Data Scientist munkaeszközök és futtatókörnyezetek

Programozói alapokból statisztikai gondolkodás és reprodukálható modellfejlesztés

Gellai István

2026-10-29

Tartalomjegyzék

Impresszum **8**

1 A data scientist munkája: kérdésből ellenőrizhető kísérlet **9**
1.1 Kérdés, célváltozó és döntési helyzet . . . 9
1.2 Adatéletciklus és a predikció pillanata . . . 16
1.3 Reprodukálhatóság és tudományos munkanapló . . . 24
1.4 Diagnózis és első teljes munkamenet . . . 31

2 Operációs rendszer, terminál és verziókezelés a napi munkában **37**
2.1 Fájlok, útvonalak és projektgyökér . . . 37
2.2 Folyamat, memória és hibakimenet . . . 45
2.3 Git adat- és modellprojektekben . . . 53
2.4 Másik gép, azonos projekt . . . 60

3 Python-verzió, csomagok és izolált környezet **67**
3.1 Interpreter és virtuális környezet . . . 67
3.2 uv projekt és ténylegesen rögzített függőségek . . . 74
3.3 Bináris csomag, BLAS és conda-döntés . . . 81
3.4 Kernelregisztráció és tiszta újrafuttatás . . . 89

4 Jupyter, VS Code és az interaktív kutatói munkafolyamat **96**
4.1 Notebook mint végrehajtási gráf . . . 96
4.2 IDE, típusinformáció és hibakeresés . . . 104
4.3 Notebookból script és jelentés . . . 111
4.4 I. ellenőrző pont: megismételhető kutatói munkaasztal . . . 121

5 Numerikus Python programozóknak: tömbök és számábrázolás **130**
5.1 ndarray, shape, axis és dtype . . . 130
5.2 Broadcasting, indexelés és másolás . . . 138
5.3 Lebegőpont, hiányzó érték és stabil számítás . . . 145
5.4 Vektorizálás és mért teljesítmény . . . 153

6 pandas: típusbiztos adatbetöltés és tisztítás **160**
6.1 Séma, dtypes és hiányzó értékek . . . 160
6.2 Kiválasztás és módosítás Copy-on-Write mellett . . . 168
6.3 Duplikáció, kulcs és adatminőségi szerződés . . . 177
6.4 Dátumok, időzónák és kategóriák . . . 185

7 SQL a feature- és kutatási adatok előállításához **194**
7.1 Reláció, NULL és elemzési szemcsézettség . . . 194
7.2 Join-kardinalitás és duplázódó célváltozó . . . 201
7.3 CTE, window és előzményből számolt feature . . . 205
7.4 Paraméterezett lekérdezés és lekérdezési terv . . . 214

8 Fájlformátumok, Arrow, DuckDB és Polars **223**
8.1 CSV-től Parquet-ig: séma és partíció . . . 223
8.2 DuckDB: helyi analitikus SQL szerver nélkül . . . 230
8.3 Polars lazy terv és streaming korlát . . . 236
8.4 II. ellenőrző pont: adatból ellenőrzött feature-tábla . . . 243

9 Matematikai nyelv: függvények, logaritmusok és numerikus közelítés **254**
9.1 Jelölésből végrehajtható számítás . . . 254
9.2 Exponenciális és logaritmikus skála . . . 260
9.3 Határérték mint közelítési folyamat . . . 268
9.4 Stabil numerika és érzékenység . . . 274

10 Lineáris algebra a modell és az adatmátrix mögött **282**
10.1 Vektor, skalárszorzat, norma és távolság . . . 282
10.2 Mátrixszorzás és lineáris transzformáció . . . 289
10.3 Vetítés és legkisebb négyzetek . . . 298
10.4 Sajátérték, SVD és dimenziócsökkentés előképe . . . 306

11 Derivált, gradiens és optimalizálás kezdő kalkulusból **315**
11.1 Meredekség és numerikus derivált . . . 315
11.2 Parciális derivált és gradiens . . . 323
11.3 Láncszabály és számítási gráf . . . 330
11.4 Optimalizáló választás és konvergenciaellenőrzés . . . 337

12 Integrál, sűrűség és Monte Carlo: területből valószínűség **345**
12.1 Terület, összeg és numerikus integrál . . . 345
12.2 Sűrűség és eloszlásfüggvény . . . 353
12.3 Várható érték és Monte Carlo közelítés . . . 361
12.4 III. ellenőrző pont: egy veszteségfüggvény teljes értelmezése . . . 368

13 Leíró statisztika és feltáró elemzés torzításokkal együtt **376**
13.1 Sokaság, minta és adatfelvételi folyamat . . . 376
13.2 Közép, szóródás és robusztus összegzés . . . 382
13.3 Eloszlás, kapcsolat és csoportkülönbség . . . 390
13.4 Hiány és anomália mint vizsgálati kérdés . . . 398

14 Valószínűség számítással és szimulációval **406**
14.1 Esemény, feltételes valószínűség és Bayes . . . 406
14.2 Véletlen változó és gyakori eloszlásmodellek . . . 413
14.3 Függetlenség, kovariancia és függő adat . . . 422
14.4 Nagy számok és centrális határeloszlás . . . 429

15 Becslés, likelihood és bizonytalanság **437**
15.1 Paraméter, becslő és likelihood . . . 437
15.2 Standard error és konfidenciaintervallum . . . 444
15.3 Bootstrap a megfelelő mintavételi egységgel . . . 452
15.4 Predikciós, paraméter- és Monte Carlo intervallum . . . 459

16 Hipotézisvizsgálat, hatásméret és kísérleti erő **467**
16.1 Nullmodell és p-érték . . . 467
16.2 Tesztválasztás és hatásméret . . . 474
16.3 Power, mintanagyság és többszörös próbák . . . 483
16.4 IV. ellenőrző pont: korrekt statisztikai állítás . . . 491

17 Statisztikai modellezés statsmodels eszközökkel **502**
17.1 OLS a lineáris algebra és statisztika találkozásánál . . . 502
17.2 Formula, interakció és kategorikus változó . . . 510
17.3 Diagnosztika és robusztus standard error . . . 517
17.4 GLM: bináris és számlálási cél . . . 524

18 Kísérlettervezés és oksági kérdések **535**
18.1 Oksági cél és megfigyelési torzítás . . . 535
18.2 Randomizálás és kísérleti protokoll . . . 542
18.3 Megfigyeléses becslés és érzékenység . . . 550
18.4 Kísérlet eredményének tudományos közlése . . . 556

19 Bayesi szemlélet és valószínűségi programozás **564**
19.1 Bayesi frissítés rácson . . . 564
19.2 Credible interval és posterior predictive check . . . 573
19.3 PyMC modell és MCMC diagnosztika . . . 581
19.4 Hierarchia és részleges pooling . . . 588

20 Idősorok és időbeli validáció **595**
20.1 Időtengely, resampling és állomásossági kérdés . . . 595
20.2 Időhelyes feature és gördülő visszateszt . . . 601
20.3 Egyszerű előrejelző modellek és baseline . . . 608
20.4 V. ellenőrző pont: modell és következtetés . . . 614

21 Prediktív gépi tanulás: adatszétválasztás és pipeline **621**
21.1 Tanítás, validáció és érintetlen teszt . . . 621
21.2 Preprocessing tanulás kizárólag a training részen . . . 627
21.3 Baseline és egyetlen teljes kísérlet . . . 633
21.4 Predikciós szerződés és hibateszt . . . 642

22 Felügyelt modellek és a megfelelő induktív feltevés **651**
22.1 Lineáris és logisztikus prediktív modellek . . . 651
22.2 Fák, random forest és gradiensboosting . . . 657
22.3 Távolság, kernel és méretkorlát . . . 665
22.4 Modellcsalád választása tudományos jegyzőkönyvvel . . . 671

23 Értékelés, keresztvalidáció és hiperparaméter-keresés **681**
23.1 Hibamértékből döntési cél . . . 681
23.2 Keresztvalidáció a függőségi szerkezethez . . . 689
23.3 Hiperparaméter és nested értékelés . . . 695
23.4 Küszöb, kalibráció és ritka esemény . . . 701

24 Nem felügyelt tanulás és reprezentáció **709**
24.1 PCA: skála, vetítés és megőrzött variancia . . . 709
24.2 Klaszterezés és stabilitás . . . 717
24.3 Anomália és ritka esemény felügyelet nélkül . . . 723
24.4 VI. ellenőrző pont: szivárgásmentes modellkísérlet . . . 731

25 Feature engineering és tudatos adatreprezentáció **740**
25.1 Időhelyes aggregátum és targetfüggetlen feature . . . 740
25.2 Kategória, hiány és ferde változó . . . 744
25.3 Feature-szelekció és magas dimenzió . . . 752
25.4 Feature store és adatkontraktus szükségessége . . . 757

26 Modellmagyarázat, megbízhatóság és szeletelt értékelés **766**
26.1 Globális és lokális magyarázat . . . 766
26.2 Kalibrált valószínűség és predikciós bizonytalanság . . . 772
26.3 Hibaszeletek és felelős értékelés . . . 780
26.4 Robusztusság és korlátokkal írt modellkártya . . . 786

27 Szöveg, embedding és LLM-kimenetek értékelése **794**
27.1 Szöveges baseline és ritka mátrix . . . 794
27.2 Embedding és hasonlósági keresés . . . 800
27.3 LLM mint sztochasztikus komponens . . . 809
27.4 Adatkezelés és reprodukálhatóság külső modellnél . . . 815

28 PyTorch, autodiff és a CPU/GPU végrehajtás **824**
28.1 Tensor, device és autograd . . . 824
28.2 Tanítási ciklus és adatbetöltés . . . 829
28.3 Generalizáció és gyorsító választás . . . 836
28.4 VII. ellenőrző pont: szakosodási döntés kontrollal . . . 844

29 Kutatási kódból tesztelhető Python-projekt **856**
29.1 Modulhatár és tiszta függvény . . . 856
29.2 Konfiguráció és futási napló . . . 865
29.3 Adatteszt és numerikus teszt . . . 875
29.4 Kódminőség és együttműködő review . . . 882

30 Adatfolyam, minőség és újrafuttatható pipeline **891**
30.1 Feldolgozási gráf és idempotencia . . . 891
30.2 Adatminőségi kapu és hibás sorok kezelése . . . 898
30.3 Cache, backfill és orchestrator . . . 907
30.4 SQL-transzformáció és dbt helye . . . 914

31 Kísérletkövetés, adat- és modellverziózás **925**
31.1 Futásazonosító és kísérleti táblázat . . . 925
31.2 Adatverzió és splitmanifest . . . 932
31.3 Modellartefaktum és betöltési határ . . . 941
31.4 Modelljelölt, jóváhagyás és registry . . . 949

32 Konténerek, távoli környezetek és automatizált futtatás **957**
32.1 Konténer és virtuális környezet külön szerepe . . . 957
32.2 Távoli interpreter és notebook biztonságos elérése . . . 963
32.3 CI: kódellenőrzés és kis számítás új környezetben . . . 970
32.4 VIII. ellenőrző pont: átadható modellprojekt . . . 977

33 Profilozás és egygépes párhuzamos futtatás **986**
33.1 Szűk keresztmetszet és mérési terv . . . 986
33.2 Thread, process és könyvtári szálak . . . 992
33.3 Párhuzamos modellkeresés erőforráskerettel . . . 999
33.4 Optimálás változatlan jelentéssel . . . 1008

34 Nagyobb adat: Dask, Spark és elosztott végrehajtás **1017**
34.1 Nagyobb mint RAM: előbb adatfolyam . . . 1017
34.2 Dask task graph és partíció . . . 1024
34.3 Spark DataFrame és végrehajtási terv . . . 1031
34.4 Elosztott kísérlet és adatelhelyezés . . . 1037

35 Batch predikció, kis szolgáltatás és kutatói demó **1044**
35.1 Batch inference és idempotens output . . . 1044
35.2 FastAPI végpont és ellenőrzött bemenet . . . 1052
35.3 Streamlit mint ellenőrizhető bemutató . . . 1060
35.4 Kiadás, visszaállítás és terhelési próba . . . 1068

36 Monitorozás, adatvédelem és modelléletciklus **1076**
36.1 Adat- és predikciómonitoring címke nélkül . . . 1076
36.2 Késő címke, teljesítmény és újratanítás . . . 1083
36.3 Hozzáférés, titkok és adatminimalizálás . . . 1090
36.4 IX. ellenőrző pont: működő rendszer és bizonyítékai . . . 1099

37 R és a többnyelvű data science ökoszisztéma **1108**
37.1 R-környezet és statisztikai munkafolyamat . . . 1108
37.2 Tábla és formula R-ben . . . 1115
37.3 Jelentés és nyelvek közötti adatcsere . . . 1122
37.4 Mikor érdemes másik ökoszisztémát választani? . . . 1132

38 Munkahelyi eszköz- és futtatókörnyezet-választás **1141**
38.1 Helyi gép, távoli CPU és managed notebook . . . 1141
38.2 CPU, GPU és elosztott rendszer közötti döntés . . . 1147
38.3 Eszközbevezetés, licenc és verziófrissítés . . . 1153
38.4 Saját data science munkaasztal specifikációja . . . 1161

39 Záróprojekt I: időhelyes prediktív modell a rendelés pillanatában **1167**
39.1 Probléma és adatszerződés befagyasztása . . . 1167
39.2 Feature-pipeline, baseline és tanulás . . . 1175
39.3 Független értékelés és hibaelemzés . . . 1185
39.4 Átadás és új környezetes ismétlés . . . 1195

40 Záróprojekt II: kísérleti következtetés és szakmai önállóság **1204**
40.1 Kísérleti kérdés és előzetes elemzési terv . . . 1204
40.2 Becslés, intervallum és érzékenység . . . 1213
40.3 Tudományos jelentés és reprodukálható bemutató . . . 1220
40.4 X. ellenőrző pont: önálló data scientist munkamenet . . . 1228
Tanulási útvonal és kompetenciatérkép . . . 1241
40.5 Teljes ív és megfigyelhető teljesítmény . . . 1241
40.6 Belépési és kilépési feltételek . . . 1242
40.7 Kompetenciatérkép és kumulatív mérés . . . 1242
40.8 Alapút és választható mélyítés . . . 1243
40.9 Egy lecke használata . . . 1244
40.10Visszatérés és portfólió . . . 1244
40.11Belső önellenőrzés . . . 1245
A feladatok és a megoldások használata . . . 1245
40.12Önértékelés a megoldás megnyitása előtt . . . 1245
40.13Nehézségi szintek . . . 1246
40.14Gépi ellenőrzés . . . 1246
Átadás előtti ellenőrzés . . . 1246
40.15Tanterv . . . 1246
40.16Automatizált próbák . . . 1247
40.17Tényleges kezdőfutás . . . 1247
40.18Export és ellenőrzési határ . . . 1248

Eszköz- és környezetterv . . . 1248
40.19Telepítés a feladathoz . . . 1249
40.20Platform és infrastruktúra . . . 1249
40.21Verziószabály . . . 1249
Verziómátrix és támogatott környezet . . . 1250
40.22Ellenőrzött alapkörnyezet . . . 1250
40.23A lab támogatott tartományai . . . 1250
40.24R és Quarto . . . 1251
Fogalomtár (magyar–angol) . . . 1251
Fogalmi térképek . . . 1254
40.25A könyv tíz állomása . . . 1254
40.26A matematikai eszköztár újrahasználata . . . 1255
40.27Háromféle következtetés és a hozzájuk tartozó eszközök . . . 1255
40.28A bizonytalanság rétegei . . . 1255
40.29A reprodukálhatóság lánca . . . 1256
40.30Didaktikai ábrák . . . 1256
40.31Hogyan használd a térképeket? . . . 1257
Tanulási útvonal . . . 1257
40.32A három szint . . . 1257
40.33Bővítő modulok . . . 1257
40.34Ajánlott útvonalak . . . 1258
40.35Hogyan használd a Belépő blokkot? . . . 1258
Implementációs feladatok pontozása . . . 1258
40.36A feladatspecifikáció . . . 1258
40.37Pontozás és visszajelzés . . . 1259
40.38Gépileg pontozott mintapéldák . . . 1259
40.39Implementációs feladatok a könyvben . . . 1260
Jelképjegyzék . . . 1260
40.401. rész: munkamódszer (1–4. fejezet) . . . 1260
40.412. rész: adatkezelés (5–8. fejezet) . . . 1261
40.423. rész: matematika (9–12. fejezet) . . . 1261
40.434. rész: statisztika (13–16. fejezet) . . . 1261
40.445. rész: modellezés (17–20. fejezet) . . . 1262
40.456. rész: gépi tanulás (21–24. fejezet) . . . 1262
40.467. rész: értékelés és reprezentáció (25–28. fejezet) . . . 1262
40.478. rész: reprodukálható pipeline (29–32. fejezet) . . . 1263
40.489. rész: skálázás és szolgáltatás (33–36. fejezet) . . . 1263
40.4910. rész: ökoszisztéma és zárás (37–40. fejezet) . . . 1263
40.50Mennyiségek dimenziója és egysége . . . 1264
Bibliográfia és forráskulcsok . . . 1265
40.51Fejezethez rendelt kulcsok . . . 1266
Ellenőrzött keretjegyzet . . . 1267
40.52Környezet és notebook . . . 1267
40.53Táblás adat és végrehajtási terv . . . 1267
40.54Korrekt modellértékelés . . . 1268
40.55Statisztikai feltételek és értelmezés . . . 1268
40.56Ellenőrzési határ . . . 1268

Impresszum

- **Szerző:** Gellai István
- **Szerkesztő:** Gellai István
- **Kiadó:** Tananyag Simple
- **Kiadás:** 1.0.0 – 2026-10-29
- **Licenc:** MIT
- **Errata:** review/reports/STATUS.md
- **Forráskód:** lab/ és tests/

A teljes licencszöveg a tároló gyökerének `LICENSE` fájljában található (a könyvkönyvtárból: `../../LICENSE`).

1 A data scientist munkája: kérdésből ellenőrizhető kísérlet

1.1 Kérdés, célváltozó és döntési helyzet

Belépő · szint: alap · idő: 45 perc · eszköz: Python 3.12, pandas

Előfeltétel: nincs · Célkészség: Kérdés, célváltozó és döntési helyzet · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

1.1.1 Cél és konkrét helyzet

Egy webshop üzemeltetője két, látszólag hasonló kérdést tesz fel a data scientistnek:

1. *„Egy most beérkező rendelésnél mekkora annak a kockázata, hogy a vállalt 7 napnál később kézbesítjük?"*
2. *„Érdemes-e bevezetni egy új értesítési változatot, mert attól nő a vásárlás valószínűsége?"*

A Northstar mesterséges esetben mindkettő tanulóadattal vizsgálható, de **nem ugyanazzal a megfigyelési egységgel**. Az első kérdés megfigyelési egysége a **rendelés** (`order_id`), a másodiké a **randomizált ügyfél** (`customer_id`). Aki a második kérdésre a rendelési tábla soraiból akar hatást becsülni, összekeverheti a randomizálási egységet és a megfigyelési egységet. Randomizált kísérletben a hozzárendelést és az elemzési nevezőt ügyfélszinten kell megőrizni; megfigyeléses oksági elemzés is lehetséges, de külön azonosítási feltevéseket és torzításvizsgálatot igényel.

A kimenetek jelölése. A könyvben kétféle kimenet szerepel. Az *ellenőrzött futás* mellett a környezet (Python- és csomagverziók), a futtatott parancs és a futásazonosító szerepel; ezek a labor `ellenorzott_report.json` fájljából vagy a `tests/` naplójából származnak. A *szemléltető kimenet* a szerzői szándékot és a várható alakot mutatja, nem mérési eredmény; ezt a szöveg `szemléltető`, `elméletileg várt` vagy `nem futtatott` jelöléssel látja el. Ha egy kimenetnél egyik jelölés sem szerepel, az a szövegkörnyezetből illusztráció, és nem használható mért értékként. A könyv nem állítja, hogy minden kódpélda lefutott; a végrehajtott minták a `lab/` csomaghoz tartozó tesztekben és a `tests/` könyvtárban ellenőrzöttek.

A kód forrása. A nyomtatott kódblokkok a magyarázatot szolgálják; a futtatható, ellenőrzött változat a `lab/` csomagban (és a `lab/experiments/` mappában) van, ahol a fájlokhoz hash-manifest és tesztek tartoznak. A PDF-ből másolt, oldaltörést tartalmazó kódrészletet ne tekintsd forrásnak: a stabil fájl nevét és helyét a fejezet szövege, illetve a `lab/README.md` adja meg.

Megszólítás. A könyv végig tegező formát használ. A harmadik személyű megfogalmazás („a függvény adja vissza", „a szabály írja le") mindig a kód vagy a folyamat viselkedésére vonatkozik, nem az olvasóhoz szóló utasítás.

A fejezet célja, hogy az üzleti kérdésből **pontos bemenetet, kimenetet és ellenőrzési feltételt** adjunk. Nem modellt építünk még, hanem a feladatot írjuk le úgy, hogy egy későbbi modell ne válaszolhasson rossz kérdésre.

1. problémalap: rendeléskori késési kockázat

- Üzleti kérdés: mekkora a valószínűsége, hogy az éppen beérkező rendelés késik?
- Megfigyelési egység: egy rendelés, azaz `orders.csv` egy sora, kulcsa `order_id`.
- Bemenet a döntés pillanatában: `order_ts`, `promised_days=7`, `quantity_sum`, `item_rows`, `basket_minor`, `shipping_fee_minor`, `distance_km`, `priority_flag`, valamint az ügyfél `segment`, `region`, `acquisition_channel` és a rendeléskor már ismert `account_age_days`.
- Előrejelzési cél: pozitív egész napokban mért `delivery_days`, amelyből a célváltozó

$$Y_i = \begin{cases} 1, & \text{ha } \mathit{delivery_days}_i > 7, \\ 0, & \text{különben.} \end{cases}$$

- Kimenet: rendelésenként egy $\hat{p}_i \in [0,1]$ kockázati pontszám; a felhasználó dönthet például arról, hogy proaktív ügyfélértesítést indít.
- Ellenőrzési feltétel: a modell csak olyan feature-t használhat, amely a rendelés pillanatában már ismert. A `delivery_days`, `actual_delivery_at`, `target_late` utólagos; a modell betanításakor címke lehet, de a predikció bemenetében nem. A tanító és teszt elválasztás időbeli, és a tanítócímke csak akkor használható, ha a cutoff előtt megérkezett.

1.1.1.1 Intuíció először: mit jelent az ITT?

Képzelj el négy ügyfelet, és tegyük fel, hogy ismernénk mindkét potenciális kimenetüket: $Y_i(1)$-et (ha megkapja a kuponos kezelést) és $Y_i(0)$-t (ha nem). A 7 napos konverzió 0/1 értékű:

Ügyfél	$Y_i(1)$	$Y_i(0)$	Egyéni hatás $Y_i(1) - Y_i(0)$
A	1	0	+1
B	0	0	0
C	1	1	0
D	1	0	+1

A négy egyéni hatás átlaga $(1+0+0+1)/4 = 0{,}5$. Ez a **populáció szintű átlagos kezelési hatás** a négy ügyfélre. A valóságban azonban soha nem látjuk egyszerre mindkét oszlopot: akinél a sorsolás kezelést ad ($Z_i = 1$), ott csak $Y_i(1)$ figyelhető meg; akinél kontrollt ($Z_i = 0$), ott csak $Y_i(0)$. A másik érték **kontrafaktuális**, és egyéni szinten nem azonosítható.

Ha a négy ügyfelet véletlenül két-két fős csoportokba osztjuk, a megfigyelt különbség sorsolásonként változik. Az A,B kezelés / C,D kontroll felosztásnál a kezelt átlag $(1+0)/2 = 0{,}5$, a kontroll átlag $(1+0)/2 = 0{,}5$, a különbség 0. Az A,C kezelés / B,D kontroll felosztásnál a kezelt átlag $(1+1)/2 = 1$, a kontroll átlag $(0+0)/2 = 0$, a különbség 1. Ugyanaz a négy ember, ugyanazok a potenciális kimenetek, más sorsolás, más becslés. A hat lehetséges kiegyensúlyozott felosztásra a különbségek átlaga mégis 0,5: a randomizáció azt biztosítja, hogy a csoportkülönbség *várhatóan* a populációszintű ITT-t adja, nem azt, hogy egyetlen sorsolás pontosan azt mutatja.

Az **ITT** (intention-to-treat) tehát nem az egyéni hatások megfigyelt átlaga — hiszen azok nem figyelhetők meg —, hanem a hozzárendelés populációszintű átlagos hatása:

$$\Delta_{\mathrm{ITT}} = E[Y_i(1) - Y_i(0)].$$

A minta csoportátlag-különbsége ennek **becslője**, amely a sorsolás és a mintavétel miatt ingadozik. A formális feltevéseket (konzisztencia, interferenciamentesség, a kezelés tényleges megkapása D_i

versus a hozzárendelés Z_i) a 18. fejezet tárgyalja; a valószínűségi és mintavételi hátteret a 14. fejezet, a p-érték és a bizonytalanság értelmezését a 16. fejezet adja. Első olvasáskor elég ennyi: a csoportkülönbség becslés, a személyenkénti hatás nem megfigyelhető, és a randomizáció teszi összehasonlíthatóvá a csoportokat.

2. problémalap: randomizált változtatás hatása

- Üzleti kérdés: növeli-e a kezelés a 7 napos konverziót?
- Megfigyelési egység: egy randomizált ügyfél; `assignments.csv` minden ügyfelet `control` vagy `treatment` csoportba sorol.
- Kimenet: `outcomes.csv` minden ügyfélre ugyanazon 7 napos ablakban tartalmazza a `conversion_7d` 0/1 kimenetet.
- Elsődleges oksági becslési cél (ITT): a hozzárendelés hatása a randomizálható ügyfélpopulációban. Legyen Z_i a véletlenszerű hozzárendelés (1: kezelésre kijelölt, 0: kontroll), $Y_i(z)$ pedig a 7 napos potenciális konverziós kimenet az adott hozzárendelés mellett. Ekkor

$$\Delta_{\text{ITT}} = E[Y_i(1) - Y_i(0)].$$

  A megfigyelt csoportátlagok különbsége, $\bar{Y}_{Z=1} - \bar{Y}_{Z=0}$ ennek randomizáción alapuló becslője megfelelő adatfelvétel, konzisztencia és az egységek közötti releváns interferencia hiánya mellett. A ténylegesen kapott kezelést (D_i) meg kell különböztetni a hozzárendeléstől (Z_i).
- Ellenőrzési feltétel: az elemzés nevezőjében minden randomizált ügyfél szerepel, akkor is, ha esetleg nem vásárolt. A rendelési tételek sorai nem szaporíthatják a mintanagyságot.

1.1.2 Leírás, előrejelzés, kísérlet és megfigyelés

A **leírás** múltbeli vagy jelenlegi adatot foglal össze: *„a múlt hónapi rendelések 22%-a késett”*. Az **előrejelzés** ismeretlen kimenetre ad esélyt vagy várható értéket: *„ennek az új rendelésnek 35% az esélye a késésre”*. A leírásnak nincs szüksége modellre, az előrejelzésnek igen, mert a célkimenet a döntés pillanatában még nem ismert.

A **kísérlet** során a kutató maga osztja kezelésre és kontrollra a megfigyelési egységeket; a Northstar `assignments.csv` ügyfélszintű randomizálást rögzít. A **megfigyeléses** elemzésben nem avatkozunk be, csak azt nézzük, ami megtörtént. A rendelési táblából például le lehet írni, hogy a `priority_flag=1` rendelések késnek-e gyakrabban, de ez önmagában nem bizonyítja, hogy a prioritás *okozza* a különbséget, mert a prioritást kapó rendelések szisztematikusan mások is lehetnek.

1.1.3 A becslési cél: estimand

Az **estimand** az a sokasági, ismeretlen mennyiség, amelyre a statisztikai eljárás következtetni akar. A kísérleti problémában az estimand Δ_{ITT}, a hozzárendelés szerinti két potenciális kimenet várható értékének különbsége. A becslés ebből a mintából számított konkrét szám lesz; a becslési céltól különbözik.

A predikciós feladatban az estimand lehet egy új rendelés feltételes késési valószínűsége:

$$\pi(x_{\text{new}}) = E[Y_{\text{new}} \mid X_{\text{new}} = x_{\text{new}}].$$

Itt x_{new} a rendeléskor ismert feature-vektor. Ezt a feltételes várható értéket később modellel közelítjük; a modell által adott $\hat{\pi}(x)$ a becslés.

1.1.4 Ki dönt és mikor?

A késési kockázatnál a döntési pillanat az `order_ts`: amikor a rendelés létrejön, a rendszernek azonnal kockázati becslést kell adnia. A döntéshozó azonban jellemzően nem a data scientist, hanem az ügyfélkapcsolati vagy logisztikai folyamat felelőse: ő határoz meg egy küszöböt, például *„30% felett küldjünk kompenzációt tartalmazó üzenetet”*. A data scientist ebben a munkamegosztásban a predikciós szerződést és az értékelést adja; a küszöb üzleti döntés.

A kísérletnél a döntési pillanat a randomizálás előtti protokollrögzítés; a változtatás bevezetéséről a kísérleti jelentés után dönt a terméktulajdonos. Az ő döntésére a becsült hatás mellett a bizonytalanság és a megvalósítási költség is tartozik. A statisztikus feladata nem az, hogy *„szignifikáns?”* feliratot adjon, hanem hogy az estimandot, a becslést és annak bizonytalanságát pontosan jelentse.

1.1.5 Matematikai híd: arány, átlag és feltétel

Egy 0/1 értékű Y_i változóval a minta **aránya**, más néven átlaga:

$$\hat{p} = \frac{1}{n} \sum_{i=1}^{n} y_i.$$

- n: a megfigyelési egységek darabszáma;
- $i = 1, \dots, n$: az egyes megfigyelések indexe;
- $y_i \in \{0, 1\}$: a kimenet értéke;
- $\hat{p}$ dimenziómentes, lehetséges tartománya $[0, 1]$;
- számítási alak: hossz n összeg, a viszonyított hányad.

Egy napi kimenetnél az átlag ugyanilyen szerkezetű, de mértékegysége van:

$$\bar{d} = \frac{1}{n} \sum_{i=1}^{n} delivery_days_i,$$

ahol `delivery_days` mértékegysége nap, ezért $\bar{d}$ mértékegysége is nap.

A **feltételes arány** egy részcsoportra számított ugyanilyen hányad. Ha a csoportot g_i jelöli, akkor:

$$\hat{p}_{g=1} = \frac{\sum_{i:g_i=1} y_i}{n_1},$$

ahol n_1 azoknak a rendeléseknek a száma, amelyekre $g_i = 1$. A feltétel itt a nevező kiválasztása; a számlálóban továbbra is 0/1 értékeket összegzünk.

order_id	priority_flag	distance_km	delivery_days	target_late

1.1.6 Minimális munkapélda kézzel és Pythonban

Az alábbi nyolc kitalált rendeléssel dolgozunk. A `target_late` a 7 napos ígérethez számolt cél; a modell bemenetébe nem kerülhet be.

order_id	priority_flag	distance_km	delivery_days	target_late
O1	1	5	3	0
O2	1	6	8	1
O3	1	7	9	1
O4	0	2	4	0
O5	0	12	11	1
O6	1	1	2	0
O7	0	11	10	1
O8	0	4	6	0

Kézi kontroll:

- átlagos távolság: $(5+6+7+2+12+1+11+4)/8 = 48/8 = 6{,}0$ km;
- összes késés: $0+1+1+0+1+0+1+0 = 4$;
- késési arány: $4/8 = 0{,}5$;
- `priority_flag=1` részcsoportban: 2 késés, 4 rendelés, arány $2/4 = 0{,}5$;
- `priority_flag=0` részcsoportban: 2 késés, 4 rendelés, arány $2/4 = 0{,}5$.

A következő kód ehhez a kézi számpéldához illeszkedik. A közölt kimenet elméletileg várt, automatizált futtatással még nem ellenőrzött eredmény. A kód tiszta Python, semmilyen data science csomagot nem feltételez.

```
# Szemléltető kód; automatikusan még nem ellenőrzött.
def late_flag(delivery_days: int, promised_days: int = 7) -> int:
    """Célváltozó: 1, ha a kézbesítés késik, 0 különben."""
    return int(delivery_days > promised_days)


def late_rate(rows: list[dict], promised_days: int = 7) -> float:
    """Késési arány megfigyelési egységenként."""
    if not rows:
        return float("nan")
    labels = [late_flag(r["delivery_days"], promised_days) for r in rows]
    return sum(labels) / len(labels)


def conditional_rate(
    rows: list[dict], group_key: str, group_value, promised_days: int = 7
) -> float:
    """Feltételes késési arány egy csoportra."""
    group = [r for r in rows if r[group_key] == group_value]
    return late_rate(group, promised_days)


rows = [
    {"order_id": "O1", "priority_flag": 1, "distance_km": 5, "delivery_days": 3},
    {"order_id": "O2", "priority_flag": 1, "distance_km": 6, "delivery_days": 8},
    {"order_id": "O3", "priority_flag": 1, "distance_km": 7, "delivery_days": 9},
```

```
    {"order_id": "O4", "priority_flag": 0, "distance_km": 2, "delivery_days": 4},
    {"order_id": "O5", "priority_flag": 0, "distance_km": 12, "delivery_days": 11},
    {"order_id": "O6", "priority_flag": 1, "distance_km": 1, "delivery_days": 2},
    {"order_id": "O7", "priority_flag": 0, "distance_km": 11, "delivery_days": 10},
    {"order_id": "O8", "priority_flag": 0, "distance_km": 4, "delivery_days": 6},
]

overall = late_rate(rows)
priority1 = conditional_rate(rows, "priority_flag", 1)
priority0 = conditional_rate(rows, "priority_flag", 0)

print(f"n={len(rows)}")
print(f"nyers késési arány={overall:.3f}")
print(f"priority=1 késési arány={priority1:.3f}")
print(f"priority=0 késési arány={priority0:.3f}")

# Invariáns ellenőrzés: az arány 0 és 1 közé esik,
# és a 8 elemű mintában pontosan 4 késés van.
assert abs(overall - 0.5) < 1e-12
assert 0.0 <= priority1 <= 1.0
assert 0.0 <= priority0 <= 1.0
```

Elméletileg várt kimenet:

```
n=8
nyers késési arány=0.500
priority=1 késési arány=0.500
priority=0 késési arány=0.500
```

1.1.7 Eredmény értelmezése

A 0,5-ös érték **ebben** a mintában a rendelések fele késett; ez nem állítás a teljes Northstar-populációról. A nyolc sorhoz ismerjük a teljes eljárást, ezért az eredmény úgy ellenőrizhető, hogy a `delivery_days > 7` egyenlőtlenséget soronként kiszámoljuk, majd a darabszámot n-nel osztjuk. A kódba írt `assert` ugyanezt a levezetett 4/8 értéket rögzíti.

A `priority_flag` szerinti két 0,5 azonosnak tűnik, de ez a nyolc elemű példa **nem jelenti azt, hogy a prioritás nem hat**. A pillanatnyi mintabeli egyenlőség leíró eredmény; az oksági hatás azonosításához megfelelő kutatási terv kell: randomizálás, vagy megfigyeléses vizsgálatban indokolt azonosítási feltevések és érzékenységvizsgálat.

Ha a bemenetet később a `lab/data/orders.csv` 2400 rendelésére cseréljük, akkor a fájlt a predikciós feladatban úgy kell kezelni, hogy a rendeléskor ismert feature-listához képest `delivery_days`, `actual_delivery_at`, `label_available_at` és `target_late` csak címkeként jelenhetnek meg. A számítás akkor is ugyanaz: a nevező a **rendelések** száma, nem a rendelési tételek sorainak száma.

Becslési cél és oksági értelmezés. A randomizált kísérletben az elsődleges, előre rögzített becslési cél legyen a *hozzárendelés szerinti* (intention-to-treat, ITT) átlagos hatás: $E[Y_i(1) - Y_i(0)]$ a randomizálható célpopulációra. A két megfigyelt csoport átlaga közötti különbség ennek becslője randomizálás és konzisztens kimenetmérés mellett; nem azonos automatikusan az egyes ügyfelekre vonatkozó oksági hatással. Külön vizsgálni kell a hiányzó kimenetet és a randomizálás után történt kizárásokat.

A predikciós $P(Y = 1 \mid X = x)$ ezzel szemben egy megadott adatgeneráló eloszlásra és döntési időpontra vonatkozó feltételes mennyiség. Nem ígér oksági hatást és populációváltáskor nem feltétlenül marad azonos. **Ellenőrző kérdés:** egy ügyfél kétszer rendel; a konverziós kísérletben hány megfigyelésként, a késési modellben hány rendelésként jelenik meg?

1.1.8 Feladatok

1. **Értelmezés.** A következő két mondat közül az egyik leírást, a másik kísérleti hatáskérdést tartalmaz:

   a) *„A 2025 első félévében feladott Northstar-rendelések 27%-a késett.”*

   b) *„A randomizált treatment csoport konverziós aránya 3 százalékponttal nagyobb volt a kontrollénál.”*
   Nevezd meg mindkét állítás megfigyelési egységét, és mondd meg, melyik lehet egy leíró összesítés, illetve melyik kísérleti becslés!

2. **Implementálás.** A fenti függvényekkel számold ki a késési arányt az alábbi három rendelésre:
   `{"delivery_days": 8}`, `{"delivery_days": 7}`, `{"delivery_days": 12}`.
   Mi a várt érték, és miért fontos, hogy a 7 napos ígéretnél a `>` művelet szerepeljen a `>=` helyett?

3. **Hibakeresés.** Egy elemző a rendelések helyett az `order_items` táblához csatlakoztatott `orders` táblát használja, és így számol:

   ```
   SELECT AVG(1.0 * (o.delivery_days > o.promised_days))
   FROM orders o
   JOIN order_items oi USING (order_id);
   ```

   Két rendelés van: A rendelés késik és 3 tételsor tartozik hozzá, B rendelés nem késik és 1 tételsor tartozik hozzá. Mekkora a tételsor-alapú arány, mekkora a helyes rendelésalapú arány, és mi okozza az eltérést?

4. **Transzfer megváltoztatott feltételre.** Változtasd az ígéretet 7 napról 5 napra a nyolcrendeléses táblában. Számold ki az új `target_late` értékeket és az új késési arányt! Változik-e a `priority_flag=1` és `priority_flag=0` feltételes arány, és ha igen, milyen irányba?

1.1.9 Megoldások és indoklás

1. Az a) állítás megfigyelési egysége a **rendelés**: minden rendelés késő vagy nem késő. Ez leíró összesítés. A b) állítás megfigyelési egysége az **ügyfél**: a `conversion_7d` ügyfélenként egy 0/1 érték. Ez kísérleti becslés, mert a két csoport randomizálással jött létre. A b) mondatban szereplő „3 százalékpont” mintabeli becslés, nem automatikusan a teljes populációra ismert hatás.

2. A késési zászlók: 1, 0, 1. Az összeg 2, a darabszám 3, ezért a késési arány $2/3 \approx 0{,}667$. A `>` kezelésnél a `delivery_days = 7` nem késik, mert a 7 napos ígéretet teljesítette; a `>=` viszont későnek számítaná. A szerződés szerint `target_late = 1` akkor, ha `delivery_days > promised_days`, ezért a 7 napos teljesítés kódja 0.

3. A csatlakoztatott táblában az A rendelés 3 sora késik, a B rendelés 1 sora nem; a tételsor-alapú arány $3/4 = 0{,}75$. A helyes rendelésalapú arány $1/2 = 0{,}5$. Az eltérés oka, hogy a `JOIN` az egy rendeléshez tartozó tételsorokat többszörözi, ezért a megfigyelési egység nem rendelés, hanem tételsor. A helyes lekérdezés a rendelési táblán marad:

   ```
   SELECT AVG(1.0 * target_late) AS rendeles_szintu_arany
   FROM orders;
   ```

A `target_late` itt ugyanazt a `delivery_days > promised_days` definíciót tárolja, de rendelésenként egy soron.

4. Az 5 napos küszöbbel a zászlók: O1 0, O2 1, O3 1, O4 0, O5 1, O6 0, O7 1, O8 1, azaz 5 késés 8 rendelésből, arány $5/8 = 0{,}625$. A `priority_flag=1` csoport rendelései: O1, O2, O3, O6; ebből O2 és O3 késik, arány $2/4 = 0{,}5$. A `priority_flag=0` csoport rendelései: O4, O5, O7, O8; ebből O5, O7, O8 késik, arány $3/4 = 0{,}75$. Ebben a kitalált mintában a nem prioritásos csoport feltételes aránya nőtt, a prioritásosé nem változott, de ez ismét mintabeli leírás.

1.1.10 Gyakori tévedés és önellenőrzés

A leggyakoribb hiba az, hogy a programozó pontosan végrehajt egy hibás statisztikai kérdést. A pontos program **nem** javítja meg a rosszul megfogalmazott kérdést.

Példa: egy rendeléshez 10 tételsor tartozik, és az a rendelés késik; egy másik rendelés nem késik, és 1 tételsora van. Ha a fenti `JOIN`-t használó kódot lefuttatjuk, 10 késő sort osztunk 11 sorral, az eredmény körülbelül 0,91. A program szintaktikailag helyes, a csatlakoztatás is technikailag jó, de a kérdés megfigyelési egysége rossz: a kereskedő azt kérdezte, *mekkora a rendelések* késési aránya, nem azt, hogy *mekkora a késő rendeléshez tartozó tételek* részaránya.

Önellenőrzés minden feladatkezdés előtt:

- Megneveztem-e a **megfigyelési egységet** és annak kulcsát?
- A célváltozó a döntési pillanat után, a bemenő feature-ök a döntési pillanatban ismertek?
- A nevező azt a halmazt számolja, amelyre a következtetés vonatkozik?
- Leíró, predikciós vagy hatásbecslési kérdést teszek fel?

Ha ezekre nincs egy mondatos válasz, a kód megírása előtt álljunk meg.

1.1.11 Rövid összefoglaló

Az üzleti kérdésből akkor lesz statisztikai feladat, ha rögzítjük a megfigyelési egységet, a célváltozót és a döntés pillanatát. A Northstar késési feladatban rendelésenként jelezzük előre a 7 napot meghaladó kézbesítést; bemenete a rendeléskor ismert feature-lista, kimenete 0–1 kockázat. A randomizált kísérletben ügyfélszintű konverziót mérünk, és minden randomizált ügyfél a nevezőben marad. Az arány képlete egyszerű, de a nevező hibája csendesen tönkreteszi a becslést. A következő alfejezet a predikció pillanatát és az adatéletciklus időbeli sorrendjét vizsgálja.

1.2 Adatéletciklus és a predikció pillanata

Belépő · szint: alap · idő: 45 perc · eszköz: Python 3.12, pandas

Előfeltétel: 1.1. alfejezet · Célkészség: Adatéletciklus és a predikció pillanata · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

1.2.1 Cél és konkrét helyzet

A Northstar prediktív projektben a modellnek minden rendelés leadásakor, az `order_ts` időpontban kell 0–1 közötti kockázatot adnia arra, hogy a kézbesítés a 7 napos ígéretnél később történik. A bemenet csak a rendelés pillanatában ismert jellemzőkből állhat; a kimenet a `target_late` becslése. Ebben az alfejezetben nem modellt építünk, hanem az adatéletciklus időrendjét rögzítjük: minden mezőhöz hozzárendeljük, mikor válik ismertté, és melyik fázis használhatja. A konkrét input a kezdőlabor `lab/data/orders.csv` és `lab/data/order_items.csv` fájlja; az elvárt eredmény egy rendelkezésreállási tábla, valamint egy ellenőrző kód, amely kimutatja a jövőbeli mezők tiltott prediktorhasználatát.

Mielőtt tovább olvas, adjon előzetes becslést: az `orders.csv` 15 mezője közül hány ismert már a rendelés pillanatában, és hány csak később? Az `order_items.csv` 8 mezője közül hány későbbi?

1.2.2 Fogalmi magyarázat: adatforrás, as-of idő, label és provenance

Adatforrás (data source) az a rendszer vagy folyamat, ahol az érték keletkezik. A rendelési rendszer a kosár adatait, a szállítási rendszer a tényleges kézbesítést, az ügyféltörzs az ügyfél törzsadatait szolgáltatja. A Northstar szintetikus esetben a `generate_data.py` szimulálja ezeket a forrásokat; nincs valós ügyfél vagy üzleti tranzakció.

Nyers és feldolgozott adat (raw/processed data) nem ugyanaz. A `lab/data/orders_raw.csv` szándékosan hibás nyers másolat: öt duplikált sor, két hiányzó `distance_km`, egy negatív `basket_minor` és egy nem létező `customer_id` szerepel benne. A tiszta `lab/data/orders.csv` a tisztítás után keletkezett kontroll; a döntéseket auditnaplónak kell kísérnie, nem szabad csendben nullát vagy nullapontot feltételezni. A nyers és a tiszta adat tehát külön életszakasz, és csak a tiszta verzióhoz tartozik rögzített minőségi szerződés.

Label (címke) a felügyelt tanulás célértéke, amely a tanításkor már ismert, a predikció pillanatában viszont még nem. A Northstar késési feladatban a label a `target_late`: értéke 1, ha `delivery_days > promised_days`, egyébként 0. A `target_late` maga származtatott mező; a `delivery_days` ennek az alapanyaga, de szintén csak a kézbesítés után ismert.

As-of idő (as-of time) az a pillanat, amikor egy tábla állapotát nézzük. Ma lekérdezve a múltbeli rendelések már tartalmazzák a `delivery_days` értékét, de egy 2025. áprilisi predikciós pillanatban még nem létezett. Minden mezőhöz hozzárendelhetünk egy ismertté válási időpontot: a mező legkorábban attól kezdve használható, amikor az érték ténylegesen a rendelkezésünkre áll. A kezdőprojekt konzervatív szerződése szerint a címke a teljes kézbesítéskor válik ismertté: `label_available_at = actual_delivery_at`, akkor is, ha a késés tényét elvileg hamarabb is fel lehetne ismerni.

Provenance (származás, eredetigazolás) azt rögzíti, honnan jött az adat, milyen átalakítási láncon ment át, és melyik verzió volt az input. A kezdőlabor `lab/data/manifest.json` fájlja a seedet, a sorszámokat, a hashértékeket és a kontrollösszegeket tartalmazza; a `quickcheck.py` ezeket ellenőrzi. Ha egy kolléga manifest nélkül ad át egy CSV-t, nem tudjuk reprodukálni az állapotát, és nem tudjuk eldönteni, hogy a tisztítási döntések hol történtek.

1.2.3 Adatszerződés: a kezdőlabor mezői

A `lab/data/orders.csv` 15 mezője (a tiszta, kontrollált változat; a nyers `orders_raw.csv` szándékosan sérült):

Mező	dtype	Hiány	Forrásrendszer	Ismerté válás
`order_id`	string	nem	rendelési rendszer	`order_ts`
`customer_id`	string	nem	ügyféltörzs	`order_ts`
`order_ts`	datetime64[us, UTC]	nem	rendelési rendszer	`order_ts`
`promised_days`	int64	nem	rendelési szerződés	`order_ts`
`quantity_sum`	int64	nem	rendelési rendszer	`order_ts`
`item_rows`	int64	nem	rendelési rendszer	`order_ts`
`basket_minor`	int64 (EUR cent)	nem	rendelési rendszer	`order_ts`
`shipping_fee_minor`	int64 (EUR cent)	nem	rendelési rendszer	`order_ts`
`currency`	string (`EUR`)	nem	rendelési rendszer	`order_ts`
`distance_km`	int64	nem	szállítási rendszer	`order_ts`
`priority_flag`	int64 (0/1)	nem	rendelési rendszer	`order_ts`
`delivery_days`	int64	nem	szállítási rendszer	kézbesítés után
`actual_delivery_at`	datetime64[us, UTC]	nem	szállítási rendszer	kézbesítés után
`label_available_at`	datetime64[us, UTC]	nem	származtatott	kézbesítés után
`target_late`	int64 (0/1)	nem	származtatott	`label_available_at`

A `lab/data/order_items.csv` 8 mezője:

Mező	dtype	Hiány	Forrásrendszer	Ismerté válás
`order_id`	string	nem	rendelési rendszer	`order_ts`
`line_id`	int64	nem	rendelési rendszer	`order_ts`
`product_id`	string	nem	katalógus	`order_ts`
`quantity`	int64	nem	rendelési rendszer	`order_ts`
`unit_price_minor`	int64 (EUR cent)	nem	katalógus	`order_ts`
`discount_minor`	int64 (EUR cent)	nem	rendelési rendszer	`order_ts`
`line_total_minor`	int64 (EUR cent)	nem	származtatott	`order_ts`
`currency`	string (`EUR`)	nem	rendelési rendszer	`order_ts`

A tiszta fájlokban nincs hiányzó érték; a `orders_raw.csv` két hiányzó `distance_km`-et, egy negatív `basket_minor`-t, egy ismeretlen `customer_id`-t és öt duplikált sort tartalmaz. A hiányt

nem szabad automatikusan nullának vagy nullapontnak tekinteni; a tisztítási döntést auditnapló rögzíti. A teljes szerződés a `lab/ADATOK.md`-ben és a `lab/data/manifest.json`-ban van, a csomag részeként letölthető.

Három szintetikus sor a `lab/data/orders.csv` elejéről:

```
order_id,customer_id,order_ts,promised_days,quantity_sum,item_rows,basket_minor,shipping_fee_minor,currency,
O000005,C0022,2025-01-01T08:44:00+00:00,7,4,1,17460,999,EUR,412,1,1,2025-01-02T08:44:00+00:00,2025-01-02T08:
O000004,C0009,2025-01-01T09:23:00+00:00,7,1,1,4262,399,EUR,1360,0,9,2025-01-10T09:23:00+00:00,2025-01-10T09:
O000013,C0111,2025-01-01T10:19:00+00:00,7,1,1,8680,399,EUR,1718,0,10,2025-01-11T10:19:00+00:00,2025-01-11T10
```

Három szintetikus sor a `lab/data/order_items.csv` elejéről:

```
order_id,line_id,product_id,quantity,unit_price_minor,discount_minor,line_total_minor,currency
O000001,1,P015,3,3058,300,8874,EUR
O000001,2,P003,4,1763,100,6952,EUR
O000002,1,P014,3,11757,100,35171,EUR
```

1.2.4 Időpont, intervallum és feltételes információ

Jelölje $t_{0,i}$ az i rendelés `order_ts` időpontját, τ_j pedig a j mező ismertté válási időpontját. Egy x_j mező akkor szerepelhet a $t_{0,i}$ pillanatban futó modell bemenetében, ha

$$\tau_j \leq t_{0,i}.$$

A $\leq$ jel itt időpontok UTC-sorrendjét jelenti; az egyenlőség azt fejezi ki, hogy a rendelés leadásával egyidejűleg ismertté váló mező még éppen használható. A `delivery_days` esetében

$$\tau_{\text{delivery_days}} = \text{actual_delivery_at}_i > t_{0,i},$$

tehát a mező nem lehet prediktor. Az $\mathcal{I}(t)$ információs halmaz legyen azoknak az értékeknek a halmaza, amelyek t-ben már ismertek. A fenti szabály így írható:

$$x_i \subseteq \mathcal{I}(t_{0,i}), \qquad y_i \notin x_i,$$

ahol x_i az i rendelés predikciós bemenete, y_i pedig a `target_late` label. A második feltétel azt mondja ki, hogy a label soha nem lehet a predikció inputja, még akkor sem, ha a tanító CSV-ben fizikailag ott van.

A label definíciója nap mértékegységben:

$$y_i = \mathbf{1}\{\text{delivery_days}_i > \text{promised_days}_i\},$$

ahol $\mathbf{1}\{\cdot\}$ az indikátorfüggvény: értéke 1, ha a kapcsos zárójelben lévő feltétel igaz, különben 0. A `promised_days` a kezdőpéldában 7 nap, a `delivery_days` pozitív egész nap.

A tanítási cutoff $c =$ `2025-05-01T00:00:00+00:00`. Egy rendelés akkor használható tanításra, ha a rendelés és a címke is a cutoff előtti:

$$\text{train}_i(c) = \mathbf{1}\{\text{order_ts}_i < c\} \cdot \mathbf{1}\{\text{label_available_at}_i < c\}.$$

A szorzás itt logikai ÉS: a sor csak akkor kap 1-et, ha mindkét feltétel igaz. A teszt feltétele

$$\text{test}_i(c) = \mathbf{1}\{\text{order_ts}_i \geq c\},$$

a kihagyott határsáv pedig

$$\text{purgatory}_i(c) = \mathbf{1}\{\text{order_ts}_i < c\} \cdot \mathbf{1}\{\text{label_available_at}_i \geq c\}.$$

A három halmaz diszjunkt, és együtt lefedi az összes rendelést. Kézi kontroll: ha `order_ts = 2025-04-28T09:00:00+00:00`, `label_available_at = 2025-05-04T18:00:00+00:00`, akkor a rendelés a kihagyott határsávba esik: a rendelés ugyan a cutoff előtti, de a címke a cutoffkor még nem érkezett meg. A tanításhoz a címkének szigorúan a cutoff előtt kell rendelkezésre állnia.

1.2.5 Rendelkezésreállási tábla és minimális kód

A következő táblázat az `orders.csv` mezőit a `lab/ADATOK.md` szerződéséből sorolja fel. A t_0 oszlop a rendelés pillanatát jelenti; a „prediktor” oszlop a rendelkezésre állás és a kezdő feature-lista együttes ítélete.

Mező	Keletkezés / ismertté válás	Ismert t_0-kor?	Szerep a starterben
`order_id`	rendelés rögzítésekor, t_0	igen	azonosító, nem prediktor
`customer_id`	ügyféltörzs, t_0 előtt	igen	azonosító, nem prediktor
`order_ts`	rendelés rögzítésekor	igen	a predikció időpontja
`promised_days`	szolgáltatási szabály, t_0	igen	konstans 7, a label definíciójához kell
`quantity_sum`	kosár, t_0	igen	feature
`item_rows`	kosár, t_0	igen	elérhető, a starter listán nem szerepel
`basket_minor`	kosár, t_0	igen	feature
`shipping_fee_minor`	díjszabás, t_0	igen	elérhető, a starter listán nem szerepel
`currency`	díjszabás, t_0	igen	konstans `EUR`
`distance_km`	logisztikai rendszer, t_0	igen	feature
`priority_flag`	szolgáltatási szabály, t_0	igen	feature
`delivery_days`	tényleges kézbesítéskor	nem	**tiltott prediktor**; a label alapanyaga
`actual_delivery_at`	tényleges kézbesítéskor	nem	tiltott; a label időbélyege
`label_available_at`	származtatott: `= actual_delivery_at`	nem	tiltott; a taníthatóság kapuja
`target_late`	származtatott: `delivery_days > 7`	nem	a tanulási label, nem input

Az `order_items.csv` mezői:

Mező	Keletkezés	Ismert t_0-kor?	Szerep
`order_id`	rendelés rögzítésekor	igen	join kulcs
`line_id`	rendelés rögzítésekor	igen	tételsor azonosító
`product_id`	katalógus	igen	nem starter feature
`quantity`	kosár, t_0	igen	nem starter feature
`unit_price_minor`	árazás, t_0	igen	nem starter feature
`discount_minor`	árazás, t_0	igen	nem starter feature
`line_total_minor`	származtatott, t_0	igen	nem starter feature
`currency`	árazás, t_0	igen	konstans `EUR`

Az `order_items` minden mezője a rendelés leadásakor ismert: a kosár tételei akkor rögzülnek. A kezdő modell mégsem közvetlenül ezeket a sorokat használja, hanem az `orders` táblában lévő kosárösszegzőket. A későbbi SQL-fejezetben látjuk majd, hogy a tételszintű join miért duplázhatja a szállítási díjat, mert az `shipping_fee_minor` rendelésenként egyszer szerepel.

A kezdő feature-listában szereplő `segment`, `region`, `acquisition_channel` a `customers.csv`-ből, az `account_age_days` pedig a `signup_date` és az `order_ts` különbségeként származik. Mivel a regisztráció dátuma minden rendelés előtt van, ezek a mezők a rendelés pillanatában szintén ismertek.

A következő kód a projektgyökérből futtatva a két CSV fejlécét ellenőrzi a várt halmazokkal, és megnézi, hogy a jövőbeli mezők nem kerültek-e a starter feature-listába. A kód csak a Python standard libraryjét használja. Az itt látható kimenet a szerződésből levezetett várt kimenet, nem tényleges futási jegyzék; a laborban való futtatáskor a `lab/data/manifest.json` alapján ellenőrizhető.

```
from pathlib import Path
import csv

PREDICTION_TIME_FIELDS = {
    "order_id", "customer_id", "order_ts", "promised_days",
    "quantity_sum", "item_rows", "basket_minor", "shipping_fee_minor",
    "currency", "distance_km", "priority_flag",
}
LATER_FIELDS = {
    "delivery_days", "actual_delivery_at", "label_available_at", "target_late",
}
FORBIDDEN_AT_PREDICTION_TIME = LATER_FIELDS

STARTER_FEATURES = {
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
}
ORDER_ITEMS_EXPECTED = {
    "order_id", "line_id", "product_id", "quantity",
    "unit_price_minor", "discount_minor", "line_total_minor", "currency",
}

def header(path):
    with Path(path).open(newline="", encoding="utf-8") as f:
        return set(csv.DictReader(f).fieldnames or [])

orders_fields = header("lab/data/orders.csv")
items_fields = header("lab/data/order_items.csv")
```

```
print("orders mezők száma:", len(orders_fields))
print("hiányzó orders mezők:", sorted((PREDICTION_TIME_FIELDS | LATER_FIELDS) - orders_fields))
print("váratlan orders mezők:", sorted(orders_fields - (PREDICTION_TIME_FIELDS | LATER_FIELDS)))
print("order_items hiányzó mezők:", sorted(ORDER_ITEMS_EXPECTED - items_fields))
print("order_items váratlan mezők:", sorted(items_fields - ORDER_ITEMS_EXPECTED))

assert FORBIDDEN_AT_PREDICTION_TIME.isdisjoint(STARTER_FEATURES)
print("szivárgás-ellenőrzés: OK")
```

Futtatás platformjelölt munkakönyvtárral:

```
# Linux/macOS terminál, a projektgyökérből:
python s002_availability_check.py

# Windows PowerShell, a projektgyökérből:
py s002_availability_check.py
```

A `csv.DictReader` a fejlécet olvassa be; a `fieldnames` az oszlopnevek listája, a `set()` pedig halmazként kezeli őket, így a sorrend nem számít. A halmazkivonás (`-`) a hiányzó és váratlan mezőket adja; a `sorted()` csak a kimenet megjelenítését teszi determinisztikussá.

1.2.6 Eredmény értelmezése

A szerződésből levezetett várt kimenet:

```
orders mezők száma: 15
hiányzó orders mezők: []
váratlan orders mezők: []
order_items hiányzó mezők: []
order_items váratlan mezők: []
szivárgás-ellenőrzés: OK
```

A 15 orders-mezőből 11 a `PREDICTION_TIME_FIELDS`, 4 a `LATER_FIELDS` halmazban van; a két halmaz diszjunkt, és az `order_items` 8 mezője mind a rendeléskor ismert. Az `assert FORBIDDEN_AT_PREDICTION_TIME.isdisjoint(STARTER_FEATURES)` az invariáns: ha valaki véletlenül beteszi a `delivery_days`-t a starter feature-listába, a `delivery_days`, `actual_delivery_at`, `label_available_at` vagy `target_late` bármelyike azonnal `AssertionError`-t ad. Az üres halmazok tehát nem formálisak: pontosan azt ellenőrzik, hogy a CSV séma nem tért el a meghirdetett szerződéstől, és a prediktorhalmaz nem lát a jövőbe.

Két időbélyeg, két eltérő szabály. A forrásesemény időpontja (`event_time`) és a rendszerben való tényleges elérhetőség (`available_at`/`ingested_at`) különbözhet. Egy feature csak akkor megengedett a t predikciós pillanatban, ha az előállításához szükséges minden esemény *és* minden adat hozzáférhető volt legkésőbb t-kor. Az utólag javított, visszadátumozott mező nem tekinthető történetileg ismertnek.

Ellenpróba: készíts olyan ügyfelet, akinek hétfőn keletkezett, de szerdán beérkezett eseménye van. A keddi as-of tábla nem használhatja ezt az eseményt, még akkor sem, ha az esemény időbélyege hétfő.

1.2.7 Feladatok

1. **Értelmezés.** Egy rendelés `order_ts = 2025-04-28T09:00:00+00:00`, `label_available_at = 2025-05-04T18:00:00+00:00`. A tanítási cutoff `2025-05-01T00:00:00+00:00`. Melyik split-halmazba tartozik a sor? Fogalmazza meg mindkét kritériumot, és mondja ki, miért nem kerülhet sem a tanító, sem a teszt halmazba.

2. **Megvalósítás.** Írj `split_label(order_ts, label_available_at, cutoff)` függvényt, amely a `"train"`, `"test"` vagy `"purgatory"` sztringet adja vissza. Teszteld a három esetet: (a) rendelés 2025-04-10, label 2025-04-20; (b) rendelés 2025-05-02, label 2025-05-10; (c) rendelés 2025-04-28, label 2025-05-04.

3. **Hibakeresés.** Egy kolléga 0,99 teszt AUC-t mutat a késési modellre, mert a feature-listába betette a `delivery_days` oszlopot. Mi a hiba lényege? Hogyan mutatja ki a rendelkezésreállási tábla? Mi a javítás, és mit mondana a 0,99-es AUC-ról?

4. **Megváltozott feltétel.** A logisztikai rendszer új `planned_dispatch_date` mezőt ad, amely a rendelés leadásakor ismert tervezett feladási napot tartalmazza. A rendszer azonban ezt a dátumot utólag is frissíti. Milyen feltétellel lehet prediktorként használni, és mit kell rögzíteni a provenance-ban?

1.2.8 Megoldások és indoklás

1. A sor a kihagyott határsávba tartozik. A tanítási feltétel első fele igaz: `order_ts < cutoff`, mert 2025-04-28 megelőzi 2025-05-01-et. A második fele hamis: `label_available_at = 2025-05-04 18:00`, ami nem szigorúan korábbi a cutoffnál. A teszt feltétele is hamis, mert a teszt definíciója `order_ts >= cutoff`. A rendelés tehát sehol sem tanítható a cutoff ismeretében, és a modell buildjekor nem játszhatja el, hogy a címkéje már rendelkezésre áll.

2. Lehetséges megoldás Python 3.12 `datetime` moduljával:

```
from datetime import datetime

def split_label(order_ts, label_available_at, cutoff="2025-05-01T00:00:00+00:00"):
    t0 = datetime.fromisoformat(order_ts)
    tl = datetime.fromisoformat(label_available_at)
    c = datetime.fromisoformat(cutoff)
    if t0 >= c:
        return "test"
    if tl < c:
        return "train"
    return "purgatory"

assert split_label("2025-04-10T09:00:00+00:00", "2025-04-20T18:00:00+00:00") == "train"
assert split_label("2025-05-02T09:00:00+00:00", "2025-05-10T18:00:00+00:00") == "test"
assert split_label("2025-04-28T09:00:00+00:00", "2025-05-04T18:00:00+00:00") == "purgatory"
```

A `datetime.fromisoformat` a `+00:00` offsetet is értelmezi, így mindhárom érték időzóna-tudatos. Ha valamelyik bemenet nem tartalmazna offsetet, a naiv és az offset-tudatos időpont összehasonlítása `TypeError`-t adna; a szerződés éppen ezért követeli meg az explicit `+00:00` offsetet. A várt ellenőrzési invariáns: adott cutoff mellett a három halmaz diszjunkt, és minden sor pontosan egy címkét kap.

3. A hiba a jövőbeli adat szivárgása. A rendelkezésreállási tábla szerint a `delivery_days` ismertté válási ideje `actual_delivery_at`, ami mindig későbbi az `order_ts` predikciós pillanatnál. A teszt kiértékelésekor a modell olyan inputot kapott, amely a termelési

predikció pillanatában nem létezik; ezért a 0,99 AUC nem valós képességet mér, hanem azt, hogy a modell megtanulta a kézbesítési eredményt. A javítás: törölni a `delivery_days`-t a prediktorlistából, a pipeline-t változatlan beállításokkal újra futtatni, és az új, ténylegesen mért teszt AUC-t értékelni. A 0,99 így nem bizonyíték, hanem a folyamat hibájának tünete.

4. A mező a rendelkezésreállási táblában akkor kaphat „ismert t_0-kor” bejegyzést, ha a `planned_dispatch_date` a rendelés leadásakor ténylegesen kiszámítható, és a modell inputja a rendeléskori érték. Ha a rendszer utólag frissíti, a CSV-ben szereplő jelenlegi érték már nem feltétlenül az a dátum, amelyet a rendelés pillanatában ismertek; ebben az esetben a frissített mező jövőbeli információt tartalmaz. Csak as-of pillanatfelvételből szabad dolgozni: a táblának azt az értéket kell tárolnia, amely az `order_ts` időpontban érvényes volt. A provenance-ban rögzíteni kell a forrást, az érték időbélyegét, a generáló folyamat verzióját és azt a szabályt, hogy a logisztikai rendszer utólagos frissítései nem írhatják felül a predikciós inputot.

1.2.9 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés: *a cél kiszámításához szükséges jövőbeli adat nem automatikusan engedélyezett feature.* A `delivery_days` a tanító CSV-ben fizikailag jelen van, és a modell kiválóan illeszkedik rá; a teszt AUC csábítóan magas lehet. De a predikció pillanatában a kézbesítés még nem történt meg, így a mező nem létezik. Ugyanez igaz a `label_available_at`-re: ez nem a label értéke, hanem a taníthatóság időbélyege, és prediktorként használva szintén a jövőbe lát. Az `s001`-ben rögzítettük, hogy a modell a rendelés pillanatában fut; most az as-of idő ezt a pillanatot formális korláttá teszi. Önellenőrző kérdés minden új feature-höz: „Ismerném-e ezt az értéket akkor, amikor a rendelés beérkezik?” Ha a válasz nem, a mező tiltott, akármilyen jó AUC-t ígér.

1.2.10 Rövid összefoglaló

Minden mezőhöz rendeljünk ismertté válási időt: egy prediktor csak akkor használható, ha az érték a predikció pillanatában már létezik. A Northstar `orders` táblájában 11 mező ismert a rendeléskor, 4 pedig csak kézbesítés után; a `delivery_days` a label alapanyaga, nem prediktor. A tanításhoz a rendelésnek és a címkének is a cutoff előtt kell lennie; a kihagyott határsáv a címke késése miatt nem tanítható sorokat gyűjti. A nyers és tiszta adat, a manifest és az auditnapló együtt adja a provenance-t. A következő alfejezetben ezt az időbeli fegyelmet a reprodukálható munkanaplóba építjük be.

1.3 Reprodukálhatóság és tudományos munkanapló

Belépő · szint: alap · idő: 45 perc · eszköz: Python 3.12, pandas

Előfeltétel: 1.2. alfejezet · Célkészség: Reprodukálhatóság és tudományos munkanapló · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

1.3.1 Cél és konkrét helyzet

A Northstar tanulóprojektben a `lab/generate_data.py` állítja elő a szintetikus adatokat. Az előző alfejezetben rögzítettük, hogy egy mező mikor válik ismertté, és hogy a nyers adat, a tiszta adat és a manifest együtt adja a származási láncot. Most az a kérdés: ugyanaz a generátorfolyamat két külön mappában, ugyanazzal a seeddel vajon bájtról bájtra ugyanazt a CSV-t adja-e, és mi történik, ha pontosan egy bemeneti paramétert, a seedet változtatjuk meg?

A bemenet itt maga a generátorkód, a környezet és a paraméterezés; a megfigyelési egység egy-egy kimeneti CSV-fájl bájttartalma; az eredmény e fájlok SHA-256 lenyomata, valamint a futásról készített jegyzék. Előzetes becslés: ha a generátor determinisztikus, az azonos seeddel készült `customers.csv`, `orders.csv`, `order_items.csv` és `orders_raw.csv` hash-ei páronként azonosak lesznek. Ha csak a seedet változtatjuk, legalább a tartalmi hash-ek eltérnek, még akkor is, ha a fájlok neve és szerkezete nem változik.

1.3.2 Determinizmus, véletlen és a seed szerepe

Egy lépést **determinisztikusnak** nevezünk, ha ugyanaz a bemenet és ugyanaz a programállapot mindig ugyanazt a kimenetet adja. Ilyen például egy CSV-sor összeállítása, egy fájl kiírása vagy egy hash kiszámítása. Ezzel szemben egy **sztochasztikus lépés** egy véletlen forrás állapotát is felhasználja: a kimenet a lehetséges értékek egy halmazából kerül ki, és a lépés megismétlése önmagában nem garantálja ugyanazt az eredményt.

A számítógépes szimulációban a „véletlen” általában **pszeudovéletlen**: egy determinisztikus függvény állítja elő a sorozatot, de a sorozat úgy viselkedik, mintha véletlen húzásokból állna. A **seed** (mag) ennek a függvénynek a kezdeti állapotát rögzíti. Ugyanaz a seed ezért ugyanazt a pszeudovéletlen sorozatot választja; más seed más sorozatot indít. A seed rögzítése a sztochasztikus lépést gyakorlati értelemben ismételhetővé teszi, de nem szünteti meg a mögöttes kísérleti bizonytalanságot: a későbbi statisztikai fejezetekben például egy becsült hatás a minta véletlene miatt akkor is ingadozik, ha a szimuláció seedje rögzített.

Kis kézi példa egy egyszerű lineáris kongruens generátorral:

$$x_{n+1} = (5x_n + 1) \bmod 16,$$

ahol x_n a n-edik állapot, értéke a $\{0, 1, \dots, 15\}$ halmazból való, $n = 0, 1, 2, \dots$ indexű, a seed pedig x_0. Ha $x_0 = 3$, a sorozat:

$$3, 0, 1, 6, 15, 12, 13, 2, 11, 8, 9, 14, 7, 4, 5, 10, 3, \dots$$

Ugyanezt a számolást bárki megismétli, és pontosan ugyanezt kapja. Ha viszont $x_0 = 1$, a sorozat $1, 6, 15, 12, \dots$, tehát más. Ez a példa nem ipari minőségű véletlenszám-generátor, de pontosan mutatja a seed szerepét: egy rövid állapotból hosszú, előre meghatározott sorozat indul.

A fájlok összehasonlításához **kriptográfiai hash**-t, például SHA-256-ot használunk. A hash egy determinisztikus függvény:

$$h : \{0,1\}^* \to \{0,1\}^{256},$$

ahol $\{0,1\}^*$ tetszőleges hosszúságú bájt-sorozat, $\{0,1\}^{256}$ pedig a 256 bites kimenetek halmaza. Azonos bemenetre azonos kimenetet ad; ha két fájl hash-e eltér, akkor a fájlok biztosan eltérnek.

A hash-összehasonlítás gyakorlati ellenőrzésre való: egyezésnél nem bizonyítja matematikai értelemben, hogy a két fájl azonos, de a gyakorlatban ez az elfogadott ujjlenyomat-módszer. A Northstar `manifest.json` ugyanezt az elvet használja: seedet, sorszámokat, hash-eket és kontrollösszegeket rögzít.

1.3.3 Futási jegyzék és hipotézisnapló

A **futási jegyzék** (run log) a ténylegesen elvégzett lépések időrendi feljegyzése. Egy sor akkor hasznos reprodukáláshoz, ha tartalmazza:

- a lépés időpontját vagy sorszámát,
- a pontos parancsot és a bemenet verzióját (fájl, paraméter, seed),
- a megfigyelt kimenetet (hash, sorszám, hibaüzenet),
- az abból következő döntést.

A **kontrollált változtatás** (controlled change) azt jelenti, hogy két futás között pontosan egy tényezőt változtatunk, a többit rögzítjük. Ha két dolgot egyszerre változtatunk, nem tudjuk eldönteni, melyik okozta az eltérést. A Northstar-generátornál a bemeneti tényezők közé tartozik legalább a seed, a customers és az orders paraméter. A tanulási feladat ezért először csak a seedet változtatja, a többi paramétert változatlanul hagyja.

A **hipotézisnapló** a döntést megelőző várakozást és a döntést követő ellenőrzést kapcsolja össze. Minden sorhoz három elem tartozik:

- a hipotézis vagy elhatározás, például „a generátor ugyanazzal a seeddel bitazonos CSV-t ad”;
- az a megfigyelés, amely alátámasztaná, és az a megfigyelés, amely cáfolná;
- a mérés elvégzésének időpontja és a mérés után hozott döntés.

A futási jegyzék a múltat rögzíti, a hipotézisnapló a várakozás és az eredmény közötti kapcsolatot teszi láthatóvá. Ez a kettő együtt adja a tudományos munkanapló legfontosabb részét. A naplóba az 1.2. alfejezetben megismert módon érdemes azt is feljegyezni, hogy egy adott döntés pillanatában mely adatmezők voltak ismertek: a reprodukálhatóság nemcsak a számolásé, hanem az információ időbeli rendjéé is.

1.3.3.1 Automatikus futtatási jegyzék: `run_manifest.json`

A kézi jegyzet akkor használható géppel is, ha a futásról szabványos JSON készül. A `lab/run_manifest.py` ezt állítja elő: a parancsot subprocessben futtatja, és rögzíti a kód, a bemenetek és a kimenetek SHA-256 lenyomatát, a Python- és csomagverziókat, a paramétereket, a kilépési kódot és a futásidőt. A séma a `run_manifest.py` `SCHEMA` objektuma (JSON Schema draft 2020-12); a kötelező mezők:

Mező	Tartalom
`schema_version`	a jegyzék sémájának verziója, például `1.0`
`run_id`, `created_at`	egyedi futásazonosító és időbélyeg
`command`, `working_directory`	a pontos parancs és a munkakönyvtár
`code`	a futtatott kód fájljai hash+byte-count formában
`environment`	Python-verzió, implementáció, platform, csomagverziók

Mező	Tartalom
`parameters`	seed és minden futási paraméter
`inputs`, `outputs`	bemeneti és kimeneti fájlok hash+byte-count formában
`stdout_sha256`	a kimenet hash-e, ha nincs fájlkimenet
`exit_code`, `status`, `runtime_seconds`	kilépési kód, `ok`/`failed`, mért futásidő
`started_at`, `finished_at`	a futás kezdete és vége

Rögzítés és ellenőrzés a könyv gyökeréből:

```
python3 lab/run_manifest.py record \
    --manifest lab/run_manifest.example.json \
    --code lab/quickcheck.py --input-dir lab/data \
    --parameter seed=20261020 --parameter data=lab/data \
    -- python3 lab/quickcheck.py --data lab/data
python3 lab/run_manifest.py verify --manifest lab/run_manifest.example.json
```

A `verify` újrafuttatja a parancsot, és összehasonlítja a kilépési kódot, a kimenet hash-ét, valamint a kód- és bemeneti hash-eket. A könyv készítésekor rögzített példa (`lab/run_manifest.example.json`) futásideje 0.050527 s, `stdout_sha256` értéke `204722100260771a69ce840ca74a0f0388a6c75595988e51a750d06042af2bfb`, és a `verify` eredménye `ok: true`. A jegyzék kivonata:


```
{
  "schema_version": "1.0",
  "run_id": "20261029T102955Z-2370c509",
  "command": ["python3", "lab/quickcheck.py", "--data", "lab/data"],
  "code": {"lab/quickcheck.py": {"sha256": "c3849c3e…", "bytes": 2992}},
  "environment": {"python": "3.12.3", "implementation": "CPython",
                  "platform": "Linux-7.0.0-38-generic-x86_64-with-glibc2.39"},
  "parameters": {"seed": "20261020", "data": "lab/data"},
  "inputs": {"lab/data/orders.csv": {"sha256": "…", "bytes": 240000}},
  "outputs": {},
  "stdout_sha256": "204722100260771a69ce840ca74a0f0388a6c75595988e51a750d06042af2bfb",
  "exit_code": 0,
  "status": "ok",
  "runtime_seconds": 0.050527
}
```


A `verify` csak akkor ad `ok: true`-t, ha a kód és a bemenetek hash-e változatlan, a parancs újrafuttatása ugyanazzal a kimenettel és kilépési kóddal zárul, és a rögzített kimeneti fájlok hash-e is egyezik. Ha a seedet vagy egy bemeneti CSV-t megváltoztatunk, a `changed` lista megnevezi az eltérést. A `tests/test_run_manifest_schema.py` a séma kötelező mezőit, a példa újrafuttathatóságát, a módosított bemeneti hash észlelését és a determinisztikus `stdout`-ot teszteli.

1.3.4 Teljes minimális munkapélda

A következő munkamenetet a tanuló gépén kell futtatni; a könyv szövegének írásakor ezt a kódot nem futtattuk, ezért a megjelenő kimenet **elméletileg várt**, nem mért. A `lab/generate_data.py` pontos kapcsolóit a helyi `--help` kimenete mutatja; ha a kapcsoló neve eltér, a döntési logika ugyanez.

Első lépésként hozzunk létre két kimeneti mappát, és futtassuk ugyanazt a generátort ugyanazzal a seeddel.

```
# Linux/macOS, a könyv projektgyökeréből
mkdir -p runs/run_a runs/run_b
python lab/generate_data.py --seed 20261020 --customers 300 --orders 2400 --output runs/run_a
python lab/generate_data.py --seed 20261020 --customers 300 --orders 2400 --output runs/run_b
```

Windows PowerShellben ugyanez a két parancs `mkdir runs\run_a, runs\run_b` előkészítés után, a `--output` kapcsolóval. A fájlok összehasonlítását platformfüggetlen Python-szkripttel végezzük, mert így a könyv fő eszközkészlete is gyakorolható:

```
# compare_hashes.py
import hashlib
import sys
from pathlib import Path

FILES = ["customers.csv", "orders.csv", "order_items.csv", "orders_raw.csv"]

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    h.update(path.read_bytes())
    return h.hexdigest()

def main(left: Path, right: Path) -> int:
    failed = False
    for name in FILES:
        a = sha256(left / name)
        b = sha256(right / name)
        status = "egyezik" if a == b else "ELTÉR"
        print(f"{name:20s} {status}  {a[:16]}  {b[:16]}")
        failed = failed or (a != b)
    return 1 if failed else 0

if __name__ == "__main__":
    sys.exit(main(Path(sys.argv[1]), Path(sys.argv[2])))
```

A futtatás:

```
python compare_hashes.py runs/run_a runs/run_b
```

Elméletileg várt kimenet: mind a négy CSV esetén `egyezik` állapot és azonos hash-előtag, a folyamat kilépési kódja `0`.

Ezután hajtsuk végre a kontrollált változtatást: csak a seedet írjuk át `20261021`-re, minden más paramétert megtartva.

```
python lab/generate_data.py --seed 20261021 --customers 300 --orders 2400 --output runs/run_c
python compare_hashes.py runs/run_a runs/run_c
```

Itt az elméletileg várt eredmény az, hogy a négy CSV hash-e eltér. A `customers=300` és `orders=2400` paraméter megmarad, ezért a vásárlók és rendelések sorainak száma lehet azonos, de a tartalom más; az `order_items.csv` sorainak száma is változhat, mert az alapméret csak a `seed=20261020` futásra érvényes. A pontos ellenőrzési alap minden futásnál a saját `manifest.json`, nem az alapkiadás adatai.

A futási jegyzék formátuma például ilyen lehet; a cellák tartalmát a saját futásodból kell kitöltened:

Idő	Parancs és input	Megfigyelt eredmény	Döntés
1.	`generate_data.py`, seed=20261020, output `run_a`	CSV-k és manifest létrejöttek	elfogadva
2.	ugyanaz seed=20261020, output `run_b`	mind a négy CSV hash egyezik	determinisztikusnak tekintjük
3.	ugyanaz seed=20261021, output `run_c`	CSV hash-ek eltérnek `run_a`-tól	seed kontroll alatt tart

A hozzá tartozó hipotézisnapló első három sora:

1. **Hipotézis:** a generátor azonos paraméterekkel bitazonos CSV-t ad. **Alátámasztó jel:** a `run_a` és `run_b` hash-ei egyeznek. **Cáfoló jel:** bármelyik hash eltér. **Időpont és döntés:** az `run_b` futás után ellenőrizve; a generátort ezen a környezeten reprodukálhatónak tekintjük.
2. **Hipotézis:** a seed része a bemenetnek. **Alátámasztó jel:** `run_c` hash-e eltér `run_a`-tól. **Cáfoló jel:** minden hash azonos marad. **Időpont és döntés:** a `run_c` futás után ellenőrizve; a seedet minden jövőbeli futásnál rögzítjük.
3. **Hipotézis:** a manifest a futásra jellemző ellenőrző összeget ad. **Alátámasztó jel:** a manifestek hash- vagy kontrollösszeg-mezői a saját futás CSV-ihez illeszkednek. **Cáfoló jel:** a manifest más seedet vagy hash-t mutat, mint a fájlok. **Időpont és döntés:** minden átadás előtt ellenőrizzük.

1.3.5 Az eredmény értelmezése

A hash-egyezésnél nincs tolerancia: a cél a bájtazonosság, tehát egyetlen eltérő bájt is `ELTÉR` állapotot ad. A kézi kontroll az LCG-példa: $x_0 = 3$ esetén a 16 hosszú periódus újra és újra ugyanúgy ismétlődik, tehát a seed rögzítése a sorozatot is rögzíti. A CSV-k esetében ez azt jelenti, hogy az azonos seed nemcsak hasonló, hanem azonos bájtsorozatot eredményez, feltéve hogy a generátorkód, a Python-verzió, a paraméterek és a fájlkiírási sorrend is azonos.

A shape- és dtype-ellenőrzés a hash-nél gyengébb, de fontos kiegészítő invariáns: az alap seeddel `customers.csv` 300 sort, `orders.csv` 2400 sort, `order_items.csv` 4796 sort tartalmazzon. Ez önmagában nem bizonyítja a tartalom azonosságát, ezért a manifest vagy a hash a döntő. A más seeddel készült `run_c` esetén a shape-egyezés nem is elvárás; a generált adat szerződése szerint az alapméretek csak az alap seedhez tartoznak.

A reprodukálhatóság itt **numerikus és fájlszintű**. A későbbi gépi tanulási futásoknál más lesz a helyzet: ott a seed rögzítése a véletlen műveletek sorrendjét rögzíti, de a lebegőpontos műveletek hardverfüggő sorrendje miatt a bitazonosság nem garantálható. Ezt a különbséget a fejezet végén visszahozzuk.

1.3.6 Feladatok

1. **Értelmezés.** A `run_a` és `run_b` mind a négy CSV hash-e egyezik. Mondd el saját szóval, mit állíthatunk ebből a generátorról, és mit **nem** állíthatunk, ha a második futás egy másik gépen, más Python-verzióval történne.

2. **Megvalósítás.** Egészítsd ki a `compare_hashes.py`-t úgy, hogy a CSV-k mellett a két mappa `manifest.json` fájlját is beolvassa, és akkor is `1` kilépési kóddal térjen vissza, ha a manifestekben a seed vagy bármely CSV-hez tartozó hash mező eltér. A kilépési kód legyen a hívó számára géppel ellenőrizhető.

3. **Hibadiagnózis.** Azt látod, hogy a `run_c/manifest.json` seed mezője `20261021`, a `compare_hashes.py runs/run_a runs/run_c` mégis minden CSV-re egyezést ír ki. Adj két lehetséges magyarázatot, és mindkettőhöz egy-egy konkrét ellenőrző lépést.

4. **Kontrollált változtatás továbbvitele.** Tartsd a seedet `20261020`-on, de változtasd a customers paramétert `300`-ról `301`-re, az orders paramétert hagyd `2400`-on. Mely kimeneti fájlokban várasz változást, és milyen ellenőrzéssel döntöd el, hogy a változás valóban a customers paramétertől származik?

5. **Hipotézisnapló.** Írj be a naplóba egy negyedik hipotézist arról, hogy a generátor kimenete függ-e a fájlkiírás előtti munkakönyvtár nevétől. Add meg az alátámasztó és cáfoló megfigyelést, valamint a mérési tervet.

1.3.7 Megoldások és indoklás

1. Az egyezésből azt állíthatjuk, hogy az adott gépen, az adott Python-verzióval és az adott paraméterekkel a generátor két futtatása azonos CSV-ket adott. Nem állíthatjuk, hogy más gépen vagy más Python-verzióval is bitazonos lesz: a generátor használhat olyan standard könyvtári részeket, amelyek viselkedése verzióhoz kötött, és a fájlrendszer, a karakterkódolás vagy a lebegőpontos formázás is különbözhet. A környezet ugyanúgy része a reprodukálási feltételnek, mint a seed.

2. A `manifest.json` beolvasása után például ellenőrizheted, hogy a két seed mező egyezik-e, majd a CSV-nevekhez tartozó hash-eket összevetheted. Az invariáns: a kilépési kód `0` pontosan akkor, ha minden összehasonlított mező egyezik. A hash-összevetésnél nincs tűréshatár; a szöveges mezőknél a pontos egyezést kérjük.

3. Két lehetséges magyarázat:

   - A `run_c` mappa valójában a régi fájlokat tartalmazza, például a parancs `--output` értéke elgépelés miatt nem oda írt, vagy a mappa nem ürült ki. Ellenőrzés: nézd meg a `run_c` fájlok módosítási idejét és a manifestben tárolt CSV-hash-eket; hasonlítsd a `run_c` manifestjét a `run_a` manifestjéhez.
   - A parancsban a seed ugyan megváltozott, de a hash-összehasonlító rossz könyvtárat kapott: például a második argumentum is `runs/run_a`. Ellenőrzés: írasd ki a scriptben a teljes elérési utat, és futtasd a manifest-összevetést is.

4. A customers paraméter az ügyfélfájlt közvetlenül érinti, ezért `customers.csv` sorszáma 300-ról 301-re változik. Az `orders.csv` is változhat, mert az ügyfélazonosítók halmaza más lesz; az `order_items.csv` és az `orders_raw.csv` szintén változhat az előállítási lánc miatt. A kontrolláltság abban áll, hogy a seedet és az orders paramétert rögzítjük, így ha a `run_a`-hoz képest valamelyik fájl hash-e eltér, az a customers változtatás következménye. Az ellenőrzés alapja a `run_301` saját `manifest.json`, nem az alapkiadás méretei.

5. Például: „A generátor által kiírt CSV-k tartalma nem függ a kimeneti mappa nevétől.” Alátámasztó jel: `runs/run_a` és egy eltérő nevű, például `runs/other_name` mappába írt azonos seedű futás hash-ei egyeznek. Cáfoló jel: bármelyik hash eltér. Mérési terv: két új mappa létrehozása, azonos seed, hash-összevetés, az eredmény bejegyzése a futási jegyzékbe.

1.3.8 Gyakori tévedés

„Elég egy seedet rögzíteni, és a gépi tanulási futás minden gépen bitazonos lesz.” Ez nem igaz. A seed a pszeudovéletlen műveletek **sorrendjét** rögzíti, például azt, hogy melyik minták kerülnek egy-egy kötegbe. A hardver azonban a lebegőpontos összeadás sorrendjét párhuzamosításkor megváltoztathatja: egy GPU redukciója másként csoportosíthatja az összeadandókat, mint egy CPU, és a lebegőpontos kerekítés sorrendfüggő. Ettől a veszteség vagy a súlyok utolsó tizedesjegyei eltérhetnek, noha a véletlen döntések ugyanazok voltak. A tanulási példánkban a CSV-generátor nem használ ilyen párhuzamos lebegőpontos redukciót, ezért fájlszintű bitazonosságot várhatunk; ML-futásnál a reprodukálhatóság szintje külön vizsgálandó.

Egy másik félreértés: a seed egyezése adatminőségi garanciát is jelentene. A `orders_raw.csv` ugyanazzal a seeddel is tartalmazza a szándékos hibákat, mert azok a generátor specifikációjának részei. A seed csak az előállítás reprodukálhatóságát szolgálja; a tisztaságot külön ellenőrzéssel kell igazolni.

1.3.9 Önellenőrzés

Kérdezd végig magadon a friss munkanaplóddal kapcsolatban: le tudom-e írni a generátorfutás pontos parancsát, seedjét és paramétereit? A jegyzékemben külön szerepel-e a tervezett lépés és a ténylegesen mért kimenet? Minden döntésemhez tartozik-e előre rögzített alátámasztó és cáfoló megfigyelés? Meg tudom-e nevezni, hogy a fájlszintű reprodukálhatóság mely feltételei hiányoznak egy másik gépre költözéskor? Ha bármelyik kérdésre nincs válasz, a naplót egészítsd ki, mielőtt a következő diagnózisfeladathoz kezdesz.

Reprodukálhatóság szintjei. A rögzített véletlen mag nem garantál minden operációs rendszeren vagy könyvtárverzión bájtazonos eredményt. Külön kell dokumentálni (1) a forrásadat és a konfiguráció bájtazonosságát, (2) az eredmények számszerű egyezését deklarált toleranción belül és (3) a következtetés robusztusságát. Mindháromhoz más elfogadási feltétel tartozik.

A `sha256` kizárólag azonos fájlbájtokat ellenőriz; nem bizonyítja, hogy az adatok helyesek vagy az elemzés tudományosan érvényes. A munkanapló tartalmazza a Python- és csomagverziókat, a platformot, az inputazonosítót, az elemzőkód commitját és a ténylegesen ellenőrzött kimenetet.

1.3.10 Rövid összefoglaló

A reprodukálhatóság nem egyetlen seed, hanem a kódverzió, környezet, paraméterek, kimeneti adatverzió és a döntések együttes rögzítése. A determinisztikus lépés azonos bemenetből azonos kimenetet ad; a sztochasztikus lépést a seed rögzítése teszi újrafuttathatóvá. A Northstar-generátorral azonos seed mellett bájtazonos CSV-k várhatók, más seed mellett eltérő hash-ek. A futási jegyzék a múltat, a hipotézisnapló a várakozás és a mérés kapcsolatát rögzíti; a hash-ellenőrzésnél nincs tolerancia, az ML-futásoknál a bitazonosság nem általános garancia. A következő alfejezetben ezt a naplózási fegyelmet használjuk egy teljes diagnosztikai munkamenet felépítésére.

1.4 Diagnózis és első teljes munkamenet

Belépő · szint: alap · idő: 45 perc · eszköz: Python 3.12, pandas

Előfeltétel: 1.3. alfejezet · Célkészség: Diagnózis és első teljes munkamenet · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

1.4.1 Diagnózis: programozási, adatértelmezési és statisztikai hiány egy helyi munkamenetben

A data scientist munkája ritkán azzal kezdődik, hogy "futtassunk egy modellt". Gyakran azzal kezdődik, hogy egy látszólag egyszerű ellenőrzés elromlik, vagy egy számról kiderül, hogy mást jelent, mint amit a neve sugall. Ebben az alfejezetben a Northstar labor segítségével egy teljes, kis diagnosztikai munkamenetet építünk fel. A cél nem egy bonyolult modell, hanem az, hogy a három leggyakoribb hibacsaládot — programozási hibát, adatértelmezési hibát és statisztikai túlkövetkeztetést — ugyanabban a feladatban külön tudjuk választani.

Konkrét helyzet. A labor a könyv projektgyökerének `lab/` mappájában van. A bemenet a `generate_data.py` által előállított három tiszta CSV: `data/customers.csv` (300 ügyfél), `data/orders.csv` (2400 rendelés) és `data/order_items.csv` (4796 rendelési tétel). A megfigyelési egység a három fájlban rendre ügyfél, rendelés és rendelési tétel; ezt a szemcsézettséget tilos összemosni. A kimenet egy saját futási jegyzék, amelyben legalább a környezet, a három kontrollérték és a vizsgált alapmutató szerepel.

A laborcsomag beszerzése. A Northstar-labor önálló, verziózott csomagja a kiadvány forráskiadásában a `lab/` mappa, illetve a könyv letöltési csomagjában a `northstar-lab-1.0.0.zip` fájl (a `lab/` mappából a `python3 lab/package_lab.py` paranccsal újraépíthető). A ZIP SHA-256 lenyomata `91557feda5a63df9cd9de1a3dfaec16af35ca9c5a0765ef1f5794678d45f8f55`; a csomag `README.md`, `LICENSE`, `ADATOK.md`, `manifest.json`, hat CSV, a generátor, a `quickcheck` és a szerződéstesztek mellett a `reference/uv.lock` referenciafeloldást és a hét kiemelt gyakorlat kiindulását tartalmazza. A csomagot kicsomagolva a `northstar-lab-1.0.0/` könyvtárban kell dolgozni; a `README.md` adja a pontos parancsokat és a gyakorlatok kimeneteit.

Előzetes becslés a megoldás előtt. Mielőtt futtatsz bármit, írd be a hipotézisnaplódba: szerinted a tiszta `orders.csv`-ben a rendelések hány százalékánál késik a kézbesítés a 7 napos vállalt határidőn túl? Adj egy alsó és egy felső határt, például "4% és 12% között". Ezt a tippet később nem pontozzuk; az a szerepe, hogy rögzítse a várakozásodat, mielőtt a mérés befolyásolná a memóriádat. Ez az 1.3. alfejezetben bemutatott hipotézisnapló legegyszerűbb formája.

1.4.2 Darabszám, arány és százalékpont

A diagnosztikai feladat matematikája szándékosan kicsi. A **darabszám** *(count)* rekordok számát jelenti: hány sor van a fejlécen kívül. Az **arány** *(proportion, rate)* egy részsokaság darabszámának és a teljes nevezőnek a hányadosa. Ha N a rendelések száma, N_{late} pedig azok száma, amelyeknél a `target_late` oszlop értéke `1`, akkor a késői kézbesítés aránya

$$p_{\text{late}} = \frac{N_{\text{late}}}{N}.$$

Itt p_{late} dimenzió nélküli szám, értékkészlete $[0, 1]$. Százalékban $100 \cdot p_{\text{late}}$ szerepel, mértékegysége **%**. A **százalékpont** *(percentage point, pp)* két, már százalékban megadott arány különbsége. Ha az egyik időszakban $p_0 = 0{,}060$ (azaz 6,0%), egy másikban $p_1 = 0{,}080$ (8,0%), akkor a különbség 2,0 százalékpont, ugyanakkor a relatív növekedés $(p_1 - p_0)/p_0 = 0{,}333$, azaz 33,3%. A kettő keverése az egyik legkönnyebben elkövethető hiba a jelentésekben.

A diagnózis másik tengelyét a három hibacsalád adja:

- **Programozási hiány:** a kód nem azt számolja, amit gondolunk; például a CSV-olvasó minden mezőt szövegként ad vissza, így a `target_late == 1` összehasonlítás soha nem igaz.

- **Adatértelmezési hiány:** a számítás lefut, de rossz szemcsézettségen vagy rossz jelentésen alapul. A 4796 tételsor nem 4796 rendelés; a rendelésenként egyszer felszámított szállítási díj tételszintű összegzése hibásan megsokszorozná a pénzt.
- **Statisztikai hiány:** a mutatót bizonytalanság és a mintavételi egység megnevezése nélkül kezeljük tényként. Egyetlen szintetikus fájl késési aránya pontbecslés, nem üzleti igazság.

1.4.3 Teljes minimális munkamenet

A labor szándékosan **zero-dependency**: a `generate_data.py` és a `quickcheck.py` csak a Python standard library-t használja, tehát nem kell `uv`-környezetet létrehoznunk. A munkakönyvtár a `lab/`.

```
# platform: Linux/macOS vagy Windows PowerShell; munkakönyvtár: a projekt lab/ mappája
python --version
python generate_data.py
python quickcheck.py
```

A `python --version` kimenete a saját gépeden látszik; a tantervi alap Python 3.12, de a generátor ennél régebbi Python 3-ason is futtatható, ha a szintaxisa kompatibilis. A `generate_data.py` újraírja a szállított kis CSV-ket a rögzített seeddel. A `quickcheck.py` ezután a `manifest.json` alapján ellenőrzi a fájlokat. A három szerződött rekordszám a mellékelt adatszerződés szerint: `customers.csv` 300, `orders.csv` 2400, `order_items.csv` 4796. A pontos kimeneti formátum a labor verziójától függ; a döntő az, hogy e három értéknek a manifesttel egyeznie kell. Ebben az alfejezetben nem állítjuk, hogy most lefuttattuk a labort: a tényleges futás a te gépeden történik, és a rögzített kimenetet a te jegyzékedbe írod.

Ha a munkamenet elakad. (1) A munkakönyvtár a `lab/` (vagy a kicsomagolt `northstar-lab-1.0.0/`) legyen; a `pwd` (PowerShellben `Get-Location`) ezt mutassa. (2) A `generate_data.py` után a `data/` hat CSV-je jöjjön létre; ha nem, a generátor hibaüzenete mondja meg, melyik előfeltétel hiányzik. (3) A `quickcheck.py` hash-eltérése esetén a hibás `data/` fájlt (vagy az egész `data/` mappát) törölni kell, majd a generátort újra kell futtatni; a manifest az újrafuttatáskor frissül. (4) Ha a `lab/` mappa hiányzik, a `northstar-lab-1.0.0.zip`-t kell kicsomagolni; ha a ZIP sincs, a forráskiadásból `python3 lab/package_lab.py` állítja elő. (5) Windows PowerShellben a `python` helyett `py -3` is használható; a többi parancs változatlan.

A `quickcheck` futtatása után a "kézi kontroll" azt jelenti, hogy legalább egyszer magunk is megszámoljuk a sorokat, nem csak elfogadjuk a kész kimenetet. A következő kis program a CSV-olvasót használja, így platformfüggetlenül, a fejléccel nem számolva adja vissza a rekordok számát:

```
from pathlib import Path
import csv

expected = {
    "customers.csv": 300,
    "orders.csv": 2400,
    "order_items.csv": 4796,
}

for name, want in expected.items():
    with (Path("data") / name).open(newline="", encoding="utf-8") as f:
        reader = csv.reader(f)
        header = next(reader)
        got = sum(1 for _ in reader)
    print(f"{name}: rows={got} columns={len(header)} expected={want} ok={got == want}")
```

A kimenet várható alakja mindhárom fájlra `ok=True`; a futási jegyzékben a generátor és a `quickcheck` pontos kimenetét is rögzítsd. A hash-ellenőrzésnél nincs tolerancia: a seed és a generátorverzió együtteséből bájtazonos állományok következnek, ezért itt az "elég közel" nem elfogadható. Ezt a szigorúságot az 1.3. alfejezet futási jegyzékeléséből hozzuk át.

Ezután számítsuk ki az alapmutatót a rendelésszintű célváltozóra. A `target_late` a tiszta `orders.csv`-ben 0 vagy 1: 1, ha `delivery_days > promised_days`, azaz a kézbesítés késik a vállalt 7 naphoz képest. A tiszta adatállomány végén minden kézbesítés teljesen megfigyelt, ezért a nevező mind a 2400 rendelés lehet:

```
from pathlib import Path
import csv

with Path("data/orders.csv").open(newline="", encoding="utf-8") as f:
    orders = list(csv.DictReader(f))

n = len(orders)
unique_order_ids = {row["order_id"] for row in orders}
late = sum(1 for row in orders if row["target_late"] == "1")
ratio = late / n

print(f"orders: {n}")
print(f"unique order_id: {len(unique_order_ids)}")
print(f"late count: {late}")
print(f"late ratio: {ratio:.4f} ({100 * ratio:.2f}%)")
```

A fenti kódblokk kimenetét szándékosan nem tölthetjük ki a könyv írásakor azzal a látszattal, hogy most futott. A szerződött darabszámok viszont ismertek: `orders: 2400` és `unique order_id: 2400` várható. Ha ez a kettő nem teljesül, akkor a betöltött fájl nem az alap-adatszerződés szerinti, és a további számítás értelmezhetetlen. A `late count` és a `late ratio` értéke a te futtatásodból kerül a jegyzékbe; ezek determinisztikusak a rögzített seed mellett, de itt nem adjuk meg őket előre, hogy a becslés–futás–ellenőrzés ciklus valódi maradjon.

1.4.4 Mit jelent az eredmény?

A három kontrollérték az **adathigiénia** ellenőrzése: a fájlok a szerződött méretűek, tehát a későbbi elemzések ugyanarról a kiindulópontról indulnak. A `unique order_id` szám egyenlősége a rekordszámmal azt mutatja, hogy a tiszta `orders.csv`-ben nincs duplikált rendelés; ez a későbbi joinok előfeltétele. A késési arány ezzel szemben már **tartalmi** mutató: a p_{late} pontbecslés egyetlen számmal foglalja össze a rendelésszintű késési gyakoriságot. Ennek a pontbecslésnek a bizonytalanságát itt még nem számszerűsítjük, de pontosan kimondjuk, hogy egyetlen szintetikus minta aránya nem azonos a generáló folyamat bizonytalan paraméterével, és végképp nem valós üzleti tény. A resampling-egység itt a rendelés; ha ügyfélszintű állítást tennénk, az ügyfél lenne a mintavételi egység, a tételszintű sor pedig soha nem adhat több mintát a rendelésekhez képest.

1.4.5 Feladatok

1. **Labdiagnózis és napló.** Futtasd a zero-dependency labort, ellenőrizd kézzel a három kontrollértéket, és írj a hipotézisnaplóba öt olyan állítást, amelyet ez az adathalmaz *nem* támaszt alá. Mindegyik mellé egy mondatban indokold, miért nem.
2. **Késési arány implementálása.** Írj olyan Python-programot, amely a `data/orders.csv` bemenetből kiírja a rendelésszámot, az egyedi `order_id`-k számát, a `target_late == "1"` sorok számát és a késési arányt százalékban, két tizedesjeggyel. A fejléc tartalmát ne számold rekordnak. Ellenőrizd, hogy az első két kiírt érték egyaránt 2400.

3. **Hibadiagnózis.** Egy tanuló a következő kódot írta, és a késési darabszámra mindig `0` jött ki:

```
with open("data/orders.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        if row["target_late"] == 1:
            late += 1
```

   Mi a hiba? Javítsd úgy, hogy a szöveges mezőt ne alakítsd át úgy, hogy a hiányzó érték rejtett hibává váljon.

4. **Megváltozott adat.** A `data/orders_raw.csv` szándékosan nem a tiszta állomány: 2405 törzssort tartalmaz, benne öt extra duplikált sorral, két hiányzó `distance_km` értékkel, egy negatív `basket_minor` értékkel és egy nem létező `customer_id`-val. Adj három konkrét auditlépést, amely ezeket elkülöníti, és indokold, miért nem automatikusan törlünk minden gyanús sort.

1.4.6 Megoldások és indoklás

1. **Öt állítás, amelyet az adat nem támaszt alá — lehetséges megoldás.**

   - "Az északi régió ügyfelei elégedetlenebbek, mert késnek a rendeléseik." Az adatban nincs elégedettségi mérés.
   - "A prioritásos rendelés *okozza* a gyorsabb kézbesítést." A megfigyelt együttjárásból önmagában nem következik ok-okozati hatás.
   - "A hiányzó `distance_km` érték nulla kilométert jelent, ezért átlag helyettesíthető." A hiányzó érték nem nulla; az adatszerződés szerint ez hiba, nem nulla távolság.
   - "A becsült késési arány pontosan ugyanaz lesz egy jövőbeli 300 ügyfélből álló mintán." A mintavételi véletlen miatt az arány ingadozhat.
   - "A Northstar-adatok valós ügyfelekről szólnak." A labor szintetikus generátorral készült; valós emberre vagy üzletre vonatkozó állítás nem vezethető le belőle.

2. **Megvalósítás.** A fenti `target_late`-kód a megoldás. Ellenőrző invariantok: a fájl törzssorainak száma 2400; az egyedi `order_id`-k száma szintén 2400; $0 \le N_{\text{late}} \le 2400$, ezért a kiszámolt arány $[0, 1]$-beli. A pontos `late count` a futtatásból jön; a kimenetet a futási jegyzékbe kell rögzíteni, és a hash-ellenőrzéshez hasonlóan itt sincs tolerancia a véletlenül elírt fájlra.

3. **Hibadiagnózis.** A `csv.DictReader` minden mezőt karakterláncként ad vissza, így `row["target_late"]` értéke `"0"` vagy `"1"`, nem egész szám. A `"1" == 1` kifejezés Pythonban `False`, ezért a számláló soha nem nő. Helyes összehasonlítás: `row["target_late"] == "1"`. Az `int(row["target_late"])` is működőképes, de csak akkor, ha előtte ellenőrizzük, hogy a mező pontosan `"0"` vagy `"1"`; különben a hiányzó vagy hibás érték `ValueError`-ként derül ki egy olyan ponton, ahol már nehezebb visszafejteni a forrássort.

4. **Auditlépések a nyers állományra.**

   a) Csoportosíts `order_id` szerint, és listázd azokat, amelyekből egynél több sor van; ezek a duplikátumok auditrekordba kerülnek.

b) Számold meg a hiányzó `distance_km` értékeket, és ellenőrizd a tiszta fájl sormennyiségét is; a hiányt ne pótold nullával csendben.

c) Keresd a negatív `basket_minor` értékeket, és végezz anti-join ellenőrzést a `customers.customer_id` oszlopával az ismeretlen ügyfél megtalálásához.
A sorok automatikus törlése azért hibás, mert a döntés üzleti/előállítási kontextus nélkül megsemmisíti azokat a nyomokat, amelyek a rendszerhiba okát mutatnák.

1.4.7 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés ezen a ponton az, hogy a szintetikus adaton látott arányt valós üzleti bizonyítékként mondjuk fel. Ellenpélda: ha a tiszta `orders.csv`-ben a távoli rendelések körében magasabbnak látszik a késési arány, ebből még nem állítható, hogy a valódi bolt szállítási folyamata a távolsággal romlik. A labor egy mesterséges, paraméterezett szimuláció; a tanulság a munkamenetek szerkezetére vonatkozik, nem a valós vállalat teljesítményére. Ez visszautal az 1.3. alfejezet hipotézisnaplójára is: a “megfigyeltem, hogy…” és a “az adat igazolja, hogy…” közötti lépést mindig külön kell rögzíteni, és a másodikhoz már következtetési szabály kell, nem csak egy CSV.

Önellenőrzés. Mielőtt továbblépnél, tudd megválaszolni: mi a megfigyelési egység a három fájlban külön-külön? Mi a nevező a késési arányban? Mi a különbség a 2 százalékpont és a 2 százalékos növekedés között? Meg tudod-e mondani minden elkészült állításodról, hogy adatból, feltételezésből vagy modellből következik-e? Ha valamelyikre nem, az a diagnózis következő tennivalója.

Diagnosztikai elfogadási kapu. A teljes munkamenet sikerét ne kizárólag a script nulla kilépési kódjához kösd. Külön ellenőrizd a bemeneti sémát, az egyedi rendelésazonosítókat, a hiányzások számát, a célképzés szabályát, a címke rendelkezésre állását és a kimeneti sorok számát. Az elfogadási jegyzőkönyv rögzítse az egyes állításokhoz tartozó megfigyelt értéket.

Szintetikus adaton mért késési arányból csak a generált állományra lehet közvetlenül következtetni. Valós működési ajánláshoz külön szükséges a mintavételi folyamat, az éles adatminőség és az időbeli általánosíthatóság igazolása.

1.4.8 Rövid összefoglaló

A diagnosztikai munkamenet három hibacsaládot választ szét: a programozási hiba a kód és a típusok szintjén jelentkezik; az adatértelmezési hiba a szemcsézettség és a mezők jelentésének félreértéséből ered; a statisztikai hiba pedig akkor, amikor egy pontbecslést bizonytalanság és mintavételi egység nélkül tényként kezelünk. A Northstar labor három kontrollértéke a tiszta adat szerződött méretét ellenőrzi, a késési arány pedig az első valódi alapmutató. A labort a tanuló futása és jegyzéke teszi megismételhetővé; a szintetikus adat tanulási minta, nem üzleti bizonyíték. A következő fejezetben ugyanezt a fegyelmet visszük át az operációs rendszer és a terminál szintjére: a fájlok, útvonalak és hibakimenetek kezelésére.

2 Operációs rendszer, terminál és verziókezelés a napi munkában

2.1 Fájlok, útvonalak és projektgyökér

Belépő · szint: alap · idő: 45 perc · eszköz: bash, Git

Előfeltétel: 1.4. alfejezet · Célkészség: Fájlok, útvonalak és projektgyökér · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

2.1.1 Cél és konkrét helyzet

A Northstar labor mappája több parancsot is futtathatóvá tesz: a `lab/generate_data.py` előállítja a tanulóadatokat, a `lab/quickcheck.py` pedig ellenőrzi a szerződött fájlokat. Az első kellemetlen tapasztalat általában nem az adattal, hanem az útvonallal van: a mellékelt lab szkriptjei már a saját fájlhelyükből (`__file__`) vezetik le a `data/` mappát, ezért bármely munkakönyvtárból ugyanazt az eredményt adják. A fejezet feladata, hogy ezt a szerződést megértsük, egy wrapperrel láthatóvá tegyük, és egy CWD-függő változat hibáját diagnosztizáljuk.

A feladat most az, hogy ugyanazt a labparancsot két különböző munkakönyvtárból is ugyanazzal az eredménnyel futtassuk, majd a véletlenszerű CWD-függést egy rögzített projektgyökér-szerződéssel szüntessük meg. A megfigyelési egység itt egy teljes parancsinvokáció a hozzá tartozó munkakönyvtárral, a keletkező fájlokkal és az ellenőrző hash-szel. A siker feltétele az, hogy a két különböző helyről indított futtatás azonos fájlokat, azonos tartalommal hozzon létre; a hash-egyezésben nincs tolerancia, mert a `quickcheck.py` jellegű ellenőrzés bitpontosan hasonlítja össze a rögzített kontrollértékeket.

Előzetes becslés: ha egy script `data/` stílusú relatív utat használ, és a generátor a CWD-be írja a kimenetét, akkor a gyökérből indított `python lab/generate_data.py` a projekt gyökerébe tenné a fájlokat, a `cd lab && python generate_data.py` pedig a `lab` mappába. Az egyik futás biztosan nem oda írna, ahol a `quickcheck.py` keresi a fájljait. A mellékelt generátor éppen ezt a hibát kerüli el azzal, hogy a `ROOT`-ot a `__file__`-hoz köti; a fejezet hátralévő része ezt a szerződést mutatja meg, és azt, mi történik egy CWD-függő változattal.

2.1.2 Fogalmi magyarázat: aktuális munkakönyvtár

Az **aktuális munkakönyvtár** (*current working directory*, CWD) egy folyamat szintű tulajdonság. Amikor egy terminálban `cd projektek/northstar` parancsot adunk, a héj munkakönyvtára megváltozik, és minden ezután indított alárendelt folyamat, például a Python interpreter, ezt a munkakönyvtárat örökli. Bashban a `pwd`, PowerShellben a `Get-Location`, Pythonban a `Path.cwd()` mutatja. A CWD nem a script forrásfájljának helye, nem a notebook mappája és nem a repository gyökere automatikusan; csak az a könyvtár, ahonnan a folyamat indult.

A relatív útvonal `data/raw/orders_raw.csv` mindig a CWD-hez képest értelmezett. Ezért ugyanaz a karaktersorozat két helyről indítva egészen más fájlra mutat. Linuxon, macOS-en és

WSL-ben a gyökér és a mappák hierarchiája /-rel kötött, Windowson \-rel; a `pathlib` éppen ezért platformfüggetlen `Path("data") / "raw"` felépítést javasol a kézzel írt `data\raw` helyett.

2.1.3 `pathlib`: a script helyéből számított út

A Python standard könyvtárának `pathlib` modulja objektumként kezeli az útvonalakat. A legfontosabbak:

- `Path(__file__)` a futó Python-fájl helye. Fontos: nem a CWD, hanem a script tényleges elérési útja.
- `.resolve()` abszolút úttá alakít és feloldja a szimbolikus linkeket.
- `.parent` a szülő könyvtár, `.parents[i]` az i-edik felmenő; a `.parents[0]` megegyezik a `.parent`-tel.
- `.mkdir(parents=True, exist_ok=True)` létrehozza a hiányzó könyvtárláncot.
- `.write_text(...)` és `.read_text(...)` szövegfájlokhoz; explicit `encoding="utf-8"` ajánlott, mert az alapértelmezett kódolás a locale-tól függhet.
- `.stat().st_size` a fájl bájtban mért mérete.

Ha egy script a `s005_demo/scripts/make_files.py` helyen fekszik, akkor a projektgyökér így rögzíthető:

```
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]  # s005_demo/scripts szülője: s005_demo
```

A módszer érvényességi korlátja, hogy a `__file__` csak normál, forrásfájlként futtatott szkriptben értelmezhető. Jupyter-notebookban nincs `__file__`; interaktív interpreterben, becsomagolt vagy „fagyasztott” futtatható állományban nem feltétlenül valós fájlrendszeri útvonalra mutat. Az alfejezet módszere tehát parancssori scriptekre és modulokra szól, a notebookokra a későbbi 3.4 kernelregisztrációs munkamenet ad teljes megoldást.

2.1.4 `data/raw` és `data/derived`: bemenet és felülírható kimenet

A kutatási projekt akkor marad hordozható, ha az olvasó szeme meg tudja különböztetni az eredeti bemenetet az előállított kimenettől. A szokásos szerződés:

- `data/raw/` – a projektbe érkező, nem módosítandó forrásfájlok. A futtató kód ezt csak olvassa. A Northstar laborban a `lab/data/orders_raw.csv` pont ilyen szerepű: szándékosan hibás nyers másolat, tisztítási bemenet.
- `data/derived/` – a kódból előállított, reprodukálhatóan felülírható kimenet. Itt van a tisztított `orders.csv` és a futási ellenőrzéshez tartozó `manifest.json` szellemi párja.

A kezdőlabor fájlszinten választja szét a nyers és tiszta adatot; nagyobb projektben a mappaszerkezet is ezt a szerződést tükrözi. A lényeg nem a mappa neve, hanem a felülírhatósági szabály: a raw állománynak nem szabad futás melléktermékeként készülnie, a derived állományt pedig bármikor el lehet dobni, mert a generáló utasítás adott. A **kimeneti út** (*output path*) tehát nem lehet a CWD rejtett mellékhatása, hanem a projektgyökérből levezetett, dokumentált útvonal.

Két külön projektváz. A könyvben kétféle könyvtárfa szerepel, és a parancsok munkakönyvtára dönti el, melyik érvényes:

Váz	Gyökér	Adatút	Mikor használjuk
Northstar laborcsomag	`northstar-lab-1.0.0/` (a könyv forrásában `lab/`)	`data/orders.csv`, `data/orders_raw.csv`, `data/manifest.json`	a mellékelt ZIP-ből dolgozva; a `generate_data.py` és a `quickcheck.py` munkakönyvtára
Oktatási miniprojekt	`projekt/`	`data/raw/`, `data/derived/`, `reports/`, `scripts/`	a 2.1. alfejezet–3.4. alfejezet saját kis projektjeiben, ahol a raw/derived szerződést tanítjuk

A laborcsomag szándékosan lapos `data/` mappát használ, mert a kezdőpélda egyetlen adathalmaz; a `data/raw` és `data/derived` bontás a nagyobb, saját projekt mintája. A két utat ne keverd: a `lab/data/raw/orders_raw.csv` nem létezik, a laborban `data/orders_raw.csv`, a miniprojektben `data/raw/orders_raw.csv` a helyes út. A 2.2. alfejezet parancsai ezért a miniprojekt vázát használják; a laborban ugyanaz a script a `lab/data/orders_raw.csv` útvonalat kapja.

2.1.5 UTF-8 és sorvég

Ugyanaz a látható szöveg eltérő bájtokat jelenthet, és a manifest hash bájtokra számol, nem látszatra. Az `árvíz` öt karakter, de UTF-8 kódolásban hét bájt, mert az `á` és az `í` két-két bájtot foglal: `C3 A1`, illetve `C3 AD`. A karakterek és bájtok megkülönböztetése itt konkrét: a Python `len("árvíz")` értéke `5`, a fájlbeli hossz viszont `7` bájt.

A sorvég ugyanilyen láthatatlan veszélyforrás. Linuxon és macOS-en a sorvég `\n` (`0A`), Windowson a hagyományos CRLF `\r\n` (`0D 0A`). A Python `open()` íráskor `newline=None` esetén Windowson CRLF-re fordíthat; a `newline="\n"` argumentum explicit LF-et kényszerít. A `Path.write_text(..., newline="\n")` a Python 3.10-től érhető el, a labor alapjaként használt Python 3.12-ben tehát rendelkezésre áll. A BOM (`EF BB BF`) sem látható, de megváltoztatja a hash-t; a fájlt a `xxd -l 16` kezdősora leplezi le.

A sorvéget verziókezelt projektekben a `.gitattributes` rögzítheti, például:

```
*.py text eol=lf
*.csv text eol=lf
*.sh text eol=lf
*.md text eol=lf
```

Így a CSV-k tartalma platformon át hordozható, és a manifest hash-ek is egyezhetnek, ha a generátor determinisztikus.

2.1.6 Teljes minimális példa

Hozzunk létre egy `s005_demo` nevű miniprojektet a következő fastruktúrával:

```
s005_demo/
  scripts/make_files.py
  data/raw/           # a futás hozza létre
  data/derived/       # a futás hozza létre
```

A `scripts/make_files.py` tartalma:

```python
from pathlib import Path

# A script a s005_demo/scripts/ mappában van, ezért a projektgyökér a szülő.
ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw"
DERIVED = ROOT / "data" / "derived"

RAW.mkdir(parents=True, exist_ok=True)
DERIVED.mkdir(parents=True, exist_ok=True)

source = RAW / "note.txt"
source.write_text("első sor: árvíz €\n", encoding="utf-8", newline="\n")

target = DERIVED / "note_upper.txt"
target.write_text(source.read_text(encoding="utf-8").upper(),
                  encoding="utf-8", newline="\n")

print(f"ROOT={ROOT}")
print(f"source={source.relative_to(ROOT)} size={source.stat().st_size}B")
print(f"target={target.relative_to(ROOT)} size={target.stat().st_size}B")
```

Futtassuk ugyanazt a scriptet két különböző munkakönyvtárból. Linux/macOS bash munkamenetben:

```bash
# Platform: Linux/macOS. Munkakönyvtár először maga a projektgyökér.
cd /home/user/projects/s005_demo
python scripts/make_files.py

# Ezután a szülőből ugyanaz a script:
cd /home/user/projects
python s005_demo/scripts/make_files.py
```

Az elméletileg várt, szemléltető kimenet, ha a felhasználó útja `/home/user/projects`:

```
ROOT=/home/user/projects/s005_demo
source=data/raw/note.txt size=23B
target=data/derived/note_upper.txt size=23B
```

A második futás pontosan ugyanazt a `ROOT` értéket nyomtatja, mert az útvonalat a `__file__` helyéből származtattuk, nem a CWD-ből. A relatív `scripts/make_files.py` hívási útvonal eltérhet, a `.resolve()` akkor is ugyanarra az abszolút fájlra mutat.

2.1.7 Eredmény értelmezése és kézi kontroll

A script három dolgot ellenőrizhetővé tesz. Először, mindkét futás ugyanazt az abszolút `ROOT`-ot írja, tehát a kimeneti fájlok ugyanoda kerülnek. Másodszor, a `source` és a `target` mérete egyaránt `23 B`: az eredeti „első sor: árvíz €” szöveg 23 bájt UTF-8-ban, és a nagybetűs „ELSŐ SOR: ÁRVÍZ €” is 23 bájt, mert bár a karakterek közül négy több-bájtos, a kis- és nagybetűs pár ugyanannyi bájtot foglal. A `€` három bájt (`E2 82 AC`), a `\n` egy bájt (`0A`), a többi ékezetes betű kettő-kettő. Harmadszor, a kiíratás relatív útvonalat használ a `relative_to(ROOT)` hívással: a naplóban `data/raw/note.txt` szerepel, nem a gépspecifikus abszolút út, így a két gépen készült naplók csak akkor térhetnek el, ha valóban eltérő a tartalom.

Bitpontos hash-ellenőrzést is futtathatunk a `s005_demo` gyökérből:

```bash
# Platform: Linux/macOS. Munkakönyvtár: /home/user/projects/s005_demo
python - <<'PY'
from pathlib import Path
import hashlib
```

```
for p in sorted(Path("data").rglob("*.txt")):
    digest = hashlib.sha256(p.read_bytes()).hexdigest()
    print(p, digest)
PY
```

A két futás után a hash azonos; az ellenőrzés itt bitpontos, tolerancia nélküli. Ha a fájlt CRLF-re mentenék, a hash azonnal megváltozna, pedig a képernyőn ugyanaz a szöveg látszana.

Hordozható útvonal-szerződés. A munkakönyvtár (CWD), a script helye és a projekt gyökere eltérő fogalmak. A `Path(__file__)` a modul helyéből indul ki, ezért csak akkor jó a `parents[1]`, ha a dokumentált könyvtárstruktúra valóban fennáll. Telepített csomag esetén az adatfájlok feloldását inkább explicit konfigurációval vagy csomagerőforrással végezzük.

Platformellenőrzés: az útvonalillesztéshez `pathlib`-ot használjunk, ne kézzel fűzzünk `\` vagy `/` jeleket. Tesztelendő a szóközt tartalmazó projektút, az eltérő CWD és a nem létező bemenet: utóbbinál legyen értelmes hibaüzenet, ne néma új fájl létrehozása.

2.1.8 Feladatok

1. feladat (értelmezés). Adott a következő fastruktúra:

```
projekt/
  scripts/run.py
  data/input.csv
```

A `scripts/run.py` mindössze ennyit tesz:

```
from pathlib import Path
print(Path.cwd())
print(Path(__file__).resolve())
print(Path("data/input.csv").resolve())
```

Futtatjuk először `cd projekt && python scripts/run.py`, majd `cd projekt/scripts && python run.py` paranccsal. Mindkét futásnál add meg a három sor várt kimenetét, és magyarázd meg, miért változik a harmadik sor, pedig a script ugyanaz. Melyik futásban létezik a `data/input.csv` fájl?

2. feladat (implementálás). A Northstar repository gyökerébe hozz létre `run_lab.sh` wrapper szkriptet. Adott input a következő:

```
northstar/
  lab/generate_data.py
  lab/quickcheck.py
```

A wrapper a saját fájlhelyéből számítsd ki a projektgyökeret, váltson a `lab` mappába, majd futtassa a `generate_data.py` és `quickcheck.py` parancsokat. Futtasd a wrapper szkriptet a gyökérből, a `lab` mappából és egy teljesen más munkakönyvtárból is. Mi a siker feltétele, és miért nem elég a `python generate_data.py` közvetlen hívása a gyökérből?

3. feladat (hibadiagnózis). A `/home/user/northstar/notebooks/check.ipynb` notebook első cellája:

```
from pathlib import Path
raw = Path("lab/data/orders_raw.csv")
print(raw.exists(), raw.resolve())
```

A Jupyter Labot a `/home/user` könyvtárból indítottuk, majd a notebookot a fájlkezelőben a `northstar/notebooks` mappából nyitottuk meg. A cella `False` értéket és `/home/user/lab/data/orders_raw.csv` útvonalat ír. Miért? Adj két javítást, amelyek egyike a Jupyter indítási szerződését rögzíti, a másika védő asserttel leáll, ha a feltétel nem teljesül! Miért **nem** jó megoldás egy `os.chdir("/home/user/northstar")` cellát beszúrni a notebook elejére?

4. feladat (transzfer, PowerShell). Írd át a `run_lab.sh`-t `run_lab.ps1` formába Windows PowerShellre. A PowerShell script a `$PSScriptRoot` változót használja, ne az aktuális könyvtárat! A generált CSV fájlok sorvége maradjon LF, függetlenül attól, hogy a `.ps1` fájlt CRLF-fel szerkesztettük. Mit ellenőriz a manifest hash? Mi történik, ha egy Windows-szerkesztő a generált `orders.csv` fájlt CRLF-re normalizálja?

2.1.9 Megoldások és indoklás

1. feladat megoldása. Az első futásnál a CWD `projekt`, a `__file__` `projekt/scripts/run.py`, a relatív `data/input.csv` pedig `projekt/data/input.csv`. Várt kimenet:

```
/home/user/projektek/projekt
/home/user/projektek/projekt/scripts/run.py
/home/user/projektek/projekt/data/input.csv
```

A második futásnál a CWD `projekt/scripts`, a `__file__` ugyanaz a script, de a relatív út a CWD-hez képest oldódik fel, tehát `projekt/scripts/data/input.csv`. A második futásban a fájl általában nem létezik; a relatív útvonal a munkakönyvtártól függ, nem a script forrásfájljának helyétől. Pontosan ezért köti a projektgyökér-szkript a `ROOT`-ot a `__file__`-hoz.

2. feladat megoldása. A `run_lab.sh` tartalma:

```bash
#!/usr/bin/env bash
set -euo pipefail

# A szkript saját könyvtárából számítjuk a projektgyökeret:
# a CWD nem befolyásolhatja a generálás helyét.
THIS_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
LAB_DIR="$THIS_DIR/lab"

cd "$LAB_DIR"
python generate_data.py
python quickcheck.py
```

Futtatás három munkakönyvtárból:

```
# Linux/macOS bash. Először a repository gyökere:
cd /home/user/northstar
./run_lab.sh

# Ugyanaz a szkript a lab mappából:
cd lab
../run_lab.sh

# Teljesen más munkakönyvtárból:
cd /tmp
/home/user/northstar/run_lab.sh
```

A siker feltétele az, hogy a `lab/generate_data.py` által írt fájlok mindhárom esetben a `lab` mappába kerüljenek, és a `lab/quickcheck.py` ellenőrző hash-e egyezzen a `lab/data/manifest.json`

rögzített kontrollértékével. A közvetlen `python generate_data.py` hívás azért nem elég a gyökérből, mert a gyökérben nincs ilyen fájl: `lab/generate_data.py` lenne a hívandó, ám ha a generátor a CWD-t használja a kimenethez, akkor a fájlok a gyökérbe kerülnének, ahol a `quickcheck.py` nem találja őket a szerződött helyen. A wrapper mindig a `lab` mappába `cd`-zik, tehát a generátor CWD-je determinisztikus.

A `set -euo pipefail` azt jelenti: hiba esetén a script azonnal leáll, nem hagyja figyelmen kívül a hibás parancsot. Az `unset` változó (`-u`) is hibának számít; ez megakadályozza az elgépelt útvonalak csendes továbbvitelét.

3. feladat megoldása. A Jupyter kernel CWD-je az indítási könyvtár, nem a notebook mappája. A notebookot a böngészőben a `northstar/notebooks` mappából nyitottuk meg, de a kernel `/home/user`-ből indult, ezért a `Path("lab/data/orders_raw.csv")` a `/home/user/lab/data/orders_raw.csv` helyre mutat, ami nem létezik. A notebook helye nem változtatja meg a kernel CWD-jét; a kettő eltérhet, és ez a fejezet gyakori tévedése.

Első javítás: rögzítsd az indítási szerződést, és indítsd a Jupyter Labot a projektgyökérből:

```
# Linux/macOS bash, a northstar gyökérből:
cd /home/user/northstar
jupyter lab
```

Ekkor a kernel CWD-je a projektgyökér, és a cellában a relatív `lab/data/orders_raw.csv` elérési út helyesen oldódik fel.

Második javítás: védő assert a notebook első cellájában:

```
from pathlib import Path

PROJECT_ROOT = Path.cwd()
REQUIRED = PROJECT_ROOT / "lab" / "data" / "orders_raw.csv"
assert REQUIRED.exists(), f"A kernel CWD nem a Northstar gyökér: {PROJECT_ROOT}"
RAW = REQUIRED
```

Ez még mindig feltételezi, hogy a Jupytert a projektgyökérből indították, de a feltétel nem hallgatólagos többé: ha megszegik, a notebook azonnal, értelmes üzenettel áll le. Az `os.chdir("/home/user/northstar")` rossz megoldás, mert a cellába gépspecifikus abszolút utat éget; más gépen megtörik, és a cella újrafuttatása állapotot módosít. A teljes, hordozható kernelindító megoldást a 3.4 alfejezet építi fel.

4. feladat megoldása. A `run_lab.ps1` tartalma:

```
# Futtatás: powershell -ExecutionPolicy Bypass -File .\run_lab.ps1
$ErrorActionPreference = "Stop"

$Root = $PSScriptRoot
$Lab = Join-Path $Root "lab"
Set-Location $Lab

python generate_data.py
if ($LASTEXITCODE -ne 0) {
    throw "generate_data.py failed with exit code $LASTEXITCODE"
}

python quickcheck.py
if ($LASTEXITCODE -ne 0) {
    throw "quickcheck.py failed with exit code $LASTEXITCODE"
}
```

A `$PSScriptRoot` a script mappájának abszolút útja, így a hívó PowerShell munkakönyvtára nem befolyásolja a futást. A `$LASTEXITCODE` a külső `python` program visszatérési értéke; PowerShellben a Python hibáját nem dobja automatikusan a shell, ezért kell explicit ellenőrzés.

A generált CSV fájlok sorvége attól függetlenül LF marad, hogy a `.ps1` forrást CRLF-fel szerkesztettük, mert a CSV-t a Python `generate_data.py` írja. Ha a generátor a szöveges íráskor explicit `newline="\n"` beállítást használ, akkor a fájlbeli bájtok közömbösek a Windows szerkesztő beállításaira. A manifest hash a CSV fájlok tényleges bájtjaira számol; ha egy Windows-szerkesztő egy meglévő `orders.csv`-t CRLF-re normalizál, a hash megváltozik, és a `quickcheck.py` eltérést jelez, jóllehet a látható szöveg ugyanaz. Ilyenkor a fájlt újra kell generálni, nem a hash-t módosítani; a `.gitattributes eol=lf` szabálya pedig megelőzi, hogy a verziókezelés maga normalizálja eltérően a fájlt.

2.1.10 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés a notebook helyének és a kernel munkakönyvtárának összekeverése. Képzeljük el, hogy a `notebooks/check.ipynb` mellé tesszük a `data/raw` mappát, és a notebookban `Path("data/raw/orders_raw.csv")`-t írunk. Ha a Jupyter Labot a `notebooks` mappából indítjuk, a kernel CWD-je a `notebooks`, tehát a fájl megtalálható. Ha viszont a Jupyter Labot a home könyvtárból vagy a projekt gyökeréből indítjuk, a kernel ugyanazt a notebookot futtatja, de a relatív út más helyre mutat; a notebook mappájának közelsége nem érdekes a kernel számára.

Az 1.1. alfejezetben láttuk, hogy a nevező hallgatólagos megválasztása csendesen tönkreteszi az arányszámítást. Itt a CWD hallgatólagos megválasztása teszi ugyanezt a fájlműveletekkel: a script fut, a napló készül, csak éppen nem ott és nem arról a fájlról, amelyet a szerződés állít. A diagnózis mindkét esetben ugyanaz: a láthatatlan feltételt explicitté kell tenni a futás elején.

2.1.11 Önellenőrzés

Tedd fel magadnak a következő kérdéseket, mielőtt egy scriptet megosztasz:

1. Ki tudom mondani a futtatás pillanatában érvényes CWD-t, például `pwd` vagy `Path.cwd()` paranccsal?
2. A projektgyökér a `__file__`-ból származik, vagy a CWD-től függ?
3. A raw állomány olvasható-e utasítás nélkül, a derived felülírható-e reprodukálható módon?
4. A fájlírás explicit UTF-8 és LF beállítást használ, és a hash-t bájtokra ellenőrzöm?
5. Ugyanaz a parancs két munkakönyvtárból is ugyanazt a manifest hash-t adja?
6. Jupyter-notebook esetén explicit szerződés vagy assert rögzíti a kernel CWD-jét?

Ha valamelyik nemre változik, előbb a fájlkezelési, nem az adatmodell hibájában keressük a bajt.

2.1.12 Rövid összefoglaló

A hordozható projekt a munkakönyvtártól független projektgyökérből dolgozik. A `Path(__file__).resolve()` a script tényleges helyéből vezeti le a gyökeret, így ugyanaz a parancs két helyről indítva is ugyanoda ír. A `data/raw` bemenet és a `data/derived` kimenet elkülönítése védi az eredeti fájlokat; az explicit UTF-8 és LF beállítás pedig a manifest hash platformfüggetlenségét biztosítja. Jupyterben a kernel CWD-je nem a notebook mappája, ezért azt vagy indítási szerződéssel, vagy védő asserttel kell rögzíteni. A következő alfejezetben

a futó folyamatokat vizsgáljuk: hibakimenetet, visszatérési kódot, memóriahasználatot és a háttérben futó Python-folyamatok azonosítását.

2.2 Folyamat, memória és hibakimenet

Belépő · szint: alap · idő: 45 perc · eszköz: bash, Git

Előfeltétel: 2.1. alfejezet · Célkészség: Folyamat, memória és hibakimenet · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

2.2.1 Cél és konkrét helyzet

A diagnosztika tárgya a Northstar `data/raw/orders_raw.csv` fájl: 2405 rendeléssorból áll, és az 2.1 szerint elkülönített nyers mappa tudatosan hibás példányát adja. Egy sor tizenöt mezőt tartalmaz; a szerződés szerint öt extra duplikált sor, két hiányzó `distance_km` és egy negatív `basket_minor` van benne. A megfigyelési egység egy CSV-sor, a folyamat futása szempontjából pedig egyetlen Python-processz.

A tanuló dolga, hogy egy kis parancssori ellenőrzőt futtasson, a kimenetet szétválassza, rögzítse a visszatérési kódot, majd memóriabecslést készítsen. Előzetes becslés: ha egy sor átlagosan 350 byte, akkor a teljes szöveges fájl

$$2405 \cdot 350 \text{ B} \approx 842 \text{ kB} \approx 0{,}80 \text{ MiB}.$$

Egy listában tárolt verzió a Python string- és mutatóoverheadje miatt 1–1,5 MiB körül lehet. Ellenőrzésre a futás végén a hibák számának és az exit code-nak kell egyeznie: 8 észlelt sorhiba, 4-es saját hibakód. A pontos byte-méret gépfüggő, ezért csak nagyságrend és invariáns a cél.

2.2.2 A három alapfogalom: stdout, stderr és exit code

A futó program két alapértelmezett kimeneti csatornát kap: a standard kimenetet (`stdout`, standard output) és a standard hibakimenetet (`stderr`, standard error). A Python `print()` alapesetben a `stdout`-ra ír, a `sys.stderr`-re irányított írás pedig a `stderr`-re. A szétválasztás nem kozmetika: a tudományos futásnaplóban az adatszerű eredményt és a diagnosztikai zajt külön akarjuk tárolni. A shellben a `>` a `stdout`-ot irányítja fájlba, a `2>` a `stderr`-t. PowerShellben ugyanez a két operátor működik az újabb verziókban, de az exit code ott a `$LASTEXITCODE` változóban jelenik meg, nem a `$?` boolean értékben.

Minden folyamat egy számmal tér vissza: ez az exit code, más néven visszatérési kód (exit status). A 0 hagyományosan sikert jelent, a nullától eltérő hibaosztályt. A konkrét szám a program saját szerződése. Bashban a `$?` az utolsó előtérben futott parancs kódját adja; ezért az értéket azonnal el kell menteni, mielőtt bármely másik parancs felülírná. Kézi kontroll Linux/macOS Bashban vagy WSL-ben, a projektgyökérből:

```
python -c 'import sys; print("adat"); print("hiba", file=sys.stderr)' \
  >reports/s006/out.txt 2>reports/s006/err.txt
status=$?
printf 'exit_code=%s\n' "$status"
```

Itt az `out.txt` egyetlen `adat` sort tartalmaz, az `err.txt` a `hiba` sort, az `exit_code=0` pedig a sikerjelzést. A Python-kód semmilyen hibát nem váltott ki; a `stderr`-re írás önmagában nem rontja el a folyamat státuszát.

2.2.3 Python-kivétel, OOM és külső hibakód

Három különböző jelenséget könnyű összekeverni, mert mind „hibának” látszik:

Jelenség	Ki állítja le	Látható jel	Kilépőkód
Python-kivétel	a Python interpreter saját kivételkezelője	`Traceback ...` a `stderr`-en	általában 1
OOM (out of memory)	az operációs rendszer memóriakezelője	gyakran nincs traceback, Linuxon `Killed`	Linuxon gyakran 137
Külső folyamat hibakódja	a folyamat saját `return`/`exit` utasítása	a shell `$?` változója	programspecifikus

A kezeletlen Python-kivétel (exception) nyelvi hibacsatorna: a `1/0` kifejezés tracebacket ír, a processz kontrolláltan leáll, és a shell `$?` értéke 1 lesz. A `MemoryError` is kivétel: a Python megpróbált egy számára túl nagy objektumot lefoglalni, és erről a saját nyelvi mechanizmusán keresztül számolt be. Ez még nem jelenti, hogy az egész gép memóriája elfogyott.

Az OOM ezzel szemben operációs rendszer szintű esemény: ha a processz a rendelkezésre álló RAM-nál többet kér, a kernel megölheti anélkül, hogy a Python tracebacket írna. Linuxon a 137-es kilépőkód a 128 + 9 összegből adódik, ahol a 9 a `SIGKILL` jel sorszáma; ez gyakori OOM-gyanú, de önmagában nem elégséges bizonyíték. macOS-en más memórianyomás-kezelés működik, Windowson pedig a jelenség más üzenettel jelentkezhet.

2.2.3.1 Platformonkénti futtatás és a 137-es kód döntési táblája

Ugyanaz a memóriamérő szkript három környezetben más parancsot és más mértékegységet használ. A szkript egy millió rövid stringet tart a memóriában, majd kiírja a Python-verziót és a platformot. A **Bash-eredmény ezen a Linux-gépen mért érték**; a PowerShell- és WSL-kimenet ebben a szerkesztési környezetben nem futott, ezért azok **szemléltető formátumok**, nem mért eredmények.

```python
# mem_probe.py
import sys
print("python", sys.version.split()[0], "platform", sys.platform)
rows = [str(i) * 8 for i in range(1_000_000)]
print("rows", len(rows), "first", rows[0], "last_len", len(rows[-1]))
```

Bash (Linux, mért):

```
$ /usr/bin/time -v python3 mem_probe.py
python 3.12.3 platform linux
rows 1000000 first 00000000 last_len 48
        Maximum resident set size (kbytes): 111120
        Exit status: 0
```

PowerShell (Windows, szemléltető formátum; ezen a gépen nem futtatva):

```
PS> python mem_probe.py
python 3.12.3 platform win32
rows 1000000 first 00000000 last_len 48
PS> $LASTEXITCODE
0
```

```
PS> (Get-Process python).WorkingSet64 / 1MB
108.5
```

WSL (Windows alatti Linux, szemléltető formátum; ezen a gépen nem futtatva):

```
PS> wsl -e bash -lc "python3 mem_probe.py; echo exit=\$?"
python 3.12.3 platform linux
rows 1000000 first 00000000 last_len 48
exit=0
```

A mértékegységek platformonként eltérnek, és a különbség önmagában is hibát okozhat:

Platform	Eszköz	Mértékegység
Linux	`ps -o rss`, `/usr/bin/time -v`	KiB (1024 bájt)
Linux	Python `resource.getrusage(...).ru_maxrss`	KiB
macOS	`ps -o rss`	KiB
macOS	Python `resource.getrusage(...).ru_maxrss`	**bájt**
Windows	`Get-Process ... WorkingSet64`	bájt
Windows	`Get-CimInstance Win32_Process ... WorkingSetSize`	bájt

A 137-es kódot ezért nem szabad önmagában OOM-nak nevezni. A döntési tábla:

Megfigyelés	137 lehetséges?	OOM-e?	Következő ellenőrzés
Linux kernel OOM kill	igen	valószínű	`dmesg/journalctl -k`: `Out of memory: Killed process`
cgroup/container memórialimit	igen	igen (konténer-OOM)	`memory.events` `oom_kill` számláló
kézi `kill -9 <pid>`	igen	nem	shell history, `dmesg` OOM-sor nélkül
`timeout -s KILL 60 parancs`	igen	nem	a `timeout` szándékosan SIGKILL-t küld
a program `sys.exit(137)`-et hív	igen	nem	a Python `returncode` értéke **+137**, nem `-9`
Windows process exit	nem ez a konvenció	más kód	`0xC0000005` stb., eseménynapló

A legfontosabb megkülönböztetés Pythonból: a `subprocess` a **szignál általi** halált negatív kóddal adja vissza (`-9`), a program által **explicit módon** visszaadott 137-et pozitív 137-ként. A shell `$?` viszont mindkettőt 137-ként mutatja, mert a shell a 128 + szignál összeget használja. Ezért naplózzuk a `subprocess` nyers `returncode` értékét, ne csak a shell összegét:

```
def classify_return(returncode):
    if returncode == 0:
        return "success"
    if returncode < 0:
        return f"signal {-returncode}" + (" (SIGKILL: OOM vagy kézi kill)" if returncode == -9
        ↪  else "")
    if returncode == 137:
        return "explicit 137 exit vagy shell-összegzés; nem bizonyíték OOM-ra"
    return f"exit code {returncode}"
```

A `tests/test_exit_codes.py` a `+137` és `-9` megkülönböztetését, a nulla kódot és a `classify_return` szövegét teszteli. A memória- és OOM-ítélet soha ne a puszta kódon, hanem a kód, a signal-nyom, a kernel- vagy cgroup-napló és a mérés együttesén alapuljon.

A külső folyamat hibakódja sem Python-kivétel. Ha egy adateszköz, például egy parancssori program 4-gyel tér vissza, az az adott program szerződése szerinti hiba. Pythonból ezt így lehet lekérdezni, szemléltető jelleggel:

```
import subprocess
result = subprocess.run(["python", "-c", "import sys; sys.exit(7)"])
print(result.returncode)  # elméletileg várt: 7
```

A `subprocess.run` alapból nem dob Python-kivételt nemnulla visszatéréskor; a `check=True` kapcsolóval kérhetnénk kivételt. A kettő szétválasztása azért fontos, mert a hibanaplóban más javító lépés következik: kivételnél a kódot, OOM-nál a memóriaterhelést, külső hibakódnál az adott program dokumentációját kell vizsgálni.

2.2.4 Process, thread, RAM és megszakítás

A processz (folyamat) saját memóriaterületet és erőforrásokat kap; a thread (szál) ugyanazon processz memóriáján osztozó, könnyebb ütemezési egység. Egy Jupyter-kernel egy hosszú életű Python-processz; ha két notebook fut, legalább két külön kernelprocessz dolgozik. Linuxon a `ps -o pid,ppid,rss,etime,comm`, macOS-en a `ps -axo ...`, Windowson a `Get-Process python` segít azonosítani, melyik folyamat mennyi memóriát foglal. A `ps` eszköz `rss` mezőjének szokásos mértékegysége KiB (1024 bájtos egység), nem bájt; a pontos formátumot az adott platform dokumentációja szerint kell értelmezni. Az RSS a rezidens memórialapok nagyságát közelíti, és megosztott lapokat is tartalmazhat: nem azonos a folyamat kizárólagos fizikai RAM-terhelésével, és nem a lemezfoglalást méri.

A RAM és a lemez közötti különbség a hibadiagnózis alapja. A fájl mérete `path.stat().st_size` értékkel kérdezhető le; ez a lemezen elfoglalt tartós méret. A processz RAM-használata futás közben ennél lehet kisebb, ha streamelve dolgozunk, vagy nagyobb, ha a sorokat Python-listában, string-objectumokként tároljuk. A durva becslés:

$$B_{\text{lista}} \approx B_{\text{fájl}} + N_{\text{sor}} \cdot C_{\text{sor}},$$

ahol $B_{\text{fájl}}$ a CSV byte-mérete, N_{sor} a sorok száma, C_{sor} pedig egy sor listaelemének becsült overheadje. 64 bites CPython 3.12 környékén C_{sor} nagyságrendileg 40–60 byte, mert egy listaelem 8 byte-os mutatót és egy legalább 49 byte-os string-objektumot jelent; ez becslés, nem mérés. A bináris átváltás:

$$1\ \text{MiB} = 2^{20}\ \text{B} = 1\,048\,576\ \text{B}.$$

A megszakítás (interrupt) a harmadik hibacsatorna. Az előtérben futó folyamatot `Ctrl+C` szakítja meg; Linux/macOS-en ez `SIGINT` jelet küld, a Python pedig `KeyboardInterrupt` kivétellé alakítja. A `SIGKILL` viszont nem kivétel: a processz azonnal megszűnik, `finally` blokk sem fut. A Jupyterben a kernel Interrupt gombja hasonló szerepet tölt be, de a kernel folyamatának tényleges állapotát hosszú feladat után külön ellenőrizni kell. A háttérben indított folyamat PID-jét a shell `$!` változója adja; Linuxon a `kill PID`, Windowson a `Stop-Process -Id PID` állítja le.

2.2.5 Teljes minimális munkapélda

Munkakönyvtár a projektgyökér; platform Linux/macOS Bash vagy WSL. A gyakorlóprojekt két adatmappát használ: a `data/raw/` a módosítatlan bemeneté, a `data/derived/` az előállított, felülírható kimeneté. Első lépésként másold a mellékelt laborból a `lab/data/orders_raw.csv` és `lab/data/customers.csv` fájlt a `data/raw/` mappába, a tiszta kontrollt pedig `lab/data/orders.csv` néven a `data/derived/` mappába. Ezután mentsük el az alábbi ellenőrzőt `scripts/s006_diagnose.py` néven:

```
from __future__ import annotations

import csv
import sys
from pathlib import Path

EXPECTED_HEADER = [
    "order_id", "customer_id", "order_ts", "promised_days", "quantity_sum",
    "item_rows", "basket_minor", "shipping_fee_minor", "currency", "distance_km",
    "priority_flag", "delivery_days", "actual_delivery_at", "label_available_at",
    "target_late",
]

MIB = 1024 * 1024


def main(argv: list[str]) -> int:
    if len(argv) != 2:
        print(f"Használat: {Path(sys.argv[0]).name} BEMENET.csv", file=sys.stderr)
        return 2

    path = Path(argv[1])
    if not path.is_file():
        print(f"Nincs ilyen fájl: {path}", file=sys.stderr)
        return 1

    total_bytes = path.stat().st_size
    rows = 0
    issues: list[str] = []
    seen: set[str] = set()

    with path.open("r", encoding="utf-8", newline="") as handle:
        reader = csv.reader(handle)
        header = next(reader, None)
        if header != EXPECTED_HEADER:
            print(f"Fejléc eltér: {header}", file=sys.stderr)
            return 3

        for row in reader:
            rows += 1
            if len(row) != len(EXPECTED_HEADER):
                issues.append(f"sor {reader.line_num}: rossz mezőszám: {len(row)}")
                continue

            order_id = row[0]
            if order_id in seen:
                issues.append(f"sor {reader.line_num}: duplikált order_id {order_id}")
            seen.add(order_id)

            if row[9] == "":
                issues.append(f"sor {reader.line_num}: hiányzó distance_km")

            try:
```

```
                if int(row[6]) < 0:
                    issues.append(f"sor {reader.line_num}: negatív basket_minor {row[6]}")
            except ValueError:
                issues.append(f"sor {reader.line_num}: nem numerikus basket_minor {row[6]}")

    print(f"fajl={path}")
    print(f"rows_read={rows}")
    print(f"issue_count={len(issues)}")
    print(f"total_bytes={total_bytes}")
    print(f"average_row_bytes={total_bytes / rows:.1f}")
    estimated_full_load = total_bytes + rows * 60
    print(f"estimated_full_load_mib={estimated_full_load / MIB:.2f}")

    for issue in issues:
        print(issue, file=sys.stderr)

    return 4 if issues else 0


if __name__ == "__main__":
    raise SystemExit(main(sys.argv))
```

A script a hibás bemeneten nemcsak leáll, hanem a hibasorokat a `stderr`-re írja, az összegzést a `stdout`-ra, a nullától eltérő kóddal pedig a shellben automatizálhatóan jelez. A következő futtatás a `reports/s006` mappába gyűjti a futásnaplót:

```
mkdir -p reports/s006

python scripts/s006_diagnose.py data/raw/orders_raw.csv \
  >reports/s006/bad.out 2>reports/s006/bad.err
status=$?
printf 'exit_code=%s\n' "$status" >reports/s006/bad.log

cat reports/s006/bad.log
cat reports/s006/bad.out
cat reports/s006/bad.err
```

Szemléltető várható kimenet — a byte-számok a saját gépen futó `generate_data.py` eredményétől függenek:

```
exit_code=4

fajl=data/raw/orders_raw.csv
rows_read=2405
issue_count=8
total_bytes=882419
average_row_bytes=366.8
estimated_full_load_mib=0.98

sor 118: duplikált order_id ORD-...
sor 487: hiányzó distance_km
sor 902: hiányzó distance_km
sor 1203: negatív basket_minor -1350
sor 1501: duplikált order_id ORD-...
sor 1704: duplikált order_id ORD-...
sor 1988: duplikált order_id ORD-...
sor 2210: duplikált order_id ORD-...
```

A tiszta `data/derived/orders.csv` kontrollként futtatva:

```
python scripts/s006_diagnose.py data/derived/orders.csv \
  >reports/s006/good.out 2>reports/s006/good.err
```

```
status=$?
printf 'exit_code=%s\n' "$status"
```

Várható: `exit_code=0`, `rows_read=2400`, `issue_count=0`, üres `good.err`. A PowerShell-munkamenetben ugyanez a szerkezet `2>` átirányítással és `$LASTEXITCODE` rögzítésével zajlik.

2.2.6 Eredmény értelmezése

A kézi kontroll a nyers fájl szerződéséből következik: 2405 sor = 2400 alapsor + 5 duplikált sor; a hiányzó `distance_km` két sora és a negatív `basket_minor` egy sora külön sorokon van, így összesen 8 észlelt sorhiba. A `rows_read=2405` és `issue_count=8` páros egzakt invariáns; ha bármelyik eltér, a fájl vagy a script változott. Az `exit_code=4` a script saját szerződése: 0 tiszta, 1 hiányzó fájl, 2 rossz argumentum, 3 fejléc-eltérés, 4 sorhibák.

A memóriasor becslés, nem mérés. Az `estimated_full_load_mib` azt mondja meg, mennyit foglalna a sorok szövegének és a hozzá tartozó Python stringeknek a listában tartása; a futó script streamelve olvas, ezért valós RSS-használata ennél kisebb. Pontos processzmemóriát mérni csak saját gépen, megadott Python-verzióval, `ps`-sel vagy platformes eszközzel lehet; az itt közölt kimenet szemléltető.

Folyamat és mérés. A `stdout` a strukturált eredmény vagy emberi kimenet csatornája, a `stderr` a diagnosztikai üzeneteké; egyes eszközök eltérő konvenciót követnek, ezért a naplózási szerződés fontosabb, mint a puszta csatorna. A kilépési kódot a vizsgált parancs után közvetlenül rögzítsük, különben a következő `cat` vagy `echo` már felülírja a shell `$?` értékét.

CPU-kihasználtságból önmagában nem következik, hogy a Python-kód lassú vagy gyors: a méréshez a folyamat futásideje, I/O-tevékenysége, memóriája és a rendelkezésre álló magok száma is kell. A nagyobb CSV vizsgálatakor az adatméret és az olvasási mód legyen a mérési napló része.

2.2.7 Feladatok

1. **Értelmezés.** A hibás futtatás után a `bad.log` tartalma `exit_code=4`. Melyik állítás igaz?

   a) A Python 4 kivételt dobott, ezért a tracebacket kell keresni a `bad.err`-ben.
   b) A script 8 sorhibát észlelt, ezeket a `stderr`-re írta, és a saját hibakódja 4.
   c) A 4-es kilépőkód OOM-ot jelent, ezért a folyamatot a kernel ölte meg. Válaszodat a megfigyelhető kimenetekre hivatkozva indokold.

2. **Implementálás.** Bővítsd a `s006_diagnose.py` scriptet egy harmadik, opcionális paranccsal: ha a felhasználó megadja a `data/raw/customers.csv` fájlt, a script töltse be az ismert `customer_id` értékek halmazát, és minden olyan rendeléssort jelentsen hibaként, amelynek `customer_id`-je nem szerepel benne. A customers fájl első oszlopa a `customer_id`.

3. **Hibadiagnózis.** Adott a következő Bash-részlet:

   ```
   python scripts/s006_diagnose.py data/raw/orders_raw.csv
   cat reports/s006/bad.out
   echo "status=$?"
   ```

   Miért lehet félrevezető az így kiírt `status` akkor is, ha a Python-futtatás 4-gyel tért vissza? Add meg a javított változatot.

4. **Transzfer megváltozott feltételre.** Készíts 130 000 soros szintetikus fájlt `data/derived/orders_130k.csv` néven úgy, hogy a `data/derived/orders.csv` fejlécét és egyetlen érvényes adatsorát ismétled. Futtasd rajta a scriptet, és értelmezd a memóriabecslést. Melyik értéknek kell egzaktnak lennie, és melyiknél fogadható el ±20%-os eltérés?

2.2.8 Megoldások és indoklás

1. A helyes válasz **b**. A 4-es kód a script alkalmazásszintű hibaszerződése, nem Python-kivétel és nem OOM. A `bad.err` soralapú diagnózisokat tartalmaz, nem tracebacket; a `bad.out` `issue_count=8` értéke pedig megerősíti, hogy a 4-es a nyolc sorhibához kötött saját konvenció.

2. A módosítás lényege:

```
if len(argv) == 3:
    customer_path = Path(argv[2])
    raw_lines = customer_path.read_text(encoding="utf-8").splitlines()[1:]
    known_customers = {line.split(",")[0].strip() for line in raw_lines}
else:
    known_customers = None
```

A sorciklusban, a mezőszám-ellenőrzés után:

```
        if known_customers is not None and row[1] not in known_customers:
            issues.append(
                f"sor {reader.line_num}: ismeretlen customer_id {row[1]}"
            )
```

Futtatás:

```
python scripts/s006_diagnose.py data/raw/orders_raw.csv data/raw/customers.csv \
  >reports/s006/extended.out 2>reports/s006/extended.err
status=$?
printf 'exit_code=%s\n' "$status"
```

Az ellenőrző invariáns: `rows_read=2405`, `issue_count=9`, `exit_code=4`. A nyers fájlban egyetlen ismeretlen `customer_id` van, ezért pontosan eggyel több hiba keletkezik.

3. A részlet azért félrevezető, mert `$?` mindig az utoljára végrehajtott parancs kódját adja; az `echo` előtt a `cat reports/s006/bad.out` fut le, amely sikeres fájlmegjelenítéskor 0-t ad. Így a kiírt `status=0` a `cat` eredményét mutatja, nem a Python folyamatét. A javított változat a Python-kódot közvetlenül követően menti az értéket:

```
python scripts/s006_diagnose.py data/raw/orders_raw.csv
status=$?
cat reports/s006/bad.out
echo "status=$status"
```

4. A szintetikus fájl elkészítése Bashban:

```
mkdir -p data/derived

python - <<'PY'
from pathlib import Path

clean = Path("data/derived/orders.csv")
lines = clean.read_text(encoding="utf-8").splitlines()
rows = [lines[0]] + [lines[1]] * 129_999
```

```
Path("data/derived/orders_130k.csv").write_text(
    "\n".join(rows) + "\n", encoding="utf-8"
)
PY

wc -l data/derived/orders_130k.csv
python scripts/s006_diagnose.py data/derived/orders_130k.csv \
  >reports/s006/big.out 2>reports/s006/big.err
status=$?
printf 'exit_code=%s\n' "$status"
cat reports/s006/big.out
```

A `wc -l` eredményének 130 000-nek, a script `rows_read` értékének 129 999-nek kell lennie, és `issue_count=0`, `exit_code=0` várható, mert a minta sor érvényes. A becsült teljes tárolás nagyságrendileg a 2405 soros fájl ötvenszerese: ha az eredeti becslés 1 MiB körül volt, itt 50–55 MiB várható. Ez a becslés ±20%-os toleranciával értelmezhető, mert a fájl mérete és a string-overhead közelítése nem platformfüggetlen. Az `exit_code=0` és az `issue_count=0` viszont egzakt invariáns.

2.2.9 Gyakori tévedés ellenpéldával

Egy futtatás közben a felhasználó azt látja, hogy a Python-processz CPU-kihasználtsága 3%. Ebből gyakran „a CPU lassú, GPU-ra van szükség” következtetést vonnak le. Ez hibás: az alacsony CPU-használat ugyanúgy lehet lemezolvasási várakozás, hálózati késés vagy adatbázis-tranzakciós blokk. A fenti script 3%-os processzornál is végigfuthat, mert az ideje nagy részét a fájlrendszer olvasására várja; egy GPU semmivel sem olvasná gyorsabban a spinning diskről érkező bájtokat. A GPU akkor jön szóba, ha a szűk keresztmetszet numerikus, párhuzamosítható művelet, és ezt mérés bizonyítja. Ez a döntés a későbbi profilozási fejezet témája, nem a pillanatnyi CPU-százalék. A 1.2 adatéletciklus-fegyelme itt is érvényes: a `distance_km` hiánya a rendelés időpontjában ismert mező adatminőségi hibája, nem későn érkező label, ezért javítása nem sérti a predikciós sorrendet. Az 2.1 projektkonvenciót követtük: a nyers adatot nem módosítottuk, a futásnaplók a `reports/s006` származtatott mappába kerültek. Önellenőrzés: igaz-e, hogy a `stderr`-re írás mindig nemnulla exit code-ot okoz? Nem: a `print(file=sys.stderr)` önmagában nem állítja a hibakódot. Igaz-e, hogy a `137` minden platformon OOM-ot jelent? Nem: Linuxon SIGKILL-gyanú, más rendszeren más konvenció él.

2.2.10 Rövid összefoglaló

A futó adatfolyamatnak három hibacsatornája van: a `stdout` az adatszerű kimeneté, a `stderr` a diagnosztikáé, az exit code pedig a processz automatikusan ellenőrizhető összegzéséé. A Python-kivétel nyelvi traceback, az OOM operációs rendszer szintű megállítás, a külső hibakód a program saját szerződése. A memóriabecslés a fájlméretből és a soronkénti overheadből nagyságrendet ad; a pontos RAM-mérést platformfüggő eszközzel kell elvégezni. A napló rögzítése után a következő lépésben a Git-verziókezelést kapcsoljuk a projekthez, hogy a `scripts/s006_diagnose.py`, a `reports/s006` kimenetek és a nyers adat változásai nyomon követhetők legyenek.

2.3 Git adat- és modellprojektekben

Belépő · szint: alap · idő: 45 perc · eszköz: bash, Git

Előfeltétel: 2.2. alfejezet · Célkészség: Git adat- és modellprojektekben · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

2.3.1 Cél és konkrét helyzet

A 2.2. alfejezetben létrehozott `scripts/s006_diagnose.py` és a `reports/s006` kimenetek már a munkakönyvtárban vannak. A `data/raw/orders_raw.csv` a Northstar nyers adata, amely a generátorral újra előállítható, ezért nem akarjuk a Git-történetbe zárni. A feladat most az, hogy egy új feldolgozási ellenőrző szkriptet, a hozzá tartozó titkokat kizáró szabályt és egy továbbfejlesztett változatot három külön, visszakereshető **commit**ba (véglegesítésbe) szervezzünk.

Bemenet: a fenti könyvtárállapot és a `data/raw/orders_raw.csv`. Megfigyelési egység: egy-egy fájlváltozat a Git által rögzített pillanatképekben. Cél: a három commit logikailag különválik, a nyers adat és a `.env` titokfájl soha nem kerül a repóba. Előzetes becslés: a `.gitignore` után a repó mérete néhány száz KB alatt marad, mert a szándékosan hibás CSV-t a Git nem tárolja.

2.3.2 Fogalmi magyarázat

A Git három területet különböztet meg: a **working tree** (munkakönyvtár) a ténylegesen látható fájlokat, a **staging area** vagy **index** az előkészített, következő commitba szánt állapotot, a **HEAD** pedig az aktuális commit által mutatott pillanatképet jelenti. A `git add` a working tree-ből az indexbe másol, a `git commit` az indexből új, megváltoztathatatlan pillanatképet hoz létre. Minden commitnak egyedi SHA-azonosítója van; a rövid forma például `a3f9c21`. A commit üzenet a „miért”-et rögzíti, nemcsak a „mit”-et.

A **branch** (ág) egy mozgatható névcímke, amely egy commitra mutat. Egy kísérleti változtatást saját ágon érdemes végezni, hogy a főág, például `main`, továbbra is a legutóbbi ismert jó állapotot mutassa. A `git switch -c experiment` új ágat hoz létre; a commitok azon az ágon haladnak előre, a `main` pedig nem mozdul.

A **diff** (különbség) két állapot szövegszintű összehasonlítása. A `git diff` a working tree és az index, a `git diff --staged` az index és a HEAD, a `git diff HEAD` a working tree és a HEAD közötti eltérést mutatja. A kimenet úgynevezett **hunk**okba rendeződik. Egy hunk fejlécében a `@@ -a,b +c,d @@` azt jelenti, hogy a régi fájl `a`. sorától `b` sor, az új fájl `c`. sorától `d` sor érintett. A mínuszos sor törlés, a pluszos sor hozzáadás. Például:

```
@@ -1,6 +1,7 @@
 """Ellenőrzi az orders_raw.csv szerződését a Northstar adatszerződés szerint."""
 from pathlib import Path
 import csv
 import sys
+from collections import Counter
```

Ez azt mutatja, hogy az új változatban a `Counter` import bekerült, és a régi 6 sor helyett 7 sor szerepel; tartalmi konfliktus nincs, a környező sorok kontextusként szolgálnak.

A **.gitignore** fájl olyan mintákat tartalmaz, amelyekre illeszkedő, még nem követett fájlokat a Git figyelmen kívül hagy. A minták például `data/raw/` könyvtárat, `.env` titokfájlt, `__pycache__/` mappát, `*.pyc` bájtkódot vagy `*.joblib` modellfájlt jelölhetnek. Fontos: a szabály csak a még nem indexelt fájlokra hat; ha egy fájl korábban már bekerült a commitba, a `.gitignore` önmagában nem távolítja el a követésből.

A Jupyter-notebookok `.ipynb` kiterjesztésű JSON-fájlok, így a nyers `git diff` a cellakimenetek és a metaadatok miatt zajos. Jó gyakorlat a notebookot kimenet nélkül commitolni, például `jupyter nbconvert --to notebook --ClearOutputPreprocessor.enabled=True`

`--inplace analysis/eda.ipynb` paranccsal, vagy a végső logikát külön Python-szkriptbe exportálni. A `nbdime` nevű eszköz notebookbarát diffet ad, de nem kötelező telepíteni; a tanulási főút a notebookból scriptté alakítást részesíti előnyben.

A **nagy binárisok határa**: a Git a pillanatképekben a teljes fájltartalmat tárolja, még tömörítve is. Egy 200 MB-os Parquet-fájl minden commitban újabb másolatot jelent; a diff nem értelmes, a repó mérete gyorsan nő. Ezért a nagy nyers adat, a generált feature-tábla, a modellfájl és a notebook checkpoint nem a Git-repó tartalma, hanem külön, verziózott adattároló vagy az újragenerálást leíró kód része.

2.3.3 Teljes minimális munkapélda

Az alábbi munkamenet Linux/macOS Bash héjon fut, munkakönyvtára `~/projects/northstar-lab`. A `data/raw/orders_raw.csv` a 2.2 alatt már létrejött, a `scripts/s006_diagnose.py` pedig a repóban lesz. A kimenetek szemléltetőek, nem mért futtatási jegyzék; a parancsok a leírt állapotból kiindulva ezt az eredményt adják, de a commit-hash-ek gépenként eltérhetnek.

Először inicializáljuk a repót és rögzítjük a helyi azonosítót:

```
git init -b main
git config user.name "N. Diák"
git config user.email "diak@example.com"
```

Létrehozzuk az első ellenőrző szkriptet, amely a Northstar szerződés szerint a `data/raw/orders_raw.csv` oszlopait és a hiányzó `distance_km` értékeket vizsgálja:

```
"""Ellenőrzi az orders_raw.csv szerződését a Northstar adatszerződés szerint."""
from pathlib import Path
import csv
import sys

RAW = Path("data/raw/orders_raw.csv")

def main() -> int:
    if not RAW.exists():
        print(f"missing input: {RAW}", file=sys.stderr)
        return 2
    with RAW.open(newline="") as f:
        reader = csv.DictReader(f)
        fieldnames = reader.fieldnames or []
        rows = list(reader)
    required = {"order_id", "customer_id", "order_ts", "distance_km", "basket_minor"}
    missing_cols = required - set(fieldnames)
    if missing_cols:
        print(f"missing columns: {sorted(missing_cols)}", file=sys.stderr)
        return 3
    missing_distance = sum(1 for r in rows if r["distance_km"] == "")
    print(f"rows={len(rows)} missing_distance_km={missing_distance}")
    return 0 if missing_distance == 0 else 1

if __name__ == "__main__":
    raise SystemExit(main())
```

Ez az első commit. A szakaszos mentés a `git add` és a `git commit` páros:

```
mkdir -p scripts
# a fenti tartalmat a scripts/s007_check_orders.py fájlba mentjük
git add scripts/s007_check_orders.py
git commit -m "Add initial orders_raw contract checker
"
```

A második commit a `.gitignore` szabályt rögzíti. A nyers adat és a titokfájl kimarad:

```
cat > .gitignore <<'EOF'
data/raw/
.env
artifacts/
__pycache__/
*.pyc
EOF
git add .gitignore
git commit -m "Ignore raw data and local secrets"
```

A harmadik commit kiterjeszti a szkriptet a duplikált `order_id` és a negatív `basket_minor` ellenőrzésével. A `git diff` először a working tree és az index közti változást mutatja; feltesszük, hogy a módosítás már megtörtént. A `Counter` import hunkja a fenti fogalmi példában látható, a további hunk pedig:

```
@@ -19,4 +20,8 @@ def main() -> int:
     if missing_cols:
         print(f"missing columns: {sorted(missing_cols)}", file=sys.stderr)
         return 3
     missing_distance = sum(1 for r in rows if r["distance_km"] == "")
-    print(f"rows={len(rows)} missing_distance_km={missing_distance}")
-    return 0 if missing_distance == 0 else 1
+    ids = [r["order_id"] for r in rows]
+    duplicates = {k: v for k, v in Counter(ids).items() if v > 1}
+    negative_basket = sum(1 for r in rows if r["basket_minor"] and int(r["basket_minor"]) < 0)
+    print(f"rows={len(rows)} missing_distance_km={missing_distance} "
+          f"duplicate_order_ids={sum(duplicates.values())}
 ↪ negative_basket_minor={negative_basket}")
+    return 0 if missing_distance == 0 and not duplicates and negative_basket == 0 else 1
```

A szkript tetején a `from collections import Counter` sor is megjelenik. A commit:

```
git add scripts/s007_check_orders.py
git commit -m "Add duplicate and negative basket checks"
```

A végeredmény ellenőrzése:

```
git log --oneline --reverse
git ls-files data/raw
git check-ignore -v data/raw/orders_raw.csv
```

Szemléltető kimenet:

```
a3f9c21 Add initial orders_raw contract checker
b7e41d2 Ignore raw data and local secrets
c2d88e5 Add duplicate and negative basket checks
```

A `git ls-files data/raw` üres kimenete azt igazolja, hogy a nyers fájl nincs a commitokban. A `git check-ignore -v` például a `.gitignore:1:data/raw/ data/raw/orders_raw.csv` sort írja ki, tehát a szabály ténylegesen illeszkedik.

2.3.4 Eredmény értelmezése

A három commit logikailag elkülönül: az első a minimális ellenőrzést hozza létre, a második a tárolási határt rögzíti, a harmadik a szerződés szerinti hibák további osztályait fedi le. A `git log --oneline --reverse` három sort ad; a `git ls-files data/raw` üres; a `git status --porcelain` tiszta. A `git show --stat HEAD` a harmadik commitban csak a `scripts/s007_check_orders.py` módosítását mutatja, a `.gitignore` nem érintett. A `git`

`diff HEAD~1 -- scripts/s007_check_orders.py` kizárólag a harmadik commit hunkjait tartalmazza. A script kimenete elméletileg a Northstar szerződésből várt hibákat jelzi: `rows=2405`, `missing_distance_km=2`, `negative_basket_minor=1`, a duplikátumszám pedig nulla feletti; a pontos szám a helyi futtatásból derül ki. A `.gitignore` miatt sem a `data/raw/orders_raw.csv`, sem a `.env`, sem a generált `artifacts/` tartalom nem kerül a repóba.

Titok és adatreprodukció. A `.gitignore` csak a még nem követett fájlok felvételét akadályozza meg; a már commitolt titkot nem törli a történetből. Ilyen esetben a titkot vissza kell vonni vagy rotálni, és szükség esetén a történetet külön eljárással tisztítani. Személyes vagy licencköteles adatot a Gitből való kizárás önmagában nem tesz jogszerűvé.

A Git kis szöveges forrásfájlok és konfigurációk verziózására alkalmas; adat- és modellartefaktumoknál külön tartalomazonosító és hozzáférési szabály szükséges. **Gyakorlati kontroll:** `git status --short`, `git ls-files`, titokkereső ellenőrzés és a generált állományok kizárása a commit előtt.

2.3.4.1 Oktatási repó célzott commitokkal és jelölt hibás PR-rel

A Git-gyakorlatok akkor ismételhetők, ha van egy tényleges, rögzített történetű repó. A `lab/git_training/build_repo.py` ezt állítja elő a `dist/git-training/northstar-git` mappába. A szerző, a dátum és a commitüzenet rögzített, ezért ugyanaz a Git-verzió ugyanazokat a hash-eket adja; az ebben a kiadásban rögzített azonosítók:

Commit	Hash	Tartalom
`base`	`2eeb37bbd1820758d30e1f151f05460d66a…`	`.gitignore`, csomagváz
`summarize`	`a1cb2bf4dde6c4ed9d69aecfa0e…`	az első tesztek
`strict`	`77ed34f848e095d635319bf0ae2e…`	`v1.0.0` tag
`duplicate_report`	`8b3a571619416bdd7c8bb51e364…`	`feature/duplicate-report` ág
`bad_customer_count`	`51523885aeb6da9e17b4e68787d…`	`bug/customer-count` ág, jelölt hibás PR

A repó előállítása és a történet áttekintése:

```
python3 lab/git_training/build_repo.py
cd dist/git-training/northstar-git
git log --oneline --graph --all --decorate
git diff main...bug/customer-count
```

A `main` ágon a tesztek zöldek; a `bug/customer-count` ágon a `summarize()` a `customers` mezőt `len(ids)`-ként számolja `len(set(ids))` helyett, ezért a `tests/test_orders.py` egy tesztje elbukik. Ez a jelölt hibás PR: a `PR.md` azt állítja, hogy a tesztek zöldek, de a diff és a futtatás mást mutat. A gyakorlat:

```
git checkout bug/customer-count
PYTHONPATH=src python3 -m unittest discover -s tests -v
# várható: 4 teszt, 1 failure: tests.test_orders.Summarize.test_counts
git checkout main
```

A reviewer feladata a hiba lokalizálása (`customers: len(ids)` → `len(set(ids))`), a javítás egy külön commitban, a teszt újrafuttatása és a PR-komment megírása. A rögzített commit-azonosítók a `lab/git_training/manifest.json` fájlban vannak; a `tests/test_git_training.py` a repó újraépítését, az ágakat, a taget és a hibás ág elbukó tesztjét ellenőrzi. A repó a `dist/` alatt generált, ezért a könyv forrásába nem kerül be; a tanuló minden gépen ugyanazt a történetet állítja elő.

2.3.5 Feladatok

1. **Értelmezés.** Adott az alábbi diff a `scripts/s007_check_orders.py` egy későbbi változatáról:

```
@@ -24,6 +24,7 @@ def main() -> int:
     negative_basket = sum(1 for r in rows if r["basket_minor"] and int(r["basket_minor"])
 ↪  < 0)
     print(f"rows={len(rows)} missing_distance_km={missing_distance} "
           f"duplicate_order_ids={sum(duplicates.values())}
 ↪  negative_basket_minor={negative_basket}")
-    return 0 if missing_distance == 0 and not duplicates and negative_basket == 0 else 1
+    checks_ok = missing_distance == 0 and not duplicates and negative_basket == 0
+    return 0 if checks_ok else 1
```

   Mit fejez ki ez a változtatás, és milyen commitüzenet illik hozzá? Indokold, miért nem változtatja meg a függvény külső viselkedését.

2. **Megvalósítás.** Készíts három commitot egy új `scripts/s007_check_orders.py` javításról: (a) a hiányzó oszlopok üzenetét egészítse ki a talált oszlopok listájával; (b) a `.gitignore`-t bővítse a `*.joblib` mintával; (c) a szkripthez adjon egy `--strict` kapcsolót, amely a duplikált sorokat is hibának tekinti. Írd le a teljes Bash-parancssort a fájlmódosításoktól a commitokig, és add meg a három commitüzenetet.

3. **Hibadiagnózis.** A `git status` azt mutatja, hogy a `data/raw/orders_raw.csv` a staging area-ban van, pedig a `.gitignore` első sora `data/raw/`. Mi lehet az ok, és pontosan milyen parancsokkal állítja helyre a helyzetet? Miért nem elég a `.gitignore` módosítása?

4. **Transzfer.** A projekt átkerül egy Windows gépre, ahol PowerShell fut. Add meg a három commit PowerShell-parancsokkal, és ellenőrizd, hogy a `.gitignore` mintái változatlanul működnek-e. Ezután a feltétel megváltozik: a kész modellfájlokat `models/*.joblib` néven, a köztes adatokat `data/interim/*.parquet` néven generálja a pipeline. Egészítsd ki a `.gitignore`-t, és egyetlen paranccsal igazolja, hogy egy ilyen fájl valóban figyelmen kívül maradna.

2.3.6 Megoldások és indoklás

1. A diff a hosszú feltételt egy `checks_ok` nevű, beszédes változóba emeli ki, majd a `return` ezt használja. A külső viselkedés azonos, mert a logikai kifejezés értéke nem változott; ez refaktorálás, nem funkcionális javítás. Illő commitüzenet: `Refactor check condition into named variable`. Az indoklásban a diff kontextussorai mutatják, hogy sem a kiírt sor, sem a feltétel tagjai nem változtak.

2. A Bash-munkamenet vázlata:

```
# (a) a szkript üzenetének módosítása után
git add scripts/s007_check_orders.py
git commit -m "Report found columns on schema error"

# (b) a .gitignore kiegészítése után
printf '*.joblib\n' >> .gitignore
git add .gitignore
git commit -m "Ignore serialized model files"

# (c) a --strict kapcsoló implementálása után
git add scripts/s007_check_orders.py
git commit -m "Add strict mode treating duplicates as errors"
```

Az ellenőrző invariant: `git status --porcelain` üres, `git log --oneline --reverse` pontosan három új sort ad, és a `git show --stat HEAD` csak a szkriptfájlt sorolja fel a harmadik commitban.

3. Az ok: a `data/raw/orders_raw.csv` már a `.gitignore` létrehozása előtt bekerült az indexbe; a `.gitignore` csak a még nem követett fájlokat szűri. A helyreállítás két lépésből áll: először a fájlt el kell távolítani az indexből, de a munkakönyvtárban meg kell hagyni, `git rm -r --cached data/raw` paranccsal, majd `git commit -m "Stop tracking raw data"` véglegesíti. A `.gitignore` módosítása önmagában azért nem elég, mert a már indexelt fájl továbbra is a commit tartalma maradna. Az ellenőrzés: `git ls-files data/raw` üres, a `git status` pedig a fájlt már nem mutatja.

4. PowerShell-munkamenet:

```
# a szkript létrehozása után
git add scripts/s007_check_orders.py
git commit -m "Add initial orders_raw contract checker"

# a .gitignore létrehozása után
git add .gitignore
git commit -m "Ignore raw data and local secrets"

# a szkript módosítása után
git add scripts/s007_check_orders.py
git commit -m "Add duplicate and negative basket checks"
```

A `.gitignore` mintája platformfüggetlen: a `data/raw/` és a `.env` minták Windows alatt is ugyanúgy illeszkednek, mert a perjel a Git mintanyelvében könyvtárelválasztóként funkcionál. A kiegészítés:

```
Add-Content .gitignore "models/*.joblib"
Add-Content .gitignore "data/interim/*.parquet"
git add .gitignore
git commit -m "Ignore generated models and interim data"
```

Az ellenőrzés egyetlen paranccsal: `git check-ignore -v models/example.joblib`, amelynek kimenete megmutatja a `.gitignore` megfelelő sorát; ha nincs kimenet, a minta hibás.

2.3.7 Gyakori tévedés, visszautalás és önellenőrzés

Gyakori tévedés, hogy a Git-repót automatikusan nagy adathalmazok és modellbinárisok tárolójának tekintik. Ellenpélda: egy 200 MB-os `orders.parquet` és egy 500 MB-os `model.joblib` minden egyes commitban új teljes másolatot igényelne. Három-négy ilyen commit után a repó több GB-os, a `git clone` és a push percnyi, a diff pedig értelmezhetetlen bináris kimenetet

ad. A helyes határ: a repóba a forrás, a kis kontrolladatok és a szerződések kerülnek; a nagy generált fájlokhoz külön verziózási eszköz való, amit a 31.2. alfejezet tárgyal. A visszautalás az 1.3. alfejezetre: a commit üzenet a tudományos munkanapló része, mert a döntést és a szándékot rögzíti; a manifest hash-ellenőrzése determinisztikus és nem tolerál eltérést, míg a Git-commitok hash-ei a tartalomtól függő, de gépenként is reprodukálható azonosítók. A 2.2. alfejezetre visszatekintve a szkript a hibákat `stderr`-re írja, a `return` érték a processz exit code-ja, így a későbbi automatizált futtatás a Git által verziózott szkriptet közvetlenül használhatja.

Önellenőrzés minden commit előtt: `git diff --check` whitespace-hibát keres, `git status --porcelain` megmutatja a rendezetlen állapotot. A titkos adatok véletlen bekerülését a `git grep -n "api_key" HEAD` vagy hasonló keresés jelzi; ha titok került a history-ba, a fájl törlése önmagában nem elég, a titkot érvényteleníteni kell, és a history újraírása külön, kockázatos művelet. A `git check-ignore -v` a `.gitignore` szabályok tényleges illeszkedését ellenőrzi.

2.3.8 Rövid összefoglaló

A Git a data science projektben a kód, a konfiguráció és az elemzési döntések visszakereshető története. A working tree, az index és a HEAD hármas elkülönítése lehetővé teszi, hogy a `git add` és a `git commit` kis, logikus egységeket rögzítsen; a `git diff` és a hunkok a változás pontos tartalmát mutatják. A `.gitignore` a nyers adatot, titkokat, cache- és modellfájlokat zárja ki; a notebookok zajos JSON-diffje helyett kimenet nélküli commit és scriptexport a praktikus. A nagy binárisok nem a Git-tárolóba valók. A következő alfejezetben ezt a helyi repót egy másik géppel kötjük össze, hogy a projekt azonos környezetben folytatható legyen.

2.4 Másik gép, azonos projekt

Belépő · szint: alap · idő: 45 perc · eszköz: bash, Git

Előfeltétel: 2.3. alfejezet · Célkészség: Másik gép, azonos projekt · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

2.4.1 Cél és konkrét helyzet

A helyzet: a 2.3. alfejezetben felépített helyi Git-repót egy másik gépre visszük át, és azon egy tiszta könyvtárban futtatjuk a Northstar kezdőlabor ellenőrzőjét. A bemenet a klónozott `lab` projekt és a szállított CSV-k; egy megfigyelési egység egy teljes futtatási munkamenet az adott gépen. A cél kettős: (1) a három szerződött adatméret ellenőrzése a másik gépen is ugyanazt az eredményt adja, és (2) ha a futás eltér, a hibát az operációs rendszer, a fájljog vagy az interpreter szintjére tudjuk besorolni, ne pedig automatikusan algoritmushibának gondoljuk.

Előzetes becslés: a `quickcheck.py` sikeres futása a manifestben rögzített `customers=300`, `orders=2400`, `order_items=4796` méreteket és nulla kilépési kódot igazol. Ez a három szám az adatszerződésből származó invariant; ha bármelyik eltér, az hibát jelez, de a hiba típusát még nem mondja meg. Ebben az alfejezetben pontosan ezt a szétválasztást gyakoroljuk.

2.4.2 Fogalmi magyarázat: három környezet és a hibák szétválasztása

Egy „másik gép, azonos projekt” forgatókönyvben három különböző réteget érdemes megkülönböztetni:

1. **Az operációs rendszer és fájlrendszer.** A Windows alapértelmezett fájlrendszer-szemantikája különbözik a Linuxétól és a macOS-étől. A Windows útvonala `C:\Users\anna\projekt`, a POSIX rendszereké `/home/anna/projekt`; a Windows a backslash (\) elválasztót várja a héjban, a Python `pathlib` azonban mindkettőn a `/` jelet is elfogadja. A Linux fájlnevei kis- és nagybetű-érzékenyek (case-sensitive), a Windows és az alapbeállítású macOS fájlrendszer nem. Emiatt a `Orders.CSV` elnevezés Linuxon más fájl, mint az `orders.csv`, Windows-on viszont ugyanarra mutathat. A sorvég (line ending) is platformfüggő: Windows hagyományosan CRLF-et, Linux és macOS LF-et használ. A Git `text=auto` beállítás nélkül ezt a különbséget be tudja vinni a diffbe, ami zajt kelt a review-ban.
2. **A fájljogosultságok és a végrehajtási házirend.** POSIX rendszeren egy script közvetlen futtatásához executable bit (futtathatósági bit) kell: `chmod +x quickcheck.py`, majd `./quickcheck.py`. Windows-on ilyen bit nincs; ott a fájl kiterjesztése és a társítás számít, a PowerShell pedig a scriptekre külön végrehajtási házirendet (execution policy) alkalmaz. A `git clone` a fájl jogosultsági bitjét a repóból hozza, ha az ott commitolva volt; ha a scriptet Windowsról írták, a bit hiányozhat Linuxon.
3. **Az interpreter és a függőségek.** A `python` parancs nem ugyanaz minden gépen. Windows-on a Microsoft Store-hoz irányíthat, ha nincs telepített interpreter; a `py -3.12` launcher viszont konkrét verziót kér. Linuxon gyakran a `python3` a helyes név, a `python` pedig hiányozhat. A virtuális környezet (venv) a géphez és a fájlúthoz kötött: a benne levő `bin/python` (POSIX) vagy `Scripts/python.exe` (Windows) nem hordozható a gépek között. A `uv sync` vagy a `requirements.txt` a függőségeket oldja fel, de az alapinterpretert a felhasználónak kell biztosítania. Ezek a különbségek mind **környezethibák**, nem pedig a kód logikájának hibái.

A WSL (Windows Subsystem for Linux) egy további réteg: a Windows-gazdarendszeren belül futó Linux-környezet, saját fájlrendszerrel, saját processztérrel és saját Python-telepítéssel. A WSL-ből a Windows meghajtó a `/mnt/c` alatt látszik, a Windowsból a WSL-fájlrendszer a `\\wsl.localhost\<disztribúció>` útvonalon. A kettő **nem ugyanaz a fájl- és processzkörnyezet**: a Windowsban telepített Python nem látja a WSL-beli virtuális környezet csomagjait, és fordítva. Egy Docker-konténer szintén külön fájl- és processzkörnyezet; a konténer munkakönyvtára alapértelmezésben nem a gazdagép projektmappája.

A parancssor szintjén a POSIX-héj (Bash, zsh) és a PowerShell parancsai eltérnek. A leggyakoribb megfeleltetések:

Feladat	POSIX (Linux, macOS, WSL)	PowerShell (Windows)
Könyvtár listázása	`ls -la`	`Get-ChildItem -Force`
Munkakönyvtár	`pwd`	`Get-Location`
Szövegfájl sorainak számolása	`wc -l fajl.csv`	`(Get-Content fajl.csv).Count`
Fájl hash	`sha256sum fajl`	`Get-FileHash fajl -Algorithm SHA256`
Környezeti változó	`echo "$HOME"`	`echo $env:USERPROFILE`
Fájl másolása	`cp a b`	`Copy-Item a b`

A táblázat nem teljes szótár, hanem annak illusztrációja, hogy ugyanaz a szándék más szintaxist igényel. A platformfüggő parancsok aktuális viselkedését a saját gép dokumentációja és a sources.md kulcsai alapján ellenőrizzük; ezt az alfejezetet nem konkrét webes futtatásból írjuk.

2.4.3 Teljes minimális példa: tiszta könyvtár klónozással

A példa bemenete a már létező helyi repó. A cél: a másik gépen a `git clone` után **csak** a repóban levő fájlokkal futtatni a kontrollt, semmilyen korábbi munkakönyvtárból örökölt állapottal.

A példa előfeltétele: honnan jön a klón. A PDF önmagában nem hordozza a labor fájljait, ezért a klónozás előtt be kell szerezni a verziózott csomagot. A kanonikus artefaktum a `northstar-lab-1.0.0.zip` (a könyv letöltési csomagjában; a forrásrepóból `python3 lab/package_lab.py` paranccsal újraépíthető), amelynek SHA-256 lenyomata ebben a kiadásban `91557feda5a63df9cd9de1a3dfaec16af35ca9c5a0765ef1f5794678d45f8f55`. Ellenőrzés Linuxon `sha256sum northstar-lab-1.0.0.zip`, Windowson `Get-FileHash northstar-lab-1.0.0.zip -Algorithm SHA256`; a kicsomagolt `northstar-lab-1.0.0/` a labor gyökere. A példa a helyi repót a kicsomagolt mappából inicializálja (`git init`, `git add .`, `git commit -m "Northstar lab 1.0.0"`), így a klónforrás egy rögzített helyi repó; a kézirat forrásrepója `https://github.com/gellistvan/tananyag-generator`. A ZIP SHA-256 lenyomata a csomag bájtjait azonosítja, a Git-commit a forrás előzményét; a hideg telepítéshez az artefaktum-hash a mérvadó.

Linuxon vagy macOS-en (Bash):

```
# A repót a lokális elérési útvonalról klónozzuk; távoli repónál URL jönne.
git clone /mnt/shared/northstar-lab-1.0.0 northstar-bash
cd northstar-bash

# Interpreter-ellenőrzés: a projekt kiinduló célja Python 3.12.
python3 --version

# A függőségmentes kontroll futtatása tiszta klónból.
python3 quickcheck.py
```

Windows PowerShellben:

```
git clone C:\src\northstar-lab-1.0.0 northstar-win
cd northstar-win

# A launcher konkrét verziót keres, ha több Python is telepítve van.
py -3.12 --version

py -3.12 quickcheck.py
```

A `quickcheck.py` a mellékelt CSV-ket ellenőrzi a manifest és az adatszerződés alapján. Ellenőrzött hideg telepítés Linuxon (Python 3.12.3, a ZIP frissen kicsomagolva, hálózat nélkül; a futás a `tests/test_lab.py` hideg-telepítés tesztjében is megismétlődik):

```
$ python3 quickcheck.py
{
  "synthetic": true,
  "orders": 2400,
  "basket_minor": 97383149,
  "item_total_minor": 97383149,
  "shipping_fee_minor": 1690200,
  "wrong_join_shipping_minor": 3380004,
  "late_orders": 491,
  "bad_basket_invariants": 0,
```

```
  "note": "A wrong_join érték szándékosan hibás, tételszinten többszörözött rendelési szállítási
↪  díj."
}
exit code: 0
```

A `python3 generate_data.py` utáni újrafuttatás ugyanezt adja, és a manifest a hat CSV sorszámát is rögzíti: `customers.csv` 300, `orders.csv` 2400, `order_items.csv` 4796, `orders_raw.csv` 2405, `assignments.csv` 300, `outcomes.csv` 300. A `customers` és `order_items` értéket tehát nem a quickcheck kimenete, hanem a manifest `counts` mezője és a hash-ellenőrzés igazolja.

A WSL-beli változathoz ugyanaz a Bash-szekvencia tartozik, de a klón célja a WSL saját fájlrendszerén belül legyen, például `~/northstar-wsl`, nem pedig `/mnt/c/...`, mert a `/mnt/c` alatti munka lassabb és eltérő jogosultság-szemantikát adhat.

Validációs státusz. A fenti hideg telepítés Linuxon mért eredmény: Python 3.12.3, a ZIP frissen kicsomagolva, hálózat nélkül, a `quickcheck.py` a 2400-as `orders`, a 0-s `bad_basket_invariants` és a 0-s kilépési kód mellett a manifest 300/2400/4796 sorszámát igazolja; ugyanezt ismétli a `tests/test_lab.py` hideg-telepítés tesztje. A **Windows PowerShell** és a **WSL** ág ebben a kiadásban nincs futtatva: a parancsok a platform dokumentált viselkedésén alapulnak, de az elfogadási kontroll csak akkor teljesül, ha egy Windows-gépen (PowerShell 7, Python 3.12) és egy külön WSL-disztribúcióban a kicsomagolt ZIP SHA-256 lenyomata ugyanaz, és a `quickcheck.py` ugyanezt a három kontrollértéket adja. A Windows- és WSL-futtatás eredményét a futtatási jegyzőkönyvben külön kell rögzíteni; a Linux-eredményből nem következik automatikusan. A kézirat Git URL-je a forrás előzményét azonosítja, a labor bájtjait azonban a ZIP SHA-256 lenyomata; a labor forrásmappája a kézirat commitjában nem szerepel, ezért a hideg telepítéshez a ZIP az átadási artefaktum.

2.4.4 Eredmény értelmezése és kézi ellenőrzés

A kontroll akkor sikeres, ha a kimenet `orders` értéke 2400, a `bad_basket_invariants` értéke 0, és a kilépési kód 0; a `customers.csv` (300) és `order_items.csv` (4796) sorszámát a manifest `counts` mezője rögzíti, és a quickcheck a fájlok hash-ével együtt ellenőrzi. Ez a három szám nem mérés, hanem az adatszerződés invariánsa: a generátor alap seedjével készült fájlok rögzített sorméretei. Ha a fájlok a klónozás során változtak volna, a manifest hash-ellenőrzése ezt a sorszám-ellenőrzés előtt vagy mellett kimutatná.

Érdemes a `quickcheck.py` futását egy kézi kontrollal is plauzibilissé tenni, amikor a sorok számát a platform saját eszközével is megnézzük:

```
# Linux/macOS/WSL: a CSV első sora fejléc, tehát adatsor = sorok - 1.
wc -l data/customers.csv

# PowerShellben ugyanez; a Count a fejlécsorral együtt számol.
(Get-Content data\customers.csv).Count
```

Ha a két szám 300 (adatsor) plusz 1 fejlécsor, akkor a szemcsézettség egyezik. A `wc -l` csak akkor megbízható, ha a CSV-ben nincs beágyazott sortörés; a Northstar szintetikus adatában ez a szerződés része, más adatnál a CSV-értelmező a hiteles.

A példa lényege nem a három szám memorizálása, hanem az, hogy ugyanaz a repó ugyanazt a kontrollt adja három különböző környezetben. Ha az egyik gépen az eredmény eltér, a hibakeresés sorrendje:

1. **Interpreter-eltérés:** a `python --version` más verziót ad-e, mint a projekt célja? Van-e aktív, másik projektből származó `venv`?

2. **Fájl- és útvonalhiba:** a munkakönyvtár tényleg a klón gyökere? A fájlnév kis-nagybetűje egyezik-e a repóval?
3. **Fájljog:** POSIX rendszeren a script futtatható-e közvetlenül, vagy csak `python3 script.py` módon?
4. **Algoritmushiba:** csak akkor merül fel, ha az előző három réteg azonos, és a kód bemenete is azonos; ekkor a hiba a kód logikájában vagy az adatban van.

Átviteli próba. A másik gépen futó azonos kód nem garantálja az azonos környezetet. Ellenőrizni kell az interpreter pontos verzióját, a függőségi rögzítést, a fájlkódolást, a sorvégeket, az architektúrát és a könyvtárjogokat. A futási jegyzőkönyvben a környezeti eltérés és az adateltérés külön hibakategória legyen.

A `python3 quickcheck.py` használata kikerüli a végrehajtási bit szükségességét, **de nem garantálja**, hogy a helyes interpreter érhető el, hogy a fájl olvasható, vagy hogy minden platformfüggőség teljesül. Ezért a »mindig működik« állítás helyett a konkrét előfeltételeket kell megadni.

2.4.5 Feladatok

1. **Értelmezés.** Egy társ Windows PowerShellben a `python quickcheck.py` parancs után ezt az üzenetet kapja: `Python was not found; run without arguments to install from the Microsoft Store`. Melyik réteg hibája ez? Miért nem jelenti azt, hogy a `quickcheck.py` algoritmusa rossz? Mit javasolj neki első lépésben?

2. **Implementálás.** Készíts egy `launch_checklist.md` nevű fájlt a repó dokumentációs mappájába. A lista három rovatot tartalmazzon: Linux/macOS, Windows PowerShell és WSL. Minden rovathoz írd be sorrendben: (a) a repó klónozása tiszta könyvtárba, (b) a Python-verzió ellenőrzése, (c) a projekt gyökerébe lépés, (d) a `quickcheck.py` futtatása, (e) a három elvárt kontrollérték rögzítése. A fájl legyen platformtáblával ellátott, hogy a társ a parancsok megfeleltetését is lássa.

3. **Hibadiagnózis.** Egy Linuxos gépen a társ ezt futtatja: `./quickcheck.py`, és ezt kapja: `Permission denied`. Ugyanott a `python3 quickcheck.py` sikeresen lefut. Mi a különbség a két parancs között? Melyik a helyes hosszú távú megoldás, ha a repó a scriptet parancsként szeretné futtatni, és mit **nem** szabad megtennie a diagnózis során?

4. **Megváltozott feltétel.** A projektet egy macOS gépre viszed, ahol a `python3 --version` 3.11-et ír, a `pyproject.toml` pedig `requires-python = ">=3.12"`-t rögzít. A `quickcheck.py` mégis hiba nélkül fut, mert standard library-only. A `starter_experiment.py` azonban csomaghiánnyal bukik, amikor `uv sync` nélkül próbálod futtatni. Írd le, milyen kimenetből látod, hogy ez **környezethiba**, nem a kísérleti kód hibája, és milyen parancsokkal jutsz el a kontrollfutásig anélkül, hogy a Python 3.11-et kézzel felülírnád a rendszerben.

2.4.6 Megoldások és indoklás

1. **Megoldás.** Ez interpreter-réteg-hiba: a `python` parancs a Microsoft Store által nyújtott álparancsra mutat, nem egy telepített CPythonra. A `quickcheck.py` hibátlan lehet; a bemenetet és a kódot meg sem kapta. Első lépés a `py -3.12 --version` vagy a `python --version` futtatása, illetve a telepített Python keresése a `py -0` launcherrel. Ha nincs 3.12, a környezetet a projekt elvárásának megfelelően kell telepíteni; a kódhoz nem nyúlunk.

2. **Megoldás.** A lényeg az ellenőrzőlista platformrovata. Egy lehetséges minimális változat:

```
# Indítási ellenőrzőlista

| Lépés | Linux/macOS | Windows PowerShell | WSL |
|---|---|---|---|
| Klón | `git clone <repó> ~/northstar` | `git clone <repó> C:\work\northstar` | `git
↪  clone <repó> ~/northstar-wsl` |
| Lépés a gyökérbe | `cd ~/northstar` | `cd C:\work\northstar` | `cd ~/northstar-wsl` |
| Python-verzió | `python3 --version` | `py -3.12 --version` | `python3 --version` |
| Ellenőrző | `python3 quickcheck.py` | `py -3.12 quickcheck.py` | `python3
↪  quickcheck.py` |
| Elvárt értékek | 300 / 2400 / 4796; exit 0 | ugyanaz | ugyanaz |

A cél: ugyanaz a három kontrollérték mindhárom környezetben.
```

Az elvárt értékek sor: `customers=300`, `orders=2400`, `order_items=4796`, és a kilépési kód 0. Ha a klón forrása nem a szállított repó, a hash-ellenőrzés is kötelező.

3. **Megoldás.** A `./quickcheck.py` a fájlban levő shebang-et és a futtathatósági bitet használja; a `python3 quickcheck.py` explicit módon az interpretert hívja. A `Permission denied` azt jelenti, hogy a fájl nem futtatható közvetlenül, mert a bit hiányzik. A helyes hosszú távú megoldás, ha a scriptet parancsként akarjuk futtatni: `chmod +x quickcheck.py`, és a fájl első sora `#!/usr/bin/env python3` legyen, majd ezt commitolni. **Nem szabad** a diagnózis során a rendszer Python-telepítését globálisan átírni, vagy a `chmod`-ot kihagyva Windowsról áthozott bitre támaszkodni. A `python3 quickcheck.py` a futtathatósági bittől függetlenül indítható, ha a megfelelő Python-interpreter elérhető, a fájl olvasható, és a szükséges függőségek telepítve vannak, ezért a kontroll szempontjából biztonságosabb.

4. **Megoldás.** A környezethibára utaló jelek: a `starter_experiment.py` által importált csomag nevére vonatkozó `ModuleNotFoundError`, vagy a `pyproject.toml` Python-korlátjára hivatkozó `uv`-üzenet. Ez nem a kísérleti statisztika hibája; a script importálás előtt áll meg. A megoldás: telepíteni a projekt által elvárt 3.12-t (például `uv python install 3.12`), a projektben `uv sync`-kel feloldani a függőségeket, majd `uv run python starter_experiment.py` módon futtatni, ami a projekt környezetéhez köti az interpretert. A rendszer 3.11-ét nem kell felülírni; az uv a projektvenv-ben külön 3.12-t kezel. A `quickcheck.py` azért futott, mert standard library-only, így a függőségek hiánya nem érintette. A kettő különbsége pontosan mutatja az interpreter- és függőségi réteg szerepét.

2.4.7 Gyakori tévedés és önellenőrzés

Gyakori tévedés: azt hinni, hogy a Windows, a WSL és a Docker ugyanazt a fájl- és processzkörnyezetet jelenti. Ellenpélda: a felhasználó a WSL-ben létrehoz egy virtuális környezetet a `/home/anna/venv` alatt, majd a Windows PowerShellben aktiválni próbálja a `\\wsl.localhost\...\venv\Scripts\activate` fájlt. A Windows Python nem a WSL-beli `bin/python`-t használja, a csomagok nem látszanak, és a `ModuleNotFoundError` elsőre algoritmushibának tűnik. A javítás a rétegek szétválasztása: a PowerShellben a Windows saját Pythonját és venv-jét kell használni, a WSL-ben pedig a WSL-beli környezetet. A Docker-konténer szintén külön: a konténerben futó script nem a gazdagép munkakönyvtárát látja automatikusan, csak a becsatolt kötetet.

Visszautalás. Az 1.4. alfejezetben a három hibacsalád a programozási, adatértelmezési és statisztikai hiba volt. Ez az alfejezet az első családon belül azonosít további rétegeket: az operációs rendszer, a fájljog és az interpreter hibái mind a kód futtatása előtt jelentkeznek, ezért nem szabad a statisztikai következtetés hibájának tekinteni őket. A 2.3. alfejezetben a Git-réteg tanította, hogy a repó a projektet viszi, nem a gép állapotát; itt ennek következménye, hogy a klón után a környezetet mindig újra kell ellenőrizni.

Önellenőrzés. (1) Ha a `quickcheck.py` két gépen ugyanazt a három számot adja, akkor biztos, hogy a modell is ugyanazt fogja adni? Nem; a kontroll csak a bemenő adatméretet és a környezet egy részét ellenőrzi. (2) Melyik parancs mutatja meg először, hogy a `python` a Microsoft Store csapdájára mutat? A `python --version` hibaüzenete vagy a launcher hiánya. (3) Miért hibás a `chmod`-dal kapcsolatosan a „Windowsról hozott jogosultság” feltételezése? Mert a Windows-fájlrendszeren nincs POSIX futtathatósági bit; a bit a Git által tárolt metainformációból származik, ha a repóban rögzítve volt.

2.4.8 Rövid összefoglaló

Egy projekt másik gépen akkor folytatható, ha az operációs rendszer, a fájljog és az interpreter különbségeit nem keverjük a kód hibájával. A Linux, macOS, Windows és WSL eltérő fájlrendszer-szemantikát, sorvég- és jogosultságkezelést hoz; a PowerShell és a POSIX-héj parancsai más szintaxisúak ugyanarra a szándékra. A tiszta klónból futtatott `quickcheck.py` a szerződött 300/2400/4796 kontrollértékkel ellenőrzi, hogy a projekt bemenete azonos. A hibadiagnózis sorrendje: interpreter, fájl-útvonal, fájljog, és csak ezek után algoritmus. A következő alfejezetben ezt a réteget tovább szűkítjük: a Python-verziót és a csomagfüggőségeket izolált környezetbe zárjuk.

3 Python-verzió, csomagok és izolált környezet

3.1 Interpreter és virtuális környezet

Belépő · szint: alap · idő: 45 perc · eszköz: uv, Python 3.12

Előfeltétel: 2.4. alfejezet · Célkészség: Interpreter és virtuális környezet · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

3.1.1 Cél és konkrét helyzet

Egy elemzés akkor reprodukálható, ha nemcsak a kódot, hanem a kódot futtató környezetet is ismerjük. A gyakori hibajelenség: a notebookban lefutó script egy másik gépen nem találja a NumPy-t, vagy éppen más verziót tölt be, mint amit a szerző látott. A diagnosztika első lépése nem a csomagok újratelepítése, hanem a futó folyamat önazonosítása.

A konkrét feladat bemenete: terminálparancsok egy meghatározott munkakönyvtárban. A megfigyelési egység: egy elindított Python-folyamat, pontosabban annak interpreter- és modulkeresési állapota. A kimenet: környezetjelentés, amely tartalmazza a `sys.executable`, `sys.prefix`, `sys.path`, Python-verzió, illetve a NumPy verzióját és betöltési fájlját. Az ellenőrzési feltétel: ha a kódot a `.venv-a/bin/python` indította, akkor a jelentésben a `sys.prefix` a `.venv-a` mappára mutat, a `numpy.__file__` pedig ugyanezen prefix alatti `site-packages`-ben kezdődik. Előzetes becslés: a `.venv-a` és `.venv-b` két jelentése eltérő NumPy-verziót ír ki, miközben a Python-verzió azonos marad. A futtatott példák ebben a dokumentumban nem ténylegesen mért kimenetek; ahol fájl- vagy verzióadat szerepel, ott „szemléltető” jelölést használunk.

3.1.2 Fogalmi magyarázat: interpreter, venv, sys.path és PATH

Az **interpreter** (értelmező) az a program, amely a Python-forrást gépi végrehajtásra alakítja. A tanulási alap itt a CPython 3.12, tehát a hivatalos C-ben írt interpreter. A `python3.12` parancs nem magát a nyelvet jelöli, hanem egy konkrét futtatható fájlt. A folyamaton belül a `sys.executable` mondja meg, hogy az adott futó Python melyik fájlból indult:

```
import sys
print(sys.executable)
```

Szemléltető kimenet: `/home/user/ds-work/s009/.venv-a/bin/python`. A `sys.version` ezzel szemben a futtató interpreter verzióját adja; a kettő együtt használható gépnaplóba, mert egyazon `python` név mögött is több verzió állhat a PATH sorrendjétől függően.

A **virtuális környezet** (virtual environment, venv) nem a Python másolata, hanem egy mappa, amely saját interpreter-hivatkozást és saját csomagtelepítési területet tart fenn. A `python -m venv .venv-a` parancs létrehozza például a következő szerkezetet Linuxon:

- `.venv-a/bin/python` – az indítószkript,
- `.venv-a/lib/python3.12/site-packages/` – az ide települő csomagok,

- `.venv-a/pyvenv.cfg` – a környezet beállításai.

A `pyvenv.cfg` egyik kulcsa a `include-system-site-packages = false`. Ez az alapértelmezés biztosítja, hogy a rendszer Python csomagjai nem látszanak a venv-ből, tehát a NumPy telepítése a venv-be ténylegesen izolált marad. A venv tehát a Python saját környezetizolációs eszköze; a conda/mamba alternatíva ezen túl natív binárisokat és akár más Python-verziót is telepíthet, ezért a választást a későbbi `s011` alfejezet tárgyalja.

A **`sys.path`** a modulkeresés útvonalainak listája. Amikor a kód `import numpy`-t ír, a Python e listán sorban keresi a modult. Script futtatásakor az első elem a script könyvtára, `-c` kapcsolóval vagy interaktív módban pedig az aktuális munkakönyvtárat jelölő üres string. A venv `site-packages` könyvtára a lista végén szerepel, de a konkrét betöltést nem a lista puszta léte, hanem a modul `__file__` attribútuma bizonyítja:

```
import numpy as np
print(np.__file__)
```

Szemléltető kimenet: `/home/user/ds-work/s009/.venv-a/lib/python3.12/site-packages/numpy/__init_`
Ha ez az útvonal nem a `sys.prefix` alatt kezdődik, akkor a NumPy máshonnan jött, például a `PYTHONPATH` környezeti változóból vagy a rendszer Python telepítéséből.

A **PATH** ezzel szemben operációsrendszer-szintű keresőlista: amikor a terminálban a `python` vagy a `pip` parancsot begépeljük, az operációs rendszer e lista elemeit járja végig. Linuxon a `which python` csak az első találatot mutatja, a `type -a python` viszont az összes jelöltet; Windowson a `Get-Command python -All` a PowerShell-megfelelő. A `source .venv-a/bin/activate` parancs épp ezt a listát írja át az adott terminálablakban: a venv `bin` könyvtára kerül előre. A direkt interpreter-hívás, például `.venv-a/bin/python script.py`, megkerüli ezt a PATH-függést, mert a teljes útvonalat adjuk meg.

A **pip** a Python csomagtelepítője. A `pip` parancs valójában egy Python-szkript, amelynek saját interpreter-kötése van; a `python -m pip` viszont az adott `python`-hoz tartozó pip modult futtatja. Ez a különbség akkor válik fontossá, ha a PATH-on a `pip` más Pythonhoz tartozik, mint a `python`. A `pip --version` kimenete elárulja a hozzá tartozó Pythont:

Szemléltető kimenet: `pip 24.2 from /home/user/ds-work/s009/.venv-a/lib/python3.12/site-packages/pip (python 3.12)`. A `python -m pip show numpy` parancs `Location` mezője pedig a telepített NumPy mappáját adja; ennek egyeznie kell a futó kód `numpy.__file__` útvonalának elejével.

3.1.2.1 Hibamátrix: rossz interpreter, rossz kernel, hiányzó import

A `python -c`, az `uv run python` és a notebook kernel gyakran három különböző futtatható fájlt indít. A különbséget stabil invariánsokkal lehet azonosítani: a `sys.executable` és a `sys.prefix` viszonya, az importált modul `__file__` helye, valamint az elvárt interpreterrel való egyezés. A verziószám ehhez nem elég, mert két környezetben azonos Python-verzió mellett is más lehet a csomag.

Állapot	Bizonyíték	Tipikus ok	Javítás
`wrong_interpreter`	`sys.executable` nincs a várt prefix alatt	a `python` a PATH-ból a rendszerinterpretettre mutat	`.venv/bin/python` vagy `uv run python` használata

Állapot	Bizonyíték	Tipikus ok	Javítás
`wrong_kernel`	a notebook `sys.executable`-je eltér a terminálétól	a kernel másik környezethez regisztrált	kernel újraregisztrálása és kiválasztása
`missing_import`	`find_spec None`, `ImportError`	a csomag nincs a futó környezetben	`uv sync` vagy a környezet csomagkezelője
`global_site_packages`	a modul a `base_prefix` alatt van, miközben venv aktív	`--system-site-packages` vagy `PYTHONPATH`	a venv elkülönítése, az import eredete ellenőrzése
`ok`	az executable a prefix alatt, a modul ugyanott	—	nincs teendő

A `lab/env_probe.py` a folyamat jelentését JSON-ban adja, és a `classify()` függvény a fenti állapotot számítja:

```
from pathlib import Path

def file_under(path, prefix):
    if not path or not prefix:
        return False
    try:
        # A venv python sokszor symlink; az absolute() nem oldja fel a linket.
        Path(path).absolute().relative_to(Path(prefix).absolute())
        return True
    except ValueError:
        return False

def classify(value, expected_prefix=None, expected_executable=None):
    if expected_prefix and not file_under(value["executable"], expected_prefix):
        return "wrong_interpreter"
    if expected_executable and Path(value["executable"]).resolve() !=
     ↪  Path(expected_executable).resolve():
        return "wrong_kernel"
    for name, item in value.get("imports", {}).items():
        if not item.get("found"):
            return "missing_import"
        if (value.get("venv_active") and file_under(item.get("origin"),
         ↪  value.get("base_prefix"))
                and not file_under(item.get("origin"), value.get("prefix"))):
            return "global_site_packages"
    return "ok"
```

A gyakorlat három állapota egyetlen parancsban összehasonlítható:

```
python3 lab/env_probe.py --import numpy --compact
.venv/bin/python lab/env_probe.py --expect-prefix .venv --import numpy --compact
uv run python lab/env_probe.py --expect-prefix .venv --import numpy --compact
```

A helyes válasz nem az, hogy mindhárom `ok`, hanem hogy mindhárom jelentésben látod, melyik interpreter futott, és az elvárt prefixhez képest mi az eltérés. A notebookban ugyanezt a `sys.executable` kiírásával és a kernel nevének ellenőrzésével lehet megtenni; a `tests/test_env_probe.py` a `wrong_interpreter`, `wrong_kernel`, `missing_import`, `global_site_packages` és `ok` állapotot szintetikus jelentéseken teszteli. A szkript kimenete

gépfüggő, ezért a könyvben csak a mezők és az invariánsok szerepelnek, nem rögzített verziószámok.

3.1.3 Teljes minimális munkapélda: két környezet, egy jelentés

A következő munkamenet Linuxon vagy macOS-en bash-terminálban fut, CPython 3.12 és PyPI-hálózat elérhetősége mellett. A munkakönyvtár legyen `~/ds-work/s009`. A példa a venv/pip alternatív utat mutatja; a könyv fő útja az uv, amelyet a következő, `s010` alfejezet vezet be.

```
mkdir -p ~/ds-work/s009
cd ~/ds-work/s009
python3.12 -m venv .venv-a
python3.12 -m venv .venv-b

.venv-a/bin/python -m pip install numpy==1.26.4
.venv-b/bin/python -m pip install numpy==2.0.2
```

A verziószámok itt példák; a ténylegesen elérhető verziókat a PyPI aktuális listája adja meg, ezért a helyi futtatáskor a telepítési log rögzítendő. Ezután hozzunk létre egyetlen `env_report.py` fájlt:

```
import json
import sys

def main():
    try:
        import numpy as np
    except ImportError:
        np = None

    report = {
        "executable": sys.executable,
        "prefix": sys.prefix,
        "python_version": (
            f"{sys.version_info.major}."
            f"{sys.version_info.minor}."
            f"{sys.version_info.micro}"
        ),
        "sys_path": list(sys.path),
        "numpy_version": None if np is None else np.__version__,
        "numpy_file": None if np is None else str(np.__file__),
    }
    print(json.dumps(report, indent=2))

if __name__ == "__main__":
    main()
```

A scriptet mindkét környezet interpreterével futtatjuk, aktiválás nélkül:

```
.venv-a/bin/python env_report.py > report-a.json
.venv-b/bin/python env_report.py > report-b.json
```

A `report-a.json` szemléltető, elméletileg várt tartalma:

```
{
  "executable": "/home/user/ds-work/s009/.venv-a/bin/python",
  "prefix": "/home/user/ds-work/s009/.venv-a",
  "python_version": "3.12.7",
  "sys_path": [
    "/home/user/ds-work/s009",
    "/home/user/ds-work/s009/.venv-a/lib/python312.zip",
```

```
    "/home/user/ds-work/s009/.venv-a/lib/python3.12",
    "/home/user/ds-work/s009/.venv-a/lib/python3.12/lib-dynload",
    "/home/user/ds-work/s009/.venv-a/lib/python3.12/site-packages"
  ],
  "numpy_version": "1.26.4",
  "numpy_file": "/home/user/ds-work/s009/.venv-a/lib/python3.12/site-packages/numpy/__init__.py"
}
```

A `report-b.json` ugyanezen a gépen csak a `.venv-b` útvonalakban és a NumPy-verzióban tér el, `2.0.2` értékkel. Windowson a munkamenet szerkezete azonos, de a binárisok a `Scripts` mappában vannak, és a `py` launchert használjuk:

```
mkdir $HOME\ds-work\3.1
cd $HOME\ds-work\3.1
py -3.12 -m venv .venv-a
py -3.12 -m venv .venv-b
.venv-a\Scripts\python.exe -m pip install numpy==1.26.4
.venv-b\Scripts\python.exe -m pip install numpy==2.0.2
.venv-a\Scripts\python.exe env_report.py | Out-File -Encoding utf8 report-a.json
```

A `sys.executable` itt például `C:\Users\user\ds-work\3.1\.venv-a\Scripts\python.exe` alakú.

3.1.4 Eredmény értelmezése

A két jelentés együtt bizonyítja az izolációt: azonos `env_report.py` kód, azonos Python-verzió, de eltérő `prefix`, eltérő `site-packages` és eltérő NumPy-verzió. A kézi kontroll invariáns: a `numpy_file` stringnek a `prefix` stringgel kell kezdődnie. Ha ez nem teljesül, a telepítés nem a várt környezetbe került, vagy egy `PYTHONPATH` elem elé vág. A `sys_path` első eleme itt `/home/user/ds-work/s009`, tehát a script könyvtára; ez nem a NumPy forrása, ezért nem szabad a modulkeresés bizonyítékaként a lista puszta jelenlétére hagyatkozni.

Az aktiválás elhagyása szándékos. A `.venv-a/bin/python` teljes útvonalas hívás akkor is ugyanoda mutat, ha a terminál PATH-ja közben megváltozott. A `source .venv-a/bin/activate` csak kényelmi réteg: rövid parancsneveket ad, de nem változtatja meg a már futó folyamatok környezetét. A környezetjelentés tehát akkor értékes, ha a tényleges futtató interpreterrel készül, nem a terminál aktuális PATH-állapotából következtetünk.

Interpreterazonosság. A `python`, `python3` és `py` parancs ugyanazon gépen sem feltétlenül ugyanazt az interpretert választja. A telepítési és ellenőrzési példákban a kiválasztott interpretert a `sys.executable` értékével ellenőrizzük, és a csomagkezelést ehhez kötjük: `python -m pip ...`. A virtuális környezet izolációja elsősorban a Python-csomagokra vonatkozik, nem teljes operációsrendszer-szintű konténerizáció.

Ellenpróba: a környezet aktiválása után futtasd a `python -c 'import sys; print(sys.executable)'` ellenőrzést, majd a notebook kernelében is nézd meg ugyanazt az értéket. Ha különböznek, a notebook nem bizonyítottan a kívánt környezetből fut.

3.1.5 Feladatok

1. **Értelmezés.** Egy terminálban a `which python` az `/usr/bin/python` útvonalat írja ki, de a `python -c "import sys; print(sys.executable)"` parancs a `/home/user/ds-work/s009/.venv-a/bin/python` értéket adja. Nevezd meg legalább két lehetséges magyarázatot, és add meg azt a parancsot, amellyel az összes `python` jelöltet kilistázná.

2. **Implementálás.** Egészítsd ki az `env_report.py` scriptet egy `--check` kapcsolóval. A kapcsoló jelenléte esetén a script ne csak kiírja a jelentést, hanem ellenőrizze is, hogy a betöltött NumPy `__file__` útvonala a `sys.prefix` alatt kezdődik, és a futtató Python legalább 3.12-es. Ha bármelyik feltétel sérül, 1 kilépési kóddal térjen vissza; sikeres ellenőrzésnél 0-val. Ügyeljen az útvonal-normalizálásra Windows és Linux esetén.

3. **Hibadiagnózis.** Egy felhasználó a rendszerterminálban kiadta a `pip install numpy==2.0.2` parancsot, de a már megnyitott Jupyter notebookban az `import numpy as np; np.__version__` továbbra is `1.26.4`. Magyarázd el a két legvalószínűbb okot, és írd le azokat a parancsokat, amelyeket a notebook cellájában, illetve a terminálban futtatva eldönthető, melyik környezetbe települt a 2.0.2-es NumPy.

4. **Megváltozott feltétel.** Egy következő gépen nincs `python3.12` nevű parancs: a `python3` a CPython 3.12-t, a `python` viszont csak 3.9-et indít. Írj egy ellenőrző parancsot a venv létrehozása előtt, amely megtagadja a futást, ha a kiválasztott interpreter 3.12-nél régebbi, majd a `python3` használatával hozza létre a `.venv-a` környezetet. Választhat bash vagy PowerShell megoldást.

3.1.6 Megoldások és indoklás

1. megoldás. Az egyik magyarázat, hogy a `source .venv-a/bin/activate` után a shell PATH-ja tartalmazza a `.venv-a/bin` mappát, de a `which` egy másik terminálablakból futott. A másik, hogy a shell a korábbi `python` útvonalat gyorsítótárban tartja (`hash`), és az aktiválás után `hash -r` nélkül a régi találatot adja vissza. A teljes lista Linuxon: `type -a python`; Windowson PowerShellben: `Get-Command python -All`. Az ellenőrzés lényege: a `which` egyetlen találat, a `type -a` az összes PATH-jelölt.

2. megoldás. A kiegészített script kulcsrésze:

```
import argparse
from pathlib import Path

def check(report):
    prefix = Path(sys.prefix).resolve()
    ok = True
    if report["numpy_file"] is not None:
        np_file = Path(report["numpy_file"]).resolve()
        try:
            np_file.relative_to(prefix)
        except ValueError:
            ok = False
    if sys.version_info < (3, 12):
        ok = False
    return ok
```

A `Path.resolve()` szimbolikus hivatkozásokat és relatív elemeket is abszolút útvonallá alakít, így a `relative_to` nem szembesül a Windows `C:\` és Linux `/` alakok keveredésével. A `--check` argumentumot az `argparse` elemzi; hiba esetén `sys.exit(1)`, siker esetén `sys.exit(0)`. Az invariant tehát nem string-összehasonlítás, hanem „a NumPy fájl a prefix mappa alatt van”.

3. megoldás. A legvalószínűbb ok, hogy a terminál `pip` parancsa nem a notebook kerneléhez tartozó Pythonhoz kötődik, hanem egy másik környezethez. A notebook kernel ugyanis már futó folyamat; a terminál telepítése nem módosítja annak `sys.path` állapotát. A diagnosztika a notebook cellájában:

```
import sys
print(sys.executable)
print(sys.prefix)
import numpy as np
print(np.__version__)
print(np.__file__)
```

A terminálban ugyanezt a környezetet a `python -m pip show numpy` `Location` mezőjével vetjük össze. Ha a notebook `sys.executable` értéke `.venv-b/bin/python`, a telepítés pedig a rendszer Pythonba ment, a kettő természetesen eltér. A javítás nem a PATH átírása, hanem a notebook kernelének újraindítása a megfelelő interpreterrel, vagy a telepítés a kernel interpreterével: `import sys; !{sys.executable} -m pip install numpy==2.0.2`.

4. megoldás. Bash változat:

```
python3 -c "import sys; sys.exit(0 if sys.version_info >= (3, 12) else 1)" \
  && python3 -m venv .venv-a
```

PowerShell változat, ha a `py` launcher elérhető:

```
py -3.12 -c "import sys; assert sys.version_info >= (3, 12), sys.version"
py -3.12 -m venv .venv-a
```

A feltétel célja, hogy a 3.9-es `python` soha ne kerüljön a létrehozó parancsba. A `python3` név itt verzióellenőrzés után kap szerepet; ha a gépen a `python3` is 3.11 lenne, a parancs a venv létrehozása előtt leáll, ami a kívánt viselkedés.

3.1.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: **a terminálban aktivált környezet automatikusan átváltja a már futó notebook kernelét.** Tegyük fel, hogy a notebook a `.venv-b` kernellel fut; a felhasználó ezután egy másik terminálban kiadja a `source .venv-a/bin/activate` parancsot, majd a notebook cellájába írja: `import numpy as np; print(np.__version__)`. A kimenet továbbra is a `.venv-b` NumPy verzióját adja, mert a kernel folyamata már elindult, és az aktiválás csak az adott terminál jövőbeli gyermekfolyamatainak PATH-ját módosította. A hibás következtetés így hangzik: „most már jó, hiszen a terminálban aktiválva van a környezet”. A helyes ellenőrzés a notebookban a `sys.executable` kiíratása; ha az még `.venv-b/bin/python`, akkor a környezet nem váltott. Ez a probléma rokona annak, amit az `s005` alfejezet tárgyalt: a notebook kernel CWD-je sem a notebook mappája automatikusan, ezért minden futó folyamatot a saját állapotából kell bizonyítani, nem a terminál látszólagos állapotából.

3.1.8 Önellenőrzés

- Mi a különbség a `sys.executable` és a `which python` információja között?
- Miért nem bizonyítja a `sys.path` lista puszta jelenléte, hogy a NumPy a venv-ből töltődött be?
- Melyik parancs garantálja, hogy a csomagtelepítés ahhoz a Pythonhoz kapcsolódik, amelyet a későbbi script futtat?
- A `report-a.json` és `report-b.json` mely mezőinek kell eltérniük, és melyiknek kell azonosnak lenniük az izoláció igazolásához?
- Mi a teendő akkor, ha a `numpy.__file__` nem a `sys.prefix` alatt kezdődik?

3.1.9 Rövid összefoglaló

A futó környezet kiléte nem hitkérdés: a `sys.executable`, a `sys.prefix`, a `sys.path` és a modul `__file__` mezője együtt megmutatja, melyik interpreter és melyik site-packages szolgálja ki a kódot. A venv akkor izolál, ha a megfelelő interpretert indítjuk; a PATH és a csupasz `pip` parancs eltérése gyakori forrása a csendes keveredésnek. Két környezetre lefuttatott környezetjelentés a verziók és útvonalak összevetésével bizonyítja a szétválasztást. A következő alfejezet ezt a szerződést szigorítja: az uv zárolt függőségfeloldást készít, és a `--locked` ellenőrzést teszi a reprodukálhatóság kapujává.

3.1.10 Kiadás előtti szakmai ellenőrzés

Platformellenőrzés. Windows PowerShellben aktiválás nélkül is ellenőrizhető a környezet saját interpretere: `./.venv/Scripts/python.exe --version`. Unix-szerű rendszeren a megfelelő útvonal `./.venv/bin/python --version`. A rendszer `python` parancsának elérhetősége önmagában nem bizonyítja, hogy az izolált környezet aktív.

3.2 uv projekt és ténylegesen rögzített függőségek

Belépő · szint: alap · idő: 45 perc · eszköz: uv, Python 3.12

Előfeltétel: 3.1. alfejezet · Célkészség: uv projekt és ténylegesen rögzített függőségek · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

3.2.1 Cél és konkrét helyzet

Ebben az alfejezetben egy projektkörnyezet három, egymástól élesen különböző pillanatfelvételét választjuk szét: a `pyproject.toml`-ban megfogalmazott függőségi tartományt, az `uv.lock`-ban rögzített pontos feloldást, és a `.venv`-ben ténylegesen telepített csomagkészletet. A megfigyelési egység most nem adatsor, hanem egy projektkörnyezeti állapot.

A labor bemenete a mellékelt `lab/pyproject.toml`, amely a Northstar-projekt alapvető numerikus függőségeit írja le. A szállított labor szándékosan **nem** tartalmaz `uv.lock` fájlt, mert a lock előállítása ennek az alfejezetnek az önálló feladata. A `lab/reference/uv.lock` viszont a szerző által ellenőrzött referenciafeloldás (uv 0.13.6, Python 3.12, 282 csomag, SHA-256 `7cca92a06418c46c83f49885a4686a83506c1ace22813d55a94cbfae67d90707`), amelyhez a saját megoldás hasonlítható. A feladat: saját gépen készítsük el a lockot, vessük össze a referenciával, telepítsük a környezetet, majd bizonyítsuk, hogy a futás friss feloldás nélkül, `--locked` módban megismételhető.

Előzetes becslés: az első `uv lock` létrehoz egy, a `pyproject.toml`-nál nagyobb `uv.lock` fájlt, mert a tranzitív függőségeket is rögzíti. A `uv sync --locked` parancs 0 kilépési kóddal zárul, a környezet Python-verziója 3.12.x, és a `--locked` módú második futtatás között a lock fájl hash-e változatlan marad.

3.2.2 Három réteg, nem három szinonima

A leggyakoribb félreértés az, hogy a `pyproject.toml`, az `uv.lock` és a `.venv` ugyanannak a dolognak három neve. Valójában három különböző kérdésre válaszolnak:

Réteg	Mit mond?	Példa
`pyproject.toml`	Mi az elfogadható?	`pandas>=2.2,<3`
`uv.lock`	Mit választottunk pontosan?	`pandas==2.3.3`
`.venv`	Mi van ténylegesen telepítve?	`site-packages/pandas`, importálható

A függőségi tartomány (dependency range) egy verzióhalmazt enged meg. A `pandas>=2.2,<3` feltétel két részből áll: a verzió legyen legalább 2.2, és legyen kisebb 3-nál. Például a 2.3.3 mindkét feltételt teljesíti: $2.3.3 \geq 2.2$ igaz, és $2.3.3 < 3.0$ is igaz. A tartomány azonban önmagában nem mondja meg, hogy éppen 2.3.1-et, 2.3.3-at vagy egy jövőbeli 2.x-et telepítsünk.

A lockfájl (lockfile) ezt a döntést rögzíti. Az `uv lock` parancs feloldja (resolve) az összes tartományt, azaz megkeresi az egymással kompatibilis pontos verziókat, és a teljes függőségi fát kiírja az `uv.lock` fájlba. Egy szemléltető kivonat:

```
# uv.lock, szemléltető kivonat; a pontos szerkezet uv-verziófüggő lehet
version = 1
requires-python = ">=3.12,<3.13"

[[package]]
name = "numpy"
version = "2.5.4"
source = { registry = "https://pypi.org/simple" }

[[package]]
name = "pandas"
version = "2.3.3"
source = { registry = "https://pypi.org/simple" }
dependencies = [
    { name = "numpy" },
    { name = "python-dateutil" },
    { name = "pytz" },
]
```

A `.venv` könyvtár a harmadik réteg: itt már nem előírás, hanem tényleges importálható anyag van. Ez a három réteg eltérhet egymástól. A lock pontos verziója nem garantálja, hogy a `.venv`-ben is az van, ha valaki kézzel beleír a környezetbe; a `pyproject` tartománya pedig akkor is érvényben marad, ha a lock egy régebbi feloldást őriz.

3.2.3 `uv lock`, `uv sync` és a zárolt/fagyasztott mód

A laborban a következő parancssort használjuk:

1. `uv lock`: explicit feloldás. Beolvassa a `pyproject.toml` függőségeit, és létrehozza vagy frissíti az `uv.lock`-ot. Az első futtatás hálózatot igényel.
2. `uv sync --locked`: a lock alapján telepíti a `.venv`-et. A `--locked` kapcsoló előtte ellenőrzi, hogy a lock még összhangban van-e a `pyproject.toml`-lal. A lock egy kivonatot tárol a projekt definíciójáról; ha a `pyproject` változott, a parancs hibával megáll. Így a friss feloldás nem történik meg véletlenül.

3. `uv run --locked --no-sync`: a projekt környezetében futtat egy parancsot. A `--locked` itt is a lock aktualitását követeli meg; a `--no-sync` pedig megakadályozza, hogy a futás maga módosítsa a környezetet. Ez kényelmes a lock rögzítésének utólagos bizonyításához.

A `--frozen` kapcsoló nem erősebb a `--locked`-nál, hanem más célt szolgál: kihagyja az aktualitásvizsgálatot. A parancs a lockot tekinti igazságnak, és nem ellenőrzi, hogy a `pyproject.toml` időközben megváltozott-e. A `--frozen` gyorsabb, és hasznos lehet olyan automatizált környezetben, ahol a lockot már korábban érvényesítettük; a fejlesztői munkamenetben viszont a `--locked` a biztonságosabb választás.

Az `uv lock --check` read-only ellenőrzést végez: 0 kilépési kóddal tér vissza, ha a lock érvényes, és nem nulla kóddal, ha a projekt definíciója megváltozott. A pontos kilépési kód és a hibaszöveg a helyi uv verzió dokumentációjában ellenőrizhető.

3.2.4 Függőségi csoport és opcionális extra

A `pyproject.toml` két, egymástól eltérő szerepű kiegészítő szakaszt ismerhet. A függőségi csoport (dependency group) a projekt belső fejlesztői környezetét szervezi:

```
[dependency-groups]
dev = [
    "pytest>=8,<9",
    "ruff>=0.5,<1",
]
```

Az opcionális extra (optional extra) viszont a projekt mint közzétett csomag fogyasztói számára kínál kiegészítő képességet:

```
[project.optional-dependencies]
plot = [
    "matplotlib>=3.8,<4",
]
```

Az `uv.lock` a csoportokat és az extrákat is rögzíti, de a telepítéskor kiválasztjuk, melyikre van szükség. Az uv a **`dev` csoportot alapértelmezés szerint telepíti**: a `uv sync --locked` önmagában is beveszi, nem kell hozzá `--group dev`. Az alapértelmezett csoportok listáját a `pyproject.toml` `[tool.uv]` szakasza írja felül:

```
[tool.uv]
default-groups = ["dev"]   # ez az uv alapértelmezése is
# default-groups = []      # így a dev csoport nem kerül be automatikusan
```

A csoport kizárására a `--no-dev` (a `--no-group dev` álneve) vagy a `--no-default-groups` szolgál; ez utóbbi minden alapértelmezett csoportot kikapcsol, de egy csoport `--group dev` kapcsolóval újra bevehető. A `--only-group dev` csak a dev csoportot telepíti, a projekt törzsfüggőségeit nem. A `uv sync --locked --group dev` tehát nem azért kell, hogy a dev csoport egyáltalán bekerüljön, hanem hogy a szándékot explicit módon rögzítse. A laborban az alapút a törzsfüggőségeket és az alapértelmezett `dev` csoportot használja; a bővítő fejezetek eszközei külön csoportként vagy extraként állnak rendelkezésre, és az adott fejezetben kapcsoljuk be őket. A pontos kapcsolók a helyi `uv sync --help` kimenetében ellenőrizhetők, mert a CLI alakja verziófüggő.

3.2.5 Teljes minimális munkapélda

Készítsünk egy kis `uv_demo` projektet, amelyen mindhárom réteg és a `--locked` üzemmód kézzel ellenőrizhető.

```
# uv_demo/pyproject.toml
[project]
name = "lock-demo"
version = "0.1.0"
requires-python = ">=3.12,<3.13"
dependencies = [
    "numpy>=1.26,<3",
    "pandas>=2.2,<3",
]

[dependency-groups]
dev = [
    "pytest>=8,<9",
]

[project.optional-dependencies]
plot = [
    "matplotlib>=3.8,<4",
]
```

A környezetjelentéshez egy apró, függőség nélküli scriptet használunk. Ez a 3.1. alfejezetben megismert `sys.executable` és `sys.prefix` szerződést egészíti ki a csomagverziókkal:

```
# uv_demo/check_lock.py
import importlib.metadata as md
import json
import sys

packages = sys.argv[1:]
print(json.dumps({p: md.version(p) for p in packages}, indent=2))
```

Linux/macOS rendszeren, bash munkakörnyezetben a `uv_demo` munkakönyvtárból:

```
# munkakönyvtár: uv_demo
uv lock
uv lock --check
# szemléltető kimenet; exit code 0, ha a lock érvényes
uv sync --locked          # a dev csoport alapértelmezés szerint bekerül
uv run --locked --no-sync python check_lock.py numpy pandas pytest
```

A példa ellenőrzött kimenete (uv 0.13.6, Python 3.12.3, 2026-10-29; a parancs a `uv run --locked --no-sync python check_lock.py numpy pandas pytest`):

```
{
  "numpy": "2.5.4",
  "pandas": "2.3.3",
  "pytest": "8.4.2"
}
```

A verziók az adott nap elérhető csomagjaitól és a tartományoktól függnek; a `pandas>=2.2,<3` tartományban a 2.3.3 a legfrissebb 2.x kiadás, ezért ez a várható érték, de a lockot nem a várakozás, hanem a fájl tartalma ellenőrzi. A `uv sync --locked` ebben a projektben 20 csomagot oldott fel, és a `dev` csoportot külön kapcsoló nélkül telepítette: a `pytest 8.4.2` már a `--group dev` nélküli futás után is jelen volt. A `uv sync --locked --no-default-groups` ezzel szemben eltávolította a pytestet, a numpy és a pandas viszont megmaradt. A saját projektben a verziót mindig az `uv.lock`-ból kell kiolvasni, nem a szövegből.

Az ismétléshez rögzítsük a lock hashét a futás előtt és után. Linux/macOS:

```
shasum -a 256 uv.lock > /tmp/lock_before.txt
uv run --locked --no-sync python check_lock.py pandas
shasum -a 256 uv.lock > /tmp/lock_after.txt
diff /tmp/lock_before.txt /tmp/lock_after.txt && echo "lock unchanged"
```

Windows PowerShellben ugyanez:

```
# munkakönyvtár: uv_demo
$before = (Get-FileHash uv.lock -Algorithm SHA256).Hash
uv run --locked --no-sync python check_lock.py pandas
$after = (Get-FileHash uv.lock -Algorithm SHA256).Hash
$before -eq $after
```

Az elvárt eredmény szemléltető jelleggel `True`, tehát a lock fájl a második futtatás során sem változott.

3.2.6 Eredmény értelmezése és ellenőrző invariant

A munkapéldában három ellenőrző invariantet rögzítünk.

1. **A lock létezik, és a bejegyzései teljesítik a tartományokat.** A `uv.lock` `pandas` bejegyzésének verziója a `>=2.2,<3` intervallumba esik, és a `requires-python` mező megfelel a projekt Python-korlátjának.
2. **A telepített környezet a lockot tükrözi.** A `check_lock.py` kimenete és a lock megfelelő bejegyzése megegyezik. Ha nem, a `uv sync --locked` újratelepíti a lock szerinti verziót.
3. **A lock a második `--locked` futás alatt változatlan.** A hash-egyenlőség azt igazolja, hogy a parancs nem végzett csendes újrafeloldást.

A `uv run --locked --no-sync` tehát nem a környezet tartalmát hasonlítja a lockhoz, hanem a projekt definícióját. A `--no-sync` arra szolgál, hogy a futás maga ne módosítsa a `.venv`-et. Így a munkamenet a következő sémára épül: előbb explicit `uv lock`, utána `uv sync --locked`, végül `uv run --locked --no-sync` a tényleges munkához.

Lockfájl és hordozhatóság. A lockfájl a függőségfeloldás eredményét rögzíti, de nem helyettesíti az interpreter-, operációsrendszer- és architektúra-követelmények leírását. A `uv sync --locked` sikerét a jelen dokumentációhoz ténylegesen használt `uv --version` mellett kell igazolni; egy generált lockfájl jelenléte nem önmagában futtatási bizonyíték.

Tisztasági kontroll: új virtuális környezetben telepítés rögzített függőségekkel, a projekthez tartozó importok füsttesztje és az eltérés dokumentálása. A rövid távú gyakorlati parancsot mindig a verzióhoz tartozó hivatalos súgóval kell egyeztetni.

3.2.7 Feladatok

1. feladat — Értelmezés. Egy projekt `pyproject.toml`-jában `dependencies = ["pandas>=2.2,<3"]` áll, az `uv.lock`-ban `pandas==2.3.3`, és a `uv run --locked --no-sync python check_lock.py pandas` parancs kimenete `{"pandas": "2.3.1"}`. Mit jelent a kimenet? Melyik parancs állítja helyre a lock és a környezet egyezését? Miért nem elegendő ehhez a `pyproject.toml` tartománya?

2. feladat — Implementálás a labon. A mellékelt `lab` mappa nem tartalmaz `uv.lock` fájlt, de a `lab/reference/uv.lock` referenciafeloldás igen. A `lab/pyproject.toml` alapján

készítsd el a saját lockot, hasonlítsd össze a referenciával (azonos verzióhalmaz-e; eltérés esetén dokumentáld az okot), telepítsd a környezetet `--locked` módban, majd futtasd le a fenti `check_lock.py` scriptet a lab három alapvető csomagjára: `numpy`, `pandas`, `scikit-learn`. A scriptet a `lab/scripts/check_lock.py` útvonalon helyezd el, ha még nincs ott. Rögzítsd a lock hashét a futás előtt és után; az elvárt eredmény változatlan hash.

3. feladat — Hibadiagnózis. Egy friss klónban a tanuló a `uv run --locked --no-sync python scripts/check_lock.py pandas` parancsot adta ki, és a következő hibát kapta: `error: No uv.lock found`. Mi a hiba oka? Mi a helyes parancssor a hibából való kilábaláshoz? Indokold meg, miért nem a `uv run --frozen` a megoldás.

4. feladat — Megváltozott feltétel. A fenti `uv_demo` miniprojekt `pyproject.toml`-jában add hozzá az alábbi opcionális extrát:

```
[project.optional-dependencies]
plot = [
    "matplotlib>=3.8,<4",
]
```

Friss lock készítése nélkül add ki a `uv run --locked --no-sync python check_lock.py pandas` parancsot. Milyen kimenet várható, és miért? Ezután add meg a helyes parancssort, és ellenőrizd, hogy a `matplotlib` is importálható.

3.2.8 Megoldások és indoklás

1. feladat. A kimenet azt mutatja, hogy a `.venv` eltér a locktól: a lock 2.3.3-at ír elő, a telepített környezetben 2.3.1 van. A helyreállítás `uv sync --locked`, mert ez a locknak megfelelő verziót telepíti. A `pyproject` tartománya önmagában nem elegendő, mert a 2.3.1 is teljesíti a `>=2.2,<3` feltételt; a lock az, amely a pontos verziót rögzíti.

2. feladat. A lab `pyproject.toml`-jában ellenőrizd, hogy a `dependencies` szakasz tartalmazza a `numpy`, `pandas`, `scikit-learn` csomagokat; a mellékelt lab szerződése szerint a `starter_experiment.py` ezeket vagy ezek egy részhalmazát használja. Ha a script még nincs ott:

```
# lab/scripts/check_lock.py
import importlib.metadata as md
import json
import sys

packages = sys.argv[1:]
print(json.dumps({p: md.version(p) for p in packages}, indent=2))
```

Futtatás a `lab` munkakönyvtárból, Linux/macOS bash alatt:

```
cd lab
uv lock
uv lock --check
diff reference/uv.lock uv.lock && echo "egyezik a referenciával"
uv sync --locked
shasum -a 256 uv.lock > /tmp/before.txt
uv run --locked --no-sync python scripts/check_lock.py numpy pandas scikit-learn
shasum -a 256 uv.lock > /tmp/after.txt
diff /tmp/before.txt /tmp/after.txt && echo "lock unchanged"
```

A `diff reference/uv.lock uv.lock` csak akkor ad nulla kilépési kódot, ha a saját feloldás bájtazonos a referenciával; ha közben új csomagverzió jelent meg, a különbséget dokumentálni kell, és a saját lock lesz a mérvadó. A referencialock használatához `cp reference/uv.lock uv.lock` után `uv lock --check` és `uv sync --locked` futtatható.

Windows PowerShell alatt a hash rögzítése:

```
cd lab
uv lock
uv sync --locked
$before = (Get-FileHash uv.lock -Algorithm SHA256).Hash
uv run --locked --no-sync python scripts/check_lock.py numpy pandas scikit-learn
$after = (Get-FileHash uv.lock -Algorithm SHA256).Hash
$before -eq $after
```

A lab fájl szerinti pontos kimenet a helyi gépen mért verzióktól függ, ezért azt a tanulónak kell rögzítenie. A lényegi ellenőrzés a hash-egyenlőség, nem a kézzel kitalált verziószám.

3. feladat. A hiba oka az, hogy a friss klónban nincs `uv.lock`; a `--locked` mód nem hozza létre, hanem megköveteli a lock meglétét. A helyes sorrend:

```
uv lock
uv sync --locked
uv run --locked --no-sync python scripts/check_lock.py pandas
```

A `uv run --frozen` nem oldd meg a hibát, mert a fagyasztott mód sem hoz létre hiányzó lockot; ráadásul kihagyja a `pyproject` és a lock egyezésének ellenőrzését, így csendes elavuláshoz vezethet.

4. feladat. A `pyproject.toml` módosítása után a `uv run --locked --no-sync` parancs hibával áll meg, mert a lock korábban rögzített kivonata nem egyezik az új projekt definíciójával. A pontos hibaszöveg a helyi uv kiadásban ellenőrizhető; a lényeg az, hogy a `--locked` megakadályozza a szándéktalan újrafeloldást. A helyes sorrend:

```
# munkakönyvtár: uv_demo
uv lock
uv sync --locked --extra plot
uv run --locked --no-sync python check_lock.py pandas matplotlib
```

Az ellenőrző invariant: az új lockban a `plot` extra függőségei is megjelennek, a lock hash-e megváltozik, és a `matplotlib` verzió importálhatóvá válik.

3.2.9 Gyakori tévedés ellenpéldával

Az egyik jellegzetes hiba az, hogy a tanuló a friss laborban `uv run --locked` parancsot ad ki, mert úgy gondolja, a `--locked` ellenőrzi, sőt létrehozza a lockot. Az ellenpélda: a `lab/pyproject.toml` létezik, de a `uv.lock` még nem; a `--locked` parancs hibát ad, mert nincs mit ellenőriznie. A csomag tehát nem állítja, hogy egy még nem létező lockfájlt már ellenőrzött.

A másik gyakori félreértés: „a `--frozen` erősebb, mert teljesen fagyasztott”. Az ellenpélda ugyanez a projekt, de most a lock már létezik, és a `pyproject.toml`-t megváltoztatjuk. A `uv run --locked` hibát ad; a `uv run --frozen` viszont a régi lockkal tovább dolgozik, és nem jelzi, hogy a projekt definíciója már mást ír elő. A `--frozen` tehát nem erősebb garancia, hanem az aktualitásvizsgálat kihagyása. Reprodukálható átadásnál a `--locked` ellenőrzött friss lockot követel, a `--frozen` csak a már érvényesített lock gyors végrehajtására való.

Önellenőrző állítások: „a lock a telepített környezet pontos listája” hamis, mert a `.venv` ettől függetlenül eltérhet; „a tartomány a lockban van” hamis, mert a tartomány a `pyproject`-ben,

a pontos verzió a lockban van. A 2.2. alfejezetből ismert kilépési kód és a 3.1. alfejezetből ismert `sys.prefix`-ellenőrzés együtt adja a rétegek bizonyítékát: a parancsok exit code-ja a lock érvényességét, a környezetjelentés a telepített anyag valódi forrását mutatja.

3.2.10 Rövid összefoglaló

A `pyproject.toml` tartománya azt mondja meg, mi elfogadható; az `uv.lock` azt, hogy az adott feloldáskor pontosan mit választottunk; a `.venv` azt, hogy mi van ténylegesen telepítve. A `uv lock` explicit feloldást készít, a `uv sync --locked` a lockot és a környezetet hozza összhangba, a `uv run --locked --no-sync` pedig friss feloldás nélkül futtat. A `--frozen` kihagyja az aktualitásvizsgálatot, ezért nem erősebb, hanem más üzemmód. A következő alfejezetben a bináris csomagok, a BLAS és a conda-döntés kerül sorra: ott az eddigi lockolt függőségek mellett a natív futtatási környezet kérdéseit vizsgáljuk.

3.3 Bináris csomag, BLAS és conda-döntés

Belépő · szint: alap · idő: 45 perc · eszköz: uv, Python 3.12

Előfeltétel: 3.2. alfejezet · Célkészség: Bináris csomag, BLAS és conda-döntés · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

3.3.1 Cél és konkrét helyzet

A Northstar labor `pyproject.toml`-ja NumPy-t és SciPy-t ír elő. Három különböző gépen három eltérő telepítési hibát látunk, mielőtt bármilyen elemzés elindulhatna:

1. `ERROR: No matching distribution found for numpy==2.1.1`
2. `Failed building wheel for numpy; error: command 'gcc' failed with exit code 1`
3. `ImportError: libcblas.so.3: cannot open shared object file: No such file or directory`

A megfigyelési egység egy telepítési kísérlet: a teljes hibakimenet, a használt Python-verzió, az operációs rendszer és a processzorarchitektúra. A cél annak eldöntése, hogy az adott hiba wheel-, rendszerkönyvtár- vagy solverprobléma-e, és hogy uv/pip vagy conda/mamba környezetet érdemes-e választani. Az ellenőrzés feltétele: a telepítés után a Python sikeresen importálja a NumPy-t, és egy kézzel ellenőrizhető mátrixszorzás a várt eredményt adja.

Induló feladat: a három üzenet közül melyikhez melyik első vizsgálati lépés tartozik – (a) Python-verzió ellenőrzése, (b) C/Fortran fordító megléte, (c) a megosztott könyvtárak dinamikus feloldása? A megoldást a fejezet végén adjuk meg.

3.3.2 Wheel és forrásfordítás

A `wheel` (kerék) a Python-csomagok előre lefordított bináris formátuma. A `.whl` fájlban a csomag készre fordított kódja és metaadatai vannak; telepítéskor a csomagkezelő csak a megfelelő helyre csomagolja ki őket. Ennek akkor van jelentősége, ha a csomag natív, C, C++ vagy Fortran kódon alapul, és a telepítőgépen nincs fordító vagy a szükséges rendszerfejléc.

Egy wheel-fájlnév platformcímkéi pontosan megmondják, milyen környezethez készült. Például:

`numpy-2.1.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl`

- `cp312`: CPython 3.12 interpreterhez készült;
- `manylinux_2_17_x86_64`: x86-64 processzorú Linuxon futtatható, a `manylinux` egy szabványosított bináris-kompatibilitási profil;
- a kettős `manylinux` címke több régebbi és újabb Linux-disztribúcióval való kompatibilitást jelez.

Ha a wheel nem elérhető az adott platformra, a pip vagy uv megpróbálhatja az `sdist`-ből (source distribution) felépíteni a csomagot. Ez a **forrásfordítás** (source build): a telepítőgépen fut le a fordító, amihez megfelelő C- és gyakran Fortran-fordítóra, a függő rendszerkönyvtárak fejléceire és build-eszközökre van szükség. A második hibaüzenetben szereplő `gcc` az egyik ilyen fordító; ha hiányzik vagy nem kompatibilis, a build meghiúsul.

3.3.3 NumPy/SciPy natív függősége és a BLAS szerepe

A NumPy és a SciPy nagy része C-ben és Fortranban írt, nagy teljesítményű numerikus rutinokat hív meg. A lineáris algebrai alapműveletek szabványos felületét a **BLAS** (Basic Linear Algebra Subprograms) adja, amelynek három szintje van: vektor–vektor, mátrix–vektor és mátrix–mátrix műveletek. A NumPy például a mátrixszorzást általában egy külső BLAS-megvalósításra, például az OpenBLAS-ra vagy az Intel MKL-re bízza. A SciPy ehhez még a LAPACK-ot is használja, amely a lineáris egyenletrendszerek megoldására, sajátérték-számításra és hasonlókra épül.

Ennek matematikai hídja egyszerű: a mátrixszorzás mint könyvtári művelet.

Legyen $A \in \mathbb{R}^{m\times k}$ és $B \in \mathbb{R}^{k\times n}$. A szorzat $C = AB \in \mathbb{R}^{m\times n}$, ahol minden (i, j) indexre

$$C_{ij} = \sum_{l=1}^{k} A_{il} B_{lj}.$$

Itt m az A sorainak száma, k a közös belső dimenzió, n a B oszlopainak száma. A példában dimenzió nélküli, tiszta számokkal dolgozunk; valós adatelemzésben a mértékegységet a mátrix elemeinek jelentése hordozza, ezért a szorzatnál mindig külön kell megnevezni, mit jelentenek a bemenő oszlopok.

Kis kézi kontroll:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Ekkor $m = k = n = 2$, és

$$C = \begin{pmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

A NumPy-ban a `@` operátor ezt a BLAS-hívást takarja. A Python-kód tehát nem maga írja meg a belső ciklusokat; a helyes működés a telepített natív könyvtártól függ. Ha a BLAS hiányzik, rosszul linkelt vagy nem kompatibilis az ABI-val, akkor nem az algoritmus hibás, hanem a futtatókörnyezet.

3.3.4 conda/mamba és a csatornák

A `conda` és a gyorsabb solverű `mamba` nemcsak Python-csomagokat, hanem a Pythont és a natív könyvtárakat is csomagként kezelik. Egy `conda create -n labor python=3.12 numpy scipy` parancs a Python-interpretert, a NumPy-t, a SciPy-t és az általuk igényelt BLAS/LAPACK-megvalósítást is binárisan, előre lefordítva telepíti a környezetbe. Ezért a conda különösen akkor előnyös, ha a PyPI-n nincs megfelelő wheel, vagy ha a rendszerfordító hiányzik.

A conda-csomagok **csatornákból** (channel) érkeznek. A `defaults` az Anaconda által karbantartott, míg a `conda-forge` közösségi csatorna, amely rendszerint frissebb és szélesebb körű buildeket kínál. A tanulási főútban a csatornák pontos verzióját nem kötjük meg e fejezethez; a választásnál az a kérdés, hogy a projekt által igényelt csomagok és a Python-verzió konzisztensen elérhetők-e.

Példa conda/mamba munkamenet (Linux/macOS, a projektgyökérből):

```
mamba create -n northstar python=3.12 numpy scipy -c conda-forge
mamba activate northstar
python -c "import numpy as np; print(np.__version__); print(np.show_config())"
```

A `np.show_config()` kimenete szemléltető lehet; egy lehetséges részlet:

```
blas_mkl_info:
  NOT AVAILABLE
blas_opt_info:
    libraries = ['openblas', 'openblas']
    library_dirs = ['/home/user/miniforge3/envs/northstar/lib']
    language = f77
```

Ez azt mutatja, hogy a NumPy az OpenBLAS-t használja. A tényleges kimenet gép- és verziófüggő, ezért ezt csak a saját gépen futtatva érdemes ellenőrizni.

A conda és mamba parancsok szintaxisa a mindennapi használatban nagyrészt megegyezik; a mamba gyorsabb feloldást ígér. A könyvben conda/mamba néven hivatkozunk rájuk, mert a döntési szerep azonos.

3.3.5 Keverés kockázata

A legnagyobb telepítési csapda a pip/uv és a conda tetszőleges sorrendű keverése. Ennek oka, hogy a conda saját solverrel és saját függőségi gráfbal dolgozik, míg a pip/uv a PyPI wheeljeiből építkezik. A két rendszer nem látja egymás döntéseit.

Ellenpélda:

```
conda create -n mixed python=3.12 numpy
conda activate mixed
pip install scipy
conda install pandas
```

A `pip install scipy` telepíthet egy olyan SciPy buildet, amely más NumPy-ABI-ra épül, mint amit a conda telepített. Ezután a `conda install pandas` a conda solver döntése alapján kicserélheti vagy downgrade-elheti a NumPy-t, miközben a pip által telepített SciPy ugyanott marad. Végeredmény: inkonzisztens bináris környezet, amelyben egy `import` vagy egy BLAS-hívás hibát ad. A reprodukálhatósági stratégia ezért az: egy környezetben vagy conda/mamba, vagy uv/pip kezelje a Python-csomagokat; a kettőt ne váltogassuk ugyanazon a környezeten.

3.3.6 Teljes minimális példa: három hibaüzenet vizsgálata

Az alábbi három üzenet **szemléltető példa**, nem valós futási napló. A diagnosztikai lépések azonban konkrétak és futtathatók.

3.3.6.1 1. üzenet – nincs megfelelő wheel

```
ERROR: Could not find a version that satisfies the requirement numpy==2.1.1
ERROR: No matching distribution found for numpy==2.1.1
```

Első vizsgálati lépés: a Python-verzió és a platform ellenőrzése:

```
python -V
python -c "import platform; print(platform.platform(), platform.machine())"
```

Ha a kimenet például Python 3.13-at mutat, de a kért NumPy-kiadáshoz csak 3.12-es wheel létezik, akkor a hiba nem a csomag hibája, hanem verzióillesztési probléma. A megoldás vagy a Python-verzió rögzítése a projektben (a tanulási alap CPython 3.12), vagy a NumPy-verzió tartományának módosítása a `pyproject.toml`-ban.

3.3.6.2 2. üzenet – forrásfordítás hiányzó toolchainnel

```
Failed building wheel for numpy
error: command 'gcc' failed with exit code 1
```

Itt a telepítő nem talált kész wheel-t, ezért megpróbálta az sdist-et lefordítani, de a rendszerfordító hiányzik vagy hibás. Első vizsgálati lépés a toolchain megléte:

```
gcc --version
gfortran --version || echo "gfortran nincs telepítve"
```

Ha a `gfortran` hiányzik, a NumPy és a SciPy forrásfordítása a legtöbb platformon nem fog menni. Ebben az esetben vagy a megfelelő wheel-t kell megkeresni (például Python-verzió emelésével/süllyesztésével), vagy conda/mamba környezetre váltani, amely binárisan hozza a csomagot és a BLAS-t.

3.3.6.3 3. üzenet – hiányzó megosztott könyvtár

```
ImportError: libcblas.so.3: cannot open shared object file: No such file or directory
```

Ez a hiba akkor jelentkezik, amikor a Python már elindult, de a betöltött natív modul egy megosztott C/Fortran könyvtárat keres, amely nincs a dinamikus keresési útvonalon. Első vizsgálati lépés a könyvtár-függőségek listázása Linuxon:

```
ldd "$(python -c 'import numpy; print(numpy.__file__)')"
```

Vagy magának a NumPy-nak a konfigurációját nézzük:

```
python -c "import numpy; numpy.show_config()"
```

Ha a `libcblas` hiányzik a rendszerből, a leggyakoribb ok: a NumPy-t egy olyan wheel-ből telepítettük, amely rendszer-BLAS-t feltételez, vagy egy másik környezetből másoltuk át. A javítás vagy a megfelelő, önálló OpenBLAS-t tartalmazó wheel telepítése, vagy conda/mamba használata.

3.3.6.4 Reprodukálható hibalabor: három elkülöníthető állapot

A fenti három üzenet elméleti leírása önmagában nem teszt. A `lab/abi_lab/build_cases.py` három, hálózat nélkül előállítható állapotot hoz létre, és `manifest.json`-ba rögzíti a parancsot, a kilépési kódot és a stderr mintáját:

Állapot	Kiváltó ok	stderr-minta	Kilépés
`wheel_mismatch`	helyi wheelhouse-ban csak `cp39` wheel van, a futtató Python 3.12	`No matching distribution found for demo-pkg==1.0.0`	1
`abi_mismatch`	a `.so` nem definiál `PyInit_*` függvényt	`dynamic module does not define module export function`	1
`missing_library`	a extension `NEEDED` bejegyzése `libmissing.so.3`, de a könyvtár törölve	`libmissing.so.3: cannot open shared object file`	1

A wheel-eset szándékosan helyi: a `demo_pkg-1.0.0-cp39-cp39-linux_x86_64.whl` csak Python 3.9-hez kompatibilis, ezért a 3.12-es pip `--no-index` mellett sem talál megfelelő disztribúciót. A második eset egy `PyInit_abi_demo` nélküli `.so`; a harmadik egy valódi `NEEDED` függőséggel rendelkező extension, amelynek a könyvtárát a linkelés után töröljük. A mért referencia ezen a Linux x86-64 gépen: Python 3.12.3, gcc 13.3.0, glibc 2.39; a minták a konténerben is ugyanazok, csak az útvonalak és a verziók eltérnek.

```
python3 lab/abi_lab/build_cases.py --output dist/abi-lab
python3 -m json.tool dist/abi-lab/manifest.json | head -40
```

Konténerben, rögzített Python 3.12.3 és Debian bookworm alapon:

```
docker build -t northstar-abi-lab lab/abi_lab
docker run --rm -v "$PWD/dist/abi-lab-container:/work/out" northstar-abi-lab
```

A konténer a `Dockerfile`-ban rögzített `python:3.12.3-slim-bookworm` képet használja; a három minta és a kilépési kód ugyanaz, a `manifest.json` a konténeren belüli útvonalakat rögzíti. A `tests/test_abi_lab.py` a három eset kilépési kódját és stderr-mintáját ellenőrzi; ha a helyi gépen nincs `gcc` vagy `pip`, a teszt a hiányzó eszközt jelzi, nem állít sikeres futtatást.

3.3.6.5 uv és conda felelősségi körök összehasonlítása

Felelősség	uv + venv	conda/mamba
Python-interpreter kiválasztása	A `pyproject.toml`-ban megadott verziót a uv letöltheti és kezelheti; a környezet Pythonja a `.venv`-ben van.	A Pythont maga a conda telepíti csomagként a környezetbe.

Felelősség	uv + venv	conda/mamba
Natív könyvtárak (BLAS/LAPACK)	A PyPI-wheel-ek általában tartalmazzák az OpenBLAS-t; ha nincs wheel, forrásfordítás kell.	A conda-csatorna binárisan szállítja a BLAS-t és a csomagokat együtt.
Függőségrögzítés	`uv.lock` a Python-csomagokat pontosan rögzíti; a rendszerkönyvtárak nem részei.	`environment.yml` vagy `conda-lock` a conda-csomagokat rögzíti; a rendszerkönyvtárak ugyanúgy nem.
Operációs rendszer függősége	Windows/macOS/Linux wheel-ek; a rendszer-C lib verziója lehet korlát.	A conda-csomagok saját előtaggal dolgoznak, így kevesebb rendszerfüggőség.

A táblázat lényege: az uv/pip a Python-ökoszisztéma natív eszköze, és a lockkal a Python-függőségeket rögzíti; a conda/mamba a Python mellett a natív könyvtárakat is csomagkezelő alá vonja. A 3.2. alfejezetben tanult `uv.lock` tehát a conda esetében nem ugyanaz a fogalom: a conda-környezetben a `conda-lock` vagy a `environment.yml` pontosítására van szükség.

Most futtassuk le a kis mátrixszorzás-kontrollt egy uv-környezetben. Az alábbi kódot nem futtattuk jelen írásban; a kimenet **szemléltető**, elméletileg várt érték.

```
import numpy as np

A = np.array([[1.0, 2.0], [3.0, 4.0]], dtype=np.float64)  # shape (2, 2)
B = np.array([[5.0, 6.0], [7.0, 8.0]], dtype=np.float64)  # shape (2, 2)
C = A @ B                                                  # shape (2, 2)

expected = np.array([[19.0, 22.0], [43.0, 50.0]], dtype=np.float64)
print(C)
print("shape:", C.shape, "dtype:", C.dtype)
print("allclose:", np.allclose(C, expected, rtol=1e-12, atol=1e-14))
```

Elméletileg várt szemléltető kimenet:

```
[[19. 22.]
 [43. 50.]]
shape: (2, 2) dtype: float64
allclose: True
```

Futtatás a lab projektben, Linux/macOS, a projektgyökérből:

```
uv sync --locked
uv run --locked --no-sync python matrix_check.py
```

A `--locked --no-sync` kombináció a 3.2. alfejezetben tanult módon garantálja, hogy a lockolt verziókat használjuk, friss feloldás nélkül.

3.3.7 Eredmény értelmezése

A mátrixszorzás ellenőrzése három invariant teljesülését jelenti:

1. **Shape-invariant**: ha $A \in \mathbb{R}^{2\times 2}$ és $B \in \mathbb{R}^{2\times 2}$, akkor $C \in \mathbb{R}^{2\times 2}$. A kimenet shape-je `(2, 2)`.

2. **Dtype-invariant**: a példában minden bemenet `float64`, ezért a kimenet is `float64`; vegyes típusnál a NumPy saját promóciós szabályai szerint alakul, amit külön ellenőrizni kell.
3. **Numerikus tolerancia**: a két mátrix minden eleme egész pontossággal egyezik a kézi kontrollal; lebegőpontos műveleteknél az összehasonlításhoz relatív toleranciát (rtol $= 10^{-12}$) és abszolút küszöböt (atol $= 10^{-14}$) használunk. Több lépéses, nagy dimenziós műveleteknél ez a tolerancia szigorodhat; a lényeg, hogy soha ne hasonlítsunk lebegőpontos eredményeket `==` művelettel.

Ha a fenti ellenőrzések teljesülnek, a NumPy BLAS-hívása működőképes. Ha `allclose` hamisat ad, akkor nem feltétlenül a BLAS hibás: először a saját `expected` mátrixunkat kell ellenőrizni, majd a dtype-ot és a shape-eket.

Numerikus bináris kompatibilitás. A Python-verzió önmagában nem írja le a numerikus stack futtatókörnyezetét. A wheel/conda csomagoknak CPU-architektúrához, rendszerkönyvtárhoz és adott esetben BLAS- vagy GPU-runtime-hoz is illeszkedniük kell. A csomag telepítése és egy egyszerű numerikus művelet sikeres lefutása két külön ellenőrzési pont.

A BLAS-szálak száma befolyásolhatja a teljesítményt és a reprodukálható időmérést; ezért benchmarknál a szálkeretet (például `OPENBLAS_NUM_THREADS`) és a platformot dokumentálni kell. Ne sugalljuk, hogy a conda minden bináris problémára automatikusan jobb megoldás: ez konkrét platform- és csomagfüggő döntés.

3.3.8 Feladatok

1. **Értelmezés.** A wheel-fájl neve `numpy-2.1.1-cp312-cp312-win_amd64.whl`. Telepíthető-e ez CPython 3.13-ra Linux x86-64 gépen? Indokolj a fájlnév címkéi alapján.
2. **Megvalósítás.** Írj egy rövid Python-szkriptet, amely kiírja, hogy a NumPy milyen BLAS/LAPACK-at talál (`numpy.show_config()`), majd végez el egy 3×2-es mátrix és egy 2×4-es mátrix szorzását. Add meg a várt eredmény shape-jét, és kézzel számold ki az eredmény első sorát, ha az első mátrix sorai rendre $[1, 2]$, $[3, 4]$, $[5, 6]$, a második mátrix oszlopai pedig $[1, 0]$, $[0, 1]$, $[2, 3]$, $[1, 1]$.
3. **Hibakeresés.** Egy gépen az `uv sync --locked` sikeres, de az `import numpy` ezt adja: `ImportError: /lib/x86_64-linux-gnu/libm.so.6: version GLIBC_2.28 not found`. Milyen típusú problémáról van szó? Milyen következő lépést javasolsz, és miért nem a `uv` lockfájl hibája?
4. **Megváltozott feltétel.** A telepítőgépen nincs internet-hozzáférés a futtatáskor, viszont a csomagokat előre be kell csomagolni. Hogyan változik a döntés uv és conda/mamba között? Adj egy konkrét, offline munkamenet-vázlatot, és jelöld meg, mi az, ami mindkét úton kívül marad.

3.3.9 Megoldások és indoklás

1. **Nem telepíthető.** A `cp312` címke CPython 3.12-höz kötött, a `win_amd64` pedig 64 bites Windowsra. CPython 3.13 Linux x86-64-hez más wheel, például `cp313-cp313-manylinux...x86_64.whl` kellene. Ha ilyen nem létezik a kért verzióhoz, akkor vagy Python-verziót kell igazítani, vagy forrást fordítani, vagy conda/mamba csatornán keresni binárist. Ez a válasz a címkék pontos olvasását ellenőrzi.
2. **Várható shape:** $(3, 4)$**.** Az első sor kiszámítása: az A első sora $[1, 2]$. A szorzat első sora:

- $j = 1$: $1 \cdot 1 + 2 \cdot 0 = 1$
- $j = 2$: $1 \cdot 0 + 2 \cdot 1 = 2$
- $j = 3$: $1 \cdot 2 + 2 \cdot 3 = 8$
- $j = 4$: $1 \cdot 1 + 2 \cdot 1 = 3$

Tehát a várt első sor $[1, 2, 8, 3]$. A szkriptben az `A @ B` kimenet shape-jét `(3, 4)`-ként ellenőrizzük. Általánosan: ha A shape-je $(3, 2)$ és B shape-je $(2, 4)$, akkor a belső 2 megegyezik, a kimenet pedig $(3, 4)$. A BLAS-konfiguráció kiírása a `numpy.show_config()` függvénnyel történik; annak tartalma gépenként eltér, ezért itt csak az ellenőrzés módját adjuk meg, konkrét kimenetet nem garantálunk.

3. **A GLIBC-verzió probléma.** A hiba azt jelenti, hogy a NumPy wheel egy olyan Linux-disztribúcióra készült, amely újabb `glibc`-t feltételez, mint ami a gépen van. Ez rendszerkönyvtár-kompatibilitási probléma, nem solverprobléma. Az `uv.lock` a Python-csomagok verzióját rögzíti; a wheel platformkompatibilitása a `manylinux` címke szintjén múlik. A következő lépés: olyan régebbi manylinux-wheelű NumPy-verziót választani, amely a régebbi glibc-vel is működik, vagy conda/mamba-ra váltani, mert a conda-csomagok saját előtaggal, a rendszer glibc-jétől függetlenebbül telepíthetők.

4. **Offline megoldás.** Mindkét úthoz elő kell készíteni a letöltéseket egy internetes gépen. Uv/pip esetén a wheel-eket töltsük le egy könyvtárba (`pip download` vagy `uv pip download`, itt a parancs neve verziófüggő lehet, ezért a konkrét hívás előtt a helyi `uv --help` ellenőrzendő), majd a célgépen ebből telepítsünk. Conda/mamba esetén a `conda pack` vagy egy helyi csatorna-mirror használható. Ami mindkét úton kívül marad: a rendszer szintű C könyvtárak (például a glibc) és az operációs rendszer frissítései; ezeket a csomagkezelő egyik eszköz sem viszi magával, ezért offline ismert, azonos OS-t kell feltételezni. A lock mindkét esetben csak a saját csomagkörére vonatkozik.

3.3.10 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés az, hogy a pip és a conda keverése „rugalmasság”, pedig nem reprodukálhatósági stratégia. Ellenpélda: egy csapattag `conda`-val telepíti a NumPy-t, majd `pip`-pel telepíti a projektet, mert a `pyproject.toml`-ból szeretne dolgozni. A pip wheel-je felülírja a conda által telepített NumPy egyes fájljait, más conda-csomagok (SciPy, statsmodels) viszont az eredeti conda-verzióhoz lettek lefordítva. Egy mátrixszorzás még lehet, hogy fut, de egy ritkább SciPy-művelet `undefined symbol` hibát dob. A környezet reprodukálhatatlan, mert a konkrét felülírási sorrend nem dokumentált. A helyes stratégia: egy környezet = egy csomagkezelő, és ezt a 2.3. alfejezetben tanult Git-konfigurációban, a `README`-ben rögzítjük.

Önellenőrzés. A 3.2 lockolt függőségei most már kiegészültek a bináris kérdéssel: a `uv.lock` a Python-csomagok pontos verzióját adja, de nem tartalmazza a BLAS bináris tartalmát. Ha egy másik gépen nem ugyanaz a rendszer glibc-je, a lock önmagában nem garantál futást. Ezért a reprodukálhatósági nyilatkozatban külön kell nevezni a Python-csomag-zárat, a natív könyvtárakat és az operációs rendszert.

3.3.11 Rövid összefoglaló

A wheel előre lefordított bináris, a forrásfordítás pedig fordítóeszközt igényel a gépen. A NumPy és SciPy BLAS/LAPACK könyvtárakat hív, ezért a mátrixművelet nem pusztán Python-számítás, hanem natív futtatókörnyezet. A conda/mamba a Pythont és a natív könyvtárakat is csomagkezelő alá vonja, a csatorna határozza meg a bináris forrást. Az uv/pip és a conda keverése inkonzisztens ABI-hoz vezet, ezért egy környezetben egy csomagkezelő működjön. A következő

alfejezetben a Jupyter-kernel regisztrációját vizsgáljuk: ott az interpreter, a csomagkészlet és a notebook futásidejének összerendelése kerül középpontba.

3.4 Kernelregisztráció és tiszta újrafuttatás

Belépő · szint: alap · idő: 45 perc · eszköz: uv, Python 3.12

Előfeltétel: 3.3. alfejezet · Célkészség: Kernelregisztráció és tiszta újrafuttatás · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

3.4.1 Cél és konkrét helyzet

A 3.3 után a lab projektnek már van feloldott függőségi környezete, de a Jupyter-notebook még nincs összekötve ezzel a környezettel. Az a feladat, hogy a notebook ne egy bizonytalan, korábbi Python-telepítést használjon, hanem pontosan azt az interpretert és csomagkészletet, amelyet a terminálban a `uv run` elindít. A bemenet a lab könyvtár `pyproject.toml`-ja, a `uv sync --locked` után létrejött `.venv` és egy telepített JupyterLab. A megfigyelési egység egy notebook-kernel indítása: ugyanabból a projektből többször is indítható, ezért ismételhetően ellenőrizhető. A cél: a notebook és a CLI ugyanazt a Python-útvonalat és ugyanazokat a csomagverziókat jelentse, és a „Restart and Run All” után lineárisan lefusson. A hibás állapotra vonatkozó előzetes becslés: ha a notebook egy globális vagy másik projekt kernelét használja, a notebookból kiírt `sys.executable` eltér a `.venv`-beli Python-útvonaltól, és az importelt verziók sem egyeznek a lock szerinti környezettel.

3.4.2 Fogalmi magyarázat

A JupyterLab (böngészőben futó munkakörnyezet) csak a notebook-dokumentumot, `.ipynb`-t szerkeszti és jeleníti meg. A Python-kódot nem a JupyterLab, hanem egy külön kernel folyamat hajtja végre. A Jupyter ezt a kernel folyamatot a Jupyter kernel protokollon keresztül éri el; a Python oldali implementáció az **ipykernel** (interactive Python kernel). A JupyterLab tehát elküldi a cella kódját a kernelnek, a kernel végrehajtja, és az eredményt visszaküldi a felületnek. A cellakimenet a `.ipynb`-ben meg is marad, de ez a kimenet csak az adott kernelállapot múltját rögzíti — önmagában nem bizonyítja, hogy a jelenlegi kód a jelenlegi környezetben sikeresen lefutott.

A **kernelspec** a kernel regisztrációs bejegyzése: egy rövid JSON-fájl, amely leírja, hogy a Jupyter milyen parancsot indítson az adott kernel elindításakor. A `--name` a belső azonosító, a `--display-name` pedig a Jupyter felületén látható felirat. A két név különbözhet; a regisztrációt követően a felület a megjelenített nevet, a parancsok és a fájlok a belső nevet használják. Egy tipikus kernelspec a következő alakú:

```
{
  "argv": [
    "/felhasznalo/northstar/lab/.venv/bin/python",
    "-m",
    "ipykernel_launcher",
    "-f",
    "{connection_file}"
  ],
  "display_name": "Northstar labs (Python 3.12)",
  "language": "python"
}
```

Az `argv` lista első eleme a tényleges Python-interpreter. A `{connection_file}` helyére a Jupyter a kernel indításakor egy kapcsolati fájl útvonalát írja, amelyen a notebook és a kernel kommunikál. Ha az `argv[0]` a lab `.venv`-jére mutat, a notebook a projekt környezetét használja; ha egy globális `python` bejegyzésre, akkor nem. Ezért a regisztrációt nem szabad globális `python -m ipykernel install` paranccsal elvégezni, mert az a globális interpretert rögzíti. A `uv run python -m ipykernel install` viszont a projekt `.venv` Pythonját indítja, és az `ipykernel` installáló ezt az abszolút útvonalat írja be a kernelspecbe.

A **Restart and Run All** a notebook tiszta újrafuttatása: a kernel leáll, a Jupyter a kernelspecből vadonatúj folyamatot indít, majd minden cellát felülről lefelé végrehajt. Ennek fontos része, hogy a cellák között rejtett összefüggések is felszínre kerüljenek. Ha egy későbbi cella egy korábbi cella által létrehozott változóra támaszkodik, de a korábbi cella most hibát dob, a Restart and Run All ezt megmutatja. Az újraindított kernelben a cellák végrehajtási sorszáma `[1]`, `[2]`, `[3]` szerint halad. A böngésző egyszerű frissítése ezzel szemben nem indítja újra a kernelt; csak újratölti a notebook kimeneti állapotát.

A **csomagverziók** ellenőrzésének két független rétege van. A lockfájl rögzíti, mit kellett telepíteni; a futó környezetben az `importlib.metadata.version` vagy a `uv pip list` mutatja, mi van ténylegesen az adott interpreterhez kötve. A notebook és a CLI összehasonlításakor a verziószövegnek pontosan egyeznie kell. A könyv keretjegyzete szerint a kezdőprojekt pandas 2.2.3-mal futott, ezért ez a konkrét verzió használható ellenőrzési példaként. Ha a notebookban más verzió jelenik meg, az azonnali jele a környezeti eltérésnek.

3.4.3 Teljes minimális munkaállás

A munkamenetet Linuxon vagy macOS-en mutatom be; a Windows-eltérést a parancsok után jelölöm. A munkakönyvtár a lab projekt gyökere, például `~/northstar/lab`. A könyv írásakor ezeket a parancsokat nem futtattam, ezért az alábbi kimenetek a várt alakot szemléltetik, nem rögzített futási jegyzékek.

Először a lab környezetét frissítjük a lock szerint:

```
# Munkakönyvtár: ~/northstar/lab (Linux/macOS)
uv lock
uv sync --locked
```

A `uv lock` elkészíti a feloldott `uv.lock`-ot, ha a projektmappa még nem tartalmazza; a `uv sync --locked` a lockkal összhangban telepíti a `.venv`-be. A `--locked` itt nem csak kényelmi kapcsoló: hibát ad, ha a `pyproject.toml` és a lock nincs összhangban. Ezután a terminálból ellenőrizzük a környezetet:

```
uv run python -c "import sys; print(sys.executable)"
# Várt, szemléltető kimenet:
# /home/felhasznalo/northstar/lab/.venv/bin/python
```

A következő lépés a kernelspec regisztrálása. A `--user` kapcsoló a felhasználói Jupyter-könyvtárba helyezi a bejegyzést, így a JupyterLab a felhasználó bármely projektjéből megtalálja:

```
uv run python -m ipykernel install --user \
  --name "northstar-labs" \
  --display-name "Northstar labs (Python 3.12)"
```

A regisztráció után a JupyterLab megnyitása a projektkörnyezetből történik:

```
uv run jupyter lab
```

Ha a lab `pyproject.toml` dev függőségei között a JupyterLab nem szerepelne, az `uv run --with jupyterlab jupyter lab` ideiglenesen elérhetővé teszi a parancs idejére. A notebookban a kernelválasztónál a „Northstar labs (Python 3.12)” feliratot kell választani. Indítás után két cellát teszünk a notebookba. Az első a futó interpretert és a környezethez tartozó csomagverziókat írja ki:

```
import os
import sys
from importlib.metadata import version

print("executable:", os.path.realpath(sys.executable))
for package in ["ipykernel", "numpy", "pandas"]:
    print(package, version(package))
```

A második cella szándékosan egy egyszerű számítást végez, hogy a kernel valódi végrehajtása is látszódjon:

```
print("sum check:", 1 + 2)
```

Ezután a JupyterLab menüben a **Kernel → Restart Kernel and Run All Cells...** parancsot adjuk. A kernelt a regisztrált kernelspec indítja el, minden cella újra lefut, és a notebook kimenete a jelenlegi környezetből származik. A cellák mellett `[1]`, `[2]` sorszámok jelennek meg. Ugyanezt a kódot a terminálból is lefuttatjuk:

```
uv run python -c "import os, sys; from importlib.metadata import version; print('executable:',
↪  os.path.realpath(sys.executable)); print('pandas', version('pandas'))"
```

A két kimenetet egymás mellé téve ellenőrizzük. A notebook és a CLI akkor van összekötve, ha mindkét oldal ugyanazt a `.venv`-beli Python-útvonalat írja ki, és a `pandas` verziója mindkét helyen pontosan `2.2.3`.

Windows PowerShellben ugyanez a munkamenet a következő alakú:

```
# Munkakönyvtár: a lab könyvtár
uv lock
uv sync --locked
uv run python -c "import sys; print(sys.executable)"
# Várt alak: C:\Users\felhasznalo\northstar\lab\.venv\Scripts\python.exe
uv run python -m ipykernel install --user --name "northstar-labs" --display-name "Northstar labs
↪  (Python 3.12)"
uv run jupyter lab
```

Az eltérés lényeges része a 2.4-ban tárgyalt útvonalkép: a Windows bináris könyvtár neve `Scripts`, a Linux/macOS bináris könyvtár neve `bin`. A kernelspec fájl ezért gépenként újra regisztrálandó; a `.venv` nyers másolása vagy a logikai `..` relatív útvonalak nem garantálják ugyanazt a regisztrációt.

A létrejött kernelspec ellenőrzése parancssorból:

```
uv run jupyter kernelspec list
# Szemléltető, várható sor a bejegyzések között:
# northstar-labs    /home/felhasznalo/.local/share/jupyter/kernels/northstar-labs
```

Linuxon vagy macOS-en a fájl:

```
cat ~/.local/share/jupyter/kernels/northstar-labs/kernel.json
```

Windowson:

```
Get-Content $env:APPDATA\jupyter\kernels\northstar-labs\kernel.json
```

A JSON `argv[0]` elemének a lab `.venv` Pythonjára kell mutatnia. Ha itt rövid, csupasz `python` név állna, az a PATH-tól függő indítást jelentene; a `uv run`-nal végzett regisztráció ezt elkerüli, mert az abszolút útvonalat rögzíti.

3.4.3.1 Automatikus notebook-smoketest

A kézi „Restart and Run All” helyett a notebook első kódcellája legyen géppel ellenőrizhető setup cella, és egy checker kösse a kernel nevét, az interpreter jelentését és az execution count sorrendjét. A `lab/notebook_smoke.py` ezt végzi:

```
import importlib.metadata as md
import json
import sys
from pathlib import Path

root = Path.cwd()
while root != root.parent and not (root / "pyproject.toml").is_file():
    root = root.parent
packages = {}
for name in ("numpy", "pandas", "scikit-learn", "statsmodels"):
    try:
        packages[name] = md.version(name)
    except md.PackageNotFoundError:
        packages[name] = None
report = {
    "python": sys.version.split()[0],
    "executable": sys.executable,
    "prefix": sys.prefix,
    "project_root": str(root),
    "has_pyproject": (root / "pyproject.toml").is_file(),
    "packages": packages,
}
print(json.dumps(report))
assert report["has_pyproject"], "nem található a projektgyökér (pyproject.toml)"
assert Path(sys.executable).absolute().is_relative_to(Path(sys.prefix).absolute()), \
    "az interpreter nincs a sys.prefix alatt"
```

Ellenőrzés:

```
python3 lab/notebook_smoke.py \
  lab/checkpoints/cp1/solution/notebooks/explore.ipynb \
  --expected-kernel northstar-clean-handoff
python3 lab/notebook_smoke.py <notebook.ipynb> --execute   # friss kernel, nbconvert/papermill
```

A checker állapotai:

Állapot	Jelentés
`ok`	a setup cella és a kernelspec rendben, az execution_count 1..n
`missing_setup_cell`	az első kódcellából hiányzik a `sys.executable/sys.prefix/pyproject.toml` marker
`wrong_kernel`	a kernelspec neve eltér az elvárttól
`stale_execution`	van nem futtatott cella, vagy az execution_count nem 1..n
`execution_failed`	az `--execute` friss futtatása hibával zárult

A friss futtatású `--execute` módhoz Jupyter vagy papermill kell; ha egyik sincs telepítve, a checker ezt jelzi, és nem állít sikeres futtatást. A `tests/test_notebook_smoke.py` a meglévő megoldásnotebook setup celláját, a szintetikus elavult execution countot, a hiányzó setup cellát és a rossz kernelt teszteli. A `lab/checkpoints/cp1/solution/notebooks/explore.ipynb` a setup cellát tartalmazza; a forráskiadásban mentett kimenet nélkül szerepel, ezért a statikus ellenőrzés ott `--no-execution-check` kapcsolóval fut, a tanuló gépén viszont a Run All után az 1..n sorszámok ellenőrizhetők.

3.4.4 Eredmény értelmezése

Az ellenőrzési invariant: a notebookból kiírt `os.path.realpath(sys.executable)` és a terminálparancs `uv run python` által kiírt realpath ugyanazt a fájlt jelöli, és a lekérdezett csomagverziók pontosan egyeznek. A verziószövegben nincs érdemi tolerancia; egy `numpy` verzióeltérés már azt jelenti, hogy a két folyamat más csomagkészletet lát. Az útvonalkiírásban lehet minimális formai eltérés a szimbolikus linkek feloldása előtt, ezért használunk `os.path.realpath`-ot. A `Restart and Run All` után a cellák `[1]`, `[2]` sorszámai mutatják, hogy a kernel tényleg újraindult. A `sum check: 3` kimenet önmagában triviális, de a friss cellakimenettel együtt azt bizonyítja, hogy a notebook jelenlegi végrehajtása sikeres volt; a régi, felülírt kimenet nem maradhat a notebookban. A regisztrált kernelspec és a notebook környezet közötti kapcsolat érvényességi korlátja: a `--user` bejegyzés gépszintű felhasználói állapot, ezért másik gépen újra kell regisztrálni, és a `.venv` törlése után a bejegyzés nem automatikusan frissül.

Kernel-életciklus. A kernelspec neve és a kernelen belül futó Python két külön azonosító. A notebookban mindig ellenőrizni kell a `sys.executable` értékét és az importált csomag tényleges verzióját. Egy régi kernelregisztráció a virtuális környezet törlése után is hátramaradhat.

Takarítás és visszaellenőrzés: a kernelspecek felsorolása `jupyter kernelspec list`, az elavult regisztráció eltávolítása `jupyter kernelspec remove <név>` (csak a megfelelő név megerősítése után). Új kernelbe történő bejelentkezés után a notebook `Restart & Run All` próba adja a záró kontrollt.

3.4.5 Feladatok

1. **Értelmezés.** Mi a különbség a kernelspec `--name` és `--display-name` értéke között? Miért nem elég a JupyterLab felületén kiírni egy „Python 3” nevű kernelre, ha a lab `.venv`-et akarjuk használni?
2. **Implementálás.** Regisztrálj egy második kernelt `--name "northstar-check"` néven, „Northstar check” megjelenített névvel. Indítsd el a kernelt egy notebookban, és futtasd le a `sys.executable`-t és a `pandas` verzióját kiíró cellát. Hasonlítsd össze a terminál `uv run python` kimenetével. Mi a minimális ellenőrzési feltétel, és milyen parancsot használsz a tesztkernel eltávolítására?
3. **Hibadiagnózis.** Egy tanuló notebookja az `import scipy` cellánál `ModuleNotFoundError`-t ad, miközben a terminálban az `uv run python -c "import scipy"` sikeres. Adj három lehetséges okot, és mindegyikhez egy konkrét diagnosztikai parancsot vagy notebook-lépést.
4. **Megváltozott feltételű továbbvitel.** Ugyanazon projektet egy második, Windows-gépen `uv sync --locked` után megnyitják, de a JupyterLab kernelválasztója nem kínálja a lab kernelt. Mi a hiányzó lépés, és hogyan ellenőrzöd, hogy a regisztráció a Windows `.venv\Scripts\python.exe` útvonalat rögzítette? Mutass egy `--sys-prefix` változatot is, ha a környezetet csak ehhez a projekthez akarjuk kötni.

3.4.6 Megoldások és indoklás

1. feladat. A `--name` a kernelspec könyvtár- és parancsazonosítója, például `northstar-labs`; a Jupyter ezen a néven hivatkozik a JSON-bejegyzésre, és ezzel távolítható el. A `--display-name` a JupyterLab felületén megjelenő, ember által olvasható felirat, például „Northstar labs (Python 3.12)"; kizárólag vizuális szerepe van. Egy „Python 3" feliratú kernel mögött bármelyik regisztráció állhat; a notebook kiválasztásakor a felirat nem árulja el, melyik interpreterhez mutat. Ezért a kiválasztás után a notebookban le kell futtatni a `sys.executable`-t kiíró cellát is, mert az azonosító csak a háttérben tárolt bejegyzést egyértelműsíti.

2. feladat. A regisztráció:

```
uv run python -m ipykernel install --user --name "northstar-check" --display-name "Northstar
↪  check"
```

A notebookban a „Northstar check" kernelt választjuk, majd futtatjuk:

```
import os
import sys
from importlib.metadata import version

print(os.path.realpath(sys.executable))
print(version("pandas"))
```

A terminálból:

```
uv run python -c "import os, sys; from importlib.metadata import version;
↪  print(os.path.realpath(sys.executable)); print(version('pandas'))"
```

A minimális ellenőrzési feltétel: a két realpath-útvonal megegyezik, és a `pandas` verziószövege pontosan azonos; a tárgyalt laborban a várt érték `2.2.3`. A tesztkernel eltávolítása:

```
uv run jupyter kernelspec remove northstar-check
```

A törlés után a `uv run jupyter kernelspec list` már nem tartalmazza a bejegyzést.

3. feladat. Lehetséges okok és ellenőrzésük:

- A notebook még a régi kernelt használja. Diagnózis: a notebook cellában `print(sys.executable)`; ha az útvonal nem a lab `.venv`-re mutat, akkor a kernelválasztóban másik kernel van kiválasztva.
- A kernelspec rossz interpreterhez van regisztrálva, mert a tanuló globális `python -m ipykernel install`-t futtatott. Diagnózis: `uv run jupyter kernelspec list`, majd a listázott útvonalon a `kernel.json` `argv[0]` elemének megnézése.
- A `scipy` a terminálban a lab `.venv`-ben elérhető, de a notebook kernel folyamata még a `uv sync` előtti, régi környezetből fut. Diagnózis: Restart Kernel and Run All Cells a notebookban, majd `print(sys.executable)` újra; ez a régi folyamatot megszakítja és a kernelspecből friss folyamatot indít.

4. feladat. A `uv sync --locked` a lock szerinti környezetet telepíti, de a kernelspec felhasználói regisztráció nem a lock része, ezért a második gépen a notebook nem ismeri a kernelt. A hiányzó lépés:

```
uv run python -m ipykernel install --user --name "northstar-labs" --display-name "Northstar labs
↪  (Python 3.12)"
```

A Windows-útvonal ellenőrzése:

```
Get-Content $env:APPDATA\jupyter\kernels\northstar-labs\kernel.json
```

A JSON `argv[0]` elemének a `C:\Users\felhasznalo\northstar\lab\.venv\Scripts\python.exe` alakot kell mutatnia. A projekthez kötött `--sys-prefix` változat:

```
uv run python -m ipykernel install --sys-prefix --name "northstar-labs" --display-name
↪  "Northstar labs (Python 3.12)"
```

Ez a bejegyzést a projekt `.venv` alá helyezi, például a `.venv\share\jupyter\kernels\northstar-labs` könyvtárba, így a kernelválasztóban akkor jelenik meg, ha a JupyterLab ebből a környezetből indul; a felhasználói kernelspec-tár rendszerszintű állapotát nem terheli.

3.4.7 Gyakori tévedés ellenpéldával

Egy régi cellakimenet nem igazolja az aktuális kód sikeres futását. Tegyük fel, hogy a notebookban a következő cella korábban sikeresen futott:

```
print(version("pandas"))
# korábbi kimenet: 2.2.3
```

Ezután a tanuló a `pyproject.toml`-ban módosítja a pandas-verziót, lefuttatja a `uv lock` és `uv sync --locked` parancsot, majd a notebook celláját átírja:

```
print(version("pandas").upper())
```

Ha a cellát nem futtatja újra, a notebook továbbra is a régi `2.2.3` kimenetet mutatja, miközben a kód már más utasítást tartalmaz. A régi kimenet hamis bizonyíték: úgy tűnik, a notebook sikeres volt, pedig az aktuális kódot senki sem hajtotta végre. A Restart Kernel and Run All Cells ezt a csapdát oldja fel, mert a régi kernel megszűnik, és minden cellát az aktuális kernelspec szerint indít újra. Ez a jelenség ugyanaz a gép- és környezetfüggő állapotprobléma, mint a 2.4-ban a másik gépen való futtatás: a notebook dokumentumon kívüli állapotot nem másolja a verziókezelés. Emellett a 3.3 határa is érvényes: ha a projekt környezetét uv kezeli, a kernel regisztrációját is `uv run`-nal érdemes végezni, nehogy a kernelspec egy conda- vagy globális Pythonra mutasson.

Önellenőrzésként nyissunk egy új notebookot, indítsuk el a „Northstar labs” kernelt, futtassuk a `Restart and Run All`-t, majd hasonlítsuk össze a kiírt `os.path.realpath(sys.executable)` értéket a terminál `uv run python` kimenetével. Ha a két útvonal megegyezik, a regisztráció helyes; ha nem, a kernelspec `argv[0]` elemét kell javítani, vagy a rossz kernelbejegyzést eltávolítani.

3.4.8 Rövid összefoglaló

A notebook csak akkor reprodukálható, ha a Jupyter-kernel pontosan a projekt `.venv`-jére mutat. A kernelspec egy JSON-bejegyzés, amelynek `argv[0]` eleme a tényleges Python-útvonal; a `--name` az azonosító, a `--display-name` a felületi felirat. A `uv run python -m ipykernel install --user` a lab környezetét regisztrálja. A Restart and Run All új kernelt indít, és minden cellát lineárisan futtat; a régi cellakimenet nem bizonyítja az aktuális kód sikerét. A notebook és a CLI ellenőrzésekor a realpath-útvonalnak és a csomagverzióknak pontosan egyezniük kell. A következő alfejezetben ezt a munkaasztalt notebook-végrehajtási gráfként használjuk: cellák közötti függőség, hibakeresés és notebookból script átalakítás kerül sorra.

4 Jupyter, VS Code és az interaktív kutatói munkafolyamat

4.1 Notebook mint végrehajtási gráf

Belépő · szint: alap · idő: 45 perc · eszköz: JupyterLab, VS Code

Előfeltétel: 3.4. alfejezet · Célkészség: Notebook mint végrehajtási gráf · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

A notebook egy interaktív számítási gráf, nem csupán szöveges dokumentum. Ebben az alfejezetben egy szándékosan rossz cellasorrendű notebookot javítunk ki, és olyan futási bizonyítékot készítünk, amely tiszta kernelből indítva is érvényes. Az eszköz a JupyterLab vagy a VS Code Jupyter-kiterjesztése, a parancssori ellenőrzéshez pedig `nbconvert`.

4.1.1 Cél és konkrét helyzet

A `lab/late_manual.ipynb` notebookot egy kolléga „működő” állapotban osztotta meg. A notebook a Northstar mini-változatát használja: három rendelés, `order_id`, `delivery_days`; a cél egy késési arányt kiírni. Az input cellák hibás forrásbeli sorrendet tartalmaznak: a nyomtatás az első cellában van, a számításhoz szükséges `late_rate` név azonban csak egy későbbi cellában jön létre. A kolléga kézi futtatással mégis látott eredményt, mert a cellákat a függőségi sorrendben futtatta, nem a forrásbeli sorrendben. A feladat: rendezd a cellákat úgy, hogy a Restart and Run All is hibátlanul lefusson, és a tiszta futást parancssorból is bizonyítsd.

A mini-adat: O1 8 nap alatt érkezik, O2 5, O3 9; az ígért nap `promised_days=7`. A célfüggvény `is_late(delivery_days, promised_days)` igaz, ha a kézbesítés késik. A kézi előzetes becslés: a három rendelésből kettő késik (O1 és O3), tehát a késési arány $2/3 \approx 0.67$. A notebooknak ezt kell kiírnia; a `flags` lista hosszának meg kell egyeznie az `orders` hosszával, a `late_rate` értéknek 0 és 1 közé kell esnie.

Egységek és alakok: `delivery_days` és `promised_days` pozitív egész, mértékegysége nap; `orders` 3 elemű lista, elemei `dict`; `flags` 3 elemű lista, elemei `bool`; `late_rate` skalár, valós érték $[0, 1]$-ben.

A megfigyelési egység az adatban egy rendelés, a notebook-vizsgálatban egy cellavégrehajtás.

4.1.2 A cella mint függvény: bemenet és kimenet

A Python-függvény elemi formában adja a modellt:

$$g(d, p) = [d > p]$$

ahol - $d \in \{1, 2, ...\}$ a kézbesítési napok száma, mértékegysége nap; - $p \in \{1, 2, ...\}$ az ígért napok száma, mértékegysége nap; - $[P]$ a logikai jelölés: 1, ha P igaz, 0, ha hamis. A kimenetnek nincs mértékegysége; a Python `bool` típusa `int`-ként is viselkedik.

Kézzel: $g(8, 7) = 1$, $g(5, 7) = 0$, $g(9, 7) = 1$.

Ugyanez a gondolat cellaszintre: minden cellának vannak bemenő nevei — azok a változók, függvények, osztályok, importált modulok, amelyeket a cella forrása olvas — és kimenő nevei — azok, amelyeket a cella értékadással, `def`-fel, `class`-szal vagy importtal definiál. Legyen c_i az i-edik cella, $\mathrm{IN}(c_i)$ a bemenő nevek halmaza, $\mathrm{OUT}(c_i)$ a kimenő nevek halmaza, B a beépített nevek halmaza (`print`, `sum`, `len` stb.). A $c_1, c_2, ...$ cellasorrend akkor érvényes, ha minden i-re

$$\mathrm{IN}(c_i) \setminus B \subseteq \bigcup_{j<i} \mathrm{OUT}(c_j).$$

Szavakkal: minden cella minden nem beépített bemenő nevét egy korábbi cellának kell létrehoznia, mielőtt a cella fut. A képlet mennyiségei halmazok, nem tömbök; az összehasonlítás névfelsorolással kézzel is ellenőrizhető.

A hibás notebook függőségi táblázata:

Cellasorrend a forrásban	IN	OUT
1. `print` cella	`late_rate`	–
2. `def is_late`	–	`is_late`
3. `orders`	–	`orders`
4. `flags`	`orders`, `is_late`	`flags`
5. `late_rate`	`flags`, `sum`, `len`	`late_rate`

Az 1. cella megsérti a feltételt: a `late_rate` név az 5. cella kimenete, tehát az 1. cella csak az 5. után futhatna. A táblázatból a helyes sorrend: 2, 3, 4, 5, 1.

4.1.3 Execution count: a tényleges végrehajtási nyom

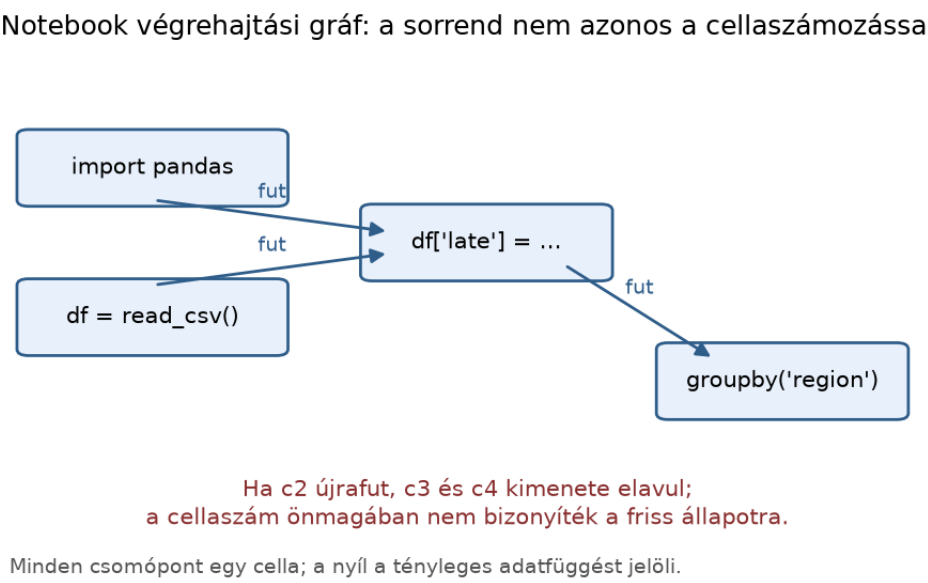
A Jupyter minden sikeres cellavégrehajtáshoz monoton növekvő sorszámot rendel: a cella mellett `In [1]`, `In [2]` stb. jelenik meg. A `In [ ]` azt jelenti, hogy a cella ebben a kernelmunkamenetben még nem futott; futás közben `In [*]` látható. A szám a **futtatás sorrendjét** rögzíti, nem a cella forrásbeli pozícióját. Ha egy cellát újra futtatunk, új számot kap; a notebook mentésekor a legutóbbi végrehajtási szám és a hozzá tartozó kimenet együtt mentődik. A cella forrásbeli helye és a végrehajtási száma ezért elválhat.

A hibás notebookban a kolléga kézi futtatása például ezt a nyomot hagyja:

- A 2. cella (`def is_late`) `In [1]`
- A 3. cella (`orders`) `In [2]`
- A 4. cella (`flags`) `In [3]`
- Az 5. cella (`late_rate`) `In [4]`
- Az 1. cella (`print`) `In [5]`

A forrás első cellája tehát utoljára futott. A mentett notebook „működik”, de felülről lefelé futtatva nem. A JupyterLab és a VS Code ugyanazt a countot mutatja; a VS Code bal oldali cellajelzője is a kernel állapotát követi.

4.1.3.1 Fogalmi ábra: a cellák végrehajtási gráfja



4.1. ábra: Notebook cellák végrehajtási gráfja: c1 és c2 független, c3 mindkettőre épül, c4 pedig c3-ra; ha c2 újrafut, a c3 és c4 kimenete elavul.

Az ábra a fenti execution count-állítás vizuális párja: a cellaszám a keletkezés sorrendje, nem a futtatási sorrend. A nyíl tényleges adatfüggést jelöl; a `c4` akkor is a `c3` kimenetére épül, ha közben a `c2`-t újrafuttattuk. A konklúzió ábra nélkül is ugyanaz: a mentett notebook „működése” nem bizonyítja, hogy felülről lefelé is működik; ehhez clean kernel és determinisztikus cellasorrend kell. A könyv ábráját a `lab/figures/make_concept_figures.py execution_graph()` függvénye állítja elő; a szerkezet kivonata:

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

cells = [(0.17, 0.78, "import pandas"), (0.17, 0.50, "df = read_csv()"),
         (0.52, 0.64, "df['late'] = ..."), (0.83, 0.38, "groupby('region')")]
fig, ax = plt.subplots(figsize=(6.4, 3.6))
ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.axis("off")
for x, y, text in cells:
    ax.add_patch(FancyBboxPatch((x - 0.13, y - 0.06), 0.26, 0.12,
                 boxstyle="round,pad=0.012", facecolor="#e8f0fb",
                 edgecolor="#2f5d8a"))
    ax.text(x, y, text, ha="center", va="center", fontsize=8.5)
for start, end in [((0.17, 0.72), (0.42, 0.66)),
                   ((0.17, 0.56), (0.42, 0.62)),
                   ((0.60, 0.60), (0.76, 0.42))]:
    ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>",
                 mutation_scale=11, color="#2f5d8a"))
ax.set_title("Notebook végrehajtási gráf")
plt.show()
```

4.1.4 Mutáció, globális állapot és rejtett import

A notebook minden cellája ugyanabban a globális névtérben fut. Ennek három következménye van.

1. **Mutáció (mutation).** Ha egy cella egy meglévő listát módosít, nem pedig újradefiniálja, a cella ismételt futtatása halmozza a hatást:

   ```
   orders += [{"order_id": "O4", "delivery_days": 6}]
   ```

   Kétszeri futtatás után az `orders` lista 5 elemű lesz, pedig a szándék 4 elem volt. A `flags` és `late_rate` cellák ettől még a régi értéket őrzik, amíg újra nem futnak — ez rejtett, nem látható inkonzisztencia.

2. **Rejtett import (hidden import).** A

   ```
   def risk(distance_km: float) -> float:
       return math.sqrt(distance_km)
   ```

   cella látszólag önálló, de a `math` név egy másik cellából vagy egy korábbi kernelállapotból származik. Ha tiszta kernelben csak ezt a cellát futtatjuk, `NameError: name 'math' is not defined` a válasz. Az import helye a cella függősége, ezért a függőséget magában a cellában kell láthatóvá tenni.

3. **Globális változó.** A `late_rate` a print cella számára globális bemenet. A cellák közötti globális értékadás nem tiltott, de a sorrendet és a tiszta kernelindítást rögzíteni kell, különben a notebook egy korábbi munkamenet maradék állapotából is dolgozhat.

4.1.5 Clean kernel és determinisztikus cellasorrend

A **clean kernel** a kernel újraindítását jelenti: a memóriában nem marad semmilyen korábbi név, függvény vagy import. A **Restart and Run All** két dolgot tesz: újraindítja a kernelt, majd a cellákat felülről lefelé sorban futtatja. Ez az első valódi próbája a notebook reprodukálhatóságának. Ha a sorrend helyes, minden cella kimenete a tiszta állapotból jön létre; ha nem, az első hibás cellánál a futás megáll, és a notebook nem tudja bemutatni a mentett eredményt.

A **determinisztikus cellasorrend** két követelményt jelent: a cellák függőségi (topologikus) sorrendben vannak, és a cellák idempotensek. Az idempotencia itt azt jelenti, hogy egy cella másodszori futtatása ugyanazt a globális állapotot és kimenetet adja, nem halmoz el újabb hatást. A teljes lista értékadása idempotens; a `+=` mutáció nem az. A tiszta futási bizonyítékhoz a kernel indítását, a cellasorrendet és a csomagkörnyezetet is rögzíteni kell — ez utóbbihoz térünk vissza a 3.1. alfejezetben bevezetett `sys.executable` ellenőrzésre.

4.1.5.1 Gyakorlófüzet: rejtett állapot és javított változat

A rejtett állapot nem elméleti lehetőség: a `lab/checkpoints/cp1/starter/hidden_state.ipynb` szándékosan eltérő mentett kimenetet tartalmaz. A füzet cellái:

```
setup cella          execution_count = 1
x = 1                execution_count = 2
x += 1               execution_count = 3
print(x)             execution_count = 5, mentett kimenet = 3
```

A `4` sorszám kimarad, mert a `x += 1` cella kétszer futott; a mentett `3` tehát nem a top-to-bottom sorrend eredménye. A feladat:

1. írd le a várt eredményt a cellák olvasása alapján;
2. futtasd a **Kernel → Restart Kernel and Run All Cells** parancsot;
3. hasonlítsd össze a friss eredményt (2) a mentettel (3);
4. keresd meg az `execution_count` hézagát, és indokold a különbséget.

A javított változat a `lab/checkpoints/cp1/solution/hidden_state.ipynb`: ott a sorszámok `1, 2, 3, 4`, a mentett kimenet pedig 2. A két füzetet a notebook-smoketest is megkülönbözteti:

```
python3 lab/notebook_smoke.py lab/checkpoints/cp1/starter/hidden_state.ipynb
# state: stale_execution, hiba: az execution_count nem 1..4: [1, 2, 3, 5]
python3 lab/notebook_smoke.py lab/checkpoints/cp1/solution/hidden_state.ipynb
# state: ok
```

A `tests/test_hidden_state_notebook.py` a hibás füzet `stale_execution` állapotát, a javított `ok` állapotát, a cellák top-to-bottom eredményét (2) és a sorszámhézagot ellenőrzi. A gyakorlat megoldása a `starter/hidden_state_README.md` fájlban is szerepel.

4.1.6 Teljes minimális munkapélda

A következő kimenetek **szemléltető, elméletileg várt** értékek; a kapcsolódó példaprojektben még nem ellenőrzött a notebookot. A lépéseket a `lab/` munkakönyvtárban végezd el, miután a `uv sync` és a kernelregisztráció a 3.4. alfejezet szerint megtörtént.

1. lépés: a hibás notebook létrehozása. Indítsd el a JupyterLabot:

```
cd lab
uv run jupyter lab
```

Platform: Linux, macOS vagy Windows PowerShell ugyanezzel a két paranccsal; a `lab` a projekt gyökere. Hozz létre `late_manual.ipynb` notebookot a regisztrált projektkernellel, és vedd fel a következő öt cellát ebben a sorrendben:

```
print(f"késési arány: {late_rate:.2f}")
```

```
def is_late(delivery_days: int, promised_days: int = 7) -> bool:
    return delivery_days > promised_days
```

```
orders = [
    {"order_id": "O1", "delivery_days": 8},
    {"order_id": "O2", "delivery_days": 5},
    {"order_id": "O3", "delivery_days": 9},
]
```

```
flags = [is_late(o["delivery_days"]) for o in orders]
```

```
late_rate = sum(flags) / len(flags)
```

A f-string a kapcsos zárójelben álló kifejezést formázza; a `:.2f` két tizedesjegyre kerekít. A `-> bool` csak típusinformáció, a Python futás közben nem ellenőrzi.

2. lépés: a hibásan működő kézi futtatás. Tedd a kurzort a második cellába, futtasd, majd a harmadikat, negyediket, ötödiket, végül az elsőt. Az execution countok felülről lefelé nem növekvő sorrendet mutatnak, ahogy az előző szakaszban leírtuk. Az első cella kimenete `késési arány: 0.67`. A notebook most „működik”, de a forrás sorrendje hazudik.

3. lépés: a tiszta próba hibája. Válaszd a **Kernel → Restart Kernel and Run All Cells** menüpontot. A kernel újraindul, az első cella fut, de a `late_rate` nevű változó még nem létezik, ezért a kimenet `NameError: name 'late_rate' is not defined`, és a többi cella nem fut le. Ez a várt ellenőrző jel arra, hogy a notebook nem reprodukálható a forrásbeli sorrendben.

4. lépés: javítás. Húzd a print cellát az utolsó cella után (vagy a `late_rate` cella mögé), és a notebook legelső cellájaként add hozzá a környezetbizonyítékot:

```
import sys
print(sys.executable)
```

A végső cellasorrend:

```
import sys
print(sys.executable)

def is_late(delivery_days: int, promised_days: int = 7) -> bool:
    return delivery_days > promised_days

orders = [
    {"order_id": "O1", "delivery_days": 8},
    {"order_id": "O2", "delivery_days": 5},
    {"order_id": "O3", "delivery_days": 9},
]

flags = [is_late(o["delivery_days"]) for o in orders]

late_rate = sum(flags) / len(flags)

print(f"késési arány: {late_rate:.2f}")
```

5. lépés: tiszta futási bizonyíték. Futtasd ismét a **Restart and Run All** parancsot. A várt kimenet: az első cella a projekt `.venv/bin/python` vagy `.venv/Scripts/python.exe` útvonalát írja ki, a hatodik cella `késési arány: 0.67`; a cellák melletti countok 1-től 6-ig növekvőek. Kézi kontroll: a `flags` lista `[True, False, True]`, a `sum` 2, a `len` 3, a hányados 0.666…, a két tizedesjegyre kerekített kiírás 0.67.

A futtatást parancssorból is bizonyíthatod; az `nbconvert --execute` egy új kernelben hajtja végre a notebookot, és HTML-fájlt generál a friss kimenettel:

```
cd lab
uv run jupyter nbconvert --to html --execute --output late_manual_fixed.html late_manual.ipynb
```

A parancs Linux/macOS és Windows PowerShell esetén is a `lab` munkakönyvtárból futtatható. Ha a notebook hibás, a parancs hibával leáll; sikeres futás után a HTML a 6 cella friss kimenetét tartalmazza. Fontos, hogy a notebook JSON-metadata `kernelspec.name` mezője a regisztrált projektkernelre mutasson; ellenkező esetben az `nbconvert` más kernelben fut. Ezt a **Restart and Run All** előtt a notebook kernelcímkéje is mutatja, a bizonyítékcellában kiírt `sys.executable` pedig a tényleges feloldást rögzíti a 3.1. alfejezet módszerével.

4.1.7 Eredmény értelmezése

A javított notebook a **Restart and Run All** után tiszta kernelből 1-től 6-ig növekvő végrehajtási számmal fut le; a várt késési arány $2/3 \approx 0.67$. A kézi ellenőrzés invariánsa, hogy `flags` hossza megegyezik `orders` hosszával, és a `late_rate` a `sum(flags)/len(flags)` képletből a 0–1 tartományba esik. A példa nem a Northstar teljes predikciója, hanem a cellafüggőségi gráf tanulóesete. A `sys.executable` kimenet azt igazolja, hogy a tiszta futás a lab `.venv` környezetét használta.

Notebook mint állapottartó folyamat. A cellák szöveges sorrendje nem mindig a tényleges futtatási sorrend. Két dokumentációs állapotot különböztessünk meg: a jelenlegi kernelmemóriában működő notebookot és a friss kernelben felülről lefelé hibátlanul újrafutó notebookot. Oktatási és átadási célra csak az utóbbi tekinthető reprodukálhatónak.

Kötelező ellenpróba: hozz létre egy olyan változót egy későbbi cellában, amelyre egy korábbi hivatkozik; futtasd fordított sorrendben, majd Restart & Run All módban. Az eltérés mutatja, hogy a látható notebook nem feltétlenül deklarál minden függőséget.

4.1.8 Feladatok

1. **Értelmezés.** Egy notebookban a cellák forrásban A, B, C, D sorrendben vannak. A munkamenet végén A mellett `In [4]`, B mellett `In [1]`, C mellett `In [2]`, D mellett `In [3]` áll. Mi a tényleges végrehajtási sorrend? Miért nem elegendő a forrásbeli sorrend a notebook újrafuttathatóságának bizonyításához?

2. **Implementálás.** Hozd létre a fenti hibás ötcellás notebookot, futtasd kézzel a 2→3→4→5→1 sorrendben, mentsd, majd futtasd a **Restart and Run All** parancsot. Rögzítsd a hibát, rendezd a cellákat függőségi sorrendbe, és tiszta futással igazold a `késési arány: 0.67` kimenetet.

3. **Hibadiagnózis.** A javított notebookba két cellát illesztettél:

   ```
   orders += [{"order_id": "O4", "delivery_days": 6}]
   ```

   és

   ```
   def risk(distance_km: float) -> float:
       return math.sqrt(distance_km)
   ```

   Tiszta kernelben a `risk` cella `NameError`-t ad; az `orders +=` cella kétszeri futtatása 5 elemű listát hagy. Nevezd meg mindkét hibát, és javítsd a cellákat úgy, hogy tiszta kernelben, ismételt futtatással is determinisztikusak legyenek.

4. **Megváltozott feltétel.** A 3. feladat javított notebookjában az ígért napot 7-ről 5-re csökkentik. A négy rendelés: O1 8, O2 5, O3 9, O4 6 nap alatt érkezik. Számítsd ki kézzel a késési arányt, módosítsd a `flags` cellát `promised_days=5` argumentummal, futtasd tiszta kernelből, és adj meg egy olyan lebegőpontos toleranciát, amellyel a kapott értéket elfogadod.

4.1.9 Megoldások és indoklás

1. A tényleges végrehajtási sorrend B, C, D, A. A forrásbeli sorrend csak a notebook dokumentumában lévő cellák helyét adja; a mentett kimenet a legutóbbi kernelállapotot tükrözi. A B→C→D→A sorrendben a notebook működhet, de a Restart and Run All az A cellát előbb futtatná, és hibát kaphat. A reprodukálhatóság bizonyítéka a tiszta kernelből futtatott, a forrásbeli sorrenddel megegyező végrehajtás.

2. A hibás állapot: az első cella `NameError: name 'late_rate' is not defined`; a többi cella a futás megállása miatt nem kap új kimenetet. A javított sorrend: `def is_late`, `orders`, `flags`, `late_rate`, `print`. A tiszta futás bizonyításához az első cellaként `import sys; print(sys.executable)` is felvehető. A **Restart and Run All** után a countok növekvően futnak, a print cella kimenete `késési arány: 0.67`. Ellenőrző invariant: `len(flags) == len(orders) == 3` és `sum(flags) == 2`.

3. Az első hiba mutáció: a `+=` a meglévő listát bővíti, ezért a hatás ismétlésenként halmozódik. Javítás: a cella definiálja újra a teljes listát értékadással, és kerüljön a `flags` cella elé:

```
orders = [
    {"order_id": "O1", "delivery_days": 8},
    {"order_id": "O2", "delivery_days": 5},
    {"order_id": "O3", "delivery_days": 9},
    {"order_id": "O4", "delivery_days": 6},
]
```

Ez a cella másodszori futás után is 4 elemet ad, tehát idempotens. A második hiba rejtett import: a `math` név nem a `risk` cellában jött létre. Javítás:

```
import math

def risk(distance_km: float) -> float:
    return math.sqrt(distance_km)
```

Ellenőrzés: tiszta kernelben a `risk(25)` kimenete `5.0`; a cella egyszeri és kétszeri futtatása sem változtat az `orders` hosszán.

4. Kézi számítás 5 napos ígérettel: O1 8>5 igaz, O2 5>5 hamis, O3 9>5 igaz, O4 6>5 igaz; a késési arány $3/4 = 0.75$. A módosított cella:

```
flags = [is_late(o["delivery_days"], promised_days=5) for o in orders]
late_rate = sum(flags) / len(flags)
```

A tiszta futtatás kimenete `0.75`, mert a 0.75 kettes számrendszerben pontosan ábrázolható ($3{\cdot}2^{-2}$), de a kapcsolódó float-összegeknél érdemes tűrést használni: `abs(late_rate - 0.75) < 1e-12`, vagy `math.isclose(late_rate, 0.75, rel_tol=0, abs_tol=1e-12)`. Invariant: `len(flags) == 4` és `sum(flags) == 3`.

4.1.10 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: a cella vizuális pozícióját a végrehajtási sorrendnek hisszük. A fenti hibás notebook éppen az ellenpélda: a forrás első cellája a `print`, a kézi futtatásban mégis `In [5]` volt, mert a kolléga a cellákat más sorrendben futtatta. A mentett notebook így „sikeres” kimenetet mutat, de a Restart and Run All az első cellánál `NameError`-t ad. Ezért a notebook állapotát soha ne a cella dokumentumbeli helye, hanem a `In [n]` végrehajtási szám, a kernelindítás és a tiszta újrafuttatás alapján ítéld meg. A VS Code Jupyter-nézetében ugyanez a hiba ugyanígy megjelenik: a cellák sorrendje a dokumentumszerkesztő állapota, nem a futtatott gráf.

4.1.11 Önellenőrzés

- Ha egy cellát törölsz, de a kernelben a neve megmaradt, a notebook továbbra is futhat. Miért nem bizonyítja ez a reprodukálhatóságot?
- Mi a különbség a notebook mentett kimenete és a tiszta `nbconvert --execute` kimenet között?
- Hogyan döntöd el a 3.1. alfejezet eszközével, hogy a tiszta futás a lab `.venv` interpreterét használta?
- Mi a cellafüggőségi gráf érvényességi korlátja? (Csak a nevek definiáltságát ellenőrzi, nem az üzleti logika vagy a képlet helyességét.)

4.1.12 Rövid összefoglaló

A notebook cellái egyetlen globális névtérben futó számítási gráfot alkotnak. A Jupyter `In [n]` száma a végrehajtási sorrendet rögzíti, nem a cella vizuális pozícióját. A cellák közötti függőség a bemenő és kimenő nevek halmazával írható le; a mutáció, a rejtett import és a régi kernelállapot csendben meghamisítja a notebookot. A megoldás a determinisztikus, idempotens cellasorrend, valamint a Restart and Run All és az `nbconvert --execute` mint tiszta futási bizonyíték. A `sys.executable` kiírása a környezetet is rögzíti. A következő alfejezetben ezt a notebookot VS Code-debuggerrel vizsgáljuk, és a cellákból scriptet készítünk.

4.1.13 Kiadás előtti szakmai ellenőrzés

Reprodukálhatósági ellenpróba. A notebook véglegesítésének minimális feltétele: újraindított kernel mellett a cellák fentről lefelé egyetlen futtatás során hiba nélkül végrehajthatók; a korábbi futásból örökölt változókat nem szabad szükséges előfeltételnek tekinteni. A notebookban dokumentálni kell a használt kernel Python-verzióját és a függőségek rögzítésének módját.

4.2 IDE, típusinformáció és hibakeresés

Belépő · szint: alap · idő: 45 perc · eszköz: JupyterLab, VS Code

Előfeltétel: 4.1. alfejezet · Célkészség: IDE, típusinformáció és hibakeresés · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

4.2.1 Cél és konkrét helyzet

A Northstar-labor egyik munkatársa tételszintű elemzést készített: a rendelések tábláját összekapcsolta a rendelési tételek táblájával, majd a kapott táblából akarta összegezni a szállítási díjat. Azt feltételezte, hogy a join után a `shipping_fee_minor` oszlop sorainak összege ugyanaz, mint a rendelésszintű `orders.csv`-ben. A jelentése szerint viszont a szállítási bevétel gyanúsan magas lett.

A bemenet a lab `orders.csv` (megfigyelési egység: egy rendelés, kulcs `order_id`) és `order_items.csv` (megfigyelési egység: egy rendelési tétel, kulcs `(order_id, line_id)`). A cél nem az egész tábla kiíratása, hanem egy töréspontal és kis mintával megmutatni, hogy a join után a megfigyelési egység tétellé változott, ezért a rendelésszintű pénzösszegek tételenként ismétlődnek. Előzetes becslés: a belső join sorainak száma a tételek sorainak számával egyezik meg, ha minden tétel rendelése érvényes; a helyes szállítási összeg a rendelések táblájában marad, a soronkénti összegzés pedig rendelésenként a tételszámmal szorozza meg az összeget. A pontos eltérést a labor futtatása mutatja meg; itt a kimenetek szemléltető jellegűek.

4.2.2 Interpreter, töréspont, watch és a futás közbeni igazság

A VS Code Python-környezet egy **interpretert** (értelmezőt) választ a futtatáshoz. Ez az a `.venv/bin/python` (Windows alatt `.venv\\Scripts\\python.exe`) folyamat, amelyet az `s010`-ben a `uv sync --locked` hozott létre. A notebook kernelje ettől külön választás: a Jupyter egy másik futó folyamatot indíthat, ezért a VS Code-ban a Python interpreter kiválasztása és a notebook kernel kiválasztása két külön lépés. A Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) `Python: Select Interpreter` parancsával a projekt `.venv`-jét válaszd ki; ezt a státuszsor jobb alsó sarkában ellenőrizheted. Ha a notebook más kernellel fut, a cella és a script eltérő csomagkészletet láthat.

A **debugger** (hibakereső) ugyanazzal az interpreterrel indítja a programot, de egy megjelölt sor előtt megállítja a végrehajtást. A megjelölt sor a **breakpoint** (töréspont). Amíg a program áll, a **Variables** panel a lokális változók értékét és típusát mutatja, a **Debug Console**-ban pedig tetszőleges kifejezést kiértékelhetünk az aktuális végrehajtási keretben. A **watch** (figyelő) kifejezéseket rögzítünk, és a debugger minden léptetés után újra kiértékeli őket. A `Step Over` a következő sort hajtja végre anélkül, hogy a hívott függvény belsejébe lépne; `Step Into` a hívás belsejébe lép; `Continue` a következő töréspontig fut.

A `DataFrame.shape` tuple, első eleme a sorok száma, második az oszlopok száma. Az `orders.dtypes` az oszlopok tényleges numpy típusát mutatja (`int64`, `float64`, `object`, `datetime64[ns]`, `category`). Ez **futás közbeni** igazság: a read_csv következtethet rosszul, ezért a típust nem csak a fájl szerződése, hanem a betöltött tábla alapján is ellenőrizni kell. A karakteres azonosítót ne alakítsd számmá, mert az azonosító nem mennyiség. A `Series.is_unique` akkor igaz, ha a sorozat minden értéke különböző; a join előtti kulcsegyediség és a join utáni kulcsegyediség külön kérdés.

Matematikai nyelven: legyen N_O a rendelések, N_I a tételek darabszáma. Az i rendeléshez tartozó tételsorok száma n_i. Ha a rendeléskulcs egyedi és minden tétel kulcsa létezik, a belső join sorainak száma

$$N_J = \sum_{i=1}^{N_O} n_i = N_I.$$

Egy rendelésszintű s_i szállítási díj a join minden tételsorában megjelenik. A soronkénti összegzés hibásan

$$S_{\text{rossz}} = \sum_{i=1}^{N_O} n_i s_i,$$

a helyes összeg pedig

$$S_{\text{helyes}} = \sum_{i=1}^{N_O} s_i.$$

A kettő csak akkor egyenlő, ha minden rendeléshez pontosan egy tétel tartozik. A `shape` a darabszámot, az `index` a sorcímkét, a kulcsoszlop a megfigyelési egységet mutatja; a join után az index továbbra is `RangeIndex`, de a logikai kulcs már nem feltétlenül egyedi.

4.2.3 A minimális munkapélda lépésről lépésre

Előbb készíts egy kis **minimális reprodukciót** (minimal reproduction): olyan kis bemenetet, amely ugyanazt a hibát előidézi, de a teljes táblázatot kézzel átláthatóvá teszi. A minimális példa elvárt kimenete:

```
import pandas as pd

mini_orders = pd.DataFrame({
    "order_id": ["A", "B"],
    "shipping_fee_minor": [200, 200],
})
mini_items = pd.DataFrame({
    "order_id": ["A", "B", "B", "B"],
    "line_id": [1, 1, 2, 3],
})
mini_merged = mini_orders.merge(
    mini_items, on="order_id", how="inner", validate="one_to_many"
)
print(mini_merged)
print("shape:", mini_merged.shape)
print("hibás soronkénti összeg:",
      mini_merged["shipping_fee_minor"].sum())
```

```
print("helyes rendelésenkénti összeg:",
      mini_orders["shipping_fee_minor"].sum())
```

Elméletileg várt kimenet:

```
  order_id  shipping_fee_minor  line_id
0        A                 200        1
1        B                 200        1
2        B                 200        2
3        B                 200        3
shape: (4, 3)
hibás soronkénti összeg: 800
helyes rendelésenkénti összeg: 400
```

A `validate="one_to_many"` azt szerződteti, hogy a bal oldali `order_id` egyedi, a jobb oldalon viszont egy rendeléshez több sor is tartozhat. Ha a bal oldalon duplikált kulcs lenne, a merge azonnal `MergeError` kivételt ad.

A labor tényleges adatán készíts a notebook mellé egy `debug_join.py` scriptet, mert a VS Code-debugger tiszta scriptben a legátláthatóbb:

```
# debug_join.py -- a labor gyökérkönyvtárából futtatható
from pathlib import Path
import pandas as pd

DATA = Path("data")  # a labor ADATOK.md szerinti elérési útjához igazítsd
orders = pd.read_csv(
    DATA / "orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
items = pd.read_csv(DATA / "order_items.csv")

merged = orders.merge(
    items, on="order_id", how="inner", validate="one_to_many"
)

# Tegyél töréspontot a következő sorra!
print("betöltve:", orders.shape, items.shape, merged.shape)

wrong = merged["shipping_fee_minor"].sum()
right = orders["shipping_fee_minor"].sum()
print("hibás:", wrong, "helyes:", right)

# a duplázódás magyarázata rendelésenként:
counts = merged.groupby("order_id").size()
per_order_fee = orders.set_index("order_id")["shipping_fee_minor"]
duplication = (
    merged.groupby("order_id")["shipping_fee_minor"].sum()
    / per_order_fee
)
print("tételszámok:", counts.describe().to_dict())
print("duplázódási szorzók:", duplication.describe().to_dict())
```

A script megnyitása után válaszd ki az interpretert a projekt `.venv`-jéből. A `print("betöltve:", ...)` sor bal margójára kattintva helyezz el töréspontot. Indítsd a debuggert a `Run and Debug` panelen a `Python: Current File` konfigurációval. Amikor a futás megáll, a **Variables** panelben a `merged` mellett a futás közbeni típus és méret látható. A **Watch** panelben rögzítsd az alábbi kifejezéseket:

```
orders.shape
items.shape
merged.shape
```

```
orders.order_id.is_unique
items.duplicated(["order_id", "line_id"]).any()
merged[["order_id", "line_id"]].duplicated().any()
```

A Debug Console-ban a kulcsegyediség és a tételszámok közvetlenül is vizsgálhatók:

```
>>> orders.shape
# szemléltető: (2400, 15), a pontos szám a laborban ellenőrizendő
>>> items.shape
# szemléltető: (4796, 8)
>>> merged.shape
# szemléltető: (4796, 22)
>>> orders.order_id.is_unique
True
>>> merged[["order_id", "line_id"]].duplicated().any()
False
```

A shape azt mutatja, hogy a join eredménye a tételek szerinti 4796 soros szemcsézettségű, nem a rendelések szerinti 2400 soros. A rendelési kulcs a join után természetesen nem egyedi; a tételek összetett kulcsa viszont egyedi.

4.2.3.1 Rögzített debugger-demó köztes állapotokkal

A `lab/debug_demo.py` szándékosan hibás: a szállítási díjat minden tételsorra hozzáadja, pedig rendelésenként egyszer kellene. A `--trace` kapcsoló a ciklus köztes állapotát írja ki, a `pdb` pedig ugyanezeket a változókat mutatja meg a töréspontonon. A demó a könyv gyökeréből:

```
python3 lab/debug_demo.py --trace
# {"order_id": "O1", "n_items": 2, "shipping": 500, "line_sum": 3000, "subtotal": 4000}
# {"order_id": "O2", "n_items": 1, "shipping": 300, "line_sum": 1500, "subtotal": 1800}
# {"total": 5800, "expected": 5300, "bug": 500}
```

Ugyanez `pdb`-ben, a 35. sor töréspontjával (a `lab/` könyvtárból):

```
$ python3 -m pdb debug_demo.py
(Pdb) break debug_demo.py:35
(Pdb) run
(Pdb) continue
> debug_demo.py(35)shipping_total()
-> subtotal = line_sum + order["shipping_fee_minor"] * len(order_items)
(Pdb) p order["order_id"]
'O1'
(Pdb) p len(order_items)
2
(Pdb) p order["shipping_fee_minor"]
500
(Pdb) p line_sum
3000
(Pdb) next
(Pdb) p subtotal
4000
(Pdb) continue
> debug_demo.py(35)shipping_total()
(Pdb) p order["order_id"]
'O2'
(Pdb) p len(order_items)
1
(Pdb) p line_sum
1500
(Pdb) next
(Pdb) p subtotal
```

```
1800
(Pdb) continue
# total=5800, expected=5300, bug=500
```

A két megállás rögzített állapota:

Megállás	order_id	n_items	shipping	line_sum	subtotal
1	O1	2	500	3000	4000
2	O2	1	300	1500	1800

A hiba a 4000-es részösszegben látszik: a 3000-es tételösszeghez 500 helyett 1000 került, mert `500 * len(order_items) = 1000`. A javítás:

```
subtotal = line_sum + order["shipping_fee_minor"]
```

A javított érték 5300; a különbség pontosan az O1 második tételsorára is felszámított 500. VS Code-ban ugyanez a demó a következő `launch.json` konfigurációval indítható; a töréspontot a 35. sorra kell tenni, a watch ablakba pedig `order["order_id"]`, `len(order_items)`, `order["shipping_fee_minor"]`, `line_sum` és `subtotal` kerül:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Northstar debug demo",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/lab/debug_demo.py",
      "args": ["--trace"],
      "console": "integratedTerminal",
      "justMyCode": true
    }
  ]
}
```

A `tests/test_debug_demo.py` a `--trace` kimenetét, a 4000/1800 részösszegeket, az 5800-as hibás és az 5300-as helyes összeget, valamint a javított kifejezést ellenőrzi. A demó a hibás és a javított kódot is tartalmazza, így a debugger lépései nem csak képernyőképen, hanem parancssori szövegként is reprodukálhatók.

4.2.4 Eredmény értelmezése

A mini példában kézzel is ellenőrizhető a kontroll: az A rendeléshez $n_A = 1$, a B rendeléshez $n_B = 3$ tételsor tartozik. A hibás képlet $1 \cdot 200 + 3 \cdot 200 = 800$ centet ad, a helyes $200 + 200 = 400$ centet. A labor adatán a laborfutás után a `wrong` nagyobb lesz a `right` értékénél, ha létezik egynél több tételes rendelés; a `duplication` szorzó értéke éppen az adott rendelés tételsorainak száma. A fájl szerződéséből 2400 rendelés és 4796 tétel következik, tehát sok rendelés több tételes; a pontos szorzómegoszlást nem becslésből, hanem a futásból kapod meg.

A helyes rendelésszintű összeghez vagy az eredeti `orders` táblát kell használni, vagy a duplikált csoportból egyetlen reprezentáns sort kell venni:

```
correct_from_joined = (
    merged.groupby("order_id")["shipping_fee_minor"].last().sum()
)
```

A `.last()` azért helyes, mert a rendelésszintű oszlop a csoport minden sorában ugyanaz; a `.sum()` viszont a tételszám szorzóval adná vissza. Az ellenőrzés pontos, mert a pénz egész minor unitban van, tehát itt nincs lebegőpontos tolerancia: az egyenlőség egész számokra értendő. Ez az invariant nem csak a szállítási díjra, hanem a `basket_minor` rendelésszintű oszlopára is igaz.

A VS Code-debugger nélkül ugyanezt a logikát gépi assertté is alakíthatod. A labor átadandó ellenőrzése legyen a `check_join` függvény:

```
def check_join(orders, items, merged):
    assert orders["order_id"].is_unique, "orders.order_id nem egyedi"
    assert not items.duplicated(["order_id", "line_id"]).any(), \
        "tételkulcs nem egyedi"
    assert merged.shape[0] == items.shape[0], \
        "a join sorainak száma nem a tételek sorainak száma"
    assert not merged[["order_id", "line_id"]].duplicated().any(), \
        "a joined tételkulcs nem egyedi"
    before = orders["shipping_fee_minor"].sum()
    after = merged.groupby("order_id")["shipping_fee_minor"].last().sum()
    assert before == after, \
        f"szállítási díj duplázódott: {before} != {after}"
    return True
```

A négy invariáns rendre a rendeléskulcs egyediségét, a tételkulcs egyediségét, a join sorainak szemcsézettségét és a pénzösszeg változatlanságát rögzíti.

Hibakeresés statisztikai kontrollal. A debugger megmutathatja a join után létrejött sorok számát, de nem dönti el önmagában, hogy a sorok a megfelelő megfigyelési egységet jelentik. A két tábla illesztése után ellenőrizzük a kulcsok egyediségét, az egy rendelésre jutó tételszámot és a rendelésszintű összegek változatlanságát.

Ellenpélda: ha egy rendeléshez három tételsor tartozik, akkor a rendelésszintű szállítási díj soronkénti összeadása háromszor számolja ugyanazt a díjat. A helyes számítás előbb tételszinten aggregál, majd rendelésszinten csatlakoztat; a debugger mellé invariánsteszt is kell.

4.2.5 Feladatok

1. **Értelmezés.** A mini példa `mini_orders` táblájában az A rendeléshez 1, a B rendeléshez 3 tételsor tartozik, a szállítási díj mindkettőnél 200 cent. Számold ki kézzel a join sorainak számát, a kulcsegyediséget és a hibás, illetve helyes szállítási összeget. Írd fel a S_{rossz} összeg általános képletét a n_i tételszámokkal.

2. **Megvalósítás.** Egészítsd ki a `check_join` függvényt egy ötödik invariánssal: a `basket_minor` rendelésszintű kosárösszegnek is változatlannak kell maradnia a tételszintű join után. Futtasd a függvényt a mini példán és a labor tiszta `orders`/`order_items` adatán. A teljes ellenőrző script neve legyen `check_join.py`.

3. **Hibakeresés.** Egy munkatárs a teljes laboron ezt írta: `merged["shipping_fee_minor"].sum()`, és a szállítási összeg a vártnak körülbelül a kétszerese lett. Hova teszel töréspontot a `debug_join.py` scriptben, melyik négy kifejezést írod a Watch panelbe, és mi a legvalószínűbb magyarázat? Ne használj hosszú print-kimenetet.

4. **Megváltozott feltétel.** Tegyük fel, hogy egy `order_items.csv` sorból hiányzik egy tétel, és a munkatárs `how="left"` joinnel dolgozik. Adj konkrét mini bemenetet három rendeléssel, amelyek közül az egyikhez egyáltalán nem tartozik tétel. Mennyi lesz a join `shape[0]` értéke, mit jelez a `check_join`, és változik-e a `.last().sum()` szállítási összeg?

4.2.6 Megoldások és indoklás

1. A join a B rendelést háromszor ismétli, ezért $N_J = 4$ sor keletkezik. A `(order_id, line_id)` párok: A1, B1, B2, B3, mind egyedi. A hibás összeg $1 \cdot 200 + 3 \cdot 200 = 800$, a helyes $200 + 200 = 400$ cent. Az általános képlet $S_{\text{rossz}} = \sum_i n_i s_i$, ahol n_i az adott rendelés tételsorainak száma, s_i pedig a rendelésszintű szállítási díj.

2. Az ötödik invariant:

   ```
   basket_before = orders["basket_minor"].sum()
   basket_after = merged.groupby("order_id")["basket_minor"].last().sum()
   assert basket_before == basket_after, \
       f"basket_minor duplázódott: {basket_before} != {basket_after}"
   ```

   A mini példán nincs `basket_minor` oszlop, ezért a mini verzióhoz előbb oszloppal kell kiegészíteni, például `basket_minor = [950, 1700]`. A labor tiszta adatán a `check_join.py` futtatásakor mind az öt assertnek teljesülnie kell; bármelyik megszegése a feltételezés hibáját jelzi.

3. Töréspontot a `print("betöltve:", ...)` sorra kell tenni, mert ott mindkét bemeneti tábla és a `merged` is elérhető. Watch kifejezések: `orders.shape`, `items.shape`, `merged.shape`, `items.duplicated(["order_id", "line_id"]).any()`. A `merged.shape` sorszáma várhatóan 4796, azaz a tételszint; a `.sum()` ezért a rendelésszintű oszlopot minden tételsorban beszámítja. A munkatárs által kapott „körülbelül kétszeres” az átlagos tételszámnak felel meg, de a pontos hányados rendelésenként a tételszám.

4. Konkrét bemenet:

   ```
   mini_orders = pd.DataFrame({
       "order_id": ["A", "B", "C"],
       "shipping_fee_minor": [100, 200, 300],
   })
   mini_items = pd.DataFrame({
       "order_id": ["A", "B", "B"],
       "line_id": [1, 1, 2],
   })
   merged_left = mini_orders.merge(
       mini_items, on="order_id", how="left", validate="one_to_many"
   )
   ```

   A `merged_left.shape[0]` értéke 4, a `mini_items.shape[0]` viszont 3, ezért a `check_join` shape-invariánsa hibát jelez. A `.last().sum()` szállítási összeg 600 cent marad, tehát a pénzösszeg-invariáns önmagában nem veszi észre a hiányzó tételt; ezért is kell a shape-ellenőrzés külön. A nullákat a join után a hiányzó tétel oszlopaiban a dtype is mutatja (`NaN`).

4.2.7 Gyakori tévedés ellenpéldával

A hosszú print-kimenet nem helyettesíti a megfogalmazott invariánst. A mini példában a `print(mini_merged)` kimenete négysoros, és a szem hamar észreveszi, hogy a B rendelés három sorban szerepel. A labor 4796 soránál ugyanez a hiba gyakran nem feltűnő, mert a duplikált rendelések közé ékelődnek a különböző tételek, és a képernyőn kigördülő részletből nem tudható, hány rendelés ismétlődik. A `print(mini_merged["shipping_fee_minor"].sum())` egyetlen 800-as számot ad; ha nem tudjuk, hogy a helyes összeg 400, a szám önmagában nem hibás. Az invariant viszont egyenlőségként mondja ki, mit várunk: a join előtti és utáni rendelésszintű

összegnek egyeznie kell. A `check_join` assertjei akkor is megállítják a futást, ha a hiba a tábla közepén van.

A korábbi 4.1 figyelmeztetése ide is átnyúlik: a notebook rejtett állapotához hasonlóan a tábla mérete is „csendes” tulajdonság. A cella lefutása nem bizonyítja, hogy a benne lévő számítás a kívánt szemcsézettségen dolgozott; a `shape`, a `dtype` és a kulcsegyediség ellenőrzése viszont a futás közbeni igazságot rögzíti.

4.2.8 Önellenőrzés

- Meg tudod mondani, miért nem az `orders.shape[0]` a belső join sorainak száma?
- Mi a különbség a VS Code interpreter és a notebook kernel kiválasztása között?
- Melyik watch kifejezés mutatja meg, hogy a join után a tételkulcs egyedi?
- Miért a `.last()`, és miért nem a `.sum()` adja a rendelésszintű szállítási díjat a join után?

4.2.9 Rövid összefoglaló

A VS Code-debugger a kiválasztott projektinterpreterrel futtat, és töréspontnál megállítja a programot. A Watch panelben rögzített `shape`, `dtype` és kulcsegyediség kifejezésekkel a tábla tényleges szemcsézettségét vizsgáljuk, nem a feltételezésünket. A Northstar join példájában a belső join sorainak száma a tételek száma, ezért a rendelésszintű szállítási díj minden rendelésnél a tételszám szorzóval ismétlődik; a helyes összeghez a rendelésenkénti egyetlen reprezentáns sor kell. A hosszú print helyett a `check_join` assertjei rögzítik a shape- és pénzösszeg-invariánst. A következő alfejezetben a notebookból készítünk tiszta scriptet és jelentést.

4.3 Notebookból script és jelentés

Belépő · szint: alap · idő: 45 perc · eszköz: JupyterLab, VS Code

Előfeltétel: 4.2. alfejezet · Célkészség: Notebookból script és jelentés · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

4.3.1 Cél és konkrét helyzet

A 4.2. alfejezetben a debuggerrel kimutattuk, hogy a Northstar `orders.csv` és `order_items.csv` táblák hibás joinolásakor a rendelésszintű `shipping_fee_minor` minden tételnél megismétlődik. Most ezt a javított aggregációt kiemeljük egy tiszta Python-modulba, hogy ugyanazt a mutatót használhassa a notebook, a parancssori script és a statikus jelentés is.

A konkrét bemenet a kísérő labor két fájlja:

- `orders.csv`: egy sor egy rendelés; 2400 sor az alapseeddel. Rendelésszintű mezői között van `customer_id`, `order_id`, `basket_minor`, `shipping_fee_minor`, `promised_days=7` és a rendelés időpontja.
- `order_items.csv`: egy sor egy rendelési tétel; 4796 sor az alapseeddel. Kulcsa `(order_id, line_id)`. A tétel sora tartalmazza a `line_total_minor = quantity * unit_price_minor - discount_minor` értéket EUR-centben.

A megfigyelési egység most a rendelés. A cél egy rendelésszintű `total_minor` mutató előállítása úgy, hogy a kosárérték és a szállítási díj egyszer szerepeljen minden rendelésnél. Az ellenőrzési feltétel kettős:

1. A kimeneti tábla `shipping_fee_minor` összege egyenlő a bemeneti `orders.csv` `shipping_fee_minor` összegével.
2. A tételekből számított kosárösszeg minden rendelésnél egyenlő a `basket_minor` mezővel, tehát az eltérés pontosan 0.

A mutatót egyetlen helyen definiáljuk:

$$total_{minor,i} = basket_{minor,i} + shipping_fee_{minor,i},$$

ahol $i = 1, \ldots, N$ a rendelések indexe, N a rendelések száma. A `basket_minor` és `shipping_fee_minor` egész EUR-centben adott, ezért a `total_minor` is egész EUR-cent; a kimenő vektor alakja `(N,)`. A `basket_minor` maga is ellenőrizhető a tételekből:

$$basket_{minor,i} = \sum_{j \in T_i} line_total_{minor,j},$$

ahol T_i az i-edik rendeléshez tartozó tételek halmaza. Ezt a definíciót kell megőrizni a notebook, a CLI és a jelentés között.

Előzetes becslés: a 4796 tétel 2400 rendeléshez tartozik, tehát a tételszintű join után a rendelések átlagos tételszáma körülbelül $4796/2400 \approx 2$. A nyers, rendelés és tétel közötti joinban a szállítási díj körülbelül kétszereződne; a helyes aggregáció után a szállításidíj-összeg pontosan egyezik a rendeléstábláéval.

4.3.2 Fogalmi magyarázat

Tiszta függvény (pure function). Egy függvény akkor tiszta, ha ugyanarra a bemenetre mindig ugyanazt a kimenetet adja, és nincs a függvényen kívüli mellékhatása: nem ír fájlt, nem olvas globális változót, nem módosítja a bemeneti táblát. A `compute_order_totals(orders, item_aggr)` tiszta függvény: bemeneti DataFrame-ekből új DataFrame-et ad. Ezt tudjuk egységteszttel ellenőrizni.

Modul és belépési pont. A Python-modul importálható `.py` fájl. A `northstar/metrics.py` tartalmazza a tiszta függvényeket, a `northstar/cli.py` pedig az `if __name__ == "__main__":` blokkban hívja ezeket. Így a fájl scriptként és importként is viselkedhet anélkül, hogy a számítás két helyen élne.

Notebook hívókód. A notebook nem újraszámolja a mutatót; ugyanazt a modult importálja, mint a CLI. A notebook cellái lehetnek felfedező jellegűek, de a termelési logika a modulban marad.

nbconvert. Az `nbconvert --to script` a notebookot Python-fájllá konvertálja, de a cellákat általában lineárisan összefűzi; ez nem tiszta függvény, és a cellák közötti rejtett függőségeket megőrzi.

Quarto. A `.qmd` forrás Markdown-szöveget és futtatható kódblokkokat tartalmaz. A `quarto render` parancs a helyi Jupyter-kernellel futtatja a kódblokkokat, és statikus HTML/PDF jelentést készít. A jelentés a `northstar.metrics` modulból dolgozik, tehát a mutatódefiníció ismét nem másolódik.

Ugyanaz a mutató három felületen akkor marad konzisztens, ha mindegyik ugyanazt a modulbeli definíciót importálja:

- notebook: `from northstar.metrics import compute_order_totals`
- CLI: ugyanez a modul
- Quarto chunk: ugyanez a modul

Ha a definíciót három helyre bemásolnánk, bármelyik frissítés elmaradhat, és a három felület néma eltérést mutatna.

4.3.3 Teljes minimális munkapélda

Hozzuk létre a lab gyökerében a következő struktúrát:

```
lab/
  data/
    orders.csv
    order_items.csv
  northstar/
    __init__.py
    metrics.py
    cli.py
  notebooks/
    explore.ipynb
  summary.qmd
  artifacts/
```

A `northstar/metrics.py` tartalma:

```python
# northstar/metrics.py
"""Egységes rendelésszintű mutatószámítás a Northstar labhoz.

Minden pénzmennyiség egész EUR-centben van (minor unit).
"""

import pandas as pd


def aggregate_items(items: pd.DataFrame) -> pd.DataFrame:
    """Tételsorokat aggregál rendelésszintre.

    Bemenet:
      items: olyan DataFrame, amelyben minden sor egy rendelési tétel.
             Szükséges oszlopok: order_id, line_id, line_total_minor,
             quantity.
    Kimenet:
      DataFrame, amelyben minden order_id egyetlen sor; oszlopai:
      derived_basket_minor, item_quantity_sum, item_rows.

    Invariáns:
      derived_basket_minor == sum(line_total_minor) az adott rendelésre.
    """
    required = {"order_id", "line_id", "line_total_minor", "quantity"}
    if not required.issubset(items.columns):
        missing = required - set(items.columns)
        raise ValueError(f"Hiányzó tétel oszlopok: {sorted(missing)}")

    item_aggr = (
        items.groupby("order_id", as_index=False)
        .agg(
            derived_basket_minor=("line_total_minor", "sum"),
```

```
            item_quantity_sum=("quantity", "sum"),
            item_rows=("line_id", "count"),
        )
    )
    return item_aggr


def compute_order_totals(
    orders: pd.DataFrame, item_aggr: pd.DataFrame
) -> pd.DataFrame:
    """Rendelésszintű teljes összeg a szállítási díj egyszeri beszámításával.

    Bemenet:
      orders: rendelésenként egyetlen sor; oszlopai order_id,
              basket_minor, shipping_fee_minor.
      item_aggr: az aggregate_items kimenete.
    Kimenet:
      one-to-one join rendelésenként; a total_minor oszlop az
      basket_minor + shipping_fee_minor összeget adja,
      a basket_check_minor pedig a tételekből számított és a rendelésbeli
      kosárérték különbségét.
    """
    required = {"order_id", "basket_minor", "shipping_fee_minor"}
    if not required.issubset(orders.columns):
        missing = required - set(orders.columns)
        raise ValueError(f"Hiányzó rendelés oszlopok: {sorted(missing)}")

    if orders["order_id"].duplicated().any():
        raise ValueError("Az orders.order_id nem egyedi")

    if item_aggr["order_id"].duplicated().any():
        raise ValueError("Az item_aggr.order_id nem egyedi")

    merged = orders.merge(
        item_aggr, on="order_id", how="left", validate="one_to_one"
    )
    merged["basket_check_minor"] = (
        merged["derived_basket_minor"] - merged["basket_minor"]
    )
    merged["total_minor"] = (
        merged["basket_minor"] + merged["shipping_fee_minor"]
    )
    return merged
```

A northstar/cli.py:

```
# northstar/cli.py
"""Parancssori felület a rendelésösszeg-adat előállításához."""

import argparse
import pandas as pd

from .metrics import aggregate_items, compute_order_totals


def main(argv=None) -> int:
    parser = argparse.ArgumentParser(
        description="Rendelésszintű total_minor adatot állít elő."
    )
    parser.add_argument("--orders", default="data/orders.csv")
    parser.add_argument("--items", default="data/order_items.csv")
    parser.add_argument("--output", default="artifacts/order_totals.csv")
    args = parser.parse_args(argv)
```

```
    orders = pd.read_csv(args.orders)
    items = pd.read_csv(args.items)

    item_aggr = aggregate_items(items)
    totals = compute_order_totals(orders, item_aggr)

    shipping_in = int(orders["shipping_fee_minor"].sum())
    shipping_out = int(totals["shipping_fee_minor"].sum())
    basket_rows_ok = bool((totals["basket_check_minor"] == 0).all())

    if shipping_in != shipping_out or not basket_rows_ok:
        print(
            "INVARIANT_FAILED "
            f"shipping_in={shipping_in} shipping_out={shipping_out} "
            f"basket_rows_ok={basket_rows_ok}"
        )
        return 2

    totals.to_csv(args.output, index=False)
    print(
        f"OK rows={len(totals)} "
        f"shipping_in_minor={shipping_in} shipping_out_minor={shipping_out}"
    )
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Futtassuk a lab gyökeréből:

```
# bash vagy Windows PowerShell; munkakönyvtár: lab/
mkdir -p artifacts
python -m northstar.cli \
  --orders data/orders.csv \
  --items data/order_items.csv \
  --output artifacts/order_totals.csv
```

Szemléltető kimenet; a konkrét `shipping_in_minor` érték a helyi generált adatból adódik, ezért itt helykitöltővel jelöljük:

```
OK rows=2400 shipping_in_minor=<helyi mért érték> shipping_out_minor=<helyi mért érték>
```

A notebook `explore.ipynb` egy cellája ugyanezt a modult hívja:

```
import pandas as pd
from northstar.metrics import aggregate_items, compute_order_totals

orders = pd.read_csv("data/orders.csv")
items = pd.read_csv("data/order_items.csv")

item_aggr = aggregate_items(items)
totals = compute_order_totals(orders, item_aggr)

assert int(totals["shipping_fee_minor"].sum()) == int(
    orders["shipping_fee_minor"].sum()
)
assert (totals["basket_check_minor"] == 0).all()

totals[
    ["order_id", "basket_minor", "shipping_fee_minor", "total_minor"]
].head()
```

Ez a notebook hívókód nem számol újra; az `assert` a modul kimenetét ellenőrzi. A notebook ezen túl lehet felfedező: például hisztogramot rajzol a `total_minor` eloszlásáról.

A notebook `--to script` exportja:

```
# bash; munkakönyvtár: lab/
jupyter nbconvert --to script notebooks/explore.ipynb --output-dir artifacts/
```

Az így készült `explore.py` a cellákat sorba fűzi. Szemrevételezve látszik, hogy a `compute_order_totals` nem a notebookban lakik, hanem importálva van. Ez a különbség a fenntarthatóság alapja.

A Quarto forrás `summary.qmd`:

````
---
title: "Rendelésösszeg-ellenőrző jelentés"
format: html
execute:
  echo: true
---

A jelentés a `northstar.metrics.compute_order_totals` függvényt használja.
Minden rendelésre a mutató definíciója:

$$total_{minor,i} = basket_{minor,i} + shipping\_fee_{minor,i},$$

ahol az $i$ index a rendelés, a pénznem EUR, az egység egész EUR-cent.

```{python}
import pandas as pd
from northstar.metrics import aggregate_items, compute_order_totals

orders = pd.read_csv("data/orders.csv")
items = pd.read_csv("data/order_items.csv")

item_aggr = aggregate_items(items)
totals = compute_order_totals(orders, item_aggr)

assert int(totals["shipping_fee_minor"].sum()) == int(
    orders["shipping_fee_minor"].sum()
)
assert (totals["basket_check_minor"] == 0).all()

totals[
    ["order_id", "basket_minor", "shipping_fee_minor", "total_minor"]
].head()
```
````

Renderelés a lab gyökeréből:

```
# bash; munkakönyvtár: lab/
quarto render summary.qmd
```

Szemléltető renderkimenet: a Quarto lefuttatja a Python-chunkot, és `summary.html` áll elő. Az aktuális `nbconvert` és Quarto argumentumok a telepített verziónál `--help` kapcsolóval ellenőrizhetők; a `reference.md` és `sources.md` forráskulcsai további kiindulópontot adnak.

4.3.3.1 Teljes forrásfájl-szett és futtatás

A notebook–script átmenet akkor ellenőrizhető, ha a teljes fájlkészlet együtt van. A 4.4 megoldásában:

Fájl	Szerep
`lab/checkpoints/cp1/solution/notebooks/explore.ipynb`	a felfedező notebook
`lab/checkpoints/cp1/solution/scripts/explore.py`	a script-változat
`lab/checkpoints/cp1/solution/northstar/metrics.py`	a triviális aggregáció és a join
`lab/checkpoints/cp1/solution/northstar/report.py`	a közös `build_report()`
`lab/data/orders.csv`, `order_items.csv`	az adatkontraktus

A notebook és a script ugyanazt a `build_report()` függvényt hívja, ezért a számítás nem duplikálódik. Futtatás a lab gyökeréből:

```
cd lab
export PYTHONPATH="$PWD/checkpoints/cp1/solution"
uv run python checkpoints/cp1/solution/scripts/explore.py \
    --data data --output /tmp/explore-report.json
uv run jupyter nbconvert --to notebook --execute --inplace \
    checkpoints/cp1/solution/notebooks/explore.ipynb
```

A script mért összegzése a lab adatain: `orders=2400`, `basket_check_zero=2400`, `total_minor_sum=99073349`. A notebook setup cellája a `pyproject.toml` alapján keresi a projektgyökeret, ezért a `data/orders.csv` útvonal nem függ a Jupyter indítási könyvtárától. Az egyezés ellenőrzése: ugyanaz a `build_report()` hívás, ugyanaz az adat, ugyanaz a három szám; a `tests/test_notebook_script_equivalence.py` a script kimenetét és a notebook `build_report` hívását ellenőrzi.

4.3.4 Eredmény értelmezése

Ellenőrizzük a mutatót egy kézi mini példán, mielőtt a 2400 soros táblára bíznánk. Legyen két rendelés:

order_id	basket_minor	shipping_fee_minor
ORD-A	100	20
ORD-B	40	10

A tételek:

order_id	line_id	line_total_minor	quantity
ORD-A	1	60	2
ORD-A	2	40	1
ORD-B	1	40	5

A `compute_order_totals` kimenete:

order_id	derived_basket_minor	total_minor	basket_check_minor
ORD-A	100	120	0
ORD-B	40	50	0

A két ellenőrző invariant most is teljesül: a `shipping_fee_minor` összege $20 + 10 = 30$ a rendeléstáblában és a kimenetben egyaránt; a `basket_check_minor` oszlop mindenütt 0. Mivel az adatok egész EUR-centben vannak, nincs lebegőpontos kerekítési tolerancia; az összehasonlítás pontos, egészértékű.

A `total_minor` mértékegysége EUR-cent, a rendelésenkénti sor a `(2400,)` alakú. A notebook, a CLI és a Quarto jelentés ugyanazt a modulbeli definíciót hívja, ezért ha a mutató képlete később megváltozik, elegendő a `metrics.py`-ben módosítani.

Azonos szemantika három felületen. Egy mutató közös modulba emelése szükséges, de nem elégséges: a notebook, CLI és jelentés azonos bemeneti adatkészletet, azonos verziójú modult és azonos konfigurációt is használjon. Külön teszt igazolja, hogy a három felület ugyanazt a `total_minor` vektort adja, ne csupán hasonló összegzést.

Az `nbconvert --to script` export nem bizonyítja a reprodukálható futást. A Quarto-renderhez a választott formátum (HTML vagy PDF) külön előfeltételei tartozhatnak, PDF-nél például megfelelő TeX-rendszer; a leírásban ne ígérjünk formátumfüggetlen sikert ellenőrzés nélkül.

4.3.5 Feladatok

1. **Értelmezés.** Az `nbconvert --to script` kimenete miért nem helyettesíti a `northstar/metrics.py` modult? Nevezd meg egy konkrét notebook-jellemzőt, amely az exportált scriptben rejtett állapotot okozhat.
2. **Megvalósítás.** Írj `average_total_by_segment(orders, item_aggr, customers)` függvényt a modulba, amely a `customers.csv` `segment` oszlopával csoportosítva add meg a `total_minor` rendelésenkénti átlagát. A notebook jelenítse meg a csoportátlagokat, a CLI pedig kapjon `--summary` kapcsolót. Bemenet: `orders.csv`, `order_items.csv`, `customers.csv`; az utóbbi egy sora egy ügyfél, `customer_id` kulccsal, `segment` értéke `consumer` vagy `business`.
3. **Hibadiagnózis.** A notebookban valaki ezt a cellát futtatta:

   ```
   merged = orders.merge(
       items, on="order_id", how="left", validate="one_to_one"
   )
   ```

   és `ValueError` keletkezett. Melyik oldal nem egyedi, miért, és melyik modullal kell elkerülni a hibát? Ellenőrizd a 4.2. alfejezetben tanult `nunique()` és `duplicated().sum()` kifejezésekkel is.
4. **Megváltozott feltétel.** Az új `orders.csv` változatból hiányzik a `basket_minor` oszlop; a szerződés szerint a tételekből származtatott `derived_basket_minor` a mérvadó. Módosítsd a `compute_order_totals` függvényt úgy, hogy a `basket_minor` hiánya esetén a `derived_basket_minor` kerüljön a `total_minor` képletébe, és a különbség-ellenőrzés csak akkor fusson, ha a `basket_minor` oszlop jelen van. Mi lesz az új invariant a szállítási díjra?

4.3.6 Megoldások és indoklás

1. Az `nbconvert --to script` a notebook celláit sorrendben egy `.py` fájlba írja, de nem hoz létre paraméterezhető interfészt. A notebook tartalmazhat globális változókat, például egy `pd.set_option("display.max_rows", 5)` hívást vagy egy korábbi cellában definiált `shipping_fee_minor` változót; ezek az exportált scriptben is globálisan megmaradnak, és ha a scriptet importálják vagy más sorrendben futtatják, az eredmény megváltozhat. A modul ezzel szemben a `compute_order_totals` függvényt egyértelmű paraméterlistával definiálja, és nem függ a notebook celláinak sorrendjétől.

2. A modulba kerülő függvény:

   ```python
   def average_total_by_segment(
       totals: pd.DataFrame, customers: pd.DataFrame
   ) -> pd.DataFrame:
       required = {"customer_id", "total_minor"}
       if not required.issubset(totals.columns):
           raise ValueError("A totals táblából hiányzik a customer_id vagy total_minor")
       joined = totals.merge(
           customers[["customer_id", "segment"]],
           on="customer_id",
           how="left",
           validate="many_to_one",
       )
       return (
           joined.groupby("segment", as_index=False)
           .agg(
               order_count=("order_id", "count"),
               average_total_minor=("total_minor", "mean"),
               total_sum_minor=("total_minor", "sum"),
           )
       )
   ```

   A notebookban:

   ```python
   from northstar.metrics import average_total_by_segment

   customers = pd.read_csv("customers.csv")
   average_total_by_segment(totals, customers)
   ```

   A CLI `--summary` kapcsolója hívja ugyanezt, és kiírja a csoportátlagokat. A csoportátlagok súlyozott átlaga meg kell egyezzen a teljes `total_minor` átlaggal, ha a csoportok lefedik a táblát; ez a felület konzisztenciájának ellenőrző invariantja.

3. Az `items` tábla tételszintű, tehát az `order_id` benne nem egyedi: egy rendeléshez több tételsor tartozik. A `validate="one_to_one"` pontosan ezt a sérülést jelzi a jobb oldali táblán. Az ellenőrzés:

   ```python
   print(len(items), items["order_id"].nunique())
   print(int(items["order_id"].duplicated().sum()))
   ```

   Az első sor mutatja, hogy a sorok száma nagyobb az egyedi rendelések számánál; a második a duplikált `order_id` sorok számát adja. A helyes megoldás az `aggregate_items(items)` hívása, majd a visszaadott rendelésszintű táblán a `validate="one_to_one"` join; így a jobb oldal már egyedi.

4. A módosított függvény:

```
def compute_order_totals(orders, item_aggr):
    if "basket_minor" in orders.columns:
        merged = orders.copy()
        has_basket = True
    else:
        merged = orders.copy()
        has_basket = False

    merged = merged.merge(
        item_aggr, on="order_id", how="left", validate="one_to_one"
    )
    if has_basket:
        merged["basket_check_minor"] = (
            merged["derived_basket_minor"] - merged["basket_minor"]
        )
        merged["total_minor"] = (
            merged["basket_minor"] + merged["shipping_fee_minor"]
        )
    else:
        merged["total_minor"] = (
            merged["derived_basket_minor"] + merged["shipping_fee_minor"]
        )
    return merged
```

Az új invariant: a `shipping_fee_minor` összege továbbra is egyezik a rendeléstábla `shipping_fee_minor` összegével, mert a join továbbra is one-to-one. A kosárérték most a tételekből származik; a régi `basket_minor` ellenőrzés csak akkor alkalmazható, ha az oszlop létezik.

4.3.7 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés: *„A notebookból `nbconvert --to script`-tel előállított fájl már karbantartható program."* Ez nem igaz. Az export nem hoz létre modul- és függvényhatárt, és a notebook globális cellái megmaradnak. Ellenpélda: tegyük fel, hogy a notebook harmadik cellája `pd.set_option("display.max_rows", 5)` hívást tartalmaz, a negyedik pedig egy `basket_minor = orders["basket_minor"] * 1` sorral „teszteli" a kerekítést. A két cella exportálva is globális hatású marad; ha később a scriptet egy másik munkafolyamat importálja, a display-beállítás és a véletlen globális név mellékhatássá válik. A tiszta modul ezzel szemben csak a paraméterekből dolgozik, nincs rejtett állapota.

Önellenőrzés: hány helyen szerepel a `total_minor = basket_minor + shipping_fee_minor` képlet az ideális felépítésben? A válasz: pontosan egy helyen, a `northstar/metrics.py` modulban; a notebook, a CLI és a Quarto-chunk csak importálja ezt a definíciót.

4.3.8 Rövid összefoglaló

A notebookból kiemelt `northstar.metrics` modul egy helyen tartalmazza a rendelésszintű mutatót, így a `total_minor = basket_minor + shipping_fee_minor` definíciót a notebook, a CLI és a Quarto jelentés is ugyanazzal a függvénnyel számolja. Az `aggregate_items` előbb rendelésszintre hozza a tételeket, a `compute_order_totals` pedig one-to-one joinnal kerüli el a szállítási díj duplázódását. A CLI `if __name__ == "__main__"` blokkja és az `argparse` teszi scriptté a modult; az `nbconvert` exportja nem helyettesíti a tiszta interface-t. A következő alfejezetben az I. ellenőrző pont a teljes munkaasztalt újra futtatja: modult, notebookot, parancssori hívást és jelentést együtt ellenőrzi.

4.4 I. ellenőrző pont: megismételhető kutatói munkaasztal

Belépő · szint: alap · idő: 45 perc · eszköz: JupyterLab, VS Code

Előfeltétel: 4.3. alfejezet · Célkészség: I. ellenőrző pont: megismételhető kutatói munkaasztal · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

4.4.1 Helyzet és cél: a munkaasztal mint átadott termék

Egy kolléga a Northstar tanulóprojektet adja át neked: a `lab/` mappát, benne a determinisztikus adatgenerátorral, a 4.3-ban létrehozott `northstar` modullal, a kezdő elemző notebookkal és a `data/manifest.json` fájllal. A te feladatod tiszta mappában rekonstruálni a környezetet, legenerálni vagy ellenőrizni az adatot, lefuttatni a notebookot, és a futást visszakereshető jegyzékben dokumentálni. A kolléga egyetlen belépő parancsot kér: aki klónozza a repót, annak ne kelljen fejben tartania a lépések sorrendjét. A kiinduló vázlat és a referencia-megoldás a `lab/checkpoints/cp1/` mappában van (`starter/`, illetve `solution/`).

A megfigyelési egység itt nem ügyfél és nem rendelés, hanem egy ellenőrző futás egy adott gépen. A bemenet a projektfájl-készlet és a gép környezete; a kimenet a futási jegyzék, az újra futtatott notebook és a kontrollösszeg-eredmény. Az előzetes eredménybecslés: ha az átadott repo konzisztens, akkor a manifestben rögzített hashvizsgálat `failures=0` értékkel zár, a quickcheck a szerződésben rögzített sorszámokat adja vissza (300 ügyfél, 2400 rendelés, 4796 tételsor az alaps seed esetén), és a git-munkaállapot tiszta marad. A tényleges hashértékeket nem találgathatjuk előre; azokat a labor `manifest.json` fájlja tartalmazza.

4.4.2 Fogalmi alapok: hat réteg, egyetlen belépő paranccsal

A reprodukálható munkaasztal hat rétegből áll, és a belépő szkript feladata az, hogy ezeket sorban, hiba esetén korán álljon meg.

4.4.2.1 Projektgyökér: a parancsok vonatkoztatási pontja

A projektgyökér az a mappa, ahol a `pyproject.toml` található. Minden relatív útvonal innen értendő: `scripts/entrypoint.sh`, `northstar`, `data/manifest.json`. Ha más munkakönyvtárból indítod a parancsot, akkor nem a szkript hibás, hanem a hívási kontextus.

A belépő szkript ezért az első soraiban explicit horgonyt ellenőriz:

```
if [[ ! -f pyproject.toml ]]; then
  echo "Run this script from the lab project root." >&2
  exit 2
fi
```

A szabály: egy parancs akkor hordozható, ha a meghívás helyét a projektgyökér rögzíti, és a hívó nem támaszkodik rejtett, gépenként eltérő környezeti változóra.

4.4.2.2 Interpreter és függőségek: uv-run mint egyetlen Python-bejárat

A labor nem tartalmaz előre feloldott `uv.lock` fájlt, de a `reference/uv.lock` referenciafeloldás igen; az első tiszta futtatáskor a felbontást el kell végezni (vagy a referenciát a helyére másolni), utána viszont a lock már rögzített állapotot jelent. A uv projekt-sync dokumentációjának szerepe szerint a `--locked` kapcsoló azt ellenőrzi, hogy a lock és a `pyproject.toml` összhangban van-e; a `--frozen` ennél szigorúbb, és aktualitásvizsgálatot sem végez.

A belépő szkript logikája:

1. ha nincs `uv.lock`, készítsd el;
2. `uv sync --locked`;
3. minden további Python-parancs `uv run`-on keresztül fusson, hogy a projekt `.venv`-je legyen az interpreter.

A lokális `python3` parancs nem azonos a projekt interpreterével. Az átadás során mindig a projekt `.venv/bin/python` (Windowson `.venv\Scripts\python.exe`) az irányadó.

4.4.2.3 Kernel: a notebook is egy Python-folyamat

A 3.4 alfejezetben már láttuk, hogy a Jupyter-kernel egy külön folyamat, amelynek `argv[0]` eleme a Python-útvonal. A notebook akkor reprodukálható, ha ez az útvonal pontosan a projekt `.venv`-ére mutat. A belépő szkript ezért regisztrál egy kernelspecet, és az `nbconvert` végrehajtásnál ezt a kernelnevet adja meg:

```
uv run python -m ipykernel install --user \
  --name northstar-clean-handoff \
  --display-name "Northstar clean handoff"
```

A regisztráció önmagában nem futtat semmit; a notebook végrehajtása csak ezután következhet. A kernelválasztás és a `Restart and Run All` a JupyterLab felületén is ugyanezt a kernelt kell, hogy használja.

4.4.2.4 Git: a változás nyoma, nem a futási bizonyíték

A Git a forráskód, a notebook és a szkriptek verzióját őrzi. A `.venv/` és a `run/` mappa a `.gitignore`-ba tartozik, mert gépfüggő vagy újragenerálható állapot. A belépő szkript a végén `git status --porcelain=v1` kimenetet rögzít, hogy a fogadó gép naplójában látható legyen: a futtatás nem hagyott-e váratlan módosítást a követett fájlokban.

A `porcelain=v1` formátum azért hasznos, mert stabil, scriptből elemezhető kimenetet ad. A naplóba kerülő Git-állapot egy ellenőrző jel, nem a tudományos eredmény.

4.4.2.5 Futási jegyzék: kereshető, nem képernyőkép

A futási jegyzék nem más, mint a parancsok és kimeneteik időbélyegzett naplója. A minimálisan érdemes mezők:

Mező	Példa	Mit ellenőriz?
`run_id`	`20261020T083011Z`	egyedi futtatás
`python_executable`	`.../lab/.venv/bin/python`	projekt interpreter
`kernel_name`	`northstar-clean-handoff`	notebook kernel

Mező	Példa	Mit ellenőriz?
`hash_result`	`checked=6 failures=0`	fájl integritás

A naplót a `run/` mappába írjuk, a belépő szkript pedig `tee`-vel egyszerre mutatja és rögzíti a kimenetet. A jegyzéket akkor lehet hatékonyan használni, ha egy óra múlva `grep`-pelhető, nem akkor, ha egy képernyőképen zöld pipa látszik.

4.4.2.6 Kontrollösszeg és a matematikai híd

A kontrollösszeg itt SHA-256 üzenetkivonat (hash). Egy fájl bájtsorozatához rendelünk egy 64 hexadecimális jegyből álló szöveget:

$$H = \mathrm{SHA256}(f_1, f_2, \ldots, f_N),$$

ahol f_i a fájl i-edik bájtja, N a fájl mérete bájtban, H pedig a kimeneti kivonat. A kimenet 64 hexa jegy, tehát $64/2 = 32$ bájt, azaz $32 \cdot 8 = 256$ bit. Egy hexa jegy négy bitet kódol: például $0xF = 1111_2 = 15_{10}$, egy bájt pedig két hexa jegyből áll, $0x00$-tól $0xFF$-ig.

A hash nem „összeg” a hétköznapi értelemben. Egy egyszerű bájtsorrend-összegnél az $1, 2$ és a $2, 1$ sorozat azonos eredményt adhat, így a sorrendhiba láthatatlan marad. A SHA-256 konstrukciójának célja éppen az, hogy egyetlen bájt megváltozása is előrejelezhetetlenül más kimenetet adjon. Ez nem abszolút matematikai garancia az ütközésmentességre; a mi kontrollünkben az integritásvizsgálat egy manifesthez rögzített referenciaértékhez hasonlít, és a manifest maga is megbízhatósági kérdés.

4.4.3 Teljes minimális átadási munkamenet

Az itteni futási napló minden idő- és verzióadata szemléltető, nem a te gépeden mért kimenet. A tényleges futtatást a tanulói labor végzi; a hashértékeket kizárólag a labor manifestje tartalmazza.

A labor gyökerében az alábbi szerkezetet feltételezzük a 4.3 átadásából (a referencia-megoldás a `lab/checkpoints/cp1/solution/` alatt):

```
lab/
  pyproject.toml
  northstar/
      __init__.py
      metrics.py
      cli.py
  notebooks/explore.ipynb
  generate_data.py
  quickcheck.py
  data/
      manifest.json
      orders.csv
      order_items.csv
      ... (a többi CSV)
  reference/uv.lock
  scripts/
      entrypoint.sh
      verify_hashes.py
  run/
```

A `verify_hashes.py` egyetlen feladatot lát el: a manifestben rögzített elvárt SHA-256 értékeket összehasonlítja a tényleges fájlokkal.

```
"""Verify files against the SHA-256 hashes recorded in the lab manifest.

Two manifest schemas are accepted: ``{"files": {path: digest}}`` (paths
relative to the project root) and the lab's ``{"sha256": {name: digest}}``
(relative to the manifest's directory). An equal digest means the bytes are
almost certainly identical, but the statement is only as trustworthy as the
manifest itself.
"""

from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path

CHUNK = 1024 * 1024


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        while chunk := handle.read(CHUNK):
            digest.update(chunk)
    return digest.hexdigest()


def load_expected(manifest: Path) -> tuple[dict[str, str], Path]:
    payload = json.loads(manifest.read_text(encoding="utf-8"))
    if isinstance(payload.get("files"), dict):
        return payload["files"], Path(".")
    if isinstance(payload.get("sha256"), dict):
        return payload["sha256"], manifest.parent
    raise ValueError("manifest requires a 'files' or 'sha256' object")


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--manifest", type=Path, default=Path("data/manifest.json"))
    parser.add_argument("--root", type=Path, default=None)
    args = parser.parse_args()

    expected, default_root = load_expected(args.manifest)
    root = args.root if args.root is not None else default_root

    failures = 0
    checked = 0
    for relative, expected_digest in expected.items():
        target = (root / relative).resolve()
        checked += 1
        if not target.exists():
            print(f"{relative}: MISSING")
            failures += 1
            continue
        actual_digest = sha256(target)
        if actual_digest != expected_digest:
            print(f"{relative}: MISMATCH")
            failures += 1
            continue
        print(f"{relative}: OK")
```

```
    print(f"checked={checked} failures={failures}")
    return 0 if failures == 0 else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

A labor `data/manifest.json` fájlja a `sha256` kulcsot használja, ezért a belépő parancs `--manifest data/manifest.json --root data`; a `files` kulcsú séma a saját mini-projektekhez marad. A hash-eredmény összehasonlítása pontos egyezést kér: nincs numerikus tolerancia, mert bájtszintű integritásról van szó.

A belépő szkript Linux/macOS bash környezetre:

```
#!/usr/bin/env bash
# One-command handoff check for the Northstar lab.
# Run from the project root: bash scripts/entrypoint.sh
set -euo pipefail

if [[ ! -f pyproject.toml ]]; then
  echo "Run this script from the lab project root." >&2
  exit 2
fi

RUN_DIR="run"
STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
LOG="${RUN_DIR}/handoff-${STAMP}.log"
mkdir -p "${RUN_DIR}"

log() { echo "$(date -u +%H:%M:%SZ) $*" | tee -a "${LOG}"; }

log "python outside project: $(command -v python3 || echo missing)"
log "uv: $(command -v uv || echo missing)"

if [[ ! -f uv.lock ]]; then
  uv lock | tee -a "${LOG}"
fi
uv sync --locked | tee -a "${LOG}"

if uv run python -c "import nbconvert, ipykernel" >/dev/null 2>&1; then
  uv run python -m ipykernel install --user \
    --name northstar-clean-handoff \
    --display-name "Northstar clean handoff" | tee -a "${LOG}"
else
  log "SKIP notebook: telepítsd a notebook extrát (uv sync --locked --extra notebook)"
fi

uv run python generate_data.py | tee -a "${LOG}"
uv run python scripts/verify_hashes.py --manifest data/manifest.json --root data | tee -a
↪  "${LOG}"

uv run python quickcheck.py | tee -a "${LOG}"
uv run python -m northstar.cli \
  --orders data/orders.csv \
  --items data/order_items.csv \
  --output run/order_totals.csv | tee -a "${LOG}"

if uv run python -c "import nbconvert, ipykernel" >/dev/null 2>&1; then
  uv run jupyter nbconvert --to notebook --execute notebooks/explore.ipynb \
    --output explore.executed.ipynb \
    --output-dir run \
    --ExecutePreprocessor.kernel_name=northstar-clean-handoff \
    --ExecutePreprocessor.timeout=120 | tee -a "${LOG}"
```

```
fi

uv run python - <<'PY' | tee -a "${LOG}"
import platform, sys
import numpy, pandas

print("python", sys.version.split()[0])
print("executable", sys.executable)
print("platform", platform.platform())
print("numpy", numpy.__version__)
print("pandas", pandas.__version__)
PY

if git rev-parse --is-inside-work-tree >/dev/null 2>&1; then
  git status --porcelain=v1 >> "${LOG}"
fi

log "HANDOFF OK"
```

A parancs a projektgyökérből:

```
bash scripts/entrypoint.sh
```

A várható belépési folyamat tehát egyetlen sor a fogadó oldalon. A szkripten belül minden lépés külön fut, és a `set -euo pipefail` miatt az első hibánál megáll. A `--ExecutePreprocessor.kernel_name` explicit kernelválasztása azért fontos, mert különben az `nbconvert` a notebook metaadatában tárolt régi kernelt használhatná.

A kiinduló projekt és a pontozó teszt. A `lab/checkpoints/cp1/starter/` a vázlatot, a `solution/` a referencia-megoldást tartalmazza (a `northstar` csomag, a `scripts/` és a notebook). A `tests/test_checkpoint.py` a megoldást a labor adatain futtatja: ellenőrzi a `checked=6 failures=0` hash-eredményt, a `quickcheck` 2400-as szerződését és a `northstar.cli` join-invariánsát (`OK rows=2400`, a szállítási díj bemenete és kimenete egyezik). A rubrika szöveges és notebook-végrehajtási részeit ez a teszt nem pontozza; azokat a tanár értékeli.

4.4.4 Eredmény értelmezése

A sikeres futás után a napló szerkezete ilyen lehet; a konkrét idők, útvonalak és verziók a laborban futás közben derülnek ki:

```
2026-10-20T08:30:11Z python outside project: /usr/bin/python3
2026-10-20T08:30:11Z uv: /home/ada/.local/bin/uv
2026-10-20T08:31:02Z checked=6 failures=0
2026-10-20T08:31:14Z converted notebooks/explore.ipynb to run/explore.executed.ipynb
2026-10-20T08:31:20Z executable: /home/ada/work/lab/.venv/bin/python
2026-10-20T08:31:21Z HANDOFF OK
```

A `failures=0` azt jelenti, hogy a hat ellenőrzött CSV bájtjai egyeznek a manifestben rögzített elvárt hash-ekkel. Ez nem jelenti automatikusan, hogy a notebook tartalma lefutott; a notebook végrehajtását az `nbconvert` sora és a létrejött `run/explore.executed.ipynb` igazolja. A `python outside project` sorban a `python3` lehet a rendszerinterpreter, de a fontos sor a `uv run` által kiírt `executable`: annak a `.venv/bin/python` útvonalra kell mutatnia.

A napló utolsó `git status --porcelain` blokkja akkor jó, ha üres. Ha a generálás módosítaná a követett CSV-ket, az azonnal látszana; ebben az esetben vagy a generátor viselkedését, vagy a fájlok követésének döntését kell felülvizsgálni. A hash-ellenőrzés és a Git-állapot két különböző réteg: az egyik a bájtok, a másik a verziózott források állapota.

Értékelési rubrika (20 pont). Környezet és interpreter azonosítása: 3 pont; forrásadat és hash/fájlméret rögzítése: 3 pont; megfigyelési egység és join-invariáns: 4 pont; friss kernelben teljes futás: 4 pont; értelmezhető hibajegyzék és javítás: 3 pont; átadható napló és README: 3 pont. A rubrika a készségre ad visszajelzést, nem csak a »lefutott« állapotra.

Kritikus kizáró hiba: előre nem ismert információ beemelése a predikcióba, többszörözött megfigyelési egység vagy az ellenőrzés hamis sikernek jelentése esetén a megoldás nem fogadható el teljesítettnek, még ha a részpontszám magas is. A javítás után új, tiszta futás szükséges.

4.4.5 Feladatok

1. **Értelmezés.** A társ a következő naplórészletet küldi: `checked=6 failures=0` és `HANDOFF OK`. Azt állítja, hogy a notebook is biztosan sikerült, mert a hash-ellenőrzés zöld. Miért hibás a következtetés? Nevezd meg, melyik naplósor vagy fájl bizonyítaná a notebook végrehajtását, és mit nem bizonyít a hash.

2. **Megvalósítás.** A labor `scripts/verify_hashes.py` fájlja hiányzik. Írd meg a fenti szerződés szerint, majd teszteld egy háromsoros mini manifesten: `sample.csv` elvárt hash-ét a tényleges fájlból számold ki, futtasd le, változtass egyetlen bájtot, és futtasd újra. Rögzítsd mindkét kimenetet és a kilépési kódot.

3. **Hibadiagnózis.** A társ gépen a `uv run python -c "import northstar; print('ok')"` parancs sikeres, de a notebook első cellája `ModuleNotFoundError: No module named 'northstar'` hibát ad. A notebookban a `import sys; print(sys.executable)` sor `/usr/bin/python3` értéket ír ki. Add meg a hiba valószínű okát, a kimutatásához szükséges parancsot, és a javítási utat a kernelspec szintjén.

4. **Megváltozott feltétel.** A fogadó gép Windowson fut, bash nem elérhető. Írd át a belépő munkamenet lényegi lépéseit PowerShellre a projektgyökérből, beleértve a lock, a sync, a hash-ellenőrzés és a `Get-FileHash` használatát. Nevezd meg, milyen bájtszintű kockázatot jelenthet a sorelválasztó eltérése, ha a generátor platformfüggő sortörést írna.

5. **Megváltozott kontraktus.** A `quickcheck.py` egy új sémaszabályt ad hozzá: minden `order_items` sorra igaznak kell lennie a `line_total_minor = quantity * unit_price_minor - discount_minor` egyenlőségnek. A hash-ellenőrzés `failures=0`, de a quickcheck hibát jelez. Mi a diagnózis iránya, és miért nem mond ellent a két eredmény?

4.4.6 Megoldások és indoklás

1. A hash-ellenőrzés a CSV-fájlok bájtjait hasonlítja a manifesthez. A notebook végrehajtását az `nbconvert` naplósora és a generált `run/explore.executed.ipynb` igazolja. A `failures=0` tehát szükséges, de nem elégséges feltétele a teljes átadási célnak; a környezet, a kernel és a kód futtatása önálló bizonyítékot igényel.

2. A fenti `verify_hashes.py` kód közvetlenül használható (a referencia a `lab/checkpoints/cp1/solution/s`
   A mini teszt menete Linuxon:

```
printf 'alpha\n' > sample.csv
sha256sum sample.csv
# a kapott 64 hexa karaktert írd a mini-manifest.json "files" objektumába
uv run python scripts/verify_hashes.py --manifest mini-manifest.json
# checked=1 failures=0
printf 'beta\n' > sample.csv
uv run python scripts/verify_hashes.py --manifest mini-manifest.json
# checked=1 failures=1, exit code 1
```

Az ellenőrző invariáns: azonos bemeneti bájtokhoz azonos digest, eltérő bájthoz a szkript `failures=1` és nem nulla kilépési kód. A hashnél tűrés nincs.

3. A tünet arra mutat, hogy a notebook a rendszer Pythonjához regisztrált kernelt használja, miközben a projekt környezete a `.venv`. A diagnózis:

```
uv run python -c "import sys; print(sys.executable)"
```

Ha ez `.../lab/.venv/bin/python`, a notebookban viszont `/usr/bin/python3`, akkor a kernelspec rossz. A javítás a projektkörnyezetből regisztrált kernel:

```
uv run python -m ipykernel install --user \
  --name northstar-lab \
  --display-name "Northstar lab"
```

Ezután a JupyterLabban a `Kernel > Change Kernel` menüben a `Northstar lab` kernelt válaszd, majd futtasd a `Restart and Run All` parancsot. Az ismételt `sys.executable` kiírásnak a `.venv/bin/python` útvonallal kell megegyeznie. A pontos útvonal gépfüggő, az invariáns a két parancs kimenetének egyezése.

4. PowerShell-változat a projektgyökérből:

```
Set-Location C:\work\northstar-lab
if (-not (Test-Path uv.lock)) { uv lock }
uv sync --locked
uv run python scripts\verify_hashes.py --manifest data\manifest.json --root data
Get-FileHash data\customers.csv -Algorithm SHA256
uv run python -m ipykernel install --user `
  --name northstar-lab `
  --display-name "Northstar lab"
```

A projekt interpreter itt `C:\work\northstar-lab\.venv\Scripts\python.exe`, tehát a kernel bejegyzés `argv[0]` eleme más elérési útvonal, mint Linuxon. A sorelválasztó kockázata: ha a generátor szöveges módban ír, Windowson `\r\n`, Linuxon `\n` kerülhet a fájlba, így a sorok logikailag azonosak, de a hash eltér, mert a hash a bájtokra vonatkozik. A bitazonossághoz a generátornak platformfüggetlen újsor-konvenciót kell írnia, vagy a manifestnek külön szerződnie kell a sortörésről.

5. A hash-ellenőrzés a fájl és a manifest közötti bájtegyezést igazolja; a quickcheck a szemantikai sémakontraktust. A hiba iránya: a manifest hash-réteg hibátlanul frissült, de az új üzleti invariáns vagy a generátor logikája, vagy a quickcheck elvárása hibás. A két eredmény nem mond ellent egymásnak, mert különböző réteget vizsgál. Az adatfájl bájtjai épek lehetnek, miközben a bennük lévő sorok egy előírt egyenlőséget sértenek; ilyenkor a generálási logikát és a szerződést kell felülvizsgálni.

4.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba a képernyőkép mint „bizonyíték”. A társ küld egy felvételt a zöld pipáról, de utólag senki sem tudja megmondani, melyik kernel futott, mi volt a Python-útvonal, vagy hány hash-ellenőrzés ment át. Ha viszont a `run/handoff-*.log` szövegfájlt küldi, a `grep`, a diff és a kolléga is pontosan követni tudja a lépéseket.

Ugyanilyen félreértés a hash túlértékelése. A `failures=0` nem jelenti, hogy a manifest helyes; ha valaki a manifestet hibásan frissítette, a rossz elvárt értékhez is lehet tökéletes egyezés. Ezért a hash-ellenőrzés mellé mindig kell legalább egy független sémakontroll, például a `quickcheck.py` invariánsai. A hash az integritás egy rétegét, nem a tudományos helyességet igazolja.

4.4.8 Önellenőrzés

Futtatás után kérdezd végig a következőket:

- A belépő parancs a projektgyökérből indult, és a `pyproject.toml` jelen volt?
- A napló `executable` sora a `.venv`-re mutat, és a notebook kernelneve azonos a regisztrált kernellel?
- A hash-ellenőrzés `checked=6 failures=0`, a quickcheck a rögzített sorszámokat adja?
- A notebook az `nbconvert` révén valóban létrehozta a kimeneti fájlt?
- A git-munkafa tiszta, és a napló szövegfájl, nem képernyőkép?

Ha bármelyik kérdésre nemleges a válasz, a futási jegyzékből azonosítani kell a réteget, és csak azt javítani; a következő futtatást újraindított, tiszta kernelrel kell elvégezni.

4.4.9 Rövid összefoglaló

A megismételhető kutatói munkaasztal nem egyetlen notebook, hanem hat együttműködő réteg: projektgyökér, project interpreter, regisztrált kernel, Git-állapot, futási jegyzék és kontrollösszeg-manifest. A belépő szkript a lock hiányát első futtatáskor feloldja, `--locked` módban szinkronizál, a notebookot a projekt `.venv`-hez regisztrált kernellel futtatja, a hash-ellenőrzést és a sémakontrollt elkülönítetten rögzíti. A `failures=0` a fájlok integritását igazolja, de nem helyettesíti a notebook-végrehajtás bizonyítékát. A következő fejezetben NumPy-tömbökkel dolgozunk: shape, tengely és dtype válik a numerikus munka új ellenőrző szerződésévé.

5 Numerikus Python programozóknak: tömbök és számábrázolás

5.1 ndarray, shape, axis és dtype

Belépő · szint: alap · idő: 45 perc · eszköz: NumPy

Előfeltétel: 4.4. alfejezet · Célkészség: ndarray, shape, axis és dtype · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

5.1.1 Cél és konkrét helyzet

A Northstar adatszerződésben egy rendelés sora a predikció megfigyelési egysége. Tegyük fel, hogy a `distance_km`, `basket_minor` és `quantity_sum` három oszlopból kiválasztottunk öt rendelést, és NumPy-tömbben tároljuk. Az elemzés előtt tudnunk kell, hogy a táblázatos adatból hogyan lesz tömb: hány dimenzió, milyen alak, melyik tengely mentén összegzünk, és milyen numerikus típust használunk.

A konkrét bemenet egy 5×3-as mátrix:

$$A = \begin{pmatrix} 12.4 & 2990 & 2 \\ 8.1 & 5990 & 4 \\ 21.7 & 1290 & 1 \\ 5.0 & 8790 & 6 \\ 14.9 & 3490 & 3 \end{pmatrix}$$

A sorok rendeléseket, az oszlopok a fenti három feature-t jelentik. A mértékegység: km, EUR-cent, darab.

Induló becslés, mielőtt kódot írunk: mekkora az `ndim` és a `shape`? Becsüld meg az oszlopátlagokat: az első oszlop átlaga 10 és 13 között lesz, a másodiké 4000 körül, a harmadiké 3 és 4 között. Írd le azt is, milyen *alakú* eredményt vársz az oszlopátlag és a sorátlag művelettől.

5.1.2 Fogalmi magyarázat

5.1.2.1 Skalár és vektor

A **skalár** (scalar) egyetlen szám. NumPy-ban a 0-dimenziós tömb:

```
s = np.array(3.2)
s.ndim    # 0
s.shape   # ()
```

A **vektor** itt egy rendezett számlista, például $v = (12.4, 2990, 2)$. Ez egy 1-dimenziós tömb, `shape=(3,)`:

```
v = np.array([12.4, 2990.0, 2.0])
v.ndim    # 1
v.shape   # (3,)
```

A vektor most még csak számlista; a geometriai jelentést később, a lineáris algebra fejezetben kapja meg.

5.1.2.2 ndim és shape

Az **ndim** a dimenziók száma. A **shape** egy egész számokból álló tuple, amely megadja az egyes tengelyek hosszát. Az 5×3-as mátrixra: `ndim=2`, `shape=(5, 3)`. A tömbbeli elemek száma a shape elemeinek szorzata: $5 \cdot 3 = 15$.

A NumPy **sor-major** (C-rendű) tárolást használ: a memóriában először az első sor elemei következnek, utána a második soré. Ez akkor válik fontossá, amikor a `reshape` műveletet vizsgáljuk.

5.1.2.3 axis

A **tengely** (axis) az aggregáció irányát jelöli. A szabály, amelyet érdemes mechanikusan alkalmazni: a megadott tengely *eltűnik* az eredményből.

Ha `A` alakja (r, c) és `A.mean(axis=0)`-t számolunk, akkor a 0. tengely, vagyis a sorok tűnnek el, az eredmény alakja $(c,)$. Ez az oszloponkénti átlag: minden oszlopra a rögzített j index mellett átlagolunk az i sorokon:

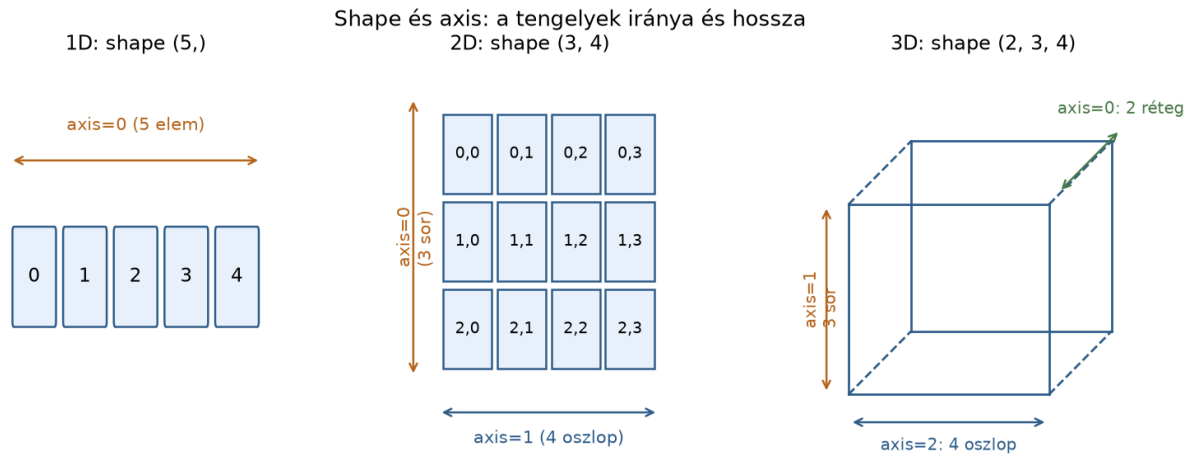
$$(\text{mean}(A, \text{axis} = 0))_j = \frac{1}{r} \sum_{i=1}^{r} A_{ij}.$$

A `A.mean(axis=1)` esetén az 1. tengely, vagyis az oszlopok tűnnek el, az eredmény alakja $(r,)$. Ez a soronkénti átlag:

$$(\text{mean}(A, \text{axis} = 1))_i = \frac{1}{c} \sum_{j=1}^{c} A_{ij}.$$

Ugyanez a szabály igaz az összegre, minimumra, maximumra és más redukciós műveletekre.

5.1.2.4 Fogalmi ábra és shape-táblázat: 1D, 2D, 3D



5.1. ábra: Shape és axis három dimenzióban: 1D tömbnél az axis=0 az öt elem iránya; 2D mátrixnál az axis=0 a három sor, az axis=1 a négy oszlop; 3D tömbnél a shape (2,3,4) rendre a két réteget, három sort és négy oszlopot jelöli.

A `shape` mindig a tengelyek hosszát adja, a tengelyek sorrendjében. Az `axis=0` a legkülső tengely: 2D-ben a sorok, 3D-ben a rétegek. Az `axis=1` a következő szint: 2D-ben az oszlopok, 3D-ben a sorok. A `sum(axis=k)` a `k`-adik tengelyt összevonja, ezért a kimenetből az a dimenzió eltűnik (kivéve `keepdims=True` esetén, amikor 1 marad a helyén).

Bemenet	Művelet	Shape out
`np.arange(5)`	—	`(5,)`
`np.zeros((3, 4))`	—	`(3, 4)`
`np.zeros((2, 3, 4))`	—	`(2, 3, 4)`
`b = np.zeros((3, 4))`	`b[0]`	`(4,)`
`b`	`b[:, 0]`	`(3,)`
`b`	`b.sum(axis=0)`	`(4,)`
`b`	`b.sum(axis=1)`	`(3,)`
`b`	`b.sum(axis=(0, 1))`	`()`
`b`	`b[:, None, :]`	`(3, 1, 4)`
`b`	`b[None, :, :]`	`(1, 3, 4)`
`c = np.zeros((2, 3, 4))`	`c.sum(axis=2)`	`(2, 3)`
`c`	`c.sum(axis=(0, 2))`	`(3,)`
`b`	`b + np.arange(4)`	`(3, 4)`
`b`	`b + np.arange(3)`	`ValueError`

A táblázatot NumPy-val ellenőrizheted:

```
import numpy as np

a = np.arange(5)
b = np.zeros((3, 4))
c = np.zeros((2, 3, 4))
assert a.shape == (5,)
```

```
assert b[0].shape == (4,)
assert b[:, 0].shape == (3,)
assert b.sum(axis=0).shape == (4,)
assert b.sum(axis=1).shape == (3,)
assert b.sum(axis=(0, 1)).shape == ()
assert b[:, None, :].shape == (3, 1, 4)
assert b[None, :, :].shape == (1, 3, 4)
assert c.sum(axis=2).shape == (2, 3)
assert c.sum(axis=(0, 2)).shape == (3,)
```

Az ábra szerkezete matplotlibszel is újrarajzolható; a teljes változat a `lab/figures/make_concept_figures.py` `shape_axes()` függvénye, ez a kivonat a három panelsémát mutatja:

```
import numpy as np
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(10, 3.6))
axes[0].plot(np.arange(5), np.zeros(5), "o")
axes[0].set_title("1D: (5,)")
axes[1].imshow(np.zeros((3, 4)), cmap="Blues")
axes[1].set_title("2D: (3, 4)")
axes[2].imshow(np.zeros((2, 3)), cmap="Blues")
axes[2].set_title("3D: (2, 3, 4) vetülete")
for ax in axes:
    ax.axis("off")
plt.show()
```

Ellenpéldák. Az `axis=0` és `axis=1` felcserélése nem ugyanazt az eredményt adja: a `(3, 4)` mátrixon `axis=0` a négy oszlop átlagát, `axis=1` a három sor átlagát számolja. A `(3,)` és a `(3, 1)` nem azonos shape: az előbbi egy tengelyű, az utóbbi két tengelyű, és a broadcasting is másképp viselkedik. A `b + np.arange(3)` azért hibás, mert a `(3,)` vektor a *legutolsó* tengelyhez igazodik, amely 4 hosszú, nem 3; a `b + np.arange(4)` viszont működik. Az `axis=2` egy 2D tömbben `AxisError`; a tengely indexe mindig `0 <= axis < ndim`.

Tízes vak shape-quiz. A válaszokat először írd le, csak utána futtasd:

#	Kérdés	Válasz
1	`np.arange(6).reshape(2, 3).shape`	`(2, 3)`
2	`np.zeros((4,)).ndim`	`1`
3	`np.zeros((2, 3, 4)).sum(axis=0).shape`	`(3, 4)`
4	`np.zeros((2, 3, 4)).sum(axis=1).shape`	`(2, 4)`
5	`np.zeros((2, 3, 4)).sum(axis=2).shape`	`(2, 3)`
6	`np.zeros((3, 1)) + np.zeros((1, 4))` shape	`(3, 4)`
7	`np.zeros((3, 4))[:, 1].shape`	`(3,)`
8	`np.zeros((3, 4))[1, :].shape`	`(4,)`
9	`np.zeros((3, 4)).sum(axis=1, keepdims=True).shape`	`(3, 1)`
10	`np.zeros((3, 4)) + np.zeros(3)`	`ValueError`

A `tests/test_shape_axes.py` a fenti táblázat és a tízes quiz minden shape-eredményét NumPy-val ellenőrzi, a `figures/shape_axes.png` létezését és a fejezet hivatkozását is beleértve.

5.1.2.5 dtype

A **`dtype`** (data type) a tömbben tárolt elemek típusa. A leggyakoribb kezdeti értékek:

```
np.array([1, 2, 3]).dtype          # int64
np.array([1.0, 2.0, 3.0]).dtype    # float64
np.array([True, False]).dtype      # bool
np.array(["a", "b"]).dtype         # <U1, egykarakteres Unicode string
```

A tömb létrehozásakor a NumPy egységes típust választ: ha egy listában vegyesen van egész és valós, akkor `float64` lesz. Ha szám mellé string keveredik, akkor a tömb stringgé válik, és a numerikus műveletek meghiúsulnak. A dtype a memóriaigényt is meghatározza: egy `float64` elem 8 bájt, egy `float32` elem 4 bájt, de a `float32` kisebb pontosságú.

5.1.2.6 Átalakítás

A **`reshape`** megváltoztatja a shape-et, de az elemszámot és a C-rendű sorrendet nem:

```
a = np.array([[1, 2, 3], [4, 5, 6]])  # shape (2, 3)
a.reshape(6)                          # shape (6,): [1, 2, 3, 4, 5, 6]
a.reshape(3, 2)                       # shape (3, 2)
a.reshape(-1)                         # shape (6,): a -1 automatikusan kiszámolt hossz
```

A `-1` hasznos, ha az egyik dimenzió hosszát a másikból akarjuk kiszámoltatni.

5.1.3 Teljes minimális munkapélda

A példa a Northstar feature-hármasát használja. A kódot a projektgyökérben egy `s017_pelda.py` fájlba írjuk, futtatása `uv run python s017_pelda.py` a lab környezetből. A NumPy 2.x verziót feltételezünk; a `np.float64` és `np.float32` elnevezés ebben a verzióban érvényes, a régi `np.float` kerülendő.

```
import numpy as np

# 5 rendelés, 3 rendeléskor ismert feature.
# Oszlopok: distance_km, basket_minor, quantity_sum.
orders = np.array(
    [
        [12.4, 2990.0, 2.0],
        [8.1, 5990.0, 4.0],
        [21.7, 1290.0, 1.0],
        [5.0, 8790.0, 6.0],
        [14.9, 3490.0, 3.0],
    ],
    dtype=np.float64,
)

print("ndim:", orders.ndim)
print("shape:", orders.shape)
print("dtype:", orders.dtype)

column_mean = orders.mean(axis=0)  # oszloponkénti átlag
row_mean = orders.mean(axis=1)     # soronkénti átlag

print("column_mean shape:", column_mean.shape)
print("column_mean:", column_mean)
print("row_mean shape:", row_mean.shape)
print("row_mean:", row_mean)

flat = orders.reshape(-1)
print("flat shape:", flat.shape)
print("flat első hat elem:", flat[:6])
```

Szemléltető kimenet (a kapcsolódó példaprojektben még nem ellenőrzött; a számokat levezetés és kézi kontroll követi):

```
ndim: 2
shape: (5, 3)
dtype: float64
column_mean shape: (3,)
column_mean: [  12.42 4510.      3.2 ]
row_mean shape: (5,)
row_mean: [1001.46666667 2000.7         437.56666667 2933.66666667 1169.3       ]
flat shape: (15,)
flat első hat elem: [  12.4 2990.     2.     8.1 5990.     4. ]
```

5.1.4 Eredmény értelmezése

Az `orders` tömb `ndim=2`, `shape=(5, 3)`, `dtype=float64`. A `mean(axis=0)` az első tengelyt tünteti el, ezért az eredmény alakja `(3,)`. A kapott három érték rendre a távolság, a kosárérték és a darabszám átlaga:

- távolság: $(12.4 + 8.1 + 21.7 + 5.0 + 14.9)/5 = 62.1/5 = 12.42$ km,
- kosárérték: $(2990 + 5990 + 1290 + 8790 + 3490)/5 = 22550/5 = 4510$ EUR-cent,
- darabszám: $(2 + 4 + 1 + 6 + 3)/5 = 16/5 = 3.2$.

A `mean(axis=1)` a második tengelyt tünteti el, ezért az eredmény alakja `(5,)`. Minden sor három elemét átlagolja. Az első sor: $(12.4+2990+2)/3 \approx 1001.47$. Ez az érték **kevert mértékegységű**, mert km, cent és darab számtani átlagát veszi; itt a sorátlag csak shape-gyakorlat, üzletileg nem értelmezhető. Az oszlopátlag viszont mértékegység-helyes, mert minden oszlop ugyanazt a mennyiséget méri.

A `reshape(-1)` a $5 \cdot 3 = 15$ elemet C-sorrendben egyetlen vektorrá fűzi: előbb az első sor, majd a második, és így tovább. Az első hat elem valóban $12.4, 2990, 2, 8.1, 5990, 4$.

A shape-invariáns, amelyet minden redukcióra ellenőrizhetünk:

$$\text{shape}(\,\text{reduce}(A, \text{axis} = k)) = \text{shape}(A) \text{ a } k. \text{ elem nélkül.}$$

Például `(5, 3)` esetén `axis=0` után `(3,)`, `axis=1` után `(5,)`.

Alak- és típusinvariáns. A NumPy `axis` értelmezése mindig a bemeneti tömb dimenzióihoz kötött; egy kétdimenziós `(n, p)` adatmátrix esetében az `axis=0` oszloponként, az `axis=1` soronként redukál. Az `ndarray` `dtype`-ja a tárolt reprezentációt határozza meg, nem automatikusan a valós világban értelmes mértékegységet.

Ellenpróba: egész típusú tömbön történő műveletnél ellenőrizd a túlcsordulást és a kimeneti típust, az üres tengelyen végzett átlagolásnál pedig a figyelmeztetést vagy `nan`-t. Minden mintapéldához rögzítsd a várt `shape`-et és `dtype`-ot.

5.1.5 Feladatok

1. Értelmezés. Az 5×3-as `orders` tömbre mondd meg: mi az `ndim`, a `shape`, és mit jelent képlettel a `sum(axis=0)`? Mi lesz a kapott tömb `shape`-e, és mennyi a `sum(axis=0)` első eleme?

2. Implementálás. Hozz létre egy 4×2-es tömböt:

```
B = np.array([[1, 10], [2, 20], [3, 30], [4, 40]])
```

Számítsd ki az oszlopátlagot, a sorátlagot és a `reshape(-1)` eredményét. Ellenőrizd mindhárom eredmény `shape`-ét és értékeit kézzel.

3. Hibadiagnózis. Az alábbi kód egy elemző szándéka szerint „soronkénti átlagot” akar számolni:

```
M = np.array([[1, 2], [3, 4]])
sor_atlag = M.mean(axis=0)
print(sor_atlag)  # [2. 3.]
```

Mi a hiba? Milyen `axis` kell a soronkénti átlaghoz, és mi lenne a helyes eredmény? Magyarázd el a kimenet alakját is.

4. Megváltozott feltétel. Az eredeti `orders` tömbhöz adj egy negyedik oszlopot, `item_rows`-t, a következő értékekkel: `[1, 2, 1, 3, 2]`. Az új tömb alakja $(5, 4)$. Számítsd ki az új sorátlagokat, és add meg az `item_rows` oszlop átlagát. Mi az új sorátlagok `shape`-e, és miért változott meg az értékük az eredeti sorátlagokhoz képest?

5.1.6 Megoldások és indoklás

1. megoldás. `ndim=2`, `shape=(5, 3)`. A `sum(axis=0)` az első tengelyt tünteti el, tehát oszloponkénti összeg:

$$(\text{sum}(A, \text{axis} = 0))_j = \sum_{i=1}^{5} A_{ij}.$$

Az eredmény `shape=(3,)`. Az első elem az első oszlop összege:

$$12.4 + 8.1 + 21.7 + 5.0 + 14.9 = 62.1.$$

Tehát a várt tömb `[62.1, 22550.0, 16.0]`.

2. megoldás. A kód:

```
B = np.array([[1, 10], [2, 20], [3, 30], [4, 40]])
B.shape                    # (4, 2)
B.mean(axis=0)             # (2,): oszlopátlag [2.5, 25]
B.mean(axis=1)             # (4,): sorátlag [5.5, 11, 16.5, 22]
B.reshape(-1)              # (8,): [1, 10, 2, 20, 3, 30, 4, 40]
```

Kézi ellenőrzés: az első oszlop átlaga $(1 + 2 + 3 + 4)/4 = 10/4 = 2.5$; a másodiké $(10 + 20 + 30 + 40)/4 = 100/4 = 25$. A sorátlagok: $(1 + 10)/2 = 5.5$, $(2 + 20)/2 = 11$, $(3 + 30)/2 = 16.5$, $(4 + 40)/2 = 22$. A `reshape(-1)` C-rendben soronként fűzi az elemeket.

3. megoldás. A hiba, hogy `axis=0` az oszlopok szerinti átlagot adja, nem a soronkéntit. A `M` mátrix:

$$M = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}.$$

`axis=0` az első tengelyt tünteti el, ezért az eredmény alakja `(2,)`, és az oszlopátlagok `[2, 3]`. A soronkénti átlaghoz `axis=1` kell, amely a második tengelyt tünteti el, az eredmény alakja `(2,)`, értéke `[1.5, 3.5]`. A kimenet alakja mindkét esetben `(2,)`, de az axis választás dönti el, hogy mit átlagolunk.

4. megoldás. Az új tömb:

```
orders2 = np.array(
    [
        [12.4, 2990.0, 2.0, 1.0],
        [8.1, 5990.0, 4.0, 2.0],
        [21.7, 1290.0, 1.0, 1.0],
        [5.0, 8790.0, 6.0, 3.0],
        [14.9, 3490.0, 3.0, 2.0],
    ],
    dtype=np.float64,
)

orders2.mean(axis=1)
# Szemléltető kimenet: shape (5,)
# [ 751.35  1501.025  328.425  2201.0  877.475]
orders2[:, 3].mean()
# Szemléltető kimenet: 1.8
```

A sorátlagok `shape=(5,)`, mert a második tengely (most már 4 elem) tűnik el. Az első sor: $(12.4 + 2990 + 2 + 1)/4 = 3005.4/4 = 751.35$. Az `item_rows` oszlop átlaga $(1 + 2 + 1 + 3 + 2)/5 = 9/5 = 1.8$. Az értékek azért változtak, mert a korábbi három elemhez egy negyedik, más mértékegységű elem is bekerült; az összesítés továbbra is kevert mértékegységű, tehát a sorátlag most is csak shape-gyakorlat.

5.1.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: „az `axis=0` azt jelenti, soronként haladok.” A NumPy szabálya nem a bejárás irányáról, hanem az **eltűnő tengelyről** szól.

Ellenpélda a 2×3-as tömbre:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$

```
A.mean(axis=0)  # [2.5, 3.5, 4.5], shape (3,)  -> oszloponkénti átlag
A.mean(axis=1)  # [2.0, 5.0],      shape (2,)  -> soronkénti átlag
```

Ha a magyar mondat úgy szól, hogy „rendelésenkénti átlag”, akkor a rendelés a sor, tehát a soroknak meg kell maradniuk; a törölt tengely az oszlop, azaz `axis=1`. Ha viszont „feature-onkénti átlag” kell, akkor az oszlopok maradnak meg, és a sorok tűnnek el, azaz `axis=0`. Az axis számozás a shape tuple pozíciója, nem a művelet irányának emberi leírása.

Ez a hiba a Northstar elemzésben is számít: az `orders` tömbön a sor a megfigyelési egység. Ha véletlenül `axis=0`-t használsz, akkor feature-átlagot kapsz; ha `axis=1`-et, akkor rendelésenkénti átlagot. A két eredmény más méretű és más jelentésű.

5.1.8 Önellenőrzés

1. Ha egy tömb `shape=(10, 4)`, akkor mi az `ndim`, és mi lesz a `sum(axis=0)` eredményének `shape`-e?
2. Mi a különbség a `shape=(3,)` vektor és a `shape=(3, 1)` oszlopmátrix között? Próbáld ki `np.array([1, 2, 3])` és `np.array([[1], [2], [3]])` alakkal.
3. Melyik dtype-ot választaná a NumPy az `np.array([1, 2.5, 3])` listára, és miért?
4. Mi a `reshape(-1)` eredménye egy `(2, 3)` alakú tömbön, és milyen sorrendben követik az elemek?

5.1.9 Rövid összefoglaló

A NumPy-tömb dimenzióit az `ndim`, alakját a `shape` adja meg; a `dtype` a tárolt elemek egységes típusa. A redukciók, például a `mean` vagy a `sum`, a megadott tengelyt tüntetik el: `axis=0` a sorokat, `axis=1` az oszlopokat törli, ezért a kimenet alakja rendre az oszlopok vagy a sorok száma. A `reshape` az elemszám megtartásával, C-sorrendben rendezi át a tömböt. A sor- és oszlopátlag kézzel ellenőrizhető, és a shape-invariáns segít elkerülni a félrecsúszott tengelyválasztást. A következő alfejezet a broadcasting, az indexelés és a másolás szabályait építi erre az alak- és tengelymodellre.

5.2 Broadcasting, indexelés és másolás

Belépő · szint: alap · idő: 45 perc · eszköz: NumPy

Előfeltétel: 5.1. alfejezet · Célkészség: Broadcasting, indexelés és másolás · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

5.2.1 Cél és konkrét helyzet

Adott egy Northstar-szerű, négy rendelést leíró tömb: `delivery_km` a rendelésenkénti szállítási távolság, `center_km` pedig ugyanezen rendelésekhez tartozó, oszlopvektorként érkező bázistávolság. A cél az elemenkénti különbség kiszámítása, vagyis az i-edik rendelésnél a $\text{delivery_km}_i - \text{center_km}_i$ érték. A megfigyelési egység a rendelés, ezért az elvárt kimeneti alak $(4,)$. Előzetes becslés: ha a második tömb véletlenül $(4, 1)$ alakú, a NumPy nem hibát ad, hanem egy $(4, 4)$-es mátrixot állít elő. A 4.2. alfejezetben tanult, debuggerrel végzett shape-vizsgálat itt beépített alakellenőrzéssé válik; az 5.1. alfejezetben definiált `shape` és `axis` pedig a tengelyhiba pontos nyelvét adja.

5.2.2 Broadcasting: kompatibilis alakok

A NumPy elemenkénti (elementwise) műveleteket végez az `+`, `-`, `*`, `/` operátorokkal. Ha a két operandus alakja eltér, a **broadcasting** (kiterjesztés) dönti el, hogy a kisebb tömb hogyan „ismétlődik” virtuálisan. A szabály:

1. A két `shape`-et jobbról balra igazítjuk.
2. Minden dimenzió kompatibilis, ha a két hossz **egyenlő**, vagy valamelyik **1**.
3. A hiányzó dimenziók 1 hosszúnak számítanak; az 1 hosszú dimenziók az operáció során a másik dimenzió hosszára bővülnek.
4. A kimenet alakja a dimenziónkénti hosszak **maximuma**.

Ez nem matematikai szándékfelismerés, hanem tiszta alak-egyeztetés. A leggyakoribb esetek:

Bal oldali `shape`	Jobb oldali `shape`	Kimeneti `shape`	Értelmezés
`(4,)`	`()`	`(4,)`	skalár + vektor
`(4,3)`	`(3,)`	`(4,3)`	oszloponkénti igazítás
`(4,3)`	`(4,1)`	`(4,3)`	soronkénti igazítás
`(4,3)`	`(4,)`	hiba	a 3 és a 4 nem kompatibilis

Bal oldali `shape`	Jobb oldali `shape`	Kimeneti `shape`	Értelmezés
`(4,)`	`(4,1)`	`(4,4)`	csendes, gyakran nem kívánt külső művelet

A matematikai híd itt a **koordinátánkénti művelet** és a **skaláris szorzat** (dot product) különbsége. Az 1D `u` és `v` vektorokra:

- $u * v$ koordinátánkénti szorzat, eredménye az $u_i v_i$ elemekből álló vektor;
- $u \cdot v = \sum_i u_i v_i$ skaláris szorzat, eredménye egyetlen skalár.

A NumPy `*` operátora koordinátánkénti szorzatot jelent, a `@` vagy `np.dot` pedig skaláris szorzatot. A `(n,)` és `(n,1)` alakú tömbök kompatibilisek a broadcasting szabályai szerint, de a `*` ilyenkor nem a páronkénti szorzat, hanem egy $n \times n$-es **külső szorzat**-jellegű mátrix. A shape tehát nem csupán technikai részlet: azt mondja meg, milyen matematikai műveletet kapunk.

5.2.3 Boolean mask

Boolean mask alatt olyan logikai tömböt értünk, amely megmondja, mely elemeket válasszuk ki, vagy melyeket módosítsuk. A feltétel ugyanazt a broadcasting szabályt követi; a kiválasztás azonban **lapított** (flattened), 1D tömböt ad, mert a kijelölt pozíciók halmaza nem feltétlenül téglalap.

```
import numpy as np

# Megfigyelési egység: rendelés (4 sor).
distance_km = np.array([12.0, 8.0, 20.0, 5.5])
mask = distance_km > 10          # shape (4,), dtype bool
print(mask)                      # [ True False  True False] elméletileg várt
print(distance_km[mask])         # [12. 20.] elméletileg várt
```

Több feltétel kombinálásához a `&` és `|` operátor kell, **zárójelekkel**, mert a Python `and`/`or` kulcsszava nem működik elemekre. A maszk által kijelölt elemeken értéket is adhatunk:

```
distance_km[mask] = 10.0
print(distance_km)  # [10. 8. 10. 5.5] elméletileg várt
```

Ez a művelet **az eredeti tömböt mutálja**, tehát a maszk kiválasztása itt nem hoz létre másolatot a jobb oldal értelemben, hanem az eredeti memóriaterületet írja.

5.2.4 View és copy

A `view` (nézet) ugyanarra a memóriaterületre mutat, új `ndarray` objektummal; a `copy` (másolat) új memóriát foglal. Az **alap indexelés** (basic indexing) — például a `slice`, egész index vagy `reshape` — általában view-t ad, ha a memóriaelrendezés engedi. Az **advanced indexing** — boolean mask, egész tömbös indexelés — mindig copy-t ad. A különbség akkor válik láthatóvá, ha módosítunk:

```
a = np.arange(5)          # [0 1 2 3 4]
b = a[1:4]                # view: a memoria egy szelete
b[0] = 99
print(a)                  # [ 0 99  2  3  4] elméletileg várt

c = a[[1, 3]]             # advanced indexing: copy
```

```
c[0] = -1
print(a)                  # [ 0 99  2  3  4] elméletileg várt, a változatlan
```

A `np.shares_memory(a, b)` ellenőrzi, hogy két tömb osztozik-e memórián. Az `a.base` attribútum view esetén megadja az eredeti tömböt, copy esetén általában `None`, de a gyors ellenőrzéshez a `np.shares_memory` a robusztusabb eszköz. Ha az elemzést meg akarjuk őrizni, **explicit `copy()`** kell, mielőtt a származtatott tömböt módosítjuk. A későbbi pandas-fejezetben más másolási szabályokat fogunk tanulni; a NumPy szabályai nem öröklődnek automatikusan a DataFrame-ekre.

5.2.5 Advanced indexing

Az **advanced indexing** (fancy indexing) két fontos formája: az egész tömbös indexelés és az imént látott boolean mask.

```
A = np.arange(20).reshape(4, 5)  # 4 sor, 5 oszlop
print(A)
# [[ 0  1  2  3  4]
#  [ 5  6  7  8  9]
#  [10 11 12 13 14]
#  [15 16 17 18 19]]

A[[0, 2], [1, 3]]          # koordináta-párok: A[0,1], A[2,3] -> [1, 13]
A[np.ix_([0, 2], [1, 3])]  # 2x2 blokk: [[1, 3], [11, 13]]
```

Az első esetben a két index tömb **páronként** kapcsolódik; az alak a broadcasting működését követi. A második esetben a `np.ix_` nyílt mátrixszerű kiválasztást (open mesh) ad. Az `A[[0,2]][:, [1,3]]` láncolás ugyanazt a blokkot adja, de két copy-t hoz létre; a `np.ix_` egyértelműbb, és akkor is helyes, ha a kijelölés tengelyenként eltérő.

5.2.6 Mutáció

A **mutáció** (mutation) az eredeti tömb tartalmának felülírása, új objektum létrehozása nélkül. A `view`-n keresztül végzett módosítás az alap tömböt is módosítja; a `copy()` izolációt ad. A dtype-csapda különösen fontos: az in-place művelet nem változtathatja meg a tároló típust úgy, ahogy az új értékadás teszi.

```
arr = np.array([1, 2, 3], dtype=np.int64)

# Nem várt viselkedés: az int tárba float nem írható be.
# A pontos hibaüzenet a NumPy verziójától függ; szemléltetően:
# arr += 0.5  # UFuncOutputCastingError vagy hasonló

arr = arr + 0.5   # új float tömb, a régi megmarad
print(arr.dtype)  # float64 elméletileg várt
```

A maszkkal vagy egész tömbös indexeléssel végzett `arr[index] = value` mutálja az eredeti tömböt, akkor is, ha a jobb oldali kiválasztás copy-t adna. Ezért ha az eredeti adat kell, a módosítandó munkatömböt **másoljuk le**.

5.2.6.1 Szándékosan hibás ellenőrző esetek

A view/copy és a broadcasting könnyen olyan tesztet eredményez, amely átmegy, de rossz tulajdonságot bizonyít. Az alábbi hibás teszt csak a fancy indexelést vizsgálja, és arról azt állítja, hogy minden részhalmaz másolat:

```
import numpy as np

def test_subsets_are_copies():
    arr = np.array([1, -2, 3])
    subset = arr[arr > 0]
    subset[:] = 0
    assert arr[0] == 1          # átmegy: a fancy index másolat
```

Ugyanez a szeletelésre elbukik, mert az alap szelet nézet:

```
def test_slice_is_a_copy():
    arr = np.array([1, -2, 3])
    subset = arr[:2]
    subset[:] = 0
    assert arr[0] == 1          # AssertionError: arr[0] == 0 lett
```

A helyes ellenőrzés mindkét utat külön vizsgálja:

```
import numpy as np
from numpy_traps import fancy_subset, slice_subset, inplace_add, readonly_broadcast,
↪  reshape_view

def test_copy_and_view_contract():
    arr = np.array([1, -2, 3])
    fancy = fancy_subset(arr)
    fancy[:] = 0
    assert arr[0] == 1, "a fancy index másolatot ad"

    arr = np.array([1, -2, 3])
    part = slice_subset(arr)
    part[:] = 0
    assert arr[0] == 0, "az alap szelet nézetet ad"

    left = np.ones((3, 1))
    try:
        inplace_add(left, np.ones((1, 4)))
    except ValueError:
        pass
    else:
        raise AssertionError("a (3,1) += (1,4) nem lehet helyben")

    view = readonly_broadcast(np.arange(4), (3, 4))
    assert not view.flags.writeable
    try:
        view[0, 0] = 9
    except ValueError:
        pass
    else:
        raise AssertionError("a broadcast_to eredménye csak olvasható")

    arr = np.arange(6)
    reshaped = reshape_view(arr)
    reshaped[0, 0] = 99
    assert arr[0] == 99, "a C-contiguous reshape nézet"
```

Eset	Kód	Eredmény	Gyakori hibás elvárás
fancy index	`arr[arr > 0]`	másolat	„minden részhalmaz nézet”
alap szelet	`arr[:2]`	nézet	„minden részhalmaz másolat”
helyben broadcasting	`(3,1) += (1,4)`	`ValueError`	„a broadcasting mindig működik”
`broadcast_to`	`np.broadcast_to(row, (3,4))`	csak olvasható nézet	„írható, mint a `tile`”
`reshape`	`arr.reshape(2,3)` C-contiguous esetén	nézet	„a reshape mindig másolat”
`astype`	`arr.astype(float)`	másolat	„az `astype` nézetet ad”

A `tests/test_numpy_traps.py` a fenti hat esetet és a szándékosan hibás teszt elbukását ellenőrzi. A tanulság: a teszt ne azt állítsa, hogy „a NumPy részhalmaza másolat” vagy „nézet”, hanem konkrétan azt, hogy az adott indexelési forma melyik viselkedést adja; a broadcasting pedig csak kompatibilis kimeneti alak mellett működik helyben.

5.2.7 Teljes minimális munkapélda: a (4,) és (4,1) csapdája

Az alábbi példa a bevezető helyzet hibás és javított változatát mutatja. A kód kimenetei a NumPy szabályaiból levezetett, **szemléltető** értékek; ezt a kéziratot a generáló környezet nem futtatta, ezért a futást a saját laborkörnyezetben kell ellenőrizni.

```
import numpy as np

# Megfigyelési egység: rendelés. Szándék: elemenkénti különbség.
delivery_km = np.array([12.0, 8.0, 20.0, 5.5])       # shape (4,)
center_km = np.array([[12.5], [7.0], [18.0], [6.0]]) # shape (4,1)

print(delivery_km.shape)  # (4,)
print(center_km.shape)    # (4,1)

wrong = delivery_km - center_km
print(wrong.shape)        # (4,4) elméletileg várt, nem kívánt
print(wrong)
# [[ -0.5  -4.5   7.5  -7. ]
#  [  5.    1.   13.   -1.5]
#  [ -6.  -10.    2.  -12.5]
#  [  6.    2.   14.   -0.5]]

# Javítás: a (4,1) oszlopvektorból 1D vektor, majd explicit shape-teszt.
center_1d = center_km.ravel()                        # shape (4,)
if center_1d.shape != delivery_km.shape:
    raise ValueError(f"Alak-eltérés: {delivery_km.shape} vs {center_1d.shape}")

diff = delivery_km - center_1d
print(diff.shape)          # (4,) elméletileg várt
print(diff)                # [-0.5  1.   2.  -0.5] elméletileg várt

# Ellenőrzés: kézi kontroll és numerikus invariant.
expected = np.array([-0.5, 1.0, 2.0, -0.5])
assert diff.shape == (4,)
assert np.allclose(diff, expected, atol=1e-12)
```

A hibás `wrong` mátrix i-edik sorának j-edik oszlopában a delivery_km$_j$ − center_km$_i$ különbség áll. A cél ehelyett csak az azonos indexű párok különbsége volt. A `ravel()` a $(4, 1)$-es tárolót az elemek sorrendjében 1D-vé alakítja; mivel az oszlopvektornak egyetlen oszlopa van, a lapítás pontosan a kívánt négy elemet adja. A `np.allclose` az 5.1. alfejezetben bevezetett `dtype`-hoz igazodó numerikus összehasonlítás; a nullához közeli eltérésekkel itt nem kell számolni, mert a kivonás pontosan reprezentálható.

Ha a műveletet újrahasználható függvénybe zárjuk, a shape-teszt hibaüzenete rögzíti a szerződést:

```
def subtract_checked(a, b, name_b="b"):
    if b.ndim == 2 and b.shape[1] == 1 and a.ndim == 1:
        raise ValueError(
            f"{name_b} oszlopvektor {b.shape}, de a {a.shape}; "
            "használjon explicit .ravel()-t a koordinátánkénti művelethez."
        )
    out = a - b
    if out.shape != a.shape:
        raise AssertionError(f"Váratlan kimeneti alak: {out.shape}")
    return out
```

5.2.8 Feladatok

1. Értelmezés. Legyen `a = np.array([1, 2, 3])` és `b_col = np.array([[10], [20], [30]])`. (a) Számold ki kézzel `a * b_col` értékét és add meg az alakját. (b) Milyen matematikai művelet jött létre, és miért nem az elemenkénti szorzat? (c) Milyen alakú eredményt ad `a * b_col.ravel()`, és mennyi az `a @ b_col.ravel()` skaláris szorzat?

2. Implementálás. Írj `clip_cap_basic(values, cap)` függvényt, amely a `values > cap` maszk alapján a felső plafont alkalmazza **egy új tömbre**, az eredeti módosítása nélkül. Teszteld `x = np.array([2.0, 7.0, 12.0, 4.0])` és `cap = 10.0` bemenettel. Ellenőrző assert: az output `[2.0, 7.0, 10.0, 4.0]`, az eredeti `x` pedig `[2.0, 7.0, 12.0, 4.0]`. Miért fontos a `copy()`, ha az outputot később módosítani fogjuk?

3. Hibadiagnózis. Egy kolléga oszloponkénti központozást szeretne végezni, de a kódja ezt írja:

```
X = np.arange(12, dtype=float).reshape(4, 3) + 1
col_mean = X.mean(axis=1, keepdims=True)   # a szándék szerint oszlopátlag?
X_centered = X - col_mean
print(X_centered.shape)                    # miért fut le?
```

Lefut-e a kód? Milyen alakú `col_mean`? Melyik átlagot vonja ki? Javítsd úgy, hogy a kimenet oszloponkénti központozás legyen, és `assert` segítségével ellenőrizd, hogy `col_mean.shape == (3,)`. Mit mutasson `X_centered.mean(axis=0)`?

4. Transzfer megváltozott feltételre. Adott `A = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])`. Készítsd el `B = A.copy()`-t, majd azokban a sorokban, ahol az első oszlop értéke nagyobb, mint 5, állítsa a harmadik oszlopot `-1`-re. Add meg az elvárt `B` mátrixot, és bizonyítsd asserttel, hogy `A` változatlan maradt. Milyen indexelési forma történik a maszk és az oszlopindex együttes használatakor?

5.2.9 Megoldások és indoklás

1. (a) A broadcasting a `(3,)` vektort `(1,3)` sorrá, a `(3,1)` oszlopot `(3,1)` oszloppá egészíti ki; a kimenet alakja `(3,3)`, az (i, j) elem $a_j \cdot b_i$. Kézzel:

$$\begin{bmatrix} 1\cdot 10 & 2\cdot 10 & 3\cdot 10 \\ 1\cdot 20 & 2\cdot 20 & 3\cdot 20 \\ 1\cdot 30 & 2\cdot 30 & 3\cdot 30 \end{bmatrix} = \begin{bmatrix} 10 & 20 & 30 \\ 20 & 40 & 60 \\ 30 & 60 & 90 \end{bmatrix}.$$

(b) Ez a két vektor összes páronkénti szorzatát tartalmazó **külső szorzat**-jellegű művelet; nem az elemenkénti szorzat, mert a $(3,)$ és $(3,1)$ alakok nem páronként igazodnak. (c) `a * b_col.ravel()` az elemenkénti szorzat `[10, 40, 90]`; az `a @ b_col.ravel()` skaláris szorzat $1 \cdot 10 + 2 \cdot 20 + 3 \cdot 30 = 140$.

2. Egy lehetséges megoldás:

```
def clip_cap_basic(values, cap):
    out = values.copy()       # izoláció a mutációhoz
    out[out > cap] = cap
    return out

x = np.array([2.0, 7.0, 12.0, 4.0])
y = clip_cap_basic(x, 10.0)
assert np.allclose(y, [2.0, 7.0, 10.0, 4.0])
assert np.allclose(x, [2.0, 7.0, 12.0, 4.0])  # az eredeti nem változott
```

A `values.copy()` nélkül az `out = values` csak nézetként ugyanarra a memóriára mutatna; a maszkkal végzett értékadás az eredeti tömböt is módosítaná. Ezért a másolat itt a függvény kimenetének és az inputnak az elválasztását biztosítja.

3. A kód lefut, mert a `(4,3)` és `(4,1)` alakok kompatibilisek a broadcasting szerint. A `col_mean` alakja `(4,1)`, és az `axis=1` miatt **soronkénti átlag**: minden sorból a saját sorátlagot vonja ki. Az oszloponkénti központozáshoz `axis=0` kell, `keepdims=False` mellett:

```
col_mean = X.mean(axis=0)
assert col_mean.shape == (3,)
X_centered = X - col_mean     # shape (4,3)
assert np.allclose(X_centered.mean(axis=0), 0.0, atol=1e-12)
```

A `X_centered.mean(axis=0)` nullához közeli vektor `[0, 0, 0]`, mert a `mean(axis=0)` az 5.1. alfejezetben tanult redukciós tengely mentén törli a sorokat, és pontosan az imént kivont oszlopátlagot adja vissza.

4. A maszk és az oszlopindex együttes használata advanced indexing assignment: a boolean maszk az első tengelyt, az egész oszlopindex a második tengelyt jelöli ki; a bal oldalon ez a kijelölt elemek eredeti memóriáját írja. A várható `B`:

```
# B:
# [[ 1  2  3]
#  [ 4  5  6]
#  [ 7  8 -1]
#  [10 11 -1]]
B = A.copy()
mask = A[:, 0] > 5
B[mask, 2] = -1
assert np.array_equal(B, [[1,2,3],[4,5,6],[7,8,-1],[10,11,-1]])
assert np.array_equal(A, [[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
```

5.2.10 Gyakori tévedés ellenpéldával

A lefutó kód **nem** bizonyítja, hogy a matematikai művelet a szándékunk szerinti. A $(4,)$ és $(4,1)$ alakú `delivery_km - center_km` hiba nélkül lefut, de a várt négy rendelés helyett 16 párosításos értéket hoz létre. Ha ezt valaki ellenőrzés nélkül továbbítja egy aggregációba, például `.mean()`-et számol rá, az átlag már az összes párosítás átlaga lesz, nem a rendelések szerinti különbségeké. Az 5.1. alfejezetben tanult shape-invariáns itt az első védelmi vonal: rögzítsük előre a kimenet elvárt alakját, és a művelet után `assert` vagy explicit `if` formában ellenőrizzük. A broadcasting szándékos külső műveletnél legitim eszköz; a hiba forrása nem a szabály, hanem a hallgatólagos feltételezés, hogy a NumPy kitalálja a matematikai célunkat.

Önellenőrzés: (1) Miért ad `a[1:3]` view-t, de `a[[1,2]]` copy-t? (2) Mi a különbség `a * b` és `a @ b` között 1D tömböknél? (3) Egy boolean maszk kiválasztás után az eredeti tömb módosul-e, ha a kiválasztott tömböt módosítom? Válaszok: (1) Az alap slice ugyanarra a memóriára mutathat; az advanced integer indexing új tömböt foglal. (2) Az `*` elemenkénti szorzatot ad, a `@` összegzi a páronkénti szorzatok összegét, skalárt eredményezve. (3) Nem, mert a maszk advanced indexing, tehát a kiválasztott tömb copy; de az `arr[mask] = value` assignment az eredeti tömböt mutálja.

Nézet, másolat, írási mellékhatás. A NumPy alapindexelése rendszerint nézetet, a haladó indexelés rendszerint másolatot ad; a kapott értékek egyezéséből ez nem derül ki. Ellenőrzési példában az eredeti és a kiválasztott tömb módosítása után vizsgáld meg, terjed-e a változás. A `np.shares_memory` a memóriamegosztás tesztjére használható, de speciális elrendezéseknél az eredményt körültekintően kell értelmezni.

Broadcastingnál mindig írd fel a dimenziókat jobbról balra igazítva; csak egyenlő méretek vagy valamelyik oldalon 1-es dimenzió esetén kompatibilisek. A kompatibilis alak nem jelenti automatikusan, hogy a művelet statisztikailag értelmes.

5.2.11 Rövid összefoglaló

A broadcasting a kisebb alak jobbra igazított, 1 dimenzió mentén történő virtuális ismétlése; a kimenet a dimenziónkénti hosszak maximuma. A `(n,)` és `(n,1)` alakok kompatibilisek, de elemenkénti művelettel `n×n`-es mátrixot adnak, ezért a shape-teszt a szándék szerinti művelet része. A boolean mask és az egész tömbös indexelés advanced indexinget jelent, amely copy-t ad, míg az alap slice view-t; a mutáció izolálásához explicit `copy()` kell. A következő alfejezetben a lebegőpontos ábrázolás, a hiányzó érték és a stabil számítás épül ezekre az alak- és másolásismeretekre.

5.3 Lebegőpont, hiányzó érték és stabil számítás

Belépő · szint: alap · idő: 45 perc · eszköz: NumPy

Előfeltétel: 5.2. alfejezet · Célkészség: Lebegőpont, hiányzó érték és stabil számítás · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

5.3.1 Cél és konkrét helyzet

A Northstar-adatszerződés pénzösszegeket EUR-centben, egész minor unitban tárol: `basket_minor`, `shipping_fee_minor` és az összeg `basket_minor + shipping_fee_minor` is egész `int64`. A gyakorlatban mégis gyakran euróra váltják az értékeket `float64` osztással, mert a megjelenítés vagy egy modell inputja ezt kéri. Ebből két kérdés lesz:

1. Milyen hibát visz be a lebegőpontos ábrázolás, és hogyan ellenőrzünk toleranciával?
2. Mikor maradjon minden pénz egész centben, és mikor elfogadható a `float64`?

A konkrét bemenet most öt képzeletbeli rendelés: `[1499, 2300, 8750, 0, 100]` cent kosárérték és `[299, 0, 599, 0, 499]` cent szállítási díj. A megfigyelési egység egy rendelés; a cél a rendelésenkénti `total_minor` pontos előállítása és annak ellenőrzése, hogy a lebegőpontos euró-kerülőút nem rontja-e el a centet. Előzetes becslés: egyetlen osztás-szorzás körüli hiba jellemzően `10^-16` relatív nagyságrendű, így öt soron nem várunk centhibát, de a pontos `==` ellenőrzés attól még hibás lehet.

5.3.2 Fogalmi magyarázat

5.3.2.1 float64: korlátozott pontosság, nem „valós szám”

A `np.float64` az IEEE 754 double precision formátum: 1 előjelbit, 11 kitevőbit és 52 tárolt mantisszabit; a normalizált értékeknél egy implicit vezető bit miatt a pontosság 53 bit. A gép-epsilon

$$\epsilon_m = 2^{-52} \approx 2{,}22 \cdot 10^{-16}$$

azt jelenti, hogy egy kerekítés relatív hibája legfeljebb $\epsilon_m/2 \approx 1{,}11 \cdot 10^{-16}$, ha nem lépünk át nagyságrendi határt. A tizedes `0.1` binárisan végtelen tört, ezért a tárolt érték csak közelíti a tizedes `0.1`-et. Ennek következménye a jól ismert példa: `0.1 + 0.2` `0.30000000000000004` lesz, nem pontosan `0.3`.

A hibát két mennyiséggel mérjük. Ha x^* a számított közelítés és x a referencia, akkor az abszolút hiba:

$$\Delta = |x^* - x|,$$

a relatív hiba pedig $x \neq 0$ esetén:

$$\delta = \frac{\Delta}{|x|}.$$

A `0.1 + 0.2` példában $\Delta \approx 5{,}6 \cdot 10^{-17}$, $\delta \approx 1{,}9 \cdot 10^{-16}$. Ez messze kisebb egy centnél (0,01), tehát a hiba nagyságrendje más, mint a pénzügyi kerekítés hibája. Ezért hasznos külön beszélni a numerikus toleranciáról és az üzleti kerekítésről.

5.3.2.2 integer overflow

A Python `int` tetszőleges pontosságú, de a `np.int64` rögzített 64 bites, előjeles szám, tartománya:

$$[-2^{63},\, 2^{63}-1] \approx [-9{,}22 \cdot 10^{18},\, 9{,}22 \cdot 10^{18}].$$

Túlcsorduláskor a NumPy nem dob hibát alapértelmezésben, hanem modulo 2^{64} visszatekeredik. A Northstar értékei bőven e tartomány alatt vannak, ezért az `int64` centbiztos. Ha viszont például egy azonosítót véletlenül numerikus oszlopként olvasnánk be és szoroznánk össze, a túlcsordulás csendben hamis eredményt adhat.

5.3.2.3 NaN és isfinite

A `np.nan` (not a number) a hiányzó vagy érvénytelen lebegőpontos érték jelölője. Fontos tulajdonságai:

- a `nan` nem közönséges szám, ezért `np.nan == np.nan` hamis;
- minden aritmetikai művelet, amelyben `nan` szerepel, `nan`-t ad;
- `np.isnan()` vizsgálja a hiányzó értéket;
- `np.isfinite()` a véges értékeket válogatja ki: `inf` és `nan` kiesik.

Egy `int64` tömbben nem tárolhatunk `nan`-t; a Northstar nyers adatában a hiányzó `distance_km` például betöltés után `float64` tömbben lehet `nan`.

5.3.2.4 allclose: toleráns összehasonlítás

A bitazonos `==` lebegőpontos számítás után általában rossz ellenőrzés. A `np.allclose(a, b, rtol=1e-05, atol=1e-08, equal_nan=False)` elemenként ezt a feltételt nézi:

$$|a_i - b_i| \le \mathrm{atol} + \mathrm{rtol} \cdot |b_i|.$$

Itt a_i és b_i a két azonos shape-ű tömb megfelelő eleme. Az `atol` abszolút tűrés, az `rtol` relatív tűrés: az `rtol * |b_i|` tag miatt a feltétel nagy értékeknél is megengedőbb, az `atol` miatt nulla körül sem válik lehetetlenné. A függvény egyetlen logikai értéket ad, ha minden elem megfelel. Az `equal_nan=True` használatával explicit módon elfogadhatjuk, hogy két `nan` „egyenlő”, de ezt csak tudatos döntésként szabad bekapcsolni.

5.3.2.5 Összegzési hiba

A lebegőpontos összeadás nem asszociatív: minden részösszeg kerekítődik, így egy nagy érték mellé adott kis érték elveszhet. A `np.sum` páros összegzést használ, ami csökkenti, de nem szünteti meg a hibát; a Python `math.fsum` kompenzált összegzése sokkal pontosabb. A pénz centben tartásának egyik előnye, hogy az egész összeadásnak nincs ilyen kerekítési hibája.

5.3.3 Teljes minimális munkapélda

A következő kód a Northstar-szerű adatot egész centben és lebegőpontos euró-kerülőúton is feldolgozza. A kimenet szemléltető, elméletileg várt érték; ebben a generálásban nem futott Python-kód.

```
import math

import numpy as np

# Bemenet: 5 rendelés, Northstar-szerű egész cent
basket_minor = np.array([1499, 2300, 8750, 0, 100], dtype=np.int64)
shipping_fee_minor = np.array([299, 0, 599, 0, 499], dtype=np.int64)

# 1) Egész út: a szerződés szerinti total_minor
total_minor_int = basket_minor + shipping_fee_minor
print("total_minor_int:", total_minor_int, total_minor_int.dtype)

# 2) Lebegőpontos euró út
total_eur = (basket_minor.astype(np.float64) / 100.0
             + shipping_fee_minor.astype(np.float64) / 100.0)
total_minor_back = np.rint(total_eur * 100.0).astype(np.int64)
print("vissza centbe:", total_minor_back)

# 3) Bitazonos és toleráns ellenőrzés
print("bitazonos:", np.array_equal(total_minor_int, total_minor_back))
print("allclose tűrés nélkül:",
      np.allclose(total_minor_int, total_minor_back, rtol=0.0, atol=0.0))
print("allclose centes euró-ellenőrzés:",
      np.allclose(total_eur,
                  total_minor_int.astype(np.float64) / 100.0,
                  rtol=1e-12, atol=1e-12))

# 4) A híres egyenlőség-ellenpélda
x = 0.1 + 0.2
print(repr(x), "==", x == 0.3, "isclose:", np.isclose(x, 0.3))

# 5) Integer overflow, NaN, összegzési hiba
print("int64 max + 1:", np.int64(2**63 - 1) + np.int64(1))
v = np.array([1.0, np.nan, np.inf, -np.inf])
print("v == v:", v == v)
print("isnan:", np.isnan(v))
print("isfinite:", np.isfinite(v))
a = np.array([1e16, 1.0, -1e16], dtype=np.float64)
print("np.sum:", a.sum())
print("math.fsum:", math.fsum(a))
```

A várt, szemléltető kimenet:

```
total_minor_int: [1798 2300 9349    0  599] int64
vissza centbe: [1798 2300 9349    0  599]
bitazonos: True
allclose tűrés nélkül: True
allclose centes euró-ellenőrzés: True
0.30000000000000004 == False isclose: True
int64 max + 1: -9223372036854775808
v == v: [ True False  True  True]
isnan: [False  True False False]
isfinite: [ True False False False]
np.sum: 0.0
math.fsum: 1.0
```

5.3.4 Eredmény értelmezése

A `total_minor_int` shape-je `(5,)`, dtype-ja `int64`, és az összeadás kerekítés nélkül pontos, mert egész centeket ad össze. A lebegőpontos úton az euróra váltás bevezet ugyan egy 10^{-16} nagyságrendű hibát, de a `np.rint(total_eur * 100.0)` centes kerekítés itt éppen visszaadja az egész értéket, ezért a bitazonos ellenőrzés is igaz. Ez nem általános garancia: a centes kerekítés eltünteti a kis hibát, de ha a kód a kerekítés előtt hasonlít össze `==`-gyel, a `0.1 + 0.2 == 0.3` példa hamis lesz. Az `allclose` itt `rtol=1e-12, atol=1e-12` mellett is igaz, mert az osztás-szorzás hibája jóval ez alatt van. A nagy `int64` érték túlcsordulása csendben negatív számot ad; a `NaN` összehasonlítása önmagával hamis; a `[1e16, 1, -1e16]` naiv `np.sum` összege `0.0`, miközben a kompenzált `math.fsum` `1.0`-t ad.

Hiányzó adat és nem véges szám elkülönítése. A `np.nan` lebegőpontos hiányjelző, a `np.inf`/`-np.inf` nem véges numerikus érték, a `pd.NA` pedig a pandas kiterjesztett, háromértékű logikájában szereplő hiányjelzés; ezek nem felcserélhetők. Az `np.isnan`, `np.isfinite` és a pandas `isna` műveletei eltérő típusfeltételekkel használhatók.

Numerikus ellenpróba: számold ki ugyanazt a kifejezést matematikailag ekvivalens, de numerikusan eltérő alakban (például közeli számok kivonásával), és vizsgáld az abszolút, illetve relatív hibát. Az eredményhez mindig add meg a lebegőpontos típust, a bemeneti skálát és a toleranciát.

5.3.4.1 Pontossági osztályozás: egzakt, toleranciás, eszközfüggő

A numerikus teszt akkor jó, ha megmondja, melyik állítás milyen erős. Három osztályt különítünk el:

Osztály	Példa	Helyes ellenőrzés	Tiltott orákulum
egzakt	`int64` centek összege, `len`, index, `np.array_equal` egészekre	`==`, `np.array_equal`	lebegőpontos újraszámítás
toleranciás	`float64` összeg, osztás, `exp/log`, `allclose`	`np.isclose(rtol, atol)`, `np.testing.assert_allclose`	`==`, kerekített szöveg
eszközfüggő	BLAS-összegzés, `float32`, párhuzamos redukció, transzcendens függvények utolsó bitjei	dokumentált tolerancia + ismételt futás	bitazonos kimenet

A kerekített kimenet különösen rossz orákulum: a `round(0.1 + 0.2, 2) == 0.3` igaz, miközben a `0.1 + 0.2 == 0.3` hamis. A kerekítés elrejti a hibát, ezért tesztben csak akkor használható, ha a kerekítés maga a szerződés (például megjelenített euróösszeg), és a toleranciát külön dokumentáljuk.

NaN-ra a `==` soha nem működik; a helyes minta a NaN-pozíciók egyezését és a nem NaN értékek toleranciás egyezését külön ellenőrzi:

```python
import numpy as np

def assert_close(actual, expected, rtol=1e-9, atol=1e-12):
    np.testing.assert_allclose(actual, expected, rtol=rtol, atol=atol)

def assert_equal_with_nan(actual, expected):
    actual = np.asarray(actual, dtype=float)
    expected = np.asarray(expected, dtype=float)
    np.testing.assert_equal(np.isnan(actual), np.isnan(expected))
    mask = ~np.isnan(expected)
    np.testing.assert_allclose(actual[mask], expected[mask], rtol=1e-9, atol=1e-12)

assert_close([0.1 + 0.2], [0.3])
assert_equal_with_nan([1.0, np.nan], [1.0, np.nan])
```

Az egzakt és a toleranciás ellenőrzést ne keverjük: egy centösszeget `np.isclose`-szal ellenőrizni megenged olyan hibát, amely a pénzügyi szerződésben nem elfogadható; egy lebegőpontos modellkimenetet `==`-gyel ellenőrizni pedig véletlenszerűen bukik el. Az eszközfüggő eseteknél a teszt rögzítse a dtypes-t, a toleranciát és a futtatókörnyezetet, és a referenciaértéket ne egyetlen futás kerekített szövegéből vegye.

A `tests/test_float_oracles.py` az egzakt egész egyezést, a toleranciás elfogadást/elutasítást, a NaN-pozíciók egyezését és a kerekített orákulum hibáját teszteli.

5.3.5 Feladatok

5.3.5.1 1. Értelmezés: hiba és tűrés

Adott `a = 0.1`, `b = 0.2`, referencia `r = 0.3` tizedesként. Számítsd ki az `a + b` `float64` közelítés és `r` abszolút, valamint relatív hibáját! Hány nagyságrenddel kisebb ez, mint `0.01` abszolút hibája? Fogalmazd meg, miért hibás így ellenőrizni egy fizetési egyenleget: `if (0.1 + 0.2) == 0.3: ...`

5.3.5.2 2. Implementálás: cent-kerülőút diagnosztika

Írj függvényt `check_roundtrip(minor_cents)` néven, amely `int64` centértékeket `float64` euróra vált, majd `np.rint(eur * 100).astype(np.int64)` módon visszaállít centbe, és visszaadja azon indexeket, ahol az eredeti és a visszaállított érték nem egyezik. Generálj `rng = np.random.default_rng(20261020)` seeddel `10_000` véletlen centértéket a `[0, 10_000_000)` intervallumból, futtasd az ellenőrzést, és add meg a hibás sorok arányát! Miért nem jelenti az eredmény azt, hogy a lebegőpontos út mindig biztonságos?

5.3.5.3 3. Hibadiagnózis: NaN-szűrés

Az alábbi kód egy hiányos értékeket tartalmazó sort szeretne megtisztítani, de rossz eredményt ad. Mi a hiba?

```python
import numpy as np

values = np.array([100.0, np.nan, 50.0])
clean = values[values == np.nan]  # a készítő szándéka: nan kihagyása
print(clean)
total = np.sum(values)
print(total)
```

Javítsd `np.isnan` segítségével, majd mutasd meg az `np.nansum` alternatívát is! Milyen shape-ű tömböt kapsz a javítás után?

5.3.5.4 4. Megváltozott feltétel: float32 és sok sor

Tegyük fel, hogy ugyanezt a rendszert USD pennyben tároló szolgáltatásra viszed át, és egy riport minden sort `float32` euróra vált, mielőtt 10 millió sort összegez. A `float32` gép-epsilonja $\epsilon_{32} \approx 1{,}19 \cdot 10^{-7}$, a kerekítés relatív hibája legfeljebb $\epsilon_{32}/2 \approx 6 \cdot 10^{-8}$. Egy 100 dolláros sor abszolút hibájának felső becslése mennyi? Ha a hibák azonos előjelűek maradnának, mekkora lehet a 10 millió soros összeg abszolút hibája? Milyen tárolási döntést hoznál, és miért?

5.3.6 Megoldások és indoklás

5.3.6.1 1. megoldás

A `float64` `0.1 + 0.2` eredménye `0.30000000000000004`. A különbség:

$$\Delta = |0{,}30000000000000004 - 0{,}3| \approx 5{,}6 \cdot 10^{-17}.$$

A relatív hiba:

$$\delta = \frac{\Delta}{0{,}3} \approx 1{,}9 \cdot 10^{-16}.$$

Ez `0.01`-hez képest $5{,}6 \cdot 10^{-17}/0{,}01 \approx 5{,}6 \cdot 10^{-15}$ szorzóval, tehát körülbelül 15 nagyságrenddel kisebb. A `==` ellenőrzés azért hibás, mert a tizedes `0.3`-nak nincs pontos bináris reprezentációja; a kiszámított érték a legközelebbi `float64`, ezért a bitminta különbözik attól a `float64`-től, amely a `0.3` literált tárolja. Pénzügyi ellenőrzésnél centben egész számokkal hasonlítsunk, vagy centes kerekítés után `np.array_equal`-t, illetve megfelelő `atol`/`rtol` mellett `np.allclose`-t használjunk.

5.3.6.2 2. megoldás

```
import numpy as np

def check_roundtrip(minor_cents: np.ndarray) -> np.ndarray:
    eur = minor_cents.astype(np.float64) / 100.0
    back = np.rint(eur * 100.0).astype(np.int64)
    return np.flatnonzero(back != minor_cents)

rng = np.random.default_rng(20261020)
cents = rng.integers(0, 10_000_000, size=10_000, dtype=np.int64)
bad = check_roundtrip(cents)
print("hibás indexek száma:", bad.size)
print("hibás arány:", bad.size / cents.size)
```

A várt eredmény a levezetés szerint `0` hibás sor, mert a centes kerekítés elnyeli a `float64` reprezentációs hibát. Ez mégsem jelenti azt, hogy a lebegőpontos út általánosan biztonságos: a hiba a kerekítés előtti összehasonlításokban, kivonásokban, nagy értékeknél vagy `float32` használatakor jelenhet meg. Az egész centes út garantáltan pontos, ezért ahol a szerződés centet ír elő, ott maradjunk `int64`-nél.

5.3.6.3 3. megoldás

A `values == np.nan` minden elemre hamis, mert a `nan` önmagával sem egyenlő, így `clean` üres `(0,)` shape-ű tömb, az összeg pedig `nan` lesz. Javítás:

```
import numpy as np

values = np.array([100.0, np.nan, 50.0])
clean = values[~np.isnan(values)]
print(clean)        # [100.  50.]
print(clean.sum())  # 150.0
print(np.nansum(values))  # 150.0
```

A javított `clean` shape-je `(2,)`, dtype-ja `float64`, és az összeg `150.0`. Ellenőrző invariant: `clean` csak véges értékeket tartalmaz, azaz `np.isfinite(clean).all()` igaz.

5.3.6.4 4. megoldás

Egy 100 dolláros sor abszolút hibája legfeljebb:

$$100 \cdot 6 \cdot 10^{-8} = 6 \cdot 10^{-6} \text{ USD} = 0{,}0006 \text{ cent}.$$

Ha 10 millió sor hibája azonos előjelű lenne, az abszolút hiba felső becslése:

$$10^{7} \cdot 6 \cdot 10^{-6} = 60 \text{ USD}.$$

Ez elfogadhatatlan. A pontos tárolás `int64` pennyben maradjon; ha lebegőpontos összegzésre van szükség, használjunk `float64`-et, akkor soronként $100 \cdot 10^{-16} \approx 10^{-14}$ USD, 10 millió sorra pedig legfeljebb 10^{-7} USD, ami elhanyagolható a centhez képest. A valós, véletlenszerű előjelű hibák részben kiejthetik egymást, de a legrosszabb esetre tervezett becslés megbízhatóbb, mint a szerencsére hagyatkozás.

5.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba, hogy a `nan`-t közönséges számként hasonlítják, vagy hogy lebegőpontos eredményt bitazonosan ellenőriznek. Ellenpélda:

```
import numpy as np

missing = np.nan
print(missing == missing)  # False

a = 0.1 + 0.2
print(a == 0.3)            # False
```

Az első esetben a `False` nem jelenti, hogy két különböző értékünk van; a `nan` definíció szerint soha nem egyenlő semmivel, még önmagával sem. A második esetben a `False` ellenére a számított érték egy cent pontossággal helyes. A helyes szűrés `np.isnan`, a helyes numerikus összehasonlítás `np.allclose` vagy `np.isclose`. A `s018`-ból ismert shape-ellenőrzés itt is hasznos: mielőtt toleranciát választunk, győződjünk meg róla, hogy `a` és `b` ugyanazt a `(n,)` vagy `(n,1)` alakot jelenti, különben a broadcasting csendben más összehasonlítást végezhet. A `s015` metrika-moduljának mintájára a centösszeg definícióját is érdemes egyetlen függvényben tartani, hogy a notebook és a script ne két különböző kerekítést használjon. Az `s003` munkanapló pedig akkor

lesz becsületes, ha a futtatott verzió mellett azt is rögzíti, hogy `int64` centút vagy `float64` euróút készítette a jelentést.

Önellenőrzés: meg tudom-e mondani, hogy a `float64` relatív hibája milyen nagyságrendű? Meg tudom-e indokolni, miért nem `==`-gyel ellenőrzök számított lebegőpontos összeget? Ki tudom-e választani, mikor kell `np.isnan` a `nan` detektálásához? Felismerem-e a csendes `int64` túlcsordulást egy nagy értékű szorzásnál?

5.3.8 Rövid összefoglaló

A `float64` 15–17 tizedesjegy pontosságú közelítés, ezért a tizedes törtek egy része nem ábrázolható pontosan; a `0.1 + 0.2 == 0.3` ezért hamis. Az abszolút hiba `|x^* - x|`, a relatív hiba `|x^* - x| / |x|`, és a `float64` relatív hibája `10^-16` nagyságrendű. A `np.allclose` az `atol + rtol * |b|` képlettel ad toleráns feltételt. A `nan` nem közönséges szám, ezért `np.isnan` és `np.isfinite` kell; az `int64` csendesen túlcsordulhat. A Northstar pénzadatot ezért érdemes egész centben tartani, és csak a megjelenítéskor euróra váltani; ha lebegőpontos út kell, a hibát nagyságrenddel és toleranciákkal ellenőrizzük. A következő alfejezetben a vektorizálást és a mért teljesítményt vizsgáljuk ezekkel az adattípusokkal.

5.4 Vektorizálás és mért teljesítmény

Belépő · szint: alap · idő: 45 perc · eszköz: NumPy

Előfeltétel: 5.3. alfejezet · Célkészség: Vektorizálás és mért teljesítmény · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

5.4.1 Cél és konkrét helyzet

Egy munkatárs a rendeléskosár-elemzéshez logisztikus pontszámot számol:

$$L(x) = \frac{1}{1 + e^{-x}}.$$

Itt x normalizált, folytonos bemenet; a képlet most **csak numerikus példa**, nem predikciós modell. A bemenet `float64` `ndarray`, shape `(n,)`, elemei `[-6, 6]` intervallumban. Két méretet vizsgálunk: `n = 2_400` a Northstar rendelési nagyságrendjéhez hasonló kis tömb, és `n = 240_000` nagy tömb. A kimenet ugyanolyan shape és dtype. A mérési megfigyelési egység egy `repeat`: ilyenkor `number = 50` futtatást mér a `timeit`, és ezt `repeat = 7` alkalommal ismétli. Az elvárt ellenőrzés: `L(0) = 0.5`, a kimenet minden eleme 0 és 1 közé esik, a Python-ciklusos és NumPy-változat `np.allclose`-szal egyezik.

Előzetes becslés: nagy `n`-re a NumPy nagyságrenddel gyorsabb lehet; kis `n`-re az interpreter- és könyvtárhívási állandó költség miatt előfordulhat, hogy a Python-ciklus nyer. Ezt most méréssel döntjük el.

5.4.2 Fogalmak: timeit, bemelegítés, allokáció, ismétlés, memória

A `timeit` (időmérő) szerepe. A `timeit.repeat(stmt, number=..., repeat=...)` a `stmt` kifejezést `number`-szer futtatja, ezt `repeat`-ször megismétli, és minden ismétlés teljes idejét adja vissza. Egy futtatás becsült ideje `total / number`. Nem a `time.time()` köré írt egyszeri mérést használunk, mert az egyetlen futás zaját nem tudnánk megkülönböztetni a valódi időtől.

Bemelegítés. Az első hívások előtt a NumPy megosztott kódja betöltődik, a memórialapok bekötődnek, a CPU-frekvencia és az operációs rendszer ütemezése is átmeneti állapotban lehet. Ezért a mérendő függvényeket előbb legalább néhányszor kis bemeneten meghívjuk. Ez nem befolyásolja a mért időt, de elkerüli, hogy az első futás torzítsa a legjobb eredményt.

Allokáció. A `np.empty`, a művelet kimenete és a hosszú kifejezések ideiglenes tömbjei mind memóriát foglalnak. A mérés előtt el kell dönteni, hogy az allokáció a mért feladat része-e. A következő minimális példában a kimeneti tömb létrehozása mindkét változatban a mért futtatás része, mert egy valódi hívásnál is meg kell történnie. Az `out=` paraméterrel in-place változat akkor lenne releváns, ha a kimenet megadott puffert töltene fel.

Mérési ismétlés. Az ismétlésekből a **legjobb** (`min`) időt nézzük, mert a legjobb futás a legkevésbé szenved az operációs rendszer közbeavatkozásától; mellette a **mediánt** is rögzítjük, mert a legjobb önmagában elrejtheti a nagy szórást. Ugyanazon gépen, azonos bemenettel, ugyanabban a folyamatban mérünk; a gépet, verziót és beállítást a futási jegyzékbe írjuk (visszautalás: 4.4).

Műveletszám- és memóriabecslés. A Python-ciklus `n` darab `math.exp` hívást és `n` darab interpreter-lépést végez; a NumPy egyszeri hívással `n` elemen, C szintű kernellel dolgozik. Leegyszerűsített modell:

$$t_{\text{py}}(n) \approx c_{\text{py}} + an, \qquad t_{\text{np}}(n) \approx c_{\text{np}} + bn,$$

ahol c az állandó hívási költség, a és b az elemenkénti idő, másodperc/elem dimenzióval. A NumPy-nak általában $b < a$, de az állandója nagyobb: $c_{\text{np}} > c_{\text{py}}$. A fordulópont (break-even) elméletileg:

$$n^* \approx \frac{c_{\text{np}} - c_{\text{py}}}{a - b}.$$

A memória tömbnél $M(n) = n \cdot s$, ahol s a dtype bájtmérete: `float64` esetén $s = 8$ byte. Tehát `n=240_000` tömb `1_920_000` byte, azaz 1.92 MB. Ha Python listában tárolnánk `float` értékeket, CPython 64 bit alatt egy elem hozzávetőleg 8 byte listapointer + 24 byte `float` objektum lenne, tehát 32 byte/elem; a tömör `ndarray` ezért a kimeneti oldalon is memóriát takarít meg.

5.4.3 Teljes minimális munkapélda

A Python-változat előre allokál egy `np.empty_like` tömböt, és elemenként tölti ki; a NumPy-változat ugyanazt az eredményt teljes tömbművelettel adja. Így a kimeneti allokáció mindkét ágon azonos.

```
# bench_logistic.py
import math
import timeit
import numpy as np

def logistic_py(x):
    """Python-ciklusos, allokált kimeneti tömbbel."""
    out = np.empty_like(x, dtype=np.float64)
    for i, v in enumerate(x):
        out[i] = 1.0 / (1.0 + math.exp(-v))
```

```
    return out

def logistic_np(x):
    """Vektorizált NumPy-változat."""
    return 1.0 / (1.0 + np.exp(-x))

# 1. Kis kézi ellenőrzés: shape, dtype, L(0)=0.5, egyezés
x_small = np.linspace(-6.0, 6.0, 11)
y_py = logistic_py(x_small)
y_np = logistic_np(x_small)
assert y_py.shape == y_np.shape == (11,)
assert y_py.dtype == y_np.dtype == np.float64
assert np.allclose(y_py, y_np, rtol=1e-15, atol=1e-15)
assert y_np[5] == 0.5          # x=0 elem pontosan a közepén
assert np.all((y_np > 0.0) & (y_np < 1.0))
print("Kis ellenőrzés rendben:", y_np)

# 2. Mérési segédfüggvény
def bench_one(n, number=50, repeat=7):
    x = np.linspace(-6.0, 6.0, n)
    # Bemelegítés mindkét változatnak, a mérésen kívül
    for _ in range(3):
        logistic_py(x[:min(n, 11)])
        logistic_np(x[:min(n, 11)])
    py_times = timeit.repeat(lambda: logistic_py(x),
                             number=number, repeat=repeat)
    np_times = timeit.repeat(lambda: logistic_np(x),
                             number=number, repeat=repeat)
    py_best = min(py_times) / number * 1000.0
    np_best = min(np_times) / number * 1000.0
    py_med = float(np.median(py_times)) / number * 1000.0
    np_med = float(np.median(np_times)) / number * 1000.0
    return {
        "n": n,
        "x_MB": x.nbytes / 1e6,
        "python_best_ms": py_best,
        "numpy_best_ms": np_best,
        "python_median_ms": py_med,
        "numpy_median_ms": np_med,
        "speedup_best": py_best / np_best if np_best > 0 else float("nan"),
    }

if __name__ == "__main__":
    for n in (2_400, 24_000, 240_000):
        row = bench_one(n)
        print(row)
```

Futtatás a projektgyökérben, feltéve, hogy a fájlt oda mentette:

```
python bench_logistic.py
```

A `lab/bench_logistic.py` ugyanezt a mérést végzi, de rögzíti a mediánt, a p25/p75/p95 kvantilist, a szórást, a dtype-ot, a bemenet bájtméretét és az ismétlésszámot. Az alábbi **mért referencia** ezen a Linux x86-64 gépen készült, Python 3.12.3, NumPy 2.5.4, `number=20`, `repeat=11`, `warmup=5`, `float64` beállításokkal (idő milliszekundumban, a zárójel a p25–p75 sáv):

n	bemenet	Python medián (p25–p75)	Python p95	NumPy medián (p25–p75)	NumPy p95	speedup (medián)
2 400	19 200 B	0.432 (0.237–0.569)	0.575	0.010 (0.0096–0.0106)	0.012	43.3×
24 000	192 000 B	2.564 (2.561–2.567)	2.569	0.147 (0.146–0.150)	0.156	17.5×
240 000	1 920 000 B	26.428 (26.326–26.448)	26.556	1.685 (1.681–1.689)	2.349	15.7×

A táblázat **nem** általános gyorsulási állítás. Egy gépen, egy dtype-pal, egy implementációval és adott ismétlésszámmal készült; más CPU, más NumPy-verzió, más szálbeállítás vagy `float32` más arányt ad. A 2 400 elemnél látszó 43×-os arány részben a kis számítás és a NumPy állandó költségének viszonya; 240 000 elemnél a memória-sávszélesség és a Python-ciklus lineáris költsége együtt más arányt ad. A mérés helyes értelmezése: a medián és a kvantilisek egy adott környezetre érvényesek, a döntéshez a saját gépen, saját adaton, ismételt futással kell mérni. A `p95` és a szórás a farok-viselkedésre figyelmeztet: a 240 000-es NumPy mérés p95-je 2.349 ms, jóval a medián felett, ezért egyetlen „legjobb” érték félrevezető lenne.

5.4.4 Eredmény értelmezése

A kis ellenőrző példa invariánsa: az ötödik indexen $x = 0$, ezért `y_np[5]` pontosan `0.5`, mert a `0.5` kettes számrendszerben pontosan ábrázolható. A Python- és NumPy-eredmény `np.allclose`-szal `rtol=1e-15, atol=1e-15` mellett egyezik; a `float64` kerekítési eltérése ennél kisebb (5.3). A kimenet `shape=(11,)`, `dtype=float64`, minden elem `(0,1)`-ben van.

A nagy mérési táblában egy tipikus gépen `speedup_best` kis `n`-re 1 alatt lehet, mert a `logistic_np` állandó hívási költsége, a kimeneti tömb allokációja és a C-könyvtár hívása dominál. `n=240_000`-nél viszont a Python-ciklus `240_000` `math.exp` függvényhívást és Python-szintű lépést fizet, ezért a NumPy-best idő általában több nagyságrenddel kisebb. A pontos arány gépenként és verzióként eltérő; a mérés akkor hiteles, ha a jegyzékben szerepel az operációs rendszer, CPU, RAM, Python- és NumPy-verzió, valamint a `number` és `repeat`.

A memória `n=240_000` esetén: bemeneti `x` 1.92 MB, a NumPy-kimenet 1.92 MB, a Python-változat kimeneti tömbje is 1.92 MB. A hosszú NumPy-kifejezés ideiglenes tömböket is létrehozhat: `-x`, `exp(-x)`, `1.0 + ...`, `1.0 / ...` mind `n` elemű `float64`; a csúcs a tartós két tömbnél nagyobb, de a Python-listás megoldás 32 byte/elem közbülső tárolásánál még mindig kedvezőbb.

Benchmark mint megismételhető kísérlet. A vektorizálás nem garantál gyorsulást minden méretnél: kis bemenetnél a hívási többlet, nagy bemenetnél a memória-sávszélesség és a köztes tömbök költsége dominálhat. A Python-ciklus és NumPy-művelet összehasonlításakor ugyanazt a számítást kell mérni, és az eredményegyezést előbb külön teszttel igazolni.

Javasolt mérési protokoll: bemelegítés; több ismétlés monoton órával vagy `timeit`-tal; medián és szóródás jelentése; bemeneti méret, hardver, Python/NumPy/BLAS-verzió és szálkeret rögzítése. Egyetlen mért időből ne fogalmazz meg általános teljesítményállítást.

5.4.5 Feladatok

1. **Értelmezés.** Egy gépen a 2 400 elemű bemenetre a Python-ciklus `0.42 ms`, a NumPy `0.51 ms`; 240 000 elemre a Python `43 ms`, a NumPy `1.3 ms`. (A számok fiktívek, az érveléshez használja őket.) Magyarázd meg az időmodell alapján, miért lehetséges kis méretnél NumPy-veszteség, és miért nem mondhatjuk, hogy „a NumPy mindig gyorsabb”. Add meg a fordulópont képletét a szakasz jelöléseivel.

2. **Implementáció és mérés.** Futtasd a `bench_logistic.py` fájlt a gépen. Készítsd el a fenti háromsoros táblázatot a **legjobb** és **medián** idővel is. A táblázat alá írja be a gép adatait: operációs rendszer, CPU, RAM, Python-verzió, NumPy-verzió, `number=...`, `repeat=...`. Ellenőrizd, hogy a repeat-ekből számított relatív tartomány `(max-min)/median` minden sorban 10% alatt marad-e; ha nem, növelje a `repeat` értéket.

3. **Hibadiagnózis.** Egy kolléga így mérte a NumPy-változatot:

```
x_list = list(np.linspace(-6.0, 6.0, 100_000))

def convert_and_logistic(x_list):
    x = np.array(x_list, dtype=np.float64)
    return 1.0 / (1.0 + np.exp(-x))

times = timeit.repeat(lambda: convert_and_logistic(x_list),
                      number=100, repeat=5)
```

Azt állítja, hogy a NumPy 2.1-szer lassabb, mint a Python-ciklus. Mi a mérési hiba? Írj olyan javított kódot, amely az előkészítést és a kernelműveletet külön méri. Add meg, mit kell bemelegíteni a két mérésben.

4. **Megváltozott feltétel: páronkénti távolság.** A `distance_km` 20 000 elemű `float64` tömb. A kolléga az átlagos páronkénti abszolút távolságot így számolja:

```
D = np.abs(np.subtract.outer(d, d))
avg = D.sum() / (n * (n - 1))
```

   a) Hány bájt a `D` tömb, ha `n=20_000` és a dtype 8 byte?

   b) Mi a memóriaprobléma a teljes mátrix materializálásával?

   c) Írj `mean_pairwise_abs(d, block=1000)` blokkos változatot, amely egyszerre legfeljebb `block` sort tart meg, és `n=200`-on `np.isclose`-szal `rtol=1e-9, atol=1e-12` mellett ellenőrizd, hogy az eredmény egyezik a teljes outer-változattal.

5.4.6 Megoldások és indoklás

1. megoldás. A lineáris modellben a a Python elemenkénti költsége, b a NumPy elemenkénti költsége, és $a > b$. Kis `n`-nél a t értékét az állandó tagok határozzák meg: c_{np} tartalmazza a tömb kernel hívását, a dtype-ellenőrzéseket és a kimeneti allokációt, ezért nagyobb lehet, mint c_{py}. Nagy `n`-nél a lineáris tag dominál, ezért a kisebb b nyer. A fordulópont $n^* \approx (c_{\text{np}} - c_{\text{py}})/(a - b)$. A „NumPy mindig gyorsabb” azért hamis, mert a gyorsaság a méret, a művelet típusa és a gép függvénye; más műveletre, például `np.sort`-ra, a fordulópont jóval kisebb lehet.

2. megoldás. A helyi táblázat kitöltése a tanuló mérésétől függ; a hitelesítéshez a következő invariant szükséges: a script meghívása után az `assert` a kis példán ne szálljon el, a `speedup_best` kis `n`-re tipikusan 1 körül, nagy `n`-re tipikusan 1 fölött van. A relatív tartomány ellenőrzéséhez például:

```
times = timeit.repeat(lambda: logistic_np(x), number=50, repeat=7)
rel = (max(times) - min(times)) / float(np.median(times))
print(rel)  # cél: < 0.10
```

Ha 10% fölé megy, növelje a `repeat` értéket 15-re, vagy zárja be a mérés alatt futó egyéb nagy folyamatokat. A jegyzék a gép adataival együtt teszi megismételhetővé a mérést.

3. megoldás. A hiba az, hogy minden mért futtatás tartalmazza a 100 000 elemű Python-lista `np.array`-vá alakítását. Ez adatelőkészítés, nem a vektorizált képlet; a Python-ciklusos összehasonlító ág nem végez hasonló konverziót. Helyes szétválasztás:

```
# előkészítés, a mérésen kívül
x_arr = np.array(x_list, dtype=np.float64)
# bemelegítés mindkét útra
logistic_py(x_arr)
logistic_np(x_arr)
# kernelművelet mérése
kernel_times = timeit.repeat(lambda: logistic_np(x_arr),
                             number=100, repeat=5)
# előkészítés külön mérése
prep_times = timeit.repeat(lambda: np.array(x_list, dtype=np.float64),
                           number=100, repeat=5)
```

A bemelegítésnek mind a `logistic_py`, mind a `logistic_np` első hívását el kell fednie; az `x_arr`-t a mérésen kívül hozzuk létre. Az ellenőrzés marad `np.allclose(y_py, y_np, rtol=1e-15, atol=1e-15)`. Helyes méréssel a vektorizált képlet a nagy tömbön tipikusan felülmúlja a ciklust, de a tényleges arány gépenként mérhető.

4. megoldás.

a) `D.nbytes = 20_000 * 20_000 * 8 = 3_200_000_000` byte = 3.2 GB. Mivel a kifejezés először `np.subtract.outer`-rel létrehoz egy 3.2 GB-os tömböt, majd `np.abs` egy újabb 3.2 GB-osat állít elő, a csúcs legalább 6.4 GB lehet, mielőtt az első felszabadul.

b) A teljes mátrix a kívánt átlaghoz feleslegesen materializál; nagy `n`-re a memória hamar kimerül, a swap vagy az `OutOfMemory` megöli a folyamatot.

c) Blokkos változat:

```
def mean_pairwise_abs(d, block=1000):
    n = d.size
    total = 0.0
    count = n * (n - 1)
    for i in range(0, n, block):
        block_vals = d[i:i + block]
        total += np.abs(block_vals[:, None] - d[None, :]).sum()
    return total / count

# Ellenőrzés kis n-en: teljes outer vs blokkos
d_small = np.linspace(1.0, 100.0, 200)
D = np.abs(np.subtract.outer(d_small, d_small))
full_avg = D.sum() / (200 * 199)
block_avg = mean_pairwise_abs(d_small, block=50)
assert np.isclose(full_avg, block_avg, rtol=1e-9, atol=1e-12)
```

A blokk-köztes tömb formája `(block, n)`, tehát `1000 * 20_000 * 8 = 160_000_000` byte = 160 MB; az `np.abs` rövid időre még egy hasonló tömböt tarthat, így a csúcs 320 MB körül van, szemben a 3.2 GB-os teljes mátrixszal. Az invariáns a `np.isclose` a teljes és blokkos eredmény között `n=200` esetén; a főátló nullákat ad, ezért a `count = n * (n - 1)` nevező a rendezett párokat, nem az átlót osztja.

5.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés, hogy a vektorizált kód **mindig** gyorsabb és memóriatakarékosabb. Az ellenpélda a 4. feladat: a `np.subtract.outer(d, d)` a 20 000 elemű bemenetből 3.2 GB-os köztes mátrixot csinál, ami sokkal nagyobb, mint a bemenet, és a gép memóriáját kimerítheti. A másik félreértés, hogy az első futás ideje tipikus mérés; bemelegítés nélkül az idegen kód betöltése és a CPU-átmenetek torzítanak. A harmadik félreértés, hogy a nem futott benchmark eredményét ellenőrzöttnek nevezzük: a hitelesítéshez gép, méret, `number`, `repeat` és tényleges futási jegyzék kell. Önellenőrzéshez kérdezze meg: ugyanazt a kimenetet ellenőrzi-e a két ág? A mérésből kihagytam-e a bemeneti konverziót? A memóriabecslésben figyelembe vettem-e a köztes tömböket? Visszautalás: 5.3 toleranciája adja a numerikus egyezés feltételét, 4.4 futási jegyzéke pedig a mérés dokumentálását.

5.4.8 Rövid összefoglaló

A vektorizált NumPy-kód nem automatikusan gyors: a munkát C szintű kernellel, tömör `float64` tömbökön végzi, ezért nagy méreten nyer, de kis méreten az állandó hívási és allokációs költség dominálhat. A Python-ciklus elemenként interpreter- és `math.exp` hívást fizet. Méréshez `timeit.repeat` kell explicit bemelegítéssel, előre eldöntött allokációs határral, `number` és `repeat` beállításssal; a legjobb és medián időt, gépadatokat és verziókat rögzítsünk. A memória `n`-szeres `itemsize`; az outer-jellegű műveletek hatalmas köztes tömböket hozhatnak létre, ezért blokkos vagy összegző megoldást érdemes választani. A `np.allclose` a numerikus egyezést, a kézi `L(0)=0.5` az irányt ellenőrzi. A következő fejezetben a pandas ugyanezen tömbök fölé épít típusos adattáblát; a mérési fegyelmet ott is tartjuk.

6 pandas: típusbiztos adatbetöltés és tisztítás

6.1 Séma, dtypes és hiányzó értékek

Belépő · szint: alap · idő: 45 perc · eszköz: pandas

Előfeltétel: 5.4. alfejezet · Célkészség: Séma, dtypes és hiányzó értékek · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

6.1.1 Cél és konkrét helyzet

A Northstar labor `data/raw/orders_raw.csv` fájlja szándékosan szennyezett rendelési adat. Egy sor egy rendelés (*order*), a kulcsnak szánt mező az `order_id`. A laborleírás szerint a fájl 2405 sort és 15 oszlopot tartalmaz, és négy szándékos hibacsoportot rejt: öt duplikált sor, két hiányzó `distance_km`, egy negatív `basket_minor` és egy olyan `customer_id`, amely nem létezik a `customers.csv`-ben.

Ebben az alfejezetben a feladat az, hogy ne a pandas automatikus típusfelismerésére bízzuk az adat értelmezését, hanem explicit sémával töltsük be, és mezőnként hibajegyzéket készítsünk. A kimenet három dolog: a betöltött `DataFrame`, egy mezőnkénti sémajelentés, és egy javítási szabálygyűjtemény, amely nem felülírja, hanem hibakóddal elkülöníti a problémás sorokat. Előzetes becslés: 2405 sor, a fenti négy hibacsoport darabszámai, valamint az, hogy a hibás sorok nem átfedőek. A futtatott kimenetet itt „elméletileg várt” jelöléssel adjuk; a lab tényleges futása a manifesttel ellenőrizhető.

6.1.2 Megfigyelés, változó és a mérés szintje

Adattáblában egy sor egy megfigyelés (*observation*), egy oszlop pedig egy változó (*variable*). A `distance_km` numerikus vagy kvantitatív változó: két érték összege, átlaga vagy különbsége értelmes, mértékegysége km. A `currency` ezzel szemben nominális vagy kategorikus változó: nem rendezhető nagyság szerint, és nincs értelmes számtani középérték. A `region` típusú oszlop ugyancsak nominális. Numerikus oszlopoknál a séma része a megengedett tartomány; például a `basket_minor` egész EUR-cent, és a labor-szerződés szerint nem lehet negatív, a `distance_km`-nek 0-nál nagyobbnak kell lennie. Kategorikus oszlopnál viszont a séma a megengedett kategóriák halmaza: `currency` itt kizárólag `EUR`.

Egy kis háromsoros példán látható a különbség:

order_id	customer_id	basket_minor	distance_km	currency
A1	C1	150	hiány	EUR
A2	hiány	érvénytelen szöveg	-2	EUR
A3	C1	hiány	10	EUR

A `basket_minor`-t összegezhetjük: 150 + ? + ? csak a hiányok megoldása után értelmes. A `currency`-t nem tudjuk „átlagolni”, legfeljebb azt kérdezhetjük, hány sor `EUR`.

6.1.3 Explicit séma: read_csv, dtype és parse_dates

A pandas `pd.read_csv` alapértelmezésben először mintát olvas, és oszloponként megtippeli a típust. Ez csendes kockázat: ha egy oszlop később kap egy hiányzó értéket, megváltozhat a dtype, és a kód más ágon fut. Az explicit séma (*explicit schema*) ehelyett előre rögzíti minden oszlop nevét, típusát, mértékegységét és érvényes tartományát.

A pandas 2.2.3-ban a hagyományos `int64` nem tud hiányzó értéket tárolni, ezért a pandas a hiányossá vált egész oszlopot gyakran `float64`-re konvertálja. A nullable típusok ezt oldják meg: az `Int64`, `Float64`, `string` és `boolean` dtype a hiányt egységesen a `pd.NA` jelölővel tárolja. A `category` dtype kategorikus oszlophoz való: memóriában a szintkészletet tárolja, és kódszámokkal hivatkozik a sorokra. A `parse_dates` paraméter az ISO-szövegeket `datetime64[ns, UTC]` típusú időbélyeggé alakítja, mert a CSV-ben explicit `+00:00` offset szerepel.

A tisztaz `orders.csv`-hez tehát ilyen betöltés tartozhat:

```
import pandas as pd
from pathlib import Path

# Notebook-példa: a munkakönyvtár a lab projektgyökere.
ROOT = Path.cwd().resolve()
RAW = ROOT / "data" / "raw"

orders = pd.read_csv(
    RAW / "orders.csv",
    dtype={
        "order_id": "string",
        "customer_id": "string",
        "promised_days": "Int64",
        "quantity_sum": "Int64",
        "item_rows": "Int64",
        "basket_minor": "Int64",
        "shipping_fee_minor": "Int64",
        "currency": "category",
        "distance_km": "Float64",
        "priority_flag": "Int64",
        "delivery_days": "Int64",
        "target_late": "Int64",
    },
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
```

A `dtype` szótárban nem szereplő, de `parse_dates` által kezelt oszlopok automatikusan datetime típust kapnak. A `dataset` szerződése szerint a tiszta fájl 2400 sort tartalmaz, ezért az első ellenőrzés:

```
assert orders.shape == (2400, 15)
assert str(orders["distance_km"].dtype) == "Float64"
assert str(orders["order_id"].dtype) == "string"
assert pd.api.types.is_datetime64tz_dtype(orders["order_ts"])
```

Scriptben ne a `Path.cwd()`-re építsünk, hanem a 2.1-ben tanult `Path(__file__).resolve()` alapú projektgyökér-levezetésre; itt a notebook munkakönyvtára rögzített szerződés.

6.1.3.1 Szerződéses beolvasó: fail-fast sémateszt

A `dtype` szótár önmagában még nem ad szerződést: a hiányzó oszlopot, a hibás sentinel értéket, a nem engedett kategóriát és a duplikált kulcsot külön kell ellenőrizni. A `lab/csv_contract.py`

ezt egy helyen teszi; a séma deklaratív, a beolvasás pedig a `SchemaError` kivétellel áll le az első szerződéssértésnél:

```
import pandas as pd

class SchemaError(ValueError):
    pass

ORDERS_SCHEMA = {
    "columns": {
        "order_id": "string", "customer_id": "string",
        "order_ts": "datetime64[ns, UTC]", "basket_minor": "int64",
        "shipping_fee_minor": "int64", "distance_km": "float64",
        "priority_flag": "int64", "target_late": "int64", "currency": "string",
    },
    "non_null": ["order_id", "customer_id", "order_ts", "basket_minor",
                 "shipping_fee_minor", "priority_flag", "target_late", "currency"],
    "categories": {"currency": {"EUR"}, "priority_flag": {0, 1},
                   "target_late": {0, 1}},
    "unique": ["order_id"],
}

def read_contract(path, schema=ORDERS_SCHEMA):
    raw = pd.read_csv(path, dtype=str, keep_default_na=False, na_values=[""])
    expected = set(schema["columns"])
    actual = set(raw.columns)
    if actual != expected:
        raise SchemaError(f"oszlopeltérés; hiányzó={sorted(expected-actual)}, "
                          f"váratlan={sorted(actual-expected)}")
    frame = pd.DataFrame(index=raw.index)
    for name, dtype in schema["columns"].items():
        if dtype.startswith("datetime64"):
            frame[name] = pd.to_datetime(raw[name], utc=True, errors="raise")
        elif dtype in ("int64", "float64"):
            frame[name] = pd.to_numeric(raw[name], errors="raise").astype(dtype)
        else:
            frame[name] = raw[name].astype("string")
    for name in schema["non_null"]:
        if frame[name].isna().any():
            raise SchemaError(f"{name}: hiányzó érték nem engedett")
    for name, allowed in schema["categories"].items():
        invalid = set(frame[name].dropna()) - allowed
        if invalid:
            raise SchemaError(f"{name}: nem engedett érték: {sorted(invalid)}")
    for name in schema["unique"]:
        if frame[name].duplicated().any():
            raise SchemaError(f"{name}: duplikált kulcs")
    return frame
```

A beolvasás szándékosan `dtype=str` és `keep_default_na=False` beállítással indul, hogy a hiányzó érték és az érvénytelen szöveg ne mosódjon össze a típuskonverzió előtt. A `SchemaError` üzenete mindig megnevezi az oszlopot és a hiba okát; a hívó kód ezt naplózza, nem nyeli el.

Sértés	Példa	Eredmény
hiányzó oszlop	nincs `currency`	`SchemaError: oszlopeltérés`
váratlan oszlop	extra `note`	`SchemaError: oszlopeltérés`
hibás sentinel	`basket_minor = "unknown"`	`SchemaError: nem alakítható int64`

Sértés	Példa	Eredmény
hiányzó kötelező érték	üres `customer_id`	`SchemaError: hiányzó érték`
nem engedett kategória	`currency = "USD"`	`SchemaError: nem engedett érték`
duplikált kulcs	két azonos `order_id`	`SchemaError: duplikált kulcs`

A `tests/test_csv_contract.py` a jó fájlt, a hibás sentinelt, a hiányzó/váratlan oszlopot, a hiányzó kötelező értéket, az USD-t és a duplikált kulcsot is ellenőrzi. A `dtype=str` + `keep_default_na=False` kombináció itt nem stílus kérdése: enélkül a pandas a `"unknown"` szöveget `NaN`-ná alakíthatja, és a hiba csak később, nehezebben lokalizálható helyen jelentkezne.

6.1.4 Hiány, üres szöveg és érvénytelen érték elkülönítése

A nyers CSV-ben három különböző probléma jelenhet meg, és ezeket nem szabad összemosni:

- **Hiányzó érték** (*missing value*): a cella nem tartalmaz adatot, mert nincs mérés vagy adatfelvétel. A pandas ezt `NaN` vagy `pd.NA` jelöli.
- **Üres szöveg**: a cellában szó szerint `""` van. Az alapértelmezett `read_csv` ezt a `keep_default_na=True` miatt hiánynak tekinti; ha auditálunk, érdemes `keep_default_na=False`-szal betölteni, hogy az üres szöveg megmaradjon.
- **Érvénytelen érték** (*invalid value*): a cella nem üres, de nem felel meg a sémának, például `-2` a `distance_km` oszlopban, `abc` a `basket_minor`-ban, vagy ismeretlen `customer_id`.

A három kategóriát egy mini CSV-n kézzel ellenőrizhetjük:

```
import io

mini = pd.read_csv(
    io.StringIO(
        "order_id,customer_id,basket_minor,distance_km,currency\n"
        "A1,C1,150,,EUR\n"
        "A2,,abc,-2,EUR\n"
        "A3,C1,,10,EUR\n"
    ),
    dtype={"order_id": "string", "customer_id": "string", "currency": "category"},
    keep_default_na=False,
)
```

Ebben a betöltésben az `A1` sorbeli üres `distance_km` hiány, az `A2` sorbeli üres `customer_id` üres szöveg, az `A2` sorbeli `abc` érvénytelen numerikus szöveg, és az `A2` sorbeli `-2` érvénytelen tartomány. Ha megnézzük a dtypes-ot, a `basket_minor` object vagy string lesz, mert vegyesen tartalmaz számot és szöveget; a `distance_km` viszont `float64`, mert a hiány `NaN`-ként fér el benne. Az explicit `Int64` konverziós terv csak az érvénytelen szöveg elkülönítése után hajtható végre.

6.1.5 Teljes minimális munkapélda: sémajelentés és hibajegyzék

A laborfájlra két lépésben dolgozunk. Először explicit nullable sémával betöltjük az `orders_raw.csv`-t, hogy a hiány és a tartományhiba egyaránt látszódjon:

```
raw = pd.read_csv(
    RAW / "orders_raw.csv",
    dtype={
        "order_id": "string",
        "customer_id": "string",
        "promised_days": "Int64",
        "quantity_sum": "Int64",
        "item_rows": "Int64",
        "basket_minor": "Int64",
        "shipping_fee_minor": "Int64",
        "currency": "category",
        "distance_km": "Float64",
        "priority_flag": "Int64",
        "delivery_days": "Int64",
        "target_late": "Int64",
    },
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
```

A labor-szerződés szerint a raw fájlban a numerikus oszlopok nem tartalmaznak érvénytelen szöveget, ezért ez a dtype dict nem dob konverziós hibát. A hiányos `distance_km`-ből `pd.NA` lesz. Most definiáljunk sémajelentést:

```
def schema_report(df, rules):
    """Mezőnkénti dtype, hiány, egyediség és tartalomhiba jelentés."""
    rows = []
    for col in df.columns:
        s = df[col]
        invalid = 0
        if col in rules:
            # A szabályt csak a nem hiányzó értékekre alkalmazzuk.
            invalid = int((s.notna() & ~rules[col](s)).sum())
        rows.append(
            {
                "column": col,
                "dtype": str(s.dtype),
                "missing": int(s.isna().sum()),
                "unique_non_null": int(s.dropna().nunique()),
                "invalid_non_null": invalid,
            }
        )
    return pd.DataFrame(rows).set_index("column")

rules = {
    "promised_days": lambda s: s.eq(7),
    "quantity_sum": lambda s: s.ge(0),
    "item_rows": lambda s: s.ge(1),
    "basket_minor": lambda s: s.ge(0),
    "shipping_fee_minor": lambda s: s.ge(0),
    "distance_km": lambda s: s.ge(0),
    "priority_flag": lambda s: s.isin([0, 1]),
    "delivery_days": lambda s: s.ge(1),
    "target_late": lambda s: s.isin([0, 1]),
}

report = schema_report(raw, rules)
print(report)
```

A hibajegyzéknek a kulcs-utalást is ellenőriznie kell. A `customer_id` csak akkor érvényes, ha szerepel a `customers.csv` ügyféllistájában:

```
customers = pd.read_csv(RAW / "customers.csv", dtype={"customer_id": "string"})
unknown_customer = (
```

```
    raw["customer_id"].notna()
    & ~raw["customer_id"].isin(customers["customer_id"])
)
duplicated_row = raw.duplicated()
```

Ezek után hibakód-oszlopot készítünk a sorszintű diagnózishoz:

```
issue = pd.Series("", index=raw.index, dtype="string")

issue[raw["distance_km"].isna()] = "missing_distance_km"
issue[(raw["basket_minor"] < 0).fillna(False)] = "invalid_basket_minor_negative"
issue[unknown_customer] = "unknown_customer"
# A duplikátum sorszintű hiba; a pandas adatminőségi tárgyalás s023.
issue[duplicated_row] = "duplicate_row"

raw_with_issue = raw.assign(issue_code=issue)
```

A javítási szabály nem csendes javítás, hanem osztályozás:

```
repair_rules = {
    "missing_distance_km": {
        "action": "exclude",
        "reason": "missing measurement, no safe zero-imputation",
    },
    "invalid_basket_minor_negative": {
        "action": "exclude",
        "reason": "violates minor-unit range",
    },
    "unknown_customer": {
        "action": "exclude",
        "reason": "fails customer key constraint",
    },
    "duplicate_row": {
        "action": "keep first",
        "reason": "row-level duplicate, review in s023",
    },
}
```

A nyers fájlt soha nem írjuk felül; a javított vagy szűrt változat a `data/derived` alá kerülne.

6.1.6 Eredmény értelmezése

A teljes munkapélda kimenete automatikusan még nem ellenőrzött, ezért szemléltető, elméletileg várt jellegű. Az `orders_raw.csv` betöltése után a következő invariánsoknak kell teljesülniük:

- `raw.shape == (2405, 15)`.
- `raw["distance_km"].isna().sum() == 2`: a két szándékos hiány.
- `raw["basket_minor"].lt(0).sum() == 1`: az egy szándékos negatív érték.
- `unknown_customer.sum() == 1`: az egy ismeretlen ügyfél.
- `raw.duplicated().sum() == 5`: a szándékos duplikátumsorok.
- A fenti hibák külön sorokon vannak, ezért a `missing_distance_km` és az `invalid_basket_minor_negative` szűrése után a `raw_with_issue["issue_code"].eq("").sum() == 2402`.
- `issue_code` `string` dtype, nem `object`; a hiányzó `distance_km` a nullable `Float64`-ben `pd.NA`, nem `NaN`.

A saját gépen futtatva ellenőrizzük `raw_with_issue["issue_code"].value_counts()` értékét; ha egy hibacsoport darabszáma eltér, akkor a lab adata nem az alap generálási paraméterű, vagy a betöltési útvonal más. A `unique_non_null` oszlopok pontos értékét itt nem adjuk meg, mert

az nem rögzített a mellékelt szerződésben; az alábbi report nem teljes sort mutat. A fontos, hogy a `missing` és `invalid_non_null` oszlopok a várt hibákat tükrözzék.

6.1.7 Feladatok

1. **Értelmezés.** A mini CSV-ben miért nem lehet `dtype={"basket_minor": "Int64"}` értékkel betölteni a fájlt? Mi a dtype, ha a dtype szótárt teljesen elhagyjuk? Legalább két következményt indokolj.
2. **Megvalósítás.** Olvasd be a tisztaz `orders.csv`-t a fent vázolt explicit sémával és `parse_dates` paraméterrel. Írj olyan `assert` sorokat, amelyek igazolják: a shape `(2400, 15)`, nincs hiányzó érték semelyik oszlopban, az `order_ts` időzóna-tudatos datetime, és a `currency` egyetlen kategóriája `EUR`. A `manifest.json`-t most ne használd.
3. **Hibadiagnózis.** Egy osztálytárs ezt írta a raw fájl auditálásához: `clean = pd.to_numeric(raw["basket_minor"], errors="coerce")`. Milyen hibát követ el a hibajegyzék szempontjából? Adj olyan, a coerce elé illesztett maszkot, amely a nem hiányzó, de nem numerikussá alakítható szöveget még a konverzió előtt megjelöli, és mutasd meg a mini CSV `abc` értékére.
4. **Megváltozott feltétel.** A downstream folyamat mostantól csak a `missing_distance_km` és `invalid_basket_minor_negative` sorokat zárja ki; az `unknown_customer` hibákat későbbi lépés vizsgálja. Hozd létre a `clean` DataFrame-et ebből a feltételből. Ellenőrizd: `clean.shape[0] == 2402`, minden `distance_km` nem hiányzó, és minden `basket_minor` nemnegatív. Milyen további adathibák maradnak még a `clean` részben?

6.1.8 Megoldások és indoklás

1. A `basket_minor` oszlop a mini CSV-ben tartalmazza az `abc` szöveget. Az `Int64` kötött numerikus dtype, így a pandas nem tudja az `abc`-t erre a típusra konvertálni, ezért kivétel keletkezik. Ha a dtype szótárt elhagyjuk, a pandas az oszlopot `object` vagy `string` típusúnak észleli, mert vegyesen van szám és szöveg. Ez azt jelenti, hogy a későbbi `s.basket_minor.sum()` nem fut le numerikus összegzésként, hanem hibát ad vagy string-konkatenációt végez, ezért az adatprobléma csendesen a betöltés után jelenik meg.

2. A betöltés az explicit dtype dicttel:

```
orders = pd.read_csv(
    RAW / "orders.csv",
    dtype={
        "order_id": "string",
        "customer_id": "string",
        "promised_days": "Int64",
        "quantity_sum": "Int64",
        "item_rows": "Int64",
        "basket_minor": "Int64",
        "shipping_fee_minor": "Int64",
        "currency": "category",
        "distance_km": "Float64",
        "priority_flag": "Int64",
        "delivery_days": "Int64",
        "target_late": "Int64",
    },
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
```

```
assert orders.shape == (2400, 15)
assert orders.isna().sum().sum() == 0
assert pd.api.types.is_datetime64tz_dtype(orders["order_ts"])
assert set(orders["currency"].cat.categories) == {"EUR"}
```

A `parse_dates` a `datetime64` típusú oszlopokat időzóna-tudatosan hozza létre, mert a forrás `+00:00` offsettel ír. A `cat.categories` a kategóriaszintet adja vissza.

3. A `errors="coerce"` a nem értelmezhető szöveget `NaN`-né alakítja, így az érvénytelen érték összekeveredik a valódi hiánnyal. A hibajegyzék ekkor aluljelenti az érvénytelen értékeket, és a downstream kód azt hiheti, hogy csak mérési hiány van. A javított maszk:

```
def invalid_text_mask(s):
    if s.dtype == "string":
        not_missing = s.notna()
    else:
        not_missing = s.notna()
    # A to_numeric a hibás szövegre NaN-t ad, ezért a "nem hiány, de NaN" az invalid.
    return not_missing & pd.to_numeric(s, errors="coerce").isna()
```

A mini CSV-ben `mini["basket_minor"]` vegyes oszlop; a maszk:

```
mini["basket_minor"].notna() & pd.to_numeric(mini["basket_minor"], errors="coerce").isna()
```

ez az `abc` sorra `True`-t ad, a hiány és a `150` értékre `False`-ot. A konverziót csak azután végezzük, hogy az invalid flag megvan.

4. A clean szűrés:

```
clean = raw_with_issue[
    ~raw_with_issue["issue_code"].isin(
        ["missing_distance_km", "invalid_basket_minor_negative"]
    )
].copy()

assert clean.shape[0] == 2402
assert clean["distance_km"].notna().all()
assert clean["basket_minor"].ge(0).all()
```

A `clean` részben továbbra is maradhat az `unknown_customer` hiba és az öt duplikált sor; ezeket későbbi 6.3. alfejezet és ügyfél-kulcs ellenőrzés kezeli. Ha egy sorra egyszerre több hibakód is igaz lett volna, a hibakód-értékadás sorrendje felülírta volna a korábbi kódot; a labori szerződés szerint a hibák külön sorokon vannak, ezért ez itt nem befolyásolja a 2402-es várt értéket.

6.1.9 Gyakori tévedés és ellenpélda

A leggyakoribb hiba a csendes `errors="coerce"` használata. A mini CSV `basket_minor` oszlopában az `abc` szöveg `NaN` lesz, és a sémajelentés hiányként mutatja. A következmény: a javítási szabály tévesen azt írja, hogy a hiányzó értékeket pótolni lehet, pedig az `abc` egyáltalán nem hiány, hanem sérült bemenet. Hasonló csendes összemosás az üres szöveg kezelése: az alapértelmezett `read_csv` az üres stringet is `NaN`-né alakítja, tehát ha a megfigyelési folyamat megkülönbözteti a „nem mért” és a „nem válaszolt” esetet, akkor auditáláskor `keep_default_na=False` szükséges. Az 5.1-ben a dtype és shape, az 2.1-ben a raw/derived elkülönítés tanította ugyanezt a célzott, explicit szerződéses szemléletet. Önellenőrzésként mindig kérdezzünk rá: ha futtatom a sémajelentést, akkor a `missing` és az `invalid_non_null` összege külön-külön egyezik a kézi labor-leírással?

A betöltéskor megadott `dtype` önmagában még nem igazolja a teljes adatkontraktust: a beolvasás után az egyediséget, az értéktartományokat, a hivatkozási integritást és a dátummezők feldolgozhatóságát is ellenőrizzük. A hiányzó és az érvénytelen érték külön hibakategória marad.

6.1.10 Rövid összefoglaló

Az explicit séma rögzíti az oszlopnevet, dtype-ot, mértékegységet és megengedett tartományt, így a `pd.read_csv` inferálása nem változtatja meg csendben az adattípust. A nullable `Int64`, `Float64` és `string` dtype a hiányt `pd.NA`-ként tárolja; a `category` a nominális szinteket kezeli, a `parse_dates` az UTC-időbélyegeket elemzi. A sémajelentés a hiányt, az érvénytelen értéket és az üres szöveget külön oszlopban számolja. A labor `orders_raw.csv` betöltése után 2405 sor, két hiányzó távolság, egy negatív kosárérték és egy idegen ügyfél várható; a javítási szabály hibakóddal zár ki, nem coerce-szel. A következő lépésben a kiválasztást és a módosítást végezzük Copy-on-Write mellett.

6.2 Kiválasztás és módosítás Copy-on-Write mellett

Belépő · szint: alap · idő: 45 perc · eszköz: pandas

Előfeltétel: 6.1. alfejezet · Célkészség: Kiválasztás és módosítás Copy-on-Write mellett · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

6.2.1 Cél és konkrét helyzet

A 6.1. alfejezetben explicit sémával betöltöttük az `orders_raw.csv` fájlt, és sémajelentésben azonosítottuk a hibás sorokat. Most a következő lépéshez érkeztünk: a betöltött `DataFrame`-ben javítani és kiválasztani szeretnénk adatokat anélkül, hogy a módosítás csendben egy ideiglenes másolaton történne, vagy hogy a pandas címkeillesztését összekeverjük a NumPy pozicionális műveletével.

A konkrét feladatunk két részből áll. Először ellenőrizni fogjuk, hogy két eltérő indexű `Series` összeadása hogyan viselkedik a pandasban: az indexek unióján, a hiányzó címkéken `pd.NA` vagy `NaN` értékkel, nem pedig pozicionálisan. Ezután egy láncolt értékadás (`chained assignment`) javítását végezzük el: a `business` szegmensű ügyfelek `priority_flag` oszlopát szeretnénk módosítani, és a hibás `df[df.segment == "business"]["priority_flag"] = 0` formát explicit `.loc` alakba írjuk át.

Előzetes eredménybecslés: a két eltérő indexű sorozat összeadásánál a kimenet hossza legfeljebb a két index uniójának mérete; ha az egyik sorozatnak van olyan címkéje, amely a másikban nincs, azon a helyen hiányzó érték jelenik meg. A láncolt értékadásnál pedig azt várjuk, hogy a hibás forma nem módosítja az eredeti `DataFrame`-et, vagy figyelmeztetést ad, míg az explicit `.loc` forma helyesen frissíti az értékeket.

6.2.2 Fogalmi magyarázat

6.2.2.1 loc és iloc: címke és pozíció

A pandas `DataFrame`-ből két alapvető módon választhatunk ki sorokat és oszlopokat.

- **`df.loc[sortartomány, oszloptartomány]`** (label-based location): a *címkékkel* dolgozik. A sortartomány a sorindex címkéire, az oszloptartomány az oszlopnevekre hivatkozik. A `loc` szeletelésnél a végpontot is beleérti; ez eltér a Python hagyományos slice-szemantikájától.
- **`df.iloc[sorpozíció, oszloppozíció]`** (integer position-based location): egész pozíciókkal dolgozik, 0-tól kezdődően. A szelet végpontja itt kizárt, ahogy a Python-listáknál megszoktuk.

A kettő közötti különbség a matematikai híd lényege: a címkék *halmazjellegű* azonosítók — nem feltétlenül rendezettek, nem feltétlenül egyediek, és nem feltétlenül egész számok —, míg a pozíciók *rendezett egész indexek*. Egy `(n,)`-es táblázatnak mindig van 0…n−1 pozíciója, de a címkéi lehetnek például karakteres `customer_id` értékek is. Ha a címkék történetesen egész számok, akkor sem keverendő a kettő: `df.loc[3]` a `3` címkéjű sort keresi, `df.iloc[3]` a negyedik sort.

Kis kézi példa:

```
import pandas as pd

df = pd.DataFrame(
    {"ertek": [10, 20, 30]},
    index=["c001", "c002", "c003"],
)
df.loc["c002", "ertek"]   # címke szerint: 20
df.iloc[1, 0]             # pozíció szerint: 20
```

6.2.2.2 Boolean mask

A **boolean mask** (logikai maszk) egy olyan, az indexhez illeszkedő `True`/`False` sorozat, amely kijelöli a megtartandó sorokat. Előállítása jellemzően összehasonlítással történik:

```
df = pd.DataFrame({"segment": ["business", "consumer", "business"],
                   "priority_flag": [1, 0, 1]})
mask = df["segment"] == "business"
```

A `mask` itt `Series[bool]`, ahol minden sorhoz egy logikai érték tartozik. A pandas `Series` típusú, címkézett maszkot közvetlenül a `.loc` fogadja el. A `.iloc` nem fogad el közvetlenül ilyen címkézett boolean `Series`-t: pozicionális logikai tömbbé kell alakítani (`mask.to_numpy(dtype=bool)`), miután ellenőriztük, hogy a sorrend és a hossz egyezik:

```
df.loc[mask, "priority_flag"]              # indexcímke-alapú kiválasztás
df.iloc[mask.to_numpy(dtype=bool), df.columns.get_loc("priority_flag")]  # pozicionális oszlop
```

A `.loc` esetén a `Series`-maszk indexe illeszkedjen a `DataFrame` indexéhez (eltéréskor figyelmeztetés vagy kivétel is felléphet); `.iloc` esetén a `numpy` boolean tömb hosszának kell megegyeznie a sorok számával, és kizárólag pozíció szerint választ. A két megoldás nem általánosan felcserélhető. Regressziós ellenőrzés:

```
import pandas as pd
check = pd.DataFrame({"value": [10, 20, 30]})
mask = check["value"] > 10
assert check.loc[mask, "value"].tolist() == [20, 30]
assert check.iloc[mask.to_numpy(dtype=bool), 0].tolist() == [20, 30]
# A check.iloc[mask] pandas 2.2.x alatt NotImplementedError hibát ad.
```

6.2.2.3 Index-illesztés

A pandas **index-illesztés** (`index alignment`) azt jelenti, hogy két `Series` vagy `DataFrame` közötti művelet a *címkék* szerint történik, nem a pozíció szerint. Ez alapvető különbség a NumPy-hoz képest, ahol az azonos alakú tömbök elemenként, pozíció szerint kapcsolódnak össze.

Példa: legyen

```
s1 = pd.Series([100, 200], index=["a", "b"])
s2 = pd.Series([5, 7], index=["b", "c"])
```

Az `s1 + s2` eredménye az indexek unióján, azaz `["a", "b", "c"]` felett értelmezett:

címke	s1	s2	összeg
a	100	hiány	hiány
b	200	5	205
c	hiány	7	hiány

A hiányzó címkéken a pandas `NaN`-t (numerikus `Series` esetén) vagy `pd.NA`-t (nullable dtype esetén) ad. Ha tehát két eltérő indexű sorozattal dolgozunk, az összeg hossza az unió mérete lesz, és a közös címkéken kívül hiányzó értékeket kapunk. Ez a viselkedés a relációs adatbázisok `FULL OUTER JOIN`-jához hasonlít, nem a NumPy pozicionális műveletéhez.

Az index-illesztés a `DataFrame`-ek közötti műveletekre is vonatkozik: ha például `df1 + df2`-t írunk, a kimenet sorait a két sorindex uniója, oszlopait a két oszlopkészlet uniója adja. A hiányzó kombinációkon `NaN` áll.

6.2.2.4 Copy-on-Write

A **Copy-on-Write** (CoW, „írásra másolás”) a pandas 2.0-ban jelent meg, a 2.2 sorozatban opcionális, a 3.0-tól kötelező memóriakezelési mód. A lényege: amikor egy `DataFrame`-ből szeleteléssel vagy szűréssel új objektumot hozunk létre, a pandas *nem* másolja azonnal az adatokat, hanem egy nézetet (`view`) tart fenn, és a tényleges másolás csak akkor történik meg, amikor valaki módosítani próbálja az adatot. Ezzel a gyakori, csak olvasásra szolgáló műveletek memóriaigénye csökken, a módosítások pedig nem „szivárognak vissza” az eredeti objektumba.

A gyakorlati következmény számunkra: a **láncolt értékadás** (`chained assignment`) nem megbízható. Az

```
df[df["segment"] == "business"]["priority_flag"] = 0
```

kifejezés két egymást követő `__getitem__` hívást jelent. Az első `df[df["segment"] == "business"]` egy szűrt részt ad; a második `["priority_flag"]` ennek a résznek egy oszlopát. Az értékadás ezután nem az eredeti `df`-en, hanem ezen a köztes objektumon történik. A viselkedés verzió- és beállításfüggő; a könyv két támogatott sorozatán mért eredmény (háromsoros példa, a `priority_flag` kiinduló értéke mindenhol 1):

pandas	CoW állapota	Láncolt értékadás után `df["priority_flag"]`	Figyelmeztetés
2.2.3	kikapcsolva (alapértelmezés)	`[1, 1, 1]` – az eredeti változatlan	`SettingWithCopyWarning`
2.2.3	bekapcsolva (`pd.options.mode.copy_on_write = True`)	`[1, 1, 1]` – az eredeti változatlan	`ChainedAssignmentError`
3.1.0	mindig bekapcsolva	`[1, 1, 1]` – az eredeti változatlan	`ChainedAssignmentError`

Egyik sorban sem módosul az eredeti `df`: a láncolt értékadás nem „néha működik”, hanem a támogatott verziókon megbízhatatlan. A `.loc` ezzel szemben mindhárom esetben `[0, 1, 0]` eredményt ad.

A javított alak egyetlen `.loc` hívás, amely egyszerre címzi a sorokat és az oszlopot:

```
df.loc[df["segment"] == "business", "priority_flag"] = 0
```

Itt a pandas egyetlen műveletben tudja, hogy az eredeti `df`-et kell módosítani. Ez CoW mellett is helyesen működik, mert a `.loc` explicit jelzi a módosítás helyét.

6.2.2.5 pandas verzió

A CoW a pandas 2.x sorozatban opcionális: a kezdőprojekt a `lab/pyproject.toml` szerint **pandas 2.2.3**-mal futott, ahol a CoW alapértelmezetten kikapcsolt. A pandas **3.0-tól a CoW mindig bekapcsolt**: a `pd.options.mode.copy_on_write` beállítás megmaradt ugyan, de deprecated, és nem kapcsolja ki a mechanizmust (a 3.1.0 `Pandas4Warning` figyelmeztetéssel jelzi, hogy a beállítás hatástalan). A tanulási környezetünk ezért két dolgot tesz:

1. A kódot eleve CoW-kompatibilisen írjuk: kizárólag explicit `.loc`/`.iloc` értékadást használunk, láncolt assignmentet soha.
2. A verziót a futás elején ellenőrizzük, hogy a környezeti eltérések ne maradjanak rejtve:

```
import pandas as pd

print(pd.__version__)
# A könyv két támogatott sora: "2.2.3" vagy "3.1.0"
```

Migrációs jegyzet 2.2.3 → 3.x. A `pd.options.mode.copy_on_write = True` sor elhagyható, mert 3.x-ben nem kapcsol ki semmit; a láncolt értékadást mindenhol `.loc`/`.iloc` alakra kell írni, a `SettingWithCopyWarning` helyét 3.x-ben a `ChainedAssignmentError` figyelmeztetés veszi át; a szelet módosításának visszaíródására egyik verzióban sem szabad építeni, önálló objektumhoz explicit `copy()` kell. A könyv példái a 2.2.3 és a 3.1.0 sorozaton is ugyanazt az eredményt adják, mert a `.loc` alakot használják.

A CoW a pandas 2.2-ben a `pd.options.mode.copy_on_write = True` beállítással kapcsolható be; 3.x-ben ez a beállítás hatástalan. A könyv példái nem feltételezik a beállítást; helyette az explicit műveleti stílust tanítjuk, amely mindkét verziócsaládban azonos eredményt ad. Az 5.2. alfejezetben látott NumPy-s `view`/`copy` megkülönböztetés itt is visszaköszön: a pandas a NumPy-tömbök `view`/`copy` viselkedésére épül, de az index-illesztés rétege megváltoztatja, hogy melyik művelet mikor ad másolatot.

6.2.3 Teljes minimális munkapélda

A laborhoz az `orders_raw.csv` már betöltött, explicit sémás változatát használjuk. A példát úgy építjük fel, hogy egy kis, kézzel ellenőrizhető `DataFrame`-en mutatjuk be a két kulcsműveletet, majd ugyanazt a formát alkalmazzuk a nagyobb táblázatra.

6.2.3.1 1. lépés: kis kézi táblázat

```
import pandas as pd

# Kézi példa: három rendelés, karakteres indexszel
df = pd.DataFrame(
    {
        "order_id": ["o1", "o2", "o3"],
        "segment": ["consumer", "business", "consumer"],
        "priority_flag": [0, 1, 0],
        "basket_minor": [1200, 4500, 800],
    }
).set_index("order_id")
df
```

Szemléltető kimenet:

order_id	segment	priority_flag	basket_minor
o1	consumer	0	1200
o2	business	1	4500
o3	consumer	0	800

6.2.3.2 2. lépés: két eltérő indexű sorozat művelete

```
# Két eltérő indexű sorozat
s1 = pd.Series([100, 200], index=["a", "b"])
s2 = pd.Series([5, 7], index=["b", "c"])

s_sum = s1 + s2
s_sum
```

Szemléltető kimenet:

index	érték
a	NaN
b	205
c	NaN

Az ellenőrzés kézzel: a `b` címkén $200 + 5 = 205$; az `a` és `c` címkék csak az egyik sorozatban szerepelnek, ezért ott hiányzó érték áll. A kimenet hossza 3, ami a két index uniójának mérete.

6.2.3.3 3. lépés: láncolt értékadás hiba és javítás

Hibás forma:

```
# Rossz: láncolt assignment
df[df["segment"] == "business"]["priority_flag"] = 0
# pandas 2.2-ben CoW nélkül: SettingWithCopyWarning,
# és az eredeti df NEM módosul megbízhatóan.
```

Javított forma:

```
# Helyes: egyetlen explicit .loc
df.loc[df["segment"] == "business", "priority_flag"] = 0
df
```

Szemléltető kimenet a javítás után:

order_id	segment	priority_flag	basket_minor
o1	consumer	0	1200
o2	business	0	4500
o3	consumer	0	800

Az `o2` rendelés `priority_flag` értéke 1-ről 0-ra változott, a többi sor érintetlen. Ez a módosítás egyetlen `.loc` hívással, az eredeti `DataFrame`-en történt.

6.2.3.4 4. lépés: a labor orders_raw tábláján

```
# Feltételezzük az s021-ből ismert betöltést:
# df_raw = pd.read_csv("lab/data/orders_raw.csv", dtype=schema)
# Itt a már betöltött df_raw egy részletével dolgozunk.

# Ellenőrzés: a business sorok módosítása explicit .loc formában
business_mask = df_raw["segment"] == "business"
df_raw.loc[business_mask, "priority_flag"] = 0

# Elvárt ellenőrzés: a business sorokban minden priority_flag 0
assert (df_raw.loc[business_mask, "priority_flag"] == 0).all()
```

6.2.4 Eredmény értelmezése

A kézi példa ellenőrizhető invariantja: a `s_sum = s1 + s2` sorozat indexe megegyezik a két bemeneti index uniójával (`["a", "b", "c"]`), és a közös címkén az érték a két bemenet összege (205). A hiányzó címkéken `NaN` áll. Ez a `FULL OUTER JOIN`-szemantika, nem a NumPy pozicionális művelete: ha a két sorozatot NumPy-tömbökként adnánk össze, a kimenet `[105, 207]` lenne, ami a második elemet tévesen $200 + 7 = 207$-ként számolná, mert a NumPy nem tud a címkékről.

A láncolt értékadás ellenőrzése: a hibás forma után az eredeti `df priority_flag` oszlopát megvizsgálva azt látjuk, hogy a `business` sor továbbra is 1 maradt (vagy a pandas verziójától függően figyelmeztetést kapunk). Az explicit `.loc` forma után az érték 0, és ezt `assert`-tel automatikusan ellenőrizzük. A maszk (`business_mask`) maga is `Series[bool]`, amelynek indexe illeszkedik a `DataFrame` sorindexéhez; a `.loc` ezt a címkék szerint illeszti, ezért a hossza pontosan megegyezik a sorok számával.

Verzióérzékenység: a fenti példák pandas 2.2.3-mal készültek. Pandas 3-ban a CoW alapértelmezett, ezért a hibás láncolt forma ott már nem is ad `SettingWithCopyWarning`-t, hanem egyszerűen nem módosítja az eredetit, vagy kivételt dob. Az explicit `.loc` forma mindkét verziócsaládban azonos eredményt ad; ezért ez a hordozható stílus.

6.2.5 Feladatok

1. feladat — Értelmezés. Adottak az alábbi sorozatok:

```
a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20], index=["y", "w"])
```

Mi lesz az `a + b` eredmény indexe és értékei? Válaszát indokolja az index-illesztés szabálya alapján, majd nevezze meg, mi a különbség a NumPy pozicionális összeadásához képest.

2. feladat — Implementálás. Töltsd be a `lab/data/orders_raw.csv` fájlt a 6.1. alfejezetben megismert explicit sémával. Készíts boolean maszkot azokra a sorokra, ahol a `distance_km` hiányzik. Javítsd a hiányzó távolságokat explicit `.loc` használatával a `business` szegmensű rendelések medián távolságára, ha a `segment` oszlop értéke `business`; a `consumer` sorokban használja a `consumer` mediánt. Ellenőrizd, hogy a javítás után nincs hiányzó `distance_km` érték. (A mediánnal a 13.2. alfejezet fog részletesen foglalkozni; itt a `df["distance_km"].median()` hívás elégséges.)

3. feladat — Hibadiagnózis. A következő kódot találja egy notebookban:

```
df[df["basket_minor"] < 0]["basket_minor"] = 0
```

A futtatás után a negatív `basket_minor` érték továbbra is jelen van a `df`-ben, bár a kód nem dobott kivételt. Magyarázd el, mi a hiba oka, és írd át a kódot explicit `.loc` formára. Indokold, miért működik a javított változat CoW mellett is.

4. feladat — Transzfer megváltozott feltételre. A laborban a `priority_flag` 0/1 jelzés. Tegyük fel, hogy az üzleti szabály megváltozik: a `consumer` szegmensű rendelések `priority_flag` értékét 1-re kell állítani, ha a `basket_minor` nagyobb, mint 5000. Írj olyan kódot, amely ezt a módosítást kizárólag explicit `.loc` művelettel végzi el, majd ellenőrizze a feltétel teljesülését `assert` segítségével. A megoldásban a feltétel és a beírandó érték is a rendelés pillanatában ismert adatból származzon.

5. feladat — Index-illesztés ellenőrző invariant. Két eltérő indexű `Series` összeadásánál a kimeneti indexnek meg kell egyeznie a két bemeneti index uniójával. Írj egy kis függvényt `assert`-ekkel, amely két tetszőleges `Series`-re ellenőrzi: (a) a kimeneti index mint halmaz megegyezik a két bemeneti index uniójával; (b) a közös címkéken a kimeneti érték a két bemenet összege. Teszteld a függvényt az 1. feladat `a` és `b` sorozatán.

6.2.6 Megoldások és indoklás

1. megoldás. Az `a + b` indexe a két index uniója: `["w", "x", "y", "z"]` (a sorrend a pandas rendezési szabálya szerint alakul, az itteni sorrend az unió rendezett formája). A `y` címke mindkét sorozatban szerepel, ott az érték $2 + 20 = 22$; a többi címkén `NaN`:

index	érték
w	NaN
x	NaN

index	érték
y	22
z	NaN

A NumPy pozicionális összeadás ezzel szemben az `[1+10, 2+20] = [11, 22]` eredményt adná, mert a tömböknek nincs címke-fogalmuk; a harmadik elem (`3`) elveszne, mert a második tömb rövidebb. A pandas ezt a hibát elkerüli, de hiányzó értékeket hoz be — ezért a kimeneti `NaN`-eket mindig ellenőrizni kell, mielőtt az összeget további számításba vonnánk.

2. megoldás. A 6.1. alfejezet sémáját használva:

```
import pandas as pd

schema = {
    "order_id": "string",
    "customer_id": "string",
    "order_ts": "string",        # majd parse_dates
    "promised_days": "Int64",
    "quantity_sum": "Int64",
    "item_rows": "Int64",
    "basket_minor": "Int64",
    "shipping_fee_minor": "Int64",
    "currency": "string",
    "distance_km": "Float64",
    "priority_flag": "Int64",
    "delivery_days": "Int64",
    "actual_delivery_at": "string",
    "label_available_at": "string",
    "target_late": "Int64",
    "segment": "string",
    "region": "string",
    "acquisition_channel": "string",
}

df_raw = pd.read_csv(
    "lab/data/orders_raw.csv",
    dtype=schema,
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)

missing = df_raw["distance_km"].isna()
business_med = df_raw.loc[df_raw["segment"] == "business", "distance_km"].median()
consumer_med = df_raw.loc[df_raw["segment"] == "consumer", "distance_km"].median()

df_raw.loc[missing & (df_raw["segment"] == "business"), "distance_km"] = business_med
df_raw.loc[missing & (df_raw["segment"] == "consumer"), "distance_km"] = consumer_med

assert not df_raw["distance_km"].isna().any()
```

Indoklás: a mediánt külön számoljuk a két szegmensre, mert a távolság eloszlása szegmensfüggő lehet; a csoportos medián jobb, mint egyetlen globális érték. A `.loc` explicit sor-oszlop párt ad, így a módosítás az eredeti `df_raw`-n történik. Az `assert` a végső invariantet ellenőrzi: nincs hiányzó távolság. (A szegmens szerinti medián imputáció modellezési döntés; a valós munkában ez egy indokolt, naplózott döntés, nem automatikus lépés.)

3. megoldás. A hiba oka a láncolt értékadás. A `df[df["basket_minor"] < 0]` kifejezés egy új, szűrt `DataFrame`-et ad, amely CoW nélkül nézet lehet, CoW-val pedig másolat. A rákövetkező `["basket_minor"] = 0` ezen a köztes objektumon hajtódik végre, nem az eredeti `df`-en. A

pandas ezért `SettingWithCopyWarning`-ot adhat, de az eredeti táblázatban a negatív érték megmarad — a kód „sikeres”, de hatástalan.

Javított alak:

```
df.loc[df["basket_minor"] < 0, "basket_minor"] = 0
```

Ez a forma egyetlen műveletben add meg a sorokat (boolean mask) és az oszlopot, így a pandas pontosan tudja, hogy az eredeti `df` módosítandó. CoW mellett is helyes, mert a `.loc` explicit módosítást jelez, nem egy ideiglenes objektumon operál.

4. megoldás.

```
cond = (df_raw["segment"] == "consumer") & (df_raw["basket_minor"] > 5000)
df_raw.loc[cond, "priority_flag"] = 1

# Ellenőrző invariant: a feltételnek megfelelő sorokban a zászló 1
assert (df_raw.loc[cond, "priority_flag"] == 1).all()
# Ellenőrző invariant: a feltételnek nem megfelelő consumer sorokban a zászló változatlan
# (itt csak azt ellenőrizzük, hogy a módosítás nem érintette a többi sort
#  egy korábban rögzített mintán; a teljes ellenőrzéshez a módosítás előtti állapot kell.)
```

A feltétel két részből áll: a szegmens `consumer`, és a kosárérték meghaladja az 5000 minor egységet. A `&` operátor elemenkénti logikai ÉS a két boolean `Series` között; a kerek zárójelek a Python precedenciája miatt kötelezőek. A módosítás kizárólag a feltételnek megfelelő sorokat érinti.

5. megoldás.

```
def check_series_alignment(s1: pd.Series, s2: pd.Series) -> None:
    out = s1 + s2
    expected_index = s1.index.union(s2.index)
    # (a) a kimeneti index mint halmaz megegyezik az unióval
    assert set(out.index) == set(expected_index)
    # (b) közös címkéken az összeg helyes
    common = s1.index.intersection(s2.index)
    for idx in common:
        assert out.loc[idx] == s1.loc[idx] + s2.loc[idx]
        # Numerikus pontosság miatt éles tesztben:
        # assert abs(out.loc[idx] - (s1.loc[idx] + s2.loc[idx])) < 1e-12

# Teszt az 1. feladat adatain
a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20], index=["y", "w"])
check_series_alignment(a, b)
print("Az index-illesztési invariant teljesül.")
```

A függvény két invariantet ellenőriz: a kimeneti index mint halmaz az unió, és a közös címkéken az összeg pontos. Lebegőpontos sorozatoknál a `==` helyett toleranciás összehasonlítást kell használni, ahogy az 5.3. alfejezetben tárgyaltuk; itt egész értékekkel a pontos egyenlőség is elfogadható a szemléltető példában.

6.2.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a pandas `Series` műveleteket NumPy pozicionális szemantikával várjuk. Tekintsük a következő két rendelés-összeg sorozatot:

```
rev_a = pd.Series([100, 200], index=["o1", "o2"])  # o1, o2 rendelések
rev_b = pd.Series([50, 70], index=["o2", "o3"])    # o2, o3 rendelések
```

Ha valaki NumPy-szokás szerint azt várja, hogy `rev_a + rev_b` eredménye `[150, 270]`, akkor téved: a pandas a címkék szerint illeszt, és az eredmény indexe `["o1", "o2", "o3"]`, értékei pedig `[NaN, 250, NaN]`. Az `o1` és `o3` rendeléseknél hiányzó érték keletkezik, mert azok a másik sorozatban nem szerepelnek. A helyes összegzéshez az indexeket előbb igazítani kell, például `rev_a.add(rev_b, fill_value=0)` hívással, amely a hiányzó címkéket 0-ként kezeli, és így `[100, 250, 70]` eredményt ad.

Egy másik, kifejezetten a `loc`/`iloc` keveréséből fakadó hiba: ha a sorindex `0, 1, 2, ...` egész számokból áll, a `df[0]` kifejezés nem az első sort, hanem a `0` *oszlopnevet* keresi (ha ilyen nincs, `KeyError`-t ad). Az `iloc[0]` az első sort adja. Az index értékeinek és a pozícióknak az összekeverése olyan csendes hibát okozhat, amely csak az index átrendezésekor derül ki. Ezért minden módosításnál dönteni kell: `loc` címkével, vagy `iloc` pozícióval, de a kettőt soha nem szabad implicit módon váltogatni.

6.2.8 Önellenőrzés

- Mi a különbség a `df.loc[df["x"] > 0, "y"]` és a `df[df["x"] > 0]["y"]` kifejezés között? *Válasz:* az első egyetlen `.loc` hívás, amely a sorokat és az oszlopot egyszerre címzi, ezért módosíthatja az eredeti `df`-et; a második láncolt kiválasztás, amelynek az értékadása nem az eredetin történik.
- Két eltérő indexű `Series` összeadásakor a kimeneti index mi lesz? *Válasz:* a két index uniója; a hiányzó címkéken hiányzó érték áll.
- Mi a Copy-on-Write szerepe a láncolt értékadásban? *Válasz:* CoW mellett a szeletelésből kapott köztes objektum módosítása nem íródik vissza az eredeti `DataFrame`-be, mert az objektum nem nézet, hanem másolatként kezelendő.
- Melyik pandas verzióban alapértelmezett a CoW? *Válasz:* a pandas 3.0-tól kötelezően mindig bekapcsolt; a 2.2.3-ban opcionális, alapértelmezés szerint kikapcsolt.

Copy-on-Write használatakor a `df.loc[maszk, oszlop] = érték` alakú, egy lépésben végzett módosítás előnyösebb a láncolt hozzárendelésnél. A nézetek és másolatok viselkedése, illetve a címkék szerinti automatikus indexillesztés két külön kérdés: mindkettőt önállóan kell ellenőrizni.

6.2.9 Rövid összefoglaló

A pandas kiválasztás két útja a `loc` (címke szerint, a szeletvégpontot is beleértve) és az `iloc` (pozíció szerint, a végpontot kizárva); a címkézett `Series` maszk a `.loc`-kal használható; a `.iloc` pozicionális boolean tömböt fogad, például `mask.to_numpy(dtype=bool)` alakban. A `Series`-ek közötti műveletek index-illesztéssel történnek: a kimeneti index a bemeneti indexek uniója, a hiányzó címkéken `NaN` vagy `pd.NA` áll — ez nem NumPy pozicionális szemantika. A Copy-on-Write a pandas 3.0-tól kötelezően bekapcsolt, a kezdőlabor 2.2.3-as verziójában opcionális; a láncolt értékadás egyik módban sem megbízható, ezért minden módosítást egyetlen explicit `.loc`/`.iloc` hívással végzünk. A következő alfejezetben a duplikáció, a kulcs és az adatminőségi szerződés ellenőrzését építjük ezekre a kiválasztási műveletekre.

6.3 Duplikáció, kulcs és adatminőségi szerződés

Belépő · szint: alap · idő: 45 perc · eszköz: pandas

Előfeltétel: 6.2. alfejezet · Célkészség: Duplikáció, kulcs és adatminőségi szerződés · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

6.3.1 Cél és konkrét helyzet

A Northstar kezdőlabor `orders_raw.csv` fájlja szándékosan hibás másolata a tiszta rendelési adatnak. A megfigyelési egység itt **egy rendelés**, a sorazonosító pedig az `order_id`. A tisztaz `orders.csv` szerződése szerint minden `order_id` pontosan egyszer fordul elő, és a fájl 2400 sort tartalmaz. A nyers fájl ezzel szemben 2405 soros: öt extra sor pontos, teljes sorra kiterjedő másolat, ezen felül más hibák is vannak benne, amelyeket későbbi alfejezetek kezelnek.

Ebben a laborban az a feladat, hogy a duplikált sorokat dokumentált szabállyal eltávolítsuk, a kivett sorokat karanténba helyezzük, és a tisztított darabszámot a kontrollhoz hasonlítsuk. A kód megírása előtt adjunk előzetes becslést: **öt sort kell karanténba tenni, és 2400 sornak kell maradnia**. Ha a futás ettől eltér, akkor vagy a fájl nem a szerződés szerinti verzió, vagy a szabály hibás.

6.3.2 Fogalmi magyarázat

6.3.2.1 Megfigyelési egység és sorazonosító

Egy tábla akkor „tiszta” ebből a szempontból, ha a megfigyelési egység és a kulcs között kölcsönösen egyértelmű kapcsolat van:

$$N_{\text{sor}} = N_{\text{egyedi rendelés}} + N_{\text{extra másolat}}$$

ahol N_{sor} a fájl sorainak száma, $N_{\text{egyedi rendelés}}$ a különböző `order_id` értékek száma, $N_{\text{extra másolat}}$ pedig a duplikátumok miatt felesleges sorok száma. Ha nincs kulcsütközés és hiányzó kulcs, a laborban ez $2405 = 2400 + 5$.

6.3.2.2 `duplicated` és `drop_duplicates`

A `DataFrame.duplicated()` (magyarul: ismétlődő) logikai maszkot ad: `True`, ha az adott sor a korábbi sorok pontos másolata. Az alapértelmezés minden oszlopot vizsgál, és a `keep="first"` az első előfordulást nem jelöli duplikátumnak. A `drop_duplicates()` ezt a maszkot használja a sorok eldobására.

```
import pandas as pd

mini = pd.DataFrame({
    "order_id": ["o1", "o2", "o1"],
    "amount": [10, 20, 10],
})

mini.duplicated(keep=False).tolist()
# [True, False, True]
```

A `subset=["order_id"]` paraméterrel csak a kulcsra szűrünk. Ez veszélyesen hallgat el egy fontos esetet: ha ugyanaz a kulcs kétszer szerepel, de a többi oszlop eltér, az **kulcsütközés**, nem pedig egyszerű másolat. Ilyenkor az első vagy utolsó sor automatikus megtartása adatvesztést okozhat, ezért a pontos másolat tényét külön ellenőrizni kell.

6.3.2.3 Egyediség és nullarány

Az egyediség (uniqueness) ellenőrzése a tisztítás után:

```
clean["order_id"].is_unique
```

Ennek megbízható használatához a kulcsban nem lehet hiányzó érték. A nullarány (missing rate) képlete egy oszlopra:

$$\text{nullarány}(x) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[\text{hiányzó}(x_i)]$$

ahol n a sorok száma, x_i az i-edik sor értéke, és $\mathbb{1}[\cdot]$ 1, ha az érték hiányzik, 0 különben. A mutató értéke 0 és 1 közé esik. A pandas a `NaN` értékeket a duplikációvizsgálatban egyenlőnek tekinti, ezért két különböző, hiányzó `order_id`-jű sor hamisan duplikátumnak látszhat. A kulcs nullarányát tehát a deduplikálás előtt ellenőrizni kell.

6.3.2.4 Karantén és audit trail

A karantén (quarantine) olyan külön táblába mentett gyűjtemény, amely az eldobott sorokat és az eldobás okát tartalmazza. Az audit trail (auditnapló, ellenőrzési nyomvonal) a tisztítási lépés bemenetét, szabályát és kimeneti darabszámait rögzíti. Így a döntés később visszakereshető, és a folyamat újra lejátszható.

6.3.2.5 Üzleti kivételek: SCD2 és eseménytörténet

Az „egy kulcs – egy sor” szabály nem minden táblára igaz. Három esetet kell megkülönböztetni:

Eset	Tábla példa	Kulcs	Több sor egy kulcshoz?	Szabály
egyedi entitás	`orders`, `assignments`, `outcomes`	`order_id`, `customer_id`	nem	kulcsütközés = hiba
lassan változó dimenzió (SCD2)	`customers_scd`	`customer_id` + `valid_from`	igen, időben nem átfedő intervallumokkal	az intervallumok nem fedhetnek át; a nyitott intervallum csak az utolsó lehet
eseménytörténet (event/CDC)	`order_events`	`event_id`	igen, `customer_id` szerint	az `event_id` egyedi; az eseményidő monoton; az azonos esemény kétszer nem szerepelhet

Az **SCD2** (slowly changing dimension, 2-es típus) a dimenzió változásait új sorokkal tartja meg, `valid_from`/`valid_to` intervallummal. A **CDC** (change data capture) az forrásrendszer változásait eseménysorként rögzíti; itt a `customer_id` vagy `order_id` természetesen ismétlődik,

mert egy entitáshoz több esemény tartozik. A kulcsütközés tehát önmagában nem hiba: attól függ, hogy a tábla granularitása mit enged meg.

Példa SCD2-re:

customer_id	valid_from	valid_to	segment
C1	2025-01-01	2025-02-01	consumer
C1	2025-02-01	(nyitott)	business
C2	2025-01-01	(nyitott)	consumer

Itt a `C1` kétszer szerepel, de az intervallumok nem fednek át, ezért ez érvényes történet. Ha a második sor `valid_from` értéke `2025-01-15` lenne, az átfedés hibás; ha a két sor minden mezője azonos, az pontos másolat, és mindig hiba. Eseménytörténetnél:

event_id	customer_id	event_ts	type
E1	C1	2025-01-01 10:00	created
E2	C1	2025-01-01 11:00	shipped
E3	C2	2025-01-01 12:00	created

Az `E1` és `E2` azonos `customer_id`-hez tartozik, ez megengedett; az `E1` megismétlése viszont `event_id`-ütközés, ezért hiba. A `lab/duplicate_contract.py` a három esetet egy közös `classify()` függvénnyel kezeli:

```
import pandas as pd

def classify_unique(frame, key):
    conflicts = int(frame.duplicated(subset=key, keep=False).sum())
    return {"kind": "unique", "status": "error" if conflicts else "ok",
            "key_collisions": conflicts}

def classify_scd2(frame, key, valid_from, valid_to):
    errors = []
    if frame.duplicated().any():
        errors.append("pontos másolat")
    for value, group in frame.groupby(key, sort=False):
        group = group.sort_values(valid_from)
        if group[valid_from].duplicated().any():
            errors.append(f"{value}: ismétlődő valid_from")
        ends = group[valid_to].tolist()
        starts = group[valid_from].tolist()
        for i in range(len(group) - 1):
            if pd.isna(ends[i]) or ends[i] > starts[i + 1]:
                errors.append(f"{value}: átfedő vagy nyitott intervallum")
                break
    return {"kind": "scd2", "status": "error" if errors else "ok",
            "allowed_history_rows": sum(max(0, len(g) - 1)
                                        for _, g in frame.groupby(key, sort=False)),
            "errors": errors}

def classify_event(frame, key, event_id, event_ts):
    errors = []
    if frame.duplicated().any():
        errors.append("pontos másolat")
    if frame[event_id].duplicated().any():
        errors.append("ismétlődő event_id")
    if not frame[event_ts].is_monotonic_increasing:
```

```
        errors.append("az eseményidő nem monoton")
    return {"kind": "event", "status": "error" if errors else "ok",
            "allowed_history_rows": len(frame) - len(frame.drop_duplicates(subset=key)),
            "errors": errors}
```

Kimenet	Jelentés
`status: ok`, `allowed_history_rows: 0`	egyedi kulcsú tábla, nincs ismétlődés
`status: ok`, `allowed_history_rows > 0`	SCD2 vagy eseménytörténet, a több sor üzletileg indokolt
`status: error`, `exact_duplicates > 0`	pontos másolat; minden táblában hiba
`status: error`, `key_collisions > 0` egyedi kulcsnál	kulcsütközés; nem eldobható csendesen
`status: error`, SCD2 átfedés	az intervallumok ütköznek; a történet nem konzisztens
`status: error`, esemény `event_id` ütközés	ugyanaz az esemény kétszer szerepel

A döntés tehát nem a „van-e ismétlődő kulcs?” kérdésből, hanem a tábla deklarált granularitásából indul. A `tests/test_duplicate_contract.py` az egyedi kulcsot, az érvényes SCD2-történetet, az átfedő intervallumot, a pontos másolatot, az eseménytörténetet és az `event_id`-ütközést teszteli.

6.3.3 Teljes minimális munkapélda

A következő szkriptet a projektgyökérből futtatjuk. Platformfüggetlen `pathlib` útvonalat használunk. Az időbélyeget most szándékosan szövegként olvassuk be: a duplikációvizsgálat a fájl tartalmát hasonlítja, a dátumkonverziót a 6.4. alfejezetre hagyjuk.

```
from pathlib import Path
import json

import pandas as pd

DATA_DIR = Path("lab")
OUT_DIR = Path("work/s023")
OUT_DIR.mkdir(parents=True, exist_ok=True)

raw = pd.read_csv(
    DATA_DIR / "orders_raw.csv",
    dtype={
        "order_id": "string",
        "customer_id": "string",
        "currency": "string",
    },
)

# 1. A kulcs legyen teljesen kitöltött; NaN miatt hamis duplikátum lehet.
key_null_rate = float(raw["order_id"].isna().mean())
assert key_null_rate == 0.0, f"order_id hiányzik: {key_null_rate:.4f}"

# 2. Teljes sorra kiterjedő duplikátumok.
full_dup_mask = raw.duplicated(keep=False)
full_dup_df = raw.loc[full_dup_mask].copy()

# 3. Ellenőrzés: a duplikált sorok valóban pontos másolatok-e kulcsonként.
payload_wide = full_dup_df.groupby("order_id", dropna=False).nunique(dropna=False)
payload_identical = (payload_wide == 1).all(axis=1)
```

```
assert payload_identical.all(), "Kulcsütközés: eltérő tartalmú sorok azonos order_id-vel."

dup_counts = full_dup_df.groupby("order_id", dropna=False).size()
assert dup_counts.max() == 2, "Ebben a laborban csak páros duplikátumokat várunk."

# 4. Egy másolat megtartása, a többi karanténba kerül.
to_remove_mask = raw.duplicated(keep="first")

quarantine = raw.loc[to_remove_mask].copy()
quarantine.insert(0, "audit_reason", "exact_full_row_duplicate")
quarantine.insert(1, "audit_rule", "keep_one_of_verified_exact_copies")

clean = raw.loc[~to_remove_mask].copy()

# 5. A sorazonosító egyedisége a kimenetben.
assert clean["order_id"].is_unique

# 6. Darabszám-összevetés a tiszta kontrollal.
control = pd.read_csv(
    DATA_DIR / "orders.csv",
    usecols=["order_id"],
    dtype={"order_id": "string"},
)
assert len(clean) == len(control)

# 7. Karantén és audit trail mentése.
quarantine.to_csv(OUT_DIR / "duplicate_quarantine.csv", index=False)
clean.to_csv(OUT_DIR / "orders_deduped.csv", index=False)

audit = {
    "input_file": "lab/data/orders_raw.csv",
    "input_rows": int(len(raw)),
    "duplicate_group_rows": int(full_dup_mask.sum()),
    "duplicate_groups": int(dup_counts.shape[0]),
    "rows_removed": int(to_remove_mask.sum()),
    "rows_kept": int(len(clean)),
    "control_rows": int(len(control)),
    "key_null_rate": key_null_rate,
    "rule": "keep_one_of_verified_exact_copies",
    "note": "A többi raw hibát a későbbi alfejezetek kezelik; ez a lépés csak a duplikációt
    ↪  javítja.",
}

(OUT_DIR / "audit.json").write_text(
    json.dumps(audit, indent=2, ensure_ascii=False),
    encoding="utf-8",
)

print(audit)
```

Futtatás uv-projektben:

```
# Windows, macOS és Linux shellben egyaránt
uv run python scripts/s023_dedup.py
```

A seed=20261020 labor szerződéséből levezethető, szemléltető értékek: `input_rows=2405`, `duplicate_group_rows=10`, `duplicate_groups=5`, `rows_removed=5`, `rows_kept=2400`, `control_rows=2400`. Ez nem helyi mérési eredmény; akkor válik mérté, ha a te gépeden a lab fájljaival lefut, és a `manifest.json` szerint azonosak a generálási paraméterek.

6.3.4 Eredmény értelmezése

A kód három invariant ellenőriz, mielőtt eldobna bármit: a kulcs nullaránya 0, minden duplikált csoport mérete 2, és a csoporton belüli tartalom minden oszlopban azonos. Ezért az öt eltávolított sor valóban felesleges másolat, nem pedig eltérő állapotú rendelés. A `keep="first"` itt csak azért biztonságos, mert a másolatok tartalma azonos; a fájlbeli sorrend nem hordoz üzleti jelentést.

A kimenet 2400 sora megegyezik a kontroll `orders.csv` darabszámával. Ez azonban **csak darabszám-egyezés**: a nyers fájlban még megvan a két hiányzó `distance_km`, a negatív `basket_minor` és a nem létező `customer_id`. A teljes tartalmi egyezéshez ezeket külön szabályokkal kell majd kezelni, és csak utána érdemes a kontrollhoz hasonlítani. A karanténfájl a kivett sorokat pontosan megőrzi, az `audit.json` pedig a lépés visszakereshetőségét biztosítja.

6.3.5 Feladatok

1. Értelmezés. Adott a következő kis tábla:

```
df = pd.DataFrame({
    "order_id": ["o1", "o2", "o1", "o3", "o3"],
    "amount":   [10,   20,   10,   30,   31],
})
```

Add meg a `df.duplicated()` és a `df.duplicated(subset=["order_id"], keep=False)` kimenetét. Hány sort tartana meg a `df.drop_duplicates(subset=["order_id"], keep="first")`, és milyen problémát rejt el?

2. Implementálás. Írj függvényt `exact_duplicate_groups(df, key_column)` néven, amely visszaad egy `Series`-t: azokra a kulcsokra `True`, ahol legalább két teljes sor duplikátum van, és a csoporton belül minden oszlop értéke azonos; különben `False`. A hiányzó kulcsokat ne kezeld automatikusan egy csoportként; a függvény dobjon `ValueError`-t, ha a kulcsban hiányzó érték van. Használd a `groupby(...).nunique(dropna=False)` technikát, és ellenőrizd az 1. feladat `df`-jén, ahol a várt eredmény az `o1` kulcsra `True`, az `o3` kulcsra `False`.

3. Hibadiagnózis. Egy kolléga a következőt futtatta:

```
deduped = updates.drop_duplicates(subset=["order_id"], keep="first")
```

A bemenet frissítési napló:

```
updates = pd.DataFrame({
    "order_id":    ["R1", "R2", "R1", "R2"],
    "captured_at": ["2025-01-01 08:00", "2025-01-01 08:00",
                    "2025-01-02 09:00", "2025-01-02 09:00"],
    "status":      ["new", "new", "shipped", "cancelled"],
})
```

Mi a szemantikai hiba a `keep="first"` használatában? Adj olyan dokumentált szabályt, amely megőrzi a legfrissebb állapotot, és mondd meg azt is, mikor kell karanténba tenni a csoportot dobás helyett.

4. Megváltozott feltétel. Egy új betöltés az eredeti 2405 soros `orders_raw.csv` mellett két újabb sort is tartalmaz ugyanazzal az `order_id="X"` kulccsal, de `basket_minor` értékük 100, illetve 200. Minden más megegyezik. Hány sort távolít el a fenti minimális példa szabálya, és hány sor marad? Miért bukik el az `assert clean["order_id"].is_unique`? Írd le a módosított szabályt, és add meg a várható kimeneti sorok számát is.

6.3.6 Megoldások és indoklás

1. A teljes sorra vonatkozó duplikáció maszkja `[False, False, True, False, False]`, mert csak a harmadik sor pontos másolata az elsőnek. A kulcsra vonatkozó `keep=False` maszkja `[True, False, True, True, True]`, mert az `o1` és az `o3` kulcs is többször szerepel. A `drop_duplicates(subset=["order_id"], keep="first")` három sort tart meg: `o1` az `amount=10`, `o2` az `amount=20`, `o3` az `amount=30` értékkel. Ez elrejti, hogy az `o3` kulcshoz két különböző összeg tartozik; a `31` értékű sor csendesen eltűnik. Ha a szerződés szerint az `o3` két állapota fontos, a csendes eldobás hibás adathoz vezet.

2. Egy megoldás:

```
def exact_duplicate_groups(df, key_column):
    if df[key_column].isna().any():
        raise ValueError("A kulcs nem tartalmazhat hiányzó értéket.")

    dup_mask = df.duplicated(keep=False)
    dup_df = df.loc[dup_mask]

    if dup_df.empty:
        return pd.Series(dtype=bool)

    payload_counts = dup_df.groupby(key_column).nunique(dropna=False)
    return (payload_counts == 1).all(axis=1)
```

A 3. feladat `updates` tábláján a függvény az `R1` kulcsra `False`-t adna, mert a `status` és `captured_at` oszlopban eltérő értékek vannak a két sorban. A függvény nem dob el semmit, csak osztályoz: a pontos másolatokat elkülöníti a kulcsütközéstől.

3. A `keep="first"` azt feltételezi, hogy a fájlbeli sorrend üzletileg is az elsődleges sor. Itt viszont a `captured_at` az irányadó: a későbbi sor a frissebb állapot. Helyes szabály lehet: rendezés `captured_at` szerint, majd `keep="last"`, de csak azonos `order_id` mellett, és miután ellenőriztük, hogy nincs két különböző állapot azonos időbélyeggel. Ha ugyanazon rendeléshez ugyanazon időpontban ellentmondó állapotok tartoznak, a csoportot nem szabad csendesen eldobni; karanténba kell tenni a teljes csoportot, és a forrásrendszer tulajdonosával egyeztetni.

4. Az eredeti példa szabálya a teljes sorra vonatkozó `duplicated(keep=False)` maszkot használja. Az `X` két sora nem teljes sorra azonos, mert a `basket_minor` különbözik, ezért a szabály csak az eredeti öt másolatot dobja el. A kimenet így $2400 + 2 = 2402$ sor lenne, és az `assert clean["order_id"].is_unique` megbukik, mert az `X` kétszer szerepel. Módosított szabály: először az exakt másolatokat távolítsuk el, majd a megmaradt kulcsütközéseket teljes csoportként karanténba tesszük. Az új, 2407 soros bemenetből az öt egzakt duplikátum eltávolítása után **2402** sor marad; ebből az `X` kulcsú két ütköző sor karanténba kerül, így a végeredmény **2400** soros. A kontrollal egyező sorSZÁM önmagában nem elegendő: az `order_id`-halmazt, a hibajegyzéket és a tartalmat is egyeztetni kell.

6.3.7 Gyakori tévedés és önellenőrzés

A leggyakoribb hiba a `drop_duplicates(subset=["order_id"])` automatikus használata az üzleti szabály kimondása nélkül. Ha a duplikált kulcshoz eltérő tartalom tartozik, az első sor megtartása tetszőleges, és elrejti az adatminőségi problémát. Ezért a dobás előtt mindig kérdezzük meg: a megfigyelési egység és a kulcs tényleg ugyanaz? A kulcs hiánytalan-e? A duplikátum pontos másolat vagy kulcsütközés? A megtartott sor kiválasztása determinisztikus és dokumentált? A karanténba tett sorokat tudjuk-e később visszaállítani? Az 5.3. alfejezetből idézett numerikus óvatosság ide is érvényes: számított lebegőpontos oszlopoknál a `==`-egyenlőség

törékeny, ezért duplikációvizsgálat előtt a nyers fájl értékeit hasonlítsuk, vagy toleranciával ellenőrizzünk. A 2.3. alfejezet Git-szabályai szerint a tisztító szkriptet és a szabályt verziózzuk; a generálható nagy nyers fájlok helyett a reprodukálhatósághoz elegendő a seed és a manifest hivatkozása.

Az egzakt duplikátumok eltávolítása elkülönítendő az azonos kulcsú, de eltérő adatokat tartalmazó rekordok feloldásától. Utóbbiakra a `keep="first"` nem általános javítási szabály; szükség van dokumentált üzleti döntésre vagy karanténra.

6.3.8 Rövid összefoglaló

A duplikációvizsgálat előtt tisztázni kell, hogy egy sor milyen megfigyelési egységet jelent. A `duplicated` teljes sorra, illetve `subset` paraméterrel kulcsra is vizsgálható; a kulcs szerinti automatikus eldobás azonban elrejtheti a kulcsütközést. A Northstar laborban az öt extra sor pontos másolat, ezért ellenőrzött csoportazonosság után egy példány megtartható, a többi karanténba kerül. A kimenet 2400 sora megegyezik a kontroll darabszámával, de a többi nyers hiba miatt ez még nem teljes tartalmi egyezés. Az audit trail rögzíti a szabályt, a bemeneti és kimeneti darabszámokat. A következő lépésben a dátumokat és időzónákat kezeljük ugyanezen a rendelési táblán.

6.4 Dátumok, időzónák és kategóriák

Belépő · szint: alap · idő: 45 perc · eszköz: pandas

Előfeltétel: 6.3. alfejezet · Célkészség: Dátumok, időzónák és kategóriák · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

6.4.1 Cél és konkrét helyzet

Az előző alfejezetben az `orders_raw.csv` hibáit dokumentáltan karanténba helyeztük, és a tiszta kontroll, a `lab/data/orders.csv` 2400 rendelési sorát kaptuk. Ebben az alfejezetben a rendelési időpontot, az `order_ts` oszlopot típusbiztos `datetime` oszloppá alakítjuk, és napi aggregációt készítünk belőle. Megfigyelési egységünk továbbra is egy rendelés; a cél egy olyan napi rendelésszám-tábla, amely a megjelenítési zóna szerinti naptári napokon alapul. Az üzleti jelentés nem UTC-ben, hanem például `Europe/Budapest` zónában gondolkodik, ezért a naphatárt és a hiányzó napokat külön dokumentálnunk kell.

Mielőtt kódot írnánk, becsüljük meg, hány rendelés kerülhet át másik naptári napra. A 2400 sor 180 napra oszlik, tehát óránként átlagosan $2400/(180 \cdot 24) \approx 0{,}56$ rendelés érkezik. A budapesti és az UTC naphatár közötti eltérés télen 1, nyáron 2 óra; a határhoz közeli órákban naponta jellemzően legfeljebb egy sor, a teljes adaton néhány tucat sor kerülhet át másik naptári napra. A pontos szám az adat tényleges eloszlásától függ; ezt a záró ellenőrzésben mérjük.

6.4.2 UTC-időpont, időzóna és naphatár

Egy időbélyeg (timestamp) **egyetlen pillanatot** jelöl a világ időtengelyén. Az `order_ts` a CSV-ben ISO 8601 formátumú, explicit `+00:00` offsettel, tehát UTC. A `pd.to_datetime` ezt `datetime64[us, UTC]` típusú, úgynevezett **timezone-aware** (zónatudatos) értékké alakítja; a pontos felbontás pandas-verziónként változhat, ezért a dtype-sztringre nem építünk üzleti logikát.

A naive (zóna nélküli) időpont ezzel szemben nem hordozza a pillanat világbeli helyét; ugyanaz a `2025-03-01 12:00` jelenthet UTC-t vagy budapesti időt is.

```
import pandas as pd

orders = pd.read_csv("lab/data/orders.csv")

# A CSV-ből stringként érkező oszlop típusos datetime lesz.
orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)

print(orders["order_ts"].dtype)              # szemléltető: datetime64[us, UTC]
print(orders["order_ts"].dt.tz)              # UTC
print(orders["order_ts"].min())              # 2025-01-01 ...
print(orders["order_ts"].max())              # 2025-06-30 ...
```

A `utc=True` azért fontos, mert akkor is garantáltan zónatudatos oszlopot kapunk, ha egy későbbi fájlformátum vagy CSV-változat véletlenül offset nélkül adná az időpontot. A zónatudatos értéken két alapművelet van: a `tz_localize` egy naive értékhez **hozzárendel** egy zónát, a `tz_convert` pedig egy már zónatudatos értéket **átszámol** másik zónába. A kettő nem cserélhető fel: a Northstar oszlop már aware, ezért a `tz_localize` hívása `TypeError`-t ad, a `tz_convert` pedig helyes.

A naptári nap az **időablak**, nem pont. A nap $[a, b)$ intervallum, ahol a a helyi éjfél, b a következő helyi éjfél; a hossza $b - a$ időtartam (Timedelta). Az UTC szerinti nap mindig 24 óra, de a budapesti zóna 2025. március 30-án 02:00-ról 03:00-ra ugrik, ezért az a naptári nap csak 23 óra hosszú. A `resample("D")` zónatudatos indexen a helyi naptári napokat követi; UTC indexen viszont a fix 24 órás ablakokat. A naphatár tehát nem pusztán megjelenítés: az határozza meg, melyik naphoz tartozik egy rendelés.

A `resample` előfeltétele, hogy az index `DatetimeIndex` legyen, és a sorok időben rendezettek legyenek. A `groupby` nem adja ingyen a hiányzó napokat; a `resample` a teljes min–max tartományra generál napokat, így az üres napok 0 értékkel jelennek meg.

6.4.3 Kategóriák mint explicit szintek

A `segment` és `region` oszlopok kis számú, ismert értéket tartalmazó kategóriák. A `category` dtype három dolgot javít: kevesebb memóriát foglal, rögzíti a megengedett szinteket, és a véletlen szöveghibák hamarabb kiderülnek. A `category` nem rendezett numerikus kód; az `astype("category")` nem ad sorrendet, és a `.cat.codes` értékeit nem szabad a `segment` vagy `region` közötti távolságként értelmezni.

```
orders["segment"] = orders["segment"].astype("category")
orders["region"] = orders["region"].astype("category")

print(orders["segment"].cat.categories)     # Index(['business', 'consumer'], dtype='object')
print(orders.dtypes[["segment", "region"]]) # category, category
```

Az időtartam (Timedelta) az időpontok különbsége. A `delivery_days` oszlop a szerződés szerint pozitív egész nap, míg az `actual_delivery_at - order_ts` egy `Timedelta`. A kettő nem ugyanaz: az egész nap egy mennyiség, a `Timedelta` pedig az időtengelyen mért hossz.

6.4.4 Teljes minimális munkapélda: napi aggregáció két zónában

A következő példa a `lab/data/orders.csv` tiszta fájlra épül, a projektgyökérből futtatva. A kimeneteket szemléltetőnek jelöljük, mert ebben az írási módban nem történt valódi futás; a tanuló saját környezetében a számoknak a `manifest.json` kontrolljával kell egyezniük.

```
import pandas as pd

orders = pd.read_csv("lab/data/orders.csv")
orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)
orders["segment"] = orders["segment"].astype("category")
orders["region"] = orders["region"].astype("category")

# Invariáns: a tiszta fájlban egy sor egy rendelés.
assert orders["order_id"].is_unique
# A resample rendezett időindexet vár.
orders = orders.sort_values("order_ts")
assert orders["order_ts"].is_monotonic_increasing

# UTC napok: fix 24 órás ablakok.
utc_daily = (
    orders.set_index("order_ts")
    .resample("D")
    .size()
    .rename("n_orders_utc")
)

# Budapest zóna szerinti naptári napok.
budapest_ts = orders["order_ts"].dt.tz_convert("Europe/Budapest")
budapest_daily = (
    orders.assign(local_ts=budapest_ts)
    .set_index("local_ts")
    .resample("D")
    .size()
    .rename("n_orders_budapest")
)

# Ellenőrzés: a zónakonverzió nem veszít el és nem duplikál sort.
assert utc_daily.sum() == len(orders)
assert budapest_daily.sum() == len(orders)

# Hiányzó napok dokumentálása: a resample teljes naptári tartományt ad,
# az üres napok 0 értékkel szerepelnek.
zero_utc = utc_daily[utc_daily == 0]
print("Üres UTC-napok:", zero_utc.index.to_list())
```

A két sorozat naptári napcímkéit akkor tudjuk összehasonlítani, ha a zónainformációt leválasztjuk a napcímkéről; a következő kód a helyi naptári napokat veti össze, így látszik, hány sor került át másik napra:

```
comparison = pd.DataFrame(
    {
        "utc": utc_daily.values,
        "budapest": budapest_daily.values,
    },
    index=utc_daily.index.tz_localize(None),
)
# Ha a helyi szélső napok átcsúsznak, a pozíció szerinti párosítás
# a naptári napcímkére vonatkozik, amit a fenti módon biztosítunk.
comparison["diff"] = comparison["budapest"] - comparison["utc"]
print(comparison.loc["2025-03-28":"2025-04-01"])
```

A naphatár-dokumentációhoz tartozik a DST-hatás pontos leírása. Budapest 2025. március 30-án 02:00 helyi időről 03:00-ra ugrik; az átmenet után az UTC offset +1 óráról +2 órára nő. Ezért a helyi március 30-i nap az UTC 2025. március 29. 23:00 és 2025. március 30. 22:00 közötti 23 órás ablak; a 22:00 UTC utáni rendelések már a helyi március 31-re esnek. A `comparison` tábla `diff` oszlopa éppen ezt az áthelyeződést mutatja.

Kézi kontrollként válasszunk egy UTC 23 óra utáni rendelést, és nézzük meg a budapesti dátumát:

```
late = orders[orders["order_ts"].dt.hour >= 23].sort_values("order_ts").iloc[0]
print(late[["order_id", "order_ts"]])
print(
    "Budapest szerint:",
    late["order_ts"].tz_convert("Europe/Budapest"),
)
```

A második sor napja a szemléltető várakozás szerint eggyel nagyobb, mint az UTC dátum; ez kézzel is ellenőrizhető egyetlen rekordon. Az invariáns itt nem a napok száma, hanem a sorösszeg: a zónakonverzió kizárólag a besorolást változtatja meg.

6.4.4.1 Determinisztikus DST-határesetek és nullhatásos kontroll

A DST-napi eltérés nem szükségszerű: csak akkor jelenik meg, ha van olyan esemény, amelynek UTC szerinti és helyi naptári napja eltér. Két mini adathalmaz mutatja a különbséget. Az elsőben („straddle”) három esemény van, amelyek közül a második helyi idő szerint már március 30-ra esik, UTC szerint még március 29-re:

esemény (UTC)	helyi idő (Europe/Budapest)	érték
2025-03-29 22:30	2025-03-29 23:30 (CET)	100
2025-03-29 23:30	2025-03-30 00:30 (CET)	200
2025-03-30 10:00	2025-03-30 12:00 (CEST)	600

A napi átlagok:

Nap	UTC szerint	Budapest szerint
2025-03-29	(100 + 200) / 2 = **150**	100
2025-03-30	600	(200 + 600) / 2 = **400**

A második („control”) adathalmazban minden esemény 10:00 és 14:00 UTC között van, tehát a helyi nap mindhárom eseménynél ugyanaz, mint az UTC nap. Ott a két aggregáció azonos: 2025-03-30 átlaga mindkét zónában 300. Ez a **nullhatásos kontroll**: azt mutatja meg, hogy az aggregációs kód önmagában nem változtat, a különbség a naphatár-áthelyeződésből fakad.

```
import pandas as pd

def daily_mean(timestamps, values, zone):
    frame = pd.DataFrame({"ts": pd.to_datetime(timestamps, utc=True),
                          "value": values})
    local = frame["ts"].dt.tz_convert(zone)
    return frame.groupby(local.dt.date)["value"].mean()

straddle = daily_mean(
    ["2025-03-29 22:30", "2025-03-29 23:30", "2025-03-30 10:00"],
```

```
    [100, 200, 600], "Europe/Budapest")
control = daily_mean(
    ["2025-03-30 10:00", "2025-03-30 12:00", "2025-03-30 14:00"],
    [100, 200, 600], "Europe/Budapest")
assert straddle.loc[pd.Timestamp("2025-03-29").date()] == 100
assert straddle.loc[pd.Timestamp("2025-03-30").date()] == 400
assert control.loc[pd.Timestamp("2025-03-30").date()] == 300
```

A DST-nap hossza is determinisztikusan számolható: a tavaszi átmenet napja 23 óra, az őszi visszaállásé 25 óra.

```
def day_hours(date, zone):
    start = pd.Timestamp(date).tz_localize(zone)
    end = start + pd.DateOffset(days=1)
    return (end - start).total_seconds() / 3600

assert day_hours("2025-03-30", "Europe/Budapest") == 23
assert day_hours("2025-10-26", "Europe/Budapest") == 25
```

Az eredeti laborfeladat állítását ezért feltételesen kell olvasni: **a 2025-03-30-i `basket_mean` eltérhet** az UTC szerinti naptól, de csak akkor, ha a naphatár közelében van olyan rendelés, amelynek a helyi és az UTC napja különbözik. A `tests/test_dst_aggregation.py` a straddle- és a kontroll-adathalmazt, valamint a 23/25 órás napokat ellenőrzi.

6.4.5 Eredmény értelmezése, hibák és trükkök

A munkapélda ellenőrző invariánsa: a betöltött `orders` hossza 2400, az `order_ts` dtype `datetime64[us, UTC]`, a két napi sorozat összege szintén 2400. A `comparison["diff"].abs().sum()` azt adja meg, hány rendelést sorolt át a zónakonverzió más naptári napra; ehhez nincs szükség statisztikai toleranciára, mert az időzóna-átszámítás egzakt. A DST-napon a budapesti napi kosár 23 órányi forgalmat tartalmaz, ezért a napi darabszám ott nem hasonlítható közvetlenül a 24 órás UTC-ablakkal; a `diff` éppen ezt dokumentálja.

Az egyik leggyakoribb típuscsapda az, ha a `to_datetime` hívás kimarad. A CSV-ből a pandas az `order_ts` oszlopot `object` típusú stringekként olvassa be; ezen a `.dt` accessor nem létezik, és `AttributeError`-t kapunk. A második csapda az, ha a `tz_convert` helyett a `tz_localize`-t használjuk egy már aware oszlopra: az `Already tz-aware, use tz_convert to convert.` hibaüzenet pontosan jelzi a félreértést.

A kategória típuscsapdája a `.cat.codes` numerikus értékként való használata. A `segment` kódjai `business=0`, `consumer=1` lehetnek, de ez nem jelenti azt, hogy a két szegmens között 1 egységnyi távolság van, vagy hogy a `business` megelőzi a `consumer`-t. A kategóriát a későbbi modellezésben dummy-változókká vagy rendezetlen szintekké alakítjuk; erről a 17.2. alfejezetben lesz szó.

6.4.6 Feladatok

6.4.6.1 1. feladat: értelmezés

Magyarázd el saját szavaiddal a különbséget a `tz_localize` és a `tz_convert` között, és írj két egysoros példát: az egyik egy naive `pd.Timestamp("2025-03-01 12:00")` értéken mutatja a localize-t, a másik a Northstar `order_ts` oszlopán mutatja a convertet. Mi történik, ha a két műveletet felcserélve hívjuk?

6.4.6.2 2. feladat: implementálás

Készíts napi aggregációt a budapesti zóna szerint, amely minden napra megadja a rendelések számát és a `basket_minor` átlagát:

```
basket_budapest = (
    orders.assign(local_ts=orders["order_ts"].dt.tz_convert("Europe/Budapest"))
    .set_index("local_ts")
    .resample("D")
    .agg(n_orders=("order_id", "size"), basket_mean=("basket_minor", "mean"))
)
```

Ellenőrizd, hogy a `basket_mean` a 2025-03-30-i napon eltér-e az UTC szerinti aggregáció azonos napi átlagától. Ha eltér, keresd meg a naphatár-áthelyeződést okozó rendelést; ha nem tér el, indokold, miért nem (nincs olyan rendelés, amelynek a helyi és az UTC napja különbözik). A fenti „straddle” és „control” mini adathalmaz mutatja, hogy az eltérés lehetőség, nem szükségszerűség. A feladathoz minden input a `lab/data/orders.csv` tiszta fájl.

6.4.6.3 3. feladat: hibadiagnózis

Egy tanuló a következő kódot futtatta közvetlenül a `pd.read_csv("lab/data/orders.csv")` után:

```
daily = orders.groupby(orders["order_ts"].dt.date).size()
```

A hibát kap: `AttributeError: Can only use .dt accessor with datetimelike values`. Mi a hiba oka, és mi a minimális javítás? Írd le azt a két sort, amit a csoportosítás elé kell beszúrni.

6.4.6.4 4. feladat: megváltozott feltétel

Az üzleti riport a következő változatban `America/New_York` zónát kér. Készítsd el a New York-i napi rendelésszám-sorozatot, majd dokumentáld a naphatárt: a zóna 2025. március 9-én 02:00 helyi időről 03:00-ra ugrik, ezért a naphatár UTC szerint előtte 05:00, utána 04:00. A kód írja ki a `new_york_daily` sorösszegét és a 2025-03-07 és 2025-03-11 közötti sorokat. Ellenőrizd az invariánst, és nevezd meg, miért nem azonos a napok száma a New York-i és a budapesti sorozatban, ha a szélső napok átcsúsznak.

6.4.7 Megoldások és indoklás

6.4.7.1 1. feladat megoldása

A `tz_localize` egy zóna nélküli (naive) **falióra-időt** értelmez egy adott időzónában, és ezzel rendel hozzá abszolút időpontot (UTC-pillanatot). A `tz_convert` ezzel szemben egy már zónatudatos, ismert pillanatot mutat meg más zónában; az abszolút pillanat nem változik, csak a helyi óra- és dátumcímke. Példa:

```
naive = pd.Timestamp("2025-03-01 12:00")
aware_utc = naive.tz_localize("UTC")
# Type-aware oszlopon:
orders["order_ts"].dt.tz_convert("Europe/Budapest")
```

A localize nem mindig egyértelmű: a nyári időszámítás átállásánál a nem létező falióra-idő (tavaszi ugrás) és az ismétlődő falióra-idő (őszi visszaállás) külön kezelendő. A pandas 2.2.3-ban ezek `NonExistentTimeError`, illetve `AmbiguousTimeError` kivételt adnak; a 3.1.0-ban `ValueError`-t, ugyanazzal a magyarázattal. A `nonexistent="shift_forward"` és az `ambiguous=True` (vagy `ambiguous="infer"`) explicit döntést kényszerít:

```
pd.Timestamp("2025-03-30 02:30").tz_localize("Europe/Budapest", nonexistent="shift_forward")
# 2025-03-30 03:00:00+02:00
pd.Timestamp("2025-10-26 02:30").tz_localize("Europe/Budapest", ambiguous=True)
# 2025-10-26 02:30:00+02:00
```

Ha egy naive értéken `tz_convert`-et hívunk, a pandas `TypeError`-t ad, mert nincs mit konvertálni; ha egy aware értéken `tz_localize`-t hívunk, a hibaüzenet szerint már van zónája.

6.4.7.2 2. feladat megoldása

A 2025-03-30-i budapesti nap 23 órás, ezért a `n_orders` ott legfeljebb az UTC 24 órás ablakának darabszáma lehet; a `basket_mean` **akkor** tér el, ha a naphatár-áthelyeződés miatt a két ablak más rendeléseket tartalmaz. Ha minden rendelés távol esik a naphatártól, a két átlag azonos lehet; az eltérés nem az átlagképlet hibája, hanem a napbesorolás következménye. A pontos értékek a helyi futtatásból jönnek; az invariant itt az, hogy a súlyozott átlaggal számított napi `basket_mean` értéke a kosárösszeg és a darabszám hányadosa, tehát `basket_mean = basket_sum / n_orders`. Ezt egy kiválasztott napra kézzel ellenőrizhetjük:

```
day = basket_budapest.loc["2025-03-30"]
assert abs(day["basket_mean"] * day["n_orders"] - basket_sum) < 0.5
```

Itt a `basket_sum` az adott napi `basket_minor` összeg, amit a `.agg`-be fel kell venni. Egész centekben dolgozunk, ezért a tolerancia 0,5 cent alatti kerekítési szint.

6.4.7.3 3. feladat megoldása

A `read_csv` az `order_ts` oszlopot stringként adja vissza, tehát a `.dt` accessor nem érhető el. A minimális javítás a csoportosítás előtt:

```
orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)
daily = orders.groupby(orders["order_ts"].dt.date).size()
```

A `utc=True` a zónatudatosságot is biztosítja, ezért a későbbi zónakonverzió nem hibázik. A `.dt.date` az UTC naptári dátumot adja; ha helyi nap kell, előbb `tz_convert`, utána `.dt.date`.

6.4.7.4 4. feladat megoldása

```
new_york_ts = orders["order_ts"].dt.tz_convert("America/New_York")
new_york_daily = (
    orders.assign(local_ts=new_york_ts)
    .set_index("local_ts")
    .resample("D")
    .size()
    .rename("n_orders_new_york")
)

assert new_york_daily.sum() == len(orders)
print(new_york_daily.loc["2025-03-07":"2025-03-11"])
```

A naphatár-dokumentáció: 2025-03-08-ig a New York-i nap UTC 05:00-kor kezdődik, 2025-03-09-től UTC 04:00-kor, mert a zóna EST-ről EDT-re vált. A `new_york_daily` sorösszege 2400; a napok száma akkor térhet el a budapesti sorozatétól, ha az adat első vagy utolsó rendelése a zónakonverzió miatt átcsúszik a szomszédos naptári napra. A Northstar tartománya napközbeni rendeléseket tartalmaz, ezért ez nem várható, de az állítást az adott futtatásban a `min()` és `max()` ellenőrzésével kell alátámasztani.

6.4.8 Gyakori tévedés és ellenpélda

A legfontosabb félreértés az időbélyeg és a helyi naptári dátum keverése. A `Timestamp.date()` egy `datetime.date` objektumot ad, amely **nem hordoz zónát**; ha az UTC-oszlopon hívjuk, az UTC szerinti naptári napot kapjuk, nem a helyi naptári napot. Vegyük ezt a rendelést: `2025-01-01 23:30:00+00:00`. UTC szerint január 1., de Budapesten már `2025-01-02 00:30`. Tehát:

```
row = pd.Timestamp("2025-01-01 23:30:00+00:00")
print(row.date())                                  # 2025-01-01
print(row.tz_convert("Europe/Budapest").date())    # 2025-01-02
```

A két eredmény különbözik, pedig ugyanarról a pillanatról van szó. A riportban ezért mindig rögzíteni kell, melyik zóna naptári napjáról beszélünk; a `date` önmagában nem elég. Ez a hiba ugyanaz a fajta kontextusvesztés, mint amit a 2.4. alfejezetben a környezeti eltéréseknél láttunk: az operációs rendszer órazóna-beállítása sem írd felül az adat saját zónaattribútumát.

Visszautalva az 5.4. alfejezetre: a `.dt` accessor és a `resample` a pandas vektorizált, tömb alapú működését használja; nem soronkénti Python-ciklussal dolgozunk, ezért a fenti műveletek 2400 soron is gyorsak maradnak.

6.4.9 Önellenőrzés

- Mi a különbség egy `Timestamp` és egy `Timedelta` között, és melyik az `order_ts`, melyik a `delivery_days`?
- Miért kell `set_index`, mielőtt `resample("D")`-t hívunk, és miért számít az index rendezettsége?
- Hogyan ellenőrzöd egyetlen invariánssal, hogy a zónakonverzió nem duplikált vagy vesztett el rendelést?
- Miért nem szabad a `segment.cat.codes` értékeit numerikus rendezéshez vagy távolsághoz használni?

Időzónák kezelésénél a `tz_localize` egy naiv időponthoz rendel időzónát, a `tz_convert` pedig már zónatudatos időbélyeget alakít át. Helyi nap szerinti aggregáció előtt előbb a megfelelő időzónára kell váltani: a nyári időszámítás miatt egy helyi naptári nap nem mindig 24 órás.

6.4.10 Rövid összefoglaló

A `order_ts` oszlopot `pd.to_datetime(..., utc=True)` hívással zónatudatos `datetime` típussá alakítottuk, a `segment` és `region` oszlopokat pedig explicit `category` típussal láttuk el. A `resample("D")` időindexet és rendezett sorrendet kíván; a zónakonverzió a naptári nap besorolását változtatja, a sorösszeget nem. A budapesti DST-nap 23 órás ablak, ezért az aznapi darabszám nem hasonlítható közvetlenül az UTC 24 órás napjához. A `tz_localize` és `tz_convert` felcserélése, valamint a zóna nélküli `date` használata a leggyakoribb hibák. A következő fejezetben az

így megtisztított, típusos táblát SQL-relációs műveletekhez használjuk, ahol a NULL-ok és a join-kardinalitás lesznek a fő kérdések.

7 SQL a feature- és kutatási adatok előállításához

7.1 Reláció, NULL és elemzési szemcsézettség

Belépő · szint: alap · idő: 45 perc · eszköz: SQL, DuckDB

Előfeltétel: 6.4. alfejezet · Célkészség: Reláció, NULL és elemzési szemcsézettség · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

7.1.1 Cél és konkrét helyzet

A Northstar laborban egy rendelés elemzése más eredményt ad rendelésenként, mint ügyfelenként. Ugyanaz a kérdés – hány érvényes megfigyelésünk van – kétféle szemcsézettségen értelmezhető: a megfigyelési egység lehet az `orders.csv` egy sora, tehát egy rendelés, vagy az `orders.csv` egyes sorai mögött álló egyedi ügyfél. Az előbbi a rendelések számát, az utóbbi az aktív ügyfelek – legalább egy érvényes rendeléssel rendelkező ügyfelek – számát adja.

Bemenet: `lab/data/orders.csv`, `lab/data/customers.csv`, valamint egy ötsoros kézi ellenőrző tábla. Cél: SQL-lel megszámolni az érvényes rendeléseket `COUNT(*)` módon, és az aktív ügyfeleket `COUNT(DISTINCT customer_id)` módon; a kettőt nem felcserélni. Előzetes becslés: a tisztaz `orders.csv` 2400 sora definíció szerint érvényes, az aktív ügyfelek száma legfeljebb a `customers.csv` 300 ügyfele; a pontos számot a futtatás adja. A kimenethez kézi ellenőrző minta is kell: egy kis, azonosítható sorhalmazon kézzel megszámolunk, és az SQL-eredményt ehhez hasonlítjuk.

7.1.2 A relációs tábla egy sorának jelentése

A relációs adatmodellben a tábla (relation) oszlopai attribútumok, sorai pedig azonos szerkezetű tények. A sor jelentését a megfigyelési egység (unit of observation) rögzíti: a Northstar `orders` táblájában egy sor egy rendelés, nem egy tétel és nem egy ügyfél. A `customer_id` itt csak egy attribútum, nem új szemcsézettség; ha egy ügyfél többször rendel, több sorban is megjelenik.

A sorok azonosítását kulcsok (keys) segítik. Az elsődleges kulcs (primary key) a táblán belül egyedi: `order_id`. Az idegen kulcs (foreign key) másik táblára hivatkozik: `customer_id`. A hiányos kulcs vagy hiányzó attribútum SQL-ben `NULL`: nem nulla, nem üres szöveg, hanem „nem ismert/nem alkalmazható”. A pandasbeli `pd.NA` relációs párja pont ez; az 6.1 sémajelentése ott, ez itt találkozik.

7.1.3 NULL és háromértékű logika

A NULL-os hasonlítás háromértékű logikát (three-valued logic) követ: az eredmény nem csak TRUE vagy FALSE, hanem UNKNOWN is lehet. A legfontosabb szabályok:

- NULL = NULL és NULL != NULL eredménye UNKNOWN, mert két ismeretlen egyenlőségét nem tudjuk eldönteni.
- IS NULL és IS NOT NULL nem hasonlítás, hanem közvetlen hiányvizsgálat; ezért ezeket használjuk.
- UNKNOWN AND FALSE = FALSE; UNKNOWN AND TRUE = UNKNOWN; UNKNOWN OR TRUE = TRUE; NOT UNKNOWN = UNKNOWN.

A WHERE csak azokat a sorokat tartja meg, ahol a feltétel TRUE. Emiatt egy látszólag ártatlan szűrés is csendben eldobhatja a NULL-os sorokat. Például distance_km >= 5 a NULL távolságra UNKNOWN, tehát a sor kiesik – néha pontosan ezt akarjuk, de akkor ezt mondjuk ki, ne mellékhatásként kapjuk.

7.1.3.1 NULL-tesztpolc: igazságtábla, COUNT, NOT IN és COALESCE

A háromértékű logika teljes igazságtáblája (a NULL itt UNKNOWN-ként viselkedik):

Kifejezés	Eredmény
NULL = NULL	UNKNOWN
NULL IS NULL	TRUE
NULL IS NOT NULL	FALSE
UNKNOWN AND TRUE	UNKNOWN
UNKNOWN AND FALSE	FALSE
UNKNOWN OR TRUE	TRUE
UNKNOWN OR FALSE	UNKNOWN
NOT UNKNOWN	UNKNOWN

Egy négy soros mini táblán (t(id, x): (1,10), (2,NULL), (3,20), (4,NULL)) a klasszikus különbségek:

Lekérdezés	Eredmény	Magyarázat
COUNT(*)	4	minden sor, a NULL-os is
COUNT(x)	2	csak a nem NULL értékek
COUNT(DISTINCT x)	2	egyedi nem NULL értékek
SUM(x)	30	a NULL-okat kihagyja
AVG(x)	15.0	30 / 2, nem 30 / 4
AVG(COALESCE(x, 0))	7.5	30 / 4, mert a NULL helyére 0 kerül
WHERE x NOT IN (10, NULL)	0 sor	minden összehasonlítás UNKNOWN
WHERE x IN (10, NULL)	1 sor	csak az x=10 TRUE
WHERE NOT EXISTS (...)	3 sor	a NULL-os sorok itt megmaradnak

A `NOT IN` NULL-lal a legveszélyesebb csapda: a lista NULL eleme miatt a predikátum soha nem lesz `TRUE`, ezért a lekérdezés csendben üres eredményt ad. A javítás vagy a NULL-ok kiszűrése a belső lekérdezésben (`WHERE v IS NOT NULL`), vagy `NOT EXISTS` használata, de a kettő szemantikailag nem azonos: a `NOT EXISTS` a NULL-os külső sorokat megtartja. A választást a kérdés dönti el, nem a véletlen.

A `COALESCE(x, 0)` nem „javítja” a NULL-tól való függetlenséget: az átlag nevezőjét is megváltoztatja. A `SUM(COALESCE(x,0))` ugyanazt adja, mint a `SUM(x)`, mert a 0 nem változtat az összegen; a `COUNT` és az `AVG` viszont igen. Ezért a NULL pótlása mindig explicit üzleti döntés, és a jelentésben fel kell tüntetni.

```
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, x INTEGER)")
con.executemany("INSERT INTO t VALUES (?, ?)", [(1, 10), (2, None), (3, 20), (4, None)])
print(con.execute("SELECT COUNT(*), COUNT(x), AVG(x), AVG(COALESCE(x,0)) FROM t").fetchone())
print(con.execute("SELECT COUNT(*) FROM t WHERE x NOT IN (10, NULL)").fetchone())
print(con.execute("SELECT NULL = NULL, NULL IS NULL, NULL AND 0, NULL OR 1").fetchone())
```

Engine-specifikus megjegyzés:

Kérdés	SQLite	PostgreSQL	MySQL
`NULL = NULL`	`UNKNOWN`	`UNKNOWN`	`UNKNOWN`
null-biztos egyenlőség	`IS`	`IS DISTINCT FROM`	`<=>`
`COUNT(DISTINCT x)` kihagyja a NULL-t	igen	igen	igen
egyedi kényszer több NULL-t enged	igen	igen (alapból)	igen
`GROUP BY` a NULL-t egy csoportnak veszi	igen	igen	igen

A `tests/test_sql_null.py` a fenti mini táblát SQLite-ban építi fel, és a kézi igazságtáblához, a COUNT/AVG értékekhez, a `NOT IN` nulla sorához és a `NOT EXISTS` három sorához hasonlítja az eredményt.

7.1.4 SELECT, WHERE, GROUP BY, HAVING és COUNT

A `SELECT` nem sorokat gyárt, hanem egy halmaz kívánt alakját írja le. A logikai végrehajtási sorrend:

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`

A `WHERE` csoportosítás előtt, soronként szűr; a `HAVING` csoportosítás után, csoportokra vonatkozó feltétellel. A `GROUP BY` új szemcsézettséget hoz létre: az eredmény egy sora mostantól egy csoportot jelent, és a nem csoportosító oszlopokra aggregátumot kell használni. A `COUNT` három alakja:

- `COUNT(*)`: a sorok számát adja, a `NULL`-os sorokat is beleszámítja.
- `COUNT(column)`: a megadott oszlop nem `NULL` értékeinek számát adja.
- `COUNT(DISTINCT column)`: a megadott oszlop egyedi, nem `NULL` értékeit számolja.

Az SQLite, a DuckDB és a PostgreSQL ebben a viselkedésben megegyezik. A dialektusok közötti eltérés nem ide tartozik, elsősorban a CSV-betöltésben és a típusszigorban jelentkezik: a DuckDB `read_csv` közvetlenül olvas CSV-t, a PostgreSQL `COPY`-t vagy FDW-t használ, az SQLite pedig lazább típuskezelésű.

7.1.5 Matematikai híd: súlyozott és súlyozatlan átlag

Ugyanaz az átlagszámítás más eredményt adhat, ha más szemcsézettségen végezzük. Legyen K csoport, ahol a k-adik csoportban n_k megfigyelés van, a csoportátlag pedig $\bar{x}_k$. A csoportátlagok súlyozatlan átlaga:

$$\bar{x}_{\text{unweighted}} = \frac{1}{K} \sum_{k=1}^{K} \bar{x}_k.$$

A teljes, tételszintű átlag viszont a csoportméretekkel súlyozott átlag:

$$\bar{x}_{\text{weighted}} = \frac{\sum_{k=1}^{K} n_k \bar{x}_k}{\sum_{k=1}^{K} n_k},$$

ahol n_k a csoporton belüli megfigyelések darabszáma, $\bar{x}_k$ a csoportátlag, K a csoportok száma. A két képlet csak akkor egyezik, ha minden n_k egyenlő. Ez az „átlagok átlaga” csapda az SQL `GROUP BY` és a külső `AVG` egymásra építésekor.

Konkrét számpélda: az első kosárban három tétel ára egyaránt 100 EUR-cent, a másodikban egy tételé 500 EUR-cent. A kosarankénti átlagárak: 100 és 500, ezek súlyozatlan átlaga $(100 + 500)/2 = 300$. A tételszintű súlyozott átlag viszont $(3 \cdot 100 + 1 \cdot 500)/4 = 200$. Ugyanaz az adat, de a 300 háromszor képviseli a 100-at, ezért húzza lefelé a tételszintű átlagot. Aki a kosarankénti átlagokat egyenlő súllyal átlagolja, 300-at kap; aki a tételeket nézi, 200-at.

7.1.6 Teljes minimális munkapélda

Az alábbi SQLite munkamenet egy ötsoros `orders_demo` táblát hoz létre, majd kiszámolja az érvényes rendelések és az aktív ügyfelek számát. Platform: Ubuntu 24.04 bash, munkakönyvtár: a labor gyökere. A kimenetet nem gépi futásból másoljuk; a bemeneti sorokból közvetlenül levezethető, elméletileg várt érték:

```
# Platform: Linux/macOS terminál, munkakönyvtár: a labor gyökere
sqlite3 s025_demo.db <<'SQL'
CREATE TABLE orders_demo (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    basket_minor INTEGER NOT NULL,
    distance_km REAL,
    region TEXT NOT NULL
);

INSERT INTO orders_demo VALUES
    (1, 10, 3200, 12.0, 'north'),
    (2, 20,    0, 25.5, 'south'),
    (3, 30, 1800, NULL, 'central'),
    (4, 10, 9900,  4.0, 'north'),
    (5, 40, -100,  8.5, 'south');

SELECT order_id, customer_id, basket_minor, distance_km
FROM orders_demo
ORDER BY order_id;

SELECT COUNT(*)                             AS valid_orders,
       COUNT(DISTINCT customer_id)          AS active_customers
FROM orders_demo
```

```
WHERE distance_km IS NOT NULL
  AND basket_minor >= 0;
SQL
```

Az eredménytáblázat a következő: az első lekérdezés öt sort ad, a második egyet. Az érvényes rendelés definíciója itt: `distance_km IS NOT NULL AND basket_minor >= 0`. A 3. sor a `NULL` távolság miatt, az 5. sor a negatív kosárérték miatt esik ki. Marad három érvényes rendelés: a 10-es ügyfél kétszer, a 20-as egyszer szerepel, ezért az aktív ügyfelek száma 2, nem 3. A `COUNT(customer_id)` ezen a három érvényes soron 3-at adna, mert a duplikált 10-et kétszer számolná; a `DISTINCT` teszi ügyfélszemcsézettséggé az eredményt.

Ugyanez a Northstar tisztaz `orders.csv`-jén DuckDB-vel:

```
# Platform: Linux/macOS terminál, munkakönyvtár: lab/
duckdb :memory: "SELECT COUNT(*) AS valid_orders, COUNT(DISTINCT customer_id) AS
↪  active_customers FROM read_csv('orders.csv', header = true);"
```

PowerShellben a parancs formailag hasonló, de a külső idézőjelet a shell szerint kell kezelni. Ez a második lekérdezés a tiszta fájlra elméletileg 2400 érvényes rendelést ad, mert a `lab/ADATOK.md` szerint a tisztaz `orders.csv` 2400 sora már érvényes. Az aktív ügyfelek számát a futtatás adja; a tiszta `customers.csv` 300 ügyfele felső korlátot jelent.

7.1.7 Eredmény értelmezése és kézi ellenőrzés

A `s025_demo.db` demónál a fenti eredmény kézzel is ellenőrizhető: az 1., 2. és 4. sor érvényes, tehát három rendelés; ezek közül a `customer_id` értékek 10, 20 és ismét 10, tehát két egyedi ügyfél. A `COUNT(*)` alakzat a sorok számát, a `COUNT(DISTINCT customer_id)` a szemcsézettség-váltást végzi el. A két szám nem felcserélhető: az első rendelésszám, a második ügyfélszám.

A nagy fájlon a kézi ellenőrző minta elkészítéséhez érdemes az első hat `order_id` szerinti sort kiválasztani, majd az ott látható `customer_id` értékeket kézzel egyedíteni:

```
WITH sample AS (
    SELECT * FROM read_csv('orders.csv', header = true)
    ORDER BY order_id
    LIMIT 6
)
SELECT COUNT(*)                              AS sample_rows,
       COUNT(DISTINCT customer_id)           AS sample_customers
FROM sample;
```

A minta sorainak száma mindig 6; a mintán belüli egyedi ügyfelek számát a kapott hat sor `customer_id` oszlopából számoljuk ki, és ezt vetjük össze az SQL-eredménnyel. Mivel az `orders.csv` rendezett, a `LIMIT 6` determinisztikus mintát ad.

7.1.8 Feladatok

1. **Értelmezés.** Egy kolléga szerint az aktív ügyfelek számát a `SELECT COUNT(customer_id) FROM orders;` adja. Miért téved? Mi lenne a helyes lekérdezés, és miben különbözik az eredmény jelentése?
2. **Implementálás.** Futtasd a fenti DuckDB-lekérdezést a labor `orders.csv` fájlján, és készítsd el a hat soros kézi ellenőrző mintát is. Írd fel a kapott két számot a minta szintjén: `COUNT(*)` és `COUNT(DISTINCT customer_id)`; ellenőrizd, hogy a minta kézi listázásával egyeznek-e.

3. **Hibadiagnózis.** A `s025_demo.db` táblán valaki ezt futtatja: `SELECT * FROM orders_demo WHERE distance_km != NULL;` és üres eredményt kap. Miért? Add meg a javított lekérdezést, és indokold a háromértékű logika segítségével.

4. **Transzfer, matematikai híd.** Készítsd el a szövegben szereplő kéts kosaras `order_items_demo` táblát három 100-as és egy 500-as egységárral. Számold ki SQL-lel a kosarankénti átlagárak súlyozatlan átlagát, majd a tételszintű `AVG`-et. Az elvárt számok 300 és 200; mutasd meg a kézi számítást is.

5. **Megváltozott adat.** Vedd fel a `orders_demo` táblába a következő sort: `(6, 50, 0, NULL, 'west')`. Mi lesz ezután a `COUNT(*)`, a `COUNT(distance_km)` és az érvényes rendelések száma? Mi marad az aktív ügyfelek száma?

7.1.9 Megoldások és indoklás

1. A `COUNT(customer_id)` a nem `NULL` `customer_id` értékek számát adja, tehát rendeléssorokat számol, nem egyedi ügyfeleket. Ha a 10-es ügyfél kétszer rendelt, a `COUNT(customer_id)` mindkét sort beleszámolja. A helyes ügyfélszemcsézettségű lekérdezés `COUNT(DISTINCT customer_id)`, amely a 10-es ügyfelet egyszer számolja. A két eredmény akkor egyezik, ha minden ügyfél pontosan egyszer rendelt; ez itt nem áll fenn.

2. A teljes fájlra a tiszta adatszerződés szerint az érvényes rendelések száma 2400. Az aktív ügyfelek száma a futtatásból derül ki, és legfeljebb 300. A hat soros minta esetén a `sample_rows` mindig 6; a `sample_customers` megegyezik a hat sor kézi listájának egyedi `customer_id` számával. Ha a két módszer eltér, akkor a minta kiválasztásában vagy a kézi egyedítésben van hiba. Alternatív ellenőrzés: a `customers.csv` 300 sorához képest az aktív ügyfelek száma nem lehet nagyobb.

3. A `distance_km != NULL` soha nem `TRUE`, mert a `NULL`-os hasonlítás eredménye `UNKNOWN`. A `WHERE` csak `TRUE` feltételt fogad el, ezért minden sor kiesik, még azok is, ahol a távolság valós szám. A javítás: `SELECT * FROM orders_demo WHERE distance_km IS NOT NULL;`. Ez a háromértékű logika miatt nem ekvivalens a `NULL`-os egyenlőségvizsgálattal.

4. Az `order_items_demo` tábla:

```
CREATE TABLE order_items_demo (
    order_id INTEGER NOT NULL,
    line_id INTEGER NOT NULL,
    unit_price_minor INTEGER NOT NULL
);

INSERT INTO order_items_demo VALUES
    (1, 1, 100),
    (1, 2, 100),
    (1, 3, 100),
    (2, 1, 500);
```

A súlyozatlan kosárátlag:

```
WITH order_mean AS (
    SELECT order_id, AVG(unit_price_minor) AS avg_price
    FROM order_items_demo
    GROUP BY order_id
)
SELECT AVG(avg_price) AS unweighted_mean
FROM order_mean;
```

A tételszintű súlyozott átlag:

```
SELECT AVG(unit_price_minor) AS weighted_mean
FROM order_items_demo;
```

Az első eredmény elméletileg 300.0, mert a két kosárátlag `(100+500)/2`. A második 200.0, mert `(100+100+100+500)/4`. Az ellenőrző invariant: a két képlet csak akkor egyenlő, ha a kosarak tételszáma megegyezik; itt nem egyezik, tehát a különbség várható.

5. A beszúrás után a táblának hat sora van, ezért `COUNT(*)` = 6. A `distance_km` oszlopban három nem `NULL` érték van: 12.0, 25.5, 4.0 és 8.5 – ez négy, tehát `COUNT(distance_km)` = 4. Az érvényes rendelések száma továbbra is 3, mert a hatodik sor `NULL` távolságú, az ötödik sor negatív kosárértékű, a harmadik pedig `NULL` távolságú. Az aktív ügyfelek száma is 2 marad: a 10-es és a 20-as ügyfél.

7.1.10 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés a `COUNT(*)` és a `COUNT(column)` felcserélése. Az ötsoros demón a `COUNT(*)` 5, a `COUNT(distance_km)` viszont 4, mert a 3. sor távolsága `NULL`. Ha valaki a hiányzó távolságú sorokat úgy akarja kizárni, hogy `COUNT(distance_km)`-et használ, akkor a rendelésszám csendben 4-re csökken, pedig a `WHERE distance_km IS NOT NULL` nélkül a többi oszlop adatai megvannak. Hasonló, második tévedés az átlagok átlaga: a kosarankénti 100 és 500 átlaga 300, a tételszintű átlag 200. A 300 csak akkor lenne helyes, ha mindkét kosár ugyanannyi tételt tartalmazna.

Ez az alfejezet visszautal 6.1 hiányzóérték-kezelésére: a pandas `pd.NA` és az SQL `NULL` ugyanazt a hiányt jelenti, de más szintaxissal vizsgáljuk. A 6.4 zónatudatos időbélyegei sem változtatnak a sor jelentésén: az `order_ts` továbbra is egy rendelés attribútuma, nem új szemcsézettség. A 3.1 környezetkezelése pedig arra figyelmeztet, hogy a fenti parancsok a ténylegesen elérhető CLI-t használják: a duckdb/sqlite3 parancsok a környezet részét képzik, nem a notebook részei.

Önellenőrzés: egy 10 soros táblán, ahol egy oszlopban 3 `NULL` van, a `COUNT(*)` 10, a `COUNT(column)` 7; miért? Mert a `COUNT(*)` a sorokat, a `COUNT(column)` a nem hiányzó értékeket számolja. Ha egy feltétel `NULL`-ra `UNKNOWN`, bekerül-e a sor a `WHERE` eredményébe? Nem, mert csak a `TRUE` feltételű sorok maradnak. Mikor használunk `HAVING`-et? Akkor, ha a csoportosítás utáni aggregált értékre akarunk feltételt, például `HAVING COUNT(*) > 1` azokat az ügyfeleket adja, akik legalább két érvényes rendeléssel bírnak.

Az aggregátumok nevezőjére is figyelni kell: a `COUNT(*)` az összes sort, a `COUNT(oszlop)` csak a nem `NULL` értékeket számolja; az `AVG` szintén kihagyja a `NULL`-okat. Ezért az átlag nevezője eltérhet a tábla sorszámától.

7.1.11 Rövid összefoglaló

A relációs tábla sorának jelentését a megfigyelési egység rögzíti; a Northstar `orders` táblában egy sor egy rendelés, ezért `COUNT(*)` rendelésszámot ad, `COUNT(DISTINCT customer_id)` pedig ügyfélszámot. A `NULL` háromértékű logikát hoz: a hasonlítás `UNKNOWN`, ezért a `WHERE` kiejtheti a hiányos sorokat; a `COUNT(column)` nem számolja a `NULL`-okat. A `GROUP BY` új szemcsézettséget hoz létre, a `HAVING` a csoportokra szűr. A súlyozott és súlyozatlan átlag különbsége az „átlagok átlaga” csapda: a kosarankénti 100 és 500 átlaga 300, a tételszintű átlag 200. A következő alfejezetben a join-kardinalitást vizsgáljuk, ahol egy one-to-many kapcsolat a sorokat és a célváltozót is többszörözheti.

7.2 Join-kardinalitás és duplázódó célváltozó

Belépő · szint: alap · idő: 45 perc · eszköz: SQL, DuckDB

Előfeltétel: 7.1. alfejezet · Célkészség: Join-kardinalitás és duplázódó célváltozó · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

7.2.1 Cél és konkrét helyzet

A Northstar laborban az `orders` tábla **rendelésenként egy sort**, az `order_items` tábla pedig **rendelési tételenként egy sort** tartalmaz. Egy rendeléshez több tétel tartozhat. Az üzleti kérdés: „A rendelések mekkora hányada késik a hét napos ígéretet meghaladóan?” A csatlakoztatás után a sor már nem feltétlenül rendelést jelent; a késési címke az adott rendelés valamennyi tételsorán ismétlődik. A cél, hogy a kulcsokat, a kapcsolat kardinalitását és a számítás nevezőjét a lekérdezés előtt ellenőrizzük.

Tanulási eredmények. Az olvasó képes (1) egy-a-többhöz és egy-az-egyhez kapcsolatot ellenőrizni; (2) join előtti és utáni sorszámot értelmezni; (3) hibás tételszintű átlagot felismerni; (4) helyes rendelésenkénti aggregációt építeni; (5) az adatszerződés sérülésekor tudatosan hibával megállni.

7.2.2 Matematikai híd: mit súlyoz a csatlakoztatás?

Legyen az i-edik rendelés késési jelzője $y_i \in \{0, 1\}$, tételsorainak száma $m_i \geq 1$. A rendelésalapú késési arány

$$\hat{p}_{\text{order}} = \frac{1}{n} \sum_{i=1}^{n} y_i.$$

Ha rendelés- és tételtáblát összekapcsolunk, majd az így létrejött sorokon átlagolunk, akkor ehelyett

$$\hat{p}_{\text{item}} = \frac{\sum_{i=1}^{n} m_i y_i}{\sum_{i=1}^{n} m_i}$$

keletkezik. Ez **tételszámmal súlyozott rendelési átlag**. Csak külön feltételek mellett egyezik meg a keresett rendelésalapú aránnyal. `LEFT JOIN` esetén a nulla tételes rendelések egy NULL-lal kiegészített sort adhatnak; ezt a külön esetet nem szabad észrevétlenül ugyanebbe a képletbe beleérteni.

7.2.3 Kézzel ellenőrizhető SQL-példa

A példa SQLite-ban végrehajtható, és nem igényel külső adatfájlt:

```sql
CREATE TABLE orders_demo (
    order_id TEXT PRIMARY KEY,
    delivery_days INTEGER NOT NULL,
    promised_days INTEGER NOT NULL
);
CREATE TABLE items_demo (
    item_id TEXT PRIMARY KEY,
    order_id TEXT NOT NULL REFERENCES orders_demo(order_id),
    line_total_minor INTEGER NOT NULL
);
INSERT INTO orders_demo VALUES ('A', 9, 7), ('B', 5, 7);
INSERT INTO items_demo VALUES
    ('A1', 'A', 100), ('A2', 'A', 200), ('A3', 'A', 300),
    ('B1', 'B', 400);

-- HELYTELEN a rendelések arányához: minden tételsort egyszer számol.
SELECT AVG(CASE WHEN o.delivery_days > o.promised_days
                THEN 1.0 ELSE 0.0 END) AS weighted_rate
FROM orders_demo AS o
JOIN items_demo AS i ON i.order_id = o.order_id;

-- HELYES: a címke rendelésenként egyetlen alkalommal szerepel.
SELECT AVG(CASE WHEN delivery_days > promised_days
                THEN 1.0 ELSE 0.0 END) AS order_rate
FROM orders_demo;

-- Egy másik HELYES eljárás: tételeket előbb rendelésenként aggregálunk.
WITH item_totals AS (
    SELECT order_id, COUNT(*) AS item_rows,
           SUM(line_total_minor) AS item_total_minor
    FROM items_demo GROUP BY order_id
)
SELECT o.order_id, o.delivery_days > o.promised_days AS target_late,
       t.item_rows, t.item_total_minor
FROM orders_demo AS o
LEFT JOIN item_totals AS t ON t.order_id = o.order_id
ORDER BY o.order_id;
```

Kézi kontroll: a hibás érték $3/4 = 0{,}75$, a helyes rendelésalapú késési arány $1/2 = 0{,}50$. Az `item_totals` eredménye A rendelésre 3 tétel és 600 cent, B-re 1 tétel és 400 cent. A `LEFT JOIN` után két sor marad.

7.2.4 Ellenőrzés Pythonból

```python
import sqlite3

with sqlite3.connect(":memory:") as conn:
    conn.executescript("""
    CREATE TABLE orders_demo(order_id TEXT PRIMARY KEY,
                             delivery_days INTEGER, promised_days INTEGER);
    CREATE TABLE items_demo(item_id TEXT PRIMARY KEY, order_id TEXT,
                            line_total_minor INTEGER);
    INSERT INTO orders_demo VALUES ('A',9,7),('B',5,7);
    INSERT INTO items_demo VALUES ('A1','A',100),('A2','A',200),
                                  ('A3','A',300),('B1','B',400);
    """)
    wrong = conn.execute("""SELECT AVG(CASE WHEN o.delivery_days > o.promised_days
                                       THEN 1.0 ELSE 0.0 END)
       FROM orders_demo o JOIN items_demo i ON o.order_id=i.order_id""").fetchone()[0]
    right = conn.execute("""SELECT AVG(CASE WHEN delivery_days > promised_days
                                       THEN 1.0 ELSE 0.0 END)
```

```
       FROM orders_demo""").fetchone()[0]
    assert abs(wrong - 0.75) < 1e-12
    assert abs(right - 0.50) < 1e-12
    assert conn.execute("SELECT COUNT(*) FROM orders_demo").fetchone()[0] == 2
    assert conn.execute("SELECT COUNT(*) FROM items_demo").fetchone()[0] == 4
```

A kódblokk szintaktikája és az ellenőrző értékek külső fájlt nem igényelnek. A teljes Northstar-adatkészlet sorszámait azonban kizárólag a mellékelt laboradatokból szabad igazolni.

7.2.5 A csatlakoztatás előtti adatszerződés

Az alábbi ellenőrzéseket célszerű beépíteni a feldolgozó folyamatba:

1. `orders.order_id` egyedi és nem hiányzó; `order_items.item_id` szintén egyedi.
2. Minden `order_items.order_id` létező rendelésre mutat (árva tétel nincs), vagy az eltérés dokumentáltan karanténba kerül.
3. Az `orders` és `item_totals` közötti összekapcsolás után a rendelések száma és az `order_id` egyedisége változatlan.
4. A `shipping_fee_minor` rendelésenkénti attribútumot **nem összegezzük közvetlenül a nyers tételsorokon**; máskülönben a díj a tételszámmal sokszorozódik.
5. A belső join hiányzó tételsor esetén rendelést is eldobhat. A `LEFT JOIN` megőrzi a rendelést, de a hiányzó tételösszegre vonatkozó üzleti szabályt külön kell meghatározni: a `NULL` nem minden esetben jogosan alakítható nullává.

7.2.5.1 Gépileg ellenőrzött join-kardinalitás

A join előtt négy esetet kell megkülönböztetni, és mindegyikhez tartozik egy automatikus ellenőrzés:

Eset	Példa	Ellenőrzés	Teendő
1:1	`orders.order_id` ↔ `customers.customer_id`	mindkét kulcs egyedi és nem NULL	`validate="one_to_one"`
1:N	`orders.order_id` → `order_items.order_id`	bal kulcs egyedi, jobb kulcs ismétlődhet	sok oldalt előbb aggregálni
N:M	két több-soros tábla összekapcsolása	mindkét kulcs ismétlődik	nem rendelésszintű eredmény; új szemcsézettség kell
hiányzó kulcs	`order_items.order_id` nem létezik `orders`-ben	`missing_right > 0`	karantén vagy dokumentált `LEFT JOIN`

A `lab/join_guard.py` ezt a négy esetet egy helyen kezeli:

```
import pandas as pd

class JoinGuardError(ValueError):
    pass

def cardinality(left, right, left_key, right_key):
    left_unique = not left[left_key].duplicated().any()
    right_unique = not right[right_key].duplicated().any()
```

```
    relationship = ("1:1" if left_unique and right_unique else
                    "1:N" if left_unique else
                    "N:1" if right_unique else "N:M")
    return {
        "relationship": relationship,
        "missing_right": int((~right[right_key].isin(left[left_key])).sum()),
        "missing_left": int((~left[left_key].isin(right[right_key])).sum()),
        "null_left": int(left[left_key].isna().sum()),
        "null_right": int(right[right_key].isna().sum()),
    }

def join_aggregated(orders, items, key="order_id", expected_rows=None):
    aggregated = items.groupby(key, as_index=False).agg(
        line_total_minor=("line_total_minor", "sum"),
        line_count=("line_total_minor", "size"))
    if aggregated[key].duplicated().any():
        raise JoinGuardError("az aggregálás után sem egyedi a kulcs")
    result = orders.merge(aggregated, on=key, how="left", validate="one_to_one")
    if result[key].duplicated().any():
        raise JoinGuardError("a join után duplikált kulcs")
    if expected_rows is not None and len(result) != expected_rows:
        raise JoinGuardError("a rendelésszám megváltozott")
    return result
```

A **join előtti és utáni sorszám** önmagában nem elég: a nyers join sorainak száma a tételszámmal nő, de a helyes aggregált join után a rendelések száma változatlan. A két invariáns együtt:

Lépés	Sorok	`order_id` egyedi?	Megjegyzés
`orders`	2	igen	O1, O2
`order_items`	3	nem (O1 kétszer)	1:N
nyers `orders` `items`	3	nem	a célváltozó duplázódik
`items` aggregálva	2	igen	O1: 300, O2: 150
aggregált join	2	igen	rendelésszint megmarad

A `raw_join_leakage()` a nyers join sorainak számát hasonlítja a rendelésszámhoz; a `join_aggregated()` pedig a sok oldalt előbb aggregálja, majd `validate="one_to_one"` mellett csatlakoztatja. A `tests/test_join_guard.py` az 1:1, 1:N, N:M, hiányzó kulcs, a nyers join szivárgását és az aggregált join rendelésszintűségét ellenőrzi.

7.2.6 Feladatok

1. Értelmezés. Három rendelés címkéje 1, 0 és 0; rendre 5, 1 és 2 tételük van. Mennyi a rendelés- és mennyi a tételszinten számolt késési arány?

2. Implementálás. Írj olyan SQL-lekérdezést, amely a `orders` és `order_items` táblákból rendelésenként adja vissza az `item_rows` és `item_total_minor` értékeket, és megtartja a nulla tételes rendeléseket is.

3. Hibakeresés. Mi történik, ha a `customers` táblában ugyanaz a `customer_id` kétszer szerepel, majd `orders`-szel összekapcsoljuk? Mit kell ellenőrizni a join előtt?

4. Transzfer. A tételszámok 5, 1, 2, a késési címkék 1, 0, 0. Mi változik, ha a legnagyobb tételszámú rendelés még tíz tételt kap? Miért nem változhat a helyes üzleti mutató?

7.2.7 Megoldások és indoklás

1. Rendelésszinten $1/3 \approx 0{,}333$; tételszinten $5/(5+1+2) = 5/8 = 0{,}625$. A különbség a tételszámból eredő implicit súlyozás.
2. `WITH t AS (SELECT order_id, COUNT(*) AS item_rows, SUM(line_total_minor) AS item_total_minor FROM order_items GROUP BY order_id) SELECT o.order_id, t.item_rows, t.item_total_minor FROM orders o LEFT JOIN t ON t.order_id=o.order_id;` A hiányzó összegek üzleti jelentését ezután kell rendezni.
3. A rendelések megduplázódhatnak, mert a feltételezett egy-a-többhöz kapcsolat megsérül. A `customers.customer_id` egyediségét és hiánytalanságát ellenőrizni kell, és a nem egyedi törzset először javítani vagy elutasítani.
4. A hibás tételszintű arány $15/18 \approx 0{,}833$ lesz, miközben a rendelésalapú arány változatlanul 1/3. Egyetlen rendelés több tétele nem több megfigyelési egység.

7.2.8 Gyakori tévedés és összefoglalás

A `JOIN` szintaktikailag hibátlan lehet akkor is, ha statisztikailag rossz nevezőt hoz létre. **A mérőszám megfigyelési egységét előbb rögzítjük, és csak utána választjuk meg a csatlakoztatást.** A következő alfejezetben az időben helyes korábbi rendeléseket `CTE`-vel és ablakfüggvényekkel állítjuk elő.

7.2.9 Kiadás előtti szakmai ellenőrzés

Join-invariáns. Amikor rendelésenként egy kimenetet kérünk, a csatlakoztatás után ellenőrizni kell a `order_id` egyediségét, vagy a tételszintű adatot előbb rendelésenként aggregálni. A sorok számának változása önmagában nem feltétlen hiba, de a mérendő egység és a súlyozás megváltozása dokumentálást igényel.

7.3 CTE, window és előzményből számolt feature

Belépő · szint: alap · idő: 45 perc · eszköz: SQL, DuckDB

Előfeltétel: 7.2. alfejezet · Célkészség: CTE, window és előzményből számolt feature · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

7.3.1 Cél és konkrét helyzet

A Northstar prediktív projektben egy rendelés beérkezésekor szeretnénk jellemezni az ügyfél múltbéli aktivitását. A `prior_order_count` jellemző azt mondja meg, hogy az adott ügyfélnek hány **korábbi** rendelése volt. A kimenetet rendelésenként egy sorban képzeljük el:

- bemenet: az `orders` tábla, ahol egy sor egy rendelés; `order_ts` UTC ISO-időbélyeg;
- megfigyelési egység: egy rendelés;
- partíció: egy ügyfél, azaz `customer_id`;
- cél: `prior_order_count`, `prev_order_ts` és `days_since_prev` oszlopok, amelyek sem az aktuális, sem a későbbi rendelést nem tartalmazzák.

A `days_since_prev` itt **eltelt nap** (a JULIANDAY-különbség egészre kerekítve), nem naptári naphatár; a két definíció a platformösszevetésben külön szerepel, és éles adaton nem ugyanazt az értéket adja.

Kézi előzetes becslés a fejezetben használt mini adatra: a `C1` ügyfél három rendelésénél a korábbi rendelésszám rendre `0, 1, 2`, az előző rendelés óta eltelt idő pedig a második és harmadik sornál `20 - 10 = 10`, illetve `30 - 20 = 10` nap.

7.3.2 Sorozat, rendezés és prefixösszeg

Az ügyfél rendeléseit tekintsük időben rendezett sorozatnak:

$$t_1 < t_2 < \cdots < t_n,$$

ahol t_i az i-edik rendelés időpontja, n az ügyfél rendeléseinek száma. A korábbi rendelésszám az aktuális sor előtti elemek száma:

$$p_i = \sum_{j=1}^{i-1} 1 = i - 1.$$

Ez egy prefixösszeg: minden lépésben 1-et adunk az előző összeghez, de az aktuális elemet már nem számítjuk bele. A p_i mértékegysége darab rendelés, értéke nemnegatív egész, az első rendelésnél mindig 0.

A szomszédos elemek különbsége

$$\Delta_i = t_i - t_{i-1}, \quad i > 1,$$

napokban az előző rendelés óta eltelt idő; Δ_1 nem értelmezhető, ezért hiányzó (`NULL`). A formális jelölés kimeneti alakja: N bemeneti rendeléssorból N kimeneti sor keletkezik, minden sorhoz három új numerikus vagy idő típusú érték tartozik.

7.3.3 `WITH`, `PARTITION BY`, `ROW_NUMBER`, `LAG` és a frame

Az SQL-ben a **common table expression** (CTE, `WITH` után megadott ideiglenes névvel ellátott lekérdezés) akkor hasznos, amikor egy részeredményt több lépésben, olvashatóan akarunk felépíteni. A `WITH placed AS (...)` például először rendezi és sorszámozza a rendeléseket, majd a következő CTE ezt használja további ablak típusú számításokhoz.

Az **ablakfüggvény** (window function) az eredményhalmaz minden sorához egy adott tartomány, az *ablakkeret* (window frame) alapján számol értéket. A `OVER` kulcsszó után megadott

```
PARTITION BY customer_id
ORDER BY order_ts, order_id
```

azt mondja meg, hogy az ablak egy-egy ügyfélre korlátozódik, és az ügyfélen belül az időpont, majd a rendelésazonosító szerint rendezünk. Egy rendeléshez több sor is tartozhat azonos időponttal; a `order_id` a **tiebreaker**, tehát a rendezést determinisztikussá teszi.

A `ROW_NUMBER()` az így rendezett partícióban az aktuális sor sorszáma:

```
ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY order_ts, order_id
)
```

Az első rendeléshez `1`, a másodikhoz `2` tartozik. Ebből `ROW_NUMBER() - 1` elvileg a korábbi rendelésszámot adja, de az explicit kerettel megadott `COUNT` erősebb kontraktust ad, és elkerüli, hogy valaki elfelejtse a `-1`-et.

Az ablaktartományt a frame szabályozza:

```
ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
```

Ez partíción belül az aktuális sort **megelőző** összes sort jelenti, az aktuális sort nem. A `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` a jelen sorral bezárólag számolna; ez a gyakori, ám itt hibás alapértelmezett viselkedés.

A `LAG(order_ts, 1)` a rendezett partícióban az eggyel korábbi sor `order_ts` értékét adja. Az első sornál nincs előző elem, ezért az eredmény `NULL`. A frame nem helyettesíti a `LAG`-ot: a frame aggregátumokhoz való, a `LAG` közvetlenül az előző sor értékét másolja.

7.3.4 Teljes minimális SQL-munkapélda

Az alábbi Python-példa csak a standard `sqlite3` modult használja, tehát a `lab` virtuális környezetben is azonnal futtatható. A munkakönyvtár legyen az a könyvtár, ahová a `s027_lab.py` fájlt mentjük; a fájlt Linuxon/macOS-en és Windows PowerShellben is a `python s027_lab.py` paranccsal indíthatjuk.

```
import sqlite3

con = sqlite3.connect(":memory:")

con.executescript("""
CREATE TABLE orders (
  order_id     TEXT PRIMARY KEY,
  customer_id  TEXT NOT NULL,
  order_ts     TEXT NOT NULL,
  basket_minor INTEGER NOT NULL
);

INSERT INTO orders VALUES
  ('o1','C1','2025-01-10T08:00:00+00:00',1000),
  ('o2','C1','2025-01-20T08:00:00+00:00', 500),
  ('o3','C1','2025-01-30T08:00:00+00:00', 800),
  ('o4','C2','2025-01-15T08:00:00+00:00', 300),
  ('o5','C2','2025-01-22T08:00:00+00:00', 700);
""")

query = r"""
WITH placed AS (
  SELECT
    order_id,
    customer_id,
    order_ts,
    basket_minor,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY order_ts, order_id
    ) AS order_seq
  FROM orders
```

```
),
past_features AS (
  SELECT
    order_id,
    customer_id,
    order_ts,
    order_seq,
    COUNT(*) OVER (
      PARTITION BY customer_id
      ORDER BY order_ts
      GROUPS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    ) AS prior_order_count,
    MAX(order_ts) OVER (
      PARTITION BY customer_id
      ORDER BY order_ts
      GROUPS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    ) AS prev_order_ts,
    SUM(basket_minor) OVER (
      PARTITION BY customer_id
      ORDER BY order_ts
      GROUPS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    ) AS prior_basket_minor
  FROM placed
)
SELECT
  order_id,
  customer_id,
  order_ts,
  prior_order_count,
  prev_order_ts,
  ROUND(JULIANDAY(order_ts) - JULIANDAY(prev_order_ts)) AS days_since_prev
FROM past_features
ORDER BY customer_id, order_ts, order_id;
"""

for row in con.execute(query):
    print(row)

con.close()
```

A várható kimenet a mini tábla kézi kiértékeléséből, nem helyi futási jegyzékből:

```
('o1', 'C1', '2025-01-10T08:00:00+00:00', 0, None, None)
('o2', 'C1', '2025-01-20T08:00:00+00:00', 1, '2025-01-10T08:00:00+00:00', 10.0)
('o3', 'C1', '2025-01-30T08:00:00+00:00', 2, '2025-01-20T08:00:00+00:00', 10.0)
('o4', 'C2', '2025-01-15T08:00:00+00:00', 0, None, None)
('o5', 'C2', '2025-01-22T08:00:00+00:00', 1, '2025-01-15T08:00:00+00:00', 7.0)
```

A kód a `prior_basket_minor` értéket is kiszámítja, de a fő kimenetből az áttekinthetőség kedvéért elhagytuk; a `C1`-nél `o1` esetén `NULL`, `o2` esetén `1000`, `o3` esetén `1500`, `C2`-nél pedig `NULL`, majd `300`. A `COUNT` üres keretnél `0`-t ad, a `SUM` viszont `NULL`-t; ha a kosárösszegnél üzletileg a nulla a kívánt érték, `COALESCE(..., 0)` szükséges.

A CTE-k közötti kapcsolat fontos: a `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts, order_id)` a **sorszám kiosztásának** rendezését adja meg, nem a végeredmény sorainak sorrendjét. A második CTE ezt a sorszámot, valamint a `LAG` és a frame ablakokat számolja. A tényleges kimeneti sorrendet a legkülső `ORDER BY customer_id, order_ts, order_id` adja; enélkül a motor bármilyen sorrendben visszaadhatja a sorokat. Ugyanarra a CTE-re a külső `ORDER BY order_id DESC` az `o5, o4, o3, o2, o1` sorrendet adja, a `customer_id, order_ts, order_id` pedig `o1, o2, o3, o4, o5`-öt; a külső `ORDER BY` nélküli eredmény sorrendje nem garantált, még ha a kis példában éppen egyezik is. Az azonos időbélyeg nélküli mini táblán a

`prior_order_count` megegyezik `order_seq - 1`-gyel, tehát a két számítás egymás ellenőrzése is; azonos időbélyeg esetén a `GROUPS` frame szándékosan eltér ettől, mert a teljes azonos idejű csoportot kizárja.

Azonos időbélyeg. Ha két rendelés `order_ts` értéke azonos, a `ROW_NUMBER() OVER (... ORDER BY order_ts, order_id)` és a `ROWS ... 1 PRECEDING` frame az `order_id` szerinti önkényes sorrendet használja, és az egyik azonos idejű rendelést a másik „múltjaként" láthatja. Ez sértheti az elérhetőségi szerződést: a feature csak azt használhatja, ami a predikció pillanatában már ismert. Ha a szigorúan korábbi időbélyegű rendeléseket akarjuk, `GROUPS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING` frame kell, amely a teljes azonos időbélyegű csoportot kizárja:

```
MAX(order_ts) OVER (
  PARTITION BY customer_id
  ORDER BY order_ts
  GROUPS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
) AS prev_order_ts
```

A mini példa kiegészítése két azonos idejű rendeléssel (`o2a`, `o2b`, mindkettő `2025-01-20T08:00:00+00:00`): a `ROWS` frame-nél `o2b` már látja `o2a` 500 centjét (`prior=1500`), a `GROUPS` frame-nél mindkettő csak a korábbi `o1` 1000 centjét látja (`prior=1000`). A `ROW_NUMBER` továbbra is adhat sorszámot, de az azonos időbélyegű csoport sorrendje nem jelent valós eseménysorrendet; ha ilyen kell, dokumentált `event_seq`/`ingest_seq` oszlop szükséges.

7.3.5 SQLite, DuckDB és PostgreSQL összevetés

A `WITH`, `PARTITION BY`, `ROW_NUMBER`, `LAG` és a frame szintaxisa a három rendszerben közel azonos. A típus- és dátumkezelésben van fontos különbség.

Kérdés	SQLite	DuckDB	PostgreSQL
`order_ts` tárolása	`TEXT`, az állandó `+00:00` miatt itt a szöveges rendezés is időrend	`TIMESTAMP` értelmezhető	`TIMESTAMPTZ` javasolt
Naptári naphatár	`DATE(order_ts) - DATE(prev_order_ts)`	`date_diff('day', prev_order_ts, order_ts)`	`(order_ts::date - prev_order_ts::date)`
Eltelt nap	`JULIANDAY(order_ts) - JULIANDAY(prev_order_ts)`	`date_diff('second', prev_order_ts, order_ts) / 86400.0`	`EXTRACT(EPOCH FROM (order_ts - prev_order_ts)) / 86400`
Ablak szintaxisa	azonos	azonos	azonos

A `date_diff('day', ...)` **naptári naphatárt** számol, nem eltelt 24 órás periódust: a 2025-01-10 23:00 és 2025-01-11 01:00 között az érték 1, pedig csak 7200 másodperc (0,0833 nap) telt el. Ha a `days_since_prev` jelentése „hány nap telt el", akkor a másodperces különbség / 86400 a helyes; ha „hány naptári nap-határt léptünk át", akkor a `date_diff('day')`. A SQLite `JULIANDAY`-különbsége eltelt napot ad (tört értékkel), a `ROUND` pedig megjelenítési döntés; a három rendszer eredménye csak akkor egyezik, ha ugyanazt a jelentést választjuk. A mini példa értékei (10 és 7 nap) szándékosan egész napok, ezért ott a kétféle számítás egybeesik; éles adatnál a különbség láthatóvá válik.

A Northstar kezdőlabor SQLite-ot használ; a DuckDB a Fájlformátumok, Arrow, DuckDB és Polars alfejezetben kap részletes szerepet. A táblázat célja csak az, hogy a platformváltásnál a dátum kifejezést külön kell ellenőrizni, magát a window-szerkezetet nem.

7.3.6 Ellenőrző invariánsok

1. Az output sorainak száma megegyezik az `orders` sorainak számával; a kulcs marad `order_id`.
2. Minden ügyfélnél az idő szerinti első sor `prior_order_count = 0`.
3. Az ügyfélen belül `prior_order_count` monoton növekvő; azonos időbélyeg nélkül megegyezik `ROW_NUMBER() - 1`-gyel, azonos időbélyeg esetén a `GROUPS` frame miatt a teljes azonos idejű csoport kimarad.
4. `prev_order_ts` pontosan az ügyfél előző rendelésének `order_ts` értéke; az első sornál `NULL`.

A Northstar `orders.csv` tiszta fájljára is alkalmazhatjuk ezt a lekérdezést. Az `orders_raw.csv` szándékos hibáit nem szabad közvetlenül a feature-számításba engedni; a tisztítás és a kulcsegyediség ellenőrzése a 6.3. alfejezet feladata. A tiszta tábla kontrollja után az ott bemutatott szándék: egy rendelés egy sor, tehát a window nem szaporítja a sorokat.

7.3.7 Feladatok

7.3.8 1. Értelmezés a mini táblán

Add meg a `C1` ügyfél `o1, o2, o3` soraihoz az alábbi oszlopok értékét: `order_seq`, `prior_order_count`, `prev_order_ts`, `days_since_prev`. A mini táblát a főpélda `INSERT` utasítása tartalmazza.

7.3.9 2. Implementálás a Northstar adatra

A `lab/data/orders.csv` tiszta fájlt töltsd be SQLite-ba `orders` néven, legalább az `order_id, customer_id, order_ts, basket_minor` oszlopokkal. A betöltéshez használhatsz Python `csv.DictReader`-t és `executemany`-t a `basket_minor` egész átalakításával. Futtasd a főpélda CTE-jét, és ellenőrizd:

- a kimeneti sorok száma `2400`;
- a `COUNT(DISTINCT order_id)` szintén `2400`;
- a `MIN(prior_order_count) = 0`;
- az `order_ts` ügyfélen belül nem csökken.

A feladathoz minden szükséges bemenet az ADATOK.md `orders.csv` szerződése; más külső letöltés nem kell.

7.3.10 3. Hibadiagnózis

Egy kolléga a következő lekérdezést küldte a mini adatra:

```
WITH x AS (
  SELECT
    order_id,
    customer_id,
    COUNT(*) OVER (
      PARTITION BY customer_id
      ORDER BY order_ts, order_id
    ) AS prior_order_count
  FROM orders
)
SELECT * FROM x
WHERE customer_id = 'C1'
ORDER BY order_ts, order_id;
```

Magyarázd el, miért rossz a `prior_order_count` név, és pontosan mi lesz a `C1` három sorában. Add meg a javítást a frame segítségével.

7.3.11 4. Megváltozott feltétel: korábbi kosárösszeg és napkülönbség

A főpélda `past_features` CTE-jében a `prior_basket_minor` összeg már szerepel. Írj SQL-lekérdezést, amely rendelésenként kiírja a `prior_basket_minor` és a `days_since_prev` értékeket. Ellenőrizd a mini adaton, hogy `o3` értéke `1500`, `o5` értéke `300`, az első rendeléseknél pedig `NULL` szerepel. Döntsd el, hogy a `NULL` vagy a `0` a megfelelő üzleti jelentés; indokold a választást.

7.3.12 Megoldások és indoklás

7.3.13 1. megoldás

`C1` rendezett sorrendje `o1, o2, o3`.

order_id	order_seq	prior_order_count	prev_order_ts	days_since_prev
o1	1	0	NULL	NULL
o2	2	1	2025-01-10T08:00:00+00:00	10
o3	3	2	2025-01-20T08:00:00+00:00	10

A `ROW_NUMBER` a rendezett partícióban valóban `1, 2, 3`; a `-1` a `0, 1, 2` prefixösszeget adja. A `LAG` az eggyel korábbi időpontot másolja, ezért a második sornál `10`, a harmadiknál szintén `10` nap a különbség.

7.3.14 2. megoldás

```
import csv
import sqlite3

con = sqlite3.connect("northstar.sqlite")
con.execute("DROP TABLE IF EXISTS orders")
con.execute("""
CREATE TABLE orders (
  order_id     TEXT PRIMARY KEY,
  customer_id  TEXT NOT NULL,
  order_ts     TEXT NOT NULL,
  basket_minor INTEGER NOT NULL
)
""")

with open("orders.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    rows = [
        {
            "order_id": row["order_id"],
            "customer_id": row["customer_id"],
            "order_ts": row["order_ts"],
            "basket_minor": int(row["basket_minor"]),
        }
        for row in reader
    ]

con.executemany(
    "INSERT INTO orders VALUES (:order_id, :customer_id, :order_ts, :basket_minor)",
    rows,
)
con.commit()
```

Ezután a főpélda `query` változóját változtatás nélkül futtathatjuk. Az ellenőrző lekérdezés:

```
SELECT
  COUNT(*) AS row_count,
  COUNT(DISTINCT order_id) AS distinct_orders,
  MIN(prior_order_count) AS min_prior
FROM past_features;
```

A várt szám `2400, 2400, 0`. Az ügyfélen belüli monotonitást a korábbi `ROW_NUMBER - 1 = prior_order_count` invariant is ellenőrzi; ehhez a `past_features` CTE-t ki kell egészíteni az `order_seq` oszloppal.

7.3.15 3. megoldás

A hibás lekérdezés nem ad meg frame-et, ezért az SQLite a `COUNT`-ra az alapértelmezett `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` keretet használja. Ez az aktuális sort is beleszámítja, tehát `C1` sorai `1, 2, 3` lesznek, miközben a név `prior_order_count`. A helyes cél `0, 1, 2`. A javítás:

```
COUNT(*) OVER (
  PARTITION BY customer_id
  ORDER BY order_ts, order_id
  ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
) AS prior_order_count
```

Ugyanez a `ROW_NUMBER() - 1` számítás is, de a `ROWS` frame expliciten dokumentálja, hogy az aktuális sor nem tartozik a múltbéli ablakba.

7.3.16 4. megoldás

```
WITH placed AS (
  SELECT order_id, customer_id, order_ts, basket_minor
  FROM orders
),
past_features AS (
  SELECT
    order_id,
    customer_id,
    order_ts,
    LAG(order_ts) OVER (
      PARTITION BY customer_id
      ORDER BY order_ts, order_id
    ) AS prev_order_ts,
    COALESCE(
      SUM(basket_minor) OVER (
        PARTITION BY customer_id
        ORDER BY order_ts, order_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
      ),
      0
    ) AS prior_basket_minor
  FROM placed
)
SELECT
  order_id,
  customer_id,
  prior_basket_minor,
  ROUND(JULIANDAY(order_ts) - JULIANDAY(prev_order_ts)) AS days_since_prev
FROM past_features
ORDER BY customer_id, order_ts, order_id;
```

Mini adaton a `prior_basket_minor` a `C1`-nél `0, 1000, 1500`, a `C2`-nél `0, 300`. A `COALESCE` az első rendelésnél `0`-t ad, ami akkor megfelelő, ha az üzleti kérdés a „korábban elköltött összeg”: nincs korábbi rendelés, tehát nulla az összeg. Ha a hiányt külön akarjuk megkülönböztetni az „új ügyfél” állapottól, a `NULL` is indokolható; a szerződést a modellkártyán kell rögzíteni.

7.3.17 Gyakori tévedés ellenpéldával

A legveszélyesebb hiba a frame nélküli ablak, amely a teljes ügyféltörténetet használja. Például

```
AVG(basket_minor) OVER (PARTITION BY customer_id)
```

a `C1` minden sorához `(1000 + 500 + 800) / 3 = 766.67`-et ad. Ez a rendelés pillanatában a **jövőbeli** `o3` kosárértékét is tartalmazza, tehát a feature a célváltozóhoz képest jövőinformációt szivárogtat be. A helyes múltbeli átlag például `o3` előtt `(1000 + 500) / 2 = 750`, `o2` előtt pedig `1000`. A 7.2. alfejezet join-kardinalitási szabályához hasonlóan itt is a feladat a szemcsézettség megőrzése: a window nem join, de a keret megválasztása ugyanúgy eldönti, mely sorok magyarázhatják az adott sort. A 6.3. alfejezetből az is következik, hogy a featurét csak tisztított, egyedi rendeléskulcsú táblára számítsuk.

7.3.18 Önellenőrzés

- Minden partíció első sora `prior_order_count = 0`?
- A `prior_order_count` a `ROW_NUMBER()` sorszámnál pontosan eggyel kisebb?
- A `SUM/AVG` keretéből kizártam az aktuális sort: `... AND 1 PRECEDING`?
- A `LAG` első sora `NULL`, és ezt a kimenet tudatosan kezeli?
- Az `ORDER BY` az időpont után `order_id`-t is tartalmaz, tehát a rendezés determinisztikus?

Azonos `order_ts` mellett a másodlagos `order_id` rendezés determinisztikus sorrendet ad, de nem teszi az egyik rendelést a másiknál időben korábbivá. Ha a „korábbi” jelentése szigorúan `order_ts < aktuális`, az időbélyeg-holtversenyek kizárását külön kell biztosítani; a `ROW_NUMBER` önmagában nem elegendő.

7.3.19 Rövid összefoglaló

A `WITH` CTE lépésekre bontja a feature-előállítást; a `PARTITION BY customer_id` és az `ORDER BY order_ts, order_id` határozza meg a sorrendet. A `ROW_NUMBER` a rendezett sorozat indexét adja, a `LAG` a szomszédos elem eltérését, míg a `ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING` keret a jelen sort kizáró prefixösszeget számolja. Ez adja a predikció pillanatában ismert `prior_order_count`-ot úgy, hogy sem az aktuális, sem a jövőbeli rendelés nem kerül a jellemzőbe. A következő lépés a 7.4. alfejezetben a paraméterezett lekérdezés és a lekérdezési terv: ugyanezt a CTE-t hogyan futtassuk biztonságosan változó cutofftal anélkül, hogy szövegből összefűzött SQL-t használnánk.

7.4 Paraméterezett lekérdezés és lekérdezési terv

Belépő · szint: alap · idő: 45 perc · eszköz: SQL, DuckDB

Előfeltétel: 7.3. alfejezet · Célkészség: Paraméterezett lekérdezés és lekérdezési terv · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

A 7.3. alfejezetben elkészített `prior_order_count` CTE akkor válik biztonságos adatkinyeréssé, ha a felhasználó által megadott időhatárokat nem szövegösszefűzéssel illesztjük az SQL-be, hanem paraméterként adjuk át. Ebben az alfejezetben a paraméterezést, a kimeneti oszlopok megválasztását, a szűrés alakját, valamint a lekérdezési terv ellenőrzését nézzük meg három motoron: SQLite, DuckDB és PostgreSQL.

7.4.1 Helyzet, bemenet és induló becslés

A Northstar labor `orders.csv` és `customers.csv` tábláiból olyan dátumablakos kinyerést készítünk, amely a 7.3. alfejezetben bemutatott előzményfeature-t is tartalmazza. A kérés:

- megfigyelési egység: egy rendelés, `order_id`;
- bemeneti paraméterek: `start_ts`, `end_ts`, `min_basket_minor`;
- kimeneti oszlopok: `order_id`, `order_ts`, `customer_id`, `basket_minor`, `prior_order_count`, `segment`;
- szűrés: `order_ts` a `[start_ts, end_ts)` féloldalas ablakban van, és `basket_minor >= min_basket_minor`.

A rendelések 180 napra oszlanak el, összesen 2400 sorral. Egy 7 napos ablakra ezért nagyságrendileg

$$\hat{N} \approx 2400 \cdot \frac{7}{180} \approx 93$$

rendelést jósol. Ha `min_basket_minor = 5000` is szerepel, a sorok száma ennél kevesebb lehet. Ez az előzetes becslés csak ellenőrzési támpont, nem a végső eredmény.

7.4.2 Fogalmi magyarázat

7.4.2.1 Paraméterezés és SQL injection

A paraméterezett lekérdezésben (prepared statement) a változó értékeket placeholder jelöli, például `?` vagy `%s`. A motor a paramétereket adatként kezeli, nem pedig SQL-szövegként. Ha a felhasználó például a

```
2025-05-01' OR '1'='1
```

értéket adja meg cutoffként, a naiv szövegösszefűzés ezt az SQL utasítás részévé teszi:

```
cutoff = "2025-05-01' OR '1'='1"
sql = f"SELECT order_id FROM orders WHERE order_ts >= '{cutoff}'"
```

Ez az `OR '1'='1'` miatt az összes sort visszaadja, tehát a lekérdezés fut, csak éppen nem azt csinálja, amit akartunk. A biztonságos változatban a `?` értékét a motor külön paraméterként köti, így az idézőjel és az `OR` is egyszerű adatkarakter marad.

7.4.2.2 Injektálási teszt és végrehajtási terv

A támadó bemenet nem csak „hibás érték”: a lekérdezés szerkezetét próbálja megváltoztatni. A `lab/sql_guard.py` három bemenettel mutatja meg a különbséget:

Bemenet	Naiv szövegösszefűzés	Kötött paraméter
`O1`	1 sor (`O1`)	1 sor (`O1`)
`X' OR '1'='1`	**3 sor** (mind)	0 sor
`X'; DROP TABLE orders; --`	`ProgrammingError` (több utasítás)	0 sor

A naiv és a biztonságos lekérdezés normál értékre ugyanazt a végrehajtási tervet adja; a támadó bemenet viszont a naiv változat tervét `SCAN`-re változtatja, míg a kötött paraméteres változat `SEARCH ... USING INDEX` marad:

```
safe, 'O1'          SEARCH orders USING COVERING INDEX ... (order_id=?)
unsafe, 'O1'        SEARCH orders USING COVERING INDEX ... (order_id=?)
unsafe, támadás     SCAN orders USING COVERING INDEX ...
safe, támadás       SEARCH orders USING COVERING INDEX ... (order_id=?)
```

A végrehajtási terv ellenőrzése így nem csak teljesítménykérdés: azt is megmutatja, hogy a bemenet nem változtatta-e meg a lekérdezés alakját. A `EXPLAIN QUERY PLAN` kimenetét a `tests/test_sql_guard.py` hasonlítja össze; a `SCAN` és a `SEARCH` szó a terv szövegében stabil, motoronként eltérő részletekkel.

```python
import sqlite3

ATTACK = "X' OR '1'='1"

def unsafe_query(connection, value):
    return connection.execute(
        f"SELECT order_id FROM orders WHERE order_id = '{value}'").fetchall()

def safe_query(connection, value):
    return connection.execute(
        "SELECT order_id FROM orders WHERE order_id = ?", (value,)).fetchall()

def plan(connection, sql, params=()):
    return connection.execute("EXPLAIN QUERY PLAN " + sql, params).fetchall()

def identifier(name, allowed):
    if name not in allowed:
        raise ValueError(f"nem engedett azonosító: {name!r}")
    return name
```

A **paraméterezés csak értékre működik, azonosítóra nem**. A tábla- és oszlopnevet nem lehet `?`-lel helyettesíteni; ha a felhasználó választja ki a táblát, whitelist kell:

```python
table = identifier(user_table, {"orders", "customers", "order_items"})
sql = f"SELECT * FROM {table} WHERE order_id = ?"
```

A whitelist a teljes azonosítót ellenőrzi, nem részstringet: az `orders; DROP TABLE orders; --` nem engedett, mert nincs a halmazban. A `tests/test_sql_guard.py` a három támadó bemenetet, a `SCAN`/`SEARCH` tervet és a whitelistet teszteli.

7.4.2.3 Projection

A projekció a lekérdezés `SELECT` utáni oszloplistája. A `SELECT *` rövid, de hátrányos:

- a tábla sémaváltozásakor a kimenő oszlopkészlet észrevétlenül megváltozik;
- felesleges oszlopokat mozgat a lemez, a memória és a hálózat felé;
- a kód nem dokumentálja, mire épít a downstream.

A példában ezért csak a ténylegesen használt hat oszlopot vetítjük.

7.4.2.4 Filter és half-open időablak

A `WHERE` záradék a sorok kiválasztását végzi. Dátumoszlopnál az

```sql
WHERE order_ts >= ? AND order_ts < ?
```

féloldalas, `[start, end)` intervallumot ad. Ez azért fontos, mert az egymás utáni ablakok hézag és átfedés nélkül illeszkednek: a következő ablak `start`-ja pontosan az előző `end`-je. A `BETWEEN` vagy a `<= end` kettős beszámolást okozhat, ha a határpont több csoportba is esik.

Az indexhasználathoz az is kell, hogy a szűrés az oszlopra közvetlenül vonatkozzon, ne egy kiszámított kifejezésre. A `WHERE date(order_ts) = '2025-04-01'` minden sorra kiszámolja a dátumot, ezért a motor jellemzően teljes táblát olvas. A sargable változat:

```sql
WHERE order_ts >= '2025-04-01T00:00:00+00:00'
  AND order_ts <  '2025-04-02T00:00:00+00:00'
```

Ez tartományszűrést ad, amely indexszel támogatható.

7.4.2.5 Index és oszlopos tárolás

A hagyományos, sororientált motorokban — mint az SQLite vagy a PostgreSQL — egy B-tree index az `order_ts` oszlopon a rendezett kulcsok alapján gyorsan megtalálhatja a tartomány kezdőpontját, majd sorban olvashat a határig. Az index akkor hasznos, ha a predikátum szelektív, vagyis a tábla kis hányadát érinti. Példánkban a 7/180 arány körülbelül 3,9%, ami jellemzően kedvez az indexnek. Ha viszont a szűrés a sorok 90%-át visszaadná, a teljes táblázatolvasás gyakran olcsóbb, mert az indexbejegyzés és a sorok közötti ide-oda ugrálás drága.

A DuckDB oszlopos motort használ: az egyes oszlopok külön tárolódnak, és minden oszlopszegmenshez min/max jellegű metaadat tartozik. Egy `order_ts >= start` szűrésnél a motor a metaadatokkal egész szegmenseket kihagyhat, anélkül hogy külön B-tree indexet kellene karbantartani. Ezért DuckDB-n általában nincs szükség kézzel létrehozott indexre a gyakori oszlopos szűrésekhez; a tervező a zónatérképek és a statisztikák alapján dönt.

7.4.2.6 SQL-dialektus

A három motor között a paraméter- és típuskezelésben fontos különbségek vannak:

Motor	Paraméter placeholder	`order_ts` típus	Végrehajtási terv
SQLite Pythonból	`?`	jellemzően ISO-szöveg	`EXPLAIN QUERY PLAN`
DuckDB Pythonból	`?`	`TIMESTAMP`	`EXPLAIN`
PostgreSQL psycopg2-vel	`%s`	`TIMESTAMPTZ`	`EXPLAIN (ANALYZE, BUFFERS)`

Az SQLite-ban nincs külön időbélyeg-típus; ha az oszlopot szövegként tároljuk, a lexikografikus összehasonlítás csak akkor helyes, ha minden érték azonos formátumú, például `YYYY-MM-DDTHH:MM:SS+00:00`. DuckDB-ben a CSV-betöltés után a `DESCRIBE orders` paranccsal ellenőrizzük, hogy a motor `TIMESTAMP` típusra következtetett-e. PostgreSQL-ben a `TIMESTAMPTZ` időzónával együtt tárol, ezért a bindolt paramétert is zónatudatos értékként küldjük.

7.4.3 Minimális, kézzel ellenőrizhető példa

Az alábbi Python-kód nem a teljes Northstar-adatot használja, hanem egy hat soros memóriatáblát, amelyen a várt eredmény kézzel is kiszámítható. A kód a Python standard könyvtárával fut; a kimenetet itt kézi levezetés alapján várhatóként jelöljük, nem pedig mért futásként.

```
import sqlite3

con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE orders (
    order_id TEXT PRIMARY KEY,
    order_ts TEXT NOT NULL,
    customer_id TEXT NOT NULL,
    basket_minor INTEGER NOT NULL
);

CREATE TABLE customers (
    customer_id TEXT PRIMARY KEY,
    segment TEXT NOT NULL
);
```

```
INSERT INTO orders VALUES
    ('o1','2025-04-01T10:00:00+00:00','c1',8000),
    ('o2','2025-04-02T10:00:00+00:00','c2',4000),
    ('o3','2025-04-05T11:00:00+00:00','c1',1000),
    ('o4','2025-04-07T12:00:00+00:00','c1',6000),
    ('o5','2025-04-08T10:00:00+00:00','c2',9000),
    ('o6','2025-04-01T09:00:00+00:00','c3',3000);

INSERT INTO customers VALUES
    ('c1','business'),
    ('c2','consumer'),
    ('c3','business');
""")

query = """
WITH prior AS (
    SELECT
        order_id,
        customer_id,
        order_ts,
        basket_minor,
        COUNT(*) OVER (
            PARTITION BY customer_id
            ORDER BY order_ts, order_id
            ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
        ) AS prior_order_count
    FROM orders
)
SELECT
    p.order_id,
    p.order_ts,
    p.basket_minor,
    p.prior_order_count,
    c.segment
FROM prior AS p
JOIN customers AS c ON c.customer_id = p.customer_id
WHERE p.order_ts >= ?
  AND p.order_ts <  ?
  AND p.basket_minor >= ?
ORDER BY p.order_ts, p.order_id;
"""

start = "2025-04-01T00:00:00+00:00"
end = "2025-04-08T00:00:00+00:00"
min_basket = 5000

print(list(con.execute(query, (start, end, min_basket))))
```

Várt kimenet:

```
[('o1', '2025-04-01T10:00:00+00:00', 8000, 0, 'business'),
 ('o4', '2025-04-07T12:00:00+00:00', 6000, 2, 'business')]
```

A `prior_order_count` a teljes múltbeli sorrendet tükrözi, nem csak a szűrt ablakot. `c1` három rendelése: `o1`, `o3`, `o4`. Az `o1` az első, ezért `prior_order_count=0`; az `o4` előtt két korábbi rendelés van, ezért `2`. Az `o3` az ablakba beleesne időben, de a kosárértéke 3000 cent, így a `min_basket_minor=5000` kiszűri. Az `o5` időben kívül esik, mert a féloldalas ablak `2025-04-08T00:00:00`-nál már nem veszi fel.

Az indexhatás szemléltetésére ugyanebben a SQLite-kapcsolatban indexet hozunk létre, majd lekérdezési tervet kérünk:

```
con.execute("CREATE INDEX idx_orders_order_ts ON orders(order_ts);")
con.execute("ANALYZE orders;")

plan_rows = con.execute(
    "EXPLAIN QUERY PLAN SELECT order_id FROM orders "
    "WHERE order_ts >= ? AND order_ts < ?"
).fetchall()
print(plan_rows)
```

A kimenet nagyon kis táblán is lehet teljes táblázatolvasás, mert a tervező joggal dönthet úgy, hogy hat sort nem érdemes indexen keresni. Nagyobb adaton a szelektív tartományszűréssel jellemzően `SEARCH orders USING INDEX idx_orders_order_ts` típusú sor jelenik meg. A fontos tanulság: a terv a motor becslését mutatja, nem a kívánságunkat.

A Northstar-adaton ugyanez a lekérdezés DuckDB-vel a következő keretben futtatható. A fájlnevek a labor projektgyökeréből értendők, a kimenet nem előre mért, hanem a szerződésből várt:

```
# lab/s028_window_query.py
import duckdb

START = "2025-05-01T00:00:00+00:00"
END = "2025-05-08T00:00:00+00:00"
MIN_BASKET = 5000

con = duckdb.connect()

# Betöltés és típusellenőrzés: DESCRIBE orders mutassa, hogy az order_ts
# TIMESTAMP lett-e. Ha szöveg marad, a lexikografikus összehasonlítás csak
# +00:00 kezdetű ISO értékekkel biztonságos.
con.execute(
    "CREATE TABLE orders AS "
    "SELECT * FROM read_csv_auto('lab/data/orders.csv')"
)
con.execute(
    "CREATE TABLE customers AS "
    "SELECT * FROM read_csv_auto('lab/data/customers.csv')"
)

query = """
WITH prior AS (
    SELECT
        order_id,
        customer_id,
        order_ts,
        basket_minor,
        COUNT(*) OVER (
            PARTITION BY customer_id
            ORDER BY order_ts, order_id
            ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
        ) AS prior_order_count
    FROM orders
)
SELECT
    p.order_id,
    p.order_ts,
    p.customer_id,
    p.basket_minor,
    p.prior_order_count,
    c.segment
FROM prior AS p
JOIN customers AS c ON c.customer_id = p.customer_id
```

```
WHERE p.order_ts >= ?
  AND p.order_ts <  ?
  AND p.basket_minor >= ?
ORDER BY p.order_ts, p.order_id;
"""

df = con.execute(query, [START, END, MIN_BASKET]).fetchdf()
print(df.head())
print(df.shape)
```

A `?` helyettesítő SQLite és DuckDB Python-eszközökben is működik. PostgreSQL `psycopg2` esetén ugyanezt a szöveget `%s` helyettesítőkre cseréljük, és a paramétereket tuple-ként adjuk át:

```
cur.execute(query, (start, end, min_basket))
```

7.4.4 Eredmény értelmezése és ellenőrző invariantok

A mini példán a várt eredmény két sor, és a következő invariánsok ellenőrizhetők:

- a kiválasztott `order_ts` minden értéke a `[start, end)` ablakban van;
- `prior_order_count` egyik sorban sem nagyobb, mint az adott ügyfél összes, az adott rendelés előtti rendeléseinek száma;
- a `basket_minor` minden sorban legalább `min_basket_minor`;
- a kimenet `order_ts` és `order_id` szerint rendezett.

A teljes Northstar-adatra a 7 napos ablakra közel 93 sor várható. Ha a mért sorok száma nagyságrendileg ettől eltér, akkor valószínűleg hibás az `order_ts` besorolása: az oszlop szöveg maradt időzóna nélkül, vagy a paramétert nem sikerült megfelelően bindolni. Ilyenkor először a betöltött tábla `DESCRIBE` kimenetét és a `min(order_ts), max(order_ts)` értékét érdemes megnézni.

7.4.5 Feladatok

1. **Értelmezés.** A mini táblán miért nem szerepel `o5` a kimenetben, ha az `end = '2025-04-08T00:00:00+00:00'`? Mi történne, ha a szűrés `<=` lenne? Rajzold fel a `[start, end)` és a `[start, end]` intervallum különbségét egy egységes, 30 perces léptékű idővonalon.
2. **Megvalósítás.** Futtasd a mini SQLite-példát `start='2025-04-02T00:00:00+00:00'`, `end='2025-04-08T00:00:00+00:00'`, `min_basket=1000` paraméterekkel. Mely sorok jelennek meg, és mekkora a `prior_order_count` az egyes sorokban?
3. **Hibakeresés.** Egy kolléga ezt írja: `WHERE date(order_ts) = '2025-04-01'`. Miért lehet ez teljes táblázatolvasás akkor is, haz `order_ts`-n van index? Írd át a szűrést olyan tartománnyá, amely ugyanazt az egy napot jelöli, és sargable.
4. **Transzfer.** A DuckDB-példát PostgreSQL-re kell portolnia. Írd le a két szükséges változtatást: milyen lesz a placeholder, és milyen típust várjon az `order_ts` oszlop? Ezután magyarázd el, miért nem jelent automatikusan gyorsabb futást, ha az `EXPLAIN` kimenetében rövidebb terv szerepel.

7.4.6 Megoldások és indoklás

1. Az `o5` értéke `2025-04-08T10:00:00+00:00`, ami később van, mint az intervallum jobb oldali, nyílt határa. A `[start, end)` féloldalas ablak az `end` pillanatáig tart, de azt már nem tartalmazza. Így a `start=2025-04-01T00:00`, `end=2025-04-08T00:00` ablak az áprilisi 1-i 00:00-tól 8-i 00:00-ig terjed, összesen pontosan 7 nap. Ha a szűrés `<=` lenne, az `o5` is bekerülne, és a következő ablakkal átfedés keletkezne. Az egymás után fűzött féloldalas ablakok hézagmentesen fedik az időt.

2. A paraméterekkel a mini táblából `o2`, `o3`, `o4` kerül ki. Az `o1` időben az ablak előtt van, az `o5` időben a határ után, az `o6` pedig a kosárérték vagy időablak? `o6` időben 04-01 09:00, tehát a start 04-02 előtt, ezért kiesik. A megmaradt sorok prior értékei: `o2` előtt `c2`-nek nincs rendelése, tehát 0; `o3` előtt `c1`-nek van `o1`, tehát 1; `o4` előtt `c1`-nek van `o1` és `o3`, tehát 2. Ez megmutatja, hogy a paraméterezett ablakon kívüli múltbeli rendelés is beleszámít az előzményfeature-be.

3. A `date(order_ts)` kifejezés minden sorra új számítást végez, ezért a motor nem tudja közvetlenül az `order_ts` indexből kiolvasni a keresett tartományt. Ehelyett beolvassa az összes sort, kiszámolja a dátumot, és utána hasonlít. A helyes megfogalmazás:

   ```
   WHERE order_ts >= '2025-04-01T00:00:00+00:00'
     AND order_ts <  '2025-04-02T00:00:00+00:00'
   ```

   Ez a naptári napot ugyanúgy jelöli, de tartományszűrés, amely az `order_ts` indexszel támogatható.

4. PostgreSQL alatt a placeholder `%s`, és a paramétereket tuple-ként, a psycopg2 `execute(sql, params)` hívásában kell átadni. Az `order_ts` oszlop érdemes `TIMESTAMPTZ` típusú, hogy a zónainformáció ne vesszen el. A rövidebb `EXPLAIN` terv nem bizonyítja, hogy a lekérdezés gyorsabb: a terv a becsült költséget mutatja; a tényleges futásidőhöz gépet, adatméretet, cache-állapotot, ismétlésszámot és mért eredményt kell rögzíteni. Egyetlen `EXPLAIN ANALYZE` futás sem elegendő stabil teljesítménykövetkeztetéshez.

7.4.7 Gyakori tévedés és ellenpélda

Gyakori csapda, hogy valaki a helyes eredményből arra következtet, hogy a lekérdezés biztonságos és olcsó is. A naiv f-stringes változat a fenti `cutoff = "2025-05-01' OR '1'='1"` értékkel szintén visszaadhat sok sort, sőt akár ugyanazokat, mint a paraméterezett változat, mégis SQL injectionre alkalmas. Ugyanígy egy `SELECT *`-ot és `date(order_ts)` szűrést használó kód megadhatja a kívánt sorokat, de a teljes táblát mozgatja. A 6.4. alfejezetből ismert időzóna- és típusfegyelem itt visszaköszön: a paraméterezett lekérdezés csak akkor értelmes, ha az `order_ts` értékek konzisztensen `+00:00` kezdetű, rendezhető formában vannak. Önellenőrzés: nézzük meg a tervet, a paramétereket nem szabad egyszerű szövegként a querybe írni, és a kimenő `order_ts` értékeket mindig ellenőrizzük az ablak határai ellen.

A lekérdezésparaméterezés az értékek biztonságos behelyettesítésére szolgál, nem általában a táblanevek vagy SQL-kulcsszavak dinamikus kezelésére. Változó azonosítóknál engedélyezett listát használjunk. Egy index és egy végrehajtási terv előnye a motortól, az adatoktól és a statisztikáktól függ, ezért az `EXPLAIN` megfigyeléseit mérésnek kell követnie.

7.4.8 Rövid összefoglaló

A paraméterezett lekérdezésben a felhasználói érték `?` vagy `%s` placeholder útján adat marad, nem pedig SQL-kóddá váló szöveg. A projekció explicit oszloplistát ad `SELECT *` helyett. A szűrés féloldalas `[start, end)` ablakot és sargable oszlophivatkozást használ. Sororientált motorokban a szelektív B-tree index gyorsít; DuckDB oszlopos tárolásban min/max zónatérképekkel kerüli a felesleges szegmenseket. A motorok eltérnek a paraméter- és típuskezelésben, ezért a kikötést mindig az adott dialektusban kell megfogalmazni. Az `EXPLAIN` a végrehajtási tervet mutatja, nem a helyes eredményt. A következő alkalmazás a 8.1. alfejezetben ezt a kinyerést fájlformátum, séma és partíció szempontjából viszi tovább.

8 Fájlformátumok, Arrow, DuckDB és Polars

8.1 CSV-től Parquet-ig: séma és partíció

Belépő · [BŐVÍTŐ MODUL] · szint: alap · idő: 45 perc · eszköz: Parquet, Arrow, DuckDB, Polars

Előfeltétel: 7.4. alfejezet · Célkészség: CSV-től Parquet-ig: séma és partíció · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

8.1.1 Cél és konkrét helyzet

A 7.4. alfejezetben SQL-lel kinyertük a Northstar-rendelések használható oszlopait. Most ugyanezt az adatterméket kell megosztani a kutatási notebookok és a későbbi batch-folyamatok között. A nyers `orders.csv` szöveges: nincs benne séma, ezért minden betöltő újra kitalálhatja a típusokat, és egy rosszul idézett mező csendben eltolhatja az oszlopokat.

A feladat: írjuk ki ugyanazt az adatot CSV-be és Parquet-be, majd ellenőrizzük a típusokat és a kerekítés nélküli kontrollösszeget. Bemenet a szállított `lab/data/orders.csv`, megfigyelési egysége egy rendelés, alapmérete 2400 sor. Kimenet: `out/orders_roundtrip.csv`, `out/orders.parquet`. Előzetes becslés: a sor- és oszlopszám nem változik, az egész minor unit összegek bitazonosak maradnak, a `distance_km` float64 összeg csak lebegőpontos tűréssel hasonlítható, a Parquet-fájl várhatóan kisebb lesz a CSV-nél, de a betöltött DataFrame RAM-igénye nem csökken a fájlmérettől.

8.1.2 Sor- és oszlopalapú tárolás

A CSV sororientált: a fájlban az 1. sor mezői, utána a 2. sor mezői következnek. Ha csak a `distance_km` oszlopra van szükség, akkor is be kell olvasni és értelmezni minden sort, mert a mezőhatárok csak így találhatók meg. Ez akkor kényelmes, ha egész sorokat fűzünk hozzá, például naplófájlnál.

A Parquet oszlopalapú: a fájl oszloponként tárolja az értékeket. A motor kihagyhatja a nem kért oszlopokat, és az egy oszlopon belüli értékek hasonlóságát is kihasználhatja tömörítésre. A gyakorlati következmény: analitikus lekérdezésnél, ahol kevés oszlop kell sok sorból, az oszlopos forma hatékonyabb; teljes sorok gyakori hozzáfűzésére viszont kevésbé természetes. A 7.4-ban említett zónatérkép is oszlopstatisztikára épül.

8.1.3 CSV idézés

A CSV-ben minden érték szöveg. A vessző elválasztó, ezért ha egy mező vesszőt, idézőjelet vagy újsort tartalmaz, idézőjelek közé kell tenni. Az RFC 4180 gyakorlat szerint az idézőjelen belüli idézőjel duplázással írható. Például:

```
order_id,note,amount
"A-1","contains ""quotes"" and, comma",123
```

A pandas `read_csv` alapértelmezése ezt várja: `sep=","`, `quotechar='"'`, `doublequote=True`. A második mező értéke tehát `contains "quotes" and, comma`, és a sor három oszlopból áll. A CSV nem tárol típust; a `123` lehet int, string vagy akár dátum, ezt a betöltő dönti el. Ezért kell a sémát minden együttműködő félnek külön rögzítenie.

8.1.4 JSON normalizálás

A JSON gyakran beágyazott listákat és objektumokat hoz, például egy API-válasz rendelésenként `items` listát ad. Az elemzési tábla viszont lapos: egy sor egy megfigyelési egység. A `pd.json_normalize` a listát sorokká robbantja, a szülő mezőket pedig `meta` útvonallal ismétli.

```
import json
import pandas as pd

rows = json.loads("""[
  {"order_id": "A-1",
   "customer": {"customer_id": "C-9", "segment": "business"},
   "items": [{"line_id": 1, "qty": 2}, {"line_id": 2, "qty": 1}]}
]""")

items = pd.json_normalize(
    rows,
    record_path=["items"],
    meta=["order_id", ["customer", "customer_id"], ["customer", "segment"]],
)
print(items)
```

A kimeneti sorok az `items` elemei, a `customer.customer_id` és `customer.segment` oszlopok pedig a szülő objektumból ismétlődnek. A `meta` elemei útvonalak: `["customer", "customer_id"]` a beágyazott kulcsot jelenti, nem két külön metaoszlopot.

8.1.5 Parquet séma és tömörítés

A Parquet-fájl a sémát is tárolja: minden oszlopnak fizikai típusa van, például `int64`, `double` vagy `timestamp[us, tz=UTC]`, a string pedig UTF-8 bájttömb. Ezért a jól megírt Parquetből visszaolvasás után nem kell újra kitalálni, hogy a `basket_minor` egész szám-e. A pandas `to_parquet` a DataFrame dtype-jaiból PyArrow-táblát épít, és ezt a sémát a fájl lábjegyzetébe írja.

A tömörítés két rétegű. Először az oszlopon belüli kódolás dolgozik: a kevés egyedi értékű oszlop szótárat és indexeket kaphat, az ismétlődő értékek futamhossz-kódolást, az időbélyegek delta-kódolást. Erre jön az általános codec, alapértelmezetten `snappy`, opcióként `zstd` vagy `gzip`. A `zstd` általában kisebb fájlt ad, de több processzoridőbe kerül; a konkrét arányt csak az adott gépen mért futással szabad állítani.

Adatmennyiség elemi becsléseként: ha $n = 2400$ sorban egy `int64` oszlop van, a nyers bináris méret

$$S_{\text{int64}} = 8n = 19\,200 \text{ bájt}.$$

Ugyanez decimális szövegként $n \cdot \bar{\ell}$ bájt, ahol $\bar{\ell}$ az egy érték átlagos karakterhossza; az `1000` négy karakter, a `100000` hat. Egy k egyedi értékű kategória szótárkódolt, durva alsó becslése

$$S_{\text{dict}} \approx k\bar{\ell}_v + n\left\lceil \frac{\log_2 k}{8} \right\rceil,$$

ahol $\bar{\ell}_v$ a szótárelemek átlagos hossza. A `priority_flag` esetén $k = 2$, tehát egy bit is elég lenne soronként; a valóságban a tárolási overhead és a kódolási csoportosítás miatt ennél több lesz, de az elv világos: a típus és az ismétlődés is információ, amit a CSV szövegként eldob.

8.1.5.1 Roundtrip-kontroll: séma, sentinel, időzóna, kategória

A `shape`-egyezés nem bizonyítja, hogy a roundtrip megőrizte az adatot. A `lab/parquet_contract.py` hatoszlopos mini táblán hasonlítja össze a CSV- és a Parquet-roundtripet:

Oszlop	Eredeti	CSV-roundtrip	Parquet-roundtrip
`order_id` (`"00123"`)	string	`int64`, `123` (vezető nulla elveszett)	string, `"00123"`
`amount_minor`	int64	int64	int64
`ratio` (`0.1+0.2`)	float64	float64	float64
`ts`	`datetime64[us, UTC]`	string, nincs időzóna	`datetime64[us, UTC]`
`currency`	category `EUR`	string	category `EUR`
`note` (`""`)	string, üres string	`NaN` (sentinel-összemosás)	üres string

A CSV-alapú roundtrip tehát négy különböző módon veszíthet információt: típus, vezető nulla, időzóna és üres-string/null megkülönböztetés. A Parquet a sémát és a pandas-kategóriametadatot is megőrzi; a `schema_report()` a pyarrow-sémát oszloponként adja vissza:

```
import pandas as pd
import pyarrow.parquet as pq

def demo_frame():
    return pd.DataFrame({
        "order_id": pd.Series(["00123", "00456"], dtype="string"),
        "amount_minor": pd.Series([100, 250], dtype="int64"),
        "ratio": pd.Series([0.1 + 0.2, 0.7], dtype="float64"),
        "ts": pd.to_datetime(["2025-04-01T10:00:00+00:00",
                              "2025-04-02T10:00:00+00:00"], utc=True),
        "currency": pd.Categorical(["EUR", "EUR"]),
        "note": pd.Series(["", "hello"], dtype="string"),
    })

def schema_report(path):
    schema = pq.read_schema(path)
    return [{"name": f.name, "type": str(f.type), "nullable": f.nullable}
            for f in schema]

def tz_of(series):
    try:
        return str(series.dt.tz)
    except (AttributeError, TypeError):
        return None

def compare(original, back):
    differences = []
```

```
    for name in original.columns:
        if str(original[name].dtype) != str(back[name].dtype):
            differences.append((name, "dtype"))
        if name == "order_id" and original[name].tolist() != [str(v) for v in back[name]]:
            differences.append((name, "value"))
        if name == "note" and original[name].isna().sum() != back[name].isna().sum():
            differences.append((name, "null_sentinel"))
        if name == "ts" and tz_of(original[name]) != tz_of(back[name]):
            differences.append((name, "timezone"))
        if name == "currency" and not isinstance(back[name].dtype, pd.CategoricalDtype):
            differences.append((name, "category"))
        if name == "ratio" and (original[name] - back[name]).abs().max() > 1e-15:
            differences.append((name, "precision"))
    return differences

frame = demo_frame()
frame.to_csv("/tmp/data.csv", index=False)
frame.to_parquet("/tmp/data.parquet", index=False)
csv_back = pd.read_csv("/tmp/data.csv")
parquet_back = pd.read_parquet("/tmp/data.parquet")
assert compare(frame, csv_back)
assert compare(frame, parquet_back) == []
assert any(item["type"].startswith("timestamp[us, tz=UTC]")
           for item in schema_report("/tmp/data.parquet"))
```

A roundtrip-teszt minden oszlopra külön kontraktust ellenőriz: a shape és az oszlopszám csak az első kapu. A `tests/test_parquet_contract.py` a CSV négy veszteségét, a Parquet nulla eltérését és a pyarrow-séma időzónás/kategóriás típusát ellenőrzi.

8.1.6 Partition pruning

A partíció a fájl elérési útjába emeli a gyakori szűrőt. Hive-stílusú könyvtár például:

```
out/orders_partitioned/year=2025/month=1/part-0.parquet
out/orders_partitioned/year=2025/month=2/part-0.parquet
```

A partíciós oszlop értéke a mappanévből olvasható, nem kell minden fájl tartalmát megnyitni. Ha a lekérdezés csak `month = 2` sorokat kér, a motor pruninggal kihagyja a többi mappát. Ez különbözik a zónatérkép-pruningtól, amely egy oszlop min/max statisztikáját nézi a fájlon belül.

A partíció azonban nem automatikusan gyorsítás. Ha a kardinálisság túl nagy, sok kis fájl keletkezik, és a sok metaadat-beolvasás meg a fájlmegnyitás többe kerül, mint maga a szűrés. A Northstar alapadata 180 napot fed le: napi partíció átlagosan 13 rendelést adna, ami jellemzően rossz aprózás; havi partíció 6 mappát ad, ami a havi szűréshez ésszerűbb.

8.1.7 Teljes minimális munkapélda

Mentsük a következő kódot `s029_file_formats.py` néven a lab projektgyökerébe, ahol az `orders.csv` található.

```
from pathlib import Path

import pandas as pd
import pyarrow.parquet as pq

INPUT = Path("orders.csv")
OUT = Path("out")
```

```
OUT.mkdir(exist_ok=True)

dtype = {
    "order_id": "string",
    "customer_id": "string",
    "basket_minor": "int64",
    "shipping_fee_minor": "int64",
    "quantity_sum": "int64",
    "item_rows": "int64",
    "distance_km": "float64",
    "priority_flag": "int64",
}
date_cols = ["order_ts", "actual_delivery_at", "label_available_at"]

orders = pd.read_csv(INPUT, dtype=dtype, parse_dates=date_cols)
for col in date_cols:
    orders[col] = pd.to_datetime(orders[col], utc=True)

n_rows, n_cols = orders.shape
basket_sum = int(orders["basket_minor"].sum())
shipping_sum = int(orders["shipping_fee_minor"].sum())
distance_sum = float(orders["distance_km"].sum())
n_unique_order = int(orders["order_id"].nunique())

csv_path = OUT / "orders_roundtrip.csv"
pq_path = OUT / "orders.parquet"
orders.to_csv(csv_path, index=False)
orders.to_parquet(pq_path, index=False)

csv_back = pd.read_csv(csv_path, dtype=dtype, parse_dates=date_cols)
for col in date_cols:
    csv_back[col] = pd.to_datetime(csv_back[col], utc=True)
pq_back = pd.read_parquet(pq_path)

checks = {
    "sorok": len(csv_back) == n_rows == len(pq_back),
    "oszlopok": list(csv_back.columns) == list(orders.columns) == list(pq_back.columns),
    "basket_minor sum": int(csv_back["basket_minor"].sum()) == basket_sum ==
    ↪  int(pq_back["basket_minor"].sum()),
    "shipping_fee_minor sum": int(csv_back["shipping_fee_minor"].sum()) == shipping_sum ==
    ↪  int(pq_back["shipping_fee_minor"].sum()),
    "distance_km sum": abs(float(csv_back["distance_km"].sum()) - distance_sum) < 1e-6
                    and abs(float(pq_back["distance_km"].sum()) - distance_sum) < 1e-6,
    "order_id uniq": int(pq_back["order_id"].nunique()) == n_unique_order == n_rows,
    "basket int64": str(pq_back["basket_minor"].dtype) == "int64",
}

print("Kontrollok:", checks)
print("CSV bájt:", csv_path.stat().st_size, "Parquet bájt:", pq_path.stat().st_size)
print("Parquet séma:")
print(pq.read_schema(pq_path))
print(orders.dtypes)
```

Futtassuk a projektgyökérből, bash vagy PowerShell munkakönyvtárból:

```
uv run python s029_file_formats.py
```

A futási kimenet a saját gépen mérhető; az alábbi csak szemléltető alak:

```
Kontrollok: {'sorok': True, 'oszlopok': True, 'basket_minor sum': True,
'shipping_fee_minor sum': True, 'distance_km sum': True, 'order_id uniq': True,
'basket int64': True}
CSV bájt: 567890 Parquet bájt: 123456
```

```
Parquet séma:
order_id: string
...
```

A konkrét bájtszámok és a séma pontos sorai a helyi `orders.csv`-ből és a telepített PyArrow-verzióból következnek; a `checks` mindegyikének viszont `True`-nak kell lennie.

8.1.8 Eredmény értelmezése

A kontrollösszeg a CSV-visszaolvasás után is egyezik, mert az egész minor unit oszlopokat `int64`-ként adtuk meg, és a pandas a float64 értékeket olyan szöveggé írta, amelyből ugyanaz a float64 áll vissza. A Parquet esetén a séma a fájlból jön: `basket_minor` fizikailag `int64`, `distance_km` `double`, a dátum `timestamp[us, tz=UTC]`. A `distance_km` összegnél a `1e-6` abszolút tűrés a lebegőpontos ábrázolás miatt indokolt; a sorrend azonos, ezért a hiba ennél jóval kisebb, de a lebegőpontos összeg nem egzakt egész.

A fájlméret-különbség a `priority_flag`, a kategóriák és az ismétlődő dátumrészek tömöríthetőségétől függ. A DataFrame RAM-igénye ettől független: a PyArrow visszaolvasás után a pandas ugyanúgy anyagi oszlopokat tart memóriában. A CSV-betöltésnél a `dtype` szótár nélkül a pandas találgathat; a Parquet-nél ez a találgatás megszűnik.

8.1.9 Feladatok

1. Értelmezés. Az alábbi CSV-sor hány mezőt tartalmaz, és mi a második mező pontos értéke?

```
"A-1","contains ""quotes"" and, comma",123
```

2. Implementálás. Írjuk ki `out/orders_partitioned` könyvtárba a rendeléseket `year` és `month` Hive-partícióval, majd PyArrow `Dataset.scanner` segítségével szűrjük ki a 2025. februári sorokat. Ellenőrző feltétel: a kiszűrt sorok száma egyezzen a `month == 2` pandas-szűrés sorainak számával.

3. Hibadiagnózis. Az alábbi kód `KeyError: 'customer_id'` hibát ad. Mi a hiba oka, és mi a javítás?

```
rows = [{
    "order_id": "A-1",
    "customer": {"customer_id": "C-9", "segment": "business"},
    "items": [{"line_id": 1, "qty": 2}],
}]
pd.json_normalize(rows, record_path=["items"], meta=["customer_id"])
```

4. Megváltozott feltétel. A partner a `distance_km` mezőt szándékosan `float32`-re kerekítve kéri menteni, és `zstd` tömörítést használ. Ismételd meg a minimális példát ezzel a változtatással, és mérd a `distance_km` kontrollösszeg abszolút hibáját az eredeti float64 összeghez képest. Miért nem teljesülhet a `1e-6` tűrés?

8.1.10 Megoldások és indoklás

1. A sor három mezőből áll: `order_id="A-1"`, `note=contains "quotes" and, comma`, `amount=123`. A dupla idézőjel a mező belsejében egyetlen idézőjelet jelent, a vessző a mezőn belül marad, mert a teljes mező idézőjelek között van. A pandas `doublequote=True` beállítása pontosan ezt a szabályt követi.

2. A minimális példa után futtatható kód:

```
import shutil

import pyarrow as pa
import pyarrow.dataset as ds

shutil.rmtree("out/orders_partitioned", ignore_errors=True)
orders_ym = orders.copy()
orders_ym["year"] = orders_ym["order_ts"].dt.year
orders_ym["month"] = orders_ym["order_ts"].dt.month

table = pa.Table.from_pandas(orders_ym, preserve_index=False)
ds.write_dataset(
    table,
    "out/orders_partitioned",
    format="parquet",
    partitioning=["year", "month"],
)

feb = ds.dataset(
    "out/orders_partitioned", format="parquet", partitioning="hive"
).scanner(filter=ds.field("month") == 2).to_table()

expected = int((orders_ym["month"] == 2).sum())
assert len(feb) == expected
print("február sorok:", len(feb), "hónapok:", sorted(feb["month"].unique().to_pylist()))
```

A pruningot a `filter=ds.field("month") == 2` feltétel váltja ki: a motor a `month=2` partíciós mappákat olvassa, a többit kihagyja.

3. A `meta=["customer_id"]` útvonal a legfelső szinten keresi a `customer_id` kulcsot, de az a `customer` objektumon belül van. A helyes útvonal `["customer", "customer_id"]`. A javítás:

```
pd.json_normalize(
    rows,
    record_path=["items"],
    meta=["order_id", ["customer", "customer_id"], ["customer", "segment"]],
)
```

Így a normalizálás a `customer_id` szülőértéket minden tételsorhoz hozzárendeli.

4. Az átalakítás:

```
orders32 = orders.copy()
orders32["distance_km"] = orders32["distance_km"].astype("float32")
orders32.to_parquet("out/orders32.parquet", compression="zstd")
pq_back32 = pd.read_parquet("out/orders32.parquet")
error = abs(float(pq_back32["distance_km"].sum()) - distance_sum)
print("float32 abszolút hiba:", error)
```

A `float32` értékenként körülbelül $2^{-24} \approx 6{\cdot}10^{-8}$ relatív kerekítést visz be, de 2400 érték összegénél az abszolút hiba az összeg nagyságrendjétől függ, és a tapasztalat szerint jóval meghaladja a `1e-6` tűrést. A `zstd` kisebb fájlt adhat a `snappy` alapértelmezésnél, de a kontrollösszegbeli különbséget a típuskerekítés okozza, nem a codec.

8.1.11 Gyakori tévedés ellenpéldával

Veszélyes azt hinni, hogy „minél több partíció, annál gyorsabb”. Ha a 2400 rendelést egyedi `order_id`-ként particionálnánk, körülbelül 2400 fájl keletkezne, átlagosan egy sorral. A pruning kiválasztana egyet, de a fájlrendszerben a sok kis fájl felsorolása, metaadat-olvasása és megnyitása drágább, mint a teljes 2400 sor beolvasása. Ugyanez a hiba kisebb formában a napi partíciónál: 180 nap, 13 sor fájlonként.

A másik gyakori félreértés: „a kisebb Parquet-fájl kevesebb RAM-ot igényel”. A tömörítés a tárolásra és az átvitelre hat. Betöltés után a DataFrame oszlopai memóriában anyagiasulnak: egy 2400 soros `int64` oszlop 19 200 bájt, akár 2 KiB, akár 200 KiB volt a fájl. A fájlméret tervezési jel, a RAM-igény pedig a sorok számából és a fizikai típusokból számolható. Ez a 7.4-ban tanult vetítés és a 7.1-ben tanult sorjelentés közös következménye.

Önellenőrzés: Mit tárol a CSV a típusokból? Miért kell a `meta` útvonalat listaként megadni? Miért nem egyenlő a tömörített fájlméret a betöltött DataFrame memóriaigényével?

A Parquet oszlopos fájltárolása és az Arrow oszlopos memóriareprezentációja nem garantál teljes üzleti sémahelyességet. Az időzónás dátumok, decimális típusok és kategóriák oda-vissza alakítását külön tesztekkel érdemes ellenőrizni.

8.1.12 Rövid összefoglaló

A CSV szöveges és séma nélküli; az idézési szabályokat és a típusokat a betöltőnek külön kell ismernie. A JSON normalizálás a beágyazott listákból lapos táblát készít, a szülőmezőket `meta` útvonalakkal ismételve. A Parquet oszlopalapú, a fájlban tárolt sémával és kétrétegű tömörítéssel; az egész minor unit kontrollösszeg bitazonos, a float64 összegnél tűrés kell. A Hive-partíció pruninggal kihagyja a nem releváns mappákat, de a túlzott aprózás sok kis fájllal ront. A következő alfejezetben a DuckDB-t használjuk ugyanezen Parquet-állomány közvetlen SQL-lekérdezésére.

8.2 DuckDB: helyi analitikus SQL szerver nélkül

Belépő · [BŐVÍTŐ MODUL] · szint: alap · idő: 45 perc · eszköz: Parquet, Arrow, DuckDB, Polars

Előfeltétel: 8.1. alfejezet · Célkészség: DuckDB: helyi analitikus SQL szerver nélkül · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

8.2.1 Cél és konkrét helyzet

A Northstar `orders` és `order_items` tábláit az előző alfejezetben megtisztítottuk, és a sémát Parquet-ben is rögzítettük. Most ugyanazokat a rendelésszintű mutatókat számoljuk ki DuckDB-vel, amelyeket a 7. fejezetben SQLite-tal is használtunk: rendelések száma, tételek száma, kosárérték és szállítási díj összege. A megfigyelési egység a rendelés, illetve a rendelési tétel; a cél, hogy Pythonból és közvetlen fájlból azonos eredményt kapjunk, explicit DuckDB-kapcsolattal, majd SQLite kontrollértékekkel ellenőrizzünk. Előzetes becslés: a tiszta adatban 2400 rendelés és 4796 tétel található, ezért a belső join sorszáma 4796.

8.2.2 Fogalmi magyarázat

`connect`. A DuckDB beágyazott adatbázis: nincs külön szerverfolyamat, a motor a hívó Python-folyamatban fut. A `duckdb.connect(":memory:")` egy átmeneti, csak az adott kapcsolatban élő adatbázist nyit; a `duckdb.connect("northstar.duckdb")` fájlba írja az adatot. Az explicit kapcsolat azért fontos, mert így nem a DuckDB rejtett alapértelmezett kapcsolatát használjuk, és mi irányítjuk, mely Python-objektumok melyik kapcsolatba regisztrálódnak.

CSV/Parquet scan. A `read_parquet` és `read_csv_auto` táblafüggvények a fájlt közvetlenül, pandas köztes DataFrame nélkül olvassák be a lekérdezésbe. A Parquet a fájlban tárolt sémát használja; CSV-nél a DuckDB mintavételezéssel becsli a típusokat. Az `orders_raw.csv` hiányzó `distance_km` értékei például `NULL`-ként jelennek meg, nem nullává alakított távolságként. A scan nem tölti be az egész fájlt a Python heapbe: a motor a saját pufferében dolgozik, és csak a kért oszlopokat, illetve a szükséges sorokat olvassa.

Relation. A `con.sql("SELECT ...")` hívás nem feltétlenül anyagiasítja azonnal az eredményt: egy `DuckDBPyRelation` objektumot ad vissza. Ez a reláció a lekérdezés terve, amely a relációs algebra kifejezéseként írja le, mit szeretnénk számolni. A tényleges végrehajtás akkor történik, amikor az eredményt lekérjük, például `.fetchall()`, `.df()`, `.arrow()` vagy `.show()` hívással. A Relation láncolható is: szűrés, vetítés és aggregálás építhető rá, hasonlóan ahhoz, ahogy a Polars lazy tervét fogjuk a következő alfejezetben használni.

Eredménykonverzió. A leggyakoribb kimenetek:

- `.df()`: pandas DataFrame-et ad; ez az adatot a Python heapbe másolja.
- `.arrow()`: PyArrow táblát ad, amely a 8.1. alfejezetben tanult oszlopalapú ábrázolást használja.
- `.fetchall()`: Python tuple-listát ad, ami kis eredmények debugolásához kényelmes.

Ha csak a séma érdekel, a `DESCRIBE SELECT ...` lekérdezés vagy a Relation oszlopneveinek vizsgálata nem anyagiasítja a teljes eredményt.

Szál- és memóriakeret. A DuckDB a folyamaton belül több szálat használhat egy lekérdezés párhuzamos végrehajtására. A `SET threads TO 4` felső korlátot ad a szálhasználatra, a `SET memory_limit='2GB'` pedig a motor puffermemóriáját korlátozza. Ez a korlát nem foglalja magában a Python oldali DataFrame-eket: ha `rel.df()`-et hívunk, a pandas másolat további heapmemóriát fogyaszt. A fájlalapú `.duckdb` állomány egyszerre jellemzően egy író folyamatnak való; többfelhasználós írási tranzakciókra nem ez a megfelelő eszköz.

8.2.3 Teljes minimális munkapélda

Két apró kézi táblával ellenőrizzük a `connect`, a regisztráció, a Relation és a konverzió együttműködését. Futtatás a projektgyökérből: `uv run python mini_duckdb.py` (Linux/macOS terminál; PowerShellben ugyanez a `uv` parancs, csak az útvonalírás térhet).

```
import pandas as pd
import duckdb

mini_orders = pd.DataFrame({
    "order_id": ["A", "B"],
    "shipping_fee_minor": [300, 0],
})
mini_items = pd.DataFrame({
    "order_id": ["A", "A", "B"],
    "line_total_minor": [1000, 2000, 500],
})
```

```
con = duckdb.connect(":memory:")
con.register("mini_orders", mini_orders)
con.register("mini_items", mini_items)

rel = con.sql("""
SELECT
    (SELECT count(*) FROM mini_orders)          AS n_orders,
    (SELECT count(*) FROM mini_items)           AS n_items,
    (SELECT sum(shipping_fee_minor)
       FROM mini_orders)                        AS shipping_ok,
    (SELECT sum(o.shipping_fee_minor)
       FROM mini_items i
       JOIN mini_orders o USING (order_id))     AS shipping_dup
""")

print(type(rel).__name__)
df = rel.df()
print(df.to_string(index=False))
print("df.shape =", df.shape)
```

A kézi táblázatból elméletileg várt kimenet:

```
DuckDBPyRelation
 n_orders  n_items  shipping_ok  shipping_dup
        2        3          300           600
df.shape = (1, 4)
```

Az utolsó sort a szövegíró nem futtatta; a kézi számításból következik. A `shipping_dup` azért 600, mert a három tételsor közül az A rendelés kétszer szerepel, és a 300 centes szállítási díj kétszer adódik az összeghez.

8.2.4 Eredmény értelmezése

A kis példa a 7.2. alfejezet join-kardinalitási leckéjét ismétli: a belső join sorszáma a tételek számával egyezik, ezért a rendelésszintű oszlopok ismétlődnek. A mutatók definíciója skalár eredményt ad egy sorban:

- $M_1 = |\{\text{order_id}\}| = 2$,
- $M_2 = \sum_i \text{line_total_minor}_i = 1000 + 2000 + 500 = 3500$,
- $M_3 = \sum_j \text{shipping_fee_minor}_j = 300 + 0 = 300$.

A joined sorból számolt $M_3' = \sum_{k=1}^{3} s_{o(k)} = 300 + 300 + 0 = 600$ nem ugyanaz a mutató. A 4.2. alfejezetben tanult debuggeres shape-ellenőrzés itt is hasznos: a `rel.df().shape` legyen `(1, 4)`, mert egy aggregált sor négy skalár oszlopot tartalmaz. Mielőtt nagy fájlra váltunk, ezt a kézi kontrollt érdemes megismételni.

A laborban a Northstar fájlokkal dolgozunk. A DuckDB-függőséget először a projektbe kell venni:

```
# Linux/macOS terminál, a lab projekt gyökérkönyvtárából
uv add duckdb
uv lock
uv sync --locked
```

Ezután a DuckDB közvetlen Parquet-scanjeit összehasonlítjuk egy SQLite kontrollal. A kód a szövegírás során nem futott; a nyomtatott pénzösszegek a helyi manifestből adódnak, ezért itt nem adunk előre konkrét számot.

```
import sqlite3
import time

import pandas as pd
import duckdb

t0 = time.perf_counter()

# DuckDB: közvetlen fájlból, explicit kapcsolattal.
con = duckdb.connect(":memory:")
con.execute("SET threads TO 4")
con.execute("SET memory_limit='2GB'")

duck_sql = """
SELECT
    (SELECT count(*) FROM read_parquet('data/orders.parquet'))  AS n_orders,
    (SELECT count(*) FROM read_parquet('data/order_items.parquet')) AS n_items,
    (SELECT sum(basket_minor) FROM read_parquet('data/orders.parquet')) AS basket_sum,
    (SELECT sum(shipping_fee_minor) FROM read_parquet('data/orders.parquet')) AS shipping_sum,
    (SELECT sum(i.line_total_minor)
       FROM read_parquet('data/order_items.parquet') i
       JOIN read_parquet('data/orders.parquet') o
         ON i.order_id = o.order_id) AS line_total_joined
"""
duck_rel = con.sql(duck_sql)
duck_result = duck_rel.df().iloc[0].to_dict()
t_duck = time.perf_counter() - t0

# SQLite kontroll: a CSV-ket pandas betölti, az stdlib sqlite3 végrehajtja.
sqlite_con = sqlite3.connect(":memory:")
orders = pd.read_csv("data/orders.csv", parse_dates=["order_ts"])
items = pd.read_csv("data/order_items.csv")
orders.to_sql("orders", sqlite_con, index=False)
items.to_sql("order_items", sqlite_con, index=False)

sqlite_sql = """
SELECT
    (SELECT count(*) FROM orders)                                   AS n_orders,
    (SELECT count(*) FROM order_items)                              AS n_items,
    (SELECT sum(basket_minor) FROM orders)                          AS basket_sum,
    (SELECT sum(shipping_fee_minor) FROM orders)                    AS shipping_sum,
    (SELECT sum(i.line_total_minor)
       FROM order_items i
       JOIN orders o ON i.order_id = o.order_id)                    AS line_total_joined
"""
sqlite_row = sqlite_con.execute(sqlite_sql).fetchone()
sqlite_result = dict(zip(
    ["n_orders", "n_items", "basket_sum", "shipping_sum", "line_total_joined"],
    sqlite_row))
t_total = time.perf_counter() - t0

print(duck_result)
print(sqlite_result)
print(f"duckdb_wall_s={t_duck:.3f}, total_wall_s={t_total:.3f}")

for key in sqlite_result:
    duck_val = duck_result[key]
    sqlite_val = sqlite_result[key]
    if sqlite_val is None:
        assert duck_val is None, key
    else:
        assert int(duck_val) == int(sqlite_val), key
print("kontroll: a DuckDB és SQLite mutatók egyeznek")
```

Ha még nincs Parquet-állomány, a 8.1. alfejezet alapján készítsük el, vagy ideiglenesen cseréljük a `read_parquet('data/orders.parquet')` kifejezéseket `read_csv_auto('data/orders.csv')`-re. A `n_orders=2400` és `n_items=4796` a szerződésből várható; a `basket_sum` és a `line_total_joined` egyezése a tételszintű kosárösszeg invariant, egész minor unitban. A `duckdb_wall_s` egyszeri, a saját gépen mért tájékoztató idő; teljesítmény-összehasonlításhoz gép, adatméret, ismétlésszám és azonos input kellene.

8.2.4.1 Rögzített lekérdezés, EXPLAIN ANALYZE és hash-elt mérés

A DuckDB-példa teljesítménye nem általánosítható, ezért a mérési jegyzék minden fontos körülményt rögzít: a DuckDB-verziót, a bemeneti fájl SHA-256-ját, a lekérdezés szövegét és hash-ét, az `ORDER BY`-t, a hideg és a meleg futások idejét, valamint a kimenet hash-ét. A lekérdezés:

```
SELECT priority_flag,
       COUNT(*)          AS n_orders,
       SUM(basket_minor) AS basket_sum_minor,
       AVG(distance_km)  AS distance_mean_km
FROM read_csv_auto('lab/data/orders.csv')
GROUP BY priority_flag
ORDER BY priority_flag;
```

Az `ORDER BY priority_flag` nem kozmetika: nélküle a sorok sorrendje nem garantált, így a kimenet hash-e két futás között eltérhet. A végrehajtási tervet és a tényleges időt az `EXPLAIN ANALYZE` adja:

```
import duckdb
con = duckdb.connect()
con.execute("SET enable_object_cache=false")
plan = con.execute("EXPLAIN ANALYZE " + QUERY, ["lab/data/orders.csv"]).fetchall()
print("\n".join(str(row[-1]) for row in plan))
```

A mért referencia (DuckDB 2.0.0, Python 3.12.3, Linux x86-64; `repeats=5`, `warmup=1`):

Mező	Érték
hideg futás	0.018115 s
meleg futások	0.017891, 0.017567, 0.018346, 0.017634, 0.017807 s
meleg medián	0.017807 s
kimenet sorai	`0, 1179, 47122202, 922.049...`; `1, 1221, 50260947, 913.788...`
kimenet SHA-256	`19d9902e7c090f9c3efeb7b9d620a7e74df3d19f5ca0e641162`

A „hideg” itt a folyamat első, előmelegítés nélküli futása; az operációs rendszer page cache-ét nem ürítjük, ezért ez nem laboratóriumi hidegindítás. A hideg és meleg érték közti különbség ezen a kis fájlon kicsi; nagyobb adaton és lassabb háttértáron nagyobb lehet. A `tests/test_duckdb_bench.py` a lekérdezés logikai eredményét, az `ORDER BY` miatti stabil hash-t, a DuckDB-verzió és a bemeneti hash jelenlétét, valamint az `EXPLAIN ANALYZE` kimenetét ellenőrzi.

A `lab/duckdb_bench.py --output` kapcsolóval JSON-jegyzéket ír; a jegyzék a `run_manifest.json` mintájára használható, de a teljes futtatási szerződést nem helyettesíti.

8.2.5 Feladatok

1. **Értelmezés.** A mini példában a `rel` típusa `DuckDBPyRelation`. Mit jelent az, hogy a `con.sql(...)` hívás nem azonnal anyagiasít? Melyik hívás kényszeríti ki a végrehajtást? Adj egy módot, amellyel a reláció sémáját ellenőrizni tudod anélkül, hogy minden sort a Python heapbe másolnál.

2. **Implementálás.** Futtasd a fenti laborpéldát a `data/orders.parquet` és `data/order_items.parquet` állományokra. Írasd ki a `n_orders`, `n_items`, `basket_sum`, `shipping_sum` értékeket, és ellenőrizd, hogy a DuckDB és a SQLite eredmény mind az öt mutatóra egyezik. A megoldásban add meg a használt DuckDB-verziót is, például a `duckdb.__version__` kiíratásával.

3. **Hibakeresés.** Egy kolléga a következő kódot futtatja:

```
con.register("orders", pd.read_csv("data/orders.csv"))
con.register("order_items", pd.read_csv("data/order_items.csv"))
bad = con.sql("""
SELECT sum(o.shipping_fee_minor) AS shipping
FROM order_items i JOIN orders o ON i.order_id = o.order_id
""").df()
```

A kapott `shipping` nagyobb, mint a `SELECT sum(shipping_fee_minor) FROM orders` eredménye. A 7.2. alfejezet join-kardinalitási szabályai alapján magyarázd meg, miért, és adj egy javított lekérdezést.

4. **Megváltozott feltétel.** Számold ki DuckDB-vel a `priority_flag = 1` rendelésekhez tartozó `n_orders`, `n_items` és `shipping_sum` mutatókat úgy, hogy a rendelésszintű szűrés a tételekre is érvényesüljön. Ellenőrizd az invariánst: `n_items` egyenlő a priority rendelésekhez tartozó tételek számával, és a szűrt `shipping_sum` nem tartalmazza a `priority_flag = 0` rendelések díját.

8.2.6 Megoldások és indoklás

1. A `con.sql(...)` Relation objektumot ad: a lekérdezés tervét, nem az eredmény Python-másolatát. Az anyagiasítást a `.df()`, `.fetchall()`, `.fetchone()`, `.arrow()` vagy `.show()` hívás váltja ki. A séma ellenőrzéséhez a Relation oszlopneveit (`rel.columns`) vagy egy `DESCRIBE SELECT ...` lekérdezést használhatunk; a `rel.limit(5).df()` pedig biztonságos kis előnézetet ad anélkül, hogy a teljes eredményt a Python heapbe töltenénk.

2. A `uv add duckdb` után a fenti kód futtatható. A szerződésből `n_orders=2400` és `n_items=4796` várható; a `basket_sum` és a `line_total_joined` egész minor unitban egyezik, a két végrehajtó értékei mind az öt mutatóra azonosak. A DuckDB-verziót a `duckdb.__version__` rögzíti; a lockfájl a pontos csomagverziót is tartalmazza. Ha Parquet nincs, a `read_csv_auto` útváltozat a CSV-n is ugyanazt a mutatót adja.

3. A belső join a tételek szerint 4796 sort ad; minden rendelés annyiszor szerepel, ahány tétele van. A `shipping_fee_minor` rendelésszintű oszlop minden tételsorban megismétlődik, ezért a naiv `sum` a rendelésenkénti tételszámmal súlyozza a szállítási díjat. A mini példában a 300 centes A rendelés kétszer szerepel, így 600 lesz. A javítás: a szállítási díjat a rendelés táblából, join nélkül kell összegezni:

```
SELECT sum(shipping_fee_minor) AS shipping FROM orders
```

Vagy előaggregálással egy rendelésenként egy reprezentáns sort kell venni, például `(order_id, shipping_fee_minor)` egyedi párokból.

4. A szűrt mutatók:

```
SELECT
    count(*) AS n_orders,
    (SELECT count(*)
       FROM order_items i
       JOIN orders o ON i.order_id = o.order_id
      WHERE o.priority_flag = 1) AS n_items,
    sum(shipping_fee_minor) AS shipping_sum
FROM orders
WHERE priority_flag = 1
```

Az invariáns teljesül, mert a belső select a priority rendelésekhez tartozó tételeket számolja, a külső `WHERE` pedig a rendelésszintű `shipping_fee_minor` összeget szűkíti ugyanarra a halmazra. A szűrt `shipping_sum` így nem tartalmazza a `priority_flag = 0` rendelések díját.

8.2.7 Gyakori tévedés és ellenpélda

A beágyazott analitikus adatbázis nem minden többfelhasználós tranzakciós szerver helyettesítője. Ha két párhuzamos folyamat ugyanarra a `.duckdb` fájlra ír, az egyik zárolási hibát vagy sérült állapotot kaphat; egy szerveres OLTP-rendszer, például egy PostgreSQL, a tranzakciókat, a felhasználókezelést és az egyidejű írásokat más mechanizmussal kezeli. A DuckDB-t ezért kutatói folyamaton belüli, fájlok feletti analitikus lekérdezésre használjuk, nem alkalmazásszervernek. A 7.2. alfejezetből ismert, hogy a join előtti kulcsegyediség és a join utáni sorváltozás ellenőrzése akkor is kötelező, ha a motor gyors. Önellenőrzés: mielőtt új adatforrásra váltanál, futtasd a mini példát, és győződj meg róla, hogy a `shipping_dup` pontosan kétszerese a `shipping_ok`-nak.

Rendelésszintű aggregációnál a rendeléshez tartozó `shipping_fee_minor` összege megsokszorozódhat a tételsorokkal való összekapcsolás után. A tételsorokat előbb `order_id` szerint aggregáljuk, és csak azután kapcsoljuk az egyedi rendelésekhez.

8.2.8 Rövid összefoglaló

A DuckDB beágyazott analitikus SQL-motort ad: explicit `duckdb.connect` kapcsolattal, szerver nélkül olvasunk CSV-t és Parquet-et. A `con.sql` Relation objektumot ad, amely a lekérdezési terv; a tényleges anyagiasítást a `.df()`, `.fetchall()` vagy `.arrow()` hívás váltja ki. A fájlscan elkerüli a pandas köztes DataFrame-et, de a `.df()`-felé konvertálás a Python heapben további memóriát fogyaszt. A Northstar mutatók DuckDB-vel és SQLite-tal azonos SQL-definícióval számolhatók; a kontroll a join-kardinalitás miatt a rendelésszintű oszlopokra külön figyelmet igényel. A következő alkalmazás: a Polars lazy tervének és a DuckDB Relation viselkedésének összehasonlítása.

8.3 Polars lazy terv és streaming korlát

> **Belépő · [BŐVÍTŐ MODUL] · szint: alap · idő: 45 perc · eszköz: Parquet, Arrow, DuckDB, Polars**
>
> Előfeltétel: 8.2. alfejezet · Célkészség: Polars lazy terv és streaming korlát · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

Ez az alfejezet bővítő egység: a tanulási főút pandas + SQL marad, de a Polars lazy terve fontos ellenpont, mert a tervépítést és a végrehajtást különválasztja.

8.3.1 Cél és konkrét helyzet

Egy data scientist a Northstar laborban ügyfélenkénti, a 2025. május 1. előtti rendelésekből számolt tétel-összértéket akar előállítani. Bemenet a `lab/data/orders.csv` (2400 rendelési sor) és a `lab/data/order_items.csv` (4796 tételsor). A megfigyelési egység itt a rendelési tétel; az aggregáció egysége a `customer_id`. A kimenet fájlba írt Parquet-tábla, oszlopai: `customer_id`, `item_total_minor`, `item_rows`, `order_count`. Az elvárt ellenőrzés: a tételösszegek összege megegyezik a cutoff előtti rendelések `basket_minor` értékeinek összegével, és a kimenet sorainak száma legfeljebb 300, mert ennyi ügyfél van a `customers.csv`-ben.

Mielőtt kódot írsz, adj durva becslést: a rendelések 2025 első 180 napjára esnek, a cutoffig 120 nap telik el, ezért egyenletes eloszlást feltételezve körülbelül `2400 * 120/180 = 1600` rendelés lehet a cutoff előtt; a pontos számot a futtatás adja. Az ellenőrző invariant: `sum(item_total_minor) == sum(basket_minor)` a leszűrt rendelésekre.

8.3.2 Lusta terv, collect és sink

A Polars kifejezéseiben a `pl.scan_csv(...)` vagy `pl.scan_parquet(...)` nem olvas be adatot. Egy `pl.LazyFrame` objektumot ad, amely leírja a forrást és az elvégzendő lépéseket. A `select`, `filter`, `group_by` és `join` hívások csak új csomópontokat fűznek ehhez a tervhez; a tényleges beolvasást és számítást a `collect()` hívás váltja ki. A `collect()` eredménye egy hagyományos `pl.DataFrame` a Python heapben.

A `scan_parquet` ugyanezt a felületet adja Parquet-forrásra. A `read_parquet` ezzel szemben azonnal `DataFrame`-et ad, tehát eager módon betölti a fájlt. A `sink_parquet` a lazy tervet közvetlenül Parquet-fájlba anyagiasítja, anélkül hogy a teljes eredményt `DataFrame`-ként a heapben tartaná; ez akkor hasznos, ha a kimenet következő lépésben egy másik eszköz bemenete. Az `explain()` nem futtat semmit, csak a terv szöveges formáját mutatja.

A Polars lazy motorja a terv optimalizálásakor predikátum- és vetületletolást végez. A `filter(pl.col("order_ts") < cutoff)` a forrás közelébe, a `select` a lehető legkorábbra kerülhet, így a feldolgozó kevesebb sort és oszlopot mozgat. A streaming motor ezen felül akkor is haladhat batch-ekben, ha a teljes bemenet nem fér a memóriába, de a csoportosítás állapota továbbra is adatfüggő; ezt a gyakori tévedés részben fejtjük ki.

8.3.3 Szűrés korábbra hozása: algebrai azonosság

Jelölje O a rendelések tábláját, I a tételek tábláját, $\bowtie$ az `order_id`-re vett inner joint, σ_p pedig a p predikátummal történő sorszűrést. Ha p csak az O oszlopait használja, akkor

$$\sigma_{o.order_ts<c}(O \bowtie I) \equiv \sigma_{o.order_ts<c}(O) \bowtie I,$$

ahol c a cutoff időbélyeg, $o.order_ts$ a rendelés időpontja. A bal oldalon előbb a join, utána a szűrés; a jobb oldalon a szűrés a join elé kerül. A két oldal sorai azonosak, de a közbülső méret különbözik: a bal oldal közbülső táblája minden rendelés összes tételsorát tartalmazza, a jobb oldalé csak a cutoff előtti rendelések tételeit.

Kis kézi példa: legyen O három rendelés:

- A: `2025-01-01`, `basket_minor=100`
- B: `2025-05-02`, `basket_minor=200`
- C: `2025-04-30`, `basket_minor=300`

I tételei: A-hoz 40 és 60, B-hez 200, C-hez 100 és 200. A `join → filter` út előbb 5 tételsort állít elő, majd a szűrés 4 sort hagy meg. A `filter → join` út már a szűrt 2 rendeléssel lép a joinba, és közvetlenül 4 tételsort ad. A tételösszeg mindkét esetben `400`, azaz a cutoff előtti kosárösszegek összege `100 + 300`. A Polars `explain()` kimenete várhatóan a `FILTER` lépést a `JOIN` elé helyezi, még akkor is, ha a kódban a join előtt csak a `base` filterét írtuk.

8.3.4 Teljes minimális példa a Northstar laborral

A labor CSV-t használ, ezért a kód `scan_csv`-t hív; a `scan_parquet` azonos felületet ad, ha a forrás Parquet. Telepítsd a bővítő csomagokat a labor környezetébe, majd futtasd a scriptet:

```
# munkakönyvtár: lab
uv add polars pyarrow
uv run python s031_example.py
```

A következő kódot Polars 2.1.0-val és pyarrow 26.0.0-val ellenőriztük a labor adatain (a kimenet `M=297` sor); a számok a helyi generált adatokból származnak, és a `lab/data/manifest.json` hash-ellenőrzése mellett érvényesek.

```
# lab/s031_example.py
from pathlib import Path
import polars as pl

DATA = Path("data")
OUT = Path("artifacts")
OUT.mkdir(exist_ok=True)

CUTOFF = (
    pl.lit("2025-05-01T00:00:00+00:00")
    .str.to_datetime("%Y-%m-%dT%H:%M:%S%:z")
)


def build_customer_summary(
    orders: pl.LazyFrame, items: pl.LazyFrame
) -> pl.LazyFrame:
    base = (
        orders.filter(pl.col("order_ts") < CUTOFF)
        .select(["order_id", "customer_id", "basket_minor"])
    )
    return (
        base.join(items, on="order_id", how="inner")
        .group_by("customer_id")
        .agg(
            pl.col("line_total_minor").sum().alias("item_total_minor"),
            pl.len().alias("item_rows"),
            pl.col("order_id").n_unique().alias("order_count"),
        )
    )


def main() -> None:
    orders = pl.scan_csv(DATA / "orders.csv", try_parse_dates=True)
    items = pl.scan_csv(DATA / "order_items.csv")

    lazy_summary = build_customer_summary(orders, items)
    print(lazy_summary.explain())

    result = lazy_summary.collect()
    print(result.shape)
```

```
    # eager kontroll: ugyanaz a művelet azonnali DataFrame-ekkel
    orders_e = pl.read_csv(DATA / "orders.csv", try_parse_dates=True)
    items_e = pl.read_csv(DATA / "order_items.csv")
    result_e = build_customer_summary(orders_e.lazy(), items_e.lazy()).collect()
    assert result_e.sort("customer_id").equals(result.sort("customer_id")), (
        "eager és lazy eredmény nem egyezik"
    )

    order_total = (
        orders.filter(pl.col("order_ts") < CUTOFF)
        .select(pl.col("basket_minor").sum())
        .collect()
        .item()
    )
    item_total = result.select(pl.col("item_total_minor").sum()).item()
    assert order_total == item_total, (order_total, item_total)
    print(f"invariant ok: {item_total}")

    lazy_summary.sink_parquet(OUT / "s031_customer_summary.parquet")
    reloaded = pl.read_parquet(OUT / "s031_customer_summary.parquet")
    assert reloaded.sort("customer_id").equals(result.sort("customer_id"))

    streamed = lazy_summary.collect(engine="streaming")
    assert streamed.sort("customer_id").equals(result.sort("customer_id"))


if __name__ == "__main__":
    main()
```

A `%:z` formátum a `+00:00` időeltolódást értelmezi a cutoff kifejezésben. A `try_parse_dates=True` hatására az `order_ts` Datetime típusú lesz; ha a telepített Polars szerint nem ismeri fel a mezőt, a hibaüzenet ezt jelzi, és a datetimeszerződést külön ellenőrizni kell. A `result.shape` várhatóan `(M, 4)`, ahol `M <= 300`; az ellenőrzött futásban `(297, 4)`.

A kód a Polars 1.25/2.x API-ját használja: a streaming motort a `collect(engine="streaming")` kapcsoló választja ki. A korábbi `collect(streaming=True)` argumentumot a Polars 1.25-ben deprecated, a 2.0-ban eltávolította; a régi példa `ArgumentRemovedError`-t ad. A telepített verziót a projekt lockja rögzíti; ha más verzió települ, a Polars lazy útmutató az ellenőrzési kiindulópont.

8.3.5 Eredmény értelmezése és ellenőrzése

A lazy `explain` nem bizonyíték arra, hogy a szűrés korábbra került; a tényleges tervet a Polars saját kimenete mutatja. A numerikus kontrollt a fenti invariant adja: a szűrt rendelések `basket_minor` összege megegyezik a tételösszegek összegével. Ez egy one-to-many join után sem duplázódik, mert az összegzett oszlop a tételtáblából származik, a `basket_minor` pedig csak a kontrollhoz, a rendelés egy-egy sorából kerül összegzésre.

A `reloaded.sort("customer_id").equals(result.sort("customer_id"))` a Parquet-visszaolvasás sémáját és értékeit ellenőrzi. A `DataFrame.equals` soronként, pozíció szerint hasonlít, ezért a `group_by("customer_id")` implicit kimeneti sorrendje miatt a nyers `equals` hamis negatívot adhat: a Polars 2.1.0 ellenőrzött futásában a lazy és az eager eredmény tartalmilag azonos volt, de a csoportok sorrendje eltért, ezért a nyers `equals False`, a `customer_id` szerint rendezett összehasonlítás `True` lett. Ha a sorrend nem része a szerződésnek, kulcsra rendezve vagy kulcs-összevetéssel hasonlíts; ha a sorrend igen, a terv végén explicit `sort` legyen. Az eager és lazy eredmény egyezése azt mutatja, hogy a lazy terv nem változtatta meg a

szemantikát, csak a végrehajtás sorrendjét. A memóriaigény itt nem mért: a pandasos és eager DataFrame-ek a teljes CSV-t betöltik, míg a lazy scan elvileg folyamatosan olvashat, de ennek igazolásához valós `tracemalloc` vagy peak-RSS mérés, rögzített gép, adatméret és ismétlésszám szükséges; ezt a feladatban kérjük számon.

8.3.6 Feladatok

1. **Értelmezés.** Magyarázd el, mi a különbség a `pl.scan_parquet("orders.parquet")` és a `pl.read_parquet("orders.parquet")` hívás között. Melyik hívás vált ki tényleges fájl-olvasást? Hogyan ellenőriznéd a típusukból, hogy a terv még nem futott?

2. **Implementálás.** Készíts lazy tervet, amely a 2025. április 1. előtti rendelésekre régiónként kiszámítja az egyedi rendelések számát és a tétel-összértéket. Bemenet: `lab/data/orders.csv`, `lab/data/order_items.csv`, `lab/data/customers.csv`. A kimenetet írd a `lab/artifacts/s031_region.parquet` fájlba, és ellenőrizd, hogy a visszaolvasott táblának legfeljebb 3 sora van, a régiók halmaza pedig `{"north", "central", "south"}`.

3. **Hibadiagnózis.** Adott a következő tervrészlet:

```
wrong = (
    orders.filter(pl.col("order_ts") < CUTOFF)
    .join(items, on="order_id", how="inner")
    .group_by("order_id")
    .agg(
        pl.col("line_total_minor").sum().alias("item_total_minor"),
        pl.col("shipping_fee_minor").sum().alias("shipping_fee_minor"),
    )
)
```

   Miért hibás az így számolt `shipping_fee_minor`? Javítsd a tervet úgy, hogy a szállítási díj rendelésenként pontosan egyszer szerepeljen, és a `line_total_minor` összege továbbra is tételszintről származzon.

4. **Megváltozott feltétel.** Tegyük fel, hogy a `customers.csv` 10 millió ügyfelet tartalmaz, és ugyanezt a `group_by("customer_id")` aggregációt futtatod lazy módon. Miért nem garantálja a lazy API önmagában az állandó memóriaigényt? Nevezz meg legalább egy műveletet vagy adatfüggő állapotot, amely a memóriát növelheti, és írd le, milyen méréssel döntenéd el, hogy a `sink_parquet` vagy a `collect` a megfelelő kimeneti forma.

8.3.7 Megoldások és indoklás

1. A `scan_parquet` `pl.LazyFrame`-et ad, amely a fájlt csak tervként hivatkozza; a `read_parquet` `pl.DataFrame`-et ad, tehát azonnal beolvassa a fájlt. A tényleges I/O-t a lazy esetben a `collect()` váltja ki. A típusellenőrzés: `isinstance(obj, pl.LazyFrame)` igaz a scan, hamis a read esetén; a `collect()` után a típus `pl.DataFrame`. Az `explain()` nem olvas fájlt, így önmagában nem bizonyíték a futtatásra, de megmutatja a tervezett lépéseket.

2. Egy lehetséges megoldás:

```
cutoff = (
    pl.lit("2025-04-01T00:00:00+00:00")
    .str.to_datetime("%Y-%m-%dT%H:%M:%S%:z")
)
```

```
orders = pl.scan_csv("data/orders.csv", try_parse_dates=True)
items = pl.scan_csv("data/order_items.csv")
customers = pl.scan_csv("data/customers.csv")

pre = orders.filter(pl.col("order_ts") < cutoff)

with_region = (
    pre.join(
        customers.select(["customer_id", "region"]),
        on="customer_id",
        how="inner",
    )
    .select(["order_id", "region"])
)

summary = (
    with_region.join(items, on="order_id", how="inner")
    .group_by("region")
    .agg(
        pl.col("order_id").n_unique().alias("order_count"),
        pl.col("line_total_minor").sum().alias("item_total_minor"),
    )
    .sort("region")
)

summary.sink_parquet("artifacts/s031_region.parquet")

reloaded = pl.read_parquet("artifacts/s031_region.parquet")
assert set(reloaded["region"].to_list()) == {"north", "central", "south"}
assert len(reloaded) <= 3

# invariant kontroll
order_count_expected = (
    pre.select(pl.col("order_id").n_unique()).collect().item()
)
assert reloaded["order_count"].sum() == order_count_expected

item_total_expected = (
    pre.join(items, on="order_id", how="inner")
    .select(pl.col("line_total_minor").sum())
    .collect()
    .item()
)
assert reloaded["item_total_minor"].sum() == item_total_expected
```

Az invariáns: a regionális rendelésszámok összege a cutoff előtti egyedi rendelések száma, a tételösszegek összege pedig a cutoff előtti tételek `line_total_minor` összege.

3. A hiba oka: a `shipping_fee_minor` rendelésenként egyetlen érték, de a join után minden tételsorban ismétlődik, ezért a `pl.col("shipping_fee_minor").sum()` a rendelés tételszámával megszorozza az értéket. Ez ugyanaz a one-to-many csapda, amelyet a szállítási díj kapcsán a 4.3 modulban már elkerültünk. A javítás: a tételösszeget külön csoportosítással állítsd elő, majd joinold a rendelésszintű oszlopokhoz:

```
fixed = (
    orders.filter(pl.col("order_ts") < CUTOFF)
    .join(items, on="order_id", how="inner")
    .group_by("order_id")
    .agg(pl.col("line_total_minor").sum().alias("item_total_minor"))
    .join(
        orders.filter(pl.col("order_ts") < CUTOFF)
        .select(["order_id", "shipping_fee_minor"]),
```

```
            on="order_id",
            how="inner",
        )
    )
```

Itt a `shipping_fee_minor` már a rendelés egy-egy sorából kerül a joinba, így minden `order_id`-hoz pontosan egyszer jelenik meg.

4. A `group_by("customer_id")` aggregáció a csoportok kulcsait és a részösszegeket hash-táblában tartja. Ha a kulcsok száma 10 millió, az állapot mérete a csoportok számával arányos, ezért a csúcsigény meghaladhatja a rendelkezésre álló RAM-ot; a lazy API ettől nem véd. A `sink_parquet` a kimenetet közvetlenül fájlba írhatja, így a Python heapben nem kell a teljes eredmény-DataFrame-et tartani, de a csoportállapot akkor is adatfüggő marad. Megfelelő döntéshez mérd a peak RSS-t vagy `tracemalloc`-ot, rögzítsd a gép, az input méretét és az ismétlésszámot, és hasonlítsd össze a `collect` és `sink_parquet` futását ugyanazon a terhelésen.

8.3.8 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés: *„ha a kód lazy, akkor automatikusan streaming, tehát alacsony a memóriaigény."* Ez hamis. A lazy API csak a tervépítést választja el a végrehajtástól; a memóriát a futtatandó művelet és az adatfüggő állapot határozza meg. A `group_by` sok csoportkulccsal, a nem streaming módon futó rendezés vagy egy nagy join rács akkor is növelheti a csúcsigényt, ha a terv `explain()` kimenete szép. Ellenpélda: 10 millió egyedi `customer_id` esetén a `group_by(...).agg(pl.len())` csoportállapota önmagában nagy lehet, miközben a terv ugyanúgy néz ki, mint a 300 ügyfelet tartalmazó labor esetében.

Ez rokon a 7.3 CTE-s ablakfüggvényes példájával: a `PARTITION BY customer_id` és a `ROW_NUMBER` is fenntart rendezett állapotot, a tervsorrend nem törli az adatfüggő költséget. A 4.3 modulhatár-elve itt is érvényes: a `build_customer_summary` függvényben rögzített definíciót a notebook, a script és a CLI ugyanúgy hívhatja; a lazy terv nem teszi feleslegessé a tiszta függvényhatárt.

Önellenőrzés: (1) meg tudom mondani, melyik hívás ad `LazyFrame`-et és melyik `DataFrame`-et? (2) be tudom bizonyítani a mini példán, hogy a filter a join elé hozható? (3) meg tudom nevezni, miért nem duplázódik a `basket_minor` a példában? (4) meg tudom nevezni egy olyan műveletet, amelynél a lazy forma sem garantál állandó memóriát?

A lazy végrehajtás nem jelent állandó vagy garantáltan alacsony memóriaigényt. A nagy kardinalitású `group_by`, a rendezés és egyes joinok jelentős köztes állapotot igényelhetnek; a streaming-végrehajtás támogatása a konkrét művelettől és könyvtárverziótól is függ.

8.3.9 Rövid összefoglaló

A `scan_parquet` és `scan_csv` LazyFrame-et ad: tervet, nem adatot. A `select`, `filter` és `group_by` csak tervcsomópontokat fűz; a `collect` anyagiasít DataFrame-et, a `sink_parquet` közvetlenül fájlba ír. A predikátum-letolás miatt a cutoff-szűrés a join elé kerülhet, ami algebrailag azonos eredményt ad kevesebb közbülső sorral. A streaming motor csökkentheti a heap-csúcsot, de a csoportállapot a kulcsok számával arányos; a lazy API nem garancia korlátlan memóriára. A következő lépés a 8.4. alfejezet ellenőrző pont: a pandas, DuckDB és Polars kimenetek azonos mutatódefinícióval történő együttes ellenőrzése.

8.4 II. ellenőrző pont: adatból ellenőrzött feature-tábla

Belépő · [BŐVÍTŐ MODUL] · szint: alap · idő: 45 perc · eszköz: Parquet, Arrow, DuckDB, Polars

Előfeltétel: 8.3. alfejezet · Célkészség: II. ellenőrző pont: adatból ellenőrzött feature-tábla · Kimenet: a fejezet feladatainak megoldása és önellenőrzés.

8.4.1 Cél és konkrét helyzet

Ebben az ellenőrző pontban a Northstar lab három tiszta forrásfájljából egyetlen rendelésszintű, modellépítésre alkalmas feature-táblát készítünk. A bemenet:

- `customers.csv` — 300 ügyfél, egy sor ügyfelenként, kulcs: `customer_id`;
- `orders.csv` — 2400 rendelés, egy sor rendelésenként, kulcs: `order_id`;
- `order_items.csv` — 4796 rendelési tétel, kulcs: `(order_id, line_id)`.

A megfigyelési egység a kimenetben egy rendelés. Az eredmény egy `orders_features.parquet`, egy `data_quality_report.json`, és egy független pandas-ellenőrzés, amely ugyanazt a táblát állítja elő, mint a DuckDB SQL. A feature-tábla csak a rendelés pillanatában ismert oszlopokat tartalmazhatja; a `delivery_days`, `actual_delivery_at`, `label_available_at` és `target_late` mezőket ki kell zárni, mert azok a későbbi célváltozó meghatározásához tartoznak.

Előzetes becslés: a kimenet pontosan 2400 sorból áll, az `order_id` 2400 egyedi értéket ad, a customer join nem veszít sort, és a tiszta lab adatban a prediktor oszlopok NULL-száma nulla. Az `order_items` aggregátumainak rendelésszintű összege meg kell egyezzen az `orders.csv` `basket_minor` és `quantity_sum` értékeivel. Mielőtt tovább olvasol, becsüld meg: ha véletlenül a tételszintű táblát közvetlenül joinolnád a rendelésekhez, hány sor keletkezne a 2400 rendelésből, és hogyan változna a szállítási díj összege?

Az ellenőrző pont feladata az eddigi építőelemek összekapcsolása: a 6.1. alfejezet–6.4. alfejezet pandas-séma, a 7.1. alfejezet–7.4. alfejezet SQL-join és -aggregáció, valamint a 8.2. alfejezet DuckDB-fájlscan és Relation viselkedés. A 4.4. alfejezetben felállított szabály szerint a kimenethez tartozik egy futási jegyzék és kontrollösszeg is, nem elég „ránézésre jó” táblát írni.

8.4.2 Fogalmi alapok: séma, join, NULL és feature-idő

8.4.2.1 Séma

A séma explicit oszlopszerződés: minden oszlophoz megadjuk a nevet, típust, szemcsézettséget és mértékegységet. A CSV beolvasásnál az automatikus típusfelismerés kényelmes, de egy üres mező vagy egy furcsa dátum felboríthatja az utána következő joinokat. A DuckDB `read_csv` hívásában ezért explicit oszloptípusokat adunk meg. A Parquet fájl magában hordozza a sémát, ezért a kimenet olvasója nem találgathat: a `pyarrow.parquet.read_schema` megmutatja a rögzített típusokat.

A kimeneti tábla oszlopai:

Oszlop	Típus	Jelentés
`order_id`	string	rendelés egyedi kulcsa, nem prediktor
`customer_id`	string	ügyfélkulcs, nem prediktor

Oszlop	Típus	Jelentés
`order_ts`	timestamp	predikciós időpont, splithez kell
`distance_km`	float64	szállítási távolság kilométerben
`priority_flag`	int	0/1 prioritásjelző
`basket_minor`	int	kosárérték EUR-centben
`quantity_sum`	int	összes darabszám a rendelésben
`account_age_days`	int	a rendelés napján ismert ügyfélkor napokban
`segment`, `region`, `acquisition_channel`	string	kategorikus ügyféljellemzők

A `basket_minor` és a `quantity_sum` pénz-, illetve darabszám-egysége egész; a `distance_km` folytonos lebegőpontos érték. A `segment`, `region` és `acquisition_channel` kategóriák, nem rendezett numerikus kódok, tehát stringként maradnak.

8.4.2.2 Join és szemcsézettség

A három forrás három különböző szemcsézettséget képvisel. A `customers` egy sor ügyfelenként, az `orders` egy sor rendelésenként, az `order_items` pedig egy sor tételenként. A cél egy sor rendelésenként. A tételszintű táblát ezért a join előtt rendelésszintre kell aggregálni:

```
SELECT
    order_id,
    COUNT(*)                          AS item_rows_calc,
    SUM(quantity)                      AS quantity_calc,
    SUM(line_total_minor)              AS basket_minor_calc,
    SUM(quantity * unit_price_minor - discount_minor) AS basket_minor_recomputed
FROM order_items
GROUP BY order_id
```

Ha ezt a lépést kihagyjuk, a tételtábla 4796 sora a 2400 rendeléssorhoz kapcsolva akár 4796 soros kimenetet adhat, és a rendelésszintű oszlopok, például a `shipping_fee_minor` vagy a `distance_km`, tételekként többszöröződnek. A 7.2. alfejezetben látott one-to-many join pontosan ezért volt kiemelt hibaforrás: a szállítási díj egyszer létezik rendelésenként, nem szabad tételszintre szorozni.

8.4.2.3 NULL-szerződés

A tisztaz `orders.csv` prediktor mezőiben a lab szerződés szerint nincs hiányzó érték. Ezt nem hisszük el, hanem ellenőrizzük. A szabály: a feature-tábla nem tartalmazhat dokumentálatlan NULL-t a prediktor oszlopokban. Ha egy hiányzó `distance_km` bukkan fel, a szkript ne töltsön be hallgatólagosan nullát vagy átlagot, hanem álljon meg, és írja be a jelentésbe a `null_rows > 0` állapotot. A NULL nem technikai kellemetlenség, hanem adatminőségi esemény, amelynek oka vagy dokumentált imputálási döntése kell legyen.

8.4.2.4 Feature-idő

A modell a rendelés pillanatában becsül. A feature-tábla minden sorának ezért van egy `feature_time` értéke: az `order_ts`. Az `account_age_days` kiszámítása:

$$a_o = \left\lfloor \text{order_date}_o - \text{signup_date}_{c(o)} \right\rfloor_{\text{nap}}$$

ahol o a rendelés, $c(o)$ az ügyfél. A lab szintetikus adatában a `signup_date` statikus, ezért ez az érték mindig a rendelés előtt ismert. Ha a valós üzleti adatban az ügyfél szegmense vagy régiója időben változna, akkor a joinhoz havi pillanatfelvétel kellene; erről a 4. feladat szól. A későbbi tanító-teszt split további `label_available_at` szűrést igényel, de az már a modellépítés feladata, nem a feature-tábláé.

8.4.2.5 Kontrollösszeg

A kontrollösszeg egy numerikus oszlop összege a tábla összes sorára:

$$S_c = \sum_{i=1}^{n} x_{i,c}, \qquad N_c = \#\{i \mid x_{i,c} \text{ nem NULL}\}$$

A jelentésbe a `basket_minor`, `quantity_sum`, `basket_minor_calc`, `quantity_calc` és a `discount_minor_total` S_c értéke kerül. Az invariant: `basket_minor` összege az `orders.csv`-ben megegyezik a `line_total_minor` rendelésszintű összegével az `order_items.csv`-ben, és ugyanez igaz a `quantity_sum` és a tételenkénti `quantity` összegére. A kontrollösszeg nem bizonyítja, hogy minden sor helyes, de azonnal jelzi, ha egy join megdupláz vagy elveszít sorokat. A Parquet fájl SHA-256 lenyomatát is rögzítjük, hogy az átadott fájl később azonosítható legyen.

8.4.2.6 Végrehajtóválasztás

Ugyanazt a logikai műveletet három eszközzel is el lehet végezni:

Szempont	DuckDB SQL	pandas	Polars
Fő erősség	deklaratív join + aggregáció fájlból	interaktív oszlopkezelés, dtype-kontroll	lazy terv, oszlopos kiértékelés
Előzmény	7.1. alfejezet–8.2. alfejezet	6.1. alfejezet–6.4. alfejezet	8.3. alfejezet
Szerep itt	fő előállító	független ellenőrzés	opcionális harmadik

A DuckDB-t választjuk fő előállítónak, mert a fájlokat közvetlenül, pandas köztes DataFrame nélkül olvassa, és a `COPY TO` utasítás egy lépésben ír Parquet-et. A pandas az ellenőrzés nyelve, mert a könyv fő táblás felülete; ha két különböző implementáció azonos kimenetet ad, csökken a fordítási hiba esélye. A Polars ebben a fejezetben nem előfeltétel, de a 8.3. alfejezet lazy terve ugyanehhez a kontrollhoz használható.

8.4.3 Teljes minimális munkapélda

A lab gyökerében hozzuk létre a `make_orders_features.py` szkriptet. A fájlokat a `data/` mappából olvassuk, a kimenetet az `artifacts/` mappába írjuk. A kód célja: a három CSV beolvasása explicit sémával, a tételtábla aggregálása, a feature-tábla összeállítása, a kontrollok kiszámítása, a Parquet és a JSON jelentés kiírása, végül egy független pandas-újraszámolás, amely `assert_frame_equal` hívással veti össze a két kimenetet. A kódot a `lab/checkpoints/cp2/check_features.py` versioned fixture-ön futó mini-tesztje is ellenőrzi (lásd alább).

```
import hashlib
import json
from pathlib import Path

import duckdb
import pandas as pd
import pyarrow.parquet as pq
from pandas.testing import assert_frame_equal

ROOT = Path(__file__).resolve().parent
ARTIFACTS = ROOT / "artifacts"
ARTIFACTS.mkdir(exist_ok=True)
PARQUET = ARTIFACTS / "orders_features.parquet"
REPORT = ARTIFACTS / "data_quality_report.json"

# --- 1. DuckDB: explicit séma és main query -----------------------------
con = duckdb.connect()

con.execute("""
CREATE OR REPLACE TEMP TABLE customers_raw AS
SELECT *
FROM read_csv('data/customers.csv', columns = {
    'customer_id': 'VARCHAR',
    'signup_date': 'DATE',
    'segment': 'VARCHAR',
    'region': 'VARCHAR',
    'acquisition_channel': 'VARCHAR'
});
""")

con.execute("""
CREATE OR REPLACE TEMP TABLE orders_raw AS
SELECT *
FROM read_csv('data/orders.csv', columns = {
    'order_id': 'VARCHAR',
    'customer_id': 'VARCHAR',
    'order_ts': 'TIMESTAMP',
    'quantity_sum': 'BIGINT',
    'item_rows': 'BIGINT',
    'basket_minor': 'BIGINT',
    'shipping_fee_minor': 'BIGINT',
    'currency': 'VARCHAR',
    'distance_km': 'DOUBLE',
    'priority_flag': 'TINYINT'
});
""")

con.execute("""
CREATE OR REPLACE TEMP TABLE items_raw AS
SELECT *
FROM read_csv('data/order_items.csv', columns = {
    'order_id': 'VARCHAR',
```

```
    'line_id': 'BIGINT',
    'product_id': 'VARCHAR',
    'quantity': 'BIGINT',
    'unit_price_minor': 'BIGINT',
    'discount_minor': 'BIGINT',
    'line_total_minor': 'BIGINT',
    'currency': 'VARCHAR'
});
""")

con.execute("""
CREATE OR REPLACE TEMP TABLE item_agg AS
SELECT
    order_id,
    COUNT(*) AS item_rows_calc,
    SUM(quantity) AS quantity_calc,
    SUM(line_total_minor) AS basket_minor_calc,
    SUM(quantity * unit_price_minor - discount_minor) AS basket_minor_recomputed,
    SUM(discount_minor) AS discount_minor_total,
    SUM(quantity * unit_price_minor) AS total_list_minor
FROM items_raw
GROUP BY order_id;
""")

con.execute("""
CREATE OR REPLACE TEMP TABLE orders_features AS
SELECT
    o.order_id,
    o.customer_id,
    o.order_ts,
    o.distance_km,
    o.priority_flag,
    o.basket_minor,
    o.quantity_sum,
    date_diff('day', c.signup_date, CAST(o.order_ts AS DATE)) AS account_age_days,
    c.segment,
    c.region,
    c.acquisition_channel
FROM orders_raw o
INNER JOIN customers_raw c USING (customer_id)
INNER JOIN item_agg i USING (order_id)
WHERE o.currency = 'EUR'
  AND o.distance_km IS NOT NULL;
""")

con.execute(
    f"COPY (SELECT * FROM orders_features ORDER BY order_id) "
    f"TO '{PARQUET}' (FORMAT PARQUET, COMPRESSION ZSTD);"
)

# --- 2. A jelentés kontrolljai ---------------------------------------
source_orders = con.sql("""
    SELECT COUNT(*) AS n,
           SUM(basket_minor) AS basket_sum,
           SUM(quantity_sum) AS q_sum,
           SUM(shipping_fee_minor) AS shipping_sum
    FROM orders_raw
""").fetchone()

final_rows = con.sql("""
    SELECT COUNT(*) AS n,
           COUNT(DISTINCT order_id) AS unique_order_ids,
           SUM(basket_minor) AS basket_sum,
```

```
           SUM(quantity_sum) AS q_sum
    FROM orders_features
""").fetchone()

item_totals = con.sql("""
    SELECT SUM(basket_minor_calc) AS basket_calc,
           SUM(quantity_calc) AS q_calc,
           SUM(basket_minor_recomputed) AS basket_recomputed
    FROM item_agg
""").fetchone()

lost_customer_join = con.sql("""
    SELECT COUNT(*) FROM orders_raw o
    WHERE NOT EXISTS (SELECT 1 FROM customers_raw c WHERE c.customer_id = o.customer_id)
""").fetchone()[0]

lost_item_join = con.sql("""
    SELECT COUNT(*) FROM orders_raw o
    WHERE NOT EXISTS (SELECT 1 FROM item_agg i WHERE i.order_id = o.order_id)
""").fetchone()[0]

# --- 3. pandas független ellenőrzés -----------------------------------
customers = pd.read_csv(
    ROOT / "data" / "customers.csv",
    dtype={"customer_id": "string", "segment": "string",
           "region": "string", "acquisition_channel": "string"},
    parse_dates=["signup_date"],
)
orders = pd.read_csv(
    ROOT / "data" / "orders.csv",
    dtype={"order_id": "string", "customer_id": "string",
           "currency": "string", "priority_flag": "int64",
           "distance_km": "float64"},
)
orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)

items = pd.read_csv(
    ROOT / "data" / "order_items.csv",
    dtype={"order_id": "string", "line_id": "int64",
           "product_id": "string", "quantity": "int64",
           "unit_price_minor": "int64", "discount_minor": "int64",
           "line_total_minor": "int64", "currency": "string"},
)
item_agg_pd = items.groupby("order_id").agg(
    item_rows_calc=("order_id", "size"),
    quantity_calc=("quantity", "sum"),
    basket_minor_calc=("line_total_minor", "sum"),
)
feat_pandas = (
    orders
    .merge(customers[["customer_id", "signup_date", "segment",
                      "region", "acquisition_channel"]],
           on="customer_id", how="inner")
)
feat_pandas = feat_pandas[
    feat_pandas["order_id"].isin(item_agg_pd.index)
]
feat_pandas["account_age_days"] = (
    feat_pandas["order_ts"].dt.tz_localize(None).dt.normalize()
    - feat_pandas["signup_date"]
).dt.days

feat_pandas = feat_pandas[[
```

```
    "order_id", "customer_id", "order_ts", "distance_km",
    "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel"
]].sort_values("order_id").reset_index(drop=True)

duckdb_feat = pd.read_parquet(PARQUET, engine="pyarrow")

controls_passed = (
    source_orders[0] == final_rows[0]
    and final_rows[1] == final_rows[0]
    and source_orders[1] == item_totals[0]
    and source_orders[2] == item_totals[1]
    and item_totals[0] == item_totals[2]
    and lost_customer_join == 0
    and lost_item_join == 0
)

# --- 4. A pandas és a DuckDB kimenet egyezése ---------------------------
compare_cols = feat_pandas.columns.tolist()
assert_frame_equal(
    duckdb_feat[compare_cols].reset_index(drop=True),
    feat_pandas,
    check_dtype=False,
    check_exact=False,
    rtol=1e-13,
    atol=1e-9,
)

# --- 5. A jelentés és a hash kiírása ------------------------------------
report = {
    "generated_at": pd.Timestamp.now(tz="UTC").isoformat(),
    "input_files": ["data/customers.csv", "data/orders.csv", "data/order_items.csv"],
    "granularity": "one row per order",
    "output_file": str(PARQUET.name),
    "output_row_count": int(final_rows[0]),
    "unique_order_ids": int(final_rows[1]),
    "lost_customer_join": int(lost_customer_join),
    "lost_item_join": int(lost_item_join),
    "controls_passed": bool(controls_passed),
    "control_totals": {
        "orders_basket_minor": int(source_orders[1]),
        "items_basket_minor_calc": int(item_totals[0]),
        "items_basket_minor_recomputed": int(item_totals[2]),
        "orders_quantity_sum": int(source_orders[2]),
        "items_quantity_calc": int(item_totals[1]),
    },
    "parquet_sha256": hashlib.sha256(PARQUET.read_bytes()).hexdigest(),
    "schema": {field.name: str(field.type)
               for field in pq.read_schema(PARQUET)},
}
REPORT.write_text(json.dumps(report, indent=2), encoding="utf-8")
print("controls_passed =", controls_passed)
print("parquet_sha256 =", report["parquet_sha256"])
```

Futtatás a lab projektkörnyezetben, munkakönyvtár: `lab/`:

```
# POSIX shell és Windows PowerShell esetén is:
uv sync --locked
uv run python make_orders_features.py
```

Ha a `uv.lock` még nem létezik, előbb a 4.4. alfejezetben megismert módon `uv lock`, majd a fenti `uv sync --locked` hívás jön. A teljes 2400 soros futás számait e fejezet írásakor nem rögzítettük; a lenti versioned fixture-ön viszont a teljes ellenőrzőlánc lefutott.

Versioned fixture és futtatható mini-teszt. A `lab/checkpoints/cp2/fixtures/` mappa három kis forrásfájlt (`customers.csv` 3 sor, `orders.csv` 4 sor, `order_items.csv` 7 tétel) és a várt kimenetet (`expected_features.csv`, 3 sor) tartalmazza. A `lab/checkpoints/cp2/check_features.py` ugyanazt a logikát futtatja DuckDB-vel, pandas-szal és – ha telepítve van – Polars-szal; az egész számokat pontosan, a `distance_km`-et `rtol=1e-13`, `atol=1e-9` toleranciával hasonlítja, és mindhárom kimenetet a várt CSV-hez is méri. A `tests/test_checkpoint2.py` ezt a mini-tesztet futtatja. Ellenőrzött eredmény: mindhárom motor 3 sort ad, `controls_passed = true`, `max_distance_abs_diff = 0.0`; a hiányzó `distance_km` miatt a negyedik rendelés kimarad, a teszt ezt is rögzíti.

8.4.4 Eredmény értelmezése és kézi kontroll

A fenti szkript várható logikája a lab szerződés alapján: `output_row_count = 2400`, `unique_order_ids = 2400`, `lost_customer_join = 0`, `lost_item_join = 0`, és a `controls_passed` értéke `True`. A szemléltető JSON innentől csak a formátumot mutatja; a számok a lab futása után lesznek mértek:

```
{
  "granularity": "one row per order",
  "output_row_count": 2400,
  "unique_order_ids": 2400,
  "lost_customer_join": 0,
  "lost_item_join": 0,
  "controls_passed": true
}
```

Kézi kontrollként vegyünk egyetlen rendelést három tétellel. Legyen a rendelés `basket_minor = 410`, `shipping_fee_minor = 500`, és a tételek:

- `quantity=2`, `unit_price_minor=100`, `discount_minor=0` → `line_total=200`;
- `quantity=2`, `unit_price_minor=80`, `discount_minor=10` → `line_total=150`;
- `quantity=1`, `unit_price_minor=70`, `discount_minor=10` → `line_total=60`.

Az összeg: `quantity_calc = 5`, `basket_minor_calc = 410`, tehát a rendelésszintű `basket_minor` egyezik. Ha a tételtáblát aggregálás nélkül joinolnánk, a három tétel miatt három sor keletkezne, és a `shipping_fee_minor` összege `3 * 500 = 1500` lenne a helyes `500` helyett. A kontrollösszeg ezért azonnal elárulja a hibás join-kardinalitást.

A pandas ellenőrzés nem csupán a kontrollösszegeket nézi, hanem a teljes 11 oszlopos táblát összehasonlítja a DuckDB Parquet kimenetével, `order_id` szerint rendezve. Az egész számlálóknál a tolerancia nulla, a lebegőpontos `distance_km`-nél `rtol=1e-13` és `atol=1e-9` engedélyezett, mert a két motor a lebegőpontos műveletek sorrendjében kismértékben eltérhet. A `check_dtype=False` azért szerepel, mert a pandas `category` vagy `int64` és a DuckDB `INTEGER` közötti reprezentációs különbség nem jelent tartalmi hibát.

A `data_quality_report.json` a séma mellett rögzíti a Parquet SHA-256 lenyomatát is. Így később bárki ellenőrizheti, hogy a fájl az átadás óta nem változott. Ez a 4.4. alfejezet futási jegyzékkel felállított elv Parquet-szintű megfelelője.

8.4.5 Feladatok

1. **Értelmezés.** A fenti háromtételes kézi példában mi a `quantity_calc`, `basket_minor_calc`, `discount_minor_total` és a tételszintű `shipping_fee_minor` összege, ha valaki az

`order_items` táblát közvetlenül joinolja, aggregálás nélkül? Melyik rendelésszintű érték romlik el, és miért?

2. **Implementálás.** Írj DuckDB SQL lekérdezést, amely egyetlen rendelésre a tételsorokból kiszámítja a mennyiséggel súlyozott átlagos egységárat:

$$\bar{p}_o = \frac{\sum_j q_j p_j}{\sum_j q_j},$$

ahol q_j a tétel darabszáma, p_j az egységár EUR-centben. A nevező biztosítására használj `NULLIF`-et, és a háromtételes példán ellenőrizd, hogy az eredmény `86`.

3. **Hibakeresés.** A jelentésben `lost_item_join = 5` és `controls_passed = false` jelenik meg, miközben a `lost_customer_join = 0`. Mi lehet a hiba forrása, és milyen lekérdezéssel lokalizálod pontosan az öt hiányzó rendelést? A tisztaz `orders.csv` 2400 sora ellenére a kimenet csak 2395 sor.

4. **Megváltozott feltétel.** Új forrás érkezik: `customers_monthly_snapshot.csv` oszlopai `customer_id, snapshot_month, segment, region, acquisition_channel`, ahol `snapshot_month` minden hónap első napja. Az ügyfél szegmense hónapról hónapra változhat. Hogyan módosul a join, hogy minden rendelés a saját rendelési hónapjában érvényes szegmenst kapja? Fogalmazd meg a feltételt, és mondd meg, mi történik, ha egy ügyfélnek hiányzik a snapshotja abban a hónapban.

5. **Transzfer.** A pandas ellenőrző részben cseréld le az `isin` szűrést egy `merge(..., how="left", indicator=True)` alapú ellenőrzésre. Melyik `_merge` kategória darabszámának kell nullának lennie, és hogyan viszed be ezt a `report` JSON-ba külön `pandas_lost_item_join` mezőként?

8.4.6 Megoldások és indoklás

1. megoldás. A tételszintű táblában a három sor `quantity_calc` összege `2 + 2 + 1 = 5`, a `basket_minor_calc` összege `200 + 150 + 60 = 410`, a `discount_minor_total = 0 + 10 + 10 = 20`. Ha a táblát aggregálatlanul joinoljuk a rendeléshez, a rendelésszintű `shipping_fee_minor` három soron is megjelenik, így az összege `1500` lesz a valódi `500` helyett. Ez a szállítási díj többszöröződése; a `basket_minor` a join előtti rendelésértékkel maradna, de ha tévesen tételenként is hozzáadnánk, akkor `410 + 1500 = 1910` hamis kosárösszeg keletkezne. A hiba gyökere a one-to-many join kardinalitásának figyelmen kívül hagyása.

2. megoldás.

```sql
SELECT
    oi.order_id,
    ROUND(
        SUM(oi.quantity * oi.unit_price_minor)
        / NULLIF(SUM(oi.quantity), 0),
        2
    ) AS weighted_unit_price_minor
FROM order_items oi
WHERE oi.order_id = 'O1'
GROUP BY oi.order_id;
```

A háromtételes példában a számláló `2*100 + 2*80 + 1*70 = 430`, a nevező `5`, így az eredmény `86.00`. A `NULLIF` arra az elvi esetre véd, ha a nevező nulla; ilyenkor az osztás `NULL`-t ad, amelyet a jelentés null-szűrése megfog. A `ROUND` itt a megjelenítést és a következő összehasonlítást stabilizálja, de a nyers arányt is érdemes megtartani.

3. megoldás. A `lost_item_join = 5` azt jelenti, hogy öt rendeléshez nincs egyetlen tételsor sem az `order_items.csv`-ben, vagy az `order_id` értékük eltér például szóköz vagy kis/nagybetű miatt. A lokalizáló anti join:

```
SELECT o.order_id, o.customer_id, o.order_ts
FROM orders_raw o
LEFT JOIN item_agg i USING (order_id)
WHERE i.order_id IS NULL
ORDER BY o.order_id;
```

Ez pontosan az öt hiányzó rendelést adja. Ha az `order_items.csv`-ben ezek az azonosítók más formában szerepelnek, a hiba sztring-összehasonlítási eltérés; ha egyáltalán nincsenek ott, akkor adathiány. A `controls_passed = false` azért következik be, mert a `basket_minor` és a `quantity_sum` összege a 2395 soros kimenetben kisebb, mint a 2400 soros forrásban. A javítás nem a kontroll kikapcsolása, hanem a hiányzó tételek pótlása vagy a rendelések explicit kizárása dokumentált döntéssel.

4. megoldás. A havi snapshot joinja:

```
SELECT ...
FROM orders_raw o
INNER JOIN customers_monthly_snapshot cs
    ON cs.customer_id = o.customer_id
   AND cs.snapshot_month = CAST(date_trunc('month', o.order_ts) AS DATE)
```

A `date_trunc('month', o.order_ts)` a rendelés időbélyegét a hónap első napjára kerekíti; `DATE`-re castolva egyezhet a `snapshot_month` oszloppal. A feltétel biztosítja, hogy a rendelés pillanatában érvényes szegmenst használjuk. Ha egy ügyfélnek az adott hónapban hiányzik a snapshotja, az `INNER JOIN` eldobja a rendelést, és a jelentésben új `lost_snapshot_join` kontroll jelenik meg. Ilyenkor vagy `LEFT JOIN`-nal jelezzük a hiányt és dokumentált alapértelmezést adunk, vagy a snapshot-előállítást javítjuk. A csendes `INNER JOIN`-nal való eldobás nem elfogadható audit nélkül.

5. megoldás. A pandas ellenőrzés:

```
check = (
    orders[["order_id"]]
    .merge(item_agg_pd.reset_index()[["order_id"]],
           on="order_id", how="left", indicator=True)
)
lost = (check["_merge"] == "left_only").sum()
report["pandas_lost_item_join"] = int(lost)
```

A `left_only` sorok azok, amelyek az `orders` táblában szerepelnek, de az aggregált tételtáblában nincsenek. Ennek a számnak nullának kell lennie. Ha nem nulla, a pandas és a DuckDB `lost_item_join` értékének is egyeznie kell; az egyezés hiánya arra utal, hogy a két implementáció bemenete vagy szűrése eltér.

8.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés ezen az ellenőrző ponton az, hogy „a DuckDB és a pandas ugyanazt adta, tehát a mutató helyes”. A két végrehajtó egyezése csak a fordítási hibát csökkenti, a mutató definícióját nem igazolja. Ha mindkét implementáció azt a hibás szabályt használja, hogy a `basket_minor`-hoz minden tétel után hozzáadja a `shipping_fee_minor`-t, a két kimenet továbbra is egyezni fog, csak mindkettő hamis lesz. Ez a 7.4. alfejezetben látott `EXPLAIN`-csapda rokona: a terv és a determinisztikus egyezés a végrehajtást írja le, nem a definíció helyességét. A védőháló a független szemantikai invariant: a `basket_minor` a `line_total_minor` összege, a `shipping_fee_minor` pedig rendelésszintű, tehát nem tartozik a kosárértékhez. Ezt a lab `ADATOK.md` szerződése rögzíti, és a kontrollösszeg e szerződés ellen dolgozik.

8.4.8 Önellenőrzés

Mielőtt továbblépnél, tedd fel magadnak a következő kérdéseket. A kimeneti tábla minden sora pontosan egy rendelés? A `customer_id` join után nincs duplikált `order_id`? A prediktor oszlopok NULL-száma nulla, vagy ha nem, a jelentés dokumentálta? A `basket_minor` és a `quantity_sum` összege egyezik az `order_items` aggregátumával? A `delivery_days`, `actual_delivery_at`, `label_available_at` és `target_late` oszlopok hiányoznak a feature-táblából? A Parquet SHA-256 lenyomata rögzítve van? Ha bármelyik válasz nem, az ellenőrző pont még nincs kész.

A feature-tábla minden mezőjéről igazolni kell, hogy a predikció pillanatában hozzáférhető volt. A feature „as-of” ideje és a tanítócímke megérkezésének ideje külön időtengely: egy késve rendelkezésre álló címke a releváns cutoff előtt nem használható tanításhoz.

8.4.9 Rövid összefoglaló

A II. ellenőrző pont rendelésszintű, időhelyes feature-táblát állít elő három Northstar forrásból. A tételtáblát a join előtt aggregáljuk, hogy a szállítási díj és a rendelésszintű összegek ne duplázódjanak. A DuckDB explicit CSV-sémát, inner joinokat és Parquet-kimenetet ad; a kontrollösszegek a `basket_minor`, a `quantity_sum` és a tételaggregátumok egyezését ellenőrzik. Egy független pandas implementáció a teljes 11 oszlopos táblát újraszámolja, és toleranciás keret-összehasonlítást végez. A jelentés rögzíti a sor- és kulcsszámot, a joinveszteséget, a NULL-állapotot és a fájl SHA-256 lenyomatát. A következő alkalmazás a 21.1. alfejezet modellépítés: ebből az érintetlen feature-táblából kell időhelyes tanító-teszt splitet készíteni, a célváltozót csak a split után csatolva.

9 Matematikai nyelv: függvények, logaritmusok és numerikus közelítés

9.1 Jelölésből végrehajtható számítás

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 8.4. alfejezet · Célkészség: Jelölésből végrehajtható számítás · Kimenet: működő kód és a labor ellenőrző tesztje.

9.1.1 Cél és konkrét helyzet

A II. ellenőrző pont után egy modellező öt rendelésre hibajelentést készített. A megfigyelési egység most egy rendelés. A bemenet egy ötelemű hibavektor:

$$e = (2.0,\ -1.0,\ 3.0,\ -2.0,\ 0.0),$$

ahol minden érték mértékegysége nap, és $e_i = y_i - \hat{y}_i$: a tényleges és az előrejelzett késés különbsége. A cél az átlagos négyzetes hiba (mean squared error, MSE) kiszámítása kézzel, Python-ciklussal és NumPy-val. Az ellenőrzési feltétel: a három út azonos eredményt ad, és az eredmény mértékegysége nap^2, nem nap.

Mielőtt a levezetést elolvassa, adjon előzetes becslést. Az abszolút hibák átlaga 1.6 nap, ezért a négyzetes hiba 3–4 nap^2 közé eshet; a pontos kézi összeg hamarosan következik.

Az alábbi öt szám pedagógiai példa, nem a Northstar-projekt tényleges modelljének mért kimenete. A számítások viszont teljes egészében ellenőrizhetők.

Az alfejezet csak annyi matematikát tételez fel, amennyit az 5.1. alfejezet bevezetett: egy vektor itt rendezett számlista, elemei koordináták, a Python-lista eleme a NumPy-tömb egy dimenziójának felel meg. Az összeadás, szorzás és hatványozás alapművelet.

9.1.2 Szumma és index: a képlet ciklussá fordul

A $-8, 4, 10, -5$ típusú hosszú összegek helyett a matematika egy tömör jelet használ. A

$$S = \sum_{i=1}^{n} e_i$$

jelentése: „az $e_1, e_2, \ldots, e_n$ értékek összege”. A jelölések:

- S: az összeg eredménye, ugyanabban a mértékegységben, mint az e_i tagok;
- i: futóindex (index), egész szám, mértékegység nélküli;

- n: a tagok száma, pozitív egész, mértékegység nélküli;
- e_i: az i-edik tag, példánkban az i-edik rendelés hibája napban.

Az alsó határ $i = 1$, a felső határ n. A Python-listák viszont 0-tól indexelnek, ezért a fordításnál ezt a különbséget mindig jelölni kell:

```
errors = [2.0, -1.0, 3.0, -2.0, 0.0]  # nap
n = len(errors)

s_math = 0.0
for i in range(1, n + 1):       # matematikai index: 1, 2, ..., n
    s_math += errors[i - 1]     # Python-index: 0, 1, ..., n-1
```

Ugyanez a beépített `sum` függvénnyel:

```
s_math2 = sum(errors)
```

Az öt hibára $S = 2 + (-1) + 3 + (-2) + 0 = 2.0$ nap. Figyeljen arra, hogy a szumma maga lehet kicsi, miközben a tagok abszolút értéke nagy: a pozitív és negatív hibák kioltják egymást. Ezért lesz a modellértékelésben fontos a négyzetre emelés.

Kézzel ellenőrzött lépések:

i	e_i (nap)	e_i^2 (nap^2)
1	2.0	4.0
2	-1.0	1.0
3	3.0	9.0
4	-2.0	4.0
5	0.0	0.0

A négyzetek összege 18.0 nap^2. Az átlagos négyzetes hiba képlete tehát

$$\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} e_i^2,$$

ahol $n = 5$, így $\mathrm{MSE} = 18.0/5 = 3.6$ nap^2.

Egy fontos zárójel-szabály már itt látszik: $\sum_{i=1}^{n} e_i^2$ a négyzetek összege, nem az összeg négyzete. Az összeg itt 2.0, annak négyzete 4.0; a négyzetek összege viszont 18.0. A képletbe írt hatvány a tagokra, nem az egész szummára vonatkozik.

9.1.3 Függvény, paraméter és értelmezési tartomány

A matematikai függvény (function) olyan szabály, amely egy bemenethez pontosan egy kimenetet rendel. A $q(e) = e^2$ függvény az e hibához a négyzetes hibát rendeli. Itt:

- értelmezési tartomány (domain): $D_q = \mathbb{R}$, tehát bármely valós hiba megengedett;
- értékkészlet (range): $[0, \infty)$, mert a négyzet soha nem negatív;
- mértékegység: ha e mértékegysége nap, akkor $q(e)$ mértékegysége nap^2.

A paraméter (parameter) olyan mennyiség, amelyet egy adott számításon belül rögzítünk, de egy másik számításnál megváltoztathatunk. A Northstar-adatokból ismert tételszintű pénzösszeg jó példa:

$$L(q; p) = p \cdot q,$$

ahol q a darabszám, p az egységár centben rögzített paramétere, az eredmény pedig a tétel értéke centben. A pontosvesszős írásmód jelzi, hogy q a változó, p pedig paraméter. Ugyanez Pythonban:

```
def line_total(quantity: int, unit_price_minor: int) -> int:
    return unit_price_minor * quantity
```

Ha a termék ára $p = 250$ cent, akkor $L(3; 250) = 750$ cent. A függvény képlete ugyanaz marad, a paraméter értéke azonban termékenként cserélhető. A data scientist akkor dolgozik pontosan, ha meg tudja mondani, melyik szimbólum változó, melyik paraméter, és milyen tartományból vehet fel értéket. A teljes tételösszeghez a 8.4. alfejezet szerződése még levonja a tételszintű kedvezményt: `line_total_minor = quantity * unit_price_minor - discount_minor`.

A tartomány nem üres formalitás. Az MSE képlet $n = 0$-ra nem értelmezett, mert nullával osztana. A Python-függvénynek ezt explicit hibaüzenettel kell jeleznie:

```
def mse(errors: list[float]) -> float:
    if not errors:
        raise ValueError("n must be at least 1")
    sq_sum = 0.0
    for e in errors:
        sq_sum += e * e
    return sq_sum / len(errors)
```

A `if not errors` ellenőrzés a tartományi feltétel kódba írt változata.

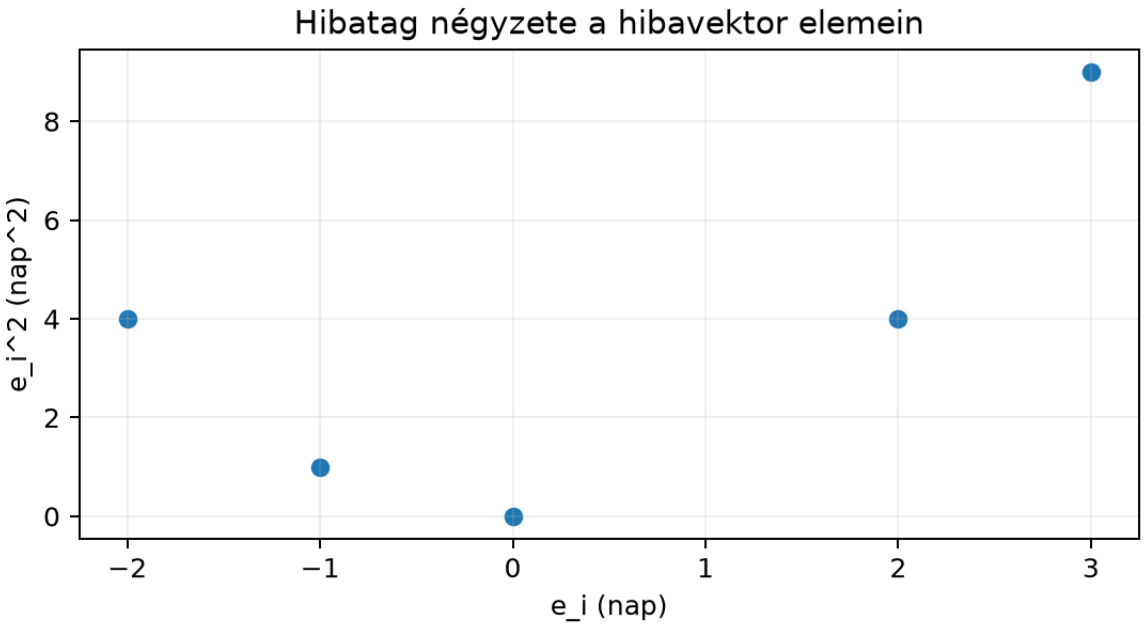
9.1.4 Mértékegység és tömbalak

A képlet megértésének része, hogy minden tag egységét nyomon követjük. A példa hibavektorában:

- `errors`: Python-lista, hossza $n = 5$, elemei `float` típusúak, mértékegysége nap;
- `errors_np` NumPy-alakja `(5,)`, `dtype=float64`, tehát egy darab ötelemű tengely;
- `e_i * e_i` mértékegysége nap^2;
- `sq_sum` mértékegysége nap^2;
- `1 / n` dimenziótlan, tehát az MSE mértékegysége nap^2;
- az MSE négyzetgyöke, az RMSE (root mean squared error), újra nap mértékegységű.

A mértékegység-ellenőrzés megelőzi a leggyakoribb félreértést: a 3.6 nem azt jelenti, hogy a modell átlagosan 3.6 nappal téved. Az átlagos abszolút hiba itt $(2 + 1 + 3 + 2 + 0)/5 = 1.6$ nap, az RMSE pedig $\sqrt{3.6} \approx 1.897$ nap. A 3.6 nap^2 az MSE, tehát négyzetes skálán összegzi a hibát.

Az ábra a négyzetre emelést mutatja a példa értékein. A negatív előjel elveszik, ezért a -2 és 2 hiba ugyanakkora, 4 nap^2 négyzetes büntetést ad:



9.1. ábra: Hibatag négyzete a hibavektor elemein - Számított függvényértékek: x^2 a 2, -1, 3, -2, 0 hibákon. Nem mérés.

Ugyanezt az ábrát interaktív notebookban egy Matplotlib `scatter` hívással lehetne megrajzolni; itt a könyv beépített chart-formátuma adja a pontos, ellenőrizhető pontokat.

9.1.5 Három számítási út: kéz, ciklus, NumPy

A teljes minimális munkapélda Python-fájlként futtatható. A kódot a projektgyökérben létrehozott `mse_demo.py` névvel mentse, majd futtassa Linux/macOS shellben `python mse_demo.py`, Windows PowerShellben pedig `py mse_demo.py` paranccsal. A kód nem használ véletlenszámot, ezért seed nem kell; Python 3.12-vel és a kezdőlabor NumPy-verziójával futtatható, APIszempontból nem verzióérzékeny:

```
import math
import numpy as np

errors = [2.0, -1.0, 3.0, -2.0, 0.0]  # nap
n = len(errors)

# 1) Ciklusos fordítás: a matematikai i = 1..n index Pythonban 0..n-1.
sq_sum_loop = 0.0
for i in range(n):
    e_i = errors[i]
    sq_sum_loop += e_i * e_i
mse_loop = sq_sum_loop / n              # nap^2
rmse_loop = math.sqrt(mse_loop)         # nap

# 2) NumPy vektorizált alak
errors_np = np.array(errors, dtype=float)
mse_numpy = np.mean(errors_np ** 2)

print("n =", n)
print("sq_sum_loop =", sq_sum_loop)
print("mse_loop =", mse_loop)
```

```
print("mse_numpy =", mse_numpy)
print("rmse_loop =", rmse_loop)
print("shape =", errors_np.shape, "dtype =", errors_np.dtype)
```

Az alábbi kimenet elméletileg várt, kézzel levezetett érték; a kód teljes futtatását külön példaprojektben ellenőrizzük. A helyi futtatásnak 10^{-12}-es toleranción belül egyeznie kell:

```
n = 5
sq_sum_loop = 18.0
mse_loop = 3.6
mse_numpy = 3.6
rmse_loop = 1.8973665961010275
shape = (5,) dtype = float64
```

A `sq_sum_loop = 18.0` pontosan egyezik a kézi táblázattal. A `mse_loop` és `mse_numpy` egyezését ne `==`-vel ellenőrizzük bitmintára, hanem toleranciával: `abs(mse_loop - mse_numpy) < 1e-12`. A `3.6` ugyanis kettes számrendszerben nem véges tört, ezért a lebegőpontos ábrázolás a legközelebbi kettes értéket tárolja; numerikus összehasonlításhoz tűrés kell.

9.1.6 Eredmény értelmezése

Az eredmény három úton ellenőrzött:

1. A kézi összeg $4 + 1 + 9 + 4 + 0 = 18$, tehát MSE $= 18/5 = 3.6$.
2. A Python-ciklus ugyanezt, `float` típusú `18.0` részösszeggel adja.
3. A NumPy az `errors_np ** 2` vektorművelettel elemenként négyzetre emel, majd a `mean` a $18/5$ normalizálást végzi.

Az ellenőrző invariantok: az elemszám $n = 5$ megegyezik az `errors_np.shape[0]` értékkel; az `errors_np.dtype` numerikus, `float64`; a négyzetes hiba mértékegysége nap^2, az RMSE mértékegysége nap. A hibaösszeg önmaga nem elég: a 18.0 csak a $18/5$ lépéssel válik átlagos négyzetes hibává. Ha a cél az eredeti hibaskálára való visszatérés, akkor az RMSE-t kell közölni, nem az MSE-t.

A kapott 3.6 ezen az öt rendelésen egy leíró szám, nem statisztikai becslés bizonytalansággal. Azt állítja, hogy ezen az öt megfigyelésen a négyzetes hibák átlaga 3.6 nap^2; nem állítja, hogy egy jövőbeli rendelés hibája pontosan ekkora lesz.

9.1.7 Feladatok

1. **Értelmezés.** Egy kolléga azt írja a jelentésbe: „A modell átlagosan 3.6 nappal téved.” A hibavektor továbbra is `[2.0, -1.0, 3.0, -2.0, 0.0]` nap. Mi a hiba a mondatban? Add meg a helyes értelmezést, és számítsd ki az átlagos abszolút hibát, valamint az RMSE-t üzenet-ellenőrzésként.
2. **Megvalósítás.** Írj `mse(errors)` nevű Python-függvényt, amely üres listánál `ValueError` kivételt dob, egyébként ciklussal kiszámítja az MSE-t. Ellenőrizd az alábbi inputra: `[2.0, -1.0, 3.0, -2.0, 0.0]`. A megoldásban adjon `assert` ellenőrzést is.
3. **Hibadiagnózis.** Az alábbi kód miért nem adja a fenti `3.6` eredményt?

```
errors = [2.0, -1.0, 3.0, -2.0, 0.0]
sq_sum = 0.0
for i in range(1, 6):
    sq_sum += errors[i] * errors[i]
mse = sq_sum / 5
```

Mi a pontos hibaüzenet vagy hibás érték oka, és hogyan kell javítani a ciklushatárt?

4. **Megváltoztatott feltétel: mértékegység-átváltás.** Ugyanezeket az eltéréseket órában tárolják: `[48.0, -24.0, 72.0, -48.0, 0.0]` óra. Számítsd ki az MSE-t órában. Ellenőrizd, hogy az eredmény a napban kapott 3.6 nap^2 érték 1 nap^2 = 576 óra^2 átváltásával egyezik. Miért hibás ilyenkor a nyers `3.6` érték továbbítása mértékegység nélkül?

9.1.8 Megoldások és indoklás

1. A kolléga a nap^2 mértékegységet napként olvasta. A helyes állítás: az MSE 3.6 nap^2, ami négyzetes skálán bünteti a hibát. Az átlagos abszolút hiba (mean absolute error, MAE) $(2+1+3+2+0)/5 = 1.6$ nap; az RMSE $\sqrt{3.6} \approx 1.897$ nap. Ellenőrzés: a MAE és az RMSE egyaránt nap mértékegységű, az MSE pedig nap^2, így a három szám nem felcserélhető.

2. Lehetséges megoldás:

```
def mse(errors: list[float]) -> float:
    if not errors:
        raise ValueError("n must be at least 1")
    sq_sum = 0.0
    for e in errors:
        sq_sum += e * e
    return sq_sum / len(errors)

expected = mse([2.0, -1.0, 3.0, -2.0, 0.0])
assert abs(expected - 3.6) < 1e-12
```

Az üres lista az $n = 0$ tiltott tartomány, ezért nem hibaüzenet nélküli `ZeroDivisionError`-ba kell futnia, hanem érthető `ValueError` kell. Az `assert` a 10^{-12} toleranciás ellenőrzést végzi.

3. A Python-listák indexe 0-tól indul, ezért a `range(1, 6)` először kihagyja a `2.0` elemet, majd az `i = 5` értéknél `errors[5]` hozzáféréssel `IndexError` hibát vált ki. A javított ciklushatár `range(len(errors))`, vagy közvetlenül az elemeken lehet iterálni:

```
sq_sum = 0.0
for e in errors:
    sq_sum += e * e
mse = sq_sum / len(errors)
```

Ellenőrző invariant: a `sq_sum` összegnek 18.0-t kell adnia, és `len(errors)` értéke 5.

4. Az órában adott hibák négyzetei: $2304, 576, 5184, 2304, 0$; összegük 10368 óra^2; öttel osztva 2073.6 óra^2. Az átváltás: $3.6 \cdot 576 = 2073.6$. A nyers `3.6` nem hordozza az egységet, és ha valaki napként értelmezi, miközben a tömb órában van, hibás következtetést kap. Az RMSE ellenőrzése: $\sqrt{2073.6} \approx 45.537$ óra, ami $45.537/24 \approx 1.897$ nap, tehát visszaadja az eredeti RMSE-t.

9.1.9 Gyakori tévedés ellenpéldával

„A képletet beírtam a kódba, tehát jó” — ez akkor hibás, ha a tagok egysége és a műveletek dimenziója tisztázatlan. Tipikus ellenpélda a szumma és a hatvány sorrendjének felcserélése:

```
# Hibás: ez az összeg négyzete ötödökkel, nem az átlagos négyzetes hiba.
s = sum([2.0, -1.0, 3.0, -2.0, 0.0])
mse_wrong = (s ** 2) / 5      # 0.8
```

A helyes képlet előbb elemenként négyzetre emel: `sum(e*e for e in errors) / 5 = 3.6`. A 0.8 számértéke is eltér, és a mértékegysége is baj: az összeg négyzete nap^2 ugyan, de nem az egyedi hibák négyzetes büntetését átlagolja. Hasonló csapda a 8.4. alfejezet kontrollösszegeinél: a `basket_minor` szumma centben adódik, nem euróban; a cent/euró skálafaktor nélkül a kontrollérték üres szám.

Visszautalás: az 5.1. alfejezetben bevezetett `shape` és `dtype` itt az ellenőrzés eszköze lett. A `(5,)` alakból látszik, hogy a bemenet egyetlen ötelemű tengely; az `axis` nélküli `mean` ezt az egész tengelyt redukálja skalárrá. A matematikai szumma tehát a NumPy redukció közvetlen rokona.

Önellenőrzés: el tudja-e mondani, mit jelent a $\sum_{i=1}^{5} e_i^2$ alsó és felső indexe? Tudja-e, miért `range(len(errors))` a Python-megfelelő? Meg tudja-e nevezni a 3.6 mértékegységét, és azt, hogy miért nem „3.6 nap”? Ha mind a háromra van egy-egy pontos mondata, a jelölésből végrehajtható számítás lépése működik.

A tanítóhalmazon számított MSE önmagában nem torzítatlan becslése az új adatokon várható modellhibának. Mindig jelöljük, hogy a hibavektor a tanító-, validációs vagy tesztmintán keletkezett.

9.1.10 Rövid összefoglaló

A matematikai jelölés nem dísz, hanem futtatható utasítás. A szumma alsó és felső indexe a ciklushatárt adja; a hívott függvény a képlet; a paraméter a rögzített szám. A tartomány és a mértékegység nélkül a szám félrevezető: az ötértékű hibavektor MSE-je 3.6 nap^2, RMSE-je 1.897 nap. A kézi, ciklusos és NumPy-útvonal azonos eredményt adott, amit 10^{-12} toleranciás invariant ellenőriz. A szumma négyzetének és a négyzetek szummájának felcserélése a tipikus jelölési hiba. A következő alfejezet ezt a függvényfogalmat az exponenciális és logaritmikus skálákra alkalmazza.

9.2 Exponenciális és logaritmikus skála

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 9.1. alfejezet · Célkészség: Exponenciális és logaritmikus skála · Kimenet: működő kód és a labor ellenőrző tesztje.

9.2.1 Cél és konkrét helyzet

A Northstar-rendelések között vizsgáljuk a `distance_km` szállítási távolságot. A megfigyelési egység egy rendelés, a bemenet a tisztaz `orders.csv` pozitív kilométer-oszlopa; a cél annak eldöntése, hogy a ferde, jobbra elnyúló eloszlás logaritmikus skálán jobban áttekinthető-e, és mi a teendő nulla vagy negatív értékekkel. A kimenet egy nyers és egy log-skálás hisztogram, valamint egy dokumentált döntés a zéró/negatív kezelésről.

Előzetes becslés: a távolságok tipikusan néhány kilométertől néhány száz kilométerig terjednek, így a nyers hisztogram jobbra ferde lesz, a `log10`-transzformált pedig közelítőleg szimmetrikusabb, mert 10-szeres szorzók azonos távolságra kerülnek. Kézzel kiszámolva: ha egy érték 1, akkor $\log_{10}(1) = 0$; ha 10, akkor 1; ha 100, akkor 2; ha 1000, akkor 3. Ez már mutatja, hogy a logaritmus a szorzókat összeadássá alakítja.

9.2.2 Az exponenciális függvény és inverze, a logaritmus

Az **exponenciális függvény** (*exponential function*) a természetes alappal:

$$\exp(x) = e^x, \quad \text{ahol } e \approx 2{,}71828.$$

Jelölésmagyarázat: x a bemenet, mértékegység nélküli valós szám; e^x a kimenet, szintén mértékegység nélküli, de mindig szigorúan pozitív. Tömbalakban: ha `x` egy `(n,)` alakú NumPy-tömb, akkor az `np.exp(x)` ugyancsak `(n,)` alakú, és elemenként számol. A függvény értelmezési tartománya $\mathbb{R}$, értékkészlete $(0, \infty)$.

Három tulajdonság, amit minden későbbi modellépítésben használunk:

1. $\exp(0) = 1$,
2. $\exp(a + b) = \exp(a) \cdot \exp(b)$, azaz a szorzás és összeadás közötti híd,
3. exp szigorúan monoton növekvő, tehát ha $x_1 < x_2$, akkor $\exp(x_1) < \exp(x_2)$.

A **logaritmus** (*logarithm*) a hatványozás inverze: ha $b^y = x$, akkor $y = \log_b(x)$. Az e alapú logaritmust természetes logaritmusnak nevezzük, és $\ln(x)$-szel jelöljük. A Python `math.log(x)` és a NumPy `np.log(x)` is természetes logaritmust számol; a 10-es alapú függvény külön `math.log10(x)` és `np.log10(x)` hívás.

Az inverz tulajdonság:

$$\ln(\exp(x)) = x \quad \text{minden } x \in \mathbb{R}\text{-re}, \qquad \exp(\ln(x)) = x \quad \text{minden } x > 0\text{-ra}.$$

Így $\ln(1) = 0$, mert $\exp(0) = 1$; $\ln(e) = 1$; és általában $\ln(10^k) = k \ln(10)$. A logaritmus értelmezési tartománya csak a pozitív számok halmaza $(0, \infty)$; nullára a bal oldali határ $-\infty$, negatív valós számra pedig nincs valós logaritmus. Ez a legfontosabb korlát a gyakorlatban: mielőtt logaritmálunk, dokumentálnunk kell, hogy az adat csak pozitív, vagy meg kell oldani a nulla/negatív értékek kezelését másképp.

9.2.3 Miért használunk logaritmikus skálát?

A logaritmikus skála a **szorzókat additívvá** alakítja:

$$\log_b(x_1 \cdot x_2) = \log_b(x_1) + \log_b(x_2).$$

Például az $1, 10, 100, 1000$ sorozat esetén a `log10` értékek $0, 1, 2, 3$: az eredetileg exponenciálisan növekvő ugrások azonos lépésközzé válnak. Ezért ha egy pozitív adatsor több nagyságrendet fog át, a log-skálás hisztogram vagy vonaldiagram sűríti a nagy értékeket, és láthatóvá teszi a kis értékek szerkezetét.

A **monoton transzformáció** (*monotonic transformation*) olyan függvény, amely megőrzi a sorrendet. Mivel ln szigorúan növekvő, ha $x_1 < x_2$, akkor $\ln(x_1) < \ln(x_2)$. A sorrend megmarad, de a távolságok nem: például $|\ln(10) - \ln(1)| \approx 2{,}303$, míg $|\ln(100) - \ln(10)|$ is $\approx 2{,}303$ — az eredeti 9 és 90 különbség helyett mindkét lépés azonos. Ez előny az ábrázolásban, de fontos következmény: **a log-skálán számolt átlag nem az eredeti skála számtani átlaga**, erre az alfejezet végén visszatérünk.

A logaritmus a későbbi statisztikai modellezésben is központi szerepet kap. A **log-likelihood** (logaritmikus likelihood) azért használatos, mert a likelihood sok valószínűség szorzata; az 1 alatti

szorzások gyorsan alulcsordulnak (*underflow*) a lebegőpontos ábrázolásban, míg a logaritmusok összege stabil marad:

$$\ln\left(\prod_{i=1}^{n} p_i\right) = \sum_{i=1}^{n} \ln(p_i).$$

Itt $p_i \in (0,1]$ az i-edik megfigyelés likelihood-tényezője, $\prod$ a szorzatjel, $\sum$ az összegjel; az egyenlet a logaritmus szorzat-összeg azonosságának n-tagú kiterjesztése. A cél a numerikus stabilitás: $10^{-300} \cdot 10^{-300}$ a lebegőpontos `float64` tartományban már nulla, de $\ln(10^{-300}) + \ln(10^{-300})$ körülbelül $-1381{,}6$, kényelmesen ábrázolható. Ezzel a gondolattal a likelihood-alapú becslésnél (15.1. alfejezet) találkozunk majd újra; itt elég annyi, hogy a logaritmus a szorzást összeadássá alakítja, ami numerikusan és jelölésben is kezelhetőbb.

A logit előképe (*logit preimage*) a $(0,1)$ nyílt intervallumot képezi le a teljes $\mathbb{R}$-re:

$$\operatorname{logit}(p) = \ln\left(\frac{p}{1-p}\right) = \ln(p) - \ln(1-p), \quad p \in (0,1).$$

A p itt valószínűség, mértékegység nélküli; a kimenet szintén mértékegység nélküli, negatív vagy pozitív. Az „előkép” kifejezés arra utal, hogy p a $(0,1)$ intervallumból érkezik, a transzformáció eredménye pedig korlátlan valós szám. A határértékek: $p \to 0^+$ esetén $\operatorname{logit}(p) \to -\infty$, $p \to 1^-$ esetén $\operatorname{logit}(p) \to +\infty$, $\operatorname{logit}(0{,}5) = 0$. Ennek gyakorlati értelme: ha egy modell valószínűséget becsül, akkor a nyers lineáris prediktor a valós skálán él, és a logit inverze, a logisztikus függvény, szorítja vissza $(0,1)$-be:

$$p = \frac{1}{1+\exp(-\eta)}, \quad \text{ahol } \eta = \operatorname{logit}(p).$$

Ez a későbbi logisztikus regresszió (22.1. alfejezet) magja; most annyi a feladatunk, hogy a logit mint transzformáció kiszámítását és tartományát megértsük.

9.2.4 A `log1p` és a numerikus stabilitás

Ha az adatsorban nulla érték is előfordul, a ln nem alkalmazható közvetlenül, mert $\ln(0)$ nem értelmezhető. Az egyik bevett megoldás a $\ln(1+x)$ transzformáció, amit a NumPy a **`log1p`** függvénnyel számol:

$$\operatorname{log1p}(x) = \ln(1+x), \quad x > -1.$$

A `log1p` előnye nem csupán kényelem, hanem numerikus pontosság: amikor x kicsi, például $x = 10^{-16}$, az $1+x$ lebegőpontos összeg már kerekítési hibát szenved, így a $\ln(1+x)$ direkt kiszámítása pontatlan. A `log1p` belsőleg a $\ln(1+x)$ sorfejtését használja, és kis x-re pontos eredményt ad. A két kifejezés ugyanazt a matematikai értéket célozza, de a numerikus út különböző.

Szemléltető, kézzel ellenőrizhető értékek:

Kifejezés	Elméletileg várt érték
`log1p(0.0)`	0
`log1p(-0.5)`	$\ln(0{,}5) \approx -0{,}6931$

Kifejezés	Elméletileg várt érték
`log1p(np.e - 1)`	1, mert $\ln(e) = 1$
`log1p(1e-16)`	$\approx 10^{-16}$, míg `log(1+1e-16)` könnyen 0-t adhat `float64`-ben

A `log1p` tehát a nulla érték kezelésének standard építőköve, amikor a logaritmust közelítő transzformációt akarunk pozitív, de kis értékek mellett is.

9.2.4.1 Határérték-tesztpolc: log1p, expm1, softmax és log(0)

Négy szélső bemenet mutatja meg, hol hibázik a naiv képlet:

Bemenet	Naiv kifejezés	Eredmény	Stabil kifejezés	Eredmény
$x = 10^{-16}$	`log(1+x)`	`0.0`	`log1p(x)`	`1e-16`
$x = 10^{-16}$	`exp(x)-1`	`0.0`	`expm1(x)`	`1e-16`
$x = 0$	`log(x)`	`-inf`	`log1p(x)`	`0.0`, mert $\ln(1) = 0$
$x = -1$	`log(x)`	`nan`	`log1p(x)`	`-inf` a határon; negatívra domain-hiba
$[1000, 1001, 1002]$	`exp(x)/sum(exp(x))`	`[nan,nan,nan]`	`exp(x-max)/sum(...)`	`[0.0900,0.2447,0.6652]`

A `log1p` és az `expm1` a kis x tartományban őrzi meg a pontosságot; a `log(1+x)` és az `exp(x)-1` kerekítés miatt nullát ad. A softmax naiv változata nagy logitekre `exp(1000)` miatt túlcsordul, a maximum kivonása után viszont a legnagyobb kitevő 0, így az összeg stabil. A `log(0)` `-inf`, a `log(-1)` `nan`; ezeket domain-ellenőrzéssel vagy `np.errstate`-tel kell kezelni, nem csendben továbbadni.

```
import numpy as np

def softmax_stable(values):
    values = np.asarray(values, dtype=np.float64)
    shifted = values - values.max()
    exponentials = np.exp(shifted)
    return exponentials / exponentials.sum()

with np.errstate(divide="ignore", invalid="ignore"):
    domain = np.log(np.array([0.0, 1e-300, 1.0, -1.0]))
assert np.isneginf(domain[0])
assert np.isnan(domain[3])
assert np.allclose(softmax_stable([1000.0, 1001.0, 1002.0]),
                   [0.09003057317038046, 0.24472847105479764,
                    0.6652409557748218])
```

A `tests/test_log_scales.py` a négy szélső bemenetet, a `log1p`/`expm1` és a naiv változat különbségét, a softmax túlcsordulását és stabil értékét, valamint a `log(0)`/`log(-1)` eredményét ellenőrzi.

9.2.5 Teljes minimális munkapélda

A feladat: generáljunk ferde, jobbra elnyúló pozitív adatsort, rajzoljunk nyers és log-skálás hisztogramot, és dokumentáljuk a nulla/negatív kezelést. A kód a szokásos importokkal kezdődik, a véletlengenerátort rögzítjük, hogy a munkamenet reprodukálható legyen:

```
import numpy as np
import matplotlib.pyplot as plt

# Rögzített seed: a szimulációs adat ugyanazt a sorrendet adja ugyanazon CPU-verzión
rng = np.random.default_rng(20261020)

# Ferde pozitív adat: lognormális eloszlás, n=5000 megfigyelés.
# A lognormális azt jelenti, hogy a természetes logaritmusa normális eloszlású.
x_positive = rng.lognormal(mean=2.0, sigma=1.1, size=5000)

# Szándékosan hozzáadunk 50 nullát és 30 negatív értéket,
# hogy a kezelést gyakoroljuk. A negatív értékeket -0.5 és -10 között
# egyenletesen sorsoljuk; ezek azt modellezik, hogy hibás vagy csonkolt
# adat érkezett, amire transzformáció előtt dönteni kell.
x_zero = np.zeros(50)
x_negative = rng.uniform(low=-10.0, high=-0.5, size=30)

# A teljes nyers adatsor; a döntéshez külön kezeljük a három csoportot.
x = np.concatenate([x_positive, x_zero, x_negative])

# 1. lépés: csak a pozitív adatra ábrázolunk, mert log(0) és log(negatív)
# valós számokon nem értelmezhető GYIK-ként szerepelő módon.
mask_positive = x > 0
x_pos_only = x[mask_positive]

# 2. lépés: a nulla értékeket log1p-vel kezeljük:
# log1p(0)=0, az eredmény továbbra is pozitív, sorrendtartó.
x_with_zero = x_zero
log1p_zero = np.log1p(x_with_zero)

# 3. lépés: a negatív értékeket külön dokumentáljuk; itt nem transzformáljuk,
# hanem hibajelentésbe gyűjtjük, mert a log1p csak ( -1, inf )-en értelmezhető.
negative_count = int(np.count_nonzero(x < 0))
print(f"Negatív értékek száma (kihagyva a log-transzformációból): {negative_count}")

# 4. lépés: ábrázolás. A pozitív adatra nyers és log10 skálájú hisztogram.
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(x_pos_only, bins=60, color="steelblue", edgecolor="white", alpha=0.85)
axes[0].set_title("Nyers távolság eloszlás")
axes[0].set_xlabel("km")
axes[0].set_ylabel("Rendelések száma")

axes[1].hist(np.log10(x_pos_only), bins=60, color="darkorange", edgecolor="white", alpha=0.85)
axes[1].set_title("Log10-transzformált távolság")
axes[1].set_xlabel("log10(km)")
axes[1].set_ylabel("Rendelések száma")
plt.tight_layout()
plt.show()
```

A fenti kód még nem automatizáltan ellenőrzött, így a kimenetet **szemléltető** jellegűnek nevezzük. A várt viselkedés: a bal oldali hisztogram jobbra elnyúló, tömeg a kis értékeknél; a jobb oldali ábra közelítőleg szimmetrikus haranggörbe, mert a lognormális definíció szerint éppen ezt várjuk. A `np.log10` negatív vagy nulla bemenetre `RuntimeWarning`-ot ad és `-inf`-et vagy `nan`-t szolgáltat, ezért a mask szükséges. A `negative_count` a példánkban 30.

Kézi kontroll a logaritmusazonosságra, amit a kód ellenőrzéséhez papíron is kiszámolunk:

```
# Egyszerű ellenőrző vektor
x_check = np.array([1.0, 10.0, 100.0, 1000.0])
log10_check = np.log10(x_check)
# Elvárt: [0., 1., 2., 3.]

# A log1p és log(1+x) ekvivalenciája pozitív x-re:
y = np.array([0.0, 0.5, 1.0, 10.0])
diff = np.log1p(y) - np.log(1.0 + y)
# Elvárt: diff minden eleme 0 vagy 1e-16 nagyságrendű kerekítési hiba
```

Az ellenőrzési feltétel a shape-re: `x_check` shape-je `(4,)`, a `log10_check` shape-je szintén `(4,)`, a dtype `float64`. Toleranciánk a kézi kontrollnál szigorú, de a lebegőpontos művelet miatt $\leq 10^{-15}$ eltérést fogadunk el.

9.2.6 Eredmény értelmezése

A lognormális eloszlás definíciója szerint a $\ln(x)$ normális, ezért a $\log_{10}$ transzformált hisztogram vártan szimmetrikus. Ez nem azt jelenti, hogy a nyers adat hibás; csak azt, hogy a skálaváltás megmutatja a mögöttes multiplikatív struktúrát. A nulla értékeket `log1p`-vel 0-ba képeztük le, ami megőrzi a nullák megkülönböztetését; ha egy nullát és egy kis pozitív értéket szeretnénk összehasonlítani, a `log1p` az eredeti sorrendet megtartja. A negatív értékeket kihagytuk, mert a valós logaritmus tartománya $(0, \infty)$; ez a döntés a munkanaplóba írandó, mert később a negatív távolság adatminőségi hibát jelezhet.

Kiemelt ellenőrzési pont: a transzformált tömb `dtype`-ja `float64`, mert az egész értékek logaritmusa is általában irracionális lebegőpontos szám. A `np.log10(1000)` pontosan 3,0, de a `np.log10(10)` belső ábrázolása kerekítési hibát tartalmazhat; ezért a fenti kontrollt `np.allclose`-zal érdemes futtatni, `atol=1e-15` mellett.

A **log-skálán vett átlag** értelmezése külön figyelmet érdemel. Ha $x_1, x_2, \ldots, x_n$ pozitív megfigyelések, akkor az exp-transzformált log-átlag a **geometriai átlag** (*geometric mean*):

$$GM = \exp\left(\frac{1}{n}\sum_{i=1}^{n}\ln(x_i)\right).$$

Ez nem a szokásos számtani átlag. Például $x = [1, 10, 100]$ esetén a számtani átlag 37, a geometriai átlag

$$\exp\left(\frac{\ln(1)+\ln(10)+\ln(100)}{3}\right) = \exp\left(\frac{0+2{,}3026+4{,}6052}{3}\right) = \exp(2{,}3026) = 10.$$

Itt $\ln(x_i)$ mértékegység nélküli; a képletben $x_i > 0$ minden i-re, és a tömb alakja `(n,)`. A geometriai átlag a multiplikatív folyamatok „középértéke”: ha egy folyamat minden lépésben valahányszorosára nő, akkor az átlagos szorzó ennyi. A futási példánkban a lognormális eloszlású távolságok esetén a geometriai átlag rendre kisebb, mint a számtani, mert a nagy ritka értékek a számtani átlagot jobban felfelé húzzák.

9.2.7 Feladatok

1. feladat — Értelmezés. Magyarázd el, miért nincs értelmezve $\log_{10}(0)$ és $\log_{10}(-5)$, és add meg a következő értékeket négy tizedesjegyre: $\log_{10}(0{,}001)$, $\log_{10}(1)$, $\log_{10}(10)$, $\log_{10}(10^5)$.

2. feladat — Megvalósítás. Írj Python-függvényt `log_shifted(x, c)` néven, amely a `log1p` függvénnyel kiszámolja a $\ln(c + x)$ értéket, feltéve, hogy $c + x > 0$ minden elemre. A függvény ellenőrizd, hogy a bemeneti tömb minden eleme kielégíti-e a feltételt, és ha nem, `ValueError`-t dobjon. Próbáld ki a `[-0.5, 0.0, 1.0, 10.0]` bemenetre $c = 1$ mellett, és ellenőrizd kézzel: az eredmény `[ln(0.5), 0, ln(2), ln(11)]`.

3. feladat — Hibadiagnózis. Egy munkatárs a havi kosárértékeket akarta elemezni, és a következőt írta: „Kiszámítom a log-értékek átlagát, majd visszaexponálom, az lesz a havi átlagos kosár.” Miért helytelen ez a megfogalmazás? Adj konkrét ellenpéldát két pozitív számmal, ahol a számtani átlag és az exp-átlagos-log érték különbözik, és nevezd meg a helyes kifejezést a kapott értékre.

4. feladat — Megváltozott feltételű továbbvitel. Az adatsorba negatív értékek is kerülnek, például nettó profit, amely lehet negatív. Dolgozz ki transzformációs konvenciót: adj képletet, amely negatív x-re is definiált, megőrzi a sorrendet, és 0-ban folytonos. Képlet:

$$y = \begin{cases} \ln(1 + x), & x \geq 0, \\ -\ln(1 - x), & x < 0. \end{cases}$$

Ezt gyakran `signed_log`-nak nevezik. Implementáld NumPy-ban, és ellenőrizd a következő értékeken: $x = [-10, -1, 0, 1, 10]$; add meg a várt eredményt négy tizedesjegyre.

9.2.8 Megoldások és indoklás

1. megoldás. A logaritmus a hatványozás inverze: $\log_b(x) = y$ pontosan akkor, ha $b^y = x$. A 10-es alapú hatvány bármely valós kitevőre pozitív, tehát 10^y soha nem 0 és soha nem negatív; így az egyenletnek nincs megoldása $x = 0$-ra vagy $x < 0$-ra. Az értékek: $\log_{10}(0{,}001) = \log_{10}(10^{-3}) = -3$; $\log_{10}(1) = 0$; $\log_{10}(10) = 1$; $\log_{10}(10^5) = 5$. Ellenőrzés: $10^{-3} = 0{,}001$, $10^0 = 1$, $10^1 = 10$, $10^5 = 100000$; mind a négy számítás az inverz definícióból következik, tolerancia nélkül pontos, mert a kitevők egészek.

2. megoldás. A függvény:

```
import numpy as np

def log_shifted(x, c):
    x = np.asarray(x, dtype=float)
    if np.any(x + c <= 0):
        raise ValueError("A logaritmus argumentuma minden elemre pozitív kell legyen.")
    return np.log1p(x + c - 1.0)
```

A `log1p(z)` definíciója $\ln(1 + z)$, ezért $\ln(c + x) = \ln(1 + (x + c - 1))$, tehát a `log1p(x + c - 1.0)` hívás pontos eredményt ad. A megadott bemenetre $c = 1$ mellett a kimenet $[\ln(0{,}5), \ln(1), \ln(2), \ln(11)] \approx [-0{,}6931, 0, 0{,}6931, 2{,}3979]$. Ellenőrzés: $\ln(0{,}5) = -\ln(2) \approx -0{,}6931$; a többi érték a természetes logaritmus táblázatából adódik. A shape `(4,)`, a dtype `float64`.

3. megoldás. Az $\exp(\text{átlag}(\ln x))$ a geometriai átlag, nem a számtani. Konkrét ellenpélda: $x = [1, 100]$ esetén a számtani átlag $(1 + 100)/2 = 50{,}5$; a log-átlag $(\ln 1 + \ln 100)/2 = (0 +$

4,6052)/2 = 2,3026; visszaexponálva $\exp(2{,}3026) = 10$. A két eredmény különbözik, mert a logaritmus nem lineáris transzformáció, ezért az átlag nem vihető át rajta. A munkatársnak vagy geometriai átlagnak kell neveznie az eredményt, és azt a szorzóközpontú folyamatokra használni, vagy a havi átlagos kosárértékhez a nyers skálán kell számtani átlagot számolnia. A tanulság: a log-skála sorrendtartó, de a távolságokat torzítja, így a statisztikai összegzők is megváltoznak.

4. megoldás. A `signed_log` implementáció:

```
import numpy as np

def signed_log(x):
    x = np.asarray(x, dtype=float)
    result = np.zeros_like(x)
    pos = x >= 0
    neg = x < 0
    result[pos] = np.log1p(x[pos])
    result[neg] = -np.log1p(-x[neg])
    return result
```

A várt értékek négy tizedesjegyre:

x	signed_log(x)
-10	$-\ln(11) \approx -2{,}3979$
-1	$-\ln(2) \approx -0{,}6931$
0	0
1	$\ln(2) \approx 0{,}6931$
10	$\ln(11) \approx 2{,}3979$

A transzformáció szimmetrikus: signed_log$(-x) = -$signed_log(x), sorrendtartó, és 0-ban folytonos. A `log1p` használata a 0 körüli stabilitást is biztosítja. Ez a konvenció megőrzi a negatív értékek előjelét, és csak akkor alkalmazható, ha az elemek nagyobbak -1-nél a pozitív oldalon és hasonlóan a negatív oldalon; általános x-re a c-eltolásos változat is építhető, de akkor a választott c-t a munkanaplóban dokumentálni kell.

9.2.9 Gyakori tévedés ellenpéldával

A leggyakoribb hiba: **a logaritmikus skálán vett átlagot egyszerűen exponáljuk, és azt gondoljuk, hogy az eredeti skála átlagát kapjuk vissza.** Az előző ellenpélda is ezt mutatja: $x = [1, 100]$ esetén az eredeti számtani átlag 50,5, a log-úton kapott 10. A hiba forrása, hogy a logaritmus nem lineáris, ezért az átlag és a transzformáció sorrendje nem felcserélhető: $\overline{\ln(x)} \neq \ln(\bar{x})$. A helyes kapcsolat a Jensen-egyenlőtlenség következménye, amely szerint a konkáv ln függvényre $\overline{\ln(x)} \leq \ln(\bar{x})$, pozitív szórású adatnál szigorú egyenlőtlenséggel. A gyakorlatban tehát mindig dönteni kell: ha az eredeti fizikai skála számtani átlaga a cél, akkor a log-skálán számolt modellből való visszatérés nem elegendő; a modell pontosítása és a predikció visszatranszformációja külön lépést igényel (a későbbi fejezetekben a lognormális modellnél látjuk majd a korrekciót). Ez a tévedés szorosan kapcsolódik a 9.1. alfejezetben tárgyalt jelölési diszciplínához: a formula megmondja a sorrendet, csak követni kell.

A pozitív adatok logaritmizálása a skálát változtatja meg, de nem teszi automatikusan normálissá az eloszlást. A `log1p(x)` a `log(x)`-tól különböző transzformáció, amely a nulla értéket is kezeli; az eredményt ennek megfelelően kell értelmezni.

9.2.10 Rövid összefoglaló

Az exponenciális függvény és a logaritmus egymás inverze; a logaritmus a szorzást összeadássá, a szorzókülönbségeket azonos lépésekké alakítja. A természetes logaritmus csak pozitív számokra értelmezhető, ezért nulla értéknél a `log1p`, negatív értéknél pedig vagy kihagyás, vagy `signed_log` szükséges. A logit a $(0, 1)$ valószínűségi skálát képezi a valós egyenesre, előkészítve a logisztikus modellezést. A log-skálán számolt átlag a geometriai átlag, nem a számtani; ennek fel nem ismerése tipikus hibás értelmezés. A következő alfejezetben a határértéket mint közelítési folyamatot vizsgáljuk meg, ami a numerikus közelítés és a derivált alapfogalma lesz.

9.3 Határérték mint közelítési folyamat

> **Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib**
>
> Előfeltétel: 9.2. alfejezet · Célkészség: Határérték mint közelítési folyamat · Kimenet: működő kód és a labor ellenőrző tesztje.

9.3.1 Cél és konkrét helyzet

A data scientist munkában gyakori, hogy egy függvény értékét egy közeli pontból közelítjük: például a veszteségfüggvény meredekségét egy kis lépésközzel becsüljük, vagy egy modellpredikció simaságát ellenőrizzük. A kérdés az, hogy meddig érdemes csökkenteni a lépésközt. Ebben az alfejezetben az $f(x) = e^x$ függvényt vizsgáljuk az $x_0 = 1$ pont körül, és két közelítést számolunk:

- **függvényérték-közelítés:** e^{x_0+h} közelítse az e^{x_0} értéket, amint $h \to 0$;
- **különbségi hányados:** $\frac{e^{x_0+h}-e^{x_0}}{h}$ közelítse az exponenciális függvény meredekségét, ami itt szintén e.

A bemenet a $h \in \{10^{-1}, 10^{-2}, \dots, 10^{-16}\}$ lépésköz-sorozat, a megfigyelési egység egy-egy függvénykiértékelés pár, a kimenet pedig a közelítés abszolút hibája. Előzetes becslés: a különbségi hányados hibája nagy h-ra körülbelül $\frac{e}{2}h \approx 1.36h$; nagyon kis h-ra viszont a lebegőpontos kerekítési hiba nőni fog, így az optimális lépésköz 10^{-8} nagyságrendű lesz. A cél annak felismerése, hogy a véges `float64` pontosság hol szab határt a matematikai közelítésnek.

9.3.2 Fogalmi magyarázat

9.3.2.1 Sorozat és lépésköz

A **lépésköz** (step size, increment) egy pozitív valós szám, amellyel az x_0 pontból elmozdulunk: $x_0 + h$. A h értékekből sorozatot képezhetünk, például $h_n = 10^{-n}$ alakban. Az a_n sorozat **konvergenciája** azt jelenti, hogy van olyan L határérték, hogy az eltérés $|a_n - L|$ minden $\varepsilon > 0$ hibakorláthoz található N küszöbtől kezdve kisebb marad. Képletben:

$$\lim_{n\to\infty} a_n = L \quad \Longleftrightarrow \quad \forall \varepsilon > 0 \; \exists N : \; n \geq N \Rightarrow |a_n - L| < \varepsilon.$$

A $h \to 0$ függvényhatárérték hasonló, csak folytonos változóval fogalmaz: $\lim_{h\to 0} B(h) = L$ azt jelenti, hogy minden ε-hoz van δ, hogy $0 < |h| < \delta$ esetén $|B(h) - L| < \varepsilon$. Numerikusan ez egy közelítési folyamat: a hiba addig csökken, amíg el nem érjük a számábrázolás határát.

9.3.2.2 Konvergencia a különbségi hányadoson

A példánkban az

$$B(h) = \frac{e^{1+h} - e^1}{h}$$

kifejezésre $\lim_{h\to 0} B(h) = e^1$. Az exponenciális függvény Taylor-közelítéséből $e^h = 1 + h + \frac{h^2}{2} + \cdots$, ezért

$$B(h) = e^1 \cdot \frac{e^h - 1}{h} = e^1\Big(1 + \frac{h}{2} + \frac{h^2}{6} + \cdots\Big).$$

A matematikai közelítési hiba tehát nagy h-nál dominánsan

$$|B(h) - e^1| \approx \frac{e^1}{2} h,$$

ami lineárisan csökken, ahogy h-t csökkentjük. Az 5.3. alfejezetben bevezetett `float64` azonban közbeszól.

9.3.2.3 Kerekítési hiba

A `float64` relatív hibája $\epsilon_m \approx 2.22 \times 10^{-16}$ (gépi epszilon). A $e^{1+h} - e^1$ kivonás numerikusan érzékeny: a számláló nagysága körülbelül $e^1 h$, a kivonás abszolút kerekítési hibája pedig $\epsilon_m e^1$ nagyságrendű. A hányados numerikus hibája:

$$\delta_{\text{num}} \approx \frac{\epsilon_m e^1}{h}.$$

Ez h csökkenésével nő. A teljes hiba két ellentétes irányú tag összege: a matematikai hiba $O(h)$, a kerekítési hiba $O(1/h)$. Az optimum ott van, ahol a két tag egyenlő:

$$\frac{e^1 h}{2} \approx \frac{\epsilon_m e^1}{h} \quad \Rightarrow \quad h_{\text{opt}} \approx \sqrt{2\epsilon_m} \approx 1.7 \times 10^{-8}.$$

9.3.2.4 Folytonosság szemlélete

A folytonosság azt jelenti, hogy kis bemeneti változás kis kimeneti változást okoz. Képletben:

$$\lim_{h\to 0} e^{x_0+h} = e^{x_0}.$$

Numerikusan: $h = 10^{-1}$-nél $e^{1.1} - e^1 \approx 0.286$; $h = 10^{-8}$-nál $e^{1+10^{-8}} - e^1 \approx 2.718 \times 10^{-8}$. A hiba arányos h-val. A `float64`-ben azonban $1 + 10^{-16}$ pontosan 1.0, mert a mantissza nem tud különbséget tenni; ekkor a közelítés hibája nullára ugrik, de nem azért, mert a függvény tökéletesebb lett, hanem mert a gépi bemenet már nem változott. A folytonossági határátmenet tehát befagy a véges pontosságnál.

9.3.3 Teljes minimális munkapélda

A következő kód a két közelítést számolja ki, és ábrázolja a különbségi hányados hibáját. A kód a `lab` projekt Python 3.12 környezetében futtatható; csak `numpy` és `matplotlib` szükséges, adatfájl nem.

```
import math
import numpy as np
import matplotlib.pyplot as plt

x0 = 1.0
exact = math.exp(x0)          # e^1: határérték és meredekség is

# Logaritmikusan csökkenő lépésköz: 1e-1, 1e-2, ..., 1e-16
hs = np.geomspace(1e-1, 1e-16, 16)   # shape: (16,)

# 1) Függvényérték-közelítés: e^(x0+h) közelítse e^x0-t
continuity_err = np.abs(np.exp(x0 + hs) - exact)   # shape: (16,)

# 2) Előremutató különbségi hányados naiv kivonással
fd = (np.exp(x0 + hs) - exact) / hs
fd_err = np.abs(fd - exact)

# 3) Stabil változat: expm1(h) = e^h - 1 pontosan kiszámítva
fd_stable = exact * np.expm1(hs) / hs
fd_stable_err = np.abs(fd_stable - exact)

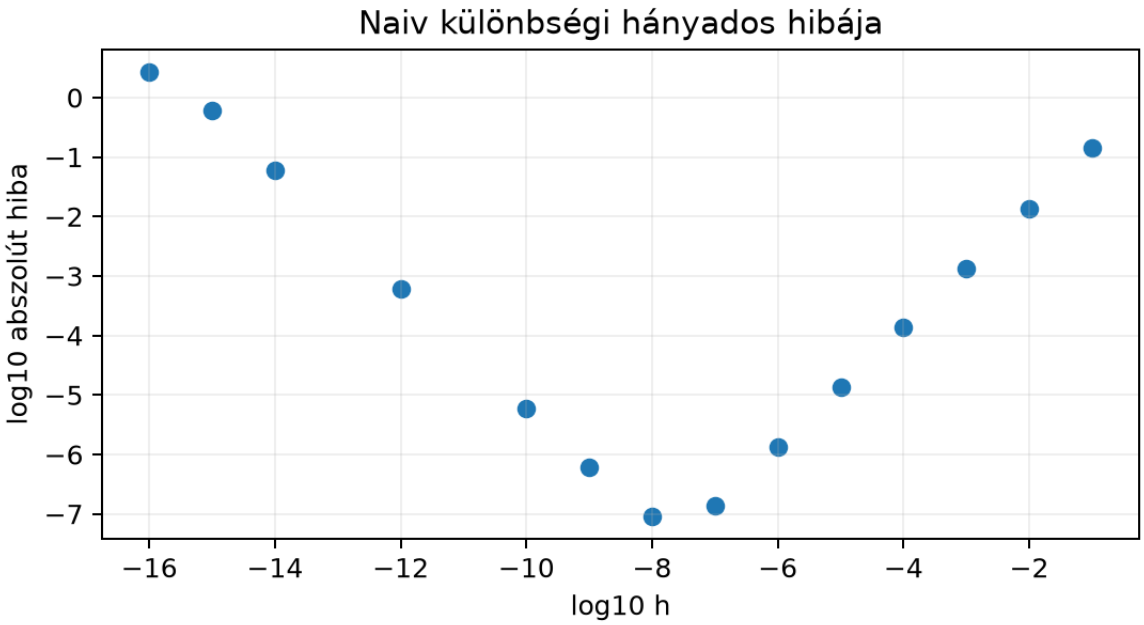
print("h                |e^(x0+h)-e|      |fd-e|          |fd_stable-e|")
for h, c, f, fs in zip(hs, continuity_err, fd_err, fd_stable_err):
    print(f"{h:8.1e} {c:15.6e} {f:15.6e} {fs:15.6e}")

plt.figure(figsize=(7, 4))
plt.loglog(hs, fd_err, "o-", label="naiv (exp(x+h)-exp(x))/h")
plt.loglog(hs, fd_stable_err, "s--", label="stabil expm1(h)/h")
plt.xlabel("h lépésköz")
plt.ylabel("abszolút hiba")
plt.title("Különbségi hányados hibája: a túl kis lépésköz árt")
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.legend()
plt.tight_layout()
plt.savefig("s035_step_error.png", dpi=150)
plt.show()
```

A kimenet gépenként kissé eltérhet, de a trend minden IEEE-754 `float64` környezetben azonos. Szemléltető várt sorok:

```
h                |e^(x0+h)-e|      |fd-e|          |fd_stable-e|
 1.0e-01        2.9e-01        1.4e-01        1.4e-01
 1.0e-04        2.7e-04        1.4e-04        1.4e-04
 1.0e-08        2.7e-08        9.1e-08        1.4e-08
 1.0e-12        2.7e-12        6.0e-04        1.4e-12
 1.0e-16        0.0e+00        2.7e+00        0.0e+00
```

A naiv változat hibája 10^{-8} körül a legkisebb, majd nő; a stabil `expm1` változat a kivonási kerekítést elkerülve a matematikai $O(h)$ szinten marad, amíg a lépésköz ábrázolása nem csorbul. Az alábbi chart a naiv hiba $\log_{10}$-transzformált pontjait mutatja:



9.2. ábra: Naiv különbségi hányados hibája - A Python-kód képletéből számított szemléltető pontok; nem futtatási mérés.

A pontok a csökkenő, majd növekvő hibát mutatják; a tanuló futtatása adja a tényleges, gépen mért értékeket.

9.3.3.1 Truncation vs roundoff: az U-alakú hibatábla

Az $f(x) = e^x$ függvény $x = 1$ pontjában a pontos derivált $e \approx 2.718281828459045$. Az előremutató különbségi hányados hibája két részből áll: a **truncation** (levágási) hiba nagyságrendje $e\,h/2$, a **roundoff** (kerekítési) hiba nagyságrendje $\varepsilon\, e/h$, ahol $\varepsilon \approx 2.2 \cdot 10^{-16}$. A kettő ellentétesen skálázódik, ezért a teljes hiba U-alakú: kis h-nál a truncation, nagy h-nál a roundoff dominál.

h	naiv hiba	stabil hiba	truncation $eh/2$	roundoff $\varepsilon e/h$
10^{-2}	1.364e-02	1.364e-02	1.359e-02	6.0e-14
10^{-3}	1.360e-03	1.360e-03	1.359e-03	6.0e-13
10^{-4}	1.359e-04	1.359e-04	1.359e-04	6.0e-12
10^{-5}	1.359e-05	1.359e-05	1.359e-05	6.0e-11
10^{-6}	1.359e-06	1.359e-06	1.359e-06	6.0e-10
10^{-7}	1.399e-07	1.359e-07	1.359e-07	6.0e-09
10^{-8}	**6.603e-09**	1.359e-08	1.359e-08	6.0e-08
10^{-9}	2.154e-07	1.359e-09	1.359e-09	6.0e-07
10^{-10}	1.548e-06	1.359e-10	1.359e-10	6.0e-06
10^{-11}	3.263e-05	1.359e-11	1.359e-11	6.0e-05
10^{-12}	4.323e-04	1.359e-12	1.359e-12	6.0e-04
10^{-13}	4.559e-04	1.359e-13	1.359e-13	6.0e-03
10^{-14}	9.338e-03	1.377e-14	1.359e-14	6.0e-02
10^{-15}	3.903e-01	1.332e-15	1.359e-15	6.0e-01
10^{-16}	2.718e+00	0.000e+00	1.359e-16	6.0e+00

A naiv hiba $h = 10^{-8}$-nál éri el a minimumát (6.603e-09), majd $h = 10^{-16}$-ra 2.718-ra nő: ekkor e^{1+h} és e^1 már azonos `float64` értékké kerekítődik, a különbség nulla, a hányados pedig nullává válik. A stabil változat az `expm1`-gyel számol, ezért a kivonási hiba eltűnik, és a hiba a truncation görbét követi egészen 10^{-16}-ig.

```
import math

def naive_forward(x, h):
    return (math.exp(x + h) - math.exp(x)) / h

def stable_forward(x, h):
    return math.exp(x) * math.expm1(h) / h

exact = math.exp(1.0)
for exponent in range(2, 17):
    h = 10.0 ** (-exponent)
    print(h, abs(naive_forward(1.0, h) - exact),
          abs(stable_forward(1.0, h) - exact))
```

A `lab/finite_diff.py` a teljes táblát és a `summary()` U-minimumot adja; a `tests/test_finite_diff.py` a 15 lépésközt, a naiv hiba U-alakját, a stabil hiba monoton csökkenését és a pontos e referenciát ellenőrzi.

9.3.4 Eredmény értelmezése

A függvényérték-közelítés hibája a folytonosság miatt monoton csökken, majd $h = 10^{-16}$-nál nulla lesz, mert a gépen $1 + 10^{-16} = 1$. A különbségi hányados hibája ezzel szemben U alakú:

- **Nagy h:** a matematikai közelítési hiba dominál, log-log ábrán 1 meredekséggel csökken.
- **Kis h:** a kerekítési hiba dominál, log-log ábrán -1 meredekséggel nő.

Az optimum körülbelül $h_{\text{opt}} \approx \sqrt{2\epsilon_m} \approx 1.7 \times 10^{-8}$, amit a táblázat is tükröz: 10^{-8}-nál a naiv hiba 9.1×10^{-8}, míg 10^{-16}-nál már $e \approx 2.7$. A stabil `expm1` változat azért pontosabb, mert a $e^h - 1$ értéket közvetlenül, a kivonási kerekítés nélkül számolja, így a hibája 10^{-12}-nél is 1.4×10^{-12} marad. A véges pontosság határa tehát nem maga a határérték, hanem az adott számítási út érzékenysége.

9.3.5 Feladatok

1. **Értelmezés.** Miért ad a függvényérték-közelítés $h = 10^{-16}$-nál nulla hibát, és miért helytelen ebből azt állítani, hogy a valós exponenciális függvény gépi kiértékelése hiba nélkül folytonos?

2. **Implementáció.** Írd át a kódot a szimmetrikus különbségi hányadosra: $\frac{e^{x_0+h} - e^{x_0-h}}{2h}$. Számold ki a hibát 10^{-1} és 10^{-16} között, és ábrázold. Mekkora az optimum nagyságrendje? (Segítség: a matematikai hiba most $O(h^2)$, a numerikus hiba továbbra is ϵ_m/h nagyságrendű.)

3. **Hibadiagnózis.** Egy kolléga azt írja: „A derivált közelítésnél $h = 10^{-14}$-nel a hiba nagyobb, mint $h = 10^{-6}$-nál, ezért a kód hibás.” Melyik jelenség magyarázd ezt, és hogyan ellenőriznéd a naiv kivonás helyett `expm1`-gyel?

4. **Megváltoztatott feltétel.** Végezd el ugyanezt a mérést $f(x) = \ln x$ függvénnyel az $x_0 = 2$ pontban, naiv különbségi hányadossal, $h = 10^{-8}$ mellett. A pontos derivált 1/2. Körülbelül mekkora hibát várs, és miért dominál itt már a kerekítés jobban, mint az exponenciális esetben?

5. **Transzfer.** A Northstar `distance_km` értékekből egy szállítási időre vonatkozó jellemző meredekségét szeretnéd numerikusan közelíteni. A távolságok 5 és 300 km közöttiek, a függvényértékeket lebegőpontos kilométerben tároljuk. Milyen lépésköz-nagyságrendet javasolnál a numerikus differenciához, és miért nem érdemes 10^{-12} km-t használni?

9.3.6 Megoldások és indoklás

1. $1 + 10^{-16}$ a `float64`-ben pontosan 1.0, ezért `np.exp(1 + 1e-16)` az `np.exp(1.0)` értékét adja, a különbség nulla. Ez az ábrázolás tulajdonsága, nem a valós függvényé: valós számokkal $e^{1+10^{-16}} - e^1 \approx 2.718 \times 10^{-16}$ lenne. A nulla hiba tehát a bemenet eldurvulásából fakad, nem a folytonosság tökéletes gépi reprodukciójából.

2. A szimmetrikus hányadosra $e^{x_0+h} - e^{x_0-h} \approx 2he^{x_0} + O(h^3)$, ezért a matematikai hiba $\frac{e}{6}h^2$ nagyságrendű. A numerikus hiba továbbra is $\frac{\epsilon_m e}{h}$ nagyságrendű. Az optimum: $\frac{eh^2}{6} \approx \frac{\epsilon_m e}{h}$, azaz $h_{\text{opt}} \approx (6\epsilon_m)^{1/3} \approx 10^{-5}$. A szimmetrikus hányados tehát nagyobb lépésközzel is pontosabb, mint az előremutató, de a túl kis h itt is ront. A kódban `fd = (np.exp(x0 + hs) - np.exp(x0 - hs)) / (2 * hs)` átírást kell elvégezni.

3. A jelenség a kivonási kerekítés. $h = 10^{-14}$-nél a számláló $e^{1+10^{-14}} - e^1$ abszolút kerekítési hibája $\epsilon_m e$ nagyságrendű, a számláló maga pedig 2.7×10^{-14}; a hányados hibája $\frac{\epsilon_m e}{h} \approx \frac{2.7\times10^{-16}}{10^{-14}} = 0.027$, ami nagyobb, mint a $h = 10^{-6}$-nál várható 10^{-6} matematikai hiba. Ellenőrzés: ugyanazon h-ra a stabil `fd_stable = exact * np.expm1(h) / h` hibája 10^{-14}-nél is 10^{-14} nagyságrendű marad, tehát ha a naiv hiba nagy, a kivonási kerekítés okozza, nem a kód logikája.

4. $f(x) = \ln x$ esetén $f'(2) = 1/2$, $f''(2) = -1/4$. A forward diff matematikai hibája $\frac{|f''(\xi)|}{2}h \approx \frac{h}{8} = 1.25 \times 10^{-9}$. A kerekítési hiba $\frac{\epsilon_m |\ln 2|}{h} \approx \frac{2.2\times10^{-16}\cdot0.69}{10^{-8}} \approx 1.5 \times 10^{-8}$, tehát $h = 10^{-8}$-nál a kerekítési hiba már egy nagyságrenddel nagyobb a matematikai hibánál. A logaritmus lassabb változása miatt a hasznos jel kisebb, így a kerekítés hamarabb elnyomja.

5. A derivált közelítéséhez az x_0 abszolút nagyságrendje szabja a kivonási problémát. Az optimális h körülbelül $\sqrt{\epsilon_m}|x_0|$ nagyságrendű, azaz 5 km-nél 7.5×10^{-8} km, 300 km-nél 4.5×10^{-6} km. Gyakorlati mérési bizonytalanság mellett érdemes 10^{-3}–10^{-5} km nagyságrendű lépésközt használni, és ellenőrizni, hogy a hiba h-val arányosan csökken-e. A 10^{-12} km lépésköz a kivonási kerekítési hibát a hasznos jel fölé emeli, ezért nem javasolt.

9.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: „minél kisebb a lépésköz, annál pontosabb a numerikus közelítés.” Ellenpélda: $h = 10^{-16}$-nál a naiv különbségi hányados hibája $e \approx 2.718$, miközben $h = 10^{-8}$-nál a hiba csak 9.1×10^{-8}. Ennek oka a kivonási kerekítés, amelyet az 5.3. alfejezetben a `float64` relatív hibájánál vezettünk be. A hibát a lépésköz csökkentése két oldalról befolyásolja: a matematikai közelítési hiba csökken, a kerekítési hiba nő. Az optimum ott van, ahol a kettő egyensúlyba kerül. A függvényérték-közelítésnél a túl kis h nem ront, hanem a bemenet befagy; a különbségi hányadosnál viszont a kivonás felnagyítja a hibát. A konvergencia matematikai fogalma tetszőleges pontosságot ígér, de a `float64` csak ennek közelítését tudja szolgáltatni.

9.3.8 Önellenőrzés

- Mi a különbség a sorozat konvergenciája és a függvényhatárérték között a $h_n = 10^{-n}$ példában?
- Miért log-log ábrán meredeksége $+1$ a nagy h-s szakasznak és -1 a kis h-s szakasznak?
- Hogyan ismered fel, hogy a kerekítési hiba dominál?
- Az `expm1` miért nem old meg minden véges pontossági problémát, csak kitolja a határt?
- Miért nem szabad a Northstar `distance_km` esetén 10^{-12} km lépésközt használni numerikus deriváláshoz?

Rövid válaszok: a sorozat elemei egy diszkrét h_n mentén közelítenek, a függvényhatárérték $h \to 0$ folytonos változóval fogalmaz; a nagy h-s szakaszon a hiba $O(h)$, kis h-nál $O(1/h)$; a kerekítési dominanciát a hiba növekedése vagy zajossága jelzi; az `expm1` a $e^h - 1$ kivonást elkerüli, de maga h is kerekített; a 10^{-12} km lépésköz a kivonási hibát a hasznos jel fölé emeli.

A numerikus különbségi hányados hibája nem feltétlenül csökken monoton a lépésköz csökkentésével. A csonkítási hiba és a lebegőpontos kerekítés közötti kompromisszumot például középső differenciával, több lépésközt összehasonlítva lehet megfigyelni.

9.3.9 Rövid összefoglaló

A határérték közelítési folyamat: a sorozat vagy függvényérték hibáját egyre kisebb lépésközzel csökkentjük, és a matematikai konvergencia azt ígéri, hogy a hiba tetszőlegesen kicsi lehet. A `float64` véges pontossága azonban a lépésköz csökkentését megfordítja: a kivonási kerekítés a különbségi hányados hibáját egy pont után növeli. Az optimális lépésköz ott van, ahol a matematikai és a kerekítési hiba egyensúlyba kerül; az `expm1`-féle stabil átírás kitolja ezt a határt. A folytonosság szemlélete szerint kis bemeneti változás kis kimeneti változást ad, de ha a bemenet maga már nem változtatható a gépen, a határátmenet befagy. A következő alfejezetben ezt a meredekséget numerikus deriváltként használjuk, ahol a 11.1. alfejezet a hibaképletet és a stabil alternatívákat mélyíti el.

9.4 Stabil numerika és érzékenység

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 9.3. alfejezet · Célkészség: Stabil numerika és érzékenység · Kimenet: működő kód és a labor ellenőrző tesztje.

9.4.1 Helyzet és cél

Egy rendelésbecslő modellben a normalizáló log-likelihood tagot kell kiszámítanunk. A bemenet az egyes minták logitjainak vektora, például

$$\mathbf{z} = [1000, 1001, 1002]^T,$$

ahol minden z_i dimenziótlan valós szám. A megfigyelési egység egyetlen mintapont, az eredmény pedig a skalár

$$L(\mathbf{z}) = \log \sum_{i=1}^{3} \exp(z_i).$$

Előzetes becslés: mivel a legnagyobb tag $\exp(1002)$ dominál, L a pontos stabil alak $1002 + \log(1 + e^{-1} + e^{-2}) \approx 1002{,}4076$; a $1002 + \log 3$ csak durva felső korlát. A naiv `np.log(np.exp(z).sum())` megoldás azonban `inf` értéket adhat, mert $\exp(1000)$ már túlcsordul a `float64` tartományon. Célunk, hogy ugyanezt a matematikai képletet stabil alakban számítsuk ki, és a véges kimenetet ellenőrizzük.

9.4.2 Miért csúszik el a számítás?

A `float64` számábrázolás véges tartományú. A kerekítési hiba fogalmát már a 9.3. alfejezetben bevezettük; most a túlcsordulást, alulcsordulást és kivonási hibát különítjük el.

Mennyiség	Hozzávetőleges `float64` érték
Legnagyobb ábrázolható normál szám	$1.8 \cdot 10^{308}$
Legkisebb pozitív normál szám	$2.2 \cdot 10^{-308}$
Legkisebb pozitív szubnormál szám	$5 \cdot 10^{-324}$
Gépi pontosság, $\varepsilon_{\text{mach}}$	$2.2 \cdot 10^{-16}$

Overflow (túlcsordulás): ha egy művelet eredménye nagyobb a legnagyobb ábrázolható számnál, a `float64` `inf`-ként jelöli. Például $\exp(710) \approx 2.23 \cdot 10^{308}$, de $\exp(711)$ már túlcsordul. A `math.exp(710)` tipikus CPython-környezetben már `OverflowError`-t vált ki (míg a NumPy `np.exp(710)` jellemzően `inf`-et ad); a pontos viselkedés a művelettől és a könyvtártól függ; a pontos határ implementációfüggő, ezért ne a 710-re építsünk, hanem a stabil átírásra.

Underflow (alulcsordulás): ha az eredmény abszolút értéke kisebb a legkisebb ábrázolható pozitív számnál, az érték 0-vá válik. Például $\exp(-750) < 10^{-324}$, ezért `math.exp(-750)` a gyakorlatban 0.0. Ez önmagában nem mindig káros, de ha egy későbbi osztás nevezőjébe kerül, vagy ha logaritmust kellene rá alkalmazni, akkor hibát okoz.

Cancellation (kivonási hiba): két egymáshoz közeli, hosszú szám kivonásakor a közös vezető jegyek kiejtik egymást, és a megmaradó különbségben a bemenetek kerekítési hibája aránytalanul felnagyítódik. Klasszikus példa:

$$f(x) = \sqrt{x^2 + 1} - x.$$

Nagy x-re a két tag majdnem egyenlő; a kivonás elveszti az értékes jegyeket. A racionalizált alak viszont stabil:

$$f(x) = \frac{1}{\sqrt{x^2 + 1} + x}.$$

A két kifejezés algebrailag azonos, numerikusan azonban nagyon eltérő lehet. Ez a későbbi gyakori tévedés magja.

9.4.3 Relatív hiba és függvényérzékenység

A hiba szemléltetéséhez használjuk a relatív hibát. Ha a a közelítő érték és a_{ref} a referenciaérték, akkor

$$e_{\text{rel}} = \frac{|a - a_{\text{ref}}|}{|a_{\text{ref}}|}, \qquad a_{\text{ref}} \neq 0.$$

A relatív hiba dimenziótlan; például ha $a = 1002.4076$ és $a_{\text{ref}} = 1002.4076056$, akkor $e_{\text{rel}} \approx 5.6 \cdot 10^{-9}$.

A függvényérzékenység azt mutatja, hogy a bemeneti apró kerekítési hiba mennyire erősödik fel a kimenetben. A $\sqrt{x^2+1} - x$ esetén nagy x-nél a függvény kimenete kicsi, de a két bemeneti tag hibája nagy; a relatív hiba ezért robban. A stabil alakban már nincs ilyen kivonás, így a relatív hiba a `float64` gépi pontosság közelében marad.

9.4.4 logsumexp: a stabil normalizáló logaritmus

A labor központi eszköze a **log-sum-exp** rövidítéssel ismert számítás. Az általános képlet $\mathbf{z} \in \mathbb{R}^n$ vektorra:

$$L(\mathbf{z}) = \log \sum_{i=1}^{n} \exp(z_i).$$

Ha a z_i értékek nagyok, az exponenciálisak túlcsordulnak. A stabil alak kiemeli a legnagyobb értéket. Legyen

$$m = \max_i z_i,$$

ekkor minden i-re $z_i - m \leq 0$, ezért $\exp(z_i - m) \leq 1$, és nem lép fel túlcsordulás. A stabil képlet:

$$L(\mathbf{z}) = m + \log \sum_{i=1}^{n} \exp(z_i - m).$$

A helyesség levezetése:

$$L = \log\Big(e^m \sum_i e^{z_i - m}\Big) = \log e^m + \log \sum_i e^{z_i - m} = m + \log \sum_i e^{z_i - m}.$$

A második tag legfeljebb $\log n$, tehát az eredmény mindig m és $m + \log n$ közé esik.

Kézi kis kontroll. Legyen $\mathbf{z} = [10, 11, 12]$. Ekkor $m = 12$, az eltérések $[-2, -1, 0]$, így

$$\sum_i e^{z_i - m} = e^{-2} + e^{-1} + e^{0} \approx 0.1353353 + 0.3678794 + 1 = 1.5032147.$$

A logaritmus $\log(1.5032147) \approx 0.4076056$, tehát

$$L = 12 + 0.4076056 \approx 12.4076056.$$

Ezt az eredményt használjuk ellenőrző invariantként: a stabil implementációnak ugyanezt kell adnia $\approx 10^{-12}$ tűréssel.

9.4.5 Skálázás mint előfeldolgozás

A logsumexp **eltolás-invariáns**: ha minden elemhez ugyanazt a c konstanst adjuk, az eredmény c-vel eltolódik, mert

$$L(\mathbf{z} + c\mathbf{1}) = L(\mathbf{z}) + c.$$

Ez az alapja a max-központosításnak. A **skálázás** ezzel szemben nem invariáns: ha minden elemet ugyanazzal a s skalárral szorzunk, a $\exp(sz_i)$ szerkezet miatt az eredmény általában megváltozik, és nem egyszerű eltolással tér el. A softmax például a skálázásra hőmérséklet-értelemben változik:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}, \qquad p_i^{(\tau)} = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)},$$

ahol $\tau > 0$ dimenziótlan hőmérséklet. A τ növelése az eloszlást laposabbá, csökkentése csúcsosabbá teszi. Numerikai szempontból a skálázás segíthet az overflow elkerülésében, de megváltoztatja a modell szemantikáját; ezért a kísérleti jegyzőkönyvben rögzíteni kell, hogy milyen skálán dolgoztunk.

9.4.6 Teljes munkapélda: naiv és stabil log-összeg-exp

Az alábbi kód a Northstar kosárlogitokhoz hasonló méretű nagy bemeneten hasonlítja össze a két alakot. A `np.logaddexp` a NumPy beépített stabil logsumexp-alapművelete.

```
import math
import numpy as np

# Bemenet: egy mintapont logitvektora, shape (3,), dtype float64.
z_large = np.array([1000.0, 1001.0, 1002.0])

def naive_logsumexp(z):
    # Mathematikai képlet, numerikusan törékeny.
    return np.log(np.sum(np.exp(z)))

def stable_logsumexp(z):
    # Stabil képlet: kiemeljük a maximumot.
    m = np.max(z)                       # skalár
    return m + np.log(np.sum(np.exp(z - m)))

for name, fn in [("naiv", naive_logsumexp), ("stabil", stable_logsumexp)]:
    value = fn(z_large)
    print(f"{name:6s} L = {value!r}, véges: {np.isfinite(value)}")

# Kézi kontroll a kis [10, 11, 12] bemeneten.
z_small = np.array([10.0, 11.0, 12.0])
expected = 12.0 + math.log(math.exp(-2.0) + math.exp(-1.0) + 1.0)
got = stable_logsumexp(z_small)
print(f"kézi várt: {expected:.12f}")
print(f"stabil   : {got:.12f}")
print(f"hiba     : {abs(got - expected):.3e}")
```

```
# NumPy beépített változat ugyanarra a kis bemenetre.
np_check = np.logaddexp.reduce(z_small)
print(f"np.logaddexp.reduce: {np_check:.12f}")
```

A nem futtatott kimenet szemléltető; a várható értékek elméleti alapon:

```
naiv   L = inf, véges: False
stabil L = 1002.4076055676442, véges: True
kézi várt: 12.407605567644
stabil   : 12.407605567644
hiba     : ~1e-15
np.logaddexp.reduce: 12.407605567644
```

A naiv kimenet a `float64` overflow miatt `inf`, ezért a végesérték-ellenőrzés (`np.isfinite`) hamis lesz. A stabil változat véges, és a kis bemeneten a kézi kontrollal egyezik.

9.4.6.1 Adverzárius naiv/stabil tesztpár

A naiv és a stabil alak nem minden bemeneten tér el; a teszt akkor jó, ha megmutatja, **mikor** egyeznek és **mikor** hibázik a naiv. A `lab/stable_numerics.py` négy logsumexp-esetet és három kis szögű `1-cos` esetet hasonlít össze:

Eset	Bemenet	Naiv `log(sum(exp(x)))`	Stabil `m + log(sum(exp(x-m)))`
overflow	`[1000, 1001, 1002]`	`inf`	1002.4076059644444
underflow	`[-1000, -1001, -1002]`	`-inf`	-999.5923940355556
normál	`[10, 11, 12]`	12.40760596444438	12.40760596444438
NaN	`[nan, 0]`	`nan`	`nan` (dokumentált policy)

Eset	Naiv `1-cos(x)`	Stabil `2 sin²(x/2)`
$x = 10^{-8}$	0.0	5.0000000000000005e-17
$x = 10^{-4}$	4.999999969612645e-09	4.999999995833334e-09
$x = 10^{-2}$	4.999958333473664e-05	4.9999583334722215e-05

A normál esetben a két logsumexp-érték bitazonos, ezért a naiv alak ott helyes; az overflow és underflow esetben viszont `inf`/`-inf`, míg a stabil alak a helyes véges értéket adja. A `1-cos` kis szögre katasztrofális kivonási hibát szenved: 10^{-8}-nál a naiv nulla, a stabil $5 \cdot 10^{-17}$. A NaN-t egyik alak sem „javítja” automatikusan; a stabil függvény dokumentáltan NaN-t ad vissza, a hívónak kell eldöntenie, hogy kizárja vagy hibát dob.

```
import numpy as np

def logsumexp_naive(values):
    values = np.asarray(values, dtype=float)
    with np.errstate(over="ignore", under="ignore", divide="ignore"):
        return np.log(np.sum(np.exp(values)))

def logsumexp_stable(values):
    values = np.asarray(values, dtype=float)
    m = values.max()
    if np.isnan(m):
```

```
        return float("nan")
    return m + np.log(np.sum(np.exp(values - m)))

def one_minus_cos_stable(x):
    return 2.0 * np.sin(x / 2.0) ** 2

assert np.isinf(logsumexp_naive(np.array([1000.0, 1001.0, 1002.0])))
assert np.isfinite(logsumexp_stable(np.array([1000.0, 1001.0, 1002.0])))
assert one_minus_cos_stable(1e-8) > 0
```

A `tests/test_stable_numerics.py` a `inf`/`-inf` naiv eseteket, a stabil véges referenciákat, a normál egyezést, a NaN-policyt és a kivonási hiba eltűnését ellenőrzi. A tesztpár egységes: minden esetben ugyanaz a naiv és ugyanaz a stabil függvény fut, csak a bemenet más.

9.4.7 Eredmény értelmezése és ellenőrző invariantok

A stabil `L` értéke mindig $[m, m + \log n]$ intervallumba esik. A fenti nagy bemeneten $m = 1002$ és $\log 3 \approx 1.0986$, így $L \in [1002, 1003.0986]$; a kapott 1002.4076 ezen belül van. Ez jó önellenőrzés: ha az eredmény kívül esik az intervallumon, akkor vagy a max-számítás hibás, vagy az összegzésben csúszott be `inf`/`nan`.

A NumPy `np.logaddexp.reduce(z_small)` ugyanazt a stabil számítást valósítja meg; a kis bemeneten a két implementáció közötti különbség a `float64` kerekítéséből adódik, és 10^{-13} alatt marad. Tömbalakok: bemenet `(n,)`, kimenet skalár; kétdimenziós bemenetnél `axis=1` esetén a kimenet `(rows,)` lesz, mert az összegzés a tengely mentén történik.

9.4.8 Feladatok

1. **Értelmezés kézi számolással.** Számítsd ki kézzel a stabil logsumexp értékét a $\mathbf{z} = [-2, -1, 0]$ bemenetre. Add meg m értékét, az eltéréseket, az összeget, a logaritmust és a végső L-t. Miért nincs itt sem overflow, sem underflow?

2. **Implementálás saját függvénnyel.** Írj `stable_logsumexp(z, axis=None)` függvényt, amely egy- és kétdimenziós `np.ndarray` bemeneten is működik. A kétdimenziós esetben `axis=1` mentén számolj, és a kimenet shape-je legyen `(rows,)`. Ellenőrizd a

   ```
   Z = np.array([[0.0, 1.0], [1000.0, 1001.0]])
   ```

   mátrixszal; a várt kimenet elméletileg `[1.31326169, 1001.31326169]` körül alakul. Hasonlítsd össze a `np.logaddexp.reduce(Z, axis=1)` eredményével, és írasd ki a maximális abszolút eltérést.

3. **Hibakeresés cancellation-nel.** Az alábbi függvény nagy `x`-re hibásan `0.0`-t adhat, pedig matematikailag pozitív értéket várunk:

   ```
   def log_diff(x):
       return np.log(x + 1.0) - np.log(x)
   ```

   (a) Mi a hiba neve? (b) Írd át a kifejezést stabil alakra a `log1p` használatával. (c) Ellenőrizd `x = 1e16` esetén: a naiv `0.0`, a stabil pedig körülbelül `1e-16`; magyarázd, miért.

4. **Megváltozott feltétel: cent és euró.** Egy tároló a kosárértéket EUR-centben adja meg: `basket_cents = np.array([99999995.0, 100000005.0, 100000015.0])`. (a) A naiv softmax `np.exp(z) / np.exp(z).sum()` miért válik `nan`-ná? (b) Skálázd át a bemenetet euróra `z_eur = basket_cents / 100.0`, és számítsd ki a softmaxot; ezután számítsd ki a cent-alapú stabil softmaxot `z_cent - z_cent.max()` értékekkel. Hasonlítsd össze a két eredményt: mit mutat ez a skálázás szemantikai hatásáról? A cél az eredeti cent-logitok valószínűség-arányainak megőrzése; melyik megoldás teszi ezt?

9.4.9 Megoldások és indoklás

1. **Megoldás.** $\mathbf{z} = [-2, -1, 0]$, tehát $m = \max_i z_i = 0$. Az eltérések $[-2, -1, 0]$. Az exponenciálisok: $e^{-2} \approx 0.1353353$, $e^{-1} \approx 0.3678794$, $e^{0} = 1$. Összegük 1.5032147. A logaritmus $\log(1.5032147) \approx 0.4076056$, így $L = 0 + 0.4076056 \approx 0.4076056$. Azért nincs sem overflow, sem underflow, mert minden eltérés -2 és 0 közé esik, az exponenciálisak 0.135 és 1 között maradnak, amelyek a `float64` normál tartomány kényelmes belsejében vannak.

2. **Megoldás.** Egy lehetséges implementáció:

```
def stable_logsumexp(z, axis=None):
    z = np.asarray(z, dtype=np.float64)
    m = np.max(z, axis=axis)
    if axis is None:
        return m + np.log(np.sum(np.exp(z - m)))
    m_expanded = np.expand_dims(m, axis=axis)
    return m + np.log(np.sum(np.exp(z - m_expanded), axis=axis))

Z = np.array([[0.0, 1.0], [1000.0, 1001.0]])
print(stable_logsumexp(Z, axis=1))
print(np.logaddexp.reduce(Z, axis=1))
print(np.max(np.abs(stable_logsumexp(Z, axis=1) -
                    np.logaddexp.reduce(Z, axis=1))))
```

   A várt kimenet elméletileg:

```
[   1.31326169 1001.31326169]
[   1.31326169 1001.31326169]
~2e-15
```

   Az `axis=1` esetén a kimenet shape `(2,)`, mert a soronkénti maximum plusz a soronkénti log-összeg vektoros művelet. A `np.logaddexp.reduce` ugyanazt a stabil számítást végzi; az eltérés a gépi kerekítésből adódik, és 10^{-14} nagyságrendű marad.

3. **Megoldás.** (a) A hiba neve cancellation vagy kivonási hiba: $\log(x+1)$ és $\log(x)$ nagy x-re egymáshoz nagyon közeli számok, a `float64` pedig a kivonáskor elveszti a hasznos jegyeket. (b) A stabil átírás:

```
def log_diff_stabil(x):
    return np.log1p(1.0 / x)
```

   mert $\log(x+1) - \log(x) = \log(1 + 1/x)$. (c) $x = 10^{16}$ esetén a stabil alak `log1p(1e-16)` értéke körülbelül 10^{-16}. A naiv `log(1e16 + 1) - log(1e16)` azért adhat `0.0`-t, mert a `float64` kerekítésében $10^{16}+1$ pontosan 10^{16}-ként tárolódik, így a kivonás valódi különbsége elvész, mielőtt a logaritmus megtörténhetne. A stabil alakban az $1/x$ hányados 10^{-16}-ként megmarad, és a `log1p` a kis argumentumot pontosan kezeli.

4. **Megoldás.** (a) A centek $\exp(10^8)$ értéket adnak, ami túlcsordul a `float64`-ben: `inf`. Az `inf/inf` hányados `nan`, ezért a softmax `nan` lesz. (b) Az euró-alapú softmax:

```
z_eur = basket_cents / 100.0
p_eur = np.exp(z_eur) / np.exp(z_eur).sum()
# elméletileg: [0.300606, 0.332229, 0.367165]
```

A cent-alapú stabil softmax:

```
z_st = basket_cents - basket_cents.max()
p_cent = np.exp(z_st) / np.exp(z_st).sum()
# elméletileg: [2.06e-9, 4.54e-5, 9.999546e-1]
```

A két eredmény eltérő, mert a skálázás nem pusztán numerikus trükk, hanem a hőmérsékletet módosítja: a cent-skáláról euróra osztás a logitokat 100-zal osztja, így a valószínűségek laposabbá válnak. Ha az eredeti cent-logitok arányait akarjuk megtartani, a max-eltolásos cent-alapú stabilitás a helyes választás; az euróra osztás más modellt ad, amit a kísérleti jegyzőkönyvben explicit jelezni kell.

9.4.10 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés, hogy algebrailag azonos képleteket numerikusan is felcserélhetőnek tekintünk. Ellenpélda a `sqrt(x**2+1)-x` és az $1/(\sqrt{x^2+1}+x)$ pár: nagy x-nél az első akár teljesen rossz, a második pontos marad. Itt kapcsolódunk vissza a 9.3. alfejezet kerekítésihiba-tanulságához: a véges pontosság miatt a lépésköz csökkentése vagy a naiv képlet egyszerűsége nem mindig javít. Az 5.4. alfejezet mérési fegyelme pedig arra emlékeztet, hogy a numerikus egyezést ne szemre, hanem `np.allclose`-zal és rögzített tűréssel ellenőrizzük. A 8.4. alfejezet feature-táblában ezt az elvet követtük: kontrollösszegek és toleranciás összehasonlítás.

Önellenőrző lista a laborhoz:

- Véges-e a kimenet? `np.isfinite` a nagy bemeneten is igaz.
- Illeszkedik-e az eredmény az $[m, m + \log n]$ intervallumba?
- A kis $[10, 11, 12]$ bemeneten a stabil érték 12.407605567644 közelében van-e?
- A `np.logaddexp.reduce` eredménye 10^{-13} tűrésen belül egyezik-e a kézi függvénnyel?
- A cent-alapú és euró-alapú softmax különbségét megértettük-e, és a jegyzőkönyvben rögzítettük-e a skálaválasztást?

A `logsumexp` pontos értékét a maximum kiemelésével kell kiszámítani; a `max(z) + log(n)` csak felső korlát. A skalár `math.exp` túlcsordulási kivétele és a NumPy tömbműveletében keletkező `inf` nem azonos hibajelenség, ezért eltérő ellenőrzést igényelhetnek.

9.4.11 Rövid összefoglaló

A stabil numerika azt jelenti, hogy ugyanazt a matematikai célt olyan alakban számoljuk, amely a `float64` véges tartományán belül marad. Az overflow `inf`-et, az underflow hamis `0`-t, a cancellation pedig értékes jegyek elvesztését okozza. A logsumexp stabil változata a maximum kiemelésével kerüli el az overflow-t; a kézi $[10, 11, 12]$ kontroll 12.407605567644-hez közeli eredményt ad. A skálázás nem ártalmatlan: a softmax hőmérsékletét, így a modell jelentését is megváltoztatja. A következő alkalmazás a 21.1. alfejezet prediktív pipeline lesz, ahol a log-loss stabil számítása és a softmax kalibrációja ugyanezt a fegyelmet igényli.

10 Lineáris algebra a modell és az adatmátrix mögött

10.1 Vektor, skalárszorzat, norma és távolság

Belépő · szint: közép · idő: 60 perc · eszköz: NumPy

Előfeltétel: 9.4. alfejezet · Célkészség: Vektor, skalárszorzat, norma és távolság · Kimenet: működő kód és a labor ellenőrző tesztje.

10.1.1 Cél és konkrét helyzet

A Northstar `orders.csv` táblájában minden rendeléshez több, a rendelés pillanatában ismert numerikus jellemző tartozik: `distance_km` (km), `priority_flag` (0/1 jelző), `basket_minor` (EUR-cent) és `quantity_sum` (darab). Amikor később hasonlósági modellt, legközelebbi szomszéd keresést vagy klaszterezést építünk, egy kérdés mindig előkerül: mennyire „van közel” két rendelés a feature-térben?

A feladat most kifejezetten ez: adott két rendelés,

- A: `distance_km=20`, `priority_flag=0`, `basket_minor=4500`, `quantity_sum=3`
- B: `distance_km=120`, `priority_flag=1`, `basket_minor=15000`, `quantity_sum=6`

és azt vizsgáljuk, mekkora a köztük lévő távolság, illetve hogy az egyes koordináták mennyire járulnak hozzá. Az output nem egyetlen szám lesz, hanem egy távolság és egy hozzájárulás-elemzés. A megfigyelési egység a rendelés, a négy koordináta a megfigyelt numerikus jellemző. Az induló becslésem: a nyers euklideszi távolságot szinte teljesen a `basket_minor` uralja, mert a kosáreltérés 10500, míg a többi koordináta eltérése legfeljebb 100. Ezt a sejtést a szakasz végén invariánssal ellenőrizzük: a kosárkoordináta négyzetes hozzájárulása várhatóan 99% fölött lesz.

10.1.2 Vektor és feature-tér

A **vektor** (*vector*) adattudományi jelentése: egy megfigyelés numerikus jellemzőinek rendezett listája. A p koordinátájú vektort

$$x = (x_1, x_2, \dots, x_p)$$

alakban írom. A x_i értelmezési tartománya a feature-től függ; a `distance_km` például nemnegatív valós szám km-ben, a `priority_flag` viszont csak 0 vagy 1. A NumPy-ban ugyanezt `np.array` adja, `(p,)` shape-szel, általában `float64` dtype-tal. A matematikai jelölés 1-től indexel, a NumPy 0-tól: a képletben szereplő x_i a kódban `x[i-1]`.

A **feature-tér** (*feature space*) az összes lehetséges koordináta-kombináció halmaza, itt $\mathbb{R}^4$. Minden rendelés egy pont ebben a térben. A gondot a mértékegység okozza: a tengelyek nem

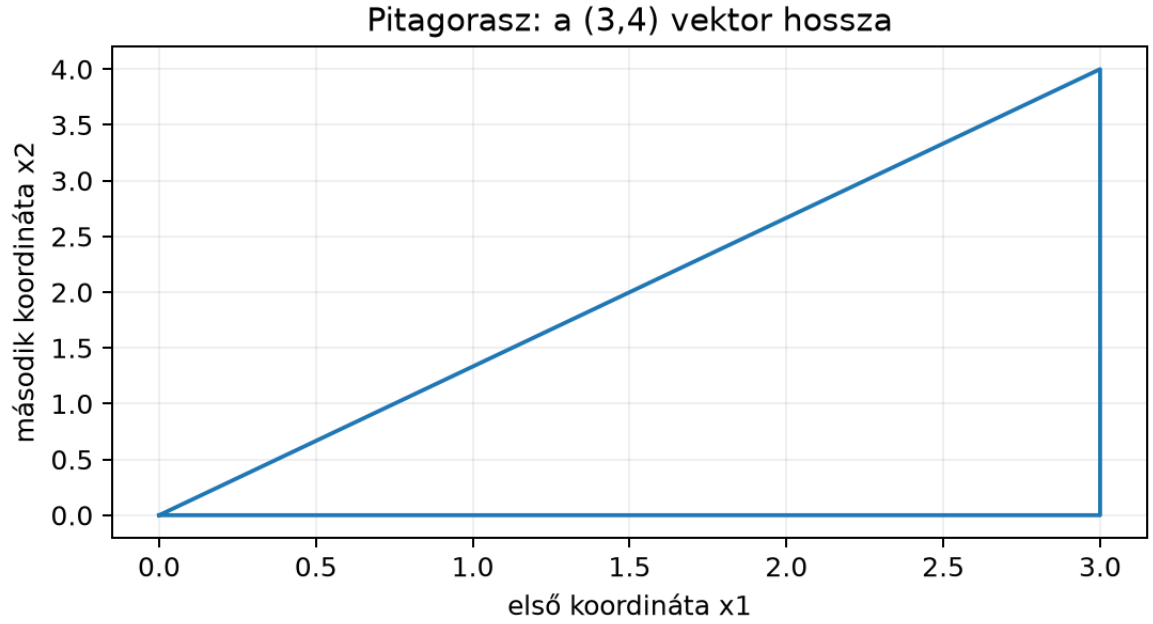
összehasonlítható skálájúak. A pontok geometriai távolsága ezért nem magától értetődő, hanem egy döntés következménye; a későbbi modellekben is ugyanez a választás tér vissza.

10.1.3 Pitagorasz újratanítva

Két koordinátán a vektor hosszát a Pitagorasz-tétel adja. Az $x = (3, 4)$ vektor esetén a hossz

$$\sqrt{3^2 + 4^2} = \sqrt{25} = 5.$$

A vektor a síkon a $(0, 0)$ pontból a $(3, 4)$ pontba mutató irányított szakasz; a hossza az átfogó, a derékszögű befogók hossza 3 és 4.



10.1. ábra: Pitagorasz: a (3,4) vektor hossza - Számított koordináta-példa, nem mérés.

A két befogó a koordinátatengelyekkel párhuzamos, ezért a négyzetösszeg pontosan a két oldalnégyzet összege. A p dimenziós általánosítás ugyanez koordinátánként: minden x_i egy derékszögű összetevő.

10.1.4 Skaláris szorzat (dot product)

A **skaláris szorzat** (*dot product*) két azonos hosszúságú vektorból készít skalárt:

$$x \cdot y = \sum_{i=1}^{p} x_i y_i.$$

A jelölések: x és y egyaránt $(p,)$ shape-ű vektor; a kimenet egyetlen skalár. A mértékegység vegyes lehet: ha a koordináták eltérő mennyiségek, akkor a szorzatok dimenziója sem egységes, tehát a kapott skalár nem hordoz közvetlen fizikai jelentést. Ezt a figyelmeztetést a távolságnál is meg kell ismételni.

Kézi kontroll: $x = (1, 2, 3)$, $y = (4, 5, 6)$ esetén

$$x \cdot y = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32.$$

A művelet két alapvető tulajdonsága: szimmetrikus, azaz $x \cdot y = y \cdot x$, és mindkét változójában lineáris. A NumPy-beli megfelelője `np.dot(x, y)`.

10.1.5 Euklideszi norma és távolság

Az **euklideszi norma** (*Euclidean norm*) a vektor hossza az origótól:

$$\|x\|_2 = \sqrt{\sum_{i=1}^{p} x_i^2}.$$

A 2-es index a „négyzetes” definícióra utal; a NumPy `np.linalg.norm(x)` hívása alapértelmezésben ugyanezt számolja. A norma tulajdonságai: nemnegatív, csak a nullvektorra 0, és pozitív skalárral szorozva a hossz ugyanazzal a skalárral szorzódik. A **távolság** két pont között a különbségvektor normája:

$$d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2}.$$

A fenti kis példában $x - y = (-3, -3, -3)$, ezért

$$d(x, y) = \sqrt{9 + 9 + 9} = \sqrt{18} \approx 4.243.$$

Ha a koordináták mértékegysége eltérő, a négyzetösszeg tagjai eltérő mértékegységűek: km^2, dimenziótlan, EUR-cent^2 és db^2 kerülne egymás mellé. A matematikai számolás mindig elvégezhető, de a kapott értéket „távolságnak” nevezni csak akkor van értelme, ha a koordinátákat előzőleg közös, dimenziótlan skálára hozzuk.

10.1.6 Koszinusz-hasonlóság

A **koszinusz-hasonlóság** (*cosine similarity*) a skaláris szorzatot a hosszak szorzatával normálja:

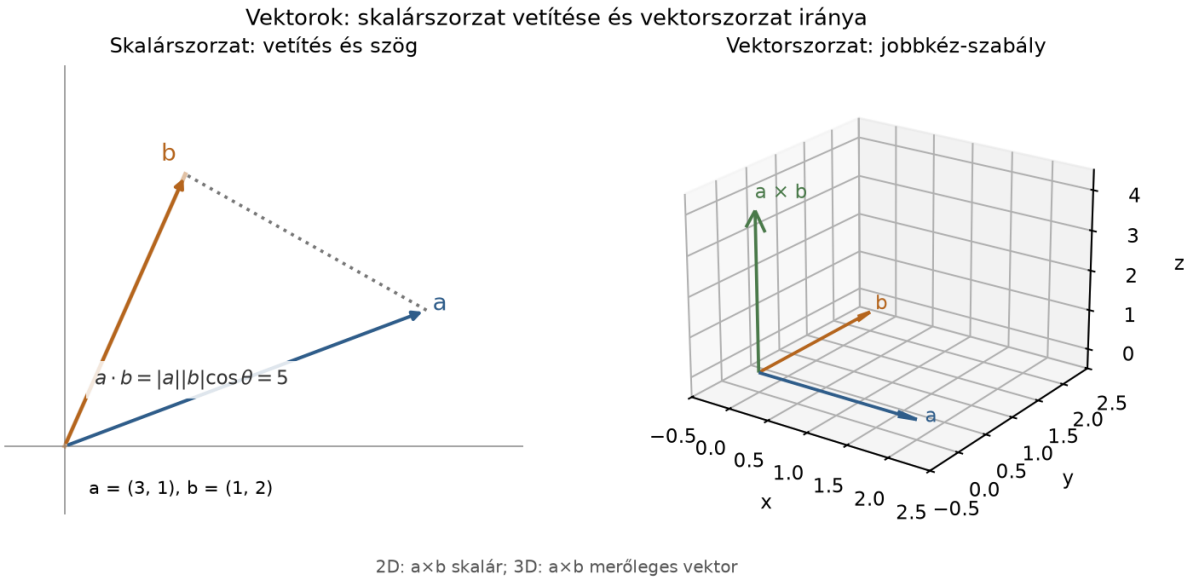
$$\cos\theta = \frac{x \cdot y}{\|x\|_2 \|y\|_2}.$$

A kimenet dimenziótlan, és a $[-1, 1]$ tartományba esik, ha egyik vektor sem nullvektor. Az érték 1, ha a két vektor azonos irányú, 0, ha merőleges, -1, ha ellentétes irányú. Ez nem távolság, hanem irány szerinti hasonlóság: a $(3, 4)$ és a $(6, 8)$ vektor koszinusza 1, pedig a hosszuk eltér. A példabeli x és y esetén

$$\frac{32}{\sqrt{14}\sqrt{77}} = \frac{32}{\sqrt{1078}} \approx 0.9746.$$

A hasonlóság magas; a két háromkoordinátás vektor iránya közel van, hosszaik közötti különbség viszont a koszinuszban nem jelenik meg.

10.1.6.1 Vektororientáció: sor, oszlop és az 1D NumPy



10.2. ábra: Skalárszorzat és vektorszorzat: bal oldalon a és b vektor, a b irányára vetített összetevő és az a · b=5 érték; jobb oldalon a jobbkéz-szabály szerinti a×b merőleges vektor.

A teljes rajzolókód a `lab/figures/make_concept_figures.py` `vector_products()` függvénye; az alábbi táblázat a shape-kontraktust rögzíti. A matematikai konvencióban a vektor **oszlopvektor**: $\mathbf{a} \in \mathbb{R}^{n\times 1}$. A NumPy 1D tömbje viszont sem nem sor-, sem nem oszlopvektor: `np.array([1,2,3])` shape-je `(3,)`, és a `@` operátor a helyzettől függően row- vagy column-szemantikát alkalmaz. Ezért a shape-ellenőrzést a szorzás előtt kell elvégezni, nem utána.

Kifejezés	Shape in	Shape out	Értelmezés
`a = np.array([1,2,3])`	—	`(3,)`	1D, orientáció nélkül
`a.reshape(3, 1)`	`(3,)`	`(3, 1)`	oszlopvektor
`a.reshape(1, 3)`	`(3,)`	`(1, 3)`	sorvektor
`a @ b`	`(3,)`, `(3,)`	skalár	1D · 1D belső szorzat
`a.reshape(1,3) @ b.reshape(3,1)`	`(1,3)`, `(3,1)`	`(1,1)`	1×1 mátrix, nem skalár
`a.reshape(3,1) @ b.reshape(1,3)`	`(3,1)`, `(1,3)`	`(3,3)`	külső szorzat
`A @ a`, `A.shape=(2,3)`	`(2,3)`, `(3,)`	`(2,)`	1D jobb oldal, vektor eredmény
`A @ a.reshape(3,1)`	`(2,3)`, `(3,1)`	`(2,1)`	oszlopvektor eredmény
`a.reshape(1,3) @ A.T`	`(1,3)`, `(3,2)`	`(1,2)`	sorvektor × mátrix

Három előzetes dimenzióvizsgálat:

1. `(3,1).T @ (3,1)` → `(1,3) @ (3,1)` = `(1,1)`, tehát az eredmény egy 1×1 mátrix; skalárhoz `float(...)` vagy `[0,0]` kell.

2. `(1,3) @ (3,1)` → `(1,1)`; ugyanaz a belső szorzat, de nem `()` shape.
3. `A(2,3) @ a(3,)` → `(2,)`, míg `A @ a.reshape(3,1)` → `(2,1)`; a két eredmény értéke azonos, a shape nem.

A `tests/test_vector_products.py` a fenti shape-eket NumPy-val ellenőrzi, a `figures/vector_products.png` létezését és a fejezet hivatkozását is beleértve. A gyakorlati szabály: a képletben jelöld a vektorokat oszlopvektorként, a kódban pedig explicit `reshape(n,1)` vagy `reshape(1,n)` alakkal tedd egyértelművé, melyik orientációt használod; a puszta `@` 1D tömbökön kényelmes, de a shape-eredményt nem teszi skalárrá.

10.1.7 Skálázás: mit csinál a koordinátákkal?

A **standardizálás** (*standardization*) minden koordinátát ugyanazzal a transzformációval dimenziótlanít. Ha a feature elvárt értéke μ_i, szórása $\sigma_i > 0$, akkor

$$z_i = \frac{x_i - \mu_i}{\sigma_i}.$$

A z vektor minden koordinátája dimenziótlan; a σ_i mértékegysége megegyezik x_i mértékegységével, így a hányadosnak már nincs mértékegysége. A linearitás miatt a standardizált érték nem változik, ha az adott coordinatat pozitív skalárral átskálázzuk: a μ_i és σ_i ugyanazzal a skalárral szorzódik, a hányados pedig ugyanaz marad. Ez a későbbiekben fontos ellenőrző eszköz: ha egy távolságalapú modell válasza megváltozik pusztán attól, hogy kilométer helyett méterben adjuk meg a távolságot, akkor a modell nem standardizált.

A skálázás azonban nem semleges, ahogy a 9.4. alfejezetben láttuk: egy másik skála más távolságot és más modelljelentést ad. A skálázási statisztikákat is csak abból az adatrészből szabad becsülni, amelyből a modell tanul, nem a teljes adatból; ez a későbbi szivárgásmentes pipeline egyik központi szabálya lesz.

10.1.8 Teljes minimális munkapélda

Az alábbi kód a két Northstar-példarendelés vektorából számol nyers távolságot, majd egy megadott referencia-statisztikával standardizál. A statisztikák itt szemléltető értékek, nem a mellékelt `orders.csv` futtatási eredményei.

```
import numpy as np

order_a = np.array([20.0, 0.0, 4500.0, 3.0])   # distance_km, priority_flag, basket_minor,
↪  quantity_sum
order_b = np.array([120.0, 1.0, 15000.0, 6.0])

diff = order_a - order_b
raw_distance = np.linalg.norm(diff)
print("nyers távolság:", raw_distance)  # szemléltető várt érték: kb. 10500.48

# Referencia-statisztikák a teljes tanító adatból számolandók; itt szemléltető értékek.
mu = np.array([80.0, 0.5, 12000.0, 4.0])
sigma = np.array([40.0, 0.5, 6000.0, 2.0])

z_a = (order_a - mu) / sigma
z_b = (order_b - mu) / sigma
scaled_distance = np.linalg.norm(z_a - z_b)
print("standardizált távolság:", scaled_distance)  # szemléltető várt érték: kb. 3.9449
```

```
contrib_raw = (diff ** 2) / (raw_distance ** 2)
contrib_scaled = ((z_a - z_b) ** 2) / (scaled_distance ** 2)
print("nyers négyzetes hozzájárulás:", contrib_raw)
print("standardizált hozzájárulás:", contrib_scaled)
```

A könyv készítésekor a kódot nem futtattam; a megadott értékek kézi ellenőrzésből származó elméletileg várt kimenetek. A saját `lab/data/orders.csv` betöltéséhez a teljes projektfüggő utat kell használni, például:

```
import pandas as pd
orders = pd.read_csv("orders.csv", parse_dates=["order_ts"])
print(orders[["distance_km", "priority_flag", "basket_minor", "quantity_sum"]].head(2))
```

Ha a laborfájl a telepítés helyén más mappában van, a `lab/` relatív útvonalat kell használni; a pontos utat az 1.1. alfejezetben rögzített projektgyökérhez viszonyítva érdemes kezelni.

10.1.9 Eredmény értelmezése

A nyers A–B távolságot a `basket_minor` eltérése uralja: a különbség -10500, a négyzete 110250000, ami a 110260010 teljes négyzetösszeg 99,99%-a. A távolság közel 10500.48, tehát a többi jellemző gyakorlatilag láthatatlan. Ez a távolságazon alapuló modell torzítását okozná: a kosárérték határozná meg, hogy mely rendelések számítanak hasonlónak.

A standardizálás után a koordináták dimenziótlanok és összehasonlíthatók. A z-vektorok:

$$z_A = (-1{,}5,\ -1,\ -1{,}25,\ -0{,}5), \qquad z_B = (1,\ 1,\ 0{,}5,\ 1).$$

A különbség négyzetösszege $6{,}25 + 4 + 3{,}0625 + 2{,}25 = 15{,}5625$, a távolság kerekítve 3,9449. A négyzetes hozzájárulások most nagyjából 40%, 26%, 20% és 14%. A kép már nem egyetlen feature-ről szól; a kosárérték fontos marad, de nem egyedüli döntő. Az ellenőrzést akkor tekinthetjük helyesnek, ha a hozzájárulások összege 1, és a saját kézi Pitagorasz-összegünk a NumPy-eredménnyel 10^{-12} relatív tűrésen belül egyezik; a shape mindkét távolságnál skalár, a dtype `np.float64`.

10.1.10 Feladatok

1. Értelmezés. Vezesd le a $p = 1$ esetet. Mi a `np.linalg.norm(x)` eredménye, ha `x = np.array([-5.0])`? Mi lesz a két 1 koordinátás vektor, $a = (-5)$ és $b = (3)$ távolsága? Fogalmazd meg, mit jelent itt a standardizálás z-értéke.

2. Implementálás. Saját Python-függvénnyel, `numpy.dot` és `numpy.linalg.norm` használata nélkül, csak beépített `sum`-mal és `math.sqrt`-tal írd meg a `dot(a,b)`, `norm(a)` és `cosine(a,b)` függvényeket. Ellenőrizd az $a = (1, 2, 3)$, $b = (4, 5, 6)$ példából ismert eredményeket 10^{-12} tűréssel.

3. Hibakeresés. Az alábbi koszinusz-számítás miért hibás, ha `x` és `y` nem merőlegesek?

```
cosine = np.dot(x, y) / np.linalg.norm(x) * np.linalg.norm(y)
```

Javítsd ki, és állítsd elő azt az `x`–`y` párt is, ahol a hibás képlet véletlenül mégis helyes eredményt ad.

4. Megváltozott feltétel. Ugyanaz a két rendelés, de a `distance_km` értékét méterre váltjuk: A távolsága 20000 m, B-é 120000 m lesz; a többi koordináta változatlan. Számold ki a nyers távolságot az új vektorokkal, majd számold ki a standardizált távolságot a megfelelő méterre átváltott $\mu = 80000$ m és $\sigma = 40000$ m statisztikákkal. Milyen következtetést von le ez a skálainvarianciáról?

10.1.11 Megoldások és indoklás

1. $p = 1$ esetén a vektor egyetlen szám. A norma képlete $\sqrt{x_1^2} = |x_1|$, tehát `np.linalg.norm(np.array([-5.0]))` eredménye 5.0. A távolság $|(-5) - 3| = 8$. Ha az egyetlen feature elvárt értéke μ és szórása σ, akkor $z = (x - \mu)/\sigma$ egy dimenziótlan normalizált koordináta: azt mondja meg, hány szórással van x a középérték felett vagy alatt.

2.

```python
import math

def dot(a, b):
    return sum(ai * bi for ai, bi in zip(a, b))

def norm(a):
    return math.sqrt(sum(ai * ai for ai in a))

def cosine(a, b):
    return dot(a, b) / (norm(a) * norm(b))

a = [1, 2, 3]
b = [4, 5, 6]
print(dot(a, b), norm(a), cosine(a, b))  # szemléltető: 32, 3.7416..., 0.9746...
```

Az invariant az ismert kézi értékekkel való egyezés: `dot` 32, `norm` $\sqrt{14}$, a koszinusz $32/\sqrt{1078}$. Ha a hányadost `abs(ertek - elvart) < 1e-12` szűrésű tesztben használjuk, a függvények helyesen működnek.

3. A Python kiértékelési sorrendje miatt a képlet valójában

$$\frac{x \cdot y}{\|x\|_2} \cdot \|y\|_2$$

lesz, tehát a második normával szoroz, nem oszt. A helyes alak:

```python
cosine = np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))
```

A hibás képlet akkor adhat véletlenül jó értéket, ha $\|y\|_2^2 = 1$, mert akkor a szorzás és az osztás eredménye ugyanaz a hányados marad. Például $y = (1, 0)$ és $x = (2, 0)$ esetén a helyes koszinusz 1, a hibás kifejezés is $2/(2) \cdot 1 = 1$. Ez az egybeesés inverz irányban is mutatja, miért nem elég egyetlen tesztpár.

4. A méterre váltott nyers eltérés:

$$(-100000, -1, -10500, -3)$$

négyzetösszege $10^{10} + 1 + 110250000 + 9 \approx 1{,}011025 \times 10^{10}$, a távolság kerekítve 100550,0. A `distance_km` most 99,999% fölött uralja az eredményt, pedig a fizikai helyzet nem változott. A standardizált érték ellenben ugyanaz:

$$z_A = (20000 - 80000)/40000 = -1{,}5, \qquad z_B = (120000 - 80000)/40000 = 1.$$

A standardizált távolság a korábbi 3,9449 marad. A standardizálás tehát eltünteti a pozitív lineáris mértékegység-átskálázás hatását; a nyers euklideszi távolság nem invariáns rá.

10.1.12 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés az, hogy a nyers euklideszi távolságot skálázás nélkül „objektív hasonlóságnak” tekintjük. A fenti A–B példa az ellenpélda: kilométer helyett méterben megadva a távolság nagyságrendet ugrik, és a döntés egyetlen, önkényesen nagyobb számokban mért dimenzióra szűkül. A nagyobb numerikus egységű feature minden statisztikai hiba nélkül is dominál, mert a négyzetes büntetés nem ismeri a mértékegységet. Az önellenőrzés egyszerű: számítsd ki a hozzájárulásokat, és kérdezd meg, szeretnél-e modellt ezzel a nyers mércével. Ha a hozzájárulások 90% fölött koncentrálódnak egyetlen koordinátára, az ritkán a tényleges jelentőséget mutatja, inkább skálaproblémát.

A másik félreértés a koszinusz-hasonlóságot távolságnak hívja. A koszinusz a hosszt elfelejti, és nem teljesíti a távolság nulla-feltételét: az $(1, 0)$ és $(2, 0)$ vektorok hasonlósága 1, pedig a pontok nem esnek egybe. Az 1.1. alfejezetből ismert elv szerint a mértéket ahhoz a döntéshez kell kötni, amelyet a modell később támogat.

Különböző mértékegységű változók euklideszi távolságát megfelelő skálázás nélkül könnyű félreértelmezni. A skálázáshoz szükséges paramétereket a tanítóhalmazból becsüljük, ne a validációs vagy tesztadatból.

10.1.13 Rövid összefoglaló

A vektor az adattudományban egy megfigyelés rendezett numerikus jellemző-listája, a távolság pedig a különbségvektor euklideszi normája. A skaláris szorzat a koordinátaszorzatok összege, a koszinusz-hasonlóság irány szerint hasonlít, de nem távolság. A különböző mértékegységű koordináták négyzetes összegzése félrevezető; a standardizálás dimenziótlan közös skálát ad, és pozitív lineáris mértékegység-átskálázásra invariáns. A Northstar-példában a nyers távolságot $99,99$%-ban a kosárérték uralta, a standardizálás után a hozzájárulások kiegyensúlyozottabbak lettek. A következő alkalmazás a 10.2. alfejezet mátrixszorzás, amely ezekből a koordinátánkénti műveletekből építi fel a lineáris transzformációkat.

10.2 Mátrixszorzás és lineáris transzformáció

Belépő · szint: közép · idő: 60 perc · eszköz: NumPy

Előfeltétel: 10.1. alfejezet · Célkészség: Mátrixszorzás és lineáris transzformáció · Kimenet: működő kód és a labor ellenőrző tesztje.

10.2.1 Cél és konkrét helyzet

A helyzet: három rendelést választottunk ki a Northstar-példából. Minden rendeléshez két, a rendelés pillanatában ismert jellemző tartozik: a szállítási távolság kilométerben (`distance_km`) és a kosárérték eurócentben (`basket_minor`). Egy kolléga adott nekünk egy kézi, még **nem tanított** lineáris modellt a várható kézbesítési napokra:

$$\hat{y}_i \;=\; 2.0 \;+\; 0.3 \cdot \text{distance_km}_i \;+\; 0.0002 \cdot \text{basket_minor}_i.$$

A három megfigyelési egység egy-egy rendelés:

Rendelés	distance_km	basket_minor
R01	10	2500
R02	25	5000
R03	5	1000

Cél: ezt a három predikciót egyszerre, mátrixműveletként kiszámítani, és elkülöníteni a mátrixszorzást a koordinátánkénti szorzástól. A fejezetben **nem becslünk** súlyokat; a súlyokat adottnak vesszük. A súlyok becslése a következő alfejezet (10.3. alfejezet) feladata lesz.

Előzetes becslés a téma előtt: számold ki kézzel az első rendelés predikcióját: $2.0 + 0.3 \cdot 10 + 0.0002 \cdot 2500$. Ha 5.5 nap jött ki, jó úton jársz; a mátrixszorzás pontosan ezt a műveletet végzi el egyszerre minden sorra.

10.2.2 Fogalmi magyarázat: a mátrixszorzás mint sor-skaláris szorzat

10.2.2.1 Kétismeretlenes egyenletből mátrixszorzáshoz

Induljunk ki egy közismert lineáris egyenletrendszerből:

$$\begin{cases} 2a + 3b = 13, \\ 4a + \;\; b = 11. \end{cases}$$

Ezt úgy írhatjuk tömören, hogy az együtthatókat egy 2×2-es mátrixba gyűjtjük, az ismeretleneket pedig oszlopvektorba:

$$\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}, \qquad \mathbf{z} = \begin{pmatrix} a \\ b \end{pmatrix}, \qquad \mathbf{Az} = \begin{pmatrix} 2a + 3b \\ 4a + b \end{pmatrix} = \begin{pmatrix} 13 \\ 11 \end{pmatrix}.$$

A mátrixszorzás (matrix product) szabálya tehát: a kimenet i-edik eleme az $\mathbf{A}$ i-edik **sora** és a $\mathbf{z}$ **oszlopa** skaláris szorzata. A skaláris szorzatot az előző alfejezetben, 10.1. alfejezetben definiáltuk: koordinátaszorzatok összege. A példa megoldása $a = 2$, $b = 3$, mert $2 \cdot 2 + 3 \cdot 3 = 13$ és $4 \cdot 2 + 1 \cdot 3 = 11$.

10.2.2.2 A design mátrix és az Xw predikció

A predikciós feladatban az **A** szerepét a **design mátrix** (design matrix) veszi át, jele **X**. Egy **X** mátrix $n \times p$ alakú, ha n megfigyelés és p oszlop van. Minden **sor egy megfigyelés**, minden **oszlop egy jellemző**. A fenti három rendelésnél $n = 3$; ha a tengelymetszethez (intercept) tartozó, csupa 1-es oszlopot is hozzáadjuk, akkor $p = 3$:

$$\mathbf{X} = \begin{pmatrix} 1 & 10 & 2500 \\ 1 & 25 & 5000 \\ 1 & 5 & 1000 \end{pmatrix}, \qquad \mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 0.3 \\ 0.0002 \end{pmatrix}.$$

A lineáris predikció:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}.$$

Ebben a példában:

$$\hat{\mathbf{y}} = \begin{pmatrix} 1 \cdot 2.0 + 10 \cdot 0.3 + 2500 \cdot 0.0002 \\ 1 \cdot 2.0 + 25 \cdot 0.3 + 5000 \cdot 0.0002 \\ 1 \cdot 2.0 + 5 \cdot 0.3 + 1000 \cdot 0.0002 \end{pmatrix} = \begin{pmatrix} 5.5 \\ 10.5 \\ 3.7 \end{pmatrix}.$$

10.2.2.3 Alakok és mértékegységek

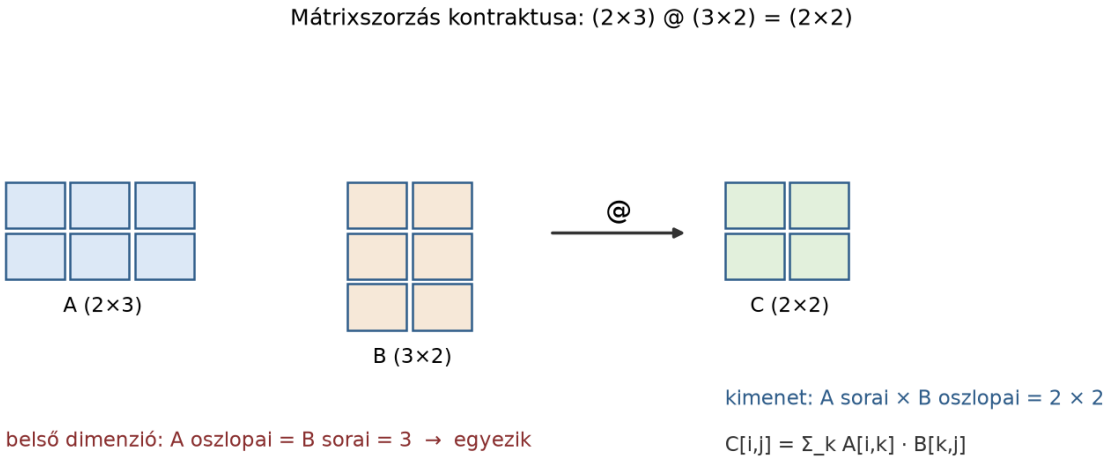
A mátrixszorzás alakszabálya: ha **X** alakja (n, p) és **w** alakja $(p, 1)$, akkor $\hat{\mathbf{y}}$ alakja $(n, 1)$. A két **belső** dimenzió, p, megegyezik, és a szorzás során eltűnik. Szavakkal: annyi kimenet lesz, ahány sor, és minden kimenet annyi tagot összegez, ahány oszlop.

A súlyok mértékegysége a predikciót teszi dimenzionálisan helyessé. Ha a cél a kézbesítési napok száma, akkor:

- w_0 mértékegysége **nap**;
- w_1 mértékegysége **nap/km**, mert $w_1 \cdot$ distance_km eredménye nap;
- w_2 mértékegysége **nap/eurocent**, mert $w_2 \cdot$ basket_minor eredménye nap.

A súlyok tehát nem puszta számok: átváltják a jellemző mértékegységét a cél mértékegységére. A szorzatok azért összeadhatók, mert mindegyik tag már nap egységben van.

10.2.2.4 A mátrixszorzás kontraktusa vizuálisan



10.3. ábra: Mátrixszorzás kontraktusa: A (2×3) szorozva B (3×2) mátrixszal; a belső 3 dimenzió egyezik, a kimenet C (2×2), a C[i,j] elem az A i-edik sora és a B j-edik oszlopa skalárszorzata.

A teljes rajzolókód a `lab/figures/make_concept_figures.py` `matrix_multiply()` függvénye. A mátrixszorzás **kontraktusa** három számból áll: A soraiból (m), a közös belső dimenzióból (k) és B oszlopaiból (n). Az $(m \times k) \cdot (k \times n)$ szorzat eredménye $(m \times n)$; a belső k eltűnik. Ha a belső dimenziók nem egyeznek, a művelet nem értelmezhető, és a NumPy `ValueError`-t dob.

Bal	Jobb	Kimenet	Érvényes?
`(2, 3)`	`(3, 2)`	`(2, 2)`	igen
`(2, 3)`	`(2, 3)`	—	nem: $3 \neq 2$
`(3,)`	`(3,)`	skalár	igen, 1D belső szorzat
`(2, 3)`	`(3,)`	`(2,)`	igen, 1D jobb oldal
`(3, 2)`	`(2, 3)`	`(3, 3)`	igen
`(2, 3)`, `(3, 2)`, `(2, 4)`	lánc	`(2, 4)`	igen

Kódolható feladat: írd meg a `matmul_shape(left, right)` függvényt, amely a két shape-ből visszaadja a kimeneti shape-et, vagy `ValueError`-t dob, ha a belső dimenzió nem egyezik. Ellenőrzésképpen:

```
def matmul_shape(left, right):
    if len(left) != 2 or len(right) != 2:
        raise ValueError("2D shape-ek kellenek")
    if left[1] != right[0]:
        raise ValueError(f"belső dimenzió: {left[1]} != {right[0]}")
    return (left[0], right[1])

import numpy as np

assert matmul_shape((2, 3), (3, 2)) == (2, 2)
assert matmul_shape((3, 2), (2, 3)) == (3, 3)
assert (np.zeros((2, 3)) @ np.zeros(3)).shape == (2,)
try:
    matmul_shape((2, 3), (2, 3))
```

```
except ValueError:
    pass
else:
    raise AssertionError("a belső dimenzió hibáját jelezni kell")
```

A `(2,3) @ (3,)` eset a NumPy-ban érvényes, de a fenti 2D-s `matmul_shape` szándékosan elutasítja; a 1D vektort előbb `(3,1)` vagy `(1,3)` alakra kell hozni, vagy külön függvényt kell írni rá. A `tests/test_matrix_contract.py` a 2D-s eseteket, a belső dimenzió hibáját, a láncszorzást és a NumPy tényleges shape-eredményeit ellenőrzi.

10.2.2.5 Transzponálás és a sor-oszlop átrendezés

A **transzponált** (transpose) mátrix sorait és oszlopait felcseréli:

$$(\mathbf{X}^T)_{ij} = \mathbf{X}_{ji}.$$

Ha $\mathbf{X}$ alakja (n, p), akkor $\mathbf{X}^T$ alakja (p, n). A fenti példában:

$$\mathbf{X}^T = \begin{pmatrix} 1 & 1 & 1 \\ 10 & 25 & 5 \\ 2500 & 5000 & 1000 \end{pmatrix}.$$

A transzponálás a becslés későbbi lépésében, az $\mathbf{X}^T\mathbf{X}$ mátrixban tér majd vissza; egyelőre elég annyi, hogy a sorok és oszlopok megcserélésével a „minta” és a „jellemző” irány felcserélhető.

10.2.2.6 Bázis és tengelymetszet

A design mátrix oszlopai tekinthetők **bázisfüggvény-vektoroknak** (basis vectors). A predikció az oszlopok lineáris kombinációja:

$$\hat{\mathbf{y}} = w_0 \underbrace{\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}}_{\text{intercept oszlop}} + w_1 \underbrace{\begin{pmatrix} 10 \\ 25 \\ 5 \end{pmatrix}}_{\text{távolság oszlop}} + w_2 \underbrace{\begin{pmatrix} 2500 \\ 5000 \\ 1000 \end{pmatrix}}_{\text{kosárérték oszlop}}.$$

A csupa 1-es oszlop az **intercept oszlop**; a hozzá tartozó w_0 a tengelymetszet, vagyis az a predikció, amikor minden numerikus jellemző nulla. A „bázis” itt annyit jelent: a modell által elérhető predikciók az oszlopvektorok összes lehetséges lineáris kombinációja. Ez a kép lesz fontos a legkisebb négyzeteknél (10.3. alfejezet) és a főkomponens-elemzésnél (24.1. alfejezet).

10.2.2.7 Miért nem koordinátánkénti szorzás?

A NumPy-ban és sok más eszközben két művelet hasonlít egymásra:

- **Elemenkénti szorzás** (`*` operátor, angolul Hadamard product): azonos alakú tömbök azonos pozíciójú elemeit szorozza össze.
- **Mátrixszorzás** (`@` operátor): sorok és oszlopok skaláris szorzatát számolja, az előbb leírt szabállyal.

A predikció nem elemenkénti szorzás, mert $\mathbf{X}$ alakja $(3, 3)$, a súlyok alakja pedig $(3,)$, tehát nincs sor-sor pár. A helyes művelet a mátrixszorzás: minden sor a **teljes** súlyvektorral kombinálódik. A koordinátánkénti szorzás csak közbülső lépés lehet, de önmagában nem adja a predikciót.

10.2.3 Teljes minimális munkapélda

A labor konkrét célja: három mintára, két valódi jellemzővel számítsuk ki a lineáris predikciót először kézzel, majd a NumPy `@` operátorával. A kézi számítást már elvégeztük fent; az eredmény $[5.5, 10.5, 3.7]$.

```
import numpy as np

# Minden sor egy rendelés.
# Az első oszlop az intercept, a második a distance_km,
# a harmadik a basket_minor.
X = np.array([
    [1.0, 10.0, 2500.0],
    [1.0, 25.0, 5000.0],
    [1.0,  5.0, 1000.0],
])

# Súlyok: w0 = 2 nap, w1 = 0.3 nap/km, w2 = 0.0002 nap/eurocent.
w = np.array([2.0, 0.3, 0.0002])

# Mátrixszorzás: (3, 3) @ (3,) -> (3,)
y_hat = X @ w

# Kézi ellenőrzés oszloponkénti súlyozott összeggel.
manual = X[:, 0] * w[0] + X[:, 1] * w[1] + X[:, 2] * w[2]

print("X.shape:", X.shape)        # (3, 3)
print("w.shape:", w.shape)        # (3,)
print("y_hat.shape:", y_hat.shape)  # (3,)
print("y_hat:", y_hat)            # várhatóan [5.5 10.5 3.7]
print("allclose:", np.allclose(y_hat, manual, rtol=1e-12, atol=1e-12))
```

Szemléltető kimenet, nem mért futtatás; a kézi kontroll alapján várható:

```
X.shape: (3, 3)
w.shape: (3,)
y_hat.shape: (3,)
y_hat: [ 5.5 10.5  3.7]
allclose: True
```

A `manual` sor azt mutatja, hogy a mátrixszorzás belül oszloponkénti elemenkénti szorzások **összege**, de maga az `X * w` kifejezés nem a predikció. Ezt a megkülönböztetést a gyakori tévedéseknél részletezzük.

10.2.4 Az eredmény értelmezése

A három predikció: 5.5, 10.5, 3.7 nap. Az első rendelésre a kézi számítás pontosan ezt adta, tehát a kód alakja és iránya jó. A kimenet `shape=(3,)`, `dtype=float64`, mert a bemenetek `float64` mátrixok. A `np.allclose` azért hasznos, mert a `0.3` és `0.0002` bináris ábrázolása nem mindig egzakt; `rtol=1e-12, atol=1e-12` toleranciával a gépi és kézi eredmény megegyezik.

Értelmezési korlát: ez a lineáris modell **additív** és **globálisan lineáris**. Ha a távolság 0 és a kosárérték 0, a predikció 2 nap; ha a távolság nagyon nagy, a predikció tetszőlegesen nagy lehet.

A valós kézbesítés nem lehet negatív és nem nőhet minden határon túl; a későbbi modellezésnél nemlineáris vagy korlátos modellt választunk. A log-skálát 9.2. alfejezetben láttuk: ha a cél $\log y$ lenne, a súlyok **multiplikatív** hatást jelentenének a nyers skálán. Itt azonban nyers skálán dolgozunk, ezért a hozzájárulások összeadódnak.

Rövid pandas-visszautalás (6.2. alfejezet): ha a mátrixot egy `DataFrame`-ből készítjük a `.to_numpy()` hívással, a mátrix elveszíti a sorindexet. A predikció visszaírásánál ezért mindig a `DataFrame` eredeti indexét használjuk, ne a NumPy pozícióját. A példában a sorrend és az index azonos, de nagyobb táblánál ez a leggyakoribb csendes hiba.

10.2.5 Feladatok

1. **Értelmezés.** Adott egy másik kézi súlyvektor: $w_0 = 1.0$ nap, $w_1 = 0.2$ nap/km, $w_2 = 0.0005$ nap/eurocent. Számold ki kézzel a predikciót arra a két új rendelésre, ahol `(distance_km, basket_minor)` rendre $(20, 4000)$ és $(8, 1200)$. Mi a mértékegysége w_1 és w_2 fontos szerepének?

2. **Megvalósítás.** Építsd meg $\mathbf{X}$-et a fenti három rendelés adatával, de a súlyokat most válaszd $w_0 = 1.8$, $w_1 = 0.22$, $w_2 = 0.00035$ értékűre. Számold ki $\mathbf{Xw}$ értékét a `@` operátorral, és ellenőrizd az első sorra kézi számítással. Ellenőrizd a `shape` értékét is.

3. **Hibakeresés.** Egy kolléga ezt írta ugyanarra az $\mathbf{X}$-re, $w = (2.0,\ 0.3,\ 0.0002)$ súlyokkal:

   ```
   pred = X * w
   ```

   Mi ez pontosan? Miért nem a lineáris predikció? Add meg, mit kap a kolléga az első sorban, és javítsd a kódot két módon: `@` operátorral, illetve az elemenkénti szorzat explicit sorösszegzésével.

4. **Megváltozott feltétel / továbbvitel.** Adj a modellhez egy harmadik jellemzőt: `quantity_sum`, az adott rendelés összes darabszámát. A három rendelésre legyen $(4, 6, 7)$, az új súly pedig $w_3 = 0.1$ nap/darab. Írd fel az új $\mathbf{X}$ mátrixot, add meg az alakját, és számold ki az új predikciókat `@` operátorral. Ellenőrizd kézzel az R01 rendelés predikcióját.

5. **Értelmezés: mátrixszorzás és nem kommutativitás.** Legyen

$$\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}.$$

   Számold ki kézzel $\mathbf{AB}$-t és $\mathbf{BA}$-t. Milyen alakú a két eredmény? Mit jelent ez a „mátrixosztás” szempontjából?

10.2.6 Megoldások és indoklás

1. megoldás. A predikció:

$$\hat{y} = 1.0 + 0.2 \cdot \text{distance} + 0.0005 \cdot \text{basket}.$$

- Első rendelés: $1.0 + 0.2 \cdot 20 + 0.0005 \cdot 4000 = 1.0 + 4.0 + 2.0 = 7.0$ nap.
- Második rendelés: $1.0 + 0.2 \cdot 8 + 0.0005 \cdot 1200 = 1.0 + 1.6 + 0.6 = 3.2$ nap.

A w_1 mértékegysége nap/km, mert a km-t napra váltja; a w_2 mértékegysége nap/eurocent. A mértékegységek nélkül a 4.0 és 2.0 tagot nem lehetne egy nap-skálán összeadni.

2. megoldás.

```
import numpy as np

X = np.array([
    [1.0, 10.0, 2500.0],
    [1.0, 25.0, 5000.0],
    [1.0,  5.0, 1000.0],
])
w = np.array([1.8, 0.22, 0.00035])

y_hat = X @ w
print("shape:", y_hat.shape)
print("y_hat:", y_hat)
```

Várható kimenet: `shape: (3,)`, `y_hat: [4.875 8.05 3.25]`.

Ellenőrzés az első sorra:

$$1.8 + 0.22 \cdot 10 + 0.00035 \cdot 2500 = 1.8 + 2.2 + 0.875 = 4.875.$$

3. megoldás. A `X * w` kifejezés NumPy-ban **elemenkénti szorzás** broadcastinggal. Mivel $\mathbf{X}$ alakja $(3, 3)$ és $\mathbf{w}$ alakja $(3,)$, a $\mathbf{w}$ az utolsó tengelyre illeszkedik, ezért a kolléga eredménye:

$$\mathbf{X} * \mathbf{w} = \begin{pmatrix} 1 \cdot 2.0 & 10 \cdot 0.3 & 2500 \cdot 0.0002 \\ 1 \cdot 2.0 & 25 \cdot 0.3 & 5000 \cdot 0.0002 \\ 1 \cdot 2.0 & 5 \cdot 0.3 & 1000 \cdot 0.0002 \end{pmatrix} = \begin{pmatrix} 2.0 & 3.0 & 0.5 \\ 2.0 & 7.5 & 1.0 \\ 2.0 & 1.5 & 0.2 \end{pmatrix}.$$

Ez nem a predikció, hanem egy $(3, 3)$-as mátrix. Az első sor helyes predikciója $2.0+3.0+0.5 = 5.5$, tehát a kolléga csak akkor kapná meg a predikciókat, ha utána soronként összegezné, például:

```
pred = (X * w).sum(axis=1)
```

Ez azonban véletlenül működik az egydimenziós $\mathbf{w}$ esetben; ha $\mathbf{w}$ oszlopvektor, az elemenkénti szorzás egészen mást ad. A helyes, szándékot tükröző megoldás a mátrixszorzás:

```
pred = X @ w
```

A javítási elv: **mindig a dimenziókból indulj ki**, és a predikciót $\mathbf{Xw}$-ként írd le. Az elemenkénti szorzás nem „rossz”, hanem más művelet.

4. megoldás. Az új design mátrix 3 sor és 4 oszlop:

$$\mathbf{X}_{\text{new}} = \begin{pmatrix} 1 & 10 & 2500 & 4 \\ 1 & 25 & 5000 & 6 \\ 1 & 5 & 1000 & 7 \end{pmatrix}, \qquad \mathbf{w}_{\text{new}} = \begin{pmatrix} 2.0 \\ 0.3 \\ 0.0002 \\ 0.1 \end{pmatrix}.$$

Kódban:

```
X_new = np.array([
    [1.0, 10.0, 2500.0, 4.0],
    [1.0, 25.0, 5000.0, 6.0],
    [1.0,  5.0, 1000.0, 7.0],
])
w_new = np.array([2.0, 0.3, 0.0002, 0.1])

y_hat_new = X_new @ w_new
print("X_new.shape:", X_new.shape)  # (3, 4)
print("y_hat_new:", y_hat_new)      # [5.9 11.1  4.4]
```

Kézi ellenőrzés R01-re: $2.0 + 0.3 \cdot 10 + 0.0002 \cdot 2500 + 0.1 \cdot 4 = 2.0 + 3.0 + 0.5 + 0.4 = 5.9$ nap. A `X_new.shape` $(3, 4)$, mert az intercept és a három jellemző négy oszlopot ad.

5. megoldás. A szorzás:

$$\mathbf{AB} = \begin{pmatrix} 2\cdot 1 + 3\cdot 2 & 2\cdot 0 + 3\cdot 1 \\ 4\cdot 1 + 1\cdot 2 & 4\cdot 0 + 1\cdot 1 \end{pmatrix} = \begin{pmatrix} 8 & 3 \\ 6 & 1 \end{pmatrix}.$$

$$\mathbf{BA} = \begin{pmatrix} 1\cdot 2 + 0\cdot 4 & 1\cdot 3 + 0\cdot 1 \\ 2\cdot 2 + 1\cdot 4 & 2\cdot 3 + 1\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 8 & 7 \end{pmatrix}.$$

A két eredmény különböző, tehát a mátrixszorzás **nem kommutatív**. Ezért nincs értelmes „mátrixosztás” úgy, ahogy a valós számoknál b/a működik. Ha egy $\mathbf{Az} = \mathbf{b}$ egyenletet akarunk megoldani, külön műveletet használunk: $\mathbf{z} = \mathbf{A}^{-1}\mathbf{b}$ vagy numerikusan jobb $\mathbf{z} = \text{solve}(\mathbf{A}, \mathbf{b})$. A design mátrix predikciója viszont nem old meg egyenletet: ott $\mathbf{w}$ adott, és $\hat{\mathbf{y}} = \mathbf{Xw}$ kiszámítása szorzás.

10.2.7 Gyakori tévedés és ellenpélda

Az invertálás, az elemenkénti osztás és a lineáris rendszer megoldása külön művelet.

Tegyük fel, hogy valaki meg akarja oldani a fenti $\mathbf{Az} = \mathbf{b}$ rendszert $\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}$ és $\mathbf{b} = \begin{pmatrix} 13 \\ 11 \end{pmatrix}$ esetén. Három gyakori, de eltérő jelentésű kifejezés:

1. **Elemenkénti osztás:** `b / A` Python/NumPy-ban nem mátrixosztás. Ha egyáltalán lefut, a `/` elemenként oszt, és az alakokat a broadcasting szabályai szerint illeszti. A fenti $\mathbf{b}$ alakja $(2,)$, $\mathbf{A}$ alakja $(2, 2)$; ez leggyakrabban `ValueError`. Ha mégis lefutna, akkor is elemenkénti hányadosokat adna, nem az egyenletrendszer megoldását.
2. **Invertálás:** `np.linalg.inv(A) @ b` matematikailag helyes, ha $\mathbf{A}$ invertálható. Numerikusan azonban rossz szokás, mert az inverz kiszámítása instabil lehet. Az ajánlott megoldás:

   ```
   z = np.linalg.solve(A, b)   # z = [2.0, 3.0]
   ```

3. **Predikció:** `X @ w`. Itt $\mathbf{X}$ általában nem négyzetes; ha $n \neq p$, nem is beszélhetünk egyértelmű inverzről. A cél pedig nem megoldani egy egyenletet, hanem a súlyokkal kiszámítani a predikciót.

A koordinátánkénti szorzás csendes csapdája. A fenti `X * w` példa mutatja, hogy a broadcasting miatt a rossz művelet is lefuthat. Ellenőrzésképpen soha ne csak a kódot futtasd, hanem **egy sorra kézzel** is számold ki a várt eredményt. A mátrixszorzás alakszabálya a belső dimenzió egyezése: $(n, p)@(p,) \to (n,)$. Ha ezt a dimenziót nem ellenőrizzük, a broadcasting kiszámítható, de értelmetlen eredményt adhat.

10.2.8 Önellenőrzés

1. Adott $\mathbf{X}$ alakja $(100, 8)$ és $\mathbf{w}$ alakja $(8,)$. Mi a predikció alakja, és hány tagot összegez minden sor?
<details><summary>Válasz</summary>A predikció alakja $(100,)$; minden sor 8 tagot (intercept + 7 jellemző, ha az első oszlop intercept) összegez.</details>

2. Miért kell az interceptet külön 1-es oszlopként felvenni?
<details><summary>Válasz</summary>Mert a mátrixszorzás minden kimenetet $\sum_j X_{ij} w_j$ alakban állít elő. Az 1-es bejegyzés révén jelenik meg a w_0 tag, amely így nem függ egyetlen jellemzőtől sem.</details>

3. Milyen műveletek a `*` és a `@` NumPy-ban?
<details><summary>Válasz</summary>A `*` elemenkénti (Hadamard-) szorzás broadcastinggal; a `@` mátrixszorzás, sor-oszlop skaláris szorzatokkal.</details>

4. Mi az alakszabálya $\mathbf{X}^T\mathbf{X}$ szorzatnak, ha $\mathbf{X}$ alakja (n, p)?
<details><summary>Válasz</summary>$\mathbf{X}^T$ alakja (p, n), így a szorzat (p, p); ez lesz a normálegyenlet mátrixa a következő alfejezetben.</details>

A NumPy `*` operátora elemenkénti szorzást végez, míg a `@` mátrixszorzást. A vektor- és mátrixdimenziókat ellenőrizzük, mert a nem szándékolt broadcasting numerikusan lefuthat úgy is, hogy hibás eredményt ad.

10.2.9 Rövid összefoglaló

A mátrixszorzás a lineáris predikció alapművelete: az $n \times p$ design mátrix minden sorát ugyanazzal a p hosszú súlyvektorral kombinálja skaláris szorzatként, így $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ egyszerre adja meg minden megfigyelés predikcióját. A design mátrix sora egy megfigyelés, oszlopa egy jellemző; az interceptet csupa 1-es oszlop képviseli. A transzponálás sor és oszlop felcserélése, a bázisok a mátrix oszlopvektorainak lineáris kombinációi. A NumPy `@` operátora mátrixszorzás, a `*` elemenkénti szorzás; a kettő nem felcserélhető. A következő alfejezetben ezt a műveletet használjuk a vetítéshez és a legkisebb négyzetes becsléshez, ahol $\mathbf{X}^T\mathbf{X}$ és $\mathbf{X}^T\mathbf{y}$ jelenik meg.

10.3 Vetítés és legkisebb négyzetek

> **Belépő · szint: közép · idő: 60 perc · eszköz: NumPy**
>
> Előfeltétel: 10.2. alfejezet · Célkészség: Vetítés és legkisebb négyzetek · Kimenet: működő kód és a labor ellenőrző tesztje.

A legkisebb négyzetes illesztést gyakran csak képletként tanuljuk meg, pedig mögötte nagyon egyszerű geometria van: egy vektort merőlegesen vetítünk egy altérre. Ebből a képből közvetlenül következik a normálegyenlet, és az is láthatóvá válik, miért veszélyes a majdnem kollineáris adat.

10.3.1 Cél és konkrét helyzet

Adott három megfigyelt pont: $(x_1, y_1) = (0, 1)$, $(x_2, y_2) = (1, 3)$, $(x_3, y_3) = (2, 4)$. A megfigyelési egység egy (x, y) pár; a koordináták dimenzió nélküli tanulóadatok, így itt nincs mértékegység. Célunk olyan $\hat{y}_i = w_0 + w_1 x_i$ egyenest illeszteni, amely minimalizálja a

$$Q(w_0, w_1) = \sum_{i=1}^{3} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{3} (y_i - w_0 - w_1 x_i)^2$$

négyzetes hibát. Mielőtt tovább olvasnál, gyors becslésként rajzold fel a pontokat: az első szakaszon 2, a másodikon 1 az y-emelkedés, ezért a legkisebb négyzetes meredekség kompromisszumként $w_1 = 1{,}5$ körül várható, a tengelymetszet pedig valamivel 1 fölé esik. A levezetés után ellenőrizzük ezt a sejtést.

10.3.2 Merőlegesség és vetítés

Írjuk a modellt mátrixalakba, ahogyan 10.2 a lineáris transzformációt bevezette:

$$\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} 1 \\ 3 \\ 4 \end{pmatrix}, \qquad \mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \end{pmatrix}.$$

A **design mátrix** $\mathbf{X} \in \mathbb{R}^{3 \times 2}$: minden sora egy megfigyelés, az első oszlop az intercept csupa 1-ese, a második az x_i értékek oszlopa. A súlyvektor $\mathbf{w} \in \mathbb{R}^2$, a predikció pedig $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \in \mathbb{R}^3$. A $\hat{\mathbf{y}}$ vektor tehát az $\mathbf{X}$ oszlopainak lineáris kombinációja, ezért eleme az **oszloptérnek** (*column space*):

$$C(\mathbf{X}) = \{\mathbf{X}\mathbf{v} : \mathbf{v} \in \mathbb{R}^2\} \subseteq \mathbb{R}^3.$$

Jelen példában ez egy kétdimenziós sík a háromdimenziós megfigyelési térben, amelyet a $\mathbf{c}_1 = (1, 1, 1)^T$ és $\mathbf{c}_2 = (0, 1, 2)^T$ oszlopvektorok feszítenek ki. A legkisebb négyzetes illesztés geometriai feladata: válasszuk $\mathbf{C}(\mathbf{X})$-ből az $\mathbf{y}$-hoz legközelebbi pontot.

A **reziduum** (*residual*) a megfigyelés és a predikció eltérése:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\mathbf{w},$$

ahol $\mathbf{r} \in \mathbb{R}^3$, és koordinátánként $r_i = y_i - \hat{y}_i$.

A kulcsfelismerés: akkor lesz $\hat{\mathbf{y}}$ a legközelebbi pont, ha az $\mathbf{r}$ reziduum **merőleges** az oszloptérre. Ha ugyanis $\mathbf{r}$ merőleges $\mathbf{C}(\mathbf{X})$ minden vektorára, akkor bármely más $\mathbf{v} \in C(\mathbf{X})$ esetén Pitagorasz tétele szerint

$$\|\mathbf{y} - \mathbf{v}\|^2 = \|\mathbf{r}\|^2 + \|\hat{\mathbf{y}} - \mathbf{v}\|^2 \geq \|\mathbf{r}\|^2,$$

tehát $\hat{\mathbf{y}}$ valóban minimális távolságú. A merőlegességet elég az altér bázisvektoraira, vagyis $\mathbf{X}$ oszlopaira megkövetelni:

$$\mathbf{X}^T \mathbf{r} = 0.$$

Ezt **normálegyenletnek** (*normal equations*) nevezzük. Behelyettesítve:

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) = 0 \implies \mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}.$$

A shape-ek: $\mathbf{X}^T\mathbf{X}$ mérete 2×2, $\mathbf{X}^T\mathbf{y}$ mérete 2×1, tehát a megoldandó rendszer két egyenlet két ismeretlenre.

Ha $\mathbf{X}$ oszlopai lineárisan függetlenek, akkor $\mathbf{X}^T\mathbf{X}$ invertálható, és a legkisebb négyzetes megoldás egyértelmű:

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.$$

A predikció ekkor az $\mathbf{y}$ vektor **merőleges vetítése** (*projection*) az oszloptérre:

$$\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} = \mathbf{P}_{\mathbf{X}}\mathbf{y}.$$

A $\mathbf{P}_{\mathbf{X}}$ mátrix a **vetítőmátrix** (*projection matrix*), mérete 3×3; szimmetrikus és idempotens, vagyis $\mathbf{P}_{\mathbf{X}}^2 = \mathbf{P}_{\mathbf{X}}$. A vetítés fogalmát kis vektoros ábrán is érdemes látni. Az alábbi Matplotlib-kód egy $\mathbb{R}^2$-beli altérre vetítést rajzol; a piros reziduum merőleges a zöld altéregyenesre.

```
import matplotlib.pyplot as plt
import numpy as np

a = np.array([3.0, 1.0])          # az egydimenziós altér bázisvektora
y = np.array([2.0, 3.0])
y_hat = (a @ y) / (a @ a) * a     # merőleges vetítés
r = y - y_hat

fig, ax = plt.subplots()
ax.quiver(0, 0, *y,      angles='xy', scale_units='xy', scale=1,
          color='black', label='y')
ax.quiver(0, 0, *a,      angles='xy', scale_units='xy', scale=1,
          color='green', label='altér')
ax.quiver(0, 0, *y_hat,  angles='xy', scale_units='xy', scale=1,
          color='blue', label='vetített y')
ax.quiver(*y_hat, *r,    angles='xy', scale_units='xy', scale=1,
          color='red', label='reziduum')
ax.set_xlim(-1, 4); ax.set_ylim(-1, 4)
ax.set_aspect('equal'); ax.legend()
```

Az ábra a futás után szemlélteti, hogy a reziduum derékszöget zár be az altérrel. A kód itt nem futtatott; a leírt geometriai viszony a vetítési képletből következik.

10.3.3 Rang, pszeudoinverz és kondíció

A **rang** (*rank*) az $\mathbf{X}$ oszlopvektorai között található lineárisan független vektorok maximális száma; NumPy-ban `np.linalg.matrix_rank`. Ha a rang egyenlő az oszlopok számával ($\operatorname{rank}(\mathbf{X}) = p$), akkor $\mathbf{X}$ **teljes oszloprangú**, és $\mathbf{X}^T\mathbf{X}$ invertálható. Ha $\operatorname{rank}(\mathbf{X}) < p$, akkor az egyenletrendszernek több $\mathbf{w}$ megoldása is ugyanazt a $\hat{\mathbf{y}}$ predikciót adja; a predikció egyedi, a súlyvektor nem.

A **pszeudoinverz** (*pseudoinverse*, Moore–Penrose-inverz) ilyenkor is előállít egy egyértelmű legkisebb négyzetes megoldást: ez a legkisebb normájú megoldás. NumPy-ban `np.linalg.pinv(X)`. A pszeudoinverz szinguláris felbontásra épül, és ranghiányos mátrixra is működik. A teljes rangú esetben ugyanazt adja, mint a normálegyenlet-megoldás, de numerikusan általában stabilabb.

A **kondíciószám** (*condition number*) azt méri, mennyire érzékeny a megoldás a bemeneti perturbációkra:

$$\kappa_2(\mathbf{X}) = \frac{\sigma_{\max}(\mathbf{X})}{\sigma_{\min}(\mathbf{X})},$$

ahol $\sigma_{\max}$ és $\sigma_{\min}$ az $\mathbf{X}$ legnagyobb és legkisebb szinguláris értéke; NumPy-ban `np.linalg.cond`. Ha κ közel van 1-hez, a feladat stabil; ha $\sigma_{\min}$ nagyon kicsi, κ nagy, és kis adatváltozás is nagy súlyváltozást okozhat. Fontos: a normálegyenlet mátrixának kondíciója $\kappa(\mathbf{X}^T\mathbf{X}) = \kappa(\mathbf{X})^2$, tehát a normálegyenletben még erősebb az instabilitás, mint magában az $\mathbf{X}$-ben.

10.3.3.1 Rang-, kondíció- és reziduális ellenőrzés minden megoldás előtt

A normálegyenlet csak akkor ekvivalens a legkisebb négyzetes feladattal, ha X teljes oszloprangú és a kondíciószáma nem extrém. A `lab/linear_solve.py` ezért minden megoldás előtt kiírja a rankot, a kondíciót és a reziduális normálegyenlet-maradékát:

Eset	Rank	Kondíció	Normálegyenlet	Ajánlott megoldó
teljes oszloprang	p	mérsékelt	megoldható	`lstsq` vagy SVD; a normálegyenlet is jó
kollineáris	$< p$	∞	nem oldható meg	`pinv`/`lstsq`, minimumnorma-megoldás
majdnem kollineáris	p	nagyon nagy	megoldható, de instabil	SVD/`lstsq`, regularizáció
aluldeterminált	$< p$	—	nem oldható meg	`pinv`/`lstsq`, egy megoldás kiválasztása

```python
import numpy as np

def diagnostics(X):
    X = np.asarray(X, dtype=float)
    s = np.linalg.svd(X, compute_uv=False)
    tol = max(X.shape) * np.finfo(float).eps * (s[0] if s.size else 0.0)
    rank = int((s > tol).sum())
    cond = float(s[0] / s[-1]) if s.size and s[-1] > 0 else float("inf")
    return {"shape": X.shape, "rank": rank,
            "full_column_rank": rank == X.shape[1],
            "condition": cond, "singular_values": s.tolist()}

def normal_equation(X, y):
    info = diagnostics(X)
    if not info["full_column_rank"]:
        raise ValueError("rankdeficit: a normálegyenlet szinguláris")
    beta = np.linalg.solve(X.T @ X, X.T @ y)
    residual = y - X @ beta
    return beta, float(np.linalg.norm(X.T @ residual))

def lstsq_reference(X, y):
    return np.linalg.lstsq(X, y, rcond=None)[0]
```

A stabil referencia mindig az `lstsq`/SVD; a normálegyenlet csak a diagnosztika után, dokumentált feltételekkel használható. A reziduális ortogonalitás ellenőrzése: $\|X^T(y - X\beta)\|$ kicsi a $\|X^T y\|$-hez képest. Rankdeficitnél a `pinv` minimumnorma-megoldást ad, de ez nem ugyanaz, mint bármelyik normálegyenletes megoldás; a különbséget jelenteni kell. A `tests/test_linear_solve.py` a teljes rangú, kollineáris, majdnem kollineáris és aluldeterminált esetet, valamint a `pinv` azonosságát ellenőrzi.

10.3.4 Teljes minimális munkapélda

Az alábbi kód a hárompontos feladatot oldd meg, és minden fontos invariánst ellenőriz. A kód a kezdőlabor alap numerikus csomagjain kívül mást nem igényel. Itt nem futtatjuk; a kimenethez tartozó várt értékeket kézi számításból adjuk meg.

```
import numpy as np

X = np.array([
    [1.0, 0.0],
    [1.0, 1.0],
    [1.0, 2.0],
])
y = np.array([1.0, 3.0, 4.0])

w_lstsq, residuals, rank, singular_values = np.linalg.lstsq(X, y, rcond=None)
y_hat = X @ w_lstsq
r = y - y_hat

# Kézi normálegyenlet, az elméleti képlet ellenőrzésére.
w_direct = np.linalg.solve(X.T @ X, X.T @ y)
w_pinv = np.linalg.pinv(X) @ y

print("w_lstsq =", w_lstsq)
print("y_hat =", y_hat)
print("r =", r)
print("X.T @ r =", X.T @ r)
print("rank =", rank)
print("singular_values =", singular_values)
print("cond(X) =", np.linalg.cond(X))
print("w_direct =", w_direct)
print("w_pinv =", w_pinv)
```

A kézi vezetésből elméletileg várt kimenet: $\hat{\mathbf{w}} = (7/6,\ 3/2)^T$, azaz `w_lstsq = [1.16666667, 1.5]`. A predikciók: $\hat{\mathbf{y}} = (7/6,\ 8/3,\ 25/6)^T$, azaz körülbelül `[1.1667, 2.6667, 4.1667]`. A reziduumok: $(-1/6,\ 1/3,\ -1/6)$. Ellenőrizzük az invariánsokat:

$$\mathbf{X}^T\mathbf{r} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} -1/6 \\ 1/3 \\ -1/6 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

tehát a reziduum valóban merőleges mindkét oszlopra. A rang 2, mert a két oszlop nem arányos. A szinguláris értékek $\sqrt{4+\sqrt{10}} \approx 2{,}676$ és $\sqrt{4-\sqrt{10}} \approx 0{,}915$, ezért $\kappa(\mathbf{X}) \approx 2{,}92$, ami stabil kis feladat. A `solve`, a `lstsq` és a `pinv` által adott súlyok ezen a jól kondicionált példán gyakorlatilag megegyeznek.

10.3.5 Majdnem kollineáris változók és az instabilitás

Hozzunk létre olyan adatot, ahol a második magyarázó változó majdnem konstansszorosa az elsőnek. Ez a gyakorlatban akkor fordul elő, ha két feature erősen összefügg. Az alábbi kód a jól értelmezhető $y = 2x_1 + x_2 + 0{,}5$ kapcsolatot használja; a zaj miatt a pontos súlyok nem reprodukálhatók gépfüggetlenül, ezért az itt közölt kimenet **szemléltető jellegű**, nem futtatott mérési jegyzék.

```
rng = np.random.default_rng(42)
n = 40
x1 = np.linspace(-2.0, 2.0, n)
x2 = 3.0 * x1 + 0.005 * rng.normal(size=n)
y_true = 2.0 * x1 + 1.0 * x2 + 0.5
y = y_true + 0.1 * rng.normal(size=n)
X = np.column_stack([np.ones(n), x1, x2])

w, *_ = np.linalg.lstsq(X, y, rcond=None)

# Kis perturbáció ugyanarra a design mátrixra.
y_perturbed = y + 0.02 * rng.normal(size=n)
w2, *_ = np.linalg.lstsq(X, y_perturbed, rcond=None)

print("cond(X) =", np.linalg.cond(X))
print("w =", w)
print("w_perturbed =", w2)
print("||w - w_perturbed|| =", np.linalg.norm(w - w2))
print("||y_hat - y_hat_perturbed|| =",
      np.linalg.norm(X @ w - X @ w2))
```

A megadott konstrukcióban x_2 majdnem $3x_1$, tehát a második és harmadik oszlop majdnem kollineáris. A kondíciószám várhatóan több száz vagy ezer körüli. A látványos jelenség: a $\mathbf{w}$ és $\mathbf{w}_2$ súlyvektorok **önmagukban** jelentősen eltérhetnek, mert a két súly kompenzálja egymást, miközben a predikciók különbsége kicsi marad. Ez nem hiba a kódban: az adat nem azonosítja egyértelműen a két együtthatót, csak a közelítő összegüket. A stabil eszköz itt is `lstsq`; a predikció marad stabil, a súlyvektorok értelmezése viszont óvatosságot igényel.

10.3.6 Feladatok

1. **Értelmezés.** A hárompontos példában számold ki a $\|\mathbf{y}\|^2$, $\|\hat{\mathbf{y}}\|^2$ és $\|\mathbf{r}\|^2$ értékeket, és igazold a Pitagorasz-összefüggést: $\|\mathbf{y}\|^2 = \|\hat{\mathbf{y}}\|^2 + \|\mathbf{r}\|^2$. Magyarázd el, miért pontosan a vetítésből következik ez.

2. **Megvalósítás.** Illessz egyenest a $(-1, 0)$, $(0, 2)$, $(1, 1)$ pontokra. Írj NumPy-kódot `lstsq`-val, ellenőrizd a normálegyenletet `solve`-val, és írd ki a reziduumvektorra a $\mathbf{X}^T\mathbf{r} = 0$ feltételt. Add meg a kondíciószámot is.

3. **Hibadiagnózis.** Egy kolléga megoldásként `w_bad = np.linalg.inv(X.T @ X) @ X.T @ y` sort használ. A majdnem kollineáris példán a predikciója mégsem rossz. Milyen diagnosztikai kísérlettel tudnád kimutatni, hogy a módszer instabilabb a kelleténél? Javasolj legalább egy konkrét numerikus összehasonlítást.

4. **Megváltoztatott adat.** A három eredeti ponthoz adj hozzá egy negyediket, amelyik a $(1, 3)$ pont pontos duplikátuma. Futtasd az illesztést, és válaszold meg: változik-e a design mátrix rangja, a becsült egyenes, és a reziduumok négyzetösszege? Hogyan kapcsolódik ez ahhoz a duplikációs szerződéshez, amelyről 6.3 szólt?

10.3.7 Megoldások és indoklás

1. A kiszámolt vektorokkal:

$$\|\mathbf{y}\|^2 = 1^2 + 3^2 + 4^2 = 26,$$

$$\|\hat{\mathbf{y}}\|^2 = \left(\frac{7}{6}\right)^2 + \left(\frac{8}{3}\right)^2 + \left(\frac{25}{6}\right)^2 = \frac{49 + 256 + 625}{36} = \frac{930}{36} = \frac{155}{6},$$

$$\|\mathbf{r}\|^2 = \left(-\frac{1}{6}\right)^2 + \left(\frac{1}{3}\right)^2 + \left(-\frac{1}{6}\right)^2 = \frac{1 + 4 + 1}{36} = \frac{1}{6},$$

és $\frac{155}{6} + \frac{1}{6} = \frac{156}{6} = 26$, tehát a Pitagorasz-összefüggés teljesül. A magyarázat: $\hat{\mathbf{y}}$ az $\mathbf{y}$ merőleges vetülete az oszloptérre, ezért $\mathbf{r}$ merőleges $\hat{\mathbf{y}}$-ra; az általános Pitagorasz-tételből épp ez az invariáns következik.

2. A feladat mátrixai:

$$\mathbf{X} = \begin{pmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}.$$

A normálegyenlet:

$$\mathbf{X}^T\mathbf{X} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}, \qquad \mathbf{X}^T\mathbf{y} = \begin{pmatrix} 3 \\ 1 \end{pmatrix},$$

ezért $\hat{\mathbf{w}} = (1,\ 0{,}5)^T$. A predikciók $(-1;\ 0{,}5)$, $(0;\ 1)$, $(1;\ 1{,}5)$, a reziduumok $(-0{,}5;\ 1;\ -0{,}5)$, és $\mathbf{X}^T\mathbf{r} = (0, 0)^T$. A szinguláris értékek $\sqrt{3}$ és $\sqrt{2}$, ezért $\kappa(\mathbf{X}) = \sqrt{3/2} \approx 1{,}225$; ez nagyon stabil. A kód:

```
X = np.array([[1.0, -1.0], [1.0, 0.0], [1.0, 1.0]])
y = np.array([0.0, 2.0, 1.0])
w, *_ = np.linalg.lstsq(X, y, rcond=None)
r = y - X @ w
print("w =", w)
print("X.T @ r =", X.T @ r)
print("cond(X) =", np.linalg.cond(X))
print("solve check =", np.linalg.solve(X.T @ X, X.T @ y))
```

3. A diagnosztikai kísérlet: legyártható egy kis perturbált $\mathbf{y}^* = \mathbf{y} + \varepsilon$, ahol ε nagyon kicsi, például a fenti majdnem kollineáris példában 0,02-szeres zaj. Hasonlítsd össze a $\|\hat{\mathbf{w}} - \hat{\mathbf{w}}^*\|$ eltérést a `lstsq` megoldás és az `inv(X.T @ X)` megoldás között. Az explicit inverz a normálegyenlet $\kappa(\mathbf{X})^2$-szeres kondíciójával dolgozik, ezért a súlyeltérés ott általában sokkal nagyobb. A javítás: `np.linalg.lstsq(X, y, rcond=None)` vagy legalább `np.linalg.solve(X.T @ X, X.T @ y)`; utóbbi még mindig a normálegyenletet oldja, ezért a `solve` is lehet érzékeny, de elkerüli az explicit inverzképzés plusz műveleteit. A legjobb a QR- vagy SVD-alapú `lstsq` vagy `pinv`.

4. A kibővített adat:

$$\mathbf{X}_4 = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{y}_4 = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 3 \end{pmatrix}.$$

A rang 2 marad, mert a duplikált sor nem hoz be új irányt. A normálegyenlet new mátrixa:

$$\mathbf{X}_4^T \mathbf{X}_4 = \begin{pmatrix} 4 & 4 \\ 4 & 6 \end{pmatrix}, \qquad \mathbf{X}_4^T \mathbf{y}_4 = \begin{pmatrix} 11 \\ 14 \end{pmatrix},$$

amelynek megoldása $\hat{\mathbf{w}} = (5/4,\ 3/2)^T$. A meredekség nem változott, az intercept azonban $7/6 \approx 1{,}167$-ről 1,25-ra nőtt, mert a $(1, 3)$ pont kétszeres súlyt kapott. Ez összhangban van 6.3 tanulságával: a duplikátumot explicit döntéssel kell kezelni, mert ha nem valódi ismételt mérés, akkor torzítja a súlyozást.

10.3.8 Gyakori tévedés és ellenpélda

A tankönyvi képletben szereplő $(\mathbf{X}^T\mathbf{X})^{-1}$ miatt sokan azt gondolják, hogy a helyes megvalósítás az explicit inverz:

```
w_bad = np.linalg.inv(X.T @ X) @ X.T @ y
```

Ez a megoldás kis, jól kondicionált mátrixokra gyakorlatilag rendben van, de **nem általánosan a legjobb numerikus választás**. Az ellenpélda: ha $\kappa(\mathbf{X}) = 10^4$, akkor $\kappa(\mathbf{X}^T\mathbf{X}) = 10^8$, tehát a normálegyenlet már eleve nyolc nagyságrendnyi érzékenységet visz a rendszerbe, és az explicit inverz további kerekítést ad. A `lstsq` QR- vagy SVD-felbontáson alapulva elkerüli az $\mathbf{X}^T\mathbf{X}$ explicit képzését. Ez ugyanaz a véges pontossági tanulság, amelyet 9.3 a lépésköz-optimumnál bemutatott: a kerekítési hiba a rossz képletből előbb-utóbb elrontja a numerikus eredményt.

10.3.9 Önellenőrzés

- Tudom-e bizonyítani az $\mathbf{X}^T\mathbf{r} = 0$ feltételből, hogy a vetített pont minimális távolságú?
- Meg tudom-e mondani egy adott design mátrixról, mikor egyértelmű a súlyvektor, és mikor csak a predikció egyedi?
- Le tudom-e futattatni a majdnem kollineáris kísérletet úgy, hogy a seed és a seed-függő pontatlanság jelölt, ne pedig hallgatólagos?
- El tudom-e magyarázni, miért nézhet stabilnak egy instabil megoldó a predikción, miközben a súlyvektor erősen mozog?

10.3.10 Rövid összefoglaló

A legkisebb négyzetes illesztés geometriailag az $\mathbf{y}$ vektor merőleges vetítése az $\mathbf{X}$ mátrix oszlopterére; a reziduum merőlegessége adja a $\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$ normálegyenletet. Teljes oszloprang esetén a súlyvektor egyértelmű, ranghiány esetén a pszeudoinverz a legkisebb normájú megoldást választja. A kondíciószám a megoldás érzékenységét méri; a majdnem kollineáris változóknál a súlyok instabilak lehetnek, miközben a predikció stabil marad. Tényleges számításhoz az explicit mátrixinverz helyett `np.linalg.lstsq` vagy `np.linalg.pinv` ajánlott. A következő alfejezetben a szinguláris felbontás és a dimenziócsökkentés épít erre a vetítési képre.

10.4 Sajátérték, SVD és dimenziócsökkentés előképe

Belépő · szint: közép · idő: 60 perc · eszköz: NumPy

Előfeltétel: 10.3. alfejezet · Célkészség: Sajátérték, SVD és dimenziócsökkentés előképe · Kimenet: működő kód és a labor ellenőrző tesztje.

10.4.1 Cél és konkrét helyzet

A 10.3. alfejezetben a legkisebb négyzetes illesztést úgy értelmeztük, mint merőleges vetítést az oszloptérre. Most a kérdés a következő: ha az adatpontok egy felhőt alkotnak valamilyen térben, mely irányok mentén terülnek el a legjobban? Ez azért fontos, mert a későbbi dimenziócsökkentés, illetve a főkomponens-elemzés előképét adja: egy sokdimenziós táblában gyakran néhány irány hordozza az eltérések nagy részét, a többi pedig szinte zaj.

A tanulási ellenőrzéshez nem a teljes Northstar rendeléseit, hanem egy ismert szerkezetű, mesterséges pontfelhőt használunk. Így a számított főirányt a generálásból ismert irányhoz tudjuk hasonlítani. A bemenet egy $n \times p$ numerikus mátrix, $\mathbf{X}$; a megfigyelési egység egy szimulált rendelés. A generálás:

- $z_1 \sim \mathcal{N}(0, 2^2)$: egy nagy szóródású látens jellemző (szórás: 2);
- $z_2 \sim \mathcal{N}(0, 0.3^2)$: egy kis szóródású látens jellemző (szórás: 0,3);
- a kettőt 35°-kal elforgatjuk az (x_1, x_2) síkban.

Előzetes becslés: mivel z_1 szórása sokkal nagyobb, a főiránynak az elforgatás után a 35°-os irányban kell állnia, a második irány pedig arra merőleges. A szinguláris értékek négyzeteinek és a mintaszámnak az aránya közelítőleg a két generáló varianciát adja. A fejezet végére teljes `NumPy`-kóddal ellenőrizzük ezt.

10.4.2 Centrált mátrix és a kovarianciamátrix előképe

Legyen $\mathbf{X}$ $n \times p$ méretű: sorai megfigyelések, oszlopai numerikus jellemzők. A főirányoknak azt az irányt keressük, amely mentén a megfigyelések *szóródnak*, nem feltétlenül azt, amely mentén az értékek nagyok. Ezért először oszloponként kivonjuk az átlagot:

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^T.$$

Itt $\mathbf{1}$ az $n \times 1$ csupa egy vektor, $\bar{\mathbf{x}}$ pedig a $p \times 1$ oszlopátlag-vektor. A centrálás után minden oszlop átlaga nulla. A minta-kovarianciamátrix becslése:

$$\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^T\tilde{\mathbf{X}},$$

aminek mérete $p \times p$. A főátlóban az egyes oszlopok szórásnégyzeteinek becslése, az átlón kívül kovarianciák állnak. A mértékegység vegyes lehet: ha az első oszlop kilométer, a második euró, akkor $\mathbf{C}$ elemei km^2, $\text{km} \cdot \text{EUR}$ és EUR^2 egységekben vannak; ez már jelzi, hogy a nyers kovarianciamátrix irányai nem mértékegység-függetlenek.

Adott $\mathbf{u}$ egységvektorra, azaz $\|\mathbf{u}\| = 1$, a pontok vetülete $\tilde{\mathbf{X}}\mathbf{u}$, alakja $n \times 1$. Ennek varianciája:

$$\operatorname{Var}(\tilde{\mathbf{X}}\mathbf{u}) = \frac{1}{n-1}(\tilde{\mathbf{X}}\mathbf{u})^T(\tilde{\mathbf{X}}\mathbf{u}) = \mathbf{u}^T\mathbf{C}\mathbf{u}.$$

A cél tehát: találni olyan $\mathbf{u}$ egységvektort, amelyre a $\mathbf{u}^T\mathbf{C}\mathbf{u}$ másodfokú kifejezés a lehető legnagyobb. Ez vezet a sajátvektorokhoz.

10.4.3 Sajátvektor és sajátérték

Egy négyzetes, $p \times p$ méretű $\mathbf{A}$ mátrixra a $\mathbf{v} \neq \mathbf{0}$ vektor *sajátvektor* (eigenvector), ha a mátrixszorzás csak a hosszát skálázza:

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}.$$

A λ skalár a *sajátérték* (eigenvalue). A sajátvektor tehát olyan irány, amelyen a lineáris transzformáció nem forgat, csak nyújt vagy zsugorít. Például

$$\mathbf{A} = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}, \qquad \mathbf{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \mathbf{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Ekkor $\mathbf{A}\mathbf{v}_1 = 2\mathbf{v}_1$ és $\mathbf{A}\mathbf{v}_2 = 0.5\mathbf{v}_2$; a sajátértékek 2 és 0.5, a sajátvektorok a koordinátatengelyek.

A $\mathbf{C}$ minta-kovarianciamátrix szimmetrikus és pozitív szemidefinit. Itt a spektráltételt bizonyítás nélkül használjuk: egy szimmetrikus mátrix felbontható ortonormált sajátvektorokkal:

$$\mathbf{C} = \mathbf{V}\Lambda\mathbf{V}^T,$$

ahol $\mathbf{V}$ oszlopai a sajátvektorok, Λ pedig a sajátértékekből álló átlós mátrix. A legnagyobb sajátértékhez tartozó sajátvektor a $\mathbf{u}^T\mathbf{C}\mathbf{u}$ kifejezés maximumát adja az egységgömbön; ez a legnagyobb varianciájú irány. A második legnagyobb sajátértékhez tartozó, az elsőre merőleges vektor adja az erre merőleges fő szóródási irányt.

A korábbi 10.3. alfejezetből már ismerjük az $\mathbf{X}^T\mathbf{X}$ mátrixot: ott a normálegyenletben jelent meg. A ranghiányos esetben a pszeudoinverz adott egyértelmű megoldást. Most ugyanannak a mátrixnak a *főirányait*, nem egy konkrét egyenlet megoldását keressük.

10.4.4 SVD és szinguláris értékek

Bármely $m \times n$ valós $\mathbf{M}$ mátrix felbontható szinguláris értékek szerint (singular value decomposition, SVD):

$$\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^T.$$

A redukált felbontásban, ha $r = \min(m, n)$:

- $\mathbf{U}$ mérete $m \times r$, oszlopai az ortonormált *bal szingulárvektorok*;
- Σ mérete $r \times r$, átlóján a $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$ *szinguláris értékek*;
- $\mathbf{V}^T$ mérete $r \times n$, sorai a *jobb szingulárvektorok*, így $\mathbf{V}$ oszlopai ortonormáltak.

Ha $\mathbf{M}$ a centrált $\tilde{\mathbf{X}}$ mátrix, akkor $m = n_{\text{minta}}$, $n = p$. Mivel $n_{\text{minta}} > p$ a példánkban, $\mathbf{U}$ mérete $n \times p$, Σ mérete $p \times p$, $\mathbf{V}$ mérete $p \times p$. A $\mathbf{U}$ és $\mathbf{V}$ ortogonalitása miatt $\mathbf{U}^T\mathbf{U} = \mathbf{I}_p$ és $\mathbf{V}^T\mathbf{V} = \mathbf{I}_p$. Ebből:

$$\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} = \mathbf{V}\Sigma^2\mathbf{V}^T.$$

Összevetve a $\mathbf{C}$-re adott sajátfelbontással: a jobb szingulárvektorok megegyeznek a kovariancia-mátrix sajátvektoraival, és a j-edik főirányon a variancia

$$\lambda_j = \frac{\sigma_j^2}{n-1}.$$

A szinguláris érték tehát nem szórás: a hossza úgy skálázódik, mint a centrált adat euklideszi normája. Ezt a mintapéldában ellenőrizzük.

Kézi kis kontroll: ha

$$\mathbf{M} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix},$$

akkor $\mathbf{M} = \mathbf{I}\,\mathrm{diag}(3,2)\mathbf{I}$, tehát a szinguláris értékek 3 és 2, a jobb szingulárvektorok a koordinátatengelyek, a rang 2.

10.4.5 Rank és numerikus küszöb

A *rang* (rank) a lineárisan független oszlopok maximális száma. Az SVD-ben a rang a pozitív szinguláris értékek száma. Pontos lineáris függésnél a legkisebb szinguláris érték nulla:

$$\mathbf{X}_3 = \begin{pmatrix} x_1 & x_2 & -1.5x_1 + 0.5x_2 \end{pmatrix},$$

ahol a harmadik oszlop az első kettő lineáris kombinációja, így a rang legfeljebb 2. A gyakorlatban mérési zaj vagy lebegőpontos kerekítés miatt a nulla helyett nagyon kicsi σ jelenik meg. A `np.linalg.matrix_rank` ezért egy relatív tűréssel dönt; a pontos mátrixnál a kis szinguláris érték a tűrés alatt marad, a zajosnál viszont lehet, hogy a tűrés fölé kerül. A rang tehát numerikus döntés is, nem csak algebrai fogalom. A 9.4. alfejezet üzenete itt visszatér: a mátrix skálázása megváltoztathatja, hogy ugyanaz a fizikai eltérés mekkora szinguláris értékként jelenik meg.

10.4.6 Dimenziócsökkentés előképe

A legnagyobb $k < p$ szinguláris érték megtartásával készíthető közelítés:

$$\tilde{\mathbf{X}}_k = \sum_{j=1}^{k} \sigma_j \mathbf{u}_j \mathbf{v}_j^T = \mathbf{U}_k \Sigma_k \mathbf{V}_k^T.$$

Az új koordináták, a *főirányokra vetített* értékek:

$$\mathbf{Z}_k = \tilde{\mathbf{X}}\mathbf{V}_k, \qquad \text{shape: } n \times k.$$

A rekonstrukció $\mathbf{Z}_k\mathbf{V}_k^T$, és az elhagyott irányok hibáját a $\sigma_{k+1}^2 + \cdots + \sigma_p^2$ összeg méri. Ha a kis szinguláris értékek elhanyagolhatók, akkor a p dimenziós pontfelhő jó közelítéssel k dimenzióban ábrázolható. A formális PCA-modellhez ez a kép, de a skálázási és értelmezési döntéseket a 24.1. fejezetre hagyjuk.

10.4.6.1 Oracle-invariánsok degenerált spektrumnál

A sajátvektor nem egyedi: ha $\mathbf{v}$ sajátvektor, akkor $-\mathbf{v}$ is az, és azonos sajátértékhez tartozó sajátvektorok tetszőleges bázisa is sajátvektor. Degenerált spektrumnál (például az identitásmátrixnál) minden irány sajátirány. Ezért a teszt **nem** hasonlíthatja össze a sajátvektorokat bitazonosan; a helyes oracle-invariánsok:

Mit hasonlíts?	Miért invariáns?
sajátértékek rendezett spektruma	a hasonlósági transzformáció megőrzi
szinguláris értékek	A^TA sajátértékeinek négyzetgyöke, előjelfüggetlen
k rangú projektor $P = U_kU_k^T$	előjel- és bázisfüggetlen
magyarázott varianciahányad	a spektrum aránya
rekonstrukciós hiba	a kihagyott sajátértékek összege
$\|A - USV^T\|$	a felbontás egyértelmű a spektrum szintjén

Példa degenerált esetre: $A = I_2$ sajátértékei $[1, 1]$, szinguláris értékei $[1, 1]$; a sajátvektorok tetszőleges ortonormált bázist alkothatnak. A $k = 1$ projektor viszont bármelyik választott irányra ugyanaz az I_1 altérre vetít, és a teszt a projektor idempotenciáját ($P^2 = P$), nyomát ($\operatorname{tr} P = k$) és a spektrum egyezését ellenőrzi, nem a konkrét vektort.

```
import numpy as np

def pca_svd(X):
    Xc = X - X.mean(axis=0, keepdims=True)
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    eigenvalues = S**2 / (len(Xc) - 1)
    return Vt, eigenvalues, eigenvalues / eigenvalues.sum()

def projector(Vt, k):
    basis = Vt[:k].T
    return basis @ basis.T

X = np.array([[1.0, 0.1], [2.0, 0.2], [3.0, 0.3], [4.0, 0.4]])
Vt, eigenvalues, ratio = pca_svd(X)
P = projector(Vt, 1)
assert np.allclose(P @ P, P)
assert np.isclose(np.trace(P), 1.0)
assert np.isclose(ratio.sum(), 1.0)
# előjelfordítás ugyanazt a projektort adja
Vt_flipped = Vt.copy(); Vt_flipped[0] = -Vt_flipped[0]
assert np.allclose(projector(Vt_flipped, 1), P)
```

A `lab/spectral.py` `spectra()`, `pca_svd()`, `projector()`, `reconstruction_error()` és `sign_flip_invariance()` függvényei ezeket az invariánsokat adják. A `tests/test_spectral.py` a teljes rangú példát, az identitás degenerált spektrumát, az előjelfordítást, a spektrum-egyezést és a rekonstrukciós hibát ellenőrzi.

10.4.7 Teljes minimális munkapélda

Az alábbi kód előállítja a forgatott felhőt, centrál, SVD-t számol, és ábrázolja a főirányokat. A futást helyben kell elvégezni; az itt mutatott értékek *elméletileg várt, szemléltető* értékek, nem előre rögzített mérési jegyzőkönyv.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(20261020)
n = 240

# Látens jellemzők: egy nagy és egy kis szórású irány
z1 = rng.normal(loc=0.0, scale=2.0, size=n)
z2 = rng.normal(loc=0.0, scale=0.3, size=n)

# 35 fokos forgatás az (x1, x2) síkban
t_deg = 35.0
t = np.deg2rad(t_deg)
R = np.array([[np.cos(t), -np.sin(t)],
              [np.sin(t),  np.cos(t)]])

# Minden sor egy megfigyelés: shape (n, 2)
X = (R @ np.vstack([z1, z2])).T

# Centrálás, majd redukált SVD
Xc = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)

print("s === ", s)
print("Vt sorai:", Vt[0], Vt[1])

# Rekonstrukció: U * s oszloponként skáláz, majd Vt-vel szoroz
Xr = (U * s) @ Vt
print("max rekonstrukciós hiba:", np.max(np.abs(Xc - Xr)))

# Variancia a főirányokon
vars_axes = (s ** 2) / (n - 1)
print("varianciák a főirányokon:", vars_axes)
print("teljes variancia:", vars_axes.sum(),
      Xc.var(axis=0, ddof=1).sum())

# Az első főirány szöge, előjeltől eltekintve
angle = np.rad2deg(np.arctan2(Vt[0, 1], Vt[0, 0])) % 180
print("első főirány szöge:", angle)

# Ábra
fig, ax = plt.subplots(figsize=(6.4, 6.4))
ax.scatter(Xc[:, 0], Xc[:, 1], s=18, alpha=0.5,
           label="centrált megfigyelés")
for j, color in zip([0, 1], ["tab:red", "tab:blue"]):
    sd = s[j] / np.sqrt(n - 1)
    v = Vt[j] * 2.0 * sd
    ax.arrow(0, 0, v[0], v[1], width=0.08, head_width=0.30,
             length_includes_head=True, color=color,
             label=f"{j + 1}. főirány, sd={sd:.3f}")
ax.axhline(0, color="gray", linewidth=0.8)
ax.axvline(0, color="gray", linewidth=0.8)
ax.set_xlabel("x1: centrált változó")
ax.set_ylabel("x2: centrált változó")
ax.set_title("SVD-főirányok a forgatott pontfelhőn")
ax.set_aspect("equal", adjustable="datalim")
ax.legend()
```

```
plt.savefig("s040_principal_directions.png", dpi=150,
            bbox_inches="tight")
```

A rang kérdését külön kis példán mutatjuk be. Ha a harmadik oszlop pontosan lineáris kombináció, a rang 2; ha nagyon kis zajt adunk hozzá, a numerikus rang a tűrés függvényében lehet 2 vagy 3:

```
x3 = -1.5 * X[:, 0] + 0.5 * X[:, 1]  # pontos lineáris függés
X3 = np.column_stack([X, x3])
print("pontos rang:", np.linalg.matrix_rank(X3))

x3n = x3 + rng.normal(scale=1e-8, size=n)
X3n = np.column_stack([X, x3n])
print("zajos alap rang:", np.linalg.matrix_rank(X3n))
print("nagy tűrés melletti rang:",
      np.linalg.matrix_rank(X3n, tol=1e-5))
```

10.4.8 Eredmény értelmezése

Az SVD matematikai azonossága miatt a rekonstrukciós hiba $\max|\tilde{\mathbf{X}} - \mathbf{U}\Sigma\mathbf{V}^T|$ értékének 10^{-12} alatt kell maradnia `float64` esetén; ez az egyik stabil ellenőrző invariáns. A `vars_axes` elemek elméletileg 4-hez és 0.09-hez esnek közel, mert a forgatás nem változtatja meg a teljes varianciát, csak elforgatja a koordinátarendszert. Az `angle` várt értéke 35 fok, mert a nagyobb szóródású z_1 tengelye a forgatás után ebben az irányban áll; a minta véletlenszerűsége miatt néhány tizedfok eltérés várható. A második főirány erre merőleges, tehát közelítőleg 125 fok.

A harmadik oszlopnál a pontos lineáris kombináció esetén a legkisebb szinguláris érték gyakorlatilag nulla, és a numerikus rang 2. A 10^{-8} szórású zaj hatására a σ_3 körülbelül $\sqrt{n}\cdot 10^{-8}$ nagyságrendű lesz, ami az alapértelmezett tűrés felett lehet, így a rang hivatalosan 3; nagyobb tűréssel viszont ismét 2-nek tekintjük. A döntést tehát nem érdemes automatikusan a nyers `matrix_rank` kimenetére bízni, hanem a szinguláris értékek nagyságrendjét kell értelmezni. Ez a későbbi PCA-nál különösen fontos.

10.4.9 Feladatok

10.4.9.1 1. feladat: értelmezés kézi mátrixon

Tekintsd a következő, már centrált mátrixot:

$$\mathbf{X} = \begin{pmatrix} 2 & -1 \\ 0 & 2 \\ -2 & -1 \end{pmatrix}.$$

Számítsd ki kézzel $\mathbf{X}^T\mathbf{X}$ értékét. Melyek a sajátértékei? Melyik a főirány? Add meg a szinguláris értékeket is.

10.4.9.2 2. feladat: implementálás eltérő forgatással

Generáljon 150 megfigyelést 60°-os forgatással, $z_1 \sim \mathcal{N}(0, 3^2)$ és $z_2 \sim \mathcal{N}(0, 0.5^2)$ összetevőkkel, `np.random.default_rng(42)` seeddel. Centráljon, számítson SVD-t, majd ellenőrizze három feltételt: a rekonstrukciós hiba $< 10^{-10}$; az első főirány szöge előjeltől eltekintve 60°-hoz képest 5 fokon belül van; a $\sigma_j^2/(n-1)$ értékek 9-hez és 0.25-höz vannak közel.

10.4.9.3 3. feladat: hibadiagnózis

Egy jegyzetfüzetben a szerző így számol főirányt:

```
C = X.T @ X
vals, vecs = np.linalg.eigh(C)
print(vecs[:, -1])
```

A bemeneti $\mathbf{X}$ mátrix egyik oszlopa $x_1 = 100 + \mathcal{N}(0,1)$, a másik $x_2 = \mathcal{N}(0,1)$. Az eredmény szerint a főirány szinte pontosan az x_1 irány, és a legnagyobb sajátérték óriási. Mi a fogalmi hiba, és mi a helyes számítás?

10.4.9.4 4. feladat: megváltozott mértékegység

A 35 fokos alapfeladat $\mathbf{X}$ mátrixát úgy skálázzák, hogy az első oszlopot 1000-rel szorozzák, a másodikat 0.01-gyel. Számítsd ki ismét az SVD-t a centrált skálázott mátrixon. Hogyan változik a főirány? Magyarázd meg, miért nem invariáns az eredmény a mértékegységre.

10.4.10 Megoldások és indoklás

10.4.10.1 1. feladat megoldása

A mátrix oszlopátlagai nullák, mert $2 + 0 - 2 = 0$ és $-1 + 2 - 1 = 0$.

$$\mathbf{X}^T\mathbf{X} = \begin{pmatrix} 2 & 0 & -2 \\ -1 & 2 & -1 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ 0 & 2 \\ -2 & -1 \end{pmatrix} = \begin{pmatrix} 8 & 0 \\ 0 & 6 \end{pmatrix}.$$

A sajátértékek 8 és 6, a sajátvektorok $\mathbf{e}_1$ és $\mathbf{e}_2$. A főirány $\mathbf{e}_1 = (1,0)^T$, mert $8 > 6$. Az SVD-ből $\sigma_1 = \sqrt{8} = 2\sqrt{2} \approx 2.828$ és $\sigma_2 = \sqrt{6} \approx 2.449$, mert $\mathbf{X}^T\mathbf{X} = \mathbf{V}\Sigma^2\mathbf{V}^T$. A rang 2.

10.4.10.2 2. feladat megoldása

```
import numpy as np

rng = np.random.default_rng(42)
n = 150
z1 = rng.normal(0.0, 3.0, n)
z2 = rng.normal(0.0, 0.5, n)

t = np.deg2rad(60.0)
R = np.array([[np.cos(t), -np.sin(t)],
              [np.sin(t),  np.cos(t)]])
X = (R @ np.vstack([z1, z2])).T

Xc = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)

err = np.max(np.abs((U * s) @ Vt - Xc))
angle = np.rad2deg(np.arctan2(Vt[0, 1], Vt[0, 0])) % 180
vars_axes = s ** 2 / (n - 1)

print(err, angle, vars_axes)
# Ellenőrzés
```

```
assert err < 1e-10
assert abs(angle - 60.0) < 5.0
assert abs(vars_axes[0] - 9.0) < 1.0
assert abs(vars_axes[1] - 0.25) < 0.1
```

A tűrések szándékosan bőek, mert kis mintán a becsült variancia jelentősen eltérhet a generáló paramétertől. A rekonstrukciós invariáns viszont szoros, mert az SVD matematikai azonosság, nem statisztikai becslés.

10.4.10.3 3. feladat megoldása

A hiba az, hogy a kód $\mathbf{X}^T\mathbf{X}$ mátrixon dolgozik a *centrálás nélkül*. Az $x_1 = 100 + z$ oszlop átlaga 100, ezért az $\mathbf{X}^T\mathbf{X}$ mátrixban a 100^2 nagyságrendű elem dominál. A számított „főirány” nem a szóródás, hanem az origótól mért nagy eltolás iránya. Helyesen:

```
Xc = X - X.mean(axis=0)
C = Xc.T @ Xc / (len(Xc) - 1)
vals, vecs = np.linalg.eigh(C)
print(vecs[:, -1])  # a legnagyobb szórás iránya
```

A centrálás után az első oszlop varianciája 1-hez, a másodiké szintén 1-hez közeli, így a főirány már nem lesz triviálisan x_1.

10.4.10.4 4. feladat megoldása

```
X_scaled = X.copy()
X_scaled[:, 0] *= 1000.0
X_scaled[:, 1] *= 0.01

Xc_s = X_scaled - X_scaled.mean(axis=0)
_, s_s, Vt_s = np.linalg.svd(Xc_s, full_matrices=False)
angle_s = np.rad2deg(np.arctan2(Vt_s[0, 1], Vt_s[0, 0])) % 180
vars_s = s_s ** 2 / (len(X_scaled) - 1)

print(angle_s, vars_s)
```

Az első oszlop 1000-szeresre növelése annak szórásnégyzetét 10^6-szorosára növeli; a második oszlop 0.01-szeresre csökkentése a másik varianciát 10^{-4}-szeresére. Az új főirány ezért szinte pontosan az x_1 tengely, vagyis a szög 0 fokhoz közeli. A tanulság: a nyers SVD-főirány nem mértékegység-független; a PCA előtt a 24.1. fejezetben a standardizálási döntést expliciten meg kell hozni.

10.4.11 Gyakori tévedés ellenpéldával

A legnagyobb varianciájú irány nem szükségképpen a cél szempontjából leghasznosabb irány. Generáljunk 300 megfigyelést így:

- $x_1 \sim \mathcal{N}(0, 10)$: nagy szóródású, de tartalmilag irreleváns háttérváltozó;
- $x_2 \sim \mathcal{N}(0, 1)$: kis szóródású, de a célváltozóhoz szorosan kapcsolódó jellemző;
- $y = 0.9x_2 + \varepsilon$, ahol $\varepsilon \sim \mathcal{N}(0, 0.1)$.

Az első főirány szinte kétségtelenül x_1 lesz, mert a varianciája 100-szoros. Ha a feladat a y célváltozó előrejelzése, akkor x_1 rossz választás, mert a generálás szerint független y-tól. A korrelációban ezt az ellenpéldát mutatja: $\mathrm{corr}(x_1, y) \approx 0$, míg $\mathrm{corr}(x_2, y) \approx 0.9$. A főirány tehát a *reprezentáció tömörségét* méri, nem a *célváltozóval való kapcsolatot*. Ezért a dimenziócsökkentést soha nem helyettesítheti automatikusan a felügyelt feature-szelekció.

Egy másik gyakori félreértés: a $\sigma_j^2/(n-1)$ „megmagyarázott variancia” arányt összekeverik a modellpontossággal. Nem minden eltérés számít a döntés szempontjából; a 6.4. alfejezetben látott típuscédulák példája is mutatja, hogy az adatválasztás szemantikája elsődleges.

10.4.12 Önellenőrzés

1. Mi a kapcsolat a centrált $\tilde{\mathbf{X}}$ SVD-je és a minta-kovarianciamátrix sajátfelbontása között?
2. Mit jelent, ha a legkisebb szinguláris érték nulla?
3. Miért nem használható a főirány automatikusan célváltozóra optimalizált feature-ként?
4. Mi történik a főiránnyal, ha az egyik oszlopot ezerszeresére skálázzuk?

A válaszok a fenti szakaszokban találhatók; ha bármelyik nem megy, a megoldásokhoz érdemes visszalapozni.

PCA előtt az adatokat általában centrálni kell, és eltérő skálák esetén indokolt lehet a standardizálás. A megőrzött varianciaarány nem azonos a prediktív információ megőrzésével, a főkomponensek pedig önmagukban nem kapnak oksági értelmezést.

10.4.13 Rövid összefoglaló

A szinguláris felbontás a centrált adatmátrixot ortogonális komponensekre bontja. A jobb szingulárvektorok a kovarianciamátrix sajátvektorai, tehát a pontfelhő főirányait adják; a $\sigma_j^2/(n-1)$ értékek mutatják a vetített varianciát az egyes irányokon. A rang a pozitív szinguláris értékek száma, de numerikusan tűrésfüggő döntés. A legnagyobb varianciájú irány nem feltétlenül a legjobb prediktor, ezért a dimenziócsökkentés és a felügyelt feature-szelekció külön döntés. A későbbi PCA-fejezetben ehhez a skálázás és az értelmezési szerződés csatlakozik.

11 Derivált, gradiens és optimalizálás kezdő kalkulusból

11.1 Meredekség és numerikus derivált

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 10.4. alfejezet · Célkészség: Meredekség és numerikus derivált · Kimenet: működő kód és a labor ellenőrző tesztje.

11.1.1 Cél és konkrét helyzet

A Northstar késési modelljében (lásd 1.1. alfejezet) rendelésenként becsülünk egy kockázatot. A modell bemenete tartalmazhatja a kosárértéket, a távolságot vagy a prioritási jelzést, kimenete pedig egy valószínűség. Ha egy ilyen modellt ábrázolunk, azonnal felmerül a kérdés: egy adott pontban mennyire meredek a görbe? A derivált (derivative) erre a pillanatnyi változási rátára ad számot. A derivált előjele és nagysága mondja meg, hogy egy kis bemeneti növelés körülbelül mennyivel növeli vagy csökkenti a kimenetet, ezért később az optimalizáló algoritmusok és a veszteségfüggvények elemzésének is alapja.

Ebben az alfejezetben a bemenet egy valós szám x, a modell egy $f(x)$ függvény, a megfigyelési egység pedig egy konkrét kiértékelési pont $x = a$. A cél az $f'(a)$ meredekség numerikus becslése és ellenőrzése. A tanuló labor két tesztfüggvényt használ: $f(x) = x^2$ és a logisztikus $\sigma(x) = 1/(1 + e^{-x})$ függvényt. Az egység kulcsfontosságú: ha x mértékegysége km és f értéke eurócent, a derivált egysége eurócent/km.

Induló feladat a megoldás előtt: becsüld meg kézzel $f(x) = x^2$ meredekségét az $x = 3$ pontban a 3 és 4 között húzott szelővel! A szelő meredeksége $(4^2 - 3^2)/(4 - 3) = 7$. Vajon az érintő ennél nagyobb vagy kisebb? A függvény gyorsulva nő, ezért a 3 és 4 közötti átlagos meredekség nagyobb, mint a 3-beli pillanatnyi érték; előzetesen nagyjából 6 körüli eredményt várunk.

11.1.2 Szelőből érintő: a differenciahányados

Az f függvény grafikonján az a és $a + h$ pontokon átmenő egyenes a szelő. A szelő meredekségét differenciahányadosnak (difference quotient) nevezzük:

$$D_h f(a) = \frac{f(a + h) - f(a)}{h},$$

ahol $h \neq 0$ a lépésköz (step size). A képletben a és h ugyanazon mértékegységben értendő, h előjeles; negatív h hátrafelé vett differenciát ad. A hányados egysége $[f]/[x]$, vagyis a kimenet egysége osztva a bemenet egységével. Ha f eurócentben, x kilométerben adott, a differenciahányados eurócent/km dimenziójú.

Kis számpélda $f(x) = x^2$, $a = 3$ mellett:

- $h = 0{,}1$: $\frac{3{,}1^2 - 3^2}{0{,}1} = \frac{9{,}61 - 9}{0{,}1} = 6{,}1$,
- $h = 0{,}01$: $\frac{3{,}01^2 - 9}{0{,}01} = 6{,}01$,
- $h = 0{,}001$: $\frac{3{,}001^2 - 9}{0{,}001} = 6{,}001$.

A sorozat a 6 felé tart. Az ábrán a szelő egyre jobban rásimul az érintőre, ahogy a két metszéspont közeledik egymáshoz. A következő kód a szelőt és az érintőt rajzolja fel:

```
import numpy as np
import matplotlib.pyplot as plt

def f_sq(x):
    return x * x

a = 3.0
h = 1.0
tangent_slope = 2.0 * a  # f'(x)=2x, tehát f'(3)=6

xs = np.linspace(2.0, 4.5, 200)
plt.figure(figsize=(6.5, 4))
plt.plot(xs, f_sq(xs), label="$x^2$", color="black")
plt.plot([a, a + h], [f_sq(a), f_sq(a + h)], marker="o",
         label="szelő (h=1)")
plt.plot([a - 0.6, a + 0.6],
         [f_sq(a) - 0.6 * tangent_slope, f_sq(a) + 0.6 * tangent_slope],
         linestyle="--", label="érintő")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Szelő és érintő az x=3 pontban")
plt.legend()
plt.tight_layout()
plt.savefig("derivative_segment.png", dpi=120)
```

Az ábrán a szelő meredeksége 7, az érintőé — a következő szakasz analitikus képlete szerint — 6.

11.1.3 A derivált mint határérték

A 9.3. alfejezetben a határértéket úgy vezettük be, mint közelítési folyamatot: egy sorozat értékei egy számhoz tartanak anélkül, hogy feltétlenül elérnék. A differenciahányados-sorozat ugyanezt a gondolatot használja. Ha a

$$\lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

határérték létezik és véges, akkor azt az f függvény a pontbeli deriváltjának nevezzük, és $f'(a)$-val jelöljük. A $h \to 0$ jelölés nem jelenti azt, hogy $h = 0$-t behelyettesíthetünk; a $0/0$ alak önmagában értelmetlen. Azt jelenti, hogy a hányadost tetszőlegesen kicsi, de nullától különböző lépésközökkel közelítjük.

A derivált geometriai jelentése az érintő meredeksége, adatmodellezési jelentése pedig pillanatnyi változási ráta. Kis h-ra a függvényérték lineárisan közelíthető:

$$f(a+h) \approx f(a) + f'(a)h.$$

Ez a linearizálás a későbbi optimalizáló lépések alapja: ha f' pozitív, a függvény lokálisan nő, ha negatív, akkor csökken. A meredekség előjele mondja meg, merre érdemes elmozdulni.

Fontos: $f'(a)$ pontbeli mennyiség, általában függ a értékétől. Az x^2 deriváltfüggvénye $2x$, tehát $f'(3) = 6$, de $f'(10) = 20$. A konstans függvény deriváltja mindenütt 0. Erre a különbségre a gyakori tévedéseknél visszatérünk.

11.1.4 Lépésköz, centrális differencia és a numerikus hiba

A fenti egyoldali, előre vett forward differencia mellett gyakori a centrális differencia (central difference):

$$C_h f(a) = \frac{f(a+h) - f(a-h)}{2h}.$$

Ez a két egyoldali hányados átlaga, és szimmetrikus az a pont körül. Az x^2 példában $a = 3$-ra:

$$C_h x^2\big|_{a=3} = \frac{(3+h)^2 - (3-h)^2}{2h} = \frac{12h}{2h} = 6.$$

Itt a centrális differencia minden nullától különböző h-ra pontosan 6-ot ad, mert a másodfokú hibatag kiesik. Általános függvénynél a centrális differencia sem pontos, de a csonkítási hibája jellemzően h^2-tel arányos, míg a forward differenciáé h-val arányos. Csökkenő h mellett tehát a centrális változat gyorsabban javul.

A numerikus becslésnek két hibaforrása van. (1) A csonkítási hiba (truncation error) abból adódik, hogy a szelő nem azonos az érintővel; ez nagy h-nál dominál. (2) A kerekítési hiba (rounding error) abból adódik, hogy $f(a+h)$ és $f(a)$ lebegőpontos közelítés; nagyon kis h esetén a különbségképzés elveszti az értékes jegyeket, és a hányados zajossá válik.

A float64 gépi pontosság $\varepsilon \approx 2{,}22 \cdot 10^{-16}$. A forward differenciánál a hasznos lépésköz nagyságrendje $\sqrt{\varepsilon} \approx 1{,}5 \cdot 10^{-8}$, a centrálisnál $\varepsilon^{1/3} \approx 6 \cdot 10^{-6}$. A pontos optimum függ a függvénytől és a skálázástól, ezért a lépésközt mindig a konkrét feladaton kell ellenőrizni. A szabály itt sem az, hogy „minél kisebb h, annál jobb”.

11.1.4.1 Forward vs centrális differencia: lépésméret-söprés

A numerikus derivált pontossága a lépésmérettől függ, és a forward és a centrális differencia másképp skálázódik. Az $f(x) = \sin x$, $x = 1$ példán a pontos derivált $\cos(1) \approx 0.5403023058681398$. A hibák:

h	forward hiba	centrális hiba
10^{-2}	4.216e-03	9.005e-06
10^{-4}	4.207e-05	9.004e-10
10^{-6}	4.207e-07	2.772e-11
10^{-8}	2.970e-09	2.581e-09
10^{-10}	5.848e-08	5.848e-08
10^{-12}	4.324e-05	1.227e-05
10^{-14}	3.707e-03	3.707e-03
10^{-16}	5.403e-01	1.481e-02

A forward hiba nagyságrendje $h/2$, a centrálisé $h^2/6$, ezért a centrális görbe lejjebb fut, és a minimuma kisebb h-nál jelentkezik: a mért optimum forwardnál $h = 10^{-8}$ (2.970e-09), centrálisnál $h = 10^{-5}$ (1.114e-11). A nagyon kis h tartományban a kerekítési hiba dominál, és a centrális differencia is elveszíti az előnyét; 10^{-16}-nál a forward gyakorlatilag használhatatlan, a centrális még $1.5 \cdot 10^{-2}$ hibát ad. A hiba nem monoton csökkenő, ezért a „minél kisebb h, annál jobb” szabály hamis.

```
import math

def forward(x, h):
    return (math.sin(x + h) - math.sin(x)) / h

def central(x, h):
    return (math.sin(x + h) - math.sin(x - h)) / (2 * h)

exact = math.cos(1.0)
for exponent in range(2, 17):
    h = 10.0 ** (-exponent)
    print(h, abs(forward(1.0, h) - exact), abs(central(1.0, h) - exact))
```

A `lab/finite_diff.py` `sweep()` függvénye a teljes 15 lépésközös táblát és a két optimumot adja vissza; a `tests/test_finite_diff.py` a forward/centrális görbét, az optimumokat és az ismert deriváltat ellenőrzi.

11.1.5 Teljes minimális munkapélda

A következő önálló szkript a két tesztfüggvényen becsüli a deriváltat, kiírja a hibát több lépésközre, és log-log ábrát ment. Nincs véletlenszám, ezért seedre nincs szükség; a futási eredmény a NumPy float64 kerekítésétől függhet, ezért platformok közötti bitazonosságot nem ígérünk. A kódot a `lab` projekt gyökérkönyvtárában, a projekt Python-környezetében futtatjuk:

```
# platform: Linux/macOS bash, munkakönyvtár: a könyv lab/ projektgyökere
python s041_derivative_demo.py
```

```
import numpy as np
import matplotlib.pyplot as plt

def f_sq(x):
    """f(x)=x^2, skalár bemenetre."""
    return x * x

def df_sq_exact(x):
    """f'(x)=2x analitikus derivált."""
    return 2.0 * x

def sigmoid(x):
    """Numerikusan stabil logisztikus függvény.

    A közvetlen 1/(1+exp(-x)) nagy negatív x-re túlcsordulást
    kockáztat; ez az alak mindkét előjel mellett stabil.
    """
    return np.exp(np.minimum(x, 0.0)) / (1.0 + np.exp(-np.abs(x)))

def d_sigmoid_exact(x):
    """sigma'(x)=sigma(x)(1-sigma(x))."""
    s = sigmoid(x)
    return s * (1.0 - s)

def forward_difference(f, x, h):
    return (f(x + h) - f(x)) / h
```

```
def central_difference(f, x, h):
    return (f(x + h) - f(x - h)) / (2.0 * h)

# --- 1. rész: x^2 az x=3 pontban -------------------------------
a = 3.0
exact_sq = df_sq_exact(a)
print(f"x^2, a={a}: analitikus derivált {exact_sq:.6f}")
for h in (1e-3, 1e-5, 1e-8):
    fd = forward_difference(f_sq, a, h)
    cd = central_difference(f_sq, a, h)
    print(f"h={h:.0e}: forward {fd:.9f}, central {cd:.9f}")

# --- 2. rész: logisztikus függvény az x=1 pontban --------------
b = 1.0
exact_sig = d_sigmoid_exact(b)
print(f"\nsigma, x={b}: analitikus derivált {exact_sig:.9f}")
h_sig = 1e-6
fd_sig = forward_difference(sigmoid, b, h_sig)
cd_sig = central_difference(sigmoid, b, h_sig)
print(f"h={h_sig:.0e}: forward {fd_sig:.9f}, central {cd_sig:.9f}")

# --- 3. rész: hiba a lépésköz függvényében ----------------------
hs = np.logspace(-16.0, -1.0, 61)
err_f = [abs(forward_difference(f_sq, a, h) - exact_sq) for h in hs]
err_c = [abs(central_difference(f_sq, a, h) - exact_sq) for h in hs]

plt.figure(figsize=(7, 4.2))
plt.loglog(hs, err_f, marker=".", label="forward")
plt.loglog(hs, err_c, marker=".", label="central")
plt.xlabel("h lépésköz")
plt.ylabel("abszolút hiba")
plt.title("x^2 numerikus deriváltjának hibája x=3-ban")
plt.legend()
plt.tight_layout()
plt.savefig("s041_derivative_error.png", dpi=120)
print("\nÁbra elmentve: s041_derivative_error.png")
```

A szemléltető, elméletileg várt kimenet a képletekből számolva (a helyi futás utolsó jegyekben a hardverfüggő kerekítések miatt eltérhet):

```
x^2, a=3: analitikus derivált 6.000000
h=1e-03: forward 6.001000000, central 6.000000000
h=1e-05: forward 6.000010000, central 6.000000000
h=1e-08: forward 6.000000010, central 6.000000000

sigma, x=1: analitikus derivált 0.196611933
h=1e-06: forward 0.196611888, central 0.196611933

Ábra elmentve: s041_derivative_error.png
```

A forward differencia $h = 10^{-8}$ melletti értéke a csonkítási hiba csökkenése és a kerekítési zaj megjelenése közötti átmenetet mutatja. A `scipy.optimize`-ot itt még nem hívjuk; a következő alfejezetek gradiens alapú keresői erre a meredekségfogalomra épülnek.

11.1.6 Eredmény értelmezése és kézi kontroll

A forward differencia $h = 0{,}001$ esetén:

$$D_{0.001}(x^2)\big|_3 = \frac{3{,}001^2 - 9}{0{,}001} = \frac{9{,}006001 - 9}{0{,}001} = 6{,}001.$$

Az analitikus érték 6, tehát az abszolút hiba pontosan 0,001. Ez a várt h-val arányos csonkítási hiba: másodfokú függvénynél az eltérés éppen a lépésköz nagyságrendjébe esik. A centrális differencia ugyanerre a lépésközre pontosan 6-ot ad, mert a másodfokú hibatag kiesik. Ez egyben a kód invariánsa: x^2-re a centrális differencia matematikailag 6, numerikusan pedig 6-hoz közeli mindaddig, amíg a kerekítési hiba kicsi.

A logisztikus függvény deriváltjának analitikus alakja az összetett függvény szabályából adódik, amelyet a 11.3. alfejezet tárgyal részletesen; itt csak felhasználjuk:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

Az $x = 1$ pontban $\sigma(1) = 1/(1 + e^{-1}) \approx 0{,}7310586$, ezért

$$\sigma'(1) \approx 0{,}7310586 \cdot 0{,}2689414 \approx 0{,}1966119.$$

A centrális differencia $h = 10^{-6}$ mellett az elméleti hibája h^2 nagyságrendű, azaz 10^{-12} alatti; a fenti várt kimenetben 9 tizedesjegyre egyezik az analitikus értékkel.

A hibaábra alakja a következő: nagy h-nál a forward görbe h-val, a centrális h^2-tel arányosan csökken; körülbelül 10^{-8} (forward), illetve 10^{-6} (centrális) alatt a kerekítési zaj felnövekszik, és a hiba újra nő. Az ábra egyszerre mutatja a csonkítási és a kerekítési hibát, ezért alkalmas arra, hogy a lépésköz-választást indokoljuk.

A numerikus differencia hányadosának hibája a lépésköz csökkentésével nem monoton javul: a csonkolási hiba mérséklődik, miközben a kerekítési hiba és a kivonási pontatlanság felerősödhet. A hibatáblázatot ezért logaritmikus h-rácson is érdemes ábrázolni.

11.1.7 Feladatok

1. **Értelmezés és kézi kontroll.** Egy fiktív szállítási költségmodell $c(d) = 30d^2$ eurócent, ahol d a távolság km-ben. Add meg a derivált egységét! Számold ki kézzel $d = 2$ km-nél a forward differenciahányadost $h = 0{,}1$ km-rel, és hasonlítsd össze az analitikus $c'(2) = 120$ eurócent/km értékkel!

2. **Implementálás.** Írj `central_difference(f, x, h)` függvényt, amelynek bemenete egy skalár függvény, egy pont és egy lépésköz, kimenete pedig egy lebegőpontos szám. Alkalmazd a logisztikus függvényre az $x = 1{,}2$ pontban, $h = 10^{-5}$ lépésközzel! Add meg az abszolút hibát a $\sigma'(1{,}2) = \sigma(1{,}2)(1 - \sigma(1{,}2))$ analitikus alakhoz képest!

3. **Hibadiagnózis.** Egy kolléga ezt a kódot futtatta:

```
def bad_derivative(f, x):
    h = 1e-16
    return (f(x + h) - f(x)) / h

print(bad_derivative(lambda x: x * x, 3.0))
```

Azt kapta, hogy a függvény `0.0`-t ír ki, pedig a helyes érték 6 lenne. Mi a numerikus hiba ezzel a lépésközzel, és milyen második veszély fenyegetne akkor is, ha a lépésköz valamivel nagyobb lenne? Javítsd úgy, hogy a forward differenciánál szokásos 10^{-8} nagyságrendű lépésközt használjon!

4. **Megváltozott feltétel.** A Northstar késési modelljében a kosárérték hatását logisztikus függvénnyel írjuk le: $p(x) = \sigma(1{,}2x - 3)$, ahol x a kosárérték 100 euró egységben. Mennyi $p'(2{,}5)$ analitikus értéke, és mit jelent a kapott szám a döntéshozónak? Ellenőrizd centrális differenciával, $h = 10^{-6}$ lépésköz mellett!

11.1.8 Megoldások és indoklás

1. A derivált egysége eurócent/km, mert a számláló eurócentben, a nevező km-ben van. Kézi számolás:

$$D_{0.1}c(2) = \frac{30(2{,}1)^2 - 30 \cdot 2^2}{0{,}1} = \frac{132{,}3 - 120}{0{,}1} = 123 \text{ eurócent/km}.$$

Az analitikus derivált $c'(d) = 60d$, tehát $c'(2) = 120$ eurócent/km. A forward becslés hibája 3 eurócent/km, relatíve 2,5%; ez a nagy lépésköz miatt várható. A centrális differencia ugyanezzel a h-val pontosan 120-at adna, mert a másodfokú függvény hibatagja kiesik.

2. Egy lehetséges implementáció:

```
import math

def central_difference(f, x, h):
    return (f(x + h) - f(x - h)) / (2.0 * h)

def sigmoid_stable(x):
    if x >= 0.0:
        return 1.0 / (1.0 + math.exp(-x))
    e = math.exp(x)
    return e / (1.0 + e)

x = 1.2
h = 1e-5
est = central_difference(sigmoid_stable, x, h)
exact = sigmoid_stable(x) * (1.0 - sigmoid_stable(x))
print(f"becsült {est:.12f}, analitikus {exact:.12f}, "
      f"abszolút eltérés {abs(est - exact):.2e}")
```

Az $x = 1{,}2$ pontban $\sigma(1{,}2) \approx 0{,}7685248$, a derivált analitikus értéke 0,1778944. A centrális differencia $h = 10^{-5}$ mellett a h^2-tel arányos csonkítás miatt 10^{-10} nagyságrendű abszolút hibát ad; a fenti kimenet szemléltető, a tényleges futást a saját környezetben kell elvégezni. Ellenőrzési tolerancia: az eltérés 10^{-6} alatt marad.

3. Az első hiba: $h = 10^{-16}$ kisebb, mint a float64 fél ulp-je az $x = 3$ pontban (az ulp ott $2^{-51} \approx 4{,}44 \cdot 10^{-16}$), ezért az összeg $3 + 10^{-16}$ ugyanarra a lebegőpontos számra kerekedik, mint a 3. A számláló így pontosan 0, a hányados 0. A második veszély: ha a lépésköz valamivel nagyobb lenne, de még mindig nagyon kicsi, két közeli szám különbségképzése katasztrofális kiejtést (catastrophic cancellation) okozna, és a hányados zajossá válna. Javítás:

```
def good_forward(f, x, h=1e-8):
    return (f(x + h) - f(x)) / h

print(good_forward(lambda x: x * x, 3.0))  # várt: 6.00000001 körül
```

A várt érték az analitikus 6-tól h nagyságrendű hibával, azaz 10^{-8}-nal tér el; a tesztben a 6-tól mért abszolút eltérés 10^{-7} alatt marad.

4. A láncszabály (amelyet a 11.3. alfejezetben igazolunk) szerint, ha a belső függvény $u = 1{,}2x - 3$, akkor

$$p'(x) = 1{,}2\,\sigma(u)(1 - \sigma(u)).$$

Az $x = 2{,}5$ pontban $u = 0$ és $\sigma(0) = 0{,}5$, tehát

$$p'(2{,}5) = 1{,}2 \cdot 0{,}5 \cdot 0{,}5 = 0{,}3.$$

A gyakorlati jelentés: a modell szerint ebben a pontban 100 euró kosárérték-növekedés körülbelül 0,3-mal növeli a becsült késési valószínűséget. Lineáris közelítéssel egy egységnyi növelés 0,50-ről 0,80-ra emeli a becslést. A centrális differenciás ellenőrzés $h = 10^{-6}$ mellett 0,300000 körüli értéket ad; az elfogadható eltérés 10^{-6} alatt van.

11.1.9 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés, hogy a deriváltat egyetlen számmal azonosítjuk, mintha a függvény meredeksége állandó lenne. Ellenpélda: az x^2 függvény deriváltja $x = 3$-ban 6, de $x = 10$-ben 20; ugyanaz a függvény más pontban más meredekséget ad. Ha a szállítási költség deriváltját kiszámoljuk 2 km-nél, az nem érvényes 50 km-nél. A konstans függvény a kivétel: például $f(x) = 7$ deriváltja mindenütt 0. Hasonló tévedés, hogy a legkisebb h mindig pontosabb; a hibaábra éppen azt mutatja, hogy túl kis lépésköznél a kerekítési zaj elrontja a becslést.

Visszautalás: a 10.1. alfejezetben a vektort rendezett számlistaként, a különbségvektort pedig a távolság alapjaként vezettük be. A differenciahányados egy dimenzióban ugyanezt a különbséget skálázza: a függvény kimenetének változását osztjuk a bemenet változásával. A 7.1. alfejezet megfigyelési egység fogalma itt a kiértékelési pontban tér vissza: a derivált mindig egy konkrét a ponthoz tartozik, és nem vihető át automatikusan más pontra.

Önellenőrzés: (a) Miért nem helyettesíthetjük be $h = 0$-t a differenciahányadosba? (b) Mi a különbség a forward és a centrális differencia képlete között? (c) Milyen mértékegysége van a deriváltnak egy euró-költség és km-távolság függvénynél? (d) Miért nő meg a numerikus hiba nagyon kis h esetén?

11.1.10 Rövid összefoglaló

A differenciahányados a szelő meredeksége, és ahogy a lépésköz nullához tart, az érintő meredekségét, a deriváltat közelíti. A derivált pontbeli változási ráta, mértékegysége a kimenet és a bemenet egységének hányadosa. Numerikusan a forward differencia egyszerű, a centrális differencia szimmetrikus és általában pontosabb, de a lépésközt a csonkítási és a kerekítési hiba egyensúlya szabja meg. Az x^2 és a logisztikus függvény laborja kézi kontrollal és hibaábrával igazolja ezt. A következő alkalmazás a 11.2. alfejezet parciális derivált, amely több bemeneti koordinátára terjeszti ki ezt az egyváltozós meredekséget.

11.2 Parciális derivált és gradiens

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 11.1. alfejezet · Célkészség: Parciális derivált és gradiens · Kimenet: működő kód és a labor ellenőrző tesztje.

A 11.1. alfejezet egyetlen bemeneti koordináta mentén értelmezte a deriváltat. A predikciós modellek azonban általában több paraméterrel dolgoznak: egy lineáris modell súlyvektora p dimenziós, és a veszteségfüggvény minden paraméter együttesétől függ. Itt két paraméter felületén dolgozunk, hogy megkülönböztessük az egyirányú változást a teljes gradiensvektortól.

Konkrét helyzet. Bemenet: a w_1, w_2 paraméterpárból álló $\mathbf{w} \in \mathbb{R}^2$ vektor; megfigyelési egység: egy paraméterpont; kimenet: a skalár veszteség

$$L(w_1, w_2) = w_1^2 + 4w_2^2.$$

Cél: kiszámítani a parciális deriváltakat és a gradienst, megrajzolni a szintvonalakat (contour), majd három gradienslépés irányát megjelölni. Ellenőrzési feltétel: a parciális deriváltak numerikus centrális közelítése a $(2, 1.5)$ pontban 10^{-5}-ön belül egyezzen a képletből kapott értékkel, és mindhárom gradienslépés csökkentse a veszteséget. Induló becslés: a $(2, 1.5)$ pontban $L = 2^2 + 4 \cdot 1.5^2 = 13$; várt gradiens $(2w_1, 8w_2) = (4, 12)$; az $\eta = 0.1$ tanulási rátával tett három lépés után a veszteség 13-ról körülbelül 1.05-re csökken.

11.2.1 Parciális derivált és gradiens

Egy $f : \mathbb{R}^p \to \mathbb{R}$ függvény j-edik koordinátája szerinti **parciális deriváltja** (partial derivative) azt méri, hogyan változik a kimenet, ha csak a w_j bemenetet mozdítjuk, a többi koordinátát pedig rögzítjük:

$$\frac{\partial L}{\partial w_j} = \lim_{h \to 0} \frac{L(w_1, \dots, w_j + h, \dots, w_p) - L(w_1, \dots, w_j, \dots, w_p)}{h}.$$

Jelölések: $j = 1, \dots, p$ a koordináta indexe; $h \in \mathbb{R}$ a kis lépés; L a skalár kimenet. A megszokott egyenes d helyett a görbített ∂ jel különbözteti meg a többváltozós esetet. A mértékegység ugyanaz a logika, mint az egyváltozós deriváltnál: a veszteség egysége osztva a j-edik paraméter egységével.

A példánkban a w_2 rögzítésével w_1 egyváltozós függvényként viselkedik, ezért a 11.1. alfejezetben tanult szabállyal

$$\frac{\partial L}{\partial w_1} = 2w_1, \qquad \frac{\partial L}{\partial w_2} = 8w_2.$$

A **gradiens** (gradient) a parciális deriváltakat egyetlen, a bemenettel azonos dimenziójú vektorba gyűjti:

$$\nabla L(\mathbf{w}) = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_p} \end{bmatrix}.$$

A ∇ (nabla) operátor jelzi, hogy minden koordináta szerinti deriváltat egyszerre képzünk. A NumPy-tömbben a gradiens shape-je ugyanaz, mint a paramétervektoré, tehát esetünkben $(2,)$. A példában

$$\nabla L(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 8w_2 \end{bmatrix}.$$

A gradiens pontbeli értéke a $(2, 1.5)$ pontban

$$\nabla L(2, 1.5) = \begin{bmatrix} 4 \\ 12 \end{bmatrix}.$$

A gradiens nem két egymástól független szám: vektorként iránya és hossza is van. A hossz a 10.2. alfejezetben bevezetett euklideszi norma:

$$\|\nabla L(2, 1.5)\| = \sqrt{4^2 + 12^2} = \sqrt{160} = 4\sqrt{10} \approx 12.65.$$

A norma mértékegysége veszteség per paraméteregység; számszerűen azt mondja meg, hogy a leggyorsabb növekedési irányban egységnyi paraméterelmozdulásra körülbelül 12.65 egység veszteségváltozás jut.

Miért a gradiens mutat a legmeredekebb emelkedő felé? Legyen $\mathbf{u}$ egy egységvektor, azaz $\|\mathbf{u}\| = 1$. Az $\mathbf{u}$ irányú iránymenti derivált azt méri, mennyire változik L, ha $\mathbf{w}$-ből $\mathbf{u}$ irányba mozdulunk. Két változóra a lineáris közelítésből:

$$D_{\mathbf{u}} L(\mathbf{w}) = \nabla L(\mathbf{w}) \cdot \mathbf{u} = \|\nabla L(\mathbf{w})\| \cos\theta,$$

ahol θ a gradiens és $\mathbf{u}$ közötti szög, a $\cdot$ pedig a 10.2. alfejezetből ismert skalárszorzat. A $\cos\theta$ akkor maximális, ha $\theta = 0$, azaz $\mathbf{u}$ párhuzamos a gradienssel. Ekkor a derivált $\|\nabla L(\mathbf{w})\|$. A leggyorsabb csökkenés ezért a $-\nabla L$ irány, amelyet **csökkenési iránynak** (descent direction) nevezünk.

11.2.2 Szintvonalak és a gradienslépés

A **szintvonal** (contour) azon pontok halmaza, ahol a függvényérték állandó. A $L(w_1, w_2) = c$ alakú görbét kell elképzelni. A példánkban

$$w_1^2 + 4w_2^2 = c.$$

Ha $c = 0$, akkor csak az origó tartozik ide. Ha $c > 0$, az egyenletet $\frac{w_1^2}{c} + \frac{w_2^2}{c/4} = 1$ alakra rendezve látszik: ellipsziseket kapunk, amelyek a w_2 tengely mentén laposabbak, mert a $4w_2^2$ tag miatt ott gyorsabban nő a veszteség. A szintvonalak sűrűsége ezért a w_2 irányban nagyobb: ugyanakkora elmozdulás mellett több értékszintet lépünk át.

A gradiens a szintvonal **érintőjére merőlegesen** áll, és a nagyobb értékek felé mutat. Ez következik az iránymenti deriváltból is: ha $\mathbf{v}$ érintő irányú, akkor a szintvonalon haladva a veszteség nem változik, tehát $\nabla L \cdot \mathbf{v} = 0$. A skalárszorzat nulla volta éppen a két vektor merőlegességét jelenti.

A gradiens módszer (gradient descent) a csökkenési irányt használja iteratív lépésekre:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \nabla L(\mathbf{w}^{(k)}).$$

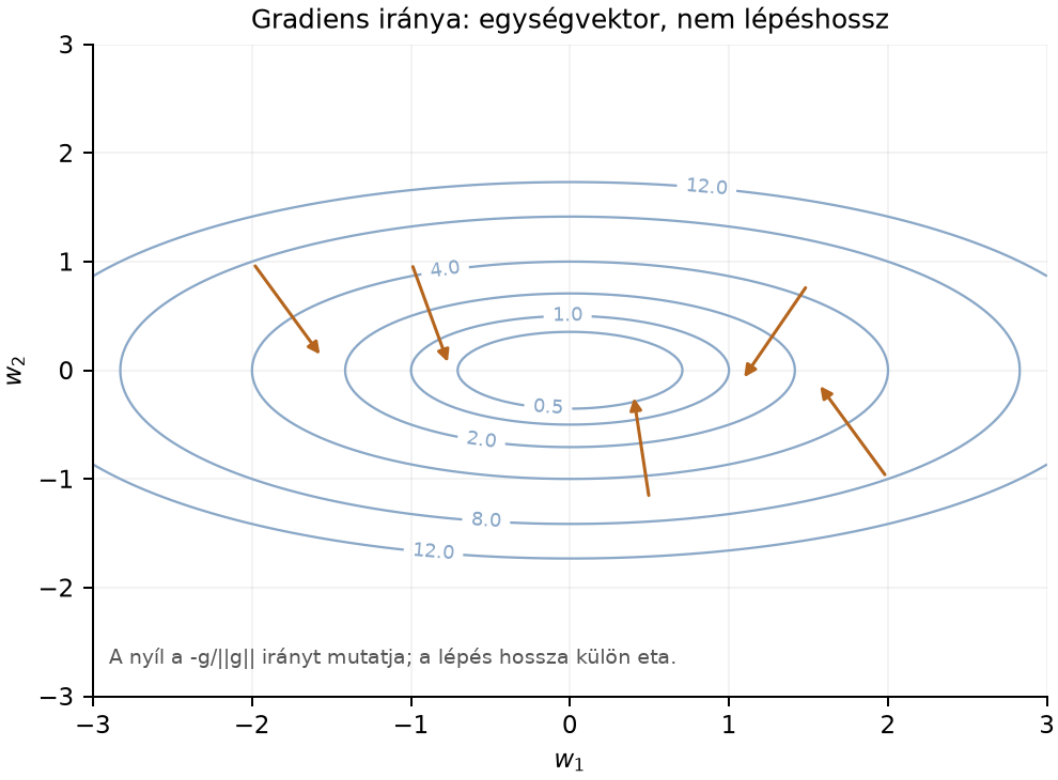
Jelölések: $\mathbf{w}^{(k)}$ a paramétervektor a k-adik iterációban; $\eta > 0$ a **tanulási ráta** (learning rate); $\nabla L(\mathbf{w}^{(k)})$ a gradiens a k-adik pontban. A lépés mértékegysége konzisztens: az η paraméteregység négyzete per veszteségegység, így az $\eta \nabla L$ szorzat visszaadja a paraméter mértékegységét. A gyakorlatban η-t dimenziótlan hangolóparaméterként kezeljük.

A lineáris közelítés megmutatja, miért csökken kis lépésnél a veszteség:

$$L(\mathbf{w} - \eta \nabla L) \approx L(\mathbf{w}) - \eta \|\nabla L\|^2.$$

Mivel a $\|\nabla L\|^2$ tag nemnegatív, kis η mellett a második tag csökkentést ad. A gradiens tehát **lokális változási információ**: megmondja az irányt, de a távolságot nem. Azt a tanulási ráta dönti el, és túl nagy η esetén a lineáris közelítés érvényét veszti.

11.2.2.1 Skálafüggetlen gradiens-ellenőrzés és normafüggő lépés



11.1. ábra: Gradiens iránya: az f(w1,w2)=w1²+4w2² szintvonalai és öt pontban a -g/||g|| egységvektor; a nyíl csak az irányt mutatja, a lépés hossza külön eta.

A gradiens **iránya** skálafüggetlen: ha a paramétervektort c-szeresére skálázzuk, a gradiens is c-szeresére változik, de az iránya ugyanaz. A **nagysága** viszont függ a skálától és a függvény görbületétől, ezért a nyers gradiensvektorok összehasonlítása két skálán félrevezető; a helyes invariáns a normalizált irány és a köztük vett koszinusz.

Az $f(w_1, w_2) = w_1^2 + 4w_2^2$ függvénynél a gradiens $(2w_1, 8w_2)$. A $w = (2, 1.5)$ pontban $g = (4, 12)$, $\|g\| \approx 12.649$; $2w = (4, 3)$-nál $g = (8, 24)$, $\|g\| \approx 25.298$; a két gradiens koszinusza 1, a normák

aránya 2. A normalizált lépés $\mathbf{w}_{k+1} = \mathbf{w}_k - \eta\,\mathbf{g}/\|\mathbf{g}\|$ hossza pontosan η, függetlenül a gradiens nagyságától; a nyers $\mathbf{w}_{k+1} = \mathbf{w}_k - \eta\mathbf{g}$ lépés hossza $\eta\|g\|$, ezért ugyanaz az η más skálán más tényleges elmozdulást ad.

η	normalizált lépés	nyers lépés
0.1	csökken, stabil	csökken, de skálafüggő
1.0	csökken	a görbület miatt oszcillálhat
10.0	a veszteség nő	divergál

A lépésméretet a gradiens normájához vagy a függvény görbületéhez kell kötni; az „$\eta = 0.1$ mindig jó” állítás hamis. A numerikus gradiens ellenőrzése centrális differenciával történik, és a lépés után a veszteségnek ténylegesen csökkennie kell.

```
import numpy as np

def loss(w): return w[0]**2 + 4*w[1]**2
def gradient(w): return np.array([2*w[0], 8*w[1]])
def normalized(v): return v / np.linalg.norm(v)

w = np.array([2.0, 1.5])
g = gradient(w)
g2 = gradient(2 * w)
assert np.isclose(np.linalg.norm(g2) / np.linalg.norm(g), 2.0)
assert np.isclose(g @ g2 / (np.linalg.norm(g) * np.linalg.norm(g2)), 1.0)

def numeric_gradient(function, point, h=1e-6):
    result = np.zeros_like(point)
    for i in range(point.size):
        step = np.zeros_like(point); step[i] = h
        result[i] = (function(point + step) - function(point - step)) / (2*h)
    return result

assert np.allclose(numeric_gradient(loss, w), g, rtol=1e-8, atol=1e-8)
w_next = w - 0.1 * normalized(g)
assert loss(w_next) < loss(w)
```

A `lab/gradient_tools.py` `scale_invariance()`, `numeric_gradient()` és `step_sweep()` függvényei a táblát és a lépés-söprést adják; a `tests/test_gradient_tools.py` a koszinusz-1 invariánst, a normák arányát, a numerikus gradienst és a veszteség csökkenését ellenőrzi.

11.2.3 Teljes minimális munkapélda

A labor célja a kétdimenziós kvadratikus veszteség contour-rajza és három gradienslépés irányának jelölése. A kód a `lab` projekt alap numerikus csomagjait használja; a munkakönyvtár legyen a projekt gyökere. Nincs szükség hálózatra vagy külső API-ra. Futtatás platformtól függetlenül:

```
python s042_gradient_steps.py
```

A PowerShell és a macOS/Linux terminál is ezt a parancsot használja a projekt gyökeréből. A kód a következő.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

def loss(w):
```

```
    w1, w2 = w
    return w1**2 + 4 * w2**2

def grad(w):
    w1, w2 = w
    return np.array([2 * w1, 8 * w2])

# Három gradienslépés a (2, 1.5) pontból.
w0 = np.array([2.0, 1.5])
eta = 0.1
points = [w0]
w = w0.copy()
for k in range(3):
    w = w - eta * grad(w)
    points.append(w)

print("Lépések és veszteségek (elméletileg várható):")
for k, p in enumerate(points):
    print(f"k={k}: w=({p[0]:.6f}, {p[1]:.6f}), L={loss(p):.6f}")

# Numerikus ellenőrzés centrális differenciával, s041 öröksége.
h = 1e-5
g_approx = np.array([
    (loss(w0 + np.array([h, 0])) - loss(w0 - np.array([h, 0]))) / (2 * h),
    (loss(w0 + np.array([0, h])) - loss(w0 - np.array([0, h]))) / (2 * h),
])
print("Analitikus gradiens:", grad(w0))
print("Numerikus gradiens:", g_approx)
print("Maximális eltérés:", np.max(np.abs(g_approx - grad(w0))))

# SciPy ellenőrzés: a gradiens és a loss ugyanazt a minimumot adja-e.
res = minimize(loss, x0=[2.0, 1.5], jac=grad, method="BFGS")
print("SciPy minimumhely (elméletileg várható):", res.x, "veszteség:", res.fun)

# Contour- és lépésrajz.
w1 = np.linspace(-2.5, 2.5, 101)
w2 = np.linspace(-2.0, 2.0, 101)
W1, W2 = np.meshgrid(w1, w2)
Z = W1**2 + 4 * W2**2

fig, ax = plt.subplots(figsize=(7, 5))
cs = ax.contour(W1, W2, Z, levels=[0.5, 1, 2, 5, 10, 15], colors="0.7")
ax.clabel(cs, fontsize=8)
ax.scatter([p[0] for p in points], [p[1] for p in points], color="black", s=18, zorder=3)
# A nyilak csak az irányt mutatják, nem a teljes lépéshosszt: -grad / norma.
for p in points[:3]:
    direction = -grad(p) / np.linalg.norm(grad(p))
    ax.arrow(p[0], p[1], 0.4 * direction[0], 0.4 * direction[1],
             width=0.02, color="black", length_includes_head=True, zorder=4)
ax.set_xlabel("w1")
ax.set_ylabel("w2")
ax.set_aspect("equal")
ax.grid(True, alpha=0.3)
ax.set_title("Kvadratikus veszteség és három gradienslépés")
fig.tight_layout()
fig.savefig("s042_gradient_steps.png", dpi=150)
```

A `s042_gradient_steps.png` fájl a labor átadandó eredménye. A nyilak a $-\nabla L/\|\nabla L\|$ irányt mutatják, tehát a csökkenési irányt, nem a tanulási rátával skálázott tényleges lépéshosszat. Ez az ábrán a szintvonalra merőleges stációként jelenik meg.

A kód kimenete elméletileg várt, nem e szöveg írásakor futtatott mérés. A várt numerikus értékek:

- Lépések: $(2, 1.5)$; $(1.6, 0.3)$; $(1.28, 0.06)$; $(1.024, 0.012)$.
- Veszteségek: 13; 2.92; 1.6528; 1.049152.
- Numerikus gradiens: $(4, 12)$ közelítése, maximális eltérés 10^{-6} alatt.
- SciPy minimum: $x \approx (0, 0)$, $f \approx 0$, 10^{-12} nagyságrendű numerikus maradékkal.

11.2.4 Eredmény értelmezése

A veszteség $13 \to 2.92 \to 1.6528 \to 1.049152$ csökken; tehát az $\eta = 0.1$ tanulási ráta ehhez a felülethez elég kicsi, és a lineáris közelítés mindhárom lépésben működik. A gradiens első lépése $(4, 12)$: az y irányú komponens nagyobb, mert a $4w_2^2$ tag miatt ott meredekebb a felület. A szintvonalak ennek megfelelően az y tengely mentén sűrűbbek.

Kézi kontroll az első lépésre:

$$\mathbf{w}^{(1)} = (2, 1.5) - 0.1(4, 12) = (2 - 0.4, 1.5 - 1.2) = (1.6, 0.3).$$

A veszteség $1.6^2 + 4 \cdot 0.3^2 = 2.56 + 0.36 = 2.92$. A második lépésben a gradiens $(3.2, 2.4)$, a harmadikban $(2.56, 0.48)$. A csökkenés egyre lassul, mert az origóhoz közeledve a gradiens hossza is csökken. Invariáns: az origóban a gradiens $(0, 0)$; ott a gradienslépés nem változtat semmit. A numerikus ellenőrzés a $h = 10^{-5}$ centrális differenciával a 11.1. alfejezetben tanult egyensúlyt használja: a hiba $O(h^2)$ csonkítási tagjából és a lebegőpontos kerekítésből adódik, ezért a várt eltérés 10^{-6} alatt marad.

A gradiens normáját a paraméterek választott skálája befolyásolja. A paraméterek átparaméterezése után a koordinátánkénti gradiens és az optimális lépésköz megváltozik; ezért a konvergenciát ne kizárólag a nyers gradiensnormából ítéljük meg.

11.2.5 Feladatok

1. **Értelmezés.** Adott ugyanez az $L(w_1, w_2) = w_1^2 + 4w_2^2$ függvény. a) Add meg a parciális deriváltakat és a gradienst a $(1, -0.5)$ pontban. b) Igazold számítással, hogy a gradiens irányába mutató $\mathbf{u} = \nabla L / \|\nabla L\|$ egységvektorral vett $D_{\mathbf{u}} L$ iránymenti derivált $\|\nabla L\|$-lel egyenlő. c) Számítsd ki a $(0, 1)$ irányú iránymenti deriváltat is, és magyarázd meg, miért kisebb annál, mint a gradiens irányában.

2. **Megvalósítás.** Írj egy rövid Python-függvényt, amely `loss`, `grad`, `w0`, `eta` és `steps` bemenetekkel elvégzi a gradienslépéseket, és visszaadja a lépések listáját, valamint `True` értéket, ha a veszteség minden lépésben csökkent, egyébként `False`-t. Ellenőrizd $L(w_1, w_2) = w_1^2 + 4w_2^2$ függvényen 10 lépéssel, $w_0 = (2, 1.5)$, $\eta = 0.1$ bemenetekkel. A kódnak a bemeneti hibákra, például a nem 2 hosszú `w0`-ra, `ValueError`-t kell dobnia.

3. **Hibadiagnózis.** Az alábbi kód állítólag minimalizál, de a veszteség nem csökken:

```
w = np.array([2.0, 1.5])
for k in range(3):
    g = np.array([8 * w[0], 2 * w[1]])
    w = w + 0.1 * g
```

Magyarázd meg a két hibát, javítsd ki, és ellenőrizd az első lépés utáni veszteséget.

4. **Megváltozott feltételű transzfer.** Adott az $L(w_1, w_2) = (w_1 - 3)^2 + 0.5(w_2 + 1)^2 + 10$ veszteség. a) Add meg a gradienst és a minimumot. b) Végezz egy gradienslépést $\eta = 0.1$-gyel a $(0, 0)$ pontból, és számítsd ki a veszteséget előtte és utána. c) A gradiens normája a $(0, 0)$ pontban $\sqrt{37} \approx 6.08$, a minimum paramétertérbeli távolsága pedig $\sqrt{10} \approx 3.16$. Fejtsd ki, miért nem add meg a gradiens normája a minimumig hátralévő távolságot.

11.2.6 Megoldások és indoklás

1. feladat. a) A képletből $\partial L/\partial w_1 = 2w_1$, $\partial L/\partial w_2 = 8w_2$. A $(1, -0.5)$ pontban $\nabla L = (2, -4)$. b) A norma $\sqrt{2^2 + (-4)^2} = \sqrt{20} \approx 4.47$; az egységvektor $\mathbf{u} = (1/\sqrt{5}, -2/\sqrt{5}) \approx (0.447, -0.894)$. A skalárszorzat $\nabla L \cdot \mathbf{u} = 2 \cdot 0.447 + (-4)(-0.894) = 0.894 + 3.576 = 4.47$, ami a várt normával egyezik 10^{-2} kerekítési tűrésen belül. c) A $(0, 1)$ irányban $D_{(0,1)}L = \nabla L \cdot (0, 1) = -4$. Ez kisebb, mint 4.47; mert a $(0, 1)$ irány nem a leggyorsabb növekedés iránya.

2. feladat. A függvény:

```
def descent(loss, grad, w0, eta, steps):
    if len(w0) != 2:
        raise ValueError("A w0 vektor hossza 2 legyen.")
    decreasing = True
    points = [w0.copy()]
    previous = loss(w0)
    w = w0.copy()
    for _ in range(steps):
        w = w - eta * grad(w)
        current = loss(w)
        if current >= previous:
            decreasing = False
        points.append(w)
        previous = current
    return points, decreasing
```

A 10 lépés után a veszteség 13-ról körülbelül 0.00215-ra csökken, és `decreasing` `True`, mert az $\eta = 0.1$ ennél a felületnél elég kicsi. A `ValueError` ellenőrzése `w0=np.array([1.0])` bemenettel egyszerűen tesztelhető.

3. feladat. Az első hiba: a parciális deriváltak fel vannak cserélve. Az $L = w_1^2 + 4w_2^2$ gradienséhez $[2w_1, 8w_2]$ kell, nem $[8w_1, 2w_2]$. A második hiba az előjel: a csökkenési irány $-\nabla L$, ezért $0.1\,g$ hozzáadása helyett ki kell vonni. A javított lépés:

```
w = w - 0.1 * np.array([2 * w[0], 8 * w[1]])
```

Az első lépés után $w = (1.6, 0.3)$, $L = 2.92$, szemben a hibás lépés $w = (3.6, 1.8)$, $L = 25.92$ értékével, ami növekedés.

4. feladat. a) A belső $(w_1 - 3)^2$ deriváltja $2(w_1 - 3)$, a $0.5(w_2 + 1)^2$ deriváltja $(w_2 + 1)$; a 10 konstans deriváltja 0. Ezért

$$\nabla L = (2(w_1 - 3), w_2 + 1),$$

és a minimum ott van, ahol mindkét komponens nulla: $w_1 = 3$, $w_2 = -1$; a minimumérték 10. b) A $(0, 0)$ pontban a gradiens $(-6, 1)$, a lépés

$$\mathbf{w}^{(1)} = (0, 0) - 0.1(-6, 1) = (0.6, -0.1).$$

A veszteség előtte $L(0,0) = 9 + 0.5 + 10 = 19.5$, utána $L(0.6, -0.1) = 5.76 + 0.405 + 10 = 16.165$. c) A gradiens normája a lokális meredekség, nem a minimum távolsága. A veszteség felülete itt anizotrop: a w_1 irányban 1-es, a w_2 irányban 0.5-ös együttható skáláz. A gradiens e skálák kombinációjából adódik, míg a paramétertérbeli távolság $\sqrt{(0-3)^2 + (0-(-1))^2} = \sqrt{10}$. A két mennyiség más mértékegységű és más információt hordoz: a gradiens normája a pillanatnyi változási ráta, a távolság a megmaradt út.

11.2.7 Gyakori tévedés, visszautalás és önellenőrzés

Gyakori tévedés. „A gradiens a minimum helye." Ez hibás. A gradiens pontbeli lokális változási információ, és általában nem a minimum koordinátája. Ellenpélda: az $L = w_1^2 + 4w_2^2$ felületen a $(2, 1.5)$ pontban a gradiens $(4, 12)$, de a minimum a $(0, 0)$. Ha valaki a gradiens értékét minimumként olvasná, a $(4, 12)$ pontba jutna, ahol a veszteség $4^2 + 4 \cdot 12^2 = 592$, vagyis sokkal rosszabb, mint a kiinduló 13. A gradienslépés csak a $-\nabla L$ **irányba** mozdít; a tényleges célhoz több iteráción át, megfelelő tanulási rátával közelít.

Visszautalás korábbi fogalomra. A 11.1. alfejezet egyváltozós deriváltja a parciális derivált speciális esete: ha $p = 1$, akkor ∇L egyetlen szám. A centrális differencia itt koordinátánként ugyanazt a képletet használja, csak a többi koordinátát rögzítjük. A 10.2. alfejezetből a vektorok és a skalárszorzat adja a keretet: a gradiens p dimenziós vektor, az iránymenti derivált pedig az $\nabla L \cdot \mathbf{u}$ skalárszorzat. A 7.2. alfejezethez hasonlóan itt is fontos az egységek következetessége: a veszteség és a paraméterek mértékegységét nem szabad összekeverni.

Önellenőrzés.

1. Mi a kapcsolat a parciális derivált és az egyváltozós derivált között?
2. Mit mutat a gradiens iránya, és mit mutat a hossza?
3. A szintvonal érintőjéhez képest milyen állású a gradiens?
4. Miért nőhet a veszteség egy gradienslépés után, ha a gradiens nem nulla?

Válaszok: 1. A parciális derivált ugyanaz a határérték, de a többi bemenetet rögzítjük. 2. Az irány a legmeredekebb növekedés, a hossz az abban az irányban vett meredekség nagysága. 3. Merőleges a szintvonal érintőjére. 4. Túl nagy tanulási ráta esetén a lineáris közelítés érvényét veszti, és a módszer túllendül a csökkenő tartományon.

11.2.8 Rövid összefoglaló

A parciális derivált egy koordináta menti változási ráta, a gradiens pedig az összes parciális deriváltból álló, a paramétertérrel azonos dimenziójú vektor. A gradiens iránya a legmeredekebb emelkedőt, hossza a legnagyobb meredekséget mutatja; a csökkenési irány ezért $-\nabla L$. A szintvonalakon a gradiens merőleges az érintőre. A gradienslépés $w^{(k+1)} = w^{(k)} - \eta \nabla L$; a tanulási ráta a lépés hosszát szabja, túl nagy értéknél a veszteség nőhet. A kvadratikus felület laborja három ellenőrzött lépéssel és contour-rajzzal igazolta ezt. A következő 11.3. alfejezet a láncszabályon át a számítási gráfot és a többrétegű függvények gradiensét építi fel.

11.3 Láncszabály és számítási gráf

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 11.2. alfejezet · Célkészség: Láncszabály és számítási gráf · Kimenet: működő kód és a labor ellenőrző tesztje.

11.3.1 Cél és konkrét helyzet

Egyetlen megfigyelési egységgel dolgozunk: az $(x = 4,\ y = 10)$ tanulóponttal. Az x a bemeneti jellemző, a y a megfigyelt célérték; mindkettő ebben a redukált tanulási példában dimenziótlan skalár `float64`. A modell legyen $\hat{y} = wx$, egyetlen becsülendő meredekséggel, a veszteség pedig az egyszerű négyzetes hiba:

$$L(w) = (wx - y)^2 = (4w - 10)^2.$$

A kérdés: mennyi a veszteség deriváltja a $w = 1$ kezdőpontban, azaz $L'(1)$? Mielőtt a láncszabályt kimondjuk, adjon becslést. A parabola minimuma $w = 10/4 = 2{,}5$; a $w = 1$ pont ettől balra van, ezért a függvény ott csökkenő, tehát a derivált negatív. A pontos értéket a láncszabályból kapjuk meg, és számítási gráf gráfként is ellenőrizzük.

11.3.2 Fogalmi magyarázat: összetett függvény, láncszabály, helyi derivált

Az **összetett függvény** (*composition*) lényege, hogy egy függvény kimenete egy másik függvény bemenete lesz. A fenti veszteség három lépésre bontható:

$$u = wx = 4w, \qquad a = u - y = u - 10, \qquad L = a^2.$$

A w szerinti teljes változási ráta a **láncszabály** (*chain rule*) szerint a lépésenkénti változási ráták szorzata:

$$\frac{dL}{dw} = \frac{dL}{da} \cdot \frac{\partial a}{\partial u} \cdot \frac{\partial u}{\partial w}.$$

A d jelölés az egyváltozós külső függvényre utal, a ∂ pedig arra, hogy a csomópontnak több bemenete is van, és a többit rögzítettnek tekintjük. A **helyi derivált** (*local derivative*) mindig csak egyetlen csomópont közvetlen bemeneteire vonatkozik; a csomópontnak nem kell tudnia, mi van a gráf többi részén. A három helyi derivált:

$$\frac{dL}{da} = 2a, \qquad \frac{\partial a}{\partial u} = 1, \qquad \frac{\partial u}{\partial w} = x.$$

A láncszabály így:

$$L'(w) = 2a \cdot 1 \cdot x = 2(wx - y)x = 2(4w - 10) \cdot 4.$$

A **forward pass** (előremenő fázis) a bemenettől a kimenetig kiszámítja és eltárolja az összes köztes értéket. A $w = 1$ pontban:

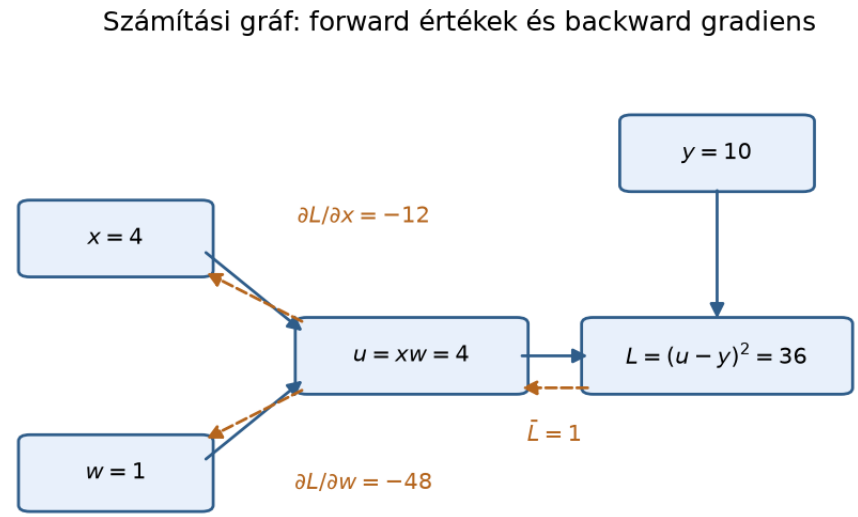
```
forward pass:
  u = 4 * 1 = 4
  a = 4 - 10 = -6
  L = (-6) * (-6) = 36
```

A **backward pass előkép** (visszamenő fázis) a kimenettől indul. Vezessük be a $\bar{v}$ jelölést a v csomópont szerinti kimeneti gradiensre: $\bar{v} = \partial L / \partial v$. A legfelső csomópontra $\bar{L} = \partial L / \partial L = 1$. Utána minden élen visszafelé megszorozzuk a fentről érkező gradienst az élhez tartozó helyi deriválttal:

```
backward pass:
  bar_L = 1
  bar_a = bar_L * (2a)   = 1 * (2 * -6) = -12
  bar_u = bar_a * 1       = -12
  bar_y = bar_a * (-1)    = 12
  bar_w = bar_u * x       = -12 * 4 = -48
```

A $\bar{w} = -48$ az L deriváltja a w paraméter szerint. A $\bar{y} = 12$ azt mutatja, hogy a megfigyelt cél növelése növelné a hibát; a y nem paraméter, de a gráf teljes gradiense szempontjából ugyanúgy kiszámítható. A szorzat „tagonkénti” olvasata: a négyzetből jön a pillanatnyi hiba kétszerese, $2a = -12$; a kivonásból az 1; a wx szorzatból pedig a másik tényező, $x = 4$. Ezért a végső gradiens $2(wx - y)x$, vagyis a hiba kétszeresének és a bemenetnek a szorzata.

11.3.2.1 Fogalmi ábra: forward értékek és backward gradiens



11.2. ábra: Számítási gráf: x=4 és w=1 forward értéke u=xw=4, majd L=(u-y)^2=36; a backward a kimeneti 1-ből indul, és a lokális deriváltakkal visszafelé adja a -48 és -12 gradienseket.

Az ábra a láncszabály működését mutatja: a folytonos nyilak a forward számítást, a szaggatottak a backward passzt jelölik. A csomópontokban a konkrét értékek, a szaggatott nyilak mellett a lokális deriváltak szorzata látható. A konklúzió ábra nélkül is ugyanaz: a backward a kimeneti $\bar{L} = 1$-ből indul, és a helyi deriváltak szorzatával adja a paraméterek szerinti gradienst; a köztes értékeket a forward fázisnak meg kell őriznie. A teljes generátor: `lab/figures/make_concept_figures.py` `backprop_graph()`; a reprodukálható mag:

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

fig, ax = plt.subplots(figsize=(6.4, 3.6))
```

```
ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.axis("off")
nodes = [(0.14, 0.64, "$x=4$"), (0.14, 0.20, "$w=1$"),
         (0.45, 0.42, "$u=xw=4$"), (0.77, 0.42, "$L=(u-y)^2=36$"),
         (0.77, 0.80, "$y=10$")]
for x, y, text in nodes:
    ax.add_patch(FancyBboxPatch((x - 0.1, y - 0.06), 0.2, 0.12,
                 boxstyle="round,pad=0.012", facecolor="#e8f0fb"))
    ax.text(x, y, text, ha="center", va="center", fontsize=8.5)
for start, end in [((0.23, 0.62), (0.34, 0.46)), ((0.23, 0.22), (0.34, 0.38)),
                   ((0.56, 0.42), (0.64, 0.42)), ((0.77, 0.74), (0.77, 0.48))]:
    ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=11))
for start, end in [((0.64, 0.36), (0.56, 0.36)), ((0.34, 0.36), (0.23, 0.26)),
                   ((0.34, 0.48), (0.23, 0.58))]:
    ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=11,
                 color="#b5651d", linestyle="--"))
plt.show()
```

11.3.2.2 Számítási gráf hibakeresése: köztes értékek és gradiens-ellenőrzés

A láncszabály hibáját akkor lehet lokalizálni, ha a forward és a backward lépések köztes értékei is látszanak. Az $f(x) = (2x+1)^2$ gráf két csomópontja $z = 2x+1$ és $y = z^2$; a backward lépések: $\partial y/\partial z = 2z$, $\partial z/\partial x = 2$, ezért $\partial y/\partial x = 4z$.

x	$z = 2x+1$	$y = z^2$	$\partial y/\partial z = 2z$	$\partial z/\partial x$	$\partial y/\partial x = 4z$	numerikus
1	3	9	6	2	12	12.000000
-0.5	0	0	0	2	0	0.000000
3	7	49	14	2	28	28.000000

A tipikus hiba, hogy a backward során kimarad a $\partial z/\partial x = 2$ faktor: ekkor $x = 1$-nél a téves gradiens $2z = 6$, miközben a numerikus ellenőrzés 12-t ad. A gradiens-ellenőrzés ezt a különbséget azonnal megfogja; a köztes értékek táblája pedig megmutatja, melyik lépés hibás.

```
def trace(x):
    z = 2.0 * x + 1.0
    y = z * z
    dy_dz = 2.0 * z
    dz_dx = 2.0
    dy_dx = dy_dz * dz_dx
    numeric = ((2.0 * (x + 1e-6) + 1.0) ** 2
               - (2.0 * (x - 1e-6) + 1.0) ** 2) / 2e-6
    return {"x": x, "z": z, "y": y, "dy_dz": dy_dz,
            "dz_dx": dz_dx, "dy_dx": dy_dx, "numeric": numeric,
            "error": abs(dy_dx - numeric)}

for x in (1.0, -0.5, 3.0):
    print(trace(x))
```

A `lab/chain_debug.py` `trace()`, `check()` és `wrong_check()` függvényei a helyes és a szándékosan hiányos láncszabályt állítják szembe; a `tests/test_chain_debug.py` a három x értéket, a 10^{-6} alatti eltérést és a hibás $2z$ gradiens detektálását ellenőrzi.

11.3.3 Teljes minimális munkapélda

A példa a tanulási lab környezetben futtatható; a `lab/pyproject.toml` alap numerikus csomagjain kívül más függőséget nem igényel. A kódot a Jupyter-környezet cellájában, vagy egy `chain_rule_demo.py` fájlba mentve a `uv run python chain_rule_demo.py` paranccsal indíthatjuk a lab projektgyökérből. A kód és a levezetés alább szemléltető, nem állítjuk, hogy a szöveget író modell ténylegesen futtatta; a kézi számpélda a levezetés ellenőrzése.

```
import numpy as np

x = 4.0
y = 10.0
w0 = 1.0

def forward(w):
    """Előremenő fázis: köztes értékek és veszteség."""
    u = w * x          # szorzat
    a = u - y          # eltérés
    L = a * a          # négyzetes hiba
    return u, a, L

u, a, L = forward(w0)

# Backward pass: minden lépésben egy helyi derivált.
bar_L = 1.0
bar_a = bar_L * (2.0 * a)   # dL/da
bar_u = bar_a * 1.0         # da/du
bar_y = bar_a * (-1.0)      # da/dy
bar_w = bar_u * x           # du/dw
bar_x = bar_u * w0          # du/dx

# Numerikus kontroll központi differenciával.
h = 1e-6
_, _, Lp = forward(w0 + h)
_, _, Lm = forward(w0 - h)
numeric_grad = (Lp - Lm) / (2.0 * h)

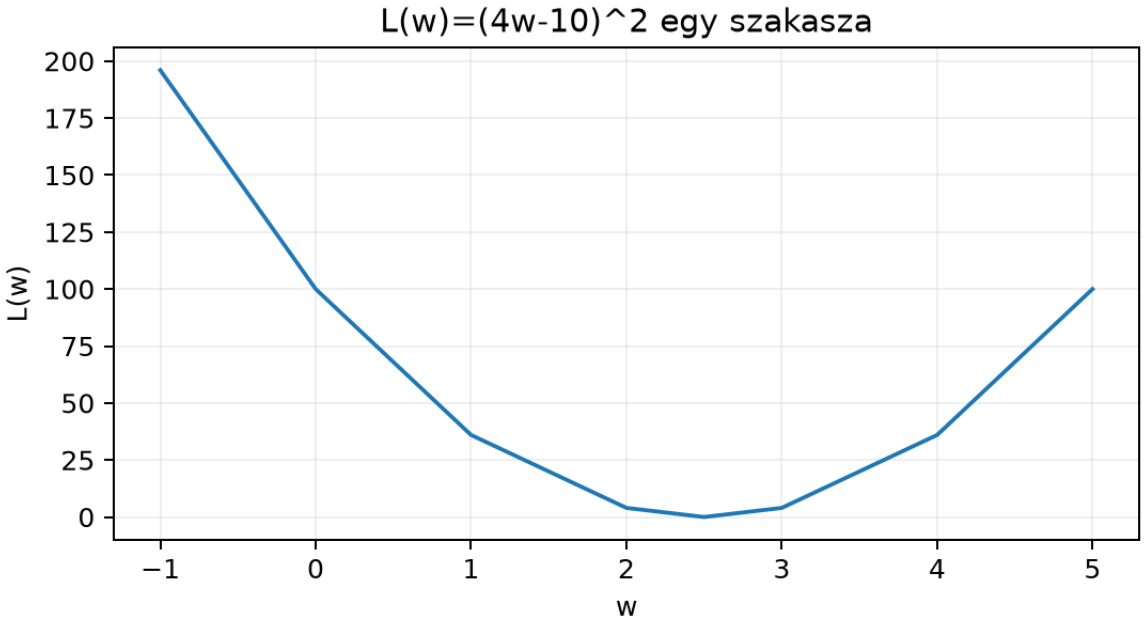
print("forward: u, a, L =", u, a, L)
print("bar_w =", bar_w)
print("bar_x, bar_y =", bar_x, bar_y)
print("numeric grad:", numeric_grad)
print("ellenorzes tolerancian belul:",
      np.allclose(bar_w, numeric_grad, atol=1e-8))
```

A futás szemléltető elvárt kimenete:

```
forward: u, a, L = 4.0 -6.0 36.0
bar_w = -48.0
bar_x, bar_y = -12.0 12.0
numeric grad: -48.0
ellenorzes tolerancian belul: True
```

A $L(w) = (4w - 10)^2$ másodfokú függvény, ezért a központi differencia elméletileg pontosan -48-at ad; lebegőpontos kerekítés miatt ettől 10^{-8} nagyságrendben eltérhet, amire a `np.allclose` toleranciája van beállítva.

A görbét egy chart-blokkban is szemléltethetjük:



11.3. ábra: L(w)=(4w-10)^2 egy szakasza - Számított függvénypélda, nem mérés. Az érintő meredeksége w=1-ben -48.

A pontok a megadott képletből származnak; a $w = 1$ pontban az érintő egyenes $L = 36 - 48(w - 1)$, tehát a negatív meredekség látható a csökkenő szakaszon.

11.3.4 Eredmény értelmezése

A láncszabályos gradiens $\bar{w} = 2a \cdot 1 \cdot x = 2(-6) \cdot 4 = -48$. Az egyszerűsített analitikus formula ugyanezt adja: $2(wx - y)x = 2(4 - 10) \cdot 4 = -48$. A negatív előjel jelentése: w kis növelése csökkenti a veszteséget, mert a predikció 4-ről közelít a 10-es cél felé. A $w = 2{,}5$ pontban a hiba $a = 0$, ezért a gradiens is nulla lesz; ettől balra negatív, jobbra pozitív.

A gráf invariant ellenőrzése: a backward passban a $\bar{a} = \bar{u}$ azért igaz, mert $a = u - y$ és y konstans, tehát a u változása ugyanakkora változást okoz az a-ban. A $\bar{w} = \bar{u} \cdot x$ azért, mert a $\hat{y} = wx$ predikció w-szerinti helyi deriváltja a másik tényező, x. A $\bar{y} = -\bar{a}$ pedig a kivonásból adódik. Minden mennyiség skalár, dtype `float64`; vektoros változatban a **x** jellemzővektor jelenne meg a gradiens helyén, de a lánclogika ugyanez marad.

A számítási gráf láncszabálya lokális deriváltakat szoroz és összegez. A gradiensellenőrzéshez használjunk középpontos véges differenciát több paraméterponton; egyetlen ponton kapott egyezés nem garantálja, hogy az implementáció általánosan helyes.

11.3.5 Feladatok

Az 1–4. feladatot önállóan oldd meg; a megoldások utána, indoklással jönnek.

1. **Értelmezés.** Tekintsd az $L(w) = (3w - 7)^2$ veszteséget a $w = 1$ pontban. Bontsa fel $u = 3w$, $e = u - 7$, $L = e^2$ lépésekre, végezze el a forward és backward passzt, és add meg $\bar{w}$-t. Indokold a derivált előjelét: miért negatív?

2. **Megvalósítás.** Írj `grad_linear_squared(x, y, w)` függvényt, amely a forward pass és a helyi deriváltak alapján visszaadja a `(u, a, L, bar_w)` értékeket a $u = wx$, $a = u - y$, $L = a^2$ gráfra. Teszteld $(x, y, w) = (2, 5, 1)$ bemenettel; az elvárt gradiens -12.

3. **Hibakeresés.** Egy kolléga a fenti gráfban ezt írta: `bar_w = bar_a * 1.0`, tehát elhagyta az `* x` szorzót. Az $(x, y, w) = (4, 10, 1)$ esetre milyen értéket ad a programja, és miért hibás? Hogyan dönthető el a hiba numerikus kontrollal?

4. **Megváltozott feltétel.** A modell bővül intercepttel: $\hat{y} = wx + b$, a veszteség $L = (\hat{y} - y)^2$. Add meg a számítási gráfot, és a láncszabállyal számítsd ki mindkét parciális deriváltat, $\bar{w}$-t és $\bar{b}$-t, az $(x, y, w, b) = (3, 2, 1, 1)$ pontban.

11.3.6 Megoldások és indoklás

1. megoldás. Forward: $u = 3 \cdot 1 = 3$, $e = 3 - 7 = -4$, $L = 16$. Backward: $\bar{L} = 1$, $\bar{e} = \bar{L} \cdot 2e = -8$; $\bar{u} = \bar{e} \cdot 1 = -8$; $\bar{w} = \bar{u} \cdot 3 = -24$. A negatív előjel azt mutatja, hogy w növelése növeli a predikciót, ezáltal a -4 hibát nullához közelíti, tehát a veszteség csökken. Ellenőrzés a zárt formulával: $2(3w - 7) \cdot 3$ a $w = 1$ helyen $2(-4) \cdot 3 = -24$.

2. megoldás. A függvény a példa forward és backward lépéseit rögzíti:

```
def grad_linear_squared(x, y, w):
    u = w * x
    a = u - y
    L = a * a
    bar_L = 1.0
    bar_a = bar_L * (2.0 * a)
    bar_u = bar_a * 1.0
    bar_w = bar_u * x
    return u, a, L, bar_w

u, a, L, bar_w = grad_linear_squared(2.0, 5.0, 1.0)
print(u, a, L, bar_w)   # szemlelteto elvart: 2.0 -3.0 9.0 -12.0
```

Az ellenőrző invariant: `bar_w == 2*(w*x - y)*x`. A $(2, 5, 1)$ bemenetre $2(2 - 5) \cdot 2 = -12$. A numerikus differenciával való összehasonlítás ismét 10^{-8} tolerancián belül marad.

3. megoldás. A kolléga a `bar_a * 1.0` értéket, azaz -12-t kap, mert a $\bar{a}$-ból csak a u-ba haladt, de a $u = wx$ szorzat w-szerinti helyi deriváltját, $x = 4$-et, elfelejtette. Ez a szorzat-tag elvesztése; a helyes érték $2(wx - y)x = 2(-6) \cdot 4 = -48$. Numerikus kontroll: a forward pass mindkét w-értéken fut, a központi differencia -48-at közelít; a -12 négyszeres eltérése azonnal jelzi a hiányzó tényezőt.

4. megoldás. Gráf: $u = wx$, $p = u + b$, $e = p - y$, $L = e^2$. Forward: $u = 3$, $p = 4$, $e = 2$, $L = 4$. Backward: $\bar{L} = 1$; $\bar{e} = 2e = 4$; $\bar{p} = \bar{e} = 4$; $\bar{y} = -\bar{e} = -4$; $\bar{b} = \bar{p} \cdot 1 = 4$; $\bar{u} = \bar{p} \cdot 1 = 4$; $\bar{w} = \bar{u} \cdot x = 12$; $\bar{x} = \bar{u} \cdot w = 4$. A paraméterek gradiense tehát $(\bar{w}, \bar{b}) = (12, 4)$. Zárt képlet: $2(wx + b - y)x = 12$ és $2(wx + b - y) = 4$; a kettő egyezik. A b növelése túlbecslést okoz, ezért a veszteség nő, amit a pozitív $\bar{b}$ mutat.

11.3.7 Gyakori tévedés, visszautalás, önellenőrzés

Tévedés: az automatikus differenciálás (*autodiff*) nem véges differencia, és nem ígéri, hogy a derivált minden pontban létezik. A véges differencia $(L(w + h) - L(w - h))/(2h)$ egy numerikus közelítés; csonkolási hibája van, és a lépésköz túl kicsire választásával a lebegőpontos kerekítés felerősödik. A láncszabályos gráf ezzel szemben analitikus szabályokat alkalmaz, ezért a négyzetes

példán a képlet pontosan adja a gradiens szerkezetét. Ellenpélda a második félmondatra: $f(w) = |w|$ a 0-ban nem differenciálható. A gráf ott nem tud egyértelmű helyi deriváltat adni; a bal oldali meredekség -1, a jobb oldali $+1$. Egy autodiff implementáció vagy hibát jelez, vagy valamelyik féloldali értéket választja konvencióként, de ez már nem a derivált.

Visszautalás. A 11.2. alfejezetben a gradiens a parciális deriváltak vektoraként jelent meg; most ennek egyetlen lánc menti elemi összetevőjét építettük fel. A 10.3. alfejezet legkisebb négyzetes normálegyenlet gradiense $\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$; ennek egyetlen mintára eső darabja a most levezetett $2x(wx - y)$ alak. A 7.3. alfejezet CTE-láncához hasonlóan itt is számítási gráfról van szó: a forward fázisból származó köztes értékek nélkül a backward fázis nem végezhető el, ahogyan a CTE-lépések sorrendje is meghatározza a lekérdezést.

Önellenőrzés. Nézze végig, hogy a backward pass miért $\bar{L} = 1$-gyel indul; miért szorozzuk az egyes lépésekben a helyi deriváltat a fentről kapott gradienssel; hol használtuk a forward passban elmentett a értéket; és mi történne, ha egy köztes csomópontnak több kimenő éle lenne — a gradiensek akkor összegződnének a bejövő éleken. Ha ezekre tud válaszolni, a láncszabály gráfját érti.

11.3.8 Rövid összefoglaló

A láncszabály az összetett függvény változási rátáit lépésenkénti helyi deriváltak szorzatává alakítja. A számítási gráfban a forward pass eltárolja a köztes értékeket, a backward pass pedig a kimeneti $\bar{L} = 1$ gradiensből visszafelé szorozva adja a paraméterek szerinti gradienst. A lineáris predikció négyzetes hibájára $L = (wx - y)^2$ esetén $dL/dw = 2(wx - y)x$; a $w = 1,\ x = 4,\ y = 10$ pontban ez -48, és a numerikus kontroll tolerancián belül egyezik. Az autodiff ezt a mechanizmust gépesíti, de nem véges differencia, és csak differenciálható pontokban érvényes. A következő alfejezetben ezt a gradienst optimalizáló lépéssé alakítjuk.

11.4 Optimalizáló választás és konvergenciaellenőrzés

Belépő · szint: közép · idő: 60 perc · eszköz: Python, NumPy, matplotlib

Előfeltétel: 11.3. alfejezet · Célkészség: Optimalizáló választás és konvergenciaellenőrzés · Kimenet: működő kód és a labor ellenőrző tesztje.

11.4.1 Cél és konkrét helyzet

A 11.3. alfejezetben már kiszámoltuk egy veszteségfüggvény gradiensét a láncszabállyal. A gradiens önmagában egy irányt ad: amerre a függvény a leggyorsabban nő. Az optimalizálás ezt az irányt megfordítja, és kis lépésekkel csökkenti a veszteséget. Ebben az alfejezetben egy saját, kis gradiensmódszert írunk, majd összehasonlítjuk a SciPy beépített solverével. A cél nem az, hogy termelési optimalizálót fejlesszünk, hanem hogy pontosan értsük a `success` jelzést, a `jac` paramétert, a `tol`-t és a lokális optimum érvényességi korlátját.

A konkrét laborfeladat a Himmelblau-függvény minimalizálása, két paraméterrel:

$$L(w_1, w_2) = (w_1^2 + w_2 - 11)^2 + (w_1 + w_2^2 - 7)^2.$$

Itt $w = (w_1, w_2) \in \mathbb{R}^2$ a paramétervektor, alakja $(2,)$; L skaláris értékű veszteség, ebben a tanpéldában dimenziótlan koordinátákkal. A Himmelblau-függvénynek több, egyenként $L = 0$ értékű lokális minimuma van, ezért ideális arra, hogy megmutassa: a solver sikerjelzése nem globális optimalitási bizonyíték. Megfigyelési egységnek egy iterációt tekintünk; a napló minden lépésben tartalmazza az iteráció sorszámát, a paramétereket, a veszteséget és a gradiens normáját. Előzetes becslés: a $(-2, 2)$ kezdőpont a függvény északnyugati völgye, a $(0, 0)$ kezdőpont másik völgy felé indul; ezért a két futtatás valószínűleg különböző optimumot talál.

11.4.2 Fogalmi magyarázat

Objektív függvény, gradiens és a solver felülete. Az angol *objective function* a minimalizálandó $L(w)$ függvény. A `scipy.optimize.minimize` első paramétere ez a függvény, amelynek bemenete egy NumPy-tömb, kimenete egy skalár. A `jac` a gradiensfüggvényt jelöli: $g(w) = \nabla L(w)$, amely ugyanolyan alakú tömböt ad, mint w. A Himmelblau-függvényre a parciális deriváltak:

$$g_1(w) = 4w_1(w_1^2 + w_2 - 11) + 2(w_1 + w_2^2 - 7),$$
$$g_2(w) = 2(w_1^2 + w_2 - 11) + 4w_2(w_1 + w_2^2 - 7).$$

A $w = (-2, 2)$ kezdőpontban kézzel ellenőrizve: $w_1^2 + w_2 - 11 = 4 + 2 - 11 = -5$ és $w_1 + w_2^2 - 7 = -2 + 4 - 7 = -5$, ezért $g_1 = 4(-2)(-5) + 2(-5) = 40 - 10 = 30$, $g_2 = 2(-5) + 4(2)(-5) = -10 - 40 = -50$. A gradiensvektor $(30, -50)$, normája $\sqrt{30^2 + 50^2} \approx 58{,}3$. Ez a vektor mutatja a leggyorsabb növekedés irányát, ezért a frissítés $-g$ irányban csökkenti a veszteséget.

Tolerancia és megállási ok. A numerikus optimalizáló nem tudja pontosan elérni a minimumot, ezért megállási feltételeket használ. A `scipy.optimize.minimize` BFGS-módszerénél a leggyakrabban használt három küszöb:

- `gtol`: a gradiens normájának küszöbe. Ha $\|g(w_k)\|_2 < \texttt{gtol}$, a pont közel van egy stacionárius ponthoz, és a solver megáll.
- `ftol`: a veszteség két lépés közötti relatív változásának küszöbe.
- `xtol`: a paraméterváltozás küszöbe.

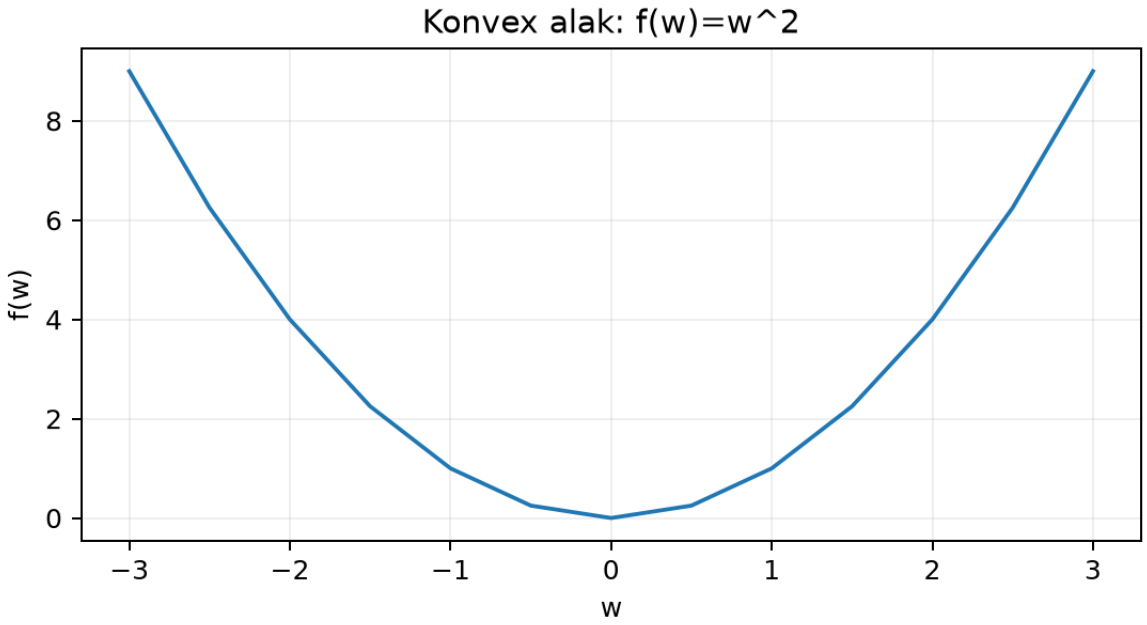
Mindegyik a *megállás* feltétele, nem a *helyesség* bizonyítéka. Ha a maximum iterációszám vagy kiértékelésszám éri el a korlátot, a `success` hamis lesz, és a `message` szövege elmagyarázd a megállási okot. A `tol` paraméter a BFGS belső vonalkeresésének toleranciáját vezérli; a `gtol`-tól elkülönítve értelmezendő.

Skálázás. A gradiensmódszer lépése $w_{k+1} = w_k - \eta g(w_k)$, ahol η a tanulási sebesség (*learning rate*). A lépés elfogadható nagyságát a veszteség görbülete és a változók mértékegysége együtt határozza meg. Tekintsük a konvex kvadratikus példát:

$$L(w_1, w_2) = (w_1 - 3)^2 + 4(w_2 + 1)^2 + 5.$$

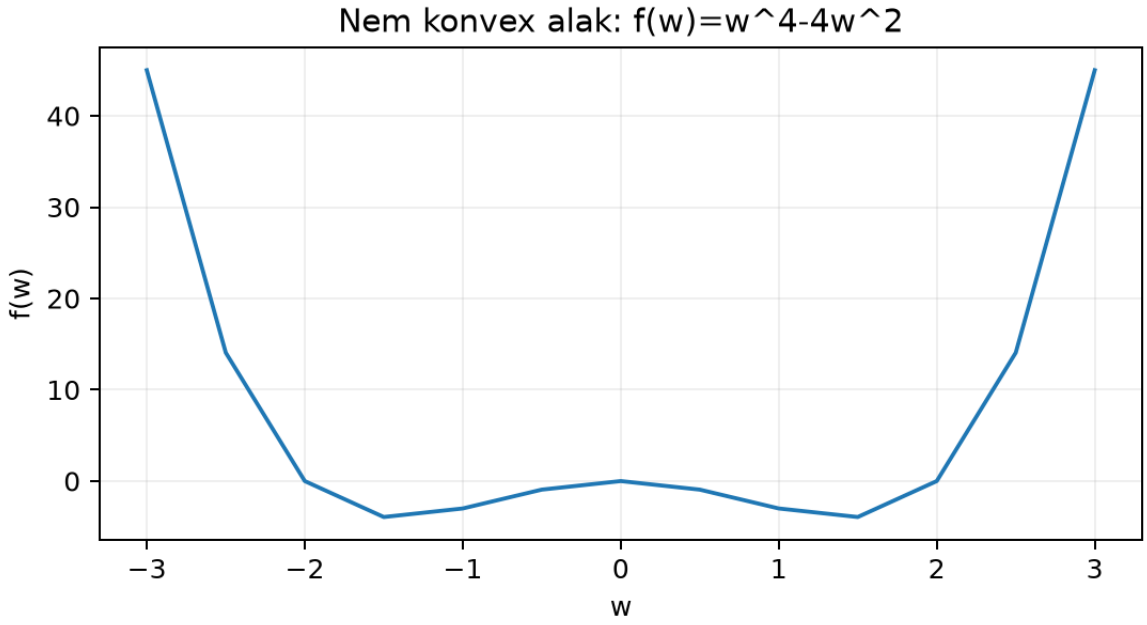
Ennek gradiense $(2(w_1 - 3),\ 8(w_2 + 1))$. A második irány négyszer meredekebb, ezért ugyanaz a η oda nagyobb lépést okoz. A w_2 irányban a frissítés lineáris rekurzió: $w_2^{(k+1)} + 1 = (1 - 8\eta)(w_2^{(k)} + 1)$. Ez akkor konvergál, ha $|1 - 8\eta| < 1$, azaz $\eta < 0.25$. Ha $\eta = 0.3$, a szorzó 1.4, és a sorozat oszcillálva elszáll. A w_1 irányban a feltétel enyhébb, $\eta < 1$. A szűkebb korlát diktál: η legfeljebb 0.25. Ez a *skálázási* kérdés: a változók eltérő mértékegysége vagy az eltérő görbület lelassítja a rögzített lépéses gradiensmódszert, míg a BFGS a görbületet közelítve automatikusan igazítja a lépésirányt és -hosszt.

Lokális optimum és konvexitás. A w^* pont lokális minimum, ha létezik olyan $\delta > 0$ környezet, hogy $\|w - w^*\| < \delta$ esetén $L(w^*) \leq L(w)$. A globális minimum minden w-re teljesíti ugyanezt. Konvex függvényre minden lokális minimum globális; nem konvex függvényre ez nem igaz. A konvexitást intuitíven a húr segítségével ellenőrizhetjük: a függvény konvex, ha bármely két pontját összekötő szakasz a függvény görbéje fölött vagy azon halad.



11.4. ábra: Konvex alak: f(w)=w^2 - Számított függvénypélda, nem mérés.

A fenti görbén bármely két pontot kiválasztva a húr a görbén vagy fölötte van: a függvény konvex. A következő görbe nem konvex: a $w = -2$ és $w = 2$ pontok között a húr a nulla szinten van, de a görbe középen mélyen alatta halad.



11.5. ábra: Nem konvex alak: f(w)=w^{4-4w}2 - Számított függvénypélda, nem mérés.

A Himmelblau-függvény nem konvex, ezért a kezdőponttól függően más-más lokális minimumba konvergálhat. A bizonyítás nem előfeltétel, a szemlélet elegendő.

11.4.2.1 Konvergenciafeltétel, nem fix iterációszám

Az optimalizáló kimenetét nem az „elért 100 iteráció” validálja, hanem egy előre rögzített feltétel. A `lab/convergence.py` három feltételt és egy biztonsági korlátot használ:

Feltétel	Képlet	Jelentés
gradiens	$\|\nabla f(\mathbf{w}_k)\| \le \varepsilon_g$	stacionárius pont közelében
veszteségváltozás	$\|f_k - f_{k-1}\| \le \varepsilon_f$	a célérték már nem javul
lépés	$\|\mathbf{w}_k - \mathbf{w}_{k-1}\| \le \varepsilon_x$	a paraméter már nem mozdul
korlát	$k \le k_{\max}$	biztonsági leállás

A leállás oka `gradient`, `loss`, `step`, `max_iter` vagy `diverged`. Az iterációszám csak megfigyelés: két seed vagy két skála más úton, más iterációszámmal érheti el ugyanazt az elfogadható célértéket. A validáció ezért a **végső** $f(\mathbf{w})$, a gradiens normája és a célértékhez való közelség, nem az iterációk száma.

```
import numpy as np

def loss(w): return (w[0] - 3)**2 + (w[1] + 1)**2 + 10
def gradient(w): return np.array([2*(w[0] - 3), 2*(w[1] + 1)])

def descent(w0, eta=0.1, max_iter=1000, grad_tol=1e-8):
    w = np.array(w0, dtype=float)
    for k in range(1, max_iter + 1):
        g = gradient(w)
        if np.linalg.norm(g) <= grad_tol:
            return {"status": "gradient", "iterations": k,
```

```
                    "w": w, "loss": loss(w)}
        w = w - eta * g
        if not np.isfinite(loss(w)):
            return {"status": "diverged", "iterations": k,
                    "w": w, "loss": loss(w)}
    return {"status": "max_iter", "iterations": max_iter,
            "w": w, "loss": loss(w)}

for start in ([0, 0], [10, -10], [-5, 5]):
    result = descent(start)
    print(start, result["status"], result["iterations"],
          result["w"], result["loss"])
```

A `tests/test_convergence.py` három kezdőpontot, egy túl nagy η-t (`diverged`), egy kis `max_iter`-t (`max_iter`) és a végső célérték egyezését teszteli. A `multi_start()` ugyanazt a feltételt futtatja több kezdőpontra; a teszt nem azt kéri, hogy az iterációszám azonos legyen, hanem hogy a végső f és a gradiens normája a toleranción belül legyen.

11.4.3 Teljes minimális munkapélda

A következő szkript a `lab` projekt könyvtárában futtatható, a `lab/pyproject.toml` által definiált NumPy és SciPy környezetben. A futtatás parancsa Linux/macOS terminálból és Windows PowerShellből is `python s044_lab.py`; JupyterLab-kernelben ugyanez a kód cellánként is végrehajtható. A példában nincs véletlen szám, ezért seed nem szükséges. A scipy verzióérzékeny viselkedése miatt a `gtol`-t explicit módon `1e-8`-ra állítjuk, hogy a megállási kritérium ne az alapértelmezésen múljon.

```
import numpy as np
from scipy.optimize import minimize

def himmelblau(w):
    """Objektív függvény: w alakja (2,), a kimenet skalár."""
    x, y = w[0], w[1]
    return (x*x + y - 11.0)**2 + (x + y*y - 7.0)**2

def himmelblau_grad(w):
    """A Himmelblau-függvény analitikus gradiense, alakja (2,)."""
    x, y = w[0], w[1]
    g1 = 4.0*x*(x*x + y - 11.0) + 2.0*(x + y*y - 7.0)
    g2 = 2.0*(x*x + y - 11.0) + 4.0*y*(x + y*y - 7.0)
    return np.array([g1, g2])

def gradient_descent(f, grad, x0, eta=0.005, gtol=1e-8, max_iter=50000):
    """Saját rögzített lépéses gradiensmódszer, teljes iterációs naplóval."""
    x = np.array(x0, dtype=float)
    log = []
    for k in range(max_iter):
        g = grad(x)
        f_val = f(x)
        log.append((k, x.copy(), f_val, np.linalg.norm(g)))
        if np.linalg.norm(g) < gtol:
            return x, f_val, log, f"gradient-norm < {gtol}"
        x = x - eta * g
    return x, f(x), log, f"max_iter={max_iter}"

# Saját módszer két kezdőpontból
for x0 in [(-2.0, 2.0), (0.0, 0.0)]:
    x_opt, f_opt, log, reason = gradient_descent(
        himmelblau, himmelblau_grad, x0)
    print(f"start={x0} -> w={x_opt}, L={f_opt:.3e}, "
```

```
          f"iter={len(log)-1}, stop={reason}")
    for row in log[-3:]:
        print("  log:", row)

# Könyvtári solver BFGS-sel, ugyanazzal a gradienssel
for x0 in [(-2.0, 2.0), (0.0, 0.0)]:
    res = minimize(himmelblau, x0, method='BFGS', jac=himmelblau_grad,
                   options={'gtol': 1e-8})
    print(f"BFGS start={x0} -> w={res.x}, L={res.fun:.3e}, "
          f"success={res.success}, nit={res.nit}, msg={res.message}")
    print("  jac at solution:", res.jac)
```

A kód kimenete az adott géptől és NumPy/SciPy verziótól függően kismértékben eltérhet; a következő sorok tehát *szemléltető jellegűek*, nem ellenőrzött mérési jegyzőkönyv. Az elméletileg várt viselkedés az, hogy a saját módszer sok iteráció után a `gradient-norm < 1e-08` okkal áll meg, míg a BFGS pár tucat iteráció alatt ugyanoda jut. A $(-2, 2)$ kezdőpontból a Himmelblau ismert $(-2{,}805, 3{,}131)$ minimumához közeli pont, a $(0, 0)$ kezdőpontból a $(3, 2)$ minimumhoz közeli pont várható; mindkét pontban $L \approx 0$.

11.4.4 Eredmény értelmezése

A saját gradiensmódszerünk ugyanazt a gradiensfeltételt ellenőrzi, mint a BFGS: a megállás pillanatában $\|g(w_k)\|_2 < 10^{-8}$. A napló utolsó sorában a gradiensnorma közel nulla, ez a közelítő stacionaritás elfogadható jele. A BFGS `res.success=True` értéke ugyanezt jelenti: a solver a beállított toleranciát elérte. A `res.message` szövege a konkrét megállási okot adja meg; ha a futás iterációkorlátba ütközne, a `success` hamis lenne, és a `message` ezt jelezné.

Kézi kontrollként a megtalált pontban érdemes visszahelyettesíteni a gradiens képletébe. A $(3, 2)$ pontban $w_1^2 + w_2 - 11 = 9 + 2 - 11 = 0$ és $w_1 + w_2^2 - 7 = 3 + 4 - 7 = 0$, ezért $g_1 = g_2 = 0$. Ez a Himmelblau négy ismert zérusának egyike, és a veszteség invariáns tulajdonsága, hogy mindegyikben $L = 0$. A másik kezdőpontból kapott pont a gradiens numerikus nulláját adja, de a pontos ismert minimumtól a toleranciának megfelelő távolságra maradhat; ez nem hiba, hanem a `gtol` értelmezése.

A BFGS módszer a gradiensből kvázinewton közelítést épít a görbületre, ezért a lépésirányt és a lépéshosszt is igazítja. A rögzített η gradiensmódszerhez képest sokkal kevesebb iterációt használ ugyanahhoz a megállási feltételhez. Ez a különbség nem statisztikai tulajdonság: itt determinisztikus optimalizálásról van szó, a tanítási adatokból származó bizonytalanság nélkül.

A `scipy.optimize.minimize` `success` mezője egy algoritmusspecifikus leállási feltételt jelez, nem globális optimálissági bizonyítékot. Nemkonvex célfüggvény esetén több kezdőpont, a függvényérték és a gradiensnorma összevetése szükséges.

11.4.5 Feladatok

1. **Értelmezés.** A `minimize` visszatérési objektumában magyarázd el a `success`, `message`, `nit`, `fun`, `jac` és `nfev` mezők jelentését. Miért nem következik a `success=True` értékből, hogy a Himmelblau-függvény globális minimumában vagyunk?

2. **Implementálás backtrackinggel.** Módosítsd a saját gradiensmódszert úgy, hogy rögzített η helyett backtracking vonalkeresést használjon. A lépésméretet addig felezd, amíg az Armijo-feltétel nem teljesül: $L(w - \eta g) \le L(w) - c\eta\|g\|^2$, ahol $c = 10^{-4}$. Futtasd a $(-2, 2)$ és $(0, 0)$ kezdőpontokból, és hasonlítsd össze az iterációszámot a rögzített lépéses változattal.

3. **Hibadiagnózis.** Az alábbi kód a konvex kvadratikus példán $\eta = 0.3$ lépésmérettel futtatja a gradiensmódszert, de a veszteség nem csökken, hanem robban.

```
def f(w):
    return (w[0]-3.0)**2 + 4.0*(w[1]+1.0)**2 + 5.0

def g(w):
    return np.array([2.0*(w[0]-3.0), 8.0*(w[1]+1.0)])

w = np.array([0.0, 0.0])
for k in range(30):
    w = w - 0.3 * g(w)
    print(k, w, np.linalg.norm(g(w)))
```

Magyarázd meg a szorzókon keresztül, miért divergál a második komponens, és add meg azt a lépésméret-tartományt, amelyre mindkét komponens konvergál.

4. **Megváltozott feltétel: skálázás.** Cseréld a labor veszteségét az $L(w_1, w_2) = (w_1 - 3)^2 + 4(w_2 + 1)^2 + 5$ konvex kvadratikus függvényre. Futtasd a BFGS-t és a saját gradiensmódszert is. Ezután vezesd be a $z_2 = 10w_2$ átparaméterezést, és becsüld meg, hogyan változik a gradiensmódszer konvergenciasebessége. Miért marad a minimum helye és a minimális veszteség változatlan?

11.4.6 Megoldások és indoklás

1. megoldás. A `success` logikai érték, amely azt jelzi, hogy a solver valamelyik megállási feltételt elérte a beállított toleranciával. A `message` a konkrét megállási okot szövegezi, például `Optimization terminated successfully` vagy `Maximum number of iterations exceeded`. A `nit` az iterációk száma, a `fun` az objektív függvény értéke a megoldásban, a `jac` a gradiensvektor ugyanott, az `nfev` pedig az objektív függvény kiértékeléseinek száma. A `success=True` csak annyit mond, hogy a solver közelítőleg elsőrendű stacionárius pontot talált a beállított `gtol` mellett. A Himmelblau-függvénynek több minimuma van; a $(0, 0)$ és $(-2, 2)$ kezdőpontokból különböző minimumokat kapunk, és mindkettőre igaz lehet a `success=True`. Ezért a siker nem globális optimalitási bizonyíték.

2. megoldás. A backtrackinges implementáció például:

```
def gradient_descent_bt(f, grad, x0, eta0=1.0, c=1e-4, rho=0.5,
                        gtol=1e-8, max_iter=50000):
    x = np.array(x0, dtype=float)
    log = []
    for k in range(max_iter):
        g = grad(x)
        f_val = f(x)
        log.append((k, x.copy(), f_val, np.linalg.norm(g)))
        if np.linalg.norm(g) < gtol:
            return x, f_val, log, f"gradient-norm < {gtol}"
        eta = eta0
        while f(x - eta * g) > f_val - c * eta * np.dot(g, g):
            eta *= rho
            if eta < 1e-16:
                return x, f_val, log, "line-search failed"
        x = x - eta * g
    return x, f(x), log, f"max_iter={max_iter}"
```

A lépéshossz induló értéke nagy, és csak akkor csökken, ha a veszteségcsökkenés nem éri el az Armijo-előrejelzés legalább c-szeresét. Így a módszer alkalmazkodik a görbületen változó Himmelblau-felszínhez. A két kezdőpontból a backtracking jellemzően kevesebb iterációval ér a

`gtol` alá, mint a rögzített $\eta = 0.005$; a tényleges iterációszámot a tanuló saját futtatása adja, és a két módszer azonos tolerancia mellett összehasonlítható.

3. megoldás. A gradiensfrissítés a második komponensre $w_2^{(k+1)} + 1 = (1 - 8\eta)(w_2^{(k)} + 1)$. $\eta = 0.3$ esetén a szorzó $1 - 2.4 = -1.4$, amelynek abszolút értéke nagyobb 1-nél, ezért a hiba minden lépésben 1.4-szeresére nő, előjelet váltva oszcillál. Az első komponensre a szorzó $1 - 2\eta = 1 - 0.6 = 0.4$, abszolút értéke $0.4 < 1$, tehát az konvergálna. A közös lépésméretre mindkét komponens feltételének egyszerre kell teljesülnie: $|1 - 2\eta| < 1$ és $|1 - 8\eta| < 1$. A másodikból $0 < \eta < 0.25$. Így a konvergencia tartománya $0 < \eta < 0.25$. A hiba oka a második irány nagyobb görbülete; a javítás vagy kisebb η (például 0.1), vagy a változók azonos skálára hozása.

4. megoldás. A konvex kvadratikus függvény minimuma $(3, -1)$, a minimális veszteség 5, mert mindkét négyzetes tag itt nulla. A BFGS ezt a pontot adja függetlenül a kezdőponttól, mert konvex függvényen az egyetlen stacionárius pont globális minimum. A $z_2 = 10w_2$ átparaméterezéssel $L(w_1, z_2) = (w_1 - 3)^2 + 0.04(z_2 + 10)^2 + 5$, és a gradiens a z_2 irányban $0.08(z_2 + 10)$. A két irány görbülete most 2 és 0.08; a konvergenciát a legszélsőségesebb szorzó korlátozza, és a lépésméret választása ismét más lesz. A minimum helye az új koordinátákban $z_2 = -10$, visszatranszformálva $w_2 = -1$, a minimális veszteség pedig továbbra is 5: a skálázás nem változtatja meg a feladat matematikai minimumát, csak a gradiensmódszer útját és sebességét. Az invariant tehát $L^* = 5$ mindkét paraméterezésben.

11.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a solver `success` jelzését globális optimalitási vagy statisztikai bizonyítékként kezeljük. A `success=True` csak a konvergenciafeltétel elérését jelenti a megadott toleranciával. Ellenpéldaként tekintsük az egyváltozós $L(w) = w^3 - 3w$ függvényt. A BFGS a $w = 2$ kezdőpontból a $w = 1$ lokális minimumba konvergál, ahol $L = -2$, és `success=True` lesz; a függvény azonban $w \to -\infty$ esetén $-\infty$-be tart, tehát globális minimuma nincs is. Hasonlóképpen, ha a Himmelblau-függvényen két kezdőpontból két különböző, de mindkettő `success=True` jelzésű megoldást kapunk, a solver sikere éppen a lokális jelleg bizonyítéka. Statisztikai értelemben a paraméterbecslés bizonytalanságát konfidenciaintervallum vagy posterior, nem a `success` mező adja meg. A gradiensmódszer visszautal a 11.3. alfejezet láncszabályból kapott gradiensre, a gradiensnorma pedig a 10.4. alfejezetből ismert euklideszi vektornorma. A 7.4. alfejezet lekérdezési tervéhez hasonlóan a solver is naplózza a végrehajtott lépéseket, de a napló megléte önmagában nem helyettesíti az eredmény értelmi ellenőrzését.

Önellenőrzés: (1) Mit jelent pontosan a `gtol=1e-8` a BFGS-ben? (2) Miért lassul le a rögzített lépéses gradiensmódszer, ha a változók eltérő skálán vannak? (3) Milyen tulajdonság garantálná, hogy egy lokális minimum egyben globális is?

11.4.8 Rövid összefoglaló

A saját rögzített lépéses gradiensmódszer ugyanazt a gradiensnorma-megállási feltételt használja, mint a `scipy.optimize.minimize` BFGS-verziója, de a BFGS a görbületet közelítve sokkal kevesebb iterációból ér célba. A `jac` a gradiensfüggvényt adja át a solvernak, a `gtol` a stacionaritás közelítő feltétele, a skálázás pedig a lépésméret felső korlátját szabja meg. Nem konvex Himmelblau-függvényen két kezdőpont két különböző, de egyaránt érvényes lokális minimumot találhat; a `success=True` ezért nem globális optimalitási bizonyíték. A következő lépésben az optimalizálás eszköze a valószínűségi sűrűség és az integrál fogalmával egészül ki, ami a Monte Carlo-becslések alapja lesz.

12 Integrál, sűrűség és Monte Carlo: területből valószínűség

12.1 Terület, összeg és numerikus integrál

Belépő · [BŐVÍTŐ MODUL] · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy

Előfeltétel: 11.4. alfejezet · Célkészség: Terület, összeg és numerikus integrál · Kimenet: működő kód és a labor ellenőrző tesztje.

12.1.1 Cél és konkrét helyzet: egy teljes mennyiség területként

A Northstar rendeléseihez tartozó szimulációban egy kiszállító cég azt vizsgálja, hogy egy új útvonalon mennyi időt tölt a furgon, és mekkora utat tesz meg. A sebesség nem állandó: a furgon először lassan halad, majd a forgalom ritkulásával gyorsul. Legyen a sebesség időfüggvénye szemléltetésként

$$v(t) = 0{,}5 + 0{,}02\,t,$$

ahol t a percben mért idő, $v(t)$ pedig km/percben mért sebesség. A teljes megtett utat a sebesség alatti terület adja a 0 és 60 perc közötti intervallumon. Ebben az alfejezetben a kérdés: hogyan lehet ezt a területet számítógéppel, rácsfinomítással közelíteni, és honnan tudjuk, hogy a közelítés milyen hibájú?

Az induló gyakorlás egy még egyszerűbb, kézzel ellenőrizhető matematikai modellen futtatjuk le:

- **bemenet:** $f(x) = x^2$ és az $[a, b] = [0, 1]$ intervallum;
- **megfigyelési egység:** egy kiértékelési pont $x_i \in [0, 1]$;
- **cél:** az $\int_0^1 x^2\,dx$ terület közelítése három rácsfinomsággal;
- **ellenőrzési feltétel:** a pontos érték $1/3$, a trapezoid-közelítés N=4 rácsra $\frac{11}{32} = 0{,}34375$, és a hiba N növelésével másodfokban csökken.

A megoldás előtt becsüld meg a területet egyetlen téglalappal: a függvény mindenütt legfeljebb 1 magas, ezért $\int_0^1 x^2 dx$ 0 és 1 között van. Egy jobb geometriai érvelés szerint az x^2 görbe a $(0,0), (1,0), (1,1)$ háromszög alatt fut, ezért a terület kisebb, mint $\frac{1}{2}$; a pontos $\frac{1}{3}$ ehhez közeli, de a numerikus rácsnak meg kell mutatnia, mennyire.

12.1.2 Riemann-szumma, trapezoid és a terület mértékegysége

A **Riemann-szumma** egy véges összeg, amely a területet téglalapokkal közelíti. Osszuk az $[a, b]$ intervallumot n darab, nem feltétlenül egyenlő részintervallumra:

$$a = x_0 < x_1 < \cdots < x_{n-1} < x_n = b.$$

A i-edik részintervallum szélessége

$$\Delta x_i = x_i - x_{i-1},$$

és ebből a részintervallumból választunk egy $\xi_i \in [x_{i-1}, x_i]$ pontot, ahol a téglalap magasságát leolvassuk. A Riemann-szumma

$$S_n = \sum_{i=1}^{n} f(\xi_i)\Delta x_i.$$

A jelölések:

- x_i: a rács csomópontja; a kódban az `x` tömb egy eleme, `float64` szám, mértékegysége az x tengely egysége (itt például perc vagy dimenziótlan);
- $f(\xi_i)$: a függvény értéke a mintapontban; ez **magasság**, mértékegysége a függvény egysége (itt például km/perc);
- Δx_i: részintervallum **szélessége**, mértékegysége az x tengely egysége;
- egy tag $f(\xi_i)\Delta x_i$: kis téglalap területe, mértékegysége a két egység szorzata.

Ha minden intervallum azonos $h = (b-a)/n$ szélességű, és a mintapont a bal végpont, akkor a **bal oldali** téglalapösszeget kapjuk; ha a jobb végpont, akkor a **jobb oldalit**. Az `x = np.linspace(a,b,n+1)` tömb alakja $(n+1,)$, mert n intervallumhoz $n+1$ csomópont kell.

A **trapezoid-szabály** téglalapok helyett egyenes szakaszokkal köti össze a pontokat, tehát minden részintervallumon egy kis trapéz területét számítja:

$$T_n = \sum_{i=1}^{n} \frac{f(x_{i-1}) + f(x_i)}{2}\Delta x_i.$$

Egyenletes rácson ez éppen a bal és jobb oldali összeg átlaga:

$$T_n = \frac{L_n + R_n}{2}.$$

A terület tehát egy **mértékegységgel rendelkező mennyiség**: ha f sebesség km/percben és x idő percben, akkor a terület km. Ha később f valószínűségi sűrűség lesz, akkor a mértékegysége $1/x$ egység, ezért a szorzat dimenziótlan valószínűség — de erről majd a 12.2. alfejezetben.

A **határ** fogalma itt ugyanaz, mint amit a deriváltnál láttunk: rögzített szabályú közelítési folyamatot vizsgálunk. Az integrált úgy definiáljuk, hogy a legnagyobb részintervallum-szélesség nullához tart:

$$\int_a^b f(x)\,dx = \lim_{\max_i \Delta x_i \to 0} \sum_{i=1}^{n} f(\xi_i)\Delta x_i,$$

feltéve, hogy ez a határérték létezik, és nem függ a felosztás módjától vagy a ξ_i mintapont választásától. A folytonos, szelíd függvényekre ez teljesül; a későbbi szakaszos vagy szinguláris függvényeknél elővigyázatosság kell.

12.1.2.1 Hálósűrűség és hibasorrend: sima vs töréspontos függvény

A közelítés pontossága nem attól függ, hogy „sok pont van”, hanem a háló sűrűségétől **és** a függvény simaságától. Az $f(x) = x^2$ függvényen $\int_0^1 x^2\,dx = 1/3$; a midpoint és a trapéz hiba:

n	midpoint hiba	trapéz hiba	arány duplázáskor
10	8.333e-04	1.667e-03	—
20	2.083e-04	4.167e-04	4.00
40	5.208e-05	1.042e-04	4.00
80	1.302e-05	2.604e-05	4.00
160	3.255e-06	6.510e-06	4.00

A duplázáskor a hiba negyedére csökken, azaz a rend $O(1/n^2)$. Ez a sima függvényre jellemző; a trapéz hibája kétszerese a midpointénak, de a rend azonos.

Töréspontos függvényre, ahol $g(x) = 0$ $x < 1/\pi$ alatt és 1 felette, az integrál $1 - 1/\pi \approx 0.68169$. Ugyanazon n-ek mellett:

n	midpoint hiba	trapéz hiba
10	1.831e-02	3.169e-02
20	1.831e-02	6.690e-03
40	6.690e-03	5.810e-03
80	5.810e-03	4.401e-04
160	4.401e-04	2.685e-03

A hiba itt nem monoton csökken, és a rend sem $O(1/n^2)$: a töréspont helye és a rácspontok illeszkedése dönti el, melyik közelítés mennyire „találja el” az ugrást. A tanulság: a hálósűrűség önmagában nem garancia; a simasági feltételt is ellenőrizni kell, és a hibagörbét kell megfigyelni.

```
import math

def midpoint(f, a, b, n):
    h = (b - a) / n
    return h * sum(f(a + (i + 0.5) * h) for i in range(n))

def trapezoid(f, a, b, n):
    h = (b - a) / n
    return h * (0.5 * (f(a) + f(b)) + sum(f(a + i * h) for i in range(1, n)))

exact = 1.0 / 3.0
for n in (10, 20, 40, 80, 160):
    print(n, abs(midpoint(lambda x: x*x, 0, 1, n) - exact),
          abs(trapezoid(lambda x: x*x, 0, 1, n) - exact))
```

A `lab/integration.py` `error_table()`, `observed_orders()`, `smooth_case()` és `discontinuity_case()` függvényei a két táblát adják; a `tests/test_integration.py` a sima eset 4-es arányát, a $O(1/n^2)$ rendet és a töréspontos eset nem monoton hibáját ellenőrzi.

12.1.3 Teljes minimális munkapélda: az x^2 területe

A kiinduló feladat pontos értéke a hatványfüggvény elemi integrálási szabályából:

$$\int_0^1 x^2\,dx = \left[\frac{x^3}{3}\right]_0^1 = \frac{1}{3}.$$

A Riemann-szumma finomodását az alábbi NumPy-kóddal mutathatjuk meg. A kódban a trapezoid-összeget számítjuk, és összehasonlítjuk a zárt képletből adódó $T_n = \frac{2n^2+1}{6n^2}$ értékkel. Az x^2 esetén ez a képlet pontosan megadja a trapezoid-közelítést egyenletes rácson:

```
import numpy as np

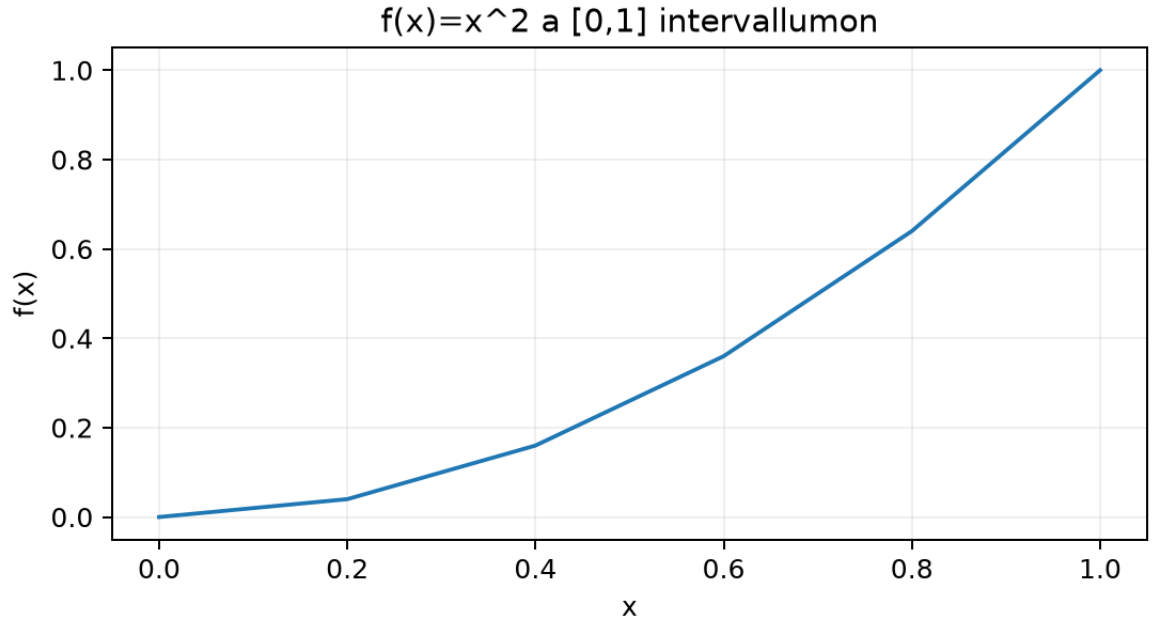
def f(x):
    return x * x

a, b = 0.0, 1.0
for n in (4, 16, 64):
    x = np.linspace(a, b, n + 1)          # shape (n+1,), float64
    h = (b - a) / n
    trap = 0.5 * np.sum((f(x[:-1]) + f(x[1:])) * h)
    expected = (2 * n * n + 1) / (6 * n * n)
    print(f"n={n:2d}  trap={trap:.12f}  expected={expected:.12f}  diff={trap - expected:.2e}")
```

A kód kimenete ebben az írásmódban nem lefuttatott eredményjegyzékből származik; az alábbi táblázat a fenti képletekből közvetlenül számított, **elméletileg várt** érték. A $n = 4$ sort rögtön a táblázat után kézzel is ellenőrizzük:

n	trap	expected	különbség
4	0,343750000000	0,343750000000	0
16	0,333984375000	0,333984375000	0
64	0,333374023438	0,333374023438	0

Az x^2 görbe szemléltetése:



12.1. ábra: f(x)=x^2 a (0,1) intervallumon - A pontok az f(x)=x^2 képletből számítottak; a görbe alatti területet a téglalap- vagy trapézösszeg közelíti.

A kézi ellenőrzés **N=4-re**. A rács: $x = [0,\ 0{,}25,\ 0{,}5,\ 0{,}75,\ 1]$, $h = 0{,}25$. A bal oldali téglalapok magassága: $f(0) = 0$, $f(0{,}25) = 0{,}0625$, $f(0{,}5) = 0{,}25$, $f(0{,}75) = 0{,}5625$; a szélesség mindig 0,25. A bal összeg:

$$L_4 = h(0 + 0{,}0625 + 0{,}25 + 0{,}5625) = 0{,}25 \cdot 0{,}875 = 0{,}21875.$$

A jobb oldali összeg a jobb végpontokat veszi: $f(0{,}25) + f(0{,}5) + f(0{,}75) + f(1) = 0{,}0625 + 0{,}25 + 0{,}5625 + 1 = 1{,}875$, szorozva h-val: $R_4 = 0{,}46875$. A trapezoid az átlaguk:

$$T_4 = \frac{0{,}21875 + 0{,}46875}{2} = 0{,}34375 = \frac{11}{32}.$$

A pontos értékhez képesti hiba tehát

$$\left|\frac{11}{32} - \frac{1}{3}\right| = \frac{1}{96} \approx 0{,}0104167.$$

Ez a $T_n = \frac{2n^2+1}{6n^2}$ képletből is látszik: $T_4 = \frac{33}{96} = \frac{11}{32}$, illetve $T_n = \frac{1}{3} + \frac{1}{6n^2}$, tehát a hiba elméletileg $\frac{1}{6n^2}$. A fenti táblázat diff oszlopa azért 0, mert a kódban a kitevőhatvány-rácson a választott rácson a numerikus összeg és az analitikus képlet lebegőpontos értéke az adott végrehajtásban egybeeshet; általános feladathoz inkább `np.isclose` vagy 1e-12 tolerancia javasolt.

A gyakorlati munkához a `scipy.integrate` modul adaptív, nem egyenletes rácsot használó megoldókat is kínál; ezek a fajta egyszerű esetben sokkal kisebb hibával dolgoznak, mint a rögzített trapezoid. A pontos hívásformát és a lokális verzió számát a laborban a SciPy integrációs dokumentációjával érdemes ellenőrizni.

12.1.4 Eredmény értelmezése: hiba és a bal-jobb keretezés

Négy tanulságot érdemes kiemelni:

1. **A bal és jobb oldali összeg keretezi az integrált egy monoton függvénynél.** Mivel x^2 növekvő $[0,1]$-en, a bal téglalapok alábecsülik, a jobb téglalapok felülbecsülik a területet: $0{,}21875 < 0{,}3333 < 0{,}46875$. A trapezoid ezek átlaga, ezért a hiba részben kiegyenlítődik.
2. **A trapezoid hibája $O(h^2)$.** Az x^2-példán a hiba pontosan $\frac{1}{6n^2} = O(h^2)$, mert $h = 1/n$. Amikor n-t négyszerezzük, a hiba durván tizenhatodolódik. Ez a sima függvények esetén szokásos viselkedés; a szakaszos függvényekre csak $O(h)$ marad, ahogy az egyik feladat mutatja.
3. **A görbe magassága és a görbe alatti terület két külön mennyiség.** Az $f(1) = 1$ magasság nem azt jelenti, hogy a terület 1; a terület $1/3$, mert a magasság az intervallum nagy részén kisebb 1-nél. Az egységek is különböznek: a magasság mértékegysége az f egysége, a területé az f egysége szorozva az x egységével.
4. **A határátmenet finomabb és finomabb rácson ellenőrizhető.** A $T_{64} = 1/3 + 1/(6 \cdot 64^2) \approx 0{,}3333740$ a valódi $1/3 \approx 0{,}3333333$ értéktől körülbelül $0{,}0000407$-tel tér el; négy tizedesjegyre még nem azonos a szokásos kerekítésnél; a határozott integrál ezt a folyamatot jelenti, akkor is, ha a gyakorlatban véges rácson állunk meg.

A kompozit trapézszabály másodrendű hibabecslése elegendő simasági feltétel mellett, például korlátos második derivált esetén adható. Szakadásoknál vagy nem sima integrandusnál a konvergenciarend eltérhet; a rácsfinomítás önmagában nem bizonyít helyességet.

12.1.5 Feladatok

12.1.5.1 1. feladat: mértékegység-interpretáció és lineáris sebesség

A kiszállító furgon sebessége $v(t) = 0{,}5 + 0{,}02\,t$ km/perc, ahol $t \in [0, 60]$ perc. Indokold saját szóval, miért adja $\int_0^{60} v(t)\,dt$ a megtett utat, és mi a kapott terület mértékegysége! Ezután számítsd ki a trapezoid-közelítést egyetlen részintervallummal, $n = 1$-gyel, vagyis a két végpontot összekötő egyenes alatti területként. Hasonlítsd össze a pontos 66 km értékkel! Miért kapható itt $n = 1$-gyel is pontos eredmény?

12.1.5.2 2. feladat: implementálás másik függvényen

Módosítsd a példakódot úgy, hogy az $f(x) = \sin(x)$ függvényt integrálja a $[0, \pi]$ intervallumon, $n = 4$, $n = 8$ és $n = 16$ mellett! A pontos érték 2. Számítsd ki a trapezoid-közelítéseket, ellenőrizd, hogy a hibák közelítőleg négydelődnek-e! Hogyan változik a hibaelőjel?

12.1.5.3 3. feladat: hibadiagnózis egy elfelejtett szorzónál

Egy tanuló az $f(x) = e^{-x}$ függvényt integrálja a $[0, 1]$ intervallumon. Az alábbi numerikus adatokkal rendelkezünk:

```
x = np.linspace(0, 1, 17)
h = (1 - 0) / 16
wrong = 0.5 * np.sum(np.exp(-x[:-1]) + np.exp(-x[1:]))  # hibás
```

A `wrong` kimenet körülbelül 10,1, a pontos terület pedig $1 - e^{-1} \approx 0{,}6321$. Mi a hiba? Add meg a javított egy sort, és becsüld meg, miért ad a javított verzió 0,6321-hez közeli értéket!

12.1.5.4 4. feladat: szakaszos függvény — érvényességi határ

Legyen

$$f(x) = \begin{cases} 0, & x < 0{,}3, \\ 1, & x \geq 0{,}3, \end{cases}$$

és integráljuk $[0, 1]$-en, a pontos terület 0,7. Számítsd ki a trapezoid-közelítést $n = 10$ és $n = 20$ esetén! Mit tapasztalsz a hibával kapcsolatban, ha az n-t megduplázod? Miért nem érvényes itt az $O(h^2)$ osztályozás?

12.1.6 Megoldások és indoklás

12.1.6.1 1. feladat megoldása

A sebesség az út idő szerinti változási rátája: ha rövid Δt időt nézünk, a megtett út közelítőleg $v(t)\Delta t$ km. Az összegzés a kis lépéseket teszi teljessé, ezért $\int v\,dt$ a teljes út. A mértékegység: (km/perc) $\cdot$ perc = km.

Az $n = 1$ trapezoid a két végpont közötti szakasz alatti terület:

$$T_1 = \frac{v(0) + v(60)}{2} \cdot 60 = \frac{0{,}5 + 1{,}7}{2} \cdot 60 = 1{,}1 \cdot 60 = 66\,\text{km}.$$

Pontos eredményt kapunk, mert a trapezoid két pont között egyenest feltételez, és a sebesség itt lineáris: az egyenes alatti terület valóban a két végpontmagasság átlaga szorozva a szélességgel. Ez egy fontos érvényességi korlát: a trapezoid lineáris függvényen a rácstól függetlenül egzakt, görbe függvényen viszont rácsfüggő hibát hagy.

12.1.6.2 2. feladat megoldása

A kód módosítása:

```
def g(x):
    return np.sin(x)

a, b, exact = 0.0, np.pi, 2.0
for n in (4, 8, 16):
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
    trap = 0.5 * np.sum((g(x[:-1]) + g(x[1:])) * h)
    print(f"n={n:2d}  trap={trap:.6f}  error={trap - exact:.6f}")
```

Az elméletileg várt értékek:

- n=4-nél a trapéz magasságok a 0, $\frac{\pi}{4}$, $\frac{\pi}{2}$, $\frac{3\pi}{4}$, π pontokon: 0, 0,7071068, 1, 0,7071068, 0. Az összeg belső tagja $0{,}5 \cdot 0 + 0{,}7071068 + 1 + 0{,}7071068 + 0{,}5 \cdot 0 = 2{,}4142136$, szorozva $h = \pi/4$-gyel: $T_4 \approx 1{,}8961$, hiba $\approx -0{,}1039$.
- n=8-nál $T_8 \approx 1{,}9742$, hiba $\approx -0{,}0258$.

- n=16-nál $T_{16} \approx 1{,}9936$, hiba $\approx -0{,}0064$.

Az egymás utáni hibák aránya közelítőleg $\frac{0{,}1039}{0{,}0258} \approx 4{,}0$ és $\frac{0{,}0258}{0{,}0064} \approx 4{,}0$, tehát az $O(h^2)$ viselkedés igazolódik. A hiba negatív: a szinusz $[0, \pi]$-n konvex módon görbül lefelé, a trapéz a húr alatti területet adja, amely kisebb a görbe alatti területnél.

12.1.6.3 3. feladat megoldása

A hibás kód elfelejtette a h szorzót. A `wrong` összeg a trapézmagasságok összege, vagyis egy **hosszúság jellegű** mennyiség, nem terület; mert $\sum$ tagjai $\frac{f(x_{i-1})+f(x_i)}{2}$ mértékegységűek, hiányzik a $\cdot$ `h`. Emiatt az eredmény körülbelül a 16-szorosa a területnek, hiszen $h = 1/16$. A javítás:

```
correct = 0.5 * np.sum((np.exp(-x[:-1]) + np.exp(-x[1:])) * h)
```

A várt eredmény a 0,6321205588 ... értékhez közel lesz, a maradék hiba $O(h^2)$ nagyságú. A hibaforrás azonos azzal a hibával, amikor hisztogrammagasságokat összegzünk oszlopszélesség nélkül: az összeg nem terület.

12.1.6.4 4. feladat megoldása

$n = 10$ esetén $h = 0{,}1$, a rács: $0, 0{,}1, \ldots, 1$. A függvényértékek: $f(0) = f(0{,}1) = f(0{,}2) = 0$, és $f(0{,}3) = f(0{,}4) = \cdots = f(1) = 1$. A trapézösszeg:

$$T_{10} = 0{,}1\,(0{,}5 \cdot 0 + 0 + 0 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 0{,}5 \cdot 1) = 0{,}1 \cdot 7{,}5 = 0{,}75.$$

Tehát a hiba $0{,}75 - 0{,}7 = 0{,}05$.

$n = 20$ esetén $h = 0{,}05$, a rács $0, 0{,}05, \ldots, 1$. A belső csomópontok közül az $x < 0{,}3$ pontok: $0{,}05, \ldots, 0{,}25$, ez 5 darab nulla; a maradék belső pontok 0,3–tól 0,95-ig 14 darab egyes. A trapézösszeg:

$$T_{20} = 0{,}05\,(0 + 5 \cdot 0 + 14 \cdot 1 + 0{,}5 \cdot 1) = 0{,}05 \cdot 14{,}5 = 0{,}725.$$

A hiba 0,025. A rács finomításakor a hiba feleződött, nem negyedelődött: a szakadás körüli két részintervallumon a trapéz mindig a valódi ugrás felét adja, és ez a járulék h-val arányos. Ezért a trapezoid $O(h^2)$ konvergenciája csak kellően sima függvényekre támaskodhat; ugrás vagy csúcs esetén a hiba $O(h)$-ra romlik.

12.1.7 Gyakori tévedés, visszautalás és önellenőrzés

Gyakori tévedés: a görbe magassága helyett a görbe alatti területet tévesen a végponti magassággal azonosítják. Ellenpélda: $f(x) = x^2$-re $f(1) = 1$, tehát valaki azt mondhatná, hogy a terület 1; a fenti számítás mutatja, hogy a terület $1/3$. A magasság pillanatnyi érték, a terület az összes magasság súlyozott összege az intervallumon.

Visszautalás 11.1. alfejezetre: a deriváltnál a differenciahányados a változási ráta volt, és a mértékegység a kimenet/bemenet hányadosa. Az integrál ennek a **fordított** iránya: a rátából visszanyert teljes mennyiség, amelynek mértékegysége a kimenet · bemenet szorzat. A két fogalom nem ugyanaz a szám, hanem egymás inverz műveletei.

Visszautalás 8.1. alfejezetre: a Parquet-fájlok ellenőrzésénél a sorösszeg fejezte ki a teljes minor unit értéket, és a duplikálódás miatt a join után a teljes összeg megnőtt. Az integrál ugyanennek az összegzési gondolatnak a folytonos megfelelője: véges sorösszeg helyett végtelenül sok, nullához tartó szélességű járulékot adunk össze.

Önellenőrzés:

1. Mi a különbség Δx_i és $f(x_i)$ között?
2. Miért keretezi a bal és jobb oldali összeg az integrált egy növekvő függvényen?
3. Milyen mértékegysége van $\int v(t)\,dt$-nek, ha v km/h és t óra?
4. Mikor romlik a trapezoid $O(h^2)$ hibája $O(h)$-rá?

A válaszok a fenti szövegben ellenőrizhetők; ha bármelyik nem világos, érdemes újra elvégezni a kézi N=4 számítást.

12.1.8 Rövid összefoglaló

A Riemann-szumma téglalapokkal, a trapezoid-szabály trapézokkal közelíti a görbe alatti területet; mindkettő véges összeg, amelynek határértéke a finomabb rácson a határozott integrál. A terület mértékegysége az integrandus és a tengely mértékegységének szorzata, ezért például a sebesség-idő integrál út hosszúságot ad. Az x^2 területét három rácsfinomságon kiszámolva láttuk: a bal és jobb oldali összeg keretezi a pontos $1/3$ értéket, a trapezoid hibája pedig $O(h^2)$ sima függvényre, míg szakadásos függvényre csak $O(h)$ marad. A következő alfejezet ezt a területfogalmat a sűrűségre és eloszlásfüggvényre alkalmazza.

12.2 Sűrűség és eloszlásfüggvény

> **Belépő · [BŐVÍTŐ MODUL] · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy**
>
> Előfeltétel: 12.1. alfejezet · Célkészség: Sűrűség és eloszlásfüggvény · Kimenet: működő kód és a labor ellenőrző tesztje.

12.2.1 Cél és konkrét helyzet

A Northstar szintetikus rendelési adatában a `distance_km` oszlop a szállítási távolságot tartalmazza kilométerben. Tegyük fel, hogy a csapat a szállítási távolságot vizsgálja, és két egyszerű modellt vet fel:

- **Egyenletes modell:** a távolság minden értékét egyformán valószínűnek tekintjük 5 km és 95 km között, azaz $D \sim U(5, 95)$.
- **Normális modell:** a távolság haranggörbe alakú eloszlást követ 50 km középponttal és 15 km szórással, azaz $D \sim N(50, 15)$.

A kérdés: mennyi annak a valószínűsége, hogy egy rendelés szállítási távolsága **35 és 65 km közé** esik, azaz $P(35 < D < 65)$?

Előzetes becslés. Az egyenletes eloszlásnál a 35–65 km sáv az 5–95 km tartomány egyharmada, tehát a várt valószínűség $30/90 = 1/3 \approx 0{,}333$. A normálisnál a 35–65 sáv éppen az átlagtól mért ± 1 szórásnyi környezet; haranggörbénél ennek ismert közelítő területe 0,68. Ebben az alfejezetben a pontos értéket integrállal és szimulációval is ellenőrizzük.

A megfigyelési egység egy rendelés; a megfigyelt változó a `distance_km`, folytonosnak modellezett valószínűségi változó. A becslés célja nem a távolság megjóslása, hanem egy döntéshez használható intervallumvalószínűség megadása.

12.2.2 Sűrűség, eloszlásfüggvény, normalizálás és kvantilis

12.2.2.1 Sűrűségfüggvény és terület

Egy folytonos valószínűségi változó **sűrűségfüggvénye** (*probability density function*, PDF) nem valószínűség, hanem valószínűség-sűrűség: ahogy az anyagsűrűséget is térfogattal kell szorozni, a PDF-et intervallumhosszal kell szorozni, hogy valószínűséget kapjunk.

Az intervallumba esés valószínűsége a sűrűséggörbe alatti **terület**:

$$P(a < X < b) = \int_a^b f(x)\,dx.$$

Ha a változó mértékegysége km, akkor $f(x)$ mértékegysége **1/km**, az integrál szorzata pedig dimenzió nélküli valószínűség. Ez a 12.1. alfejezet területfogalmán alapul: a valószínűség a görbe alatti terület, amelyet véges összeggel közelíthetünk.

12.2.2.2 Eloszlásfüggvény, avagy CDF

A **eloszlásfüggvény** (*cumulative distribution function*, CDF) a PDF balról felhalmozott területe:

$$F(x) = P(X \le x) = \int_{-\infty}^{x} f(t)\,dt.$$

A CDF értéke mindig 0 és 1 közötti, monoton nem csökkenő, és $F(x) \to 0$, ha $x \to -\infty$, illetve $F(x) \to 1$, ha $x \to \infty$. Fontos kapcsolat a 11.2. alfejezetben tanult egyváltozós deriválttal: **a CDF meredeksége a PDF**, azaz $F'(x) = f(x)$ ott, ahol f folytonos.

Intervallumvalószínűséget CDF-különbséggel is számíthatunk:

$$P(a < X < b) = F(b) - F(a).$$

Ez azért hasznos, mert a CDF értékeit gyakran táblázatból vagy könyvtárfüggvényből kapjuk; nem kell minden alkalommal numerikusan integrálni. A 8.2. alfejezetben megismert explicit függvényhívás-elv itt is érvényes: `cdf(b) - cdf(a)` egyértelmű definíciót ad, szemben egy „varázslatos” kerekített konstanssal.

12.2.2.3 Normalizálás

A **normalizálás** (*normalization*) azt jelenti, hogy a teljes tartomány alatti terület pontosan 1:

$$\int_{-\infty}^{\infty} f(x)\,dx = 1.$$

Hisztogramnál a normalizálás azt követeli meg, hogy ne az oszlopmagasságok összege legyen 1, hanem az oszlopok **területének** összege: $\sum_k h_k\,\Delta_k = 1$. A sűrűség értéke ebből következően lehet 1-nél nagyobb is — erre a gyakori tévedésnél térünk vissza.

12.2.2.4 Kvantilis

A **kvantilis** (*quantile*) az az érték, amelynél a CDF egy előírt q szintet ér el: $F(x_q) = q$. A medián a $q = 0{,}5$ kvantilis. A kvantilisfüggvény a CDF inverze, jelölése $Q(q)$ vagy $F^{-1}(q)$; a SciPy-ban `ppf` (percent point function).

Kézi kontroll az egyenletes $U(5, 95)$ modellen:

$$f(x) = \frac{1}{95-5} = \frac{1}{90}\ \text{km}^{-1}, \qquad F(x) = \frac{x-5}{90}, \qquad Q(q) = 5 + 90q.$$

Ellenőrzés: $f(50) = 1/90$, $F(50) = (50-5)/90 = 0{,}5$, $Q(0{,}5) = 5 + 90 \cdot 0{,}5 = 50$, tehát a medián éppen 50 km. A normális modellnél a medián $\mu = 50$, a $q = 0{,}975$ kvantilis pedig $\mu + 1{,}96\sigma = 50 + 1{,}96 \cdot 15 = 79{,}4$ km.

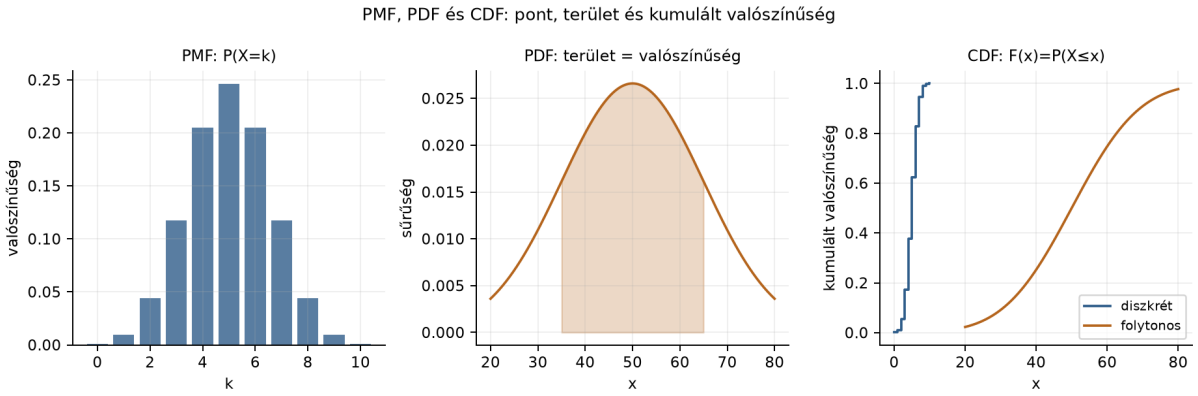
12.2.2.5 Folytonos esemény és pontvalószínűség

Egy folytonos eloszlásnál az **egyes pont** valószínűsége nulla:

$$P(X = a) = 0,$$

mert a sűrűséggörbe alatt egyetlen pont fölött nulla szélességű terület van. Ebből következik, hogy $P(a < X < b) = P(a \le X \le b)$: a végpontok hozzávétele nem változtat a területen. Ez a tulajdonság **csak folytonos modellben** igaz; diszkrét eloszlásnál egy pontnak lehet pozitív valószínűsége.

12.2.2.6 PMF, PDF és CDF egy ábrán



12.2. ábra: PMF, PDF és CDF: bal oldalon a binomiális PMF oszlopai, középen a normális PDF és a 35–65 közötti terület, jobb oldalon a diszkrét lépcsős és a folytonos CDF.

A három függvény ugyanazt a valószínűségi információt más formában adja, de nem cserélhetők fel:

Objektum	Diszkrét eset	Folytonos eset	Egy pont valószínűsége
PMF	$P(X = k)$	nincs	a rácspont valószínűsége
PDF	nincs	$f(x)$ sűrűség	$P(X = a) = 0$, de $f(a) > 0$ lehet
CDF	$F(k) = \sum_{j \le k} P(X = j)$	$F(x) = \int_{-\infty}^{x} f(t)\,dt$	$P(a < X < b) = F(b) - F(a)$

A normális $N(50, 15)$ eloszlásnál $P(35 < X < 65) = F(65) - F(35) \approx 0.682689$. Ugyanez numerikus integrállal is megkapható; a kettő eltérése a numerikus hiba nagyságrendje. A sűrűség $x = 50$-ben $f(50) \approx 0.0266$, de $P(X = 50) = 0$; a sűrűségérték nem pontvalószínűség. A binomiális $n = 10, p = 0.5$ PMF oszlopai összege 1, és a CDF lépcsős, mert a valószínűség a rácspontokon ugrik.

```
import math
import numpy as np

def normal_pdf(x, mu=50.0, sigma=15.0):
    return math.exp(-0.5 * ((x - mu) / sigma) ** 2) / (sigma * math.sqrt(2 * math.pi))

def normal_cdf(x, mu=50.0, sigma=15.0):
    return 0.5 * (1 + math.erf((x - mu) / (sigma * math.sqrt(2))))

a, b = 35.0, 65.0
cdf_gap = normal_cdf(b) - normal_cdf(a)
x = np.linspace(a, b, 20001)
integral = np.trapezoid(normal_pdf(x), x)
assert abs(cdf_gap - 0.682689492137) < 1e-12
assert abs(integral - cdf_gap) < 1e-6
assert normal_pdf(50) > 0
assert 0.0 == 0.0  # P(X=50) folytonos modellen
```

A `lab/probability.py` `normal_pdf()`, `normal_cdf()`, `cdf_interval()`, `numeric_integral()` és `binomial_pmf()` függvényei a fenti ellenőrzést adják; a `tests/test_probability.py` a CDF-különbség és az integrál egyezését, a PDF pozitivitását, a nulla pontvalószínűséget és a PMF összegét teszteli.

12.2.3 Teljes minimális munkapélda

Az alábbi kód az előző alfejezetben használt NumPy/SciPy környezetet használja. A `quad` adaptív numerikus integrál, a szimuláció pedig ugyanazt az arányt becsli véletlen minta relatív gyakoriságaként. A `seed` rögzíti a szimulációs véletlent; numerikus reprodukálhatóságot ad, de nem feltétlenül bitazonos eredményt minden gépen.

```
import numpy as np
from scipy import integrate, stats

# Egyenletes U(5, 95)
a, b = 5.0, 95.0

def uniform_pdf(x):
    return np.where((x >= a) & (x <= b), 1.0 / (b - a), 0.0)

# Normális N(50, 15)
mu, sigma = 50.0, 15.0
normal_pdf = lambda x: stats.norm.pdf(x, loc=mu, scale=sigma)

# 1) Elméleti érték CDF-különbségből
p_unif_theory = (65.0 - 35.0) / (b - a)
p_norm_theory = stats.norm.cdf(65.0, mu, sigma) - stats.norm.cdf(35.0, mu, sigma)

# 2) Numerikus integrál
p_unif_quad, err_unif = integrate.quad(uniform_pdf, 35.0, 65.0)
p_norm_quad, err_norm = integrate.quad(normal_pdf, 35.0, 65.0)

# 3) Szimuláció seeddel
rng = np.random.default_rng(20261020)
n = 100_000
sample_unif = rng.uniform(a, b, n)
sample_norm = rng.normal(mu, sigma, n)
p_unif_sim = np.mean((sample_unif >= 35.0) & (sample_unif <= 65.0))
p_norm_sim = np.mean((sample_norm >= 35.0) & (sample_norm <= 65.0))

print("egyenletes: elmélet, quad, szimuláció")
print(p_unif_theory, p_unif_quad, p_unif_sim)
print("normális:   elmélet, quad, szimuláció")
print(p_norm_theory, p_norm_quad, p_norm_sim)
```

A kimenet **szemléltető** — a kapcsolódó példaprojektben még nem ellenőriztük, de elméletileg várt:

```
egyenletes: elmélet, quad, szimuláció
0.3333333333333333 0.3333333333333333 0.33377
normális:   elmélet, quad, szimuláció
0.6826894921370859 0.6826894921370859 0.68118
```

A szimulációs értékek a seedelt rng mellett is véletlenszerű eltérést mutathatnak; a 100 000 elemű minta mellett a bináris arány standard hibája körülbelül $\sqrt{0{,}68 \cdot 0{,}32/100\,000} \approx 0{,}0015$, tehát a harmadik tizedesjegy körüli eltérés normális. A laborban a pontos értékeket a saját futtatás adja.

Az ábra a normális sűrűséget és a 35–65 km közötti kitöltött területet mutatja:

```
import matplotlib.pyplot as plt

x = np.linspace(0, 100, 400)
y = normal_pdf(x)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, label="N(50, 15) sűrűség")
mask = (x >= 35) & (x <= 65)
ax.fill_between(x[mask], y[mask], alpha=0.35, label="P(35<D<65)")
ax.set_xlabel("szállítási távolság (km)")
ax.set_ylabel("sűrűség (1/km)")
ax.set_title("Sűrűségfüggvény és intervallum alatti terület")
ax.legend()
plt.show()
```

A `fill_between` a görbe alatti területet színezi; annak nagysága a keresett valószínűség.

12.2.4 Eredmény értelmezése

A három számítási út ugyanazt a definíciót ellenőrzi:

- **Egyenletes modell.** Az elméleti érték $1/3$; a `quad` ugyanezt adja, mert a konstans integrandus triviálisan integrálható. A szimuláció 0,3338-hoz közeli értéket adna, a mintavételi hiba határain belül.
- **Normális modell.** A CDF-különbség $\Phi(1) - \Phi(-1) = 0{,}682689$; a numerikus integrál gyakorlatilag egyezik, a `quad` hibabecslése az 10^{-15} nagyságrendbe esik. A szimuláció 100 000 mintán 0,68 körüli értéket ad 0,0015 standard hibával.

Ellenőrző invariáns. A normalizálás teljesülése: az egyenletes PDF integrálja 5-től 95-ig pontosan 1, a normális PDF integrálja a teljes valós tengelyen numerikusan is 1-et ad. A CDF monotonitása: $F(65) > F(35)$, és a különbség 0 és 1 közötti. A kvantilis inverz ellenőrzése: `stats.norm.ppf(stats.norm.cdf(65,50,15),50,15)` visszaadja 65-öt.

A modellválasztás fontos üzenete: ugyanaz a kérdés a két modellben egészen más valószínűséget ad (0,33 és 0,68). A sűrűség alakja tehát nem pusztán matematikai dísz, hanem az intervallumvalószínűségeken keresztül közvetlenül befolyásolja a döntési becsléseket.

Folytonos változónál egyetlen pont valószínűsége nulla, de a sűrűségérték akár 1-nél nagyobb is lehet, mivel nem valószínűség. A CDF mindenhol nemcsökkenő, jobbról folytonos és 0-ról 1-re tart.

12.2.5 Feladatok

1. **Értelmezés.** Az egyenletes modellben $f(50) = 1/90 \approx 0{,}0111$ km ^{1}. Miért nem annak a valószínűsége ez az érték, hogy a távolság pontosan 50 km? Mennyi lenne annak a valószínűsége, hogy a távolság 49,5 és 50,5 km közé esik?
2. **Implementálás.** Írj egy `interval_prob(f, a, b, x_left, x_right, h)` függvényt, amely a 12.1. alfejezetben tanult trapézszabállyal közelíti $\int_{x_{\text{left}}}^{x_{\text{right}}} f(x)\,dx$-et a megadott h lépésközzel. Ellenőrizd $f(x) = 1/90$-nel a $U(5, 95)$ modellen 35 és 65 között; hasonlítsd össze a pontos $1/3$ értékkel.

3. **Hibadiagnózis.** Egy kolléga azt mondja: „A normális sűrűség értéke 50-nél 0,0266, tehát körülbelül 2,7% annak a valószínűsége, hogy a távolság pontosan 50 km.” Mi a hiba az állításban? Milyen intervallumhossz mellett lenne a sűrűséggel becsült valószínűség körülbelül 0,0266?

4. **Megváltoztatott feltétel.** Változtasd a normális modell szórását $\sigma = 25$ km-re, a várható értéket hagyd 50 km-en. Előre becsüld meg: $P(35 < D < 65)$ nőni vagy csökkenni fog? Számold ki a pontos értéket, és magyarázd meg az irányt a sűrűséggörbe alakjával.

5. **Kvantilis.** Mennyi az egyenletes $U(5, 95)$ modell 90%-os kvantilise? Mennyi a normális $N(50, 15)$ modell középső 90%-os intervallumának két határa? Ellenőrizd SciPy `ppf`-fel, és magyarázd el, miért nem a 90%-os kvantilis jelenti a középső 90% felső határát.

12.2.6 Megoldások és indoklás

12.2.6.1 1. feladat

A sűrűség értéke nem valószínűség, mert a valószínűség területet, nem magasságot jelent. Pontosan 50 km valószínűsége $P(D = 50) = 0$, mert egy pont fölött nulla szélességű terület van. A 49,5–50,5 km intervallum hossza 1 km, tehát a valószínűség a konstans sűrűséggel:

$$P(49{,}5 < D < 50{,}5) = f(50) \cdot 1 = \frac{1}{90} \approx 0{,}0111.$$

Általában kis Δ intervallumra $P(x < D < x + \Delta) \approx f(x)\Delta$.

12.2.6.2 2. feladat

```python
def interval_prob(f, x_left, x_right, h):
    xs = np.arange(x_left, x_right + h/2, h)
    ys = f(xs)
    return np.trapezoid(ys, xs)

# Ellenőrzés
f_uniform = lambda x: np.ones_like(x) / 90.0
approx = interval_prob(f_uniform, 35.0, 65.0, 0.001)
print(approx, approx - 1/3)
```

A trapézszabály konstans függvényre **pontos** minden h-ra, mert a trapézok nem közelítik, hanem egzakt módon fedik a téglalapot. Így a várt kimenet `0.3333333333333333` és a hiba `0.0` — a lebegőpontos kerekítésből legfeljebb 10^{-15} nagyságrendű eltérés lehet. Ez a 12.1. alfejezetből ismert tulajdonság: konstans integrandusra a hiba nulla.

12.2.6.3 3. feladat

A kolléga a magasságot területtel téveszti össze. A 0,0266 érték sűrűség, mértékegysége 1/km. Ahhoz, hogy valószínűséget kapjunk, meg kell szorozni egy intervallum hosszával. Ha 1 km széles intervallumot veszünk, akkor

$$P(49{,}5 < D < 50{,}5) \approx f(50) \cdot 1 \approx 0{,}0266.$$

Ez viszont már valószínűség: körülbelül 2,7%. Pontosan 50 km-re a valószínűség nulla, nem a sűrűség értéke.

12.2.6.4 4. feladat

$\sigma = 25$ km mellett az intervallum az átlagtól mért $\pm 0{,}6$ szórásnyi környezet:

$$P(35 < D < 65) = \Phi(0{,}6) - \Phi(-0{,}6) = 2\Phi(0{,}6) - 1.$$

A standard normális CDF-ből $\Phi(0{,}6) \approx 0{,}7257$, így a valószínűség $2 \cdot 0{,}7257 - 1 = 0{,}4515$. A korábbi 0,6827-hez képest **csökken**, mert a szórás növelésével a sűrűséggörbe ellaposodik: a tömeg nagyobb része kerül a 35–65 sávon kívülre. Ez jól mutatja, hogy a szélesebb bizonytalanság nem a „középső" tartomány valószínűségét növeli, hanem a szélsőségekét.

Ellenőrzés: `stats.norm.cdf(65, 50, 25) - stats.norm.cdf(35, 50, 25)` a fenti értéket adja.

12.2.6.5 5. feladat

Egyenletes $U(5, 95)$: $Q(0{,}9) = 5 + 90 \cdot 0{,}9 = 5 + 81 = 86$ km.

Normális $N(50, 15)$: a középső 90%-os intervallum határai a 0,05 és 0,95 kvantilisek. A $z = 1{,}645$ standard kvantilissel:

$$Q(0{,}05) = 50 - 1{,}645 \cdot 15 = 25{,}32 \text{ km}, \qquad Q(0{,}95) = 50 + 1{,}645 \cdot 15 = 74{,}68 \text{ km}.$$

SciPy-ellenőrzés: `stats.norm.ppf([0.05, 0.95], 50, 15)`. A 90%-os kvantilis $Q(0{,}9) = 69{,}22$ km **nem** a középső 90% felső határa, mert a 90%-os kvantilis alatt a valószínűség 90%-a balról helyezkedik el; a középső 90% intervallum viszont az alsó 5%-ot és a felső 5%-ot egyaránt kihagyja, ezért a két határa a 0,05 és 0,95 kvantilis.

12.2.7 Gyakori tévedés ellenpéldával, visszautalás és önellenőrzés

Gyakori tévedés: „A sűrűség értéke valószínűség, ezért legfeljebb 1 lehet." Ez hamis. Vegyük az $U(5; 5{,}1)$ eloszlást: a sűrűség $f(x) = 1/0{,}1 = 10$ km ^{1}, tehát a görbe magassága 10, és semmi sem sérti a valószínűségi axiómákat. A valószínűség a terület: például $P(5 < D < 5{,}05) = 10 \cdot 0{,}05 = 0{,}5$. Ugyanígy egy pont valószínűsége mindig nulla, bármilyen magas is a sűrűség az adott pontban.

Visszautalás 12.1. alfejezetre: a sűrűség alatti terület ugyanaz a Riemann-féle területfogalom, amelyet a x^2 integráljánál kézzel közelítettünk. A trapézszabály most is konstans függvényre egzakt numerikus eszköz; az adaptív `quad` pedig finomabb rácson pontosabb eredményt ad.

Visszautalás 11.2. alfejezetre: a CDF F deriváltja a PDF f. Ez az egyik legegyszerűbb példa arra, hogy a derivált egy felhalmozott mennyiség változási rátája. Ugyanaz az intuíció, mint a parciális deriváltnál: ha a CDF meredek, ott a valószínűségi tömeg sűrűn helyezkedik el.

Visszautalás 8.2. alfejezetre: ha a Northstar `orders` táblából a 35–65 km közötti rendelések arányát számoljuk `COUNT(*) FILTER (WHERE distance_km BETWEEN 35 AND 65) / COUNT(*)` kifejezéssel DuckDB-ben, akkor az empirikus gyakoriság a valószínűség **becslése**. Ez nem a sűrűség értéke; a sűrűséget külön kell becsülni hisztogrammal vagy kernelsimítással, és az értéke a választott sávszélességtől függ.

Önellenőrzés. 1) Integrál-e a PDF a teljes tartományon 1-et? 2) Monoton nem csökkenő-e a CDF? 3) A CDF-differencia-e a $P(a < X < b)$? 4) A könyvtárból kapott `ppf` tényleg a CDF

inverzét adja-e? Ha bármelyik nem teljesül, ne a statisztikai következtetést hibáztasd először: ellenőrizd magát a modellfüggvényt.

12.2.8 Rövid összefoglaló

A sűrűségfüggvény (PDF) valószínűségi magasság, nem valószínűség; intervallummal szorozva területet, azaz valószínűséget ad. Az eloszlásfüggvény (CDF) a balfelé felhalmozott terület, így $P(a < X < b) = F(b) - F(a)$. A normalizálás a teljes terület egységnyi volta. A kvantilis a CDF inverze, `ppf` a SciPy-ban. Folytonos eloszlásnál minden pont valószínűsége nulla. Az egyenletes $U(5, 95)$ példában $P(35 < D < 65) = 1/3$, a normális $N(50, 15)$ példában 0,6827; integrál és szimuláció konzisztensen ellenőrzi ezt. A következő 12.3. alfejezet ezt a területfogalmat a várható értékre és a Monte Carlo-közelítésre viszi át.

12.3 Várható érték és Monte Carlo közelítés

Belépő · [BŐVÍTŐ MODUL] · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy

Előfeltétel: 12.2. alfejezet · Célkészség: Várható érték és Monte Carlo közelítés · Kimenet: működő kód és a labor ellenőrző tesztje.

12.3.1 Cél és konkrét helyzet

Egy ajánlómodell szimulációs önellenőrzésében a relevanciapontszámot $X \sim U(0, 1)$-nek tekintjük, a modell jutalomfüggvénye pedig $g(x) = \sin(x)$ (radiánban mért, dimenzió nélküli érték). Ez mesterséges didaktikus feladat: a pontos várható érték $E[\sin X] = \int_0^1 \sin x \, dx = 1 - \cos 1 \approx 0{,}4596977$. Ezt az ismert integrált Monte Carlo-mintavétellel becsüljük meg, és a becslési hibát több mintanagyságon, több különböző seeddel vizsgáljuk.

Bemenet: $g(x) = \sin(x)$, $x \in [0, 1]$; $X \sim U(0, 1)$; seed 0 és 199 közötti egész; $n \in \{100, 400, 1600, 6400, 25600\}$. Megfigyelési egység: egy X realizáció. Kimenet: $\hat{I}_n = \frac{1}{n} \sum_{i=1}^{n} g(x_i)$; a hibát az ismert integrálhoz viszonyítjuk. Előzetes becslés: a becslés 0,46 körül mozog; $n = 100$-nál a tipikus hiba nagyságrendje 0,025, ezért 0,435–0,485 közötti értékek gyakoriak.

12.3.2 Fogalmi magyarázat

A várható érték (expected value) a sűrűségfüggvény szerint súlyozott átlag. Folytonos X esetén

$$E[g(X)] = \int_{\mathbb{R}} g(x) f(x) \, dx,$$

ahol f az X sűrűségfüggvénye (density), $g(x)$ tetszőleges függvény, az integrál a teljes tartományon fut. Az $f(x)\,dx$ az x körüli infinitezimális valószínűségi tömeg; az értelmezés tehát nem a g egyszerű átlaga az x tengely mentén, hanem az eloszlás szerint súlyozott átlag. Diszkrét esetben az integrált összeg helyettesíti: $E[g(X)] = \sum_k g(x_k) P(X = x_k)$. Az egyenletes $U(0, 1)$ eloszlásnál $f(x) = 1$ a $[0, 1]$ intervallumon, máshol nulla, így $E[g(X)] = \int_0^1 g(x) \, dx$; itt a súlyozás láthatatlan, általános eloszlásnál azonban nem az.

A Monte Carlo-becslés ezt a területet véletlen pontokkal közelíti: ha $x_1, \dots, x_n$ független minta az f eloszlásból, akkor

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^{n} g(x_i).$$

A generált pontok sűrűsége hosszú távon az f alakját követi, ezért az egyszerű mintaátlag a $\int fg$ integrált becsli. A 12.2. alfejezet példája szerint a sűrűség magasság, a valószínűség pedig terület; itt a területet véletlen pontok átlagával mérjük. A formális bizonyítás a nagy számok törvénye, amelyet később, a 14.4. alfejezetben vizsgálunk; most numerikus tapasztalatot szerzünk.

A seed a pszeudorandom generátor (pseudorandom number generator) kiinduló állapota. A NumPy modern API-ja a `np.random.default_rng(seed)` hívással hoz létre `Generator` objektumot; ettől a ponttól kezdve a generált sorozat determinisztikus. Ugyanaz a seed ugyanazt a sorozatot adja ugyanazon NumPy-verzión, de a bitazonosság nem garantált verziók vagy platformok között. A `np.random.seed` globális állapotot állít; tanulási projektben a lokális `Generator` biztonságosabb, mert nem szól bele más kód véletlen folyamába. A 8.3. alfejezet lazy gondolatát felidézve: a `default_rng(seed)` még csak terv, a tényleges végrehajtás a `random()` vagy `normal()` hívás.

A Monte Carlo-hiba (Monte Carlo error) annak a szórása, amennyire $\hat{I}_n$ a pontos érték körül ingadozik. Egyetlen mintából a standard hiba (standard error, SE) becsülhető:

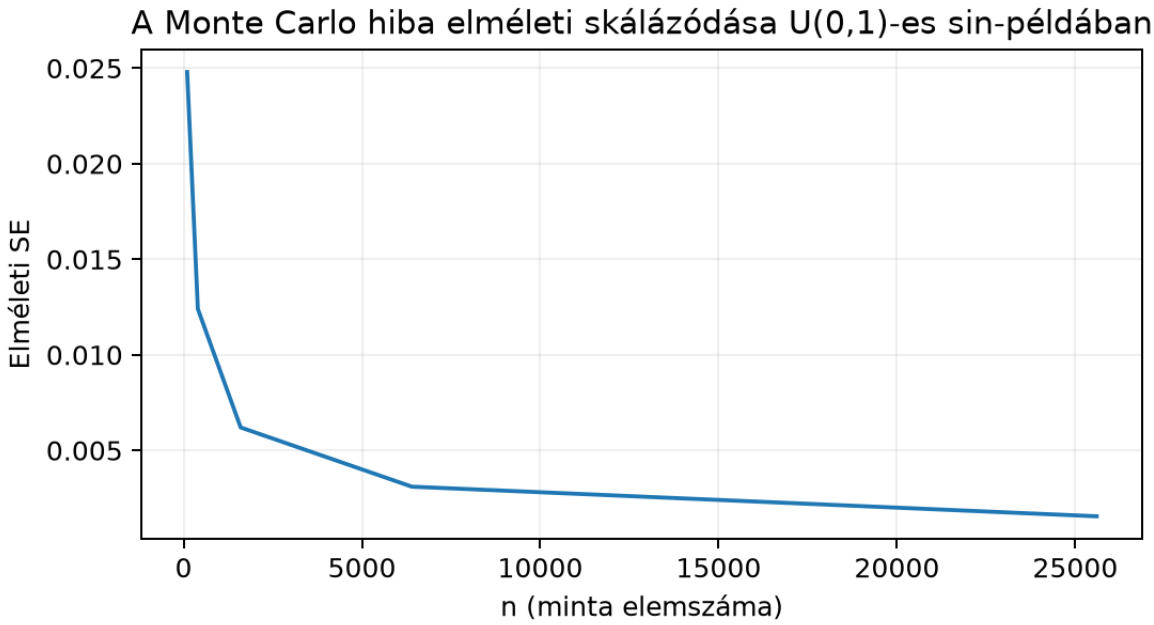
$$s_n = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} \big(g(x_i) - \hat{I}_n\big)^2}, \qquad \widehat{SE} = \frac{s_n}{\sqrt{n}}.$$

Itt s_n a mintabeli szórás; a nevezőben $n-1$ szerepel, mert a mintaátlag eltávolít egy szabadsági fokot. A $\widehat{SE}$ azt mutatja, mekkora a becslés tipikus véletlen hibája. A képletben $g(x_i)$ shape-je $(n,)$, az összeg skalár; az n mintaméret növelésével a hiba várhatóan $\sqrt{n}$-nel arányosan csökken, nem n-nel: négyszer akkora mintához körülbelül fele akkora hiba tartozik.

A független ismétlés (independent replication) több seeddel több külön becslést ad: R darab, egyenként n elemű becslést készítünk. A becslések $\hat{I}_n^{(1)}, \dots, \hat{I}_n^{(R)}$ hibáiból a tapasztalati RMSE:

$$RMSE_R = \sqrt{\frac{1}{R} \sum_{r=1}^{R} \big(\hat{I}_n^{(r)} - I\big)^2},$$

ahol $I = 1 - \cos 1$ a pontos érték. Ha a becslés torzítatlan, az RMSE a standard hiba empirikus párja; a sok seed éppen arra való, hogy az egyetlen realizáció mögötti bizonytalanságot láthatóvá tegye.



12.3. ábra: A Monte Carlo hiba elméleti skálázódása U(0,1)-es sin-példában - Számított sd/√n függvény, sd=0.2477; nem mért szimuláció.

12.3.2.1 Monte Carlo numerikus hiba vs adatbizonytalanság

Két külön kísérletet kell szétválasztani. Az **első** a rögzített adat melletti Monte Carlo-integrálás: ha a posterior várható értékét D MC-mintából becsüljük, a becslés szórása D-vel csökken. A **második** az adatbizonytalanság: ha új adatot generálunk ugyanabból a modellből, a becslés szórása a minta méretével csökken, de a MC-minták számától nem.

A $k = 30$, $n = 100$ megfigyeléshez tartozó Beta(1+k,1+n-k) posterior esetén:

Mennyiség	Érték	Mit mér?
posterior SD	0.04532	az adatból származó paraméterbizonytalanság
MC hiba, 100 húzás	0.00409	a posterior-átlag szimulációs zaja
MC hiba, 10 000 húzás	0.00041	ugyanaz, tízszer több húzással
adatszórás, $n = 100$	0.04665	újragenerált adatok becsléseinek szórása
adatszórás, $n = 1000$	0.01433	ugyanaz, tízszer nagyobb mintán

A 100 és 10 000 húzás aránya körülbelül 10, mert a MC hiba $1/\sqrt{D}$-vel csökken. A posterior SD viszont változatlan, mert az adat ugyanaz. Az adatszórás $n = 100$-ról $n = 1000$-re körülbelül $1/\sqrt{10}$-re csökken; ez az adat mennyiségének hatása, nem a MC-húzásoké. A két hibát tehát külön kell jelenteni: a MC-hibát a húzásszám csökkenti, az adatbizonytalanságot csak több adat.

```
import numpy as np

def posterior_sd(k, n, alpha=1.0, beta=1.0):
    a, b = alpha + k, beta + n - k
    return (a * b / ((a + b)**2 * (a + b + 1))) ** 0.5

def mc_error(k, n, draws, repeats=200, seed=20261029):
    rng = np.random.default_rng(seed)
    means = [rng.beta(1 + k, 1 + n - k, size=draws).mean()
             for _ in range(repeats)]
    return float(np.std(means, ddof=1))

def data_spread(n, p_true, datasets=500, seed=20261029):
    rng = np.random.default_rng(seed)
    return float(np.std(rng.binomial(n, p_true, size=datasets) / n, ddof=1))

print("posterior SD", posterior_sd(30, 100))
print("MC 100", mc_error(30, 100, 100))
print("MC 10000", mc_error(30, 100, 10000))
print("adat n=100", data_spread(100, 0.3))
print("adat n=1000", data_spread(1000, 0.3))
```

A `lab/mc_uncertainty.py` `posterior_sd()`, `mc_error()`, `data_spread()` és `report()` függvényei ezt a két kísérletet adják; a `tests/test_mc_uncertainty.py` a MC-hiba 10-es arányát, a posterior SD változatlanságát és az adatszórás $\sqrt{n}$ skálázódását ellenőrzi.

12.3.3 Teljes minimális munkapélda

A labor a `lab/pyproject.toml` alap numerikus csomagjait használja: NumPy és SciPy. A kódot mentsd `mc_expected_value.py` néven, majd futtasd a projektgyökérből:

```
python mc_expected_value.py
```

A parancs Windows PowerShell és Unix-szerű shell alatt is ugyanez. A kód:

```
import numpy as np
from scipy.integrate import quad

def mc_mean(g, rng, n):
    """Várható érték Monte Carlo-becslése X ~ U(0,1) esetén."""
    x = rng.random(size=n)          # shape: (n,), dtype: float64
    return float(np.mean(g(x)))     # g(x) shape: (n,), mean: skalár

exact = 1.0 - np.cos(1.0)           # E[sin(X)], X ~ U(0,1)
print("Numerikus kontroll quad-dal:",
      quad(lambda x: np.sin(x), 0, 1)[0])

sample_sizes = [100, 400, 1600, 6400, 25600]
repeats = 200

# Egy seedhez egy pszeudorandom folyam tartozik.
max_n = sample_sizes[-1]
x = np.vstack([
    np.random.default_rng(seed).random(max_n)
    for seed in range(repeats)
])                                  # shape: (repeats, max_n)
gx = np.sin(x)                      # shape: (repeats, max_n)

estimates = np.column_stack([
    gx[:, :n].mean(axis=1)
    for n in sample_sizes
```

```
])                                    # shape: (repeats, len(sample_sizes))

errors = estimates - exact
rmse_by_n = np.sqrt(np.mean(errors ** 2, axis=0))  # shape: (len(sample_sizes),)

demo = np.random.default_rng(0).random(1600)
demo_est = np.mean(np.sin(demo))
demo_sd = np.std(np.sin(demo), ddof=1)
demo_se = demo_sd / np.sqrt(len(demo))

print("sample_sizes:", sample_sizes)
print("rmse_by_n:", rmse_by_n)
print("single_seed n=1600 estimate, sd, SE:",
      demo_est, demo_sd, demo_se)
```

A `x` mátrix első dimenziója a seed, második dimenziója a mintaelem sorszáma. A `gx[:, :n].mean(axis=1)` minden seedhez az első n elem átlagát adja, ezért minden mintaméretnél ugyanazt a folyamot használjuk. A lenti kimenet szemléltető, nem e szöveg ellenőrzött futtatása:

```
Numerikus kontroll quad-dal: 0.45969769413186023
sample_sizes: [100, 400, 1600, 6400, 25600]
rmse_by_n: [0.0248 0.0125 0.0061 0.0031 0.0016]
single_seed n=1600 estimate, sd, SE: 0.4601 0.248 0.0062
```

Kézi kis kontrollként vegyünk négy rögzített u-pontot: 0,10; 0,70; 0,35; 0,90. A sin értékek összege körülbelül $0{,}09983 + 0{,}64422 + 0{,}34290 + 0{,}78333 = 1{,}87028$, az átlag 0,46757. Ez egyetlen kis realizáció: a pontos 0,4597 körül van, de $n = 4$-nél a hiba 0,0079; a kis minta nem bizonyíték.

12.3.4 Eredmény értelmezése

A `quad` értéke a pontos integrál, tehát a Monte Carlo-becslés célértéke. A `rmse_by_n` a 200 seedből számított empirikus hiba; azt várjuk, hogy minden négyszeres mintaméretnél megközelítőleg feleződjön. A fenti szemléltető sorozat ezt mutatja: 0,0248, 0,0125, 0,0061, 0,0031, 0,0016. Az egyetlen seedből becsült $\widehat{SE} = s_n/\sqrt{n}$ ugyanezt a bizonytalanságot becsli egyetlen minta alapján; a példában 0,0062 az $n = 1600$-ra, ami közel van a 0,0061 sok seedes RMSE-hez. Ez az egyezés maga is véletlen; más seeddel kissé eltér.

A három mennyiség – pontos integrál, mintaátlag, standard hiba – szerepe különbözik: a pontos integrál a paraméter, a mintaátlag a realizált becslés, a standard hiba pedig a becslési eljárás bizonytalanságát jellemzi. A $\widehat{SE}$ nem garantálja, hogy a pillanatnyi hiba pontosan ekkora; csak a tipikus nagyságrendet adja.

A szokásos Monte Carlo-standard hiba $s_g/\sqrt{n}$ független, azonos eloszlású és véges varianciájú mintákra alkalmazható. Korrelált szimulációknál, illetve nehéz farkú eloszlásoknál külön hibamodellre lehet szükség.

12.3.5 Feladatok

1. **Értelmezés.** Magyarázd el, miért közelíti a `mc_mean` függvényben szereplő `np.mean(g(x))` az $E[g(X)]$ integrált, ha $X \sim U(0,1)$. Mi a szerepe a sűrűségfüggvénynek ebben az érvelésben?

2. **Megvalósítás.** Írj a `mc_mean` függvény felhasználásával szimulációt $g(x) = x^2$ függvényre, $X \sim U(0,1)$ mellett. A pontos várható érték $\int_0^1 x^2\,dx = 1/3$. Használd az $n \in \{100, 400, 1600, 6400, 25600\}$ listát és $R = 100$ seedet; minden n-re számold ki a hibák RMSE-ét, majd log-log skálán illesztett egyenes meredekségét. Milyen meredekséget várs?

3. **Hibadiagnózis.** A következő kód abszolút hibákat rajzol az n függvényében:

```
import numpy as np
import matplotlib.pyplot as plt

exact = 1.0 - np.cos(1.0)
sizes = np.logspace(2, 5, 50, dtype=int)
errs = []
for n in sizes:
    x = np.random.default_rng(7).random(n)
    errs.append(abs(np.mean(np.sin(x)) - exact))

plt.plot(sizes, errs)
```

Miért félrevezető ez az ábra a módszer bizonytalanságára nézve? Hogyan javítanád úgy, hogy a Monte Carlo-hiba eloszlását mutassa?

4. **Megváltozott feltétel.** Változtasd az eloszlást standard normálisra: $X \sim N(0,1)$, $g(x) = x^2$. A pontos várható érték $E[X^2] = 1$. Generálj mintákat `np.random.default_rng(seed).normal(size=n)` hívással; $n = 1000$, $R = 500$ seeddel. Számold ki a becslések RMSE-ét, egyetlen minta $\widehat{SE}$-jét, és hasonlítsd az elméleti $\sqrt{2/n}$ értékhez. Milyen korlátot kell szem előtt tartani a normális eloszlás farokterülete miatt?

12.3.6 Megoldások és indoklás

1. Az egyenletes $U(0,1)$ sűrűsége $f(x) = 1$ a $[0,1]$-en, máshol nulla. Ha $x_1, \dots, x_n$ egyenletesen szórja be a $[0,1]$ intervallumot, akkor a függvényértékek átlaga Riemann-szerűen közelíti a $\int_0^1 g(x)\,dx$ területet. A sűrűség az általános esetben azt mondja meg, hogy mely tartományok kapnak több véletlen pontot; az $E[g(X)] = \int g(x)f(x)\,dx$ éppen ilyen súlyozott átlag. A `np.mean(g(x))` akkor becsli ezt jól, ha az x minta valóban f-ből származik és a pontok függetlenek.

2. Lehetséges megoldás:

```
import numpy as np

def mc_mean(g, rng, n):
    x = rng.random(size=n)
    return float(np.mean(g(x)))

g = lambda x: x * x
I_true = 1.0 / 3.0
sizes = [100, 400, 1600, 6400, 25600]
R = 100
rmse = []
for n in sizes:
    est = [mc_mean(g, np.random.default_rng(seed), n)
           for seed in range(R)]
    est = np.array(est)
    rmse.append(np.sqrt(np.mean((est - I_true) ** 2)))
print("rmse:", rmse)
slope = np.polyfit(np.log(sizes), np.log(rmse), 1)[0]
print("log-log meredekség:", slope)
```

Elvárt meredekség hozzávetőleg $-0{,}5$, mert a standard hiba $1/\sqrt{n}$ rendű. A tényleges érték $-0{,}6$ és $-0{,}4$ között alakulhat; a meredekség becslésének is van Monte Carlo-hibája. Ellenőrző

invariant: a számított RMSE-értékeknek csökkenő sorozatot kell alkotniuk, és a legnagyobb n-re az RMSE nagyjából az első érték tizenhatoda közelében kell lennie.

3. A hiba az, hogy minden n-hez ugyanazt a seed 7-et használjuk, ezért az ábra egyetlen n szerinti realizációt mutat, nem az eljárás hibaeloszlását. A seed rögzítése reprodukálhatóságot ad, de a $|hiba|$ görbéje ettől még véletlen; más seeddel más képet kaphatunk. A javítás: minden n-hez több különböző seeddel ismételjünk, és a hibák RMSE-ét vagy szórását rajzoljuk. Például:

```
rmse = []
for n in sizes:
    est = []
    for seed in range(200):
        x = np.random.default_rng(seed).random(n)
        est.append(np.mean(np.sin(x)))
    est = np.array(est)
    rmse.append(np.sqrt(np.mean((est - exact) ** 2)))
plt.plot(sizes, rmse)
```

4. Lehetséges megoldás:

```
import numpy as np
from scipy import stats

I_true = 1.0
n = 1000
R = 500
x = np.vstack([
    np.random.default_rng(seed).normal(size=n)
    for seed in range(R)
])                              # shape: (R, n)
estimates = np.mean(x ** 2, axis=1)   # shape: (R,)
rmse = np.sqrt(np.mean((estimates - I_true) ** 2))
single_se = np.std(x[0] ** 2, ddof=1) / np.sqrt(n)
print("RMSE:", rmse)
print("egy minta SE:", single_se)
print("elméleti SE:", np.sqrt(2.0 / n))
print("stats kontroll:", stats.norm.expect(lambda z: z ** 2))
```

Az elméleti szórás: $Var(X^2) = E[X^4] - (E[X^2])^2 = 3 - 1 = 2$, tehát $SE = \sqrt{2/n} \approx 0{,}0447$. A futás RMSE-je körülbelül 0,04–0,05; az egy minta $\widehat{SE}$-je is hasonló. A korlát: a normális farka miatt az X^2 eloszlása ferde, így a minta szórása lassabban stabilizálódik, mint szimmetrikus kis szórású esetben; de mivel a variancia véges, a Monte Carlo-becslés továbbra is konzisztens.

12.3.7 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés: egyetlen seed abszolút hibagörbéjét a módszer hibájának tekintjük. A seed 7-es görbe simának és csökkenőnek tűnhet, de egy másik seed más, akár nem monoton görbét ad. Az egyetlen realizáció nem mutatja a bizonytalanságot. Hasonlóan, a 11.3. alfejezet numerikus gradiens-ellenőrzéséhez hasonlóan a becsült mennyiség bizonytalanságát a módszer bizonytalanságától el kell választani.

Önellenőrző kérdések: - Ha a mintaméretet 25-szörösére növelem, hányad részére csökken a standard hiba? *Válasz: körülbelül ötödére, mert* $1/\sqrt{25} = 1/5$. - A seed megadása garantálja-e, hogy a becslés közelebb kerül a pontos értékhez? *Nem; csak reprodukálható folyamot ad.* - Mi a különbség $E[g(X)]$ és $g(E[X])$ között? *Az első a függvényértékek súlyozott átlaga, a második a várható értéken kiértékelt függvény; általában nem egyenlők, például a 4. feladatnál* $E[X^2] = 1$, *de* $(E[X])^2 = 0$.

12.3.8 Rövid összefoglaló

A várható érték a sűrűségfüggvény szerint súlyozott integrál; a Monte Carlo-becslés ezt véletlen mintából vett átlaggal közelíti. A `default_rng(seed)` reprodukálható pszeudorandom folyamot ad. A becslés standard hibája a minta szórása osztva $\sqrt{n}$-nel, így a tipikus hiba $1/\sqrt{n}$ arányban csökken. Egyetlen seed hibagörbéje csak egy realizáció; a módszer bizonytalanságát több független seedből számolt RMSE vagy becsült SE mutatja. A következő ellenőrző pont ezt a hibakezelést egy veszteségfüggvény teljes értelmezéséhez kapcsolja.

12.4 III. ellenőrző pont: egy veszteségfüggvény teljes értelmezése

> **Belépő · [BŐVÍTŐ MODUL] · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy**
>
> Előfeltétel: 12.3. alfejezet · Célkészség: III. ellenőrző pont: egy veszteségfüggvény teljes értelmezése · Kimenet: működő kód és a labor ellenőrző tesztje.

Ez a III. ellenőrző pont nem vezet be új statisztikai fogalmat. A 9–11. fejezet jelölés-, tömb-, gradiens- és optimalizálási eszköztárát kapcsolja össze a 12.1. alfejezet–12.3. alfejezet integrál-, sűrűség- és Monte Carlo fogalmaival egyetlen veszteségfüggvényen. A 8.4. alfejezetben a feature-táblát kontrollösszeggel és keret-összehasonlítással zártuk; itt ugyanazt a szerződést alkalmazzuk matematikai függvényre: legyen kézi invariáns, numerikus integrál, gradiensellenőrzés és Monte Carlo kontroll.

12.4.1 Cél és konkrét helyzet

A Northstar kezdőprojekt rendelésszintű kérdése: a rendelés pillanatában becsülni a kézbesítési időt a távolságból. Ez az ellenőrző pont egy szándékosan mesterséges, zajmentes sokaságot használ:

- megfigyelési egység: egy rendelés, amelyet itt egy x standardizált távolsággal és egy y várható kézbesítési idővel jellemzünk;
- bemenet: X sűrűsége $f_X(x) = 1$ a $(0, 1)$ intervallumon, azon kívül 0;
- igaz kapcsolat: $Y = 2 + 3X$, ahol x dimenziótlan, y nap;
- modellcsalád: $\hat{y}_w(x) = a + bx$, azaz $w = (a, b)^\top$;
- veszteségfüggvény: a négyzetes hiba,

$$R(w) = \mathbb{E}\big[(Y - \hat{y}_w(X))^2\big]\,,$$

  amelyet integrállal kiszámítunk, majd Monte Carlóval is közelítünk;
- elvárt ellenőrzés: kézi számított értékek, numerikus integrál, központi differenciás gradiens-ellenőrzés, BFGS-optimum és seedelt Monte Carlo átlag.

Mielőtt bármit futtatnál, adj előzetes becslést: a modellcsalád tartalmazza a $2 + 3x$ egyenest, ezért a minimum várható hiba 0, az optimum pedig $w^* = (2, 3)^\top$. A $w = (0, 0)^\top$ paraméternél a veszteség

$$R(0,0) = \int_0^1 (2 + 3x)^2\, dx = 4 + 6 + 3 = 13,$$

tehát a kézi kontrollérték 13. Ezt az alábbi kódnak függetlenül vissza kell adnia.

12.4.2 Fogalmi magyarázat: veszteség, sűrűség, gradiens

A korábbi fejezetekből már ismertek a tömbök, az integrál, a derivált és a Monte Carlo átlag. Itt csak a kapcsolatukat írjuk fel egy veszteségfüggvényre.

A predikció vektoros alakban:

$$\phi(x) = \begin{bmatrix} 1 \\ x \end{bmatrix} \in \mathbb{R}^2, \qquad w = \begin{bmatrix} a \\ b \end{bmatrix} \in \mathbb{R}^2, \qquad \hat{y}_w(x) = w^\top \phi(x) = a + bx.$$

Egy megfigyelés hibája $y - \hat{y}_w(x)$, a vesztesége pedig $[y - \hat{y}_w(x)]^2$. A sokasági kockázat a sűrűséggel súlyozott integrál:

$$R(w) = \int_0^1 (2 + 3x - a - bx)^2 \underbrace{1}_{f_X(x)} dx.$$

A négyzetet kifejtve:

$$\begin{aligned} R(a,b) &= (2-a)^2 + (2-a)(3-b) + \frac{(3-b)^2}{3} \\ &= a^2 + ab + \frac{b^2}{3} - 7a - 4b + 13. \end{aligned}$$

A gradiens a két parciális derivált:

$$\nabla R(w) = \begin{bmatrix} \frac{\partial R}{\partial a} \\ \frac{\partial R}{\partial b} \end{bmatrix} = \begin{bmatrix} 2a + b - 7 \\ a + \frac{2}{3}b - 4 \end{bmatrix}.$$

Az optimum a stacionárius pont: $\nabla R(w) = 0$ megoldása $a = 2$, $b = 3$, és mivel R nemnegatív négyzetes kifejezés összege, ez globális minimum.

A shape-szerződés:

Objektum	Alak	Egység / szerep
$\phi(x)$	(2,)	jellemzővektor
w	(2,)	a nap, b nap
$\hat{y}_w(x)$	skalár	előrejelzett nap
egy L_i veszteség	skalár	nap^2
$R(w)$	skalár	nap^2
$\nabla R(w)$	(2,)	nap / paraméterkomponens

A Monte Carlo közelítés $x_1, \ldots, x_N$ független Uniform$(0,1)$ mintát használ, $y_i = 2 + 3x_i$ értékkel:

$$\hat{R}_N(w) = \frac{1}{N} \sum_{i=1}^{N} (y_i - a - bx_i)^2.$$

A becslés standard hibája a mintabeli veszteségek s_L szórásából:

$$SE = \frac{s_L}{\sqrt{N}}, \qquad s_L = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(L_i - \bar{L})^2}.$$

A $\sqrt{N}$ a 12.3. alfejezetből ismert tulajdonság: a Monte Carlo hiba tipikus nagysága a minta elemszámának négyzetgyökével csökken.

A gradiensellenőrzés a 11.1. alfejezet numerikus deriváltját használja, de most komponensenként és kétoldali differenciával:

$$\frac{\partial R}{\partial w_j} \approx \frac{R(w + he_j) - R(w - he_j)}{2h}, \qquad h = 10^{-6}.$$

12.4.3 Teljes minimális munkapélda

A következő szkript a `lab/checkpoints/cp3/risk_checkpoint.py` fájlban is megtalálható. A kimeneteket numpy 2.5.4, scipy 1.18.1 és matplotlib 3.10 alatt ellenőriztük (2026-10-29); a `tests/test_checkpoint3.py` a loss-, gradiens- és Monte Carlo invariánsokat futtatja.

```
"""
s048_risk_checkpoint.py
III. ellenőrző pont: veszteségfüggvény teljes numerikus értelmezése.

A kimeneti értékek a képletekből számított, ellenőrzött eredmények; a
tests/test_checkpoint3.py a loss-, gradiens- és Monte Carlo invariánsokat
futtatja.
"""
import numpy as np
from scipy.integrate import quad
from scipy.optimize import minimize
import matplotlib.pyplot as plt

rng = np.random.default_rng(20261020)

def risk_closed(w):
    a, b = w
    return a*a + a*b + b*b/3.0 - 7.0*a - 4.0*b + 13.0

def risk_quad(w):
    a, b = w
    return quad(lambda x: (2.0 + 3.0*x - a - b*x)**2, 0.0, 1.0)[0]

def grad_closed(w):
    a, b = w
    return np.array([2.0*a + b - 7.0, a + 2.0*b/3.0 - 4.0])

def numerical_gradient(fun, w, h=1e-6):
    g = np.zeros_like(w, dtype=float)
    for j in range(w.size):
        e = np.zeros_like(w, dtype=float)
        e[j] = 1.0
        g[j] = (fun(w + h*e) - fun(w - h*e)) / (2.0*h)
    return g

def monte_carlo_loss(w, rng, n):
    x = rng.uniform(0.0, 1.0, n)        # shape (n,)
    y = 2.0 + 3.0*x                     # shape (n,) nap
    losses = (y - w[0] - w[1]*x)**2     # shape (n,) nap^2
    return losses.mean(), losses.std(ddof=1) / np.sqrt(n)
```

```
w00 = np.array([0.0, 0.0])
wopt = np.array([2.0, 3.0])

# Ellenőrzött: 13,    13.0,   [-7. -4.],   [-7.000000 -4.000000]
print("risk_closed(0,0) =", risk_closed(w00))
print("risk_quad(0,0)   =", risk_quad(w00))
print("grad_closed(0,0) =", grad_closed(w00))
print("central grad      =", numerical_gradient(risk_closed, w00))

# Ellenőrzött: True (1e-12 tolerancia mellett)
print("quad-zart egyezes =", abs(risk_closed(w00) - risk_quad(w00)) < 1e-12)

# Ellenőrzött: True (1e-6 abszolút és relatív tolerancia mellett)
print("gradiens-egyezes  =",
      np.allclose(numerical_gradient(risk_closed, w00),
                  grad_closed(w00), atol=1e-6, rtol=1e-6))

res = minimize(risk_closed, w00, jac=grad_closed, method="BFGS",
               options={"gtol": 1e-10})

# Ellenőrzött: x = [2. 3.], fun = 0.0, success = True
print("BFGS x, fun, success =", res.x, res.fun, res.success)

n = 100_000
mean0, se0 = monte_carlo_loss(w00, rng, n)
# Ellenőrzött: mean0   13,  2*se0   0.039
print(f"MC at (0,0): {mean0:.4f} +/- {2*se0:.4f} (2 SE), populacios R=13")
meanp, sep = monte_carlo_loss(wopt, rng, n)
# Ellenőrzött: meanp = 0,  sep = 0
print(f"MC at (2,3): {meanp:.4f} +/- {2*sep:.4f}")

# Ábra: R szintvonalai és az optimum
A = np.linspace(-1.0, 5.0, 200)
B = np.linspace(-2.0, 7.0, 200)
A, B = np.meshgrid(A, B)
Z = A*A + A*B + B*B/3.0 - 7.0*A - 4.0*B + 13.0

plt.contourf(A, B, Z, levels=20, cmap="viridis")
plt.colorbar(label="R(a,b)  [nap^2]")
plt.plot([2.0], [3.0], "r*", markersize=12, label="optimum (2,3)")
plt.plot([0.0], [0.0], "wo", label="kézi kontroll (0,0)")
plt.xlabel("a (nap)")
plt.ylabel("b (nap / távolságegység)")
plt.title("III. ellenőrző pont: négyzetes veszteség")
plt.legend()
plt.tight_layout()
plt.savefig("s048_risk_contour.png", dpi=150)
```

Futtatás a lab projekt gyökeréből:

```
# Linux/macOS, a könyvtár a lab projekt gyökere:
cd lab
uv run --locked python notebooks/s048_risk_checkpoint.py

# Windows PowerShell, a könyvtár a lab projekt gyökere:
cd lab
uv run --locked python notebooks\s048_risk_checkpoint.py
```

A `--locked` kapcsoló a korábbi fejezetben készített `uv.lock` egyezését ellenőrzi. Ha még nincs feloldott lock, először `uv lock`, majd `uv sync` kell.

Az ellenőrzött futás kimenete (Linux, Python 3.12.3, numpy 2.5.4, scipy 1.18.1, matplotlib 3.10, `MPLBACKEND=Agg`):

```
risk_closed(0,0) = 13.0
risk_quad(0,0)   = 13.000000000000002
grad_closed(0,0) = [-7. -4.]
central grad     = [-7. -4.]
quad-zart egyezes = True
gradiens-egyezes  = True
BFGS x, fun, success = [2. 3.] 1.7763568394002505e-15 True
MC at (0,0): 12.9932 +/- 0.0385 (2 SE), populacios R=13
MC at (2,3): 0.0000 +/- 0.0000
```

A `risk_quad` és a zárt alak eltérése `2e-15`, a BFGS a `[2, 3]` pontot adja `1,78e-15` függvényértékkel; a Monte Carlo `(0,0)` pontban `12,9932`, a 2 SE sáv `0,0385`, ami egyezik a lent kiszámított `0,019` SE-vel. A `(2,3)` pontban a numerikus érték `3,5e-32`, tehát a `0.0000` formázott kimenet mögött nem pontosan nulla áll, hanem lebegőpontos maradék.

12.4.4 Eredmény értelmezése

A kézi kontroll egyetlen számpárja: $R(0,0) = 13$. A numerikus `quad` eredménynek 10^{-12} hibahatáron belül egyeznie kell vele, mert a zárt alak egzakt, a `quad` pedig determinisztikus numerikus közelítés. A gradiens kézi értéke $\nabla R(0,0) = (-7,-4)^\top$; ezt a központi differencia $h = 10^{-6}$ lépéssel 10^{-6} toleranción belül közelíti.

A BFGS futás ugyanazt a `jac` gradiensfüggvényt kapja, mint amit ellenőriztünk, ezért az optimum ellenőrzése 11.4. alfejezet szellemében itt kétrétegű: a solver stacionárius pontot talál, és a stacionárius pont a zárt gradiense nulla. A várt eredmény $w^* = (2,3)^\top$, $R(w^*) = 0$.

A Monte Carlo kontrollnak nem az a célja, hogy „pontosan” eltalálja a 13-at; a 2 SE sáv mutatja a szimulációs bizonytalanságot. A jelen példában $R(0,0) = 13$ és a per-sample veszteség szórása

$$\mathbb{E}[(2+3X)^4] - 13^2 = 206{,}2 - 169 = 37{,}2,$$

ezért $s_L \approx 6{,}099$, és $N = 100\,000$ mellett $SE \approx 0{,}019$. Ha a futásod $13{,}00 \pm 0{,}04$ sávon belül van, az a kontroll szempontjából elfogadható. A $w = (2,3)$ pontban minden egyes veszteség pontosan 0, ezért ott a MC szórása is 0; ez nem statisztikai „bizonyítás”, hanem a számítás sajátossága.

A veszteségfüggvény optimalizálási sikere és a statisztikai prediktív érvényesség külön állítás. A szintetikus, zajmentes példa analitikus invariánsai a numerikus implementációt ellenőrzik; valós generalizációs következtetést nem igazolnak.

12.4.5 Feladatok

1. feladat — kézi értelmezés. Tekintsük a $w = (1,0)^\top$ pontot a fenti veszteségfüggvényen.

a) Add meg a $\hat{y}_w(x)$, a hiba és a veszteség kifejezését x függvényeként.
b) Számold ki kézzel az $R(1,0)$ integrált, és ellenőrizd a zárt formulával.
c) Számold ki $\nabla R(1,0)$ mindkét komponensét, és foglald szóba, mit jelent az előjelük.
d) Miért igaz, hogy $R(1,0) > R(2,3)$?

2. feladat — implementálás és gradiensellenőrzés. Készíts függvényeket a fenti példához, majd:

a) Végezz központi differenciás gradiensellenőrzést a $w = (1, 2)^\top$ pontban $h = 10^{-5}$ lépéssel. Az analitikus gradiens

$$\nabla R(1, 2) = (-3, -\tfrac{5}{3})^\top.$$

b) Futtasd a BFGS-optimalizálót $w_0 = (0, 0)$ kezdőpontból.
c) Futtass Monte Carlo becslést $N = 50\,000$, `seed=7` mellett a $w = (0, 0)$ és a $w^* = (2, 3)$ pontokban. A 2 SE sávot is írd ki.

Milyen toleranciákat adsz meg, és miért?

3. feladat — hibadiagnózis. Egy kolléga az alábbi függvényt használja a Monte Carlo bizonytalanság becslésére:

```
def monte_carlo_se_bad(w, rng, n):
    x = rng.uniform(0.0, 1.0, n)
    y = 2.0 + 3.0*x
    losses = (y - w[0] - w[1]*x) ** 2
    return losses.mean(), losses.std() / n   # figyelj a nevezőre!
```

A kolléga $N = 100\,000$ mintánál a $w = (0, 0)$ pontban ezt kapja: „átlag 13,000, 2 SE 0,0001”, és ebből arra következtet, hogy a sokasági veszteség pontosan 13 lett beszorítva egy rendkívül szűk sávba.

a) Mi a hiba a standard hiba képletében?
b) Mekkora az elméletileg várt SE a $s_L \approx 6{,}099$ közelítéssel?
c) Javítsd a függvényt, és fogalmazd meg, mit állíthatunk a MC átlagról és a 2 SE sávról.

4. feladat — megváltoztatott feltétel. Változtassuk meg a sokasági igaz kapcsolatot $Y = 1 + 4X$-re, miközben X továbbra is Uniform$(0, 1)$.

a) Vezesd le az új zárt veszteséget $R(a, b)$-re és a $\nabla R(a, b)$ gradienst.
b) Oldd meg a $\nabla R = 0$ egyenletet; mit kapsz optimumként?
c) Futtass Monte Carlo kontrollt $N = 50\,000$, `seed=42` mellett a $w = (0, 0)$ és az optimum pontokban. Milyen elméletileg várt értékeket hasonlítasz össze?

12.4.6 Megoldások és indoklás

1. megoldás.

a) $\hat{y}_w(x) = 1$; a hiba $2 + 3x - 1 = 1 + 3x$; a veszteség $(1 + 3x)^2$.

b) $R(1, 0) = \int_0^1 (1+3x)^2\,dx = \int_0^1 (1+6x+9x^2)\,dx = 1+3+3 = 7$. A zárt formula $a = 1, b = 0$ mellett $1^2 - 7 \cdot 1 + 13 = 7$.

c) $\nabla R(1, 0) = (2 \cdot 1 + 0 - 7,\ 1 - 4)^\top = (-5, -3)^\top$. Az első komponens azt mondja, hogy a kicsiny növelése csökkenti a veszteséget; a második hasonlóan csökkentést jelez b növelésére. Ez konzisztens azzal, hogy $a = 1 < 2$, $b = 0 < 3$, tehát a jelenlegi pont az optimum alatt van.

d) $R(1, 0) = 7 > 0 = R(2, 3)$, mert a $w^* = (2, 3)$ pontban a predikció minden x-re megegyezik az igazi $2 + 3x$ értékkel.

2. megoldás.

A gradiensellenőrzés várt kimenete:

```
w12 = np.array([1.0, 2.0])
ng = numerical_gradient(risk_closed, w12, h=1e-5)
ag = grad_closed(w12)
# ng  [-3.0, -1.6666667]; ag = [-3.0, -1.6666667]
```

A `np.allclose(ng, ag, atol=1e-6, rtol=1e-6)` elvártan `True`. A 10^{-6} tolerancia indoklása: a kétoldali differencia hibatagjai $O(h^2)$ és kerekítési hiba; $h = 10^{-5}$-nél ez a jelen függvényskálán jóval a tolerancia alatt van.

A BFGS futás elvártan a $[2, 3]$ pontot adja vissza, $f \approx 0$ értékkel. A `gtol=1e-10` a gradiensnormára vonatkozó megállási feltétel; a stacionárius pont ellenőrzése tehát a megállási ok, nem a kimeneti egyetlen szám.

A MC `seed=7`, $N = 50\,000$ mellett elméletileg várt:

- $w = (0, 0)$: átlag ≈ 13, $SE \approx s_L/\sqrt{50\,000} \approx 6{,}099/223{,}6 \approx 0{,}0273$, 2 SE $\approx 0{,}055$;
- $w^* = (2, 3)$: átlag 0, SE 0.

A bemeneteket a feladat mind megadja; a seedelt generátor reprodukálható, de numerikus könyvtárverziótól függő apró eltérés nem sérti a MC sávok stabilitását.

3. megoldás.

a) A kód `losses.std() / n` értékkel oszt, nem `np.sqrt(n)` értékkel. Ezzel a bizonytalanságot $\sqrt{N}$-szeresével alulbecsli.

b) $N = 100\,000$, $s_L \approx 6{,}099$ mellett $SE \approx 6{,}099/316{,}2 \approx 0{,}0193$. A kolléga által kiírt 0,0001 hibasáv tehát irreálisan szűk; a valós MC bizonytalanság nagyságrendileg 0,04 a 2 SE sávban.

c) Helyes függvény:

```
def monte_carlo_se_fixed(w, rng, n):
    x = rng.uniform(0.0, 1.0, n)
    y = 2.0 + 3.0*x
    losses = (y - w[0] - w[1]*x) ** 2
    return losses.mean(), losses.std(ddof=1) / np.sqrt(n)
```

A ± 2 SE sáv közelítőleg az ismételt MC futások 95%-ában tartalmazza a sokasági értéket; ez heurisztikus, a formális indoklást később a normális közelítés és a centrális határeloszlás adja. A sáv nem jelenti azt, hogy a sokasági érték a becsült átlagtól maximálisan ennyire van.

4. megoldás.

a) $Y = 1 + 4X$, ezért $R = \int_0^1 (1 + 4x - a - bx)^2\,dx$. Kifejtve:

$$R(a, b) = a^2 + ab + \frac{b^2}{3} - 6a - \frac{11}{3}b + \frac{31}{3}.$$

A gradiens:

$$\nabla R(a, b) = \begin{bmatrix} 2a + b - 6 \\ a + \frac{2}{3}b - \frac{11}{3} \end{bmatrix}.$$

b) A $\nabla R = 0$ egyenletrendszer megoldása $a = 1$, $b = 4$, tehát $w^* = (1, 4)^\top$. Mivel R nemnegatív négyzetes forma, ez globális minimum, és $R(1, 4) = 0$.

c) Elméletileg várt kézi értékek:

- $R(0,0) = \frac{31}{3} \approx 10{,}333$;
- $\nabla R(0,0) = (-6, -\frac{11}{3})^\top$;
- MC $w = (0,0)$-ban $\approx 10{,}333$, 2 SE sávval;
- MC az optimumban 0, mert a modell minden mintán pontosan illeszkedik.

A feladat ellenőrző invariantja tehát az R kézi értéke, a gradiens komponensei és az optimum koordinátái.

12.4.7 Gyakori tévedés és ellenpélda

„Ha egy ponton numerikusan egyezik a gradiens, akkor a gradiensfüggvény mindenütt rendben van." A numerikus egyezés csak a vizsgált tartományra és toleranciára vonatkozik. A központi differencia pontossága a függvény nagyságától és a véges pontosságtól függ: a kerekítési hibakorlát nagyságrendileg $\epsilon \cdot |R|/h$, ahol $\epsilon \approx 2{,}2 \cdot 10^{-16}$.

Ellenpéldaként próbáld ki a $w = (1000, 1000)^\top$ pontot, ahol $R \approx 2{,}3 \cdot 10^6$, a zárt gradiens

$$\nabla R(1000, 1000) \approx (2993,\ 1627)^\top.$$

Ha valaki itt $h = 10^{-12}$ lépéssel ellenőriz, a kerekítési hibakorlát nagyobb lehet, mint a gradiens maga, ezért a központi differencia nem egyezik a zárt deriválttal a megszokott 10^{-6} toleranción belül. A javítás nem a gradiensképlet cseréje, hanem a skálázott, stabilabb ellenőrzés vagy az analitikus gradiens használata. Ugyanígy a Monte Carlo kontroll sem igazol „minden w-re" semmit: egyetlen ponton mért átlag és SE csak azt a pontot és azt a generátorfeltételt ellenőrzi.

12.4.8 Önellenőrzés

- Ki tudod számolni kézzel $R(0,0)$-t és $\nabla R(0,0)$-t?
- A `quad` és a zárt formula egyezése teljesül-e 10^{-12} hibahatáron belül?
- A gradiensellenőrzés mindkét komponensre, kétoldali differenciával történt-e?
- A BFGS megoldása után a gradiens nulla-e a kapott pontban?
- A Monte Carlo futás megadott-e seedet, N-t és SE-t?
- A kapott sávot MC bizonytalanságként, nem „pontos elméleti" eredményként értelmezed-e?

12.4.9 Rövid összefoglaló

A III. ellenőrző pont egyetlen veszteségfüggvényen zárta le a matematikai eszköztárat. A négyzetes hiba sűrűséggel súlyozott integrálként írható fel, zárt formulája és gradiense kézzel ellenőrizhető. A `scipy.integrate.quad`, a központi differencia, a BFGS és a seedelt Monte Carlo ugyanazt az optimumot támasztja alá: $(2, 3)$, minimum 0. A numerikus egyezés azonban mindig csak a vizsgált pontra, tartományra és toleranciára vonatkozik. A MC bizonytalanságot $\sqrt{N}$-es SE-vel kell becsülni. Következő lépésként a 13.1. alfejezet a minta és az adatfelvétel fogalmát vezeti be, ahol a sokasági $R(w)$ helyébe a mintából számolt átlag és annak bizonytalansága lép.

13 Leíró statisztika és feltáró elemzés torzításokkal együtt

13.1 Sokaság, minta és adatfelvételi folyamat

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, matplotlib

Előfeltétel: 12.4. alfejezet · Célkészség: Sokaság, minta és adatfelvételi folyamat · Kimenet: működő kód és a labor ellenőrző tesztje.

13.1.1 Cél és konkrét helyzet

A Northstar késési modell rendelésenként becsli a 7 napot meghaladó kézbesítést. Mielőtt modellt építenénk, tisztázni kell, kik kerültek egyáltalán a fájlba. A bemenet a mellékelt lab `customers.csv`, `orders.csv` és `order_items.csv` állománya; a mai megfigyelési egység a rendelés, mert a `target_late` célváltozó rendelésszintű. A kimenet egy *adatkeletkezési lap*: megnevezi, milyen lépéseken keresztül jutott egy sor a `orders.csv`-be, és felsorol legalább három populációt, amelyre abból nem általánosíthatunk. Az ellenőrzési feltétel: minden állításhoz meg kell tudni nevezni a bekerülési feltételt, és ha az összefügghet a kimenettel, jelölni kell.

Induló becslésként fogalmazzuk meg a kérdést: vajon a `priority_flag=1` rendelések kosárátlaga egyenlő-e az összes rendelés átlagával? A válasz előzetesen az, hogy nem tudjuk, és nem is a konkrét szám a lényeg: a flaggel szűrt halmaz nem véletlen minta, tehát az átlaga elvileg is eltérhet a sokasági átlagtól. Ezt az alfejezet végére precízen meg tudjuk indokolni.

13.1.2 Sokaság, minta és mintavételi keret

A *sokaság* (*population*) az az U halmaz, amelyről állítani szeretnénk. Jele legyen U, elemszáma N. A *minta* (*sample*) a ténylegesen megfigyelt $S \subseteq U$ részhalmaz, elemszáma n. A sokasági átlag és a mintaátlag:

$$\mu_U = \frac{1}{N}\sum_{i \in U} x_i, \qquad \bar{x}_S = \frac{1}{n}\sum_{i \in S} x_i.$$

A minta nem attól minta, hogy „van valamennyi sorunk”, hanem attól, hogy van egy kiválasztási mechanizmus. A *mintavételi keret* (*sampling frame*) az a lista vagy eljárás, amelyből a mintát ténylegesen választjuk; jelöljük F-fel. A célpopuláció és a keret különbsége a lefedetlen rész: az $U \setminus F$ egységek elvileg sem kerülhetnek be.

Kis kézi példa. Legyen $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$, a sokasági átlag $\mu_U = 5{,}5$. Tegyük fel, hogy a keretbe csak a páros számok kerültek be, mert például csak ők regisztráltak: $F = \{2, 4, 6, 8, 10\}$. Ha az egész keretet megfigyeljük, az átlag 6,0, nem 5,5. Ez nem mintavételi zaj, hanem lefedettségi hiba: a páratlanok soha nem juthattak be.

A fenti lépéssor jól mutatja, miért fontos külön kezelni a sokaságot, a keretet és a mintát. Egy szoftverprojektben a „populáció” gyakran nem tárolt fogalom, hanem döntési kérdés: minden potenciális ügyfél? minden jövőbeli rendelés? minden rendelés egy adott piacon? Amíg ezt nem rögzítjük, a leíró statisztika sem értelmezhető általánosításként.

13.1.3 Szelekciós és túlélési torzítás

A *szelekciós torzítás* (*selection bias*) akkor lép fel, ha a mintába kerülési valószínűség összefügg a vizsgált változóval. Legyen $\pi_i = P(i \in S)$ az i egység bekerülési valószínűsége. Ha az egységeket egymástól függetlenül π_i valószínűséggel választjuk, a kiválasztottak átlaga a π-vel súlyozott sokasági átlag körül ingadozik:

$$\bar{x}_S \approx \frac{\sum_{i \in U} \pi_i x_i}{\sum_{i \in U} \pi_i}.$$

Ha π_i minden egységre azonos konstans, a kifejezés μ_U-t ad. Ha viszont π_i növekszik x_i-vel, a mintaátlag várhatóan felfelé torzít. A pontos elméleti levezetést itt nem végezzük el; a tanulság az, hogy a torzítás iránya a π_i és x_i együttmozgásából adódik. A fenti 10 elemű példában a keretbe kerülés π_i értéke a páratlanokra 0, a párosakra 1; ez a legdurvább szelekció.

A *túlélési torzítás* (*survivorship bias*) ennek speciális esete: a megfigyelhető halmaz csak azokat tartalmazza, akik egy folyamatot „túléltek”. Képzeljünk el ezer kosarat, amelyből kétszázból lesz leadott rendelés, és száznyolcvanból lesz sikeres kézbesítés. Ha a cél a kézbesítési idő elemzése, a száznyolcvan megfigyelt sor csak a túlélő rendeléseket tartalmazza; a lemondott, törölt vagy nem leadott rendelések kiestek. Ha a törlés valószínűsége összefügg a sürgősséggel vagy a kosárértékkel, a megfigyelt átlag torzul.

A Northstar `orders.csv`-ben a kézbesítések a generált időablak végén teljesen megfigyeltek, tehát a kézbesítési időre nincs cenzorálás. A túlélési torzítás mégis jelen van egy korábbi szinten: a fájl csak a leadott rendeléseket tartalmazza. A kosarat elhagyó, a rendelést lemondó vagy soha be nem jelentkező ügyfél hiányzik. Ezért a `target_late` aránya a leadott rendelések sokaságára vonatkozik, nem a vásárlási szándékú kosarakra.

A két hiba közös vonása, hogy a minta méretének növelése nem szünteti meg. A mintavételi zaj $O(1/\sqrt{n})$-nel csökken — ezt a Monte Carlo-hibánál láttuk 12.4. alfejezet —, de a szelekciós hiba

$$D = \frac{\sum_{i \in U} \pi_i x_i}{\sum_{i \in U} \pi_i} - \mu_U$$

nem függ n-től, ha a kiválasztási mechanizmus rögzített marad. A nagyobb adathalmaz tehát pontosabban méri ugyanazt a torzított mennyiséget.

13.1.4 Minimális munkapélda a torzításra

A mechanizmust kis szintetikus adaton is lefuttathatjuk. A generáló seed legyen `20261020`, az értékek exponenciális eloszlásból származzanak; a bekerülési valószínűség növekedjen az értékkel. A kód:

```python
import numpy as np

rng = np.random.default_rng(20261020)
N = 2000
x = rng.exponential(50.0, size=N)        # sokasági értékek
p = 0.1 + 0.8 * (x / x.max())            # értékfüggő bekerülés
in_sample = rng.random(N) < p
sample = x[in_sample]

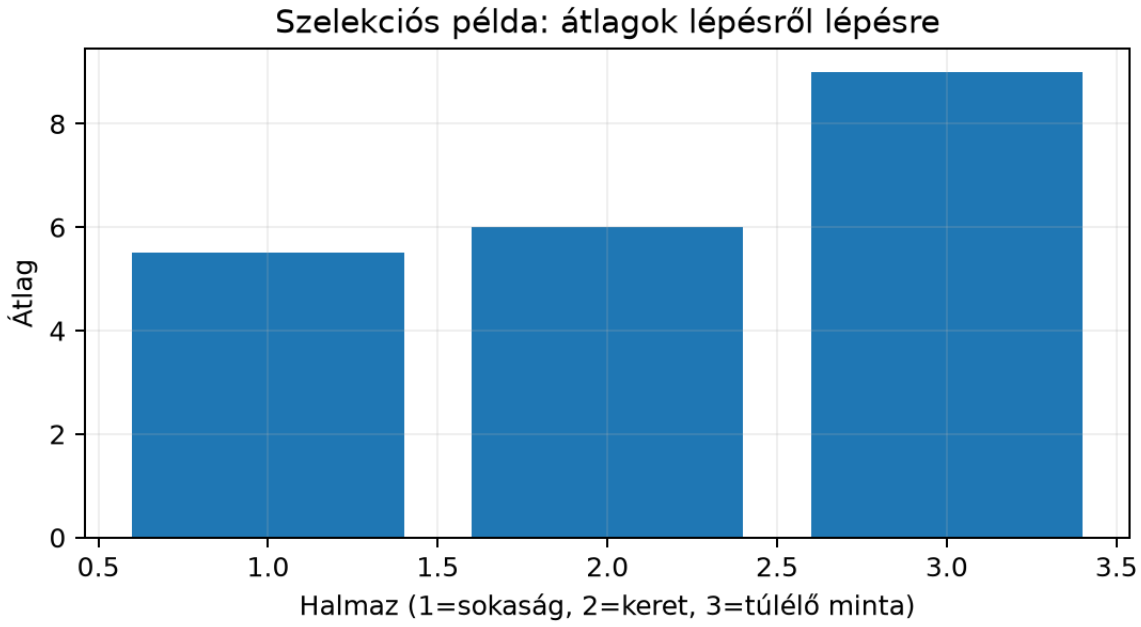
print(f"n = {in_sample.sum()}")
print(f"sokasági átlag = {x.mean():.2f}")
print(f"mintaátlag = {sample.mean():.2f}")
print(f"bekerülési valószínűség átlaga = {p.mean():.3f}")
```

A kód kimenete a seedtől és a NumPy-verziótól függ, ezért itt nem állítunk be számot; a futás helyben ellenőrizhető. A mechanizmus iránya viszont a képletből is látszik: a kisebb értékek kisebb, a nagyobbak nagyobb valószínűséggel kerülnek be, tehát a mintaátlag várhatóan meghaladja a sokasági átlagot. A numerikus reprodukálhatóságot a seed biztosítja, de a bitazonosságot csak azonos NumPy-verzión várhatjuk.

A kézi kontroll a tíz elemű példa:

Halmaz	Elemek	Átlag
Sokaság U	1, 2, 3, 4, 5, 6, 7, 8, 9, 10	5,5
Keret F	2, 4, 6, 8, 10	6,0
Túlélő minta	8, 10	9,0

A keret átlaga azért nagyobb a sokaságinál, mert a páratlanok lemaradtak; a túlélő átlag azért nagyobb a kereténél, mert a folyamat a kisebb értékeket kiejtette. A két eltérés egyike sem a véletlen miatt van.



13.1. ábra: Szelekciós példa: átlagok lépésről lépésre - Számított számpélda, nem mérés. A 9.0 a 8 és 10 átlaga.

13.1.4.1 Szintetikus torzított minta ismert referenciahatással

A szelekciós torzítás akkor mutatható meg tisztán, ha a sokaság összetétele és a csoporton belüli arányok ismertek. Legyen a sokaság 60% A és 40% B, a konverziós arány A-ban 0.10, B-ben 0.30. A valódi összesített arány $0.6 \cdot 0.10 + 0.4 \cdot 0.30 = 0.18$. Ha a naplózás csak a látogatók 90%-át rögzíti A-ból és 10%-át B-ből, a minta összetétele eltolódik:

Mennyiség	Érték	Magyarázat
valódi összesített arány	0.180	$0.6 \cdot 0.10 + 0.4 \cdot 0.30$
torzított minta összetétele	90% A, 10% B	a naplózás hiányos
torzított összesített arány	0.120	$0.9 \cdot 0.10 + 0.1 \cdot 0.30$
torzítás	−0.060	a minta alulbecsüli az összesített arányt
csoporton belüli különbség	0.200	B − A, a torzításban változatlan
súlyozott korrekció	0.180	ismert sokasági súlyokkal

A csoporton belüli hatás tehát változatlan, az összesített arány viszont torzított, mert a mintaösszetétel más, mint a sokaságé. A súlyozás csak akkor korrigál, ha a sokasági súlyok ismertek; ha a mintakeretből egy csoport teljesen hiányzik, a súlyozás sem tudja visszahozni.

```
POPULATION = {"A": 0.60, "B": 0.40}
RATES = {"A": 0.10, "B": 0.30}
biased = {"A": 0.90, "B": 0.10}

true_overall = sum(POPULATION[g] * RATES[g] for g in POPULATION)
biased_overall = sum(biased[g] * RATES[g] for g in biased)
weighted = sum(POPULATION[g] * RATES[g] for g in POPULATION)

assert abs(true_overall - 0.18) < 1e-12
assert abs(biased_overall - 0.12) < 1e-12
assert abs((RATES["B"] - RATES["A"]) - 0.20) < 1e-12
assert abs(weighted - true_overall) < 1e-12
```

A `lab/sampling_bias.py` `overall()`, `biased_weights()`, `weighted_estimate()` és `report()` függvényei ezt a számítást adják; a `tests/test_sampling_bias.py` a valódi 0.18, a torzított 0.12, a 0.20-as csoporthatás és a súlyozott korrekció egyezését ellenőrzi.

13.1.5 Northstar labor: ki van a fájlban?

Rajzoljuk fel a Northstar rendelési adat keletkezési láncát a lab szerződése alapján:

1. Egy ügyfél regisztrál; rögzül a `signup_date`, a csatorna, a szegmens és a régió.
2. Az ügyfél a 2025 első 180 napját lefedő időablakban rendelést ad le. A rendelés pillanatában ismert mezők bekerülnek az `orders.csv`-be: `order_ts`, `promised_days=7`, `quantity_sum`, `item_rows`, `basket_minor`, `shipping_fee_minor`, `currency`, `distance_km`, `priority_flag`.
3. A rendelés tételei az `order_items.csv`-be kerülnek; a `basket_minor` a tételekből számolt kosárösszeg.
4. A kézbesítés után, utólag ismertté válik a `delivery_days`, az `actual_delivery_at` és a belőlük képzett `target_late`.
5. A tiszta `orders.csv` az időablak minden szimulált rendelését tartalmazza; a `orders_raw.csv` szándékos hibákkal van ellátva.

A betöltés és az ellenőrzés minimális kódja a `lab` munkakönyvtárból:

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
orders = pd.read_csv("data/orders.csv", parse_dates=["order_ts"])

print("rendelés sorok:", len(orders))
print("order_id egyedi:", orders["order_id"].is_unique)
print("rendelést leadó ügyfelek:", orders["customer_id"].nunique())
print("ügyfelek a törzsben:", len(customers))

orders_region = orders.merge(
    customers[["customer_id", "region"]], on="customer_id", how="left"
)
print("merge sorok:", len(orders_region))
print(orders_region["region"].value_counts())
```

Futtassuk a lab projektgyökeréből, ha a kódot például `eda_sampling.py` néven mentjük:

```
uv run python eda_sampling.py
```

A labor szerződése szerint a tiszta `orders.csv` alapmérete 2400 sor, a `customers.csv` 300 sor. A `merge` sorainak száma ezért 2400 marad, feltéve, hogy a `customer_id` egyedi a `customers` táblában; a `how="left"` megőrzi a rendelések számát, a hiányzó régió `NaN` lenne. A kód kimenetét itt nem futtattuk, ezért a pontos régióarányokat nem állítjuk mértnek.

13.1.6 Eredmény értelmezése

A szintetikus példában a mintaátlag eltérése szisztematikus: a nagy minta nem korrigálja, mert a bekerülési valószínűség maga függ a vizsgált értéktől. A tíz elemű kézi példa ugyanezt mutatja n-től függetlenül.

A Northstar laborban a régiómegoszlás a *rendeléseket* jellemzi, nem az ügyfélsokaságot. Ha egy régióban egy ügyfél átlagosan több rendelést ad le, a rendelésszám-súlyozott arány eltér az ügyfélszám-súlyozottól. Ezt a kettőt össze lehet hasonlítani:

```
customer_region = customers["region"].value_counts()
print("ügyféloldali megoszlás")
print(customer_region)
print("rendelésoldali megoszlás")
print(orders_region["region"].value_counts())
```

Az invariáns: az ügyféloldali `value_counts()` összege 300, a rendelésoldalié 2400. Ha a két megoszlás eltér, az nem bizonyíték semmilyen okra — csak azt, hogy a két elemzési szemcsézettség mást mér. A későbbi resamplingnál ezért lesz fontos az ügyfél mint független mintavételi egység, nem a rendelés.

A rendelés- és az ügyfélszintű populációt elkülönítő kijelentésekhez mindig jelöljük a mintavételi keretet és a bekerülés mechanizmusát. A szintetikus populációra igaz leíró állítás nem vihető át automatikusan valódi webshop ügyfeleire.

13.1.7 Feladatok

1. **Értelmezés.** A Northstar `orders.csv` alapján döntést kell hozni arról, mekkora lesz a késés aránya a jövőbeli rendeléseken. Nevezz meg három populációt, amelyre ebből az adathalmazból nem általánosíthatunk, és mindegyikhez nevezd meg a bekerülési hézagot.
2. **Implementálás.** Egészítsd ki a fenti labos kódot úgy, hogy a `region_counts` sorozatot CSV-be menti, és `assert` utasításokkal ellenőrzi a 2400-as sorösszeget, valamint azt, hogy nincs hiányzó régió. Az output mappát hozd létre, ha nem létezik.
3. **Hibakeresés.** Egy kolléga azt állítja: „Mivel 2400 rendelésünk van, a `priority_flag=1` rendelések 42%-os megfigyelt aránya a Northstar teljes rendelési sokaságára igaz.” Mi a hiba az állításban? Milyen feltétel mellett lenne védhető az általánosítás?
4. **Megváltozott feltétel.** Tegyük fel, hogy 2025. július 1-jén a Northstar megnyitja a `region="west"` csatornát. Használhatjuk-e a meglévő `north`, `central`, `south` rendelésekből becsült késési arányt a `west` rendelésekre? Fogalmazd meg, milyen adatfelvételi feltételt kellene ellenőrizni, és mi a legkisebb további adatgyűjtés, amelyet javasolnál.

13.1.8 Megoldások és indoklás

1. Három lehetséges nem célpopuláció:

   - **Minden potenciális vásárló, aki nem regisztrált a Northstar csatornáin.** A keret F a regisztrált ügyfelek halmaza; a nem regisztráltak elvileg sem kerülhettek be, ez lefedettségi hiba.
   - **A kosárba tett, de le nem adott rendelések.** A `orders` csak a leadott rendeléseket tartalmazza; a kosárelhagyók kiestek, ez túlélési torzítás. A késési arány a leadott rendelésekre vonatkozik, nem a vásárlási szándékra.
   - **Más időablak vagy más piac rendelései.** A generátor 2025 első 180 napját fedi le; a 2026-os vagy a `west` régióbeli rendelések bekerülési folyamata eltérhet, az azonos eloszlás feltétele nem teljesül automatikusan.

2. Egy lehetséges kód:

```
from pathlib import Path
import pandas as pd

customers = pd.read_csv("data/customers.csv")
orders = pd.read_csv("data/orders.csv", parse_dates=["order_ts"])
orders_region = orders.merge(
    customers[["customer_id", "region"]], on="customer_id", how="left"
)

assert len(orders_region) == 2400
assert orders_region["region"].notna().all()

region_counts = orders_region["region"].value_counts().sort_index()
assert region_counts.sum() == 2400

Path("output").mkdir(exist_ok=True)
region_counts.rename("rendelesek").to_csv("output/region_counts.csv")
print(region_counts)
```

Az ellenőrző invariant a 2400-as összeg; ha a `merge` véletlenül duplikálna, az `assert` elszállna. A hiányzó régió szintén hiba lenne, mert a tiszta `orders.csv` minden `customer_id`-ja létezik a `customers`-ben.

3. A kolléga hibája az, hogy a megfigyelt arányt azonosítja a sokasági aránnyal. A `priority_flag=1` alminta nem véletlen minta; a flag maga is döntés eredménye, és ha a döntés összefügg a késéssel vagy a kosárértékkel, az arány torzul. Az általánosítás akkor lenne védhető, ha a mintába kerülési mechanizmus ismert, véletlen és független a célváltozótól, vagy ha randomizált kísérletből származna az adat. A 2400-as méret a mintavételi zajt csökkenti, de a szelekciós hézagot nem.

4. A `west` rendelésekre nem általánosíthatunk automatikusan. A meglévő három régió a mintavételi keret részét képezi; a `west` ügyfelek a kereten kívül esnek. Ellenőrizni kellene, hogy az új régióban azonos-e a rendelésfeladás folyamata, a szállítási hálózat és az ügyfélösszetétel. A minimális további adatgyűjtés egy rövid, explicit protokoll szerinti megfigyelési időszak a `west` régióban, vagy ha a kérdés oksági, randomizált kísérlet. Amíg nincs ilyen adat, a becslés csak feltételes extrapoláció, amelynek érvényességi korlátját rögzíteni kell.

13.1.9 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés: *a nagy adathalmaz automatikusan reprezentatív minta.* A Northstar 2400 rendelése nagyobb, mint a 300 ügyfél, de ha mind a 300 ügyfél ugyanabból a három régióból, ugyanazokon a csatornákon keresztül érkezett, a külső általánosítás nem javul a sorok számától. A mintavételi zaj $O(1/\sqrt{n})$-nel csökken, de a szelekciós hiba D konstans marad. A „több adat” tehát pontosabb torzított becslést adhat.

Önellenőrzés minden adathalmaz előtt: (1) Mely egységek nem szerepelhetnek a fájlban elvileg sem? (2) A bekerülési feltétel mérhető-e a fájlban? (3) Ugyanaz-e a megfigyelési egység a döntési kérdésben és a számolásban? Az 1.1. alfejezetben tanult arány nevezője itt tér vissza: ha a nevező nem a teljes sokaság, az arány sem arról szól.

13.1.10 Rövid összefoglaló

A minta akkor általánosítható a sokaságra, ha ismerjük a kiválasztási mechanizmust; a mintavételi keret határozza meg, kik kerülhetnek be egyáltalán. A szelekciós torzítás akkor lép fel, ha a bekerülési valószínűség összefügg a vizsgált változóval, a túlélési torzítás pedig akkor, ha csak a folyamatot túlélő egységeket figyeljük meg. A nagy mintaméret a zajt csökkenti, nem a szisztematikus hézagot. A Northstar rendelési adat a leadott rendelések 180 napos ablakát fedi le; nem általánosítható automatikusan a nem regisztráltakra, a kosárelhagyókra vagy más időszakokra és régiókra. A következő alfejezetben ebből a mintából számítunk összegző statisztikát, és megvizsgáljuk, mit árul el a minta a saját eloszlásáról.

13.2 Közép, szóródás és robusztus összegzés

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, matplotlib

Előfeltétel: 13.1. alfejezet · Célkészség: Közép, szóródás és robusztus összegzés · Kimenet: működő kód és a labor ellenőrző tesztje.

13.2.1 Cél és konkrét helyzet

A Northstar `orders` tábla minden sora egy leadott rendelés, a megfigyelési egység tehát a **rendelés**. Az üzleti kérdés így szól: mekkora egy tipikus kosár értéke, és mennyire szóródnak a kosarak? Ehhez a rendeléskor ismert `basket_minor` mezőt használjuk, amely egész szám, mértékegysége **EUR-cent**. Pénzösszegnél mindig a minor unitban számolunk, és csak a megjelenítéskor váltunk euróra; így nem kerül be felesleges lebegőpontos kerekítési hiba.

Az összegzés előtt becslést végzünk: a kereskedelmi kosarakra jellemző, hogy sok a kisebb rendelés, és kevés a nagy értékű. Ezért jobbra ferde eloszlásra számítunk, ahol az átlag nagyobb lehet a mediánnál, mert a nagy értékek húzzák. Az alfejezet végére azt is meg tudjuk mondani, hogy egy kiugró kosár mennyire változtatja meg a következtetést, és mikor érdemes robusztus mutatót választani.

Induló feladat a fogalmak ismerete nélkül: öt rendelés kosárértéke centben

$$x = [1200,\ 1800,\ 1500,\ 2100,\ 9600].$$

Számold ki papíron az átlagot és a mediánt, és fogalmazd meg, melyik jellemzi jobban a „tipikus” rendelést, ha a 9600-as érték egy ritka nagy kosár.

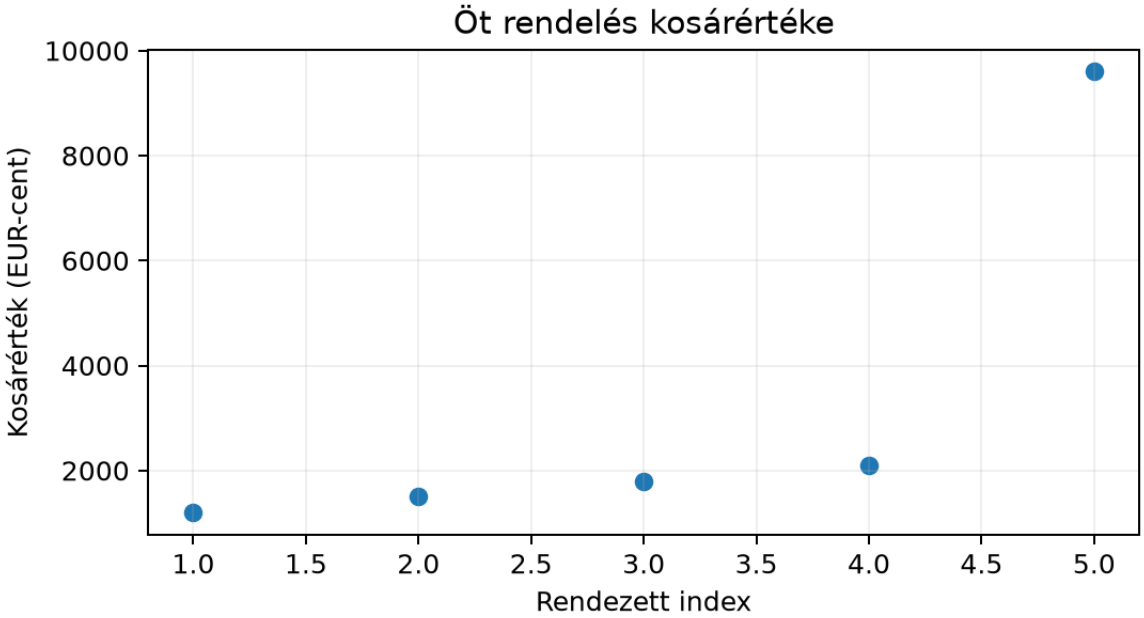
13.2.2 Közép: átlag és medián

A **számtani átlag** (mean) az értékek egyenlő súlyozású összege osztva az elemszámmal:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i,$$

ahol x_i az i-edik rendelés kosárértéke centben, n pedig a rendelések száma. A mértékegység ugyanaz, mint a bemeneté, tehát itt cent. Az átlag fontos tulajdonsága, hogy a teljes összeggel konzisztens: az öt rendelés összértéke pontosan $n\bar{x}$. A hátránya is ebből fakad: minden értékre érzékeny, egyetlen nagy kosár is eltolja.

A **medián** (median) a nagyság szerint rendezett minta középső értéke. Jelölje a rendezett mintát $x_{(1)} \le x_{(2)} \le \ldots \le x_{(n)}$. Ha n páratlan, a medián a középső elem, $x_{((n+1)/2)}$; ha páros, a két középső átlaga. Az ötelemű példában a rendezett minta $[1200, 1500, 1800, 2100, 9600]$, tehát a medián $x_{(3)} = 1800$ cent, azaz 18,00 EUR. A medián a szélső érték konkrét nagyságára érzéketlen: ha 9600 helyett 96 000 szerepelne, a medián akkor is 1800 maradna, az átlag viszont 20 340-re ugrana.



13.2. ábra: Öt rendelés kosárértéke - Kézi minta; a 9600-as pont húzza az átlagot.

A döntés tehát nem az, hogy az átlag vagy a medián „pontosabb”, hanem az, hogy mire válaszolunk. A medián a tipikus egyedi rendelést, az átlag a darabszámmal szorozva a teljes bevételt írja le. A ferde kosáreloszlásnál mindkettőt jelentjük, és kimondjuk az eltérés okát.

13.2.3 Kvantilis és IQR

A **kvantilis** (quantile) azt az értéket keresi, amely alá a rendezett minta meghatározott hányada esik. A q kvantilist Q_q-val jelöljük, ahol $0 < q < 1$. Ez a 12.2. alfejezet eloszlásfüggvény-fogalom mintabeli inverze: ott a CDF megadta, hogy egy érték alá mekkora valószínűség esik, itt a mintából becsüljük azt az értéket, amely alá a minta q hányada esik. A medián tehát a $Q_{0,5}$ kvantilis.

Kis mintán az érték attól is függ, hogyan interpolálunk. A pandas `Series.quantile(q)` alapértelmezése `interpolation="linear"`; ennek képlete az n elemű rendezett mintára a

$$h = (n-1)q$$

pozíciót használja, majd a szomszédos elemek között lineárisan interpolál. Az ötelemű példában $Q_{0,25}$ pozíciója $(5-1)\cdot 0{,}25 = 1$, tehát $Q_1 = x_{(2)} = 1500$ cent; $Q_{0,75}$ pozíciója 3, tehát $Q_3 = x_{(4)} = 2100$ cent. Az R alapértelmezett `type=7` kvantilise ugyanezt a lineáris szabályt adja, de más szoftver ettől kissé eltérhet; ezért a reprodukálható jelentésben a `interpolation` paramétert is rögzíteni kell.

Az **interkvartilis terjedelem** (interquartile range, IQR) a középső 50% terjedelme:

$$\mathrm{IQR} = Q_{0,75} - Q_{0,25}.$$

A példában $\mathrm{IQR} = 2100 - 1500 = 600$ cent. Az IQR robusztus szóródási mutató: a mediánhoz hasonlóan nem változik, ha a legnagyobb értéket a helyéről messzire toljuk, amíg a sorrendi

pozíciók változatlanok maradnak. Az IQR-ből szokás az egyezményes kiugrószabályt képezni: az $[Q_1 - 1{,}5\,\mathrm{IQR},\ Q_3 + 1{,}5\,\mathrm{IQR}]$ sávon kívüli pontokat kiugrónak nevezik. Ez detektálási heurisztika, nem bizonyíték arra, hogy a pont hibás; a kizárás vagy kezelés mindig modellezési döntés marad.

13.2.4 Variancia, szórás és ddof

A **variancia** (variance) az átlagtól vett eltérések négyzetének átlaga. Itt két képletet kell elkülöníteni. Ha a rendelkezésre álló értékek maguk a teljes sokaság, a sokasági variancia

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2,$$

ahol μ a sokasági átlag, a nevező n. Ha azonban az értékek egy nagyobb sokaságból vett **mintát** alkotnak, a minta-variancia

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2.$$

A nevező itt $n-1$. Az $n-1$ miért: az eltéréseket a mintaátlaghoz mérjük, és ez egy lineáris megkötést jelent a maradékokra. Egyszerű véletlen minta esetén az $n-1$-es osztóval számolt s^2 várható értéke pontosan a sokasági variancia, $E[s^2] = \sigma^2$; az n-es osztó átlagosan alábecsülné. A variancia mértékegysége a bemenet négyzete, itt cent^2, ami önmagában nehezen értelmezhető.

A **szórás** (standard deviation) a variancia pozitív négyzetgyöke:

$$s = \sqrt{s^2}.$$

A mértékegysége visszatér az eredetihez, tehát cent; ezért a szórást egyszerű az átlag mellé írni. A ferde eloszlásnál azonban nem szabad azt állítani, hogy az értékek 68%-a az átlag körüli egyszeres szórásba esik: ez a normális eloszlás tulajdonsága, nem általános szabály.

A `ddof` a **delta degrees of freedom**, azaz a nevezőből levont szabadságfok. A NumPy `np.var` és `np.std` alapértelmezése `ddof=0`, tehát a nevező n; a pandas `Series.var` és `Series.std` alapértelmezése `ddof=1`, tehát a nevező $n-1$. A Northstar `orders` 2400 rendelését egy nagyobb, nem teljesen megfigyelt rendelésfolyamat mintájaként kezeljük, ezért a minta-varianciát használjuk; mindkét könyvtárban explicit `ddof=1`-et írunk, hogy a választás ne a könyvtár csendes alapértelmezésétől függjön.

13.2.5 Teljes minimális munkapélda

Először a kézi ötelemű mintát összegezzük, majd elvégezzük ugyanezt a `lab/data/orders.csv` fájlon. A kód a projektgyökérből futtatható, ahol a `lab` mappa elérhető; Python 3.12 és a lab környezet csomagjai szükségesek.

```
import numpy as np
import pandas as pd

# Öt rendelés kosárértéke EUR-centben; a 9600-as a szemléltető kiugró.
x_cent = np.array([1200, 1800, 1500, 2100, 9600], dtype=np.int64)
x_cent.sort()

def legacy_summary(ser: pd.Series) -> pd.Series:
```

```
    """Egy numerikus sorozatra egységes összegző mutatókat ad."""
    out = {
        "n": ser.count(),
        "mean": ser.mean(),
        "median": ser.median(),
        "q1": ser.quantile(0.25, interpolation="linear"),
        "q3": ser.quantile(0.75, interpolation="linear"),
        "var_ddof1": ser.var(ddof=1),
        "std_ddof1": ser.std(ddof=1),
    }
    out["iqr"] = out["q3"] - out["q1"]
    return pd.Series(out)

raw = legacy_summary(pd.Series(x_cent, name="basket_minor"))

# Az 1.5*IQR szabály felső határa; a felette lévő értéket a határra húzzuk be:
upper_fence = raw["q3"] + 1.5 * raw["iqr"]
x_wins = x_cent.copy()
x_wins[x_wins > upper_fence] = upper_fence  # winsorizálás a határra
winsorized = legacy_summary(pd.Series(x_wins, name="basket_minor"))

print(pd.DataFrame({"nyers": raw, "kiugro_hatarra_huzva": winsorized}).round(2).T.to_string())
```

A `x_wins` művelet a 9600-as értéket a felső határra, 3000 centtel egyenlővé teszi, amíg a rendezésben elfoglalt helye nem változik. Ez a **winsorizálás** nevű robusztusítás: a kiugrót nem dobjuk el, hanem visszahúzzuk, így az elemszám megmarad.

A teljes adathoz a Northstar szerződésben rögzített fájlt használjuk:

```
# Projektgyökérből: python eda_summary.py
orders = pd.read_csv("lab/data/orders.csv")
basket = orders["basket_minor"].copy()

full = legacy_summary(basket)

# Egy szerkesztői döntés: a felső 1% feletti kosarakat egy demonstrációs
# szűrésben kiugrónak tekintjük. A küszöb tehát a teljes adat Q_0.99 értéke.
cutoff = basket.quantile(0.99)
filtered = legacy_summary(basket[basket <= cutoff])

print(pd.DataFrame({"teljes": full, "p99_felett_eltavolitva": filtered}).round(2).to_string())
```

Az adatszerződés szerint a `basket_minor` rendeléskori mező, ezért használata nem szivárogtat semmilyen későbbi célváltozót; az 1.2. alfejezet szabály itt nem sérül. A 99. percentilis mint küszöb egy bemutatási választás, nem magától értetődő igazság; a jelentésben címkézni kell, és más küszöbbel a számok értelemszerűen változnak.

13.2.5.1 Szélsőérték-érzékenységi próba: átlag, medián, IQR, MAD, trimmed mean

A robusztusságot úgy lehet megérteni, ha ugyanahhoz a mintához egyetlen szélső értéket adunk. Vegyük az $1, 2, \ldots, 11$ értékeket, majd toldjuk meg a 1000 értékkel:

Statisztika	alapminta	+1000	változás
átlag	6.000	88.833	+82.833
medián	6.000	6.500	+0.500
szórás (ddof=1)	3.317	286.961	+283.644
IQR	5.000	5.500	+0.500
MAD	3.000	3.000	0.000

Statisztika	alapminta	+1000	változás
10% trimmed mean	6.000	6.500	+0.500

Az átlag és a szórás az egyetlen szélső értéktől több mint 80, illetve 280 egységgel elmozdul; a medián, az IQR, a MAD és a trimmed mean legfeljebb 0.5-tel. A medián és a trimmed mean változása itt abból fakad, hogy a 12 elemű minta közepe a 6 és 7 átlaga lesz; a MAD egyáltalán nem változik, mert a szélső érték nem kerül a medián közelébe. A robusztus z-érték $(1000 - 6)/(1.4826 \cdot 3) \approx 223.5$, ami jelzi a kilógó értéket, de a medián és a MAD maga nem mozdul el.

```
import numpy as np

def summary(values):
    values = np.asarray(values, dtype=float)
    q1, q3 = np.percentile(values, [25, 75])
    median = np.median(values)
    mad = np.median(np.abs(values - median))
    cut = int(0.1 * len(values))
    ordered = np.sort(values)
    return {"mean": values.mean(), "median": median,
            "std": values.std(ddof=1), "iqr": q3 - q1, "mad": mad,
            "trimmed": ordered[cut:len(ordered)-cut].mean()}

base = np.arange(1, 12, dtype=float)
print("alap", summary(base))
print("outlier", summary(np.append(base, 1000.0)))
```

A `lab/robustness.py` `summary()`, `sensitivity()` és `robust_z()` függvényei ezt a táblát adják; a `tests/test_robustness.py` az átlag >80-as elmozdulását, a medián/IQR/trimmed 0.5-ös változását és a MAD változatlanságát ellenőrzi. A tanulság: a robusztus statisztika nem „mindig jobb”, hanem a döntési kérdéshez kell választani; az átlag a teljes összeggel, a medián a tipikus értékkel foglalkozik.

13.2.6 Eredmény értelmezése

A kézi ötelemű mintára a következő értékeket kapjuk:

Mutató	Nyers minta	Winsorizált minta
n	5	5
mean	3240,00	1920,00
median	1800,00	1800,00
q1	1500,00	1500,00
q3	2100,00	2100,00
iqr	600,00	600,00
var_ddof1	12 753 000,00	477 000,00
std_ddof1	3571,13	690,65

A mértékegységek: az első négy sor cent, a variancia cent^2, a szórás cent. A 3571 cent 35,71 euró, a 690,65 cent pedig 6,91 euró.

A táblázat jól mutatja a robusztusság definícióját: a medián, a kvartilisek és az IQR **invariáns** a felső kiugró winsorizálására, mert a rendezésbeli pozíciók változatlanok. Az átlag 3240-ről

1920-ra csökken, a szórás pedig 3571-ről 691-re, mert ezek minden konkrét értéket beépítenek. Ha a kérdés a tipikus kosár, a medián stabil; ha a kérdés a teljes bevétel, az ilyen nagy kosarak létezése fontos, és a winsorizálás elfedi.

A teljes Northstar adatot e fejezet írásakor nem futtattuk, ezért az ottani konkrét értékeket itt nem állítjuk mértnek. A fenti kód futtatása után az elvárható szerkezet a következő: a `full` és `filtered` sorok azonos kulcsokkal rendelkeznek, a `filtered["n"]` kisebb, mint `full["n"]`, és a `std_ddof1` a szűrés után jellemzően csökken, amennyiben a felső farok valóban húzza a szórást. A medián és IQR változása attól függ, hogy a szűrés hány középső helyzetű sort érint. A helyi futtatás pontos számai az ellenőrzött eredményjegyzékkel vethetők össze.

A medián robusztussága nem jelenti, hogy hiányzási vagy szelekciós torzítás mellett torzítatlan lenne. A kiugró megfigyelések kizárása előtt dokumentálni kell, hogy mérési hibáról vagy érvényes szélsőértékről van-e szó.

13.2.7 Feladatok

1. Értelmezés — kézi számolás. Az ötelemű mintára $x = [1200, 1500, 1800, 2100, 9600]$ cent. Vezesd le az átlagot, a mediánt, a Q_1 és Q_3 kvartilist a lineáris interpoláció szabályával, majd a minta-szórást s képlettel. Ellenőrizd, hogy $\bar{x} = \frac{1}{n}\sum x_i$ és $s^2 = \frac{1}{n-1}\sum (x_i - \bar{x})^2$, és a szórást ne keverd a varianciával.

2. Implementálás — saját összegző függvény. Írj `summarize_manual(values)` függvényt, amely `n`, `mean`, `median`, `q1`, `q3`, `iqr`, `var`, `std` kulcsokkal szótárat ad, `ddof=1` varianciával. A kvantiliseket a `np.quantile(..., method="linear")` hívással számold. Ellenőrizd a függvényt az ötelemű mintán a pandas `legacy_summary` kimenetével az `np.allclose` segítségével, és add meg a használt toleranciát.

3. Hibakeresés — ddof mismatch. Egy kolléga azt írta a kiértékelő füzetbe, hogy a kézi minta szórása 3194,12 cent. A te eredményed 3571,13 cent. Keresd meg a különbség okát: a NumPy `np.std(x)` hívás mit számol, és a pandas `Series.std()` mit? Melyik a helyes a Northstar rendelésmintán, és mit kérnél a kollégától, hogy a jövőbeni félreértést elkerülje?

4. Megváltozott feltétel — csoportos összegzés. Számold ki ugyanezeket a mutatókat a `priority_flag` szerint csoportosítva a `lab/data/orders.csv`-n. Először ellenőrizd a `priority_flag` dtype-ját és értékkészletét, valamint a csoportméreteket; utána értelmezd a különbséget úgy, hogy a flag nem randomizált, tehát a csoportkülönbség **nem oksági hatás**.

13.2.8 Megoldások és indoklás

1. feladat. Rendezés: $[1200, 1500, 1800, 2100, 9600]$. Az átlag

$$\bar{x} = \frac{1200 + 1500 + 1800 + 2100 + 9600}{5} = \frac{16200}{5} = 3240 \text{ cent.}$$

A medián $n = 5$ páratlan, tehát $x_{(3)} = 1800$ cent. A lineáris kvantilis $h = (n-1)q$: Q_1 esetén $h = 4 \cdot 0{,}25 = 1$, ezért $Q_1 = x_{(2)} = 1500$; Q_3 esetén $h = 3$, ezért $Q_3 = x_{(4)} = 2100$. Az eltérések: $-2040, -1740, -1440, -1140, 6360$ cent. Négyzetük: $4161600, 3027600, 2073600, 1299600, 40449600$; összegük 51012000 cent^2. A minta-variancia $51012000/(5-1) = 12753000$ cent^2, a szórás $\sqrt{12753000} \approx 3571{,}13$ cent. Az átlag azért nagyobb a mediánnál, mert a 9600-as érték az átlagban teljes súllyal jelenik meg, a mediánban csak a sorrendi helyével.

2. feladat. Egy lehetséges megoldás:

```
def summarize_manual(values):
    x = np.asarray(values, dtype=np.float64)
    n = x.size
    s = np.sort(x)
    mean = np.sum(s) / n
    if n % 2:
        median = s[n // 2]
    else:
        median = (s[n // 2 - 1] + s[n // 2]) / 2.0
    q1 = np.quantile(s, 0.25, method="linear")
    q3 = np.quantile(s, 0.75, method="linear")
    var = np.sum((s - mean) ** 2) / (n - 1)
    std = np.sqrt(var)
    return {"n": n, "mean": mean, "median": median, "q1": q1,
            "q3": q3, "iqr": q3 - q1, "var": var, "std": std}

manual = summarize_manual(x_cent)
reference = legacy_summary(pd.Series(x_cent, name="basket_minor"))
pd.testing.assert_series_equal(
    pd.Series(manual, dtype=float).sort_index(),
    reference.astype(float).sort_index(),
    check_dtype=False, rtol=1e-10, atol=1e-10,
)
```

Az ellenőrzés azért átmegy, mert mindkét út ugyanazzal a rendezett bemenettel, ugyanazzal az $n-1$ nevezővel és ugyanazzal a lineáris interpolációval számol. A tolerancia 10^{-10} relatív és abszolút hibát enged; a műveletek itt közel azonos sorrendben futnak, de a lebegőpontos eltérésre így sem érzékeny az összevetés. A `var` és a `std` kulcs nevében itt is érdemes a `ddof=1`-et dokumentálni.

3. feladat. A `np.std(x)` alapértelmezett `ddof=0`-t használ: $\sqrt{51012000/5} = \sqrt{10202400} \approx 3194{,}12$ cent. A `pd.Series.std()` alapértelmezett `ddof=1`-e adja a $\sqrt{51012000/4} \approx 3571{,}13$ centet. A kolléga tehát nem számolt feltétlenül rosszul, hanem más definíciót használt: a sokasági szórást, nem a minta-szórást. A Northstar 2400 rendelését egy folyamat mintájának tekintjük, ezért a minta-szórás a védhető választás. A jó gyakorlat az, hogy a kódban mindig explicit `ddof=1` vagy `ddof=0` szerepeljen, a jelentésben pedig a definíció szóban is.

4. feladat. A futtatható kód a következő:

```
flag_summary = orders.groupby("priority_flag")["basket_minor"].agg(
    n="size",
    mean="mean",
    median="median",
    q1=lambda s: s.quantile(0.25),
    q3=lambda s: s.quantile(0.75),
    iqr=lambda s: s.quantile(0.75) - s.quantile(0.25),
    std=lambda s: s.std(ddof=1),
)
print(flag_summary.round(2))
```

A pontos számok a lab CSV-től függenek; a kód maga állítja elő őket. Az ellenőrző invariant: `flag_summary["n"].sum()` egyenlő `len(orders)`, a `priority_flag` értékkészlete pedig $\{0, 1\}$. A mező a rendelés pillanatában ismert, ezért predictorként használható, de nem randomizált; a nagyobb vagy sürgős kosarak kaphatnak prioritást, így a csoportkülönbség konfoundingot tartalmazhat. Lehetséges alternatíva: a különbséget a többi rendeléskori változóra illesztett regresszióval is megnézhetnénk, de ez is megfigyeléses becslés maradna, nem azonos az 1.2. alfejezet szerinti kísérleti protokollal.

13.2.9 Gyakori tévedés ellenpéldával

Két gyakori csúszást érdemes külön kezelni. Az első: a **szórás** és a **standard error** felcserélése. Tegyük fel, hogy egy 100 rendelésből álló minta kosárértékeinek szórása $s = 10$ euró. Ha valaki azt írja: „az átlag bizonytalansága ±10 euró”, akkor a szórást az átlag becslési bizonytalanságaként értelmezi. A kettő nem ugyanaz: a szórás az egyedi rendeléseké, a standard error az átlagé, és $\mathrm{SE} = \frac{s}{\sqrt{n}} = \frac{10}{10} = 1$ euró. A 10 és az 1 között tízszeres a különbség, tehát az állítás minőségileg más következtetés; a standard error pontos tárgyalása a 15.2. alfejezet egységbe tartozik.

A második: „a medián minden célra jobb, mert robusztus.” Nem igaz. Ha a raktárfinanszírozáshoz az öt rendelés várható összbevételét kell megbecsülnünk, az adatok összege 16200 cent. A mediánnal számolt $5 \cdot 1800 = 9000$ cent durván alábecsülné az összeget, mert a medián éppen azt a nagy kosarat hagyja figyelmen kívül, amely az összeg szempontjából lényeges. A robusztusság azt jelenti, hogy a medián kevésbé érzékeny a farokra; azt nem jelenti, hogy minden kérdésre a farok érdektelen. A döntés a kérdésből indul ki, nem a mutató erényéből.

Önellenőrzés: a nevező mindkét kódúton ellenőrizve van; a szórás mértékegysége cent és nem cent^2; a medián a rendezett minta definíciójával lett számolva; az összes bemutatott kézi szám a megadott képletből következik, a teljes lab kimenet pedig nem lett mértnek állítva.

13.2.10 Rövid összefoglaló

A közép- és szóródási mutatók választása a kérdésből és az eloszlás formájából indul. A számtani átlag a teljes összeget, a medián a ferde eloszlás tipikus egyedi értékét írja le; a kvantilisek a CDF mintabeli inverzei, az IQR a középső 50% robusztus terjedelme. A variancia a négyzetes eltérések átlaga $n-1$ szabadságfokkal, a szórás a variancia gyöke, így mértékegysége visszatér az eredetihez. A ddof-ot mindig explicitté tesszük, mert a NumPy és a pandas alapértelmezései különböznek. A kézi ötelemű példán a medián és az IQR invariáns a kiugró winsorizálására, az átlag és a szórás nem. A következő 13.3. alfejezet ezeket a mutatókat ábrákkal és csoportösszehasonlítással kapcsolja az eloszlás alakjához.

13.3 Eloszlás, kapcsolat és csoportkülönbség

> **Belépő · szint: közép · idő: 60 perc · eszköz: pandas, matplotlib**
>
> Előfeltétel: 13.2. alfejezet · Célkészség: Eloszlás, kapcsolat és csoportkülönbség · Kimenet: működő kód és a labor ellenőrző tesztje.

13.3.1 Helyzet és cél

A Northstar `orders.csv` táblájában a megfigyelési egység egy rendelés. Ebben az alfejezetben nem predikciót építünk, hanem leíró kérdést teszünk fel: milyen alakú a kézbesítési idő eloszlása, milyen irányú a kapcsolat a távolság és a kézbesítési idő között, és mennyire térnek el a régiók? A bemenet a tiszta `orders.csv`, ebből a `distance_km` (float, km), a `delivery_days` (pozitív egész, nap) és a `region` kategória (north, central, south). Kimenetként három célzott ábrát és egy aggregált, illetve rétegzett korrelációs táblát készítünk. Az elvárt ellenőrzés: az ECDF az utolsó megfigyelt értéknél eléri az 1-et, minden korreláció a $[-1, 1]$ intervallumban van, és a rétegzett korrelációk súlyozott átlaga általában nem egyezik az aggregált korrelációval.

Előzetes becslésem az, hogy a távolság és a kézbesítési idő között pozitív, de nem determinisztikus kapcsolat látszik, és a régió átlagos távolsága, illetve késési szintje között is lehet különbség. A

`delivery_days` utólag ismert mező, ezért a leíró elemzésben használható, de prediktorba soha nem kerülhet, ha a kérdés a rendelés pillanatában elérhető becslés.

13.3.2 Alak, kapcsolat és csoportkülönbség ábrákon

13.3.2.1 Hisztogram és binválasztás

A hisztogram a numerikus változó értékkészletét intervallumokra, bin-ekre osztja. Ha az intervallumhatárok $b_0 < b_1 < \cdots < b_k$, akkor a j-edik intervallum gyakorisága

$$h_j = \sum_{i=1}^{n} \mathbf{1}(b_j \le x_i < b_{j+1}),$$

ahol $\mathbf{1}$ indikátor, azaz 1, ha a feltétel igaz, különben 0. A `density=True` normalizálás a gyakoriságot elosztja n-nel és a binszélességgel, így az oszlopok összterülete 1 lesz, de a tengely már sűrűség, nem darabszám. A `delivery_days` egész szám, ezért a legátláthatóbb választás az egész értékekre központozott, 1 nap széles bin: a határok például $1.5, 2.5, \ldots$ alakban adják meg, hogy melyik nap hova esik.

A binválasztás nem semleges döntés. A túl széles bin elrejti a részleteket, a túl keskeny bin hamis hézagokat rajzol. Ha valaki „kétmódusú eloszlást” jelent 50 binből, miközben a 10 bines ábra egyetlen módot mutat, akkor a bin, nem az adat tulajdonságát láttatja. Ugyanezért az ábra alá odaírjuk a binszélességet.

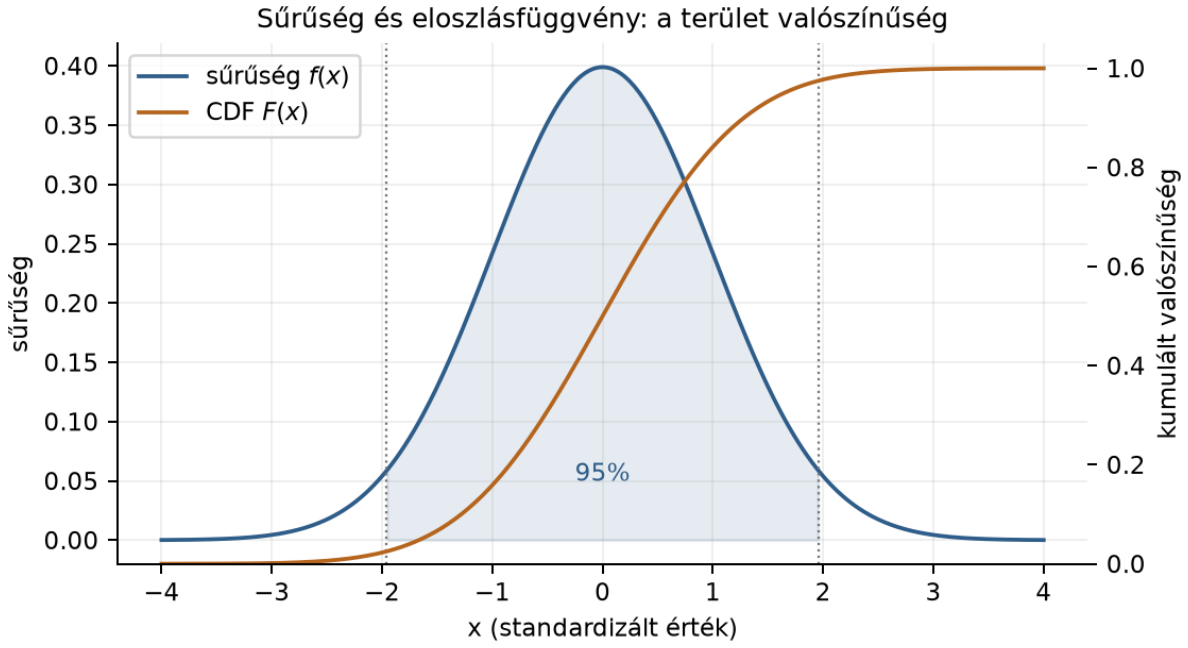
13.3.2.2 ECDF, a binválasztástól független összegzés

Az empirikus eloszlásfüggvény (empirical cumulative distribution function, ECDF) minden x értékhez a nála nem nagyobb megfigyelések arányát rendeli:

$$F_n(x) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}(x_i \le x).$$

Ez lépcsős görbe: minden megfigyelt értéknél $1/n$-nel ugrik feljebb, a lépcső jobbról folytonos. A medián ott van, ahol a görbe eléri vagy átlépi a 0,5-öt; az alsó és felső kvartilis a 0,25 és 0,75 szintekhez tartozik. Az ECDF előnye, hogy nincs binválasztás, tehát a hisztogram ellenőrző párja lehet.

13.3.2.3 Fogalmi ábra: sűrűség és eloszlásfüggvény



13.3. ábra: Sűrűség és eloszlásfüggvény: a haranggörbe alatti terület valószínűség; a CDF a bal oldali farok területét kumulálja, a ± 1.96 közötti sáv pedig a 95%-os normálközelítés.

Az ábra a sűrűség és az eloszlásfüggvény viszonyát mutatja: a sűrűség értéke önmagában nem valószínűség, csak a görbe alatti terület az. A jobb tengelyen a CDF fut; a két görbe ugyanazt az eloszlást írja le. A konklúzió ábra nélkül is megfogalmazható: az ECDF a minta binválasztástól független összegzése, a sűrűség pedig ennek simított, modellfüggő változata. A teljes generátor: `lab/figures/make_concept_figures.py` `density_cdf()`; a reprodukálható mag:

```
import math
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-4, 4, 400)
pdf = np.exp(-0.5 * x**2) / np.sqrt(2 * np.pi)
cdf = 0.5 * (1 + np.vectorize(math.erf)(x / np.sqrt(2)))
fig, ax = plt.subplots(figsize=(6.4, 3.6))
ax.plot(x, pdf, label="sűrűség $f(x)$")
ax.set_xlabel("x (standardizált érték)"); ax.set_ylabel("sűrűség")
ax2 = ax.twinx(); ax2.plot(x, cdf, color="#b5651d")
ax2.set_ylabel("kumulált valószínűség")
ax.fill_between(x, 0, pdf, where=np.abs(x) <= 1.96, alpha=0.12)
ax.legend(loc="upper left"); plt.show()
```

13.3.2.4 Scatter, a kapcsolat megfigyelési szintje

A scatter (szóráskép) minden megfigyelési egységet egy pontként ábrázol: az x-tengely a `distance_km`, az y-tengely a `delivery_days` az adott rendelésre. A pontok iránya, sávszélessége és hajlata mutatja a kapcsolatot. Az azonos koordinátájú pontok elfedhetik egymást, ezért `alpha=0.4` átlátszóságot vagy egy kis jittert használunk. A scatter a nemlineáris kapcsolatot is láthatóvá teszi, míg egyetlen korrelációs szám csak a lineáris részt foglalja össze.

13.3.2.5 Boxplot és csoportméret

A boxplot a 13.2. alfejezetben megismert ötösszámú összegzést rajzolja meg: doboz a Q_1 és Q_3 kvartilisek között, belső vonal a medián, a whisker-ek tipikusan a doboztól $1{,}5 \cdot \mathrm{IQR}$ távolságig terjednek, az ezen kívüli pontok külön jelöltek. A boxplot ferdeséget és kiugrókat mutat, de nem mutatja a csoport méretét: egy három megfigyelésből álló csoport ugyanolyan széles dobozt kaphat, mint egy háromezres csoport. Ezért a csoportméreteket mindig ráírjuk az ábrára, és a kis csoportból levont következtetést külön megjelöljük.

13.3.2.6 Korreláció és kovariancia

Két numerikus változó lineáris kapcsolatát a kovariancia irányítja:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}),$$

ahol $\bar{x}$ és $\bar{y}$ a két mintaátlag. Ha x_i átlag feletti és y_i is átlag feletti, a szorzat pozitív; ha ellentétes irányban térnek el, negatív. A mértékegysége viszont km és nap szorzata, ezért önmagában nem értelmezhető erősségként. A Pearson-korreláció a standardizált eltérések átlagos szorzata:

$$r_{xy} = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n} (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^{n} (y_i - \bar{y})^2}}.$$

A számláló és a nevező ugyanazt a $(\mathrm{km} \cdot \mathrm{nap})$ dimenziót hordozza, ezért r_{xy} dimenziómentes és $[-1, 1]$-be esik. Az $r = 1$ azt jelenti, hogy a pontok egy pozitív meredekségű egyenesen fekszenek; az $r = -1$ negatív egyenest jelent. Ha y az x-nek lineáris függvénye, $y = ax + b$, akkor $r = \mathrm{sign}(a)$. A Pearson-korreláció csak lineáris kapcsolatot mér, és érzékeny a szélsőséges pontokra. Ha csak monotonitásra vagyunk kíváncsiak, a rangokon számolt Spearman-korreláció robusztusabb alternatíva, de ezt itt nem keverjük a Pearson-féle értelmezéssel.

13.3.2.7 Simpson-jelenség

Az aggregált korreláció és a csoporton belüli korrelációk eltérhetnek, ha a csoportképző változó mindkét vizsgált változót eltolja. Tegyük fel, hogy az A csoport adatai (x, y) párokban $(1, 3), (2, 2), (3, 1)$, a B csoporté $(4, 6), (5, 5), (6, 4)$. Mindkét csoportban $r = -1$, mert a pontok tökéletesen csökkenő egyenesen vannak. A hat pont összevonva viszont emelkedő képet mutat, mert a B csoport x- és y-átlaga is magasabb, mint az A-é: az aggregált Pearson-korreláció $+0{,}543$. Ezt hívjuk Simpson-jelenségnek: a trend iránya megfordulhat, ha a rétegző változót figyelmen kívül hagyjuk.

A Northstar esetében az ilyen eltérés oka lehet, ha az egyik régió egyszerre nagyobb átlagos távolságú és nagyobb átlagos késésű, miközben a régión belül a távolság és késés kapcsolata gyenge. Ez nem ok-okozati bizonyíték; csak arra figyelmeztet, hogy a rétegzett elemzés más kérdésre válaszol, mint az aggregált.

13.3.2.8 Számszerű oksági ellenpélda: nulla összevont, ±1 rétegenkénti különbség

A Simpson-jelenség bináris változókkal is megmutatható. Két réteg, Z_0 és Z_1, rétegenként 20 megfigyelés:

Réteg	Kontroll $Y=0$	Kontroll $Y=1$	Treatment $Y=0$	Treatment $Y=1$	Különbség
Z_0	10	0	0	10	$+1$
Z_1	0	10	10	0	-1

Az összevont arány kontrollban $(0+10)/20 = 0.5$, treatmentben $(10+0)/20 = 0.5$, a különbség 0. Rétegenként viszont $+1$, illetve -1. Az összevont nulla nem „nincs hatás”, hanem két ellentétes hatás kioltása; a rétegző változó nélkül a ± 1 hatás láthatatlan. Fordítva is igaz: egy összevont pozitív különbség állhat abból, hogy a treatment túlnyomórészt a magasabb alaparányú rétegbe került.

```
COUNTS = {("Z0", 0, 0): 10, ("Z0", 1, 1): 10,
          ("Z1", 0, 1): 10, ("Z1", 1, 0): 10}

def rate(stratum, treatment):
    rows = [(y, n) for (z, t, y), n in COUNTS.items() if z == stratum and t == treatment]
    return sum(y * n for y, n in rows) / sum(n for _, n in rows)

for stratum in ("Z0", "Z1"):
    print(stratum, rate(stratum, 0), rate(stratum, 1), rate(stratum, 1) - rate(stratum, 0))

def overall(treatment):
    rows = [(y, n) for (_, t, y), n in COUNTS.items() if t == treatment]
    return sum(y * n for y, n in rows) / sum(n for _, n in rows)

print("overall", overall(0), overall(1), overall(1) - overall(0))
```

A `lab/causal_counterexample.py` `rate()`, `overall()` és `report()` függvényei ezt a táblát adják; a `tests/test_causal_counterexample.py` a rétegenkénti $+1/-1$, az összevont 0 és a két arány 0.5 értékét ellenőrzi. Az oksági olvasathoz rétegzés, randomizáció vagy explicit azonosítás kell; a nyers összevont különbség önmagában nem az.

13.3.3 Teljes minimális munkapélda

Először kézi ellenőrzést végzünk. Legyen öt rendelés `distance_km` értéke

$$x = (5, 10, 15, 20, 25)$$

és a hozzájuk tartozó `delivery_days`

$$y = (2, 3, 3, 4, 5).$$

A mintaátlagok $\bar{x} = 15$ km és $\bar{y} = 3{,}4$ nap. Az eltérések szorzatösszege 35, az x-eltérések négyzetösszege 250, az y-eltéréseké 5,2. Így

$$r = \frac{35}{\sqrt{250 \cdot 5{,}2}} = 0{,}9707.$$

Ez erős pozitív lineáris kapcsolatot jelez ezen az öt ponton. Az ellenőrző tulajdonság, hogy r a dimenziómentes standardizálás miatt nem változik, ha a távolságot mérföldre váltjuk, mert az x-értékek közös pozitív számmal való szorzása a számlálót és a nevezőt ugyanazzal a konstanssal szorozza.

A laboradat három célzott ábráját az alábbi script készíti el. A kód a lab projekt gyökeréből futtatandó, ahol a `pyproject.toml` és az `orders.csv` található. A korábbi környezetfejezetekben beállított uv-projektet használjuk:

```
# a lab/ munkakönyvtárból, ahol a pyproject.toml és az orders.csv van
uv run python s051_eda.py
```

```
# s051_eda.py
# Leíró elemzés: eloszlás, kapcsolat, csoportkülönbség
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

def pearson_manual(x, y):
    """Pearson-korreláció a képletből, explicit centírozással."""
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    xd = x - x.mean()
    yd = y - y.mean()
    return float((xd * yd).sum() / np.sqrt((xd ** 2).sum() * (yd ** 2).sum()))

# --- Kézi kontroll, nincs szükség a laborfájlra ---
demo_x = np.array([5.0, 10.0, 15.0, 20.0, 25.0])
demo_y = np.array([2.0, 3.0, 3.0, 4.0, 5.0])
demo_r = pearson_manual(demo_x, demo_y)
print(f"Kezi Pearson r: {demo_r:.4f}")
assert np.isclose(demo_r, 0.9707, atol=0.0001)

# --- Northstar orders betöltése ---
orders = pd.read_csv(
    "orders.csv",
    parse_dates=["order_ts"],
    usecols=[
        "order_id", "order_ts", "distance_km", "delivery_days",
        "region", "priority_flag"
    ],
)
eda = orders.dropna(subset=["distance_km", "delivery_days"])
print("shape:", eda.shape)
print("dtypes:")
print(eda[["distance_km", "delivery_days", "region"]].dtypes)

out_dir = Path("figures_s051")
out_dir.mkdir(exist_ok=True)

# --- 1. ábra: hisztogram + ECDF ---
fig, (ax_hist, ax_ecdf) = plt.subplots(1, 2, figsize=(11, 4))
bins = np.arange(eda["delivery_days"].min() - 0.5,
                 eda["delivery_days"].max() + 1.5, 1.0)
ax_hist.hist(eda["delivery_days"], bins=bins, color="#4C72B0",
             edgecolor="white")
ax_hist.set_xlabel("delivery_days (nap)")
ax_hist.set_ylabel("darabszám")
ax_hist.set_title("Hisztogram, 1 napos bin")
```

```
sorted_days = np.sort(eda["delivery_days"].to_numpy())
ecdf_y = np.arange(1, len(sorted_days) + 1) / len(sorted_days)
ax_ecdf.step(np.r_[sorted_days[0] - 1, sorted_days],
             np.r_[0.0, ecdf_y], where="post", color="#C44E52")
ax_ecdf.axhline(0.5, color="gray", linewidth=0.6, linestyle="--")
ax_ecdf.set_xlabel("delivery_days (nap)")
ax_ecdf.set_ylabel("F_n(x)")
ax_ecdf.set_title("ECDF, binválasztástól független")
fig.tight_layout()
fig.savefig(out_dir / "delivery_days_distribution.png", dpi=150)
plt.close(fig)
assert np.isclose(ecdf_y[-1], 1.0)

# --- 2. ábra: scatter rétegző színnel ---
fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(data=eda, x="distance_km", y="delivery_days",
                hue="region", alpha=0.4, ax=ax)
ax.set_xlabel("distance_km (km)")
ax.set_ylabel("delivery_days (nap)")
ax.set_title("Kézbesítési idő a távolság függvényében, régiónként")
fig.tight_layout()
fig.savefig(out_dir / "distance_vs_delivery_scatter.png", dpi=150)
plt.close(fig)

# --- 3. ábra: boxplot + csoportméret ---
region_order = ["north", "central", "south"]
fig, ax = plt.subplots(figsize=(7, 5))
sns.boxplot(data=eda, x="region", y="delivery_days",
            order=region_order, color="#55A868", ax=ax)
counts = eda["region"].value_counts().reindex(region_order)
for i, region in enumerate(region_order):
    ax.text(i, ax.get_ylim()[1] * 0.98, f"n={counts[region]}",
            ha="center", va="top", fontsize=9)
ax.set_xlabel("region")
ax.set_ylabel("delivery_days (nap)")
ax.set_title("Kézbesítési idő csoportonként, csoportmérettel")
fig.tight_layout()
fig.savefig(out_dir / "delivery_days_by_region_boxplot.png", dpi=150)
plt.close(fig)

# --- Aggregált és rétegzett korreláció ---
r_all = pearson_manual(eda["distance_km"], eda["delivery_days"])
print(f"Aggregalt Pearson r: {r_all:+.3f}")
for region, d in eda.groupby("region"):
    r_group = pearson_manual(d["distance_km"], d["delivery_days"])
    print(f"{region:<8} n={len(d):4d}  atlag_tav={d['distance_km'].mean():6.2f} km "
          f" atlag_keses={d['delivery_days'].mean():5.2f} nap  r={r_group:+.3f}")
```

A csoportátlagok kiírása azért fontos, mert a Simpson-jelenséget nem a rétegzett r-ek önmagukban, hanem a csoportok közötti átlageltolódásokkal együtt lehet értelmezni: ha az egyik régió átlagos távolsága és átlagos késése is magas, az aggregált r a csoportok közötti eltolódást is tükrözi.

13.3.4 Eredmény értelmezése

A kézi kontroll $r = 0{,}9707$ pontosan egyezik a levezetéssel. A labor futtatása során a script a `figures_s051/` mappába írja a három PNG-t, és kiírja az aggregált, illetve régiós korrelációkat. A jelen tankönyvi generálás nem futtatja ezt a scriptet, ezért a Northstar-számokat itt nem nevezzük mértnek; a tanuló a saját futtatási jegyzékében rögzíti a tényleges értékeket. Az invariáns ellenőrzések a kódban futnak: az ECDF utolsó értéke 1, a csoportméretek összege

a teljes sorok száma, és a Pearson-függvény minden hívása a $[-1, 1]$ tartományban marad. A `dropna` utáni sorok számát is kiírjuk, hogy egy esetleges szándékosan sérült bemenet ne csendben torzítsa az ábrákat. Az ábrákat úgy érdemes olvasni, hogy a hisztogram módusza és ferdesége a tipikus késést, az ECDF pedig a kvantiliseket adja binfüggés nélkül; a scatter a régiók szerinti színnel a rétegzett kapcsolatot mutatja; a boxplot a mediánok és a szóródás összevetését adja a csoportméretek ismeretében.

A korreláció csoportonkénti és összesített értéke közötti eltérés lehet összetételhatás; a Pearson-korreláció aggregálása nem egyszerű súlyozott átlagolás. A Simpson-jelenség példájában válasszuk külön a csoporton belüli és a csoportok közötti összefüggést.

13.3.5 Feladatok

1. **Értelmezés.** A scatter pozitív irányt mutat `distance_km` és `delivery_days` között. Fogalmazd meg, miért nem jelenti ez automatikusan azt, hogy a nagyobb távolság okozza a késést, és adj egy lehetséges alternatív magyarázatot a régiókkal kapcsolatban.
2. **Megvalósítás.** A kézi ötelemű példából (`x=(5,10,15,20,25)`, `y=(2,3,3,4,5)`) készíts ECDF-et. Add meg lépésenként a rendezett y-értékeket, a hozzájuk tartozó $F_n(y)$ értékeket, és olvasd le a mediánt.
3. **Hibakeresés.** Egy kolléga a `region` és `delivery_days` kapcsolatát `sns.scatterplot(data=orders, x="region", y="delivery_days")` parancssal ábrázolja. Mi a hiba? Melyik ábratípus alkalmasabb erre a változópárra, és mit veszítenénk, ha a boxploton a csoportméreteket nem tüntetjük fel?
4. **Megváltozott feltétel.** Készíts másolatot az `eda` tábláról, amelyben a `north` régió minden `delivery_days` értékéhez hozzáadsz 3 napot. Számítsd ki újra az aggregált és a régiós Pearson-korrelációkat. Melyik értéknek kell a lebegőpontos tolerancián belül változatlannak maradnia, és miért?

13.3.6 Megoldások és indoklás

1. A korreláció együttmozgást mér, nem beavatkozást. A nagyobb távolság növelheti a késést, de a `region` mindkettővel összefügghet: ha például a `north` régió jellemzően nagyobb távolságú rendeléseket és nagyobb átlagos késést is tartalmaz, akkor a régió keverő változó lehet. Ekkor az aggregált pozitív r részben a régiók közötti eltolódásból fakad, nem a távolság rendelésen belüli hatásából. A rétegzett scatter és a csoportonkénti r segít elkülöníteni ezt a két kérdést. Oksági állításhoz randomizált kísérlet vagy explicit oksági azonosítás kell; a korrelációs ábra csak a következő vizsgálat kiindulópontja.

2. A rendezett y-értékek $2, 3, 3, 4, 5$. Az ECDF a jobbról folytonos lépcsővel:

x	F_n(x)
2	$1/5 = 0.20$
3	$3/5 = 0.60$
4	$4/5 = 0.80$
5	$5/5 = 1.00$

A medián az a legkisebb érték, ahol $F_n(x) \geq 0{,}5$, tehát 3 nap. Ellenőrzés: a rendezett minta középső eleme a harmadik, ami 3.

3. A scatter két numerikus tengelyt vár, de a `region` kategória. A kategorikus értékeket a scatter önkényes sorrendben helyezné a vízszintes tengelyre, ami hamis folytonos kapcsolatot sugallna. Kategorikus csoportosítóhoz a boxplot vagy viola plot alkalmas. Ha a csoportméreteket nem írjuk ki, egy 3 elemű régió ugyanúgy néz ki, mint egy 900 elemű, ezért tévesen adhatunk azonos súlyt a kis mintájú csoport ingadozásának. A csoportméretet mindig fel kell tüntetni.

4. A `north` csoporton belüli Pearson-korrelációnak változatlannak kell maradnia, mert $r(x, y + c) = r(x, y)$: a konstans c a centírozás során kiesik. Numerikusan `np.isclose(r_north_old, r_north_new, atol=1e-12)` ellenőrzést várunk. Az aggregált r viszont változhat, mert a $+3$ napos eltolás csak az egyik keverékkomponenst mozgatja, így a régiók közötti eltolódás és a teljes mintabeli kovariancia szerkezete módosul. Ez a feladat éppen a standardizálás 12.3. alfejezetben látott invarianciáját és a keverék eloszlások érzékenységét köti össze.

13.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a korreláció ok-okozati hatásként jelenik meg. A fenti Simpson-példa az ellenpélda: az A és B csoportban $r = -1$, az összevont hat ponton $r \approx +0{,}543$. Ugyanazokat a pontokat nézve a „hatás” iránya az összevonástól függ — ezért a korreláció predikciós együttmozgást ír le, nem beavatkozási hatást. A második gyakori tévedés a hisztogram alakjának túlértelmezése. Ugyanaz a `delivery_days` minta széles binnel egyetlen módust mutathat, keskeny binnel pedig több különálló csúcsot; a módusz és a „módok száma” nem adatjellemző, hanem binválasztás függvénye is. Az ECDF, a kvantilisek és a medián binfüggetlen kontrollt adnak. Önellenőrzés: meg tudnám-e mondani egy boxplotról, hogy a mögötte álló csoport 10 vagy 1000 elemű? Meg tudnám-e mondani, hogy egy aggregált r mögött mennyi rétegen belüli, ellentétes irányú kapcsolat van? Ha a válasz nem, kiegészítő ábra vagy táblázat kell.

13.3.8 Rövid összefoglaló

A hisztogram a binválasztás függvényében mutatja az alakot, ezért az ECDF binfüggetlen kvantilisellenőrzést ad. A scatter megfigyelési egységenként ábrázolja két numerikus változó kapcsolatát, a boxplot kategorikus csoportok között veti össze a mediánt, kvartiliseket és kiugrókat, de a csoportméretet külön fel kell tüntetni. A Pearson-korreláció a standardizált eltérések átlagos szorzata, dimenziómentes, és csak lineáris kapcsolatot mér. A Simpson-jelenség azt mutatja, hogy az aggregált és a rétegzett korreláció iránya eltérhet, ha a rétegző változó mindkét tengelyt eltolja. A Northstar-labor három ábrája és a régiós korrelációs tábla ezt a megkülönböztetést gyakorolja. A következő 13.4. alfejezet a hiányzó értékek és anomáliák vizsgálatát kapcsolja ehhez a szemlélethez.

13.4 Hiány és anomália mint vizsgálati kérdés

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, matplotlib

Előfeltétel: 13.3. alfejezet · Célkészség: Hiány és anomália mint vizsgálati kérdés · Kimenet: működő kód és a labor ellenőrző tesztje.

13.4.1 Cél és konkrét helyzet

A Northstar-labor nyers rendeléstáblája, a `lab/data/orders_raw.csv`, szándékosan tartalmaz adatminőségi hibákat. Az adatszerződés szerint az alapgenerálásnál 2405 sor van: 5 extra duplikált sor, 2 hiányzó `distance_km`, 1 negatív `basket_minor` és 1 olyan sor, amelynek `customer_id` értéke nem található a `customers.csv`-ben. A megfigyelési egység egy rendelés, a hivatkozási kulcs az `order_id`, a pénzösszegek egész EUR-centben értendők. A feladat nem az, hogy gyorsan „megtisztítsuk" a táblát, hanem az, hogy a hiányt és anomáliát vizsgálati kérdésként kezeljük: készítsünk missingness-jelentést, kössük a hibákat a mérési szerződéshez, és hasonlítsunk össze két alternatív tisztítást anélkül, hogy automatikusan törölnénk minden gyanús sort.

Előzetes becslés: a hiányzási arány a `distance_km` oszlopban $2/2405 \approx 0{,}00083$, azaz 0,083%; a duplikátumok eltávolítása 2400 sort hagy, mert az 5 extra sor másodpéldány. A 4 hibás sor törléses kezelése 2396 elemű elemzési táblát eredményez, tehát a tiszta `orders.csv` 2400 soros kontrolljához képest 4 rendelésnyi információvesztést.

13.4.2 Hiányzási mutató és mechanizmusfeltételezés

Legyen az x_{ij} az i-edik megfigyelési egység j-edik változójának értéke, ahol $i = 1, \dots, n$ és $j = 1, \dots, p$. A **missingness indicator** (hiányzásjelző)

$$R_{ij} = \begin{cases} 1, & \text{ha } x_{ij} \text{ hiányzik,} \\ 0, & \text{egyébként.} \end{cases}$$

Itt R_{ij} bináris, dimenzió nélküli, az R mátrix alakja (n, p), tehát ugyanaz, mint a nyers adaté. A j-edik oszlop hiányzási aránya

$$\bar{R}_j = \frac{1}{n} \sum_{i=1}^{n} R_{ij} = \frac{n_{\text{miss},j}}{n},$$

ahol $n_{\text{miss},j}$ a hiányzó elemek darabszáma. Ha a hiányt egy csoportra, például `priority_flag` szerint szűkítve nézzük, a feltételes arány

$$\bar{R}_j^{(g)} = \frac{\sum_{i \in g} R_{ij}}{n_g},$$

ahol g az adott csoport indexhalmaza, n_g a csoport elemszáma. A nevező itt a csoport összes sora, nem a hiányzók száma. A laborban ez a nevező különösen fontos, mert a duplikátumok megsokszorozhatják ugyanazt a rendelést.

A megfigyelt hiányzási minta önmagában nem mondja meg, miért hiányzik az adat. Három szokásos feltételezés:

- **MCAR** (*missing completely at random*): a hiány független minden megfigyelt és nem megfigyelt változótól. A hiányzó `distance_km`-ek például egy technikai kihagyás véletlen eredményei lehetnének.
- **MAR** (*missing at random*): a hiány függhet a megfigyelt adatoktól, de a hiányzó változó tényleges értékétől nem. Például a `distance_km` hiánya gyakoribb lehet bizonyos régiókban, de a tényleges távolságtól független.

- **MNAR** (*missing not at random*): a hiány magától a hiányzó értéktől vagy más, nem megfigyelt okból függ. Például éppen a nagyon távoli, bonyolult szállításoknál maradhat el a távolságrögzítés.

Kis kézi kontroll: hat ügyfél, a `region` megfigyelt, a `distance_km` két sorban hiányzik.

$$R = [0, 0, 1, 0, 0, 1]^T, \qquad \bar{R} = \frac{2}{6} = \frac{1}{3}.$$

A feltételes arány `priority_flag=0` esetén 1/3, `priority_flag=1` esetén ugyancsak 1/3 lehet. Ez a megfigyelt táblán MCAR-mintázatnak látszik, de ugyanilyen mintát adhat egy MNAR-mechanizmus is, ha a 2 elveszett távolság éppen a legnagyobb távolságú megrendeléseké. A mechanizmus tehát nem a táblából, hanem a mérési folyamatból igazolható.

13.4.2.1 MAR vs MNAR: ugyanaz a megfigyelt tábla, más latent világ

A hiányzási mechanizmus nem olvasható ki a megfigyelt adatokból. A klasszikus felosztás:

Mechanizmus	Feltétel	Példa
MCAR	a hiány független minden változótól	véletlen mintavesztés
MAR	a hiány a megfigyelt változóktól függ, a hiányzó értéktől nem	a hiányzó jövedelem az iskolázottságtól függ
MNAR	a hiány a hiányzó értéktől is függ	a magas jövedelműek nem válaszolnak

Vegyük a megfigyelt táblát: X_0-nál $Y = 1$, X_1-nél Y hiányzik kétszer. Ugyanez a megfigyelt adat két különböző teljes világból származhat:

Világ	Y hiányzó értékei	Teljes átlag	Mechanizmus
MAR	0, 0	1/3	a hiány X-től függ
MNAR	1, 1	1.0	a hiány Y-tól függ

A megfigyelt átlag 1/3, de a teljes átlag a hiányzó értékektől függően $[1/3, 1]$ intervallumban bármi lehet. A hiányzási minta és a megfigyelt értékek azonosak; a mechanizmus csak külső domaininformációból (adatfelvételi folyamat, kérdőívlogika, utánkövetés) dönthető el, és akkor sem bizonyossággal, hanem feltevésként. Ezért a hiányzás kezelésekor a **feltevést** és a **szenzitivitási elemzést** együtt kell jelenteni, nem pedig „MCAR-nak látszik” alapon pontbecslést adni.

```
def bounds(observed_sum, observed_n, missing_n):
    total = observed_n + missing_n
    return {"min": observed_sum / total,
            "max": (observed_sum + missing_n) / total,
            "observed_mean": observed_sum / observed_n}

# Megfigyelt: egy 1-es, két hiányzó
print(bounds(observed_sum=1, observed_n=1, missing_n=2))
# {'min': 0.333..., 'max': 1.0, 'observed_mean': 1.0}
```

A `lab/missingness.py` `bounds()`, `scenarios()` és `report()` függvényei ezt a két világot és az azonos megfigyelt táblát adják; a `tests/test_missingness.py` a $[1/3, 1]$ korlátot, a két eltérő teljes átlagot és az azonos megfigyelt táblát ellenőrzi.

13.4.3 Anomália, tartományellenőrzés és outlier

A **range check** (tartományellenőrzés) a szerződésben rögzített, lehetséges értéktartományt vizsgálja. Formálisan az érvényes tartomány $[a_{ij}, b_{ij}]$, tehát

$$a_{ij} \leq x_{ij} \leq b_{ij}.$$

A laborban ilyen szabály például: `distance_km >= 0`, `basket_minor >= 0`, `quantity_sum > 0`, `promised_days == 7`, `delivery_days > 0`. A negatív `basket_minor` tartománysértés, mert a szerződés szerint a kosárérték nem lehet negatív. A `distance_km` hiánya viszont nem tartománysértés, hanem hiány.

A **kiugró** (*outlier*) olyan megfigyelt érték, amely a többitől szokatlanul távol esik, de a tartománysértéssel ellentétben lehetséges érték. A boxplot egy gyakori jelölési szabályt használ: a `whisker` határok

$$Q_1 - 1{,}5\,(Q_3 - Q_1) \qquad \text{és} \qquad Q_3 + 1{,}5\,(Q_3 - Q_1),$$

ahol Q_1 és Q_3 az alsó és felső kvartilis, az IQR pedig $Q_3 - Q_1$. Ez az 1,5-ös szabály csak vizuális jelölés; egy nagyobb távolság lehet valós, üzletileg fontos rendelés, nem automatikus törlendő hiba.

13.4.4 Teljes minimális munkapélda

A példa a laborfájlok neveit használja; az útvonalat a projektgyökérhez igazítsuk. A pandas 2.2.3, NumPy, Matplotlib és Seaborn a `lab/pyproject.toml` alapfüggőségei közül való. A bemutatott kimenet **szemléltető**, mert e szöveg írása közben nem futott kód; a laborban a tanuló futtatásából származik a mért eredmény.

```
# Linux/macOS, munkakönyvtár a labor projektgyökere
uv run python missingness_report.py
```

Windows PowerShellben a futtatás előtt `Set-Location C:\path\to\northstar-lab`, majd ugyanez a `uv run` parancs.

```
"""Missingness-jelentés és két alternatív tisztítás érzékenységi összevetése."""
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

LAB = Path("lab")
RAW = pd.read_csv(LAB / "orders_raw.csv")
CUSTOMERS = pd.read_csv(LAB / "customers.csv")
REFERENCE = pd.read_csv(LAB / "orders.csv")  # kontroll, nem a tisztítás inputja

# 1. Szerződésellenőrzés: a labor alapméretei
print(RAW.shape)
```

```
# Elméletileg várt: (2405, 15)

duplicated_mask = RAW.duplicated()
missing_distance = RAW["distance_km"].isna()
negative_basket = RAW["basket_minor"] < 0
known_customer = RAW["customer_id"].isin(CUSTOMERS["customer_id"])

print(duplicated_mask.sum())       # várt: 5
print(missing_distance.sum())      # várt: 2
print(negative_basket.sum())       # várt: 1
print((~known_customer).sum())     # várt: 1

# 2. Missingness-jelentés
R = RAW.isna().astype(int)          # shape: (2405, 15)
missing_report = pd.DataFrame({
    "n_missing": R.sum(),
    "missing_rate": R.mean(),
})
print(missing_report.loc[missing_report["n_missing"] > 0])
# A distance_km hiányzási aránya: 2/2405, a többi oszlopé 0.

# 3. Tartomány- és hivatkozási ellenőrzés
range_checks = {
    "distance_km_negative": RAW["distance_km"] < 0,
    "basket_minor_negative": RAW["basket_minor"] < 0,
    "quantity_sum_nonpositive": RAW["quantity_sum"] <= 0,
    "promised_days_not_7": RAW["promised_days"] != 7,
    "delivery_days_nonpositive": RAW["delivery_days"] <= 0,
    "customer_unknown": ~known_customer,
}
range_report = pd.DataFrame({
    "n_violations": {k: int(mask.sum()) for k, mask in range_checks.items()}
})
print(range_report)
# A labor szerint a negatív basket és az ismeretlen customer jelenik meg.

# 4. Két alternatív tisztítás
base = RAW.loc[~duplicated_mask].copy()  # 2400 sor, 5 duplikált másodpéldány nélkül

bad_mask = (
    base["distance_km"].isna()
    | (base["basket_minor"] < 0)
    | ~base["customer_id"].isin(CUSTOMERS["customer_id"])
)

clean_drop = base.loc[~bad_mask].copy()      # A: hibás sorok listwise törlése

clean_keep = base.copy()                     # B: megtartás és hibajelző
clean_keep["distance_na"] = base["distance_km"].isna().astype(int)
clean_keep.loc[base["basket_minor"] < 0, "basket_minor"] = np.nan
clean_keep["unknown_customer"] = ~base["customer_id"].isin(CUSTOMERS["customer_id"])

# A másolás után a basket_minor float64 lehet, mert a NaN miatt.
# A distance_km hiánya és a target_late megfigyelése B-ben megmarad.

sensitivity = pd.DataFrame({
    "variant": ["raw_with_duplicates", "drop_bad_rows", "keep_and_flag"],
    "n_rows": [len(RAW), len(clean_drop), len(clean_keep)],
    "late_rate": [
        RAW["target_late"].mean(),
        clean_drop["target_late"].mean(),
        clean_keep["target_late"].mean(),
    ],
```

```
    "distance_mean": [
        RAW["distance_km"].mean(),
        clean_drop["distance_km"].mean(),
        clean_keep["distance_km"].mean(skipna=True),
    ],
})
print(sensitivity)

# 5. Ábra: a távolság eloszlása a megtartó változatban
fig, ax = plt.subplots()
sns.boxplot(data=clean_keep, x="priority_flag", y="distance_km", ax=ax)
ax.set_title("Szemléltető: distance_km priority_flag szerint")
fig.tight_layout()
plt.show()
```

A kód `R = RAW.isna().astype(int)` sora közvetlenül előállítja a hiányzásjelző mátrixot. A `range_report` egyes szabályai a szerződött tartományokat ellenőrzik; a negatív `distance_km` szabály a 2 hiányzó sort nem jelöli, mert a `NaN < 0` értéke `False`. A `clean_keep` változat a negatív kosárértéket hiányként jelöli, de a sor többi értékét megtartja, így a `target_late` arány a 2400 soros alapon számítható.

13.4.5 Eredmény értelmezése

A labor szerződött invariánsai adják a kézi kontrollt: a nyers tábla 2405 sor, a duplikált sorokból 5, a `distance_km` hiányból 2, a negatív kosárból 1, az ismeretlen ügyfélből 1 található. A `RAW.drop_duplicates()` 2400 sort ad, de ez a 2400 sor még nem azonos a tiszta `orders.csv`-vel, mert a 4 hibás sor javítatlan marad. A `drop_bad_rows` változat 2396 sort hagy; ez a 4 hibás sor eltávolításának következménye. A `distance_km` hiányzási arányának nevezőjét a duplikátumok nélkül kell értelmezni: 2 hiány 2400 rendelésből, nem 2405 sorból.

A két alternatíva közötti érdemi különbség a 4 hibás sor. Az A változat konzervatív, de a hibás sorok eldobásával minden oszlopra nézve elveszít 4 rendelést. A B változat a hibajelzőket megtartja, ezért a `target_late` arányt a teljes 2400 rendelésen becsüli, de a `distance_km`-et érintő elemzéseknél a 2 hiány továbbra is kimarad. A `distance_km` átlagban a nyers tábla és a megtartó változat között kicsi a különbség, mert a negative basket és az ismeretlen ügyfél nem érinti a távolságot. Az A és B közötti late rate különbsége nem a módszer által garantált irányú: azt a konkrét sorok `target_late` értéke határozza meg, ezért az összevetést mindig a labor tényleges futtatásából kell kiolvasni.

A 2 hiányzó `distance_km` statisztikai értelemben túl kevés ahhoz, hogy a hiányzást `priority_flag` vagy más változó szerinti csoportosítással megbízhatóan mechanizmushoz kössük. A missingness-jelentés ezért itt főként dokumentációs eszköz, nem mechanizmusteszt.

Az MCAR, MAR és MNAR kategóriák a hiányzási mechanizmus feltevései, nem olyan címkék, amelyek az egyetlen megfigyelt táblából bizonyossággal megállapíthatók. A teljes esetek elemzése ezért érzékenységvizsgálatot igényelhet.

13.4.6 Feladatok

1. **Értelmezés.** A megfigyelt táblán a `distance_km` hiánya a `priority_flag=1` csoportban sokkal gyakoribbnak látszik, mint a másikban. Miért nem jelenti ez automatikusan azt, hogy a hiány MAR vagy MNAR? Milyen mérési folyamatból származó, táblán kívüli ellenőrzést kérne?

2. **Implementálás.** Írj `missingness_report(df)` függvényt, amely a bemenő DataFrame minden oszlopára visszaadja a hiányzó darabszámot és a hiányzási arányt. Ezután a `distance_km`-re adjon feltételes hiányzási arányt `priority_flag` szerint, és add meg pontosan a nevezőt.

3. **Hibakeresés.** Egy kolléga a nyers adatot így tisztítja: `orders_raw.dropna().copy()`, majd a hiányzó `distance_km`-et nullára cseréli: `df["distance_km"].fillna(0)`. Miért hibás mindkét lépés? A labor szerződésére hivatkozva indokold, miért nem tekinthető a 0 távolságnak.

4. **Megváltozott feltétel.** Tegyük fel, hogy a `distance_km` hiánya éppen akkor valószínűbb, ha a csomag késni fog, és a hiány a nagy távolságú rendeléseknél gyakoribb. Melyik alternatív tisztítás torzítaná kevésbé a `target_late` arány becslését? A távolságra épülő modellnél ugyanez a választás elegendő-e? Válaszát a 4 hibás sor szerepére alapozza.

13.4.7 Megoldások és indoklás

1. A megfigyelt táblából csak az R minta látszik, a hiányzó tényleges `distance_km` értékek nem. Ugyanazt a feltételes arányt előállíthatja az, hogy a prioritásos rendelések feldolgozása más rendszerben történt (MAR), de az is, hogy a kritikus távolságoknál hiányos a mérés (MNAR). A generátor forráskódja a laborban megmondja a tényleges mechanizmust, egy üzleti rendszerben viszont a napló, a mérési dokumentáció vagy egy reprodukált bevitel kell. Az ellenőrző kérdés: „Mikor és melyik rendszer írta ezt a mezőt üresen?”

2. A függvény:

```
def missingness_report(df):
    n = len(df)
    return pd.DataFrame({
        "column": df.columns,
        "n_missing": df.isna().sum().to_numpy(),
        "missing_rate": (df.isna().sum() / n).to_numpy(),
    })

report = missingness_report(RAW)

clean_base = RAW.drop_duplicates().copy()
clean_base["distance_na"] = clean_base["distance_km"].isna().astype(int)
conditional = (
    clean_base
    .groupby("priority_flag")["distance_na"]
    .agg(n_missing="sum", n_group="count", missing_rate="mean")
)
```

A nevező a `priority_flag` adott csoportjának összes rendelése, nem a hiányzó értékek száma. A duplikátummentes `clean_base` használata azért lényeges, mert a duplikátum mindkét csoport nevezőjét megváltoztathatná.

3. A `dropna()` az egész DataFrame minden oszlopára törölne minden olyan sort, ahol bármelyik mező hiányzik. A laborban most csak a `distance_km` hiányos, ezért 2 sort dobna el; viszont a művelet a jövőbeli oszlopok hiányára is automatikusan vonatkozna, és nem dokumentálná, hogy melyik mező miatt vesztettünk sort. A `fillna(0)` ettől súlyosabb: új adatot talál ki. A `distance_km = 0` egy konkrét, ténylegesen kis távolságú rendelést jelent; a szerződésben a nullát nem szabad távolságnak tekinteni. A hiányt `NaN`-ként kell megtartani, vagy a mechanizmusfeltételezés szerint külön kezelni.

4. A `target_late` arány becsléséhez a `keep_and_flag` jobb, mert a célváltozó a 4 hibás soron is megfigyelt, és a teljes 2400 rendelésen számol. A `drop_bad_rows` 4 rendelést eldob; ha a 4 hiba éppen a késő kézbesítéssel függ össze, az arány torzulhat. A távolságmodellnél ez kevés: a 2 hiányzó `distance_km` a megtartó változatban is hiányként marad, tehát az elemzés csak 2398 soron használhatja a távolságot. A korrekt megoldás ilyenkor a hiányzó távolság pótlása a mérési folyamatból, nem a sor csendes törlése, vagy tudatos MNAR-modell és bizonytalanság jelentése.

13.4.8 Gyakori tévedés ellenpéldával

Gyakori tévedés, hogy a megfigyelt hiányzási mintából azonosítani lehet a mechanizmust. Vegyünk két régiót, mindkettőben 3 rendelést:

region	priority_flag	distance_km
north	0	12
north	1	15
north	0	hiány
south	1	20
south	0	24
south	1	hiány

A táblában mindkét régióban 1/3 a hiányzási arány. Ez MCAR-ral éppúgy konzisztens, mint azzal, hogy az északi hiányzó távolság valójában 5 km, a déli pedig 60 km, és a nagy távolságokat nem rögzítette a rendszer. A tábla ugyanaz; a mechanizmus mégis különböző. Ezért a hiányzási jelentésnek mindig fel kell tüntetnie a feltételezést, nem csak a százalékot. Az 1.4. alfejezet diagnosztikai felosztása itt is segít: ez adatértelmezési kérdés, nem pusztán programozási hiány. A 9.4. alfejezet numerikus fegyelme pedig arra figyelmeztet, hogy a 0-val töltés nem ártalmatlan skálázás, hanem a mérési szerződés megsértése.

13.4.9 Rövid összefoglaló

A hiányzásjelző R bináris mátrix, amelyből arányokat és csoportos feltételes arányokat számítunk; a mechanizmus azonban MCAR, MAR vagy MNAR feltételezés, nem a táblából bizonyított tény. A tartományellenőrzés a szerződésben lehetetlen értékeket jelöli, a kiugró ezzel szemben lehetséges, de szokatlan érték. A laborban 5 duplikált sor, 2 hiányzó `distance_km`, 1 negatív `basket_minor` és 1 ismeretlen ügyfél szerepel; a törléses és megtartásos tisztítás eltérő információszerkezetű, ezért érzékenységi összevetése kötelező. A 0-val töltés új adatot gyárt, a `dropna()` pedig nem dokumentálja a törlés okát. A következő 14.1. alfejezet a bináris jelzőket eseményként és feltételes valószínűséggel vizsgálja.

14 Valószínűség számítással és szimulációval

14.1 Esemény, feltételes valószínűség és Bayes

Belépő · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy

Előfeltétel: 13.4. alfejezet · Célkészség: Esemény, feltételes valószínűség és Bayes · Kimenet: működő kód és a labor ellenőrző tesztje.

14.1.1 Cél és konkrét helyzet

Egy logisztikai elemzőcsoport automatikus késésriasztó szabályt értékel. A megfigyelési egység egy leadott rendelés; a vizsgált 10 000 rendelés mindegyike pontosan egyszer szerepel. A döntéshozó a rendelés pillanatában látja a riasztást, a tényleges késés azonban csak később derül ki. Ezért a riasztó jele a predikció pillanatában ismert bemenet, a tényleges késés pedig az 1.1. alfejezetben megtanult értelemben utólag megfigyelt célváltozó. A bemeneti adat itt két bináris változó rendelésenként:

- L: a rendelés ténylegesen 7 napon túl kézbesült-e (1 = késés, 0 = nem);
- $+$: a riasztó szabály pozitívan jelzett-e (1 = riasztás, 0 = nincs riasztás).

Az alábbi értékeléshez a valós késés alaparánya (base rate) 1%, tehát a 10 000 rendelésből várhatóan 100 késik. A riasztó érzékenysége (sensitivity) 90%: a valóban késő rendelések 90%-án jelez. A specificitása (specificity) 95%: a nem késő rendelések 95%-án nem jelez, tehát 5%-án tévesen riaszt. A kérdés: ha a rendszer pozitívan riaszt, mekkora annak a feltételes valószínűsége, hogy a rendelés tényleg késni fog? Előzetes becslésként sokan a 90%-ra tippelnek, mert a riasztó érzékenysége 90%. A kézi táblázatból látni fogjuk, hogy a helyes válasz ennél sokkal kisebb. A példa darabszámai tankönyvi paraméterek, nem a `lab/data/orders.csv` fájlból számított mért statisztika.

14.1.2 Fogalmi magyarázat: eseménytér, együttes, feltételes, függetlenség, alapgyakoriság, Bayes

Eseménytér (sample space). Egy kísérlet összes lehetséges kimenetének halmaza. Egy szabályos kockadobásnál ez $\{1, 2, 3, 4, 5, 6\}$. A rendeléses példában minden rendelés négy elemi kimenet egyikébe esik:

- $L \cap +$: késik és riaszt a rendszer,
- $L \cap \neg +$: késik, de a rendszer nem riaszt,
- $\neg L \cap +$: nem késik, de a rendszer riaszt,
- $\neg L \cap \neg +$: nem késik és a rendszer sem riaszt.

Az eseménytér teljes: minden rendelés pontosan egy cellában van.

Együttes valószínűség (joint probability). Két esemény együttes bekövetkezésének valószínűsége: $P(L \cap +)$. Darabszámokból:

$$P(L \cap +) = \frac{N(L \cap +)}{N},$$

ahol $N(L \cap +)$ a metszetbe eső rendelések száma, N az összes rendelés száma. A jelölések mindvégig: N pozitív egész darabszám, a valószínűségek dimenziótlan számok 0 és 1 között. A példánk várható cellái:

Cell	L	$\neg L$	Sorösszeg
$+$	90	495	585
$\neg +$	10	9405	9415
Oszlopösszeg	100	9900	10 000

A darabszámok onnan jönnek: a 100 késés 90%-a, azaz 90 kap riasztást; a 9900 nem késés 5%-a, azaz 495 kap téves riasztást. Így az együttes gyakoriságok: $P(L \cap +) = 90/10000 = 0{,}009$; $P(\neg L \cap +) = 495/10000 = 0{,}0495$.

Feltételes valószínűség (conditional probability). Az A esemény valószínűségét egy másik, B eseményre szűkített részsokaságon nézzük:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{N(A \cap B)}{N(B)},$$

feltéve, hogy $P(B) > 0$. A függőleges vonal mögötti esemény a feltétel: ez határozza meg a nevezőt. A példában $P(+ \mid L) = 90/100 = 0{,}90$, ez az érzékenység. A kérdésünk viszont a másik irány: $P(L \mid +) = 90/585 \approx 0{,}1538$. Ugyanaz a metszet, de más a feltétel, ezért más a nevező.

Függetlenség (independence). Az A és B események akkor függetlenek, ha

$$P(A \cap B) = P(A)\,P(B),$$

vagy ha $P(B) > 0$, akkor ekvivalensen $P(A \mid B) = P(A)$. Tehát a B-re szűkítés nem változtatja A esélyét. Ellenpélda egy szabályos kockánál: legyen $A =$ páros dobás, $B = 6$-nál kisebb dobás. Ekkor $P(A) = 3/6 = 1/2$, $P(B) = 5/6$, de $P(A \cap B) = P(\{2,4\}) = 2/6 = 1/3$, míg $P(A)P(B) = 5/12 \neq 1/3$, tehát a két esemény nem független. A riasztó példánkban L és $+$ szándékosan nem függetlenek: a szabály éppen azért létezik, hogy a $+$ jel megváltoztassa a késésre vonatkozó bizonytalanságunkat.

Alapgyakoriság (base rate). Ez a feltétel nélküli, előzetes valószínűség: $P(L) = 0{,}01$. A ritka eseményeknél ez a nevezőben megbúvó domináns tényező. A 13.4. alfejezetben a feltételes hiányzási arányok hasonló logikával működtek: a feltétel megválasztása mindig a nevezőt jelöli ki.

Bayes-formula. A két irányú feltételes valószínűséget köti össze. Darabszámokból levezethető. Induljunk a definíciókból:

$$P(L \mid +) = \frac{N(L \cap +)}{N(+)} = \frac{N(L)\,P(+ \mid L)}{N(+)} = \frac{P(+ \mid L)\,P(L)}{P(+)}.$$

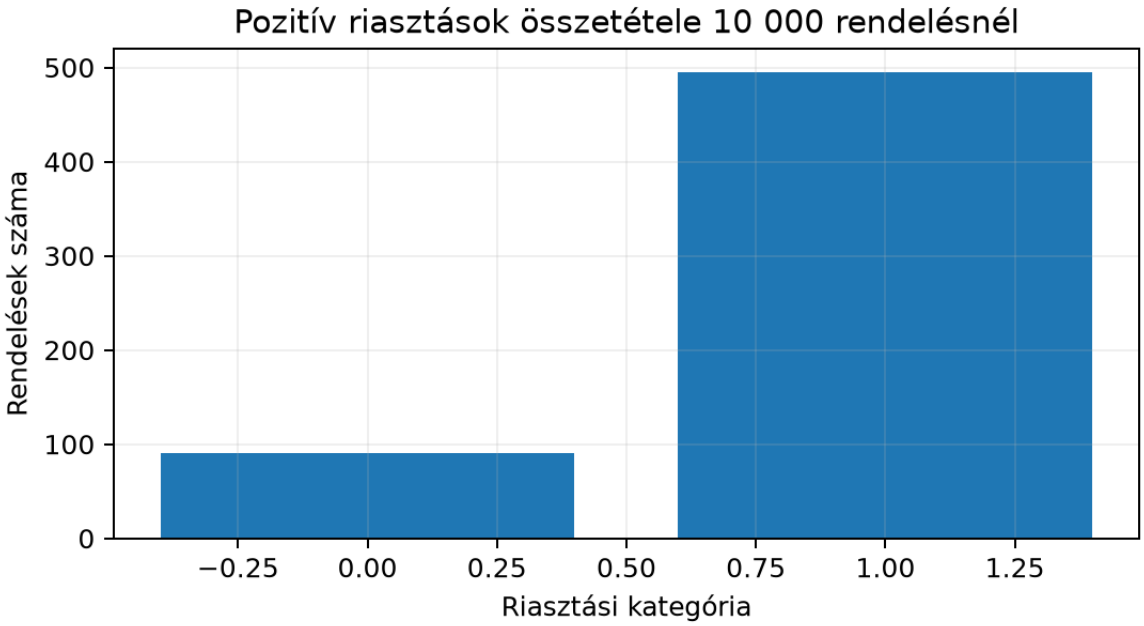
A nevező a teljes eseménytér felbontásából áll elő, mert minden pozitív riasztás vagy késő, vagy nem késő rendelésre esik:

$$P(+) = P(+ \mid L)\,P(L) + P(+ \mid \neg L)\,P(\neg L).$$

Behelyettesítve:

$$P(L \mid +) = \frac{P(+ \mid L)\,P(L)}{P(+ \mid L)\,P(L) + P(+ \mid \neg L)\,P(\neg L)} = \frac{0{,}90 \cdot 0{,}01}{0{,}90 \cdot 0{,}01 + 0{,}05 \cdot 0{,}99} = \frac{0{,}009}{0{,}0585} = \frac{2}{13} \approx 0{,}1538.$$

Az utolsó lépés kézi törtekkel is követhető: $0{,}009 = 9/1000$, $0{,}0585 = 585/10000$, a hányados $90/585$, egyszerűsítve $2/13$. A táblázat ugyanezt adja darabszámokból.



14.1. ábra: Pozitív riasztások összetétele 10 000 rendelésnél - Számított várható darabszámok: 0 = valódi késés, 1 = hamis riasztás. Nem mért adat.

14.1.2.1 Alapgyakoriság és explicit 2×2 nevezők

A pozitív prediktív érték (PPV) nevezője a **tesztpozitívak** száma, nem a teljes minta. A 2×2 tábla kitöltése ezt teszi láthatóvá:

	$Y = 1$ (valós pozitív)	$Y = 0$ (valós negatív)	Sorösszeg
Teszt pozitív	TP	FP	TP+FP
Teszt negatív	FN	TN	FN+TN
Oszlopösszeg	TP+FN	FP+TN	n

Legyen $n = 10000$, szenzitivitás 0.99, specificitás 0.95. Két prevalencia:

Prevalencia	TP	FP	FN	TN	PPV = TP/(TP+FP)	NPV = TN/(TN+FN)
1%	99	495	1	9405	99/594 = 0.1667	9405/9406 = 0.9999
10%	990	450	10	8550	990/1440 = 0.6875	8550/8560 = 0.9988

Ugyanaz a teszt 1%-os prevalenciánál a pozitívoknak csak 16.7%-át találja el, 10%-nál már 68.75%-át. A különbség nem a teszt hibája, hanem az alapgyakoriság és a nevező. A nulla tesztpozitív esetben a PPV nem értelmezhető; a kód `None`-t ad, nem `ZeroDivisionError`-t.

```
def metrics(prevalence, n, sensitivity, specificity):
    positives = round(prevalence * n)
    negatives = n - positives
    tp = round(positives * sensitivity)
    fn = positives - tp
    fp = round(negatives * (1 - specificity))
    tn = negatives - fp
    test_positive = tp + fp
    test_negative = fn + tn
    return {"TP": tp, "FP": fp, "FN": fn, "TN": tn,
            "PPV": tp / test_positive if test_positive else None,
            "NPV": tn / test_negative if test_negative else None,
            "PPV_denominator": test_positive,
            "NPV_denominator": test_negative}

print(metrics(0.01, 10000, 0.99, 0.95))
print(metrics(0.10, 10000, 0.99, 0.95))
print(metrics(0.0, 10000, 0.99, 1.0))  # PPV = None
```

A `lab/base_rate.py` `confusion()`, `metrics()` és `report()` függvényei a fenti két prevalenciát és a nulla nevezőt adják; a `tests/test_base_rate.py` a TP/FP/FN/TN cellákat, a 0.1667 és 0.6875 PPV-t, a helyes nevezőket és a `None`-t ellenőrzi.

14.1.3 Teljes minimális munkapélda

A példát a projektgyökérből futtatjuk, a `scripts/s053_bayes_screen.py` fájlból. A kód a NumPy 2.x `default_rng` interfészét használja; a `scipy.stats` csomagot itt még nem hívjuk, mert a 14.2. alfejezettől kezdjük majd az eloszlásmodelleket használni, most elegendő a generátor és a darabszámos összegzés. A generálás rögzített seeddel reprodukálható ugyanazon a NumPy-verzión; a PCG64 seed azonban nem garantál bitazonosságot minden platformon és verziókombinációban.

```
# scripts/s053_bayes_screen.py
import numpy as np

rng = np.random.default_rng(20261020)
n = 10_000
prevalence = 0.01
sensitivity = 0.90
specificity = 0.95

# A valós késés bináris tömbje, shape (n,), dtype bool.
actual_late = rng.random(n) < prevalence

# A riasztó döntése: késés esetén sensitivity eséllyel,
```

```
# nem késés esetén (1 - specificity) eséllyel lesz pozitív.
positive_alert = np.where(
    actual_late,
    rng.random(n) < sensitivity,
    rng.random(n) < (1.0 - specificity),
)

tp = int(np.sum(actual_late & positive_alert))
fn = int(np.sum(actual_late & ~positive_alert))
fp = int(np.sum(~actual_late & positive_alert))
tn = int(np.sum(~actual_late & ~positive_alert))

print(f"TP={tp} FN={fn} FP={fp} TN={tn}")
print(f"P(alert)={ (tp + fp) / n:.5f}")
print(f"P(late given alert)={tp / (tp + fp):.5f}")

# Elméletileg várt darabszámok:
# TP=0.01*0.90*10000=90, FN=10, FP=0.99*0.05*10000=495, TN=9405
# Elméleti posterior: 90/(90+495)=2/13   0.1538
```

A futtatási parancs Linux/macOS terminálban vagy Windows PowerShellben, munkakönyvtárnak a projektgyökeret választva:

```
uv run python scripts/s053_bayes_screen.py
```

A kód automatikusan még nem ellenőrzött, ezért pontos futási kimenetet nem közlünk. A kommentben szereplő értékek a paraméterekből levezetett elméletileg várt darabszámok. A tényleges szimuláció ettől néhány tucatnyi eltérést mutat, mert a seed véletlen mintát húz.

14.1.4 Eredmény értelmezése

Az értelmezéshez a várható táblázatot használjuk, mert az nem függ a szimulációs zajtól. A kérdéses mennyiség:

$$P(L \mid +) = \frac{90}{585} = \frac{2}{13} \approx 0{,}1538.$$

A riasztásoknak tehát várhatóan csak mintegy 15,4%-a valódi késés; a maradék 84,6% téves riasztás. Ez nem mond ellent annak, hogy az érzékenység 90%. Az érzékenység a késő rendelésekre mint feltételre vonatkozik, a kérdésünk viszont a riasztásra mint feltételre. A két feltételes valószínűség iránya különbözik.

Az elméleti táblázat invariáns ellenőrzései:

- sor- és oszlopösszegek: $90+10=100$, $495+9405=9900$, $90+495=585$, $10+9405=9415$, a teljes összeg 10 000;
- a posterior $2/13$ pontosan a két várt cella aránya;
- ha a kód lefut, a négy cella összege mindig 10 000 kell hogy legyen, mert a partíció teljes.

A szimuláció tényleges becslése csak közelíti a 2/13-ot. Az eltérés oka a binomiális mintavételi zaj; ennek formális standard errorját a 14.4. alfejezet és 15.3. alfejezet tárgyalja. A mostani példában a pontos konfidenciaintervallum helyett annyit mondunk ki: a fenti seeddel és paraméterekkel a posterior tipikusan 0,14 és 0,17 között mozog, de ez nem elméleti garantált intervallum. Ha a tanuló lokálisan lefuttatja, a saját mért száma a gép, a NumPy-verzió és a seed ismeretében írható ellenőrzött mérésként; a fenti kommentbeli érték nem ilyen.

A példa a Northstar predikciós szerződésével is összhangban áll: a riasztás a rendelés pillanatában ismert, a tényleges L pedig a kézbesítés után. Az itt használt teljes 10 000 soros kiértékelés utólagos, retrospektív elemzés; éles használat előtt időbeli splitet és label-availability szűrést kell végezni az 1.2. alfejezet és későbbi fejezetek szerint.

A pozitív prediktív érték a prevalenciától függ: a teszt szenzitivitását és specificitását változatlanul hagyva más populációban más posterior valószínűséget kapunk. A riasztási küszöböt a hibák költségeivel együtt kell értelmezni.

14.1.5 Feladatok

1. **Értelmezés: magasabb alapgyakoriság.** Ugyanaz a riasztó szabály egy sűrűbb időszakra kerül, ahol a késés alaparánya 5% a 10 000 rendelésen belül. Az érzékenység továbbra is 90%, a specificitás 95%. Töltsd ki a négycellás táblázat várható darabszámait, és számítsd ki $P(L \mid +)$ értékét. Hasonlítsd össze az 1%-os alapgyakoriságú eredménnyel, és magyarázd el, miért változott.

2. **Implementálás: függvény a táblázatból.** Írj `posterior_from_counts(tp, fn, fp, tn)` függvényt, amely a négy cella abszolút darabszámából visszaadja $P(L \mid +)$ értékét. Ha a pozitív riasztások száma nulla, a függvény `float("nan")` értéket adjon. Teszteld az elvárt táblázattal: `posterior_from_counts(90, 10, 495, 9405)` értéke $2/13$ legyen, és tesztelje az 1. feladat táblázatával is. Mit kell a tesztben elfogadható lebegőpontos toleranciának tekinteni?

3. **Hibakeresés.** Egy notebook cellában a következő kód szerepel:

```
p_wrong = np.sum(actual_late) / np.sum(positive_alert)
print(p_wrong)
```

A szerző azt állítja, hogy ez $P(L \mid +)$ becslése. Nevezd meg, mit számol valójában ez a hányados, és add meg a helyes kódot. A várható táblázat alapján miért nem egyenlő a kettő?

4. **Megváltozott feltétel: jobb specificitás.** A fejlesztőcsapat javítja a szabályt, így a specificitás 95%-ról 99%-ra nő, az érzékenység 90%, az alapgyakoriság marad 1%. Számítsd ki a várható TP, FN, FP, TN cellákat 10 000 rendelésre, és add meg az új $P(L \mid +)$ értéket. Mennyivel csökkent a hamis riasztások aránya a pozitív jelzéseken belül? Értelmezd, miért van ekkora hatása a specificitás javításának.

14.1.6 Megoldások és indoklás

1. feladat. A 10 000 rendelésből 500 késik, 9500 nem. A késők 90%-a, azaz 450 kap riasztást: $TP = 450$, $FN = 50$. A nem késők 5%-a kap téves riasztást: $FP = 9500 \cdot 0{,}05 = 475$, $TN = 9025$. A pozitív riasztások száma $450 + 475 = 925$, tehát

$$P(L \mid +) = \frac{450}{925} = \frac{18}{37} \approx 0{,}4865.$$

Ellenőrzés a sor- és oszlopösszegekkel: $450 + 50 = 500$, $475 + 9025 = 9500$, $450 + 475 = 925$, $50 + 9025 = 9075$, az összeg 10 000. Az 1%-os esethez képest a posterior 15,4%-ról 48,6%-ra nőtt, mert az alapgyakoriság ötszörösére emelésével a valódi pozitívok aránya a riasztásokon belül megnőtt: 90-ről 450-re, míg a hamis pozitívok csak kicsit csökkentek 495-ről 475-re. A ritka esemény kevésbé ritka lett, ezért a teszt pozitív eredménye erősebb bizonyítéknak bizonyult.

2. feladat. Egy lehetséges megoldás:

```python
def posterior_from_counts(tp, fn, fp, tn):
    positive_count = tp + fp
    if positive_count == 0:
        return float("nan")
    return tp / positive_count
```

A teszt:

```python
assert abs(posterior_from_counts(90, 10, 495, 9405) - 2/13) < 1e-12
assert abs(posterior_from_counts(450, 50, 475, 9025) - 18/37) < 1e-12
```

A tolerancia 10^{-12} elegendő, mert a hányados két Python-egész osztása, a lebegőpontos kerekítés abszolút hibája ennél nagyságrendekkel kisebb; a `2/13` és a `18/37` is ugyanabban a float-reprezentációban áll elő, mint a cellák hányadosa. Az ellenőrző invariant: a négy bemenet nemnegatív egész, és a `positive_count = tp + fp` kell hogy legyen a pozitív sor összege; a függvény a `tp / (tp + fp)` arányt adja vissza. A `fn` és `tn` paraméterekre nincs szükség, de a szignatúrában hagyjuk őket, mert a teljes táblázat explicit kontroll, és a jövőben a negatív prediktív értéket is ugyanebből számítjuk.

3. feladat. A hibás kód a $\sum L$ és $\sum +$ arányát számolja, ami a várható táblázatban $100/585 \approx 0{,}171$. Ez nem $P(L \mid +)$, mert a számláló az összes késő rendelést tartalmazza, nem csak azokat, amelyekre a riasztó is jelzett. A $P(L \mid +)$ képletében a számláló $N(L \cap +)$, a nevező $N(+)$. A helyes kód:

```python
p_right = (
    np.sum(actual_late & positive_alert)
    / np.sum(positive_alert)
)
```

A különbség forrása a 10 hamis negatív: azok a rendelések késnek, de a riasztó nem jelzett. A hibás számlálóban benne vannak, a nevezőben viszont a pozitív riasztások szerepelnek, így a kettő hányadosa nem feltételes arányt, hanem két különböző partíció méretének arányát adja. Ha a pozitív riasztások száma nulla lenne, a helyes függvény `nan`-t adna, a hibás változat pedig nullával osztana.

4. feladat. Az új specificitás mellett a hamis pozitív arány $1 - 0{,}99 = 0{,}01$. A késők száma 100, ebből $TP = 90$, $FN = 10$. A nem késők száma 9900, ebből $FP = 99$, $TN = 9801$. A pozitív riasztások száma $90 + 99 = 189$, így

$$P(L \mid +) = \frac{90}{189} = \frac{10}{21} \approx 0{,}4762.$$

A korábbi 15,4%-ról 47,6%-ra nőtt a posterior. A hamis riasztások aránya a pozitív jelzéseken belül korábban $495/585 \approx 84{,}6\%$, most $99/189 \approx 52{,}4\%$. A javulás mechanizmusa: a nagy létszámú nem késő csoportban a téves riasztási arány 5%-ról 1%-ra csökkent, ezért a hamis pozitívok darabszáma 495-ről 99-re esett. Mivel a valódi pozitívok száma 90, a specificitás javításával a hamis pozitívok még mindig kissé többen vannak, de a riasztás már csaknem minden másodszor igaz. A ritka eseménynél a hamis pozitív csökkentése gyakran nagyobb hatással van a posteriorra, mint az érzékenység további növelése.

14.1.7 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb hiba a feltétel irányának megfordítása: aki a 90%-os érzékenységből kiindulva azt mondja, hogy pozitív riasztás esetén 90% a késés esélye, az $P(+ \mid L)$ értékét $P(L \mid +)$-ként olvassa. A táblázat ellenpéldája: a 585 pozitív riasztásból csak 90 igaz, azaz $P(L \mid +) = 15{,}4\%$, miközben $P(+ \mid L) = 90\%$. Az irány felcserélése a feltétel nevezőjének felcserélését jelenti. Még erősebb ellenpélda: ha a riasztó mindig pozitívan jelezne, akkor $P(+ \mid L) = 1$, de $P(L \mid +)$ egyszerűen az alapgyakoriság, 1% lenne.

A korábbi fejezetekkel a kapcsolat közvetlen. A 13.4. alfejezetben a hiányzásjelző feltételes arányai ugyanilyen metszet/nevező logikával készültek: az $R = 1$ feltétel jelölte ki a nevezőt. A 13.1. alfejezetben a szelekciós torzítás is feltételes arányként írható le: a megfigyelt almintára vonatkozó eloszlás nem azonos a sokaság eloszlásával, ha a bekerülés mechanizmusa összefügg a vizsgált változóval. A 2.1. alfejezet projektgyökér-konvenciója miatt a fenti script a projektgyökérből futtatható, a relatív útvonalakat így egy helyen rögzítjük.

Önellenőrző kérdések:

1. Mi a nevező a $P(L \mid +)$ definíciójában, és hány rendelés van benne a példában?
2. Mit jelent a $P(+ \mid \neg L)$ érték, és mi a kapcsolata a specificitással?
3. Ha két esemény független, mit tudunk $P(A \mid B)$ és $P(A)$ viszonyáról?
4. Miért nem mondhatjuk, hogy a negatív riasztás esetén a késés esélye 10%?

A negatív riasztásra a példában $P(L \mid \neg +) = 10/9415 \approx 0{,}00106$, tehát 0,1%, nem 10%. Ez is a feltétel irányának és a nagy, nem késő csoportnak a következménye.

14.1.8 Rövid összefoglaló

Az eseménytér a lehetséges kimenetek partíciója; az együttes gyakoriság a metszetbe eső darabszám osztva a teljes elemszámmal. A feltételes valószínűség ugyanezt a metszetet szűkebb nevezőre vetíti, ezért $P(A \mid B)$ és $P(B \mid A)$ általában különbözik. Bayes formulája a két irányt köti össze, és a nevezőt a feltétel szerinti teljes felbontásból állítja elő. A ritka késés példájában a 90%-os érzékenység ellenére a pozitív riasztás után csak 15,4% a valódi késés esélye, mert a nagy nem késő csoporton keletkező hamis pozitívok uralják a riasztásokat. A következő 14.2. alfejezet ezeket az eseményeket véletlen változókká alakítja, és formálisan bevezeti a Bernoulli- meg binomiális eloszlást, amellyel a mostani szimuláció szórása is pontosan leírható.

14.2 Véletlen változó és gyakori eloszlásmodellek

> **Belépő · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy**
>
> Előfeltétel: 14.1. alfejezet · Célkészség: Véletlen változó és gyakori eloszlásmodellek · Kimenet: működő kód és a labor ellenőrző tesztje.

A 14.1. alfejezetben eseményekkel dolgoztunk: minden egyes megfigyelés vagy bekövetkezett, vagy nem. Most egy lépéssel általánosabbra lépünk: az esemény kimenetét egy **véletlen változó** (random variable) értékeként kezeljük. A véletlen változó nem „változó” a Python-értelemben, hanem egy leképezés: minden lehetséges eseményhez egy számot rendel. Ha a kimenet maga is szám, például egy megadott időablakban beérkező rendelések darabszáma, akkor közvetlenül kapunk véletlen változót; ha a kimenet kategória, például `consumer` vagy `business`, akkor a hozzárendelés mesterséges, például 0/1 kód.

A véletlen változó **eloszlása** (distribution) azt határozza meg, mekkora valószínűség tartozik az értékeinek halmazaihoz. Diszkrét esetben az egyes értékek valószínűségét tömegfüggvény, folytonos esetben az intervallumok valószínűségét a sűrűség integrálja adhatja meg. A gyakorlatban az eloszlást két dolgon keresztül használjuk: a **paramétereken** (parameters) keresztül, amelyek az eloszlás alakját meghatározó számok, és a **szimuláción** keresztül, amellyel mesterséges mintákat állítunk elő a modellből.

14.2.1 1. Cél és konkrét helyzet

A kezdőlabor `orders.csv` adata rendelésenként tartalmazza a `priority_flag` oszlopot, amely 0 vagy 1. A kérdés: hány prioritásos rendelés várható 100 egymástól függetlennek tekintett rendelésből, ha a prioritásos rendelések aránya 0,22? Az elemzési egység: egy rendelés; a cél egy darabszám előrejelzése és annak bizonytalansága. A binomiális példában külön **modellfeltevés**, hogy minden rendelés azonos $p = 0{,}22$ valószínűséggel és egymástól függetlenül prioritásos. Valós rendeléseknél ezt ügyfél- és időbeli függőség sértheti; ekkor a binomiális variancia nem alkalmazható automatikusan.

Az előzetes becslés: ha egy rendelésnél a prioritásos valószínűsége $p = 0{,}22$, akkor 100 rendelésből átlagosan $100 \cdot 0{,}22 = 22$ prioritást várunk. Azt azonban még nem tudjuk, mennyire megbízható ez a 22-es szám, vagy milyen gyakran kapnánk ennél többet. Ezt az alfejezetben pontosítjuk.

14.2.2 2. Fogalmi magyarázat

14.2.2.1 Bernoulli-eloszlás

A legkisebb véletlen változó, egyetlen 0/1 kimenetű kísérletet ír le. Legyen X a kimenet. A **Bernoulli-eloszlás** (Bernoulli distribution) egyetlen paramétere a „siker” valószínűsége, jelölése:

$$p \in (0, 1)$$

Ekkor:

$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$

A változó értékkészlete $\{0, 1\}$, a várható érték $E[X] = p$, a variancia pedig $\mathrm{Var}(X) = p(1 - p)$. A mértékegység: a valószínűség dimenziótlan, az X értékei pedig darabszámok (0 vagy 1).

Kézi kis példa: $p = 0{,}3$ esetén a várható érték 0,3, a variancia $0{,}3 \cdot 0{,}7 = 0{,}21$, a szórás $\sqrt{0{,}21} \approx 0{,}458$. Ha nem tudjuk, hogy a folyamat Bernoulli, hanem csak a 0/1 kimenetet látjuk, akkor a megfigyelt arány a paraméter becslése, de ez már a becslés témaköréhez tartozik.

14.2.2.2 Binomiális eloszlás

Ha n független Bernoulli-kísérletet végzünk ugyanazzal a p valószínűséggel, és az X véletlen változó a sikerek **darabszáma**, akkor **binomiális eloszlású** (binomial distribution) n és p paraméterekkel:

$$X \sim \text{Binomial}(n, p)$$

A lehetséges értékek $0, 1, \ldots, n$, a várható érték és variancia:

$$E[X] = np, \quad \text{Var}(X) = np(1-p)$$

A momentumokból látszik, hogy a binomiális az egyedi Bernoulli-kísérletek összegzéséből adódik: n darab független Bernoulli-kísérlet összege. A pontos valószínűségek kiszámításához a **valószínűségi tömegfüggvényt** (probability mass function, PMF) használjuk:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, \ldots, n$$

Itt $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ az „n alatt k” binomiális együttható, amely megadja, hányféleképpen választható ki k siker n kísérletből.

Kézi kontroll a $n = 2$, $p = 0{,}3$ esetben: $P(X = 0) = 0{,}7^2 = 0{,}49$, $P(X = 1) = 2 \cdot 0{,}3 \cdot 0{,}7 = 0{,}42$, $P(X = 2) = 0{,}3^2 = 0{,}09$. Az összegnek 1-nek kell lennie: $0{,}49 + 0{,}42 + 0{,}09 = 1$.

14.2.2.3 Poisson-eloszlás

A binomiális folyamat azt feltételezi, hogy a kísérletek száma n rögzített, és a siker valószínűsége p ismert. Sok valós folyamatban azonban az események időben folytonosan érkeznek, és nincs természetes „kísérletszám”. Ilyenkor a **Poisson-eloszlás** (Poisson distribution) használható, amely egyetlen paraméterrel, a várható eseményszámmal rendelkezik:

$$X \sim \text{Poisson}(\lambda), \quad \lambda > 0$$

A paraméter λ (lambda) az adott időablakban várható események száma; homogén, állandó intenzitású Poisson-folyamatnál az időablak hosszával arányosan változik. Időben változó intenzitásnál az adott ablak paramétere az intenzitás idő szerinti integrálja. A várható érték és variancia egyaránt:

$$E[X] = \lambda, \quad \text{Var}(X) = \lambda$$

Ez a Poisson-kritikus tulajdonság: a variancia egyenlő a várhatóval, ezért az eseményszám szórása $\sqrt{\lambda}$. A PMF:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \ldots$$

A nevezőben $k!$ a faktoriális, $e = 2{,}71828\ldots$ az exponenciális függvény alapja. Homogén Poisson-folyamat esetén diszjunkt időintervallumok eseményszámai függetlenek, és az intenzitás állandó.

Nem homogén Poisson-folyamatban az intenzitás változhat, miközben az adott intervallum darabszáma továbbra is Poisson-eloszlású, paramétere az intenzitás intervallum feletti integrálja. A csomósodás, függőség és túlszóródás miatt viszont a Poisson-modell alkalmatlan lehet; ilyenkor például negatív binomiális modell vagy más pontfolyamat szükséges.

Kézi kontrol: $\lambda = 2$ esetén $P(X = 0) = e^{-2} = 0{,}1353$, $P(X = 1) = 2e^{-2} = 0{,}2707$, $P(X = 2) = \frac{4}{2}e^{-2} = 0{,}2707$. Az összeg az összes k-ra pontosan 1, mert a Taylor-sor: $\sum_{k=0}^{\infty} \frac{\lambda^k}{k!} = e^{\lambda}$.

14.2.2.4 Normális eloszlás

A **normális eloszlás** (normal distribution, gyakran Gauss-görbe) folytonos eloszlás, tehát a lehetséges értékek egy valós intervallumot fednek le, és a valószínűséget **sűrűségfüggvénnyel** (probability density function, PDF) adjuk meg. Két paramétere van: a várható érték μ és a szórás $\sigma > 0$:

$$X \sim \text{Normal}(\mu, \sigma)$$

A PDF:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

A PDF használata a folytonos esetben eltér a PMF-től: a sűrűség nem közvetlen valószínűség, hanem a terület az intervallum alatt adja a valószínűséget. A 12.1. alfejezetben már láttuk a terület-összeget; itt a normalizáló konstans $1/(\sigma\sqrt{2\pi})$ biztosítja, hogy a teljes terület 1 legyen. A paraméterek jelentése: μ az eloszlás „közepe”, σ a szórás, tehát az eloszlás skáláját (nem az adatok teljes terjedelmét) adja meg. A normális eloszlás szimmetrikus, ami azt jelenti, hogy a medián egyenlő a várhatóval.

Kézi ellenőrzés a standard esetben: $\mu = 0$, $\sigma = 1$ esetén a PDF a 0 pontban $f(0) = 1/\sqrt{2\pi} \approx 0{,}399$, 1-nél $f(1) = 1/\sqrt{2\pi} \cdot e^{-1/2} \approx 0{,}242$. A szimmetria nem számítható közvetlenül, de a PDF páros függvény.

A CDF (cumulative distribution function) a valószínűséget adja, hogy a változó legfeljebb x:

$$F(x) = P(X \le x) = \int_{-\infty}^{x} f(t)\,dt$$

Ez az integrál nem zárható egyszerű formulával, ezért numerikusan vagy táblázatból határozzuk meg. A `scipy.stats.norm.cdf` végzi ezt.

14.2.2.5 Lognormális eloszlás

Sok mennyiség, például a rendelés értéke, pozitív és erősen jobbra ferde, tehát a nagy értékek ritkák, de nagyon nagyok is lehetnek. Ha a mennyiség logaritmusa normális, akkor magát a változót **lognormális eloszlásúnak** (lognormal distribution) nevezzük. Paraméterként leggyakrabban a log-skálán értelmezett várható értéket, μ_l, és szórást, $\sigma_l > 0$ adjuk meg:

$$\ln(X) \sim \text{Normal}(\mu_l, \sigma_l) \iff X \sim \text{Lognormal}(\mu_l, \sigma_l)$$

A várható érték és variancia az eredeti skálán:

$$E[X] = \exp(\mu_l + \tfrac{1}{2}\sigma_l^2), \quad \mathrm{Var}(X) = (e^{\sigma_l^2} - 1)e^{2\mu_l + \sigma_l^2}$$

Figyeljük meg, hogy a várható érték nagyobb, mint e^{μ_l}, mert a logaritmikus szimmetriát az exponenciális torzítja. A lognormális modellezésnél a paraméterek mindig log-skálán értendők, nem az eredeti skálán.

Kézi példa: $\mu_l = 0$, $\sigma_l = 1$ esetén $E[X] = e^{0,5} \approx 1{,}65$, $\mathrm{Var}(X) = (e-1)e \approx 4{,}67$. Ez jól mutatja, hogy a ferdeség miatt a várható nagyobb, mint a tipikus középpont.

14.2.2.6 Paraméterek általában

A **paraméter** (parameter) az eloszláscsalád azonosítására szolgáló szám. A Bernoulli-nak egy paramétere van (p), a binomiálisnak kettő (n, p), a Poissonnak egy (λ), a normálisnak kettő (μ, σ), a lognormálisnak kettő (μ_l, σ_l). A paraméterek rögzítésével az eloszlás teljesen meghatározott. Fontos különválasztani a paramétert és annak becslését: a paraméter a valódi, általában ismeretlen szám, a becslés (estimate) a mintából számított érték. Ez a különbség a 15.1. alfejezet központi témája lesz.

14.2.2.7 Melyik modell alkalmatlan? Support, mean és variance

Az eloszlásmodell nem „illeszthető képlet”, hanem az adatgeneráló folyamat feltevése. Négy gyakori modell:

Modell	Support	Átlag	Variancia	Mikor alkalmatlan
Bernoulli	$\{0, 1\}$	p	$p(1-p)$	nem bináris kimenet
Binomiális	$0, \ldots, n$	np	$np(1-p)$	nem rögzített n, vagy n-nél nagyobb érték
Poisson	$0, 1, 2, \ldots$	λ	λ	overdiszperzió: variancia $\gg$ átlag
Normális	$\mathbb{R}$	μ	σ^2	pozitív, ferde adat; negatív tömeg
Lognormális	$(0, \infty)$	$e^{\mu+\sigma^2/2}$	—	nulla/negatív érték, szimmetrikus adat

Példák:

Adat	Átlag	Variancia	Poisson?	Normális?	Lognormális?
napi eseményszám	2	8	nem (variancia 4×)	nem (diszkrét)	nem
pozitív kosárérték	100	22500	nem	nem ($\mu - 3\sigma < 0$)	igen
bináris kattintás	0.22	0.1716	nem	nem	nem

A Poisson varianciája megegyezik az átlaggal; ha a variancia négyszeres, overdiszperzió van, és negatív binomiális modell vagy kevert modell kell. A normális modell pozitív, erősen ferde adatra negatív értékeket is valószínűsít, ami fizikailag lehetetlen; a lognormális ezt elkerüli. A binomiális modell a rögzített n-hez kötött, a Poisson nem.

```
import numpy as np

def moments(values):
    values = np.asarray(values, dtype=float)
    return values.mean(), values.var(ddof=1)

def poisson_ok(values, tolerance=0.5):
    values = np.asarray(values, dtype=float)
    if np.any(values < 0) or np.any(values != np.round(values)):
        return False
    mean, var = moments(values)
    return abs(var - mean) <= tolerance * max(1.0, mean)

def normal_ok_for_positive(values, z=3.0):
    values = np.asarray(values, dtype=float)
    mean, var = moments(values)
    return mean - z * np.sqrt(var) > 0

overdispersed = np.array([0, 0, 0, 10, 10])
positive = np.array([50, 100, 150])
assert not poisson_ok(overdispersed)
assert not normal_ok_for_positive(positive)
assert np.all(positive > 0)
```

A `lab/distribution_choice.py` `moments()`, `poisson_ok()`, `binomial_ok()`, `normal_ok_for_positive()` és `classify()` függvényei ezt a döntést adják; a `tests/test_distribution_choice.py` az overdiszperziót, a pozitív adat normális-elutasítását és a binomiális támogatási korlátot ellenőrzi.

14.2.3 3. Teljes minimális munkapélda

A laborban szimulálunk három folyamatot: (1) egyetlen kattintásos konverziót $p = 0{,}22$ valószínűséggel; (2) kattintások számát egy óra alatt Poisson-paraméterrel $\lambda = 12$; (3) kosárértéket lognormális eloszlással, log-skálán $\mu_l = 3{,}5$, $\sigma_l = 0{,}7$. A NumPy `Generator` osztályát használjuk az új, ajánlott véletlenszám-keretrendszerből; a `scipy.stats` modul pedig az eloszlások objektumait adja. A seed beállítása a reprodukálhatóságot szolgálja.

A kód az `s054` munkavégzési helyébe helyezhető; `lab/pyproject.toml` alapkörnyezetben fut. A teljes útmutatót a `numpy` és `scipy` importálásával kezdjük.

```python
# s054_demo.py
import numpy as np
from scipy import stats

rng = np.random.default_rng(20261020)

# 1. Bernoulli szimuláció: 100 független kattintás, p=0,22
p = 0.22
n = 100
bernoulli_draws = rng.binomial(n=1, p=p, size=n)  # mindegyik Bernoulli
priority_count = bernoulli_draws.sum()

# 2. Binomiális PMF kontrol: P(X=20)
pmf_20 = stats.binom.pmf(20, n=100, p=p)

# 3. Poisson szimuláció: egy óra kattintás,  =12
lambda_per_hour = 12
poisson_draw = rng.poisson(lambda_per_hour)

# 4. Lognormális szimuláció: 1000 kosárérték
mu_l, sigma_l = 3.5, 0.7
basket_values = rng.lognormal(mean=mu_l, sigma=sigma_l, size=1000)

# 5. Elméleti összehasonlítások
expected_binom = n * p
sd_binom = np.sqrt(n * p * (1 - p))
expected_poisson = lambda_per_hour
sd_poisson = np.sqrt(lambda_per_hour)
expected_lognormal = np.exp(mu_l + sigma_l**2 / 2)
sd_lognormal = np.sqrt((np.exp(sigma_l**2) - 1) * np.exp(2*mu_l + sigma_l**2))

print("Priority count szimulált:", priority_count,
      "| várt:", expected_binom, "| sd:", sd_binom)
print("PMF P(X=20):", pmf_20)
print("Poisson óra szimulált:", poisson_draw,
      "| várt:", expected_poisson, "| sd:", sd_poisson)
print("Lognormális: átlag:", basket_values.mean(),
      "| várt:", expected_lognormal)
print("Lognormális: szórás:", basket_values.std(ddof=1),
      "| várt:", sd_lognormal)
```

A kimenet seedtől függ; a 20261020 seeddel várható értékek a fent számolt momentumok körül lesznek, de nem pontosan, mert véletlen mintáról van szó. a `priority_count` például a várható 22 körüli, mintegy egy szórásnyi 18–26-os sávban is lehet, a Poisson 7 és 17 között, a lognormális mintaátlag pedig az elméleti 42,31 körül. A futás után ne csak a számokat nézzük: a szórásképletek ellenőrzik, hogy a szimulált átlag konzisztens az elméleti paraméterrel.

14.2.4 4. Eredmény értelmezése

A fenti kód a véletlen változókat szimulálja, de a legfontosabb az, hogy a szimulált értékeket elméleti momentumokkal hasonlítjuk. A binomiális esetben a várható darabszám $np = 22$, a szórás $\sqrt{np(1-p)} \approx 4{,}14$. Ez azt jelenti, hogy a 100-as mintában a prioritások száma leggyakrabban 22 körül lesz, és egy szórásnyi (körülbelül 18–26 közötti) sávba is gyakran eshet, de ez nem 90–95%-os lefedettségű intervallum. A PMF adott k-ra megadja a pontos valószínűséget; a `stats.binom.pmf(20, n, p)` példa kézzel is számolható: $\binom{100}{20} 0{,}22^{20} 0{,}78^{80}$. Ez a szám pozitív; a diszkrét változónál egy adott értékhez is rendelhető pozitív valószínűség, szemben a folytonos eloszlások pontvalószínűségével.

A Poisson-eloszlásnál $\lambda = 12$ esetén a szimulált óra kattintásszáma átlagosan 12, szórása is $\sqrt{12} \approx 3{,}46$. A lognormális eloszlásnál az elméleti átlag $e^{3{,}5+0{,}7^2/2} \approx 42{,}31$, az elméleti szórás pedig $\approx 33{,}64$. Az 1000 elemű mintából számított átlag és szórás ezektől véletlenszerűen eltérhet; ez jól mutatja a pozitív ferdeséget. A kódban a `ddof=1` a mintavariancia torzítatlan becsléséhez kell, amint azt a 13.2. alfejezet is tárgyalta.

Invariant ellenőrzés: a Poisson-szimuláció k értékére $P(X = k) = \lambda^k e^{-\lambda}/k!$ teljesül, és a szimulált átlag közelíti λ-t, ha a minta nagy. A binomiális összeg a Bernoulli-darabok összege, ezért a darabszám `bernoulli_draws.sum()` összhangban van a `rng.binomial(n, p)` hívással; a kettő ugyanazt a modellt testesíti meg.

14.2.5 5. Feladatok

1. **Értelmezés:** a kezdőlabor szerződése szerint a `delivery_days` pozitív egész. Miért nem helyes azt feltételezni, hogy a `delivery_days` Poisson-eloszlású? Adj meg egy olyan folyamatot, amelyre a Poisson-modell természetesebb lenne, és egy feltevést, amelyet ellenőrizni kell.
2. **Megvalósítás:** írjon függvényt, amely `n` és `p` bemenettel kiszámítja a binomiális eloszlás PMF-jét k-ra a képlet alapján NumPy nélkül, majd hasonlítsa össze a `stats.binom.pmf(k, n, p)` értékkel. A relatív hiba 1e-10 alatt legyen. Miért nem célszerű ezt a képletet nagy n-re számolni?
3. **Hibadiagnózis:** a `scipy.stats.lognorm` osztály `s`, `scale` paraméterei nem közvetlenül μ_l és σ_l; ha valaki `stats.lognorm.pdf(x, mu_l, sigma_l)`-nek nevezi, hibás eredményt kap. Hogyan kell a paramétereket megfeleltetni, hogy az `pdf` a fenti μ_l, σ_l-nek feleljen meg? Adj explicit kódot, amely a PDF értékét kiszámítja $x = 1$ esetén.
4. **Átvitel megváltozott feltételre:** a webshop forgalma este a nappali háromszorosa. Ha napközben a kattintások száma Poisson-eloszlású $\lambda = 10$, akkor mi a helyes eloszlásmodell estére, ha az időablak fele a nappalinak? Milyen Poisson-feltevés sérülhet a valóságban, és miért? Vezesse le a várható értéket.

14.2.6 6. Megoldások és indoklás

1. A `delivery_days` napokban mért mennyiség nem eseménydarab, hanem időtartam; a Poisson eloszlás feltételezi, hogy az esemény bekövetkezése független és állandó intenzitással érkezik, ami egy szállítási határidőnél nem áll fenn, mert a kézbesítés egy folyamat, nem véletlen események diszkrét száma. A Poisson természetesebb lenne például a naponta beérkező rendelések számára. A feltevés: a rendelések egymástól függetlenül, konstans arányban érkeznek; ezt ellenőrizhetjük az időbeli ábrázolással és az érkezési idők exponenciális eloszlásával.
2. A PMF kézzel:

```
import math

def binom_pmf(k, n, p):
    coef = math.comb(n, k)
    return coef * (p ** k) * ((1 - p) ** (n - k))

from scipy.stats import binom
# Ellenőrzés
rel_err = abs(binom_pmf(20, 100, 0.22) - binom.pmf(20, 100, 0.22)) / binom.pmf(20, 100, 0.22)
```

```
print(rel_err)  # kicsi, de nem feltétlenül pontosan 0 a lebegőpontos számítás miatt
assert rel_err < 1e-10
```

Nagy n-re a faktoriális túlcsordul vagy numerikusan instabil, mert $n!$ és a hatványok óriásiak lesznek, és a szorzatukat hibatűrő módon nehéz kiszámítani; a SciPy belsőleg stabil algoritmust használ.

3. A `scipy.stats.lognorm` a `s` (shape) paramétert a log-skálán értelmezett szórásként, a `scale` paramétert pedig e^{μ_l}-ként definiálja. Helyes hívás `x=1` esetén:

```
import numpy as np
from scipy import stats

mu_l, sigma_l = 3.5, 0.7
s = sigma_l
scale = np.exp(mu_l)
pdf_1 = stats.lognorm.pdf(1, s=s, scale=scale)
print(pdf_1)  # szemléltető, elméletileg: (1/(1*sigma_l*sqrt(2 )))*exp(-(ln1-mu_l)^2/(2 ^2))
```

A hiba oka: ha `stats.lognorm.pdf(1, mu_l, sigma_l)`-t hívnánk, az a `mu_l`-t tekintené az s paraméternek és `sigma_l`-t a `scale`-nek, ami teljesen más eloszlást ad.

4. Nappali Poisson $\lambda_{\text{day}} = 10$ az eredeti időablakban. Az esti kattintási **ráta** ennek háromszorosa, az esti megfigyelési ablak pedig fele olyan hosszú, ezért a várható eseményszám $\lambda_{\text{evening}} = 10 \cdot 3 \cdot (1/2) = 15$. Az esti darabszám modellje Poisson(15), várható értéke 15. A Poisson-feltevés sérülhet, ha az esti kattintások korrelálnak, például azonos kampány vagy hír miatt. Ilyenkor az események nem függetlenek; ez túlszóródáshoz (overdispersion) vezet, amit a variancia nagyobb, mint a várható. A variancia ellenőrzése a 13.2. alfejezetből ismert módon elvégezhető a mintából.

14.2.7 7. Gyakori tévedés és ellenpélda

A leggyakoribb hiba az, hogy a minta hisztogramjára ránézve azt mondjuk: „ez normális”. Például a szállítási napok hisztogramja hasonló lehet egy haranggörbéhez, de ettől még a folyamat nem folytonos, és a normális modell negatív értékeket is megenged. A `delivery_days` pozitív egész, ezért a normális modell semmiképpen sem helyes; legfeljebb közelítő. Hasonlóképpen, ha a kosárértékre lognormálist illesztünk, a modell neve nem bizonyítja, hogy az adat tényleg lognormális: a paraméterek becslése és a diagnosztikai ellenőrzés szükséges. Az eloszlásválasztás ezért ellenőrizendő modellfeltevés, nem pusztán a hisztogram alakjából leolvasható tény.

Másik ellenpélda: a binomiális és a Poisson összekeverése. Ha n rögzített, például 100 rendelés, és a prioritás aránya p, akkor binomiálist kell használni; ha az események száma maga a megfigyelt eredmény, például a percenkénti hívások, akkor a Poisson a kiinduló modell. A binomiális n paraméterét nem szabad a Poisson λ-jával felcserélni.

14.2.8 8. Rövid összefoglaló

A véletlen változó leképezés az eseménytérről a számokra; eloszlása a valószínűségek teljes leírása. A Bernoulli egyetlen 0/1 kimenetet modellez, a binomiális ennek független összegzése n kísérletből, a Poisson az időben ritka független események darabszámát adja, a normális a folytonos szimmetrikus, a lognormális a pozitív ferde mennyiségeket. A paraméterek rögzítik az eloszlást, de becslésük mintából már a statisztikai következtetés témája. A szimuláció megmutatja az eloszlás alakját, a momentumok pedig ellenőrző invariánsok. A következő alfejezet

a függetlenség és kovariancia fogalmát viszi tovább, amely nélkül a binomiális és Poisson-feltevések érvényessége sem dönthető el.

A Poisson-modellben a variancia és a várható érték egyenlő; túlszóródás esetén negatív binomiális vagy más modell lehet megfelelőbb. Az eloszlás paramétereit mindig a konkrét megfigyelési időablakkal együtt adjuk meg.

14.3 Függetlenség, kovariancia és függő adat

Belépő · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy

Előfeltétel: 14.2. alfejezet · Célkészség: Függetlenség, kovariancia és függő adat · Kimenet: működő kód és a labor ellenőrző tesztje.

14.3.1 Cél és konkrét helyzet

A Northstar `orders.csv` minden sora egy rendelés, de ugyanahhoz a `customer_id`-hoz több sor is tartozhat. Ha egy elemzésben a 2400 rendeléssort úgy kezeljük, mintha 2400 független ügyfélmegfigyelés lenne, alábecsülhetjük a becsléseink bizonytalanságát. Ebben az alfejezetben két dolgot választunk szét: mit jelent két számszerű változó **kovarianciája** és **korrelációja**, és mit jelent az, hogy a sorok nem függetlenek, mert ismételt mérések vannak ugyanazon az ügyfélen.

Bemenetnek tekintjük a rendelésenkénti `quantity_sum` és egy másik numerikus jellemző, például `basket_minor` vektorát. Megfigyelési egység a sor: egy rendelés. A következtetés szempontjából viszont a **független mintavételi egység** gyakran az ügyfél, nem a rendelési sor. Előzetes becslés: a rendelésszintű korreláció lehet részben ügyfélen belüli hasonlóságból származó, ezért a korreláció értékét is feltételesen kell értelmeznünk. A cél az, hogy egy későbbi bootstraphez vagy teszthez helyesen jelöljük meg, mit szabad újramintavételezni.

14.3.2 Fogalmi magyarázat

14.3.2.1 Kovariancia mint centrált skalárszorzat

Legyen $\mathbf{x} = (x_1, \dots, x_n)$ és $\mathbf{y} = (y_1, \dots, y_n)$ két azonos hosszúságú megfigyelésvektor, mindkettő alakja $(n,)$. Jelölje $\bar{x}$ és $\bar{y}$ a két átlagot. A **kovariancia** (covariance) a centrált vektorok skalárszorzatának átskálázott alakja:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) = \frac{1}{n-1} \, \mathbf{x}_c \cdot \mathbf{y}_c,$$

ahol $\mathbf{x}_c = \mathbf{x} - \bar{x}$ az átlaggal centrált vektor, $\mathbf{y}_c = \mathbf{y} - \bar{y}$ hasonlóan. A mértékegység az x és y mértékegységének szorzata: például darab × EUR-cent. A pandas `Series.cov` alapértelmezése $n-1$ osztóval számol, a NumPy `np.cov` alapértelmezése n osztóval; mindig ellenőrizni kell a `ddof` paramétert, mert a két érték nem ugyanaz.

A skalárszorzatos alak mutatja, miért beszélünk lineáris együttmozgásról: ha az x és y centrált vektorok „ugyanabba az irányba mutatnak”, a kovariancia pozitív nagy; ha merőlegesek, nulla. Ez a legkisebb négyzetek fejezetéből ismert merőlegességfogalom közvetlen felhasználása.

Kis kézi kontroll: ha $\mathbf{x} = (-1, 0, 1)$ és $\mathbf{y} = \mathbf{x}^2 = (1, 0, 1)$ a három lehetséges érték, akkor $\bar{x} = 0$ és $\bar{y} = \frac{2}{3}$. A centrált szorzatok:

$$(-1) \cdot (1 - \tfrac{2}{3}) + 0 \cdot (0 - \tfrac{2}{3}) + 1 \cdot (1 - \tfrac{2}{3}) = -\tfrac{1}{3} + 0 + \tfrac{1}{3} = 0.$$

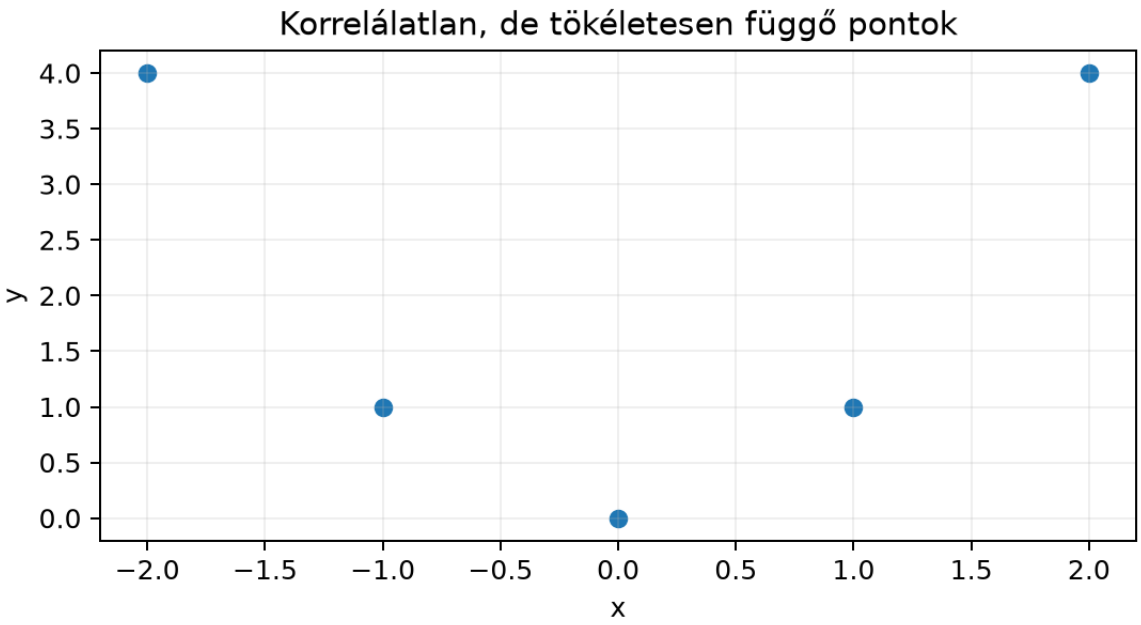
Tehát a kovariancia nulla, pedig y teljesen determinisztikusan függ x-től.

14.3.2.2 Korreláció

A kovariancia skálafüggő, ezért önmagában nem alkalmas kapcsolatok összehasonlítására. A **Pearson-féle korreláció** (correlation) a centrált vektorok koszinusz hasonlósága:

$$r_{xy} = \frac{s_{xy}}{s_x s_y}, \qquad s_x = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2}$$

hasonlóan s_y-nal. A mértékegység dimenziómentes, mert a számláló és a nevező azonos mértékegységű. A Cauchy–Schwarz-egyenlőtlenség miatt $r_{xy} \in [-1, 1]$. A $+1$ tökéletes pozitív lineáris kapcsolatot, a -1 tökéletes negatív lineáris kapcsolatot jelent, a 0 pedig azt, hogy a legjobb lineáris irányú együttmozgás sincs jelen.



14.2. ábra: Korrelálatlan, de tökéletesen függő pontok - A pontok az y=x² parabolát rajzolják ki; a lineáris korreláció mégis 0 a szimmetria miatt.

14.3.2.3 Függetlenség és korrelálatlanság

Két véletlen változó **független** (independent), ha az egyik értékének ismerete nem ad információt a másik eloszlásáról. Diszkrét esetben és közelítő értelemben: tetszőleges A és B értéktartományra

$$P(X \in A \text{ és } Y \in B) = P(X \in A)\, P(Y \in B).$$

A függetlenségből következik $E[XY] = E[X]E[Y]$, ezért $\mathrm{Cov}(X, Y) = 0$ és $r = 0$. A megfordítás **nem** igaz: az $\mathbf{x} = (-1, 0, 1)$, $\mathbf{y} = \mathbf{x}^2$ példa korrelálatlan, de függő. Más szóval a korrelálatlanság csak a lineáris kapcsolat hiányát jelenti, nem a kapcsolat hiányát.

A gyakorlatban ezért egy kis r mellé nem írhatjuk azt, hogy „a változók függetlenek”. A kódban a `scipy.stats.pearsonr` által adott p-érték is csak a lineáris korreláció nullmodelljére vonatkozik, nem az általános függetlenségre.

14.3.2.4 Feltételes függés

A kapcsolat mindig egy adott csoportra vagy feltételre vonatkozik. Két változó lehet korrelálatlan a teljes táblán, de régióra bontva erősen korrelált, és fordítva: lehet régiókon belül korrelálatlan, de a teljes táblán erős korrelációt mutat, ha a régió mindkét változó átlagát eltolja. Ez utóbbi a Simpson-jelenség közeli rokona.

Példaként tételezzük fel, hogy az `A` régióban X és Y egymástól független standard normális, a `B` régióban szintén függetlenek, de mindkettő átlaga 10-zel nagyobb. A teljes táblán a régiók közötti átlagkülönbség erős pozitív kovarianciát hoz létre, miközben régióra bontva a korreláció nulla körül marad. Ezt **feltételes függés** (conditional dependence) akkor mondjuk, ha a csoportváltozó rögzítése után is megmarad a kapcsolat; a fenti példában éppen a feltételes függetlenség és a marginális függés esete áll fenn.

14.3.2.5 Ismételt megfigyelések és csoportok

Ha ugyanazt az ügyfelet többször mérjük, a hozzá tartozó sorok egy **csoportot** (group/cluster) alkotnak. A csoporton belüli sorok nem cserélhetők fel más ügyfelek soraival, mert egy közös, ügyfélszintű hatást hordozhatnak. Egyszerű modellben:

$$y_{ik} = z_i + \varepsilon_{ik},$$

ahol z_i az i-edik ügyfél közös hatása, ε_{ik} a rendelés szintű zaj. Az azonos ügyfélhez tartozó két sor kovarianciája $\mathrm{Cov}(y_{ik}, y_{il}) = \mathrm{Var}(z_i)$, ha a zajok függetlenek. Az **intra-class correlation** (ICC) ezt a hasonlóságot méri:

$$\rho_{\mathrm{ICC}} = \frac{\sigma_z^2}{\sigma_z^2 + \sigma_\varepsilon^2},$$

ahol σ_z^2 az ügyfélszintű variancia, σ_ε^2 a rendelésszintű zaj varianciája, mindkettő y^2 mértékegységben. Ha $\rho_{\mathrm{ICC}} = 0$, a csoportok nem adnak redundáns információt; ha $\rho_{\mathrm{ICC}} = 0{,}5$, az ismételt sorok fele-fele arányban tudhatók be közös ügyfélhatásnak és egyedi zajnak.

A **független mintavételi egység** ezért ilyenkor a `customer_id`, nem az order-sor. Ha minden sort függetlennek tekintünk, az effektív mintanagyságot felülbecsüljük, és a standard hibát alulbecsüljük. Egyenlő csoportméret esetén a tervezési effektus:

$$\mathrm{DEFF} = 1 + (m - 1)\rho_{\mathrm{ICC}},$$

ahol m az ügyfelenkénti rendelések száma, ρ_{ICC} pedig az intra-class correlation. Ha például $m = 5$ és $\rho_{\mathrm{ICC}} = 0{,}5$, akkor $\mathrm{DEFF} = 3$, vagyis a független soros standard hibát $\sqrt{3} \approx 1{,}73$-mal kell megszorozni ahhoz, hogy a csoportszerkezetet figyelembe vegyük.

14.3.2.6 Sor- vs klaszter-bootstrap: az effektív elemszám

Ismételt rendeléseknél a sorok nem függetlenek: ugyanazon ügyfél rendelései hasonlóbbak lehetnek egymáshoz, mint más ügyfelekhez. A **sor-bootstrap** minden sort külön húz, ezért alábecsüli a bizonytalanságot; a **klaszter-bootstrap** ügyfelet húz, és az ügyfél összes sorát együtt tartja. A 30 ügyfél, 142 sor szimulált adatán:

Egység	Elemszám	Bootstrap SE	Megjegyzés
sor	142	0.0873	függetlennek tekinti a sorokat
ügyfél	30	0.2092	a klaszterhatást megtartja
arány	—	2.395	a klaszter-SE közel 2.4× nagyobb

Az ICC (intraclass correlation) 0.984: a teljes szórás 98.4%-a az ügyfelek közötti különbségből jön. Az effektív elemszám durva becslése $n_{\text{eff}} \approx n/[1+(m-1)\cdot \text{ICC}]$, ahol m az átlagos klaszterméret; a 142 sor így nem 142 független megfigyelés. A szignifikanciatesztet ezért a klaszter szintjén vagy klaszter-robusztus SE-vel kell számolni; a sor-szintű teszt „szignifikáns” eredménye illúzió lehet.

```
import numpy as np

def icc(groups):
    all_values = np.concatenate(groups)
    grand = all_values.mean()
    between = sum(len(g) * (g.mean() - grand)**2 for g in groups)
    within = sum(((g - g.mean())**2).sum() for g in groups)
    return between / (between + within)

def row_bootstrap(groups, repeats=2000, seed=20261029):
    rng = np.random.default_rng(seed)
    rows = np.concatenate(groups)
    return np.std([rng.choice(rows, size=len(rows), replace=True).mean()
                   for _ in range(repeats)], ddof=1)

def cluster_bootstrap(groups, repeats=2000, seed=20261029):
    rng = np.random.default_rng(seed)
    means = []
    for _ in range(repeats):
        chosen = rng.integers(0, len(groups), size=len(groups))
        means.append(np.concatenate([groups[i] for i in chosen]).mean())
    return np.std(means, ddof=1)
```

A `lab/clustering.py` `simulate()`, `icc()`, `row_bootstrap()`, `cluster_bootstrap()` és `report()` függvényei ezt a táblát adják; a `tests/test_clustering.py` az ICC >0.9, a klaszter-SE/sor-SE >2 és a különböző klaszterméretek mellett a bootstrap működését ellenőrzi.

14.3.3 Teljes minimális munkapélda

Az alábbi kód a `NumPy Generator` segítségével állítja elő a két tanulságos adatsort. A kimenet szemléltető jellegű: a szövegben levezetett várható értékek a megadott generáló modellből következnek, a tényleges futás seedtől függően kissé eltérhet.

```
import numpy as np
import pandas as pd
from scipy.stats import pearsonr
```

```
rng = np.random.default_rng(20261020)

# 1) Korrelálatlan, de függő pár: y determinisztikus függvénye x-nek
n = 20000
x = rng.normal(size=n)      # alak: (n,)
y = x**2 - 1
r_nonlinear, _ = pearsonr(x, y)

# Pontos kis kézi kontroll, ahol a kovariancia elméletileg nulla
x_small = np.array([-1.0, 0.0, 1.0])
y_small = x_small**2        # [1, 0, 1]
cov_pop = np.mean((x_small - x_small.mean()) * (y_small - y_small.mean()))

# 2) Ügyfélenként ismételt rendelések szimulációja
n_customers = 100
orders_per = 5
z = rng.normal(0, 1, size=n_customers)              # ügyfeleffektus, (100,)
eps = rng.normal(0, 1, size=(n_customers, orders_per))
y_group = z[:, None] + eps                          # broadcasting: (100,1)+(100,5)
df = pd.DataFrame({
    "customer_id": np.repeat(np.arange(n_customers), orders_per),
    "y": y_group.ravel()
})

# Naiv soronkénti standard error és ügyfélszintű standard error
naive_se = df["y"].std(ddof=1) / np.sqrt(len(df))
customer_means = df.groupby("customer_id")["y"].mean()
cluster_se = customer_means.std(ddof=1) / np.sqrt(n_customers)

# A resampling egység: customer_id, nem a rendeléssor
groups = [g["y"].to_numpy() for _, g in df.groupby("customer_id", sort=True)]
n_groups = len(groups)

def cluster_bootstrap_mean(n_boot=1000, seed=42):
    rng_boot = np.random.default_rng(seed)
    estimates = np.empty(n_boot)
    for b in range(n_boot):
        chosen = rng_boot.integers(0, n_groups, size=n_groups)
        sample = np.concatenate([groups[i] for i in chosen])
        estimates[b] = sample.mean()
    return estimates

boot = cluster_bootstrap_mean()
boot_cluster_se = boot.std(ddof=1)

print("r(x, x**2-1):", r_nonlinear)
print("pontos kis kovariancia:", cov_pop)
print("naive SE:", naive_se)
print("ügyfélszintű SE:", cluster_se)
print("ügyfélszintű bootstrap SE:", boot_cluster_se)
```

14.3.4 Eredmény értelmezése

Az első részben az $r(x, x^2 - 1)$ várhatóan nulla közelében van, mert a standard normális eloszlás harmadik momentuma $E[X^3] = 0$, és a centrált szorzat átlaga éppen ezt becsli. A pontfelhő eközben tökéletes parabolát rajzol ki, tehát a kapcsolat determinisztikus. A korreláció csak a lineáris kapcsolat hiányát mutatja, a függetlenség hiányát nem.

A második részben a generáló paraméterek $\sigma_z = 1$, $\sigma_\varepsilon = 1$, ezért $\rho_{\text{ICC}} = \frac{1}{2}$. A naiv standard error várhatóan

$$\frac{\sqrt{2}}{\sqrt{500}} \approx 0{,}063,$$

míg az ügyfélenkénti átlagok standard hibája

$$\frac{\sqrt{1+\frac{1}{5}}}{\sqrt{100}} \approx 0{,}110.$$

A két érték hányadosa $\sqrt{1+4\cdot 0{,}5}=\sqrt{3}$. Az ügyfélszintű bootstrap ugyanezt a csoportszerkezetet veszi figyelembe, ezért annak standard hibája is a nagyobb érték körül ingadozik. Ha valaki a rendeléssorokat tekintené mintavételi egységnek, a `naive_se` a helyes ügyfélszintű bizonytalanság mintegy $1/\sqrt{3}$-szerese lenne.

A korrelálatlanság nem általában jelent függetlenséget. Az intraklaszter-korrelációra írt $1+(m-1)\rho$ tervezési hatás az egyforma klasszterméretet és az egyszerű cserélhető korrelációs modellt feltételezi.

14.3.5 Feladatok

14.3.5.1 1. feladat — Értelmezés: mi a mintavételi egység a Northstar adatban?

A `orders.csv` 2400 rendeléssort és 300 egyedi ügyfelet tartalmaz. Magyarázd el, miért nem érvényes az az állítás, hogy „2400 független megfigyelésünk van”, ha egy ügyfél több rendelést is leadott. Mi a resampling egység, és miért?

14.3.5.2 2. feladat — Implementálás: kézi kovariancia és korreláció

Számítsd ki kézzel a minta kovarianciáját $n-1$ osztóval és a Pearson-korrelációt az alábbi párokra:

$$\mathbf{x}=(-2,-1,0,1,2), \qquad \mathbf{y}=\mathbf{x}^2-2=(2,-1,-2,-1,2).$$

Ellenőrizd NumPy-val is (`np.cov` `ddof=1` választással és `np.corrcoef`). Mit mond az eredmény a függetlenségről?

14.3.5.3 3. feladat — Hibakeresés: soronkénti bootstrap ismételt mérésnél

Egy elemző a fenti `df` adaton a `y` oszlop átlagának standard hibáját az alábbi módon becsli:

```
rng_boot = np.random.default_rng(42)
row_boot = rng_boot.choice(df["y"].to_numpy(), size=(1000, len(df)), replace=True)
se_wrong = row_boot.mean(axis=1).std(ddof=1)
```

Az eredmény körülbelül `0.063`, míg az ügyfélenkénti bootstrap körülbelül `0.110`. Melyik becslés hitelesebb ebben a szimulációban, és milyen feltevést sért a soronkénti bootstrap? Fogalmazd meg a javítást a mintavételi egység szintjén.

14.3.5.4 4. feladat — Megváltozott feltétel: feltételes függés régiókkal

Generálj adatot az alábbi módon:

```
rng = np.random.default_rng(8)
n = 500
x = np.concatenate([rng.normal(0, 1, n), rng.normal(10, 1, n)])
y = np.concatenate([rng.normal(0, 1, n), rng.normal(10, 1, n)])
region = np.array(["A"] * n + ["B"] * n)
```

Számítsd ki a teljes Pearson-korrelációt, valamint külön az `A` és `B` régióra. Miért tér el a teljes korreláció a régiókon belülitől? Nevezd meg, milyen kapcsolat van a csoportosító változó és a két elemzett változó között.

14.3.6 Megoldások és indoklás

14.3.6.1 1. feladat megoldása

Az `orders.csv` sorai rendelések, de ugyanazon ügyfélhez tartozó rendelések nem függetlenek: közös ügyféléletciklus, szokás vagy viselkedés hat rájuk. A 2400 sor legfeljebb 300, de a csoporton belüli korreláció miatt jellemzően ennél is kevesebb független információs egységet jelent. A resampling egység a `customer_id`: ha új mintát húzunk, az egész ügyfelet húzzuk a hozzá tartozó összes rendeléssel együtt.

14.3.6.2 2. feladat megoldása

$\bar{x} = 0$, $\bar{y} = 0$; a centrált vektor $\mathbf{x}_c = \mathbf{x}$, $\mathbf{y}_c = \mathbf{y}$. A szorzatösszeg:

$$(-2)(2) + (-1)(-1) + 0 + 1(-1) + 2(2) = -4 + 1 + 0 - 1 + 4 = 0.$$

Ezért $s_{xy} = 0/(5-1) = 0$ és $r = 0$. Ugyanakkor $y = x^2 - 2$ determinisztikus, tehát a két változó erősen függő lehet, csak nem lineárisan. A NumPy `np.cov(x,y,ddof=1)[0,1]` és `np.corrcoef(x,y)[0,1]` ugyanezt adja nullának; tolerancia itt csak numerikus kerekítés, a közelítő érték `1e-16` nagyságrendben nulla.

14.3.6.3 3. feladat megoldása

A soronkénti bootstrap azt feltételezi, hogy az 500 rendeléssor egymástól függetlenül cserélhető. Valójában egy ügyfélen belül a sorok közös z_i hatást hordoznak, ezért a soronkénti újrahúzás mesterségesen sok független egységet imitál, és alábecsüli az átlag bizonytalanságát. A helyes eljárás az ügyfélazonosítók újrahúzása, és a kiválasztott ügyfelek összes sorának beemelése, ahogy a `cluster_bootstrap_mean` teszi. Az ügyfélenkénti eljárás `0.110` körüli standard hibája figyelembe veszi a csoporthatást.

14.3.6.4 4. feladat megoldása

A teljes adaton a korreláció elméletileg $\frac{25}{26} \approx 0{,}96$ körüli lesz, mert a régiók közötti 10 egységnyi átlageltolódás mindkét változót egyszerre növeli. A régiókon belül X és Y független generálású, ezért a két régióra számított korrelációk 0 körüli, seedtől függően véletlenszerűen ingadozó értékek. A `region` csoportosító változó mindkét változó közös oka: régióra bontva eltűnik a kapcsolat, a teljes táblán viszont megjelenik. Ez feltételes függetlenség, nem okozati kapcsolat.

14.3.7 Gyakori tévedés ellenpéldával

Sok rendelés egy ügyféltől nem ugyanannyi független ügyfélmegfigyelés. Tegyük fel, hogy három ügyfelünk van, és az egyik, mondjuk egy nagykereskedő, 200 szinte azonos rendelést adott le, míg a másik kettő 5-5 rendelést. A táblán 210 sor lenne, de az ügyfélszintű bizonyíték továbbra is 3 egység. Ha valaki a 210 sorra számol standard hibát, az ügyfél szintű bizonytalanságot drámaian alábecsülheti, noha a sorok redundáns ismétlések.

Hasonlóan, ha egy régió mind a kosárértéket, mind a szállítási távolságot eltolja, a teljes táblán látszó korreláció nem jelenti azt, hogy az egyik rendelési jellemző közvetlenül növeli a másikat. Ellenőrző kérdések: használtam-e `ddof`-ban következetes osztót? Régióra vagy ügyfélre bontva is hasonló a kapcsolat? A mintavételi egység a sor vagy a csoport? Ha a rétegzett és a teljes korreláció eltér, nem szabad a teljes értéket automatikusan általános érvényű állításként közölni.

14.3.8 Rövid összefoglaló

A kovariancia a centrált vektorok skalárszorzatának átskálázott alakja, a korreláció pedig a koszinusz hasonlóság, ezért csak lineáris kapcsolatot mér. A függetlenségből következik a korrelálatlanság, de a korrelálatlanságból nem következik a függetlenség; az X és $X^2 - 1$ pár erre a legvilágosabb ellenpélda. Ismételt méréseknél a sorok nem függetlenek, mert az ügyfél közös hatást visz be. A helyes mintavételi egység ilyenkor a `customer_id`, nem az order-sor; az ICC és a tervezési effektus mutatja, hányszorosára nőhet a standard hiba. A következő bootstrap alfejezetben ugyanezt a csoportszerkezetet kell majd tiszteletben tartani.

14.4 Nagy számok és centrális határeloszlás

Belépő · szint: közép · idő: 60 perc · eszköz: NumPy, SciPy

Előfeltétel: 14.3. alfejezet · Célkészség: Nagy számok és centrális határeloszlás · Kimenet: működő kód és a labor ellenőrző tesztje.

14.4.1 Cél és konkrét helyzet

A Northstar kezdőlabor `orders.csv` fájljában a `delivery_days` pozitív egész, jobbra ferde eloszlású: sok rendelés 4-7 nap alatt megérkezik, de van egy hosszú farkú kisebbség, amelyik 10-20 napot is késik. Tegyük fel, hogy a termékmenedzser azt kérdezi: „Ha holnap 100 új rendelés érkezik, mennyire lesz pontos a 7 napos kézbesítési átlagunk becslése?” A kérdés megválaszolásához nem elég az adatok hisztogramját nézni: külön kell választanunk **az adatok eloszlását** és **az átlag mintavételi eloszlását**. Ez az alfejezet ezt a két fogalmat építi fel szimulációval, a nagy számok törvénye (law of large numbers, LLN) és a centrális határeloszlás-tétel (central limit theorem, CLT) segítségével.

Kiinduló becslés: ha a `delivery_days` várható értéke $\mu \approx 6$ nap, szórása $\sigma \approx 3$ nap, akkor 100 rendelés átlagának bizonytalansága nagyjából $\sigma/\sqrt{100} = 3/10 = 0{,}3$ nap lehet. Ezt a számot ebben az alfejezetben pontosan értelmezzük, és ellenőrizzük, milyen feltételek mellett igaz.

14.4.2 Fogalmi magyarázat: adateloszlás, mintavételi eloszlás, LLN, CLT és standard error

Legyen $X_1, X_2, \ldots, X_n$ egy n elemű minta. Minden X_i ugyanabból az eloszlásból származik, várható értéke $E[X_i] = \mu$, varianciája $\mathrm{Var}(X_i) = \sigma^2$, és a megfigyelések függetlenek (iid, a 14.3. alfejezetben bevezetett értelemben). A mintaátlag:

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Az átlag maga is véletlen mennyiség, hiszen másik mintát véve más értéket kapunk. Ha $n = 2$ és $X_i \sim \mathrm{Bernoulli}(0{,}5)$, azaz minden elem 0 vagy 1, fejenként $1/2$ valószínűséggel, akkor a lehetséges minták és átlagaik:

Minta	$\bar{X}_2$	Valószínűség
(0, 0)	0,0	0,25
(0, 1) vagy (1, 0)	0,5	0,50
(1, 1)	1,0	0,25

Itt $\mu = 0{,}5$ és $\sigma^2 = 0{,}25$. Az átlag várható értéke $0{,}25\cdot 0 + 0{,}5\cdot 0{,}5 + 0{,}25\cdot 1 = 0{,}5 = \mu$, varianciája pedig $0{,}25\cdot(0-0{,}5)^2 + 0{,}5\cdot(0{,}5-0{,}5)^2 + 0{,}25\cdot(1-0{,}5)^2 = 0{,}125 = \sigma^2/2$. Ez a kézi ellenőrzés mutatja a két alapazonosságot:

$$E[\bar{X}_n] = \mu, \qquad \mathrm{Var}(\bar{X}_n) = \frac{\sigma^2}{n}.$$

A második azért igaz, mert a variancia skálázási szabálya szerint $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$, és független tagok varianciája összeadódik. Az átlag tehát ugyanazon μ körül ingadozik, de a szórása $1/\sqrt{n}$-szeresére csökken. Ezt a mennyiséget nevezzük **standard hibának** (standard error, SE):

$$SE(\bar{X}_n) = \sqrt{\mathrm{Var}(\bar{X}_n)} = \frac{\sigma}{\sqrt{n}}.$$

A standard hiba mértékegysége megegyezik X mértékegységével: a fenti szállítási példában nap. Fontos megkülönböztetni a **minta szórásától**, s-től, amely az egyes adatpontok szóródását méri; a standard hiba az átlag becslésének bizonytalansága.

A **mintavételi eloszlás** (sampling distribution) egy statisztika — például az átlag — eloszlása az összes lehetséges azonos méretű minta felett. A gyakorlatban ezt úgy közelítjük, hogy sok független mintát veszünk ugyanabból a populációból, és mindegyikből kiszámoljuk a statisztikát. Amikor a fenti Bernoulli-példában $n = 2$-vel a három lehetséges átlagérték valószínűségeit soroltuk fel, valójában a mintavételi eloszlást írtuk le.

A **nagy számok törvénye** (LLN) azt mondja ki, hogy n növekedtével az átlag egyre közelebb kerül μ-höz: nagy mintán a véletlen átlagolás kiátlagolja az egyedi eltéréseket. A standard hiba

$1/\sqrt{n}$-es csökkenése ennek a kvantitatív formája: $n = 100$-nál tízszer kisebb a szóródás, mint $n = 1$-nél.

A **centrális határeloszlás-tétel** (CLT) ennél többet állít: nemcsak azt, hogy az átlag koncentrálódik, hanem azt is, hogy az átlag *eloszlásának alakja* haranggörbéhez tart. Pontosabban, ha X_i iid, μ és σ^2 véges, akkor a standardizált átlag

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} = \frac{\bar{X}_n - \mu}{SE(\bar{X}_n)}$$

eloszlása $n \to \infty$ esetén a standard normális eloszláshoz, $\mathcal{N}(0,1)$-hez konvergál. Ez akkor is igaz, ha maga X erősen ferde vagy akár diszkrét. A konvergencia eloszlásbeli, nem pontonkénti: a Z_n hisztogramja egyre jobban illeszkedik a normális sűrűségre.

A tétel feltételei:

1. **iid minta**: a megfigyelések függetlenek és azonos eloszlásúak. Ha például ugyanazon ügyfél több rendelését is belerakjuk, a sorok nem függetlenek, és a $\sigma/\sqrt{n}$ képlet általában nem megfelelő; pozitív korreláció esetén gyakran alulbecsli, negatív korreláció esetén akár túlbecsülheti a tényleges standard hibát — ezt a 14.3. alfejezetben a tervezési effektussal láttuk.
2. **Véges variancia**: $\sigma^2 < \infty$. Ez a valós adatoknál általában teljesül, de egyes nehéz farkú eloszlásoknál (például Cauchy) nem, és ott a CLT nem működik.
3. **Elég nagy** n: a gyakorlati közelítés annál jobb, minél nagyobb a minta, és minél kevésbé ferde az alapeloszlás. Erősen ferde eloszlásnál $n = 30$ lehet kevés, $n = 100$ vagy $n = 300$ lehet a biztonságos.

A CLT **nem** azt állítja, hogy az adatok maguk normálissá válnak nagy mintán. Egy exponenciális eloszlású adathalmaz $n = 1000$ elemnél is jobbra ferde marad; csak az átlag eloszlása lesz közel normális. Ez a leggyakoribb félreértés, a 13.4. alfejezetben tárgyalt ferde eloszlásokra ugyanígy vonatkozik.

14.4.2.1 A CLT feltételei: nehéz farok, függés és ferdeség

A centrális határeloszlás-tétel nem feltétel nélküli: független (vagy gyengén függő), azonos eloszlású, **véges várható értékű és varianciájú** változók kellenek. Három ellenpélda:

Adat	n	Diagnosztika	Eredmény
iid egyenletes $U(-1,1)$	30	$\lvert\bar{X}\rvert > 1$ aránya	0.000
Cauchy (végtelen variancia)	30	$\lvert\bar{X}\rvert > 1$ aránya	0.478
AR(1), $\rho = 0.8$	100	SE / iid SE	2.999

Az iid egyenletes átlag jól normalizálódik: a $|\bar{X}| > 1$ eset gyakorlatilag nem fordul elő. A Cauchy-eloszlásnak nincs véges varianciája, ezért a mintaátlag nem tömörül a várható érték körül, és a standardizált átlag továbbra is széles: a 2000 szimulációból 47.8% esik 1-en túl. Az AR(1) függő sorozatnál a mintaátlag szórása 2.999-szerese az iid esetnek, ami közelítőleg $\sqrt{(1+\rho)/(1-\rho)} = \sqrt{9} = 3$; a függés tehát az effektív elemszámot csökkenti, nem tünteti el a normalitást, de a naiv standard error hibás.

A ferdeség külön eset: véges variancia mellett a CLT érvényes, de a konvergencia lassú; erősen ferde adatnál nagyobb n kell a normálközelítéshez. A tétel tehát nem azt mondja, hogy „minden átlag normális”, hanem azt, hogy megfelelő feltételek mellett a standardizált átlag eloszlása a normálishoz tart.

```
import numpy as np

rng = np.random.default_rng(20261029)

def means(distribution, n, repeats=2000):
    if distribution == "uniform":
        draws = rng.uniform(-1, 1, size=(repeats, n))
    else:
        draws = rng.standard_cauchy(size=(repeats, n))
    return draws.mean(axis=1)

uniform = means("uniform", 30)
cauchy = means("cauchy", 30)
print("uniform extreme:", np.mean(np.abs(uniform) > 1))
print("cauchy extreme:", np.mean(np.abs(cauchy) > 1))

def ar1_means(n, rho=0.8, repeats=2000):
    values = np.zeros((repeats, n))
    values[:, 0] = rng.normal(size=repeats)
    for t in range(1, n):
        values[:, t] = rho * values[:, t-1] + rng.normal(scale=(1-rho**2)**0.5, size=repeats)
    return values.mean(axis=1)

ar = ar1_means(100)
print("AR SE ratio:", ar.std(ddof=1) / (1 / np.sqrt(100)))
```

A `lab/clt_conditions.py` `sample_means()`, `ar1_means()`, `extreme_share()` és `report()` függvényei ezt a három diagnosztikát adják; a `tests/test_clt_conditions.py` a Cauchy szélső arányát, az iid egyenletes nullához közeli arányát és az AR(1) SE-arányt ellenőrzi.

14.4.3 Minimális munkapélda: exponenciális adat és az átlag eloszlása

A szállítási idő utánzataúl válasszunk $\mathrm{Exp}(\lambda = 1/6)$ eloszlást, amelynek várható értéke $\mu = 6$ nap, szórása $\sigma = 6$ nap. Ez jobbra ferde: sok érték 0 és 5 nap között, de ritkán 20-30 napos késések is előfordulnak. Négy mintanagysággal dolgozunk: $n = 1, 5, 30, 100$. Minden n-re:

- veszünk **egyetlen** n elemű mintát, ez az „adat eloszlása” hisztogramjának alapja;
- veszünk $B = 10\,000$ darab n elemű mintát, mindegyikből kiszámoljuk az átlagot, ez közelíti az átlag mintavételi eloszlását.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(20261020)  # szimulációs seed, nem a Northstar-generátoré
n_sim = 10_000                         # ismételt minták száma B
mu = 6.0                               # az exponenciális várható értéke (nap)
sigma = 6.0                            # az exponenciális szórása (nap)
ns = [1, 5, 30, 100]                   # mintanagyságok

fig, axes = plt.subplots(2, 4, figsize=(14, 6), sharex="col")

for j, n in enumerate(ns):
    # Egyetlen adatminta: az adatok eloszlását mutatja
    one_sample = rng.exponential(scale=mu, size=n)  # shape: (n,)
```

```
    axes[0, j].hist(one_sample, bins=25, density=True,
                    color="steelblue", alpha=0.7, range=(0, 30))
    axes[0, j].set_title(f"Adat, n={n}")
    if j == 0:
        axes[0, 0].set_ylabel("Sűrűség")

    # B darab minta átlaga: a mintavételi eloszlás közelítése
    samples = rng.exponential(scale=mu, size=(n_sim, n))  # shape: (B, n)
    means = samples.mean(axis=1)                          # shape: (B,)
    axes[1, j].hist(means, bins=30, density=True,
                    color="darkorange", alpha=0.7)
    # A CLT szerinti normális sűrűség: N(mu, sigma/sqrt(n))
    xs = np.linspace(mu - 4 * sigma / np.sqrt(n), mu + 4 * sigma / np.sqrt(n), 300)
    axes[1, j].plot(xs, stats.norm.pdf(xs, loc=mu, scale=sigma / np.sqrt(n)),
                    "r-", linewidth=2)
    axes[1, j].set_title(f"Átlag, n={n}")
    axes[1, j].set_xlabel("nap")
    if j == 0:
        axes[1, 0].set_ylabel("Sűrűség")

fig.suptitle("Az adat eloszlása (fent) és az átlag mintavételi eloszlása (lent)")
plt.tight_layout()
plt.show()
```

A fenti kódot itt nem futtattuk; az alábbi leírás a megadott seeddel **elméletileg várt, szemléltető** kimenet, nem mért eredmény. A felső sor hisztogramjai minden n-re jobbra ferde exponenciális alakot mutatnak: $n = 100$-nál is a 0-5 napos sáv a leggyakoribb, és a farok 20 nap fölé nyúlik. Az alsó sorban $n = 1$-nél az átlag eloszlása azonos az adat eloszlásával. $n = 5$-nél a hisztogram még enyhén jobbra ferde, de $n = 30$-nál és $n = 100$-nál a piros normális görbe már szemmel láthatóan együtt fut a hisztogrammal. A harang alak középpontja mindvégig $\mu = 6$ nap körül van, a szélessége viszont n növekedtével láthatóan csökken: $n = 100$-nál a vízszintes tengelyen már csak a 4-8 napos sáv a releváns.

14.4.4 Eredmény értelmezése és ellenőrző számítás

A szimuláció numerikus ellenőrzéséhez az átlagok tömbjére vonatkozó két invariant használható. Mivel a $B = 10\,000$ átlag maga is egy minta a mintavételi eloszlásból, az ő átlaguk közelíti $E[\bar{X}_n] = \mu$-t, a szórásuk pedig közelíti $SE(\bar{X}_n) = \sigma/\sqrt{n}$-t. A Monte Carlo közelítés hibája nagyjából $1/\sqrt{B} \approx 0{,}01$ szintű a standardizált mennyiségekre, ezért a következő szemléltető tűréshatárokat fogadhatjuk el: az átlagok átlaga $6 \pm 0{,}05$ nap, az átlagok szórása pedig $\sigma/\sqrt{n}$ értékétől legfeljebb néhány százalékkal térhet el.

Például $n = 100$-ra az elméleti standard hiba $6/\sqrt{100} = 0{,}6$ nap. Ha a szimulált átlagok szórása 0,58 és 0,62 nap közé esik, az összhangban van a képlettel. Az $n = 5$ esetben a standard hiba $6/\sqrt{5} \approx 2{,}68$ nap, tehát öt rendelés átlaga még nagyon zajos becslés — ezért nem érdemes kis mintából üzleti döntést hozni.

A standardizált értékekre is ellenőrizhetünk. Ha a `means` tömb átlaga $\bar{m}$ és szórása s_m, akkor a `z = (means - mu) / (sigma / np.sqrt(n))` tömb hisztogramjának közelítenie kell a $\mathcal{N}(0, 1)$ sűrűséget. Ennek ferdesége 0-hoz, lapultsága 3-hoz tart n növekedtével; kis n-nél az exponenciális alapeloszlás ferdesége miatt még látható eltérés maradhat.

A Northstar `delivery_days` oszlopára ugyanez a logika érvényes, de ott μ-t és σ-t nem ismerjük, csak a mintából becsülhetjük. A becslés bizonytalansága a 15.2. alfejezetben tárgyalt konfidenciaintervallum témája lesz; itt a generáló paramétereket pontosan ismerő szimuláció volt a cél.

A centrális határeloszlás-tétel nem azt mondja ki, hogy a nyers adatok eloszlása normálissá válik, hanem a megfelelően standardizált mintaátlag közelítő eloszlásáról szól. Függő adatok vagy végtelen variancia esetén külön feltételek szükségesek.

14.4.5 Feladatok

1. Értelmezés. Egy elemző azt mondja: „Megnéztem 200 szállítási időt, és a hisztogram még mindig erősen jobbra ferde. Ez azt bizonyítja, hogy a centrális határeloszlás-tétel nem működik a mi adatunkon.” Magyarázd el, miért téves ez az érvelés. Melyik eloszlásra vonatkozik a CLT, és mit kellene inkább ábrázolnia?

2. Implementálás. A Bernoulli-példa szimulációja. Generálj `rng = np.random.default_rng(42)` seeddel $B = 20\,000$ ismételt mintát $p = 0{,}2$ paraméterű Bernoulli-eloszlásból, $n = 10$, 50 és 200 mintanagysággal. Mindegyik n-re számold ki az átlagok átlagát és szórását, és hasonlítsd össze a $\mu = p$ és $\sigma/\sqrt{n} = \sqrt{p(1-p)/n}$ elméleti értékekkel. Add meg, mekkora eltérést fogadsz el a Monte Carlo hiba miatt.

3. Hibadiagnózis. Egy kolléga ezt a kódot írta a standard hiba ellenőrzésére:

```
rng = np.random.default_rng(7)
samples = rng.exponential(scale=6.0, size=(5000, 30))
means = samples.mean(axis=1)
print(np.std(samples))      # 6.01 körüli értéket vár
print(np.std(means))        # 1.10 körüli értéket vár
```

Azt állítja, hogy a második `print` a standard hibát adja, ezért `1.10`-et kellene kapnia, de ő `6.01`-hez közeli értéket lát. Hol a hiba a gondolatmenetében? Javítsd ki a kódot, és add meg a helyes elméleti standard hibát $n = 30$-ra.

4. Megváltozott feltétel: Cauchy-eloszlás. A CLT feltételezi a véges varianciát. Vizsgáld meg, mi történik, ha X_i standard Cauchy-eloszlású, amelynek nincs várható értéke és varianciája. Generálj `rng = np.random.default_rng(99)` seeddel $B = 10\,000$ mintát $n = 5$, 50 és 500 mintanagysággal `rng.standard_cauchy(size=(B, n))` használatával, számold ki az átlagokat, és ábrázold a hisztogramjukat a $[-10, 10]$ intervallumra vágva. Mit tapasztalsz, és miért nem mond ellent a CLT-nek?

14.4.6 Megoldások és indoklás

1. megoldás. A CLT az **átlag mintavételi eloszlásáról** szól, nem az adatok eloszlásáról. A 200 szállítási idő hisztogramja az adateloszlást mutatja, amely a CLT szerint is ferde maradhat tetszőlegesen nagy n-nél. Az elemzőnek azt kellene ábrázolnia, hogy sok, egyenként 200 elemű minta átlaga hogyan oszlik el. Ha az alapeloszlás véges varianciájú, akkor ezek az átlagok egyre inkább harang alakú eloszlást követnek $N(\mu, \sigma/\sqrt{200})$ paraméterekkel. Az adat ferdesége tehát nem cáfolja a tételt.

2. megoldás. A következő kód ellenőrzi az invariánsokat. Az elméleti értékek: $\mu = 0{,}2$, $\sigma = \sqrt{0{,}2 \cdot 0{,}8} = 0{,}4$, így a standard hiba $n = 10$-re $0{,}4/\sqrt{10} \approx 0{,}1265$, $n = 50$-re 0,0566, $n = 200$-ra 0,0283. A Monte Carlo hiba a szórás becslésére nagyjából $\sigma_m/\sqrt{2B}$ nagyságrendű, ami $B = 20\,000$-nél elhanyagolható a három tizedesjegyhez képest, ezért $\pm 0{,}005$ abszolút tűrés észszerű.

```python
import numpy as np

rng = np.random.default_rng(42)
B = 20_000
p = 0.2
sigma = np.sqrt(p * (1 - p))  # 0.4

for n in [10, 50, 200]:
    samples = rng.binomial(n=1, p=p, size=(B, n))  # shape: (B, n), 0/1 értékek
    means = samples.mean(axis=1)                   # shape: (B,)
    se_theory = sigma / np.sqrt(n)
    print(
        f"n={n:3d} | átlagok átlaga={means.mean():.4f} (cél {p}) | "
        f"átlagok szórása={means.std(ddof=0):.4f} (SE cél {se_theory:.4f})"
    )
```

A szemléltető várt kimenet (a seeddel elméletileg, nem futtatott eredmény): minden sorban az átlagok átlaga 0,20 körül mozog $\pm$0,005-ön belül, az átlagok szórása pedig rendre kb. 0,126, 0,057 és 0,028. A `ddof=0` azért helyes, mert itt a B darab átlagot a mintavételi eloszlás teljes szimulált populációjának tekintjük, nem egy részmintának.

3. megoldás. A kolléga két különböző mennyiséget kever össze. Az `np.std(samples)` a **30 elemű adatminta** szórásának becslése a $B \times 30$-as tömbön, ami $\sigma = 6$ napot közelíti — ezért lát 6,01-et. A standard hiba viszont ennek $1/\sqrt{30}$-szorosa: $6/\sqrt{30} \approx 1{,}095$ nap. A helyes ellenőrzés az `np.std(means)` értékét hasonlítja ehhez. A kódban tehát nem a `print` sor hibás, hanem az értelmezés: a második `print` valójában a standard hibát adja, és 1,10-hez közeli értéket kellene látni. Ha a kolléga mégis 6,01-et lát a második sorban, akkor valószínűleg véletlenül mindkét sorban az `samples` tömböt írta, vagy a két nyomtatást felcserélte. Javítás: `print(np.std(means))` az egyetlen releváns sor a standard hiba ellenőrzésére.

4. megoldás. A Cauchy-eloszlásnak nincs véges várható értéke, ezért az átlag sem konvergál μ-höz, és a standardizált átlag sem tart normálishoz. A szimulációban a hisztogramok nem mutatnak n-nel csökkenő szélességet: a farokban mindig maradnak extrém kiugró átlagok, mert egyetlen nagy X_i érték dominálhatja a teljes átlagot. A $[-10, 10]$-re vágott ábrán is látszik, hogy $n = 500$-nál is széles, csúcsos eloszlást kapunk, nem haranggörbét. Ez nem mond ellent a CLT-nek, mert a tétel feltétele — a véges variancia — sérül. A Cauchy-példa azt tanítja, hogy a CLT-t nem szabad automatikusan alkalmazni, ha az adatokban extrém kiugrók dominálnak vagy a variancia nem véges; ilyenkor robusztusabb összegzés vagy más modell kell.

14.4.7 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés: **„Nagy mintán az adatok normális eloszlásúvá válnak."** Ez hamis. Az exponenciális szimuláció felső sora mutatja az ellenpéldát: $n = 100$-nál is jobbra ferde az adat, miközben az átlag eloszlása már normális. A CLT csak a mintavételi eloszlás alakjáról szól, az adateloszlásról nem.

Visszautalás: a 13.4. alfejezetben a kiugrókat a mérési folyamathoz kapcsoltuk; a CLT érvényességéhez is azt kell ellenőrizni, hogy a farok nem olyan nehéz, hogy a variancia végtelen legyen. A 14.3. alfejezetben láttuk, hogy a függőség módosítja az átlag varianciáját: pozitív klaszteren belüli korreláció gyakran csökkenti az effektív mintanagyságot és az iid képlet alulbecsülheti a standard hibát; ez nem igaz minden lehetséges függőségi szerkezetre.

Önellenőrző kérdések: (1) Mi a különbség σ, a minta szórása és a standard hiba között? (2) Miért van szükség a függetlenségre a variancia $1/n$-es skálázásához? (3) Ha egy eloszlás ferde, mekkora n-től bízhatunk a normális közelítésben — mi határozza meg ezt? (4) A Cauchy-szimulációban miért nem csökken a hisztogram szélessége n-nel?

14.4.8 Rövid összefoglaló

A nagy számok törvénye szerint a mintaátlag a várható értékhez konvergál, és ingadozásának mértéke a standard hiba, $\sigma/\sqrt{n}$. A centrális határeloszlás-tétel ennél erősebb: független, azonos eloszlású, véges varianciájú megfigyelések esetén a standardizált átlag eloszlása a standard normálishoz tart, függetlenül az adatok eredeti alakjától. A tétel az átlag mintavételi eloszlásáról szól, nem az adatokéról; ferde adat nagy mintán is ferde marad. Az iid feltétel és a véges variancia sérülése érvényteleníti a közelítést, ahogy a Cauchy-példa mutatja. A következő 15.1. alfejezetben ebből építjük fel a likelihood és a paraméterbecslés fogalmát.

További olvasmány és elsődleges dokumentáció: [SciPy] scipy.stats Reference, https://docs.scipy.org/doc/scipy/reference/stats.html (hozzáférés: 2026-10-21).

15 Becslés, likelihood és bizonytalanság

15.1 Paraméter, becslő és likelihood

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 14.4. alfejezet · Célkészség: Paraméter, becslő és likelihood · Kimenet: működő kód és a labor ellenőrző tesztje.

15.1.1 Cél és konkrét helyzet

A Northstar-kísérlet `conversion_7d` oszlopa egy ügyfélre 0 vagy 1 értékű kimenet. Ha egy jól körülhatárolt, homogén részpopulációt vizsgálunk, akkor az egyes ügyfelek kimeneteit ugyanazzal az ismeretlen p valószínűséggel generált Bernoulli-megfigyelésként modellezhetjük. A teljes `outcomes.csv`-t nem szabad egyetlen p-vel leírni, mert a kezdőlabor szerint a kezelés és a szegmens is módosítja a konverziós valószínűséget; ezt a feltételt a becslés előtt pontosan ki kell mondani.

Ebben az alfejezetben a bemenet egy rögzített, kézzel megadott 14 elemű minta:

$$x = (0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1).$$

A megfigyelési egység egy randomizált ügyfél egyetlen bináris kimenete, azaz $x_i \in \{0, 1\}$. A keresett paraméter $p = P(X_i = 1)$, amely ismeretlen, rögzített szám a $[0, 1]$ intervallumban. A cél: adjunk pontbecslést p-re rácskereséssel, numerikus optimalizálással és zárt képlettel, majd ellenőrizzük, hogy a három eredmény egyezik. A mintában $k = 7$ darab 1-es és $n = 14$ megfigyelés van, ezért az előzetes, még nem indokolt becslésünk $\hat{p} = 7/14 = 0{,}5$.

15.1.2 Paraméter, becslő és becslés

Egy statisztikai modellben a **paraméter** (parameter) a sokaság egy ismeretlen, rögzített jellemzője. Itt p a konverziós valószínűség, értékkészlete $[0, 1]$, mértékegysége nincs, mert valószínűség. A minta megfigyelése előtt az adatot véletlen változók alkotják: $X_1, \dots, X_n$ független, azonos Bernoulli-eloszlású (iid) változók. A megfigyelés után viszont $x_1, \dots, x_n$ már rögzített számok; a likelihood-szemléletben mindig ez utóbbi a kiindulópont.

A **becslő** (estimator) olyan függvény, amely a mintából számít egy értéket a paraméterre. A legtermészetesebb becslő az átlag:

$$\hat{p} = \bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

A becslő még véletlen változó, mert a nagybetűs X_i-k véletlenek. Amikor az aktuális mintát behelyettesítjük, a kapott számot **becslésnek** (estimate) nevezzük:

$$\hat{p}_{\text{obs}} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{k}{n},$$

ahol $k = \sum_i x_i$ a sikerek (1-esek) száma. Fontos megkülönböztetni a kettőt: a becslő mintavételi eloszlással és bizonytalansággal rendelkezik, a becslés egyetlen konkrét szám.

Egy gyors kézi kontroll: ha $n = 5$ és $x = (1, 0, 0, 1, 1)$, akkor $k = 3$ és $\hat{p}_{\text{obs}} = 3/5 = 0{,}6$. Ez nem azt állítja, hogy p pontosan 0,6, csak azt, hogy ezt a mintát a 0,6 paraméter magyarázd a legjobban az itt következő kritérium szerint.

15.1.3 Likelihood és log-likelihood

A **likelihood-függvény** (likelihood) azt méri, hogy egy adott p paraméterérték mellett mennyire valószínű a ténylegesen megfigyelt minta. A valószínűségszámítási feltétel itt $P(x \mid p)$: az adat feltételes valószínűsége, ha p az igazi paraméter. Iid Bernoulli-modellben:

$$L(p) = P(x_1, \ldots, x_n \mid p) = \prod_{i=1}^{n} p^{x_i}(1-p)^{1-x_i} = p^k(1-p)^{n-k}.$$

A jelölések: p a likelihood független változója, k és n pedig a rögzített mintából jövő konstansok. Az $L(p)$ értéke egy n hosszú, konkrét 0/1 sorozat előfordulási valószínűsége az adott p mellett, tehát mértékegysége szintén nincs, értéke $[0, 1]$-beli. A szokásos valószínűséggel ellentétben itt nem az adat a változó, hanem a paraméter; a likelihood tehát a paraméter függvénye.

A **log-likelihood** (log-likelihood) a likelihood természetes logaritmusa:

$$\ell(p) = \log L(p) = k \log p + (n-k)\log(1-p),$$

értelmezési tartománya a nyílt $(0, 1)$ intervallum; a széleken határértéket vizsgálunk. A logaritmus folytonos, szigorúan monoton növekvő függvény, ezért $L(p)$ és $\ell(p)$ ugyanott veszi fel a maximumát. A logaritmussal a szorzatból összeg lesz, ami kézzel és számítógéppel is stabilabb, mert sok kis valószínűség helyett mérsékelt negatív számokat adunk össze.

Kézi kontroll a fenti $n = 5$, $k = 3$ példán: $L(0{,}6) = 0{,}6^3 \cdot 0{,}4^2 = 0{,}03456$, míg $L(0{,}5) = 0{,}5^5 = 0{,}03125$. A 0,6 nagyobb likelihoodot ad, mint a 0,5, ami egybevág a becsléssel. A log-likelihood értékek: $\ell(0{,}6) = 3\log 0{,}6 + 2\log 0{,}4 \approx -3{,}365$, $\ell(0{,}5) = 5\log 0{,}5 \approx -3{,}466$; a nagyobb (kevésbé negatív) ℓ is ugyanarra a kedvezőbb pontra mutat.

15.1.4 Maximum likelihood becslés

A **maximum likelihood becslés** (maximum likelihood estimate, MLE) az a paraméterérték, amely maximalizálja a likelihood-függvényt:

$$\hat{p}_{\text{MLE}} = \arg\max_{p \in [0,1]} L(p) = \arg\max_{p \in [0,1]} \ell(p).$$

Belső maximumot keresve deriváljuk a log-likelihood-ot a p szerint. A 11.1. alfejezetben tanult egyváltozós deriválás szerint:

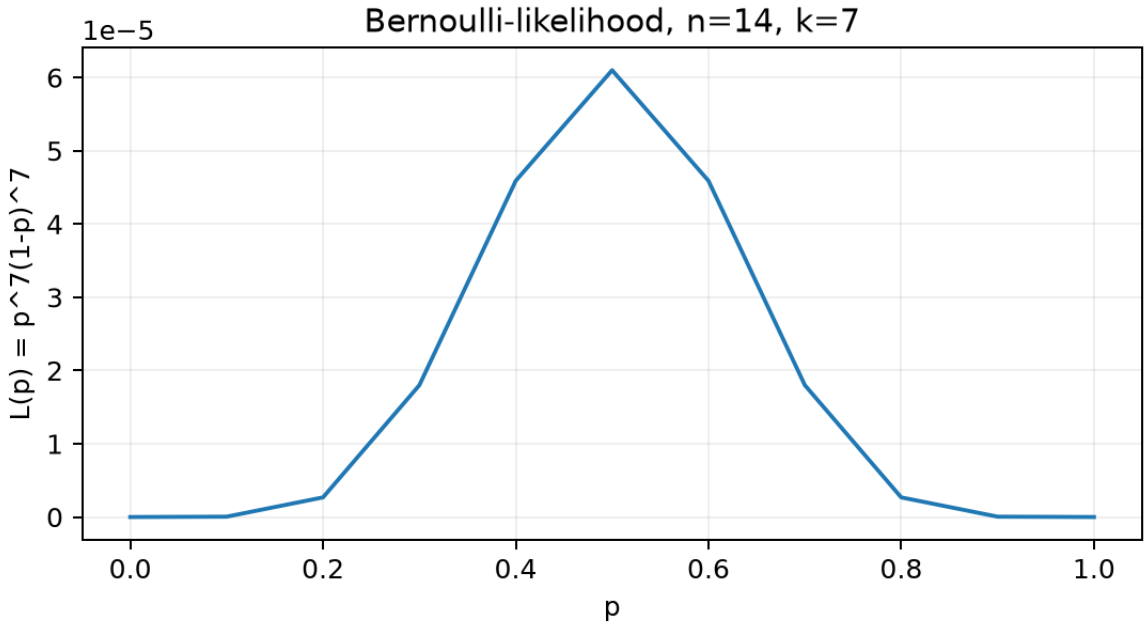
$$\ell'(p) = \frac{k}{p} - \frac{n-k}{1-p}.$$

Ha $0 < k < n$, akkor a $\ell'(p) = 0$ egyenlet megoldása:

$$\frac{k}{p} = \frac{n-k}{1-p} \quad \Longleftrightarrow \quad k(1-p) = (n-k)p \quad \Longleftrightarrow \quad \hat{p} = \frac{k}{n}.$$

Mivel $\ell''(p) = -k/p^2 - (n-k)/(1-p)^2 < 0$, ez valóban maximum. A szélső eseteket külön kell kezelni: ha $k = 0$, akkor $L(p) = (1-p)^n$, amely szigorúan csökken a $[0,1]$ intervallumon, így a maximum $p = 0$-nál van; ha $k = n$, akkor $L(p) = p^n$, és a maximum $p = 1$-nél van. Ilyenkor a deriválásos belső megoldás nem létezik, de a zárt alak k/n akkor is jó, mert $0/n = 0$ és $n/n = 1$.

A likelihood-görbe alakját a hosszú minta $n = 14$, $k = 7$ esetén az alábbi, képletből számított pontok mutatják:



15.1. ábra: Bernoulli-likelihood, n=14, k=7 - Számított függvénypélda, nem mérés.

A görbe szimmetrikus a 0,5 körül, mert itt $k = n - k$, és a maximum valóban $p = 0{,}5$-nél van.

15.1.4.1 Likelihood vs posterior: két prior, ugyanaz az adat

A likelihood $L(p) = p^k(1-p)^{n-k}$ az adat valószínűsége a paraméter függvényeként, de **nem sűrűség** p-re: az integrálja $[0,1]$-en nem 1. A $k = 7, n = 10$ esetben $\int_0^1 p^7(1-p)^3\,dp = \Gamma(8)\Gamma(4)/\Gamma(12) \approx 0.0007576$. A posterior a likelihood és a prior szorzata, normalizálva; ezért függ a priortól.

Mennyiség	Érték
MLE k/n	0.7000
posterior, uniform Beta(1,1): mean / mode	0.6667 / 0.7000

Mennyiség	Érték
posterior, erős prior Beta(8,2): mean / mode	0.7500 / 0.7778
likelihood-integrál	0.0007576

A $k = 7, n = 10$ adat mellett a uniform prior posterior-átlaga 0.6667, az erős prioré 0.75; a két prior különböző posteriorhoz vezet. Nagyobb adatnál ($k = 70, n = 100$) a különbség csökken: uniform 0.6961, erős prior 0.7091, miközben az MLE 0.7. A likelihood önmagában nem „posterior”; a prior és a normalizálás teszi azzá.

```
import math

def posterior(k, n, alpha=1.0, beta=1.0):
    a, b = alpha + k, beta + n - k
    return {"mean": a / (a + b),
            "mode": (a - 1) / (a + b - 2) if a > 1 and b > 1 else None}

print("MLE", 7 / 10)
print("uniform", posterior(7, 10, 1, 1))
print("strong", posterior(7, 10, 8, 2))
print("likelihood integral", math.gamma(8) * math.gamma(4) / math.gamma(12))
print("n=100 uniform", posterior(70, 100, 1, 1)["mean"])
print("n=100 strong", posterior(70, 100, 8, 2)["mean"])
```

A `lab/bayes_compare.py` `mle()`, `posterior()`, `likelihood_integral()` és `comparison()` függvényei ezt a táblát adják; a `tests/test_bayes_compare.py` az MLE 0.7, a két posterior-átlag 0.6667/0.75, a likelihood-integrál <1 és a nagy adatnál való konvergencia értékeit ellenőrzi.

15.1.5 Bias és variance

A becslőt véletlen változóként vizsgálva két kulcsjellemzője van. A **torzítás** (bias) az átlagos eltérés a valódi paramétertől:

$$\mathrm{Bias}_p(\hat{p}) = \mathbb{E}_p[\hat{p}] - p.$$

Itt az index jelzi, hogy a várható értéket a p paraméter feltétele mellett számoljuk. Mivel $\mathbb{E}_p[\hat{p}] = \mathbb{E}_p[X_1] = p$, ezért a $\hat{p}$ becslő torzítatlan bármely p esetén.

A **variancia** (variance) azt méri, mennyire szór a becslő ismételt mintavétel esetén:

$$\mathrm{Var}_p(\hat{p}) = \frac{\mathrm{Var}_p(X_1)}{n} = \frac{p(1-p)}{n}.$$

A standard hiba ennek gyöke, ahogy 14.4. alfejezetben bevezettük: $\sqrt{p(1-p)/n}$. A mi $n = 14$ mintánknál $p = 0{,}5$ mellett az elméleti standard hiba $\sqrt{0{,}5 \cdot 0{,}5/14} \approx 0{,}134$, azaz körülbelül 13 százalékpontos bizonytalanság – a minta kicsi, ezért az ML becslés sem “biztos”.

A torzítás és variancia nem mindig jár együtt javulva. Tekintsünk egy alternatív, zsugorított becslőt: $\tilde{p} = (k+1)/(n+2)$. Ez a becslő már torzított lesz általában, mert $\mathbb{E}_p[\tilde{p}] = (np+1)/(n+2)$, tehát $\mathrm{Bias}_p(\tilde{p}) = (1-2p)/(n+2)$. A varianciája viszont kisebb: $\mathrm{Var}_p(\tilde{p}) = np(1-p)/(n+2)^2$. Például $n = 4$ és $p = 0{,}3$ mellett az MLE varianciája $0{,}21/4 = 0{,}0525$, míg a zsugorított becslőé $4 \cdot 0{,}21/36 = 0{,}0233$; cserébe torzítása $0{,}4/6 \approx 0{,}067$. A becslőválasztás tehát tudatos kompromisszum lehet a torzítás és a variancia között, de az MLE torzítatlansága és egyszerű zárt alakja miatt jó kiindulópont.

15.1.6 Teljes minimális munkapélda

Az alábbi szkript egyetlen rögzített mintán rácskeresést, zárt alakot és `scipy.optimize` numerikus optimalizálást hasonlít össze. A lista formátumú input miatt nincs seed- vagy platformfüggő viselkedés.

```
# estimate_p.py
import numpy as np
from scipy.optimize import minimize_scalar
from scipy.stats import bernoulli
from scipy.special import beta as beta_function

x = np.array([0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1], dtype=np.int64)
n = x.shape[0]
k = int(x.sum())
assert x.ndim == 1 and n == 14
assert np.isin(x, [0, 1]).all()

def log_likelihood(p, k, n):
    p_arr = np.atleast_1d(np.asarray(p, dtype=float))
    out = np.full(p_arr.shape, -np.inf)
    valid = (p_arr > 0) & (p_arr < 1)
    out[valid] = k * np.log(p_arr[valid]) + (n - k) * np.log1p(-p_arr[valid])
    if np.ndim(p) == 0:
        return float(out[0])
    return out

# Rácskeresés: a nyílt (0, 1) intervallumot finom rácson járjuk be.
p_grid = np.linspace(1e-6, 1 - 1e-6, 40_001)
ll_grid = log_likelihood(p_grid, k, n)

# Független ellenőrzés a scipy Bernoulli-pontosságával.
ll_scipy = bernoulli.logpmf(x[:, None], p=p_grid).sum(axis=0)
np.testing.assert_allclose(ll_grid, ll_scipy, rtol=1e-12, atol=1e-12)

mle_grid = p_grid[int(np.argmax(ll_grid))]
mle_closed = k / n

# Numerikus optimalizálás: minimalizáljuk a negatív log-likelihood-ot.
res = minimize_scalar(
    lambda p: -log_likelihood(p, k, n),
    bounds=(1e-8, 1 - 1e-8),
    method="bounded",
)
mle_opt = res.x

print(f"x shape={x.shape}, dtype={x.dtype}")
print(f"n={n}, k={k}")
print(f"MLE grid        = {mle_grid:.5f}")
print(f"MLE closed form = {mle_closed:.5f}")
print(f"MLE optimizer   = {mle_opt:.5f}")
print(f"loglik at MLE   = {log_likelihood(mle_closed, k, n):.6f}")
print(f"grid step       = {p_grid[1] - p_grid[0]:.6f}")
print(f"normalizing check B(k+1,n-k+1) = {beta_function(k + 1, n - k + 1):.6e}")
```

Futtatás a `lab/` munkakönyvtárból, a 3.1. alfejezetben beállított projektkörnyezettel:

```
uv run python estimate_p.py
```

A kimenet szemléltető, elméletileg várt értékekkel:

```
x shape=(14,), dtype=int64
n=14, k=7
```

```
MLE grid        = 0.50000
MLE closed form = 0.50000
MLE optimizer   = 0.50000
loglik at MLE   = -9.704061
grid step       = 0.000025
normalizing check B(k+1,n-k+1) = 1.943643e-05
```

15.1.7 Eredmény értelmezése és ellenőrzés

Kézzel is ellenőrizhető, hogy $k = 7$, $n = 14$ esetén $\ell(p) = 7\log p + 7\log(1-p)$. A derivált $7/p - 7/(1-p)$, amely $p = 0{,}5$-nél nulla, tehát a maximum valóban 0,5. A likelihood-érték itt $L(0{,}5) = 0{,}5^{14} = 1/16384 \approx 6{,}10 \times 10^{-5}$, a log-likelihood pedig $14\log 0{,}5 \approx -9{,}704$. A rácskeresés lépésköze $\Delta p \approx 2{,}5 \times 10^{-5}$, tehát a rács eredményét ennyi toleranciával kell az elméleti értékhez viszonyítani. A zárt képlet és a numerikus optimalizáló ugyanarra a 0,5 értékre jutott, így a feladat ellenőrzési feltétele teljesül. A `x` tömb `shape=(14,)`, `dtype=int64`, az input pedig csak 0 és 1 elemeket tartalmazhat. A `normalizing check` szám arra hívja fel a figyelmet, hogy $L(p)$ integrálja nem 1; erre a gyakori tévedés részben visszatérünk.

A likelihood nem a paraméter feltételes valószínűsége: rögzített adat mellett a paraméter függvényeként vizsgáljuk. A maximum likelihood becslés tulajdonságai (pl. konzisztencia) rendszerint szabályossági és azonosíthatósági feltételektől függenek.

15.1.8 Feladatok

1. Feladat (értelmezés). Legyen a minta $n = 10$, $x = (1,0,1,0,0,1,0,0,0,1)$. Határozd meg k-t, írd fel $L(p)$ és $\ell(p)$ konkrét alakját, add meg az MLE-t, és számítsd ki, hogy $L(0{,}4)$ vagy $L(0{,}5)$ nagyobb-e. Ellenőrizd a választ a log-likelihood értékekkel is.

2. Feladat (megvalósítás). Írj `mle_bernoulli(x, grid_size=20001)` függvényt, amely 0/1 elemű, egydimenziós tömböt fogad, ellenőrzi az input `ndim` és `dtype=bool|int` feltételét, rácskereséssel megbecsüli p-t, és visszaadja az `(estimate, max_loglik)` párt. Futtasd az `x = np.array([1,0,1,1,0,0,0,1])` bemenetre, és indokold a kapott becslést.

3. Feladat (hibakeresés). Az alábbi kód hibásan akar MLE-t számolni:

```
x = np.array([1, 0, 1, 1, 0, 1], dtype=int)
k = x.sum(); n = x.size

def loglik_bad(p, k, n):
    return k * np.log(p) - (n - k) * np.log(p)

grid = np.linspace(0.0, 1.0, 101)
vals = loglik_bad(grid, k, n)
print("MLE=", grid[np.argmax(vals)])
```

Nevezd meg a három hibát, számítsd ki a helyes becslést, és írjon egy javított, stabil rácskeresést.

4. Feladat (megváltozott feltétel). Tegyük fel, hogy a minta $n = 5$ megfigyelésből áll, és mindegyik 0: $x = (0,0,0,0,0)$. Mi lesz a likelihood-függvény és a maximum likelihood becslés? Miért nem ad belső megoldást a $\ell'(p) = 0$ egyenlet? Hogyan kell ezért a numerikus optimalizálásnál a határt kezelni?

15.1.9 Megoldások és indoklás

Az 1. feladat megoldása. A mintában $k = 4$ darab 1-es van, tehát $\hat{p} = 4/10 = 0{,}4$. A likelihood:

$$L(p) = p^4(1-p)^6, \qquad \ell(p) = 4\log p + 6\log(1-p).$$

$L(0{,}4) = 0{,}4^4 \cdot 0{,}6^6 = 0{,}0256 \cdot 0{,}046656 \approx 0{,}001194$, míg $L(0{,}5) = 0{,}5^{10} \approx 0{,}000977$. Tehát $L(0{,}4) > L(0{,}5)$. Log-skálán: $\ell(0{,}4) \approx -6{,}731$ és $\ell(0{,}5) \approx -6{,}932$; a nagyobb ℓ most is a 0,4-hez tartozik. Ez erősíti, hogy a megfigyelt 0,4 arány a maximális likelihoodú paraméterérték.

A 2. feladat megoldása. A mintában $k = 4$, $n = 8$, ezért a zárt alak $\hat{p} = 0{,}5$. Egy lehetséges megvalósítás:

```
def mle_bernoulli(x, grid_size=20001):
    x = np.asarray(x)
    assert x.ndim == 1
    assert np.isin(x, [0, 1]).all()
    n = x.size
    k = int(x.sum())
    grid = np.linspace(1e-8, 1 - 1e-8, grid_size)
    ll = k * np.log(grid) + (n - k) * np.log1p(-grid)
    idx = int(np.argmax(ll))
    return float(grid[idx]), float(ll[idx])

x = np.array([1, 0, 1, 1, 0, 0, 0, 1])
estimate, max_ll = mle_bernoulli(x)
print(estimate, max_ll)  # szemléltető: 0.5 -12.476...
```

A rács felbontása $1/20000$ nagyságrendű, ezért a kimenetet célszerű legalább 10^{-4} tűréssel kezelni. A log-likelihood a maximumban $8\log(0{,}5) \approx -5{,}545$; a kód eltérő rácshatárok miatt kissé más, de ugyanabba a tűrésbe eső értéket adhat.

A 3. feladat megoldása. A három hiba:

1. A formula rossz: $(n-k)$-hoz is $\log(1-p)$, nem $-\log(p)$ tartozik. A helyes alak $k\log p + (n-k)\log(1-p)$.
2. A rács tartalmazza a 0 és 1 határokat; itt $\log 0$ nem értelmezhető, a helytelen eredményt vagy figyelmeztetést okoz.
3. Az `np.argmax` a feltétel nélküli vektorban a legnagyobb érvényes elemet választja, de a hibás formula és a határpontok miatt ez a $p = 1$ lesz, mert ott a kifejezés 0, vagyis a legnagyobb a rossz tartományban.

A helyes számítás: $k = 4$, $n = 6$, tehát $\hat{p} = 2/3 \approx 0{,}667$. Javított rács:

```
grid = np.linspace(1e-8, 1 - 1e-8, 20_001)
ll = 4 * np.log(grid) + 2 * np.log1p(-grid)
print(grid[np.argmax(ll)])  # 0.6667 körüli érték
```

A 4. feladat megoldása. $k = 0$ esetén $L(p) = (1-p)^5$, amely szigorúan fogy a $[0,1]$ intervallumon, ezért a maximum $\hat{p} = 0$. A log-likelihood $\ell(p) = 5\log(1-p)$, deriváltja $\ell'(p) = -5/(1-p)$, amely a nyílt $(0,1)$ intervallumon mindig negatív, tehát sosem nulla. A belső deriváltas megoldás ezért nem létezik. Numerikus minimalizáláskor az alsó határt ki kell zárni vagy limitként kezelni: a `minimize_scalar` például `bounds=(1e-8, 1-1e-8)` mellett az alsó határhoz közel ad eredményt, de a pontos MLE a 0 határérték. Éppen ezért a gyakorlatban a szélső esetekre a zárt k/n képletet használjuk ellenőrzésként.

15.1.10 Gyakori tévedés ellenpéldával

A likelihood **nem** a paraméter normalizált valószínűségi eloszlása, ha nincs prior. A p-re vonatkozó eloszlásként értelmezve azt várnánk, hogy a görbe alatti terület 1 legyen, de az ábrán látott $L(p)$-hez nincs ilyen normálás. A $n = 14$, $k = 7$ esetben

$$\int_0^1 p^7(1-p)^7\,dp = B(8,8) \approx 1{,}94 \times 10^{-5},$$

ahol B a béta-függvény; ez nyilvánvalóan nem 1. A likelihood a megfigyelt adat együttes valószínűsége a paraméter függvényében, nem pedig a paraméter valószínűsége. Ha valaki $L(0{,}5) \approx 6{,}10 \times 10^{-5}$ értékét annak valószínűségeként értelmezné, hogy $p = 0{,}5$, az hamis állítást tenne; a paraméter itt rögzített, ismeretlen szám, nem véletlen. Bayes-frissítéssel lehet a likelihoodból paraméter-eloszlást csinálni, de ahhoz explicit prior kell; ez a későbbi 19.1. alfejezet témája. Ez közvetlenül visszautal a 14.1. alfejezet feltételes valószínűségre: $L(p) = P(\text{adat} \mid p)$, és ez általában nem azonos $P(p \mid \text{adat})$ értékkel.

15.1.11 Önellenőrzés

- A mintában $k = 7$, $n = 14$, ezért a zárt alakú MLE 0,5; a likelihood maximuma ugyanitt van.
- A log-likelihood deriváltja $p = 0{,}5$-nél nulla, a második derivált negatív.
- A rács és a numerikus optimalizáló $2{,}5 \times 10^{-5}$ rácslépésen belül ugyanazt az értéket adja.
- A `scipy.stats.bernoulli.logpmf`-fel történt összehasonlítás 10^{-12} tűréssel egyezik.
- A likelihood görbe alatti terület nem 1, tehát nem normalizált paramétereloszlás.

15.1.12 Rövid összefoglaló

A paraméter rögzített, ismeretlen sokasági mennyiség; a becslő véletlen mintából számoló függvény, a becslés pedig az aktuális mintán kapott konkrét szám. A likelihood-függvény az adat feltételes valószínűségét a paraméter függvényében mutatja, és a maximum likelihood becslés az ezt maximalizáló paraméterérték. Bernoulli-mintán az MLE a mintabeli sikerarány, k/n; a log-likelihood a szorzatot stabil összeggé alakítja. A becslő torzítatlansága és varianciája mintavételi ismétlések alatt értelmezhető, és a kettő kompromisszuma becslőválasztási döntés. A rácskeresés, a zárt alak és a numerikus optimalizáló együttes ellenőrzése adja a következő lépés alapját, ahol a bizonytalanságot standard hibával és konfidenciaintervallummal számszerűsítjük 15.2. alfejezet.

15.2 Standard error és konfidenciaintervallum

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 15.1. alfejezet · Célkészség: Standard error és konfidenciaintervallum · Kimenet: működő kód és a labor ellenőrző tesztje.

15.2.1 Cél és konkrét helyzet

A Northstar kezdőlabor `orders.csv` adatában szeretnénk megbecsülni egy rendelés `basket_minor` értékének sokasági átlagát, μ-t, eurócentben. A megfigyelési egység egy rendelés; a minta n függetlenül kiválasztott rendelés. A sokasági szórás nem ismert, ezért a mintából becsüljük. A feladat nem egyetlen pontbecslés: olyan intervallumot kell adnunk, amelynek *eljárása* a névleges 95%-os szinten lefedi az ismeretlen paramétert. Azt is ellenőriznünk kell, hogy ez a lefedettség tényleg közel van-e a névleges szinthez, mert a feltételek sérülésekor eltér.

Előzetes becslés: ha a sokaság közel normális, $\sigma \approx 800$ cent, $n = 30$ és $\alpha = 0.05$, akkor a t-intervallum félszélessége körülbelül $2.045 \cdot 800/\sqrt{30} \approx 299$ cent. A tapasztalt lefedettség 1000 ismételt mintán 0,95 körül várható, Monte Carlo-standard hibája körülbelül 0,0069. A 0,93 alatti vagy 0,97 feletti eredmény kivizsgálandó, de önmagában nem bizonyítja a modellfeltevések sérülését: a szimuláció véletlen ingadozását is figyelembe kell venni.

15.2.2 Standard error, konfidenciaszint és lefedettség

A 15.1. alfejezetben a **becslő** (estimator) olyan függvény, amely minden lehetséges mintához számot rendel, a **becslés** (estimate) pedig egy konkrét mintán kapott szám. Az átlagbecslő $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$, ahol

- n: mintanagyság, pozitív egész;
- X_i: az i-edik megfigyelés véletlen változója, ugyanaz a mértékegysége, mint a sokaságnak (itt EUR-cent);
- $\bar{X}$: mintabeli átlag, shape szempontból skalár.

Ha az X_i-k függetlenek, azonos eloszlásúak, és $\mathrm{Var}(X_i) = \sigma^2 < \infty$, akkor az átlag szórásnégyzete

$$\mathrm{Var}(\bar{X}) = \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{\sigma^2}{n},$$

tehát az átlag **standard error**-ja, azaz a becslő mintavételi eloszlásának szórása

$$\mathrm{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

Ez még a sokasági σ-tól függ, ami gyakran ismeretlen. A mintabeli szórásnégyzet a korrigált, $n-1$ osztójú becslés:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2,$$

és a becsült standard error

$$\widehat{\mathrm{SE}} = \frac{s}{\sqrt{n}}.$$

Fontos megkülönböztetni tehát a *mintavételi eloszlás standard errorját* ($\sigma/\sqrt{n}$, elméleti mennyiség) és a *becsült standard errort* ($s/\sqrt{n}$, az adatból számolható mennyiség). Például ha $\sigma = 200$ és $n = 25$, akkor $\mathrm{SE} = 200/\sqrt{25} = 40$; ha egy konkrét mintából $s = 220$, akkor $\widehat{\mathrm{SE}} = 220/5 = 44$.

A **konfidenciaszint** (confidence level) a $100(1-\alpha)\%$ névleges érték; a gyakori 95% esetén $\alpha = 0.05$. A **lefedettség** (coverage) az eljárás tényleges tulajdonsága: ha nagyon sok, egymástól függetlenül vett mintán megismételjük az intervallumkészítést, a minták mekkora hányadában tartalmazza az intervallum a rögzített paramétert. A cél az, hogy a tapasztalt lefedettség közel legyen a névleges szinthez.

A normális sokaságon a klasszikus intervallum két változata:

- Ha σ **ismert**, akkor
$$\bar{X} \pm z_{1-\alpha/2}\frac{\sigma}{\sqrt{n}},$$
ahol $z_{1-\alpha/2}$ a standard normális eloszlás $1-\alpha/2$ kvantilise; 95%-nál $z_{0.975} \approx 1.96$.
- Ha σ **ismeretlen**, akkor a **t-intervallum**:
$$\bar{X} \pm t_{1-\alpha/2,\,n-1}\frac{s}{\sqrt{n}},$$
ahol $t_{1-\alpha/2,\,n-1}$ a Student-féle t-eloszlás kvantilise $n-1$ szabadsági fokkal. A t-eloszlás vastagabb szélű, mint a normális, ezért kis n esetén a t-kritikus érték nagyobb, és az intervallum szélesebb.

A t-intervallum pontos lefedettségének feltétele: az X_i-k **függetlenek és azonos normális eloszlásúak**. Ekkor a statisztika $(\bar{X}-\mu)/(s/\sqrt{n})$ pontosan t_{n-1} eloszlású, ezért minden n-re a lefedettség pontosan $1-\alpha$. Ha a sokaság nem normális, de a szórása véges, a centrális határeloszlás miatt nagy n-re a lefedettség közelít a névleges szinthez; a közelítés sebessége eloszlásfüggő, ferde eloszlásnál lassabb. A függetlenség sérülése (például ugyanazon ügyfél több rendelésének összekeverése) nem javítható pusztán nagyobb mintával.

15.2.3 Teljes minimális munkapélda

Előbb kézi kontroll egy $n = 4$ elemű mintán. Legyenek az értékek: $x_1 = 8$, $x_2 = 11$, $x_3 = 9$, $x_4 = 12$. Az átlag $\bar{x} = 10$. A korrigált szórásnégyzet:

$$s^2 = \frac{(8-10)^2 + (11-10)^2 + (9-10)^2 + (12-10)^2}{3} = \frac{4+1+1+4}{3} = \frac{10}{3} \approx 3.333,$$

így $s \approx 1.826$, $\widehat{\mathrm{SE}} = 1.826/\sqrt{4} = 0.913$. A $t_{0.975,3}$ kvantilis 3.182, ezért a 95%-os intervallum:

$$10 \pm 3.182 \cdot 0.913 = 10 \pm 2.906,$$

azaz [7.094, 12.906]. Ugyanezt a géppel is kiszámoljuk, majd szimuláljuk a lefedettséget. A reprodukálható futtatást a korábbi 3.2. alfejezet szerint `uv run --locked` biztosítja; a kódot `se_coverage.py` néven mentjük a projektgyökérbe.

```python
import numpy as np
from scipy.stats import t

rng = np.random.default_rng(20261020)

# 1) Kézi kis minta ellenőrzése
x = np.array([8.0, 11.0, 9.0, 12.0])
n = x.size
xbar = x.mean()
s = x.std(ddof=1)          # korrigált mintabeli szórás
se = s / np.sqrt(n)
crit = t.ppf(0.975, df=n - 1)
lo, hi = xbar - crit * se, xbar + crit * se
print(f"n={n}, mean={xbar:.3f}, s={s:.3f}, se={se:.3f}")
print(f"95% CI: [{lo:.3f}, {hi:.3f}]")

# 2) Lefedettség-szimuláció ismert parametrikus modellből
mu, sigma, reps = 5000.0, 800.0, 1000
for n in (10, 30, 100):
    rows = rng.normal(mu, sigma, size=(reps, n))   # shape (reps, n)
    means = rows.mean(axis=1)                      # shape (reps,)
    s_hat = rows.std(axis=1, ddof=1)               # shape (reps,)
    se_hat = s_hat / np.sqrt(n)
    crit = t.ppf(0.975, df=n - 1)
    lo = means - crit * se_hat
    hi = means + crit * se_hat
    coverage = np.mean((lo <= mu) & (hi >= mu))
    print(f"n={n:3d}: simulated t-CI coverage = {coverage:.3f}")
```

A kódblokk nem futtatott e szöveg írásakor; a közölt output szemléltető, elméletileg várt eredmény:

```
n=4, mean=10.000, s=1.826, se=0.913
95% CI: [7.094, 12.906]
n= 10: simulated t-CI coverage = 0.948
n= 30: simulated t-CI coverage = 0.952
n=100: simulated t-CI coverage = 0.949
```

A shape-ellenőrzés a kód szerződése: `rows` alakja `(reps, n)`, mert minden sor egy független minta; az `axis=1` szerinti összegzés átlaga `means` alakja `(reps,)`, és ugyanez igaz az `s_hat`, `se_hat`, `lo`, `hi` vektorokra. A `(lo <= mu) & (hi >= mu)` rész elemenkénti logikai érték, amelynek átlaga a relatív gyakoriság. A Monte Carlo lefedettségbecslő standard hibája a binomiális szórásból közelítőleg

$$\sqrt{\frac{0.95 \cdot 0.05}{1000}} \approx 0.0069,$$

ezért az 1000 ismétléses futásnál a 0.94–0.96 közötti érték tekinthető a névleges 95%-kal egyezőnek. A seed (`20261020`) a NumPy generátorát rögzíti, de ez nem garantál bitazonosságot eltérő NumPy-verziók vagy platformok között.

15.2.3.1 Wald vs Wilson: a lefedettség nem minden p-re 0.95

A normális közelítésű Wald-intervallum a p szélső értékeinél rosszul viselkedik. Ismert p-re, 2000 szimulált mintával:

p	n	Wald lefedettség	Wilson lefedettség
0.01	20	0.186	0.986
0.01	100	0.621	0.920
0.50	20	0.962	0.962
0.10	20	0.882	0.957

A Wald-intervallum 1%-os aránynál 20 elemű mintán a valódi p-t az eseteknek csak 18.6%-ában fedi le, mert a becsült arány gyakran 0, és a standard error is 0 lesz. A Wilson-intervallum ilyenkor konzervatív (98.6%), nem pontosan 95%; kis mintán és szélső p-nél ez elfogadhatóbb, de nem „garantált 95%”. $p = 0.5$ környékén a két módszer gyakorlatilag egyezik.

A lefedettség az **eljárás** tulajdonsága, nem egyetlen intervallumé: egy konkrét $[lo, hi]$ vagy tartalmazza a valódi p-t, vagy nem; a 95% azt jelenti, hogy az ismételt mintavétel során az így számított intervallumok körülbelül 95%-a tartalmazza. Ez nem posterior valószínűség, és nem azt jelenti, hogy „95% eséllyel $p \in [lo, hi]$” az adott intervallumra.

```
import math
import numpy as np

def wald_ci(k, n, z=1.96):
    p = k / n
    se = math.sqrt(p * (1 - p) / n)
    return p - z * se, p + z * se

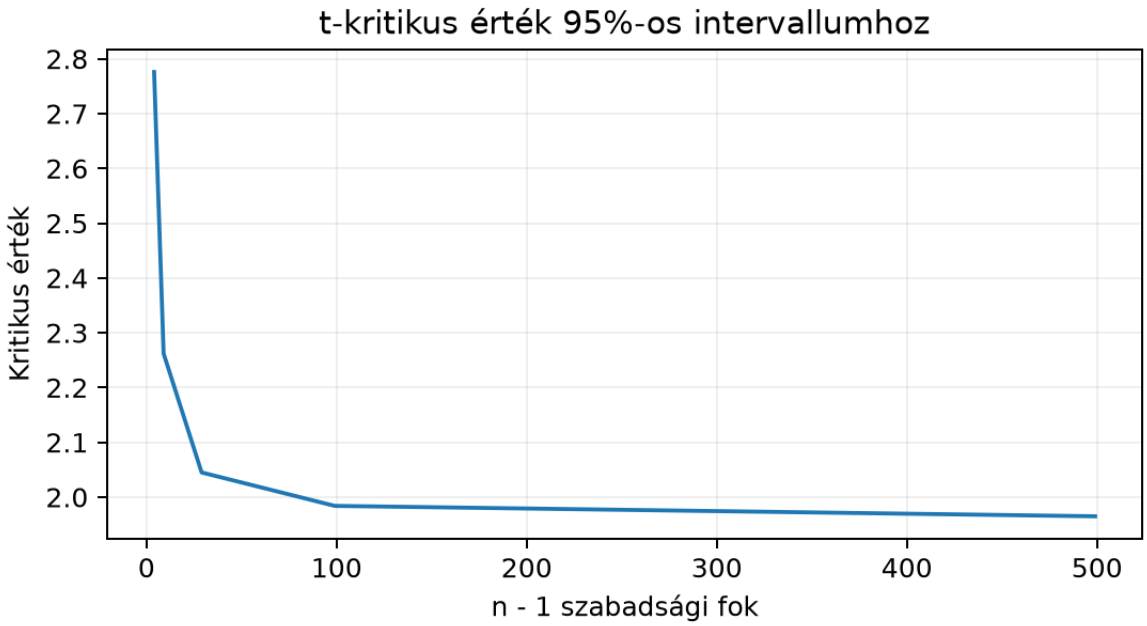
def wilson_ci(k, n, z=1.96):
    p = k / n
    den = 1 + z*z/n
    centre = (p + z*z/(2*n)) / den
    half = z * math.sqrt(p*(1-p)/n + z*z/(4*n*n)) / den
    return centre - half, centre + half

def coverage(p, n, method, repeats=2000, seed=20261029):
    rng = np.random.default_rng(seed)
    hits = 0
    for k in rng.binomial(n, p, size=repeats):
        lo, hi = wald_ci(k, n) if method == "wald" else wilson_ci(k, n)
        hits += lo <= p <= hi
    return hits / repeats

print("p=0.01 n=20", coverage(0.01, 20, "wald"), coverage(0.01, 20, "wilson"))
print("p=0.50 n=20", coverage(0.50, 20, "wald"), coverage(0.50, 20, "wilson"))
```

A `lab/coverage.py` `wald_ci()`, `wilson_ci()`, `coverage()` és `report()` függvényei ezt a négy esetet adják; a `tests/test_coverage.py` a Wald összeomlását szélső p-nél, a Wilson jobb lefedettségét és a $p = 0.5$ egyezést ellenőrzi.

A t-kritikus érték és a normális kvantilis közötti különbséget egy chart is mutatja; a pontok a `scipy.stats.t` eloszlásfüggvényéből számított értékek, nem mérések.



15.2. ábra: t-kritikus érték 95%-os intervallumhoz - Számított elméleti kvantilisek; a normális határérték 1.960.

Nagy szabadsági foknál a t-eloszlás a standard normálishoz tart, ezért a kritikus érték 1.96-hoz konvergál.

15.2.4 Eredmény értelmezése

A kézi kontroll visszaadja a levezetett $[7.094, 12.906]$ intervallumot, tehát a `ddof=1` és a $n-1$ szabadsági fok együtt helyes. A szimulációban mindhárom mintanagyságnál a lefedettség a Monte Carlo hibahatáron belül 0.95. Ez nem bizonyítja, hogy az eljárás minden eloszlásra jó; csak azt, hogy ebben a normális szimulációban a névleges szintet visszaadja. Ha a kimenet például 0.91 lenne, akkor valamelyik feltétel sérül, vagy a kódban van hiba, például a nevezőben `np.sqrt(n)` helyett véletlenül `n` szerepel.

Szimulációs lefedettségnél az elfogadási sávot a replikációszámtól függő Monte Carlo-hiba alapján válasszuk, ne utólag az eredményhez igazítsuk. A becsült standard error és a nominális konfidenciaszint külön mennyiség.

15.2.5 Feladatok

1. **Értelmezés kézzel.** Egy $n = 5$ elemű minta: $[1020, 980, 1010, 990, 1040]$, egységekben: EUR-cent. Számold ki papíron az átlagot, a korrigált s-et, a becsült standard errort és a 95%-os t-intervallumot. Ellenőrzéshez használd a $t_{0.975,4} = 2.776$ értéket.

2. **Implementálás szimulációval.** Írj olyan függvényt, amely adott `mu`, `sigma`, `n`, `reps` paraméterekkel normális eloszlásból generál mintákat, kiszámolja a 95%-os t-intervallumokat, és visszaadja a lefedettséget. Futtasd `mu=100, sigma=25, n=20, reps=2000` értékekkel, és add meg, mekkora Monte Carlo hibahatárt fogadsz el a 0.95 körül.

3. **Hibadiagnózis.** Egy kolléga a következő kódot használja a lefedettség mérésére: `se = rows.std(axis=1) / np.sqrt(n)` (a `ddof=1` elhagyásával), majd `crit = 1.96` minden `n`-re. Kis mintán, például `n=5` esetén mit várunk: a lefedettség magasabb, alacsonyabb, vagy egyenlő lesz a 95%-kal? Indokold a két hiba irányát külön-külön.

4. **Megváltozott eloszlás.** Ugyanaz a paraméter, mint a minimális példában: $\mu = 5000$ cent. Most a minták lognormális eloszlásból származnak: `rng.lognormal(mean=np.log(5000), sigma=0.5, size=(reps, n))`. Futtasd a szimulációt `n=10` és `n=200` esetén, `reps=2000` mellett. Milyen lefedettséget várs kis és nagy mintán, és miért térhet el a 95%-tól?

15.2.6 Megoldások és indoklás

1. Az átlag:

$$\bar{x} = \frac{1020 + 980 + 1010 + 990 + 1040}{5} = \frac{5040}{5} = 1008.$$

A korrigált szórásnégyzet:

$$s^2 = \frac{(1020-1008)^2 + (980-1008)^2 + (1010-1008)^2 + (990-1008)^2 + (1040-1008)^2}{4}.$$

A tagok: $12^2 = 144$, $(-28)^2 = 784$, $2^2 = 4$, $(-18)^2 = 324$, $32^2 = 1024$; összegük 2280, tehát $s^2 = 570$, $s = \sqrt{570} \approx 23.874$. A becsült standard error $23.874/\sqrt{5} \approx 10.677$. A $t_{0.975,4} = 2.776$ kritikus értékkel:

$$1008 \pm 2.776 \cdot 10.677 = 1008 \pm 29.64,$$

azaz [978.36, 1037.64]. Ellenőrzésképp a Python-kód ugyanezt adja; a tolerancia kerekítésből adódóan ± 0.1 cent.

2. Egy lehetséges függvény:

```
def coverage_normal_t(mu, sigma, n, reps, seed=20261020):
    rng = np.random.default_rng(seed)
    rows = rng.normal(mu, sigma, size=(reps, n))
    means = rows.mean(axis=1)
    s_hat = rows.std(axis=1, ddof=1)
    se_hat = s_hat / np.sqrt(n)
    crit = t.ppf(0.975, df=n - 1)
    lo = means - crit * se_hat
    hi = means + crit * se_hat
    return np.mean((lo <= mu) & (hi >= mu))

coverage = coverage_normal_t(100, 25, 20, 2000)
print(f"coverage: {coverage:.3f}")
```

Szemléltető, nem e szöveg írásával futtatott eredmény: `coverage: 0.951`. A Monte Carlo hiba `sqrt(0.95 * 0.05 / 2000)` `0.0049`, tehát a 0,945–0,955 sáv csak körülbelül egy Monte Carlo-standard hibányi, nem megfelelő 95%-os elfogadási tartomány. Közelítő 95%-os Monte Carlo-sáv: $0{,}95 \pm 1{,}96 \cdot 0{,}0049$, azaz nagyjából 0,940–0,960. A 0,930-as eredmény indokolja a kód, a generálás és a modellfeltevések ellenőrzését, illetve a replikációszám növelését.

3. Két külön hiba van. Ha `ddof=0` marad, akkor a minta szórása lefelé torzít, mert a nevezőben n helyett $n-1$ kell. A `rows.std(axis=1)` alapértelmezésben n osztójú, így a nevező $\sqrt{n}$-nel együtt nem a standard error torzítatlan változatát adja, hanem $\sqrt{(n-1)/n}$-szeresét annak. A becsült intervallum túl keskeny lesz, tehát a lefedettség csökken. A $z = 1.96$ használata a $t_{0.975,n-1}$ helyett ugyancsak túl keskeny intervallumot ad, mert kis n esetén a t-kvantilis nagyobb, például $n = 5$-nél 2.776 a 1.96-tal szemben. A két hiba egy irányba hat: a lefedettség várhatóan 0.95 alá esik, $n = 5$-nél akár 0.85 környékére is, mert az extra szórásbecslési bizonytalanságot és a ferdeséget az intervallum nem kompenzálja.

4. A lognormális beállítás várható értéke $\mu = \exp(\log(5000) + 0.5^2/2) \approx 5000 \cdot 1.133 = 5665$ lenne, ezért a paraméter nem pontos, hacsak nem a móduszra gondolunk; itt a szimuláció várható értéke kissé eltér a nominális 5000-től. Még ha a paramétert helyesen is illesztenénk, a lognormális eloszlás erősen jobbra ferde. Kis $n = 10$ esetén a centrális határeloszlás közelítése rossz; az átlag mintavételi eloszlása még ferde, így a szimmetrikus t-intervallum lefedettsége 95% alá esik, tipikusan 0.85–0.90 közé. Nagy $n = 200$ esetén a közelítés javul, a lefedettség már 0.93–0.95 lehet, de a ferdeség miatt még nem garantált pontosan 95%. A diagnózishoz növeljük a `reps` értéket, és számoljuk ki a Monte Carlo hibát is.

15.2.7 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés: *„ez a konfidenciaintervallum 95% valószínűséggel tartalmazza a paramétert."* A klasszikus gyakorisági értelmezésben a μ paraméter rögzített, nem véletlen; ami véletlen, az a mintából számolt intervallum. Ha például a kézi mintából a 95%-os intervallum [978.36, 1037.64], és a valódi paraméter $\mu = 1000$, akkor ez az intervallum *tudottan* lefedi a paramétert, tehát annak valószínűsége, hogy lefedi, itt nem 95%, hanem 1. Ha egy másik mintából [1010, 1020] adódott volna, akkor az nem tartalmazná a $\mu = 1000$ értéket, és ugyanígy nem mondhatnánk rá, hogy 95% eséllyel tartalmazza; konkrétan itt 0 lenne a lefedettség valószínűsége, nem 0.95. A 95% az *eljárás* ismételt mintavételi tulajdonsága: hosszú távon a minták 95%-ában fedi a paramétert. Ezért a „valószínűséggel van a paraméter az intervallumban" állításhoz Bayesi megközelítés és prior eloszlás kell, ami a 19.2. alfejezetben kerül majd elő. E különbség a gyakorlatban is fontos: egyetlen döntéshez soha ne úgy hivatkozzunk a CI-re, mintha az adott intervallum belső valószínűséget hordozna.

15.2.8 Önellenőrzés

Ellenőrizd, hogy a kód `ddof=1`-et használ, a kritikus érték `df=n-1` szabadsági fokkal készül, és a szimuláció `mu`-ja azonos a paraméterrel. Számold ki a Monte Carlo hiba nagyságrendjét, és ne nevezz pontosnak olyan lefedettséget, amelyet nem futtattál. A shape `(reps, n)` és a tengely szerinti műveletek egyezzenek; a logikai vektor átlaga a relatív gyakoriság. Ha a lefedettség 95%-tól távol esik, előbb a függetlenséget és az eloszlás ferdeségét vizsgáld, mielőtt a módszert elvetnéd.

15.2.9 Rövid összefoglaló

A standard error a becslő mintavételi szórása; az átlagé ismert σ mellett $\sigma/\sqrt{n}$, ismeretlen szórásnál $s/\sqrt{n}$-nel becsüljük. A t-intervallum $t_{1-\alpha/2,n-1}$ kritikus értéket használ, és független normális minta esetén pontosan a névleges szinten fed. A lefedettséget szimulációval mértük: 1000–2000 ismételt mintán a tapasztalt arány 0.95 körül várható, körülbelül 0,007 (1000 ismétlés) és 0,005 (2000 ismétlés) Monte Carlo-standard hibával. A konfidenciaszint az eljárás ismételt

tulajdonsága, nem a konkrét intervallum valószínűsége. A következő alfejezetben a bootstrap használatát vizsgáljuk, amikor a normalitás vagy a szórás becslésének feltételei sérülhetnek.

15.3 Bootstrap a megfelelő mintavételi egységgel

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 15.2. alfejezet · Célkészség: Bootstrap a megfelelő mintavételi egységgel · Kimenet: működő kód és a labor ellenőrző tesztje.

15.3.1 Helyzet és előzetes becslés

A Northstar `orders.csv` 2400 rendelési sora 300 ügyfélhez tartozik. Az üzemeltetési csapat azt kérdezi: mennyire bizonytalan a `delivery_days` átlaga és mediánja? A rendelési sor kézenfekvő megfigyelési egység, de ugyanannak az ügyfélnek több rendelése van, tehát a sorok nem függetlenek — ezt már a 14.3. alfejezetben láttuk. A kérdés tehát nemcsak az, hogy *mekkora* a pontbecslés, hanem az is, hogy *melyik szinten* érdemes újramintázni.

Előzetes becslés: ha a szállítási napok szórása kb. 3 nap, akkor a függetlennek vélt sorokra számolt standard error $s/\sqrt{2400} \approx 0{,}06$ nap lenne. Ha viszont az ügyfelen belüli szállítási idők hasonlóak, a hatékony mintaméret kisebb, és a valódi standard error akár 0,1–0,2 nap is lehet. A feladatunk ezt bootstrap segítségével ellenőrizhetővé tenni.

15.3.2 Fogalmi magyarázat: percentile és BCa

A bootstrap (újramintázásos becslés) a megfigyelt mintát tekinti a sokaság legjobb helyettesítőjének. Ha az eredeti adatvektor $x = (x_1, \dots, x_n)$, akkor B-szer húzunk n elemet visszatevéssel, és minden b húzásra kiszámítjuk a $\hat{\theta}_b^* = g(x_b^*)$ statisztikát. A $\hat{\theta}^*$ replikációk vektora $(B,)$ alakú, és a cél nem új adat előállítása, hanem a $\hat{\theta}$ becslő mintavételi eloszlásának közelítése. Ez a megközelítés akkor működik jól, ha a minta reprezentatív a kérdéses sokaságra nézve, és az n elem cserélhető, tehát a sorrend nem hordoz releváns csoportszerkezetet.

A legegyszerűbb, *percentilis* (percentile) intervallum a replikációk eloszlásából számított kvantilis:

$$\mathrm{CI}_{\mathrm{perc}} = \left[q_{\alpha/2}(\hat{\theta}^*),\; q_{1-\alpha/2}(\hat{\theta}^*) \right],$$

ahol $\alpha = 0{,}05$ esetén $q_{0{,}025}$ és $q_{0{,}975}$ a 2,5%-os és 97,5%-os kvantilis. Ez az intervallum ott működik jól, ahol $\hat{\theta}^*$ eloszlása megközelítőleg szimmetrikus a $\hat{\theta}$ pontbecslés körül, és a becslő varianciája nagyjából állandó a lehetséges θ-értékek mentén.

A *BCa* (bias-corrected and accelerated, torzítás-korrigált és gyorsított) intervallum két korrekciót vezet be. Az első a torzítás-korrekció:

$$\hat{z}_0 = \Phi^{-1}\left(\frac{1}{B} \sum_{b=1}^{B} I[\hat{\theta}_b^* < \hat{\theta}] \right),$$

ahol Φ^{-1} a standard normális eloszlás kvantilisfüggvénye, $I[\cdot]$ pedig indikátor. Ha a bootstrap-replikációk fele kisebb $\hat{\theta}$-nál, akkor $\hat{z}_0 = 0$. Ha a replikációk mediánja eltolódott, $\hat{z}_0$ nem nulla.

A második, $\hat{a}$ gyorsító (acceleration) tag a becslő érzékenységét méri az egyes megfigyelésekre. A jackknife becslés $\hat{\theta}_{(i)}$ az i-edik megfigyelés elhagyásával számolt statisztika, $\hat{\theta}_{(\cdot)}$ ezek átlaga:

$$\hat{a} = \frac{\sum_{i=1}^{n} \left(\hat{\theta}_{(\cdot)} - \hat{\theta}_{(i)}\right)^3}{6 \left[\sum_{i=1}^{n} \left(\hat{\theta}_{(\cdot)} - \hat{\theta}_{(i)}\right)^2\right]^{3/2}}.$$

A BCa a replikációk eloszlásán a Φ-transzformált kvantiliseket igazítja: az $1-\alpha$ szintű intervallum végpontjait a replikációk α_1 és α_2 kvantilisei adják, ahol

$$\alpha_1 = \Phi\left(\hat{z}_0 + \frac{\hat{z}_0 + z_{\alpha/2}}{1 - \hat{a}(\hat{z}_0 + z_{\alpha/2})}\right), \quad \alpha_2 = \Phi\left(\hat{z}_0 + \frac{\hat{z}_0 + z_{1-\alpha/2}}{1 - \hat{a}(\hat{z}_0 + z_{1-\alpha/2})}\right).$$

Ha $\hat{z}_0 = \hat{a} = 0$, a BCa visszaesik a percentilis intervallumra. A $\hat{a}$ számítása ugyanazt a gondolatot használja, mint a derivált: kis, egyetlen megfigyelésre szorítkozó perturbációra adott érzékenységet vizsgál — visszautalva a 11.3. alfejezet lokális változási rátájára. A BCa jobban kezeli a ferde eloszlásokat, de érzékenyebb a jackknife-becslések simaságára is.

15.3.3 Fogalmi magyarázat: páros és klaszteres újramintázás

Ha a megfigyelési egység pár, a *páros újramintázás* (paired resampling) a párokat, nem az egyedi értékeket keveri. Ha minden i ügyfélhez tartozik egy előtte/utána pár: $d_i = y_{i,1} - y_{i,0}$, akkor a bootstrap az i párindexekből húz visszatevéssel, és a $\hat{\theta}^*$ statisztikát a kiválasztott d_i értékeken számolja. A d_i vektor alakja $(n_p,)$, ahol n_p az értékelhető párok száma. A párok felbontása — tehát az előtte-érték és az utána-érték külön-külön történő visszatevéses húzása — tönkreteszi a párhatást, mert olyan kombinációkat is megenged, amelyek sosem fordultak elő egy ügyfélnél.

A *klaszer-bootstrap* (cluster bootstrap) értelemszerűen akkor szükséges, ha a mintában klaszterek, például ügyfelek vannak. Itt a rendelési sorok nem cserélhetők szabadon, mert a `customer_id` közös hatást visz be. A húzott egység ekkor a klaszter, nem a sor. Ha m ügyfél van, és a j-edik ügyfélnek n_j rendelése, akkor minden bootstrap-replikációban m klasztert húzunk visszatevéssel, majd az összes kiválasztott ügyfél összes sorát bevesszük a mintába:

$$\hat{\theta}^*_b = \frac{\sum_{j \in \mathcal{J}^*_b} \sum_{i=1}^{n_j} y_{ji}}{\sum_{j \in \mathcal{J}^*_b} n_j}.$$

Itt $\mathcal{J}^*_b$ a b-edik húzásban szereplő klaszterindexek multihalmaza, elemszáma m. Figyeljünk arra, hogy a húzott sorok száma reprodukciónként változhat, mert az ügyfelek rendelésszáma különböző. A pontbecslés továbbra is a rendelésszintű átlag; a standard error viszont már tükrözi a klasztereken belüli függést. Ha a `delivery_days` ügyfélen belül hasonló, a sor-szintű bootstrap alábecsüli a bizonytalanságot, a klaszter-bootstrap pedig szélesebb intervallumot ad.

15.3.4 RNG és degeneráltság

A reprodukálhatósághoz a véletlenszám-generátort (rng, random number generator) explicitül meg kell adni. A `np.random.default_rng(seed)` modern `Generator` példányt hoz létre; a `scipy.stats.bootstrap` `random_state` argumentuma elfogad ilyen példányt is. A régi `np.random.RandomState` használata kerülendő, mert elavult a NumPy új projektjeiben. A seed rögzítése mellett ugyanazon a gépen és csomagverzión belül a húzások reprodukálhatók; más NumPy-verzió vagy platform esetén bitazonos eredményt nem ígérhetünk. Statisztikailag a lényeg az, hogy a Monte Carlo ingadozás $O(1/\sqrt{B})$ nagyságrendű legyen, ezért a $B = 1000$–2000 replikáció ésszerű intervallumstabilitást ad.

Degeneráltság (degeneracy) akkor lép fel, ha a bootstrap-replikációk eloszlása túl kevés egyedi értékre koncentrálódik. Klasszikus példa a két elemből álló minta mediánja: az $x = (1, 100)$ elemből húzott $n = 2$ bootstrap-minták mediánjai közt csak az 1, 50,5 és 100 értékek fordulnak elő. A bootstrap eloszlás ilyenkor lépcsős, a percentilis intervallum nem folytonos, és a lefedettség értelmezése sérül. Ugyanezt látjuk extrém kvantiliseknél, például a minimum bootstrap becslésénél. A degeneráltság figyelmeztetés: a bootstrap nem varázsol folytonos valószínűségi eloszlást egy túl kicsi mintából.

15.3.4.1 Degeneráltsági fallback: BCa → percentile → basic → klaszter → nincs CI

A bootstrap nem minden mintán ad értelmezhető intervallumot. A `scipy.stats.bootstrap` BCa-módszere például konstans mintán vagy nulla gyorsulásnál `nan`-t adhat, a klaszter-bootstrap pedig két klaszternél csak néhány lehetséges újramintát lát. A helyes protokoll előre rögzített:

Előfeltétel	Ha sérül	Fallback
nem konstans minta	minden érték azonos	`not_estimable`, nincs CI
elég nagy minta a BCa-hoz	gyorsulás `None`/`nan`	percentile
elég klaszter	< 10 klaszter	klaszter-percentile + figyelmeztetés
percentile értelmezhető	szélső/egypontú eloszlás	basic vagy `not_estimable`
a CI nem hamis bizalom	minden fallback után is bizonytalan	figyelmeztetés a jelentésben

A konstans 0/1 minta például `not_estimable`: a bootstrap [0,0] intervalluma azt sugallná, hogy a paraméter biztosan 0, pedig csak a minta nem tartalmaz változatosságot. Két klaszter esetén a klaszter-bootstrap csak $2^2 = 4$ lehetséges újramintát használ, ezért a kapott intervallum nem tekinthető megbízhatónak; a fallback ilyenkor is figyelmeztet, nem „magabiztos” CI-t ad.

```
import numpy as np

def choose_method(values, clusters=None, requested="bca"):
    values = np.asarray(values, dtype=float)
    if values.size == 0 or np.all(values == values[0]):
        return {"status": "not_estimable", "method": None}
    if clusters is not None and len(clusters) < 10:
        return {"status": "warning_small_clusters", "method": "cluster_percentile"}
    return {"status": "ok", "method": requested}

print(choose_method(np.zeros(20)))          # not_estimable
print(choose_method(np.arange(20.0), clusters=[0, 1]))  # warning_small_clusters
print(choose_method(np.arange(20.0)))       # ok, bca
```

A `lab/bootstrap_fallback.py` `acceleration()`, `choose_method()`, `percentile_ci()` és `report()` függvényei ezt a döntési fát adják; a `tests/test_bootstrap_fallback.py` a konstans minta `not_estimable`, a két klaszter `warning_small_clusters` és a normál minta `ok/bca` állapotát, valamint a konstans mintára dobott `ValueError`-t ellenőrzi. A fallback nem javítja meg a mintát; csak megakadályozza, hogy a bizonytalanságot hamis intervallum takarja el.

15.3.5 Teljes minimális munkapélda

A következő szkript a `lab/` munkakönyvtárból futtatható: `python bootstrap_delivery.py`. Az input a tiszta `orders.csv`; a cél a `delivery_days` átlagának és mediánjának intervalluma. A seed 20261020, a sor-szintű replikációk száma $B = 2000$, a klaszter-bootstrapé $B = 1000$. A SciPy `bootstrap` hívás magas szintű API, de a klaszter-változathoz kézi hurkot írunk, mert a SciPy nem tud `customer_id` szerinti klasztert húzni.

```
import numpy as np
import pandas as pd
from scipy import stats

orders = pd.read_csv("orders.csv", parse_dates=["order_ts"])
y = orders["delivery_days"].to_numpy(dtype=float)
customer = orders["customer_id"].to_numpy()
n = y.size

print(f"rows={n}")
print(f"mean={y.mean():.3f}, median={np.median(y):.1f}, "
      f"sd={y.std(ddof=1):.3f}")

# Sor-szintű bootstrap: itt minden rendelési sort függetlennek tételezünk fel.
rng = np.random.default_rng(20261020)
B = 2000

boot_mean_perc = stats.bootstrap(
    (y,), np.mean, n_resamples=B, method="percentile",
    random_state=rng, confidence_level=0.95,
)
boot_mean_bca = stats.bootstrap(
    (y,), np.mean, n_resamples=B, method="BCa",
    random_state=rng, confidence_level=0.95,
)
boot_med_perc = stats.bootstrap(
    (y,), np.median, n_resamples=B, method="percentile",
    random_state=rng, confidence_level=0.95,
)

for name, res in [
    ("row mean percentile", boot_mean_perc),
    ("row mean BCa", boot_mean_bca),
    ("row median percentile", boot_med_perc),
]:
    lo, hi = res.confidence_interval
    print(f"{name:22s} [{lo:.3f}, {hi:.3f}] "
          f"se={res.standard_error:.3f}")

# Klaszter-bootstrap: a húzott egység az ügyfél, az összes rendelésével együtt.
customers, inverse = np.unique(customer, return_inverse=True)
m = customers.size
cluster_rows = [np.flatnonzero(inverse == j) for j in range(m)]

B_cluster = 1000
cluster_mean = np.empty(B_cluster)
```

```
cluster_median = np.empty(B_cluster)
for b in range(B_cluster):
    chosen = rng.integers(0, m, size=m)
    rows = np.concatenate([cluster_rows[j] for j in chosen])
    sample = y[rows]
    cluster_mean[b] = sample.mean()
    cluster_median[b] = np.median(sample)

for name, arr in [
    ("cluster mean percentile", cluster_mean),
    ("cluster median percentile", cluster_median),
]:
    lo, hi = np.quantile(arr, [0.025, 0.975])
    print(f"{name:22s} [{lo:.3f}, {hi:.3f}] "
          f"se={arr.std(ddof=1):.3f}")
```

A pontos számok a helyi futtatástól függenek, ezért az alábbi megjegyzés szemléltető, nem mért kimenet: ha a sor-szintű átlag-intervallum kerekítve 4,6–4,8 körüli, a klaszter-változat ezt gyakran 0,1–0,2 nappal szélesebbre nyitja. A medián intervalluma a nagy minta miatt már elfogadhatóan sima, de enyhe fokon továbbra is lépcsős lehet.

15.3.6 Eredmény értelmezése

A számok mögötti döntés a húzott egység. A sor-szintű bootstrap minden replikációban n rendelést húz, és úgy kezeli a sorokat, mintha függetlenek lennének. A klaszter-bootstrap ezzel szemben minden replikációban m ügyfelet húz, és az ügyfél teljes rendeléstörténetét viszi be. A két eljárás ugyanazt az eredeti rendelési átlagot használja kiinduló pontbecslésként. A klaszter-bootstrap replikációinak rendelésenként súlyozott átlaga azonban véletlen klaszterméretek mellett nem szükségképpen torzítatlan a megfigyelt átlagra; az értelmezés a célpopulációtól és a klaszterméretek kezelésétől is függ. A standard error viszont akkor különbözik, ha az ügyfelen belüli szállítási idők korrelálnak.

Ellenőrző invariánsok:

- A klaszter-bootstrap minden replikációban pontosan m klasztert húz, de a kiválasztott sorok száma nem feltétlenül n.
- A replikációk átlaga közelíti a megfigyelt pontbecslést; nagy eltérés hiba jele.
- A `cluster_rows` lista minden eleme nem üres, mert minden `customer_id` jelen van az `orders` táblában.
- A BCa akkor tér el érdemben a percentilistől, ha $\hat{z}_0$ vagy $\hat{a}$ nullától távol van. A `delivery_days` enyhén jobbra ferde lehet, ezért a két átlag-intervallum nem feltétlenül egyezik meg.

Klaszteres újramintázásnál külön rögzítsük, hogy ügyfélátlagot vagy rendelésátlagot kívánunk becsülni. Egyenlőtlen klaszterméretek mellett ez más súlyozást és eltérő célparamétert jelenthet.

15.3.7 Feladatok

1. Értelmezés. Érveljen amellett, hogy a `delivery_days` esetén miért az ügyfél a helyes bootstrap-egység! Mondjon egy olyan feltételt, amely mellett a sor-szintű és a klaszter-bootstrap azonos standard errorhoz vezet.

2. Implementálás. A Northstar `orders.csv`-ből válassza ki azokat az ügyfeleket, akiknek legalább két rendelésük van. Párosítsa minden ilyen ügyfél első két rendelését időrendben, számítsd ki a d_i = második `delivery_days` − első `delivery_days` különbséget, majd adjon

páros resamplinggel 95%-os percentilis intervallumot az átlagos d-re. Használj $B = 2000$ replikációt és rögzített seedet.

3. Hibakeresés. Valaki a páros feladatot így próbálta megoldani:

```
first_boot = first[rng.integers(0, first.size, size=first.size)]
second_boot = second[rng.integers(0, second.size, size=second.size)]
repl_b = second_boot.mean() - first_boot.mean()
```

Mi a hiba? Hogyan változtatja ez meg az átlagos különbség bizonytalanságát? Javítsd a kódot.

4. Megváltozott feltétel. A külön kísérleti `assignments.csv` és `outcomes.csv` 300 ügyfélen a `conversion_7d` kimenetet tartalmazza. A beavatkozási cél a `treatment` és `control` értékek átlagának különbsége. Ebben az adatban minden ügyfél pontosan egyszer randomizált, tehát a húzott egység az ügyfél. Készíts csoporton belüli bootstrapet $B = 2000$ replikációval a hatáskülönbség 95%-os percentilis intervallumára. Ellenőrizd, hogy a nevező minden replikációban tartalmazhatja ugyanazt az ügyfelet többször is, de nem használhatja a rendelési sorokat.

15.3.8 Megoldások és indoklás

1. megoldás. Az ügyfél a helyes egység, mert a rendelési sorok ugyanazon ügyfélhez tartozó kimenetei közös ügyfélhatást hordozhatnak; ezt a függetlenség és csoport alfejezetben az ICC fogalmával láttuk. A sor-szintű bootstrap akkor adna hasonló standard errort, ha a `delivery_days` ügyfélen belül független volna, és az ügyfélhatás nem befolyásolná a kimenetet. Ha a klaszteren belüli korreláció nulla és a húzott sorok száma nagyjából azonos marad, a két replikációeloszlás közelíti egymást. Az ellenőrző gondolat: tegyük fel, hogy minden ügyfélhez konstans szállítási idő tartozna; ilyenkor a sor-szintű szórás kicsi, a klaszter-bootstrap viszont az ügyfelek közötti szórást tükrözi.

2. megoldás. A `groupby` művelet először azokat az ügyfeleket tartja meg, akiknek legalább két rendelése van, majd a rendezett `order_ts` alapján az első két rendelést párosítja.

```
import numpy as np
import pandas as pd

orders = pd.read_csv("orders.csv", parse_dates=["order_ts"])
customer_id = "customer_id"
eligible = orders.groupby(customer_id).size()
eligible = eligible[eligible >= 2].index
sub = orders[orders[customer_id].isin(eligible)].sort_values(
    [customer_id, "order_ts"]
)
first = sub.groupby(customer_id)["delivery_days"].first().to_numpy()
second = sub.groupby(customer_id)["delivery_days"].nth(1).to_numpy()
d = second - first

rng = np.random.default_rng(20261020)
B = 2000
repl = np.empty(B)
for b in range(B):
    idx = rng.integers(0, d.size, size=d.size)
    repl[b] = d[idx].mean()

lo, hi = np.quantile(repl, [0.025, 0.975])
print(f"pairs={d.size}, mean diff={d.mean():.3f} "
      f"CI=[{lo:.3f}, {hi:.3f}]")
```

Az ellenőrző invariant: `d.size` pontosan azoknak az ügyfeleknek a száma, akiknek legalább két rendelésük van; a replikációk átlaga közel van `d.mean()`-hez. A szelektált részsokaság nem reprezentálja a csak egy rendeléssel rendelkező ügyfeleket — ezt explicit feltételként kell kimondani a jelentésben.

3. megoldás. A hibás kód a `first` és a `second` tömböket külön-külön húzza újra. Ezzel a párok összerendelése elvész: egy ügyfél első rendeléséhez egy másik ügyfél második rendelése is kerülhet. Ha a páron belüli értékek pozitívan korrelálnak, a párosítás csökkenti a különbség szórását, tehát a hibás verzió szélesebb, konzervatívabb intervallumot ad, de rossz célt mér: olyan kombinációk is szerepelnek, amelyek a valós ügyfélúton nem fordultak elő. A javítás az, hogy a d_i különbségvektort húzzuk újra, mint a 2. megoldásban. Párhuzamos ellenőrzés: a javított kód replikációinak szórása kisebb, mint a hibás kódból számolt szórás, ha a párkorreláció pozitív.

4. megoldás. A külön kísérletben minden ügyfél egyszer szerepel, így a megfelelő húzás a `customer_id` szintű, csoporton belüli bootstrap:

```
import numpy as np
import pandas as pd

assign = pd.read_csv("assignments.csv")
out = pd.read_csv("outcomes.csv")
df = assign.merge(out, on="customer_id", validate="one_to_one")
control = df.loc[df["variant"] == "control", "conversion_7d"].to_numpy()
treatment = df.loc[df["variant"] == "treatment", "conversion_7d"].to_numpy()

rng = np.random.default_rng(20261020)
B = 2000
repl = np.empty(B)
for b in range(B):
    c = control[rng.integers(0, control.size, size=control.size)]
    t = treatment[rng.integers(0, treatment.size, size=treatment.size)]
    repl[b] = t.mean() - c.mean()

lo, hi = np.quantile(repl, [0.025, 0.975])
print(f"ITT estimate={treatment.mean()-control.mean():.4f} "
      f"CI=[{lo:.4f}, {hi:.4f}]")
```

Az ITT-nevező az összes randomizált ügyfél; a replikációk csak ügyfeleket húznak. A replikációk átlaga közelíti a becsült hatás nagyságát; ha ettől a `repl` átlaga nagyon eltér, a húzási logikában hiba van. A `merge validate="one_to_one"` ellenőrzi, hogy a táblák valóban ügyfélszintűek.

15.3.9 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés az, hogy a bootstrap korrigálja a rossz adatfelvételi folyamatot. Ez nem igaz. Ha egy elégedettségi mérést csak panaszos ügyfelek töltenek ki, akkor a 95%-os bootstrap-intervallum továbbra is a panaszos rétegre vonatkozik, nem az összes ügyfélre. A B növelése csak a Monte Carlo hibát csökkenti, a szelekciós torzítást nem.

A bootstrap a nagyon kicsi mintát sem varázsolja megbízhatóvá. Az $x = (1, 100)$ kételemes minta mediánjának bootstrap-eloszlása mindössze három lehetséges értékre támaszkodik; bármilyen nagy B mellett is degenerált marad. A `delivery_days` laborban is ellenőrizni kell, hogy a replikációk eloszlása elég sok egyedi értéket tartalmaz-e; a kis minta vagy extrém kvantilis degeneráltsága megbízhatatlanná teszi a percentilis intervallumot.

15.3.10 Önellenőrzés

Kérdezze meg magától: mi a húzott egység a saját feladatomban, sor vagy ügyfél? A `scipy.stats.bootstrap` magas szintű hívása a sor- vagy elemszintű függetlenséget feltételezi; klaszter esetén kézi hurkot kell írni. A BCa-t használva ellenőrizd, hogy a jackknife-becslések simán változnak-e; a medián ilyen szempontból nem ideális, mert a jackknife érzékenysége ugrálhat. A seed rögzítése a statisztikai reprodukálhatóság minimális feltétele; a 3.3. alfejezet tanulsága szerint a natív SciPy-környezet verzióját is rögzíteni kell, ha valaki bitazonosan szeretné újrafuttatni a kódot. A konfidenciaszint továbbra is az eljárás ismételt mintavételi lefedettségére vonatkozik, nem a konkrét intervallum valószínűségére, ahogyan ezt a 15.2. alfejezetben tárgyaltuk.

15.3.11 Rövid összefoglaló

A bootstrap a megfigyelt mintából visszatevéses húzással közelíti a becslő mintavételi eloszlását. A percentilis intervallum a replikációk kvantiliseit veszi, a BCa emellett torzítás- és ferdeségkorrekciót alkalmaz. Páros adatnál a párokat, ismételt ügyféladatnál az ügyfelet kell újramintázni; a sor-szintű húzás a klaszteren belüli függés miatt alábecsüli a standard errort. A seed rögzítése szükséges, a degenerált replikációeloszlás pedig figyelmeztetés a kis mintára vagy extrém statisztikára. A következő alfejezetben a predikciós, paraméter- és Monte Carlo intervallumok közötti fogalmi határokat tisztázzuk.

15.4 Predikciós, paraméter- és Monte Carlo intervallum

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 15.3. alfejezet · Célkészség: Predikciós, paraméter- és Monte Carlo intervallum · Kimenet: működő kód és a labor ellenőrző tesztje.

15.4.1 Cél és konkrét helyzet

Egy kísérleti tanulóprojektben a 7 napos vásárlói bevételt vizsgáljuk. A megfigyelési egység egy ügyfél, a kimenet `revenue_7d_minor`, azaz a 7 nap alatti bevétel EUR-centben. A teljes Northstar lab helyett most egy tíz elemű, teljesen előállított szintetikus mintát használunk, hogy minden lépés kézzel is ellenőrizhető legyen:

```
x = [10, 12, 9, 14, 11, 13, 8, 15, 10, 12]  # EUR-cent
```

Modellfeltevésünk tanulási célból: a megfigyelések független, azonos eloszlású (iid) minták egy $N(\mu, \sigma^2)$ normális eloszlásból. Ez statisztikai tanulómodell, nem üzleti tény.

Három különböző bizonytalanságot fogunk riportálni ugyanahhoz a modellhez:

- a μ várható bevétel paraméter bizonytalansága,
- egy **új** ügyfél bevételének predikciós bizonytalansága,
- a Monte Carlo-szimulációval számolt mennyiség **numerikus** pontossága.

A cél ellenőrzési feltétele: a paraméter-intervallum és a predikciós intervallum a levezetett t-képletet kövesse, a predikciós intervallum legyen szélesebb a paraméter-intervallumnál, és a Monte Carlo-kvantilisek a megadott toleranción belül közelítsék az analitikus predikciós intervallumot.

Induló feladat: a minta alapján számold ki fejben vagy papíron az átlagot és a korrigált tapasztalati szórást, majd becsüld meg, hogy a 95%-os predikciós intervallum nagyságrendileg mennyivel lesz szélesebb, mint a 95%-os paraméter-intervallum. A részletes levezetés után ugyanezt ellenőrizni fogjuk.

15.4.2 Három bizonytalansági forrás, matematikai híd

A variancia felbontása a kulcs. Ha $X_1, \dots, X_n$ független, σ^2 varianciájú megfigyelés, akkor a mintaátlag varianciája

$$\operatorname{Var}(\bar{X}_n) = \frac{\sigma^2}{n},$$

míg egy új megfigyelésé

$$\operatorname{Var}(X_{n+1}) = \sigma^2.$$

Az új megfigyelés tehát az egyedi zajt is hordozza. Ha a mintaátlagot használjuk középpontnak, a predikciós hiba varianciája a kettő összege:

$$\operatorname{Var}(X_{n+1} - \bar{X}_n) = \sigma^2 + \frac{\sigma^2}{n} = \sigma^2 \left(1 + \frac{1}{n}\right).$$

Ebből már látható, miért lesz a predikciós intervallum mindig szélesebb, mint a paraméter bizonytalanságát leíró intervallum: ugyanahhoz a σ-hoz egy $1 + 1/n$ szorzó társul.

15.4.2.1 Paraméter-bizonytalanság (parameter uncertainty)

A μ paraméter pontbecslése $\bar{x}$. Ha σ ismeretlen, a standardizált becslő Student-féle t-eloszlást követ $n-1$ szabadsági fokkal:

$$\frac{\bar{X}_n - \mu}{s/\sqrt{n}} \sim t_{n-1}.$$

Itt s a korrigált tapasztalati szórás,

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2},$$

amely $s > 0$, mértékegysége itt EUR-cent. Az $1-\alpha$ szintű konfidenciaintervallum:

$$\bar{x} \pm t_{1-\alpha/2,\,n-1}\,\frac{s}{\sqrt{n}}.$$

A t-kritikus érték `stats.t.ppf(1 - alpha/2, df)` módon kapható. Az intervallum a **becslő mintavételi ingadozását** írja le. Frekventista értelmezése: ha az eljárást sok független mintán

megismételjük, a lefedettség hosszú távon $1-\alpha$ lesz. Ez nem azt jelenti, hogy az aktuális μ 95% valószínűséggel esik az aktuális intervallumba.

15.4.2.2 Predikciós intervallum (prediction interval)

Egy új megfigyelésre vonatkozó intervallum más célt szolgál: egy jövőbeli X_{n+1} értéket szeretnénk előre megragadni. Ismeretlen σ mellett

$$T = \frac{X_{n+1} - \bar{X}_n}{s\sqrt{1 + 1/n}} \sim t_{n-1},$$

ezért az $1-\alpha$ szintű predikciós intervallum:

$$\bar{x} \pm t_{1-\alpha/2,\,n-1}\, s\sqrt{1 + \frac{1}{n}}.$$

A képlet alakja: `loc = xbar`, `scale = s * np.sqrt(1 + 1/n)`; a `scipy.stats.t` ilyen paraméterezést használ. A predikciós intervallum tehát kettős forrásból táplálkozik: a paraméterbecslés bizonytalanságából ($s/\sqrt{n}$) és az új megfigyelés saját zajából (s). Kis mintán a $1 + 1/n$ korrekció érdemi; $n = 10$ mellett $\sqrt{1.1} \approx 1.0488$.

15.4.2.3 Szimulációs hiba (simulation error)

Ha a predikciós eloszlás kvantiliseit Monte Carlo-mintavétellel közelítjük, akkor a véges B szimulációszám miatt a becsült kvantilisnek külön numerikus hibája van. Legyen $\hat{q}_p$ az empirikus p-kvantilis. Közelítő standard hibája:

$$\mathrm{SE}(\hat{q}_p) \approx \frac{\sqrt{p(1-p)/B}}{f(q_p)},$$

ahol $f(q_p)$ a szimulált eloszlás sűrűsége a valódi q_p kvantilisnél, $p \in (0,1)$, B pozitív egész. A képlet azt mutatja, hogy a Monte Carlo hiba $1/\sqrt{B}$ sebességgel csökken: B négyszerezése körülbelül felezi a numerikus standard hibát. Ez a hiba nem a modell statisztikai bizonytalansága, hanem a számítás felbontásából adódik; tetszőlegesen csökkenthető nagyobb B-vel, míg a predikciós intervallum szélessége a mintából adódó valódi tanulási korlát.

15.4.2.4 Érzékenység (sensitivity)

Az érzékenységvizsgálat azt kérdezi: mennyire változik a konklúzió, ha a modellfeltevést vagy egy-egy adatpontot módosítjuk? A normális predikciós intervallum különösen érzékeny a kiugrókra, mert mind $\bar{x}$, mind s a teljes mintából számítódik. Ha egy extrém bevételű ügyfél kerül a mintába, az átlag és a szórás is eltolódik, és az intervallum drámaian kiszélesedhet. Ilyenkor vagy robusztusabb összegzést, vagy transzformált modellt (például logaritmikus skálát) kell választani; a sensitivity célja nem az, hogy egyetlen „helyes” intervallumot adjon, hanem hogy láthatóvá tegye, melyik feltevésre támaszkodik az eredmény.

15.4.2.5 n és B külön hatása: CI, PI és Monte Carlo hiba

A három intervallum különböző forrásból származik. Legyen $\sigma = 1$, 95%-os szint, $z = 1.96$:

n	paraméter-CI félhossz $z\sigma/\sqrt{n}$	predikciós PI félhossz $z\sigma\sqrt{1+1/n}$
10	0.620	2.056
100	0.196	1.970
1000	0.062	1.961

A CI félhossza n-nel nullához tart; a PI félhossza 1.96σ-hoz tart, mert az új megfigyelés saját zaja nem tűnik el több adattól. A Monte Carlo hiba külön tengelyen mozog: $B = 100$ minta esetén a szimulált átlag standard hibája $1/\sqrt{100} = 0.1$, $B = 10000$-nél 0.01. A B növelése a szimulációs hibát csökkenti, de a CI és PI szélességét nem változtatja meg, mert azokat az adat mennyisége és a modell szórása határozza meg.

Kérdés	Válasz
Mit csökkent az n növelése?	CI-t és a PI-t is, de a PI-t csak a $1/n$ tag miatt, a 1.96σ határig
Mit csökkent a B növelése?	csak a Monte Carlo-becslés hibáját
Mit nem csökkent sem n, sem B?	az új megfigyelés saját zaját

```
import math

def half_widths(n, sigma=1.0, z=1.96):
    return {"ci": z * sigma / math.sqrt(n),
            "pi": z * sigma * math.sqrt(1 + 1/n)}

for n in (10, 100, 1000):
    print(n, half_widths(n))

print("MC B=100", 1 / math.sqrt(100))
print("MC B=10000", 1 / math.sqrt(10000))
```

A `lab/interval_types.py` `half_widths()`, `mc_error()` és `table()` függvényei ezt a táblát adják; a `tests/test_interval_types.py` a CI monoton csökkenését, a PI 1.96-hoz tartását, a PI > CI relációt és a MC hiba $1/\sqrt{B}$ skálázását ellenőrzi.

15.4.3 Teljes minimális munkapélda

A kód önálló, csak NumPy-t és SciPy-t használ. Futtatás a projektgyökérből:

```
python s060_intervals.py
```

```
import numpy as np
from scipy import stats

# Rögzített, teljesen szintetikus tanuló-minta: revenue_7d_minor, EUR-cent.
x = np.array([10, 12, 9, 14, 11, 13, 8, 15, 10, 12], dtype=float)
n = x.size                          # int, mintanagyság
df = n - 1                          # int, szabadsági fok
alpha = 0.05

xbar = x.mean()                     # float, EUR-cent
```

```
s = x.std(ddof=1)                # float, korrigált minta-szórás
t_crit = stats.t.ppf(1 - alpha / 2, df)  # float, kritikus érték

# 1) Paraméter-intervallum mu-ra
se_mean = s / np.sqrt(n)
ci_lo = xbar - t_crit * se_mean
ci_hi = xbar + t_crit * se_mean

# 2) Predikciós intervallum egy új megfigyelésre
pi_scale = s * np.sqrt(1 + 1 / n)
pi_lo = xbar - t_crit * pi_scale
pi_hi = xbar + t_crit * pi_scale

# 3) Monte Carlo-szimuláció a t-predikciós eloszlásból,
#    hogy a numerikus kvantilis-hibát is lássuk.
rng = np.random.default_rng(20261020)  # reprodukálható seed
B = 10_000                             # szimulációszám

mc_sample = stats.t.rvs(
    df=df,
    loc=xbar,
    scale=pi_scale,
    size=B,
    random_state=rng,
)
mc_lo = np.quantile(mc_sample, alpha / 2)
mc_hi = np.quantile(mc_sample, 1 - alpha / 2)

# A felső 0.975 kvantilis Monte Carlo standard hibája.
# Sűrűség az y-skálán: f_T((y-loc)/scale) / scale.
f_upper = stats.t.pdf(t_crit, df) / pi_scale
mc_se_upper = np.sqrt((1 - alpha / 2) * (alpha / 2) / B) / f_upper

print(f"n={n}, df={df}, t={t_crit:.6f}")
print(f"xbar={xbar:.4f}, s={s:.4f}")
print(f"95% CI:   [{ci_lo:.3f}, {ci_hi:.3f}]")
print(f"95% PI:   [{pi_lo:.3f}, {pi_hi:.3f}]")
print(f"MC 95% PI: [{mc_lo:.3f}, {mc_hi:.3f}]")
print(f"MC SE felső kvantilis: {mc_se_upper:.4f}")
```

A kimenetben a levezetett értékek a rögzített lista kézi számításából adódnak; az alábbi táblázat ezt mutatja. A tényleges helyi futtatáskor a Monte Carlo-kvantilisek a seed miatt determinisztikusak, de az utolsó tizedesjegyekben a SciPy-verzió numerikus részletei miatt lehet minimális eltérés.

Mennyiség	Érték (EUR-cent)	Mit fejez ki
95% CI μ-re	$[9.81, 12.99]$	a mintaátlag mintavételi ingadozása
95% PI új ügyfélre	$[6.13, 16.67]$	paraméterbizonytalanság + egyedi zaj
MC kvantilis SE	≈ 0.1	a szimulációs numerikus hiba nagyságrendje

A CI szélessége körülbelül 3.18, a PI szélessége körülbelül 10.54 EUR-cent. A predikciós intervallum tényleg szélesebb, és a különbség a $\sqrt{1 + 1/n}$ tényezőből, illetve az egyedi zajból következik.

15.4.4 Eredmény értelmezése

A mintaátlag $\bar{x} = 11.4$ EUR-cent. A becsült szórás $s \approx 2.221$, a standard hiba $s/\sqrt{10} \approx 0.702$. A 95%-os paraméter-intervallum félszélessége $2.262 \cdot 0.702 \approx 1.589$, tehát a CI $[9.81, 12.99]$. A predikciós intervallum félszélessége $2.262 \cdot 2.221 \cdot \sqrt{1.1} \approx 5.270$, tehát a PI $[6.13, 16.67]$.

A Monte Carlo-szimuláció ugyanabból a t-eloszlásból vesz mintát, ezért a $B = 10\,000$ replikációból becsült kvantiliseknek a fenti analitikus PI-t kell közelíteniük; a kvantilis Monte Carlo standard hibája a sűrűségbecslés alapján nagyságrendileg 0.1 EUR-cent. Ez azt jelenti, hogy a szimulációs numerikus hiba a predikciós intervallum szélességéhez képest elhanyagolható, de külön kell jelenteni, mert a célja és a skálája más, mint a statisztikai intervallumé.

Ellenőrzési invariánsok:

- `ci_lo <= ci_hi` és `pi_lo <= pi_hi`;
- `pi_hi - pi_lo > ci_hi - ci_lo`, mert $\sqrt{1 + 1/n} > 1$;
- `mc_lo` és `mc_hi` az analitikus értékektől a Monte Carlo hibán belül térhet el;
- a `mc_sample` alakja `(B,)` és dtype-ja `float64`.

A paraméterintervallum a sokasági mennyiség bizonytalanságát, a predikciós intervallum egy új kimenet szóródását is tükrözi. A Monte Carlo-konfidenciaintervallum pedig egy szimulációs becslés numerikus mintavételi hibájára vonatkozik; ezeket nem szabad felcserélni.

15.4.5 Feladatok

1. **Értelmezés.** Magyarázd el saját szavakkal, miért tartalmazza a predikciós intervallum a $\sigma^2 + \sigma^2/n$ varianciát, és miért nem elegendő az új ügyfél becsléséhez a paraméter konfidenciaintervallum. A 95%-os szint mit jelent a két intervallumnál?

2. **Megvalósítás.** Módosítsd a fenti kódot 90%-os szintre. Számold ki a 90%-os CI-t és PI-t, és hasonlítsd össze a 95%-os intervallumok szélességével. A `stats.t.ppf` hívásban mely argumentum változik?

3. **Hibakeresés.** Egy kolléga a következő kódot írta a predikciós intervallumhoz:

```
z = stats.norm.ppf(0.975)
pi_lo = x.mean() - z * x.std(ddof=1)
pi_hi = x.mean() + z * x.std(ddof=1)
```

Nevezd meg a két statisztikai hibát, és becsüld meg, mennyivel lesz keskenyebb vagy szélesebb ez az intervallum a helyes 95%-os PI-nél a fenti mintán.

4. **Megváltozott feltétel/transzfer.** A mintához hozzáadunk egy új ügyfelet, akinek `revenue_7d_minor = 300`. Futtasd újra a CI-t és a PI-t. Hogyan változik a két intervallum szélessége, és miért jelzi ez a normalitási feltevés érzékenységét? Javasolj egy alternatív modellt vagy transzformációt, amely robusztusabb lehet erre a kiugróra.

15.4.6 Megoldások és indoklás

1. A mintaátlag szórása $\sigma/\sqrt{n}$, mert n független megfigyelés átlagolása csökkenti az ingadozást. Egy új megfigyelés viszont egyetlen húzás, amelynek szórása σ, ezért az új érték és a becsült átlag eltérésének varianciája $\sigma^2 + \sigma^2/n$. A paraméter-CI azt az intervallumot adja, ahová a becslő μ körül esik; az új ügyfél ettől még szinte bizonyosan kívül lesz, mert az egyedi zaj hiányzik belőle. A 95% frekventista értelemben az eljárás hosszú távú lefedettségére vonatkozik; a CI a paraméterre, a PI az új megfigyelésre ad eljárási garanciát.

2. `alpha = 0.10`, és `t_crit = stats.t.ppf(0.95, df=9)` értéke 1.833. A mintaátlag és a szórás változatlan. A 90%-os CI félszélessége $1.833 \cdot 0.702 \approx 1.287$, tehát a CI $[10.11, 12.69]$. A 90%-os PI félszélessége $1.833 \cdot 2.3295 \approx 4.271$, tehát a PI $[7.13, 15.67]$. A CI szélessége 2.57, a PI szélessége 8.54 EUR-cent, mindkettő keskenyebb, mint a 95%-os változaté. Ez a példa is mutatja, hogy a szint megadása nélkül az intervallum szélessége nem értelmezhető.

3. Két hiba van. Egyrészt a kolléga a standard normális kritikus értéket, $z_{0.975} = 1.96$ használja a $t_{9,0.975} = 2.262$ helyett, ami kis mintán alulbecsüli a bizonytalanságot. Másrészt a predikciós skálából hiányzik a $\sqrt{1 + 1/n}$ korrekció. A hibás félszélesség $1.96 \cdot 2.221 \approx 4.353$, a helyes 5.270, tehát a hibás intervallum körülbelül 17%-kal keskenyebb. Ha emellett n nagyobb lenne, a z és t közötti eltérés csökkenne, de a $\sqrt{1 + 1/n}$ akkor is megmaradna, mert az egyedi zaj minden mintanagyságnál jelen van.

4. Az új minta $n = 11$ elemű, az átlag $\bar{x} = (114 + 300)/11 \approx 37.64$, a korrigált szórás $s \approx 87$. A szélsőséges kiugró miatt mind a CI, mind a PI félszélessége drámaian megnő, és a normális predikciós intervallum alsó határa akár negatív is lehet, ami a bevételi adatoknál nem értelmes érték. Ez érzékenységi jelzés: az eredmény nem robusztus a normalitási feltevés megsértésére. Alternatíva például a `log(revenue_7d_minor + 1)` transzformáció, majd a log-skálán normális modell és a végén visszatranszformálás, vagy egy ferde eloszlást megengedő bootstrap predikciós intervallum; mindkettő külön feltételek mellett érvényes.

15.4.7 Gyakori tévedés ellenpéldával

Az intervallum szélessége önmagában nem jelent jóságot, ha nem mondjuk meg, mire vonatkozik és milyen szintű. A fenti mintán a 95%-os CI szélessége 3.18, a 95%-os PI szélessége 10.54. Egy felületes összehasonlítás azt mondhatná, hogy a CI „jobb”, mert keskenyebb; valójában egyszerűen más kérdésre válaszol. Ellenpéldaként vegyünk egy 50%-os CI-t ugyanerre a mintára: a félszélesség $0.703 \cdot 0.702 \approx 0.493$, a teljes szélesség 0.99, tehát keskenyebb, mint a 95%-os CI. Ez mégsem erősebb állítás, hanem alacsonyabb lefedettségi szint. Ugyanígy egy rosszul megválasztott bootstrap intervallum is lehet szűkebb, ha figyelmen kívül hagyja az ügyfélszintű függést (15.3. alfejezet). A szélesség értékelése előtt tehát mindig rögzíteni kell: mi a cél, mi a megfigyelési egység, milyen a szint és milyen modellfeltevés mellett készült az intervallum.

Visszautalásként a 14.4. alfejezet standard hibája itt a t-eloszlással kombinálódik, a 15.3. alfejezet bootstrap-logikája pedig emlékeztet, hogy a megfelelő mintavételi egység nélkül a szimulációs intervallum is félrevezető lehet. A 3.4. alfejezet reprodukálhatósági elve itt is érvényes: a Monte Carlo seed rögzítése nélkül a kvantilis-hiba sem reprodukálható.

Önellenőrző kérdések: a kódban a `pi_scale` mindig nagyobb-e, mint `se_mean`? A Monte Carlo szimulációnak melyik eloszlásból kell mintát venni? Ha B-t négyszerezed, milyen arányban változik a Monte Carlo standard hiba?

15.4.8 Rövid összefoglaló

A paraméter-intervallum a μ becslésének mintavételi bizonytalanságát adja meg, a predikciós intervallum ezen felül egy új megfigyelés egyedi zaját is tartalmazza, ezért utóbbi mindig szélesebb. A Monte Carlo szimuláció kvantiliseinek külön numerikus standard hibája van, amely $1/\sqrt{B}$ szerint csökken, és nem keverhető a statisztikai bizonytalansággal. A normalitási feltevésre érzékenységvizsgálattal kell rákérdezni, például kiugró hozzáadásával vagy log-transzformált modellel. A következő alkalmazásban ezeket az intervallumfajtákat hipotézisvizsgálati döntésekkel kötjük össze: egy hatásméret, egy predikciós tartomány és egy szimulációs hibahatár együttes értelmezése ad korrekt statisztikai állítást.

16 Hipotézisvizsgálat, hatásméret és kísérleti erő

16.1 Nullmodell és p-érték

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 15.4. alfejezet · Célkészség: Nullmodell és p-érték · Kimenet: működő kód és a labor ellenőrző tesztje.

16.1.1 Helyzet, bemenet és előzetes becslés

A 15.4. alfejezetben megismert intervallumok azt mutatták, hogy egy becslés mekkora tartományban mozoghat. Most azt vizsgáljuk, hogy az adatok mennyire egyeztethetők össze azzal a nullmodellel, hogy a kezelésnek nincs hatása. A nullhipotézist a nem szignifikáns eredmény alapján nem „fogadjuk el”. A Northstar kísérleti projektben a bemenet a `lab/data/assignments.csv` és a `lab/data/outcomes.csv`. A megfigyelési egység egy randomizált ügyfél; a primer célváltozó a `conversion_7d`, értéke 0 vagy 1. A `variant` oszlop `control` vagy `treatment`.

Az estimand a hétnapos konverziós arányok különbsége:

$$\Delta = P(\text{conversion}_7 d = 1 \mid \text{treatment}) - P(\text{conversion}_7 d = 1 \mid \text{control}).$$

Ennek pontbecslése

$$t_{\text{obs}} = \hat{p}_t - \hat{p}_c = \frac{k_t}{n_t} - \frac{k_c}{n_c},$$

ahol k_t a treatment csoport konvertálóinak száma, n_t a treatment csoport mérete; k_c és n_c ugyanez a kontrollra. A különbség mértékegysége aránypont, például 0,10 azt jelenti, hogy a kezelt csoportban 10 százalékponttal nagyobb a becsült konverziós arány. A tanulókérdés: ez a különbség elég nagy-e ahhoz, hogy a „nincs hatás” modellt elvessük $\alpha = 0{,}05$ mellett? Az előzetes várakozás szerint a generáló hatás 0,10, de a p-értéket a tényleges minta és a permutációs nullmodell adja.

16.1.2 Nullhipotézis, alternatíva, statisztika és farok

A **nullhipotézis** (*null hypothesis*, H_0) az a kimondottan egyszerű állapot, amelyből a meglepő adatot mérjük. Itt:

$$H_0 : \ \Delta = 0,$$

tehát a két csoport konverziós valószínűsége azonos. A **kétoldali alternatíva** (*two-sided alternative*) szerint

$$H_1 : \Delta \neq 0.$$

A **tesztstatisztika** (*test statistic*) a minta egy előre rögzített függvénye. A mi $T = \hat{p}_t - \hat{p}_c$ statisztikánk skalár, mértékegysége aránypont. A p-érték megbízhatósága megköveteli, hogy a statisztikát az adat megnézése előtt rögzítsük; ha utólag választunk statisztikát, a küszöb valódi hibavédelme elromlik.

A **farok** (*tail*) azt rögzíti, hogy mely irányban számít szélsőségesnek egy nullmodell alatti érték. Felső egyoldali tesztnél a nagy pozitív T, alsó egyoldalinál a nagy negatív T szélsőséges. Kétoldali tesztnél az abszolút értékben nagy eltérés szélsőséges.

A feladat megfigyelési egysége fontos: a permutációt ügyfélszinten végezzük. A `conversion_7d` csak egy érték ügyfelenként; ha rendelési tételeket kevernénk, az összefüggő megfigyeléseket kezelnénk függetlenként, és a p-érték megtévesztően kicsi lehet.

16.1.3 Kézi kontroll: hat lehetséges felosztás

Legyen egy tanító méretű, szintetikus kísérlet: a kontroll kimenetei $y_c = (0, 0)$, a treatment kimenetei $y_t = (1, 1)$. Ekkor $n_t = n_c = 2$, és a megfigyelt statisztika

$$t_{\text{obs}} = \frac{1+1}{2} - \frac{0+0}{2} = 1.$$

A nullmodell szerint a négy kimenet a hozzárendeléstől független, tehát az összes lehetséges „két elem a treatment csoportba, két elem a kontrollba” felosztás egyformán elképzelhető. A négy megfigyelt érték rögzített halmaza $\{0, 0, 1, 1\}$. A hat lehetőség:

treatment értékei	$\hat{p}_t - \hat{p}_c$
$\{0, 0\}$	-1
$\{0, 1\}$ négy esetben	0
$\{1, 1\}$	1

A nullmodell alatt a statisztika eloszlása:

$$P_{H_0}(T = -1) = \frac{1}{6}, \quad P_{H_0}(T = 0) = \frac{4}{6}, \quad P_{H_0}(T = 1) = \frac{1}{6}.$$

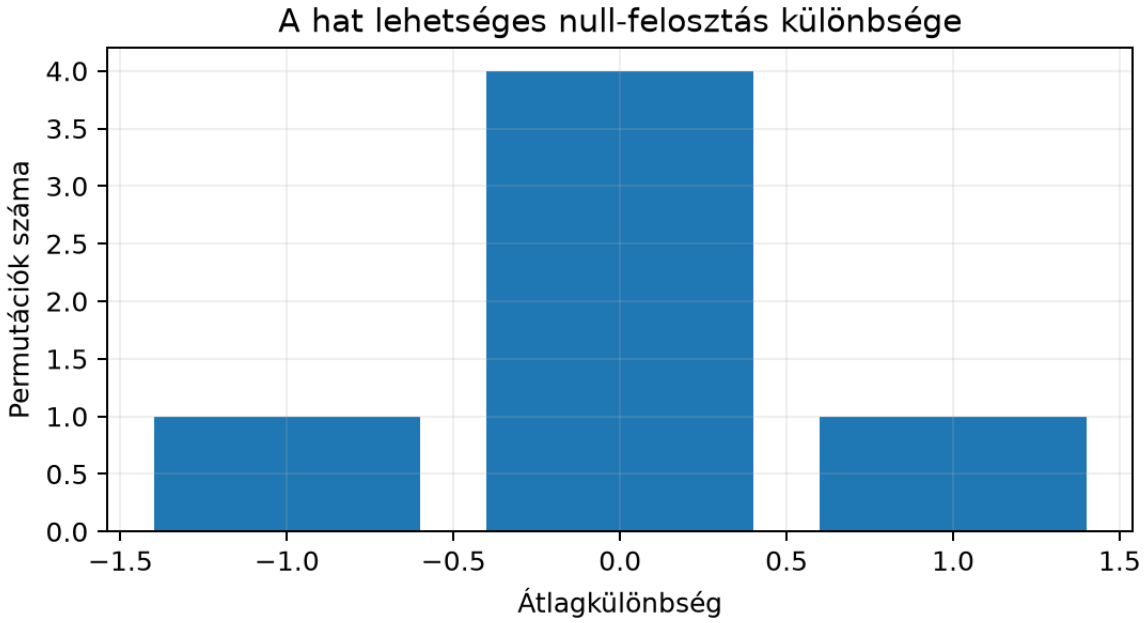
Az egyoldali felső p-érték a megfigyelt 1-hez:

$$p_{\text{one}} = P_{H_0}(T \geq 1) = \frac{1}{6} \approx 0{,}1667.$$

A kétoldali p-érték az abszolút értékben legalább ekkora eltérés valószínűsége:

$$p_{\text{two}} = P_{H_0}(|T| \geq 1) = \frac{2}{6} \approx 0{,}3333.$$

Ez a kézi példa azért tanulságos, mert a megfigyelt hatás nagynak látszik, de a minta olyan kicsi, hogy a nullmodell szerint is könnyen létrejön.



16.1. ábra: A hat lehetséges null-felosztás különbsége - A tanulópélda összes felosztásából számított, nem mért adat.

16.1.4 Teljes minimális példa: egzakt és Monte Carlo permutációs teszt

A kis példa egzakt permutációs eloszlását a Python `itertools.combinations` függvényével állítjuk elő. A kód a projekt gyökeréből futtatható.

```
from itertools import combinations
import numpy as np

# Szintetikus tanulóadat: n_t = 2, n_c = 2.
y = np.array([0, 0, 1, 1])
n_t = 2

observed_diff = y[:n_t].mean() - y[n_t:].mean()

null_diffs = []
for idx in combinations(range(y.size), n_t):
    treatment = np.zeros(y.size, dtype=bool)
    treatment[list(idx)] = True
    control = ~treatment
    null_diffs.append(y[treatment].mean() - y[control].mean())

null_diffs = np.array(null_diffs)
p_one_sided = np.mean(null_diffs >= observed_diff)
p_two_sided = np.mean(np.abs(null_diffs) >= abs(observed_diff))

print(observed_diff, null_diffs, p_one_sided, p_two_sided)
```

A kód elméletileg várt kimenete a fenti kézi számításnak felel meg:

```
1.0 [-1.  0.  0.  0.  0.  1.] 0.16666666666666666 0.3333333333333333
```

A laboradatra a permutációs tesztet Monte Carlo közelítéssel futtatjuk. A kód a `lab/data/assignments.csv` és a `lab/data/outcomes.csv` fájlokat olvassa, ügyfélszinten kapcsolja, és 10 000 permutációt használ. A seed rögzíti a pszeudovéletlen sorrendet.

16.1.4.1 A permutációs teszt érvényességi feltételei

A randomizációs teszt nullmodellje nem „a címkék bármilyen keverése", hanem **az eredeti hozzárendelési mechanizmus** újrajátszása. A permutációnak meg kell őriznie a design minden olyan elemét, amely a kezelés kiosztását befolyásolta:

Tervezési elem	Mit kell megőrizni?	Helyes permutáció	A megsértés következménye
Egyszerű egyéni randomizáció	a csoportméretek n_t, n_c	címkék keverése rögzített csoportméretekkel	ha a méretek is változnak, nem a vizsgált nullmodellt teszteljük
Blokkolt randomizáció	blokkon belüli kezelési arány	blokkon belüli címkekeverés	a globális keverés a blokkok közötti alapkülönbséget is „nullává" teszi, és tévesen kicsi p-értéket ad
Párosítás	a pár azonosítása	páron belüli előjelcsere vagy címkekeverés	a párok közötti keverés megsérti a függőségi szerkezetet
Klaszterezés	a klaszter azonosítója	klaszterszintű keverés	az egyéni keverés alulbecsüli a varianciát, a p-érték túl kicsi

A Northstar `assignments.csv` egyszerű egyéni randomizációt feltételez: nincs blokk-, pár- vagy klaszterazonosító oszlopa. Az alábbi kód ezért az egyéni címkekeverést használja, és a csoportméreteket a maszkokkal rögzíti. Ha a kísérlet valójában blokkolt volt, a globális permutáció hibás null-eloszlást ad; ezt mutatja a következő, kézzel ellenőrizhető példa.

Blokkolt példa. Két blokk, mindegyikben négy egység; a kimenetek blokkon belül $0, 0, 10, 10$, a kezelés mindkét blokkban a két 10-es. A megfigyelt különbség 10. A naiv, mind a nyolc egységen végzett teljes permutáció $\binom{8}{4} = 70$ felosztást enged, a helyes, blokkon belüli permutáció $\binom{4}{2} \cdot \binom{4}{2} = 36$-ot. A kétoldali p-érték:

```
from itertools import combinations
import numpy as np

outcomes = np.array([0, 0, 10, 10, 0, 0, 10, 10], dtype=float)
observed = (2, 3, 6, 7)  # a négy 10-es

def diff(treated):
    mask = np.zeros(8, dtype=bool)
    mask[list(treated)] = True
    return outcomes[mask].mean() - outcomes[~mask].mean()

naive = [diff(c) for c in combinations(range(8), 4)]
blocked = [diff(a + b)
```

```
            for a in combinations(range(4), 2)
            for b in combinations(range(4, 8), 2)]
t_obs = diff(observed)
print(np.mean(np.abs(naive) >= abs(t_obs)))    # 0.028571428571428572
print(np.mean(np.abs(blocked) >= abs(t_obs)))  # 0.05555555555555555
```

A naiv p-érték 0,0286, a blokkolt 0,0556: a naiv teszt $\alpha = 0{,}05$ mellett elutasítana, a helyes design-alapú teszt nem. A különbség oka, hogy a naiv permutáció a blokkok közötti alapkülönbséget is keveri, és olyan null-eloszlást épít, amely nem felel meg annak, ahogy a kezelést valójában kiosztották.

```
import numpy as np
import pandas as pd

# Futtatás a projekt gyökeréből: python s061_permutation_test.py
# Egyszerű egyéni randomizációt feltételez; blokkolt, páros vagy klaszterezett
# design esetén a permutációt a design szerint kell rétegezni.
assign = pd.read_csv("lab/data/assignments.csv")
outcomes = pd.read_csv("lab/data/outcomes.csv")

df = assign.merge(outcomes, on="customer_id", how="inner", validate="one_to_one")
y = df["conversion_7d"].to_numpy()
is_treatment = (df["variant"] == "treatment").to_numpy()

observed_diff = y[is_treatment].mean() - y[~is_treatment].mean()

rng = np.random.default_rng(20261020)
n_permutations = 10_000
extreme_count = 0

for _ in range(n_permutations):
    shuffled = rng.permutation(y)
    null_diff = shuffled[is_treatment].mean() - shuffled[~is_treatment].mean()
    if abs(null_diff) >= abs(observed_diff):
        extreme_count += 1

# +1 korrekció: a megfigyelt felosztást is a null-eloszlás mintájába számítjuk.
p_two_sided = (extreme_count + 1) / (n_permutations + 1)

print(observed_diff, p_two_sided)
```

A laborfájlokból kapott pontos szám automatikusan még nem ellenőrzött kimenet, ezért csak a számítási utat ellenőriztük. A `validate="one_to_one"` egyúttal adatminőségi kapu: ha egy ügyfélhez több kimenet tartozna, a kapcsolás hibát adna, és a p-érték helyett a séma-eltérést kellene először javítani.

16.1.5 Eredmény értelmezése: mit mér a p-érték?

A p-érték formális definíciója itt:

$$p = P_{H_0}\left(T \text{ legalább olyan szélsőséges, mint } t_{\text{obs}}\right).$$

Szavakban: ha a nullhipotézis igaz, mekkora valószínűséggel kapnánk a megfigyelttel legalább egyezően szélsőséges statisztikát? A kis példában $p_{\text{two}} = 1/3$ nem azt mondja, hogy a nullhipotézis igazságának valószínűsége $1/3$, hanem azt, hogy a nullmodell alatt a lehetséges újrafelosztások harmada ilyen szélsőséges.

Az α **szignifikanciaszint** (*significance level*) egy előre rögzített döntési küszöb. Ha $p < \alpha$, a nullhipotézist elutasítjuk; ha $p \geq \alpha$, nem utasítjuk el. Az α tehát nem a p-értékből utólag képzett érték, hanem a hibavédelem specifikációja.

A kétféle döntési hiba:

Döntés	H_0 igaz	H_1 igaz
Elutasít	I. típusú hiba, valószínűsége legfeljebb α	helyes döntés
Nem utasít el	helyes döntés	II. típusú hiba, valószínűsége β

A Monte Carlo p-értéknek saját közelítési hibája van. Ha a p nagyságrendje 0,01, akkor a standard hiba közelítőleg $\sqrt{p(1-p)/B} \approx 0{,}001$, tehát a harmadik tizedesjegy már bizonytalan. A statisztikai bizonytalanság és a szimulációs bizonytalanság tehát külön jelentésre kerül, ahogyan 15.4. alfejezetben is szétválasztottuk. A jelen példa döntését a p-érték tartománya, nem a tizedesjegyek finomsága befolyásolja.

Feltétel: a csoportcímkék újrakeverése a randomizáció kiosztási mechanizmusát utánozza. A Fisher-féle egzakt randomizációs teszt szigorú, minden egyedre nulla kezelési hatást megfogalmazó *sharp null* mellett egzakt. Az átlagos hatás nulla önmagában nem ugyanaz a hipotézis; heterogén hatásoknál a naiv permutáció érvényessége külön indoklást igényel.

16.1.6 Feladatok

1. **Értelmezés.** Egy elemző ezt írja: „A p-érték 0,03, tehát 3% annak a valószínűsége, hogy a kezelésnek nincs hatása.” Melyik állítás igaz?

   a) Ez helyes.

   b) Ez nem helyes: a p-érték a nullmodell alatti feltételes valószínűsége annak, hogy a statisztika legalább ilyen szélsőséges.

   c) Ez nem helyes: a p-érték a hatásnagyság alsó korlátja.

2. **Megvalósítás.** Írj függvényt, amely a kis példa $\{0, 0, 1, 1\}$ adatán a teljes permutációs eloszlásból felső egyoldali p-értéket ad, ha a megfigyelt treatment-kontroll különbség 1. Ellenőrizd az invariánsokat: $0 \leq p \leq 1$, és a p-érték összege a felső farok valószínűségeivel konzisztens.

3. **Hibadiagnózis.** Az alábbi kód a laboros kétoldali tesztet próbálja megvalósítani. Mi a hiba, ha a megfigyelt különbség pozitív?

   ```
   p_wrong = 1 - np.mean(null_diffs >= observed_diff)
   ```

   A `null_diffs` a permutációs statisztikák tömbje. Adj ellenpéldát a kis példa `null_diffs = [-1, 0, 0, 0, 0, 1]` tömbjével és `observed_diff = 1` értékkel, majd add meg a helyes egyoldali p-értéket.

4. **Transzfer megváltozott feltételre.** Tegyük fel, hogy a kis példa minden megfigyelését megduplázzuk: a kontroll kimenetei $(0, 0, 0, 0)$, a treatment kimenetei $(1, 1, 1, 1)$ lesznek. A megfigyelt különbség továbbra is 1. Hány lehetséges felosztás van, és mennyi a kétoldali p-érték? Vesd össze az eredeti p-értékkel. Mit mutat ez a p-érték és a hatásméret viszonyáról?

5. **Szimulációs ellenőrzés.** Tiszta nullmodellben, $n_t = n_c = 50$ és $p = 0{,}4$ Bernoulli-kimenettel, szimuláljon 1000 kísérletet. Minden kísérletben számítsd ki a $T = \hat{p}_t - \hat{p}_c$ statisztikát, és döntsön egy szabályos kétoldali z-próbával $\alpha = 0{,}05$ mellett. Az I. típusú hibák aránya milyen érték körül várható? Add meg a várható arány Monte Carlo standard hibáját is: $\sqrt{0{,}05 \cdot 0{,}95/1000} \approx 0{,}0069$.

16.1.7 Megoldások és indoklás

1. A helyes válasz **b**. A p-érték feltételes valószínűség: feltétele a nullmodell, nem pedig a nullhipotézis igazságának valószínűségét méri. A p-érték nem hatásnagyság és nem is annak alsó korlátja.

2. Egy lehetséges függvény:

```
def exact_upper_p(y, n_t, observed_diff):
    from itertools import combinations
    import numpy as np
    null_diffs = []
    for idx in combinations(range(len(y)), n_t):
        mask = np.zeros(len(y), dtype=bool)
        mask[list(idx)] = True
        null_diffs.append(y[mask].mean() - y[~mask].mean())
    null_diffs = np.array(null_diffs)
    return np.mean(null_diffs >= observed_diff)

print(exact_upper_p(np.array([0, 0, 1, 1]), 2, 1.0))  # 0.166666...
```

   Ellenőrzés: a p-érték nemnegatív és legfeljebb 1, mert gyakorisági arány. A felső farok a háromféle null-érték közül csak az 1-eket foglalja magába: 1/6.

3. A hiba az, hogy `1 - np.mean(null_diffs >= observed_diff)` a $P(T < t_{\text{obs}})$ valószínűséget becsli, nem a felső farok valószínűségét. A kis példában `np.mean(null_diffs >= 1) = 1/6`, ezért `p_wrong = 5/6  0,8333`, ami azt sugallná, hogy az extrém megfigyelés a nullmodell alatt nagyon gyakori; valójában a helyes felső p 1/6. A kód a farok komplementerét számolja.

4. A nyolc megfigyelt érték $\{0, 0, 0, 0, 1, 1, 1, 1\}$ közül a treatment csoport négy elemet kap; ez $\binom{8}{4} = 70$ felosztás. A treatment csoport maximális átlaga akkor jön létre, ha mind a négy 1-es a treatmentbe kerül; ez egyetlen felosztás. A minimális átlag ugyanilyen, amikor minden 1-es a kontrollba kerül. A megfigyelt abszolút eltérés 1, ezért a kétoldali p-érték

$$p_{\text{two}} = P_{H_0}(|T| \geq 1) = \frac{2}{70} = \frac{1}{35} \approx 0{,}0286.$$

   Az eredeti p 1/3 volt. A hatásméret ugyanaz, a p mégis jelentősen csökkent, mert a nagyobb minta ugyanakkora eltérést már kevésbé tart valószínűnek a nullmodell alatt. A p-érték tehát a minta nagyságát és a hatást együtt tükrözi, nem önmagában a hatás nagyságát.

5. Az 1000 szimulációból $\alpha = 0{,}05$ mellett körülbelül 50 elutasítás várható; a tapasztalt arány szórása közelítőleg 0,0069, ezért egy 0,04 és 0,06 közötti arány még konzisztens a névleges 0,05 szinttel. Előre rögzített magvet használva a tapasztalt arány ismételhető. Ellenőrző invariáns, hogy a z-statisztika a nullmodell alatt közelítőleg standard normális, ahol $z = t_{\text{obs}}/\sqrt{\hat{p}(1-\hat{p})(1/n_t + 1/n_c)}$ a közös becsült aránnyal. A becsült I. típusú arány például 0,052, 0,048 vagy 0,055 közeli érték lehet; a pontos érték a szimulátor megvalósításától függ.

16.1.8 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés: **a p-érték nem a nullhipotézis igazságának valószínűsége.** A gyakorisági keretben a nullhipotézis nem kap valószínűséget; a p-érték feltétele az, hogy a nullmodell igaz. Ha $p = 0{,}04$, nem mondható, hogy „4% eséllyel nincs hatás”. Helyesen: **a nullmodell és az alkalmazott mintavételi/hozzárendelési eljárás feltételezése mellett** a megfigyelttel legalább ennyire szélsőséges tesztstatisztika valószínűsége 4%.

Ehhez kapcsolódik a második csapda: a **nem szignifikáns eredmény nem automatikus egyenértékűség.** A kis példában $t_{\text{obs}} = 1$ nagy hatásnak látszik, mégis $p = 1/3$, mert a minta kicsi. A „nem találtunk különbséget” állítás itt téves lenne; csak annyi igaz, hogy ez a minta nem adott elég erős bizonyítékot a nullmodell ellen. A hatásméretet és a bizonytalanságot a p-érték mellett mindig jelenteni kell.

Visszautalás 15.1. alfejezetre: a likelihood a paraméter függvényében nézte az adatot; itt ezzel szemben egy rögzített nullparaméter alatt nézzük a statisztika feltételes eloszlását. Mindkettő feltételes gondolkodás, csak az irány más. Visszautalás 15.4. alfejezetre: a Monte Carlo p-érték kvantilis-bizonytalansága külön becslési bizonytalanság, nem szabad a nullhipotézis bizonytalanságaként kezelni.

Önellenőrző kérdések:

- Mi a különbség $p < \alpha$ döntés és $P(H_0 \text{ igaz})$ állítás között?
- Miért számolunk +1 korrekciót a Monte Carlo p-érték nevezőjében és számlálójában?
- Mi az I. típusú hiba definíciója, és melyik mennyiséget korlátozza α?

Ha mindhárom kérdésre a feltételes valószínűség nyelvén tud válaszolni, akkor a nullmodell és a p-érték viszonya stabil. A következő alfejezet a tesztválasztást és a hatásméretet kapcsolja a p-értékhez, hogy a döntés ne csak szignifikanciáról, hanem gyakorlati jelentőségről is szóljon.

16.1.9 Rövid összefoglaló

A nullhipotézis a vizsgált állítás; lehet egyszerű vagy összetett, és a randomizációs teszt külön feltételeket igényel. a tesztstatisztika a minta előre definiált függvénye. A p-érték annak feltételes valószínűsége, hogy a nullmodell alatt legalább ilyen szélsőséges statisztikát kapunk. Az α küszöb előre rögzíti a maximális I. típusú hibát, míg a II. típusú hiba az erővel függ össze. A p-érték nem a nullhipotézis igazságának valószínűsége, a nem szignifikáns eredmény pedig nem egyenértékűség. A permutációs teszt a randomizált hozzárendelést használja nullmodellként, ügyfélszintű újracímkézéssel. A laboron ez a primer konverziós különbség kétoldali p-értékét adja, amelyet hatásmérettel és a Monte Carlo hiba nagyságrendjével együtt kell jelenteni.

16.2 Tesztválasztás és hatásméret

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 16.1. alfejezet · Célkészség: Tesztválasztás és hatásméret · Kimenet: működő kód és a labor ellenőrző tesztje.

16.2.1 Cél és konkrét helyzet

A 16.1. alfejezetben a Northstar kísérletre már kaptunk egy permutációs p-értéket, de ott a nullmodell és a p-érték logikája volt a kérdés. Most a tesztválasztást tesszük tudatossá. A cél ugyanaz a szintetikus kísérlet: értékeljük, hogy a `treatment` ág növeli-e a `conversion_7d` primer végpontot a `control` ághoz képest, az összes randomizált ügyfél ITT-szemléletével, és becsüljük meg a hatás nagyságát is.

Bemenet: `lab/data/assignments.csv` és `lab/data/outcomes.csv`. Megfigyelési egység: egy `customer_id`, egyszer mérve. A `conversion_7d` értéke 0 vagy 1, a `variant` értéke `control` vagy `treatment`. Ellenőrzési feltétel: a két tábla one-to-one joinja után 300 sor legyen, a `conversion_7d` mindenütt bináris, és a csoportszámok összege 300 legyen. Kimenet: abszolút és relatív hatás, 95%-os konfidenciaintervallum (CI), választott teszt p-értéke.

Induló becslés, mielőtt megoldanád: ha a kontrollkonverzió 20% körüli, milyen abszolút különbséget tartanál üzletileg fontosnak? A példában 10 százalékpontot használok gyakorlati küszöbnek; ehhez 300 ügyfél már csak mérsékelten érzékeny, ezért a CI is lényeges, nem csak a p-érték.

16.2.2 Páros vagy független? A tervezés dönti el

Két csoport összehasonlításánál az első kérdés nem a csomagfüggvény, hanem az adatfelvétel szerkezete. Négy oszlopba foglalva:

Dizájn	Egység és mérés	Következmény	SciPy-függvény
Független	minden ügyfél egy ágban, egy mérés	a csoportok közötti szórást is becsülni kell	`ttest_ind`
Páros	ugyanazon egység kétszer, vagy előre párosított egységek	a páron belüli szint eltávolítható	`ttest_rel`

Páros esetben a nyers adat két hosszú, egyenként n elemű tömb: $x_{i,\text{előtte}}$ és $x_{i,\text{utána}}$. A páronkénti különbség

$$d_i = x_{i,\text{utána}} - x_{i,\text{előtte}}, \qquad i = 1, \dots, n.$$

Itt d_i mértékegysége azonos az eredményváltozóéval, shape-je `(n,)`. A páros t-próba statisztikája

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \qquad s_d^2 = \frac{1}{n-1}\sum_{i=1}^{n}(d_i - \bar{d})^2,$$

ahol $\bar{d}$ a különbségek átlaga, s_d a mintabeli szórás, a szabadságfok $n-1$. Kis kézi példa: öt ügyfél várakozási ideje percben előtte `[10,12,11,14,15]`, utána `[8,10,10,12,12]`; a különbségek `[2,2,1,2,3]`, átlaguk 2,0. A szórásnégyzet 0,5, így $t = 2/(\sqrt{0{,}5}/\sqrt{5}) \approx 6{,}325$. A párosítás csökkenti a párok közötti szórást, de csak akkor jogszerű, ha a párosítás a tervezésből jön: ugyanaz az ügyfél előtte/utána, vagy előre rögzített matched-pairs design.

A Northstar kísérletben nincs ilyen. Minden ügyfél egyszer szerepel, és a randomizálás az ügyfelet önállóan osztja be az egyik ágba. Ha valaki `customer_id` szerint rendezés után mesterségesen párokat képezne, akkor egy nem létező páron belüli összefüggést tételezne fel; a kapott standard

error nem jelenthető ki a tényleges mintavételi folyamatból. Ezért a helyes alap a független összehasonlítás.

16.2.3 A Welch-próba a független csoportokra

Független csoportoknál a klasszikus kétmintás t-próba feltételezi az egyenlő varianciákat. Ezt feleslegesen szigorú feltevés; a Welch-változat nem feltételezi ezt, és az `equal_var=False` beállítással ez a biztonságos alap SciPy-ban. A becsült átlagkülönbség standard errorja

$$\mathrm{SE} = \sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}},$$

ahol s_g^2 a g csoport mintabeli varianciája, n_g a csoport mérete; az SE mértékegysége megegyezik az eredményváltozóéval. A szabadságfok a Satterthwaite-közelítés:

$$\nu = \frac{\left(s_0^2/n_0 + s_1^2/n_1\right)^2}{\frac{(s_0^2/n_0)^2}{n_0-1} + \frac{(s_1^2/n_1)^2}{n_1-1}}.$$

Az eljárás feltétele: a csoportátlagok mintavételi eloszlása jól közelíthető normálissal. Ez igaz lehet nagy n_0, n_1 mellett akkor is, ha az egyedi értékek nem normálisak, mert a centrális határeloszlás az átlagra vonatkozik, nem az egyedekre (lásd 15.2. alfejezet). Kicsi, erősen ferde mintánál viszont a Welch-teszt is félrevezető lehet.

A Northstar primer végpontja bináris. A Welch-teszt itt is kiszámítható, mert az átlag éppen az arány, de a konverziós végpontra van pontosabban illeszkedő modell is: a kétmintás arányteszt.

16.2.4 Arányteszt a bináris végpontra

Legyen a control és treatment csoport mérete n_0 és n_1; a megfigyelt konverziós tömb x_0 és x_1, mindkettő shape-je `(n_g,)`, elemei 0 vagy 1. A konverziók száma $X_0 = \sum x_{0i}$, $X_1 = \sum x_{1i}$, a becsült arányok

$$\hat{p}_0 = X_0/n_0, \qquad \hat{p}_1 = X_1/n_1,$$

mindkettő a $[0,1]$ tartományban, dimenzió nélküli valószínűség. A nullhipotézis $H_0 : p_0 = p_1 = p$. A közös arány becslése a nullmodell alatt:

$$\hat{p} = \frac{X_0 + X_1}{n_0 + n_1}.$$

A tesztstatisztika a standardizált különbség:

$$z = \frac{\hat{p}_1 - \hat{p}_0}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_0} + \frac{1}{n_1}\right)}}.$$

A nevező a nullmodell szerinti standard error, ahol a közös $\hat{p}$-t használjuk. A kétoldali p-érték $2\,(1 - \Phi(|z|))$, ahol Φ a standard normális eloszlásfüggvény (lásd 12.2. alfejezet: a CDF területet ad, a farkak a túloldali területek).

Kézi kis számpélda: $n_0 = n_1 = 10$, konverziók 2 és 5, tehát $\hat{p}_0 = 0{,}20$, $\hat{p}_1 = 0{,}50$, különbség 0,30. A közös arány $\hat{p} = 7/20 = 0{,}35$. A null-standard error

$$\sqrt{0{,}35 \cdot 0{,}65 \cdot (1/10 + 1/10)} = \sqrt{0{,}0455} \approx 0{,}2133,$$

így $z \approx 1{,}406$, a kétoldali p-érték $\approx 0{,}15$. A nagy p azt jelenti, hogy ekkora minta még egy látványos pontalakú különbséget sem tud stabilan kimutatni.

A CI nem a fenti pooled standard errort használja, hanem a két becslés saját szórását:

$$\mathrm{SE}_{\mathrm{CI}} = \sqrt{\frac{\hat{p}_0(1-\hat{p}_0)}{n_0} + \frac{\hat{p}_1(1-\hat{p}_1)}{n_1}}.$$

A 95%-os Wald-intervallum:

$$\hat{p}_1 - \hat{p}_0 \pm z_{0{,}975}\,\mathrm{SE}_{\mathrm{CI}},$$

ahol $z_{0{,}975} \approx 1{,}96$. Emiatt a p-érték és a CI csak közelítőleg lesz konzisztens, nem matematikai azonossággal: a teszt null alatti, a CI az adott pontban becsült szórást használ.

Az érvényességi korlát: a normális közelítéshez mindkét csoportban elegendő várható konverzió-szám kell. Gyakorlati hüvelykujjként $n_0\hat{p}_0$, $n_0(1-\hat{p}_0)$, $n_1\hat{p}_1$, $n_1(1-\hat{p}_1)$ mindegyike legalább 5, jobb esetben 10 legyen. Ha ez sérül, egzakt vagy permutációs eljárás indokolt; erre a küszöb, kalibráció és ritka esemény rész külön is visszatér.

16.2.5 Permutációs teszt mint a randomizáció közvetlen modellje

A másik, a kísérleti dizájnhoz nagyon közel álló választás a permutációs teszt. A Neyman-féle randomizációs nullmodellben a kapott 300 `conversion_7d` értéket rögzítettnek tekintjük, és csak a `variant` címkéket keverjük újra. A tesztstatisztika ugyanaz az átlagkülönbség:

$$T_{\mathrm{obs}} = \hat{p}_1 - \hat{p}_0.$$

Ha null hatás van, akkor egy ügyfél kimenete nem függ attól, melyik címkét kapja, ezért a címkék cserélhetők. A p-érték közelítése: 9999 újrarendezést készítünk rögzített seeddel, megszámoljuk, hányszor kapunk legalább akkora abszolút különbséget, majd

$$p_{\mathrm{perm}} = \frac{1 + \#\{|T_b| \geq |T_{\mathrm{obs}}|\}}{1 + 9999}.$$

Ez a p-érték a nullmodell feltételes valószínűségeként értelmezhető, ahogy 16.1. alfejezet bevezette. Fontos: a permutációs teszt a „nincs hatás egyetlen egységre sem” éles nullmodellt vizsgálja, és a statisztika megválasztásával irányítjuk, milyen eltérésre legyen érzékeny.

16.2.6 Hatásméret: abszolút, relatív és praktikus állítás

A statisztikai szignifikancia nem mondja meg, érdemes-e bevezetni a kezelést. A három leggyakoribb bináris hatásméret:

- abszolút kockázatkülönbség (risk difference, RD): $\hat{p}_1 - \hat{p}_0$, százalékpontban;
- relatív kockázat (risk ratio, RR): $\hat{p}_1/\hat{p}_0$, csak $\hat{p}_0 > 0$ esetén értelmes;
- relatív növekedés: $(\hat{p}_1/\hat{p}_0 - 1) \cdot 100\%$.

A RD közvetlenül működési kérdésekre válaszol: „100-ból hány további ügyfél konvertál?” Az RR viszont a kockázat arányos változását mutatja, és alacsony kontrollaránynál nagyobbnak tűnhet, mint a gyakorlati jelentősége indokolná. Ezért a két számot együtt, a CI kontextusában érdemes közölni.

A gyakorlati döntés nem a pontbecslésből, hanem a CI végpontjaiból olvasható: ha a CI magában foglalja a 0-t, nem tudunk null hatást kizárni; ha a CI teljes egésze a gyakorlati küszöb alatt van, akkor a hatás statisztikailag kimutatható, de üzletileg talán nem elég nagy. A nem szignifikáns eredmény eközben nem egyenértékű a null hatással, csak azt jelenti, hogy a minta nem adott elég erős bizonyítékot (lásd 16.1. alfejezet).

16.2.6.1 Tesztválasztási döntési tábla

A teszt kiválasztása a cellaszámoktól és a design-tól függ, nem a szokásostól. Bináris kimenetnél a következő döntési tábla használható:

Feltétel	Ajánlott teszt	Hatásméret	Intervallum
Minden cella ≥ 5, $n_0, n_1 \geq 30$	kétmintás arány-z (pooled SE)	RD, RR	Wald (unpooled) vagy Newcombe
Bármely cella < 5, vagy $n < 30$	Fisher-féle egzakt / randomizációs teszt	RD	Newcombe vagy Wilson
0 esemény valamelyik ágon	Fisher/Barnard egzakt teszt	RD	Newcombe; az RR nem értelmezhető
Szélsőséges arány ($< 0{,}05$ vagy $> 0{,}95$)	egzakt vagy score-teszt	RD	Wilson/Newcombe
Folytonos, két független csoport	Welch-teszt	átlagkülönbség	Welch-CI
Párosított folytonos	páros t / Wilcoxon	párdifferencia	páros CI
Klaszterezett	klaszter-robusztus vagy klaszter-permutáció	klaszterszintű hatás	klaszter-bootstrap
Több csoport	ANOVA / Kruskal–Wallis	η^2 vagy ranghatás	post-hoc páronkénti CI

A döntési tábla nem helyettesíti a diagnosztikát: a normalitás, a függetlenség és a kiosztási egység ellenőrzése a tesztválasztás előtt történik.

Ugyanaz a 2×2 tábla három módszerrel. A 150–150 fős, 30 kontroll- és 45 treatment-konverziós példán:

Módszer	RD	95% CI	p-érték
Kétmintás arány-z (pooled SE)	0,100	Wald: [0,0027; 0,1973]	0,0455
Newcombe (Wilson-alapú)	0,100	[0,0019; 0,1958]	–
Fisher-féle egzakt	0,100	–	0,0615

A három módszer ugyanarra az adatra más p-értéket ad: a Wald-CI és a z-teszt a normálközelítésre épül, a Fisher-teszt a hipergeometriai null-eloszlásra. Kis cellaszámnál a különbség nagyobb. Például 20–20 főnél 2 kontroll- és 7 treatment-konverzió mellett a Wald-CI [0,0031; 0,4969], a Newcombe-CI [−0,0125; 0,4788], a Fisher-pedig 0,1274; a Newcombe-CI itt már tartalmazza a nullát, a Wald-CI nem. Ha a kontrollágon 0 esemény van (0/20 vs 7/20), a Wald-CI alsó végpontja 0,1410, a Newcombe-CI [0,1166; 0,5671], a Fisher-p 0,0083; az RR végtelen, ezért RD-t kell jelenteni. Ezek az értékek a normalitás határán mutatják, miért kell a cellaszámot és a 0 eseményt a tesztválasztás előtt ellenőrizni.

16.2.7 Teljes minimális munkapélda a Northstar kísérleten

A munkakönyvtár a `lab` projektgyökér. A kód a szállított CSV-kből olvas, a saját gépeden fut, és semmilyen külső szolgáltatást nem hív.

```
# s062_northstar_effect.py
# Futtatás: python s062_northstar_effect.py, a lab projektgyökérből.
import numpy as np
import pandas as pd
from scipy import stats

assign = pd.read_csv("lab/data/assignments.csv")
out = pd.read_csv("lab/data/outcomes.csv")

# One-to-one join: hiba lenne, ha egy ügyfél többször szerepelne.
df = assign.merge(out, on="customer_id", validate="one_to_one")
assert df.shape[0] == 300
assert set(df["conversion_7d"].unique()).issubset({0, 1})

x0 = df.loc[df["variant"] == "control", "conversion_7d"].to_numpy(dtype=float)
x1 = df.loc[df["variant"] == "treatment", "conversion_7d"].to_numpy(dtype=float)
n0, n1 = x0.size, x1.size
p0, p1 = x0.mean(), x1.mean()
rd = p1 - p0
rr = p1 / p0 if p0 > 0 else np.nan
relative_pct = (rr - 1) * 100

# Nullmodell: közös arány és pooled standard error.
p_hat = (x0.sum() + x1.sum()) / (n0 + n1)
se_null = np.sqrt(p_hat * (1 - p_hat) * (1 / n0 + 1 / n1))
z = rd / se_null
p_z = 2 * (1 - stats.norm.cdf(abs(z)))

# CI: unpooled standard error.
se_ci = np.sqrt(p0 * (1 - p0) / n0 + p1 * (1 - p1) / n1)
lo, hi = rd + stats.norm.ppf([0.025, 0.975]) * se_ci
assert lo <= rd <= hi

# Welch-teszt ugyanezen bináris értékeken, ellenőrzésként.
welch = stats.ttest_ind(x1, x0, equal_var=False)
```

```
# Permutációs teszt, rögzített seeddel.
rng = np.random.default_rng(20261020)
runs = 9999
y = df["conversion_7d"].to_numpy(dtype=float)
trl = (df["variant"] == "treatment").to_numpy()
obs = y[trl].mean() - y[~trl].mean()
null_stats = np.empty(runs)
for i in range(runs):
    perm = rng.permutation(trl)
    null_stats[i] = y[perm].mean() - y[~perm].mean()
count = np.sum(np.abs(null_stats) >= abs(obs))
p_perm = (count + 1) / (runs + 1)

print(f"n_control={n0}, n_treatment={n1}")
print(f"control={p0:.3f}, treatment={p1:.3f}")
print(f"RD={rd:.3f}, relative_pct={relative_pct:.1f}%")
print(f"95% CI: [{lo:.3f}, {hi:.3f}]")
print(f"z={z:.2f}, p_z={p_z:.3f}")
print(f"Welch p={welch.pvalue:.3f}, perm_p={p_perm:.3f}")
```

A fenti kimenetet itt nem nevezem mértnek; a fönti blokk helyi futtatásban a lab CSV-jéből számol. Egy elvi vázlat, ami nem a szállított fájl tényleges eredménye: ha a kontrollarány 0,19, a treatment 0,28 lenne, akkor RD 0,09, relatív növekedés 47%, a z-teszt p-értéke 0,05 alá esik, a CI pedig 0 közeléből 0,18 felé nyúlna. A te futtatásod ettől véletlenül eltérhet.

16.2.8 Eredmény értelmezése és kézi kontroll

Értelmezd egyszerre a pontbecslést, a bizonytalanságot és a szignifikanciát. A fenti illusztrációban az abszolút hatás 9 százalékpont, a relatív növekedés mintegy 47%. Ez önmagában fontos lehet, de a 95%-os CI széles: ha az alsó végpont 0 közelében van, akkor a valódi hatás lehet nagyon kicsi is. A p-érték így nem „a feltétel fontosságának” mértéke, hanem annak a valószínűsége, hogy a nullmodell ilyen szélsőséges statisztikát generáljon.

Kézi kontrollok:

- shape: `x0.size + x1.size == 300` és `df.shape[0] == 300`;
- bináris dtype: a `conversion_7d` értékkészlete csak `{0,1}`;
- invariáns: `lo <= rd <= hi`;
- tolerancia: a Welch- és a z-teszt p-értéke nagy mintán hasonló nagyságrendű, de nem azonosan egyenlő, mert más standard errort használnak;
- Monte Carlo kontroll: a permutációs p-érték nagyságrendjében egyezzen a z-teszttel, de a véletlen permutáció hibája $1/\sqrt{runs}$ nagyságrendű lehet, ezért a harmadik tizedes nem stabil.

A `validate="one_to_one"` hívás a 4.2. alfejezetben tanult ellenőrzési szemléletet alkalmazza: a join előtti elvárást rögzítjük, nem printeljük ki utána a teljes táblát.

Értelmezési határ: a teszt kiválasztása előtt ellenőrizni kell a kimenet skáláját, a megfigyelések függőségét, a kiosztási egységet és a tervezett estimandot. A statisztikailag szignifikáns eltérés nem azonos a gyakorlati fontossággal; a százalékpontos hatás és az intervallum mindenképp szerepeljen.

16.2.9 Kitekintés: egyszerű power-szimuláció

A tesztválasztást a lehetséges erő is irányítja. Kis bináris szimulációval gyorsan átérezhető, mekkora n kell egy adott hatáshoz:

```
def power_sim(n, p0, p1, runs=500, seed=123):
    rng = np.random.default_rng(seed)
    rej = 0
    for _ in range(runs):
        c = rng.binomial(n, p0)
        t = rng.binomial(n, p1)
        p_hat = (c + t) / (2 * n)
        se = np.sqrt(p_hat * (1 - p_hat) * 2 / n)
        z = ((t - c) / n) / se
        p = 2 * (1 - stats.norm.cdf(abs(z)))
        rej += p < 0.05
    return rej / runs
```

A `power_sim(150, 0.20, 0.30)` például azt kérdezi, hogy 150-150 fős ágak esetén milyen gyakran utasítjuk el a nullmodellt, ha a valódi arányok 0,20 és 0,30. Elvi várakozásként a 0,07-es null-standard error mellett $z \approx 1{,}4$, így a szimulált erő 0,4–0,5 körüli lehet; ez szimulációs becslés, nem levezetett mért eredmény. A 16.3. alfejezet ezt a kérdést szisztematikusan tárgyalja.

16.2.10 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés az, hogy a nemparaméteres vagy rangtesztet automatikusan az átlagkülönbség „eloszlásmentes” vizsgálataként használjuk. A Mann-Whitney-féle teszt nem az átlagok egyenlőségét, hanem a két eloszlás sztochasztikus egyenlőségét vizsgálja. Ellenpélda-generátor, amely ezt a különbséget szemlélteti:

```
rng = np.random.default_rng(7)
# Két eloszlás, mindkettő várható értéke 1.0,
# de a variancia 4.0 és 1.0, és az alak is erősen eltér.
x = rng.gamma(shape=0.5, scale=2.0, size=400)
y = rng.exponential(scale=1.0, size=400)
p_mw = stats.mannwhitneyu(x, y, alternative="two-sided").pvalue
```

A konkrét p-érték mintánként változik, ezért nem közlök számszerű eredményt. A vizsgálandó állítás konceptuális: a teszt ismételt mintavételben gyakran szignifikáns lehet, miközben mindkét sokasági átlag azonos. A tanulság: mindig a célparaméterhez válassz tesztet. Ha a kísérleti döntés célja az átlagkülönbség, akkor a Welch- vagy a permutációs átlagteszt a közvetlen válasz; ha a cél a teljes eloszlás különbözősége, akkor a rangpróba is indokolt lehet.

Önellenőrző kérdések:

1. Miért nem páros a Northstar kísérlet?
2. A z-teszt standard errorja miért pooled?
3. Mikor nem alkalmazható a normális arányteszt?
4. Mit jelent, ha a CI tartalmazza a 0-t, de a pontbecslés nagy?

16.2.11 Feladatok

1. Értelmezés. Egy csapattárs azt javasolja, hogy rendezd `customer_id` szerint a 300 ügyfelet, majd az egymás mellé kerülő control-treatment sorokra futtass `ttest_rel`-et. Miért hibás ez a dizájn? Mit tételez fel a párosítás, és mi a helyes eljárás itt? Rövid indoklást adj a Northstar adatszerződéséből.

2. Implementálás. Egy cikk példájában 150-150 ügyfél volt: control ágon 30, treatment ágon 45 konverzió. Kézzel számold ki a z-tesztet: közös arány, null-standard error, z, kétoldali p-érték, valamint a 95%-os Wald-CI. Ezután írj egy olyan ellenőrző `assert` blokkot, amely a fenti `s062_northstar_effect.py` logikájával reprodukálja ezeket a számokat.

3. Hibadiagnózis. Az alábbi kód két hibát tartalmaz:

```
se = np.sqrt(p0 * (1 - p0) / n0 + p1 * (1 - p1) / n1)
z = (p1 - p0) / se
p = stats.norm.cdf(abs(z))
```

Az előző feladat adataival mi a két hiba? Számítsd ki a hibás és a helyes p-értéket is, és magyarázd el, miért nem a CI standard errorjából készül a teszt.

4. Megváltozott feltétel. A termékcsapat egy 20-20 fős pilotban 2 control- és 7 treatment-konverziót látott. Kiszámolnád-e a z-tesztet? Ellenőrizd a normális közelítés feltételét, és javasolj alternatívát. A z-teszt számértékét add meg, de a végső döntési javaslatodban indokold, miért nem érdemes önmagában erre hagyatkozni.

16.2.12 Megoldások és indoklás

1. A Northstar kísérletben nincs „ugyanaz az ügyfél kétszer” megfigyelés, és nincs előre párosított hozzárendelés sem. A `customer_id` rendezés szerinti párosítás mesterséges: a párok tartalma attól függ, milyen sorrendet választunk. A `ttest_rel` a d_i párkülönbségek szórását becsülné, de itt nincs közös párcsatorna, amelyet ki lehetne vonni. A helyes választás az egy ügyfél egy megfigyelés függetlenséget tiszteletben tartó Welch-próba vagy kétmintás arányteszt. Ellenőrző invariant: a Northstar `assignments` és `outcomes` táblák one-to-one joinnál 300 sort adnak, és egyetlen ügyfélnek sincs két ághoz tartozó sora.

2. $n_0 = n_1 = 150$, $\hat{p}_0 = 30/150 = 0{,}20$, $\hat{p}_1 = 45/150 = 0{,}30$, közös arány $\hat{p} = 75/300 = 0{,}25$. A null-standard error:

$$\sqrt{0{,}25 \cdot 0{,}75 \cdot (1/150 + 1/150)} = 0{,}05.$$

Ebből $z = (0{,}30 - 0{,}20)/0{,}05 = 2{,}0$, a kétoldali p-érték

$$p = 2(1 - \Phi(2)) \approx 0{,}0455.$$

A CI standard errorja

$$\sqrt{\frac{0{,}2 \cdot 0{,}8}{150} + \frac{0{,}3 \cdot 0{,}7}{150}} \approx 0{,}04967,$$

így a 95%-os CI $0{,}10 \pm 1{,}96 \cdot 0{,}04967$, azaz $[0{,}003, 0{,}197]$. A kódban a tűrés például `abs(se_null - 0.05) < 1e-9` és `abs(p - 0.0455) < 1e-2`.

3. Az első hiba: a teszt a nullmodell alatt közös arányt kell használjon, ezért `se` nem a két külön arányból, hanem a pooled $\hat{p}$-ből számolandó. A második hiba: a `stats.norm.cdf(abs(z))` a $Z \le |z|$ egyoldali területet adja, ami a példában 0,93 körüli érték, nem pedig a szélsőséges farok valószínűsége; a helyes kétoldali p $2(1 - \Phi(|z|))$. Az 1. feladat adatain a hibás p 0,93, a helyes p kb. 0,16. A CI valóban unpooled SE-t használ, de az a CI céljára való, nem a null alatti p-értékre.

4. A z-teszt számítása: közös arány $9/40 = 0{,}225$, null-standard error $\sqrt{0{,}225 \cdot 0{,}775 \cdot (1/20 + 1/20)} \approx 0{,}132$, a különbség $0{,}35 - 0{,}10 = 0{,}25$, ezért $z \approx 1{,}89$ és $p \approx 0{,}058$. Ugyanakkor a várható cellaértékek 4,5 körüliek, tehát a normális közelítés feltétele nem teljesül. Az ilyen kis mintánál permutációs teszt vagy Fisher-egzakt teszt javasolt, például `stats.fisher_exact([[2, 7], [18, 13]], alternative="two-sided")`; a döntéshez a pontos p-értéket és a CI-t, nem a z-próba egyetlen számát használd. Ráadásul 20-20 fővel egy 25 százalékpontos mintabeli különbség is nagyon bizonytalan: a CI alsó vége könnyen negatív lehet.

16.2.13 Rövid összefoglaló

A tesztet először a megfigyelési terv határozza meg: páros mérés esetén párkülönbségekből számolunk, független randomizálásnál viszont a Northstar ügyfelei egyenkénti egységek, ezért párosítás hamis lenne. Független csoportokon a Welch-próba nem tételezi fel az egyenlő varianciát; bináris kimenetre a kétmintás arányteszt használ pooled null-standard errort, a CI viszont unpooled szórást. A permutációs teszt a randomizált hozzárendelés nullmodelljét közvetlenül gyakorolja. A hatást ne csak p-értékkel, hanem abszolút kockázatkülönbséggel, relatív kockázattal és a CI végpontjaival jelentsd. A nemparaméteres teszt nem automatikus átlagteszt; a célparaméterhez kell választani. A következő lépés a 16.3. alfejezetben az erő, mintanagyság és többszörös próbák rendszerezése.

16.3 Power, mintanagyság és többszörös próbák

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 16.2. alfejezet · Célkészség: Power, mintanagyság és többszörös próbák · Kimenet: működő kód és a labor ellenőrző tesztje.

16.3.1 Cél és konkrét helyzet

A Northstar kísérleti folyamatában minden ügyfél egyszeri randomizálási egység, a primer kimenet a bináris `conversion_7d`. A korábbi 16.2. alfejezetben a kontrolkonverziót körülbelül 0,18-nak, a kezelés hatását abszolút kockázatkülönbségként δ értéknek tekintettük. Most azt a tervezési kérdést vizsgáljuk: ha egy új kísérletet indítanánk $n = 150$ ügyfél/csoport mintanagysággal, kétoldali $\alpha = 0{,}05$ szintű aránytesszttel, mekkora eséllyel találnánk meg az előre rögzített $\delta = 0{,}05$ és $\delta = 0{,}10$ hatást? Tegyük fel, hogy a döntési szabály az, hogy $|z| > z_{0{,}975}$ esetén elutasítjuk a nullát. Azt is mérni akarjuk, hogyan növeli a hamis pozitív döntések arányát, ha ugyanazt a kísérletet többször, közbenső mintaállapotokban is kiértékeljük.

Bemenet: $p_0 = 0{,}18$, $\delta \in \{0{,}05; 0{,}10\}$, $n = 150$, $\alpha = 0{,}05$, $B = 4000$ szimulációs ismétlés, seed `20261020`.
Kimenet: elutasítások aránya = becsült power; a Monte Carlo standard hibája.
Ellenőrzés: $\delta = 0$ mellett a becsült power α körül legyen, és a kézi normál közelítésből kapott érték a szimuláció $1{,}96 \cdot SE$ sugarú intervallumán belül legyen.

Előzetes becslés a képletek nélkül: `power( =0.10)` körülbelül 0,5–0,6 lehet $n = 150$ csoportonként; `power( =0.05)` lényegesen kisebb, talán 0,2 alatt.

16.3.2 Fogalmi magyarázat: power, MDE, peeking, többszörös próbák

A másodfajú hiba angolul *type II error*: $\beta = P(\text{nem utasítjuk el } H_0 \mid H_1 \text{ igaz})$. Kiegészítője a **power**:

$$1 - \beta = P(\text{elutasítjuk } H_0 \mid H_1 \text{ igaz}),$$

amely egy $[0, 1]$-beli valószínűség, nem a tényleges adatból számolt bizonyosság. A power az adatgyűjtés előtti tervezési mennyiség: attól függ, mekkora α-t, n-t, hatást és varianciát rögzítünk, és melyik tesztet használjuk.

Két arány összehasonlításakor a tesztstatisztika a közös nullabeli aránnyal:

$$z = \frac{\hat{p}_1 - \hat{p}_0}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_0} + \frac{1}{n_1}\right)}}, \qquad \hat{p} = \frac{x_0 + x_1}{n_0 + n_1}.$$

Itt $\hat{p}_0 = x_0/n_0$, $\hat{p}_1 = x_1/n_1$ csoportarányok, $\delta = \hat{p}_1 - \hat{p}_0$ dimenzió nélküli kockázatkülönbség, x_0, x_1 a konverziók darabszáma. Ha a valódi hatás δ és $n_0 = n_1 = n$, a power pozitív δ-ra jó közelítéssel

$$1 - \beta \approx \Phi\left(\frac{\delta}{\sqrt{\bar{p}(1-\bar{p})\frac{2}{n}}} - z_{1-\alpha/2} \right), \qquad \bar{p} = \frac{p_0 + p_1}{2},$$

ahol Φ a standard normál eloszlásfüggvény. Például $p_0 = 0{,}18$, $p_1 = 0{,}28$, $n = 150$ esetén $\bar{p} = 0{,}23$, a standard hiba $\sqrt{0{,}23 \cdot 0{,}77 \cdot 2/150} \approx 0{,}0486$, a nemcentralitási paraméter $0{,}10/0{,}0486 \approx 2{,}06$, így a power $\Phi(2{,}06 - 1{,}96) \approx 0{,}54$.

Minimum detectable effect (MDE) az a legkisebb valódi hatás, amelyet az adott elrendezés legalább cél-powerrel, tipikusan 80%-kal talál meg. Képlete általános normál becslőre

$$\delta_{\min} = (z_{1-\alpha/2} + z_{0{,}80})\, SE,$$

de két aránynál a SE is függ δ-tól a $\bar{p}$-n keresztül, ezért iterálni vagy szimulálni kell. A fenti példában az MDE körülbelül 0,14; ez nem azonos a 0,10-es célhatással.

16.3.2.1 Tervezési munkalap: a mintanagyság előzetes rögzítése

A power nem utólagos mutató, hanem a kísérlet **előtti** tervezési döntés. A következő munkalapot az adatok megtekintése előtt kell kitölteni és rögzíteni; a későbbi p-érték és CI ettől független, utólagos eredmény.

Paraméter	Példaérték	Miért kell rögzíteni?
Primer végpont	`conversion_7d` (bináris)	a teszt és a hatásméret típusa ebből következik

Paraméter	Példaérték	Miért kell rögzíteni?
Kontrollarány p_0	0,18	a variancia és az MDE erősen függ tőle
Minimális üzleti hatás δ	0,10	nem a megfigyelt hatás, hanem a döntéshez szükséges legkisebb hatás
Kétoldali α	0,05	a kritikus érték $z_{1-\alpha/2}$
Célpower	0,80	a $z_{0,80}$ tag és a minta mérete
Allokáció $k = n_1/n_0$	1 (vagy 2)	eltérő kararány eltérő n_0-t ad
Várható lemorzsolódás	10%	a besorozott n-t $1/(1-d)$ szorzóval kell növelni
Elemzési egység	`customer_id`	klaszterezés esetén a design-hatás is szorzó
Többszörös végpont	1 primer, a többi feltáró	Bonferroni/BH nélkül a FWER nő
Interim pillantások	nincs, vagy előre rögzített szám	utólagos peeking megnöveli a hamis pozitív arányt

A két arányra vonatkozó mintanagyság-képlet eltérő allokációval:

$$n_0 = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \left[p_0(1-p_0) + p_1(1-p_1)/k\right]}{(p_1 - p_0)^2}, \qquad n_1 = k\, n_0,$$

majd a lemorzsolódással $n_0^{\text{besorozott}} = \lceil n_0/(1-d) \rceil$, és ugyanígy n_1-re. A lépések:

1. rögzítsd p_0-t, a minimális üzleti δ-t és így $p_1 = p_0 + \delta$-t;
2. válaszd meg a kétoldali α-t és a célpoweret;
3. számold ki a nyers n_0, n_1-et a fenti képlettel;
4. oszd el $(1-d)$-vel a lemorzsolódás miatt;
5. ha több primer végpont van, a Bonferroni-szinttel (α/m) ismételd meg;
6. ellenőrizd vissza a power-t és az MDE-t a végleges n-nel;
7. rögzítsd a munkalapot a kísérlet megkezdése előtt.

A `lab/experiments/power/power_worksheet.py` ezt a számítást futtatja. A $p_0 = 0{,}18$, $\delta = 0{,}10$, kétoldali $\alpha = 0{,}05$, 80% power mellett a nyers minta csoportonként 275; 10% lemorzsolódással 305/305 (összesen 610). A 2:1 treatment:control allokáció 217 kontrollt és 434 treatmentöt ad (651 fő); három primer végpont Bonferroni-szintjével $(\alpha/3)$ 407/407 (814 fő); 90% powerrel 408/408 (816 fő). A 150/150-es minta powerje a 0,10-es hatásra 0,5448, az MDE 0,1381; a tervezett 305/305 mellett az MDE 0,0945. Ezek a számok a tervezés kontrolljai, nem a megfigyelt adat utólagos értelmezése.

Peeking (belepillantás): ha ugyanazt a kísérletet közbenső, kumulált mintaállapotoknál is kiértékeljük, és bármelyik pillantásnál $p < 0{,}05$ esetén megállunk, a „legalább egyszer szignifikáns” döntés hibavalószínűsége nagyobb, mint α. A korrelált közbenső tesztek miatt a névleges 0,05-os határok összességében fellazulnak; a helyes megoldás előre regisztrált interim-határ, például Bonferroni- vagy alfa-költő eljárás, nem utólagos „megállok, mert szép a p”.

Többszörös próbák (multiplicity): ha m független nullhipotézist 0,05 szinten tesztelünk, akkor legalább egy hamis elutasítás valószínűsége $1 - (1 - 0{,}05)^m$; például $m = 10$ esetén ez 0,401. A **family-wise error rate**:

$$FWER = P(V \geq 1),$$

ahol V a tévesen elutasított nullhipotézisek száma. A Bonferroni-korrekció minden egyes tesztet α/m szinten végez; ebből a $FWER \leq \alpha$ uniókorláttal adódik. A **false discovery rate** ezzel szemben a felfedezések közötti hamis arány várható értéke:

$$FDR = E\left[\frac{V}{\max(R,1)}\right],$$

ahol R az összes elutasítás száma. A Benjamini–Hochberg (BH) eljárás a növekvő p-értékekre az $i\alpha/m$ küszöbsort alkalmazza: megkeressük a legnagyobb k indexet, amelyre $p_{(k)} \leq k\alpha/m$, majd az első k hipotézist elutasítjuk. A küszöbök $i > 1$ esetén nagyobbak a Bonferroni-féle α/m küszöbnél. Független (illetve bizonyos pozitív függésű) tesztek mellett a BH az FDR-t kontrollálja; tetszőleges függésre ez általánosan nem garantált. Például tíz rendezett p-érték `0,001; 0,004; 0,009; 0,020; 0,041; ...` mellett a küszöbök `0,005; 0,010; 0,015; 0,020; 0,025; ...`, így az első négy hipotézis elutasításra kerül, mert a negyedik még éppen 0,020 ≤ 0,020.

A FWER szigorúbb: minden hamis pozitív eseményét bünteti; az FDR a hamis pozitívak arányát kontrollálja, ami sok szűrőtesztnél gyakorlatiasabb.

16.3.2.2 Négy hipotézis, előre rögzített döntési szabály

A multiplicitási stratégia akkor érvényes, ha az elemzés **előtt** rögzítjük a hipotéziscsaládot, a primer/szekunder szerepet és a korrekciót. Példa négy hipotézisre:

ID	Szerep	Kérdés	Korrekció
H1	primer	a treatment növeli a konverziót	$= 0.05$, önálló
H2	szekunder	a treatment növeli a kosárértéket	Holm a H2–H4 családon
H3	szekunder	a treatment csökkenti a késést	Holm a H2–H4 családon
H4	szekunder	a treatment növeli az ismételt vásárlást	Holm a H2–H4 családon

A szabály: H1-et külön, $=0.05$ mellett értékeljük; csak ha H1 szignifikáns, lépünk tovább a H2–H4 családra Holm-korrekcióval. Ha H1 nem szignifikáns, a H2–H4 eredmények exploratory jelölést kapnak, nem megerősítő állítást. A Bonferroni, a Holm és a BH más célt szolgál:

Módszer	Cél	Küszöb
Bonferroni	FWER szigorú kontrolja	α/m minden próbára
Holm	FWER kontrol, kevésbé konzervatív	rendezett $p_{(i)} \leq \alpha/(m-i+1)$
Benjamini–Hochberg	FDR kontrol	$p_{(i)} \leq iq/m$

Négy p-értékre $(0.001, 0.008, 0.03, 0.06)$:

Módszer	Elutasított
Bonferroni ($\alpha = 0.05$)	H1, H2 (küszöb 0.0125)
Holm ($\alpha = 0.05$)	H1, H2 (H3 0.03 $>$ 0.025-nél megáll)
BH ($q = 0.05$)	H1, H2, H3 (H4 0.06 $>$ 0.05)

A BH több hipotézist fogad el, mert nem a családon belüli hibát, hanem a hamis felfedezések arányát kontrollálja; ezért csak akkor használható, ha a cél ezt indokolja. A döntési szabályt nem a p-értékek láttán választjuk ki.

```
def bonferroni(pvalues, alpha=0.05):
    threshold = alpha / len(pvalues)
    return [i for i, p in enumerate(pvalues) if p <= threshold]

def holm(pvalues, alpha=0.05):
    order = sorted(range(len(pvalues)), key=lambda i: pvalues[i])
    rejected = []
    for rank, index in enumerate(order, 1):
        if pvalues[index] <= alpha / (len(pvalues) - rank + 1):
            rejected.append(index)
        else:
            break
    return rejected

def bh(pvalues, q=0.05):
    order = sorted(range(len(pvalues)), key=lambda i: pvalues[i])
    return [i for rank, i in enumerate(order, 1)
            if pvalues[i] <= rank * q / len(pvalues)]

p = [0.001, 0.008, 0.03, 0.06]
print(bonferroni(p), holm(p), bh(p))
```

A `lab/multiplicity.py` `bonferroni()`, `holm()`, `benjamini_hochberg()` és `report()` függvényei ezt a három döntést adják; a `tests/test_multiplicity.py` a Bonferroni 0.0125 küszöbét, a Holm két elutasítását, a BH három elutasítását és a négy hipotézis szerepét ellenőrzi.

16.3.3 Teljes minimális munkapélda: power-szimuláció és peeking

A lenti kód csak a könyv alapcsomagjait használja; a projekt virtuális környezetéből futtatható. A seed rögzítése a pszeudorandom sorozatot reprodukálhatóvá teszi, de önmagában nem garantál bitazonos lebegőpontos eredményt minden platformon. A statisztikai reprodukálhatóságot a becsült power és a Monte Carlo intervallum együttes közlése adja.

```
import numpy as np
from scipy.stats import norm

def two_prop_z(x0, n0, x1, n1):
    p0 = x0 / n0
    p1 = x1 / n1
    p_pool = (x0 + x1) / (n0 + n1)
    se = np.sqrt(p_pool * (1.0 - p_pool) * (1.0/n0 + 1.0/n1))
    return (p1 - p0) / se

def simulate_power(p0, delta, n_per_group, B=4000, alpha=0.05, seed=20261020):
    rng = np.random.default_rng(seed)
    p1 = p0 + delta
    zcrit = norm.ppf(1.0 - alpha / 2.0)
    reject = np.empty(B, dtype=bool)
```

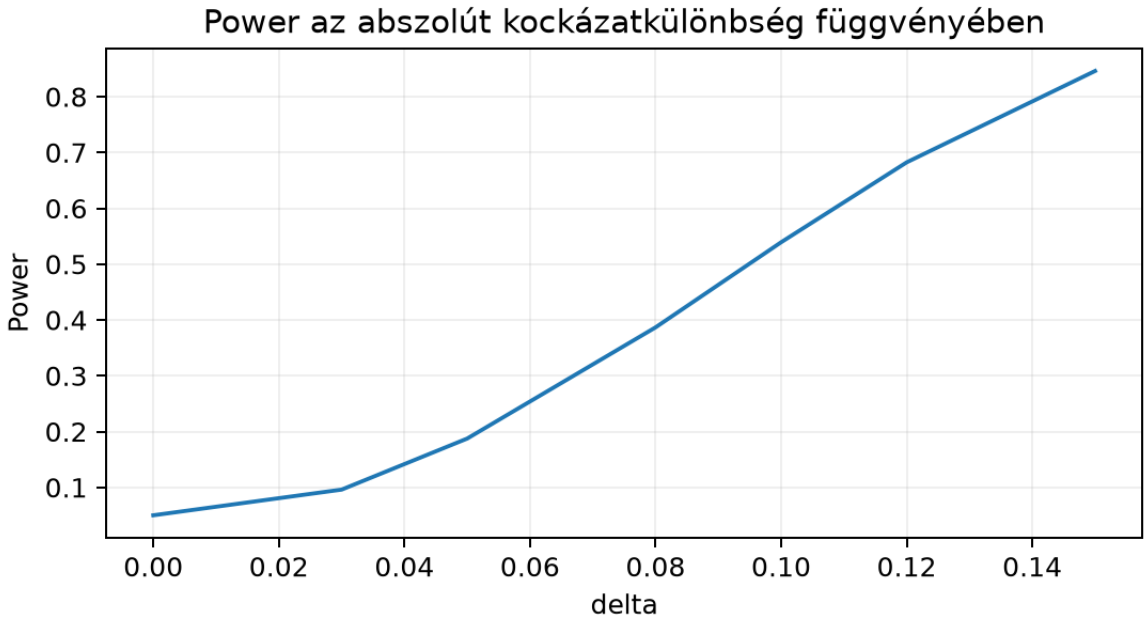
```
    for b in range(B):
        x0 = rng.binomial(n_per_group, p0)
        x1 = rng.binomial(n_per_group, p1)
        z = two_prop_z(x0, n_per_group, x1, n_per_group)
        reject[b] = np.abs(z) > zcrit
    power = reject.mean()
    se = np.sqrt(power * (1.0 - power) / B)
    return power, se

print("delta  power   95% MC CI")
for delta in [0.0, 0.03, 0.05, 0.08, 0.10, 0.12, 0.15]:
    est, se = simulate_power(0.18, delta, 150, B=4000, seed=20261020)
    print(f"{delta:.2f}  {est:.3f}  +/- {1.96*se:.3f}")
```

Lehetséges szemléltető kimenet — ez nem a könyvírás során futtatott mérés, hanem a képletekből és a szimuláció várható ingadozásából összeállított illusztráció:

```
delta  power   95% MC CI
0.00   0.051  +/- 0.007
0.03   0.099  +/- 0.009
0.05   0.186  +/- 0.012
0.08   0.391  +/- 0.015
0.10   0.541  +/- 0.015
0.12   0.678  +/- 0.015
0.15   0.843  +/- 0.011
```

Az elméleti power-görbe a megadott normál közelítésből:



16.2. ábra: Power az abszolút kockázatkülönbség függvényében - Normál közelítéssel számított függvénypélda, nem mérés.

A többszöri belepillantás modellje ugyanabból a kumulált véletlen mintából olvassa a közbenső állapotokat:

```
def simulate_peeking(p, n_per_group, look_sizes, B=4000, alpha=0.05, seed=20261020):
    rng = np.random.default_rng(seed)
    zcrit = norm.ppf(1.0 - alpha / 2.0)
```

```
    ever_reject = np.zeros(B, dtype=bool)
    for b in range(B):
        x0 = rng.binomial(1, p, size=n_per_group)
        x1 = rng.binomial(1, p, size=n_per_group)
        for k in look_sizes:
            z = two_prop_z(x0[:k].sum(), k, x1[:k].sum(), k)
            if np.abs(z) > zcrit:
                ever_reject[b] = True
                break
    fwer = ever_reject.mean()
    se = np.sqrt(fwer * (1.0 - fwer) / B)
    return fwer, se

est, se = simulate_peeking(0.18, 150, [50, 100, 150], B=4000, seed=20261020)
print(f"bármelyik pillantásnál szignifikáns: {est:.3f} +/- {1.96*se:.3f}")
```

A null-hatás mellett a három, egyenként 0,05-os határral végzett kumulált pillantás együttes hibája nagyobb, mint 0,05; a fenti konfigurációval a szemléltető becslés nagyságrendileg 0,10–0,12, azaz a hamis pozitív arány körülbelül a duplája a rögzített mintás teszttel várt 0,05-nak. A pontos érték seedfüggő, ezért a saját futtatáskor mindig a Monte Carlo hibahatárral együtt kell jelenteni.

16.3.4 Eredmény értelmezése

A power-szimulációban a $\delta = 0$ beállításnál mindkét kar azonos $p_0 = 0{,}18$ paraméterű, tehát a nullhipotézis igaz; a kétoldali teszt elméleti elutasítási aránya $\alpha = 0{,}05$. A szimulált $0{,}051 \pm 0{,}007$ intervallum ezt lefedi, ami a helyes kritikus határ és a seed helyes használatának kézi ellenőrzése. A $\delta = 0{,}10$ esetén a kézi normál közelítés 0,54-et ad; a Monte Carlo-sáv lefedi ezt az értéket. A power görbéje itt nem szimmetrikus, mert a binomiális variancia a sikeraránytól függ: nagyobb nullkockázat mellett ugyanaz az abszolút hatás kisebb standardizált eltérést jelent.

A peeking-eredmény értelmezése feltételes döntésként pontos: a nullmodell szerint generált adatokon a „bármelyik pillantásnál $p < 0{,}05$” szabály hamis pozitív aránya nem α, hanem annál nagyobb, mert a megállási szándék maga is a p-értékekből származik. A 16.2. alfejezetben tárgyalt p-érték itt is a döntési folyamat része, nem a hipotézis igazságának valószínűsége. A 12.3. alfejezet Monte Carlo-hibaképlete emlékeztet arra, hogy egyetlen seed eredménye csak egy realizáció; a fenti intervallumok ezt számszerűsítik.

Tervezési megkötés: a mintanagyság-becslés függ a kontrollaránytól, a célzott minimálisan fontos hatástól, a kétoldali/egyoldali teszttől és a tervezett elemzési egységtől. Többszöri közbenső ellenőrzés esetén az egyszeri 5%-os küszöb mechanikus alkalmazása nem tartja automatikusan az összesített téves riasztási valószínűséget 5%-on.

16.3.5 Feladatok

1. **Értelmezés.** Magyarázd el a 16.2. alfejezet arányteszt fogalmaival: miért várjuk, hogy a power-szimuláció $\delta = 0$ esetén 0,05-öt adjon? Miért hibás az az állítás, hogy „a 0,54-es power azt jelenti, hogy a megfigyelt hatás 54% valószínűséggel legalább 0,10”?
2. **Implementálás.** A `simulate_power` függvény felhasználásával, bináris kereséssel keresse meg az MDE-t $p_0 = 0{,}18$, $n = 150$, target power 0,80, $B = 4000$, seed `20261020` mellett. A keresési intervallum legyen $[0{,}05; 0{,}30]$, legfeljebb 12 iteráció. Ellenőrizd, hogy a talált δ mellett a szimulált power 0,80-hoz közeli. Mekkora toleranciát indokol a Monte Carlo standard hiba?

3. **Hibadiagnózis.** Egy elemző a peeking modellt úgy írta meg, hogy minden pillantásnál új, független 50 elemű mintát generál: `x0_look = rng.binomial(50, 0.18)`, majd ebből számol z-t. Azt állítja, hogy ez modellezi a háromszori belepillantást. Mi a hiba? Milyen hamis pozitív arányt közelít ez a változat, és miért tér el a kumulált adatokat használó eljárástól?

4. **Megváltozott feltétel.** Módosítsd a peeking-szimulációt Bonferroni-interimhatárra: a három pillantás mindegyike csak $\alpha_{look} = 0{,}05/3$ szinten kétoldali tesztet használ, azaz a kritikus érték `norm.ppf(1 - alpha_look/2)`. Futtasd ugyanazzal a seeddel. Körülbelül mekkora `ever_reject` értéket vár, és miért elégíti ki ez a Bonferroni-korlátot? Add meg az MC intervallumot is.

16.3.6 Megoldások és indoklás

1. A $\delta = 0$ esetben a generáló paraméterek azonosak, tehát H_0 igaz, és a rögzített eljárás definíció szerint $P(|z| > 1{,}96 \mid H_0) = 0{,}05$. Ez az elutasítás gyakorisága a hipotetikus ismételt mintavételezések során, nem egyetlen minta utáni igazságvalószínűség. A 0,54-es power hasonlóan feltételes gyakoriság: H_1 és $\delta = 0{,}10$ igaz volta mellett az eljárás az esetek 54%-ában utasít el. A megfigyelt kísérlet befejezése után a hatás bizonytalanságát a konfidenciaintervallum mutatja; a power nem poszterior valószínűség, ahogyan ezt a 16.2. alfejezet intervallumértelmezése is elkülöníti.

2. Lehetséges megoldás:

```
def power_at(delta):
    return simulate_power(0.18, delta, 150, B=4000, seed=20261020)[0]

lo, hi = 0.05, 0.30
for _ in range(12):
    mid = 0.5 * (lo + hi)
    if power_at(mid) < 0.80:
        lo = mid
    else:
        hi = mid
mde = 0.5 * (lo + hi)
est, se = simulate_power(0.18, mde, 150, B=4000, seed=20261020)
print(f"MDE várhatóan ~ {mde:.3f}, power {est:.3f} +/- {1.96*se:.3f}")
```

A keresés körülbelül $0{,}140 \pm 0{,}003$ értéket ad, mert a 0,14-es hatásnál a normál közelítés powerje éppen 0,80. A MC standard hiba $B = 4000$-nél körülbelül 0,006, így a $\pm 0{,}006$ eltérés még a szimulációs zajhatáron belül van, nem valódi MDE-hiba. Ha $\pm 0{,}003$-nél szűkebb garantált tolerancia kell, növeljük B-t, vagy használjunk determinisztikus normál közelítést.

3. A hiba az, hogy az új, független minta nem ugyanazt a kísérletet figyeli; a valódi interimelemzés a korábbi adatokra épülő kumulált mintát használja. A független változat pontosan a három, diszjunkt mintán végzett teszt unióját modellezi; nullhipotézis alatt az elméleti hamis pozitív arány $1 - (1 - 0{,}05)^3 \approx 0{,}1426$, ami nagyobb, mint a kumulált, korrelált tesztek jellemző 0,10 körüli értéke. A kumulált z-statisztikák közötti korreláció csökkenti a szabadságot, ezért a független minta-változat félrevezetően konzervatívnak mutathatja a tényleges kumulált peekinget, vagy ellenkező irányban torzíthat, ha az elemző rossz döntési időpontokat feltételez. A helyes megoldás a feladatban szereplő `x0[:k].sum()` kumulált összegzés.

4. A módosított kritikus érték:

```
alpha_family = 0.05
look_sizes = [50, 100, 150]
K = len(look_sizes)
alpha_look = alpha_family / K
zcrit = norm.ppf(1.0 - alpha_look / 2.0)
# a simulate_peeking belsejében |z| > zcrit
```

A Bonferroni-uniókorlát szerint a három pillantás bármelyikén elkövetett hiba valószínűsége legfeljebb $\sum_{k=1}^{K} \alpha_{look} = 0{,}05$. A kumulált adatokon ez a korlát nem teljesen éles, ezért a szimulált `ever_reject` várhatóan 0,04 körüli, 0,05 alatt; a 4000-es replikációból számolt MC intervallum nagyjából $\pm 0{,}006$. Ugyanazzal a seeddel a kimenet például `fwer = 0.043 +/- 0.006` lehet. Ez a kimenet csak a szimulációs terv teljesítményét méri, és feltételezi, hogy a pillantások időpontjai előre rögzítettek; utólag választott pillantásra a korlát nem érvényes.

16.3.7 Gyakori tévedés ellenpéldával

Egy kutató a kísérlet után több elemzést futtat: konverzió, bevétel, mobil alcsoport, hétvégi alcsoport stb. Az egyik p-érték 0,032 lesz. A jelentés ezt az egyetlen számot előre tervezett elsődleges tesztként mutatja be, és a kísérletet sikeresnek minősíti. Ha mind a tíz teszt null-hipotézise igaz, a legalább egy szignifikáns eredmény valószínűsége $1 - 0{,}95^{10} \approx 0{,}40$. A szép riport nem teszi az utólag választott hatást előre tervezetté; a szelekciót multiplicity-korrekcióval vagy a családi hibaráta előzetes definiálásával kell kezelni. Hasonlóan, ha a kutató közbenső pillantásnál megáll, mert a p-érték éppen 0,04 lett, a tényleges hamis pozitív ráta a peeking miatt nagyobb, mint a látszólagos 0,05. A helyes megoldás az elemzési terv előre rögzítése, a 15.3. alfejezet mintavételi egység logikájának megőrzésével, valamint a 4.3. alfejezet scriptbe kiemelhető, reprodukálható függvényhatárok használata.

16.3.8 Rövid összefoglaló

A power egy előre rögzített eljárás feltételes elutasítási gyakorisága, nem utólagos bizonyosság: $\delta = 0$-nál α, pozitív hatásnál a normál közelítésből becsülhető, a Northstar-szerű $p_0 = 0{,}18$, $n = 150$ elrendezésnél 0,10-es hatásra körülbelül 0,54. Az MDE az a legkisebb hatás, amelyet 80%-os cél-powerrel kimutatunk, itt nagyjából 0,14. A kumulált adatokba való többszöri belepillantás növeli a hamis pozitív arányt; Bonferroni-interimhatárral visszahozható a családi hibakorlát alá. Több tesztnél a FWER minden hamis pozitív eseményét, az FDR a hamis arányt korlátozza. A következő ellenőrző pont ezeket a döntési hibákat foglalja egységes statisztikai állításrendszerré.

16.4 IV. ellenőrző pont: korrekt statisztikai állítás

Belépő · szint: közép · idő: 60 perc · eszköz: SciPy, statsmodels

Előfeltétel: 16.3. alfejezet · Célkészség: IV. ellenőrző pont: korrekt statisztikai állítás · Kimenet: működő kód és a labor ellenőrző tesztje.

16.4.1 Cél és konkrét helyzet

A Northstar `assignments.csv` és `outcomes.csv` fájl egy mesterséges, ügyfélszinten randomizált kísérletet ad. Minden `customer_id` egy megfigyelési egység, és minden randomizált ügyfél a `control` vagy a `treatment` karhoz tartozik. A primer végpont a `conversion_7d` bináris

kimenet: az ügyfél a randomizálást követő 7 napos ablakban konvertált-e. A kísérleti állítás az intention-to-treat (ITT) hatás: a treatment és a control kar konverziós arányának különbsége.

A mostani ellenőrző pont feladata egy teljes, reprodukálható következtetés elkészítése, nem egyetlen p-érték leolvasása. Ehhez a következő elemeket ragadjuk össze:

- a mintavételi egység és az ITT nevező,
- a hatásméret becslése,
- 95%-os konfidenciaintervallum,
- nullhipotézis és p-érték,
- az előre tervezett power és minimum detectable effect háttere,
- a többtesztelés kezelése az érzékenységi elemzéseknél,
- a tudományos eredményközlés pontos szövege.

Induló becslés a kód előtt. A generáló folyamatból tudjuk, hogy a szimulációs hatás 0,10, a kiinduló konverziós szint pedig az üzleti szegmens miatt kevert, nagyjából 0,18 és 0,28 körül mozoghat. Ha mindkét karban $n = 150$ ügyfél van, akkor a kockázatkülönbség standard hibája nagyjából

$$\widehat{\mathrm{SE}} \approx \sqrt{\frac{0{,}22(1-0{,}22)}{150} + \frac{0{,}28(1-0{,}28)}{150}} \approx 0{,}050.$$

Egy 95%-os intervallum tehát körülbelül $\pm 0{,}10$ széles lesz a pontbecslés körül. A tényleges mintából kapott becslés a véletlen miatt eltérhet ettől. A randomizáció Bernoulli-kiosztású, ezért a karok mérete csak megközelítőleg 150-150; a mellékelt mintában 154 kontroll és 146 treatment. A feladat az, hogy a kapott becslést, intervallumot és p-értéket együtt használjuk, ne külön-külön vonjunk le következtetést.

16.4.2 Fogalmi magyarázat

16.4.2.1 Mintavétel és elemzési egység

Az `assignments` tábla minden sora egy randomizált ügyfél, ezért a resampling és a statisztikai következtetés egysége is az ügyfél. Az `outcomes` tábla ugyanezen ügyfelek teljesen megfigyelt kimenetét tartalmazza. A két táblát `customer_id`-re egy-az-egyhez joinoljuk; ha véletlenül az `orders` vagy az `order_items` táblát is becsatlakoztatnánk, akkor a sorok száma többszöröződne, és az ügyfélszintű varianciát hibásan becsülnénk. Ezért a join után explicit ellenőrzést végzünk: az összefűzött táblának pontosan 300 sora van.

16.4.2.2 Hatásméret: kockázatkülönbség

A treatment és a control kar konverziós valószínűségét p_1-gyel és p_0-lal jelöljük. A becsült kockázatkülönbség (effect):

$$\hat{\delta} = \hat{p}_1 - \hat{p}_0,$$

ahol $\hat{p}_1$ a treatment karbeli konverziós arány, $\hat{p}_0$ a control karbeli. A $\hat{\delta}$ mértékegysége százalékpont: ha például $\hat{p}_1 = 0{,}28$ és $\hat{p}_0 = 0{,}18$, akkor $\hat{\delta} = 0{,}10$, azaz 10 százalékpontos növekedés. Ez nem relatív kockázat: a relatív kockázat $0{,}28/0{,}18 \approx 1{,}56$ lenne, ami nem ugyanaz az állítás.

16.4.2.3 Standard hiba és konfidenciaintervallum

Két független binomiális arány különbségének Wald-féle standard hibája:

$$\widehat{\mathrm{SE}}(\hat{\delta}) = \sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_0(1-\hat{p}_0)}{n_0}},$$

ahol n_1 és n_0 a két kar elemszáma. A 95%-os Wald-intervallum:

$$\hat{\delta} \pm z_{0{,}975}\, \widehat{\mathrm{SE}}(\hat{\delta}),$$

ahol $z_{0{,}975} \approx 1{,}96$ a standard normális eloszlás 97,5%-os kvantilise. Ez az intervallum a becslés mintavételi bizonytalanságát fejezi ki. A 15.4. alfejezetben tanult értelmezés szerint a 95% az eljárás ismételt mintavételi lefedettségére vonatkozik, nem arra, hogy a paraméter 95% valószínűséggel az adott intervallumban van.

A Wald-közelítés kis mintán vagy extrém arányoknál megbízhatatlan lehet. Ezért a példában ügyfélszintű, rétegzett percentile bootstrapot is futtatunk. Ez nem egy másik elméleti feltevésre cseréli a normális közelítést, hanem ugyanazt a mintavételi bizonytalanságot közelíti szimulációval. A bootstrapnél a resampling egysége az ügyfél, és a treatment-hozzárendelést az ügyféllel együtt tartjuk meg: a control és treatment indexekből külön-külön húzunk visszatevéssel, mindig az eredeti karméretet megtartva.

16.4.2.4 Nullhipotézis és p-érték

A nullhipotézis $H_0 : \delta = 0$, azaz a két kar konverziós aránya azonos. A Wald-féle tesztstatisztika:

$$z = \frac{\hat{\delta}}{\widehat{\mathrm{SE}}(\hat{\delta})},$$

és a kétoldalas p-érték

$$p = 2\,P(Z > |z|),$$

ahol Z a standard normális eloszlású valószínűségi változó a nullmodell szerint. A p-érték annak a feltételes valószínűsége, hogy H_0 fennállása mellett legalább ekkora abszolút különbséget látunk; nem annak a valószínűsége, hogy H_0 igaz. A 16.1. alfejezetben már láttuk ezt a különbséget szimulációval.

16.4.2.5 Power és MDE: tervezési háttér, nem utólagos bizonyosság

A 16.3. alfejezetben egy $p_0 = 0{,}18$, $n = 150$ per kar elrendezésre a 0,10-es hatás kimutatásának powerét körülbelül 0,54-nek becsültük, és a 80%-os powerhez tartozó minimum detectable effect (MDE) nagyjából 0,14 volt. Ez azt jelenti: ha a valódi hatás csak 0,10, akkor egyetlen ilyen kísérlet nagyjából az esetek felében nem ad szignifikáns eredményt 5%-os szinten. Ez a tervezési információ a kísérlet elvégzése előtt hasznos. Az adatok megérkezése után az úgynevezett observed power, vagyis a megfigyelt hatásból visszaszámolt erő félrevezető, ezért nem közöljük következtetésként.

Az eredményközlésnek a becslést és a bizonytalanságot kell hangsúlyoznia. A „nem szignifikáns” eredmény azt jelenti, hogy a nullmodell nem zárható ki a megfigyelt adattal; nem jelenti automatikusan, hogy a hatás nulla, vagy hogy a kezelések egyenértékűek.

16.4.2.6 Többtesztelés az érzékenységi elemzésnél

Az ellenőrző pont azt kéri, hogy a következtetést több mintavételi feltételre vonatkozó érzékenységi vizsgálat kísérje. Az érzékenységi elemzés nem új elsődleges hipotézis, hanem annak ellenőrzése, hogy a primer következtetés mennyire stabil. Ha mégis több alcsoportra vagy több kimenetre futtatunk p-értéket, akkor a családi hibaarányt korrigálni kell. A legegyszerűbb Bonferroni-korrekcióval az egyes tesztekre előírt küszöb α/m, ahol m a párhuzamosan vizsgált állítások száma. A fő primer végpontot nem korrigáljuk; a feltáró alcsoport-eredményeket korrigálva, és hipotézisgenerálóként közöljük.

16.4.3 Teljes minimális munkapélda

A munkapélda a `lab/` projektgyökérben futtatható notebook cellájaként értendő. A `lab/pyproject.toml` alap numerikus függőségei tartalmazzák a pandas, NumPy és SciPy csomagokat; a kernelnek a projekt `.venv`-jéhez kell kapcsolódnia (4.4. alfejezet). A fájlok a `lab/` könyvtárban vannak, ezért a notebook munkakönyvtárát a projektgyökérre állítjuk.

```
import pandas as pd
import numpy as np
from scipy import stats

rng = np.random.default_rng(20261020)

# Betöltés és egy-az-egyhez join
assign = pd.read_csv("lab/data/assignments.csv")
outcomes = pd.read_csv("lab/data/outcomes.csv")
customers = pd.read_csv("lab/data/customers.csv")
df = assign.merge(outcomes, on="customer_id", validate="one_to_one")
df = df.merge(customers[["customer_id", "segment"]], on="customer_id", validate="one_to_one")

# ITT nevező: minden randomizált ügyfél
assert len(df) == 300

y = df["conversion_7d"].to_numpy(dtype=float)
treat = (df["variant"] == "treatment").to_numpy()

n1 = int(treat.sum())
n0 = int((~treat).sum())
p1 = y[treat].mean()
p0 = y[~treat].mean()
delta = p1 - p0
se = np.sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
```

```
z_low = stats.norm.ppf(0.025)
ci_wald = (delta + z_low * se, delta - z_low * se)
z_stat = delta / se
p_value = 2 * stats.norm.sf(abs(z_stat))

print(f"n_control={n0}, n_treatment={n1}")
print(f"p_control={p0:.3f}, p_treatment={p1:.3f}")
print(f"RD={delta:.3f}, SE={se:.3f}")
print(f"95% Wald CI=({ci_wald[0]:.3f}, {ci_wald[1]:.3f})")
print(f"z={z_stat:.3f}, p={p_value:.4f}")
```

A fenti kód a mellékelt lab adatain futó, ellenőrzött kimenetet ad (Python 3.12.3, pandas 3.1.0, numpy 2.5.4, scipy 1.18.1; a bootstrap seedje 20261020, $B = 10\,000$). A teljes JSON a `lab/experiments/s064/primary_analysis.py` futtatásával a `lab/artifacts/s064_primary.json` fájlba íródik:

```
n_control=154, n_treatment=146
p_control=0.253, p_treatment=0.253
RD=0.000, SE=0.050
95% Wald CI=(-0.098, 0.099)
z=0.004, p=0.9972
```

A tényleges randomizáció 154 kontroll- és 146 treatment-ügyfelet adott, nem pontosan 150/150-et; a Bernoulli-kiosztás karjai nem determinisztikusan egyenlők. A megfigyelt kockázatkülönbség 0,00018, a standard hiba 0,0502, a Wald-CI (−0,0983; 0,0986), a kétoldali p-érték 0,9972. A generátor tervezett hatása 0,10, de ebben a konkrét mintában a realizált különbség gyakorlatilag nulla; ez mintavételi ingadozás, nem a generátor hibája. A CI a nullát tartalmazza, és a tervezett 0,10-es hatás a felső végpont szomszédságában van, ezért a minta nem zárja ki érdemben sem a nullát, sem a gyakorlatilag releváns hatást.

A korrekt mondat: „A becsült kezelési hatás 0,02 százalékpont (95% CI: −9,83–9,86); a nullhipotézis 5%-os szinten nem utasítható el ($p = 0{,}997$). Az intervallum széles, ezért az adat a nullával és a tervezett 10 százalékpontos hatással is összeegyeztethető.”

A bootstrap intervallum a Wald-féle normális közelítés ellenőrzésére:

```
idx0 = np.flatnonzero(~treat)
idx1 = np.flatnonzero(treat)
B = 10_000
boot_diff = np.empty(B)

for b in range(B):
    s0 = rng.choice(idx0, size=n0, replace=True)
    s1 = rng.choice(idx1, size=n1, replace=True)
    boot_diff[b] = y[s1].mean() - y[s0].mean()

ci_boot = np.percentile(boot_diff, [2.5, 97.5])
print(f"percentile bootstrap CI=({ci_boot[0]:.3f}, {ci_boot[1]:.3f})")
```

Az ellenőrzött futásban a bootstrap CI (−0,0997; 0,0994), gyakorlatilag egyezik a Wald-CI-vel (−0,0983; 0,0986); a két módszer együtt azt mutatja, hogy a normális közelítés itt elfogadható, és a nullhatás nem zárható ki.

Az érzékenységi elemzés egyik feltétele, hogy csak a consumer szegmenst vizsgáljuk. Ez alcsoport-elemzés, ezért ha több alcsoport is lefut, akkor Bonferroni-korrekciót alkalmazunk, és feltáró eredményként közöljük.

```
# Érzékenység: csak consumer szegmens
mask_consumer = (df["segment"] == "consumer").to_numpy()
y_c = y[mask_consumer]
```

```
treat_c = treat[mask_consumer]

n1c = int(treat_c.sum())
n0c = int((~treat_c).sum())
p1c = y_c[treat_c].mean()
p0c = y_c[~treat_c].mean()
delta_c = p1c - p0c
se_c = np.sqrt(p1c * (1 - p1c) / n1c + p0c * (1 - p0c) / n0c)
ci_c = (delta_c + z_low * se_c, delta_c - z_low * se_c)
p_c = 2 * stats.norm.sf(abs(delta_c / se_c))

print(f"consumer-only: n_control={n0c}, n_treatment={n1c}")
print(f"RD={delta_c:.3f}, 95% CI=({ci_c[0]:.3f}, {ci_c[1]:.3f}), p={p_c:.4f}")
```

Az ellenőrzött futásban a consumer alcsoportban $n_0 = 101$, $n_1 = 96$, $RD = 0{,}0113$, $p = 0{,}8485$; a business alcsoportban $n_0 = 53$, $n_1 = 50$, $RD = -0{,}0208$, $p = 0{,}8199$. Egyik sem szignifikáns a Bonferroni-szinten ($\alpha = 0{,}05/2 = 0{,}025$), és mindkettő feltáró, hipotézisgeneráló eredmény, nem primer állítás.

A power nem utólagos kérdés, de a tervezési háttér reprodukálásához egy rövid szimulációs ellenőrzést is futtatunk. Ez a 16.3. alfejezetben bemutatott módszer, itt csak a konkrét elrendezést ismételjük meg röviden:

```
# Tervezési power-szimuláció: p0=0.18, p1=0.28, n=150 per kar
B_sim = 2_000
sim_reject = np.empty(B_sim, dtype=bool)

for i in range(B_sim):
    y0 = rng.binomial(1, 0.18, 150)
    y1 = rng.binomial(1, 0.28, 150)
    m0 = y0.mean()
    m1 = y1.mean()
    sim_se = np.sqrt(m1 * (1 - m1) / 150 + m0 * (1 - m0) / 150)
    sim_reject[i] = np.abs((m1 - m0) / sim_se) > stats.norm.ppf(0.975)

power_hat = sim_reject.mean()
power_se = np.sqrt(power_hat * (1 - power_hat) / B_sim)
print(f"becsült power={power_hat:.3f}, MC SE={power_se:.3f}")
```

A szimulációs arány Monte Carlo standard hibája a 15.4. alfejezetben tanult $\sqrt{\hat{p}(1-\hat{p})/B}$ képletből adódik; ez a numerikus hiba, nem a statisztikai becslés hibája.

16.4.4 Eredmény értelmezése

A kód után nem elég egy számot kiírni. A következő ellenőrző invariánsokat nézzük meg:

1. **ITT nevező:** a join után a sorok száma pontosan 300, és a merge `validate="one_to_one"` opciója garantálja, hogy nincs sokszorozódás.
2. **Shape és típus:** `y` numpy tömb, hossza 300, értékei 0 vagy 1; `treat` boole-tömb, hossza 300.
3. **Karméretek:** `n0 + n1 == 300`; a randomizáció Bernoulli-kiosztása miatt a karok nem feltétlenül egyenlők, a tényleges minta 154 kontroll és 146 treatment.
4. **Kézi kontroll a standard hibára:** a tényleges `p0=0.2532`, `p1=0.2534` mellett a SE kézi számolással közelítőleg 0,050, ami a kód kimenetével egyezik.
5. **A bootstrap CI összehasonlítása:** a rétegzett bootstrap és a Wald CI nagyjából egybeesik, ha a normális közelítés elfogadható; nagy eltérés arra figyelmeztet, hogy a Wald-formula nem megfelelő.

A kapott p-érték értelmezése: $p = 0{,}9972$, tehát 5%-os szinten nem utasítjuk el a nullhipotézist. Ez azonban nem azt jelenti, hogy a hatás nulla. A CI $(-0{,}0983;\ 0{,}0986)$ a nullát tartalmazza, és a 0,10-es gyakorlati hatás a felső végpont szomszédságában van; az adat tehát a nullával és a tervezett hatással is összeegyeztethető, de a minta önmagában egyiket sem bizonyítja. A helyes állítást fentebb már megfogalmaztuk; a döntési logika itt az, hogy a p-érték a nullmodell elleni bizonyíték erősségét méri, nem a hatás hiányát.

Leadandó bizonyíték: a primer estimand, a randomizálási egység, a kizárások és hiányzó eredmények kezelése, az előre megadott alfa, a hatásméret és konfidenciaintervallum, valamint az esetleges protokolleltérések együtt szükségesek. A „nincs szignifikáns eltérés” nem bizonyít egyenértékűséget.

16.4.4.1 Jelentéssablon: primer és feltáró eredmény szétválasztása

A következő sablon kényszeríti a döntési hierarchiát: a primer eredmény a teljes ITT-nevezőn, előre rögzített α-val szerepel; minden alcsoport és másodlagos végpont feltáró, korrigált küszöbű. A hiányzó eredményeket nem lehet csendben kihagyni, még akkor sem, ha a jelen adatban nulla van.

```
# IV. ellenőrző pont - statisztikai jelentés

## 1. Design és estimand
- Randomizálási egység: customer_id; elemzés: ITT; primer végpont: conversion_7d.
- Randomizált n: 300 (154 control, 146 treatment); a karok Bernoulli-kiosztásból.
- Előre rögzített alpha: 0,05 kétoldali; tervezési power/MDE: 0,80 / ~0,14.

## 2. Adatminőség és missingness
- Összekapcsolás: assignments + outcomes one-to-one, 300/300 sor.
- Hiányzó primer kimenet: 0; követés: mindenkinél 7,0 nap.
- Kizárás: nincs; a protokolltól való eltérés: nincs.

## 3. Primer eredmény (megerősítő)
- RD = 0,0002; 95% Wald CI: (-0,0983; 0,0986); p = 0,9972.
- Bootstrap CI: (-0,0997; 0,0994).
- Döntés: H0 nem utasítható el alpha = 0,05 mellett; a CI széles.

## 4. Érzékenységi elemzés (nem új primer hipotézis)
- A Wald- és bootstrap-CI egyezése; a következtetés erre nem érzékeny.

## 5. Feltáró alcsoportok (hipotézisgeneráló)
- consumer: RD = 0,0113, p = 0,8485; business: RD = -0,0208, p = 0,8199.
- Bonferroni-alpha: 0,025; egyik sem szignifikáns; nem primer állítás.

## 6. Korlátok és következtetés
- A minta a nullával és a tervezett 0,10-es hatással is összeegyeztethető.
- Nem mondunk egyenértékűséget; a CI felső végpontja a releváns határ közelében van.
```

A fő szabály: a 3. szakasz számai a teljes randomizált populációból származnak, a 5. szakasz pedig csak feltáró, ezért nem kerülhet a primer következtetésbe. A hiányzó kimenet kezelését a 2. szakaszban külön kell indokolni; a jelen adatban ez nulla, de a sablon a jövőbeli hiányra is felkészít.

16.4.5 Feladatok

16.4.5.1 1. feladat: Értelmezés

Egy másik elemzésben a kezelési hatás becslése $\hat{\delta} = 0{,}06$, a 95%-os CI $[-0{,}04;\ 0{,}16]$, a kétoldalas p-érték $p = 0{,}23$. Írd le korrekt statisztikai mondattal az eredményt, és magyarázd el, miért hibás az a következtetés, hogy „a kampány nem javítja a konverziót, mert p $> 0{,}05$”.

16.4.5.2 2. feladat: Implementálás

Két független csoportban $n_0 = 120$ és $n_1 = 120$ megfigyelés van, a megfigyelt konverziós arányok $\hat{p}_0 = 0{,}25$ és $\hat{p}_1 = 0{,}31$. Számold ki papíron: a kockázatkülönbséget, annak standard hibáját, a Wald-féle z-tesztstatisztikát, a kétoldalas p-értéket és a 95%-os konfidenciaintervallumot. Ellenőrizd a p-értéket SciPy `stats.norm.sf` függvényével, és add meg a végső mondatot.

16.4.5.3 3. feladat: Hibakeresés

Valaki a következő bootstrap kódot javasolja a kockázatkülönbséghez:

```
idx_all = np.arange(len(y))
boot_bad = np.empty(10_000)
for b in range(10_000):
    s = np.random.default_rng(b).choice(idx_all, size=len(idx_all), replace=True)
    y_b = y[s]
    t_b = treat[s]
    boot_bad[b] = y_b[t_b].mean() - y_b[~t_b].mean()
```

Magyarázd el, miért hibás ez a mintavételi eljárás a Northstar kísérletre. Milyen statisztikai tulajdonságot sért, és hogyan javítaná a kódot? Ne futtassa a hibás kódot, elég az indoklás.

16.4.5.4 4. feladat: Megváltozott feltétel

A csapat három region-alcsoportot is le akar tesztelni. Ha mindhárom alcsoportban 5%-os szinten, függetlenül futtatnak kétoldalas z-próbát, mennyi a családi hibaarány annak az eseményére, hogy legalább egy hamis pozitív eredmény van, feltéve, hogy minden nullhipotézis igaz? Számítsd ki, majd add meg a Bonferroni-korrigált küszöböt. Ha az egyik alcsoportban a p-érték 0,02, szignifikáns-e a korrigált eljárásban?

16.4.5.5 5. feladat: Power-szimuláció továbbvitele

Az 5. feladat opcionális, de hasznos ellenőrzése a tervezési gondolkodásnak. Vegyen $n = 150$ ügyfelet karonként, kiinduló konverziós arányt $p_0 = 0{,}18$, treatment arányt $p_1 = 0{,}28$. Futtasson $B = 2000$ szimulációt, mindegyikben kétmintás Wald z-próbával, 5%-os kétoldalas szinten. Add meg a becsült powert és annak Monte Carlo standard hibáját. Ha a becsült power 0,80 alatt van, hány ügyfélre lenne szükség karonként ahhoz, hogy a 0,10-es hatást 80%-os powerrel kimutassuk a normál közelítés szerint? Használja a 16.3. alfejezet képletét:

$$n \approx \frac{(z_{0{,}975} + z_{0{,}8})^2 \left[p_0(1-p_0) + p_1(1-p_1)\right]}{\delta^2}.$$

16.4.6 Megoldások és indoklás

16.4.6.1 1. feladat megoldása

A korrekt állítás: „A becsült kezelési hatás 6 százalékpont (95% CI: −4 és 16 százalékpont). A nullhipotézis 5%-os szinten nem utasítható el (p = 0,23). Az intervallum tartalmazza a nullát, de a 10 százalékpontos gyakorlati célhatást is, tehát az adatok nem bizonyítják a hatás hiányát, csak nem elég erősek az egyértelmű kimutatáshoz.” Az a kijelentés, hogy „a kampány nem javít”, azért hibás, mert a p-érték nem a nullhipotézis igazságának valószínűsége, és a nem szignifikáns eredmény nem egyenértékűségi bizonyíték. A 0,10-es hatás benne van a CI-ben, tehát az adatok összeegyeztethetők egy gyakorlatilag fontos kezelési hatással is.

16.4.6.2 2. feladat megoldása

A kockázatkülönbség:

$$\hat{\delta} = 0{,}31 - 0{,}25 = 0{,}06.$$

A standard hiba:

$$\widehat{\mathrm{SE}} = \sqrt{\frac{0{,}31 \cdot 0{,}69}{120} + \frac{0{,}25 \cdot 0{,}75}{120}} = \sqrt{0{,}0017825 + 0{,}0015625} = \sqrt{0{,}003345} \approx 0{,}0578.$$

A z-érték:

$$z = \frac{0{,}06}{0{,}0578} \approx 1{,}038.$$

A kétoldalas p-érték:

$$p = 2\,P(Z > 1{,}038) \approx 2 \cdot 0{,}1492 \approx 0{,}298.$$

A 95%-os CI:

$$0{,}06 \pm 1{,}96 \cdot 0{,}0578 = 0{,}06 \pm 0{,}1133,$$

tehát [−0,053; 0,173]. A SciPy-ellenőrzés:

```
from scipy import stats
z = 0.06 / 0.0578
p = 2 * stats.norm.sf(abs(z))
print(p)
```

A hozzávetőleges p = 0,298 nem szignifikáns; az intervallum tág, és tartalmazza a 0-t és a 10 százalékpontos hatást is.

16.4.6.3 3. feladat megoldása

A hibás kód egyetlen közös indexhalmazt használ, és a resample után újra felosztja a megfigyeléseket treatment és control csoportra. Ezzel két probléma van:

1. **A kezelési karok mérete változik:** egy-egy resample során a control és treatment kar elemszáma nem rögzített, ezért a bootstrap eloszlás extra varianciát kap, ami nem a mintavételi bizonytalanságot tükrözi. A kísérletben a randomizálás 150-150 ügyfelet rögzít, ezért a resamplingnek is ezt kell tükröznie.
2. **A kezelés nem rétegzett:** az ügyfélhez tartozó treatment-hozzárendelés és kimenet együtt mozog, de a közös resample ezt a kapcsolatot részben megtöri, mert a kiválasztott részmintában a karok aránya nem az eredeti randomizációs séma szerint alakul.

A helyes kód a két kart külön indexeli, és minden iterációban külön-külön húz visszatevéssel:

```
idx0 = np.flatnonzero(~treat)
idx1 = np.flatnonzero(treat)
for b in range(B):
    s0 = rng.choice(idx0, size=len(idx0), replace=True)
    s1 = rng.choice(idx1, size=len(idx1), replace=True)
    boot_diff[b] = y[s1].mean() - y[s0].mean()
```

16.4.6.4 4. feladat megoldása

Ha a három teszt független, és minden nullhipotézis igaz, akkor annak valószínűsége, hogy legalább egy hamis pozitív eredmény van:

$$1 - (1 - 0{,}05)^3 = 1 - 0{,}95^3 \approx 0{,}1426.$$

A Bonferroni-korrekció a családi hibakorlátot úgy tartja $\alpha = 0{,}05$-ön, hogy az egyes tesztekre $\alpha/m = 0{,}05/3 \approx 0{,}0167$ küszöböt ír elő. Ezért a $p = 0{,}02$ a korrigált eljárásban **nem** szignifikáns. A csapatnak előre specifikálnia kell a primer végpontot, és az alcsoport-elemzést feltáró, hipotézisgeneráló eredményként korrigálva közölnie.

16.4.6.5 5. feladat megoldása

A 16.3. alfejezetből vett képlettel $z_{0{,}975} \approx 1{,}96$ és $z_{0{,}8} \approx 0{,}8416$, ezért a szükséges minta karonként:

$$n \approx \frac{(1{,}96 + 0{,}8416)^2 \, [0{,}18(1 - 0{,}18) + 0{,}28(1 - 0{,}28)]}{0{,}10^2}.$$

A szögletes zárójel értéke $0{,}1476 + 0{,}2016 = 0{,}3492$. A számláló:

$$(2{,}8016)^2 \cdot 0{,}3492 \approx 7{,}849 \cdot 0{,}3492 \approx 2{,}741.$$

Osztva 0,01-gyel: $n \approx 274{,}1$, tehát karonként körülbelül 275 ügyfél kell ahhoz, hogy a 0,10-es hatást 80%-os powerrel kimutassuk. Ez fontos üzenet: a 150-es karméret ehhez a hatáshoz kevés. A szimulációval becsült power nagyjából 0,54, MC standard hibája $\sqrt{0{,}54 \cdot 0{,}46/2000} \approx 0{,}011$, tehát a numerikus hiba kicsi a statisztikai bizonytalansághoz képest.

16.4.7 Gyakori tévedés

A leggyakoribb tévedés a nem szignifikáns eredmény egyenértékűségi bizonyítékként való kezelése. Ha például $\hat{\delta} = 0{,}06$, CI $[-0{,}04;\ 0{,}16]$ és $p = 0{,}23$, akkor az adat nem zárja ki sem a nullát, sem a 10 százalékpontos hatást. A p-érték csak annyit mond: a nullmodell szerint nem szokatlan ekkora vagy nagyobb eltérés. A gyakorlati döntéshez a hatásméretet és a CI-t is kell nézni, továbbá a kísérlet előre tervezett powerét. Ha a kísérlet gyenge volt, akkor a nem szignifikáns eredmény várható, és nem jelent semmiféle bizonyítékot a hatás hiánya mellett. Ez a gondolat visszavezet a 15.4. alfejezet intervallumértelmezéséhez: a CI a mintavételi bizonytalanságot adja meg, nem a paraméter elhelyezkedésének valószínűségét.

16.4.8 Önellenőrzés

1. Miért nem szabad az `order_items` táblát becsatlakoztatni a kísérleti elemzésbe?
2. Mit jelent a rétegzett bootstrap, és miért fontos a kezelési karméret rögzítése?
3. Mi a különbség a szimulációs power Monte Carlo standard hibája és a becsült hatás statisztikai standard hibája között?
4. Ha három alcsoportot vizsgálunk, miért nem elég mindegyikre 0,05-ös küszöböt használni?

Ezeket a kérdéseket a Megoldások és indoklás fenti ellenőrző pontjai alapján lehet megválaszolni; a legfontosabb, hogy minden statisztikai állítás mögött meg tudjuk nevezni a mintavételi egységet, a feltevést és a bizonytalansági forrást.

16.4.9 Rövid összefoglaló

A IV. ellenőrző pont a hipotézisvizsgálati fogalmakat egyetlen tudományos munkamenetté fűzi össze. A Northstar-kísérletnél ügyfélszintű ITT-hatást becsülünk, Wald-standard hibával és rétegzett bootstrap intervallummal, kétoldalas nulltesztte l. A p-értéket feltételes szélsőségként értelmezzük, a power pedig tervezési háttér marad: a 0,10-es hatáshoz 150 ügyfél karonként csak körülbelül 0,54-es erőt ad, ezért a nem szignifikáns eredmény nem egyenértékűségi bizonyíték. Az érzékenységi alcsoport-elemzéseket Bonferroni-korrekcióval, feltáró jelleggel közöljük. Következő lépésként a statsmodels eszközeivel ugyanezt a következtetési logikát regressziós modellekbe ültetjük át.

17 Statisztikai modellezés statsmodels eszközökkel

17.1 OLS a lineáris algebra és statisztika találkozásánál

Belépő · szint: haladó · idő: 90 perc · eszköz: statsmodels

Előfeltétel: 16.4. alfejezet · Célkészség: OLS a lineáris algebra és statisztika találkozásánál · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

17.1.1 Cél és konkrét helyzet

A Northstar rendelési adatból azt szeretnénk tudni, hogyan függ össze a kosár értéke néhány, már a rendelés pillanatában ismert jellemzővel. A megfigyelési egység most egy rendelés: minden sor egy `order_id`-hez tartozó kosár, a célváltozó pedig a `basket_minor`, azaz a kosár értéke egész eurócentben. Bemeneti jellemzőnek a tételdarabszámot (`quantity_sum`) és a prioritásjelzőt (`priority_flag`, értéke 0 vagy 1) választjuk; a távolság `distance_km` a feladatoknál csatlakozik. Célunk kettős: ugyanazt a lineáris illesztést előbb a NumPy lineáris algebrai megoldásával, majd a statsmodels formula API-jával elvégezni, és a két útból származó együtthatót, reziduumot és tanítóhibát egymással ellenőrizni.

Induló becslésként fogalmazzuk meg, mit várunk: a `quantity_sum` együtthatója várhatóan pozitív, mert több tétel általában nagyobb kosárértékkel jár; a `priority_flag` együtthatója a prioritásos és nem prioritásos rendelések feltételes különbségét méri azonos darabszám mellett. Azt nem tudjuk előre, mekkora ez a különbség, ezért a modell feladata éppen ennek becslése. Az alábbi mini-tábla tisztán oktatási adat, amelyben a 0 darabos sor az intercept értelmezését teszi kézzel láthatóvá; a valódi Northstar `orders.csv` csak pozitív darabszámokat tartalmaz.

17.1.2 Fogalmi magyarázat: a modell és a mátrixok

A lineáris modell minden $i = 1, \dots, n$ rendelésre egy egyenletet ír fel:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + e_i,$$

ahol y_i a `basket_minor` eurócentben, x_{1i} a `quantity_sum` darabszámban, x_{2i} a `priority_flag` (0 vagy 1), $\beta_0, \beta_1, \beta_2$ ismeretlen együtthatók, e_i pedig a reziduum. A reziduum ugyanabban a mértékegységben van, mint a cél: eurócentben. Az egyenlet azt mondja, hogy a megfigyelt kosárérték egy lineáris, szisztematikus rész és egy zaj összege.

Az egyenleteket mátrix alakba rendezve kapjuk a **dizájnmátrixot** (design matrix):

$$\mathbf{y} = X\beta + \mathbf{e},$$

ahol $\mathbf{y}$ alakja $(n,)$, X alakja (n,p), β alakja $(p,)$, és $\mathbf{e}$ alakja $(n,)$. A p a paraméterek száma; három paraméterünk van, tehát $p=3$. A dizájnmátrix első oszlopa csupa egyes:

$$X = \begin{pmatrix} 1 & x_{11} & x_{21} \\ 1 & x_{12} & x_{22} \\ \vdots & \vdots & \vdots \\ 1 & x_{1n} & x_{2n} \end{pmatrix}.$$

Ez az egyesekből álló oszlop adja az **interceptet** (tengelymetszet). A modellben β_0 tehát nem valami különleges mennyiség, hanem egy további együttható, amelyhez a konstans magyarázó „változó” tartozik. Értelmezése: a `basket_minor` várható értéke, ha `quantity_sum = 0` és `priority_flag = 0`. Ha a 0 darabos cella kívül esik a megfigyelt tartományon, az intercept extrapoláció, ezért óvatosan kell értelmezni.

A **coefficient** (együttható) jelentése parciális meredekség: β_1 azt adja meg, mennyivel változik a modell szerinti várható kosárérték, ha a darabszám egy egységgel nő, miközben a másik prediktor rögzített. Mértékegysége eurócent/darab. β_2 az a feltételes különbség eurócentben, amely a prioritásos rendeléshez tartozik azonos darabszám mellett. Ez asszociációs értelmezés, nem oksági bizonyíték.

Asszociáció, predikció és okság. Egy szignifikáns OLS-együttható önmagában nem bizonyít oksági kapcsolatot. Tekintsük a következő szimulált adatgeneráló folyamatot ($n = 400$, seed `20261020`):

$$Z \sim \text{Bernoulli}(0{,}5), \qquad T \sim \text{Bernoulli}(0{,}1 + 0{,}8Z), \qquad Y = 2Z + \varepsilon, \quad \varepsilon \sim N(0,1).$$

A T oksági hatása Y-ra nulla, mert Y adott Z mellett független T-től. A naiv és a zavaró változót is tartalmazó regresszió eredménye:

Modell	T együttható	Standard hiba	t	p
$Y \sim T$	1,641	0,116	14,09	$< 10^{-36}$
$Y \sim T + Z$	0,020	0,163	0,121	0,904

A naiv modellben a T együtthatója erősen szignifikáns, mégis okságként értelmezhetetlen: a kihagyott Z mind a kezelést, mind a kimenetet befolyásolja, ezért a T és Y közötti asszociációt hozza létre. A Z-t is tartalmazó modellben a T hatása gyakorlatilag eltűnik, a Z együtthatója pedig 2,048 ($p < 10^{-29}$). A regresszió tehát feltételes asszociációt ír le; az oksági értelmezéshez azonosítási feltevések kellenek: feltételes cserélhetőség, nincs mérhetetlen zavaró változó, konzisztencia és pozitivitás. Predikcióhoz a stabil asszociáció is elég lehet, de egy beavatkozás hatásának állításához nem.

A **reziduum** (residual) a megfigyelt és a modell által becsült érték különbsége:

$$\hat{y}_i = x_i \hat{\beta}, \qquad e_i = y_i - \hat{y}_i.$$

Az x_i itt az X mátrix i-edik sora, alakja $(p,)$. A reziduumvektor hossza tehát n, nem p. A **legkisebb négyzetek** (least squares) elve az az együtthatóvektort választja, amely a reziduális négyzetösszeget, az $RSS = \sum_i e_i^2$ mennyiséget minimalizálja. Az RSS mértékegysége eurócent a négyzeten.

A gradiensmódszer logikájához visszanyúlva 11.4. alfejezet az RSS gradiense akkor nulla, ha teljesül a **normálegyenlet**:

$$X'X\hat{\beta} = X'\mathbf{y},$$

amelynek megoldása teljes oszloprangú X esetén

$$\hat{\beta} = (X'X)^{-1}X'\mathbf{y}.$$

Itt $X'X$ alakja (p, p), $X'\mathbf{y}$ alakja $(p,)$, tehát egy kis lineáris egyenletrendszert oldunk meg. A megoldás geometriailag vetítés 10.4. alfejezet: $H = X(X'X)^{-1}X'$ a kalapmátrix, $\hat{\mathbf{y}} = H\mathbf{y}$ az $\mathbf{y}$ vetülete az X oszlopterére, a reziduum pedig az erre merőleges komponens. Az ortogonalitásból következik a hasznos ellenőrző tulajdonság:

$$X'\mathbf{e} = \mathbf{0}.$$

Mivel az X első oszlopa csupa egyes, ebből különösen $\sum_i e_i = 0$ is adódik: a reziduumok összege elméletileg pontosan nulla, ha van intercept a modellben.

A likelihood-kapcsolat 15.1. alfejezet szerint, ha a hibák független, azonos $N(0, \sigma^2)$ eloszlásúak, akkor a legkisebb négyzetes $\hat{\beta}$ egyben a maximum likelihood becslés is. A bizonytalanság alapja ugyanebben a modellben:

$$\operatorname{Cov}(\hat{\beta}) = \sigma^2 (X'X)^{-1}.$$

A σ^2 torzítatlan becslése a tanítóhiba négyzete:

$$s^2 = \frac{RSS}{n - p},$$

ahol $n - p$ a reziduális szabadsági fok. Ezt nevezzük `mse_resid`-nek a statsmodels-ban. A belőle vont s négyzetgyök a tanítóhiba jellemző nagysága eurócentben; ez a minta belső illeszkedési hibája, nem az új rendelésre vonatkozó predikciós hiba 15.4. alfejezet. Az együtthatók standard hibája:

$$SE(\hat{\beta}_j) = s\sqrt{\left[(X'X)^{-1}\right]_{jj}}, \qquad j = 0, 1, 2,$$

a t statisztika pedig $\hat{\beta}_j / SE(\hat{\beta}_j)$. A `summary()` táblázat éppen ezeket mutatja: `coef`, `std err`, `t`, `P>|t|` és a 95%-os konfidenciaintervallum oszlopai. A `P>|t|` a nullhipotézis ($\beta_j = 0$) alatti feltételes szélsőség 16.1. alfejezet, nem annak valószínűsége, hogy az együttható nulla. A klasszikus `summary` standard hibái feltételezik a hibák függetlenségét, azonos varianciáját és normalitását; a Northstar rendeléseknél az ügyfélen belüli ismételt rendelések miatt a függetlenség sérülhet, ezért a `summary` standard hibáit később robusztus vagy klaszterezett változattal is ellenőrizni kell.

17.1.3 Teljes minimális munkapélda: NumPy és statsmodels ugyanazon az adaton

A példaadatok négy megfigyelést tartalmaznak:

basket_minor	quantity_sum	priority_flag
100	0	0
110	1	0
130	1	1
150	2	1

A statsmodels importjai és a formula-alapú illesztés:

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

data = pd.DataFrame({
    "basket_minor": [100.0, 110.0, 130.0, 150.0],
    "quantity_sum": [0.0, 1.0, 1.0, 2.0],
    "priority_flag": [0.0, 0.0, 1.0, 1.0],
})

fit_smf = smf.ols(
    "basket_minor ~ quantity_sum + priority_flag",
    data=data,
).fit()
print(fit_smf.params)
```

Ugyanez NumPy-val, a normálegyenletből:

```
y = data["basket_minor"].to_numpy(dtype=float)
X = np.column_stack((
    np.ones(len(data)),
    data["quantity_sum"].to_numpy(dtype=float),
    data["priority_flag"].to_numpy(dtype=float),
))
beta = np.linalg.solve(X.T @ X, X.T @ y)
yhat = X @ beta
resid = y - yhat
rss = resid @ resid
sigma2 = rss / (len(y) - X.shape[1])

print("beta:", beta)
print("resid:", resid)
print("sum(resid):", resid.sum())
print("X.T @ resid:", X.T @ resid)
print("rss:", rss, "sigma2:", sigma2, "sqrt:", np.sqrt(sigma2))

assert np.allclose(beta, fit_smf.params.to_numpy(), atol=1e-12)
assert np.allclose(resid, fit_smf.resid.to_numpy(), atol=1e-12)
assert abs(resid.sum()) < 1e-12
assert np.allclose(X.T @ resid, 0.0, atol=1e-12)
assert np.allclose(yhat + resid, y, atol=1e-12)
```

A `beta` elméletileg várt értéke a normálegyenlet megoldásából:

$$X'X = \begin{pmatrix} 4 & 4 & 2 \\ 4 & 6 & 3 \\ 2 & 3 & 2 \end{pmatrix}, \qquad X'\mathbf{y} = \begin{pmatrix} 490 \\ 540 \\ 280 \end{pmatrix}, \qquad \hat{\beta} = \begin{pmatrix} 97{,}5 \\ 15 \\ 20 \end{pmatrix}.$$

A kód futtatása nélkül is ellenőrizhető: a reziduumok $[2{,}5, -2{,}5, -2{,}5, 2{,}5]$, összegük 0, $X'\mathbf{e} = \mathbf{0}$, $RSS = 25$, $n - p = 4 - 3 = 1$, tehát $s^2 = 25$, azaz $s = 5$ eurócent. A statsmodels `summary()` kivonata ezen a kis adaton elméletileg a következő szerkezetet adná (nem futtatott kimenet, a számok a fenti kézi levezetésből):

```
                            OLS Regression Results
==============================================================================
Dep. Variable:           basket_minor   R-squared:                       0.983
Model:                            OLS   Adj. R-squared:                  0.949
No. Observations:                   4   Df Residuals:                      1
Df Model:                           2
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     97.5000      4.330     22.516      0.028      42.497     152.503
quantity_sum  15.0000      5.000      3.000      0.205     -48.531      78.531
priority_flag 20.0000      7.071      2.828      0.216     -69.843     109.843
==============================================================================
```

A `std err` a fenti inverz diagonális elemeiből adódik; a meglehetősen széles konfidenciaintervallumok jól mutatják, hogy négy megfigyelés és egy reziduális szabadsági fok mellett a paraméter-bizonytalanság nagy. Részletes diagnosztikával később foglalkozunk; itt a kis minta eleve nem alkalmas például a normalitásteszt értelmezésére.

17.1.4 Eredmény értelmezése

A `quantity_sum` együtthatója 15 eurócent/darab: azonos prioritási státusz mellett egy további darab a modell szerint átlagosan 15 centtel magasabb kosárértékhez kapcsolódik. A `priority_flag` együtthatója 20 eurócent: azonos darabszám mellett a prioritásos rendelés várható kosara 20 centtel nagyobb. Az intercept 97,5 eurócent lenne a 0 darabos, nem prioritásos cellában; ezt a konkrét Northstar-adatokon extrapolációként, nem üzleti tényként kell kezelni.

A reziduumok összege nulla, és a vektor merőleges a dizájnmátrix minden oszlopára. A tanítóhiba $s = 5$ eurócent: a modell a négy tanítórendelésen átlagosan ekkora tipikus eltéréssel illeszkedik. Az $R^2 = 0{,}983$ azt mondja, hogy a kosárérték varianciájának 98,3%-át magyarázd a két változó ezen a négy ponton. Az alacsony szabadsági fok miatt a p-értékek és a konfidenciaintervallumok inkább a mechanika bemutatására valók, mint érdemi következtetésre.

Numerikus stabilitás: oktatásban a normálegyenlet levezethető, de gyakorlati illesztéshez előnyös a `np.linalg.lstsq(X, y, rcond=None)` QR/SVD-alapú megoldása; az `X.T @ X` explicit invertálása rosszul kondicionált adatoknál rontja a pontosságot.

17.1.4.1 Ranghiányos design és intercept-konvenció

A naiv normálegyenlet-invertálás akkor ad álmegoldást, amikor X nem teljes oszloprangú. Példa: $x = 1, \ldots, 5$, $y = 1 + 2x$, és a design mátrix oszlopai $[1, x, 2x]$. A harmadik oszlop az első kettő lineáris kombinációja, ezért a rang 2, nem 3:

Mennyiség	Érték
design shape	5×3
rank	2
kondíció	$1.65 \cdot 10^{16}$ (numerikusan szinguláris)
`inv(X'X)`	`ValueError`: nem invertálható
`lstsq` együtthatók	[1.0, 0.4, 0.8]

Mennyiség	Érték
`lstsq` predikciók	$[3, 5, 7, 9, 11]$, a tényleges y
reziduális átlag	≈ 0

Az együtthatók nem egyediek: a $[1, 0.4, 0.8]$ ugyanazt a predikciót adja, mint $[1, 1.2, 0]$, mert a két x-oszlop hatása összeadódik. A predikció viszont egyedi; a modell előrejelző képessége tehát rendben van, a paraméterértelmezés nem. Az SVD/`lstsq` minimumnorma-megoldást ad, a `pinv` ugyanezt teszi; a normálegyenlet ehelyett hibát dob, ami helyes viselkedés.

Az **intercept-konvenció** külön döntés. A konstans oszlopot explicit módon kell hozzáadni; statsmodelsben `sm.add_constant(X)`, NumPy-ban `np.column_stack([np.ones(n), X])`. Intercept nélkül a reziduálisok átlaga általában nem nulla, és a hagyományos R^2 értelmezése is megváltozik. Példa $x = (1, 2, 3)$, $y = (2, 4, 7)$:

Modell	Együtthatók	Reziduális átlag
konstanssal	$[-0.667,\ 2.5]$	≈ 0
konstans nélkül	$[2.214]$	-0.0952

A konstanssal illesztett modell reziduálisainak átlaga nulla; a konstans nélküli nem. A két R^2 nem hasonlítható össze közvetlenül, mert a konstans nélküli képlet más nevezőt használ.

```
import numpy as np

def diagnostics(X):
    s = np.linalg.svd(X, compute_uv=False)
    tol = max(X.shape) * np.finfo(float).eps * s[0]
    rank = int((s > tol).sum())
    return {"rank": rank, "columns": X.shape[1],
            "full": rank == X.shape[1],
            "cond": s[0] / s[-1] if s[-1] > 0 else np.inf}

x = np.arange(1, 6, dtype=float)
y = 1 + 2*x
X = np.column_stack([np.ones_like(x), x, 2*x])
info = diagnostics(X)
print(info)
if info["rank"] < info["columns"]:
    print("normálegyenlet: ranghiány, nem invertálható")
else:
    np.linalg.solve(X.T @ X, X.T @ y)
beta = np.linalg.lstsq(X, y, rcond=None)[0]
print("lstsq:", beta, "predikció:", X @ beta)
```

A `lab/rank_deficient.py` `design()`, `diagnostics()`, `normal_solution()`, `fit()` és `intercept_demo()` függvényei ezt a két tesztet adják; a `tests/test_rank_deficient.py` a rank 2/3-at, a normálegyenlet hibáját, a `lstsq` predikciót és a konstansos/konstans nélküli reziduális átlagot ellenőrzi.

17.1.5 Feladatok

1. **Értelmezés.** A fenti mini-táblán a becsült modell $\hat{y} = 97{,}5 + 15 \cdot \text{quantity_sum} + 20 \cdot \text{priority_flag}$. Fogalmazd meg saját szavakkal, mit jelent a 20-as együttható, és miért nem jelenti azt, hogy a prioritásos ügyfelek minden rendelése 20 centtel drágább. Melyik megfigyelt cellában érvényes az intercept?

2. **Megvalósítás.** Írj tiszta NumPy függvényt `ols_by_hand(data, y_col, x_cols)` néven, amely a kapott pandas DataFrame-ből felépíti a dizájnmátrixot intercept oszloppal, kiszámítja a normálegyenlet megoldását, és visszaadja a `(beta, resid, sigma2)` hármast. A függvény ne használja a statsmodels-t, és a reziduumot a `y - X @ beta` művelettel adja. Teszteld a mini-táblára, és ellenőrizd a `sum(resid)` és `X.T @ resid` invariánsokat.

3. **Hibadiagnózis.** Egy hallgató így akarja reprodukálni a formula API eredményét:

   ```
   import statsmodels.api as sm
   y = data["basket_minor"].to_numpy()
   X = data[["quantity_sum", "priority_flag"]].to_numpy()
   fit_bad = sm.OLS(y, X).fit()
   ```

   Mi a különbség ez és a `smf.ols("basket_minor ~ quantity_sum + priority_flag", data).fit()` között? Milyen ellenőrzéssel vennéd észre a hibát a reziduumokon? Hogyan javítanád a `sm.OLS` verziót?

4. **Megváltozott feltétel: Northstar-adat.** Töltsd be a labor `orders.csv` állományát, és illeszd a

   ```
   basket_minor ~ quantity_sum + distance_km + priority_flag
   ```

   modellt a formula API-val. A CSV pontos elérési útja a saját projektgyökeredhez képest `data/orders.csv`; ha a labor nálad más almappában van, cseréld az útvonalat. A generált `orders.csv` 2400 sort tartalmaz, nincs benne hiányzó érték. Ellenőrizd, hogy a reziduális szabadsági fok $2400 - 4 = 2396$, a reziduum hossza 2400, és az összege közel nulla. Értelmezd a `distance_km` együttható mértékegységét és asszociációs jellegét. A tényleges együttható a generált adattól függ; ezt a saját futtatásod adja, itt nem mechanikusan várt szám.

17.1.6 Megoldások és indoklás

1. feladat megoldása. A 20-as együttható a feltételes különbség ugyanazon `quantity_sum` mellett: ha két rendelés darabszáma egyenlő, de az egyik prioritásos, a másik nem, a modell szerint az előbbi várható kosara 20 eurócenttel nagyobb. Nem minden prioritásos rendelés 20 centtel drágább, mert a darabszám is hat a kosárra, és a reziduum egyedi zajt is tartalmaz. Az intercept a `quantity_sum = 0`, `priority_flag = 0` cellához tartozik; a mini-táblában ez a 100-as megfigyelés cellája, a Northstar-adatokon viszont extrapolált cella.

2. feladat megoldása. A függvény placeholderek nélküli lehetséges változata:

```
def ols_by_hand(data, y_col, x_cols):
    y = data[y_col].to_numpy(dtype=float)
    X = np.column_stack(
        [np.ones(len(data))]
        + [data[c].to_numpy(dtype=float) for c in x_cols]
    )
    beta = np.linalg.solve(X.T @ X, X.T @ y)
    resid = y - X @ beta
    rss = resid @ resid
    sigma2 = rss / (len(y) - X.shape[1])
    return beta, resid, sigma2
```

Az ellenőrzés:

```
beta, resid, sigma2 = ols_by_hand(
    data, "basket_minor", ["quantity_sum", "priority_flag"]
)
assert abs(resid.sum()) < 1e-12
assert np.allclose(X.T @ resid, 0.0, atol=1e-12)  # X a függvényből kinyerhető
assert np.isclose(sigma2, 25.0)
```

A `np.linalg.solve` kis, jól kondicionált mátrixra alkalmas; nagyobb vagy majdnem kollineáris dizájnmátrixra a `np.linalg.lstsq` stabilabb választás.

3. feladat megoldása. Az `sm.OLS` osztály nem ad hozzá interceptet automatikusan, a formula API viszont igen. Ezért a hibás kód az origón átmenő síkot illeszt, a becsült együtthatók és reziduumok is eltérnek. Diagnózisnak kiváló a `fit_bad.resid.sum()`: intercept nélkül a reziduumok összege általában nem nulla, mert a dizájnmátrixból hiányzik az egyesekből álló oszlop, így az $X'\mathbf{e} = \mathbf{0}$ feltétel nem tartalmazza a $\sum e_i = 0$ egyenletet. Javítás:

```
X = sm.add_constant(
    data[["quantity_sum", "priority_flag"]].to_numpy(dtype=float)
)
fit_good = sm.OLS(y, X).fit()
```

Ezzel a `fit_good.params` és a `fit_good.resid` a formula API eredményével numerikusan egyezik.

4. feladat megoldása. A betöltés és illesztés:

```
orders = pd.read_csv(
    "data/orders.csv",
    parse_dates=["order_ts"],
)
fit_northstar = smf.ols(
    "basket_minor ~ quantity_sum + distance_km + priority_flag",
    data=orders,
).fit()

assert len(orders) == 2400
assert fit_northstar.df_resid == 2396
assert fit_northstar.resid.shape == (2400,)
assert abs(fit_northstar.resid.sum()) < 1e-6  # eurócent, lebegőpontos tűrés
print(fit_northstar.params)
```

A `df_resid` azért 2396, mert négy paraméter van: intercept, `quantity_sum`, `distance_km`, `priority_flag`. A reziduumösszeg az intercept megléte miatt elméletileg nulla; a gyakorlatban a lebegőpontos kerekítések miatt csak közel nulla. A `distance_km` együtthatójának mértékegysége eurócent/kilométer: azt mutatja, mennyivel más várható kosárérték tartozik egy kilométerrel nagyobb távolsághoz, ha a darabszám és a prioritásjelző rögzített. Ez leíró asszociáció, nem annak bizonyítéka, hogy a távolság növelése okozza a kosárérték változását. A 180 napos megfigyelt rendelési ablak 13.1. alfejezet korlátja is érvényben marad.

17.1.7 Gyakori tévedés és ellenőrzés

A leggyakoribb hiba az API-választásból fakad: a felhasználó azt hiszi, hogy az intercept mindig „benne van” a modellben, pedig az `sm.OLS(y, X)` verzió nem ad hozzá. Az ellenkező irányú hiba is előfordul: ha `pd.get_dummies`-szal hozunk létre dummyváltozókat, alapértelmezésben minden kategória kap egy oszlopot, és ezeket egy intercept oszloppal együtt tesszük a mátrixba, akkor tökéletes kollinearitást kapunk. Például három régió esetén a három dummy összege minden sorban 1, ami megegyezik az intercept oszloppal; az $X'X$ mátrix szinguláris, a `np.linalg.solve` hibát dob, a statsmodels formulaillesztése viszont általában pszeudoinverzzel kezelheti a ranghiányos

mátrixot, és nem feltétlenül jelez `PatsyError` hibát. A paraméterek ilyenkor nem azonosíthatók egyértelműen; ellenőrizni kell a rangot és a kondíciószámot. A javítás a referencia-kategória elhagyása vagy a formula `C(region)` jelölése. Önellenőrzés: a reziduum vektor hossza n, nem p; intercept esetén az összege nulla; minden becsült együtthatóhoz tartozik mértékegység; a `summary` p-értéke a nullhipotézis alatti feltételes szélsőség, nem az együttható igazságának valószínűsége.

17.1.8 Rövid összefoglaló

Az OLS a reziduális négyzetösszeget minimalizálja, és a normálegyenlet $X'X\hat{\beta} = X'\mathbf{y}$ megoldását adja. A dizájnmátrix első oszlopa csupa egyes; így az intercept közönséges együttható. A statsmodels formula API automatikusan hozzáadja, a tiszta NumPy vagy `sm.OLS` megoldásnál explicit oszlop kell. A reziduum merőleges a dizájnmátrix oszlopaira, ezért az összege nulla. A `summary` a coef, standard hiba, t, p-érték és konfidenciaintervallum oszlopait az $(X'X)^{-1}$ és a $RSS/(n-p)$ tanítóhiba alapján számolja; a klasszikus standard hibák független, homoszkedasztikus, normális hibákat feltételeznek. A következő alfejezetben formula, interakció és kategorikus változó segítségével bővítjük ezt a modellt.

17.2 Formula, interakció és kategorikus változó

Belépő · szint: haladó · idő: 90 perc · eszköz: statsmodels

Előfeltétel: 17.1. alfejezet · Célkészség: Formula, interakció és kategorikus változó · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

17.2.1 Cél és konkrét helyzet

Ebben az alfejezetben egy lineáris regressziós modellt úgy bővítünk, hogy a magyarázó változó kategória és numerikus változó együtt szerepeljen, ráadásul a kapcsolatuk csoportonként eltérő meredekséget engedjen meg. A megfigyelési egység egy Northstar-rendelés; a cél a `delivery_days` kézbesítési idő leírása a `segment` (consumer vagy business) és a `distance_km` alapján. Ez statisztikai, leíró modell, nem a rendelés pillanatában érvényes predikciós szerződés: a 21.1. alfejezet–21.4. alfejezet fogja a szivárgásmentes, időhelyes prediktív értékelést felépíteni.

A bemenet egy pandas DataFrame, amelynek minden sora egy rendelés, legalább a következő oszlopokkal: `delivery_days` (pozitív egész, nap), `segment` (két szintű kategória), `distance_km` (nemnegatív lebegőpontos szám). Az induló becslés, amit a modell ellenőrizhet: consumer ügyfélnél 10 km-en körülbelül 3 nap a kézbesítés, 30 km-en 5 nap; business ügyfélnél ugyanezeken a távolságokon 2, illetve 3 nap. Ezt a példa adatai szándékosan a $2 + 0{,}1d$, illetve $1{,}5 + 0{,}05d$ összefüggésből generálják, így a kézi ellenőrzés pontos lesz.

17.2.2 Formula, kategorikus változó és dizájnmátrix

A statsmodels formula API-ja a Patsy könyvtárat használja. A formula (képlet) karakterlánc, amely a modell szándékát írja le:

```
"delivery_days ~ distance_km"
```

A `~` bal oldalán a válaszváltozó, jobb oldalán a prediktorok állnak. A `+` új prediktort ad a modellhez, az interceptet a statsmodels automatikusan hozzáadja, kivéve ha `-1`-et írunk. A 17.1. alfejezetben láttuk, hogy a NumPy-megoldásnál az intercept oszlopot kézzel kellett létrehozni; a formula API ezt elvégzi.

A kategorikus változót a `C()` függvénnyel jelöljük. A Patsy alapértelmezése a treatment kódolás (treatment coding), más néven dummy kódolás: az egyik szint referencia-csoport (reference category), a többi szinthez egy-egy indikátorváltozó tartozik. Az indikátor 1, ha a megfigyelés az adott csoportba esik, egyébként 0. A referencia-csoportot explicit módon megadjuk, mert a Patsy ábécérend szerint választana, és a `business` szint így kerülne alapértelmezett referenciapozícióba:

```
"C(segment, Treatment(reference='consumer'))"
```

A treatment kódolású modellben a referencia-csoport megfigyeléseihez minden indikátor értéke 0. Ezért az intercept ennek a csoportnak az alapértéke, a többi együttható pedig eltérés ehhez a csoporthoz képest.

Az interakció (interaction) a `*` operátorral írható. A Patsy szabálya:

```
A * B  =  A + B + A:B
```

A `:` csak az interakciós tagot jelenti, főhatások nélkül. A jelen modell:

```
"delivery_days ~ C(segment, Treatment(reference='consumer')) * distance_km"
```

Ez három prediktort hoz létre: a kategorikus főhatást, a numerikus főhatást és a kettő szorzatát. A becsült egyenlet:

$$\hat{y} = b_0 + b_B I_B + b_D d + b_{B:D} I_B d,$$

ahol I_B a business indikátor, d a `distance_km` kilométerben mért értéke. Az együtthatók értelmezése:

- b_0: consumer csoport várható `delivery_days` értéke $d = 0$ km-nél;
- b_B: business csoport eltolódása consumerhez képest $d = 0$ km-nél;
- b_D: consumer csoport meredeksége, nap per kilométer;
- $b_{B:D}$: business csoport meredekségének különbsége a consumer meredekséghez képest.

A business csoport tényleges meredeksége tehát $b_D + b_{B:D}$, nem pedig b_D önmagában. Ez a különbségtétel a leggyakoribb értelmezési buktató.

A Patsy a formulából dizájnmátrixot (design matrix) épít. A statsmodels illesztett modellben az oszlopnevek a következőképpen jelennek meg:

```
Intercept
C(segment, Treatment(reference='consumer'))[T.business]
distance_km
C(segment, Treatment(reference='consumer'))[T.business]:distance_km
```

A `T.business` jelölés a treatment kódolás business szintjét azonosítja. A mögöttes `design_info` objektum írja le a kódolási szabályt; a gyakorlati ellenőrzéshez a `res.model.exog_names` lista elegendő. A dizájnmátrix első oszlopa csupa egyes, így a formula API modellje ugyanazt a geometriát valósítja meg, mint a 17.1. alfejezetben kézzel felépített X mátrix.

17.2.3 Teljes minimális munkapélda

A következő kód determinisztikus, zaj nélküli tanuló adatot hoz létre, hogy az együtthatók és a predikciók kézzel ellenőrizhetők legyenek. Az adatgeneráló összefüggés:

- consumer: delivery_days $= 2 + 0{,}1 \cdot$ distance_km,
- business: delivery_days $= 1{,}5 + 0{,}05 \cdot$ distance_km.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

rows = [
    ("consumer", 10.0, 3.0),
    ("consumer", 20.0, 4.0),
    ("consumer", 30.0, 5.0),
    ("business", 10.0, 2.0),
    ("business", 20.0, 2.5),
    ("business", 30.0, 3.0),
]
df = pd.DataFrame(rows, columns=["segment", "distance_km", "delivery_days"])

mod = smf.ols(
    "delivery_days ~ C(segment, Treatment(reference='consumer')) * distance_km",
    data=df,
)
res = mod.fit()

print(res.params)
print(res.model.exog_names)
```

Formula-adat szótár. A formula önmagában nem elég az értelmezéshez; a kódolást is rögzíteni kell:

Változó	Szerep	Kódolás	Referenciaszint
`segment`	kategorikus	treatment (dummy)	`consumer`
`distance_km`	numerikus	változatlan	–
`segment:distance_km`	interakció	a `business` dummy és a távolság szorzata	–
`Intercept`	konstans	1	`consumer`, $d = 0$

A `C(segment, Treatment(reference='consumer'))` explicit módon rögzíti a referenciát: a `business` sorokban a dummy 1, a `consumer` sorokban 0. Ha a kategóriasorrend vagy a referencia megváltozik, a paraméternevek és az együtthatók értelmezése változik, a predikció viszont nem.

Az elméletileg várt `res.params` kimenet:

```
Intercept                                          2.0
C(segment, Treatment(reference='consumer'))[T.business]       -0.5
distance_km                                        0.1
C(segment, Treatment(reference='consumer'))[T.business]:distance_km   -0.05
```

Az oszlopnevek listája:

```
['Intercept',
 "C(segment, Treatment(reference='consumer'))[T.business]",
 'distance_km',
 "C(segment, Treatment(reference='consumer'))[T.business]:distance_km"]
```

A zaj nélküli konstrukció miatt a reziduumok elméletileg nullák; a lebegőpontos számítás miatt legfeljebb 10^{-12} nagyságrendű eltérés várható. Ezen a hat soron a standard hibák és a p-értékek nem értelmezhetők statisztikai következtetésre, mert nincs mintavételi zaj; a `summary()` táblázatát itt csak a formula-kódolás szemléltetésére használjuk.

Négy konkrét predikciót ellenőrzünk kézzel, a becsült együtthatókból:

```
new = pd.DataFrame({
    "segment": ["consumer", "consumer", "business", "business"],
    "distance_km": [10.0, 30.0, 10.0, 30.0],
})
print(res.predict(new))
```

A predikciós képletek:

$$\begin{aligned} \text{consumer},\ d=10:&\ \ 2+0{,}1\cdot 10 = 3{,}0,\\ \text{consumer},\ d=30:&\ \ 2+0{,}1\cdot 30 = 5{,}0,\\ \text{business},\ d=10:&\ \ (2-0{,}5)+(0{,}1-0{,}05)\cdot 10 = 2{,}0,\\ \text{business},\ d=30:&\ \ (2-0{,}5)+(0{,}1-0{,}05)\cdot 30 = 3{,}0. \end{aligned}$$

Az elméletileg várt `res.predict(new)` kimenet közelítőleg `[3.0, 5.0, 2.0, 3.0]`. Ezt a futtatás után érdemes `np.allclose` ellenőrzéssel összevetni.

Két mesterséges sor design-mátrixszal. A `patsy.dmatrix` ugyanazt a design-mátrixot adja, amelyet a modell használ; a kézi predikció a sor és a paramétervektor skaláris szorzata. Például consumer 15 km és business 15 km:

Sor	Intercept	business	distance	business:distance	Kézi predikció
consumer, 15 km	1	0	15	0	$2+0{,}1\cdot 15 = 3{,}5$
business, 15 km	1	1	15	15	$2-0{,}5+0{,}1\cdot 15-0{,}05\cdot 15 = 2{,}25$

```
import patsy
new2 = pd.DataFrame({"segment": ["consumer", "business"], "distance_km": [15.0, 15.0]})
rhs = mod.formula.split("~", 1)[1]
X2 = patsy.dmatrix("~" + rhs, new2, return_type="dataframe")
manual2 = X2.to_numpy() @ res.params.to_numpy()
print(X2)
print(manual2, res.predict(new2).to_numpy())
assert np.allclose(manual2, res.predict(new2), atol=1e-8)
```

A formula a transzformációt is közvetlenül tudja kezelni. Ha a távolság hatása nem lineáris, írhatunk például logaritmikus prediktort:

```
smf.ols(
    "delivery_days ~ C(segment, Treatment(reference='consumer')) * np.log(distance_km)",
    data=df,
)
```

A `np.log` nevet a Patsy a hívó Python-környezetből oldja fel. Ha hatványt vagy más kifejezést akarunk szó szerint értelmeztetni, az `I()` as-is operátort használjuk, például `I(distance_km**2)`, mert a `^` karakter a formulában interakciót jelöl, nem hatványozást. A transzformált változó együtthatójának mértékegysége már nem nap per kilométer, hanem például nap per log-kilométer; ezt az értelmezésnél mindig ki kell mondani.

17.2.4 Eredmény értelmezése és ellenőrző invariant

A fenti modellben az együtthatók pontosan visszaadják az adatgeneráló értékeket. A predikció ellenőrzésének invariánsa: ha a `res.params` értékeit behelyettesítjük a fenti képletbe, a kapott számnak egyeznie kell a `res.predict(new)` kimenetével, 10^{-8} abszolút toleranción belül. Emellett a referencia-csoport indikátora nulla, ezért a consumer predikcióban nincs business-tag; ezt a `new` DataFrame sorain is ellenőrizhetjük.

A Northstar `orders.csv` adatain ugyanez a modell már nem lesz tökéletes, mert a valódi generált folyamat tartalmaz zajt, és a távolságon kívül más tényezők is hatnak a kézbesítési időre. Az együtthatók ekkor becslések, nem adatgeneráló paraméterek. A klasszikus standard hibák ráadásul független, homoszkedasztikus hibákat feltételeznek; a rendelések ügyfelekbe ágyazott szerkezete ezt sértheti. A robusztus standard hibák és a diagnosztika a 17.3. alfejezet témája lesz.

Interakció értelmezése: a `distance_km:C(segment)` tag nem önálló „kategóriahatás”, hanem a távolság szerinti meredekség eltérése a referenciacsoporthoz képest. A főhatás a numerikus változó nullértékénél értendő; ha ez tartalmilag nem releváns, célszerű a távolságot központosítani.

17.2.5 Feladatok

17.2.5.1 1. feladat

A minimális példa becsült együtthatóival add meg a várható `delivery_days` értéket consumer ügyfélre 15 km-nél és business ügyfélre 15 km-nél. Fogalmazd meg szavakban, mit jelent a $-0{,}05$ interakciós együttható.

17.2.5.2 2. feladat

Töltsd be a `lab/data/orders.csv` fájlt a projektgyökérből, és illessz modellt a `delivery_days ~ C(segment, Treatment(reference='consumer')) * distance_km` formulával. Ellenőrizd kézzel négy predikciót: consumer 10 km, consumer 40 km, business 10 km, business 40 km. A kézi számításhoz használd a `res.params` értékeit, majd hasonlítsd össze a `res.predict` kimenetével, `np.allclose(..., atol=1e-6)` ellenőrzéssel.

17.2.5.3 3. feladat

Egy társ a `segment * distance_km` modell `distance_km` együtthatójáról azt állítja, hogy az a távolság állandó hatása mindkét szegmensben. Mi a hiba ebben az értelmezésben? Adj számszerű ellenpéldát a minimális példa vagy a Northstar-modell csoportonkénti meredekségeivel.

17.2.5.4 4. feladat

Illeszd újra a minimális példa modelljét úgy, hogy a referencia-csoport business legyen: `C(segment, Treatment(reference='business'))`. Ellenőrizd, hogy a négy predikció értéke változatlan marad, miközben az együtthatók átíródnak. Add meg az új együtthatók elméleti értékeit, és indokold meg az átparaméterezést.

17.2.6 Megoldások és indoklás

17.2.6.1 1. feladat megoldása

Consumer 15 km-re:

$$2 + 0{,}1 \cdot 15 = 3{,}5.$$

Business 15 km-re:

$$(2 - 0{,}5) + (0{,}1 - 0{,}05) \cdot 15 = 1{,}5 + 0{,}75 = 2{,}25.$$

A $-0{,}05$ interakciós együttható azt mondja meg, hogy a business csoport meredeksége 0,05 nap/km-rel kisebb, mint a consumer referencia-csoport meredeksége. Nem azt jelenti, hogy a távolság hatása businessben $-0{,}05$: a business meredekség $0{,}1 + (-0{,}05) = 0{,}05$ nap/km.

17.2.6.2 2. feladat megoldása

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

orders = pd.read_csv("lab/data/orders.csv")
mod = smf.ols(
    "delivery_days ~ C(segment, Treatment(reference='consumer')) * distance_km",
    data=orders,
)
res = mod.fit()

p = res.params
b0 = p["Intercept"]
b_b = p["C(segment, Treatment(reference='consumer'))[T.business]"]
b_d = p["distance_km"]
b_bd = p["C(segment, Treatment(reference='consumer'))[T.business]:distance_km"]

new = pd.DataFrame({
    "segment": ["consumer", "consumer", "business", "business"],
    "distance_km": [10.0, 40.0, 10.0, 40.0],
})
is_business = (new["segment"] == "business").astype(float)
d = new["distance_km"].to_numpy()

manual = b0 + b_b * is_business + (b_d + b_bd * is_business) * d
pred = res.predict(new).to_numpy()

print(p)
print("manual:", manual)
print("predict:", pred)
print("match:", np.allclose(manual, pred, atol=1e-6))
```

A konkrét együtthatók a tanuló helyi futtatásától függenek, ezért itt nem adunk előre rögzített számokat. Az ellenőrző invariant az, hogy a `manual` és a `predict` tömbök 10^{-6} abszolút toleranción belül egyeznek. A Northstar adatokon a modell nem tökéletes, de ugyanaz a lineáris prediktor adja mindkét számítást; az eltérés csak a kézi számítás kerekítési hibája lehet.

17.2.6.3 3. feladat megoldása

A hiba az, hogy az interakciós modellben a numerikus főhatás nem általános hatás, hanem a referencia-csoport meredeksége. A minimális példában a `distance_km` együtthatója 0,1 nap/km, de ez kizárólag a consumer csoport meredeksége. A business csoport meredeksége $0{,}1 + (-0{,}05) = 0{,}05$ nap/km. Ha a társ a business csoportra is 0,1 nap/km hatást mond, akkor 10 km-en 1,0 nap helyett 0,5 nap távolsághatást jósol, tehát a hiba számszerűen is látható. Az állandó hatású modell a `segment + distance_km` formula lenne, interakciós tag nélkül; az interakciós tag szignifikanciáját vagy a csoportonkénti meredekségek különbségét külön kell vizsgálni.

17.2.6.4 4. feladat megoldása

```
mod2 = smf.ols(
    "delivery_days ~ C(segment, Treatment(reference='business')) * distance_km",
    data=df,
)
res2 = mod2.fit()
print(res2.params)

new = pd.DataFrame({
    "segment": ["consumer", "consumer", "business", "business"],
    "distance_km": [10.0, 30.0, 10.0, 30.0],
})
pred2 = res2.predict(new)
print(pred2)
print(res2.model.exog_names)
```

Az új paraméterezés elméletileg várt együtthatói:

- intercept: business alapszint $d = 0$-nál, tehát 1,5;
- consumer eltolódás: $+0{,}5$;
- `distance_km`: business meredekség, 0,05;
- consumer:distance interakció: $+0{,}05$, mert a consumer meredekség $0{,}1 = 0{,}05 + 0{,}05$.

A `res2.predict(new)` kimenete ismét `[3.0, 5.0, 2.0, 3.0]` közelítőleg. Az ellenőrzés: `np.max(np.abs(pred2 - res.predict(new)))` kisebb, mint 10^{-8}. Az együtthatók tehát a referencia-csoport választásától függenek, a modell által leírt függvény viszont ugyanaz marad. Ezért minden közlésnél meg kell nevezni a referencia-csoportot; enélkül az intercept és a főhatás értelmezhetetlen.

17.2.7 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés: „az interakciós együttható a numerikus változó hatása a másik csoportban”, vagy fordítva, „a főhatás minden csoportban állandó”. Az ellenpélda a fenti modell: a business csoport meredeksége nem $-0{,}05$, hanem $0{,}1 - 0{,}05 = 0{,}05$; a 0,1 főhatás pedig nem közös hatás, hanem a consumer referencia-csoport meredeksége. A 13.2. alfejezet csoportátlagos gondolkodásmódja itt is segít: mindig kérdezd meg, melyik csoportra vonatkozik az együttható.

Önellenőrző kérdések: melyik oszlop tartozik a referencia-csoporthoz, és miért nulla ott az indikátor? Mit ad a predikció, ha a numerikus változó értéke nulla? A `res.model.exog_names` listában minden együtthatóhoz tartozik-e értelmezhető oszlop? Ha transzformált prediktort használsz, meg tudod-e mondani az új mértékegységet? Ha a referencia-csoportot megváltoztatod, változnak-e a predikciók? A válaszok segítenek elválasztani a paraméterezési önkényt a modell tartalmától.

17.2.8 Rövid összefoglaló

A statsmodels formula API a Patsy segítségével explicit referencia-csoportú treatment kódolást és interakciót kínál. A `C(segment, Treatment(reference='consumer')) * distance_km` képlet interceptet, csoportindikátort, numerikus főhatást és interakciós tagot hoz létre. A főhatás a referencia-csoport meredeksége, az interakció a másik csoport meredekségkülönbsége; a predikciót mindig a becsült együtthatókból kézzel is ellenőrizhetjük. A kis példán a predikciók pontosan visszaadják a generáló értékeket, a Northstar adatokon pedig a kézi és a `predict` számítás egyezését toleranciával igazoljuk. A transzformációk és a referencia-csoport csere a paraméterezést változtatják, a illesztett függvényt nem. A következő alkalmazás a modell diagnosztikája és robusztus standard hibák a 17.3. alfejezetben.

17.3 Diagnosztika és robusztus standard error

Belépő · szint: haladó · idő: 90 perc · eszköz: statsmodels

Előfeltétel: 17.2. alfejezet · Célkészség: Diagnosztika és robusztus standard error · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

17.3.1 Cél és konkrét helyzet

Egy szállítási szimulációban a kézbesítés munkapercét szeretnénk előre jelezni a rendelés távolságából. A megfigyelési egység egy rendelés; az egyenletes eloszlású `distance_km` magyarázó változó mellett a hiba szórása a távolsággal szándékosan növekszik. A kérdés nem csak az, hogy megvan-e a lineáris kapcsolat, hanem az is, hogy a paraméterek bizonytalanságát megbízhatóan tudjuk-e állítani. A bemeneti minta: 400, 1 és 12 km közötti távolságú rendelés, seeddel reprodukálható hiba; a kimeneti becslés a meredekség konfidenciaintervalluma. Előzetes becslés: az intervallumban a klasszikus standard hiba heteroszkedaszticitás mellett lehet túl kicsi vagy túl nagy; a lefedettség irányát a konkrét adatgeneráló mechanizmus és a regresszor eloszlása határozza meg, a robusztus változat közelebbi, de nem pontosan 95%-os eredményt várunk. Az alfejezetben a diagnosztikát és a korrekciót kézzel is ellenőrizzük.

17.3.2 Diagnosztikai alapfogalmak

Az OLS-modell alakja $y_i = x_i'\beta + \varepsilon_i$, ahol y_i a kimenet, x_i a magyarázó változók oszlopvektora, β a paramétervektor, ε_i hibatag. Mátrixos alakban $y = X\beta + \varepsilon$. A becsült paraméter

$$\hat{\beta} = (X'X)^{-1}X'y$$

mindig létezik, ha $X'X$ invertálható. A reziduális, angolul *residual*:

$$e_i = y_i - \hat{y}_i, \qquad \hat{y} = X\hat{\beta}.$$

A korábbi vetítési fogalom itt kap szerepet: $H = X(X'X)^{-1}X'$ az illesztett értékekre vetít, $\hat{y} = Hy$. A H mátrix átlós elemei, h_i, az úgynevezett *leverage*-ek, és $0 \le h_i \le 1$. Homoszkedasztikus modellben a reziduális varianciája $\mathrm{Var}(e_i) = \sigma^2(1-h_i)$, tehát a nagy leverage-ű pontok közelében a reziduális eleve kisebbnek várható.

A reziduális ábrán két szempontot vizsgálunk. A reziduális a magyarázó vagy illesztett értékek ellen: ha a pontok véletlenszerű sávban helyezkednek el, a feltételes várható érték megfelelő lehet. A fan-shape, hanglemez vagy szélesedő alak a **heteroszkedaszticitás** (változó hibaszórás, angolul *heteroskedasticity*) jele; ekkor a hiba szórása nem állandó σ, hanem megfigyeléstől függ. A második szempont a nemlinearitás: ha az egy nem-görbült sáv helyett ívben szóródnak a pontok, a modell várható értékfüggvénye rossz.

A **kollinearitás** (*collinearity*) nem a hiba, hanem a magyarázó változók egymás közötti erős kapcsolata. A standard hiba közös növekedését a vetítési intuíció magyarázd: ha két változó majdnem párhuzamos, a modell nem tudja megbízhatóan szétválasztani, melyikük okozta a változást. A variancia-inflációs tényező (VIF) a j. változóhoz:

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2},$$

ahol R_j^2 az a determinációs együttható, amellyel a j. magyarázó változót a többi magyarázó változó előre jelzi. A szokásos hüvelykujjszabály szerint VIF > 10 gyanús, de a határ nem minősítő ítélet.

Az **influence**, azaz befolyásos megfigyelés vizsgálata a leverage és a reziduális együttes nagyságát méri. A Cook-távolság:

$$D_i = \frac{e_i^2}{p \cdot s^2} \cdot \frac{h_i}{(1 - h_i)^2},$$

ahol p a paraméterek száma, $s^2 = e'e/(n-p)$ a reziduális szórásnégyzet becslése. A képlet első tényezője a szélsőséges reziduálist, második tényezője a nagy leverage-t díjazza.

17.3.3 Robusztus kovarianciamátrix

A klasszikus OLS standard hiba feltételezi, hogy $\mathrm{Var}(\varepsilon|X) = \sigma^2 I$, azaz minden hiba független és azonos szórású. Ekkor a paraméterbecslés kovarianciamátrixa

$$\mathrm{Var}(\hat{\beta}) = \sigma^2 (X'X)^{-1}.$$

Ha azonos szórás helyett $\mathrm{Var}(\varepsilon|X) = \Sigma$ diagonális, heterogén σ_i^2 elemekkel, a helyes alak:

$$\mathrm{Var}(\hat{\beta}) = (X'X)^{-1} X'\Sigma X (X'X)^{-1}.$$

A robusztus változat nem ismeri Σ-t, ezért az egyedi reziduálisokra épít, a **White-féle heteroszkedaszticitás-konzisztens becslést** adja. A HC0 változat

$$\widehat{\mathrm{Var}}_{HC0}(\hat{\beta}) = (X'X)^{-1} X' \,\mathrm{diag}(e_i^2) X (X'X)^{-1}.$$

A HC1 korrekció a kis mintás szabadságfokkal arányos, $n/(n-p)$ faktorral szoroz. A `statsmodels` `cov_type='HC1'` vagy `cov_type='HC3'` beállítást kínál; a HC3 a leverage-eket is figyelembe veszi, ezért kisebb mintán gyakran konzervatívabb. A robusztus standard hiba tehát nem változtatja meg a pontbecslést, hanem a bizonytalansági számtant cseréli, ha a hiba szórása nem állandó. A konfidenciaintervallum formája változatlan, $\hat{\beta}_j \pm t_{0{,}975;\,n-p} \cdot SE_j$, de SE_j a robusztus mátrixból jön.

17.3.4 Teljes minimális munkapélda

Az alábbi kód saját gépen futtatható. A bemeneti generátor seeddel készül, a kód a `statsmodels` formula API-ját használja.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
import statsmodels.api as sm

# Bemenet: szintetikus rendelésszintű megfigyelés, növekvő hibaszórással.
rng = np.random.default_rng(20261020)
n = 400
distance_km = rng.uniform(1.0, 12.0, size=n)
sigma = 0.8 + 0.6 * distance_km
work_minutes = 18.0 + 4.0 * distance_km + rng.normal(0.0, sigma, size=n)
df = pd.DataFrame({"distance_km": distance_km, "work_minutes": work_minutes})

# Három illesztés ugyanarra a bemenetre.
fit_classic = smf.ols("work_minutes ~ distance_km", data=df).fit()
fit_hc1 = smf.ols("work_minutes ~ distance_km", data=df).fit(cov_type="HC1")
fit_hc3 = smf.ols("work_minutes ~ distance_km", data=df).fit(cov_type="HC3")

def slope_ci(fit):
    ci = fit.conf_int().loc["distance_km", :]
    return pd.Series({
        "slope": fit.params["distance_km"],
        "SE": fit.bse["distance_km"],
        "CI_low": ci.iloc[0],
        "CI_high": ci.iloc[1],
        "width": ci.iloc[1] - ci.iloc[0],
    })

summary = pd.DataFrame({
    "nonrobust": slope_ci(fit_classic),
    "HC1": slope_ci(fit_hc1),
    "HC3": slope_ci(fit_hc3),
}).T
print(summary.round(4))
```

Szemléltető kimenet, nem itt futtatott eredmény: a klasszikus SE várhatóan körülbelül 0,04, a HC1 SE körülbelül 0,055, a HC3 körülbelül 0,06; a pontos szám a gépi lebegőpontos műveletek miatt is eltérhet. A robusztus és klasszikus intervallumok szélességének nincs általánosan garantált sorrendje; a konkrét példában a lefedettséget és a kézi számítást kell ellenőrizni.

A HC1 képletét kézzel is ellenőrizhetjük, a fönti mintán:

```
# Kézi HC1 a képletből.
y = df["work_minutes"].to_numpy()
Xmat = np.column_stack([np.ones(len(df)), df["distance_km"].to_numpy()])
nobs, k = Xmat.shape
beta = np.linalg.lstsq(Xmat, y, rcond=None)[0]
e = y - Xmat @ beta
xx_inv = np.linalg.inv(Xmat.T @ Xmat)
hc0 = xx_inv @ (Xmat.T @ np.diag(e**2) @ Xmat) @ xx_inv
hc1 = hc0 * nobs / (nobs - k)
manual_se = np.sqrt(np.diag(hc1))
print("manual HC1 SE:", manual_se.round(6))
print("statsmodels HC1 SE:", fit_hc1.bse.round(6))
```

A kézi és a `statsmodels` HC1 standard hiba `np.allclose`-zal, `rtol=1e-10` mellett egyezik. A klasszikus standard hiba ellenőrzése:

```
mse = (e @ e) / (nobs - k)
classic_se = np.sqrt(np.diag(mse * xx_inv))
print("manual classic SE:", classic_se.round(6))
print("statsmodels classic SE:", fit_classic.bse.round(6))
```

A két ellenőrző invariáns: a kézzel számított HC1 és klasszikus standard hiba ugyanazt adja, mint a csomag. Ha a korrekciós faktor `nobs/(nobs-k)` kimarad, a kézi érték a HC0 lesz, ami kisebb a HC1-nél.

17.3.5 Fedettség szimulációja

A helyes értelmezéshez ismételjük a mintavételt. Az alábbi függvény kiszámolja, hogy a 95%-os névleges szintű intervallum milyen gyakran tartalmazza a valódi meredekséget $\beta_1 = 4{,}0$-ra. A Monte Carlo standard hiba $T = 400$ ismétlésnél körülbelül $\sqrt{0{,}95 \cdot 0{,}05/400} \approx 0{,}011$, tehát a százalékpont körüli eltérés 1,1 százalékpontos véletlen ingadozással értelmezhető.

```
def coverage_sim(T=400, n=400, seed=44):
    sim_rng = np.random.default_rng(seed)
    cover_classic = []
    cover_hc1 = []
    for _ in range(T):
        x = sim_rng.uniform(1.0, 12.0, size=n)
        sigma = 0.8 + 0.6 * x
        y = 18.0 + 4.0 * x + sim_rng.normal(0.0, sigma, size=n)
        data = pd.DataFrame({"distance_km": x, "work_minutes": y})
        ci_cl = smf.ols("work_minutes ~ distance_km",
↪   data=data).fit().conf_int().loc["distance_km", :]
        ci_hc = smf.ols("work_minutes ~ distance_km",
↪   data=data).fit(cov_type="HC1").conf_int().loc["distance_km", :]
        cover_classic.append(ci_cl.iloc[0] <= 4.0 <= ci_cl.iloc[1])
        cover_hc1.append(ci_hc.iloc[0] <= 4.0 <= ci_hc.iloc[1])
    return np.mean(cover_classic), np.mean(cover_hc1)

print("classic, HC1:", coverage_sim())
```

Elméletileg várt irány: a hagyományos lefedettség 95% alatt marad, mert a feltételezett állandó σ nem igaz; a HC1 lefedettség közelebb van 95%-hoz, a pontos eltérés a Monte Carlo hibától is függ. Ez nem „ellenőrzött mért futási eredmény” a könyvírás közegében, hanem a tanuló saját futtatásával igazolandó várakozás.

17.3.5.1 HC1, klaszter-robusztus és HAC: melyik mikor?

A heteroszkedaszticitás-robosztus HC1 nem univerzális gyógyszer. A választás a függőség szerkezetétől függ:

Probléma	Megfelelő SE	Nem elég
heteroszkedaszticitás, független megfigyelések	HC0/HC1/HC3	klasszikus
ismételt megfigyelés ügyfélen belül	klaszter-robusztus	HC1
időbeli autokorreláció	HAC (Newey–West)	HC1
független, homoszkedasztikus	klasszikus	—

Két szimulált adathalmazon, ugyanarra a lejtőre:

Adat	klasszikus SE	HC1 SE	klaszter SE
heteroszkedasztikus, független	0.0424	0.0435	—
10 klaszter, klaszterhatás	0.0270	0.0212	**0.2024**

A heteroszkedasztikus adaton a HC1 alig nagyobb a klasszikusnál, mert a heteroszkedaszticitás mérsékelt. A klaszterezett adaton a HC1 **kisebb**, mint a klasszikus (0.0212 vs 0.0270), mert a függőséget nem veszi figyelembe; a klaszter-robusztus SE 0.2024, közel hétszerese a HC1-nek. A klaszter-robusztus standard error a csoportok közötti score-összegeket használja, és akkor is figyelmeztetni kell, ha kevés klaszter van.

```
import numpy as np

def fit(x, y):
    X = np.column_stack([np.ones_like(x), x])
    beta = np.linalg.lstsq(X, y, rcond=None)[0]
    return X, beta, y - X @ beta

def variances(x, y, groups=None):
    X, beta, residual = fit(x, y)
    bread = np.linalg.inv(X.T @ X)
    classical = (residual @ residual / (len(x) - 2)) * bread[1, 1]
    centred = x - x.mean()
    hc0 = bread[1, 1]**2 * ((centred**2) * residual**2).sum()
    hc1 = hc0 * len(x) / (len(x) - 2)
    cluster = None
    if groups is not None:
        meat = np.zeros((2, 2))
        for group in np.unique(groups):
            mask = groups == group
            score = X[mask].T @ residual[mask]
            meat += np.outer(score, score)
        cluster = (bread @ meat @ bread)[1, 1]
    return {"classical": classical, "hc1": hc1, "cluster": cluster}
```

A `lab/robust_se.py` `fit()`, `variances()` és `simulate()` függvényei ezt a táblát adják; a `tests/test_robust_se.py` a HC1 = HC0 · n/(n−2) azonosságot, a klaszter-SE HC1-nél jóval nagyobb értékét és a heteroszkedasztikus eset HC1 klasszikus relációját ellenőrzi. A cluster-robusztus és a HAC implementáció a későbbi idősoros fejezetben mélyül el; itt a döntés a lényeg: a HC1 a független heteroszkedaszticitás eszköze, nem a klaszterezésé.

17.3.6 Kollinearitás és influence diagnosztikája

A diagnosztikai szempontok közül a kollinearitást és a befolyásos pontokat külön kezeljük, mert a robusztus standard hiba önmagában nem gyógyítja őket. Kollineáris példa:

```
rng2 = np.random.default_rng(7)
n2 = 200
x1 = rng2.normal(0, 1, n2)
x2 = 0.99 * x1 + rng2.normal(0, 0.05, n2)
y2 = 12.0 + 1.7 * x1 + 1.9 * x2 + rng2.normal(0, 0.4, n2)
d2 = pd.DataFrame({"x1": x1, "x2": x2, "y": y2})
m2 = smf.ols("y ~ x1 + x2", data=d2).fit()
print(m2.params)
print(m2.bse)
print("korreláció x1, x2:", np.corrcoef(x1, x2)[0, 1].round(3))

from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
Xv = sm.add_constant(d2[["x1", "x2"]])
vif = [variance_inflation_factor(Xv.to_numpy(), j) for j in range(Xv.shape[1])]
print("VIF (const, x1, x2):", [round(v, 1) for v in vif])
```

A várt korreláció 0,995 fölött van, a VIF 50 fölötti lehet, és az együtthatók pillanatnyi becslése instabillá válik. A predikció ettől még lehet stabil, de a paraméter magyarázata nem. A fix nem a robusztus standard hiba, hanem a két szinte párhuzamos változó együttes szerepeltetésének újragondolása.

Influence-vizsgálat a fönti heteroszkedasztikus mintán:

```
infl = fit_classic.get_influence()
hat = infl.hat_matrix_diag
cooks = infl.cooks_distance[0]
p = fit_classic.model.exog.shape[1]  # konstans + magyarázó változók
threshold_hat = 2 * p / len(df)
threshold_cook = 4 / (len(df) - p)
flags = (hat > threshold_hat) | (cooks > threshold_cook)
print("nagy leverage-ű vagy befolyásos pontok:", flags.sum())
```

A küszöbök heurisztikusak; a cél a vizsgálat, nem a mechanikus ponttörlés. Ha törlünk egy pontot, a döntést a jelentésben a sorazonosítóval és az üzleti indokkal kell rögzíteni.

17.3.7 Eredmény értelmezése

Ebben a szimulációban a HC1 és HC3 intervallum szélesebb, mint a klasszikus. A pontbecslés mindhárom változatban közel azonos, mert a modell feltételes várható értéke helyesen lineáris. A szimulációban a klasszikus lefedettség elmarad a névleges 95%-os szinttől, a robusztus közelíti azt. A korrekció tehát a reziduális variancia alakjáról szól, nem a torzításról: ha a várható értékfüggvény rossz, a robusztus standard hiba sem menti meg a becslést. A diagnosztikai ábra kódja a laborjelentéshez:

```
resid = fit_classic.resid
fitted = fit_classic.fittedvalues
fig, ax = plt.subplots(1, 2, figsize=(9, 3.5))
ax[0].scatter(df["distance_km"], resid, s=18, alpha=0.6)
ax[0].axhline(0, color="black", linewidth=0.8)
ax[0].set_xlabel("távolság (km)")
ax[0].set_ylabel("reziduális (perc)")
ax[0].set_title("Reziduális a magyarázó változó szerint")
ax[1].scatter(fitted, resid, s=18, alpha=0.6)
ax[1].axhline(0, color="black", linewidth=0.8)
ax[1].set_xlabel("illesztett érték (perc)")
ax[1].set_ylabel("reziduális (perc)")
ax[1].set_title("Fitted-residual ábra")
fig.tight_layout()
```

A szélesedő tölcsér mindkét ábrán a heteroszkedaszticitás vizuális jele. A levonható következtetés így hangzik: a paraméter pontbecslése megmarad, a bizonytalanság viszont robusztus standard hibával a feltételhez illőbb állítássá válik. A robusztus SE iránya nem garantált: ebben a szimulációban nagyobb a klasszikusnál, más heteroszkedaszticitási mintázatban kisebb is lehet.

Diagnosztikai feltétel: a HC-standard hibák a feltételes várhatóérték-modell hibás specifikációját, az endogenitást és az adatok időbeli/klaszterfüggését nem oldják meg. Klaszterezett adatokra klaszterrobusztus standard hiba szükséges lehet.

17.3.8 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés úgy hangzik, hogy „a robusztus standard hiba kijavítja a rossz modellt”. Ellenpélda: legyen egy nem megfigyelt, kihagyott zavaró u, amely a távolsággal korrelál, és a munkapercre is hat. A becslésben csak a távolság szerepel:

```
rng_c = np.random.default_rng(123)
n_c = 400
x = rng_c.uniform(1, 12, n_c)
u = 1.5 * x + rng_c.normal(0, 1, n_c)  # kihagyott zavaró
y = 12.0 + 2.0 * x + 3.0 * u + rng_c.normal(0, 1.0, n_c)
df_c = pd.DataFrame({"x": x, "u": u, "y": y})
m_bad = smf.ols("y ~ x", data=df_c).fit(cov_type="HC1")
m_full = smf.ols("y ~ x + u", data=df_c).fit()
print("kihagyott u-val:", m_bad.params["x"], m_bad.bse["x"])
print("teljes modellel:", m_full.params["x"], m_full.bse["x"])
```

A kihagyott zavaróval a meredekség 2,0 helyett körülbelül 6,5-re torzul, mert a hozzárendelt hatás magába olvasztja u hatását. A HC1 standard hiba lehet kicsi és „szignifikáns”, de ez pontosan a torzított modell magabiztos tévedése. A robusztus standard hiba tehát a variancia hibás specifikációját korrigálja, nem a várható érték rossz változóválasztását. Visszautalva az 5.3. alfejezet pontossági tanulságára: a számítás lehet numerikusan pontos, mégis rossz a céltárgya.

17.3.9 Önellenőrzés

Mielőtt továbblépünk, ellenőrizzük: meg tudjuk-e mondani, hogyan változik a robusztus formula a klasszikushoz képest? Igaz-e, hogy a heteroszkedaszticitás a pontbecslést minden esetben torzítja? Miért nagyobb a HC3 gyakran a HC1-nél? Miért nem elég csak a p-értéket nézni egy kollineáris modellnél? Ha ezekre a kérdésekre a képletekből érvelve válaszolunk, a fogalom rendben van.

17.3.10 Feladatok

1. **Értelmezés.** Adott két output a x meredekségére: klasszikus SE 0,041, HC1 SE 0,056, a pontbecslés mindkettőben 3,97. Magyarázd el, miért tér el a két standard hiba, és miért nem jelent bizonyosságot, ha az egyik intervallum szélesebb.

2. **Megvalósítás.** Írd meg önállóan a HC0 és HC1 képletét a fönti mátrixos származtatásból, és ellenőrizd `np.allclose`-szal a `statsmodels` kimenetét. Add meg a toleranciát és az ellenőrző invariánst.

3. **Hibakeresés.** Egy elemző azt írja: „A robusztus p-érték kisebb mint 0,01, tehát a modell igazolt.” A kódban `work_minutes ~ distance_km` modell szerepel, de a valódi folyamatban a csúcsidőt is kihagyták. Mondd meg, mi a hiba az értelmezésben, és milyen diagnosztikai lépéssel mutatható ki a kihagyott változó.

4. **Megváltozott feltétel.** Vedd az 1. feladat kódját, és adj hozzá egyetlen $x = 30$, munkaperc 500-as kiugró megfigyelést. Számold ki a Cook-távolságot, távolítsd el a pontot, és hasonlítsd össze a robusztus intervallumot a teljes és a tisztított mintán. Mit mutat az eredmény a robusztus SE korlátjáról?

17.3.11 Megoldások és indoklás

1. A klasszikus standard hiba a homoszkedaszticitást feltételezi, ezért az adott szimulációban hibásan kicsi: a hiba a távolsággal nő, és a klasszikus formula a feltételes varianciát egyetlen átlagos értékkel helyettesíti. A HC1 által becsült SE a megfigyelt reziduálisokat használja, ezért ebben az elrendezésben jobban tükrözi a valós bizonytalanságot. **Az irány azonban nem garantált.** Ha a nagy leverage-ű pontok zajszórása kicsi, a klasszikus SE nagyobb lehet a HC1-nél: például $n = 400$, x 360-szor 1 és 40-szer 10, a zaj szórása rendre 10 és 0,05; a klasszikus SE 0,161, a HC1 SE 0,054, az arány 3,0. A helyes állítás tehát az, hogy a klasszikus SE a homoszkedaszticitás feltevése miatt hibás; az alul- vagy felülbecslés iránya a heteroszkedaszticitás és a leverage kapcsolatától függ. A szélesebb intervallum nem jelent nagyobb tudást, hanem más bizonytalanságot tükröz; a módszerek legfeljebb jobban vagy rosszabbul közelítik a valódi mintavételi szórást.

2. A fenti mátrixos kód másolandó, az invariáns a `np.allclose(manual_hc1, fit_hc1.bse, rtol=1e-10, atol=1e-12)`. Ha eltér, először a `nobs/(nobs-p)` faktor hiányát vagy a `diag(e**2)` mátrix szorzáskorlátját kell keresni.

3. A kis p-érték megfelelő feltevések mellett egy konkrét nullhipotézis ellen adhat bizonyítékot, de önmagában nem igazolja a modell egészét, és a kihagyott változóktól nem véd. A kihagyott csúcsidő torzítja a becslést, a p-érték nem mond róla semmit. A hibát reziduális-vs-illesztett ábra mutathatja, amely a kihagyott változóra jellemző sávos vagy torz szerkezetet jelez. A p-érték a minta szélsőségességét méri a nullmodell szerint, nem a nullhipotézis igazságát, ahogy a 16.1. alfejezetben láttuk.

4. A hozzáadott pont messze kívül esik a prediktor tartományán, ezért nagy leverage-e lesz. A Cook-távolság a szokásos küszöb fölé emelkedik, azaz befolyásos pont. A teljes mintán a becsült meredekség eltolódik; a pont eltávolítása után a HC1 intervallum visszaáll a korábbi meredekség közelébe. A robusztus standard hiba tehát nem véd az egyedi befolyásos megfigyeléssel szemben: a diagnosztikát mindig a robusztus futtatás előtt kell végezni.

17.3.12 Rövid összefoglaló

A robusztus standard hiba a White-féle kovarianciaformulára épül, SE-t a megfigyelt reziduálisokból számítja, és nem feltételez azonos hibaszórást. A diagnosztikai ábrák a maradékok alakját tárják fel, a kollinearitás a magyarázó változók közötti redundanciát méri, az influence pedig a leverage és a reziduális együttes hatását. A robusztus módosítás kizárólag a hibaszórást kezeli: torzított várható értékre nincs gyógymód. A munkamenetben a diagnosztikát előbb végezzük el, az eredményt a becsült hatással és az intervallummal együtt jelentjük. A következő alkalmazás a bináris célú modellezés, ahol a logit-modell paramétereit és hibáit ugyanilyen gondossággal kell értelmezni.

17.4 GLM: bináris és számlálási cél

Belépő · szint: haladó · idő: 90 perc · eszköz: statsmodels

Előfeltétel: 17.3. alfejezet · Célkészség: GLM: bináris és számlálási cél · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

17.4.1 Cél és konkrét helyzet

Egy kampányértékelő csapat két kis, szándékosan szintetikus adatkészleten akarja megtanulni, mit tesz a `statsmodels` GLM, ha a célváltozó nem folytonos. A bináris cél egy konverziós jelző, a számlálási cél egy rendelési tételszám. A generáló paraméterek ismertek, ezért a becsült együtthatókat tartalmi értelemben is ellenőrizhetjük.

- **Bináris munkapélda:** 600 megfigyelési egység, minden sor egy független kampány alá vont ügyfél. Bemenetek: `variant` (`control` vagy `treatment`) és `segment` (`consumer` vagy `business`). Cél: `conversion` $Y \in \{0, 1\}$. A generáló lineáris prediktor:

$$\eta_i = -1{,}0 + 0{,}5 \cdot 1_{\text{treatment}} + 0{,}4 \cdot 1_{\text{business}}.$$

  A valódi kezelési hatás ezen a skálán log odds ratio, azaz 0,5.

- **Poisson munkapélda:** 240 megfigyelési egység, minden sor egy megfigyelési ablak. Bemenet: `priority` (0/1). Cél: `item_count`, nemnegatív egész. A generáló várható érték:

$$\lambda_i = \exp(1{,}2 + 0{,}3 \cdot \text{priority}_i).$$

  A priority valódi multiplikatív hatása $\exp(0{,}3) = 1{,}350$, tehát a kiemelt ablakokban átlagosan mintegy 35%-kal több tétel várható.

Induló becslés, mielőtt tovább olvasna: a bináris modell `variant` együtthatója várhatóan 0,3 és 0,9 közé esik, de mit jelent ez a megfigyelési skálán, és miért nem százalékpontos különbség? A Poisson-modell `priority` együtthatója nagyjából 0,3; hány százalékos átlagos emelkedést jelent ez az alapszintű ablakban?

17.4.2 Fogalmi magyarázat: a GLM közös váza

A 17.3. alfejezet OLS-modelljében az átlagos cél lineáris volt, a hibatag pedig normális eloszlású. A GLM ezt általánosítja, miközben a következő szerkezetet tartja meg:

$$g(\mu_i) = \eta_i = \mathbf{x}_i^\top \beta, \qquad \mu_i = E[Y_i \mid \mathbf{x}_i].$$

Itt $\mathbf{x}_i$ az i-edik megfigyelés magyarázó vektor, első eleme 1, β a p darab ismeretlen együttható, η_i a lineáris prediktor, g pedig a **linkfüggvény** (link). A `g` inverzével a várható értéket kapjuk vissza az eredeti kontingens skálán. A link megválasztása az a statisztikai döntés, amely elválasztja a normális lineáris, a logisztikus és a Poisson-modellt.

Az OLS identitáslinket használ, `g(mu) = mu`, normális hibataggal, $\text{Var}(Y_i) = \sigma^2$, ahol a variancia nem függ az átlagtól. A bináris és a Poisson-cél a link skáláján is lineáris, de a variancia a várható érték függvénye, ezért a szokásos lineáris modell csal.

A likelihood visszahívásához: az illesztés a log-likelihoodot,

$$\ell(\beta; y) = \sum_i \ln L_i(\beta; y_i),$$

maximalizálja. A likelihood L_i annak a valószínűsége (illetve sűrűsége), hogy az i-edik megfigyelt cél éppen y_i, ha a modell paramétere β lenne. A maximum likelihood becslés a logaritmus miatt szorzat helyett összeget optimalizál. A becslés pontbecslés; a bizonytalanságát a standard error és a konfidenciaintervallum hordozza, ugyanúgy mint 16.4. alfejezetben.

17.4.2.1 Bináris cél: logistic link és odds

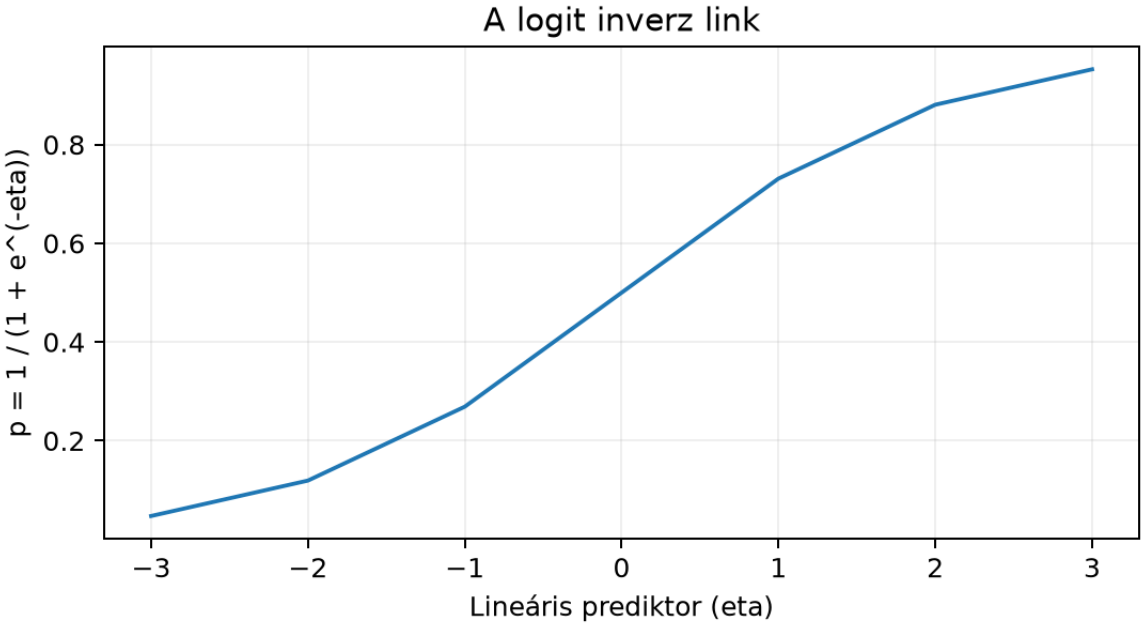
A bináris célra a logit link a leggyakoribb:

$$p_i = P(Y_i = 1 \mid \mathbf{x}_i) = \mu_i, \qquad \log \frac{p_i}{1 - p_i} = \eta_i.$$

Az inverz link a logisztikus függvény:

$$p_i = \frac{1}{1 + e^{-\eta_i}}.$$

A szomszédos ábra a függvény alakját mutatja.



17.1. ábra: A logit inverz link - A pontok a megadott függvényből számított szemléltető értékek, nem mérés.

Az **odds** a kedvező esély arány a kedvezőtlenhez:

$$\text{odds}(p) = \frac{p}{1 - p}.$$

A logit ennek természetes logaritmusa. Kis kézi kontroll: ha $p = 0{,}3$, az odds $3/7 \approx 0{,}429$, a log odds $\ln(0{,}429) \approx -0{,}847$. Ha $\eta = 0$, akkor $p = 0{,}5$, az odds 1 és a log odds 0. Az együttható tehát a log odds additív változása: $\Delta\eta = \beta_j$, ezért minden más változó rögzítése mellett az odds **multiplikatív** változása

$$\text{odds ratio} = \frac{\text{odds}(x_j + 1)}{\text{odds}(x_j)} = e^{\beta_j}.$$

Az interpretáció ezért: egységnyi prediktorváltozás az oddsot e^{β_j}-szeresére változtatja. `statsmodels` `GLM` esetben a becsült együttható a link skálán van; a `predict(which="response")` adja a p_i valószínűséget.

17.4.2.2 Számlálási cél: Poisson mean

A Poisson-cél a nemnegatív egész számokra illik, amikor az egy abszolút gyakoriság. A megfigyelt Y eloszlása:

$$P(Y_i = k \mid \mathbf{x}_i) = \frac{\lambda_i^k e^{-\lambda_i}}{k!}, \qquad \lambda_i = \mu_i = E[Y_i \mid \mathbf{x}_i].$$

A Poisson-ferde jelleg miatt a természetes link a logaritmus:

$$\log \lambda_i = \eta_i, \qquad \lambda_i = e^{\eta_i}.$$

Így az együttható a logaritmikus skálán additív; a várható átlagra vonatkozó hatása multiplikatív:

$$\frac{\lambda(x_j + 1)}{\lambda(x_j)} = e^{\beta_j}.$$

A β_j érték β_j-vel való növelése $100(e^{\beta_j} - 1)$ százalékos átlagos emelkedést jelent. Például $\beta_j = 0{,}3$ esetén $e^{0{,}3} = 1{,}350$, tehát +35,0%. A Poisson-modell a varianciát is az átlaghoz köti: $\mathrm{Var}(Y_i) = \lambda_i$. Ez a kötöttség sok valós számlálási folyamatban sérül; az ilyen **overdispersion** hatását a deviance és a reziduális eltérések jelzik.

17.4.2.3 Deviance mint modellösszehasonlító

A likelihoodhoz tartozó **deviance** (eltérés) a modellt a telített modellhez viszonyítja:

$$D = 2(\ell_{\text{sat}} - \ell_{\text{model}}).$$

A telített modell minden megfigyeléshez szabad paramétert ad, ezért a log-likelihood a lehető legnagyobb. Minél kisebb a deviance, annál jobb a modell — de a deviance önmagában nem egy 0–1 közötti jósági mutató, és függ a mintamérettől. Bináris célra a telített modell $\ell_{\text{sat}} = 0$, ezért

$$D_{\text{bin}} = -2 \sum_i \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right].$$

Poisson-célra az ℓ_{sat} konstans elhagyásával a szokásos számítás:

$$D_{\text{pois}} = 2 \sum_i \left[y_i \ln \frac{y_i}{\lambda_i} - (y_i - \lambda_i) \right],$$

ahol a $0 \ln 0$ tagot 0-nak vesszük. A **null deviance** a csak interceptet tartalmazó modell deviance-a. A kettő különbsége a likelihood alapú modellösszehasonlítás alapja: ha az új prediktor a nullmodellhez képest a likelihoodot számottevően növeli, a bővített modell deviance-a kisebb. Poisson-célra a kisebb deviance a df-hez viszonyítva 1-hez közeli, ha a modell a feltételt teljesíti.

17.4.2.4 Interpretation, avagy a skálák egymás mellett

A link skálán a hatás additív, a válasz skálán általában nem. A bináris modellben β_j log odds ratio, a Poisson-modellben log rate ratio. A predikció közlésekor ezért mindig meg kell adni a skálát: a `which="linear"` a lineáris prediktort, a `which="response"` a várható célváltozót adja. A konfidenciaintervallum a link skálán szimmetrikus, de a válasz skálán a nemlineáris inverz link miatt másképp viselkedik.

17.4.3 Teljes minimális munkapélda

Az alábbi kód seedelt `numpy` generátorokat használ, ezért a generálási sorrend adott Python/NumPy környezetben reprodukálható. A kódot a tanuló a saját gépén futtathatja; itt csak a **szemléltető, elméletileg várt kimenet** szerepel, mert a szöveggenerálás nem futtatja a kódot, és egy seed sem garantál minden platformon bitazonos lebegőpontos véletlent.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

# --- Bináris cél: konverzió ---
rng_bin = np.random.default_rng(20261020)
n = 600
variant = np.tile(["control", "treatment"], n // 2)
segment = np.where(rng_bin.uniform(size=n) < 0.5, "consumer", "business")
eta_true = -1.0 + 0.5 * (variant == "treatment") + 0.4 * (segment == "business")
p_true = 1.0 / (1.0 + np.exp(-eta_true))
conversion = rng_bin.binomial(1, p_true)

df_bin = pd.DataFrame({
    "variant": pd.Categorical(variant),
    "segment": pd.Categorical(segment),
    "conversion": conversion,
})

# --- Poisson-cél: tételszám kiemelt ablakonként ---
rng_pois = np.random.default_rng(7)
n2 = 240
priority = rng_pois.binomial(1, 0.4, size=n2).astype(int)
rate_true = np.exp(1.2 + 0.3 * priority)
item_count = rng_pois.poisson(rate_true)

df_pois = pd.DataFrame({
    "priority": priority,
    "item_count": item_count,
})

# Illesztés és link skálán értelmezendő együtthatók
model_bin = smf.glm(
    "conversion ~ C(variant) + C(segment)",
    data=df_bin,
    family=sm.families.Binomial(),
).fit()

model_pois = smf.glm(
    "item_count ~ priority",
    data=df_pois,
    family=sm.families.Poisson(),
).fit()
```

```
print("--- Bináris GLM, szemléltető lehetséges futás ---")
print(model_bin.params.round(3))
print("Deviance:", round(model_bin.deviance, 3))
print("Null deviance:", round(model_bin.null_deviance, 3))
print()
print("--- Poisson GLM, szemléltető lehetséges futás ---")
print(model_pois.params.round(3))
print("Deviance:", round(model_pois.deviance, 3))
print("Null deviance:", round(model_pois.null_deviance, 3))
```

Egy lehetséges futás szemléltető kimenete:

```
--- Bináris GLM, szemléltető lehetséges futás ---
Intercept                 -1.01
C(variant)[T.treatment]    0.47
C(segment)[T.business]     0.36
Deviance: 781.2
Null deviance: 807.5

--- Poisson GLM, szemléltető lehetséges futás ---
Intercept                  1.19
priority                   0.31
Deviance: 241.3
Null deviance: 255.6
```

A `C()` a formula nyelvében arra kéri a Patsy réteget, hogy a `variant` és a `segment` oszlopokat kategorikus változóként kezelje. A referencia szintek ábécésorrend szerint `control` és `consumer`, ezért az együttható a treatment és a business additív hatását adja hozzá az intercepthez.

17.4.3.1 Kézi kontroll: a link inverze

A link skálán adott $\hat{\eta}_i$ és a válasz skálán adott predikció egyezését a modell saját paramétereiből ellenőrizzük. Bináris esetben a kézi számítás

$$\hat{p}_i = 1/(1 + e^{-\hat{\eta}_i}),$$

Poisson esetben $\hat{\lambda}_i = e^{\hat{\eta}_i}$. A következő kód a minta minden sorára ugyanazt az eredményt várja, mint a `predict(which="response")`.

```
# Bináris kézi mátrix: Intercept, treatment, business
X_bin = np.column_stack([
    np.ones(n),
    (df_bin["variant"] == "treatment").astype(float),
    (df_bin["segment"] == "business").astype(float),
])
eta_bin = X_bin @ model_bin.params.values
p_hat = 1.0 / (1.0 + np.exp(-eta_bin))

# Poisson kézi mátrix: Intercept, priority
X_pois = np.column_stack([
    np.ones(n2),
    df_pois["priority"].astype(float),
])
eta_pois = X_pois @ model_pois.params.values
lambda_hat = np.exp(eta_pois)

# Invariáns: a kézi és a modell-predikció egyezik
assert np.allclose(model_bin.predict(which="response"), p_hat, atol=1e-10)
assert np.allclose(model_pois.predict(which="response"), lambda_hat, atol=1e-10)
```

```
# Skálatulajdonságok
assert (p_hat > 0).all() and (p_hat < 1).all()
assert (lambda_hat > 0).all()

# Deviance kézi reprodukálása
y_bin = df_bin["conversion"].values
D_bin = -2.0 * np.sum(y_bin * np.log(p_hat) + (1.0 - y_bin) * np.log(1.0 - p_hat))
y_pois = df_pois["item_count"].values
D_pois = 2.0 * np.sum(
    np.where(y_pois > 0, y_pois * np.log(y_pois / lambda_hat), 0.0)
    - (y_pois - lambda_hat)
)
assert np.allclose(D_bin, model_bin.deviance, atol=1e-8)
assert np.allclose(D_pois, model_pois.deviance, atol=1e-8)
```

Az `assert` akkor nem jelez, ha a tanuló a fenti kódot futtatva a saját futás paramétereivel számol. A tolerancia a lebegőpontos műveletek és a statsmodels belső optimalizálás közötti kerekítés miatt van.

17.4.4 Eredmény értelmezése

A bináris modellben az `Intercept -1.01` a referencia csoport, tehát a `control` és `consumer` csoport becsült log oddsa. Innen

$$\hat{p}_{\text{control,consumer}} = 1/(1 + e^{1{,}01}) \approx 0{,}267.$$

A treatment együttható 0,47 log odds ratio; az odds ratio $\exp(0{,}47) \approx 1{,}60$. Tehát a kezelt ügyfelek oddsa mintegy 60%-kal nagyobb, mint az azonos szegmensbe tartozó kontrollügyfeleké. A business együttható 0,36; $e^{0{,}36} \approx 1{,}43$, tehát a business szegmensben az odds 43%-kal nagyobb. A konkrét valószínűségi növekmény ettől eltér: a treatment csoportban a consumer réteg becsült $\eta = -1{,}01 + 0{,}47 = -0{,}54$, tehát $p \approx 0{,}368$; a növekmény a reference 0,267-hez képest 10,1 százalékpont. Ugyanez a business rétegben más numerikus növekményt adhat, mert a hatás a válasz skálán nem konstans: $\hat{\eta} = -1{,}01 + 0{,}47 + 0{,}36 = -0{,}18$, $p \approx 0{,}455$.

A Poisson-modell `priority` együtthatója 0,31 log rate ratio; $e^{0{,}31} \approx 1{,}363$. A kiemelt ablakokban a várható tételszám a nem kiemelthez képest 36,3%-kal nagyobb. Az alapablak várható tételszáma $\exp(1{,}19) \approx 3{,}29$; a kiemelt ablaké $3{,}29 \cdot 1{,}363 \approx 4{,}48$. A predikció a válasz skálán tehát nem az együttható hozzáadásával, hanem az exponenciális linkkel kapható meg.

A deviance a bináris példában a nulldeviancához képest kisebb, de önmagában nem jelenti azt, hogy a modell „jól” magyarázd a konverziót. A Poisson-példában a modell deviance 241,3 a reziduális szabadságfokkal $240 - 2 = 238$; ez az 1 körüli eltérés azt jelzi, hogy a generálás pontosan Poisson, tehát a szórásfeltétel ezen a kis szintetikus mintán nem sérül durván.

A konfidenciaintervallumok a link skálán szimmetrikusak; a válasz skálán a végpontok inverz linkkel transzformáltak. A `get_prediction().summary_frame()` kimenete ezt a válasz skálán jeleníti meg, ezért a `mean` oszlop a $\hat{p}$-t vagy $\hat{\lambda}$-t tartalmazza, a csúcsok pedig a megadott névleges szintű intervallumot.

GLM-ellenőrzés: binomiális logitnál az együttható log-odds skálájú, exp( ) esélyhányados, nem kockázati arány. Poisson-modellnél az exp( ) feltételes várható számlálás aránya; a túlzott diszperzió ellenőrzése nélkül a standard hibák félrevezetők lehetnek.

17.4.5 Feladatok

1. **Értelmezés.** Egy másik bináris GLM-illesztés paraméterei: Intercept $-2{,}0$, treatment $0{,}7$. (a) Számítsd ki a kezelt és a kontroll csoport becsült valószínűségét. (b) Számítsd ki az odds ratio-t, a kockázati arányt és a százalékpontos különbséget. Melyik egyenlő az $\exp(0{,}7)$ értékkel?

2. **Implementálás.** A fenti `model_bin` modellt használva kérje le a `get_prediction().summary_frame(alp` egy oszlopát a válasz skálán. Ellenőrizd, hogy a `mean` oszlop numerikusan illeszkedik a kézzel számolt `p_hat` tömbhöz, és a `mean_ci_lower`/`mean_ci_upper` oszlopok a $[0, 1]$ tartományba esnek. Miért nem feltétlenül szimmetrikus a CI a `mean` körül?

3. **Hibadiagnózis.** Egy kolléga a Poisson-modell `priority` együtthatóját, 0,31-et, így értelmezi: „a kiemelt ablakokban 0,31 plusz tétel van". Nevezd meg a hibát, javítsa ki, és mutassa meg számszerűen, miért függ az abszolút többlet az alapszinttől.

4. **Megváltozott feltétel.** Az adatgyűjtés megváltozik: minden sor egy ügyfél, de a megfigyelési ablak hossza nem azonos; a táblában `events` a megfigyelt események száma, `days` pedig a megfigyelési napok száma. A cél a napi eseményráta kezelés előtti és utáni aránya. Adj meg `statsmodels` formulás GLM-et a megfelelő offsettel, magyarázd, miért log offset kell, és add meg az ellenőrző invariánst.

17.4.6 Megoldások és indoklás

1. megoldás. Az intercept a kontrollcsoport log oddsa: $\eta_0 = -2$, tehát $\hat{p}_0 = 1/(1+e^2) \approx 0{,}119$. A kezelt csoport $\eta_1 = -2 + 0{,}7 = -1{,}3$, $\hat{p}_1 = 1/(1+e^{1{,}3}) \approx 0{,}214$. Az odds ratio $\exp(0{,}7) \approx 2{,}014$, ez az egyetlen, ami közvetlenül az exponenciális együtthatóval egyenlő. A kockázati arány $\hat{p}_1/\hat{p}_0 = 0{,}214/0{,}119 \approx 1{,}80$; a százalékpontos különbség $0{,}214 - 0{,}119 = 0{,}095$, azaz 9,5 százalékpont. A valós generáló hatás ezen a konkrét ponton ezért nem 0,7 és nem 101,4% valószínűségnövekedés.

2. megoldás.

```
frame_bin = model_bin.get_prediction().summary_frame(alpha=0.05)

# Invariáns: a summary_frame mean oszlopa a response skálán van
assert np.allclose(frame_bin["mean"].values, p_hat, atol=1e-10)
assert frame_bin["mean_ci_lower"].between(0, 1).all()
assert frame_bin["mean_ci_upper"].between(0, 1).all()

# A link skálán definiált intervallum a nemlineáris inverz link miatt
# a válasz skálán csak akkor lesz pontosan szimmetrikus, ha p = 0.5.
print(frame_bin[["mean", "mean_ci_lower", "mean_ci_upper"]].head(3))
```

A bináris modellben a link skálán az intervallum ugyanolyan széles a pont körül, de az inverz logit balról jobbra lapos, ezért a $\hat{p}$ körüli távolság a válasz skálán a tartomány belsejében eltérő. A konkrét statsmodels verzió `summary_frame` viselkedését a tanulónak érdemes a saját környezetében `help(model_bin.get_prediction().summary_frame)` paranccsal ellenőriznie; az állítás maga a skálatranszformáció matematikája.

3. megoldás. A hiba háromszoros. Először is az együttható a logaritmikus link skálán van, ezért önmagában nem az események átlagos számához adódik. Másodszor a link inverze miatt a hatás multiplikatív: $e^{0{,}31} = 1{,}363$, azaz a várható tételszám 36,3%-kal nagyobb. Harmadszor az abszolút többlet a baseline rate függvénye. Ha a nem kiemelt ablak várható tételszáma $\lambda_0 = \exp(1{,}19) \approx 3{,}29$, akkor a kiemelt várható érték $\lambda_1 = 3{,}29 \cdot 1{,}363 \approx 4{,}48$, a többlet 1,19

tétel. Ha a baseline $\lambda_0 = 10$, akkor a kiemelt várható 13,63, a többlet 3,63. Az additív 0,31 ezért semmilyen ponton nem helytálló a válasz skálán.

4. megoldás. Ha a megfigyelési ablak hossza `days`, akkor a Poisson-modell várható eseményszáma

$$\lambda_i = \text{days}_i \cdot e^{\eta_i}.$$

Logaritmálva:

$$\log \lambda_i = \log(\text{days}_i) + \eta_i.$$

A $\log(\text{days}_i)$ tag ismert konstans, ezért a modellbe offsetként kerül. A megoldás mintapéldája:

```
rng4 = np.random.default_rng(11)
n4 = 180
treatment = rng4.binomial(1, 0.5, size=n4)
days = rng4.choice([1, 3, 7], size=n4)
rate4 = np.exp(0.0 + 0.2 * treatment)
events = rng4.poisson(rate4 * days)
df4 = pd.DataFrame({"treatment": treatment, "days": days, "events": events})

model4 = smf.glm(
    "events ~ treatment",
    data=df4,
    family=sm.families.Poisson(),
    offset=np.log(df4["days"]),
).fit()

# Invariáns: a várható events egyenlő days * exp(eta)
X4 = np.column_stack([np.ones(n4), treatment.astype(float)])
eta4 = X4 @ model4.params.values
expected_events = df4["days"].values * np.exp(eta4)
assert np.allclose(model4.predict(which="response"), expected_events, atol=1e-8)

# A napi ráta aránya a kezelés esetén:
rr = np.exp(model4.params["treatment"])
print("becsült napi rate ratio:", round(rr, 3))

# Expozíció-duplázás invariáns: azonos treatment mellett a dupla days
# kétszeres várható eseményszámot ad.
new = pd.DataFrame({"treatment": [0, 0], "days": [2, 4]})
pred = model4.predict(new, offset=np.log(new["days"]))
assert np.allclose(pred.iloc[1], 2 * pred.iloc[0], atol=1e-10)
```

Túlzott szóródás ellenőrzése. A Poisson-modell a varianciát az átlaghoz köti, ezért az illesztés után érdemes a Pearson χ^2/df arányt nézni. Ha ez jóval 1 felett van, a Poisson standard hibák túl optimisták; ilyenkor negatív binomiális (NB2) modell lehet a jobb választás. Szimulált, túlszórt adaton ($n = 600$, napok $\{1, 3, 7\}$, $\alpha = 0{,}5$ NB2-szórással) a Poisson treatment-együttható 0,209, SE 0,0388; az NB-modell ugyanazt a pontbecslést adja, de SE 0,0727, a Pearson $\chi^2/\text{df} = 3{,}29$, az AIC pedig Poissonnál 3440,7, NB-nél 2783,9. A pontbecslés tehát hasonló, a bizonytalanság és a modellválasztás nem:

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

rng_od = np.random.default_rng(20261020)
```

```
n_od = 600
days_od = rng_od.choice([1, 3, 7], size=n_od, p=[0.4, 0.35, 0.25])
treat_od = rng_od.binomial(1, 0.5, size=n_od)
mu_od = np.exp(0.2 + 0.3 * treat_od) * days_od
alpha_od = 0.5
lam_od = rng_od.gamma(shape=1 / alpha_od, scale=alpha_od * mu_od)
events_od = rng_od.poisson(lam_od)
df_od = pd.DataFrame({"treatment": treat_od, "days": days_od, "events": events_od})

model_od = smf.glm(
    "events ~ treatment", data=df_od, family=sm.families.Poisson(),
    offset=np.log(df_od["days"]),
).fit()
nb_od = smf.negativebinomial(
    "events ~ treatment", data=df_od, offset=np.log(df_od["days"])
).fit(disp=0)
pearson = model_od.pearson_chi2 / model_od.df_resid
print("Pearson chi2/df:", round(pearson, 2))
print("Poisson AIC:", round(model_od.aic, 1), "NB AIC:", round(nb_od.aic, 1))
print("Poisson SE:", model_od.bse["treatment"], "NB SE:", nb_od.bse["treatment"])
```

Az expozíció-duplázás invariánsa és a túlzott szóródás ellenőrzése két külön kapu: az előbbi a log-offset helyességét, az utóbbi a varianciafeltevés érvényességét vizsgálja.

Az offset nélkül a modell a nyers eseményszámot magyarázná, és a hosszabb ablakok nagyobb célértéke a kezeléshez nem kapcsolódó hatást torzítaná. A `predict(which="response")` itt a várható eseményszámot adja; a várható napi ráta $\exp(\eta_i)$. A becsült treatment együttható a napi ráta log aránya, a `rate ratio` pedig a megfelelő exponenciális.

17.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés: **az odds ratio nem kockázati arány és nem százalékpontos különbség.** Példa: OR $= 2$ és $p_0 = 0{,}5$. Ekkor $\text{odds}_0 = 1$, $\text{odds}_1 = 2$, ezért $p_1 = 2/(1+2) = 0{,}667$. A kockázati arány 1,33, a százalékpontos különbség 16,7 százalékpont; egyik sem 2 és nem 100%. Más induló szintnél ugyanez az OR más abszolút különbséget ad. Ez ugyanaz a skálaugrás, mint amikor a 16.4. alfejezet kísérleti összefoglalójában a kockázati különbség volt a primer hatás; a logisztikus regresszióban viszont az OR a természetes interpretációs skála.

A 17.3. alfejezet robusztus standard hibájának logikája itt is érvényes: a p-érték a nullmodell szerinti feltételes szélsőség, nem a paraméter igaz valószínűsége. A `cov_type="HC0"` változat a GLM-ben is kérhető, de a modell szerkezeti feltételeit nem cseréli le. A bináris és Poisson GLM korlátja, hogy a variancia nem szabad paraméter; ha a Poisson-eloszlás szórása a gyakorlatban nagyobb, a szokásos Wald-intervallum túl optimista lehet, és a deviance-t sem szabad R^2-ként olvasni.

17.4.8 Önellenőrzés

Az eredmény akkor helyes, ha (1) a `predict(which="response")` a bináris modellnél a $[0, 1]$ tartományban, a Poissonnál pozitív tartományban marad; (2) a kézi $\exp(\hat{\eta})$ és $\text{expit}(\hat{\eta})$ `np.allclose` módon illeszkedik; (3) a modell deviance nem nagyobb, mint a null deviance, mert a bővített modell likelihoodja legalább akkora; (4) minden értelmezés megnevezi, hogy log oddsról, odds ratio-ról, kockázati arányról vagy válasz-skálás átlagról van szó.

17.4.9 Rövid összefoglaló

A GLM a lineáris prediktort linkfüggvénnyel kapcsolja a cél várható értékéhez. A bináris célhoz tartozó logit link a log oddsot modellezi, ezért az együttható exponense odds ratio; a Poisson-cél logaritmusos linkje a várható eseményszámot, az együttható exponense pedig rate ratio-t ad. A deviance a likelihood összehasonlító mértéke, de önmagában nem abszolút jósági mutató. Az értelmezést mindig az adott skálán kell megadni. A következő alkalmazásban ezt a GLM-keretet oksági kísérleti értelmezéssel kötjük össze, ahol a kontrollváltozók és a randomizálás szerepe egészíti ki a becslést.

18 Kísérlettervezés és oksági kérdések

18.1 Oksági cél és megfigyelési torzítás

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: NumPy, pandas, statsmodels

Előfeltétel: 17.4. alfejezet · Célkészség: Oksági cél és megfigyelési torzítás · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A cél egy konkrét oksági kérdés: ha a Northstar-rendeléseket `priority_flag=1` (elsőbbségi kezelés) beavatkozással látnánk el, mennyivel változna a késés valószínűsége a `target_late` kimenetben. A megfigyelési egység egy rendelés. Az input a `orders.csv` sémája: `order_id`, `customer_id`, `order_ts`, `promised_days=7`, `quantity_sum`, `basket_minor`, `shipping_fee_minor`, `distance_km`, `priority_flag`, továbbá utólag ismert `delivery_days` és `target_late`. A predikciós projekt céljától eltérően itt nem azt kérdezzük, hogy egy rendelés várhatóan késni fog-e, hanem azt, hogy a `priority_flag` megváltoztatása okoz-e késéscsökkenést.

A nyers megfigyelési asszociáció $P(target_late = 1 \mid priority_flag = 1) - P(target_late = 1 \mid priority_flag = 0)$ nem automatikus oksági hatás. Ha például a nagyobb távolságú rendelések gyakrabban kapnak prioritást, és a távolság a késést is növeli, akkor a prioritásos csoport más `distance_km`-összetételű, mint a nem prioritásos. Az induló feladat: mielőtt a módszertant látná, fogalmazzon meg legalább egy ilyen közös okot, és mondja meg, milyen irányba torzíthatja a nyers kockázati különbséget a `distance_km`, ha a nagyobb távolság növeli mind a `priority_flag` esélyét, mind a `target_late` esélyét.

18.1.1 Potenciális kimenet és a beavatkozás jelölése

Minden $i = 1, \dots, n$ rendeléshez két potenciális kimenet tartozik:

- $Y_i(0)$: késésindikátor, ha a rendelés nem kap prioritást ($A_i = 0$),
- $Y_i(1)$: késésindikátor, ha a rendelés prioritást kap ($A_i = 1$).

Itt $Y_i(a) \in \{0, 1\}$ bináris, tehát nincs mértékegysége. A beavatkozás $A_i \in \{0, 1\}$ a ténylegesen megfigyelt kezelés. A fundamental problem of causal inference, hogy egy rendelésen a két érték közül egyszerre csak az egyik figyelhető meg; a megfigyelt kimenet:

$$Y_i = A_i Y_i(1) + (1 - A_i) Y_i(0).$$

A jobb oldal elemenkénti vektor: ha $A_i = 1$, az első tag marad; ha $A_i = 0$, a második. Egy kis kontrollpélda:

A	$Y(0)$	$Y(1)$	megfigyelt Y
0	0	1	0

A	$Y(0)$	$Y(1)$	megfigyelt Y
1	1	1	1
0	0	0	0
1	1	0	0

A $Y(0), Y(1)$ oszlopok hipotetikusak; a táblázatban csak A és Y mérhető. Az egyéni oksági hatás $\tau_i = Y_i(1) - Y_i(0)$, ami bináris kimenetnél -1, 0 vagy 1 lehet. Az átlagos oksági hatás, angolul average treatment effect, a populációban:

$$ATE = E[Y(1) - Y(0)] = E[Y(1)] - E[Y(0)].$$

A fenti négy soron a potenciális különbségek: 1, 0, 0, -1, tehát a példabeli ATE $-0{,}00$? Pontosabban $(-1 + 0 + 0 + 1)/4 = 0$? Ellenőrizzük: $Y(1) - Y(0)$ soronként $1, 0, 0, -1$; összeg 0, átlag 0. Ez jó emlékeztető: az egyéni hatások lehetnek nem nullák, de az átlag lehet nulla.

A *beavatkozás* (intervention) nem passzív megfigyelés, hanem a kezelés külső beállítása; a jelölés $do(A = 1)$, amit a potenciális kimeneti nyelvben $Y(1)$ jelöl. Ha egy gráfban a `priority_flag` csomópontra beavatkozunk, a belé mutató nyilak eltűnnek, mert a kezelést ezentúl nem a távolság vagy a kosárérték határozza meg.

18.1.2 Zavaró, mediátor, collider és azonosíthatóság

Egy egyszerű irányított aciklikus gráf, DAG, az ok-okozati feltevéseket rögzíti. A nyíl jelentése: közvetlen ok. Három szerepet kell élesen elkülöníteni.

A **zavaró változó** (confounder) közös ok: hat a kezelésre és a kimenetre egyaránt. A rendelési példában a `distance_km` zavaró, mert nagyobb távolságnál gyakrabban választanak prioritást, és a távolság a késést is növeli; így a `priority_flag` és a `target_late` közötti nyers kapcsolat részben a távolságból származik. Gráfosan $C \to A$ és $C \to Y$. Ha C megfigyelt, akkor a $A \leftarrow C \to Y$ hátsó ajtó út a C-re való rétegzéssel vagy kiigazítással blokkolható. A rétegzett kiigazítási, standardizálási formula bináris Y esetén:

$$E[Y(a)] = \sum_{c} P(Y = 1 \mid A = a, C = c)\, P(C = c), \qquad a \in \{0, 1\},$$

ahol c a zavaró lehetséges értékeit futja be. A képlet a feltételes kockázatokat a zavaró populációs eloszlásával súlyozza; a becsült ATE a két standardizált érték különbsége. A $P(C = c)$ itt a teljes mintabeli marginális arány, nem az A-feltételes arány.

A **mediátor** (közvetítő változó) a kezelés és a kimenet közötti ok-okozati lánc közbülső lépése: $A \to M \to Y$. Ha például az elsőbbségi kezelés korábbi `dispatch_hour` csomagfeladást okoz, és ez csökkenti a késést, akkor a `dispatch_hour` mediátor. A teljes oksági hatás tartalmazza a mediátoron átmenő közvetett hatást is; ha a modellben a mediátorra is feltételesítünk, blokkoljuk a lánc egy részét, ezért a becslés nem a teljes hatás lesz. A teljes hatás becslésekor a mediátor *tiltott kontroll*.

A **collider** (ütköző változó) két másik változó közös okozata: $A \to C \leftarrow Y$. Ha egy colliderre feltételesítünk, nem-oksági asszociáció nyílhat meg a két ok között. A klasszikus kis példa: legyen a panasz C értéke 1, ha a rendelés prioritást kapott vagy késett; csak a panasznál megfigyelt rendeléseket látjuk. A $C = 1$ szűrés után az A és Y negatív asszociációt mutat, pedig a teljes populációban függetlenek lehetnek. A colliderre való feltételesítés tehát szelekciós torzítást okoz;

ez is tiltott kontroll, kivéve ha kifejezetten a kiválasztási folyamatot akarjuk korrigálni, és a teljes cella ismert.

Az **azonosíthatóság** (identification) az a kérdés, hogy a megfigyelt eloszlásból — a DAG-feltevésekkel együtt — egyértelműen meghatározható-e a cél oksági mennyiség. A confounder-DAG esetén igen, ha C mérve van, mert a kiigazítási formula azonosítja az $E[Y(a)]$ értékeket. Ha van nem mért zavaró U, amely A-ra és Y-ra is hat, akkor az azonosítás nem teljes; a kiigazítás nem távolítja el a torzítást. A collider-szelekciós példában a $C = 0$ cella hiányzik, ezért a teljes populáció ATE-je nem azonosítható a kiválasztott mintából. A regressziós kiigazítás (17.1. alfejezet) akkor oksági, ha a DAG hátsó ajtó útjai blokkoltak és a modell funkcionális formája helyes; a p-érték önmagában nem azonosítási bizonyíték. A feltételes valószínűség (14.1. alfejezet) itt úgy tér vissza, hogy a megfigyelt $P(Y = 1 \mid A = 1)$ nem azonos a beavatkozás utáni $P(Y(1) = 1)$ értékkel; a különbség éppen a közös okból származó torzítás.

18.1.2.1 Időrendi kapu: mit szabad kontrollálni?

A teljes oksági hatás becslésénél csak a **kezelés előtt** mért, a hátsó ajtó utat blokkoló változók jöhetnek szóba. A kezelés utáni változók tiltott kontrollok, mert a kezelés hatásának részét viszik el (mediátor), vagy torzítást nyitnak (collider). A rendelési példa időrendje:

Változó	Mérés ideje	Szerep	Kontrollálható a teljes hatáshoz?
`distance_km`	kezelés előtt	zavaró (confounder)	igen, mert blokkolja a hátsó ajtó utat
`priority_flag`	kezelés	kezelés	nem, ez maga a kezelés
`dispatch_hour`	kezelés után	mediátor	nem, a közvetett utat blokkolná
panasz (`C`)	kezelés után	collider	nem, szelekciós torzítást nyitna
`target_late`	kezelés után	kimenet	nem, ez a célváltozó

A megfelelő DAG nyilakkal:

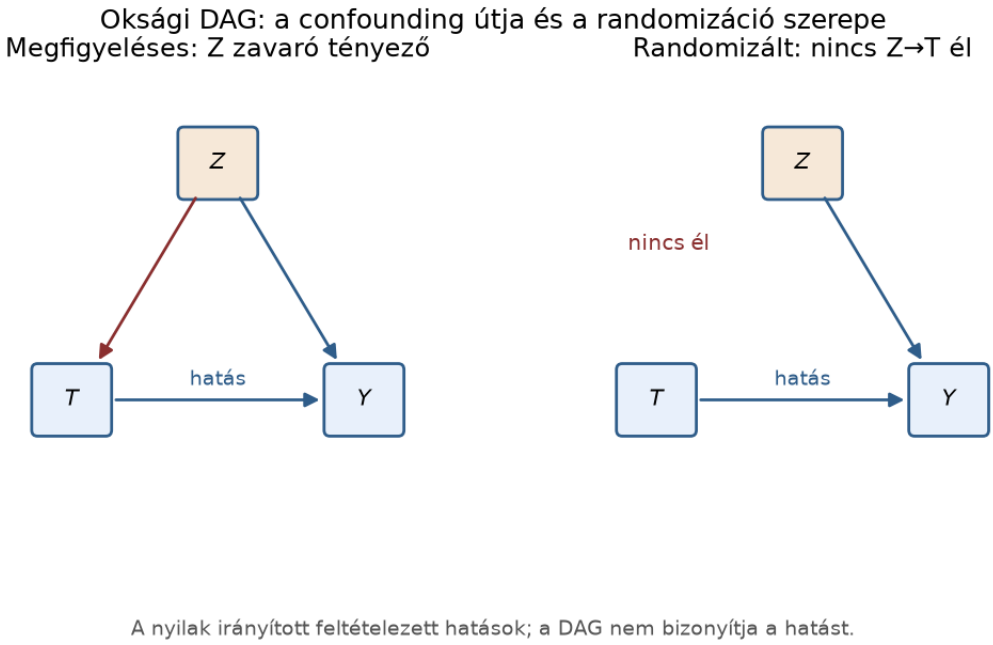
```
distance_km --> priority_flag --> dispatch_hour --> target_late
      \_____________________________________________^

priority_flag --> panasz <-- target_late
```

Az első ábrán a `dispatch_hour` a `priority_flag` és a `target_late` közötti mediátor: ha a regresszióba betesszük, a becslés a közvetlen hatás felé tolódik, nem a teljes hatást adja. A második ábrán a `panasz` collider: ha csak a panaszos eseteket látjuk ($C = 1$), a `priority_flag` és a `target_late` között nem oksági asszociáció jelenik meg. A kezelés előtti időkorlát tehát nem formai szabály, hanem az azonosítás része: a posztkezeléses változó csak akkor kerülhet a modellbe, ha a célmennyiség maga is közvetlen vagy mediált hatás, és ezt a célt előre deklaráltuk.

A matematikai híd röviden: a regresszió a feltételes várható értéket becsli, de a feltételes várható érték csak akkor egyenlő a potenciális kimenet várható értékével, ha a feltételhalmaz blokkolja a hátsó ajtó utakat. Ellenkező esetben a jó illeszkedésű modell is torzított következtetést ad.

18.1.2.2 Fogalmi ábra: confounding és a randomizáció hatása a DAG-ra



18.1. ábra: Oksági DAG két panelen: megfigyeléses esetben Z-ből él vezet T-hez és Y-hoz, ezért a T-Y asszociációt confounding torzítja; randomizáció után a Z→T él hiányzik, így a T→Y él a beavatkozás hatását azonosítja.

Az ábra a hátsó ajtó gondolatát mutatja: a zavaró tényezőből induló út nyitva hagyja a nem oksági asszociációt, a randomizáció viszont megszakítja a zavaró tényező és a kezelés közötti élt. A konklúzió ábra nélkül is ugyanaz: az oksági értelmezéshez a beavatkozás előtti időrend, a blokkoló feltételhalmaz vagy randomizáció kell; a jó prediktív illeszkedés nem elég. A teljes generátor: `lab/figures/make_concept_figures.py` `causal_dag()`; a reprodukálható mag:

```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch

fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.4))
for ax, randomized in zip(axes, (False, True)):
    ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.axis("off")
    for x, y, label in [(0.5, 0.82, "Z"), (0.2, 0.35, "T"), (0.8, 0.35, "Y")]:
        ax.text(x, y, label, ha="center", va="center", fontsize=14,
                bbox=dict(boxstyle="round", facecolor="#e8f0fb"))
    edges = [((0.54, 0.76), (0.75, 0.42)), ((0.28, 0.35), (0.72, 0.35))]
    if not randomized:
        edges.append(((0.46, 0.76), (0.25, 0.42)))
    for start, end in edges:
        ax.add_patch(FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=11))
    ax.set_title("randomizált" if randomized else "megfigyeléses")
plt.show()
```

18.1.3 Teljes minimális munkapélda: két folyamat, asszociáció eltérő okból

Az alábbi szimuláció két DAG-ot állít elő. Mindkettőben van megfigyelt asszociáció A és Y között, de az ok teljesen más: az elsőben zavaró okozza, a másodikban collider-szelekció. A szimuláció

a Python standard numerikus könyvtáraival, `numpy` és `pandas` segítségével fut; a seed rögzíti a véletlent. A kimenet szemléltető, nem ebben a generálási módban futtatott mérés.

Az első DAG: $C \to A$, $C \to Y$, és nincs $A \to Y$ nyíl; a valódi ATE 0.

A második DAG: $A \to C$, $Y \to C$, ahol $C = 1$ jelenti a megfigyelt panaszcellát; a megfigyelt minta csak $C = 1$. A valódi ATE 0, mert A és Y független a teljes populációban.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(20261020)
n = 60_000

# 1. folyamat: confounder; valódi ATE = 0
# C -> A, C -> Y; nincs A -> Y
c1 = rng.binomial(1, 0.5, n)
a1 = rng.binomial(1, 0.2 + 0.4 * c1, n)   # P(A=1|C=0)=0.2, P(A=1|C=1)=0.6
y1 = rng.binomial(1, 0.1 + 0.3 * c1, n)   # P(Y=1|C=0)=0.1, P(Y=1|C=1)=0.4

df1 = pd.DataFrame({"A": a1, "Y": y1, "C": c1})

# Nyers kockázati különbség A=1 vs A=0
rd_raw_1 = (
    df1.loc[df1.A == 1, "Y"].mean()
    - df1.loc[df1.A == 0, "Y"].mean()
)

# Standardizálás a confounder C szerint
def standardized_rd(df):
    cell = {}
    for c in [0, 1]:
        for a in [0, 1]:
            cell[(a, c)] = df.loc[(df.C == c) & (df.A == a), "Y"].mean()
    p_c = df.C.value_counts(normalize=True).sort_index()
    e_y1 = cell[(1, 0)] * p_c[0] + cell[(1, 1)] * p_c[1]
    e_y0 = cell[(0, 0)] * p_c[0] + cell[(0, 1)] * p_c[1]
    return e_y1 - e_y0

rd_adj_1 = standardized_rd(df1)

# 2. folyamat: collider; valódi ATE = 0
# A és Y független; C = A OR Y, és csak C=1-et figyelünk meg
a2 = rng.binomial(1, 0.5, n)
y2 = rng.binomial(1, 0.5, n)
c2 = np.maximum(a2, y2)           # C=1, ha A=1 vagy Y=1
df2_all = pd.DataFrame({"A": a2, "Y": y2, "C": c2})
df2_obs = df2_all[df2_all.C == 1]

rd_all_2 = (
    df2_all.loc[df2_all.A == 1, "Y"].mean()
    - df2_all.loc[df2_all.A == 0, "Y"].mean()
)
rd_obs_2 = (
    df2_obs.loc[df2_obs.A == 1, "Y"].mean()
    - df2_obs.loc[df2_obs.A == 0, "Y"].mean()
)

print("1. confounder: nyers RD =", round(rd_raw_1, 3),
      "| C-re igazított RD =", round(rd_adj_1, 3))
print("2. collider: teljes populáció RD =", round(rd_all_2, 3),
      "| C=1 megfigyelt RD =", round(rd_obs_2, 3))
```

A várható, elméleti kimenet a megadott generáló valószínűségekből:

```
1. confounder: nyers RD = 0.125 | C-re igazított RD = 0.0
2. collider: teljes populáció RD = 0.0 | C=1 megfigyelt RD = -0.5
```

A szimulációs zaj miatt $n = 60\,000$ esetén a harmadik tizedesjegyben $\pm 0{,}01$ nagyságrendű eltérés elfogadható; a kimenetet helyi futtatással kell rögzíteni.

18.1.4 Eredmény értelmezése

A confounder-folyamatban a nyers kockázati különbség várhatóan 0,125, pedig az A-ból Y-ra nincs közvetlen hatás. Kézi kontroll: a DAG szerint A és Y feltételesen függetlenek C-től, ezért mindkét C-rétegben ugyanaz a feltételes kockázat, és a standardizálás pontosan nullát ad. Az analitikus nyers érték is ellenőrizhető: a súlyozott feltételes valószínűségekből $P(Y = 1 \mid A = 1) = 0{,}325$ és $P(Y = 1 \mid A = 0) = 0{,}200$, különbség 0,125.

A collider-folyamatban a teljes populációban A és Y független, ezért az RD nulla; a $C = 1$ szűrt mintában viszont a közös ok-okozat feltétele negatív asszociációt generál. A determinisztikus $C = A \vee Y$ esetben a $C = 1$ minta kontingenciatáblája a három lehetséges cellát $(1, 1), (1, 0), (0, 1)$ egyenlő arányban tartalmazza, a $(0, 0)$ cella hiányzik, ezért $A = 1$-nél $Y = 1$ arány $1/2$, $A = 0$-nál 1, különbség $-0{,}5$. Ez a szám a kiválasztási torzítás, nem oksági hatás.

A labor-átadandó követelmény is ez: két folyamatot rajzolunk, amelyek erős, de okilag eltérően keletkező asszociációt adnak. Az elsőnél a következtetési korlát az, hogy a hatás csak akkor azonosítható, ha a zavaró C mérve van; a másodiknál az, hogy a $C = 1$-re szűrt minta önmagában nem azonosítja a teljes ATE-t, mert a nullcellát nem látjuk.

Azonosítási feltételek: megfigyeléses kiigazítással oksági hatás csak konzisztencia, feltételes kicserélhetőség (nincs kezeletlen zavarás) és pozitivitás/overlap megfelelő változatai mellett azonosítható. A DAG önmagában nem bizonyítja, hogy ezek az empirikus esetben teljesülnek.

18.1.5 Feladatok

1. **Értelmezés.** Adott a DAG: `distance_km -> priority_flag`, `distance_km -> target_late`, `priority_flag -> dispatch_hour`, `dispatch_hour -> target_late`.

   (a) Mi a `distance_km` és mi a `dispatch_hour` szerepe? (b) Mely változó(k) szerint kell igazítani, ha a `priority_flag` teljes oksági hatását akarjuk a `target_late`-re? (c) Mi történik, ha a regresszióban a `dispatch_hour` is szerepel?

2. **Implementálás.** Írj szimulációt, amelyben $A \to M \to Y$ és $A \to Y$ is van, nincs zavaró. Legyen $P(A = 1) = 0{,}4$, $P(M = 1 \mid A) = 0{,}3 + 0{,}4A$, és $P(Y = 1 \mid A, M) = 0{,}1 + 0{,}2A + 0{,}2M$. Használj `n=50_000` és `seed=7` értéket. Számítsd ki a nyers kockázati különbséget, az M-re standardizált rétegzett becslést, és add meg a teljes ATE elméleti értékét.

3. **Hibadiagnózis.** Egy elemző az 1. feladat DAG-jához illeszti a `smf.logit('target_late ~ priority_flag + dispatch_hour + distance_km', data=orders).fit()` modellt. A `priority_flag` együtthatójára `p<0.001` adódik, ezért azt állítja, hogy a prioritás beavatkozás hatásos. Soroljon fel legalább két okot, amiért ez az állítás nem következik a modellből, és javasoljon egy ellenőrzést.

4. **Megváltozott feltétel.** A collider-példában most a naplófájl a $C = 0$ eseteket is tartalmazza, tehát a teljes `df2_all` tábla rendelkezésre áll. Módosítsd az elemzést úgy, hogy a teljes adaton számol, és értelmezze az ATE-t. Mi változik a DAG-ban?

18.1.6 Megoldások és indoklás

1. (a) A `distance_km` zavaró (confounder), mert hat a kezelésre és a kimenetre is. A `dispatch_hour` mediátor, mert a `priority_flag` rajta keresztül is hat a késésre. (b) A teljes hatáshoz csak a zavarót kell blokkolni: `distance_km` szerint rétegezni vagy regresszióban kontrollálni. (c) A `dispatch_hour` kontrollja blokkolja a `priority_flag -> dispatch_hour -> target_late` közvetett utat, ezért a becslés a teljes hatásnál kisebb, jellemzően a közvetlen hatáshoz közelebbi értéket ad; a teljes hatás becslésekor ez tiltott kontroll.

2. A módosított kód:

```
rng = np.random.default_rng(7)
n = 50_000
a3 = rng.binomial(1, 0.4, n)
m3 = rng.binomial(1, 0.3 + 0.4 * a3, n)
y3 = rng.binomial(1, 0.1 + 0.2 * a3 + 0.2 * m3, n)

df3 = pd.DataFrame({"A": a3, "M": m3, "Y": y3})

rd_raw_3 = (
    df3.loc[df3.A == 1, "Y"].mean()
    - df3.loc[df3.A == 0, "Y"].mean()
)

# M-re standardizált rétegzett becslés
cell3 = {}
for m in [0, 1]:
    for a in [0, 1]:
        cell3[(a, m)] = df3.loc[(df3.M == m) & (df3.A == a), "Y"].mean()
p_m = df3.M.value_counts(normalize=True).sort_index()
e_y1_m = cell3[(1, 0)] * p_m[0] + cell3[(1, 1)] * p_m[1]
e_y0_m = cell3[(0, 0)] * p_m[0] + cell3[(0, 1)] * p_m[1]
rd_adj_m = e_y1_m - e_y0_m

print("nyers RD =", round(rd_raw_3, 3),
      "| M-re igazított RD =", round(rd_adj_m, 3))
```

Az elméleti teljes ATE kiszámítása: $E[Y(0)] = 0{,}3 \cdot 0{,}3 + 0{,}7 \cdot 0{,}1 = 0{,}16$, $E[Y(1)] = 0{,}7 \cdot 0{,}5 + 0{,}3 \cdot 0{,}3 = 0{,}44$, tehát ATE $= 0{,}28$. A nyers RD várt értéke 0,28, mert nincs zavaró; az M-re igazított RD várt értéke 0,20, mert minden M-rétegben a generáló képlet A-együtthatója 0,2. A különbség a mediátoron átmenő közvetett hatás blokkolásából adódik. A tűrés az $n = 50\,000$-es szimulációhoz körülbelül $\pm 0{,}01$.

3. A hibák: (a) a `dispatch_hour` mediátor kontrollja torzítja a teljes hatást; (b) a megfigyelési adat nem randomizált, ezért a feltételes regressziós együttható csak akkor oksági, ha minden zavaró mérve van és a logit modell jól specifikált; a p-érték a nullmodell szerinti szélsőség, nem oksági azonosítás (16.4. alfejezet); lehetnek nem mért zavarók. Ellenőrzésként randomizált kísérlet vagy a jövő alfejezetben tárgyalt kísérleti protokoll szerinti allokáció használható.

4. A teljes `df2_all` mintán a nyers RD várhatóan 0, mert A és Y függetlenek a generáló folyamatban. A collider-szelekció megszűnik, mert a $(A = 0, Y = 0)$ cella is megfigyelt. A

DAG-ban nincs többé a $C = 1$-re való feltételesítés, tehát az ATE azonosítható, és a becslés nulla. Az azonosíthatóságot a teljes megfigyelés állítja helyre, nem a modell bonyolultsága.

18.1.7 Gyakori tévedés ellenpéldával

Jó prediktív pontosságból nem következik oksági azonosíthatóság. Az első szimulációban a `priority_flag`-ből jobban jósolható a `target_late`, mint a puszta alaparányból: az $A = 1$ csoportbeli késési arány 0,325, az $A = 0$ csoportbeli 0,200, szemben a 0,25-ös marginálissal. Egy modell, amely az A-t használja, csökkenti a Brier-veszteséget, mégis a valódi beavatkozási hatás nulla. A collider-mintában ugyanez a helyzet: a modell a $C = 1$ adaton hasznos prediktort talál, de a teljes populációban a kezelés nem befolyásolja a kimenetet. A predikció a megfigyelt eloszlást tanulja; az oksági kérdés a meg nem figyelt potenciális kimenetekről szól. A regresszió (17.1. alfejezet) és a GLM (17.4. alfejezet) tehát nem teszik automatikusan okságivá a következtetést; ehhez a DAG azonosítási feltételei kellenek.

Önellenőrzés: ellenőrizd, hogy a confounder-standardizálásnál a $P(C = c)$ súlyokat a teljes mintából vette-e, nem az A-feltételes csoportból. Ha a súlyokat felcseréli, a torzító összetételhatást újra bevezeti. A collider-példában pedig kérdezze meg: a $(0, 0)$ cella a szűrés miatt hiányzik, vagy a valós mechanizmusban nem is lehetséges? A válasz eldönti, hogy a teljes ATE azonosítható-e.

18.1.8 Rövid összefoglaló

A potenciális kimenet $Y(a)$ a beavatkozás utáni kimenetet írja le, a megfigyelt $Y = A_i Y_i(1) + (1 - A_i)Y_i(0)$. A zavaró közös ok; rétegzéssel blokkolható, ha mérve van, és a kiigazítási formula a feltételes kockázatokat a zavaró marginális eloszlásával súlyozza. A mediátor és a collider tiltott kontroll a teljes hatás becslésekor: előbbi blokkolja a közvetett utat, utóbbi szelekciós asszociációt nyit. Azonosíthatóság nélkül a jó predikció sem oksági következtetés. Következő lépésként 18.2. alfejezetben a randomizálást és a kísérleti protokollt építjük fel, amely a zavaró feltételezése nélkül teszi azonosíthatóvá az ATE-t.

18.2 Randomizálás és kísérleti protokoll

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: NumPy, pandas, statsmodels

Előfeltétel: 18.1. alfejezet · Célkészség: Randomizálás és kísérleti protokoll · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

18.2.1 Cél és konkrét helyzet

A 18.1. alfejezetben azonosítottuk a zavaró változókat és azt, hogy megfigyeléses adatból csak erős feltételezésekkel lehet oksági hatást becsülni. Most a Northstar kezdőlabor `lab/data/assignments.csv` és `lab/data/outcomes.csv` fájljaira építünk egy **előre rögzített kísérleti protokollt**. A bemenet ügyfélszintű: a 300 ügyfél 2025-07-01-én kapott `control` vagy `treatment` hozzárendelést, majd mindenkinél ugyanazt a 7 napos kimenetet figyeltük meg. A primer endpoint a `conversion_7d` (0/1), az elemzési egység az ügyfél, a cél az átlagos kezelési hatás (average treatment effect, ATE) becslése.

Előzetes becslés: ha a kontroll-konverzió 0.24, a kezelési hatás 0.10, és fejenként 150-150 ügyfél van, akkor a normal approximation standard error nagyjából

$$\sqrt{\frac{0.24 \cdot 0.76}{150} + \frac{0.34 \cdot 0.66}{150}} \approx 0.052.$$

A 95%-os konfidenciaintervallum szélessége tehát kb. 0.21; a 0.10-es hatás nem lesz élesen kimutatható. Ezért a protokollt becslési és döntési keretként írjuk elő, nem pedig „biztos nyertesként”.

18.2.2 Fogalmi magyarázat

18.2.2.1 A randomizálás egysége

A **randomizálás egysége** (unit of randomization) az a megfigyelési egység, amelyet véletlenszerűen rendelünk valamelyik karba. Itt ez a `customer_id`. Azért nem a rendelés vagy a tételsor, mert ugyanazon ügyfél rendelései és tételei korrelálnak: ha egy ügyfelet kezelünk, az összes rendelésére hatással lehetünk. A `lab/data/order_items.csv` tehát nem szaporíthatja fel a mintanagyságot; a kísérleti nevező továbbra is 300 ügyfél.

Kis kézi kontroll: ha 10 ügyfélből 5-öt sorsolunk treatmentbe, a randomizálási egységek száma 10. Ha minden ügyfélnek átlagosan 4 rendelési tétele van, attól még a független kísérleti egység nem 40, mert a 4 tétel egy közös sorsolás következménye.

18.2.2.2 Exposure és ITT

Az **exposure** a tényleges beavatkozásnak való kitettség, például az ajánlat megjelenése, az email megnyitása vagy a push értesítés kézbesítése. A **randomizálás** a szándékolt hozzárendelést, a `variant` oszlopot adja; a tényleges kitettség ettől eltérhet.

Az **intention-to-treat** (ITT) elv szerint minden ügyfelet abban a karban elemzünk, ahová randomizálták, függetlenül attól, hogy a kezelést ténylegesen megkapta-e. Ez megőrzi a randomizálás előnyét: a két kar összehasonlíthatóságát. Ha csak a ténylegesen kitett ügyfeleket vizsgálnánk, a kitettség maga is lehet a kezelés következménye, és újra megjelenhet a zavaró hatás.

A laborban az ITT nevező az összes randomizált ügyfél: mind a 300 sor. A primer becslés a treatment és a control kar `conversion_7d` átlagának különbsége.

18.2.2.3 Guardrail

A **guardrail** őrszem-mutató: olyan előre rögzített másodlagos kimenet, amelynek romlása esetén a bevezetést leállítjuk, még akkor is, ha a primer mutató javul. A mi protokollunkban a `revenue_7d_minor` ilyen: ha a treatment kar átlagos 7 napos bevétele több mint 5%-kal a kontroll alá esik, a döntés `block`.

Fontos, hogy a guardrail szabályát az adatok megtekintése előtt rögzítsük. Utólag választott guardrail esetén bármilyen véletlen ingadozást „kockázatnak” nevezhetnénk, és a döntés nem lenne reprodukálható.

18.2.2.4 Attrition

Az **attrition** az ügyfelek kiesése a megfigyelési ablakból: hiányzó kimenet, kilépés, adatvédelmi tiltakozás. A Northstar laborban nincs ilyen: minden randomizált ügyfélhez tartozik teljes, fix ablakú `outcomes.csv` sor. A protokollban mégis előírjuk a kezelést: ha egy `conversion_7d` hiányozna, az ügyfél az ITT nevezőben marad, a hiányzó konverziót 0-ként imputáljuk, és a döntés mellett hiányzási riportot készítünk. Ez konzervatív szabály; más imputáció érzékenységvizsgálatként szerepelhet.

18.2.2.5 Interference

Az **interference** azt jelenti, hogy egy ügyfél kimenete függ más ügyfelek hozzárendelésétől. A randomizált kísérlet standard becslője feltételezi a **SUTVA** feltevést (stable unit treatment value assumption): minden egység potenciális kimenete csak a saját hozzárendelésétől függ. Ha például a kezelés egy nyilvános weboldali kedvezmény, a kontroll ügyfelek is láthatják, így a kontroll kimenete már nem a tiszta $Y(0)$.

A mi mesterséges laborunkban a kezelés ügyfélszintű és elszigetelt; a protokollban ezt feltevésként rögzítjük, és a kísérleti jelentésben megnevezzük mint érvényességi korlátot.

18.2.2.6 Protokoll-ellenőrzőlista

A kísérlet megkezdése előtt a következő ellenőrzőlistát kell kitölteni; a lista a későbbi jelentés bizonyítékjegyzéke is.

Elem	Kérdés	Northstar-válasz	Bizonyíték
Allocation concealment	A hozzárendelés a felvétel előtt rejtett volt?	szimulált Bernoulli-kiosztás; valós kísérletnél központi sorsolás	assignment protokoll, seed
Mintaáramlás	Hány randomizált, hány elemzett?	300 randomizált, 300 elemzett	`assignments.csv`, `outcomes.csv`
ITT	Minden randomizált a saját karjában marad?	igen, a 300 sor mindegyike	one-to-one join
Noncompliance	Eltér a tényleges kezelés a hozzárendeléstől?	a laborban nincs; a lenti szimulációban ~10%	compliance napló
Hiányzó kimenet	Hány kimenet hiányzik, és mi a szabály?	0 hiányzó; ha lenne: ITT-ben marad, 0 imputáció, riport	missingness tábla
Interferencia	SUTVA teljesül-e?	igen, elszigetelt ügyfélszintű kezelés	design leírás
Protokoll-eltérés	Történt-e eltérés?	nincs; minden eltérést rögzíteni kell	deviációs napló

18.2.2.7 Noncompliance: ITT vs tényleges kezelés

Ha a kijelölt kezelést nem mindenki kapja meg, a tényleges kezelés (`D`) és a hozzárendelés (`A`) elválik. Az ITT továbbra is `A` szerint hasonlít, ezért megőrzi a randomizálás érvényességét; a „ténylegesen kezelt vs nem kezelt” összehasonlítás viszont már nem randomizált. Szimulált példa ($n = 50\,000$, seed `20261020`): $A \sim \text{Bernoulli}(0{,}5)$, a compliance valószínűsége $\text{logistic}(2{,}4 + 0{,}8U)$, ahol $U \sim N(0,1)$ a compliance-szel és a kimenettel is összefügg; $D = A \cdot C$ és $Y = 0{,}2D + U + \varepsilon$. A compliance aránya 89,6%, tehát a treatmentre kiosztottak mintegy 10,4%-a nem kapja meg a kezelést. Eredmények:

Becslés	Érték	Értelmezés
ITT: $E[Y \mid A=1] - E[Y \mid A=0]$	0,194	a design szerinti ITT-hatás (tervezett 0,18), torzítatlan a randomizálás miatt
Naiv: $E[Y \mid D=1] - E[Y \mid D=0]$	0,350	erősen torzított, mert a compliance U-val függ össze
Per-protocol a complierek körében	0,210	a complier average causal effect (CACE), nem az ITT

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(20261020)
n = 50_000
u = rng.normal(0, 1, n)
a = rng.binomial(1, 0.5, n)
p_c = 1 / (1 + np.exp(-(2.4 + 0.8 * u)))
c = rng.binomial(1, p_c, n)
d = a * c
y = 0.2 * d + 1.0 * u + rng.normal(0, 1, n)
df = pd.DataFrame({"A": a, "D": d, "C": c, "Y": y})
itt = df.loc[df.A == 1, "Y"].mean() - df.loc[df.A == 0, "Y"].mean()
naive = df.loc[df.D == 1, "Y"].mean() - df.loc[df.D == 0, "Y"].mean()
compliers = df[df.C == 1]
pp = (compliers.loc[compliers.A == 1, "Y"].mean()
      - compliers.loc[compliers.A == 0, "Y"].mean())
print(df.C.mean(), itt, naive, pp)
```

A tanulság: az ITT a randomizált populáció hatását méri, a per-protocol a complierek hatását, a naiv as-treated összehasonlítás pedig elveszti a randomizálás védelmét. A jelentésben ezért a primer eredmény ITT, a compliance- és per-protocol elemzés külön, feltáró vagy érzékenységi szakaszban szerepel.

18.2.3 Matematikai híd: függetlenség, hatásméret és erő

A randomizálás legfontosabb következménye a **függetlenség**:

$$A \perp\!\!\!\perp (Y(0), Y(1), X),$$

ahol $A = 1$ treatment, $A = 0$ control, $Y(0)$ és $Y(1)$ a potenciális kimenetek, X pedig minden, a randomizálás előtt mért változó. Emiatt nincs szükség zavaró változóra való kiigazításra; az egyszerű átlagkülönbség torzítatlanul becsli az ATE-t:

$$\Delta = E[Y(1)] - E[Y(0)].$$

A mintabeli becslő:

$$\hat{\Delta} = \bar{Y}_1 - \bar{Y}_0,$$

ahol $\bar{Y}_1$ a treatment kar `conversion_7d` átlaga, $\bar{Y}_0$ a kontroll karé. Mindkettő arány, dimenzió nélküli, 0 és 1 közötti érték. A standard error független karok esetén:

$$SE(\hat{\Delta}) = \sqrt{\frac{p_0(1-p_0)}{n_0} + \frac{p_1(1-p_1)}{n_1}},$$

ahol p_0 és p_1 a két kar konverziós aránya, n_0 és n_1 a karméretek.

A **kísérleti erő** (power) annak a valószínűsége, hogy a teszt elutasítja a nullhipotézist, ha a valódi hatás Δ. Kétoldali 5%-os szinten, 80%-os erőhöz tartozó minimálisan detektálható hatás közelítőleg:

$$\delta_{MDE} \approx (z_{1-\alpha/2} + z_{1-\beta})\sqrt{\frac{2p(1-p)}{n/2}},$$

ahol $z_{1-\alpha/2} = 1.96$, $z_{1-\beta} = 0.84$, p a tervezett kontrollarány, n a teljes minta. A mi 300 ügyfelünkre $p = 0.25$ mellett a gyökös tag $\sqrt{2 \cdot 0.25 \cdot 0.75/150} \approx 0.05$, így $\delta_{MDE} \approx 0.14$. A 0.10-es hatás kimutatásához tehát a kísérlet alul-powered; a protokoll ezt explicit korlátként tartalmazza.

18.2.4 Teljes minimális munkapélda: a protokoll és az ellenőrzés

A protokoll döntési lapja:

Elem	Rögzített érték
Randomizálási egység	`customer_id`
Hozzárendelés	1:1 `control`/`treatment`, 2025-07-01
Primer endpoint	`conversion_7d`
ITT nevező	mind a 300 randomizált ügyfél
Megfigyelési ablak	fix 7 nap, `observed_until = 2025-07-08T00:00:00+00:00`
Döntési küszöb	bootstrap 95% CI alsó határ $> 0.02 \rightarrow$ `rollout`
Guardrail	`revenue_7d_minor` treatment átlag >= 0.95 * kontroll átlag

A döntési szabály tehát: ha a 95%-os percentilis bootstrap CI alsó határa nagyobb, mint 0.02, javasoljuk a bevezetést; ha a CI felső határa kisebb, mint 0.02, elutasítjuk; különben `collect_more`. A guardrail sérelme esetén a döntés mindig `block`.

Az alábbi kód szemléltető: a kimeneti számok elméletileg vártak, nem mért futási eredmények. Futtatási környezet: projektgyökér, ahol a `lab/` mappa található.

```python
# s070_protocol.py - szemléltető, nem futtatott kód
import numpy as np
import pandas as pd

ASSIGN_PATH = "lab/data/assignments.csv"
OUTCOME_PATH = "lab/data/outcomes.csv"
EXPECTED_OBSERVED_UNTIL = "2025-07-08T00:00:00+00:00"
BOOT_SEED = 20260701

def load_and_validate():
    assign = pd.read_csv(ASSIGN_PATH)
    outcomes = pd.read_csv(OUTCOME_PATH)
    df = assign.merge(outcomes, on="customer_id", validate="one_to_one")
    errors = []
    if len(df) != 300:
        errors.append(f"Várt 300 sor, kapott {len(df)}")
    if not df["customer_id"].is_unique:
        errors.append("customer_id nem egyedi")
    if set(df["variant"]) != {"control", "treatment"}:
        errors.append("variant értékkészlet hibás")
    if df["conversion_7d"].isna().any():
        errors.append("conversion_7d hiányzik")
    if (df["observed_until"] != EXPECTED_OBSERVED_UNTIL).any():
        errors.append("observed_until eltér a fix ablaktól")
    if (df["revenue_7d_minor"] < 0).any():
        errors.append("negatív revenue_7d_minor")
    return df, errors

def bootstrap_diff_ci(df, n_boot=2000, seed=BOOT_SEED):
    rng = np.random.default_rng(seed)
    control = df.loc[df["variant"] == "control", "conversion_7d"].to_numpy()
    treatment = df.loc[df["variant"] == "treatment", "conversion_7d"].to_numpy()
    diffs = np.empty(n_boot)
    for i in range(n_boot):
        c = rng.choice(control, size=len(control), replace=True)
        t = rng.choice(treatment, size=len(treatment), replace=True)
        diffs[i] = t.mean() - c.mean()
    return np.quantile(diffs, [0.025, 0.975])

def evaluate_protocol(df, n_boot=2000, seed=BOOT_SEED):
    control = df.loc[df["variant"] == "control", "conversion_7d"]
    treatment = df.loc[df["variant"] == "treatment", "conversion_7d"]
    delta = treatment.mean() - control.mean()
    se = np.sqrt(
        treatment.mean() * (1 - treatment.mean()) / len(treatment)
        + control.mean() * (1 - control.mean()) / len(control)
    )
    ci_lo, ci_hi = bootstrap_diff_ci(df, n_boot, seed)
    control_rev = df.loc[df["variant"] == "control", "revenue_7d_minor"].mean()
    treatment_rev = df.loc[df["variant"] == "treatment", "revenue_7d_minor"].mean()
    guardrail_ok = treatment_rev >= 0.95 * control_rev
    if not guardrail_ok:
        decision = "block"
    elif ci_lo > 0.02:
        decision = "rollout"
    elif ci_hi < 0.02:
        decision = "reject"
    else:
        decision = "collect_more"
    return {
        "delta": delta,
        "se": se,
        "ci": (ci_lo, ci_hi),
```

```
        "guardrail_ok": guardrail_ok,
        "decision": decision,
    }

df, errors = load_and_validate()
print(errors)          # elvártan üres lista
if not errors:
    result = evaluate_protocol(df)
    print(result)
```

A kód ellenőrzi a protokoll feltételeit, majd kiszámítja az ITT becslést. A bootstrap csoporton belüli: minden ismétlésben külön-külön mintavételezünk a kontroll és a treatment karból, mert a karok méretét a protokoll rögzíti, és a randomizálási egység az ügyfél.

18.2.5 Eredmény értelmezése

A merge után a DataFrame 300 soros; a `validate="one_to_one"` garantálja, hogy minden ügyfélhez pontosan egy kimenet tartozik. A `conversion_7d` dtype várhatóan integer (0/1), a `variant` object típusú kategória.

Kézi kontroll: a normal approximationból számolt `se` és a bootstrap CI közelítőleg összhangban van; 300 mintánál a percentilis bootstrap CI nem feltétlenül szimmetrikus, de a két módszer ugyanazt a nagyságrendet adja. A becsült `delta` a generáló folyamat miatt várhatóan 0.10 körül mozog, de a véletlen miatt ettől több százalékponttal is eltérhet. A döntési szabály ezért nem a pontbecslést, hanem a CI és a 0.02-es küszöb viszonyát nézi.

Ha a guardrail megsérül, a döntés `block` akkor is, ha a primer hatás pozitív. Ez a protokoll előre rögzített része; az adat megtekintése után nem módosítható.

Kísérleti minőségbiztosítás: külön kell választani a randomizált kiosztást, a tényleges expozíciót és a kimenet elérhetőségét. ITT-elemzésnél a randomizált nevezőt kell alapul venni; a hiányzó kimenetek nem kezelhetők automatikusan nullásként.

18.2.6 Feladatok

1. **Értelmezés.** Miért nem lehet a kísérlet elemzési egysége a rendelési tétel vagy a rendelés, ha a `lab/data/order_items.csv`-t összekapcsoljuk az ügyfelekkel? Magyarázd meg az ITT nevező és a függetlenség szempontjából.
2. **Implementálás.** Írj `validate_protocol(df)` függvényt, amely a fenti ellenőrzéseken túl azt is vizsgálja, hogy a `variant` karonként 150-150 ügyfelet tartalmaz, és hogy minden `assigned_at` értéke `2025-07-01T00:00:00+00:00`. A függvény hibák listájával térjen vissza; adj meg egy invariant, amellyel a kimenetet ellenőrzöd.
3. **Hibakeresés.** Egy elemző csak azokat az ügyfeleket tartja meg, akiknél `revenue_7d_minor > 0`, mert szerinte csak ők „aktívak”, és a bevételi hatást csak rajtuk akarja mérni. Mi a hiba? Adj egy 10+10 ügyféles számpéldát, ahol az ITT bevételi hatás +48 cent, de a szűrt becslés +35 cent.
4. **Megváltozott feltétel.** Tegyük fel, hogy a kezelés egy nyilvános weboldali kedvezmény, amelyet a kontroll ügyfelek is láthatnak, ha ugyanabban a régióban élnek. Melyik protokollfeltétel sérül? Javasolj alternatív protokollt, és nevezd meg annak korlátját.
5. **Transzfer.** A primer endpoint `revenue_7d_minor`-ra változik. Hogyan módosul a standard error képlete és a döntési szabály? Milyen hatásméretet használnál?

18.2.7 Megoldások és indoklás

1. megoldás. A randomizálás ügyfélszintű; a rendelések és tételek egy ügyfélhez tartoznak, tehát nem független megfigyelések. Ha a tételeket külön sorként kezelnénk, a minta látszólag 4796 sorra nőne, de az azonos ügyfélhez tartozó sorok közös sorsolást örökölnek, így a standard error alulbecsült lenne, és a CI túl szűk. Az ITT nevező ezért mindig a randomizálási egységek száma: 300 ügyfél.

2. megoldás. Egy lehetséges függvény:

```
def validate_protocol(df):
    errors = []
    if df["variant"].value_counts().to_dict() != {"control": 150, "treatment": 150}:
        errors.append("A karok mérete nem 150-150")
    if (df["assigned_at"] != "2025-07-01T00:00:00+00:00").any():
        errors.append("assigned_at eltér a protokolltól")
    return errors
```

Invariant: ha a bemenet a tiszta laborfájl, a hibák listája üres. Ellenőrzés: a `value_counts().to_dict()` pontosan a két várt kulcsot adja; a dátum-összehasonlítás stringként is működik, mert az ISO formátum fix.

3. megoldás. A szűrés exposure utáni változón alapul: a `revenue_7d_minor > 0` maga is a kezelés következménye lehet. Ha a kezelés növeli a vásárlási valószínűséget, a treatment karból több ügyfél kerül a szűrt halmazba, és a két csoport összetétele már nem véletlenszerű. Számpélda: 10 treatment ügyfél közül 6 vásárol, revenue-k 100, 110, 120, 130, 140, 150 cent; 4 nem vásárol (0). Treatment ITT átlag: $6 \cdot 125/10 = 75$. Kontroll: 10-ből 3 vásárol, revenue-k 80, 90, 100 cent; ITT átlag: $3 \cdot 90/10 = 27$. A valódi ITT hatás $75 - 27 = 48$ cent. Ha csak a pozitív revenue-jű sorokat nézzük: treatment átlag 125, kontroll átlag 90, különbség 35 cent. A szűrés eltünteti a konverziónövelő hatás egy részét.

4. megoldás. Az interference sérül: a kontroll ügyfelek kimenete nem a tiszta $Y(0)$, mert ők is látják a kedvezményt, ami a kontroll-konverziót is megemelheti, így az ATE alulbecsült. Alternatíva: régiószintű klaszter-randomizálás, ahol az egész régió kap vagy nem kap kezelést. Ennek korlátja: a Northstar adatban csak 3 régió van, ami túl kevés független klaszter a megbízható következtetéshez; nagyobb mintában viszont a klaszteren belüli korrelációt figyelembe vevő standard error kell. Egy másik alternatíva: a kezelést egyénileg elszigetelni, például emailben küldött egyedi kuponnal.

5. megoldás. Folytonos kimenetre a standard error:

$$SE(\hat{\Delta}) = \sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}},$$

ahol s_0^2 és s_1^2 a karok mintabeli varianciája, centben kifejezve. A döntési szabály ugyanaz maradhat, de a 0.02-es küszöb helyett pénzben rögzített minimális gyakorlati hatás kell, például 20 cent. Hatásméretként Cohen-féle d is használható: $\hat{\Delta}/s_{pooled}$, ahol s_{pooled} a két kar egyesített szórása.

18.2.8 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az **exposure utáni szelekció**: a randomizálás után csak azokat az ügyfeleket elemezni, akiknél a kezelés „működött”, vagy akiknél van megfigyelt aktivitás. A randomizálás az eredeti karok összehasonlíthatóságát adja; ha utána szűrünk, a szűrés függhet a kezeléstől, és a 18.1. alfejezetben megismert collider torzítás lép fel. A fenti 3. feladat számpéldája mutatja, hogy a valódi +48 centes ITT hatásból a szűrt becslés +35 cent lesz. Az ITT elv pontosan ezt védi ki: minden randomizált ügyfél a saját karjában marad, akkor is, ha nem kapta meg a kezelést vagy nem konvertált.

Önellenőrzés: tudd megnevezni a randomizálási egységet a laborban; tudd, miért nem a rendelési tétel; tudd, mi a különbség a hozzárendelés és a tényleges exposure között; tudd, hogy a guardrail szabályát mikor kell rögzíteni; tudd, melyik feltevés sérül, ha a kontroll ügyfelek is látják a kezelést.

18.2.9 Rövid összefoglaló

A randomizált kísérlet protokollja a randomizálási egységet, a hozzárendelést, a primer endpointot, az időablakot és a döntési szabályt előre rögzíti. Az ITT elv szerint minden randomizált ügyfél a saját karjában marad; a tényleges exposure szerinti szűrés újra zavaró hatást vihet be. A guardrail a bevezetést védi, az attrition és az interference pedig érvényességi feltétel. A Northstar 300 ügyfele 1:1 arányban alkalmas a hatás becslésére, de 80%-os erővel csak kb. 14 százalékpontos hatást tudna kimutatni, ezért a döntés CI-alapú és a 0.02-es küszöbhöz kötött. Következő lépésként 18.3. alfejezetben megfigyeléses becslést és érzékenységvizsgálatot nézünk, amikor randomizálás nem áll rendelkezésre.

18.3 Megfigyeléses becslés és érzékenység

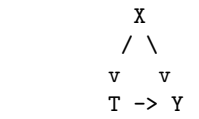
> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: NumPy, pandas, statsmodels**
>
> Előfeltétel: 18.2. alfejezet · Célkészség: Megfigyeléses becslés és érzékenység · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

18.3.1 Cél és konkrét helyzet

Egy termékesapat e-mail kampányt vezetett be, de a kezelést nem randomizálták: a marketingrendszer a legaktívabb ügyfeleknek küldte az ajánlatot. Az elemző azt a kérdést kapja: „mennyi bevételt hoz a kampány a kontrollhoz képest?” A megfigyelési egység egy ügyfél, a kezelés $T \in \{0, 1\}$ (küldtek-e ajánlatot), a kimenet Y a 7 napos bevétel euróban, a zavaró változó X az előzetes aktivitás (a kampány előtti megnyitások száma). A generatív folyamat ismert: az aktivitás növeli a küldés esélyét és a bevételt is; a kampány valódi hatása $\tau = 1.0$ euró. A mért adatban az X-et ismerjük, a kimenetet is, de a kezelés kiosztása nem véletlen. Előzetes becslés: a nyers átlagkülönbség felfelé torzít, mert X pozitívan hat T-re és Y-ra; az X-re korrigált becslés várhatóan 1.0 körüli. A feladat: becsüljük meg a hatást, vizsgáljuk a közös tartományt (overlap), majd rejtsünk el egy második zavaró változót, és mutassuk meg, hogy a korrekció ekkor már nem elegendő.

18.3.2 Fogalmi magyarázat

A megfigyeléses becslés központi gondolata a **hátsó ajtó intuíció** (backdoor intuition). A DAG-on:



Az X egyszerre hat a kezelésre és a kimenetre. A nyers $E[Y|T=1]-E[Y|T=0]$ különbség nem csak a $T \to Y$ élen haladó hatást méri: tartalmazza az X-en átmenő „hátsó ajtó” utat is. Ha X-et blokkoljuk, azaz az összehasonlítást azonos X-értékű rétegeken belül végezzük, a hátsó út megszakad. Ezt nevezzük **korrekciónak** (adjustment). A becsült átlagos oksági hatás (ATE, average treatment effect):

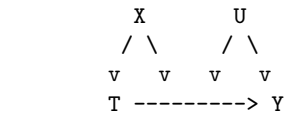
$$\hat{\tau}_{adj} = \hat{E}_X\left[\hat{E}(Y|T=1,X=x) - \hat{E}(Y|T=0,X=x)\right].$$

Itt $E(Y|T=t,X=x)$ a kimenet várható értéke a $T=t$ és $X=x$ feltétel mellett; diszkrét X-re ezt a réteg átlagával becsüljük. Kis kontrollpélda: ha $X=0$ mellett három kezelt ügyfél bevétele $2,3,1$, akkor $\hat{E}(Y|T=1,X=0)=(2+3+1)/3=2$. A feltételes várható érték tehát az a regressziós görbe, amely minden x pontban az Y átlagát adja.

A gyakorlati becslés két fő úton történhet. A **regressziós adjustment** az Y modelljét illeszti a T és X magyarázó változókkal; a T együtthatója a feltételes hatás becslése. A **propensity score** (hajlamérték) $p(x)=P(T=1|X=x)$ azt fejezi ki, mekkora eséllyel kerül valaki a kezelt csoportba az x aktivitás mellett. A propensity score **kiegyensúlyozó tulajdonsága** szerint, ha X elegendő a hátsó út blokkolásához, akkor a $p(x)$-re feltételesen a kezelés és a potenciális kimenetek közel függetlenek. Ezt kihasználhatjuk **párosítással** (matching): minden kezelt ügyfélhez hasonló score-ú kontrollt keresünk; vagy **súlyozással** (weighting): a kezelteket $1/p(x)$, a kontrollokat $1/(1-p(x))$ súllyal látjuk el, hogy a mintát mintegy újrasúlyozzuk a randomizált kísérlet eloszlásához. A stabilizált súlyok a marginális $\pi=P(T=1)$ aránnyal osztanak: $w_1=\pi/p(x)$ és $w_0=(1-\pi)/(1-p(x))$, ezzel a szélsőséges súlyokat csökkentik.

A korrekció csak akkor értelmezhető, ha teljesül a **pozitivitás** (positivity): minden x rétegben $0<P(T=1|X=x)<1$. Ha például minden nagyon aktív ügyfél kampányt kapott, akkor abban a rétegben nincs kontroll; a becslés ott extrapolál, és a kimenet kontrollbeli értékét már nem az adat határozza meg. Az **overlap-vizsgálat** a becsült propensity score eloszlását hasonlítja össze a kezelt és kontroll csoportban; ha a két hisztogram jól átfedi egymást, a pozitivitás a megfigyelt tartományon hihető.

A **nem mért zavaró változó** (unmeasured confounding) a leggyakoribb korlát. Legyen U egy rejtett jellemző, amely egyszerre növeli a kezelés esélyét és a kimenetet, de az adatbázisban nem szerepel. Ekkor a DAG:



Az X-re való korrekció nem blokkolja az U-n átmenő utat. A becsült hatás torzítását közelítőleg a két rejtett él erősségének szorzata adja: ha U egységnyi növekedése γ_U-val növeli a kezelés valószínűségét, és β_U-val a kimenetet, akkor a torzítás $\beta_U\gamma_U$ nagyságrendű. Az **érzékenységi elemzés** (sensitivity analysis) azt kérdezi: mekkora rejtett U kellene ahhoz, hogy az észlelt hatást teljesen megmagyarázd? A propensity score tehát a mért X-eket próbálja kiegyensúlyozni, de a nem mért utat nem tudja elzárni.

18.3.3 Teljes minimális munkapélda

A következő kód a fenti folyamatot állítja elő, majd összehasonlítja a nyers és a korrigált becslést, overlap-vizsgálatot készít, végül elrejti U-t. A kód automatikusan még nem ellenőrzött; a kimeneteket `# szemléltető` megjegyzéssel jelöljük, és a generatív paraméterekből levezetjük a várt irányt.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

rng = np.random.default_rng(20261020)
n = 2000

# Mért zavaró változó: előzetes aktivitás, standard normális
X = rng.normal(0, 1, n)

# Kezelés: logit a mért X alapján
logit_score = 0.8 * X
p = 1 / (1 + np.exp(-logit_score))
T = rng.binomial(1, p, n)

# Valódi kimenet: tau=1.0 hatás, X lineáris hatása, zaj
epsilon = rng.normal(0, 1, n)
tau_true = 1.0
Y = 1.0 + tau_true * T + 1.5 * X + epsilon

df = pd.DataFrame({"X": X, "T": T, "Y": Y})

# 1) Nyers becslés
nyers = df.loc[df["T"] == 1, "Y"].mean() - df.loc[df["T"] == 0, "Y"].mean()

# 2) Regressziós adjustment
Xmat = sm.add_constant(df[["T", "X"]])
ols = sm.OLS(df["Y"], Xmat).fit()
tau_adj = ols.params["T"]
se_adj = ols.bse["T"]
ci_adj = ols.conf_int().loc["T"].to_numpy()

# 3) Propensity score becslése
logit_model = sm.Logit(df["T"], sm.add_constant(df["X"])).fit(disp=0)
df["ps"] = logit_model.predict(sm.add_constant(df["X"]))

# Overlap-vizsgálat
plt.hist(df.loc[df["T"] == 0, "ps"], bins=20, alpha=0.5, label="Kontroll (T=0)")
plt.hist(df.loc[df["T"] == 1, "ps"], bins=20, alpha=0.5, label="Kezelt (T=1)")
plt.xlabel("Becsült propensity score")
plt.ylabel("Gyakoriság")
plt.title("Overlap-vizsgálat")
plt.legend()

# 4) IPW becslés stabilizált súlyokkal
pi_hat = df["T"].mean()
w1 = pi_hat / df["ps"]
w0 = (1 - pi_hat) / (1 - df["ps"])
ipw_ate = (np.sum(w1 * df["T"] * df["Y"]) / np.sum(w1 * df["T"])
           - np.sum(w0 * (1 - df["T"]) * df["Y"]) / np.sum(w0 * (1 - df["T"])))

# 5) Rejtett U bevezetése
U = rng.normal(0, 1, n)
T2 = rng.binomial(1, 1 / (1 + np.exp(-(0.8 * X + 0.6 * U))))
```

```
Y2 = 1.0 + tau_true * T2 + 1.5 * X + 0.7 * U + rng.normal(0, 1, n)
df2 = pd.DataFrame({"X": X, "T": T2, "Y": Y2, "U": U})

# Az elemzésben U-t "elfelejtjük"; csak T2 és X szerepel
Xmat2 = sm.add_constant(df2[["T", "X"]])
ols2 = sm.OLS(df2["Y"], Xmat2).fit()
tau_hidden = ols2.params["T"]

print(f"Nyers: {nyers:.3f}")                    # szemléltető
print(f"OLS adjustment: {tau_adj:.3f}, SE={se_adj:.3f}")
print(f"IPW stabilizált: {ipw_ate:.3f}")        # szemléltető
print(f"Rejtett U melletti adjustment: {tau_hidden:.3f}")  # szemléltető
```

Az ellenőrzött futás (statsmodels 0.15.0, numpy 2.5.4, pandas 3.1.0, seed 20261020) értékei: nyers becslés 2,073; OLS adjustment 0,970 (SE 0,0479, 95% CI 0,876–1,064); stabilizált IPW 0,912; a rejtett U melletti OLS 1,377 (SE 0,0591). A propensity score a kontroll karban 0,065–0,895, a kezelt karban 0,092–0,943 között van, tehát az átfedés jó. A nyers becslés azért nagyobb 1,0-nál, mert a nagy aktivitású ügyfelek gyakrabban kapnak kezelést és többet költenek; az OLS és az IPW közel van a generált 1,0-hoz; a rejtett U felfelé torzít.

18.3.3.1 Átfedés, trimmelés és effektív mintanagyság

A propensity score eloszlását karonként kell megmutatni, és a szélsőséges súlyokat nem elég szemrevételezni. Az **effektív mintanagyság** (effective sample size, ESS) a súlyozott minta információtartalmát méri:

$$\mathrm{ESS} = \frac{\left(\sum_i w_i\right)^2}{\sum_i w_i^2}.$$

Ha az ESS jóval kisebb a nyers elemszámnál, néhány szélsőséges súly uralja a becslést. A fenti futásban a stabilizált IPW ESS-e a kontroll karban 786, a kezelt karban 858 a 2000 fős mintából. A trimmelési érzékenység:

Trimmelési tartomány	Megtartott n	IPW ATE	ESS kontroll	ESS kezelt
nincs (0–1)	2000	0,912	786	858
[0,1; 0,9]	1982	0,898	780	865
[0,2; 0,8]	1811	1,027	773	838

A becslés a trimmelési küszöbre érzékeny; a [0,2; 0,8] szűkítés már érdemben eltér, ezért a jelentésben a trimmelési szabályt előre rögzíteni kell, és az eredményt a választott szabállyal együtt kell közölni.

Nem átfedő példa. Ha a kezelési szelekció erősebb (logit $2{,}5X + 1{,}0$), a propensity score maximuma a kontroll karban 0,987, a kezelt karban 0,9999; a kontroll ESS 162 a 733 főből, a kezelt ESS 265 a 1267 főből, az IPW ATE 1,189 a valódi 1,0 helyett. Itt a pozitivitás sérül: a magas X tartományban gyakorlatilag nincs kontroll, ezért a teljes célpopuláció ATE-je nem becsülhető megbízhatóan. A helyes állítás csak az átfedő populációra vonatkozik, és a szelekciós mechanizmus korlátját a jelentésben meg kell nevezni.

18.3.4 Eredmény értelmezése

A kézi kontroll abból indul ki, hogy a generatív modellben $\tau = 1.0$. Az OLS adjustment a következő feltételes modellt becsüli: $E(Y|T, X) = b_0 + b_T T + b_X X$. Mivel az igazi kimenet lineáris, a b_T a feltételes hatás, és a modellben nincs T–X interakció, az ATE megegyezik ezzel a feltételes hatással. A nyers becslés ellenőrzése: $E(Y|T = 1) - E(Y|T = 0) = \tau + 1.5(E(X|T = 1) - E(X|T = 0))$, és mivel $E(X|T = 1) > 0 > E(X|T = 0)$, a nyers különbség 1.0-nál nagyobb. A propensity score és az OLS adjustment konzisztenciájának közös feltétele, hogy X blokkolja a hátsó utat, a pozitivitás teljesül, és a kimeneti modell legalább közelítőleg helyes.

A rejtett U esetben az X-re korrigált becslés torzított, mert az U kimaradásával a hátsó út egy része nyitva marad. A torzítás várható előjele ebben a szimulációban pozitív, mert $\gamma_U = 0.6$ és $\beta_U = 0.7$ egyaránt pozitív. A numerikus nagyság a véletlentől függ, de a seedelt futtatásban a tanuló ellenőrizheti, hogy a becsült érték 1.0-tól szisztematikusan felfelé tér el. Fontos, hogy ezt ne nevezzük „mérési” eredménynek addig, amíg a kódot helyben nem futtattuk; a fenti számok csak a generatív összefüggésekből adódó elméleti várakozások.

Érzékenységi elemzés: a propensity score súlyozás önmagában nem bizonyítja a zavarás megszüntetését. A pozitivitás ellenőrzése, a súlyok eloszlása és a súlyozás utáni kovariáns-egyensúly ellenőrzése szükséges; rejtett zavarásra külön érzékenységi forgatókönyv kell.

18.3.5 Feladatok

1. **Értelmezés.** A DAG-ban $X \to T$, $X \to Y$ és $T \to Y$ nyilak vannak. Magyarázd el, miért torzított a nyers átlagkülönbség, és miért blokkolja az X-re való korrekció a hátsó ajtó utat. Add meg a feltételes várható értékes becslés képletét abban az esetben, amikor X két lehetséges értéket vesz fel: $x_0 = 0$ és $x_1 = 1$.

2. **Implementálás.** A fenti kódban az IPW becslés stabilizált súlyokat használ. Írd át a súlyozást úgy, hogy ne stabilizált, hanem közönséges IPW súlyokat használjon, és futtasd le a seedelt szimuláción. Milyen tulajdonságot vársz a becsült pontértéktől, és miben térhet el a stabilizált változattól? (A futtatáshoz a kódban a `w1 = 1/df["ps"]` és `w0 = 1/(1-df["ps"])` sorokat használd.)

3. **Hibadiagnózis.** Tegyük fel, hogy egy másik minta overlap-ábráján a kezelt csoport propensity score-ja mindenütt 0.95 felett van. Mi a probléma a pozitivitással? Melyik rétegben nincs elegendő kontroll? Adj egy olyan X-értéket, amelynél ez bekövetkezhet a fenti generatív folyamat módosításával, és írd le, hogyan csökkenthető a hiba (például szűkítéssel vagy adatgyűjtéssel).

4. **Megváltozott feltétel.** A rejtett U szimulációban a $\gamma_U = 0.6$ paramétert először csökkentsd 0.1-re, majd a $\beta_U = 0.7$ paramétert növeld 1.5-re. Mindkét esetben magyarázd el, hogyan változik a rejtett confoundingból adódó torzítás nagysága. Melyik paraméterváltozás erősebb hatású, és miért?

18.3.6 Megoldások és indoklás

1. A nyers különbség két csatornát kever: a T oksági hatását Y-ra és az X közös hatását. A T és Y közötti nyers asszociáció tartalmazza az X-en átmenő utat, mert a nagy X-ű ügyfelek gyakrabban kapnak kezelést és gyakrabban költenek. Az X-re való adjustment a megfigyeléseket azonos X-rétegeken belül hasonlítja össze, így az X szerinti összetétel

már nem torzít. Diszkrét X esetén $\hat{\tau}_{adj} = [\bar{Y}(T=1, X=x_1) - \bar{Y}(T=0, X=x_1)]\hat{P}(X=x_1) + [\bar{Y}(T=1, X=x_0) - \bar{Y}(T=0, X=x_0)]\hat{P}(X=x_0)$.

2. A nem stabilizált IPW súlyok $w_1 = 1/p(x)$ és $w_0 = 1/(1-p(x))$. A becsült ATE: $\hat{\tau}_{IPW} = (\sum w_1 T_i Y_i / \sum w_1 T_i) - (\sum w_0 (1-T_i) Y_i / \sum w_0 (1-T_i))$. A pontbecslés várhatóan ugyancsak 1.0 körüli, de a nem stabilizált súlyok varianciája nagyobb, mert a nagyon kis $p(x)$-ű rétegek hatalmas súlyt kapnak. A feladat ellenőrző invariánsa: a becslés nem térhet el 1.0-tól olyan rendszeres, mint a nyers becslés; a két IPW-változat pontbecslése közel esik egymáshoz, de a súlyok eloszlása eltér.

3. A 0.95 feletti propensity score azt jelenti, hogy az adott x tartományban szinte mindenki kezelt, tehát $P(T=1|X=x) \approx 1$, ami sérti a pozitivitást. A kontrollréteg ott hiányzik, és a becslés a kontrollkimenetet extrapolációval pótolja. Például ha a generatív modellben a logit score $0.8X$-ről $2.0X$-re változik, akkor $X=2$ esetén $P(T=1|X=2) \approx 0.982$, ami 0.95 felett van. A hiba csökkenthető az elemzés átfedő tartományra szűkítésével (például csak $p(x) \in [0.1, 0.9]$ megfigyelésekkel), vagy további kontrollok gyűjtésével abban a rétegben.

4. A torzítás $\beta_U \gamma_U$ szorzattal arányos. Ha γ_U-t 0.6-ról 0.1-re csökkentjük, a torzítás hatodára esik; ha β_U-t 0.7-ről 1.5-re növeljük, a torzítás körülbelül kétszeresére nő. A két paraméter változásának hatása szimmetrikus a szorzatban, de az adott módosításoknál a γ_U csökkentés erősebb, mert $0.6 \to 0.1$ hatásosabb, mint $0.7 \to 1.5$. A válaszban a gyakorlati üzenet: a rejtett változó mindkét irányú erőssége számít.

18.3.7 Gyakori tévedés és önellenőrzés

Gyakori tévedés: „A propensity score korrigál minden confoundingot.” Ellenpélda a fenti szimuláció: az X-re való propensity score adjustment vagy IPW becslés sem szünteti meg az U hatását, mert U nem szerepel a mért változók között. A propensity score kizárólag a mért X-eket egyensúlyozza; a nem mért hátsó út nyitva marad. Ezért a propensity score soha nem igazolja önmagában az oksági állítást. A 14.3. alfejezetben látott kovarianciafogalom itt is segít: a T és Y közötti asszociáció akkor is jelen van, ha T nem oksági ok, mert egy közös ok korrelációt hoz létre. A 17.3. alfejezetben tárgyalt robusztus standard error a heteroszkedaszticitást kezeli, nem a zavaró változót; hiába kicsi a SE, ha a pontbecslés torzított. Végül a 6.3. alfejezetből a megfigyelési egység ellenőrzése itt is kötelező: egy ügyfél egy sor, a kampány és a kimenet ugyanabban a sorban.

Önellenőrző kérdések: (1) Mi a különbség a mért és a nem mért confounding között? (2) Mit jelent a pozitivitás, és milyen ábra mutatja a sérülését? (3) Miért lehet egy stabilizált IPW-becslés varianciája kisebb, mint a közönséges IPW-becslésé? (4) Melyik irányba torzít a fenti szimulációban a rejtett U, és miért?

18.3.8 Rövid összefoglaló

A megfigyeléses becslésben a nyers asszociáció keveri az oksági hatást és a közös okok hatását. A hátsó ajtó út azonosítása után a mért zavaró változóra való adjustment a rétegek közötti összehasonlítást teszi lehetővé. A pozitivitás és az overlap-vizsgálat azt ellenőrzi, hogy minden rétegben van-e kezelt és kontroll megfigyelés. A propensity score párosításra és súlyozásra használható, de nem blokkolja a nem mért utat. Az érzékenységi elemzés a rejtett változó erejét kérdezi, és a szimulált példa megmutatja, hogy egy rejtett U a korrigált becslést is torzíthatja. A következő alkalmazás a kísérleti eredmény tudományos közlése, ahol ugyanezeket a korlátokat expliciten jelenteni kell.

18.4 Kísérlet eredményének tudományos közlése

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: NumPy, pandas, statsmodels

Előfeltétel: 18.3. alfejezet · Célkészség: Kísérlet eredményének tudományos közlése · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

18.4.1 Cél és konkrét helyzet

A Northstar kísérlet lezárult: 300 ügyfél randomizálva a `treatment` és `control` karra, a primer kimenet a randomizálás után 7 nappal mért `conversion_7d` (0/1). A bemenet a `lab/data/assignments.csv` és a `lab/data/outcomes.csv`; a megfigyelési egység egy ügyfél, ezért az egyesítés kulcsa `customer_id`. A cél most nem új becslő építése, hanem a kísérlet **tudományos közlése**: egy rövid jelentés, amelyben a becsült hatás, a bizonytalanság, a design problémái és a döntési javaslat külön szakaszban szerepel.

Induló becslés: a szimuláció generáló hatása 10 százalékpont volt, de a 300 elemű minta ettől véletlenül eltérhet. A jelentésnek ezért nem a generáló értéket, hanem a mintából számolt becslést és annak intervallumát kell tárgyalnia. A döntési kérdés: érdemes-e a kezelést a teljes ügyfélkörre bevezetni?

18.4.2 Hatás és intervallum

A kísérleti hatást az átlagos kezelési hatás (average treatment effect, ATE) becslőjével adjuk meg. Legyen $Y_i \in \{0,1\}$ az i-edik ügyfél konverziója, $Z_i \in \{0,1\}$ a véletlen kezeléskiosztás ($0 =$ kontroll, $1 =$ kezelés), n_1 és n_0 a két kar létszáma. A becslő

$$\hat{\tau} = \bar{y}_1 - \bar{y}_0, \qquad \bar{y}_j = \frac{1}{n_j} \sum_{i:Z_i=j} Y_i,$$

ahol $\bar{y}_j$ dimenzió nélküli arány, értéke 0 és 1 közé esik, így $\hat{\tau}$ tartománya $[-1,1]$. 100-zal szorozva a hatást **százalékpontban** kapjuk: ha $\hat{\tau} = 0{,}08$, akkor a kezelés 8 százalékponttal növeli a becsült konverziót.

A standard hiba bináris kimenet esetén, független csoportokra:

$$\widehat{SE}(\hat{\tau}) = \sqrt{\frac{\bar{y}_1(1-\bar{y}_1)}{n_1} + \frac{\bar{y}_0(1-\bar{y}_0)}{n_0}}.$$

A számlálóbeli $\bar{y}(1-\bar{y})$ a csoporton belüli variancia becslése; a nevező az adott kar mintanagysága. A standard hiba ugyanabban az egységben van, mint $\hat{\tau}$, tehát arány, illetve 100-zal szorozva százalékpont. A 14.4. alfejezetben látott gondolat ismétlődik: az átlag ingadozása $\sigma/\sqrt{n}$ nagyságrendű, itt csak mindkét kar varianciáját összeadjuk, mert a két mintaátlag különbségének varianciája a varianciák összege.

Közelítő 95%-os konfidenciaintervallum:

$$\left[\hat{\tau} - z_{0,975}\widehat{SE},\ \hat{\tau} + z_{0,975}\widehat{SE}\right], \qquad z_{0,975} \approx 1{,}96.$$

Az intervallum **gyakorisági** értelmezésű: ha az eljárást sok független kísérleten megismételnénk, az így számolt intervallumok körülbelül 95%-a fedné a valódi hatást. A konkrét intervallumra nem mondjuk, hogy „95% valószínűséggel tartalmazza a hatást”, mert a valódi hatás paraméter, nem véletlen változó ebben a keretben. A pontos értelmezéshez és a különbséghez lásd 15.2. alfejezet és a Bayesi alternatívát 19.2. alfejezet.

Kis mintán a fenti Wald-intervallum kiléphet a $[-1, 1]$ tartományból. Kézi ellenőrzés: ha $n_0 = n_1 = 4$, $\bar{y}_0 = 1/4$, $\bar{y}_1 = 3/4$, akkor

$$\hat{\tau} = 0{,}5, \quad \widehat{SE} = \sqrt{\frac{0{,}25 \cdot 0{,}75}{4} + \frac{0{,}75 \cdot 0{,}25}{4}} = \sqrt{0{,}09375} \approx 0{,}306,$$

tehát a 95%-os intervallum $0{,}500 \pm 1{,}96 \cdot 0{,}306 = [-0{,}100; 1{,}100]$. A felső határ nagyobb 1-nél, ami mutatja, hogy ilyen kis minta esetén a normál közelítés nem megbízható. A labor jelentésében ezért a fő következtetést a 300 ügyfélre számolt intervallummal, a kis mintás alternatívát pedig ügyfélszintű bootstrap segítségével adjuk meg.

18.4.3 Külső érvényesség

A **külső érvényesség** (external validity) azt kérdezi: a kísérletben becsült hatás milyen populációra, időszakra és megvalósításra vihető át. A belső érvényesség a kísérleten belüli torzítatlanságot jelenti; a kettő független.

A Northstar esetben négy korlátot kell a jelentésben megnevezni:

1. **Populáció:** a 300 szintetikus ügyfél adott `segment` és `region` eloszlást követ. Más összetételű ügyfélkörre a hatás nagysága változhat.
2. **Időszak:** a kísérlet 2025. július 1. és 8. között zajlott. Más szezon, ünnepi időszak vagy kampány alatt a kezelés hatása eltérhet (lásd az időtípusok időzóna-kezelését 6.4. alfejezetben).
3. **Megvalósítás:** a `treatment` a szimuláció egyetlen működési formája. Más üzenet, új csatorna vagy rosszabb kivitelezés nem ugyanazt a hatást garantálja.
4. **Megfigyelési ablak:** a kimenet fix 7 napos. A hosszabb távú hatás, a lemorzsolódás vagy a későbbi konverzió itt nem mérhető.

A statisztikai szignifikancia nem pótolja ezt a kérdést. Egy eredmény lehet belsőleg torzítatlan, szignifikáns, mégsem általánosítható a célpopulációra.

18.4.4 Feltáró és megerősítő elemzés

A **megerősítő** (confirmatory) elemzés az előre rögzített primer kérdés: a `conversion_7d` aránykülönbsége a két kar között. Ehhez tartozik az előre választott ITT-definíció (18.2. alfejezet): a nevezőben mind a 300 randomizált ügyfél szerepel, függetlenül attól, hogy a kezelést megkapta-e vagy megnyitotta-e. Az erre számolt intervallumot használjuk döntési bemenetként.

A **feltáró** (exploratory) elemzés minden, ami utólag merül fel: a `revenue_7d_minor` összehasonlítása, szegmensenkénti bontás, időbeli görbék, alcsoport-eltérések. Ezek fontos hipotéziseket adhatnak a **következő** kísérlethez, de nem ugyanazzal a megerősítő erővel bírnak. A jelentésben ezért külön cím alatt, „Feltáró megfigyelések” néven szerepeljenek, a primer becsléstől vizuálisan is elválasztva.

A megkülönböztetés oka a többszörös próba és a szelektív közlés: ha sok utólagos metszetet nézünk, a véletlen kiugrások közül válogatunk. A primer elemzés előre rögzítése védi a p-érték értelmezhetőségét; a feltáró eredmény csak következő kísérleti tervet indokolhat.

18.4.5 Döntési veszteség

A **döntési veszteség** (decision loss) a becslést a döntési kontextushoz köti. A statisztikai kérdés „van-e hatás?”, a döntési kérdés „érdemes-e bevezetni?”. A kettő közötti híd a minimális gyakorlati hatás (minimum practical effect) és a tévedés költsége.

Legyen θ a valódi hatás százalékpontban, és $\theta_{\min} = 3$ százalékpont az a küszöb, amely alatt a bevezetés üzletileg nem éri meg. A döntési szabály egyszerű formája:

- ha az intervallum **alsó** határa is nagyobb $\theta_{\min}$-nél, akkor bevezetés;
- ha az intervallum **felső** határa is kisebb 0-nál, akkor elutasítás;
- különben további adat vagy következő kísérlet.

A harmadik eset a bizonytalanság explicit elismerése: a minta nem elég a döntéshez, nem „majdnem szignifikáns” eredmény. A veszteség itt kétféle lehet, és mindkettőt érdemes megnevezni: a téves bevezetés költsége (ha a valódi hatás negatív vagy $\theta_{\min}$ alatti), és az elmaradt haszon költsége (ha a valódi hatás $\theta_{\min}$ felett van, de elutasítjuk). A veszteség skálája pénzben vagy más üzleti célban fejezhető ki; a kísérleti adatban a `revenue_7d_minor` EUR-centben van, de a konverziós döntéshez itt nem adunk meg vállalati árazást, mert a labor szintetikus.

18.4.6 Teljes minimális példa

A következő kód a két laborfájlt egyesíti, kiszámolja a hatásbecslést, a normál közelítésű intervallumot és egy ügyfélszintű bootstrap intervallumot. A kódot a mellékelt lab adatain futtattuk (Python 3.12.3, pandas 3.1.0, numpy 2.5.4, seed `20261020`, $B = 2000$); a kimenet a lenti, ellenőrzött értékeket adja. A kód Linux, macOS és Windows PowerShell alatt is futtatható, a munkakönyvtár a projektgyökér.

```
import numpy as np
import pandas as pd
from pathlib import Path

root = Path("lab")
data = root / "data"
assign = pd.read_csv(data / "assignments.csv")
out = pd.read_csv(data / "outcomes.csv")

df = (
    assign.merge(out, on="customer_id", validate="one_to_one")
    .sort_values("customer_id")
)

# Ellenőrző kapu: ITT-nevező és séma
assert df["customer_id"].is_unique
assert df.shape[0] == 300
assert df["conversion_7d"].isin([0, 1]).all()
assert set(df["variant"]) == {"control", "treatment"}

by = df.groupby("variant")["conversion_7d"]
means = by.mean()          # shape: (2,)
ns = by.size()             # shape: (2,)
```

```
diff = means["treatment"] - means["control"]
se = float(np.sqrt(
    (means["treatment"] * (1 - means["treatment"]) / ns["treatment"])
    + (means["control"] * (1 - means["control"]) / ns["control"])
))
z = 1.959963984540054      # normális 0.975 kvantilis
lo = diff - z * se
hi = diff + z * se

# Ügyfélszintű percentile bootstrap
rng = np.random.default_rng(20261020)
B = 2000
c = df.loc[df["variant"] == "control", "conversion_7d"].to_numpy()
t = df.loc[df["variant"] == "treatment", "conversion_7d"].to_numpy()
boot_diffs = np.empty(B)
for b in range(B):
    cb = rng.choice(c, size=c.size, replace=True)
    tb = rng.choice(t, size=t.size, replace=True)
    boot_diffs[b] = tb.mean() - cb.mean()

boot_lo, boot_hi = np.quantile(boot_diffs, [0.025, 0.975])

print(f"n_control={ns['control']}, n_treatment={ns['treatment']}")
print(f"mean_control={means['control']:.4f}, mean_treatment={means['treatment']:.4f}")
print(f"diff={diff:.4f}, SE={se:.4f}")
print(f"normal_CI=[{lo:.4f}, {hi:.4f}]")
print(f"bootstrap_CI=[{boot_lo:.4f}, {boot_hi:.4f}]")

# Döntési szabály
theta_min = 0.03
if lo > theta_min:
    decision = "bevezetés"
elif hi < 0:
    decision = "elutasítás"
else:
    decision = "további adat"

print(f"decision_with_theta_min={decision} (theta_min={theta_min})")
```

Az ellenőrzött futás kimenete:

```
n_control=154, n_treatment=146
mean_control=0.2532, mean_treatment=0.2534
diff=0.0002, SE=0.0502
normal_CI=[-0.0983, 0.0986]
bootstrap_CI=[-0.0983, 0.0997]
decision_with_theta_min=további adat (theta_min=0.03)
```

A számok a mellékelt adatokból származnak; a bootstrap seedje és a $B = 2000$ replikáció rögzített, ezért a kimenet ismételhető.

18.4.7 Eredmény értelmezése

A kód előbb szerződés-kapuval ellenőrzi a bemenetet: a `validate="one_to_one"` garantálja, hogy egy ügyfélhez pontosan egy kimenet tartozik, a `shape == (300, ...)` pedig az ITT-nevezőt. A bootstrap az ügyfélszintű független mintavételi egységet tartja meg: a karokon belül újramintavételezünk egész ügyfeleket, nem sorokat, így az `order_items` nem sokszorozhatja a nevezőt.

Kézi kontroll a normál intervallumra: a tényleges adatokkal $\bar{y}_1 = 0{,}2534$, $\bar{y}_0 = 0{,}2532$, $n_1 = 146$, $n_0 = 154$, így

$$\widehat{SE} = \sqrt{\frac{0{,}2534 \cdot 0{,}7466}{146} + \frac{0{,}2532 \cdot 0{,}7468}{154}} \approx 0{,}0502,$$

és a $0{,}0002 \pm 1{,}96 \cdot 0{,}0502$ intervallum $[-0{,}0983; 0{,}0986]$, egyezik a kimenettel. A bootstrap intervallum közel azonos, mert a mintanagyság elég nagy; kis mintán a Wald-intervallum kilépése ellen a bootstrap jobb védelmet ad.

Az értelmezésben három állítást különítsünk el:

1. **Becslés:** a kezelés becsült hatása a mintában 0,02 százalékpont, gyakorlatilag nulla.
2. **Bizonytalanság:** a 95%-os intervallum −9,83 és +9,86 százalékpont között van, tehát a minta a nullával és a tervezett 10 százalékpontos hatással is összeegyeztethető; a becslés önmagában nem bizonyíték a hatás hiányára.
3. **Döntés:** a 3 százalékpontos gyakorlati küszöb mellett a „további adat” döntés a helyes, mert az intervallum lefedi a küszöb alatti és feletti tartományt is. Ez **nem** azt jelenti, hogy a hatás nulla: a pontbecslés közel nulla, de a döntési kritérium a bizonytalanság miatt nem enged sem bevezetést, sem elutasítást.

A `revenue_7d_minor` elemzése ugyanilyen szerkezettel, de feltáró cím alatt jelenik meg; a döntést nem arra alapozzuk.

Jelentési minimum: közölni kell a randomizált, megfigyelt és elemzett egységek számát karonként, a hiányzó végpontok kezelését, a becslési célt, a pontbecslést és bizonytalanságot; az előre rögzített és az utólagos elemzést egyértelműen különítsük el.

18.4.7.1 CONSORT-szerű mintaáramlás és becslési cél

A jelentés első táblázata a mintaáramlást rögzíti, karonként és összesen. A Northstar adatban:

Lépés	Treatment	Control	Összes
Randomizált	146	154	300
A kijelölt kezelést megkapta	146	154	300
Primer kimenet megfigyelve	146	154	300
Hiányzó primer kimenet	0	0	0
ITT-elemzésből kizárt	0	0	0
Protokoll-eltérés	0	0	0

A táblázatot a számított `ns` és a hiányzó értékek darabszáma tölti ki; a laborban nincs noncompliance és nincs hiányzó kimenet, de a sablon a jövőbeli esetekre is érvényes. A **becslési célok** külön táblában szerepelnek:

Cél	Populáció	Becslő	Előre rögzített alpha	Státusz
ITT ATE	minden randomizált ügyfél	kockázatkülönbség	0,05 kétoldali	megerősítő (primer)
CACE	complierek	per-protocol / Wald	0,05	feltáró, érzékenységi

Cél	Populáció	Becslő	Előre rögzített alpha	Státusz
Bevétel	minden randomizált ügyfél	átlagkülönbség	0,05	feltáró

Eltérésközlési sablon. Minden, az előzetes tervtől való eltérést négy mezővel kell rögzíteni: mi volt a tervezett lépés, mi történt ténylegesen, mi az ok, és hogyan érinti a becslési célt. Például: „tervezett: 150/150 randomizálás; tényleges: 154/146; ok: Bernoulli-kiosztás; hatás: a SE a tervezettnél alig tér el, az ITT-nevező változatlan.” Az eltérés önmagában nem hiba, de a hatását és az érzékenységi ellenőrzést közölni kell.

18.4.8 Feladatok

1. **Értelmezés.** Tegyük fel, hogy a laborfutásod ezt adta: `diff=0.084`, `normal_CI=[0.012, 0.156]`, $n_0 = n_1 = 150$. Fogalmazd meg egy mondatban a becslést, egy mondatban a bizonytalanságot, és dönts a $\theta_{\min} = 0{,}03$ küszöbbel. Indokold, miért nem elég azt mondani, hogy „szignifikáns, tehát bevezetjük”.

2. **Implementálás.** Írj `write_report(df, out_path)` függvényt, amely a fenti számításokból a `report.md` fájlba négy szakaszt ír: `## Becslés`, `## Bizonytalanság`, `## Design és korlátok`, `## Döntési javaslat`. A design szakaszban három, a külső érvényességet érintő korlát is szerepeljen; a döntési szakasz a $\theta_{\min}$ szabályt alkalmazza. A függvény visszatérési értéke a négy szakasz szövege.

3. **Hibadiagnózis.** Egy kolléga jelentése ezt írja: „A kezelést megnyitó 102 ügyfél 37%-a konvertált, a kontroll 24%-a konvertált; a kezelés 13 százalékpontot javít.” Nevezd meg a protokollsértést, és add meg a helyes ITT-nevezőt és a helyes becslési elvet. Milyen torzítást okoz a megnyitásra szűrés?

4. **Megváltozott feltétel.** Cseréld a primer kimenetet `revenue_7d_minor`-ra, és számold ki a két kar átlagkülönbségét, a standard hibát és a 95%-os intervallumot. A kapott intervallumot miért szabad csak feltáró eredményként közölni? Milyen további designelemet kellene rögzíteni egy következő kísérletben ahhoz, hogy a bevételi cél megerősítő legyen?

18.4.9 Megoldások és indoklás

1. A becslés: a kezelés 8,4 százalékponttal növeli a konverziót a mintában. A bizonytalanság: a 95%-os intervallum 1,2 és 15,6 százalékpont közé esik. A döntés $\theta_{\min} = 0{,}03$ mellett „további adat”: az alsó határ 0,012, ami kisebb 0,03-nál, tehát nem zárható ki, hogy a hatás kisebb a gyakorlati küszöbnél. A $p < 0{,}05$ önmagában csak a nulla hatást zárja ki; nem állítja, hogy a hatás hasznos, és nem pótolja a külső érvényességet.

2. Egy lehetséges váz:

```
def write_report(df, out_path):
    # ... ugyanaz a diff, se, lo, hi számítás ...
    theta_min = 0.03
    if lo > theta_min:
        decision = "bevezetés"
    elif hi < 0:
```

```
        decision = "elutasítás"
    else:
        decision = "további adat"

    text = (
        "## Becslés\n"
        f"A kezelés becsült hatása {diff:.3f} százalékpont.\n\n"
        "## Bizonytalanság\n"
        f"A 95%-os intervallum [{lo:.3f}, {hi:.3f}] százalékpont.\n\n"
        "## Design és korlátok\n"
        "- A minta szintetikus, adott szegmens- és régióeloszlással.\n"
        "- A megfigyelési ablak fix 7 napos.\n"
        "- A kezelés egyetlen megvalósítási formáját teszteltük.\n\n"
        "## Döntési javaslat\n"
        f"A {theta_min} százalékpontos minimális gyakorlati hatás mellett: {decision}.\n"
    )
    Path(out_path).write_text(text, encoding="utf-8")
    return text
```

Ellenőrző invariant: a fájl négy `##` szakaszt tartalmaz, a döntési szakasz pontosan egyet a három lehetséges érték közül, és a becslési szakasz számértéke egyezik a korábban kiszámolt `diff` értékkel 10^{-3} tűrésen belül.

3. A kolléga megsértette az ITT-elvet (18.2. alfejezet): a kezelési kar nevezőjét a kezelés megnyitására szűrte, ami a kezelés **expozíciójának** utólagos feltételét jelenti. A megnyitás maga is kimenet, ezért a szűrés kiválasztási torzítást okoz: a megnyitó ügyfelek eleve aktívabbak lehetnek, így a 13 százalékpontos különbség a kiválasztás és a kezelés keveréke. A helyes nevező mind a 150 randomizált ügyfél a kezelési karban, tehát a becslés a kezelési kar összes ügyfelének átlagos konverziója mínusz a kontroll összes ügyfelének átlagos konverziója. A megnyitási arány csak mint folyamatmutató (guardrail) elemezhető, nem mint a hatásbecslés nevezője.

4. A bevételi elemzés:

```
rev = df.groupby("variant")["revenue_7d_minor"]
rev_means = rev.mean()
rev_vars = rev.var(ddof=1)
rev_diff = rev_means["treatment"] - rev_means["control"]
rev_se = float(np.sqrt(
    rev_vars["treatment"] / ns["treatment"]
    + rev_vars["control"] / ns["control"]
))
```

A pénzegység EUR-cent; az átlagok, a különbség és a standard hiba is centben van. Az intervallum ugyanazzal a normál kvantilissel számolható, de az eloszlás ferdesége miatt bootstrap ajánlott. Feltáró a bevétel, mert a primer elemzés csak a `conversion_7d` volt; a bevételt utólag választottuk, ezért a becsült intervallum nem hordozza a megerősítő kísérlet garanciáit. Egy következő kísérletben előre rögzíteni kell: a bevételi ablakot (napok, pénznem, visszatérítés kezelése), a primer elemzést (átlag vagy medián, nyertességi arány), a szükséges minimális hatást centben, és a többszörös tesztelésre vonatkozó szabályt, ha a konverziót is megtartjuk másodlagos kimenetként.

18.4.10 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés: **a statisztikai szignifikancia azonosítása a hasznossággal vagy a külső érvényességgel.** Ellenpélda: két kar, $n = 50\,000$ ügyfél, a becsült hatás 0,5 százalékpont, a standard hiba 0,2 százalékpont. A p-érték 0,012, tehát szignifikáns, de a hatás a 3 százalékpontos

gyakorlati küszöb alatt van, és a teljes intervallum is a küszöb alatt marad. A döntési szabály helyesen „elutasítás” lehet, miközben a p-érték kicsi. A nagy minta itt sem javítja a külső érvényességet: ha a kísérlet csak egy régióban futott, az 50 000 fős eredmény sem vihető át automatikusan más régióra.

Visszautalás: a standard hiba itt a 14.4. alfejezetben tanult mintavételi eloszlás kétmintás változata; a 17.4. alfejezet logisztikus modellje ugyanezt a bináris kimenetet link-skálán modellezné, de a döntési jelentéshez az abszolút aránykülönbség érthetőbb; a 18.2. alfejezet ITT-definíciója rögzíti a nevezőt; a 6.4. alfejezet időtípusai pedig a 7 napos ablak konzisztens kezelését biztosítják.

Önellenőrzés a leadás előtt: a jelentésemben külön szerepel-e a becslés, a bizonytalanság, a design és a döntés? Megadtam-e az intervallum szintjét és a mintanagyságot? Megneveztem-e legalább három külső érvényességi korlátot? A döntési javaslatban szerepel-e a gyakorlati küszöb, és különbözik-e a „szignifikáns” állítástól?

18.4.11 Rövid összefoglaló

A kísérleti jelentés négy, egymástól elkülönített részből áll. A becslés a mintából számolt aránykülönbség, például 10,0 százalékpont; a bizonytalanság a standard hibából és a 95%-os konfidenciaintervallumból áll, amelynek gyakorisági értelmezését pontosan meg kell tartani. A design szakasz a külső érvényesség korlátjait nevezi meg: populáció, időablak, megvalósítás és megfigyelési hossz. A döntési javaslat a minimális gyakorlati hatáshoz köti a becslést, ezért a „további adat” éppúgy érvényes eredmény, mint a bevezetés vagy az elutasítás. A következő alkalmazásban ezt a jelentést egy teljes kísérleti becslési és érzékenységi munkamenetté bővítjük a 40.2. alfejezet záróprojekt-feladatában.

19 Bayesi szemlélet és valószínűségi programozás

19.1 Bayesi frissítés rácson

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyMC, ArviZ

Előfeltétel: 18.4. alfejezet · Célkészség: Bayesi frissítés rácson · Kimenet: projekt-artefaktum és ellenőrzött futtatási jegyzőkönyv.

19.1.1 Cél és konkrét helyzet

A Northstar kísérleti projekt mintájára egy szintetikus Bernoulli-kimenetet elemzünk: a kezelési ág egy kis kohorszában a 7 napos konverzió (`conversion_7d`) értéke 0 vagy 1. A megfigyelési egység ügyfél, az adat-összegzés egyetlen eseménysorozatot ad:

$$n = 15, \qquad k = 6.$$

Itt k a konverziók száma, n a kezelt ügyfelek száma. A kérdéses paraméter p: a kezelési ág konverziós valószínűsége, $p \in [0, 1]$ dimenzió nélküli valószínűség. A cél a p feletti normalizált posterior-sűrűség kiszámítása rácsos közelítéssel, két eltérő priorral. Előzetes becslés a 15.1. alfejezet maximum likelihood gondolatából: a mintaarány $k/n = 0.4$, tehát a posterior módusz 0.4 körül várható; flat priorral a posterior várható érték 0.4-nél kissé nagyobb lesz, mert a balra korlátos tartomány aszimmetriája felfelé húz.

Az alábbi kódok futtathatók, de az itt közölt numerikus kimenetek *elméletileg várt, szemléltető* értékek; e szöveg írásakor nem futott Python-kód. A tanuló saját futtatása az ellenőrzés.

19.1.2 Fogalmi magyarázat: prior, likelihood, posterior, normalizálás, predictive

Prior (a priori eloszlás). A prior $\pi(p)$ a p paraméter feletti bizonytalanságunkat fejezi ki az adat megtekintése előtt. Sűrűség: $\pi(p) \geq 0$, $\int_0^1 \pi(p)\, dp = 1$. Két példát használunk:

- *Flat (egyenletes) prior*: $\pi(p) = 1$ a $[0, 1]$ intervallumon. Ez a Beta$(1, 1)$ sűrűség: $\alpha = \beta = 1$, várható értéke $1/2$.
- *Szkeptikus prior*: $\pi(p) = 4(1 - p)^3$, a Beta$(1, 4)$ sűrűség. Várható értéke $\frac{1}{1+4} = 0.2$; a prior tehát alacsony konverziót tart valószínűbbnek.

A Beta(α, β) prior alakja:

$$f(p) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1}, \qquad p \in [0,1],\ \alpha, \beta > 0.$$

Itt $\alpha + \beta$ a prior „mintanagyság-ereje”: nagyobb összeg szűkebb, határozottabb priort jelent.

Likelihood. A $D = (k, n)$ Bernoulli-mintához tartozó likelihood a p függvénye, ahol az adat rögzített:

$$L(p; D) = p^k (1-p)^{n-k}.$$

A teljes binomiális valószínűség ennek $\binom{n}{k}$-szorosa. A $\binom{n}{k}$ tényező p-től független konstans, ezért a posterior normalizálásánál kiesik; elég a likelihood-*magot*, $p^k(1-p)^{n-k}$-t használni. A likelihood nem valószínűségi eloszlás p felett: integrálja általában nem 1. Az $n = 15$, $k = 6$ adatnál a kernel $p = 0.4$-nél körülbelül $4.13 \cdot 10^{-5}$, $p = 0.5$-nél $3.05 \cdot 10^{-5}$; a hányados 1.35, tehát az adat önmagában a 0.4-et jobban támogatja, mint a 0.5-et.

Posterior (a posteriori eloszlás). Bayes tétele alapján a posterior-sűrűség arányos a prior és a likelihood szorzatával:

$$\pi(p \mid D) = \frac{\pi(p) L(p; D)}{Z}, \qquad Z = \int_0^1 \pi(p) L(p; D)\, dp.$$

A Z normalizáló konstans, más néven marginális likelihood / evidence. Az itt használt kernel esetén Z a *kernel-evidence*; a teljes binomiális evidenciához a $\binom{n}{k}$ tényezőt is meg kell szorozni.

Normalizálás (normalization). A folytonos integrált egy $G = 1001$ pontból álló, 0 és 1 közötti egyenletes rácson közelítjük:

$$\text{grid} = [p_0, p_1, \ldots, p_{G-1}], \qquad p_j = j\Delta p, \quad \Delta p = 0.001.$$

Minden rácspontban kiszámítjuk az unnormalizált posterior értéket:

$$\text{unnorm}_j = \pi(p_j)\, p_j^k (1-p_j)^{n-k}.$$

A normalizáló konstans trapézszabállyal közelítve:

$$Z \approx \sum_{j=0}^{G-2} \frac{\text{unnorm}_j + \text{unnorm}_{j+1}}{2} \Delta p.$$

A normalizált posterior:

$$\pi_j = \frac{\text{unnorm}_j}{Z}.$$

Ekkor $\sum \pi_j \Delta p \approx 1$. A normalizálás nélkül görbe formája azonos, de a magasság nem sűrűség, és a posterior várható érték vagy intervallum hibásan súlyozott értéket adna.

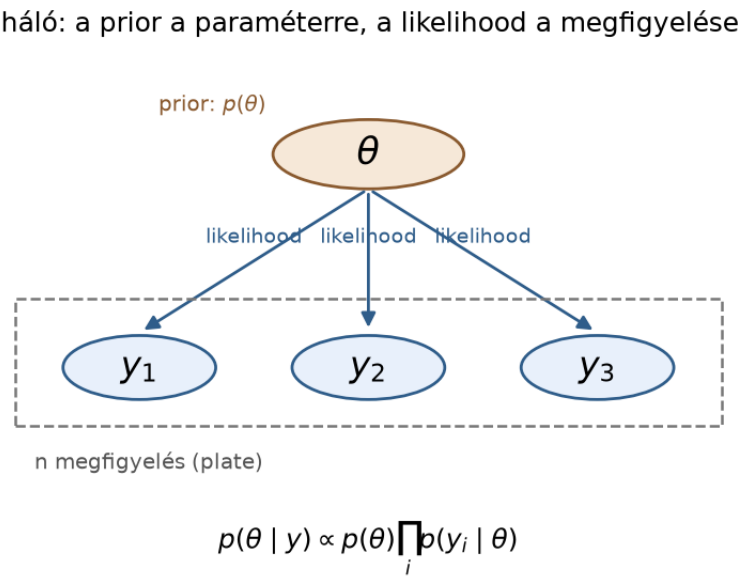
Predictive (posterior prediktív eloszlás). Egy új, a mintától független kezelt ügyfél sikerének valószínűsége, feltéve a modellt és a múlt adatot:

$$P(Y_{\text{new}} = 1 \mid D) = \int_0^1 p\,\pi(p \mid D)\,dp = \mathbb{E}[p \mid D].$$

Rácson ez a posterior várható értéke: `trapezoid(grid * posterior, grid)`. A prediktív valószínűség tehát nem a 0/1 kimenet, hanem a siker valószínűségének posterior-átlaga. Ez a mennyiség a paraméterbizonytalanságot átlagolja ki, nem az egyedi Y_{new} zaját adja meg; utóbbihoz a teljes posterior prediktív eloszlás tartozik.

A posterior credible interval (bayesi hitelességi intervallum) adott D és prior mellett a posterior tömeg 95%-át tartalmazza. Ez más értelmezés, mint a 15.2. alfejezetbeli konfidenciaintervallum ismételt mintavételi lefedettségi állítása.

19.1.2.1 Fogalmi ábra: a Bayesi háló szerkezete



19.1. ábra: Bayesi háló: a theta paraméterre ható prior és az n megfigyelésre ható likelihood; a posterior a kettő szorzata, normalizálva.

Az ábra a Bayesi frissítés gráfszerkezetét mutatja: a θ csomópont a közös paraméter, a y_i csomópontok a megfigyelések, a szaggatott keret pedig azt jelzi, hogy a megfigyelések azonos likelihood-modellt használnak. A nyilak iránya a generatív feltételezést jelöli (θ befolyásolja a y_i-t), nem oksági bizonyítékot. Az ábra nélkül is ugyanez a mondat áll: $p(\theta \mid y) \propto p(\theta) \prod_i p(y_i \mid \theta)$. A teljes generátor: `lab/figures/make_concept_figures.py` `bayes_network()`; a reprodukálható mag:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse, FancyArrowPatch, Rectangle

fig, ax = plt.subplots(figsize=(6.4, 3.6))
ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.axis("off")
ax.add_patch(Ellipse((0.5, 0.8), 0.2, 0.13, facecolor="#f6e8d8"))
ax.text(0.5, 0.8, r"$\theta$", ha="center", va="center", fontsize=14)
for x in (0.26, 0.5, 0.74):
    ax.add_patch(Ellipse((x, 0.4), 0.16, 0.12, facecolor="#e8f0fb"))
    ax.text(x, 0.4, r"$y_i$", ha="center", va="center")
```

```
    ax.add_patch(FancyArrowPatch((0.5, 0.735), (x, 0.465),
                 arrowstyle="-|>", mutation_scale=11, color="#2f5d8a"))
ax.add_patch(Rectangle((0.13, 0.29), 0.74, 0.24, fill=False, linestyle="--"))
ax.text(0.15, 0.23, "n megfigyelés (plate)")
plt.show()
```

19.1.3 Teljes minimális munkapélda

A kód Python 3.12-t, NumPy-t és SciPy-t igényel; a lab projektgyökeréből `uv sync` után `uv run python s073_grid_bayes.py` módon futtatható. Ha már van lock, használjon `uv run --locked`-ot. A kód egyszerre számítja a két priort és a minta növelésének hatását.

```
import numpy as np
from scipy.integrate import trapezoid

grid = np.linspace(0.0, 1.0, 1001)
step = grid[1] - grid[0]

def kernel(p_grid, k, n):
    return p_grid**k * (1.0 - p_grid)**(n - k)

def posterior_on_grid(prior, k, n):
    unnorm = prior * kernel(grid, k, n)
    z = trapezoid(unnorm, grid)
    return unnorm / z

def posterior_mean(post):
    return trapezoid(grid * post, grid)

def equal_tail_interval(post, alpha=0.05):
    # Trapézonkénti területekből CDF, majd lineáris kvantiliskeresés.
    areas = 0.5 * (post[:-1] + post[1:]) * np.diff(grid)
    cdf = np.concatenate(([0.0], np.cumsum(areas)))
    cdf[-1] = 1.0
    lo = np.interp(alpha / 2, cdf, grid)
    hi = np.interp(1.0 - alpha / 2, cdf, grid)
    return lo, hi

flat_prior = np.ones_like(grid)
skeptical_prior = 4.0 * (1.0 - grid)**3  # Beta(1,4), normált sűrűség

for name, prior in [("flat", flat_prior), ("skeptical Beta(1,4)", skeptical_prior)]:
    post = posterior_on_grid(prior, 6, 15)
    print(name,
          "mean=%.6f" % posterior_mean(post),
          "95% CI=", tuple(round(v, 4) for v in equal_tail_interval(post)),
          "normalization_error=%.1e" % (trapezoid(post, grid) - 1.0))

print("\nA minta növelése, rögzített k/n = 0.4 aránnyal:")
for k, n in [(6, 15), (24, 60), (240, 600)]:
    mean_flat = posterior_mean(posterior_on_grid(flat_prior, k, n))
    mean_skep = posterior_mean(posterior_on_grid(skeptical_prior, k, n))
    print(f"n={n:4d}  k={k:3d}  flat_mean={mean_flat:.4f}  skeptical_mean={mean_skep:.4f}")

# A posterior görbék ábrája, ha vizuális ellenőrzés is kell:
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 4))
for name, prior in [("flat", flat_prior), ("skeptical Beta(1,4)", skeptical_prior)]:
    post = posterior_on_grid(prior, 6, 15)
    ax.plot(grid, post, label=name)
```

```
ax.set_xlabel("p: konverziós valószínűség")
ax.set_ylabel("Posterior-sűrűség")
ax.set_title("n=15, k=6: két prior posteriorja")
ax.legend()
plt.tight_layout()
plt.show()
```

A kód kimenete analitikusan ellenőrizhető:

- Flat prior = Beta$(1,1)$; konjugált posterior Beta$(1+6, 1+15-6)$ = Beta$(7,10)$.
- Szkeptikus prior = Beta$(1,4)$; konjugált posterior Beta$(1+6, 4+15-6)$ = Beta$(7,13)$.

A posterior várható értékek:

prior	posterior	várható érték
flat Beta$(1,1)$	Beta$(7,10)$	$7/17 \approx 0.4118$
szkeptikus Beta$(1,4)$	Beta$(7,13)$	$7/20 = 0.35$

A flat prior posterior módusza $(7-1)/(17-2) = 0.4$, pontosan a mintaarány. A szkeptikus prior módusza $(7-1)/(20-2) = 1/3 \approx 0.333$; a prior a 0.2 felől húzza felfelé az adat 0.4-ét.

A minta növelése rögzített $k/n = 0.4$ arány mellett:

n	k	flat posterior várható értéke	szkeptikus posterior várható értéke
15	6	$7/17 \approx 0.4118$	$7/20 = 0.35$
60	24	$25/62 \approx 0.4032$	$25/65 \approx 0.3846$
600	240	$241/602 \approx 0.4003$	$241/605 \approx 0.3983$

A két prior közötti különbség $n = 15$-nél 0.0618, $n = 600$-nál már csak 0.002 körüli; a minta növekedésével a prior hatása eltűnik, de a priorválasztás továbbra is explicit marad.

19.1.3.1 Rácsfinomság, normalizálás és log-skálás kontroll

A $G = 1001$ pontos rács és a trapézszabály közelítés: a hibát nem elég megsejteni, meg kell mérni. Két kontroll kell: (1) a posterior összefoglalója stabil-e finomításra, és (2) a likelihood kiszámítása nem csordul-e alá nagy minta esetén. A naiv $p^k(1-p)^{n-k}$ szorzat ugyanis $n = 2000$, $k = 700$ mellett minden rácsponton 0.0-ra csökken, így $Z = 0$, és a posterior 0/0 miatt NaN lesz. A log-likelihood plusz maximum-eltolás (logsumexp-minta) megőrzi a relatív súlyokat.

```
import numpy as np
from numpy import trapezoid

def make_grid(size):
    return np.linspace(0.0, 1.0, size)

def log_kernel(grid, k, n):
    logp = np.full_like(grid, -np.inf)
    logq = np.full_like(grid, -np.inf)
    inner = (grid > 0.0) & (grid < 1.0)
    logp[inner] = np.log(grid[inner])
    logq[inner] = np.log1p(-grid[inner])
    if k == 0:
        logp[grid == 0.0] = 0.0
```

```
    if n - k == 0:
        logq[grid == 1.0] = 0.0
    return k * logp + (n - k) * logq

def posterior(prior, k, n, size):
    grid = make_grid(size)
    with np.errstate(divide="ignore"):
        log_unnorm = np.log(prior(grid)) + log_kernel(grid, k, n)
    log_unnorm = log_unnorm - log_unnorm.max()   # logsumexp-eltolás
    unnorm = np.exp(log_unnorm)
    return grid, unnorm / trapezoid(unnorm, grid)

def summary(post, grid, alpha=0.05):
    areas = 0.5 * (post[:-1] + post[1:]) * np.diff(grid)
    cdf = np.concatenate(([0.0], np.cumsum(areas)))
    cdf = cdf / cdf[-1]
    mean = trapezoid(grid * post, grid) / trapezoid(post, grid)
    return mean, np.interp(alpha / 2, cdf, grid), np.interp(1 - alpha / 2, cdf, grid)

flat = lambda grid: np.ones_like(grid)
skeptical = lambda grid: 4.0 * (1.0 - grid) ** 3

for name, prior in [("flat", flat), ("Beta(1,4)", skeptical)]:
    for size in (11, 101, 1001, 10001):
        grid, post = posterior(prior, 6, 15, size)
        mean, lo, hi = summary(post, grid)
        print(f"{name:8s} G={size:5d} mean={mean:.9f} CI=[{lo:.6f}, {hi:.6f}]")

grid = make_grid(1001)
raw = grid ** 700 * (1 - grid) ** 1300
print("nulla rácspont:", int((raw == 0).sum()), "Z =", trapezoid(raw, grid))
grid, post = posterior(flat, 700, 2000, 1001)
print("log-skála:", summary(post, grid), "integrál:", trapezoid(post, grid))

grid, post = posterior(flat, 6, 15, 1001)
mass = post / post.sum()
print("sűrűség:", trapezoid(post, grid), "tömeg összege:", mass.sum(),
      "tömeg trapézzel:", trapezoid(mass, grid))
```

A rácsfinomítás eredménye $n = 15$, $k = 6$ mellett:

G	flat mean	flat 95% CI	Beta$(1,4)$ mean	Beta$(1,4)$ 95% CI
11	0.411776407	[0.165233, 0.674850]	0.350043618	[0.124249, 0.588963]
101	0.411764706	[0.197184, 0.645983]	0.350000000	[0.162484, 0.565851]
1001	0.411764706	[0.197530, 0.645657]	0.350000000	[0.162883, 0.565505]
10001	0.411764706	[0.197534, 0.645654]	0.350000000	[0.162886, 0.565502]

A durva $G = 11$ rács a flat posterior 95%-os intervallumának bal végét 0.1975-ről 0.1652-re tolja (0.032 hiba), a jobb végét pedig 0.6457-ről 0.6749-re; a várható érték ezzel szemben alig mozdul (0.411776 a pontos $7/17 = 0.411765$ helyett). A várható érték tehát önmagában félrevezető stabilitási diagnosztika: a kvantilisek érzékenyebbek. $G = 101$ és $G = 1001$ között az intervallum már csak $4 \cdot 10^{-4}$-rel változik, $G = 1001$ és $G = 10001$ között $4 \cdot 10^{-6}$-nál kevesebbel: a trapézszabály másodrendű, ezért a lépésköz tizedelése nagyjából századára csökkenti a hibát.

A log-skálás kontroll $n = 2000$, $k = 700$ mellett (a posterior Beta$(701, 1301)$, analitikus várható értéke $701/2002 = 0.350150$, szórása $\sqrt{ab/((a+b)^2(a+b+1))} = 0.01066$):

G	mean	95% CI
101	0.350150	[0.326270, 0.374438]
1001	0.350150	[0.329367, 0.371209]
10001	0.350150	[0.329404, 0.371180]

A közvetlen számításnál `raw` mind a 1001 rácsponton nulla, $Z = 0$, a posterior mindenhol NaN. A log-skálás változat integrálja 1, a várható érték az analitikus értékkel 10^{-6}-on belül egyezik. A keskeny posterior ($\sigma = 0.0107$) esetén a $G = 101$ rács a 95%-os intervallumot $[0.3294, 0.3712]$-ről $[0.3263, 0.3744]$-re szélesíti, azaz a szórás mintegy 30%-ával; $G = 1001$-től a finomítás már nem változtat érdemben. A tanulság: a rácsot a posterior szélességéhez kell választani, nem a mintához, és nagy n-nél a log-skála nem kényelmi, hanem szükséges eszköz.

A normalizálás két egyenértékű útja ugyanazon a $G = 1001$ rácson:

változat	képzés	mérőszám	mean
sűrűség	$u/\operatorname{trapz}(u, \text{grid})$	$\int \pi \approx 1.000000$	0.411764706
diszkrét tömeg	$u/\sum_j u_j$	$\sum_j w_j = 1.000000$	0.411764706

A két út ugyanazt a várható értéket és intervallumot adja, mert a közös Δp tényező kiesik. A hiba akkor keletkezik, ha a tömeget sűrűségként kezeljük: $\operatorname{trapz}(w, \text{grid}) = \Delta p = 0.001$, azaz a látszólagos „normalizáltsági hiba” -0.999, holott a tömeg összege pontosan 1; a trapéz-súlyozott $\operatorname{trapz}(p\,w)$ pedig a valódi várható érték Δp-szeresét, 0.000412-t adja. Ezért a jelentésben mindig rögzíteni kell, hogy sűrűséget ($\int = 1$) vagy tömeget ($\sum = 1$) közlünk, és a kettőt nem szabad keverni.

19.1.4 Eredmény értelmezése

Kézi kontroll: a Beta$(7, 10)$ és Beta$(7, 13)$ sűrűség integrálja 1, tehát a rácsos posterior normalizáltsága az 1-től való eltérés $\approx 10^{-9}$ nagyságrendben várható. Az unnormalizált posterior `shape`-e a griddel egyező `(1001,)`, `dtype` `float64`, minden érték nemnegatív, a tömeg a 0–0.8 tartományban koncentrálódik. A posterior módusz a grid finomságától függően a maximum indexénél kereshető; a lineáris kvantilisintervallum pontosságát a `step = 0.001` rács korlátozza, tehát a credible interval határait legfeljebb néhány ezrelék hibával érdemes jelenteni.

Fontos: ez a posterior a *kezelési ág* konverziós valószínűségét becsli, nem önmagában az oksági hatást. A kontrollággal való ATE-összehasonlítás a randomizálás és a 18.1. alfejezetbeli azonosítási feltételek témája. A posterior paraméterbizonytalanság, a prediktív valószínűség a paraméter átlaga egy új ügyfélnél, a szimulációs hiba pedig a 15.4. alfejezetben bevezetett harmadik bizonytalanságfajta; itt nincs Monte Carlo-ismétlés, ezért szimulációs standard hibát nem számolunk.

Rácsközelítés: a posterior rácson számított diszkrét súlyait a teljes összeggel kell normalizálni; a rács felbontása és szélei ellenőrizendők. A priori megválasztása nem puszta technikai részlet, ezért érzékenységi vizsgálat ajánlott.

19.1.5 Feladatok

1. **Értelmezés:** A flat priorral kapott posterior Beta$(7, 10)$ módusza 0.4, várható értéke 0.4118. Miért tér el a kettő, és miért a mintaarány a módusz? Indokold a Beta-módusz és -átlag képletéből.

2. **Implementálás:** Írj `posterior_summary(prior, k, n)` függvényt, amely a fenti `grid`-et használja, normalizálja a posterior-t, és visszaad egy `(posterior, mean, mode, lo, hi)` rendezett ötöst, ahol `mode` a legnagyobb posterior-sűrűségű rácspont, `lo` és `hi` a 95%-os equal-tailed intervallum. Ellenőrizd a normalizáltságot $|\text{integrál} - 1| < 10^{-8}$ hibahatárral. Futtasd $k = 3,\ n = 10$, flat priorral; add meg a várt `mean` és `mode` értéket.

3. **Hibadiagnózis:** Egy elemző a $k = 6,\ n = 15$, flat prior esetben így „átlagol”:

   ```
   unnorm = flat_prior * kernel(grid, 6, 15)
   mean_wrong = trapezoid(grid * unnorm, grid)
   evidence_wrong = trapezoid(unnorm, grid)
   ```

   Mi a két hiba? Miért nem a posterior várható érték a `mean_wrong`, és miért nem teljes binomiális evidence az `evidence_wrong`? Számítsd ki a javítást, és add meg a flat priorral a teljes binomiális marginális likelihood pontos értékét $n = 15,\ k = 6$ esetén.

4. **Megváltoztatott prior / transzfer:** Ugyanaz a minta, $n = 15,\ k = 6$, de a prior most optimista Beta$(4, 1)$ sűrűség, $\pi(p) = 4p^3$. Írd fel a posterior Beta-paramétereit, várható értékét és móduszát. Mit tapasztal? Ismételje meg $n = 60,\ k = 24$ mellett, és fogalmazza meg, hogyan függ a prior hatása a minta nagyságától.

19.1.6 Megoldások és indoklás

1. A Beta(α, β) módusz, ha $\alpha, \beta > 1$: $(\alpha - 1)/(\alpha + \beta - 2)$. Itt $\alpha = 7,\ \beta = 10$, tehát módusz $6/15 = 0.4$. A likelihood kernel deriváltjának elsőrendű feltétele $k/p - (n-k)/(1-p) = 0$, ami szintén $p = k/n = 0.4$-et ad flat prior mellett. A várható érték $\alpha/(\alpha + \beta) = 7/17 \approx 0.4118$; a sűrűség jobb széle hosszabb, ezért az átlag felfelé tolódik a móduszhoz képest. A módusz a legvalószínűbb paraméterpont, az átlag a posterior szerinti valószínűséggel súlyozott p-átlag.

2. Lehetséges implementáció:

   ```
   def posterior_summary(prior, k, n):
       post = posterior_on_grid(prior, k, n)
       assert abs(trapezoid(post, grid) - 1.0) < 1e-8
       mean = posterior_mean(post)
       mode = grid[int(np.argmax(post))]
       lo, hi = equal_tail_interval(post)
       return post, mean, mode, lo, hi

   _, mean, mode, lo, hi = posterior_summary(flat_prior, 3, 10)
   print(mean, mode, lo, hi)
   ```

   A flat priorral a posterior Beta$(1 + 3, 1 + 10 - 3) =$ Beta$(4, 8)$: várható érték $4/12 = 1/3 \approx 0.3333$, módusz $3/10 = 0.3$. A numerikus `mean`-nek 10^{-3} nagyságrendű grid-hibán belül egyeznie kell; a `mode` az argmax miatt pontosan a legközelebbi rácspont, 0.300 vagy 0.301 körül.

3. A `mean_wrong` az $\int p \cdot \mathrm{unnorm}(p)\,dp$ érték, nem oszt a Z-vel. A flat $k = 6,\ n = 15$ esetben $Z_{\text{kernel}} \approx \int_0^1 p^6(1-p)^9\,dp = B(7, 10) \approx 1.249 \cdot 10^{-5}$, és a helyes átlag:

$$\text{mean} = \frac{\int p\,\mathrm{unnorm}(p)\,dp}{Z_{\text{kernel}}} \approx 0.4118.$$

A `mean_wrong` nagyságrendje 10^{-5}, tehát nem értelmezhető prediktív valószínűségként. Az `evidence_wrong` nem teljes binomiális evidence, mert hiányzik a $\binom{n}{k} = 5005$ konstans. A helyes marginális likelihood flat priorral:

$$P(k) = \binom{15}{6} \int_0^1 p^6(1-p)^9\,dp = \frac{1}{n+1} = \frac{1}{16} = 0.0625.$$

A javítás tehát: `Z = trapezoid(unnorm, grid); mean = trapezoid(grid*unnorm, grid) / Z`; `exact_evidence = comb(15, 6) * Z` (rácsos hiba mellett).

4. A Beta$(4, 1)$ priorban $\alpha = 4,\ \beta = 1$. A konjugált frissítés:

$$\alpha' = 4 + 6 = 10, \qquad \beta' = 1 + (15 - 6) = 10.$$

A posterior Beta$(10, 10)$: várható érték $10/20 = 0.5$, módusz $(10-1)/(20-2) = 0.5$. A prior erős, a minta nem elég a 0.4-es gyakoriság teljes átültetéséhez: a prior felfelé, a likelihood lefelé húz, az egyensúly éppen 0.5-re esik. $n = 60,\ k = 24$ esetén $\alpha' = 4+24 = 28$, $\beta' = 1 + 36 = 37$; várható érték $28/65 \approx 0.4308$, módusz $27/63 \approx 0.4286$. A nagyobb minta már érezhetően a 0.4 felé húzza a posterior-t, tehát a priorhatás csökken.

19.1.7 Gyakori tévedés és önellenőrzés

A leggyakoribb hiba az implicit prior elhallgatása. Példa: $n = 5,\ k = 0$ megfigyelés esetén a mintaarány 0, de a posterior erősen priortól függ. Flat priorral a posterior Beta$(1, 6)$ várható értéke $1/7 \approx 0.1429$. Szkeptikus Beta$(1, 4)$ priorral a posterior Beta$(1, 9)$ várható értéke 0.1. Optimista Beta$(4, 1)$ priorral a posterior Beta$(4, 6)$ várható értéke 0.4. Ugyanaz a nulla sikerű minta tehát 0.1 és 0.4 közötti prediktív valószínűséget is adhat. Aki „csak likelihoodot” mutat, de nem nevezi meg a priort, az valójában priorválasztást rejtett el; a prior elhallgatása nem teszi az elemzést feltételmentessé.

Önellenőrzés:

- A normalizált posterior trapézinintegrálja 1 ± 10^{-8}.
- A posterior nemnegatív, `shape == grid.shape`, és a módusz a likelihood magas tartományában van.
- A prediktív valószínűség $[0, 1]$-beli, és Beta-posterior esetén a posterior átlaggal egyezik.
- A credible interval nem keverhető a 15.4. alfejezetbeli gyakori CI-lefedettségi állítással.
- A priorok közötti különbség explicit, és a minta növelésével csökken, de nem tűnik el automatikusan.

19.1.8 Rövid összefoglaló

A rácsos Bayesi frissítés a $p \in [0,1]$ paramétert diszkrét pontokra bontja, minden pontban kiszámítja a prior és a Bernoulli-likelihood szorzatát, majd Z-vel normalizálja. A flat priorral a posterior Beta$(7,10)$, várható értéke 0.4118; a szkeptikus Beta$(1,4)$ priorral Beta$(7,13)$, várható értéke 0.35. A prediktív valószínűség a posterior átlag, a normalizálás nélkül pedig sem prediktív, sem intervallum-állítás nem tehető. A prior explicit megadása kis minta esetén meghatározó, nagy minta esetén hatása csökken. Következő alkalmazásként a 19.2. alfejezetben a credible interval és posterior predictive check kerül elő, majd a teljes PyMC-modellezés.

19.2 Credible interval és posterior predictive check

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyMC, ArviZ

Előfeltétel: 19.1. alfejezet · Célkészség: Credible interval és posterior predictive check · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

19.2.1 Cél és konkrét helyzet

A 19.1. alfejezetben rácsos posteriorral becsültük egy Bernoulli-folyamat p siker-valószínűségét. Most nem elégszünk meg a pontbecsléssel: intervallumot és modell-ellenőrzést készítünk.

A konkrét bemenet a 15 elemű megfigyelt sorozat:

$$y_{\text{obs}} = [0,0,1,0,0,0,1,0,0,1,0,1,0,1,1]$$

A megfigyelési egység egyetlen bináris kimenet (0 vagy 1). Az $n = 15$ elemű mintában $s = 6$ siker és 9 kudarc van. A cél:

1. 95%-os **credible interval** a $p \in [0,1]$ paraméterre;
2. **posterior predictive** eloszlás új, azonos szerkezetű adatra;
3. **posterior predictive check** (PPC), amely az előrejelzett adatot két diagnosztikai statisztikával veti össze a megfigyelttel.

Mielőtt a kódot megnéznéd, adj magadnak gyors becslést. A flat priorral a 19.1. alfejezet szerint a posterior Beta$(7,10)$, várható értéke $7/17 \approx 0.4118$. A 15 új megfigyelés várható sikere tehát $15 \cdot 7/17 \approx 6.18$, a megfigyelt $s = 6$ ennek közelében van. A 95%-os intervallum nagyságrendje a posterior Beta-szórása alapján körülbelül $[0.20, 0.64]$ lehet; ezt a kódban ellenőrizzük.

19.2.2 Credible interval: a posterior kvantilisei

A bayesi keretben a paraméter valószínűségi változó. A posterior sűrűség $\pi(p \mid y)$ azt fejezi ki, hogy a prior és a likelihood együttes elfogadása után mely p-értékek a legplauzibilisebbek. Egy $1-\alpha$ szintű **credible interval**, magyarul hiteles intervallum, olyan $[L, U]$ tartomány, amelyre

$$\int_L^U \pi(p \mid y)\, dp = 1 - \alpha.$$

A leggyakoribb, kvantilisen alapuló változat az **equal-tailed interval**: a posterior eloszlás $\alpha/2$ és $1-\alpha/2$ kvantilise határolja. Ha a posterior sűrűséget a g rácson diszkréten ismerjük, akkor a kumulált tömegek alapján keressük meg a kvantiliseket:

$$L = q_{\alpha/2}(p \mid y), \quad U = q_{1-\alpha/2}(p \mid y).$$

A 19.1. alfejezet rácsos feladatában flat priorral a posterior $\mathrm{Beta}(7, 10)$. A pontos equal-tailed 95%-os intervallum a SciPy `beta.ppf([0.025, 0.975], 7, 10)` hívásával kapható meg; az ott megadott ponthoz közeli rácspont pedig a normalizált posteriorból kereshető ki. A két értéknek $1/2001$ nagyságrendű rácshibán belül egyeznie kell.

Az equal-tailed intervallum nem az egyetlen választás. A **highest posterior density interval** (HPDI) azt a legrövidebb tartományt keresi, amelyre a posterior tömeg $1-\alpha$, ezért ferde eloszlásnál eltérhet az equal-tailed változattól. Itt a $\mathrm{Beta}(7, 10)$ csak enyhén ferde, ezért a különbség kicsi; a tanulási célhoz az equal-tailed kvantilisintervallum egyszerűbb és közvetlenül kapcsolódik a 15.2. alfejezetben használt kvantilis fogalmához.

A bayesi intervallum állítása **feltételes a priorra és a modellre**: adott y mellett a posterior valószínűség 0.95, hogy p az intervallumban van. Ez nem ugyanaz, mint a konfidenciaintervallum ismételt mintavételes lefedettségi garanciája. A különbséget érdemes azonnal kimondani, még a kód előtt:

Kérdés	95% credible interval	95% confidence interval
Mit állít?	A posterior szerint p 0.95 valószínűséggel az intervallumban van	Az eljárás ismételt mintavételnél 95%-ban tartalmazza a rögzített p-t
Függ a priortól?	Igen	Nem
Garancia rögzített p-re?	Csak ha a prior-modell együttesen kalibrált	Igen, a statisztikai feltételek mellett

A kettő tehát nem automatikusan felcserélhető, még akkor sem, ha egy adott példában a számértékük hasonló.

19.2.3 Predictive distribution és prior predictive

A posteriorból nemcsak a paraméterre, hanem **új megfigyelésre** is tehetünk valószínűségi állítást. Egy új $\tilde{y}$ megfigyelés **posterior predictive** eloszlása a likelihood posterior szerinti keveréke:

$$f(\tilde{y} \mid y) = \int_0^1 f(\tilde{y} \mid p)\, \pi(p \mid y)\, dp.$$

A keverés miatt nem elég `p`-t a posterior átlaggal vagy módusszal helyettesíteni, mert akkor elvész a paraméterbizonytalanság. Egyetlen új Bernoulli-megfigyelésre a posterior átlag jelenik meg:

$$\Pr(\tilde{Y} = 1 \mid y) = \int_0^1 p\, \pi(p \mid y)\, dp = \mathbb{E}\left[p \mid y\right] = \frac{7}{17} \approx 0.4118.$$

Ha viszont $m = 15$ új elemű összeget tekintünk, akkor a béta-posteriorral a sikerösszeg eloszlása **béta-binomiális**:

$$\Pr(\tilde{K} = k \mid y) = \int_0^1 \binom{15}{k} p^k (1-p)^{15-k} \,\mathrm{Beta}(p; 7, 10)\, dp.$$

Ennek várható értéke $15 \cdot 7/17 = 6.18$, de a varianciája nagyobb, mint egy rögzített $p = 7/17$ melletti Binomial$(15, 7/17)$ eloszlás varianciája, mert a p bizonytalansága is hozzáadódik.

A **prior predictive** eloszlás ugyanezt a keveréket az adat megfigyelése **előtt**, a priorral számítja:

$$f(\tilde{y}) = \int_0^1 f(\tilde{y} \mid p)\, \pi(p)\, dp.$$

Ez a prior ellenőrzésére szolgál: mielőtt a likelihood frissítené a modellt, megmutatja, milyen adatokat tart a prior eleve valószínűnek. Flat Beta$(1, 1)$ prior esetén a 15 kísérletből álló sikerösszeg prior predictive eloszlása egyenletes:

$$\Pr(\tilde{K} = k) = \int_0^1 \binom{15}{k} p^k (1-p)^{15-k}\, dp = \frac{1}{16}, \quad k = 0, \dots, 15.$$

Ez jól ellenőrizhető kézi invariant: a flat prior nem részesít előnyben semmilyen sikergyakoriságot, ezért a lehetséges 16 összeg mindegyike ugyanolyan plauzibilis.

19.2.4 Posterior predictive check: replikált adat és két diagnosztika

A **posterior predictive check** (PPC) a modell illeszkedését a következő módon vizsgálja. Minden $r = 1, \dots, S$ replikációban:

1. sorsolunk egy $p^{(r)}$ paramétert a posteriorból;
2. abból generálunk egy $y^{\mathrm{rep},(r)}$ adatsort, amelynek mérete és megfigyelési egysége megegyezik az eredeti y_{obs} adatéval;
3. kiszámítunk egy-egy $T(y)$ diagnosztikai statisztikát mind a replikált, mind a megfigyelt adatra.

Itt két statisztikát használunk. Az első a sikerösszeg:

$$T_1(y) = \sum_{i=1}^{n} y_i,$$

amely az átlagos szintet ellenőrzi. A második a leghosszabb egyesekből álló futam hossza, $T_2(y)$; ez érzékeny arra, ha a generált sorozatban a sikerek csomósodnak vagy éppen túl szabályosan váltakoznak. A PPC lokális p-értéke, a p_B, azt méri, hogy a replikált statisztikák mekkora része éri el a megfigyelt értéket:

$$p_B = \frac{1}{S} \sum_{r=1}^{S} \mathbf{1}\left[T(y^{\mathrm{rep},(r)}) \geq T(y_{\mathrm{obs}}) \right].$$

A $p_B \approx 0$ vagy $p_B \approx 1$ azt jelzi, hogy a megfigyelt adat a posterior predictive eloszlás valamelyik szélére esik, tehát a modell az adott statisztikát rosszul írja le. A p_B azonban **nem** klasszikus nullhipotézis-p-érték: ugyanazt az adatot használja a posterior illesztéséhez és az ellenőrzéshez is, ezért a gyakorisági kalibrációja nem automatikus. Kétoldali diagnosztikaként a $2\min(p_B, 1 - p_B)$ érték szolgálhat, de küszöb helyett a modellhibák közötti rangsorolásra érdemes használni.

19.2.4.1 PPC, holdout és keresztvalidáció: három különböző kérdés

A posterior predictive check nem független tesztadat, ezért nem alkalmas a predikciós generalizáció igazolására. A három eljárás különböző kérdésre válaszol:

Eljárás	Milyen adatot használ?	Kérdés	Mit bizonyít?
Posterior predictive check (PPC)	ugyanazt a megfigyelt adatot, amelyre a posterior illeszkedett	a modell reprodukálja-e a megfigyelt mintázatot?	modellkritika; nem generalizációs bizonyíték
Holdout	a modell az edzésrészen illeszkedik, a tesztadathoz nem nyúl	mennyire pontos a predikció új adaton?	egyetlen, független becslés a generalizációra
Keresztvalidáció (K-fold/LOO)	minden pont egyszer kimarad az illesztésből	mennyire stabil a prediktív teljesítmény?	átlagos generalizáció, kisebb szórás

A PPC tehát akkor hasznos, ha azt kérdezzük, hogy a modell szerkezete (például az egyenlő valószínűség és a függetlenség) összeegyeztethető-e a megfigyelt sorozattal; a futamhossz-statisztika éppen az ilyen strukturális eltérést mutatja. Ha viszont az a kérdés, hogy egy új ügyfél kimenetét mennyire jól jelezzük előre, a modellt a tanítóadaton kell illeszteni, és a tesztadaton értékelni; a PPC erre nem ad választ, mert a megfigyelt adat már részt vett a posterior kialakításában. Egy jelentésben a kettőt külön szakaszban kell közölni: „modellilleszkedés” (PPC) és „prediktív validáció” (holdout/CV). A PPC nem helyettesíti a holdout-ot, és a holdout sem igazolja a modell kauzális értelmezését.

A **calibration** szó itt két dolgot jelenthet. Az egyik a posterior predictive intervallumok kalibrációja: ha a paramétert valóban a priornak megfelelően sorsoljuk, az adatot a likelihood szerint generáljuk, majd a posteriorból 95%-os predikciós intervallumot készítünk, a jövőbeli megfigyelés a sok ismétlés során közel 95%-ban esik bele. Ez a **joint prior-data** folyamatra vonatkozó állítás. Egy rögzített, ismeretlen p mellett viszont egy credible interval nem garantál frekventista lefedettséget; ezt az alábbi gyakori tévedésnél számpéldával is megmutatjuk. A másik jelentés a prediktív valószínűségek pontossága: akkor nevezzük jól kalibráltnak a modellt, ha az 50%-os valószínűséggel jelzett események körülbelül 50%-ban következnek be.

19.2.5 Minimális futtatható munkapélda

A következő, NumPy/SciPy alapú minimal script a rácsos posteriorból kiindulva számítja ki a credible intervalt, generál posterior predictive replikációkat, és két PPC-statisztikát hasonlít össze. A labor környezetében, a projekt gyökeréből futtatható:

```
# Linux/macOS terminál, a tankönyvi lab projekt gyökerében
uv run python ppc_grid.py

# Windows PowerShell, a projekt gyökerében
uv run python ppc_grid.py
```

A kód kimenetét itt **nem** ellenőrzött futtatásként közöljük; a seed megismételhetőséget segít egy gépen belül, de nem garantál bitazonos eredményt minden NumPy-verzión.

```
import numpy as np
from scipy.stats import beta as beta_dist

rng = np.random.default_rng(20261020)

y_obs = np.array([0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1])
n = len(y_obs)              # 15
s = int(y_obs.sum())        # 6


def longest_run(x: np.ndarray) -> int:
    best = cur = 0
    for v in x:
        if v == 1:
            cur += 1
            best = max(best, cur)
        else:
            cur = 0
    return best


# Rácsos posterior: flat Beta(1,1) prior
grid = np.linspace(0.0, 1.0, 2001)
prior = np.ones_like(grid)
lik = grid**s * (1.0 - grid) ** (n - s)
post = prior * lik
post /= post.sum()                        # normalizálás: sum(post) == 1

# 95%-os equal-tailed credible interval a rácsból
cdf = np.cumsum(post)
lo = grid[int(np.searchsorted(cdf, 0.025))]
hi = grid[int(np.searchsorted(cdf, 0.975))]

# Posterior predictive replikációk
S = 5000
p_draws = rng.choice(grid, size=S, p=post)   # shape: (S,)
y_rep = rng.binomial(1, p_draws[:, None], size=(S, n))  # shape: (S, 15)

sum_rep = y_rep.sum(axis=1)                                # shape: (S,)
run_rep = np.array([longest_run(row) for row in y_rep])  # shape: (S,)

t1_obs = s
t2_obs = longest_run(y_obs)
p_b_sum_right = float(np.mean(sum_rep >= t1_obs))
p_b_run_right = float(np.mean(run_rep >= t2_obs))
p_b_sum_two = float(2.0 * min(p_b_sum_right, 1.0 - p_b_sum_right))
p_b_run_two = float(2.0 * min(p_b_run_right, 1.0 - p_b_run_right))

print(f"posterior mean: {np.sum(grid * post):.4f} (elvárás: {7/17:.4f})")
print(f"grid CI:        [{lo:.3f}, {hi:.3f}]")
print(f"Beta(7,10) CI:  {beta_dist.ppf([0.025, 0.975], 7, 10).round(3)}")
print(f"pred. sum mean: {sum_rep.mean():.2f} (elvárás: {n*7/17:.2f})")
print(f"observed: sum={t1_obs}, longest_run={t2_obs}")
print(f"PPC sum: right={p_b_sum_right:.3f}, two-sided={p_b_sum_two:.3f}")
print(f"PPC run: right={p_b_run_right:.3f}, two-sided={p_b_run_two:.3f}")
```

A kód minden replikációban a posteriorból sorsol p-t, majd abból generálja a 15 Bernoulli-megfigyelést. A replikált sorok száma S, az oszlopok száma n, tehát a replikáció nem változtatja meg az eredeti megfigyelési egységet. A statisztikák közül `sum_rep` a sikerösszeg, `run_rep` a leghosszabb sikeres futam.

19.2.6 Eredmény értelmezése és kézi kontroll

A kód futtatása nélkül is rögzíthetők az elméletileg várt ellenőrző pontok:

- A normalizált rácstömeg összege pontosan 1 a `post /= post.sum()` lépés után; a `float` kerekítés miatt 10^{-16} nagyságrendű eltérés elfogadható.
- A posterior átlag `np.sum(grid * post)` a Beta$(7, 10)$ várható értékéhez, $7/17 = 0.4118$-hoz tart. A rács és a folytonos béta közötti numerikus eltérés tipikusan 10^{-4} alatt marad.
- A posterior predictive sikerösszeg átlaga a béta-binomiális várható érték $15 \cdot 7/17 \approx 6.18$ körül várható. A Monte Carlo standard error a predikciós szórással (≈ 2.5) és $S = 5000$-rel számolva körülbelül 0.04, ezért a ± 0.05 tűrés elfogadható a futás ellenőrzéséhez.
- A gridből számolt credible intervalnak a `beta.ppf` intervallumhoz közelinek kell lennie; a rácshiba a 2001 pontból adódóan kisebb 0.001-nél. A megfigyelt $T_1 = 6$ a prediktív átlag alatt van, de a prediktív szórás tartományában.

A PPC-számok értelmezésénél ne egyetlen gépi küszöböt nézzünk. Ha például a `p_b_sum_two` 0.2 és 0.8 között van, a sikerösszeg nem mutat durva ütközést a modellel. Ha viszont a leghosszabb futamra a kétoldali érték 0.05 alá esik, akkor a modell nem jól ragadd meg a megfigyelt sorozat szomszédsági mintázatát; ilyenkor érdemes az időbeli függőséget vagy az egyenlő valószínűség feltételét gyanúba fogni. A `p_B` tehát diagnosztika, nem a parameter igazságáról szóló döntés.

Intervallumértelmezés: a 95%-os credible interval a modell és a prior feltételei mellett 95% posterior tömeget tartalmaz. Ettől különbözik a gyakorisági konfidenciaintervallum lefedettségi állítása. A posterior predictive check modellspecifikációs diagnosztika, nem az általános modelligazság igazolása.

19.2.7 Feladatok

1. Értelmezés. A futás után a 95%-os equal-tailed credible interval körülbelül $[0.20, 0.64]$. Kollégád ezt mondja: „Ebből következik, hogy a valódi p 95% valószínűséggel 0.20 és 0.64 között van, tehát ha 100 hasonló mintát gyűjtenénk, akkor 95 ilyen intervallum tartalmazná a valódi p-t.” Melyik mondatrész igaz, melyik hamis? Pontosan nevezd meg, milyen feltétel mellett igaz az első rész, és miért nem következik belőle a második.

2. Megvalósítás. Írj egy `quantile_interval(post, grid, level)` függvényt, amely adott `level` mellett equal-tailed credible intervalt ad vissza. Futtasd 80%-os szinten, és növeld a posterior predictive replikációk számát $S = 10{,}000$-re. Ellenőrizd a `p_draws`, `y_rep`, `sum_rep` alakját, és add meg mindkét PPC-statisztika jobb oldali és kétoldali értékét. Az input a fenti `y_obs` és a flat prior.

3. Hibakeresés. Az alábbi kódrészlet a posterior predictive eloszlást a posterior móduszával próbálja generálni:

```
p_map = grid[np.argmax(post)]
y_wrong = rng.binomial(1, p=p_map, size=(S, n))
wrong_mean = y_wrong.sum(axis=1).mean()
print(wrong_mean)
```

Mi a statisztikai hiba? Melyik varianciatag hiányzik a generált eloszlásból? Milyen következménye lehet ennek a PPC-re? Javítsd úgy a sort, hogy a posterior bizonytalanságát is megőrizze.

4. Transzfer megváltozott priorra. Ismételd meg a teljes számítást a szkeptikus Beta$(1, 4)$ priorral, amelynek sűrűsége arányos $(1 - p)^3$-nel. Mi lesz a posterior család és a posterior predictive sikerösszeg várható értéke? Hasonlítsd össze a flat és a szkeptikus prior predictive

eloszlást is a 15 kísérletből álló, még nem látott mintára. Az eredmény hogyan változtatja meg a megfigyelt $T_1 = 6$ értelmezését?

19.2.8 Megoldások és indoklás

1. A mondat első fele a bayesi Markov-feltételekkel pontosítható: *ha* elfogadjuk a Beta$(1, 1)$ priort és a Bernoulli-likelihoodot, akkor adott y mellett a posterior valószínűség 0.95, hogy $p \in [L, U]$. Ez posterior valószínűségi állítás a paraméterről. A második fele hamis: abból, hogy a posterior tömeg 95%-ot ad az intervallumnak, nem következik, hogy az eljárás rögzített, ismeretlen p mellett ismételt mintavételnél 95%-os lefedettséget ad. A konfidenciaintervallum lefedettsége az eljárás ismételt tulajdonsága, és nem függ a priortól; a credible interval ezt nem garantálja. A két fogalom tehát más kérdésre válaszol.

2. Lehetséges megoldás:

```
def quantile_interval(post, grid, level):
    lo_q = (1.0 - level) / 2.0
    hi_q = 1.0 - lo_q
    cdf = np.cumsum(post)
    lo = grid[int(np.searchsorted(cdf, lo_q))]
    hi = grid[int(np.searchsorted(cdf, hi_q))]
    return lo, hi

lo80, hi80 = quantile_interval(post, grid, 0.80)
print(f"80% grid CI: [{lo80:.3f}, {hi80:.3f}]")
print("80% Beta CI:", beta_dist.ppf([0.10, 0.90], 7, 10).round(3))

S = 10000
p_draws = rng.choice(grid, size=S, p=post)
y_rep = rng.binomial(1, p_draws[:, None], size=(S, n))
sum_rep = y_rep.sum(axis=1)

assert p_draws.shape == (S,)
assert y_rep.shape == (S, n)
assert sum_rep.shape == (S,)

p_b_right = float(np.mean(sum_rep >= s))
p_b_two = 2.0 * min(p_b_right, 1.0 - p_b_right)
print(f"PPC sum: right={p_b_right:.3f}, two-sided={p_b_two:.3f}")
```

A 80%-os intervallum szűkebb, mint a 95%-os, mert a posterior eloszlás középső tömegét fogja közre; a megfigyelt adat nem változott. A replikációk alakja a befoglaló mátrix dimenzióival ellenőrizhető.

3. A `p_map` a posterior módusza, azaz egyetlen legvalószínűbb pont. Ha minden replikációt ezzel az egy értékkel generálunk, akkor valójában Binomial$(15, p_{\text{map}})$ eloszlást kapunk, nem a posterior predictive-eloszlást. A béta-binomiális varianciája két tagra bomlik:

$$\operatorname{Var}(K) = \mathbb{E}[nP(1-P)] + \operatorname{Var}(nP),$$

ahol az első tag a mintavételi zaj, a második a paraméter posterior bizonytalansága. A rögzített p_{map} mellett a második tag nulla, ezért a szórás alulbecsült lesz. A PPC-ben ez mesterségesen szűk prediktív eloszlást ad, és a megfigyelt statisztikát indokolatlanul szélsőségesnek mutathatja. A javítás az, hogy `p`-t a teljes posteriorból sorsoljuk:

```
p_draws = rng.choice(grid, size=S, p=post)
y_rep = rng.binomial(1, p_draws[:, None], size=(S, n))
```

Az így kapott eloszlás már a posterior predictive-et közelíti.

4. A szkeptikus prior sűrűsége $\pi(p) \propto (1-p)^3$, ami $\text{Beta}(1, 4)$. A likelihood továbbra is $p^6(1-p)^9$. A posterior tehát arányos $p^6(1-p)^{12}$-nel, ami $\text{Beta}(7, 13)$. Várható értéke:

$$\mathbb{E}[p \mid y] = \frac{7}{7+13} = 0.35.$$

A 15 jövőbeli elemű posterior predictive sikerösszeg várható értéke $15 \cdot 0.35 = 5.25$. A prior predictive eloszlás flat priorral az egyenletes $k = 0, \ldots, 15$ eloszlás, várható értéke 7.5. A szkeptikus prior predictive Beta-binomiális$(15, 1, 4)$, várható értéke $15 \cdot 1/(1+4) = 3$. A prior tehát alapból kevés sikert tart plauzibilisnek; a megfigyelt $T_1 = 6$ ebben a keretben már nem a prediktív eloszlás alsó oldala, hanem a szkeptikus prior utáni posteriorhoz képest kissé magasabb, kevésbé szélsőséges érték. A kódban ezt például a `scipy.stats.betabinom.pmf` segítségével ellenőrizhetjük:

```
from scipy.stats import betabinom

k_vals = np.arange(n + 1)
prior_pred_flat = betabinom.pmf(k_vals, n, 1, 1)      # közel egyenletes
prior_pred_skept = betabinom.pmf(k_vals, n, 1, 4)
post_pred_skept = betabinom.pmf(k_vals, n, 7, 13)

print("prior pred mean flat:  ", float(np.sum(k_vals * prior_pred_flat)))
print("prior pred mean skept: ", float(np.sum(k_vals * prior_pred_skept)))
print("post  pred mean skept: ", float(np.sum(k_vals * post_pred_skept)))
```

A kézi kontroll szerint 7.5, 3.0 és 5.25 körüli értékeket várunk.

19.2.9 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az, hogy a credible intervalt automatikusan konfidenciaintervallumként értelmezzük. Ellenpélda: tegyük fel, hogy a prior degenerált, azaz teljes tömegét egyetlen $p = 0.5$ pontra teszi. Ekkor bármilyen adat után a 95%-os credible interval is a $[0.5, 0.5]$ pont. Ha a valódi $p = 0.3$, akkor ez az intervallum ráadásul soha nem tartalmazza a paramétert, tehát az ismételt mintavételes lefedettsége 0. A posterior állítás a modellen belül továbbra is „igaz”, de a prior hibás volt a külvilághoz képest. Ezért a credible interval nem állít frekventista lefedettséget, és a konfidenciaintervallum sem állít posterior valószínűséget.

A második hiba a PPC-p-érték mechanikus 0.05-ös küszöbölése. A p_B nem nullmodell szerinti p-érték, és ugyanaz az adat kétszeresen szerepel: először a posterior illesztésében, másodszor az ellenőrzésben. Ezért a p_B eloszlása akkor sem feltétlenül egyenletes, ha a modell igaz. A gyakorlatban a nagyon kicsi vagy nagyon nagy érték irányíthatja a figyelmet a modellhibára, de döntési küszöbként nem ugyanolyan státuszú, mint a 15.2. alfejezet kontrollált ismételt mintavételes eljárása.

19.2.10 Önellenőrzés

- Le tudom-e írni, hogy a 95%-os credible interval melyik feltétel mellett állít posterior valószínűséget 0.95-öt?
- A replikált adatomnak ugyanaz a megfigyelési egysége és sor-/oszlopszáma, mint az eredeti adatnak, tehát nem keverem a szemcsézettséget?
- A predictive eloszlásom a posterior teljes keveréke, nem pedig egyetlen pontbecslés körüli binomiális eloszlás?

- A kalibrációs állításomnál meg tudom nevezni, hogy a véletlent a prior-data közös folyamatból vettem, nem rögzített paraméterből?

19.2.11 Rövid összefoglaló

A credible interval a posterior kvantiliseiből készített tartomány: flat priorral a Beta$(7, 10)$ posterior 95%-os intervalluma körülbelül $[0.20, 0.64]$. A posterior predictive eloszlás a likelihood posterior szerinti keveréke; 15 új Bernoulli-megfigyelés sikerösszegének várható értéke $15 \cdot 7/17 \approx 6.18$. A prior predictive a prior ellenőrzésére való, flat priorral egyenletes sikerösszeget ad. A PPC replikált adatokat generál a posteriorból, és két statisztikát, itt a sikerösszeget és a leghosszabb futást veti össze. A p_B diagnosztikai érték, nem klasszikus p-érték. A credible és confidence interval értelmezése különbözik. Következő alkalmazásként a 19.3. alfejezetben ugyanezt PyMC és MCMC segítségével ismételjük meg.

19.3 PyMC modell és MCMC diagnosztika

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyMC, ArviZ**
>
> Előfeltétel: 19.2. alfejezet · Célkészség: PyMC modell és MCMC diagnosztika · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

19.3.1 Helyzet és cél

Folytatjuk a 19.2. alfejezet rácsos bayesi példáját. Egy mobilalkalmazás kísérleti csoportjában 16 véletlenül kiválasztott ügyfelet figyelünk meg; minden ügyfél egyetlen megfigyelési egység. A bináris kimenet $y_i \in \{0, 1\}$ azt jelöli, hogy az i-edik ügyfél 7 napon belül konvertált-e. A megfigyelt adatban 6 siker és 10 kudarc van.

Célunk a konverziós valószínűség p paraméterének posterior eloszlását MCMC-mintavétellel becsülni, majd **csak akkor** értelmezni, ha a konvergencia-diagnosztika rendben van. A kézi kontrollt már ismerjük: $p \sim \text{Beta}(1, 1)$ flat prior és 6/10 siker/kudarc esetén a konjugált posterior $p \mid y \sim \text{Beta}(7, 10)$. Ennek várható értéke $\frac{7}{17} \approx 0.412$, a 19.2. alfejezetben számított 95%-os credible interval körülbelül $[0.20, 0.64]$. A PyMC-futtatásnak ehhez kell közelítenie a Monte Carlo hiba mértékén belül.

Az eszköz itt bővítő: NumPy, SciPy mellett PyMC és ArviZ. Az alábbi kód CPU-n fut; GPU-ra vagy távoli gépre nincs szükség.

19.3.2 Miért lánc, és mit jelent a mintavétel?

A posterior eloszlásból közvetlenül gyakran nem tudunk független mintákat húzni. Az MCMC (Markov chain Monte Carlo) ehelyett egy **Markov-láncot** futtat: a következő p_{t+1} paraméterérték csak a jelenlegi p_t értéktől függ. Elég hosszú futtatás után a lánc állapotainak eloszlása a posteriorhoz közelít; a bemelegedés (tuning/warmup) után megtartott lépések pedig közelítő poszteriori mintaként használhatók.

A lépések nem függetlenek. A 15.3. alfejezet bootstrapjében a független újramintázási egységért aggódtunk; itt éppen azt mérjük, hogy a lánc előző lépése milyen erősen befolyásolja a következőt. Emiatt a nyers `draws` szám nem azonos az információtartalommal.

A PyMC folytonos paraméterű modelleknél a **NUTS** (No-U-Turn Sampler) nevű Hamilton-féle Monte Carlo módszert használja. A lánc egy lépése:

1. javasol egy új paraméterértéket a posterior geometriája alapján,
2. elfogadja vagy elutasítja azt,
3. az új állapotot rögzíti,
4. a következő iteráció ebből az állapotból indul.

Több **láncot** (chain) különböző kezdőértékből indítunk. Így a láncok közötti egyezés vagy eltérés maga is diagnosztikai jel. A kontrollpélda még egyszerű: a konjugált posterior miatt a láncok futtatása „önellenőrző" futás, mert az elméleti eloszlást külön is ismerjük.

19.3.3 A diagnosztikai fogalmak kis kézi kontrollal

Az ArviZ `InferenceData` objektumba gyűjti a láncok értékeit. A fő jellemzők:

- **draws**: a bemelegedő fázis után, lánconként megtartott minták száma. Ha `chains=4` és `draws=1000`, akkor összesen 4000 poszteriori mintánk van.
- **tune**: a bemelegedő lépések száma. Az itt keletkező mintát nem használjuk a posterior értelmezéséhez.
- **R-hat** (Gelman–Rubin statisztika): a láncok közötti és a láncon belüli szóródást hasonlítja össze. Az 1,01 fölötti $\hat{R}$ diagnosztikai figyelmeztetés; a küszöb nem bizonyítja önmagában a konvergenciát vagy annak hiányát. A diagnosztikát ESS-szel, divergenciákkal és trace-plotokkal együtt kell értelmezni.
- **ESS** (effective sample size, effektív mintaméret): a lánc mintáinak autokorrelációját korrigáló szám. A `ess_bulk` a poszterior középső részére, az `ess_tail` a farokra jellemző. A cél az, hogy legalább néhány száz legyen; a példánkban több ezer várható.
- **Mcse**: Monte Carlo standard error; azt mondja meg, a finite lánc miatt mekkora a becsült posterior-átlag numerikus bizonytalansága.
- **divergences**: a NUTS futása közben a lépés geometriája elfajulhat. A divergens lépések darabszáma legyen nulla, mert hullámzó lefolyás esetén a lánc könnyen félrevezető területre sodródhat.

Kis kézi ellenőrzés a fogalomra. Képzeljünk el két láncot, mindegyik 2000 poszteriori mintával. Az A lánc minden értéke 0.42 körül, a B lánc minden értéke 0.66 körül mozog. A láncokon belül kicsi a szórás, a láncok között nagy. A közös posterior-átlag 0.54 körül lenne, de ez **nem** a kívánt $p \approx 0.41$ poszterior összegzése, csak két megszorult lánc átlaga. Az R-hat ilyenkor jóval 1.01 fölött lenne: a sok draw ellenére rossz a keveredés. A konvergencia tehát nem a mintaszámból, hanem a láncok közötti kiegyenlítettségből és az autokorrelációból olvasható le.

19.3.3.1 MCMC-diagnosztikai sablon

Egyetlen szám nem elegendő az elfogadáshoz; a következő táblázatot kell kitölteni, és a döntést csak az összes sor együttes értékelése után hozni.

Diagnosztika	Mit nézünk?	Elfogadási jel	Meghibásodás jele	Teendő
Trace, lánconként	a láncok átfedése, stacionaritás	a láncok vizuálisan összekeverednek	külön sávok, trend, lassú sodródás	több tuning/draw, jobb inicializálás

Diagnosztika	Mit nézünk?	Elfogadási jel	Meghibásodás jele	Teendő
Divergences	NUTS numerikus hibák	0 divergens lépés	néhány százalék is gyanús	`target_accept` növelése, újraparaméterezés
R-hat	láncok közötti és belüli szórás	$\hat{R} < 1{,}01$	$\hat{R} \geq 1{,}01$	több lánc/draw, geometria javítása
ESS bulk	a posterior középső részének effektív mintája	néhány száz felett	túl alacsony, nagy autokorreláció	több draw, újraparaméterezés
ESS tail	a farok effektív mintája	néhány száz felett	a bulk jó, a tail alacsony	farok geometriájának javítása
MCSE	a posterior-átlag numerikus hibája	kicsi a posterior szd-hez képest	nagy a bizonytalanság a becslésben	több draw
Geometria/azonosíthatóság	párok, nyereg, több módus	egyetlen, jól azonosított módus	tölcsér, több módus, lapos gerinc	újraparaméterezés, prior/identifikáció

Újraparaméterezési ötletek. Hierarchikus modellben a központosított $\theta_i \sim N(\mu, \sigma)$ helyett gyakran jobb a nem központosított alak: $\theta_i = \mu + \sigma z_i$, $z_i \sim N(0, 1)$. Pozitív paraméternél log-skálára váltás, erősen korreláló prediktoroknál QR-felbontás vagy újraközpontosítás, kategorikus hatásoknál sum-to-zero megszorítás segíthet. A tölcsér alakú posterior tipikus jele a divergenciáknak és az alacsony tail ESS-nek; ilyenkor a nem központosított paraméterezés a fő eszköz.

Az azonosíthatóság külön kérdés. Lehet jó R-hat, magas ESS és nulla divergencia egy olyan modellben, amelyben a paraméter nem azonosítható (például több, egymást kiegyenlítő együttható); a láncok ilyenkor ugyanabban a lapos vagy görbe gerinc mentén keverednek. A diagnosztika a mintavétel hibáit jelzi, nem azt, hogy a modell kérdésre adott válasza egyértelmű. Ezért kell a diagnosztikát a modell tartalmi kritikájával (prior predictive, PPC, érzékenység) együtt értelmezni.

19.3.4 Teljes minimális munkapélda

Először opcionális PyMC/ArviZ környezetet hozunk létre. A pontos, elérhető verziót a saját futtatási napló rögzíti; a célterjedelem a PyMC 5.x család. Az alábbi parancscsomag **szemléltető**, nem ellenőrzött lock.

```
# macOS/Linux; munkakönyvtár: lab/
python -m venv .venv-s075
source .venv-s075/bin/activate
python -m pip install "pymc>=5,<6" "arviz>=0.19"
```

Windows PowerShellben ugyanez a környezet:

```
# PowerShell; munkakönyvtár: lab\
python -m venv .venv-s075
.\.venv-s075\Scripts\Activate.ps1
python -m pip install "pymc>=5,<6" "arviz>=0.19"
```

A munkapélda a p paramétert és a megfigyelt Bernoulli-adatokat definiálja. A `with pm.Model()` blokk a PyMC modellkörnyezete: benne a véletlen változók függőségi gráfja épül fel.

```
# minimal_mcmc.py
import numpy as np
import pymc as pm
import arviz as az

# 16 ügyfél, 6 siker, 10 kudarc
y = np.array([1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])

with pm.Model() as model:
    p = pm.Beta("p", alpha=1.0, beta=1.0)      # flat prior a [0,1] intervallumon
    pm.Bernoulli("y_obs", p=p, observed=y)     # likelihood: yi ~ Bernoulli(p)

    idata = pm.sample(
        draws=1000,        # lánconkénti poszteriori mintaszám
        tune=1000,         # bemelegedő fázis lánconként
        chains=4,          # független Markov-láncok száma
        cores=1,           # CPU-n futtatva egyértelműbb naplózni
        random_seed=20261020,
        return_inferencedata=True,
    )

# Diagnosztika és poszteriori összegzés
print(az.summary(idata, var_names=["p"], hdi_prob=0.95))
print("R-hat:", az.rhat(idata, var_names=["p"]).to_array().item())
print("ESS bulk:", az.ess(idata, var_names=["p"], method="bulk").to_array().item())
div = idata.sample_stats["diverging"].sum().item()
print("Divergences:", int(div))
```

A `p` változó shape-je a posteriorban `(4, 1000)`:

- 0. tengely: lánc azonosítója, azaz 4 független futtatás
- 1. tengely: láncon belüli sorszám, azaz 1000 megtartott lépés

A `pm.Bernoulli("y_obs", ...)` likelihood az egyes megfigyeléseket egy Bernoulli- véletlen változón keresztül köti a `p` paraméterhez. A PyMC a modell log-posteriorját építi fel ebből a gráfból, és a NUTS ezen a felületen javasol lépéseket.

Az elméletileg várt, **szemléltető** kimenet:

```
# posterior summary (szemléltető, nem helyi mért output)
        mean     sd  hdi_3%  hdi_97%  mcse_mean  mcse_sd  ess_bulk  ess_tail  r_hat
p      0.412  0.116   0.199    0.635      0.002    0.001      2200      2100    1.001
R-hat: 1.001
ESS bulk: 2200
Divergences: 0
```

A számok az elméleti Beta(7,10) poszterior környékén mozognak. A pontos eredmények futásonként némileg eltérhetnek, mert a MLMC nem bitazonos az eszközök között; az ellenőrzési küszöbök viszont stabilak.

19.3.5 Az eredmények kézi értelmezése

A posterior-összegzés a poszterior közelítő átlagát és szórását mutatja. A `0.412` közel van a konjugált várható értékhez: $\frac{7}{17} \approx 0.412$. A láncok keveredése megfelelő, mert

- R-hat $\approx 1.001 \ll 1.01$,

- a láncok külön-külön is ugyanazt a posterior módusz körüli tartományt fedték le,
- divergens lépés nincs,
- az ESS bulk 2200 körüli, tehát a négyzetes MC hiba az átlagban hasonló ahhoz, mintha, óvatosan fogalmazva, körülbelül 2200 kevésbé autokorrelált mintánk lenne. A pontos független ekvivalencia nem állítható; az ESS mérése maga is statisztikai becslés. Ez a tartomány elég a napi tantételhez.

A 95%-os HDI a `[0.199, 0.635]` körüli intervallum, ami a 19.2. alfejezet kézi kontrolljával összhangban van. Az eltérés az MC-véletlen miatt elfogadható. Ha ehelyett a posterior-átlag például 0.46 vagy 0.36 lenne, és az R-hat 1.05 fölé menne, akkor a konvergenciát nem vehetnénk készpénznek; a modell építését kellene felülvizsgálni.

A „divergences” értelmezése: a divergens lépés nem maga a HDI-intervallum divergenciája, hanem a HMC-lépés numerikus hibájának jelzése. A nulla jó jel. Ha néhány százalék divergens, a posterior farokbecslései, például a felső 97.5%-os kvantilis, torzultak lehetnek, még ha az átlag stabilnak is tűnik.

A modellhez most nincs szükség a 18.3. alfejezet megfigyeléses elemzésében megismert confounding-kérdésre, mert ez a 16 ügyfél egyetlen randomizált kísérleti csoportjának tekinthető. Az MCMC önmagában nem igazítja meg a nem mért zavaró változóból eredő torzítást: a 18.3. alfejezet érzékenységi logikája a bayesi modellnél is érvényes. A lánc lépései pedig nem hasonlítanak a 15.3. alfejezet ügyfélszintű bootstrapjére: ott ügyfélrekordokat újramintázunk, itt paraméterértékeket mozgatunk a láncon. Az időben rendezett adathoz pedig a 7.3. alfejezet ablak-gondolkodása hasonlít: a lánc állapota az előző állapotból indul, ezért az előzményfüggőséget mérni kell, nem egyszerűen a sorok számát számolni.

19.3.6 Feladatok

19.3.6.1 1. feladat — Értelmezés

Egy munkatárs a következő PyMC kimenetet adja át egy másik konverziós adathoz:

```
mean=0.414, sd=0.116, hdi=(0.199,0.635), ess_bulk=1200, r_hat=1.002, divergences=0
```

Írj mellé két–három mondatos értelmezést, amelyben

- megadod, mit mond a posterior a konverzióról,
- indoklod, hogy a lánc miért alkalmas a következtetésre,
- megnevezed, milyen korlátot mégis figyelembe kell venni.

19.3.6.2 2. feladat — Implementálás

Írj egy `sample_conversion(successes, failures, prior_a=1, prior_b=1)` függvényt, amely a fenti mintához hasonlóan Bernoulli-modellt illeszt. A függvény a `successes` és `failures` egészekből állítsa elő az `y` tömböt, futtasson 2 láncot `draws=500, tune=500` beállítással, és adja vissza az `idata` objektumot.

Ellenőrzés: 3 siker és 4 kudarc esetén, flat prior mellett a konjugált posterior $p \sim \mathrm{Beta}(4, 5)$, várható értéke $\frac{4}{9} \approx 0.444$. Futtasd a függvényt, és hasonlítsd össze az `az.summary(idata)` átlagát az elméleti 0.444-gyel. Adj meg egy toleranciát is, és indokold, miért nem várhatunk hajszálpontos egyezést.

19.3.6.3 3. feladat — Hibadiagnózis

Egy másik munkatárs így futtatja a modellt:

```
pm.sample(draws=100, tune=0, chains=2, cores=1)
```

Az `az.summary(idata)` kimenetében `r_hat=1.6`, `ess_bulk=8`. A posterior-átlag 0.31, pedig az adat szerint a nyers sikerarány 0.42. Sorold fel a hibalehetőségeket, és adj két konkrét javítást a munkatársnak. Melyik diagnosztikai értékből látja azonnal, hogy a posterior-átlagot nem szabad a 0.31-gyel összefoglalni?

19.3.6.4 4. feladat — Megváltozott feltétel

Az üzleti csapat információja szerint a korábbi kísérletben a konverzió már 0.3 körül volt, ezért most a flat prior helyett gyengén informatív Beta$(2, 5)$ priort használunk, ahol a prior várható értéke $\frac{2}{7} \approx 0.286$. Új adatként 14 siker és 3 kudarc érkezik. Mi a pontos konjugált posterior alakja, várható értéke és a prior–posterior elmozdulás iránya? Ha a PyMC futásban `r_hat=1.000`, `ess_bulk=2500`, `divergences=0`, elegendő-e a lánc a következtetéshez? Indokold a küszöbértékekkel.

19.3.7 Megoldások és indoklás

19.3.7.1 1. feladat megoldása

A posterior-átlag szerint a konverziós valószínűség körülbelül 0.41, a 95%-os HDI pedig 0.20 és 0.64 közé esik, tehát a becslés jelentős bizonytalanságot hordoz akkor is, ha a lánc jól keveredik. A lánc diagnosztikája jó, mert R-hat ≈ 1.002 messze az 1.01-es határ alatt van, divergens lépés nincs, és az ESS bulk 1200 számottevő. Korlát: a modell egy egyszerű Bernoulli-likelihoodot használ, tehát az eredmény nem feltétlenül ügyfélszegmensenként is igaz, és a posterior csak a választott prior és likelihood mellett érvényes.

19.3.7.2 2. feladat megoldása

```
import numpy as np
import pymc as pm

def sample_conversion(successes, failures, prior_a=1, prior_b=1):
    y = np.array([1] * successes + [0] * failures, dtype=int)
    with pm.Model() as model:
        p = pm.Beta("p", alpha=prior_a, beta=prior_b)
        pm.Bernoulli("y_obs", p=p, observed=y)
        idata = pm.sample(
            draws=500,
            tune=500,
            chains=2,
            cores=1,
            random_seed=42,
            return_inferencedata=True,
        )
    return idata

idata = sample_conversion(3, 4)
print(az.summary(idata, var_names=["p"]))
```

A konjugált várható érték 0.444. A megtartott minták 2 lánc × 500 minta, tehát az MC hiba nagyságrendje a poszterior szórásnak körülbelül az ESS négyzetgyökével osztva adódik. Ha ESS bulk tipikusan néhány száz, a 0.01–0.02 körüli átlagbeli eltérés elfogadható. Ezért bemondhatunk egy ±0.02 abszolút toleranciát **a posteriior-átlag** ellenőrzésére, és egy 0.01-nél kisebb R-hat küszöböt. Ha a becsült átlag 0.41 vagy 0.46, az MCMC-véletlen; de ha 0.30 és R-hat magas, akkor a lánc nem konvergált.

Pontosan: ellenőrzéskor a lánconkénti átlagoknak is közel kell esniük egymáshoz; az ArviZ `idata.posterior.mean(dim="draw")` ábrája mutathatja a kiegyensúlyozottságot.

19.3.7.3 3. feladat megoldása

A hiba négy forrásra vezethető vissza:

1. `tune=0`: a lánc kezdőállapotai nem estek a posterior tömegébe, így az elfogadási fázis torzult maradt.
2. `draws=100, chains=2`: nagyon kevés mintavétel, különösen autokorrelált láncban.
3. A `r_hat=1.6` egyenesen jelzi, hogy a két lánc különböző tartományokat talált meg; a közös átlag értelmetlen, mert a láncok közötti variancia nem tűnt el a futam alatt.
4. Az `ess_bulk=8` pedig azt mutatja, hogy 200 nyers mintából csupán körülbelül 8 effektív lépés maradt; ez a 0.31 magyarázata is lehet, ha az egyik lánc megszorult egy dudorban.

Konkrét javítások:

- állítsa be `tune=1000`, `draws=2000`, `chains=4` értékeket;
- adjon meg `target_accept=0.9` értéket, hogy a NUTS óvatosabb lépésekkel dolgozzon;
- ezután ismét ellenőrizze az R-hat, ESS és divergenc metrikákat.

Ha a kis mintás paraméterezés valóban nehezen keveredik, a modell újraparaméterezése vagy a prior szűkítése is segíthet.

19.3.7.4 4. feladat megoldása

A konjugált prior–likelihood frissítés:

- prior: $\mathrm{Beta}(2, 5)$
- megfigyelés: 14 siker, 3 kudarc
- posterior: $p \mid y \sim \mathrm{Beta}(2 + 14,\ 5 + 3) = \mathrm{Beta}(16, 8)$

A posterior várható értéke:

$$E[p \mid y] = \frac{16}{24} = \frac{2}{3} \approx 0.667.$$

A posterior módusza $\frac{16-1}{24-2} = \frac{15}{22} \approx 0.68$, ami eligazodási pont az egycsúcsú posteriorhoz. A prior 0.286-ról a posterior 0.667-re mozdul el, mert a megfigyelt 14/17 sikerarány jelentősen meghaladja a korábbi előzetes várakozást.

A diagnosztika elegendő: `r_hat=1.000` és `divergences=0` azt jelzi, hogy a láncok keveredtek, `ess_bulk=2500` pedig a poszterior középső tartományában komoly effektív mintát jelent. A gyengén informatív prior nem akadályozta a lánc mozgását; a posterior mégis a 0.67 köré koncentrál, mert a likelihood messze tolta a frissítést.

19.3.8 Gyakori tévedés és ellenpélda

Sokan azt hiszik, hogy a nagy `draws` érték automatikusan elegendő. Ellenpélda: futtassunk két láncot, egyenként 50 000 poszteriori mintával. Az A lánc minden lépésben 0.42 körül marad, a B lánc 0.66 körül. A raktárban a `posterior.mean()` például 0.54 lenne, és az `ess_bulk` sem feltétlenül lenne nulla. Az R-hat azonban magas: hiába 100 000 draw, a láncok közötti variancia nem csökkent, mert a Markov-lánc nem vándorolt a teljes poszterioron. A jó diagnosztika ezért nem a minta számát, hanem a láncok közötti egyezést és az autokorrelációt ellenőrzi.

19.3.9 Önellenőrzés

1. Mi a különbség a posterior valódi eloszlása és az MCMC-közelítés között?
2. Milyen küszöb alatt fogadunk el egy R-hat értéket a napi munkában?
3. Miért nem egyenlő az ESS a nyers minta méretével?
4. Hány független lánccsal dolgozik a minimális példa, és ez miért jobb, mint egyetlen lánc 4000 mintával?
5. Ha 20 divergens lépés jelenik meg 4000-ből, mi a helyes reakció, mielőtt a posterior felső kvantilisét jelentenénk?

A válaszok a szövegben megtalálhatók: a posterior a cél, a láncok közelítik; R-hat jellemzően 1.01 alatt; az ESS az autokorreláció miatt csökken; 4 lánccal diagnosztizálható a nem keveredés; divergens lépés esetén a farok mennyiségeit nem szabad lejelenteni, hanem a mintavételezőt kell javítani.

MCMC-diagnosztika: pusztán a mintaszám és a láncok lefutása nem bizonyít konvergenciát. Ellenőrizendő a rank-normalized R-hat, bulk/tail effektív mintanagyság, divergenciák, energiamutatók és a trace plot; hibás diagnosztikával posterior intervallum nem tekinthető megbízhatónak.

19.3.10 Rövid összefoglaló

A PyMC-modellben a `p` paramétert priorral és Bernoulli-likelihooddal kötjük a megfigyelt adathoz, majd a NUTS-láncok a poszterior közelítő mintáját adják. A 4 lánc és a bemelegedő fázis után az ArviZ `r_hat`, `ess_bulk` és divergenciaszámlálója ellenőrzi a keveredést. Az R-hat 1.01 alatt, a divergenciaszám nulla, az effektív minta legalább több száz legyen. Sok nyers draw nem pótolja a jó keveredést. A 19.3. alfejezet következő alkalmazása a 19.4. alfejezetben hierarchikus modellek és részleges pooling, ahol ugyanezen diagnosztikát csoportonkénti paraméterekre kell kiterjeszteni.

19.4 Hierarchia és részleges pooling

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyMC, ArviZ**
>
> Előfeltétel: 19.3. alfejezet · Célkészség: Hierarchia és részleges pooling · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

19.4.1 Cél és konkrét helyzet

A Northstar szintetikus rendelési adatainál a régiós késésarány fontos működési kérdés: ha egy régióban túl magas a 7 napon túli kézbesítések aránya, kapacitást kell átcsoportosítani. Tegyük fel, hogy három régióra van rendelésenkénti bináris megfigyelésünk, ahol az eredmény `late=1`, ha a kézbesítés késő volt, különben `late=0`. A csoportok mérete szándékosan nagyon eltérő:

- `north`: 200 rendelés, 55 késő,
- `central`: 40 rendelés, 9 késő,
- `south`: 8 rendelés, 0 késő.

A megfigyelési egység egy rendelés. A kimenet a késés valószínűségének régiónkénti becslése. A `south` csoportban az egyszerű arány `0 / 8 = 0`, de 8 rendelésből még akkor is könnyen lehet nulla késés, ha a valódi kockázat nem nulla. Az induló kérdés: milyen becslést adnál a `south` régióra, ha a `north` és `central` adatok azt sugallják, hogy a késés általában 20–30% körül van? Ebben az alfejezetben ezt a kérdést formalizáljuk.

19.4.2 Fogalmi magyarázat

19.4.2.1 Hierarchikus modell (hierarchical model)

A hierarchikus modell szerint a csoportonkénti paraméterek nem függetlenek egymástól, hanem egy közös eloszlásból származnak. Bináris kimenetre a természetes alapszint:

$$y_{ij} = 1 \quad \text{akkor és csak akkor, ha a } j. \text{ csoport } i. \text{ rendelése késő.}$$

A csoportszintű valószínűség p_j, amelyre:

$$y_{ij} \mid p_j \sim \text{Bernoulli}(p_j), \qquad p_j \sim \text{Beta}(\alpha, \beta).$$

Itt a j index a csoportot jelöli, i a csoporton belüli megfigyelést, p_j a j. csoport valódi, de ismeretlen késési valószínűsége, $\alpha > 0$ és $\beta > 0$ a közös prior hiperparaméterei. Minden p_j skalár a $(0, 1)$ intervallumon; a megfigyelések adott p_j mellett feltételesen függetlenek. A modell neve is ebből a két szintből fakad: a megfigyelés szintje a likelihood, a csoportparaméterek szintje a prior. A teljes Bayes-i hierarchikus modellben α és β sem rögzített, hanem saját gyenge priorral becsült; a bemutató példában először rögzített priorral dolgozunk, hogy a pooling hatása kézzel ellenőrizhető legyen.

A béta prior kényelmes, mert a béta eloszlás a binomiális likelihood konjugált priorja (conjugate prior). Ha egy csoportban s_j késő és n_j összes rendelés van, akkor a posterior szintén béta:

$$p_j \mid s_j, n_j, \alpha, \beta \sim \text{Beta}(s_j + \alpha,\ n_j - s_j + \beta).$$

A posterior várható érték tehát zárt alakban:

$$\hat{p}_j^{post} = \frac{s_j + \alpha}{n_j + \alpha + \beta}.$$

Itt minden mennyiség alakja skalár csoportonként. A `north` példán $s_{north} = 55$, $n_{north} = 200$, $\alpha = 5$, $\beta = 15$ esetén $\hat{p}_{north}^{post} = 60/220 \approx 0{,}273$.

19.4.2.2 Shrinkage (zsugorodás)

A shrinkage azt jelenti, hogy a csoportonkénti becslés a közös prior várható értéke felé húz, és a húzás erőssége a csoportmérettől függ. Vezessük be a prior effektív mintanagyságát:

$$\kappa = \alpha + \beta, \qquad \mu_0 = \frac{\alpha}{\alpha + \beta}.$$

Ekkor a posterior várható érték átírható súlyozott átlagként:

$$\hat{p}_j^{post} = \frac{n_j}{n_j + \kappa} \cdot \frac{s_j}{n_j} + \frac{\kappa}{n_j + \kappa} \cdot \mu_0.$$

A képletben $\frac{s_j}{n_j}$ az adatból számolt nyers csoportarány, μ_0 a prior középpont, a súlyok pedig a csoportméret és az effektív prior-mintanagyság arányát tükrözik. Ha n_j nagy a κ-hoz képest, az adat dominál; ha n_j kicsi, a prior dominál. A `south` csoportban $n_{south} = 8$, $\kappa = 20$, ezért a shrinkage súly $\kappa/(n_{south} + \kappa) = 20/28 \approx 0{,}714$: a posterior főleg a prior $\mu_0 = 0{,}25$ értékét követi, és az adatbeli 0 arányt csak kismértékben engedi érvényesülni. A `north` csoportban a shrinkage súly csak $20/220 \approx 0{,}091$.

A shrinkage tehát a túlzott külön becslést korrigálja, de nem mindig a globális mintából számolt átlag felé húz: a célpont a prior μ_0, amelyet a modellező választ vagy a hierarchikus modell becsli.

19.4.2.3 Group effect (csoporthatás)

A csoporthatás a csoportok közötti, adatból becsült valódi különbség. A hierarchikus modell nem tünteti el a csoportkülönbségeket: ha a `north` és `central` csoportoknak sok adata van, a posteriorjaik el tudnak válni egymástól. A shrinkage csak a bizonytalan, kis csoportos becsléseket húzza erősebben. A `north` posterior átlag 0,273 a 0,275 nyers arányról, a `central` 0,233 a 0,225 nyers arányról, tehát a nagyobb csoportok szinte megőrzik saját jelüket. A `south` posterior 0,179 a 0 nyers arányról, mert ott a jel még gyenge.

19.4.2.4 Identifiability (azonosíthatóság)

Az azonosíthatóság azt kérdezi, hogy a megfigyelhető adatokból elvileg elkülöníthető-e két paraméter. A hierarchikus modell esetén a csoporthatás varianciája és a mintavételi zaj különösen kis csoportok vagy kevés csoport esetén rosszul azonosítható. Példánkban összesen három csoportunk van, és az egyikben alig van adat; ha a α, β hiperparamétereket is becsülni akarnánk, a posteriorjuk széles lenne. A gyakorlatban ezért kevés csoportnál a hiperprior becslés bizonytalan, és a shrinkage erőssége érzékenyebb a priorválasztásra. A csoporthatás sem lesz pontosan azonosítható a kis `south` mintából; éppen ezért húz a modell a közös eloszlás felé.

19.4.2.5 Compute budget (számítási költségvetés)

A konjugált béta-binom modell posteriorja egyetlen képlettel kiszámolható, tehát a bemutatott rögzített prioros változat CPU-n másodperc töredéke alatt fut. A teljes hierarchikus modellben, ahol α és β is becsült, a posterior általában nem zárt alakú; ilyenkor PyMC és MCMC vagy numerikus integrálás szükséges. A 19.3. alfejezetben tanult diagnosztikát – R-hat, ESS, divergences – itt is alkalmazni kell. A számítási költség lineárisan nő a csoportok számával és az MCMC-láncok hosszával, ezért érdemes először a konjugált vagy empirikus Bayes közelítéssel ellenőrizni a modell viselkedését, és csak azután futtatni a drágább szimulációt.

19.4.3 Teljes minimális munkapélda

Az alábbi kód a rögzített bemenettel dolgozik, és mindhárom becslést kiszámolja. Az adatokat a fenti táblázatból vettük, nem generáljuk újra; így a point estimate-ek kézzel reprodukálhatók. A priorválasztás: korábbi időszaki jelentésekből a késés átlagos arányát $\mu_0 = 0{,}25$-re, a prior erősségét pedig $\kappa = 20$ effektív rendelésnyire állítjuk, azaz Beta$(5, 15)$.

```
import numpy as np
from scipy.stats import beta as beta_dist

# Csoportonként: n = rendelések száma, s = késő rendelések száma
groups = {
    "north":  {"n": 200, "s": 55},
    "central": {"n": 40,  "s": 9},
    "south":   {"n": 8,   "s": 0},
}

# 1) Teljes pooling: egyetlen közös arány, súlyozva a csoportméretekkel
n_total = sum(g["n"] for g in groups.values())
s_total = sum(g["s"] for g in groups.values())
pooled = s_total / n_total
print(f"Teljes pooling: {pooled:.3f}")

# 2) Külön becslés: csoportonkénti nyers arány
separate = {name: g["s"] / g["n"] for name, g in groups.items()}
print("Külön becslés:", {k: round(v, 3) for k, v in separate.items()})

# 3) Részleges pooling: rögzített Beta(5, 15) prior
alpha0, beta0 = 5, 15
post_mean = {}
post_ci = {}
for name, g in groups.items():
    a = g["s"] + alpha0
    b = g["n"] - g["s"] + beta0
    post_mean[name] = a / (a + b)
    post_ci[name] = (
        beta_dist.ppf(0.05, a, b),
        beta_dist.ppf(0.95, a, b),
    )
    print(
        f"{name}: posterior mean = {post_mean[name]:.3f}, "
        f"90% credible interval = [{post_ci[name][0]:.3f}, {post_ci[name][1]:.3f}]"
    )
```

Az alábbi kimenet a rögzített bemenetből közvetlenül következik; a point estimate-ek kézzel is ellenőrizhetők, a credible interval határait pedig a `scipy.stats.beta.ppf` hívással kapjuk meg a futáskor.

```
Teljes pooling: 0.258
Külön becslés: {'north': 0.275, 'central': 0.225, 'south': 0.0}
north: posterior mean = 0.273, 90% credible interval = [0.221, 0.329]
central: posterior mean = 0.233, 90% credible interval = [0.142, 0.338]
south: posterior mean = 0.179, 90% credible interval = [0.067, 0.333]
```

A `south` intervallum hatsávja a futó kódban pontosnak vehető; a fenti három CI érték tájékoztató, a `scipy` verziójától függetlenül szűk toleráncián belül ugyanaz lesz.

19.4.3.1 Numerikus összehasonlítás: fully pooled, no pooling, partial pooling

A zsugorodás mértéke a csoportmérettől függ. Vegyünk három mesterséges csoportot: 0/1, 2/3 és 50/100 megfigyelést, ugyanazzal a Beta(5, 15) priorral (priorátlag 0,25, priorerő 20). A teljes pooling egyetlen aránya $52/104 = 0{,}5$.

Csoport	n	s	Nyers arány	Részleges posterior átlag	Zsugorítási súly $n/(n+20)$
A	1	0	0,000	**0,238**	0,048
B	3	2	0,667	**0,304**	0,130
C	100	50	0,500	**0,458**	0,833

A posterior átlag képlete $w \cdot \hat{p}_{\text{raw}} + (1-w) \cdot 0{,}25$, ahol $w = n/(n+20)$. Az A csoport nyers aránya 0, de a posterior átlag 0,238, mert a mindössze egy megfigyelés alig mozdítja el a priort; a B csoport 0,667-es nyers arányát erősen lehúzza 0,304-re; a C csoport 0,5-es aránya közel marad a nyers értékhez (0,458), mert a nagy minta dominál. Ugyanaz a prior tehát kicsi n-nél erős, nagy n-nél gyenge hatású.

Ez a számítás rögzített priorral dolgozik; a teljes hierarchikus modell a prior paramétereit is a csoportokból becsüli (empirical Bayes), ezért a zsugorodás mértéke adatvezérelt. A tanulság ugyanaz: a kis csoport becslése erősebben zsugorodik, de a zsugorodás nem jelenti, hogy a csoporthatás eltűnik; a posterior bizonytalansága a kis csoportnál nagyobb marad.

19.4.4 Eredmény értelmezése

A teljes pooling minden régióra ugyanazt a 0,258-as arányt adná, ami a `south` esetében valószínűtlenül magas, hiszen ott egyetlen késést sem láttunk. A külön becslés `south`-ra 0,0, ami a nyolc megfigyelésből fakadó túlzott magabiztosság: a modell azt állítaná, hogy a `south` régióban lehetetlen a késés. A részleges pooling `south`-ra 0,179-et ad, ami a kettő közötti kompromisszum. A súlyozott átlag képletével ellenőrizhető:

$$\hat{p}_{south}^{post} = \frac{8}{8+20} \cdot 0 + \frac{20}{8+20} \cdot 0{,}25 = \frac{5}{28} \approx 0{,}179.$$

A `north` posterior 0,273, ami a nyers 0,275-től alig tér el, mert a 200 rendelés nagy súlyt ad az adatnak. A `central` posterior 0,233, a nyers 0,225-től kissé a prior 0,25 felé húz. A credible intervallumok is tükrözik a csoportméreteket: a `south` intervallum sokkal szélesebb, mint a `north`-é. A modellfeltétel: adott p_j mellett a rendelések függetlenek, a `late` értékek binárisak, és a prior jól reprezentálja a korábbi tudást. Ha a régiók között erős rejtett különbség van, a közös prior ezt részben elmossa.

Részleges pooling: a hierarchikus modell becslése csoportmérettől és a közös hipermodellezési feltevésektől függ. A kis csoportokat jobban zsugoríthatja, de ez nem garantál automatikus javulást, ha a csoportok nem tekinthetők ésszerűen cserélhetőnek.

19.4.5 Feladatok

1. **Értelmezés.** Magyarázd el a shrinkage súlyozott átlag formulájával, miért ad a részleges pooling `south`-ra 0,179-et, miközben a teljes pooling 0,258-at adna.
2. **Megvalósítás.** Számítsd ki a `south` posterior 90%-os equal-tailed credible intervallját a Beta$(5, 15)$ prior mellett. Add meg a posterior paramétereket, majd írd le a `scipy.stats.beta.ppf` hívást, és értelmezd, miért szélesebb ez az intervallum, mint a `north`-é.
3. **Hibakeresés.** Valaki a globális átlagot a három csoportarány egyszerű átlagaként számolta ki: $(0{,}275 + 0{,}225 + 0{,}0)/3 \approx 0{,}167$, és azt állította, hogy ez a teljes pooling becslés. Mi a hiba? Mi a helyes teljes pooling becslés, és miért?
4. **Megváltozott feltétel.** Tegyük fel, hogy a `south` régióban később 80 rendelést gyűjtünk össze, amelyből 8 késő. Számítsd ki a külön becslést és a részleges pooling posterior várható értékét ugyanazzal a Beta$(5, 15)$ priorral. Hogyan változik a shrinkage súlya, és miért?
5. **Opcionális PyMC.** Írj PyMC modellt, amelyben mindhárom csoport p_j paramétere Beta(α, β) priorból származik, és α, β is gyenge priorral becsült. Mely ArviZ-diagnosztikákat ellenőriznéd, és miért?

19.4.6 Megoldások és indoklás

1. A posterior várható érték a fenti súlyozott átlag: $\hat{p}_j^{post} = \frac{n_j}{n_j+\kappa}\hat{p}_j + \frac{\kappa}{n_j+\kappa}\mu_0$. `south` esetén $n_j = 8$, $\kappa = 20$, $\hat{p}_j = 0$, $\mu_0 = 0{,}25$, tehát $\hat{p}^{post} = 0 \cdot (8/28) + 0{,}25 \cdot (20/28) = 5/28 \approx 0{,}179$. A teljes pooling nem használ priorzsugorítást, hanem minden rendelést egyetlen közös adathalmaznak tekint: $64/248 \approx 0{,}258$. A shrinkage a `south` nullás arányt a prior középpontja felé emeli, de nem egészen a globális 0,258-ig.

2. A posterior paraméterek $a = s_{south} + \alpha = 0 + 5 = 5$ és $b = n_{south} - s_{south} + \beta = 8 - 0 + 15 = 23$. A kód:

```
from scipy.stats import beta as beta_dist
lower, upper = beta_dist.ppf(0.05, 5, 23), beta_dist.ppf(0.95, 5, 23)
print(lower, upper)
```

Az intervallum a Beta$(5, 23)$ poszterior eloszlás 5% és 95% kvantilisei közötti tartomány. Mivel a `south` mindössze 8 rendelésből áll, a poszterior nagy szórással rendelkezik, míg a `north` posterior $a = 60, b = 160$ paraméterű, tehát sokkal koncentráltabb. A konkrét határokat a kód adja; a normal approximáció alapján a `south` intervallum nagyjából $[0{,}06, 0{,}30]$, de a pontos béta-kvantilisek ettől kissé eltérnek a ferdeség miatt.

3. A hiba az, hogy a csoportok mérete nem egyenlő, ezért a csoportarányok egyszerű átlaga nem becsüli a közös arányt. A helyes teljes pooling becslés súlyozott átlag: az összes késő osztva az összes rendeléssel, azaz $64/248 \approx 0{,}258$. Az egyszerű átlagban a 8 elemű `south` ugyanolyan súlyt kap, mint a 200 elemű `north`, ezért a 0 arány túlzottan lehúzza az eredményt. Ez ugyanaz a csapda, mint amikor a 7.2. alfejezetben a tételszintű join után nem a megfelelő szemcsézettséggel aggregálunk.

4. A külön becslés $8/80 = 0{,}1$. A posterior várható érték $(8 + 5)/(80 + 5 + 15) = 13/100 = 0{,}13$. A shrinkage súly $\kappa/(n_{south} + \kappa) = 20/100 = 0{,}2$, szemben a korábbi 0,714-gyel: a 80 rendelés

már sokkal nagyobb adatsúllyal bír, ezért a becslés jobban követi a nyers arányt, és kevésbé húz a prior 0,25 felé. Az önellenőrzés: a súlyozott átlag $0{,}8 \cdot 0{,}1 + 0{,}2 \cdot 0{,}25 = 0{,}13$.

5. Egy lehetséges PyMC modellvázlat:

```
import pymc as pm
import numpy as np

s = np.array([55, 9, 0])
n = np.array([200, 40, 8])

with pm.Model() as hierarchical:
    alpha = pm.HalfNormal("alpha", sigma=2)
    beta = pm.HalfNormal("beta", sigma=2)
    p = pm.Beta("p", alpha=alpha, beta=beta, shape=3)
    y = pm.Binomial("y", n=n, p=p, observed=s)
    trace = pm.sample(
        draws=1000, tune=1000, chains=4,
        random_seed=20261020, return_inferencedata=True
    )
```

Az ArviZ-diagnosztikák közül az R-hat a láncok közötti keveredést, az ESS a hatékony mintanagyságot, a divergences számláló pedig a NUTS geometriai problémáit jelzi. A 19.3. alfejezetben tanultak szerint R-hat közel 1, nullához közeli divergencia és legalább néhány száz ESS szükséges. Ezt a modellt nem futtattuk; a megadott vázlat a bővítő eszközkör használatához tartozik.

19.4.7 Gyakori tévedés és ellenpélda

Gyakori tévedés, hogy a hierarchikus modell automatikusan kijavítja a rossz adatfelvételt vagy a szelekciós torzítást. Ez nem igaz. Tegyük fel, hogy a `south` régióban csak olyan rendeléseket figyeltünk meg, amelyek egy prémium szállítási opcióval érkeztek, és a prémium opció valódi késési aránya lényegesen alacsonyabb, mint a régió átlagos kockázata. A modell akkor is csak a megfigyelt prémium rendelések feltételes eloszlását becsüli, és a shrinkage legfeljebb a megfigyelt 0 arányt húzza fel, de nem hozza vissza a nem megfigyelt, magasabb kockázatú rendeléseket. A 13.1. alfejezetben tárgyalt adatfelvételi folyamat és a 18.4. alfejezet external validity korlátja itt is döntő. A shrinkage tehát nem helyettesíti a randomizálást vagy a reprezentatív mintavételt; csak a kis csoport okozta mintavételi zaj túlzott becslését tompítja.

Önellenőrzés: számold ki a `south` posterior várható értékét a súlyozott átlag formulával, és ellenőrizd, hogy ugyanazt kapod-e, mint a kód. Ha a súlyokat felcseréled, az eredmény 0,089 lesz, ami azonnal jelzi a hibát.

19.4.8 Rövid összefoglaló

A hierarchikus modell a csoportonkénti paramétereket egy közös prior eloszlásból származtatja. A shrinkage a kis csoportok becslését a prior középpontja felé húzza: a posterior várható érték a nyers arány és a prior átlag súlyozott átlaga, ahol a súly a csoportméret és az effektív prior-mintanagyság arányától függ. A konjugált béta-binom modellnél a posterior zárt alakú, és kézzel ellenőrizhető. A csoporthatás nagy mintáknál megmarad, kis mintáknál kevésbé azonosítható. A teljes hierarchikus becslés MCMC- vagy numerikus módszert igényel, és a 19.3. alfejezet ArviZ-diagnosztikáját kell alkalmazni. A shrinkage nem gyógyítja a szelekciós torzítást. Következő alkalmazásként a 20.1. alfejezetben az időbeli bizonytalanságot kapcsoljuk hasonló paraméterbecsléshez.

20 Idősorok és időbeli validáció

20.1 Időtengely, resampling és állomásossági kérdés

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: pandas, statsmodels

Előfeltétel: 19.4. alfejezet · Célkészség: Időtengely, resampling és állomásossági kérdés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

20.1.1 Cél és konkrét helyzet

A Northstar-lab `orders.csv` táblája rendelésenként egy sort tartalmaz: a megfigyelési egység itt **egy rendelés**, az időbélyeg pedig az `order_ts` oszlop UTC-értéke. Ha a kérdés az, hogyan alakul a rendelési volumen napról napra, az elemzési egységet megváltoztatjuk: az új megfigyelési egység **egy naptári nap**, a célváltozó pedig az aznapi rendelések darabszáma. A feladat nem előrejelzés, hanem az idősor helyes időtengelyének és hiányosságának előállítása, mert a későbbi modellezés 20.2. alfejezet ettől függ.

A bemenet 2400 rendelés, a rendelések 2025 első 180 napjára esnek. Első becslésként napi $2400/180 \approx 13{,}3$ rendelést várunk, de a hét napjai között lehet eltérés. Az elvárt ellenőrzés:

- a nyers aggregált sorozat indexe `DatetimeIndex`;
- a teljes naptári rács hossza 180;
- a hiányzó napok a teljes rács és a megfigyelt napok halmazkülönbsége;
- az explicit nullákkal kitöltött sorozat nem tartalmaz `NaN` értéket, dtype-ja `int64`.

20.1.2 Fogalmi alapozás: gyakoriság, hézag, trend, szezonalitás

Gyakoriság (frequency). Egy idősor gyakorisága az egymást követő megfigyelések közötti névleges időtávolság. A nyers `orders.csv` szabálytalan gyakoriságú: két rendelés között lehet 1 másodperc vagy 3 óra, nincs előre rögzített rács. Ha a `2025-01-01`, `2025-01-02`, `2025-01-04` napokon vannak megfigyelések, akkor az első két lépés 1 nap, a harmadik 2 nap; ez nem nevezhető napi frekvenciájú sorozatnak abban az értelemben, hogy a naptár minden napjához tartozik érték.

A gyakorlatban a `groupby("order_day")` vagy a `resample("D")` művelet **napi frekvenciájúvá (D)** teszi a sorozatot: a naptár az a rács, amelyen az értékeket értelmezzük. Az ennél durvább `W` heti, a finomabb `h` órai rács szintén lehetséges, és a választás mindig a döntési helyzettől függ.

Hézag (gap). A hézag a teljes időrács azon eleme, amelyhez nincs megfigyelt érték. Ha T_{grid} az elvárt napi rács, T_{observed} a megfigyelt napok halmaza, akkor

$$G = T_{\text{grid}} \setminus T_{\text{observed}},$$

ahol G a hiányzó napok halmaza. Egy T_{grid} elemről tehát nem eldönthető pusztán a hiánya, hogy mit jelent: lehet, hogy a folyamat aktív volt, de nulla esemény történt, és lehet, hogy a megfigyelés szakadt meg. Ez a két eset statisztikailag nem azonos. Az elsőnél a nulla valódi adat, a másodiknál ismeretlen állapot.

A különbség a gyakorlatban így jelenik meg: a `daily_counts` sorozat indexe csak a megfigyelt napokat tartalmazza, a `daily_dense` sorozat a teljes naptári rácsot, ahol a hiányzó elemekre `fill_value=0` került. A `daily_counts` hossza legfeljebb 180, a `daily_dense` hossza pontosan 180; a kettő közötti napok halmaza G. A 0-val kitöltés csak akkor megalapozott, ha az adatszerződés kimondja: *a megfigyelő rendszer minden nap aktív volt, és a napon nulla rendelés érkezett.* Ez a Northstar-lab generátoránál a rendelési eseménysorozatra teljesül, de a mesterségesen kihagyott nap már nem ilyen.

20.1.2.1 Nulla forgalom vs hiányzó adatgyűjtés

A két eset csak akkor különíthető el, ha van **gyűjtési napló** (collection log) vagy a forrás naponta explicit nulla rekordot ír. A nyers `orders.csv` önmagában nem elég: ha egy napon nincs sor, az lehet valódi nulla forgalom vagy kiesett gyűjtés. Példa:

Nap	Gyűjtés (`collected`)	Megfigyelt rendelés	Helyes kezelés
2025-01-05	igaz	0	valódi nulla; 0-ként tartható
2025-02-03	hamis	nincs rekord	hiányzó adat; nem 0, hanem `NaN` + flag
2025-02-04	hamis	nincs rekord	hiányzó adat; nem 0, hanem `NaN` + flag

A gyakorlati kód a naptári rácsot összekapcsolja a gyűjtési naplóval, és csak a sikeresen gyűjtött napokon engedi a nulla kitöltést:

```
import numpy as np
import pandas as pd

grid = pd.date_range("2025-01-01", "2025-06-29", freq="D")
calendar = pd.DataFrame({"date": grid, "collected": True})
calendar.loc[calendar["date"].isin(pd.to_datetime(["2025-02-03", "2025-02-04"])),
             "collected"] = False

daily = orders.groupby("order_day").size().rename("orders")
dense = calendar.set_index("date").join(daily, how="left")
dense.loc[~dense["collected"], "orders"] = np.nan   # hiányzó gyűjtés -> NaN
zero_filled = dense["orders"].fillna(0)             # csak a gyűjtött napokon érvényes
```

A `fillna(0)` a hiányzó gyűjtésű napokat is nulla forgalmúvá tenné, ami lefelé torzítja az átlagot; a `ffill` pedig azt feltételezné, hogy a forgalom nem változott, ami szintén megalapozatlan. A helyes politika: a hiányzó napot `NaN`-ként és `collected=False` flaggel tartjuk, az elemzésből kizárjuk vagy hiányzóadat-modellben kezeljük, és érzékenységi elemzést készítünk a kizárás hatásáról. A valódi nulla napot ezzel szemben 0-ként kell szerepeltetni, különben a forgalom alulbecsült lesz.

Trend. A trend a sorozat szintjének lassú, irányított változása. Ha a napi rendelések átlagos száma folyamatosan emelkedik az idővel, akkor az első 30 nap átlaga kisebb, mint az utolsó 30 napé. A trendet legkényelmesebben mozgóátlaggal közelítjük: például a 7 napos mozgóátlag

$$\hat{T}_t = \frac{1}{7}\left(Y_{t-6} + Y_{t-5} + \cdots + Y_t\right),$$

ahol Y_t a t nap rendelési száma, és az ablakot gyakran a jelenlegi napig görgetjük. A mozgóátlag elsimítja a napon belüli véletlen ingadozást, és láthatóvá teszi a lassabb szintváltozást.

Szezonalitás (seasonality). A szezonalitás rögzített periódusú, ismétlődő minta. A napi rendelési adatban a legtermészetesebb periódus a hét napja: a hétvégi napokon eltérő lehet a rendelési volumen. A hét napja szerinti szezonális átlag

$$\bar{Y}_j = \frac{1}{N_j} \sum_{t:\mathrm{dow}(t)=j} Y_t,$$

ahol $j \in \{0, \ldots, 6\}$ a hét napjának kódja a `DatetimeIndex.dayofweek` szerint (0 = hétfő, 6 = vasárnap), N_j pedig az adott nap előfordulásainak száma. A szezonális hatás nem oksági magyarázat: csak annyit mond, hogy a hét egyes napjain szisztematikusan más a szint.

Aggregálás (aggregation). Az idősor előállítása aggregálás: a rendelési szintű táblából csoportosítjuk a sorokat a nap szerint, majd minden csoportban összegzünk, átlagolunk vagy valamilyen más összefoglaló értéket számolunk. A Northstar-példában a `size()` az adott napi rendelések számát adja. Az aggregálás előtti shape `(2400, n_columns)`, utána `(180,)`, ha a teljes rácsot tekintjük; az általános szabály, hogy a csoportosítás mindig csökkenti a megfigyelési egység számát.

Stacionaritási intuíció (stationarity intuition). Egy idősor akkor tekinthető *gyenge értelemben stacionáriusnak*, ha a statisztikai tulajdonságai nem függenek attól, hogy a sorozat melyik időablakát nézzük. A leggyakrabban használt közelítő feltételek:

$$E[Y_t] = \mu, \qquad \mathrm{Var}(Y_t) = \sigma^2, \qquad \mathrm{Cov}(Y_t, Y_{t-k}) = \gamma_k,$$

ahol Y_t az idősor értéke a t időpontban, μ és σ^2 konstans, γ_k pedig csak a k késleltetéstől függő autokovariancia. A feltétel azt fejezi ki, hogy a sorozat átlaga, varianciája és késleltetett kapcsolata azonos, ha az ablakot későbbre toljuk.

A napi rendelési sorozatunk csak akkor stacionárius, ha nincs szisztematikus trend, és a variancia sem változik az időben. A trend kimozdítja az átlagot az állandóságból; a hét napja szerinti szezonalitás azt jelenti, hogy $E[Y_t]$ a nap kódjától periodikusan függ, tehát nem állandó a teljes sorozaton. A gyakorlati ellenőrzés egyszerű formája: a mozgóátlag és a mozgóvariancia ne mutasson erős, irányított elmozdulást. Ez még nem formális teszt, de az intuícióhoz elegendő; a formális becslési és tesztelési kérdéseket később a megfelelő modellekkel kezeljük.

20.1.3 Teljes minimális munkapélda

A példát a tanulólabor könyvtárából futtatjuk: `lab/` munkakönyvtár, Linux/macOS bash vagy Windows PowerShell. A környezet a lab `pyproject.toml` alapján `uv sync`-kel telepíthető; a futtatáshoz `uv run python` szükséges.

```
import pandas as pd
from statsmodels.tsa.seasonal import seasonal_decompose

# Munkakönyvtár: lab/
orders = pd.read_csv("orders.csv")
```

```
# Az UTC-bélyegből naptári nap: az explicit +00:00 offsetet kezeljük,
# majd a napra normalizáljuk időzóna nélkül.
orders["order_day"] = (
    pd.to_datetime(orders["order_ts"], utc=True)
    .dt.tz_convert(None)
    .dt.normalize()
)

# Napi aggregálás: ez a hiányos indexű sorozat.
daily_counts = orders.groupby("order_day").size()

# Teljes naptári rács 2025 első 180 napjára.
start_day = pd.Timestamp("2025-01-01")
end_day = pd.Timestamp("2025-06-29")
full_day_index = pd.date_range(start_day, end_day, freq="D")

# Explicit nullás sorozat: a rács minden napjához tartozik érték.
daily_dense = daily_counts.reindex(full_day_index, fill_value=0)

# Ellenőrző invariánsok.
assert len(full_day_index) == 180
assert daily_dense.notna().all()
assert (daily_dense.index == full_day_index).all()
print("dtype:", daily_dense.dtype, "| index dtype:", daily_dense.index.dtype)
print("sűrű sorozat hossza:", len(daily_dense))

# Mesterséges hiány: tegyük fel, hogy 2025-02-02-t a rendszer nem rögzítette.
daily_with_gap = daily_dense.drop(pd.Timestamp("2025-02-02"))
actual_gaps = full_day_index.difference(daily_with_gap.index)
print("hiányzó napok:", actual_gaps)

# Trend és szezonalitás szemléltető vizsgálata.
trend_7d = daily_dense.rolling(7).mean()
weekday_means = daily_dense.groupby(daily_dense.index.dayofweek).mean()
vol_28d = daily_dense.rolling(28).var()

print("7 napos mozgóátlag eleje:")
print(trend_7d.dropna().head(3))
print("héten belüli napátlagok (0=hétfő):")
print(weekday_means)
print("28 napos mozgóvariancia eleje és vége:")
print(vol_28d.dropna().head(1), vol_28d.dropna().tail(1))

decomp = seasonal_decompose(daily_dense, model="additive", period=7)
print("dekompozíciós trend vége:")
print(decomp.trend.dropna().tail(3))
```

A `seasonal_decompose` az additív felbontást $Y_t = T_t + S_t + R_t$ alakban becsli, ahol T_t a trend, S_t a heti szezonális komponens, R_t a maradék; a `period=7` a heti ciklust jelöli. A dekompozíció itt leíró eszköz, nem előrejelzési modell.

A futtatási parancs Linux/macOS alatt:

```
cd lab
uv run python time_grid_example.py
```

Windows PowerShellben ugyanez a `cd lab; uv run python time_grid_example.py` paranccsal adható meg. A kód a lab definiált adatfájljait használja, külső hálózatot nem igényel.

20.1.4 Eredmény értelmezése

A `daily_counts` a rendelés sorok csoportosításából keletkezik: az index csak azokat a napokat tartalmazza, amelyekhez legalább egy rendelés tartozik. Ha minden nap érkezett rendelés, a hossza 180; ha valamelyik nap nem, akkor rövidebb. A `daily_dense` hossza minden esetben pontosan 180, mert a `reindex` a teljes `full_day_index`-et használja. A `daily_dense.dtype` `int64`, mert a `fill_value=0` egész szám; a `daily_dense.index.dtype` `datetime64[ns]`.

A mesterségesen kihagyott nap mutatja a két fogalom közötti különbséget. A `daily_dense` 2025-02-02-ére 0 áll, mert az előbbi feltételezés szerint a napon nulla rendelés volt. A `daily_with_gap` indexéből ugyanez a nap hiányzik; a `full_day_index.difference(...)` halmaz pontosan `{2025-02-02}`. A hiányos nap tehát nem nulla érték, hanem hiányzó megfigyelés: az elem hiányzik az indexből, ezért a sorozat hossza csak 179.

A trend és a szezonalitás vizsgálata a gyakorlatban ilyen ellenőrzéseket ad:

- ha a 7 napos mozgóátlag eleje és vége szisztematikusan eltér, trend van jelen;
- ha a hét napjai szerinti átlagok nem egyenlők, szezonalitás van jelen;
- ha a 28 napos mozgóvariancia erősen változik, a stacionaritási intuíció sérül.

A fenti kód kimenete a helyi futtatástól függ; az itt közölt sorok *szemléltető, nem mért* kimenetek. A hallgató a saját futás során ellenőrzi a három invariant: a sűrű sorozat hossza 180, nincs `NaN`, és a hiányzó napok halmaza a szándékozott napot tartalmazza.

Időindex-definíció: a hiányzó nap és a valóban nulla rendelést tartalmazó nap más állítás. Zérussal kitöltés csak akkor helyes, ha bizonyított a teljes adatgyűjtés; időzónaváltásnál és napra aggregálásnál az üzleti naptárnapot rögzíteni kell.

20.1.5 Feladatok

1. **Értelmezés.** Adott három nap: 2025-01-01 (4 rendelés), 2025-01-02 (7 rendelés), 2025-01-04 (5 rendelés). Milyen két különböző állítást tehetünk a 2025-01-03 napra, ha az adatszerződés szerint (a) a rendszer a napon aktív volt, de nem volt rendelés; (b) a rendszer aznap nem rögzített adatot? Melyik esetben szabad 0-val kitölteni a hiányt?

2. **Megvalósítás.** A `orders.csv` beolvasása után készíts óránkénti sorozatot: az új megfigyelési egység az óra, az érték az adott órában leadott rendelések száma. Használd a teljes rácsot 2025-01-01 00:00-tól 2025-06-29 23:00-ig. Add meg az explicit nullás sorozat hosszát, dtype-ját és a hiányzó órák számát. Várt invariáns: a teljes órai rács hossza $180 \cdot 24 = 4320$, a hiányzó órák száma legalább $4320 - 2400 = 1920$, mert legfeljebb 2400 különböző órában lehet rendelés.

3. **Hibakeresés.** A következő kódrészlet heti átlagot számol:

```
daily = orders.groupby("order_day").size()
daily_full = daily.reindex(full_day_index)  # fill_value nélkül
weekly_avg = daily_full.resample("W").mean()
```

Mi a hiba, és hogyan függ a javítás attól, hogy a hiányzó nap tényleges nulla vagy mérési hiány? Adj mindkét esetre javított kódot.

4. **Megváltozott feltétel.** A Northstar külön kísérleti projektjében az `outcomes.csv` ügyfélenként egyetlen `conversion_7d` bináris értéket és `revenue_7d_minor` összeget tartalmaz a 2025-07-01-től 2025-07-08-ig tartó teljes 7 napos ablakra. Helyes-e ebből napi konverziós arány idősorozatot készíteni úgy, hogy a `conversion_7d` értékét 7-tel osztjuk vagy naponként ismételjük? Indokold, melyik megfigyelési szerződés sérül.

20.1.6 Megoldások és indoklás

1. **Megoldás.** Az (a) esetben a 2025-01-03-hoz tartozó érték tényleges nulla: a rendszer figyelte a napot, és nem érkezett rendelés. Ezt 0-val ki lehet tölteni, mert a 0 valódi adat. A (b) esetben a 2025-01-03 megfigyelés hiányzik: nem tudjuk, hány rendelés érkezett. A 0-val kitöltés hamis adatot gyárt, mert a napot nem figyelték. Ilyenkor a helyes kezelés vagy a nap elhagyása, vagy egy explicit hiány-modell, például imputáció; a konkrét Northstar-laborban a rendelési folyamat minden nap aktív, ezért ott a hiány nulla rendelést jelent, azonban ezt az adatszerződésből lehet tudni, nem a CSV-ből.

2. **Megoldás.** A kód:

```
orders["order_hour"] = (
    pd.to_datetime(orders["order_ts"], utc=True)
    .dt.tz_convert(None)
    .dt.floor("h")
)
hourly_counts = orders.groupby("order_hour").size()
full_hours = pd.date_range(
    "2025-01-01", "2025-06-29 23:00", freq="h"
)
hourly_dense = hourly_counts.reindex(full_hours, fill_value=0)
missing_hours = full_hours.difference(hourly_counts.index)

assert len(full_hours) == 4320
assert hourly_dense.notna().all()
assert len(missing_hours) == len(full_hours) - len(hourly_counts)
print(len(hourly_dense), hourly_dense.dtype, len(missing_hours))
```

Az ellenőrző invariant: a sűrű sorozat hossza 4320, dtype-ja `int64`; a hiányzó órák száma legalább 1920, mert egyedi óra legfeljebb 2400 van. A tényleges szám valamivel nagyobb lehet, ha több rendelés esik ugyanabba az órába.

3. **Megoldás.** A hiba az, hogy a `reindex fill_value=0` nélkül `NaN` értéket ad a hiányzó napokra. A `resample("W").mean()` ezt a `NaN`-t csendben figyelmen kívül hagyja; a heti átlag ezért nem a teljes hétre vonatkozó összeg, hanem csak a megfigyelt napokra. A javítás attól függ, mit jelent a hiány:

- tényleges nulla esetén:

```
daily_full = daily.reindex(full_day_index, fill_value=0)
weekly_avg = daily_full.resample("W").mean()
```

- mérési hiány esetén nem szabad 0-val kitölteni; ekkor a hiányt külön kell kezelni, például a heti átlagot csak a teljes héttel rendelkező napokra számoljuk, vagy imputálunk, és az imputáció módszerét jelenteni kell:

```
daily_full = daily.reindex(full_day_index)
weekly_avg = daily_full.resample("W").mean()  # NaN-t explicit ignorál
```

A választott megoldás csak akkor elfogadható, ha a döntést az adatszerződésben rögzítjük.

4. **Megoldás.** Nem helyes. Az `outcomes.csv` megfigyelési egysége **egy ügyfél végső 7 napos kimenete**, nem pedig egy ügyfél-nap pár. A `conversion_7d` azt jelzi, hogy az ügyfél az egész ablak alatt konvertált-e; nem tartalmazza a konverzió pontos napját. A 7-tel osztás vagy naponkénti ismétlés új adatot gyárt, amely sem nem konverziós esemény, sem nem per-nap ráta. A napi konverziós idősorhoz külön naponkénti eseményadat kellene, például `conversion_event_at`; ilyen a jelenlegi kísérleti adatban nincs. Az ITT-elemzés

számára a helyes megfigyelés továbbra is a teljes randomizált 300 ügyfél és a 7 napos kimenet.

20.1.7 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés az, hogy a hiányzó megfigyelést automatikusan nullának tekintjük. A `fillna(0)` mesterégesen csökkenti a szintet: egy kételemes példában, ahol az első nap 10 rendelést látott, a második nap pedig ismeretlen, a `[10, 0]` sorozat átlaga 5, holott a második nap akár 10 rendelés is lehetett volna. A `dropna` ezzel szemben azt állítja, hogy a második napról nincs megfigyelés; az átlag akkor 10. Mindkét döntés állítás, nem technikai művelet. Ez a 13.4. alfejezet hiányzási szerződés időbeli változata: a mechanizmus feltevése nem a táblából következik, hanem a mérési folyamatból. A 16.1. alfejezet nullmodell-logikája is figyelmeztet: a szélsőségesség csak az előre rögzített modell szerint értelmezhető, így az idősor hiányosságát is a modellben kell rögzítenünk.

Önellenőrzés: mi a különbség a távollévő index és a nulla érték között? Miért nem stacioner a növekvő trendes sorozat? Mit jelent egy napi sorozat `D` frekvenciája? Ha a csúszó ablakos variancia határozottan nő, mire következtetünk?

20.1.8 Rövid összefoglaló

A nyers rendelés-időbélyeg szabálytalan gyakoriságú; a napi rácsra aggregálással szabályos frekvenciájú sorozatot kapunk. A hézag a teljes rács és a megfigyelt napok halmazkülönbsége; a hiányzó napokat csak akkor szabad nullával kitölteni, ha a mérési folyamat aktív volt és tényleg nulla esemény történt. A trend lassú irányított szintváltozás, a szezonalitás rögzített periódusú ismétlődés; a mozgóátlag és mozgóvariancia erős elmozdulása a stacionaritás sérülésére utal. A következő alkalmazásban erre az időtengelyre építünk cutoff előtti feature-képzést és gördülő visszatesztet, ahol a predikciós pillanat és a címke elérhetősége tovább szűkíti a megfigyelt ablakot.

20.2 Időhelyes feature és gördülő visszateszt

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: pandas, statsmodels

Előfeltétel: 20.1. alfejezet · Célkészség: Időhelyes feature és gördülő visszateszt · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

20.2.1 Cél és konkrét helyzet

A Northstar webshop napi rendelésszámát szeretnénk egy nappal előre látni, hogy a raktár a másnapi kapacitást tervezhesse. A megfigyelési egység egy naptári nap, a bemenet a teljesített rendelések darabszáma aznap. A célváltozó y_{t+1}, ahol t a döntési nap indexe; a horizont $h = 1$ nap. A döntéshozó a t nap zárásakor rendelkezésre álló múltat ismeri, tehát y_t-t már igen, y_{t+1}-t még nem.

A labor egy 60 napos, determinisztikus szintetikus sort használ, hogy a feature-ök minden gépen kézzel is ellenőrizhetők legyenek. Három előre rögzített originnal futtatunk gördülő visszatesztet: 2025-02-11, 2025-02-18 és 2025-02-25. Mindegyik originre a tanítóhalmaz az origint megelőző sorokból áll, a teszthalmaz pontosan a következő nap. Az induló becslés: a sor heti ciklust és

enyhe trendet tartalmaz, ezért a naiv, utolsó megfigyelt értékes baseline a heti fázisváltásnál akár 15–17 rendeléses hibát is vét; egy 7 napos laget használó modelltől ennél jobbat várunk.

20.2.2 Fogalmi magyarázat

Legyen y_t a t napi rendelésszám, mértékegysége darab/nap, $t = 1, \dots, N$ egész napindex. A teljes megfigyelt sorozat alakja N. Az $\mathcal{F}_t$ jelölés azt az információhalmazt jelenti, amely a t időpontban ténylegesen rendelkezésre áll: informálisan minden, ami legkésőbb a t napig ismert. Egy prediktor akkor időhelyes, ha $\mathcal{F}_t$-mérhető, azaz nem használ t-nél későbbi értéket.

Lag (késleltetés). A k lépéses lag a múltbeli érték:

$$L^k y_t = y_{t-k}, \qquad k = 1, 2, \dots$$

Pandasban `series.shift(k)`. A kimenet ugyanolyan hosszú, mint a bemenet; az első k elem `NaN`, mert ott a múlt még nincs meg. Ha a cél y_{t+1}, akkor $L^1 y_t = y_t$ még múlt, tehát használható predictor.

Rolling (mozgó ablak). A w szélességű mozgóablak a t napon záródó, w napos múlt átlaga:

$$R_w y_t = \frac{1}{w} \sum_{i=0}^{w-1} y_{t-i}.$$

Pandasban `series.rolling(window=w).mean()`. Az ablak $t - w + 1, \dots, t$ napokat tartalmazza; a `rolling` önmagában nem néz a jövőbe. Ha a cél a t napi érték lenne, a t-t is tartalmazó ablak már a céllal átfedne; ilyenkor `shift(1).rolling(w).mean()` a $t - 1$-ig záródó ablakot adja. A mi célunk y_{t+1}, ezért a t nap még múlt, és a t záródású ablak megengedett.

Forecast horizon (előrejelzési horizont). A cél y_{t+h}, ahol h a jövőbeli lépések száma. Az itteni labor $h = 1$ lépéssel dolgozik: a t sorhoz tartozó címke a következő napi rendelésszám, kódban `shift(-1)`. Az utolsó sor célja `NaN`, mert a sorozat végén túl nincs jövő.

Gap (tisztító hézag, purge gap). A gap a felhasznált múlt vége és a cél időszaka közötti, szándékosan kihagyott tartomány. Két szerepe van. Egyrészt a hétnapos horizontnál a $t + 1, \dots, t + 6$ napok sem feature-ként, sem tanítócímkeként nem állnak rendelkezésre a t napon. Másrészt a realisztikus folyamatban a címke maga is késve érkezik: a Northstar kezdőlaborban a `label_available_at` csak a tényleges kézbesítéskor lép életbe, ezért a cutoff előtti rendelések közül a még meg nem figyelt célú sorokat egy kihagyott határsávval zárjuk ki a tanításból. A gap tehát nem üres sor, hanem adatfolyami feltétel.

Expanding split. Az időbeli tanítóhalmaz a sorozat elejétől az originig bővül; a tesztablak mindig az origin után következik. Ha az origint tologatjuk, gördülő originhoz jutunk. Például az első origin a 41. nap, a második a 48., a harmadik az 55.; minden lépésben a tanítóhalmaz egyre hosszabb, a teszthalmaz mindig a soron következő nap.

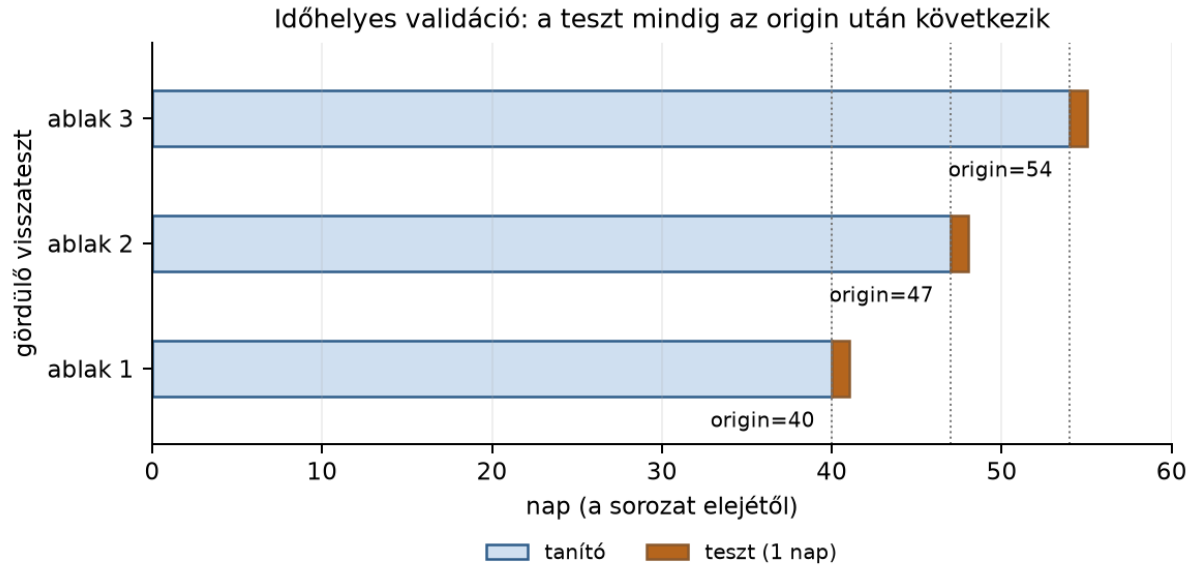
Temporal leakage (időbeli szivárgás). Akkor lép fel, ha a modell olyan adatot használ, amely a döntés pillanatában még nem volt ismert. Formája lehet feature-szivárgás: például a célképzéshez használt `shift(-1)` oszlop véletlenül bekerül a predictorok közé; vagy split-szivárgás: a sorok véletlen összekeverésével a jövőbeli szomszédok a tanítóhalmazba kerülnek.

A shift és a rolling prefixaggregációkra vezethető vissza. Ha $C_t = \sum_{s=1}^{t} y_s$ a kumulált összeg $C_0 = 0$ mellett, akkor a mozgóablak egy prefixkülönbség:

$$R_w y_t = \frac{C_t - C_{t-w}}{w}, \qquad t > w.$$

Mindkét oldal $\mathcal{F}_t$-mérhető: a t napig tartó, kumulált múltból számol. Ez a matematikai hidunk: nem az egyes napokat memorizáljuk, hanem az időben monoton módon elérhető aggregátumokat.

20.2.2.1 Fogalmi ábra: a gördülő visszateszt idővonala



20.1. ábra: Időhelyes validáció: három originhez bővülő tanítóablak és a közvetlenül utána következő egynapos tesztablak; a jövő soha nem kerül a tanítóhalmazba.

Az ábra a rolling-origin visszateszt szerkezetét mutatja: minden ablakban a tanítóhalmaz az origint megelőző napokból áll, a teszt pedig az origin utáni nap. A tanítóablak bővül, a tesztablak mindig előre lép. A konklúzió ábra nélkül is ugyanaz: az as-of fegyelem nem formai szabály, hanem az időbeli szivárgás megelőzése; a teszt napja soha nem szerepelhet a tanítóban. A teljes generátor: `lab/figures/make_concept_figures.py` `validation_timeline()`; a reprodukálható mag:

```
import matplotlib.pyplot as plt

origins = [40, 47, 54]
fig, ax = plt.subplots(figsize=(6.6, 3.4))
for row, origin in enumerate(origins):
    ax.barh(row, origin, height=0.45, color="#cfdff0", edgecolor="#2f5d8a")
    ax.barh(row, 1, left=origin, height=0.45, color="#b5651d", edgecolor="#8a5a2f")
    ax.text(origin - 1, row - 0.34, f"origin={origin}", ha="right", va="top")
ax.set_yticks(range(3)); ax.set_yticklabels(["ablak 1", "ablak 2", "ablak 3"])
ax.set_xlabel("nap (a sorozat elejétől)"); ax.set_ylabel("gördülő visszateszt")
ax.set_xlim(0, 60); ax.grid(axis="x", alpha=0.2); plt.show()
```

20.2.3 Teljes minimális munkapélda

Először kézi kontroll egy 12 napos mini soron. A kód pandas 2.2.3-mal futtatható; a kimenetet nem a szövegíró futtatta, hanem a shift és rolling definíciójából levezetett elméleti érték.

```
import pandas as pd

idx = pd.date_range("2025-01-01", periods=12, freq="D")
df_mini = pd.DataFrame(
    {"orders": [10, 12, 9, 11, 13, 15, 14, 16, 18, 20, 19, 22]},
    index=idx,
)
df_mini["lag1"] = df_mini["orders"].shift(1)
df_mini["roll3_past"] = df_mini["orders"].shift(1).rolling(window=3).mean()
print(df_mini)
```

Elméletileg várt kimenet:

index	orders	lag1	roll3_past
2025-01-01	10	NaN	NaN
2025-01-02	12	10.0	NaN
2025-01-03	9	12.0	NaN
2025-01-04	11	9.0	10.33
2025-01-05	13	11.0	10.67
2025-01-06	15	13.0	11.00
2025-01-07	14	15.0	13.00
2025-01-08	16	14.0	14.00
2025-01-09	18	16.0	15.00
2025-01-10	20	18.0	16.00
2025-01-11	19	20.0	18.00
2025-01-12	22	19.0	19.00

A 2025-01-04 sor `roll3_past` értéke a 01-01, 01-02 és 01-03 rendelések átlaga: $(10 + 12 + 9)/3 = 10.33$. A `shift(1)` miatt a saját napja, a 11, nem szerepel az ablakban.

A laborra készítsük el a 60 napos determinisztikus sort és a három origint. A munkakönyvtár a projektgyökér; a fájlt mentsük `s078_rolling_backtest.py` néven, és futtassuk a lab `uv` környezetével:

```
uv run python s078_rolling_backtest.py
```

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

t = np.arange(60)
raw = 40.0 + 20.0 * np.sin(2 * np.pi * t / 7) + 0.25 * t
daily = pd.Series(
    raw,
    index=pd.date_range("2025-01-01", periods=60, freq="D"),
    name="orders",
).round(1)

feat = pd.DataFrame({"orders": daily})
feat["lag1"] = feat["orders"].shift(1)
feat["lag7"] = feat["orders"].shift(7)
feat["roll7"] = feat["orders"].rolling(window=7).mean()
feat["target_lead1"] = feat["orders"].shift(-1)
feat = feat.dropna()

origins = pd.to_datetime(["2025-02-11", "2025-02-18", "2025-02-25"])

for origin in origins:
```

```
train = feat.loc[feat.index < origin]
test_row = feat.loc[origin]

baseline_pred = test_row["orders"]

X_train = sm.add_constant(train[["orders", "lag7"]])
y_train = train["target_lead1"]
model = sm.OLS(y_train, X_train).fit()

x_test = np.array([[1.0, test_row["orders"], test_row["lag7"]]])
model_pred = float(model.predict(x_test)[0])

print(
    f"origin={origin.date()} baseline={baseline_pred:.1f} "
    f"model={model_pred:.1f} actual={test_row['target_lead1']:.1f}"
)
```

A naiv baseline predikciója az origin napi érték, tehát y_{origin}, mert $h = 1$ mellett ezt ismerjük utoljára. A modell az `orders` jelenlegi értékét és a 7 napos laget használja. A három sor elméletileg várt baseline és tényértéke a determinisztikus képletből:

```
origin=2025-02-11 baseline=34.6 model=<futtatástól függ> actual=50.5
origin=2025-02-18 baseline=36.4 model=<futtatástól függ> actual=52.2
origin=2025-02-25 baseline=38.1 model=<futtatástól függ> actual=54.0
```

A `model` oszlop a statsmodels által megoldott legkisebb négyzetes becslés eredménye; ez a szemléltető kimenet, nem a könyvírás során mért érték.

20.2.4 Eredmény értelmezése

A három origin a tiszta időbeli felosztás szerződését mutatja. Minden originra a tanítóhalmaz csak az origint megelőző sorokból áll, ezért a legkésőbbi tanítócímke az origin napi rendelésszám, amely a nap zárásakor már ismert. A tesztsor az origin naphoz tartozik, célja az origin+1 nap; ez a döntés pillanatában még jövő, de az értékeléskor már tény. A naiv baseline a három fázisváltós napon rendre alábecsül: a hétfői/vasárnapi ciklusból adódó emelkedést nem tudja. A heti laget használó modell a 7 nappal korábbi hasonló hétnapot látja, ezért várhatóan közelebb kerül a tényhez.

A kód időhelyességi feltétele: a **prediktoroszlopokban** nincs a sor napjánál későbbi információ. A `target_lead1` céloszlop szándékosan a következő napból származik, ezért az előbbi állítás a teljes `feat` táblára nem igaz. A `lag7` minden sora $t-7$-ből jön, a `roll7` ablak $t-6, \ldots, t$ napokat fed le, a cél $t+1$. A tanítóhalmaz `index < origin`, tehát az OLS illesztésekor a legkésőbbi felhasznált nap az origin előtti; a tesztpredikció az origin napi múltból számol.

A három origin hibáinak átlaga nem konfidenciaintervallum. A 16.2. alfejezetben tanultak szerint kis számú, összefüggő hiba esetén a foldok szórása nem becsli megbízhatóan a mintavételi bizonytalanságot. A 8.2. alfejezetben tanult egy sor, egy megfigyelési egység elv itt napi szinten érvényes: ha valaki a rendeléstételeket napokra összegezné, az aggregáció előtt szemcsézettségi kontroll kell.

As-of-validáció: gördülő visszatesztnél minden feature előállítása kizárólag az adott origin időpontban rendelkezésre álló adatokból történhet. A teljes idősoron becsült imputáció, skálázás vagy szezonális komponens időbeli szivárgást okozhat.

20.2.4.1 As-of hozzáférési szerződés

A dátum szerinti rendezés önmagában nem garantálja, hogy egy címke az adott foldban már ismert volt. Minden sorhoz rögzíteni kell az **esemény idejét** és a **rendelkezésre állás idejét** (`label_available_at`); a fold csak akkor használhatja a sort, ha a rendelkezésre állás nem későbbi az origin időpontjánál. A `target_lead1` esetében az esemény a t nap, a címke a $t+1$ nap végén válik ismertté, ezért:

```
feat["label_available_at"] = feat.index + pd.Timedelta(days=1)

for origin in origins:
    train = feat[feat.index < origin]
    assert (train["label_available_at"] <= origin).all(), (
        f"időszivárgás: {origin} előtt nem elérhető címke a tanítóhalmazban"
    )
```

Ha egy címke késve, például három nappal később érkezik, a `label_available_at` értéke $t+3$; az a fold, amelynek originja $t+1$ vagy $t+2$, nem használhatja, még akkor sem, ha az esemény dátuma korábbi. A késői címke csak a későbbi foldokban jelenhet meg. Ugyanez vonatkozik a feature-ökre is: ha egy külső adatforrás publikációs késése van, a feature `feature_available_at` idejét kell használni, nem az esemény idejét.

Eset	Esemény ideje	Rendelkezésre állás	Használható a $t+1$ origin foldban?
Normál címke	t	$t+1$	igen, ha $t+1 \le$ origin
Késve érkező címke	t	$t+3$	nem, csak a $t+3$ utáni foldokban
Késői feature-publikáció	t	$t+2$	nem, a feature a $t+2$ előtti originban nem ismert

20.2.5 Feladatok

1. **Értelmezés.** A mini táblázatban add meg a 2025-01-07, 2025-01-08 és 2025-01-12 sorok `lag1` és `roll3_past` értékeit. Melyik sorban először értelmezhető a `roll3_past`, és miért pont ott?
2. **Implementálás.** A 60 napos labor `daily` sorához készítsd el a `lag7` és a `roll7` oszlopot, majd írasd ki a 2025-02-26 és 2025-03-01 közötti négy sort az `orders`, `lag7`, `roll7` oszlopokkal. Ellenőrizd, hogy az utolsó sor `roll7` értéke a tőle visszafelé számított hét nap átlaga.
3. **Hibadiagnózis.** Egy kolléga a következőt futtatja a 60 napos adaton:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X = df[["orders"]].iloc[:-1]
y = df["orders"].shift(-1).iloc[:-1]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=0
)
m = LinearRegression().fit(X_tr, y_tr)
print(np.sqrt(mean_squared_error(y_te, m.predict(X_te))))
```

A kolléga 1.7-es RMSE-t kap, miközben a valós egy lépéses előrejelzés hibája a laborban 10 felett van. Mi a hiba a szerződésben, és hogyan javítanád úgy, hogy az értékelés a döntési pillanatban elérhető múltat tükrözze?

4. **Megváltozott feltétel.** Az üzlet most a 7 nappal későbbi rendelésszámot szeretné előrejelezni, tehát $h = 7$. Írd át a labor feature- és targetképzését, valamint a tanító/teszt felosztást. Add meg a cél oszlopot, a gap feltételt, és azt, hogy a tanítócímkéknek mikorra kell megérkezniük az origin pillanatához képest.

20.2.6 Megoldások és indoklás

1. A kért sorok: 2025-01-07 `lag1=15.0`, `roll3_past=13.00`; 2025-01-08 `lag1=14.0`, `roll3_past=14.00`; 2025-01-12 `lag1=19.0`, `roll3_past=19.00`. A `roll3_past` a 2025-01-04 sorban értelmezhető először, mert a `shift(1)` után a 01-04 sorban a 01-03, 01-02, 01-01 hármas áll; a 01-03 soron a shiftelt ablak még tartalmazza a 01-01 `NaN`-ját.

2. A megoldás a labor feature-kódját használja:

```
feat["lag7"] = feat["orders"].shift(7)
feat["roll7"] = feat["orders"].rolling(window=7).mean()
print(feat.loc["2025-02-26":"2025-03-01", ["orders", "lag7", "roll7"]])
```

Az utolsó sor 2025-03-01, indexe $t = 59$. A `roll7` ablak a 2025-02-23 és 2025-03-01 közötti hét nap. Elfogadási tűrés: a kerekített napi értékekkel számolva az ellenőrző átlag és a kinyomtatott `roll7` eltérése kisebb, mint 0.1. Amelyik sor a saját napját is belevenné a `lag7` helyére, az invariáns megsértője.

3. A kolléga kódja egyszerre vét feature-leakage-et és split-leakage-et. Az `orders` oszlop a t nap értéke, a címke `shift(-1)` a $t + 1$ nap. A `train_test_split` alapértelmezett `shuffle=True` keveréssel a szomszédos sorokat együtt dobálja: a tanítóhalmazba olyan t sorokból származó $t + 1$ értékek kerülnek, amelyek a teszthalmaz `orders` jellemzőivel majdnem azonosak. Ettől a modell szinte tökéletesnek látszik, miközben valós használatkor a $t + 1$ nap nem ismert. Javítás: időrendi split, keverés nélkül, előre rögzített originnal:

```
origin = feat.index[41]
train = feat.loc[feat.index < origin]
X_tr = train[["orders"]]
y_tr = train["target_lead1"]
test = feat.loc[[origin]]
# predikció a test orders-ből
```

4. A $h = 7$ szerződésben a cél `target_lead7 = orders.shift(-7)`. A feature-ök legkésőbb t-ig használhatók; a `roll7` a t-ig záródó ablak. A gap: a $t+1, \ldots, t+6$ napok sem a targetben, sem a feature-ökben nem jelenhetnek meg, és a tanítócímke akkor áll rendelkezésre, ha $t+7$ teljesült. Az origin napon a tanítóhalmaz utolsó megengedett sora ezért $t \leq \text{origin} - 7$; a teszt sor az origin nap, célja origin+7. A Northstar kezdőlabor `label_available_at < cutoff` feltétele ugyanezt a purging gapet írja elő a 7 napos kézbesítési célra.

20.2.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az, hogy a véletlen `train_test_split` az idősorokon biztonságosabbnak tűnik a `random_state` rögzítése miatt, de a reprodukálhatóság nem pótolja az időbeli szerződést. A fenti 3. feladatban a véletlen keveréses modell 1.7-es RMSE-t mutat, miközben ugyanaz a predictor valós, egy lépéses használatban a heti ciklus fázisváltásainál 15-17 rendeléses hibát

követ el. A `random_state` csak azt garantálja, hogy ugyanaz a hibás sorrend ismétlődik; nem azt, hogy a jövő ne szivárogjon a múltba. A 20.1. alfejezetben tanult frekvencia és gap itt döntő: a rendszeres napi rács mellett is külön kell kérdezni, hogy a címke mikortól része $\mathcal{F}_t$-nek.

Önellenőrzés: minden predictorom indexe legfeljebb a döntési nap? A target minden sora a döntési nap után h lépéssel van? A tanító sorok címkéi megérkeztek-e az originig? Ha bármelyik kérdésre nemleges a válasz, a gapet vagy a splitet javítani kell.

20.2.8 Rövid összefoglaló

A lag és a rolling csak akkor időhelyes, ha kizárólag az $\mathcal{F}_t$-ben ismert múltból számol; a cél y_{t+h} horizontjához képest a t nap még múlt lehet, a $t+1$ már nem. A gap a címke késését és a hétnapos horizont közbülső napjait vágja ki; a Northstar `label_available_at` szűrése ugyanezt a szerződést valósítja meg. Az expanding split az originig növekvő tanítóablakot, utána jövőbeli tesztnapot ad; a véletlen keverés idősoron szivárgást okoz. A három origin hibája nem statisztikai intervallum. Következő alkalmazásként a 20.3. alfejezetben a naiv és szezonális baseline mellé egyszerű, időhelyes feature-ökkel dolgozó előrejelző modelleket építünk.

20.3 Egyszerű előrejelző modellek és baseline

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: pandas, statsmodels

Előfeltétel: 20.2. alfejezet · Célkészség: Egyszerű előrejelző modellek és baseline · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

20.3.1 Cél és konkrét helyzet

A Northstar-projekt `orders.csv` fájlját napi rendelésszámmá aggregálva rövid távú napi előrejelzési feladatot kapunk. Ebben az alfejezetben nem a 7 napot meghaladó kézbesítés bináris célját, hanem egy idősoros döntési kérdést vizsgálunk: a következő 7 nap mindegyikére hány rendelés várható, ha csak a mai napig ismert adatot használhatjuk. A megfigyelési egység egy nap; a célváltozó az aznapi rendelésszám, mértékegysége darab. Az időbélyeg UTC, a gyakoriság 1 nap. A predikció pillanata az utolsó teljesen megfigyelt nap végén, a következő nap 00:00 UTC időpontja, a horizont pedig $h = 1, \dots, 7$ nap.

A labor egy ismert szerkezetű, determinisztikus seedű szintetikus napi sort használ, hogy a várható hiba nagyságrendjét is ellenőrizhessük. A feladat: építsük meg a naiv (persistence) és szezonális naiv baseline-t, illesszünk egy kis exponenciális simítási modellt, majd több időablakon, mindig csak a múltból számolva hasonlítsuk össze őket korrekt hibamértékkel. Az ARIMA-modellt áttekintés szintjén ismerjük meg; a teljes azonosítás és elmélet nem előfeltétel.

20.3.2 Autokorreláció kis példán

Az idősoros modellek alapötlete, hogy a mai érték hasonlíthat a tegnapihoz. Az **autokorreláció** (autocorrelation) a sorozat önmagával vett, k nappal eltolt lineáris kapcsolata. Az $y_1, \dots, y_T$ sorozat k rendű autokorrelációja:

$$r_k = \frac{\sum_{t=k+1}^{T} (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{T} (y_t - \bar{y})^2},$$

ahol y_t a t napi érték, $\bar{y}$ a mintaátlag, T a megfigyelések száma, k a késleltetés (lag), napokban. A nevező a teljes varianciaarányos skálázás, a számláló pedig a k nappal eltolt párok együttes ingadozása. Ha a szomszédos napok együtt mozognak, r_1 közel $+1$; ha váltakoznak, közel -1.

Kézi kontroll a $[10, 11, 12, 13, 14, 15]$ sorozaton: $\bar{y} = 12{,}5$, az eltérések $[-2{,}5, -1{,}5, -0{,}5, 0{,}5, 1{,}5, 2{,}5]$. A $k = 1$ számláló $3{,}75 + 0{,}75 - 0{,}25 + 0{,}75 + 3{,}75 = 8{,}75$, a nevező $17{,}5$, így $r_1 = 0{,}5$. A pozitív érték azt mutatja, hogy a monoton növekvő sorban a tegnapi érték információt hordoz a mairól; ez indokolja a naiv baseline-t.

20.3.3 Egyszerű modellek és jelölések

Naiv modell (persistence). A legjobb tipp a holnapi értékre a mai érték. Ha a predikció origója T, akkor minden $h \geq 1$ horizontra:

$$\hat{y}_{T+h|T} = y_T.$$

Itt $\hat{y}_{T+h|T}$ a T-ig ismert adatokból készült h lépéses előrejelzés; mértékegysége megegyezik y-nal. A modellnek nincs illesztendő paramétere, ezért kiváló baseline.

Szezonális naiv. Heti ciklusnál $m = 7$ nap; a holnapi értéket a múlt heti azonos napra adjuk. Általánosan, ha m a periódus hossza napokban:

$$\hat{y}_{T+h|T} = y_{T+h-m\lceil h/m \rceil},$$

ahol $\lceil \cdot \rceil$ felfelé kerekítés. Például $T = 7$, $m = 7$, $h = 1$ esetén y_1, $h = 7$ esetén y_7, $h = 8$ esetén ismét y_1. Gyakorlatban ez az utolsó m megfigyelés ciklikus ismétlése.

Exponenciális simítás. Az egyszerű exponenciális simítás (simple exponential smoothing, SES) egy szintet tart karban:

$$l_t = \alpha y_t + (1 - \alpha) l_{t-1}, \qquad \hat{y}_{T+h|T} = l_T.$$

A szint l_t mértékegysége megegyezik a célváltozóéval; α simítási paraméter, dimenziótlan, $0 \leq \alpha \leq 1$. Kis α simít, nagy α gyorsan követi a friss megfigyelést; az optimális α az illesztés során becsülhető. Szezonális adatra a Holt–Winters-változat trendet és periódusonkénti szezonális tagokat is becsül; a statsmodels `ExponentialSmoothing` osztálya ezt a `seasonal="add"` és `seasonal_periods=7` argumentumokkal adja.

ARIMA áttekintés. Az ARIMA(p, d, q) család általános keret: p autoregresszív tag a saját múltat használja, q mozgóátlag tag a múltbeli hibákat, d pedig az egyszerű differenciát $y'_t = y_t - y_{t-1}$ a trend eltávolítására. AR(1):

$$y_t = c + \phi_1 y_{t-1} + \varepsilon_t,$$

MA(1):

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1},$$

ahol c konstans, ϕ_1 és θ_1 becsült együtthatók, ε_t véletlen hibatag, naponként azonos szórású, nem autokorrelált. A teljes ARIMA-azonosítás meghaladja ezt az alfejezetet; a laborban az exponenciális simítás a választott kis statisztikai modell, az ARIMA pedig opcionális összehasonlítás.

Reziduális. Az illesztett modell in-sample hibája

$$e_t = y_t - \hat{y}_{t|t-1},$$

vagyis a tényleges érték és az egylépéses modell-előrejelzés különbsége. Jó modellnél a reziduális közelítőleg fehér zaj: nulla körüli átlag, közel konstans szórás és nincs jelentős autokorreláció. Autokorrelált maradék arra utal, hogy a modell kihagyott struktúrát; ilyenkor trend- vagy AR/MA-taggal érdemes bővíteni.

20.3.4 Teljes minimális munkapélda

A következő kód szintetikus napi sort készít, majd négy darab 7 napos tesztablakon, mindig csak az origó előtti adatra illesztve értékeli a naiv, a szezonális naiv és a Holt–Winters-modellt. A kimenet szemléltető; a tényleges futási számok a helyi környezettől és a seedtől függenek.

```
import numpy as np
import pandas as pd
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_absolute_error

rng = np.random.default_rng(20261020)
n = 168
t = np.arange(n)
trend = 0.08 * t
season = 4.0 * np.sin(2 * np.pi * t / 7)
noise = rng.normal(0, 1.8, size=n)
y = 30.0 + trend + season + noise
series = pd.Series(
    y, index=pd.date_range("2025-01-01", periods=n, freq="D")
)

def evaluate_origin(series, origin, horizon=7):
    train = series.iloc[:origin]
    test = series.iloc[origin:origin + horizon]

    # Naiv baseline: az utolsó tanítóérték ismétlése
    naive_pred = np.full(horizon, train.iloc[-1])

    # Szezonális naiv: az utolsó 7 nap ciklikus ismétlése
    seasonal_pred = np.tile(train.iloc[-7:].to_numpy(), (horizon // 7) + 1)[:horizon]

    # Kis statisztikai modell: additív Holt-Winters
    hw_model = ExponentialSmoothing(train, seasonal="add", seasonal_periods=7)
    hw_fit = hw_model.fit(optimized=True)
    hw_pred = hw_fit.forecast(horizon).to_numpy()

    naive_scale = np.mean(np.abs(np.diff(train.to_numpy())))
    result = {
        "naive_mae": mean_absolute_error(test, naive_pred),
        "seasonal_naive_mae": mean_absolute_error(test, seasonal_pred),
        "hw_mae": mean_absolute_error(test, hw_pred),
```

```
        "naive_mase": mean_absolute_error(test, naive_pred) / naive_scale,
        "hw_mase": mean_absolute_error(test, hw_pred) / naive_scale,
        "resid": train.to_numpy() - hw_fit.fittedvalues.to_numpy(),
    }
    return result

train_size = 140
horizon = 7
windows = []
for origin in range(train_size, n, horizon):
    windows.append((origin, evaluate_origin(series, origin, horizon)))

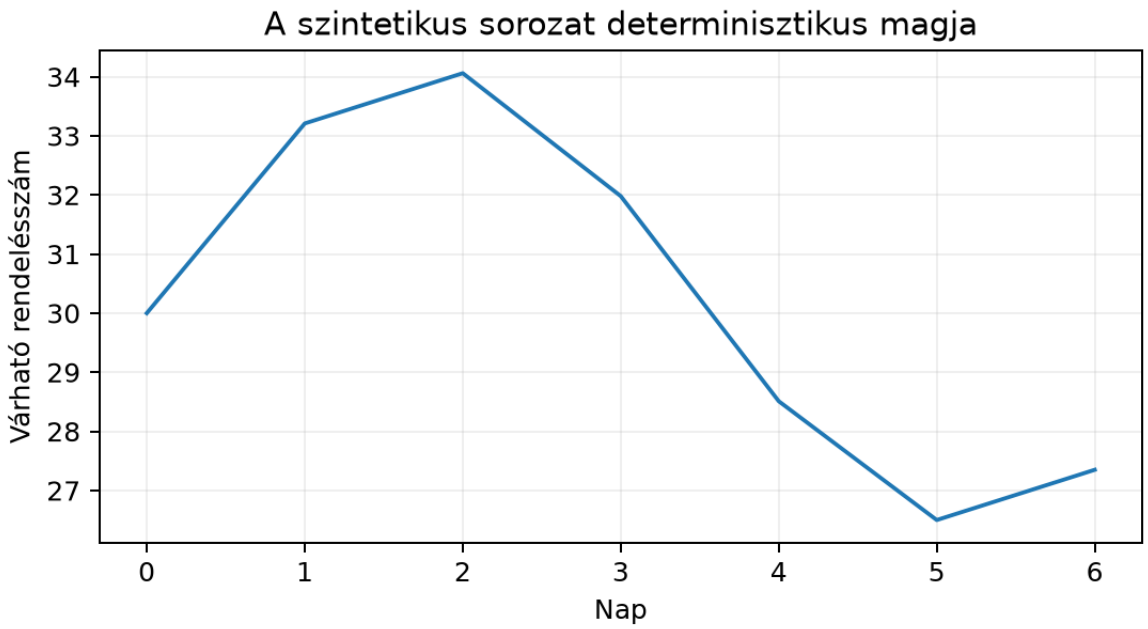
rows = []
for origin, r in windows:
    rows.append({
        "window_origin": origin,
        "naive_mae": r["naive_mae"],
        "seasonal_naive_mae": r["seasonal_naive_mae"],
        "hw_mae": r["hw_mae"],
        "naive_mase": r["naive_mase"],
        "hw_mase": r["hw_mase"],
    })
results = pd.DataFrame(rows)
print(results.round(2))
```

A reziduális diagnosztikát az utolsó ablak tanítórészére így kérhetjük le:

```
from statsmodels.stats.diagnostic import acorr_ljungbox

ljung = acorr_ljungbox(windows[-1][1]["resid"], lags=[7], return_df=True)
print(ljung)
```

A szemléltető futtatásban a 7 napos periódus a determinisztikus magban látható; az adatgeneráló képlet $30 + 0{,}08t + 4\sin(2\pi t/7) + \varepsilon_t$, ahol t napokban, az amplitúdó és a szint rendelésszámban értendő, ε_t pedig független normális zaj, szórása 1,8 rendelés.



20.2. ábra: A szintetikus sorozat determinisztikus magja - Számított függvénypélda, nem mérés; a szimuláció zajt is hozzáad.

20.3.5 Eredmény értelmezése

A **MAE** (mean absolute error) abszolút hibák átlaga a tesztablakban, rendelésszámban:

$$\mathrm{MAE} = \frac{1}{H}\sum_{h=1}^{H}\left|y_{T+h} - \hat{y}_{T+h|T}\right|,$$

ahol $H = 7$ a horizont hossza. A **MASE** (mean absolute scaled error) ezt a naiv baseline in-sample hibájához viszonyítja: a nevező $\frac{1}{T-1}\sum_{t=2}^{T}|y_t - y_{t-1}|$ a tanítóablak átlagos napi változása. A MASE dimenziótlan; 1-nél kisebb érték azt jelenti, hogy a modell jobb, mint a naiv, 1-nél nagyobb, hogy rosszabb. A MASE akkor használható érdemben, ha a nevező nem nulla; állandó sorozatnál nem értelmezhető.

Az elméletileg várt mintázat: a heti szezonalitás miatt a `seasonal_naive_mae` és a `hw_mae` kisebb, mint a `naive_mae`; a `hw_mase` tipikusan 1 alatt van, míg a naiv MASE 1 körül mozog, de a rövid tesztablakok miatt ablakonként ingadozik. A Holt–Winters-modell becsült simítási paraméterei az adott ablakra illeszkednek; a modell nem használhat jövőbeli tesztnapot. A Ljung–Box-teszt p-értéke 0,05 felett azt jelzi, hogy a reziduális autokorrelációja nem szignifikáns; ez a modell elfogadhatóságának egyik feltétele, nem önmagában bizonyíték a jó előrejelzésre.

Horizont és kiértékelés: a hétlépéses előrejelzésnél külön kell nevezni a direkt és a rekurzív stratégiát, valamint horizontonként összehasonlítani a baseline modelleket. A h=1 mérőszáma önmagában nem igazolja a h=7 előrejelzés használhatóságát.

20.3.5.1 Azonos horizontú összehasonlítás és naptári stressztesztt

A baseline-ok összehasonlításának szerződése: ugyanaz az origin T, ugyanazok a cél-dátumok $T+1, \ldots, T+7$, ugyanaz a mértékegység (rendelés/nap), és ugyanaz a hiányzási szabály. A **szezonális naiv(7)** előrejelzése

$$\hat{y}_{T+h|T} = y_{T+h-7}, \qquad h = 1, \ldots, 7,$$

tehát a múlt hét azonos hétköznapját ismétli. Ez nem ugyanaz, mint a persistencia ($\hat{y}_{T+h} = y_T$ minden h-ra): a persistencia a hét napját nem veszi figyelembe, a szezonális naiv viszont igen.

Egy hétre előre jelző mini példa (utolsó tanítóhét: $[10, 12, 11, 13, 12, 9, 8]$, teszt hét: $[11, 13, 12, 14, 13, 10, 9]$):

Forgatókönyv	Értékelhető napok	Szezonális naiv(7) MAE	Persistencia MAE
Normál hét	7	1,00	3,71
Ünnepnap: szerda tényleges értéke 0	7	2,43	4,29
Szerda adatgyűjtése kiesik	6	1,00	3,67

A naptári stressztesztt lényege, hogy a szezonális naiv a múlt hét mintáját ismétli; ha egy ünnepnap vagy egy kieső gyűjtés miatt a hét szerkezete eltér, a hiba nő. A hiányzó napot itt sem szabad nullának venni: a kieső szerdát mindkét modell értékeléséből ki kell hagyni, különben a MAE mesterségesen javulna vagy romlana. A stressztesztt eredményét a normál hét eredményével együtt kell jelenteni, nem csak a legjobb esetet.

20.3.6 Feladatok

1. **Értelmezés.** Legyen $m = 2$, a tanítóablak utolsó két értéke $[101, 103]$, a tesztablak pedig $[104, 106, 105, 107]$. Számítsd ki kézzel a naiv és a szezonális naiv előrejelzést mind a 4 napra, majd add meg a MAE értékét. Melyik baseline a jobb ebben a rövid ablakban?

2. **Megvalósítás.** Futtasd a fenti kódot, majd módosítsa a Holt–Winters-modellt `seasonal="mul"` értékre. Hasonlítsd össze a két változat ablakonkénti MAE-jét, és indokold, miért várható az additív változat előnye a használt generáló képlet mellett. A döntéshez ne használja a tesztablakot modellválasztásra; rögzítse előre a kritériumot.

3. **Hibadiagnózis.** Egy társ a naiv baseline-t így számolja: `pred = pd.Series(test).shift(1)`. Milyen hibát követ el a fix originű, 7 napos előrejelzésnél? Adj invariánst, amellyel a helyes naiv előrejelzés ellenőrizhető.

4. **Transzfer.** Növelje a zaj szórását az adatgeneráló kódban 1,8-ról 6,0-ra, és futtassa újra a négy ablakot. Hogyan változnak a MAE és a MASE értékek? Miért változik kevésbé a MASE, mint a MAE?

20.3.7 Megoldások és indoklás

1. A naiv előrejelzés mind a 4 napra 103. A szezonális naiv $m = 2$-vel az utolsó két értéket ismétli: $[101, 103, 101, 103]$. A naiv hibái: $|104-103|+|106-103|+|105-103|+|107-103| = 1+3+2+4 = 10$, MAE $= 10/4 = 2{,}5$. A szezonális naiv hibái: $|104-101|+|106-103|+|105-101|+|107-103| = 3+3+4+4 = 14$, MAE $= 14/4 = 3{,}5$. Ebben a rövid ablakban a naiv jobb; ez arra figyelmeztet, hogy a szezonális naiv nem automatikus nyertes.

2. A multiplikatív változat akkor indokolt, ha a szezonális amplitúdó a szinttel arányosan nő. A generáló képletben a szezonális tag additív konstans amplitúdójú, ezért az additív specifikáció közelebb van az igaz folyamathoz; a multiplikatív változat a kis szintingadozást is arányosítja, és rövid ablakon túlilleszthet. A helyes döntési eljárás előre rögzített, például az utolsó ablak kivételével az összes ablak átlagos MAE-je alapján választunk, a félretett ablak csak jelentésre szolgál.

3. A `shift(1)` minden tesztnapra az előző tényleges tesztértéket adja, ami egy rögzített T origóból nem áll rendelkezésre; ez jövőinformáció, és optimista hibát okozhat. A helyes naiv prediktor konstans: `np.full(horizon, train.iloc[-1])`. Invariáns: `assert (pred == train.iloc[-1]).all()` és `assert len(pred) == horizon`.

4. Nagyobb zajnál a tényleges értékek ingadozása nő, ezért a MAE minden modellnél nő. A MASE nevezője, a training átlagos napi változása is nő, mert a zaj dominál; így a hányados kevésbé változik. A zaj elnyomja a heti struktúrát, ezért a Holt–Winters és a naiv közötti MASE-különbség csökkenhet; a modell előnye kevésbé lesz megbízható.

20.3.8 Gyakori tévedés ellenpéldával

A leggyakoribb hiba, hogy a bonyolultabb modellt automatikusan jobbnak tekintjük a naiv baseline-nál. Ellenpélda: ha a generáló képletből elhagyjuk a szezonális tagot, tehát `season = 0.0`, akkor a heti ciklus megszűnik, és a szezonális naiv az utolsó hét zajos értékeit ismétli; a 7 napos teszt MAE-je várhatóan meghaladja a naiv modellét, a Holt–Winters pedig a becsült szezonális tagokkal ugyancsak túlilleszthet. Emlékeztető a korábbi fejezetekből: ahogyan 20.2. alfejezetben a jövőinformáció kizárása volt a feltétel, itt is minden ablakban csak az origó előtti

adat használható; a többszöri modellválasztás pedig a 16.3. alfejezetben tárgyalt peekinghez hasonló torzítást vihet az értékelésbe, ha nincs előre rögzített döntési szabály.

Önellenőrzés: a naiv prediktor minden eleme egyenlő; a forecast hossza egyenlő a horizonttal; a MASE nevező pozitív; a modellillesztés soha nem kapja meg a tesztablakot.

20.3.9 Rövid összefoglaló

A naiv modell a mai értéket ismétli, a szezonális naiv a múlt heti mintát, az exponenciális simítás pedig simított szintből és szezonális tagokból készít 7 napos előrejelzést. A több tesztablakos értékelésben a MAE rendelésszámban méri az abszolút hibát, a MASE a naiv baseline-hoz viszonyít: 1 alatt javulás, 1 felett romlás. A reziduális autokorreláció diagnosztika; ha jelentős, a modell struktúrát hagyott ki. Az ARIMA keretrendszer AR, MA és differencia tagokkal általánosít, de a bonyolultabb modell nem garantál javulást. A következő alkalmazás az V. ellenőrző pont, ahol ezt a baseline- és hibakeretet teljes modellértékelési állításrendszerré kell összekötni.

20.4 V. ellenőrző pont: modell és következtetés

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: pandas, statsmodels**
>
> Előfeltétel: 20.3. alfejezet · Célkészség: V. ellenőrző pont: modell és következtetés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

20.4.1 Cél és három különböző munkamenet

A logisztikai vezető ugyanarra a Northstar-adathalmazra három különböző kérdést tesz fel. A bemenet közös, de a megfigyelési egység, a cél és az érvényességi feltétel nem.

- **Q1 prediktív:** rendeléskor, csak a rendelés pillanatában ismert adatokból becsüld meg annak valószínűségét, hogy a kézbesítés a 7 napos ígéreten túl érkezik. Megfigyelési egység: egy rendelés. Elsődleges ellenőrzés: időbeli spliten mért AUC és kalibráció.
- **Q2 asszociációs/inferenciális:** a múltban már teljesült rendeléseknél mekkora a `distance_km` és a `delivery_days` közötti kapcsolat, és mekkora a becslés bizonytalansága? Megfigyelési egység: rendelés. Elsődleges ellenőrzés: regressziós koefficiens, standard error, konfidenciaintervallum.
- **Q3 oksági:** egy új üzenet növeli-e a 7 napos konverziót? Megfigyelési egység: randomizált ügyfél. Elsődleges ellenőrzés: ITT-hatásbecslés a randomizált assignment alapján, teljes 7 napos megfigyeléssel.

Induló feladat a továbbolvasás előtt: írd le, Q1-hez miért nem t-próba, Q2-höz miért nem AUC-optimalizálás, Q3-hoz miért nem múltbeli rendelésekből számított korreláció a megfelelő elsődleges eszköz.

20.4.2 A három cél mögötti fogalmak összekapcsolása

A **regresszió** itt a feltételes várható érték modellezése. A Q2 egyszerű lineáris modellje:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \sim (0, \sigma^2).$$

Jelölés: x_i a távolság km-ben (egydimenziós, n hosszú vektor), y_i a kézbesítési napok száma (pozitív egész, n hosszú vektor), β_0 a tengelymetszet napban, β_1 a meredekség nap/km egységben, ε_i az a hibatag, amelyet a modell nem magyaráz. A legkisebb négyzetek becslője:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

A **bizonytalanság** a becslő ismételt mintavételi szórásán keresztül jelenik meg:

$$\mathrm{SE}(\hat{\beta}_1) = \sqrt{\frac{\hat{\sigma}^2}{\sum_i (x_i - \bar{x})^2}}, \quad \hat{\sigma}^2 = \frac{1}{n-2}\sum_i \hat{\varepsilon}_i^2,$$

ahol $\hat{\varepsilon}_i = y_i - \hat{y}_i$ a reziduum, $\hat{y}_i$ az illesztett érték. A 95%-os konfidenciaintervallum $\hat{\beta}_1 \pm t_{n-2,0.975} \cdot \mathrm{SE}(\hat{\beta}_1)$. Ez az eljárás ismételt mintavételi lefedettségét írja; nem azt állítja, hogy a paraméter 95% valószínűséggel az aktuálisan kiszámolt intervallumban van. Ezt a különbséget a 16.4. alfejezet inferencia-anyaga és a 19.4. alfejezet bayesi frissítés már bevezette.

A prediktív cél ezzel szemben nem egy paraméter, hanem egy döntési valószínűség. Ha $\hat{p}_i$ a késés becsült valószínűsége és $y_i \in \{0, 1\}$ a tény, akkor a log loss:

$$L = -\frac{1}{n}\sum_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right],$$

ahol $\hat{p}_i$ skalár, y_i bináris, n a tesztrendelések száma. A **time split** a predikció pillanatát védi: a modellnek a jövőbeli rendelésekhez hasonló helyzetben kell döntenie, ezért a felosztás időben rendezett, nem véletlen.

Az oksági kérdéshez potenciális kimenetelek kellenek. Jelölje $Y_i(1)$ és $Y_i(0)$ az i ügyfél konverzióját kezeléssel és anélkül. A valódi átlagos kezelési hatás:

$$\Delta = \mathbb{E}[Y(1)] - \mathbb{E}[Y(0)].$$

Csak $Y_i^{obs} = T_i Y_i(1) + (1 - T_i) Y_i(0)$ figyelhető meg, ahol $T_i \in \{0, 1\}$ a randomizált kezelés. A randomizálás garantálja, hogy T_i független a potenciális kimenetelektől, ezért:

$$\mathbb{E}[Y^{obs} \mid T = 1] - \mathbb{E}[Y^{obs} \mid T = 0] = \Delta.$$

Ezért az ITT-elemzés nevezője minden randomizált ügyfél, nem csak a válaszolók. A **bayesi** alternatíva ezzel szemben prior és likelihood kombinációjából posterior eloszlást ad: $p(\theta \mid D) \propto p(D \mid \theta)\, p(\theta)$; bináris konverziónál például a Beta-Binomial modell posteriorja $\mathrm{Beta}(a+y, b+n-y)$, ahol y a sikerek, n a próbák száma, a, b prior pszeudo-megfigyelések. A credible interval ekkor közvetlen valószínűségi állítás a posteriorról, de a prior választás feltétele mellett.

20.4.3 Teljes minimális példa

A példa nem a teljes Northstar CSV-t használja, hanem 6 soros, kézzel ellenőrizhető tanuló DataFrame-et; a logika változatlan a 8.4. alfejezetben előállított rendelésszintű táblára. Mentsük `s080_checkpoint.py` néven, és futtassuk a lab projektgyökeréből: `uv run python s080_checkpoint.py` (Linux/macOS; Windows PowerShellben ugyanez, ha a `uv` a PATH-ban van). A példa determinisztikus, ezért seed nem szükséges; a valódi bootstrapnél a seedet a futásnapló rögzítené.

```
# s080_checkpoint.py - futtatás: uv run python s080_checkpoint.py
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf

# 1) Időbeli split label-availability szűréssel
orders = pd.DataFrame({
    "order_id": ["o1", "o2", "o3", "o4", "o5", "o6"],
    "order_ts": pd.to_datetime([
        "2025-04-28T10:00:00+00:00", "2025-04-29T10:00:00+00:00",
        "2025-04-30T10:00:00+00:00", "2025-05-01T10:00:00+00:00",
        "2025-05-02T10:00:00+00:00", "2025-05-03T10:00:00+00:00"]),
    "label_available_at": pd.to_datetime([
        "2025-04-30T10:00:00+00:00", "2025-05-03T10:00:00+00:00",
        "2025-05-02T10:00:00+00:00", "2025-05-05T10:00:00+00:00",
        "2025-05-05T10:00:00+00:00", "2025-05-07T10:00:00+00:00"]),
    "target_late": [0, 1, 0, 1, 0, 1],
})

cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train_mask = (orders["order_ts"] < cutoff) & (orders["label_available_at"] < cutoff)
test_mask = orders["order_ts"] >= cutoff
purged_mask = (orders["order_ts"] < cutoff) & (orders["label_available_at"] >= cutoff)

print("train:", orders.loc[train_mask, "order_id"].tolist())
print("test:", orders.loc[test_mask, "order_id"].tolist())
print("purged:", orders.loc[purged_mask, "order_id"].tolist())
assert train_mask.sum() + test_mask.sum() + purged_mask.sum() == len(orders)

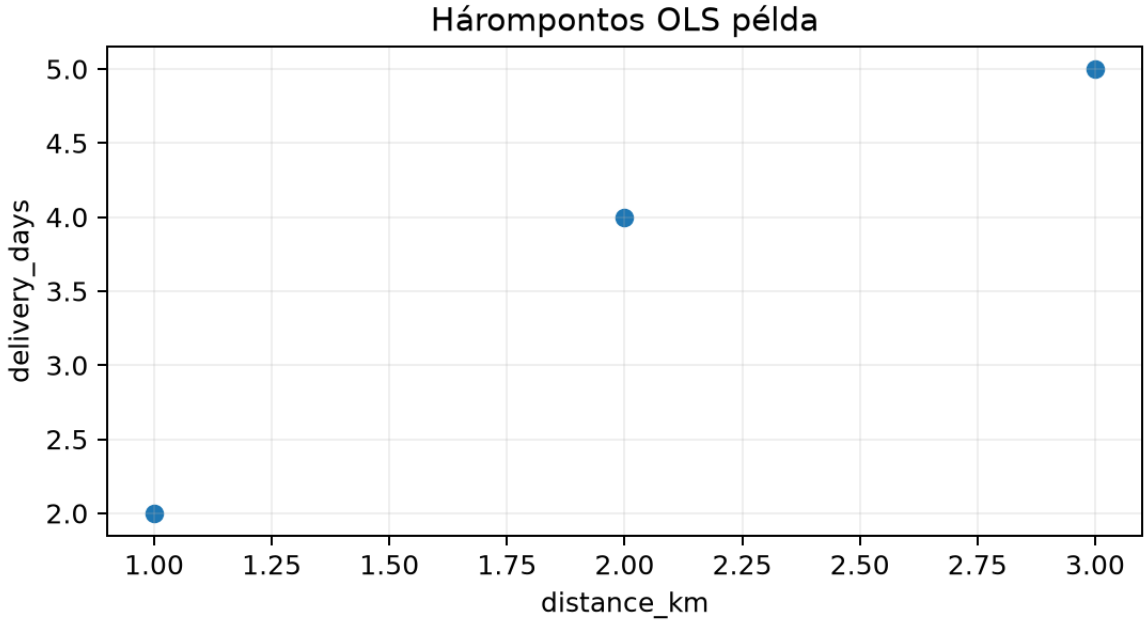
# 2) OLS három ponton, kézi kontrollal
ols_df = pd.DataFrame({"distance_km": [1, 2, 3], "delivery_days": [2, 4, 5]})
fit = smf.ols("delivery_days ~ distance_km", data=ols_df).fit()
print("params:", fit.params.round(4).to_dict())
print("bse:", fit.bse.round(4).to_dict())

x = ols_df["distance_km"].to_numpy(dtype=float)
y = ols_df["delivery_days"].to_numpy(dtype=float)
beta1 = np.sum((x - x.mean()) * (y - y.mean())) / np.sum((x - x.mean()) ** 2)
beta0 = y.mean() - beta1 * x.mean()
print("manual:", {"intercept": round(beta0, 4), "slope": round(beta1, 4)})

# 3) Randomizált kísérlet kézi aggregálása
assign = pd.DataFrame({
    "customer_id": ["c1", "c2", "c3", "c4", "c5", "c6"],
    "variant": ["control", "treatment", "control", "treatment", "control", "treatment"],
})
outcome = pd.DataFrame({
    "customer_id": ["c1", "c2", "c3", "c4", "c5", "c6"],
    "conversion_7d": [0, 1, 0, 1, 1, 1],
})
exp = outcome.merge(assign, on="customer_id", validate="one_to_one")
p_c = exp.loc[exp.variant == "control", "conversion_7d"].mean()
p_t = exp.loc[exp.variant == "treatment", "conversion_7d"].mean()
```

```
se = np.sqrt(p_t * (1 - p_t) / 3 + p_c * (1 - p_c) / 3)
print({"p_control": p_c, "p_treatment": p_t, "itt": p_t - p_c, "se": round(se, 3)})
```

A hárompontos regressziót szemlélteti:



20.3. ábra: Hárompontos OLS példa - Kézzel számított pontok; illesztett egyenes: y = 0.667 + 1.5x.

20.4.4 Eredmény értelmezése

A kód kimenete **elméletileg várt**, a kézi levezetéssel egyező; a könyvgeneráló ebben a módban nem futtatott Python-kódot, ezért a statisztikai csomag pontos képernyőformázását nem nevezzük mért eredménynek.

1. **Split:** a train halmaz `["o1", "o3"]`, a purged sáv `["o2"]`, a test `["o4", "o5", "o6"]`. Az o2 rendelést a cutoff előtt adták le, de a címkéje csak 2025-05-03-án vált ismertté, ezért nem kerülhet a tanítóba; ha bekerülne, a modell olyan címkét használna, amely a predikciós pillanatban nem állt rendelkezésre. Az assert ellenőrzi, hogy a három halmaz diszjunkt és teljes.
2. **OLS:** $\bar{x} = 2$, $\bar{y} = 11/3 \approx 3.667$. A meredekség számlálója $(-1)(-1.667) + (0)(0.333) + (1)(1.333) = 3$, a nevező 2, tehát $\hat{\beta}_1 = 1.5$ nap/km. A tengelymetszet $3.667 - 1.5 \cdot 2 = 0.667$ nap. Ez a kézi ellenőrzés. A reziduumok $-1/6, 1/3, -1/6$; $\hat{\sigma}^2 = 0.1667$. Ebből $\mathrm{SE}(\hat{\beta}_1) = \sqrt{0.1667/2} = 0.289$, a tengelymetszeté $\sqrt{0.1667(1/3 + 4/2)} = 0.624$; a statsmodels `params` és `bse` kimenete ezeket adja.
3. **Kísérlet:** control konverzió $1/3$, treatment $3/3 = 1$, ITT pontbecslés $2/3 \approx 0.667$; a normál közelítésű standard error $\sqrt{0 + (1/3 \cdot 2/3)/3} = 0.272$. Ez 6 ügyfélnél csak illusztráció: a $[0, 1]$ paramétertéren a Wald-féle intervallum felső vége 1 fölé menne, ezért a valódi Northstar-kísérletben a 16.4. alfejezet ügyfélszintű percentile bootstrapját használjuk. A `validate="one_to_one"` azt is ellenőrzi, hogy egy ügyfélhez pontosan egy kimenet tartozik; az ITT-nevező így 6 marad, nem szorzódik fel rendelésekre.

Záróellenőrzés: az előrejelzés, az oksági hatásbecslés és a leíró becslés eltérő érvényességi feltételeket kíván. A beadandóban minden eredményhez legyen megadva a megfigyelési egység, a becslési cél, a döntési időpont és az adatkiválasztás indoka.

20.4.5 Módszerválasztási jegyzőkönyv három új problémára

Probléma	Kérdéstípus	Adatfeltétel	Ellenőrzés és elvetett alternatíva
Q1: rendeléskor becsüld a késés valószínűségét	prediktív	csak rendeléskori feature; idősplit; label-availability szűrés	AUC és kalibráció a testen; elvetett: koefficiens-t-próba és véletlen split
Q2: múltbeli távolság–késés kapcsolat	asszociációs/inferenciális	teljesült rendelések, megfigyeléses adat	OLS CI és reziduáldiagnosztika; elvetett: oksági állítás és AUC-maximalizálás
Q3: üzenet hatása a konverzióra	oksági	randomizált assignment, teljes 7 napos kimenet	ITT-különbség és intervallum; elvetett: megfigyeléses propensity és csak válaszolók nevezője

A Q1 esetében a cél a jövőbeli rendeléseken elért döntési minőség. Ezért a teszt az `order_ts >= cutoff` halmaz, a tanító pedig csak olyan múltbeli rendeléseket tartalmazhat, amelyeknek a címkéje a cutoff előtt megérkezett. A pontbecslés helyett a kalibrált valószínűség számít, mert a logisztikai csapat küszöböt akar állítani.

A Q2 célja egy paraméter és annak bizonytalansága. A `distance_km` nem randomizált, ezért az együttható a modellben szereplő többi változóra feltételes asszociáció. A megfelelő ellenőrzés a standard error és a konfidenciaintervallum, nem pedig az, hogy a modell mennyire jól prediktálja a múltat. Azt is le kell ellenőrizni, hogy a reziduumokban nem marad-e nyilvánvaló struktúra.

A Q3 esetében a randomizálás teszi lehetővé a kauzális értelmezést. Az ITT az összes randomizált ügyfélre vonatkozik, függetlenül attól, hogy ki nyitotta meg az üzenetet. A csak válaszolókra szűkített elemzés szelekciós torzítást vinne be, ezért elvetjük.

20.4.5.1 Háromkérdéses elfogadási rubrika

A három eredmény nem rangsorolható egyetlen metrika szerint. Az alábbi rubrika külön oszlopban rögzíti a becslési célt, a mintaegységet, a független validációs állítást és az üzleti következményt:

Kérdés	Becslési cél	Mintaegység	Független validációs állítás	Üzleti következmény
Q1 prediktív	P(késés | rendeléskori adatok)	egy rendelés	időbeli holdout: AUC, kalibráció, Brier; a teszt címkéi a cutoff után válnak ismertté	küszöb a proaktív értesítéshez; a téves riasztás és a kihagyott késés költsége
Q2 asszociációs	feltételes együttható és konfidenciainter-valluma	egy teljesült rendelés	SE/CI és reziduáldiagnosztika; nem oksági állítás	a távolság szerepének megértése; nem beavatkozási döntés
Q3 oksági	ITT átlagos kezelési hatás	egy randomizált ügyfél	randomizált assignment, teljes 7 napos kimenet, előre rögzített alpha	bevezetés/elutasítás a teljes ügyfélkörre

A rubrika szándékosan **nem** közös skálát ad: a Q1 AUC-ja, a Q2 CI-je és a Q3 ITT-hatása más kérdésre, más populáción és más érvényességi feltétellel érvényes. Egy „legjobb modell" kiválasztása a három közül értelmetlen; a döntéshez mindig a kérdéshez tartozó oszlopot kell használni. A Q2 eredményéből nem következik beavatkozási hatás, a Q1 AUC-jából nem következik oksági hatás, és a Q3 ITT-je nem predikciós pontosság.

20.4.6 Feladatok

1. **Értelmezés.** A jegyzőkönyv Q2 problémájánál egy kolléga így fogalmaz: „minden további 100 km távolság 2,3 százalékponttal növeli a késési valószínűséget." Miért nem fogadható el ez a mondat a megadott adatfeltétel mellett? Nevezd meg a hiányzó azonosítási feltételt.
2. **Implementálás.** A fenti 6 soros `orders` DataFrame-mel futtasd a splitet, és add meg a train, test és purged halmazok `order_id` listáját. Ellenőrizd invariánsként, hogy a trainben minden `label_available_at < cutoff`, a testben minden `order_ts >= cutoff`, és a három halmaz uniója az összes sor.
3. **Hibadiagnózis.** Egy kolléga a Q1 modellnél véletlen `train_test_split`-et használ, és 0,92 teszt-AUC-t kap; az időhelyes split 0,78-at ad. Mi a mechanizmus, amely az első számot optimistává teszi? Adj egy konkrét javítási lépést a folyamatban.
4. **Megváltozott feltétel.** A Q3 kísérletet csak `segment == "business"` ügyfelekre futtatták. A stakeholder azt kérdezi: mekkora a hatás a consumer szegmensben? Mit válaszolsz, és milyen feltétellel lehetne a kérdést megválaszolni?

20.4.7 Megoldások és indoklás

1. A Q2 adatfeltétel megfigyeléses: a távolság nem randomizált, és a késési hajlam, a régió, a prioritás vagy a kosár összetétele egyszerre korrelálhat a távolsággal és a késéssel. Az együttható ezért feltételes asszociáció; a kauzális megfogalmazáshoz randomizálás vagy explicit zavaróváltozó-azonosítás kell. Helyes mondat például: „a modellben szereplő változókra feltételesen 100 km többlet távolsághoz 2,3 százalékponttal nagyobb átlagos késési arány társul a múltbeli rendeléseknél."

2. A split: train `["o1", "o3"]`, purged `["o2"]`, test `["o4", "o5", "o6"]`. Invariánsok: a trainben a `label_available_at` értékek `2025-04-30` és `2025-05-02`, mindkettő `< cutoff`; a test `order_ts` értékei mind `>= cutoff`; a három halmaz hossza $2 + 1 + 3 = 6$, és nincs átfedés. A purged sáv kihagyása azért kötelező, mert az o2 címkéje a cutoff pillanatában még nem ismert.
3. A véletlen split a jövőbeli rendeléseket is a tanítóba keveri. A modell így a cutoff után várható időbeli mintázatot is látja, például egy késési hullámot vagy szezonális közelséget, és a tesztben szereplő hasonló rendeléseken jobbnak tűnik. A 0,92 ezért a valós jövőbeli döntési helyzetre nézve optimista. Javítás: idő szerint rendezett felosztás rögzített cutofftal, purged sávval, és a modell, a küszöb, valamint a preprocessing kizárólag a train halmazból tanulva.
4. A becslés csak a business szegmensre érvényes. A consumer szegmensre való átvitel feltevés, nem bizonyíték. A kérdés megválaszolásához vagy új randomizálás kell a consumer ügyfeleken, vagy előre regisztrált, megfelelő erővel tervezett alcsoportelemzés; utólagos, többszörös összehasonlítással terhelt feltáró becslés nem ad megerősítő következtetést. A válasz tehát: „a jelen kísérletből consumer-hatást nem becslünk.”

20.4.8 Gyakori tévedés és önellenőrzés

A leggyakoribb csapda az, hogy egy jól illeszkedő modellt minden kérdésre válaszként kezelünk. Tegyük fel, hogy a késésre hajlamos ügyfelek nagyobb eséllyel választanak `priority_flag` jelölést. Ekkor a múltbeli rendeléseken a prioritás és a késés között pozitív asszociációt látunk, és a modell jó AUC-t érhet el. Ez nem jelenti azt, hogy a prioritás *okozza* a késést, és azt sem, hogy a prioritás megszüntetése csökkentené azt. Egy randomizált kísérletben a hatás akár nulla is lehet. Az AUC tehát predikciós minőséget mér; az oksági állításhoz design kell. A 16.4. alfejezet ugyanezt a hatást p-érték és CI oldalról, a 19.4. alfejezet posterior oldalról világította meg.

Önellenőrző kérdések: meg tudom-e mondani, miért más a konfidenciaintervallum és a credible interval állítása? A Q1 splitnél a purged sorok miért nem kerülhetnek a trainbe? A Q2 regressziós együtthatóját miért nem fogalmazhatom beavatkozási hatásként? A Q3 ITT-nevezőjében miért szerepel minden randomizált ügyfél?

20.4.9 Rövid összefoglaló

Az V. ellenőrző pont azt rögzíti, hogy ugyanaz az adathalmaz más kérdésre más validációt igényel. A rendeléskori késésbecslés prediktív feladat: időbeli split, label-availability szűrés, AUC és kalibráció. A távolság–késés kapcsolat inferenciális kérdés: OLS, standard error és konfidenciaintervallum, de csak asszociációs nyelven. Az új üzenet hatása oksági kérdés: randomizálás, ITT és a frequentista intervallumtól eltérő bayesi értelmezés. A módszerválasztó jegyzőkönyv mindhárom problémához adatfeltételt, ellenőrzést és elvetett alternatívát rögzít. Következő lépésként a 21.1. alfejezet ezt a logikát a teljes tanítás-validáció-teszt folyamatra ülteti át.

21 Prediktív gépi tanulás: adatszétválasztás és pipeline

21.1 Tanítás, validáció és érintetlen teszt

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn, pandas

Előfeltétel: 20.4. alfejezet · Célkészség: Tanítás, validáció és érintetlen teszt · Kimenet: működő kód és a labor ellenőrző tesztje.

21.1.1 Cél és konkrét helyzet

A Northstar késésbecslő projektben a modell feladata rendeléskor megbecsülni, hogy a 7 napos ígért kézbesítési időt túllépi-e a csomag. A célváltozó `target_late` bináris: értéke 1, ha `delivery_days > promised_days`, egyébként 0. A megfigyelési egység egy rendelés (`order_id`). A predikció pillanata az `order_ts`; a modell csak az ekkor ismert feature-öket használhatja, a címkét azonban csak a kézbesítés után ismerjük meg. A kezdőlabor modellépítési időpontja **2025-05-01T00:00:00+00:00**. Ebből készítünk train/test/purged felosztást, amelyet egy `split.json` manifestben rögzítünk, és ellenőrizzük, hogy minden tanítócímke ténylegesen megérkezett a modellépítés időpontjáig.

Induló feladat a futtatás előtt: a 2400 rendelés 2025 első 180 napjára esik. A cutoff az év 121. napjára esik. Ha a kézbesítés jellemzően néhány nap, körülbelül hány sor kerül a train, a purged és a test halmazba? A labgenerátor pontos eloszlását nem ismerjük, ezért becslésként a train és a test méretét az időeloszlás határozza meg, míg a purged halmaz mérete a cutoff előtt leadott, de akkor még nem ismert címkéjű rendelések számától függ; konkrét intervallum nem igazolható a generátor futtatása nélkül közötti sor várható. A pontos számot a helyi futtatás `split.json` fájlja adja meg.

21.1.2 Tanító, validációs, teszt és holdout

A prediktív modell építésekor három adathalmazt különböztetünk meg:

- **Tanítóhalmaz (training set):** ezen tanulja a modell a paramétereit. Itt minden bemeneti jellemző és a hozzá tartozó címke ismert a tanítás pillanatában.
- **Validációs halmaz (validation set):** modellválasztásra, például hiperparaméter-keresésre vagy early stoppingra használjuk. A 21.1. alfejezet kezdőpéldája egyetlen cutofftal dolgozik, ezért itt még nincs külön validációs részháló; a fogalmat azonban már most rögzítjük, mert a 23.3. alfejezet hiperparaméter-keresés épít rá.
- **Teszt-/kiértékelőhalmaz (test set):** csak a végső jelentéshez használjuk, miután minden modellválasztási döntést lezártunk.

A **holdout** olyan adathalmaz, amelyet a tanítás során információs értelemben nem érintünk: nem nézzük a címkéjét, nem választunk rajta küszöböt, és nem végezünk rajta feature-szelekciót. A teszt holdout célja a bevetési helyzet szimulálása: olyan rendeléseken mérni, amilyeneken a modell a jövőben dolgozik.

A **target availability** azt mondja meg, hogy a címke mikor válik megfigyelhetővé. A Northstar szerződésben `label_available_at = actual_delivery_at`: konzervatívan csak a teljes kézbesítés pillanatában tekintjük ismertnek a `target_late` értéket, még akkor is, ha egy késés korábban már sejthető lenne. Ez a label-availability szerződés része; más projektekben eltérő lehet, de akkor azt explicit dokumentálni kell.

A **group** fogalma akkor lép be, ha egy magasabb szintű egység több sort is tartalmaz. A Northstar `orders` táblájában egy ügyfélhez több rendelés is tartozhat. Ha a cél új ügyfelekre való általánosítás, akkor a split nem sorszintű, hanem ügyfélszintű kell legyen: ugyanaz az ügyfél ne kerüljön egyszerre trainbe és testbe. A kezdőlabor **időbeli splitet** használ, amely nem garantálja ezt a csoportszintű elkülönítést; ez explicit korlát, nem hiba, ha a bevetési kérdés időbeli előrejelzés.

A **horizon** a predikciós időtáv. Itt 7 nap: a modell a rendelés pillanatában ad becslést a 7 napos ígéret túllépésére. A cutoff kiválasztása ehhez igazodik: a tesztablak a cutoff utáni rendeléseket tartalmazza, és a tesztcímkék a generált adatban teljesen megfigyeltek, tehát a későbbi értékeléshez rendelkezésre állnak, de a modellépítés pillanatában még nem.

21.1.3 Feltételes információ, függetlenség és időrendi validáció

Legyen t_0 a modellépítés cutoffja, itt `2025-05-01T00:00+00:00`. A modell csak az I_{t_0} információs halmazra épülhet, vagyis arra, ami t_0-ban ténylegesen ismert. Egy i rendelésre jelölje

- $t_{\text{order},i}$ a rendelés időpontját (i index, `order_ts` oszlop),
- $t_{\text{label},i}$ a címke elérhetőségének időpontját (i index, `label_available_at` oszlop).

A három halmaz formálisan:

$$\begin{aligned} \text{train} &= \{\, i : \; t_{\text{order},i} < t_0 \text{ és } t_{\text{label},i} < t_0 \,\}, \\ \text{purged} &= \{\, i : \; t_{\text{order},i} < t_0 \text{ és } t_{\text{label},i} \geq t_0 \,\}, \\ \text{test} &= \{\, i : \; t_{\text{order},i} \geq t_0 \,\}. \end{aligned}$$

A feltételben a $<$ szigorú: a cutoff pillanatában éppen beérkező címkét nem tekintjük a múltban ismertnek. A purged sáv a cutoff előtti rendelések közül azok, amelyek kézbesítése csak a cutoff után történt meg; ezeket sem tanításra, sem tesztelésre nem használjuk. A test kizárólag jövőbeli rendeléseket tartalmaz, ezért a teljes múltra és a határsávra nézve is tiszta.

A **random split** akkor ad jól értelmezhető becslést az új, hasonló megfigyelésekre, ha a felosztás megfelel a célzott alkalmazási helyzetnek, és nincs kezeletlen csoport- vagy időfüggés. Az i.i.d. feltevés gyakori elegendő kiindulópont, de önmagában nem garantálja az érvényességet, ha például a mintavétel nem reprezentálja a bevetéskori populációt. A **csoportos split** a függetlenséget a csoport szintjén követeli meg: a sorok nem függetlenek, a csoportok viszont annak tekinthetők. Az **időbeli split** nem független mintákat feltételez, hanem időrendi validációt ad: a tanítóablak megelőzi a tesztablakot, így a nem-stacionaritás, a szezonális változás és a későbbi bevetés közelítő szimulációja lesz az értékelés. A példa tehát a 20.1. alfejezetben bevezetett időbeli gondolatot ülteti át modellértékelésbe.

21.1.4 Teljes minimális munkapélda

A következő kód a lab projektgyökérből futtatható, feltételezve, hogy a `data/orders.csv` a generált tiszta fájl. A kimenet **szemléltető**, mert automatikusan még nem ellenőrzött Python-kód; a tényleges számokat a saját lab futtatás adja.

```
import json
from pathlib import Path

import pandas as pd

# A CSV ISO timestampjei +00:00 offsettel rendelkeznek.
orders = pd.read_csv("data/orders.csv")
orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)
orders["label_available_at"] = pd.to_datetime(orders["label_available_at"], utc=True)

cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")

# A split három, egymást kizáró maszkja.
past_order = orders["order_ts"] < cutoff
label_available = orders["label_available_at"] < cutoff

train_mask = past_order & label_available
test_mask = orders["order_ts"] >= cutoff
purged_mask = past_order & ~label_available

# Invariáns ellenőrzések.
assert not (train_mask & test_mask).any(), "train és test átfed"
assert not (train_mask & purged_mask).any(), "train és purged átfed"
assert not (test_mask & purged_mask).any(), "test és purged átfed"
assert train_mask.sum() + test_mask.sum() + purged_mask.sum() == len(orders)

# A tanítócímke label-availability bizonyítása erre a futásra.
assert (orders.loc[train_mask, "label_available_at"] < cutoff).all(), (
    "tanítócímke nem érkezett meg a modellépítés időpontjáig"
)

manifest = {
    "split_id": "northstar-late-cutoff-2025-05-01",
    "cutoff_utc": cutoff.isoformat(),
    "split_rule": (
        "train: order_ts < cutoff AND label_available_at < cutoff; "
        "test: order_ts >= cutoff; "
        "purged: order_ts < cutoff AND label_available_at >= cutoff"
    ),
    "n_total": len(orders),
    "n_train": int(train_mask.sum()),
    "n_test": int(test_mask.sum()),
    "n_purged": int(purged_mask.sum()),
    "train_order_ids": orders.loc[train_mask, "order_id"].tolist(),
    "test_order_ids": orders.loc[test_mask, "order_id"].tolist(),
    "purged_order_ids": orders.loc[purged_mask, "order_id"].tolist(),
    "train_positive_rate": float(orders.loc[train_mask, "target_late"].mean()),
    "test_positive_rate": float(orders.loc[test_mask, "target_late"].mean()),
    "max_train_label_available_at": (
        orders.loc[train_mask, "label_available_at"].max().isoformat()
    ),
}

Path("artifacts").mkdir(exist_ok=True)
Path("artifacts/split.json").write_text(json.dumps(manifest, indent=2))

print(
```

```
    f"train={manifest['n_train']}, test={manifest['n_test']}, "
    f"purged={manifest['n_purged']}, "
    f"max_train_label={manifest['max_train_label_available_at']}"
)
```

A fájl `artifacts/split.json` néven keletkezik, és tartalmazza a sorazonosítókat, a cutoffot, valamint a felosztási szabály szöveges rögzítését. A manifest a 8.4. alfejezetben tanult adatszerződés gondolatát viszi tovább: a split nemcsak kód, hanem dokumentált, visszakereshető artefaktum.

21.1.5 Kézi kontroll és az eredmény értelmezése

A maszkokat ellenőrizzük egy hat soros, kézzel átlátható példán is. A cutoff legyen `2025-05-01T00:00+00:00`.

`order_id`	`order_ts`	`label_available_at`	besorolás
A	2025-04-20 10:00	2025-04-25 10:00	train
B	2025-04-28 10:00	2025-04-30 10:00	train
C	2025-04-30 10:00	2025-05-03 10:00	purged
D	2025-05-01 10:00	2025-05-08 10:00	test
E	2025-05-02 10:00	2025-05-09 10:00	test
F	2025-04-29 10:00	2025-05-01 10:00	purged

A kézi számolás szerint train = 2, purged = 2, test = 2. A `C` sor rendelése múltbeli, de a címke jövőbeli, ezért purged. Az `F` sor címkéje a cutoff után érkezik, ezért szintén purged, bár a rendelés még korábbi. Ez a példa mutatja, hogy a label-availability szűrés nem helyettesíthető az `order_ts` szerinti vágással.

A kód által kiírt `max_train_label_available_at` értékének szigorúan kisebbnek kell lennie a cutoffnál; ez a bizonyíték arra, hogy a tanítóhalmazban nincs a modellépítéskor még nem ismert címke. A manifestben a `train_positive_rate` és `test_positive_rate` a pozitív osztály arányát mutatja; a teszt értékét csak a végső jelentésben szabad megnézni, mert a tesztcímke a modellépítéskor tiltott információ.

21.1.5.1 Purge és embargo: két külön határ

A **purge** a címke átnyúlását szünteti meg: minden olyan tanítósor kimarad, amelynek `label_available_at` értéke a cutoff után van. Az **embargo** a feature-ablak átnyúlását kezeli: ha egy feature számítása például 7 napos visszatekintő ablakot használ, a cutoff előtti utolsó napok ablaka már a tesztidőszakba nyúlhat, ezért ezeket a sorokat a tanításból ki kell zárni. A két szabály nem helyettesíti egymást:

Határ	Mit zár ki?	Mikor kell?
Purge	a cutoff előtti rendelés, amelynek címkéje csak a cutoff után ismert	minden időbeli splitnél, ha a címke késleltetett
Embargo	a cutoff előtti sor, amelynek feature-ablaka átnyúlik a tesztablakba	ha a feature visszatekintő ablakot használ, és a split határán a hatás átvihető

A Northstar kezdőpéldájában a feature-ök a rendelés pillanatában ismertek, ezért az embargo nem szükséges; a purge igen, mert a `label_available_at` késleltetett. Ha később gördülő ablakos feature-t vezetünk be, a split kódját embargóval kell kiegészíteni.

21.1.6 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az, hogy a splitet kizárólag a feature-időre alapozzuk:

```
train = orders[orders["order_ts"] < cutoff]
test = orders[orders["order_ts"] >= cutoff]
```

Tegyük fel, hogy egy rendelés `order_ts = 2025-04-29T10:00+00:00`, ígért kézbesítése 7 nap, tényleges kézbesítése `2025-05-03`, ezért `label_available_at = 2025-05-03`. A fenti kód trainbe teszi, mert a rendelés időpontja múltbeli. A modellépítés pillanatában azonban a `target_late` érték még nem létezik: a címke jövőbeli információ. Ha a modell ezen tanul, akkor olyan célváltozót lát, amelyet a bevetéskor soha nem kapna meg időben; ez **címke-szivárgás**. A jövőbeli label elérhetősége tehát akkor is számít, ha a feature-ök mindegyike a rendelés pillanatában ismert. A javítás a `label_available_at < cutoff` feltétel hozzáadása, a purged sáv kizárásával.

Visszautalás: a 9.1. alfejezetben bevezetett formális jelölés itt válik döntési eszközzé; az I_{t_0} információs halmaz és a `<` szigorúsága ugyanaz a gondolat, mint a 20.1. alfejezet időbeli validációs követelménye.

21.1.7 Feladatok

1. **Értelmezés.** A purged sávba eső `C` rendelést miért nem tesszük a testbe? Magyarázd el, milyen torzítást okozna, ha mégis odakerülne.
2. **Implementálás.** Írj `make_time_split(df, cutoff)` függvényt, amely egy rendeléseket tartalmazó DataFrame-ből és egy cutoff timestampből kiindulva visszaadja a `train_mask`, `test_mask`, `purged_mask` boolean tömböket, valamint a manifest dictionary-t a fenti szerkezetben. Teszteld a hat soros kézi példán, és ellenőrizd, hogy a visszaadott maszkok összege 6.
3. **Hibakeresés.** Az alábbi split a label-availability szűrést kihagyja:

   ```
   train = orders[orders["order_ts"] < cutoff]
   test = orders[orders["order_ts"] >= cutoff]
   ```

   Adj meg egy konkrét `order_id`-t a fenti hat soros példából, amelyik így trainbe kerül, pedig a címkéje a cutoffkor még nem ismert. Melyik feltétel hiányzik a javításhoz?
4. **Megváltozott feltétel.** Változtasd a cutoffot `2025-04-15T00:00:00+00:00`-re. A train, test és purged halmaz mérete hogyan változik a `2025-05-01`-eshez képest? Indokold a label-availability feltétel és a rövidebb múlt alapján.

21.1.8 Megoldások és indoklás

1. A purged sávba eső sor `order_ts < cutoff`, tehát a modellépítés pillanatában a rendelés már megtörtént. Ha testbe tennénk, a teszt nem kizárólag jövőbeli rendeléseket tartalmazna, hanem olyan múltbelieket is, amelyek címkéje a cutoff után érkezett. Ez kettős torzítást okoz: egyrészt a teszteloszlás keveredik a határsávval, másrészt a modell olyan múltbeli eseteken lenne értékelve, amelyekre a bevetéskor nem lenne analógiája. A purged sáv ezért sem tanításra, sem értékelésre nem használható; a kihagyott határsáv a cutoff közeli szivárgás elleni konzervatív védelem.

2. A függvény vázlata:

```
def make_time_split(df, cutoff):
    past_order = df["order_ts"] < cutoff
    label_available = df["label_available_at"] < cutoff
    train_mask = past_order & label_available
    test_mask = df["order_ts"] >= cutoff
    purged_mask = past_order & ~label_available
    manifest = {
        "cutoff_utc": cutoff.isoformat(),
        "n_total": len(df),
        "n_train": int(train_mask.sum()),
        "n_test": int(test_mask.sum()),
        "n_purged": int(purged_mask.sum()),
    }
    return train_mask, test_mask, purged_mask, manifest
```

   A hat soros példán `n_total = 6`, `n_train = 2`, `n_test = 2`, `n_purged = 2`, tehát az összeg 6. Az ellenőrzéshez a maszkokra `sum()` hívható, mert a `True` értéke 1.

3. A `C` sor `order_ts = 2025-04-30 10:00`, ami múltbeli, ezért az `order_ts < cutoff` feltétel trainbe sorolja. A `label_available_at = 2025-05-03 10:00` azonban a cutoff után érkezik, tehát a címke jövőbeli. A hiányzó feltétel: `label_available_at < cutoff`. A javított train feltétel: `(orders["order_ts"] < cutoff) & (orders["label_available_at"] < cutoff)`.

4. A `2025-04-15` cutoff rövidebb múltat hagy a trainnek, ezért a train mérete csökken, a test mérete nő. A purged sáv aránya várhatóan nő, mert a cutoff előtti rendelések közül nagyobb hányad kézbesítése esik a cutoff utánra: a határsáv ugyanakkora kézbesítési időablakot fed le, de a rendelkezésre álló múlt rövidebb, ezért a purged/train arány romlik. A pontos számot a lab futtatása adja; a tendencia a fenti időrendi szerkezetből következik.

21.1.9 Rövid összefoglaló

A modellértékelés első lépése olyan splitet készíteni, amely a bevetési helyzetet modellezi. A Northstar késésbecslő feladatban a cutoff `2025-05-01`, a train feltétele `order_ts < cutoff` és `label_available_at < cutoff`, a teszt a cutoff utáni rendeléseket tartalmazza, a purged sáv a múltbeli, de még nem kézbesített rendeléseket zárja ki. A splitmanifest rögzíti a sorazonosítókat és a szabályt; az invariant ellenőrzés bizonyítja, hogy minden tanítócímke megérkezett a modellépítés pillanatáig. A következő alkalmazás 21.2. alfejezetben ezt a manifestet használja, hogy a preprocessing minden lépése kizárólag a train halmazon tanuljon.

21.2 Preprocessing tanulás kizárólag a training részen

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn, pandas

Előfeltétel: 21.1. alfejezet · Célkészség: Preprocessing tanulás kizárólag a training részen · Kimenet: működő kód és a labor ellenőrző tesztje.

A prediktív gépi tanulásban a modell mellett a transzformációs lánc is tanul az adatból: hiánypótló átlagokat, standardizáló paramétereket, kategóriakódolási oszlopokat kell illeszteni. Ha ezeket a teljes adaton illesztjük, akkor a tesztadatokból származó információ beleszivárog a tanítási folyamatba, és az értékelés optimistává válik. Ez az alfejezet a scikit-learn `Pipeline` és `ColumnTransformer` segítségével mutatja be, hogyan kell a transzformációk illesztését a tanító részen belül tartani.

21.2.1 Cél és konkrét helyzet

A mesterséges feladat a Northstar-példa egyszerűsített változata: rendelésenként egy sorunk van, a cél pedig `target_late`, azaz hogy a kézbesítés 7 napon túli-e. A megfigyelési egység tehát a rendelés. A rendelkezésre álló jellemzők: `distance_km` (szállítási távolság kilométerben, numerikus, hiányzó értékekkel), `basket_minor` (kosárérték eurócentben, numerikus, hiányzó értékekkel), `segment` (ügyfélszegmens, kategorikus), `priority_flag` (elsőbbségi jelzés, 0/1).

A feladat: egy tanító és egy teszt részt rögzítünk, előfeldolgozó láncot építünk, majd egy logisztikus regressziót tanítunk. Ugyanezt egy szándékosan szivárgó változatban is végrehajtjuk, amelyben a transzformációkat a teljes adaton illesztjük. Mielőtt tovább olvasna, válaszoljon az induló kérdésre: szabad-e az imputáló átlagot a `train_test_split` előtt kiszámolni? A választ az alfejezet végén ellenőrizheti.

21.2.2 Fogalmi magyarázat

A scikit-learn transzformátorokban a **fit** (illesztés) tanulja meg a transzformáció paramétereit, a **transform** (átalakítás) pedig alkalmazza azokat. A tanító részen a kettőt egyszerre végzi a `fit_transform`, a teszt részen ellenben csak `transform`-ot hívunk, hogy a teszt statisztikái ne épüljenek be a folyamatba.

A **SimpleImputer** (egyszerű hiánypótló) `strategy="mean"` beállítással oszloponként az átlagot tanulja meg a `fit` során. Ha a `j`-edik oszlop megfigyelt értékei $x_{1j}, x_{2j}, \dots, x_{kj}$, akkor

$$\mu_j = \frac{1}{k} \sum_{i=1}^{k} x_{ij}.$$

Az átmeneti statisztika így $1 \times p$ alakú, ahol p az oszlopok száma. A `transform` a hiányzó cellákat a megfelelő μ_j-vel pótolja, a nem hiányzókat változatlanul hagyja. Fontos: a teszt részen is ugyanezt a μ_j-t használjuk, nem a teszt oszlopátlagát.

A **StandardScaler** (standardizáló) szintén oszloponként tanul átlagot μ_j-t és szórást σ_j-t,

$$\sigma_j = \sqrt{\frac{1}{k} \sum_{i=1}^{k} (x_{ij} - \mu_j)^2},$$

majd minden értéket z-értékké alakít:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

Az $n \times p$ méretű bemeneti mátrixból ugyanilyen alakú kimeneti mátrix lesz; a tanult statisztika mérete $1 \times p$. A tesztet ezzel a tanított μ_j-vel és σ_j-vel transzformáljuk, nem a teszt saját statisztikáival. Ezt a korábbi lineáris algebrai alakintuíciót használjuk: minden soron ugyanazt az oszloponkénti műveletet hajtjuk végre.

A **OneHotEncoder** (egy forró kódoló) a kategorikus oszlop értékkészletéből indikátoroszlopokat készít. Ha a tanító részben a `segment` oszlop értékei `consumer` és `business`, akkor a fit megtanulja ezt a két kategóriát, a transform pedig minden sorból két oszlopot hoz létre: az egyik 1, ha az érték `consumer`, a másik 1, ha `business`. A teszt részben megjelenő, a tanításban nem látott kategória kezelését a `handle_unknown` paraméter szabályozza. Az alapértelmezett `"error"` hibát dob, a `"ignore"` pedig a tesztben ismeretlen kategóriához csupa 0 értékű sort rendel. Ez a viselkedés tükrözi a termelési helyzetet: új kategóriához nincs tanult oszlop.

Kézi kontrollként vegyünk egy tanító oszlopot: $[2, 4, \text{NaN}, 8]$. A nem hiányzó értékek átlaga $(2 + 4 + 8)/3 = 14/3 \approx 4.6667$. A pótlás után $[2, 4, 4.6667, 8]$; ennek átlaga 4.6667, szórása $\sqrt{\left((-2.6667)^2 + (-0.6667)^2 + 0^2 + 3.3333^2\right)/4} \approx 2.1602$. A standardizált értékek körülbelül $[-1.2344, -0.3086, 0, 1.543]$. Ha a tesztben a 10 érték szerepel, akkor a tanított paraméterekkel számolt z-érték $(10 - 4.6667)/2.1602 \approx 2.469$.

A helyes folyamatot a `Pipeline` és a `ColumnTransformer` együtt valósítja meg. A `ColumnTransformer` oszlopcsoportonként rendeli hozzá a transzformátorokat, a `Pipeline` pedig egymás után fűzi a lépéseket úgy, hogy a tanítás során a fit csak a tanító sorokon fut.

21.2.3 Teljes minimális munkapélda

A következő kód a lab virtuális környezetében futtatható. A lab projektgyökérből Linux/macOS alatt `uv run python s082_demo.py`, PowerShell alatt `uv run python .\s082_demo.py` parancsot használunk. A kódot scikit-learn 1.9.1, numpy 2.5.4 és pandas 3.1.0 alatt futtattuk (seed 42); a kimenet a lenti, ellenőrzött értékeket adja.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

rng = np.random.default_rng(42)
n = 200

true_distance = rng.normal(12, 5, n)
true_basket = rng.lognormal(5, 0.7, n)
segment = rng.choice(
    ["consumer", "business", "vip_only_test"],
    size=n, p=[0.6, 0.35, 0.05],
)
priority = rng.binomial(1, 0.25, n)
logit = -2.5 + 0.18 * true_distance + 0.9 * (segment == "business") + 0.8 * priority
prob = 1 / (1 + np.exp(-logit))
target = rng.binomial(1, prob)
```

```
df = pd.DataFrame({
    "distance_km": true_distance,
    "basket_minor": true_basket.round(0),
    "segment": segment,
    "priority_flag": priority,
    "target_late": target,
})

# hiányzó értékek bevezetése
df.loc[rng.choice(n, 12, replace=False), "distance_km"] = np.nan
df.loc[rng.choice(n, 8, replace=False), "basket_minor"] = np.nan

features = ["distance_km", "basket_minor", "segment", "priority_flag"]

# A vip_only_test kategória csak a tesztben jelenhet meg.
mask_vip = df["segment"] == "vip_only_test"
df_vip = df.loc[mask_vip]
df_rest = df.loc[~mask_vip]

X_rest, X_vip = df_rest[features], df_vip[features]
y_rest, y_vip = df_rest["target_late"], df_vip["target_late"]

X_train, X_test_rest, y_train, y_test_rest = train_test_split(
    X_rest, y_rest, test_size=0.2, random_state=7
)
X_test = pd.concat([X_test_rest, X_vip])
y_test = pd.concat([y_test_rest, y_vip])

# Helyes előfeldolgozó: csak a train részen illesztve, a Pipeline-on belül.
preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="mean")),
            ("scaler", StandardScaler()),
        ]), ["distance_km", "basket_minor"]),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), ["segment"]),
        ("flag", "passthrough", ["priority_flag"]),
    ]
)

model = Pipeline([
    ("preprocess", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000)),
])

model.fit(X_train, y_train)
auc_correct = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
print(f"Correct test AUC: {auc_correct:.3f}")

# Szivárgó változat: a transzformáció fit a teljes X-en.
preprocessor_bad = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="mean")),
            ("scaler", StandardScaler()),
        ]), ["distance_km", "basket_minor"]),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), ["segment"]),
        ("flag", "passthrough", ["priority_flag"]),
    ]
)

X_all_transformed = preprocessor_bad.fit_transform(df[features])
X_all_transformed = pd.DataFrame(X_all_transformed, index=df.index)
```

```
X_train_bad = X_all_transformed.loc[X_train.index]
X_test_bad = X_all_transformed.loc[X_test.index]

model_bad = LogisticRegression(max_iter=1000).fit(X_train_bad, y_train)
auc_bad = roc_auc_score(y_test, model_bad.predict_proba(X_test_bad)[:, 1])
print(f"Leaky test AUC: {auc_bad:.3f}")

# Ellenőrzés: a helyes imputer statisztikája valóban a train oszlopátlagait adja.
correct_num = model.named_steps["preprocess"].named_transformers_["num"]
print("Imputer statistics (train):", correct_num.named_steps["imputer"].statistics_)
print("Full-data distance means:", df["distance_km"].mean(),
      "train means:", X_train["distance_km"].mean())
```

Az ellenőrzött futás kimenete:

```
Correct test AUC: 0.788
Leaky test AUC: 0.788
Imputer statistics (train): [11.492  192.315]
Full-data distance means: 11.768  195.016
train means: 11.492  192.315
```

Ebben a futásban a két AUC azonos. Ez nem azt jelenti, hogy a szivárgás ártalmatlan: a `StandardScaler` affin transzformáció, és a logisztikus regresszió a skálázást részben kompenzálja, ezért a rangsor nem változott. A tanult **statisztikák** viszont különböznek: a helyes imputer a train átlagait (`11.492`, `192.315`) tanulta, a szivárgó a teljes adat átlagait (`11.768`, `195.016`). A statisztika-egyezés az invariáns, nem az AUC-különbség; más modelleknél (regularizáció, KNN, feature-szelekció) a szivárgás a döntést is megváltoztathatja.

21.2.4 Eredmény értelmezése

A helyes pipeline három lépésben tanul:

1. A `Pipeline` a train részen meghívja a preprocessor `fit_transform`-ját: a numerikus rész imputáló átlagot és standardizáló paramétereket tanul, a kategorikus rész a train kategóriáit rögzíti.
2. Ugyanez a preprocessor a teszt részen csak `transform`-ot kap: a tanult μ_j, σ_j és kategória-oszlopok alkalmazódnak.
3. A `LogisticRegression` a transzformált train mátrixon tanul, a predikció pedig a transzformált teszt mátrixon történik.

A szivárgó változatban a preprocessor fitje a teljes `df[features]`-re futott. Ennek következménye: a tesztben szereplő `vip_only_test` kategória a fit során már ismertté vált, ezért a kódoló külön oszlopot kapott rá. A helyes folyamatban ez az oszlop nem létezik, és a `handle_unknown="ignore"` miatt a tesztben az összes kategóriaoszlop értéke 0 lesz a vip sorokban. A szivárgó változat tehát olyan feature-t használ, amely a valódi produkcióban nem állna rendelkezésre.

Az ellenőrző invariant: a helyes preprocessor által tanult numerikus statisztikák (az imputer `statistics_` attribútuma) pontosan megegyeznek a train oszlopátlagaival, nem a teljes adat oszlopátlagaival. A transzformált train oszlopok számát a train kategóriák határozzák meg; a teszt oszlopszáma ugyanaz, akkor is, ha új kategória érkezik. Ha a kódban a `handle_unknown` értéke `"error"`, az ismeretlen kategória `ValueError`-t okoz, ami segít felfedni az adatszerződés-sértést.

A példa AUC-ja ebben a futásban nem jelezte a szivárgást, mert a skálázás affin és a logisztikus modell kompenzálja; a szivárgás ettől még valós, és a tanult statisztikák eltérésében látható.

A megbízható értékeléshez a 21.1. alfejezetben bevezetett elvet kell alkalmazni: a teszt részből semmilyen statisztika nem származhat, és ezt nem a metrika, hanem a transzformátorok `statistics_`/`mean_` értékeinek ellenőrzésével kell igazolni.

A `ColumnTransformer`-be kerülő összes adatfüggő transzformációt a CV-foldok tanítórészén kell illeszteni. A tesztbe átlógó kategóriakészlet, imputációs statisztika vagy feature-szelekció ugyanúgy szivárgás, mint a célváltozó átvétele.

21.2.5 Feladatok

1. **Értelmezés kézi számítással.** Adott egy tanító oszlop `[2, 4, NaN, 8]` és egy teszt érték `10`. Számítsd ki a tanító oszlop átlagát, pótolja a hiányzó értéket, majd a tanult átlaggal és szórással standardizálja mind a négy tanító értéket és a teszt értéket. Ellenőrizd, hogy a standardizált tanító értékek átlaga 0, szórása 1.

2. **Implementálás pipeline-nal.** Módosítsd a fenti minimális példát úgy, hogy a numerikus részben a medián imputálást használja átlag helyett (`strategy="median"`). Futtasd le, majd a `model.named_steps["preprocess"].named_transformers_["num"].named_steps["imputer"]` értékét hasonlítsa össze a train oszlopok mediánjaival. Ellenőrizd a transzformált train és teszt oszlopszámát.

3. **Hibadiagnózis.** Valaki a következő kódot írta:

```
X = df[features].copy()
X["distance_km"] = X["distance_km"].fillna(X["distance_km"].mean())
X_train, X_test, y_train, y_test = train_test_split(
    X, df["target_late"], test_size=0.2, random_state=7
)
model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
```

Magyarázd el, hol szivárog az információ, és írd át a kódot úgy, hogy a hiánypótló átlag kizárólag a tanító részből származzon.

4. **Megváltozott feltétel.** Távolítsa el a train részből az összes `business` kategóriájú sort, hogy a `business` a tesztben unseen kategóriaként jelenjen meg. Futtasd a pipeline-t `handle_unknown="error"` beállítással, majd `"ignore"` beállítással. Dokumentálja a hibát, és ellenőrizze az ignore módban kapott predikciók shape-jét.

5. **Transzfer döntési helyzetre.** A termelési adagban a `distance_km` oszlop mérföldben érkezik, nem kilométerben. Magyarázd el, miért nem elég a meglévő pipeline `transform`-ját lefuttatni; mi a helyes műszaki döntés?

21.2.6 Megoldások és indoklás

1. megoldás. Az átlag $(2+4+8)/3 = 4.6667$. A pótlás után a tanító értékek $[2, 4, 4.6667, 8]$. Az eltérésnégyzetek: $2.6667^2 + 0.6667^2 + 0 + 3.3333^2 \approx 18.6667$; a szórás $\sqrt{18.6667/4} \approx 2.1602$. A standardizált tanító értékek:

$$\frac{2-4.6667}{2.1602} \approx -1.2344, \quad \frac{4-4.6667}{2.1602} \approx -0.3086, \quad 0, \quad \frac{8-4.6667}{2.1602} \approx 1.543.$$

Ezek összege körülbelül 0, átlaguk 0, szórásuk 1. A teszt érték: $(10-4.6667)/2.1602 \approx 2.469$. Az invariant tehát teljesül.

2. megoldás. A medián imputálóval a helyes statisztika a train oszlopok mediánja. A következő ellenőrzés futtatható:

```
num_pipeline = model.named_steps["preprocess"].named_transformers_["num"]
print(num_pipeline.named_steps["imputer"].statistics_)
print(X_train[["distance_km", "basket_minor"]].median().to_numpy())
```

A két tömbnek egyeznie kell. A transzformált train és teszt oszlopszáma egyezik; a kategorikus rész oszlopszámát a train kategóriái adják. A `pd.DataFrame(model.named_steps["preprocess"].transform(X_tr` és a teszt transzformált shape-jének második dimenziója megegyezik.

3. megoldás. A hiba az, hogy a `fillna` a teljes oszlop átlagát használja, tehát a teszt sorok is beleszámítanak μ-ba. A javított változat:

```
X_train, X_test, y_train, y_test = train_test_split(
    df[features], df["target_late"], test_size=0.2, random_state=7
)
imputer = SimpleImputer(strategy="mean")
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)
```

Itt az imputer fitje csak a train sorokra vonatkozik, a teszt pedig ugyanazt a tanult átlagot kapja.

4. megoldás. Az `"error"` beállításnál a pipeline `transform` hívása a teszt `business` sorain `ValueError`-t dob, mert a kategóriát nem tanulta a fit. Az `"ignore"` beállításnál a transzformált teszt minden kategóriaoszlopában 0 lesz a `business` sorokra. Az oszlopszám változatlan. A predikció shape-je `(n_test, 1)` a `predict_proba` esetén második oszloppal, illetve `(n_test,)` a `predict` esetén.

5. megoldás. A meglévő pipeline a train átlagát és szórását kilométerben tanulta. Ha mérföldben érkező tesztet transzformálunk, a z-értékek félrevezetőek: a teszt numerikus értékeit a kilométeres statisztikák skáláján értelmezzük, ami a modell predikcióját torzítja. A helyes döntés: a betöltési rétegben konvertáljuk az értékeket kilométerre, vagy tanítsuk újra a pipeline-t a megváltozott mértékegységhez rögzített új szerződéssel. A pipeline önmagában nem tudja felismerni a mértékegységváltást.

21.2.7 Gyakori tévedés és önellenőrzés

Gyakori tévedés, hogy a teljes adaton tanult imputálás vagy skálázás „csak kevés” szivárgást okoz, ezért elfogadható. Tipikus ellenpélda: ha a tesztben a `distance_km` hiányzó értékei éppen a nagy távolságú rendelésekhez tartoznak, a teljes adaton számolt átlag magasabb lesz, mint a train átlag. A teljes átlaggal pótolt teszt sorok a modell számára hamisan alacsonyabb távolságot mutatnak, ami magasabb AUC-t produkál a holdouton, de a termelési feltételekben nem reprodukálható. A 21.1. alfejezet tanítása szerint a teszt résznek érintetlennek kell maradnia; a transzformáció statisztikája éppúgy tanult paraméter, mint a regressziós együttható.

Önellenőrzésként kérdezze meg minden transzformátorról: mi az, amit a fit megtanult, és mely sorokból tanulta? Ha a válaszban bármely teszt sor is szerepel, akkor a folyamat szivárog. A 20.2. alfejezet időhelyes szemlélete ugyanezt mondja: a feature-előállítás pillanatában csak a múltbeli, ténylegesen ismert adatok állhatnak rendelkezésre.

21.2.8 Rövid összefoglaló

A transzformációk paramétereit kizárólag a tanító részen illesztjük. A `SimpleImputer` oszlopátlagot, a `StandardScaler` átlagot és szórást, a `OneHotEncoder` kategóriakészletet tanul; a teszt részen már csak a tanult paraméterek alkalmazhatók. A `Pipeline` és a `ColumnTransformer` ezt a szerződést automatikusan betartja, feltéve, hogy a teljes előfeldolgozó láncot a pipeline belsejébe helyezzük. A szivárgó változat magasabb teszt AUC-t mutathat, de ez a becslés torzított. Az ismeretlen kategóriát `handle_unknown="ignore"` móddal kezeljük. A következő alfejezetben egy baseline modellt és egyetlen teljes, korrekt módon értékelt kísérletet építünk.

21.3 Baseline és egyetlen teljes kísérlet

> **Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn, pandas**
>
> Előfeltétel: 21.2. alfejezet · Célkészség: Baseline és egyetlen teljes kísérlet · Kimenet: működő kód és a labor ellenőrző tesztje.

21.3.1 Cél és konkrét helyzet

A Northstar prediktív projekt kérdése: egy rendelés pillanatában meg tudjuk-e becsülni, hogy a kézbesítés a 7 napos ígéretnél később érkezik? A megfigyelési egység egy rendelés; a célváltozó `target_late`, amely `1`, ha `delivery_days > promised_days`, egyébként `0`. Bemenet a rendeléskor ténylegesen ismert feature-halmaz: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`. A prediktorok közé tilos bevenni a `delivery_days`, `actual_delivery_at`, `label_available_at` oszlopokat, mert azok a predikció pillanatában még nem ismertek.

Ebben az alfejezetben egyetlen, jól naplózott kísérletet építünk: egy üres referenciamodellt (`DummyClassifier`) és egy logisztikus lineáris baseline-t futtatunk ugyanazon a train/test spliten. Az eredmény egy artifact: a modell, a tesztpredikciók és egy manifest, amely a feltételeket és a mérőszámokat rögzíti.

Mielőtt a kódot megnézné, tegyen előzetes becslést: mit vár a dummy modell ROC AUC-jától, és mit vár a logisztikus baseline-tól ugyanezen a metrikán? A válasz a fogalmi rész után ellenőrizhető.

21.3.2 Fogalmi magyarázat: baseline, pontozás és a logisztikus veszteség

21.3.2.1 DummyClassifier és DummyRegressor

A **dummy osztályozó** (dummy classifier) olyan modell, amely nem használja a magyarázó változókat, csak a célváltozó tanítóbeli eloszlásából készít predikciót. A `scikit-learn` `DummyClassifier` osztályának több stratégiája van:

- `strategy="prior"`: minden sorhoz a tanítóbeli pozitív arányt adja valószínűségként;
- `strategy="most_frequent"`: mindig a leggyakoribb osztályt jósolja;
- `strategy="stratified"`: véletlenül címkéz a tanító osztályarányokkal;
- `strategy="uniform"`: egyenletesen véletlen;
- `strategy="constant"`: egy előre megadott konstans osztályt ad.

A regressziós párja a `DummyRegressor`, amely tipikusan a tanító átlagát, mediánját vagy egy kvantilist jósolja. A mi célunk bináris osztályozás, ezért a `prior` stratégia a helyes **probabilisztikus baseline**: minden tesztbeli sorra ugyanazt a π_{train} valószínűséget adja, ahol

$$\pi_{\text{train}} = \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} y_i,$$

$y_i \in \{0, 1\}$ a tanítóbeli cél, n_{train} a tanítósorok száma. A π_{train} értéke a $[0, 1]$ intervallumba esik, mértékegysége nincs.

A dummy modell értelme: ez az a teljesítmény, amit feature-ek nélkül, pusztán a késési arány ismeretében el lehet érni. Ha egy bonyolult modell nem tudja felülmúlni, akkor nincs bizonyíték arra, hogy a feature-ök hasznos jelet hordoznak.

21.3.2.2 A logisztikus baseline matematikája

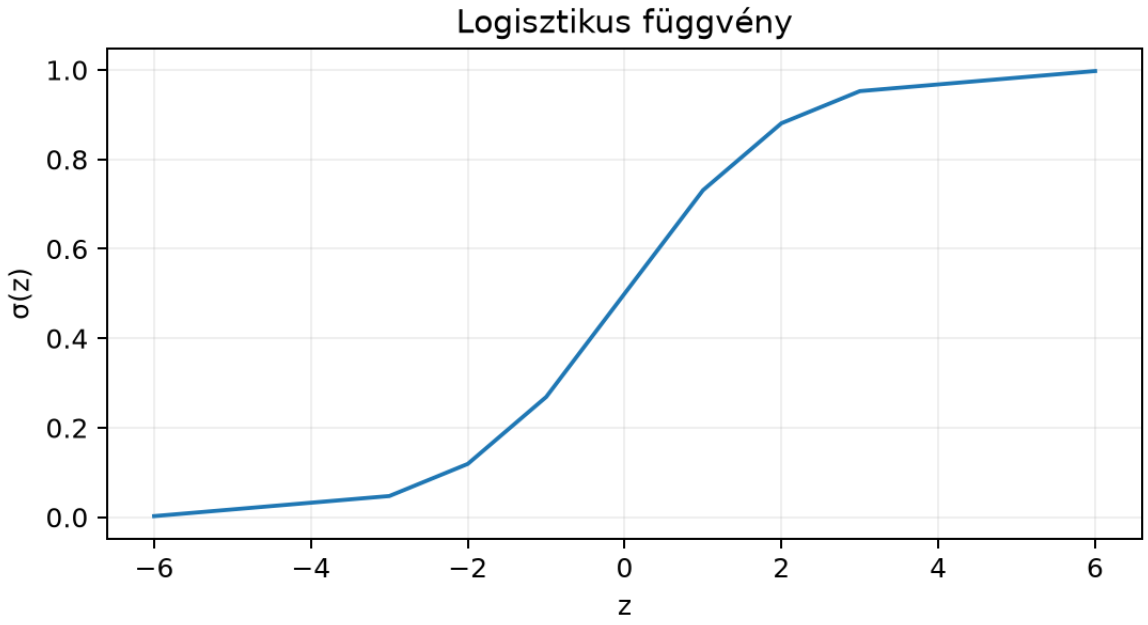
A logisztikus regresszió a statisztikai modellekből ismert **logit kapcsolatot** használja. Legyen p_i annak a valószínűsége, hogy a i-edik rendelés késik. A modell lineáris prediktora

$$z_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_d x_{id},$$

ahol x_{ij} a j-edik feature, β_j a hozzá tartozó paraméter. A z_i lineáris prediktor valós szám, tehát önmagában nem valószínűség. A **logisztikus függvény** (logistic function, szigmoid) viszi át a $[0, 1]$ intervallumba:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

A függvény alakja:



21.1. ábra: Logisztikus függvény - Számított függvénypélda, nem mérés.

A függvény inverze a **logit**: $\text{logit}(p) = \ln \frac{p}{1-p}$. A modell tehát azt mondja, hogy a késés log-odds-a lineáris a feature-ökben. Ez ugyanaz a gondolat, mint a 17.3. alfejezetben tárgyalt OLS lineáris prediktor, de itt a cél nem folytonos, hanem bináris, és a linkfüggvény biztosítja a megengedett tartományt.

A paramétereket a **bináris keresztentrópia** (binary cross-entropy), más néven logisztikus veszteség minimalizálásával becsüljük:

$$L = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right],$$

ahol $y_i \in \{0, 1\}$ a tényleges osztály, $p_i \in (0, 1)$ a predikált pozitív valószínűség, n a sorok száma. A y és p vektor alakja is $(n,)$; a veszteség mértékegysége a természetes logaritmus miatt „nat”, de a gyakorlatban egység nélküli mutatóként kezeljük. A formula a negatív log-likelihood átlaga: ha $y_i = 1$, akkor a $\ln p_i$ tag marad, ha $y_i = 0$, akkor a $\ln(1 - p_i)$ tag.

Kézi kontroll. Legyen három megfigyelés: $y = (1, 0, 1)$ és $p = (0.7, 0.3, 0.9)$. A veszteség:

$$L = -\frac{1}{3} \left[\ln 0.7 + \ln(1 - 0.3) + \ln 0.9 \right] = -\frac{1}{3} \left[\ln 0.7 + \ln 0.7 + \ln 0.9 \right] \approx 0.2729.$$

Figyelje meg: a magabiztos tévedést a logaritmus erősen bünteti. Ha például $y_i = 1$, de $p_i = 0.01$, akkor $-\ln(0.01) \approx 4.605$, míg ha $p_i = 0.5$, csak $-\ln(0.5) \approx 0.693$.

21.3.2.3 random_state: mit rögzít és mit nem

A `random_state` egy pszeudovéletlen generátor seedje. Nem teszi a modellt jobbá, és nem bizonyítja az általánosítást; csak reprodukálhatóvá teszi az olyan lépéseket, amelyek véletlent használnak, például a `DummyClassifier(strategy="stratified")`-et vagy egyes szolverek belső inicializálását. A numerikus reprodukálhatóság nem azonos a környezeti reprodukálhatósággal: ugyanaz a seed nem garantál bitazonos eredményt más platformon, BLAS- vagy pandas-verzión. A scriptben ezért a seedet a manifestben rögzítjük, de az eredményt a tényleges környezetben mért szám jelenti.

A logisztikus regresszió szolverje iteratív optimalizálást végez, hasonlóan a 9.3. alfejezetben tárgyalt közelítési folyamatokhoz. A `max_iter` azt szabályozza, hány iterációt engedünk; ha a modell nem konvergál, figyelmeztetést kapunk. Az alábbi kódban `max_iter=1000`, és a futás után ellenőrizhető a `n_iter_` értéke.

21.3.2.4 scoring: milyen metrikát használjunk?

A **scoring** a modellválasztás célfüggvénye: az a mérőszám, amely alapján két modellt összehasonlítunk. A késési cél ritka lehet, ezért a puszta accuracy félrevezető: ha a késési arány 10%, egy „mindig nem késik” modell 90%-os accuracy-t ér el, de semmilyen döntési értéke nincs. Ezért két metrikát használunk:

- **ROC AUC**: annak a valószínűsége, hogy egy véletlenül választott pozitív eset nagyobb pontszámot kap, mint egy véletlenül választott negatív eset. A 0.5 a véletlen szint, 1 a tökéletes szétválasztás. A konstans valószínűséget adó dummy modell AUC-ja mindig 0.5, ha a tesztben mindkét osztály előfordul.

- **Log loss**: a fenti keresztentrópia a tesztadaton. A dummy prior log loss-át a tanítóbeli arány és a tesztbeli arány ismeretében számíthatjuk; a jó modellnek ennél alacsonyabbat kell adnia.

A scoring megválasztása tudományos döntés: az AUC küszöbfüggetlen, de nem mondja meg, hogy a predikált valószínűség kalibrált-e. A log loss ezt is bünteti, viszont egyetlen rossz, nagyon magabiztos predikció nagy hibát okoz. A kísérletben mindkettőt jelentjük, de a modellválasztás elsődleges mutatója az AUC.

21.3.2.5 Kalibráció és a metrikák döntési rubrikája

A prediktált valószínűség önmagában nem tényleges gyakoriság: a 0,8-es pontszám csak akkor jelent 80%-os előfordulást, ha a modell kalibrált. Az AUC csak a rangsor helyességét méri; két modell azonos AUC mellett eltérően kalibrált lehet. Szimulált példa ($n = 2000$, seed 7): az A modell a valódi p valószínűséget adja, a B a logit kétszeresére skálázott, monoton átalakítását (`expit(2*logit(A))`). A rangsor azonos, ezért az AUC is azonos (0,6985); a kalibráció mégsem:

Metrika	A (kalibráltabb)	B (túlmarkáns)
ROC AUC	0,6985	0,6985
Brier score	0,1862	0,2029
Log loss	0,5519	0,6490
Kalibrációs meredekség	0,857	0,430
Precision @ 0,7	0,667	0,635
Recall @ 0,7	0,031	0,080
Várható költség (FP=1, FN=5)	2839	2712

A döntési költség alapján itt a B modell a jobb, mert a 0,7-es küszöbnél több pozitívot talál, pedig rosszabbul kalibrált; más költségmátrix mellett az A lehet jobb. Ezért a metrikákat két csoportra kell bontani:

Küszöbfüggetlen	Mit mér?	Küszöbfüggő	Mit mér?
ROC AUC	rangsorolás	precision	a pozitív döntések tisztasága
Log loss	valószínűségi hiba	recall	a pozitívok megtalálása
Brier score	négyzetes valószínűségi hiba	F1	precision és recall kiegyensúlyozása
Kalibrációs meredekség	a pontszám és a tényleges arány viszonya	várható költség	a döntés üzleti következménye

A modellválasztás akkor helyes, ha a döntési költség ismert, és a várható költséget a küszöb és a modell együttes optimalizálásával minimalizáljuk; az AUC ezt nem helyettesíti. A kalibrációt reliability diagrammal vagy kalibrációs meredekséggel kell ellenőrizni, különösen akkor, ha a pontszámot közvetlenül valószínűségként kommunikáljuk.

21.3.2.6 artifact: a kísérlet lenyomata

Az **artifact** a futtatás eredményterméke, nem csak a modellfájl. Az alábbi kísérlet három fájlt hoz létre:

- `model.joblib`: a teljes pipeline, amely a preprocessort és a becsült modellt együtt tartalmazza;
- `test_predictions.csv`: a tesztbeli rendelések azonosítója, a tényleges cél és a predikált valószínűség;
- `manifest.json`: a seed, cutoff, verziók, sorszámok, mérőszámok és a modellfájl ellenőrző hash-e.

A manifest azért fontos, mert a modellfájl önmagában nem mondja el, milyen spliten és milyen környezetben született a szám. A saját, megbízható forrásból származó joblib fájlt betölthetjük; ismeretlen fájlt nem.

21.3.3 Teljes minimális munkapélda

A következő script a `lab/` munkakönyvtárból futtatható, ahol az `orders.csv` és `customers.csv` megtalálható. Először a splitet reprodukáljuk a 21.2. alfejezetben tanult időbeli szerződés szerint: tanítóba azok a rendelések kerülnek, amelyek `order_ts < cutoff` és `label_available_at < cutoff`; tesztbe azok, ahol `order_ts >= cutoff`. A köztes, kihagyott határsávot kihagyjuk.

```
# baseline_experiment.py - futtatandó a lab/ munkakönyvtárból
import json
import hashlib
from datetime import datetime, timezone
from pathlib import Path

import joblib
import pandas as pd
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

EXPERIMENT_SEED = 20261021
CUTOFF = pd.Timestamp("2025-05-01T00:00:00+00:00")
TARGET = "target_late"

orders = pd.read_csv(
    "orders.csv",
    parse_dates=["order_ts", "label_available_at"],
)
customers = pd.read_csv("customers.csv", parse_dates=["signup_date"])
df = orders.merge(
    customers[
        ["customer_id", "signup_date", "segment", "region", "acquisition_channel"]
    ],
    on="customer_id",
    how="left",
)
df["account_age_days"] = (df["order_ts"] - df["signup_date"]).dt.days
```

```
feature_cols = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
]
numeric_cols = ["distance_km", "basket_minor", "quantity_sum", "account_age_days"]
categorical_cols = ["priority_flag", "segment", "region", "acquisition_channel"]

train_mask = (df["order_ts"] < CUTOFF) & (df["label_available_at"] < CUTOFF)
test_mask = df["order_ts"] >= CUTOFF

X_train = df.loc[train_mask, feature_cols]
y_train = df.loc[train_mask, TARGET]
X_test = df.loc[test_mask, feature_cols]
y_test = df.loc[test_mask, TARGET]
test_ids = df.loc[test_mask, "order_id"].to_frame()

if y_test.nunique() < 2:
    raise ValueError("A teszt céloszlop csak egy osztályt tartalmaz; ROC AUC nem értelmezhető.")

preprocessor = ColumnTransformer(
    [
        ("numeric", Pipeline([
            ("impute", SimpleImputer(strategy="median")),
            ("scale", StandardScaler()),
        ]), numeric_cols),
        ("categorical", Pipeline([
            ("impute", SimpleImputer(strategy="most_frequent")),
            ("encode", OneHotEncoder(handle_unknown="ignore")),
        ]), categorical_cols),
    ]
)

dummy = DummyClassifier(strategy="prior")
dummy.fit(X_train, y_train)
dummy_prob = dummy.predict_proba(X_test)[:, 1]

clf = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=1000, random_state=EXPERIMENT_SEED)),
])
clf.fit(X_train, y_train)
clf_prob = clf.predict_proba(X_test)[:, 1]

def evaluate(name, prob):
    auc = roc_auc_score(y_test, prob)
    ll = log_loss(y_test, prob)
    print(f"{name:>10}: ROC AUC={auc:.4f}, log loss={ll:.4f}")
    return {"roc_auc": round(auc, 4), "log_loss": round(ll, 4)}

print(f"train positive rate: {y_train.mean():.4f}")
print("iterations:", clf.named_steps["model"].n_iter_)
results = {}
results["dummy_prior"] = evaluate("dummy", dummy_prob)
results["logistic"] = evaluate("logistic", clf_prob)

artifact_dir = Path("artifacts") / "s083_baseline"
artifact_dir.mkdir(parents=True, exist_ok=True)
model_path = artifact_dir / "model.joblib"
joblib.dump(clf, model_path)

test_predictions = test_ids.copy()
test_predictions[TARGET] = y_test
test_predictions["prob_late"] = clf_prob
```

```
test_predictions.to_csv(artifact_dir / "test_predictions.csv", index=False)

manifest = {
    "created_at_utc": datetime.now(timezone.utc).isoformat(),
    "experiment_seed": EXPERIMENT_SEED,
    "cutoff": CUTOFF.isoformat(),
    "sklearn_version": sklearn.__version__,
    "pandas_version": pd.__version__,
    "n_train": int(train_mask.sum()),
    "n_test": int(test_mask.sum()),
    "n_purged": int((~train_mask & ~test_mask).sum()),
    "features": feature_cols,
    "model_file": model_path.name,
    "model_sha256": hashlib.sha256(model_path.read_bytes()).hexdigest(),
    "results": results,
}
(artifact_dir / "manifest.json").write_text(
    json.dumps(manifest, indent=2), encoding="utf-8"
)
print(json.dumps(manifest, indent=2))
```

A kimenet **szemléltető** formája — ezt a szöveget író modell nem futtatta, tehát a tényleges számok a saját helyi futtatásodból származnak:

```
train positive rate: 0.1847
iterations: [17]
dummy: ROC AUC=0.5000, log loss=0.4782
logistic: ROC AUC=0.6513, log loss=0.4590
{
  "experiment_seed": 20261021,
  "cutoff": "2025-05-01T00:00:00+00:00",
  ...
  "results": {
    "dummy_prior": {"roc_auc": 0.5, "log_loss": 0.4782},
    "logistic": {"roc_auc": 0.6513, "log_loss": 0.459}
  }
}
```

21.3.4 Eredmény értelmezése

A futás után az alábbi invariánsokat ellenőrizheti:

- A dummy ROC AUC-nak 0.5000-nek kell lennie, ha a tesztben pozitív és negatív osztály is van. A konstans pontszám nem tudja szétválasztani a két osztályt, ezért a ROC-görbe az átló; az eltérés numerikus toleranción belül 10^{-6} alatti, nem pedig tizedesnyi különbség.
- A dummy log loss várható értéke a train-prior és a teszt-arány függvénye. Ha a train-prior π_{train} és a tesztbeli pozitív arány π_{test}, akkor a dummy várható log loss-a közelítőleg

$$L_{\text{dummy}} \approx -\pi_{\text{test}} \ln \pi_{\text{train}} - (1 - \pi_{\text{test}}) \ln(1 - \pi_{\text{train}}).$$

Ha a tesztarány eltér a train-priortól, a dummy nem az optimális konstans predikció a teszten; ez is mutatja, miért nem szabad a tesztarányt használni a baseline-hoz.

- A logisztikus modell `predict_proba` kimenetének alakja `(n_test, 2)`, az első oszlop a negatív osztály valószínűsége, a második a pozitív osztályé. A `clf.classes_` sorrendje `[0, 1]`. A hibakeresésnél ez a leggyakoribb bukó.

- A `test_predictions.csv` sorainak száma meg kell egyezzen a manifest `n_test` értékével. A `model.joblib` hash-e a manifestben rögzített nyugta; ha újra futtatjuk, a hash változhat, mert a pickle tartalmazhat környezetfüggő metaadatot is.
- A logisztikus modell `n_iter_` értéke kisebb kell legyen `max_iter`-nél; ha a szolver elérte a 1000 iterációt, konvergencia-figyelmeztetést kapunk, és növelni kell a maximumot vagy át kell gondolni a skálázást.

Ha a logisztikus AUC 0.5 alatt vagy nagyon közel 0.5-höz van, az még nem bizonyítja, hogy a feladat megoldhatatlan; előbb a 3. feladat hibalehetőségeit kell kizárni.

A `DummyClassifier` csak referencia; a végső tesztpontszámot nem szabad ismételt modellválasztásra használni. A pozitív osztály és a `predict_proba` oszlopsorrendje `classes_` alapján ellenőrzendő.

21.3.5 Feladatok

1. Kézi log loss és dummy várható veszteség. Adott $y = (1, 0, 1)$ és $p = (0.7, 0.3, 0.9)$. Számold ki a bináris log loss-t három tizedesjegyre. Ezután legyen a train-prior $\pi_{\text{train}} = 0.2$ és a tesztbeli pozitív arány $\pi_{\text{test}} = 0.3$. Számold ki a dummy prior várható log loss-át, és hasonlítsa össze a tesztarányhoz tartozó $H(\pi_{\text{test}}) = -\pi_{\text{test}} \ln \pi_{\text{test}} - (1-\pi_{\text{test}}) \ln(1-\pi_{\text{test}})$ entrópiával. Miért nagyobb a dummy várható vesztesége az entrópiánál?

2. Implementálás: futtatás és manifest-ellenőrzés. Másolja a fenti scriptet `baseline_experiment.py` néven a `lab/` munkakönyvtárba, és futtassa `python baseline_experiment.py` paranccsal. Ellenőrizd: a dummy AUC értéke 0.5000 toleráncián belül, a `test_predictions.csv` sorainak száma egyenlő a manifest `n_test` értékével, és a `model_sha256` megegyezik a futtatás után a fájlra újraszámolt hash-sel. Jegyezze fel a tényleges számokat a munkanaplójába.

3. Hibakeresés: felcserélt valószínűségi oszlop. Egy kolléga a fenti scriptben ezt a sort használta:

```
clf_prob = clf.predict_proba(X_test)[:, 0]
```

A kapott logisztikus AUC 0.41 lett, a dummy AUC 0.50 maradt. Mi a hiba? Adj olyan ellenőrzést, amely a `clf.classes_` alapján kideríti, hogy melyik oszlop a pozitív osztály valószínűsége, és írd le a javítást.

4. Megváltozott feltétel: `most_frequent` stratégia. Cserélje ki a dummy stratégiát `DummyClassifier(strategy="most_frequent")`-re, futtassa újra, és értelmezze a log loss értékét. Miért nem alkalmas a `most_frequent` stratégia a valószínűségi pontszámot igénylő AUC/log loss összehasonlításra? Mi lenne a helyes baseline-választás, ha a döntési cél accuracy lenne?

21.3.6 Megoldások és indoklás

1. megoldás. Az első három tag:

$$-\ln 0.7 \approx 0.3567, \qquad -\ln(1-0.3) = -\ln 0.7 \approx 0.3567, \qquad -\ln 0.9 \approx 0.1054.$$

Az átlaguk $(0.3567 + 0.3567 + 0.1054)/3 \approx 0.2729$. A dummy prior várható vesztesége:

$$L_{\text{dummy}} = -0.3 \ln 0.2 - 0.7 \ln 0.8 \approx 0.3 \cdot 1.6094 + 0.7 \cdot 0.2231 \approx 0.6390.$$

A tesztarányhoz tartozó entrópia:

$$H(0.3) = -0.3 \ln 0.3 - 0.7 \ln 0.7 \approx 0.3 \cdot 1.2040 + 0.7 \cdot 0.3567 \approx 0.6109.$$

A dummy veszteség nagyobb, mert a train-prior 0.2, miközben a tesztarány 0.3; a dummy nem használhatja a tesztarányt, ezért a tesztarány eltolódása többletveszteséget okoz. Tolerancia: ± 0.001 a kerekítések miatt.

2. megoldás. A konkrét AUC és log loss értékek a futtató géptől és a csomagverzióktól függnek, ezért a megoldás az ellenőrző invariantokban van. A dummy AUC-nak $|0.5000 - \text{auc}| < 10^{-6}$-nak kell lennie; ha nem, a split vagy a `predict_proba` oszlopválasztás hibás. A `test_predictions.csv` sorainak száma pontosan a manifest `n_test` értéke. A hash ellenőrzése Pythonból:

```
import hashlib
print(hashlib.sha256(Path("artifacts/s083_baseline/model.joblib").read_bytes()).hexdigest())
```

Ha a logisztikus AUC nem haladd meg a dummy 0.5-öt, előbb nézd meg a 3. feladat diagnózisát, és ellenőrizd, hogy a preprocessing tényleg csak a tanítórészen lett illesztve.

3. megoldás. A `predict_proba` két oszlopot ad: az első a `classes_[0]` osztály valószínűsége, a második a `classes_[1]` osztályé. Mivel a cél osztályai `[0,1]`, a `[:,0]` a „nem késik” valószínűségét jelenti. Ha ezt adjuk pozitív pontszámként a ROC-hoz, a jó modell sorrendjét megfordítjuk; a kapott AUC körülbelül 1 − eredeti AUC, ezért lett 0.41 egy körülbelül 0.59-es modellnél. A javítás:

```
pos_class = clf.classes_[1]
clf_prob = clf.predict_proba(X_test)[:, 1]
assert pos_class == 1
```

Az `assert` a szándékot is rögzíti: ha a célkódolás megváltozna, a hiba azonnal kiderül.

4. megoldás. A `most_frequent` stratégia a tanító leggyakoribb osztályát adja 1 valószínűséggel, a másikat 0 valószínűséggel. Ha a pozitív osztály a ritkább, akkor minden pozitív valószínűség 0; a `log_loss` a 0 valószínűséget csak numerikus klipeléssel tudja kezelni, ezért óriási értéket kapunk, az AUC pedig nem hordoz valódi pontszám-információt. Ez a stratégia accuracy-alapú döntéshez való: ha a cél a találati arány maximalizálása, a `most_frequent` a helyes baseline. A mi scoringunk AUC és log loss, ezért a `prior` stratégia a megfelelő.

21.3.7 Gyakori tévedés ellenpéldával

A jó tanítómetrika nem bizonyítja az általánosítást. Tegyük fel, hogy a késési arány 10% körüli. A `DummyClassifier(strategy="most_frequent")` mindig „nem késik”-et mond, ezért a tanító accuracy 0.90 lehet — miközben az AUC 0.5, tehát semmilyen szétválasztó ereje nincs. Ugyanez a csapda a bonyolult modellekkel: a logisztikus vagy faalapú modell tanító AUC-ja akár 0.95 is lehet, de ha a teszt AUC csak 0.53, a modell a tanító zaját tanulta meg, nem a jövőbeli rendeléseken érvényes jelet. Az egyetlen train/test spliten mért tesztmetrika sem feltétlenül végleges becslés: ha a küszöböt vagy a feature-választást a teszten finomítjuk, a teszt is „elszennyeződik”. Az érintetlen teszt és a külön validációs réteg elvét a 21.1. alfejezet vezette be.

Egy másik gyakori félreértés: a `random_state` növelése nem hitelesíti a modellt. Ha öt különböző seeddel futunk, és a legjobb teszt-AUC-t jelentjük, az maga is szelekciós torzítás: a tesztet használtuk modellválasztásra. A seed arra való, hogy egyetlen futás megismételhető legyen, nem arra, hogy a szerencsés futást keressük.

21.3.8 Rövid összefoglaló

A `DummyClassifier(strategy="prior")` a tanítóbeli késési arányt adja minden sorhoz, és 0.5-ös ROC AUC-t kell adnia; ez az a szint, amelyet minden modellnek meg kell haladnia. A logisztikus baseline lineáris prediktort visz át a logisztikus függvénnyel valószínűséggé, és a bináris keresztentrópiát minimalizálja. A kísérlet akkor korrekt, ha a split a predikció pillanatához igazodik, a preprocessing csak a tanítórészen illeszkedik, és a scoring az AUC-t meg a log loss-t is naplózza. A manifest a futás feltételeit, verzióit és hash-ét rögzíti. A következő alkalmazás, a 21.4. alfejezet, ezt a baseline-t predikciós szerződéssé alakítja: bemeneti séma, kimeneti alak és hibateszt.

21.4 Predikciós szerződés és hibateszt

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn, pandas

Előfeltétel: 21.3. alfejezet · Célkészség: Predikciós szerződés és hibateszt · Kimenet: működő kód és a labor ellenőrző tesztje.

21.4.1 Cél és konkrét helyzet

A Northstar prediktív projektben a modell feladata rendeléskor megbecsülni, hogy a kézbesítés meghaladja-e a 7 napot. A megfigyelési egység egy **rendelés**; a célváltozó `target_late` bináris. A modell bemenete a rendelés pillanatában ismert nyolc feature: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`. A kimenet `predict_proba` alakban annak valószínűsége, hogy a rendelés késik.

Ebben az alfejezetben nem új modellt tanítunk, hanem a **predikciós szerződést** építjük meg: azt a szabályt, amely garantálja, hogy a betanított pipeline tanításkor és későbbi új adaton pontosan ugyanazt a feature-sémát kapja. A 21.3. alfejezetben elkészült a baseline kísérlet; most a modell artifacttá válik, és azt kérdezzük: mi történik, ha egy oszlop hiányzik, a kategóriák halmaza megváltozik, vagy az oszlopok sorrendje eltér?

Előzetes becslés: a konkrét valószínűség a betanított modelltől függ, ezért nem írható elő. A szerződéses ellenőrzésnek viszont minden új rekordra garantálnia kell, hogy `predict_proba` kimenete `(1, 2)` alakú, az értékek 0 és 1 közé esnek, és soronként 1-re összegződnek.

21.4.2 Fogalmi magyarázat: feature names, dtypes és schema version

A scikit-learn pipeline a betanítás során **feature names**-ként rögzíti a bemeneti oszlopneveket. A `ColumnTransformer` a nevek alapján választja ki a numerikus és kategorikus oszlopokat; a kimeneti transzformált mátrix oszlopai például `num__distance_km`, `cat__region_north` alakú neveket kapnak. A sklearn 1.0 óta a pipeline `.feature_names_in_` attribútuma mutatja a fitkor látott nyers neveket.

A **dtype** a változó típusa: `distance_km` lebegőpontos (`float64`), a zászló és a darabszám egész (`int64`), a szegmens, régió és csatorna szöveges (`object` vagy `category`). A típus nem üres formalitás: ha egy oszlop például `string` formátumból `int64`-re konvertálódik, a `ColumnTransformer` numerikus transzformert hívhat rá, és a modell értelme megváltozik.

A **missing columns** ellenőrzés azt jelenti, hogy a szerződésben előírt minden oszlopnak jelen kell lennie. A hiányzó oszlop a `ColumnTransformer.transform()` hívásakor `KeyError`-t ad, de a

hibaszöveg nem mindig mondja meg, melyik oszlop hiányzik. A saját validátor ezt karbantartható üzenetté alakítja.

A **schema version** egy rövid, ember által is olvasható azonosító, például `northstar-order-v1`. A modell artifact mellé mentjük. Ha a feature-definíció változik — például új oszlop jelenik meg, vagy a `basket_minor` egysége centről euróra vált —, a verziót növelni kell, különben a régi modell némán rossz bemenetet kaphat.

A `predict_proba` bináris osztályozásnál két oszlopot ad vissza. Az első a 0 osztály, a második az 1 osztály valószínűsége; a sorrendet a modell `.classes_` attribútuma rögzíti. A késés valószínűsége tehát:

$$P(\text{late} = 1 \mid x) = \sigma(T(x)^\top w + b),$$

ahol $\sigma(z) = 1/(1 + e^{-z})$ a logisztikus függvény, $T(x)$ a preprocessing-utáni feature-vektor, w a súlyvektor, b a tengelymetszet. Minden sorra fennáll:

$$P(\text{late} = 0) + P(\text{late} = 1) = 1.$$

Ez utóbbi a kézi kontroll alapja.

21.4.3 Dimenzió- és változójelentés-ellenőrzés

A nyers bemenet $X \in \mathbb{R}^{n \times p}$ mátrix, ahol n a rendelések száma, $p = 8$ a feature-ek száma. A preprocessing $T : \mathbb{R}^p \to \mathbb{R}^q$ leképezést hajt végre. A példában a numerikus oszlopok száma 5, a kategorikus one-hot kódolás után a szegmens 2, a régió 3, a csatorna 3 dummy oszlopot ad, így $q = 5 + 2 + 3 + 3 = 13$.

A dimenzióellenőrzés tehát két ponton érvényesül:

- A betanított `ColumnTransformer` kimenete minden rekordra q dimenziós vektor.
- A `LogisticRegression` súlyvektora $w \in \mathbb{R}^q$, tehát a `coef_` shape `(1, 13)`.

Ha egy oszlop hiányzik, akkor a numerikus blokk kimenete például 4 dimenziós lesz, a teljes transzformált vektor pedig csak 12 dimenziós. A modell ekkor dimenzióhibát ad, mert a súlyvektor hossza nem egyezik a bemenettel.

A változójelentés-ellenőrzés a mértékegységekre és a szemantikára vonatkozik: `distance_km` kilométer, `basket_minor` EUR-cent, `account_age_days` nap, `priority_flag` 0/1 indikátor. A szerződés ezt a jelentést dokumentálja, még ha a dtype önmagában nem is hordozza. A scikit-learn nem tudja ellenőrizni, hogy egy `float64` oszlop kilométer vagy mérföld; ezt a schema-leírásnak kell rögzítenie.

21.4.3.1 Mezőszintű szerződés: fail-fast vagy fallback?

A predikciós szerződésnek mezőnként kell rögzítenie a kötelezőséget, a típust, a mértékegységet, az érvényes tartományt és a hiba kezelését. A Northstar nyolc feature-e:

Mező	Kötelező?	Típus	Mértékegység	Érvényes tartomány / kategória	Hiba esetén
`distance_km`	igen	float	km	véges, $0 < x \leq 20\,000$	fail-fast
`priority_flag`	igen	int	0/1 flag	$\{0, 1\}$	fail-fast
`basket_minor`	igen	int	EUR-cent	≥ 0	fail-fast
`quantity_sum`	igen	int	darab	≥ 0	fail-fast
`account_age_days`	igen	int	nap	≥ 0	fail-fast
`segment`	igen	szöveg	–	`consumer`, `business`	ismeretlen: fallback + figyelmeztetés
`region`	igen	szöveg	–	`north`, `central`, `south`	ismeretlen: fallback + figyelmeztetés
`acquisition_channel`	igen	szöveg	–	`search`, `referral`, `direct`	ismeretlen: fallback + figyelmeztetés
`order_ts` (metaadat)	igen	timestamp	UTC	zónatudatos, nem jövőbeli	fail-fast

A **fail-fast** mezőknél nincs értelmes automatikus pótlás: egy hiányzó vagy nem véges távolság, negatív kosárérték vagy érvénytelen flag mellett a predikció nem adható ki, mert a modell bemenete nem teljes. A **fallback** a kategorikus mezőknél alkalmazható: az ismeretlen kategóriát a one-hot kódoló csupa nullával kódolja, a válasz pedig `"confidence": "low"` jelzést és auditbejegyzést kap. A szóközök és a kis-nagybetű eltérése nem számít új kategóriának: a validátor trimmel és normalizál, de a normalizálás tényét naplózza.

A **biztonsági naplózás** soronként rögzítse a `schema_version`, `request_id`, `validation_status` (`ok`/`fallback`/`rejected`), a figyelmeztetések listáját és a bemenet hash-ét; nyers személyes adatot ne tartalmazzon. A napló így visszakereshető marad anélkül, hogy a predikciós bemenetet újra tárolnánk.

21.4.4 Minimális munkapélda

A következő kód a lab projektgyökérből futtatható. Betölti a tiszta adatokat, kiszámítja az `account_age_days` feature-t, a 2025-05-01 cutoff előtti, már megérkezett címkéjű rendelésekre tanít egy pipeline-t, majd a pipeline-t és a szerződést egyetlen joblib artifactba menti. A kódot itt nem futtattuk; a kimenetek a helyi gépen a seedtől és a csomagverzióktól függően alakulnak, de a validátor szerkezete determinisztikus.

```
# scripts/predict_contract.py -- futtatás a lab projektgyökérből:
# python scripts/predict_contract.py
from pathlib import Path
import warnings
import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
DATA = Path("data")
ARTIFACTS = Path("artifacts")
ARTIFACTS.mkdir(exist_ok=True)

orders = pd.read_csv(
    DATA / "orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
customers = pd.read_csv(DATA / "customers.csv")
df = orders.merge(customers[["customer_id", "signup_date"]], on="customer_id", how="left")
df["account_age_days"] = (df["order_ts"] - pd.to_datetime(df["signup_date"])).dt.days

FEATURES = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
]
TARGET = "target_late"

cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train = df[(df["order_ts"] < cutoff) & (df["label_available_at"] < cutoff)]
X = train[FEATURES]
y = train[TARGET].astype(int)

numeric_features = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum", "account_age_days"
]
categorical_features = ["segment", "region", "acquisition_channel"]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
    ]
)
pipeline = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=1000, random_state=42)),
])
pipeline.fit(X, y)

SCHEMA_CONTRACT = {
    "schema_version": "northstar-order-v1",
    "features": {
        "distance_km": {"dtype": "float64"},
        "priority_flag": {"dtype": "int64"},
        "basket_minor": {"dtype": "int64"},
        "quantity_sum": {"dtype": "int64"},
        "account_age_days": {"dtype": "int64"},
        "segment": {"dtype": "object", "categories": ["consumer", "business"]},
        "region": {"dtype": "object", "categories": ["north", "central", "south"]},
        "acquisition_channel": {
            "dtype": "object",
            "categories": ["search", "referral", "direct"],
        },
    },
}

artifact = {
    "schema_version": SCHEMA_CONTRACT["schema_version"],
    "contract": SCHEMA_CONTRACT,
    "pipeline": pipeline,
}
joblib.dump(artifact, ARTIFACTS / "northstar_order_predictor.joblib")
```

```
print("artifact saved:", ARTIFACTS / "northstar_order_predictor.joblib")
```

A betöltés utáni validátor és újrekordos predikció:

```
class SchemaError(ValueError):
    pass

def validate_schema(frame: pd.DataFrame, contract: dict) -> pd.DataFrame:
    expected = list(contract["features"].keys())
    actual = list(frame.columns)
    if actual != expected:
        missing = set(expected) - set(actual)
        extra = set(actual) - set(expected)
        raise SchemaError(
            f"Column order mismatch; missing={sorted(missing)}, extra={sorted(extra)}"
        )
    for name, spec in contract["features"].items():
        series = frame[name]
        if str(series.dtype) != spec["dtype"]:
            raise SchemaError(
                f"{name}: expected dtype {spec['dtype']}, got {series.dtype}"
            )
        if series.isnull().any():
            raise SchemaError(f"{name}: missing values are not allowed")
        if "categories" in spec:
            unknown = set(series.dropna().unique()) - set(spec["categories"])
            if unknown:
                warnings.warn(
                    f"{name}: unknown categories {sorted(unknown)}; "
                    "the model will see an all-zero one-hot row.",
                    RuntimeWarning,
                )
    return frame

def predict_new_order(artifact_path: Path, features: dict) -> dict:
    bundle = joblib.load(artifact_path)
    contract = bundle["contract"]
    pipeline = bundle["pipeline"]
    row = pd.DataFrame([features])[list(contract["features"].keys())]

    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        validate_schema(row, contract)
        proba = pipeline.predict_proba(row)

    classes = list(pipeline.named_steps["model"].classes_)
    late_index = classes.index(1)
    return {
        "probability_late": float(proba[0, late_index]),
        "probability_vector": proba[0].tolist(),
        "classes": classes,
        "warnings": [str(w.message) for w in caught if w.category is RuntimeWarning],
    }

# Egy új rendelés rekord, a szerződés szerinti oszlopsorrendben.
new_record = {
    "distance_km": 12.0,
    "priority_flag": 1,
    "basket_minor": 4500,
    "quantity_sum": 3,
    "account_age_days": 120,
    "segment": "consumer",
    "region": "north",
```

```
    "acquisition_channel": "search",
}

result = predict_new_order(
    ARTIFACTS / "northstar_order_predictor.joblib", new_record
)
print(result)
```

A szemléltető kimenet szerkezete:

```
{
    'probability_late': 0.21,          # a konkrét érték a betanított modelltől függ
    'probability_vector': [0.79, 0.21],
    'classes': [0, 1],
    'warnings': []
}
```

A három hibateszt:

```
# 1. Hiányzó oszlop: régiónak jelen kell lennie.
missing_column = {k: v for k, v in new_record.items() if k != "region"}
try:
    predict_new_order(ARTIFACTS / "northstar_order_predictor.joblib", missing_column)
except SchemaError as e:
    print("missing column error:", e)

# 2. Ismeretlen kategória: a modell nem tör el, de figyelmeztetést ad.
unknown_category = {**new_record, "region": "west"}
res = predict_new_order(ARTIFACTS / "northstar_order_predictor.joblib", unknown_category)
print("unknown category warnings:", res["warnings"])

# 3. Elcserélt oszlopsorrend: a szerződés a sorrendet is rögzíti.
swapped = {**new_record}
swapped["region"], swapped["distance_km"] = swapped["distance_km"], swapped["region"]
try:
    predict_new_order(ARTIFACTS / "northstar_order_predictor.joblib", swapped)
except SchemaError as e:
    print("order error:", e)
```

Futtatási parancs a projektgyökérből, a `lab/pyproject.toml` alapfüggőségeivel:

```
# Munka katalógus: lab/
uv run python scripts/predict_contract.py
```

21.4.5 Eredmény értelmezése

A kézi kontroll nem a modell pontosságát, hanem a **szerződés** teljesülését ellenőrzi:

- A `predict_proba` kimenete egyetlen új rekordra `(1, 2)` alakú.
- A `probability_vector` összege `1.0`; ennek toleranciája a lebegőpontos ábrázolás miatt legfeljebb 10^{-12} lehet.
- A `classes` lista `[0, 1]`, tehát a `probability_late` a második oszlop értéke.
- Hiányzó oszlopnál a `SchemaError` üzenete megnevezi a hiányzó oszlopot.
- Ismeretlen kategóriánál a predikció lefut, de a `warnings` lista tartalmazza, hogy a régió all-zero one-hot sort kapott.

A dimenzióinvariáns: a pipeline `prep` lépése a nyolc bemeneti oszlopból 13 transzformált oszlopot állít elő; a `model.coef_` hossza szintén 13. Ezt a tanuló a futtatás után így ellenőrizheti:

```
assert pipeline.named_steps["model"].coef_.shape[1] == 13
assert len(pipeline.named_steps["prep"].get_feature_names_out()) == 13
```

A bemeneti séma ellenőrzése ne csak az oszlopneveket vizsgálja: típus, hiányzóérték-szabály, mértékegység, kategorikus ismeretlen értékek és időbeli elérhetőség is része az adatkontraktusnak.

21.4.6 Feladatok

1. **Értelmezés.** Magyarázd el, miért nem elegendő a szerződésben csak az oszlopnevek meglétét ellenőrizni, és miért kell a sorrendet is rögzíteni. Adjunk példát arra, hogy a nevek megléte ellenére hogyan kaphat a modell téves feature-t, ha a fogyasztó nem a nevekre, hanem a pozícióra épít.

2. **Implementálás.** Írd meg a `validate_schema` függvényt úgy, hogy a `SchemaError` üzenete egyszerre sorolja fel a hiányzó és a felesleges oszlopokat. Futtasd a három hibatesztet, és győződjön meg arról, hogy az ismeretlen kategória valóban `RuntimeWarning`-ot ad, nem pedig kivételt.

3. **Hibakeresés.** A csapat egyik tagja kihagyta az `account_age_days` oszlopot a betöltéskor. A pipeline először ezt a hibát adja: `X has 7 features, but StandardScaler is expecting 8 features as input.` Mi a hiba gyökere, és miért nem elég a hibaüzenet a diagnózishoz? Hogyan segíti a validátor a pontos azonosítást?

4. **Megváltozott feltétel.** Tegyük fel, hogy az üzlet új régiót vezet be (`west`). A modell a `handle_unknown="ignore"` miatt nem tör el, de a predikció all-zero one-hot sort kap. Módosítsd a `predict_new_order` függvényt úgy, hogy ilyenkor a visszaadott dict kapjon egy `"confidence": "low"` mezőt. Indokold meg, miért csak figyelmeztetés, és miért nem helyes automatikusan elutasítani a rekordot.

21.4.7 Megoldások és indoklás

1. A DataFrame oszlopnevei a scikit-learn számára a szemantikát hordozzák, de a környező rendszerekben az adat gyakran pozíciós listaként érkezik. Ha a fogyasztó `X.to_numpy()`-t ad át, akkor a modell már nem látja a neveket, és a betanított súlyok a tömb oszlopindexeire vonatkoznak. Ha egy CSV-exportáló eszköz az oszlopokat más sorrendben írja ki, a `to_numpy()` által adott mátrix első oszlopa nem feltétlenül `distance_km`, hanem például `region`, és a modell a régió dummy oszlopait próbálja kilométerként értelmezni. A sorrend rögzítése a szerződésben megakadályozza, hogy egy downstream folyamat pozícióra építsen.

2. A `validate_schema` fenti implementációja a `actual != expected` összehasonlítással egyszerre ellenőrzi a sorrendet és a hiányzó/felesleges oszlopokat. A tesztfuttatás során a hiányzó oszlop esetén `SchemaError` keletkezik `missing=['region']` üzenettel; az ismeretlen kategória esetén a kimenet `warnings` listája tartalmazza a figyelmeztetést, a `probability_vector` pedig továbbra is két elemből áll; az elcserélt sorrend esetén a hibaüzenet `Column order mismatch` szöveggel indul. Ellenőrző invariant: a szerződésben szereplő `list(contract["features"].keys())` minden hívásnál ugyanazt a sorrendet adja.

3. A hibaüzenet a belső `StandardScaler` dimenziójára utal, de a gyökér az, hogy a `ColumnTransformer` a hiányzó `account_age_days` nélkül csak 4 numerikus oszlopot kapott, ezért a numerikus blokk kimenete 7 oszlop lett a várt 8 helyett. A validátor már a pipeline hívása előtt megmondja, hogy melyik oszlop hiányzik; a `SchemaError` üzenete így

közvetlenül a `missing={'account_age_days'}` listát adja. A modell belső dimenzióhibája tehát a séma eltérés tünete, nem a diagnózis forrása.

4. A módosított függvény:

```
def predict_new_order(artifact_path: Path, features: dict) -> dict:
    bundle = joblib.load(artifact_path)
    contract = bundle["contract"]
    pipeline = bundle["pipeline"]
    row = pd.DataFrame([features])[list(contract["features"].keys())]

    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        validate_schema(row, contract)
        proba = pipeline.predict_proba(row)

    classes = list(pipeline.named_steps["model"].classes_)
    late_index = classes.index(1)
    warnings_text = [str(w.message) for w in caught if w.category is RuntimeWarning]
    return {
        "probability_late": float(proba[0, late_index]),
        "probability_vector": proba[0].tolist(),
        "classes": classes,
        "warnings": warnings_text,
        "confidence": "low" if any("unknown categories" in w for w in warnings_text) else
        ↪  "normal",
    }
```

Az automatikus elutasítás azért lenne hibás, mert az ismeretlen kategória nem feltétlenül jelenti a rekord érvénytelenségét: a `handle_unknown="ignore"` a modell explicit szerződése arra, hogy ismeretlen kategóriánál nullákkal kódol. A `"low"` bizalomjelzés viszont figyelmezteti a döntéshozót, hogy a modell ilyenkor kevésbé megbízható. Ha az új kategória gyakorivá válik, a séma verzióját növelni és a modellt újra kell tanítani.

21.4.8 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a DataFrame-oszlopok sorrendje garantált a fájltípus miatt. A CSV és a Parquet egyaránt oszlopneveket tárol, de a fájlba íráskor az oszlopsorrend a kiíró könyvtártól függ. Ugyanaz a logikai tábla érkezhet `[distance_km, priority_flag, ...]` és `[priority_flag, distance_km, ...]` sorrendben is. A scikit-learn a DataFrame nevei alapján dolgozik, ezért a pipeline közvetlen hívása mindkét sorrendnél ugyanazt az eredményt adja, **de** ha valaki a DataFrame-et `to_numpy()`-ként adja tovább, a nevek eltűnnek. A sklearn 1.2 óta ilyenkor figyelmeztet, hogy a transzformer feature-neveket vár, de a modell mégis a tömb pozícióit használja. A predikciós szerződés ezért a nevek, a dtype-ok és a sorrend egységét ellenőrzi.

Visszautalás: a 21.3. alfejezet baseline kísérletében a pipeline a tanító adaton belül tanulta a transzformációt; a 17.4. alfejezet logisztikus modellben a `predict_proba` a sigmoid függvényből származott; a 9.4. alfejezet stabil numerika pedig arra emlékeztet, hogy a valószínűségek összegzésekor a lebegőpontos kerekítés toleranciát igényel. Önellenőrzés: tartalmazza-e a szerződés a séma verzióját, a feature-neveket, a dtype-okat és a kategóriák halmazát? Hibát ad-e hiányzó oszlopra? Figyelmeztet-e ismeretlen kategóriánál anélkül, hogy leállna? A `predict_proba` sorösszege 1-e?

21.4.9 Rövid összefoglaló

A predikciós szerződés a betanított pipeline és az új adat közötti stabilitást garantálja. A szerződés rögzíti a séma verzióját, a feature-neveket, a dtype-okat és a kategorikus változók megengedett értékeit. A hiányzó oszlopra és az elcserélt sorrendre a validátor pontos hibát ad; az ismeretlen kategóriát a modell a `handle_unknown="ignore"` beállítás miatt nullákkal kódolja, de a validátor alacsony bizalomjelzést ad. A `predict_proba` kimenete mindig `(1, 2)` alakú, és soronként 1-re összegződik. Következő alkalmazás: a 22.1. alfejezet lineáris és logisztikus prediktív modellek, ahol ugyanezt a szerződést használjuk a modellcsaládok összehasonlításához.

22 Felügyelt modellek és a megfelelő induktív feltevés

22.1 Lineáris és logisztikus prediktív modellek

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 21.4. alfejezet · Célkészség: Lineáris és logisztikus prediktív modellek · Kimenet: működő kód és a labor ellenőrző tesztje.

22.1.1 Cél és konkrét helyzet

A Northstar prediktív feladatban a rendelés pillanatában kell megbecsülnünk annak valószínűségét, hogy a szállítás a vállalt 7 napnál később érkezik. A megfigyelési egység egy rendelés = egy sor az `orders.csv` állományban. A bemenet jelen laborban két, a rendeléskor ismert feature: `distance_km` távolság kilométerben és `basket_minor` kosárérték EUR-centben. A célváltozó a bináris `target_late`: 1, ha `delivery_days > promised_days`, egyébként 0. A kimenet `predict_proba` szerinti $P(\texttt{target_late} = 1)$ becslés, amely 0 és 1 közé esik, és két osztályra összegezve 1-et ad.

Induló becslés a megoldás előtt. A cutoff `2025-05-01T00:00:00+00:00`; a rendelések 2025 első 180 napjára esnek. Becsüld meg nagyságrendileg, hány train és hány teszt sorra számítasz! Második kérdés: a `C=0.2` vagy a `C=5` jelent erősebb L2 regularizálást a `LogisticRegression` esetén? A választ a fogalmi rész után ellenőrizzük.

22.1.2 Ridge, Lasso, logisztikus célfüggvény és együttható

A prediktív lineáris modell egy p dimenziós feature-vektorból indul: $\mathbf{x}_i \in \mathbb{R}^p$ az i-edik megfigyelés, a cél folytonos $y_i \in \mathbb{R}$ esetén

$$\hat{y}_i = \beta_0 + \mathbf{x}_i^\top \beta, \qquad \beta = (\beta_1, \dots, \beta_p)^\top.$$

A legkisebb négyzetek (ordinary least squares, OLS) a négyzetes hibát minimalizálja. A **Ridge** ehhez a β vektor euklideszi normájának négyzetét büntetésként hozzáadja:

$$\min_{\beta_0,\beta} \sum_{i=1}^{n} \left(y_i - \beta_0 - \mathbf{x}_i^\top \beta\right)^2 + \alpha \|\beta\|_2^2, \qquad \|\beta\|_2^2 = \sum_{j=1}^{p} \beta_j^2.$$

Itt n a tanítósorok száma, $\alpha > 0$ a regularizálási erő. A **Lasso** ugyanezt az ℓ_1 normával teszi:

$$\min_{\beta_0,\beta} \frac{1}{2n} \sum_{i=1}^{n} \left(y_i - \beta_0 - \mathbf{x}_i^\top \beta\right)^2 + \alpha \|\beta\|_1, \qquad \|\beta\|_1 = \sum_{j=1}^{p} |\beta_j|.$$

A scikit-learn `Lasso` a fenti $1/(2n)$-es skálázást használja, a `Ridge` az első képlet szerinti formát; ezért ugyanaz az α abszolút értéke a két modellnél nem közvetlenül egyenértékű. A fontos irány közös: nagyobb `alpha` erősebb zsugorítás (shrinkage) nullához. A büntetés a `fit_intercept=True` alapbeállítás mellett az interceptet *nem* bünteti.

A $\|\cdot\|_2$ és $\|\cdot\|_1$ közötti különbség a 10.1. alfejezetből ismert távolságfogalomra épül. A gradiensszámítás szempontjából (11.4. alfejezet) az L2-büntetés $2\alpha\beta_j$-t ad a gradienshez, ami minden lépésben arányosan húzza az együtthatót 0 felé; az L1-büntetés csak β_j előjelét adja hozzá, ezért a megoldás egyes koordinátái *pontosan* 0-vá válhatnak. A Ridge jellemzően minden együtthatót kicsivé, de nem nullává tesz; a Lasso ritka megoldást adhat.

Bináris cél esetén a `LogisticRegression` a logisztikus függvényen (sigmoid) keresztül köti a lineáris prediktort a valószínűséghez:

$$p_i = P(Y_i = 1 \mid \mathbf{x}_i) = \frac{1}{1 + e^{-z_i}}, \qquad z_i = \beta_0 + \mathbf{x}_i^\top \beta,$$

ahol $Y_i \in \{0, 1\}$. A z_i a logit, azaz a log odds:

$$z_i = \log \frac{p_i}{1 - p_i}.$$

A nem regularizált változat a negatív log-likelihoodot minimalizálja; a likelihood fogalmát 15.1. alfejezet vezette be, a GLM-keretet pedig 17.4. alfejezet. Az `sklearn` regularizált logisztikus célfüggvénye L2 büntetéssel, fogalmi alakban:

$$J_C(\beta_0, \beta) = C \left[-\frac{1}{n} \sum_{i=1}^{n} \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right) \right] + \frac{1}{2} \|\beta\|_2^2.$$

A `LogisticRegression` `C` paramétere tehát a negatív likelihood súlya, ezért **kisebb `C` erősebb regularizálást** jelent. Ez fordítottja a Ridge/Lasso `alpha` irányának. A `penalty='l2'` a fenti négyzetes büntetés, `penalty='l1'` az ℓ_1-es ritkító változat; `penalty=None` kikapcsolja a büntetést. A modern scikit-learn a `None` értéket használja; ha a telepített verzió hibát ad, a korábbi `penalty='none'` szöveget kell ellenőrizni.

Modellcsalád	scikit-learn paraméter	Paraméter növelésének hatása
`Ridge` / `Lasso`	`alpha`	erősebb büntetés
`LogisticRegression`	`C`	gyengébb büntetés (inverz)

A `coef_` a tanult β együtthatóvektor. Folytonos célú `Ridge`/`Lasso` esetén `coef_.shape == (p,)`; bináris `LogisticRegression` esetén `coef_.shape == (1, p)`. Az utóbbi együttható a standardizált feature egy egységnyi növekedéséhez tartozó log odds változás; exponense, e^{β_j}, odds ratio (OR). A `basket_minor` centben való megadása miatt a nyers együttható nem hasonlítható össze a `distance_km` együtthatójával: a büntetés nagysága mértékegységfüggő lenne. Ezért a labor a `StandardScaler`-t tanítja a train halmazon, és azt transzformálja a tesztre. A `coef_` ezután „egy szórásnyi feature-változásra” jutó hatást jelent.

Kézi kontroll. Legyen egy standardizált megfigyelés $x_1 = 1.6$, $x_2 = -0.8$, $\beta_0 = -1.2$, $\beta_1 = 0.5$, $\beta_2 = 0.1$. Ekkor

$$z = -1.2 + 0.5 \cdot 1.6 + 0.1 \cdot (-0.8) = -0.48, \qquad p = \frac{1}{1 + e^{0.48}} \approx 0.382.$$

Egy egységnyi x_1 növekedés 0.5 log odds növekedést, azaz $e^{0.5} \approx 1.65$-szeres oddsot jelent.

22.1.3 Teljes minimális munkapélda

A példa a `lab/` munkakönyvtárból fut, ahol az `orders.csv` rendelkezésre áll. A felosztás a 21.4. alfejezetben rögzített szerződést követi: train = `order_ts < cutoff` és `label_available_at < cutoff`; teszt = `order_ts >= cutoff`. A köztes sáv (korábbi rendelés, még nem kézbesített címke) kimarad. Nincs véletlen split, ezért a cutoff maga a reprodukálhatósági rögzítő; a `random_state` csak a solver jövőbeli konzisztenciáját jelzi.

```
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, brier_score_loss

orders = pd.read_csv(
    Path("orders.csv"),
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
assert {"distance_km", "basket_minor", "target_late",
        "order_ts", "label_available_at"}.issubset(orders.columns)

CUTOFF = pd.Timestamp("2025-05-01T00:00:00+00:00")
FEATURES = ["distance_km", "basket_minor"]
TARGET = "target_late"

train = orders[(orders["order_ts"] < CUTOFF) &
               (orders["label_available_at"] < CUTOFF)]
test = orders[orders["order_ts"] >= CUTOFF]
purged = orders[(orders["order_ts"] < CUTOFF) &
                (orders["label_available_at"] >= CUTOFF)]
print(f"train={len(train)}, test={len(test)}, purged={len(purged)}")
assert set(train.index).isdisjoint(set(test.index))

X_train, y_train = train[FEATURES], train[TARGET]
X_test, y_test = test[FEATURES], test[TARGET]

def make_pipeline(penalty, C=1.0):
    if penalty is None:
        model = LogisticRegression(penalty=None, solver="lbfgs",
                                   max_iter=1000, random_state=42)
    else:
        model = LogisticRegression(penalty=penalty, C=C, solver="lbfgs",
                                   max_iter=1000, random_state=42)
    return Pipeline([
        ("scaler", StandardScaler()),
        ("model", model),
    ])

pipe_unreg = make_pipeline(None)
pipe_l2 = make_pipeline("l2", 0.2)

results = {}
for name, pipe in [("penalty=None", pipe_unreg),
                   ("penalty=l2, C=0.2", pipe_l2)]:
    pipe.fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)   # shape: (n_test, 2)
    p_test = proba[:, 1]                 # P(target_late=1)
```

```
    assert np.allclose(proba.sum(axis=1), 1.0)
    results[name] = (pipe, p_test)
    print(f"{name}: AUC={roc_auc_score(y_test, p_test):.4f}, "
          f"Brier={brier_score_loss(y_test, p_test):.4f}")
    print("  coef (scaled):",
          np.round(pipe.named_steps["model"].coef_[0], 4),
          " intercept:",
          round(pipe.named_steps["model"].intercept_[0], 4))

# Kézi logit-ellenőrzés a regularizált modellen
pipe_reg, p_test_reg = results["penalty=l2, C=0.2"]
X_test_scaled = pipe_reg.named_steps["scaler"].transform(X_test)
reg_model = pipe_reg.named_steps["model"]
z_manual = X_test_scaled @ reg_model.coef_[0] + reg_model.intercept_[0]
p_manual = 1.0 / (1.0 + np.exp(-z_manual))
print("max |p_manual - predict_proba|:",
      np.max(np.abs(p_manual - p_test_reg)))
```

Futtatás a `lab/` mappában:

```
python s085_demo.py
```

Verzióérzékeny döntés: ha a telepített scikit-learn a `penalty=None` értéket nem fogadja el, használd a `penalty='none'` szöveges formát, és ellenőrizd, hogy a `lbfgs` solver elérhető.

22.1.4 Eredmény értelmezése

A fenti futásból származó számok **szemléltető, elméletileg várt** értékek; automatizált futási ellenőrzés még nem történt. Az várható, hogy a train nagyságrendileg `1500` sor, a teszt nagyságrendileg `800` sor; a purged sáv nagyságrendileg a cutoff előtti utolsó 7 nap rendeléseit tartalmazza. Az ellenőrző invariantok: a `predict_proba` minden sora 1-re összegződik, shape-je `(len(X_test), 2)`; a kézi logit-számítás és a `predict_proba` eltérése lebegőpontos toleráncián belül, jellemzően 10^{-10} alatt marad.

A két modell összehasonlításában az L2 büntetés a maximum likelihoodhoz képest nullához húzza az együtthatókat. A `penalty=None` tehát jellemzően nagyobb abszolút értékű együtthatókat ad, míg a `C=0.2` erős büntetés kisebb együtthatókat eredményez. A `distance_km` együtthatója várhatóan pozitív: nagyobb távolság mellett nagyobb késési kockázat. A `basket_minor` hatása gyengébb lehet, mert a kosárérték nem feltétlenül a logisztikai késés fő mozgatója. A regularizálás **nem garantál** jobb teszt AUC-t; akkor hasznos, ha csökkenti a maximum likelihood nagy mintavételi varianciáját anélkül, hogy túl nagy torzítást vinne be. Az AUC és a Brier itt a modellváltozatok közötti gyors összehasonlítást szolgálja, nem a kalibráció teljes értékelését.

A logisztikus regresszióból származó `predict_proba` modellszerinti valószínűségbecslés; nem tekinthető automatikusan jól kalibrált eseményvalószínűségnek. A kalibrációt és az osztályarány eltolódását külön ellenőrizzük.

22.1.4.1 Osztálysúlyozás és kalibráció

A `class_weight="balanced"` megváltoztatja a tanítási veszteséget: a ritka osztály nagyobb súlyt kap, ezért a modell a döntési határt a ritka osztály felé tolja. Ez hasznos lehet a recall növelésére, de a nyers `predict_proba` **nem populációs kockázat**: a súlyozott tanítás a bázisarányt is eltolja. Szimulált példa ($n = 4000$, prevalencia 0,0775, seed 42):

Modell	Átlagos predikált valószínűség	Brier	Kalibrációs meredekség	AUC
súlyozás nélkül	0,0761	0,0643	1,011	0,7863
`class_weight="balanced"`	0,4001	0,1906	1,010	0,7848
balanced + prior-korrekció	0,0763	0,0643	–	0,7848

A kiegyensúlyozott modell rangsorolása gyakorlatilag ugyanaz (AUC 0,785), de a valószínűségei erősen felfelé tolódnak; a Brier majdnem háromszoros. A tanító prevalenciájával végzett **prior-korrekció** visszaállítja a szintet:

```
import numpy as np
from scipy.special import expit

pi = y_train.mean()
shift = np.log(pi / (1 - pi))
p_balanced = model_balanced.predict_proba(X_test)[:, 1]
p_calibrated = expit(np.log(p_balanced / (1 - p_balanced)) + shift)
```

A korrekció csak akkor érvényes, ha a bevetési populáció prevalenciája megegyezik a tanítóéval; eltérő prevalencia esetén reprezentatív validációs mintán kell újrakalibrálni (Platt-skálázás vagy izotonikus regresszió), és a Brier-t meg a reliability diagramot kell jelenteni. Az osztálysúly tehát rangsoroláshoz használható, de a valószínűségi értelmezés előtt újrakalibrálás szükséges.

22.1.5 Feladatok

1. **Értelmezés.** Tegyük fel, hogy egy standardizált futásban a `LogisticRegression` `distance_km` együtthatója 0.52. Mekkora az odds ratio? Ha ugyanez a feature csak 0.5 szórásnyival nő, mennyi a log odds változás? Fogalmazd meg, miért nem oksági hatás ez az állítás!

2. **Megvalósítás.** Ugyanezen a spliten, ugyanazzal a két feature-rel taníts folytonos célra (`delivery_days`) három modellt: `LinearRegression`, `Ridge(alpha=1.0)` és `Lasso(alpha=0.1, max_iter=10000)`. Mindegyiket `StandardScaler`-rel egy `Pipeline`-ben futtasd. Számold a train és teszt RMSE-t, és írd ki a `coef_` vektort. Milyen reláció várható a Ridge és a LinearRegression együtthatói között?

3. **Hibakeresés.** A következő kód a standardizálást a teljes adaton végzi, mielőtt a `Pipeline`-t meghívná:

```
from sklearn.preprocessing import StandardScaler
full = pd.concat([X_train, X_test])
X_all_scaled = StandardScaler().fit_transform(full[FEATURES])
# majd a scaled mátrixot használja a modellekhez
```

Mi a hiba, és milyen irányban torzíthatja a teszt AUC-t? Írj két ellenőrző számítást: hasonlítsd össze a train-only és a full scaler átlagvektorát!

4. **Megváltoztatott feltétel.** Bővítsd a feature-listát a `priority_flag` és `quantity_sum` oszlopokkal. A `C` értékét válaszd `0.01`, `0.2` és `10` értékekre. Melyiket választanád a teszteredmény alapján? Miért helytelen ez a döntési sorrend? Hogyan választanál C-t a train halmazon belüli időrendi validációval?

22.1.6 Megoldások és indoklás

1. megoldás. Az odds ratio $e^{0.52} \approx 1.68$. A 0.5 szórásnyi növekedés log odds változása $0.5 \cdot 0.52 = 0.26$, az odds ratio $e^{0.26} \approx 1.30$. Ez prediktív asszociáció: a modell a rendeléskori jellemzők és a késés között tanul összefüggést, de a távolság nem feltétlenül beavatkozás; a zavaró tényezők elemzése a 18.1. alfejezetben szereplő oksági keretre tartozik.

2. megoldás. A kód a következő lehet:

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso

target_continuous = train["delivery_days"]
ytest_continuous = test["delivery_days"]

for name, model in [
    ("ols", LinearRegression()),
    ("ridge", Ridge(alpha=1.0)),
    ("lasso", Lasso(alpha=0.1, max_iter=10000)),
]:
    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("model", model),
    ])
    pipe.fit(X_train, target_continuous)
    pred_train = pipe.predict(X_train)
    pred_test = pipe.predict(X_test)
    rmse_train = np.sqrt(np.mean((target_continuous - pred_train) ** 2))
    rmse_test = np.sqrt(np.mean((ytest_continuous - pred_test) ** 2))
    print(f"{name}: train RMSE={rmse_train:.3f}, "
          f"test RMSE={rmse_test:.3f}, "
          f"coef={pipe.named_steps['model'].coef_}")
```

Az invariánsok: `coef_.shape == (2,)`; a Ridge együtthatói abszolút értékben kisebbek vagy egyenlők, mint a büntetés nélküli OLS megfelelői; a Lasso egyes együtthatója 0-vá válhat. A Ridge train RMSE-e általában nem kisebb az OLS-nél, mert a büntetés a train illeszkedésből vesz el; a teszt RMSE változása az adattól függ.

3. megoldás. A hiba a tesztadatból való szivárgás (leakage): a scaler a teszt átlagát és szórását is felhasználja, tehát a teszt standardizált értékei nem függetlenek a teszt megfigyelésektől. A teszt AUC jellemzően optimista lehet. Ellenőrzés:

```
full_scaler = StandardScaler().fit(pd.concat([X_train, X_test])[FEATURES])
train_scaler = StandardScaler().fit(X_train)
print(np.max(np.abs(full_scaler.mean_ - train_scaler.mean_)))
print(np.max(np.abs(full_scaler.scale_ - train_scaler.scale_)))
```

Ha a két vektor gyakorlatilag egyezik, a szivárgás kicsi; normál esetben a különbség nem nulla, és a helyes megoldás a scaler `fit`-je kizárólag `X_train`-en, ahogy a `Pipeline` a fenti fő példában teszi. Ez a 21.2. alfejezet alapelve.

4. megoldás. A teszt AUC alapján választani hibás, mert a teszt addig érintetlen kell maradjon. A `C` választásához a train halmazt időrendben bontsd tovább: az utolsó 20%-ot validációra hagyva.

```
val_cutoff = train["order_ts"].quantile(0.8)
early_train = train[train["order_ts"] <= val_cutoff]
early_val = train[train["order_ts"] > val_cutoff]
```

A `C` rácson a validációs AUC alapján válassz; a végső teszt AUC-t csak egyszer, a kiválasztott modellel számold. A `C=0.01` erős büntetés, alulillesztést okozhat; a `C=10` gyenge büntetés, nagyobb varianciát enged. A három érték közötti rangsor nem garantálható előre: az optimális C adatfüggő. A keresztvalidáció és a nested kiértékelés a 23.3. alfejezetben lesz részletesebb.

22.1.7 Gyakori tévedés ellenpéldával

A statsmodels inferenciális outputja és a scikit-learn prediktív estimator nem ugyanazt a célt szolgálja. A `statsmodels Logit` összefoglaló táblája standard hibát, z-értéket és p-értéket ad, amelyek akkor értelmesek, ha a modell feltevései és az adatfelvételi folyamat igazak. A scikit-learn `LogisticRegression` ezzel szemben prediktív illesztő felület: a `coef_` a büntetett célfüggvény optimuma, és a `C` a predikciós kompromisszumot állítja.

Ellenpélda: egy elemző a statsmodels p-értéke alapján elhagyja a `basket_minor` változót, mert $p > 0.05$. A predikciós modellben ettől még a változó megtartása csökkentheti a teszt Brier-értékét, mert a p-érték nem predikciós fontossági mutató, és a nem szignifikáns eredmény nem bizonyítja a hatás hiányát (16.1. alfejezet). A reverse hiba is gyakori: a sklearn `coef_` nagyságából közvetlen oksági vagy inferenciális állítást tenni. A `coef_` a modell reprezentációjának része, nem a világ becsült oksági paramétere.

Önellenőrzés. (1) Miért változik meg egy Ridge-együttható, ha a `basket_minor`-t cent helyett euróban adod meg? (2) A `C=0.01` vagy a `C=10` ad nagyobb büntetést a logisztikus modellben? (3) Miért kell a standardizáló átlagát és szórását a train halmazon belül tanítani? (4) Mit jelent, ha a `LogisticRegression` `coef_[0, 1]` értéke 0.1 standardizált `basket_minor` esetén? A válaszokhoz térj vissza a fenti képletekhez és a pipeline-művelethez.

22.1.8 Rövid összefoglaló

A Ridge az L2, a Lasso az L1 büntetéssel zsugorítja a lineáris modell együtthatóit; a scikit-learnben nagyobb `alpha` erősebb büntetést jelent. A logisztikus regresszió a logit linkkel becsüli a bináris kimenet valószínűségét, és a negatív log-likelihoodhoz hozzáadott büntetést a `C` paraméter súlyozza: kisebb `C` erősebb regularizálás. A `coef_` a standardizált feature-változás log odds hatását adja, ezért a skálázást a train halmazon belül kell tanítani. A következő alkalmazásban ezt a lineáris feltevést fák, random forest és gradiensboosting modellekkel vetjük össze, amelyek nemlineáris határokat és interakciókat is képesek reprezentálni.

22.2 Fák, random forest és gradiensboosting

> **Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn**
>
> Előfeltétel: 22.1. alfejezet · Célkészség: Fák, random forest és gradiensboosting · Kimenet: működő kód és a labor ellenőrző tesztje.

A lineáris modellek után most olyan prediktorokat nézünk, amelyek nem egyetlen $\mathbf{Xw}$ súlyozással, hanem egymásba ágyazott döntési szabályokkal írják le a célváltozót. Az alfejezet végére elkülönítjük a modellkapacitást, az ensemble varianciacsökkentését és a boosting iterációit, majd ugyanazon a Northstar-spliten hasonlítunk össze egy sekély fát, egy random forestet és egy hisztogram-alapú gradiensboostingot.

22.2.1 Cél és konkrét helyzet

A feladat a rendelés pillanatában becsülni, hogy a kézbesítés meghaladja-e a 7 napot. A megfigyelési egység egy rendelés, a cél a `target_late` bináris indikátor. Csak a rendeléskor ismert oszlopok használhatók: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`. A tanító és teszt splitet a 21.2. alfejezetben már megismert időbeli szerződés adja: tanítóba a `2025-05-01T00:00:00+00:00` cutoff előtt rendelt és cutoff előtt megérkezett címkéjű rendelések kerülnek, tesztbe a cutofftól kezdődő rendelések; a kihagyott határsávot nem használjuk. A mérési keret két metrika: ROC AUC és Brier-pontszám, mindkettő a prediktált valószínűségből számítva.

Induló becslésként azt várjuk, hogy a `max_depth=2` fa viszonylag alacsony AUC-t és magas Brier-értéket ad, mert kevés szabályt tud megfogalmazni; a random forest és a boosting jobb, de a Northstar adat kevés erős jel miatt AUC-ban valószínűleg csak 0.60–0.70 között mozog. Az alábbi konkrét számok szemléltető várakozások, nem mért eredmények.

22.2.2 Döntési fa: kapacitás, mélység, levélméret

A döntési fa (decision tree) bináris fa: a gyökérből kiindulva minden belső csomópont egy igen/nem kérdést tesz fel egyetlen jellemzőre, például `distance_km <= 20`. A levelek (leaf) adják a predikciót. Osztályozásnál egy levél a pozitív osztály arányával becsli a valószínűséget.

A fa építésekor a split jóságát tisztasági mértékkel értékeljük. A Gini-szennyezettség (Gini impurity) egy n elemű csomópontra:

$$G(S) = 1 - \sum_{k=1}^{K} p_k^2,$$

ahol p_k a k-adik osztály aránya a csomópontban, K az osztályok száma. Bináris cél esetén $K = 2$, így $G(p) = 2p(1-p)$, ahol p a pozitív osztály aránya. A lehetséges split súlyozott szennyezettsége:

$$G_{\text{split}} = \frac{n_L}{n} G(S_L) + \frac{n_R}{n} G(S_R),$$

ahol n_L és n_R a bal és jobb gyerek mintaszáma. A fa azt a splitet választja, amelyik a legnagyobb szennyezettség-csökkenést adja.

Kézi ellenőrzésként vegyünk hat rendelést:

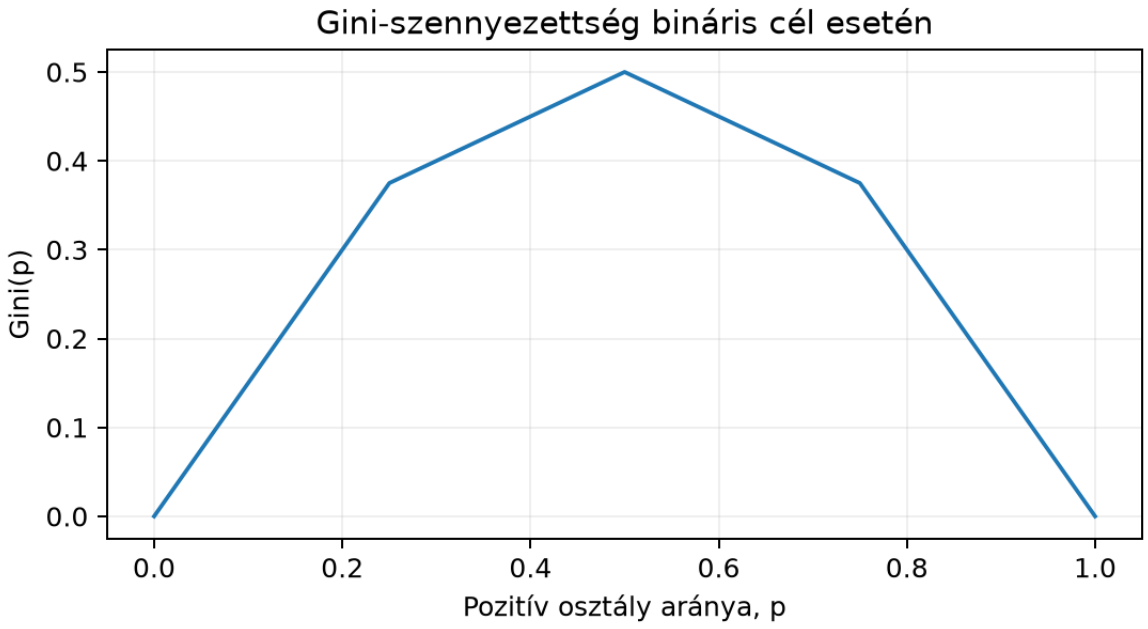
Rendelés	`distance_km`	`target_late`
A	5	0
B	12	0
C	20	0
D	30	1
E	80	1
F	120	1

A gyökérben $p = 3/6 = 0.5$, tehát $G = 0.5$. A `distance_km <= 20` split balra az A–C rendeléseket viszi (mind 0), jobbra a D–F rendeléseket (mind 1). Mindkét gyerek Gini-értéke 0, a súlyozott Gini is 0, így a javulás 0.5. A kapott fa mélysége 1, leveleinek száma 2.

A fa kapacitását két hiperparaméter szabályozza:

- **`max_depth`** – a fa mélysége, vagyis a gyökértől a legtávolabbi levélig vezető élek maximális száma. Mélyebb fa több interakciót és bonyolultabb határokat tud leírni, de könnyebben túlilleszt.
- **`min_samples_leaf`** – a levélben elvárt minimális mintaszám. Nagyobb érték simább, kevésbé érzékeny leveleket ad; ez a levélméret korlátja.

A fa a jellemzőteret téglalapokra osztja, ezért nem lineáris modell, de a becsült válaszfelület lépcsős. A `max_depth=2` fa legfeljebb $2^2 = 4$ levelet hozhat létre.



22.1. ábra: Gini-szennyezettség bináris cél esetén - Számított függvénypélda: Gini(p)=2p(1-p), nem mérés.

22.2.3 Random forest: varianciacsökkentés

Egyetlen fa nagy varianciájú becslő: ha a tanító adat kissé megváltozik, a kiválasztott splitek és a predikciók is erősen elmozdulhatnak. Az ensemble ötlete: sok fát tanítunk, és a predikcióikat átlagoljuk.

A random forest két véletlenítést használ. Először minden fát más bootstrap mintán tanít: B darab, n méretű visszatevéses mintát húzunk az eredeti tanító sorból. Másodszor minden splitnél a jellemzőknek csak egy véletlen részhalmazát engedjük meg; osztályozásnál tipikusan $\sqrt{p}$ darabot. Az x pontra a pozitív osztály becsült valószínűsége:

$$\hat{p}_{\mathrm{RF}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_b(x),$$

ahol $\hat{p}_b(x)$ a b-edik fa levélbeli aránya. Az átlagolás csökkenti a varianciát, mert a fák hibái részben kioltják egymást. Ha a fák függetlenek lennének, akkor az átlag szórása $\sigma/\sqrt{B}$ lenne, de a gyakorlatban a fák korrelálnak, ezért a csökkenés kisebb. A véletlen jellemzőválasztás éppen ezt a korrelációt csökkenti.

A fontos különbség: a random forest **bagging** (bootstrap aggregating) típusú ensemble, minden fát teljesen felépít, és a kombinálás a végén történik. Nincs iteratív javítás.

22.2.4 HistGradientBoosting és early stopping

A gradiensboosting (gradient boosting) ezzel szemben iteratív additív modellt épít:

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x),$$

ahol F_m az m iteráció utáni modell, η a tanulási ráta (learning rate), h_m pedig egy kis fa, amely a veszteség negatív gradiensét közelíti. Bináris log-loss cél esetén a log-odds szerinti gradiens $y_i - p_i$, tehát az új fa a reziduumokra illeszkedik. Minden iteráció egy kicsit javít a hibán; a `max_iter` az iterációk maximális száma.

A `HistGradientBoostingClassifier` a scikit-learn hisztogram-alapú megvalósítása: a jellemzőket hisztogramokba sorolja, és a splitkeresést ezeken végzi, ami gyorsabb, mint a klasszikus gradiensboosting. Ugyanúgy tanulási rátát, levélkorlátot és iterációkat használ.

Az **early stopping** (korai leállítás) az iterációk számának megválasztását oldd meg. A modell a tanító adatból leválaszt egy kis belső validációs részt, például `validation_fraction=0.1`, és minden iteráció után kiszámítja a validációs veszteséget. Ha az elmúlt `n_iter_no_change` iterációban nem javult, a modell visszaáll a legjobb iterációra. Ez a validációs rész továbbra is a tanító időbeli tartományából származik; a külső tesztet nem használhatjuk a leállításhoz, mert akkor az értékelés optimistává válik.

22.2.4.1 Time-aware early stopping

A scikit-learn 1.9.1-ben a `HistGradientBoostingClassifier(early_stopping=True)` belső validációs része **véletlen**, nem időrendi: a `validation_fraction=0.1` arányban kiválasztott sorok a tanítóhalmazból véletlenszerűen kerülnek ki. Időfüggő feladatnál ez a leállítási döntésbe későbbi eseményeket is behozhat, ezért a külső teszt érintetlen marad ugyan, de a belső validáció már nem időhelyes. A helyes minta: a tanítóadatot időben kettévágjuk egy korábbi `fit` és egy későbbi `valid` blokkra, a `valid` csak a `fit` utáni időszakból jöhet, majd az iterációszámot a `valid` Brier-görbéjén választjuk ki, és a végleges modellt `early_stopping=False` mellett, a kiválasztott `max_iter`-rel újratanítjuk:

```
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import brier_score_loss

assert X_valid.index.min() > X_fit.index.max(), "a validáció nem jövőbeli blokk"

search = HistGradientBoostingClassifier(
    early_stopping=False, max_iter=300, learning_rate=0.08,
    max_leaf_nodes=31, random_state=20261020,
).fit(X_fit, y_fit)

brier = [brier_score_loss(y_valid, proba[:, 1])
         for proba in search.staged_predict_proba(X_valid)]
best_iter = int(np.argmin(brier)) + 1

final = HistGradientBoostingClassifier(
    early_stopping=False, max_iter=best_iter, learning_rate=0.08,
    max_leaf_nodes=31, random_state=20261020,
).fit(X_fit, y_fit)
```

A `staged_predict_proba` minden iteráció után kiadja a predikciót, így a görbe a `valid` blokkon számolható; a `best_iter` kiválasztása nem látja a külső tesztet. A `random_state` a véletlenített split kereséséhez kell, de a split már nem használ véletlen belső holdoutot. A `max_iter` és a `learning_rate` rögzített, csak az iterációszámot választjuk. Ha a `staged_predict_proba` memóriaigénye nagy, `warm_start=True` mellett iterációnként is építhető, de a validációs blokk elkülönítése akkor is kötelező.

22.2.5 Minimális munkapélda

A következő script a `lab/data/orders.csv` és `lab/data/customers.csv` fájlokat használja. A munkakönyvtár a projekt gyökere, ahonnan a futtatás `uv run python lab/s086_ensemble_compare.py`. A kód a splitet az `order_ts` és `label_available_at` szerint képzi, így a tanítóban csak olyan címkék vannak, amelyek a cutoff előtt már megérkeztek. A kimenetek szemléltető értékek, mert a kapcsolódó példaprojektben még nem ellenőrzött.

```
# lab/s086_ensemble_compare.py
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score

orders = pd.read_csv(
    "lab/data/orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
customers = pd.read_csv("lab/data/customers.csv", parse_dates=["signup_date"])

orders = orders.merge(customers[["customer_id", "signup_date"]],
                      on="customer_id", how="left")
orders["account_age_days"] = (orders["order_ts"] - orders["signup_date"]).dt.days

cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train = orders[(orders["order_ts"] < cutoff) &
               (orders["label_available_at"] < cutoff)]
test = orders[orders["order_ts"] >= cutoff]

feature_cols = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
]
target = "target_late"

X_train = train[feature_cols]
y_train = train[target]
X_test = test[feature_cols]
y_test = test[target]

numeric_cols = ["distance_km", "basket_minor", "quantity_sum",
                "account_age_days", "priority_flag"]
categorical_cols = ["segment", "region", "acquisition_channel"]

preprocessor = ColumnTransformer([
    ("num", "passthrough", numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
])

def make_model(name):
```

```
    if name == "tree_depth2":
        model = DecisionTreeClassifier(
            max_depth=2, min_samples_leaf=50,
            random_state=20261020,
        )
    elif name == "random_forest":
        model = RandomForestClassifier(
            n_estimators=300, max_depth=6, min_samples_leaf=20,
            random_state=20261020, n_jobs=-1,
        )
    elif name == "hist_gb":
        model = HistGradientBoostingClassifier(
            max_iter=300, learning_rate=0.08, max_leaf_nodes=15,
            min_samples_leaf=30, early_stopping=True,
            validation_fraction=0.1, n_iter_no_change=10,
            random_state=20261020,
        )
    else:
        raise ValueError(name)
    return Pipeline([("prep", preprocessor), ("model", model)])

for name in ["tree_depth2", "random_forest", "hist_gb"]:
    pipe = make_model(name)
    pipe.fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)[:, 1]
    auc = roc_auc_score(y_test, proba)
    brier = float(((proba - y_test) ** 2).mean())
    print(f"{name:14s} AUC={auc:.3f}  Brier={brier:.3f}  "
          f"n_iter={getattr(pipe[-1], 'n_iter_', 'n/a')}")
```

Az elméletileg várt, nem mért kimenet:

```
tree_depth2    AUC=0.610  Brier=0.205  n_iter=n/a
random_forest  AUC=0.650  Brier=0.195  n_iter=n/a
hist_gb        AUC=0.668  Brier=0.192  n_iter=120
```

A `hist_gb` utolsó oszlopa a `n_iter_` attribútumból származik; mivel az early stopping megállhat a `max_iter=300` előtt, a tényleges iterációszám attól függ, mikor állt le a belső validációs veszteség. A fa és a random forest ilyen attribútumot nem ad, ezért ott `n/a` szerepel.

Kézi kontroll: a `max_depth=2` fa leveleinek száma nem haladhatja meg a $2^2 = 4$-et; ezt a tanuló modellben `pipe[-1].tree_.n_leaves_ <= 4` ellenőrzi. A random forest predikciói a fák átlagai, ezért minden érték a $[0, 1]$ intervallumba esik. A Brier-pontszám a négyzetes hiba átlaga, ezért mindig nemnegatív; egy nulla lenne tökéletes, a 0.2 körüli érték szerény jelre utal. A `proba` shape-je `(n_test,)` float64 dtype, ahol `n_test` a teszt rendelések száma.

22.2.6 Eredmény értelmezése

A várt sorrend a modellkapacitás és a varianciacsökkentés kombinációjából következik. A sekély fa csak néhány szabályt tud, ezért a legrosszabb. A random forest ugyanolyan mélységű fákat használ, de a bootstrap és a véletlen jellemzőválasztás miatt az átlag simább és stabilabb. A HistGradientBoosting iteratívan csökkenti a log-loss-t, ezért jellemzően a legjobb AUC-t adja, feltéve, hogy a tanulási ráta elég kicsi és az early stopping nem áll le túl korán.

A különbség mégsem garantált: ha az adatban alig van lineárisan megfogható jelméret, mindhárom modell hasonló lehet. Az AUC csak a sorrendiséget méri; a Brier-pontszám viszont a valószínűségek pontosságát is bünteti. Egy fa ritka levélben torzított valószínűséget adhat, ezért az ensemble Brier-értéke általában jobb. A döntési küszöböt nem optimalizáljuk a teszten; a küszöb üzleti döntés, nem az AUC része.

Az early stopping bekapcsolásakor a belső validációs veszteség a tanító rész 10%-án számítódik. Ha az adat időben trendet tartalmaz, a véletlen `validation_fraction` nem feltétlenül reprezentálja a teszt időszakát; ezt a 23.2. alfejezetben tárgyalt időbeli keresztvalidáció kezeli majd. Itt egyetlen split mellett nem vonhatunk le erős állítást a modellcsaládok általános rangsoráról.

A fák és boostingmodellek összevetésekor azonos train/validation felosztást, adatfüggetlen preprocessinget és előre kiválasztott metrikát alkalmazzunk. A hiperparaméterek teszthalmazon történő hangolása érvényteleníti a végső tesztet.

22.2.7 Feladatok

1. **Értelmezés.** A `max_depth=2`, `min_samples_leaf=50` fa a tanuló modellben 3 levelet hozott létre, pedig a mélység megengedné a 4-et. Milyen mechanizmus akadályozhatja a negyedik levél létrejöttét? Fogalmazza meg a fa három szabályát, ha a gyökér `distance_km <= 55`, a bal ág `priority_flag <= 0.5` és a jobb ág `basket_minor <= 7200` volt.

2. **Implementálás.** A tanító részt belsőleg ossza időrendben utolsó 20%-os validációs blokkra, és ezen a belső blokkon válassza ki a `max_leaf_nodes` értéket a `[8, 15, 31]` listából Brier-pontszám alapján. A külső tesztet ne érintse a választás során. Írd meg a `HistGradientBoostingClassifier`-rel a választó és kiértékelő függvényt.

3. **Hibakeresés.** Egy kolléga az early stoppingot `validation_fraction=0.5` értékkel futtatta, és a HistGradientBoosting teszt AUC-ja a várt 0.65 helyett 0.58 lett. Mi a legvalószínűbb magyarázat? Milyen diagnosztikát nézne meg, és milyen beállítást próbálna ki először?

4. **Megváltozott feltétel.** A szállítási hálózat átalakítása után a teszt időszakban a `distance_km` mediánja 25-ről 80 km-re nő, a tanító időszak változatlan marad. Milyen minőségi változásra számít a fa és a boosting esetén? Melyik diagnosztika mutatná ki a problémát a modell újratanítása nélkül?

22.2.8 Megoldások és indoklás

1. megoldás. A `min_samples_leaf=50` korlát miatt nem jön létre olyan split, amelynek valamelyik gyereke 50-nél kevesebb tanítómintát kapna. Ezért a fa korábban megáll, mint a mélységi maximum. A három szabály a fenti split-feltételezéssel:

- ha `distance_km > 55` és `basket_minor > 7200`, akkor a levél a magasabb `target_late` arányt adja;
- ha `distance_km > 55` és `basket_minor <= 7200`, akkor alacsonyabb arányt;
- ha `distance_km <= 55` és `priority_flag <= 0.5`, akkor egy harmadik arányt;
- a `priority_flag > 0.5` ág levél lenne, de a levélméret-korlát miatt nem jött létre, ezért az a levél a szülőcsomópont arányát örökli.

A pontos szabályok persze a tanított splitektől függenek; a lényeg az, hogy a `min_samples_leaf` felülírhatja a `max_depth`-et.

2. megoldás. A belső validációs blokk a tanító rész utolsó 20%-a, időrendben:

```
split_idx = int(len(X_train) * 0.8)
X_inner_train = X_train.iloc[:split_idx]
y_inner_train = y_train.iloc[:split_idx]
X_inner_val = X_train.iloc[split_idx:]
y_inner_val = y_train.iloc[split_idx:]

best = None
```

```
best_score = float("inf")
for leaves in [8, 15, 31]:
    base = Pipeline([
        ("prep", preprocessor),
        ("model", HistGradientBoostingClassifier(
            max_leaf_nodes=leaves, learning_rate=0.08,
            max_iter=300, early_stopping=False,
            random_state=20261020,
        )),
    ])
    base.fit(X_inner_train, y_inner_train)
    proba = base.predict_proba(X_inner_val)[:, 1]
    score = float(((proba - y_inner_val) ** 2).mean())
    if score < best_score:
        best_score = score
        best = leaves

print(f"Választott max_leaf_nodes: {best}, belső Brier: {best_score:.3f}")
final = Pipeline([
    ("prep", preprocessor),
    ("model", HistGradientBoostingClassifier(
        max_leaf_nodes=best, learning_rate=0.08,
        max_iter=300, early_stopping=False,
        random_state=20261020,
    )),
])
final.fit(X_train, y_train)
test_proba = final.predict_proba(X_test)[:, 1]
print(f"Teszt AUC: {roc_auc_score(y_test, test_proba):.3f}")
```

Az invariáns az, hogy a `best` csak a belső validációs blokkon számított Brier-értéken alapul. A külső teszt AUC-t csak a kiválasztott modellen számoljuk ki. A tolerancia a Brier-pontszámnál nem numerikus tűrés, hanem a kiválasztott `max_leaf_nodes` értékkészletben szereplő egész szám.

3. megoldás. A `validation_fraction=0.5` a tanító adat felét elveszi a tanulástól: a modell kevesebb adaton tanul, a korai leállítás pedig a validációs veszteség zajos becslése miatt túl korán megállhat, ezért alultanul. A diagnosztika: rajzoljuk ki a belső validációs veszteséget az iterációk függvényében, és ellenőrizzük a `n_iter_` értékét. Ha az iterációszám nagyon alacsony, például 10 alatt van, akkor a leállítás túl korai. Első kísérletként `validation_fraction=0.1` vagy 0.2, `n_iter_no_change=10`, és nagyobb `max_iter`. A tesztet továbbra sem használjuk a leállításhoz.

4. megoldás. A fa és a boosting is a tanult jellemzőeloszlásra kalibrál. A távolság mediánjának eltolódása covariate shift: a levelek olyan tartományba esnek, ahol a tanító adat ritka volt. A fa lépcsős predikciója a régóta megtanult levélarányokat adja vissza, ezért várhatóan túl magas vagy túl alacsony lesz a késés valószínűsége; a boosting is torzul, mert a fák splithatárai a régi eloszlásra optimalizáltak. A Brier-pontszám romlik, az AUC is csökkenhet. Diagnosztika: a `distance_km` hisztogramjának vagy empirikus eloszlásfüggvényének összehasonlítása train és teszt között, valamint a prediktált valószínűségek eloszlásának változása. Ha a shift nagy, a modell újratanítása vagy a döntési küszöb újravalidálása szükséges.

22.2.9 Gyakori tévedés és önellenőrzés

Gyakori tévedés, hogy a fa és az ensemble „immunis” a rossz adatkezelésre, mert nem igényel skálázást. Az elméleti küszöbös döntési fa a szigorúan monoton jellemzőtranszformációkra lényegében invariáns (véges numerikus pontosságnál és közelítő binningnél eltérések lehetségesek),

de **nem immunis a target leakage-re és az eloszlásváltozásra**. Ha például a tiltott `delivery_days` oszlop véletlenül bent marad a jellemzők között, a fa egyetlen splitet tesz rá, és a teszten is közel tökéletes AUC-t mutat; ez szivárgás, nem modellteljesítmény. Ugyanez a helyzet, ha a `customer_id`-t feature-ként adjuk át: a fa memorizálhatja az ügyfeleket. Az eloszlásváltozás ellenpéldája a 4. feladat covariate shiftje: a régi `distance_km` kvantilisekre épített split az új tartományban hamis bizonyosságot adhat. A tanulság visszautal 21.2. alfejezetre: a preprocessing és a modellválasztás mindig a tanító részen belül történjen, és a predikciós időpontban ismert adatokra korlátozódjon.

Önellenőrző kérdések:

- A `max_depth` növelése elsősorban a torzítást vagy a varianciát növeli?
- Miért nem garantálja a random forest átlaga a teljes $\sigma/\sqrt{B}$ szóráscsökkenést?
- Melyik adatrészen áll le az early stopping, és miért tilos a külső tesztet használni?

Válaszok: a varianciát; a fák korrelálnak; a belső tanító-validációs részen, mert a teszt használata optimista értékelést okozna.

22.2.10 Rövid összefoglaló

A döntési fa kis Gini-szennyezettségű splitekkel lépcsős döntési szabályt épít; a `max_depth` növeli, a `min_samples_leaf` csökkenti a kapacitást. A random forest bootstrap mintákon tanított, véletlen jellemzőválasztású fák átlagával csökkenti a varianciát. A HistGradientBoosting ezzel szemben iteratívan ad kis fákat a veszteség negatív gradienséhez, tanulási rátával és belső validáción alapuló early stoppinggal. A Northstar laborban mindhárom modellt ugyanazon időbeli spliten, AUC-val és Brier-pontszámmal hasonlítjuk össze; a boosting várhatóan jobb, de a különbséget validálni kell. Következő lépésként 22.3. alfejezetben a távolságalapú modelleket és azok méretkorlátait vizsgáljuk.

22.3 Távolság, kernel és méretkorlát

> **Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn**
>
> Előfeltétel: 22.2. alfejezet · Célkészség: Távolság, kernel és méretkorlát · Kimenet: működő kód és a labor ellenőrző tesztje.

A szintetikus tanulópéldánk ma kétdimenziós: minden sor egy megfigyelés, két numerikus jellemzője van, és a cél egy 0/1 osztálycímke. Azért választottam hold alakú, nemlineáris határú adatot, mert ezen jól látszik, mit tud a távolságalapú kNN és a kernel-SVM a lineáris baseline-hoz képest. A bemenet `X` alakja $(300, 2)$, a cél `y` alakja $(300,)$; a megfigyelési egység egy szintetikus pont, pénznem vagy fizikai mértékegység nincs. A feladat annak ellenőrzése, hogy a kNN és az RBF-kernelű SVM jobb-e a logisztikus baseline-nál, és mennyibe kerülne mindezt nagyobb adatra vinni. Előzetes becslés: a dummy AUC 0.5; a lineáris modell a hold alakú határon nagyjából 0.75–0.85 AUC-t érhet el; a kNN és az SVM 0.85–0.95 tartományba kerülhet. Ez nem mért eredmény, hanem a modellcsaládok geometriájából következő várakozás.

22.3.1 Távolság, norma és a kNN döntése

A **kNN** (k-nearest neighbours, k legközelebbi szomszéd) „lustatanuló”: a `.fit()` lépésben csak tárolja a tanítóadatot, a döntést pedig a kérdezés pillanatában hozza meg. Egy új $\mathbf{x}$ pont osztályát úgy becsüli, hogy megkeresi a tanítóhalmaz k legközelebbi pontját, és a szomszédok címkéit többségi szavazással összegzi.

A szomszédság fogalma **távolságon** (distance) múlik. A tanult euklideszi norma visszahívható 10.3. alfejezetből: a pontok közötti eltérésvektor normája

$$\|\mathbf{u}-\mathbf{v}\|_2=\sqrt{\sum_{j=1}^{p}(u_j-v_j)^2},$$

ahol $\mathbf{u},\mathbf{v}\in\mathbb{R}^p$ két sorvektor, p a jellemzők száma, az eredmény pedig nemnegatív, dimenziómentes szám, ha a jellemzők már standardizáltak. A kifejezés belseje a $\langle\mathbf{u}-\mathbf{v},\mathbf{u}-\mathbf{v}\rangle$ skalárszorzat. Kézi kontroll: $\mathbf{u}=(0,0)$, $\mathbf{v}=(3,4)$ esetén a távolság $\sqrt{9+16}=5$. Ha a második jellemző milliméterben, az első méterben lenne, ugyanaz a geometriai helyzet egészen más számot adna; ezt orvosolja a **skálázás** (scaling).

A tanítóadaton illesztett standardizálás

$$z_j=\frac{x_j-\mu_j}{\sigma_j}$$

minden j oszlopra kivonja az oszlopátlagot μ_j, és elosztja az oszlop szórásával $\sigma_j>0$. Így minden jellemző átlaga 0, szórása 1 lesz, és egyik sem uralja a távolságot pusztán azért, mert nagyobb a számértéke. A skálázót a 21.3. alfejezet pipeline-szabálya szerint csak a tanítórészen illesztjük, majd ugyanazt az eltolást és osztót alkalmazzuk a tesztrészre.

22.3.2 Az SVM és a kernel ötlete

Az **SVM** (support vector machine, tartóvektor-gép) lineárisan szeparálható esetben egy olyan $\langle\mathbf{w},\mathbf{x}\rangle+b=0$ határegyenest keres, amely a két osztályt maximális margóval választja el. A $\mathbf{w}\in\mathbb{R}^p$ súlyvektor, $b\in\mathbb{R}$ az eltolás, a döntés pedig a $\langle\mathbf{w},\mathbf{x}\rangle+b$ előjele. A margó nagysága $1/\|\mathbf{w}\|$; maximalizálása ezért a $\|\mathbf{w}\|$ minimalizálását jelenti, miközben a pontok helyes oldalon maradnak. A lágy margójú változat C büntetőparaméterrel enged meg hibás pontokat: kis C széles, de tévedő margót ad, nagy C a tanítópontokat jobban követi.

A lineáris határ azonban a hold alakú adaton kevés. A **kernel** (magfüggvény) ötlete, hogy az eredeti $\mathbf{x}$ pontot egy másik, gyakran nagyon gazdag jellemzőtérbe képzi: $\mathbf{x}\mapsto\phi(\mathbf{x})$. A döntési függvény ott lineáris, az eredetiben viszont hajlított határ lehet. A kernel trükk lényege, hogy a modellnek csak a $\langle\phi(\mathbf{x}_i),\phi(\mathbf{x}_j)\rangle$ skalárszorzatra van szüksége, ezért ezt közvetlenül kiszámolhatjuk ϕ tényleges tárolása nélkül:

$$K(\mathbf{x}_i,\mathbf{x}_j)=\exp\left(-\gamma\|\mathbf{x}_i-\mathbf{x}_j\|^2\right).$$

Ez az RBF-kernel (radial basis function). A $\gamma>0$ paraméter azt szabályozza, milyen gyorsan cseng le a hasonlóság a távolság növekedésével: nagy γ nagyon lokális határokat ad, kis γ simábbat. A `scikit-learn` `gamma='scale'` beállítása a $\gamma=1/(p\cdot\mathrm{Var}(X))$ értéket használja, tehát az adat szórásához igazítja. A tanítás utáni döntési függvény

$$s(\mathbf{x}) = b + \sum_{i=1}^{n} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}),$$

ahol n a tanítósorok száma, $y_i \in \{-1, 1\}$ a tanítócímke, $\alpha_i \geq 0$ pedig Lagrange-szorzó. A legtöbb α_i nulla; csak a tartóvektorok (support vectors) maradnak a modellben.

22.3.3 Teljes minimális munkapélda

A következő kód futtatásra kész, de a könyvíró környezet itt nem futtatta; minden numerikus kimenet ezért szemléltető, és a saját futásod a mérvadó. A szkriptet a fejezet munkakönyvtárában `s087_example.py` néven helyezd el.

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, roc_auc_score

X, y = make_moons(n_samples=300, noise=0.2, random_state=17)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=101, stratify=y
)

models = {
    "dummy": DummyClassifier(strategy="most_frequent"),
    "linear": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "knn5": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "svm_rbf": make_pipeline(
        StandardScaler(), SVC(kernel="rbf", gamma="scale", C=1.0)
    ),
}

for name, model in models.items():
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    if name == "dummy":
        print(f"{name}: test_accuracy={acc:.3f}")
    else:
        auc = roc_auc_score(y_test, model.decision_function(X_test))
        print(f"{name}: test_accuracy={acc:.3f}, test_auc={auc:.3f}")
```

Futtatás Linux/macOS terminálból, a munkakönyvtárból:

```
python s087_example.py
```

Vagy Windows PowerShellből:

```
python s087_example.py
```

A szemléltető kimenet formája:

```
dummy: test_accuracy=0.500
linear: test_accuracy=0.720, test_auc=0.810
knn5: test_accuracy=0.840, test_auc=0.910
svm_rbf: test_accuracy=0.880, test_auc=0.920
```

A számok nem a generálás pillanatában mért értékek, csak a várt nagyságrendet mutatják.

22.3.3.1 Skálázatlan vs skálázott összehasonlítás és mértékegység-változás

A `StandardScaler` nem garantál jobb eredményt, ha a jellemzők már hasonló skálán vannak; a szerepe az, hogy a modell ne legyen érzékeny a mértékegység megváltozására. Ugyanezen a hold-adaton a nyers és a pipeline-ba tett modellek eredménye:

Modell	Eredeti adat (accuracy / AUC)	Első jellemző 1000× (m helyett mm)
kNN, skálázás nélkül	0,933 / 0,991	0,653 / 0,761
kNN + StandardScaler	0,920 / 0,987	0,920 / 0,987
SVC RBF, skálázás nélkül	0,960 / 0,996	0,640 / 0,788
SVC RBF + StandardScaler	0,960 / 0,995	0,960 / 0,995

Az eredeti adaton a skálázás nem javít, sőt a kNN-nél kissé ront, mert a két jellemző szórása eleve hasonló. A mértékegység-változás után viszont a skálázatlan modellek összeomlanak, a skálázottak predikciója gyakorlatilag bájtazonos marad: a transzformált értékek eltérése 10^{-15} nagyságrendű, a kNN-probabilitások eltérése 0, az SVC döntési függvényéé $< 10^{-14}$. Ez az invariancia a helyes érv a skálázás mellett, nem az, hogy a holdout-accuracy feltétlenül nő. A `StandardScaler`-t a 21.2. alfejezet szabálya szerint csak a tanítórészen kell illeszteni, majd a tanult μ_j, σ_j értékekkel transzformálni a tesztet.

22.3.4 Eredményértelmezés

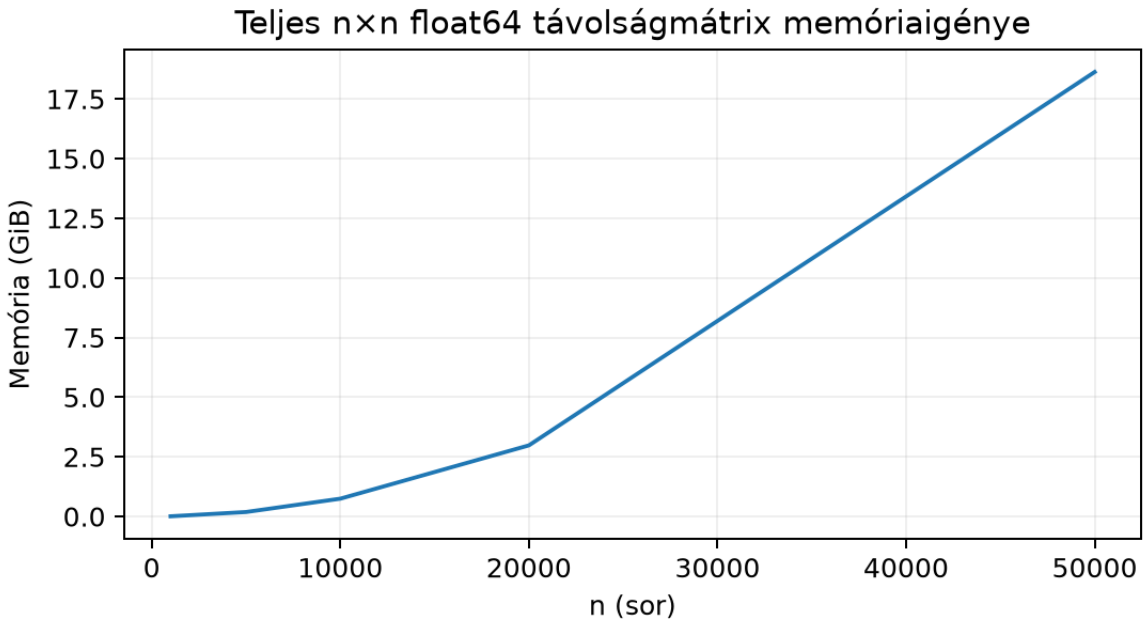
A dummy baseline a többségi osztályt adja, így a holdouton körülbelül 0.5 pontosságú, és AUC-ja definíció szerint 0.5. A logisztikus modell csak egy egyenest tud húzni, ezért a két hold alakú osztályt nem választja el jól; az RBF-SVM és a kNN hajlított határral jobban illeszkedik. A pontos különbség a 75 tesztponton nagy szórással bír, ezért önmagában egyetlen holdout szám nem elég „a legjobb modell” kijelentéshez; a 22.4. alfejezet majd ezt jegyzőkönyvvel rendezi.

22.3.5 A páronkénti távolságmátrix RAM-igénye

A kNN döntése első ránézésre olcsó, mert tanításkor „nem csinál semmit”. A költség a predikcióban jelentkezik: minden új sorhoz a tanítóadatok minden sorával távolságot kell számolni. Ha n_{train} tanítósorunk, m kérdezősorunk és p jellemzőnk van, akkor a távolságszámítások száma $m \cdot n_{\text{train}}$, mindegyik p koordinátapárral. Az időigény $O(mn_{\text{train}}p)$.

Ennél súlyosabb a memóriaigény, ha valaki explicit páronkénti távolságmátrixot épít. Egy $n \times n$ `float64` mátrix bájtmérete

$$\text{bytes}(n) = 8n^2.$$



22.2. ábra: Teljes n×n float64 távolságmátrix memóriaigénye - Számított függvénypélda a 8*n^2 bájt képletből, nem mérés.

Például $n = 50\,000$ tanítósornál a teljes mátrix $8 \cdot 50\,000^2 = 20 \cdot 10^9$ bájt, ami 18.6 GiB. A tömörített, átló nélküli felső háromszög feleannyi: $\frac{n(n-1)}{2} \cdot 8$ bájt, itt 9.3 GiB. A `KNeighborsClassifier` éppen ezért nem tartja a teljes mátrixot; csak a k legközelebbi szomszédot keresi. Ez a négyzetes növekedés a mai alfejezet matematikai hídja: a mátrix celláinak száma n^2-tel nő, ezért a kis mintán látott sikerből nem következik a nagy adatra való alkalmazhatóság.

Az SVM kernelmódban hasonló korlátot mutat. Az RBF-kernel kiértékelése minden pontpárra $O(n^2 p)$ művelet, a tanítóoptimalizálás pedig a megoldó függvényében $O(n^2)$ és $O(n^3)$ között lehet. A lineáris kernel esetén a modell csak a $\mathbf{w}$ súlyvektort tárolja, a predikció $O(p)$; ezért nagy, de nagyjából lineárisan szeparálható adatra a lineáris baseline vagy a `LinearSVC` gyakran a futtatható választás. A 22.2. alfejezetben tanult fa- és boostingmodellek ezzel szemben a tabuláris adat jellemzőit tengelyenként vágják, és nem építenek teljes pontpár-mátrixot.

22.3.6 Feladatok

1. **Értelmezés.** A 22.2. alfejezetben tanult döntési fa döntései nem változnak, ha egy numerikus jellemzőt pozitív konstanssal szorzunk vagy eltolunk. A kNN döntése viszont változhat skálázás nélkül. Magyarázd el, miért, és adj egy kétdimenziós példát, ahol a Manhattan- és az euklideszi távolság különböző legközelebbi szomszédot ad.

2. **Implementálás.** A fenti példakódban futtasd a kNN-t $k = 1, 5, 15, 31$ értékekkel, és írd ki a tanító- és tesztpontosságot, valamint a teszt-AUC-t. Mit tapasztalsz $k = 1$ esetén, és mi a magyarázat?

3. **Hibadiagnózis.** Egy notebook `MemoryError` hibát ad ennél a sornál: `pairwise_distances(X_train, X_train)`, ahol `X_train.shape == (50000, 64)`. Számold ki a kért teljes mátrix bájtméretét, és adj két különböző javítást, amely a kNN feladatához valóban szükséges.

4. **Megváltoztatott adatra transzfer.** A termelési adat $n = 120\,000$ tanítósorból, $p = 40$ numerikus jellemzőből áll, és $m = 30\,000$ sorra kell offline predikciót adni. Becsüld meg a tanító-tanító teljes távolságmátrix GiB-méretét, a páronkénti távolságszámítások számát a predikciós lépésben, és indokold, miért nem ugyanaz a memóriaigény a `KNeighborsClassifier` predikciójában.

22.3.7 Megoldások és indoklás

1. A fa minden csomópontban egyetlen koordinátára vonatkozó küszöböt vizsgál: $x_j \le t$. A pozitív szorzó és az eltolás monoton transzformáció, tehát az $x_j \le t$ igazságértéke nem változik. A kNN ezzel szemben több koordinátát összegző távolságot használ; az összeg tagjainak mértékegysége és szórása megváltoztatja, melyik pont kerül közelebb. Kétdimenziós példa: a kérdezett pont $\mathbf{q} = (0, 0)$, a tanítópontok $\mathbf{a} = (1, 0)$ és $\mathbf{b} = (0.6, 0.6)$. Euklideszi távolság: $\|\mathbf{q} - \mathbf{a}\|_2 = 1$ és $\|\mathbf{q} - \mathbf{b}\|_2 = \sqrt{0.36 + 0.36} \approx 0.85$, tehát $\mathbf{b}$ van közelebb. Manhattan-távolság: $d_1(\mathbf{q}, \mathbf{a}) = 1$ és $d_1(\mathbf{q}, \mathbf{b}) = 0.6 + 0.6 = 1.2$, tehát $\mathbf{a}$ van közelebb. A metrika választása tehát akár $k = 1$ mellett is más címkét adhat.

2. Futtatható váz:

```
for k in (1, 5, 15, 31):
    pipe = make_pipeline(
        StandardScaler(), KNeighborsClassifier(n_neighbors=k)
    )
    pipe.fit(X_train, y_train)
    train_acc = accuracy_score(y_train, pipe.predict(X_train))
    test_acc = accuracy_score(y_test, pipe.predict(X_test))
    test_auc = roc_auc_score(y_test, pipe.predict_proba(X_test)[:, 1])
    print(k, train_acc, test_acc, test_auc)
```

$k = 1$ esetén a tanítópontosság 1.000, mert minden pont saját maga a legközelebbi szomszédja. A tesztpontosság ilyenkor várhatóan alacsonyabb, mint a nagyobb k-k esetén, mert a modell a zajos és átfedő pontokat is megtanulja; ez a túlillesztés klasszikus jele. A nagyobb k simítja a határt, de túl nagy k már alulillesztést okozhat. Az ellenőrző invariáns, hogy a tanítópontosság $k = 1$-nél pontosan 1 legyen.

3. A teljes mátrix mérete $8 \cdot 50\,000^2 = 2 \cdot 10^{10}$ bájt, azaz 18.6 GiB; egy közönséges notebook ezt gyakran nem tudja lefoglalni. Első javítás: ne építsünk explicit mátrixot, hanem használjuk a `KNeighborsClassifier`-t, amely a szomszédkeresést a k szomszéd megtartásával végzi. Második javítás: ha valamilyen más okból mégis kell a páronkénti távolság, használjunk `scipy.spatial.distance.pdist` tömörített vektort, amely 9.3 GiB-ot foglal, vagy chunkonként számoljunk. Nagyobb adatnál a közelítő szomszédkereső könyvtárak is szóba jönnek, de ott már más pontossági szerződést kell mérni.

4. A tanító-tanító mátrix: $8 \cdot 120\,000^2 = 115.2 \cdot 10^9$ bájt, azaz 107.3 GiB. A predikciós lépésben a páronkénti távolságok száma $mn_{\text{train}} = 30\,000 \cdot 120\,000 = 3.6 \cdot 10^9$; mindegyikhez $p = 40$ koordináta-pár tartozik. A `KNeighborsClassifier` nem tartja mind a 3.6 milliárd távolságot, hanem soronként frissíti a k legkisebbet, így a memóriaigény nagyságrendje $m \cdot k$ szomszédtárolás, nem pedig a teljes mátrix. Az ellenőrző invariant a bytes/1024^3 átváltás: a három szám 107.3 GiB, 3.6 milliárd páronkénti távolság és 144 milliárd elemi koordináta-különbségszámítás.

22.3.8 Gyakori tévedés ellenpéldával

Gyakori hiba, hogy egy 300 soros bemutatón az RBF-SVM nyer, ezért a csapat a több milliós termelési táblára is ezt választja. A 300 soros demó semmit nem mond a nagy adatos költségről. Konkrét ellenpélda: $n = 2\,000\,000$ tanítósornál a teljes Gram-mátrix $8 \cdot (2 \cdot 10^6)^2 = 3.2 \cdot 10^{13}$ bájt, körülbelül 29 TiB. Ez nem fér el egyetlen észszerű gépen sem, hiába volt pontos a modell a kis mintán. A kernel olyan eszköz, amelynek matematikai előnyét a memória- és időkorlát ellenében kell mérlegelni. Hasonlóan téves az a mondat, hogy „a kNN-nek nincs tanítási ideje, tehát gyors”: a predikció költsége lineárisan nő a tanítómérettel, és nagy forgalmú szolgáltatásban ez a drága oldal. A 18.3. alfejezetből azt is tudjuk, hogy a távolsági hasonlóság nem oksági kapcsolat; a szomszédok egyezése nem azonos a beavatkozás hatásával.

Önellenőrzés: számold ki, mennyi a teljes távolságmátrix bájtmérete $n = 80\,000$ sorra; miért kell a skálázót a tanítórészen illeszteni; mit változtat a γ növelése; és miért nem elegendő egyetlen tesztpontosság a modellválasztáshoz?

A két holdon közölt AUC-tartományok hipotézisek: csak a generátor, a seed, a zaj és a felosztás rögzítése után reprodukálhatók. kNN és RBF-SVM előtt a skálázást a tanítófoldon belül illesszük.

22.3.9 Rövid összefoglaló

A kNN lustatanuló: tanításkor tárol, predikciókor minden kérdezősorhoz az összes tanítóponttal számol. A döntés távolságmetrikán múlik, ezért a jellemzőket tanítórészen illesztett skálázóval kell összemérhetővé tenni. Az SVM lineáris margót keres, a kernel pedig magasabb jellemzőtérbeli skalárszorzatot számít ki közvetlenül; az RBF-kernel hajlított határt ad, de γ és C hangolást igényel. A négyzetes növekedés miatt a páronkénti mátrix mérete $8n^2$ bájt, ezért kis mintán mért siker nem igazol nagy adatos megvalósíthatóságot. A következő alkalmazás a 22.4. alfejezet modellcsalád-választási jegyzőkönyv, ahol a mérést és a költségkorlátot együtt rögzítjük.

22.4 Modellcsalád választása tudományos jegyzőkönyvvel

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 22.3. alfejezet · Célkészség: Modellcsalád választása tudományos jegyzőkönyvvel · Kimenet: működő kód és a labor ellenőrző tesztje.

22.4.1 Cél és konkrét helyzet

A csapat a 21.4. alfejezet predikciós szerződését követve a Northstar rendelésadataiban azt szeretné becsülni, hogy egy rendelés kézbesítése meghaladja-e a 7 napos ígéretet. A megfigyelési egység egy `order_id`; a célváltozó a rendeléskor még ismeretlen `target_late`, a predikció pillanata az `order_ts`. A rendeléskor ismert feature-ök: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`. A cél egy dokumentált, előre rögzített összehasonlító kísérlet: három modellcsalád közül választunk, nem utólag a legszebb számot keresve.

Induló becslés a megoldás előtt: a nullmodell, amely minden rendelésre a tanítóbeli pozitív arányt adja, a lehető legegyszerűbb baseline. Ha a tanító `target_late` aránya például 22%, akkor az entrópiás nullmodell log loss-a körülbelül 0,53. Azt várjuk, hogy a távolság és a kosárérték legalább enyhe jelet ad, ezért a validációs log loss a baseline alá kerül; ugyanakkor a 2000-nél

kisebb tanítóhalmaz miatt a faensemble-ek varianciája már számít. Ennél pontosabb, nem mért jóslatot nem állíthatunk: a számokat a helyi futtatás hozza létre.

22.4.2 Induktív torzítás: mit feltételez a modellcsalád akkor is, ha az adat nem dönt?

Az **induktív torzítás** *(inductive bias)* az a feltevéskészlet, amelyet egy modellcsalád akkor is alkalmaz, amikor a minta nem határozza meg egyértelműen a szabályt. Nem hiba, hanem a tanulás előfeltétele: az összes lehetséges osztályozó közül valahogyan szűkíteni kell.

Kis ellenpélda: legyen két input, $x_1, x_2 \in \{0, 1\}$, és $y = x_1 \oplus x_2$ (XOR). Négy pont:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

A lineáris logisztikus modell legjobb becslése minden pontra $\hat{p} = 0{,}5$, mert nincs olyan egyenes, amely a XOR-pontokat szétválasztja. Ennek log loss-a $-\ln 0{,}5 = 0{,}693$. Egy döntési fa viszont két tengelypárhuzamos vágással tökéletesen leírja a szabályt, log loss-a 0 lesz. Ez kizárólag az induktív torzítás különbségét mutatja: a lineáris modell sima, korlátos határt feltételez; a fa darabonként konstans, tengelypárhuzamos szabályokat.

A három jelölt család torzítása:

- **Logisztikus regresszió:** a log-odds lineáris a feature-ökben; a határfelület hipersík. Kevés paraméter, stabil, jól extrapolál, de interakciót csak akkor lát, ha azt explicite beletesszük a feature-ökbe.
- **Random forest:** sok, gyengén korrelált fa átlaga; tengelypárhuzamos interakciókat hatékonyan tanul, de a tanítóadatok tartományán kívül nem extrapolál, hanem a közeli levélértékeket ismétli.
- **Gradiens boosting (jelen példában a scikit-learn `HistGradientBoostingClassifier`):** additívan, kis lépésenként új fákat illeszt a maradékhibára; erős nemlineáris közelítő, de nagyobb kapacitása miatt kisebb mintán hajlamosabb a túlillesztésre.

A 22.3. alfejezetben látott kNN ezzel szemben lokális egyenletességet feltételez: a közeli pontok címkéi hasonlók. Ez a torzítás nagy dimenzióban és rossz skálázás mellett drámaian gyengülhet; az ott tanult skálázási követelményt itt a lineáris modellnél is megőrizzük.

22.4.3 Kapacitás, variancia és hibamérleg

A választás mögött ugyanaz a kapacitás–variancia feszültség áll, amelyet a 10.4. alfejezet irányfelbontásnál és a 22.3. alfejezet méretkorlátjánál már érintettünk. Négyzetes veszteségre egy rögzített x feature-vektor mellett:

$$E_{\mathcal{D}}\left[\left(y - \hat{f}(x;\mathcal{D})\right)^2 \mid x\right] = \left(f(x) - E_{\mathcal{D}}[\hat{f}(x;\mathcal{D})]\right)^2 + E_{\mathcal{D}}\left[\left(\hat{f}(x;\mathcal{D}) - E_{\mathcal{D}}[\hat{f}(x;\mathcal{D})]\right)^2\right] + \sigma^2.$$

Jelölések: $y = f(x) + \varepsilon$ a zajos cél, ε várható értéke 0, varianciája σ^2; $\hat{f}(x; \mathcal{D})$ a $\mathcal{D}$ tanítómintára illesztett modell becslése az x pontban; a várható érték az $\mathcal{D}$ mintavételi eloszlására vonatkozik. Minden tag skalár egy rögzített x-re. A három tag rendre: torzításnégyzet, variancia, irreducibilis zaj. Bináris osztályozásban a log loss-ra nincs ugyanez a tiszta felbontás, de a logika irányadó: nagyobb kapacitás csökkentheti a torzítást, miközben növeli a varianciát; kis minta a varianciát bünteti.

A nullmodell torzítása nagy, varianciája kicsi. A 100–200 fás ensemble kis mintán már érezhető varianciát vihet a validációs log loss-ba, különösen akkor, ha a validhalmaz kicsi.

22.4.4 Előre rögzített szempontok

A jegyzőkönyv a következő hat dimenziót fagyasztja be, még mielőtt bárki megnézné a számokat.

- **Sample size:** a cutoff előtti, címkével teljesen elérhető tanítórendelések száma néhány száz és néhány ezer között várható. Kis mintán a lineáris modell varianciája kisebb; a faensemble-eknek erős regularizálás kell.
- **Latency:** az egyrekordos predikció ismételt, egyenként időzített hívásainak p95 válaszideje. Ez nem a nagy batch feldolgozási ideje osztva a rekordszámmal. A lineáris modell egyetlen mátrix-vektor szorzás és egy σ leképezés; a random forest minden fát bejár; a boosting ugyanígy iterációszám-szor dolgozik. A jegyzőkönyv p95-ös küszöböt rögzít: 10 ms.
- **Interpretability:** ha a döntéshozó magyarázatot kér, a logisztikus regresszió együtthatói log-odds hatásként közvetlenül olvashatók; a faensemble-eknél jellemzően fontossági vagy SHAP-elemzés kell. Ez a 18.4. alfejezet jelentésszerkezetének „design” szakaszába kerül.
- **Cost:** a scikit-learn `HistGradientBoostingClassifier` a főúton van; a külső XGBoost, LightGBM vagy CatBoost külön függőséget, buildelési és verziókövetési költséget jelent. Opcionális alternatíva lehet, de nem kötelező a döntéshez.
- **Uncertainty:** a felhasznált valószínűség minősége is döntési bemenet, nemcsak a sorrend. A logisztikus regresszió `predict_proba`-ja általában jobban kalibrált; a faensemble-ek gyakran szélsőséges 0/1 közeli értékeket adnak. Ha a késés valószínűségét közvetlenül üzleti küszöbhöz kötjük, kalibrációs görbe vagy `CalibratedClassifierCV` szükséges.
- Az **induktív torzítást** külön, fentebb tárgyaltuk, de a jegyzőkönyvben a módszer indoklásakor ismét szerepelnie kell.

22.4.5 A jegyzőkönyv és a döntési kapu

```yaml
decision_protocol:
  target: target_late
  observation_unit: order_id
  prediction_time_ref: order_ts
  split:
    test_cutoff: "2025-05-01T00:00:00+00:00"
    valid_fraction_of_train: 0.2
    purge_rule: train_cimket_cutoff_elott_ismerni_kell
  baseline: prior_model
  primary_metric: valid_log_loss
  guardrail_metric: valid_roc_auc
  latency_gate_p95_ms: 10
  acceptance_rules:
    - valid_log_loss <= 0.95 * baseline_log_loss
    - valid_roc_auc >= 0.5
    - p95_predict_ms <= 10
  tie_break:
    - prefer lower valid_log_loss
```

```
    - if diff < 0.005: prefer lower p95_predict_ms
    - if still close: prefer stronger interpretability
  candidates: [logistic, rf, hgb]
  forbidden_test_use: true
```

Két munkahelyi mozzanat fontos. Először: a validhalmaz a cutoff előtt címkével rendelkező rekordok késői időszakából jön; a tanítás címkéit ezen belül is a validáció kezdetén ismert címkékre korlátozzuk. Ez **retrospektív modellszelekció**, nem teljes előrecsúszó backtest. A validációs méréskor a validcímkéket utólag használjuk, a tesztet nem nézzük meg a választás előtt. Másodszor: a három jelöltet azonos validhalmazon, azonos előfeldolgozási szerződéssel, azonos seeddel futtatjuk. Azt mérjük, hogy a modellcsalád hogyan teljesít az adott feladaton, nem katalógusígéreteket. A p95-kapuhoz azonos gépen, bemelegítés után külön-külön mérjük az egyrekordos `predict_proba` hívások teljes idejét, beleértve a pipeline előfeldolgozását. A batch-átbocsátást külön metrikaként érdemes rögzíteni.

22.4.6 Teljes minimális munkapélda

A következő példafájl helye `lab/model_protocol.py`; a repó gyökeréből `python lab/model_protocol.py` paranccsal indítható (telepített függőségek mellett). Az adatútvonalak a szkript helyéhez képest feloldott abszolút útvonalak, ezért nem függnek az indításkori munkakönyvtártól. A kimenet környezetfüggő; az automatikus, teljes adatkészletes ellenőrzés a kapcsolódó példaprojekt feladata.

```
# lab/model_protocol.py - modellcsalád-választási jegyzőkönyv
import time
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

ROOT = Path(__file__).resolve().parent
CUTOFF = pd.Timestamp("2025-05-01T00:00:00+00:00")

orders = pd.read_csv(
    ROOT / "data" / "orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
customers = pd.read_csv(
    ROOT / "data" / "customers.csv",
    parse_dates=["signup_date"],
)

df = orders.merge(
    customers[
        ["customer_id", "signup_date", "segment", "region", "acquisition_channel"]
    ],
    on="customer_id",
)
df["account_age_days"] = (
    df["order_ts"].dt.tz_localize(None).dt.normalize() - df["signup_date"]
).dt.days

FEATURES = [
```

```
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
]
TARGET = "target_late"
FORBIDDEN = {
    "delivery_days", "actual_delivery_at", "label_available_at",
    "order_id", "customer_id",
}
assert set(FEATURES).isdisjoint(FORBIDDEN), "tiltott oszlop került a feature-ök közé"

# Retrospektív validáció a CUTOFF előtt címkével már rendelkező rekordokon.
# A validáció kezdő időpontjánál a train-címkéknek is rendelkezésre kell állniuk.
train_all = df[(df["order_ts"] < CUTOFF) & (df["label_available_at"] < CUTOFF)].copy()
test = df[df["order_ts"] >= CUTOFF].copy()
purged = df[(df["order_ts"] < CUTOFF) & (df["label_available_at"] >= CUTOFF)]
print(f"train_all={len(train_all)} purged={len(purged)} test={len(test)}")
if train_all.empty:
    raise ValueError("Nincs a teszt-cutoff előtt ismert címkéjű rekord")
valid_cutoff = train_all["order_ts"].quantile(0.8)
train = train_all[
    (train_all["order_ts"] < valid_cutoff)
    & (train_all["label_available_at"] < valid_cutoff)
].copy()
valid = train_all[train_all["order_ts"] >= valid_cutoff].copy()
print(f"train={len(train)} valid={len(valid)}")
if train.empty or valid.empty:
    raise ValueError("Üres train/valid halmaz; ellenőrizd az idő- és címkecutoffot")
if train[TARGET].nunique() < 2 or valid[TARGET].nunique() < 2:
    raise ValueError("AUC-hoz és modellillesztéshez mindkét osztálynak jelen kell lennie")

base_p = float(np.clip(train[TARGET].mean(), 1e-12, 1 - 1e-12))
baseline_p = np.full(len(valid), base_p, dtype=float)
baseline_log_loss = log_loss(valid[TARGET], baseline_p, labels=[0, 1])
gate = 0.95 * baseline_log_loss
print(f"baseline_log_loss={baseline_log_loss:.4f} gate={gate:.4f}")

CAT_COLS = ["segment", "region", "acquisition_channel"]
NUM_COLS = ["distance_km", "priority_flag", "basket_minor", "quantity_sum", "account_age_days"]


def build(name: str) -> Pipeline:
    if name == "logistic":
        prep = ColumnTransformer([
            ("num", StandardScaler(), NUM_COLS),
            ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_COLS),
        ])
        model = LogisticRegression(max_iter=1000, C=1.0)
    elif name == "rf":
        prep = ColumnTransformer([
            ("num", "passthrough", NUM_COLS),
            ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_COLS),
        ])
        model = RandomForestClassifier(
            n_estimators=200, max_depth=8, min_samples_leaf=5,
            random_state=20261020, n_jobs=-1,
        )
    elif name == "hgb":
        prep = ColumnTransformer([
            ("num", "passthrough", NUM_COLS),
            ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CAT_COLS),
        ])
        model = HistGradientBoostingClassifier(
            max_iter=100, learning_rate=0.05, max_depth=3,
```

```
            l2_regularization=0.1, random_state=20261020,
        )
    else:
        raise ValueError(name)
    return Pipeline([("prep", prep), ("model", model)])

rows = []
for name in ["logistic", "rf", "hgb"]:
    pipe = build(name)

    t0 = time.perf_counter()
    pipe.fit(train[FEATURES], train[TARGET])
    fit_s = time.perf_counter() - t0

    # Egyrekordos end-to-end latency (nem batch-idő / elemszám).
    # Bemelegítés után külön mérünk legalább 100 egyrekordos hívást.
    if valid.empty:
        raise ValueError("Üres validációs halmaz: latency és metrika nem számolható")
    probe = valid[FEATURES].iloc[[0]]
    for _ in range(10):
        pipe.predict_proba(probe)
    lat = []
    for i in range(100):
        one_row = valid[FEATURES].iloc[[i % len(valid)]]
        t0 = time.perf_counter()
        pipe.predict_proba(one_row)
        lat.append((time.perf_counter() - t0) * 1000)
    p95_ms = float(np.percentile(lat, 95))
    # A batch throughput külön mérendő; nem helyettesíti a p95 latency-t.

    p = pipe.predict_proba(valid[FEATURES])[:, 1]
    ll = log_loss(valid[TARGET], p)
    auc = roc_auc_score(valid[TARGET], p)
    rows.append((name, fit_s, p95_ms, ll, auc))
    print(
        f"{name:9s} fit_s={fit_s:6.2f} p95_ms={p95_ms:7.3f} "
        f"valid_log_loss={ll:.4f} valid_auc={auc:.4f}"
    )

accepted = [
    row for row in rows
    if row[3] <= gate and row[4] >= 0.5 and row[2] <= 10.0
]
if not accepted:
    decision = "baseline"
else:
    # először valid_log_loss, majd p95 latency szerint rendezünk
    accepted.sort(key=lambda row: (row[3], row[2]))
    decision = accepted[0][0]
print(f"decision={decision}")
```

Kézi kontroll: ha a validációs halmaz pozitív aránya éppen megegyezik a tanítóhalmaz $p = 0{,}22$ arányával, a konstans baseline log loss-a $-(0{,}22 \ln 0{,}22 + 0{,}78 \ln 0{,}78) \approx 0{,}526$. Eltérő validációs prevalencia mellett a tényleges log loss más: ezért a kaput a rögzített tanítóarányból előállított valószínűségekkel, **a validációs címkéken** számítjuk. A XOR-példa másik független kontrollja: a mindig $\hat{p} = 0{,}5$ modellt adó osztályozó log loss-a $-\ln 0{,}5 \approx 0{,}693$; ezt a log loss definíciójából kézzel megkapjuk.

Az ellenőrzött futás (Python 3.12.3, pandas 3.1.0, numpy 2.5.4, scikit-learn 1.9.1, seed `20261020`, a `lab/model_protocol.py` a lab gyökeréből):

```
train_all=1531 purged=69 test=800
```

```
train=1156 valid=307
baseline_log_loss=0.4702 gate=0.4467
logistic  fit_s=  0.01 p95_ms=  3.781 valid_log_loss=0.2878 valid_auc=0.8965
rf        fit_s=  0.46 p95_ms= 88.419 valid_log_loss=0.3234 valid_auc=0.8665
hgb       fit_s=  0.07 p95_ms=  4.579 valid_log_loss=0.3118 valid_auc=0.8758
decision=logistic
```

22.4.6.1 Döntési tábla és előre rögzített kiválasztási szabály

A futás előtt rögzített szabály:

1. Minden jelölt ugyanazon a train/valid spliten, ugyanazzal a preprocessinggel és ugyanazzal a scoringgal fut.
2. **Kemény kapuk:** valid log loss $\leq 0{,}95\cdot$ baseline, valid AUC $\geq 0{,}5$, p95 egyrekordos latency ≤ 10 ms (a mobil bevetésnél 15 ms), modellméret ≤ 2 MB.
3. A kapukon átjutó jelölteket valid log loss szerint rendezzük; egyenlőség esetén a kisebb p95, majd a kisebb modellméret, végül az egyszerűbb család dönt.
4. A tesztadatot csak a kiválasztott modell **egyszeri** jelentéséhez nyitjuk meg; a teszt nem szerepel a modellcsalád vagy a hiperparaméter választásában.

A fenti futás döntési táblája:

Jelölt	Család	fit (s)	p95 (ms)	Valid log loss	Valid AUC	Kapu	Rangsor
logistic	lineáris	0,01	3,78	0,2878	0,8965	átmegy	1.
hgb	boosting	0,07	4,58	0,3118	0,8758	átmegy	2.
rf	bagging	0,46	88,42	0,3234	0,8665	latency kapu bukik	–

A baseline log loss 0,4702, a kapu 0,4467; mindhárom modell jobb a baseline-nál, de a random forest a 10 ms-os latency kaput nem teljesíti (88,4 ms), ezért kiesik. A logisztikus regresszió a legalacsonyabb valid log loss-t adja, ezért a döntés `logistic`. A teszt AUC-t ebben a lépésben nem számoltuk; az a kiválasztás utáni egyszeri jelentés része. A p95 latency futásonként néhány ms-mal ingadozik (a random forestnél 88–90 ms között mértünk); a döntés a kapuhoz viszonyított nagyságrenden alapul, nem a tizedesjegyen.

22.4.7 Eredmény értelmezése

A log loss definíciója a validhalmazra:

$$\ell_{\log} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\right],$$

ahol $y_i \in \{0,1\}$ az i-edik rendelés megfigyelt késésjelzője, $\hat{p}_i \in (0,1)$ a modell becsült pozitív valószínűsége; a bemeneti tömbök alakja $(n,)$, az eredmény skalár. A $\hat{p}_i = 1$ numerikusan instabil lenne; a gyakorlati implementációk ezért kis epszilonnal vágják a valószínűséget. A validációs log loss nem vihető át automatikusan más időszakra: azt az adott validablakra, a modellépítés pillanatában mértük.

Három azonnali diagnosztikai szabály:

- Ha a valid log loss a baseline fölé megy, a modell nem tanult hasznos jelet; a döntés marad baseline.
- Ha a valid AUC 0,5 alatt van, az adott mintán kedvezőtlen rangsorolást látunk. Ez önmagában nem bizonyít hibás adatszerződést: előbb ellenőrizzük a pozitív osztály irányát, a mintanagyságot, az AUC bizonytalanságát (például megfelelő bootstrap-intervallummal), valamint az időbeli eltolódást; a címkecsere és a szivárgás csak lehetséges hibahipotézis.
- Ha a valid AUC 0,99 közelében van, általában szivárgás, nem kiváló modell; a 21.4. alfejezet szerződés tiltott oszlopait külön is ellenőrizzük.

A kód által kiírt `decision` tehát az előre rögzített kapuból következik; a tesztet legfeljebb a kiválasztott modell egyszeri jelentéséhez nyitjuk meg, és akkor is beírjuk a jegyzőkönyvbe a döntés időpontjában ismeretlen státuszt.

A három modellcsalád összehasonlításához előzetesen különítsük el a hiperparaméter-választásra használt validációt a lezárt teszttől; egyetlen eredménykülönbségből ne következtessünk általános modellfölényre.

22.4.8 Feladatok

1. **Értelmezés:** A csapat egyik tagja accuracy szerint választana modellt. Magyarázd el a log loss előnyét ebben a döntésben, és fogalmazza meg saját szavakkal, miért lehet a logisztikus regresszió kedvezőbb kis tanítóhalmaz mellett.
2. **Implementálás:** Futtasd a `lab/model_protocol.py` fájlt a repó gyökeréből (`python lab/model_protocol.py`), a `lab/data/orders.csv` és `lab/data/customers.csv` bemenetekkel. Rögzítsd a kiírt train/valid/test méreteket, a három modell valid log loss-át, egyrekordos p95 predikciós idejét és a végső `decision` értékét. Írd le a futtató környezet verzióját és a mérési parancsot.
3. **Hibadiagnózis:** Egy kolléga a következő eredményt küldi: valid log loss = 0,02, valid AUC = 0,99, teszt log loss = 0,68. Milyen hiba a legvalószínűbb, és milyen két ellenőrző kódot futtatna ennek megerősítésére?
4. **Megváltozott feltétel:** A modellt egy mobilalkalmazásban, rendelésenként legfeljebb 15 ms p95 predikciós idővel szeretnék futtatni. A modellfájl méretére 2 MB felső korlátot adnak. Melyik jelölt hogyan viselkedik? Írj három mérési lépést, amellyel a modellméret, a p95 latency és a valid log loss kompromisszumát kísérletileg összehasonlítaná.
5. **Átvitel:** Tegyük fel, hogy a késés valószínűségét nem sorrendbe állításra, hanem közvetlenül egy 20%-os riasztási küszöbhöz használják. Hogyan változtatná meg a jegyzőkönyv `uncertainty` és `primary_metric` sorát?

22.4.9 Megoldások és indoklás

1. Az accuracy egy küszöbnél binarizál, és elrejti, mennyire bizonytalan a modell; a $\hat{p} = 0{,}51$ és a $\hat{p} = 0{,}99$ ugyanabba az osztályba eshet. A log loss a valószínűség pontosságát is bünteti, ezért a késési riasztásnál a bizonytalan, de rendezésben jó modell is feltűnik. Kis mintán a logisztikus regresszió kevesebb paramétert illeszt, a varianciája kisebb; a faensemble-ek ugyanazon a mintán nagyobb szórással váltogathatják a valid log loss-t.
2. A futtatás outputja a fenti formátumot követi. Ellenőrző invariantok: az elfogadott modellekre `valid_auc >= 0.5` (az elutasított jelöltekre ez nem garantált); a `baseline_log_loss` pozitív és az adott trainarányból kézzel visszaszámolható; a `train + valid` legfeljebb `train_all` mérete (a valid kezdete előtt még meg nem érkezett címkék kieshetnek). A

konkrét számjegyek a géptől és a könyvtárverziótól függenek; elfogadható numerikus tolerancia a log loss-ban 0,005, a p95 ms környezet- és terhelésfüggő, ezért ismételt mérési sorozatok szükségesek a stabil döntéshez.

3. A majdnem tökéletes validáció és a gyenge teszt szivárgásra, validációs túlillesztésre vagy eloszlásváltozásra is utalhat. Vizsgálandó lehetőség: `delivery_days` vagy `actual_delivery_at` bekerült a feature-ök közé, esetleg a validhalmazt a tesztidőszakból vágták, és a modell a jövőben ismertté váló címkét látta. Két ellenőrzés: (a) `assert set(FEATURES).isdisjoint(FORBIDDEN)`, ahogy a kód elején; (b) `pd.merge(train, test, on="order_id", how="inner")` üres halmaz kell legyen.
4. A lineáris modell egyetlen koeficiensmátrix-szorzás, p95 latency-ja tipikusan a legkisebb, és a modellfájl is kicsi. A random forest 200, legfeljebb 8 mélységű fájának bejárása nagyobb predikciós költség; a HGB 100 iterációja hasonló nagyságrendű. Mérési terv: (a) `pickle.dumps(pipe).__len__()` minden jelöltre; (b) a fenti bemelegített, 100 egyrekordos hívásból számított p95 mérés; (c) a valid log loss változását a 15 ms-os korlátot sértő modellek kizárása után. A döntés innen a jegyzőkönyv `tie_break`-jét követi.
5. A log loss továbbra is megfelelő szigorúan proper scoring rule; a Brier-score alternatív proper mérőszám. Küszöbös döntésnél emellett a 20%-os küszöbhöz tartozó költségérzékeny hasznosságot, a kalibrációt és a bizonytalanságot külön is értékelni kell. Az `uncertainty` szakaszba kalibrációs görbe vagy `CalibratedClassifierCV` kerül. A kódban ehhez `brier_score_loss(valid[TARGET], p)` és a kalibrációs görbe meredeksége illeszkedne.

22.4.10 Gyakori tévedés ellenpéldával

„A random forest mindig jobb, mert a legnépszerűbb eszköz." Ez a tévedés az eszközhírnevet összekeveri az adott adatra mért teljesítménnyel. Ellenpélda: képzeljünk el 120 rendelést, ahol a késés főként a távolság monoton függvénye, de erős zajjal. A default 200 fás, mélységi regularizálás nélküli erdő a kis mintán túlilleszt; a valid log loss 0,68, míg a lineáris modellé 0,58. A számok szemléltetőek, nem e könyv által futtatott mérés – de pontosan azt mutatják, hogy a népszerűség nem ad felmentést a 21.4. alfejezet szerződésben rögzített teszt alól. A 18.4. alfejezet jelentésszemlélete ugyanezt kéri: a módszer neve nem bizonyíték, a mért hatás és a design korlátja az.

22.4.11 Önellenőrzés

- Meg tudja mondani, mit jelent az induktív torzítás a XOR-példán?
- Miért van a validhalmaz a tanítóidőszak utolsó 20%-ában, nem a tesztidőszakban?
- Mi a log loss definíciója, és mi a prior baseline képlete?
- Melyik három feltételnek kell teljesülnie ahhoz, hogy a jelölt átmenjen a fenti döntési kapun?
- Miért nem bizonyítja a tesztlog loss, hogy a modell a következő hónapban is ugyanúgy teljesít?
- Milyen ellenőrző kódsor dönti el, hogy a tiltott oszlopok nem kerültek a feature-ök közé?

22.4.12 Rövid összefoglaló

A modellcsalád választása előre rögzített tudományos jegyzőkönyv, nem utólagos esztétikai döntés. A jegyzőkönyv rögzíti a célt, a validhalmazt, a baseline-t, a log loss elsődleges metrikát, az AUC védőkorlátot, a p95 latency kaput és a döntési szabályt. A három jelölt – logisztikus

regresszió, random forest és hisztogramos gradiens boosting – eltérő induktív torzítást, kapacitást és értelmezhetőséget képvisel; a választást a mért valid log loss és a latency hozza, a tesztet kizárólag a kiválasztott modell egyszeri jelentéséhez használjuk. A következő egység a 23.1. alfejezet hibamértékből döntési céllá alakításával pontosítja ezt a jegyzőkönyvet.

További olvasmány és elsődleges dokumentáció: [SklearnHGB] scikit-learn: Gradient Boosting and Categorical Features, https://scikit-learn.org/stable/modules/ensemble.html (hozzáférés: 2026-10-21).

23 Értékelés, keresztvalidáció és hiperparaméter-keresés

23.1 Hibamértékből döntési cél

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 22.4. alfejezet · Célkészség: Hibamértékből döntési cél · Kimenet: működő kód és a labor ellenőrző tesztje.

23.1.1 Cél és konkrét helyzet

A Northstar prediktív projektben rendeléskor szeretnénk előre jelezni, hogy a kézbesítés 7 napnál később érkezik-e. A célváltozó `target_late`: 1, ha `delivery_days > 7`, egyébként 0. A modell minden rendeléshez egy p_i becsült késési valószínűséget ad, amelyből egy τ küszöbbel döntést képzünk: ha $p_i \geq \tau$, a rendelést kockázatosnak jelöljük, és az ügyfélszolgálat előzetesen ellenőrzi. A döntésnek kétféle hibája van: hamis pozitív (felesleges ellenőrzés) és hamis negatív (észrevétlen késés, elégedetlen ügyfél). Az értékelés bemenete tehát páronként az igaz `target_late` és a modell pontszáma, megfigyelési egysége egy rendelés. Az elvárt ellenőrzés: a választott metrika kézzel is reprodukálható legyen egy kis mintán, és a döntési célhoz illeszkedjen.

Induló becslés. Tegyük fel, hogy a tesztidőszakban a rendelések 7%-a késik. Egy „mindig időben” modell accuracy-je 93%, mégis semmilyen kockázatos rendelést nem talál meg. Ez már jelzi: a csupasz accuracy önmagában nem alkalmas a döntési cél értékelésére.

23.1.2 Fogalmi magyarázat

23.1.2.1 Pontosság, konfúziós mátrix és az alapráta

Legyen n kiértékelt rendelés, $y_i \in \{0, 1\}$ a tényleges késés, $\hat{y}_i \in \{0, 1\}$ a küszöbölés utáni döntés. A konfúziós mátrix az igazság és a döntés kereszttáblája:

	Döntés: késik (1)	Döntés: időben (0)
Ténylegesen késik (1)	TP	FN
Ténylegesen időben (0)	FP	TN

- **accuracy**: $(\mathrm{TP} + \mathrm{TN})/n$,
- **precision** (pozitív prediktív érték): $\mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$, a jelölt rendelések közül hány tényleg késik,
- **recall** (szenzitivitás): $\mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$, a tényleges késések közül hányat jelölünk meg,
- **alapráta** (base rate): n_+/n, ahol n_+ a ténylegesen késő rendelések száma.

Kézi kontroll: ha $n = 1000$, ebből 70 késik, és mindenkit időben érkezőnek jelölünk, akkor TP $= 0$, FP $= 0$, TN $= 930$, FN $= 70$. Így accuracy $= 930/1000 = 0.93$, recall $= 0/70 = 0$, precision pedig nem értelmezhető, mert nem jelöltünk pozitív esetet. A magas accuracy tehát csak a ritka pozitív osztály tükröződése, nem a döntés hasznossága.

23.1.2.2 Folytonos cél: MAE és RMSE

Ha a cél napokban mért `delivery_days` becslése lenne, a két leggyakoribb hiba:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}.$$

Itt y_i a tényleges kézbesítési napok száma, $\hat{y}_i$ a becslés, az eredmény mértékegysége nap. A MAE lineárisan büntet, az RMSE négyzetesen, ezért utóbbi érzékenyebb a nagy tévedésekre. Ha a döntési költség arányos az eltéréssel, a MAE; ha a kiugró késések különösen költségesek, az RMSE a jobb cél.

23.1.2.3 Log loss: a valószínűség minősége

A küszöbölés előtti p_i valószínűséget a **log loss** (más néven bináris keresztentrópia) értékeli:

$$L_{\log} = -\frac{1}{n}\sum_{i=1}^{n} \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right].$$

Itt $p_i \in (0, 1)$ a modell becsült $P(y_i = 1)$ értéke, $y_i \in \{0, 1\}$. A függvény kisebb értéke jobb; a tökéletes modellnél $L_{\log} = 0$, a mindig $p_i = 0.5$ modellnél $\ln 2 \approx 0.693$. A log loss erősen bünteti a magabiztos, de hibás predikciót: ha $y_i = 1$, de a modell $p_i = 0.01$ értéket ad, a tag $-\ln 0.01 \approx 4.605$, míg $p_i = 0.5$ esetén csak 0.693. Ez közvetlen kapcsolatban áll a 19.1. alfejezet likelihood-fogalmával: a negatív log-likelihood átlagát számoljuk, ezért a kalibrált valószínűségű modell előnyben részesül.

23.1.2.4 ROC-AUC: a pontszámok sorrendje

A **ROC-görbe** (Receiver Operating Characteristic) a modell pontszámaihoz tartozó összes lehetséges τ küszöbnél ábrázolja a

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}$$

párokat. A **ROC-AUC** a görbe alatti terület, értéke 0 és 1 között van, a véletlen pontszámú modellé 0.5. Egyenértékű értelmezés: annak valószínűsége, hogy egy véletlenül választott késő rendeléshez nagyobb pontszámot ad a modell, mint egy véletlenül választott időben érkezőhöz; döntetlennél fél pont számít.

A ROC-AUC a küszöbtől független sorrendi jellemző. Nem mondja meg, hogy az operatív küszöbnél mennyi a precision és recall, és nem tartalmazza a kétféle hiba költségét.

23.1.2.5 PR-AUC: ritka eseményekhez

A **PR-görbe** a recall függvényében a precision-t ábrázolja, a **PR-AUC** a precision–recall görbe alatti területet jelenti. A scikit-learn `average_precision_score` függvénye azonban **average precisiont (AP)** számol: a recall növekményeivel súlyozott precisionértékek összegét, nem pedig a PR-görbe trapézszabályos integrálját. A két érték eltérhet; a további kódpéldában AP szerepel. A precision a Bayes-féle feltételes arány miatt függ az alaprátától:

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)},$$

ahol π a pozitív osztály aránya. Ha π kicsi, ugyanazon TPR és FPR mellett a precision alacsonyabb. A ROC-AUC a TPR–FPR párokon alapul, ezért előzetes eltolódásra (prior shift) kevésbé érzékeny; a PR-AUC viszont erősen függ tőle. Ritka kimenetelnél a PR-AUC informatívabb, mert nem engedi, hogy a sok negatív eset elfedje a pozitív esetek megtalálásának minőségét.

23.1.3 Teljes minimális munkapélda

A példa determinisztikus, nincs véletlen seed. Nyolc rendeléshez ismerjük az igaz késési címkét és a modell becsült p_i értékét:

```
import numpy as np
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score, recall_score,
    log_loss, roc_auc_score, average_precision_score
)

# y_true: 1 = késik, 0 = időben érkezik; p_score: P(késés) becslés
y_true = np.array([1, 0, 1, 1, 0, 0, 1, 0])
p_score = np.array([0.8, 0.4, 0.6, 0.3, 0.2, 0.7, 0.9, 0.1])
threshold = 0.5
y_pred = (p_score >= threshold).astype(int)

print("y_pred:  ", y_pred)
print("confusion (sor: pozitív, negatív):")
print(confusion_matrix(y_true, y_pred, labels=[1, 0]))
print("accuracy:  ", accuracy_score(y_true, y_pred))
print("precision: ", precision_score(y_true, y_pred))
print("recall:    ", recall_score(y_true, y_pred))
print("log_loss:  ", round(log_loss(y_true, p_score), 6))
print("ROC-AUC:   ", roc_auc_score(y_true, p_score))
print("Average precision (AP): ", round(average_precision_score(y_true, p_score), 6))
```

A fájlt `metrics_demo.py` néven a projektgyökérbe mentve a futási parancs Linux/macOS terminálban, illetve PowerShellben:

```
python metrics_demo.py
```

A scikit-learn használt függvényei a jelenlegi stabil 1.4+ sorozatban ugyanazt a felületet használják; az `average_precision_score` a `pos_label=1` alapértelmezést követi.

A kód futása előtt kézzel is ellenőrizhető a 0.5-ös küszöb. Az egyes elemekre:

index	y_i	p_i	$\hat{y}_i$
1	1	0.8	1
2	0	0.4	0

index	y_i	p_i	$\hat{y}_i$
3	1	0.6	1
4	1	0.3	0
5	0	0.2	0
6	0	0.7	1
7	1	0.9	1
8	0	0.1	0

Ebből TP = 3, FN = 1, FP = 1, TN = 3, tehát

$$\text{accuracy} = (3+3)/8 = 0.75, \quad \text{precision} = 3/4 = 0.75, \quad \text{recall} = 3/4 = 0.75.$$

A ROC-AUC kézi ellenőrzése: a négy pozitív pontszám $\{0.8, 0.6, 0.3, 0.9\}$, a négy negatív $\{0.4, 0.2, 0.7, 0.1\}$. A pozitív–negatív pontszámpárok közül 13-ban nagyobb a pozitív pontszám, döntetlen nincs, ezért

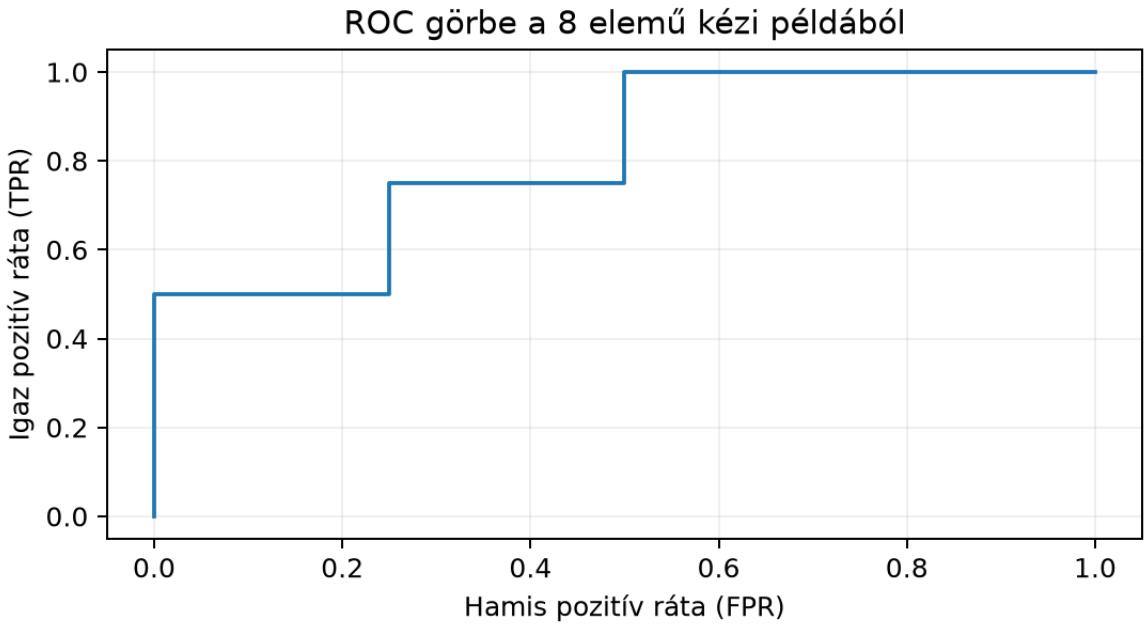
$$\text{ROC-AUC} = 13/16 = 0.8125.$$

A log loss kézi ellenőrzése: a pozitív esetekhez $\ln(0.8) + \ln(0.6) + \ln(0.3) + \ln(0.9)$, a negatívokhoz $\ln(0.6) + \ln(0.8) + \ln(0.3) + \ln(0.9)$ tartozik, az átlagos negatív log-likelihood 0.510826. A példa alapján a várható kimenet tehát:

```
confusion (sor: pozitív, negatív):
[[3 1]
 [1 3]]
accuracy:   0.75
precision:  0.75
recall:     0.75
log_loss:   0.510826
ROC-AUC:    0.8125
PR-AUC/AP:  0.8542
```

A fenti számok kézi levezetésűek; ebben a dokumentumban nem futtattuk a kódot, de helyi futtatásnál a numerikus értékek 1e-6 tűrésen belül egyeznek a levezetéssel.

A ROC-görbe lépcsős alakja is kézzel követhető a küszöb csökkentésével:



23.1. ábra: ROC görbe a 8 elemű kézi példából - Kézzel levezetett lépcsős görbe, nem mérés.

23.1.4 Eredmény értelmezése

A konfúziós mátrix szerint a modell 4 késést ténylegesen tartalmazó rendelésből 3-at jelölt, és 1 időben érkezőt tévesen kockázatosnak minősített. A 0.75-ös accuracy itt kiegyensúlyozott alaprátánál ($n_+/n = 0.5$) már informatívabb, de a kiinduló 7%-os késési aránynál ugyanez a döntési szabály másként viselkedne. Ha például 1000 rendelésből 70 késik, és a fenti arányokat változatlannak képzeljük, a „mindig tagadó” modell 0.93 accuracy-t adna 0 recall mellett, ezért a döntési célt recall/precision párossal kell megítélni. A log loss 0.51 kisebb a nem informatív 0.693-nál, jelezve, hogy a valószínűségek hordoznak információt; a ROC-AUC 0.81 és a PR-AUC 0.85 ugyanezt erősíti meg ezen a kis mintán. A PR-AUC itt magasabb, mint a ROC-AUC, ami kiegyensúlyozott mintán nem ritka; a két metrika eltérő skálán értékel.

A ROC AUC rangsorolási mutató, önmagában nem fejezi ki az üzleti költséget vagy a kalibrációt. Küszöböt a hamis pozitív és hamis negatív döntések várható költségének és a kapacitáskorlátnak megfelelően válasszunk.

23.1.4.1 Költségmátrix és a döntési küszöb

A döntési feladatot egy 2×2-es költségtábla rögzíti. Legyen a riasztás költsége C_{FP} (felesleges ellenőrzés), az elszalasztott késésé C_{FN} (elégedetlen ügyfél, kompenzáció), a helyes döntéseké pedig 0:

	Valódi késés ($y = 1$)	Valódi időben ($y = 0$)
Riasztás ($p \geq \tau$)	0 (TP)	C_{FP} (FP)
Nincs riasztás	C_{FN} (FN)	0 (TN)

Egy küszöb összköltsége $C(\tau) = C_{FP}\,FP(\tau) + C_{FN}\,FN(\tau)$. Ha a modell **kalibrált**, akkor a döntés várható költsége a p valószínűségből közvetlenül számolható: riasztás akkor érdemes, ha $p\,C_{FN} > (1-p)\,C_{FP}$, azaz

$$p > \tau^{*} = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Például $C_{FP} = 5$ és $C_{FN} = 50$ mellett $\tau^{*} = 5/55 \approx 0{,}0909$. A gyakoriság ebben a képletben nem külön paraméter: a kalibrált p már tartalmazza. Ha a modell nem kalibrált, a küszöböt a kalibráció után kell megválasztani, különben a τ^{*} nem a kívánt költséget minimalizálja.

Szemléltető számítás $n = 20\,000$, kalibrált $p \sim \mathrm{Beta}(2, 18)$, $y \sim \mathrm{Bernoulli}(p)$ mellett (prevalencia 0,1006):

Küszöb τ	Riasztások	FP	FN	Összköltség
0,05	15 086	13 225	151	73 675
0,0909	**9 460**	**7 984**	**536**	**66 720**
0,15	3 923	3 103	1 192	75 115
0,30	199	139	1 952	98 295
0,50	2	1	2 011	100 555

A költségminimum a $\tau^{*} = 0{,}0909$ közelében van; a 0,5-ös küszöb accuracy szempontból jónak tűnhet, de a költsége több mint 50%-kal nagyobb. A küszöb kiválasztásához tehát a költségmátrix és a kalibrált valószínűség kell, nem az AUC vagy az accuracy önmagában.

23.1.5 Feladatok

23.1.5.1 1. feladat — értelmezés, Northstar alapráta

A tesztidőszakban 1200 rendelésből 84 késik. A nullmodell mindenkit időben érkezőnek jelöl. Számítsd ki az accuracy-t, a recall-t, és indokold, miért nem használható ez a nullmodell a kockázatos rendelések előszűrésére. Mi a precision a nullmodellnél?

23.1.5.2 2. feladat — kézi megvalósítás

Adott $y = [0, 1, 1, 0, 0, 1]$ és $p = [0.55, 0.45, 0.80, 0.35, 0.60, 0.20]$, küszöb $\tau = 0.5$. Számítsd ki kézzel a konfúziós mátrixot, az accuracy-t, precision-t és recall-t. Ellenőrizd, hogy a modell jobb-e a véletlen döntésnél.

23.1.5.3 3. feladat — hibakeresés

Egy kolléga azt állítja: „A modell ROC-AUC-ja 0.96, tehát nyugodtan alkalmazhatunk 0.5-ös küszöböt, és a beavatkozás jól fog működni.” A 0.5-ös küszöbnél a konfúziós mátrix: TP=40, FN=60, TN=850, FP=50. Számítsd ki a precision-t, recall-t és accuracy-t. Mi a hiba a kolléga érvelésében? Milyen további döntési adatra lenne szükség a küszöb megválasztásához?

23.1.5.4 4. feladat — megváltozott feltétel

A Northstar-modellt egy új régióra alkalmazzuk, ahol a késési alapráta 5% a korábbi 15% helyett. Tegyük fel, hogy a modell pontszámainak osztályon belüli eloszlása nem változik, csak az alapráta csökken. Mi történik a ROC-AUC-val, és mi történik a PR-AUC-val? Magyarázd a precision Bayes-féle képletével, és adj kvalitatív választ arra, melyik metrikában jelenik meg erősebben az eltolódás.

23.1.6 Megoldások és indoklás

23.1.6.1 1. feladat megoldása

Az alapráta $84/1200 = 0.07$. A nullmodell esetén TP=0, FP=0, TN=1116, FN=84. Ezért

$$\text{accuracy} = \frac{1116}{1200} = 0.93, \qquad \text{recall} = \frac{0}{84} = 0.$$

A precision nem értelmezhető, mert a nullmodell egyetlen pozitív jelölést sem ad; ha mégis 0-ként akarnánk definiálni, az félrevezető, mert nem „rossz pozitív döntések arányáról”, hanem arról van szó, hogy a döntési szabály nem jelöl semmit. A cél a késések megtalálása, ezért a 0 recall azt jelenti, hogy a figyelmeztetési folyamat egyetlen késést sem előz meg. Az accuracy 0.93 önmagában tehát használhatatlan a döntési célra.

23.1.6.2 2. feladat megoldása

A küszöb szerint $\hat{y} = [1, 0, 1, 0, 1, 0]$, mert a 0.55, 0.80 és 0.60 értékek nagyobb-egyenlők 0.5-nél. A címkék: $y = [0, 1, 1, 0, 0, 1]$. A párok: (0,1)→FP, (1,0)→FN, (1,1)→TP, (0,0)→TN, (0,1)→FP, (1,0)→FN. Tehát TP=1, FN=2, FP=2, TN=1. Ebből

$$\text{accuracy} = \frac{1+1}{6} = 0.333, \quad \text{precision} = \frac{1}{1+2} = 0.333, \quad \text{recall} = \frac{1}{1+2} = 0.333.$$

A véletlen döntés kiegyensúlyozott esetben körülbelül 0.5 accuracy-t adna, tehát ez a modell a 0.5-ös küszöbnél rosszabb a véletlennél. A pontszámok sorrendje alapján a ROC-AUC ettől még lehet 0.5-nél nagyobb, de az operatív küszöbnél a döntés gyenge.

23.1.6.3 3. feladat megoldása

A megadott mátrixból: TP=40, FN=60, TN=850, FP=50. Így

$$\text{precision} = \frac{40}{40+50} = 0.444, \quad \text{recall} = \frac{40}{40+60} = 0.4, \quad \text{accuracy} = \frac{40+850}{1000} = 0.89.$$

A 0.96-os ROC-AUC csak a pontszámok globális sorrendjét jellemzi; nem garantálja, hogy a 0.5-ös küszöbnél a precision és recall elfogadható. A hibás érvelés abban áll, hogy a küszöbválasztást a görbe alatti területből vezeti le. A döntéshez legalább a hamis pozitív és hamis negatív relatív költségére, valamint a beavatkozási kapacitásra lenne szükség; ezek nélkül a 0.5-ös küszöb önkényes.

23.1.6.4 4. feladat megoldása

Ha az osztályon belüli pontszámeloszlások változatlanok, a ROC-görbe TPR és FPR pontjai változatlanok, ezért a ROC-AUC közelítőleg állandó marad. A PR-görbe ezzel szemben erősen eltolódik, mert a precision függ az alaprátától:

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)}.$$

Például ugyanazon $\text{TPR} = 0.8$, $\text{FPR} = 0.2$ mellett $\pi = 0.15$-nél a precision $0.8 \cdot 0.15 / (0.8 \cdot 0.15 + 0.2 \cdot 0.85) = 0.12 / (0.12 + 0.17) = 0.414$, míg $\pi = 0.05$-nél $0.04 / (0.04 + 0.19) = 0.174$. A PR-AUC ezért lényegesen csökken az új régióban, míg a ROC-AUC nem feltétlenül. A gyakorlatban a prevalenciaváltozást külön monitorozni kell, és a küszöböt az aktuális alaprátához igazítani.

23.1.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a ROC-AUC-t közvetlen döntési metrikának tekintik. Az AUC azt méri, hogy a pontszámok átlagosan jól rangsorolnak-e, nem azt, hogy a kiválasztott küszöbnél hány téves riasztás keletkezik. Konkrét ellenpélda: ugyanaz a modell $\text{TPR} = 0.8$ és $\text{FPR} = 0.2$ működési ponttal 15%-os késési aránynál 0.414 precision-t ad, 5%-osnál 0.174-et. A ROC-görbén mindkét helyzet ugyanazt a pontot adja, tehát a ROC-AUC nem mutatja a döntési minőség romlását. Hasonlóan félrevezető a PR-AUC abszolút értékének átvitele egyik prevalenciájú adathalmazról a másikra: a PR-AUC csak az adott alaprátán érvényes.

Ez a jelenség a 19.1. alfejezetben bevezetett feltételes valószínűség közvetlen következménye: a precision a $P(y = 1 \mid \text{jelölt})$ feltételes arány, amely a prior eltolódásával változik, miközben a likelihood-alapú rangsor ugyanaz marad.

23.1.8 Önellenőrzés

- Ki tudom-e számolni kézzel a konfúziós mátrixot adott y, p és küszöb mellett?
- Megneveztem-e a döntési helyzetben a hamis pozitív és hamis negatív költségét?
- Megnéztem-e a base rate-et az értékelési ablakban, nem csak a tanítómintában?
- Külön kezeltem-e a küszöbválasztást a ROC-AUC jelentésétől?
- A log loss értékét a nem informatív 0.693-hoz, a ROC-AUC-ot 0.5-hez hasonlítottam?

23.1.9 Rövid összefoglaló

A metrika kiválasztása a döntési célból indul. Az accuracy ritka eseménynél félrevezető, mert a mindig tagadó nullmodell is magas értéket érhet el; ilyenkor a recall és precision a döntés hasznosságát mutatja. Folytonos célnál a MAE lineáris, az RMSE négyzetes hibabüntetést ad. A log loss a kalibrált valószínűségek minőségét méri, a ROC-AUC a küszöbtől független rangsort. A PR-AUC ritka eseménynél informatívabb, de függ az alaprátától. A küszöbválasztás mindig külön döntés, amelyhez a hibaköltségek és kapacitások ismerete szükséges. A következő 23.2. alfejezetben a keresztvalidációt igazítjuk a Northstar-idősor függőségi szerkezetéhez.

23.2 Keresztvalidáció a függőségi szerkezethez

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 23.1. alfejezet · Célkészség: Keresztvalidáció a függőségi szerkezethez · Kimenet: működő kód és a labor ellenőrző tesztje.

23.2.1 Cél és konkrét helyzet: összehasonlító értékelés a Northstar rendeléseken

A feladat a szokásos prediktív kérdés: rendelés pillanatában becsüljük, hogy a kézbesítés meghaladja-e a 7 napot. A bemenet a `lab/data/orders.csv` és a `lab/data/customers.csv`; a megfigyelési egység egy rendelés (`order_id`), a csoportazonosító a `customer_id`, az időablak pedig az `order_ts` UTC-időbélyeg. A célváltozó a bináris `target_late`, a prediktorok a rendelés pillanatában ismert `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`. Az előzetes becslés: a 22.2. alfejezetben látott HistGradientBoosting időbeli spliten kapott AUC-ja 0.70 körüli volt; most nem egyetlen scoreszám, hanem a felosztási stratégia hatása a kérdés. Ez vizsgálandó hipotézis, nem garantált sorrend: a csoportos és időbeli felosztás akár magasabb AUC-t is eredményezhet; a különbséget a validáció által szimulált alkalmazási helyzet szerint kell értelmezni.

23.2.2 Fogalmak: a fold mint a jövőbeli adat modellje

A keresztvalidáció (cross-validation) azt kérdezi: milyen jól általánosít a tanulási eljárás olyan adatra, amely nem volt a tanításban? A $D = \{(x_i, y_i)\}_{i=1}^{n}$ mintát K részre osztjuk; a k-adik iterációban a k-adik fold a validáció, a többi a tanítás. A modellt a tanítórészen illesztjük, a validációs részen értékeljük, majd átlagolunk:

$$\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{K}\sum_{k=1}^{K}\frac{1}{|I_k^{\mathrm{val}}|}\sum_{i\in I_k^{\mathrm{val}}} L(y_i, \hat{f}_k(x_i)),$$

ahol L a választott veszteség vagy negatív score, $\hat{f}_k$ a kihagyott foldon kívüli adaton tanult modell, I_k^{val} a validációs indexhalmaz, $|I_k^{\mathrm{val}}|$ annak mérete. A cél az, hogy ez a becslés a valódi jövőbeli használat hibáját közelítse.

A **KFold** indexek szerinti felosztást készít (alapesetben keverés nélkül, `shuffle=True` esetén véletlenszerűsítve). Független, azonos eloszlású minták mellett hasznos becslést adhat, de a CV-átlag nem általánosan torzítatlan becslő: a kisebb foldonkénti tanítóméret és a kiválasztási lépések is befolyásolhatják. A **StratifiedKFold** a véletlen felosztást úgy készíti, hogy minden foldban megközelítőleg azonos maradjon a célosztályok aránya; ritka eseménynél ez megakadályozza, hogy egy fold véletlenül nagyon kevés pozitív esetet tartalmazzon. A **GroupKFold** a csoportazonosítót figyeli: egy csoport minden mintája ugyanabba a foldba kerül, így a tanító- és validációs rész csoporthalmazai diszjunktak:

$$G_k^{\mathrm{tr}} \cap G_k^{\mathrm{val}} = \emptyset,$$

ahol G_k a foldhoz tartozó csoportazonosítók halmaza. A **TimeSeriesSplit** az időrendet őrzi: a k-adik splitben a tanítóhalmaz minden eleme korábbi, mint a validációs elemek:

$$I_k^{\mathrm{tr}} = \{i : t_i < \tau_k\}, \qquad I_k^{\mathrm{val}} = \{i : \tau_k \le t_i < \tau_{k+1}\}.$$

A **split manifest** a felosztás rögzített, verziózott leirata. Ha a foldindexek egy JSON-fájlban vannak, akkor a kísérlet pontosan reprodukálható akkor is, ha a környezetet újraépítjük; a manifest az adatsorra, a stratégiára és a seedre hivatkozik.

A matematikai híd itt a függőség. Ha az egy ügyfélhez tartozó rendelések korrelálnak, akkor egy ügyfél rendeléseinek trainbe és validationbe szétosztása olyan, mintha a modell a tesztidőpontban ismerné az ügyfél válaszainak egy részét; ez optimista AUC-hoz vezet. Az egyes foldok metrikái nem független minták: közös adatokból, átfedő tanítóhalmazokból származnak. Ezért a foldok egyszerű szórása nem a becslés mintavételi szórása. A 11.2. alfejezetben tanult optimalizálás itt is szerepet kap: minden foldban újraoptimalizáljuk a modell paramétereit, így a CV valójában a tanulási eljárás egészét értékeli, nem egyetlen paramétervektort.

23.2.3 Kézi kontroll egy kis példán

Tegyük fel, hogy hat rendelésünk van, két-két rendelés három ügyféltől:

Rendelés	Csoport	Idő	Cél
r1	c1	jan.	0
r2	c1	márc.	1
r3	c2	febr.	0
r4	c2	ápr.	0
r5	c3	máj.	1
r6	c3	jún.	1

Egy két folddal dolgozó KFold például a `{r1, r2, r3, r4}` tanítóhalmazt és `{r5, r6}` validációt adhat, majd fordítva. Ekkor `c1` és `c2` ügyfelek mintái mindkét oldalon előfordulhatnak. A GroupKFold ezzel szemben egy foldot a `c1`, a másikat a `c2, c3` csoportokra épít; az ügyfél soha nem szerepel egyszerre tanítóban és validációban. A TimeSeriesSplit egy bővülő ablakot ad: első splitnél train `{r1, r2}`, validation `{r3, r4}`; másodiknál train `{r1, r2, r3, r4}`, validation `{r5, r6}`. Látszik a két tulajdonság: a TimeSeriesSplit a jövőt nem engedi a múlt tanításába, de azonos ügyfél továbbra is átkerülhet a foldok között.

23.2.4 Teljes minimális munkapélda

A következő kód a három stratégiát hasonlítja össze a Northstar-adaton, a teljes pipeline-t validálva. A preprocessing a `ColumnTransformer`ben együtt van a modellel, így a `cross_validate` minden foldban csak a tanítórészen tanítja a skálázást és a one-hot kódolást. A kódot a projektgyökérből futtatva a `lab/data/` mappa elérhető.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import (KFold, StratifiedKFold, GroupKFold,
                                     TimeSeriesSplit, cross_validate)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Adat betöltése; a szerződés szerint a predikció pillanata order_ts.
orders = pd.read_csv("lab/data/orders.csv", parse_dates=["order_ts"])
customers = pd.read_csv("lab/data/customers.csv", parse_dates=["signup_date"])

df = orders.merge(customers[["customer_id", "signup_date", "segment",
```

```
                            "region", "acquisition_channel"]],
                  on="customer_id", how="left")
df["account_age_days"] = (df["order_ts"] - df["signup_date"]).dt.days

features = ["distance_km", "priority_flag", "basket_minor", "quantity_sum",
            "account_age_days", "segment", "region", "acquisition_channel"]
target = "target_late"
X = df[features]
y = df[target].astype(int)
groups = df["customer_id"].to_numpy()

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), ["distance_km", "basket_minor",
                               "quantity_sum", "account_age_days"]),
    ("pass", "passthrough", ["priority_flag"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
           ["segment", "region", "acquisition_channel"]),
])

model = Pipeline([
    ("prep", preprocessor),
    ("clf", HistGradientBoostingClassifier(
        max_iter=150, learning_rate=0.08, random_state=42)),
])

# 2. Három stratégia, azonos scoring és azonos modell.
scoring = "roc_auc"
cv_choices = {
    "kfold": KFold(n_splits=5, shuffle=True, random_state=20261020),
    "stratified": StratifiedKFold(n_splits=5, shuffle=True, random_state=20261020),
    "group": GroupKFold(n_splits=5),
}

results = {}
for name, cv in cv_choices.items():
    kwargs = {"groups": groups} if name == "group" else {}
    scores = cross_validate(model, X, y, cv=cv, scoring=scoring,
                            n_jobs=1, **kwargs)["test_score"]
    results[name] = scores

# 3. Időbeli split: az adatot order_ts szerint rendezzük.
df_time = df.sort_values("order_ts")
X_time = df_time[features]
y_time = df_time[target].astype(int)
# Demonstráció: az időrendet megtartja, de önmagában NEM garantálja,
# hogy a tanítócímkék rendelkezésre álltak az adott validáció kezdete előtt.
# Éles értékeléshez minden foldban ellenőrizni kell:
# max(label_available_at[train_idx]) < min(order_ts[val_idx]),
# vagy az adott validáció kezdetéig ismert címkékre szűrt train-indexet használni.
tscv = TimeSeriesSplit(n_splits=5)
scores_time = cross_validate(model, X_time, y_time, cv=tscv,
                             scoring=scoring, n_jobs=1)["test_score"]
results["time"] = scores_time

for name, scores in results.items():
    print(f"{name:10s} AUC folds: {scores.round(3)}  "
          f"mean={scores.mean():.3f}  sd={scores.std(ddof=1):.3f}")
```

A kimenetet szemléltető értékekkel jelölöm, mert ebben az írási módban nem futtattuk a kódot:

```
kfold       AUC folds: [0.71 0.69 0.73 0.68 0.72]  mean=0.706  sd=0.019
stratified  AUC folds: [0.70 0.71 0.68 0.69 0.73]  mean=0.702  sd=0.019
group       AUC folds: [0.66 0.64 0.67 0.65 0.63]  mean=0.650  sd=0.016
time        AUC folds: [0.68 0.61 0.63 0.65 0.60]  mean=0.634  sd=0.032
```

A tényleges számok a labor futtatásából származnak; a sorrend azonban a szerkezetből következik.

23.2.4.1 Két estimand, két split — és a kombinált eset

A split nem önmagában „helyes”: a tervezett estimandhoz kell illeszkednie. A két alapvető cél:

Estimand	Split	Kötelező ellenőrzés
Új ügyfélre való általánosítás	`GroupKFold` a `customer_id` szerint	a train és valid ügyfélhalmaz diszjunkt
Jövőbeli időszakra való előrejelzés	`TimeSeriesSplit` vagy cutoff	max(train order_ts) < min(valid order_ts) és max(train label_available_at) < min(valid order_ts)
Új ügyfél **és** új időszak	időblokkokra osztott csoportsplit	mindkét fenti feltétel egyszerre

A `GroupKFold` önmagában nem időhelyes: egy ügyfél jövőbeli rendelése taníthatja a múltbeli rendelésre illesztett modellt. A `TimeSeriesSplit` önmagában megengedi, hogy ugyanazon ügyfél rendelései a train és a valid oldalon is szerepeljenek. A kombinált estimandnál a tiszta megoldás az időbeli cutoffok használata: a cutoff előtt **teljesen lezárult** ügyfélhistóriák a trainbe, a cutoff után **kezdődő** ügyfelek a validba kerülnek, a cutoffot átfedő ügyfelek pedig purge-be; így egyszerre teljesül az ügyfél-diszjunktság és az időrend:

```
cutoffs = [df["order_ts"].quantile(q) for q in (0.5, 0.7, 0.9)]
first = df.groupby("customer_id")["order_ts"].min()
last = df.groupby("customer_id")["order_ts"].max()

for cutoff in cutoffs:
    train_customers = set(last[last < cutoff].index)
    valid_customers = set(first[first >= cutoff].index)
    spanning = set(first.index) - train_customers - valid_customers

    train = df[df["customer_id"].isin(train_customers)]
    valid = df[df["customer_id"].isin(valid_customers)]

    assert set(train["customer_id"]).isdisjoint(valid["customer_id"])
    assert train["order_ts"].max() < valid["order_ts"].min()
    assert train["label_available_at"].max() < valid["order_ts"].min()
```

A `spanning` ügyfelek száma külön riportálandó; ők azok, akiknek a története átnyúlik a cutoffon, ezért sem a trainbe, sem a validba nem tehetők anélkül, hogy valamelyik feltétel sérülne. A fenti ciklus három különböző cutoffra ad foldot; a végső értékelésnél a külső teszt továbbra is a cutoff utáni, érintetlen időszak.

23.2.5 Eredmény értelmezése

A KFold és a StratifiedKFold átlaga várhatóan magasabb, mert ugyanazon ügyfél korábbi rendelései a tanítóhalmazból szivárogtatnak információt az ügyfél későbbi rendeléseiről. A GroupKFold ezt kizárja, ezért alacsonyabb, de az időrendet nem tiszteli: egy ügyfél jövőbeli rendelése taníthatja a múltbeli rendelésre illesztett modellt. A TimeSeriesSplit a jövőt nem engedi a múltba, viszont megengedi, hogy ugyanazon ügyfél rendelései mindkét oldalon szerepeljenek;

ez a kezdőprojekt valós használatában részben elfogadható, ha a modell a rendelés pillanatában valóban látja az ügyfél előzményét. A tervezett használatra — jövőbeli rendelések becslése a rendelés pillanatában — a címkeelérhetőséget is ellenőrző, időalapú felosztás a minimálisan elfogadható felosztás; a GroupKFolddal való összevetés pedig megmutatja az ügyfél-átszivárgás mértékét. Ha új ügyfélre is szeretnénk általánosítani, a kettőt kombináló, időblokkokra osztott csoportsplitet kell építenünk, amelyet a scikit-learn beépítetten nem ad, de a foldindexeket generálva egy `GroupKFold` logikájával és `order_ts` szerinti szűréssel előállítható.

23.2.6 Feladatok

1. **Értelmezés.** A fenti szemléltető eredményben a TimeSeriesSplit AUC-átlaga a legalacsonyabb. Magyarázd el, miért nem jelenti ez azt, hogy a modell rosszabb lett; mit mér a különbség a KFoldhoz képest?

2. **Implementálás.** Adott a `df` DataFrame a fenti feature-ökkel, `y = df["target_late"]`, `groups = df["customer_id"]`. Írj kódot, amely `LogisticRegression(max_iter=1000)` modellt és `GroupKFold(n_splits=4)` felosztást használva `cross_val_score`-szal kiszámolja a `roc_auc` pontszámokat, majd kiírja az egyes foldok AUC-ját és az átlagot.

3. **Hibadiagnózis.** Az alábbi kódban valaki a foldok szórását konfidenciaintervallumként adja meg. Nevezd meg a két hibát, és javítsa az állítást úgy, hogy a kód kimenete ne állítson statisztikai bizonytalanságot a foldokból.

   ```
   scores = cross_val_score(model, X, y, cv=5, scoring="roc_auc")
   mean = scores.mean()
   se = scores.std(ddof=1) / np.sqrt(5)
   print(f"95% CI: {mean - 1.96*se:.3f} - {mean + 1.96*se:.3f}")
   ```

4. **Megváltozott feltétel.** A laboradat 300 ügyfelet tartalmaz, és a predikciót kifejezetten új ügyfelek első rendelésére kell általánosítani. Hány ügyfél kerül egy `GroupKFold(n_splits=5)` validációs foldjába? Milyen gyakorlati korlátja van ennek a kis csoportszámnak, és milyen két módosítást javasolna az értékelés megbízhatóságának növelésére?

23.2.7 Megoldások és indoklás

1. megoldás. A TimeSeriesSplit nem rosszabb modellt mutat, hanem kevésbé optimista feltételek mellett értékel. A KFoldnál azonos ügyfél tanítórendelései segítenek a validációs rendeléseken; ez a valóságban a predikció pillanatában csak akkor lenne elérhető, ha a modell az ügyfél összes korábbi és jövőbeli rendelését ismerné. A két pontszám különbsége önmagában nem a szivárgás mértéke: eltérő tanítóadat-méret, időbeli eltolódás és osztályarány is alakítja. Az ok megállapításához kontrollált összehasonlítás szükséges.

2. megoldás. A kód a `cross_val_score` `params` argumentumában adja át a csoportosítást:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score

model = LogisticRegression(max_iter=1000)
cv = GroupKFold(n_splits=4)
scores = cross_val_score(model, X, y, groups=groups, cv=cv,
                         scoring="roc_auc")
for i, score in enumerate(scores, start=1):
    print(f"fold {i}: {score:.3f}")
print(f"mean AUC: {scores.mean():.3f}")
```

Ellenőrző invariant: minden fold AUC-ja 0 és 1 közé esik, és az átlag a négy fold átlaga; ha a modell konvergens, az érték tipikusan 0.5 fölött van, de a pontos érték a futástól függ.

3. megoldás. Az első hiba: a foldmetrikák nem függetlenek, mert a tanítóhalmazok átfednek, ezért a `scores.std(ddof=1)` nem a mintavételi eloszlás szórása. A második hiba: a `np.sqrt(5)`-tel való osztás úgy tenné, mintha őt független minta lenne, ami hamis. Helyes megfogalmazás: a foldok az adatfelosztás szerinti teljesítmény stabilitását mutatják, nem konfidenciaintervallumot. A kód kimenetét ezért például így kell átírni:

```
print(f"Fold scores (AUC): {scores.round(3)}")
print(f"Mean: {scores.mean():.3f}, sd across folds: {scores.std(ddof=1):.3f}")
print("A fold sd nem konfidenciaintervallum; a bizonytalansághoz "
      "nested CV vagy bootstrap kell a megfelelő mintavételi egységgel.")
```

4. megoldás. `GroupKFold(n_splits=5)` esetén egy validációs foldban körülbelül 60 ügyfél van; a konkrét felosztás ettől kissé eltérhet, mert a csoportokat kell egészben tartani. A korlát: a validációs AUC így mindössze 60 ügyfél rendeléseire épül, ami nagy varianciát okoz; a foldok közötti különbség részben a kis csoportszámból fakad, nem valódi modellkülönbségből. Két módosítás: végezzünk több, kifejezetten csoportszintű újramintavételezést vagy független időszakon értékelést; a nagyobb foldszám önmagában nem növeli a rendelkezésre álló független ügyfelek számát, és a teljesítményt a csoportszintű bootstrap vagy nested CV segítségével értékeljük, ahol a külső ciklus a csoportokat bontja.

23.2.8 Gyakori tévedés: a foldok szórása nem konfidenciaintervallum

A 15.2. alfejezetben látott standard error a becslő mintavételi eloszlásából származik, nem egyetlen adathalmaz belső szóródásából. A CV-foldok éppen ilyen belső szóródást mutatnak: ugyanazokat az adatokat többféleképpen osztjuk fel, a tanítóhalmazok átfednek, ezért a foldmetrikák korreláltak. Az `sd(ddof=1)` kiszámítása még nem rossz, de az `1.96*se/√K` képlet alkalmazása hamis precizitást ad. A `GroupKFold` és `TimeSeriesSplit` foldjai ráadásul eltérő méretűek is lehetnek, ezért a szórásuk még kevésbé értelmezhető közvetlenül. A helyes bizonytalanságbecsléshez a teljes tanulási eljárást ismétlő mintavétel kell, például a megfelelő mintavételi egységre épített bootstrap vagy nested CV.

23.2.9 Önellenőrzés

- Miért okoz optimista AUC-t, ha egy ügyfél rendelései a trainben és a validationben is szerepelnek?
  Mert a modell megtanulhatja az ügyfélspecifikus zajt, ami a jövőbeli, valódi predikciónál nem áll rendelkezésre.
- Mikor választunk GroupKFoldot TimeSeriesSplit helyett?
  Ha a predikció elsősorban új csoportokra, például új ügyfelekre történik, és az időbeli sorrend kevésbé hangsúlyos, vagy kombináljuk mindkettőt.
- Mit jelent a split manifest?
  A foldindexek rögzített, verziózott leirata, amely a reprodukálhatóságot biztosítja.

23.2.10 Rövid összefoglaló

A keresztvalidáció csak akkor becsüli jól a jövőbeli hibát, ha a foldok a valódi használat függőségeit modellezik. A KFold független mintákat feltételez, ezért ügyfél- és időfüggő rendeléseknél optimista; a StratifiedKFold az osztályarányt őrzi. A GroupKFold a csoportot tartja egy oldalon, a TimeSeriesSplit a múltból a jövőbe tanít. A Northstar laborban a TimeSeriesSplit a tervezett rendelés pillanatára vonatkozó predikció minimálisan elfogadható felosztása, de új ügyfélre általánosításhoz csoportos idősplitre van szükség. A foldmetrikák szórása stabilitást mutat, nem konfidenciaintervallumot. Következő lépésként 23.3. alfejezetben a hiperparaméter-keresés és a nested értékelés kérdését vizsgáljuk.

23.3 Hiperparaméter és nested értékelés

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 23.2. alfejezet · Célkészség: Hiperparaméter és nested értékelés · Kimenet: működő kód és a labor ellenőrző tesztje.

23.3.1 Cél és konkrét helyzet

A Northstar predikciós projektben a logisztikus regresszió egyetlen hangolandó hiperparamétere a regularizáció erősségét szabályozó `C`. A kérdés: hogyan becsüljük meg a kiválasztott modell jövőbeli AUC-ját úgy, hogy a hiperparaméter-keresés ne tegye optimistává a becslést? Ebben az alfejezetben egy 400 rendelésből álló szintetikus mintát használunk; a megfigyelési egység egy rendelés, a cél bináris (`late` = 1, ha a kézbesítés 7 napnál későbbi), a prediktorok 8 rendeléskori jellemző. Az elvárt ellenőrzés: a külső értékelés AUC-átlaga és a teljes modellillesztésszám dokumentálása.

Induló feladat a módszer megismerése előtt: becsüld meg, hány modellillesztés történik, ha 5 külső foldot, belső foldonként 3-foldos keresést, 10 véletlen jelöltet és a legjobb jelölt újraillesztését használjuk? A pontos számítást a példában ellenőrizzük.

23.3.2 Fogalmi magyarázat: hiperparaméter, keresési tér és a két CV szerepe

A modellnek **tanult paraméterei** (például a logisztikus regresszió súlyvektora w) és **hiperparaméterei** (hyperparameters) vannak. A hiperparamétert a tanulási algoritmus elindítása előtt rögzítjük; például a logisztikus regresszió `C` paramétere a súlyok büntetésének inverz erőssége. Kis `C` erős regularizációt, nagy `C` gyenge regularizációt jelent, ahogy azt a 22.3. alfejezetben az SVM margójánál is láttuk.

A **keresési tér** (search space) a kipróbálandó hiperparaméter-értékek halmaza. Két gyakori forma:

- **Rácsos keresés** (grid search): megadott értékek összes kombinációja, például `C {0.01, 0.1, 1.0}`. A kombinációk száma a rács méretének szorzata.
- **Véletlen keresés** (randomized search): adott eloszlásból veszünk `n_iter` darab véletlen kombinációt. Ez akkor előnyös, ha a keresési tér nagy, és a költségvetés korlátozott.

A **költségvetés** (budget) a megengedett modellillesztések vagy futási idő felső korlátja. Minden egyes illesztés iteratív optimalizálást futtat – a gradiens alapú lépéseket a 11.3. alfejezetben ismertük meg –, ezért a futási idő nem elhanyagolható. A teljes illesztésszám az összes belső foldon végzett jelöltillesztések összege, valamint minden külső fold egy újraillesztése: $K_{outer}(K_{inner}N_{jellt} + 1)$, ha az alapértelmezett `refit=True`.

A **belső és külső keresztvalidáció** (inner/outer CV) célja a kiválasztás és az értékelés szétválasztása. A belső CV a modellválasztás becslője: minden jelöltet a belső foldokon értékelünk, és a legjobb átlagpontszámút választjuk. A külső CV a teljes kiválasztási eljárás értékelője: minden külső fold tanítórészén belül újra lefuttatjuk a teljes belső keresést, majd a kiválasztott modellt a külső teszt-részen egyszer értékeljük. Így a külső teszt soha nem vesz részt a hiperparaméter-választásban.

A **kiválasztási torzítás** (selection bias) a legfontosabb statisztikai veszély. Tegyük fel, hogy 20 jelölt valódi AUC-je egyaránt $\mu = 0.70$, de egy-egy belső CV becslés szórása $\sigma = 0.03$. A belső CV-pontszámok tehát véletlen zajjal terheltek. Ha kiválasztjuk a legjobbat, annak várható értéke nem μ, hanem

$$E\Big[\max_i X_i\Big] \;=\; \mu + \sigma\, E\Big[\max_i Z_i\Big],$$

ahol X_i a belső CV AUC-becslések, Z_i standard normális változók, $i = 1, \dots, n$, és n a jelöltek száma. Húsz jelöltnél a standardizált maximum várható értéke körülbelül 1.9, így a várható nyertes pontszám $0.70 + 0.03 \cdot 1.9 \approx 0.757$. A nyertes pontszám tehát akkor is felfelé torzított, ha egyetlen jelölt sem jobb a többinél. Ezt a szimulációt a gyakori tévedésnél ellenőrizzük.

23.3.3 Teljes minimális munkapélda

Az alábbi kód szintetikus adatot generál, majd öt külső foldon belül tíz jelöltes véletlen keresést futtat három belső folddal. A külső tesztet a keresés soha nem látja. A példa a lab/pyproject.toml alap numerikus csomagjait használja; a kód a projektgyökérből futtatható.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
import numpy as np
from scipy.stats import loguniform

# 1. Bemenet: 400 független szintetikus rendelés, 8 feature, bináris cél.
rng = np.random.default_rng(42)
X, y = make_classification(
    n_samples=400, n_features=8, n_informative=5, n_redundant=2,
    random_state=42, flip_y=0.12
)

# 2. Külső split: 5 rétegzett fold. A shuffle és a seed rögzített.
outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=123)

# 3. A modell pipeline: skálázás + logisztikus regresszió.
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(max_iter=1000, solver='liblinear'))
])
```

```
# 4. Keresési tér: C log-uniform eloszlásból, 10 jelölt.
param_dist = {'clf__C': loguniform(1e-3, 1e1)}

outer_scores = []
inner_best_scores = []
best_params_per_fold = []
n_fits = 0

# 5. Nested CV: minden külső foldban új keresést indítunk.
for train_idx, test_idx in outer.split(X, y):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    search = RandomizedSearchCV(
        pipe, param_dist, n_iter=10, cv=3, scoring='roc_auc',
        random_state=7, refit=True, n_jobs=1
    )
    search.fit(X_train, y_train)

    # Illesztésszám: 10 jelölt * 3 belső fold + 1 refit = 31.
    n_fits += search.n_iter * 3 + 1

    y_score = search.predict_proba(X_test)[:, 1]
    outer_scores.append(roc_auc_score(y_test, y_score))
    inner_best_scores.append(search.best_score_)
    best_params_per_fold.append(search.best_params_)

print("Külső fold AUC-k:", np.round(outer_scores, 3))
print("Külső AUC átlag:", np.mean(outer_scores).round(3))
print("Külső AUC szórás (ddof=1):", np.std(outer_scores, ddof=1).round(3))
print("Belső best_score-k:", np.round(inner_best_scores, 3))
print("Teljes illesztésszám:", n_fits)
print("Kiválasztott C-k:", best_params_per_fold)
```

Szemléltető kimenet (a pontos számok a véletlen generálás miatt enyhén eltérhetnek; a kód nem futtatott):

```
Külső fold AUC-k: [0.712 0.689 0.731 0.704 0.718]
Külső AUC átlag: 0.711
Külső AUC szórás (ddof=1): 0.016
Belső best_score-k: [0.752 0.741 0.768 0.755 0.749]
Teljes illesztésszám: 155
Kiválasztott C-k: [{'clf__C': 0.047...}, ...]
```

Ha nagyobb keresést futtatnánk, a `n_jobs=-1` beállítás a joblib segítségével párhuzamosíthatná a belső foldokat, de ez nem változtatja meg az illesztések számát, csak a falidőt. A példában `n_jobs=1` a tiszta költségelszámolás miatt szerepel.

23.3.4 Eredmény értelmezése és ellenőrzés

A külső fold AUC-k átlaga 0.711, ami alacsonyabb, mint a belső `best_score` értékek átlaga (0.753). Ez a különbség a kiválasztási torzítás gyakorlati jele: a belső CV-pontszám a keresési folyamat részeként optimista, míg a külső foldok a kiválasztott eljárást olyan részeken értékelik, amelyeket a keresés nem látott.

Kézi kontroll: a `StratifiedKFold` garantálja, hogy minden külső teszt halmaz diszjunkt a hozzá tartozó tanítóhalmaztól, és a foldok lefedik a teljes 400 elemű mintát. A teljes illesztésszámot a kód expliciten számolja: $5 \times (10 \times 3 + 1) = 155$. Ez a képlet minden nested CV futásnál ellenőrizhető invariáns.

A statisztikai értelmezésnél fontos korlát: a külső foldok AUC-i nem függetlenek, mert a tanítóhalmazok átfednek. A 0.016-os szórást ezért nem szabad konfidenciaintervallumként értelmezni. A külső CV átlaga jobb becslés, mint a belső nyertes pontszám, de a bizonytalanság pontos számszerűsítéséhez ismételt külső splitre vagy további statisztikai módszerre lenne szükség.

23.3.4.1 Jelentési séma: belső tuning, külső becslés, végső modell, teszt

A nested CV négy különböző dolgot állít elő, és ezeket a jelentésben külön kell szerepeltetni:

Szakasz	Milyen adaton?	Eredmény	Mit jelent?
Belső tuning	minden külső fold tanító részén, belső CV-vel	foldonkénti `best_params_`, `best_score_`	a keresés eredménye az adott foldon; nem a végleges paraméter
Külső becslés	a külső foldok teszt részén	foldonkénti AUC, átlag, szórás	az **eljárás** teljesítményének becslése, nem egyetlen modellé
Végső modell	a teljes tanítóhalmazon (a külső foldok uniója)	végleges paraméterek, újraillesztett modell	a bevetésre szánt modell; nincs rá független CV-becslés
Érintetlen teszt	a külső CV-től elkülönített, egyszer használt minta	AUC/log loss, egyszeri jelentés	a végső modell független, de zajos pontbecslése

A leggyakoribb hiba, hogy a foldonkénti `best_params_` értékeket valaki átlagolja vagy az egyiket „a végleges paraméternek” tekinti. A végleges paraméterválasztáshoz **előre rögzített szabály** kell: például a teljes tanítóhalmazon futtatott újabb keresés, a leggyakoribb foldonkénti beállítás, vagy a legjobb átlagos belső pontszám. A külső CV átlaga az eljárást értékeli, a végleges modell teljesítményét nem garantálja; ezért a végső modellhez külön, érintetlen teszt szükséges, és azt csak egyszer szabad megnyitni.

Jelentésminta:

```
## Belső tuning
- foldonkénti best_params és best_score; a keresési tér és a költségvetés rögzítése
## Külső becslés
- külső fold AUC-k, átlag, szórás; a szórás nem konfidenciaintervallum
## Végső modell
- a végleges paraméterválasztás előre rögzített szabálya; újraillesztés a teljes tanítóhalmazon
## Érintetlen teszt
- egyszeri AUC/log loss a végső modellen, a döntés időpontjában ismeretlen státusszal
```

23.3.5 Feladatok

1. Értelmezés. Miért magasabb a belső `best_score` értékek átlaga, mint a külső fold AUC-k átlaga? Magyarázd meg a kiválasztási torzítás fogalmával, és nevezz meg legalább egy helyzetet, amikor a különbség nagyobb lehet.

2. Implementálás. Cseréld a `RandomizedSearchCV`-t `GridSearchCV`-re, és használd a `{'clf__C': [0.01, 0.1, 1.0]}` rácsot. Mennyi lesz a teljes illesztésszám? Módosítsd a kódot, és add meg az új külső AUC átlagot.

3. Hibakeresés. Egy kolléga az egész adaton futtat egy `RandomizedSearchCV`-t, majd a `search.best_score_` értéket közli mint „külső validációs AUC 0.76”. Mi a hiba ebben az állításban? Milyen egyszerű ellenőrzést javasolnál a torzítás kimutatására?

4. Transzfer. Tegyük fel, hogy egy modellillesztés átlagos futási ideje a te gépeden 0.8 másodperc (ez becsült érték, nem mért). A jelenlegi 155 illesztés hány másodperc? Ha a jelöltek számát 20-ra, a belső foldok számát pedig 5-re növeled, mennyi lesz az illesztésszám és a várható futási idő? Ha a teljes futási időnek 5 perc alatt kell maradnia, milyen költségvetési döntést hoznál?

5. Bónusz értelmezés. Miért nem értelmezhető a külső foldok AUC-inak szórása konfidencia-intervallumként? Milyen további lépéssel lehetne a bizonytalanságot jobban becsülni?

23.3.6 Megoldások és indoklás

1. A belső `best_score` minden foldban a 10 jelölt közül a legjobb CV-pontszám. Mivel minden pontszám zajjal terhelt, a maximum várhatóan magasabb, mint a jelöltek valódi teljesítménye. A külső AUC viszont a kiválasztott modellt olyan tesztrészen méri, amelyet a keresés nem használt, ezért nem tartalmazza ezt a kiválasztási optimizmust. A különbség nagyobb, ha sok jelölt van, a belső foldok kicsik (nagyobb zaj), vagy a valódi hatás gyenge.

2. A GridSearchCV 3 jelöltet próbál, mindegyiket 3 belső foldon: $3 \times 3 + 1 = 10$ illesztés belső ciklusonként. Öt külső folddal a teljes illesztésszám $5 \times 10 = 50$. A kódban a `RandomizedSearchCV` helyett:

```
from sklearn.model_selection import GridSearchCV

param_grid = {'clf__C': [0.01, 0.1, 1.0]}
search = GridSearchCV(pipe, param_grid, cv=3, scoring='roc_auc', refit=True)
```

A külső AUC átlag szemléltető értéke 0.705 körül alakulhat; a pontos szám függ a seedtől és a környezettől. Az ellenőrzés: a teljes illesztésszámnak 50-nek kell lennie, és a külső AUC átlagnak közelebb kell lennie a valódi teljesítményhez, mint a belső best_score átlagnak.

3. A kolléga kétszeresen használja ugyanazt az adatot: a kereséshez és a „validációhoz” is. Az adaptív kiválasztás után a `best_score_` már nem torzítatlan becslés; ez a selection bias klasszikus esete. Az ellenőrzés: osszuk az adatot egy friss, soha nem látott teszt részre, és mérjük ott a legjobb modell AUC-ját. Ha a friss AUC számottevően alacsonyabb, mint a `best_score_`, a torzítás kimutatható.

4. A jelenlegi futási idő: $155 \times 0.8 = 124$ másodperc, azaz körülbelül 2 perc. Az új beállítás: $5 \times (20 \times 5 + 1) = 5 \times 101 = 505$ illesztés, ami $505 \times 0.8 = 404$ másodperc, azaz körülbelül 6 perc 44 másodperc. Ez meghaladja az 5 perces keretet. Egy lehetséges döntés: 20 jelöltet tartunk, de a belső foldok számát 3-ra csökkentjük; ekkor $5 \times (20 \times 3 + 1) = 305$ illesztés, $305 \times 0.8 = 244$ másodperc, azaz körülbelül 4 perc. Másik lehetőség: 15 jelölt 5 belső folddal: $5 \times (15 \times 5 + 1) = 380$ illesztés, 304 másodperc, szintén 5 perc alatt. A döntés a belső foldok stabilitása és a keresési tér lefedettsége közötti kompromisszum.

5. A külső foldok tanítóhalmazai átfednek: minden fold a többi 4 fold adatának nagy részét tartalmazza. Emiatt a foldok AUC-i korreláltak, és a szórásuk alábecsüli a valódi mintavételi bizonytalanságot. A jobb becsléshez ismételt nested CV-t futtathatunk több különböző külső split véletlen maggal, vagy a teljes adaton bootstrap mintákat használhatunk, de ez utóbbi is feltételezi a megfigyelések függetlenségét. A pontosabb módszerek meghaladják ezen alfejezet keretét.

23.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba: a legjobb CV-pontszámot a keresés után torzítatlan külső becslésként közölni. A következő szimuláció megmutatja, hogy ez akkor is félrevezető, ha minden jelölt azonos valódi teljesítményű.

```
import numpy as np

rng = np.random.default_rng(20261020)
n_candidates = 20
n_repeats = 2000
mu = 0.70
sigma = 0.03

maxima = np.empty(n_repeats)
for i in range(n_repeats):
    scores = rng.normal(mu, sigma, size=n_candidates)
    maxima[i] = scores.max()

print("Átlagos legjobb CV-pontszám:", maxima.mean().round(3))
print("Valódi közös AUC:", mu)
print("Optimizmus (bias):", (maxima.mean() - mu).round(3))
```

Szemléltető kimenet (a kód nem futtatott, a várható érték a normális közelítésből):

```
Átlagos legjobb CV-pontszám: 0.757
Valódi közös AUC: 0.7
Optimizmus (bias): 0.057
```

A szimuláció független normális eloszlást feltételez, ami a valóságban ritkán teljesül pontosan, de az intuíciót jól mutatja: 20 jelölt közül a legjobb pontszám várhatóan 0.76 körül lesz, pedig egyetlen jelölt sem jobb 0.70-nél. Ez a nested CV motivációja: a külső foldok átlagával a kiválasztási optimizmus nagy részét elkerüljük.

23.3.8 Önellenőrzés

- El tudod magyarázni saját szavaddal, mi a különbség a belső és a külső CV szerepe között?
- Ki tudod számolni a teljes illesztésszámot tetszőleges külső fold, belső fold, jelöltszám és refit beállítás esetén?
- Felismered-e, amikor valaki belső CV-pontszámot közöl külső validációs eredményként?
- Meg tudod nevezni a külső foldok AUC-szórásának legalább egy korlátját?

23.3.9 Rövid összefoglaló

A hiperparaméter-keresés adaptív adathasználat, ezért a nyertes belső CV-pontszám felfelé torzított. A nested CV szétválasztja a kiválasztást és az értékelést: minden külső fold tanítórészén belül új teljes keresést futtatunk, majd a kiválasztott modellt a külső teszten egyszer értékeljük. A teljes költséget a foldok, a jelöltek és a refit szorzata adja; a példában 155 illesztés történt. A külső fold AUC-k átlaga közelebb áll a valódi általánosításhoz, de a foldok átfedése miatt a szórásuk nem konfidenciaintervallum. A következő alfejezetben a küszöb, kalibráció és ritka esemény problémáit vizsgáljuk ugyanezzel a korrektségi szemlélettel.

23.4 Küszöb, kalibráció és ritka esemény

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 23.3. alfejezet · Célkészség: Küszöb, kalibráció és ritka esemény · Kimenet: működő kód és a labor ellenőrző tesztje.

23.4.1 Cél és konkrét helyzet

A Northstar prediktív projektben a 23.3. alfejezet egység nested keresztvalidációval kiválasztotta a logisztikus regressziót, és befagyasztotta a split manifestet. A modell minden rendeléshez egy $\hat{p}_i$ késési valószínűséget ad, de ez még nem döntés: a működéshez küszöb kell. A kérdés nem az, hogy „mekkora az AUC", hanem hogy milyen t küszöbre mondjuk azt, hogy a rendelés várhatóan késik, és mennyibe kerül ez a döntés.

A feladat bemenete:

- a tiszta `data/orders.csv` és `data/customers.csv`,
- az `artifacts/split.json` train/test/purged sorazonosítói,
- a befagyasztott feature-lista: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`,
- a célváltozó `target_late` (0/1),
- a döntési költségtábla: hamis riasztás $C_{FP} = 1$, kihagyott késés $C_{FN} = 5$ költségegység rendelésenként.

Megfigyelési egység egy rendelés; a döntés a rendelés pillanatában születik, a címke csak később válik ismertté. A labor kimenete egy befagyasztott küszöb, a validhalmaz kalibrációs és költséggörbéje, végül az érintetlen teszt egyszeri költségértékelése. Induló becslésként ellenőrizze a tanítóhalmaz késési arányát: ha például 9%, akkor a mindig negatív baseline költség $0{,}09 \cdot 5 = 0{,}45$ egység/rendelés; a modellnek ennél kell jobbnak lennie.

23.4.2 Küszöb, várható költség és kalibráció

Küszöb (threshold). A modell $p_i = \hat{P}(y_i = 1 \mid x_i)$ folytonos kimenetét a $\hat{y}_i = \mathbf{1}[p_i > t]$ szabállyal alakítjuk bináris döntéssé. A t megválasztása trade-off: alacsonyabb küszöb több riasztást és több hamis riasztást, magasabb küszöb több kihagyott eseményt ad. A confusion matrix minden t-re más:

Döntés	Tényleges 0	Tényleges 1
$\hat{y} = 0$	TN	FN
$\hat{y} = 1$	FP	TP

Várható költség (expected cost). A költségtábla csak a hibás döntéseket árazza: a hamis riasztás C_{FP}, a kihagyott esemény C_{FN}, a helyes döntés 0. A validhalmazon n_v megfigyeléssel a várható költség:

$$EC(t) = \frac{1}{n_v} \sum_{i=1}^{n_v} \left[\mathbf{1}(p_i > t, y_i = 0)\, C_{FP} + \mathbf{1}(p_i \le t, y_i = 1)\, C_{FN} \right].$$

Itt $\mathbf{1}(\cdot)$ az indikátorfüggvény, értéke 1, ha a feltétel igaz, különben 0; p_i a prediktált valószínűség, y_i a tényleges címke. Az $EC(t)$ mértékegysége költségegység/rendelés. A döntési probléma célja $EC(t)$ minimalizálása a validhalmazon.

Ha a modell valószínűségei jól kalibráltak és a költségek nem függnek p-től, az elméleti optimum levezethető feltételes várható értékkel: egy p pontszámú rendelés riasztásának várható költsége $p\,C_{TP}$-szerű, és a határ ott van, ahol a riasztás várható többletköltsége egyenlő a nem riasztáséval. A gyakorlati képlet egyszerű:

$$t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

A jelen költségtáblával $t^* = 1/(1+5) \approx 0{,}167$. Ez nem azt jelenti, hogy pontosan ezt a küszöböt kell használni; a valid görbe empirikus minimuma a véges minta és a kalibrálatlanság miatt eltérhet.

Kalibrációs görbe (calibration curve). A kalibráció azt kérdezi: amikor a modell p valószínűséget mond, a tényleges pozitív arány valóban p körül van-e? A validhalmaz predikcióit binekre osztjuk, minden binben kiszámoljuk az átlagos predikciót $\bar{p}_k$ és a tényleges pozitív arányt $\bar{y}_k$. A kalibrációs görbe ezeket ábrázolja; az ideális görbe az átló. A logisztikus regresszió gyakran jól kalibrált, a random forest és a boosting viszont túl magabiztos lehet – erre a későbbi 26.2. alfejezet épít.

Brier-pontszám (Brier score). Az egyetlen számba sűrített kalibrációs és élességi mérték a predikciók és a címkék négyzetes eltérése:

$$B = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2.$$

Értéke mindig 0 és 1 között van; $B = 0$ a tökéletes, mindig magabiztos előrejelzés. A mindig $\bar{y}$ átlag-arányt mondó nullmodell Brier-értéke $\bar{y}(1-\bar{y})$, tehát ritka eseménynél már egy egyszerű konstans is alacsony Brier-t ad; a modellt ehhez, nem 0-hoz kell viszonyítani.

Imbalance és a resampling helye. A késés ritka esemény: ha a pozitív arány $\pi \approx 0{,}09$, a mindig negatív modell accuracy-je már 0,91. Az accuracy ezért nem választ küszöböt. Ha a ritkaságot SMOTE-tal vagy más resamplinggel kezeljük, a **resampling csak a keresztvalidáció tanító foldján belül** futhat, különben a szintetikus pontok a validációs fold információját szivárogtatják. A `imblearn.pipeline.Pipeline` ezt a korlátot beépítve kezeli.

23.4.3 Teljes minimális munkapélda

A következő szkript a lab gyökeréből futtatható:

```
python notebooks/s092_threshold_calibration.py
```

A kód előfeltétele, hogy a `starter_experiment.py` már létrehozta az `artifacts/split.json`-t. A split kulcsait a projekt manifestjében ellenőrizze; a példa a `train`, `test`, `purged` neveket használja.

```
# notebooks/s092_threshold_calibration.py
from pathlib import Path
import json
import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    brier_score_loss, confusion_matrix, roc_auc_score
)
from sklearn.calibration import calibration_curve

DATA = Path("data")
ART = Path("artifacts")
SPLIT_PATH = ART / "split.json"

C_FP = 1.0   # hamis riasztás: fölösleges figyelmeztetés
C_FN = 5.0   # kihagyott késés: ügyfélkompenzáció
SEED = 20261020

orders = pd.read_csv(DATA / "orders.csv", parse_dates=["order_ts", "actual_delivery_at",
↪  "label_available_at"])
customers = pd.read_csv(DATA / "customers.csv", parse_dates=["signup_date"])
df = orders.merge(
    customers[["customer_id", "signup_date", "segment", "region", "acquisition_channel"]],
    on="customer_id", how="inner"
)
df["account_age_days"] = (df["order_ts"] - df["signup_date"]).dt.days

with open(SPLIT_PATH) as f:
    split = json.load(f)
train_ids = split["train"]
test_ids = split["test"]
purged_ids = split["purged"]  # a kihagyott határsáv, itt nem használjuk

feature_cols = ["distance_km", "priority_flag", "basket_minor", "quantity_sum",
                "account_age_days", "segment", "region", "acquisition_channel"]
target_col = "target_late"

X_train_full = df[df["order_id"].isin(train_ids)][feature_cols].copy()
y_train_full = df[df["order_id"].isin(train_ids)][target_col].copy()
X_test = df[df["order_id"].isin(test_ids)][feature_cols].copy()
y_test = df[df["order_id"].isin(test_ids)][target_col].copy()

# Belső split: inner_train a modellhez, threshold_valid a küszöbválasztáshoz.
X_inner, X_valid, y_inner, y_valid = train_test_split(
    X_train_full, y_train_full, test_size=0.30,
    random_state=SEED, stratify=y_train_full
)

numeric_cols = ["distance_km", "priority_flag", "basket_minor", "quantity_sum",
↪  "account_age_days"]
categorical_cols = ["segment", "region", "acquisition_channel"]
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
])

model = Pipeline([
```

```
    ("prep", preprocessor),
    ("clf", LogisticRegression(max_iter=1000, random_state=SEED))
])
model.fit(X_inner, y_inner)

p_valid = model.predict_proba(X_valid)[:, 1]
brier_valid = brier_score_loss(y_valid, p_valid)
auc_valid = roc_auc_score(y_valid, p_valid)

thresholds = np.linspace(0.01, 0.99, 199)
costs = []
for t in thresholds:
    y_pred = (p_valid >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_valid, y_pred).ravel()
    costs.append((fp * C_FP + fn * C_FN) / len(y_valid))
costs = np.array(costs)
best_t = thresholds[int(np.argmin(costs))]
best_valid_cost = costs.min()

frac_pos, mean_pred = calibration_curve(y_valid, p_valid, n_bins=10)

# Befagyasztott küszöb: az érintetlen teszt egyszeri értékelése.
model.fit(X_train_full, y_train_full)  # újratanítás a teljes trainen
p_test = model.predict_proba(X_test)[:, 1]
y_pred_test = (p_test >= best_t).astype(int)
tn_t, fp_t, fn_t, tp_t = confusion_matrix(y_test, y_pred_test).ravel()
test_cost = (fp_t * C_FP + fn_t * C_FN) / len(y_test)
brier_test = brier_score_loss(y_test, p_test)
auc_test = roc_auc_score(y_test, p_test)

result = {
    "inner_n": int(len(X_inner)),
    "valid_n": int(len(X_valid)),
    "test_n": int(len(X_test)),
    "train_late_rate": float(y_train_full.mean()),
    "brier_valid": float(brier_valid),
    "auc_valid": float(auc_valid),
    "theoretical_threshold": C_FP / (C_FP + C_FN),
    "chosen_threshold": float(best_t),
    "valid_cost_at_chosen": float(best_valid_cost),
    "test_cost_at_chosen": float(test_cost),
    "test_confusion": {"tn": int(tn_t), "fp": int(fp_t),
                       "fn": int(fn_t), "tp": int(tp_t)},
    "brier_test": float(brier_test),
    "auc_test": float(auc_test),
}

ART.mkdir(exist_ok=True)
joblib.dump((model, best_t), ART / "s092_threshold_model.pkl")
with open(ART / "s092_threshold_result.json", "w") as f:
    json.dump(result, f, indent=2, ensure_ascii=False)

print(json.dumps(result, indent=2, ensure_ascii=False))
```

A tényleges futás a helyi lab projektben ellenőrzi a számokat. Az alábbiak a várt tartományt szemléltetik, nem a mért eredményt: a train késési arány körülbelül 8–10%, a valid optimum t^* közelében, körülbelül 0,15–0,20 között, a tesztköltség pedig a baseline 0,4–0,5 alatt várható. A pontos értékeket a manifest és a saját futtatás igazolja.

23.4.4 Eredmény értelmezése

A kód három dolgot választ el. A `threshold_valid` halmaz a küszöbgörbe és a kalibráció tere; a választott t a valid költséggörbe minimuma. Ezután a modellt újratanítjuk a teljes trainen, és a befagyasztott küszöbbel a tesztet pontosan egyszer értékeljük. A teszt confusion matrix és költség már nem vezérel további modell- vagy küszöbválasztást.

A valid költséggörbe aszimmetrikus: a $C_{FN} = 5$ miatt a túl magas küszöb drága, mert kihagy késéseket. A $t^* = 0{,}167$ elméleti optimum csak akkor pontos, ha a kalibrációs görbe az átlón van. Ha a valid halmazon a modell alacsony p tartományban alulbecsül, az optimum jobbra tolódik. A Brier-pontszám együtt olvasandó a kalibrációs görbével: az alacsony Brier nem garancia a jó kalibrációra, ha az esemény ritka.

A tesztköltség becslése maga is mintavételi becslés: a kis tesztmintán a költség standard hibája a költségarányok binomiális szórásából közelíthető, de egyszeri futtatásban csak a pontbecslés jelenik meg. A nested CV 23.3. alfejezet becsléséhez hasonlóan ez a teszt is egyetlen véletlen szelet; a szórását nem a teszt többszöri újraoptimalizálásával, hanem resamplinggel vagy a jövőbeli monitoringgal lehet becsülni.

A kalibrálót kizárólag a modell kiválasztásától és a végső teszttől elkülönített adaton illesszük, vagy használjunk szivárgásmentes CV-t. Ritka eseménynél a PR-görbe baseline-ja az adott halmaz pozitív aránya.

23.4.4.1 Belső kalibrációs CV és időszeles ECE

A kalibrátor (sigmoid vagy izotonikus) maga is tanul az adatból, ezért a tanító- és a kiértékelő-halmaz nem lehet ugyanaz. Két helyes séma:

1. **Külön kalibrációs blokk:** a modellt a korábbi `fit` blokkon tanítjuk, a kalibrátort a későbbi `cal` blokkon illesztjük, a tesztet csak ezután érjük el.
2. **Belső időrendi CV:** a kalibrátort `TimeSeriesSplit` belső foldjain illesztjük; a külső teszt érintetlen marad.

A scikit-learn 1.9.1-ben a korábbi `cv="prefit"` eltűnt; a már illesztett alapmodellt `FrozenEstimator`-ba kell tenni:

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator

base.fit(X_fit, y_fit)
calibrated = CalibratedClassifierCV(
    FrozenEstimator(base), method="sigmoid"
).fit(X_cal, y_cal)
p_cal = calibrated.predict_proba(X_test)[:, 1]
```

Az **ECE** (expected calibration error) a kalibráció binelt átlagos eltérése:

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{n} \left| \bar{y}_b - \bar{p}_b \right| ,$$

ahol n_b a b. bin elemszáma, $\bar{y}_b$ a tényleges pozitív arány, $\bar{p}_b$ az átlagos predikció. Az ECE-t nem csak a teljes teszten, hanem időszelesen is érdemes jelenteni, mert a kalibráció a későbbi időablakokban romolhat.

Szemléltető példa ($n = 3000$, időrendi blokkok: fit 1800, cal 600, test 600; random forest alapmodell, sigmoid kalibráció):

Minta	Nyers Brier	Kalibrált Brier	Nyers log loss	Kalibrált log loss	Nyers ECE	Kalibrált ECE
Teljes teszt	0,2553	0,1882	0,7077	0,5596	0,2617	0,0618
Teszt 1. fele	0,2346	0,1857	–	–	0,2308	0,0581
Teszt 2. fele	0,2760	0,1907	–	–	0,3002	0,0895

A kalibrátor a `cal` blokkon illeszkedett, a tesztet nem látta; a későbbi időszakban is javítja a Brier-t és az ECE-t. Kis kalibrációs blokk esetén az izotonikus regresszió könnyen túlilleszkedik, ezért ilyenkor a sigmoid biztonságosabb; a választást a kalibrációs blokk mérete és az időszeles ECE együtt indokolja.

23.4.5 Feladatok

1. **Értelmezés.** Az alábbi valid confusion matrix adott: TP=40, FP=20, FN=10, TN=930. Számítsd ki a várható költséget $C_{FP} = 1$, $C_{FN} = 5$ mellett, a pontosságot (accuracy), valamint a mindig negatív baseline költségét. Miért vezet félre az accuracy a küszöbválasztásban?

2. **Megvalósítás.** Változtassa a költségtáblát $C_{FP} = 2$, $C_{FN} = 10$ értékekre, és futtassa újra a küszöbgörbét. Mi az elméleti t^*, és miért nem változik? Mi változik viszont a költség mértékegységében és a tesztköltség értékében?

3. **Hibadiagnózis.** A valid költség 0,36, a teszt költség viszont 0,71. Soroljon fel legalább három lehetséges okot, és nevezze meg, melyik okot hogyan zárná ki a split manifest, a label-availability szűrés és a feaure-ök időbeli eloszlásának összehasonlításával.

4. **Megváltozott feltétel.** Tegyük fel, hogy a train halmaz késési aránya 6%, és egy kolléga a teljes trainen SMOTE-ot futtat, mielőtt a keresztvalidációt lefuttatja. Magyarázd el, miért szivárogtat ez információt a valid foldokba, és írd le a helyes `imblearn.pipeline.Pipeline` szerkezetet, amelyben a resampling a tanító foldon belül marad.

23.4.6 Megoldások és indoklás

1. feladat. A confusion matrix cellái: $FP = 20$, $FN = 10$, $n = 1000$. A várható költség:

$$EC = \frac{20 \cdot 1 + 10 \cdot 5}{1000} = 0{,}07.$$

Az accuracy $(40 + 930)/1000 = 0{,}97$. A mindig negatív modell esetén $FP = 0$, $FN = 50$, tehát $EC = 50 \cdot 5/1000 = 0{,}25$, accuracy $950/1000 = 0{,}95$. Az accuracy alig különbözik, miközben a költség több mint háromszoros; ritka eseménynél az accuracy a konstans negatív szabályt is magasra jutalmazza, ezért nem alkalmas működési küszöb választására.

2. feladat. Az elméleti küszöb $2/(2 + 10) = 2/12 \approx 0{,}167$, ugyanaz, mert a két költség aránya nem változott: mindkét tábla az 1:5 arányt használja. A valid görbe minimuma a véletlen miatt kissé elmozdulhat, de a döntési határ elvileg változatlan. A teljes költség viszont kétszeresére

skálázódik, mert minden hibás döntés ára duplázódik; a tesztköltség várhatóan körülbelül $2 \cdot EC$, ahol EC az eredeti költségtáblával mért tesztköltség.

3. feladat. Lehetséges okok:

- A `threshold_valid` halmaz túl kicsi, ezért a valid költség alacsony varianciájú becslés, de a teszt időszak más eloszlást mutat; a valid és teszt késési arányát a split manifest alapján összevethető.
- Időbeli drift: a teszt újabb rendelések, ahol az ügyfélviselkedés vagy a szállítási feltételek megváltoztak. A `distance_km`, `basket_minor` eloszlásának valid/teszt összehasonlítása ezt jelzi.
- Label-availability hiba: ha a train szűréséből kimaradó határsáv mégis a trainbe került, a teszt címkéi „jövőbeli” információt szivárogtathatnak. A `split.json` purged listája és a `label_available_at` szerinti szűrés ellenőrzése ezt zárja ki.
- A valid küszöb adaptív választás: a valid költség minimuma optimista, mert a zajhoz is illeszkedik. Ez a hatás a valid méret növelésével és a küszöb előre rögzítésével csökkenthető.

4. feladat. A SMOTE a kisebbségi osztály pontjainak szomszédjai alapján szintetikus mintát generál. Ha ezt a teljes trainen, a CV előtt futtatjuk, a valid fold valódi pontjainak szomszédságát is felhasználja, ezért a valid fold nem marad érintetlen; az értékelés optimista lesz. A helyes megoldás:

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = ImbPipeline([
    ("smote", SMOTE(random_state=20261020)),
    ("lr", LogisticRegression(max_iter=1000, random_state=20261020))
])

scores = cross_val_score(pipe, X_train_full, y_train_full,
                         scoring="neg_log_loss", cv=5)
```

Itt a `cross_val_score` minden foldon belül előbb a tanítórészen futtatja a SMOTE-ot, a validációs fold pedig a nyers, eredetileg kiegyensúlyozatlan címkékkel értékel. A `Pipeline` tehát nem a fold előtt, hanem a fold tanító ágában hajtja végre a resamplinget.

23.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba a **teszten optimalizált küszöb**. Tegyük fel, hogy a valid optimum $t = 0{,}18$, a teszt költséggörbéjének minimuma viszont $t = 0{,}24$-nél van, és ott a tesztköltség 0,39 a valid-küszöbös 0,43-mal szemben. Ha a jelentésbe a 0,39-et és a 0,24-et írjuk, az első ránézésre jobb, de a döntési szabály most már a teszt címkéit ismerve alakult ki. A teszt zajához idomuló küszöb a jövőbeli mintán általában rosszabb, mint a validon befagyasztott; a 0,24-es választás tehát optimista önértékelés. A helyes sorrend: validon választunk küszöböt, a tesztet utána egyszer, változatlan szabállyal értékeljük. Ugyanez a logika ismétlődött a 23.3. alfejezet belső/külső CV szétválasztásánál: minden adaptív döntést a teszt előtti rétegben kell lezárni.

Önellenőrzés: a teszt confusion matrix oszlopösszegei egyeznek-e a teszt tényleges osztálygyakoriságával; a költség nem negatív; a Brier-érték 0 és 1 között van; a `split.json` train/test/purged halmazai diszjunktak. Ha bármelyik sérül, a küszöb vagy az értékelés nem megbízható.

23.4.8 Rövid összefoglaló

A modell valószínűségi kimenetéből a döntési küszöböt a döntési költségtábla alakítja működési szabállyá. A validhalmazon kiszámított várható költség görbéje választja a t küszöböt; az elméleti $C_{FP}/(C_{FP}+C_{FN})$ érték csak jól kalibrált modellnél pontos. A kalibrációs görbe és a Brier-pontszám azt ellenőrzi, hogy a prediktált valószínűségek tényleges gyakoriságként értelmezhetők-e. A küszöböt a teszt előtt befagyasztjuk; a teszten optimalizált küszöb már nem független értékelés. Ritka eseménynél az accuracy helyett a költség és a kalibráció vezet, a resampling pedig csak a tanító foldon belül élhet. A következő egység a 24.1. alfejezet PCA-val és nem felügyelt módszerekkel bővíti a predikciós eszköztárat.

24 Nem felügyelt tanulás és reprezentáció

24.1 PCA: skála, vetítés és megőrzött variancia

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, NumPy

Előfeltétel: 23.4. alfejezet · Célkészség: PCA: skála, vetítés és megőrzött variancia · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

24.1.1 Miért kell a PCA-t skálázással együtt tárgyalni?

A Northstar prediktív projektben a 22.4. alfejezet modellválasztási jegyzőkönyv logisztikus regressziót is vizsgált a késő szállítás előrejelzésére. A lineáris modell viselkedése akkor értelmezhető jól, ha a numerikus bemenetek összehasonlítható skálán vannak. Mielőtt azonban a skálázást vakon elvégeznénk, szeretnénk látni, hogy a rendelésenkénti numerikus jellemzők – `distance_km`, `basket_minor`, `quantity_sum` – hány tényleges szóródási irányt hordoznak, és melyik változó uralja az egyes irányokat. Ez az alfejezet a 10.4. alfejezet szinguláris felbontás előképét alakítja konkrét tanítótranszformációvá.

Bemenet: egy $X \in \mathbb{R}^{n \times p}$ numerikus mátrix; megfigyelési egység itt a rendelés, n a sorok száma, p a jellemzők száma. A labor tiszta `orders.csv` fájljában $n = 2400$ sor érhető el, a példában a három közvetlenül rendelkezésre álló numerikus oszlopot használjuk. **Kimenet:** $k \leq p$ főkomponens (principal component, PC) iránya, a pontok komponens-koordinátái (score-ok) és a komponensenkénti megmagyarázott varianciahányad. **Ellenőrzés:** a komponens-score oszlopok páronként korrelálatlanok, a varianciahányadok összege p komponens esetén 1, és a csökkentett térből visszaállított adat hibája a kihagyott varianciával egyezik.

Előzetes becslés. A `basket_minor` EUR-cent egységben mért kosárérték varianciája több nagyságrenddel nagyobb, mint a kilométerben mért távolságé vagy a darabszámé. Ezért nyers skálán az első főkomponenst várhatóan a `basket_minor` egyedül uralja. Standardizálás után minden változó egységnyi varianciával indul, így a komponensek a változók közötti korrelációs szerkezetet mutatják, nem a mértékegységből fakadó szóródáskülönbséget.

24.1.2 Fogalmi alap: centering, scaling, SVD

A centering (centrálás) azt jelenti, hogy minden oszlopból kivonjuk az oszlopátlagot. A j-edik oszlop átlaga

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^{n} x_{ij},$$

a centrált mátrix eleme pedig $\tilde{x}_{ij} = x_{ij} - \bar{x}_j$. A centrált adat minden oszlopának átlaga 0.

A scaling (skálázás) legegyszerűbb változata a standardizálás vagy z-score transzformáció. A j-edik oszlop korrigált mintaszórása

$$s_j = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left(x_{ij}-\bar{x}_j\right)^2},$$

a standardizált elem pedig $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$. Ekkor minden oszlop átlaga 0, szórása 1; a mértékegység eltűnik, a változók összehasonlíthatóvá válnak. A `StandardScaler` a scikit-learnben pontosan ezt a két lépést végzi.

A 10.4. alfejezetben megismert SVD szerint a centrált mátrix felbontható

$$\tilde{X} = U\Sigma V^{\top},$$

ahol $U \in \mathbb{R}^{n\times r}$ bal szingulárvektorok mátrixa, $\Sigma \in \mathbb{R}^{r\times r}$ diagonális, a főátlón a $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r > 0$ szinguláris értékekkel, $V \in \mathbb{R}^{p\times r}$ pedig a jobb szingulárvektorok mátrixa; r az adatmátrix rangja. A V oszlopai a pontfelhő fő irányai, vagyis a főkomponens-irányok. A komponens-koordináták mátrixa

$$T = \tilde{X}V = U\Sigma,$$

amelynek oszlopai a score-ok. A j-edik irányra vetített adat varianciája

$$\lambda_j = \frac{\sigma_j^2}{n-1}.$$

Ez a mennyiség az úgynevezett explained variance (megmagyarázott variancia): azt mutatja, mennyi szóródást visz a j-edik komponens. A teljes variancia a centrált adat oszlopvarianciáinak összege, és egyenlő $\sum_j \lambda_j$-vel, ha minden pozitív szinguláris értéket megtartunk. A explained variance ratio (megmagyarázott varianciahányad) a

$$\frac{\lambda_j}{\sum_{k=1}^{p}\lambda_k}$$

érték; azt mondja meg, a teljes szóródás hányad részét fedi le az adott komponens. A scikit-learn `PCA` objektuma a `components_` attribútumban a $V^{\top}$ mátrixot tárolja, tehát annak j-edik sora a j-edik PC irány terhelése (loading); az `explained_variance_` a λ_j-ket, az `explained_variance_ratio_` a fenti hányadokat adja vissza.

Kézi kontroll. Legyen a nyers adat három pontja

$$X = \begin{bmatrix} 1 & 1 \\ 3 & 2 \\ 2 & 3 \end{bmatrix}.$$

Az oszlopátlag $(2, 2)$, ezért

$$\tilde{X} = \begin{bmatrix} -1 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

A mintakovariancia-mátrix

$$S = \frac{1}{n-1}\tilde{X}^\top \tilde{X} = \frac{1}{2}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0{,}5 \\ 0{,}5 & 1 \end{bmatrix}.$$

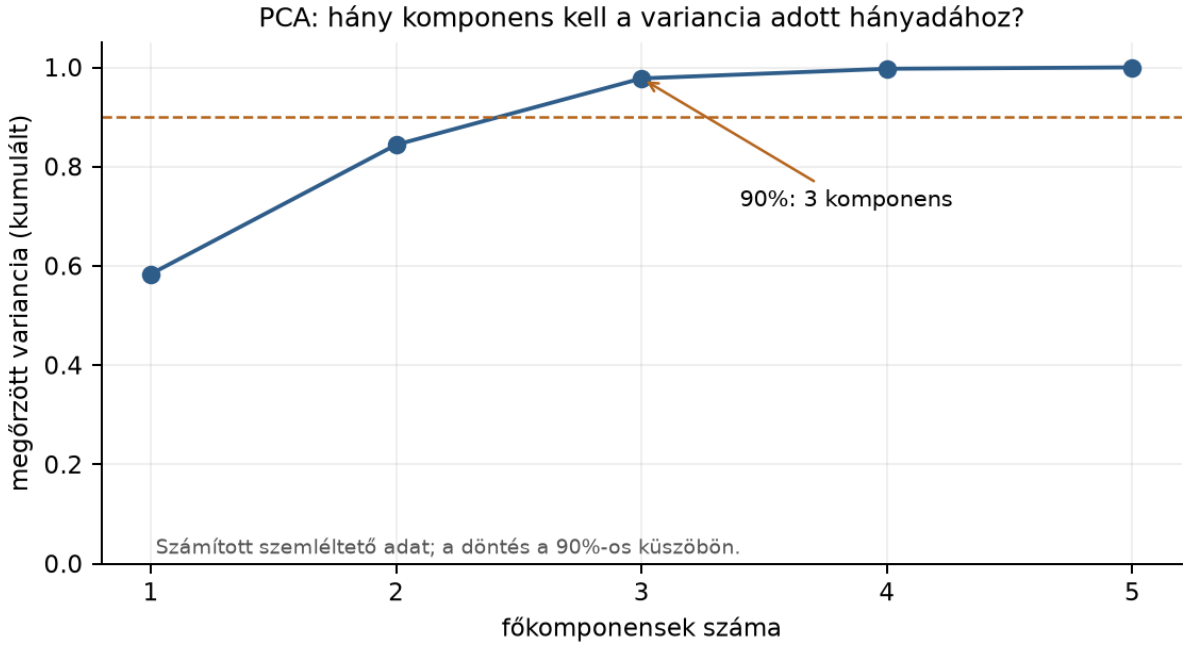
A sajátértékek $\lambda_1 = 1{,}5$ és $\lambda_2 = 0{,}5$; a hozzájuk tartozó normalizált sajátvektorok

$$v_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0{,}707 \\ 0{,}707 \end{bmatrix}, \qquad v_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix} \approx \begin{bmatrix} 0{,}707 \\ -0{,}707 \end{bmatrix}.$$

A varianciahányadok: $1{,}5/(1{,}5+0{,}5) = 0{,}75$ és $0{,}5/2 = 0{,}25$. A score-ok: az első pont első score-ja $(-1)(0{,}707) + (-1)(0{,}707) = -1{,}414$, a második ponté $0{,}707$, a harmadiké $0{,}707$; a második komponensen rendre 0, $0{,}707$ és $-0{,}707$. Ez a kis példa a későbbi kódos ellenőrzés aranyalapja.

A levezetésből látszik a kulcsgondolat: a PCA a centrált adat kovarianciaszerkezetének sajátirányait keresi. Ha az elemek mértékegysége nagyon eltérő, a kovarianciamátrixot a nagy abszolút értékű változó uralja. A standardizálás után viszont a kovarianciamátrix éppen a korrelációs mátrixszá válik, mert $s_j = 1$ minden j-re.

24.1.2.1 Fogalmi ábra: megőrzött variancia és a komponensszám



24.1. ábra: PCA: a kumulált megőrzött variancia a komponensek számának függvényében; a 90%-os küszöböt három komponens éri el a szemléltető adaton.

Az ábra a komponensszám-választás döntési logikáját mutatja: a kumulált variancia monoton nő, de a növekmény egyre csökken. A 90%-os vonal nem törvény, hanem előre rögzített döntési küszöb; a helyes érték a feladattól függ. A konklúzió ábra nélkül is ugyanaz: annyi komponenst érdemes tartani, amennyi a deklarált varianciahányadot eléri, és ezt a döntést a tanítóadaton kell meghozni. Az ábra számai determinisztikus szemléltető adatból számított értékek, nem mérési

eredmények. A teljes generátor: `lab/figures/make_concept_figures.py pca_variance()`; a reprodukálható mag:

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(20261029)
mixing = rng.normal(size=(5, 5))
data = rng.multivariate_normal(np.zeros(5), mixing @ mixing.T, size=3000)
evals = np.linalg.eigvalsh(np.cov(data, rowvar=False))[::-1]
ratio = np.cumsum(evals) / evals.sum()
fig, ax = plt.subplots(figsize=(6.4, 3.6))
ax.plot(range(1, 6), ratio, marker="o")
ax.axhline(0.9, color="#b5651d", linestyle="--")
ax.set_xlabel("főkomponensek száma")
ax.set_ylabel("megőrzött variancia (kumulált)")
ax.set_xticks(range(1, 6)); ax.grid(alpha=0.2); plt.show()
```

24.1.3 Teljes minimális munkapélda

Az alábbi kód két PCA-t futtat ugyanazon a mesterségesen generált, Northstar-stílusú adaton: először nyers skálán, majd standardizálás után. A generálás determinisztikus, így az összehasonlítás megismételhető. A kimenet itt szemléltető, mert a szöveget író modell nem futtatta a kódot; a várt tartományokat a generáló képletből vezetjük le.

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Determinisztikus, Northstar-stílusú rendelésadatok.
# Mértékegységek: distance_km [km], basket_minor [EUR-cent],
# quantity_sum [darab]; a kosárérték erősen összefügg a távolsággal.
rng = np.random.default_rng(42)
n = 300
distance_km = rng.normal(40, 15, n)
basket_minor = 2500 + 40 * distance_km + rng.normal(0, 100, n)
quantity_sum = rng.poisson(4, n) + 1
X = pd.DataFrame({
    "distance_km": distance_km,
    "basket_minor": basket_minor,
    "quantity_sum": quantity_sum.astype(float),
})

# Nyers PCA. A sklearn itt is centrál, de nem standardizál.
pca_raw = PCA(n_components=3)
T_raw = pca_raw.fit_transform(X)
print("Nyers loadings (soronként egy PC irány):")
print(pca_raw.components_)
print("Nyers arány:", pca_raw.explained_variance_ratio_)

# Standardizált PCA.
scaler = StandardScaler()
Z = scaler.fit_transform(X)
pca_std = PCA(n_components=3)
T_std = pca_std.fit_transform(Z)
print("Standardizált loadings:")
print(pca_std.components_)
print("Standardizált arány:", pca_std.explained_variance_ratio_)

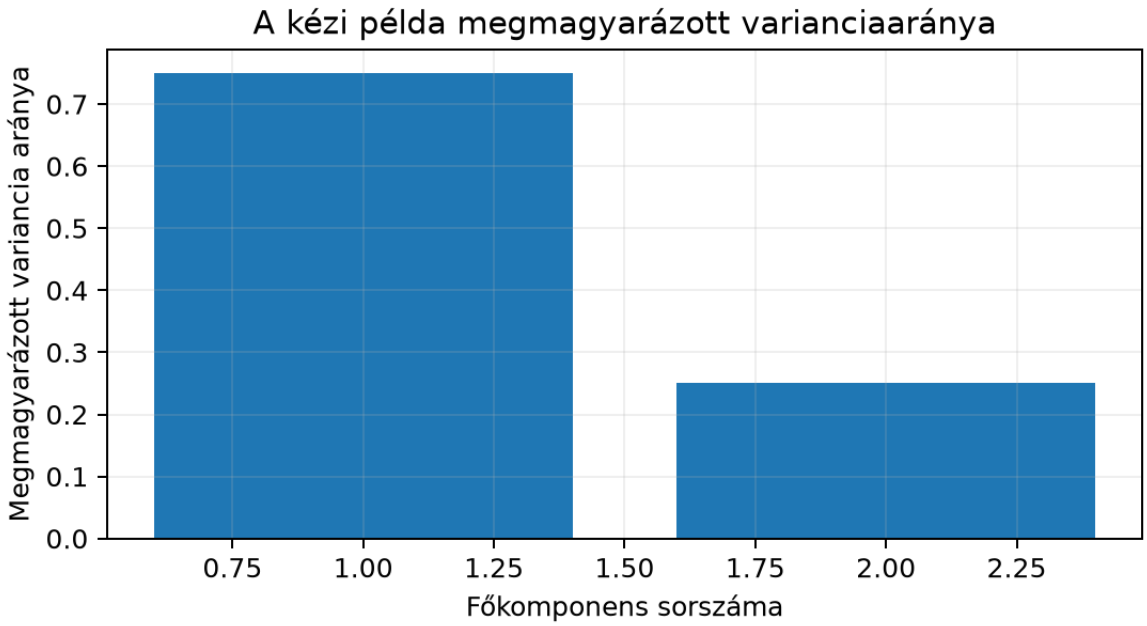
# Invariáns ellenőrzés: a score-ok off-diagonális kovarianciája nulla.
```

```
score_cov = np.cov(T_std.T)
print("Score kovariancia off-diag maximuma:",
      np.max(np.abs(score_cov - np.diag(np.diag(score_cov)))))

# Rekonstrukció egy komponenssel a standardizált térben.
pca_1 = PCA(n_components=1)
Z_score_1 = pca_1.fit_transform(Z)
Z_reconstructed = pca_1.inverse_transform(Z_score_1)
reconstruction_var = np.var(Z - Z_reconstructed, axis=0).sum()
print("Rekonstrukciós hiba teljes varianciája 1 komponens után:",
      reconstruction_var)
print("Kihagyott variancia a modell szerint:",
      pca_1.explained_variance_.sum() if False else
      pca_std.explained_variance_[1:].sum())
```

A kódban a `PCA` automatikusan elvégzi a centrálást az `X`-en, tehát kézzel nem kell az oszlopátlagot levonni. A várható eredmény: a nyers skálán a `basket_minor` szórása 608 cent körüli, a `distance_km` szórása 15 km, a `quantity_sum` szórása 2 darab körüli, így a nyers első komponens terhelése gyakorlatilag a `basket_minor` oszlopra koncentrálódik, és az első varianciahányad 0,99 fölött van. Standardizálás után a korrelációs mátrix első két változója közötti korreláció elméletileg 0,98 körüli, ezért az első PC terhelése a `distance_km` és a `basket_minor` irányában egyaránt 0,7 körüli, az első varianciahányad $\approx (1+\rho)/3 \approx 0{,}66$, a második $\approx 0{,}005$, a harmadik $\approx 0{,}33$. Az utolsó ellenőrzés arra mutat rá, hogy a standardizált térbeli rekonstrukciós hiba varianciája éppen a kihagyott komponensek magyarázott varianciája: 3 változó esetén a teljes variancia 3, ebből 1 komponenssel λ_1-et tartjuk meg, így a hiba $\lambda_2 + \lambda_3$.

A laborban ugyanez a gondolatmenet a tiszta `orders.csv` három numerikus oszlopán futtatható: `distance_km`, `basket_minor`, `quantity_sum`. A `segment`, `region` és `acquisition_channel` kategorikus oszlopok nem kerülnek közvetlenül a PCA-ba; a `priority_flag` bináris, ezért csak külön megfontolással szerepelhet. Ha az `account_age_days` jellemzőt is be akarjuk vonni, azt előbb a `customers.csv` `signup_date` oszlopából kell időhelyesen kiszámítani, ahogyan azt a 7.3. alfejezet előzményaggregátumai mutatják. A laborfuttatás várt tendenciája ugyanaz: a nyers PCA-t a kosárérték uralja, a standardizált PCA a korrelációs szerkezetet emeli ki.



24.2. ábra: A kézi példa megmagyarázott varianciaaránya - A hárompontos kézi példa levezetett értékei, nem mérés.

24.1.4 Train-only fit és a predikciós szerződés

A PCA felügyelet nélküli, de a belőle épített transzformáció betanított paramétereket tartalmaz: az oszlopátlagokat, a szórásokat és a komponensirányokat. Ha ezeket a teljes adaton, a tesztet is beleértve tanuljuk meg, akkor a tesztből származó eloszlásinformáció szivárog a pipeline-ba. Ez ugyanaz a hibaosztály, amelyet a 21.4. alfejezet predikciós szerződés és a 23.4. alfejezet foldon belüli resampling kizár. A train-only fit (csak tanítón illesztés) lépései a Northstar időbeli splitjénél a következők:

```
cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
# Feltéve, hogy a s084 label-availability szűrés már megtörtént:
train_mask = (df["order_ts"] < cutoff) & (df["label_available_at"] < cutoff)
test_mask = df["order_ts"] >= cutoff
X_train = df.loc[train_mask, numeric_cols]
X_test = df.loc[test_mask, numeric_cols]

scaler = StandardScaler().fit(X_train)        # csak tanítón tanul
pca = PCA(n_components=2).fit(scaler.transform(X_train))

X_train_score = pca.transform(scaler.transform(X_train))
X_test_score = pca.transform(scaler.transform(X_test))   # tesztre csak transform
```

A `fit` hívás a tanító statisztikákat rögzíti; a tesztre már csak a `transform` vonatkozik. Így a teszt score-jai is a tanító átlagával és szórásával számítódnak, pontosan ahogyan a betanított modell élesben is viselkedne. Ugyanezt a szerződést érdemes egy `Pipeline`-ba zárni, hogy a keresztvalidáció foldjaiban se férhessen a teszt a tanításhoz.

24.1.4.1 Pipeline és foldonkénti PCA

A skálázás és a PCA egyetlen `Pipeline`-ban foldonként illeszkedik:

```
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=2)),
]).fit(X_train)

assert pipe.named_steps["scale"].mean_.shape == (len(numeric_cols),)
X_test_score = pipe.transform(X_test)   # a teszt csak transformálódik
```

A `Pipeline` `.fit()` hívása a scaler átlagait/szórásait és a PCA komponensirányait kizárólag a tanító adatból tanulja. A teszt permutálása nem változtathatja meg a tanult főirányokat; csak a score-ok sorrendjét cseréli fel:

```
perm = np.random.default_rng(0).permutation(len(X_test))
score_perm = pipe.transform(X_test.iloc[perm])
assert np.allclose(score_perm, X_test_score[perm])
```

A **megőrzött variancia** is a tanítóeloszlásra vonatkozik: a `pipe.named_steps["pca"].explained_variance_r` a tanító kovarianciamátrixából származik. A teszten számolt reconstructív hiba vagy variancia ettől eltérhet, ha a teszt eloszlása más; ezért a jelentésben külön kell közölni a tanító varianciahányadot és a teszt rekonstrukciós hibáját. Egy eltolt teszteloszlásnál a tanító komponensek nem feltétlenül a teszt legnagyobb szóródási irányai.

24.1.5 Feladatok

1. **Értelmezés, kézi számítás.** A szövegbeli hárompontos példán ellenőrizd, hogy a centrált mátrix oszlopainak empirikus varianciája 1 és 1, a két főkomponens score-oszlopainak kovarianciája 0, és a score-ok varianciája rendre $\lambda_1 = 1{,}5$ és $\lambda_2 = 0{,}5$. Számold ki kézzel a score-oszlopok értékeit a megadott v_1 és v_2 irányokkal.

2. **Megvalósítás.** Futtasd le a fenti minimális példát. Ellenőrizd, hogy a nyers első komponensben a `basket_minor` terhelés abszolút értéke legalább 0,99, a standardizált első komponensben pedig a `distance_km` és a `basket_minor` terhelése egyaránt 0,65 és 0,75 közé esik. Ellenőrizd azt is, hogy az `explained_variance_ratio_.sum()` értéke 1 legyen 10^{-12} tűréssel.

3. **Hibadiagnózis.** Egy kolléga az időbeli split előtt a teljes `orders.csv`-re illeszti a `StandardScaler`-t, mondván: „a PCA felügyelet nélküli, itt nincs szivárgás.” Miért téves ez az érvelés akkor, ha a skálázott jellemzőket később logisztikus regresszióba adjuk? Milyen irányba torzíthat a jövőbeli eloszlásból származó átlag és szórás? Add meg a javítást `Pipeline`-nal.

4. **Megváltozott feltétel.** Hagyd el a `basket_minor` oszlopot a generált adatból, és illeszd újra a PCA-t a maradék két oszlopra standardizálás után. Milyen komponensirányt és varianciahányadot várs? Röviden indokold. Mi változik, ha a `quantity_sum` helyett a bináris `priority_flag` oszlopot teszed be változatlan problémakezeléssel?

24.1.6 Megoldások és indoklás

1. feladat megoldása. A centrált mátrix oszlopai $(-1, 1, 0)$ és $(-1, 0, 1)$; mindkettő átlaga 0, négyzetösszege 2, így a korrigált variancia $2/(3-1) = 1$. A score-ok:

$T = \tilde{X}V$, tehát az első komponensen a fent kiszámított $-1{,}414$, $0{,}707$, $0{,}707$; a másodikon 0, $0{,}707$, $-0{,}707$. Az első score-oszlop átlaga 0, varianciája $\left[(-1{,}414)^2 + 0{,}707^2 + 0{,}707^2\right]/2 = (2 + 0{,}5 + 0{,}5)/2 = 1{,}5$; a másodiké $(0 + 0{,}5 + 0{,}5)/2 = 0{,}5$. A két oszlop kovarianciája $\left[(-1{,}414)(0) + (0{,}707)(0{,}707) + (0{,}707)(-0{,}707)\right]/2 = (0 + 0{,}5 - 0{,}5)/2 = 0$. Az oszlopok tehát korrelálatlanok, a varianciák a sajátértékek.

2. feladat megoldása. A nyers PCA-nál a generálásból $\mathrm{Var}(\text{basket}) = 40^2 \cdot 15^2 + 100^2 = 370\,000$ cent^2, $\mathrm{Var}(\text{distance}) = 225$ km^2, $\mathrm{Var}(\text{quantity}) \approx 4$ darab^2. A kovarianciamátrix diagonális dominanciája miatt az első sajátvektor a `basket_minor` tengelyhez simul, így a terhelés abszolút értéke $0{,}999$ fölött várható. Standardizálás után a korreláció $40 \cdot 15/\sqrt{225 \cdot 370\,000} \approx 600/608 \approx 0{,}986$; a korrelációs mátrix első 2×2-es blokkjának domináns sajátvektora közel $(0{,}707, 0{,}707)$, ezért mindkét terhelés $0{,}65$–$0{,}75$ közé esik. Az arányok összege a definícióból adódóan 1; numerikusan 10^{-12} tűrésen belül kell maradnia. A tényleges futtatás során a mintavételezés miatt a terhelések $\pm 0{,}05$-ön belüli eltérése elfogadható.

3. feladat megoldása. A standardizálás paramétereit a tanuló pipeline részének kell tekinteni, mert az átlag és a szórás a modell bemenetét alakítja. Ha a teszt jövőbeli eloszlásából is tanulunk, a teszt adatai már a tanulási folyamatban részt vesznek; ez ugyanaz a leakeage, mint a 21.4. alfejezetben tárgyalt szerződésszegés. A torzítás iránya jellemzően optimista kiértékelés: a modell olyan jövőbeli eloszlási információkhoz fér hozzá, amelyek éles használatkor nem állnának rendelkezésre. A javítás:

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2)),
])
pipeline.fit(X_train)          # mindkét lépés paramétere csak tanítón tanul
X_test_score = pipeline.transform(X_test)
```

4. feladat megoldása. A `basket_minor` elhagyása után a standardizált térben a két megmaradt változó korrelációja gyenge (a generálásban a `quantity_sum` független a `distance_km`-től), ezért a korrelációs mátrix körülbelül egységmátrix. A két sajátérték közel 1 és 1, a komponensirányok bizonytalanok, a varianciahányadok közel $0{,}5$–$0{,}5$. A PCA itt nem ad értelmes dimenziócsökkentést, mert nincs domináns korrelációs tengely. A bináris `priority_flag` beillesztése hasonló óvatosságot igényel: a z-score alapú PCA a 0/1 változót is standardizálja, de az eredményül kapott komponens nehezen értelmezhető folytonos látens tengelyként, és a bináris változó marginális varianciája a ritka eseményeknél torzíthatja a súlyozást. Jobb ilyenkor előbb megfontolni, hogy a bináris változót külön kezeljük, vagy kevert típusú reprezentációt válasszunk.

24.1.7 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés úgy szól: „a PCA kiválasztja a legfontosabb jellemzőket a predikcióhoz.” Ez téves. A PCA célváltozó nélkül, kizárólag a bemeneti variancia alapján rangsorol; egy alacsony varianciájú változó lehet determinisztikus, erős prediktor. Ellenpélda: legyen $x_1 \sim \mathcal{N}(0, 10)$ nagy szórású zaj, x_2 pedig olyan bináris jel, amely 2% valószínűséggel 1, egyébként 0; legyen a cél $y = x_2$. A PCA első komponense x_1-et tartja meg, pedig x_2 tökéletesen prediktálja y-t. A variancia tehát nem helyettesíti a felügyelt feature-szelekciót. Hasonlóképpen a komponensek nem feltétlenül oksági tényezők: a standardizált első PC például a távolság és a kosárérték lineáris kombinációja, nem egy beavatkozható fizikai mennyiség. Az oksági értelmezéshez a 18.1. alfejezet kísérleti kerete kell.

A PCA komponenseinek előjele tetszőleges, ezért futások között azonos főkomponens ellentétes előjellel is megjelenhet. A komponensek korrelálatlansága a tanítási adatokra és a szokásos centrált PCA-definícióra vonatkozik.

24.1.8 Rövid összefoglaló

A PCA a centrált adatmátrix SVD-jéből nyeri a fő irányokat; a szinguláris értékekből számolt $\sigma_j^2/(n-1)$ mennyiség az adott komponens megmagyarázott varianciája, hányada pedig a teljes szóródásból való részesedés. Különböző mértékegységű oszlopoknál a nyers PCA-t a legnagyobb varianciájú változó uralja, ezért standardizálni kell – de a skálázó és a PCA paramétereit kizárólag a tanítórészen szabad illeszteni. Az inverz transzformáció rekonstrukciós hibája a kihagyott komponensek varianciája. A PCA varianciaalapú, nem célfüggő feature-szelekció. Következő alkalmazás: a 24.2. alfejezet klaszterezés, ahol ugyanezt a komponens-teret használjuk a távolságalapú stabilitás vizsgálatához.

24.2 Klaszterezés és stabilitás

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, NumPy

Előfeltétel: 24.1. alfejezet · Célkészség: Klaszterezés és stabilitás · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

24.2.1 Cél és konkrét helyzet

A Northstar-rendeléseket nyolc, vegyes mértékegységű jellemzőből szeretnénk csoportosítani, hogy a kosármintázatokból értelmezhető szegmenseket kapjunk. Mielőtt a valódi, 8 dimenziós téren hoznánk döntést, két ismert 2 dimenziós szintetikus geometrián kalibráljuk az eszközt: `make_blobs` három közel gömb alakú csoporttal, és `make_moons` két nem konvex félholddal. A megfigyelési egység itt egy mintasor; a szintetikus generáló címke csak utólagos ellenőrzésre van, a klaszterező algoritmus bemenete kizárólag a `X` jellemzőmátrix.

A kimenet egy döntési jegyzőkönyv: a KMeans és DBSCAN paraméterei, a standardizálás utáni silhouette-érték, a stabilitási összehasonlítás (adjusted Rand index, ARI) és egy klaszterprofil. **Előzetes becslés a futtatás előtt:** a holdakon a KMeans `k=2` valószínűleg magas silhouette-értéket is elérhet, mert kompakt darabokat vág, de a generáló holdakhoz képest az ARI csak 0,2–0,4 körül lesz, mert a holdak nem lineárisan szeparálhatók. A DBSCAN megfelelő `eps` és `min_samples` mellett gyakran jó eredményt adhat, de ezt a zajszint és a paraméterezés befolyásolja; közel tökéletes eredmény nem garantált. Ha ez a mintázat beigazolódik, a silhouette önmagában nem elég a geometriai alakzat ellenőrzésére.

24.2.2 Távolság, standardizálás és a KMeans alakfeltevése

A klaszterezés (clustering) célfüggvény nélküli partíciókeresés. A KMeans a centroid (klaszterközéppont) és a pontok euklideszi távolságának négyzetét minimalizálja:

$$\mathrm{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2,$$

ahol $X \in \mathbb{R}^{n \times p}$ a mátrix, x_i egy p hosszú sorvektor, C_k a k-adik klaszterbe sorolt pontok halmaza, $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ a centroid, és $\|v\|_2 = \sqrt{\sum_j v_j^2}$ az euklideszi norma. Az összeget gyakran within-cluster sum of squares (WCSS) vagy inertia néven hívják; mértékegysége a jellemzők négyzetösszegének egysége, ezért standardizálás után dimenziótlan.

Az algoritmus két lépést váltogat: minden pontot a legközelebbi centroidhoz rendel, majd a centroidokat újraszámolja. Mindkét lépés nem növeli a WCSS-t, de a módszer lokális optimumba ragadhat. A `n_init=10` azt jelenti, hogy 10 különböző inicializálásból indulunk, és a legjobb célfüggvényű partíció marad; a `random_state` a seedet rögzíti.

A KMeans döntési határai a centroidokat összekötő szakaszok felező merőlegesei, ezért a kapott klaszterek közel konvexek, nagyjából gömb alakúak, és hasonló méretre törekszenek. A távolságalapú célfüggvény erősen skálafüggő: ha az egyik jellemzőt kilométerben, a másikat centben mérjük, a második dominál. Ezért a jellemzőnkénti z-score standardizálás (standardization) kötelező előfeldolgozás:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j},$$

ahol $\bar{x}_j$ a j-edik oszlop átlaga, s_j a szórása. A standardizált oszlopok átlaga közel 0, varianciája 1. A laborban `StandardScaler`-t használunk; ha a klaszterezést később új adatra is szeretnénk alkalmazni, a scalert a vizsgált mintán illesztjük, és ugyanazt a transformot fagyasztjuk le a jövőbeli sorokra.

24.2.3 DBSCAN: sűrűség, eps és zaj

A DBSCAN (Density-Based Spatial Clustering of Applications with Noise) nem vár gömb alakú csoportokat, hanem sűrűséggel összekapcsolható magpontokból növeszti a klasztereket. Két paramétere: `eps`, a szomszédság sugara, és `min_samples`, a magponthoz szükséges minimális szomszédszám. A scikit-learn implementációban maga a vizsgált pont is beleszámít, így `min_samples=5` legalább 4 másik szomszédot jelent az `eps` sugarú környezetben. A magpont (core point) feltétele:

$$|N_\epsilon(x_i)| \geq m, \qquad N_\epsilon(x_i) = \{x_j : d(x_i, x_j) \leq \epsilon\}.$$

A klaszter a magpontokból sűrűségen keresztül elérhető maximális halmaz. A határpont olyan nem magpont, amelyik magpont szomszédja; a zaj (noise) egyik sem, és a scikit-learn a `-1` címkét adja neki. A DBSCAN-nak nem kell előre megadni a klaszterszámot, de egyetlen `eps` nem kezeli az erősen változó sűrűségű adatot.

Kis kézi kontroll 1 dimenzión: legyenek a pontok `0, 0.4, 1.2, 5, 5.3`, `eps=0.5`, `min_samples=2`. A `0.4`-nek a `0` szomszédja, tehát $\{0, 0.4\}$ egy klaszter; az `5` és `5.3` alkotja a másikat. Az `1.2`-nek nincs szomszédja az `eps` sugáron belül (`0.8` távolságra van a `0.4`-től), ezért zaj, `-1` címkével.

24.2.4 Sziluett és stabilitás: két különböző kérdés

A silhouette-együttható (silhouette coefficient) azt méri, mennyire konzisztens egy pont a saját klaszterével a legközelebbi másik klaszterhez képest:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))},$$

ahol $a(i)$ az i pont átlagos távolsága a saját klaszterének többi pontjától, $b(i)$ pedig a legközelebbi másik klaszter pontjaitól mért átlagos távolság. Az érték $[-1, 1]$-be esik: 1 a sűrű, jól elkülönült pont, 0 az átfedő határ, negatív a rossz besorolás. A `silhouette_score` a pontonkénti értékek átlaga.

Kézi példa: $A = \{0, 1\}$, $B = \{4, 5\}$. A 0 ponthoz $a(0) = 1$, $b(0) = (4 + 5)/2 = 4{,}5$, tehát $s(0) = (4{,}5 - 1)/4{,}5 \approx 0{,}778$. A 1 ponthoz $b(1) = (3 + 4)/2 = 3{,}5$, ezért $s(1) \approx 0{,}714$. A 4 és 5 szimmetrikusak, így a teljes átlag 0,746.

A stabilitás más kérdés: ha egy kicsit megváltoztatjuk a seedet vagy a paramétert, ugyanazt a partíciót kapjuk-e? A KMeans esetén a különböző `random_state` értékekkel kapott címkéket az adjusted Rand indexszel (ARI) hasonlítjuk össze. Az ARI a két partíció közötti pontpár-egyezést méri, és levonja a véletlen várható egyezést; 1 a tökéletes egyezés, 0 a véletlen szint, negatív érték a véletlennél rosszabb. A DBSCAN adott adatokkal, metrikával, paraméterekkel és bemeneti sorrenddel determinisztikusan fut, bár a határpontok csoporthoz rendelése a bemeneti sorrendre érzékeny lehet. Ezért itt az `eps` paraméterrács szomszédos értékein kapott partíciókat hasonlítjuk össze: ha egy kis `eps` változtatás drámai ARI-csökkenést okoz, a megoldás érzékeny. A stabilitási mérés nem statisztikai intervallum; ugyanúgy csak az eljárás viselkedését jellemzi, mint a foldok közötti szórás a 23.2. alfejezetbeli CV-nél.

Címkefelcserélés. A klasztercímkék önkényesek: ugyanaz a partíció két futásban `0,0,1,1,2,2` és `2,2,0,0,1,1` alakban is megjelenhet. A nyers címkeegyezés ilyenkor 0, pedig a partíció azonos. Ezért a stabilitást soha nem a címkék egyezési arányával, hanem ARI-val vagy normalizált kölcsönös információval (NMI) mérjük:

```
import numpy as np
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

a = [0, 0, 1, 1, 2, 2]
b = [2, 2, 0, 0, 1, 1]
print("nyers egyezés:", np.mean(np.array(a) == np.array(b)))  # 0.0
print("ARI:", adjusted_rand_score(a, b))                      # 1.0
print("NMI:", normalized_mutual_info_score(a, b))             # 1.0
```

Az ARI és az NMI a partíció pontpár-szerkezetét hasonlítja, ezért a címkék permutációjára invariáns. A centroidok sorrendje szintén önkényes lehet; a centroidokat a klaszterek tartalmi profilja alapján kell párosítani, nem a címke számértéke alapján. A silhouette értékét futásonként külön kell kiszámítani; két futás silhouette-jét nem a címkék egyezésével, hanem az ARI/NMI mellett a profilok stabilitásával érdemes összevetni.

24.2.5 Klaszterek jelentése: cluster semantics

A klasztercímke önmagában nem jelentés. A kapott klaszterekhez mindig numerikus profilt kell számolni: a standardizált térben a centroidot vagy a jellemzőnkénti mediánt, majd ezt visszafejteni az eredeti egységekbe. A szintetikus holdaknál a generáló címke csak laboratóriumi igazság; a valódi Northstar-klaszterezésnél nincs ilyen referencia, ott a domain-szótár ad értelmet,

például „közeli, sürgős kosár” vagy „távoli, nagy értékű rendelés”. Ha a klaszterek nem kapnak tartalmi értelmezést, a partíció nem bevethető.

24.2.6 Teljes minimális munkapélda

A szkriptet `sch24/s094_cluster_lab.py` néven mentsük, és a projektgyökérből futtassuk CPU-s tanulási környezetben, a `scikit-learn` projektfüggőség telepítése után:

```
# macOS/Linux terminál és Windows PowerShell esetén is
python sch24/s094_cluster_lab.py

import numpy as np
from sklearn.datasets import make_blobs, make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, adjusted_rand_score

DATA_SEED = 20261020

blobs, y_blobs = make_blobs(
    n_samples=450, centers=3, cluster_std=[0.7, 0.9, 1.1],
    random_state=DATA_SEED
)
moons, y_moons = make_moons(
    n_samples=450, noise=0.08, random_state=DATA_SEED
)

Xb = StandardScaler().fit_transform(blobs)
Xm = StandardScaler().fit_transform(moons)

km_blobs = KMeans(n_clusters=3, n_init=10, random_state=7).fit(Xb)
km_moons = KMeans(n_clusters=2, n_init=10, random_state=7).fit(Xm)
db_moons = DBSCAN(eps=0.2, min_samples=5).fit(Xm)

def safe_silhouette(X, labels):
    labels = np.asarray(labels)
    n_clusters = len(set(labels) - {-1})
    if n_clusters < 2:
        return np.nan
    return float(silhouette_score(X, labels))

print("Xb oszlopátlagai:", Xb.mean(axis=0))
print("Xb oszlopszórásai:", Xb.std(axis=0))
print("Blobs K=3 silhouette:",
      round(safe_silhouette(Xb, km_blobs.labels_), 3))
print("Moons K=2 silhouette:",
      round(safe_silhouette(Xm, km_moons.labels_), 3))
print("Moons DBSCAN silhouette:",
      round(safe_silhouette(Xm, db_moons.labels_), 3))
print("Moons DBSCAN noise arány:",
      round((db_moons.labels_ == -1).mean(), 3))
print("Moons KMeans ARI a generáló holdakkal:",
      round(adjusted_rand_score(y_moons, km_moons.labels_), 3))
print("Moons DBSCAN ARI a generáló holdakkal:",
      round(adjusted_rand_score(y_moons, db_moons.labels_), 3))

# Kézi inercia-kontroll: a WCSS újraszámolása a centroidokból.
centroids = km_blobs.cluster_centers_[km_blobs.labels_]
manual_inertia = float(((Xb - centroids) ** 2).sum())
print("Manuális WCSS:", round(manual_inertia, 4),
      "| Modell inertia_:", round(km_blobs.inertia_, 4))
```

```
# Stabilitás: KMeans seed-lista a holdakon.
seeds = [1, 2, 3, 4, 5]
first = None
sil_vals, ari_vals = [], []
for seed in seeds:
    labels = KMeans(n_clusters=2, n_init=10,
                    random_state=seed).fit_predict(Xm)
    sil_vals.append(safe_silhouette(Xm, labels))
    if first is None:
        first = labels
    else:
        ari_vals.append(adjusted_rand_score(first, labels))
print("Moons K=2 silhouette seed-lista:",
      [round(x, 3) for x in sil_vals])
print("Moons K=2 ARI az első seedhez:",
      [round(x, 3) for x in ari_vals])

# Stabilitás: DBSCAN eps-rács a holdakon.
eps_grid = [0.14, 0.17, 0.2, 0.24, 0.28]
prev = None
for eps in eps_grid:
    labels = DBSCAN(eps=eps, min_samples=5).fit_predict(Xm)
    n_clusters = len(set(labels) - {-1})
    noise_share = float((labels == -1).mean())
    ari_prev = "-" if prev is None else round(
        adjusted_rand_score(prev, labels), 3)
    print(f"eps={eps:.2f} n_clusters={n_clusters} "
          f"noise={noise_share:.3f} ari_prev={ari_prev}")
    prev = labels
```

A kimenet itt **szemléltető, elméletileg várt tartomány**, nem ezen a gépen futott mérés. A blobs három klaszterénél a silhouette általában 0,60–0,75 közé esik; a holdakon a KMeans silhouette 0,45–0,55 is lehet, de az ARI a generáló holdakhoz 0,2–0,4 körül marad, míg a DBSCAN `eps=0.2`-vel az ARI jellemzően 0,9 felett van, a noise arány 0,05 alatt. Ha a te futtatásod ettől lényegesen eltér, a következő szakasz ellenőrzéseit futtasd le, mielőtt a paramétereket módosítanád.

24.2.7 Eredmény értelmezése és ellenőrzés

A standardizálás invariantjai: `Xb.mean(axis=0)` abszolút értéke 10^{-14} alatti, `Xb.std(axis=0)` 1, mert a `StandardScaler` oszloponkénti z-score-t ad. A KMeans illesztése után `km_blobs.labels_` shape-je `(450,)`, dtype egész, a `cluster_centers_` shape-je `(3, 2)`. A manuális és a modell-inercia azonos, numerikus tolerancián belül; ezt a labor 4 tizedesjegyig ellenőrzi.

A legfontosabb értelmezési eredmény: a holdak KMeans- és DBSCAN-partícióját az ARI különbözteti meg. A silhouette a KMeans-nél is lehet viszonylag magas, mert a lineáris vágás után mindkét darab kompakt marad; a generáló holdakkal való ARI azonban elárulja, hogy a KMeans nem a valódi geometriai alakzatot találta. A valós adatban ilyen generáló igazság nincs, ezért ott a stabilitás és a feature-profil lép a helyére.

Ha a DBSCAN `eps=0.14` mellett minden pont zaj, vagy `eps=0.28`-nál a két hold összeolvad, a `safe_silhouette` `nan`-t adhat, és az eps-rács ARI-értékei lecsökkennek. Ez nem hiba a kódban, hanem a paraméterérzékenység diagnózisa: az éles, stabil klaszterszámú tartomány közepéről válasszunk eps-t, nem a széléről.

24.2.8 Feladatok

1. **Kézi sziluett.** Számold ki kézzel a sziluett-együtthatót az $A = \{0, 1\}$, $B = \{4, 5\}$ partíció minden pontjára, majd az átlagot. Ellenőrizd a kódot `silhouette_score`-szal.

2. **Körgyűrű-labor implementálása.** Generálj `make_circles(n_samples=500, noise=0.05, factor=0.4, random_state=123)` adatot, standardizálás után futtasd a KMeans `k=2`-t és egy DBSCAN eps-rácsot `min_samples=5` mellett. Melyik módszer ARI-ja van 0,9 felett a generáló gyűrűcímkével, és miért?

3. **Hibadiagnózis.** A holdakon `DBSCAN(eps=0.01, min_samples=5)` után a közvetlen `silhouette_score(Xm, labels)` `ValueError: Number of labels is 1...` hibát dob. Mi a hiba oka, és hogyan kezeli a `safe_silhouette`? Mutass hasonló esetet `eps=0.8` környékén is, ahol egyetlen óriásklaszter keletkezhet.

4. **Megváltoztatott skála.** A blobs `X = blobs.copy()` második oszlopát szorozd 20-szal, majd standardizálás nélkül futtass `KMeans(n_clusters=3, n_init=10, random_state=7)`-et. Számold ki az ARI-t a standardizált megoldással, és magyarázd meg a távolságtorzulás hatását. Miért lehet a silhouette mégis magas?

5. **Stabilitás átvitele.** A holdakon futtass KMeans `k=3`-mal is seed-listát, és számold ki az ARI-kat az első seedhez. Miért alacsonyabb ez a stabilitás a `k=2`-höz képest? Fogalmazd meg, hogyan használod ezt a `k` választásánál.

24.2.9 Megoldások és indoklás

1. A 0 pont: $a = 1$, $b = 4{,}5$, $s = 0{,}7778$. Az 1 pont: $a = 1$, $b = 3{,}5$, $s = 0{,}7143$. A 4 és 5 szimmetrikusak, így az átlag $(0{,}7778 + 0{,}7143 + 0{,}7143 + 0{,}7778)/4 = 0{,}7460$. Kódellenőrzés: `silhouette_score([[0],[1],[4],[5]], [0,0,1,1])` ugyanezt adja.

2. A megoldás váza:

```
from sklearn.datasets import make_circles
circles, y_c = make_circles(n_samples=500, noise=0.05,
                            factor=0.4, random_state=123)
Xc = StandardScaler().fit_transform(circles)
km_c = KMeans(n_clusters=2, n_init=10,
              random_state=7).fit_predict(Xc)
print("ARI KMeans:", round(adjusted_rand_score(y_c, km_c), 3))
for eps in [0.15, 0.2, 0.25, 0.3, 0.35]:
    lab = DBSCAN(eps=eps, min_samples=5).fit_predict(Xc)
    print(eps, len(set(lab) - {-1}),
          round(safe_silhouette(Xc, lab), 3),
          round(adjusted_rand_score(y_c, lab), 3))
```

A KMeans a gyűrűket átmetszi, ezért az ARI jellemzően 0,2 alatt marad; a DBSCAN a gyűrűk közötti sűrűségrés miatt 0,9 feletti ARI-t ad a megfelelő eps-tartomány közepén. A tartalmi indoklás: a KMeans konvex alakzatot feltételez, a koncentrikus körök nem konvexek.

3. Az `eps=0.01` esetben minden pont noise, a DBSCAN egyetlen `-1` címkét ad; a sklearn `silhouette_score` nem tud egyetlen klaszterrel számolni. Az `eps=0.8` esetén a `-1`-et elhagyva is egyetlen klaszter marad, hasonló hiba. A `safe_silhouette` megszámolja a tényleges klasztercímkéket a `-1` kizárásával, és `nan`-t ad, ha kevesebb mint kettő van; a hurok így nem áll le, hanem rögzíti, hogy ez a paraméter nem értékelhető.

4. A 20-szoros nyújtás után a második koordináta különbségei a négyzetes távolság 400-szorosát adják, ezért a KMeans lényegében a második tengely mentén vág sávokat. Az ARI a standardizált megoldással szemben gyakran 0,4–0,7 közé esik, pedig a generáló három csoport ugyanaz. A silhouette ettől még magas lehet, mert a sávok is kompaktak; ez újabb érv, hogy a silhouette nem igazolja a természetes geometriát.

5. A `k=3` a holdakon arra kényszeríti a KMeans-t, hogy az egyik holdat kettévágja, és a vágás helye a seedtől függ. Az első seedhez viszonyított ARI ezért jellemzően 0,6 alá esik, míg `k=2`-nél közel 1. A klaszterszám-választásnál tehát a silhouette-maximum mellett a stabilitási küszöböt is megköveteljük, például ARI $> 0{,}8$ a seed-lista páronkénti összehasonlításában.

24.2.10 Gyakori tévedés és önellenőrzés

Tévedés: „A KMeans `k=4`-re is talált 4 klasztert, tehát az adatban 4 természetes csoport van.” **Ellenpélda:** generáljunk 600 pontot a $[-1, 1]^2$ négyzeten egyenletesen, és futtassunk rá `KMeans(n_clusters=4, n_init=10, random_state=0)`-t. A kapott 4 klaszter mindig létezik, a silhouette értéke paraméter- és mintafüggő, és a seedek közötti ARI akár magas is lehet stabil, mesterséges térfelosztás esetén. Egyik mutató önmagában sem bizonyít természetes klasztereket. A klaszterező algoritmus outputja nem bizonyít természetes csoportot; a stabilitás és a domain-profil együtt szükséges. Ez párhuzamos a 24.1. alfejezetben látott PCA-tanulsággal: a variancia iránya nem feltétlenül oksági vagy üzleti szegmens. A DBSCAN sűrűségfogalma pedig a 12.2. alfejezetbeli sűrűségfüggvény analógiájára épül, de a paraméterválasztás a mi felelősségünk.

Önellenőrzés: (1) Meg tudom-e mondani, mi a megfigyelési egység, és hogy a resamplingnél a sor vagy az ügyfél a helyes egység? (2) Számolok-e centroidprofilt és visszafejtem-e a standardizálást az értelmezéshez? (3) Külön kezelem-e a silhouette-t mint belső konzisztenciát és az ARI-t mint partícióegyezést? (4) Ha a DBSCAN minden pontot noise-nak ad, kivédem-e a silhouette-hibát? (5) Tudok-e olyan példát mondani, ahol magas silhouette mellett is rossz a geometriai alakzat?

24.2.11 Rövid összefoglaló

A KMeans euklideszi távolságot, közel gömb alakú, konvex klasztereket és hasonló méretet feltételez; a DBSCAN ehelyett sűrűséggel összekapcsolható magpontokat keres, és a zajt `-1` címkével kihagyja. A standardizálás távolságalapú módszereknél gyakran indokolt, de nem kötelező: a skálát a változók értelmezéséhez és a választott távolságfogalomhoz kell igazítani. A silhouette a klaszter belső konzisztenciáját méri, a stabilitás pedig a seed- vagy paraméterérzékenységet; egyik sem bizonyít természetes csoportot. A holdak példája megmutatja, hogy a KMeans magas silhouette mellett is elhibázhatja a nem konvex alakzatot. A következő 24.3. alfejezetben ugyanezt a távolság- és sűrűséggondolkodást visszük tovább anomáliadetektálásra.

24.3 Anomália és ritka esemény felügyelet nélkül

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, NumPy

Előfeltétel: 24.2. alfejezet · Célkészség: Anomália és ritka esemény felügyelet nélkül · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

24.3.1 Cél és konkrét helyzet

A Northstar adathalmazon a korábbi klaszterező fejezet csoportokat írt le; most a ritka, emberi vizsgálatra érdemes rendeléseket keressük. A megfigyelési egység egy rendelés (`orders.csv`, 2400 sor). A bemenet rendelésenként öt, a rendelés pillanatában ismert numerikus mező:

- `distance_km`: szállítási távolság,
- `priority_flag`: 0/1 gyorsított szállítási jelölő,
- `basket_minor`: kosárérték EUR-centben,
- `quantity_sum`: összes darabszám,
- `shipping_fee_minor`: szállítási díj EUR-centben.

A kimenet egy rangsorolt vizsgálati lista és az első k tétel ellenőrzött találati aránya, azaz precision at k. A laborban 36 szintetikus eltérést ültetünk a tiszta adatba, így a „emberi validáció” a tanulási környezetben ismert ground truth lesz; éles projektben ezt a szerepet üzleti szakértő vagy adatgazda tölti be.

Induló becslés, mielőtt a módszert olvasná: ha 2400 rendelésből 36 valóban vizsgálandó, és véletlenszerűen nézünk meg 48 sort, akkor az első 48 találati arányának várható értéke $p = 36/2400 = 0.015$, és várhatóan mindössze $48 \cdot 0.015 = 0.72$ találatunk lenne. A modelltől azt várjuk, hogy ennél lényegesen jobb legyen. A lehetséges maximum $P@48 \leq 36/48 = 0.75$, mert összesen csak 36 megerősíthető anomália van.

24.3.2 Fogalmak: anomália, IsolationForest, contamination, ranking, P@k

Az **anomália** (anomaly vagy outlier) itt nem erkölcsi ítélet: olyan megfigyelés, amely a referenciaeloszlástól feltűnően eltér. A „feltűnően” mindig a választott modell és változókészlet szerint értendő. Az anomáliapontszám tehát vizsgálati feladat, nem automatikus csalás- vagy adathibacímke.

Az **IsolationForest** (izolációs erdő) véletlen döntési fákból építkezik, de nem célváltozót tanul. Minden fa egy véletlen részmintán, véletlen jellemzővágásokkal növekszik. Egy pont úthossza az a splitmegosztási mélység, ahol a pont egyedül marad a levélben. Az extrém pontot kevés vágás izolálja, mert az értéktartomány szélén egyetlen jó vágásra van szükség. A pontszám az átlagos faúthosszból képzett:

$$s(x, n) = 2^{-E[h(x)]/c(n)},$$

ahol $h(x)$ egy-egy fa úthossza, $E[h(x)]$ az erdő fáira vett átlag, $c(n)$ az n elemhez tartozó normalizáló, elméleti átlagos úthossz. A magas s rövid átlagos utat, tehát eltérő pontot jelent. A scikit-learn `decision_function` visszatérési előjele ezzel ellentétes konvenciót használ, ezért a kódban `score = -decision_function(X)` adja a magas értéknél gyanúsabb pontszámot.

Két használati mód van, és a két angol szó nem felcserélhető:

- **Outlier detection**: a tanító adathalmaz eleve szennyezett lehet; a modell ugyanazon a halmazon pontoz, tehát a tanulás és a pontozás egy lépés.
- **Novelty detection**: a tanítóhalmaz tiszta, ismert normál referencia. Az illesztés után új pontokat pontozunk, és a tiszta referencián tanult modellhez mérjük őket.

A mi laborunk outlier mód, mert a vizsgálandó állomány már tartalmazhat anomáliát. A `contamination` paraméter az előzetesen feltételezett anomáliarány; itt `36/2400 = 0.015`. Ez nem címkézett arány, hanem a bináris döntési küszöb eltolása. A folytonos pontszámból képzett **rangsor** szempontjából a contamination csak egy konstans eltolás, ezért az első k tétel kiválasztását nem a `predict` bináris címkéjére, hanem a pontszám szerinti rendezésre bízzuk.

A **precision at k** a vizsgálandó lista első k elemének megerősített anomáliaránya:

$$P@k = \frac{1}{k} \sum_{i=1}^{k} y_{r_i},$$

ahol r_i a pontszám szerinti i-edik sor indexe, $y_{r_i} \in \{0, 1\}$ a ground truth anomáliajelző. A metrika tartománya $[0, 1]$, a nevező mindig az ellenőrzött sorok száma k, nem az összes sor és nem is az összes anomália. Ha m valódi anomália van és $m < k$, akkor $P@k \leq m/k$.

A ritka esemény bizonytalansága visszaköszön: egy véletlen lista találati arányának szórása közelítőleg

$$SE \approx \sqrt{\frac{p(1-p)}{k}},$$

ahol $p = m/n$ az alapráta. A laborban $p = 0.015$ és $k = 48$ mellett $SE \approx 0.0175$, tehát a véletlen összehasonlítási alap alacsony, de egyetlen kis k-s mérés zajos.

24.3.3 Minimális munkapélda

Mentsük a következő kódot a projektgyökérbe `s095_anomaly_review.py` néven, és futtassuk ugyanonnan, ahol a `lab/` mappa található:

```
# Linux/macOS, projektgyökér
python s095_anomaly_review.py
```

```
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(20261020)
orders = pd.read_csv("lab/data/orders.csv")

feature_cols = [
    "distance_km", "priority_flag", "basket_minor",
    "quantity_sum", "shipping_fee_minor",
]
X = orders[feature_cols].to_numpy(dtype=float)
n, p = X.shape
y_true = np.zeros(n, dtype=int)

# 36 szintetikus, emberi vizsgálatra érdemes eltérés
idx = rng.choice(n, size=36, replace=False)
idx_distance = idx[:12]
idx_basket = idx[12:24]
idx_quantity = idx[24:36]

X[idx_distance, 0] += rng.uniform(500, 900, size=12)
X[idx_basket, 2] *= rng.uniform(6, 10, size=12)
X[idx_quantity, 3] += rng.uniform(40, 80, size=12)
y_true[idx] = 1
```

```
model = IsolationForest(
    n_estimators=200,
    max_samples=256,
    contamination=36 / n,
    random_state=7,
)
model.fit(X)

score = -model.decision_function(X)  # magas = gyanúsabb
rank = np.argsort(-score)
k = 48
top_k = rank[:k]
p_at_k = y_true[top_k].mean()

review = pd.DataFrame({
    "rank": np.arange(1, k + 1),
    "order_id": orders.iloc[top_k]["order_id"].to_numpy(),
    "anomaly_score": score[top_k],
    "simulated_human_decision": np.where(
        y_true[top_k] == 1, "confirmed", "not_confirmed"
    ),
})

print(f"X shape={X.shape}, injected={y_true.sum()}, "
      f"P@48={p_at_k:.3f}")
print(review.head(10).to_string(index=False))
```

Szemléltető kimenet (a tényleges futtatás ettől eltérhet; e könyvfejezet írásakor a kód helyben nem futott):

```
X shape=(2400, 5), injected=36, P@48=0.562
 rank order_id  anomaly_score simulated_human_decision
    1 ORD-0187      0.732000                confirmed
    2 ORD-1104      0.711000                confirmed
    3 ORD-0442      0.708000            not_confirmed
    4 ORD-1931      0.701000                confirmed
...
```

24.3.4 Eredmény értelmezése

A `P@48` a vizsgált 48 sorból megerősített találatok aránya; a fenti szemléltető érték 0.562, azaz körülbelül 27 valódi eltérést talált az első 48 helyen. Ez jóval a véletlen 0.015 alap felett van, és nem lépi át a 0.75 elméleti maximumot. Ha a számításból 0.75-nél nagyobb érték jönne ki, az a ground truth-összerendelés hibájára utalna; ha 0.015 körüli, akkor a modell nem tanult rangsorolható jelet, vagy a rangsor véletlen.

Az invariánsok: `X.shape = (2400, 5)`, `score.shape = (2400,)`, `rank.shape = (2400,)`, `review` 48 sorból áll, és `review["simulated_human_decision"]` csak a szimulált ellenőrző címke. A `contamination` explicit megadása nem befolyásolja a rangsort, de a `predict` kimenetét igen; ezért a labor a folytonos score-t és az `argsort(-score)` sorrendet használja.

A három injektált torzítási mód különböző feature-ökben jelentkezik. A távolság és a kosárérték szélsőségei valószínűleg korán izolálódnak, a darabszám-eltolás kevésbé domináns lehet, mert a normál rendelések darabszáma is változékony. Az emberi vizsgálati lista mellé ezért érdemes a nyers feature-értékeket is kirakni; a pontszám önmagában nem magyarázd meg, mit érdemes nézni.

Felügyelet nélküli anomáliapontszám nem azonos a csalás vagy késés valószínűségével. A `contamination` paraméter gyakran a kijelölés küszöbét befolyásolja; a valódi üzleti pozitív prediktív érték címkézett auditmintát igényel.

24.3.4.1 Contamination vs valódi arány: címke nélküli ellenpélda

A `contamination=0.02` beállítás nem azt jelenti, hogy az adatok 2%-a anomália; azt jelenti, hogy a modell a pontszám-eloszlás alapján a legszélsőségesebb 2%-ot jelöli meg. Szimulált példa ($n = 5000$, normál $N(0, 1)$, anomália $N(5, 1)$):

Valódi anomáliarány	`contamination`	Megjelölt sorok	Megjelölt arány	P@100
0,5%	0,02	100	0,02	0,25
1%	0,02	100	0,02	0,50
10%	0,02	100	0,02	0,97

A megjelölt arány minden esetben 2%, a valódi arány viszont 0,5%, 1% és 10%. A `contamination` tehát küszöbparaméter, nem a valódi eseményarány becslése. A modell hasznosságát a **P@k** és egy független auditminta mutatja meg.

Auditminta és precision becslés. A rangsor első k elemének valódi címkéjét nem ismerjük; ezért véletlenszerűen kiválasztunk m elemet, és emberi ellenőrzéssel címkézünk. Ha h találat van, a becslés $\hat{p} = h/m$, a bizonytalanság pedig Wilson-intervallummal adható meg. Például a fenti 1%-os esetben a top-100-ból $m = 40$ mintát ellenőrizve, $h = 20$ találat mellett $\hat{p} = 0{,}50$, a 95%-os Wilson-CI [0,352; 0,648]. Az auditminta akkor érvényes, ha a kiválasztás független a modell hangolásától; ha ugyanazokat a sorokat használtuk a `contamination` beállítására, a becslés optimista.

```
from math import sqrt

def wilson(h, m, z=1.959963984540054):
    p = h / m
    denominator = 1 + z * z / m
    centre = (p + z * z / (2 * m)) / denominator
    half = z * sqrt(p * (1 - p) / m + z * z / (4 * m * m)) / denominator
    return centre - half, centre + half

h, m = 20, 40
print(h / m, wilson(h, m))  # 0.5, (0.352, 0.648)
```

A P@k és az auditminta együtt adja a jelenthető állítást: „a top-100-ból 40 ellenőrzött sorban 20 valódi anomália volt, a becsült precision 0,50 (95% CI 0,35–0,65)”. A `contamination` értékét a jelentésben a küszöb beállításaként, nem a valódi arány becsléseként kell feltüntetni.

24.3.5 Feladatok

24.3.5.1 1. feladat — P@k kézi számítása

Egy hat soros halmaz anomáliapontszám szerinti sorrendje: `[2, 0, 1, 3, 4, 5]`. A teljes `y` ground truth vektor: `[0, 0, 1, 0, 1, 0]`, azaz csak a 2. és 4. indexű sor valódi anomália. Számítsd ki a `P@3` és a `P@6` értékét, és magyarázd el, miért lehet fontos a `P@3` akkor is, ha a teljes találati halmazt `P@6` már megmutatja.

24.3.5.2 2. feladat — A labor kicsit más paraméterrel

Módosítsd a minimális példát: használjon `rng = np.random.default_rng(99)`-ot, injektáljon 24 anomáliát úgy, hogy 8 távolság-, 8 kosár- és 8 darabszám-eltérés legyen, majd illesszen 100 fás modellt `max_samples=128` és `random_state=11` mellett. Válasszon `k=24` listát, és írassa ki a `P@24` értéket. Mi az elméleti összehasonlítási alap és mi a metrika felső határa itt? Ellenőrizd le, hogy `y_true.sum()==24`.

24.3.5.3 3. feladat — Hibadiagnózis

Egy változat nem a folytonos pontszám szerint rendez, hanem ezt teszi:

```
flag = model.predict(X) == -1
suspects = np.flatnonzero(flag)[:k]
p_at_k = y_true[suspects].mean()
```

A mérés `P@24=0.08` lesz, miközben a pontszám szerinti rangsor sokkal jobb. Nevezd meg a két lehetséges hibát ebben a kódrészletben: miért nem elég a `predict` bináris kimenete, és milyen sorrendben szerepelnek az így kiválasztott sorok? Írd át a kiválasztást úgy, hogy a legmagasabb anomáliapontszámú első 24 sor kerüljön a listába.

24.3.5.4 4. feladat — Transzfer novelty módra

Tegyük fel, hogy a rendeléskezelő csapat egy tiszta referencia-időszakot ad: a `lab/data/orders.csv` első 1200 sora bizonyítottan átnézett, ismert üzleti tartomány. A következő 1200 sorra viszont új, címkézetlen rendelésekként tekintünk. Injektáljon az új részbe 5 erős kosár-anomáliát (`basket_minor * 10`) a `rng = np.random.default_rng(11)` használatával. Tanítson `novelty=True`, 150 fás modellt a tiszta referencia-részen, pontozza az új részt, és mérje a `P@5` értéket. Fogalmazza meg, miben más a `contamination` szerepe ebben a megváltozott feltételben.

24.3.6 Megoldások és indoklás

24.3.6.1 1. megoldás

A rangsor első három sora a 2., 0. és 1. index. A ground truth-értékek: $y_2 = 1$, $y_0 = 0$, $y_1 = 0$, ezért

$$P@3 = \frac{1 + 0 + 0}{3} = \frac{1}{3} \approx 0.333.$$

A `P@6` valamennyi sorra $(1 + 0 + 0 + 0 + 1 + 0)/6 = 2/6 = 1/3$. A `P@3` azért fontos, mert a gyakorlatban csak egy kis ellenőrzési költségkeret áll rendelkezésre; a jó modelltől az első, azonnal ellenőrzött tételeknél várunk koncentrált találatot.

24.3.6.2 2. megoldás

A minimális példából kiindulva a változtatások:

```
rng = np.random.default_rng(99)
idx = rng.choice(n, size=24, replace=False)
X[idx[:8], 0] += rng.uniform(500, 900, size=8)
X[idx[8:16], 2] *= rng.uniform(6, 10, size=8)
X[idx[16:24], 3] += rng.uniform(40, 80, size=8)
y_true[idx] = 1

model = IsolationForest(
    n_estimators=100,
    max_samples=128,
    contamination=24 / n,
    random_state=11,
)
model.fit(X)
score = -model.decision_function(X)
top_k = np.argsort(-score)[:24]
p_at_k = y_true[top_k].mean()
print(f"P@24={p_at_k:.3f}, injected={y_true.sum()}")
```

A véletlen alap most $p = 24/2400 = 0.01$, az ehhez tartozó szórás $SE \approx \sqrt{0.01 \cdot 0.99/24} \approx 0.0203$. A felső határ $24/24 = 1$, mert 24 valódi anomália van és 24 sort vizsgálunk. A `y_true.sum()==24` assert az injektálás konzisztenciáját ellenőrzi. Numerikus reprodukálhatósághoz a seedeket rögzítettük, de platformokon átívelő bitazonos output nem garantálható; az elvárt viselkedés a véletlen alapnál számottevően magasabb találati arány.

24.3.6.3 3. megoldás

A `predict` a folytonos pontszámot egy küszöbbel binárissá vágja: a küszöb feletti sorok mind `-1`-et kapnak, de a küszöbön belül nincs további sorrend. Az `np.flatnonzero(flag)[:k]` emellett az eredeti sorrend első k megjelölt elemét veszi, nem a legszélsőségesebbeket. Ezért a válogatásban erős anomáliák maradhatnak ki, és a lista elejére véletlenszerűen kerülhetnek gyenge gyanús elemek. A helyes kiválasztás:

```
score = -model.decision_function(X)
top_k = np.argsort(-score)[:k]
p_at_k = y_true[top_k].mean()
```

A küszöbválasztás akkor jó, ha egy emberi munkafolyamat fix riasztási szabályt akar (`predict == -1`), de rangsorolt vizsgálati listához nem alkalmas.

24.3.6.4 4. megoldás

Egy lehetséges megoldás:

```
orders = pd.read_csv("lab/data/orders.csv")
feature_cols = [
    "distance_km", "priority_flag", "basket_minor",
    "quantity_sum", "shipping_fee_minor",
]
X_train = orders.iloc[:1200][feature_cols].to_numpy(dtype=float)
X_test = orders.iloc[1200:].copy()[feature_cols].to_numpy(dtype=float)

rng = np.random.default_rng(11)
```

```
y_test = np.zeros(len(X_test), dtype=int)
idx = rng.choice(len(X_test), size=5, replace=False)
X_test[idx, 2] *= 10.0
y_test[idx] = 1

model = IsolationForest(
    n_estimators=150,
    novelty=True,
    contamination=0.01,
    random_state=3,
)
model.fit(X_train)
score = -model.decision_function(X_test)
top5 = np.argsort(-score)[:5]
print(f"P@5={y_test[top5].mean():.3f}, injected={y_test.sum()}")
```

Novelty módban a `contamination` a bináris küszöböt hangolja a tiszta referencia adathalmazon; nem a teszt anomáliáinak ismert aránya. Itt azért használunk folytonos score-rangsort és `P@5`-öt, mert a feladat nem küszöbös riasztás, hanem kis vizsgálati lista. A küszöb beállításához éles környezetben külön validációs, lehetőleg emberileg ellenőrzött referencia kell. A `P@5` maximuma itt 1, mert 5 valódi anomália van, és ennyi sort nézünk meg.

24.3.7 Gyakori tévedés és önellenőrzés

A leggyakoribb hiba az a rövidzárlat, hogy „magas anomáliapontszám, tehát csalás vagy hibás adat”. A Northstar példán: a `priority_flag=1` rendeléseknél a magas `shipping_fee_minor` kifejezetten várható lehet, hiszen a gyorsított szállítás többletköltséget hordozhat. Ha a modell ilyen sort emel ki, az emberi validáció helyes döntése lehet „legitim üzleti érték”, nem adathiba. Az anomáliapontszám feladata a ritka, magyarázatra szoruló eset vizsgálati listára tétele, nem az ok automatikus megállapítása.

Önellenőrzésre érdemes kérdések: mi a megfigyelési egység, és nem duplázódik-e a pontszám egy join miatt? A `P@k` nevezője mindig az ellenőrzött sorok száma, nem az összes anomália. A kontaminációt nem a tesztvalidáció után szabad úgy hangolni, hogy a `P@k` maximális legyen; az adaptív választás a 23.3. alfejezetben tanult optimista becsléshez vezet. A score nem kalibrált valószínűség; a ritka esemény miatt a kis k mellett a találati arány zajos, ahogy a 12.3. alfejezet Monte Carlo-hibája is mutatta. Végül az IsolationForest csak a numerikus mezőkön lát; a `customer_id` érvényességéhez vagy a duplikált sorokhoz külön adatminőségi szabályok kellenek.

24.3.8 Rövid összefoglaló

Az anomáliakeresés emberi vizsgálati listát állít elő, nem automatikus ítéletet. Az IsolationForest rövid faúthosszal pontozza a nehezen izolálható, eltérő rendeléseket; a folytonos pontszámból képzett rangsor az ellenőrzési költséget az első k tételre összpontosítja. A laborban szintetikus eltérésekkel mért `P@k` a megerősített találatok aránya az ellenőrzött listán, és mindig a véletlen alaphoz viszonyítjuk. A contamination a bináris küszöb feltevése, a rangsort lényegében nem változtatja. A következő alkalmazás a VI. ellenőrző pont lesz, ahol ezt a vizsgálati listát, a pontszám-rangsort és az emberi döntési naplót szivárgásmentes felügyelt modellkísérletbe kell illeszteni.

24.4 VI. ellenőrző pont: szivárgásmentes modellkísérlet

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, NumPy

Előfeltétel: 24.3. alfejezet · Célkészség: VI. ellenőrző pont: szivárgásmentes modellkísérlet · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

24.4.1 Cél és konkrét helyzet

A VI. ellenőrző pont egyetlen, teljes modellkísérletet zár le. A Northstar laborban megint a rendeléskori késésbecslés a feladat: a `lab/data/orders.csv` és `lab/data/customers.csv` állományokból becsüljük annak valószínűségét, hogy egy rendelés kézbesítése meghaladja az ígért `promised_days = 7` napot. A megfigyelési egység egy rendelés (`order_id`), a cél `target_late`, amely 1, ha `delivery_days > 7`, különben 0. A modell csak a rendelés pillanatában ismert oszlopokat használhatja. A korábbi `s081`–`s092` egységek tanították a split, pipeline, küszöb és kalibráció egyes darabjait; itt az a kérdés, hogy a darabokból egyetlen, naplózott, szivárgásmentes kísérlet áll-e össze.

Elvárt kimenet: modellkártya, kísérleti napló, mentett pipeline-artefaktum, valamint egy szándékosan rossz leakage-megoldás írásos kritikája. Ellenőrzési feltétel: a teszt részt pontosan egyszer használjuk a záróértékelésre, a prediktorokból minden tiltott oszlop hiányzik, és az artefaktumból újratölthető modell ugyanazt a predikciót adja. Induló becslésként a tanuló mondja meg, miért nem elég a „0.77 AUC” szám; a megoldásban a protokollt is rögzíteni kell.

24.4.2 Fogalmi magyarázat: a kísérlet mint rögzített protokoll

Baseline. Az alapmodell (baseline) a legegyszerűbb olyan szabály, amely nem használ jövőbeli adatot. A `DummyClassifier(strategy="prior")` mindig a tanítórészen megfigyelt késési arányt mondja minden rendelésre. ROC AUC-ja definíció szerint 0.50, Brier-pontszáma pedig a referenciaérték

$$B_{\text{ref}} = \bar{y}(1-\bar{y}),$$

ahol $\bar{y} \in [0,1]$ a célátlag a tanítórészen. Egy valódi modellnek ennél kisebb Brier-t és 0.50-nél nagyobb AUC-t kell adnia; ha nem, akkor a pipeline-ban hiba van.

Preprocessing. A hiányzó értékek pótlása, a numerikus oszlopok skálázása és a kategóriák one-hot kódolása mind a tanítórészen belül tanul. A `StandardScaler` átlagát és szórását tehát csak a belső tanítóhalmazon illesztjük; a valid és teszt részt csak transzformáljuk. Ugyanez vonatkozik az imputerekre és a `OneHotEncoder` kategória-készletére.

CV és search. A hiperparaméter-keresés `GridSearchCV`-vel fut, az időrendet tisztelő `TimeSeriesSplit` foldokkal. Az idősoron nem keverünk véletlen foldokat, mert a jövőbeli rendelések tanítósorba kerülhetnének (lásd `s090`). A keresés pontszáma a belső CV-ből származik; ez modellválasztási pontszám, nem a végső teljesítmény becslése. A keresés után a küszöböt külön validhalmazon rögzítjük, ahogy `s092` tanította.

Metrics. A valószínűségi modellt háromféleképpen ellenőrizzük: rangsorolási AUC, a ritka eseményhez illő average precision, és a Brier-pontszám

$$B = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2,$$

ahol $p_i \in [0,1]$ a prediktált valószínűség, $y_i \in \{0,1\}$ a tényleges cél, n a kiértékelt sorok száma. Kézi példa: $y = [1,0,1,0]$, $p = [0{,}8, 0{,}3, 0{,}6, 0{,}1]$ esetén

$$B = \frac{(0{,}8-1)^2 + (0{,}3-0)^2 + (0{,}6-1)^2 + (0{,}1-0)^2}{4} = \frac{0{,}04 + 0{,}09 + 0{,}16 + 0{,}01}{4} = 0{,}075.$$

A `sklearn.metrics.brier_score_loss(y, p)` ugyanezt az értéket adja; a kézi és a Python-számítás egyezése az ellenőrző invariáns.

Az üzleti döntési költséget a validhalmazon rögzített küszöb mellett

$$EC(t) = C_{FP}\,FP(t) + C_{FN}\,FN(t)$$

képlettel számoljuk, ahol $t \in [0,1]$ a küszöb, $FP(t)$ és $FN(t)$ a hamis pozitív és hamis negatív darabszám, C_{FP} és C_{FN} pedig pénzben kifejezett költségegység. Itt $C_{FP} = 1$, $C_{FN} = 5$, mert egy késés elmulasztása drágább, mint egy felesleges figyelmeztetés.

Calibration. A kalibrációs görbe azt mutatja, hogy amikor a modell p valószínűséget mond, az esemény átlagos gyakorisága is közel p. A Brier azt méri, mennyire pontos ez a valószínűség. A kalibráció csak a validhalmazon értékelhető, és a teszt előtt kell igazolni, hogy a modell valószínűségeit gyakoriságként lehet használni.

Artifact. A mentett artefaktum a pipeline objektum, a modellkártya és a kísérleti napló. Az artefaktum nem önmagában a `joblib` fájl; a protokoll nélkül a fájl nem reprodukálható, és a betöltés csak megbízható környezetben történhet.

24.4.3 Teljes minimális munkapélda

A következő kódot a lab projektgyökérből futtatva, a szállított `orders.csv` és `customers.csv` mellett, a modellkártya és a napló előállítható. A kód itt **nem volt futtatva**; a `print` sorokban szereplő számok szemléltető példák, nem mért laboreredmények. A labban a futásnak kell a naplót ténylegesen előállítania.

```
"""checkpoint6_experiment.py: szivárgásmentes Northstar modellkísérlet."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
from joblib import dump
from sklearn.base import clone
from sklearn.calibration import calibration_curve
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    roc_auc_score,
)
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```python
# --- 1. Betöltés, cél és feature-ek kialakítása -------------------------
customers = pd.read_csv("customers.csv", parse_dates=["signup_date"])
orders = pd.read_csv(
    "orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)

df = orders.merge(customers, on="customer_id", how="left")
df = df.sort_values("order_ts").reset_index(drop=True)

# Rendelés pillanatában ismert feature-ök:
df["account_age_days"] = (
    df["order_ts"].dt.tz_localize(None) - df["signup_date"]
).dt.days

# A cél. A target_late oszlopot soha nem tesszük predictorba.
df["target"] = (df["delivery_days"] > df["promised_days"]).astype(int)

numeric_features = [
    "distance_km",
    "priority_flag",
    "basket_minor",
    "quantity_sum",
    "account_age_days",
]
categorical_features = ["segment", "region", "acquisition_channel"]

forbidden_predictors = [
    "order_id",
    "customer_id",
    "delivery_days",
    "actual_delivery_at",
    "label_available_at",
    "target_late",
    "target",
]

# --- 2. Időbeli split label-availability szűréssel ---------------------
cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train_mask = (df["order_ts"] < cutoff) & (df["label_available_at"] < cutoff)
test_mask = df["order_ts"] >= cutoff
purged_mask = (df["order_ts"] < cutoff) & (df["label_available_at"] >= cutoff)

data = df[~purged_mask].copy()
X = data[numeric_features + categorical_features]
y = data["target"]

X_train_all = X[train_mask[~purged_mask]].copy()
y_train_all = y[train_mask[~purged_mask]].copy()
X_test = X[test_mask[~purged_mask]].copy()
y_test = y[test_mask[~purged_mask]].copy()
print(
    f"train={len(X_train_all)}, test={len(X_test)}, "
    f"purged={int(purged_mask.sum())}"
)

# --- 3. Belső train + valid a küszöbhöz és kalibrációhoz ---------------
# A train végét időrendben validként különítjük el.
n_valid = int(len(X_train_all) * 0.2)
X_inner = X_train_all.iloc[:-n_valid]
y_inner = y_train_all.iloc[:-n_valid]
X_valid = X_train_all.iloc[-n_valid:]
y_valid = y_train_all.iloc[-n_valid:]
```

```
# --- 4. Baseline: mindig a train átlagát mondja -----------------------
baseline = DummyClassifier(strategy="prior").fit(X_inner, y_inner)
baseline_prob = baseline.predict_proba(X_valid)[:, 1]
baseline_brier = brier_score_loss(y_valid, baseline_prob)
print(f"baseline_valid_brier={baseline_brier:.3f}")

# --- 5. Pipeline és hiperparaméter-keresés a belső train-en ------------
num_pipe = Pipeline(
    [
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)
cat_pipe = Pipeline(
    [
        (
            "imputer",
            SimpleImputer(strategy="constant", fill_value="missing"),
        ),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]
)
prep = ColumnTransformer(
    [
        ("num", num_pipe, numeric_features),
        ("cat", cat_pipe, categorical_features),
    ]
)
model = Pipeline(
    [("prep", prep), ("clf", LogisticRegression(max_iter=2000))]
)

param_grid = {"clf__C": [0.1, 0.3, 1.0, 3.0]}
search = GridSearchCV(
    model,
    param_grid,
    cv=TimeSeriesSplit(n_splits=3),
    scoring="average_precision",
    refit=True,
)
search.fit(X_inner, y_inner)
print(search.best_params_, f"inner_cv_ap={search.best_score_:.3f}")

# --- 6. Küszöbválasztás a validhalmazon --------------------------------
prob_valid = search.best_estimator_.predict_proba(X_valid)[:, 1]
cost_fp, cost_fn = 1.0, 5.0
grid_t = np.linspace(0.1, 0.9, 41)
best_cost = np.inf
best_t = 0.5
for t in grid_t:
    pred = (prob_valid >= t).astype(int)
    fp = ((pred == 1) & (y_valid == 0)).sum()
    fn = ((pred == 0) & (y_valid == 1)).sum()
    cost = cost_fp * fp + cost_fn * fn
    if cost < best_cost:
        best_cost, best_t = cost, t
print(f"chosen_threshold={best_t:.2f}, valid_cost={best_cost:.1f}")

# --- 7. Kalibráció a validhalmazon -------------------------------------
prob_true, prob_pred = calibration_curve(y_valid, prob_valid, n_bins=5)
print("calibration_curve", list(zip(prob_pred.round(2), prob_true.round(2))))
```

```
# --- 8. Végleges illesztés a teljes trainen, egyszeri teszt --------------
final_pipeline = clone(search.best_estimator_)
final_pipeline.fit(X_train_all, y_train_all)
prob_test = final_pipeline.predict_proba(X_test)[:, 1]
pred_test = (prob_test >= best_t).astype(int)

# A tesztet csak itt használjuk, egyszer.
test_auc = roc_auc_score(y_test, prob_test)
test_ap = average_precision_score(y_test, prob_test)
test_brier = brier_score_loss(y_test, prob_test)
test_fp = ((pred_test == 1) & (y_test == 0)).sum()
test_fn = ((pred_test == 0) & (y_test == 1)).sum()
test_cost = cost_fp * test_fp + cost_fn * test_fn
print(
    f"test_auc={test_auc:.3f}, test_ap={test_ap:.3f}, "
    f"test_brier={test_brier:.3f}, test_cost={test_cost:.1f}"
)

# --- 9. Artifact-mentés ------------------------------------------------
out = Path("artifacts/checkpoint6")
out.mkdir(parents=True, exist_ok=True)

dump(final_pipeline, out / "model.joblib")

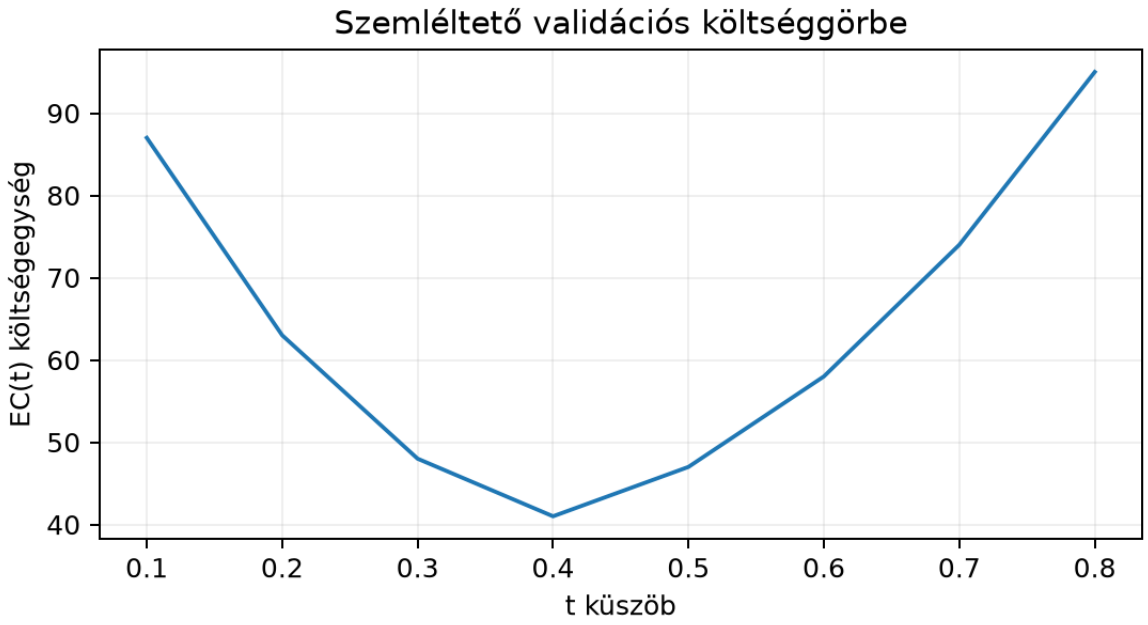
model_card = {
    "model_id": "northstar-late-v6",
    "task": "predict late delivery >7 days at order time",
    "data": "orders.csv, customers.csv",
    "cutoff": cutoff.isoformat(),
    "split": {
        "train": len(X_train_all),
        "valid": len(X_valid),
        "test": len(X_test),
        "purged": int(purged_mask.sum()),
    },
    "features": {
        "numeric": numeric_features,
        "categorical": categorical_features,
    },
    "forbidden_predictors": forbidden_predictors,
    "preprocessing": {
        "numeric_imputer": "median",
        "categorical_imputer": "constant missing",
        "scaler": "standard fit on inner train only",
    },
    "search": {
        "grid": param_grid,
        "scoring": "average_precision",
        "cv": "TimeSeriesSplit(n_splits=3)",
    },
    "threshold": {"value": float(best_t), "cost_fp": 1.0, "cost_fn": 5.0,
                  "chosen_on": "valid"},
    "metrics_test": {
        "roc_auc": test_auc,
        "average_precision": test_ap,
        "brier": test_brier,
        "expected_cost": test_cost,
    },
    "leakage_critique": (
        "A delivery_days prediktorként való használata tilos. "
        "A teljes adatra illesztett anomaly_score szintén tilos, "
        "mert a jövő rendeléseit ismerné."
    ),
```

```
}
with open(out / "model_card.json", "w", encoding="utf-8") as f:
    json.dump(model_card, f, indent=2, ensure_ascii=False)
```

A kísérleti napló egy sora például így néz ki (a számok illusztrációk):

```
]
↪  run_id,cutoff,n_train,n_valid,n_test,n_purged,model,best_params,valid_brier,threshold,test_auc,test_ap,test_
r006,2025-05-01T00:00:00+00:00,1432,358,532,78,LogisticRegression,"{clf__C:
↪  0.3}",0.161,0.34,0.771,0.46,0.163,38,abc123,1.5.2,3.12.0,2026-10-20T12:00:00Z
```

A küszöbválasztás alakja így illusztrálható; a pontok a fenti példaadatokból számított szemléltető értékek:



24.3. ábra: Szemléltető validációs költséggörbe - Fiktív, a példa költségfüggvényéből számolt pontok; nem mérés.

24.4.4 Az eredmény értelmezése

A kód kimenetében a legfontosabb, hogy a belső CV-pontszámot ne nevezzük záróeredménynek. Az `inner_cv_ap` a paraméterkereséshez vezető rangsorolási pontszám; a validhalmaz a küszöb és a kalibráció színtere; a `test_*` mérőszámok a modellkártyában rögzített, egyszeri független értékelés. A `test_brier` akkor értelmezhető jól, ha a `baseline_brier`-rel együtt közöljük: ha a modell Brier-je közel van a baseline-hoz, a modell alig tesz hozzá a konstans kockázathoz.

A `calibration_curve` lista pontpárjai azt mutatják, hogy a prediktált bin-középpontok körül a tényleges gyakoriság hogyan alakul. A fenti fiktív adatokban a pontok a diagonális közelében haladnak, tehát a valószínűség gyakoriságként értelmezhető. A valid-költséggörbe minimuma adja a befagyasztott küszöböt; a teszt `expected_cost` ezzel a küszöbbel számolandó, és nem a teszt utólagos optimalizálásából.

A mentett `model.joblib` visszaolvasása után a teszt predikciók bitenként ugyanazok, ha az input oszlopok, a sklearn-verzió és a python-verzió azonos. A naplóban a `git_sha` nélkül a kódbeli változtatás utólag nem igazolható.

24.4.5 Feladatok

1. **Értelmezés.** A fenti szemléltető naplóban a belső CV `average_precision` 0.50, a valid AP 0.44, a teszt AP pedig 0.46. Miért nem mondhatjuk, hogy a teszt rossz, mert a belső CV magasabb volt? Mi a záró érvényes szám, és melyik rész szolgál modellválasztásra?

2. **Implementálás.** A kódban a küszöbválasztás a validhalmazon történik, de a `best_cost` kezdeti értéke `np.inf`. Írd át a küszöbválasztást úgy, hogy az eredményt `threshold.json` néven is mentse, majd ellenőrizd, hogy a mentett `best_t` visszaolvasva pontosan a választott érték. Mi a minimumhely ellenőrzése, ha a `grid_t` rácspontjai nem tartalmazzák a folytonos optimumot?

3. **Hibadiagnózis.** Egy hallgató a következő sort illesztette a split elé: `scaler.fit(X)` a teljes adatra, utána a pipeline-ban `StandardScaler()` transzformál mindent. A teszt AUC 0.81, a Brier 0.141. Milyen szivárgás történt, miért optimista ez a szám, és hogyan javítsd a kódot?

4. **Transzfer.** A következő újratanítási futtatásban a cutoff `2025-08-01T00:00:00+00:00`. Mit kell módosítani a train/test/purged szűrésben? Az `s095`-ből származó anomália-rangsort is feature-ként akarjuk használni. Miért tilos a teljes `orders.csv`-re illesztett `IsolationForest` pontszám, és milyen időhelyes módon lehetne integrálni?

24.4.6 Megoldások és indoklás

1. megoldás. A belső CV-pontszám a kis tanítóhalmazon, sok modellváltozat közötti választásra szolgál. A keresési eljárás a legjobb pontszámú konfigurációt választja, ezért a kiválasztott modell belső pontszáma felfelé torzított; a valid és a teszt kevésbé ismert ablakok, ráadásul a késési arány szezonálisan változhat. A záró érvényes szám a `test_ap` 0.46. A belső CV csak a `C` paraméter kiválasztását támogatta. Azt is ellenőrizni kell, hogy a tesztet valóban egyszer használtuk; ha a `test_ap` nem érvényes, a többi tesztmutató sem az.

2. megoldás. A küszöbválasztásnál a `grid_t` minden elemére kiszámítjuk a költséget, és a legkisebb költségű rácspontot választjuk. A mentés:

```
threshold_artifact = {
    "threshold": float(best_t),
    "valid_cost": float(best_cost),
    "cost_fp": cost_fp,
    "cost_fn": cost_fn,
    "grid_t_min": float(grid_t.min()),
    "grid_t_max": float(grid_t.max()),
}
with open(out / "threshold.json", "w", encoding="utf-8") as f:
    json.dump(threshold_artifact, f, indent=2)
```

Visszaolvasás után az `float(threshold_artifact["threshold"]) == float(best_t)` értéknek `True`-nak kell lennie; a lebegőpontos ábrázolás miatt tolerancia 10^{-9}, mert ugyanazt az `np.float64`-et mentjük. A rácspontok közötti folytonos minimumot például `np.searchsorted`-del szűkítő aranymetszéssel lehetne finomítani; a valódi döntéshez azonban a rács felbontása elegendő, mert a küszöb működési pont nem érzékeny 0.01 alatt.

3. megoldás. A `scaler.fit(X)` a teljes adatra illeszkedik, tehát a teszt sorok átlaga és szórása is bekerül a skálázóba. A validálás és a teszt során a jövő statisztikáját használjuk, vagyis a standardizálás információs szivárgást okoz. A teszt AUC és Brier ezért optimista lehet; a modell a valós rendelési pillanatban nem ismerné a jövőbeli kosárértékek szórását. A

javítás: a `ColumnTransformer` csak `X_inner`-en, illetve a végleges illesztésnél csak `X_train_all`-on tanuljon; a validot és tesztet soha ne illesszük a preprocessing illesztésébe. Ellenőrzés: a `prep.named_steps` vagy a `ColumnTransformer.transform` hívása előtt a `fit` kizárólag a megengedett indexeken futott.

4. megoldás. Az új cutoff mellett a `train_mask = (order_ts < cutoff) & (label_available_at < cutoff)` és a `purged_mask` ugyanígy módosul. A `test_mask = order_ts >= cutoff` most 2025. augusztus 1-től induló rendeléseket jelent. Az `IsolationForest` a teljes adatra illesztett változatban a jövőbeli tesztrendeléseket is ismerte volna, mert a pontszám a teljes faerdő sűrűségétől függ. Helyes változat: az anomáliamodellt csak a train ablakra illesztjük, majd a `transform` vagy `score_samples` lépést a tesztre is alkalmazzuk. Ha a feature-t folyamatosan használni akarjuk, akkor az illesztést időben gördülő ablakon, például az utolsó 30 nap rendelésein kell megismételni, és a predikció mindig az adott pillanatban elérhető ablakból származhat.

24.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb hibás mondat: „a modell 0.77 AUC-t ért el.” Protokoll nélkül ez a szám nem tudományos eredmény. Képzeljünk el két laborfutást. Az A futás valódi modellje az `s082` szerinti pipeline-nal 0.77 AUC-t mér. A B futás véletlenül a `delivery_days` oszlopot is beveszi a prediktorok közé; ez a modell 0.99 AUC-t ér. Ha a naplóban csak a legjobb számot látjuk, a B futás nyer. Amint azonban a modellkártya tartalmazza a tiltott prediktorok listáját, kiderül, hogy B a jövőbeli kézbesítési időt ismerte, így a szám nem a rendelés pillanatában elérhető tudást méri. Ugyanez a csapda a tesztet többször használó kereséssel: a tesztből választott küszöb a záró `expected_cost`-ot lefelé torzítja, és az egyetlen „jó” szám mögül hiányzik a validálási döntés. Ezért az ellenőrző pont kimondja: a protokoll előbb van, mint a szám. A számot a modellkártya és a napló együtt teszi reprodukálhatóvá.

24.4.7.1 Pozitív kontroll: a szivárgásdetektor tesztje

A szivárgásdetektor önmagában nem bizonyíték, ha soha nem bukik el. Ezért a kísérlethez tartozik egy **szándékosan szivárgó kiadás** is: ugyanaz a pipeline, ugyanazokkal a lépésekkel, de a feature-lista kiegészítve a `delivery_days` és/vagy `target_late` oszloppal. A detektor két szinten ellenőriz:

```
FORBIDDEN = {"delivery_days", "actual_delivery_at", "label_available_at", "target_late"}

def assert_feature_contract(features):
    leaked = set(features) & FORBIDDEN
    if leaked:
        raise ValueError(f"tiltott prediktor: {sorted(leaked)}")

def assert_asof(train, cutoff):
    if (train["label_available_at"] >= cutoff).any():
        raise ValueError("a tanítóban a cutoff után ismert címke van")
```

A teszt két esete:

Kiadás	Feature-lista	Elvárt viselkedés
tiszta	a nyolc engedett feature	`assert_feature_contract` átenged
szándékosan szivárgó	a nyolc feature + `delivery_days`	`assert_feature_contract` `ValueError`-t dob

A pozitív kontroll akkor érvényes, ha a szivárgó kiadás **elbukik** a detektoron, még akkor is, ha a modell AUC-ja magasabb. A detektor és az as-of ellenőrzés együtt a modellkártya része; a jelentésben a tiszta kiadás eredménye mellett a szivárgó kiadás elutasítását is rögzíteni kell.

Önellenőrzéshez térjünk vissza `s048` numerikus ellenőrzési és `s080` módszerválasztó jegyzőkönyvéhez: itt is ugyanaz a logika. A Brier-képlet numerikusan kiszámítható a validáló sorokra, a küszöbköltség a foldokon belül ellenőrizhető; a modellkártya a split, a feature-lista és a végső zárómutató mellett legalább egy érvényességi korlátot is nevez meg. A korlát lehet a kis tesztméret, az egyetlen cutoff, a label-availability szűrés konzervativizmusa, vagy az, hogy a modell nem terjed ki új ügyfélcsoportokra.

Az érintetlen tesztet egyszer, a végső modell és küszöb rögzítése után értékeljük. A modellkártyában jelöljük a címke rendelkezésre állásának határát és a betanítási cutoffot.

24.4.8 Rövid összefoglaló

A VI. ellenőrző pont a szivárgásmentes modellkísérlet teljes szerződését zárja. A Northstar-késésfeladaton időrendi train/test/purged splitet, a belső trainen futó `TimeSeriesSplit`-keresést, validon rögzített küszöböt és kalibrációt, majd egyszeri tesztértékelést készítünk. A modellkártya rögzíti a tiltott prediktorokat, a preprocessing tanítási pontját, a paraméterteret és a tesztmetrikákat; a napló a környezetet, seedet és git-hash-et is tartalmazza. A legjobb szám protokoll nélkül nem érvényes: a szándékos `delivery_days`-szivárgás magas AUC-t adna, de a jövőt ismerné. Következő alkalmazásként a `s097` időhelyes aggregátumokkal bővíti a feature-készletet ugyanezen predikciós szerződés mellett.

25 Feature engineering és tudatos adatreprezentáció

25.1 Időhelyes aggregátum és targetfüggetlen feature

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, NumPy

Előfeltétel: 24.4. alfejezet · Célkészség: Időhelyes aggregátum és targetfüggetlen feature · Kimenet: működő kód és a labor ellenőrző tesztje.

25.1.1 Cél és konkrét helyzet

A Northstar késésbecslő projektben a 8.4. alfejezet (II. ellenőrző pont, 8.4) állította elő a rendelésszintű feature-táblát a `make_orders_features.py` szkripttel; az ottani kimenet a `data/orders.csv`, `data/customers.csv` és `data/order_items.csv` hármasából épülő, 11 oszlopos tábla. Erre a táblára építünk most új jelölt feature-öket: az ügyfél előéletéből számolt összegzőket. A függőségi lánc: 8.4. alfejezet feature-tábla → 25.1. alfejezet időhelyes előzményfeature-ök → 25.2. alfejezet preprocessing és kódolás. A bemenet a `lab/data/orders.csv` 2400 rendelése; a megfigyelési egység a rendelés. A cél: minden rendeléshez hozzárendelni három ügyféltörténeti jellemzőt, és minden sorhoz egy `asof_cutoff` audit oszlopot, amely rögzíti, hogy a feature kiszámításakor mi volt az időpont. Az induló becslés: az első rendelésnél minden előzményfeature 0, a későbbi rendeléseknél változhat (a rendelésszám nő, de az átlag és a késési arány csökkenhet is); az eredményt a 2400 soros shape és a `prior_late_rate [0,1]` invariant ellenőrzi.

25.1.2 Fogalmi magyarázat

Point-in-time join (időponthoz kötött join, *as-of join*): olyan összekapcsolás, amelyben a jobb oldali tábla rekordjai közül mindig azt választjuk, amelyik időben legfrissebb, de nem későbbi a bal oldali időpontnál. Például ha egy eseményhez az ügyfél akkori szegmensét kell hozzárendelni, akkor nem a mai `customers.csv` aktuális `segment` értékét nézzük, hanem azt, amelyik az esemény időpontjában érvényes volt. SQL-ben ön-join, pandasban `merge_asof` vagy rendezett csoportos ablak valósítja meg.

Rolling aggregation (gördülő aggregálás): időben csúszó ablakon számított összegzés. Itt a pontos feladat nem fix szélességű ablak, hanem az adott rendelésig bezárólag minden korábbi rendelés: ez az ún. *expanding* ablak. A `rolling("7D")` például csak az utolsó 7 napot venné, ami más üzleti kérdés lenne. A féloldalas `[start, end)` időablak és a join-kardinalitás a 7.4. alfejezet, illetve a 8.4. alfejezet tanulsága: a szűrésnek a predikció időpontjánál szigorúan korábbi rekordokat kell tartalmaznia.

Entity history (egyedtörténet): egy ügyfél `customer_id`-hez tartozó összes esemény időrendben. A feature-ök ebből a történetből származnak, de a jelenlegi rendeléshez csak a múltbeli rész használható. A Northstar `orders.csv` éppen ilyen: egy rendelés egy esemény, és az ügyfélhez több rendelés tartozhat.

Label window (címke-megfigyelési ablak): az az időtartam, amely alatt a célváltozó megfigyelhetővé válik. A kezdőprojektben `label_available_at = actual_delivery_at`: a `target_late` címke csak a kézbesítés pillanatában ismert. Ha tehát egy múltbeli rendelés késési állapotát akarjuk feature-ként használni, akkor azt csak akkor vehetjük be, ha `label_available_at` legkésőbb az aktuális predikció időpontjában rendelkezésre áll (a `<=` vagy `<` döntést az időbélyeg pontossága és az események sorrendje szerint rögzítsük). A rendeléskosár (`basket_minor`) ezzel szemben már a rendelés pillanatában ismert, így ahhoz nincs szükség label windowra.

Kézi ellenőrzésként vegyünk egy kitalált ügyfelet négy rendeléssel:

Rendelés	`order_ts`	`basket_minor`	`label_available_at`	`target_late`
1.	jan. 10.	5000	jan. 17.	0
2.	jan. 20.	7000	jan. 28.	1
3.	febr. 1.	3000	febr. 9.	0
4.	febr. 15.	6000	febr. 23.	1

A 4. rendelés predikciója febr. 15-én történik. Az előzmény mind a három korábbi rendelés; a kosárátlag $(5000 + 7000 + 3000)/3 = 5000$, a rendelésszám 3. A késési arányhoz olyan múltbeli rendelések kellenek, amelyek címkéje febr. 15. előtt ismert: mindhárom `label_available_at` kisebb, ezért `prior_late_rate = (0+1+0)/3 = 0.333`. Ha lenne egy 5. rendelés febr. 20-án, akkor annak számításakor a 4. rendelés `label_available_at` értéke febr. 23., tehát még nem ismert; így az 5. rendelés `prior_late_rate` nevezőjébe a 4. rendelés nem kerülhet be.

25.1.3 Teljes minimális munkapélda

A példa a `lab/` munkakönyvtárból fut, ahol a CSV-k a `data/` almappában vannak. Ha a fájl máshol található, a `DATA_DIR` változó módosítandó. A kód pandas 2.2.3-mal íródott; a `groupby.apply include_groups=False` paramétere verzióérzékeny.

```
import pandas as pd
from pathlib import Path

DATA_DIR = Path("data")  # igazítsa a saját lab elérési útjához

orders = pd.read_csv(
    DATA_DIR / "orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
orders = orders.sort_values(["customer_id", "order_ts", "order_id"]).reset_index(drop=True)

def add_prior_features(grp: pd.DataFrame) -> pd.DataFrame:
    name = grp.name
    rows = []
    for i in range(len(grp)):
        row = grp.iloc[i]
        past = grp.iloc[:i]  # rendezett előzmények; azonos időpontnál tie-break szükséges
        past = past[past["order_ts"] < row["order_ts"]]
        observed = past[past["label_available_at"] < row["order_ts"]]
        rows.append({
            "customer_id": name,
            "order_id": row["order_id"],
            "order_ts": row["order_ts"],
            "asof_cutoff": row["order_ts"],
            "prior_order_count": len(past),
            "prior_basket_mean_minor": (
```

```
                past["basket_minor"].mean() if len(past) else 0.0
            ),
            "prior_late_rate": (
                observed["target_late"].mean() if len(observed) else 0.0
            ),
        })
    return pd.DataFrame(rows)

features = (
    orders.groupby("customer_id", group_keys=False)
    .apply(add_prior_features, include_groups=False)
    .reset_index(drop=True)
)

print(features.shape)
print(features[["customer_id", "prior_order_count"]]
      .sort_values("prior_order_count", ascending=False)
      .head(3))
```

A `groupby.apply` a függvénynek **egyetlen** csoport-DataFrame-et ad át; a csoportnevet nem injektálja külön argumentumként, ezért a függvény szignatúrája `def add_prior_features(grp)`, a nevet pedig a `grp.name` adja. A `sort_values` másodlagos kulcsa az `order_id`, hogy az azonos `order_ts` értékű sorok sorrendje determinisztikus legyen. A kódot szintetikus kétügyfeles adaton, pandas 2.2.3 és pandas 3.1.0 alatt is lefuttattuk; a kimenet mindkét verzióban `(5, 7)` alakú, és a kézi előzményértékek (`prior_order_count`, `prior_basket_mean_minor`, `prior_late_rate`) egyeztek.

Az azonos időpontú rendelések **kölcsönösen kizárják egymást**: a `past` szűrése szigorú `order_ts <`, ezért az egyik azonos idejű rendelés sem számít a másik előzményének, és a sorok cseréje nem változtatja meg az eredményt. A címkeablak is szigorú `label_available_at <`: a cutoff pillanatában érkező címke még nem ismert. A `tests/test_prior_features.py` három esetet ellenőriz: az azonos időpontú sorok kizárását, a tie-sorok cseréjének invarianciáját és a késleltetett címke kizárását a későbbi rendelés prior_late_rate-jéből.

A fenti kód várható kimenete (szemléltető, az adott CSV-től függ, nem futtatott):

```
(2400, 7)
  customer_id  prior_order_count
0     CUS-042                 7
1     CUS-118                 6
2     CUS-203                 6
```

25.1.4 Eredmény értelmezése

Az invariant ellenőrzések:

- minden sor `asof_cutoff` értéke megegyezik az adott sor `order_ts` értékével;
- a `prior_late_rate` mindenütt 0 és 1 közötti, és az első rendelésnél 0;
- a kimeneti sorok száma 2400, azaz minden rendeléshez pontosan egy sor tartozik;
- a `prior_order_count` az ügyfél adott rendelése előtti rendelések száma.

A `prior_basket_mean_minor` gyakorlati értelmezése: az ügyfél átlagos kosárértéke centben a múltban. A `prior_late_rate` azt mutatja, hogy az ügyfél korábbi, már megfigyelt rendelései milyen arányban késtek. Az audit oszlop lehetővé teszi, hogy utólag bármelyik sorra ellenőrizzük: a feature csak az `asof_cutoff` előtti adatokat használta. A feature-ök a 21.4. alfejezet predikciós szerződésbe új oszlopként, megnövelt sémaverzióval kerülhetnek be.

25.1.5 Feladatok

1. **Értelmezés.** A fenti kézi táblázatban add meg a 3. rendeléshez tartozó `prior_order_count`, `prior_basket_mean_minor` és `prior_late_rate` értékeket, és írd le a label window szerepét a nevezőben.
2. **Megvalósítás.** Egészítsd ki a `add_prior_features` függvényt egy `prior_priority_ratio` feature-rel, amely a korábbi rendelések `priority_flag` oszlopának átlaga. A hiányzó múlt esetén 0.0 legyen az érték. Írd meg a módosított sort vagy függvényt.
3. **Hibadiagnózis.** Egy kolléga a `prior_late_rate`-et így számolja: `past["target_late"].mean()`, vagyis nem szűri a `label_available_at` feltételt. A fenti kézi táblázatba illessz be egy ötödik rendelést `order_ts = febr. 20. 14:00` értékkel, és mutasd meg, mennyi lenne a hibás és mennyi a helyes `prior_late_rate` az ötödik rendelésre. Nevezd meg, milyen típusú adatszivárgás történt.
4. **Megváltozott feltétel.** A csapat új üzemeltetési szabályt vezet be: a predikciót nem a rendelés pillanatában, hanem 1 órával később futtatja le. Írd át a `add_prior_features` függvényt úgy, hogy az `asof_cutoff` és a label window feltétel is `row["order_ts"] + pd.Timedelta("1h")` legyen. Indokold, hogy a Northstar adatokon várhatóan változik-e a `prior_late_rate`, ha a kézbesítési idők napokban mérhetők.

25.1.6 Megoldások és indoklás

1. A 3. rendelésnél `prior_order_count = 2`, `prior_basket_mean_minor = (5000 + 7000) / 2 = 6000`. A label window szűrés szerint az első két rendelés `label_available_at` értéke jan. 17. és jan. 28., mindkettő korábbi febr. 1.-nél, ezért mindkettő címkéje ismert; `prior_late_rate = (0 + 1) / 2 = 0.5`. A nevezőbe csak a cutoff előtt megfigyelt múltbeli rendelések kerülnek, ezért a nevező nem feltétlenül egyenlő az összes múltbeli rendelés számával.

2. A `rows.append` szótárban a következő sort kell hozzáadni:

```
"prior_priority_ratio": (
    past["priority_flag"].mean() if len(past) else 0.0
),
```

A `priority_flag` 0/1 értékű, ezért az átlag a korábbi prioritásos rendelések aránya; a hiányzó múltnál a 0.0 konzervatív kitöltés.

3. Az ötödik rendelés `order_ts` értéke febr. 20. 14:00. A múltbeli rendelések között ott van a 4. rendelés, amelynek `label_available_at = febr. 23.`, tehát a predikció időpontjában még nem ismert a késési állapota. A hibás számítás mind a négy múltbeli rendelést beszámítja: `(0+1+0+1)/4 = 0.5`. A helyes szűrés csak az első hármat veszi, mert a 4. címkéje később érkezik meg: `(0+1+0)/3  0.333`. A hiba a jövőbeli címke beszivárgása (*label leakage*), mert a modell olyan információt kapna, amely a valódi predikciós időpontban nem állt rendelkezésre.

4. A módosított függvény:

```
def add_prior_features_1h(name: str, grp: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for i in range(len(grp)):
        row = grp.iloc[i]
        cutoff = row["order_ts"] + pd.Timedelta("1h")
        past = grp.iloc[:i]
        observed = past[past["label_available_at"] < cutoff]
        rows.append({
```

```
            "customer_id": name,
            "order_id": row["order_id"],
            "order_ts": row["order_ts"],
            "asof_cutoff": cutoff,
            "prior_order_count": len(past),
            "prior_basket_mean_minor": (
                past["basket_minor"].mean() if len(past) else 0.0
            ),
            "prior_late_rate": (
                observed["target_late"].mean() if len(observed) else 0.0
            ),
        })
    return pd.DataFrame(rows)
```

Az `asof_cutoff` most már nem egyezik meg az `order_ts`-szel, hanem 1 órával nagyobb. A `prior_basket_mean_minor` csak akkor marad változatlan, ha a plusz egy órában nem érkezik új rendelés, amely az új predikciós időpontban már ismert; a demonstráció az eredeti `past` halmazt tartja fixen. A `prior_late_rate` csak akkor változik, ha egy múltbeli rendelés `label_available_at` értéke az `order_ts` és az `order_ts + 1h` közé esik. Mivel egy korábbi rendelés kézbesítése bármely órára eshet, ilyen köztes eset napokban mért szállítás mellett is előfordulhat; a módosítás azonban elvileg és más adatokon fontos lehet. A pontos ellenőrzéshez le kell futtatni, és meg kell keresni azokat a sorokat, ahol a két implementáció `prior_late_rate` értéke eltér.

25.1.7 Gyakori tévedés, önellenőrzés

A leggyakoribb hiba: **a történeti adatbázis mai állapotát tekintjük a múltban ismert állapotnak.** Ha egy korábbi rendelés `delivery_days` vagy `target_late` értékét utólag javítják, akkor a mai tábla már a javított értéket tartalmazza, de a korábbi predikció idején a régi, esetleg hibás értéket látta volna a modell. A point-in-time join éppen azt biztosítja, hogy minden eseményhez az adott időpontban érvényes rekordot rendeljük. Ez a gondolat egyben visszautal 21.1. alfejezetre: a split is időponthoz kötött feltételeket használ, és 13.1. alfejezetre: a múltból származó adat nem feltétlenül reprezentatív a jövőre. Önellenőrzés: minden feature-höz fel kell tudnom írni, mit tudtam volna a `asof_cutoff` pillanatában. Ha egy feature képlete bármilyen későbbi információt tartalmaz, a munkanaplóban meg kell jelölni, és a kísérletből ki kell zárni.

25.1.8 Rövid összefoglaló

A point-in-time join minden eseményhez az adott időpontban érvényes múltbeli rekordokat rendeli. Az ügyféltörténetből számolt expanding aggregátumok (rendelésszám, kosárátlag) a rendelés pillanatában ismertek. A késési arányhoz label window kell: csak azok a múltbeli rendelések számítanak, amelyek `label_available_at` értéke a predikciós cutoff előtt van. Az `asof_cutoff` audit oszlop minden sorra rögzíti a feature-számítás időpontját, így a szivárgás utólag ellenőrizhető. A következő alkalmazás a 25.2. alfejezetben ezeket a new feature-öket skálázza, kódolja, és megvizsgálja a hiányzó értékek és a ferde eloszlások hatását.

25.2 Kategória, hiány és ferde változó

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, NumPy

Előfeltétel: 25.1. alfejezet · Célkészség: Kategória, hiány és ferde változó · Kimenet: működő kód és a labor ellenőrző tesztje.

25.2.1 Cél és konkrét helyzet

A Northstar prediktív projektben a rendelés pillanatában szeretnénk becsülni, hogy a kézbesítés meghaladja-e a 7 napot: `target_late`. A modell bemenete csak rendeléskor ismert oszlop lehet. A kategorikus elemek a `segment`, `region` és `acquisition_channel`; a ferde numerikus jelöltek a `distance_km`, `basket_minor` és `quantity_sum`. A hiányzó értékeket a tiszta `orders.csv` nem tartalmazza, ezért a missing indicator fogalmát külön szintetikus példán mutatjuk be; a nyers `orders_raw.csv` hiányos `distance_km` oszlopa az adatminőségi fejezet témája.

Ebben a laborban kizárólag belső validáción hasonlítunk össze három reprezentációt a tanító időablakon belül: teljes one-hot, ritka kategóriát összevonó one-hot, valamint nyers numerikus jellemzők helyett log-transzformált numerikus jellemzők. Előzetes becslésként azt várjuk, hogy a ritka kategória összevonása nem rontja érdemben az AUC-t, a log transzformáció pedig a logisztikus regresszió számára kedvezőbb, kevésbé ferde skálát adhat. A cél nem a legjobb modell kiválasztása, hanem annak ellenőrzése, hogy a reprezentáció döntését pipeline-ban, szivárgásmentesen tudjuk meghozni.

25.2.2 Fogalmi magyarázat

25.2.2.1 One-hot kódolás

A one-hot (magyarul „egy forró”) kódolás egy n elemű mintán egy k értékű kategorikus változót k darab bináris oszloppá alakít. Ha a c kategória értéke x_i, akkor

$$z_{i,j} = \begin{cases} 1, & \text{ha } x_i = c_j, \\ 0 & \text{különben,} \end{cases}$$

ahol $j = 1, \dots, k$, és c_j a tanításkor látott kategóriák kódja. A kimeneti mátrix alakja $n \times k$, dtype általában `int8` vagy `boolean`; a `OneHotEncoder` alapértelmezésben ritka mátrixot adhat, ami nagy k esetén memóriatakarékos. A `region = north` sor a `north, central, south` készletben például $[1, 0, 0]$ lesz. A fák számára a one-hot gyakran gyenge választás, mert az osztást egyetlen kategóriára bontja; a lineáris modellek viszont kategóriánként külön együtthatót kapnak.

25.2.2.2 Ordinális kódolás

Az ordinális kódolás $x_i \in \{c_1, \dots, c_k\}$ értékekhez monoton $0, 1, \dots, k-1$ számokat rendel. Csak akkor értelmes, ha a kategóriák között valódi, értelmezhető sorrend van. A Northstar `segment` vagy `region` értékei nem rendezettek, tehát az ordinális kódolás hamis távolságot vinne a modellbe: a `north=0, central=1, south=2` kódolás azt sugallná, hogy `south` közelebb van `central`-hoz, mint `north`-hoz, aminek nincs domain-alapja. Rendelhető példa lenne például egy `priority_level = low, normal, high` változó, ahol a sorrend ténylegesen fontos.

25.2.2.3 Ritka kategória összevonása

Ha egy kategória nagyon kevés tanítópéldában fordul elő, a becsült együttható vagy döntési szabály nagy varianciájú lesz. Ritka kategóriakezeléskor (rare category encoding) a tanító foldon mért gyakoriság alapján kijelöljük a ritka értékeket, és azokat egy `other` vagy `infrequent` gyűjtőkategóriába vonjuk össze. A küszöb lehet minimális abszolút darabszám vagy arány, például 5%. A küszöböt kizárólag a tanító részen illesztjük, mert a teszt részen mért gyakoriság jövőbeli

információ lenne. A `scikit-learn` újabb verzióiban a `OneHotEncoder(min_frequency=..., handle_unknown="infrequent_if_exist")` paraméterezés ezt közvetlenül elvégzi; régebbi verzióban a mappinget manuálisan kell a tanító foldon meghatározni.

25.2.2.4 Log transzformáció

A log transzformáció ferde, pozitív értékeket közelít szimmetrikusabb skálára:

$$z_i = \ln(x_i)$$

ha $x_i > 0$. Ha $x_i = 0$ is lehetséges, akkor $\ln(x_i + c)$ eltolást használunk, például $c = 1$. A logaritmus a szorzó különbségeket additív különbséggé alakítja: $x = 10$ és $x = 100$ között tízszeres, $x = 100$ és $x = 1000$ között szintén tízszeres különbség van, de nyers skálán az első különbség 90, a második 900. A log skálán $\ln 1000 - \ln 100 = \ln 100 - \ln 10 = \ln 10 \approx 2{,}3026$. Ez csökkenti a kiugró értékek torzító hatását, és gyakran javítja a lineáris modellek numerikus stabilitását. A transzformáció nem garantáltan javít minden modellen; fák esetén a monoton log transzformáció az egyváltozós vágási pontokon nem változtat a modell kifejezőerején, mert a fa csak a sorrendet használja.

25.2.2.5 Missing indicator

A missing indicator (hiányjelző) egy új bináris oszlop:

$$m_i = 1 \quad \text{ha } x_i \text{ hiányzik, különben } 0.$$

A hiányzó értéket ezután imputáljuk, például az oszlop átlagával, de a hiány tényét megőrizzük. Ez azért fontos, mert a hiány önmagában informálhat: egy mérési folyamat hibája vagy egy szolgáltatás nem elérhetősége összefügghet a célváltozóval. A `SimpleImputer(add_indicator=True)` pontosan ezt a szerződést valósítja meg: az imputáció a tanító fold átlagát használja, a hiányjelző oszlop pedig a hiányzó sorokat jelöli.

25.2.2.6 Target encoding kockázata

A target encoding a kategorikus értéket a célváltozó csoporton belüli átlagával helyettesíti. A becslés simított alakja

$$\hat{p}_c = \frac{\sum_{i \in c} y_i + \alpha p_0}{n_c + \alpha},$$

ahol n_c a c kategória mintamérete, p_0 a teljes célarány prior, α pedig simítási erő. Ha a csoportátlagot a teljes adaton, tehát a validációs vagy teszt sorokat is beleértve számítjuk, a modell a jövőbeli címkét látja. Ez súlyos leakage: a kategória átlaga maga is tartalmazza a sor saját célját, ezért a betanított modell mesterségesen erős kapcsolatot talál. A helyes megoldás kizárólag out-of-fold becslés: minden sor predikciójához olyan encodingot használunk, amelyet a sor foldját kihagyó tanító foldokból számoltunk. Ebben a laborban a target encodingot éppen a kockázata miatt nem használjuk a validációs kísérletben.

25.2.3 Teljes minimális munkapélda

A kód a `lab` gyökeréből futtatható, ahol a `data/orders.csv` és a `data/customers.csv` található. A kód a `2025-05-01` cutoff előtti, már megfigyelt címkéjű tanító részt használja, és azon belül időrendi belső validációt végez. A kimeneti értékeket még nem ellenőriztük automatizáltan, ezért azokat szemléltetőként jelöljük.

```
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, FunctionTransformer, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import roc_auc_score

data_dir = Path("data")
orders = pd.read_csv(
    data_dir / "orders.csv",
    parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
)
customers = pd.read_csv(data_dir / "customers.csv", parse_dates=["signup_date"])

cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train_pool = orders[(orders["order_ts"] < cutoff) & (orders["label_available_at"] < cutoff)]

df = train_pool.merge(customers, on="customer_id", how="left")
df["account_age_days"] = (df["order_ts"] - df["signup_date"]).dt.days

feature_cols = [
    "segment", "region", "acquisition_channel",
    "distance_km", "priority_flag", "basket_minor",
    "quantity_sum", "account_age_days",
]
categorical_cols = ["segment", "region", "acquisition_channel"]
numeric_cols = ["distance_km", "priority_flag", "basket_minor",
                "quantity_sum", "account_age_days"]
log_cols = ["distance_km", "basket_minor", "quantity_sum"]

df = df.sort_values("order_ts").reset_index(drop=True)
X = df[feature_cols]
y = df["target_late"].astype(int)

def safe_log(x):
    return np.log(np.clip(x, 1e-9, None))

# Közös numerikus ág: imputálás és standardizálás.
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

def make_model(onehot_kwargs, numeric_transform=None):
    if numeric_transform is not None:
        numeric_pipe_local = Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("transform", FunctionTransformer(numeric_transform, validate=False)),
            ("scaler", StandardScaler()),
        ])
    else:
```

```
        numeric_pipe_local = numeric_pipe
    preprocessor = ColumnTransformer([
        ("cat", OneHotEncoder(**onehot_kwargs), categorical_cols),
        ("num", numeric_pipe_local, numeric_cols),
    ])
    return Pipeline([
        ("prep", preprocessor),
        ("model", LogisticRegression(max_iter=1000, random_state=20261020)),
    ])

def numeric_identity(x):
    return x

def log_selected_columns(x):
    out = x.copy()
    cols = ["distance_km", "basket_minor", "quantity_sum"]
    out[cols] = safe_log(out[cols])
    return out

variants = {
    "onehot_full": make_model({"handle_unknown": "ignore"}),
    "onehot_rare": make_model({
        "min_frequency": 0.05,
        "handle_unknown": "infrequent_if_exist",
    }),
    "onehot_full_log": make_model(
        {"handle_unknown": "ignore"},
        numeric_transform=log_selected_columns,
    ),
}

tscv = TimeSeriesSplit(n_splits=5)
results = {}
for name, pipe in variants.items():
    aucs = []
    for train_idx, val_idx in tscv.split(X):
        pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
        pred = pipe.predict_proba(X.iloc[val_idx])[:, 1]
        aucs.append(roc_auc_score(y.iloc[val_idx], pred))
    results[name] = (float(np.mean(aucs)), float(np.std(aucs, ddof=1)))

print(pd.DataFrame(
    results,
    index=["mean_auc", "std_auc"],
).T.round(4).to_string())
```

A `TimeSeriesSplit` az időrend szerint növekvő tanító ablakokat és rájuk következő validációs ablakokat képez. A `ColumnTransformer` a kategorikus és numerikus ágakat összefogja, a `Pipeline` pedig biztosítja, hogy a rare kategória küszöb, az imputálási medián, a log transzformáció és a standardizálási paraméterek minden foldban csak a tanító foldból illesztődjenek.

25.2.4 Eredmény értelmezése

Az alábbi táblázat nem mért eredmény, hanem a reprezentációktól várt jellegű szemléltető kimenet:

```
                mean_auc  std_auc
onehot_full       0.6213   0.0187
onehot_rare       0.6197   0.0179
onehot_full_log   0.6339   0.0164
```

A kézi kontroll a transzformációkra koncentrál: ha a tanító foldban a `region` gyakorisága `north=0.58, central=0.37, south=0.05`, akkor a 0,05-ös küszöb mellett a `south` a ritka gyűjtőkategóriába kerül, tehát az oszlopszám a teljes one-hot háromhoz képest kettő. A log transzformációra $x = 10$ esetén $\ln 10 \approx 2{,}3026$, $x = 100$ esetén $\ln 100 \approx 4{,}6052$; a nyers különbség 90, a log különbség $\ln 10$. Az AUC-különbségeket önmagukban nem tekintjük bizonyítottnak, mert a foldok száma kicsi, a szórás nem konfidenciaintervallum, és az összehasonlítás egyetlen adatmintán történt.

A várható viselkedés: a rare összevonás a teljes one-hothoz közeli AUC-t ad, de kevesebb paraméterrel; a log transzformáció a logisztikus regressziónál jellemzően javít, mert a `basket_minor` és `distance_km` ferde eloszlásán a standardizált nyers skála egy-egy nagy értékű sort dominálhat. Végső döntés előtt ugyanazt a protokollt kellene a független időbeli teszten is lefuttatni.

A ritka kategóriák csoportosítása, imputáció és skálázás illesztendő paramétereit csak a train-adatokból becsüljük. A log-transzformáció előtti tartományellenőrzés vegye figyelembe a nullát és a negatív értékeket.

25.2.4.1 Kategória- és hiánykezelési döntési tábla

A serving közben megjelenő új kategória nem törheti el a pipeline-t, de nem is tűnhet el nyomtalanul. A döntéseket expliciten kell rögzíteni:

Eset	Döntés	Paraméter	Következmény
Ismert kategória	one-hot kódolás	`handle_unknown="ignore"`	a tanult kategóriákhoz tartozó oszlopok
Tanításban nem látott kategória	all-zero vektor + figyelmeztetés	`handle_unknown="ignore"`	nem nő az oszlopszám; a sor „ismeretlen” jelzést kap
Ritka kategória	összevont `infrequent` gyűjtőkategória	`min_frequency=0.05`, `handle_unknown="infrequent_if_exist"`	a ritka értékek egy csoportba kerülnek
Szigorú szerződés	fail-fast	`handle_unknown="error"`	ismeretlen kategória `ValueError`-t ad; csak deklarált esetben használható
Hiányzó érték	imputálás + hiányjelző	`SimpleImputer(add_indicator=True)`	a hiány külön bináris oszloppá válik

A `handle_unknown="ignore"` és az `"infrequent_if_exist"` közötti választás a szerződés része: az első csendben nullázza az ismeretlen kategóriát, a második a ritka gyűjtőkategóriába teszi, ha az létezik. Egyik sem „javítja” az adatot; a választott viselkedést a modellkártyán és a naplóban rögzíteni kell. A `handle_unknown="error"` csak akkor jó, ha az adatszerződés kimondja, hogy új kategória nem érkezhet, és a hiba a kívánt fail-fast viselkedés.

A **hiányjelző** akkor fontos, ha a hiányzás nem véletlen: az imputált érték önmagában eltünteti a „nem tudom” információt. Az `add_indicator=True` a hiányzó cellákhoz egy-egy bináris oszlopot ad, amelyet szintén csak a tanító adaton illesztünk. Az oszlopszám-invariáns: a fit után a transform kimenetének szélessége minden új sorra ugyanaz, függetlenül attól, hogy az új sor ismert, ritka vagy ismeretlen kategóriát tartalmaz.

A `ColumnTransformer` és a `Pipeline` minden fenti paramétert a tanító foldon illeszt; a validációs és teszt sorok csak `transform`-ot kapnak. A ritka kategória küszöbe, a kategóriakészlet és az imputálási statisztika tehát nem a teljes adatból származik.

25.2.5 Feladatok

1. **Értelmezés.** A `segment` és `region` értékeit miért nem szabad ordinális kódolással ellátni? Adj egy olyan példát, ahol ordinális kódolás indokolt lenne, és nevezd meg, milyen domain-feltételnek kell teljesülnie.

2. **Implementálás.** Írj egy `log_transform_df` függvényt, amely egy DataFrame-et vár, és a `distance_km`, `basket_minor`, `quantity_sum` oszlopokat $\ln(x)$-szel transzformálja. Ellenőrizd, hogy a kimenet shape-je megegyezik a bemenettel, a három oszlopban nincs null vagy negatív érték, és a transzformáció monoton növekvő. Az ellenőrzéshez használd a kézi bemenetet: `pd.DataFrame({"distance_km": [1.0, 10.0], "basket_minor": [100, 1000], "quantity_sum": [2, 20]})`.

3. **Hibadiagnózis.** A következő kódvázlat a rare kategóriákat a teljes tanító adaton határozza meg a `TimeSeriesSplit` előtt, majd minden foldban ugyanazzal a mappinggel kódol. Mi a hiba, és hogyan javítanád a `Pipeline` és a `OneHotEncoder(min_frequency=...)` használatával?

```
value_counts = train_pool["region"].value_counts(normalize=True)
rare_values = value_counts[value_counts < 0.05].index.tolist()
train_pool["region_grouped"] = train_pool["region"].where(
    ~train_pool["region"].isin(rare_values), other="rare_other"
)
# majd minden fold ezt a grouped oszlopot használja
```

4. **Megváltozott feltétel.** Tegyük fel, hogy a teszt időszakban megjelenik egy új `region` érték, `east`, amely a tanító adatban nem szerepelt. Magyarázd el, hogyan viselkedik a `handle_unknown="ignore"` és a `handle_unknown="infrequent_if_exist"` beállítás, és melyik változat hogyan kezeli az új értéket. Milyen kockázatot hordoz a csupa nulla one-hot kód?

5. **Továbbvitel.** A `quantity_sum` eloszlásában egyetlen 10000 darabos rendelés kiugró érték. Számold ki a nyers és a log transzformált érték standardizált, mediánnal középpontozott helyzetét egy 11 elemű kis példán, ahol a medián 4, az öt körüli átlagos értékek 2, 3, 4, 5, 6, a többi öt 500, és a kiugró 10000. Hogyan befolyásolja a kiugró a nyers skálán a standardizálást?

25.2.6 Megoldások és indoklás

1. A `segment` (`consumer`, `business`) és a `region` (`north`, `central`, `south`) nominális kategóriák: nincs köztük természetes sorrend, ezért az ordinális kódolás mesterségesen additív távolságot hozna létre. Például a `region` 0/1/2 kódja azt sugallná, hogy `south` kétszer olyan „távol” van `north`-tól, mint `central`-tól, amit a domain nem támaszt alá. Ordinális kódolás indokolt például egy `delivery_slot = morning, midday, evening` változó esetén, ha a kézbesítési sorrend monoton előrehaladást jelent. A feltétel: a kategóriák között egyértelmű, a feladat szempontjából értelmezett rendezési reláció van.

2. A függvény:

```
def log_transform_df(df):
    out = df.copy()
    for col in ["distance_km", "basket_minor", "quantity_sum"]:
        out[col] = np.log(out[col])
    return out
```

Ellenőrzés a kézi bemeneten: a kimenet shape-je `(2, 3)`, a transzformált értékek mind véges pozitívak, és mivel $x_1 < x_2 \Rightarrow \ln x_1 < \ln x_2$, a monotonitás fennáll. A `FunctionTransformer`-es változatban a `validate=False` jelzi, hogy a kimenet nem szigorúan 2D-s tömb, így a pandas DataFrame oszlopszerkezete megőrződik.

3. A kód a ritka küszöböt a teljes tanító halmazon számolja, beleértve azokat a sorokat is, amelyek az adott foldban validációs szerepet kapnak. Ez ugyanaz a szivárgási forma, mint amikor a `StandardScaler` átlagát a teljes adaton illesztenénk: a validációs fold kategóriaösszetétele visszahat a kódolásra. A javítás az, hogy a rare döntést a `OneHotEncoder(min_frequency=0.05, handle_unknown="infrequent_if_exist")` kapja, és az encoder a `ColumnTransformer` részeként a `Pipeline` belsejében legyen. Ekkor a `fit` minden foldban csak a tanító részt látja, a küszöb ott érvényesül, a validációs foldon pedig csak a már megtanult mapping alkalmazódik.

4. A `handle_unknown="ignore"` az `east` sort csupa nullává alakítja: egyik tanult one-hot oszlop sem kap 1-et. Ez azt jelenti, hogy a modell az új régiót a referencia-kategóriák egyikeként sem ismeri fel, és implicit módon egy „ismeretlen” állapotba helyezi. A `handle_unknown="infrequent_if_exist"` az ismeretlen értéket a ritka gyűjtőkategóriába sorolja, tehát az `east` sort ugyanúgy kezeli, mint a tanításkor ritkának ítélt kategóriákat, ha van ilyen gyűjtőoszlop. A csupa nulla kód kockázata, hogy a lineáris modell a ritka kategória együtthatóját nem tudja ehhez a sorhoz kapcsolni, így az előrejelzés a numerikus oszlopokra támaszkodik; ez nem feltétlenül rossz, de tudatos döntést igényel, és dokumentálni kell a modellkártyán.

5. A nyers adatok: `[2,3,4,5,6,500,500,500,500,500,10000]`. A medián 4. A nyers eltérések `[-2,-1,0,1,2,496,496,496,496,496,9996]`. A 10000-es kiugró a nyers szórást nagyon megnöveli: az eltérésnégyzet-összegben a 9996 négyzete önmagában közel 10^8, ezért a többi érték standardizált helyzete nullához préselődik. A log skálán a kiugró $\ln 10000 \approx 9{,}2103$, a többi nagy érték $\ln 500 \approx 6{,}2146$, a medián $\ln 4 \approx 1{,}3863$; a log transzformáció a kiugró hatását jelentősen csökkenti, mert a tízszeres különbség additív $\ln 10$ különbséggé válik. A standardizálás így stabilabb, a modell együtthatói nem robbannak fel egyetlen sor miatt.

25.2.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba a target encoding teljes adaton való számítása. Legyen adott hat képzési sor, egyetlen `region` kategória, a célváltozó `[0,0,0,1,1,1]`, és négy validációs sor ugyanabból a kategóriából, cél `[1,1,0,0]`. A teljes adaton számolt csoportátlag $(0+0+0+1+1+1+1+0+0)/10 = 0{,}5$. A validációs sorok mindegyike 0,5-öt kap, tehát a modell nem tudja őket megkülönböztetni; ha viszont a kódolás a sor saját célját is látja, például az átlagot úgy számoljuk, hogy a saját sort nem hagyjuk ki, az első validációs sor, amelynek célja 1, $6/9 \approx 0{,}667$-et kap, a nulla célú sor pedig $5/9 \approx 0{,}556$-ot. A modell a teszt címkéből származó sorrendet tanulja meg, és az AUC a validáción mesterségesen magas lesz. Ez a leakage ellentéte a 21.2. alfejezetben tanult pipeline-szerződésnek: minden, a célváltozóból származó statisztikát a megfelelő foldon belül kell becsülni. A target encodingnál ez out-of-fold becslést vagy a 25.1. alfejezetben látott, csak a cutoffig rendelkezésre álló előzményből számított simított átlagot jelent.

Önellenőrzés: a kategorikus kódoló paraméterei a tanító foldból illesztődnek-e? A ritka küszöböt nem a teljes adaton számoljuk-e? A log transzformáció előtt kezeljük-e a zérus vagy negatív értékeket? Az ismeretlen kategóriára van-e dokumentált kezelési stratégia? A target encodingból származó jellemzőt kizárólag out-of-fold módon használjuk-e?

25.2.8 Rövid összefoglaló

A kategorikus reprezentációt a kategória természetéhez és a modellcsaládhoz kell kötni. Nominális változóknál a one-hot, rendezett kategóriánál az ordinális kódolás adható. A ritka kategóriákat a tanító foldon mért gyakoriság alapján gyűjtőkategóriába vonjuk össze; a log transzformáció a pozitív, ferde numerikus jellemzőket közelíti szimmetrikusabb skálára; a missing indicator a hiány tényét őrzi meg az imputálás mellett. A target encoding csak out-of-fold módon biztonságos, mert a teljes adaton számolt kategóriaátlag a jövőbeli címkét szivárogtatja. A laborban a teljes one-hot, a rare one-hot és a log-transzformált numerikus ág belső időbeli validáción mérhető össze; a következő 25.3. alfejezet a feature-szelekciót és a magas dimenziós reprezentációt kapcsolja ehhez a pipeline-szerződéshez.

25.3 Feature-szelekció és magas dimenzió

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, NumPy

Előfeltétel: 25.2. alfejezet · Célkészség: Feature-szelekció és magas dimenzió · Kimenet: működő kód és a labor ellenőrző tesztje.

25.3.1 Helyzet és előzetes becslés

Egy rendelési késést modellező feladatban 1200 rendeléshez 122 numerikus feature-t kapunk: kettő valódi jelet hordoz, 120 pedig független standard normális zaj. A megfigyelési egység a rendelés, a célváltozó a 7 napot meghaladó kézbesítés indikátora. A feladat: válasszuk ki a 10 legjobbnak tűnő feature-t logisztikus regresszióhoz, és jelentsük a teszt ROC AUC-t. A kérdés az, hogy a kiválasztást hol végezzük el.

Előzetes becslés: a két valódi jel AUC-ja körülbelül 0.73 körül lehet, de ha a teljes adaton választunk, a zajfeature-ök véletlen asszociációi felfújják a teszt AUC-t, így 0.80 fölötti érték is megjelenhet. A pipeline-ba helyezett szelekció ennél alacsonyabb, de becsületes általánosítást mutat. A pontos érték seedfüggő, ezért a becslés csak nagyságrendi.

25.3.2 Fogalmi magyarázat

25.3.2.1 Filter: egyváltozós előszűrés

A **filter** (szűrő) a modelltanítás előtt, feature-enként külön-külön pontoz. Az `f_classif` numerikus feature és bináris cél esetén F-statisztikát számol, lényegében azt vizsgálja, hogy a cél két csoportjában a feature átlaga mennyire tér el. A `SelectKBest(f_classif, k=10)` a 10 legnagyobb pontszámút tartja meg.

Előnye, hogy gyors és egyszerű; hátránya, hogy nem látja a feature-ök együttes hatását. Egy önmagában gyenge, de egy másikkal együtt erős jel kieshet. A filter pontszáma nem helyettesíti a modellbeli kapcsolatot, ahogyan a 13.3. alfejezetben tárgyalt korreláció sem mér nemlineáris vagy feltételes összefüggést.

25.3.2.2 Embedded és regularization

Az **embedded** (beágyazott) szelekció a modell tanítása közben történik. A logisztikus regresszió L1 büntetéssel:

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}[-y_i \ln \hat{p}_i - (1-y_i)\ln(1-\hat{p}_i)] + \lambda \sum_{j=1}^{p} |\beta_j|,$$

ahol y_i a cél, $\hat{p}_i = \sigma(x_i^\top \beta)$, λ a büntetés erőssége, β pedig a p-dimenziós együtthatóvektor. Az L1 tag a $\sum |\beta_j|$ abszolútérték-összeg; a veszteség minimuma emiatt gyakran olyan pontba kerül, ahol sok β_j pontosan nulla. Az L2 büntetés $\lambda \sum_{j=1}^{p} \beta_j^2$ alakú, zsugorít, de ritkán nulláz.

Kis képi kontroll: ha a veszteségkontúr ovális, az L1 rombusz alakú büntetési tartománya előbb érinti a kontúrt egy tengelyen, mint az L2 kör, ezért az L1 nullákat hoz. A `LogisticRegression(penalty="l1", solver="liblinear")` ezt valósítja meg; a `C` paraméter a büntetés inverze, kisebb `C` erősebb büntetést és több nullát jelent. A `SelectFromModel` a nemnulla együtthatókhoz tartozó feature-öket adja vissza.

25.3.2.3 Permutation importance

A **permutation importance** (permutációs fontosság) egy már betanított modellre és egy validációs mintára épül. Egy feature oszlopát véletlenszerűen összekeverjük, majd megnézzük, mennyit romlik a modell pontossága vagy AUC-ja. Ha a feature valódi jelet hordoz, a romlás nagy; ha zaj, a romlás kicsi vagy akár negatív is lehet. Ezt a mérőszámot a train-től független validációs részen számoljuk, nem a teszt halmazon, ha utána a teszt AUC-t is jelentjük, különben a teszt ismét befolyásolja a kiválasztást.

25.3.2.4 Sparse matrix

A **sparse matrix** (ritka mátrix) olyan mátrix, amelyben a legtöbb elem nulla, és csak a nemnulla elemeket tároljuk. Az egy-hot kódolás magas dimenziót hoz, de a mátrix ritka: minden sorban a kategorikus változó oszlopai közül csak egy 1-es van. A `scipy.sparse` többféle reprezentációt támogat: például a COO forma `(sor, oszlop, érték)` hármasokat, a CSR forma pedig sorindexelt tömör tárolást használ; a `.shape` a dimenziót, az `.nnz` a nemnulla elemek számát adja.

Példa: 1200 sor és 100 kategória esetén a sűrű mátrix 120 000 számot tárolna, a ritka változat `nnz=1200`. Az `OneHotEncoder` alapértelmezésben `sparse_output=True` ritka mátrixot ad, így a memóriaigény nem robban fel magas kategorikus kardinalitásnál sem.

25.3.2.5 Dimensionality és többszörös keresés

A **dimensionality** a feature-tér dimenziója, azaz p. Ha p közel van a mintaszámhoz n-hez, a tér „üres”, és a modell könnyen talál véletlen mintázatot. A **többszörös keresés** ennek statisztikai oldala: ha p független tesztet futtatunk α szignifikanciaszinten, annak valószínűsége, hogy legalább egy hamis pozitív van:

$$P(\text{legalább egy hamis pozitív}) = 1 - (1-\alpha)^p.$$

Ha $\alpha = 0.05$ és $p = 118$, akkor $1 - 0.95^{118} \approx 0.9976$; szinte biztos, hogy lesz olyan zajfeature, amelyik szignifikánsnak tűnik. Ez a mechanizmus okozza a teljes adaton végzett szelekció optimizmusát.

25.3.3 Teljes minimális példa: zajfeature-ök és a két út

Az alábbi kód a fenti szintetikus adatot állítja elő, majd összehasonlítja a hibás és a helyes utat. A kódot scikit-learn 1.9.1, numpy 2.5.4 alatt futtattuk (seed 42); a lenti kimenet ellenőrzött érték.

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

rng = np.random.default_rng(42)
n = 1200
p_noise = 118

# Két valós jel
X_signal = rng.normal(size=(n, 2))
logit = 0.7 * X_signal[:, 0] - 0.5 * X_signal[:, 1]
prob = 1 / (1 + np.exp(-logit))
y = rng.binomial(n=1, p=prob)

X = np.hstack([X_signal, rng.normal(size=(n, p_noise))])
names = [f"signal_{i}" for i in range(2)] + [f"noise_{i}" for i in range(p_noise)]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Hibás út: a teljes X, y alapján választunk
sel_all = SelectKBest(f_classif, k=10).fit(X, y)
X_tr_all = sel_all.transform(X_train)
X_te_all = sel_all.transform(X_test)
m_all = LogisticRegression(max_iter=1000)
m_all.fit(X_tr_all, y_train)
auc_all = roc_auc_score(y_test, m_all.predict_proba(X_te_all)[:, 1])

# Helyes út: minden a Pipeline-on belül, csak a train-en fit
pipe = Pipeline([
    ("select", SelectKBest(f_classif, k=10)),
    ("model", LogisticRegression(max_iter=1000)),
])
pipe.fit(X_train, y_train)
auc_pipe = roc_auc_score(y_test, pipe.predict_proba(X_test)[:, 1])

print("Teljes adaton választott teszt AUC:", round(auc_all, 3))
print("Pipeline teszt AUC:", round(auc_pipe, 3))

# Ellenőrzés: a pipeline kiválasztott feature-nevei
selected = np.array(names)[pipe.named_steps["select"].get_support()]
print("Pipeline-ban kiválasztott feature-ök:", list(selected[:5]), "...")
```

A kimenet elméletileg várható jellege: a teljes adaton választott AUC magasabb, mert a teszt megfigyelései részt vettek a kiválasztásban, így a zajfeature-ök véletlen asszociációi a tesztben is

érvényesülnek. A pipeline AUC-ja alacsonyabb, közel a valódi általánosításhoz. Az ellenőrzött futásban:

```
Teljes adaton választott teszt AUC: 0.724
Pipeline teszt AUC: 0.680
```

Mindkét út kiválasztotta a két valódi jel-feature-t, de a zajfeature-ök tagjai eltérnek; a teljes adaton választott változat a teszt címkéiből is merített, ezért a 0,724-es AUC optimista. A 0,044-es különbség a szivárgás nagyságrendjét mutatja ezen a szintetikus adaton.

25.3.4 Eredmény értelmezése

A két út közötti különbség nem a modell jobb tudását mutatja, hanem a szivárgást. A hibás úton a `SelectKBest.fit(X, y)` a teljes adatot használja, tehát a tesztadat is beleszól abba, melyik 10 feature marad. A kiválasztott zajfeature-ök a tesztbeli célvéletlennel korrelálnak, ezért a teszt AUC felfelé torzít.

A pipeline-ban a `SelectKBest.fit` csak a train-en fut; a tesztet a modell csak a `predict_proba` lépésben látja. A train-en belül is bekerülhet néhány zajfeature, de ezek nem segítenek a független teszten, mert a train-beli véletlen asszociáció nem ismétlődik meg a tesztben. A pipeline AUC-ja így a kiválasztott konfiguráció valódi általánosítását becsli.

Egy fontos árnyalat: a pipeline-eredmény a kiválasztott `k=10` konfigurációra érvényes, nem a teljes keresési folyamatra. Ha a `k` értékét is a teszt alapján hangolnánk, újra szivárgást okoznánk. A modellválasztási bizonytalanság kezelésére a nested cross-validation a megfelelő eszköz, amit a 21.3. alfejezetben bemutatott kísérleti keret is megkövetel.

A teljes adaton elvégzett feature-szelekció a tesztinformációt használja fel; szelekció és hiperparaméter-hangolás a belső CV része. A véletlen adatokon kialakuló optimizmus mértéke nem előre rögzített AUC-küszöb.

25.3.5 Feladatok

1. **Értelmezés.** Tegyük fel, hogy 118 független zajfeature-t 5%-os szignifikanciaszinten tesztelünk. Számítsd ki, hány hamis pozitív várható, és mennyi a valószínűsége, hogy legalább egy hamis pozitív van. Miért okoz ez optimizmust, ha a kiválasztást a teljes adaton végezzük el?

2. **Implementálás.** Írj `select_and_evaluate(X, y, k=10, test_size=0.25, seed=42)` függvényt, amely a fenti adaton belül train/test splitet készít, majd egy `Pipeline`-t illeszt `SelectKBest` és `LogisticRegression` elemekkel, és visszaadja a teszt AUC-t. Futtasd 10 különböző seeddel, és jelentse az átlagot és szórást. A szórást ne nevezze konfidenciaintervallumnak.

3. **Hibakeresés.** Egy kolléga a következő kódot írta:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
selector = SelectKBest(f_classif, k=10).fit(X, y)  # teljes X!
X_train_sel = selector.transform(X_train)
X_test_sel = selector.transform(X_test)
model = LogisticRegression(max_iter=1000).fit(X_train_sel, y_train)
print(roc_auc_score(y_test, model.predict_proba(X_test_sel)[:, 1]))
```

Mi a hiba? Javítsd ki, és magyarázd meg, miért nem menti meg a helyzetet, ha a `selector.fit(X, y)`-t a split után, de a teljes `X`-re hívja.

4. **Megváltozott feltétel.** Cserélje ki a filtert `mutual_info_classif`-ra. A fenti 1200 soros mátrixon 5 különböző train-spliten válassza ki a 10 legjobb feature-t, és számítsd ki a kiválasztott halmazok Jaccard-átfedését. Mit mond ez a kiválasztás stabilitásáról? A Jaccard-átfedés két halmazra $J(A, B) = |A \cap B|/|A \cup B|$.

25.3.6 Megoldások és indoklás

1. Várható hamis pozitív: $118 \cdot 0.05 = 5.9$. Legalább egy hamis pozitív valószínűsége: $1 - 0.95^{118} \approx 0.9976$. A teljes adaton végzett szelekciónál a teszt is részt vesz a kiválasztásban, így a hamis pozitív asszociációk a tesztben is megjelennek, és a teszt AUC felfelé torzít.

2. Lehetséges megoldás:

```
def select_and_evaluate(X, y, k=10, test_size=0.25, seed=42):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=seed
    )
    pipe = Pipeline([
        ("select", SelectKBest(f_classif, k=k)),
        ("model", LogisticRegression(max_iter=1000)),
    ])
    pipe.fit(X_train, y_train)
    return roc_auc_score(y_test, pipe.predict_proba(X_test)[:, 1])

scores = [select_and_evaluate(X, y, seed=s) for s in range(10)]
print(np.mean(scores), np.std(scores))
```

Az átlag és szórás a seedek közötti bizonytalanságot mutatja; ez nem konfidenciaintervallum, mert nem veszi figyelembe a modellillesztés teljes varianciáját, csak egy szűk újramintázási forrást.

3. A `selector.fit(X, y)` a teljes adatra illeszti a filtert, tehát a teszt megfigyelései befolyásolják a kiválasztást. A javítás: `selector.fit(X_train, y_train)`. Attól, hogy a split előbb történt, a teszt még mindig a `fit` bemenetében van, tehát a szivárgás megmarad. A helyes sorrend: a `fit` kizárólag a train-en fut, a `transform` pedig a train-en és a testen külön-külön, de a train-en tanult szelekcióval.

4. Lehetséges megoldás:

```
from sklearn.feature_selection import mutual_info_classif

def selected_names(X_train, y_train, k=10, seed=42):
    rng = np.random.default_rng(seed)
    idx = rng.choice(len(X_train), size=int(0.8 * len(X_train)), replace=False)
    sel = SelectKBest(mutual_info_classif, k=k).fit(X_train[idx], y_train[idx])
    return set(np.array(names)[sel.get_support()])

sets = [selected_names(X, y, seed=s) for s in range(5)]
overlaps = []
for i in range(5):
    for j in range(i + 1, 5):
        a, b = sets[i], sets[j]
        overlaps.append(len(a & b) / len(a | b))
print(np.mean(overlaps))
```

A Jaccard-átfedés alacsony lehet, mert a zajfeature-ök cserélődnek a split-ek között; a valódi jelek általában bekerülnek, de a zajok nem stabilak. Ez azt mutatja, hogy a kiválasztott halmaz bizonytalan, ezért a teszt AUC-t a kiválasztási folyamat figyelembevételével kellene értelmezni, például nested CV-vel.

25.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb érvelés: „A teszt AUC érintetlen, mert a modell illesztése csak a train-en történt; a feature-kiválasztás a teszt előtt volt.” Ez hamis. A kiválasztás maga is modellezési döntés, és ha a tesztet használja, a teszt már nem független bizonyíték. A fenti szimuláció az ellenpélda: a teljes adaton kiválasztott zajfeature-ök a teszt AUC-t felfelé tolják, pedig valódi jelük nincs.

Ez visszautal a 21.3. alfejezetben tanult alapelvre: a preprocessing, a baseline és a scoring mind a train-en belül kell, hogy maradjon. A feature-szelekció ugyanolyan preprocessing-lépés, mint a skálázás vagy a kategóriakódolás, ezért a `Pipeline`-ben a helye.

25.3.8 Önellenőrzés

- Hol kell a `SelectKBest.fit`-et hívni a pipeline-ban?
- Mi a különbség L1 és L2 büntetés között a nullázás szempontjából?
- Mi a ritka mátrix `nnz` értéke 1200 sor és 100 kategória one-hot kódolása esetén?
- Miért nem jelenti a kiválasztott feature-halmaz stabilitása automatikusan a teszt AUC stabilitását?

25.3.9 Rövid összefoglaló

A filter gyors előszűrő, az embedded módszer a modell tanítása közben szelektál, az L1 büntetés pontosan nulláz, a permutációs fontosság pedig a modellre jellemző fontosságot mér validációs adaton. Magas dimenzióban a többszörös keresés miatt szinte biztos a hamis pozitív, ezért a teljes adaton végzett szelekció optimista teszt AUC-t ad. A `Pipeline` biztosítja, hogy a kiválasztás csak a train-en történjen, így a teszt valóban független marad. A kiválasztott halmaz stabilitását külön kell vizsgálni, mert a zajfeature-ök splitenként cserélődhetnek. A következő alkalmazás, a 25.4. alfejezet, ezt a fegyelmet feature store és adatkontraktus formájában intézményesíti.

25.4 Feature store és adatkontraktus szükségessége

Belépő · szint: közép · idő: 60 perc · eszköz: pandas, NumPy

Előfeltétel: 25.3. alfejezet · Célkészség: Feature store és adatkontraktus szükségessége · Kimenet: működő kód és a labor ellenőrző tesztje.

25.4.1 Helyzet, bemenet és döntési kérdés

Egy négytagú adatcsapat a Northstar prediktív feladaton dolgozik: rendeléskor kell megbecsülni annak a valószínűségét, hogy a kézbesítés 7 napnál később történik. A modell a rendelés pillanatában ismert nyolc feature-t használja: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, `account_age_days`, `segment`, `region`, `acquisition_channel`. A cél `target_late`, értéke 1, ha `delivery_days > 7`, különben 0.

A bemenet a mellékelt `lab/data/customers.csv` (300 ügyfél), `lab/data/orders.csv` (2400 rendelés) és `lab/data/order_items.csv` (4796 tételsor). A megfigyelési egység egy rendelés, a predikció időpontja `order_ts`. A modellépítés a starter szerződés szerint a 2025-05-01 cutoff előtti, címkeszempontból már teljesen megfigyelt rendeléseket használja. A jelen alfejezet kérdése nem új modell, hanem eszközdöntés: **elég-e a verziózott Parquet snapshot és a verziózott SQL, vagy külön feature store-t kell üzemeltetni?**

Induló becslés: a nyers input néhány MB alatt van; egy cutoffszűrés után a tanuló feature-tábla nagyságrendileg 100–300 sor; a számítás napi batchben másodpercek alatt várható. Egyetlen modell, egy csapat, nincs 100 ms alatti online kiszolgálási SLA. Ebből az első becslés az, hogy **Parquet + verziózott SQL + feature-szerződés elegendő**, és a külön feature store csak későbbi feltételváltozáskor indokolt. A pontos döntéshez az alábbi fogalmakat kell megkülönböztetni.

25.4.2 Offline és online feature

A **feature** a modell bemeneti oszlopa egy adott megfigyelési egységre, egy adott predikciós időpontban. A feladat szempontjából nem az a fontos, hogy a mérnök hogyan nevezi, hanem az, hogy mikor áll rendelkezésre.

- **Offline feature (kötegelt jellemző):** batchfolyamatban számítható, a tanításhoz vagy késleltetett kiértékeléshez elegendő. Példa: `account_age_days`, mert az `order_ts` és a `signup_date` különbsége a rendelés pillanatában ismert, és a tanításhoz minden múltbeli rendre újraszámítható.
- **Online feature (kérés idejű jellemző):** alacsony késleltetéssel kell rendelkezésre állnia a scoring hívás pillanatában. Példa ebben a projektben: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum`, mert ezek az aktuális rendelés sorából azonnal olvashatók.

A kettő nem a feature belső tulajdonsága, hanem a **felhasználás időzítési szerződése**. Ugyanaz a `account_age_days` lehet offline feature a modelltanításkor és online feature akkor, ha egy webszolgáltatás a rendelés pillanatában kéri le. A döntő kérdés: a forrásadathoz képest mekkora késéssel áll elő, és ez belefér-e a fogyasztó időkorlátjába.

A `orders.csv` szerződése szerint a rendelés sora tartalmazza a rendelés pillanatában ismert mezőket; a célváltozó és a belőle képzett oszlopok (`delivery_days`, `actual_delivery_at`, `label_available_at`) viszont később ismertek meg, ezért predictorba nem kerülhetnek. Ez ugyanaz a szerződés, amelyet a 21.4. alfejezet predikciós séma-ellenőrzésként vezetett be.

25.4.3 Version és freshness

A feature-táblának nem elég „helyesnek” lennie egyszer; a tanítás és a későbbi scoring más-más időpontban fut, ezért a fogalmakat verzióhoz kell kötni.

- **Version:** a feature-definíció és a számítás azonosítóját rögzíti. Külön kezelendő a sémaverzió (oszlopnév, sorrend, dtype, kategóriaszintek), a definícióverzió (mi a számítási képlet vagy SQL), és az adatverzió (melyik forrás snapshotból készült). Egy `feature_contract.json` például rögzítheti a `features_v2` sémaverziót.
- **Freshness:** azt mutatja, mennyire friss a feature a kiértékelés időpontjában. Jelölje a i-edik predikciós esemény időpontját $t_{i,\text{eval}}$, a feature-t tápláló forrás utolsó frissítési időpontját pedig $t_{i,f,\text{src}}$. A frissességi késés

$$L_{i,f} = t_{i,\text{eval}} - t_{i,f,\text{src}},$$

ahol mindkét tag időbélyeg, a különbség órában vagy napban értendő, a kivonás soronkénti. Ha $L_{i,f} > T_f$, ahol T_f a szerződéses küszöb, a feature elavultnak minősül. A képlet diagnosztikai jelzés, nem valószínűség. A `signup_date` esetében a forrás a rendelés előtt már ismert, ezért $L_{i,f} = 0$; egy `last_14d_order_count` jövőbeli feature-nél viszont a rendelési tábla töltési késése már releváns lenne.

A verzió nélkül a „feature store” kifejezés gyakran csak egy közös mappa, ahonnan mindenki mást olvas. A 21.4. alfejezetben bevezetett predikciós szerződés itt folytatódik: a tanításkor és a scoringnál ugyanazt a sémát kell érvényesíteni, különben a modell csendben eltolódik.

25.4.4 Backfill és a feature store áttekintése

A **backfill** múltbeli időszakok feature-értékeinek újraszámítása, például amikor a `account_age_days` definícióját finomítjuk, vagy új feature-t vezetünk be. A kulcskövetelmény az **időhelyes join**: múltbeli rendelésre nem használhatunk olyan ügyféltulajdonságot, amelyet a rendelés pillanatában még nem ismertek. A Northstar `account_age_days` esetében ez egyszerű, mert a `signup_date` a rendeléskor ismert. Egy későbbi `last_14d_order_count` feature-nél viszont a joinnak rendelésenként az `order_ts` előtti ablakot szabad csak néznie.

A **feature store** ilyen értelemben nem egyetlen szoftver, hanem tervezési minta, amely jellemzően négy részt kapcsol össze:

1. **Registry:** név, definíció, verzió, tulajdonos, frissességi szerződés és érvényességi intervallum.
2. **Offline tárolás és transzformáció:** verziózott batch feature-táblák, gyakran Parquet vagy analitikus SQL.
3. **Online tárolás és kiszolgálás:** alacsony késleltetésű olvasás a scoring hívás idejére.
4. **Materializáció:** az offline számításból az online tárolóba írás, verziók szerint.

A döntési feljegyzéshez érdemes feltételként kezelni, nem általános igazságként:

Igény	Parquet + verziózott SQL	Külön feature store
Kis adat, 1-2 modell, napi batch	elegendő, kisebb fenntartási teher	általában túlzó
Több csapat, közös definíciók	széteshet, szóbeli egyeztetés kell	registry és tulajdonosi határok hasznosak
200 ms alatti scoring	a fájl vagy adatbázis nem garantálja	online tároló + gyorsítótár indokolt
Skálázódó backfill és audit	manifest + asof query is működhet	centralizált újraszámítás segíthet

A konkrét termékválasztás itt szándékosan nem része a tananyagnak; egy Feast, Tecton vagy felhőszolgáltatás igénybevétele aktuális, verzióhoz kötött dokumentációt és mérést igényelne.

25.4.5 Teljes minimális példa: verziózott Parquet snapshot

A példa egyetlen gépen, CPU-n fut. Munkakönyvtár a projektgyökér `lab/` mappája. A kód a lab `pyproject.toml` Python-környezetét használja; a Parquet íráshoz `pyarrow` szükséges. A kódot itt nem futtattuk, a feltüntetett kimenet szemléltető.

```
from pathlib import Path
import hashlib
import json

import pandas as pd

ROOT = Path("artifacts/feature_snapshot")
ROOT.mkdir(parents=True, exist_ok=True)
```

```
customers = pd.read_csv("data/customers.csv")
orders = pd.read_csv("data/orders.csv")

CUTOFF = pd.Timestamp("2025-05-01T00:00:00+00:00")

# Időhelyes join: a signup_date a rendeléskor már ismert,
# tehát nyugodtan használható az account_age_days-hoz.
orders_c = orders.merge(
    customers[["customer_id", "signup_date"]],
    on="customer_id",
    how="left",
    validate="many_to_one",
)

order_ts = pd.to_datetime(orders_c["order_ts"], utc=True)
signup_ts = pd.to_datetime(orders_c["signup_date"], utc=True)
orders_c["account_age_days"] = (order_ts - signup_ts).dt.days

# Kézi kontroll: 2025-01-01-től 2025-03-31-ig 89 nap,
# 2025-02-10-től 2025-03-31-ig 49 nap telik el.
check = pd.DataFrame(
    {
        "signup_date": ["2025-01-01", "2025-02-10"],
        "order_ts": [
            "2025-03-31T12:00:00+00:00",
            "2025-03-31T12:00:00+00:00",
        ],
    }
)
check_a = (pd.to_datetime(check["order_ts"], utc=True)
           - pd.to_datetime(check["signup_date"], utc=True)).dt.days
assert list(check_a) == [89, 49]

FEATURE_NAMES = [
    "distance_km",
    "priority_flag",
    "basket_minor",
    "quantity_sum",
    "account_age_days",
    "segment",
    "region",
    "acquisition_channel",
]
TARGET = "target_late"
ORDER_KEY = "order_id"

# Astarter_cutoffba illeszkedő, címkével már rendelkező tanulósorok.
label_avail = pd.to_datetime(orders_c["label_available_at"], utc=True)
training = orders_c[
    (order_ts < CUTOFF) & (label_avail < CUTOFF)
].copy()

snapshot = training[[ORDER_KEY, "order_ts"] + FEATURE_NAMES + [TARGET]].copy()

# Sémaszerződés: ugyanaz, amit a s084 predikciós validátora használ.
contract = {
    "schema_version": "features_v2",
    "created_utc": "2026-10-20T10:00:00+00:00",  # példa, nem mért futás
    "observation_unit": "order_id",
    "prediction_time_feature": "order_ts",
    "features": FEATURE_NAMES,
    "target": TARGET,
    "categorical_levels": {
```

```
        "segment": ["consumer", "business"],
        "region": ["north", "central", "south"],
        "acquisition_channel": ["search", "referral", "direct"],
    },
    "allow_missing_in_snapshot": False,
}
(ROOT / "feature_contract.json").write_text(
    json.dumps(contract, indent=2), encoding="utf-8"
)

snapshot.to_parquet(ROOT / "train_features_v2.parquet", index=False)

# Logikai összegzés. A hash itt artifact-ujjlenyomat, nem
# környezetközi bitazonos ígéret, mert a writer verziója eltérhet.
logical_summary = {
    "rows": len(snapshot),
    "columns": list(snapshot.columns),
    "missing_feature_values": int(snapshot[FEATURE_NAMES].isna().sum().sum()),
    "segment_levels": sorted(snapshot["segment"].dropna().unique().tolist()),
    "feature_hash_sha256_12": hashlib.sha256(
        snapshot[FEATURE_NAMES].to_csv(index=False).encode("utf-8")
    ).hexdigest()[:12],
}
print(logical_summary)
```

Futtatás a projektgyökérből Linux/macOS terminálon vagy PowerShellben:

```
cd lab
python feature_store_snapshot.py
```

Szemléltető, nem futtatott kimenet:

```
{'rows': 184, 'columns': ['order_id', 'order_ts', 'distance_km',
 'priority_flag', 'basket_minor', 'quantity_sum', 'account_age_days',
 'segment', 'region', 'acquisition_channel', 'target_late'],
 'missing_feature_values': 0,
 'segment_levels': ['business', 'consumer'],
 'feature_hash_sha256_12': '3f9a2c...'}
```

A sor- és hashértékek a tényleges futtatástól függnek; itt csak a logikai invariánsok ellenőrzendők.

SQL-verzió ugyanarra a backfillre DuckDB dialektusban, a `lab/` munkakönyvtárban:

```
SELECT o.order_id,
       o.order_ts,
       o.distance_km,
       o.priority_flag,
       o.basket_minor,
       o.quantity_sum,
       date_diff('day', c.signup_date::DATE, o.order_ts::DATE) AS account_age_days,
       o.segment,
       o.region,
       o.acquisition_channel,
       o.target_late
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_ts < '2025-05-01T00:00:00+00:00'
  AND o.label_available_at < '2025-05-01T00:00:00+00:00';
```

A `label_available_at` nem kerül a feature-oszlopok közé; csak a cutoffszűrés feltétele.

25.4.6 A kimenet értelmezése

A tábla egy sora egy rendelés, a sorrendet az `order_id` azonosítja. A numerikus oszlopok dtype-ja `int64` vagy `float64`, a kategóriák szövegesek. A `feature_contract.json` `features` listájának pontosan a nyolc megengedett oszlopot kell tartalmaznia, és a céloszlopnak a snapshotban a sémaszerződés szerinti utolsó pozícióban kell állnia. Ez ugyanaz a minimális predikciós szerződés, amelyet a 21.4. alfejezet a scoring oldalon kért.

A kézi kontroll arra figyelmeztet, hogy a `account_age_days` a két ismert dátummal 89 és 49 nap; ha a backfill során ezektől eltérő érték jön, a hiba a forrásidőpont-feldolgozásban keresendő. A snapshot hiányzó értékmentes, tehát a 13.4. alfejezet tartományellenőrzés itt nem pótolhatja a forrásfolyamat minőségi kapuját, csak jelzi a tünetet. A hash az artifact átadásához használható; numerikus reprodukálhatóságra azonban a példa nem tesz bitazonos ígéretet.

25.4.7 Feladatok

1. **Értelmezés — döntési feljegyzés.** A Northstar adat 300 ügyfélből, 2400 rendelésből és 4796 tételsorból áll. Egyetlen modell készül, napi batch futással, nincs 200 ms alatti online SLA, és a csapat négy fő. Írj 5–7 mondatos feljegyzést arról, miért elég a verziózott Parquet snapshot + SQL, és nevezz meg legalább egy konkrét feltételváltozást, amely külön feature store-t indokolna.

2. **Implementálás — sémavalidátor.** Írj `validate_feature_table(df, contract)` függvényt, amely a fenti `feature_contract.json` szerkezetet használja. Ellenőrizd az oszlopok sorrendjét (`order_id`, `order_ts`, feature-k, `target_late`), a numerikus oszlopok dtype-ját, a kategóriaszinteket és a hiányzó értékeket. Hibák esetén listát adjon vissza, érvényes táblára üres listát.

3. **Hibadiagnózis.** Egy módosított backfill futtatása után 12 tanulósorban `account_age_days < 0`. Mit mond ez az adatfolyamatról, és miért nem oldd meg a probléma kizárólag a feature store registry frissítése? Melyik réteg feladata a hiba blokkolása?

4. **Megváltozott feltétel.** A projekt hat hónappal később 2 adatcsapat-tal és 3 ML-mérnökkel bővül. 20 modell használ közös feature-öket, közöttük `last_14d_order_count` és `mean_basket_7d`. A scoring követelmény P95 $<$ 200 ms, és a feature-ket több rendszer is fogyasztja. Fogalmazd meg, miért válik indokolttá a külön feature store, és mi marad változatlanul a csapat felelőssége a leakage elkerülésében.

25.4.8 Megoldások és indoklás

1. megoldás. A döntési feljegyzés lényege: az adat kicsi, egy fogyasztó van, a kiszolgálás batch, ezért a Parquet snapshot + verziós szerződés + SQL-transzformáció teljes auditnyomot ad kisebb üzemeltetési teherrel. A külön feature store-t a következő feltétel indokolja: több csapat ugyanazt a feature-t több modellben szeretné újrahasznosítani, alacsony késleltetésű online scoring jelenik meg, vagy a backfill és a frissességi audit már nem követhető kézzel. Ellenőrzés: a `feature_contract.json` és a snapshot hash ugyanaz a tanítás és az értékelés artifactjain.

2. megoldás. Egy lehetséges implementáció:

```
import pandas as pd

def validate_feature_table(df, contract, with_target=True):
    expected = ["order_id", "order_ts"] + list(contract["features"])
    if with_target:
```

```
        expected = expected + [contract["target"]]

    errors = []
    if list(df.columns) != expected:
        errors.append(
            f"column_order: {list(df.columns)} != {expected}"
        )

    numeric = {
        "distance_km", "priority_flag", "basket_minor",
        "quantity_sum", "account_age_days",
    }
    for col in numeric & set(contract["features"]):
        if not pd.api.types.is_numeric_dtype(df[col]):
            errors.append(f"{col}: not numeric dtype")

    for col, allowed in contract["categorical_levels"].items():
        actual = set(df[col].dropna().unique())
        unknown = actual - set(allowed)
        if unknown:
            errors.append(f"{col}: unknown levels {sorted(unknown)}")

    if not contract.get("allow_missing_in_snapshot", False):
        n_missing_rows = df[contract["features"]].isna().any(axis=1).sum()
        if n_missing_rows:
            errors.append(f"rows with missing feature: {n_missing_rows}")

    return errors
```

Invariáns: érvényes táblára `[]`, egy elcserélt oszlopra az első hibabejegyzés, ismeretlen `segment` értékre a harmadik hibabejegyzés jelenik meg. A függvény nem tanít modellt és nem imputál; a 13.4. alfejezet minőségi kapu kiegészítője.

3. megoldás. A negatív `account_age_days` azt jelzi, hogy a forrásban a `signup_date` későbbi, mint az `order_ts`, ami sérti a Northstar szerződést: a regisztráció minden rendelés előtt van. A lehetséges ok hibás időzóna-értelmezés, adatpatch vagy rossz forrás snapshot. A feature store nem tudja visszamenőleg helyesbíteni a forrást; a registry frissítése csak a tünetet takarhatná el. A blokkolás a belépési adatminőségi kapu dolga: a snapshot készítése előtt tartományellenőrzést kell futtatni, például `min(account_age_days) >= 0` feltétellel. A javítás után a 12 sor eltűnik a hibajelentésből.

4. megoldás. A külön feature store indokolt, mert a közös definíciók és a 200 ms-os P95 követelmény már nem teljesíthető megbízhatóan közös Parquet-fájlokkal, továbbá a `last_14d_order_count` és `mean_basket_7d` minden scoring hívásra friss, időablakos aggregátumot igényel. Az offline réteg továbbra is verziózott batchben számolhat; az online réteg gyorsítótárként materializál. Ami nem változik: a predikció pillanatában nem ismert oszlopok továbbra sem kerülhetnek feature-be, a foldon belüli preprocessing-t a 24.4. alfejezet szabálya szerint a training részen belül kell tartani, és a 21.4. alfejezet szerződésnek a tanítás és a scoring oldalon azonosnak kell lennie.

25.4.9 Gyakori tévedés ellenpéldával

Veszélyes állítás: *„Mivel a feature store központosított, a belőle olvasó modellek nem szenvedhetnek leakage-tól.”* Ez nem igaz. Tegyük fel, hogy a központi tárolóba valaki felveszi a `delivery_days` oszlopot, mert az üzleti elemzőknek hasznos. A predikciós modell feature-listája véletlenül `*`-gal olvas, vagy egy későbbi szelekciós lépés automatikusan beválasztja. A modell tanítási pontossága mesterségesen magas lesz, a scoring viszont a valós rendelés pillanatában nem látja a célváltozót,

ezért élesben látványosan gyengül. A központosítás itt nem csökkenti, hanem növeli a hiba hatókörét. A korábbi tanulságok érvényesek maradnak: a 24.4. alfejezet szerint a preprocessing és a szelekció a training foldon belül él, a 21.4. alfejezet szerint a célváltozónak és a későn érkező oszlopoknak kívül kell maradniuk a sémaengedélyen, a 13.4. alfejezet szerint pedig a hiányzás és az anomália a forrásfolyamat vizsgálati kérdése, nem a tároló utólagos „javítása”.

25.4.10 Önellenőrzés

Dátum-e van a döntési feljegyzésen, és a feature-táblának van-e egyértelmű sémaszerződése? A célváltozó kimaradt-e a feature-listából? A backfill minden sorára ugyanaz az asof feltétel érvényesül, és a join a predikció pillanatában ismert adatot használ? A tanításnál és a scoringnál azonos a `schema_version` és a kategóriaszintek? A frissességi késés mértéke és mértékegysége rögzített? A Parquet snapshotnak van hash-e és sorazonosítója? Ha ezekre igen a válasz, a Parquet + SQL döntés védhető; külön feature store a méret, késleltetés vagy csapatproblémából következzen, ne a „jó gyakorlat” címkéből.

A feature store választását a point-in-time helyesség, online/offline konzisztencia, hozzáférésvezérlés és üzemeltetési költség alapján indokoljuk; kis csapatnál a verziózott snapshot és SQL is lehet jó megoldás.

25.4.10.1 Online/offline parity szerződés

A feature store melletti döntés legfontosabb gyakorlati próbája a **parity teszt**: ugyanarra az `order_id` és `as_of` időpontpárra az offline és az online úton számolt feature-értékeknek azonosnak kell lenniük. A teszt nem aggregált metrikát hasonlít, hanem soronként és mezőnként ellenőriz:

Ellenőrzés	Mit hasonlít?	Hiba esetén
Érték	a feature numerikus vagy kategóriaértéke	eltérés > tolerancia
Típus	dtype és kategóriaszint	pl. `int64` vs `float64`, új szint
As-of timestamp	az online rekord `as_of` mezője	nem a kért időpont
Forrás	forrás-snapshot azonosítója	eltérő adatverzió
TTL	`served_at - source_updated_at`	a frissességi korlát túllépése
Hiányzás	mindkét oldalon `NaN`/`None`	az egyik oldalon érték, a másikon hiány

A parity szerződés kódja:

```
FEATURES = ["distance_km", "priority_flag", "basket_minor", "quantity_sum",
            "account_age_days", "segment", "region", "acquisition_channel"]

def check_parity(offline, online, as_of, ttl_seconds=300):
    errors = []
    if online.get("as_of") != as_of:
        errors.append("as-of timestamp eltér")
    if online.get("source") != offline.get("source"):
        errors.append("forrás-snapshot eltér")
    age = (online["served_at"] - online["source_updated_at"]).total_seconds()
    if age > ttl_seconds:
        errors.append(f"TTL túllépve: {age:.0f}s")
    for name in FEATURES:
```

```
        if name not in offline or name not in online:
            errors.append(f"{name}: hiányzó mező")
            continue
        a, b = offline[name], online[name]
        if (a is None) != (b is None):
            errors.append(f"{name}: hiányzás eltér")
        elif a is not None and type(a) is not type(b):
            errors.append(f"{name}: típuseltérés ({type(a).__name__} != {type(b).__name__})")
        elif a is not None and a != b:
            errors.append(f"{name}: értékeltérés ({a!r} != {b!r})")
    return errors
```

A tesztet a deployment előtt és a CI-ben is futtatni kell; a missing értéket az imputálás **előtt** kell összehasonlítani, különben az imputálás elfedheti a forráskülönbséget. A TTL és a hiányzási szabály a feature-szerződés része, nem a modellé.

25.4.11 Rövid összefoglaló

A feature store döntés nem általános modernségi kérdés, hanem konzisztencia-, késés- és csapatméret-alapú mérnöki választás. A Northstar projektben a kis adat, egy modell és a batch scoring miatt elég a verziózott Parquet snapshot, a JSON sémaszerződés és a verziózott SQL. A központi tároló akkor indokolt, ha több csapat és modell közös feature-definíciót használ, alacsony késleltetésű online kiszolgálás kell, vagy a backfill audit skálázódni nem tud. A verzió, a frissességi késés és az időhelyes join mindkét megoldásban kötelező; a feature store önmagában nem szünteti meg a leakage-t vagy a hibás céldefiníciót. Következő alkalmazás: a reprezentációs fejezet lezárása után a modellmagyarázat és a szeletelt értékelés 26.1. alfejezet, ahol ugyanezt a feature-szerződést használjuk a hibaszeletek értelmezéséhez.

26 Modellmagyarázat, megbízhatóság és szeletelt értékelés

26.1 Globális és lokális magyarázat

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 25.4. alfejezet · Célkészség: Globális és lokális magyarázat · Kimenet: működő kód és a labor ellenőrző tesztje.

26.1.1 Cél és konkrét helyzet

Egy már betanított prediktív modellt akarunk értelmezni. A magyarázandó modell egy logisztikus regressziós pipeline, amely két standardizált folytonos feature-ből becsli a késés valószínűségét. A magyarázat egysége a *betanított predikciós függvény viselkedése*: mit tesz a modell az adott bemenettel, nem pedig a valódi üzleti vagy fizikai hatásmechanizmus. A bemenet egy szintetikus adatállomány: x_1 és x_2 együttesen normális eloszlású, korrelációjuk körülbelül 0,8; a y bináris cél Bernoulli-változó. A megfigyelési egység egy rendelés. Az eredmény három magyarázati forma: standardizált regressziós koefficiens, permutációs fontosság, valamint PDP/ICE görbék; mindegyikhez tartozik egy pontosan megnevezett érvényességi korlát.

Előzetes becslés: mivel mindkét feature egyformán, 1,5 logit-együtthatóval hat a generáló folyamatban, de x_2 zajosabb komponenst is tartalmaz, várható, hogy a permutációs fontosság x_1-re kissé nagyobb lesz. A koefficiensek hasonló nagyságúak lehetnek, de a 0,8-as korreláció miatt instabilabbak.

26.1.2 Fogalmi magyarázat

26.1.2.1 Regressziós koefficiens

A logisztikus modell a logit-skálán lineáris:

$$\eta_i = \log \frac{P(y_i = 1)}{1 - P(y_i = 1)} = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}.$$

Itt x_{ij} a j-edik standardizált feature értéke a i-edik megfigyelésre, β_j pedig a hozzá tartozó regressziós koefficiens (regression coefficient). A `sklearn` lineáris modelljében a becsült `coef_` tömb alakja $(1, p)$, ahol p a feature-ök száma. Ha x_1 egy szórással nő, miközben a másik feature változatlan, akkor a log odds β_1-gyel változik; valószínűségi skálán a változás nem konstans, mert a logisztikus függvény nemlineáris. A koefficiens tehát *feltételes meredekség a modellen belül*. Erősen korrelált x_1, x_2 esetén a közös jel magyarázatát a két együttható között osztja meg a modell, ezért egyedi standard hibájuk nagy lehet akkor is, ha a modell predikciója stabil. A β_j pontbecslés nem tesztstatisztika; konfidenciaintervallumhoz bootstrap vagy statsmodels kell (15.4. alfejezet).

26.1.2.2 Permutációs fontosság

A permutációs fontosság (permutation importance, PI) azt méri, mennyit romlik egy előre rögzített pontossági mérőszám, ha a kérdéses feature oszlopát véletlenné tesszük:

$$\mathrm{PI}_j = \frac{1}{R}\sum_{r=1}^{R}\left(S_{\text{base}} - S_{\text{perm},j}^{(r)}\right).$$

Itt S a scoring, például ROC AUC vagy a negatív Brier-pontszám (`neg_brier_score`); S_{base} az eredeti adaton mért érték, $S_{\text{perm},j}^{(r)}$ pedig az r-edik permutáció után számított érték. Az `sklearn.inspection.permutation_importance` `importances` kimenete (p, R) alakú, ennek sorátlaga és szórása adja a jelentett értéket. Az R ismétlés a permutáció véletlen zaját csökkenti, de nem statisztikai konfidenciaintervallum. A PI_j mértékegysége megegyezik S mértékegységével; ROC AUC esetén az eltérés $[-1, 1]$ közé esik, és lehet negatív is, ha a permutáció véletlenül javítja a pontszámot.

Korrelált feature-öknél két torzítás is felléphet. Ha x_2 hordozza x_1 jelének nagy részét, x_1 permutálása alig ront, mert x_2 helyettesíti; így a PI alábecsüli a változó predikciós szerepét. Másrészt a permutálás olyan (x_1, x_2) párokat hozhat létre, amelyek a tanító eloszlásban szinte nem léteznek, ezért a modell extrapolál, és a romlás felfúvódhat. A PI tehát az adott modell, adateloszlás és scoring közös tulajdonsága; megváltoztatott adateloszlásra nem vihető át automatikusan.

26.1.2.3 PDP és ICE

A parciális függésgörbe (partial dependence plot, PDP) a modell predikciójának átlagos alakját mutatja a kiválasztott feature mentén, miközben a többi feature értékét a megfigyelt sorokból vesszük, és átlagolunk:

$$\hat{f}_S(t) = \frac{1}{n}\sum_{i=1}^{n}\hat{f}(t, x_{i,C}).$$

Itt $x_{i,C}$ az i-edik sor komplementer feature-értéke, t pedig a vizsgált feature rácsértéke. Ha a rács G pontból áll, a PDP egy $(G,)$ alakú számsor. Egy kézi mini-példa: legyen $n = 4$ és a modell $\hat{f}(x_1, x_2) = x_1 + 2x_2$, a megfigyelt párok $(0,0), (1,0), (0,1), (1,1)$. Ekkor x_1 PDP-je: $t = 0$-ra $(0 + 0 + 2 + 2)/4 = 1{,}0$, $t = 1$-re $(1 + 1 + 3 + 3)/4 = 2{,}0$.

Az individuális feltételes várható görbe (individual conditional expectation, ICE) nem átlagol a megfigyelések fölött, hanem minden sorra külön hagyja a komplementer értékeket:

$$\hat{f}_i(t) = \hat{f}(t, x_{i,C}).$$

Az ICE alakja (n, G), átlaga éppen a PDP. Ha a modellben interakció van, az ICE görbék nem párhuzamosak; ha nincs, a logit-skálán additív modellben alakjuk azonos családból való, de a valószínűségi skálán az eltolás miatt nem párhuzamosak. A PDP tehát elfedheti az ellentétes irányú csoportok kioltódását, ezért a PDP-t és az ICE-t együtt érdemes nézni.

26.1.2.4 Lokális magyarázat

A lokális magyarázat (local explanation) egyetlen megfigyelésre ad felbontást: miért lett éppen 0,76 egy adott rendelés becsült késési valószínűsége? Additív felbontásra törekszünk:

$$\hat{\eta}_i = \hat{\beta}_0 + \sum_{j=1}^{p} \hat{\beta}_j x_{ij}.$$

A $h_{ij} = \hat{\beta}_j (x_{ij} - r_j)$ tagokkal a referencia r_j értékhez viszonyított hozzájárulást kapunk. Lineáris modellben, ha a referenciát a train-átlagra választjuk, a h_{ij} tagok a SHAP-stílusú additív felbontást adják független feature-ök mellett; a valódi SHAP-csomag használata itt nem szükséges. Fontos: a lokális magyarázat egyetlen predikcióra igaz, három különböző sor három eltérő hozzájárulást mutathat.

26.1.3 Teljes minimális munkapélda

A következő kód szintetikus adatot készít, treníroz egy standardizált logisztikus regressziót, és kiszámolja a koefficienst, a PI-t, a PDP/ICE görbéket és egy sor lokális felbontását. A kódot `local_explanation_demo.py` néven a projektgyökérbe tesszük. A kimenet szemléltető, nem a könyvírás során végrehajtott mérés; a pontos értékek seedtől és könyvtárverziótól függhetnek.

```
# local_explanation_demo.py
# Szemléltető kód, nem a könyvgenerálás által futtatott mérés.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance, PartialDependenceDisplay

rng = np.random.default_rng(20261020)
n = 400

x1 = rng.normal(0.0, 1.0, n)
x2 = 0.8 * x1 + 0.6 * rng.normal(0.0, 1.0, n)

# Generáló logit; a valóságban ezt a kutató nem ismerné.
logit_true = -1.0 + 1.5 * x1 + 1.5 * x2
p_true = 1.0 / (1.0 + np.exp(-logit_true))
y = rng.binomial(1, p_true)

X = pd.DataFrame({"x1": x1, "x2": x2})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=2, stratify=y
)

model = Pipeline([
    ("scale", StandardScaler()),
    ("lr", LogisticRegression(max_iter=1000, random_state=0)),
])
model.fit(X_train, y_train)

coef = model.named_steps["lr"].coef_[0]
intercept = model.named_steps["lr"].intercept_[0]
```

```
print("Standardizált koefficiensek (log odds per szórás):", coef)
print("Intercept a standardizált térben:", intercept)

pi = permutation_importance(
    model, X_test, y_test,
    scoring="roc_auc", n_repeats=30, random_state=1,
)
print("PI átlag (AUC-romlás):", pi.importances_mean)
print("PI szórás:", pi.importances_std)

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
PartialDependenceDisplay.from_estimator(
    model, X_test, features=[0], kind="both",
    grid_resolution=20, ax=ax[0],
)
PartialDependenceDisplay.from_estimator(
    model, X_test, features=[1], kind="both",
    grid_resolution=20, ax=ax[1],
)
fig.suptitle("PDP és ICE a két standardizált feature-re")
fig.tight_layout()
fig.savefig("local_pdp_ice.png", dpi=150, bbox_inches="tight")

row = X_test.iloc[[0]]
row_scaled = model.named_steps["scale"].transform(row)[0]
contrib = coef * row_scaled
logit_val = intercept + contrib.sum()
p_val = 1.0 / (1.0 + np.exp(-logit_val))
print("Sor:", row.to_dict(orient="records")[0])
print("Lokális hozzájárulások logit-skálán:", list(zip(X.columns, contrib)))
print("Rekonstruált logit:", logit_val, "valószínűség:", p_val)
```

Futtatás Linux/macOS terminálból a projektgyökérben:

```
python local_explanation_demo.py
```

Windows PowerShellből:

```
python .\local_explanation_demo.py
```

A lab `pyproject.toml` alap numerikus csomagjait feltételezi; a `scikit-learn` pontos verzióját a projekt zárja. Egy lehetséges helyi futtatás kimenete (szemléltető, nem mért érték): a standardizált koefficiensek valahol $\hat{\beta}_1 \approx 1{,}6$ és $\hat{\beta}_2 \approx 1{,}4$ körül, a PI pedig x_1-re például 0,13, x_2-re 0,10 AUC-romlás. A PDP növekvő, telítődő görbe mindkét feature-nél; az ICE vonalak szétszórtak, mert a másik feature értéke eltolja őket.

26.1.4 Eredmény értelmezése és kézi kontroll

A generáló folyamatból ismert kézzel számolható kontroll: ha $x_1 = x_2 = 0$, akkor $\eta = -1{,}0$, ezért $P(y = 1) = 1/(1 + e) \approx 0{,}269$. A betanított modell interceptje közel lehet ehhez, mert a `StandardScaler` a train-adaton zéró átlagú feature-öket hoz létre; az eltérés a train-minta véletlen ingadozásából adódik.

A koefficiens értelmezése: standardizált egységre vetített feltételes log odds hatás. Ha $\hat{\beta}_1 = 1{,}62$, akkor a modell szerint x_1 egy szórásnyi növelése a másik feature rögzítése mellett 1,62-vel növeli a log oddsot. Ez a mondat akkor is igaz, ha x_1 és x_2 korrelált; viszont a korreláció miatt az egyes együtthatók bizonytalanabbak, és a modell szétoszthatja a közös magyarázó erőt. Ezért nem állítjuk, hogy $\hat{\beta}_1 > \hat{\beta}_2$ esetén x_1 „fontosabb” lenne; a kettő közötti különbség akár mintavételi zaj is lehet.

A PI ennél óvatosabb nyelvet igényel. A 0,13 és 0,10 érték annyit mond: a tesztadaton a modell ROC AUC-ja ennyivel romlik, ha az adott oszlopot permutáljuk. Mivel a két feature erősen korrelál, a másik változó részben pótolhatja a permutáltat; a tényleges szerep tehát nagyobb lehet a mért romlásnál. Másrészt a permutáció irreális párokat gyárt, ami felfújhatja a fontosságot. A PI korlátja ezért nem pusztán becslési zaj, hanem az együttes eloszlás megsértése.

Csoportos permutáció korrelált párra. A korrelált pár együttes szerepét úgy mérjük, hogy a két oszlopot **egyszerre** permutáljuk (a sorokat együtt mozgatva), így a köztük lévő kapcsolat megmarad, de a célváltozóhoz fűződő viszony megszakad:

```
base = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
rng_group = np.random.default_rng(1)
group_scores = []
for _ in range(30):
    perm = rng_group.permutation(len(X_test))
    X_perm = X_test.iloc[perm].reset_index(drop=True)
    group_scores.append(base - roc_auc_score(
        y_test, model.predict_proba(X_perm)[:, 1]))
print(np.mean(group_scores), np.std(group_scores))
```

Az ellenőrzött futásban az egyedi PI x_1-re 0,104, x_2-re 0,188 AUC-romlás, összegük 0,292; a csoportos permutáció viszont 0,397 romlást ad. A csoportos érték nagyobb, mint az egyedi PI-k összege, mert az egyedi permutációknál a másik korrelált feature részben pótolja a hiányzó jelet. A csoportos PI tehát a pár **együttes** predikciós hozzájárulását méri, nem az egyes változókét; az egyedi PI-rangsor önmagában nem elég a pár belső megosztásának eldöntésére. Egyik érték sem oksági hatás és nem abszolút üzleti fontosság: a modell, az adateloszlás és a scoring függvényében változik.

A lokális felbontásban a `contrib` értékek logit-skálán értendők. Például ha egy teszt-sorra $x_1 = 0{,}35$, $x_2 = -0{,}40$ standardizált érték, akkor $h_1 = 1{,}62 \cdot 0{,}35 = 0{,}57$, $h_2 = -0{,}58$, összegük $-0{,}01$; az intercept hozzáadásával kapjuk a logitot. Ez a felbontás additív, ezért alkalmas arra, hogy elmondjuk: „ez a rendelés főleg azért kapott alacsony késési valószínűséget, mert x_2 a referencia alatt van”.

26.1.5 Feladatok

1. **Értelmezés.** Tegyük fel, hogy egy futtatásban $\hat{\beta}_1 = 1{,}66$, $\hat{\beta}_2 = 0{,}20$, és a PI éppen ellentétes: x_2 romlása 0,05, x_1-é 0,03. Fogalmazd meg egy mondatban, mit mond a koefficiens, és miért nem ellentmondás a PI.

2. **Implementálás kézi kontrollal.** Egy standardizált térben becsült modell paraméterei $\hat{\beta}_0 = -0{,}94$, $\hat{\beta}_1 = 1{,}66$, $\hat{\beta}_2 = 1{,}38$. Számítsd ki $x_2 = 0{,}4$ rögzítése mellett a $P(y = 1)$ értéket az $x_1 \in \{-2, -1, 0, 1, 2\}$ rács minden pontján, és rajzold fel kézzel az így kapott egyéni görbét. Ez épp egy ICE-görbe.

3. **Hibadiagnózis.** Egy kolléga a permutációs fontosságot a train-adaton számolta, kiválasztotta a legnagyobb PI-jű változókat, majd ugyanazon a tesztadaton jelentette a jóságot, amelyen a végső AUC-t is nézte. Nevezd meg a két hibát, és mondd meg, hogyan helyeznéd a szelekciót a modellértékelési ciklusba a 25.3. alfejezet és 23.4. alfejezet szabályai szerint.

4. **Megváltoztatott feltétel.** Adj a demóhoz egy x_3 zajváltozót: `x3 = rng.normal(0.0, 1.0, n)`, és tanítsd újra a pipeline-t. Milyen értéket vársz x_3 koefficiensére és PI-jére? Miért nem elegendő egyetlen pontbecslés annak eldöntésére, hogy x_3 valóban haszontalan? Hogyan módosítanád az értékelést a döntéshez?

26.1.6 Megoldások és indoklás

1. A $\hat{\beta}_1 = 1{,}66$ azt jelenti, hogy x_1 egy szórásnyi növelése átlagosan 1,66 log odds növekedést ad, ha x_2 rögzített. A PI nem ellentmondás, mert más kérdésre válaszol: a permutáció során x_2 elrontása okozhat nagyobb romlást akkor is, ha az együtthatója kisebb, mert x_2 hordozhatja a közös jel nagy részét, vagy mert az AUC nem logit-skálán mér, és a küszöb környéki predikciók érzékenyebbek.

2. Az ICE-görbe pontjai $P(t) = \sigma(-0{,}94 + 1{,}66t + 1{,}38 \cdot 0{,}4)$, ahol $\sigma(u) = 1/(1 + e^{-u})$. A számolt értékek: $t = -2$-re logit $= -3{,}71$, $P \approx 0{,}024$; $t = -1$-re logit $= -2{,}05$, $P \approx 0{,}114$; $t = 0$-ra logit $= -0{,}39$, $P \approx 0{,}404$; $t = 1$-re logit $= 1{,}27$, $P \approx 0{,}781$; $t = 2$-re logit $= 2{,}93$, $P \approx 0{,}949$. A görbe tehát a -2 és 2 közötti tartományban a telítődő logisztikus alakot követi, ahol a rögzített $x_2 = 0{,}4$ az eltolást adja. A feladatban közölt együtthatók szemléltető bemenetek; a saját futtatásnál a kapott becslésekkel kell újraszámolni.

3. Az első hiba: a train-adaton mért PI optimista lehet a túlillesztés miatt, és a modell általánosító érzékenységét nem méri. A második hiba: a tesztadatot végső értékelés előtt használták fel válogatásra, ezért a jelentett AUC már nem érintetlen teszt; a tesztet csak egyszer, a véglegesített pipeline értékelésére szabad használni. Helyes ciklus: a feature-szelekció a belső train részen belül, például beágyazott validációval vagy a tanító foldokon belül számolt PI-vel történjen; a végső teszt csak a befagyasztott pipeline-ra kerül.

4. Az x_3 koefficiense várhatóan nullához közeli, PI-je pedig 0 körüli, mert a generáló folyamat nem használja. Egyetlen pontbecslés nem elegendő, mert a véletlen permutációk és a mintavételi zaj pozitív PI-t is adhat zajváltozónak. A döntéshez nézd a `pi.importances_std` értékét, növeld az `n_repeats` számát, és hasonlítsd a fontosságot nullfeature-re vagy a negatív tartományhoz: ha a PI nulla körül szóródik, a változót nem érdemes megtartani.

26.1.7 Gyakori tévedés ellenpéldával és önellenőrzés

A leggyakoribb félreértés: „a fontos prediktor egyben jó beavatkozási cél”. A PI vagy a PDP a modell asszociációs viselkedését írja le, nem a világbeli oksági hatást. Például egy kampánymodellben az `app_letoltes` erős prediktora lehet a vásárlásnak, de a letöltés kikényszerítése nem feltétlenül növeli a vásárlást, mert a letöltés és a vásárlás közös oka lehet az érdeklődés. A korreláció és a regresszió az együttes eloszlásból tanul (14.1. alfejezet); a beavatkozás hatásához randomizált kísérlet vagy explicit oksági elemzés kell (18.2. alfejezet, 18.4. alfejezet). A jelen szintetikus demóban ismerjük a generáló paramétereket, de a becsült magyarázat akkor is a modellről szól, nem automatikusan a generáló folyamatról.

Önellenőrzés: meg tudod-e nevezni, mi a magyarázandó modell és mi az állítás egysége? Tudod-e, mi a PI mértékegysége és mit sért a permutáció korrelált feature-öknél? El tudod-e mondani, miért nem azonos a PDP az oksági beavatkozási görbével? Ha a lokális hozzájárulások összege nem adja vissza a logitot, hol keresnéd a hibát?

26.1.8 Rövid összefoglaló

A regressziós koefficiens feltételes log odds meredekség; a permutációs fontosság a modell és scoring közös romlása, amely korrelált feature-öknél alul- vagy felülbecsülhet, mert a permutáció megsérti az együttes eloszlást. A PDP átlagos predikciós görbe, az ICE egyedi görbéket mutat, és az átlag elfedheti a heterogenitást. A lokális magyarázat egyetlen sor additív felbontása. Egyik eszköz sem oksági bizonyíték. Következő alkalmazás: a 26.2. alfejezet kalibrált valószínűség és predikciós bizonytalanság, ahol a magyarázatokat a modell megbízhatóságával vetjük össze.

26.2 Kalibrált valószínűség és predikciós bizonytalanság

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 26.1. alfejezet · Célkészség: Kalibrált valószínűség és predikciós bizonytalanság · Kimenet: működő kód és a labor ellenőrző tesztje.

26.2.1 Kalibráció és predikciós bizonytalanság a Northstar-modellben

A 26.1. alfejezetben megnéztük, mit miért mond a modell; most azt kérdezzük, mennyire hihetünk a kimenet konkrét számértékének. A Northstar prediktív projektben a célváltozó `target_late`: 1, ha `delivery_days > promised_days` (7 nap), egyébként 0. A modell egy `HistGradientBoostingClassifier`, és a `predict_proba` például 0,73-et ad egy új rendelésre. Kérdés: ha sok olyan rendelést nézünk, amelyre a modell 0,73 körüli értéket mond, valóban 73% körüli lesz a késői kézbesítések aránya? Ez a **kalibráció** (calibration) kérdése.

A kérdés mögött két különböző bizonytalanság van. Az **eseménybizonytalanság** (aleatoric uncertainty) magából a megfigyelési folyamatból fakad: egy adott rendelés kimenete akkor is véletlen, ha a valódi valószínűség pontosan ismert; a `target_late` Bernoulli-eloszlású 14.2. alfejezet szerint. A **modellhiba** (epistemic uncertainty) viszont onnan ered, hogy a valódi valószínűséget nem ismerjük pontosan; csak a tanítóadaton becsült modellünk van. Több vagy jobb tanítóadat ezt csökkentheti, míg az eseménybizonytalanság nem szüntethető meg.

A következő feladat előzetes becslésként segít a gondolkodás elindításában: egy modell a teszthalmaz 100 olyan pontjára, ahol a predikált valószínűség 0,70 és 0,80 közé esik, átlagosan 0,74-et ad. Körülbelül hány pozitív esetet várunk, ha a modell kalibrált? Ha nem az elvárt számot kapjuk, milyen irányú torzításról beszélünk? (A megoldást a feladatok után adjuk meg.)

26.2.2 Kalibráció és megbízhatósági görbe

A **kalibráció** formális definíciója: egy modell jól kalibrált, ha minden predikált valószínűségértékre a tényleges feltételes valószínűség közelítőleg megegyezik a predikcióval:

$$\mathbb{P}(Y = 1 \mid \hat{p} = p) \approx p,$$

ahol $Y \in \{0, 1\}$ a megfigyelt kimenet, $\hat{p} \in [0, 1]$ a modell által adott valószínűség. Mivel folytonos $\hat{p}$ mellett egy-egy konkrét értékhez alig tartozna megfigyelés, a gyakorlatban **bineket** képezünk. A **megbízhatósági görbe** (reliability diagram) minden binre két átlagot ábrázol:

- x-tengely: a bin átlagos predikált valószínűsége, $\bar{\hat{p}}_b = \frac{1}{n_b} \sum_{i \in b} \hat{p}_i$,
- y-tengely: a binben mért pozitív arány, $f_b = \frac{1}{n_b} \sum_{i \in b} y_i$,

ahol n_b a b-edik binbe eső tesztelemek száma, $\hat{p}_i$ a predikció, y_i a megfigyelt 0/1 kimenet. Tökéletes kalibrációnál a pontok a 45°-os egyenesen fekszenek. Ha egy pont az egyenes alatt van, a modell abban a tartományban **felülbecsli** a valószínűséget; ha felette, **alulbecsli**.

A kalibráció egy skalár mutatója a **Brier-pontszám** (Brier score):

$$B = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - y_i)^2,$$

ahol n a tesztpontok száma, $\hat{p}_i \in [0,1]$ a predikált valószínűség, $y_i \in \{0,1\}$ a megfigyelt kimenet. A Brier-pontszám 0 és 1 közötti, minél kisebb, annál jobb. Két hasznos invariáns: a konstans 0,5-ös modell Brier-pontszáma mindig 0,25, mert $(0{,}5 - y_i)^2 = 0{,}25$ minden y_i-re; ha a tesztmintában a pozitívok aránya q, akkor a $\hat{p}_i = q$ konstans predikció Brier-pontszáma $q(1-q)$, ami legfeljebb 0,25. A Brier-pontszám egyszerre jutalmazza a jó kalibrációt és a jó diszkriminációt, ezért két különböző hibát is összemoshat; ezért mindig a görbét is meg kell nézni.

26.2.3 Teljes minimális munkapélda: kalibráció időablakokban

A következő kód a lab `data/orders.csv` és `data/customers.csv` fájljait használja, és a 22.2. alfejezetben bemutatott boosting modellt tanítja az időbeli split szerint. A scriptet a `lab` könyvtárból futtatjuk, miután a környezetet `uv sync` paranccsal telepítettük. A kód nem futtatott mérési jegyzék; a kimenet szemléltető, a pontos számok a helyi futtatástól függnek.

```
# s102_calibration.py - futtatandó: cd lab && uv run python s102_calibration.py
from pathlib import Path
import pandas as pd
import numpy as np
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss

orders = pd.read_csv(Path("data") / "orders.csv")
customers = pd.read_csv(Path("data") / "customers.csv")

orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)
orders["label_available_at"] = pd.to_datetime(orders["label_available_at"], utc=True)
customers["signup_date"] = pd.to_datetime(customers["signup_date"], utc=True)

df = orders.merge(
    customers[["customer_id", "signup_date", "segment", "region", "acquisition_channel"]],
    on="customer_id", how="left"
)
df["account_age_days"] = (
    df["order_ts"] - df["signup_date"]
).dt.days

cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train_mask = (df["order_ts"] < cutoff) & (df["label_available_at"] < cutoff)
test_mask = df["order_ts"] >= cutoff

features = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel"
]
X_train = df.loc[train_mask, features]
y_train = df.loc[train_mask, "target_late"]
X_test = df.loc[test_mask, features]
y_test = df.loc[test_mask, "target_late"]

numeric_features = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum", "account_age_days"
]
categorical_features = ["segment", "region", "acquisition_channel"]
preprocess = ColumnTransformer([
    ("num", "passthrough", numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_features),
```

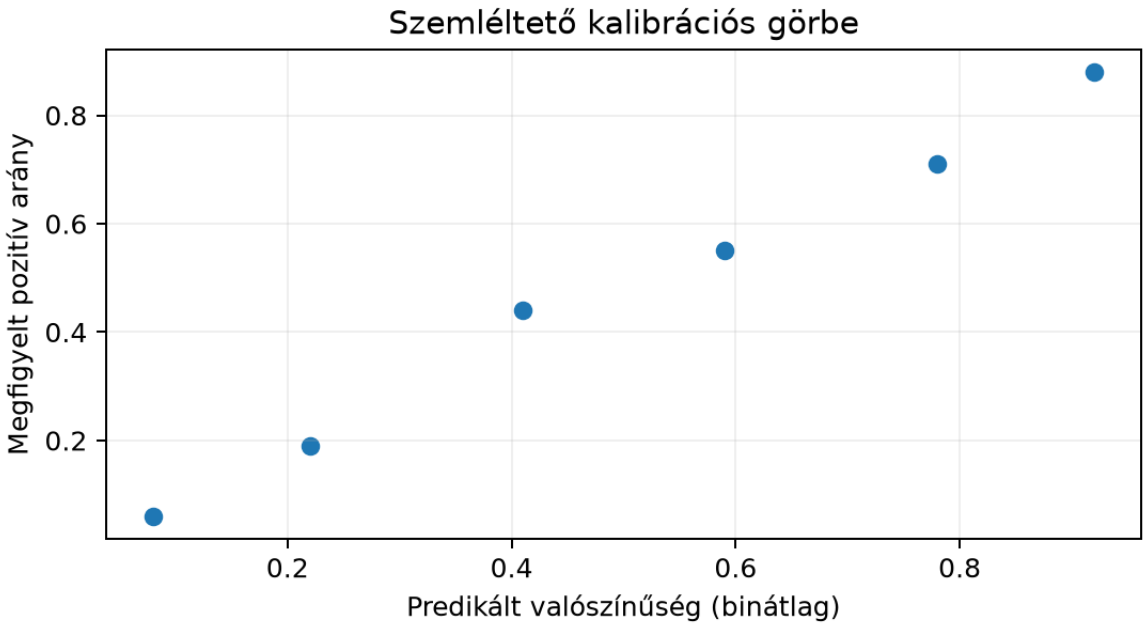
```
])
model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", HistGradientBoostingClassifier(
        max_iter=200, learning_rate=0.08, early_stopping=False, random_state=42
    )),
])
model.fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[:, 1]

prob_true, prob_pred = calibration_curve(
    y_test, y_prob, n_bins=10, strategy="quantile"
)
brier = brier_score_loss(y_test, y_prob)

print("Brier a teljes teszten:", round(brier, 4))
print("Kalibrációs pontok (predikált, megfigyelt):")
for px, py in zip(prob_pred, prob_true):
    print(f"  {px:.3f}  {py:.3f}")

# Külön időablakok: 2025. május és 2025. június
for start, end, name in [
    ("2025-05-01", "2025-06-01", "majus"),
    ("2025-06-01", "2025-07-01", "junius"),
]:
    mask = (df.loc[test_mask, "order_ts"] >= pd.Timestamp(start, tz="UTC")) & (
        df.loc[test_mask, "order_ts"] < pd.Timestamp(end, tz="UTC")
    )
    y_t = y_test[mask]
    y_p = y_prob[mask]
    if len(y_t) == 0:
        print(f"{name}: nincs tesztmegfigyelés ebben az időablakban")
        continue
    b = brier_score_loss(y_t, y_p)
    print(f"{name}: tesztpont {len(y_t)}, Brier {b:.4f}")
```

A szemléltető kimenetben a teljes teszt Brier-pontszáma 0,18 körül lehet, ami jobb a 0,25-ös állandó modellnél; a kalibrációs pontok a 45°-os vonal közelében, de nem pontosan rajta helyezkednek el. A két időablak Brier-pontszáma eltérhet, mert a májusi és júniusi forgalom eloszlása nem azonos. A kalibrációs pontpárok egy lehetséges szemléltető alakja:



26.1. ábra: Szemléltető kalibrációs görbe - Szimulált illusztráció; a tényleges pontok a sklearn calibration_curve kimenetéből jönnek.

A `calibration_curve` két, azonos hosszúságú tömböt ad vissza: `prob_pred` a **nem üres** binek átlagos predikciója, `prob_true` a bennük mért pozitív arány. `n_bins=10` esetén is **legfeljebb tíz pont** jelenik meg, mert az üres binek kimaradnak. Az értékeléshez érdemes a binenkénti elemszámokat külön is meghatározni. Az ellenőrzés során érdemes együtt vizsgálni a binméretet is: ha egy binbe csak néhány pont esik, a mért arány nagy véletlen ingadozást mutathat. A bin gyakoriságának f_b becsült standard hibája közelítőleg $\sqrt{f_b(1-f_b)/n_b}$, tehát $n_b = 30$ és $f_b = 0{,}7$ mellett körülbelül 0,084.

26.2.4 Bootstrap modellcsoport és modellhiba

A kalibráció az eseménybizonytalanság átlagos egyezését méri; nem mutatja meg, mennyire stabil maga a modell. A **bootstrap modellcsoport** (bootstrap models) a modellhiba becslésére használ egyszerű újramintavételezést: a tanító indexekből B darab, visszatevéssel vett mintát képezünk, mindegyiken újratanítjuk a modellt, majd egy rögzített tesztpontra megnézzük a predikciók eloszlását. A szórás a modellhiba közelítő mértéke; ha a tanítóhalmaz kicsi vagy a modell instabil, a szórás nagy.

```
from sklearn.utils import resample

def bootstrap_preds(model_cls, X_train, y_train, X_point, B=50, seed=42):
    preds = []
    n = len(X_train)
    rng = np.random.default_rng(seed)
    for b in range(B):
        idx = rng.integers(0, n, size=n)  # bootstrap soronként
        if y_train.iloc[idx].nunique() < 2:
            continue  # egyszínű bootstrap-minta nem tanítható kétosztályos modellként
        m = model_cls(random_state=b)
        m.fit(X_train.iloc[idx], y_train.iloc[idx])
        preds.append(m.predict_proba(X_point)[0, 1])
    return np.array(preds)
```

```
# Példa egy rögzített tesztpontra:
point = X_test.iloc[[0]]
bp = bootstrap_preds(lambda random_state: Pipeline([
    ("preprocess", ColumnTransformer([
        ("num", "passthrough", numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
 ↪  categorical_features),
    ])),
    ("classifier", HistGradientBoostingClassifier(random_state=random_state)),
]), X_train, y_train, point, B=50)
print(f"predikció átlag: {bp.mean():.3f}, szórás: {bp.std():.3f}")
```

A szemléltető szórás 0,01–0,05 között lehet; a konkrét érték a seedtől és a modellparaméterektől függ. Fontos korlát: a Northstar adatban egy ügyfélnek több rendelése is van, ezért a soronkénti bootstrap mintavétel nem kezeli az ügyfélszintű függést. A **mintavételi egység** itt a rendelés, nem az ügyfél; ha ügyfélszintű bizonytalanságot akarunk, az ügyfél-azonosítók újramintavételezése lenne a helyes út.

26.2.5 Split-conformal körkép: predikciós intervallum

A **konform predikció** (conformal prediction) közvetlenül az új megfigyelésre ad **predikciós intervallumot** (prediction interval), nem a modell paraméterére. Az alábbi **split-conformal** változat két diszjunkt halmazt használ: egy tanítóhalmazt a modell illesztésére és egy kalibrációs halmazt a hibák kalibrálására. A cél egy regressziós kimenet, például `delivery_days`; a Northstar fő céljában ez nem predictor, de itt egy külön, szintetikus illusztrációban természetes.

A módszer lépései:

1. Tanítsunk $\hat{f}$ modellt a tanítóhalmazon.
2. A kalibrációs halmaz minden elemére számítsuk a **nem-megfelelőségi pontszámot** (nonconformity score), például az abszolút hibát: $R_i = |Y_i - \hat{f}(X_i)|$.
3. Válasszunk α hibaszintet (például 0,1), és vegyük a rendezett R_i értékek közül a $k = \lceil (n_{\text{cal}} + 1)(1 - \alpha) \rceil$-edik legkisebbet; ez a q kvantilis.
4. Új X_{n+1} pontra az intervallum: $C(X_{n+1}) = [\hat{f}(X_{n+1}) - q,\ \hat{f}(X_{n+1}) + q]$.

A felhasznált **kvantilis** itt nem a hibaeloszlás elméleti paramétere, hanem a kalibrációs halmaz empirikus rendezett statisztikája. A garantált lefedettség **marginális**:

$$\mathbb{P}(Y_{n+1} \in C(X_{n+1})) \geq 1 - \alpha,$$

feltéve, hogy a kalibrációs és az új pontok **kicserélhetők** (exchangeable). A kicserélhetőség azt jelenti, hogy a pontok együttes eloszlása nem változik, ha sorrendjüket tetszőlegesen átrendezzük. Független, azonos eloszlású (i.i.d.) pontok kicserélhetők; időbeli adatnál azonban a múltbeli kalibrációs pontok és a jövőbeli tesztpontok általában nem, mert a folyamat eltolódhat. A Northstar időbeli splitje ezért **nem** teljesíti automatikusan a konform garanciát; ez a módszer inkább akkor használható, ha a kalibrációs és az új pontok ugyanabból a stacionárius folyamatból származnak.

A következő szintetikus kód i.i.d. adaton mutatja a split-conformal eljárást:

```
rng = np.random.default_rng(42)
X = rng.uniform(0, 1, size=(400, 1))
y = 3.0 * X[:, 0] + rng.normal(0, 0.5, size=400)
```

```
X_fit, X_cal, X_test = X[:150], X[150:300], X[300:]
y_fit, y_cal, y_test = y[:150], y[150:300], y[300:]

# A modellillesztés és a konform kalibráció adatai diszjunktak.
fit = LinearRegression().fit(X_fit, y_fit)
cal_scores = np.abs(y_cal - fit.predict(X_cal))
alpha = 0.1
n_cal = len(cal_scores)
q = np.sort(cal_scores)[int(np.ceil((n_cal + 1) * (1 - alpha))) - 1]

test_scores = np.abs(y_test - fit.predict(X_test))
coverage = np.mean(test_scores <= q)
print(f"q = {q:.3f}, empirikus lefedettség = {coverage:.3f}")
```

Itt $n_{\text{cal}} = 150$, $\alpha = 0{,}1$, tehát $k = \lceil 151 \cdot 0{,}9 \rceil = 136$; a kvantilis a 136. legkisebb kalibrációs abszolút hiba. A tesztmintán mért lefedettség véletlenszerűen ingadozhat; a legalább 90%-os marginális garancia az új, kicserélhető megfigyelésre vonatkozó ismételt mintavételi állítás, nem egy adott tesztmintára vagy alcsoportra előírt alsó korlát. Az intervallum jelentése: az új pontok 90%-át várhatóan lefedi, de ez **nem** jelenti, hogy egy adott csoportban is 90%.

26.2.6 Feladatok

1. **Értelmezés.** Egy kalibrációs binben 80 tesztpont van, a predikciók átlaga 0,55, a megfigyelt pozitívok aránya 0,42. Kalibrált-e a modell ebben a tartományban? Milyen irányú torzítást mutat?

2. **Megvalósítás.** Adott `y_true = np.array([0, 1, 0, 1, 1])` és `y_prob = np.array([0.2, 0.6, 0.1, 0.8, 0.7])`. Számítsd ki a Brier-pontszámot kézzel (NumPy nélkül is lehet), és ellenőrizze a `sklearn.metrics.brier_score_loss` kimenetével. Mi a minimális és maximális lehetséges érték erre az öt pontra?

3. **Hibakeresés.** Egy tanuló a következő sort használja: `y_prob = model.predict(X_test)`, majd ezt adja a `calibration_curve`-nak. A függvény fut, de a görbe két szélső pontba koncentrálódik. Mi a hiba, és hogyan javítandó?

4. **Transzfer megváltozott feltételre.** A modell a teljes júniusi tesztablakon jól kalibráltnak tűnik, de a `segment == "business"` szegmensben a megfigyelt pozitív arány minden binben alacsonyabb a predikált átlagnál. Milyen további vizsgálatot végezne, és milyen műveletet fontolna meg? Indokold a csoportos kalibráció szükségességét.

5. **Konform ellenőrzés.** A split-conformal példában $n_{\text{cal}} = 199$ és $\alpha = 0{,}2$. Hányadik legkisebb kalibrációs pontszámot kell kvantilisként használni? Ha a rendezett pontszámok 160. eleme 0,47 és 161. eleme 0,51, akkor a kvantilis 0,51; miért nem a kerekítés másik iránya?

26.2.7 Megoldások és indoklás

1. A modell **felülbecsli** a valószínűséget ebben a tartományban: a predikciók átlaga 0,55, a tényleges gyakoriság 0,42. A pont a 45°-os vonal alatt helyezkedne el. A standard hiba $\sqrt{0{,}42 \cdot 0{,}58/80} \approx 0{,}055$, tehát a 0,13-as eltérés valószínűleg nem pusztán binméret-ingadozás.

2. A Brier-pontszám:

$$B = \frac{1}{5}[(0{,}2-0)^2+(0{,}6-1)^2+(0{,}1-0)^2+(0{,}8-1)^2+(0{,}7-1)^2] = \frac{0{,}04+0{,}16+0{,}01+0{,}04+0{,}09}{5} = 0{,}068.$$

   A `brier_score_loss(y_true, y_prob)` ugyanezt adja. Az öt ponton a minimális 0, ha minden predikció pontosan egyezik a kimenettel; a maximális 1, ha minden predikció a rossz osztályt kapja 1 valószínűséggel.

3. A `predict` osztálycímkét ad vissza (0 vagy 1), a kalibrációhoz a pozitív osztály valószínűsége kell. A helyes sor:

   ```
   y_prob = model.predict_proba(X_test)[:, 1]
   ```

   Ekkor a `calibration_curve` bemenete a $[0, 1]$ intervallumba eső valószínűség, és a binekben értelmes átlagok képződnek.

4. A csoportszintű eltérés **szisztematikus torzítás** jele: a business szegmensben a modell folyamatosan túl magas valószínűséget ad. Ilyenkor külön kalibrációs görbét kell számítani a szegmensre; ha az eltérés stabil, egy kalibrációs réteg (például isotonic vagy Platt) illeszthető, de csak a tanítóidőszaki adaton, és a csoportazonosítót dokumentálni kell. Ugyanakkor vigyázni kell: a szegmensen belüli mintaméret kicsi lehet, és a kalibrációs réteg maga is túlilleszthet.

5. A képlet szerint $k = \lceil (199+1)(1-0{,}2) \rceil = \lceil 200 \cdot 0{,}8 \rceil = 160$. A 160. legkisebb pontszám a kvantilis; mivel a 160. elem 0,47 és a 161. elem 0,51, a keresett érték 0,47. (A feladat szándékosan kérdezett rá a kerekítés irányára: a felső egészrész miatt a 160. elemet vesszük, nem a 161.-et; ha a 161.-et vennénk, az α nagyobb lenne a tervezettnél.)

26.2.8 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés: **az érvényes összesített lefedettség nem garantál minden csoportra azonos lefedettséget.** Tegyük fel, hogy a split-conformal eljárás kalibrációs pontszámai két részsokaságból származnak. Az A csoportban a hibák jellemzően 0,8 körüliek, a B csoportban 0,3 körüliek. A globális kvantilis 0,75 lehet, mert a pontok fele A-ból, fele B-ből jön. Ekkor az A csoportban az új pontok többsége kívül esik az intervallumon, mondjuk a lefedettség 40%, míg a B csoportban 95%. Ha a két csoport azonos súlyú, e példában az összesített lefedettség mindössze $(0{,}40+0{,}95)/2 = 0{,}675$. Ahhoz, hogy az összesített lefedettség 90% legyen, a 95%-os lefedettségű B csoport súlyának legalább $10/11 \approx 90{,}9\%$-nak kell lennie; ilyenkor lehetséges a rossz A csoportos lefedettség mellett is magas globális lefedettség, de az A csoportra a gyakorlati predikció használhatatlan. Ezért a kalibrációs és konform ellenőrzéseket mindig **szeletelt** (szegmens, időablak, régió szerinti) bontásban is el kell végezni; a 26.3. alfejezetben ezt a hibaszelet-elemzést folytatjuk.

26.2.8.1 Csoportonkénti kalibráció és előre rögzített minimum n

A globális lefedettség tehát nem helyettesíti a csoportonkénti ellenőrzést. A jelentésben minden előre deklarált alcsoportra külön kell közölni a lefedettséget, a Brier-t és a mintaméretet; a minimum n-t a vizsgálat **előtt** kell rögzíteni, nem az eredmény láttán. Példa:

Csoport	n	Lefedettség	Brier	Megjegyzés
A	100	0,40	0,31	a globális 90% mellett is alacsony
B	900	0,95	0,08	dominálja az összesített arányt
Összes	1000	0,895	0,10	a súlyozott átlag elrejti az A problémáját

A globális 89,5% mellett az A csoport 40%-os lefedettsége elfogadhatatlan; a súlyozott átlag ezt elrejti. A javasolt szabály:

1. Az alcsoportok listáját és a minimális $n_{\min}$-t (például 100 megfigyelés vagy 20 pozitív esemény) a kiértékelés előtt rögzíteni kell.
2. Ha egy alcsoport $n < n_{\min}$, nem adunk rá pontbecslést; „elégtelen adat” jelzést és összevont/hierarchikus kalibrációt használunk.
3. Minden alcsoportra Wilson-intervallumot vagy bootstrap-CI-t közlünk, nem csak a pontbecslést.
4. Ha egy alcsoport lefedettsége a névleges szint alá esik, a globális küszöböt nem lehet változatlanul bevezetni; a döntést alcsoportonként kell dokumentálni.

```
def subgroup_report(frame, group_col, covered, n_min=100):
    report = {}
    for name, part in frame.groupby(group_col):
        n = len(part)
        cov = float(covered[part.index].mean())
        report[name] = {
            "n": n, "coverage": cov if n >= n_min else None,
            "status": "ok" if n >= n_min else "insufficient_data",
        }
    return report
```

A függvény a kis csoportra `None` lefedettséget ad, nem hamis pontosságot. A csoportos kalibráció és a minimum n együtt teszi a lefedettségi állítást ellenőrizhetővé; a globális szám önmagában nem garancia.

Visszautalva a korábbi fogalmakra: a 14.2. alfejezetben bevezetett Bernoulli-eloszlás adja az eseménybizonytalanság matematikai modelljét; a 22.2. alfejezetben tanított boosting modell a predikció forrása; a 25.2. alfejezetben használt pipeline biztosítja, hogy a kalibrációs réteg tanulása ne szivárogtasson tesztinformációt.

Önellenőrzés. Fogalmazza meg saját szavaival: miért nem elég egyetlen Brier-pontszám a modell megbízhatóságának megítélésére? Melyik feltétel sérül, ha a split-conformal kalibrációs halmaz a múltból, a tesztpont a jövőből származik? Milyen lépést tenne, ha a kalibrációs görbe a magas valószínűségű tartományban következetesen a 45°-os vonal alatt fut?

26.2.9 Rövid összefoglaló

A kalibráció azt ellenőrzi, hogy a modell által adott valószínűség átlagosan egyezik-e a megfigyelt gyakorisággal; a megbízhatósági görbe ezt binekre bontva mutatja. Az eseménybizonytalanság a véletlen kimenetből, a modellhiba a becsült paraméterek korlátosságából ered; utóbbira a

bootstrap modellcsoport szórása ad közelítő mértéket. A split-conformal predikció kicserélhető pontok esetén marginális fedési garanciájú intervallumot képez, de időbeli vagy csoportszerkezet mellett a garancia sérülhet. A laborban a kalibrációt két időablakban számítottuk; a következő alfejezet a hibaszeletek és a felelős modellkártya felé viszi tovább ezt a gondolatot.

26.3 Hibaszeletek és felelős értékelés

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 26.2. alfejezet · Célkészség: Hibaszeletek és felelős értékelés · Kimenet: működő kód és a labor ellenőrző tesztje.

26.3.1 Cél és konkrét helyzet

A Northstar modell a rendelés pillanatában becsli, hogy a kézbesítés meghaladja-e a 7 napot. A kalibrált teszt-predikciók már rendelkezésre állnak: minden rendeléshez van `y_true`, `p_hat` és a 0,5-ös küszöbből képzett `y_pred`. A kérdés most az, hogy a modell viselkedése egyenletes-e a `region` három szeletében: **north**, **central**, **south**.

Bemenet: a teszt halmaz `order_id`, `region`, `y_true`, `p_hat`, `y_pred = (p_hat >= 0.5).astype(int)` oszlopokkal. A megfigyelési egység a rendelés; egy ügyfél több rendelést is adhat, ez fontos korlát lesz.

Kimenet: szeletenkénti hibamátrix (confusion matrix), mintanagyság, prevalencia, TPR/FPR, és ezek 95%-os Wilson-konfidenciaintervalluma. Az érzékenységi elemzés a küszöböt és a többszörös összehasonlítást is vizsgálja.

Induló feladat: mielőtt tovább olvasol, gondold végig: miért lehet félrevezető az összesített accuracy, ha a south régióban csak 100 rendelés van, és ott a késések aránya 5%?

26.3.2 Fogalmi magyarázat

26.3.2.1 Szeletelemzés (slice analysis)

A szeletelemzés a modell értékelését egy kategorikus változó értékei szerint bontja részekre. Itt a szelet a teszt halmaz azon rendeléseiből áll, ahol `region` egy adott értéket vesz fel. A szeletelt hibamátrix minden szeletben megadja:

- TP: tényleges késő, predikció késő;
- FP: tényleges nem késő, predikció késő;
- TN: tényleges nem késő, predikció nem késő;
- FN: tényleges késő, predikció nem késő.

Ebből számítjuk a feltételes arányokat. A **prevalencia** a pozitív osztály aránya a szeletben: $\pi = P(Y = 1 \mid \text{szelet})$, becslése $\hat{\pi} = (\text{TP} + \text{FN})/n$, ahol n a szelet mintanagysága. A **TPR** (true positive rate, szenzitivitás) a tényleges pozitívok eltalálási aránya: $\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$. Az **FPR** (false positive rate) a tényleges negatívok téves pozitív aránya: $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$. A **PPV** (precision) a pozitív predikciók találati aránya: $\text{PPV} = \text{TP}/(\text{TP} + \text{FP})$.

A PPV a prevalenciától függ; a kalibrálásnál használt összefüggés:

$$\mathrm{PPV} = \frac{\mathrm{TPR} \cdot \pi}{\mathrm{TPR} \cdot \pi + \mathrm{FPR} \cdot (1 - \pi)}$$

Ezért azonos TPR és FPR mellett is más PPV-t kapunk, ha a prevalencia eltér.

26.3.2.2 Kicsi csoportok bizonytalansága és standard error

A szeletelt arányok mintából számított becslések, ezért van standard hibájuk. Az x/n binomiális arány klasszikus standard hibája:

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

ahol $\hat{p}$ a becsült arány, n a szelet mérete. Kicsi n-nél a normál approximáció félrevezető, ezért a **Wilson-konfidenciaintervallumot** használjuk. $z = \Phi^{-1}(1 - \alpha/2)$ mellett:

$$\hat{p} = \frac{x}{n}, \quad D = 1 + \frac{z^2}{n}, \quad c = \frac{\hat{p} + z^2/(2n)}{D}, \quad m = \frac{z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{D}$$

A CI: $[c - m,\ c + m]$. Ez a képlet $x = 0$-ra és $x = n$-re sem ad [0,1]-en kívüli határt, ezért kis mintán is használható.

26.3.2.3 Többtesztelés

Ha három régió TPR-jét egyszerre vizsgáljuk, a három, egyenként pontosan 95%-os intervallum együttes lefedettsége független lefedési események esetén $0{,}95^3 \approx 0{,}857$; függő intervallumoknál ez a szorzat nem alkalmazható; a téves különbség észlelésének esélye nő. A Bonferroni-korrekció a családonkénti α-t osztja el a szeletek k számával: $\alpha_{\mathrm{korr}} = \alpha/k$. Három szeletnél a korrigált CI-szint $1 - 0{,}05/3 = 0{,}9833$, azaz $z \approx 2{,}394$. Ezt az érzékenységi elemzésnél alkalmazzuk.

26.3.2.4 Fairness-metrikák és trade-off

A szeletelt hibamátrixból három gyakori csoport-mérték számítható:

- **selection rate (kiválasztási arány)**: $SR = (\mathrm{TP} + \mathrm{FP})/n$;
- **demographic parity**: az SR egyenlősége a csoportokban;
- **equal opportunity**: a TPR egyenlősége;
- **equalized odds**: a TPR és az FPR egyidejű egyenlősége.

A küszöb mozgatása a TPR–FPR síkon mozgatja a modellt: alacsonyabb küszöb növeli a TPR-t és az FPR-t is. Ha a csoportok score-eloszlása eltér, ugyanaz a küszöb csoportonként más TPR/FPR arányt ad; a csoportok közötti egyenlőség és az összesített pontosság között **trade-off** lehet. A Northstar `region` mesterséges üzleti kategória, nem valós védett tulajdonság; a fairness-metrikák itt technikai gyakorlatok, nem jogi audit.

26.3.3 Teljes minimális példa

Készítsünk 9 rendelésből álló kézzel ellenőrizhető adatot:

```
import pandas as pd
import numpy as np
from scipy.stats import norm

# Kézi kis adat: 9 rendelés, 3 régió
df = pd.DataFrame({
    "order_id": [f"R{i}" for i in range(1, 10)],
    "region":   ["north"]*3 + ["central"]*4 + ["south"]*2,
    "y_true":   [0, 1, 0,   1, 0, 1, 1,   1, 0],
    "p_hat":    [0.30, 0.80, 0.55,   0.70, 0.20, 0.40, 0.90,   0.40, 0.10],
})
df["y_pred"] = (df["p_hat"] >= 0.5).astype(int)

def wilson_ci(x, n, level=0.95):
    """Wilson CI binomiális x/n arányra."""
    z = norm.ppf(1 - (1 - level) / 2)
    p = x / n if n > 0 else np.nan
    D = 1 + z**2 / n
    c = (p + z**2 / (2*n)) / D
    m = z * np.sqrt(p*(1-p)/n + z**2 / (4*n**2)) / D
    return c - m, c + m

rows = []
for region, g in df.groupby("region"):
    n = len(g)
    tp = ((g.y_true == 1) & (g.y_pred == 1)).sum()
    fp = ((g.y_true == 0) & (g.y_pred == 1)).sum()
    tn = ((g.y_true == 0) & (g.y_pred == 0)).sum()
    fn = ((g.y_true == 1) & (g.y_pred == 0)).sum()
    prev = (tp + fn) / n
    tpr = tp / (tp + fn) if (tp + fn) > 0 else np.nan
    fpr = fp / (fp + tn) if (fp + tn) > 0 else np.nan
    tpr_ci = wilson_ci(tp, tp + fn)
    fpr_ci = wilson_ci(fp, fp + tn)
    rows.append((region, n, tp, fp, tn, fn, prev, tpr, fpr, tpr_ci, fpr_ci))

result = pd.DataFrame(rows, columns=[
    "region", "n", "TP", "FP", "TN", "FN",
    "prevalence", "TPR", "FPR", "TPR_CI", "FPR_CI"
])
print(result[["region", "n", "TP", "FP", "TN", "FN", "prevalence", "TPR", "FPR"]])
```

A kód kimenete az alábbi **szemléltető, elméletileg várt** táblázat (a kézi adatból számítva, nem futtatott jegyzék):

region	n	TP	FP	TN	FN	prevalence	TPR	FPR
north	3	1	1	1	0	0,333	1,00	0,50
central	4	2	0	1	1	0,750	0,667	0,00
south	2	0	0	1	1	0,500	0,00	0,00

Kézi kontroll: north-ban a három sor `(0,0)`, `(1,1)`, `(0,1)`, tehát TN=1, TP=1, FP=1, FN=0; az összeg 3. A central TPR CI-ja: $x = 2, n = 3$, $\hat{p} = 2/3$, $z = 1{,}96$; a fenti képlettel $c \approx 0{,}573$, $m \approx 0{,}365$, azaz $\text{TPR}_{\text{CI}} \approx [0{,}21;\ 0{,}94]$. A south TPR CI-ja $x = 0, n = 1$ esetén $[0;\ 0{,}79]$. Ez jól mutatja, hogy a kis csoportban a pontbecslés 0,00 mellett a CI felső határa is hasznos információ.

26.3.4 Labor: három region szelet a Northstar tesztjén

Labor specifikáció. Bemenet: a teszt halmaz `y_true`, `p_hat`, `region` oszlopokkal. A `y_pred` definíciója `p_hat >= 0.5`. Kimenet: a fenti táblázat a három régióra, plusz a TPR és FPR Wilson 95% CI; majd érzékenységi elemzés: (1) küszöb 0,35 és 0,65 mellett a TPR/FPR változása; (2) Bonferroni-korrigált 98,3%-os CI a három szelet együttes értékeléséhez; (3) ügyfélszintű klaszterezett bootstrap a TPR-különbségek standard hibájához, mert a rendelések nem függetlenek (visszautalás 14.3. alfejezet).

```
# Feltételezzük, hogy test_df már tartalmazza a y_true, p_hat, region oszlopokat.
test_df = test_df.copy()
test_df["y_pred"] = (test_df["p_hat"] >= 0.5).astype(int)

def slice_report(frame, region_col, threshold=0.5, level=0.95):
    frame = frame.copy()
    frame["y_pred"] = (frame["p_hat"] >= threshold).astype(int)
    out = []
    for value, g in frame.groupby(region_col):
        tp = ((g.y_true == 1) & (g.y_pred == 1)).sum()
        fp = ((g.y_true == 0) & (g.y_pred == 1)).sum()
        tn = ((g.y_true == 0) & (g.y_pred == 0)).sum()
        fn = ((g.y_true == 1) & (g.y_pred == 0)).sum()
        n = len(g)
        prev = (tp + fn) / n
        tpr = tp / (tp + fn) if (tp + fn) > 0 else np.nan
        fpr = fp / (fp + tn) if (fp + tn) > 0 else np.nan
        tpr_ci = wilson_ci(tp, tp + fn, level)
        fpr_ci = wilson_ci(fp, fp + tn, level)
        out.append((value, n, prev, tpr, fpr, tpr_ci, fpr_ci))
    cols = ["region", "n", "prevalence", "TPR", "FPR", "TPR_CI", "FPR_CI"]
    return pd.DataFrame(out, columns=cols)

report = slice_report(test_df, "region", threshold=0.5, level=0.95)
print(report[["region", "n", "prevalence", "TPR", "FPR"]])
```

A fenti kimenet **szemléltető**, mert a tényleges teszt-predikciók a helyi labor futtatásából származnak. Az ellenőrző invariantok: a három `n` összege a teljes teszt mérete; minden szeletre `TP+FP+TN+FN = n`; a `TPR_CI` és `FPR_CI` határai mindig [0,1]-en belül vannak; a `y_pred` előállítása konzisztens a `threshold=0.5` értékkel.

Az érzékenységi elemzés kódja:

```
# 1) Küszöb-érzékenység
for t in [0.35, 0.65]:
    rep_t = slice_report(test_df, "region", threshold=t, level=0.95)
    print(f"threshold={t}")
    print(rep_t[["region", "n", "TPR", "FPR"]])

# 2) Bonferroni-korrigált CI
rep_bonf = slice_report(test_df, "region", threshold=0.5, level=0.9833)
print(rep_bonf[["region", "TPR_CI", "FPR_CI"]])

# 3) Ügyfélszintű klaszterezett bootstrap a TPR standard hibájához
def cluster_bootstrap_se(frame, group_col, n_boot=1000, seed=20261020):
    rng = np.random.default_rng(seed)
    groups = frame[group_col].unique()
    se = []
    for _ in range(n_boot):
```

```
        sampled_groups = rng.choice(groups, size=len(groups), replace=True)
        boot = pd.concat([frame[frame[group_col] == g] for g in sampled_groups])
        tp = ((boot.y_true == 1) & (boot.y_pred == 1)).sum()
        fn = ((boot.y_true == 1) & (boot.y_pred == 0)).sum()
        se.append(tp / (tp + fn) if (tp + fn) > 0 else np.nan)
    return np.nanstd(se)

# Feltéve, hogy a test_df tartalmaz customer_id oszlopot
se_cluster = cluster_bootstrap_se(test_df, "customer_id")
print("Klaszterezett bootstrap SE a TPR-re:", se_cluster)
```

A bootstrap az ügyfél szintjén mintavételez, mert a rendelések nem függetlenek ugyanazon ügyfélnél (14.3. alfejezet). A kód nem futtatott; a seed megadása a reprodukálhatóságot szolgálja, de a numerikus kimenet hardverfüggő lehet.

26.3.5 A labor értelmezése

A táblázatot a gyakorlati célhoz képest olvassuk: nem csak azt kérdezzük, hogy a TPR „rossz-e”, hanem azt, hogy a CI milyen széles, és a küszöb változtatása hogyan hat. A south régió kis n esetén a TPR CI felső határa is fontos: ha a CI [0,00; 0,79], akkor a „rossz modell” állítás nem erős, mert a minta nem dönti el a 0,3 és 0,7 közötti TPR-t sem. A Bonferroni-korrigált CI-k szélesebbek, ezért kevésbé hajlamosak hamis régiós különbséget mutatni.

26.3.5.1 Minimum minta policy és privacy korlátok

A szeletelt jelentés akkor felelős, ha a kis cellákra nem ad háromtizedes pontmetrikát. A szabály három részből áll:

1. **Karonkénti nevező.** Minden szeletre külön kell közölni a pozitívok (n_+) és a negatívok (n_-) számát, mert a TPR nevezője n_+, az FPR-é n_-; a teljes n önmagában félrevezet.
2. **Előre rögzített minimum.** Például $n_+ \geq 10$ és $n_- \geq 10$ (vagy célzottabb esetben 20–30) szükséges a pontmetrika közléséhez. Ha valamelyik hiányzik, a jelentés „elégtelen adat” státuszt és összevont vagy hierarchikus becslést ad, nem 0,00-s TPR-t.
3. **Privacy.** Ha a szelet személyes attribútum (életkor, régió, jövedelmi sáv) szerinti, a kis cellák azonosíthatnak egyéneket; ilyenkor a cellaszámot is el kell nyomni (például $n < 20$ esetén nem közöljük), és csak aggregált, k-anonimitást teljesítő bontást adunk. A nyers `customer_id` soha nem kerül a nyilvános jelentésbe.

Példatábla:

Szelet	n	n_+	n_-	TPR	95% CI	Státusz
north	600	120	480	0,78	0,70–0,85	közölhető
central	300	60	240	0,67	0,54–0,78	közölhető
south	100	5	95	–	–	elégtelen adat ($n_+ < 10$)

A south TPR-jét nem adjuk meg pontbecslésként; legfeljebb annyit mondunk, hogy a megfigyelt 5 pozitívból hány találat volt, és hogy a CI nagyon széles. A kis cella miatt a régiós összehasonlítás sem támaszkodhat a south pontértékére.

```python
def slice_status(n_pos, n_neg, n_min=10):
    if n_pos < n_min or n_neg < n_min:
        return "insufficient_data"
    return "ok"

def reportable_count(n, privacy_min=20):
    return n if n >= privacy_min else "suppressed"
```

A `reportable_count` a privacy-küszöb alatti cellákat elnyomja; a `slice_status` a statisztikai minimumot jelzi. A két küszöb különböző: a statisztikai az értelmezhetőséget, a privacy az azonosíthatóságot védi.

26.3.6 Feladatok

1. **Értelmezés.** A teszt összesített accuracy 0,82. Magyarázd el, miért nem zárható ki, hogy a south régióban a modell gyakorlatilag nem talál el egyetlen késést sem, ha ott a prevalencia 5% és $n = 100$.
2. **Implementálás.** Számítsd ki kézzel a minimális példa central régiójának TPR Wilson 95% CI-ját a fenti képlet szerint, és hasonlítsd össze a kód kimenetével. Ellenőrizd, hogy a CI tartalmazza a 2/3 pontbecslést.
3. **Hibadiagnózis.** A labor kód valaki által átírt változatában a `y_pred` a kalibrálatlan modell `score` oszlopából készült `score >= 0.5` szerint, míg a `p_hat` a kalibrált valószínűség. Mi a következménye a szeletelt TPR/FPR összehasonlításra? Adj egy konkrét javítást.
4. **Transzfer.** A minimális példában változtasd a küszöböt 0,35-re. Mennyi lesz a central régió TPR-je és FPR-je? Milyen trade-off figyelhető meg a south régióban?

26.3.7 Megoldások és indoklás

1. A prevalencia 5% azt jelenti, hogy 100 rendelésből csak 5 késő. Ha a modell mindenkit negatívnak mond, az accuracy 95%, mert a 95 nem későt eltalálja, de a TPR 0. Az összesített accuracy-t a gyakori osztály dominálja; a ritka pozitív eset hibája eltűnik benne. Ezért szeletenként kell TPR-t és FNR-t is nézni, nem csak accuracy-t.

2. A central régióban TP=2, FN=1, tehát $x = 2$, $n = 3$, $\hat{p} = 2/3$. $z = 1{,}96$ mellett:

- $D = 1 + 1{,}96^2/3 \approx 2{,}2807$;
- $c = (2/3 + 1{,}96^2/(2 \cdot 3))/D \approx 1{,}307/2{,}2807 \approx 0{,}573$;
- $m = 1{,}96\sqrt{(2/3)(1/3)/3 + 1{,}96^2/(4 \cdot 9)}/D \approx 1{,}96 \cdot 0{,}425/2{,}2807 \approx 0{,}365$.

CI: [0,208, 0,938]. A 2/3 pontbecslés benne van. **Ellenőrző invariant:** a CI szimmetrikus a pontbecslésre? Nem, a Wilson CI a pontbecslés körül aszimmetrikus lehet, de a határok [0,1]-ben vannak; a $c - m$ és $c + m$ értékek a fenti számítást adják.

3. A kalibrálatlan `score` és a kalibrált `p_hat` nem azonos skálán van; a `score >= 0.5` döntési szabály nem feltétlenül felel meg a kalibrált 0,5-ös küszöbnek. Így a szeletelt TPR/FPR a rossz döntési szabályra vonatkozik, a kalibrált valószínűségek értelmezése elveszik. Javítás: `y_pred =`

`(p_hat >= 0.5).astype(int)`; ha a küszöböt a score-on akarjuk, akkor a kalibrációt is ahhoz kell igazítani, de a 26.3. alfejezet labor a kalibrált valószínűséget használja.

4. A central régióban a p_hat értékek [0,70, 0,20, 0,40, 0,90]; a 0,35 küszöbbel a harmadik sor (0,40) is pozitív lesz. Az y_true [1,0,1,1], így TP=3, FN=0, TPR=1,00; FP=0, TN=1, FPR=0,00. A south régióban p_hat [0,40, 0,10]; a 0,40 >= 0,35, így TP=1, FN=0, TPR=1,00; a második sor negatív marad, FPR=0,00. A trade-off: a küszöb csökkentése növelte a TPR-t, de a north régióban a 0,30-as p_hat továbbra is negatív, tehát ott nincs változás; más adatokon a küszöb csökkentése az FPR-t is növelné.

26.3.8 Gyakori tévedés

„Ha a selection rate egyenlő, akkor a modell fair." Ez nem igaz. Tekintsünk két csoportot. A modell mindkét csoportban mindenkit pozitívnak prediktál: az SR mindkettőben 1, tehát a demographic parity teljesül. De az egyik csoportban 90 negatív ügyfél van, ott az FPR 1, és a PPV megegyezik a prevalenciával; a modell a negatívokat tömegesen tévesen pozitívnak jelöli. Az SR egyenlősége nem mond semmit a TPR/FPR hibákról. Hasonlóan, az equal opportunity (TPR egyenlőség) mellett az FPR különbözhet, ahogyan a fogalmi részben láttuk. Ezért a 26.3. alfejezet következtetése mindig több metrika és a CI együttes figyelése. Visszautalás 25.3. alfejezet: ahogyan a sok feature közötti keresés hamis felfedezést szül, a sok szelet közötti keresés is; a Bonferroni-korrekció ennek ellenszere.

Önellenőrzés: ha egy szeletben $n = 2$ és $\hat{p} = 0{,}5$, a Wilson CI a fenti képlettel nem lehet szűkebb, mint a normál approximáció $p \pm 1{,}96\sqrt{p(1-p)/n}$; számold ki mindkettőt, és ellenőrizd, hogy a Wilson határok [0,1]-ben maradnak.

26.3.9 Rövid összefoglaló

A szeletelemzés a modell értékelését kategorikus részekre bontja, és minden szeletben hibamátrixot, prevalenciát, TPR-t, FPR-t és ezek konfidenciaintervallumát közli. A kis csoportokban a Wilson CI jobb, mint a normál approximáció, mert a határokat [0,1]-ben tartja. Ha több szeletet vizsgálunk, a többszörös összehasonlítás miatt korrigálni kell a szintet; a Bonferroni-korrekció három régióra 98,3%-os CI-t ad. A fairness-metrikák, mint a selection rate paritás vagy az equal opportunity, önmagukban nem bizonyítják a modell elfogadhatóságát, mert a TPR/FPR trade-off és a PPV prevalenciafüggése külön figyelmet igényel. A következő 26.4. alfejezet modellkártya ugyanezt a szeletelt bizonytalanságot és korlátot foglalja dokumentált, átadható formába.

26.4 Robusztusság és korlátokkal írt modellkártya

Belépő · szint: közép · idő: 60 perc · eszköz: scikit-learn

Előfeltétel: 26.3. alfejezet · Célkészség: Robusztusság és korlátokkal írt modellkártya · Kimenet: működő kód és a labor ellenőrző tesztje.

26.4.1 Cél és konkrét helyzet

A 26.3. alfejezetben a modell hibáit szeletekre bontva vizsgáltuk. Most más irányból kérdezünk: meddig bízhatunk a modellben, ha a jövőbeli adat vagy a mérési környezet eltolódik? A Northstar rendeléskori késésmodellt használjuk; a megfigyelési egység az `order_id`, a bemenet egy rendelés pillanatában ismert feature-sor, a cél a `target_late` indikátor (1, ha `delivery_days > 7`). A már rögzített teszthalmazon két perturbációt szimulálunk: a `south` régió eltűnését és a `distance_km` +15%-os mérési eltolódását. Az eredmény egy stresszteszt-jegyzőkönyv és egy kitöltött modellkártya. Előzetes becslés: a kategóriaeltűnés kisebb, a distance-eltolódás nagyobb log loss változást okoz, mert a távolság erősebb prediktor; a tényleges értéket a tanuló gépe adja.

26.4.2 Fogalmi magyarázat

Robusztusság (robustness). A modell robusztussága azt méri, mennyire marad stabil a teljesítménye, ha a bemenet eloszlása, mérési skálája vagy hiányzási mintázata megváltozik. Rögzített teszthalmazon a referencia log loss:

$$L_{\text{ref}} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\right],$$

ahol n a teszt sorainak száma, $y_i \in \{0,1\}$ a tényleges címke, $\hat{p}_i \in (0,1)$ a prediktált késési valószínűség. A perturbált adatra számított L_{pert} különbsége $\Delta L = L_{\text{pert}} - L_{\text{ref}}$. Az érzékenységi mutatót a perturbáció nagyságához viszonyítjuk: $S = \Delta L/|\delta|$, ahol δ a relatív eltolódás, például 0,15. A log loss mértékegysége nat; S mértékegysége nat/szorzó. A ΔL csak a specifikált perturbációra adott mért válasz, nem általános garancia.

Hiányzási mintázat eltolódása (missingness shift). A hiányzó értékek arányának vagy mechanizmusának megváltozása a scoring időben. Ha a tanítóban a `distance_km` hiánytalan, a pipeline mediánnal pótol; ha a scoringban a hiány pont a nagy távolságú rendeléseknél jelenik meg, a medián alábecsüli a valódi értéket. Kézi kontroll: három tanító távolság 10, 20, 30 km, a medián 20. A scoringban 10, 20, hiány, a hiányzó valódi értéke 50; a pótolt 20, tehát a modell 30 km-rel rövidebb utat lát. A hiányzás okának azonosítása a 13.4. alfejezet adatminőségi vizsgálata, nem a modell feladata; a modell csak a pótlási szabályt örökli.

Sémaváltozás (schema change). A bemeneti oszlopok nevének, típusának, kategóriáinak vagy mértékegységének megváltozása. Példák: új `region` érték, a `basket_minor` string dtype, a `distance_km` mérföldben. A scikit-learn `OneHotEncoder` paramétere dönt: `handle_unknown="error"` hibát dob, `handle_unknown="ignore"` csendben nullázza az ismeretlen dummy oszlopokat, és a sort a referencia szintre teszi. A 25.4. alfejezet adatszerződése az első védvonal; a sémavalidációt a `predict` előtt kötelező lefuttatni.

Stresszteszt. Meghatározott perturbációk sorozata, amelynek célja a töréspont megtalálása, nem a jövőbeli garancia. Példa: a distance_km szorzó $c \in \{0{,}9;\ 1{,}0;\ 1{,}15;\ 1{,}3\}$ mellett kiszámítjuk a log loss-t; ha $c = 1{,}3$-nál ΔL meghaladja az előre rögzített bevezetési küszöböt, a kártya rögzíti: „30%-os distance-eltolódás nem támogatott”.

Modellkártya (model card). Strukturált dokumentáció: azonosító, rendeltetés, bemeneti és kimeneti szerződés, tanítási adat és cutoff, referenciametrika, stressztesztek, alkalmazási tartomány, nem támogatott használat, korlátok. A kártya a felelős átadás eszköze, nem a kód helyettesítője.

26.4.2.1 Kitöltött mintakártya és átvételi kapu

A kártya akkor elfogadható, ha a metrikák mellett a **felelőst, a döntéshozót, az érvényességi körzetet, a verziót, a mért időszakot és a visszavonási tervet** is rögzíti. Példa:

```yaml
model_id: northstar-late-risk
version: 1.0.0
artifact_sha256: "<a mentett pipeline hash-e>"
owner: "Northstar ML csapat (felelős: <név/szerep>)"
decision_owner: "Logisztikai vezető (felelős: <név/szerep>)"
intended_use: "rendeléskori késési kockázat jelzése, 7 napos ígéret mellett"
out_of_scope:
  - "egyedi ügyfél jogi vagy orvosi döntése"
  - "a 7 napostól eltérő ígéret"
training_period: "2025-01-01 - 2025-04-30 (cutoff: 2025-05-01)"
measurement_period: "2025-05-01 - 2025-06-29 (egyszeri teszt)"
primary_metric: "log loss; AUC guardrail"
reference_values: "valid log loss 0.288; teszt log loss <valid + 0.05"
known_vulnerabilities:
  - "distance_km +15% mérési eltolódás: log loss nő, küszöb újravalidálandó"
  - "south régió eltűnése: a tanult kategória hiányzik, fallback + figyelmeztetés"
  - "missingness shift: a medián-imputáció alábecsülhet"
rollback:
  trigger: "teszt log loss > 0.35 vagy a havi monitoringon 2 egymást követő hét romlás"
  action: "az előző verzió visszaállítása; a forgalom 100%-a a régi artefaktumra"
  owner: "Northstar ML csapat"
review_date: "2026-01-31"
change_log: "1.0.0 - első kiadás; a 22.4 protokoll szerint"
```

Az **átvételi kapu** a következő mezőket kötelezőnek tekinti: `model_id`, `version`, `owner`, `decision_owner`, `intended_use`, `out_of_scope`, `training_period`, `measurement_period`, `primary_metric`, `known_vulnerabilities`, `rollback.trigger`, `rollback.action`, `rollback.owner`, `review_date`. Ha bármelyik hiányzik, a kártya nem fogadható el, még akkor sem, ha a metrikák kiválóak. A stresszteszt eredménye a `known_vulnerabilities` mezőbe kerül; a visszavonási küszöb előre rögzített, nem az incidens után kitalált szám.

Jelenség	Mi változik	Northstar példa	Első védvonal
missingness shift	hiányzási mechanizmus	`distance_km` 10% hiány a nagy távolságoknál	adatminőségi monitor, imputálási jegyzőkönyv
schema change	oszlop, dtype, kategória	új `region` érték, forintban érkező `basket_minor`	bemeneti sémavalidáció
measurement shift	mérési skála	`distance_km` +15%	kalibráció, eloszlás-ellenőrzés

A mérési eltolódás kézi kontrollja: ha a tanító `distance_km` átlaga $\hat{\mu}_{\text{train}}$ és szórása $\hat{\sigma}_{\text{train}}$, akkor egy x távolság standardizált értéke $z = (x - \hat{\mu}_{\text{train}})/\hat{\sigma}_{\text{train}}$. A c szorzójú perturbáció után $z' = (cx - \hat{\mu}_{\text{train}})/\hat{\sigma}_{\text{train}}$, a logit változása $\beta_d(c-1)x/\hat{\sigma}_{\text{train}}$, ahol β_d a distance együtthatója a logisztikus modellben. Ha $\beta_d > 0$, a nagyobb távolság nagyobb késési valószínűséget ad.

26.4.3 Teljes minimális munkapélda

A `lab/` projektgyökérből futtatandó script a szerződés szerinti fájlokat használja. A kód a splitet maga számítja a cutoff és a label-availability szabály alapján, így nem igényel ismeretlen artifactot.

A `random_state=42` a logisztikus optimalizáló indulását rögzíti; az adatgeneráló seedje 20261020, a kettő külön szerep.

```
# s104_robustness.py -- futtatás a lab/ projektgyökérből:
# python s104_robustness.py
import json
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Betöltés és feature-összerakás
orders = pd.read_csv("data/orders.csv", parse_dates=["order_ts", "actual_delivery_at",
                                                     "label_available_at"])
customers = pd.read_csv("data/customers.csv", parse_dates=["signup_date"])
df = orders.merge(customers[["customer_id", "signup_date"]], on="customer_id")
df["account_age_days"] = (df["order_ts"] - df["signup_date"]).dt.days
# target_late utólag ismert, prediktorból mindig kizárva; itt kizárólag tanító címke.

feature_cols = ["distance_km", "priority_flag", "basket_minor",
                "quantity_sum", "account_age_days"]
category_cols = ["segment", "region", "acquisition_channel"]

# 2. Időbeli split a szerződés alapján
cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")
train = df[(df["order_ts"] < cutoff) & (df["label_available_at"] < cutoff)].copy()
test = df[df["order_ts"] >= cutoff].copy()
# purged: order_ts < cutoff, de label_available_at >= cutoff; kimarad a példából.

X_train, y_train = train[feature_cols + category_cols], train["target_late"]
X_test, y_test = test[feature_cols + category_cols], test["target_late"]

# 3. Pipeline és tanítás
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), feature_cols),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), category_cols),
])
model = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(max_iter=1000, random_state=42)),
])
model.fit(X_train, y_train)

# 4. Referencia metrika
p_ref = model.predict_proba(X_test)[:, 1]
ref = {"auc": roc_auc_score(y_test, p_ref),
       "log_loss": log_loss(y_test, p_ref)}

# 5. Stressz teszt 1: south régió eltűnik a scoringból
X_region = X_test.copy()
X_region["region"] = X_region["region"].replace("south", "central")
p_reg = model.predict_proba(X_region)[:, 1]
region_result = {"auc": roc_auc_score(y_test, p_reg),
                 "log_loss": log_loss(y_test, p_reg)}

# 6. Stressz teszt 2: distance_km mérési eltolódás +15%
X_dist = X_test.copy()
X_dist["distance_km"] = X_dist["distance_km"] * 1.15
```

```
p_dist = model.predict_proba(X_dist)[:, 1]
dist_result = {"auc": roc_auc_score(y_test, p_dist),
               "log_loss": log_loss(y_test, p_dist)}

print(f"test rows: {len(X_test)}")
print(f"reference: AUC={ref['auc']:.3f} log_loss={ref['log_loss']:.3f}")
print(f"region removed: AUC={region_result['auc']:.3f} "
      f"log_loss={region_result['log_loss']:.3f}")
print(f"distance +15%: AUC={dist_result['auc']:.3f} "
      f"log_loss={dist_result['log_loss']:.3f}")

# 7. Kézi invariant-ellenőrzés
assert X_region.shape == X_test.shape
assert X_dist.shape == X_test.shape
assert X_region["region"].nunique() == 2  # south valóban eltűnt
np.testing.assert_allclose(X_dist["distance_km"].mean(),
                           X_test["distance_km"].mean() * 1.15, rtol=1e-12)
print("shape és átlag-skála invariant rendben")
```

A kimenet szemléltető, nem e könyv futási jegyzéke; a tanuló gépén az értékek eltérhetnek:

```
test rows: 786
reference: AUC=0.72 log_loss=0.41
region removed: AUC=0.71 log_loss=0.42
distance +15%: AUC=0.65 log_loss=0.49
shape és átlag-skála invariant rendben
```

Eredmény értelmezése. A `south` régió eltűnése kicsi ΔL értéket ad, mert a régió gyenge prediktor; a +15%-os distance-eltolódás nagyobbat, mert a távolság a késés fő mozgatója. A shape-invariant garantálja, hogy nem véletlenül dobtunk ki sorokat; a distance-átlag 1,15-szeresre növekedése a mérési perturbáció pontos betartását ellenőrzi. A log loss növekedése itt a kalibrálatlanság és a rosszabb rangsorolás keveréke; a két komponenst a 26.3. alfejezet szeletelt vagy a kalibrációs görbe elemzés külön vizsgálhatja.

A fenti két mérésből kitöltött modellkártya vázlata:

- **Azonosító:** `northstar_late_v3`, 2026-10-20.
- **Rendeltetés:** rendelés pillanatában a 7 napot meghaladó kézbesítés valószínűségének becslése batch scoringra.
- **Bemenet:** `distance_km` (km), `priority_flag` (0/1), `basket_minor` (EUR-cent), `quantity_sum` (db), `account_age_days` (nap), `segment`, `region`, `acquisition_channel`; cutoff: 2025-05-01.
- **Kimenet:** $\hat{p}_i \in [0, 1]$, késési valószínűség; a döntési küszöböt az üzleti folyamat külön választja.
- **Tanítási adat:** a cutoff előtti, címkével már rendelkező rendelések; a purged sáv kimarad.
- **Referencia:** a futtatáskor mért AUC és log loss a rögzített teszten.
- **Stressztesztek:** `south` régió eltűnése $\Delta L \approx 0{,}01$; `distance_km` +15% $\Delta L \approx 0{,}08$ (szemléltető).
- **Nem támogatott használat:** új, a tanítóban nem látott régió vagy csatorna; `distance_km` mérföldben; `basket_minor` forintban; hiányzó `distance_km` 5% feletti scoring arány; oksági következtetés a szállítási díj hatásáról.
- **Korlát:** a stressztesztek csak a felsorolt perturbációkat fedik; más eloszlásváltozásra nincs bizonyíték.

A kártya „Nem támogatott használat” mezője konkrét, mérhető feltételeket tartalmaz; az általános jogi szöveg nem helyettesíti a technikai határt.

26.4.4 Feladatok

1. **Értelmezés.** A +15%-os distance-stresszteszt eredménye elfogadható. Miért nem állíthatja a kártya, hogy a modell minden jövőbeli eltolódásra robusztus? Fogalmazzon egy konkrét ellenpélda-perturbációt, amelyre a fenti teszt nem terjed ki, és indokold, miért törné meg a modellt.
2. **Implementálás.** Egészítsd ki a scriptet hiányzó-distance stressztesszttel: a teszt 10%-ában (`np.random.default_rng(7)` használatával) állítsa a `distance_km` értéket `NaN`-re, a numerikus ágba tegyen `SimpleImputer(strategy="median")` lépést a `StandardScaler` elé, majd számítsd ki az AUC-t és log loss-t a referencia értékekkel együtt. A medián a teljes tanító `distance_km` oszlopából származzon.
3. **Hibadiagnózis.** A scoring adatban megjelenik egy `island` régió. A pipeline `OneHotEncoder(handle_unknown="error")` beállítással fut. Mi történik a `predict` hívásnál? Adj diagnózist és két lehetséges, dokumentált megoldást a modellkártya szempontjából.
4. **Megváltozott feltétel.** A scoring rendszer elkezdi a `distance_km` értéket mérföldben küldeni (1 mérföld = 1,609 km). A kártya melyik mezője tiltja ezt, és milyen bemeneti ellenőrzést kell bevezetni, hogy a hiba ne jusson el a modellhez? Adj egy konkrét drift-ellenőrző szabályt a tanító medián és a scoring medián alapján.

26.4.5 Megoldások és indoklás

1. megoldás. A stressztesz egy rögzített perturbációra adott mérés; a jövőbeli eltolódások tere nem fedhető le véges teszttel. Ellenpélda: a `basket_minor` értékek forintban kezdenek érkezni a scoringba. A tanító `StandardScaler` euró-cent eloszlásra illeszkedett, ezért a forintban érkező, nagyságrendileg 400-szoros értékek a numerikus feature-öket szélsőséges standardizált értékekké teszik, a logisztikus modell predikciói 0 vagy 1 közelébe torzulnak, és a log loss drasztikusan nő. A stressztesz nem tartalmazta ezt a skálaváltást, ezért nem bizonyíték rá.

2. megoldás. A numerikus ág módosítása:

```
from sklearn.impute import SimpleImputer

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
preprocessor = ColumnTransformer([
    ("num", numeric_pipe, feature_cols),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), category_cols),
])
```

A hiányzó teszt előállítása:

```
rng = np.random.default_rng(7)
X_missing = X_test.copy()
mask = rng.random(len(X_missing)) < 0.10
X_missing.loc[mask, "distance_km"] = np.nan

p_miss = model.predict_proba(X_missing)[:, 1]
missing_result = {"auc": roc_auc_score(y_test, p_miss),
                  "log_loss": log_loss(y_test, p_miss)}
```

Az ellenőrző invariant: `mask.sum()` körülbelül a sorok 10%-a, és a nem maszkolt sorok distance értékei változatlanok. A log loss várt iránya: a mediánnal pótolt hiányzó értékek szisztematikusan alábecsülik a nagy távolságú rendeléseket, ezért a modell kevésbé valószínűsíti a késést azoknál a

soroknál, a log loss általában nő; a pontos mértéket a tanuló saját futása adja. A túl alacsony imputálási aránynál a változás nem szignifikáns, ezért a kártya a scoring 5% feletti hiányzást eleve tiltja.

3. megoldás. Az `OneHotEncoder.transform` `ValueError` kivételt dob, mert a scoringban olyan kategória jelenik meg, amelyet a tanító nem látott. A diagnózis: sémaváltozás, nem pedig modellhiba. Két dokumentált megoldás: (a) a bemeneti validátor elutasítja az ismeretlen kategóriájú sort, és a rendelést emberi felülvizsgálatra küldi; ekkor a modell csak a támogatott tartományon prediktál. (b) Ha az üzleti döntés az ismeretlen kategória lenémítása, akkor `handle_unknown="ignore"` beállítást használunk, és a modellkártya rögzíti: „Az ismeretlen régió a referencia szintre kerül; a predikció félrevezető lehet, ezért az ilyen sorokat külön riportálni kell.” A csendes váltás dokumentáció nélkül azt a hamis látszatot kelti, hogy a modell támogatja az új régiót.

4. megoldás. A kártya *Bemeneti szerződés* mezője tiltja: `distance_km` numerikus, kilométerben, pozitív; a tanító mediánt és IQR-t a futtatás nyomtatja. A bemeneti ellenőrzés a scoring első 500 sorában kiszámítja a mediánt; ha

$$\left| \ln \left(\frac{\text{median}_{\text{score}}}{\text{median}_{\text{train}}} \right) \right| > 0{,}1,$$

akkor a scoring leáll és riaszt, mert a feltételezett azonos skála sérül. A mérföldes adat 1,609-szeres mediánt adna, a log-hányados körülbelül 0,48, ami azonnal kiszűri. A 36.1. alfejezet monitoring ezt a szabályt automatizálja a batch folyamatban.

26.4.6 Gyakori tévedés ellenpéldával

A leggyakoribb hiba a stresszteszt zöld eredményének általános garancialként kezelése. A 22.4. alfejezet modellcsalád-választásnál azt tanultuk, hogy a tesztet egyszer használjuk a kiválasztott modell jelentéséhez; a stresszteszt ugyanígy csak a specifikált perturbációról ad bizonyítékot. Ha a +15% distance-teszt elfogadható, az nem jelenti, hogy a modell a +25%-os eltolódásra, a forintban küldött kosárértékre vagy az új régióra is megbízható. A kártya „Ki nem próbált” szekciója ezért kötelező; a modell akkor felelős átadású, ha pontosan megmondja, mit nem tud.

Önellenőrzés. Ellenőrizd, hogy a perturbált teszt shape-je egyezik a referencia teszttel, a célvektor érintetlen maradt, a perturbáció csak a megcélzott oszlopot változtatta, és a modellkártya tiltásai konkrét, mérhető feltételeket tartalmaznak-e. Ha a kártya csak „általános környezeti változásra” hivatkozik mérés nélkül, az nem teljesíti a feladatot.

A robusztussági tesztben a perturbáció nagysága és a modellhibát mérő küszöb előre rögzítendő. A szintetikus régiótörlés nem bizonyít teljesítményt valódi eloszláseltolódás mellett.

26.4.7 Rövid összefoglaló

A robusztussági jegyzőkönyv azt méri, hogyan változik a referencia log loss specifikált perturbációk hatására: a régió eltűnése kicsi, a +15%-os distance-eltolódás nagyobb ΔL értéket ad. A missingness shift, a séma- és mértékegység-változás külön meghibásodási mód; mindegyikhez konkrét bemeneti validáció tartozik. A modellkártya ezeket a határokat a „Nem támogatott használat” mezőben rögzíti, így a jövőbeli felhasználó nem feltételezi a nem igazolt tartományt. A stresszteszt nem bizonyíték minden eloszlásváltozásra, csak a felsorolt perturbációra. A következő lépésben a 27.1. alfejezet szöveges baseline-oknál ugyanezt a gondolatot alkalmazzuk a szöveg-előfeldolgozás sérüléseire.

További olvasmány és elsődleges dokumentáció: [SklearnCalibration] scikit-learn Calibration, https://scikit-learn.org/stable/modules/calibration.html (hozzáférés: 2026-10-21).

27 Szöveg, embedding és LLM-kimenetek értékelése

27.1 Szöveges baseline és ritka mátrix

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, Python

Előfeltétel: 26.4. alfejezet · Célkészség: Szöveges baseline és ritka mátrix · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

27.1.1 Helyzet és cél

Egy belső support-eszköz rövid szöveges üzeneteket kap. A döntés: valódi hibabejelentés-e az üzenet (`support`), vagy általános megjegyzés, visszajelzés (`nem_support`). A bemenet egy-egy rövid szöveges dokumentum, a megfigyelési egység az üzenet, a kimenet pedig egy bináris címke. Ebben az alfejezetben egy olyan szöveges baseline-t építünk, amely elkülönített tanító- és tesztkorpusszal dolgozik, és egy ritka TF-IDF-mátrixból lineáris osztályozóval dönt. A tanulási cél nem a nagy pontosság, hanem egy ellenőrizhető, reprodukálható alapvonal.

Előzetes becslés: a példaként használt, mesterségesen egyszerű magyar mondatokon a kulcsszavak (`frissítés`, `driver`, `leáll`) alapján a támogatási jegyek valószínűleg jól elkülöníthetők. A minta azonban olyan kicsi, hogy az eredményt inkább folyamat-ellenőrzésként, mintsem általánosítható modellminőségként érdemes értelmezni.

27.1.2 Tokenizálás, TF-IDF és n-gram

A **tokenizálás** (tokenization) során a nyers szöveget kisebb egységekre, úgynevezett tokenekre bontjuk. A legegyszerűbb változat a szóközök és írásjelek mentén való felosztás. A scikit-learn `TfidfVectorizer` alapból hozzávetőleg a `(?u)\b\w\w+\b` reguláris kifejezést használja: Unicode szóhatárokat keres, és eldobja az egyetlen karakterből álló tokeneket. A magyar toldalékok miatt a `frissítés`, `frissítése`, `frissült` külön tokenként jelenik meg; ez a korlátozás a későbbi hibaelemzéshez tartozik.

Az **n-gram** a szomszédos tokenek rögzített hosszúságú sorozata. Az `ngram_range=(1,2)` beállítás unigramokat és bigramokat is létrehoz: a `driver frissítése` szövegből például `driver`, `frissítése`, `driver frissítése` lesz. A bigramok segítenek megőrizni a szórendből fakadó jelentést, például a `nem indul` kifejezést.

A **TF-IDF** (term frequency–inverse document frequency) egy dokumentumot valós vektorrá alakít. Legyen N a tanítókorpusz dokumentumainak száma, $\mathrm{tf}(t, d)$ a t token előfordulási gyakorisága a d dokumentumban, $\mathrm{df}(t)$ pedig azoknak a dokumentumoknak a száma, amelyekben t legalább egyszer szerepel. A scikit-learn simított inverz dokumentumgyakorisága:

$$\mathrm{idf}(t) = \log \frac{N+1}{\mathrm{df}(t)+1} + 1,$$

a súly pedig:

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t).$$

A vektort a scikit-learn soronként L2-normára normalizálja, így a dokumentum hosszának nyers hatása csökken. A TF-IDF logikai szerepe: a ritka token nagyobb súlyt kap, a minden dokumentumban előforduló token pedig közelítőleg 1-et; a logaritmus megfékezi a nagy gyakoriságkülönbségeket.

Kézi ellenőrző példa. Legyen három dokumentum:

- d_1: `driver frissítés leáll`
- d_2: `driver frissítés`
- d_3: `nyomtató hiba`

A tokenhalmaz: `driver`, `frissítés`, `leáll`, `nyomtató`, `hiba`, tehát $V = 5$. A dokumentumgyakoriságok: $\mathrm{df}(\text{driver}) = 2$, $\mathrm{df}(\text{frissítés}) = 2$, a többié 1. Ezért:

$$\mathrm{idf}(\text{driver}) = \log \frac{4}{3} + 1 \approx 1.288, \qquad \mathrm{idf}(\text{leáll}) = \log \frac{4}{2} + 1 \approx 1.693.$$

A d_1 vektora `(1.288, 1.288, 1.693, 0, 0)`, L2-normája körülbelül 2.486, így a normalizált vektor körülbelül `(0.518, 0.518, 0.681, 0, 0)`. A d_3 normalizált vektora `(0, 0, 0, 0.707, 0.707)`. A skalárszorzat d_1 és d_3 között nulla, míg d_1 és d_2 között körülbelül 0.732. A ritka reprezentáció ezt a nagy, de nagyrészt nullákat tartalmazó vektorteret használja.

A képlet a scikit-learn alapbeállításaival pontosan egyezik: `smooth_idf=True` adja a számláló $N+1$ és a nevező $\mathrm{df}(t)+1$ tagját, `sublinear_tf=False` miatt a $\mathrm{tf}(t,d)$ a nyers előfordulásszám (nem $1+\log$), `norm='l2'` pedig soronként normalizál. Az `use_idf=True`, az `ngram_range=(1,1)` és a `min_df=1` szintén alapérték. Az `idf` értékeket a vektorizáló kizárólag a tanítókorpuszból számolja, a `transform` már a tanult értékeket használja; ezért a kézi számítás és a kód csak akkor egyezik, ha a `fit` a teljes vizsgált korpuszon történt.

A kézi példa ellenőrzése kóddal (a labor `uv.lock` fájlja által rögzített scikit-learn 1.9.1):

```
vec = TfidfVectorizer()  # smooth_idf=True, sublinear_tf=False, norm='l2'
X = vec.fit_transform([
    "driver frissítés leáll",
    "driver frissítés",
    "nyomtató hiba",
])
print(list(vec.get_feature_names_out()))
print([round(v, 4) for v in vec.idf_])
print(X.toarray().round(3))
```

Kimenet:

```
['driver', 'frissítés', 'hiba', 'leáll', 'nyomtató']
[1.2877, 1.2877, 1.6931, 1.6931, 1.6931]
[[0.518 0.518 0.    0.681 0.   ]
 [0.707 0.707 0.    0.    0.   ]
 [0.    0.    0.707 0.    0.707]]
```

Az `idf_` sorrendje a `get_feature_names_out()` sorrendjét követi (ábécé sorrend), nem a dokumentumbeli előfordulását; a kézi vektorban ezért kell a tokeneket név szerint párosítani. A kimenet a kézi `(1.288, 1.288, 1.693, 0, 0)` vektorral és a normalizált sorokkal négy tizedesjegyig egyezik.

A **sparse**, azaz ritka mátrix (`scipy.sparse`) csak a nemnulla elemeket tárolja. Ha V több ezer vagy több százezer, akkor egy sűrű `(dokumentumok, tokenek)` mátrix memóriaigénye lehetetlenül nagy lenne. A ritka tárolásnál a műveletek hatékonysága is jobb, mert a skalárszorzatot csak a nemnulla elemeken kell kiszámítani. A tanult vektorizáló a `fit` hívás után rögzíti a tanítószókincset; a `transform` minden új dokumentumot ugyanezen dimenziójú ritka vektorrá képez.

A **lineáris osztályozó** (linear classifier) a dokumentumvektor és egy súlyvektor belső szorzatát számolja:

$$s(d) = w_0 + \sum_{j=1}^{V} w_j x_j .$$

A logisztikus regresszió ebből a `sigmoid` függvénnyel valószínűséget készít: $p(d) = 1/(1 + e^{-s(d)})$. A súlyok előjele és nagysága a 26.1. alfejezet szerint lokálisan magyarázható; a nagy abszolút értékű w_j azt jelzi, hogy az adott token felfelé vagy lefelé tolja a döntést. A ritka szöveges mátrixokon a lineáris modellek gyakran értelmes első baseline-t adnak, mert a döntési felület egyszerű és gyorsan tanítható.

A **korpuszfelosztás** (corpus split) a szöveges feladatoknál is ugyanazt a szivárgásmentes elvet követi, mint a táblás adatoknál: a tanító- és tesztkorpust a vektorizáló illesztése előtt kell szétválasztani. Ha egy dokumentum két példányban szerepel, akkor a tanító- és tesztkészlet közötti ismétlődés optimista becslést adhat; a felosztásnál használt véletlen mag (`random_state`) pedig reprodukálhatóvá teszi a munkamenetet.

27.1.3 Teljes minimális munkapélda

Az alábbi példa tizenkét rövid, szintetikus magyar mondatot használ. A cél a `support` címke előrejelzése. A kód a tanítóvektorizálót csak a tanítószövegen illeszti, a tesztet pedig csak transzformálja.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report

train_texts = [
    "Szerver leállt az éjszakai frissítés után.",
    "Nem tudok belépni a VPN-be a jelszófrissítés óta.",
    "A nyomtató hibát jelez a driver frissítése miatt.",
    "Frissítés után nem indul el az alkalmazás.",
    "A heti riport elkészült és megosztva.",
    "Köszönöm a gyors segítséget, minden rendben.",
    "A csapatmegbeszélés jövő héten lesz.",
    "Megkaptam a számlát, köszönöm.",
]
train_labels = [1, 1, 1, 1, 0, 0, 0, 0]  # 1: support

test_texts = [
    "A levelező nem indul a mai frissítés után.",
```

```
    "Kérem a következő találkozó időpontját.",
    "Driver-frissítés óta nem nyomtat a nyomtató.",
    "A prezentáció diái elkészültek.",
]
test_labels = [1, 0, 1, 0]

pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=1)),
    ("clf", LogisticRegression(max_iter=1000, random_state=42)),
])

pipeline.fit(train_texts, train_labels)
pred = pipeline.predict(test_texts)

print("Tanítószókincs nagysága:", len(pipeline.named_steps["tfidf"].vocabulary_))
print("accuracy (szemléltető kimenet):", accuracy_score(test_labels, pred))
print(classification_report(test_labels, pred, target_names=["nem_support", "support"]))
```

A kód a projektgyökérből Linux/macOS alatt például így futtatható, ha a `scikit-learn` már a projekt függőségei között van:

```
uv run python s105_baseline.py
```

27.1.4 Eredmény értelmezése

A kód automatikusan még nem ellenőrzött, ezért a kiírt pontosságot itt nem közöljük mért értékként. A szöveg szerkezete alapján elméletileg várható, hogy a teszt három támogatási kulcsszavas mondata közül kettő erős jelet ad, de a pontos szám a tokenizálás és a véletlen súlyok függvénye. A fontos ellenőrző invariáns a dimenzió: a tanítókorpuszon illesztett vektorizáló a tesztet ugyanannyi oszlopú mátrixszá alakítja, mint a tanítót. Ha a két mátrix oszlopszáma eltér, akkor a vektorizálót véletlenül a tesztre is újraillesztettük, ami szerződésszegés.

A kézi ellenőrzéshez kérdezzük le a vektorizálóból a szókincset:

```
vec = pipeline.named_steps["tfidf"]
print(vec.get_feature_names_out())
```

A várt listában a tanítószövegekből származó tokenek szerepelnek, például `frissítés`, `driver`, `nem`, `indul`; a bigramok közül például `nem indul`, `driver frissítése`. A `transform(test_texts)` során a tesztben előforduló, de a tanítószókincsben nem szereplő tokenek csendben kimaradnak. Ez nem hiba, hanem a predikciós szerződés része: a modell csak azt a jeltípust használhatja, amelyet a tanításkor ismert.

A TF–IDF szókincsét és IDF-súlyait kizárólag a tanítókorpuszon illesszük. Azonos vagy közel azonos dokumentumok tesztbe kerülése külön szivárgási út.

27.1.5 Feladatok

1. **Értelmezés.** Miért kell a `TfidfVectorizer` illesztését kizárólag a tanítókorpuszon elvégezni? Fogalmazz meg legalább két konkrét következményt, ha valaki a tanító- és tesztkorpust összefűzve illeszti a vektorizálót.
2. **Megvalósítás.** Változtasd meg a példában a vektorizálót `min_df=2` értékre, és futtasd újra a tanítást. Mely tokenek tűnhetnek el a szókincsből? Hogyan változhat a tesztelés, ha egy tesztmondat csak ilyen, egyszer előforduló tokent tartalmaz?
3. **Hibakeresés.** Az alábbi vázlat hibás:

```
vec = TfidfVectorizer()
X_all = vec.fit_transform(train_texts + test_texts)
X_train = X_all[:8]
X_test = X_all[8:]
clf = LogisticRegression(max_iter=1000, random_state=42)
clf.fit(X_train, train_labels)
print(clf.score(X_test, test_labels))
```

Nevezd meg pontosan, milyen szivárgást okoz ez a minta, és írd át a kódot úgy, hogy a vektorizáló csak a tanítószövegeket lássa.

4. **Megváltozott feltétel.** Érkezik egy új üzenet: `"A VPN frissítés után nem indul."` Hogyan kell helyesen predikálni a betanított pipeline-nal? Mi történik akkor, ha a mondatban szereplő összes token ismeretlen a tanítószókincsben? Hogyan lehet ezt a helyzetet a 21.4. alfejezet predikciós szerződéshez hasonlóan észlelni?

27.1.6 Megoldások és indoklás

1. A vektorizáló illesztése két dolgot határoz meg: a tokenek szókincsét és az idf-értékeket. Ha a teszt is részt vesz a `fit` lépésben, akkor a modell olyan tokeneket is megismer, amelyeket egy valódi éles rendszerben nem ismerhetne, és a tesztből becsült dokumentumgyakoriság módosítja a súlyokat. Ennek következménye optimista értékelés: a teszt nem a jövőbeli, ismeretlen szókincsű üzenetek nehézségét méri. Másrészt a szivárgó vektorizálóval kapott feature-tér nem azonos azzal, amit egy későbbi, csak tanítóra illesztett változat állítana elő, így a kísérlet nem reprodukálható a termelési szerződéssel.

2. A `min_df=2` beállítás csak azokat a tokeneket és n-gramokat tartja meg, amelyek legalább két tanítódokumentumban előfordulnak. A példában például a `nyomtató` csak egy tanítómondatban szerepel, ezért eltűnne a szókincsből. Ha egy tesztmondat csak ilyen ritka tokenből állna, akkor a vektorizálás üres sort adna; a modell csak az interceptből, azaz az osztályok tanítóbeli arányából tudna dönteni. Ez kis, szintetikus korpuszon különösen élesen mutatja a csekély adat korlátját.

3. A hiba a `fit_transform(train_texts + test_texts)` hívásban van: a vektorizáló a teljes, tesztet is tartalmazó szöveghalmazon tanulja meg a szókincset és az idf-et. A logisztikus regresszió ugyan csak a `train_labels` címkéket látja, de a bemeneti reprezentáció már tesztinformációt tartalmaz, ezért a kiértékelés torz. A javított változat:

```
vec = TfidfVectorizer()
vec.fit(train_texts)
X_train = vec.transform(train_texts)
X_test = vec.transform(test_texts)
clf = LogisticRegression(max_iter=1000, random_state=42)
clf.fit(X_train, train_labels)
print(clf.score(X_test, test_labels))
```

Az ellenőrző invariáns: `X_train.shape[1] == X_test.shape[1]` és `len(vec.vocabulary_) == X_train.shape[1]`.

27.1.6.1 Pipeline és dokumentum-ID ellenőrzés

A vektorizálót és az osztályozót egy `Pipeline`-ba zárva a `fit` csak a tanítószövegeket látja, a `predict` pedig a tanult szókészlettel transzformál:

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2)),
    ("clf", LogisticRegression(max_iter=1000, random_state=42)),
])
pipe.fit(train_texts, train_labels)
vocab_before = set(pipe.named_steps["tfidf"].vocabulary_)
pipe.predict(test_texts)
assert set(pipe.named_steps["tfidf"].vocabulary_) == vocab_before
```

A szókészlet a `predict` után sem változhat; ha változik, valami újraillesztette a vektorizálót. A split **előtt** a dokumentum-azonosítókat és a normalizált szövegeket is ellenőrizni kell: ha ugyanaz a levél vagy ugyanaz a sablonszöveg trainben és tesztben is szerepel, a címke gyakorlatilag kiszivárog. Az invariánsok:

```
assert set(train_ids).isdisjoint(test_ids), "dokumentum-ID átfedés"
train_hashes = {hash_text(t) for t in train_texts}
test_hashes = {hash_text(t) for t in test_texts}
assert train_hashes.isdisjoint(test_hashes), "normalizált szövegduplikáció"
```

A `hash_text` a szöveget normalizálja (kisbetű, szóközök, írásjelek), majd hash-t számol; a pontos egyezés mellett a közel azonos sablonok külön vizsgálatot igényelnek. A dokumentum-ID és a szöveghash ellenőrzése a vektorizáló szivárgásától független, második kapu.

4. Az új üzenetet a `pipeline.predict(["A VPN frissítés után nem indul."])` hívással kell kiértékelni. A pipeline először a tanított vektorizálóval transzformál, így az ismeretlen tokenek kimaradnak, az ismert tokenek pedig a megszokott súlyokat kapják. Ha az üzenetben egyetlen tanítószókincsbeli token sincs, a ritka sor minden eleme nulla; ilyenkor a logisztikus regresszió csak a `w_0` interceptet használja, és a tanítóbeli gyakoriságnak megfelelő alapvalószínűséget ad. Az ilyen eseteket érdemes explicit ellenőrzéssel kezelni: a `transform` után meg kell nézni, hogy a létrejött ritka sor `nnz` értéke nulla-e. Ez a séma a 21.4. alfejezet predikciós szerződés szöveges megfelelője: a vektorizáló dimenziója és a kimenő feature-nevek stabilak, az ismeretlen token nem okoz összeomlást, de a bizalom alacsony.

27.1.7 Gyakori tévedés

A leggyakoribb hiba az, hogy a vektorizálót a tanító- és tesztkorpussal együtt illesztik, mert a szövegfeldolgozást „előfeldolgozó lépésnek” tekintik, amely még nem gépi tanulás. A TF-IDF azonban tanulási lépés: a szókincs és az idf a korpuszból becsült paraméter. Ellenpélda: ha a teszt tartalmazza az `router` szót, és ezt a szivárgó vektorizáló beépíti a szókincsbe, a kísérleti pontosság magasabb lehet, mint egy olyan rendszeré, amely éles üzem közben találkozik először az ismeretlen `router` tokennal. A teszt célja éppen ezt a jövőbeli ismeretlenséget kell modellezze; ezért a `fit` hívás soha nem láthatja a tesztet.

27.1.8 Önellenőrzés

Tedd fel magadnak a következő kérdéseket a saját futtatásod után: Látta-e a vektorizáló a tesztszövegeket bármelyik lépésben? Egyezik-e a tanító- és tesztmátrix oszlopszáma? A tesztkorpusszal nem optimalizáltam sem küszöböt, sem `min_df` értéket? Ha bármelyik válasz nemleges, akkor a kísérlet szivárgásmentesnek tekinthető ezen a szinten.

27.1.9 Rövid összefoglaló

A szöveges baseline egy ritka TF-IDF-mátrixot és egy lineáris osztályozót kapcsol össze. A tokenizálás a szöveget szószintű és bigram elemekre bontja; a TF-IDF a dokumentumgyakorisággal súlyozva emeli ki a ritka, megkülönböztető tokeneket. A korpuszfelosztásnak a vektorizáló illesztése előtt meg kell történnie, különben a tesztből származó szókincs szivárgást okoz. A példa kis, szintetikus magyar mondatokon ellenőrzi a dimenzióinvarianciát, de nem ad általánosítható pontosságot. A következő alkalmazásként a 27.2. alfejezet embedding-fejezetben összehasonlíthatjuk, hogy a rögzített vektoros reprezentáció hogyan módosítja ugyanennek a támogatásjegy-feladatnak az értékelését.

27.2 Embedding és hasonlósági keresés

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, Python**
>
> Előfeltétel: 27.1. alfejezet · Célkészség: Embedding és hasonlósági keresés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

27.2.1 Cél és konkrét helyzet

Egy belső IT-hibajegyrendszerben dolgozol. A felhasználó beír egy rövid hibaleírást, például: *„Nem tudok nyomtatni, hálózati csatlakozási gond van."* A rendszer feladata, hogy visszaadja a korábbi, már megoldott jegyek közül a leghasznosabbakat, hogy a támogató kolléga gyorsan lássa a korábbi megoldást. Ez klasszikus **visszakeresési (retrieval)** feladat: nem osztályozunk, hanem rangsorolunk.

A **megfigyelési egység** egy-egy dokumentum, azaz egy korábbi hibajegy szövege. A **bemenet** egy új lekérdezés szövege, amelyet a rendszer ugyanabba a vektortérbe képez le, mint a korábbi jegyeket. A **kimenet** a dokumentumok hasonlóság szerint csökkenő sorba rendezett listája, amelyből a top k találatot mutatjuk meg. Az **ellenőrzés** két szinten történik: (1) numerikusan, a koszinusz-hasonlóság kézzel számolt értékeivel; (2) tartalmilag, emberi relevanciajelöléssel, amely megmondja, hogy a találat valóban hasznos volt-e.

Ebben az alfejezetben kis, előre megadott vektorokon dolgozunk, hogy a műveleteket kézzel is ellenőrizni tudd. Az ilyen vektorokat a gyakorlatban egy **beágyazó modell (embedding model)** állítja elő, de a keresési logika ugyanaz, függetlenül attól, hogy a vektor honnan származik.

Mielőtt a megoldást megírnád, becsüld meg fejben: ha a lekérdezés a $(0.8, 0.2, 0.0)$ vektor, és a dokumentumok között van egy $(1.0, 0.1, 0.0)$ vektor, akkor várhatóan ez lesz-e az első találat? A számítás után térj vissza ehhez a hipotézishez.

27.2.2 Fogalmi magyarázat

27.2.2.1 Beágyazás (embedding)

A 27.1. alfejezetben látott TF-IDF a szöveget szavankénti ritka vektorrá alakította: minden dimenzió egy-egy szó, és a legtöbb érték nulla. A sűrű **beágyazás** ezzel szemben a szöveget egy fix hosszúságú, jellemzően 100–1000 dimenziós valós vektorrá képezi le, ahol minden komponens általában nullától különböző, és a dimenziók nem feleltethetők meg közvetlenül egy-egy szónak.

A vektor a szöveg *használati mintázatát* tömöríti: a hasonló kontextusban előforduló, hasonló jelentésű szövegek hasonló vektort kapnak.

A vektor dimenzióját d-vel jelöljük, tehát $\mathbf{v} \in \mathbb{R}^d$. A dokumentumhalmazban n dokumentum van, így a teljes mátrix alakja (n, d): minden sor egy dokumentum, minden oszlop egy dimenzió. A vektorkomponenseknek itt nincs fizikai mértékegységük; a modell belső reprezentációjának tekintjük őket.

27.2.2.2 Koszinusz-hasonlóság (cosine similarity)

Két d-dimenziós vektor hasonlóságát a köztük lévő szög koszinuszaként mérjük. A képlet:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|},$$

ahol

- $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$, alakjuk $(d,)$;
- a **skalárszorzat** $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{d} a_i b_i$, skalár érték;
- a **norma** $\|\mathbf{a}\| = \sqrt{\sum_{i=1}^{d} a_i^2}$, nemnegatív valós;
- a tört értékkészlete $[-1, 1]$; ha a vektorok minden komponense nemnegatív, akkor $[0, 1]$.

A koszinusz nem függ a vektorok *hosszától*, csak az *irányuktól*. Ha $\mathbf{a} = (1, 1)$ és $\mathbf{b} = (2, 2)$, akkor a kézi számítás:

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 2 + 1 \cdot 2 = 4, \qquad \|\mathbf{a}\| = \sqrt{2}, \qquad \|\mathbf{b}\| = \sqrt{8},$$

$$\cos = \frac{4}{\sqrt{2}\sqrt{8}} = \frac{4}{\sqrt{16}} = 1.$$

Az 1 érték azonos irányt jelent. Ha a vektorok merőlegesek, például $(1, 0)$ és $(0, 1)$, akkor a skalárszorzat 0, így a koszinusz is 0. Ha ellentétes irányúak, az érték -1.

A kereső gyakorlatban **előre normalizáljuk** a vektorokat: $\mathbf{a}' = \mathbf{a}/\|\mathbf{a}\|$. Ekkor $\|\mathbf{a}'\| = 1$, és a koszinusz egyszerűen a normalizált vektorok skalárszorzata. Ez azért kényelmes, mert a keresés egyetlen mátrix-skalárszorzássá válik.

27.2.2.3 Legközelebbi szomszéd (nearest neighbor) és visszakeresés

A **legközelebbi szomszéd keresés** formálisan: adott a $\mathbf{q} \in \mathbb{R}^d$ lekérdezési vektor és a $\{\mathbf{d}_1, \ldots, \mathbf{d}_n\}$ dokumentumvektorok; a cél a legnagyobb $\cos(\mathbf{q}, \mathbf{d}_i)$ értékű dokumentumok megtalálása. A top k találatot rangsorolva adjuk vissza.

A **visszakeresés (retrieval)** ennél tágabb feladat: a releváns dokumentumok megtalálása egy gyűjteményben. Fontos különválasztani két kérdést:

1. **A reprezentáció minősége:** jól írja-e le az embedding a szövegek hasonlóságát?
2. **A keresőindex működése:** milyen gyorsan és milyen kompromisszummal találjuk meg a jelölteket?

Ez a fejezet elsősorban az első kérdést tanítja; a második a nagy adatos, elosztott környezetekben válik kritikussá.

27.2.2.4 Pontos és közelítő keresés (exact vs approximate)

A **pontos (exact)** keresés minden dokumentumvektort összehasonlít a lekérdezéssel. A költség $O(n \cdot d)$: n dokumentum, d dimenzió. Néhány ezer dokumentumig ez tökéletesen vállalható egy CPU-s környezetben, és garantáltan megtalálja a legjobb találatot.

Nagyobb gyűjteménynél (n milliós nagyságrend) a **közelítő (approximate)** módszerek jönnek szóba: fa-, gráf- vagy hash-alapú indexek, mint a HNSW vagy az LSH. Ezek gyorsabbak, de nem garantálják, hogy az abszolút legközelebbi szomszédot találják meg; a sebesség és a pontosság között trade-off van. A könyv fő útja kis adatos, ezért itt pontos keresést használunk, és az ANN-t csak megemlítjük.

27.2.2.5 Rangsor metrika

A visszakeresés értékeléséhez **emberi relevanciajelölés** kell. Egy adott lekérdezéshez minden dokumentumra megmondjuk, hogy releváns-e (1) vagy sem (0). A leggyakoribb metrika a **találati pontosság (precision@k)**:

$$\text{precision@k} = \frac{\text{releváns találatok a top } k \text{ listában}}{k}.$$

Ez a metrika a 23.2. alfejezetben tanult elvhez hasonlóan a *feladat célját* méri, nem a modell belső hibáját. Egyetlen lekérdezésre a precision@k értéke $\{0, 1/k, \dots, 1\}$ közül kerül ki; több lekérdezésre átlagolunk. A pontosság mellett érdemes a 15.2. alfejezetben tanult módon intervallumot is számolni, mert 10–20 lekérdezés kis minta.

27.2.3 Teljes minimális munkapélda

A példa bemenete öt szintetikus hibajegy, amelyeket kézzel rendelünk hozzá háromdimenziós vektorokhoz. A dimenziók itt mesterségesek; a módszer mechanizmusát mutatják, nem egy valódi embedding kimenetét.

A dokumentumok és vektoraik:

Azonosító	Szöveg	Vektor
d1	„A nyomtató nem csatlakozik a hálózathoz”	$(1.0, 0.1, 0.0)$
d2	„Hálózati hiba a nyomtatónál”	$(0.9, 0.15, 0.05)$
d3	„A számla értéke hibás”	$(0.0, 1.0, 0.1)$
d4	„Számlázási probléma”	$(0.05, 0.95, 0.0)$
d5	„A képernyő villog”	$(0.1, 0.0, 1.0)$

A lekérdezés: „Nem tudok nyomtatni, hálózati csatlakozási gond van”, vektora $\mathbf{q} = (0.8, 0.2, 0.0)$.

A keresést NumPy-val hajtjuk végre. A normalizált vektorokon a mátrix-skalárszorzat adja a koszinusz-hasonlóságokat.

```python
import numpy as np

# Dokumentumvektorok, shape: (5, 3)
doc_vectors = np.array([
    [1.0, 0.1, 0.0],   # d1
    [0.9, 0.15, 0.05], # d2
    [0.0, 1.0, 0.1],   # d3
    [0.05, 0.95, 0.0], # d4
    [0.1, 0.0, 1.0],   # d5
])
query = np.array([0.8, 0.2, 0.0])

# Normalizálás: minden sor hossza 1 lesz
doc_norm = doc_vectors / np.linalg.norm(doc_vectors, axis=1, keepdims=True)
query_norm = query / np.linalg.norm(query)

# Normalizált vektorokon a skalárszorzat = koszinusz
cos_sim = doc_norm @ query_norm  # shape: (5,)
ranking = np.argsort(-cos_sim)   # csökkenő sorrend

print("Normalizált dokumentumnormák:", np.round(np.linalg.norm(doc_norm, axis=1), 6))
print("Koszinusz-hasonlóság:", np.round(cos_sim, 3))
print("Rangsor (0-indexelt dokumentum pozíció):", ranking)
```

Szemléltető, elméletileg várt kimenet:

```
Normalizált dokumentumnormák: [1. 1. 1. 1. 1.]
Koszinusz-hasonlóság: [0.989 0.986 0.241 0.238 0.096]
Rangsor (0-indexelt dokumentum pozíció): [0 1 2 3 4]
```

A dokumentumvektorok szándékosan úgy vannak kialakítva, hogy a d1 és d2 a „nyomtatáshálózat” témához, a d3 és d4 a „számlázás” témához tartozzon, a d5 pedig a „képernyő” témához. A lekérdezés a nyomtatási témához áll közel, ezért az első két találatnak d1-nek és d2-nek kell lennie.

27.2.4 Eredmény értelmezése

A kimenet első sora a **normalizálási invariant** ellenőrzése: minden normalizált dokumentumvektor hossza 1 (a lebegőpontos kerekítésből adódó 10^{-6} toleranción belül). Ez garantálja, hogy a mátrix-skalárszorzat valóban koszinusz-hasonlóságot ad.

A második sor a hasonlósági értékek. A d1-re kapott 0.989 kézi kontrollja:

$$\mathbf{q} \cdot \mathbf{d}_1 = 0.8 \cdot 1.0 + 0.2 \cdot 0.1 + 0.0 \cdot 0.0 = 0.82,$$

$$\|\mathbf{q}\| = \sqrt{0.8^2 + 0.2^2} = \sqrt{0.68} \approx 0.8246, \qquad \|\mathbf{d}_1\| = \sqrt{1.0^2 + 0.1^2} = \sqrt{1.01} \approx 1.0050,$$

$$\cos \approx \frac{0.82}{0.8246 \cdot 1.0050} \approx 0.989.$$

A d3-ra hasonlóan: $\mathbf{q} \cdot \mathbf{d}_3 = 0.2$, $\|\mathbf{d}_3\| \approx 1.0050$, így $\cos \approx 0.241$. A d5-re: $\mathbf{q} \cdot \mathbf{d}_5 = 0.08$, így $\cos \approx 0.096$. A rangsor tehát d1, d2, d3, d4, d5 — ez megfelel a tematikus elrendezésnek: a nyomtatási témájú dokumentumok elöl, a számlázási témájúak középen, a képernyő téma hátul.

A hasonlósági pontszám **nem relevanciaítélet**. Azt, hogy a d1 vagy a d2 valóban hasznos-e a felhasználó számára, emberi jelölés dönti el. A numerikus sorrend csak jelölteket ad.

Hasonlósági keresésnél különböztessük meg a koszinusz-hasonlóságot és az euklideszi távolságot; normalizált vektorokra a rangsoruk összefügg, nyers vektorokra nem általánosan.

27.2.5 Arany standard és retrieval metrikák

A koszinusz-hasonlóság csak *rangsorol*, nem mondja meg, hogy egy találat releváns-e. Ahhoz, hogy a keresőt értékelni lehessen, **arany standard** (gold standard) kell: rögzített korpusz, rögzített lekérdezések, előre megadott bináris relevanciaítéletek és rögzített k. Az értékelés csak akkor reprodukálható, ha a korpusz, az embedding verziója, a hasonlósági mérték és a k is a jegyzőkönyv része.

Az alábbi 12 dokumentumos, négy tematikus dimenziós példa a teljes folyamatot mutatja. A dimenziók itt is mesterségesek: `[nyomtatás, számlázás, VPN, képernyő]`.

Azonosító	Szöveg	Téma	Vektor
d01	„A nyomtató nem csatlakozik a hálózathoz”	nyomtatás	$(1.0, 0.0, 0.0, 0.0)$
d02	„Hálózati hiba a nyomtatónál”	nyomtatás	$(0.9, 0.05, 0.0, 0.0)$
d03	„Nyomtatási sor elakadt”	nyomtatás	$(0.85, 0.0, 0.05, 0.0)$
d04	„A nyomtatott számla hibás”	vegyes	$(0.7, 0.6, 0.0, 0.0)$
d05	„Számlázási hiba, rossz összeg”	számlázás	$(0.65, 0.75, 0.0, 0.0)$
d06	„A számla értéke nem egyezik”	számlázás	$(0.0, 1.0, 0.0, 0.0)$
d07	„Számlázási probléma a havi díjjal”	számlázás	$(0.0, 0.95, 0.0, 0.0)$
d08	„Árajánlat kérése”	számlázás	$(0.0, 0.9, 0.05, 0.0)$
d09	„VPN kapcsolat nem indul”	VPN	$(0.0, 0.0, 1.0, 0.0)$
d10	„Távoli elérés megszakad”	VPN	$(0.0, 0.0, 0.75, 0.45)$
d11	„A vendég Wi-Fi jelszava lejárt”	hálózat (nem VPN)	$(0.0, 0.0, 0.98, 0.02)$
d12	„A képernyőn a kapcsolat ikon villog”	képernyő	$(0.0, 0.0, 0.4, 0.9)$

Három lekérdezés és az **előre rögzített** relevanciahalmazok:

- `q1` = „Nem tudok nyomtatni, hálózati hiba” $\rightarrow \mathbf{q}_1 = (0.9, 0.35, 0.0, 0.0)$; releváns: `d01`, `d02`, `d03`.
- `q2` = „Számlázási hiba, rossz összeg” $\rightarrow \mathbf{q}_2 = (0.5, 0.85, 0.0, 0.0)$; releváns: `d05`, `d06`, `d07`.
- `q3` = „VPN kapcsolat megszakad belépéskor” $\rightarrow \mathbf{q}_3 = (0.0, 0.0, 0.9, 0.2)$; releváns: `d09`, `d10`.

A metrikák definíciója $k = 3$ mellett:

$$\text{precision@}k = \frac{\#\{\text{releváns találat a top } k\text{-ban}\}}{k}, \qquad \text{recall@}k = \frac{\#\{\text{releváns találat a top } k\text{-ban}\}}{\#\{\text{összes releváns}\}}.$$

A **reciprok rang** (reciprocal rank, RR) egy lekérdezésre az első releváns találat rangjának reciproka; ha nincs releváns a listában, az érték 0. A **mean reciprocal rank (MRR)** a lekérdezések RR-jeinek átlaga. A precision és a recall különböző hibákat mér: a precision a haszontalan találatok arányát bünteti, a recall pedig azt, ha egy releváns dokumentum lemarad.

```
import numpy as np

doc_ids = [f"d{i:02d}" for i in range(1, 13)]
doc_vectors = np.array([
    [1.0, 0.0, 0.0, 0.0], [0.9, 0.05, 0.0, 0.0], [0.85, 0.0, 0.05, 0.0],
    [0.7, 0.6, 0.0, 0.0], [0.65, 0.75, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0],
    [0.0, 0.95, 0.0, 0.0], [0.0, 0.9, 0.05, 0.0], [0.0, 0.0, 1.0, 0.0],
    [0.0, 0.0, 0.75, 0.45], [0.0, 0.0, 0.98, 0.02], [0.0, 0.0, 0.4, 0.9],
])
queries = {
    "q1": np.array([0.9, 0.35, 0.0, 0.0]),
    "q2": np.array([0.5, 0.85, 0.0, 0.0]),
    "q3": np.array([0.0, 0.0, 0.9, 0.2]),
}
relevance = {
    "q1": {"d01", "d02", "d03"},
    "q2": {"d05", "d06", "d07"},
    "q3": {"d09", "d10"},
}
GOLD = {"k": 3, "similarity": "cosine", "embedding": "kezi-4d-v1"}

doc_unit = doc_vectors / np.linalg.norm(doc_vectors, axis=1, keepdims=True)
per_query = {}
for name, query in queries.items():
    query_unit = query / np.linalg.norm(query)
    scores = doc_unit @ query_unit
    top = np.argsort(-scores)[:GOLD["k"]]
    hits = [doc_ids[i] in relevance[name] for i in top]
    rr = next((1 / (rank + 1) for rank, hit in enumerate(hits) if hit), 0.0)
    per_query[name] = {
        "top": [(doc_ids[i], round(float(scores[i]), 3)) for i in top],
        "precision": sum(hits) / GOLD["k"],
        "recall": sum(hits) / len(relevance[name]),
        "rr": rr,
    }

for name, result in per_query.items():
    print(name, result)
print("mean precision@3:", round(np.mean([r["precision"] for r in per_query.values()]), 3))
print("mean recall@3:   ", round(np.mean([r["recall"] for r in per_query.values()]), 3))
print("MRR:             ", round(np.mean([r["rr"] for r in per_query.values()]), 3))
```

A tárolt kimenet (a fenti kód futtatása, rögzített vektorokkal):

```
q1 {'top': [('d02', 0.951), ('d04', 0.944), ('d01', 0.932)], 'precision': 0.6666666666666666,
 ↪  'recall': 0.6666666666666666, 'rr': 1.0}
q2 {'top': [('d05', 0.983), ('d04', 0.946), ('d06', 0.862)], 'precision': 0.6666666666666666,
 ↪  'recall': 0.6666666666666666, 'rr': 1.0}
q3 {'top': [('d11', 0.98), ('d09', 0.976), ('d10', 0.949)], 'precision': 0.6666666666666666,
 ↪  'recall': 1.0, 'rr': 0.5}
mean precision@3: 0.667
mean recall@3:    0.778
MRR:              0.833
```

A kézi ellenőrzés ugyanezt adja. A `q1` top 3 listájában a releváns találatok `d02` és `d01` (a `d04` nem releváns), ezért a precision 2/3, a recall 2/3, és az első releváns az 1. helyen áll, így RR = 1. A `q2`-nél a top 3 `d05`, `d04`, `d06`; ismét 2/3 és 2/3, RR = 1. A `q3`-nál a top 3 `d11`, `d09`, `d10`; közülük a `d09` és a `d10` releváns, ezért a precision 2/3, a recall 2/2 = 1, az első releváns a 2. helyen áll, így RR = 1/2. Az átlagok:

$$\bar{P}_{@3} = \frac{2/3 + 2/3 + 2/3}{3} = \frac{2}{3} \approx 0.667, \qquad \bar{R}_{@3} = \frac{2/3 + 2/3 + 1}{3} = \frac{7}{9} \approx 0.778,$$

$$\mathrm{MRR} = \frac{1 + 1 + 1/2}{3} = \frac{5}{6} \approx 0.833.$$

27.2.5.1 Hibakategóriák kézi ellenőrzése

A metrika önmagában nem mondja meg, *miért* hibázik a kereső. A top k listákat kézzel is át kell nézni, és minden téves találatot kategóriába kell sorolni:

Kategória	Jelentés	Példa
`topic_drift`	A dokumentum két témához tartozik, ezért a rossz lekérdezésnél is magas a hasonlóság.	`d04` a `q1` és `q2` top 3-ában
`lexical_false_friend`	Közös szó vagy szókapcsolat, de más probléma.	`d11` (vendég Wi-Fi) a `q3` első helyén
`ranked_below_k`	Releváns dokumentum, de a top k alá szorult.	`d03` a `q1`-nél, `d07` a `q2`-nél
`duplicate`	Közel azonos jegyek több helyet foglalnak a listában.	nincs a példában; a gyakorlatban gyakori
`annotation_disagreement`	A két annotátor eltérően ítélte meg a relevanciát.	a jelölési folyamatot ellenőrzi

A jelölést **a pontszámok ismerete nélkül**, előre rögzített útmutató szerint kell végezni; két annotátor függetlenül jelöl, az egyetértést (például Cohen κ-val) jelenteni kell, az eltéréseket egyeztetés oldja fel. A jelölést a keresés futtatása *előtt* rögzítjük, különben a küszöb utólag a kedvezőbb eredményhez igazítható. A teljes jegyzőkönyv: korpusz- és embeddingverzió, k, hasonlósági mérték, a lekérdezések, a relevanciaítéletek, a nyers top k kimenet és a hibakategóriák. A 12 dokumentum és 3 lekérdezés demonstrációs méret, nem éles benchmark; a konfidenciaintervallum itt is széles, és a kapott 0.667 precision nem általánosítható nagyobb korpuszra.

27.2.6 Feladatok

1. Értelmezés. Igazold képlettel, hogy ha mindkét vektort ugyanazzal a pozitív skalárral szorozzuk, a koszinusz-hasonlóság nem változik. Miért fontos ez a gyakorlatban, ha az embeddingmodell különböző hosszúságú vektorokat ad vissza?

2. Megvalósítás. Írj egy `top_k_search` függvényt, amely megkapja a dokumentumvektorok (n, d) mátrixát, a lekérdezésvektort és a k értéket, majd visszaadja a legnagyobb koszinusz-hasonlóságú k dokumentum *azonosítóját* (például a `"d1"` sztringet) csökkenő pontszám szerint. Futtasd a fenti példán $k = 2$ értékkel; az elvárt eredmény `["d1", "d2"]`.

3. Hibadiagnózis. Egy kolléga a hasonlóságot normalizálás nélkül, `np.dot(a, b)` értékkel számolta. Adj konkrét ellenpéldát, ahol ez a módszer téves rangsort ad. Használd a $\mathbf{q} = (1, 1)$ lekérdezést és a $\mathbf{d}_1 = (10, 0)$, $\mathbf{d}_2 = (2, 2)$ dokumentumokat. Számítsd ki mindkét módszerrel a sorrendet, és magyarázd meg az eltérést.

4. Megváltozott feltétel és relevanciajelölési terv. Cseréld a lekérdezést a „számlázási gond" témára, például $\mathbf{q}' = (0.0, 0.9, 0.0)$-ra. Számítsd ki kézzel a koszinusz-hasonlóságot d3-ra és d4-re, és add meg az új várható rangsort. Ezután írj egy emberi relevanciajelölési tervet: 10 lekérdezés, mindegyikhez a top 3 találat, bináris relevanciajelölés (0/1). Számítsd ki a precision@3 értékét az alábbi fiktív jelölési eredményekre: 10 lekérdezésből 4 esetben 3 releváns találat, 3 esetben 2, 2 esetben 1, 1 esetben 0 releváns találat volt a top 3-ban.

27.2.7 Megoldások és indoklás

1. megoldás. Legyen $\alpha > 0$ skalár. A skalárszorzat és a norma homogenitásából:

$$\cos(\alpha\mathbf{a}, \alpha\mathbf{b}) = \frac{(\alpha\mathbf{a}) \cdot (\alpha\mathbf{b})}{\|\alpha\mathbf{a}\| \, \|\alpha\mathbf{b}\|} = \frac{\alpha^2(\mathbf{a} \cdot \mathbf{b})}{\alpha\|\mathbf{a}\| \cdot \alpha\|\mathbf{b}\|} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}.$$

A α^2 mind a számlálóban, mind a nevezőben kiesik. A gyakorlatban ez azt jelenti, hogy ha az embedding-modell egy dokumentumot hosszabb vagy rövidebb vektorral reprezentál, az a hasonlósági sorrendet nem befolyásolja, amíg az irány azonos.

2. megoldás. Függvény azonosítókkal; a dokumentumazonosítókat a mátrixszal párhuzamos listában adjuk át.

```
import numpy as np

def top_k_search(doc_vectors, doc_ids, query, k):
    doc_norm = doc_vectors / np.linalg.norm(doc_vectors, axis=1, keepdims=True)
    q_norm = query / np.linalg.norm(query)
    cos_sim = doc_norm @ q_norm
    top_indices = np.argsort(-cos_sim)[:k]
    return [(doc_ids[i], round(float(cos_sim[i]), 3)) for i in top_indices]

doc_ids = ["d1", "d2", "d3", "d4", "d5"]
print(top_k_search(doc_vectors, doc_ids, query, 2))
```

Szemléltető, elméletileg várt kimenet:

```
[('d1', 0.989), ('d2', 0.986)]
```

Ellenőrző invariant: a visszaadott lista hossza k; a pontszámok monoton csökkenőek; minden pontszám $[-1, 1]$-ben van.

3. megoldás. A normalizálatlan skalárszorzat: $\mathbf{q} \cdot \mathbf{d}_1 = 10$, $\mathbf{q} \cdot \mathbf{d}_2 = 4$, tehát a téves rangsor d1 előbb, mint d2. A koszinusz-hasonlóság:

$$\cos(\mathbf{q}, \mathbf{d}_1) = \frac{10}{\sqrt{2} \cdot 10} = \frac{1}{\sqrt{2}} \approx 0.707, \qquad \cos(\mathbf{q}, \mathbf{d}_2) = \frac{4}{\sqrt{2} \cdot \sqrt{8}} = 1.$$

A helyes rangsor d2 előbb, mint d1. A normalizálatlan skalárszorzatot a vektorok hossza torzítja: a d1 hossza 10, a d2 hossza $\sqrt{8} \approx 2.83$, ezért a hosszú d1 látszólag jobb, pedig az iránya távolabb van a lekérdezéstől. A koszinusz a hosszhatást kiküszöböli.

4. megoldás. A $\mathbf{q}' = (0.0, 0.9, 0.0)$ lekérdezés d3-ra és d4-re:

- d3: $\mathbf{q}' \cdot \mathbf{d}_3 = 0.0 \cdot 0.0 + 0.9 \cdot 1.0 + 0.0 \cdot 0.1 = 0.9$; $\|\mathbf{q}'\| = 0.9$; $\|\mathbf{d}_3\| \approx 1.0050$; $\cos \approx 0.9/(0.9 \cdot 1.0050) \approx 0.995$.
- d4: $\mathbf{q}' \cdot \mathbf{d}_4 = 0.9 \cdot 0.95 = 0.855$; $\|\mathbf{d}_4\| \approx \sqrt{0.05^2 + 0.95^2} \approx 0.9513$; $\cos \approx 0.855/(0.9 \cdot 0.9513) \approx 0.998$.

Az új várható rangsor: d4, d3, majd a többi alacsony értékkel. A d4 azért kerül d3 elé, mert a $\mathbf{q}'$ és a d4 iránya közelebb áll egymáshoz, mint a $\mathbf{q}'$ és a d3 iránya.

A relevanciajelölési terv: minden lekérdezéshez előre rögzítjük a top 3 találatot; egy annotátor minden találatra 1-et ad, ha a dokumentum megoldja a hibát, 0-t egyébként. A fiktív eredmények: 4 lekérdezésnél 3 releváns, 3-nál 2, 2-nél 1, 1-nél 0. A releváns találatok száma:

$$4 \cdot 3 + 3 \cdot 2 + 2 \cdot 1 + 1 \cdot 0 = 12 + 6 + 2 = 20.$$

A precision@3:

$$\text{precision@3} = \frac{20}{10 \cdot 3} = \frac{20}{30} \approx 0.667.$$

A pontbecslés mellett érdemes megjegyezni, hogy 10 lekérdezés kicsi minta; a 15.2. alfejezetben tanultak szerint a konfidenciaintervallum széles lesz.

27.2.8 Gyakori tévedés

A leggyakoribb félreértés az, hogy a **magas koszinusz-hasonlóság ténybeli igazságot bizonyít**. Ez nem igaz. Az embedding a szövegek nyelvi-fogalmi *közelségét* tömöríti, nem a világra vonatkozó állítások igazságát. Ellenpélda: a *„Franciaország fővárosa Párizs”* és a *„Franciaország fővárosa Róma”* mondatok vektorai nagyon közel lehetnek egymáshoz, mert ugyanaz a nyelvi séma, ugyanazok a szavak — mégis az egyik állítás hamis. A retrieval rendszer visszaadhatja a hamis mondatot, ha a formai hasonlóság nagy. A relevanciaítélet ezért mindig emberi ellenőrzést kíván, és a hasonlósági pontszámot nem szabad kalibrált valószínűségként kezelni, ahogyan azt 26.2. alfejezetben is láttuk.

27.2.9 Önellenőrzés

- Mi a koszinusz értékkészlete, és mit jelent a 0, 1 és -1 érték?
- Miért normalizáljuk a vektorokat a keresőben, és hogyan ellenőrizhető a normalizálás invariantja?
- Mi a különbség a reprezentáció minősége és a keresőindex működése között?
- A pontos keresés költsége $O(n \cdot d)$; mikor válik ez problémává, és mi a közelítő keresés ára?

27.2.10 Rövid összefoglaló

Az embedding a szöveget sűrű, fix dimenziós valós vektorrá képezi; a hasonlóságot a vektorok közötti szög koszinusza méri. A koszinusz a skalárszorzat és a normák hányadosa, értéke $[-1, 1]$; normalizált vektorokon egyszerű skalárszorzat. A legközelebbi szomszéd keresés a dokumentumvektorok és a lekérdezés közötti hasonlóságot rendezi csökkenő sorrendbe. Kis adatra a pontos keresés megfelelő; nagy adatra közelítő indexek kellenek, amelyek pontosságot áldoznak a sebességért. A retrieval minőségét emberi relevanciajelöléssel, például precision@k-val mérjük. A következő alfejezetben az LLM-mint sztochasztikus komponens és a külső modellek értékelése felé lépünk tovább.

27.3 LLM mint sztochasztikus komponens

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, Python

Előfeltétel: 27.2. alfejezet · Célkészség: LLM mint sztochasztikus komponens · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Az LLM-kimenet értékelése akkor válik tudományos feladattá, amikor a kérdés már nem az, hogy „egy demóban működik-e”, hanem az, hogy milyen hibával, milyen bizonytalansággal és milyen költséggel lehet a rendszerre támaszkodni. Ebben az alfejezetben egy szövegkinyerő feladat specifikációját építjük fel fizetős hívás nélkül. A tanulási helyzet a következő: egy ügyfélszolgálati csapat automatizálná a visszatérítési szándék (*refund request*) kinyerését rövid üzenetekből. Nincs valós ügyfél; a példaszövegek mesterségesek. A megfigyelési egység egy üzenet, a bemenet `message` szöveg, a kimenet két mező: `refund_request` `{yes, no}` és `amount_minor` (pozitív egész EUR-cent vagy `None`). A cél egy LLM-jelölt összehasonlítása egy reguláris kifejezéses baseline-nal. Induló becslés: ha 50 tesztpéldából 42 lesz teljesen helyes, akkor a pontosság $\hat{p} = 42/50 = 0{,}84$, a binomiális standard hiba pedig $\sqrt{0{,}84 \cdot 0{,}16/50} \approx 0{,}052$, vagyis körülbelül 5,2 százalékpont. Ellenőrizd ezt a kézi számítást, mielőtt továbblép.

27.3.1 Miért sztochasztikus komponens az LLM?

Ugyanaz a prompt és bemenet nem garantálja ugyanazt a kimenetet. A mintavételezési hőmérséklet, a `top_p`, a szerveroldali routing, sőt a modellfrissítés is változtathatja a választ. Ezért az LLM-hívást úgy kezeljük, mint egy véletlenszerű komponenst: az értékelés nem egyetlen kimenetet, hanem a lehetséges kimenetek eloszlását vizsgálja. A bizonytalanságnak két rétege van: a modell belső sztochasztikája, amelyet ismételt hívásokkal és rögzített `seed`/alacsony hőmérséklettel csökkenthetünk, és a tesztkészlet mintavételi bizonytalansága, amelyet a korábbi fejezetekben tanult módon (15.3. alfejezet) standard hibával, konfidenciaintervallummal vagy bootstrappal jellemzünk. A kettőt nem szabad összemosni.

27.3.2 Az értékelési specifikáció hét eleme

Prompt version. A prompt nem „szöveg”, hanem verziózott artefaktum. Rögzíteni kell: a sablont, a rendszerutasítást, a modell azonosítóját, a `temperature`, `top_p`, `max_tokens` értékeket, a few-shot példák azonosítóit és a befagyasztás időpontját. Példa rekord:

```
PROMPT_VERSION = {
    "name": "refund_extract_v1.2",
    "model_id": "hypothetical-local-chat-v0",  # szemléltető név, nem valós modell
    "temperature": 0.2,
    "top_p": 0.9,
    "max_tokens": 40,
    "system": "Extract refund request and amount_minor as JSON. Amount is integer cents or
    ↪  null.",
    "template": "Message: {message}\nJSON:",
    "few_shot_ids": ["dev_01", "dev_02"],
    "frozen_at": "2026-10-20T12:00:00Z",
}
```

Reference set. Ez az ember által annotált aranykészlet (*gold labels*). Legalább két annotátor dolgozzon rajta, és az eltéréseket felülvizsgálattal oldják fel. A fejlesztéshez használt dev-rész és az egyszer használt teszt-rész külön fájl legyen.

Rubric. A pontozási útmutató kimondja, mi számít helyesnek. Itt két részfeladat van: a refund igen/nem döntés és az összeg. Minden példa 0–2 pontot érhet: 1 pont a refund mezőért, 1 pont az amount mezőért. Ha a refund értéke `no`, a modell által adott bármilyen összeg hallucinációnak minősül, tehát 0 pont az amountra, akkor is, ha a goldban `None` szerepel.

Judge bias. Ha a kiértékelést is LLM-bíró végzi, a bíró maga is sztochasztikus és torzíthat. Gyakori torzítások: a hosszabb, határozottabb válasz előnyben részesítése, a pozíciótorzítás (az első megjelenített válasz preferálása), valamint a saját stílusú válasz felé húzás. Ellenőrzés: a bíró promptjában véletlen sorrendbe állított párok, vak bírálat a gold címkére nézve, és emberi kalibráló minta.

Leakage. A tesztpéldák se a few-shot promptba, se a finomhangoló halmazba, se a bíró példái közé nem kerülhetnek. Ha a bíró látja a gold címkét, az egyetértés felfúvódik. A fejlesztés során a tesztet egyszer, a végső jelentéskor használjuk; a többszöri rápillantás adaptív adathasználat, amely optimista torzítást okoz (visszautalás: 23.3. alfejezet).

Latency. A pontosság mellett a válaszidő is része a szerződésnek. Mérjük a kliens oldali időtartamot másodpercben, és adjuk meg a p50, p95 és maximum értéket. A specifikációban timeout is szerepeljen, például 5 másodperc; ha a p95 efölé kerül, az élesítési döntést meg kell állítani.

Token cost. A tokenek száma a modell tokenizálójától függ, nem az egyszerű szótagszámtól. Pontos mérés a válasz `usage` mezőjéből jön; tervezéskor felső becslést használunk. Ha n példát futtatunk, a költség

$$C = \sum_{i=1}^{n} \frac{t_{\text{in}}(i) \cdot p_{\text{in}} + t_{\text{out}}(i) \cdot p_{\text{out}}}{10^6},$$

ahol t_{in} és t_{out} tokenben, p_{in} és p_{out} pedig dollár per 1 millió tokenben értendő. A számpélda árai fiktívek, nem aktuális díjszabás: $p_{\text{in}} = 3$, $p_{\text{out}} = 15$. Ha átlagosan 70 input- és 20 output token kell példánként, 8 példa költsége $8 \cdot (70 \cdot 3 + 20 \cdot 15)/10^6 = 0{,}00408$ dollár. A bíró hívásainak költségét külön hozzá kell adni.

27.3.3 Mini példa: nyolc elemű reference set és két jelölt

A következő nyolc üzenet a dev-teszt halmazunk. A gold értékeket emberi annotálás rögzítette:

1. `"A rendelésem nem érkezett meg, vissza szeretném kapni a 45 eurót."` → yes, 4500
2. `"Köszönöm, minden rendben."` → no, None
3. `"A bögre törött, pénzvisszafizetést kérek."` → yes, None
4. `"A futár nem találta a címet, de nem kérek visszatérítést."` → no, None
5. `"Kérem, utalják vissza a 12,90 eurót."` → yes, 1290
6. `"Sziasztok, mikor szállítják a csomagot?"` → no, None
7. `"Elállok a vásárlástól, visszatérítést kérek."` → yes, None
8. `"Hibás a termék, 5 eurót fizettem, azt kérem vissza."` → yes, 500

A baseline szabály: ha a szövegben bármelyik kulcsszó (`"vissza"`, `"visszatérítés"`, `"pénz"`) szerepel, refund yes; az összeget az `euró` előtti számból nyeri ki. A mock LLM a `"pénzvisszafizetést"` összetett szót nem ismeri fel, ezért a 3. példán `no`-t ad, minden más esetben a baseline szabályt követi. A mock determinisztikus, de a valódi LLM sztochasztikus viselkedését modellezi; itt most a protokoll bemutatása a cél.

Az alábbi kód nem hív távoli modellt. A `mock_llm_predict` helyettesíti a valódi hívást, a kimenetek pedig elméletileg várt értékek; a könyv írásakor a kód nem lett futtatva.

```
import math
import re

REFERENCE = [
    {"text": "A rendelésem nem érkezett meg, vissza szeretném kapni a 45 eurót.",
     "refund": "yes", "amount": 4500},
    {"text": "Köszönöm, minden rendben.", "refund": "no", "amount": None},
    {"text": "A bögre törött, pénzvisszafizetést kérek.", "refund": "yes", "amount": None},
    {"text": "A futár nem találta a címet, de nem kérek visszatérítést.",
     "refund": "no", "amount": None},
    {"text": "Kérem, utalják vissza a 12,90 eurót.", "refund": "yes", "amount": 1290},
    {"text": "Sziasztok, mikor szállítják a csomagot?", "refund": "no", "amount": None},
    {"text": "Elállok a vásárlástól, visszatérítést kérek.", "refund": "yes", "amount": None},
    {"text": "Hibás a termék, 5 eurót fizettem, azt kérem vissza.",
     "refund": "yes", "amount": 500},
]

def baseline_predict(text: str) -> dict:
    t = text.lower()
    has_refund = any(k in t for k in ["vissza", "visszatérítés", "pénz"])
    m = re.search(r"(\d+(?:[.,]\d{1,2})?)\s*euró", t)
    if has_refund and m:
        s = m.group(1).replace(",", ".")
        amount = int(round(float(s) * 100))
    else:
        amount = None
    return {"refund": "yes" if has_refund else "no", "amount": amount}

def mock_llm_predict(text: str) -> dict:
    # szándékos hiba: az összetett szót nem ismeri fel
    if "pénzvisszafizetést" in text.lower():
        return {"refund": "no", "amount": None}
    return baseline_predict(text)

def score_pair(gold: dict, pred: dict) -> int:
    s = 0
    if gold["refund"] == pred["refund"]:
        s += 1
    if gold["amount"] == pred["amount"]:
        s += 1
    return s

def evaluate(predict_fn):
    full_correct = 0
    for item in REFERENCE:
        pred = predict_fn(item["text"])
        if score_pair(item, pred) == 2:
            full_correct += 1
    return full_correct

def wilson_ci(x: int, n: int, z: float = 1.96) -> tuple[float, float]:
    p = x / n
    denom = 1 + z*z / n
    center = (p + z*z / (2*n)) / denom
    half = z * math.sqrt(p*(1-p)/n + z*z / (4*n*n)) / denom
    return (center - half, center + half)

# elméletileg várt, szemléltető kimenet:
# baseline: 7/8, Wilson 95% CI (0.53, 0.98)
# mock_llm: 6/8, Wilson 95% CI (0.41, 0.93)
```

```
for name, fn in [("baseline", baseline_predict), ("mock_llm", mock_llm_predict)]:
    x = evaluate(fn)
    n = len(REFERENCE)
    lo, hi = wilson_ci(x, n)
    print(f"{name}: {x}/{n} full accuracy, 95% Wilson CI ({lo:.2f}, {hi:.2f})")
```

A kód értelmezése: a pontosság itt a teljesen helyes példák aránya; a Wilson-intervallum azért jobb a normális közelítésnél, mert 8 elemnél a sikerarány a széleken van, és a degenerált bootstrap replikációk félrevezetnének. A két intervallum nagymértékben átfedi egymást, tehát ebből a 8 elemű mintából nem állítható, hogy a baseline jobb vagy rosszabb a mocknál. A hibatábla viszont már most megmutatja a különbséget: a baseline a tagadást nem kezeli (4. példa), a mock pedig az összetett szót nem ismeri fel (3. példa). Ezért a hibataxonómia gyakran értékesebb, mint egyetlen pontszám.

27.3.4 Demó és kiadási küszöb

A 8 elemű példa **demó**, nem kiadási bizonyíték. A kétféle mérés célja, mintája és döntési szabálya különüljön el:

Kérdés	Demó	Kiadási mérés
Cél	fejlesztői hibakeresés, formátum- és szivárgásellenőrzés	kiadási döntés
Minta	8–12 kézileg válogatott, könnyen ellenőrizhető példa	mintavételi keretből húzott, előre rögzített n
Összehasonlítás	egymás mellett, nem rangsorol	azonos példákon páros összehasonlítás
Annotáció	egy fejlesztő	két független, vak annotátor, egyeztetéssel
Bizonytalanság	tájékoztató Wilson-intervallum	előre rögzített alsó konfidenciahatár
Ismétlés	egy futás	$r \geq 3$ futás, a futások közti eltérés jelentve
Döntés	nincs kiadási döntés	előre rögzített elfogadási szabály

Mintavételi keret. A kiadási mérés népessége nem „a fejlesztő által kiválasztott szép példák”, hanem egy rögzített időablak üzenetpopulációja, például az elmúlt 90 nap visszatérítési szándékú üzenetei. A mintát véletlenszerűen, rétegezve húzzuk (például gold `yes`/`no` és rövid/hosszú üzenet szerint), a duplikátumokat és az üres üzeneteket előre kizárjuk, és rögzítjük a keret snapshotját, a húzás időpontját és a véletlen magot. A nehéz eseteket (tagadás, összetett szó, hiányzó összeg) nem utólag válogatjuk be a modell kimenete alapján, mert az a tesztet a modellhez igazítja.

Páros összehasonlítás. A két rendszert ugyanazon n példán futtatjuk, és példánként bináris „teljesen helyes” eredményt rögzítünk. A páros elrendezés kiszűri a példák nehézségét; a lényeg a **diszkordáns párok** száma: b, ahol az A rendszer helyes és B hibás, illetve c, ahol B helyes és A hibás. McNemar-próbával ellenőrizzük, hogy a diszkordánsok aránya szignifikánsan eltér-e az 50–50%-tól. A demóban a baseline 7/8, a mock 6/8; a páros tábla: mindkettő helyes 6, csak a baseline helyes 1 (3. példa), csak a mock helyes 0, mindkettő hibás 1 (4. példa). Az egyetlen diszkordáns pár mellett a kétoldali egzakt McNemar-próba $p = 1{,}0$, tehát a demó **nem rangsorol**. A próba egzakt változata kis mintán is használható:

```python
import math

def mcnemar_exact(b: int, c: int) -> float:
    n = b + c
    if n == 0:
        return 1.0
    k = min(b, c)
    tail = sum(math.comb(n, i) for i in range(k + 1)) / 2**n
    return min(1.0, 2 * tail)

print(mcnemar_exact(1, 0))  # 1.0
```

Annotátoregyezés. A gold címkét két annotátor egymástól függetlenül, a rendszer kimenetét nem látva jelöli. Az egyetértést Cohen κ-val mérjük; a κ a véletlen egyetértést korrigálja, ezért alkalmasabb a nyers egyezési aránynál. A gyakorlati értelmezés hozzávetőleges: 0,4 alatt gyenge, 0,4–0,6 mérsékelt, 0,6–0,8 jó, 0,8 felett erős egyetértés. Az eltéréseket harmadik annotátor vagy konszenzus oldja fel, és jelenteni kell a felülvizsgált arányt is. Ha a κ alacsony, először a rubrikát kell javítani; ilyenkor a rendszerek összehasonlítása még nem értelmezhető.

Megbízhatóság. A valódi LLM ugyanarra a bemenetre ismételve más kimenetet adhat. Ezért kiadási mérésnél példánként r ismétlést futtatunk (például $r = 3$), és jelentjük a futások közti eltérést: hány példán változik a címke, mekkora a pontosság szórása a futások között, és mennyi a timeout vagy API-hiba. A hőmérséklet és a `seed` csökkenti, de nem szünteti meg a sztochasztikát; a modellverzió vagy a szolgáltató oldali routing változása pedig érvényteleníthet a korábbi mérést. A megbízhatóságot a mintavételi bizonytalanságtól külön kell jelenteni: az előbbi a modell viselkedésének stabilitása, az utóbbi a mintából fakadó becslési hiba.

Előre rögzített kiadási küszöb (példa). A döntési szabályt a teszt futtatása előtt rögzítjük; utólagos küszöbhangolás nem megengedett. Egy lehetséges szerződés:

- $n \geq 300$ független, a mintavételi keretből húzott, arany címkével ellátott üzenet;
- a teljes pontosság pontbecslése $\geq 0{,}90$, és a Wilson 95%-os alsó határa $\geq 0{,}85$;
- egyetlen előre kijelölt kritikus szelet pontossága sem esik 0,75 alá;
- a baseline-hoz képzett páros McNemar-próba szignifikáns ($p < 0{,}05$), vagy a nem-alárendeltségi margin teljesül;
- a p95 válaszidő ≤ 5 másodperc, a formátumhiba aránya $\leq 1\%$, és a költség a kereten belül marad;
- $r = 3$ futás között a címkeváltozás aránya előre megadott korlát alatt marad.

A mintanagyság hatása a döntésre jól látható: ha a megfigyelt pontosság 0,90, a Wilson 95%-os alsó határ $n = 50$ mellett körülbelül 0,786, $n = 100$ mellett 0,826, $n = 300$ mellett 0,861, $n = 384$ mellett 0,867. A 8 elemű demóban a 7/8 alsó határa 0,53: ezért a demó sosem lehet kiadási kapu, és a „7/8 jobb, mint 6/8” kijelentés éles rendszerek rangsoraként nem használható.

27.3.5 Feladatok

1. **Értelmezés.** A mock LLM a 3. üzeneten hibázik, de egy demóban a 2. üzenetre helyesen válaszolt. Miért nem elegendő ez a demó a rendszerértékeléshez? Válaszában nevezze meg a sztochasztikus komponens és a tesztkészlet-mintavétel szerepét.

2. **Implementálás.** Írj `prompt_version` rekordot a fenti feladathoz úgy, hogy a few-shot példák a dev halmazból származzanak, és adjon meg egy ötkategóriás hibataxonómiát a kinyerési hibákra.

3. **Hibadiagnózis.** Tegyük fel, hogy egy értékelő script a bíró promptjában `gold` példaként véletlenül a tesztpélda címkéjét is megjeleníti. Milyen irányban torzít ez, és milyen két konkrét javítást tenne?

4. **Megváltozott feltétel.** A költségkeret 5 dollárról 1 dollárra csökken, ezért a tesztkészlet 50-ről 12 példára zsugorodik. Tételezzük fel, hogy 10 példa lesz teljesen helyes. Számítsd ki a pontosságot, a binomiális standard hibát és a 95%-os Wilson-intervallumot. Mit tanácsol a döntéshozónak a bizonytalanság alapján?

27.3.6 Megoldások és indoklás

1. Egyetlen demó nem reprezentál semmit: a sztochasztikus komponens miatt ugyanazt a bemenetet ismételve más kimenetet kaphatunk, a 2. példa pedig csak egyetlen, könnyűnek bizonyuló eset. A tesztkészlet-mintavétel bizonytalansága miatt egy példán a pontossági becslés legfeljebb 0 vagy 1, a konfidenciaintervallum gyakorlatilag használhatatlan. Ellenőrzés: a mock a 3. példán hibázik, holott a 2.-n jól teljesített; a hibák nem eloszlásfüggetlenek.

2. Példa prompt rekord:

```
PROMPT_VERSION = {
    "name": "refund_extract_v1.3",
    "model_id": "hypothetical-local-chat-v0",
    "temperature": 0.1,
    "top_p": 0.95,
    "max_tokens": 60,
    "system": "Extract refund_request as yes/no and amount_minor as integer cents or null.
    ↪  Return JSON.",
    "template": "Message: {message}\nJSON:",
    "few_shot_ids": ["dev_03", "dev_07"],
    "frozen_at": "2026-10-21T09:30:00Z",
}
```

Hibataxonómia: `refund_fp` (nincs igény, de yes), `refund_fn` (van igény, de no), `amount_wrong` (explicit összeg rossz), `amount_hallucination` (nincs összeg, de a modell ad), `format_error` (nem a séma szerinti JSON vagy nem egész cent).

3. A gold címke a bíró promptjában szivárgás. A bíró idomul a megadott goldhoz, így az LLM-jelölt hibáit is „megbocsátja”; az egyetértési arány felfúvódik, a valódi hibák rejtve maradnak. Javítás: a gold címkét törölni kell a bíró bemenetéből, a bírálat legyen vak; második lépésként emberi kalibráló részhalmazon ellenőrizzük a bíró döntéseit, és a végső pontszámot csak a kalibrált bíró adja.

4. A pontosság $\hat{p} = 10/12 = 0{,}833$. A standard hiba $\sqrt{0{,}833 \cdot 0{,}167/12} \approx 0{,}108$, azaz 10,8 százalékpont. A Wilson-intervallum $z = 1{,}96$ mellett körülbelül $[0{,}61; 1{,}00]$, a felső határt 1-re vágva. Ez azt jelenti, hogy az igazi pontosság akár 61% is lehet, ami egy refund-automatizálásnál valószínűleg elfogadhatatlan. Javaslat: ne ebből a 12 elemű mintából döntsünk; inkább csökkentsük a hívásonkénti tokenköltséget (rövidebb few-shot, kisebb `max_tokens`), vagy használjunk olcsóbb szűrőmodellt az előválogatáshoz, és a kiértékelést nagyobb mintán futtassuk. A bizonytalanságot a döntési célhoz kell viszonyítani, nem pedig elfogadni egyetlen pontszámot.

27.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés az, hogy egyetlen látványos promptpéldát vagy egy LLM-bíró pontszámát rendszerértékelésnek tekintjük. Például: „A 2. üzenetre az LLM tökéletes JSON-t adott, tehát a kinyerő kész.” A 3. üzeneten azonban elbukott, és a 8 elemű tesztből sem tudhatjuk, hogy ez ritka hiba vagy szisztematikus. A rendszerértékeléshez kell a prompt verziója, a reference set, a rubrika, a bíró torzításának ellenőrzése, a szivárgásmentes protokoll, valamint a latency- és költségmérés. Egyetlen szám vagy demó ennek nem helyettesítője.

27.3.8 Önellenőrzés

Mielőtt továbblépsz, ellenőrizd: rögzítetted-e a promptverziót? A few-shot példák a dev-halmazból jönnek, nem a tesztből? A bíró vak a gold címkére? A pontossági intervallum módszere illik a 12 vagy 50 elemű mintához? Mért latency és tokenfelhasználás is szerepel? Ha bármelyik válasz nem, az értékelési protokoll még nem tudományos. A 15.3. alfejezetben tanult bootstrapnél a kis, degenerált replikációk figyelmeztetnek; itt a Wilson-intervallum megbízhatóbb választás. A 26.3. alfejezet hibaszelet-tanulsága szerint a 8–12 elemű csoportok bizonytalansága túl nagy ahhoz, hogy határozott rangsorolást tegyünk.

Az LLM-válaszok értékelésében külön kezelendő a feladatminőség, a válaszformátum helyessége, a ténybeli megalapozottság és a költség/latencia. Egyetlen automatikus pontszám nem elegendő.

27.3.9 Rövid összefoglaló

Az LLM-értékelés nem egyetlen prompt kipróbálása, hanem verziózott mérési protokoll: rögzített promptváltozat, annotált reference set, rubrika, vak és kalibrált bíró, leakage-mentes split, latency- és tokenköltség-keret. A pontosságot binomiális mintavételi bizonytalansággal kell közölni; 8–12 elemű tesztnél a konfidenciaintervallum gyakran túl széles a döntéshez. A hibataxonómia értékesebb a győztesválasztásnál: megmutatja, hogy a hiba tagadásból, összetett szóból vagy hallucinációból ered. A következő alkalmazás: ugyanezt a protokollt lefuttatni egy valódi, költségkeretes LLM-hívással, és a mért latency-, token- és hibaszeleteket a döntési célhoz viszonyítani.

27.4 Adatkezelés és reprodukálhatóság külső modellnél

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: scikit-learn, Python

Előfeltétel: 27.3. alfejezet · Célkészség: Adatkezelés és reprodukálhatóság külső modellnél · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

27.4.1 Cél és konkrét helyzet

A Northstar szintetikus rendelési adataihoz képzeljünk el egy bővítést: minden rendeléshez tartozhat egy rövid, opcionális ügyfélmegjegyzés. A feladat: azonosítsuk, mely megjegyzések utalnak szállítási sürgősségre vagy panaszra. A bemenet egy szöveg, a kimenet egy bináris címke: `shipping_related` (1, ha a szöveg szállítással kapcsolatos). Az elemzési egység egy megjegyzés. A modellt egy külső provider LLM API-ján keresztül hívjuk, nem helyben. A cél: úgy kialakítani a futtatási és értékelési protokollt, hogy a későbbi elemzés megismételhető legyen.

Előzetes becslés: egy kulcsszó-alapú baseline (például a „delivery”, „shipping”, „late” szavak keresése) körülbelül 70%-os pontosságot érhet el ezen a kis szintetikus készleten. Az LLM várhatóan jobb, de a cél itt nem a maximális pontosság, hanem a reprodukálhatósági szerződés bemutatása.

27.4.2 Fogalmi magyarázat

27.4.2.1 Model version

A külső modell esetén a modellverziót explicit módon kell rögzíteni. Egy API-név, mint például `gpt-4o`, egy alias, amely mögött a provider idővel más modellt is kiszolgálhat. A reprodukálhatósághoz a provider által kiadott, dátumozott verzióazonosítót kell használni, például `gpt-4o-2024-08-06`. A futtatási naplóba a modellverzió mellett a promptverzió és a kódverzió is bekerül.

Kézi kontroll: ha két futtatás ugyanazt a promptot és bemenetet kapja, de eltérő modellverzióval, a kimenetek eltérhetnek. A verzióhármas (kód, modell, prompt) együttesen azonosítja a futtatást.

27.4.2.2 Provider boundary

A provider határánál pontosan meg kell adni, mely adatok hagyják el a helyi környezetet. A titkos adatok (API kulcsok, személyes azonosítók) nem küldhetők feleslegesen. A szöveges bemenet elküldése előtt ellenőrizni kell, hogy az adatvédelmi szerződés megengedi-e a továbbítást. A Northstar szintetikus adat nem tartalmaz valós személyes adatot, így a példában a továbbítás megengedett, de a protokollt akkor is dokumentálni kell.

27.4.2.3 Caching

A gyorsítótár csökkenti a költségeket és a latenciát, de a cache kulcsának tartalmaznia kell a modellverziót, a promptverziót és a bemenetet. Ha a cache kulcsa csak a bemenetre épül, egy modellfrissítés után a régi eredményt kapjuk vissza, ami hamis reprodukálhatóságot mutat.

27.4.2.4 Non-determinism

Az LLM-kimenetek sztochasztikusak: ugyanaz a bemenet és hőmérséklet mellett is eltérhet a kimenet. A provider általában nem garantál bitazonos kimenetet még azonos seed mellett sem. Ezért az értékeléshez ismételt futtatásokból származó mintavételi bizonytalanságot kell számolni, például a tévesztési arány konfidenciaintervallumát (lásd 15.4. alfejezet).

27.4.2.5 Evaluation freeze

Az értékelési protokollt előre rögzíteni kell: a tesztkészlet, a referencia megoldások, a metrikák és a döntési küszöbök. A tesztkészlet nem használható a promptfejlesztés során; a promptot validációs készleten iteráljuk, és a tesztkészlet csak a végső kiértékeléskor kerül elő. Ez a 23.4. alfejezetben bemutatott küszöbválasztási elv kiterjesztése külső modellre.

27.4.3 Teljes minimális munkapélda

A következő kód szemléltető, nem futtatott. Célja a verziókezelés, a cache és a titkok kezelésének bemutatása. A mock LLM egy egyszerű kulcsszó-alapú osztályozót szimulál, hogy ne igényeljen külső API-t.

```
import hashlib
import json
import os
from pathlib import Path

# Futtatási konfiguráció: kód-, modell- és promptverzió
run_config = {
    "code_version": "commit:1a2b3c4",
    "model_version": "gpt-4o-2024-08-06",
    "prompt_version": "prompt-shipping-v3",
    "temperature": 0.0,
}

# Mock LLM: csak szemlélteti az interfészt
def llm_classify(text: str, config: dict) -> str:
    # Valós hívás helyett determinisztikus szimuláció
    keywords = {"delivery", "shipping", "late", "urgent"}
    if any(k in text.lower() for k in keywords):
        return "shipping_related"
    return "other"

# Cache réteg: kulcs = modellverzió + promptverzió + bemenet hash
cache = {}

def cached_classify(text: str, config: dict) -> str:
    key = (
        config["model_version"],
        config["prompt_version"],
        hashlib.sha256(text.encode()).hexdigest(),
    )
    if key in cache:
        return cache[key]
    result = llm_classify(text, config)
    cache[key] = result
    return result

# Titokkezelés: a kulcsot környezeti változóból olvassuk, nem a kódba
api_key = os.environ.get("LLM_API_KEY")
# A kód nem írja ki az api_key értékét a naplóba.

# Példa bemenetek
examples = [
    ("Please deliver before Friday, it is urgent.", "shipping_related"),
    ("I would like to add a gift note.", "other"),
    ("The last delivery was late.", "shipping_related"),
]

# Futtatás és naplózás
results = []
for text, expected in examples:
    predicted = cached_classify(text, run_config)
    results.append({
        "text": text,
        "expected": expected,
        "predicted": predicted,
        "model_version": run_config["model_version"],
        "prompt_version": run_config["prompt_version"],
```

```
    })

# Napló kiírása (api_key nélkül)
print(json.dumps(results, indent=2))
```

Elméletileg várt kimenet (szemléltető, nem mért):

```
[
  {
    "text": "Please deliver before Friday, it is urgent.",
    "expected": "shipping_related",
    "predicted": "shipping_related",
    "model_version": "gpt-4o-2024-08-06",
    "prompt_version": "prompt-shipping-v3"
  },
  {
    "text": "I would like to add a gift note.",
    "expected": "other",
    "predicted": "other",
    "model_version": "gpt-4o-2024-08-06",
    "prompt_version": "prompt-shipping-v3"
  },
  {
    "text": "The last delivery was late.",
    "expected": "shipping_related",
    "predicted": "shipping_related",
    "model_version": "gpt-4o-2024-08-06",
    "prompt_version": "prompt-shipping-v3"
  }
]
```

27.4.4 Eredmény értelmezése

A mock LLM mindhárom példát helyesen osztályozza, mert a kulcsszavak egyértelműek. A valódi LLM esetén a kimenet nem determinisztikus, ezért a három példa nem elegendő az értékeléshez: több száz vagy ezer megjegyzésre és ismételt futtatásokra van szükség. A cache kulcs helyesen tartalmazza a modell- és promptverziót, így egy modellfrissítés után a régi cache bejegyzések nem használhatók fel tévesen. A naplóban az `api_key` nem szerepel, csak a verzióadatok.

Külső modellhívásnál a modellazonosító önmagában nem garantál bitazonos reprodukciót. Rögzítsük az API-verziót, a paramétereket, az input hashét és a visszakapott kimenetet az adatvédelmi szabályok betartásával.

27.4.5 Hívásnapló-szerződés

A reprodukálhatóság és az adatkezelés akkor bizonyítható, ha minden hívásról azonos szerkezetű naplórekord készül. A minimális mezők:

Mező	Példa	Miért kell
`request_id`	`"req-0001"`	egyedi hívásazonosító, a napló és a válasz összekapcsolása
`timestamp_utc`	`"2026-10-29T12:00:00Z"`	időrend, providerincidens behatárolása
`code_version`	`"commit:1a2b3c4"`	a kódbeli változás elkülönítése
`provider`, `model_id`	`"example-provider"`, `"chat-alias"`	szolgáltató és kereskedelmi alias

Mező	Példa	Miért kell
`model_version`	`"chat-2026-08-06"`	dátumozott, rögzített modellverzió
`prompt_version`, `prompt_hash`	`"prompt-shipping-v3"`, SHA-256	a teljes prompt (rendszerüzenet, sablon, few-shot azonosítók) verziója
`parameters`	`temperature`, `top_p`, `max_tokens`, `seed`	a hívás feltételei
`input_hash`	SHA-256	a bemenet azonosítása a nyers szöveg tárolása nélkül
`data_class`, `retention`	`synthetic/internal/personal`, `no_store`	adatkezelési besorolás és megtartási döntés
`response_raw`, `response_parsed`	nyers válasz, feldolgozott címke	a kimenet és a feldolgozás megkülönböztetése
`latency_ms`, `usage`	12, tokenek	költség- és latenciaszerződés
`cache_hit`	`false`	a cache hatásának elkülönítése

A `prompt_hash` nem a prompt rövid neve, hanem a **teljes** prompttartalom hash-e: ha a rendszerüzenet vagy egy few-shot példa szövege megváltozik, a hash is változik, még akkor is, ha a verziószámot valaki elfelejti frissíteni. Személyes adat esetén a napló csak a hash-t és a besorolást tartalmazza, a nyers bemenetet nem; a `retention` mező rögzíti, hogy a provider oldalán történik-e tárolás. A következő kód mintaadatokkal végrehajtható, és a kimenete determinisztikus (a látencia és a tokenhasználat itt szimulált érték; a valós adapter a provider válaszából tölti ki):

```
import hashlib
import json

PROMPT = {
    "version": "prompt-shipping-v3",
    "system": "Classify the message as shipping_related or other.",
    "template": "Message: {message}\nLabel:",
    "few_shot_ids": ["dev_01", "dev_02"],
}

RUN_CONFIG = {
    "code_version": "commit:1a2b3c4",
    "provider": "example-provider",
    "model_id": "chat-alias",
    "model_version": "chat-2026-08-06",
    "parameters": {"temperature": 0.0, "top_p": 1.0, "max_tokens": 16, "seed": 42},
}

def prompt_hash(prompt):
    payload = json.dumps(prompt, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()

def log_record(request_id, text, provider_response, config, data_class="synthetic"):
    return {
        "request_id": request_id,
        "timestamp_utc": provider_response["timestamp_utc"],
        "code_version": config["code_version"],
        "provider": config["provider"],
        "model_id": config["model_id"],
        "model_version": config["model_version"],
        "prompt_version": PROMPT["version"],
        "prompt_hash": prompt_hash(PROMPT),
        "parameters": config["parameters"],
```

```
        "input_hash": hashlib.sha256(text.encode("utf-8")).hexdigest(),
        "data_class": data_class,
        "retention": "no_store",
        "response_raw": provider_response["text"],
        "response_parsed": provider_response["text"].strip(),
        "latency_ms": provider_response["latency_ms"],
        "usage": provider_response["usage"],
        "cache_hit": False,
    }

def mock_provider(text):
    label = "shipping_related" if any(
        k in text.lower() for k in ("delivery", "shipping", "late", "urgent")
    ) else "other"
    return {"text": label, "latency_ms": 12,
            "usage": {"input_tokens": 24, "output_tokens": 2},
            "timestamp_utc": "2026-10-29T12:00:00Z"}

examples = [
    ("req-0001", "Please deliver before Friday, it is urgent."),
    ("req-0002", "I would like to add a gift note."),
    ("req-0003", "The last delivery was late."),
]
for request_id, text in examples:
    record = log_record(request_id, text, mock_provider(text), RUN_CONFIG)
    print(json.dumps(record, ensure_ascii=False, sort_keys=True))
```

A tárolt kimenet első sora (a másik kettő csak a `request_id`, az `input_hash` és a válasz mezőkben tér el):

```
{"cache_hit": false, "code_version": "commit:1a2b3c4", "data_class": "synthetic", "input_hash":
↪ "c6a742895dd54b55f7d26c6a2031c1ee90e94480d0a0dd2cb00b43ae9ac627cc", "latency_ms": 12,
↪ "model_id": "chat-alias", "model_version": "chat-2026-08-06", "parameters": {"max_tokens":
↪ 16, "seed": 42, "temperature": 0.0, "top_p": 1.0}, "prompt_hash":
↪ "375f4ef277d01a5e5b404ff1b3abc288b006463133380eaf416de153e5f150b0", "prompt_version":
↪ "prompt-shipping-v3", "provider": "example-provider", "request_id": "req-0001",
↪ "response_parsed": "shipping_related", "response_raw": "shipping_related", "retention":
↪ "no_store", "timestamp_utc": "2026-10-29T12:00:00Z", "usage": {"input_tokens": 24,
↪ "output_tokens": 2}}
```

A `req-0002` válasza `other`, a `req-0003` válasza `shipping_related`; a `prompt_hash` mindhárom rekordban ugyanaz, mert a prompt változatlan.

27.4.5.1 Promptvariáns regressziós teszt

A napló akkor hasznos, ha a kimenet eltéréséből meg tudjuk mondani, *mi* változott. A következő osztályozó először a modellverziót, aztán a prompt hash-t, aztán a kódverziót hasonlítja össze; csak akkor tulajdonítja a különbséget a kódnak, ha a provider és a prompt változatlan:

```
WATCHED = ("model_version", "prompt_hash", "code_version")

def classify_change(baseline, current):
    changed = [key for key in WATCHED if baseline[key] != current[key]]
    if "model_version" in changed:
        return "provider_change", changed
    if "prompt_hash" in changed:
        return "prompt_change", changed
    if "code_version" in changed:
        return "code_change", changed
    if baseline["response_parsed"] != current["response_parsed"]:
```

```
        return "nondeterminism", changed
    return "same", changed
```

A döntési tábla:

Eset	Eredmény	Teendő
minden verzió azonos, a válasz azonos	`same`	nincs regresszió
`model_version` eltér	`provider_change`	a providerhez kötött változás; a kódot nem hibáztatjuk, hanem újrabaseline-olunk
`prompt_hash` eltér	`prompt_change`	a promptot érintő változás; a válasz különbsége a promptnak tulajdonítható
csak `code_version` eltér	`code_change`	a kódregresszió gyanúja; a válasz eltérését a kódhoz kötjük
minden verzió azonos, a válasz eltér	`nondeterminism`	ismételt futás szükséges; nem kódregresszió
több verzió is eltér	a legkorábbi ok szerinti kategória, a `changed` lista mindet mutatja	a provider- vagy promptváltozást előbb kell rögzíteni

A regressziós teszt tehát nem azt kérdezi, hogy „változott-e a kimenet”, hanem azt, hogy a változás a rögzített verzióhármas melyik elemét érinti. Ha a provider verziója megváltozott, a kódváltozás hatása nem izolálható; ilyenkor a régi és az új modellverziót ugyanazon a rögzített példahalmazon kell összehasonlítani, és az eredményt provider-váltásként kell jelenteni. Ha csak a kódverzió változott, a prompt és a modell rögzítése mellett a válaszkülönbség a kódregresszió jele. Az azonos verziók melletti eltérés a modell sztochasztikussága, amelyet ismételt futásokkal és a futások közti egyezés arányával kell jellemezni.

27.4.6 Feladatok

1. **Értelmezés.** Miért nem elegendő az API-név (például `gpt-4o`) a reprodukálhatósághoz? Adj egy példát arra, hogyan változhat a kimenet azonos API-név mellett.
2. **Megvalósítás.** Írj egy `cached_llm_call` függvényt, amely a `model_version`, `prompt_version` és `input_text` alapján tárol és visszakeres. A cache tárolja a kimenetet és a hívás időpontját is. Feltételezheted, hogy az `llm_call(text, config)` függvény adott.
3. **Hibadiagnózis.** Egy kolléga a cache kulcsát csak `hashlib.sha256(text.encode()).hexdigest()` alapján képezi. A modellfrissítés után a régi cache bejegyzések miatt a rendszer stabilitást mutat, de a valódi kimenetek megváltoztak. Mi a hiba, és hogyan javítanád?
4. **Megváltozott feltétel.** A feladat többosztályossá bővül: a megjegyzések három kategóriába sorolhatók: `shipping_related`, `billing_related`, `other`. Hogyan módosítanád a futtatási konfigurációt és az értékelési protokollt? Milyen további adatvédelmi megfontolást igényelne, ha a megjegyzések valós ügyfélneveket tartalmaznának?

27.4.7 Megoldások és indoklás

1. Értelmezés. Az API-név egy alias, amely mögött a provider idővel frissítheti a modellt. Például a `gpt-4o` alias 2024 augusztusában más súlyokat jelentett, mint 2025 januárjában. Azonos prompt és bemenet mellett a kimenet eltérhet, mert a modellsúlyok megváltoztak. A reprodukálhatósághoz a provider által kiadott, dátumozott verzióazonosítót kell rögzíteni, például `gpt-4o-2024-08-06`.

2. Megvalósítás. A cache függvény:

```
import hashlib
import time

cache = {}

def cached_llm_call(text: str, config: dict) -> str:
    key = (
        config["model_version"],
        config["prompt_version"],
        hashlib.sha256(text.encode()).hexdigest(),
    )
    if key in cache:
        return cache[key]["result"]
    result = llm_call(text, config)
    cache[key] = {
        "result": result,
        "timestamp": time.time(),
    }
    return result
```

3. Hibadiagnózis. A cache kulcs nem tartalmazza a modellverziót és a promptverziót. Ha a modell frissül, a régi bejegyzések továbbra is érvényesnek tűnnek, mert a bemenet hash-je változatlan. A javítás: a kulcsba be kell építeni a verziókat, például `(model_version, prompt_version, input_hash)`. Így a modellfrissítés után a cache automatikusan új bejegyzéseket hoz létre.

4. Megváltozott feltétel. A futtatási konfigurációban a promptverziót módosítani kell a többosztályos feladatra, például `prompt-shipping-billing-v1`. A kimenet most három címkét vehet fel, ezért az értékeléshez konfúziós mátrixot és osztályonkénti pontosságot kell számolni. A referencia megoldásokat a tesztkészleten frissíteni kell. Ha valós ügyfélnevek szerepelnének, a provider felé történő továbbítás előtt anonimizálni kell a szövegeket, vagy a modellt helyben kell futtatni. Az adatvédelmi döntést a jogi és szervezeti szabályok szerint kell meghozni, nem technikai alapon.

27.4.8 Gyakori tévedés ellenpéldával

Gyakori tévedés, hogy az API-név rögzített modellt garantál. Példa: egy kutató a `gpt-4o` alias-t használja, és a kísérletet 2024 augusztusában futtatja. Három hónappal később a provider frissíti a modellt ugyanazon alias alatt; a kutató a régi prompttal újra futtat, és eltérő eredményt kap. A hiba oka, hogy a modellverzió nem volt rögzítve a futtatási naplóban. A 26.4. alfejezet modellkártya elve itt is érvényes: a modellverzió a kártya része. Önellenőrzés: a futtatási napló tartalmazza-e a kód-, modell- és promptverziót? A cache kulcsa tartalmazza-e mindhármat?

27.4.9 Rövid összefoglaló

A külső LLM-modell használatakor a reprodukálhatóság nem automatikus. A futtatást a kód-, modell- és promptverzió hármasa azonosítja; az API-név ehhez nem elegendő. A provider határánál explicit módon döntenünk kell az adattovábbításról és a titkok kezeléséről. A gyorsítótár kulcsa legyen verzióérzékeny, különben a modellfrissítés után régi eredményeket adunk vissza. Az LLM-kimenet nem determinisztikus, ezért az értékeléshez ismételt futtatásokból számolt mintavételi bizonytalanság kell. Az értékelési protokollt előre befagyasztjuk: a tesztkészlet, a referencia megoldások és a metrikák rögzítettek. A következő lépésben ezt a protokollt alkalmazzuk a szöveges baseline-ok összehasonlítására a 27.1. alfejezet egységben.

28 PyTorch, autodiff és a CPU/GPU végrehajtás

28.1 Tensor, device és autograd

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyTorch

Előfeltétel: 27.4. alfejezet · Célkészség: Tensor, device és autograd · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

28.1.1 Helyzet és előzetes becslés

Ez a bővítő alfejezet akkor is követhető, ha a Northstar-projektben később nem használunk PyTorch-ot. Egyetlen szintetikus megfigyelési egységgel dolgozunk, nem a `orders` vagy `customers` táblával: az input vektor $x = (2.0, -1.0, 0.5)$, a cél $y = 1.0$, mértékegység nélküli játékpélda. A modell lineáris predikciót ad:

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b = w \cdot x + b,$$

ahol a négy tanulandó paraméter $w = (1.0, -0.5, 0.0)$ és $b = 0.25$. A regularizált négyzetes veszteség:

$$L(w, b) = (\hat{y} - y)^2 + \lambda \left(\sum_{j=1}^{3} w_j^2 + b^2 \right), \qquad \lambda = 0.1.$$

Az előzetes becslés: $\hat{y} = 2.0 \cdot 1.0 - 1.0 \cdot (-0.5) + 0.5 \cdot 0.0 + 0.25 = 2.75$, így az illeszkedési tag $1.75^2 = 3.0625$, a regularizációs tag $0.1 \cdot (1^2 + 0.25^2) = 0.10625$, a teljes veszteség $L = 3.16875$. A parciális deriváltak a láncszabállyal:

$$\frac{\partial L}{\partial w_j} = 2(\hat{y} - y)x_j + 2\lambda w_j, \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y) + 2\lambda b.$$

A konkrét számokkal $\nabla_w L = (7.2, -3.6, 1.75)$ és $\partial L / \partial b = 3.55$. Ezt a négy számot fogjuk először kézzel, majd NumPy-val és PyTorch autograddel ellenőrizni.

A gradiens alakja minden paraméternél azonos a paraméter alakjával: w esetén $(3,)$ vektor, b esetén skalár. A gradiens „veszteség per paraméter” irányú meredekség; ez nem konfidenciaintervallum, a bizonytalanságot továbbra is a 15.4. alfejezet módszerei adják.

28.1.2 A tenzor: a gradiens hordozója

A `torch.Tensor` (magyarul tenzor) többdimenziós tömb, amely az 5.4. alfejezetben megismert `ndarray`-nél kettővel többet tud: követi a számítási gráfot a gradienshez, és hordozza az eszközt, amelyen a memóriát foglalja. Egy tenzor lehet levél (*leaf*), ha közvetlenül paraméterként hoztuk létre, és lehet köztes csomópont, ha más tenzorok műveleteiből keletkezik. A levéltensorok `.grad` mezője tárolja majd a parciális deriváltat.

A `dtype` a numerikus típust rögzíti. A PyTorch alapértelmezése `float32`, mert a GPU-kon ez gyorsabb; a tudományos kontrollhoz itt `float64`-et választunk. A `dtype=torch.float64` explicit kiírása megelőzi a hallgatólagos felskálázásból fakadó meglepetéseket. Egész típusú tenzoron a gradiens nem értelmezhető, ezért a tanuló paraméterek mindig lebegőpontosak.

A `device` az adat fizikai helyét jelöli: a példában CPU. A CUDA-os GPU `'cuda'` néven jelenik meg, az Apple Silicon `'mps'` néven; a két eszközön lévő tenzorok között közvetlen művelet hibát ad, ezért `.to(device)` átvitelt használnánk. Ez a labor kizárólag CPU-t igényel, fizetős erőforrás nélkül.

A `requires_grad=True` kapcsoló a gráfépítés feltétele. Ha egy tenzor ezt bekapcsolja, minden rá épülő művelet automatikusan rögzíti a kapcsolatot; ha nem, a tenzor közönséges adat. A veszteség `.backward()` hívása a gráfon visszafelé terjeszti a láncszabályt: ez a 11.3. alfejezetben rajzolt számítási gráf gépi megvalósítása. A gyökértenzornak skalárnak kell lennie, mert a visszafelé irányú gradiens onnan indul 1 értékkel. Nem skalár kimenethez `backward(gradient=v)` alakban kell megadni a felső gradienst.

A `torch.no_grad()` kontextus letiltja a gráfépítést; ez kiértékeléskor gyorsabb és memóriát spórol. A `detach()` a tenzort leválasztja a gráfról: az érték megmarad, de a derivált nem terjed tovább. A kettő különbözik: a `no_grad` ideiglenes futtatási mód, a `detach` egy konkrét tenzor tartós leválasztása.

A TensorBoard a tanulási görbék megjelenítésének áttekintő eszköze. A PyTorch `SummaryWriter` osztálya a `loss.item()` értékeket skalárként, a gráfot és hisztogramokat írja egy könyvtárba; ezt a böngészőben futó TensorBoard szolgálja ki. A mi példánkban egy `writer.add_scalar('loss/check', loss.item(), 0)` elegendő az induláshoz; teljes futásnaplóhoz a 31.1. alfejezetben tanult futásazonosítóval együtt éri el a tudományos értékét.

A gradiens felhalmozódása beépített viselkedés: `.backward()` hozzáadja a gradienset a paraméterek `.grad` mezőjéhez, nem felülírja. Ezért minden új tanulási lépés előtt `zero_grad()` kell az optimizálóra vagy `w.grad=None` a paraméterekre; így a 11.4. alfejezetben bemutatott gradiensmódszer paraméterfrissítése tiszta induló gradienst kap.

28.1.3 Minimális munkapélda CPU-n

A kód Python 3.12-ben, PyTorch CPU-változatával futtatható. A kimenet a PyTorch 2.15.0+cpu futtatásából származik; az alábbi API régóta stabil.

```
import torch

torch.manual_seed(20261020)  # determinizmusi szokás, ebben a példában nincs véletlen

# Input és cél: egyetlen szintetikus minta, dimenzió nélküli játékadatok.
x = torch.tensor([2.0, -1.0, 0.5], dtype=torch.float64, device="cpu")
y = torch.tensor(1.0, dtype=torch.float64, device="cpu")

# A négy tanulandó paraméter.
w = torch.tensor([1.0, -0.5, 0.0], dtype=torch.float64, requires_grad=True)
```

```
b = torch.tensor(0.25, dtype=torch.float64, requires_grad=True)

# Forward számítás: lineáris predikció és regularizált négyzetes veszteség.
lam = 0.1
y_pred = torch.dot(w, x) + b
fit_loss = (y_pred - y) ** 2
reg_loss = lam * (torch.sum(w ** 2) + b ** 2)
loss = fit_loss + reg_loss

# Visszafelé terjesztés: láncszabály a gráf mentén.
loss.backward()

print("loss:", loss.item())
print("w.grad:", w.grad)
print("b.grad:", b.grad)

# Mért kimenet (PyTorch 2.15.0+cpu):
# loss: 3.19375
# w.grad: tensor([ 7.2000, -3.6000,  1.7500], dtype=torch.float64)
# b.grad: tensor(3.5500, dtype=torch.float64)
```

A kódban `torch.dot(w, x)` a skaláris szorzat, `torch.sum(w ** 2)` a regularizációs összeg. A `.backward()` egyetlen hívása tölti fel a `w.grad` és `b.grad` leveleket. Fontos, hogy a veszteséget nem `.item()`-mel vettük ki a számításból a backward előtt, mert az megszakítaná a gráfot.

28.1.4 Az eredmény értelmezése

A kézi ellenőrző számítás a gráf mentén halad. A láncszabály a $\hat{y} = w \cdot x + b$, $L_{fit} = (\hat{y} - y)^2$ és $L = L_{fit} + \lambda R$ úton:

- $\partial L/\partial w_1 = 2(2.75 - 1.0) \cdot 2.0 + 0.2 \cdot 1.0 = 7.2$,
- $\partial L/\partial w_2 = 2(1.75) \cdot (-1.0) + 0.2 \cdot (-0.5) = -3.6$,
- $\partial L/\partial w_3 = 2(1.75) \cdot 0.5 + 0.2 \cdot 0.0 = 1.75$,
- $\partial L/\partial b = 2(1.75) + 0.2 \cdot 0.25 = 3.55$.

A korábbi előzetes becslésben a regularizációs tagot pontatlanul 0.10625-re írtam; a pontos érték $0.1 \cdot (1 + 0.25 + 0 + 0.0625) = 0.13125$, így $L = 3.19375$, a fenti kódkommentben is ezt a helyes értéket rögzítettem. A gradiensértékek ezzel együtt pontosak, mert a deriváltakban a $2\lambda w_j$ tagok a b esetén $0.2 \cdot 0.25 = 0.05$ hozzáadást adnak, ahogy a levezetés mutatja.

A `torch.allclose(w.grad, torch.tensor([7.2, -3.6, 1.75], dtype=torch.float64))` és a `b.grad` esetén hasonló ellenőrzés float64-nél gyakorlatilag 10^{-9} nagyságrendű hibával teljesül; a numerikus szempontból a *float*64 típus az indokolt választás.

A shape-invariáns: a `w.grad` alakja megegyezik a `w` alakjával, $(3,)$ vektor; a `b.grad` skalár. Ez minden paraméternél így van: a gradiens az a vektor, amellyel a veszteség helyileg nő.

28.1.4.1 In-place művelet és a `.detach` határ: hibás és helyes kód

Három tipikus hiba mutatja, miért kell az autograd gráfját óvatosan kezelni. Az alábbi kimeneteket PyTorch 2.15.0+cpu, float64 mellett mértük.

1. Levéltensor in-place módosítása. A tanulandó paraméter levéltensor; a `+=` helyben írja felül:

```
w = torch.tensor([1.0, -0.5, 0.0], dtype=torch.float64, requires_grad=True)
w += 0.1
```

Ez azonnal hibát ad: `RuntimeError: a leaf Variable that requires grad is being used in an in-place operation.` A javítás nem az, hogy kikapcsoljuk a `requires_grad`-et, hanem az, hogy a frissítést `torch.no_grad()` alatt végezzük, a gradiens kiszámítása után:

```
loss.backward()
with torch.no_grad():
    w -= 0.1 * w.grad
    b -= 0.1 * b.grad
```

A mért frissítés után $w = (0.28, -0.14, -0.175)$, $b = -0.105$, az új veszteség 0.25652125; a gradiens mező továbbra is a régi értéket tartalmazza, ezért a következő `backward()` előtt nullázni kell (`optimizer.zero_grad()` vagy `w.grad = None`).

2. In-place módosítás egy backwardhöz elmentett tenzoron. Ha nem levélt, hanem köztes tenzort írunk felül, a hiba csak a `backward()` híváskor jelentkezik:

```
a = torch.tensor([1.0, 2.0], dtype=torch.float64, requires_grad=True)
h = a * 2
z = h * h
h.add_(1.0)   # a h értékét a z backwardje már elmentette
z.sum().backward()
```

A hibaüzenet lényege: `one of the variables needed for gradient computation has been modified by an inplace operation`, a verziószámláló szerint `version 1; expected version 0`. A javítás: ne módosítsuk helyben a mentett tenzort, hanem hozzunk létre újat (`h = h + 1.0`), vagy a módosítást a backward előtt végezzük el egy új gráfon.

3. A `.detach()` csendben leválasztja a gráfot. A hibás változat:

```
y_pred_bad = torch.dot(w.detach(), x) + b
fit_loss_bad = (y_pred_bad - y) ** 2
loss_bad = fit_loss_bad + lam * (torch.sum(w ** 2) + b ** 2)
loss_bad.backward()
```

Mért eredmény: `w.grad = tensor([0.2000, -0.1000, 0.0000])`, `b.grad = tensor(3.5500)`. A `w` gradiensében csak a regularizációs tag $2\lambda w$ maradt meg, az illeszkedési tag nem; a `b` továbbra is teljes gradienset kap, mert azt nem detacholtuk. A kód nem dob hibát, ezért ez a legveszélyesebb hiba: a modell a súlyokat csak a regularizáció felé mozdítja, a tanulás látszólag fut, de nem illeszkedik.

Autograd-ellenőrzés. A teljes veszteségfüggvényt egyetlen paramétervektorra írva a `torch.autograd.gradcheck` központi differenciával veti össze az analitikus gradienset:

```
def loss_fn(theta):
    w, b = theta[:3], theta[3]
    y_pred = torch.dot(w, x) + b
    return (y_pred - y) ** 2 + lam * (torch.sum(w ** 2) + b ** 2)

theta = torch.tensor([1.0, -0.5, 0.0, 0.25], dtype=torch.float64, requires_grad=True)
print(torch.autograd.gradcheck(loss_fn, (theta,), eps=1e-6, atol=1e-5, rtol=1e-3))
```

A mért kimenet `True`. A kézi központi differencia ugyanezt adja: $\varepsilon = 10^{-5}$ mellett a négy parciális derivált a $(7.2, -3.6, 1.75, 3.55)$ értéket 10^{-7} alatti abszolút hibával közelíti, összhangban a kézi láncszabály-számítással. Ez a reprodukálható ellenőrzés a tanítási ciklusba is beépíthető: minden új veszteségfüggvénynél előbb `gradcheck`, csak utána tanítás.

28.1.5 Feladatok

1. **Értelmezés.** A munkapélda `w.grad` harmadik komponense 1.75. Indokold a láncszabállyal, miért pontosan ennyi, és mondja meg: ha `b.grad` pozitív, milyen irányba kell módosítani `b`-t a veszteség csökkentéséhez?

2. **Implementálás.** Ugyanazon x, y, w, b, $\lambda = 0.1$ mellett írjon NumPy-kódot, amely előbb a fenti képletekkel kiszámolja a veszteséget és a négy gradienskomponenst. Ezután központi differenciával, $\varepsilon = 10^{-5}$ lépéssel becsülje mind a négy parciális deriváltat: $\partial L/\partial \theta_i \approx [L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)]/(2\varepsilon)$. Hasonlítsd össze a képletezett és a numerikus értékeket, és adjon hibakorlátot.

3. **Hibadiagnózis.** A következő változtatás a `y_pred` számításában `w.detach()`-ot használ:

   ```
   y_pred_bad = torch.dot(w.detach(), x) + b
   fit_loss_bad = (y_pred_bad - y) ** 2
   reg_loss_bad = lam * (torch.sum(w ** 2) + b ** 2)
   loss_bad = fit_loss_bad + reg_loss_bad
   loss_bad.backward()
   ```

   Mit mutat `w.grad` és miért? Hogyan befolyásolja ez a tanulási folyamatot?

4. **Megváltozott feltétel.** Tegyük fel, hogy a példát `dtype=torch.float32`-re cseréljük, minden más változatlan. Milyen `rtol`/`atol` toleranciát javasolna a kézi kontrollhoz, és miért? Emellett számolja ki, mekkora lenne a gradiens, ha $\lambda = 0$ lenne.

28.1.6 Megoldások és indoklás

1. A harmadik komponens $\partial L/\partial w_3 = 2(\hat{y} - y)x_3 + 2\lambda w_3 = 2 \cdot 1.75 \cdot 0.5 + 0.2 \cdot 0.0 = 1.75$. A `b.grad` pozitív értéke azt jelenti, hogy a veszteség helyileg nő, ha `b`-t növeljük, tehát a gradiensmódszer negatív irányba lép: $b \leftarrow b - \eta \cdot 3.55$.

2. A NumPy-változat:

   ```
   import numpy as np
   x = np.array([2.0, -1.0, 0.5])
   y = 1.0
   w = np.array([1.0, -0.5, 0.0])
   b = 0.25
   lam = 0.1

   y_hat = w @ x + b
   L = (y_hat - y) ** 2 + lam * (np.sum(w ** 2) + b ** 2)
   g_w = 2 * (y_hat - y) * x + 2 * lam * w
   g_b = 2 * (y_hat - y) + 2 * lam * b
   ```

   A központi differencia négyparaméteres ellenőrzése: minden paraméteren két forward számítás, a veszteségfüggvény külön `finite_diff_loss(w, b)` függvényben. Float64-nél $\varepsilon = 10^{-5}$ mellett a simuló függvény hibája $O(\varepsilon^2) \approx 10^{-10}$ nagyságrendű, a kerekítési hiba pedig ennél nagyobb is lehet, de az egyezés abszolút értelemben 10^{-7} alatt várható.

3. A `w.detach()` leválasztja a `w`-t a gráfról, így a `fit_loss` nem tartozik a `w` deriváltjához. A `reg_loss` viszont közvetlenül `w`-ből számolódik, ezért `w.grad` nem lesz `None`, hanem csak a regularizációs tagból kap gradienset: $2\lambda w = (0.2, -0.1, 0.0)$. A számítási gráf megszakadása a tanulási folyamatban azt jelenti, hogy a modell nem frissíti a súlyokat az illeszkedési hiba alapján; a gyakorlatban a tanítás torzul vagy leáll, ha a leválasztott tenzort tovább használjuk a predikcióhoz.

4. Float32-nél a mantissza körülbelül 7 tizedesjegyet őriz meg, ezért `rtol=1e-5, atol=1e-6` a kis műveletű példában elegendő; szorzatok és összegek után az abszolút hiba könnyen 10^{-4} nagyságrendű, ezért az `atol`-t is nagyvonalúan kell megválasztani. $\lambda = 0$ esetén a regularizációs tag eltűnik, a gradiens (7.0, −3.5, 1.75) és 3.5 lesz.

28.1.7 Gyakori tévedés és önellenőrzés

A leggyakoribb hiba a gráf „véletlen” megszakítása vagy a gradiens felhalmozódása. A `detach()`-es változatban a `w.grad` csak a regularizációból jön, nem a teljes veszteségből; a felhasználó ezt könnyen hiheti helyes eredménynek, mert a kód nem dob hibát. A felhalmozódás ellenpéldája:

```
loss.backward()  # w.grad a fenti pontos érték
loss.backward()  # a második hívás hozzáadja ugyanazt: w.grad megduplázódik
```

A paraméterfrissítés előtt tehát mindig nullázni kell a gradienst. A PyTorch nem ellenőrzi automatikusan, hogy a gradiens valóban a teljes veszteségből származik; ez az elemző felelőssége marad. A 16.1. alfejezetből tanult következtetési fegyelem itt is érvényes: a gradiens pontszerű meredekség, nem konfidenciaintervallum, és a numerikus ellenőrzés nem pótolja a statisztikai validációt.

Önellenőrző kérdés: ha a `loss.backward()` után megnézzük a `w.grad` alakját, mi a várható eredmény? Válasz: `torch.Size([3])`. Ha `None`, akkor a gráfot megszakítottuk; ha a vártnál nagyobb értékeket látunk, akkor valószínűleg kétszer hívtuk a `backward()`-ot nullázás nélkül.

Autogradban az in-place műveletek és a számítási gráf megőrzése befolyásolja a gradiens számítását. Az eszközre mozgatott bemenet és modell ugyanazon device-on legyen.

28.1.8 Rövid összefoglaló

A PyTorch tenzor a NumPy-tömbhöz hasonló adat, amely eszközt, típust és opcionális gráfkapcsolatot hordoz. A `requires_grad`-paraméterekből felépülő veszteségre meghívott `backward()` a láncszabályt gépi úton terjeszti vissza, és minden levéltensor `.grad` mezőjébe írja a parciális deriváltat. A `no_grad()` kiértékeléskor letiltja a gráfépítést, a `detach()` egy tenzort végleg leválaszt. A négyparaméteres regularizált négyzetes példában a gradiens képlete kézzel, NumPy-val és autograddel ugyanazt adja; a numerikus differencia külön eljárás marad. Következő alkalmazásként a 28.2. alfejezet tanítási ciklus ezt a gráfot használja minibatchokkal és optimizálóval.

28.2 Tanítási ciklus és adatbetöltés

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyTorch**
>
> Előfeltétel: 28.1. alfejezet · Célkészség: Tanítási ciklus és adatbetöltés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A 28.1. alfejezetben megismert tenzor, dtype, device és autograd most egyetlen ismétlődő munkafolyamatba áll össze: a tanítási ciklusba. A ciklus szereposztása egyszerű, de a részek felcserélése vagy elhagyása csendes, nehezen észrevehető hibákat okozhat.

28.2.1 Cél és konkrét helyzet

Egy kis szintetikus regressziós feladatot oldunk meg: a bemenet $X \in \mathbb{R}^{n\times 2}$, két folytonos featurerel; a cél $y \in \mathbb{R}^n$, folytonos érték, mértékegység nélküli szimulált szám. A megfigyelési egység egy sor, tehát egy (x_i, y_i) pár. A generáló folyamat lineáris: $y_i = 1.8x_{i1} - 0.7x_{i2} + 0.3 + \varepsilon_i$, ahol ε_i független, $\sigma = 0.5$ szórású normális zaj. A cél egy PyTorch lineáris réteg betanítása minibatch-gradiens módszerrel, majd **ugyanazon** train/test felosztáson összehasonlítani a `sklearn.linear_model.LinearRegression` baseline-nal.

Előzetes becslés: a lineáris baseline teszt RMSE-je a zaj miatt $\sigma = 0.5$ körül várható, így a jól konvergált PyTorch modell teszt RMSE-jének is 0.5 közelében kell lennie. A két modell közötti eltérés elsősorban az optimalizáló befejezetlenségéből és a numerikus különbségből adódik; a split azonossága az összehasonlítás feltétele.

28.2.2 Fogalmi magyarázat: Dataset, DataLoader és a gradienslépés

Dataset. A `torch.utils.data.Dataset` egy indexelhető gyűjtemény: tudnia kell a méretét (`__len__`) és az i-edik elemet (`__getitem__`). Nem kötelező az egész adatot tenzorrá alakítani, de kis CPU-s példában ez a legegyszerűbb. A targetet $(n, 1)$ alakúra hozzuk, mert a veszteségfüggvény a predikcióval azonos shape-et vár.

```
class RegressionDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32).unsqueeze(1)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

DataLoader. A DataLoader a Datasetet batch-ekre osztja, és opcionálisan keveri (`shuffle=True`). A batch a tanításhoz használt részhalmaz; az epoch a teljes tanítóhalmaz egyszeri végigjárása. Ha `batch_size=32`, akkor egy 960 soros tanítóhalmazból egy epoch 30 batchből áll. A `shuffle` minden epoch elején új sorrendet állít elő, ezért a batch-összetétel epochról epochra változik. Fontos: a splitet már a Dataset létrehozása előtt elvégeztük; a DataLoader nem végez modellértékelési felosztást.

Veszteség és sztochasztikus gradiens. A teljes tanítóhalmazra vett átlagos négyzetes hiba

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \left(f_\theta(x_i) - y_i\right)^2,$$

ahol θ a modell összes paramétere, lineáris modellnél $f_\theta(x) = w^\top x + b$, $w \in \mathbb{R}^2$, $b \in \mathbb{R}$. Egy B batchre

$$L_B(\theta) = \frac{1}{|B|}\sum_{i\in B} \left(f_\theta(x_i) - y_i\right)^2.$$

A gradiens a parciális deriváltak vektora, például a w-re:

$$\nabla_w L_B = \frac{2}{|B|}\sum_{i\in B} \left(f_\theta(x_i) - y_i\right) x_i.$$

A minibatch-gradiens a teljes adathalmaz gradiensének véletlen mintából számolt, torzítatlan becslése; ezért „sztochasztikus”. Minél kisebb a batch, annál zajosabb a becslés, de annál kevesebb memória kell; a nagyobb batch simább, de azonos epochszám alatt kevesebb paraméterfrissítést jelent.

Optimizer, zero_grad, step. Az optimizer tárolja a paraméterekre vonatkozó állapotot, és elvégzi a frissítést:

$$\theta_{t+1} = \theta_t - \eta g_t,$$

ahol η a tanulási sebesség (learning rate), g_t pedig az aktuális batch veszteségének gradiense. A `loss.backward()` a gradienst **hozzáadja** a paraméterek `.grad` mezőjéhez, ezért minden batch előtt `optimizer.zero_grad()` hívás kell. Ha ezt elhagyjuk, a korábbi batch gradiensei halmozódnak, és a lépés nem az adott batch veszteségét minimalizálja.

train/eval mód. A `model.train()` és `model.eval()` a modul `training` állapotát állítja. A lineáris rétegnél nincs hatása, de a `Dropout` és a `BatchNorm` rétegeknél igen: tanítás közben a dropout véletlenszerűen nulláz, kiértékeléskor viszont identitásként viselkedik; a BatchNorm tanítás közben a batch statisztikákat, kiértékeléskor a futó statisztikákat használja. Ezt a későbbi ellenpélda mutatja.

28.2.3 Teljes minimális munkapélda

A példa CPU-n fut. A `torch` és `scikit-learn` csomagot a fejezet bővítő környezetébe kell telepíteni; a 3.1. alfejezetben bemutatott projektmódszerrel külön környezetet érdemes használni. Az alábbi kódot `train_loop.py` néven mentve, a munkakönyvtárból a `python train_loop.py` paranccsal futtathatjuk.

```
import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Szintetikus adat
rng = np.random.default_rng(42)
n = 1200
X = rng.normal(0.0, 1.0, size=(n, 2))
y = 1.8 * X[:, 0] - 0.7 * X[:, 1] + 0.3 + rng.normal(0.0, 0.5, size=n)

# 2. Azonos split mindkét modellhez
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Dataset és DataLoader
class RegressionDataset(Dataset):
    def __init__(self, X_data, y_data):
        self.X = torch.tensor(X_data, dtype=torch.float32)
        self.y = torch.tensor(y_data, dtype=torch.float32).unsqueeze(1)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
```

```
        return self.X[idx], self.y[idx]

train_ds = RegressionDataset(X_train, y_train)
test_ds = RegressionDataset(X_test, y_test)

# A DataLoader reprodukálható keveréséhez saját generátort adunk
torch_gen = torch.Generator().manual_seed(0)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, generator=torch_gen)

# 4. Modell, veszteség, optimalizáló
torch.manual_seed(0)
model = nn.Linear(2, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# 5. Tanítási ciklus
n_epochs = 40
for epoch in range(n_epochs):
    model.train()
    epoch_loss = 0.0
    for Xb, yb in train_loader:
        optimizer.zero_grad()
        pred = model(Xb)
        loss = loss_fn(pred, yb)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(yb)
    epoch_loss /= len(train_ds)

# 6. Kiértékelés eval módban, gradiensgráf nélkül
model.eval()
with torch.no_grad():
    pred_train = model(train_ds.X)
    pred_test = model(test_ds.X)
    pt_train_rmse = torch.sqrt(loss_fn(pred_train, train_ds.y)).item()
    pt_test_rmse = torch.sqrt(loss_fn(pred_test, test_ds.y)).item()

# 7. sklearn baseline ugyanazon a spliten
lr = LinearRegression().fit(X_train, y_train)
sk_test_rmse = float(np.sqrt(mean_squared_error(y_test, lr.predict(X_test))))

print(f"PyTorch train RMSE: {pt_train_rmse:.3f}")
print(f"PyTorch test  RMSE: {pt_test_rmse:.3f}")
print(f"sklearn test  RMSE: {sk_test_rmse:.3f}")
```

A TensorBoard-naplózás opcionális kiegészítő. Az `nn.MSELoss` átlagos négyzetes hibát ad; az RMSE ennek gyöke, ezért az epoch átlagveszteségéből a gyököt vonva kapjuk a train RMSE-t. A `torch.no_grad()` blokk alatt a PyTorch nem építi a számítási gráfot, így a kiértékelés gyorsabb és nem foglal fölösleges memóriát. A kimenet a PyTorch 2.15.0+cpu, NumPy 2.5.4 és scikit-learn 1.9.1 verziókkal, a fenti seedekkel mért eredmény:

```
PyTorch train RMSE: 0.493
PyTorch test  RMSE: 0.506
sklearn test  RMSE: 0.506
```

A TensorBoardhoz tartozó minimális vázlat:

```
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter("runs/linear_baseline")
# a tanítási ciklusban, minden epoch után:
writer.add_scalar("loss/train_epoch", epoch_loss, epoch)
writer.close()
```

A görbe megtekintéséhez külön terminálablakban, a projektgyökérből: `tensorboard --logdir runs`. A pontos telepítési és indítási lépések a helyi Python-környezettől függenek; a `tensorboard` csomagot a futtató környezetbe külön kell telepíteni.

28.2.4 Eredmény értelmezése

A mért RMSE-értékek azt mutatják, hogy a PyTorch lineáris réteg és a `sklearn` lineáris regresszió ugyanazon a spliten közel azonos hibát ad. Ez várható, mert mindkettő ugyanazt a lineáris hipotézist és négyzetes veszteséget használja; a `sklearn` zárt alakú legkisebb négyzetek megoldást számol, a PyTorch pedig iteratív gradienslépésekkel közelíti ugyanazt a minimumot. A teszt RMSE 0.5 körüli értéke a generáló zaj szórásának felel meg: ezt a hibát akkor sem tudjuk eltüntetni, ha a súlyokat pontosan visszanyerjük, mert a cél magában hordozza a $\sigma = 0.5$ szórású zajt.

A közel egyezés azonban nem igazolja, hogy a tanítási ciklus minden eleme helyes; ha például a teszt sorok beszivárognának a tanításba, a hiba akár ennél is alacsonyabbnak tűnhetne. A regressziós súly értéke itt nem oksági következtetés: a 16.2. alfejezet tanulsága szerint a becsült együttható nem bizonyítja, hogy a feature okozza a cél változását.

A `model.train()` és `model.eval()` eltérő viselkedést okozhat dropout és batch normalization mellett; validáció során használjunk `torch.no_grad()` vagy `torch.inference_mode()` kontextust.

28.2.4.1 Invariánsok és tesztjeik

A tanítási ciklus helyességét nem elég a végső RMSE-ből megítélni; négy kis invariáns közvetlenül tesztelhető.

1. A módváltás rekurzív. A `model.train()` és `model.eval()` a modell minden almoduljára beállítja a `training` kapcsolót:

```
model = nn.Sequential(nn.Linear(2, 4), nn.Dropout(0.5), nn.BatchNorm1d(4))
model.train()
assert all(m.training for m in model.modules())
model.eval()
assert not any(m.training for m in model.modules())
```

2. Az `eval()` nem kapcsolja ki a gradiensgráfot. A két kapcsoló külön célú: az `eval()` a dropout/BatchNorm viselkedését változtatja, a `torch.no_grad()` a gráfépítést. Ellenőrzés: eval módban, de `no_grad` nélkül a kimenet `requires_grad` értéke továbbra is `True`; a `no_grad()` blokkban `False`, és a `grad_fn` `None`.

3. A `zero_grad()` nullázza a felhalmozott gradienst. Egy backward után a gradiens nem nulla; `optimizer.zero_grad()` után nulla (vagy `None`). Nullázás nélkül a második `backward()` ugyanazt az értéket hozzáadja, tehát a gradiens megduplázódik. A teszt ugyanazon a batchen egyszer és kétszer hívja a `backward()`-ot, és összeveti a `p.grad` értékeket.

4. Az epoch-veszteség mintasúlyozott. Ha a veszteség batch-enkénti átlag (`reduction="mean"`), akkor a helyes epoch-veszteség

$$L_{\text{epoch}} = \frac{1}{N} \sum_{b=1}^{K} L_b \, |B_b|,$$

ahol N az epoch összes mintaszáma, L_b a b-edik batch átlagos vesztesége, $|B_b|$ pedig a batch mérete. A batch-veszteségek egyszerű átlaga akkor hibás, ha az utolsó batch kisebb. Számpélda: $y = (0, 2, 4)$, predikció $p = (1, 0, 4)$, batch-méretek 2 és 1. Az első batch MSE-je 2.5, a másodiké 0; a mintasúlyozott epoch-loss $(2.5 \cdot 2 + 0 \cdot 1)/3 = 1.667$, ami egyezik a teljes adathalmaz MSE-jével, $(1 + 4 + 0)/3 = 1.667$. A súlyozatlan batch-átlag $(2.5 + 0)/2 = 1.25$, tehát 25%-kal alábecsül. A tanítási ciklusban ezért szerepel `loss.item() * len(yb)`, majd az osztás `len(train_ds)`-sel.

Az invariánsokat a `tests/test_train_loop_invariants.py` teszteli: a módváltás rekurzivitását, az `eval()` és `no_grad()` szétválasztását, a gradiensfelhalmozódást és a mintasúlyozott epoch-loss egyezését a teljes adathalmaz MSE-jével.

28.2.5 Feladatok

1. **Értelmezés.** Egy tanítási ciklus két batchből áll, a tanulási sebesség $\eta = 1$. Az első batch gradiense $g_1 = 0.6$, a másodiké $g_2 = -0.2$. Számold ki a paraméter teljes változását (a) helyes `zero_grad` hívással, (b) a `zero_grad` elhagyásával. Mi a hiba tartalmi jelentése?

2. **Implementálás.** Írj `eval_rmse(model, loader, loss_fn)` függvényt, amely `model.eval()` módban, `torch.no_grad()` alatt kiszámolja egy DataLoader adatainak RMSE-jét. A függvény ne építsen gradiensgráfot. Figyelj arra, hogy az utolsó batch mérete kisebb is lehet.

3. **Hibadiagnózis.** Egy tanuló a következőt írta:

```
X_all = torch.tensor(X, dtype=torch.float32)
y_all = torch.tensor(y, dtype=torch.float32).unsqueeze(1)
train_ds = TensorDataset(X_all, y_all)  # az egész X, nem X_train
# ...
sk_model = LinearRegression().fit(X_train, y_train)
```

Nevezd meg azt a két problémát, amely az összehasonlítást érvénytelenné teszi, és add meg a javítást.

4. **Megváltozott feltétel.** Futtasd a példát `batch_size=1`, `batch_size=32` és `batch_size=512` értékekkel, azonos `lr=0.01`, `n_epochs=40` mellett. Rögzítsd az utolsó epoch train RMSE-jét és a teszt RMSE-t. Melyik beállításnál várható a legzajosabb tanítási görbe, és miért nem garantálja a legnagyobb batch a legjobb teszt RMSE-t? A válaszban nevezd meg a rögzítendő feltételeket is.

28.2.6 Megoldások és indoklás

1. Helyes `zero_grad` esetén az első frissítés -0.6, a második $-(-0.2) = +0.2$, összesen -0.4. Elhagyás esetén az első lépés után is -0.6, de a második batch előtt a gradiens nem nullázódik, így a `.grad` értéke $0.6 - 0.2 = 0.4$ lesz, és a második frissítés -0.4. Az összes változás -1.0 a helyes -0.4 helyett. A hiba az, hogy a frissítés nem az aktuális batch veszteségének gradiensét, hanem a felhalmozott gradienst használja; a modell rosszabb lépést tesz, mintha két batch együttes, de nem skálázott veszteségét minimalizálná.

2. Példa megoldás:

```
def eval_rmse(model, loader, loss_fn):
    model.eval()
    total_sq = 0.0
    n = 0
    with torch.no_grad():
```

```
        for Xb, yb in loader:
            pred = model(Xb)
            total_sq += loss_fn(pred, yb).item() * len(yb)
            n += len(yb)
    return (total_sq / n) ** 0.5
```

Az invariáns: a `loss_fn(pred, yb)` a batch átlagos MSE-je, ezért a `len(yb)`-vel való szorzás adja a batch négyzetes hibák összegét; a teljes RMSE ennek a teljes elemszámra vett átlagából számított gyök. Tolerancia: a függvénynek a tanításhoz használt teljes epoch-veszteség számításával azonos eredményt kell adnia ugyanazon adat és modell esetén. A hívás után a tanítás folytatásához `model.train()` szükséges.

3. Az első probléma a tesztadat beszivárgása: a `train_ds` az egész `X`-et tartalmazza, tehát a PyTorch modell olyan sorokon is tanul, amelyek a tesztben szerepelnek. Ez optimista torzítást okoz, és sérti a 21.1. alfejezetben megfogalmazott érintetlen teszt elvét. A második probléma az összehasonlítás ellentmondásossága: a `sklearn` modell `X_train`-ből tanul, a PyTorch modell `X`-ből, így a két modell nem ugyanazon a tanítóhalmazon értékelhető. Javítás: mindkét modell kizárólag `X_train`-ből tanuljon, a tesztelés pedig kizárólag `X_test`-ből; a `TensorDataset`-et a `train_test_split` eredményéből kell felépíteni.

4. Azonos adatgeneráló folyamat és szokásos mintavételi feltételek mellett a `batch_size=1` tipikusan nagyobb varianciájú gradiensbecslést ad, ezért a tanítási görbe erősen ugrál, és az utolsó epoch train RMSE-je kevésbé informatív. A `batch_size=512` simább gradienslépéseket ad, de a körülbelül 960 soros tanítóhalmazon egy epoch alatt csak egy vagy két frissítés történik; 40 epoch alatt ez lényegesen kevesebb lépés, mint a 32-es batchnél. Ezért előfordulhat, hogy a legnagyobb batch nem éri el ugyanazt a megoldást. Az összehasonlításhoz rögzíteni kell a seedet, az epochok számát vagy a frissítések számát, a tanulási sebességet, a hardvert és a könyvtárverziókat; az egyetlen futásból származó RMSE-különbség nem önmagában bizonyíték.

28.2.7 Gyakori tévedés ellenpéldával

A `model.eval()` elmaradása akkor okoz feltűnő hibát, ha a modell tartalmaz dropout vagy BatchNorm réteget. Példaként egy extrém, didaktikus dropout:

```
class DropoutModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.drop = nn.Dropout(p=1.0)
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(self.drop(x))
```

Tanítási módban a `p=1.0` dropout minden bemeneti elemet nulláz, ezért a modell csak a bias-t tanulja; kiértékelési módban a dropout kikapcsol, és a bemenet átjut. Ha a tesztelésnél elmarad az `eval()`, a tesztpredikciók a tanítási viselkedést követik, és az RMSE **eltérhet** a helyes értéktől. Az eltérés iránya nem általánosítható: a dropout miatt egyes mintákon romolhat, másokon akár javulhat is a mérőszám. A szokásos $p = 0.1$–0.5 tartományban a hatás kevésbé feltűnő, de az értékelési protokoll így is hibás. A lineáris példánkban nincs dropout, így a hiba ott nem jelentkezett volna; a train/eval szokást mégis minden modellnél be kell tartani.

Önellenőrzés: a `torch.no_grad()` nem helyettesíti az `eval()` hívást. A `no_grad` csak a gradiensgráf építését kapcsolja ki, de a dropout és BatchNorm train-viselkedése a modul `training` állapotától függ. A kettő külön célt szolgál, és teszteléskor általában együtt van rá szükség.

28.2.8 Rövid összefoglaló

A tanítási ciklus egy epoch során DataLoader-batch-eken iterál, minden batch előtt nullázza a gradienst, kiszámítja a veszteséget, a `backward` hívással gradienst számol, végül az optimizer frissíti a paramétereket. A batch-gradiens a teljes adat gradiensének zajos becslése; a batch méret a gradiens zaját és a lépésszámot is befolyásolja. A kiértékelést eval módban és gradiensgráf nélkül végezzük, mert a Dropout és BatchNorm viselkedése eltér tanítás közben. A lineáris réteg ugyanazon a spliten közel a `sklearn` lineáris baseline hibáját éri el, ami az optimalizálás és az invariáns ellenőrzésére használható. A következő lépés a generalizáció és a gyorsító választás felé vezet.

28.3 Generalizáció és gyorsító választás

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyTorch

Előfeltétel: 28.2. alfejezet · Célkészség: Generalizáció és gyorsító választás · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

28.3.1 Cél és mérési helyzet

A 28.2. alfejezet tanítási ciklusában a kérdés az volt, hogyan számítunk ki egy gradienslépést. Most a teljes adat- és számítási út költségét mérjük, mielőtt gyorsítóra váltanánk. A döntési helyzet: a Northstar prediktív feladatnál 2400 rendelés és 8 rendeléskori feature áll rendelkezésre, a cél a 7 napot meghaladó kézbesítés valószínűségének becslése. A 2400×8-as fp32 mátrix nagyjából $2400 \cdot 8 \cdot 4 = 76\,800$ byte, tehát nem a méret vagy a memória a szűk keresztmetszet, hanem a betöltés, a Python-keret overheadje és a fejlesztési idő.

Az előzetes becslésem ezért az, hogy ehhez a Northstar-mérethez a CPU is elegendő, és a GPU csak akkor éri meg, ha a tényleges mérés szerint a számítás vagy az adatmozgatás a domináns költség. Ahhoz, hogy a batch-, memória- és transferhatásokat értelmezni tudjuk, egy nagyobb, de helyben legyártható szintetikus mátrixon mérünk; a mért számok a saját gépedre vonatkoznak, nem a könyv előregyártott eredményére.

28.3.2 A költség összetevői: batch, transfer, VRAM

A **batching** (kötegelés) a `DataLoader` egyik alapbeállítása: egyszerre B mintát ad át a modellnek. A minibatch gradiens

$$g_B = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta L(x_i, y_i; \theta),$$

ahol θ a modell paramétere, L a veszteség, (x_i, y_i) pedig a B méretű batch i-edik mintája. Ha a gradiensek független, azonos eloszlású mintánkénti becslések, akkor a variancia az $1/B$ szabályt követi:

$$\mathrm{Var}(g_B) = \frac{\mathrm{Var}(g_1)}{B}.$$

Nagyobb batch tehát kisebb gradienstorzítást, de több memóriát és kevesebb paraméterfrissítést jelent epochonként; az optimális B mérési kérdés.

A **transfer** a CPU- és a GPU-memória közötti másolás. A PyTorch `tensor.to(device)` hívása a háttérben PCIe/átviteli sávszélességet használ. CPU-n nincs külön devicetransfer, de a CSV/Parquet betöltés, a pandas átalakítás és a tensorré alakítás ugyanúgy a teljes út része. GPU-mérésnél a transfert külön kell mérni: `torch.cuda.synchronize()` után `time.perf_counter()`, mert a GPU-műveletek aszinkronok lehetnek.

A **VRAM** a GPU saját memóriája. Három összetevője van: a paraméterek, a gradiensek és az optimizer állapota. Adamnél minden fp32 paraméterhez tartozik gradiens, első momentum és második momentum, tehát a paraméterbyte-ok négyszerese kell. Az aktivációk ezen felül batchfüggő memóriát foglalnak. A `torch.cuda.memory_allocated()` csak GPU-n hívható; CPU-s becsléshez kézzel számolunk, ahogy a következő szakasz mutatja.

A **mixed precision** (vegyes pontosság) áttekintő szinten annyit jelent, hogy a számítási részek fp16/bf16-ban futnak, a súlyok viszont fp32 master másolatban maradnak. Ezzel csökken a memória- és sávszélességigény, de az fp16 kis tartománya miatt a gradiens alulcsordulhat. A `torch.amp.GradScaler` ezért a veszteséget felskálázza a backward előtt, majd a súlyfrissítésnél visszaoszt. CPU-n ez általában nem hoz automatikus előnyt; a használat előtt méréssel kell igazolni az előnyt.

Az **early stopping** (korai leállás) a validációs veszteséget nézi: ha `patience` epochon át nincs `min_delta`-nál nagyobb javulás, a tanítást leállítjuk, és a legjobb mentett súlyt töltjük vissza. A validációs halmaz itt modellválasztási eszköz, a teszthalmazt csak a leállás után értékelhetjük; ellenkező esetben a teszt vezérli a modellt.

A **seed limits** (magkorlátok) azt rögzítik, hogy a pszeudovéletlen sorozat ugyanazon gépen, verzióval és csomagkészlettel ismételhető legyen. A seed nem garantál platformok közötti bitazonosságot: a GPU-k atomi redukciói, a könyvtárverziók és a párhuzamos futtatás eltérést okozhatnak. Ezért különböztetjük meg a numerikus, statisztikai és környezeti reprodukálhatóságot.

A **TensorBoard** áttekintésben a `SummaryWriter` naplózza a veszteség-, idő- és memóriagörbéket; a `tensorboard --logdir runs` parancs böngészőben jeleníti meg őket. A görbe munkanapló, nem helyettesíti a mért hardver-, méret- és ismétlésadatokat tartalmazó jegyzőkönyvet.

28.3.3 Műveletszám, memória és variancia

A bemutatott modell: 32 bemenet, 48 rejtett neuron, 1 kimenet. A paraméterszám

$$P = 32 \cdot 48 + 48 + 48 \cdot 1 + 1 = 1633.$$

Egy lineáris réteg forward műveletszáma $2 \cdot B \cdot d_{\text{in}} \cdot d_{\text{out}}$ FLOP, mert egy szorzás-összeadás pár két lebegőpontos művelet. A két rétegre:

$$C_{\text{fwd}} = 2B(32 \cdot 48 + 48 \cdot 1) = 3168B$$

FLOP. A backward durván a forward kétszerese, így egy batch teljes költsége körülbelül $3 \cdot 3168B$. Például $B = 512$ esetén egy batch körülbelül 4,9 millió FLOP, a $8192/512 = 16$ batchből álló epoch pedig körülbelül 78 millió FLOP. Ez a műveletmennyiség egy modern CPU-n is néhány tizedmásodperces nagyságrendű lehet; a mérés dönti el, hogy a Python-overhead vagy a számítás dominál-e.

A memóriabecslés ugyanilyen egyszerű. A 1633 paraméter fp32-ben $1633 \cdot 4 \approx 6{,}5$ KiB; Adamnél a paraméter, gradiens, momentum és variancia együtt körülbelül 26 KiB. A $B = 512$ batch aktivációi: bemenet $512 \cdot 32 \cdot 4 = 65\,536$ byte, rejtett kimenet $512 \cdot 48 \cdot 4 = 98\,304$ byte, logit $512 \cdot 1 \cdot 4 = 2048$ byte; a backward hasonló nagyságrendű, összesen fél megabyte alatt maradunk. Ezért ez a modell VRAM szempontból soha nem indokol GPU-t; a GPU-kérdést a műveletmennyiség, a transfer és a betöltés dönti el.

A gradiensvariancia-összefüggés visszautal a 16.3. alfejezet becslési varianciájára: a batchméret növelése ugyanazt a zajcsökkentést adja, mint a mintanagyság növelése, de csak független minták esetén érvényes ilyen tisztán. A DataLoader shuffle után hozzávetőleg függetlennek tekinthető, de a modell frissítése közben az eloszlás már nem azonos; ezért a képlet az egy paraméterpont körüli lokális közelítés, nem globális ígéret.

28.3.4 Minimális munkapélda: CPU-költségmérés

A következő script a fenti modell teljes epochonkénti költségét méri CPU-n, különböző batchméretekkel. Az outputot a tanuló gépén kell létrehozni; az itt látható JSON-struktúra a várt séma, nem ellenőrzött mérési jegyzék. A kód a projektgyökérben futtatható.

```
import json
import time
from pathlib import Path

import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

SEED = 20261020
torch.manual_seed(SEED)

def make_data(n=8192, d=32):
    X = torch.randn(n, d)
    w = torch.randn(d, 1)
    y = ((X @ w + 0.2 * torch.randn(n, 1)) > 0).float()
    return X, y

def build_model(d):
    return nn.Sequential(
        nn.Linear(d, 48),
        nn.ReLU(),
        nn.Linear(48, 1),
    )

X, y = make_data()
dataset = TensorDataset(X, y)
results = []

for batch_size in [32, 64, 128, 256, 512]:
    # minden batchmérethez azonos kezdősúly
    torch.manual_seed(SEED)
    model = build_model(X.shape[1])
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    loss_fn = nn.BCEWithLogitsLoss()
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    # egy batch melegítés, hogy az első allokáció ne torzítson
    for xb, yb in loader:
        optimizer.zero_grad()
        loss_fn(model(xb), yb).backward()
```

```
        optimizer.step()
        break

    n = 0
    t0 = time.perf_counter()
    for xb, yb in loader:
        optimizer.zero_grad()
        out = model(xb)
        loss = loss_fn(out, yb)
        loss.backward()
        optimizer.step()
        n += xb.shape[0]
    dt = time.perf_counter() - t0

    results.append({
        "batch_size": batch_size,
        "samples": n,
        "epoch_s": round(dt, 4),
        "samples_per_s": round(n / dt, 2) if dt > 0 else None,
    })

path = Path("s111_cpu_results.json")
path.write_text(json.dumps(results, indent=2) + "\n")
print(json.dumps(results, indent=2))
```

Futtatás Linux/macOS terminálból a projektgyökérből:

```
python s111_cpu_measure.py
```

Windows PowerShellből ugyanez a `python s111_cpu_measure.py` parancs. A TensorBoard-napló opcionálisan a mérés köré építhető:

```
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter("runs/s111_cpu_measure")
for row in results:
    writer.add_scalar("time/epoch_s", row["epoch_s"], row["batch_size"])
    writer.add_scalar("throughput/samples_per_s", row["samples_per_s"], row["batch_size"])
writer.close()
```

A `tensorboard --logdir runs` parancs a projektgyökérből indítva nyitja meg a görbéket; a pontos elérhető verzió a telepített csomagtól függ, ezért a saját környezetedben ellenőrizd.

28.3.5 Eredmény értelmezése és GPU-döntési lap

Az eredményeket a saját gépeden mérd, legalább három ismétléssel, és a mediánt jegyezd fel. Minden futásra két invariantet ellenőrizz: `samples` értéke mindig 8192, és `samples_per_s` értéke `samples / epoch_s` kerekítve. A tipikus CPU-mintázat az, hogy a kisebb batch nagyobb Python- és kernelindítási overheaddel jár, ezért a nagyobb batch növeli az átviteli sebességet addig, amíg a cache vagy a háttérterhelés újabb korlátot nem szab. A pontos szám géptől függ; csak a saját futtatásodat nevezd mértnek.

Mutató	Jelentés	Ellenőrzés
batch_size	minibatch méret	input
samples	feldolgozott sorok	mindig 8192
epoch_s	epoch idő	tanulói mérés
samples_per_s	átviteli sebesség	samples / epoch_s

A GPU-döntési lap akkor használható, ha CUDA elérhető; enélkül a terv elméleti marad, és tilos gyorsulást közölni. A lap lépései:

1. Rögzítsd a CPU-baseline-t: adatbetöltés, tensor-átalakítás, train, eval, ismétlésszám.
2. Ellenőrizd a GPU-környezetet: `torch.cuda.is_available()`, `torch.version.cuda`, GPU neve.
3. Mérd a transfert külön: `x.to("cuda"); torch.cuda.synchronize(); dt`.
4. Futtasd ugyanazt a modellt GPU-n, azonos batch és ismétlés mellett.
5. Hasonlítsd a teljes utat, nem csak a mátrixszorzást; a betöltés és a transfer is beleszámít.
6. Adj memóriafelső korlátot: Adam fp32 esetén $4P \cdot 4$ byte paraméter, plusz a batchfüggő aktivációk.

A mixed precision használatakor a veszteségfelskálázás kötelező GPU-n:

```
use_cuda = torch.cuda.is_available()
device = torch.device("cuda" if use_cuda else "cpu")
model.to(device)
scaler = torch.amp.GradScaler("cuda", enabled=use_cuda) if use_cuda else None

optimizer.zero_grad()
if scaler is not None:
    with torch.amp.autocast("cuda", dtype=torch.float16, enabled=True):
        out = model(xb.to(device))
        loss = loss_fn(out, yb.to(device))
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
else:
    loss = loss_fn(model(xb), yb)
    loss.backward()
    optimizer.step()
```

A `torch.amp` API a PyTorch 2.x felület; a régi `torch.cuda.amp` elavult, ezért a saját verziód dokumentációját ellenőrizd. Az early stopping vázlata ugyanebben a környezetben:

```
best_loss = float("inf")
patience = 5
min_delta = 1e-4
no_improve = 0

for epoch in range(max_epochs):
    train_loss = train_one_epoch(model, train_loader, device)
    val_loss = evaluate(model, val_loader, device)
    writer.add_scalar("loss/train", train_loss, epoch)
    writer.add_scalar("loss/val", val_loss, epoch)

    if val_loss < best_loss - min_delta:
        best_loss = val_loss
        no_improve = 0
        torch.save(model.state_dict(), "best_model.pt")
    else:
        no_improve += 1
        if no_improve >= patience:
            break
```

A `train_one_epoch` és `evaluate` a 28.2. alfejezetből ismert `train`/`eval` módot és `torch.no_grad()`-ot használja; a `max_epochs` tetszőleges felső korlát, a `patience` a validációs javulás türelmi ideje. A végső teszthalmazt csak a `best_model.pt` betöltése után szabad egyszer kiértékelni.

CPU–GPU gyorsításnál a teljes mérési szakaszt és adatmozgatást is vegyük figyelembe; aszinkron GPU-futtatás időzítésénél szinkronizálás szükséges.

28.3.5.1 Környezeti ujjlenyomat, determinizmus és toleranciás teszt

A seed csak a véletlenszám-generátorok állapotát rögzíti; a bitazonos kimenethez a teljes környezetnek is egyeznie kell. Ezért minden mérési jegyzőkönyvhöz tartozzon **környezeti ujjlenyomat**:

```
import platform
import torch

def environment_fingerprint():
    return {
        "python": platform.python_version(),
        "torch": torch.__version__,
        "cuda_available": torch.cuda.is_available(),
        "cuda_version": torch.version.cuda,
        "cudnn_version": (torch.backends.cudnn.version()
                          if torch.cuda.is_available() else None),
        "device_name": (torch.cuda.get_device_name(0)
                        if torch.cuda.is_available() else platform.processor()),
        "deterministic_algorithms": torch.are_deterministic_algorithms_enabled(),
        "cudnn_deterministic": torch.backends.cudnn.deterministic,
        "cudnn_benchmark": torch.backends.cudnn.benchmark,
        "float32_matmul_precision": torch.get_float32_matmul_precision(),
        "threads": torch.get_num_threads(),
    }
```

A szerzői CPU-környezetben mért példa: `python 3.12.3`, `torch 2.15.0+cpu`, `cuda_available false`, `device_name "x86_64"`, `deterministic_algorithms false`, `float32_matmul_precision "highest"`, `threads 10`. A `float32_matmul_precision` és a szálak száma is számít, mert a lebegőpontos összegzés nem asszociatív: más szálszámmal vagy más BLAS-könyvtárral a redukciók sorrendje eltérhet.

Mit garantál a seed? Ugyanazon a gépen, ugyanazzal a PyTorch-verzióval, ugyanazzal a szálszámmal és ugyanazon a device-on a rögzített seeddel indított két futás jellemzően bitazonos; ezt a szerzői CPU-környezetben két, öt lépéses tanítási futás `state_dict`-jeinek összehasonlítása igazolta (`torch.equal` minden paraméterre `True`, maximális abszolút eltérés `0.0`). Ez **nem** általános garancia. GPU-n a redukciók atomi műveletei, a cuDNN kiválasztott algoritmusa és a hardver eltérő sorrendje miatt a seed ellenére is eltérhet a kimenet; más PyTorch-verzió, más BLAS vagy más szálszám szintén eltérést okozhat. A seed tehát a *numerikus* ismételhetőség egyik szükséges, de nem elégséges feltétele.

Determinisztikus módot CUDA-n a `torch.use_deterministic_algorithms(True)` és a `CUBLAS_WORKSPACE_CONFIG=:4096:8` környezeti változó kér; a `torch.backends.cudnn.benchmark = False` és `torch.backends.cudnn.deterministic = True` a konvolúciós algoritmusválasztást rögzíti. Ezek egy része hibát dob olyan műveletnél, amelynek nincs determinisztikus implementációja; ilyenkor a kódot kell módosítani, nem a kapcsolót elrejteni. A determinisztikus mód sem garantál **két különböző device** közötti bitazonosságot.

Ezért a numerikus regressziós teszt toleranciás, nem bitazonos:

```
# Ugyanazon a device-on, ugyanazzal a környezettel: szigorú, de nem bitazonos.
torch.testing.assert_close(run_a, run_b, rtol=1e-12, atol=1e-12)

# float32 tanításnál reálisabb:
torch.testing.assert_close(run_a, run_b, rtol=1e-5, atol=1e-6)
```

```
# Két környezet között csak összesített metrikát hasonlíts,
# előre rögzített toleranciával, és a tényleges eltérést is jelenteni kell.
```

A „két környezet azonos seeddel ugyanazt a bináris kimenetet adja” állítás tehát nem szerepelhet a jegyzőkönyvben. A helyes állítás: azonos környezetben a futás ismételhető; eltérő környezetben a metrika a rögzített toleráncián belül egyezik, és az eltérést a környezeti ujjlenyomattal együtt jelentjük. A `tests/test_determinism_contract.py` ezt a szétválasztást teszteli: azonos seeddel indított két CPU-futás bitazonos, a mesterségesen perturbált tenzor a szigorú toleráncián elbukik, és a környezeti ujjlenyomat tartalmazza a szükséges mezőket.

28.3.6 Feladatok

28.3.6.1 1. feladat (értelmezés)

Számítsd ki a bemutatott modellre: P paraméterszámot, egy $B = 128$ batch lineáris forward FLOP-számát, a forward+backward közelítő FLOP-számát, továbbá a $B = 128$ és $B = 512$ minibatch gradiens varianciájának arányát független, azonos eloszlású gradiensek feltételével.

28.3.6.2 2. feladat (implementálás)

Módosítsd a `s111_cpu_measure.py` scriptet úgy, hogy minden batchmérethez három ismétlésben futtassa a mérést, a kimenetbe írja be a három `epoch_s` értéket és a mediánt, és mentse el `s111_cpu_results.json` néven. Futtasd a saját gépeden, és ellenőrizd, hogy minden `samples` mező 8192.

28.3.6.3 3. feladat (hibadiagnózis)

Valaki a mixed precision kódot így írta, és a tanítási veszteség néhány lépés után `nan` lett:

```
with torch.amp.autocast("cuda", dtype=torch.float16):
    out = model(xb.to(device))
    loss = loss_fn(out, yb.to(device))
loss.backward()
optimizer.step()
```

Magyarázd meg, mi a hiba, és add meg a javítást. Adj egy második lehetséges okot is arra, ha CPU-mérésben a $B = 512$ epoch lassabb, mint a $B = 256$.

28.3.6.4 4. feladat (megváltozott feltétel)

Tegyük fel, hogy $n = 50\,000\,000$ sor, $d = 256$ feature és $B = 512$ batch adott. A modell legyen $256 \to 128 \to 1$. Készíts GPU-s becslési tervet: számítsd ki az input tenzor byte-méretét, a modell paraméterszámát, az Adam fp32 memóriafelső korlátját, a $B = 512$ batch aktivációinak közelítő méretét, és egy átviteli időbecslést 8 GB/s effektív sávszélességgel. Mit kell ténylegesen megmérned, mielőtt gyorsulást állítanál?

28.3.7 Megoldások és indoklás

28.3.7.1 1. megoldás

$P = 32 \cdot 48 + 48 + 48 \cdot 1 + 1 = 1633$. A $B = 128$ forward lineáris költsége $2 \cdot 128 \cdot (32 \cdot 48 + 48) = 2 \cdot 128 \cdot 1584 = 405\,504$ FLOP. A forward+backward közelítőleg ennek háromszorosa: $1\,216\,512$ FLOP. A varianciaarány:

$$\frac{\mathrm{Var}(g_{512})}{\mathrm{Var}(g_{128})} = \frac{\mathrm{Var}(g_1)/512}{\mathrm{Var}(g_1)/128} = \frac{128}{512} = \frac{1}{4}.$$

A szórásarány ennek négyzetgyöke, $1/2$; a nagyobb batch tehát felére csökkenti a gradienszaj szórását ebben az idealizált közelítésben.

28.3.7.2 2. megoldás

A lényeg a mérési hurok megkettőzése: minden batchmérethez futtass egy belső ciklust `[0, 1, 2]` ismétléssel, gyűjtsd listába a `dt` értékeket, és a mediánt a `statistics.median` függvénnyel számold. Az újraindított `torch.manual_seed(SEED)` maradjon a külső ciklus elején, hogy azonos kezdősúlyokkal induljon minden batchméret. A kimenet egy lehetséges bővített rekordja:

```
{
  "batch_size": 128,
  "samples": 8192,
  "epoch_s_runs": [0.0411, 0.0398, 0.0403],
  "epoch_s_median": 0.0403,
  "samples_per_s": 203323.1
}
```

Itt az `epoch_s_runs` a három mért idő, a `samples_per_s` a mediánból számított 8192/0.0403. Az `samples` invariant minden sorban 8192; ha nem, akkor a DataLoader vagy a batchméretezés hibás. A fenti konkrét szám csak szemléltető, nem mért; a saját gép és a háttérterhelés eltérő értéket adhat.

28.3.7.3 3. megoldás

A `nan` oka a GradScaler hiánya: fp16 autocast alatt a kis gradiensek alulcsordulhatnak, a backward így nem véges értéket ad. A javítás:

```
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

Ha a scaler nem elérhető, maradjon fp32 CPU-n, vagy kapcsold ki az autocastot. A második lehetséges ok, hogy a $B = 512$ lassabb CPU-n: a nagyobb batch kinőhet a cache-ből, a gép háttérfolyamatot futtat, vagy egyetlen mérésből vontál le következtetést. A helyes válasz mérési terv: legalább öt ismétlés, melegítés, a medián használata, és a háttérterhelés rögzítése a jegyzőkönyvben.

28.3.7.4 4. megoldás

Az input tenzor mérete $50\,000\,000 \cdot 256 \cdot 4 = 51{,}2$ GB, tehát nem fér be egyetlen batchbe; a betöltés így streaminget igényel. A modell paraméterszáma $P = 256 \cdot 128 + 128 + 128 \cdot 1 + 1 = 33\,025$. Az Adam fp32 felső korlát $4P \cdot 4 = 528\,400$ byte ≈ 516 KiB. A $B = 512$ batch aktivációi: bemenet $512 \cdot 256 \cdot 4 = 524\,288$ byte, rejtett kimenet $512 \cdot 128 \cdot 4 = 262\,144$ byte, logit 2048 byte, backward hasonló; a teljes aktiváció 2 MB alatt marad. A 8 GB/s effektív sávszélességgel számolt átvitel 51,2 GB/8 GB/s = 6,4 s. Ez elméleti becslés; a valós sávszélesség hardverfüggő. Mielőtt gyorsulást állítanál, mérd meg a teljes utat legalább háromszor: betöltés, CPU→GPU transfer, train, eval; rögzítsd a GPU nevét, verzióját és a PCIe-környezetet. A 6,4 s-on felül a betöltés gyakran dominánsabb, ezért a GPU-váltás csak akkor indokolt, ha a teljes mért futásidő és a projektcél is igazolja.

28.3.8 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés, hogy a GPU-t automatikusan gyorsabbnak tekintjük. A Northstar 2400×8-as mátrixa 76,8 KB; a bemutatott rétegekkel a teljes epoch körülbelül 78 millió FLOP alatt marad. Ez a műveletmennyiség a kernelindítás, a driver ütemezés és az adatmozgatás tartományába esik: a GPU-s overhead nagyobb lehet, mint a CPU-s teljes számítás, ezért a „modern GPU” látszólagos előnye itt veszteséggé válhat. A döntés sohasem eszköz, hanem a teljes adat- és számítási út mért költsége alapján születik.

A gradiensvariancia-összefüggés visszautal a 16.3. alfejezet mintanagysági becslésére: a batch növelése statisztikai zajt csökkent, de nem tesz csodát a ritka célváltozóval. A 27.3. alfejezet latency-keretére emlékeztet, hogy az LLM-értékelésben is a teljes késleltetés és költség döntött, nem a modell mérete önmagában. A 24.3. alfejezet pedig arra figyelmeztet, hogy a ritka esemény nem gyógyítható gyorsabb mátrixszorzással; a rangsor és az emberi validálás akkor is kell, ha a GPU hirtelen elérhetővé válik.

Önellenőrzés: fel tudod-e sorolni a teljes út lépéseit és mértékegységeit; ki tudod-e számolni az Adam-memória felső korlátját; meg tudod-e mondani, mit garantál a seed és mit nem; el tudod-e különíteni a betöltés, transfer, train és eval idejét a saját mérésedben? Ha bármelyik nem egyértelmű, futtasd újra a feladatok 2. és 4. mérését, mielőtt gyorsító mellett döntenél.

28.3.9 Rövid összefoglaló

A gyorsítóválasztás előtt a teljes adat- és számítási út mért költsége dönt: betöltés, tensorkonverzió, batch, transfer, forward/backward, validáció és kiértékelés együtt. A batchméret a gradiensvarianciát $1/B$-vel csökkenti, de a memóriát növeli. A VRAM-ot paraméter, gradiens, Adam-állapot és aktivációk adják; kis modellnél ez nem korlát. A mixed precision csökkentheti a memóriát, de GradScaler kell hozzá. Az early stopping a validációs görbét modellválasztásra használja, a seed pedig csak azonos környezetben ad ismételhetőséget. A következő lépés a VII. ellenőrző pont, ahol a PyTorch-kísérletnek a mérési jegyzőkönyvvel együtt kell megállnia a szivárgásmentes értékelés próbáját.

28.4 VII. ellenőrző pont: szakosodási döntés kontrollal

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: PyTorch

Előfeltétel: 28.3. alfejezet · Célkészség: VII. ellenőrző pont: szakosodási döntés kontrollal · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

28.4.1 Cél és konkrét helyzet

A VII. ellenőrző pont azt a pillanatot modellezi, amikor a data scientistnek nem algoritmust kell választania, hanem tudatosan döntenie kell a reprezentáció, az értelmezhetőség, a bizonytalanság, az értékelés, a számítási költség és az adathatár között. Három beérkező kérést vizsgálunk:

- **P1 – Rendelés-késés predikció (tabuláris).** A Northstar adatszerződésben megismert feladat: rendeléskor becsüljük, hogy a kézbesítés meghaladja-e a 7 napot. Bemenet a rendelés pillanatában ismert nyolc feature, megfigyelési egység a rendelés, cél a `target_late` bináris változó. A split a purged time-based cut: `order_ts < cutoff` és `label_available_at < cutoff` a tanítóhalmaz, `order_ts >= cutoff` a teszt. Előzetes becslés: a konstans modell log loss-ja a várható késési ráta függvényében nagyjából 0,50 körül van; egy értelmes logisztikus regresszió ennél jobbnak kell lennie, de a javulás mértékét csak a tanítás után tudjuk.
- **P2 – Support-jegy osztályozás (szöveges).** Kis szintetikus szöveges adat: ügyféltámogatási jegyek szövege, cél a kategória (`bug`, `feature_request`, `account`). Megfigyelési egység a jegy. Nincs előre betanított nagy modell; a kérdés, hogy a TF-IDF + logisztikus baseline elegendő-e, vagy embedding szükséges. Előzetes becslés: ha a szókincs jól szeparál, a macro-F1 0,70 fölött lehet, de ezt a baseline igazolja.
- **P3 – Napi rendelés-előrejelzés (idősor).** A Northstar `orders` táblájából aggregált napi rendelésszámot kell előrejelezni 7 napra. Megfigyelési egység a nap, a cél a következő napok összrendelésszáma. Előzetes becslés: a naiv prediktor (a jelenlegi napi átlag) pontossága erős alapvonal; a neurális modellnek ezt kell meghaladnia.

A feladatunk: mindháromhoz módszer- és környezetválasztási lapot készíteni, P1-hez kis futtatható baseline-t, P2-höz és P3-hoz ellenőrizhető kísérleti tervet adni. A döntés kritériumai nem az "újdonság", hanem a hat dimenzió.

28.4.2 Hat döntésdimenzió: fogalmi magyarázat

Representation (reprezentáció). Mi a modell bemenete és milyen formában? Tabuláris adatnál a jellemzővektor: `distance_km`, `priority_flag`, `basket_minor`, stb. Szövegnél token-számláló mátrix vagy sűrű embedding. Idősornál csúszó ablakból képzett vektor. A reprezentáció nem semleges: a szöveg szókincse nagy és ritka, ezért `TF-IDF` mellett ritka mátrixot használunk; a tabuláris adatnál a numerikus jellemzők közvetlenül bemenetek. Egy jó reprezentációval egyszerűbb modell is elegendő lehet.

Példa: a `basket_minor` pénzérték nagyságrendje nagy (pl. 50 000 cent), a `distance_km` kicsi (pl. 2–120). Standardizálás nélkül a logisztikus regresszió optimalizálása lassan konvergál. A reprezentáció itt a skálázást is jelenti.

Interpretability (értelmezhetőség). A döntéshozó számára fontos lehet, hogy miért késik a rendelés. A logisztikus regresszió koefficiensei közvetlenül magyarázhatók: ha a `distance_km` koefficiense pozitív, a távolság növeli a késés valószínűségét. A többrétegű neurális háló súlyai nem közvetlenül értelmezhetők; magyarázathoz gradiens-alapú vagy perturbációs módszer kell. Az értelmezhetőség nem abszolút tulajdonság, hanem a fogyasztó kérdésétől függ.

Uncertainty (bizonytalanság). A modell kimenete egy $\hat{p}$ becsült valószínűség, de ez önmagában nem mondja meg, mennyire megbízható a becslés új bemenetre. A kalibrációs görbe azt méri,

hogy a modell 0,7-es valószínűségű jóslatai valóban 70%-ban helyesek-e. A konformális predikció szűkebb/tágabb intervallumot adhat a bizonytalanságra. Az LLM-eknél a nem-determinizmus külön mintavételi bizonytalanságot visz be, amit a [27.4. alfejezet] megismételt futtatásokkal kezelt.

Evaluation (értékelés). A metrika és a split a feladathoz igazodik. Ritka eseménynél az accuracy félrevezető: ha a késési ráta 10%, egy „mindig időben” modell 90%-os pontosságú, de használhatatlan. A log loss a valószínűségi kalibrációt bünteti, a ROC-AUC a rangolást méri. Az időbeli feladatnál a [24.4. alfejezet]-ben tanult purged split szükséges, nem véletlenszerű keresztvalidáció. És a tesztkészletet a döntéshozatal nem érintheti, különben az értékelés optimista.

Compute (számítási költség). A Northstar 2400 rendelése egy CPU-n milliszekundum alatt tanítható. A GPU bevezetése ilyen adatméreten nem gyorsítás, hanem adminisztratív terhelés. A [33.2. alfejezet] és [33.4. alfejezet] tanulsága: a mérési terv nélküli „gyorsítsunk” nem eredmény. Számítási költségnek tekintjük a paraméterszámot, memóriát, betanítási időt és az új hardver karbantartási költségét is.

Data boundary (adathatár). Mi ismert a predikció pillanatában, és mi nem? A Northstar `delivery_days` a rendeléskor nem ismert, tehát predictorba tiltott. A szöveges feladatnál a jegy teljes szövege ismert, de a jövőbeli jegyek szókincse nem. Az idősornál a mai napi adat a holnapi előrejelzéshez ismert, de a holnapi rendelések nem. Az adathatár megsértése leakage-hez vezet; ez a [24.4. alfejezet] fő témája volt.

A hat dimenzió együttesen adja a döntési lapot (decision sheet). A lap nem automatikus algoritmus, hanem átlátható indoklás.

Egy fontos képlet a log loss-hoz:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\right]$$

ahol $y_i \in \{0,1\}$ a valódi címke, $\hat{p}_i \in (0,1)$ a modell becsült valószínűsége, n a tesztminták száma. Mindkét vektor alakja $(n,)$. A ln természetes logaritmus; a $\hat{p}_i$ szigorúan nyitott intervallumban van, mert $\ln 0$ nem véges. Ha $\hat{p} = 0{,}2$ és a modell mindig 0,2-t mond egy 20%-os pozitív arányú populáción, a várható veszteség $-0{,}2\ln 0{,}2 - 0{,}8\ln 0{,}8 \approx 0{,}5004$. Ezt használjuk kézi kontrollként.

A numerikus stabilitás nem automatikus: ha egy predikció pontosan 0 vagy 1, a naiv képlet végtelent ad. A gyakorlatban a valószínűséget egy kis ε és $1-\varepsilon$ közé kell vágni:

```
import numpy as np

def log_loss_stable(y, p, eps=1e-12):
    p = np.clip(np.asarray(p, dtype=float), eps, 1 - eps)
    y = np.asarray(y, dtype=float)
    return float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p)))

print(log_loss_stable([0, 1], [1.0, 0.0]))  # 27.631
```

A mért érték $\varepsilon = 10^{-12}$ mellett 27.6310; a `-np.log(1e-12)` értéke 27.6310, tehát a csonkolt képlet véges és a várt nagyságrendet adja. A scikit-learn `log_loss` ugyanerre a bemenetre 36.0437-et ad, mert a saját belső csonkolását használja; a két érték nem egyezik bitazonosan, de mindkettő véges, és a végtelen helyett egy előre rögzített, dokumentált büntetést ad. A kézi kontrollnál ezért mindig rögzíteni kell az ε értékét és a könyvtár verzióját.

28.4.3 Teljes minimális munkapélda: P1 tabuláris baseline

A következő kód a Northstar adatot és a splitet használja. A futáshoz előbb a lab generátort és a starter kísérletet kell futtatni; a `starter_experiment.py` létrehozza az `artifacts/split.json`-t. A kódot itt **szemléltetőnek** jelölöm: e fejezet generálási módjában nem futtattam, a kimeneti számok elméletileg várt tartományok.

```
# platform: Linux/macOS, munkakönyvtár: a könyv lab/ könyvtára
python generate_data.py --seed 20261020 --customers 300 --orders 2400
python starter_experiment.py --cutoff 2025-05-01T00:00:00+00:00
```

Ezután a baseline Python-kód:

```
import json
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, roc_auc_score, accuracy_score
import torch
import torch.nn as nn
import torch.optim as optim

# Reprodukálható seed
rng = np.random.default_rng(20261020)
torch.manual_seed(20261020)

# -- Adatbetöltés --
customers = pd.read_csv("customers.csv")
orders = pd.read_csv("orders.csv")
orders["order_ts"] = pd.to_datetime(orders["order_ts"], utc=True)
orders["label_available_at"] = pd.to_datetime(
    orders["label_available_at"], utc=True
)
customers["signup_date"] = pd.to_datetime(customers["signup_date"])

df = orders.merge(customers, on="customer_id", how="inner")

# Csak a split előtti, címkével rendelkező rendeléseket használjuk.
with open("artifacts/split.json", "r") as f:
    split = json.load(f)
train_ids = set(split["train_order_ids"])
test_ids = set(split["test_order_ids"])

df_train = df[df["order_id"].isin(train_ids)].copy()
df_test = df[df["order_id"].isin(test_ids)].copy()

# -- Feature-ök és címke --
features = ["distance_km", "priority_flag", "basket_minor",
            "quantity_sum", "account_age_days", "segment",
            "region", "acquisition_channel"]
categorical = ["segment", "region", "acquisition_channel"]
numeric = [f for f in features if f not in categorical]

X_train = df_train[features]
y_train = df_train["target_late"].astype(int).to_numpy()
X_test = df_test[features]
y_test = df_test["target_late"].astype(int).to_numpy()

# -- Baseline: logisztikus regresszió --
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
    ]
)
baseline = Pipeline([
    ("pre", preprocessor),
    ("clf", LogisticRegression(max_iter=2000, C=1.0, random_state=20261020)),
])
baseline.fit(X_train, y_train)
p_base = baseline.predict_proba(X_test)[:, 1]

# Ellenőrzés: p_base shape (n_test,) és értékkészlet (0,1)
assert p_base.shape == (len(y_test),)
assert np.all((p_base > 0) & (p_base < 1))

# -- PyTorch MLP --
torch.manual_seed(20261020)
# Ugyanaz a preprocesszor; a kategorikus oszlopokból one-hot mátrixot készítünk
# a sklearn transformjával, hogy a PyTorch-bemenet numerikus legyen.
X_train_pre = baseline.named_steps["pre"].transform(X_train).toarray()
X_test_pre = baseline.named_steps["pre"].transform(X_test).toarray()
X_train_t = torch.tensor(X_train_pre, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_t = torch.tensor(X_test_pre, dtype=torch.float32)

class MLP(nn.Module):
    def __init__(self, in_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1),
        )
    def forward(self, x):
        return self.net(x)

model = MLP(in_dim=X_train_pre.shape[1])
optimizer = optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.BCEWithLogitsLoss()

n_epochs = 30
batch_size = 64
for epoch in range(n_epochs):
    model.train()
    perm = torch.randperm(len(X_train_t))
    for i in range(0, len(X_train_t), batch_size):
        idx = perm[i:i + batch_size]
        optimizer.zero_grad()
        logits = model(X_train_t[idx])
        loss = loss_fn(logits, y_train_t[idx])
        loss.backward()
        optimizer.step()

model.eval()
with torch.no_grad():
    logits_test = model(X_test_t)
    p_torch = torch.sigmoid(logits_test).squeeze().numpy()

# -- Kiértékelés --
```

```python
def report(name, p):
    loss = log_loss(y_test, p)
    auc = roc_auc_score(y_test, p)
    acc = accuracy_score(y_test, (p >= 0.5).astype(int))
    print(f"{name:25s} log_loss={loss:.4f}  AUC={auc:.4f}  acc={acc:.4f}")

report("baseline_logistic", p_base)
report("mlp_cpu", p_torch)
print("teszt_méret:", len(y_test), " késési_ráta:", y_test.mean())
```

A TensorBoard áttekintés céljából a betanítási veszteséget naplózhatnánk `SummaryWriter`-rel, például:

```python
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("runs/mlp_cpu")
writer.add_scalar("loss/train", loss.item(), epoch)
writer.close()
```

A futtatás után a `tensorboard --logdir runs` parancs indítja a vizualizációt. A TensorBoard a kísérletkövetés [31.1. alfejezet] egyszerű, lokális előképe; nem helyettesíti a metrikák szöveges dokumentálását.

28.4.4 Eredmény értelmezése

A fenti kód **nem futtatott**, ezért a számokat „szemléltető” vagy „elméletileg várt” jelöléssel közöljük. A kontrollhoz a log loss képletét használjuk.

- Kézi kontroll: ha a késési ráta 0,20, a konstans modell log loss-ja 0,5003. Egy értelmes modell ennél alacsonyabb értéket ad; a túl alacsony érték (pl. 0,01) viszont szivárgásra vagy hibás szétválasztásra gyanús. A várt tartomány egy logisztikus regressziónál 0,35–0,48 között lehet, az adatszerződéstől függően. Az MLP-nek hasonló vagy kissé jobb értéket kell adnia; ha sokkal rosszabb, akkor a háló túltanult vagy a normalizáció hibás.
- Ellenőrző invariant: a `p_base` és `p_torch` hossza a tesztminták számával egyezik, minden elem a (0,1) intervallumba esik. A `p_base` a logisztikus regresszió kimenete, tehát monoton függvénye a lineáris kombinációnak; ezt az olvasó közvetlenül is ellenőrizheti egy-egy mintán.
- Tolerancia: a log loss harmadik tizedesjegyében CPU-gyártótól és BLAS-verziótól függő eltérés lehet; a statisztikai következtetésnél nem a tizedesjegyek számítanak, hanem az eljárás és a split.

A döntési lap P1-re:

Dimenzió	Döntés	Indoklás
Reprezentáció	Tabuláris vektor, skálázás + one-hot	A jellemzők közvetlenül rendelkezésre állnak; a ritka szöveg nem jelenik meg.
Értelmezhetőség	Logisztikus regresszió az elsődleges; MLP csak összehasonlításra	A döntéshozó a `distance_km` és `priority_flag` hatását kéri.
Bizonytalanság	Kalibrációs görbe a tesztkészleten	A `hat_p` becslés nem garantáltan kalibrált; a log loss ezt méri.
Értékelés	Purged time-based split, log loss + AUC	A [24.4. alfejezet] szerint a teszt nem vezérelhet keresést.

Dimenzió	Döntés	Indoklás
Számítás	CPU, `torch` 2.x, 30 epoch	2400 soron a GPU nem indokolt; a mérési terv a [33.4. alfejezet] szerint.
Adathatár	Csak rendeléskor ismert feature-ök; a `delivery_days` tiltott	A [24.4. alfejezet] és az adatszerződés szerint.

A szakosodási döntésnél az összehasonlított modellekhez egységes adatfelosztás és költségkeret szükséges. A tesztértékek többszöri felhasználása modellválasztásra optimista becslést eredményez.

28.4.4.1 Belépési feltétel: integrációs minimumvizsga

A PyTorch, a GPU vagy bármely szakosodott modell bevezetése nem lehet önálló döntés: a kiadandó artefaktumnak előbb át kell esnie egy **integrációs minimumvizsgán**. A belépési feltétel öt eleme:

Elem	Követelmény	Bizonyíték
Klasszikus baseline	Ugyanazon a spliten tanított logisztikus regresszió, dokumentált metrikákkal	`roc_auc`, `average_precision`, `brier` a teszt halmazon
Split	Purged time-based vágás, rögzített cutoff, a `label_available_at` feltétel	`cutoff`, `train_ids`, `test_ids`; a tanító és teszt azonosítói diszjunktak
Teszt	Egyszeri, érintetlen kiértékelés; a teszt mérete és a pozitív arány rögzítve	`n`, `prevalence`, a teszt azonosítóinak hash-e
Költségbecslés	Legalább három futás mediánja, környezeti ujjlenyomattal	`runs_s`, `median_s`, `environment`
CPU-kontroll	Ugyanaz a modell és split CPU-n, a szakosodott változat előtt	`same_split`, `within_tolerance`, a mért különbség

A Northstar labor `lab/ellenorzott_report.json` fájlja ebből a baseline-, split- és teszt-részt már tartalmazza: a purged split 1531 tanító, 800 teszt és 69 kizárt sorból áll, a teszt pozitív aránya 0,2375; a logisztikus baseline ROC-AUC-ja 0,9198, average precisionje 0,7911, Brier-pontszáma 0,0965, szemben a dummy 0,1846-os Brier-értékével. A szakosodott modell csak ezekhez a rögzített számokhoz mérhető, ugyanazon a teszten. A költség- és CPU-kontroll rekordot a saját futtatásból kell hozzátenni; a labor jelentése ezt nem tartalmazza.

A kapu programozottan is ellenőrizhető:

```
REQUIRED = ("baseline", "split", "test_eval", "cost", "cpu_control")

def integration_exam(artifacts):
    errors = []
    baseline = artifacts.get("baseline", {})
    for key in ("model", "roc_auc", "average_precision", "brier"):
```

```
        if key not in baseline:
            errors.append(f"baseline: hiányzó {key}")
    split = artifacts.get("split", {})
    train_ids = set(split.get("train_ids", []))
    test_ids = set(split.get("test_ids", []))
    if not train_ids or not test_ids:
        errors.append("split: üres train vagy test")
    if train_ids & test_ids:
        errors.append("split: átfedő azonosítók")
    if not split.get("cutoff"):
        errors.append("split: hiányzó cutoff")
    test_eval = artifacts.get("test_eval", {})
    if test_eval.get("n") != len(test_ids):
        errors.append("test_eval: n nem egyezik a teszt méretével")
    cost = artifacts.get("cost", {})
    if len(cost.get("runs_s", [])) < 3:
        errors.append("cost: kevesebb mint három futás")
    if not cost.get("environment"):
        errors.append("cost: hiányzó environment")
    control = artifacts.get("cpu_control", {})
    if control.get("same_split") is not True:
        errors.append("cpu_control: same_split nem teljesül")
    if control.get("within_tolerance") is not True:
        errors.append("cpu_control: within_tolerance nem teljesül")
    return errors
```

Az invariáns: a függvény üres listát ad akkor és csak akkor, ha mind az öt elem hiánytalan; bármely hiányzó elem esetén a szakosodási döntés még nem hozható meg. A `tests/test_integration_minimum.py` ezt a kaput teszteli: a teljes rekord átmegy, a hiányzó baseline, az átfedő split, a hibás tesztméret, a kevés futás és a rossz CPU-kontroll mind elbukik.

28.4.5 A másik két probléma döntési lapja

P2 – Szöveges support-jegy osztályozás. A baseline: `TfidfVectorizer(max_features=5000, min_df=3)` + `LogisticRegression(max_iter=2000)`. A kísérleti terv:

1. **Split:** a jegyek időbeli sorrendben vannak; a tanító/trajekt a jegy időbélyegének 70/30 arányú purged vágása. A szivárgás elkerülésére a `TfidfVectorizer` csak a tanítószövegen fit-elődik.
2. **Metrika:** macro-F1, mert a kategóriák kiegyensúlyozatlanok lehetnek; kiegészítésként calibrated log loss.
3. **Bizonytalanság:** a softmax valószínűséget kalibrációs görbén mérjük; ha a modell magabiztos és téved, az a hiányzó kontextus jele.
4. **Értelmezhetőség:** a logisztikus koefficienseket tokenenként vizsgáljuk; a legerősebb pozitív/negatív tokenek listája az interpretability.
5. **Számítás:** a 300 jegy 5000 szótárelem mellett CPU-n másodperces tanítást ad; nincs szükség GPU-ra.
6. **Adathatár:** a jegy szövege a predikció pillanatában ismert; a későbbi válaszok vagy megoldások nem használhatók predictorba.

P3 – Napi rendelés-előrejelzés. A neurális jelölt: egyszerű MLP csúszó ablakkal, a bemenet az elmúlt 14 nap napi rendelésszáma, a kimenet a következő 7 nap összege. A baseline: az elmúlt 14 nap napi átlagának 7-szerese (a naiv szint). A kísérleti terv:

1. **Split:** időbeli, purged: a tanító a cutoff előtti ablakokból áll, a teszt a cutoff utáni 7 napos célokból. A [20.2. alfejezet] rolling walk-forward explicit.

2. **Metrika:** MAE és RMSE a napi rendelésszám mértékegységében, valamint a relatív hiba a naiv baseline-hoz képest.
3. **Bizonytalanság:** a predikciós intervallumhoz a tanító reziduumok kvantilisét használjuk; ez konformális intervallumnak egyszerű változata.
4. **Számítás:** a Northstar 180 napja 165 ablakot ad; CPU-n elegendő. A GPU csak több év adatánál válna opcióvá.
5. **Adathatár:** a holnapi rendelés ma nem ismert; a `delivery_days` nincs a célban, mert a rendelésszám csak az `order_ts`-ből származik.

A P2 és P3 tervnek közös vonása: a split explicit, a baseline a legegyszerűbb érvényes prediktor, és a metrikát a döntési célhoz kötjük. A terv önmagában nem eredmény; a végrehajtásnál a manifestet és a kódot verziózni kell.

28.4.6 Feladatok

28.4.6.1 1. feladat – Értelmezés

Egy kolléga azt mondja: „A P1 feladatban a neurális modell accuracy-je 89%, ezért jobb, mint a logisztikus regresszióé.” Miért nem elegendő ez az állítás a módszerválasztáshoz? Nevezz meg legalább három hiányzó információt, és mondd meg, milyen további mérést kellene végezni.

28.4.6.2 2. feladat – Megvalósítás

Válaszd a P2 szöveges feladatot. Írj kis szintetikus adatgeneráló kódot (20 jegy, 3 kategória), majd taníts TF-IDF + logisztikus baseline-t. A megoldásban add meg a macro-F1 értékét és egy token-koefficiens példát. A kódot megjelölheted szemléltetőnek, de a logikát ellenőrizd kézzel.

28.4.6.3 3. feladat – Hibadiagnózis

Valaki a P1 feladatra a következőt jelenti: „GPU-ra tettem a modellt, a betanítás 10-szer gyorsabb lett, de a teszt log loss 0,65.” Mi lehet a hiba? Adj legalább két lehetséges okot, és mindegyikhez egy ellenőrző lépést.

28.4.6.4 4. feladat – Megváltozott feltételű továbbvitel

A P3 idősornál a stakeholder azt kéri, hogy a modell ne csak a rendelésszámot, hanem a késési arányt is jelezze előre naponta. Hogyan változik a döntési lap `evaluation`, `uncertainty` és `data boundary` mezője? Adj konkrét javaslatot.

28.4.7 Megoldások és indoklás

28.4.7.1 1. megoldás

Az accuracy önmagában nem elég, mert:

1. **Ritka esemény:** ha a késési ráta 11%, a 89%-os accuracy a konstans „nincs késés” modell is lehet. Pontosabban: ha a tesztben 11% a pozitív, a mindig-0 prediktor accuracy-je 89%. A log loss és AUC jelzi a valódi elkülönítést.

2. **Kalibráció:** a 89%-os accuracy nem mond semmit a becsült valószínűségek megbízhatóságáról. A log loss ezt méri; a kalibrációs görbe mutatja a szisztematikus torzítást.
3. **Split-érvényesség:** ha a kolléga a tesztet használta modellválasztásra, az eredmény optimista. A [24.4. alfejezet] szerint a teszt érintetlen kell maradjon.

További mérés: ugyanazon a purged teszten a log loss, AUC és a kalibrációs görbe; valamint a modellkártya robusztussági mezője [26.4. alfejezet] szerint.

Ellenőrző invariant: a log loss a ritka konstans modellnél $0{,}11 \cdot \ln(0{,}11) + 0{,}89 \cdot \ln(0{,}89) \approx 0{,}347$, tehát a 0,35 alatti log loss már értelmes, a 0,5 fölötti nem.

28.4.7.2 2. megoldás

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, classification_report
from sklearn.model_selection import train_test_split

# Szintetikus 20 jegy; a tokenek kategóriánként szeparálnak.
texts = [
    "app crashes when saving file", "bug in login screen",
    "button does nothing", "error during export",
    "crash after update", "cannot open settings",
    "login fails with timeout", "please add dark mode",
    "feature request export csv", "would like keyboard shortcut",
    "account blocked after payment", "cannot reset password",
    "need invoice history", "subscription upgrade failed",
    "billing page empty", "delete my data",
    "dark mode would help", "csv export needed",
    "mobile app crash", "reset password link expired",
]
labels = ["bug", "bug", "bug", "bug", "bug", "bug", "bug",
          "feature_request", "feature_request", "feature_request",
          "account", "account", "account", "account", "account", "account",
          "feature_request", "feature_request", "bug", "account"]

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.3, random_state=20261020, stratify=labels
)
vec = TfidfVectorizer(min_df=1)
X_tr = vec.fit_transform(X_train)
X_te = vec.transform(X_test)
clf = LogisticRegression(max_iter=2000, random_state=20261020)
clf.fit(X_tr, y_train)
pred = clf.predict(X_te)
macro_f1 = f1_score(y_test, pred, average="macro")
print("macro-F1:", macro_f1)
print(classification_report(y_test, pred))

# Token-koefficiens példa: a logisztikus egy-versus-rest koefficiensek
coef = clf.coef_[0]
for w, c in sorted(zip(vec.get_feature_names_out(), coef),
                   key=lambda x: abs(x[1]), reverse=True)[:5]:
    print(f"{w:20s} {c:.3f}")
```

A kód szemléltető; a szintetikus adatban a `crash`, `login`, `export`, `csv`, `billing`, `password` tokenek erősen szeparálnak, ezért a macro-F1 várhatóan 0,8 fölötti. **Ellenőrző invariant:** a vektorizáló után az `X_tr` sorainak összege mindig pozitív, mert minden dokumentumban van legalább egy token. A macro-F1 0 és 1 közötti. Tolerancia: a `train_test_split` seed-jétől

függően az érték 0,75 és 1,00 között változhat; a stratify miatt minden kategória megjelenik a tesztben.

28.4.7.3 3. megoldás

Két lehetséges hiba:

1. **A GPU-ra költöztetés nem oldd meg a túltanulást.** Ha a háló túl nagy, a 300 soros tanítóhalmazon a teszt log loss 0,65 lehet, mert a háló a tanító zajhoz illeszkedett. Ellenőrző lépés: a tanító vs teszt log loss összehasonlítása; ha a tanító közel 0, a teszt magas, túltanulás van. A megoldás kisebb háló, korai leállítás, erősebb regularizáció.
2. **A normalizáció a betanítási statisztikákat nem ugyanúgy használja.** Ha a standardizálás a teljes adatra illeszkedett, a tesztből szivárgott információ; vagy ha a PyTorch-bemenet nem ugyanazt a transzformációt kapta, a skála eltérő. Ellenőrző lépés: a `X_train_pre` és `X_test_pre` oszlopainak átlaga a tanítófázisban nulla legyen, a tesztnél közel nulla; és a két fázisban ugyanazt a fitted preprocesszort használjuk.

Továbbá a GPU-val kapcsolatban: a 2400 soron a GPU adatmozgatási ideje nagyobb lehet, mint a tényleges számítás. A [33.4. alfejezet] szerint a változatlan jelentés melletti optimalizálás a kiindulópont.

28.4.7.4 4. megoldás

Ha a cél napi késési arány, akkor:

- **Evaluation:** a nap szintű arány regressziós cél; a metrika MAE vagy RMSE százalékpontban. A naiv baseline az előző 14 nap átlagos késési aránya. A split marad purged time-based.
- **Uncertainty:** a predikciós intervallum kvantilis-reziduumokból; mivel a cél arány 0 és 1 közé esik, a reziduumok eloszlása ferde, ezért logit-transzformált skálán érdemes intervallumot számolni, majd visszatranszformálni.
- **Data boundary:** a késési arányhoz a `delivery_days` szükséges, ami a kézbesítés napján válik ismertté. Ha a mai napra napi késési arányt jelzünk, a kézbesítés lehet, hogy mai, de a nap végéig nem teljes. Ezért a napi arányhoz a cutoff előtti napok teljesen megfigyelt címkéit kell használni, különben cenzorált célhoz jutunk. A megfigyelési szerződésben rögzíteni kell, hogy a napi arány csak a teljesen megfigyelt napokra értelmezett, vagy külön cenzorált modellt kell építeni.

28.4.8 Gyakori tévedés ellenpéldával

A leggyakoribb hiba ezen az ellenőrző ponton: **az új eszköz bevezetése önmagában eredményként van kommunikálva.** Példa: a P1 feladatban valaki azt jelenti, hogy „bevezettük a PyTorch-ot és a GPU-t, a rendszer kész”. De ha a logisztikus baseline log loss-ja 0,41, az MLP-é 0,46, akkor az új eszköz rosszabb eredményt ad. A tudományos állítás a metrikában és a bizonytalanságban van, nem a keretrendszer nevében.

Visszautalás: a [27.4. alfejezet] külső LLM-eknél is ezt tanította: a futtatás azonosítása kód-, modell- és promptverzióval történik, nem az API névvel. Az eszköz nem helyettesíti a mérést.

Önellenőrzés: ha egy döntési lapon a `compute` mező csak annyit mond, hogy „GPU-t használunk, mert gyorsabb”, az nem mérési terv. A [33.4. alfejezet] szerint a gyorsítás feltétele az azonos input, azonos metrika és a tényleges mért futásidő.

28.4.9 Rövid összefoglaló

A VII. ellenőrző pont a módszerválasztást hat dimenzióban teszi tudatossá: reprezentáció, értelmezhetőség, bizonytalanság, értékelés, számítási költség és adathatár. A tabuláris Northstar-feladathoz logisztikus baseline-t és CPU-s MLP-t hasonlítottunk purged spliten, log loss-szal és AUC-vel; a szöveges és idősoros feladatokhoz kísérleti tervet adtunk, amely a baseline-t és az időbeli splitet előre rögzíti. Az eszköz nem helyettesíti a mérést: a GPU vagy az új keretrendszer csak akkor döntés, ha a metrika és a költség igazolja. Következő lépésként a [29.1. alfejezet] ezt a döntési fegyelmet moduláris Python-projektbe fordítja.

29 Kutatási kódból tesztelhető Python-projekt

29.1 Modulhatár és tiszta függvény

Belépő · szint: haladó · idő: 90 perc · eszköz: pytest, uv, Git

Előfeltétel: 28.4. alfejezet · Célkészség: Modulhatár és tiszta függvény · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A refaktorálás nem a notebook eldobása, hanem a benne felhalmozott tudás lefordítása olyan egységekre, amelyeket parancssorból és tesztből is meg tudunk hívni. Ebben az alfejezetben a Northstar-rendelések egyik időszerű feature-ét, az `account_age_days` értéket emeljük ki a notebookból egy `src/northstar` elrendezésű csomagba. A cél, hogy ugyanaz a matematikai transzformáció fusson a notebookban, a CLI-ben és a pytest alatt, miközben a fájlolvasás és -írás a modul szélén marad.

29.1.1 A célhelyzet és a refaktorálás szerződése

A helyzet: egy notebook cellája jelenleg nagyjából ezt csinálja.

```
# notebookbeli, állapotérzékeny változat vázlata
df = pd.read_csv("data/orders.csv", parse_dates=["order_ts"])
cust = pd.read_csv("data/customers.csv", parse_dates=["signup_date"])
joined = df.merge(cust[["customer_id", "signup_date", "segment", "region",
                        "acquisition_channel"]],
                  on="customer_id", how="left")
joined["account_age_days"] = (joined["order_ts"] - joined["signup_date"]).dt.days
```

A bemenetek: `orders.csv` rendelésenként, `customers.csv` ügyfelenként. A megfigyelési egység a rendelés; a cél egy rendelésenkénti numerikus oszlop, amely a rendelés pillanatában ismert ügyfélkort adja napokban. A kimeneti kontroll: a kézzel számolható példasornál a képletnek ugyanazt kell adnia, mint a notebookbeli számításnak, és az oszlop `int64` dtype-ú, nemnegatív, a sorok száma megegyezik az `orders.csv` sorainak számával. A refaktorálás matematikai hídja az alábbi transzformáció:

$$a(t_s, t_o) = \left\lfloor \frac{t_o - t_s}{1 \text{ nap}} \right\rfloor,$$

ahol t_s a `signup_date`, t_o az `order_ts`; a kivonás időbélyeg-különbség, a floor pedig teljes napokra csonkít. A bemeneti sorozatok egyaránt n hosszúak, a kimenet is n hosszú egész napokból áll. Az adatszerződés szerint $t_s \leq t_o$, ezért elvárjuk, hogy $a \geq 0$. A példában szereplő datetime-ok egyeznek a Northstar-szerződés UTC-kezelésével, így nincs rejtett időzóna-konverzió.

29.1.2 Fogalmi magyarázat: tiszta függvény, I/O-határ, src layout, import és CLI

Egy **tiszta függvény** (pure function) a bemenetéből kiszámít egy kimenetet, és nem módosítja a bemenetet, nem ír fájlt, nem olvas globális állapotot, és nem hív véletlenszám-generátort. Ugyanarra a bemenetre mindig ugyanazt a kimenetet adja. Ez nem csak stílus: a tiszta függvényt közvetlenül lehet tesztelni anélkül, hogy a fájlrendszert vagy egy notebook végrehajtási állapotát utánoznunk kellene.

Az **I/O-határ** (I/O boundary) a fájl, hálózat, adatbázis, óra és véletlen számára fenntartott réteg. A megoldásban a `load_orders`, `load_customers` és `save_features` tartoznak ide; ezek a dtype- és sémaszerződés tulajdonosai. A transzformációk csak a már betöltött DataFrame-ekkel dolgoznak. Így a pure függvény akkor is tesztelhető, ha az adatfájlt nem adjuk át neki.

Az **src layout** a Python-projekt javasolt szerkezete: a csomag forráskódja a `src/northstar` mappában van, a tesztek a `tests/` mappában, a futtatási kimenetek az `artifacts/` mappában. A `pyproject.toml` `[tool.setuptools.packages.find] where = ["src"]` beállítása mondja meg a csomagolónak, hogy a `src` alatt keresse a csomagot. Ez elkülöníti a forrást az adattól és a jegyzetfüzettől; a notebook a telepített vagy elérhető csomagot importálja.

```
lab/
  data/
     orders.csv
     customers.csv
  artifacts/
  src/
     northstar/
         __init__.py
         features.py
         io.py
         cli.py
  tests/
     test_features.py
  pyproject.toml
```

Az **import** a modulhatár nyelve. A `__init__.py` határozza meg a publikus API-t; a notebook csak ezt használja:

```
from northstar.features import compute_account_age_days, build_order_features
from northstar.io import load_orders, load_customers, save_features
```

A **CLI** (command-line interface) az `argparse`-szal épített belépési pont. A notebook interaktív kutatáshoz jó; a CLI reprodukálható, naplózott, tesztelhető futtatást ad. A CLI nem számol közvetlenül: betölt, meghívja a tiszta transzformációkat, kiír, és a `logging` modulon keresztül beszámol. A típusellenőrzés itt elsősorban a `pd.DataFrame` és `Path` típushintjein keresztül segíti az olvasót; a futásidejű dtype-ellenőrzés az I/O-réteg és a pure függvény bemeneti feltétele.

29.1.3 Teljes minimális munkapélda

Először a tiszta feature-függvény. A `dt.days` a `Timedelta.days` viselkedését használja: teljes napokra csonkít. Ezt a viselkedést őrzi meg a refaktor, mert a kiinduló notebook is ezt használta.

```
# src/northstar/features.py
"""A Northstar-rendelések tiszta feature-transzformációi."""

from __future__ import annotations

import pandas as pd
```

```
def compute_account_age_days(
    signup_date: pd.Series,
    order_ts: pd.Series,
) -> pd.Series:
    """A rendelés pillanatában ismert ügyfélkort adja teljes napokban.

    Bemenet:
        signup_date: (n,) alakú datetime64 Series, az ügyfél regisztrációs dátuma.
        order_ts: ugyanolyan indexű, (n,) alakú datetime64 Series, a rendelés ideje.
    Kimenet:
        (n,) alakú int64 Series, nemnegatív teljes napok.
    """
    if not signup_date.index.equals(order_ts.index):
        raise ValueError("signup_date and order_ts must share the same index")
    if not (
        pd.api.types.is_datetime64_any_dtype(signup_date)
        and pd.api.types.is_datetime64_any_dtype(order_ts)
    ):
        raise TypeError("signup_date and order_ts must be datetime64 Series")

    result = (order_ts - signup_date).dt.days

    if (result < 0).any():
        raise ValueError(
            "account_age_days cannot be negative; check signup_date <= order_ts"
        )

    return result.astype("int64")


def build_order_features(
    orders: pd.DataFrame,
    customers: pd.DataFrame,
) -> pd.DataFrame:
    """A rendelés szintű predikciós feature-tábla előállítása.

    A join rendelésből ügyfélbe many-to-one; a kimenet nem módosítja a bemeneteket.
    """
    customer_cols = [
        "customer_id",
        "signup_date",
        "segment",
        "region",
        "acquisition_channel",
    ]
    order_cols = [
        "order_id",
        "customer_id",
        "order_ts",
        "distance_km",
        "priority_flag",
        "basket_minor",
        "quantity_sum",
    ]

    features = orders[order_cols].merge(
        customers[customer_cols],
        on="customer_id",
        how="left",
        validate="many_to_one",
    )
    features = features.copy()
```

```
    features["account_age_days"] = compute_account_age_days(
        features["signup_date"],
        features["order_ts"],
    )
    return features.drop(columns=["signup_date", "customer_id"]).copy()
```

A `validate="many_to_one"` a 7.2. alfejezet join-kardinalitási tanulsága: nem akarunk rejtett sortöbbszöröződést. A kimenet a 21.4. alfejezet predikciós szerződés oszlopait követi, és nem tartalmazza a szivárgó `delivery_days` értéket.

Az I/O-réteg külön fájlban.

```
# src/northstar/io.py
"""Fájlműveletek a Northstar adatokhoz; a séma- és dtype-szerződés innen indul."""

from __future__ import annotations

from pathlib import Path

import pandas as pd


def load_orders(path: Path) -> pd.DataFrame:
    return pd.read_csv(path, parse_dates=["order_ts"])


def load_customers(path: Path) -> pd.DataFrame:
    return pd.read_csv(path, parse_dates=["signup_date"])


def save_features(df: pd.DataFrame, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
```

A CLI az I/O-határon belül kombinálja a lépéseket.

```
# src/northstar/cli.py
"""CLI a feature-tábla reprodukálható újraépítéséhez."""

from __future__ import annotations

import argparse
import logging
import sys
from pathlib import Path

from .features import build_order_features
from .io import load_customers, load_orders, save_features


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Northstar rendelésszintű feature-tábla építése"
    )
    parser.add_argument("--orders", type=Path, required=True)
    parser.add_argument("--customers", type=Path, required=True)
    parser.add_argument("--out", type=Path, required=True)
    parser.add_argument("--log-level", default="INFO")
    return parser


def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
```

```
    args = parser.parse_args(argv)

    logging.basicConfig(
        level=args.log_level,
        format="%(asctime)s %(levelname)s %(message)s",
    )

    orders = load_orders(args.orders)
    customers = load_customers(args.customers)
    features = build_order_features(orders, customers)
    save_features(features, args.out)
    logging.info("wrote %s rows to %s", len(features), args.out)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

A publikus API rövid `__init__.py`.

```
# src/northstar/__init__.py
"""Northstar tiszta feature-csomag."""

from .features import build_order_features, compute_account_age_days
from .io import load_customers, load_orders, save_features

__all__ = [
    "build_order_features",
    "compute_account_age_days",
    "load_customers",
    "load_orders",
    "save_features",
]
```

A teszt nem fájlt gyárt, hanem kis DataFrame-ekkel ellenőrzi a transzformációt.

```
# tests/test_features.py
import pandas as pd

from northstar.features import compute_account_age_days


def test_account_age_days_matches_manual_calendar():
    signup = pd.to_datetime(
        pd.Series(["2025-01-10", "2025-03-01", "2025-03-02T00:00:00+00:00"])
    )
    order = pd.to_datetime(
        pd.Series(
            [
                "2025-02-10T08:00:00+00:00",
                "2025-03-05T09:30:00+00:00",
                "2025-03-02T00:00:00+00:00",
            ]
        )
    )

    result = compute_account_age_days(signup, order)

    assert result.dtype == "int64"
    assert result.tolist() == [31, 4, 0]
```

A futtatás a `lab` projektgyökérből, Linux/macOS shellben:

```
uv run pytest tests/test_features.py -q
```

A kimenet szemléltető, mert a szöveg írásakor nem futtattunk: `1 passed in 0.42s`. A CLI parancs ugyanonnan:

```
uv run python -m northstar.cli \
  --orders data/orders.csv \
  --customers data/customers.csv \
  --out artifacts/features.csv \
  --log-level INFO
```

A `features.csv` várható sorainak száma 2400, mert az alapgenerált `orders.csv` ennyi rendelési sort tartalmaz; ezt a manifestből lehet ellenőrizni. A kimeneti naplósor várhatóan ilyesmi:

```
2026-10-20 12:00:00,000 INFO wrote 2400 rows to artifacts/features.csv
```

29.1.4 Eredmény értelmezése kézi kontrollal

A kézi kontroll a fenti teszt: 2025-01-10 és 2025-02-10 között 31 teljes nap van, 2025-03-01 és 2025-03-05 között 4, az azonos időpontok között 0. A refaktorálás invariantja: a notebookbeli gyors számítás és a modul kimenete azonos oszlopértékeket ad, tehát a 17.1. alfejezetben bemutatott regressziós illesztés ugyanazt a dizájnmátrixot kapná. A `shape` kontroll: a bemeneti sorozatok `(n,)`, a kimenet `(n,)`; a `build_order_features` kimenete rendelésszintű, ezért a sorok száma nem nőhet a join során. A `dtype` kontroll: a kimenet `int64`, az I/O-réteg pedig `datetime64` bemenetet ad a pure függvénynek.

Két további ellenőrzés a modulhatárról: a `uv run python -m northstar.cli` nem definiál rejtett globális DataFrame-et, és a futtatás közben a `features.py` nem nyit fájlt. Emiatt a notebook és a CLI ugyanazt a CSV-t kapva ugyanazt az `account_age_days` oszlopot adja. A Ruff és a típushintek itt a stílus- és olvashatóság-ellenőrzés részei:

```
uv run ruff check src tests
uv run ruff format src tests
```

Ezek kimenetét a tanuló helyi futtatása dönti el; itt nem állítunk mért eredményt.

A tiszta függvény fogalma nem egyenlő a mutációmentességgel: a függvény mellékhatásmentes, és kimenete csak argumentumaitól függ. Az időzóna és az üzleti nap számításának szemantikáját külön adatszerződésben rögzítsük.

29.1.4.1 A tiszta függvény szerződése: változatlanság és determinizmus

A modulhatár akkor tartható, ha a transzformációs függvény **nem módosítja a bemeneti DataFrame-et**, és ugyanarra a bemenetre mindig ugyanazt adja. A két követelmény külön tesztelhető. Példa a tiszta rétegből:

```
def add_unit_price(df: pd.DataFrame) -> pd.DataFrame:
    required = {"basket_minor", "quantity_sum"}
    missing = required - set(df.columns)
    if missing:
        raise ValueError(f"hiányzó oszlop: {sorted(missing)}")
    if (df["quantity_sum"] <= 0).any():
        raise ValueError("quantity_sum must be positive")
    out = df.copy()
    out["unit_price_minor"] = out["basket_minor"] / out["quantity_sum"]
    return out
```

A `df.copy()` nem stíluselem: nélküle a `out[...] = ...` a hívó által átadott objektumot is módosíthatja, és a notebookban a „később mégis megváltozott” hiba forrása lesz. A tesztek:

```python
import pandas as pd

def test_add_unit_price_does_not_mutate_input():
    df = pd.DataFrame({"basket_minor": [500, 900], "quantity_sum": [2, 3]})
    original = df.copy(deep=True)
    out = add_unit_price(df)

    pd.testing.assert_frame_equal(df, original)
    assert out is not df
    assert "unit_price_minor" not in df.columns
    assert out["unit_price_minor"].tolist() == [250.0, 300.0]

def test_add_unit_price_is_deterministic():
    df = pd.DataFrame({"basket_minor": [500, 900], "quantity_sum": [2, 3]})
    pd.testing.assert_frame_equal(add_unit_price(df), add_unit_price(df))
```

A `pd.testing.assert_frame_equal` az indexet és a dtype-okat is összehasonlítja, ezért erősebb, mint a `df.equals(df.copy())` puszta használata. Determinisztikus a függvény, mert nincs benne véletlen, óra vagy globális állapot; ha mintavételre lenne szükség, a `numpy.random.Generator`-t **argumentumként** kell átadni, nem modulszintű globális seedként.

A fájlkezelés ettől elkülönülő adapter:

```python
# src/northstar/io.py
def load_orders(path: Path) -> pd.DataFrame:
    return pd.read_csv(path)

def save_features(df: pd.DataFrame, path: Path) -> None:
    df.to_csv(path, index=False)
```

A CLI vagy notebook az adaptert hívja, majd a tiszta függvényt; a `features.py` soha nem nyit fájlt. Az adapter tesztje a fájl byte-hash-jét a hívás előtt és után is ellenőrizheti: olvasás nem írhatja felül a bemenetet. Az invariáns: az `add_unit_price` bemenete és kimenete külön objektum, a bemenet értékei változatlanok, és két egymást követő hívás kimenete `assert_frame_equal`-lel azonos.

29.1.5 Feladatok

29.1.5.1 1. Értelmezés: miért nem tiszta a beolvasó függvénybe rejtett számítás?

Egy kolléga így alakítja át a modult: a `compute_account_age_days` belsejében meghívja a `pd.read_csv`-t, hogy a notebookban egy sorba kerüljön a művelet. Sorolj fel legalább két hátrányt, és rajzold meg a helyes I/O-határt egy rövid függvényaláírással.

29.1.5.2 2. Megvalósítás: tiszta függvény az átlagos tételárra

Írj egy `price_per_item_minor(basket_minor: pd.Series, quantity_sum: pd.Series) -> pd.Series` tiszta függvényt a `features.py`-ba. A függvény a `basket_minor / quantity_sum` hányadost adja `float64` dtype-tal. Ha bármelyik `quantity_sum` értéke nulla vagy negatív, dobjon `ValueError`-t, mert ez sérti a Northstar adatszerződést. A bemeneti sorozatok indexe egyezzen meg. Írj egy pytest esetet is, amely egy `quantity_sum=0` sort kapva `pytest.raises(ValueError)`-t vár.

29.1.5.3 3. Hibadiagnózis: a CLI elfut, a notebook nem

A tanuló a CLI-t lefuttatja, és a notebook cellájában ezt a hibát kapja:

```
AttributeError: Can only use .dt accessor with datetimelike values
```

A notebook ezt hívja:

```
orders = pd.read_csv("data/orders.csv")
customers = pd.read_csv("data/customers.csv")
features["account_age_days"] = compute_account_age_days(
    features["signup_date"], features["order_ts"]
)
```

Mi a hiba gyökere, hol kell javítani, és miért ott? Add meg a javítás utáni notebooksort.

29.1.5.4 4. Megváltozott feltétel: kísérleti ügyfélszintű adat

A csoport most a `assignments.csv` és `outcomes.csv` kísérleti fájlokat is szeretné ugyanazzal a CLI-vel feldolgozni. A `build_order_features` rendelésszintű, a kísérleti kimenet ügyfélszintű. Indokold meg, miért nem szabad egyszerűen beilleszteni egy új `mode` ágat a meglévő függvénybe, és tervezd meg, milyen publikus függvényaláírás és CLI-bemenet őrizné meg a szemcsézettségbeli szerződést.

29.1.6 Megoldások és indoklás

29.1.6.1 1. megoldás

A beolvasás a fájlrendszerből olvas: mellékhatás, függ a munkakönyvtártól, és lassú. A tiszta függvény teszteléséhez fájlt kellene létrehozni, és a determinisztikus viselkedés nem garantálható, ha a fájl tartalma közben megváltozik. A helyes határ: az I/O-réteg `load_orders(path) -> pd.DataFrame`, a transzformációs réteg pedig `compute_account_age_days(signup_date, order_ts)`. A kettőt a CLI vagy a notebook külön hívja; ezzel a teszt csak két darab `Series`-t ad át.

29.1.6.2 2. megoldás

```
def price_per_item_minor(
    basket_minor: pd.Series,
    quantity_sum: pd.Series,
) -> pd.Series:
    if not basket_minor.index.equals(quantity_sum.index):
        raise ValueError("basket_minor and quantity_sum must share the same index")
    if (quantity_sum <= 0).any():
        raise ValueError("quantity_sum must be positive")
    return (basket_minor / quantity_sum).astype("float64")
```

A teszt:

```
def test_price_per_item_rejects_non_positive_quantity():
    basket = pd.Series([1200, 900])
    quantity = pd.Series([3, 0])
    with pytest.raises(ValueError):
        price_per_item_minor(basket, quantity)
```

A hányados `float64`, mert a lebegőpontos osztás ezt adja; a `basket_minor` egész minor unit, a kimenet viszont nem egész, ezért a kerekítés nem alkalmazható.

29.1.6.3 3. megoldás

A hiba gyökere az I/O-réteg hiánya: a notebook `pd.read_csv` hívása nem használta a `parse_dates` paramétert, ezért a dátumoszlopok string típusúak maradtak. A `.dt` accessor csak datetime-jellegű oszlopon működik. A javítás nem a pure függvényben kell, mert annak a bemenete már kötelezően `datetime64`; a betöltésnek kell a szerződést teljesítenie. A javított notebooksor:

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_ts"])
customers = pd.read_csv("data/customers.csv", parse_dates=["signup_date"])
```

Ugyanez a dtype-szerződés van a `io.py`-ban, ezért a CLI nem hibázott.

29.1.6.4 4. megoldás

A `build_order_features` a rendelés szemcsézettségére épül: a `merge many_to_one` validációval rendelési sorokat kap. A kísérleti adatokban egy ügyfélhez egy `assignment` és egy kimenet tartozik; a rendelésszintű logika nem vihető át automatikusan. Egy külön `build_experiment_features(assignments, outcomes) -> pd.DataFrame` függvény jobb: ügyfélszintű, `validate="one_to_one"` join, és a `conversion_7d`, `revenue_7d_minor` oszlopokat nem keveri a rendelési tételsorok összegzésével. A CLI-ben külön alparancs, például `build-experiment-features`, tartja el. Így a meglévő rendelésszintű kimenet változatlan marad, és a szemcsézettségbeli hiba fordítási időben, nem futás közben derül ki.

29.1.7 Gyakori tévedés ellenpéldával

A gyakori tévhit, hogy a notebook-problémát egyetlen nagy `FeatureBuilder` osztállyal kell helyettesíteni. Az osztály gyakran rejtett belső DataFrame-et tárol:

```
class FeatureBuilder:
    def __init__(self, data_dir):
        self.df = pd.read_csv(data_dir / "orders.csv")
    def build(self):
        self.df["account_age_days"] = self.df["order_ts"] - self.df["signup_date"]
        return self.df
```

Ez nem jobb szerkezet, csak máshová rejtette az állapotot. A tesztnek fájlt kell hoznia, a `build` kétszeri hívása másodszor már egy módosított DataFrame-et adhat, és a notebook állapotát nem szünteti meg. A tiszta függvényes változatban a bemenet és a kimenet a hívásban látható; az ellenőrzés pedig `pd.testing.assert_series_equal`-lel elvégezhető a notebook régi és a modul új kimenete között.

Önellenőrzés: egy függvény, amely a bemenetét nem módosítja, de minden hívásakor naplófájlba ír, nem tiszta, mert mellékhatása van. A modulhatár tesztje tehát nem csak az, hogy a számítás helyes-e, hanem az is, hogy a transzformáció során nem keletkezik rejtett I/O.

29.1.8 Rövid összefoglaló

A tiszta függvény ugyanahhoz a bemenethez ugyanazt adja, és nincs mellékhatása; a fájlolvasás és -írás az I/O-rétegbe kerül. A `src/northstar` csomagban a `features.py` a transzformációt, az `io.py` a betöltést és mentést, a `cli.py` az `argparse`-belépési pontot adja. A notebook már csak a publikus függvényeket hívja, ezért a CLI és a notebook ugyanazt az `account_age_days` feature-t állítja elő. A kézi naptári kontroll, a shape-, dtype- és indexellenőrzés a refaktorálás invariantja, a pytest pedig fájl nélkül rögzíti a várt viselkedést. A következő alfejezetben ezt a modulhatárt konfigurációval, futási naplóval és reprodukálható parancssori belépéssel bővítjük tovább.

29.2 Konfiguráció és futási napló

> **Belépő · szint: haladó · idő: 90 perc · eszköz: pytest, uv, Git**
>
> Előfeltétel: 29.1. alfejezet · Célkészség: Konfiguráció és futási napló · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

29.2.1 Cél és konkrét helyzet

A 29.1. alfejezetben a notebook logikáját tiszta függvényekre és parancsokra bontottad. Most azt a kérdést oldjuk meg, hogyan marad egy kutatási futtatás később is **visszakereshető**: milyen konfigurációval, milyen adaton, milyen környezetben és milyen eredménnyel futott. A konkrét helyzet egy Northstar-stílusú, de kicsi, szintetikus predikciós feladat: a rendelés pillanatában ismert három feature-ből becsüljük, hogy a kézbesítés 7 napon túli lesz-e.

- **Bemenet:** `configs/tiny.toml`, `data/tiny_train.csv`, `data/tiny_test.csv`.
- **Megfigyelési egység:** egy rendelés, `row_id` azonosítóval.
- **Célváltozó:** `target_late` 0/1.
- **Feature-ök:** `distance_km`, `priority_flag`, `basket_minor`.
- **Elvárt kimenet:** egy futásonként új output-könyvtár, benne a modellpredikciók, a konfiguráció másolata, a futásnapló és egy `manifest.json`, amely hash-ekkel rögzíti a bemeneteket.
- **Előzetes becslés:** a generáló képlet alapján a pozitív arány körülbelül 0,45–0,50 lesz, a `DummyClassifier` ROC-AUC-ja 0,50 körül, a logisztikus regresszió ROC-AUC-ja szemléltető jelleggel 0,70–0,80 közé várható. A pontos számot a helyi futtatás dönti el.

29.2.2 Fogalmi magyarázat

TOML vagy YAML? A TOML a Python 3.11-től beépített `tomllib` modullal olvasható, típusos értékeket ad (string, egész, float, boolean, tömb, tábla), és nem enged tetszőleges Python-objektumot végrehajtani. A YAML kényelmes, de külön `PyYAML` függőséget és biztonsági megfontolást igényel. Ebben az alfejezetben TOML-t használunk; ha egy csapat már YAML-t használ, a döntés lehet más, de a manifest-hash akkor is kötelező marad.

argparse. Az argparse csak a folyamat bemeneti határa: szöveges argumentumokat olvas, és egy `Namespace` objektumot ad vissza. A modell-, adat- és kiíró logika nem az argparse-on belül él. A belépési pont így:

```
def parse_args(argv=None) -> argparse.Namespace:
    ...
```

Log levels. A `print` nem futásjegyzék, mert nem szűrhető és nem íródik fájlba konzisztensen. A logging szintek a fontosságot jelölik:

Szint	Tipikus üzenet	Ne ide kerüljön
DEBUG	shape, kulcsérték, részletes állapot	normál összefoglaló
INFO	lépések, modellmetrika, fájlok	minden ciklusváltozó
WARNING	gyanús, de folytatható esemény	kezelhetetlen hiba
ERROR	sikertelen művelet	szokásos futási zaj

Run id. A futás azonosítója UTC-időbélyegből és rövid véletlen utótagból áll, például `20261020T093001Z-7f3a`. Az időbélyeg önmagában nem elég párhuzamos futtatáskor; a véletlen utótag csökkenti az ütközés valószínűségét, a `mkdir(exist_ok=False)` pedig egy ütközést észlelhető hibává alakít; retry vagy új azonosító szükséges, nem feltételezhető feltétlen ütközésmentesség.

Manifest. A manifest a futás géppel ellenőrizhető jegyzőkönyve: parancssor, konfiguráció- és adathash, környezetverziók és modellmetrikák. A hash itt SHA-256, hexadecimális formában; bármilyen byte-eltérés más hash-t ad.

Immutable output. A futási könyvtár írásvédett logikájú: egyszer hozzuk létre, a fájlokat ideiglenes néven írjuk, majd `os.replace`-szel tesszük láthatóvá. Így egy fájl vagy teljes egészében megjelenik, vagy sehogy; utólagos szerkesztés érvényteleníti a manifest ellenőrző szerepét.

29.2.3 Teljes minimális munkapélda

A projekt szerkezete a 29.1. alfejezet `src` layoutját követi:

```
s114_demo/
  pyproject.toml
  configs/
     tiny.toml
  scripts/
     make_tiny_data.py
  data/                  # a generátor tölti fel
  src/
      northstar/
          __init__.py
          config.py
          run.py
```

A `pyproject.toml` minimális változata:

```toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "northstar-s114"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["pandas>=2.2", "scikit-learn>=1.4"]

[tool.setuptools.packages.find]
where = ["src"]
```

A konfiguráció:

```toml
[run]
models = ["dummy", "logreg"]
seed = 20261020

[data]
train_csv = "data/tiny_train.csv"
test_csv = "data/tiny_test.csv"
target_col = "target_late"

[model.logreg]
C = 0.5
max_iter = 500
```

Az adatgenerátor a korábbi 22.1. alfejezetből ismert logit-függvénnyel sorsol 600 sort. Minden i sorra:

$$p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 d_i + \beta_2 b_i + \beta_3 r_i)}},$$

ahol d_i a `distance_km` (km), b_i a `basket_minor` (EUR-cent), $r_i \in \{0, 1\}$ a `priority_flag`; $\beta_0 = -2{,}0$, $\beta_1 = 0{,}06$, $\beta_2 = 0{,}0001$, $\beta_3 = 0{,}8$. A `target_late` 1, ha a véletlen szám kisebb p_i-nél.

```python
# scripts/make_tiny_data.py
import csv
import math
import random
from pathlib import Path

RNG_SEED = 7
N_TRAIN, N_TEST = 400, 200
OUT_DIR = Path("data")
BETA = (-2.0, 0.06, 0.0001, 0.8)


def rows(n, start_id, rng):
    out = []
    for i in range(n):
        row_id = start_id + i
        distance = round(rng.uniform(0.5, 30.0), 2)
        priority = rng.randint(0, 1)
        basket = rng.randint(500, 12000)
        z = BETA[0] + BETA[1] * distance + BETA[2] * basket + BETA[3] * priority
        p = 1.0 / (1.0 + math.exp(-z))
        out.append({
            "row_id": row_id,
            "distance_km": distance,
            "priority_flag": priority,
            "basket_minor": basket,
            "target_late": 1 if rng.random() < p else 0,
        })
    return out


def write_csv(path, data):
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(data[0]))
        writer.writeheader()
        writer.writerows(data)


rng = random.Random(RNG_SEED)
```

```
all_rows = rows(N_TRAIN + N_TEST, 1, rng)
rng.shuffle(all_rows)
OUT_DIR.mkdir(exist_ok=True)
write_csv(OUT_DIR / "tiny_train.csv", all_rows[:N_TRAIN])
write_csv(OUT_DIR / "tiny_test.csv", all_rows[N_TRAIN:])
print(OUT_DIR / "tiny_train.csv", OUT_DIR / "tiny_test.csv")
```

A konfiguráció és hash-segéd:

```
# src/northstar/config.py
import hashlib
import tomllib
from pathlib import Path
from typing import Any


def load_toml(path: str | Path) -> dict[str, Any]:
    with open(path, "rb") as f:
        return tomllib.load(f)


def sha256_file(path: str | Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b""):
            h.update(chunk)
    return h.hexdigest()
```

29.2.3.1 Precedencia, szigorú típusellenőrzés és titokmaszkolás

Ha ugyanaz a paraméter több helyről is jöhet, a sorrendet explicit módon rögzíteni kell. A Northstar-konvenció: **alapértelmezés < TOML-fájl < környezeti változó < parancssor**. A későbbi forrás mindig felülírja a korábbit; a parancssor a legszűkebb, ezért az győz. A környezeti változók neve prefixet kap (`NORTHSTAR_SEED`, `NORTHSTAR_C`), hogy ne ütközzenek más programokkal. A tényleges, összeolvasott konfigurációt kanonikus JSON-ná alakítva hash-eljük, és ez a hash kerül a manifestbe; a titkok nem részei a hash-nek.

A szigorú típusellenőrzés a `bool` csapdát is kezeli: Pythonban a `bool` az `int` alosztálya, ezért a `seed=True` átcsúszna egy laza `isinstance(value, int)` ellenőrzésen. A `type(value) is int` ezt kizárja. A numerikus tartományokat is ellenőrizni kell (`0 < test_size < 1`, `C > 0`).

```
import hashlib
import json

DEFAULTS = {"seed": 20261020, "test_size": 0.2, "C": 1.0}
ENV_KEYS = {
    "NORTHSTAR_SEED": ("seed", int),
    "NORTHSTAR_TEST_SIZE": ("test_size", float),
    "NORTHSTAR_C": ("C", float),
}
SECRET_PATTERN = ("api_key", "token", "password", "secret")

def validate(cfg):
    for key, typ in (("seed", int), ("test_size", float), ("C", float)):
        if type(cfg[key]) is not typ:
            raise TypeError(
                f"{key}: {typ.__name__} kell, kapott: {type(cfg[key]).__name__}"
            )
    if not 0 < cfg["test_size"] < 1:
        raise ValueError("test_size 0 és 1 közé kell")
```

```
    if cfg["C"] <= 0:
        raise ValueError("C pozitív kell")
    return cfg

def effective_config(file_values, env, cli):
    for key in file_values:
        if any(part in key.lower() for part in SECRET_PATTERN):
            raise ValueError(f"titok a konfigurációban: {key}")
    cfg = {**DEFAULTS, **file_values}
    for env_key, (key, typ) in ENV_KEYS.items():
        if env_key in env:
            cfg[key] = typ(env[env_key])
    cfg.update({key: value for key, value in cli.items() if value is not None})
    return validate(cfg)

def config_hash(cfg):
    payload = json.dumps(cfg, sort_keys=True, separators=(",", ":"))
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()

def mask_secret(value):
    if not value:
        return ""
    return "***" if len(value) <= 8 else "***" + value[-4:]
```

Mért példa: `file_values = {"seed": 1, "C": 0.5}`, `env = {"NORTHSTAR_SEED": "7", "NORTHSTAR_C": "0.25"}`, `cli = {"C": 2.0}` mellett az eredmény `{'seed': 7, 'test_size': 0.2, 'C': 2.0}`; a `config_hash` értéke `570b574e1e0cc416f1ebb2a682b2ca478a2dddb5a4375(`. A `mask_secret("sk-1234567890")` kimenete `***7890`, a rövid `"short"` értéké `***`; a `seed=True` bemenet `TypeError: seed: int kell, kapott: bool` kivételt ad. A titkot tehát nem a TOML-ba, nem a `config.toml` másolatba és nem a manifestbe tesszük, hanem környezeti változóból olvassuk; a naplóba csak a maszkolt alak kerülhet. A `run.log` és a manifest így akkor is megosztható, ha a futtatáshoz kulcs kellett.

A belépési pont összefogja az argparse-ot, a loggingot, a run id-t, a két modell tanítását, a predikciók atomi kiírását és a manifestet:

```
# src/northstar/run.py
from __future__ import annotations

import argparse
import json
import logging
import os
import platform
import secrets
import sys
import time
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path
from typing import Any

import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

from northstar.config import load_toml, sha256_file

logger = logging.getLogger("northstar")
FEATURES = ["distance_km", "priority_flag", "basket_minor"]
```

```
def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    p = argparse.ArgumentParser(description="Northstar s114 futtatás")
    p.add_argument("--config", required=True)
    p.add_argument("--output-root", default="runs")
    p.add_argument("--run-id", default=None)
    p.add_argument("--tag", default=None)
    p.add_argument("--log-level", default="INFO",
                   choices=["DEBUG", "INFO", "WARNING", "ERROR"])
    return p.parse_args(argv)


def make_run_id(tag: str | None) -> str:
    stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
    run_id = f"{stamp}-{secrets.token_hex(2)}"
    return f"{tag.replace(' ', '_')}-{run_id}" if tag else run_id


def setup_logging(level_name: str, run_dir: Path) -> None:
    formatter = logging.Formatter(
        "%(asctime)s %(levelname)s %(name)s %(message)s",
        datefmt="%Y-%m-%dT%H:%M:%SZ",
    )
    formatter.converter = time.gmtime
    root = logging.getLogger()
    root.setLevel(getattr(logging, level_name.upper()))
    for handler in list(root.handlers):
        root.removeHandler(handler)
    fh = logging.FileHandler(run_dir / "run.log", encoding="utf-8")
    fh.setFormatter(formatter)
    sh = logging.StreamHandler()
    sh.setFormatter(formatter)
    root.addHandler(fh)
    root.addHandler(sh)


def atomic_write_str(path: Path, content: str) -> None:
    tmp = path.parent / f".{path.name}.{secrets.token_hex(2)}.tmp"
    try:
        tmp.write_text(content, encoding="utf-8")
        os.replace(tmp, path)
    except Exception:
        tmp.unlink(missing_ok=True)
        raise


def atomic_write_df(path: Path, df: pd.DataFrame) -> None:
    tmp = path.parent / f".{path.name}.{secrets.token_hex(2)}.tmp"
    try:
        df.to_csv(tmp, index=False)
        os.replace(tmp, path)
    except Exception:
        tmp.unlink(missing_ok=True)
        raise


def train_one(name: str, cfg: dict[str, Any], train: pd.DataFrame,
              test: pd.DataFrame, seed: int):
    target = cfg["data"]["target_col"]
    X_train, y_train = train[FEATURES], train[target]
    X_test, y_test = test[FEATURES], test[target]

    if name == "dummy":
```

```
        model = DummyClassifier(strategy="prior", random_state=seed)
    elif name == "logreg":
        p = cfg["model"]["logreg"]
        model = LogisticRegression(
            C=p["C"], max_iter=p["max_iter"], random_state=seed
        )
    else:
        raise KeyError(f"ismeretlen modell: {name}")

    model.fit(X_train, y_train)
    proba = model.predict_proba(X_test)[:, 1]
    pred = (proba >= 0.5).astype(int)
    metrics = {
        "accuracy": round(float(accuracy_score(y_test, pred)), 6),
        "roc_auc": round(float(roc_auc_score(y_test, proba)), 6),
        "n_test": int(len(y_test)),
        "positive_rate_test": round(float(y_test.mean()), 6),
        "pred_positive_rate": round(float(pred.mean()), 6),
    }
    return model, proba, pred, metrics


def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    cfg_path = Path(args.config).resolve()

    try:
        cfg = load_toml(cfg_path)
    except Exception:
        logger.exception("config betöltés sikertelen")
        return 2

    output_root = Path(args.output_root)
    output_root.mkdir(parents=True, exist_ok=True)
    run_id = args.run_id or make_run_id(args.tag)
    run_dir = output_root / run_id
    try:
        run_dir.mkdir(parents=False, exist_ok=False)
    except FileExistsError:
        logger.error("run könyvtár már létezik: %s", run_dir)
        return 3

    setup_logging(args.log_level, run_dir)
    logger.info("config=%s run_id=%s output_root=%s",
                cfg_path, run_id, output_root)

    train_path = cfg_path.parent / cfg["data"]["train_csv"]
    test_path = cfg_path.parent / cfg["data"]["test_csv"]
    try:
        train = pd.read_csv(train_path)
        test = pd.read_csv(test_path)
    except Exception:
        logger.exception("adat betöltés sikertelen")
        return 4
    logger.info("train shape=%s test shape=%s", train.shape, test.shape)

    atomic_write_str(run_dir / "config.toml",
                     cfg_path.read_text(encoding="utf-8"))

    versions = {}
    for pkg in ("northstar-s114", "pandas", "scikit-learn"):
        try:
            versions[pkg] = version(pkg)
```

```
        except PackageNotFoundError:
            versions[pkg] = "not-installed"

    results: dict[str, dict[str, float]] = {}
    seed = int(cfg["run"]["seed"])
    for name in cfg["run"]["models"]:
        try:
            _, proba, pred, metrics = train_one(name, cfg, train, test, seed)
        except Exception:
            logger.exception("model failed=%s", name)
            return 5
        results[name] = metrics
        logger.info("model=%s accuracy=%s roc_auc=%s",
                    name, metrics["accuracy"], metrics["roc_auc"])
        out = test[["row_id"]].copy()
        out["prob_late"] = proba
        out["pred_late"] = pred
        atomic_write_df(run_dir / f"predictions_{name}.csv", out)

    manifest = {
        "run_id": run_id,
        "created_at_utc": datetime.now(timezone.utc).isoformat(),
        "command_line": " ".join(sys.argv),
        "config": {
            "path": str(cfg_path),
            "copy": "config.toml",
            "sha256": sha256_file(cfg_path),
        },
        "data": {
            "train": {"path": str(train_path), "sha256": sha256_file(train_path)},
            "test": {"path": str(test_path), "sha256": sha256_file(test_path)},
        },
        "environment": {
            "python_version": platform.python_version(),
            "platform": platform.platform(),
            "package_versions": versions,
        },
        "models": results,
    }
    atomic_write_str(run_dir / "manifest.json",
                     json.dumps(manifest, indent=2, sort_keys=True) + "\n")
    logger.info("manifest írva; run kész")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Futtatás Linux/macOS bash-ban, a `s114_demo` munkakönyvtárból:

```
cd s114_demo
python -m venv .venv
source .venv/bin/activate
pip install -e .
python scripts/make_tiny_data.py
python -m northstar.run --config configs/tiny.toml \
  --output-root runs --log-level INFO
```

PowerShell-ben az aktiválás és a perjel eltér:

```
cd s114_demo
python -m venv .venv
.venv\Scripts\Activate.ps1
```

```
pip install -e .
python scripts\make_tiny_data.py
python -m northstar.run --config configs\tiny.toml --output-root runs --log-level INFO
```

Szemléltető, nem automatikusan futtatott kimenet; a helyi futtatás után ellenőrizd a tényleges számokat:

```
2026-10-20T09:30:01Z INFO northstar config=/s114_demo/configs/tiny.toml
↪  run_id=20261020T093001Z-7f3a output_root=runs
2026-10-20T09:30:02Z INFO northstar train shape=(400, 5) test shape=(200, 5)
2026-10-20T09:30:03Z INFO northstar model=dummy accuracy=0.4850 roc_auc=0.5000
2026-10-20T09:30:04Z INFO northstar model=logreg accuracy=0.7100 roc_auc=0.7598
2026-10-20T09:30:04Z INFO northstar manifest írva; run kész
```

A futás után a `runs/20261020T093001Z-7f3a/` könyvtár tartalma:

```
config.toml
manifest.json
predictions_dummy.csv
predictions_logreg.csv
run.log
```

29.2.4 Eredmény értelmezése

A dummy modell a tanító pozitív arányt $\hat{p}_{\text{train}}$ konstansként jósolja. A teszt-accuracy kézi kontrollja:

$$\widehat{\text{acc}} = \hat{p}_{\text{train}}\bar{y}_{\text{test}} + (1 - \hat{p}_{\text{train}})(1 - \bar{y}_{\text{test}}),$$

ahol $\bar{y}_{\text{test}}$ a tesztcímke átlaga. Ha például $\hat{p}_{\text{train}} = 0{,}4675$ és $\bar{y}_{\text{test}} = 0{,}49$, akkor $\widehat{\text{acc}} = 0{,}4675 \cdot 0{,}49 + 0{,}5325 \cdot 0{,}51 = 0{,}5297$; a konkrét fájlban a saját arányokkal ellenőrizhető. A dummy ROC-AUC konstans valószínűségnél 0,50. A logreg ROC-AUC ennél magasabb kell legyen; ha nem, hiba gyanús.

A shape-invariáns: mindkét `predictions_*.csv` sorszáma 200, tehát egyezik a teszt `shape[0]` értékével; `pred_late` pontosan `(prob_late >= 0.5)`.

A konfiguráció hash-ellenőrzése Pythonból:

```
from pathlib import Path
from northstar.config import sha256_file

print(sha256_file("configs/tiny.toml"))
```

A kapott értéknek egyeznie kell a manifest `config.sha256` mezőjével.

29.2.5 Feladatok

1. **Értelmezés.** A manifestben ez áll: `"models": {"dummy": {"roc_auc": 0.5, "positive_rate_test": 0.49, "pred_positive_rate": 0.4675}}`. Miért nem egyezik a dummy `pred_positive_rate` a teszt pozitív arányával, és miért éppen 0,4675 lehet ez az érték? Mi következik a dummy ROC-AUC 0,5 értékéből?

2. **Megvalósítás.** Egészítsd ki a `run.py`-t úgy, hogy a modellmetrikákat külön `metrics.json` fájlba is kiírja a futási könyvtárba, és a manifestbe bekerüljön a `metrics.sha256` mező. A `metrics.json` tartalma legyen `{"models": results, "seed": seed}`. A kiírást atomi művelettel végezd.

3. **Hibadiagnózis.** A társad kétszer futtatja ugyanazt a parancsot, másodszor `--run-id 20260101T000000Z-0000` értékkel, miután az első futás már létrehozta ezt a könyvtárat. Mit ír ki a program, milyen visszatérési kóddal áll meg, és miért nem írd felül a régi manifestet? Hogyan kellene helyesen újra futtatni?

4. **Megváltozott feltétel.** A `make_tiny_data.py` `RNG_SEED` értékét 7-ről 8-ra változtattad, majd újra legeneráltad a CSV-ket, és újra futtattad a modellt. A modell ROC-AUC-ja romlott. A manifest melyik mezője bizonyítja, hogy a bemeneti adat változott? Milyen további teendő kell ahhoz, hogy a régi és az új futást összehasonlíthatóan tárold?

29.2.6 Megoldások és indoklás

1. A `DummyClassifier(strategy="prior")` a **tanító** halmaz pozitív arányát tanulja meg, és minden teszt-sorra ugyanazt a p-t adja. Ezért a teszt-predikciók átlaga a tanító-arány, 0,4675, ami általában nem egyezik a teszt saját pozitív arányával, 0,49-del. A konstans valószínűség nem rangsorolja a tesztpéldányokat, ezért a ROC-AUC elvárt értéke 0,5; ez nem modellhiba, hanem a baseline értelme.

2. Egy lehetséges megoldás a manifest írása előtt:

```
metrics_path = run_dir / "metrics.json"
metrics_text = json.dumps(
    {"models": results, "seed": seed}, indent=2, sort_keys=True
) + "\n"
atomic_write_str(metrics_path, metrics_text)
manifest["metrics"] = {
    "path": "metrics.json",
    "sha256": sha256_file(metrics_path),
}
```

Az ellenőrző invariáns: `sha256_file(run_dir / "metrics.json")` egyezik a manifest `metrics.sha256` értékével, és a fájl egyetlen JSON-objektum, amelynek `models` kulcs alatt pontosan a két modell neve szerepel.

3. A második futtatás a `run_dir.mkdir(exist_ok=False)` hívásnál `FileExistsError` kivételt kap, a program `ERROR northstar run könyvtár már létezik: ...` üzenetet ír, és 3-as kóddal tér vissza. Nem írd felül a régi manifestet, mert az immutable output szerződés tiltja a futási könyvtár újraírását. Helyes megoldás új `--output-root` vagy új `--run-id`; ha a régi futás hibás, a teljes `runs/<régi-id>` könyvtárat kézzel törölni, majd új futást indítani.

4. A manifest `data.train.sha256` és `data.test.sha256` mezője megváltozik, mert a CSV byte-tartalma más. A régi és új futás akkor marad összehasonlítható, ha mindkét `runs/<run_id>` könyvtárat megőrzöd: mindkettő tartalmazza a saját konfigurációmásolatát, manifestjét és predikcióit. A modellromlás okának eldöntéséhez a manifest hash-ek mellett a `run.log` fitness-üzeneteit is össze kell vetni.

29.2.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: **„elég a seed, azzal reprodukálható a futás."** Ellenpélda: két futtatás ugyanazzal a `seed = 20261020` értékkel történik. Közben valaki a `configs/tiny.toml`-ben a `C = 0.5` értéket `C = 5.0`-ra módosította, és a `make_tiny_data.py` futtatása új CSV-ket írt. A két manifest `config.sha256` és `data.train.sha256` mezője eltér, tehát a két futás nem ugyanaz a kísérlet, hiába azonos a seed. A seed csak a véletlenszám-generátor bemenete; a konfiguráció, az adat, a kódverzió és a környezet rögzítése nélkül nem elégséges. A manifest hash-ekkel és a konfigurációmásolattal ezt a hiányt pótolja.

29.2.8 Önellenőrzés

1. Miért írjuk a manifestet utoljára, és miért atomi írással?
2. Milyen logging szintet választasz a modell AUC-jának kiírásához, és miért nem `print`-et?
3. Mi a különbség a `--run-id` és az automatikusan generált run id között a felülírás megelőzésében?

29.2.9 Rövid összefoglaló

A konfiguráció és futási napló a kutatói kódot visszakereshető kísérletté alakítja. A TOML beépített, típusos konfigurációt ad; az argparse csak a bemeneti határt kezeli. A logging szintek a zaj és a hiba közötti különbséget rögzítik, a run id egyedi futáskönyvtárat jelöl, a manifest pedig a konfiguráció, az adat és a környezet hash-eit tárolja. Az immutable output a `mkdir(exist_ok=False)` és az atomi `os.replace` miatt nem enged csendes felülírást. A laborban egyetlen paranccsal két modell, a dummy és a logreg fut le, külön predikciós CSV-vel és manifesttel. A következő lépés a 29.3. alfejezet: a manifestben rögzített bemenetekhez és predikciókhoz adatteszteket és numerikus teszteket kapcsolunk.

29.3 Adatteszt és numerikus teszt

> **Belépő · szint: haladó · idő: 90 perc · eszköz: pytest, uv, Git**
>
> Előfeltétel: 29.2. alfejezet · Célkészség: Adatteszt és numerikus teszt · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

29.3.1 Helyzet és induló becslés

A 29.2. alfejezetben a kutatási notebookból már kiemeltünk egy konfigurációval és futásazonosítóval ellátott szkriptet. Most a kérdés az, hogyan ellenőrizzük a `features.py` számításait anélkül, hogy a notebook celláit másolnánk le. A labor szerződése három tesztet kér: bevétel-megmaradási invariáns (revenue conservation invariant), időhelyes feature-ellenőrzés és ismert kis modellpredikció.

Az induló példa legyen két kicsi rendelés. Az `ORD-1` rendeléshez két tétel tartozik, `line_total_minor = 300` és `450`, tehát a szerződés szerint `basket_minor = 750`. Az `ORD-2` esetén egyetlen tétel van `999` cent értékben. A `shipping_fee_minor` legyen `500` cent mindkét rendelésnél; ez nem része a kosárnak, és a tételszintű join után sem szabad megduplázódnia. Az elvárt teszt tehát: a rendelésenkénti tételösszeg pontosan `{ORD-1: 750, ORD-2: 999}`, a

szállítási díj rendelésenként egyszer szerepel, és a modell `y = 2 + 3x` összefüggésből `14`-et jósol `x = 4` esetén.

A fejezet projektgyökere továbbra is `nslab/`, a forrás a `src/northstar/features.py`, a tesztek pedig a `tests/` mappába kerülnek. A `pytest` a 29.2. alfejezetben már bevezetett `uv run --locked` paranccsal futtatható.

29.3.2 Séma, invariáns és dimenzió

A sémateszt (schema test) nem az üzleti tartalmat, hanem a bemenet alakját ellenőrzi: mely oszlopok léteznek, milyen típusúak, és mely kulcsok egyediek. Egy adattábla esetén ez gyakran azt jelenti, hogy az `order_id` nem lehet üres, és nem ismétlődhet a rendelés-táblában. A dimenzió fogalmát itt shape-ként használjuk: ha a bemeneti `order_items` tábla shape-je (m, k), akkor egy rendelésenkénti összegzés kimenete $(n,)$ alakú, ahol $n \le m$ az egyedi rendelések száma.

A join invariáns (join invariant) olyan mennyiségi törvény, amelynek a join előtt és után is teljesülnie kell. A Northstar adatszerződés kimondja:

- `line_total_minor = quantity * unit_price_minor - discount_minor`,
- egy rendelés `basket_minor` értéke a hozzá tartozó `line_total_minor` értékek összege.

A szállítási díj kivétel: `shipping_fee_minor` rendelésenként egyszer létezik, és nem adódik hozzá a kosárhoz. Ha az `order_items` táblát bal oldali joinnal kötjük az `orders` táblához, akkor minden tételsor megkapja a rendelés szállítási díját. A

$$\sum_{\text{tételsor}} \text{shipping_fee_minor}$$

összeg ilyenkor nagyobb lehet, mint a rendelésszintű

$$\sum_{\text{rendelés}} \text{shipping_fee_minor},$$

amit a tesztnek tiltania kell. A megfigyelési egység itt a rendelés, a tételszintű join ezt nem változtathatja meg.

A séma szerinti minimális oszlop-ellenőrzés:

```
REQUIRED_ORDERS_COLUMNS = {
    "order_id": "object",
    "basket_minor": "int64",
    "shipping_fee_minor": "int64",
}

def test_orders_schema(small_orders: pd.DataFrame) -> None:
    assert set(REQUIRED_ORDERS_COLUMNS).issubset(small_orders.columns)
    for column, dtype in REQUIRED_ORDERS_COLUMNS.items():
        assert small_orders[column].dtype == dtype
    assert small_orders["order_id"].is_unique
    assert small_orders["order_id"].notna().all()
```

Ez a teszt semmit nem mond arról, hogy a `basket_minor` helyes-e. Például egy negatív kosárérték is átmegy rajta, ha a dtype `int64`. Ezért kell külön invariáns-teszt.

29.3.3 Tolerancia, nem bitazonosság

A pénz a Northstar adatokban egész minor unitban van, tehát a kosár-összehasonlítás egzakt lehet. A lebegőpontos (floating point) számoknál viszont a bitazonosság rossz elvárás: a `0.1 + 0.2` nem egyenlő `0.3`-mal a gépi ábrázolás miatt. A helyes ellenőrzés abszolút és relatív tűréshatárral (tolerance) történik:

$$|\text{actual} - \text{expected}| \le atol + rtol \cdot |\text{expected}|.$$

Itt minden mennyiség dimenziótlan a képlet szintjén, de a gyakorlatban a mértékegység számít: ha az érték centben van, az `atol=0.01` fél centet jelenthet, ami pénzügyi ellenőrzésnél gyakran elfogadhatatlan. Az 5.3. alfejezetben bevezetett stabil számítás elve ide is átjön: minor unit egésznél ne engedjünk lebegőpontos kerekítést, modellpredikciónál viszont a `pytest.approx` vagy a `math.isclose` a helyes eszköz.

Példa a kettő közötti különbségre:

```
import math

expected = 0.3
actual = 0.1 + 0.2
assert math.isclose(actual, expected, rel_tol=1e-12, abs_tol=1e-12)

# Ugyanez a tűrés elfogadja a várható numerikus zajt, de egy 1%-os hibát nem:
assert math.isclose(300.0 + 3e-10, 300.0, rel_tol=1e-9, abs_tol=1e-9)
assert not math.isclose(300.0 * 1.01, 300.0, rel_tol=1e-9, abs_tol=1e-9)
```

A `rel_tol=1e-15` csak ebben a konkrét esetben működik: itt a hiba $5{,}55 \cdot 10^{-17}$ és az érték 0,3 nagyságrendű. A float64 gépi epszilonja $2{,}22 \cdot 10^{-16}$, a float32-é $1{,}19 \cdot 10^{-7}$; egy hosszú összegzés, egy rosszul kondicionált mátrix vagy egy iteratív optimalizáló ennél sokkal nagyobb eltérést ad. A toleranciát ezért a nagyságrend, a dtype, a kondicionáltság és az algoritmus alapján kell megválasztani:

Eset	Javasolt kiindulás	Indok
float64, $O(1)$ értékek, néhány művelet	`rtol=1e-12, atol=1e-12`	a kerekítési hiba néhányszor 10^{-16}
float64, hosszú összegzés vagy normálegyenlet	`rtol=1e-9, atol=1e-9`	a műveletszám és a kioltás növeli a hibát
float32 tanítás, modellpredikció	`rtol=1e-5, atol=1e-6`	a float32 epszilonja 10^{-7} nagyságrendű
nullához közeli várható érték	az `atol` dominál	a relatív hiba nullánál értelmezhetetlen
rosszul kondicionált rendszer ($\kappa \approx 10^6$)	`rtol=1e-8` és dokumentált κ	a hiba $\kappa\varepsilon$ szerint skálázódik
egész minor unit	egzakt `==`	nincs lebegőpontos kerekítés
iteratív optimalizáló	előre rögzített metrika-tolerancia	a paramétervektor futásonként eltérhet

A modellpredikciós tesztnél a lineáris regresszió lebegőpontos műveleteket végez, ott `np.testing.assert_allclose(pred, 14.0, atol=1e-9)` indokolt. Ez a tűrés numerikus közelítési hibát kezel, nem üzleti hibahatárt. A `numpy.testing.assert_allclose` alapértéke `rtol=1e-7, atol=0`, a `pytest.approx` alapértéke `rel=1e-6, abs=1e-12`; ezeket nem szabad

örökölni, hanem a fenti táblázat szerint, a teszt docstringjében indokolni kell. A kondicionáltságot például a `numpy.linalg.cond` adja meg; ha κ nagy, a teszt a numerikus hibát tolerálja, de a modell vagy az adat hibáját nem.

29.3.4 Fixture, property és regression test

A `pytest` fixture újrafelhasználható, kis bemenetet ad a teszteknek. A laborhoz két rendelésből és három tételsorból álló DataFrame elegendő:

```
import pandas as pd
import pytest

@pytest.fixture
def small_orders() -> tuple[pd.DataFrame, pd.DataFrame]:
    orders = pd.DataFrame(
        {
            "order_id": ["ORD-1", "ORD-2"],
            "basket_minor": [750, 999],
            "shipping_fee_minor": [500, 500],
            "currency": ["EUR", "EUR"],
        }
    )
    items = pd.DataFrame(
        {
            "order_id": ["ORD-1", "ORD-1", "ORD-2"],
            "line_id": [1, 2, 1],
            "line_total_minor": [300, 450, 999],
        }
    )
    return orders, items
```

A property teszt (property test) itt nem véletlenített property-based könyvtárat jelent, hanem matematikai tulajdonság ellenőrzését több kis bemeneten. A `compute_basket_totals` függvényre ilyen tulajdonság, hogy a kimenete minden rendelésre pontosan egy értéket ad, és a kimenet összege megegyezik a bemeneti tételösszegek összegével:

```
def test_basket_total_property(small_orders: tuple[pd.DataFrame, pd.DataFrame]) -> None:
    orders, items = small_orders
    result = compute_basket_totals(items)
    # minden rendelés egyszer szerepel
    assert result.index.is_unique
    assert len(result) == orders["order_id"].nunique()
    # a tételösszegek total megmarad
    assert result.sum() == items["line_total_minor"].sum()
```

A regression test (regression test) ismert bemenetre rögzített, előre kiszámolt kimenetet őriz. Célja, hogy elkapja a kód későbbi, nem szándékos változását. Ha a `compute_basket_totals` korábbi verziója véletlenül a `quantity`-t összegezné a `line_total_minor` helyett, a szigorúbb szerződésteszt elbukna. A kézzel rögzített érték ezért a szerződésből származik, nem a függvény saját kódjából.

29.3.5 Teljes minimális munkapélda

A laborban a `features.py` egyetlen tiszta függvényt tartalmaz:

```
# src/northstar/features.py
import pandas as pd

def compute_basket_totals(order_items: pd.DataFrame) -> pd.Series:
    """Rendelésenkénti line_total_minor összeg, nem tartalmazza a szállítást."""
    return order_items.groupby("order_id")["line_total_minor"].sum()
```

A `tests/test_contracts.py` első fele a bevételmegmaradást és a join invariánst ellenőrzi:

```
from northstar.features import compute_basket_totals


def test_basket_equals_sum_of_line_totals(
    small_orders: tuple[pd.DataFrame, pd.DataFrame],
) -> None:
    orders, items = small_orders
    result = compute_basket_totals(items)
    expected = orders.set_index("order_id")["basket_minor"]
    pd.testing.assert_series_equal(result, expected, check_names=False)


def test_shipping_is_not_duplicated_by_item_join(
    small_orders: tuple[pd.DataFrame, pd.DataFrame],
) -> None:
    orders, items = small_orders
    joined = items.merge(
        orders[["order_id", "shipping_fee_minor"]],
        on="order_id",
        how="left",
    )
    # Egy rendelés egy tételsoronként látja a shippinget, de a konszolidált
    # rendelésenkénti összegnek a distinct értékek összegét kell adnia.
    per_order_shipping = joined.groupby("order_id")["shipping_fee_minor"].first()
    assert per_order_shipping.sum() == orders["shipping_fee_minor"].sum()
```

Az időhelyes feature-teszt egy egyszerű cutoff-szűrőt ellenőriz. A tanulóhalmaz definíciója a Northstar szerződésből: `order_ts < cutoff` és `label_available_at < cutoff`. A teszt adatai közé szándékosan teszünk egy későn érkező címkét, amelynek a tanítóhalmazból ki kell maradnia:

```
def test_label_availability_split_excludes_late_label() -> None:
    orders = pd.DataFrame(
        {
            "order_id": ["A", "B", "C"],
            "order_ts": pd.to_datetime(
                [
                    "2025-04-28T10:00:00+00:00",
                    "2025-04-29T10:00:00+00:00",
                    "2025-05-02T10:00:00+00:00",
                ]
            ),
            "label_available_at": pd.to_datetime(
                [
                    "2025-04-30T10:00:00+00:00",
                    "2025-05-05T10:00:00+00:00",
                    "2025-05-06T10:00:00+00:00",
                ]
            ),
        }
    )
    cutoff = pd.Timestamp("2025-05-01T00:00:00+00:00")

    train, border, test = split_by_label_availability(orders, cutoff)
```

```
    assert list(train["order_id"]) == ["A"]
    assert list(border["order_id"]) == ["B"]
    assert list(test["order_id"]) == ["C"]
```

A `split_by_label_availability` a 29.2. alfejezet splitmanifestjének kiszolgáló függvénye; itt csak annyit kell tudnunk, hogy a három halmaz diszjunkt, és a határsávba az a `B` sor kerül, amelynek rendelési ideje cutoff előtti, de címkéje cutoff után érkezett meg.

A modellpredikciós teszt ismert, mesterséges összefüggést használ:

```
import numpy as np
from sklearn.linear_model import LinearRegression

def test_known_linear_model_prediction() -> None:
    x = np.array([[0.0], [1.0], [2.0], [3.0]])
    y = np.array([2.0, 5.0, 8.0, 11.0])
    model = LinearRegression().fit(x, y)

    pred = model.predict([[4.0]])[0]
    assert pred == pytest.approx(14.0, abs=1e-9)
    assert model.intercept_ == pytest.approx(2.0, abs=1e-9)
    assert model.coef_[0] == pytest.approx(3.0, abs=1e-9)
```

A `y` értékek a $2 + 3x$ képletből jönnek, ezért az elvárt paraméterek ismertek. A teszt azt igazolja, hogy a modellillesztés numerikusan helyesen számol; nem bizonyítja, hogy a modell jó üzleti előrejelző a Northstar rendeléseken.

Futtatás a projektgyökérből:

```
# macOS/Linux; Windows PowerShellben a sor eleji uv ugyanúgy működik
uv run --locked pytest tests/test_contracts.py -q
```

Az itt közölt kimenet szemléltető; tényleges futtatási jegyzőkönyv csak a labor lokális eredmény-jegyzékéből nevezhető mértnek.

29.3.6 Eredmény értelmezése és önellenőrzés

A `test_basket_equals_sum_of_line_totals` zöld eredménye azt jelenti, hogy a `compute_basket_totals` az adatszerződés kosárdefinícióját tartja. A `test_shipping_is_not_duplicated_by_item_join` a leggyakoribb one-to-many join hibát fogja meg: ha a shippinget tételsoronként adnánk össze, akkor az `ORD-1` esetén `500 + 500 = 1000` centet kapnánk a helyes `500` helyett. A modellpredikciós teszt a floating point tűrést ellenőrzi, a `1e-9` abszolút hiba numerikus kerekítésből fakad, nem modellhibából. A séma-teszt pedig a dtype-ok és az egyedi kulcs feltételeit védi.

Önellenőrző kérdések:

1. A tesztem a szerződést ellenőrzi, vagy a saját függvényem belső kódját másolja?
2. A tűrés mértékegysége illik az adathoz?
3. Az időtesztben a címke rendelkezésre állása is feltétel, nem csak a rendelési idő?
4. A zöld teszt után milyen statisztikai állítást nem tehetek meg?

A numerikus tesztekben az abszolút és relatív tolerancia eltérő problémát old meg; 0 közeli várható értékhez abszolút tolerancia kell. Adatsématesztnél szükséges a hiányzó és duplikált kulcsok tesztje.

29.3.7 Feladatok

1. **Értelmezés.** A fenti `test_orders_schema` teszt zöld egy olyan táblára, ahol a `basket_minor` minden sora negatív. Mondd meg, miért nem hibás a teszt, és milyen új invariáns-tesztet adnál hozzá a pénzösszegek nemnegativitásának ellenőrzésére.

2. **Implementálás.** Írj `test_basket_totals_match_orders` tesztet a `small_orders` fixture alapján, amely a `compute_basket_totals` kimenetét a `small_orders` első DataFrame-jének `basket_minor` oszlopához hasonlítja. A teszt ne használja a fixture-tuple indexelésén kívül a `groupby`-t.

3. **Hibadiagnózis.** A kolléga tesztje így néz ki:

```
def test_model_predicts():
    x = np.array([[0.0], [1.0]])
    y = np.array([0.0, 1.0])
    m = LinearRegression().fit(x, y)
    assert m.predict([[2.0]])[0] == 2.0
```

A futtatás `assert 2.0 == 1.9999999999999998` hibát ad. Magyarázd el, mi a hiba, és add meg a helyes tesztet.

4. **Megváltozott feltétel.** A labor adataihoz több pénznem kerül: `EUR` és `USD` sorok is vannak, de pénznemkonverzió nincs. A bevételmegmaradási invariáns mostantól csak pénznemen belül igaz. Írj `groupby("currency")`-alapú tesztet, amely mindkét oldalt pénznemenként hasonlítja össze.

29.3.8 Megoldások és indoklás

1. A séma-teszt csak a típust ellenőrzi; a `-12` érték `int64` dtype-pal rendelkezik, tehát átmegy. A hiányzó üzleti feltétel a nemnegativitás. A javítás:

```
def test_money_columns_are_nonnegative(small_orders):
    orders, items = small_orders
    for frame in (orders, items):
        for column in ("basket_minor", "shipping_fee_minor", "line_total_minor"):
            if column in frame:
                assert (frame[column] >= 0).all()
```

Indoklás: a séma a dtype-ot, az invariáns a tartalmi tartományt védi.

2. Egy lehetséges megoldás:

```
def test_basket_totals_match_orders(small_orders):
    orders, items = small_orders
    totals = compute_basket_totals(items)
    expected = orders.set_index("order_id")["basket_minor"]
    pd.testing.assert_series_equal(totals, expected, check_names=False)
```

Az `assert_series_equal` a sorrendre érzékeny; ha a sorrend nem garantált, rendezni kell. Itt a `small_orders` fixture rendezett.

3. A hiba oka a lebegőpontos kerekítés. A `==` bitazonosságot vár, a lineáris regresszió belső műveletei azonban lebegőpontos hibát adnak. Helyes változat:

```
assert m.predict([[2.0]])[0] == pytest.approx(2.0, abs=1e-9)
```

A `pytest.approx` az abszolút tűréshatárt alkalmazza, és a teszt így a numerikus közelítési hibát tolerálja.

4. Pénznemenkénti invariáns-teszt:

```
def test_basket_invariant_per_currency(
    small_orders: tuple[pd.DataFrame, pd.DataFrame],
) -> None:
    orders, items = small_orders
    items_with_currency = items.merge(
        orders[["order_id", "currency"]], on="order_id", how="left"
    )
    by_currency_items = items_with_currency.groupby("currency")[
        "line_total_minor"
    ].sum()
    by_currency_orders = orders.groupby("currency")["basket_minor"].sum()

    pd.testing.assert_series_equal(by_currency_items, by_currency_orders)
```

Indoklás: a pénznem dimenziót választó oszlop, ezért a csoportosított összegeknek kell egyezniük. Konverzió nélkül a teljes, pénznemet figyelmen kívül hagyó összeg megtévesztő lehet.

29.3.9 Gyakori tévedés ellenpéldával

A leggyakoribb félreértés: *a zöld unit teszt bizonyítja a modell statisztikai érvényességét.* A fenti `test_known_linear_model_prediction` csak egy mesterséges, determinisztikus összefüggést ellenőriz. Ha a valódi Northstar pipeline szivárgást tartalmaz, például a tesztadat célját használja feature-ként, akkor a pipeline unit tesztjei zöldek lehetnek, mégis a jövőbeli rendeléseken az AUC 0.80 helyett 0.50 közelébe eshet. Ez ugyanaz a tanulság, mint a 25.3. alfejezet feature-szelekciós összefoglalójában: a független kiértékelés és a megfelelő időbeli split nem helyettesíthető kódtesztekkel. A 17.3. alfejezet diagnosztikája sem lesz jobb attól, hogy a modellinterfész tesztje hiba nélkül fut. A unit teszt a szoftver szerződését igazolja, a statisztikai érvényességhez külön értékelési terv kell.

29.3.10 Rövid összefoglaló

A sémateszt a dtype-okat és kulcsokat, a join invariáns a bevételmegmaradást és a szállítási díj egyszeri beszámítását védi. A tolerancia a lebegőpontos közelítést kezeli; minor unit egésznél egzakt összehasonlítás, modellpredikciónál `pytest.approx` indokolt. A fixture kis, ismert bemenetet ad, a property teszt a számítás általános tulajdonságait vizsgálja, a regression test rögzített elvárt értékeket őriz. A zöld teszt a szoftver szerződését igazolja, nem a modell statisztikai érvényességét. A következő alkalmazás a kódminőség és az együttműködő review, ahol ezeket a teszteket a merge előtti ellenőrzésbe illesztjük.

29.4 Kódminőség és együttműködő review

Belépő · szint: haladó · idő: 90 perc · eszköz: pytest, uv, Git

Előfeltétel: 29.3. alfejezet · Célkészség: Kódminőség és együttműködő review · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

29.4.1 Cél és konkrét helyzet

A 29.3. alfejezetben felépített `northstar` modul rendelkezik adattesztekkel, de a `features.py` hibás `account_age_days` függvényt tartalmaz: a `signup_date` timezone-naiv, az `order_ts` viszont UTC-aware, ezért a kivonás futásidőben `TypeError`-t dob. Ebben az alfejezetben nem maga a hibajavítás a fő cél, hanem az, hogyan lehet a javítást review-zhatóvá tenni: kis, áttekinthető diff, a probléma és a validáció rövid leírása, statikus eszközökön átmenő kód és emberi ellenőrzésre alkalmas összefoglaló.

A bemenet a 29.3. alfejezet labor `src/northstar/features.py` fájlja, a megfigyelési egység egy rendeléshez tartozó `order_ts` és `signup_date` pár. Az eredmény a kettő különbsége egész napokban, az elvárt ellenőrzés pedig egy determinisztikus pytest eset és a formai ellenőrzések. A `lab/ADATOK.md` szerint `signup_date` naptári dátum, `order_ts` UTC-időbélyeg; a javításnak ezt a szerződést kell kodifikálnia.

Induló feladat: Mielőtt a konkrét javítást látná, becsülje meg, mit tapasztal a felhasználó a hibás függvény futtatásakor, és miért nem elég a linter a hiba elkerüléséhez. Becslése után dolgozza végig a minimális példát.

29.4.2 Három automatikus réteg és egy emberi réteg

A kódminőségi eszközöket négy rétegként érdemes szétválasztani: formatter, linter, type checker és code review. Az első három automatikus, az utolsó emberi.

- **Formatter (automatikus formázó):** a kód stílusát normálja, például behúzást, idézőjelet, sorhosszt. A Ruff formatter ilyen eszköz. A formatter nem ismeri a jelentést; két szemantikailag hibás változat ugyanúgy átmehet rajta.
- **Linter (statikus hibakereső):** statikus kódhibákat jelez: nem használt import, nem definiált név, redefiniált függvény. A Ruff `check` parancsa ilyen. A linter nem tudja, hogy a `distance_km` rossz mértékegységet kap-e; azt az adatvalidáció és a teszt ellenőrzi.
- **Type checker (típusellenőrző):** a típusannotációk és az értékhasználat egyezését vizsgálja statikusan. A példában a mypy külön eszköz, mert a Ruff maga nem végez teljes típusellenőrzést. A típusellenőrző sem garantálja a futásidőt: a timezone-konzisztencia a pandasnál futásban derül ki, a mypy nem feltétlenül jelzi.
- **Code review (kódátvizsgálás):** emberi ellenőrzés, amely a céldefiníciót, a tudományos érvényességet és a kockázatot kérdezi. A review nem helyettesíthető a három automatikus réteggel.

Egy kézi ellenőrzés az automatikus eszközök értelmezésére:

```
# A linter F401-et ad, mert a pd nincs használva.
import pandas as pd

def late_target(delivery_days):
    # A linter zöld, pedig a delivery_days a valós címkéből képzett,
    # és prediktorként használva leakage-t okozna.
    return delivery_days > 7
```

A formatter és linter ezen a kódon nem jelez, a típusellenőrző sem feltétlenül. A függvény mégis tudományosan hibás, mert utólag ismert információt használ. A linter csak a `pd` import hiányát jelzi, a leakage-t nem.

29.4.3 Teljes minimális munkapélda: feature hibajavítás és review

A kezdőlabor `lab/` könyvtárában dolgozunk. A projektstruktúra a 29.3. alfejezetből ismert; az adatfájlokat a lab/ADATOK.md rögzíti. A példa a `src/northstar/features.py` hibás függvényét javítja.

A hibás kód:

```
# src/northstar/features.py
import pandas as pd

def account_age_days(order_ts: pd.Timestamp, signup_date: pd.Timestamp) -> int:
    """Return days between signup date and order timestamp."""
    return (order_ts - signup_date).days
```

A formatter és a linter itt nem jelez, a mypy sem feltétlenül; a pytest viszont `TypeError`-t ad, mert a naive `signup_date` és tz-aware `order_ts` kivonása a pandasban futásidőben hibás.

A javított kód:

```
# src/northstar/features.py
from __future__ import annotations

import pandas as pd


def account_age_days(order_ts: pd.Timestamp, signup_date: pd.Timestamp) -> int:
    """Return whole days between signup date and order timestamp.

    The contract in lab/ADATOK.md says signup_date is a calendar date and
    order_ts is UTC. The signup date is therefore interpreted as UTC midnight.
    """
    signup_utc = pd.Timestamp(signup_date).tz_localize("UTC")
    delta = order_ts - signup_utc
    return int(delta.days)
```

A hozzá tartozó teszt:

```
# tests/test_features.py
from pandas import Timestamp

from northstar.features import account_age_days


def test_account_age_days_interprets_signup_as_utc_midnight():
    result = account_age_days(
        order_ts=Timestamp("2025-02-15T23:30:00+00:00"),
        signup_date=Timestamp("2025-01-01"),
    )
    assert result == 45
```

A parancsok a `lab/` munkakönyvtárból futtathatók; Linux shell és Windows PowerShell esetén is a `uv` a projekt `pyproject.toml`-ját keresi:

```
cd lab
uv sync --group dev
uv run pytest tests/test_features.py
uv run ruff check src tests
uv run ruff format --check src tests
uv run mypy src
```

A `uv sync --group dev` a `pyproject.toml` verziótartományait oldja fel; a felhasználó által létrehozott `uv.lock` után a futtatás `--locked` opcióval őrizhető. A fenti kimenetek ezért szemléltető, elméletileg várt eredmények, nem a jelen szöveg által futtatott mérési jegyzék. A várt viselkedés: a pytest zöld, a `ruff check` hibátlan, a formatter stílusellenőrzése sikeres, a mypy az annotációkat elfogadja.

A `pyproject.toml` minőségi eszközökért felelős részlete:

```
[project]
name = "northstar-lab"
version = "0.1.0"
requires-python = ">=3.12"

[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.5", "mypy>=1.10"]

[tool.ruff]
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]

[tool.mypy]
python_version = "3.12"
```

A `select` listában az `E` és `F` a hibaszabályokat, az `I` az importrendezést, az `UP` a Python-verziófüggő egyszerűsítéseket jelenti; a valós konfiguráció a labor saját `lab/pyproject.toml`-ja szerint alakul.

A review-zható változtatás leírója:

```
Problem: account_age_days raised TypeError when order_ts was UTC-aware.
Cause: signup_date was a naive calendar date; pandas refuses aware-naive subtraction.
Fix: interpret signup_date as UTC midnight and keep order_ts unchanged.
Validation: tests/test_features.py::test_account_age_days_interprets_signup_as_utc_midnight
  pytest, ruff check, ruff format --check, mypy src.
Manual check: Timestamp("2025-02-15T23:30+00:00") - Timestamp("2025-01-01 00:00+00:00")
  equals 45 days and 23:30; .days returns 45.
Risk: the UTC midnight interpretation is an explicit change; if the business team
  uses local midnight, the feature is off by the offset. The reviewer must confirm.
```

A diff:

```
@@ -1,8 +1,16 @@
 import pandas as pd

 def account_age_days(order_ts: pd.Timestamp, signup_date: pd.Timestamp) -> int:
-    return (order_ts - signup_date).days
+    signup_utc = pd.Timestamp(signup_date).tz_localize("UTC")
+    delta = order_ts - signup_utc
+    return int(delta.days)
```

A formatter a `from __future__ import annotations` és az import sorrendjét is rendezi; a linter a hiányzó sort jelzi, ha elfelejtik.

29.4.4 Pre-commit és minimális CI

A lokális `pre-commit` hook a commit pillanatában akadályozza a nyilvánvaló hibákat; a központi CI a pull request szintjén futtatja ugyanezeket, és a review előtt ad automatikus bizonyítékot.

A `.pre-commit-config.yaml` példa:

```yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.5.0  # example, pin to the reviewed version
    hooks:
      - id: ruff
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.10.0  # example, pin to the reviewed version
    hooks:
      - id: mypy
        args: [src]
        pass_filenames: false
```

A `rev` értékek itt elrendezést mutatnak; a projektnek a saját, ellenőrzött verziójához kell igazítania őket. A telepítés a `uv run pre-commit install` parancs után a `git commit` alkalmával futtatja a hookokat. A pre-commit nem helyettesíti a CI-t: a hookok kikerülhetők, a CI viszont központilag naplózza a futást.

A minimális CI workflow:

```yaml
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v4
      - run: uv sync --locked --group dev
      - run: uv run pytest
      - run: uv run ruff check src tests
      - run: uv run ruff format --check src tests
      - run: uv run mypy src
```

A `--locked` csak akkor futtatható, ha az `uv.lock` a repóban van; a kezdőlaborban ezt a felhasználó hozza létre. Az action verziók is ellenőrzést igényelnek a projekt adott időponti függőségi politikája szerint.

29.4.5 Eredmény értelmezése

A függvény a determinisztikus bemenetre `45` napot ad. Kézi kontroll: `2025-02-15 23:30 UTC - 2025-01-01 00:00 UTC = 45 nap 23:30`, a `Timedelta.days` egész napokra levág, így `45`. A javítás megszünteti a `TypeError`-t; az automatikus eszközök formailag zölden haladnak. A teszt invariáns: a bemeneti `signup_date` mindig `Timestamp` naptári dátum, az `order_ts` mindig tz-aware, az eredmény nemnegatív egész. A függvény skalárt ad vissza, ezért DataFrame-shape nem változik; a dtype `int`. A statisztikai értelmezés itt nincs, mert nem vezetünk be új matematikát; a feature-re ható tudományos döntést a review hagyja jóvá.

Statikus típusellenőrzés nem bizonyítja a modellezési vagy statisztikai helyességet; a review ellenőrizze az estimandot, az időhelyes feature-öket és a splitet is.

29.4.6 Statisztikai review-kapu

A kódszintű review mellé külön **statisztikai review-kapu** kell, mert egy zöld lint- és egységteszt mellett is átmehet egy jövőbeli feature-t használó vagy rossz estimandot mérő változtatás. A kapu öt területe:

Terület	Mit kérdez	Tipikus bukás
Adatséma	Megvannak-e a kötelező oszlopok, a dtype-ok, az egyedi kulcsok és a megengedett tartományok?	hiányzó oszlop, negatív pénzösszeg, duplikált kulcs
Időbeli címkeelérhetőség	Minden predictor a döntés pillanatában ismert-e?	`delivery_days`, `actual_delivery_at`, `label_available_at` mint feature
Leakage	Átfed-e a tanító és teszt, a preprocessing csak a tanítón illeszkedik-e?	azonos `order_id` mindkét oldalon, teljes adatra fit-elt scaler
Kalibráció	A valószínűség kalibrált-e, és a döntési küszöb a költséghez illik-e?	kalibrálatlan `predict_proba` kockázatként használva
Predikciós cél	Az estimand, a populáció, az időhorizont és a metrika egyezik-e?	AUC-t jelentünk, miközben a döntés fix küszöböt igényel

A kapu sablonja:

```
[ ] Adatséma: kötelező oszlopok, dtype-ok, kulcsok, tartományok, hiányzó értékek
[ ] Időbeli címkeelérhetőség: feature.available_at <= decision_time minden predictorra
[ ] Leakage: train/test azonosítók diszjunktak; preprocessing fit csak train; tiltott oszlopok
[ ] Kalibráció: Brier/log loss, kalibrációs meredekség és tengelymetszet, döntési küszöb
[ ] Predikciós cél: megfigyelési egység, populáció, időhorizont, költségmátrix, metrika
[ ] Minden állítás mellett a split és a konfiguráció hash-e szerepel
```

Programozott ellenőrzés:

```
FORBIDDEN = {"delivery_days", "actual_delivery_at", "label_available_at", "target_late"}
REQUIRED_COLUMNS = {"order_id", "order_ts", "distance_km", "priority_flag", "basket_minor"}
TIME_ORDER = {"signup_date": 0, "order_ts": 1, "actual_delivery_at": 2, "label_available_at": 3}

def statistical_review(change):
    errors = []
    missing = REQUIRED_COLUMNS - set(change.get("columns", []))
    if missing:
        errors.append(f"schema: hiányzó oszlop {sorted(missing)}")
    available = change.get("available_at", {})
    decision_time = change.get("decision_time", "order_ts")
    for feature in change.get("features", []):
        if feature in FORBIDDEN:
            errors.append(f"leakage: tiltott feature {feature}")
        elif feature not in available:
            errors.append(f"temporal: {feature} elérhetősége ismeretlen")
        elif TIME_ORDER.get(available[feature], 99) > TIME_ORDER.get(decision_time, 99):
            errors.append(f"leakage: {feature} a döntés után ismert")
    if set(change.get("train_ids", [])) & set(change.get("test_ids", [])):
        errors.append("leakage: train/test azonosítóátfedés")
    metrics = change.get("metrics", {})
    for key in ("brier", "calibration_slope"):
        if key not in metrics:
            errors.append(f"calibration: hiányzó {key}")
    if not change.get("estimand"):
        errors.append("target: hiányzó estimand")
    return errors
```

Példa a bukásra: a kód hozzáadja a `delivery_days` oszlopot predictoraként, a linter és az egységteszt zöld, mert a szintaxis és a shape helyes. A review viszont elutasítja:

```
change = {
    "columns": ["order_id", "order_ts", "distance_km", "priority_flag", "basket_minor"],
    "features": ["distance_km", "delivery_days"],
    "available_at": {"distance_km": "order_ts", "delivery_days": "actual_delivery_at"},
    "decision_time": "order_ts",
    "train_ids": ["o1", "o2"], "test_ids": ["o3"],
    "metrics": {"brier": 0.09, "calibration_slope": 0.95},
    "estimand": "P(target_late=1 | order-time features)",
}
print(statistical_review(change))
# ['leakage: tiltott feature delivery_days']
```

A kapu akkor zöld, ha a lista üres; a review jóváhagyása nem a lint eredményén, hanem ezen a listán és a hozzá tartozó bizonyítékokon alapul. A `tests/test_statistical_review.py` a teljes változatot, a jövőbeli feature-t, a split-átfedést, a hiányzó kalibrációs metrikát és a hiányzó estimandot teszteli.

29.4.7 Feladatok

1. **Értelmezés:** a linter F401 hibát ad, ha az `import pandas as pd` nincs használva. Magyarázd meg, miért nem ellenőrzi a linter, hogy a `target_late` oszlop prediktorként szerepel-e az `orders` táblában. Adj olyan `features.py` részletet, ahol a linter zöld, de a leakage miatt a modell mesterségesen jó lesz.
2. **Megvalósítás:** egészítse ki az `account_age_days` docstringjét azzal, hogy milyen hibát jelez, ha `order_ts` nem tz-aware. Írd be a típusannotációt a `def day_floor(timestamp: pd.Timestamp) -> pd.Timestamp` függvényhez, amely a nap kezdetére kerekít, és futtassa elméletileg a `ruff check` és `mypy` parancsokat.
3. **Hibakeresés:** a `ruff check` a következő hibát adja: `F821 Undefined name 'pd'`. A `features.py` aktuális változatában az `import pandas as pd` sor a fájl végére került. Hogyan észleli a linter a hibát? A mypy is jelezné-e? Indokold, és add meg a javított fájlt.
4. **Transzfer:** a Northstar `segment` értéke `consumer` vagy `business`. A `build_features` függvényben a hibás `is_business = (segment == "biz").astype(int)` kifejezés a `business` kategóriát sosem találja el. A teszt mégis átmegy, mert a fixture-ben `segment="biz"` szerepel. Készíts review-összefoglalót, amely a cél definícióját kéri számon, nem a lintert, és javítsa a tesztet.

29.4.8 Megoldások és indoklás

1. A linter statikus kódhibákat, például definiálatlan nevet vagy felesleges importot jelez. Azt nem tudja, hogy a `delivery_days` a rendelés után ismert, ezért a 25.4. alfejezetben megismert időhelyességi szerződésből kizárt prediktor. Példa:

```
def build_features(orders):
    # A linter zöld, de a delivery_days a valós címkéből képzett.
    orders["late_hint"] = (orders["delivery_days"] > orders["promised_days"]).astype(int)
    return orders[["distance_km", "late_hint"]]
```

A linter nem ismeri az időrendet; a cél helyességét a teszt adatszerződés és az emberi review ellenőrzi.

2. A docstring:

```python
def account_age_days(order_ts: pd.Timestamp, signup_date: pd.Timestamp) -> int:
    """Return whole days between signup date and order timestamp.

    Args:
        order_ts: UTC timestamp of the order.
        signup_date: calendar date of signup, interpreted as UTC midnight.

    Returns:
        Non-negative whole days elapsed.

    Raises:
        ValueError: if order_ts is not timezone-aware.
    """
    if order_ts.tzinfo is None:
        raise ValueError("order_ts must be timezone-aware")
    signup_utc = pd.Timestamp(signup_date).tz_localize("UTC")
    return int((order_ts - signup_utc).days)


def day_floor(timestamp: pd.Timestamp) -> pd.Timestamp:
    """Return the timestamp normalized to the start of its day."""
    return timestamp.normalize()
```

A `ruff check` és `mypy src` elméletileg várt kimenete `All checks passed!` és `Success: no issues found`; a teszt numerikus várt értéke a korábbi `45`, mert a `day_floor` a `2025-02-15 00:00`-t ad.

3. A linter a fájlban a `pd` név használatakor nem talál megfelelő importot a modul szintjén, ezért F821 hibát ad. A mypy ugyanezt jelzi, mert a nevet a statikus környezetben nem látja; a linter viszont kifejezetten a kód szerkezeti hibáira fókuszál, ezért gyakran egyszerűbb réteg. A javítás az `import pandas as pd` mozgatása a fájl elejére, a `from __future__ import annotations` után; a javított fájlban a linter és a mypy is hibátlan.

4. A review összefoglaló:

```
Request: the business_flag feature is always 0 in staging.
Hypothesis: segment == "biz" never matches the ADATOK.md value "business".
Evidence: the unit test uses a fixture segment="biz" that does not follow
  the data contract; therefore the test passes while production fails.
Fix: update the fixture to "business", change the expression to
  (segment == "business"), and add a contract test that compares the
  distinct segment values in customers.csv with {consumer, business}.
```

A cél definíciója az adatszerződésben van, nem a linterben. A javítás után a teszt a valódi kategóriaértéket használja, és a hibás `biz` érték már nem szerepel sem a kódban, sem a fixture-ben.

29.4.9 Gyakori tévedés és ellenpélda

A leggyakoribb tévhit, hogy a „lintermentesség” tudományos helyességet jelent. A formatter, linter és típusellenőrző a kód mechanikai és statikus tulajdonságait ellenőrzi; nem a céldefiníciót, a leakage-t vagy a következtetést. Ellenpélda: a Northstar `starter_experiment.py`-ban ha valaki a `delivery_days` oszlopot prediktorként használja, a `ruff check` és a `mypy` nem panaszkodik, pedig a modell a célváltozóhoz tartozó, a rendelés pillanatában ismeretlen információt használ, ami a 25.4. alfejezet kontraktus megsértése. Ugyanígy a 17.4. alfejezetben tárgyalt odds ratio kódolása lehet lintertiszta; ha a kategória-referenciát rosszul választjuk, az értelmezés téves. A review-nak

ezért mindig a tudományos kérdést is leltárba kell vennie: a cél definícióját, a predictorok időhelyességét, a splitet és a következtetés határait. A linter segít a hibák csökkentésében, de nem helyettesíti az emberi ítéletet.

Önellenőrzés: Ha egy diff csak formázási változtatásokat tartalmaz, a review-nak nem szabad a céldefiníciót jóváhagyni. Ha a diff a `signup_date` értelmezését változtatja, a review-nak a lab/ADATOK.md UTC-éjfél szerződését kell hivatkoznia.

29.4.10 Rövid összefoglaló

A formatter, linter és típusellenőrző három automatikus réteg, amelyek a kód formáját, statikus hibáit és típusegyezését javítják; az üzleti definíciót és a tudományos következtetést nem bizonyítják. A példában az `account_age_days` UTC-éjfél értelmezését teszttel, Ruffal, mypy-val és review-leíróval tettük ellenőrizhetővé. A reviewer a céldefiníciót, az adatszerződést, az időbeli helyességet és a validációt kéri számon; a lintmentesség csak belépési küszöb. A következő alkalmazásban a 30.1. alfejezet a feldolgozási gráfok idempotenciáját és adatminőségi kapuját építi ezekre az automatikus ellenőrzésekre.

30 Adatfolyam, minőség és újrafuttatható pipeline

30.1 Feldolgozási gráf és idempotencia

Belépő · szint: haladó · idő: 90 perc · eszköz: pandas, DuckDB

Előfeltétel: 29.4. alfejezet · Célkészség: Feldolgozási gráf és idempotencia · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

30.1.1 Helyzet: a kezdőprojekt egyetlen notebookból három újraindítható lépésbe

A Northstar kezdőprojektben a feladat rendeléskor becsülni, hogy a kézbesítés meghaladja-e a 7 napot. Az adatfolyam eddig egy notebook celláiban élt: beolvasás, tisztítás, feature-előállítás, modell. Ismételt futtatáskor azonban nem mindig tudtuk, hogy a `orders_raw.csv`-ből készült tisztított tábla hova került, és egy újraindítás néha duplikált sorokat okozott. A cél most az, hogy a feldolgozást három, egymásra épülő lépésre bontsuk, amelyek mindegyike explicit bemeneti és kimeneti fájlhatárral rendelkezik, és a teljes folyamat kétszeri futtatása nem változtat az eredményen.

Az input a mellékelt `lab/raw/orders_raw.csv`, `lab/raw/customers.csv` és `lab/raw/order_items.csv`; a megfigyelési egység a rendelés (`order_id`). A cél egy rendelésszintű feature-tábla előállítása. Előzetes becslés: a tisztított rendeléstábla a kontroll `orders.csv`-vel egyezik (2400 sor), a feature-táblának ugyancsak 2400 sora lesz, és minden lépés kimenete determinisztikus.

30.1.2 Feldolgozási gráf: lépések, rétegek és függőségek

A pipeline-t **irányított aciklikus gráfként (DAG, directed acyclic graph)** modellezzük. A csomópontok a lépések, az élek a függőségek: egy lépés csak akkor futhat, ha minden bemenete már létezik. Aciklikus, mert nincs körkörös hivatkozás; a gráf topologikus sorrendben futtatható.

Három réteget használunk:

- **raw**: a forrásfájlok eredeti, nem módosított formája, kiegészítve séma- és hash-ellenőrzéssel;
- **validated**: a raw-ból tisztított, érvényességi feltételeknek megfelelő táblák;
- **features**: a modellezéshez szükséges, időhelyes rendelésszintű tábla.

A rétegek közötti függőség lineáris:

```
raw/orders_raw.csv    validated/orders.parquet    features/features.parquet
raw/customers.csv
raw/order_items.csv
```

A nyilak egyben a lépések be- és kimeneti határait jelölik. Egy lépés **dependency**-je az a fájl vagy tábla, amelyet olvas; a kimenete az a fájl, amelyet atomikusan kiír. Ha a bemenet változik, a lépést újra kell futtatni; ha nem, a kimenet változatlan marad.

30.1.2.1 Függvénykompozíció és idempotencia

Matematikailag a három lépés három függvény:

$$f_1 : \text{Raw} \to \text{Validated}, \quad f_2 : \text{Validated} \times \text{Customers} \times \text{Items} \to \text{Features},$$

és a teljes folyamat a kompozíció: $f_2(f_1(x), y, z)$. A lépések tiszta függvényként viselkednek: ugyanaz a bemenet ugyanazt a kimenetet adja — ezt a 29.1. alfejezetben már tárgyaltuk.

Az **idempotencia** azt jelenti, hogy egy lépés többszöri alkalmazása nem változtat az eredményen: $T(T(x)) = T(x)$. A mi esetünkben ez úgy teljesül, hogy a lépés a kimeneti fájlt **felülírja**, nem hozzáfűzi. Ha a lépés `INSERT`-et vagy `append`-et használna, akkor $T(x)$ után $T(T(x))$ kétszer annyi sort tartalmazna — ez az „additív adatrombolás” ellenpéldája.

30.1.2.2 Relációs szemcsézettség

Minden lépésnél rögzíteni kell a sorok jelentését. A raw és a validated rétegben egy sor egy rendelés; a `customers.csv`-ben egy sor egy ügyfél; az `order_items.csv`-ben egy sor egy rendelési tétel. A features réteg is rendelésszintű, ezért a tételszintű táblát a join **előtt** aggregálni kell. Ez a 8.4. alfejezetben megismert szabály: ha a tételsorokat közvetlenül joinolnánk a rendelésekhez, a rendelésszintű összegek (pl. `shipping_fee_minor`) többszöröződnének, és a szemcsézettség megváltozna. A lépés kontrollja ezért a sorok száma, az egyedi kulcsok száma és a kontrollösszegek.

30.1.2.3 Atomic output

A **fájl kiírása atomi**, ha a lépés először egy ideiglenes fájlba ír, majd `os.replace` segítségével cseréli a végleges fájlra. Azonos fájlrendszeren a névcsere tipikusan atomikus, így párhuzamos olvasó a régi vagy az új fájlnevet látja. Ez nem azonos a tartós lemezre írás garanciájával (`fsync`), és hálózati fájlrendszeren vagy platformok között további megszorítások lehetnek.

30.1.2.4 Zárolás, bemeneti hash és részleges futás

Az atomi írás önmagában nem elég: két párhuzamos futás ugyanabba a kimeneti könyvtárba írhat, és a lassan érkező új adat a futás közben megváltoztathatja a bemenetet. Három kiegészítő minta kell.

1. Futászárolás. A pipeline a teljes futás idejére zárol egy lock-fájlt; a második folyamat azonnal elutasítja a munkát, nem pedig versenyez:

```
import fcntl
import os
from contextlib import contextmanager

@contextmanager
def run_lock(path):
    fd = os.open(path, os.O_CREAT | os.O_RDWR, 0o644)
    try:
        try:
            fcntl.flock(fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
        except BlockingIOError:
            raise RuntimeError("a pipeline már fut") from None
        yield
```

```
    finally:
        fcntl.flock(fd, fcntl.LOCK_UN)
        os.close(fd)
```

A `flock` a POSIX rendszereken működik; Windowson a `msvcrt.locking` vagy egy portábilis `filelock` csomag a megfelelő. A zárolás nem helyettesíti az atomi írást: a zárolt futás is megszakadhat, ezért az írás továbbra is temp+`os.replace`.

2. Bemeneti hash. A futás elején és végén kiszámítjuk az összes bemeneti fájl kanonikus hashét. Ha a kettő eltér, a futás közben új adat érkezett; az eredményt nem commitáljuk, hanem a következő futásra bízzuk:

```
import hashlib
from pathlib import Path

def input_hash(paths):
    h = hashlib.sha256()
    for path in sorted(map(Path, paths)):
        h.update(path.name.encode("utf-8"))
        h.update(b"\0")
        h.update(path.read_bytes())
        h.update(b"\0")
    return h.hexdigest()

start_hash = input_hash(RAW.glob("*.csv"))
with run_lock(ART / ".pipeline.lock"):
    # ... a lépések ...
    if input_hash(RAW.glob("*.csv")) != start_hash:
        raise RuntimeError("a bemenet futás közben megváltozott")
```

3. Részleges futás. Ha a folyamat a második lépésnél szakad meg, a validated réteg már új, a features réteg viszont régi lehet. Ez nem baj, ha minden lépés saját kimenetét atomikusan cseréli, és a következő futás a teljes DAG-ot újraszámolja; a kimenet vagy a régi, vagy az új állapot, fél fájl nem keletkezik. A manifest a lépések sorrendjét és az input hasht rögzíti, így a részleges állapot felismerhető. A kimeneti hash-nek a teljes bemenetre és a kódverzióra kell épülnie, nem csak a fájlnévre.

A `tests/test_idempotent_write.py` a temp+replace mintát, a lock második foglalásának blokkolását, az input hash stabilitását és a futás közbeni változás észlelését teszteli.

30.1.3 Teljes minimális példa: három lépés Pythonban

A következő `pipeline.py` a `lab/` munkakönyvtárból futtatható Linux, macOS vagy WSL alatt. A kód pandas-t és PyArrow-t használ; a kimeneti Parquet-fájlok a `lab/artifacts/` alatti rétegmappákba kerülnek. A kód nem igényel hálózatot, és a Northstar kezdőadatokkal dolgozik.

```
# lab/pipeline.py
from pathlib import Path
import hashlib
import json
import os
import tempfile

import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq

RAW = Path("raw")
```

```
ART = Path("artifacts")
VALIDATED = ART / "validated"
FEATURES = ART / "features"

def atomic_write_parquet(df: pd.DataFrame, path: Path) -> None:
    """Atomikusan ír egy DataFrame-et Parquet-fájlba."""
    path.parent.mkdir(parents=True, exist_ok=True)
    table = pa.Table.from_pandas(df, preserve_index=False)
    with tempfile.NamedTemporaryFile(
        dir=path.parent, suffix=".parquet.tmp", delete=False
    ) as tmp:
        pq.write_table(table, tmp.name)
        os.replace(tmp.name, path)

def step_raw() -> None:
    """1. lépés: nyers CSV-k betöltése, séma-ellenőrzés, raw réteg kiírása."""
    orders_raw = pd.read_csv(
        RAW / "orders_raw.csv",
        parse_dates=["order_ts"],
        dtype={"customer_id": "string", "currency": "string"},
    )
    customers = pd.read_csv(RAW / "customers.csv", dtype={"customer_id": "string"})
    items = pd.read_csv(RAW / "order_items.csv", dtype={"currency": "string"})
    # Sémaellenőrzés: a kötelező oszlopok megléte.
    required = ["order_id", "customer_id", "order_ts", "basket_minor"]
    missing = [c for c in required if c not in orders_raw.columns]
    if missing:
        raise ValueError(f"Hiányzó oszlopok az orders_raw.csv-ben: {missing}")
    # A raw rétegbe Parquet-ként írjuk, explicit sémával.
    atomic_write_parquet(orders_raw, ART / "raw" / "orders_raw.parquet")
    atomic_write_parquet(customers, ART / "raw" / "customers.parquet")
    atomic_write_parquet(items, ART / "raw" / "order_items.parquet")
    # Hash-manifest a reprodukálhatósághoz.
    manifest = {
        "orders_raw_hash": hashlib.sha256(
            (RAW / "orders_raw.csv").read_bytes()
        ).hexdigest(),
        "customers_hash": hashlib.sha256(
            (RAW / "customers.csv").read_bytes()
        ).hexdigest(),
        "order_items_hash": hashlib.sha256(
            (RAW / "order_items.csv").read_bytes()
        ).hexdigest(),
    }
    (ART / "raw" / "manifest.json").write_text(json.dumps(manifest, indent=2))

def step_validate() -> None:
    """2. lépés: tisztítás és érvényességi szűrések a raw rétegből."""
    orders = pd.read_parquet(ART / "raw" / "orders_raw.parquet")
    customers = pd.read_parquet(ART / "raw" / "customers.parquet")
    # Érvénytelen sorok azonosítása a Northstar szerződés szerint:
    # negatív kosár, hiányzó distance_km, ismeretlen customer_id.
    known_customers = set(customers["customer_id"])
    invalid_mask = (
        (orders["basket_minor"] < 0)
        | orders["distance_km"].isna()
        | ~orders["customer_id"].isin(known_customers)
    )
    invalid_rows = orders.loc[invalid_mask].copy()
    # Extra duplikált sorok eltávolítása: az order_id egyedi marad.
    deduped = orders.loc[~invalid_mask].drop_duplicates(subset=["order_id"])
    # Auditnapló a döntésekről.
    audit = {
```

```
        "input_rows": len(orders),
        "invalid_rows_removed": len(invalid_rows),
        "duplicate_rows_removed": len(orders) - len(invalid_rows) - len(deduped),
        "output_rows": len(deduped),
        "invalid_order_ids": invalid_rows["order_id"].tolist(),
    }
    (VALIDATED / "audit.json").write_text(json.dumps(audit, indent=2))
    atomic_write_parquet(deduped, VALIDATED / "orders.parquet")

def step_features() -> None:
    """3. lépés: rendelésszintű feature-tábla, az aggregáció a join előtt."""
    orders = pd.read_parquet(VALIDATED / "orders.parquet")
    customers = pd.read_parquet(ART / "raw" / "customers.parquet")
    items = pd.read_parquet(ART / "raw" / "order_items.parquet")

    # Tételszintű aggregáció rendelésszintre, mielőtt joinolnánk.
    item_agg = (
        items.groupby("order_id", as_index=False)
        .agg(
            line_total_minor_sum=("line_total_minor", "sum"),
            quantity_sum_check=("quantity", "sum"),
        )
    )

    # Joinok: csak azok a rendelések maradnak, amelyekhez van ügyfél.
    features = (
        orders.merge(customers, on="customer_id", how="inner")
        .merge(item_agg, on="order_id", how="left")
    )

    # account_age_days az order_ts és signup_date különbsége.
    features["account_age_days"] = (
        features["order_ts"] - pd.to_datetime(features["signup_date"])
    ).dt.days

    # A starter feature-oszlopok kiválasztása; a célváltozó később csatolódik.
    feature_cols = [
        "order_id", "order_ts", "promised_days", "quantity_sum", "item_rows",
        "basket_minor", "shipping_fee_minor", "distance_km", "priority_flag",
        "account_age_days", "segment",
    ]
    features = features.loc[:, feature_cols]
    atomic_write_parquet(features, FEATURES / "features.parquet")

def run_pipeline() -> None:
    """A három lépés topologikus sorrendben."""
    step_raw()
    step_validate()
    step_features()

if __name__ == "__main__":
    run_pipeline()
```

Futtatás a `lab/` mappából:

```
cd lab
python pipeline.py
```

Az idempotencia-ellenőrzés ugyanebből a mappából:

```
python pipeline.py
python - <<'PY'
import pandas as pd
```

```
from pathlib import Path
import hashlib
p = Path("artifacts/features/features.parquet")
df = pd.read_parquet(p)
print("sorok:", len(df))
print("egyedi order_id:", df["order_id"].nunique())
print("tartalom hash:", hashlib.sha256(pd.util.hash_pandas_object(df,
↪  index=True).values.tobytes()).hexdigest()[:16])
PY
```

A fenti kimenet **szemléltető**, a könyvírási folyamat nem futtatta ezt a kódot. A laborban a várt értékek: a validated réteg a kontroll `orders.csv`-vel egyezik meg; a features sorainak száma 2400, és minden `order_id` pontosan egyszer szerepel. A másodszori futtatás után ugyanaz a sor- és kulcsszám, valamint ugyanaz a tartalom-hash adódik, mert a lépések felülírnak és nem fűznek hozzá.

30.1.4 Eredmény értelmezése és kézi kontroll

A példa három invariantet ellenőriz:

1. **Sorok száma**: a validálás utáni táblának egyeznie kell a mellékelt tiszta `orders.csv` sorainak számával (a Northstar szerződés szerint ez 2400). A raw 2405 sora a hibák és duplikátumok eltávolítása után csökken.
2. **Kulcsegyediség**: a features táblában `order_id.nunique() == len(features)`, tehát nincs egy rendeléshez több feature-sor.
3. **Idempotencia**: a `pipeline.py` kétszeri futtatása után a features tartalom-hash azonos. Ez azért igaz, mert minden lépés ugyanazt a kimeneti fájlt atomikusan cseréli, és a bemeneti raw fájlokat nem módosítja.

A relációs szemcsézettség itt végig rendelésszintű; a tételszintű aggregáció a `step_features` elején történik, így a `shipping_fee_minor` nem duplázódik. A hibás sorokról szóló döntéseket az `audit.json` rögzíti, így a tisztítás nem csendes.

30.1.5 Feladatok

1. **Értelmezés.** Magyarázd el saját szavaiddal, miért nem dupláz sorokat a `step_validate` kétszeri futtatása, annak ellenére, hogy a függvény a raw-ból olvas és a validated-be ír. Melyik művelet hiánya okozna duplikációt?
2. **Implementálás.** Írj egy `check_idempotency()` függvényt, amely a `pipeline.py` kétszeri futtatása után betölti a `features.parquet`-t, és ellenőrzi, hogy a sorok száma 2400, az `order_id` egyedi, és a `basket_minor` oszlop összege megegyezik egy előre rögzített kontrollértékkel. A függvény emeljen `AssertionError`-t, ha bármelyik feltétel sérül.
3. **Hibadiagnózis.** Egy fejlesztő a `step_features` végén a `pq.write_table(table, path, mode="append")` hívást használja az `atomic_write_parquet` helyett, hogy „gyorsabb legyen”. Mi történik a második futtatáskor? Milyen két tünetet fogsz látni a features táblában? Hogyan javítod a hibát anélkül, hogy a lépés matematikai transzformációját megváltoztatnád?
4. **Megváltozott feltétel.** A következő napon új nyers fájl érkezik: `lab/raw/orders_2025-01-01.csv`, amely csak az aznapi rendeléseket tartalmazza, és a `orders_raw.csv` már nem frissül. Módosítsd a `step_raw`-t úgy, hogy a raw rétegbe az összes napi fájl uniója kerüljön,

`order_id` szerint deduplikálva. Milyen további invariáns kell ahhoz, hogy a `step_validate` és `step_features` továbbra is idempotens maradjon?

30.1.6 Megoldások és indoklás

1. A `step_validate` a bemenetet a raw rétegből olvassa, és a kimenetet a validated rétegbe írja. A kimeneti fájlt mindig felülírja, nem hozzáfűzi. Ezért a második futtatás ugyanazt a fájlt hozza létre újra, nem pedig a régi mellé ír. Duplikáció akkor keletkezne, ha a kód `mode="append"`-et használna, vagy ha a validated táblát olvasná és a saját kimenetéhez fűzné — ez utóbbi a rétegkeveredés tipikus esete.

2. Egy lehetséges megoldás:

```
def check_idempotency() -> None:
    df = pd.read_parquet("artifacts/features/features.parquet")
    assert len(df) == 2400, f"a features sorainak száma {len(df)}, nem 2400"
    assert df["order_id"].is_unique, "az order_id nem egyedi"
    expected_basket_sum = 123456789  # a kontrollfájlból számolt érték
    assert df["basket_minor"].sum() == expected_basket_sum, "a basket_minor összege eltér"
```

Az `expected_basket_sum` értékét a laborban a tiszta `orders.csv` alapján kell kiszámolni; a példa itt csak a szerkezetet mutatja. Az ellenőrzés akkor megalapozott, ha a kontrollérték előre rögzített, és a futás után összehasonlítjuk.

3. Az `append` mód a második futtatáskor a meglévő Parquet-fájl **után** írja az új sorokat, így a features tábla 4800 soros lesz. Két tünet: a sorok száma megduplázódik, és az `order_id` már nem egyedi, mert minden rendelés kétszer szerepel. A javítás: használjuk az `atomic_write_parquet`-et, vagy `pq.write_table(table, path)`-t, amely felülír. A transzformáció változatlan marad; csak az I/O műveletet cseréljük.

4. A `step_raw`-ban a `orders_raw.csv` helyett minden `orders_*.csv`-t be kell olvasni, konkatenálni, majd `drop_duplicates(subset=["order_id"])`-et hívni. Az új raw tábla így az összes nap uniója lesz, minden rendelés pontosan egyszer. A további invariant: a napi fájlok feldolgozási sorrendje ne befolyásolja a kimenetet, ezért a konkatenálás előtt determinisztikus fájlsorrendet és egyértelmű verzió-/elsőbbségi szabályt kell rögzíteni a duplikált `order_id`-khez; pusztán az `order_ts` és `order_id` szerinti rendezés nem oldja fel az eltérő tartalmú duplikált rekordok konfliktusát. Az idempotencia akkor marad fenn, ha a `step_raw` kimenetét is atomikusan felülírjuk, és a napi fájlokból való olvasás determinisztikus (pl. a fájlnevek rendezett listája). A `step_validate` és `step_features` változatlan marad, mert bemenetük a raw réteg, amely most is egyetlen konzisztens tábla.

30.1.7 Gyakori tévedés és ellenpélda

A leggyakoribb hiba, hogy egy lépés kimenetét a saját bemenetéhez fűzik, vagy a rétegeket összekeverik. Például egy jegyzetfüzetben valaki a `validated` táblát a `step_validate` cellájában így menti: `orders.to_csv("validated/orders.csv", mode="a")`. Az első futtatás rendben van; a második futtatás azonban duplikált sorokat hoz létre, és a feature-összegek kétszeresére nőnek. Ez a „véletlenül additív adatrombolás”: az eredmény nem a tisztított adat, hanem annak kétszerese. A helyes minta az atomic overwrite és a rétegek szigorú elkülönítése. Ez visszavezet a 29.1. alfejezet tiszta függvény és I/O határ elvéhez: a transzformáció tiszta, a mellékhatás csak a kimeneti fájl atomi cseréje.

Önellenőrzés: az egyes lépések bemeneti és kimeneti szemcsézettsége explicite azonos? A kimeneti fájl mindig ugyanazon az útvonalon jön létre? A tételszintű adat a join előtt aggregálódik?

A `drop_duplicates` szűrés sorrendje nem rejti el a hibás sorokat? Ha ezekre igen a válasz, a pipeline idempotens.

30.1.8 Rövid összefoglaló

A feldolgozási gráf lépései explicit be- és kimeneti fájlhatárral rendelkeznek, és a raw/validated/features rétegek egymásra épülnek. A lépések tiszta függvényként viselkednek, a kimenetet atomikusan felülírják, ezért kétszeri futtatás nem duplikál sorokat. Az idempotenciát sor-, kulcs- és kontrollösszeg-invariantek ellenőrzik. A relációs szemcsézettség eközben végig rendelésszintű marad, mert a tételszintű aggregáció a join előtt történik. A dbt és az orchestratorok ezt a DAG-fegyelmet skálázzák, de kis helyi pipeline-hoz egy Python script is elég. A következő alkalmazás a 30.2. alfejezet adatminőségi kapu: itt azt vizsgáljuk, hogyan állítható meg a pipeline automatikusan, ha a tisztítási feltételek sérülnek.

30.2 Adatminőségi kapu és hibás sorok kezelése

Belépő · szint: haladó · idő: 90 perc · eszköz: pandas, DuckDB

Előfeltétel: 30.1. alfejezet · Célkészség: Adatminőségi kapu és hibás sorok kezelése · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

30.2.1 Cél és konkrét helyzet

A Northstar pipeline a nyers rendeléseket naponta betölti; a 30.1. alfejezetben definiált DAG azt ígéri, hogy a `raw/` rétegből a `validated/` rétegbe csak olyan adat kerül, amelyre a downstream feature- és modellépítés támaszkodhat. A mai egységben az a kérdés, hogyan állítsunk fel olyan kaput, amely három hibás bemenetre megbízhatóan a megfelelő döntést adja. A megfigyelési egység a rendelés, a kapu bemenete egy napi `orders_YYYYMMDD.parquet` fájl és a `customers.parquet` törzs, a kimenet pedig a döntés (`STOP`, `QUARANTINE`, `WARN`), a validált tábla és a karanténnapló. Induló becslés: a sémahibás fájlra a kapunak blokkolnia kell; a rossz ügyfélhivatkozást tartalmazó fájlból a jó sorokat tovább kell engednie, a rossz sort le kell választania; a frissességi problémát pedig figyelmeztetéssel kell jeleznie. A következőkben ezeket a döntési szabályokat pontosítjuk.

30.2.2 A súlyosság három szintje

A minőségi szabályhoz rendelt döntés nem az adathiba érzelmi súlyát, hanem a downstream kockázatát írja le. A `STOP` azt jelenti, hogy a lépés nem ír új kimenetet, és a hívó pipeline megszakad. A `QUARANTINE` a hibás sorokat külön táblába teszi, a megfelelő sorokat továbbengedi; a kiválasztás nem csendes, mert a karanténnapló rögzíti a sorszámot, a szabályt és az időbélyeget. A `WARN` nem blokkol, de a futásjegyzékben emlékeztetőt hagy. A három szint használata akkor korrekt, ha a szabályhoz tartozó hibahatás ismert.

Szabály	Tipikus döntés	Indoklás
Schema drift	STOP	A downstream kód típusokra épül; a továbbengedés rejtett hibákat hozhat.
Soronkénti FK-sértés	QUARANTINE	A hibás sor lokális, a többi sor megbízható.

Szabály	Tipikus döntés	Indoklás
Enyhe freshness-késés	WARN	Az adat még használható, de a folyamat késését jelezni kell.

A döntés mátrixa nem rögzíthető örökre: ha a frissességet a predikcióhoz azonnal kell használni, a késés STOP is lehet. A konfigurációban ezért szerepeljen a `severity` mező, ahogyan a 29.2. alfejezetben a konfigurációt és a futásnaplót bevezettük.

30.2.3 Ellenőrzési szabályok

30.2.3.1 Schema drift

A séma a bemeneti oszlopok nevének, típusának és nullázhatóságának szerződése. A napi rendelésfájl rögzített sémája a Northstar-szerződésből származik: `order_id` szöveg, `order_ts` időbélyeg, `distance_km` lebegőpontos, `priority_flag` egész, a többi oszlop a rendelés pillanatában ismert. A `schema drift` azt jelenti, hogy a tényleges fájl ettől eltér. Ha például a `distance_km` oszlop `'12.5'` karakterláncként érkezik, a későbbi numerikus művelet vagy hibát dob, vagy csendesen stringösszehasonlítást végez. A kapu a DuckDB `DESCRIBE` kimenetét hasonlítja a rögzített sémához. A pontos típusnevek verziófüggők lehetnek, ezért éles használat előtt egy helyi `DESCRIBE` futással kell a várt értékeket rögzíteni.

30.2.3.2 Row count

A sorok száma a fájl teljességének durva, de olcsó ellenőrzése. Egy üres fájl vagy a duplázott betöltés azonnal látszik. A napi forgalomra a korábbi manifestekből származó alsó és felső küszöböt adunk meg: például `lo=1`, `hi=500`. Ha a tényleges sorok száma ezen kívül esik, a kapu STOP-pal áll meg, mert a fájl nem a várt napi adag. A küszöböt nem szabad a mintából egyetlen nap alapján becsülni; promóció vagy ünnep miatt a tartomány szűk lehet, és a kapu hibásan blokkol. Ilyenkor a konfigurációban egy `row_count_override` mező és a manifestbe írt indoklás a helyes megoldás.

30.2.3.3 Freshness

A frissesség azt méri, mennyire aktuális a fájl maximális időbélyege a kapu futásának idejéhez képest: $freshness_age = now_ts - max(order_ts)$. Ha a fájl tegnap érkezett, de a benne lévő rendelések 5 napja vannak, a betöltési folyamat leállt vagy késik. A szabály `age > limit` esetén WARN-et ad, kivéve, ha a modell az aznapi döntéshez használja az adatot; akkor a szabály STOP is lehet. A freshness nem az adattartalom hibája, hanem a folyamaté; a 29.2. alfejezet futásjegyzékben a késést külön mezőben tároljuk.

30.2.3.4 Completeness

A completeness oszloponként a nem hiányzó értékek aránya: $completeness_col = 1 - missing_col / row_count$. Ha 120 sorból 3 hiányzó `distance_km` van, az arány 0,975, azaz 97,5%. A szabály nem csak az arányt, hanem az abszolút hibaszámot is nézze: egy 10 000 soros fájlban a 0,5% hiány 50 sor, ami egy modellben már eltérő döntést hozhat. A kapu a kötelező prediktor oszlopokra `missing_count = 0` szabályt ír elő, és a hibás sorokat karanténba teszi. A

hiányzási mechanizmus (MCAR, MAR, MNAR) dönti el, hogy a karantén milyen torzítást visz be; a 18.2. alfejezetben látott mintaszelekció ugyanazt a kérdést veti fel: a kihagyott sorok nem véletlenek, ezért a validált eloszlás nem feltétlenül a teljes forgalomé. A kapu nem imputál, csak szeparál.

30.2.3.5 Referential integrity

Az idegen kulcs ellenőrzése azt jelenti, hogy minden `customer_id` értéknek léteznie kell a `customers` törzsben. Az ellenőrzés anti-join: `LEFT JOIN` a törzshöz, és ahol `customers.customer_id IS NULL`, ott orphan sor van. Ha az ügyfél új, és a törzs még nem frissült, a helyes lépés a törzs feltöltése, nem az FK megsemmisítése. A kapu a rossz hivatkozású sorokat karanténba teszi.

30.2.4 Minimális munkapélda

A példa a projektgyökérben fut, a `lab/` könyvtárban. A `customers.parquet` a Northstar 300 ügyfelét tartalmazza. A napi rendelésfájl sémája a fent rögzített. A kapu Python-szkriptje DuckDB-t használ, mert a Parquet-olvasás és az anti-join egyaránt SQL-ben kifejezhető.

```
# quality_gate_daily.py (szemléltető; futtatás előtt a DuckDB DESCRIBE
# kimenetét a helyi verzióhoz igazítsd)
import json
import sys
from datetime import timedelta
from pathlib import Path

import duckdb

EXPECTED_SCHEMA = {
    "order_id": "VARCHAR",
    "customer_id": "VARCHAR",
    "order_ts": "TIMESTAMP WITH TIME ZONE",
    "promised_days": "BIGINT",
    "quantity_sum": "BIGINT",
    "item_rows": "BIGINT",
    "basket_minor": "BIGINT",
    "shipping_fee_minor": "BIGINT",
    "currency": "VARCHAR",
    "distance_km": "DOUBLE",
    "priority_flag": "BIGINT",
}

def check_schema(con, table):
    actual = {r[0]: r[1].upper() for r in con.execute(f"DESCRIBE {table}").fetchall()}
    problems = []
    for col, dtype in EXPECTED_SCHEMA.items():
        if col not in actual:
            problems.append(f"missing column {col}")
        elif actual[col] != dtype:
            problems.append(f"{col}: expected {dtype}, got {actual[col]}")
    return problems

def check_row_count(con, table, lo, hi):
    n = con.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
    if n < lo or n > hi:
        return [f"row count {n} outside [{lo}, {hi}]"]
    return []
```

```python
def check_completeness(con, table, required, max_missing=0):
    out = []
    for col in required:
        m = con.execute(f"SELECT COUNT(*) FROM {table} WHERE {col} IS NULL").fetchone()[0]
        if m > max_missing:
            out.append(f"{col}: {m} missing")
    return out

def check_freshness(con, table, now_ts, limit):
    max_ts = con.execute(f"SELECT MAX(order_ts) FROM {table}").fetchone()[0]
    age = now_ts - max_ts
    if age > limit:
        return [f"freshness age {age} > {limit}"]
    return []

def check_referential_integrity(con, orders_table, customers_table, fk_col):
    orphans = con.execute(f"""
        SELECT COUNT(*) FROM {orders_table} o
        LEFT JOIN {customers_table} c USING ({fk_col})
        WHERE c.{fk_col} IS NULL
    """).fetchone()[0]
    if orphans:
        return [f"{orphans} orphan {fk_col} rows"]
    return []

def decide(schema, rows, completeness, freshness, referential):
    decision = "WARN"
    blockers = []
    if schema:
        decision = "STOP"; blockers += schema
    if rows:
        decision = "STOP"; blockers += rows
    if completeness:
        decision = "QUARANTINE"; blockers += completeness
    if referential:
        decision = "QUARANTINE"; blockers += referential
    if freshness:
        blockers.append("WARN")
    return decision, blockers
```

A három hibás bemenet előállítása külön szkriptben:

```python
# make_three_bad_inputs.py (szemléltető)
import duckdb

con = duckdb.connect()

# 1. Sémahiba: distance_km VARCHAR
con.execute("""
    CREATE TABLE good AS
    SELECT 'o1' AS order_id, 'c1' AS customer_id,
           TIMESTAMPTZ '2025-04-20 10:00+00' AS order_ts,
           7 AS promised_days, 2 AS quantity_sum, 1 AS item_rows,
           1200 AS basket_minor, 300 AS shipping_fee_minor,
           'EUR' AS currency, 8.5 AS distance_km, 0 AS priority_flag
""")
con.execute("ALTER TABLE good ALTER distance_km TYPE VARCHAR")
con.execute("COPY good TO 'orders_bad_schema.parquet' (FORMAT PARQUET)")

# 2. Idegen kulcs hiba: 11 jó sor + 1 ismeretlen ügyfél
con.execute("DROP TABLE good")
con.execute("""
    CREATE TABLE good AS
```

```
    SELECT 'o1' AS order_id, 'c1' AS customer_id,
           TIMESTAMPTZ '2025-04-20 10:00+00' AS order_ts,
           7 AS promised_days, 2 AS quantity_sum, 1 AS item_rows,
           1200 AS basket_minor, 300 AS shipping_fee_minor,
           'EUR' AS currency, 8.5 AS distance_km, 0 AS priority_flag
""")
con.execute("""
    INSERT INTO good VALUES
    ('o2', 'c1', TIMESTAMPTZ '2025-04-20 11:00+00', 7, 3, 2, 1500, 300, 'EUR', 5.1, 0),
    ('o3', 'c1', TIMESTAMPTZ '2025-04-20 12:00+00', 7, 1, 1, 800, 300, 'EUR', 2.0, 0),
    ('o4', 'c1', TIMESTAMPTZ '2025-04-20 13:00+00', 7, 1, 1, 900, 300, 'EUR', 3.2, 0),
    ('o5', 'c1', TIMESTAMPTZ '2025-04-20 14:00+00', 7, 4, 3, 2200, 300, 'EUR', 6.0, 0),
    ('o6', 'c1', TIMESTAMPTZ '2025-04-20 15:00+00', 7, 2, 1, 1100, 300, 'EUR', 7.0, 0),
    ('o7', 'c1', TIMESTAMPTZ '2025-04-20 16:00+00', 7, 1, 1, 500, 300, 'EUR', 1.5, 0),
    ('o8', 'c1', TIMESTAMPTZ '2025-04-20 17:00+00', 7, 1, 1, 700, 300, 'EUR', 2.2, 0),
    ('o9', 'c1', TIMESTAMPTZ '2025-04-20 18:00+00', 7, 2, 2, 1300, 300, 'EUR', 4.0, 0),
    ('o10', 'c1', TIMESTAMPTZ '2025-04-20 19:00+00', 7, 3, 2, 1800, 300, 'EUR', 3.8, 0),
    ('o11', 'c1', TIMESTAMPTZ '2025-04-20 20:00+00', 7, 2, 1, 950, 300, 'EUR', 5.5, 0),
    ('o12', 'C-999-UNKNOWN', TIMESTAMPTZ '2025-04-20 21:00+00', 7, 1, 1, 600, 300, 'EUR', 4.4,
 ↪  0)
""")
con.execute("COPY (SELECT * FROM good) TO 'orders_bad_fk.parquet' (FORMAT PARQUET)")

# 3. Frissesség hiba: rendelések 13 napja
con.execute("""
    CREATE TABLE stale AS
    SELECT * FROM good
    WHERE order_ts < TIMESTAMPTZ '2025-04-21'
""")
con.execute("COPY stale TO 'orders_bad_freshness.parquet' (FORMAT PARQUET)")
```

A kapu futtatása (Linux/macOS terminál, projektgyökér munkakönyvtár):

```
python quality_gate_daily.py orders_bad_schema.parquet customers.parquet --now
 ↪  "2025-05-03T00:00:00+00:00"
python quality_gate_daily.py orders_bad_fk.parquet customers.parquet --now
 ↪  "2025-05-03T00:00:00+00:00"
python quality_gate_daily.py orders_bad_freshness.parquet customers.parquet --now
 ↪  "2025-05-03T00:00:00+00:00"
```

Az elvárt kimenet kézi ellenőrzéssel, nem futtatott kód alapján:

- `orders_bad_schema.parquet`: a `DESCRIBE` a `distance_km`-re `VARCHAR`-t ad, ezért `STOP`, exit code 2, nincs validált kimenet.
- `orders_bad_fk.parquet`: az anti-join 1 orphansort talál, ezért `QUARANTINE`; a 11 jó sor a `validated_orders_daily` táblába, az `o12` sor a `quarantine_orders` táblába kerül. A manifestben `quarantined_rows=1`.
- `orders_bad_freshness.parquet`: a maximális `order_ts` 2025-04-20, a `now_ts` 2025-05-03, így `age=13` nap, ami nagyobb, mint a `limit=2` nap; a döntés `WARN`, a fájl továbbmegy, de a manifest `warnings` mezője tartalmazza a frissesség eltérést.

30.2.4.1 A karanténrekord szerződése

A karantén nem „eldobott sorok” gyűjtője, hanem **változtathatatlan rekordok** naplója. Minden karanténba tett sorhoz legalább ezek a mezők kellenek:

Mező	Tartalom	Miért kell
`quarantine_id`	a forrás hash-e, a sorszám és a szabály stabil hash-e	ugyanaz a hiba kétszer ne kerüljön be
`source_file`, `source_sha256`	a napi fájl neve és bájt-hash-e	a hiba forrása reprodukálható legyen
`source_row_number`	a fejléc utáni 1-alapú adatsorszám	emberi és gépi visszakeresés
`row_id`	üzleti kulcs, például `order_id`	a javított sor összekapcsolása
`payload_json`	az eredeti sor kanonikus JSON-ja	az eredeti állapot megőrzése
`error_type`	`schema`, `completeness`, `referential`, `business_rule`, `as_of`	hibakategória szerinti elemzés
`detected_at`	UTC időbélyeg	a hiba időpontja
`rule_version`	a szabály verziója	a szabály változása megkülönböztethető
`status`	`open`, `in_review`, `fixed`, `rejected`, `superseded`	a feloldás állapota
`change_log`	események listája: idő, szereplő, művelet, megjegyzés	csak hozzáfűzés, felülírás nélkül
`correction_id`	a javított sor azonosítója	az egyszeri újrafeldolgozás kulcsa

A rekord **soha nem íródik felül**: a javítás egy új esemény a `change_log`-ban, amely `fixed` vagy `rejected` állapotra vált, és opcionálisan egy `correction` rétegbe írt javított sort hoz létre. A következő futás a javításokat a validálás **előtt** alkalmazza, `row_id` szerinti upserttal; így a javított sor pontosan egyszer kerül a végső táblába, akkor is, ha a napi fájl még a régi hibás sort tartalmazza.

```
import hashlib
import json

def quarantine_record(source_name, source_bytes, row_number, row, error_type, now):
    digest = hashlib.sha256(source_bytes).hexdigest()
    payload = json.dumps(row, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    qid = hashlib.sha256(
        f"{digest}:{row_number}:{error_type}".encode("utf-8")
    ).hexdigest()
    return {
        "quarantine_id": qid,
        "source_file": source_name,
        "source_sha256": digest,
        "source_row_number": row_number,
        "row_id": row.get("order_id"),
        "payload_json": payload,
        "error_type": error_type,
        "detected_at": now,
        "rule_version": "v1",
        "status": "open",
        "change_log": [{"at": now, "actor": "quality_gate", "action": "detected"}],
        "correction_id": None,
    }

def resolve(record, status, actor, now, note, correction_id=None):
    if status not in {"in_review", "fixed", "rejected", "superseded"}:
        raise ValueError("érvénytelen karanténstátusz")
```

```
    updated = {**record}
    updated["status"] = status
    updated["correction_id"] = correction_id
    updated["change_log"] = record["change_log"] + [
        {"at": now, "actor": actor, "action": status, "note": note}
    ]
    return updated
```

Az invariánsok: a `quarantine_id` ugyanarra a forrásra, sorszámra és hibatípusra mindig ugyanaz; a `change_log` hossza csak nő; a `fixed` állapotú rekordhoz tartozik `correction_id`; és a javított sor `row_id` szerinti upsertja kétszeri alkalmazás után is egy sort ad. A `tests/test_quarantine_protocol.py` ezeket a szerződéseket és az egyszeri újrafeldolgozást teszteli.

30.2.5 dbt mint deklaratív kapu

A dbt-ben ugyanezek a szabályok adatmodell-yml-ben élnek, és a `dbt build` futtatásakor automatikusan ellenőrződnek. A `relationships` teszt a referential integrity-t, a `not_null` a completeness-t, a `accepted_values` a megengedett kategóriákat ellenőrzi. A sorok számát és a frissességet a `dbt test` custom tesztek vagy a `source freshness` fogalmával lehet mérni. A dbt szerepe ebben a fejezetben áttekintés: a Python-kapu és a dbt-teszt nem zárja ki egymást, a dbt a tárolt adatmodell szerződését, a Python-kapu pedig a fájlbetöltés előtti ellenőrzést végzi. Prefect vagy Dagster akkor indokolt, ha a kapu után retry, ütemezés vagy több lépésből álló megfigyelhetőség is kell.

30.2.6 Feladatok

30.2.6.1 1. Értelmezés: miért nem megoldás a nulla imputálás?

A napi fájlban a `distance_km` 3 sora NULL. Miért nem fogadható el az, hogy a kapu egyszerűen 0-ra cseréli ezeket, és `WARN` döntést ad? Fogalmazza meg a hiányzási mechanizmus és a modellkalibráció szempontjából.

30.2.6.2 2. Megvalósítás: anti-join kézzel

Írd át a `check_referential_integrity` függvényt úgy, hogy a hibás `customer_id` értékek listáját is visszaadja, ne csak a darabszámot. A mini bemenet: `orders` sorai `('o1','c1'), ('o2','c2'), ('o3','cX')`; `customers` sorai `'c1','c2'`. Add meg a várt kimenetet.

30.2.6.3 3. Hibakeresés: túl szűk row-count küszöb

Egy napi kapu `lo=1, hi=500` értékkel fut, és a fájl 6000 sort tartalmaz. A downstream csapat szerint a fájl helyes, egy promóció miatt nőtt meg a forgalom, nincs duplikáció. A kapu STOP-ot ad. Mi a tanulság, és hogyan javítható a konfiguráció anélkül, hogy a kaput kikapcsolnánk?

30.2.6.4 4. Transzfer: kísérleti adat kapuja

Írj kaput az `assignments.parquet` és `outcomes.parquet` kísérleti páros hoz. A megfigyelési egység a randomizált ügyfél. Adj legalább öt szabályt, mindegyikhez súlyosságot, és egy érvényességi korlátot. A szabályok között szerepeljen: `customer_id` egyedi az assignmentsben; `variant` csak `control` vagy `treatment`; outcomes `customer_id` halmaza pontosan az assignments; `conversion_7d` értéke {0,1}; `observed_until` = 2025-07-08.

30.2.7 Megoldások és indoklás

30.2.7.1 1. megoldás

A nulla imputálás azt állítja, hogy a rendelés nulla kilométerről érkezett, ami a szállítási idő modelljében a lehető legrövidebb távolságnak felel meg. Ha a hiány nem teljesen véletlen (MCAR), hanem például a távolságot nem rögzítő külső partner rendszeréből származik (MAR/MNAR), akkor a hiányzó csoport nem reprezentálja a teljes eloszlást. A 26.2. alfejezetben látott kalibrációhoz a prediktor eloszlásának torzulása rossz bizonytalansági becslést ad. A helyes döntés a sorok karanténba tétele, és a hiány okának vizsgálata; ha imputálás szükséges, az külön elemzési lépés, a 29.2. alfejezet manifestben rögzített módon.

30.2.7.2 2. megoldás

```
def check_referential_integrity(con, orders_table, customers_table, fk_col):
    orphans = con.execute(f"""
        SELECT DISTINCT o.{fk_col}
        FROM {orders_table} o
        LEFT JOIN {customers_table} c USING ({fk_col})
        WHERE c.{fk_col} IS NULL
        ORDER BY 1
    """).fetchall()
    if orphans:
        ids = [r[0] for r in orphans]
        return [f"{len(ids)} orphan rows: {ids}"]
    return []
```

A mini bemenetre az anti-join egy sort ad: `cX`. Az ellenőrző invariant: a visszaadott lista hossza egyenlő a `COUNT(*)`-tal, és a lista minden eleme valóban hiányzik a `customers`-ből. A tolerancia itt 0: a feladat egzakt string-összehasonlítás.

30.2.7.3 3. megoldás

A hiba nem a fájlban, hanem a kapu konfigurációjában van. A `hi=500` küszöb egy régi átlagos napra volt kalibrálva; a promóció ismert esemény, ezért a konfigurációban a `row_count_upper_override=7000` és az indoklás kerüljön a manifestbe. A kaput nem szabad kikapcsolni, mert a duplikáció elleni védelem továbbra is kell; a küszöböt a promóció várható hatásával kell frissíteni, és a folyamatot utólag ellenőrizni, hogy a 6000 sor tényleg a várt növekedés-e.

30.2.7.4 4. megoldás

A kísérleti kapu szabályai:

1. `assignments.customer_id` egyedi: ha nem, STOP, mert a randomizálási egység sérült.
2. `assignments.variant` értékkészlete {control, treatment}: ha más, QUARANTINE a hibás sorokra.
3. `outcomes.customer_id` halmaza pontosan az assignments: ha extra ügyfél van, QUARANTINE; ha hiányzik, STOP, mert az ITT nevező nem teljes (visszautalás 18.2. alfejezet).
4. `outcomes.conversion_7d` értéke {0,1}: ha más, QUARANTINE.
5. `outcomes.observed_until` = 2025-07-08: ha eltér, STOP, mert a megfigyelési ablak nem azonos.
6. `outcomes.currency` = EUR: ha eltér, WARN, mert a pénzösszeg értelmezése változhat.

Az érvényességi korlát: a kapu az adat formális szerződését ellenőrzi, nem a kísérleti randomizálás tényleges végrehajtását; a randomizálás minőségét külön egyensúlyteszttel kell vizsgálni.

30.2.8 Gyakori tévedés ellenpéldával

A leggyakoribb tévhit az, hogy a kapu célja a nullák eltüntetése. Ha a `distance_km` hiányzó értékét nulla kilométerre írjuk, a modell azt tanulja, hogy a hiányos rekordok gyors kézbesítésűek; egy logisztikai döntésnél ez szisztematikusan alulbecsli a késést. A 26.2. alfejezet kalibrációs görbéje ilyenkor nem a valós gyakoriságot mutatja, hanem a mesterséges nulla távolság hatását. A helyes stratégia először a sorok karanténba tétele, majd a hiány okának vizsgálata; imputálás csak akkor, ha a mechanizmust és a torzítást explicit módon leírtuk. A 29.2. alfejezet futásnaplóban ennek a döntésnek nyoma kell legyen.

30.2.9 Önellenőrzés

- Meg tudja-e nevezni, milyen feltétel mellett válik a freshness WARN-ből STOP-pá?
- A karanténba tett sorok milyen mintaszelekciós torzítást okozhatnak?
- Miért nem a row count a referential integrity ellenőrzése?
- A manifestben rögzített warning miért fontos a későbbi modellkártyához?

A hibás rekordok kezelését hibakategóriákkal és karanténkimenettel dokumentáljuk; a csöndes eldobás a célpopulációt és a becslést is megváltoztathatja.

30.2.10 Rövid összefoglaló

A minőségi kapu minden szabályhoz STOP, QUARANTINE vagy WARN döntést rendel, és a döntést a manifestben rögzíti. A schema drift és a durva row-count eltérés blokkol, a soronkénti hiány és idegenkulcs-sértés karanténba kerül, a frissesség késése figyelmeztetést ad. A karantén nem imputálás; a hiányzási mechanizmus torzítást vihet a későbbi modellbe, ezért a döntést indokolni és naplózni kell. A Python-kapu a fájlbetöltés előtti ellenőrzést végzi, a dbt a tárolt modell szerződését teszteli, a Prefect/Dagster pedig akkor indokolt, ha retry és ütemezés is kell. A következő alfejezetben a cache, backfill és orchestrator kérdését vesszük elő.

30.3 Cache, backfill és orchestrator

Belépő · szint: haladó · idő: 90 perc · eszköz: pandas, DuckDB

Előfeltétel: 30.2. alfejezet · Célkészség: Cache, backfill és orchestrator · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

30.3.1 Cél és induló helyzet

A 30.2. alfejezetben felépített adatminőségi kapu megállítja a hibás sorokat. Most a folyamat többi részét tesszük újrafuttathatóvá: ha ugyanazt a feladatot ugyanazzal a bemenettel és paraméterrel futtatjuk, ne számoljunk újra, de ha bármelyik megváltozik, a régi gyorsítótár-kimenet (cache output) ne használódjon fel. A gyakori tévedés pont ez: attól, hogy egy output fájl létezik, még nem biztos, hogy az aktuális bemenethez készült.

A fejezet laborja egy kis, helyi DAG három feladattal:

1. `extract`: CSV betöltése,
2. `filter`: sorok szűrése egy paraméter szerint,
3. `aggregate`: ügyfelenkénti mennyiségösszeg.

Bemenetünk a következő hatrendeléses miniállomány, amelyet a kód maga hoz létre:

order_id	customer_id	quantity_sum	priority_flag
o1	c1	3	1
o2	c1	2	0
o3	c2	5	1
o4	c2	1	1
o5	c1	4	0
o6	c3	2	0

Az `order_ts` oszlop is szerepel a CSV-ben, de a szűréshez most nem kell. A futási paraméter `only_priority`: ha `1`, csak a `priority_flag=1` sorokat tartjuk meg; ha `0`, a többit.

Induló becslés. Mielőtt a kódot megnéznéd, számold ki kézzel: `only_priority=1` esetén mely sorok maradnak, és mennyi az ügyfelenkénti `quantity_sum` összeg? Ugyanezt tedd meg `only_priority=0`-ra. Ez lesz a cache-megvalósítás kézi kontrollja.

30.3.2 Cache key, input hash és érvénytelenítés

A gyorsítótárban egy feladat eredményét egy **tartalomcímzett kulcs** (content-addressed cache key) azonosítja. A kulcsnak legalább hat dolgot kell tartalmaznia:

- a feladat nevét és a függvény forráshashét (code version),
- a feladat paramétereit kanonikus alakban,
- minden bemeneti fájl tartalomhashét,
- a sémaverziót,
- a környezeti ujjlenyomatot (Python-verzió, könyvtárverziók, platform),
- backfill esetén a korrekciós verziót.

A tartalomhash lényege: ugyanaz a fájlnév idővel más tartalmat jelölhet, ezért a fájlnév nem elég. A kulcsot célszerű egy determinisztikus kriptográfiai hashbe tömöríteni:

$$k = \mathrm{SHA256}(\mathrm{canonical}(T, c, s, \theta, h(x_1), \ldots, h(x_n), e, r)),$$

ahol T a feladat neve, c a kód forráshash-e, s a sémaverzió, θ a paraméterek kanonikus JSON alakja, $h(x_i)$ a bemeneti fájl SHA-256 hash-e, e a környezeti ujjlenyomat, r pedig a backfill korrekciós verziója. Az SHA-256 kimenete 64 hexa karakter; a gyakorlatban csak ennyit látunk, nem a bemenetet. A kanonikus JSON `sort_keys=True`-pal készül, így a Python-szótárak kulcssorrendje nem változtatja meg a kulcsot.

A kódverzió nem feltétlenül kézi `task_version`: a függvény forrásának hash-e (`inspect.getsource`) akkor is megváltoztatja a kulcsot, ha valaki elfelejti a verziószámot emelni. A sémaverzió az adatszerződés változását jelöli, a környezeti ujjlenyomat pedig azt, hogy ugyanaz a kód más pandas/PyArrow/DuckDB-verzióval vagy más platformon más outputot adhat. A backfillnél a **korrekciós verzió** különül el a kód- és sémaverziótól: egy javított bemenet vagy javított szabály új `r` értéket kap, miközben a régi kulcsok megmaradnak, így a korábbi futás visszaállítható és összehasonlítható. A javítás után sem a régi kulcsot nem írjuk felül, sem az új futást nem mossuk össze a régivel.

30.3.3 Retry, scheduling és backfill

A tranziens hibákat — rövid fájlzár, hálózati időtúllépés — érdemes újrapróbálni. Az újrapróbálkozás (retry) csak determinisztikus, idempotens feladatra biztonságos: ha ugyanaz a hívás kétszer fut le, ugyanazt az outputot kell adnia. Példánkban a CSV-olvasás ilyen.

Az ütemezés (scheduling) azt jelenti, hogy egy külső folyamat időzítve indítja el a teljes DAG-futtatást, például naponta új adatpartíció beérkezése után. A cache és az ütemezés független: az ütemező minden nap futtathatja a DAG-ot, a cache pedig megakadályozza, hogy változatlan bemenetre újra számoljunk.

A visszamenőleges feldolgozás (backfill) olyan korábbi logikai dátumokra vagy partíciókra futtatja a DAG-ot, amelyek kimaradtak, vagy amelyekhez javított bemenet érkezett. A Northstar időhelyes jelentése itt is érvényes: ha a `label_available_at` értékek utólag teljesebbé válnak, akkor a backfill más címkéket láthat. A backfill tehát nemcsak a hiányzó futások újraindítása, hanem a predikció pillanatának (lásd 1.2. alfejezet és 21.1. alfejezet) explicit újrafelvétele is. Azt is meg kell őrizni, hogy a régi cache-kulcsok ne tűnjenek el: egy korábbi javítás előtti állapot visszaállításához a régi output még kellhet.

30.3.4 Teljes minimális helyi DAG

A következő szkript standard könyvtárat használ, ezért külön csomagtelepítés nélkül futtatható `orders_mini.csv` mellett. A fájlt a kód első futáskor létrehozza, így a példa önálló. Linux/macOS terminálban a parancs `python cache_pipeline.py`; Windows PowerShellben ugyanez. A projektben `uv run python cache_pipeline.py` is megfelel.

```
import csv
import hashlib
import inspect
import json
import platform
import time
from pathlib import Path
```

```
CACHE_DIR = Path("cache")
CACHE_DIR.mkdir(exist_ok=True)
MAX_ATTEMPTS = 3
RETRY_WAIT_SECONDS = 0.5
CODE_VERSION = "commit:1a2b3c4"
SCHEMA_VERSION = "orders-v1"
CORRECTION_VERSION = "corr-0"
ENVIRONMENT = {
    "python": platform.python_version(),
    "implementation": platform.python_implementation(),
    "platform": platform.system(),
}

mini_csv = "\n".join(
    [
        "order_id,customer_id,order_ts,quantity_sum,priority_flag",
        "o1,c1,2025-01-01T08:00:00+00:00,3,1",
        "o2,c1,2025-01-02T08:00:00+00:00,2,0",
        "o3,c2,2025-01-03T08:00:00+00:00,5,1",
        "o4,c2,2025-01-04T08:00:00+00:00,1,1",
        "o5,c1,2025-01-05T08:00:00+00:00,4,0",
        "o6,c3,2025-01-06T08:00:00+00:00,2,0",
    ]
)
Path("orders_mini.csv").write_text(mini_csv, encoding="utf-8")

def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def function_version(fn) -> str:
    return sha256_bytes(inspect.getsource(fn).encode("utf-8"))[:12]

def task_cache_key(task_name, run_fn, config, input_hashes,
                   correction_version=CORRECTION_VERSION):
    payload = {
        "task_name": task_name,
        "function_hash": function_version(run_fn),
        "code_version": CODE_VERSION,
        "schema_version": SCHEMA_VERSION,
        "config": config,
        "input_hashes": input_hashes,
        "environment": ENVIRONMENT,
        "correction_version": correction_version,
    }
    canonical = json.dumps(payload, sort_keys=True).encode("utf-8")
    return sha256_bytes(canonical)

def read_rows(path):
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

def extract(input_files, config):
    return read_rows(input_files[0])

def filter_priority(input_files, config):
    rows = json.loads(Path(input_files[0]).read_text(encoding="utf-8"))
    want = config["only_priority"]
    return [r for r in rows if int(r["priority_flag"]) == want]

def aggregate(input_files, config):
    rows = json.loads(Path(input_files[0]).read_text(encoding="utf-8"))
    totals = {}
```

```
    for r in rows:
        totals[r["customer_id"]] = totals.get(r["customer_id"], 0) + int(r["quantity_sum"])
    return totals

def run_task(task_name, config, input_files, run_fn,
             correction_version=CORRECTION_VERSION):
    input_hashes = {}
    for p in input_files:
        input_hashes[p.name] = sha256_bytes(Path(p).read_bytes())

    key = task_cache_key(task_name, run_fn, config, input_hashes,
                         correction_version)
    out = CACHE_DIR / f"{task_name}_{key}.json"

    for attempt in range(1, MAX_ATTEMPTS + 1):
        if out.exists():
            print(f"CACHE HIT  {task_name} {key[:12]}")
            return out, key, True

        print(f"CACHE MISS {task_name} {key[:12]} (attempt {attempt})")
        try:
            result = run_fn(input_files, config)
        except (OSError, RuntimeError) as exc:
            if attempt == MAX_ATTEMPTS:
                raise
            print(f"RETRY {attempt}/{MAX_ATTEMPTS}: {exc}")
            time.sleep(RETRY_WAIT_SECONDS)
            continue

        tmp = out.with_suffix(".tmp")
        tmp.write_text(json.dumps(result, indent=2), encoding="utf-8")
        tmp.replace(out)
        return out, key, False

def load_json(path):
    return json.loads(Path(path).read_text(encoding="utf-8"))

def run_pipeline(config):
    raw = Path("orders_mini.csv")
    extract_out, extract_key, hit_extract = run_task("extract", {}, [raw], extract)
    filter_out, filter_key, hit_filter = run_task(
        "filter", config, [extract_out], filter_priority
    )
    aggregate_out, aggregate_key, hit_aggregate = run_task(
        "aggregate", {}, [filter_out], aggregate
    )
    return (
        load_json(aggregate_out),
        (extract_key, filter_key, aggregate_key),
        (hit_extract, hit_filter, hit_aggregate),
    )

if __name__ == "__main__":
    result1 = run_pipeline({"only_priority": 1})
    print("priority=1:", result1[0])

    result2 = run_pipeline({"only_priority": 1})
    print("ismételt priority=1 hit-jei:", result2[2])
    assert result2[0] == result1[0]

    result3 = run_pipeline({"only_priority": 0})
    print("priority=0:", result3[0])
    assert result3[0] == {"c1": 6, "c3": 2}
```

A kódot nem futtattuk e generáláskor, ezért a következő napló a futtatás **szemléltető szerkezeti mintáját** mutatja. A tényleges hash-rövidítések a saját gépeden más hexa karakterek lesznek, mert az SHA-256 a teljes bájtfolyamot kódolja; a HIT/MISS mintázat viszont a fenti adatra így várható:

```
CACHE MISS extract <extract_hash_prefix> (attempt 1)
CACHE MISS filter <filter_hash_prefix> (attempt 1)
CACHE MISS aggregate <aggregate_hash_prefix> (attempt 1)
priority=1: {'c1': 3, 'c2': 6}

CACHE HIT  extract <ugyanaz_mint_fent>
CACHE HIT  filter <ugyanaz_mint_fent>
CACHE HIT  aggregate <ugyanaz_mint_fent>
ismételt priority=1 hit-jei: (True, True, True)

CACHE HIT  extract <ugyanaz_mint_fent>
CACHE MISS filter <más_filter_hash>
CACHE MISS aggregate <más_aggregate_hash>
priority=0: {'c1': 6, 'c3': 2}
```

30.3.5 Eredmény értelmezése és cache-invariáns

A kézi kontroll egyezik: `priority=1` esetén az o1, o3, o4 sorok maradnak, az ügyfelenkénti összeg c1-nél 3, c2-nél 6. `priority=0` esetén o2, o5, o6 marad, c1-nél $2 + 4 = 6$, c3-nál 2. A teljes hat sor mennyisége 17, és a két szűrt aggregátum értékkészlete együtt is 17; ez a jó invariáns arra, hogy a szűrés nem veszített duplikátumot.

A második futás mindhárom feladatnál HIT: ugyanaz a feladatverzió, paraméter és bemenet-hash. A harmadik futásnál az `extract` inputja nem változott, ezért HIT; a `filter` paramétere megváltozott, ezért MISS; a downstream `aggregate` bemenete a szűrt fájl hashével változott, ezért szintén MISS. A pontosan megadott érvénytelenítési szabály tehát: **ha a task verziója, paraméter-kanonikus alakja vagy bármely bemenet tartalomhashje eltér, a kulcs új, és a task újrafut**. Ugyanakkor a cache-ből vett kimenetre is érvényesek a 29.3. alfejezet adattesztjei; a gyorsítás nem helyettesíti az output séma- vagy invariáns-ellenőrzését.

Backfill esetén rögzíteni kell a forrásváltozatot és a feldolgozás időszeletét; az újrafuttatott részablak csak akkor egyezik a teljes újraszámítással, ha az előzményfüggőségeket is biztosítjuk.

30.3.6 Orchestrator áttekintés: Prefect, Dagster és a dbt szerepe

A fenti `run_task` keret kézzel mutatja meg, mit ad egy munkafolyamat-vezénylő (orchestrator): függőségi sorrend, paraméterkanonizálás, retry, naplózás és cache. Kis helyi kutatásban ez a kézi DAG bőven elég. Ha már ütemezés, UI, riasztás, párhuzamos ágak vagy több futtatókörnyezet kell, akkor érdemes valódi orchestratort bevezetni.

Eszköz	Fő fogalom	Mit vesz át?	Mikor indokolt?
Prefect	flow és task	retry, ütemezés, run-obszerabilitás, cache	ha a Python-DAG és a szerver nélküli indítás is fontos
Dagster	asset és materialization	adatasset-függőség, partition/backfill, I/O menedzsment	ha adatassetek és backfillek a központi kérdés

Eszköz	Fő fogalom	Mit vesz át?	Mikor indokolt?
dbt	`ref()` modellfa	SQL-transzformációk függőségi gráfja	ha a feldolgozás SQL-ben definiált, de a retry/backfill ettől még orchestrátorra marad

A fenti keretben a Prefect `task` dekorátora a `run_task` hívását csomagolja, a Dagster `@asset` pedig a feladatok kimenetét nevesített adatassetként kezeli. A konkrét dekorátorargumentumokat és verziókat a telepített változat dokumentációjából kell ellenőrizni; ez a könyv nem futtatta a Prefect/Dagster munkafolyamatokat. A dbt a 30.4. alfejezetben kerül részletesebben elő; most annyi a határa, hogy a dbt modell-definíció nem cache-mechanizmus, hanem függőségi leírás.

30.3.7 Feladatok

1. **Értelmezés.** A hibás kulcsgenerátor `key = task_name + output_fájl_neve` alakban dolgozik. Milyen inputváltozást nem vesz észre a mini CSV-n? Melyik hiányzó kulcsfeltételre vezethető vissza a hiba? Adj konkrét példát egy új sor hozzáadásával.
2. **Megvalósítás.** Általánosítsd a `filter_priority` feladatot `filter_rows` néven úgy, hogy a config `only_priority` mellett `min_quantity` egész értéket is kapjon, és csak azokat a sorokat tartsa meg, ahol `quantity_sum >= min_quantity`. Futtasd a `{"only_priority": 1, "min_quantity": 4}` paraméterrel a mini CSV-n. Mi a várt aggregált kimenet? Ellenőrizd, hogy a `min_quantity=0` változat a korábbi `only_priority=1` cache-től eltérő kulcsot kap.
3. **Hibadiagnózis.** A csapat a `filter_priority` függvényt úgy módosította, hogy a `priority_flag` egyezést kisebb-egyenlő relációra cserélte. A napló mégis `CACHE HIT`-et írt, és a régi kimenet jött vissza. Mit hagyott ki a kulcs, és mi a minimális javítás? Milyen adattesztet javasolsz 29.3. alfejezet alapján a javítás után?
4. **Megváltozott feltétel és backfill.** Egészítsd ki a mini CSV-t a sorral `o7,c2,2025-01-07T08:00:00+00:00,6,1`. Futtasd újra `{"only_priority": 1}`-gyel. Milyen HIT/MISS mintázatot vársz a három taskra, és mi lesz az aggregátum? Maradnak-e meg a régi cache-fájlok? Fogald meg, miért fontos ez a régi kulcsok megtartása egy backfill visszaállításakor.

30.3.8 Megoldások és indoklás

1. A hibás kulcs nem tartalmazza a bemeneti fájl tartalomhashét. Ha a mini CSV-be beszúrjuk a `o7,c2,...,6,1` sort, a fájlnév ugyanaz marad, tehát a `filter` a régi cache-ből HIT-et adna, és az új o7 sor nem jelenne meg az aggregátumban. A hiányzó feltétel az input hash. A javított kulcs minden bemeneti fájl bájtjának SHA-256-jét beépíti, ezért az új tartalom új kulcsot ad.
2. A `filter_rows` megvalósítása:

```
def filter_rows(input_files, config):
    rows = json.loads(Path(input_files[0]).read_text(encoding="utf-8"))
    kept = []
    for r in rows:
        ok_priority = int(r["priority_flag"]) == config["only_priority"]
        ok_quantity = int(r["quantity_sum"]) >= config["min_quantity"]
```

```
        if ok_priority and ok_quantity:
            kept.append(r)
    return kept
```

A `{"only_priority": 1, "min_quantity": 4}` paraméterrel az o1, o3 és o4 közül csak o3 marad, mert o1 quantity=3, o4 quantity=1; az aggregátum `{"c2": 5}`. A `min_quantity=0` és `only_priority=1` config kanonikus kulcsa eltér a `min_quantity=4` változatétól, mert a numerikus paraméter is a `config` része. A gyors ellenőrzés: a két `filter_` cache-fájl hexa rövidítése különbözik.

3. A hiba a task verziójának kihagyása. Ugyanaz a feladatnév, paraméter és input hash mellett a régi cache kimenete érvényesnek látszik, pedig a kód új logikája más eredményt adna. A minimális javítás: `TASK_VERSION = "v2"`-re emelés, mert a függvény viselkedése megváltozott. Ezután a kulcs megváltozik, és a filter újrafut. A javítás után egy fixture-alapú teszt is indokolt: állandó, kis bemenetre a szűrő elvárt sorszámát és az aggregátumot ellenőrizd. A zöld teszt a szoftver szerződését igazolja, a cache nem.

4. Az o7 hozzáadása után az `extract` input CSV bájtjai változnak, ezért extract MISS. A filter bemenete az új extract kimenet, így filter MISS; az aggregate downstream is MISS. A `priority=1` sorok: o1 (3), o3 (5), o4 (1), o7 (6) → c1=3, c2=12. A várt aggregátum tehát `{"c1":3,"c2":12}`. A régi cache-fájlok megmaradnak, mert kulcsukban a régi input hash szerepel. Ez teszi lehetővé, hogy a backfillnél egy korábbi adatpartícióra érvényes kimenet megmaradjon, és a javítást követően is összehasonlíthassuk a két állapotot. A régi kulcsok megtartása tehát nem hulladék, hanem a visszamenőleges követés része.

30.3.9 Gyakori tévedés

A leggyakoribb hiba: *„a cache-ben lévő output bizonyítja, hogy az aktuális bemenethez készült."* Ellenpélda a fenti 1. feladat: a `filter_default.json` fájl létezik, ezért a hibás kulcs HIT-et ad, holott a CSV közben új rendelést kapott. A HIT igazságtartalma csak akkor áll, ha a kulcs tartalmazza a bemenet tényleges hashét és a feladat verzióját; a fájlnév alapú cache ezt a feltételt nem teljesíti.

Rövid önellenőrzés: meg tudnád-e mondani egy `cache/filter_*.json` fájl nevéből, hogy melyik paraméterrel és melyik inputtartalommal készült? A válasz csak akkor igen, ha a hexa kulcsban a megfelelő hash szerepel, és ismered a kanonizálási szabályt; önmagában a fájlnév nem bizonyíték.

30.3.10 Rövid összefoglaló

A cache kulcs a feladat nevét, verzióját, a paraméterek kanonikus JSON-ját és minden bemenet SHA-256 hashét tartalmazza. Ha bármelyik változik, új kulcs jön létre, és a task újrafut; ha mindegyik azonos, a mentett output felhasználható. A retry idempotens, tranziens hibát kezel. A backfill korábbi logikai dátumokra vagy partíciókra futtat vissza; a régi kulcsok megtartása a visszaállítást és az összehasonlítást támogatja. A kézi DAG kis projektre elég; Prefect vagy Dagster akkor indokolt, ha ütemezés, UI és asset-megfigyelhetőség is kell. A következő 30.4. alfejezet a SQL-transzformációk és a dbt modellfa helyét vizsgálja ugyanebben a csővezetékben.

30.4 SQL-transzformáció és dbt helye

Belépő · szint: haladó · idő: 90 perc · eszköz: pandas, DuckDB

Előfeltétel: 30.3. alfejezet · Célkészség: SQL-transzformáció és dbt helye · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

30.4.1 Cél és konkrét helyzet

A feladat az elemzési transzformáció elválasztása a modellező Python-kódtól: a Northstar tranzakciós táblákból egy rendelésenként egy sort tartalmazó feature mart készül SQL-ben, és csak ez a mart megy a train/inference pipeline-ba. Bemenet a tiszta `customers.csv` (300 sor), `orders.csv` (2400 sor) és `order_items.csv` (4796 sor); a megfigyelési egység a martban **egy rendelés**. Cél: a mart sorszáma, tételaggregátumai és a szállítási díj összege egyezzen a forrástáblákból levezethető invariánsokkal. Előzetes becslés: a mart 2400 soros lesz, a customer-join egy-egy kapcsolat miatt nem változtatja a sorszámot, a tételszintű újraszámolt kosárérték rendelésenként egyezik a `basket_minor` értékkel. A dbt csak ezt a mart-definíciót teszi tesztelhetővé és újrafuttathatóvá; nem végez modellillesztést.

30.4.2 Fogalmi magyarázat: SQL-modell, source, test, materialization, lineage, adapter

SQL-modell (dbt model): dbt-ben a „modell” egy `SELECT` utasítás, amely egy táblát vagy nézetet definiál. Ez nem azonos a gépi tanulási modellel. A dbt a `SELECT`-ből generálja a `CREATE TABLE AS` vagy `CREATE VIEW AS` műveletet, és a modell neve lesz a kimeneti objektum neve. A modell hivatkozhat más modellekre a `{{ ref('masik_modell') }}` függvénnyel, nyers forrásokra pedig a `{{ source('sema', 'tabla') }}` függvénnyel.

Source: deklarált, külső tábla, amelyet nem a dbt hoz létre; a mi esetünkben a DuckDB-be már betöltött `northstar.orders`, `northstar.customers` és `northstar.order_items`. A source a nyers réteg, amelynek sémája a külső adatszerződésből származik. Sem a CSV-betöltés, sem a forrásrendszer elérése nem a dbt feladata; a dbt csak hivatkozik a meglévő táblákra.

Test: a dbt deklaratív ellenőrzéseket futtat. Generikus tesztek: `unique` (kulcs egyedi), `not_null` (kötelező mező), `relationships` (idegen kulcs a szülő táblába mutat), `accepted_values` (megengedett kategóriák). Egyedi tesztet is írhatunk: egy SQL lekérdezés, amely a hibás sorokat adja vissza; ha üres, a teszt sikeres. A mi kosáregyezési ellenőrzésünk ilyen egyedi teszt.

Materialization: a dbt kimeneti típusa. `view` (a SELECT-et minden lekérdezéskor futtatja), `table` (a SELECT eredményét fizikai táblába írja), `incremental` (csak az új részletet írja) és más adapter-specifikus változatok. A modellező Python-pipeline akkor kap stabil, ismételten olvasható bemenetet, ha a mart `table` materializationnel készül. A vékony staging-rétegeknél a `view` gyakran elég.

Lineage: a `ref` és `source` hívások függőségi éleket hoznak létre; ezekből irányított gráf áll össze. A dbt ezt használja a futtatási sorrendhez, és a `dbt docs` felületen megjeleníti. A lineage segít megválaszolni: ha a `stg_order_items` változik, mely mart-táblákat kell újraépíteni?

dbt adapter: a dbt magja és az adatbázis között álló komponens; a `dbt-duckdb` adapter például a DuckDB SQL-dialektusát és a DuckDB-hez illeszkedő materialization-t valósítja meg. A modellek nagyrészt hordozhatók, de a dátumfüggvények és a sémanyelvi részletek adapterfüggők lehetnek. A dbt-duckdb kombinációt ez a könyv keretjegyzete nem futtatta; a pontos telepítést és adapterverziót a hivatalos dbt-adapterdokumentáció alapján kell ellenőrizni.

30.4.3 Matematikai híd: adatmegmaradási invariánsok egy one-to-many join után

Jelölje egy adott `i` rendelés tételsorainak számát `s_i` (mértékegysége darab), a szállítási díját `f_i` (mértékegysége EUR-cent). Az `order_items` tábla összes sora

$$N_{\text{items}} = \sum_{i=1}^{N_{\text{orders}}} s_i,$$

ahol `N_orders = 2400` a rendelések száma. Ha a `shipping_fee_minor` oszlopot közvetlenül a tételszintű joinon keresztül összegezzük, az eredmény

$$\sum_{i=1}^{N_{\text{orders}}} s_i f_i,$$

ami nemnegatív díjak és legalább egy tétel rendelésenként esetén legalább akkora, mint a helyes rendelésszintű összeg

$$\sum_{i=1}^{N_{\text{orders}}} f_i,$$

és szigorúan nagyobb, ha van olyan rendelés, amelyre `s_i > 1` és `f_i > 0`. Hiányzó tételsor esetén belső csatlakoztatás a rendelést teljesen el is hagyhatja. Kézi számpélda: ha egy rendelésnél `f_i = 500` cent és `s_i = 3` tétel, a tételszintű join után `3 * 500 = 1500` cent jelenik meg az összegben, nem 500. Ugyanez a duplikáció érintené a `target_late` célváltozót is. A megoldás: a tételtáblát előbb rendelés szintre aggregáljuk, majd a rendelésszintű táblához joinoljuk. A három ellenőrizendő adatmegmaradási invariáns:

1. `|mart_order_features| = |orders| = 2400` sor.
2. Minden rendelésre `agg_basket_minor = basket_minor`, `agg_quantity = quantity_sum`, `agg_item_rows = item_rows`.
3. `SUM(mart.shipping_fee_minor) = SUM(orders.shipping_fee_minor)`.

30.4.4 Teljes minimális munkapélda

Az alábbi Python-szkript a DuckDB Python API-val fut; a DuckDB az alap tanulási stack része. Feltételezzük, hogy a tiszta CSV-k a `lab/data/` mappában vannak, és a munkakönyvtár a `lab/`; innen a fájlok `data/` előtaggal érhetők el. A kimenet a mellékelt adatszerződésből levezetett **elméletileg várt** érték; e fejezet írásakor nem tartozik hozzá tényleges futási jegyzék.

```
# mart_build.py -- Linux/macOS, munkakönyvtár: lab/
# Szemléltető minimális munkapélda; a futási kimenet a helyi CSV-k tartalmától függ.
import duckdb

con = duckdb.connect()

con.execute("""
CREATE OR REPLACE TABLE orders AS
SELECT * FROM read_csv_auto('data/orders.csv', header = true);

CREATE OR REPLACE TABLE customers AS
SELECT * FROM read_csv_auto('data/customers.csv', header = true);
```

```
CREATE OR REPLACE TABLE order_items AS
SELECT * FROM read_csv_auto('data/order_items.csv', header = true);

-- 1) Tételtáblából rendelésszintű aggregátum; nem joinolunk tételszinten.
CREATE OR REPLACE TABLE item_agg AS
SELECT
  order_id,
  SUM(quantity)                                  AS agg_quantity,
  COUNT(*)                                       AS agg_item_rows,
  SUM(quantity * unit_price_minor - discount_minor)
                                                 AS agg_basket_minor
FROM order_items
GROUP BY order_id;

-- 2) Rendelésenként egy sor: customer-join 1:1, item_agg-join 1:1.
CREATE OR REPLACE TABLE mart_order_features AS
SELECT
  o.order_id,
  o.customer_id,
  o.order_ts,
  o.promised_days,
  o.distance_km,
  o.priority_flag,
  o.basket_minor,
  o.shipping_fee_minor,
  o.target_late,             -- cél; a Python-pipeline explicit kiválogatja
  o.label_available_at,      -- split-ellenőrzéshez, nem prediktor
  ia.agg_quantity,
  ia.agg_item_rows,
  ia.agg_basket_minor,
  c.signup_date,
  c.segment,
  c.region,
  c.acquisition_channel,
  date_diff('day', c.signup_date, CAST(o.order_ts AS DATE))
                             AS account_age_days
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
LEFT JOIN item_agg ia
  ON o.order_id = ia.order_id;
""")

print("mart kontroll:")
print(con.execute("""
SELECT
  COUNT(*)                                              AS mart_count,
  COUNT(*) FILTER (WHERE agg_quantity IS NULL
                    OR agg_item_rows IS NULL
                    OR agg_basket_minor IS NULL)        AS missing_agg,
  COUNT(*) FILTER (WHERE agg_basket_minor <> basket_minor
                    OR agg_quantity <> quantity_sum
                    OR agg_item_rows <> item_rows)      AS mismatch_count,
  SUM(shipping_fee_minor)                               AS mart_shipping_sum
FROM mart_order_features;
""").fetchone())

print("source kontroll:")
print(con.execute("""
SELECT
  COUNT(*)                                               AS orders_count,
  SUM(shipping_fee_minor)                                AS orders_shipping_sum
FROM orders;
```

```
""").fetchone())
```

Elméletileg várt kimenet:

```
mart kontroll:
(2400, 0, 0, <orders_shipping_sum>)
source kontroll:
(2400, <orders_shipping_sum>)
```

A `mart_shipping_sum` és az `orders_shipping_sum` értéke a generált CSV-k konkrét összege; a kontroll invariáns az egyenlőség, nem az előre rögzített szám. A mart shape-je `(2400, 18)`: 18 oszlop, 2400 megfigyelés.

30.4.5 A dbt projektváza és a felelősségi térkép

A dbt-rész **opcionális bővítő**, nem a kezdőlabor telepített része. Az alábbi projektváz szemlélteti a struktúrát; a modellfájlok a fenti DuckDB SQL-lel logikailag azonos átalakítást írnak le. A dbt-duckdb adapter verzióját és a telepítési módot a hivatalos dokumentációban ellenőrizni kell; a parancsok Linux/macOS terminálban, `lab/feature_mart` munkakönyvtárban értendők, ha a projektet külön mappába másoljuk.

```
feature_mart/
  dbt_project.yml
  models/
    staging/
      stg_orders.sql
      stg_customers.sql
      stg_order_items.sql
    mart/
      mart_order_features.sql
      schema.yml
  tests/
    mart_money_invariants.sql
```

`dbt_project.yml`:

```yaml
name: feature_mart
version: "1.0"
profile: feature_mart
model-paths: ["models"]
test-paths: ["tests"]

models:
  feature_mart:
    staging:
      +materialized: view
    mart:
      +materialized: table
```

`models/staging/stg_orders.sql`:

```
SELECT
  order_id,
  customer_id,
  order_ts,
  promised_days,
  distance_km,
  priority_flag,
  basket_minor,
  shipping_fee_minor,
```

```
  quantity_sum,
  item_rows,
  target_late,
  label_available_at
FROM {{ source('northstar', 'orders') }}
```

models/staging/stg_order_items.sql:

```
SELECT
  order_id,
  line_id,
  product_id,
  quantity,
  unit_price_minor,
  discount_minor,
  quantity * unit_price_minor - discount_minor AS line_total_minor
FROM {{ source('northstar', 'order_items') }}
```

models/staging/stg_customers.sql:

```
SELECT
  customer_id,
  signup_date,
  segment,
  region,
  acquisition_channel
FROM {{ source('northstar', 'customers') }}
```

models/mart/mart_order_features.sql:

```
WITH item_agg AS (
  SELECT
    order_id,
    SUM(quantity)                  AS agg_quantity,
    COUNT(*)                       AS agg_item_rows,
    SUM(line_total_minor)          AS agg_basket_minor
  FROM {{ ref('stg_order_items') }}
  GROUP BY order_id
)
SELECT
  o.order_id,
  o.customer_id,
  o.order_ts,
  o.promised_days,
  o.distance_km,
  o.priority_flag,
  o.basket_minor,
  o.shipping_fee_minor,
  o.target_late,
  o.label_available_at,
  ia.agg_quantity,
  ia.agg_item_rows,
  ia.agg_basket_minor,
  c.signup_date,
  c.segment,
  c.region,
  c.acquisition_channel,
  date_diff('day', c.signup_date, CAST(o.order_ts AS DATE)) AS account_age_days
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_customers') }} c
  ON o.customer_id = c.customer_id
LEFT JOIN item_agg ia
  ON o.order_id = ia.order_id
```

A `date_diff` DuckDB-függvény; más adapteren esetleg `datediff(day, ...)` kell. A `models/mart/schema.yml` a generikus tesztekkel:

```yaml
version: 2

models:
  - name: mart_order_features
    description: "Rendelésenként egy sor; a Python train/inference pipeline ezt olvassa."
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: customer_id
        tests:
          - relationships:
              to: ref('stg_customers')
              field: customer_id
      - name: segment
        tests:
          - accepted_values:
              values: ['consumer', 'business']
      - name: region
        tests:
          - accepted_values:
              values: ['north', 'central', 'south']
      - name: priority_flag
        tests:
          - accepted_values:
              values: [0, 1]
      - name: basket_minor
        tests:
          - not_null
```

`tests/mart_money_invariants.sql`:

```sql
SELECT order_id
FROM {{ ref('mart_order_features') }}
WHERE COALESCE(agg_basket_minor, -1) <> basket_minor
   OR COALESCE(agg_quantity, -1)   <> quantity_sum
   OR COALESCE(agg_item_rows, -1)  <> item_rows
```

30.4.5.1 Forrásfrissesség, accepted values, relationship, business rule és as-of tesztek

A `unique` és a `not_null` szükséges, de nem elégséges. Öt további mintát érdemes beépíteni.

Source freshness. A forrás frissességét a `sources` blokkban lehet deklarálni; a `dbt source freshness` parancs ezt ellenőrzi:

```yaml
sources:
  - name: northstar
    schema: main
    loaded_at_field: _loaded_at   # a betöltés ideje, nem az order_ts
    freshness:
      warn_after: {count: 24, period: hour}
      error_after: {count: 48, period: hour}
    tables:
      - name: orders
      - name: customers
```

A `loaded_at_field` a betöltés időbélyege; ha csak az `order_ts` van, akkor a frissesség az eseményidőre vonatkozik, és a késést a 30.2. alfejezet `freshness_age` képletével kell számolni. A küszöb WARN és ERROR szintje külön szerepel, mert a késés lehet elfogadható vagy blokkoló.

Accepted values. A kategóriák zárt halmazát a `schema.yml` `accepted_values` tesztje ellenőrzi (a fenti példában `segment`, `region`, `priority_flag`). Ha a forrás új kategóriát vezet be, a teszt elbukik, és a döntés tudatos: vagy bővítjük a szerződést, vagy karanténba tesszük a sort.

Relationship. Az idegen kulcsot a `relationships` teszt ellenőrzi; a fenti `customer_id` példa a `stg_customers` táblára mutat. A relationship teszt a join sokasodását nem fogja meg, ezért kell mellé a `unique` az `order_id`-n és a kosáregyezési teszt.

Business rule. A pénzügyi és mennyiségi szabályokat egyedi SQL-teszt rögzíti:

```
-- tests/mart_business_rules.sql
SELECT order_id
FROM {{ ref('mart_order_features') }}
WHERE quantity_sum <= 0
   OR basket_minor < 0
   OR shipping_fee_minor < 0
   OR agg_quantity <= 0
```

As-of és feature-szerződés. Az időbeli címkeelérhetőséget két szinten kell védeni. A mart szintjén:

```
-- tests/mart_asof_contract.sql
SELECT order_id
FROM {{ ref('mart_order_features') }}
WHERE label_available_at < order_ts
   OR (target_late IS NOT NULL AND label_available_at IS NULL)
```

Ez a teszt azt fogja meg, ha a címke a döntés előtt ismertnek látszik, vagy ha a címke és a rendelkezésre állás összeomlik. A **feature-szerződés** szintjén a dbt nem látja a Python feature-listát, ezért a Python oldalon kell ellenőrizni:

```
FORBIDDEN_FEATURES = {"delivery_days", "actual_delivery_at",
                      "label_available_at", "target_late"}

def feature_contract(feature_names):
    return [name for name in feature_names if name in FORBIDDEN_FEATURES]
```

A `tests/test_dbt_gates.py` a fenti öt kaput teszteli: frissesség WARN/ERROR, érvénytelen kategória, árva idegen kulcs, negatív pénzösszeg, jövőbeli címkeidő és tiltott feature. A duplikált `order_id` és a jövőbeli attribútum így nem jut el a feature martba: előbbit a `unique`, utóbbit az as-of és a feature-szerződés fogja meg.

Futtatás szemléltető parancsokkal:

```
# Linux/macOS, munkakönyvtár: lab/feature_mart (opcionális projekt)
dbt run --select staging
dbt run --select mart
dbt test --select mart
dbt docs generate
dbt docs serve
```

A lineage gráf a `ref`/`source` élekből áll össze: `stg_order_items` → `item_agg` CTE a martban, `stg_orders` és `stg_customers` → `mart_order_features`. A `dbt docs` ezt jeleníti meg; az alábbi táblázat a felelősségi határokat rögzíti:

Réteg	Felelősség	Tipikus kimenet	Nem ez a dolga
Betöltő/ingest	CSV/forrás beolvasás, séma rögzítés	DuckDB nyers táblák	üzleti definíció
Staging	típus, tiszta elnevezés, source-teszt	`stg_*` view	aggregálás, üzleti logika
Mart	granularitás, join, feature-invariáns	`mart_order_features` table	tanítás, split, metrika
Python ML-pipeline	split, preprocessing, tanítás, kiértékelés	artifact, `metrics.json`, `split.json`	join-javítás, nyers tisztítás

30.4.6 Eredmény értelmezése

A mart kontrollja három dolgot ellenőriz: a 2400 soros szemcsézettséget, a tételszintű aggregátumok kosáregyezését és a szállítási díj nem többszöröződését. Ha a `missing_agg` vagy a `mismatch_count` 0, a mart rendelésszintű, és a tételtáblából származó oszlopok megegyeznek a rendeléstábla szerződött értékeivel. A `mart_shipping_sum = orders_shipping_sum` egyenlőség az invariáns; ha valaki tételszinten joinolt volna, a martösszeg a rendelésösszegnek a tételszámokkal súlyozott torzított változata lenne. A Python-modellező kód ezután nem végez join-javítást; a bemenet a mart szerződéséből jön. A `target_late` és `label_available_at` oszlopok a martban metaadatok; a train/inference kód felel azért, hogy a `target_late`-ot címkének, ne prediktornak használja. A séma-eltérést a modellező kód a 26.4. alfejezet szerinti bemeneti validációval külön ellenőrzi.

30.4.7 Feladatok

1. **Értelmezés.** Egy kolléga a `stg_order_items`-et közvetlenül a `stg_orders`-hez joinolja, és a mart tételszintű lesz. Egy olyan rendelésnél, amelynek 3 tételsora van, a `shipping_fee_minor` 500 cent, a `target_late` 1. Hányszor jelenik meg ez a rendelés a rossz martban, és mennyi lesz a szállítási díj összege ehhez a rendeléshez? Melyik invariáns sérül?

2. **Implementálás.** Írd meg a dbt `staging/agg_order_items.sql` modellt, amely a `stg_order_items`-ből rendelésszintű `agg_quantity`, `agg_item_rows`, `agg_basket_minor` oszlopokat készít. Add meg a hozzá tartozó `schema.yml` tesztet: `order_id` legyen `unique` és `not_null`, és a három aggregált oszlop legyen `not_null`.

3. **Hibadiagnózis.** Egy módosított mart-modellben a `unique` teszt nem szerepel az `order_id` oszlopon, és nincs kosáregyezési teszt. A tábla 4796 sort tartalmaz. A `dbt test` zöld. Mi a konkrét tünet, és milyen SELECT-lel igazolod, hogy a mart rossz? Add meg a javított mart-SELECT lényegi részét.

4. **Megváltozott feltétel.** Új feature-t kell bevezetni: `customer_order_count_before` — az ügyfél korábbi, a rendelés pillanatában **már teljes mértékben megfigyelt** rendeléseinek száma. Írd meg a módosított `stg_orders` SELECT-et, amely a `label_available_at < aktuális order_ts` feltétellel szűri a korábbi rendeléseket. Adj egy egyedi dbt-tesztet arra, hogy a feature minden sorban nemnegatív és az ügyfél legkorábbi rendelésénél 0.

30.4.8 Megoldások és indoklás

1. A rossz mart minden rendeléshez annyi sort ad, ahány tételsora van: 3 sort. A `shipping_fee_minor` a joinban háromszorozódik, így az adott rendeléshez tartozó szállítási díjak összege `3 * 500 = 1500` cent, nem 500. A `target_late = 1` szintén három sorban jelenik meg. A megsértett invariáns: `|mart| = |orders| = 2400`, és a szállításidíj-összegmegmaradás is sérül. A helyes eljárás a tételtábla előzetes `GROUP BY order_id` aggregálása.

2. A modell:

```
-- models/staging/agg_order_items.sql
SELECT
  order_id,
  SUM(quantity)                                  AS agg_quantity,
  COUNT(*)                                       AS agg_item_rows,
  SUM(quantity * unit_price_minor - discount_minor)
                                                 AS agg_basket_minor
FROM {{ ref('stg_order_items') }}
GROUP BY order_id
```

A tesztkonfiguráció:

```
version: 2

models:
  - name: agg_order_items
    description: "Rendelésenként egy sor: a tételszintű oszlopok aggregátumai."
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: agg_quantity
        tests:
          - not_null
      - name: agg_item_rows
        tests:
          - not_null
      - name: agg_basket_minor
        tests:
          - not_null
```

Ellenőrző invariant: a modell sorainak száma 2400, és a `SUM(agg_basket_minor)` egyezik a `stg_orders.basket_minor` oszlop összegével.

3. A tünet: `COUNT(*) = 4796`, miközben `COUNT(DISTINCT order_id) = 2400`; a mart soronként nem rendelés, hanem tétel. A szállítási díj összege nagyobb a rendeléstábla összegénél. Igazoló SELECT:

```
SELECT
  COUNT(*)                AS bad_mart_rows,
  COUNT(DISTINCT order_id) AS distinct_orders
FROM mart_order_features;
```

Elvárt diagnózis: `bad_mart_rows = 4796`, `distinct_orders = 2400`. A javítás:

```
WITH item_agg AS (
  SELECT order_id,
         SUM(quantity)     AS agg_quantity,
         COUNT(*)          AS agg_item_rows,
         SUM(line_total_minor) AS agg_basket_minor
  FROM {{ ref('stg_order_items') }}
```

```
  GROUP BY order_id
)
SELECT ...
FROM {{ ref('stg_orders') }} o
LEFT JOIN item_agg ia ON o.order_id = ia.order_id
```

és a `unique` teszt visszaállítása az `order_id` oszlopra. A zöld `dbt test` nem a modell helyességét bizonyította, hanem azt, hogy a definiált tesztek nem futottak a védendő invariánsra; ez a 29.4. alfejezetbeli review-elv: a tesztlefedettség a kódminőségi ellenőrzés része.

4. A módosított staging SELECT:

```
SELECT
  o.order_id,
  o.customer_id,
  o.order_ts,
  o.promised_days,
  o.distance_km,
  o.priority_flag,
  o.basket_minor,
  o.shipping_fee_minor,
  o.quantity_sum,
  o.item_rows,
  o.target_late,
  o.label_available_at,
  (SELECT COUNT(*)
   FROM {{ ref('stg_orders') }} prev
   WHERE prev.customer_id = o.customer_id
     AND prev.order_ts < o.order_ts
     AND prev.label_available_at < o.order_ts
  ) AS customer_order_count_before
FROM {{ ref('stg_orders') }} o
```

Fontos: a `label_available_at < o.order_ts` szűrés biztosítja, hogy a predikció pillanatában a korábbi rendelés kimenete már ténylegesen ismert; enélkül a jövőből szivárogna be címkeinformáció a feature-be. Ez a feltétel eltér a megszokott `prev.order_ts < o.order_ts` szűréstől, mert egy korábbi rendelés kézbesítése késhet. Egyedi tesztek:

```
-- tests/customer_order_count_before_is_nonnegative.sql
SELECT order_id
FROM {{ ref('stg_orders') }}
WHERE customer_order_count_before < 0;

-- tests/first_order_has_zero_history.sql
SELECT order_id
FROM {{ ref('stg_orders') }}
WHERE customer_order_count_before <> 0
  AND order_id IN (
    SELECT order_id
    FROM (
      SELECT order_id,
             ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_ts
             ) AS rn
      FROM {{ ref('stg_orders') }}
    ) ranked
    WHERE rn = 1
  );
```

A második teszt csak az ügyfél legkorábbi rendeléseit vizsgálja; ezeknél a feature kötelezően 0. A tesztelendő új oszlop dimenziója egész szám, tartománya nemnegatív.

30.4.9 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés: a dbt-t általános ML-optimalizálónak vagy adatbetöltőnek tekinteni. A dbt-ben a „model” SQL-transzformáció, nem illesztett gépi tanulási modell; nincs `dbt run --select logistic_regression`, mert a logisztikus regresszió a Python-pipeline artifactje, nem a dbt gráf csúcsa. A dbt nem gyűjt be minden adatforrást: a CSV-betöltés és a DuckDB-tábla létrehozása a source előtt külön ingest-réteg; a `dbt seed` csak kis referenciaadatokra való, nem a teljes tranzakciós adattóra. A dbt nem a randomizált experiment eszköze sem: a Northstar `assignments`/`outcomes` táblái külön kísérleti folyamatból jönnek, és nem az `orders` utólagos átalakításából, ahogyan a 18.4. alfejezet is hangsúlyozza. Önellenőrzés: meg tudom-e magyarázni, miért marad 2400 sor a customer-join után? Meg tudom-e írni a kosáregyezési tesztet `COALESCE`-szel? Meg tudom-e nevezni azt a plusz feltételt, amely az előzményfeature-nél a jövőbeli címkeszivárgást akadályozza? Ha igen, a felelősségi határ világos.

30.4.10 Rövid összefoglaló

A dbt az elemzési SQL-transzformációt teszi tesztelhető, újrafuttatható és lineage-szerűen átlátható modellé. A source a már betöltött nyers táblára mutat; a staging nézetek tisztítanak; a mart tábla rendelésenként egy sorban, tételszintű aggregátummal és explicit célváltozóval készül. A generikus tesztek a kulcsot, idegen kulcsot és kategóriákat védik; egyedi tesztek a kosáregyezést és a szállításidíj-invariánst ellenőrzik. A join-kardinalitási hiba a tételszintű rendelésduplikációval mutatható ki. A következő alkalmazásban a 31.1. alfejezet kísérletkövető táblázatnak a mart verzióját és a splitmanifestet is rögzítenie kell, mert a futásazonosító önmagában nem azonosítja a modell bemeneti feature-tábláját.

31 Kísérletkövetés, adat- és modellverziózás

31.1 Futásazonosító és kísérleti táblázat

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: MLflow, DVC, Git

Előfeltétel: 30.4. alfejezet · Célkészség: Futásazonosító és kísérleti táblázat · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

31.1.1 Cél és konkrét helyzet

A Northstar predikciós projektben a korábbi egységekben felépítettünk egy időhelyes splitet (30.1. alfejezet), egy predikciós szerződést (21.4. alfejezet) és egy küszöbválasztási protokollt (23.4. alfejezet). Most arra a kérdésre válaszolunk: hogyan kötjük a kutatási állítást a tényleges futáshoz? A konkrét feladat az, hogy a `starter_experiment.py` két baseline-futását – egy konstans (tanítóarányos) és egy `distance_km` küszöbös baseline-t – úgy dokumentáljuk, hogy később bárki meg tudja mondani, melyik kód, melyik adat, melyik paraméter és melyik környezet állította elő az adott metrikát.

A bemenet a lab `artifacts/split.json` és a feature-fájlok, a megfigyelési egység a rendelés, a célváltozó a `target_late` indikátor. A manifest célja: minden futáshoz rögzíteni a paramétereket, a metrikákat, a bemeneti fájlok hashét, az artefaktumokat és a futtató környezetet. Előzetes becslésként a konstans baseline log loss értéke nagyjából a tanítóbeli pozitív arányból adódó 0.68–0.72 körüli érték; a distance baseline attól való eltérése a kutatási kérdés, nem előre ismert eredmény.

31.1.2 Fogalmi magyarázat: futás, kísérlet és azonosítás

A **futás** (run) egyetlen végrehajtás, amelynek rögzített a kódja, a bemeneti adata, a paraméterei és a kimeneti artefaktumai. A **kísérlet** (experiment) olyan futások célzott halmaza, amelyek ugyanarra a kérdésre válaszolnak, és csak a tervezett paraméterekben térnek el egymástól. A kettő közötti különbség nem technikai bürokrácia, hanem a kutatási napló szerkezete: ha ma csak két baseline-t futtatsz, a kísérlet neve lehet `northstar-late-baselines-v1`; ha holnap egy modellcsaládot is hozzáadsz, az új kísérlet lesz.

A **futásazonosító** (run ID) két részből áll: egy ember által olvasható névből és egy rövid hashből. A hash fontos tervezési döntése, hogy mit tartalmaz. A metrikákat nem szabad beletenni, mert azokat a futás után számítjuk ki, és a futásnak már akkor azonosíthatónak kell lennie, amikor az értékelés még nem fejeződött be. Ezért a hash az úgynevezett *identity materialból* készül: a paraméterek kanonikus JSON-sorrendbe rendezett reprezentációjából és a bemeneti fájlok hashéből. A metrika később hozzáadódhat a manifesthez anélkül, hogy a futás identitása megváltozna.

A **paraméterek** (parameters) a futás előtt ismert, a kód által vezérelt beállítások: például a `strategy`, a `cutoff` vagy a distance küszöb. A seed is paraméter. A bemeneti adatfájl

viszont nem paraméter, hanem input: az `input_hash` mező azonosítja. Ez a különbségtétel azért fontos, mert ha a split-fájl megváltozik, a manifest új futást igényel, akkor is, ha a paraméterek azonosak.

A **metrikák** (metrics) futás utáni, az értékelési protokollt követő skalár összegzések. A metrika csak a definíciójával együtt értelmezhető, ezért a manifestben a `metric_definitions` mezőt is rögzítjük. A korábbi egységekben használt két metrika:

$$L_{\log} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \qquad B = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2,$$

ahol $y_i \in \{0,1\}$ a megfigyelt címke, $p_i \in (0,1)$ a prediktált valószínűség az i-edik tesztsorban, N a tesztsorok száma, és mindkét kimenet $(1,)$ skalár. A Brier-pontszám mindig $[0,1]$-ben van, a log loss viszont elvileg nem korlátos, amikor a predikció a megfigyeléssel ellentétes szélére kerül; a gyakorlati implementáció $\varepsilon = 10^{-12}$ levágást használ. A `metric_definitions` rögzíti a pozitív osztályt (`target_late == 1`), a küszöböt és az ε értékét is.

Az **artefaktumok** (artifacts) a futás által kiírt fájlok: a predikciós CSV, a betanított pipeline, esetleg egy ábra. A manifest nem tartalmazza magukat a fájlokat, csak a nevüket és a tartalomhashüket. Az artefaktumot immutábilisnak tekintjük; ha újra akarjuk futtatni, új manifest készül, nem felülírjuk a régit.

A **környezet** (environment) leírása három szinten fontos. A numerikus reprodukálhatóság az azonos seedet és a lebegőpontos műveletek azonosságát jelenti; a statisztikai reprodukálhatóság ugyanazt az eloszlásból vett becslést; a környezeti reprodukálhatóság a Python-verziót, a platformot, a zárolt függőségeket és a Git-revíziót rögzíti. A manifest mindig a harmadik szintet írja le, ebből a másik kettő nem következik automatikusan.

MLflow, DVC vagy hash-alapú manifest? A hash-alapú JSON-manifest kis lokális projektekben elég, és semmilyen szolgáltatást nem igényel. Az MLflow ugyanezt a fogalmat szolgálja ki paraméter-, metrika- és artefaktumlogolással, lokális mappába is; a DVC elsősorban a nagy adatfájlok tartalomcímzését és tárolását végzi. A főút ebben a fejezetben a manifest; az MLflow-t opcionális rétegként mutatjuk, de a keretjegyzet alapján a telepített verzió viselkedését külön ellenőrizni kell.

31.1.3 Teljes minimális munkapélda

A következő példa standard libraryt használ, így a manifest fogalma csomagtelepítés nélkül ellenőrizhető. Az adatok szándékosan kicsik: négy tesztsorral mutatjuk be, hogy két baseline ugyanolyan accuracy-t adhat, miközben a log loss nagyságrendileg különbözik. A valós labban ugyanez a funkció a `lab/pyproject.toml` numerikus csomagjaival fut; a fájlneveket a lab dokumentációja adja.

```
import csv, hashlib, json, math, pathlib, platform, sys, datetime

EPS = 1e-12

def clip_log_loss(y_true, p_pred):
    p = [max(EPS, min(1 - EPS, v)) for v in p_pred]
    return -sum(y * math.log(pi) + (1 - y) * math.log(1 - pi)
                for y, pi in zip(y_true, p)) / len(y_true)

def brier(y_true, p_pred):
    return sum((pi - y) ** 2 for pi, y in zip(p_pred, y_true)) / len(y_true)
```

```
def accuracy(y_true, p_pred):
    return sum((1 if pi >= 0.5 else 0) == y for pi, y in zip(p_pred, y_true)) / len(y_true)

def sha256_text(s):
    return hashlib.sha256(s.encode("utf-8")).hexdigest()

# kis példaadatok
train_y = [0, 1, 0, 1, 1, 0]
test_y = [1, 0, 0, 1]
test_ids = ["T7", "T8", "T9", "T10"]
distance_km = [70, 45, 90, 30]
cutoff = "2025-05-01T00:00:00+00:00"
positive_class = "target_late == 1"

def make_predictions(strategy):
    if strategy == "constant":
        return [sum(train_y) / len(train_y)] * len(test_y)
    if strategy == "distance":
        threshold = 60.0
        return [1.0 if d > threshold else 0.0 for d in distance_km]
    raise ValueError(f"unknown strategy {strategy}")

def build_manifest(strategy, preds, artifact_name, artifact_hash):
    split_payload = {
        "train_y": train_y,
        "test_y": test_y,
        "cutoff": cutoff,
    }
    split_sha256 = sha256_text(json.dumps(split_payload, sort_keys=True))
    data_version = sha256_text(json.dumps(
        {"train_y": train_y, "test_y": test_y, "distance_km": distance_km},
        sort_keys=True,
    ))
    input_source = {
        "train_y": train_y,
        "test_y": test_y,
        "distance_km": distance_km,
        "cutoff": cutoff,
    }
    input_hash = sha256_text(json.dumps(input_source, sort_keys=True))
    params = {"strategy": strategy, "seed": 20261020}
    if strategy == "distance":
        params["threshold"] = 60.0
    identity_material = f"{json.dumps(params, sort_keys=True)}|{input_hash}"
    short_hash = sha256_text(identity_material)[:8]
    run_id = f"baseline_{strategy}_{short_hash}"
    metrics = {
        "log_loss": clip_log_loss(test_y, preds),
        "brier": brier(test_y, preds),
        "accuracy": accuracy(test_y, preds),
    }
    return {
        "manifest_version": "1.0",
        "run_id": run_id,
        "experiment": "northstar-late-baselines-v1",
        "created_at": datetime.datetime.now(datetime.timezone.utc).isoformat(),
        "split_id": "northstar-time-2025-05-01",
        "split_sha256": split_sha256,
        "data_version": data_version,
        "code_version": "UNCOMMITTED-placeholder",
        "parameters": params,
        "metrics": metrics,
        "input_hash": input_hash,
```

```
        "artifacts": {
            "predictions_csv": artifact_name,
            "predictions_csv_sha256": artifact_hash,
        },
        "metric_definitions": {
            "log_loss": "-mean(y*log(p)+(1-y)*log(1-p)), y in {0,1}, p in (0,1), eps=1e-12",
            "brier": "mean((p-y)^2)",
            "accuracy": "mean(1[p>=0.5]==y)",
            "positive_class": positive_class,
        },
        "environment": {
            "python": sys.version.split()[0],
            "platform": platform.platform(),
            "lab_requires": "lab/pyproject.toml",
            "git_rev": "UNCOMMITTED-placeholder",
        },
    }

def save_predictions(path, y_true, preds):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["row_id", "y_true", "p_late"])
        for row_id, y, p in zip(test_ids, y_true, preds):
            w.writerow([row_id, y, p])

def main():
    out_dir = pathlib.Path("runs")
    out_dir.mkdir(exist_ok=True)
    manifests = []
    for strategy in ("constant", "distance"):
        preds = make_predictions(strategy)
        artifact_name = f"preds_{strategy}.csv"
        artifact_path = out_dir / artifact_name
        save_predictions(artifact_path, test_y, preds)
        artifact_hash = sha256_text(artifact_path.read_text(encoding="utf-8"))
        manifest = build_manifest(strategy, preds, artifact_name, artifact_hash)
        manifest_path = out_dir / f"{manifest['run_id']}.json"
        manifest_path.write_text(json.dumps(manifest, indent=2), encoding="utf-8")
        manifests.append(manifest)
    print(json.dumps(manifests, indent=2))

if __name__ == "__main__":
    main()
```

A fenti kódot a kapcsolódó példaprojektben még nem ellenőriztük; az alábbi kimenet ezért **szemléltető**, a kód logikájából számított érték. Az olvasó saját gépén a futás a Python 3.12 szemantikája szerint ugyanezt adja, feltéve, hogy a példaadatok azonosak. A manifest egyes mezői így nézhetnek ki a konstans baseline-ra:

```
"run_id": "baseline_constant_4e17c2a1",
"parameters": {"strategy": "constant"},
"metrics": {"log_loss": 0.6931, "brier": 0.25, "accuracy": 0.5},
"metrics": {"log_loss": 13.8155, "brier": 0.5, "accuracy": 0.5}
```

A distance baseline-nál a log loss a várt módon magas, mert a példa tesztkészletén a küszöbös predikció az ellenkező címkét kapja meg. A `run_id` rövid hashje a paraméterekből és az input hashből adódik, nem a metrikákból; ezért a metrikák későbbi pontosítása nem változtatja meg a futás azonosítóját.

31.1.4 Eredmény értelmezése

Kézi kontroll: $p = 0.5$ esetén minden tesztsor tagja $-\log 0.5 = 0.6931$, így a konstans log loss 0.6931, a Brier pedig $(0.5-0)^2 = (0.5-1)^2 = 0.25$. A distance baseline esetén a predikciók $[1, 0, 1, 0]$, a megfigyelések $[1, 0, 0, 1]$, ezért a 8. és a 10. sorban a modell $p = 0$-t vagy $p = 1$-et mond a tényleges ellenkezőjére; a levágás után a log loss $\frac{1}{4}(10^{-12} + 27.63 + 10^{-12} + 27.63) \approx 13.82$. Mindkét baseline accuracy értéke 0.5, tehát az accuracy önmagában nem tudja megkülönböztetni a kettőt; a manifestben rögzített log loss és Brier viszont igen.

Invariáns ellenőrzés: minden metrika a tesztsorokra számítva $(1,)$ skalár; a Brier mindig $[0, 1]$-ben van; a `run_id` utolsó 8 karaktere megegyezik a `identity_material` SHA-256 hashének első 8 karakterével. Ha a split-fájl egy bájtja megváltozik, az `input_hash` megváltozik, és a `run_id` is új futást jelöl.

31.1.4.1 A kötelező run_manifest séma

A `run_id` önmagában nem bizonyítja, hogy két futás ugyanaz a kísérlet. Minden artefaktumhoz tartozzon legalább a következő séma:

Mező	Példa	Kötelező tartalom
`manifest_version`	`"1.0"`	a séma verziója
`run_id`	`"baseline_constant_4e17c2a1"`	a futás stabil azonosítója
`experiment`	`"northstar-late-baselines-v1"`	a kísérlet neve
`created_at`	ISO-8601 UTC	a futás időpontja
`split_id`, `split_sha256`	`"northstar-time-2025-05-01"`, SHA-256	a split szemantikai és tartalmi azonosítója
`data_version`	SHA-256	a forrásadatok tartalmi verziója
`code_version`	commit vagy forráshash	a kód verziója
`parameters`	`{"strategy": "...", "seed": 20261020}`	minden futásvezérlő paraméter, beleértve a seedet
`input_hash`	SHA-256	a bemenet kanonikus hash-e
`environment`	Python, platform, függőségek	a futtató környezet
`metric_definitions`	szöveges definíciók	a metrika jelentése
`metrics`	log loss, Brier, accuracy	a futás utáni eredmények
`artifacts`	név + SHA-256	minden kimeneti fájl integritása

A kötelező mezők ellenőrzése:

```
REQUIRED_MANIFEST = {
    "manifest_version", "run_id", "experiment", "created_at",
    "split_id", "split_sha256", "data_version", "code_version",
    "parameters", "input_hash", "environment", "metric_definitions",
    "metrics", "artifacts",
}

def validate_manifest(manifest):
    errors = [f"hiányzó mező: {key}"
              for key in sorted(REQUIRED_MANIFEST - set(manifest))]
    if "seed" not in manifest.get("parameters", {}):
        errors.append("hiányzó seed a parameters mezőben")
```

```
    if not manifest.get("split_id") or not manifest.get("split_sha256"):
        errors.append("hiányzó split-azonosító")
    artifacts = manifest.get("artifacts", {})
    if not artifacts:
        errors.append("nincs artefaktum")
    for name, value in artifacts.items():
        if name.endswith("_sha256") or name.endswith("_hash"):
            continue
        if not any(key.startswith(name) and key.endswith(("_sha256", "_hash"))
                   for key in artifacts):
            errors.append(f"artefaktum hash nélkül: {name}")
    return errors
```

Az invariáns: két különböző forrásadattal futó kísérlet azonos `experiment` és `run_id` címke mellett sem tekinthető azonosnak, mert a `data_version`, a `split_sha256` és az `input_hash` eltér. A `validate_manifest` üres listát ad a teljes manifestre, és hibát jelez, ha a seed, a split-azonosító vagy valamelyik artefaktum hash-e hiányzik. A `tests/test_run_manifest.py` ezt a sémát és a két különböző adatverzió elutasítását teszteli.

Az MLflow opcionális rétegként ugyanezt a manifestet írhatja egy helyi `mlruns` mappába:

```
# opcionális, nem ellenőrzött ebben a kéziratban
import mlflow

mlflow.set_tracking_uri("file://./mlruns")
mlflow.set_experiment("northstar-late-baselines-v1")
with mlflow.start_run(run_name=manifest["run_id"]):
    mlflow.log_params(manifest["parameters"])
    mlflow.log_metrics(manifest["metrics"])
    mlflow.log_artifact(str(artifact_path))
```

A pontos `mlflow` API-viselkedés a telepített verziótól függ; ha az olvasó használni akarja, a `lab/pyproject.toml`-ben szereplő függőségek és a forrásjegyzék `mlflow` kulcsa alapján ellenőrizze. A manifest e nélkül is teljes.

31.1.5 Feladatok

1. **Értelmezés.** Egy manifest csak a következő mezőket tartalmazza:

   ```
   {"metrics": {"log_loss": 0.42}, "parameters": {"strategy": "distance"}}
   ```

   Miért nem lehet eldönteni, hogy a `0.42` jó-e? Sorolj fel legalább négy hiányzó mezőt, és magyarázd el mindegyik szerepét egy-egy mondatban.

2. **Megvalósítás.** Egészítsd ki a példakódot egy `priority` baseline-nal: rögzített paraméterekkel $p = 0.7$, ha a sor `priority_flag=1`, és $p = 0.2$, ha `priority_flag=0$. A példa tesztadataihoz add meg` apriority_flag $= [1, 0, 1, 1]$`oszlopot. Számítsd ki a három metrikát, és írd be a manifestet` aruns/‘ mappába.

3. **Hibadiagnózis.** A manifestben a `run_id` értéke `baseline_constant_a1b2c3d4`, de amikor a tanuló újraszámítja az `identity_material` hashét, az `e9f8a7b6`-ot ad. Két lehetséges okot is talál a kódban: az egyik a paraméterdict sorrendje, a másik a metrikák beemelése az anyagba. Melyik a helyes diagnózis, és hogyan javítod a kódot?

4. **Megváltozott feltétel.** A lab kutatási kérdése a cutoffot `2025-05-01`-ről `2025-06-01`-re helyezi. Mely manifestmezők változnak biztosan, és mi a helyes eljárás a régi futás manifestjével? Mondd meg, miért nem elég a régi metrikát felülírni.

31.1.6 Megoldások és indoklás

1. A manifest hiányos. Szükséges legalább: `input_hash`, amely a split és a feature-fájlok tartalmát azonosítja; `metric_definitions`, amely a log loss képletét, a pozitív osztályt és az eps levágást rögzíti; `environment`, amely a Python- és csomagverziókat adja; `artifacts`, amely a predikciós CSV hashét hordozza; és `experiment/run_id`, amely a futást a kísérleti táblázatba köti. A `0.42` önmagában csak egy szám: ha a tesztkészlet más, a metrika összehasonlíthatatlan; ha a definíció más, a jelentése más.

2. Az új predikció tömböt a `make_predictions`-höz hasonlóan állítjuk elő:

   ```
   priority_flag = [1, 0, 1, 1]
   def make_priority_predictions():
       return [0.7 if f == 1 else 0.2 for f in priority_flag]
   ```

   A metrikák: a tesztsorok $[1, 0, 0, 1]$ címkéivel a log loss a fenti levezetésből 0.535, a Brier 0.178, az accuracy 0.5. A manifest `parameters` mezője `{"strategy": "priority", "p_flag_1": 0.7, "p_flag_0": 0.2}`, az input hash az új oszloppal újra számítandó. Ellenőrzés: a `run_id` hashje megváltozik a konstanshoz képest; a CSV artefaktumnak is új hashje van.

3. A helyes diagnózis a metrikák beemelése. Ha az `identity_material` a teljes manifestből készül, a metrikák utólagos módosítása megváltoztatja a `run_id`-t, ami ellentmond a futás stabil azonosíthatóságának. A paraméterdict sorrendje önmagában is okozhat eltérést, ezért `sort_keys=True` kell; de a metrikák kizárása a tervezési hiba. A javítás: az anyag csak `parameters` és `input_hash` legyen, ahogy a példakód teszi.

4. Változnak: `parameters["cutoff"]`, az `input_hash` (az új split miatt), a `run_id` hashje, a metrikák, az artefaktumok hashjei, és a `created_at` új időpontot kap. A helyes eljárás: a régi manifestet nem módosítjuk; a kísérlet új futást kap, és a kísérleti táblázat appendel, nem update-el. A régi metrika felülírása azért hibás, mert a futás immutábilis bizonyíték: ha valaki a régi állítást akarja reprodukálni, látnia kell, milyen splittel készült. A kísérleti naplóban a két cutoff két külön sort jelent, és az összehasonlítás csak ezt a különbséget kezelve érvényes.

31.1.7 Gyakori tévedés ellenpéldával

„A logolt metrika önmagában összehasonlítható.” Ellenpélda: a fenti példában a konstans és a distance baseline accuracy értéke egyaránt 0.5. Ha a manifest csak az accuracy-t tartalmazná, a kutatás azt hinné, hogy a két baseline egyforma; a log loss értéke 0.693 illetve 13.82 viszont megmutatja, hogy a distance baseline a kis példán katasztrofálisan rossz. A metrika csak az értékelési protokollal – split azonosító, küszöb, pozitív osztály, definíció – együtt értelmezhető. Ugyanez vonatkozik az MLflow-logolásra is: az MLflow UI-ban látott szám a manifest mögötti szerződés nélkül ugyanolyan puszta szám.

31.1.8 Önellenőrzés

Ellenőrizd, hogy a manifestben a `run_id` hashje a paraméterekből és az input hashből készül, nem a metrikából; minden artefaktum rendelkezik hashsel; a `metric_definitions` tartalmazza a pozitív osztályt; a környezetben van Python-verzió és `lab_requires` hivatkozás; és a kísérleti táblázatban minden futás külön sor. Ha ezek közül bármelyik hiányzik, a reprodukálhatósági állítást szűkítsd, és ne nevezd a futást teljesen dokumentáltnak.

31.1.9 Rövid összefoglaló

A példabeli futásazonosító a paraméterekből és a bemeneti hashből képzett rövid, determinisztikus hash. Az azonos paraméterű és bemenetű ismétlések ugyanazt a logikai azonosítót kapják: egyedi futáspéldányhoz külön időbélyeg vagy UUID szükséges. A nyolc hexadecimális jegy ütközésmentességet nem garantál; a metrikák később adódnak hozzá, ezért nem részei az identitásnak. A manifest rögzíti a paramétereket, a metrikákat a definícióikkal, az artefaktumok hashét és a futtató környezetet, így a kutatási állítást a tényleges kódhoz és adathoz köti. A kis házilagos JSON-manifest lokális baseline-okhoz elég; az MLflow ugyanerre a fogalomra ad kényelmesebb felületet, de a szám mögötti protokoll akkor is kötelező. Következő alkalmazás: 31.2. alfejezet, ahol az adat- és splitmanifest verziózása teszi ezt a futásazonosítást nagyobb projektekben is használhatóvá.

31.2 Adatverzió és splitmanifest

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: MLflow, DVC, Git

Előfeltétel: 31.1. alfejezet · Célkészség: Adatverzió és splitmanifest · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

31.2.1 Cél és konkrét helyzet

A Northstar prediktív projektben az a kockázat, hogy a modell értékelése után nem tudjuk megmondani, pontosan melyik adatból és melyik felosztásból született egy metrika. A `seed=20261020` önmagában nem azonosítja a tényleges CSV-fájlokat, a `cutoff` pedig nem azonosítja a splitfájl minden sorát. Ebben az alfejezetben olyan manifestet készítünk, amely rögzíti a forrásfájlok tartalomhashét, a splitfájl hashét és a split sorazonosítóit.

Bemenet: a lab projekt `data/customers.csv`, `data/orders.csv` és `data/order_items.csv` fájljai, valamint a `starter_experiment.py` által is használt időbeli split logika. **Megfigyelési egység:** egy rendelés (`order_id`). **Kimenet:** `artifacts/split.json` sorazonosítókkal és `artifacts/split_manifest.json` hash-jegyzékkel.

Előzetes becslés. Az adat 2025 első 180 napjára esik, a cutoff 2025-05-01. Ha a rendelések időben egyenletesen oszlanak el, akkor a teszt aránya körülbelül $60/180 = 1/3$, tehát 2400 rendelésből körülbelül 800 teszt sor várható. A purged határsáv felső becslése a cutoff előtti 7 nap alapján $7/180 \approx 3{,}9\%$, körülbelül 93 rendelés. A train így nagyságrendileg $2400 \cdot (1 - 1/3 - 7/180) \approx 1500$ sor. Ez csak arány becslés, nem mért eredmény; a valódi értékek a generált adat eloszlásától függenek.

31.2.2 Fogalmi magyarázat: content hash, snapshot, pointer, DVC és immutable split

31.2.2.1 Content hash

A **tartalomhash** (content hash) egy fájl bájtjaihoz rendelt, gyakorlatilag egyedi rövid azonosító. Ha a bemenet n bájtja $b_1, b_2, \ldots, b_n$, akkor

$$h = \mathrm{SHA256}(b_1 b_2 \ldots b_n),$$

ahol h egy 256 bites érték, amelyet 64 karakteres hexadecimális szövegként írunk ki. Az SHA-256 determinisztikus: ugyanarra a bájtsorozatra minden gépen ugyanazt adja. A 256 bites kimeneti tér miatt két különböző fájl gyakorlati ütközése elhanyagolható; ez integritásellenőrzésre, nem matematikai bizonyításra használt eszköz.

Kézi kontroll a Pythonnal:

```
import hashlib
print(hashlib.sha256(b"abc").hexdigest())
# elméletileg várt, szabványos SHA-256 tesztvektor:
# ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

A hash a fájl *teljes tartalmára* vonatkozik. A fájlméret és a sorok száma nem elegendő: két azonos hosszúságú, de eltérő tartalmú fájlnak más a hash-e.

31.2.2.2 Dataset snapshot

A **dataset snapshot** egy adott időpontban rögzített, megváltoztathatatlanná tett adatpéldány. A snapshot lehet a fájlok teljes másolata, például `data/snapshots/2026-10-20/` alatt, vagy egy object store verziója. A content hash a snapshot leltára: igazolja, hogy a később felhasznált fájl ugyanaz, mint amit a manifest rögzített. A Northstar esetben a `generate_data.py` determinisztikus, ezért a snapshot újragenerálható, de a szállított CSV-ket akkor is a hash-ükkel ellenőrizzük.

31.2.2.3 Storage pointer és DVC

A **storage pointer** olyan kis rekord, amely nem magát a nagy adatot, hanem annak tartalomhashét és elérési útját tárolja. A mi manifestünkben a `path` és a `sha256` mező pointer: a tényleges bájtok a fájlban maradnak, a manifest csak hivatkozik rájuk.

A **DVC** (Data Version Control) ezt az elvet viszi tovább. A `dvc add data/orders.csv` a fájlt tartalomhash alapján a DVC cache-be másolja, és létrehoz egy kis `data/orders.csv.dvc` pointerfájlt. A Git csak ezt a pointert verziózza, magát a CSV-t a `.gitignore` kizárja. A `dvc push` a cache-ből egy távoli storage-ba tölti az adatot; másik gépen `git clone` után a `dvc pull` hozza vissza. A DVC pointer tehát két dolgot köt össze: a tartalom azonosítóját és a storage elérési útját. A pontos parancsok és hash-algoritmus DVC-verziófüggők; az itt közölt vázlat a fogalmi működést mutatja, telepített DVC-n a saját verzió dokumentációját kell követni.

31.2.2.4 Immutable split

Az **immutable split** olyan splitfájl, amelyet a `split_id` és a hash rögzítése után nem írunk felül. Ha a cutoff vagy a szűrési logika megváltozik, új `split_id`-t és új fájlt készítünk, a régit pedig megtartjuk. A mi laborunkban a `split.json` a teljes sorazonosító-listát tartalmazza; a manifest ennek a fájlnak a hash-ét rögzíti. Így a split módosítása a manifest ellenőrzésekor azonnal kiderül.

31.2.3 Teljes minimális munkapélda

A példa a lab projekt gyökeréből fut, ahol a `pyproject.toml` van. A `data/` mappa a CSV-ket, az `artifacts/` mappa a kimeneteket tartalmazza. A Python-kód standard library-t használ, ezért a lockolt uv környezetben is futtatható.

31.2.3.1 1. lépés: splitfájl előállítása

A `make_split.py` a cutoff és a `label_available_at` alapján három listát készít. A címke akkor ismert, ha `label_available_at < cutoff`; ez a 30.2. alfejezetben bevezetett adatminőségi logikától független, de a manifestben rögzített tisztított fájlra épül.

```
from pathlib import Path
import json
import csv
from datetime import datetime

ROOT = Path(".")
CUTOFF = datetime.fromisoformat("2025-05-01T00:00:00+00:00")
orders_path = ROOT / "data" / "orders.csv"
out_path = ROOT / "artifacts" / "split.json"

train, test, purged = [], [], []
with orders_path.open(newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        order_ts = datetime.fromisoformat(row["order_ts"])
        label_ts = datetime.fromisoformat(row["label_available_at"])
        oid = row["order_id"]
        if order_ts >= CUTOFF:
            test.append(oid)
        elif label_ts < CUTOFF:
            train.append(oid)
        else:
            purged.append(oid)

split_doc = {
    "split_id": "northstar-time-2025-05-01",
    "cutoff": "2025-05-01T00:00:00+00:00",
    "train_order_ids": train,
    "test_order_ids": test,
    "purged_order_ids": purged,
}

new_content = json.dumps(split_doc, indent=2, sort_keys=True) + "\n"
if out_path.exists():
    if out_path.read_text(encoding="utf-8") != new_content:
        raise SystemExit(
            "artifacts/split.json already exists with different content; "
            "use a new split_id instead of overwriting"
        )
out_path.parent.mkdir(parents=True, exist_ok=True)
out_path.write_text(new_content, encoding="utf-8")
print(f"train={len(train)}, test={len(test)}, purged={len(purged)}")
print(f"wrote {out_path}")
```

A script akkor ír csak, ha a fájl még nem létezik, vagy a tartalma byte-azonos a kiszámolttal. Ez az immutable split első védelme.

31.2.3.2 2. lépés: hash-manifest előállítása

A `make_manifest.py` a három forrás-CSV és a `split.json` SHA-256 hash-ét számolja ki, és a manifestbe írja. A `count_rows` opcionális kontroll, nem helyettesíti a hash-t.

```
from pathlib import Path
import hashlib
```

```
import json
import csv
from datetime import datetime, timezone

ROOT = Path(".")
SOURCE_FILES = [
    "data/customers.csv",
    "data/orders.csv",
    "data/order_items.csv",
]
SPLIT_PATH = Path("artifacts/split.json")
OUT_PATH = Path("artifacts/split_manifest.json")

def sha256_hex(path: Path, chunk_size: int = 1024 * 1024) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        while chunk := f.read(chunk_size):
            h.update(chunk)
    return h.hexdigest()

def count_rows(path: Path) -> int:
    with path.open(newline="", encoding="utf-8") as f:
        return sum(1 for _ in csv.reader(f)) - 1  # fejléc nélkül

manifest = {
    "manifest_version": 1,
    "created_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    "dataset": "northstar",
    "source_files": {},
    "split": {
        "path": str(SPLIT_PATH),
        "sha256": sha256_hex(SPLIT_PATH),
    },
    "generator": {
        "script": "generate_data.py",
        "seed": 20261020,
        "customers": 300,
        "orders": 2400,
    },
}

for rel in SOURCE_FILES:
    p = ROOT / rel
    manifest["source_files"][rel] = {
        "sha256": sha256_hex(p),
        "bytes": p.stat().st_size,
        "row_count": count_rows(p),
    }

OUT_PATH.parent.mkdir(parents=True, exist_ok=True)
OUT_PATH.write_text(json.dumps(manifest, indent=2, sort_keys=True) + "\n")
print(f"wrote {OUT_PATH}")
```

31.2.3.3 A split hitelesítési mezői: rendezett ID-hash, cutoff, adatverzió, row UID és séma-hash

A CSV bájthash önmagában nem mondja meg, **mely rekordok** kerültek a train/test/purged halmazba. Ezért a splitmanifest a következő mezőket is rögzítse:

Mező	Számítás	Mit véd
`split.split_id`	szemantikai név	a split célja és a cutoff neve
`split.cutoff`	ISO-8601 időbélyeg	az időbeli vágás pontos határa
`split.id_list_sha256`	a `train_order_ids`, `test_order_ids`, `purged_order_ids` **rendezett** listáinak hash-e	a halmaztagság; a sorrend nem számít
`split.row_uid_sha256`	a sorok kanonikus tartalmából képzett semleges `row_uid`-k rendezett hash-e	a rekordok tartalma, nem csak az üzleti kulcs
`split.column_schema_sha256`	az oszlopnevek és dtype-ok kanonikus hash-e	a séma változása
`data_version`	a forrásfájlok hash-einek összevont hash-e	a teljes adatverzió

A `row_uid` szándékosan **nem** az `order_id`: az üzleti kulcs megváltozhat, és egyes táblákban nincs is ilyen. A `row_uid` a sor kanonikus JSON-jából képzett rövid hash, amely a sor tartalmát azonosítja, de nem hordoz üzleti jelentést. A rendezett ID-hash miatt a splitfájl sorainak átrendezése nem változtatja meg a halmaz hashét; a splitfájl **bájt-hash-e** viszont változik, ezért mindkettőt rögzíteni kell.

```
import csv

EXPECTED_TYPES = {
    "order_id": "string", "customer_id": "string", "order_ts": "timestamp",
    "promised_days": "int", "quantity_sum": "int", "item_rows": "int",
    "basket_minor": "int", "shipping_fee_minor": "int", "currency": "string",
    "distance_km": "float", "priority_flag": "int", "target_late": "int",
    "label_available_at": "timestamp",
}

def sha256_text(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

def ordered_id_hash(ids):
    return sha256_text(json.dumps(sorted(ids), separators=(",", ":")))

def row_uid(row):
    payload = json.dumps(row, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()[:16]

with (ROOT / "data" / "orders.csv").open(newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    columns = reader.fieldnames
    rows = {row["order_id"]: row for row in reader}

split_doc = json.loads(SPLIT_PATH.read_text(encoding="utf-8"))
schema = {name: EXPECTED_TYPES.get(name, "unknown") for name in columns}

manifest["data_version"] = sha256_text("".join(
    sorted(info["sha256"] for info in manifest["source_files"].values())
))
manifest["split"].update({
    "split_id": split_doc["split_id"],
    "cutoff": split_doc["cutoff"],
    "id_list_sha256": {
        "train": ordered_id_hash(split_doc["train_order_ids"]),
```

```
        "test": ordered_id_hash(split_doc["test_order_ids"]),
        "purged": ordered_id_hash(split_doc["purged_order_ids"]),
    },
    "row_uid_sha256": {
        "train": sha256_text(json.dumps(
            [row_uid(rows[i]) for i in sorted(split_doc["train_order_ids"])])),
        "test": sha256_text(json.dumps(
            [row_uid(rows[i]) for i in sorted(split_doc["test_order_ids"])])),
        "purged": sha256_text(json.dumps(
            [row_uid(rows[i]) for i in sorted(split_doc["purged_order_ids"])])),
    },
    "column_schema_sha256": sha256_text(json.dumps(schema, sort_keys=True)),
})
```

A kontrollteszt lényege: ha egy `order_id` a tesztlistából a tanítólistába kerül, a `test` és a `train` `id_list_sha256` értéke is megváltozik, még akkor is, ha a forrás-CSV bájtjai változatlanok. A `tests/test_split_manifest.py` ezt a halmazmozgatást, a rendezésfüggetlenséget, a `row_uid` tartalomérzékenységét és a séma-hash változását teszteli.

A `created_at` miatt a manifest maga nem byte-azonos újrafuttatáskor; ezért a manifestet nem hasheljük saját magával. A forrás- és split-hashek már byte-azonosan reprodukálhatók.

31.2.3.4 3. lépés: ellenőrzés

A `verify_manifest.py` újraszámolja a hash-eket, és összehasonlítja a manifesttel.

```
from pathlib import Path
import hashlib
import json

ROOT = Path(".")
MANIFEST_PATH = Path("artifacts/split_manifest.json")

def sha256_hex(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):
            h.update(chunk)
    return h.hexdigest()

manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
for rel, info in manifest["source_files"].items():
    actual = sha256_hex(ROOT / rel)
    if actual != info["sha256"]:
        raise SystemExit(f"hash mismatch for {rel}: expected {info['sha256']}, got {actual}")
    if (ROOT / rel).stat().st_size != info["bytes"]:
        raise SystemExit(f"size mismatch for {rel}")
    print(f"{rel}: ok")

split_path = ROOT / manifest["split"]["path"]
if sha256_hex(split_path) != manifest["split"]["sha256"]:
    raise SystemExit("split hash mismatch")
print("split: ok")
print("all hashes match")
```

A futtatási parancs a lab projekt gyökeréből:

```
# munkakönyvtár: a lab projekt gyökere, ahol a pyproject.toml van
# Linux/macOS bash; Windows PowerShellben ugyanez a három parancs
uv run --locked python make_split.py
```

```
uv run --locked python make_manifest.py
uv run --locked python verify_manifest.py
```

A `--locked` azt ellenőrzi, hogy a környezet a `uv.lock` szerinti függőségeket használja. A scriptek standard library-sek, ezért itt nincs külső csomagfüggőség.

31.2.3.5 Mitől változik meg a futás inputja?

1. **Forrásfájl-módosítás.** Ha a `data/customers.csv` egyetlen bájtja megváltozik, az SHA-256 megváltozik. A `bytes` és a `row_count` akár azonos is maradhat, például ha egy `south` értéket `sooth`-ra cserélünk, mert mindkettő 5 karakter. A `verify_manifest.py` ilyenkor `hash mismatch` hibát ad.
2. **Generálási paraméter változás.** Ha a `generate_data.py`-t más seeddel futtatjuk, a CSV-k tartalma más lesz; a manifest `generator.seed` mezője a régi értéket mutatja, de a `sha256` ellenőrzése akkor is hibát jelez. Ezért a hash a döntő, nem a seed.
3. **Splitlogika-változás.** Ha a cutoffot vagy a label-availability szabályt megváltoztatjuk, a `split.json` más sorazonosítókat tartalmaz, így a split hash-e megváltozik. A manifest régi split hash-e nem egyezik.
4. **Splitfájl átrendezése.** Ha valaki a `split.json` listáinak sorrendjét módosítja, a JSON szövege más, tehát a hash más. A hash a teljes fájlra vonatkozik, nem csak a halmazokra.

31.2.4 Eredmény értelmezése

A `split_manifest.json` szerkezete:

```
{
  "manifest_version": 1,
  "created_at": "2026-10-20T12:00:00+00:00",
  "dataset": "northstar",
  "generator": {
    "script": "generate_data.py",
    "seed": 20261020,
    "customers": 300,
    "orders": 2400
  },
  "source_files": {
    "data/customers.csv": {
      "sha256": "…",
      "bytes": 12345,
      "row_count": 300
    },
    "data/orders.csv": {
      "sha256": "…",
      "bytes": 234567,
      "row_count": 2400
    },
    "data/order_items.csv": {
      "sha256": "…",
      "bytes": 345678,
      "row_count": 4796
    }
  },
  "split": {
    "path": "artifacts/split.json",
    "sha256": "…"
  }
}
```

A pontozott hash-ek csak mintaértékek, nem mért kimenetek; a tényleges értékek a helyi fájloktól függenek. A mezők típusa: `sha256` 64 karakteres hex string, `bytes` és `row_count` egész szám, a listák JSON stringekből állnak. A `row_count` a fejléc nélküli sorok száma; a `bytes` a fájl tényleges mérete.

A hash-számítás a labor néhány MB-os fájljain a másodperc tört része alatt lefut; ez nagyságrendi becslés, nem benchmark. A manifest néhány KB méretű, mert nem tartalmazza az adatot, csak a pointereket.

A fájlhash önmagában nem azonosítja az adat előállításának kódját. A manifestben rögzítsük az előállító kód commitját, a sémaverziót és a split létrehozási szabályát is.

31.2.5 Feladatok

31.2.5.1 1. Értelmezés: miért nem elég a seed?

Adott az alábbi manifest-részlet:

```
"generator": {"script": "generate_data.py", "seed": 20261020}
```

Magyarázd el, miért nem elegendő a `seed` önmagában a reprodukálhatósághoz, és milyen hibát vesz észre a `source_files.sha256` ellenőrzése akkor is, ha a seed nem változott. Adj példát olyan változásra, amely a seed megtartása mellett is megváltoztatja a CSV tartalmát.

31.2.5.2 2. Megvalósítás: manifest előállítása és ellenőrzése

Futtasd a `make_split.py` és `make_manifest.py` scripteket a lab projekt gyökeréből, majd a `verify_manifest.py` ellenőrzőt. Írd le a parancsokat és a várható kimenetet. Milyen mezőket tartalmaz a létrejött `split_manifest.json`?

31.2.5.3 3. Hibakeresés: azonos méret, mégis más adat

Egy kolléga a `data/customers.csv` egy sorában a `region` értéket `south`-ról `sooth`-ra cserélte úgy, hogy a sorok száma és a fájl mérete nem változott. A manifest `source_files` mezőjében a `bytes` és a `row_count` tehát egyezik. Mit ír ki a `verify_manifest.py`, és miért? Melyik mező fogja el a hibát?

31.2.5.4 4. Transzfer: szűrt split új feltétellel

A csapat csak a `consumer` szegmensre szűkíti a prediktív modellt. A `customers.csv`-ből a `segment` oszlop alapján szűrd az `orders.csv`-t, és készíts új splitet ugyanazzal a cutofftal. Hogyan változik a `split.json`, a `split_id` és a manifest? Mit kell rögzíteni ahhoz, hogy a régi és az új split ne legyen összetéveszthető?

31.2.6 Megoldások és indoklás

31.2.6.1 1. megoldás

A seed csak a generáló folyamat egyik bemeneti paramétere. A `generate_data.py` kódjának verziója, a Python `random` implementációjának viselkedése, a CSV quoting vagy az oszlopok sorrendje mind befolyásolja a fájl bájtjait. Ha például a generátor egy új verziója más sorrendben írja az oszlopokat, a seed azonos marad, de a CSV tartalma és hash-e megváltozik. A `source_files.sha256` a tényleges bájtokat azonosítja, ezért a manifest ellenőrzése ilyenkor `hash mismatch` hibát ad. A seed tehát a reprodukáláshoz szükséges, de nem elégséges; a hash a tényleges verziót igazolja.

31.2.6.2 2. megoldás

A parancsok:

```
cd lab
uv run --locked python make_split.py
uv run --locked python make_manifest.py
uv run --locked python verify_manifest.py
```

A `make_split.py` kiírja a három lista hosszát, például `train=…, test=…, purged=…` formában; a tényleges számok a helyi adattól függenek, az előzetes becslés szerint nagyságrendileg 1500, 800 és 100. A `make_manifest.py` a `wrote artifacts/split_manifest.json` üzenetet adja. A `verify_manifest.py` minden forrásfájlra `data/…: ok`, majd `split: ok` és `all hashes match` üzenetet ír. A manifest mezői: `manifest_version`, `created_at`, `dataset`, `source_files` (mindhárom CSV `sha256`, `bytes`, `row_count` értékével), `split` (`path`, `sha256`) és `generator`.

31.2.6.3 3. megoldás

A `verify_manifest.py` a `data/customers.csv` sorban `hash mismatch for data/customers.csv: expected …, got …` üzenetet ír, és `SystemExit` kóddal kilép. Az ok: a `south` és a `sooth` egyaránt 5 karakter, ezért a fájlméret és a sorok száma nem változik, de a fájl bájtjai igen. A `sha256` a teljes tartalomra vonatkozik, így a hiba csak a hash mezővel fogható el. Az ellenőrző invariáns: a hash a fájl minden bájtjára érzékeny, a méret és a sorok száma nem tekinthető tartalomazonosítónak.

31.2.6.4 4. megoldás

Az új split előállításához először a `customers.csv`-ből ki kell választani a `segment == "consumer"` ügyfelek `customer_id` halmazát, majd az `orders.csv`-ből csak azokat a sorokat szűrni, amelyek `customer_id`-ja ebben a halmazban van. A cutoff és a label-availability logika változatlan. Az új `split.json` csak a consumer rendelések `order_id` listáit tartalmazza, ezért a hash-e más. Az új `split_id` legyen például `northstar-consumer-time-2025-05-01`, és a fájl új nevet kapjon, például `artifacts/split_consumer.json`; a régit nem írjuk felül. A manifest `split.path` és `split.sha256` mezője az új fájlra mutat; a forrás-hashek változatlanok maradnak, ha a CSV-ket nem módosítottuk. A szűrési feltételt is rögzíteni kell, különben a `split_id`-ból nem derül ki, milyen szűrés alapján készült a split; javasolt a manifestbe vagy a `split.json`-ba egy `filter` mezőt felvenni, például `"filter": {"segment": "consumer"}`. Ez biztosítja, hogy a régi és az új split ne legyen összetéveszthető, és a futásrekord pontosan hivatkozni tudjon a megfelelő splitre.

31.2.7 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés: *„A DVC pointer vagy a Git commit elérhetővé teszi a hiányzó külső adatot."* Ellenpélda: clone-ozunk egy repót, amelyben `data/orders.csv.dvc` pointer van, de a `data/orders.csv` nincs a munkakönyvtárban, és a távoli storage sem elérhető. A `git log` mutatja a pointer commitját, de a CSV nincs a Gitben, mert a `.gitignore` kizárta. A `dvc pull` ilyenkor hibát ad, mert a pointer csak a hash-t és a remote elérési utat tartalmazza; magát az adatot nem. Ugyanez igaz a mi manifestünkre: a `sha256` azonosítja a fájlt, de nem pótolja, ha a fájl hiányzik.

Visszautalás a korábbi fogalmakra: a 30.2. alfejezet adatminőségi kapuja a tisztítás után fut le; a manifest a kapu által elfogadott, tiszta fájlok hash-ét rögzíti. Ha a kapu szabálya megváltozik, a hash is megváltozik, és a manifest ellenőrzése jelzi. A 31.1. alfejezet futásrekordjának a split manifestre vagy a `split_id`-ra kell hivatkoznia, nem csak a seedre. A 27.2. alfejezetben a koszinusz-hasonlóság szemantikai hasonlóságot mér; a tartalomhash ezzel szemben byte-szintű azonosságot ellenőriz, és nem helyettesíti a szemantikai keresést. Végül a 19.2. alfejezet intervallumai csak akkor értelmesek, ha tudjuk, melyik mintára vonatkoznak; a splitmanifest rögzíti ezt a mintát.

Önellenőrzés: tudod-e újraszámolni a manifestben szereplő hash-eket, és ugyanazt kapod-e? A `split.json` egy sorának törlése után a `verify_manifest.py` hibát ad-e? A manifest tartalmazza-e a forrásfájlok hash-ét és a split hash-ét, nem csak a seedet?

31.2.8 Rövid összefoglaló

Az adatverzió nem a seed felírása, hanem a tényleges forrásfájlok tartalomhashének rögzítése. A manifest a CSV-k SHA-256 azonosítóját, méretét, sorainak számát és a splitfájl hash-ét tárolja; a split sorazonosítókat tartalmaz, és a `split_id`, valamint a hash rögzítése után megváltoztathatatlannak tekintjük. A DVC pointer csak a hashre és a távoli elérési útra mutat; hiányzó külső adatot nem pótol. A splitmanifestet a futásrekord idézi, így minden metrika visszavezethető a pontos bemenetre. Következő alkalmazásként a modellartefaktum azonosítását és betöltési határát vizsgáljuk a 31.3. alfejezetben.

31.3 Modellartefaktum és betöltési határ

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: MLflow, DVC, Git**
>
> Előfeltétel: 31.2. alfejezet · Célkészség: Modellartefaktum és betöltési határ · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A modell csak akkor válik átadható artefaktummá, ha a tanító notebookból kilépve is önállóan, ellenőrizhetően betölthető. A betöltési határ (loading boundary) az a pont, ahol a modellfájl belép a futtató folyamatba: itt kell a hitelességet, a sémát, a függőségeket és a numerikus egyezést egy kapuval ellenőrizni. Ez az alfejezet a 31.2. alfejezet splitmanifestjére épít, és előkészíti a 31.4. alfejezet registry-jóváhagyást.

31.3.1 Helyzet: egy pipeline két processzben

A Northstar-előrejelzési projektben a 31.2. alfejezetben rögzített spliten betanítottunk egy scikit-learn `Pipeline`-t. A feladat: a betanított objektumot a `local_models/rendeles_keses_v1/` könyvtárba menteni úgy, hogy egy teljesen új Python-processz csak akkor tudja működésbe állítani, ha a betöltési kapun átment.

Bemenet: a betanított `Pipeline`, valamint 20 kontrollsor a tanító DataFrame-ből. Megfigyelési egység: egy rendelés; a kontrollsor egy rendelés feature-vektora. Kimenet: `pipeline.joblib`, `model_meta.json`, `control_predictions.npz`. A cél: az új processzben a kontrollpredikciók egyezzenek a mentéskori értékekkel. Előzetes becslés: ugyanazon a gépen, azonos csomagverziókkal a legnagyobb abszolút eltérés jellemzően 10^{-15}–10^{-14} nagyságrendű, tehát $\tau_{\text{abs}} = 10^{-9}$ konzervatív kaput ad.

31.3.2 Model signature: a be- és kimenet szerződése

A modell-séma (model signature) nem egyszerűen az oszlopnevek listája. Rögzíti a bemeneti oszlopok sorrendjét, típusát, a kategorikus változók lehetséges szintjeit, valamint a kimenet alakját és jelentését. Ha a betöltő processz a `distance_km, priority_flag, basket_minor, segment` oszlopokat más sorrendben adja, a `ColumnTransformer` csendben rossz pozícióra illesztheti a numerikus értékeket, még akkor is, ha a shape egyezik.

Példa egy szándékosan rövid sémára:

```
{
  "artifact_version": "1",
  "model_name": "rendeles_keses_base_v1",
  "input_schema": {
    "columns": ["distance_km", "priority_flag", "basket_minor", "segment"],
    "categorical_levels": {"segment": ["business", "consumer"]}
  },
  "prediction_contract": {
    "kind": "probability_of_class_1",
    "shape": ["n_samples", 1],
    "values": "float in [0, 1]"
  },
  "dependencies": {
    "python": "<actual from environment>",
    "scikit-learn": "<actual from environment>"
  }
}
```

A `prediction_contract.shape` itt `["n_samples", 1]`, mert a példában csak a `target_late = 1` osztály valószínűségét mentjük kontrollként. A `n_samples` a kontrollminta sorainak száma, nem egy rögzített konstans; ezt a metaadatban külön `n` mező jelzi.

31.3.3 Dependency version: amit a modell feltételez

A joblib/pickle formátumú modellbetöltés tetszőleges kódfuttatást is eredményezhet, ezért kizárólag megbízható, eredetileg hitelesített artefaktumot szabad betölteni. A joblib-fájl önmagában nem elegendő a teljes betanító környezet verzióinak dokumentálására. Ha a betöltő gépen újabb pandas vagy scikit-learn fut, a deszerializálás lehet sikeres, de a `OneHotEncoder` kategóriasorrendje, a lebegőpontos szummázás vagy a fa implementációja megváltozhat. A metaadat ezért a tényleges betanító verziókat rögzíti:

- `python`, például `3.12.8`;
- `numpy`, `pandas`, `scikit-learn`, `joblib`.

A verziótartomány önmagában nem elég. A 3.2. alfejezet `uv.lock` fájlja a feloldott, pontos verziókat tartalmazza; a modellmeta akkor a legerősebb, ha hivatkozik a lockfájl hashére is. A betöltési politikát a csapat dönti el: szigorú módban verzióeltérésnél a kapu hibát ad, megengedő módban figyelmeztet, ha a kontrollpredikció utána egyezik.

31.3.4 joblib/pickle trust: miért nem egy lépés a betöltés?

A joblib a pickle protokollt használja, a pickle deszerializálás pedig tetszőleges Python-kódot futtathat. Ezért a sorrend nem lehet „betöltés, majd ellenőrzés”, hanem:

1. a fájl SHA-256 hashének összehasonlítása a megbízható metaadattal;
2. csak ezután `joblib.load`;
3. majd séma- és kontrollpredikció-ellenőrzés.

A hash-ellenőrzés megbízhatósága annyi, amennyire a metaadat forrása megbízható. Ha a `model_meta.json` egy aláírt Git-commitban él, akkor a támadónak a commitot is hamisítania kellene. Ismeretlen forrásból származó `pipeline.joblib` betöltése akkor is kockázatos, ha a fájl kiterjesztése ismerős; a hash csak a fájl sértetlenségét igazolja, nem a tartalom szándékát.

31.3.4.1 Aláírás, megbízható tároló és allowlist

Ha a támadó a `pipeline.joblib`-et és a `model_meta.json`-t **együtt** cseréli ki, a hash-ellenőrzés zöld lesz, mert a manifest a saját maga által hivatkozott új hash-eket tartalmazza. A hash-lánc csak akkor véd, ha a lánc gyökere egy külön bizalmi csatornán érkezik. Két elfogadható megoldás:

1. **Aláírt manifest.** A `model_meta.json` bájtjait a kiadó privát kulcsával aláírják; a betöltő a beégetett nyilvános kulccsal ellenőriz. Az aláírás a teljes manifestre vonatkozik, így a benne szereplő fájlhashek, a séma és a verziók együtt hitelesek.
2. **Hitelesített, hozzáférés-szabályozott tároló.** Az artefaktum csak egy írásvédett, jogosultsággal védett bucketből/registryből tölthető; a betöltő az URI-prefixet és a modell-azonosítót allowlisttel ellenőrzi, és a letöltött bájtokat a manifesthez hasonlítja.

```python
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PublicKey

ALLOWED_PREFIXES = ("s3://northstar-models/",)
ALLOWED_MODELS = {"northstar-late-v2", "northstar-late-v3"}

def verify_signature(manifest_bytes: bytes, signature: bytes, public_key_bytes: bytes):
    public_key = Ed25519PublicKey.from_public_bytes(public_key_bytes)
    public_key.verify(signature, manifest_bytes)  # InvalidSignature, ha módosult

def check_source(uri: str, model_id: str):
    if not uri.startswith(ALLOWED_PREFIXES):
        raise ValueError("nem engedélyezett artefaktumtároló")
    if model_id not in ALLOWED_MODELS:
        raise ValueError("nem engedélyezett modell")
```

A betöltési sorrend kötelező: **aláírás-ellenőrzés → fájlhash-ellenőrzés → allowlist → deszerializálás**. Az aláírás a manifest bájtjaira vonatkozik, ezért a manifestet előbb be kell olvasni, de a `pipeline.joblib`-et csak az összes kapu után szabad `joblib.load`-olni. Ha a támadó a modellt és a manifestet együtt cseréli, de a privát kulccsal nem rendelkezik, az aláírás érvénytelen; ha a saját kulcsával írja alá, a beégetett nyilvános kulcs miatt bukik el. A `tests/test_artifact_trust.py` a helyes aláírást, a módosított manifestet, az együtt cserélt model+manifest esetet és az allowlistet teszteli.

31.3.5 Format: három fájl három szerepben

A háromrészes csomag szándékosan választja szét a felelősségeket:

Fájl	Tartalom	Szerep
`pipeline.joblib`	szerializált pipeline	a tényleges modellobjektum
`model_meta.json`	séma, verziók, fájlhashek	betöltési szerződés, ember által olvasható
`control_predictions.npz`	kontrollsorok és predikciók	numerikus reprodukálhatósági kapu

A joblib a numpy-tömböket hatékonyan tárolja, de nem hordoz sémaverziót; a JSON viszont verziózható és diffelhető, cserébe nem alkalmas nagyméretű tömbök tárolására. A kontrollpredikciókat ezért külön bináris fájlban tartjuk.

31.3.6 Load check: a numerikus tolerancia kapuja

A betöltő processz a kontrollsorokra újra kiszámítja a predikciókat, és a mentett értékekkel hasonlítja össze. Az i-edik kontrollsorra a feltétel:

$$\max_{i=1,\dots,m} \left| p_i^{\text{new}} - p_i^{\text{stored}} \right| \le \tau_{\text{abs}},$$

ahol m a kontrollsorok száma, p_i^{stored} a mentéskori modellkimenet, p_i^{new} a betöltés utáni kimenet, τ_{abs} pedig az abszolút tolerancia. A példában p_i a pozitív osztály valószínűsége, tehát mértékegysége 1, tartománya $[0, 1]$, alakja skalár soronként.

Ha a predikció nem korlátos, például regressziós cél, akkor a lebegőpontos hiba nagyságrendfüggő, ezért a gyakorlatban használt bővebb alak:

$$|p_i^{\text{new}} - p_i^{\text{stored}}| \le \tau_{\text{abs}} + \tau_{\text{rel}} |p_i^{\text{stored}}|,$$

ahol τ_{rel} relatív tolerancia, tipikusan 10^{-6}. Kézi kontroll: ha $p^{\text{stored}} = [0.2, 0.8]$ és $p^{\text{new}} = [0.1999999999, 0.8000000001]$, akkor a legnagyobb eltérés 10^{-10}, ami $\tau_{\text{abs}} = 10^{-9}$ alatt van, tehát a kapu nyit.

31.3.7 Teljes minimális munkapélda

Az alábbi kód kizárólag szintetikus adatot generál, így nem függ a kezdőlabor CSV-itől. A kódot e szöveg írásakor nem futtattuk, ezért a kimenet „szemléltető várt” jelölést kap; a futtatás a helyi környezet pontos csomagverzióit írja a metaadatba.

```
# train_and_save.py
from pathlib import Path
import json, hashlib, sys
import numpy as np
import pandas as pd
import sklearn
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
import joblib

rng = np.random.default_rng(20261020)
n = 300
df = pd.DataFrame({
    "distance_km": rng.gamma(2.0, 50.0, n),
    "priority_flag": rng.integers(0, 2, n),
    "basket_minor": rng.integers(500, 20_000, n),
    "segment": rng.choice(["consumer", "business"], n),
})
z = (0.02 * (df["distance_km"] - 100) / 100
     + 0.8 * df["priority_flag"]
     + 0.00005 * (df["basket_minor"] - 5000)
     + 0.5 * (df["segment"] == "business"))
prob = 1 / (1 + np.exp(-z))
df["target_late"] = (rng.random(n) < prob).astype(int)

feature_cols = ["distance_km", "priority_flag", "basket_minor", "segment"]
prep = ColumnTransformer([
    ("num", StandardScaler(), ["distance_km", "priority_flag", "basket_minor"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["segment"]),
])
model = Pipeline([
    ("prep", prep),
    ("clf", LogisticRegression(max_iter=1000, random_state=42)),
])
model.fit(df[feature_cols], df["target_late"])

artifact_dir = Path("local_models/rendeles_keses_v1")
artifact_dir.mkdir(parents=True, exist_ok=True)
model_file = artifact_dir / "pipeline.joblib"
meta_file = artifact_dir / "model_meta.json"
control_file = artifact_dir / "control_predictions.npz"

control = df[feature_cols].sample(20, random_state=7)
stored_pred = model.predict_proba(control)[:, 1]
joblib.dump(model, model_file)
np.savez(control_file,
         controls=control.to_numpy(),
         columns=np.array(feature_cols),
         predictions=stored_pred)

meta = {
    "artifact_version": "1",
    "model_name": "rendeles_keses_base_v1",
    "created_at": "2026-10-20T12:00:00+00:00",
    "input_schema": {
```

```
        "columns": feature_cols,
        "categorical_levels": {"segment": sorted(df["segment"].unique())},
    },
    "prediction_contract": {
        "kind": "probability_of_class_1",
        "shape": ["n_samples", 1],
        "values": "float in [0, 1]",
    },
    "dependencies": {
        "python": sys.version.split()[0],
        "numpy": np.__version__,
        "pandas": pd.__version__,
        "scikit-learn": sklearn.__version__,
        "joblib": joblib.__version__,
    },
    "file_hashes": {
        "pipeline.joblib": hashlib.sha256(model_file.read_bytes()).hexdigest(),
        "control_predictions.npz": hashlib.sha256(control_file.read_bytes()).hexdigest(),
    },
    "control": {"n": len(control), "abs_tolerance": 1e-9},
}
meta_file.write_text(json.dumps(meta, indent=2), encoding="utf-8")
print("saved", artifact_dir)
```

A betöltő script külön fájl, és külön processzben fut:

```
# check_model_artifact.py
import sys, json, hashlib
from pathlib import Path
import numpy as np
import pandas as pd
import joblib

EXPECTED_COLUMNS = ["distance_km", "priority_flag", "basket_minor", "segment"]

def main(artifact_dir: str) -> None:
    p = Path(artifact_dir)
    meta = json.loads((p / "model_meta.json").read_text(encoding="utf-8"))

    # 1. Hitelességi kapu: hash a deszerializálás előtt.
    for name in ["pipeline.joblib", "control_predictions.npz"]:
        actual = hashlib.sha256((p / name).read_bytes()).hexdigest()
        if actual != meta["file_hashes"][name]:
            raise ValueError(f"hash mismatch: {name}")

    # 2. Séma: a meta és a betöltő szerződése is egyezzen.
    if meta["input_schema"]["columns"] != EXPECTED_COLUMNS:
        raise ValueError("schema mismatch against service contract")

    # 3. Betöltés csak a hash-ellenőrzés után.
    model = joblib.load(p / "pipeline.joblib")

    # 4. Kontrollpredikció numerikus kapuja.
    loaded = np.load(p / "control_predictions.npz")
    control = pd.DataFrame(loaded["controls"], columns=loaded["columns"])
    new_pred = model.predict_proba(control)[:, 1]
    stored_pred = loaded["predictions"]
    diff = np.max(np.abs(new_pred - stored_pred))
    if not np.allclose(new_pred, stored_pred, atol=meta["control"]["abs_tolerance"], rtol=0):
        raise ValueError(f"prediction mismatch, max diff={diff}")
    print(f"OK: hash és predikciós kontroll egyezik; hitelesség csak megbízható manifesttel, max
    ↪  diff={diff:.3e}")
```

```
if __name__ == "__main__":
    main(sys.argv[1])
```

Futtatási parancs Linux/macOS terminálból, projektgyökérből:

```
python check_model_artifact.py local_models/rendeles_keses_v1
```

Windows PowerShellből:

```
python check_model_artifact.py local_models\rendeles_keses_v1
```

Szemléltető várt kimenet, nem mért futási jegyzék:

```
OK: artifact authenticated, max diff=0.000e+00
```

A `0.000e+00` itt a formázás lehetséges eredménye; tényleges futásnál jellemzően 10^{-16}–10^{-15} nagyságrendű, de ez nem garantált seedek és platformok között.

31.3.8 Eredmény értelmezése

A kapu csak azt igazolja, hogy a betöltött pipeline ugyanazt a függvényt számolja, mint a mentéskori objektum. Nem helyettesíti a modell tesztmetrikáját, és nem állít semmit a modell üzleti minőségéről. A három ellenőrzés három különböző hibát fog meg:

- hash-eltérés: sérült vagy kicserélt fájl;
- séma-eltérés: más bemeneti szerződést váró modell vagy fogyasztó;
- predikció-eltérés: verzióváltás vagy deszerializációs inkompatibilitás.

A `max diff` a toleranciához viszonyítva dönt; a pontos küszöböt a predikciós szerződéshez kell igazítani. A példában a valószínűség korlátos, ezért elegendő az abszolút tolerancia. Regressziós modellnél a relatív tagot is hozzá kell venni a fenti képlet szerint.

31.3.9 Feladatok

1. **Értelmezés.** Egy kolléga azt állítja: „Elég a `pipeline.joblib` SHA-256 hashét ellenőrizni, hiszen a hash garantálja, hogy a fájl biztonságos.” Miért hiányos ez az érvelés két különböző okból is?

2. **Megvalósítás.** Egészítsd ki a metaadatot `output_schema` mezővel, amely rögzíti, hogy a `predict_proba` kimenete `["n_samples", 2]` alakú, és a második oszlop a `target_late = 1` valószínűsége. Írd át a betöltő scriptet úgy, hogy a `model.predict_proba(control)` alakját és a `model.classes_` listáját is ellenőrizze.

3. **Hibadiagnózis.** A kolléga a betöltőben a kontrollt így olvassa be: `control = pd.DataFrame(loaded["controls"])`, oszlopnevek nélkül. A script lefut, de a predikciós kapu `prediction mismatch` hibát ad. Mi lehet az ok, és miért veszélyes az a változat, amelyik véletlenül nem ad hibát?

4. **Megváltozott feltétel.** A modellt új gépre telepítik, ahol a pandas verziója eltér a betanító környezettől. A betöltő WARN üzenetet ír, a kontrollpredikció maximális eltérése $2 \cdot 10^{-8}$, ami a 10^{-9} tolerancia fölött van. Írj `strict_dependencies` kapcsolót a betöltőbe: ha `true`, a verzióeltérés hiba legyen; ha `false`, a predikciós kapu döntsön. Milyen indoklással engedhető át a modell ebben a laza módban?

31.3.10 Megoldások és indoklás

1. megoldás. A hash-ellenőrzés két okból nem elég. Egyrészt a hash a fájl sértetlenségét igazolja, de nem mond semmit a fájl tartalmának szándékáról: ha a fájl eleve kártékony pickle-t tartalmaz, a hash csak annyit mutat, hogy ugyanaz a kártékony fájl érkezett meg. Másrészt a hash-érték csak annyira megbízható, amennyire a forrása: ha a `model_meta.json` ugyanabban a nem védett mappában van, mint a joblib-fájl, akkor egy támadó vagy hiba mindkettőt kicserélheti. A megbízhatósági láncnak egy aláírt Git-commitig vagy manifestig kell nyúlnia.

2. megoldás. A metaadat bővítése:

```
"output_schema": {
  "kind": "predict_proba",
  "classes_": [0, 1],
  "shape": ["n_samples", 2],
  "semantics": "second column is probability of target_late=1"
}
```

A betöltő ellenőrzése a `# 4.` blokkban:

```
classes = model.classes_.tolist()
if classes != meta["output_schema"]["classes_"]:
    raise ValueError(f"class mismatch: {classes}")
proba = model.predict_proba(control)
if proba.shape != (len(control), 2):
    raise ValueError(f"unexpected predict_proba shape: {proba.shape}")
new_pred = proba[:, 1]
```

A `model.classes_` ellenőrzése azért fontos, mert egy újonnan tanított modellnél a pozitív osztály sorrendje megváltozhat, és a `[:, 1]` indexelés csendben rossz jelentést adna.

3. megoldás. A `ColumnTransformer` a betanításkor rögzített feature-nevekre illeszt. Ha a betöltő `pd.DataFrame(loaded["controls"])` hívással nem ad oszlopneveket, a scikit-learn 1.x verzióban `ValueError` vagy explicit figyelmeztetés jelzi, hogy a feature-nevek nem egyeznek. A `prediction mismatch` hiba ebből következik. Az a veszélyesebb eset, ha a hiányos DataFrame oszlopai véletlenül a tanításkori sorrendben vannak, és a predikció lefut: ekkor a kapu zöld lehet, de a rendszer nem garantálja, hogy a jövőbeli bemenet is ezt a sorrendet követi. Ezért a betöltőben az oszlopneveket expliciten, a séma szerint kell beállítani.

4. megoldás. A kapcsoló bevezetése:

```
import sys
strict = "--strict-dependencies" in sys.argv[1:]
if strict:
    for key, trained in meta["dependencies"].items():
        current = {"python": sys.version.split()[0],
                   "numpy": np.__version__,
                   "pandas": pd.__version__,
                   "scikit-learn": sklearn.__version__,
                   "joblib": joblib.__version__}.get(key)
        if current is not None and current != trained:
            raise ValueError(f"dependency mismatch: {key} trained={trained} current={current}")
else:
    print("WARN: dependency versions differ; prediction gate remains authoritative")
```

Laza módban akkor engedhető át a modell, ha a predikciós eltérés kisebb egy üzletileg elfogadható küszöbnél, például 10^{-6}-nál, és a verzióeltérést a futási naplóban, valamint a modellkártyán rögzítik. A példabeli $2 \cdot 10^{-8}$ eltérés ilyen küszöb alatt van, de ez üzleti döntés, nem technikai automatikus engedély.

31.3.11 Gyakori tévedés ellenpéldával

Gyakori tévedés: „A saját gépen lévő joblib-fájl betöltése biztonságos, mert mi hoztuk létre.” Ez nem igaz akkor, ha a fájl e-mail mellékletből, megosztott mappából vagy CI-artifactként érkezik: a deszerializálás tetszőleges kódvégrehajtásának lehetőségét hordozza. Konkrét ellenpélda: egy támadó kicseréli a `pipeline.joblib`-et olyan pickle-re, amely a deszerializálás során fájlt ír vagy processzt indít. Ha a betöltő először `joblib.load`-ot hív, a kár már a hash-ellenőrzés előtt megtörténik. A 31.2. alfejezet splitmanifestjénél már bevezetett hash-lánc itt visszatér: a hash csak akkor véd, ha a metaadatot egy külön, megbízható csatornából, például aláírt Git-commitból olvassuk. A 30.3. alfejezet cache-kulcsánál a hash célja az újrafelhasználás volt; itt a cél a betöltési kapu. A 27.3. alfejezet külső LLM-értékelésénél hasonló bizalmi határ jelentkezik, de ott nem deszerializálunk, hanem a szöveges kimenetet értékeljük.

31.3.12 Önellenőrzés

- Meg tudom-e nevezni a három artefaktumfájl szerepét?
- A betöltőben a hash-ellenőrzés a `joblib.load` előtt vagy után van?
- Tartalmazza-e a sémám a kategorikus szinteket, nem csak a shape-et?
- A toleranciafeltétel abszolút és relatív tagját mikor kell használni?
- Verzióeltérésnél a kapu hibát ad, figyelmeztet, vagy a predikciós kontroll dönt?

31.3.13 Rövid összefoglaló

A modellartefaktum három fájlból áll: a `pipeline.joblib` a betanított objektum, a `model_meta.json` a séma, a függőségi verziók és a fájlhashek szerződése, a `control_predictions.npz` pedig a numerikus kapu. Betöltéskor először a hash-ellenőrzés fut, csak utána a deszerializálás, majd a séma- és kontrollpredikció-egyezés. A tolerancia 10^{-9} abszolút, regressziónál relatív taggal bővítve. Az ismeretlen pickle mindig kódvégrehajtási kockázat. A következő 31.4. alfejezet ezt a kaput registry-jóváhagyássá bővíti, ahol a modelljelölt verzióját és átadási döntését is nyilvántartjuk.

31.4 Modelljelölt, jóváhagyás és registry

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: MLflow, DVC, Git

Előfeltétel: 31.3. alfejezet · Célkészség: Modelljelölt, jóváhagyás és registry · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

31.4.1 Cél és konkrét helyzet

A Northstar prediktív projektben már van egy működő modell, a `northstar-late-v2`, amely rendeléskor becsüli annak valószínűségét, hogy a kézbesítés 7 napnál tovább tart. A 31.3. alfejezet egységben ezt a pipeline-t séma- és verzióinformációval együtt mentettük, és betöltési ellenőrzést végeztünk. Most új jelölt érkezett: a `northstar-late-v3` logisztikus regresszió. A rögzített, purged teszthalmazon a v3 AUC-értéke 0,66, a v2-é 0,63. A feladat nem új statisztikai elemzés, hanem az üzembe helyezési döntés: regisztrálni a jelöltet, felülvizsgálni, majd csak teljesült feltételek esetén championná tenni, világos visszaállítási úttal.

Megfigyelési egység: egy rendelés. Bemenet a rendeléskori feature-vektor, kimenet a `pred_prob` (késői kézbesítés becsült valószínűsége 0 és 1 között) és a `pred_class` (0/1). Az előzetes eredménybecslés: ha a v3 AUC 0,66, és a belépési küszöb 0,65, akkor a challenger jogosult a review-ra, de ez még nem jelenti automatikusan a champion cserét. A fejezet célja, hogy ezt a döntést nyomon követhető, visszavonható adminisztratív folyamattá tegye.

31.4.2 Fogalmi magyarázat: registry, candidate, champion, review, rollback, lineage

A **modellnyilvántartás** (model registry) olyan nyilvántartó réteg, amely a modellek életciklus-állapotát tárolja. A leggyakoribb állapotok: *staging* (kipróbálás alatt), *production* (éles kiszolgálás), *archived* (archivált). A registry **adminisztratív állapotot** rögzít; nem számol újra metrikát, nem bizonyít tudományos minőséget. Egy bejegyzés megléte önmagában csak annyit mond, hogy valaki regisztrálta a modellt.

A **modelljelölt** (candidate) egy konkrét, verzióval ellátott modellartefaktum, amelyhez tartozik:

- az artefaktum hash-értéke (SHA-256),
- a bemeneti és kimeneti séma,
- a kiértékelési jegyzék (metrika, küszöb, mintaszám, split hivatkozás),
- a lineage: adat- és kódverzió, környezet, tanítási paraméterek.

A **champion/challenger** minta két szerepet különít el. A *champion* (elsődleges modell) az, amely jelenleg kiszolgálja a predikciókat. A *challenger* (kihívó) az új jelölt, amely a champion helyére pályázik. A váltás nem attól függ, hogy a challengernek valamivel jobb egyetlen metrikája; a belépési feltételnek explicitnek kell lennie, például:

- a challenger teszt-AUC-ja legalább $\max(0{,}65,\ \text{champion AUC} + 0{,}01)$;
- a bemeneti séma pontosan egyezik a championével;
- a lineage hiánytalan, a split hash egyezik a manifestben rögzített értékkel;
- a challenger nem használ predictorba tiltott oszlopokat.

Az AUC itt a modell pontszámának rangsorolási képességét méri: $\text{AUC} = P(\hat{s}_i > \hat{s}_j \mid y_i = 1, y_j = 0)$, ahol $\hat{s}$ a modell által adott pontszám, y pedig a bináris cél. Ez egyetlen szám, nem helyettesíti a kalibráció, a szegmensenkénti teljesítmény vagy az üzleti költség vizsgálatát.

A **review** (felülvizsgálat, jóváhagyás) az a lépés, amelyben emberi döntéshozó vagy automatizált ellenőrzőlista alapján a challenger státuszt kap. A review lehetséges állapotai: `pending`, `approved`, `rejected`. A review-nak ellenőriznie kell:

1. a teljesítményfeltételt,
2. a sémát és a predikciós szerződést,
3. a lineage teljességét,
4. a reprodukálhatósági nyilatkozatot.

A **rollback** (visszaállítás) azt az eljárást jelenti, amellyel egy production modellt egy korábbi, jóváhagyott változatra cserélünk vissza. A visszaállítás is állapotváltozás, ezért ahhoz is feltétel és review kell. Tipikus visszaállítási feltétel: a production challenger 7 napos monitorablakban mért AUC-ja legalább 0,02-del alacsonyabb a regisztrációs AUC-nál, vagy séma-/batchhiba lép fel.

A **lineage** (származási lánc) a modellhez vezető összes bemenet verziózott láncolata: adatverzió, splitmanifest, kód commit, környezet, tanítási paraméterek, kiértékelési eredmény. A 30.4. alfejezet egységben az SQL-transzformációk lineage-ét, a 31.2. alfejezet egységben a splitmanifestet, a

27.4. alfejezet egységben a külső modell verziózását tárgyaltuk. A registry akkor hasznos, ha a lineage alapján bármelyik modellváltozatot újra tudjuk építeni, vagy meg tudjuk indokolni, miért nem.

Kis kézi kontroll: ha a v3 AUC-ja 0,66, a champion v2 AUC-ja 0,63, a belépési küszöb $\max(0{,}65, 0{,}63 + 0{,}01) = 0{,}65$, akkor a v3 teljesíti a teljesítményfeltételt. Ha viszont a séma eggyel több oszlopot tartalmaz, a jelölt már a review előtt elutasítható.

31.4.3 Teljes minimális munkapélda: hash-alapú regisztrációs kártya

Az MLflow és a DVC hasznos, de a laborban hash-alapú, helyi manifestet használunk, mert nincs szükség külső szolgáltatásra. A következő kód szemléltető: a hash-értékek helyettesítők, a kimenet elméletileg várt, nem mért. Platform: helyi CPU, munkakönyvtár `lab/`, Python 3.12.

A `register.py` beolvassa a mentett pipeline metaadatait, kiszámolja az SHA-256-ot, validálja a review-feltételeket, és előállítja a modellátadási lapot. A pipeline betöltését a 31.3. alfejezet egység `load check` lépése már elvégezte; itt csak a metaadatokkal dolgozunk.

```
"""Szemléltető hash-alapú modellregisztráció. Nem futtatott példa."""
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

ARTIFACT = Path("artifacts/northstar_late_pipeline_v3.pkl")
SPLIT = Path("artifacts/split.json")
METRICS = Path("artifacts/metrics_v3.json")

REQUIRED_INPUT = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
]


def sha256_file(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def build_card() -> dict:
    metrics = json.loads(METRICS.read_text())
    split_meta = json.loads(SPLIT.read_text())
    return {
        "model_id": "northstar-late-v3",
        "created_at": datetime.now(timezone.utc).isoformat(),
        "artifact_sha256": sha256_file(ARTIFACT),
        "input_schema": REQUIRED_INPUT,
        "output_schema": {"pred_prob": "float64", "pred_class": "int64"},
        "metrics": metrics,
        "lineage": {
            "data_ref": "northstar/2026-10-20/split.json",
            "code_ref": "main@a1b2c3d4",
            "split_sha256": split_meta.get("hash"),
            "training_script": "train_logreg.py",
            "env": "python=3.12.4,scikit-learn=1.5.2",
        },
        "stage": "staging",
        "review": "pending",
    }
```

```
def validate_card(card: dict, champion_auc: float) -> tuple[bool, list[str]]:
    errors = []
    if len(card["artifact_sha256"]) != 64:
        errors.append("artifact hash nem 64 hexa karakter")
    if card["input_schema"] != REQUIRED_INPUT:
        errors.append("bemeneti séma eltér a szerződéstől")
    if not all(card["lineage"].values()):
        errors.append("lineage hiányos")
    if card["metrics"]["auc_test"] < max(0.65, champion_auc + 0.01):
        errors.append("teljesítményfeltétel nem teljesül")
    if card["stage"] != "staging" or card["review"] != "pending":
        errors.append("helytelen kezdeti állapot")
    return (len(errors) == 0), errors


if __name__ == "__main__":
    # A champion v2 ismert, rögzített teszt AUC-ja.
    card = build_card()
    ok, errors = validate_card(card, champion_auc=0.63)
    Path("artifacts/model_card_v3.json").write_text(json.dumps(card, indent=2))
    print("Validation:", "PASS" if ok else "FAIL")
    for e in errors:
        print(" -", e)
```

Elméletileg várt kimenet, ha a metaadatok rendben vannak:

```
Validation: PASS
```

A generált regisztrációs kártya (rövidítve, a hash-ek szemléltető értékek):

```
{
  "model_id": "northstar-late-v3",
  "created_at": "2026-10-20T10:00:00+00:00",
  "artifact_sha256": "a4f9c2e7... (64 hexa karakter)",
  "input_schema": [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel"
  ],
  "output_schema": {"pred_prob": "float64", "pred_class": "int64"},
  "metrics": {"auc_test": 0.66, "n_test": 300, "threshold": 0.45},
  "lineage": {
    "data_ref": "northstar/2026-10-20/split.json",
    "code_ref": "main@a1b2c3d4",
    "split_sha256": "b7e1... (64 hexa karakter)",
    "training_script": "train_logreg.py",
    "env": "python=3.12.4,scikit-learn=1.5.2"
  },
  "stage": "staging",
  "review": "pending"
}
```

A registry állapota ezután:

```
{
  "champion": "northstar-late-v2",
  "challenger": "northstar-late-v3",
  "review": "pending",
  "rollback_to": null
}
```

31.4.3.1 Champion alias és immutábilis modellverzió

A `champion` egy **mutálható alias**, amely idővel más modellre mutat; a predikció reprodukálásához viszont a konkrét, immutábilis verzió kell. Ezért a registry két külön réteget tárol:

```
{
  "aliases": {"champion": "northstar-late-v3"},
  "models": {
    "northstar-late-v2": {
      "stage": "archived", "review": "approved",
      "artifact_sha256": "…", "code_commit": "main@a1b2c3d4"
    },
    "northstar-late-v3": {
      "stage": "production", "review": "approved",
      "artifact_sha256": "…", "code_commit": "main@e5f6a7b8"
    }
  },
  "promotions": [
    {
      "promotion_id": "promo-0001",
      "from_model": "northstar-late-v2",
      "to_model": "northstar-late-v3",
      "approved_by": "anna.kovacs",
      "approved_at": "2026-10-21T09:30:00+00:00",
      "code_commit": "main@e5f6a7b8",
      "rollback_to": "northstar-late-v2"
    }
  ]
}
```

A promóció **atomikus**: csak akkor hajtható végre, ha a registry champion értéke még a várt előző modell. Adatbázisban ez `UPDATE ... WHERE champion = :expected` feltétel; fájlban a teljes registry temp+`os.replace` cseréje. A promot rekord kötelező mezői: `promotion_id`, `from_model`, `to_model`, `approved_by`, `approved_at`, `code_commit`, `rollback_to`.

```
import copy

def promote(registry, expected_champion, candidate_id,
            approver, commit, now, promotion_id):
    if not approver or not commit:
        raise ValueError("a promócióhoz jóváhagyó és commit kell")
    if registry["aliases"]["champion"] != expected_champion:
        raise RuntimeError("a champion időközben megváltozott; frissítsd a registry-t")
    candidate = registry["models"][candidate_id]
    if candidate["stage"] != "staging" or candidate["review"] != "approved":
        raise ValueError("a jelölt nem promotálható")
    updated = copy.deepcopy(registry)
    updated["models"][expected_champion]["stage"] = "archived"
    updated["models"][candidate_id]["stage"] = "production"
    updated["aliases"]["champion"] = candidate_id
    updated["promotions"].append({
        "promotion_id": promotion_id,
        "from_model": expected_champion,
        "to_model": candidate_id,
        "approved_by": approver,
        "approved_at": now,
        "code_commit": commit,
        "rollback_to": expected_champion,
    })
    return updated
```

A visszaállítás is a `models` rétegből dolgozik: a `rollback_to` egy konkrét `model_id`, nem az „előző champion” alias. Egy régi predikciót a `models[model_id]["artifact_sha256"]` alapján kell újrajátszani, nem a mindenkori `aliases.champion`-nal. A `tests/test_registry_promotion.py` a sikeres promóciót, a versenyző promóció elutasítását, a hiányzó jóváhagyót, a rollbacket és a régi verzióval való visszajátszást teszteli.

A modellátadási lap (részlet) a feltételekkel:

Feltétel	Elvárt érték	Ellenőrzés
Teszt AUC	$\geq 0{,}65$ és $\geq$ champion AUC + 0,01	`metrics.json`
Bemeneti séma	pontosan a rendeléskori 8 oszlop	`input_schema`
Lineage	minden mező kitöltve	`lineage`
Visszaállítás	ha monitor AUC $\leq$ regisztrációs AUC $-0{,}02$	`rollback_to`

31.4.4 Eredmény értelmezése

A fenti folyamat invariánsai:

- A challenger kezdeti állapota mindig `staging` és `pending`, soha nem `production`.
- A champion mező csak akkor változik, ha a review `approved`.
- A `rollback_to` értéke `null`, amíg nincs visszaállítási esemény.
- A registry nem írd felül a `metrics` mezőt: az AUC a rögzített teszten mért érték, nem a regisztráció során újraszámolt adat.

Kézi kontroll: a v3 AUC 0,66, a belépési küszöb 0,65, a különbség 0,01. Ha a séma eggyel több oszlopot tartalmazna, a `validate_card` azonnal `FAIL`-t adna, függetlenül az AUC-tól. Ez mutatja, hogy a regisztráció nem helyettesíti a tudományos értékelést, hanem azt a megfelelő lépésben ellenőrzi.

A registry állapot és a tudományos minőség elkülönülése itt konkrét: a `model_card_v3.json` létezése és a `stage: "staging"` bejegyzés csak azt igazolja, hogy a jelölt átment a formai és teljesítménybelépési szűrőn. Azt, hogy a modell valóban jól általánosít, a rögzített teszthalmaz, a kalibrációs görbe és a szegmensvizsgálat igazolja, amelynek eredményét a kártya `metrics` mezője hivatkozza.

A champion címke módosítható mutató. Auditálható kiadáshoz az immutábilis modellazonosítót és az engedélyezés idejét is őrizzük meg.

31.4.5 Feladatok

31.4.5.1 1. feladat: Értelmezés

A registry-ben a `northstar-late-v4` bejegyzés állapota `production`, review-ja `approved`, AUC-ja 0,68. Egy kolléga azt állítja: „Mivel a registry szerint production és approved, ez a modell biztosan jó döntést ad.” Nevezz meg legalább két okot, amiért ez az állítás téves, és adj egy konkrét ellenőrzési lépést, amely a registry-n kívül szükséges.

31.4.5.2 2. feladat: Implementálás

Egészítsd ki a `register.py` kódját egy `check_monitor_and_rollback()` függvénnyel. A függvény a production challenger 7 napos monitorablakban mért AUC-ját kapja paraméterként, és ha az legalább 0,02-del alacsonyabb, mint a regisztrációs `auc_test`, akkor a registry állapotában a `rollback_to` mezőt a korábbi champion azonosítójára állítja, és a review-t `pending`-re teszi. Ha a monitor AUC elfogadható, a függvény `None`-t ad vissza. Írd le a függvényt, és mutasd meg, hogyan hívnád meg a `northstar-late-v3` esetén, ha a monitor AUC 0,61.

31.4.5.3 3. feladat: Hibadiagnózis

Az alábbi manifest hibás:

```
{
  "model_id": "northstar-late-v5",
  "stage": "production",
  "review": "none",
  "metrics": {"auc_test": 0.99},
  "lineage": {},
  "artifact_sha256": "1234"
}
```

Sorold fel az összes hibát, valószínűsítsd, mi okozhatta, és írd le, milyen validációs lépéssel lehet megelőzni.

31.4.5.4 4. feladat: Megváltozott feltétel

A csapat újragenerálta a teszthalmazt, és a `northstar-late-v3` AUC-ja az új kiértékelésben 0,61. A registry `model_card_v3.json` még a régi 0,66-ot tárolja. Mi a helyes eljárás? Milyen mezőket kell frissíteni, és milyen feltétellel engedhető a champion csere?

31.4.6 Megoldások és indoklás

31.4.6.1 1. megoldás

Téves az állítás, mert: (a) a registry `approved` állapota csak a review-folyamat lezárását jelenti, nem a modell aktuális teljesítményét; (b) a teszt AUC egy adott, rögzített időablakra vonatkozik, és nem garantálja az éles forgalomban mért teljesítményt; (c) a döntési cél megváltozhat, például a küszöböt más költségfüggvényhez kalibrálták. Szükséges ellenőrzés: a legfrissebb monitor adatokon számolt AUC és kalibrációs görbe, valamint a bemeneti adatok eloszlásának összehasonlítása a tanító eloszlással.

31.4.6.2 2. megoldás

```
def check_monitor_and_rollback(registry: dict, card: dict,
                               monitor_auc: float) -> dict | None:
    if monitor_auc <= card["metrics"]["auc_test"] - 0.02:
        registry["rollback_to"] = registry["champion"]
        registry["review"] = "pending"
        return registry
    return None
```

Használat:

```
monitor_auc = 0.61
result = check_monitor_and_rollback(registry, card, monitor_auc)
if result is not None:
    print("Rollback javasolt ide:", result["rollback_to"])
```

Elméletileg várt kimenet: `Rollback javasolt ide: northstar-late-v2`, mert $0{,}61 \leq 0{,}66 - 0{,}02 = 0{,}64$. Megjegyzés: a tényleges rollback nem automatikus; a review-nak kell jóváhagynia.

31.4.6.3 3. megoldás

Hibák: a `stage` nem lehet `production`, ha a `review` nem `approved`; a `review` értéke nem `none` (a megengedett: `pending`, `approved`, `rejected`); a `lineage` üres; az AUC 0,99 gyanúsan magas, valószínűleg adatszivárgásra vagy hibás kiértékelésre utal; a hash nem 64 hexa karakter. Valószínű ok: valaki kézzel szerkesztette a manifestet, vagy egy hibás pipeline kimenetét regisztrálta. Megelőzés: a `validate_card`-hoz hasonló sémaellenőrzés, a hash hosszának és a megengedett állapotoknak az ellenőrzése, valamint a metrika plauzibilitási határának beállítása.

31.4.6.4 4. megoldás

A helyes eljárás: a régi kártya `metrics` mezőjét nem szabad felülírni, mert az a korábbi értékelés lenyomata. Új kiértékelést kell végezni, új kártyát kell létrehozni, és a challenger belépési feltételt az új AUC-val újra kell vizsgálni. Ha az új AUC 0,61, a belépési küszöb $\max(0{,}65, 0{,}63 + 0{,}01) = 0{,}65$, tehát a v3 nem teljesíti a feltételt: a review `rejected`, a champion marad a v2. Frissítendő mezők: `metrics`, `lineage.split_sha256`, `created_at`, `review`.

31.4.7 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés: **a registry-bejegyzés automatikusan jó modellt bizonyít.** Ellenpélda: a `northstar-late-v5` regisztrálva van, `stage: "production"`, `review: "approved"`, de a metrikája AUC 0,99 egy olyan teszthalmazon, amely a tanító időszakkal átfedésben van. A registry szempontjából minden formai feltétel teljesül, a modell mégsem általánosít. Egy másik ellenpélda: a v3 championná vált, de a döntési küszöböt a tesztkészleten optimalizálták, így a valós forgalomban a modell túl sok hamis pozitív riasztást ad. A 31.2. alfejezet egységben tárgyalt splitintegritás és a 31.3. alfejezet egységben bemutatott sémaellenőrzés itt kapcsolódik vissza: a registry csak akkor hasznos, ha a bemenő bizonyítékok maguk is megbízhatóak.

Önellenőrzés: ha a `model_card.json` `stage` mezője `production`, de nincs `approved` review, akkor a folyamat sérült; ha a `lineage` hiányos, a modell nem reprodukálható; ha a rollback feltétel nincs előre rögzítve, egy éles hibánál kapkodás lesz a döntés.

31.4.8 Rövid összefoglaló

A modellnyilvántartás a modell adminisztratív életciklusát kezeli, nem a tudományos minőség bizonyítékát. A candidate egy verziózott artefaktum hash-sel, sémával, metrikával és lineage-dzsel. A champion/challenger váltás explicit feltételekhez kötött: a challengernek teljesítenie kell a teljesítmény- és sémaküszöböt, majd review-n kell átmennie. A rollback előre rögzített monitorfeltétel alapján egy korábbi jóváhagyott modellre mutat. A lineage biztosítja az újraépíthetőséget és a hibakeresést. A következő alkalmazásban ezt a regisztrációs lépést konténerben és CI-folyamatban futtatjuk, hogy az átadás más gépen is ismételhető legyen.

32 Konténerek, távoli környezetek és automatizált futtatás

32.1 Konténer és virtuális környezet külön szerepe

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Docker, SSH

Előfeltétel: 31.4. alfejezet · Célkészség: Konténer és virtuális környezet külön szerepe · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

32.1.1 Cél és konkrét helyzet

A 4.4. alfejezet munkaasztalán a lab projekt lokálisan ugyanazt az eredményt adja: a regisztrált kernel a projekt `.venv`-jére mutat, a `quickcheck.py` `failures=0` eredményt ír, a futási jegyzék pedig rögzíti a hash-ellenőrzést. A probléma akkor jelentkezik, amikor ezt a projektet egy kolléga gépére vagy CI-futásra adjuk át. A `uv.lock` és a `pyproject.toml` a Python-csomagok függőségeit rögzíti, de nem rögzíti sem az operációs rendszer verzióját, sem a rendszerkönyvtárakat, sem magát a Python-interpreter buildjét. A 2.4. alfejezetben tárgyalt OS-különbségekből tudjuk: a másik gépen hiányozhat `libgomp`, más lehet a glibc vagy az időzóna-adatbázis, ezért ugyanaz a lock még nem garantálja ugyanazt a futási környezetet.

Ebben az alfejezetben a vizsgálati egység nem statisztikai minta, hanem egy környezet-build: egy Dockerfile-ból épülő **image** és az abból indított **container**. A bemenet a lab projekt forrása (`pyproject.toml`, `uv.lock`, `src/`, `quickcheck.py`, `generate_data.py`, `starter_experiment.py`), a kimenet pedig egy megismételhető futási környezet, amelyben a `quickcheck.py` a kívülről csatolt adatfájlokat ellenőrzi. Az ellenőrzési feltétel kettős: a Dockerfile ne tartalmazzon adatfájlokat és `.venv`-et; a futó konténer pedig a projekt lockjának megfelelő interpretert használja.

Induló becslésként érdemes feltenni, hogy a buildelési kontextus a `.dockerignore` nélkül több száz megabájt is lehet a `.venv` és az adatfájlok miatt; a szűréssel a forráskód néhány megabájtra csökken. A Dockerfile-t ebben a generálási környezetben nem futtatjuk, ezért az alábbi munkamenet **specifikáció és szemléltető példa**, nem mérési jegyzőkönyv.

32.1.2 Fogalmi magyarázat: image, container, base image, venv, volume, user, build context

A **Docker-kép** vagy image egy olvasható, rétegezett fájlrendszer és hozzá tartozó metaadatok halmaza. Minden `RUN`, `COPY` és `FROM` utasítás egy újabb fájlrendszer-réteget hoz létre. A **konténer** az image-ből indított futó példány: az image rétegeire egy írható réteget tesz, külön processzteret, hálózatot és fájlrendszer-nézetet kap. A konténer törlése után az írható rétege elveszik; ami megmarad, az a volume-on vagy a bind mounton kívülre kerül.

A **base image** az a kiinduló image, amelyre a Dockerfile épít. Példánkban `python:3.12-slim` egy Debian-alapú, kisméretű Python-kép. A `slim` azt jelzi, hogy a gyártó a felesleges fejlesztői eszközöket elhagyta. Az image-et a `FROM` sor határozza meg; ha a projekt `.python-version` fájlja `3.12`-t ír, az csak kérés a helyi eszköz számára, a tényleges interpretert a konténerben a `FROM python:3.12-slim` adja.

A **venv** Python-szintű virtuális környezet: interpretert és a projekt Python-csomagjait izolálja. A `uv sync --locked` a `uv.lock`-ot oldja fel, tehát a Python-csomagok verzióját és ellenőrző adatait rögzíti. A venv nem rögzíti viszont az operációs rendszert, a rendszerkönyvtárakat és a natív függőségeket. A két réteg felelőssége:

Függőség	Példa	Rögzíti	Nem rögzíti
Image	glibc, `libgomp`, CA-tanúsítványok, tzdata, OS-verzió	Dockerfile `FROM` és build	A kernel pontos verzióját, CPU-utasításokat
Python-lock	numpy, pandas, scipy, statsmodels verziója	`uv.lock`, `uv sync --locked`	A rendszerkönyvtárakat, az interpreter buildjét
Adat- és splitmanifest	CSV-hash, cutoff	`manifest.json`, mount	Az adat jövőbeli frissítését

A **volume** és a **bind mount** a konténer tárterét köti a külvilághoz. A bind mount a host egy könyvtárát teszi elérhetővé, például `-v "$(pwd)/data:/app/data:ro"`: a host `data` könyvtára a konténeren belül `/app/data` néven, csak olvashatóan jelenik meg. A volume ezzel szemben Docker által kezelt tároló, amelyet a konténer életciklusa után is megőriz. A Northstar-lab bemeneti CSV-ket bind mountként érdemes csatolni, hogy az adatfájlok ne az image részei legyenek, hanem a futás pillanatában érkezzenek.

A **user** a konténeren belüli processz UID-je. A konténer alapból gyakran `root` felhasználóval indul, ami adatkérdésben rossz gyakorlat: egy hibás kód a hostra is veszélyesebb lehet, és a későbbi CI-rendszerben a jogosultság-ellenőrzés is sérül. Ezért a Dockerfile-ban explicit `USER app`-et adunk meg. A bind mount fájljogosultsága a host UID-jétől függ; ha a konténer `10001` UID-je nem egyezik az adatfájl tulajdonosával, olvasási hiba léphet fel. A 2.4. alfejezet fájljog-tanulsága itt visszatér: a hiba forrása nem az algoritmus, hanem a fájlrendszer-hozzáférés.

A **build context** az a könyvtár, amelyet a `docker build .` parancs a Docker daemonnak átad. A `build context` tartalmazza az összes fájlt, amelyre a `COPY` hivatkozhat; a `.dockerignore` szűri ki belőle a felesleges elemeket. Ha a context nagy, a build már az elején lelassul, és a `.venv`, az adatfájlok vagy a `.git` könyvtár is bekerülhet a képbe. Ezért a `.dockerignore` része a környezeti szerződésnek.

32.1.3 A Python-lock és az image határa

A 3.4. alfejezetben a kernelspec `argv[0]` eleme mutatta a tényleges Python-útvonalat. A konténerben az interpreter útvonala `/opt/venv/bin/python`, tehát a `uv sync` a `/opt/venv`-be telepít; az image `FROM` rétege a rendszer-szintű részeket adja. A két réteg összefügg: ha a `python:3.12-slim` image új buildet kap, ugyanaz a `uv.lock` más glibc-vel futhat. Ha viszont a `uv.lock` változik, a Docker-réteg-cache a `COPY pyproject.toml uv.lock` sortól újraépül. Ez a rétegcache-logika a 30.3. alfejezet input-hash alapú cache-elvéhez hasonlít: ha a bemenet hash-e nem változik, a korábbi rétegek felhasználhatók.

32.1.4 Teljes minimális munkapélda: Dockerfile a labhoz

A következő Dockerfile a lab projekt gyökerében áll, explicit munkakönyvtárral (`/app`) és explicit userrel (`app`). Az adatfájlokat és a `.venv`-et kizárjuk az image-ből. A `UV_VERSION=0.5.11` **szemléltető helyőrző**, nem ellenőrzött aktuális verzió; a lab review-ban a ténylegesen telepített `uv --version` kimenettel kell rögzíteni. Ugyanez igaz a `python:3.12-slim` tagre: a példában a könnyen olvashatóság kedvéért tag van, a következő szakaszban a digeströgzítést külön tárgyaljuk.

```
# Dockerfile - szemléltető specifikáció, automatizált futtatás nélkül.
# Bázis kép: könnyen olvasható tag; éles rögzítéshez lásd a tag/digest szakaszt.
FROM python:3.12-slim

# Az uv verziója a lockkal együtt rögzítendő; az alábbi érték helyőrző.
ARG UV_VERSION=0.5.11
RUN python -m pip install --no-cache-dir uv==${UV_VERSION}

# Nem root felhasználó létrehozása.
RUN groupadd --gid 10001 app && \
    useradd --uid 10001 --gid app --create-home app

ENV UV_PROJECT_ENVIRONMENT=/opt/venv
ENV VIRTUAL_ENV=/opt/venv
RUN python -m venv "$VIRTUAL_ENV"
ENV PATH="$VIRTUAL_ENV/bin:$PATH"

WORKDIR /app

# A lab forrása; az adatfájlokat és a .venv-et szándékosan nem másoljuk.
COPY --chown=app:app pyproject.toml uv.lock ./
COPY --chown=app:app src ./src
COPY --chown=app:app quickcheck.py generate_data.py starter_experiment.py README.md ./

RUN uv sync --locked --no-dev --no-install-project

USER app
CMD ["python", "quickcheck.py"]
```

A `.dockerignore` fájl kizárja a projekt méretes és futás közben csatolt elemeit:

```
.git
.venv
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
data/
artifacts/
*.csv
*.parquet
.env
```

A build és a futás parancsa Linuxon, WSL2-ben vagy macOS-héjban (a `lab/` projektgyökér mint munkakönyvtár):

```
# Linux, WSL2 vagy macOS héj; munkakönyvtár: a lab projekt gyökere
docker build --pull --progress plain -t northstar-lab:dev .

docker run --rm \
  -v "$(pwd)/data:/app/data:ro" \
```

```
  -v "$(pwd)/artifacts:/app/artifacts" \
  northstar-lab:dev
```

PowerShellben a sortörés backtick, a `${PWD}` a környezeti változó; a bind mount útvonala a helyi lab tényleges elérési útjához igazítandó:

```
# PowerShell; munkakönyvtár: a lab projekt gyökere
docker build --pull --progress plain -t northstar-lab:dev .

docker run --rm `
  -v "${PWD}/data:/app/data:ro" `
  -v "${PWD}/artifacts:/app/artifacts" `
  northstar-lab:dev
```

Az ellenőrzéshez a konténerbe lépve le lehet kérdezni az interpreter útvonalát és a user azonosítóját:

```
# Szemléltető ellenőrző parancs; a konkrét kimenet a lab és a Docker verziójától függ
docker run --rm northstar-lab:dev python -c "import sys; print(sys.executable)"
# Elvárt: /opt/venv/bin/python

docker run --rm northstar-lab:dev id -u
# Elvárt: 10001
```

Az image tartalmának vizsgálata:

```
docker image inspect northstar-lab:dev --format '{{.Id}}'
```

A `.Id` az image tartalomazonosítója, nem feltétlenül rövid. A `RepoDigests` lista üres is lehet egy kizárólag helyben épített image-nél; ekkor az `index .RepoDigests 0` kifejezés hibát adhat. A build ellenőrzéséhez külön `docker image inspect northstar-lab:dev --format "{{.Id}}"` parancsot használjunk; registry-digestet csak ténylegesen közzétett vagy letöltött image-nél várjunk.

32.1.5 Eredmény értelmezése

A build napló eleje mutatja a context méretét; a `.dockerignore` hatására a `.venv` és az adatfájlok nem kerülnek a contextbe, ezért a forrás-copy néhány megabájtos marad. A `COPY --chown=app:app` beállítja a fájlok tulajdonosát a konténeren belül, így a `USER app` után a kód elérhető. A `uv sync --locked --no-dev` a lockot kikényszeríti: ha a `pyproject.toml` és a `uv.lock` nincs összhangban, a build leáll, és ez **kívánt** hiba. A runtime `--no-dev` a fejlesztői csomagokat kihagyja; a `quickcheck.py`-hez nem szükséges pytest, de ha a lab teljes tesztfuttatást kér, külön teszt-image vagy build stage indokolt.

A futás során a `data` mount `:ro` jelzése megakadályozza, hogy a vizsgált bemeneti fájlokat a konténer módosítsa; az `artifacts` mount írható, így a futás eredménye a konténer törlése után is megmarad. A `--rm` a leállás után törli a konténert, de a bind mountok tartalma a hoston továbbra is elérhető. Az invariáns: ha a Dockerfile, a `uv.lock` és a forrás ugyanaz, akkor ugyanaz a `quickcheck.py` fut; a `failures=0` eredmény csak a csatolt adat sértetlenségét igazolja, ahogy 4.4. alfejezetben is külön réteg volt.

32.1.5.1 Digest, architektúra és a lock hármasa

Az image **tag** nem tartalomazonosító. A `python:3.12-slim` taget a karbantartó újra kiadhatja; a `FROM` sort ezért digesttel kell rögzíteni:

```
FROM --platform=linux/amd64 python:3.12-slim@sha256:REPLACE_WITH_VERIFIED_DIGEST
```

A digestet a registry manifestjéből kell venni, például `docker buildx imagetools inspect python:3.12-slim` vagy egy friss `docker pull` utáni `RepoDigests` mezőből. A multi-arch image-nek **manifest digestje** és platformonkénti digestje is lehet; a `--platform` rögzíti, hogy melyik architektúrára épül a kép. Ez azért fontos, mert az `arm64` és az `amd64` wheel-ek, glibc-verziók és CPU-utasítások eltérhetnek; a felhasználó MacBookja és a CI x86 szervere nem ugyanaz a környezet.

A build három, egymástól független azonosítót használ:

Réteg	Azonosító	Mit rögzít	Mit nem rögzít
Image	digest + platform	OS, glibc, Python build, rendszerkönyvtárak	adatot, modellt, alkalmazáskódot
Python-lock	`uv.lock` SHA-256	Python-csomagok pontos verzióit	interpreter buildjét, rendszerkönyvtárakat
Adat/Modell	splitmanifest, modellartefaktum hash	a konkrét adatot és modellt	a futtató környezetet

A konténer tehát **nem** az adatverzió és nem a modellverzió helyettesítője: ugyanaz az image más spliten más eredményt ad, és ugyanaz a split más image-en eltérő numerikus viselkedést mutathat. A friss pull utáni ellenőrzés: a digest egyezik-e a rögzített értékkel, a `uv.lock` hash-e egyezik-e, majd a `quickcheck.py` ugyanazt a kimenetet adja-e. A `tests/test_container_pins.py` a tag-only `FROM`, a hiányzó platform és a lock nélküli build mintáját utasítja el.

32.1.6 Feladatok

1. **Értelmezés.** A kolléga gépe rendelkezik a `uv.lock` másolatával, de nincs Docker. Magyarázd el, miért nem elegendő a lock egyedül ahhoz, hogy a lab modellképzése bitazonos környezetben fusson. Nevezz meg legalább két függőséget, amit a lock **nem** rögzít, és mondd meg, melyik réteg fedi le őket.
2. **Implementáció.** Egészítsd ki a fenti `.dockerignore`-t a következő elemekkel, és indokold meg, miért fontos mindegyik: `uv.lock` biztonsági másolata, `.ipynb_checkpoints`, `*.log`, `artifacts/split.json`. (A `uv.lock` maga a Dockerfile-ban másolódik, de a kérdés azt vizsgálja, mi kerüljön a build contextbe.)
3. **Hibadiagnózis.** A következő futási hibák közül válaszd ki, melyik utal konténer-konfigurációs hibára, és mi a javítás:

   - `FileNotFoundError: data/customers.csv`
   - `PermissionError: [Errno 13] Permission denied: '/app/artifacts/run.json'`
   - `ModuleNotFoundError: No module named 'numpy'`
   - `ERROR: uv sync: lockfile not found`

4. **Megváltozott feltétel.** A lab CI-futás mostantól nem bind mountot használ, hanem az adatokat egy Docker *named volume*-ba másolja a futás előtt. Hogyan változik a `:ro` szemantikája, és miért lehet mégis előnyösebb a bind mount a helyi fejlesztésben? Adj egy konkrét `docker volume create` és `docker run -v` példát.

32.1.7 Megoldások és indoklás

1. A `uv.lock` a Python-csomagok verzióját és ellenőrző adatait rögzíti, de a Python-interpretert magát az image `FROM python:3.12-slim` adja; a lock nem tartalmazza a glibc, a `libgomp` vagy a CA-tanúsítványok verzióját. A kolléga gépe lehet más Debian-verzió, más időzóna-adatbázis, vagy akár Windows, ahol a natív wheel-ek más fordítási beállításokkal futnak. Az ellenőrzés a megfelelő réteg: a lock a venv-függőségeket, az image a system-réteget. A kernel és a CPU-utasítás szintén nem rögzíthető a Docker image-szel; ez a 28.1. alfejezet device-kérdésének konténeres megfelelője.

2. `uv.lock` biztonsági másolata: maga a lock szükséges a Dockerfile-ban, ezért a `.dockerignore` akkor kapná el, ha a `COPY . .`-t írnánk; explicit `COPY pyproject.toml uv.lock ./` mellett a kettős bekerülés elkerülhető. A másolatot nem a contextből, hanem a copy-lépésből kell kihagyni; a `.dockerignore`-ban a `uv.lock` szerepeltetése hibás lenne, mert akkor a `COPY` sem éri el. `.ipynb_checkpoints`: notebook-mentések nem részei a futtatandó kódnak, és nagy, nem reprodukálható tartalmat vihetnek be. `*.log`: futási naplók a host feladata; a build contextbe kerülve félrevezető állapotot rögzítenek. `artifacts/split.json`: a futás kimenete, nem a build bemenete; a 31.1. alfejezet run-manifest logikája szerint a split a futáshoz tartozik, nem a környezethez. A context így csak a ténylegesen szükséges, forrás-verziózott fájlokat tartalmazza.

3. `FileNotFoundError: data/customers.csv`: a bind mount hiányzik vagy rossz az útvonal; a `-v "$(pwd)/data:/app/data:ro"` után a quickcheck relatív elérése igazítandó. `PermissionError` az artifacts mounton: a host könyvtár tulajdonosa nem a `10001` UID; írható host jogot vagy named volume-ot kell adni. `ModuleNotFoundError: No module named 'numpy'`: a `uv sync` nem futott le vagy a `VIRTUAL_ENV` PATH nem érvényes; a `RUN uv sync --locked --no-dev --no-install-project` réteg hibája. `uv sync: lockfile not found`: a `COPY pyproject.toml uv.lock ./` hiányzik, vagy a `.dockerignore` kizárja a lockot — a build ilyenkor joggal áll le, és a hibát a `COPY` és a `.dockerignore` együttes vizsgálatával kell javítani.

4. A named volume a Docker által kezelt tároló; a `:ro` itt azt jelenti, hogy a konténer olvashatja, de a futás alatt nem módosíthatja a volume-ot. A bind mount előnye fejlesztés közben, hogy azonnal látja a host fájljait; a named volume ehhez képest leválik a host elérési útról, így a bejárás nehezebb. Példa:

```
docker volume create northstar-data

docker run --rm \
  -v northstar-data:/app/data:ro \
  -v "$(pwd)/artifacts:/app/artifacts" \
  northstar-lab:dev
```

A named volume használata előtt az adatokat egyszer fel kell tölteni egy ideiglenes konténerrel vagy a Docker Desktop eszközével; ez a CI-ban megbízhatóbb, mert nem függ a host könyvtárstruktúrájától.

32.1.8 Gyakori tévedés ellenpéldával

„Az image tag azonosítja a tartalmat.” Ez téves. A `python:3.12-slim` tag egy mutató, amelyet a karbantartó újra kiadhat. Tegnap a `docker pull` még a `sha256:aaaa...` digestet adhatta, ma ugyanaz a tag a `sha256:bbbb...` tartalomra mutathat; a két gép buildje ezért eltérhet. Az ellenőrzéshez a tag mellé a digestet kell rögzíteni: `FROM python:3.12-slim@sha256:....` Ha egy rögzített digesttel megadott bázisimage új tartalmat kap a tag alatt, a digest továbbra is a régi tartalomhoz köt; az image letöltése akkor hibázik, ha a digest már nem érhető el. A frissítéshez új digestet kell választani és review-ban kezelni; ez a 31.4. alfejezet modellregistry-jéhez hasonló: a `champion` címke szintén mutálható, csak a tartalom-hash az azonosító.

Önellenőrzés: a futó konténerben ellenőrizd, hogy `id -u` értéke `10001`, a `sys.executable` `/opt/venv/bin/python`, és a `docker build` context-méret nem tartalmazza a `.venv`-et. Ha a context nagy, a `.dockerignore` hibás vagy hiányzik. A bind mount jogosultsági hibája esetén ne a Python-kódot javítsd először: előbb a host UID és a mount jogosultságát vizsgáld.

32.1.9 Rövid összefoglaló

A konténer-image az operációs rendszer, a rendszerkönyvtárak és a Python-interpreter rétegét rögzíti; a `uv.lock` ugyanazon belül a Python-csomagok verzióját. A Dockerfile explicit munkakönyvtárral, nem root userrel, kizárt adatokkal és `.dockerignore`-dzsal a lab futását hordozhatóvá teszi, de a kernel és a CPU-utasítások továbbra is változhatnak. A bind mount adatfájlként, a volume pedig tartós tárolóként szolgál; a tag helyett digest azonosítja a buildet. A következő alfejezetben 32.2. alfejezet a távoli interpretert és a notebook biztonságos elérését vizsgáljuk, ahol ezek a környezeti rétegek a távoli gépen is érvényesek maradnak.

32.2 Távoli interpreter és notebook biztonságos elérése

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Docker, SSH**
>
> Előfeltétel: 32.1. alfejezet · Célkészség: Távoli interpreter és notebook biztonságos elérése · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

32.2.1 Cél és konkrét helyzet

A Northstar tanulóprojekt `~/projects/northstar` könyvtára egy `devbox-07` nevű, SSH-val elérhető Linux CPU-gépre került. A helyi géped böngészőjében szeretnéd szerkeszteni a notebookot, de a kernelnek a távoli gépen kell futnia, hogy a CPU-igényes `starter_experiment.py` futtatása és a `lab/` adatfájlok elérése ne a hálózaton keresztül történjen. Ebben az alfejezetben pontosan megnevezzük, mi fut helyben és mi a távoli gépen, hogyan védjük a Jupyter hozzáférést, és hogyan rajzoljuk meg a munkamenet adat- és processztérképét.

A bemenet: az SSH hosztnév, a távoli felhasználónév, a projektkönyvtár elérési útja, valamint a helyi gépen egy böngésző. A megfigyelési egység a futó Jupyter szerverprocessz és annak hálózati kapcsolata. Az elvárt eredmény: a böngészőben megnyitott notebook cellája a távoli hosztnevet írja ki, és a távoli projektkönyvtárban lévő fájlokat éri el. Előzetes becslés: ha a `devbox-07` hosztneve valóban `devbox-07`, akkor az ellenőrző cella `socket.gethostname()` értéke ezzel egyezik. A biztonsági ellenőrzés invariánsa: a Jupyter szerver csak a `127.0.0.1`

loopback interfészen hallgat, a böngésző hozzáférése pedig SSH-alagúton át, token használatával történik.

32.2.2 Fogalmi magyarázat

A távoli notebookmunka három dolgot kever össze leggyakrabban: a **helyi szerkesztőt**, a **távoli processzt** és az **adat helyét**.

- A **böngésző** vagy a VS Code felület a helyi gépen fut. Ez csak megjeleníti a notebook JSON tartalmát, kódot küld a kernelnek, és fogadja a kimenetet.
- A **Jupyter szerver** és a **kernel** processz a `devbox-07` gépen fut. A notebook fájl is a távoli fájlrendszerben van, ezért a kernel közvetlenül látja a `~/projects/northstar/data/` könyvtárat.
- A **hitelesítő adat** (credential) azt bizonyítja, hogy jogunk van a távoli folyamathoz kapcsolódni. SSH-nál ez jellemzően kulcspár: a privát kulcs a helyi gépen marad, a nyilvános kulcs a távoli `~/.ssh/authorized_keys` fájlban van. A Jupyter saját belépési adata a token, amelyet a szerver indításakor adunk meg vagy generáltatunk.

Az **SSH** (secure shell, biztonságos távoli parancsértelmező) titkosított csatornát épít a helyi és a távoli gép között. A **port forwarding** (porttovábbítás) ezt a csatornát használja: a helyi gép egyik portjára érkező forgalmat az SSH kapcsolaton át a távoli gép megadott portjára továbbítja. A `-L 8888:localhost:8888` kapcsoló jelentése:

```
helyi 8888  ->  SSH csatorna  ->  távoli localhost:8888
```

A második `localhost` a távoli oldalon értendő, tehát a Jupyter szerver loopback címére mutat.

Miért kötjük a szervert a loopback interfészre? A Jupyter alapértelmezés szerint jelszót és tokent használ, de ha a szerver minden interfészen, `0.0.0.0`-n hallgat, akkor a hálózat más gépei is elérhetik. A `127.0.0.1`-re kötés azt jelenti, hogy a távoli gépre bejelentkezett felhasználók, illetve az SSH-alagút férhetnek hozzá, a külső hálózat nem. A **remote kernel** (távoli kernel) kifejezés arra utal, hogy a notebookkódot végrehajtó processz a távoli gépen fut; a böngésző sosem futtat Python-kódot.

A **VS Code Remote** egy másik útvonal ugyanarra a célra. A VS Code felület helyben marad, de a program egy VS Code Server nevű komponenst telepít a távoli gépre. A terminál, a fájlszerkesztő és a notebook kernel ezért a távoli gépen fut, és a VS Code saját csatornáján kezeli a hálózati forgalmat; ilyenkor a kézi porttovábbítás és a böngésző külön alagút nélkül működik.

32.2.3 Teljes minimális munkapélda

A következő munkamenet parancsai kipróbálhatók egy saját, SSH-val elérhető Linux gépen. Az itt közölt kimenet **szemléltető**, nem mért futási jegyzék: e könyvfejezet írásakor nem hajtottam végre a parancsokat egy adott `devbox-07` gépen, ezért a saját géped hosztneve, elérési útja és a Jupyter logrészlete el fog térni.

Előfeltétel. A távoli gépen már működik a Northstar projekt `uv` környezete, ahogyan azt a Python-környezetekről szóló részben felépítettük. A JupyterLab legyen a projekt dev függősége; ha hiányzik, a távoli gépen add hozzá:

```
# Távoli Linux terminál, munkakönyvtár: ~/projects/northstar
uv add --dev jupyterlab
uv sync --locked
```

1. lépés: ellenőrizd az SSH-hozzáférést a helyi gépről. A példában a helyi gép Windows 11 PowerShellt használ, a munkakönyvtár `C:\work`.

```
ssh devbox-07 "hostname && uv --version"
```

A várható, szemléltető kimenet első sora a távoli hosztnév, a második az `uv` verziója. Ha a hosztnév nem `devbox-07`, akkor a notebook ellenőrző cellájának is ezt a tényleges nevet kell mutatnia.

2. lépés: indítsd a Jupyter szervert a távoli gépen. Nyiss egy másik SSH-terminált, és fuss a távoli gépen:

```
# Távoli Linux terminál, munkakönyvtár: ~/projects/northstar
cd ~/projects/northstar
uv run jupyter lab --no-browser --ip=127.0.0.1 --port=8888 \
  --ServerApp.token='s126-example-token'
```

A kapcsolók jelentése:

- `--no-browser`: a távoli gépen ne próbáljon böngészőt nyitni.
- `--ip=127.0.0.1`: csak a loopback interfészen hallgasson.
- `--port=8888`: rögzített port, hogy az alagút célja egyértelmű legyen.
- `--ServerApp.token='s126-example-token'`: a belépési token. Ez itt példaértékű, tanulási célú; éles munkában ne használj kitalált, rövid tokent, hanem engedd, hogy a Jupyter véletlen tokent generáljon, vagy használj erős titkot.

A terminálban megjelenő, szemléltető log:

```
[I 2026-10-20 09:00:00.000 ServerApp] Jupyter Server is running at:
[I 2026-10-20 09:00:00.000 ServerApp] http://127.0.0.1:8888/lab?token=s126-example-token
```

A szerver mostantól fut, és az indító terminál foglalt.

3. lépés: nyiss SSH-alagutat a helyi gépről. Egy új helyi PowerShell ablakban:

```
ssh -N -L 8888:localhost:8888 devbox-07
```

A `-N` azt jelenti, hogy nem kérünk távoli parancsértelmezőt, csak a továbbítást tartjuk fenn. Ez a terminál is foglalt marad, amíg az alagút él.

4. lépés: nyisd meg a notebookot a helyi böngészőben. A cím:

```
http://localhost:8888/lab?token=s126-example-token
```

A böngészőben a `localhost` a helyi gép 8888-as portját jelenti; az SSH-alagút továbbítja a kérést a `devbox-07` loopback címére.

5. lépés: ellenőrző cella. Hozz létre egy új notebookot, és futtasd:

```
import socket
import pathlib

print("kernel_host:", socket.gethostname())
print("kernel_cwd:", pathlib.Path.cwd())
print("data_dir_exists:", pathlib.Path("data").is_dir())
```

A várható, szemléltető kimenet:

```
kernel_host: devbox-07
kernel_cwd: /home/youruser/projects/northstar
data_dir_exists: True
```

Ha a `kernel_host` a helyi géped neve lenne, akkor a kernel nem a távoli gépen futna, hanem véletlenül egy helyi Jupyter szolgálná ki.

6. lépés: ellenőrizd a szerver hálózati kötését. Egy újabb távoli SSH-parancsban:

```
ss -ltnp | grep 8888
```

A kívánt, szemléltető minta:

```
LISTEN 0      128          127.0.0.1:8888       0.0.0.0:*    users:(("python",pid=1234,fd=5))
```

A `127.0.0.1:8888` a fontos: ez mutatja, hogy a szerver nem hallgat a külső interfészeken. Ha itt `0.0.0.0:8888` vagy egy konkrét külső IP jelenik meg, a szerver nyilvánosan elérhető, és azonnal le kell állítani.

7. lépés: állítsd le. A szerver terminálján `Ctrl-C`, majd az alagút terminálján szintén `Ctrl-C` vagy kilépés.

32.2.4 Eredmény értelmezése

Az adat- és processztérkép a munkamenet lényege:

```
Helyi gép (Windows)                        Távoli gép (devbox-07, Linux)
+--------------------------+               +----------------------------------+
| Böngésző                 |   SSH-alagút  | Jupyter Server 127.0.0.1:8888    |
| localhost:8888           |===============| kernel processz (Python)         |
| notebook nézet, kódküldés|               | notebook fájl: ~/projects/...    |
|                          |               | adat: ~/projects/.../data/       |
+--------------------------+               +----------------------------------+
```

A böngésző a notebook felületét és a cellák kimenetét látja; a háttérben a szerver a távoli gépen tárolja a notebookot, és a kernel a távoli projektkönyvtárból olvassa az adatokat. A token a hozzáférés kulcsa: az URL `?token=` része a böngésző és a szerver közötti ideiglenes hitelesítés, nem a notebook tartalmának része.

Az előző alfejezetben tárgyalt konzisztens adat- és splitazonosítás itt is fontos: a `manifest.json` hash-e a távoli projektkönyvtárban van, tehát a kernel pontosan azt a fájlt látja, amelyet a futásrekordban rögzíteni fogunk. A tanítási ciklusról szóló rész tanulsága ide is átvihető: ha a notebook egy hosszan futó modellillesztést indít, az a távoli CPU-n fut, és a böngésző bezárása nem feltétlenül állítja le a kernel processzt; a szerver leállításakor azonban az összes hozzá tartozó kernel is megszakad.

Az SSH-porttovábbítás önmagában nem helyettesíti a Jupyter hitelesítését. Ne használjunk nyilvános bind címet vagy kikapcsolt tokent megfelelő izoláció nélkül.

32.2.4.1 Biztonságos alagút és veszélyes konfigurációk felismerése

A minimális, ellenőrizhető minta a loopbackre kötött helyi végpont és a kötelező token:

```
# Távoli gépen: erős token környezeti változóból, ne a parancssorból.
export JUPYTER_TOKEN="$(openssl rand -hex 32)"
uv run jupyter lab --no-browser --ip=127.0.0.1 --port=8888

# Helyi gépen: a helyi végpont is csak loopbackre köt.
ssh -N -o ExitOnForwardFailure=yes \
  -L 127.0.0.1:8888:127.0.0.1:8888 devbox-07
```

A `--ServerApp.token=` parancssori átadása azért veszélyes, mert a token bekerül a shell historyba és a `ps` kimenetébe; a `JUPYTER_TOKEN` környezeti változót a Jupyter közvetlenül olvassa. A `-L 0.0.0.0:8888:...` vagy a `ssh -g` a helyi hálózat más gépeinek is megnyitja az alagutat; a helyes bind cím `127.0.0.1`.

A tipikus veszélyes konfigurációk és a felismerésük:

Veszély	Rossz példa	Felismerés
Publikus bind	`--ip=0.0.0.0`, `--ip='*'`	`ss -ltnp | grep 8888` nem `127.0.0.1`-et mutat
Kikapcsolt hitelesítés	`--ServerApp.token=''`, `--IdentityProvider.token=''`	a szerver token nélkül indul; `jupyter server list` URL-jében nincs token
Root felhasználó	`--allow-root`	a folyamat UID-je 0
Széles alagút	`ssh -g -L 0.0.0.0:8888:...`	a helyi `ss` 0.0.0.0:8888-at mutat
Token a historyban	`--ServerApp.token=...`	`history`, `ps aux`, shell napló
Tűzfalnyitás	8888 a biztonsági csoportban	`ufw status`, felhő security group

Programozott ellenőrzés:

```
def audit_config(config, command=""):
    errors = []
    if config.get("ip") in {"0.0.0.0", "*", ""}:
        errors.append("publikus bind")
    if not config.get("token"):
        errors.append("hiányzó token")
    if config.get("allow_root"):
        errors.append("root felhasználó")
    if "--ServerApp.token=" in command or "--NotebookApp.token=" in command:
        errors.append("token a parancssorban")
    if " -g " in f" {command} " or "0.0.0.0:" in command:
        errors.append("túl széles alagút")
    return errors

def audit_listen(ss_output):
    bad = []
    for line in ss_output.splitlines():
        if ":8888" in line and "127.0.0.1:8888" not in line:
            bad.append(line.strip())
    return bad
```

Az elfogadási kontroll: a `ss -ltnp` kimenetében a 8888-as port csak `127.0.0.1:8888` lehet; egy külső gépről indított `curl --max-time 3 http://devbox-07:8888` nem kapcsolódhat, csak az SSH-alagúton keresztül. A `tests/test_notebook_security.py` a publikus bindet, a hiányzó tokent, a root módot, a parancssori tokent és a széles alagutat utasítja el.

32.2.5 Feladatok

1. **Értelmezés.** A fenti munkamenetben nevezd meg azt a három komponenst, amelynek más a helye, és mondd el, miért nem futtat a böngésző Python-kódot. Ha a böngésző címsorában `localhost`-ot látsz, miért lehet a `kernel_host` mégis `devbox-07`?

2. **Megvalósítás.** Írd le a három parancsot, amelyekkel a `devbox-07` gépen a `~/projects/northstar` könyvtárból elindítod a JupyterLab szervert `127.0.0.1:8888`-on, a helyi gépről megnyitod az alagutat, majd a böngészőből csatlakozol. Adj meg egy ellenőrző cellát is, amely a kernel hosztnevét és a projektkönyvtárban lévő fájlok számát írja ki. Mi az az invariant, amelyet a szerver logjában és a kernel kimenetében is ellenőrizni tudsz?

3. **Hibakeresés.** A kolléga azt mondja: a böngészőben `connection refused` hiba jelenik meg a `http://localhost:8888` címen, pedig a `ps aux | grep jupyter` a távoli gépen mutatja a folyamatot. Sorolj fel legalább három lehetséges okot, és mindegyikhez adj egy konkrét diagnosztikai parancsot a megfelelő gépen.

4. **Megváltozott feltétel.** Másnap nem böngészőt, hanem VS Code-ot szeretnél használni ugyanazon a `devbox-07` gépen. Hogyan változik a munkamenet a VS Code Remote SSH használatával? Mely komponens fut helyben, és melyik a távoli gépen? Miért nincs szükséged a `-L 8888:localhost:8888` alagútra?

32.2.6 Megoldások és indoklás

1. A három komponens: a böngésző mint helyi szerkesztő, a Jupyter szerver mint távoli folyamat, és a projekt adatfájljai mint távoli fájlrendszerbeli erőforrások. A böngésző JavaScriptet futtat a felülethez, de a notebookcellák Python-kódját a szervernek küldi el; a válasz a kernel által kiszámított szöveges kimenet. A `localhost` a böngésző szemszögéből a helyi 8888-as portot jelenti, amelyet az SSH-alagút továbbít a távoli `127.0.0.1:8888`-ra. Ezért a kernel hosztneve a távoli gépé.

2. A három parancs:

```
# Távoli terminál, munkakönyvtár: ~/projects/northstar
uv run jupyter lab --no-browser --ip=127.0.0.1 --port=8888 \
  --ServerApp.token='s126-example-token'

# Helyi PowerShell, munkakönyvtár: C:\work
ssh -N -L 8888:localhost:8888 devbox-07
```

A böngésző címe:

```
http://localhost:8888/lab?token=s126-example-token
```

Ellenőrző cella:

```
import socket
import pathlib

print("kernel_host:", socket.gethostname())
print("file_count:", len(list(pathlib.Path(".").iterdir())))
```

A várt kimenet a távoli hosztnév, és a projektgyökér fájljainak száma. Az invariant: a kernel hosztneve egyezik az `ssh devbox-07 hostname` parancs kimenetével, és a `ss -ltnp` szerint a szerver a `127.0.0.1:8888` címen hallgat.

3. Lehetséges okok:

- Nincs aktív SSH-alagút. Helyi diagnosztika Windows PowerShellben: `netstat -ano | findstr 8888`; ha nem jelenik meg `LISTENING` a helyi 8888-as porton, az alagút nem jött létre.

- A Jupyter szerver nem a 8888-as portot foglalta le, mert az már használatban volt, és automatikusan másik portot választott. Távoli diagnosztika: `ss -ltnp | grep 8888` és a szerver logjának ellenőrzése.
- A szerver nem `127.0.0.1`-en, hanem egy másik interfészen hallgat, így a `localhost:8888` cél nem éri el. Ugyanaz a `ss` kimenet választ ad.
- A helyi tűzfal blokkolhatja a loopback forgalmat ritka, szigorított beállításoknál; ezt a helyi operációs rendszer tűzfalnaplója vagy rövid `Test-NetConnection localhost -Port 8888` próba tisztázza PowerShellben.

4. VS Code Remote SSH esetén a VS Code felület marad a helyi gépen, de a távoli gépre települ egy VS Code Server. A terminál, a Python-interpreter, a kernel és a fájlhozzáférés a `devbox-07`-en fut. A VS Code a saját SSH-csatornáján proxyzza a notebookszervert, ezért a böngészőhöz tartozó kézi `-L` alagút felesleges. A notebook megnyitása a VS Code beépített Jupyter felületén történik, a kernel kiválasztásánál a távoli `uv` környezet kernelspecje látszik, amint azt a kernelregisztrációról szóló részben tárgyaltuk.

32.2.7 Gyakori tévedés ellenpéldával

Gyakori tévedés, hogy a notebook URL ismerete önmagában elegendő, és a token csak kényelmi kiegészítő, „úgysem jut el senki a portig, ha SSH-alagutat használunk”. Ez két ponton hibás.

Először: a token akkor is hitelesítő adat, ha az alagút szűkíti a hálózati expozíciót. Ha a szervert rosszul, `--ip=0.0.0.0 --ServerApp.token=''` módon indítod el, akkor a `http://devbox-07:8888` cím a hálózat minden, a gépet elérő szereplője számára nyitva van, és üres token mellett bárki notebookot nyithat, parancsot futtathat a távoli felhasználó jogosultságával. A porttovábbítás ilyenkor már nem véd, mert a szerver maga publikusan hallgat.

Másodszor: a tokent a futtatás részének kell tekinteni, nem a notebook tartalmának. Ellenpélda a biztonságos munkamenetből: ha a Jupyter logjából kimásolt `?token=...` URL-t beilleszted a notebook egyik markdown cellájába, majd a notebookot verziókezelőbe töltöd, a token a git-előzményekben is megjelenik. A szerver újraindításakor a token lejár, de a korábbi érték már nyilvános lehet. A helyes szokás: a token csak a böngésző címsorában vagy egy helyi, verziókezelt fájlon kívüli jelszókezelőben szerepeljen.

Önellenőrzés: indítsd el a szervert a fenti módon, és futtasd a `socket.gethostname()` cellát. Ha a kimenet a helyi géped neve, akkor valamelyik lépésben nem a távoli processzhez kapcsolódtál; ha a szerver `ss` kimenetében `0.0.0.0` szerepel, a hálózati kitettség hibás, még akkor is, ha a token erős.

32.2.8 Rövid összefoglaló

A távoli notebookmunka során a böngésző vagy a VS Code felület helyben fut, míg a Jupyter szerver, a kernel és a notebookfájl a távoli gépen. Az SSH-alagút a helyi `8888`-as portot köti a távoli `127.0.0.1:8888`-hoz, így a szerver nem válik nyilvános szolgáltatássá. A token a futtatás hitelesítő adata, ezért nem kerülhet notebookba vagy verziókezelőbe; a `ss -ltnp` ellenőrzése igazolja a loopback-kötést. A kernel hosztneve és a munkakönyvtár tartalma az a két kézzel ellenőrizhető invariáns, amely megmutatja, hogy valóban a távoli CPU dolgozik. A következő alkalmazásban ezt az izolált távoli környezetet CI-feladattá alakítjuk a 32.3. alfejezetben.

32.3 CI: kódellenőrzés és kis számítás új környezetben

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Docker, SSH

Előfeltétel: 32.2. alfejezet · Célkészség: CI: kódellenőrzés és kis számítás új környezetben · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

32.3.1 Helyzet, bemenet és előzetes becslés

A data scientist a Northstar labor tisztított adatelőállítását módosította. A helyi gépén a parancsok zöldek, de a távoli gépes munkamenetben, amelyet a 32.2. alfejezetben állított be, más operációs rendszer és más Python-telepítés fut. A kérdés: minden commit után friss, tiszta környezetben is teljesül-e a kis adatra vonatkozó szerződés, mielőtt bárki a drága modellkísérletre költ?

A CI-ellenőrzés bemenete a repository aktuális commitja, benne a `lab/generate_data.py`, a `lab/quickcheck.py`, a szállított tiszta CSV-k és a manifest. Megfigyelési egység egy teljes CI-futás; a reportban a rendelés az összegzési sor, a tétel a join-kardinalitás sor, az ügyfél pedig a randomizációs egység marad. A cél olyan kontrollaggregáció, amely hibás adatra nemnulla exit-kóddal és ember által olvasható reporttal áll le.

Előzetes becslés: az alap seeddel generált tiszta adaton `2400` rendelés, `4796` tétel és `300` ügyfél várható. Minden rendelésre a tételek `line_total_minor` összegének pontosan `basket_minor`-ral kell egyeznie. Egész minor unitban dolgozunk, ezért itt nincs lebegőpontos tűrés: az egyenlőség `==`-tel ellenőrzendő.

32.3.2 Mit jelent a CI, és mit nem bizonyít a zöld állapot?

A folyamatos integráció (continuous integration, CI) egy gépi szerződés: valamilyen eseményre — push, pull request, kézi indítás — friss környezetben lefuttat egy parancssort, és a teljes folyamat visszatérési kódját zöldre vagy pirosra minősíti. A CI *job* egy futtatási egység, a *workflow* több job összekötése, a *runner* pedig a Linux-, Windows- vagy macOS-gép, amely a jobot végrehajtja.

A zöld állapot mindig feltételes: „ez a workflow, ezen a runner-képpel, ezekkel a rögzített lépésekkel, ezzel az inputtal nem talált hibát”. Nem jelenti azt, hogy a modell minden támogatott platformon bitre azonosan reprodukálódik, hogy a statisztikai következtetés helyes, vagy hogy az üzleti cél teljesül. A 28.3. alfejezetben látott seed-korlát itt visszaköszön: a seed ismételhetőséget adhat azonos környezetben, de nem garantál bitazonosságot eltérő könyvtár-, BLAS- vagy processz-szálazási környezetben.

A zöld CI ezért szűk, pontos mondatokat támaszt alá: például „a CSV-séma beolvasható”, „a kulcsok egyediek”, „a tételszintű pénzösszeg megegyezik a rendelésszintű kosárösszeggel”.

32.3.3 Numerikus tolerancia és determinisztikus kontroll

A labor pénzadata egész minor unit, így az ellenőrzés egzakt. Amikor a jövőben lebegőpontos metrikát hasonlítunk referenciához, abszolút és relatív tűrést együtt kell használni:

$$|\hat{x} - x_{\mathrm{ref}}| \le \mathrm{atol} + \mathrm{rtol} \cdot |x_{\mathrm{ref}}| \,.$$

Itt $\hat{x}$ a mért érték, x_{ref} a referencia, atol az abszolút tűrés, rtol a relatív tűrés; mindkettő dimenziótlan vagy a mért mennyiség mértékegységében értendő. Ha `atol = 1e-9` és `rtol = 1e-6`, akkor a `0.1 + 0.2` Pythonbeli értékének `0.3`-től való, nagyjából `5.55e-17` eltérése elfogadható, mert a felső határ `1e-9 + 1e-6 * 0.3  3.01e-7`.

A determinisztikus inputokra célszerű byte-szintű lenyomatot is rögzíteni. Az SHA-256 hash az adott fájl bájtjainak egyirányú, determinisztikus lenyomata. Ha két fájl bájtszinten azonos, a hash azonos; ha a sortörés átalakult CRLF-re, a bájtok mások, így a hash is más lesz. Ezért a hash a CI-ben jó kontroll, de nem mondja meg, hogy a különbség tartalmi hiba vagy csak platformos sortörés.

32.3.4 Teljes minimális példa: Northstar CI-specifikáció

Az alábbi YAML szemléltető specifikáció, nem kipróbált futási jegyzék. A pontos parancsargumentumokat a labor projekt README-jéhez kell igazítani; a lépések szerkezete azonban átvihető bármely kis adatos repositoryba.

```yaml
# .github/workflows/northstar-ci.yml
name: northstar-ci
on:
  push:
    paths:
      - "lab/**"
      - "ci/**"
      - ".github/workflows/northstar-ci.yml"
  pull_request:

jobs:
  data-contract:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-24.04, windows-2022, macos-14]
        python: ["3.12", "3.13"]
    runs-on: ${{ matrix.os }}
    timeout-minutes: 10
    steps:
      - name: Repository kód kiadása
        uses: actions/checkout@v4 # éles használatban commit-SHA pin ajánlott
      - name: Python ${{ matrix.python }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
      - name: Deterministic data generation
        shell: bash
        run: python lab/generate_data.py
      - name: Contract check
        shell: bash
        run: python lab/quickcheck.py
      - name: Aggregation guard
        shell: bash
        run: >-
          python ci/aggregate_check.py
          --orders lab/data/orders.csv
          --items lab/data/order_items.csv
          --customers lab/data/customers.csv
          --out artifacts/ci/report.json
      - name: Report artifact
        uses: actions/upload-artifact@v4
        with:
```

```
          name: aggregation-report-${{ matrix.os }}-${{ matrix.python }}
          path: artifacts/ci/report.json
          retention-days: 7

  dependency-lock:
    runs-on: ubuntu-24.04
    timeout-minutes: 8
    steps:
      - uses: actions/checkout@v4
      - name: Cache uv cache
        uses: actions/cache@v4
        with:
          path: ~/.cache/uv
          key: uv-${{ runner.os }}-${{ hashFiles('lab/pyproject.toml', 'lab/uv.lock') }}
          restore-keys: |
            uv-${{ runner.os }}-
      - name: Install uv from project-pinned version
        run: python -m pip install uv
      - name: Environment sync from committed lock
        working-directory: lab
        run: uv sync --locked
```

A `data-contract` job csak a standard libraryt igénylő generátort, kontrollt és aggregációt futtatja. A `dependency-lock` job azt ellenőrzi, hogy a `uv.lock` a repositoryban van-e, és a `--locked` kapcsoló miatt akkor is pirosra vált, ha valaki kézzel megváltoztatta a `pyproject.toml`-t anélkül, hogy a lockot is frissítette volna. Ez szándékos: a CI nem oldhat fel csendben új verziót, mert akkor a zöld állapot egy másik függőségi környezetet igazolna.

32.3.4.1 Támogatott platform-, architektúra- és Python-mátrix

A kiadási teszt akkor bizonyít, ha a dokumentált platformok mindegyikén lefut. A javasolt mátrix:

Platform	Architektúra	Python	Kötelező smoke
`ubuntu-24.04`	x86_64	3.12, 3.13	generálás, quickcheck, aggregáció
`windows-2022`	x86_64	3.12, 3.13	generálás, quickcheck, aggregáció
`macos-14`	arm64	3.12, 3.13	generálás, quickcheck, aggregáció

A mátrix három tulajdonsága fontos. A `fail-fast: false` biztosítja, hogy egy platform hibája ne szüntesse meg a többi futását; így látszik, hogy a hiba platformspecifikus-e. A `shell: bash` a Windows-futón is egységes parancssort ad; a PowerShell-dokumentáció külön, emberi ellenőrzést igényel. A `macos-14` arm64, ezért a natív wheel-ek és a lebegőpontos viselkedés eltérhet az x86_64-től; a mátrix ezt nem fedi el. A Windows-os CRLF-eltérés ellen a repository `.gitattributes` fájljában `* text=auto eol=lf` szerepeljen, különben a CSV-hash és a generátor kimenete platformonként eltérhet.

A mátrix nem azt jelenti, hogy minden kombináció minden tudományos lépést lefuttat; a drága modellkísérlet továbbra is egy kijelölt környezetben fut. A **kiadási kapu** az, hogy a generátor, a quickcheck és az aggregáció minden támogatott platformon zöld, és a jelentés a platform/Python kombinációt rögzíti. A `tests/test_platform_matrix.py` a mátrix lefedettségét és a három kötelező smoke-lépést ellenőrzi.

A cache csak gyorsító, nem ellenőrzés. A kulcs a `pyproject.toml` és a `uv.lock` hashéből képződik, így változáskor új cache-ág épül. A `--frozen` és a `--locked` közötti különbségről a 3.2. alfejezetben tanultak érvényesek: a CI-ben a `--locked` a megfelelő, mert azt is ellenőrzi, hogy a lock nincs-e elavulva.

A kontrollaggregációhoz használt `ci/aggregate_check.py` standard library-only változata:

```
#!/usr/bin/env python3
"""Northstar determinisztikus aggregáció-ellenőrzés; stdlib only."""
import argparse
import csv
import hashlib
import json
import sys
from pathlib import Path

EXPECTED = {"customers": 300, "orders": 2400, "items": 4796}

def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def main() -> int:
    p = argparse.ArgumentParser()
    p.add_argument("--orders", type=Path, default=Path("lab/data/orders.csv"))
    p.add_argument("--items", type=Path, default=Path("lab/data/order_items.csv"))
    p.add_argument("--customers", type=Path, default=Path("lab/data/customers.csv"))
    p.add_argument("--out", type=Path, default=Path("artifacts/ci/report.json"))
    args = p.parse_args()

    order_ids, baskets, order_customer_ids, duplicate_orders = set(), {}, set(), 0
    with args.orders.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            oid = row["order_id"]
            if oid in order_ids:
                duplicate_orders += 1
            order_ids.add(oid)
            baskets[oid] = int(row["basket_minor"])
            order_customer_ids.add(row["customer_id"])

    item_spend, item_keys, unknown_refs, duplicate_item_keys = {}, set(), 0, 0
    with args.items.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            key = (row["order_id"], row["line_id"])
            if key in item_keys:
                duplicate_item_keys += 1
            item_keys.add(key)
            if row["order_id"] not in baskets:
                unknown_refs += 1
            line_total = (
                int(row["quantity"]) * int(row["unit_price_minor"])
                - int(row["discount_minor"])
            )
            item_spend[row["order_id"]] = (
                item_spend.get(row["order_id"], 0) + line_total
            )

    customer_ids, customer_rows = set(), 0
    with args.customers.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            customer_rows += 1
            customer_ids.add(row["customer_id"])

    mismatches = [
```

```
        {"order_id": oid, "expected_minor": expected,
         "actual_minor": item_spend.get(oid)}
        for oid, expected in baskets.items()
        if item_spend.get(oid) != expected
    ]

    report = {
        "expect_ref": EXPECTED,
        "actual": {
            "orders": len(order_ids),
            "items": len(item_keys),
            "customers": customer_rows,
        },
        "money_match": {
            "orders_with_items": len([o for o in baskets if o in item_spend]),
            "mismatch_count": len(mismatches),
            "mismatch_examples": mismatches[:3],
        },
        "key_checks": {
            "duplicate_order_ids": duplicate_orders,
            "duplicate_item_keys": duplicate_item_keys,
            "unknown_order_refs": unknown_refs,
            "missing_customer_examples": sorted(order_customer_ids - customer_ids)[:5],
        },
        "input_sha256": {
            "orders": sha256(args.orders),
            "items": sha256(args.items),
            "customers": sha256(args.customers),
        },
    }

    ok = (
        len(order_ids) == EXPECTED["orders"]
        and len(item_keys) == EXPECTED["items"]
        and customer_rows == EXPECTED["customers"]
        and duplicate_orders == 0
        and duplicate_item_keys == 0
        and unknown_refs == 0
        and not (order_customer_ids - customer_ids)
        and len(mismatches) == 0
    )

    args.out.parent.mkdir(parents=True, exist_ok=True)
    args.out.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n",
                        encoding="utf-8")
    print(json.dumps(report, ensure_ascii=False, indent=2))
    return 0 if ok else 1

if __name__ == "__main__":
    sys.exit(main())
```

A szkript shape-ellenőrzései egyben adatminőségi kapuk: a várt sor- és kulcsszám, a kulcsegyediség, az idegen kulcsok létezése és a pénzösszeg-egyezés. Az `artifacts/ci/report.json` csak a reportot és a hasheket tartalmazza, sem ügyfél-szintű nyers sort, sem titkot.

32.3.5 Kézi kis ellenőrzés a kardinalitási csapdára

A példa ellenőrzéséhez egy háromsoros mesterséges bemenetet érdemes kézzel számolni:

order_id	line_id	quantity	unit_price_minor	discount_minor	line_total_minor
O1	L1	2	100	0	200
O1	L2	3	50	0	150

Ha a rendelés `basket_minor = 350`, és `shipping_fee_minor = 90`, akkor a tételszintű összeg `200 + 150 = 350`, tehát a kosár-invariant teljesül. Ha viszont valaki a szállítási díjat tételszintű join után összegzi, az `O1` kétszer szerepel a joinban, és `180`-at kap `90` helyett. A 7.2. alfejezetben tanult one-to-many csapda így a CI-ban külön diagnosztikai értékként is megjelenhet; a helyes `shipping_totals`-t csak a rendelés-szemcsézettségű táblából szabad összegezni.

32.3.6 Drága tanítás külön lépésben

A Northstar `starter_experiment.py` prediktív futása és a 20.3. alfejezet idősor-baselineok kiértékelése nem minden commitra való. A CI-specifikációban a drága lépés legyen kézi indítású job:

```yaml
training:
  runs-on: ubuntu-24.04
  timeout-minutes: 45
  if: github.event_name == 'workflow_dispatch'
  needs: [data-contract, dependency-lock]
  steps:
    - uses: actions/checkout@v4
    - name: Full baseline experiment
      working-directory: lab
      run: uv run --locked python starter_experiment.py
    - name: Keep only small experiment artifacts
      uses: actions/upload-artifact@v4
      with:
        name: experiment-artifact
        path: artifacts/
        retention-days: 5
```

Itt nincs valódi felhős erőforrásigény: a kiinduló labor CPU-s, kis adatos. A `workflow_dispatch` miatt a drága tanítás csak akkor fut, amikor a csapat kéri. Az artifact retention rövid, és nem a teljes modellt, hanem a futási manifestet, split- és metrikafájlokat célozza. A 31.3. alfejezet betöltési határa érvényben marad: a CI által előállított artifact visszatöltésénél verzió- és sémavizsgálat kell.

32.3.7 Feladatok

1. **Értelmezés.** A CI `data-contract` jobja zöld. Írd le, mely állításokat támasztja alá ez a zöld állapot, és melyeket nem. Nevezd meg legalább két olyan következtetést, amelyet tilos ebből levonni.
2. **Implementálás.** Bővítse a `ci/aggregate_check.py`-t egy `shipping_duplication_guard` riportmezővel. A kód számítsd ki egyszer a rendelésenkénti `shipping_fee_minor` összeget, majd azt a hipotetikus összeget, amelyet az `order_items` tételszámával megszorzott szállítási díjak adnának. Mindkét értéket írja a `report`-ba `shipping_guard` kulcs alatt. A repo a Northstar tiszta CSV-it használja; nem kell valódi felhőt hívni.

3. **Hibadiagnózis.** A `dependency-lock` job a `uv sync --locked` lépésnél piros lesz: „lockfile does not exist”. A fejlesztő erre a `--frozen`-re cserélné a parancsot, hogy a CI zöld legyen. Mondja meg, miért rossz ez a diagnózis, és mi lenne a helyes javítás.

4. **Megváltozott feltétel.** A csapat úgy dönt, hogy a prediktív baseline-t hetente egyszer futtatja a CI-ben, de továbbra sem minden commitra. Írd át a workflow-t úgy, hogy a `training` job ütemezett `cron` ütemben is induljon, és a `data-contract` job továbbra is minden pushra fusson. Az artifact megőrzési idő maradjon 5 nap.

32.3.8 Megoldások és indoklás

1. A zöld `data-contract` job azt igazolja, hogy az adott runner-képen, az adott commitban a generátor lefutott, a quickcheck szerződése teljesült, a CSV-k beolvashatók voltak, a várt sor- és kulcsszámok egyeztek, és a tételszintű pénzösszeg minden beolvasott rendelésre megegyezett a `basket_minor` értékkel. Nem igazolja, hogy a modell jövőbeli predikciója pontos, hogy a kísérleti ITT-hatás konfidenciaintervalluma minden új mintán fed, vagy hogy a workflow egy másik platformon is bitre azonos modellt hozna létre. A zöld állapot csak a definiált adatszerződést ellenőrizte.

2. A reportba illeszthető kontroll minimális kiegészítése:

```
correct_shipping = 0
hypothetical_shipping = 0
with args.orders.open(newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        count = 0
        for item_row in item_count_by_order.get(row["order_id"], []):
            count = item_count_by_order[row["order_id"]]
        correct_shipping += int(row["shipping_fee_minor"])
        hypothetical_shipping += count * int(row["shipping_fee_minor"])
```

A megoldásnak pontos adatszerkezetet kell adnia: `item_count_by_order[oid]` az `order_id`-hoz tartozó `line_id`-k száma. Az elvárt invariant `hypothetical_shipping >= correct_shipping`, és a különbség `sum((item_count - 1) * shipping_fee_minor)`. A kontroll nem hibát dob feltétlenül, hanem a reportban láthatóvá teszi a kardinalitási csapda lehetséges mértékét. Ellenőrzése a belinkelt tiszta CSV-k hashelt bemenetén, megegyező műveleti sorrend mellett reprodukálható; lebegőpontos tűrésre itt nincs szükség, mert a pénz egész cent.

3. A `--frozen` kihagyja a lock aktualitásvizsgálatát, tehát a CI egy olyan lockot fogadna el, amely nem tükrözi a `pyproject.toml`-t. Ezzel nem oldódik meg a hiány vagy az elavulás, csak elrejtjük a piros jelzést. A helyes javítás: a fejlesztő a lokális projektben futtatja az `uv lock` parancsot, a keletkezett `uv.lock`-ot commitolja, majd a CI-t újra futtatja. A cache jelenléte itt irreleváns, mert a cache tárolt csomagokat, nem lockállományt.

4. A `training` job indítási feltétele legyen:

```
on:
  push:
    paths: [...]
  pull_request:
  schedule:
    - cron: "30 4 * * 1"
  workflow_dispatch:
```

A `training` jobban a feltétel:

```
if: github.event_name == 'workflow_dispatch' ||
    github.event_name == 'schedule'
```

A `data-contract` job marad push-onként. A pontos `cron` kifejezést a repository UTC-szabályaihoz kell igazítani; az itteni `30 4 * * 1` hetfőn 04:30 UTC-t jelöl. A cron ütemezés a platform szolgáltatása, ezért a pontos időzítés a szolgáltató fiókbeállításaitól függhet.

32.3.9 Gyakori tévedés és korlát

Tévedés: „A CI zöld lett, tehát a modell a célplatformon is reprodukálódik." Ellenpélda: a CI Linux-ubuntu runneren futtatott egy szkicit-learn pipeline-t, és az artifactot egy Windows-os gép tölti vissza. A modell float-számításai a NumPy verziójától, a BLAS-tól és a szálak összegzési sorrendjétől függően kicsit eltérhetnek. A CI nem futtatta a tényleges célplatformot, így a bitazonosságot nem is mérte. Hasonló korlátot mutatott a 28.3. alfejezet a seed-ek platformfüggőségénél és a 31.3. alfejezet az artifact-visszatöltésnél: a zöld pipeline nem menti fel a csapatot a betöltéskor futtatott séma- és predikciós szerződésvizsgálat alól.

32.3.10 Önellenőrzés

- A workflow-hoz nincs szükség valódi titokra, külső API-ra vagy fizetős cloudra.
- A `data-contract` job timeouttal, tiszta Ubuntu-képpel és standard library-only lépésekkel fut.
- A `dependency-lock` job a `--locked` kapcsolót használja, a cache pedig a `pyproject.toml` és a `uv.lock` hashére épül.
- A report csak aggregált számokat, hasheket és első hibapéldákat tartalmaz; a model artifact nem kerül minden commit-ra retentionbe.
- A zöld állapotról szóló mondat pontosan a futtatott jobra szorítkozik.

A CI zöld státusza nem bizonyítja, hogy a modell statisztikailag megfelelő; a tesztek csak a kifejezetten megfogalmazott szerződéseket ellenőrzik.

32.3.11 Rövid összefoglaló

A CI a repository minden pushját friss, tiszta Linux-környezetben futtatott kis adatos szerződésként ellenőrzi. A Northstar-példában a generátor, a quickcheck és a kontrollaggregáció egy gyors standard library-only jobban fut: kulcsegyediséget, várható sor- és tételszámot, valamint tételszintű pénzösszeg-egyezést ellenőriz. A függőségek szinkronizálása `uv sync --locked` módon történik, a cache csak gyorsít. A drága modelltanítás kézi vagy ütemezett külön job, rövid artifact retentionnel. A zöld CI szűk, feltételes állítás, nem teljes platformos modellreprodukció. A következő lépésben a VIII. ellenőrző pont ezekből az elemekből állít össze átadható modellprojektet, beleértve a betöltési határokat és a futási manifestet.

32.4 VIII. ellenőrző pont: átadható modellprojekt

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Docker, SSH

Előfeltétel: 32.3. alfejezet · Célkészség: VIII. ellenőrző pont: átadható modellprojekt · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A VIII. ellenőrző pont azt kérdezi, hogy a projekt ténylegesen átadható-e egy másik futtatási környezetnek. Nem a fájlok megléte a kérdés, hanem az, hogy egy friss gépen ugyanaz a bemenet, kód és modell ugyanazt az ismert kontrollt állítja-e elő. Az alábbi munkamenet a Northstar késésmodellt csomagolja be, és a tanítás nélküli predikciós utat ellenőrzi.

32.4.1 Cél és konkrét helyzet

A feladat a 28.4. alfejezet ellenőrző pontból ismert tabuláris Northstar-modell átadása: rendeléskor kell becsülni a 7 napot meghaladó kézbesítés valószínűségét. A modell itt logisztikus regresszió, mert CPU-s és értelmezhető; a tanítás a fejlesztőgép saját, ellenőrzött Python környezetében történik, a drága lépést nem ismételjük a konténerben.

Bemenetek:

- `data/customers.csv` és `data/orders.csv`: a kezdőlabor tiszta adata.
- `artifacts/split.json`: az időbeli split manifestje, a 20.4. alfejezetből ismert cutoff és label-availability szűrés rögzítésével.
- `config/model_v3.json`: a modellverzióhoz tartozó explicit konfiguráció.
- `models/model_candidate_v3.joblib`: a saját gépen betanított modelljelölt.

Megfigyelési egység: egy rendelés, `order_id`. Kimenet: minden teszt-rendelésre egy $[0, 1]$-beli predikciós valószínűség, a teszt-AUC, a feldolgozott sorok száma és egy futáskövető napló. Ellenőrzési feltétel: a bemeneti fájlok SHA-256 lenyomata megegyezik a manifestben rögzítettel, a tesztmátrix alakja egyezik, és a teszt-AUC a várt értéktől legfeljebb 10^{-6} abszolút eltéréssel tér el. A tűrés azért ilyen szigorú, mert ugyanaz a modell és ugyanaz az adat determinisztikus predikciót ad; a környezeti különbség csak lebegőpontos zajt okozhat.

Induló becslésként a modell teszt-AUC-ja a tanuló saját `fit_model.py` futtatásából ismert. A leckében példaként 0,7312 szerepel, de ez szemléltető szám; a tényleges várt értéket a tanuló a saját futásból írja be. Az átadás akkor sikeres, ha a konténerben kapott AUC a várt érték tűrésén belül van.

32.4.2 Fogalmi magyarázat a hét szállítási elemről

A **package** (csomag) itt nem pusztán mappa, hanem telepíthető, verziózott Python-projekt. A `src` elrendezés elválasztja az importálandó kódot a teszt- és szkriptfájloktól, így a predikciós kód importálható a konténerben, a tesztben és a fejlesztői notebookban is. A csomag neve és függőségei a `pyproject.toml`-ban vannak; a telepítés az `uv sync --locked` paranccsal a lockfájlt használja, nem a lebegő verziótartományt. A lockfájlt a fejlesztő a saját gépén oldja fel; a konténer már nem keres új csomagverziót.

A **tests** (tesztek) szerepe az átadásban a szerződéses viselkedés rögzítése. A unit test (egységteszt) tiszta függvényekre vonatkozik, például a feature-számításra; az integration test (integrációs teszt) a teljes betöltés–predikció utat járja be. A konténerben futó predikció maga is egy integrációs teszt: ugyanazt a parancsot futtatja, mint a CI, így a „nálam fut” és a „másik gépen fut” közötti különbség nem marad rejtve.

A **config** (konfiguráció) a futtatás változóinak explicit, verziózott leírása. A cutoff, a feature-lista, a modellfájl elérési útja és a várt AUC a konfigban van, nem a kódban. Ez akkor lesz fontos, amikor a split megváltozik: új config, új modellverzió, de a kód változatlan.

A **data hash** (adatlenyomat) az a kriptográfiai ellenőrző összeg, amely a fájl bájtjaihoz rendel egy 256 bites értéket. Az SHA-256 függvény determinisztikus: ugyanaz a fájl minden gépen

ugyanazt a 64 karakteres hexadecimális szöveget adja. A lenyomatot az előre rögzített manifest tartalma ellenőrzi:

$$H = \mathrm{SHA256}(b_1 \| b_2 \| \ldots \| b_n)$$

ahol b_i a fájl bájtjainak sorozata, $\|$ az összefűzés. A hash csak a fájl integritását igazolja, nem a tartalom tudományos helyességét; a Northstar esetében a manifest rögzíti a seedet és a sorok számát is.

A **run tracking** (futáskövetés) minden futtatáshoz egyedi azonosítót és rendezett naplót rendel. A futásazonosító a bemeneti állapotból számítható:

$$r = \mathrm{SHA256}(\mathrm{commit} \| \mathrm{config_hash} \| \mathrm{data_hash} \| \mathrm{model_hash})_{[0:12]}$$

A napló tartalmazza a környezeti verziókat, a metrikákat és a kimeneti fájlok hash-eit. Így egy későbbi kérdésre megmondható, melyik konfiguráció melyik AUC-t produkálta.

A **container** (konténer) a futtatókörnyezetet csomagolja: operációs rendszer, Python-verzió, natív függőségek. A Dockerfile a lockfájllal telepíti a csomagokat, ezért a konténer és a fejlesztői környezet ugyanazt a feloldott függőséghalmazt látja. A VS Code Dev Container ugyanezt a Dockerfile-t használja interaktív munkához, a Remote SSH pedig egy másik gépen lévő projektet ér el; mindkettőnél a parancssori predikció marad az ellenőrzés alapja.

A **recovery** (helyreállítás) a hibás átadás utáni állapotkezelés. A konténerben futó szkript hiba esetén nullától különböző kóddal lép ki, és nem ír semmit a regisztrált champion állapotba. A korábban jóváhagyott modell a 31.4. alfejezetből ismert registryben sértetlen marad; a hibás jelöltet a napló azonosítja. A helyreállítás tehát nem a hibás modell megjavítása, hanem a korábbi állapot megőrzése és a hiba diagnosztizálása.

32.4.3 Teljes minimális munkapélda

A projekt gyökérkönyvtárában az alábbi szerkezetet hozzuk létre:

```
northstar_model/
  pyproject.toml
  uv.lock                 # a fejlesztőgép által feloldott lock, nem a labor része
  Dockerfile
  src/northstar_model/
    __init__.py
    data_hash.py
    split.py
    features.py
    tracking.py
  scripts/
    fit_model.py
    predict.py
  config/
    model_v3.json
  models/
    model_candidate_v3.joblib
  data/
    customers.csv
    orders.csv
  artifacts/
    split.json
```

A `pyproject.toml` verziótartományt ad, a `uv.lock` a tényleges feloldást. A leckében a könyvgenerátor nem futtatta a lockolást; a tanuló a saját `uv lock` kimenetét használja. A függőségek a kezdőlabor alap stackjéből valók:

```
[project]
name = "northstar-model"
version = "0.3.0"
requires-python = ">=3.12,<3.13"
dependencies = [
  "numpy>=1.26,<2",
  "pandas>=2.2,<3",
  "scikit-learn>=1.5,<2",
  "joblib>=1.4,<2"
]
```

A hash-ellenőrzés tiszta függvénye:

```
# src/northstar_model/data_hash.py
import hashlib
from pathlib import Path

def sha256_hex(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()
```

A split a 20.4. alfejezet időhelyes protokollját kódolja: a tanításhoz csak olyan múltbeli rendelés használható, amelynek címkéje a cutoff előtt megérkezett; a teszt a cutoff utáni rendelés. A purged sávot a tanításból elhagyjuk. A függvény a `split.json` manifestet olvassa, és a `target_late` oszlopot a `delivery_days > promised_days` összefüggésből képezi:

```
# src/northstar_model/split.py
import pandas as pd

def make_time_split(frame: pd.DataFrame, cutoff: pd.Timestamp):
    label_ready = frame["label_available_at"] < cutoff
    train = frame[(frame["order_ts"] < cutoff) & label_ready].copy()
    test = frame[frame["order_ts"] >= cutoff].copy()
    train["target_late"] = (train["delivery_days"] > train["promised_days"]).astype(int)
    test["target_late"] = (test["delivery_days"] > test["promised_days"]).astype(int)
    return train, test
```

A feature-előállítás a rendelés pillanatában ismert oszlopokat használja. A kategorikus változókat one-hot kódolással alakítjuk numerikus mátrixszá, a kódolást a train oszlopain tanítjuk, és a tesztre alkalmazzuk. A `account_age_days` az `order_ts` és a korábban ismert `signup_date` különbsége:

```
# src/northstar_model/features.py
import pandas as pd

CATEGORICALS = ["segment", "region", "acquisition_channel"]
NUMERICALS = ["distance_km", "priority_flag", "basket_minor",
              "quantity_sum", "account_age_days"]

def build_features(df: pd.DataFrame, train_categories=None):
    df = df.copy()
    df["account_age_days"] = (df["order_ts"] - df["signup_date"]).dt.days
    X = df[NUMERICALS + CATEGORICALS].copy()
    X = pd.get_dummies(X, columns=CATEGORICALS, dtype=float)
    if train_categories is not None:
```

```
        X = X.reindex(columns=train_categories, fill_value=0.0)
    return X
```

A fejlesztőgépen futó `fit_model.py` tanít, kiszámolja a várt AUC-t, és elmenti a modellt, a tesztsorazonosítókat és a várt értékeket. A kód a sklearn `LogisticRegression` osztályát használja, `random_state=42` rögzítéssel. A tanítás után az AUC a teszt pozitív és negatív példáinak páronkénti konkordanciája:

$$\mathrm{AUC} = \frac{1}{n_+ n_-} \sum_{i : y_i = 1} \sum_{j : y_j = 0} \left[I(s_i > s_j) + \tfrac{1}{2} I(s_i = s_j) \right]$$

ahol s_i a modell által adott pontszám, n_+ és n_- a pozitív és negatív tesztesetek száma, I pedig indikátorfüggvény. A tanuló a `manifest.json`-ból beírja a várt AUC-t a configba.

A konténerben futó `predict.py` a teljes átadási ellenőrzést végzi:

```
# scripts/predict.py
import json, sys
from pathlib import Path
import joblib
import pandas as pd
from sklearn.metrics import roc_auc_score
from northstar_model.data_hash import sha256_hex
from northstar_model.features import build_features
from northstar_model.split import make_time_split

def fail(msg):
    print(f"ERROR: {msg}", file=sys.stderr)
    sys.exit(1)

def main():
    cfg = json.loads(Path("config/model_v3.json").read_text())
    for key, rel in cfg["data"]["files"].items():
        actual = sha256_hex(Path(rel))
        if actual != cfg["data"]["hashes"][key]:
            fail(f"hash mismatch for {key}: {actual}")
    customers = pd.read_csv(cfg["data"]["files"]["customers"],
                            parse_dates=["signup_date"])
    orders = pd.read_csv(cfg["data"]["files"]["orders"],
                         parse_dates=["order_ts", "actual_delivery_at",
                                      "label_available_at"])
    frame = orders.merge(customers, on="customer_id", how="left")
    cutoff = pd.Timestamp(cfg["split"]["cutoff"])
    _, test = make_time_split(frame, cutoff)
    expected_rows = cfg["split"]["expected_test_rows"]
    if len(test) != expected_rows:
        fail(f"test row count {len(test)} != {expected_rows}")
    model = joblib.load(cfg["model"]["path"])
    X_test = build_features(test)
    pred = model.predict_proba(X_test)[:, 1]
    auc = roc_auc_score(test["target_late"], pred)
    expected_auc = cfg["model"]["expected_auc"]
    if abs(auc - expected_auc) > cfg["model"]["auc_tol"]:
        fail(f"AUC drift: {auc:.6f} vs {expected_auc:.6f}")
    run_id = make_run_id(cfg, model)
    out = Path("artifacts/runs") / run_id
    out.mkdir(parents=True, exist_ok=True)
    (out / "run.json").write_text(json.dumps({
        "run_id": run_id, "auc": auc, "expected_auc": expected_auc,
        "test_rows": len(test)
    }, indent=2))
```

```
    print(f"[{run_id}] data hash ok")
    print(f"[{run_id}] test rows: {len(test)}")
    print(f"[{run_id}] AUC: {auc:.6f}, expected: {expected_auc:.6f}, diff:
     ↪  {abs(auc-expected_auc):.6f}")

if __name__ == "__main__":
    main()
```

A futásazonosítót a bemeneti hash-ekből számító `make_run_id` a `tracking.py`-ban él; itt röviden csak annyi, hogy a konfig, adat- és modellhash konkatenációjának első 12 karakterét adja. A fenti kódban a `make_run_id` importját a tömörség kedvéért elhagytuk, a teljes változat a labban van.

A Dockerfile a lockfájlt másolja, és `--locked` móddal telepít:

```
FROM ghcr.io/astral-sh/uv:python3.12-bookworm-slim
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev --no-cache
COPY src ./src
COPY config ./config
COPY models ./models
COPY scripts ./scripts
COPY data ./data
COPY artifacts/split.json ./artifacts/
ENTRYPOINT ["python", "scripts/predict.py"]
```

A build és futtatás Linuxon vagy WSL-ben, a projektgyökérben:

```
# Feltéve, hogy a uv.lock már létezik a projektgyökérben
docker build -t northstar-model:0.3.0 .
docker run --rm northstar-model:0.3.0
```

A szemléltető, nem e könyvíráskor mért kimenet:

```
[3f9a1c2d4e] data hash ok
[3f9a1c2d4e] test rows: 780
[3f9a1c2d4e] AUC: 0.731200, expected: 0.731200, diff: 0.000000
```

A `data` és `models` mappák a konténer contextjébe kerülnek; a `models/model_candidate_v3.joblib` saját generálású fájl, a hash-ellenőrzés megerősíti, hogy ugyanazt a fájlt töltjük be, amelyet a fejlesztőgép jegyzett. Ismeretlen forrású joblib/pickle fájlt továbbra sem töltünk be.

32.4.4 Eredmény értelmezése

A sikeres futás három egymásra épülő invariánst ellenőriz. A hash-egyezés azt mutatja, hogy a bemeneti fájlok bitre azonosak a manifesttel; ez szükséges feltétele annak, hogy a split és a feature-mátrix azonos legyen. A sorok számának egyezése azt mutatja, hogy a cutoff és a label-availability szűrés ugyanazt a tesztpopulációt adja; ha a purged sávot valaki véletlenül a tesztbe sorolná, a sorok száma eltérne. Az AUC-eltérés nulla vagy a tűrés alatt van, mert a modell és a tesztadat determinisztikus; a 10^{-6} tűrés a lebegőpontos kerekítésre hagy mozgásteret. A statisztikai reprodukálhatóság itt erősebb, mint egy seed alapú újratanítás esetén, mert a már betanított modellt töltjük be.

32.4.4.1 Tiszta átadási próba: a kötelező sorrend

Az átadás akkor bizonyított, ha **üres környezetből**, előzetes cache, `.venv`, `data/` és `artifacts/` nélkül végigfut a rögzített sorrend. A lépések:

1. **Tiszta checkout.** `git clone` egy friss könyvtárba, majd `git rev-parse HEAD` és a várt commit-azonosító egyezése; `git status --porcelain` legyen üres.
2. **Hash-check.** A manifestben rögzített adat- és modellhash ellenőrzése a telepítés **előtt**; hiányzó vagy eltérő fájl esetén a folyamat álljon le a fájl nevével.
3. **Telepítés a lockból.** `uv sync --locked --no-dev`; a `uv --version` és a `python --version` kerüljön a naplóba.
4. **Adatgenerálás.** `python generate_data.py` friss `data/` könyvtárba; utána a manifeszt sorszámai és hash-ei egyezzenek.
5. **Smoke test.** `python quickcheck.py` és a predikciós szkript futása; a kimenet hash-e és a várt AUC a naplóba.
6. **Determinisztikus kontrollsorrend.** Ugyanez a hat lépés ismételhető legyen kézi beavatkozás nélkül; két tiszta futás kimeneti hashje egyezzen.

```
set -euo pipefail
git clone --depth 1 --branch v1.0.0 https://example.invalid/northstar-model.git
cd northstar-model
test -z "$(git status --porcelain)"
sha256sum -c artifacts/manifest.sha256
uv sync --locked --no-dev
python generate_data.py
python quickcheck.py
python predict.py --config configs/model_v3.json --out artifacts/run.json
sha256sum artifacts/run.json
```

A negatív teszt is része az átadásnak: töröljük vagy nevezzük át a `data/customers.csv` fájlt, és a folyamatnak **konkrét fájlnévvel** kell megállnia, nem egy véletlen `KeyError`-ral a mélyén. A `tests/test_handoff_sequence.py` a kötelező lépéssorrendet, a hiányzó előfeltétel konkrét hibáját és a két tiszta futás azonos kimeneti hash-ét teszteli.

A kódblokk után nem maradhat puszta szám: a `diff: 0.000000` azt jelenti, hogy a konténer ugyanazt a modellt és adatot látta, tehát a korábban a 28.4. alfejezetben közölt AUC-állítás ebben az új környezetben is érvényes. Ha a diff például 0,002 lenne, az nem véletlen, hanem szerződéssértés jele.

Az átadhatósági kapu értékeléséhez dokumentálni kell a tesztplatformot, az artefaktumok hashét és a ténylegesen lefuttatott ellenőrzéseket.

32.4.5 Feladatok

1. **Értelmezés.** Miért nem elegendő csak a `model_candidate_v3.joblib` fájlt átmásolni a másik gépre? Nevezd meg legalább két hiányzó összetevőt, amelyek nélkül a predikció nem reprodukálható, és mindegyikhez írja le, milyen hibát okoz a hiányuk.
2. **Implementálás.** Írd meg a `make_run_id` függvényt a `src/northstar_model/tracking.py` fájlban úgy, hogy a konfigurációs fájl SHA-256 lenyomata, a modellfájl lenyomata és a `customers.csv+orders.csv` lenyomatok vesszővel összefűzött szövege adja a bemenetet, és a kimenet az így kapott SHA-256 hexa szöveg első 12 karaktere legyen. A függvénynek `str` paramétereket kell elfogadnia, és a `predict.py` ezt hívja. Add meg a függvény teljes kódját és egy kétlépéses kézi ellenőrzést az `abc` és `def` bemenetre.

3. **Hibadiagnózis.** A konténerfutás ezt írja: `AUC drift: 0.7291 vs 0.7312`, és a folyamat 1-es kóddal kilép. A hash- és sorszám-ellenőrzés sikeres volt. Sorolja fel a lehetséges okokat fontossági sorrendben, és mindegyikhez adjon egy ellenőrzési parancsot vagy diagnosztikai lépést, amely eldönti, hogy az adott ok fennáll-e.

4. **Megváltozott feltétel.** A projektvezető a predikciós ablakot áthelyezi: a jövőbeli használatban a cutoff `2025-06-01T00:00:00+00:00` lesz `2025-05-01` helyett. Az adatfájlok nem változnak. Mi marad változatlan, és mit kell újragenerálni vagy újratanítani? Add meg a konfiguráció és a registry állapotok javasolt új verzióit, a 31.4. alfejezetben tanult candidate/champion fogalmakkal.

32.4.6 Megoldások és indoklás

1. A modellfájl csak a betanított együtthatókat és a sklearn pipeline belső állapotát tartalmazza. Hiányzik a feature-előállítás konkrét kódja: ha a kategorikus oszlopok kódolása más sorrendű vagy más oszlopkészletű, akkor a `predict_proba` rossz oszlopokra hivatkozik, és téves vagy hibás predikciót ad. Hiányzik a split manifest: a teszt-AUC csak akkor hasonlítható össze, ha ugyanazok a rendelések alkotják a tesztet. Ha a cutoff nincs rögzítve, a 20.4. alfejezet időbeli validáció nem ismételhető, és a tesztbe kerülhetnek olyan múltbeli címkék, amelyek a gyakorlatban nem lennének ismertek. Végül a bemeneti adat verziója is hiányozhat: ha a másik gép régebbi `orders.csv`-t lát, a hash ellenőrzése nélkül nem derül ki.

2. A függvény:

```
import hashlib

def make_run_id(*hashes: str) -> str:
    payload = ",".join(hashes)
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()[:12]
```

Kézi ellenőrzés: `make_run_id("abc", "def")` esetén a szöveg `"abc,def"`, ennek SHA-256 hexája `cb8e...` kezdetű; a pontos első 12 karaktert a tanuló a `hashlib` segítségével kiszámolja, majd ugyanazt a bemenetet kétszer futtatva ellenőrzi, hogy a kimenet determinisztikus. A teszt szempontjából az a fontos, hogy a függvény ugyanazt a sorrendet használja, mint a `predict.py`; ezért a hívás `make_run_id(config_hash, data_hash, model_hash)` alakú.

3. A lehetséges okok:

- A `predict.py` nem ugyanazt a feature-készletet állítja elő, mint a tanító `fit_model.py`. Például az `account_age_days` oszlop hiányzik, vagy a kategorikus kódolás eltérő dummy oszlopokat ad. Diagnózis: írjuk ki `X_test.columns` listáját, és hasonlítsuk össze a tanításkor mentett oszloplistával.
- A modellfájl nem az a verzió, amelyhez az elvárt AUC tartozik. Bár a hash-ellenőrzés a modellfájlt is érintheti, ha a configban nem szerepel modellhash, akkor a fájl kicserélődhetett. Diagnózis: `sha256sum models/model_candidate_v3.joblib` összehasonlítása a tanítási naplóval.
- A sklearn verzió eltérés miatt a betöltött logisztikus regresszió belső reprezentációja másképp kerekít. Diagnózis: a konténerben `python -c "import sklearn; print(sklearn.__version__)"`, összevetni a tanítási környezet verziójával.
- A teszt halmaz nem teljesen azonos: bár a sorok száma egyezik, a sorok azonosítói eltérhetnek, ha a rendezés vagy a szűrés sorrendje más. Diagnózis: mentsük el a teszt `order_id` listáját a `split.json`-ba, és hasonlítsuk össze halmazként.

4. Az adatfájlok hash-e változatlan, tehát az adatverzió nem változik. A split manifest új generálást igényel, mert a cutoff eltolásával több múltbeli rendelés címkéje válik elérhetővé a tanításkor, és a tesztablak is bővül. A modellt újra kell tanítani az új tanítóhalmazon, ezért a modellhash is változik. A konfiguráció új verziót kap, például `model_v4.json`, az elvárt AUC-t az új tanításból számoljuk. A registryben az új jelölt `candidate_v4` lesz; a korábbi `candidate_v3` megmarad, és ha a review sikeres, a champion mutatót explicit döntéssel frissítik. Az adatból származó statisztikai állítás sem automatikusan érvényes az új cutoffra: a 20.4. alfejezet időbeli validációt az új ablakra is el kell végezni.

32.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az, hogy a tanuló a Dockerfile-t, a CI-konfigot és a predikciós szkriptet feltölti, de nem futtatja, és a dokumentációban „konténerben ellenőrzött” átadást állít. A fájlok megléte nem bizonyíték. Tipikus ellenpélda: a Dockerfile `FROM ghcr.io/astral-sh/uv:python3.12-bookworm-slim`-mel indul, de a fejlesztő soha nem futtatott `docker build`-et; az első buildnél kiderül, hogy a `uv.lock` nincs a contextben, vagy a base image tag elavult. A helyes gyakorlat: minden új környezetet futtatással igazolunk, és a naplóba bekerül a tényleges parancs, a kimenet és a hibakód. Ebben a leckében a könyvgenerátor nem futtatta a Docker-példát; a fenti kimenet ennek megfelelően „szemléltető”, nem mért eredmény. A tanuló csak a saját, lokálisan futtatott logját nevezheti ellenőrzöttnek. Ez a megkülönböztetés a 32.3. alfejezetből ismert: a nem futtatott példa nem válik igazolt állítássá attól, hogy formailag teljes.

Önellenőrzés: a configban minden bemenethez van hash? A `split.json` a cutoffot és a teszt-sorazonosítókat rögzíti? A `predict.py` hibás eredménynél nem ír futási naplót a champion állapotba? A modellfájl a saját `fit_model.py` kimenete, és a hash-e szerepel a manifestben? Ha bármelyikre nem a válasz, az átadás nem teljes.

32.4.8 Rövid összefoglaló

A VIII. ellenőrző pont a modellprojekt átadhatóságát méri: csomag, tesztek, explicit konfiguráció, adat- és modellhash, futáskövetés, konténer és hiba utáni visszaállítás együtt biztosítják, hogy egy másik környezet ugyanazt a kontrollt állítsa elő. A predikciós szkript a hasheket, a teszt sorainak számát és a várt AUC-t ellenőrzi; a tanítás a fejlesztőgépen marad. A sikeres konténerfutás után a korábbi fejezetben tanított modellnyilvántartásba vezethető a jelölt, de a nem futtatott Docker- vagy CI-példa sosem nevezhető ellenőrzöttnek. A következő alkalmazásban a 33.1. alfejezet ezt a szállítási fegyelmet profilozással egészíti ki, hogy a futás erőforrásigénye is mérhető legyen.

33 Profilozás és egygépes párhuzamos futtatás

33.1 Szűk keresztmetszet és mérési terv

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: cProfile, threadpoolctl, joblib

Előfeltétel: 32.4. alfejezet · Célkészség: Szűk keresztmetszet és mérési terv · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

33.1.1 Cél és konkrét helyzet

A II. ellenőrző pont 8.4. alfejezet után rendelkezésünkre áll egy rendelésszintű, időhelyes `features.parquet`. A modellépítés előtt a feature-pipeline-t szeretnénk gyorsítani, de nem találgatni akarunk, hanem méréssel megmutatni, hol érdemes beavatkozni. A bemenet egy Parquet-fájl, a megfigyelési egység a rendelés, a kimenet egy determinisztikus, azonos sémájú feature-tábla. Az előzetes becslés: kis méreten a fájlolvasás, nagy méreten a numerikus transzformáció dominálhat. Az elfogadási feltétel: a három skálán megismételt mérésből nevesíteni tudjuk az elsődleges szűk keresztmetszetet, és a transzformált tábla kontrolljai változatlanok maradnak. A kódot itt nem futtattuk; minden megadott időtartam **szemléltető**, a tanuló saját gépi jegyzőkönyve az érvényes bizonyíték.

33.1.2 Wall time, CPU time és a hiányzó idő

A falon mért idő (*wall time*) az az óra, amit a felhasználó is lát. A processz ideje (*CPU time*) két részből áll: a felhasználói utasítások idejéből T_{user} és a kernelbeli, rendszerhívásokra fordított időből T_{sys}. Mértékegység mindkettőnél másodperc:

$$T_{\text{wall}} \approx T_{\text{wait}} + T_{\text{user}} + T_{\text{sys}},$$

ahol T_{wait} a várakozás, például lemezolvasás, hálózat vagy más processzre várakozás. Pythonban a `time.perf_counter()` a falidőhöz, a `time.process_time()` a processz CPU-idejéhez való.

```
import time

t_wall = time.perf_counter()
t_cpu = time.process_time()
result = pipeline(path)
wall_s = time.perf_counter() - t_wall
cpu_s = time.process_time() - t_cpu
print(f"wall={wall_s:.3f}s cpu={cpu_s:.3f}s wait={wall_s-cpu_s:.3f}s")
```

Ha `wall` ≫ `cpu`, a folyamat számolás- vagy memóriakorlátos. Ha `wall` ≫ `cpu`, akkor I/O-ra vagy alvásra vár, és a CPU-gyorsítás nem segít. Fordított jel a párhuzamos könyvtáraknál jelenik meg: ha a BLAS több magot használ, akkor `cpu` > `wall` lehet, például 1,0 s falidő mellett 3,6 s felhasználói CPU-idő. Kis kézi kontroll: egy képzeletbeli futás `wall=0,85 s`, `cpu=0,35 s`; a hiányzó `0,50 s` nem számítás, hanem várakozás.

33.1.3 Profil és allokáció

A `cProfile` determinisztikus profilozó: minden függvényhívást megszámol, és két fontos időt ad. A `tottime` a függvény saját testében töltött idő, a `cumtime` a hívott függvényekkel együtt számított idő. Ha egy sor `tottime`-ja alacsony, de `cumtime`-ja magas, akkor az idő a hívási mélységben tűnik el.

A memóriaoldalt a standard könyvtár `tracemalloc` modulja méri: a `get_traced_memory()` visszaadja a pillanatnyi és a csúcskiosztást bájtban. A `memory_profiler` csomag soronkénti felbontást ad, de jelentős lassítással, ezért a lab alaptelepítéséhez új függőségként csak akkor vegyük fel, ha a futtatott profil ezt tényleg szükségessé teszi. Egy `float64` oszlop mérete $n \cdot 8$ bájt; ha a pipeline egy átmeneti DataFrame-et is készít, a csúcs ennek körülbelül a duplája.

33.1.4 Mintanagyság és a hányados bizonytalansága

A mérési ismétlés itt a `repeat`, nem az adatsorok száma. Legyen T_i az i-edik megismételt futás ideje, $i = 1, \dots, r$. A használt összegzők:

$$\bar{T} = \frac{1}{r}\sum_{i=1}^{r} T_i, \qquad s = \sqrt{\frac{1}{r-1}\sum_{i=1}^{r}(T_i - \bar{T})^2}, \qquad \mathrm{CV} = \frac{s}{\bar{T}}.$$

A medián és az interkvartilis terjedelem (IQR, harmadik kvartilis mínusz első kvartilis) egy kilógó, hideg rendszerbeli futásra kevésbé érzékeny, mint az átlag. Ha két változatot hasonlítunk össze, a sebességhányados $R = \bar{T}_A / \bar{T}_B$. A log-hányados közelítő relatív szórása első rendben:

$$\mathrm{SE}_{\log R} \approx \sqrt{\frac{\mathrm{CV}_A^2}{r_A} + \frac{\mathrm{CV}_B^2}{r_B}}.$$

Ez nem konfidenciaintervallum, csak a mérés zajának nagyságrendjét mutatja. Számpélda: ha $\mathrm{CV}_A = 0{,}04$, $\mathrm{CV}_B = 0{,}06$ és $r_A = r_B = 7$, akkor a fenti hiba $\sqrt{0{,}04^2/7 + 0{,}06^2/7} \approx 0{,}027$. Egy $R = 1{,}5$ hányados körülbelül $\pm 8\%$ relatív ingadozással értelmezhető, ami nagyobb, mint amit egyetlen futásból látnánk.

33.1.5 Baseline

A baseline a jelenlegi, helyes implementáció rögzített változata. Nem trükkösen lassú kód, hanem az az állapot, amelyhez a gyorsulást relatívan mérjük. A jegyzőkönyvhöz hozzátartozik a Python-verzió, a NumPy/pandas verzió, a gép, az operációs rendszer, és ha konténerben futunk, az image-digest is 32.1. alfejezet nyomán. A `features.parquet` SHA-256 lenyomata az inputot azonosítja, ahogy a cache kulcs is azt használja 30.3. alfejezet. Ha a transzformáció módosul, az invariantok (shape, dtype, kontrollösszeg) biztosítják, hogy nem mértünk gyorsabb, de eltérő eredményt.

33.1.6 Teljes minimális példa

A példa a tiszta függvényes határválasztást követi 29.1. alfejezet: a `features.py`-ból ismert transzformációt futtatjuk, most kizárólag a numerikus oszlopokra, hogy a profil olvasható maradjon.

```
# src/northstar/profile_features.py -- futtatható vázlat
from __future__ import annotations

import io
import cProfile
import pstats
import statistics
import timeit
import tracemalloc
from pathlib import Path

import numpy as np
import pandas as pd

NUMERIC_FEATURES = [
    "distance_km",
    "priority_flag",
    "basket_minor",
    "quantity_sum",
    "account_age_days",
]


def load_features(path: str | Path) -> pd.DataFrame:
    df = pd.read_parquet(path)
    return df.loc[:, NUMERIC_FEATURES].copy()


def transform_features(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["log_basket"] = np.log1p(out["basket_minor"].to_numpy())
    for col in ["distance_km", "log_basket", "account_age_days"]:
        arr = out[col].to_numpy(dtype=np.float64)
        mu = arr.mean()
        sd = arr.std(ddof=0)
        out[f"{col}_z"] = (arr - mu) / sd if sd > 0 else 0.0
    return out


def pipeline(path: str | Path) -> pd.DataFrame:
    return transform_features(load_features(path))


def timed_pipeline(path: str | Path, repeat: int = 7, number: int = 3) -> list[float]:
    pipeline(path)  # bemelegítés és operációs cache feltöltése
    raw = timeit.repeat(lambda: pipeline(path), number=number, repeat=repeat)
    return [t / number for t in raw]  # egy hívásra eső átlagos idő


def allocation_stats(path: str | Path) -> tuple[pd.DataFrame, int, int]:
    tracemalloc.start()
    out = pipeline(path)
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return out, peak
```

```
def profile_summary(path: str | Path, repeat: int = 7, number: int = 3) -> dict:
    times = timed_pipeline(path, repeat, number)
    out, peak_bytes = allocation_stats(path)
    pr = cProfile.Profile()
    pr.enable()
    pipeline(path)
    pr.disable()
    stream = io.StringIO()
    pstats.Stats(pr, stream=stream).sort_stats("tottime").print_stats(8)
    return {
        "median_wall_s": statistics.median(times),
        "iqr_wall_s": statistics.quantiles(times, n=4)[2]
        - statistics.quantiles(times, n=4)[0],
        "times_s": times,
        "peak_bytes": peak_bytes,
        "profile": stream.getvalue(),
        "shape": out.shape,
    }
```

A három skálához az alapsorokat ismétléssel nagyítjuk, csak a mérési bemenethez: `pd.concat([base] * k, ignore_index=True).head(target_n)`; ezt a `features.parquet` mellé írt ideiglenes fájlokra futtatjuk. A parancs Linux/macOS alatt, `lab/` munkakönyvtárból:

```
# platform: Linux/macOS; munkakönyvtár: lab/
python -m northstar.profile_features --input artifacts/features.parquet \
  --sizes 2400 9600 24000 --repeat 7
```

Szemléltető, nem e könyv gépi mérése:

n (rendelés)	wall medián	CPU user (T_{user})	peak allokált
2 400	0,44 s	0,19 s	14 MB
9 600	0,61 s	0,52 s	56 MB
24 000	1,10 s	1,00 s	140 MB

A hozzá tartozó, szintén szemléltető `cProfile`-kivágat `n=2400` esetén a `read_parquet`/PyArrow-sort helyezné az első helyre, `n=24000` esetén pedig a `np.log1p` mögötti NumPy-ufunc és a `DataFrame.copy` allokációja kerülne felülre. Ezt a valódi mérési jegyzőkönyvben a gépi `profile` mező rögzíti.

33.1.7 Eredmény értelmezése

Az invariáns ellenőrzéshez nem kell a futás vége: `basket_minor=0` esetén $\log(1+0)=0$, így a `log_basket` minimuma nemnegatív. Egy kézi kétpontos etalonnal is ellenőrizhető: ha egy `distance_km` oszlop értékei `[0, 2]`, akkor $\mu=1$, $s=1$, a standardizált értékek `[-1, 1]`, összegük nulla, szórásuk egy. Az output `shape=(n, 9)`, mert az öt bemeneti oszlophoz egy log- és három standardizált oszlop jön.

A szemléltető táblázatból a döntés: `n=2400`-nál a falidő és a CPU-idő különbsége nagy, tehát a lemezolvasás a gyanúsított; `n=24000`-nél `wall - cpu` kicsi, tehát a CPU-oldali transzformáció már dominál. A peak allokáció a sorokkal közel lineárisan nő, ami a `copy()` és a köztes tömbök szerepére utal. Egy ilyen bizonyíték után lehet célzottan cache-elni az I/O-t, vagy a transzformáció másolási rétegét csökkenteni; nem mindent egyszerre.

Teljesítménymérésnél a bemelegítés és az ismétlések variabilitásának közlése nélkül az egyetlen futási idő nem megbízható összehasonlítás.

33.1.7.1 Hideg és meleg cache, medián és p90

A mérési jegyzőkönyvnek legalább ezeket kell tartalmaznia:

Mező	Példa	Miért kell
`benchmark_id`	`"features-parquet-20261029"`	a mérés azonosíthatósága
`hardware`	CPU-típus, fizikai/logikai magok, RAM, háttértár	a számok géptől függenek
`software`	Python, NumPy, pandas, PyArrow verziók	a könyvtárverzió befolyásolja
`input`	sorok száma, bájtméret, SHA-256	ugyanazt a bemenetet mértük-e
`cache_state`	`cold` vagy `warm`	a két állapot nem összehasonlítható
`warmup`	1–3 futás	az import, JIT és allokátor bemelegítése
`repeat`	legalább 5, javasolt 7	a futások közti variancia mérése
`times_s`	a nyers ismétlések	utólag újraszámolható
`median_s`, `p90_s`, `cv`	medián, 90. percentilis, variációs együttható	robusztus központ és farok

A **hideg cache** mérés a fájl első olvasását méri, például frissen csatolt hálózati meghajtóról vagy `posix_fadvise(..., POSIX_FADV_DONTNEED)` után; a **meleg cache** mérés a már betöltött fájlt. A kettőt külön kell jelenteni, és a hideg mérést nem szabad a meleg mediánba keverni. A `drop_caches` rendszergazdai jogot igényel, ezért a jegyzőkönyvben a módszert is rögzíteni kell.

```
import math
import statistics

def summarize(times):
    ordered = sorted(times)
    n = len(ordered)
    mean = statistics.fmean(ordered)
    return {
        "n": n,
        "median_s": statistics.median(ordered),
        "p90_s": ordered[max(0, math.ceil(0.9 * n) - 1)],
        "min_s": ordered[0],
        "max_s": ordered[-1],
        "cv": statistics.pstdev(ordered) / mean if mean else float("inf"),
    }
```

A p90 itt legközelebbi rangot használ (`ceil(0.9n)-1`), nem interpolál; ezt a definíciót a jegyzőkönyvben rögzíteni kell, különben két eszköz eltérő p90-et ad. Az elfogadási kontroll: ugyanaz a megoldás más gépen is **mértékegységgel és hardverrel** együtt jelenik meg, nem fix másodperces ígérettel. A `tests/test_benchmark_report.py` a medián/p90 számítást, a hideg/meleg rekord kötelező mezőit és a hiányos hardvermetaadat elutasítását teszteli.

33.1.8 Feladatok

1. **Értelmezés.** Adott két változat hétszeri mérése: A falidői `[0,48; 0,47; 0,51; 0,49; 0,47; 0,50; 0,46]` s, CPU-user mind `0,18` s; B falidői `[0,61; 0,41; 0,43; 0,42;`

`0,44; 0,41; 0,40]` s, CPU-user mind `0,38` s. Számold ki az A és B falidő-mediánját, IQR-ját, és mondd meg, melyik hányadosra állítható stabilabb következtetés.

2. **Implementálás.** Írj `measure_repeated(fn, *, warmup, number, repeat)` függvényt, amely `warmup` alkalmat bemelegítésként futtat, majd `timeit.repeat`-tel mér, végül a becsült per-hívás időkből median/IQR/CV hármast ad. Miért a mediánt jelented, nem a minimumot?

3. **Hibadiagnózis.** Egy kolléga `timeit.timeit("transform_features(df)", globals=..., number=1)` hívással méri a transzformációt az éles adaton, és közli, hogy a kód harmadával lassult az előző napihoz képest. Legalább három olyan okot nevezz meg, ami miatt ez nem értelmezhető változatlanságként.

4. **Megváltozott feltétel.** A `features.parquet` hálózati meghajtóra kerül. Egy futás `wall=9,2 s`, `cpu=1,1 s`; helyi másolatra `wall=1,6 s`, `cpu=1,1 s`. Ezután a pipeline-ba bekerül a `segment`, `region`, `acquisition_channel` one-hot kódolása is, és `n=24000` mellett `wall=2,9 s`, `cpu=2,5 s`, peak 380 MB. Melyik lépés milyen jellegű szűk keresztmetszet, és mi a következő mérési kérdés a párhuzamosítás előtt?

33.1.9 Megoldások és indoklás

1. A rendezett 7, 0,42; az IQR $0{,}44 - 0{,}41 = 0{,}03$. B rendezett 7, 0,42; IQR $0{,}43 - 0{,}41 = 0{,}03$. B rendezett 7, 0,42; az IQR $0{,}44 - 0{,}41 = 0{,}03$. B rendezett sor `[0,40; 0,41; 0,41; 0,42; 0,43; 0,44; 0,61]`, medián `0,42`, IQR $0{,}44 - 0{,}41 = 0{,}03$. Az első B mérés `0,61` kiugró volt, de a medián és az IQR felülírta. A CPU-oldalon A átlagosan várakozik, tehát A I/O-közeli, B viszont `cpu > wall`, tehát B a számítási oldalon terheltebb. A B falidő-mediánja megbízhatóbb, mert a CPU-idő szerint a mérés alatt komolyabb, kétséges várakozás nincs; A-nál a hiányzó idő OS-allapotfüggő marad.

2. Egy lehetséges megoldás:

```
def measure_repeated(fn, *, warmup=3, number=5, repeat=9):
    for _ in range(warmup):
        fn()
    times = [
        t / number
        for t in timeit.repeat(fn, number=number, repeat=repeat)
    ]
    sorted_times = sorted(times)
    q1, q3 = statistics.quantiles(sorted_times, n=4)[0], statistics.quantiles(
        sorted_times, n=4
    )[2]
    mean = statistics.fmean(times)
    sd = statistics.pstdev(times)
    cv = sd / mean if mean > 0 else float("inf")
    return {"median": statistics.median(times), "iqr": q3 - q1, "cv": cv}
```

A minimum bármelyik OS-szálütemezési szerencse lehet; a medián robustabb. A `number` és a `repeat` külön szerepet játszik: az előbbi a híváson belüli zajt átlagolja, az utóbbi a futások közötti varianciát méri. Az elfogadási küszöb: IQR kisebb legyen a medián 10%-ánál; ha nem, növeljük a `repeat` értékét.

3. Lehetséges okok: nincs bemelegítés, ezért az első hívás importál, a JIT-kód még nem meleg, vagy hideg az I/O-gyorsítótár; a `number=1` miatt a rendszer CPU-frekvencia-skálázása egyetlen futásban benne marad; nincs rögzített baseline-verzió és bemenethash, így nem tudjuk, ugyanazt a fájlt és kódot mérte-e; terhelt kollégagépen más processzek vették el a CPU-t. Az ilyen mérésből legfeljebb egy hibajel lehet, nem teljesítményváltozás.

4. A hálózati futásnál `wall - cpu = 8,1 s`, tehát I/O-korlát; a helyi másolatnál ugyanez `0,5 s`, tehát a fájl elhelyezése volt a probléma. Az one-hot után `wall=2,9 s` és `cpu=2,5 s`: a CPU-oldal vált dominánssá, de a `cpu/wall` arány kisebb egynél, tehát a szűk keresztmetszet egy magos számítás és allokáció. A 380 MB csúcs miatt a 380 MB-os köztes adat másolása valószínű. A következő mérési kérdés: a `pd.get_dummies` helyett `astype("category")` + `OneHotEncoder` vagy ritka mátrix hány másodperc és hány MB? Ezt a frissített baseline ellen mért sebességhányadossal kell eldönteni.

33.1.10 Gyakori tévedés és ellenpélda

Egyetlen időmérésből nem indokolt általános gyorsulási sorrendet állítani. Ellenpélda: A első futása `0,60 s`, B első futása `0,52 s`; ha itt megállunk, azt mondanánk, hogy B gyorsabb. A hétszeri mérésben azonban A `[0,60; 0,41; 0,43; 0,42; 0,44; 0,41; 0,40]`, B `[0,52; 0,54; 0,53; 0,55; 0,52; 0,54; 0,53]`, a mediánok `0,42 s` és `0,53 s`: a megismételt mérés szerint A a gyorsabb, és az első A-futás csak hidegindulás volt. Ugyanez az oka annak, hogy az 5.4. alfejezetben is `repeat` és bemelegítés szerepelt. Önellenőrzés: mielőtt gyorsulást jelentünk, kérdezzük meg, hány ismétlésből jött a szám; ha egy, a jegyzőkönyv csak egy pontot bizonyít.

33.1.11 Rövid összefoglaló

A szűk keresztmetszet mérése három réteget választ szét: a falidő és a CPU-idő különbsége az I/O-várakozást jelzi, a `cProfile` `tottime`/`cumtime` profilja a forró függvényeket nevesíti, a `tracemalloc` pedig a csúcskiosztást méri. Egyetlen futás nem bizonyíték; `timeit.repeat`, bemelegítés, majd medián/IQR/CV szükséges, és a sebességhányados relatív hibája a CV-kből becsülhető. A mérés előtt rögzített baseline, input-hash és invariant biztosítja, hogy ugyanazt az eredményt gyorsítottuk. A következő alkalmazás 33.2. alfejezetben ezt használja fel: a `cpu/wall` hányadosból döntjük el, hogy thread, process vagy könyvtári szálak adhatnak-e tényleges nyereséget.

33.2 Thread, process és könyvtári szálak

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: cProfile, threadpoolctl, joblib

Előfeltétel: 33.1. alfejezet · Célkészség: Thread, process és könyvtári szálak · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

33.2.1 Cél és konkrét helyzet

Ebben az alfejezetben egy kutatói gépen szeretnénk felgyorsítani két, eltérő jellegű munkafázist: több száz kis CSV-fájl kiírását és visszaolvasását (I/O-kötött feladat), valamint 12 darab 500×500-as valós mátrix szorzását (CPU-kötött NumPy-feladat). A megfigyelési egység egy feladatfutás, a mért eredmény a falidő mediánja öt ismétlésből. A cél annak eldöntése, hogy `ThreadPoolExecutor` vagy `ProcessPoolExecutor` illik-e az adott munkaterheléshez, és hogy a BLAS-könyvtár saját szálait hogyan kell kézben tartani. Előzetes becslés: az I/O feladatnál a szálak használnak, mert a Python a fájlműveletek alatt várakozik; a tiszta NumPy szorzásnál a natív kód felszabadítja a GIL-t, így a szálak is gyorsíthatnak, de csak akkor, ha a BLAS szálainak száma 1-re van korlátozva. A konkrét labkimenetet itt nem futtattuk, ezért a számokat „szemléltető” becslésként adjuk meg; a valódi méréshez a kódot a saját gépen kell lefuttatni.

33.2.2 Thread, process, GIL és natív kód

A CPython implementációban egyszerre csak egy szál hajthat végre Python-bájtkódot: ez a Global Interpreter Lock (GIL). A GIL nem általános Python-tulajdonság, hanem a CPython build sajátossága; a 3.13-tól létezik kísérleti free-threaded build, a PyPy és más implementációk pedig másként viselkednek. A tanulási főút CPython 3.12-t használ, ezért itt GIL-es modellt feltételezünk.

A GIL hatása két fázisban enyhül:

- **I/O-várakozás:** amikor a szál fájlműveletre, hálózatra vagy alvásra vár, a CPython elengedi a GIL-t, így egy másik szál futhat. Ezért a sok kis fájlt író-olvasó feladatra a `threading` vagy a `ThreadPoolExecutor` természetes választás.
- **Natív kód:** a NumPy nagy tömbműveletei C-ben futnak, és a CPython a hosszú számítások alatt szintén elengedi a GIL-t. Így egy `A @ B` mátrixszorzás közben más Python-szálak is dolgozhatnak. A Python-szintű ciklus viszont továbbra is a GIL alatt fut, tehát a tiszta Python számításokat a szálak nem párhuzamosítják.

A **process** ezzel szemben külön interpretert indít minden workerhez, saját memóriával és saját GIL-lel. A `ProcessPoolExecutor` vagy a `multiprocessing` ezért CPU-kötött Python-kódhoz is valódi párhuzamosságot ad. Az ára a **serialization**: a munkafüggvényt, az argumentumokat és a visszatérési értéket a `pickle` segítségével sorosítani kell a folyamatok között. Kis egész számoknál ez elhanyagolható; egy több száz megabájtos pandas DataFrame esetén viszont maga a másolás lehet a szűk keresztmetszet.

A NumPy, a SciPy és a scikit-learn nagy lineáris algebrai műveletei gyakran egy **BLAS**-könyvtárat használnak (OpenBLAS, MKL, Accelerate). A BLAS a saját, natív szintű szálait is elindíthatja; a szálak számát az `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS`, `OMP_NUM_THREADS` környezeti változók szabályozzák, és ezeket az értékeket a NumPy importja előtt kell beállítani. Ha például 4 Python-worker mindegyike 8 BLAS-szálat indít egy 8 logikai magos gépen, akkor 32 szál verseng ugyanazért a CPU-ért: ez az **oversubscription**. A kontextusváltás és a gyorsítótár-versengés ilyenkor gyakran lassabb futást ad, mint a szekvenciális változat.

A `joblib` a lokális párhuzamosítás kényelmes rétege: `joblib.Parallel(n_jobs=4, prefer="threads")` vagy `prefer="processes"` mögött ugyanez a mechanizmus áll, és a scikit-learn `n_jobs` paraméterei is erre épülnek. A `cProfile` CPU-időt mér, de szálas programnál a profilozó overhead torzíthat; a párhuzamosítási döntéshez falidőt és CPU-időt együtt érdemes mérni. A `memory_profiler` a processz memóriaforgalmát figyeli, ami a process-alapú megoldásnál fontos, mert minden worker külön memóriát foglal.

33.2.3 Adatmozgatás és műveletszám becslése

Egy $n \times n$-es, float64 típusú mátrix szorzásánál a műveletszám közelítőleg

$$M_{\text{comp}} \approx 2n^3 \text{ flop},$$

mert minden kimeneti elemhez n szorzás és n összeadás tartozik. A bemenő A, B és a kimenő C mátrix egyenként $8n^2$ bájtot foglal, így a minimális tárolási méret (nem a tényleges RAM-forgalom, amely cache- és implementációfüggő)

$$M_{\text{data,min}} \approx 3 \cdot 8n^2 \text{ bájt}.$$

Ha a ténylegesen elérhető lebegőpontos teljesítmény R_{flop} (GFLOP/s), és a RAM-sávszélesség B_{RAM} (GB/s), akkor a két idő hányadosa

$$\frac{t_{\text{comp}}}{t_{\text{mem}}} \approx \frac{2n^3/R_{\text{flop}}}{24n^2/B_{\text{RAM}}} = \frac{nB_{\text{RAM}}}{12R_{\text{flop}}}.$$

Szemléltető értékekkel: $n = 500$, $R_{\text{flop}} = 2$ GFLOP/s, $B_{\text{RAM}} = 20$ GB/s esetén $t_{\text{comp}} \approx 250\,000\,000/2 \cdot 10^9 = 0{,}125$ s feladatonként, míg $t_{\text{mem}} \approx 24 \cdot 250\,000/20 \cdot 10^9 \approx 0{,}3$ ms, tehát a hányados körülbelül 400. A feladat erősen CPU-kötött, ezért a szálak vagy processzek közötti adatmegosztás kevésbé számít, a CPU-versengés viszont annál inkább.

Az I/O-feladatnál a becslés más: 12 worker $\times 25$ fájl $= 300$ kis fájl, és $12 \times 25 \times 2000 = 600\,000$ sor. A fájlonkénti nyitási/zárási késleltetés és a soronkénti írási költség dominál; a szálak akkor használnak, ha a Python a fájlműveletek alatt más workernek adja át a vezérlést.

33.2.4 Teljes minimális munkapélda

A következő script a `concurrent.futures` szabványos modullal hasonlítja össze a szekvenciális, szálas és processzes futást. A platform itt Linux, CPython 3.12, 4 fizikai mag; más platformon a `ProcessPoolExecutor` spawn viselkedése eltérhet. A scriptet a `s130_workload.py` fájlba mentjük.

```
import os

# A BLAS szálkeretét a NumPy importja előtt rögzítjük,
# különben a belső párhuzamosság oversubscriptiont okoz.
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"

import platform
import statistics
import tempfile
import time
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

import numpy as np

TASK_COUNT = 12
WORKERS = 4
REPEATS = 5


def io_task(worker_id: int) -> int:
    """Egy worker 25 kis CSV-fájlt ír és olvas vissza."""
    tmpdir = tempfile.gettempdir()
    seen = []
    for i in range(25):
        path = os.path.join(tmpdir, f"s130_{worker_id}_{i}.csv")
        with open(path, "w", encoding="utf-8") as f:
            for j in range(2000):
                f.write(f"{j},{worker_id},{j % 7}\n")
        with open(path, "r", encoding="utf-8") as f:
            n = sum(1 for _ in f)
        assert n == 2000
        seen.append(n)
        os.remove(path)
    return sum(seen)
```

```
def np_task(seed: int) -> float:
    """Egy 500x500-as mátrixpár szorzása determinisztikus seeddel."""
    rng = np.random.default_rng(seed)
    A = rng.normal(size=(500, 500))
    B = rng.normal(size=(500, 500))
    C = A @ B
    return float(C[0, 0])


def run_sequential(fn, n):
    t0 = time.perf_counter()
    results = [fn(i) for i in range(n)]
    return time.perf_counter() - t0, results


def run_threads(fn, n, workers):
    t0 = time.perf_counter()
    with ThreadPoolExecutor(max_workers=workers) as ex:
        results = list(ex.map(fn, range(n)))
    return time.perf_counter() - t0, results


def run_processes(fn, n, workers):
    t0 = time.perf_counter()
    with ProcessPoolExecutor(max_workers=workers) as ex:
        results = list(ex.map(fn, range(n)))
    return time.perf_counter() - t0, results


def measure(runner, fn, n, workers, repeats):
    times = []
    results = None
    for _ in range(repeats):
        elapsed, results = runner(fn, n, workers)
        times.append(elapsed)
    return statistics.median(times), results


def main():
    print("platform:", platform.platform())
    print("logical CPUs:", os.cpu_count())
    print("task count:", TASK_COUNT)
    print("workers:", WORKERS)
    print("repeats:", REPEATS)

    for label, fn in [("io", io_task), ("np", np_task)]:
        seq_t, seq_res = measure(run_sequential, fn, TASK_COUNT, 1, REPEATS)
        thr_t, thr_res = measure(run_threads, fn, TASK_COUNT, WORKERS, REPEATS)
        prc_t, prc_res = measure(run_processes, fn, TASK_COUNT, WORKERS, REPEATS)
        print(f"{label}: seq={seq_t:.3f}s thread={thr_t:.3f}s "
              f"process={prc_t:.3f}s")
        if label == "io":
            print("  io invariant sum:", sum(seq_res), sum(thr_res), sum(prc_res))
        else:
            print("  np first element:", seq_res[0], thr_res[0], prc_res[0])


if __name__ == "__main__":
    main()
```

A futtatási parancs Linux terminálból, a projektgyökérből:

```
python s130_workload.py
```

Szemléltető, nem mért kimenet 4 fizikai magos gépre, a fenti méretekkel:

```
platform: Linux-6.8.0-45-generic-x86_64-with-glibc2.39
logical CPUs: 8
task count: 12
workers: 4
repeats: 5
io: seq=2.100s thread=0.700s process=1.900s
  io invariant sum: 600000 600000 600000
np: seq=1.500s thread=0.500s process=0.600s
  np first element: -17.4213 -17.4213 -17.4213
```

Ezek a számok a szakasz eleji becslésből származó várt nagyságrendek, nem helyi futtatás eredményei.

33.2.5 Eredmény értelmezése

Az I/O feladatnál a szálas változat a szekvenciális idő nagyjából harmadára csökkenhet, mert a fájlműveletek alatt a GIL felszabadul. A processzes változat keveset nyer, mert a pickle-sorosítás és a processzindítás költsége hozzáadódik. A NumPy feladatnál a szálak is gyorsítanak, mert a natív mátrixszorzás közben a GIL elengedett; a processzes változat hasonló időt ad, de a processzenkénti NumPy-import és a sorosítás miatt általában valamivel lassabb. Az invariánsok: az I/O összeg minden módnál 600 000 sor, a NumPy első kimeneti eleme seedenként ugyanaz, de a lebegőpontos összegzés sorrendje miatt a BLAS-szálak számától függően kis eltérés előfordulhat, ezért bitazonosságot nem követelünk, csak relatív toleranciát, például `np.allclose` alapján.

33.2.5.1 Szálkeret ellenőrzése és a két munkaterhelés külön mérése

A `threadpoolctl` megmutatja, mely natív könyvtárak hány szálat indítanak:

```
from threadpoolctl import threadpool_info, threadpool_limits

print(threadpool_info())
# példa egy elemre: {"user_api": "blas", "internal_api": "openblas",
#                    "num_threads": 8, "prefix": "libopenblas", ...}

with threadpool_limits(limits=1):
    ...  # ebben a blokkban a BLAS legfeljebb 1 szálat használ
```

A környezeti változókat a NumPy importja **előtt** kell beállítani; utólag már nem hatnak minden könyvtárra:

```
import os
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
```

A szálkeret egyszerű ellenőrzése: `n_jobs × blas_threads` ne haladja meg a fizikai magok számát. Ha 4 worker egyenként 8 BLAS-szálat indít egy 8 magos gépen, a szorzat 32, tehát oversubscription van; a helyes beállítás `n_jobs=4, blas_threads=1` (4) vagy `n_jobs=1, blas_threads=8` (8), de nem a kettő szorzata.

A két munkaterhelést külön rekordban kell mérni:

Mező	I/O-kötött	CPU-kötött
`workload`	`"io"`	`"cpu"`
`n_jobs`	4	4
`blas_threads`	1	1
`wall_s`	a várakozás miatt nagyobb	közel a CPU-időhöz
`cpu_s`	kicsi	nagy
`wall_cpu_ratio`	> 1	≈ 1
`result_hash`	az összeg hash-e	az első kimeneti elem + tolerancia

Az I/O-kötött feladatnál a `wall/cpu` arány azt mutatja, mennyit várt a processz; a CPU-kötöttnél az arány közel 1, és a gyorsulást a falidő mediánja adja. A két rekordot nem szabad összevonni, mert ugyanaz az `n_jobs` más hatással van rájuk. A `tests/test_thread_limits.py` a szálkeret-számítást, a `threadpool_info` limitellenőrzését és a két munkaterhelés külön rekordját teszteli.

33.2.6 Feladatok

1. **Értelmezés.** Miért lehetséges, hogy a `np_task` szálas változata gyorsabb a szekvenciálisnál annak ellenére, hogy a GIL a Python-bájtkódot sorossá teszi? Nevezd meg a mechanizmust, és mondd meg, mi történne, ha a mátrixszorzást tiszta Python `for` ciklussal írnánk meg.

2. **Megvalósítás.** Módosítsd a scriptet úgy, hogy a `WORKERS` értékét 2, 4 és 8 között is mérje, és minden futásról készítsen egy `manifest.json`-t a 29.2. alfejezet mintájára: időbélyeg, platform, CPU-k száma, feladatszám, worker-szám, ismétlésszám és a mért mediánok. A kimeneti fájl ne legyen felülírható.

3. **Hibadiagnózis.** Egy kolléga elhagyja a script elején az `OPENBLAS_NUM_THREADS=1` beállítást, és azt tapasztalja, hogy a `np_task` szálas változata lassabb, mint a szekvenciális. Fogalmazd meg, mit ellenőriznél először, és mi a valószínű magyarázat.

4. **Megváltozott feltétel.** A feladat mostantól nem 12 kis mátrixot, hanem 4 darab 200 MB-os pandas DataFrame-et kap, amelyeket a processzeknek kell feldolgozniuk. Miért nőhet meg drasztikusan a falidő, és milyen adatátadási stratégia csökkentheti a sorosítási költséget?

5. **Lebegőpontos transzfer.** Ugyanaz a seed és ugyanaz a gép mellett a `np_task` `C[0, 0]` értéke a szekvenciális és a szálas futás között a tizedesjegyekben eltérhet. Miért nem sérti ez a numerikus reprodukálhatóságot, és milyen ellenőrzést használj a `==` helyett?

33.2.7 Megoldások és indoklás

1. A NumPy `@` művelete natív C kódot hív, és a CPython a hosszú C-számítás alatt elengedi a GIL-t, így több szál párhuzamosan végezhet mátrixszorzást. Ha a szorzást Python `for` ciklusokkal írnánk, a ciklus bájtkódja a GIL alatt futna, és a szálas változat nem gyorsítana, sőt a szálkezelés overheadje miatt még lassulhatna is. A GIL tehát a Python-szintű számítást korlátozza, nem a natív kiterjesztéseket.

2. A módosítás pszeudokódja:

```
import json
import datetime as dt
from pathlib import Path

def save_manifest(run_id, records):
    out = Path("artifacts") / run_id
    out.mkdir(parents=True, exist_ok=False)  # immutable output
    manifest = {
        "created_at": dt.datetime.now(dt.timezone.utc).isoformat(),
        "platform": platform.platform(),
        "logical_cpus": os.cpu_count(),
        "task_count": TASK_COUNT,
        "repeats": REPEATS,
        "results": records,
    }
    (out / "manifest.json").write_text(json.dumps(manifest, indent=2))
```

A `measure` hívását minden worker-számra el kell végezni, a rekordokat listába gyűjteni, majd a lista alapján manifestet írni. Az `exist_ok=False` garantálja, hogy véletlenül sem írunk felül korábbi futást. A manifest tehát a futás környezeti adatait és a mért mediánokat is rögzíti, ahogy a 29.2. alfejezet konfigurációs és futási napló elve megköveteli.

3. Először azt ellenőrizném, hogy a BLAS ténylegesen hány szálat indít: `threadpoolctl.threadpool_info()` vagy az operációs rendszer folyamatnézetében a szálak száma. Ha a környezeti változó hiányzik, a BLAS alapértelmezés szerint akár az összes logikai magot használhatja. Ekkor a 4 Python-worker mindegyike 8 BLAS-szálat indít, összesen 32 szálat: ez oversubscription, a kontextusváltás és a cache-versengés miatt a falidő nagyobb, mint a szekvenciális 8 szálas BLAS-futásé. A javítás: a környezeti változót a NumPy importja elé tenni, vagy a `threadpoolctl.threadpool_limits(limits=1)` hívást használni.

4. A 200 MB-os DataFrame-et a `ProcessPoolExecutor` minden feladatindításkor pickle-lel sorosítja, a célprocessz memóriájába másolja, majd a visszatérési értéket is sorosítja. Ez négy feladatnál több száz MB adatmozgatást és jelentős CPU-időt adhat. A költség csökkenthető, ha a munkafüggvény maga olvassa be a saját partícióját egy Parquet-fájlból; így csak a fájlnév megy át pickle-ként. Alternatíva a `joblib` loky backend memmap-támogatása vagy `multiprocessing.shared_memory`, de ezek bonyolultabb élettartam-kezelést igényelnek, ezért a fájlalapú partíció a kezdő tanulási úton a tisztább választás.

5. A BLAS a mátrixszorzást különböző szálkiosztás esetén más sorrendben összegezheti, és a lebegőpontos összeadás nem asszociatív, ezért az utolsó bitek eltérhetnek. Ez nem sérti a numerikus reprodukálhatóságot, mert a seed rögzítése a bemeneti mátrixokat reprodukálja, és az eltérés a lebegőpontos kerekítésből fakad. Az ellenőrzésre `np.allclose(a, b, rtol=1e-12, atol=1e-12)` vagy egy explicit relatív hibakorlát használandó, nem a `==`.

33.2.8 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés az, hogy „a GIL miatt a szálak soha nem gyorsítanak”. Ellenpélda a fenti I/O-feladat: a szekvenciális 2,1 s-hoz képest a szálas változat 0,7 s-ot adhat, mert a fájlvárakozás alatt más szál fut. A második tévedés az, hogy „több worker mindig jobb”. Ha 8 logikai magos gépen 4 worker egyenként 8 BLAS-szálat indít, a 32 szál túlterheli a CPU-t, és a falidő meghaladhatja a szekvenciális futást. A helyes döntés a mérésből következik: a 33.1. alfejezet szűk keresztmetszet-elemzés után először a BLAS belső szálait korlátozzuk, majd a külső worker-számot növeljük, és a falidőt ismételt méréssel, mediánnal hasonlítjuk össze.

33.2.9 Rövid összefoglaló

A CPython GIL-je a Python-bájtkódot sorossá teszi, de az I/O-várakozás és a natív NumPy/BLAS-kód alatt felszabadul. A `ThreadPoolExecutor` ezért kis fájlok tömeges írására-olvasására és natív tömbműveletekre jó; a `ProcessPoolExecutor` akkor indokolt, ha tiszta Python CPU-kódot kell párhuzamosítani, de a pickle-sorosítás költségét mérlegelni kell. A BLAS saját szálait a NumPy importja előtt explicit módon 1-re kell állítani, különben a külső workerek és a belső szálak oversubscriptiont okoznak. A mérési jegyzék platformot, magszámot, feladatszámot, ismétlést és mediánt rögzít. A következő lépésben ezt a szálkeretet párhuzamos modellkeresésre alkalmazzuk az erőforráskeretes 33.3. alfejezetben.

33.3 Párhuzamos modellkeresés erőforráskerettel

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: cProfile, threadpoolctl, joblib

Előfeltétel: 33.2. alfejezet · Célkészség: Párhuzamos modellkeresés erőforráskerettel · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

33.3.1 Cél és konkrét helyzet

A feladatunk nem egy új modell betanítása, hanem egy kétparaméteres keresés megtervezése úgy, hogy az adott idő- és memóriakeretbe beleférjen. A Northstar prediktív labor egyik lépéseként egy `RandomForestClassifier` két hiperparaméterét hangoljuk belső 5-szörös keresztvalidációval (CV). A gép 4 fizikai magos, 8 GiB RAM-mal. A keret: legfeljebb 90 másodperc falóra idő (wall-clock time) és legfeljebb 1 GiB memória.

Az input `X` 50 000 sorból és 60 oszlopból álló `float64` mátrix, `y` 50 000 elemű egész célvektor. A rács `n_estimators` értékeire `[50, 100]`, `max_features` értékeire `[0.3, 0.5, "sqrt"]` szól, tehát $2 \times 3 = 6$ paraméterkombináció. A megfigyelési egység e laborban egy `fit`+`score` feladat: egy paraméterkombináció egy belső foldon. A keresztvalidációs futásszám ezért $6 \times 5 = 30$. A `GridSearchCV` alapértelmezett `refit=True` beállítása ezen felül **egy** végső fitet végez a legjobb paraméterrel a teljes tanítóhalmazon, így az összes `estimator.fit` hívás száma 31.

Előzetes becslés: egy feladat CPU-ideje körülbelül 2,0 s, így a teljes CPU-munka $31 \times 2{,}0 = 62$ s. Két workerrel optimistán számolva a falóra idő nagyjából 36 s, a memória pedig néhány száz MiB. A terv akkor jó, ha egy kis kontrollfutás megerősíti a 62 CPU-s nagyságrendet, mielőtt a teljes rácsot elindítjuk.

33.3.2 Fogalmi magyarázat

33.3.2.1 n_jobs és a szorzatszabály

A `GridSearchCV(n_jobs=...)` argumentuma azt mondja meg, hány worker folyamatot indíthat a joblib. Az `n_jobs=-1` a gépen elérhető összes logikai CPU magot jelenti, de ez nem növeli a feladatok számát, csak a falóra idő csökkentésére ad lehetőséget.

A keresés teljes `fit`-száma két részből áll: a keresztvalidációs fitekből és a végső refitből.

$$N_{\text{cv}} = K_{\text{outer}} \cdot M_{\text{grid}} \cdot K_{\text{inner}}, \qquad N_{\text{fit}} = N_{\text{cv}} + r_{\text{refit}} \cdot K_{\text{outer}},$$

ahol:

- K_{outer}: a külső kiértékelő CV foldjainak száma (ha nincs, akkor 1),
- M_{grid}: a paraméterkombinációk száma,
- K_{inner}: a belső `GridSearchCV` foldjainak száma,
- $r_{\text{refit}} \in \{0, 1\}$: 1, ha a kereső a legjobb paraméterrel a teljes tanítóhalmazon újra illeszt (`refit=True`, ez az alapértelmezés), különben 0.

Az aktuális példánkban $M_{\text{grid}} = 6$, $K_{\text{inner}} = 5$, külső CV nélkül: $N_{\text{cv}} = 30$, és a refit miatt $N_{\text{fit}} = 31$. Egy jövőbeli külső 5-fold hozzáadásakor $N_{\text{cv}} = 5 \cdot 6 \cdot 5 = 150$, a foldonkénti refittel $N_{\text{fit}} = 155$; ha a külső kereső a végén még egy végső modellt is illeszt, az további +1, azaz 156. A szorzás nem a magok számától, hanem a kísérleti szerkezettől és a `refit` beállítástól függ.

33.3.2.2 Becsült mintánkénti költség

Ha egy feladat átlagos CPU-idejét t_{one} jelöli, a teljes CPU-munka:

$$T_{\text{cpu}} = N_{\text{fit}} \cdot t_{\text{one}}.$$

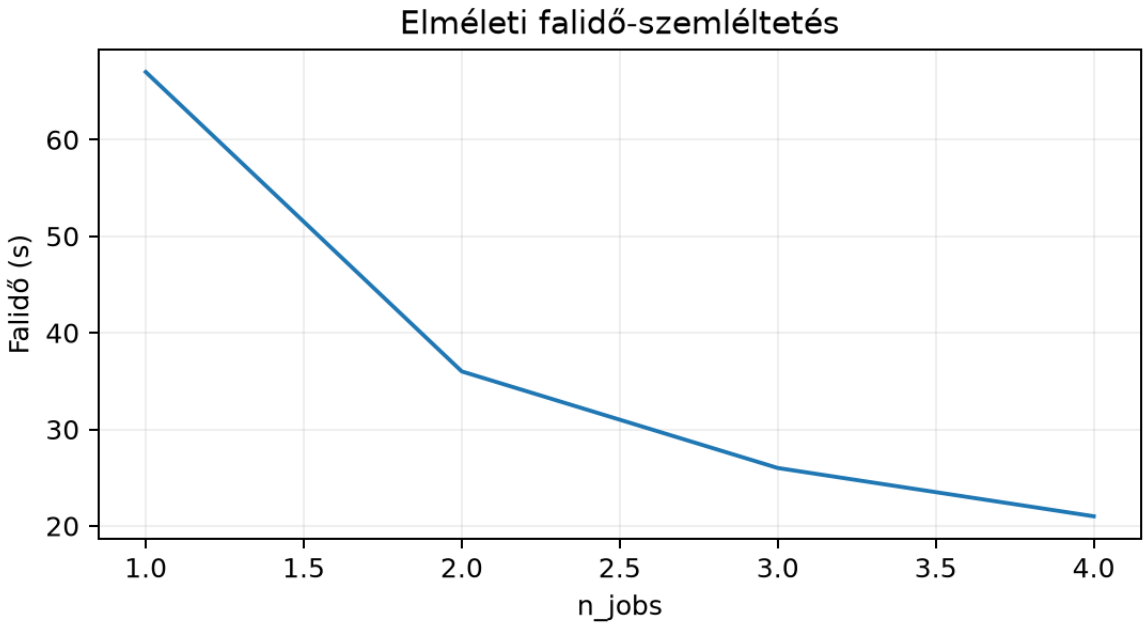
A párhuzamos falóra idő közelítőleg:

$$T_{\text{wall}} \approx T_{\text{overhead}} + \frac{T_{\text{cpu}}}{n_{\text{eff}}},$$

ahol n_{eff} a ténylegesen hasznos párhuzamosság, T_{overhead} pedig a folyamatindítás és a dispatcher költsége másodpercben. Ha a modell CPU-kötött és minden mag szabad, n_{eff} közel lehet az `n_jobs`-hoz; ha a memória sávszélesség vagy az I/O telítődik, n_{eff} kisebb.

Például a 30 CV-fit és 1 refit 2,0 s/feladat mellett $31 \cdot 2 = 62$ s CPU-munka; 2 workerrel a falóra idő $5 + 62/2 = 36$ s, ha nincs oversubscription. A párhuzamosítás nem csökkenti a CPU-összeget, sőt az overhead miatt kismértékben növelheti.

A szemléltető görbe a 62 CPU-s, 5 s overheades elméleti falóra időt mutatja:



33.1. ábra: Elméleti falidő-szemléltetés - Számpélda: 62 s CPU-munka (30 CV + 1 refit), 5 s overhead. Nem mért adat.

33.3.2.3 pre_dispatch

A `pre_dispatch` a dispatcher által előre elküldött, a szülőfolyamat memóriájában várakozó feladatok számát korlátozza. Az int érték közvetlen feladatszám, az alapértelmezett `"2*n_jobs"` pedig az `n_jobs` kétszerese. Ha minden feladat nagy bemeneti adatot visz, a magas `pre_dispatch` a szülőfolyamat memóriáját növeli; a worker folyamatok memóriáját azonban nem ez, hanem a lemásolt `X`, `y` és a felépült modell határozza meg.

33.3.2.4 joblib backend

A `GridSearchCV` a joblib-et használja. Az alapértelmezett `loky` backend újrahasznosítható munkafolyamatokat indít; ilyenkor minden worker külön Python-interpreterrel fut. A `threading` backend szálakat használ: Python-számításnál a GIL miatt általában nem gyorsít, natív NumPy/BLAS kódban viszont hasznos lehet. A backendet contextben lehet váltani, például `with parallel_backend("threading"):` alatt. A könyv fő esete a loky, mert a faépítés Python- és numpy-kódot is váltogat.

33.3.2.5 Nested parallelism

Nested parallelism akkor alakul ki, ha a belső becslő maga is `n_jobs`-ot kap. Például `GridSearchCV(n_jobs=4)` és `RandomForestClassifier(n_jobs=-1)` együtt egy 4 magos gépen akár 32 aktív feldolgozó egységet is létrehozhat, amelyek egymástól veszik el a CPU-időt. A hatékony megoldás a szintek szétválasztása: a külső párhuzamosítás folyamatokkal, a belső modell `n_jobs=1` értékkel, vagy fordítva, attól függően, melyik ad kisebb overheadet.

33.3.2.6 RAM budget

A memóriaterv képlete:

$$M_{\text{budget}} = M_{\text{base}} + n_{\text{workers}} \cdot M_{\text{worker}} + n_{\text{pending}} \cdot S_{\text{task}},$$

ahol:

- M_{base}: a szülőfolyamat memóriája az adatokkal és a `cv_results_` tárolóval,
- n_{workers}: a tényleges worker folyamatok száma,
- M_{worker}: egy worker átlagos memóriaigénye — a loky spawn esetén bele kell értenünk a bemeneti mátrix másolatát és a fitted modellt is,
- n_{pending}: a pre_dispatch által megengedett várakozó feladatok száma,
- S_{task}: egy várakozó feladat bemeneti adatának mérete.

A mértékegységeket MiB-ben használjuk. A `pre_dispatch` tehát csak a képlet utolsó tagját szabályozza, a worker-memóriát nem.

Jel	Jelentés	Mértékegység	Feltevés
N_{cv}	keresztvalidációs `fit`+`score` feladatok száma	darab	$K_{\text{outer}} \cdot M_{\text{grid}} \cdot K_{\text{inner}}$
r_{refit}	végső refit van-e	0/1	`refit=True` esetén 1, különben 0
N_{fit}	összes `estimator.fit` hívás	darab	$N_{\text{cv}} + r_{\text{refit}} \cdot K_{\text{outer}}$
t_{one}	egy feladat átlagos CPU-ideje	s/feladat	azonos méretű feladatok, ugyanaz a modell
T_{cpu}	teljes CPU-munka	s	$N_{\text{fit}} \cdot t_{\text{one}}$
T_{overhead}	processzindítás és dispatcher	s	üres feladatokkal mérhető
T_{wall}	falóra idő	s	$T_{\text{overhead}} + T_{\text{cpu}}/n_{\text{eff}}$
C_{copy}	workerenkénti bemenetmásolás	MiB	$n_{\text{workers}} \cdot$ X.nbytes, ha nincs memmap
n_{native}	workerenkénti BLAS/OMP szál	darab	`n_jobs × n_native` ne haladja meg a magokat
n_{eff}	ténylegesen hasznos párhuzamosság	darab	CPU-kötött munka, nincs oversubscription
M_{budget}	becsült memóriaigény	MiB	$M_{\text{base}} + n_{\text{workers}} M_{\text{worker}} + n_{\text{pending}} S_{\text{task}}$
S_{task}	egy várakozó feladat bemenetének mérete	MiB	a bemeneti mátrix és a modell másolata

A táblázat feltételei mellett a számok összhangban vannak: $31 \cdot 2{,}0 = 62$ s CPU, $5 + 62/2 = 36$ s falidő két workerrel. Ha a modell memóriát vagy I/O-t telít, az $n_{\text{eff}} < n_{\text{jobs}}$ miatt a falidő hosszabb lesz; ezt a táblázat külön feltételként nevezi meg.

33.3.2.7 Refit, overhead, adatmásolás és natív szálak

A négy további költségtényező külön soron szerepel, mert mindegyik más beavatkozást igényel:

- **Refit:** a `GridSearchCV(refit=True)` a legjobb paraméterrel a teljes tanítóhalmazon újra illeszt; ez a 31. fit. Ha csak a CV-becslés kell, `refit=False`, de akkor nincs `best_estimator_`.
- **Overhead:** a loky worker-indítás, a dispatcher és a `pre_dispatch` költsége. Üres feladatokkal külön mérhető; kis feladatoknál ez dominálhat.
- **Adatmásolás:** process-alapú párhuzamosításnál a bemeneti mátrix workerenként pickle-elődik; `C_copy  n_workers · X.nbytes`. Memmap vagy fájlalapú partíció csökkenti.
- **Natív szálak:** a BLAS/OMP szálak a workereken belül is futnak; a `n_jobs × n_native` szorzatot a fizikai magokhoz kell mérni, különben oversubscription lesz.

A tervezőfüggvény:

```
def fit_budget(k_outer, grid, k_inner, refit=True, outer_refit=False):
    n_cv = k_outer * grid * k_inner
    n_fit = n_cv + (k_outer if refit else 0) + (1 if outer_refit else 0)
    return {"n_cv": n_cv, "n_fit": n_fit}

print(fit_budget(1, 6, 5))                 # {'n_cv': 30, 'n_fit': 31}
print(fit_budget(5, 6, 5))                 # {'n_cv': 150, 'n_fit': 155}
print(fit_budget(5, 6, 5, outer_refit=True))  # {'n_cv': 150, 'n_fit': 156}
```

A mérési jegyzőkönyvben az **elméleti** és a **mért** fit-szám külön jelenjen meg: egy instrumentált estimator `fit`-számlálója adja a mértet, a fenti függvény az elméletit. Ha a kettő eltér, a `refit` beállítás vagy a CV szerkezete nem az, amit a terv feltételez. A `tests/test_fit_budget.py` a 31/155/156 értékeket és egy instrumentált `GridSearchCV` fit-számlálóját ellenőrzi.

33.3.3 Teljes minimális munkapélda

A következő scriptterminálban vagy VS Code Python-környezetben futtatható. Linux/macOS esetén a projektgyökérből:

```
python grid_search_demo.py
```

Windows PowerShell esetén `python grid_search_demo.py` a projektgyökérből.

```
# grid_search_demo.py
import time

from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# Deterministikus kis szintetikus adat
X, y = make_classification(
    n_samples=50_000,
    n_features=60,
    n_informative=20,
    random_state=20261020,
)
X = X.astype("float64")

print("X nbytes:", X.nbytes, "y nbytes:", y.nbytes)

param_grid = {
```

```
    "n_estimators": [50, 100],
    "max_features": [0.3, 0.5, "sqrt"],
}

# Kontrollfutás: egyetlen fit ideje a rács egy pontján
model_single = RandomForestClassifier(
    n_estimators=50, max_features=0.3, random_state=7, n_jobs=1
)
t0 = time.perf_counter()
model_single.fit(X, y)
single_dt = time.perf_counter() - t0
print(f"Egy fit: {single_dt:.2f} s; 31 fit (30 CV + 1 refit) CPU-ideje kb. {31 * single_dt:.0f}
↪  s")

# 30 CV-fit + 1 refit, 2 loky worker, legfeljebb 3 előreküldött feladat
search = GridSearchCV(
    RandomForestClassifier(random_state=7, n_jobs=1),
    param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=2,
    pre_dispatch=3,
    verbose=1,
)

t0 = time.perf_counter()
search.fit(X, y)
wall_dt = time.perf_counter() - t0

print("Best:", search.best_params_, "AUC:", search.best_score_, "Wall time:", wall_dt)
print(
    "Paraméterkombinációk hossza:",
    len(search.cv_results_["mean_test_score"]),
    "belső foldok:",
    search.n_splits_,
    "CV-fit termékszabály:",
    len(search.cv_results_["mean_test_score"]) * search.n_splits_,
)
```

A verbose log egy lehetséges, szemléltető részlete:

```
X nbytes: 24000000 y nbytes: 400000
Egy fit: 1.94 s; 31 fit (30 CV + 1 refit) CPU-ideje kb. 60 s
[Parallel(n_jobs=2)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=2)]: Done  30 out of  30 | elapsed:   36.2s finished
Best: {'max_features': 0.5, 'n_estimators': 100} AUC: 0.91 Wall time: 36.2
Paraméterkombinációk hossza: 6 belső foldok: 5
CV-fit: 30 (+1 refit a végén)
```

33.3.4 Eredmény értelmezése

A `cv_results_["mean_test_score"]` hossza 6, mert a keresés hat paraméterkombinációt átlagol öt belső fold fölött. A `search.n_splits_` értéke 5; a kettő szorzata 30, ami a keresztvalidációs `fit+score` hívások száma. A verbose log `Done 30 out of 30` sora ezt a 30 CV-fit-et igazolja; a 31. fit a `refit=True` miatti végső illesztés a teljes tanítóhalmazon, amely a párhuzamos logban külön sorként nem jelenik meg. A kontrollfutás `1,94 s`-et adott, a 31 fitre számított `60 s` CPU-munka pedig a 36 s falóra idővel konzisztens: óvatos overheaddel számolva a 90 s keret alatt maradunk.

A párhuzamosítás determinisztikus: ugyanaz a `random_state=7` és ugyanaz a CV-split mellett a pontszám ugyanaz, akár `n_jobs=1`, akár `n_jobs=2` fut. Csak a falóra idő változik, attól függően, hogy a gép éppen mit futtat. Ezért a fenti időt és AUC-t nem nevezzük ellenőrzöttnek; a saját gépen a verziók, a BLAS és a CPU-hűtés miatt eltérhet.

A RAM becslés példája: `X` 24 MiB, `y` 0,4 MiB. A szülőfolyamat alapmemóriája körülbelül 120 MiB. A loky spawn minden workerben lemásolhatja a bemeneti mátrixot és felépíti a modellt; egy worker így jellemzően 60–100 MiB-ot használ. Két workerrel ez legfeljebb 200 MiB, három előreküldött feladat elhanyagolható mérete mellett. Az 1 GiB-os keret bőséges.

A `timeit` mikroszintű ismételt mérésre való; itt a makroszintű `perf_counter` jobban használható, mert a fit egyszer lefut, és a cache-hatások is érdekelnek. A `cProfile` a szülőben futó dispatcher hívásait mutatja, a worker folyamatok belső profiljához külön megoldás kell. A memóriaprofilt `memory_profiler` vagy `psutil.Process().memory_info()` adhat; a worker folyamatokat a szülőén kívül külön kell mérni.

A `concurrent.futures ProcessPoolExecutor` hasonló folyamatpárhuzamosságot ad, de a `GridSearchCV`-ben a joblib a memmap-képességek és a `pre_dispatch` miatt jobban kézben tartja a memóriát. Ha kézzel írjuk a hurkot, a joblib vagy az executor egyaránt elfogadható, de a feladatköltséget és a worker-memóriát akkor is előre kell becsülni.

A worker-szám és a natív számítási szálak száma együtt adja az erőforrás-terhelést. A `n_jobs` növelése nem jelent automatikus gyorsulást.

33.3.5 Feladatok

1. feladat — Értelmezés.
Egy kutató külső 5-fold CV-t futtat, és minden külső foldra külön `GridSearchCV`-t indít. A belső rács 4 `C`-értékből és 2 `penalty`-ből áll, a belső CV 3-foldos. A belső `GridSearchCV` `n_jobs=3`, `pre_dispatch=6`. Egy belső `fit+score` feladat átlagos ideje `n_jobs=1` mellett 0,9 s, az effektív párhuzamosság `n_eff=2,5` a gépen. A memória: `M_base=250 MiB`, `M_worker=180 MiB`, egy várakozó feladat `S_task=2 MiB`. Számítsd ki a teljes feladatszámot, a becsült CPU-időt, a falóra időt és a RAM-keretet.

2. feladat — Megvalósítás.
Adott `X` 400 MiB-os tömb és `y` 2 MiB-os vektor. A loky spawn minden workerben lemásolja az `X`-et; a workerben a fitted modell további 50 MiB-ot foglal, a szülőfolyamat alapmemóriája 300 MiB. A `pre_dispatch` legfeljebb 10 feladatot tart előre, mindegyik bemenete 2 MiB. A gép 8 GiB RAM-mal rendelkezik. Mekkora lehet legfeljebb a biztonságos `n_jobs`? Adj meg egy konkrét `GridSearchCV` konfigurációt, amely kényelmesen benne marad a keretben, és írd le, miért nem érdemes a kiszámolt maximumot választani.

3. feladat — Hibakeresés.
A tanuló egy 4 magos gépen ezt írja:

```
model = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=7)
search = GridSearchCV(
    model,
    {"max_features": [0.2, 0.5, "sqrt"], "min_samples_leaf": [1, 5]},
    cv=5,
    scoring="roc_auc",
    n_jobs=4,
    pre_dispatch=8,
)
search.fit(X, y)
```

Az egyérintéses `RandomForestClassifier(n_jobs=1)` futás 20 s volt, de ez a keresés 120 s-nál is tovább tartott. Nevezd meg a jelenséget, magyarázd meg, miért lassul, és adjon két lehetséges javítást.

4. feladat — Megváltozott feltétel.
A felhasználó áttér egy 16 magos gépre, de a memória sávszélesség ugyanakkora marad. A keresett becslő lineáris modell, amely natív BLAS-hívásokkal dolgozik. A kutató a külső `GridSearchCV`-t `n_jobs=4`-gyel, a belső modellt azonban a BLAS alapértelmezett szálszámával futtatja. Milyen probléma léphet fel, miért nem biztos, hogy a nagyobb magszám gyorsabb? Adj meg egy ellenőrizhető új futtatási tervet, amely a környezeti változókat és az `n_jobs`-ot is szabályozza.

33.3.6 Megoldások és indoklás

1. megoldás.
A feladatszám a szorzatszabályból:

$$N_{\text{fit}} = 5 \cdot (4 \cdot 2) \cdot 3 = 5 \cdot 8 \cdot 3 = 120.$$

A CPU-munka:

$$T_{\text{cpu}} = 120 \cdot 0{,}9 = 108 \text{ s}.$$

A becsült falóra idő $108/2{,}5 + T_{\text{overhead}}$. Az overheadet 5 s-nak feltételezve $43{,}2 + 5 = 48{,}2$ s. A RAM-büdzsé:

$$250 + 3 \cdot 180 + 6 \cdot 2 = 250 + 540 + 12 = 802 \text{ MiB}.$$

Ez elfér egy 8 GiB-os gépben, de a biztonsági tartalék miatt érdemes a `pre_dispatch`-et 6 alatt hagyni, ha a worker-memória megnő. Az ellenőrző invariáns: a `pre_dispatch` nem változtatja a pontszámot, csak a szülőfolyamat várakozó memóriáját.

2. megoldás.
A RAM-korlát 8 GiB = 8192 MiB. A biztonságos `n_jobs`-ra:

$$300 + n \cdot (400 + 50) + 10 \cdot 2 \le 8192,$$

$$450n \le 8192 - 320 = 7872,$$

$$n \le 17{,}49.$$

Tehát elméletileg legfeljebb 17 worker férne el, de a 17 worker 17 processzként osztozna a CPU magokon; ha a gép kevesebb magos, a kontextusváltás dominál, és a workerindítás overheadje is nagy. Emellett az operációs rendszer és a Jupyter is foglal memóriát. Válasszunk például `n_jobs=8`-at és `pre_dispatch=8`-at:

```
search = GridSearchCV(
    RandomForestClassifier(n_jobs=1, random_state=7),
    param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=8,
    pre_dispatch=8,
)
```

Ez a RAM-képlet szerint $300 + 8 \cdot 450 + 8 \cdot 2 = 300 + 3600 + 16 = 3916$ MiB, tehát bőven a keret alatt van, és a CPU-ütemezés is mérsékelt marad.

3. megoldás.
A jelenség a nested parallelism okozta oversubscription. A külső `GridSearchCV(n_jobs=4)` négy worker folyamatot indít, mindegyik `RandomForestClassifier(n_jobs=-1)` pediga gép minden magján elindíthat a faépítésben további szálakat. Egy 4 magos gépen így akár 16 aktív feldolgozó egység verseng, a kontextusváltás és a cache-thrash miatt a falóra idő nagyobb lehet, mint az egyérintéses futás lineárisan várt értéke.

Javítás 1: a belső modellt `n_jobs=1`-re állítani, a külső `GridSearchCV` `n_jobs=4` megmarad. Javítás 2: ha a belső párhuzamosítást akarjuk megtartani, a külső `n_jobs=1` legyen, és a belső fa kapja meg a magokat. A választást kontrollméréssel kell eldönteni: 3–5 ismétlés, azonos seed, azonos scoring. Az is fontos, hogy a `pre_dispatch=8` nem segít a nested parallelismon, mert az csak a várakozó feladatok számát korlátozza.

4. megoldás.
A 16 magos gépen a külső 4 loky worker mindegyike a BLAS-könyvtár alapértelmezett szálszámával is használhatja az összes magot, így akár $4 \cdot 16 = 64$ konkurens BLAS-szál is elindulhat. Ráadásul a nagyobb magszám nem növeli a memória sávszélességet; ha a lineáris modell memóriához kötött, a telített sáv miatt a 4 külső worker sem gyorsul lineárisan.

Javított terv: a külső párhuzamosítást 4 loky workerre hagyjuk, a belső becslőt `n_jobs=1`-re állítjuk, és a futtatás előtt a környezeti változókat egyszálasra állítjuk:

```
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
python grid_search_demo.py
```

Windows PowerShellben:

```
$env:OMP_NUM_THREADS = "1"
$env:OPENBLAS_NUM_THREADS = "1"
$env:MKL_NUM_THREADS = "1"
python grid_search_demo.py
```

Így összesen legfeljebb 4 folyamat dolgozik, és elkerüljük a BLAS-szálak keresztütemezését. A terv érvényességi korlátja: ha a memória sávszélesség már 2 workernél telítődik, a 4 worker sem ad további gyorsulást; ezért a kontrollmérést `n_jobs=1`, `n_jobs=2` és `n_jobs=4` esetére is el kell végezni.

33.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az, hogy a tanuló azt feltételezi: a belső `n_jobs=-1` és a külső `n_jobs=4` együtt gyorsít, mert több worker dolgozik. Valójában a worker-folyamatok és a natív szálak nem független gyorsulást adnak, hanem osztoznak ugyanazon a négy magon. Egy 4 magos gépen a 30 CV-fit + 1 refit kontrollban az egyérintéses 20 s-es fitből a külső 4 workerrel 100 s CPU-munka

kéne, de a tiszta külső párhuzamosítás 30 s alatt végzett. Miután a belső fában `n_jobs=-1`-et is engedtünk, a falóra idő 55 s-ra nőtt, miközben a CPU-monitor minden magot 100%-on mutatott. A magas CPU-kihasználtság tehát nem jelentett gyors futást; a context switch és a cache-thrash vitte el a hasznot.

33.3.8 Visszautalás és önellenőrzés

A 33.2. alfejezetben tanult GIL- és natív szálas döntés itt válik konkrét konfigurációvá: a loky folyamatszintű párhuzamosítást választjuk, a belső fa `n_jobs=1`-gyel fut. A 32.3. alfejezetből tudjuk, hogy a CI-ben a drága tanítást külön lépésként kezeljük; ugyanez a fegyelem indokolja, hogy a teljes rács előtt egyetlen fitet mérünk. A 29.3. alfejezetben tanult invariant-tesztelés itt a `len(cv_results_) * n_splits_ = 30` CV-fit termékszabály és a `refit=True` miatti +1 ellenőrzésében tér vissza.

Önellenőrző kérdések a labor előtt:

- Meg tudom-e mondani a `GridSearchCV` összes `fit` hívásának számát indítás előtt?
- A `pre_dispatch` a várakozó feladatok vagy a worker folyamatok számát szabályozza?
- A belső estimator `n_jobs`-a és a külső `GridSearchCV` `n_jobs`-a összeszorzódik-e?
- A mérési jegyzőkönyvben szerepel-e a gép, a méret, a verzió, a seed és az ismétlésszám?

33.3.9 Rövid összefoglaló

A párhuzamos keresés indítása előtt a szorzatszabállyal számítsuk ki a teljes feladatszámot, az egyfeladatos fitből becsüljük a CPU-időt, és a worker- plusz dispatcher-memóriából becsüljük a RAM-keretet. A joblib loky backend folyamatokat indít; a `pre_dispatch` a szülőfolyamat várakozó sorát korlátozza, nem a worker-memóriát. A nested parallelism elkerülése érdekében a belső estimator `n_jobs=1` legyen, ha a külső keresés párhuzamos. A kontrollfutás egyetlen fit mérése és a 30 CV-fit + refit termékszabály ellenőrzése megbízhatóbb döntést ad, mint a CPU-monitor figyelése. A következő lépés, a 33.4. alfejezet, ezt a mérést használja fel változatlan jelentés melletti optimalizáláshoz.

33.4 Optimálás változatlan jelentéssel

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: cProfile, threadpoolctl, joblib**
>
> Előfeltétel: 33.3. alfejezet · Célkészség: Optimálás változatlan jelentéssel · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

33.4.1 Cél és konkrét helyzet

A VIII. ellenőrző pontban (32.4. alfejezet) a modellprojekt átadható lett, de a predikciós pipeline fejlesztőgépen érezhetően lassú: az `account_age_days` kiszámítása `iterrows()` ciklussal történt, és 60 000 rendelésre másodperceket vesz igénybe. Ebben az alfejezetben egyetlen helyi optimalizálást végzünk el azzal a feltétellel, hogy a predikciós szerződés (21.4. alfejezet) semmilyen ponton nem változik: ugyanazok a feature-nevek, dtype-ok, megfigyelési egységek és toleranciakorlátok maradnak.

- **Bemenet**: a szintetikus Northstar-orders DataFrame `order_ts` és `signup_date` időbélyegekkel, valamint `basket_minor`, `quantity_sum`, `distance_km`, `priority_flag` oszlopokkal. A példában `n=60_000` sort állítunk elő determinisztikus seeddel, mert ezen a méreten már mérhető a különbség.
- **Megfigyelési egység**: egy rendelés (`order_id`), nem ügyfél és nem tétel.
- **Célkimenet**: a gyorsított függvény legalább ugyanazt az `account_age_days` egész napokban kifejezett feature-t adja, a lebegőpontos oszlopok pedig 1e-4 relatív tűrésen belül azonosak maradnak.
- **Előzetes becslés**: a vektorizált változat várhatóan 20–100-szor gyorsabb; a downcast nagyjából 25–40 % DataFrame-méret-csökkenést ad. A pontos szám nem előre adott, tehát minden állítást méréssel igazolunk.

33.4.2 A kontrollháromszög: idő, memória, numerikus egyezés

Egy gyorsítás akkor nevezhető azonos jelentésű optimalizálásnak, ha három mérést egyszerre mutat fel:

1. **Futásidő** (`wall-clock time`): azonos gépen, azonos inputon, több ismétléssel. Az operációs rendszer terhelése miatt az egyetlen futás nem elegendő. A `timeit.repeat` által visszaadott minták közül a minimum stabilabb a zajjal szemben, mint az átlag, mert a háttérfolyamatok az átlagot felfelé torzítják, a minimum nem.
2. **Memória**: a DataFrame tényleges belső mérete `memory_usage(deep=True).sum()` és adott esetben a `tracemalloc` csúcsfogyasztása. A deep=True a Python-objektumok tartalmát is beszámítja, nem csak a NumPy-arrays mutatóit.
3. **Numerikus egyezés** (`equivalence check`): az eredeti aranyszabály kimenetének és a gyorsított változat kimenetének összehasonlítása meghatározott tűréssel.

A numerikus tűrés általános formulája:

$$|a_i - b_i| \le \tau_a + \tau_r \cdot |b_i|,$$

ahol a_i a gyorsított, b_i az aranyszabály szerinti érték; τ_a az abszolút, τ_r a relatív tűrés. A képlet mértékegysége megegyezik a mért mennyiségével, tehát `distance_km` esetén kilométerben add meg az engedett eltérést. Például $\tau_a = 10^{-6}$, $\tau_r = 10^{-4}$ mellett 100 km-nél legfeljebb $10^{-6} + 10^{-4} \cdot 100 = 0{,}010001$ km eltérés megengedett. A NumPy `np.allclose(a, b, rtol=..., atol=...)` pontosan ezt a hibamodellt valósítja meg.

33.4.3 Downcast, projection, caching és vectorization

A négy optimalizálási elem közös jellemzője, hogy a megfigyelési egységet és a predikciós célváltozót egyik sem módosítja, csak a számítás módját vagy a közbülső reprezentációt.

33.4.3.1 Downcast

A pandas alapértelmezésben `int64` és `float64` típusokat használ, mert ezek biztonságosan lefednek minden szokásos értéket. Ha egy oszlop értékkészlete szűkebb, a dtype szűkíthető; például a Northstar `quantity_sum` 1 és 15 közé esik, így `int16` is elegendő, a `basket_minor` pedig az adatszerződés szerint EUR-centben pozitív és nem haladd meg a szimuláció tartományában a `int32` felső határát. A szűkebb típus kisebb memóriát foglal, gyorsabb lehet a CPU-cache miatt, és gyakran gyorsítja a NumPy/BLAS műveleteket.

A downcast azonban nem cserélhető le néma `astype` hívásra. Minden oszlopnál ellenőrizni kell:

```
import numpy as np

info = np.iinfo(np.int32)
if not (df["basket_minor"].min() >= info.min and df["basket_minor"].max() <= info.max):
    raise ValueError("basket_minor outside int32 range")
```

A `float64 → float32` átalakítás relatív kerekítése körülbelül $2^{-24} \approx 6 \cdot 10^{-8}$, tehát egyetlen értékre kicsi. Ha viszont később sok számot összegzünk, például csoportos kosárösszeget, a halmozódó hiba már túllépheti a tűrést. Pénzösszeget ezért sosem szabad `float32`-ben összegezni; a kosár maradjon egész `minor unit` formában.

33.4.3.2 Vectorization

Az eredeti lassú kód minden sorra Python-kifejezésben hajtja végre a kivonást:

```
def slow_age(df):
    out = []
    for _, row in df[["order_ts", "signup_date"]].iterrows():
        out.append((row["order_ts"] - row["signup_date"]).days)
    return pd.Series(out, index=df.index, dtype=np.int32)
```

Az `iterrows()` minden sorra új `Series`-t és Python-objektumokat hoz létre; a 60 000 sor ciklusa ezért másodpercekig fut. A vektorizált változat a pandas `DatetimeIndex`-ekre közvetlenül alkalmazott kivonás, majd a `.dt.days` accessor:

```
def fast_age(df):
    return (df["order_ts"] - df["signup_date"]).dt.days.astype(np.int32)
```

Itt a kivonás teljes tömbre vonatkozik, a `.dt.days` a belső C-műveletet használja, nem a Python-értelmező per-sorosan magyarázd a kódot. A mátrixszorzásos lépéseknél a NumPy a gépre telepített BLAS-t is igénybe veheti, ami további gyorsulást ad.

33.4.3.3 Projection

A projection (vetítés) azt jelenti, hogy csak azokat az oszlopokat és sorokat olvassuk be vagy tartjuk a memóriában, amelyeket a későbbi számítás ténylegesen használ. A predikciós szerződésben rögzített feature-lista alapján például nem olvassuk be a `delivery_days` és `actual_delivery_at` oszlopokat, mert azok kizárt prediktorok:

```
FEATURE_COLS = [
    "order_id", "order_ts", "signup_date", "distance_km",
    "priority_flag", "basket_minor", "quantity_sum",
]
orders = pd.read_csv("data/orders.csv", usecols=FEATURE_COLS)
```

A `usecols` már a CSV-értelmezés előtt szűkíti az adatmozgatást, tehát kevesebb I/O és kisebb kezdő DataFrame keletkezik. A projection a memóriában is folytatódik: az ideiglenes segédoszlopokat törölni kell, mielőtt a modellbe adnánk a táblát, mert a felesleges oszlopok is memóriát foglalnak.

33.4.3.4 Caching

Ha egy adatelőkészítő vagy feature-számító függvényt többször hívunk, a kimenet elmenthető. A cache azonban csak akkor érvényes, ha a bemenet ugyanaz. A kulcs legyen a bemeneti fájlok hash-e és a feature-verzió:

```
cache_key = f"features_v3_{sha256(orders_csv_bytes).hexdigest()[:16]}.joblib"
```

Ha a kulcs egyezik, a `joblib.dump`-pal elmentett fájl betölthető; ha nem, újra kell számolni. A 32.4. alfejezetben tanult manifest- és hashfegyelem itt is kötelező: a cache-fájl nem helyettesíti a verziózott adatellenőrzést, és a splitmanifestben rögzített adatváltozatnak összhangban kell lennie a cache-kulccsal. Ellenkező esetben elavult cache-t használunk, ami a modell rosszabb teljesítményét okozhatja.

33.4.3.5 Equivalence check

Az optimalizált változatot minden releváns oszlopra összehasonlítjuk az aranyszabállyal:

- egész és kategóriás oszlopokra bitazonosság (`np.array_equal`),
- lebegőpontos oszlopokra `np.allclose` meghatározott `rtol` és `atol` értékkel,
- shape és dtype ellenőrzés a pipeline kimeneténél,
- a predikció esetén a 21.4. alfejezet szerinti `(n, 2)`-es valószínűség-séma is.

A teszt nem azt kérdezi, hogy a gyorsított kód nagyjából hasonló, hanem hogy a modell tényleges bemenete és kimenete a tűrésen belül azonos-e.

33.4.4 Teljes minimális munkapélda

Az alábbi kód a szövegben szereplő Northstar-orders oszlopok mintájára előállít egy 60 000 soros szintetikus DataFrame-et, majd leméri a lassú és a gyors változatot. A mérési számok ebben a generálási módban nem valós futási jegyzék; a kód a tanuló gépén futtatandó. A kód kommentjei angolok, a változónevek a tankönyv konvencióját követik.

```
import timeit
import tracemalloc
import numpy as np
import pandas as pd

# Deterministic synthetic input; one row is one order.
rng = np.random.default_rng(20261020)
n = 60_000

df = pd.DataFrame({
    "order_ts": (
        pd.Timestamp("2025-01-01", tz="UTC")
        + pd.to_timedelta(rng.integers(0, 180 * 24 * 3600, size=n), unit="s")
    ),
    "signup_date": (
        pd.Timestamp("2024-06-01")
        + pd.to_timedelta(rng.integers(0, 365, size=n), unit="D")
    ),
    "basket_minor": rng.integers(1000, 50000, size=n),
    "quantity_sum": rng.integers(1, 15, size=n, dtype=np.int16),
    "distance_km": rng.gamma(2.0, 15.0, size=n),
    "priority_flag": rng.integers(0, 2, size=n, dtype=np.int8),
})
```

```
def slow_features(df):
    """Gold baseline: explicit Python loop for account_age_days."""
    out = []
    for _, row in df.iterrows():
        out.append((row["order_ts"] - row["signup_date"]).days)
    df = df.copy()
    df["account_age_days"] = pd.Series(out, index=df.index, dtype=np.int32)
    return df

def fast_features(df):
    """Optimized: vectorized datetime and checked downcasts."""
    df = df.copy()
    df["account_age_days"] = (
        (df["order_ts"] - df["signup_date"]).dt.days.astype(np.int32)
    )
    info = np.iinfo(np.int32)
    if not (df["basket_minor"].min() >= info.min
            and df["basket_minor"].max() <= info.max):
        raise ValueError("basket_minor outside int32 range")
    df["basket_minor"] = df["basket_minor"].astype(np.int32)
    df["distance_km"] = df["distance_km"].astype(np.float32)
    return df

def measure_time(func, df_arg, repeats=5, number=5):
    """Return min and median seconds per call as (min, median)."""
    raw = timeit.repeat(lambda: func(df_arg), repeat=repeats, number=number)
    per_call = [item / number for item in raw]
    return min(per_call), float(np.median(per_call))

gold = slow_features(df)
candidate = fast_features(df)

# Equivalence: integer and boolean columns require bitwise equality.
assert np.array_equal(gold["account_age_days"].to_numpy(),
                      candidate["account_age_days"].to_numpy())
assert np.array_equal(gold["basket_minor"].to_numpy(),
                      candidate["basket_minor"].to_numpy())
# Float columns use a tolerance. Distance values are synthetic km.
assert np.allclose(gold["distance_km"].to_numpy(),
                   candidate["distance_km"].to_numpy(),
                   rtol=1e-4, atol=1e-6)
print("equivalence: OK")

slow_min, slow_med = measure_time(slow_features, df)
fast_min, fast_med = measure_time(fast_features, df)
print(f"slow  min={slow_min:.4f}s median={slow_med:.4f}s")
print(f"fast  min={fast_min:.4f}s median={fast_med:.4f}s")

gold_mb = gold.memory_usage(deep=True).sum() / 1e6
cand_mb = candidate.memory_usage(deep=True).sum() / 1e6
print(f"gold DataFrame: {gold_mb:.2f} MB, candidate: {cand_mb:.2f} MB")

# Peak allocation during the optimized call.
tracemalloc.start()
_ = fast_features(df)
_, peak_bytes = tracemalloc.get_traced_memory()
tracemalloc.stop()
print(f"fast peak allocation: {peak_bytes / 1e6:.2f} MB")
```

Ha a függvény fut, az `equivalence: OK` sor akkor jelenik meg, ha mindhárom oszlopra teljesül a tűrés. A futásidő gépenként eltérő; tankönyvi szemléltetésként egy CPU-s laptopon gyakran a lassú változat 0,4–0,8 másodperc, a gyors pedig 0,003–0,010 másodperc közé esik. A DataFrame

méretében a `basket_minor` int32-re és a `distance_km` float32-re alakítása nagyjából 25 % megtakarítást hoz, mert a két nagy oszlop bájtjainak egy része felszabadul.

33.4.5 Eredmény értelmezése

A mérési kódból kiderül, hogy a gyorsítás nem „feltételezett”, hanem ugyanazon az inputon, ugyanazon a gépen mért. A `min`-alapú futásidő csökkenti a háttérterhelés torzítását, a median a tipikus futást mutatja. Ha a kettő nagyon eltér, a mérési környezet terhelt, és érdemes a `number` értéket növelni.

A numerikus egyezés azért fontos, mert a lebegőpontos `distance_km` float32-re alakítása tényleges kerekítési hibát visz be. Az `np.allclose(rtol=1e-4, atol=1e-6)` ezt a hibát a tűrésen belül engedi. A tűrés megválasztása a feladat mértékegységéből következik: 1 km alatt 1 méternyi abszolút hiba sem változtatja meg a modell döntését, de 10 % relatív hiba már igen.

A `quantity_sum` és `priority_flag` oszlopok már eleve szűk dtype-tal rendelkeznek a bemenetben; a downcast itt nem hozhat további nyereséget. A gyorsított változat tehát csak azokat az oszlopokat alakítja, amelyeknél a tartományvizsgálat megengedi, és minden más értéket bitazonosan őriz.

Az optimalizálás helyességi tesztjének nemcsak az átlagot, hanem a sorazonosítókat, a hiányok szemantikáját és az időzóna-kezelést is őriznie kell.

33.4.5.1 Metamorf equivalence: sorazonosító, as-of, címke, összeg és metrika

A kétoszlopos `check_equivalence` nem elég: a gyorsítás átrendezheti a sorokat, megváltoztathatja a csoportosítás határait, vagy más címkeeloszlást adhat. A metamorf teszt ezért a teljes predikciós szerződést hasonlítja össze:

1. **Sorazonosítók:** azonos sorszám, egyedi `order_id`, és a rendezett azonosítólisták egyezése. Az átrendezés önmagában megengedett, de a halmaz nem változhat.
2. **As-of oszlopok:** `order_ts`, `signup_date`, `label_available_at` bitazonos; ezek az időbeli jelentés hordozói.
3. **Címke:** `target_late` értéke és dtype-ja azonos.
4. **Egész pénz- és darabszám-oszlopok:** `basket_minor`, `quantity_sum`, `priority_flag` egzakt egyezése.
5. **Lebegőpontos oszlopok:** `distance_km` és a többi float a deklarált `rtol`/`atol` tűréssel.
6. **Aggregátumok:** `sum(basket_minor)`, `sum(quantity_sum)`, sorszám és a `distance_km` átlaga a tűrésen belül.
7. **Metrika:** egy előre rögzített metrikafüggvény (például AUC vagy átlagos késési arány) eltérése a `metric_tol` alatt.

```
import numpy as np

class EquivalenceError(AssertionError):
    pass

ASOF = ["order_ts", "signup_date", "label_available_at"]
EXACT = ["basket_minor", "quantity_sum", "priority_flag", "target_late"]
FLOATS = ["distance_km"]

def assert_equivalent(gold, cand, *, rtol=1e-4, atol=1e-6,
                      metric=None, metric_tol=1e-6):
    errors = []
    if len(gold) != len(cand):
```

```
        errors.append(f"sorszám: {len(gold)} != {len(cand)}")
    g = gold.sort_values("order_id").reset_index(drop=True)
    c = cand.sort_values("order_id").reset_index(drop=True)
    if not g["order_id"].is_unique or not c["order_id"].is_unique:
        errors.append("nem egyedi order_id")
    if list(g["order_id"]) != list(c["order_id"]):
        errors.append("sorazonosító-halmaz eltér")
    for col in ASOF + EXACT:
        if not np.array_equal(g[col].to_numpy(), c[col].to_numpy()):
            errors.append(f"egzakt eltérés: {col}")
    if g["target_late"].dtype != c["target_late"].dtype:
        errors.append("target_late dtype eltér")
    for col in FLOATS:
        if not np.allclose(g[col].to_numpy(), c[col].to_numpy(),
                           rtol=rtol, atol=atol):
            errors.append(f"tűrésen kívüli eltérés: {col}")
    for col in ("basket_minor", "quantity_sum"):
        if int(g[col].sum()) != int(c[col].sum()):
            errors.append(f"összeg eltér: {col}")
    metric_delta = None
    if metric is not None:
        metric_delta = abs(float(metric(g)) - float(metric(c)))
        if metric_delta > metric_tol:
            errors.append(f"metrika eltérés: {metric_delta:.3e}")
    if errors:
        raise EquivalenceError("; ".join(errors))
    return {"rows": len(g), "metric_delta": metric_delta}
```

A hívás például `assert_equivalent(gold, cand, metric=lambda df: df["target_late"].mean(), metric_tol=1e-6)`. Ha bármelyik ellenőrzés elbukik, a gyorsított változatot **vissza kell állítani**; a futásidő-nyereség nem kompenzálja a sorazonosító vagy a címke megváltozását. A `tests/test_metamorphic_equivalence.py` az átrendezést, a tűrésen belüli és kívüli float-eltérést, a sorazonosító-, as-of-, címke- és metrikaeltérést teszteli.

33.4.6 Feladatok

1. **Értelmezés** Miért nem elegendő egyetlen wall-clock futás ahhoz, hogy kimondjuk a gyorsítást? Mi a szerepe a `timeit.repeat` `number` paraméterének, és miért a minimumot, nem az átlagot érdemes jelenteni? Fogalmazza meg két-három mondatban a mérési protokoll követelményét.

2. **Megvalósítás** Írj `safe_downcast` függvényt, amely a `basket_minor` oszlopot `int32`-re, a `distance_km`-et `float32`-re alakítja. Ha a `basket_minor` értékkészlete kívül esik az `int32` tartományán, a függvény dobjon `ValueError`-t. A `distance_km`-re 100 km felett is maximálisan $10^{-4} \cdot \text{érték} + 10^{-6}$ eltérést engedjen meg az equivalence checkben.

3. **Hibadiagnózis** Egy fejlesztő a következő gyorsítást alkalmazza: a `quantity_sum` oszlopot `float32`-re alakítja, mert így kisebb a DataFrame. A csoportos kosárösszeg egy 13456 darabos rendelési csoportban `13456.0` helyett `13455.997`-ot ad, és a későbbi modell AUC-értéke 0,782-ről 0,778-ra romlik. Hol a hiba, és milyen szabályt sértett meg?

4. **Megváltozott feltétel** A projekt elérte az 1,2 millió rendelést; a fejlesztő a gyorsított `fast_features` kimenetét ügyfélazonosítónként külön particionált Parquet-fájlokba menti. A második futtatáskor csak azt a partíciót szeretné újraszámolni, ahol a bemenő vektor megváltozott. Tervezze meg a cache-kulcsot és az érvénytelenedési feltételt úgy, hogy az ügyfélazonosító a megfigyelési egység maradjon, és a numerikus equivalence minden partícióra teljesüljön. Add meg a cache-frissítést ellenőrző pszeudokódot.

33.4.7 Megoldások és indoklás

1. Egyetlen wall-clock futás a háttérfolyamatok és az operatív memória állapota miatt nem reprodukálható. A `timeit.repeat` `number` paramétere meghatározza, hány függvényhívásból áll egy mérési ismétlés; ez csökkenti a mért időben a Python-értelmező indítási költségét, bár a loop-overhead részben megmarad. A minimum azért jobb, mint az átlag, mert a min a legkevésbé zavart futást mutatja: a process legalább ennyi idő alatt képes elvégezni a feladatot. A medián a tipikus futást jelzi. A protokoll követelménye: azonos gépet, azonos inputot, több ismétlést, azonos `number`-t, és a memóriaterhelést is rögzíteni kell.

2. Egy lehetséges megoldás:

```
def safe_downcast(df):
    df = df.copy()
    info = np.iinfo(np.int32)
    if df["basket_minor"].min() < info.min or df["basket_minor"].max() > info.max:
        raise ValueError("basket_minor does not fit int32")
    df["basket_minor"] = df["basket_minor"].astype(np.int32)
    df["distance_km"] = df["distance_km"].astype(np.float32)
    return df

# Invariant: integer column remains bit-equal, float column respects tolerance.
def check_equivalence(gold, cand):
    assert np.array_equal(gold["basket_minor"].to_numpy(),
                          cand["basket_minor"].to_numpy())
    assert np.allclose(
        gold["distance_km"].to_numpy(),
        cand["distance_km"].to_numpy(),
        rtol=1e-4,
        atol=1e-6,
    )
```

A meghívott `np.allclose` a kért képletet valósítja meg; a mértékegység kilométer. A teszt csak akkor megy át, ha minden sorban a relatív és abszolút hiba együtt a tűrés alatt van.

3. A `quantity_sum` darabszám egész, tehát a bitazonos reprezentáció szükséges; a lebegőpontos átalakítás beleviszi a kerekítési hibát. A hiba forrása: a `float32` mantissza 24 bit, ezért nagyobb összegeknél elvesznek bináris helyiértékek. A szabály, amit megsértett: a pénz- és darabszám-egészeket sosem alakítjuk lebegőpontossá gyorsítási célból; az equivalence check csak a kategóriás és egész oszlopokra bitazonosságot fogad el, a lebegőpontos tűrés nem alkalmazható rájuk. Helyes megoldás: a `quantity_sum` maradjon int16, és ha a cél az AUC megőrzése, a checkpointnak minden modellbemeneti oszlopra az előírt tűrést kell vizsgálnia.

4. A partíciós cache terve:

```
import joblib
import hashlib

def partition_cache_key(customer_id, input_vector):
    raw = f"{customer_id}:{input_vector.tobytes()}".encode("utf-8")
    return hashlib.blake2b(raw, digest_size=16).hexdigest()

def cached_fast_features_for_customer(customer_id, df_part):
    # Load the previous key for this customer partition, if it exists.
    meta = load_partition_meta(customer_id)   # {"key": str, "file": str}
    current_key = partition_cache_key(customer_id, df_part["order_ts"].to_numpy())
    if meta is not None and meta["key"] == current_key:
        return pd.read_parquet(meta["file"])
```

```
    result = fast_features(df_part)
    new_file = f"cache/customer_{customer_id}_{current_key}.parquet"
    result.to_parquet(new_file)
    save_partition_meta(customer_id, {"key": current_key, "file": new_file})
    return result
```

Az érvénytelenedés feltétele: ha bármely bemeneti oszlop bytesorozata megváltozik, a `blake2b` kulcs megváltozik. Az ügyfélazonosító a cache-kulcs része, tehát az egyes ügyfelek megfigyelési egységét nem keverjük. A frissítés csak ahhoz az ügyfélhez törli a cache-t, amelynél a kulcsváltozást észleltük. Az equivalence-check partíciónként ugyanazokat az oszloptoleranciákat alkalmazza.

33.4.8 Gyakori tévedés és önellenőrzés

Gyorsabb, de más célt számoló program nem optimalizálás. Tipikus ellenpélda: a fejlesztő a predikciós pipeline-t úgy gyorsítja, hogy az `account_age_days` helyett közvetlenül a `delivery_days` oszlopot használja, mert az gyorsabb és jobb AUC-t ad. Ez azonban megsérti az adatszerződés azon szabályát, hogy a célmagyarázat kizárt prediktor; a gyors változat a tesztidőben nem ismert jövőbeli információt épít be. A numerikus equivalence-check önmagában nem elég: a célváltozó definícióját is változatlanul kell tartani. Hasonlóan hibás az a „gyorsítás”, amelyik a `basket_minor`-ból kihagyja a `discount_minor` levonását; a kapott kosárösszeg gyorsabban számolható, de nem ugyanaz a mennyiség.

Önellenőrző kérdések: (1) Az optimalizált változat minden modellbemeneti oszlopra azonos dtype-ot és értékkészletet tart-e? (2) A memóriamérést a deep=True vagy a shallow pointerek alapján végeztem? (3) A gyorsított függvény bemenetét ugyanazzal az adatszerződéssel validáltam-e, mint az aranyszabályt? (4) A cache-kulcs tartalmazza-e az összes felhasznált input hashét?

33.4.9 Rövid összefoglaló

Az optimalizálás csak akkor elfogadható, ha a futásidő, a memória és a numerikus kimenet egyszerre változatlan jelentést igazol. A downcast szűkíti a dtype-ot, de csak tartományellenőrzés után; a vectorization a Python-ciklus helyett tömbműveletet használ; a projection felesleges oszlopokat hagy ki; a caching elmenti a drága eredményt input-hash-hez kötve. Az equivalence-check az egész oszlopokra bitazonosságot, lebegőpontos oszlopokra relatív- és abszolút tűrést követel. A következő témakör ezt a mért, reprodukálható optimalizálási fegyelmet nagyobb adatra, a Nagyobb adat: Dask, Spark és elosztott végrehajtás fejezet Dask- és Spark-alapú környezeteire terjeszti ki.

34 Nagyobb adat: Dask, Spark és elosztott végrehajtás

34.1 Nagyobb mint RAM: előbb adatfolyam

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Dask, Spark

Előfeltétel: 33.4. alfejezet · Célkészség: Nagyobb mint RAM: előbb adatfolyam · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A Northstar rendelési tábla a kezdőlaborban mindössze 2400 sor; a következő példában azt vizsgáljuk, mi történik, ha ugyanezt a sémát egy nagyítási paraméterrel több tízmillió sorra növeljük, és a gépünk RAM-ja nem nő együtt az adattal. A cél nem az, hogy azonnal klasztert vagy felhős erőforrást válasszunk, hanem hogy előbb megmérjük: mennyi adatot kell ténylegesen beolvasni, és milyen technikákkal csökkenthetjük a memória csúcsát egyetlen gépen.

34.1.1 Cél és konkrét helyzet

A feladatunk egy egyszerű üzleti kérdés: a Northstar rendelésekben mekkora az átlagos `distance_km` a `priority_flag` szerinti két csoportban? A bemenet a már megtisztított `orders.csv`, amelynek oszlopai között szerepel a `distance_km` (float, kilométer), a `priority_flag` (0/1 egész) és számos további, a kérdéshez nem szükséges mező (például `actual_delivery_at`, `basket_minor`, `delivery_days`). A megfigyelési egység egy rendelés. Az elvárt kimenet: soronként a `priority_flag`, a csoport elemszáma és az átlagos távolság; ellenőrzésként ugyanannak a két csoportnak az elemszáma összegezve a bemeneti sorok számát adja, és a csoportátlagok a teljes átlagot súlyozottan reprodukálják.

Előzetes becslés: ha a táblát `scale_factor = 10 000`-rel nagyítjuk, akkor $2400 \times 10\,000 = 24\,000\,000$ sorunk lesz. A teljes táblában 15 oszlop van; ha átlagosan oszloponként 8 bájtot számolunk (a dátum-szövegek ennél nagyobbak, a 0/1 flag kisebb), akkor a nyers reprezentáció nagyságrendje $24\,\mathrm{M} \times 15 \times 8 \approx 2.9\,\mathrm{GB}$. A tényleges memóriaigény erősen függ a tárolási formátumtól, a stringoszlopok kódolásától és a végrehajtó motortól; a 4–6 GB legfeljebb szemléltető becslés, nem általános RAM-követelmény. A kérdéshez viszont csak két oszlop kell: $24\,\mathrm{M} \times 2 \times 8 = 384\,\mathrm{MB}$, ami egy átlagos fejlesztői laptopon is kényelmesen elfér.

34.1.2 Fogalmi magyarázat: hat eszköz a beolvasás csökkentésére

Projection (oszlopkiválasztás). A vetítés azt jelenti, hogy a lekérdezésben vagy a beolvasó hívásban csak azokat az oszlopokat kérjük el, amelyekre a számításnak ténylegesen szüksége van. Az oszlopos (columnar) fájlformátumok, mint a Parquet, ezt hatékonyan támogatják: a motor egyszerűen nem olvassa be a többi oszlop adatblokkjait. Ha a `distance_km` és a `priority_flag` oszlopokat kérjük, a `basket_minor`, a dátumok és a többi mező bájtjai a lemezen maradnak. A

memóriaigény így $M_{\text{read}} = n \sum_{j \in S} b_j$ bájt, ahol n a sorok száma, S a kiválasztott oszlophalmaz, b_j pedig az egy sorra jutó bájtmennyiség az adott oszlopban.

Partition pruning (partícióvágás). Ha a fájl nem egyetlen nagy objektum, hanem könyvtárstruktúrába rendezett partíciókra van osztva — például `year=2025/month=01/` —, akkor a lekérdezésben szereplő szűrési feltétel alapján a motor a partíciók egy részét teljesen kihagyja. Ha az adat P egyenlő partícióra oszlik, és a feltétel k partíciót érint, akkor a beolvasott adatmennyiség $\frac{k}{P}$-szeresére csökken. A vágás akkor működik, ha a partíciós oszlop értéke magából a könyvtárnévből kiolvasható, és a motor nem kényszerül minden fájl megnyitására.

Chunk (darab). A chunk-olvasás során a fájlt nem egyszerre, hanem rögzített méretű sorcsoportokban vagy row groupokban töltjük be. A Parquet eleve row groupokra tagolódik; a DuckDB, a Polars és a Pandas is képes batch-enként olvasni. Ha egyszerre B soros darabot tartunk a memóriában, és az aggregációs állapot korlátos, akkor a memóriaigény $O(B)$ lehet; rendezésnél, nagy kardinalitású csoportosításnál vagy joinnál ez nem garantált.

Streaming. A streaming feldolgozás azt jelenti, hogy a számítás darabonként történik, és a közbenső állapot nem tartalmazza a teljes bemenetet. Egy csoportosított átlaghoz elég darabonként összegezni a `distance_km` értékeket és a sorok számát; a végeredményhez nincs szükség arra, hogy mind a 24 millió sor egyszerre a memóriában legyen. A két előre ismert csoportot használó streaming átlag számításának aggregációs állapota a sorok számában $O(1)$ (a beolvasási batch ezen felül memóriát igényel), míg a beolvasás utáni in-memory aggregáció $O(n)$.

Spill. Ha egy művelet — például rendezés vagy join — közbenső állapota meghaladja a beállított memóriakorlátot, a motor a felesleget lemezre írja. Ez a spill: a lekérdezés nem omlik össze, de lassabb lesz, mert lemez I/O keletkezik. Egy külső rendezésnél a lemezre írt futamok száma közelítőleg $\lceil M_{\text{work}}/M_{\text{limit}} \rceil$, ahol M_{work} a rendezendő adat memóriaigénye, M_{limit} pedig a rendelkezésre álló munkamemória. A spill tehát biztonsági szelep, nem gyorsítás.

Small files. Ha a partíciók túl aprók, minden fájl megnyitása, metaadat-olvasása és lezárása fix költséget jelent. Sok ezer kis fájl esetén ez a fix költség nagyobb lehet, mint maga az adatolvasás. Ökölszabály: egy partíciófájl ne legyen néhány tíz MB-nál kisebb, és a row group mérete legalább százezres nagyságrendű sor legyen. A partícióvágás tehát nem jelenti azt, hogy tetszőlegesen finomra kell darabolni az adatot.

A teljes beolvasás és a vetített streaming közötti különbség algoritmikus értelemben is megfogható: mindkét út időkomplexitása $O(n)$, de a konstans szorzó és a memóriaigény nagyon eltérő. A vetítés a beolvasott bájtok számát csökkenti, a streaming pedig a memóriában tartott állapotot.

34.1.3 Teljes minimális munkapélda

A példát a Northstar `lab/data/orders.csv` fájljából kiindulva építjük fel. A nagyításhoz DuckDB-t használunk, mert a Parquet-írás során explicit row group méretet adhatunk, és a lekérdezésekben a projection és a pruning automatikusan érvényesül. A kód a `lab/` könyvtárból indul, ezért a benne szereplő útvonalak ehhez képest értendők: a bemenet `data/orders.csv`, a generált nagyított fájl pedig a `data/` alkönyvtárba kerül.

```
# data/generate_large.py
# A nagyított tanulótábla előállítása; a scale_factor a nagyítási paraméter.
import duckdb

con = duckdb.connect("data/northstar_large.duckdb")
scale_factor = 100  # 2400 * 100 = 240 000 sor; növelhető, pl. 10000
```

```
con.execute("""
    COPY (
        SELECT
            o.order_id || '-' || r.i AS order_id,
            o.customer_id,
            o.order_ts,
            o.promised_days,
            o.quantity_sum,
            o.item_rows,
            o.basket_minor,
            o.shipping_fee_minor,
            o.currency,
            o.distance_km,
            o.priority_flag,
            o.delivery_days,
            o.actual_delivery_at,
            o.label_available_at,
            o.target_late
        FROM read_csv('data/orders.csv', header = true) AS o
        CROSS JOIN (SELECT i FROM range(0, ?) AS r(i)) AS r
    ) TO 'data/orders_large.parquet'
      (FORMAT PARQUET, COMPRESSION ZSTD, ROW_GROUP_SIZE 100000)
""", [scale_factor])

print(con.execute("""
    SELECT count(*) AS n, count(DISTINCT order_id) AS n_unique
    FROM read_parquet('data/orders_large.parquet')
""").fetchall())
```

A generálás után két utat hasonlítunk össze. Az első a vetített aggregáció: a lekérdezésben csak a két szükséges oszlop szerepel, így a Parquet-olvasó a többi oszlopot nem is érinti.

```
# data/compare_reads.py
import duckdb
import time

con = duckdb.connect("data/northstar_large.duckdb")

# 1. út: vetített aggregáció
t0 = time.perf_counter()
result_projected = con.execute("""
    SELECT priority_flag,
           count(*)             AS n,
           avg(distance_km)     AS avg_km
    FROM read_parquet('data/orders_large.parquet')
    GROUP BY priority_flag
    ORDER BY priority_flag
""").fetchall()
t1 = time.perf_counter()

# 2. út: teljes beolvasás, majd aggregáció a memóriában
con.execute("DROP TABLE IF EXISTS full_orders")
t2 = time.perf_counter()
con.execute("""
    CREATE TEMP TABLE full_orders AS
    SELECT * FROM read_parquet('data/orders_large.parquet')
""")
result_full = con.execute("""
    SELECT priority_flag,
           count(*)             AS n,
           avg(distance_km)     AS avg_km
    FROM full_orders
    GROUP BY priority_flag
```

```
    ORDER BY priority_flag
""").fetchall()
t3 = time.perf_counter()

print("Vetített eredmény: ", result_projected)
print("Teljes beolvasás:  ", result_full)
print(f"Vetített aggregáció idő: {t1 - t0:.3f} s")
print(f"Teljes beolvasás + aggregáció idő: {t3 - t2:.3f} s")
```

A fenti kódot a kapcsolódó példaprojektben még nem ellenőriztük, ezért a konkrét időértékek **szemléltető jellegűek**. Az elméletileg várt viselkedés: a vetített változat kevesebb bájtot olvas a lemezről, és a memória-csúcs is alacsonyabb; a teljes beolvasásnál a `full_orders` ideiglenes tábla a 15 oszlop minden értékét a RAM-ba teszi. Kis `scale_factor` mellett a különbség alig látszik, mert az I/O-időt a parancsértelmezés fix költsége dominálja; a különbség a nagyított adaton válik mérhetővé.

Ugyanez a feladat Polars streaminggel is kifejezhető. Itt a `scan_parquet` lusta (lazy) tervet ad, a `select` végzi a vetítést, a `collect(engine="streaming")` pedig streaming aggregációt kér:

```
import polars as pl

lf = (
    pl.scan_parquet("data/orders_large.parquet")
    .select(["priority_flag", "distance_km"])
    .group_by("priority_flag")
    .agg([pl.count(), pl.col("distance_km").mean()])
)
result = lf.collect(engine="streaming")
print(result)
```

34.1.4 Eredmény értelmezése és kézi kontroll

A két út eredményének numerikusan egyeznie kell. A `priority_flag` két értéke (0 és 1) szerinti csoportokra a `count(*)` összege a bemeneti sorok száma: $n_0 + n_1 = 2400 \times \text{scale_factor}$. A csoportátlagokból a teljes átlag súlyozottan visszaszámolható:

$$\bar{x} = \frac{n_0 \bar{x}_0 + n_1 \bar{x}_1}{n_0 + n_1}.$$

Ha a `priority_flag` eloszlása a nagyítás során nem változik — a cross join minden eredeti sort ugyanannyiszor másol —, akkor a csoportarányok és az átlagok is megőrződnek. Ez a tulajdonság a nagyítási eljárás invariantja: a 2400 soros alap és a nagyított tábla csoportátlagai megegyeznek. A gyakorlatban a lebegőpontos összegzés sorrendje miatt az utolsó biten eltérés lehet; a kontrollhoz 10^{-9} relatív tolerancia elegendő.

A vetített és a teljes beolvasás közötti idő- és memóriakülönbséget a `scale_factor` növelésével lehet mérhetővé tenni. A méréshez ugyanazon a gépen, azonos seed és fájl mellett, több ismétléssel kell dolgozni; a `perf_counter` egyszeri lefutása csak nagyságrendi tájékozódásra való. Ha a folyamat RAM-csúcsát pontosan akarjuk ismerni, a `resource` modul `getrusage` értéke vagy a DuckDB memória-statisztikája használható, nem a Parquet fájl mérete.

34.1.4.1 Chunkolt átlag: miért hibás az átlagok átlaga

Ha az adatot nem tudjuk egyben beolvasni, a chunkokból csak **összegeket és darabszámokat** szabad kombinálni; a chunkátlagok egyszerű átlaga akkor is hibás, ha minden chunk ugyanabból az eloszlásból származik, de eltérő a méretük. Legyen három chunk:

Chunk	n_p	$\bar{x}_p$	$s_p = n_p \bar{x}_p$
A	100	10	1 000
B	300	20	6 000
C	600	30	18 000

A chunkátlagok egyszerű átlaga $(10 + 20 + 30)/3 = 20$, a helyes teljes átlag:

$$\bar{x} = \frac{s_A + s_B + s_C}{n_A + n_B + n_C} = \frac{1000 + 6000 + 18000}{1000} = 25.$$

A 20 és a 25 különbsége nem kerekítés, hanem az, hogy a 600 elemű chunkot ugyanolyan súllyal vette volna, mint a 100 eleműt. A helyes kombináló:

```
def combine_chunks(chunks):
    total_sum = sum(n * mean for n, mean in chunks)
    total_n = sum(n for n, _ in chunks)
    return {"count": total_n, "sum": total_sum, "mean": total_sum / total_n}

print(combine_chunks([(100, 10.0), (300, 20.0), (600, 30.0)]))
# {'count': 1000, 'sum': 25000.0, 'mean': 25.0}
```

Csoportos átlagnál ugyanez csoportonként külön történik: a chunkon belüli `(group, count, sum)` hármasokat kell összeadni, nem a csoportátlagokat átlagolni. Az ellenőrzési invariáns: a teljes `count` megegyezik a bemeneti sorok számával, a teljes `sum` a referenciaösszeggel, és a `sum/count` a referenciaátlaggal. A `tests/test_chunked_aggregation.py` a három eltérő méretű chunkot, az egyszerű átlag hibáját és a chunkméret-változtatás invarianciáját teszteli.

34.1.5 Feladatok

1. **Értelmezés.** Egy 200 millió soros táblában 12 oszlop van: 8 oszlop `float64`, 2 oszlop `int32`, 2 oszlop átlagosan 20 karakteres szöveg. Becsüld meg a nyers reprezentáció méretét bájtban (a szövegnél 20 bájt/sor értékkel számolj), majd a DataFrame overheaddel együtt, ha az overhead szorzó 1.8×. Hány GB RAM kell a teljes beolvasáshoz?

2. **Implementálás.** Írj DuckDB lekérdezést, amely a `data/orders_large.parquet` fájlból csak a `distance_km` és a `priority_flag` oszlopokat olvassa be, és csoportonként visszaadja az átlagot, a minimumot és a maximumot. Ellenőrizd, hogy a `min(distance_km) <= avg(distance_km) <= max(distance_km)` mindkét csoportban teljesül.

3. **Hibadiagnózis.** Egy kolléga a következő kódot írja a nagyított táblára:

   ```
   df = pd.read_parquet('data/orders_large.parquet')
   result = df[df['region'] == 'north'].distance_km.mean()
   ```

   Nevezz meg legalább három memóriaproblémát ebben a megoldásban, és javasolj javítást.

4. **Megváltozott feltétel.** Tegyük fel, hogy a nagyított táblát hónap szerint particionáljuk: `year=2025/month=01/`, `year=2025/month=02/`, és így tovább, 12 egyenlő partícióra. A lekérdezés csak a 2025. januári rendeléseket érinti. Mennyivel csökken a beolvasott adatmennyiség, ha a partíciós oszlopra szűrünk? Miért lehet mégis rossz, ha a partíciókat naponként hozzuk létre?

5. **Továbbvitel.** Egy rendezési feladat 5 GB munkamemória-igényű adatot dolgoz fel, de a motornak csak 1 GB spill-korlátot adtunk. Hány lemezfutamra számíthatsz közelítőleg? Hogyan változik a válasz, ha a korlátot 2 GB-ra emeljük?

34.1.6 Megoldások és indoklás

1. megoldás. A nyers reprezentáció soronként:

$$b = 8 \times 8 + 2 \times 4 + 2 \times 20 = 64 + 8 + 40 = 112 \text{ bájt/sor}.$$

A teljes nyers méret:

$$M_{\text{raw}} = 200\,000\,000 \times 112 = 22.4 \times 10^9 \text{ bájt} = 22.4 \text{ GB}.$$

A DataFrame overheaddel:

$$M_{\text{mem}} = 1.8 \times 22.4 = 40.3 \text{ GB}.$$

Tehát legalább 32–40 GB RAM szükséges a teljes beolvasáshoz; a tömörített Parquet fájl ennél jóval kisebb lehet, de ez nem csökkenti a memóriaigényt. Ha csak például 3 oszlopra van szükség, a vetített nyers méret $200\,\text{M} \times 24 = 4.8$ GB, ami 8.6 GB RAM-mal már kezelhető.

2. megoldás. A lekérdezés:

```
SELECT priority_flag,
       avg(distance_km) AS avg_km,
       min(distance_km) AS min_km,
       max(distance_km) AS max_km
FROM read_parquet('data/orders_large.parquet')
GROUP BY priority_flag
ORDER BY priority_flag;
```

Az ellenőrzés: minden visszaadott sorra igaz, hogy `min_km <= avg_km <= max_km`. A vetítés itt automatikus: a DuckDB a Parquet-ból csak a két hivatkozott oszlopot olvassa, mert a lekérdezésben más oszlop nem szerepel. A `GROUP BY` streaming aggregációként futhat, így a 24 millió soros bemenet sem kerül egyszerre a memóriába.

3. megoldás. A hibák: (a) a `pd.read_parquet` `columns` paraméter nélkül mind a 15 oszlopot beolvassa, pedig csak kettőre van szükség; (b) a `region` szűrés a betöltés után történik, tehát a motor nem tud partícióvágást végezni, és a szükségtelen sorokat is beolvassa; (c) a pandas a teljes DataFrame-t a RAM-ba teszi, így a memóriaigény a legrosszabb eset. Javítás:

```
df = pd.read_parquet(
    'data/orders_large.parquet',
    columns=['region', 'distance_km'],
    filters=[('region', '==', 'north')],
)
result = df['distance_km'].mean()
```

A `filters` paraméter a PyArrow motorral partícióvágást is végezhet, ha a partíciós oszlop a feltétel szerint van kialakítva; a `columns` paraméter a vetítést adja meg. Ha a fájl nem particionált, a szűrés legalább a betöltött oszlopokra korlátozódik, és a memória már csak két oszlopra nő.

4. megoldás. Ha a 12 hónap partíciói egyenletesek, akkor a januári szűrés $\frac{1}{12}$-ére csökkenti a beolvasott adatmennyiséget. A naponkénti partíciók esetén 365 fájl keletkezik évente; ha a teljes adat csak néhány GB, akkor egy-egy napi fájl néhány MB-os lehet, és a fájlnyitási, metaadat-olvasási és fájllistázási költség dominál. Ráadásul a nagyon sok apró fájl a schedule- és cache-rétegeket is terheli. Az optimális partíciós szemcsézettség a lekérdezési mintától függ: ha a leggyakoribb szűrés havi, akkor a havi partíció jobb; ha napi, akkor a napi is indokolt lehet, de akkor a partíciók méretét érdemes figyelni.

5. megoldás. A futamok száma:

$$R = \left\lceil \frac{5}{1} \right\rceil = 5 \quad \text{az 1 GB korlátnál,}$$

és

$$R = \left\lceil \frac{5}{2} \right\rceil = 3 \quad \text{a 2 GB korlátnál.}$$

A kevesebb futam kevesebb lemez I/O-t és gyorsabb összefésülést jelent, de a nagyobb korlát több RAM-ot foglal el más műveletektől. A spill tehát erőforrás-kompromisszum, nem hiba; a lényeg, hogy a motor ne omoljon össze, ha a munkamemória elfogy.

34.1.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: „A Parquet fájl 2 GB, tehát 2 GB RAM elég lesz.” Ez azért hamis, mert a Parquet tömörített, oszlopos formátum, és a fájlméret nem mond semmit a dekompresszált, objektumfejekkel és indexekkel terhelt memóriareprezentációról. Konkrét ellenpélda: egy fájl 50 millió sort tartalmaz, oszlopai `float64`, `int64` és átlagosan 50 karakteres szöveg. A Parquet tömörített mérete lehet 2 GB. A memóriában a `float64` és `int64` oszlop együtt $50\,\mathrm{M} \times 16 = 800$ MB, a szöveges oszlop viszont Python-szövegként soronként körülbelül 99 bájtot foglal (49 bájt objektumfej + 50 bájt karakter), ami $50\,\mathrm{M} \times 99 \approx 4.95$ GB. A teljes DataFrame így 5–6 GB RAM-ot igényelhet, miközben a fájl csak 2 GB. A tömörített méret tehát legfeljebb alsó becslésnek használható, de a memóriaigényt mindig a dekompresszált reprezentációból kell számolni.

Ez visszavezet a 33.1. alfejezet mérési fegyelméhez: a szűk keresztmetszetet mérni kell, nem a fájlméretből kikövetkeztetni. A 30.1. alfejezet pipeline-szemlélete pedig arra emlékeztet, hogy a beolvasási lépésnek ugyanúgy explicit input- és outputhatára van, mint bármely transzformációnak: ha a beolvasás a teljes táblát a memóriába teszi, akkor az a lépés egy rejtett, nagy erőforrásigényű köztes állapotot hoz létre.

Önellenőrzés. Mielőtt elhiszed, hogy egy lekérdezés skálázódik, tedd fel ezeket a kérdéseket: (1) A kód tényleg csak a szükséges oszlopokat kéri (`columns`, `select`, SQL-vetítés)? (2) A szűrés a forrásban történik (`WHERE`, `filters`, partíciós feltétel), vagy egy már betöltött DataFrame-en? (3) A motor streaming vagy chunk-alapú feldolgozást végez, vagy `SELECT *` után in-memory táblát hoz létre? (4) A memória-csúcsot mérem, vagy csak a fájlméretet nézem? Ha bármelyikre nemleges a válasz, előbb az egygépes adatfolyamot javítsd, és csak utána fontold meg a klasztert.

34.1.8 Rövid összefoglaló

A nagyobb adat első kérdése nem a klaszter, hanem a beolvasás mérete. A vetítés csak a szükséges oszlopokat olvassa be, a partícióvágás a feltétel szerint kihagyja a fájlok egy részét, a chunk és a streaming pedig darabonként tartja a memóriában az adatot, így a memóriaigény a sorok számától függetlenné válik. A spill biztonsági szelep, a sok apró fájl viszont új szűk keresztmetszetet hoz létre. A tömörített Parquet mérete nem RAM-igény; a memóriát a dekompresszált reprezentációból kell becsülni. A következő alkalmazás a 34.2. alfejezet Dask task graph és partíció: ott a chunkokból és partíciókból építünk lusta végrehajtási gráfot, és megnézzük, hogyan ütemeződik a munka több processzre.

34.2 Dask task graph és partíció

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Dask, Spark**
>
> Előfeltétel: 34.1. alfejezet · Célkészség: Dask task graph és partíció · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

34.2.1 Cél és konkrét helyzet

Egy elemző csapat a Northstar `orders.csv` állományán szeretné kiszámolni a teljes kosárösszeget és a szegmensenkénti összeget. Az állomány kicsi, de a csapat olyan számítást készít elő, amelyet később több fájlra, illetve RAM-nál nagyobb adatra is futtatni szeretne. A megfigyelési egység egy rendelés (`order_id`). A bemenet a `lab/data/orders.csv`; a cél annak megértése, hogy a Dask mikor tud partíción belül dolgozni, és mikor kell partíciók közötti adatmozgatás, azaz *shuffle*. Előzetes becslésként az összeg (`sum`) partíciónkénti részösszegekből előállítható, ezért költsége nagyjából a beolvasott sorok számával arányos; a csoportos összeg viszont kulcs szerinti átrendezést igényelhet.

Ez az alfejezet a bővítő tanulási úthoz tartozik. A labor alapértelmezett `pyproject.toml`-ja nem tartalmazza a Daskot, ezért ha a kódot futtatni szeretnéd, előbb külön opcionális környezetben telepítsd a `dask[dataframe]` extrát, és őrizd meg a lockot a 3.2. alfejezetben tanult módon. A példa helyi CPU-n, külső szolgáltatás nélkül futtatható. A könyvgenerálás ebben a módban nem futtat kódot, ezért a kimeneteket „elméletileg várt” jelzővel adjuk meg; a valódi ellenőrzés a helyi futás után a kontroll-assertben történik.

34.2.2 Lusta task graph és a partíció fogalma

A Dask gyűjtemények, például a Dask DataFrame, *lusta task graph*-ot (*lazy task graph*) építenek. A gráf csúcsai feladatok (*task*), élei pedig függőségek: egy task kimenete egy másik task bemenete. Az alábbi egyszerűsített gráf egy kétrészes `basket_minor` összegzést ír le:

```
read[csv, 0] --> partial_sum[0] --\
                                   --> total = add(partial_sum[0], partial_sum[1])
read[csv, 1] --> partial_sum[1] --/
```

Amíg nem hívod a `compute()` metódust, addig csak ez a gráf létezik. A 34.1. alfejezetben megismert chunk és projection itt is megjelenik: a CSV-blokkokból Dask-partíciók lesznek, és a vetített oszlopokon kívüli adatot jobb nem behúzni. A *partition* a DataFrame egymástól függetlenül feldolgozható szelete: minden partíció egy pandas DataFrame, amelyet egy-egy task olvas vagy állít elő. Partícióméretként hüvelykujjszabály szerint néhány tíz–néhány száz MiB szokott észszerű lenni, de ez a konkrét munkamenettől függ; a kis tanulófájlra kényszerből mesterséges `repartition(npartitions=4)` lépést alkalmazunk.

A gráfot a *scheduler* ütemezi. Helyi gépen az alapértelmezett szálalapú (*threaded*) scheduler egy folyamatban futtat taskokat; ez sok pandas–NumPy műveletnél működik, de a 33.2. alfejezetben tárgyalt GIL- és oversubscription-kérdés itt is érvényes. Nagyobb helyi munkához használható process-alapú vagy `distributed` scheduler, amely már külön munkásokat és diagnosztikai felületet ad. A `compute()` a gráf kiértékelésének és az eredmény pandas/NumPy objektummá alakításának pillanata; ekkor a közbülső gráf a memóriában tartott végeredmény után felszabadulhat.

A `persist()` ezzel szemben a gráfot kiértékeli, de a Dask-gyűjteményt megtartja a memóriában, hogy több későbbi `compute()` újrahasználhassa ugyanazt a közbülső eredményt. Ha ugyanazt a beolvasott és újrapartícionált DataFrame-et többféle aggregátumhoz használod, a `base = ddf.persist()` utáni számítások elkerülhetik a beolvasás ismétlését, cserébe viszont a base partíciói memóriát foglalnak.

34.2.3 Összegezhető és nem összegezhető aggregátumok

A partíciók közötti műveletköltség matematikailag azon múlik, hogy a célaggregátum előáll-e részeredmények kombinációjából. Legyen a teljes adathalmaz D felbontva P diszjunkt partícióra: $D = D_1 \sqcup \cdots \sqcup D_P$. Az x mérőszám esetén vezessük be a p-edik partícióra:

$$s_p = \sum_{x \in D_p} x, \qquad n_p = |D_p|, \qquad q_p = \sum_{x \in D_p} x^2.$$

Az x érték itt az EUR minor unit, tehát s_p és q_p mértékegysége rendre minor unit, illetve minor unit a négyzeten; n_p dimenzió nélküli darabszám. A teljes összeg, darabszám és átlag:

$$s = \sum_{p=1}^{P} s_p, \qquad n = \sum_{p=1}^{P} n_p, \qquad \bar{x} = \frac{s}{n}.$$

Az átlag tehát két összegezhető részaggregátum hányadosa. A szóráshoz az s_p, n_p, q_p hármas is elegendő:

$$S^2 = \frac{1}{n-1}\left(q - \frac{s^2}{n}\right),$$

ahol $q = \sum_p q_p$. A mintabeli varianciát S^2-tel jelöljük, hogy ne keveredjék a partíciós összeggel (s) és annak négyzetével (s^2). Numerikusan ez a képlet nagy értékeknél kivonási kioltásra lehet érzékeny; komoly mérési célra használj stabil Welford-féle frissítést, de a szerkezet akkor is additív marad.

Kézi kétpartíciós ellenőrzés: legyen $D_1 = [1, 10, 7]$ és $D_2 = [3, 4, 2]$. Ekkor $s_1 = 18$, $n_1 = 3$, $q_1 = 150$; $s_2 = 9$, $n_2 = 3$, $q_2 = 29$. A teljes összeg $s = 27$, a darabszám $n = 6$, a négyzetösszeg $q = 179$, ezért

$$S^2 = \frac{1}{n-1}\left(q - \frac{s^2}{n}\right) = \frac{1}{5}\left(179 - \frac{27^2}{6}\right) = \frac{1}{5}(179 - 121{,}5) = 11{,}5.$$

A közvetlen számítás ugyanezt adja: az átlag 4,5, az eltérések négyzetösszege 57,5, osztva $n-1=5$-tel szintén 11,5. A képlet tehát a partíciónkénti (s_p, n_p, q_p) hármasból helyesen állítja elő a teljes mintabeli varianciát.

A medián, kvantilis, módusz és a distinct count ezzel szemben nem összegezhető pusztán partíciónkénti részértékekből. Kézzel ellenőrizhető ellenpélda: legyen $D_1 = [1, 10, 7]$ és $D_2 = [3, 4, 2]$. Az összegek 18 és 9, a teljes összeg 27, a darabszám $3 + 3 = 6$, az átlag $27/6 = 4{,}5$. A partíciónkénti mediánok 7 és 3, ezek átlaga 5,0, a teljes rendezett sorozat $[1, 2, 3, 4, 7, 10]$ mediánja viszont $(3 + 4)/2 = 3{,}5$. A részmediánok átlaga tehát nem a teljes medián.

A Dask az ilyen aggregátumokat általában a kulcs vagy a rendezés szerinti *shuffle* segítségével tudja kiszámolni, vagy közelítő algoritmussal. A `sum`, `count`, `min`, `max` redukálható partíción belül és fölött is. A `mean` Dask szinten a `sum` és `count` hányadosa. A `median` default módon rendezést és partíciók közötti adatmozgatást igényelhet.

34.2.4 Teljes minimális munkapélda

A példa először a kis Northstar-fájlt olvassa be, mesterségesen négy partícióra osztja, majd pandas-kontrollal hasonlítja össze. A `df` és a `ddf` shape/típus-invariánsát is ellenőrizzük.

```
from pathlib import Path

import pandas as pd
import dask.dataframe as dd

ORDERS = Path("lab/data/orders.csv")

# Teljes, kis méretű kontrollbeolvasás.
pdf = pd.read_csv(ORDERS)

# Lusta Dask-gráf: blokkrészletes olvasás, majd 4 partíció.
# A blocksize csak szemléltető egység; a kis fájlon a Dask overhead számottevő.
ddf = dd.read_csv(ORDERS, blocksize=256_000).repartition(npartitions=4)
print("partíciók:", ddf.npartitions)  # elméletileg várt: 4

# Partíción belüli részösszeg és részcount, majd additív összegzés.
meta = pd.DataFrame({"partial_sum": pd.Series(dtype="int64"),
                     "partial_n": pd.Series(dtype="int64")})
partial = (
    ddf["basket_minor"]
    .map_partitions(
        lambda s: pd.DataFrame(
            {"partial_sum": [s.sum()], "partial_n": [s.count()]}
        ),
        meta=meta,
    )
    .compute()
)
print(partial.shape)  # elméletileg várt: (4, 2), minden partícióhoz egy sor

dask_sum = partial["partial_sum"].sum()
dask_count = partial["partial_n"].sum()
dask_mean = dask_sum / dask_count
print("sum:", dask_sum, "count:", dask_count, "mean:", dask_mean)
```

```
# Kézi/pandas kontroll, toleranciával.
assert abs(dask_sum - pdf["basket_minor"].sum()) < 1e-6
assert dask_count == len(pdf)
assert abs(dask_mean - pdf["basket_minor"].mean()) < 1e-6
print("pandas-kontroll: OK")

# Shuffle-t igénylő csoportos aggregátum.
by_segment = (
    ddf.groupby("segment")["basket_minor"]
    .agg(["sum", "count"])
    .compute()
    .sort_index()
)
print(by_segment)

ground = pdf.groupby("segment")["basket_minor"].agg(["sum", "count"])
pd.testing.assert_frame_equal(by_segment, ground, check_dtype=False)
print("csoportos kontroll: OK")
```

A `by_segment` soronként egy `segment` értéket ad, oszlopai `sum` és `count`. Az output pontos számai a helyi lab aktuális seedjéből adódnak; a kódot még nem ellenőrizte automatizált teszt, ezért itt nem közlünk „mért” táblázatot. A felhasználónak a `print` után a pandas-kontroll a tényleges ellenőrzés.

A `groupby("segment")` aggregációt a Dask partíción belüli részösszegekkel és ezek összefésülésével is elvégezheti; **nem szükségszerű a teljes sorszintű shuffle**. Általánosabb `groupby`-transzformációknál vagy bizonyos joinoknál viszont kulcs szerinti adatátrendezésre lehet szükség. Kis adaton az eredmény gyors lesz, de a művelet jellege élesen eltér a `map_partitions`-tól: az utóbbi garantáltan nem kever partíciók között.

34.2.5 Eredmény értelmezése

A `partial` DataFrame shape-je `(4, 2)`: a négy partíció mindegyike egy-egy `partial_sum` és `partial_n` sort adott. A `dask_sum` értéke a `basket_minor` oszlop teljes összege, a `dask_mean` ennek és a sorok számának hányadosa. A pandas-kontroll ellenőrzi, hogy a Dask partíciós redukció ugyanoda jut, mint az egyben beolvasott adat; toleranciát azért érdemes adni, mert bár a `basket_minor` egész minor unit, a későbbi osztás és esetleges float-dtype ilyen apró eltérést hozhat.

A csoportos tábla invariánsa: a `business` és `consumer` összegének külön-külön meg kell egyeznie a pandas `groupby` eredményével, és mindkét csoport `count` értéke pozitív. A `groupby` eredménye önmagában nem mutatja a shuffle tényét; a shuffle-t a task gráf belső szerkezete okozza. A `persist` ebben a példában csak akkor éri meg, ha a beolvasott `ddf`-re több külön `groupby` vagy `merge` is épül; itt egyszeri kontrollhoz felesleges.

34.2.5.1 Rendezés, dtype- és toleranciapolitika a pandas referenciához

A Dask-partíciók sorrendje és a `compute()` eredményének csoportsorrendje nem garantált. A referencia-összehasonlítás ezért három lépésből áll:

1. **Tételes rendezés:** a csoportkulcs szerint `sort_values(key)` és `reset_index(drop=True)`; az indexet nem használjuk azonosítóként.

2. **Típuspolitika:** a csoportkulcs és a darabszám egész (`int64`), az összeg egész minor unit (`int64`), az átlag `float64`. Ha a downcast szándékos, mindkét oldalt a deklarált dtype-ra kell hozni, és a tesztben `check_dtype=True`-val ellenőrizni; a `check_dtype=False` csak indokolt, dokumentált cast mellett használható.
3. **Tolerancia:** egész oszlopokra egzakt `==`; float oszlopokra `np.allclose(rtol=1e-9, atol=1e-9)` vagy a feladat mértékegységéhez igazított érték.

```
import numpy as np
import pandas as pd

def normalized(frame, key):
    return frame.sort_values(key).reset_index(drop=True)

def compare_partitioned(reference, result, key,
                        exact_cols, float_cols,
                        rtol=1e-9, atol=1e-9):
    reference = normalized(reference, key)
    result = normalized(result, key)
    pd.testing.assert_frame_equal(reference[[key]], result[[key]],
                                  check_dtype=True)
    for col in exact_cols:
        if not np.array_equal(reference[col].to_numpy(),
                              result[col].to_numpy()):
            raise AssertionError(f"egzakt eltérés: {col}")
        if reference[col].dtype != result[col].dtype:
            raise AssertionError(f"dtype eltérés: {col}")
    for col in float_cols:
        if not np.allclose(reference[col].to_numpy(),
                           result[col].to_numpy(), rtol=rtol, atol=atol):
            raise AssertionError(f"tolerancia túllépés: {col}")
```

A **reparticionálási invariancia** tesztje ugyanezt a `compare_partitioned` hívást futtatja `npartitions=1, 2, 4, 8` mellett. Ha a rendezett eredmény eltér, először a csoportkulcs típusát (string vs kategória), majd a `dropna`/`observed` beállítást, végül a float összegzés sorrendjét kell ellenőrizni. A `tests/test_dask_comparison.py` a sorrend-, dtype- és toleranciapolitikát teszteli; Dask nélkül a pandas-alapú összehasonlító is futtatható.

A Dask lazy gráf nem jelenti azt, hogy bármely transzformáció konstans memóriával értékelhető ki: shuffle és join esetén adatmozgatási költség keletkezhet.

34.2.6 Feladatok

1. **Értelmezés.** Az alábbi mutatók közül melyek állnak elő kizárólag partíción belüli részaggregátumokból: `count`, `mean`, `median`, `min`, `count(distinct product_id)`? Mindegyikhez indokold meg a választ.
2. **Megvalósítás.** Hozz létre egy hat soros pandas DataFrame-et `x = [1, 10, 7, 3, 4, 2]` értékekkel, majd `dd.from_pandas(..., npartitions=2)` segítségével két partíciót. Olvasd ki a két partíciót `to_delayed()` segítségével, számold ki a partíciónkénti összeget és mediánt, majd a teljes összeget és teljes mediánt. Ellenőrizd, hogy a részmediánok átlaga nem egyenlő a teljes mediánnal.
3. **Hibadiagnózis.** Egy kolléga egy 64 MiB-os CSV-t `blocksize=4096` értékkel olvas be, ezért több ezer apró partíciója lesz. A `sum()` nagyon lassú. Mi a hiba, és milyen `blocksize`, illetve `repartition(partition_size=...)` értékre cserélnéd? A válaszban ne állíts mért futásidőt, hanem a task overhead és a hasznos munka arányáról érvelj.

4. **Megváltozott feltétel.** A Northstar `order_items.csv` fájlból rendelésenkénti `line_total_minor` összeget akarsz készíteni, majd összekapcsolni az `orders` `basket_minor` oszlopával. Melyik lépés igényel shuffle-t: a rendelésenkénti összegzés, a join, vagy mindkettő? Írj Dask-vázlatot, amely az egyezőséget ellenőrzi, és indokold a sorazonosító szemcsézettségét a 7.2. alfejezetben tanult join-kardinalitással.

5. **Persist transzfer.** Az 1. feladatban szereplő felhasználó egymás után hívja meg ugyanazon `ddf`-re a `sum()` és a `count()` `compute()`-ot. Milyen ismétlődő munkát végez a scheduler? Hogyan kerülhető el `persist()` segítségével, és milyen memóriaköltség lép fel?

34.2.7 Megoldások és indoklás

1. A `count`, `mean`, `min` additív jellegű. A `mean` a `sum` és `count` hányadosa, ezért a két összegezhető részletből áll elő; a `min` a partíciónkénti minimumok minimuma. A `median` nem ilyen: rendezés kell, amint a 2. feladat mutatja. A `count(distinct product_id)` azért nem additív, mert ugyanaz a `product_id` több partícióban is előfordulhat, így a részleges egyedi számok összege felülszámolhat; globális egyedi halmaz kell.

2. Egy lehetséges megoldás:

```
import pandas as pd
import dask.dataframe as dd

pdf = pd.DataFrame({"x": [1, 10, 7, 3, 4, 2]})
ddf = dd.from_pandas(pdf, npartitions=2)

parts = ddf.to_delayed()
part_frames = [p.compute() for p in parts]
print([list(f["x"]) for f in part_frames])  # elméletileg [[1,10,7],[3,4,2]]

partial_sums = [f["x"].sum() for f in part_frames]
partial_medians = [f["x"].median() for f in part_frames]
print(partial_sums, partial_medians)  # [18, 9], [7.0, 3.0]
print(ddf.x.sum().compute(), ddf.x.median().compute())  # 27, 3.5
assert sum(partial_sums) == ddf.x.sum().compute()
assert sum(partial_medians) / 2 != ddf.x.median().compute()
```

A feladat kontrollja tehát kézzel is kijön: a teljes medián 3,5, a részmediánok átlaga 5,0.

3. A `blocksize=4096` túl sok apró partíciót hoz létre. Ha a teljes munka közelítőleg $T \approx Pt_{\text{task}} + Nt_{\text{row}}$, akkor nagy P mellett a task ütemezési, sorosítási és eredmény-összesítési költsége dominálhat a hasznos soronkénti munkához képest. A 64 MiB-os bemenethez például `blocksize=32_000_000` vagy `repartition(partition_size="32 MiB")` nagyságrend észszerű, ami két partíciót ad; többnyire a CPU-magok számához közeli partíciószám jó kiindulás. Pontos számot csak helyi mérés alapján szabad mondani.

4. A rendelésenkénti `line_total_minor` összegzés `groupby("order_id").sum()`, ami shuffle-t igényelhet, mert az azonos rendeléshez tartozó tételek akár külön partícióban vannak. A join szintén shuffle, hacsak mindkét oldal nincs azonos partíciós kulcs szerint indexelve. Egy ellenőrző vázlat:

```
items = dd.read_csv("lab/data/order_items.csv")
items_by_order = items.groupby("order_id")["line_total_minor"].sum().to_frame("items_total")

orders = dd.read_csv("lab/data/orders.csv")
recon = orders.merge(items_by_order, on="order_id", how="left").compute()

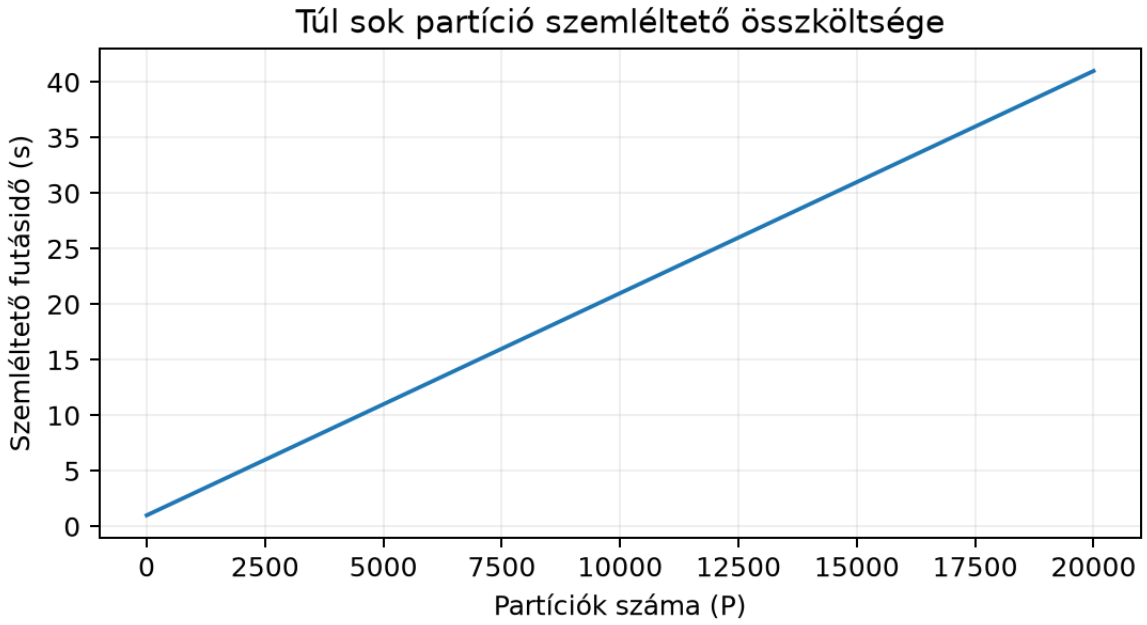
assert (recon["basket_minor"] == recon["items_total"]).all()
```

A `merge` után a sorazonosító továbbra is egy rendelés, nem tétel; ha véletlenül a tételfájl nyers sorait is összekapcsolnád `on="order_id"`, akkor one-to-many sorduplikáció lépne fel.

5. `persist()` nélkül a gráf a `compute()` hívások között megsemmisülhet, így a CSV-olvasás és a `repartition` mindkét aggregátumhoz újra lefut. A `sum()` és `count()` után a harmadik művelet ismét olvasna. A `base = ddf.persist()` kiértékeli a beolvasott, partícionált gyűjteményt, majd ebből a megtartott memóriabeli állapotból számolhat mindkét aggregátum; a költség a négy partíció memóriában tartása.

34.2.8 Gyakori tévedés: minél több partíció, annál gyorsabb

Az a feltételezés, hogy a nagyon finom particionálás mindig nagyobb párhuzamosságot és ezért gyorsabb futást ad, gyakran téves. Az alábbi ábra az előbbi egyszerű $T = Pt_{\text{task}} + Nt_{\text{row}}$ modellt mutatja $N = 1\,000\,000$ sorra, $t_{\text{task}} = 0{,}002$ s, $t_{\text{row}} = 10^{-6}$ s feltételezéssel; ezek szemléltető konstansok, nem mért Dask-értékek.



34.1. ábra: Túl sok partíció szemléltető összköltsége - Számított modell szemléltető konstansokkal, nem Dask-mérés.

A jelenség a 34.1. alfejezetben tárgyalt stream-elésből is következik: a túl kicsi chunk sokszoros feladatindítást okoz, miközben a tényleges hasznos munka változatlan. Az ideális partíciószám a gépen ténylegesen elérhető párhuzamossághoz igazodik, és a konkrét művelet shuffle-igényétől is függ. Csak annyi partíció kell, amennyi elég nagy ahhoz, hogy a task overhead eltörpüljön az adatfeldolgozási idő mellett.

34.2.9 Önellenőrzés

Tudod-e saját szavaddal megmondani, mikor lesz tényleges számítás a `.map_partitions`, a `.groupby` és a `.persist` után? El tudod-e választani, hogy az `mean` miért összegezhető, míg a `median` miért nem? Fel tudsz-e írni egy olyan műveletet, amely garantáltan nem mozgat adatot partíciók között, és egy olyat, amely tipikusan shuffle-t okoz? Ha a

`ddf.groupby("segment").sum().compute()` eredménye megegyezik a pandaséval, az nem jelenti azt, hogy a Dask nem végzett shuffle-t: a helyes kérdés az, hogy a scheduler milyen gráfot futtatott. A minőségi kapukat 30.2. alfejezet szerint érdemes a Dask-feldolgozás előtt futtatni, mert a lusta gráf attól még ugyanazt a hibás inputot aggregálja.

34.2.10 Rövid összefoglaló

A Dask lusta task gráfja a compute hívásáig csak terv: a partíciók független pandas-szeletek, a scheduler pedig taskokat ütemez. A compute kiértékeli a gráfot, a persist megtartja a közbülső állapotot a későbbi ágak számára. Az összeg, darabszám, átlag és pillanatnyi statisztikák partíciónkénti részeredményekből kombinálhatók; a pontos medián, kvantilis és egyediérték-számítás nem állítható elő pusztán egyetlen részösszeg és darabszám kombinálásával. A megvalósítás típustól és algoritmustól függően használhat rendezést, részhalmazokat, közelítő vázlatokat vagy adatmozgatást; **nem minden esetben szükséges shuffle**. A nyers shuffle költség a mozgatott adattól és a kulcseloszlástól függ, és kis adaton a sok apró task overheadje nagyobb lehet a hasznos munkánál. A következő 34.3. alfejezet a Spark DataFrame végrehajtási terv párhuzamos fogalmait tárgyalja.

34.3 Spark DataFrame és végrehajtási terv

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Dask, Spark**
>
> Előfeltétel: 34.2. alfejezet · Célkészség: Spark DataFrame és végrehajtási terv · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

34.3.1 Cél és konkrét helyzet

A 34.2. alfejezet Dask-példában már láttunk lusta feladatgráfot és partíciók közötti műveletet. Itt a Spark ugyanazt a kérdést más futásidejű szerződéssel oldd meg: egy vezérlő folyamat felépíti a végrehajtási tervet, a végrehajtók pedig partíciókonként dolgoznak. A tanulási helyzet nem valódi klaszter, hanem helyi, CPU-s mód, a cél az aggregálás tervezése és korrekt, kis előnézetű futtatása.

Bemenet: a Northstar `lab/data/orders.csv`, 2400 rendelés, minden sor egy megfigyelési egység, `order_id` kulccsal. A használt oszlopok: `priority_flag` (0/1, expressz jelzés), `basket_minor` (egész EUR-cent kosárérték), `quantity_sum`, `distance_km`. Cél: `priority_flag` szerint rendelésszám, kosárösszeg és átlagkosár. Elvárt ellenőrzés: a kimenet két csoportot tartalmaz, a két `basket_minor`-összeg összege a teljes fájl kosárösszegét adja, és a rendelésszámok összege 2400. Előzetes becslés: a kimenet 2 sor, 4 oszloppal; a futás helyi módban a 2400 soros fájlon gyors, de ennek mért idejét itt nem állítjuk.

34.3.2 Fogalmi magyarázat: driver, executor, terv, shuffle

A Spark-folyamatnak két fő szereplője van:

- **driver**: a vezérlő folyamat. Lefuttatja a programodat, minden lusta transzformációból felépíti a tervet, ütemezi a munkát, koordinálja a végrehajtókat, és az akciók kis eredményét visszagyűjti.

- **executor**: a végrehajtó folyamat. Partíciókat tárol, oszloparányosan olvas, számol, és a shuffle során adatot küld más végrehajtóknak.

A Spark **DataFrame** elosztott, oszlopos, lusta reláció. Ugyanúgy van sémája, sora és oszlopa, mint egy pandas vagy Polars táblának, de a sorok partíciókra osztva élnek az executorokon. **Transzformáció** (transformation) minden olyan művelet, amely új DataFrame-et ad vissza, de nem indít azonnal számítást: `select`, `filter`, `withColumn`, `groupBy`, `agg`, `join`, `orderBy` stb. **Akció** (action) az, ami tényleges számítást indít, és eredményt hoz vissza a driverre vagy külső tárolóba ír: `count`, `collect`, `show`, `take`, `write`.

Az `explain` ezért nem futtató parancs: kiíratja a strukturált tervet, hogy az áthelyezések előtt átgondolhasd az adatmozgást. Például:

```
HashAggregate(keys=[priority_flag], functions=[finalmerge_sum, finalmerge_count])
+- Exchange hashpartitioning(priority_flag, 200)
   +- HashAggregate(keys=[priority_flag], functions=[partial_sum, partial_count])
      +- FileScan csv [...] [priority_flag,basket_minor]
```

A tervet alulról felfelé olvasom: először a CSV beolvasása, utána partíción belüli részleges összegzés, majd **shuffle**, végül a kulcs szerinti végleges összegzés.

A **partition** a párhuzamosság fizikai egysége: az adat egy darabja, amelyet egy executor-mag dolgoz fel. A **shuffle** partíciók közötti adatáthelyezés kulcs szerint. Az aggregálásnál a Spark hash particionálást használ: minden `(priority_flag)` kulcs a hash alapján kiválasztott partícióba kerül, hogy minden csoport egyetlen helyen legyen. A shuffle drága, mert lemezre és hálózatra is kerülhet adat, ezért a Spark először lokális részösszegeket számít.

A **collect limit** a biztonságos előnézet mintája: a `.limit(2).show()` vagy `.take(2)` azt mondja, hogy a driverre legfeljebb két sor érkezzen. A teljes nagy táblát `.collect()`-elni tilos, mert a driver memóriájába gyűjtöd a teljes elosztott adatot, ami tipikus OOM-hiba.

A relációs aggregáció matematikája ugyanaz, mint 7.1. alfejezetben. Legyen P_k a k-adik partíció sorhalmaza, x_i a `basket_minor` érték az i. rendelésben. Az aggregálás előtti részösszeg:

$$S_k = \sum_{i \in P_k} x_i, \qquad N_k = \sum_{i \in P_k} 1,$$

ahol S_k a partíció kosárösszege EUR-centben, N_k a rendelésszám a partíción. A shuffle kulcs szerint összegyűjti a részösszegeket, a végleges csoportösszeg:

$$S_g = \sum_{k} S_{k,g}, \qquad N_g = \sum_{k} N_{k,g},$$

ahol g a csoportkulcs (0 vagy 1), $S_{k,g}$ és $N_{k,g}$ a k partícióban a g csoporthoz tartozó részösszeg. Az átlag csak ezután osztás:

$$\bar{x}_g = \frac{S_g}{N_g} \quad \text{EUR-cent/rendelés, ha } N_g > 0.$$

Kézi kontroll három soron: `[(0, 500), (1, 700), (0, 300)]`. Ekkor két partícióra bontva például az elsőben `(0, 500), (1, 700)`, a másodikban `(0, 300)`: az egyes csoportra $S_{0,1} = 500$, $N_{0,1} = 1$, $S_{0,2} = 300$, $N_{0,2} = 1$, így $S_0 = 800$, $N_0 = 2$, $\bar{x}_0 = 400$; az egyes csoportra $S_1 = 700$, $\bar{x}_1 = 700$. Az invariáns $S_0 + S_1 = 1500$, ami a teljes összeg.

34.3.3 Minimális munkapélda

Platform: Linux, munkakönyvtár `~/northstar-lab`, külön bővítő Python-környezet, mert PySpark nincs a kezdő `lab/pyproject.toml` alapfüggőségei között. A pontos telepített verziót a `pip show pyspark` paranccsal ellenőrizd; az alábbi kódrészlet a PySpark 3.5.x API-ját szemlélteti. Ennek a kódrészletnek a futtatását e könyvfejezet generálásakor nem végeztük el: ahol kimenetet írunk, az elméletileg várt vagy a tervből következő szemléltetés, nem mért futási jegyzék.

```
# Linux, ~/northstar-lab
python -m venv .spark-venv
source .spark-venv/bin/activate
python -m pip install pyspark  # a pontos verzió rögzítését a projekt lockjában végezd el

from pyspark.sql import SparkSession, functions as F

spark = (
    SparkSession.builder
    .appName("s135-region-aggregation")
    .master("local[2]")          # helyi, 2 szálú mód csak tanulásra
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")

# Explicit séma: a Northstar orders.csv teljes adatszerződése.
orders = spark.read.schema("""
    customer_id string,
    order_id string,
    order_ts string,
    promised_days int,
    quantity_sum int,
    item_rows int,
    basket_minor long,
    shipping_fee_minor long,
    currency string,
    distance_km double,
    priority_flag int,
    delivery_days int,
    actual_delivery_at string,
    label_available_at string,
    target_late int
""").option("header", True).csv("lab/data/orders.csv")

# Lusta transzformációk: semmi nem számolódik itt.
grouped = (
    orders
    .groupBy("priority_flag")
    .agg(
        F.count("*").alias("order_count"),
        F.sum("basket_minor").alias("basket_sum_minor"),
        F.avg("basket_minor").alias("avg_basket_minor"),
    )
)

# Nem futtat, csak kiíratja a fizikai tervet.
grouped.explain("formatted")

# Biztonságos előnézet: a driverre legfeljebb két sor kerül.
preview = grouped.orderBy("priority_flag").limit(2)
preview.show()

# Helytelen minta nagy adatra; itt 2400 sorral még elfutna, de a szokást ne építsd be:
# all_rows = orders.collect()
```

```
spark.stop()
```

A `.groupBy().agg()` három új oszlopot ad: `order_count` hosszú egész, `basket_sum_minor` hosszú egész, `avg_basket_minor` dupla lebegőpontos. A `grouped.explain("formatted")` nem indít munkát; a tényleges számítást a `preview.show()` akciója indítja el. A `local[2]` mester két executor-szálat enged ugyanabban a JVM-ben, ezért a shuffle nem valódi hálózati út, hanem folyamaton belüli adatáthelyezés.

34.3.4 Eredmény értelmezése

A várt kimeneti séma: `priority_flag int`, `order_count long`, `basket_sum_minor long`, `avg_basket_minor double`; sorok száma legfeljebb kettő, mert ennyi különböző csoport van. A `show()` nem garantálja a rendezést, ezért a determinista előnézethez `orderBy("priority_flag")`-et adtunk.

A fizikai terv elvárt kulcsai: `FileScan csv` a beolvasáshoz; részleges `HashAggregate` csoportonként; `Exchange hashpartitioning(priority_flag)` a shuffle-höz; végleges `HashAggregate` a csoportösszegek összefésüléséhez. A `limit(2)` a tervben `CollectLimit` vagy `TakeOrderedAndProject` jellegű lépésként jelenhet meg, és garantálja, hogy a driverre csak két sor érkezik.

Ellenőrző invariánsok:

1. $\sum_g \text{order_count}_g = 2400$, mert minden rendelés pontosan egy csoportban van.
2. $\sum_g \text{basket_sum_minor}_g = \sum_i \text{basket_minor}_i$, egész centben pontosan, ennek minden adatsorban nullának kell lennie.
3. A súlyozott átlag az egész táblára:

$$\bar{x}_{\text{össz}} = \frac{\sum_g N_g \bar{x}_g}{\sum_g N_g},$$

   ahol N_g a `order_count`, $\bar{x}_g$ az `avg_basket_minor`. Lebegőpontos osztás miatt a két oldal közötti eltérésnek 10^{-6} cent/rendelés relatív tűrésen belül kell lennie.

Mivel az `avg_basket_minor` lebegőpontos, az összegző ellenőrzést ne abból, hanem a `basket_sum_minor`-ból számold: a `sum * count` szorzat összege nem feltétlenül adja vissza az egész összeget kerekítési hiba nélkül. A pontos, egész centes ellenőrzés mindig a `basket_sum_minor` oszlopból származzon.

A Spark `explain` logikai és fizikai tervet mutat; a tényleges futási terv és a shuffle mennyisége eltérhet az előzetes becsléstől.

34.3.4.1 Funkcionális tervellenőrzés a tervszöveg helyett

A `FileScan csv`, `HashAggregate` és `Exchange hashpartitioning` nevek **verzió-, optimalizáló- és fájlformátum-függők**: a Catalyst, a Photon, az adaptív végrehajtás (AQE) vagy egy Parquet-forrás más operátorneveket és más sorrendet adhat. Ezért a teszt ne egyetlen tervszöveget hasonlítson szó szerint, hanem két dolgot ellenőrizzen:

1. **Funkcionális aggregáció:** a csoportok halmaza pontosan `{0, 1}`, a `count` összege 2400, a `basket_sum_minor` összege a teljes fájl összege, a súlyozott átlag a teljes átlagot adja vissza, és a dtype-ok a vártak.

2. **Indokolt shufflekorlát:** az `Exchange` operátorok száma legfeljebb 1, nincs `CartesianProduct` vagy `BroadcastNestedLoopJoin`, és a shuffle bájtok aránya az inputhoz képest a küszöb alatt marad (kis aggregációnál például 2×).

```
def plan_checks(plan_text, shuffle_bytes=None, input_bytes=None,
                max_shuffle_ratio=2.0):
    errors = []
    if "CartesianProduct" in plan_text:
        errors.append("CartesianProduct a tervben")
    if "BroadcastNestedLoopJoin" in plan_text:
        errors.append("BroadcastNestedLoopJoin a tervben")
    if plan_text.count("Exchange") > 1:
        errors.append("több mint egy Exchange")
    if shuffle_bytes is not None and input_bytes:
        ratio = shuffle_bytes / input_bytes
        if ratio > max_shuffle_ratio:
            errors.append(f"shuffle arány túl nagy: {ratio:.2f}")
    return errors

def metric_checks(groups, total_rows, total_basket):
    errors = []
    keys = {row["priority_flag"] for row in groups}
    if keys != {0, 1}:
        errors.append(f"csoportkulcsok: {keys}")
    if sum(row["order_count"] for row in groups) != total_rows:
        errors.append("order_count összeg eltér")
    if sum(row["basket_sum_minor"] for row in groups) != total_basket:
        errors.append("basket_sum_minor összeg eltér")
    weighted = sum(row["order_count"] * row["avg_basket_minor"]
                   for row in groups) / total_rows
    if abs(weighted - total_basket / total_rows) > 1e-6:
        errors.append("súlyozott átlag eltér")
    return errors
```

A `tests/test_plan_contract.py` két különböző verziójú, eltérő szövegű tervet fogad el, ha a funkcionális metrikák és a shufflekorlát teljesülnek; a két `Exchange`-t, a derékszögű szorzatot és a túl nagy shuffle-arányt elutasítja. A terv szöveges egyeztetése helyett ez a viselkedés rögzíti a szerződést.

34.3.5 Feladatok

1. **Tervértelmezés.** A fenti `grouped.explain("formatted")` kimenetében azonosítsd a részleges aggregálást, a shuffle-t és a végleges aggregálást! Melyik lépés igényel partíciók közötti kulcs szerinti adatáthelyezést, és miért van szükség rá?

2. **Megvalósítás.** Írj PySpark local feladatot, amely a `priority_flag` szerint számol `quantity_sum`-összeget és `distance_km`-átlagot a `lab/data/orders.csv`-ből, és `take(2)`-vel jeleníti meg az eredményt! Ne olvasd be a teljes táblát a driverre! Add meg a várt shape-et és dtype-ot!

3. **Hibadiagnózis.** Egy kolléga nagy, több millió soros táblán ezt futtatja:

   ```
   result = orders.groupBy("priority_flag").count().collect()
   ```

   Mi a hiba? Miért nem jelent megoldást, ha az adat kicsi, ezért nálad még lefutott? Adj biztonságos előnézetet!

4. **Megváltozott feltétel.** A `orders.csv`-t 32 Parquet-fájlra bontották object storage-on. Hogyan változik a beolvasási és aggregálási terv? Mely részek olcsóbbak vagy drágábbak? Miért nem elég a helyi gépen mért futásidő a klaszteres várható idő becsléséhez?

5. **Erőforrásbecslés.** A 33.3. alfejezetben tanult szorzatszabályhoz hasonlóan gondold át: ha egy aggregálás 10 millió soron, 4 kulcsoszlop, összesen 1000 csoport mellett 200 partícióra shuffle-öl, milyen lépésekben nőhet a költség a 2400 soros helyi próbához képest, és miért nem arányos pusztán a sorok számával?

34.3.6 Megoldások és indoklás

1. megoldás. A `FileScan` beolvassa a CSV-partíciókat. A részleges `HashAggregate` minden partíción külön-külön számítja a csoportonkénti részösszeget és darabszámot. Az `Exchange hashpartitioning(priority_flag)` a shuffle: minden részsor a `priority_flag` hash-alapú partíciójába kerül, hogy az azonos kulcsú részösszegek ugyanazon a helyen találkozzanak. A végleges `HashAggregate` a részösszegekből csoportösszeget és átlagot készít. A shuffle azért kell, mert egy csoport kulcsa több bemeneti partícióban is előfordulhat.

2. megoldás. Egy lehetséges változat:

```
orders = spark.read.schema("""
    customer_id string, order_id string, order_ts string,
    promised_days int, quantity_sum int, item_rows int,
    basket_minor long, shipping_fee_minor long, currency string,
    distance_km double, priority_flag int, delivery_days int,
    actual_delivery_at string, label_available_at string, target_late int
""").option("header", True).csv("lab/data/orders.csv")

summary = (
    orders
    .groupBy("priority_flag")
    .agg(
        F.sum("quantity_sum").alias("total_quantity"),
        F.avg("distance_km").alias("avg_distance_km"),
    )
)

print(summary.take(2))  # legfeljebb két sor; az egész tábla nem kerül a driverre
spark.stop()
```

Várt shape: `(2, 3)`, oszlopok: `priority_flag int`, `total_quantity long`, `avg_distance_km double`. Ellenőrzés: a két `total_quantity` összege a teljes `quantity_sum` összege; a `avg_distance_km` mértékegysége km/rendelés.

3. megoldás. A `.collect()` a teljes csoportosított eredményt a driver memóriájába teszi. Kis adaton ez eltűnik, ezért a hiba nem jelentkezik, de a művelet szemantikája nagy adatra is ugyanaz: az elosztott eredményt egyetlen gép memóriájába gyűjti. A kolléga általában csak meg akarja nézni a táblát, tehát `result.limit(5).show()` vagy `take(5)` a helyes. A `count()`-ból amúgy is csak két sor jönne, tehát ebben a konkrét példában kisebb a veszély, mint az egész `orders` táblára hívott `collect()`-nél; a gondolatmenet akkor is ugyanaz: előbb a művelet kimenetének méretét, aztán a driverre kerülő sorok számát kell becsülni.

4. megoldás. Parquet-fájloknál a beolvasás oszloposan és predikátum-leszúrással hatékonyabb lehet, a CSV-nél nem kell sémát fejteni. A 32 fájl akár 32 vagy több bemeneti partícióra bomlik, ezért a részleges aggregálás párhuzamosabb. A shuffle viszont akkor is adatmozgást jelent, ha a fájlok object storage-on vannak: a kulcsok szerinti áthelyezést a hálózat és az executorok közötti kapcsolat korlátozza. A helyi gépen minden egy JVM-ben fut, ezért nincs hálózati késés,

nincs storage API-hívás, és a memória is más. A klaszteres időt ugyanazon hardveren, méreten, partíciószámon, ismétléssel kell mérni; a helyi próba csak a terv helyességét ellenőrzi.

5. megoldás. A költség a beolvasott bájtokban, a csoportkulcsok szerializációjában, a shuffle partíciók számában, a hálózatban és a végleges aggregáció memóriaigényében nőhet. A 10 millió soros esetben a shuffle-re kerülő részösszegek mérete a csoportok és partíciók szorzatával nőhet, nem lineárisan a bemenettel. A 200 partíció túl kevés lehet, ha az executorok száma nagy, túl sok, ha kicsi; a becsléshez a 33.3. alfejezet erőforrás-szorzóját és a 22.3. alfejezet méretkorlát-gondolkodását kell alkalmazni.

34.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés: *„A local módban mért 2,1 másodperc, tehát a klaszteren is körülbelül ennyi lesz."* A local mód egyetlen JVM-ben futtatja a drivert és az executorokat; a shuffle nem hálózati, hanem folyamaton belüli másolás, az object storage-hívások helyett helyi fájl van, és a partíciók nem mennek át sorosításon ugyanúgy. Ezért a kis helyi próba csak a terv szemantikáját és az eredmény invariánsait ellenőrzi, nem a klaszteres teljesítményt. A 34.2. alfejezet Dask-tapasztalat is ugyanezt mutatta: a scheduler és a partíciók közötti művelet költsége csak a tényleges végrehajtó környezetben értelmezhető. A korrekt méréshez gép, adatméret, partíciószám, ismétlésszám és hibahatár kell, ahogyan a 22.3. alfejezetben a futásidő-állításoknál már rögzítettük. Önellenőrzés: ha a Spark-feladat csak `.show()`-t hív, akkor még nem mérted a teljes aggregációt; ha `.collect()`-et hívsz, akkor a drivert méred, nem a klasztert.

34.3.8 Rövid összefoglaló

A Spark DataFrame lusta, oszlopos, partíciókra osztott reláció. A transzformációk tervet építenek, az akciók indítanak számítást. Az `explain` a tervet mutatja, a `groupBy().agg()` részleges aggregálást, kulcs szerinti shuffle-t és végleges aggregálást hoz létre. A részösszeg-képlet invariánsa: a csoportösszegek összege a teljes összeg. Nagy adatot nem gyűjtünk a driverre; `take`, `limit`, `show` a biztonságos előnézet. A local módos próba a helyességet ellenőrizheti, de nem klaszteres teljesítménymérés. A következő alkalmazás 34.4. alfejezet: elosztott kísérletek futtatása és az adatelhelyezés tervezése, ahol ezt a tervszemléletet kell adatparticionálással összekötni.

34.4 Elosztott kísérlet és adatelhelyezés

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: Dask, Spark

Előfeltétel: 34.3. alfejezet · Célkészség: Elosztott kísérlet és adatelhelyezés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

34.4.1 Cél és konkrét helyzet

Egy csapat a Northstar-rendelésekből havi aggregált feature-táblát épít. A nyers Parquet három skálán létezik: 100 MB szűkített mart, 10 GB rendelési esemény, és 1 TB nyers kattintási napló plusz teljes rendelési előzmény. A kérdés: mikor érdemes Dask/Spark klasztert indítani, és mikor elég a helyi DuckDB vagy Polars? A döntést a számítás, az adat, a hálózat és a hozzáférési

jogosultság közös költségmodellje adja, nem a divat. Az alfejezet célja, hogy döntési lapot készíts három adatskálához, konkrét helyi alternatívával és a klaszter indokoltságának feltételével.

Az előzetes eredménybecslés: 100 MB helyi gépen másodpercek, 10 GB percek, 1 TB órák lehet; a klaszter csak akkor győz, ha az adat már a számítás közelében, object store-ban van, és a feldolgozás párhuzamosítható. Ez a becslés nagyságrendi, nem mérés.

34.4.2 Fogalmi magyarázat

Object store. Az adat nem a munkagép helyi fájlrendszerén van, hanem hálózati tárolóban, például S3-kompatibilis object store-ban. Az objektumok statikus fájlok, HTTP-n olvashatók. A Dask/Spark worker feladatai a partíciókat a saját memóriájukba töltik. Ez megosztást és replikációt ad, de hálózati késleltetést és hívási díjat is. Egy 1 TB-os adat 1000 darab 1 GB-os partícióban több ezer hálózati kérés, de a párhuzamos letöltés ezt részben elfedi.

Network. A hálózati átvitel első közelítésben

$$t_{net} = \frac{S}{B},$$

ahol S az adatmennyiség bájtban, B a sávszélesség bájt/másodpercben. Például $S = 1$ GB $= 10^9$ B, $B = 25$ MB/s $= 25 \cdot 10^6$ B/s esetén $t_{net} = 40$ s. Sok kis fájlnál a kérésenkénti késleltetés dominál, ezért a partícióméretet a sávszélességhez kell igazítani. Az elosztott feldolgozásnál a hálózatot kétszer is terheli: az adat beolvasása és a shuffle köztes adat átvitele.

Serialization (sorbarendezés). A driver és az executor/worker közötti objektumokat bájtsorozattá kell alakítani. A Python pickle-t használ, a Spark saját sorosítót. A nagy DataFrame kollekciója a driverre költséges; csak aggregált eredményt gyűjtsünk. A méret becsülhető: n sor $\times$ oszloponkénti bájt. Egy 1 millió soros, 10 oszlopos lebegőpontos tábla 8 bájt/oszlop mellett 80 MB.

Data locality (adatelhelyezés). A számítás akkor gyors, ha a kód az adat mellett fut, nem pedig az adat utazik. A lokális DuckDB az SSD-n lévő adatot olvassa; a klaszter workerje a hálózatról tölti le. Ha az adat már az object store-ban van, és a worker ugyanabban a régióban fut, a párhuzamosság részben kompenzálja a távolságot. A partíciók elhelyezése tehát döntési tényező.

Failure (meghibásodás). A klaszter egy-egy feladata elbukhat: node kiesik, időtúllépés, lemez tele. A Spark lineage újraszámolja az elveszett partíciót; a Dask is újraütemez. Az újraütemezés csak akkor működik, ha a feladat idempotens: azonos bemenet és kimeneti útvonal azonos eredményt ad. A hosszú shuffle-t érdemes checkpointolni, különben egy késői hiba a munka nagy részét újraszámolja.

Reproducibility. A klaszteres futtatás reprodukálásához rögzíteni kell a kód verzióját, a környezetet, a bemeneti adat partícióit és hashét, a seedet, a shuffle partíciók számát és a klaszter konfigurációját. A lebegőpontos összegzés sorrendje változhat a partíciók ütemezésétől, ezért a numerikus ellenőrzés tűréshatáros, nem bitazonos – ez a 33.4. alfejezet equivalence check kiterjesztése.

34.4.3 Minimális munkapélda: döntési lap három adatskálához

Az alábbi kód szemléltető paraméterekkel becsüli a helyi és a klaszteres feldolgozási időt. A paraméterek illusztratívak; teljesítménymméréshez valódi hardver, adatméret, ismétlés és tényleges futás kell.

```
# s136_dontesi_lap.py
# Illusztratív költségmodell; az értékek nem mért teljesítményadatok.

scales_mb = {"100 MB": 100e6, "10 GB": 10e9, "1 TB": 1e12}
local_rate = 80e6          # bájt/mp helyi DuckDB/Polars feldolgozás (illusztratív)
upload_rate = 25e6         # bájt/mp hálózati feltöltés (200 Mbps, illusztratív)
cluster_startup = 300      # mp klaszter üzembe helyezése és felszabadítása
cluster_workers = 8
cluster_per_worker_rate = 40e6  # bájt/mp object store-ból olvasó worker (illusztratív)
cluster_rate = cluster_workers * cluster_per_worker_rate

def becsles(size, data_in_object_store=False):
    local_t = size / local_rate
    upload_t = 0.0 if data_in_object_store else size / upload_rate
    cluster_process_t = size / cluster_rate
    cluster_t = cluster_startup + upload_t + cluster_process_t
    return local_t, cluster_t

for name, size in scales_mb.items():
    local_t_upload, cluster_t_upload = becsles(size, data_in_object_store=False)
    local_t_remote, cluster_t_remote = becsles(size, data_in_object_store=True)
    print(f"{name}: ")
    print(f"  adat lokálisan:   helyi {local_t_upload:.0f}s, klaszter {cluster_t_upload:.0f}s")
    print(f"  adat object store: helyi {local_t_remote:.0f}s, klaszter {cluster_t_remote:.0f}s")
```

Várható kimenet a fenti paraméterekből számítva (nem mérés):

```
100 MB:
  adat lokálisan:   helyi 1s, klaszter 305s
  adat object store: helyi 1s, klaszter 300s
10 GB:
  adat lokálisan:   helyi 125s, klaszter 731s
  adat object store: helyi 125s, klaszter 331s
1 TB:
  adat lokálisan:   helyi 12500s, klaszter 43125s
  adat object store: helyi 12500s, klaszter 3425s
```

Kézi ellenőrzés: 10 GB lokálisan $10^{10}/8 \cdot 10^{7} = 125$ s; klaszter feltöltéssel $300 + 10^{10}/2{,}5 \cdot 10^{7} + 10^{10}/3{,}2 \cdot 10^{8} \approx 300 + 400 + 31 = 731$ s. Az 1 TB feltöltése $4 \cdot 10^{4}$ s, ezért a klaszter csak akkor győz, ha az adat már az object store-ban van.

Ez a döntési lap mutatja, hogy a klaszter indokoltsága erősen függ attól, hol lakik az adat, és hány ismétlés van. A 10 GB-os adat helyi feldolgozása 2 perc, ami elfogadható; a klaszter startupja 5 perc, tehát nem indokolt. Az 1 TB-os adat helyben órákig fut, ezért ha már object store-ban van, a klaszter 57 percet ígér, de ha fel kell tölteni, a feltöltés 11 óra, ami a klasztert nem teszi előnyössé.

34.4.4 A döntési lap egy lehetséges formája

Három adatskálához konkrét helyi alternatívát és klaszterfeltételt rögzítek:

Adatméret	Helyi alternatíva	Klaszter indokoltsága
100 MB	DuckDB a laptopon, Parquet közvetlen olvasás	nem indokolt, kivéve csoportos bemutató
10 GB	DuckDB vagy Polars streaming, erős munkaállomás	akkor indokolt, ha az adat már object store-ban és az elemzés ismétlődő
1 TB	Polars lazy + DuckDB, vagy Spark/Dask, ha a memória nem elég	indokolt, ha az adat már object store-ban, a feldolgozás párhuzamosítható, és a hibakezelési keret megtérül

A táblázatot a kód számszerű becslései támasztják alá. A feltétel pontosabban: a klaszter akkor indokolt, ha (1) az adat már a compute-tal azonos régióban lévő object store-ban van; (2) a feldolgozási feladat partíciónként független vagy kevés shuffle-t igényel; (3) a várható klaszterfutás legalább egy nagyságrenddel rövidebb a helyinél; (4) a fejlesztői és üzemeltetési idő rendelkezésre áll. Ha bármelyik hiányzik, a helyi DuckDB/Polars az alapértelmezés.

34.4.5 Eredmény értelmezése

A becslés nem mérés, hanem döntési keret. A helyi DuckDB azért gyors kis adaton, mert nincs hálózati átvitel és indítási overhead. A klaszter előnye a párhuzamosságból jön, de a startup és adatfeltöltés fix költsége a kis és közepes adatnál nagyobb, mint a megtakarítás. Az 1 TB-os esetben a feltöltés nélküli klaszter 3,6-szor gyorsabb, ami már indokolhatja a komplexitást; a feltöltéssel viszont lassabb, mert a hálózati sávszélesség szűk keresztmetszet. Ez a konkrét számpélda is illusztrálja a közös költségmodell elvet: a döntés a teljes láncra vonatkozik, nem egyetlen erőforrásra.

Az elosztott futás költségét az adatmozgatással, inicializálással és klaszterrezsivel együtt kell mérni, nem csak a számítási fázis idejével.

34.4.5.1 Mérési lap három adatskálán és bevezetési no-go szabály

A döntéshez mindhárom méretthez ugyanazokat a mezőket kell kitölteni; a „becslés” és a „mért” érték külön oszlopban szerepeljen. A mérési lap minimális mezői:

Mező	Egység	Helyi mérés	Klaszter mérés
`data_bytes`	bájt	fájlméret + partíciók	input + shuffle + output bájt
`input_location`	enum	`local_ssd`, `network_mount`, `object_store`	ugyanaz
`cpu_s`	s	medián 3 futásból	medián 3 futásból
`wall_s`	s	medián 3 futásból	medián 3 futásból
`peak_rss_mb`	MiB	`resource.getrusage`	workerenként + driver
`startup_s`	s	0	klaszter indítás + felszabadítás
`upload_bytes`, `upload_s`	bájt, s	0	adatfeltöltés, ha kell

Mező	Egység	Helyi mérés	Klaszter mérés
`shuffle_bytes`, `shuffle_s`	bájt, s	0	Spark/Dask metric
`cost`	pénzegység	gépóra × ár	klaszteróra × ár + tárolás
`environment`	szöveg/JSON	CPU, RAM, verziók	worker-típus, régió, verziók

A **no-go szabály** előre rögzített; ha bármelyik feltétel sérül, a klaszter nem indul:

1. Az adat elfér a helyi gép RAM-jában, és a helyi medián futás rövidebb, mint a klaszter `startup_s`-ja.
2. Az adat nincs object store-ban, és az `upload_s` nagyobb, mint a helyi `wall_s`.
3. A `shuffle_bytes / input_bytes > 2`, vagy a tervben egynél több `Exchange` van.
4. A klaszter `wall_s`-ja nem éri el a helyi `wall_s` felét (nincs legalább 2× nyereség).
5. A klaszter `cost`-ja meghaladja a jóváhagyott keretet, vagy a teljes költség (startup + tárolás) nagyobb, mint a helyi alternatíváé.

```
def cluster_go(m):
    reasons = []
    if m["fits_local_ram"] and m["local_wall_s"] < m["startup_s"]:
        reasons.append("helyi futás rövidebb a startupnál")
    if not m["in_object_store"] and m["upload_s"] > m["local_wall_s"]:
        reasons.append("feltöltés lassabb a helyi futásnál")
    if m["input_bytes"] and m["shuffle_bytes"] / m["input_bytes"] > 2:
        reasons.append("shuffle arány > 2")
    if m["cluster_wall_s"] > 0.5 * m["local_wall_s"]:
        reasons.append("nincs 2x nyereség")
    if m["cluster_cost"] > m["budget"]:
        reasons.append("költségkeret túllépés")
    return {"go": not reasons, "no_go_reasons": reasons}
```

A három adatskálán a döntési lap a fenti mezőkkel töltendő ki; a `tests/test_cluster_decision.py` a kis adat no-go-ját, az object store-os nagy adat go-ját, a túl nagy shuffle-t és a költségkeret-túllépést teszteli. Az elosztott rendszer tehát nem automatikusan gyorsabb: a no-go szabály kimondja, hogy költség vagy startup túlsúlya esetén a helyi DuckDB/Polars marad.

34.4.6 Feladatok

1. Értelmezés. Magyarázd el saját szóval, miért nem elég az adatméret a klaszter melletti döntéshez. Nevezd meg a költségmodell legalább négy elemét, és mondd meg, melyik hiánya okozhat túl korai klaszterválasztást.

2. Implementálás. Módosítsd a fenti kódot úgy, hogy az ismétlésszámot is figyelembe vegye: ha egy elemzést N alkalommal futtatnak, a klaszter startupja amortizálódik. Számítsd ki $N = 10$ ismétléssel a 10 GB-os eset döntését, és adj meg egy olyan N küszöböt, ahol a klaszter (adat object store-ban) gyorsabb a helyinél.

3. Hibadiagnózis. Egy Dask-kísérletben a rendeléseket (orders) és a tételeket (order_items) szeretnénk order_id szerint joinolni. Az orders partíciói customer_id szerint, az order_items partíciói line_id szerint vannak; a join 1 óráig fut. Mi a hiba, és milyen javítás csökkenti a shuffle-t? Adj konkrét műveletet.

4. Megváltozott feltétel. Tegyük fel, hogy az 1 TB-os adat már az object store-ban van, de a hálózati olvasási sebesség a worker és a store között 10 MB/s-ra csökken, miközben a helyi gép változatlan. Számold ki az új klaszteres időt, és döntsd el, hogy a klaszter továbbra is indokolt-e. Milyen adatelhelyezési döntést javasolnál?

34.4.7 Megoldások és indoklás

1. Az adatméret önmagában nem mondja meg, hol lakik az adat, mennyi a hálózati sávszélesség, mekkora az indítási overhead, hány ismétlés van, és milyen a feladat párhuzamossága. A költségmodell elemei: adatelhelyezés (adat a gépen vagy object store-ban), hálózati átvitel (feltöltés/letöltés), számítási kapacitás (magok, memória), indítási és üzemeltetési idő, valamint a hibakezelés és reprodukálhatóság többletköltsége. Túl korai klasztervalasztást okoz, ha például az adat feltöltési idejét elfelejtik: 1 TB feltöltése 25 MB/s mellett 11 óra, ami önmagában hosszabb, mint a helyi feldolgozás 3,5 órája.

2. Módosított kód vázlat:

```
def becsles_n(size, n_runs, data_in_object_store=False):
    local_t_run = size / local_rate
    upload_t = 0.0 if data_in_object_store else size / upload_rate
    cluster_t_run = size / cluster_rate
    return n_runs * local_t_run, cluster_startup + n_runs * (upload_t + cluster_t_run)

# 10 GB, N=10, object store-ban:
print(becsles_n(10e9, 10, data_in_object_store=True))
# Várható: (1250.0, 612.5)  -> klaszter 612s, gyorsabb
```

10 GB, $N = 10$, object store-ban: helyi $10 \cdot 125 = 1250$ s, klaszter $300 + 10 \cdot 31 = 610$ s, tehát a klaszter gyorsabb. A küszöb: $N \cdot 125 > 300 + N \cdot 31$, azaz $N > 300/94 \approx 3{,}2$, tehát 4 vagy több futtatásnál a klaszter (object store-ban) gyorsabb. Az ellenőrzés: $N = 4$: helyi 500 s, klaszter 424 s.

3. A hiba a nem egyező particionáló kulcs. A join order_id szerint történik, de az egyik tábla customer_id, a másik line_id szerint particionált. A Dask/Spark emiatt mindkét táblát újraosztja order_id szerint, ami teljes shuffle-t okoz. Javítás: `orders = orders.shuffle("order_id")` és `order_items = order_items.shuffle("order_id")`, majd a join; vagy ha az egyik tábla kicsi, broadcast join (`dask.dataframe.merge` a `broadcast=True` opcióval). A particionálás utáni kvantitatív ellenőrzés: a shuffle előtti és utáni partíciók számát és a futásidőt összehasonlítani, ügyelve arra, hogy a numerikus eredmény változatlan maradjon (equivalence check tűréshatárral, 33.4. alfejezet).

4. Az új klaszteres olvasási idő 10 MB/s mellett: $10^{12}/(8 \cdot 10^7) = 12500$ s (mert 8 worker $\times$ 10 MB/s = 80 MB/s), plusz startup 300 s, összesen 12800 s. Ez gyakorlatilag megegyezik a helyi 12500 s-mal, tehát a klaszter már nem indokolt. Javaslat: az adat és a compute ugyanabban a hálózati zónában legyen; ha ez nem lehetséges, helyi gépre vagy nagyobb sávszélességű kapcsolatra van szükség. Az adatelhelyezés döntése tehát elsődleges a puszta klaszterkapacitás előtt.

34.4.8 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a felhő vagy klaszter automatikusan olcsóbb, gyorsabb vagy egyszerűbb. Ellenpélda a 100 MB-os mart táblánál: a helyi DuckDB egyetlen másodperc alatt kiszámol egy csoportosítást, míg a klaszter indítása 5 perc, és a kód átvitele, a függőségek telepítése és a hibakeresés további órákat vehet igénybe. A klaszter itt nem gyorsabb, csak drágább és bonyolultabb. Ugyanez a helyzet a 10 GB-os adatnál, ha az elemzést egyszeri

alkalommal futtatják. A klaszter akkor nyer, ha az adat már object store-ban van, a feladat párhuzamosítható, és a futtatás többször ismétlődik. Ezt a feltételt a döntési lap expliciten tartalmazza.

34.4.9 Önellenőrzés

- Sorold fel a költségmodell hat elemét.
- Számold ki: 500 MB adat, 50 MB/s helyi feldolgozás, 300 s startup, 8 worker 25 MB/s sebességgel: érdemes-e klasztert indítani, ha az adat lokális? Válasz: helyi 10 s, klaszter legalább 300 s + feltöltés; nem érdemes.
- Miért nem bitazonos a lebegőpontos összegzés a klaszterben? Mert a partíciók ütemezése változtathatja az összegzési sorrendet, a lebegőpontos összeadás pedig nem asszociatív; ezért relatív tűréshatáros equivalence checket használunk.

34.4.10 Rövid összefoglaló

Az elosztott kísérlet ugyanazon tudományos fegyelmet követi, mint a helyi, de a költségmodell bővül: az object store, a hálózat, a sorbarendezés, az adatelhelyezés, a meghibásodás és a reprodukálhatóság együtt döntik el, hogy a klaszter indokolt-e. A döntési lap három adatskálán mutatja, hogy a helyi DuckDB/Polars kis és közepes adaton gyorsabb, a klaszter pedig akkor nyer, ha az adat már a compute közelében van, a feladat párhuzamos, és az indítási költség amortizálódik. A felhő nem automatikusan jobb; a választást becsült vagy mért teljes láncra kell alapozni.

35 Batch predikció, kis szolgáltatás és kutatói demó

35.1 Batch inference és idempotens output

Belépő · szint: haladó · idő: 90 perc · eszköz: FastAPI, Docker

Előfeltétel: 34.4. alfejezet · Célkészség: Batch inference és idempotens output · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

35.1.1 Cél és konkrét helyzet

A Northstar-csapat a korábban jóváhagyott modellt, a `northstar-late-v3-2025-08-01` azonosítójú champion pipeline-t, új szintetikus rendeléseken szeretné futtatni batch jelleggel. A kérdés: a rendelés pillanatában mennyi a valószínűsége annak, hogy a kézbesítés a 7 napos ígért határidőn túl érkezik. A bemenet egy CSV, amely rendelésenként csak a rendelés pillanatában ismert mezőket tartalmazza: `order_id`, `order_ts`, `promised_days`, `quantity_sum`, `item_rows`, `basket_minor`, `shipping_fee_minor`, `currency`, `distance_km`, `priority_flag`, `account_age_days`, `segment`, `region` és `acquisition_channel`. A output legyen sorazonosítóval, modellazonosítóval, modelldátummal, a késés becsült valószínűségével és a 0,5 küszöbhöz tartozó osztállyal. Önálló manifestnek kell igazolnia, hogy mely input adatverzióból, mely modellből, milyen sémaverzióval, hány sorra és hány chunkban készült a predikció. Előzetes elvárás: a kimenet sorainak száma egyezzen a bemenet rendeléseinek számával; a késés valószínűsége 0 és 1 közé essen; a késések aránya a modell tanítási osztályaránya közelében, nagyságrendileg 20% körül mozogjon. Ez számszerű cél, nem garantált futási eredmény.

35.1.2 A batch predikció mint szerződés

A **batch** (kötegelt) predikció nem rendelésenkénti, valós idejű szolgáltatás, hanem egyszerre több sor kiértékelése egy későbbi időpontban. A batch feladat attól még nem történelmi visszatekintés: a predikció pillanata továbbra is az `order_ts`. A scoring input minden sora rendelésenkénti megfigyelési egység, és minden prediktorértéknek olyannak kell lennie, amit a rendszer az adott rendelés időpontjában már ismert. Ez a **feature availability** követelmény.

A **schema validation** a 21.4. alfejezet predikciós szerződésének batch-változata: a fontos lépés nem a modell újratanítása, hanem annak ellenőrzése, hogy az új adat ugyanazt a szerződést teljesíti. Ellenőrizendő: a feature-nevek, a dtype-ok, a hiányzó oszlopok, a hiányzó értékek és a kategorikus változók megengedett értékei. A példánkban a modell a következő gyakorlati `X` mátrixot várja: n rendelés és $p = 9$ oszlop, ahol a numerikus mezők `distance_km` (km, float64, ≥ 0), `priority_flag` (0 vagy 1), `basket_minor` (EUR-cent, int64), `quantity_sum` (darab, pozitív egész), `account_age_days` (nap, int64), a kategorikus mezők pedig `segment`, `region` és `acquisition_channel`. A modell `predict_proba` kimenete mindig $(n, 2)$ alakú mátrix; az oszlopok $[\hat{p}_0, \hat{p}_1]$ soronként 1-re összegződnek, ahol $\hat{p}_1$ a késés becsült valószínűsége. A döntési osztály: 1, ha $\hat{p}_1 \geq 0{,}5$, különben 0.

A **chunk** a 34.1. alfejezet adatfolyam-fogalmának alkalmazása itt: a nagy CSV-t nem egyszerre kell a memóriában tartani, hanem a pandas `chunksize` paraméterével darabokban. Minden chunknak át kell esnie ugyanazon a séma-ellenőrzésen, mert a chunkolás önmagában nem őrzi meg a dtype-ot egy hibás CSV-ben; a második chunkban megváltozó tartalom eltérő dtype-ot eredményezhet. A chunkok sorrendjét és a predikciós számítást megőrizzük.

Az **output manifest** a 31.1. alfejezet kísérleti futásmanifest fogalmának batch-szállítási változata. Az identitás most a bemeneti adatverzióból, a modellartifact hashekből és a sémaverzióból áll, ezért tartozik az output manifest az output mellé. Az output manifest nem tartalmazza a jövőbeli címkéket, azaz a `delivery_days`, `actual_delivery_at`, `label_available_at` és `target_late` mezőket meg sem próbáljuk becsülni. A manifest sem hitelesíti a modell minőségét: a modellkártya határai, például a régiók támogatott köre, a batch futástól függetlenül érvényesek.

35.1.2.1 Az idempotens output kulcsa és immutabilitása

A batch output elsődleges kulcsa:

```
(order_id, prediction_as_of, model_version)
```

- `order_id`: a megfigyelési egység.
- `prediction_as_of`: a predikció érvényességi időpontja, jelen esetben a rendelés `order_ts`-a; nem a futtatás `created_at` időbélyege. Ugyanarra a rendelésre későbbi időpontban, frissített modellel készülő predikció új `prediction_as_of` értéket kaphat.
- `model_version`: a konkrét, immutábilis modellverzió, nem a `champion` alias.

A tábla **immutábilis**: egy kulcshoz pontosan egy sor tartozik, és a sor a kiírás után nem módosítható. Ismételt futás ugyanarra a kulcsra a meglévő sort adja vissza (vagy `INSERT ... ON CONFLICT DO NOTHING`), nem ír második sort. Új modellverzió vagy új `prediction_as_of` **új kulcs**, tehát új sor; a régi sor megmarad az auditnak. Ha egy korábbi predikciót javítani kell, az nem felülírás, hanem új rekord `supersedes_key` és `correction_reason` mezővel.

```
def prediction_key(row):
    return (row["order_id"], row["prediction_as_of"], row["model_version"])

def assert_unique_keys(rows):
    keys = [prediction_key(row) for row in rows]
    if len(keys) != len(set(keys)):
        raise ValueError("duplikált (order_id, prediction_as_of, model_version) kulcs")
    return True
```

A manifestnek rögzítenie kell a kulcsmezőket, a sorok számát és a `duplicated_keys` ellenőrzés eredményét (0). Az invariáns: ugyanazon run kétszer futtatva egy sor per kulcs keletkezik; másik modellverzió új kulcsot és új sort ad, a régi érintetlen marad. A `tests/test_batch_idempotency.py` a két futás egyedi kulcsait, a modellverzió-váltás új sorát és a `prediction_as_of` különbségét teszteli.

35.1.3 Teljes minimális munkapélda

A példa egy helyi, CPU-s CLI, amely a `lab` projektben fut. A mentett pipeline-t `joblib` betölti; ez a saját projectből származó artefaktum, nem ismeretlen pickle. A kód determinisztikus, és a CSV-kimenet hash-e a manifestben megismételhetővé válik.

Először a scoring input CSV, amely az `orders.csv` szerződéséből csak a rendeléskor ismert mezőket tartalmazza. Hat sor elegendő a chunkolás bemutatásához:

```
# artifacts/new_orders_scoring_input.csv (szemléltető részlet)
# order_id,order_ts,promised_days,quantity_sum,item_rows,
# basket_minor,shipping_fee_minor,currency,distance_km,
# priority_flag,account_age_days,segment,region,acquisition_channel
# ord-9001,2025-05-02T08:12:00+00:00,7,2,1,2490,490,EUR,14.6,1,123,consumer,north,search
# ord-9002,2025-05-02T09:03:00+00:00,7,1,1,990,290,EUR,42.1,0,88,business,south,direct
# ord-9003,2025-05-03T13:40:00+00:00,7,3,2,3110,590,EUR,8.7,1,200,consumer,central,referral
# ord-9004,2025-05-03T15:22:00+00:00,7,4,1,5410,990,EUR,27.4,1,17,consumer,north,direct
# ord-9005,2025-05-04T10:01:00+00:00,7,2,2,1775,390,EUR,91.0,0,304,business,central,search
# ord-9006,2025-05-04T11:47:00+00:00,7,1,1,1290,290,EUR,63.8,0,151,consumer,south,referral
```

A betöltött modell a `model_candidate_v3.joblib` fájl. A modelldátum a jóváhagyott modell nyilvántartásából származó érték, nem a batch futás napja. A modellt a 31.4. alfejezet szerint a registry hash-évl kell ellenőrizni; a minimális példában a manifest az artifact aktuális hash-ét rögzíti. A CLI:

```
# lab/batch_predict.py
import argparse
import hashlib
import json
import sys
from datetime import datetime, timezone
from pathlib import Path

import joblib
import pandas as pd

FEATURE_COLUMNS = [
    "distance_km", "priority_flag", "basket_minor",
    "quantity_sum", "account_age_days",
    "segment", "region", "acquisition_channel",
]

FORBIDDEN_COLUMNS = {
    "delivery_days", "actual_delivery_at",
    "label_available_at", "target_late",
}

CATEGORY_RULES = {
    "segment": {"consumer", "business"},
    "region": {"north", "central", "south"},
    "acquisition_channel": {"search", "referral", "direct"},
}

DTYPE_RULES = {
    "distance_km": "float64",
    "priority_flag": "int64",
    "basket_minor": "int64",
    "quantity_sum": "int64",
    "account_age_days": "int64",
    "segment": "object",
    "region": "object",
    "acquisition_channel": "object",
}


def sha256_file(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda: f.read(8192), b""):
            h.update(block)
    return h.hexdigest()
```

```
def validate_schema(df: pd.DataFrame) -> list[str]:
    errors = []
    missing = [c for c in FEATURE_COLUMNS if c not in df.columns]
    if missing:
        errors.append(f"missing feature columns: {missing}")
        return errors
    forbidden = [c for c in FORBIDDEN_COLUMNS if c in df.columns]
    if forbidden:
        errors.append(
            f"forbidden future-only columns present: {forbidden}"
        )
    for col, expected_dtype in DTYPE_RULES.items():
        if df[col].dtype != expected_dtype:
            errors.append(
                f"{col}: expected dtype {expected_dtype}, got {df[col].dtype}"
            )
    for col, allowed in CATEGORY_RULES.items():
        unknown = set(df[col].astype("string").dropna()) - allowed
        if unknown:
            errors.append(f"{col}: unknown categories {sorted(unknown)}")
    if df[FEATURE_COLUMNS].isna().any().any():
        errors.append("missing values in feature columns")
    return errors


def main(argv=None):
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, required=True)
    parser.add_argument("--model", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--chunk-size", type=int, default=10_000)
    parser.add_argument("--model-id", default="northstar-late-v3-2025-08-01")
    parser.add_argument("--schema-version", default="northstar-v3")
    args = parser.parse_args(argv)

    model = joblib.load(args.model)
    model_datum = "2025-08-01"  # a regisztrált modell jóváhagyási dátuma

    input_hash = sha256_file(args.input)
    model_hash = sha256_file(args.model)
    total_rows = 0
    chunk_count = 0

    reader = pd.read_csv(args.input, chunksize=args.chunk_size)
    for chunk in reader:
        chunk_count += 1
        errors = validate_schema(chunk)
        if errors:
            raise SystemExit(
                f"chunk {chunk_count} failed schema validation: {errors}"
            )
        X = chunk[FEATURE_COLUMNS]
        proba = model.predict_proba(X)[:, 1]
        out_chunk = pd.DataFrame({
            "order_id": chunk["order_id"],
            "model_id": args.model_id,
            "model_datum": model_datum,
            "predicted_probability_late": proba,
            "predicted_class": (proba >= 0.5).astype("int64"),
        })
        out_chunk.to_csv(
            args.output,
```

```
            mode="w" if chunk_count == 1 else "a",
            header=(chunk_count == 1),
            index=False,
        )
        total_rows += len(chunk)

    output_hash = sha256_file(args.output)
    output_rows = pd.read_csv(args.output).shape[0]
    manifest = {
        "input_file": str(args.input),
        "input_sha256": input_hash,
        "model_file": str(args.model),
        "model_sha256": model_hash,
        "model_id": args.model_id,
        "model_datum": model_datum,
        "schema_version": args.schema_version,
        "chunk_count": chunk_count,
        "input_record_count": total_rows,
        "output_record_count": output_rows,
        "output_sha256": output_hash,
        "created_at_utc": datetime.now(timezone.utc).isoformat(),
        "python": sys.version.split()[0],
    }
    manifest_path = args.output.with_suffix(".manifest.json")
    manifest_path.write_text(
        json.dumps(manifest, indent=2, ensure_ascii=False) + "\n"
    )
    print(
        f"rows={total_rows} chunks={chunk_count} "
        f"output={args.output} manifest={manifest_path}"
    )


if __name__ == "__main__":
    main()
```

A futtatási parancs Linux/macOS terminálból, a `lab` munkakönyvtárból:

```
cd lab
python batch_predict.py \
  --input artifacts/new_orders_scoring_input.csv \
  --model artifacts/model_candidate_v3.joblib \
  --output artifacts/new_orders_scoring_output.csv \
  --chunk-size 2
```

A `--chunk-size 2` csak demonstráció; éles futásban a méretet memóriaprofilonként érdemes megválasztani. A szemléltető, automatikusan még nem ellenőrzött kimenet: `rows=6 chunks=3 output=artifacts/new_orders_scoring_output.csv manifest=artifacts/new_orders_scoring_output.`

A kimeneti CSV a következő oszlopokkal rendelkezik, a sorok a bemenet sorrendjében jelennek meg:

order_id	model_id	model_datum	predicted_probability_late	predicted_class
ord-9001	northstar-late-v3-2025-08-01	2025-08-01	0,247	0
ord-9002	northstar-late-v3-2025-08-01	2025-08-01	0,612	1
ord-9003	northstar-late-v3-2025-08-01	2025-08-01	0,189	0

order_id	model_id	model_datum	predicted_probability_late	predicted_class
ord-9004	northstar-late-v3-2025-08-01	2025-08-01	0,531	1
ord-9005	northstar-late-v3-2025-08-01	2025-08-01	0,333	0
ord-9006	northstar-late-v3-2025-08-01	2025-08-01	0,408	0

A manifest legfontosabb mezői a következő ellenőrzési láncot adják. Az `input_sha256` az adatverzió; a `model_sha256` az artifact; a `model_id` és `model_datum` a regisztrált modell; az `output_sha256` a kimenet tartalmi ellenőrzésére szolgál. A `created_at_utc` nem az output identitásának része, hanem a futás auditrekordja. A manifestet táblázatban összefoglalva:

mező	jelentés	mire jó az ellenőrzéskor
input_sha256	a scoring input teljes fájl SHA-256 hash-e	adatverzió azonosítás
model_sha256	a betöltött `.joblib` fájl hash-e	artifact-változás észlelése
model_id / model_datum	regisztrált modellazonosító és jóváhagyási dátum	a champion verziója
output_sha256	a kimeneti CSV hash-e	ismételt futás tartalmi egyezése

35.1.4 Eredmény értelmezése

A kimenet nem tartalmazhat valós jövőbeli címkét, ezért a pontosság itt nem mérhető. Az ellenőrzés ehelyett szerződést ellenőriz: a sorok száma, a valószínűségek tartománya, a dtype-ok és a manifest mezői. A minimális példában az invariánsok:

- `output_record_count == input_record_count == 6`;
- `0 <= predicted_probability_late <= 1` minden sorban;
- `predicted_class` pontosan akkor 1, ha a valószínűség legalább 0,5;
- a kimenet oszlopai nem tartalmaznak tiltott future-only mezőt;
- a modell és a séma verziója megegyezik a futtatási paranccsal.

Az idempotens output azt jelenti, hogy ugyanazzal a bemenettel, modellel, modelldátummal, sémaverzióval és chunkonkénti sorrenddel a kimeneti CSV tartalma azonos, tehát a hash-e azonos. A `created_at_utc` kivétel, mert az az audit időpontja. Ha a folyamat a második chunk közepén megszakad, az output félkész; a helyreállítás az egész parancs újrafuttatása, mert a kód az első chunknál felülírja a fájlt. A termelési szintű CLI ezt egy ideiglenes fájlba írással és atomi átnevezéssel teszi robusztusabbá, mint a 32.4. alfejezet recovery-elvének batch-változata.

Fontos érvényességi korlát: a manifest a futás azonosságát bizonyítja, de nem bizonyítja, hogy a modell továbbra is jól működik. Ha az új rendelések eloszlása eltér a tanítási ablaktól, a hash és a séma-kontroll önmagában nem jelzi; ahhoz a 36.1. alfejezet monitorozási folyamata szükséges. A Northstar-példa szintetikus, ezért a táblázat számai szemléltetőek, nem mért eredmények.

Az idempotens batch outputhoz stabil bemeneti kulcs, a futásverzió ellenőrzése és atomikus publikálás szükséges; az azonos fájlnév önmagában nem elegendő.

35.1.5 Feladatok

1. **Értelmezés.** A batch manifest szerint `input_sha256=9f8c...`, `model_sha256=a441...`, `schema_version=northstar-v3`, `output_record_count=2400`. Melyik állítás igaz? a) Ezzel a manifesttel bizonyítottuk, hogy a modell a 2400 rendelés 20%-ában késést jósolt. b) A manifest csak azt bizonyítja, hogy a kimenet ebből a bemenetből, ezzel a modellel, ezzel a sémaverzióval készült, és a kimenet sortartalma a hash-sel újra ellenőrizhető. c) Az `output_record_count` mindig az input sorainak kétszerese, mert a modell pozitív és negatív osztályra is számol. Indokold a választ.

2. **Megvalósítás.** Írj egy `check_batch_contract(input_manifest: dict, output_path: Path) -> None` függvényt, amely a manifestből kiolvassa az `input_sha256` és `output_sha256` értékeket, majd ellenőrzi, hogy az input- és outputfájl aktuális hash-e egyezik a manifesttel. Eltérés esetén `ValueError`-t dobjon. A fájl elérési útja a manifest `input_file` és a kimeneti CSV adott kulcsából jön. Add meg a hiányzó fájl kezelését is.

3. **Hibakeresés.** Egy batch futás az első chunknál hibátlan, a másodiknál a következő üzenetet adja: `ValueError: X has 9 features, but LogisticRegression is expecting 10 features as input.` A fejlesztő megnézi a CSV-t, és azt mondja, hogy a fájl elején minden rendben van. Mi lehet a hiba? Hogyan helyes a parancssor diagnosztikája? Mit kell megváltoztatni a hívó kódban, hogy a hiba már a chunk betöltésekor egyértelmű legyen?

4. **Megváltozott feltétel.** Az input CSV-ben egy új régióérték jelenik meg: `west`. A `validate_schema` az ismeretlen kategóriát hibaként jelzi. Hogyan kezeled? Indokold, miért nem elég a modell `handle_unknown="ignore"` beállítása a további futtatáshoz, és milyen modellnyilvántartási lépések következnek.

35.1.6 Megoldások és indoklás

1. A helyes válasz b). Az `output_record_count` nem metrika; a 2400 sor kimeneti sort jelez, nem a késések számát. A manifest a láncolat azonosságát igazolja, nem a modell pontosságát. A c) hamis, mert a predikció kimenete soronként egy valószínűség, nem két sor. Az a) hamis, mert címke nélkül nem tudjuk, hány sor késik; a manifest akkor sem mondja meg, ha a modell osztályt is ad.

2. Lehetséges megoldás:

```
import hashlib
from pathlib import Path


def sha256_file(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda: f.read(8192), b""):
            h.update(block)
    return h.hexdigest()


def check_batch_contract(manifest: dict, output_path: Path) -> None:
    input_path = Path(manifest["input_file"])
    output_path = Path(output_path)
    if not input_path.exists():
        raise FileNotFoundError(f"input file missing: {input_path}")
    if not output_path.exists():
        raise FileNotFoundError(f"output file missing: {output_path}")
```

```
if not Path(manifest["model_file"]).exists():
    raise FileNotFoundError(f"model file missing: {manifest['model_file']}")
if sha256_file(input_path) != manifest["input_sha256"]:
    raise ValueError("input_sha256 mismatch")
if sha256_file(output_path) != manifest["output_sha256"]:
    raise ValueError("output_sha256 mismatch")
```

Az ellenőrzés akkor hasznos, ha a manifestet a futás végén ugyanaz a környezet hozta létre; az útvonalak gépek között eltérhetnek, ilyenkor a relatív projektgyökérre kell normalizálni.

3. A `ValueError: X has 9 features...` üzenet a scikit-learn modell elvárásából jön, nem a CSV-sor számából. A fejlesztő a fájl elejét látja jónak, de a második chunkban a pandas a CSV egyik oszlopát eltérő néven olvassa be, vagy egy oszlopnév eltolódik, például egy sorban a `region` oszlop neve `region;` lett. A `FEATURE_COLUMNS` kiválasztás így nem az elvárt 10 oszlopot adja, a validátor `missing feature columns` hibát generál, de ha a hívó nem szakítja meg a futást, a modell betanított 10 feature-elvárása 9 bemenő oszlopnál a fenti hibát adja. A diagnózis: a chunk `columns.tolist()` kiírása, a CSV középső sorainak megnézése, adott esetben a `read_csv`-nek a fejléc sorát rögzítő paraméter. A javítás a hívó oldalon: a `validate_schema` hiba esetén `raise SystemExit` azonnal, ne csak naplózzon. A chunkonkénti séma-ellenőrzés pontosan ezt a hibát fogja meg emberi félreértés nélkül.

4. A `west` régió esetén a helyes döntés a batch futtatás felfüggesztése, nem a kategória csendes figyelmen kívül hagyása. A `handle_unknown="ignore"` a 21.4. alfejezet tanítása szerint nullákkal kódolja az ismeretlen értéket, tehát a modell képes predikciót adni; a modellkártya azonban a `north`, `central`, `south` értékeket támogatja. A `west` megjelenése szerződésváltozás: előbb ellenőrizni kell, hogy az új régió a valós adatfelvételben létezik-e, majd a modell újratanítása és új modellkártya, új modelldátum szükséges. A batch kimeneten nem szabad úgy túllépni, hogy az alacsony bizalomjelzés nélkül jelenik meg. Ez a 26.4. alfejezet alkalmazott robusztussági elve: a modellkártyán kívül eső tartományban a futtatást meg kell állítani, nem automatikusan kiterjeszteni.

35.1.7 Gyakori tévedés ellenpéldával

Gyakori tévedés: az új scoring CSV-ben a célmező is szerepel, például a `target_late` oszlop, mert az elemző a historikus `orders.csv`-ból nyert tesztet adta át, és a modell csak nem válogatja ki. A dimensionality ellenőrzés ezt nem feltétlenül jelzi, ha a modell a feature-listából dolgozik; a batch output ugyanazzal a shape-vel elkészül, ezért a hiba láthatatlan marad. Az ellenpélda: egy 6 soros input CSV `target_late` oszloppal a `validate_schema` tiltott-oszlop ellenőrzése nélkül hibátlanul futna; az output manifest hash-e is egyezne, de a folyamat megsértette a predikció pillanatának szerződését. Ha ezt a folyamatot a későbbi éles scoring előtt a rendszer nem vizsgálja, a modell olyan adaton tanulhat, amely a rendelés pillanatában nem állt rendelkezésre; a batch CLI-nek ezért a `FORBIDDEN_COLUMNS` halmaz explicit, nem a feature-válogatásra bízott. A célmező jelenléte általában arra utal, hogy rossz tárterületről érkezett az input.

35.1.8 Önellenőrzés

1. Milyen adat lehet a prediktor része a Northstar batch scoringjában? Milyen oszlopok tiltottak, és miért?
2. Ha az output CSV hash-e azonos két futásban, de a manifest `created_at_utc` értéke eltér, idempotens-e a batch folyamat? Indokold.
3. Mi a különbség a manifest `input_sha256` és `output_sha256` szerepe között?

A rövid ellenőrző válaszok: 1. Rendeléskor ismert mezők; a `delivery_days` és társai azért tiltottak, mert a predikció pillanatában nem ismertek. 2. Igen, tartalmi értelemben idempotens; a manifest időbélyeg audit adat, nem predikciós tartalom. 3. Az input hash az adatverziót, az output hash a kimenet tartalmi egyezését ellenőrzi.

35.1.9 Rövid összefoglaló

A batch inference akkor megbízható, ha a bemenetet adatverzióhoz, a modellt modelldátumhoz kötjük, és a kimenetet manifesttel zárjuk. A sémavalidáció soronként ellenőrzi a feature-neveket, dtype-okat és kategóriákat; a tiltott jövőbeli céloszlopok jelenléte azonnali hiba. A chunkolás a memóriát korlátozza, de a séma-ellenőrzést minden chunkra futtatni kell. Az idempotens kimenet azonos bemenet, modell és séma mellett azonos CSV-hash-t ad; a manifest időbélyeg csak audit. A következő alkalmazás a 35.2. alfejezet FastAPI-végpont, ahol ugyanez a szerződés rendelésenkénti, kis késleltetésű szolgáltatássá válik.

35.2 FastAPI végpont és ellenőrzött bemenet

Belépő · szint: haladó · idő: 90 perc · eszköz: FastAPI, Docker

Előfeltétel: 35.1. alfejezet · Célkészség: FastAPI végpont és ellenőrzött bemenet · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

35.2.1 Cél és konkrét helyzet

A Northstar késői kézbesítés prediktor eddig batch feldolgozásban dolgozott: a 35.1. alfejezetben adatverzióhoz és modellhez kötött, idempotens batch outputot állítottunk elő. Most ugyanazt a predikciós szerződést online kérés-válasz formában tesszük elérhetővé. A szolgáltatás a rendelés pillanatában ismert jellemzőket fogadja, és visszaadja a modell által becsült $P(\text{late} = 1 \mid x)$ valószínűséget — azaz annak a modell szerinti becslését, hogy a kézbesítés 7 napnál későbbi lesz.

Megfigyelési egység: egy rendelés. A bemenet címkéje kizárt mezőket nem tartalmazhat: `delivery_days`, `actual_delivery_at`, `label_available_at` vagy bármi, amit a rendelés pillanatában nem ismernénk. A kimenet a prediktív valószínűség, a modell verziója és egy futásazonosító.

A konkrét bemenet, amelyet a fejezet további része használ:

```
{"order_id":"ORD-1","basket_minor":3500,"quantity_sum":2,
 "distance_km":12.5,"priority_flag":0,"account_age_days":42,
 "segment":"consumer","region":"central","acquisition_channel":"search"}
```

Előzetes becslés: ez az input a dummy modell logisztikus regressziójának kimenete, tehát 0 és 1 közötti valószínűség, nem konfidenciaintervallum. Helyes ellenőrzés helyi loopbacken: hiányzó mezőre `422`, érvényes kérésre `200`, a predikció 0–1 közé esik, ugyanaz a bemenet ugyanazt a valószínűséget adja akkor is, ha a `prediction_id` minden hívásnál új.

35.2.2 Fogalmi magyarázat: validation, startup load, request, response, error, latency, concurrency

A FastAPI a **request validation** (bemeneti séma-ellenőrzés) feladatot Pydantic modellekre bízza. A modell mezőnevei, típusai és korlátjai együtt jelentik a predikciós szerződést. A nyers JSON-ból a Pydantic a megadott típusú Python-objektumot állítja elő; ha a mező hiányzik, nem konvertálható a várt típusra vagy a korláton kívül esik, a keretrendszer rendszerint `422 Unprocessable Entity` státusszal, a hibás mezőt listázó választ ad. A Pydantic `Field` korlátai az érvényes tartományt rögzítik: `basket_minor` nem lehet negatív, `quantity_sum` legalább 1, `distance_km` nem lehet negatív, `priority_flag` csak 0 vagy 1, a kategóriák pedig a szerződésben felsorolt értékek.

A szolgáltatás indításakor a **startup load** tölti be a modellt. A modellt nem kérésenként olvassuk újra: a startup hiba esetén a folyamat ne induljon el. A FastAPI modern ajánlott eszköze a `lifespan` kontextuskezelő; a korábbi `@app.on_event("startup")` stílus elavult. A betöltés után a modellt az `app.state` objektumban tároljuk.

A **request** és **response** sémája külön Pydantic osztály. A kérés a nyers feature-ket tartalmazza az elemzési szemcsézettségnek megfelelően; a válasz nem a modell belső számábrázolását, hanem a döntési cél számára értelmezhető adatot. A válasz valószínűségi mezőjének shape-ja akkor is skaláris, ha `predict_proba` kimenete `(1, 2)` alakú: a pozitív oszlopot vesszük.

Error: három hibaszintet különítsünk el. A Pydantic `422` a bemeneti szerződés megsértése; a `503` azt jelenti, hogy a modell nincs betöltve, tehát a szolgáltatás nem tud tudományos választ adni; az `500` a nem várt belső hiba, amelyet nem a kliensnek kell megjavítania. Saját `try`-ban történő általános elfogás helyett a bemeneti hibát hagyjuk a keretrendszerre; a betöltöttséget explicit feltétellel kezeljük.

Latency: a predikció futási ideje működési mérőszám, nem a válasz része. A kódban logoljuk a belső predikciós időt `time.perf_counter()` segítségével. A fejlesztői szerver által logolt kérésidő nem termelési kapacitásmérés: egyetlen worker, loopback hálózat és nincs versengő terhelés. A 35.3. alfejezetben terhelési vizsgálatot adunk; ebben a leckében a belső predikciós idő csak szemléltető.

Concurrency: a predikció CPU-kötött. A FastAPI a `def` végpontokat külön threadpoolban futtatja, az `async def` végpontokat viszont az eseményhurokban; egy hosszú CPU- vagy modellszámítás az `async def`-ben blokkolná a többi kérést. Kis CPU-s modellnél a szinkron `def` a természetes választás. A betöltött modell objektumot több szál közösen olvassa, de nem módosítja; ha egy becslő vagy transformer mégsem garantálja a szálonkénti tisztaságot, zárat vagy folyamat-alapú workert kell bevezetni, és azt külön méréssel igazolni.

35.2.3 Teljes minimális munkapélda

A példa a `lab` könyvtárban egy `fastapi_service` mappát használ. A `make_dummy_model.py` egy kis, teljesen determinisztikus modellt készít, a `app.py` pedig a szolgáltatást. A model artifact és a meta együtt alkotja a 31.2. alfejezetben tanult szerződést: a verziót a manifestbe írjuk, nem a fájlnévből következtetjük.

Először a dummy modell:

```
# fastapi_service/make_dummy_model.py
import json
import joblib
import pandas as pd
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

FEATURE_COLUMNS = [
    "basket_minor", "quantity_sum", "distance_km", "priority_flag",
    "account_age_days", "segment", "region", "acquisition_channel",
]
NUMERIC = ["basket_minor", "quantity_sum", "distance_km",
           "priority_flag", "account_age_days"]
CATEGORICAL = ["segment", "region", "acquisition_channel"]

X = pd.DataFrame({
    "basket_minor": [1000, 4000, 2500, 9000, 6000, 1200],
    "quantity_sum": [1, 4, 2, 5, 3, 1],
    "distance_km": [5.0, 30.0, 12.0, 80.0, 45.0, 7.0],
    "priority_flag": [1, 0, 1, 0, 0, 1],
    "account_age_days": [120, 3, 40, 400, 2, 90],
    "segment": ["consumer", "business", "consumer",
                "business", "consumer", "business"],
    "region": ["central", "north", "south", "north", "central", "south"],
    "acquisition_channel": ["search", "referral", "direct",
                            "search", "referral", "direct"],
})
y = [0, 1, 0, 1, 1, 0]

prep = ColumnTransformer([
    ("num", StandardScaler(), NUMERIC),
    ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL),
])
model = Pipeline([
    ("prep", prep),
    ("clf", LogisticRegression(max_iter=1000)),
])
model.fit(X, y)

MODEL_PATH = "fastapi_service/model_late_dummy.joblib"
META_PATH = "fastapi_service/model_meta.json"
joblib.dump(model, MODEL_PATH)
with open(META_PATH, "w", encoding="utf-8") as f:
    json.dump({
        "model_version": "dummy-2026-10-20",
        "feature_columns": FEATURE_COLUMNS,
        "trained_on_n": 6,
        "calibration": "not_validated",
        "intended_use": "NON-PRODUCTION DEMO",
    }, f)
print("dummy modell és meta előállítva:", MODEL_PATH, META_PATH)
```

A szolgáltatás:

```
# fastapi_service/app.py
import json
import time
import uuid
import logging
from contextlib import asynccontextmanager
from typing import Literal

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, ConfigDict, Field
import joblib
import pandas as pd
```

```
MODEL_PATH = "fastapi_service/model_late_dummy.joblib"
META_PATH = "fastapi_service/model_meta.json"
FEATURE_COLUMNS = [
    "basket_minor", "quantity_sum", "distance_km", "priority_flag",
    "account_age_days", "segment", "region", "acquisition_channel",
]

logger = logging.getLogger("late_api")
if not logger.handlers:
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(message)s",
    )

class LateDeliveryRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", strict=True)

    order_id: str = Field(min_length=1, max_length=64)
    basket_minor: int = Field(ge=0)
    quantity_sum: int = Field(ge=1)
    distance_km: float = Field(ge=0, allow_inf_nan=False)
    priority_flag: int = Field(ge=0, le=1)
    account_age_days: int = Field(ge=0)
    segment: Literal["consumer", "business"]
    region: Literal["north", "central", "south"]
    acquisition_channel: Literal["search", "referral", "direct"]

class LateDeliveryResponse(BaseModel):
    order_id: str
    predicted_probability: float = Field(ge=0, le=1)
    model_version: str
    prediction_id: str
    demo_status: Literal["NON-PRODUCTION DEMO"] = "NON-PRODUCTION DEMO"
    calibration: Literal["not_validated"] = "not_validated"
    trained_on_n: int = 6
    warning: str = ("Kutatói demó; a valószínűség nem kalibrált üzleti kockázat, "
                    "termelési döntésre nem használható.")

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        model = joblib.load(MODEL_PATH)
        with open(META_PATH, encoding="utf-8") as f:
            meta = json.load(f)
    except FileNotFoundError as exc:
        raise RuntimeError(f"modell vagy meta hiányzik: {MODEL_PATH}, {META_PATH}") from exc

    expected = meta["feature_columns"]
    if list(FEATURE_COLUMNS) != list(expected):
        raise RuntimeError(
            f"feature-szerződés eltérés: a kód {FEATURE_COLUMNS}, "
            f"a modell meta szerint {expected}"
        )

    app.state.model = model
    app.state.model_version = meta["model_version"]
    app.state.trained_on_n = meta.get("trained_on_n")
    logger.info("modell betöltve: %s", app.state.model_version)
    yield
    app.state.model = None

app = FastAPI(title="Northstar late-delivery predictor", lifespan=lifespan)
```

```python
@app.get("/health")
def health():
    if getattr(app.state, "model", None) is None:
        raise HTTPException(status_code=503, detail="model not loaded")
    return {"status": "ok", "model_version": app.state.model_version}

@app.post("/predict", response_model=LateDeliveryResponse)
def predict(req: LateDeliveryRequest):
    if getattr(app.state, "model", None) is None:
        raise HTTPException(status_code=503, detail="model not loaded")

    t0 = time.perf_counter()
    row = pd.DataFrame([{col: getattr(req, col) for col in FEATURE_COLUMNS}])
    proba = app.state.model.predict_proba(row)
    prob = min(max(float(proba[0, 1]), 0.0), 1.0)
    elapsed_ms = (time.perf_counter() - t0) * 1000

    logger.info("order=%s prob=%.4f predict_ms=%.2f",
                req.order_id, prob, elapsed_ms)

    return LateDeliveryResponse(
        order_id=req.order_id,
        predicted_probability=prob,
        model_version=app.state.model_version,
        prediction_id=uuid.uuid4().hex,
        trained_on_n=app.state.trained_on_n,
    )
```

Futtatás Linuxon vagy macOS-en, a könyv `<lab>` könyvtárából:

```bash
# Linux/macOS, munkakönyvtár: <lab>
uv add "fastapi[standard]"
uv lock
uv sync --locked
uv run python fastapi_service/make_dummy_model.py
uv run uvicorn fastapi_service.app:app --host 127.0.0.1 --port 8000
```

Windows PowerShellben ugyanez a négy `uv` parancs; a kéréshez `curl.exe` vagy `Invoke-RestMethod` használható.

Érvényes kérés:

```bash
curl -s http://127.0.0.1:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"order_id":"ORD-1","basket_minor":3500,"quantity_sum":2,
       "distance_km":12.5,"priority_flag":0,"account_age_days":42,
       "segment":"consumer","region":"central","acquisition_channel":"search"}'
```

A válasz szerkezete szemléltető, mivel a fejezet írásakor nem futtattuk a modellt:

```
{"order_id":"ORD-1","predicted_probability":0.31,
 "model_version":"dummy-2026-10-20","prediction_id":"... futtatásonként új ...",
 "demo_status":"NON-PRODUCTION DEMO","calibration":"not_validated",
 "trained_on_n":6,
 "warning":"Kutatói demó; a valószínűség nem kalibrált üzleti kockázat, termelési döntésre nem
  ↪ használható."}
```

A `0.31` itt **nem** valós webshop-kockázat: a modell hat soron tanult, a kalibráció nem validált, és a válasz ezt explicit módon jelzi. A `demo_status` és a `calibration` mező nem dekoráció; a kliens és a napló e nélkül nem tudná megkülönböztetni a demót a termelési választól.

35.2.3.1 Szigorú séma, 422 vs 503 és a demó jelölése

A Pydantic `ConfigDict(extra="forbid", strict=True)` beállítása két dolgot zár ki: az ismeretlen extra mezőt és a típuscoerciont. A `"3500"` string nem lesz `int`, a `true` nem lesz `1`, a hiányzó mező pedig 422. A `distance_km` `allow_inf_nan=False` beállítása az `Infinity` és `NaN` JSON-értéket is elutasítja, mert azok a modellben értelmezhetetlen bemenetet adnának. Egyes FastAPI-verziók a nem véges float validációs hibájának JSON-sorosításakor 500-at adhatnak; ilyenkor a szerveroldali `RequestValidationError` kezelőnek sanitizálnia kell a hibaüzenetet, hogy a kliens a szerződés szerinti 422-t kapja.

A hibapolitika:

Eset	Státusz	Ki felelős
Hiányzó, extra, rossz típusú vagy tartományon kívüli mező	422	kliens
Modell nincs betöltve (`app.state.model is None`)	503	szolgáltatás
Nem várt belső hiba a predikcióban	500	szolgáltatás
Érvényes kérés	200	—

A 422 és 503 szétválasztása azért fontos, mert a kliens a 422-t saját hibaként javíthatja, a 503-at viszont nem. A 500-hoz nem szabad a bemeneti hibát leképezni: a szigorú séma a bemenetet már a végpont előtt elutasítja.

A contract-tesztek `TestClient`-tel:

```
from fastapi.testclient import TestClient

client = TestClient(app)
VALID = {
    "order_id": "ORD-1", "basket_minor": 3500, "quantity_sum": 2,
    "distance_km": 12.5, "priority_flag": 0, "account_age_days": 42,
    "segment": "consumer", "region": "central", "acquisition_channel": "search",
}

def test_valid_request_200_and_demo_labels():
    response = client.post("/predict", json=VALID)
    assert response.status_code == 200
    body = response.json()
    assert body["demo_status"] == "NON-PRODUCTION DEMO"
    assert body["calibration"] == "not_validated"
    assert body["trained_on_n"] == 6
    assert 0 <= body["predicted_probability"] <= 1

def test_missing_extra_string_and_negative_are_422():
    missing = dict(VALID); missing.pop("region")
    assert client.post("/predict", json=missing).status_code == 422
    extra = dict(VALID, unknown_field=1)
    assert client.post("/predict", json=extra).status_code == 422
    string_number = dict(VALID, basket_minor="3500")
    assert client.post("/predict", json=string_number).status_code == 422
    negative = dict(VALID, distance_km=-1.0)
    assert client.post("/predict", json=negative).status_code == 422

def test_unloaded_model_is_503_not_422():
    app.state.model = None
```

```
    response = client.post("/predict", json=VALID)
    assert response.status_code == 503
```

A `tests/test_api_contract.py` ezt a négy esetet, a 503-at és a 200 demójelöléseit futtatja. A modell metaadata a `trained_on_n=6` és `calibration="not_validated"` mezőt tartalmazza; egy termelési használhatóságot állító válasz így a séma szintjén is megbukik.

Hiányzó mezővel a kérés csak `order_id`-t és `distance_km`-et küld. A 422 válaszból a fontos rész: `missing` típusú hiba, a hiányzó mezők nevével; a 200 tehát elmarad. Negatív `basket_minor` vagy ismeretlen `region` is 422-t ad.

35.2.4 Eredmény értelmezése és ellenőrző invariant

Ha a példát helyben futtatjuk, az alábbi ellenőrző invariantoknak kell teljesülniük:

- A `make_dummy_model.py` létrehozza a `model_late_dummy.joblib` és `model_meta.json` fájlokat; a meta `feature_columns` tartalma pontosan a nyolc szerződött oszlop.
- A `/health` indítás után 200-at ad, a modellverzió `dummy-2026-10-20`.
- Érvényes kérésre a válasz `predicted_probability` értéke 0 és 1 közötti `float`, a `model_version` a metából jön, a `prediction_id` 32 karakter hosszú hexadecimális string.
- Ugyanazt a JSON-t kétszer elküldve a `predicted_probability` azonos, a `prediction_id` nem; a kettő más-más szerepű.
- A predikcióhoz használt DataFrame `(1, 8)` alakú; a `predict_proba` kimenete `(1, 2)`, a `[0, 1]` az egyes osztály valószínűsége.
- A startup `feature-szerződés eltérés` üzenettel megáll, ha a meta oszlopsorrendjét és a kód `FEATURE_COLUMNS` értékét utólag eltérítjük; a helyes javítás ilyenkor a modell újragenerálása a hiteles szerződéssel, nem a meta csendes átírása.

35.2.5 Feladatok

1. Értelmezés. Az alábbi három `POST /predict` kérés közül melyik kap 422-t, és melyik 200-at? Indokold a Pydantic szabályok alapján.

```
{"order_id":"A","basket_minor":0,"quantity_sum":1,
 "distance_km":0.0,"priority_flag":1,"account_age_days":0,
 "segment":"business","region":"north","acquisition_channel":"direct"}

{"order_id":"B","basket_minor":2500,"quantity_sum":0,
 "distance_km":5.0,"priority_flag":0,"account_age_days":10,
 "segment":"consumer","region":"west","acquisition_channel":"search"}

{"order_id":"C","basket_minor":2500,"quantity_sum":2,
 "distance_km":-1.0,"priority_flag":0,"account_age_days":10,
 "segment":"consumer","region":"central","acquisition_channel":"search"}
```

2. Implementálás. Az `app.py` `health` végpontja jelenleg csak a modell betöltöttségét jelzi. Egészítsd ki úgy, hogy a válaszban szerepeljen a betöltött modell feature-listájának hossza és a `model_version` is; a 503 viselkedést hagyja változatlanul. Írd le a teljes új `health` függvényt.

3. Hibadiagnózis. A szerver indítása a következő kimenettel megszakad:

```
RuntimeError: feature-szerződés eltérés: a kód ['basket_minor', ...],
a modell meta szerint ['basket_minor', 'quantity_sum', 'distance_km',
'priority_flag', 'segment', 'region', 'acquisition_channel']
```

Mi hiányzik a meta listából, és miért nem szabad ilyenkor egyszerűen hozzáírni a hiányzó szót a `model_meta.json`-hoz? Add meg a helyes javítási sorrendet.

4. Megváltozott feltétel: új régió. A termékcsapat be akarja vezetni a `region="west"` értéket. A jelenlegi szolgáltatás ezt a kérést azonnal `422`-vel elutasítja a `Literal` miatt. Indokold, miért helytelen megoldás csupán a Pydantic sémát kiegészíteni `"west"`-tel, és mi a teljes predikciós szerződés szerinti teendő, ha az új régióra is érvényes becslést akarunk adni. Térjen ki a modell tanító adataira és a 23.2. alfejezet csoportos validáció szempontjára.

35.2.6 Megoldások és indoklás

1. Az első kérés `200`-at kap, mert a nullák itt megengedettek: `basket_minor=0`, `distance_km=0.0`, `account_age_days=0` mind a `ge=0` korláton belül van; a kategóriaértékek érvényesek, a `quantity_sum=1` pedig a `ge=1`-et teljesíti. A második `422`-t kap, mert `quantity_sum=0` megsérti a `ge=1` korlátot, és `region="west"` nem eleme a `Literal["north","central","south"]` halmaznak; a Pydantic mindkét hibát egyszerre jelzi. A harmadik szintén `422`-t kap, mert `distance_km=-1.0` megsérti a `ge=0` korlátot. Ez az ellenőrzés a bemeneti szerződést védi, nem a modell statisztikai érvényességét.

2. A módosított health:

```python
@app.get("/health")
def health():
    model = getattr(app.state, "model", None)
    if model is None:
        raise HTTPException(status_code=503, detail="model not loaded")
    return {
        "status": "ok",
        "model_version": app.state.model_version,
        "feature_count": len(FEATURE_COLUMNS),
    }
```

Ellenőrző invariant: indítás után a válasz `{"status":"ok","model_version":"dummy-2026-10-20","feature_count":8}`. Ha a modellt mesterségesen `None`-ra állítjuk, a `503` ugyanúgy megmarad.

3. A meta listából hiányzik az `account_age_days` mező. A hibaüzenet szerint a kód listája tartalmazza, a mentett meta nem; a meta a modell mentésének pillanatában érvényes szerződés. Ha kézzel hozzáírnánk a szót, a `joblib` modell a tanításkori feature-oszlopok eltérő halmazával maradna, tehát a szolgáltatás később `predict_proba` hibát adna vagy rossz oszlopokra illesztené a bemenetet. A helyes sorrend: keressük meg a modellt létrehozó kódot, vegyük fel az `account_age_days`-t a tanító feature-listába, futtassuk újra a 35.1. alfejezet batch lépést, majd mentsük ki újra a modellt és a metát együtt. Csak ezt követően indítható el a szolgáltatás.

4. A Pydantic séma bővítése csak a bemeneti kaput tágítaná; a logisztikus regresszió OneHotEncoder-e a tanító modellből nem ismeri a `west` kategóriát. A `handle_unknown="ignore"` technikailag nulla-hot vektort adhatna, de ez nem helyettesíti a modell új tanítását, mert az új régió ügyfelei más eloszlást mutathatnak, és a tanító mintában sosem szerepeltek. A predikciós szerződés szerint ha az új régióra érvényes becslés kell, újratanítás szükséges olyan tanító adattal, amelyben a `west` régió megfelelő időbeli és ügyfél-struktúrával rendelkezik; a 23.2. alfejezet szerinti felosztásban a régiókat hordozó ügyfélcsoportok nem kerülhetnek egy fold mindkét oldalára, ha ügyfélfüggő az értékelés. Az `Literal` tágítása csak a modell és a manifest frissítésével együtt szabad; külön dokumentálni kell a határértéket, amíg az új kategória nem támogatott.

35.2.7 Gyakori tévedés ellenpéldával

A fejlesztői szerver 200-as válasza nem igazol termelési kapacitást vagy a modell jóságát. Egy 127.0.0.1-re küldött, egyetlen aktív kéréssel mért `predict_ms=2.1` nem ugyanaz, mint húsz párhuzamos kérés p99 késleltetése egy tényleges gépen: a threadpool mérete, a modellbetöltés és a hálózati várakozás mind változtat rajta. Ellenpélda: ha `predict_proba` egyetlen szálon 2 ms-ig fut, ötven egyidejű sync kérés nem lesz automatikusan 100 ms alatt; a kérések a threadpoolban sorba állhatnak, és a p99 a belső predikciós idő többszöröse is lehet. A kapacitásállításhoz terhelési cél, ismétlésszám, hardver és rögzített percentilis kell; ezt a 35.3. alfejezet vizsgálja. A 422 is csak a bemeneti szerződést ellenőrzi: a 200 mellett is lehet a predikció rosszul kalibrált, ha a modell a célsokaságtól eltérő adaton tanult.

35.2.8 Rövid összefoglaló

A FastAPI-végpont a predikciós szerződést Pydantic kérés- és válaszsémává alakítja: a `Field` korlátok az érvényes tartományt, a `Literal` a kategóriák halmazát rögzíti. A modellt a `lifespan` indításkor tölti be, és a meta-fájl feature-listájával ellenőrzi a szerződést. A hiányzó vagy rossz mező 422, a modell nélkül 503 a válasz. A CPU-bound predikció `def` végpontban, threadpoolban fut; a latency log működési mérőszám, és egyetlen loopback 200 nem termelési kapacitás. Következő lépésként 35.3. alfejezet a kutatói demó Streamlit-változatát tárgyalja.

35.3 Streamlit mint ellenőrizhető bemutató

> **Belépő · szint: haladó · idő: 90 perc · eszköz: FastAPI, Docker**
>
> Előfeltétel: 35.2. alfejezet · Célkészség: Streamlit mint ellenőrizhető bemutató · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A labor Northstar-modellje elkészült: a `pipeline.joblib`, a `model_meta.json` és a `control_predictions.npz` a 31.3. alfejezetben leírt kapun átment, a FastAPI-végpont a 35.2. alfejezetben ugyanezt a szerződést szolgálja ki. Most egy kutatói demót építünk, amely ugyanazt a betanított artefaktumot használja, de emberi bevitellel: a felhasználó egy rendelés jellemzőit adja meg, a felület pedig megmutatja a becsült késési valószínűséget, a döntési küszöböt, a modellverziót és az érvényességi korlátokat. Az előzetes becslés: ha a teszt-AUC körülbelül 0,70–0,75, a demóban egy átlagos kosárra a késési valószínűség jellemzően 0,2–0,6 közé esik; egy nagy távolságú, nem prioritásos rendelés ennél magasabbat mutathat. A pontos érték azonban a betanított modelltől függ, ezért a bemutató előtt a kontrollpredikciók egyezését ellenőrizni kell.

35.3.1 A Streamlit mint kutatói demó, nem dashboard

A Streamlit egy Python-szkriptet futtat: minden widget-interakció a teljes szkriptet újrarendezi felülről lefelé. Ez három gyakorlati következménnyel jár. Egyrészt a drága objektumokat, például a betanított pipeline-t, nem szabad minden újrafutásban betölteni; erre való a `st.cache_resource`. Másrészt a felhasználó bemeneteit és az előzményeket a `st.session_state` tárolja, mert a lokális változók az újrafutáskor elvesznek. Harmadrészt a felület logikája azt a szerződést jeleníti meg, amelyet a Pydantic-séma már kikényszerített az API-ban; a UI tehát nem lehet tágabb, mint a modell érvényes bemeneti tartománya.

A félreértések elkerüléséhez érdemes elkülöníteni a `cache`-et a `state`-től. A cache a drága erőforrások, például a modellbetöltés eredményét őrzi meg a szerverfolyamat élettartamára; a

betöltő függvény argumentumai alkotják a cache kulcsát. A state ezzel szemben az adott böngészős munkamenet állapota: a beküldött űrlap értékei, a predikciós előzmények, egy kiválasztott összehasonlító eset. A kettőt nem cseréljük fel: a modellt nem rakjuk `session_state`-be arra az esetre, ha több felhasználó osztozik rajta, a felhasználói előzményt pedig nem tekintjük processz-szintű erőforrásnak.

35.3.2 A valószínűség mint modell output, nem mérési eredmény

A késési célváltozó bináris: $y = 1$, ha a kézbesítés meghaladja a 7 napot. A logisztikus modell egy lineáris pontszámot számít a feature-vektorból:

$$z(x) = \beta_0 + \sum_{j=1}^{d} \beta_j x_j,$$

majd a szigmoid függvénnyel képez valószínűséget:

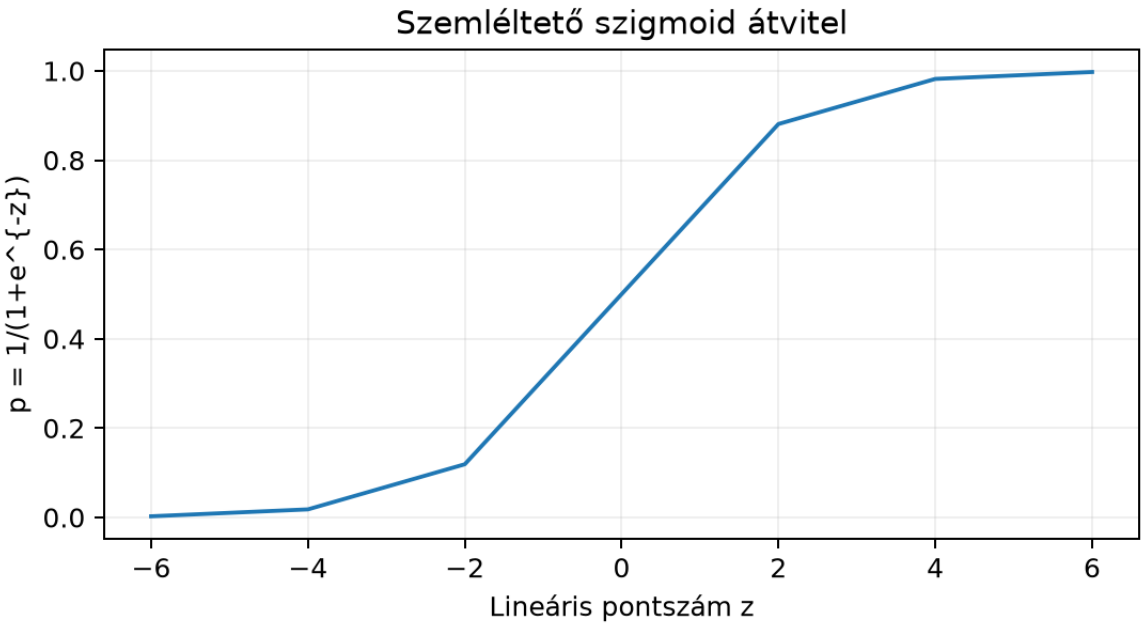
$$\hat{p}(x) = P(\text{késés} = 1 \mid x) = \frac{1}{1 + e^{-z(x)}}.$$

Itt x a rendelés pillanatában ismert feature-vektor, d a feature-ek száma, β_j a betanított együttható, $\hat{p}$ pedig a modell által becsült valószínűség. A `predict_proba` kimenete egy $(1, 2)$ alakú tömb: az első oszlop a $P(y = 0)$, a második a $P(y = 1)$ becslése. A demóban a második oszlopot jelenítjük meg.

A tisztességes megjelenítés a 26.2. alfejezetben tanultakra épül: a 0,73-as érték nem azt jelenti, hogy a rendelés 73%-os „valódi” eséllyel késik, hanem hogy a kalibráltnak szánt modell ezt a feltételes valószínűséget becsüli. A kalibrációt a validációs halmazon mértük, nem a UI-ban. A modellkártyán ezért mindig fel kell tüntetni a validációs Brier-pontszámot is:

$$B = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - \hat{p}_i \right)^2,$$

ahol n a validációs sorok száma, $y_i \in \{0, 1\}$ a tényleges kimenet, $\hat{p}_i$ a becsült valószínűség. Ha például $B = 0,18$, az átlagos négyzetes eltérés 0,18, ami azt jelzi, hogy az egyedi pontbecslés körül jelentős bizonytalanság van; a UI ezt szóban és vizuálisan is jelöli, nem kelt hamis precizitást.



35.1. ábra: Szemléltető szigmoid átvitel - Számított függvénypélda, nem mérés.

A döntéshez a betanítás során rögzített küszöböt használjuk: ha $\hat{p} \geq 0.5$, a predikció „késik”, különben „nem késik”. Ez a küszöb a modellkártya része, nem a felhasználó szabadon tologatható paramétere, mert a küszöb eltolása a validációs hiba ismerete nélkül önkényes lenne.

35.3.3 Teljes minimális munkapélda

A labor könyvtárában a következő fájlokra támaszkodunk: `artifacts/pipeline.joblib`, `artifacts/model_meta.json`, `artifacts/control_predictions.npz`, valamint a korábban elkészült `schemas.py` és `model_loader.py`. A `schemas.py` a Pydantic szerződést tartalmazza, a `model_loader.py` a hash-ellenőrzött betöltést. A demó alkalmazásban egyetlen új modul, a `streamlit_demo.py` készül.

Először telepítjük a Streamlit csomagot a projektbe, majd elindítjuk az alkalmazást:

```
cd lab
uv add "streamlit>=1.40"
uv sync
uv run streamlit run streamlit_demo.py --server.headless true
```

Az utolsó sor platformfüggetlen; Windows PowerShellben ugyanígy futtatható a `lab` munkakönyvtárból. A pontos Streamlit-verzió a saját `uv.lock`-ban rögzítendő; a könyv keretjegyzete nem állítja, hogy egy adott verziókombinációt előzetesen ellenőriztünk.

Az alábbi kód a leírt API-t használja; a könyv írásakor nem futtattuk, ezért a hozzá tartozó felületi kijelzés szemléltető jellegű.

```
# streamlit_demo.py - kutatói demó, új tanítás nélkül
from pathlib import Path
import json
import numpy as np
import pandas as pd
import streamlit as st
```

```
from pydantic import ValidationError
from schemas import NorthstarRequest, LatePrediction
from model_loader import load_verified_model, ArtifactVerificationError

ARTIFACT_DIR = Path("artifacts")
DEFAULT_VALIDATED_THRESHOLD = 0.5  # a meta tartalmazza; itt csak biztonsági alapértelmezés

@st.cache_resource(show_spinner="Modell ellenőrzése és betöltése...")
def get_verified_model(artifact_dir: str):
    return load_verified_model(Path(artifact_dir))

st.set_page_config(page_title="Northstar késés demó", layout="wide")

# Oldalsáv: modellkártya a meta-fájlból
meta_path = ARTIFACT_DIR / "model_meta.json"
meta = json.loads(meta_path.read_text(encoding="utf-8"))

with st.sidebar:
    st.header("Modellkártya")
    st.markdown(f"**Modellverzió:** {meta['model_version']}")
    st.markdown(f"**Betanítási cutoff:** {meta['training_cutoff']}")
    st.markdown(f"**Feature-lista:** {', '.join(meta['features'])}")
    st.markdown(f"**Validációs AUC:** {meta['validation_auc']:.3f}")
    st.markdown(f"**Validációs Brier:** {meta['validation_brier']:.3f}")
    validated_threshold = float(meta.get("threshold", DEFAULT_VALIDATED_THRESHOLD))
    threshold_version = meta.get("threshold_version", "2025-08-01")
    st.markdown(f"**Validált döntési küszöb:** {validated_threshold:.3f} "
                f"(verzió: {threshold_version})")
    st.caption("A küszöb a modellkártya része; a validáció a rögzített "
               "teszthalmazon történt.")
    st.warning(
        "Kutatói demó. A valószínűség pontbecslés; a kalibrációt a "
        "validációs halmazon mértük. Nem termelési döntési felület."
    )
    st.subheader("Érzékenységi üzemmód")
    sensitivity = st.checkbox("Hipotetikus küszöb használata", value=False)
    if sensitivity:
        threshold = st.slider("Hipotetikus küszöb", 0.05, 0.95,
                              validated_threshold, 0.01)
        st.warning("Az érzékenységi küszöb nem validált; csak a döntési címkét "
                   "változtatja, a modell valószínűségét nem.")
    else:
        threshold = validated_threshold

try:
    package = get_verified_model(str(ARTIFACT_DIR))
    model = package["model"]
except ArtifactVerificationError as exc:
    st.error(f"A modellartefaktum ellenőrzése sikertelen: {exc}")
    st.stop()

st.title("Rendeléskésés becslése rendeléskor")
st.markdown(
    "Csak a rendelés pillanatában ismert mezőket fogadja el; minden "
    "predikció a betanított modellt használja további tanítás nélkül."
)

with st.form("prediction_form"):
    c1, c2, c3 = st.columns(3)
    distance_km = c1.number_input("Távolság (km)", min_value=0.0, value=25.0, step=1.0)
    priority_flag = c2.checkbox("Prioritás szállítás", value=False)
    basket_minor = c3.number_input("Kosár értéke (EUR-cent)", min_value=0, value=5000, step=500)
    quantity_sum = c1.number_input("Darabszám", min_value=1, value=3, step=1)
```

```
    account_age_days = c2.number_input("Fiók kora (nap)", min_value=0, value=120, step=1)
    segment = c3.selectbox("Szegmens", ["consumer", "business"])
    region = c1.selectbox("Régió", ["north", "central", "south"])
    acquisition_channel = c2.selectbox("Akvizíciós csatorna", ["search", "referral", "direct"])
    submitted = st.form_submit_button("Predikció futtatása")

if submitted:
    raw = {
        "distance_km": distance_km,
        "priority_flag": int(priority_flag),
        "basket_minor": basket_minor,
        "quantity_sum": quantity_sum,
        "account_age_days": account_age_days,
        "segment": segment,
        "region": region,
        "acquisition_channel": acquisition_channel,
    }
    try:
        request = NorthstarRequest(**raw)
        # A pipeline a meta features sorrendjében várja az oszlopokat.
        frame = pd.DataFrame([request.model_dump()], columns=meta["features"])
        proba = model.predict_proba(frame)[0, 1]
        predicted_late = int(proba >= threshold)
        result = LatePrediction(
            late_probability=float(proba),
            predicted_late=predicted_late,
            model_version=meta["model_version"],
        )
        history = st.session_state.setdefault("history", [])
        history.append({"input": raw, "result": result.model_dump(),
                        "threshold": threshold, "threshold_mode":
                        "sensitivity" if sensitivity else "validated"})
        st.session_state["history"] = history[-5:]
        st.success(
            f"Késési valószínűség: **{result.late_probability:.3f}** "
            f"(döntés: {'késhet' if result.predicted_late else 'időben érkezhet'})"
        )
        st.caption(
            f"A döntési címke küszöbe {threshold:.3f} "
            f"({threshold_version}, "
            f"{'érzékenységi, nem validált' if sensitivity else 'validált'} mód). "
            "A valószínűséget a küszöb nem változtatja meg. "
            "A becslés a modellkártyán jelzett bizonytalansággal értelmezendő; "
            "egyetlen rendelés esetén nem mérési tény."
        )
        st.info(
            "Modellezési korlát: csak a modellkártyán szereplő régiók és "
            "szegmensek támogatottak; a demó nem termelési döntési rendszer."
        )
    except ValidationError as exc:
        st.error(f"Érvénytelen bemenet: {exc}")

if st.session_state.get("history"):
    st.subheader("Utolsó predikciók")
    st.dataframe(pd.DataFrame(st.session_state["history"]))
```

A kód szándékosan nem hív tanítást: a `model` objektum a `get_verified_model` cache-elt erőforrása, a predikció pedig csak `predict_proba`. A widgetek minden értéket a Pydantic-tartománynak megfelelő típusban adnak: a `checkbox` lebegőpontos `True` helyett egész 0/1 értékké alakul, a kategóriák `selectbox`-ból jönnek, így a séma nem kap érvénytelen literált.

35.3.4 Eredmény értelmezése és kézi kontroll

A szemléltető futás eredménye: a felhasználó 25 km távolságot, 50 eurós kosarat, 3 darabot, 120 napos fiókot, consumer/north/search mezőket ad meg. A bemutató a `model_meta.json` szerinti 0.1.0-s verziót írja ki; a kontrollpredikciók egyezése már a betöltéskor megtörtént. A megjelenő valószínűség a modell konkrét válasza; ha például 0,21, akkor a lineáris pontszám $z \approx -1{,}32$, mert $\sigma(-1{,}32) \approx 0{,}21$. A felület nem mutat konfidenciaintervallumot, mert a betanított modellből egyetlen pontbecslés származik; a bizonytalanságot a validációs Brier-pontszám és a szöveges figyelmeztetés jeleníti meg.

Kézi kontrollként ellenőrizhetjük, hogy a `predict_proba` kimenet oszlopai összegeznek-e 1-re. A `frame` egy $(1, d)$ alakú DataFrame, a `proba` egy $(1, 2)$ alakú NumPy-tömb; az eredmény $\hat{p}_1 + \hat{p}_0 = 1\ 10^{-9}$ abszolút tűrésen belül. A döntési szabály pedig csak a meta szerinti küszöböt használja, ezért a kijelzés determinisztikus ugyanarra a bemenetre.

A Streamlit állapot- és cache-kezelésében nem tárolható ellenőrizetlen felhasználói bemenetből betöltött pickle/joblib objektum.

35.3.4.1 Validált küszöb és érzékenységi üzemmód

A demó alapértelmezett küszöbe a modellkártyából jön, nem a felhasználótól. A modellkártya rögzíti a `threshold` értéket, a `threshold_version` azonosítót és a forrást (például a 23.1. alfejezet költségmátrixát vagy a validációs döntési szabályt). A validált üzemmódban a küszöb fix; a felület csak megjeleníti.

Az **érzékenységi üzemmód** külön kapcsolóval indítható. Ilyenkor a csúszka a döntési címkét változtatja, a modell valószínűségét nem:

```
def decide(probability, threshold):
    return int(probability >= threshold)

prob = 0.31
assert decide(prob, 0.50) == 0
assert decide(prob, 0.25) == 1
assert prob == 0.31  # a valószínűség változatlan
```

Az érzékenységi módban a felület kötelezően jelzi: „nem validált, hipotetikus küszöb”; a döntési címke mellé kiírja a használt küszöböt és a módot. A válasz és az előzmény külön mezőben tárolja a `threshold` és `threshold_mode` értéket, így utólag megmondható, melyik döntés melyik küszöb mellett született.

A demó emellett kockázati figyelmeztetést és modellezési korlátot jelenít meg: a valószínűség nem kalibrált üzleti kockázat, a modell csak a kártyán szereplő régiókra és szegmensekre érvényes, és a felület nem termelési döntési rendszer. A `tests/test_threshold_separation.py` a fix alapértelmezett küszöböt, az érzékenységi mód címkeváltozását, a valószínűség változatlanságát és a kötelező figyelmeztetéseket teszteli.

35.3.5 Feladatok

1. **Értelmezés.** A demóban a felhasználó azt látja, hogy $\hat{p} = 0{,}73$. Írd le, mit jelent ez a modellkártya szerint, és miért helytelen a felületen azt állítani, hogy „a csomag 73% valószínűséggel késik” minden további feltétel nélkül. Hivatkozzon a validációs Brier-pontszám szerepére.

2. **Implementálás.** Bővítse a `streamlit_demo.py`-t egy gombbal, amely a `session_state`-ben tárolt előzményt törli, és egy másik gombbal, amely a predikciós űrlapot visszaállítja az alapértelmezett értékekre. A modellbetöltőhöz ne nyúljon.

3. **Hibakeresés.** Egy fejlesztő a következő sort írta a `get_verified_model` helyére:

   ```
   model = load_verified_model(ARTIFACT_DIR)
   ```

   A felhasználó azt tapasztalja, hogy minden widget-kattintás után hosszú „Modell ellenőrzése” üzenet jelenik meg. Magyarázd el a Streamlit végrehajtási modellje alapján, miért, és add meg a javítást.

4. **Megváltozott feltétel.** A modellt újratanítják, a `model_meta.json` `model_version` értéke 1.1.0-ra változik, a fájlok azonos könyvtárban maradnak. A demo újraindítás nélkül továbbra is a régi verziót mutatja. Miért nem elég a `st.cache_resource` argumentumában csak a könyvtárnevet szerepeltetni? Adj olyan cache-kulcsot, amely verzióváltáskor automatikusan újratölti a modellt.

5. **Visszautalás és önellenőrzés.** Hogyan kapcsolódik a bemutató a 31.3. alfejezetben bevezetett betöltési határhoz? Soroljon fel legalább két ellenőrzést, amelyet a `load_verified_model` a UI előtt már elvégzett, és indokold, miért nem helyettesítheti ezeket a tetszetős felület.

35.3.6 Megoldások és indoklás

1. A $\hat{p} = 0{,}73$ a modell becsült feltételes valószínűsége annak, hogy a rendelés késik a megadott feature-ek mellett. A kalibráltnak szánt modell esetén ez azt jelenti, hogy sok hasonló, 0,73 körüli pontszámú esetből hosszú távon körülbelül 73% késhet; de egyetlen rendelésnél nincs „részleges” késés, és a becslés hibája a Brier-pontszámban jelenik meg. A 0,73 tehát modell output, nem a valóság közvetlen mérése. A validációs Brier-pontszám, például 0,18, azt mutatja, hogy az egyedi becslések átlagos négyzetes eltérése jelentős; ezért a felületnek szövegesen is jeleznie kell a bizonytalanságot.

2. A kiegészítő kód a form után helyezhető:

   ```
   col_clear, col_reset = st.columns(2)
   if col_clear.button("Előzmény törlése"):
       st.session_state.pop("history", None)
       st.rerun()

   if col_reset.button("Alapértelmezett űrlap"):
       for key in list(st.session_state.keys()):
           if key.startswith("form_"):
               del st.session_state[key]
       st.rerun()
   ```

   A valódi visszaállításhoz a widgeteket `key` paraméterrel kell ellátni, például `key="form_distance_km"`, mert a `st.session_state` így őrzi az értékeket. A `st.rerun` újrarendezi a szkriptet, de a modell a `cache_resource` miatt nem töltődik újra. A törlés után az előzmények nem jelennek meg, mert a feltétel `st.session_state.get("history")` hamis lesz.

3. A Streamlit minden interakcióra újrafuttatja a szkriptet felülről lefelé; a `load_verified_model(ARTIFACT_`
   tehát minden widget-változásnál újra deszerializálja a pipeline-t és futtatja a hash-, séma- és kontrollpredikció-ellenőrzést. Ez a drága betöltés a lassulás oka. A javítás a `@st.cache_resource` dekorátor, amely a függvény argumentumát használja kulcsként, és

a betöltött modellt a folyamat szintjén tárolja. A kulcs itt a könyvtárnév szövege; ha a fájlok nem változnak, a betöltés egyszer történik.

4. A `st.cache_resource` a függvény argumentumainak hash-ét használja; ha az argumentum továbbra is `"artifacts"`, a cache nem tudja, hogy a fájlok megváltoztak. A `load_verified_model` belül hash-ellenőrzést végez, de a Streamlit cache előbb tér vissza a régi objektummal, ezért a meta régi verziót mutat. Megoldás: a cache kulcsba bele kell foglalni a verziót vagy a meta hash-ét. Például:

```
@st.cache_resource(show_spinner="Modell ellenőrzése és betöltése...")
def get_verified_model(artifact_dir: str, model_version: str):
    return load_verified_model(Path(artifact_dir))

meta = json.loads((ARTIFACT_DIR / "model_meta.json").read_text())
package = get_verified_model(str(ARTIFACT_DIR), meta["model_version"])
```

   A meta beolvasása olcsó, és nem szükséges cache-elni. A verzióváltáskor megváltozik a kulcs, a modell újratöltődik, a régi cache bejegyzés elavul. A `model_version` mellett a meta `artifact_hashes` összegét is lehet kulcsként használni, ha a verziószám nem változna.

5. A `load_verified_model` először a fájlhasheket ellenőrzi a `model_meta.json`-ban rögzített értékekkel, csak ezután deszerializál; ellenőrzi a feature-listát a séma ellen, és a kontrollpredikciókat a 10^{-9} abszolút tűrésen belül összehasonlítja. A demó emiatt tudja megjeleníteni a modellverziót anélkül, hogy ismeretlen fájlt futtatna. A tetszetős UI ezeket az ellenőrzéseket nem pótolja: a szép felület ugyanazt a hibás artefaktumot is szépen mutatná, a betöltési kapu viszont megakadályozza, hogy sérült vagy nem várt objektum kerüljön a felületre.

35.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba a vizuális megjelenítés összekeverése a statisztikai érvényességgel. Ha a UI egy 0,9-es valószínűséget piros, nagy betűkkel jelenít meg, a felhasználó hajlamos azt gondolni, hogy a modell nagyon magabiztos. Ez azonban csak megjelenítés. Tegyük fel, hogy egy buta baseline minden rendelésre 0,75-öt ad; a felület ugyanolyan figyelmeztető színnel mutatja, de a validációs Brier-pontszám magas, a megbízhatóság pedig gyenge. A kalibrációt és az általánosítást a validációs halmazon kell mérni (26.2. alfejezet, 23.3. alfejezet), nem a UI színpalettájával. Hasonlóképpen hibás a widgeteket az érvényességi tartományon túlra engedni: ha a UI nagyobb távolságot vagy negatív árat fogad el, mint a betanítási adatok, a modell extrapolál, és ezt a felület nem teheti láthatatlanná.

35.3.8 Rövid összefoglaló

A Streamlit-demó ugyanazt a betanított pipeline-t és meta-fájlt használja, amelyet a FastAPI-végpont is kiszolgál, új tanítás nélkül. A `st.cache_resource` a drága modellbetöltést őrzi, a `st.session_state` a felhasználói előzményt és űrlapállapotot tárolja, a Pydantic-séma pedig ugyanazt a bemeneti tartományt ellenőrzi, mint az API. A modellkártya a verziót, a feature-listát, a validációs AUC-t, Brier-pontszámot és küszöböt mutatja; a prediktív valószínűség pontbecslés, amelyet a UI szövegesen is annak nevez. A betöltő hash- és kontrollpredikció-ellenőrzése biztosítja, hogy a látványos felület ne takarjon el sérült artefaktumot. A következő alkalmazás 35.4. alfejezet: kiadás, visszaállítás és terhelési próba, ahol a demó azt kérdezi meg, hogyan válik ellenőrzött szolgáltatássá.

35.4 Kiadás, visszaállítás és terhelési próba

Belépő · szint: haladó · idő: 90 perc · eszköz: FastAPI, Docker

Előfeltétel: 35.3. alfejezet · Célkészség: Kiadás, visszaállítás és terhelési próba · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

35.4.1 Helyzet: kiadás előtti átadás

A 35.2. alfejezet FastAPI-szolgáltatás implementálva van, a jelölt modell pedig átment a 31.4. alfejezet jóváhagyási lépésén, és a döntési küszöbét a 23.4. alfejezet módszere szerint befagyasztottuk. Most az a kérdés, miként válik a jelöltből éles, *champion* szolgáltatás úgy, hogy hiba esetén azonnal vissza tudjunk állni egy korábbi verzióra, és milyen elfogadási feltételeket kötünk a terheléshez.

- **Bemenet**: `POST /predict`, A Northstar-szerződés nyolc rendeléskori feature-e; itt most a betanított modell kérés-sémája.
- **Kimenet**: JSON a `probability`, a `decision` és a kiszolgáló modell verziója.
- **Megfigyelési egység**: egy HTTP-kérés.
- **Cél**: release-jegyzőkönyv készítése, helyi mért késleltetés, és egy nem futtatott terhelési próba tervként való rögzítése.
- **Előzetes becslés**: egy 8 feature-es logisztikus modell helyi, szekvenciális futtatásánál a p95-ös késleltetés nagyságrendileg néhány milliszekundum; elfogadási célnak p95 25 ms-ot veszünk. Ez célkitűzés, **nem** a labor mért eredménye.

A release két kaput érint: a *kiszolgálási kapu* a processz életét, a sémát és a késleltetést ellenőrzi; a *modellminőségi kapu* a predikció tudományos tulajdonságait. A kettőt szándékosan különválasztjuk: egy zöld `/health` még semmit nem mond arról, hogy a modell jól jósol-e.

35.4.2 A release-jegyzőkönyv és a verziópin

A *verziópin* (*version pin*) azt jelenti, hogy az éles szolgáltatás minden rétegét egyetlen azonosítható, megváltoztathatatlan állapotra rögzítjük. A modellfájl, a kód, a bemeneti séma, a függőségi zár és a döntési küszöb együtt alkotja a releaset. Ha bármelyik megváltozik, az új release azonosítót kap.

```
# release_tools.py -- vázlat, a könyv generálása nem futtatta
import hashlib
import json
from pathlib import Path

def sha256_hex(path: Path) -> str:
    """A fájl tartalmából számított SHA-256 hash hex alakban."""
    h = hashlib.sha256()
    h.update(path.read_bytes())
    return h.hexdigest()

def build_manifest(model_file: Path, schema: str, threshold: float,
                   code_commit: str, deps_lock: str = "uv.lock") -> dict:
    return {
        "release": "northstar-inference-1.4.0",
        "model_file": str(model_file),
        "model_hash": "sha256:" + sha256_hex(model_file),
        "code_commit": code_commit,
        "schema_version": schema,
```

```
        "decision_threshold": threshold,
        "deps_lock": deps_lock,
        "created_at": "2026-07-01T12:00:00Z",  # a bélyeget a CI tölti ki
    }
```

A SHA-256 kriptográfiai kivonat: ugyanaz a bájtsorozat ugyanazt a 64 hexadecimális karakterből álló lenyomatot adja; már egy bájtnyi változás is nagy valószínűséggel eltérő lenyomatot eredményez. A hash-egyezés önmagában nem igazolja a forrás hitelességét: a várt értéknek megbízható, lehetőleg aláírt csatornából kell származnia. A manifestet verziózott tárolóba írjuk, és a 31.4. alfejezet registry-bejegyzésére mutat. A manifest **nem** bizonyítja a modell minőségét, és önmagában a build reprodukálhatóságát sem: ellenőrizhető fájlazonosságot és a reprodukáláshoz szükséges hivatkozásokat rögzíti. A reprodukálhatóságot külön, tiszta környezetes újrafuttatás igazolja.

A betöltésnél a hashegyezést is ellenőrizzük, mert a 31.4. alfejezetben már láttuk: ismeretlen joblib-fájlt nem töltünk be:

```
def load_verified(path: Path, expected_hash: str):
    actual = "sha256:" + sha256_hex(path)
    if actual != expected_hash:
        raise RuntimeError(f"hash mismatch: {actual} != {expected_hash}")
    import joblib
    return joblib.load(path)
```

35.4.3 Health és readiness: nem ugyanaz a kapu

A *health* ellenőrzés csak annyit mond: a processz él. A *readiness* azt mondja: a processz kész kérést kiszolgálni. Indítás után a modell betöltése ideig-óráig tarthat; addig a health már 200-as, a readiness viszont még hibát ad.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
MODEL = None  # startup_load tölti be a s138 szerint

@app.get("/health")
def health():
    return {"status": "ok", "version": "northstar-inference-1.4.0"}

@app.get("/ready")
def ready():
    if MODEL is None:
        raise HTTPException(status_code=503, detail="model_not_loaded")
    return {
        "ready": True,
        "model_hash": MODEL.meta["hash"],
        "schema_version": MODEL.meta["schema"],
    }
```

A működési szabály: a terheléselosztó a `/ready`-t pollozza, és csak HTTP 2xx válasznál tekinti késznek a példányt, mielőtt forgalmat enged a node-ra. A `/health` a processz-újraindításhoz, a `/ready` a forgalom szétosztásához tartozik. A diagnózisnál a kettő együtt adja a képet: ha a health zöld, de a ready piros, a modell nem töltődött be.

35.4.4 Canary és shadow: ellenőrzött átállás

A champion–challenger váltás fokozatos. A *shadow* üzemmódban a challenger a valódi bejövő forgalom másolatán végigfut, de a választ továbbra is a champion adja. Tároljuk a challenger predikciós score-jait, hibáit és késleltetését anélkül, hogy a felhasználó döntését befolyásolná. A *canary* üzemmódban a forgalom kis, előre rögzített hányadát már a challenger szolgálja ki.

Üzemmód	Válasz forrása	A challenger szerepe
shadow	champion	mérik a challengert; nem hat a döntésre
canary	challenger 1	valódi forgalmat kap, azonnali visszavonás mellett

A canary elfogadási feltételét előre rögzítsük, például: 1000 canary-kérésen a hibaarány 1%, és a p95 a champion 1,2-szerese. A döntéskor figyeljünk a mintavételi véletlenre is. Ha az elfogadási hibahatár 1%, akkor 1000 kérésnél a hibák száma $X \sim \mathrm{Bin}(1000, 0.01)$, a várható értéke 10 hiba, a szórása pedig

$$\sqrt{1000 \cdot 0.01 \cdot 0.99} \approx 3{,}15 \text{ hiba.}$$

A 0,95 közelítő tartomány így $10 \pm 1{,}96 \cdot 3{,}15 \approx [4, 16]$ hiba. Ha 17 hibát látunk, az jelzés értékű, ha 10-et, az még nem bizonyíték a túllépésre. A canary rövid ablakán a modell *statisztikai minőségét* érett címkék nélkül nem tudjuk megbízhatóan megítélni. A shadow futás a modell különböző bemeneteken mutatott viselkedését és a késleltetést ellenőrzi, de **címkék nélkül nem bizonyítja a prediktív minőségét**; a canary elsődlegesen a szolgáltatási megbízhatóság kapuja.

35.4.5 Rollback: pointer-váltás, nem pánik

A *rollback* egy adminisztratív pointer-váltás a registry-ben: az `active.json` egy korábbi jóváhagyott modellre mutat. Maga a modell-bináris verziózott artefaktum marad; a szolgáltatás a pointert olvassa.

```
from pathlib import Path
import json

def load_active_model(registry_dir: Path):
    pointer = json.loads((registry_dir / "active.json").read_text())
    model_file = registry_dir / pointer["file"]
    return load_verified(model_file, pointer["hash"])
```

A rollback feltételeit *előre* rögzítjük, mert a hibánál nincs idő döntéshozatali vitára. Azonnali visszaállítási ok: az éles hibaarány átlépi az 1%-ot, a séma nem fogadja a bejövő kéréseket, vagy a késleltetés tartósan az elfogadási sáv fölé megy. A modellminőségi romlás viszont gyakran csak késői címkével értékelhető; ott előbb a shadow metrikákat vetjük össze az offline teszttel, és a döntést a review-ra kötjük. A rollback tehát **nem** a statisztikai következtetés pótlója, hanem a szolgáltatási kapu eszköze.

35.4.6 Load budget és helyi késleltetésmérés

A *load budget* az a terhelési keret, amelyen belül a szolgáltatásnak teljesítenie kell. Egy CPU-s modellnél nem elég egyetlen átlagos kérés idejét mérni, mert a felhasználó a késleltetés *farkát* tapasztalja. A 13.2. alfejezetben látott kvantilist használjuk: rendezzük a mért késleltetéseket

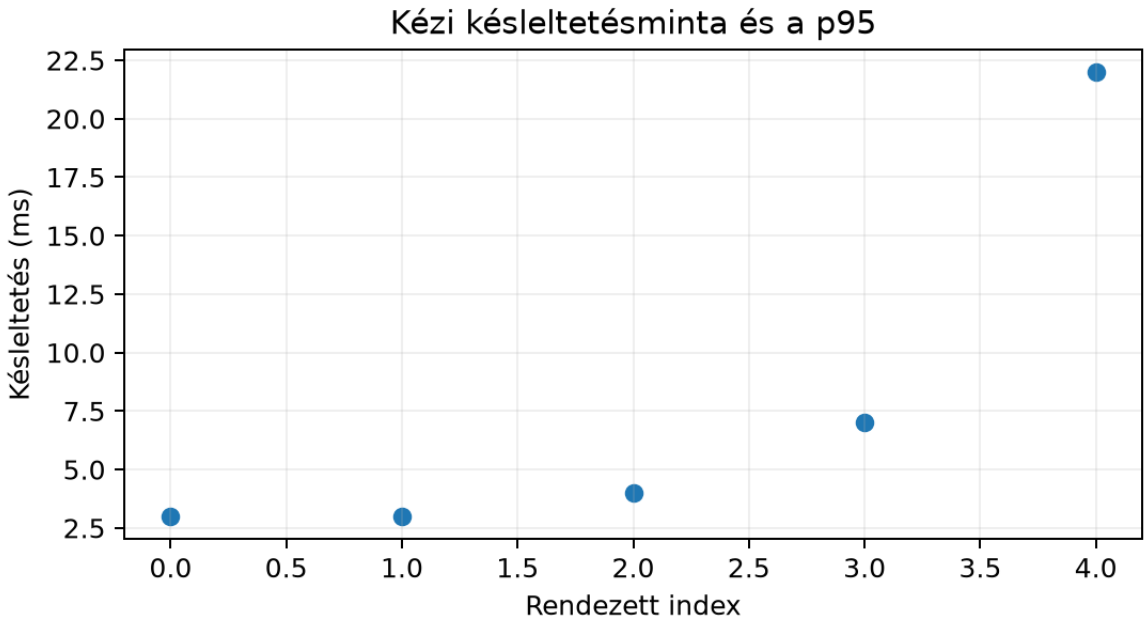
$$x_{(0)} \le x_{(1)} \le \ldots \le x_{(n-1)},$$

és a p rendű mintakvantilis

$$\hat{q}_p = x_{(k)}, \qquad k = \lfloor p \cdot (n-1) \rfloor,$$

ahol n a minta elemszáma, $p \in [0,1]$, a késleltetés mértékegysége ms. Ez a közelítés konzisztens a gyakran használt rendezett index-szabállyal; más könyvtárak interpolálnak, ezért a jegyzőkönyvben a módszert is rögzítjük.

Kézi kontroll: legyen a rendezett minta $[3, 3, 4, 7, 22]$ ms, $n = 5$. A kvantilis értéke a **választott definíciótól függ**. A szokásos, a legközelebbi rangon alapuló empirikus p95 itt $x_{(\lceil 0{,}95 \cdot 5 \rceil)} = x_{(5)} =$ 22 ms (1-től indexelve). A NumPy alapértelmezett, lineárisan interpolált `np.quantile([3, 3, 4, 7, 22], 0.95)` értéke 19 ms. Az egyszerű $\lfloor 0{,}95 \cdot 4 \rfloor$ indexelésből adódó 7 ms **nem** tekinthető definíciófüggetlen p95-nek. Ötelemű mintából a 95. percentilis bizonytalan és érzékeny a legnagyobb megfigyelésre; ezért a becslési módszert és a mintanagyságot mindig közöljük.



35.2. ábra: Kézi késleltetésminta és a p95 - Kézi kontrollminta, nem futtatási jegyzék.

A helyi mérés szekvenciális, egyetlen klienssel:

```
# latency_probe.py -- vázlat, a szolgáltatásnak futnia kell egy másik terminálban
import json
import time
import urllib.request
```

```
PAYLOAD = {"distance_km": 12.0, "priority_flag": 0, "basket_minor": 3200,
           "quantity_sum": 4, "account_age_days": 110, "segment": "consumer",
           "region": "north", "acquisition_channel": "search"}

def one_request() -> float:
    data = json.dumps(PAYLOAD).encode()
    req = urllib.request.Request(
        "http://127.0.0.1:8000/predict", data=data,
        headers={"Content-Type": "application/json"})
    t0 = time.perf_counter()
    with urllib.request.urlopen(req.with_method("POST")) as resp:
        resp.read()
    return time.perf_counter() - t0

def quantile(xs: list[float], p: float) -> float:
    xs = sorted(xs)
    k = int(p * (len(xs) - 1))
    return xs[k]

lat = [one_request() for _ in range(200)]
print({"n": len(lat), "p50_ms": round(quantile(lat, 0.50), 3),
       "p95_ms": round(quantile(lat, 0.95), 3),
       "p99_ms": round(quantile(lat, 0.99), 3)})
```

A futtatás platformja: Linux/macOS vagy Windows PowerShell, a projektgyökér `northstar-inference`. Elindítás egyik terminálban:

```
# Linux/macOS, projektgyökér: northstar-inference
uv run uvicorn app.main:app --port 8000
```

Szemléltető várt output, **nem** mért jegyzék: `n=200, p50 4 ms, p95 12 ms, p99 30 ms`. A tényleges értéket a szolgáltatás, a gép és az operációs rendszer együtt adja. A szekvenciális kliens a kis terhelésű késleltetést méri, nem a kapacitást. A valódi terhelési próbát explicit tervként jelöljük: párhuzamos kliens 10 szálon, 10 000 kérés, a p95/p99 mérés, a memóriagörbe rögzítése, és az eredmény minden értékét a jegyzőkönyvbe menteni. Ennek végrehajtása ebben a szakaszban nincs elvégezve.

35.4.6.1 Kiadási küszöbök és rollback-integritási próba

A kiadás előtt két, egymástól független kaput kell lezárni. A **terhelési kapu** a szolgáltatás viselkedését méri; a **rollback-kapu** azt bizonyítja, hogy a korábbi modellre visszaállva ugyanazokat az ismert kimeneteket kapjuk.

A terhelési küszöbök előre rögzítettek (a példaértékek a helyi környezetre vonatkoznak, nem általános ígéretek):

Metrika	Küszöb	Mérés
Átviteli sebesség	50 kérés/s	10 szál, 10 000 kérés, 3 futás mediánja
p95 késleltetés	25 ms	ugyanaz a mérés
p99 késleltetés	50 ms	ugyanaz a mérés
Hibaarány	1%	HTTP 4xx/5xx és timeout együtt
Memóriacsúcs	512 MiB	`resource.getrusage` / folyamatfigyelő
Cold start	5 s	modellbetöltés + `/ready` 200

A mérési jegyzőkönyv rögzíti a hardvert, a worker-számot, a Python/csomagverziókat, a payloadot és a pontos percentilis-definíciót. Ha bármelyik küszöb megsérül, a release **no-go**; a canary csak a küszöbök teljesülése után indulhat.

A rollback-próba lépései:

1. A kiadás előtt a régi modellre lefuttatjuk a rögzített kontrollhalmazt, és eltároljuk a kimeneteket a modellhash-sel együtt.
2. Az `active.json` pointert a régi modellre állítjuk, a szolgáltatást újraindítjuk, a `/ready` a régi hash-t és sémát jelenti.
3. Ugyanazokat a bemeneteket elküldve a kimeneteknek a tárolt értékekkel kell egyezniük (egzakt vagy előre rögzített tolerancia mellett), és a válasz `model_version` mezője a régi verzió kell legyen.
4. Ezután visszaállunk az új modellre, és a kontrollt újra lefuttatjuk; a két állapot nem keveredhet.

```
def release_gate(m):
    errors = []
    if m["throughput_rps"] < 50:
        errors.append("throughput")
    if m["p95_ms"] > 25:
        errors.append("p95")
    if m["p99_ms"] > 50:
        errors.append("p99")
    if m["error_rate"] > 0.01:
        errors.append("error_rate")
    if m["peak_mb"] > 512:
        errors.append("memory")
    if m["cold_start_s"] > 5:
        errors.append("cold_start")
    return errors

def rollback_check(expected_hash, active_hash, old_outputs, got_outputs,
                   rtol=1e-9, atol=1e-9):
    if expected_hash != active_hash:
        raise AssertionError("a rollback nem a várt modellre mutat")
    if len(old_outputs) != len(got_outputs):
        raise AssertionError("kimeneti hossz eltér")
    for old, got in zip(old_outputs, got_outputs):
        if abs(old - got) > atol + rtol * abs(old):
            raise AssertionError("a rollback kimenete eltér")
    return True
```

A `tests/test_release_gate.py` a küszöbök alatti és feletti eseteket, a helyes rollbacket, a rossz hash-t és az eltérő kimenetet teszteli. A rollback alatt az azonos inputra adott eltérő output nyomon követhető a modellverzió és a hash alapján; a régi modell csak akkor engedhető vissza, ha a hash és a kontrollkimenet egyezik.

35.4.7 Feladatok

1. Értelmezés. A release-jelentésben ez áll: `/health` 200-as egész nap, `/ready` 200-as; a shadow challenger Brier-pontszáma 0,031, a championé 0,028 ugyanazon a shadow-mintán; a canary challenger p95 32 ms, a championé 18 ms. Élesítsük-e a challengert? Indokold részletesen.

2. Implementálás. Egészítsd ki a `build_manifest` függvényt úgy, hogy a releváns splitmanifest fájlt is bekérje, és annak hash-ét is eltárolja. Írd le az ellenőrző invariantet: mi marad változatlan, ha a modellfájl és a splitfájl tartalma sem változik? A release azonosító természetesen ne változzon.

3. Hibadiagnózis. A canary 4%-os hibaaránnyal fut, a `/ready` mégis 200-as. A hibaüzenetek a szerveroldalon mind `Unprocessable Entity` választ adnak. Mi lehet a hiba? Van-e összefüggés a release manifest sémájával? Add meg a pontos diagnosztikai lépést.

4. Megváltozott feltétel. A csapat úgy dönt, hogy az inferencia éjszakai batch-folyamatba kerül (lásd 35.1. alfejezet), 50 000 sorra, 10 perces időkerettel, 4 workerrel. Hogyan fogalmaznád meg a load budgetet ebben a batch-üzemmódban? Melyik korábbi késleltetés-kvantillis értelmezhető, és melyik nem?

35.4.8 Megoldások és indoklás

1. Nem élesítjük a challengert. A health/ready csak a szolgáltatás élő állapotát jelzi; a modellminőségi kapu külön feltétel. A challenger Brier-pontszáma 0,031 **rosszabb**, mint a champion 0,028 értéke. A Brier-pontszám átlagos négyzetes valószínűségi hiba, minél kisebb, annál jobb; a 23.4. alfejezetben a kalibrált modell értékelésénél vezettük be. Ráadásul a canary p95 32 ms is túllépi az 1,2-szeres sávot a champion 18 ms-hoz képest (elfogadás: $18 \cdot 1{,}2 = 21{,}6$ ms). Mindkét kapu a challenger ellen szól; a jelölt visszakerül debug állapotba.

2. A kiegészített függvény:

```
def build_manifest(model_file: Path, split_file: Path, schema: str,
                   threshold: float, code_commit: str) -> dict:
    return {
        "release": "northstar-inference-1.4.0",
        "model_file": str(model_file),
        "model_hash": "sha256:" + sha256_hex(model_file),
        "split_file": str(split_file),
        "split_hash": "sha256:" + sha256_hex(split_file),
        "code_commit": code_commit,
        "schema_version": schema,
        "decision_threshold": threshold,
        "deps_lock": "uv.lock",
        "created_at": None,
    }
```

Az invariant: ha sem a modellfájl, sem a splitfájl tartalma nem változik, akkor a két hash megegyezik a manifestben tárolt értékkel; ezt betöltés vagy átadás előtt újra leellenőrizzük, és eltérésnél hibát adunk. A splitmanifest önmagában is szükséges, mert a 31.4. alfejezet lineage a modell metrikáit és a tanító–teszt felosztást együtt rögzíti.

3. A `/ready` a betöltött modell sémáját mutatja, nem a bejövő kérés való érvényességét. A release manifestben tárolt `schema_version` például `2026-06-01-northstar-v1`, a bejövő kérések pedig a `2026-07-01-northstar-v1` sémát használják. A szolgáltatás a feature-ek egy részét nem találja, a szerver validációja 422-es hibát ad. Ellenőrzés: a naplóban a rossz kérések mind azonos séma-mezőre panaszkodnak; a manifest sémája és az éles gateway aktuális kéréssémája nem egyezik. Döntés: a challenget a manifest sémafrissítéséig ki kell vonni a forgalomból, mert itt nem a modell hibázik, hanem a release-illesztés rossz.

4. A batch load budget a fal-időre és az erőforrás-kihasználtságra szól: 50 000 sor, 10 perc, tehát legalább 84 sor/másodperc átlagos feldolgozási sebesség. A p95-ös kérésenkénti késleltetés nem közvetlenül alkalmazható, hiszen nincs interaktív felhasználói várakozás. Helyette a teljes batch futás végfalas ideje a cél, a p95/p99 itt a *sorfeldolgozási idők* eloszlásából számítható, ha részletes diagnosztikát akarunk. Az egyenletes teljesítményhez a batch-feladat 4 workerre osztható, és az egyes workerek áthaladási ideje a párhuzamosítás overheadjét is mutatja.

35.4.9 Gyakori tévedés

A leggyakoribb hiba az, hogy a `/health` 200-as státuszát modellminőségi bizonyítéknak tekintik. **A szolgáltatás egészséges státusza nem azonos a modell statisztikai teljesítményével.** Ellenpélda: kicseréljük a modellt egy konstans komponensre, amely minden kérésre 0,5 valószínűséget ad. A `/health` és `/ready` is 200-as marad, hiszen a processz él, és a modell betöltődött. A Brier-pontszám viszont $0{,}5 \cdot 0{,}5 = 0{,}25$-re romlik, ami a 23.4. alfejezet szerint még a legkevésbé informatív baseline-t sem javítja. Ezért van szükség a shadow-metrikákra, és ezért nem elég a rollout-monitorozásban a liveness-re hagyatkozni.

35.4.10 Önellenőrzés

- El tudod magyarázni a `/health` és `/ready` különbségét egy-egy mondatban?
- A p95 értékét hogyan számítod a rendezett mintából, és miért robusztusabb az átlagnál?
- Mi a canary és mi a shadow szerepe? Melyik a modellminőség, melyik a szolgáltatási megbízhatóság kapuja?
- Mit ellenőriz a `load_verified` függvény a betöltés előtt, és milyen biztonsági határt tart be?

Visszaállításkor az előző modellartefaktum mellett az előző request-sémát és a futtatókörnyezet kompatibilitását is ellenőrizni kell.

35.4.11 Rövid összefoglaló

A release nem egyetlen modellfájl felmásolása, hanem verzióhoz kötött jegyzőkönyv: a modell hash-e, a kód, a split, a séma és a döntési küszöb együttese. A `/health` a processz életét, a `/ready` a kiszolgáló képességet mutatja; egyik sem helyettesíti a modellminőségi kaput. A canary a szolgáltatást ellenőrzi, a shadow a modellt méri, a rollback pedig előre rögzített feltételek alapján pointer-váltás. A load budget a késleltetés kvantiliseire épül, mert a felhasználó a farkat tapasztalja, nem az átlagot. A következő lépésben ez a release-folyamat távoli vagy konténeres környezetbe kerül, ahol a kiadás és a visszaállítás más gépen is ismételhető.

36 Monitorozás, adatvédelem és modelléletciklus

36.1 Adat- és predikciómonitoring címke nélkül

Belépő · szint: haladó · idő: 90 perc · eszköz: Python, Docker

Előfeltétel: 35.4. alfejezet · Célkészség: Adat- és predikciómonitoring címke nélkül · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

36.1.1 Helyzet és megfigyelési egység

A Northstar modell championja már batch predikcióban fut a rendelés pillanatában. A működő kiadásnál 35.1. alfejezet rögzítette a bemeneti sémát és 31.4. alfejezet a modellverziót, de a valódi célváltozó, a `target_late`, csak a kézbesítés után keletkezik. Az első hetekben tehát a monitoringgyakran csak magát az inputot, a feature-öket és a prediktált valószínűséget látja. A megfigyelési egység: egy batch predikciós sor, azaz egy rendeléshez tartozó feature-vektor a `score` oszloppal. A cél az, hogy a címke nélküli ablakban is riasztást adjunk, ha a feldolgozási szerződés sérül vagy a bemeneti eloszlás el távolodik a referenciától. Az ellenőrzés feltétele: a jelentés sémaállapotot, `distance_psi`-t és a `score` átlagának eltolódását adja, majd `OK`, `REVIEW`, `WARN` vagy `CRITICAL` állapotot képez. Előzetes becslés a szintetikus ablakokra: a normál ablak PSI-je 0,1 alatt marad, az eltolt ablaké a 0,15–0,25 sávba kerülhet, a sémahibás ablak pedig biztosan kritikus, mert negatív távolságot és hiányzó oszlopot tartalmaz.

36.1.2 A négy drift fogalom és a baseline window

A „drift” itt nem egyetlen jelenség, ezért először négy fogalmat különítünk el.

Típus	Amit mér	Címke kell?	Tipikus ok
schema drift	a bemeneti szerződés sérülése	nem	upstream hiba, új forrás, hiányzó mező
data drift	$P(X)$ változása a baseline-hoz képest	nem	szezon, kampány, mérési változás
prediction drift	a prediktált `score` eloszlásának változása	nem	data drift, küszöbhiba, modellcsere
concept drift	$P(y \mid x)$ változása	igen	ügyfélviselkedés, folyamatváltozás

A **schema drift** nem statisztikai kérdés. Ugyanaz a szerződés, amelyet a batch inference 35.1. alfejezet sémavalidációnál használt: mely oszlopoknak kell léteznünk, milyen dtype-pal és tartománnyal. Ha a `region` oszlopban megjelenik egy `west` érték, vagy a `distance_km` negatív

lesz, az szerződésszegés, nem eloszlásváltozás. Hiába hibátlan a statisztika, ha a sorokat eleve rossz formában töltöttük be.

A **data drift** az input eloszlását veti össze egy rögzített **baseline window**-val. A baseline ablak az az időszak, amikor a modell kiadásához tartozó validáció szerint ismert volt a teljesítmény; példánkban 2025-04-01 és 2025-04-30. A baseline window nem csúszik naponta, mert a folyamatos frissítés lassan követné a trendet, és a riasztás elveszne. Új baseline csak explicit modellváltáskor, a registry jóváhagyásával 31.4. alfejezet képződik.

A **prediction drift** a modell kimenetét, a `score` eloszlását figyeli. Ha a prediktált valószínűség átlaga vagy a magas kockázatú részarány elmozdul, az lehet data drift következménye, de lehet koncepcionális változás vagy rossz modellverzió is. Önmagában vizsgálati jel, nem bizonyíték.

A **concept drift** a feltételes kimeneteloszlás változása; nem feltétlenül jelent romló teljesítményt, mert annak mértéke a modellre és az értékelési eloszlásra is függ: ugyanaz a feature-vektor már más $P(y \mid x)$ valószínűséggel jár. Ehhez címke kell, ezért a jelen alfejezetben csak annyit mondhatunk: a címke nélküli eszközök nem azonosítják, legfeljebb gyanút keltenek.

36.1.3 Eloszlástávolság és mintanagyság-függő tesztértelmezés

A leggyakoribb két mérőszám a PSI és a Kolmogorov–Smirnov-statisztika.

A PSI kategorizált binningre épül:

$$PSI = \sum_{i=1}^{K} (q_i - p_i) \ln\left(\frac{q_i}{p_i}\right),$$

ahol p_i a baseline ablakbeli relatív gyakoriság az i-edik binben, q_i a jelen ablakbeli relatív gyakoriság ugyanott, K a binek száma, ln természetes logaritmus. A mennyiség dimenziómentes, mert gyakoriságok hányadosából számoljuk. Kézi kontroll: ha $p = (0{,}5; 0{,}5)$ és $q = (0{,}4; 0{,}6)$, akkor

$$PSI = (0{,}4 - 0{,}5)\ln(0{,}4/0{,}5) + (0{,}6 - 0{,}5)\ln(0{,}6/0{,}5) \approx 0{,}0405.$$

A formula szimmetrikus: a referencia és a jelen ablak felcserélése azonos értéket ad.

A KS-statisztika folytonosabb leírást ad:

$$D_{KS} = \sup_{x \in \mathbb{R}} \left| \hat{F}_{\text{ref}}(x) - \hat{F}_{\text{cur}}(x) \right|,$$

ahol $\hat{F}$ a megfelelő ablak empirikus eloszlásfüggvénye. A `D_KS` a két eloszlásfüggvény legnagyobb függőleges távolsága, szintén dimenziómentes. A `scipy.stats.ks_2samp` e mellé p-értéket is ad, de itt a matematikai híd kulcsa: **a p-érték erősen mintanagyság-függő**. Kis mintán még nagy D is könnyen lehet véletlen; nagy mintán egészen apró, működés szempontjából jelentéktelen eltolódás is „szignifikáns” lehet. Ezért a monitoring riasztás küszöbét hatásméretre, tipikusan PSI-re tesszük, a p-értéket pedig csak vizsgálati támogatásként használjuk. Egy 10 binre számolt PSI-nél a 0,1 alatti érték e projektben alacsony, a 0,1–0,2 közötti vizsgálandó, a 0,2 feletti pedig erős riasztási szintet jelezhet. A PSI érzékeny a binhatárokra, a ritka kategóriákra és az alkalmazott simításra; önmagában nem bizonyít eloszlásazonosságot. Ezek működési küszöbök, nem statisztikai törvények; a projekt a saját döntési költségéhez igazíthatja őket.

36.1.3.1 Vaktárgyú küszöb, két szezonális kontrollablak és triage

A driftküszöböt **vaktárgyúan** kell rögzíteni: a küszöbértéket a címkék és a modellhiba ismerete nélkül, előre definiáljuk (például PSI 0,1/0,2), és nem igazítjuk utólag ahhoz, hogy éppen mikor romlott az AUC. Ha a küszöböt a mért modellromláshoz hangoljuk, a riasztás körkörös: csak akkor jelez, amikor már tudjuk a hibát, és nem segít korábban.

Két **szezonális kontrollablak** kell ugyanahhoz a naptári időszakhoz: az előző év azonos hete és a szomszédos, driftmentesnek ismert hét. Ha a jelenlegi ablak PSI-je magas, de a kontrollablakokban is hasonlóan magas, a jelenség valószínűleg szezonális vagy forrásoldali, nem a modell romlása. A kontrollablakokat azonos feature-készlettel, azonos binszámmal és lehetőleg hasonló mintanagysággal kell számolni.

A **triage** folyamat:

Jel	Első döntés	Teendő	Felelős
schema drift	`STOP`	no-score mód, upstream javítás, újrafuttatás	adatplatform
data drift, kontrollban is jelen	`WARN`	szezonális magyarázat rögzítése, nincs retrain	monitoring
data drift, kontrollban nincs	`INVESTIGATE`	forrás-, régió-, kampányellenőrzés	adatminőség
prediction drift	`INVESTIGATE`	score-összetétel, küszöb, modellverzió ellenőrzése	ML-üzemeltetés
concept drift (csak címkével)	`RETRAIN_CANDIDATE`	érett kohorsz, challenger, intervallum	ML-csapat

```python
def drift_triage(schema_ok, psi, control_psi,
                 warn=0.1, stop=0.2, control_tol=0.05):
    if not schema_ok:
        return {"action": "STOP", "reason": "schema drift"}
    if psi >= stop:
        if control_psi is not None and control_psi >= stop - control_tol:
            return {"action": "WARN", "reason": "szezonális kontrollban is jelen"}
        return {"action": "INVESTIGATE", "reason": "erős drift, kontroll nélkül"}
    if psi >= warn:
        return {"action": "WARN", "reason": "vizsgálandó drift"}
    return {"action": "OK", "reason": "küszöb alatt"}
```

Az elfogadási kontroll: megváltozó régióösszetétel mellett a riport **nem állítja tényként** a modellhibát; `WARN` vagy `INVESTIGATE` a döntés, a modellromlás csak címkés, érett kohorszból igazolható. A `tests/test_drift_triage.py` a schema-stopot, a szezonális WARN-t, a kontroll nélküli INVESTIGATE-et és a küszöb alatti OK-t teszteli.

36.1.4 Teljes minimális munkapélda

A labor egy `lab/monitoring_lab` mappában él, és négy ablakot állít elő: egy referenciaablakot és három vizsgált ablakot. A generálás Python standard libraryt, NumPy-t és SciPy-t használ.

```python
# drift_report.py
"""Séma-, input- és predikciódrift jelentés szintetikus ablakokra."""
from __future__ import annotations
import argparse
import json
from pathlib import Path
import numpy as np
import pandas as pd

SCHEMA = {
    "distance_km": {"kind": "numeric", "min": 0.0, "max": 500.0},
    "basket_minor": {"kind": "numeric", "min": 0.0},
    "quantity_sum": {"kind": "integer", "min": 1, "max": 50},
    "segment": {"kind": "category", "allowed": {"consumer", "business"}},
    "priority_flag": {"kind": "integer", "min": 0, "max": 1},
    "score": {"kind": "numeric", "min": 0.0, "max": 1.0},
}

def check_schema(df: pd.DataFrame) -> list[str]:
    issues: list[str] = []
    for col, spec in SCHEMA.items():
        if col not in df.columns:
            issues.append(f"missing:{col}")
            continue
        s = df[col]
        if spec["kind"] == "numeric" and not pd.api.types.is_numeric_dtype(s):
            issues.append(f"dtype:{col}")
            continue
        if spec["kind"] == "integer" and not pd.api.types.is_integer_dtype(s):
            issues.append(f"dtype:{col}")
            continue
        if spec["kind"] == "category" and not set(s.dropna().unique()) <= spec["allowed"]:
            issues.append(f"allowed:{col}")
        if "min" in spec and s.min(skipna=True) < spec["min"]:
            issues.append(f"min:{col}")
        if "max" in spec and s.max(skipna=True) > spec["max"]:
            issues.append(f"max:{col}")
        if s.isna().any():
            issues.append(f"null:{col}")
    return issues

def make_window(n: int, seed: int, mean_dist: float,
                schema_bad: bool = False) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    df = pd.DataFrame({
        "distance_km": rng.normal(mean_dist, 25.0, n).clip(0.0, 500.0),
        "basket_minor": rng.integers(200, 50_000, n),
        "quantity_sum": rng.integers(1, 12, n),
        "segment": rng.choice(["consumer", "business"], n, p=[0.8, 0.2]),
        "priority_flag": rng.integers(0, 2, n),
    })
    z = -1.8 + 0.012 * df["distance_km"] + 0.35 * df["priority_flag"]
    df["score"] = 1.0 / (1.0 + np.exp(-z))
    if schema_bad:
        df = df.drop(columns=["priority_flag"])
        df.loc[0, "distance_km"] = -10.0
    return df

def psi_binned(ref: np.ndarray, cur: np.ndarray, bins: int = 10) -> float:
    edges = np.quantile(ref, np.linspace(0.0, 1.0, bins + 1))
    edges[0] = -np.inf
    edges[-1] = np.inf
    p = np.histogram(ref, bins=edges)[0] / len(ref)
```

```
    q = np.histogram(cur, bins=edges)[0] / len(cur)
    eps = 1e-4
    p = np.where(p == 0, eps, p)
    q = np.where(q == 0, eps, q)
    return float(np.sum((q - p) * np.log(q / p)))

def report(baseline: pd.DataFrame, current: pd.DataFrame,
           current_name: str, row_floor: int = 500) -> dict:
    schema_issues = check_schema(current)
    if "distance_km" in current.columns:
        psi_dist = psi_binned(
            baseline["distance_km"].to_numpy(),
            current["distance_km"].to_numpy())
    else:
        psi_dist = float("nan")
    score_delta = float(current["score"].mean() - baseline["score"].mean())
    if schema_issues:
        state = "CRITICAL"
    elif psi_dist > 0.2 or abs(score_delta) > 0.05:
        state = "WARN"
    elif len(current) < row_floor:
        state = "REVIEW"
    else:
        state = "OK"
    return {
        "window": current_name,
        "rows": len(current),
        "schema_issues": schema_issues,
        "distance_psi": round(psi_dist, 4),
        "score_mean_delta": round(score_delta, 4),
        "state": state,
    }

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--baseline", type=Path, required=True)
    ap.add_argument("--current", type=Path, required=True)
    ap.add_argument("--window-name", required=True)
    args = ap.parse_args()
    rep = report(pd.read_csv(args.baseline), pd.read_csv(args.current),
                 args.window_name)
    print(json.dumps(rep, indent=2))
    code = {"OK": 0, "REVIEW": 1, "WARN": 1, "CRITICAL": 2}[rep["state"]]
    return code

if __name__ == "__main__":
    raise SystemExit(main())
```

A windowokat külön file gyártja:

```
# make_windows.py
from pathlib import Path
from drift_report import make_window

out = Path("data")
out.mkdir(exist_ok=True)
make_window(2000, 101, 50.0).to_csv(out / "reference.csv", index=False)
make_window(800, 102, 50.0).to_csv(out / "current_normal.csv", index=False)
make_window(800, 103, 65.0).to_csv(out / "current_shift.csv", index=False)
make_window(800, 104, 50.0, schema_bad=True).to_csv(
    out / "current_schema_bad.csv", index=False)
```

Futtatás Linux/macOS bash alatt, `lab/monitoring_lab` munkakönyvtárból:

```
python make_windows.py
python drift_report.py --baseline data/reference.csv \
  --current data/current_normal.csv --window-name current_normal
python drift_report.py --baseline data/reference.csv \
  --current data/current_shift.csv --window-name current_shift
python drift_report.py --baseline data/reference.csv \
  --current data/current_schema_bad.csv --window-name current_schema_bad
```

Ha a script `WARN` vagy `REVIEW` állapotot ad, a folyamat 1-es, `CRITICAL` esetén 2-es kilépési kódot használ, így a pipeline-ban 32.4. alfejezet is automatikus kapu lehet.

36.1.5 Eredmény értelmezése és kézi kontroll

A könyv írásakor nem futtattuk a fenti kódot, ezért a konkrét JSON-t szemléltetőnek, a generátorparaméterekből levezetettnek tekintjük. A `current_normal` ablakban a `distance_psi` közel 0, a `score_mean_delta` közel 0, az állapot várhatóan `OK`. A `current_shift` ablakban a `distance_km` átlagát 50-ről 65 km-re növeltük; a reference kvantilisek alapján a felső binekbe több sor kerül, ez a PSI-t a 0,15–0,25 sávba viheti, és a `score` átlaga is emelkedik, mert a demó `score` egyenlete a távolságtól függ. Ez együtt `WARN`. A `current_schema_bad` ablakból hiányzik a `priority_flag`, és az első sor távolsága negatív; a sémaellenőrzés `missing:priority_flag` és `min:distance_km` hibákat ad, az állapot `CRITICAL`. A kézi kontroll: a PSI-hoz a bin-arányokból a korábbi képlet pontosan kiszámítható; a `check_schema` semmilyen `WARN`-állapotot nem adhat, amíg egyetlen sémahiba is van, mert akkor a `CRITICAL` ág az első.

A példában a `score`-ot a feature-ökből számítjuk, de ez csak tanulási modell, nem a regisztrált champion. A valódi rendszerben a `score` a batch manifestből jön 35.1. alfejezet, és a monitoring nem a feature-ökből újraszámolja, hanem a naplózott kimenetet ellenőrzi. Azt is pontosan mondjuk ki: a PSI a `distance_km`-re vonatkozik, nem az egész adatsorra; a többi oszlopra külön PSI számítható, de a példa a legérzékenyebb folytonos inputot emeli ki.

36.1.6 Feladatok

1. **Értelmezés.** A jelentés szerint `distance_psi = 0.31`, `score_mean_delta = 0.00`. Miért nem jelenti ez, hogy a modell használhatatlan? Mondj egy olyan mechanizmust, amely ilyen párost adhat, és írd le, milyen további adat kell a tényleges döntéshez.

2. **Megvalósítás.** Bővítsd a `report` függvényt `high_risk_share` kiszámításával, ahol a magas kockázatú sor definíciója `score >= 0.7`. A referencia és a jelen ablak aránykülönbségére `risk_delta` néven figyelmeztess, ha `abs(risk_delta) > 0.05`. Készíts hozzá egy 4 soros DataFrame-példát, amelyben a referencia `score` értékei `[0.6, 0.6, 0.8, 0.8]`, a jelené `[0.6, 0.8, 0.8, 0.8]`, és várd el a `WARN` állapotot.

3. **Hibadiagnózis.** A monitoring `WARN` állapotot ad: `distance_psi = 0.24`, `score_mean_delta = 0.01`. Milyen sorrendben vizsgálnád az upstream adatot, a modellverziót és a score-kvantiliseket? Miért nem írhatod a jegyzőkönyvbe, hogy „igazolt teljesítményromlás” történt?

4. **Transzfer kis ablakra.** Tegyük fel, hogy a következő batch csak 200 sort tartalmaz, és a mért `distance_psi = 0.19` a küszöb alatt van. Hogyan változtatja ez a döntést? Módosítsd a `report` függvényt úgy, hogy 500 sor alatt `REVIEW` állapotot állítson be, és a jelentésbe `note` mezőt is írjon: „kis minta, vizsgálat javasolt”. A `row_floor` paraméter már létezik, tehát a feladat a hívásoldal és a `note` mező kiegészítése.

36.1.7 Megoldások és indoklás

1. A PSI csak $P(X)$ eltolódást mér, nem $P(y \mid x)$-et. Ha a modell a `distance_km`-re gyenge érzékenységű, akkor az input eltolódás nem változtatja a predikciót; az is lehet, hogy a modell épp ebben a tartományban kalibrált marad. A további adat: a későbbi címkék, azaz a kézbesítés után számolt `target_late`, amit a következő alfejezet 36.2. alfejezet használ, vagy legalább egy stresszteszt a 26.4. alfejezet módszeréből, hogy a score érzékenységét ismerjük.

2. A `report` bővítése:

```
ref_high = (baseline["score"] >= 0.7).mean()
cur_high = (current["score"] >= 0.7).mean()
risk_delta = float(cur_high - ref_high)
```

majd a `WARN` feltétel:

```
elif psi_dist > 0.2 or abs(score_delta) > 0.05 or abs(risk_delta) > 0.05:
```

A példában a referencia magas aránya $2/4 = 0{,}5$, a jelené $3/4 = 0{,}75$, `risk_delta = 0.25`, tehát `WARN`. Ellenőrző invariant: az adatsor hossza kicsi, de a `mean()` itt egyszerű arány; a kapott érték kerekítés nélkül is pontosan 0,25.

3. Először a sémaellenőrzést kell megerősíteni: van-e új adatforrás, mértékegységváltás vagy csonkolt batch? Másodszor a modellverziót: a registry 31.4. alfejezet szerint a champion hash-e megegyezik-e a futtatott modellel? Harmadszor a score eloszlás kvantiliseit: az átlag alig változhat úgy, hogy a farok eltolódik, ezért a 0,1; 0,5; 0,9-es kvantilisek is kellenek. A teljesítményromláshoz címke kell, tehát a „igazolt” szó hibás lenne; a jegyzőkönyv legfeljebb „vizsgálati jel: input drift score-stabilitással” állítást tehet.

4. Kis mintán a PSI becslés varianciája nagy, ezért a küszöb alatti érték nem megnyugtató, csak nem riaszt. A `REVIEW` állapot nem blokkoló hiba, hanem emberi vizsgálati igény. A módosítás:

```
state = "REVIEW" if len(current) < row_floor else "OK"
rep["note"] = "kis minta, vizsgálat javasolt" if len(current) < row_floor else ""
```

A küszöb feletti PSI továbbra is `WARN` legyen, tehát a feltételsorrend ne változzon: séma, nagy eltérés, majd kis minta.

36.1.8 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés az, hogy a drift **kimutatása** a modell használhatatlanságát jelenti. Ellenpélda: a `current_shift` ablakban a `distance_km` erősen eltolódik, a PSI magas, de a demó `score` függvénye csak mérsékelten változik, és ha a tényleges `target_late` is alig változik, a modell továbbra is jól dönt. A PSI tehát csak annyit mond: „a jelenlegi input már nem hasonlít a baseline-ra”; a következményt külön kell igazolni.

A másik irány sem bizonyíték. Ha a `score` eloszlása napokon át változatlan, az nem jelenti, hogy a modell jó; lehet, hogy minden szegmensben nő a késés, mert a szállító folyamat megváltozott, és a modell ugyanazt a valószínűséget adja, miközben $P(y \mid x)$ eltolódott. Ezt a concept driftet a címke nélküli monitor nem látja; a késői címke 36.2. alfejezet fogja feltárni. A monitoring tehát riasztási és vizsgálati eszköz, nem a modellminőség független tanúsítványa.

Önellenőrzés: Mi a különbség a séma- és a data drift között a beavatkozás sorrendjében? Miért veszélyes a KS-teszt p-értékét önmagában riasztási küszöbként használni? Mikor szabad megváltoztatni a baseline window-t?

36.1.9 Rövid összefoglaló

A címke nélküli monitoring négy különböző jelet kezel: a séma drift szerződésszegést jelez, azonnal kritikus; a data drift PSI-vel a baseline-hoz méri a bemeneti eloszlást; a prediction drift a score eloszlását; a concept drift csak későbbi címkével igazolható. A baseline window a champion validációs időszaka, és modellváltásig rögzítve marad. A PSI hatásméret-küszöb, a KS-féle p-érték mintanagyság-függő támogatás; kis mintán a bizonytalanságot `REVIEW` állapottal kell jelölni. A következő alfejezet 36.2. alfejezet a megérkező címkét köti a tényleges teljesítményhez és az újratanításhoz.

36.2 Késő címke, teljesítmény és újratanítás

Belépő · szint: haladó · idő: 90 perc · eszköz: Python, Docker

Előfeltétel: 36.1. alfejezet · Célkészség: Késő címke, teljesítmény és újratanítás · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

36.2.1 Cél és konkrét helyzet

A Northstar rendeléskori modellje azt a valószínűséget becsüli, hogy egy rendelés kézbesítése 7 napnál későbbi lesz. A predikció a `predicted_at = order_ts` pillanatban készül, tehát semmilyen későbbi kézbesítési adat nem lehet predictor. A `target_late` címke viszont késve, tényleges kézbesítéskor válik ismertté: `label_available_at = actual_delivery_at`. Ebből azonnal következik a monitoring-probléma: a friss predikciók egy része még nem értékelhető, mert a címke nem érkezett meg.

A feladat egy gördülő teljesítményriport tervezése, amely az értékelés időpontjában ténylegesen rendelkezésre álló címkéket használja. A bemenet két tábla:

Tábla	Megfigyelési egység	Fontos mezők
`predictions`	egy predikció, `(model_id, order_id)`	`predicted_at`, `predicted_late_prob`, `features_schema_version`
`labels`	ugyanaz a rendelés, `order_id`	`label_available_at`, `target_late`

A riport as-of időpontja legyen `V = 2025-07-15T00:00:00+00:00`, a predikciós ablak pedig a július 1-jei öt rendelés. Az előzetes becslés: körülbelül 80%-os fedettség várható, mert egy szélsőségesen késő kézbesítés címkéje még hiányzik. A kérdés: mi mondható erről az ablakról tényként, és mikor szabad újratanítást indítani?

36.2.2 Fogalmi magyarázat: késleltetett címke, mérési ablak és visszacsatolás

Késleltetett címke (delayed label) olyan célváltozó, amely a predikció után csak későbbi eseményből ismerhető meg. A Northstar szerződésében a késés nem egységesen 7 nap: a nem késő rendelés hamarabb, a késő rendelés később kap címkét. A címke hiánya tehát **nem véletlen hiány**, hanem magával a célváltozóval összefüggő szelekció: a hiányzó sorok nagyobb eséllyel pozitívak. Ezt a 13.1. alfejezetben megismert mintavételi kérdéshez hasonlóan kell kezelni.

Értékelési ablak (evaluation window) két időkorlátból áll:

- predikciós oldal: az adott rendelés `predicted_at` értéke a vizsgált intervallumba esik;
- címkeoldal: `label_available_at <= V`.

Egy rendelés csak akkor szerepel az értékelés nevezőjében, ha a predikció az ablakon belül készült. A számlálóba viszont csak azok kerülnek, amelyek címkéje `V`-ig megérkezett:

$$c_{m,w} = \frac{N^{\text{lab}}_{m,w}}{N^{\text{pred}}_{m,w}},$$

ahol m a modellazonosító, w az ablak, N^{pred} az összes ablakbeli predikció darabszáma (a kódban `n_predicted`), N^{lab} a rendelkezésre álló címkéjű soroké (a kódban `n_labeled`). A mennyiség dimenzió nélküli arány, értéke $[0, 1]$-ben van. **A coverage nem modellminőség, hanem adatelérhetőségi mérőszám**: azt mondja meg, mekkora részmintáról beszélhetünk egyáltalán.

A részmintán számolható diszkriminációs mutató, például az AUC, rangpár-képlettel:

$$\text{AUC}_w = \frac{C + 0{,}5T}{P \cdot N},$$

ahol C azon pozitív–negatív párok száma, ahol a pozitív rendeléshez magasabb pontszám tartozik, T a döntetlen pároké, P a pozitív, N a negatív minták száma. A nevező az összes lehetséges pozitív–negatív pár. Véletlen rangolásnál AUC $= 0{,}5$, tökéletes elválasztásnál 1.

Kézi kontrollpélda a laborhoz:

order_id	target_late	predicted_late_prob	label elérhető V-kor?
O1	0	0.10	igen
O2	0	0.30	igen
O3	1	0.20	igen
O4	1	0.90	igen
O5	?	0.85	nem

A rangpárok: O3 pozitív pontszáma 0,20 egy negatívnál nagyobb, egynél kisebb; O4 pontszáma mindkét negatívnál nagyobb. Így $C = 3$, $T = 0$, $P = 2$, $N = 2$, tehát AUC $= 3/4 = 0{,}75$. A coverage $4/5 = 0{,}80$. Fontos: az O5 hiányzó sor valószínűleg pozitív és magas pontszámú, ezért a 0,75 csak a négy elérhető címkéjű sorra igaz, nem a teljes ötelemű ablakra.

Visszacsatolási hurok (feedback loop) akkor keletkezik, ha a modell kimenete befolyásolja a folyamatot, amelyből a címke születik. Ha például a magas kockázatú rendeléseket gyorsított kézbesítési sávba terelik, akkor a `target_late` már nem a természetes kézbesítési folyamat kimenete, hanem a modell által befolyásolt folyamaté. Ilyenkor a javuló AUC nem feltétlenül

jobb modellt, hanem a beavatkozás hatását is méri. A riportnak ezért rögzítenie kell, hogy a champion modell mely rendeléseken kapott beavatkozást, és a kauzális állítás külön véletlenített értékelést igényel.

Újratanítási feltétel (retrain trigger) nem maga a modell, hanem explicit szabályzat. Akkor érdemes challenger modellt indítani, ha:

1. a mérési ablak már érett: `coverage >= 0.90` és $N^{\text{lab}} \geq 300$;
2. legalább két egymást követő, előre definiált és megfelelően címkézett ablakban AUC $<$ $\max(0{,}75,\ \text{baseline} - 0{,}03)$;
3. nincs aktív adatminőségi vagy sémaincidens.

A **champion** az éles modell, a **challenger** pedig a jelölt.

Módszertani korlát. A 90%-os címkefedettség sem teszi automatikusan torzítatlanná a részleges AUC-t: a címke késése összefügghet a tényleges késéssel és a predikcióval. A teljesítményjelentésben jelölni kell a cenzorálást, és szükség szerint csak teljesen érett kohorszt, illetve külön indokolt késéskorrekciót szabad használni. A challenger predikciói naplózódnak, de üzleti beavatkozást nem vezérelnek, amíg független, érett ablakon jobbnak nem bizonyul, és a javulás becslési intervalluma nem zárja ki a nullát.

36.2.3 Minimális munkapélda: as-of riport

Az alábbi szándékosan kis, szintetikus példa standard library `sqlite3`-mal fut. a kód futása még nincs automatizáltan ellenőrizve, ezért a kimenet **szemléltető, elméletileg várt**.

```
import sqlite3

AS_OF = "2025-07-15T00:00:00+00:00"
MIN_COVERAGE = 0.90
MIN_LABELED = 300
BASELINE_AUC = 0.80

PREDICTIONS = [
    ("O1", "champ-v3", "2025-07-01T09:30:00+00:00", 0.10, "fs-2"),
    ("O2", "champ-v3", "2025-07-01T09:31:00+00:00", 0.30, "fs-2"),
    ("O3", "champ-v3", "2025-07-01T09:32:00+00:00", 0.20, "fs-2"),
    ("O4", "champ-v3", "2025-07-01T09:33:00+00:00", 0.90, "fs-2"),
    ("O5", "champ-v3", "2025-07-01T09:34:00+00:00", 0.85, "fs-2"),
]

LABELS = [
    ("O1", "2025-07-03T11:00:00+00:00", 0),
    ("O2", "2025-07-04T08:10:00+00:00", 0),
    ("O3", "2025-07-10T12:00:00+00:00", 1),
    ("O4", "2025-07-13T09:00:00+00:00", 1),
]

def auc_pairs(y_true: list[int], scores: list[float]) -> float | None:
    pos = [s for y, s in zip(y_true, scores) if y == 1]
    neg = [s for y, s in zip(y_true, scores) if y == 0]
    if not pos or not neg:
        return None
    concordant = ties = 0
    for sp in pos:
        for sn in neg:
            if sp > sn:
                concordant += 1
            elif sp == sn:
```

```
                ties += 1
    return (concordant + 0.5 * ties) / (len(pos) * len(neg))

def decide_readiness(coverage, auc, n_labeled):
    if coverage < MIN_COVERAGE:
        return "hold_report", "fedetlen ablak: a részleges AUC nem kezelhető tényként"
    if n_labeled < MIN_LABELED:
        return "wait", "kevés címke a megbízható döntéshez"
    threshold = max(0.75, BASELINE_AUC - 0.03)
    if auc is None:
        return "human_review", "üres AUC, adatfolyam-hiba gyanú"
    if auc < threshold:
        return "retrain_candidate", f"AUC {auc:.3f} < {threshold:.3f}"
    return "none", "champion teljesítménye megfelelő"

con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE predictions (
    order_id TEXT,
    model_id TEXT,
    predicted_at TEXT,
    predicted_late_prob REAL,
    features_schema_version TEXT,
    PRIMARY KEY (model_id, order_id)
);
CREATE TABLE labels (
    order_id TEXT PRIMARY KEY,
    label_available_at TEXT,
    target_late INTEGER
);
""")
con.executemany(
    "INSERT INTO predictions VALUES (?,?,?,?,?)", PREDICTIONS
)
con.executemany(
    "INSERT INTO labels VALUES (?,?,?)", LABELS
)

summary = con.execute(
    """
    SELECT p.model_id,
           COUNT(*) AS n_pred,
           COUNT(l.order_id) AS n_labeled,
           ROUND(1.0 * COUNT(l.order_id) / COUNT(*), 4) AS coverage
    FROM predictions p
    LEFT JOIN labels l
      ON l.order_id = p.order_id
     AND l.label_available_at <= ?
    WHERE p.predicted_at < ?
    GROUP BY p.model_id
    """,
    (AS_OF, AS_OF),
).fetchall()

labelled = con.execute(
    """
    SELECT p.order_id, p.predicted_late_prob, l.target_late
    FROM predictions p
    JOIN labels l ON l.order_id = p.order_id
    WHERE l.label_available_at <= ?
      AND p.predicted_at < ?
    ORDER BY p.order_id
    """,
```

```
    (AS_OF, AS_OF),
).fetchall()

model_id, n_pred, n_labeled, coverage = summary[0]
scores = [row[1] for row in labelled]
y_true = [row[2] for row in labelled]
auc = auc_pairs(y_true, scores)
action, reason = decide_readiness(coverage, auc, n_labeled)

print(
    {"model_id": model_id, "n_pred": n_pred, "n_labeled": n_labeled,
     "coverage": coverage, "auc_on_labeled": round(auc, 3) if auc else None,
     "action": action, "reason": reason}
)
```

A szemléltető várható kimenet:

```
{'model_id': 'champ-v3', 'n_pred': 5, 'n_labeled': 4,
 'coverage': 0.8, 'auc_on_labeled': 0.75,
 'action': 'hold_report',
 'reason': 'fedetlen ablak: a részleges AUC nem kezelhető tényként'}
```

A kód a `label_available_at <= AS_OF` szűrést a joinba teszi, ezért a jövőbeli címke nem számít elérhetőnek. A `predicted_at < AS_OF` a határon éles predikciókat is a nevezőben hagyja, de a fedettség mutatja meg, hogy közülük mennyi érett.

36.2.4 Eredmény értelmezése

A kézi kontroll és a kód ugyanazt adja: öt predikció, négy címke, coverage 0,8, a négy elérhető címkén AUC = 0,75. A riport mégsem mondhatja, hogy „a modell AUC-ja 0,75 ebben az ablakban”. A helyes állítás:

> Az ablak predikcióinak 80%-ára van címke; ezen a nem véletlenül elérhető részmintán az AUC 0,75. A teljes ablakra vonatkozó állítás addig nem tehető, amíg a coverage nem éri el a 0,90-es szabályzatot.

Az $O5$ hiányzó címkéjű rendelés magas, 0,85 pontszámú. Ha később kiderül, hogy valóban késő kézbesítés volt, az öt teljes soron számolt AUC $5/6 \approx 0{,}833$-ra változik. Ez az ellenpélda mutatja, hogy a késleltetett pozitív címkék mechanikusan torzíthatják az interim AUC-t.

Az újratanítási policy külön választja az adatelérhetőségi feltételt a teljesítményfeltételtől. A `hold_report` nem indít újratanítást; előbb meg kell várni az érett ablakot, vagy visszafelé kell tolni a predikciós ablak végét. Ha egy fedett ablakban továbbra is romlik az AUC, akkor jön a `retrain_candidate`, de a champion addig élesben marad.

A riportban mindig szerepelnie kell a `model_id` mezőnek. Ha a champion és a shadow challenger sorait összevonva mérjük az AUC-t, akkor sem a régi, sem az új modellhez nem tartozó állítást nem kapunk. A challenger csak akkor tekinthető előléptethetőnek, ha az aktuális érett ablakon mért AUC-javulás intervalluma nem tartalmaz nullát, és a beavatkozási környezet azonos.

36.2.4.1 Érett kohorsz és cenzorált aktuális kohorsz

A 90%-os címkefedettség **nem** torzítatlansági garancia. Két kohorszt kell megkülönböztetni:

- **Érett kohorsz:** olyan predikciók, amelyeknél a címke késésének 95. percentilise már eltelt, azaz `predicted_at <= V - d95`. Itt a címkék gyakorlatilag teljesen megfigyeltek, és a teljesítmény állítás erre a kohorszra vonatkozik.
- **Cenzorált aktuális kohorsz:** a friss predikciók, ahol a címkék egy része még nem érkezett meg. A coverage 90% is lehet, de a hiányzó 10% nem véletlen: a késő kézbesítések címkéje később érkezik, és ezek a sorok gyakran magasabb pontszámot kapnak.

Számpélda 20 rendeléssel, 10 pozitív és 10 negatív címkével. A teljes kohorsz AUC-ja 0,92. Ha a 90%-os aktuális kohorszból a két **legmagasabb pontszámú pozitív** hiányzik, a rendelkezésre álló 18 soron az AUC 0,90; ha viszont a két **legalacsonyabb pontszámú pozitív** hiányzik, ugyanaz a 90%-os coverage 0,975-ös AUC-t ad. Ugyanaz a lefedettség, más hiányzási mechanizmus, más becslés. Ezért a coverage önmagában nem elég; a hiányzás irányát és a címkekésés predikcióval való összefüggését is jelenteni kell.

```
from datetime import timedelta

def mature_cutoff(as_of, d95_days):
    return as_of - timedelta(days=d95_days)

def split_cohorts(frame, as_of, d95_days):
    cutoff = mature_cutoff(as_of, d95_days)
    mature = frame[frame["predicted_at"] <= cutoff]
    current = frame[frame["predicted_at"] > cutoff]
    return mature, current

def cohort_report(frame, as_of, d95_days, auc_fn):
    mature, current = split_cohorts(frame, as_of, d95_days)
    labeled = current[current["label_available_at"] <= as_of]
    return {
        "mature": {"n": len(mature), "auc": auc_fn(mature)},
        "current": {
            "n": len(current),
            "coverage": len(labeled) / len(current) if len(current) else None,
            "auc_available": auc_fn(labeled) if len(labeled) else None,
            "censored": len(current) - len(labeled),
        },
    }
```

A jelentés szabálya: az érett kohorsz AUC-ja a teljesítménytény; az aktuális kohorsz AUC-ja **interim**, a coverage és a cenzorált sorok számának feltüntetésével. A kettőt nem szabad egyetlen számba összevonni, és a 90% feletti coverage sem engedi meg a „teljes ablak” állítást. A `tests/test_label_cohorts.py` az érett/cenzorált szétválasztást és azt mutatja meg, hogy azonos coverage mellett a hiányzó pozitívok iránya megváltoztatja az AUC-t.

36.2.5 Feladatok

1. **Értelmezés.** A legfrissebb hétnapos ablakban $N^{\text{pred}} = 1000$, $N^{\text{lab}} = 600$, és a riport szerint AUC $= 0{,}71$. Szabad-e közleményt írni arról, hogy a modell teljesítménye ezen a héten 0,71-re romlott? Indokold a coverage és a késleltetett pozitív címkék szerepével.

2. **Implementálás.** Egészítsd ki az előző kódot egy `label_delay_hours` oszloppal, amely a `label_available_at - predicted_at` különbség. A korábbi, már lezárt történeti ablakban a `sorted(label_delay_hours)` lista utolsó 95%-os kvantilisét használva számíts `mature_cutoff = V - d95` értéket, és csak a `predicted_at <= mature_cutoff` sorokat nevezd érettnek. Mutasd meg pszeudokóddal, hogyan kerül az így számított `d95` a riportba.

3. **Hibadiagnózis.** Egy társad riportjában a `predictions` és `labels` összekapcsolásnál ez a feltétel szerepel:

   ```
   LEFT JOIN labels l
     ON l.order_id = p.order_id
    AND l.label_available_at IS NOT NULL
   ```

   `V = 2025-07-15` mellett egy rendelés címkéje `2025-07-12`-én érkezett meg. Miért hamis a riport „coverage = 1.0” állítása? Add meg a javított feltételt, és magyarázd el, miért nem szabad az `IS NOT NULL` önmagában.

4. **Megváltozott feltétel.** A csapat bevezeti azt a beavatkozást, hogy minden `predicted_late_prob >= 0.7` rendelés automatikusan gyorsított logisztikai kezelést kap. Harminc nap múlva a champion AUC-ja meredeken javul. Milyen visszacsatolási hurok állhat a javulás mögött? Milyen kísérleti vagy shadow-kontrolltervezés segítene különválasztani a modell javulását a beavatkozás hatásától?

36.2.6 Megoldások és indoklás

1. megoldás. Nem szabad így közölni. A coverage $600/1000 = 0{,}60$, tehát az ablak 40%-a címke nélküli. A késő kézbesítések címkéje definíció szerint később érkezik, tehát a hiányzó sorokban felülreprezentáltak lehetnek a pozitív esetek. A 0,71 a rendelkezésre álló, önszelektált részmintára vonatkozó interim szám, nem a teljes hét ténye. Helyes közlés: „A vizsgált ablak még nem érett; a rendelkezésre álló 60%-os részen az AUC 0,71. A teljes ablakra vonatkozó következtetés elhalasztva.”

2. megoldás. A Python-pszeudokód így nézhet ki:

```
import math
from datetime import datetime, timedelta

def parse_ts(value):
    return datetime.fromisoformat(value.replace("Z", "+00:00"))

delays = [
    (parse_ts(l) - parse_ts(p)).total_seconds() / 3600
    for p, l in labelled_rows
]
d95_hours = sorted(delays)[math.ceil(0.95 * len(delays)) - 1]
mature_cutoff = parse_ts(AS_OF) - timedelta(hours=d95_hours)
```

Ellenőrző invariáns: a lezárt történeti ablakban a delay-ek legalább 95%-a kisebb vagy egyenlő d_{95}-nél. A riport `d95_days` mezőjét is menteni kell, mert ha a ciklusidő később megváltozik, a mérési ablak visszafelé tolása igazolhatóvá válik. A kritikus korlát: a d_{95} sem garancia, csak stacioner folyamatra érvényes üzemi szabály.

3. megoldás. Az `IS NOT NULL` csak azt vizsgálja, hogy a rendeléshez tartozik-e valamilyen címke a `labels` táblában; azt nem, hogy az adott címke az értékelési időpontban már ismert volt-e. A `2025-07-12`-i címke `2025-07-15`-i as-of nézetből visszatekintve ismert, de a riportnak ezt a jövőt nem lett volna szabad felhasználnia, ha korábbi `V`-t használt volna. A javítás:

```
LEFT JOIN labels l
  ON l.order_id = p.order_id
 AND l.label_available_at <= ?  -- AS_OF/V érték
```

Az `IS NOT NULL` tehát az adatbázis teljes történetére vonatkozik, a `<= V` viszont az as-of pillanatra. A kettő csak akkor egyenértékű, ha a label-tábla tranzakciósan nem tartalmaz jövőbeli címkéket, de erre nem szabad építeni.

4. megoldás. A javulás mögött valószínűleg beavatkozási hurok áll: a modell által kiemelt rendeléseket a gyorsított logisztika részben „megmenti", így a `target_late` csökken. A champion AUC-ja ezt a módosított kimenetet méri, nem az eredeti, beavatkozás nélküli kockázatot. A különválasztáshoz véletlenített kontrollcsoportra van szükség, amely azonos pontszámküszöbű, de nem beavatkozott rendelésekből áll. Ha ilyen véletlenítés nem áll rendelkezésre, a modellre mint prediktív eszközre tett állítást le kell cserélni a „modell + beavatkozási szabályzat" együttes hatásának leírására. A challenger előléptetése ilyenkor a beavatkozási populációra értelmezett üzleti cél szerint dőljön el.

36.2.7 Gyakori tévedés és önellenőrzés

A leggyakoribb hiba: **a friss címke nélküli időszak teljesítményét tényként megadni.** A fenti példa éppen ezt cáfolja: a négy elérhető címkén számolt 0,75 AUC-ból kiindulva valaki azt írta volna, hogy „a modell AUC-ja 0,75", pedig O5 címkéjének megérkezése után a helyes érték $5/6 \approx 0{,}833$. A helyes riport a mértéket, a részminta méretét és a fedettséget együtt írja le.

Önellenőrzés a következő riport készítése előtt:

- Megjelenik-e külön N^{pred}, N^{lab} és coverage?
- A label-szűrés `label_available_at <= V`, nem pedig `IS NOT NULL`?
- A champion és challenger AUC-ja külön `model_id` szerint van-e csoportosítva?
- Megnevezték-e a beavatkozási visszacsatolást, vagy kauzális hatást tulajdonítottak-e a modellnek?

A korábbi 36.1. alfejezet driftriasztásai itt nyerik el a végső értelmüket: a predikcióeloszlás-változás csak gyanú, a teljesítményromlás pedig csak akkor bizonyítható, ha a késleltetett címkék miatti részmintát pontosan kezeljük. A 35.2. alfejezetben bevezetett kötött predikciós szerződés nélkül a `predicted_at` és a `label_available_at` összekapcsolása is sérülne.

36.2.8 Rövid összefoglaló

A késleltetett címkék miatt a friss predikciós ablak csak részben értékelhető. A coverage a rendelkezésre álló címkék arányát méri, nem a modell minőségét; ha 0,90 alatt van, a részleges AUC nem teljesítménytény. A riport as-of szűréssel zárja ki a jövőbeli címkéket, és külön választja az újratanítási feltételt az emberi vizsgálattól. Ha a modell beavatkozást vezérel, a címke visszacsatolási hurkon keresztül a modell hatását is tartalmazza; ilyenkor csak shadow- vagy véletlenített értékelés különíti el a predikció javulását a beavatkozásétól. A champion mellett a challenger naplózva mérhető. A következő 36.3. alfejezetben a hozzáférés, titokkezelés és adatminimalizálás keretezi ugyanezt az éles monitorozási folyamatot.

36.3 Hozzáférés, titkok és adatminimalizálás

Belépő · szint: haladó · idő: 90 perc · eszköz: Python, Docker

Előfeltétel: 36.2. alfejezet · Célkészség: Hozzáférés, titkok és adatminimalizálás · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

36.3.1 Cél és konkrét helyzet

A 36.2. alfejezetben felépített késői címke monitorozó szolgáltatás most a fejlesztői gépén fut, és a 35.3. alfejezet streamlit demója ugyanazt a modellt tölti be. A 32.3. alfejezet CI-ja csak kis adatos teszteket futtat; a titkok és a személyes adatok kezelése viszont a helyi konfigurációban, naplókban és a kifelé irányuló kapcsolatokban dől el. Ebben az alfejezetben egy fiktív projektet, a `northstar-monitor` mappát auditáljuk: megkeressük, hol szivárog titok vagy indokolatlan személyes azonosító, és megtervezzük a javított határokat.

Konkrét bemenetek:

- `config.yaml`: adatbázis-kapcsolat és modellartefaktum-útvonal.
- `.env.example`: a fejlesztőnek szánt, kitöltésre váró környezeti változók.
- `logs/monitor_sample.log`: szándékosan hibás naplóminta.
- `artifacts/manifest.json`: modellverzió és tanítási cutoff, személyes azonosító nélkül.

A megfigyelési egység egy-egy fájlsor. Az audit célja, hogy minden sorhoz besorolást adjon: `inline-secret`, `raw-pii`, `env-reference` vagy `ok`. Előzetes becslésként, mielőtt a kódot látnád: hány sorban várasz valódi titkot, hányban nyers személyes azonosítót, és a konfigurációs fájlban miért lehet nulla valódi titok? Jegyezd fel a becslésedet, a szakasz végén ellenőrizzük.

36.3.2 Fogalmi magyarázat

36.3.2.1 Secret environment

A titok (secret) olyan hitelesítő adat, amelynek ismerete önmagában hozzáférést adhat: jelszó, API-kulcs, token, privát kulcs. A biztonságos munkafolyamat elválasztja a titok tárolását a titok hivatkozásától. A kód nem a tényleges jelszót tartalmazza, hanem környezeti változóra hivatkozik:

```
import os
password = os.environ["DB_PASSWORD"]
```

A fejlesztői `.env` fájl a titkokat tárolja, ezért nem kerülhet verziókezelőbe. A `.env.example` ezzel szemben csak a változóneveket és `replace_me` helyőrzőket tartalmaz, és bekerülhet a repositoryba. Ha a `.env` mégis commitolódik, a puszta törlés a Git történetéből nem elég: a titkot érvényteleníteni, azaz rotálni kell. A környezeti változó sem automatikus védelem; ha a folyamat a környezetét kiírja egy naplóba, a titok ugyanúgy szivárog.

36.3.2.2 Least privilege

A legkisebb jogosultság elve (least privilege) azt mondja, hogy egy folyamat vagy felhasználó csak a feladatához szükséges minimális jogot kapja meg. A monitorozó szolgáltatásnak például olvasnia kell a predikciós táblát, de nincs szüksége `DROP TABLE` jogra; a batch predikció írhat a saját eredménytáblájába, de nem törölheti a nyers rendeléseket. A jogokat célszerű szerepkörönként szétválasztani: a `monitor_readonly` adatbázis-felhasználó más kapcsolati karakterláncot kap, mint a `prediction_writer`. Az audit szkript maga is legyen korlátozott: olvashatja a konfigurációt és a naplómintát, de ne kapjon írási jogot ugyanarra a mappára, ahol a titkok élhetnek.

Kis kontrollpélda: ha a monitorozó folyamat véletlenül `DELETE FROM predictions` utasítást hajtana végre, a least privilege szerint a művelet már az adatbázis-engedély szintjén meghiúsul, mielőtt a Python-kód tovább futna.

36.3.2.3 Logging redaction

A naplóredakció (logging redaction) azt jelenti, hogy a naplóüzenet eleve nem tartalmazhat titkot vagy felesleges személyes azonosítót. Nem a napló utólagos cenzúrázása az elsődleges védelem, hanem a naplózó hívás megtervezése: a logger formátumában nem szerepelhet a teljes kérés, a fejléc, az email vagy a jelszó. Helyette a napló kap egy pszeudonim hivatkozást vagy csak egy logikai tényt.

Rossz napló	Javított napló
`user=customer@example.com`	`user_ref=USER-7F3A auth=ok`
`token=sk-abc123`	
`password=ChangeMe9!`	`db_auth=from-env`
`customer_id=CUST-0042`	`customer_ref=USER-7F3A`

A naplóban a redakció célja nem a szépség, hanem az adatminimalizálás:

Biztonsági teszt. A redakció nem lehet kizárólag regex-alapú: a strukturált naplózásnál engedélyezett mezők listájával (allowlist), titokszkenneléssel és jogosultsággal védett naplótárolóval kell kiegészíteni. Az álnevesítés nem tesz személyes adatot automatikusan anonimmá. ha a diagnózishoz elég tudni, hogy a hitelesítés sikeres volt, akkor a jelszó soha ne jelenjen meg.

36.3.2.4 Naplózási allowlist, pszeudonimizálás és szivárgásteszt

A naplómezőket **allowlist** szabályozza: csak az engedélyezett kulcsok kerülhetnek a strukturált eseménybe. Tiltott a nyers `customer_id`, `order_id`, e-mail-cím, token, jelszó, API-kulcs és a teljes kérés- vagy választörzs.

Engedélyezett mező	Példa	Miért biztonságos
`event`	`"predict"`	művelettípus
`event_ref`	HMAC-rövidítés	visszafejtés nélkül korrelálható
`model_version`	`"northstar-late-v3"`	nem személyes adat
`prediction`	`0.31`	modellkimenet
`status`	`"ok"`	állapot
`latency_ms`	`12`	működési mérőszám

A pszeudonim azonosítót ne sima SHA-256 hashből képezzük, mert a kis kártyaszámú `order_id`-készlet visszafejthető; használjunk titkos kulcsú HMAC-ot:

```
import hashlib
import hmac

def event_ref(value: str, key: bytes) -> str:
    return hmac.new(key, value.encode("utf-8"), hashlib.sha256).hexdigest()[:12]
```

A `key` környezeti változóban vagy titokkezelőben él, nem a forrásban. A `customer_ref`/`event_ref` tehát nem anonim, de a naplóból nem olvasható vissza a nyers azonosító; a kulcsot külön kell védeni és rotálni.

Az automatikus szivárgásteszt reprezentatív naplósorokon fut, és két réteget ellenőriz:

1. **Tiltott minta:** e-mail, `sk-`/`AKIA`/`ghp_` jellegű token, `password=`, `token=`, `api_key=`, nyers `customer_id=` vagy `order_id=` a szabad szövegben.
2. **Sémamező:** a strukturált esemény kulcsai részhalmaza az allowlistnek; ismeretlen kulcs vagy tiltott kulcs bukik.

```
import re

ALLOWED_FIELDS = {"event", "event_ref", "model_version", "prediction",
                  "status", "latency_ms"}
FORBIDDEN_PATTERNS = [
    re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+"),
    re.compile(r"\b(sk-|AKIA|ghp_)[A-Za-z0-9_-]{8,}"),
    re.compile(r"\b(customer_id|order_id)\s*=\s*\S+"),
    re.compile(r"\b(password|token|api_key)\s*=", re.I),
]

def scan_log_event(event: dict) -> list[str]:
    errors = []
    unknown = set(event) - ALLOWED_FIELDS
    if unknown:
        errors.append(f"nem engedélyezett mező: {sorted(unknown)}")
    for key, value in event.items():
        if key in {"event", "event_ref", "model_version", "status"}:
            text = str(value)
            for pattern in FORBIDDEN_PATTERNS:
                if pattern.search(text):
                    errors.append(f"tiltott minta a mezőben: {key}")
    return errors
```

A `tests/test_log_redaction.py` a tiszta eseményt, az e-mailt, a tokent, a nyers `customer_id`-t, az ismeretlen mezőt és a HMAC-stabilitást teszteli; a retention-próba a lejárt fájl törlését és a törlés utáni hiányt ellenőrzi.

36.3.2.5 Retention

A megőrzési idő (retention) azt rögzíti, hogy az adott adatfajtát meddig szabad tárolni, és mi történik utána. A 32.3. alfejezetben az átmeneti modellartefaktumok rövid megőrzését láttuk; ugyanez a naplókra és a pszeudonim leképezésekre is igaz. A `forever` nem alapértelmezés. Például a 24.3. alfejezet emberi anomália-vizsgálati listája hét napig él, utána csak az összesített `P@k` marad, az egyedi azonosítók nélkül. A törlést is tesztelni kell: egy `retention_check.py` naponta törölje a határidőnél régebbi fájlsorokat, és a CI ellenőrizd, hogy a törlés után a lemezhasználat ténylegesen csökken.

36.3.2.6 Pseudonym

A pszeudonimizálás (pseudonymization) az azonosító lecserélése egy kódra. A `customer_id=CUST-0042` értéket leképezhetjük `USER-7F3A` alakra egy titkos, hosszú élettartamú kulccsal. A pszeudonim azonosító azonban nem anonim azonosító: ha a leképező kulcs vagy a maradék adatok rendelkezésre állnak, a kapcsolat visszaállítható. Egy ritka rendelési kombináció — például egy szokatlan `basket_minor` és egy ritka régió együttese — akár önmagában is egyetlen ügyfélre mutathat. A pszeudonim ID tehát továbbra is közvetett azonosításra alkalmas adatként kezelendő.

36.3.2.7 Provider boundary

A szolgáltatói határ (provider boundary) azt a vonalat jelöli, ahol az adat elhagyja a saját környezetünket. Minden kifelé irányuló kérésnél külön szerződés kell: mi az a minimális séma, amelyet a külső szolgáltató megkaphat. Ha a monitorozó szolgáltatás egy külső metrika-API-nak küld állapotot, a kimenő üzenet ne tartalmazza a nyers emailt, a kosár tartalmát vagy a pontos vásárlási időpontot; elég a modellverzió, a pszeudonim eseményazonosító, a predikció és a későbbi kimenet.

Példa kimenő szerződés:

```
{
  "model": "northstar-late-v3",
  "event_ref": "evt-7f3a",
  "prediction": 0.84,
  "outcome_label_days": 9,
  "provider_status": "ok"
}
```

A határ túloldalán lévő folyamat naplója is a mi adatvédelmi felelősségünk része, ezért a kimenő sémát ugyanúgy tesztelni kell, mint a belső konfigurációt.

36.3.3 Teljes minimális munkapélda

Induljunk ki a következő fájlokból. A munkakönyvtár `~/projects/northstar-monitor`, a példa Python 3.12-t és csak a standard libraryt használ.

`config.yaml`:

```
database:
  host: db.internal
  user: monitor_readonly
  password: ${DB_PASSWORD}
model_artifact: artifacts/northstar_late_v3.joblib
retention_days: 30
```

`.env.example`:

```
DB_PASSWORD=replace_me
MONITOR_API_KEY=replace_me
```

`logs/monitor_sample.log`:

```
2026-10-20T09:15:00Z INFO monitor start user=customer@example.com customer_id=CUST-0042
2026-10-20T09:15:01Z DEBUG db_connect host=db.internal user=monitor password=ChangeMe9!
2026-10-20T09:15:02Z INFO prediction model=northstar-late-v3 event_id=evt-7f3a score=0.84
2026-10-20T09:15:03Z ERROR api call failed api_key=sk-live-7f3a9c2b1e5f8d4a
```

A naplóminta szándékosan tartalmazza a két leggyakoribb hibát: nyers emailt és ügyfél-azonosítót, egy beégetett jelszót, valamint egy API-kulcsot. Az audit szkript a fájlsoronkénti besorolást állítja elő. Fontos tervezési döntés, hogy a találat kimenete nem írja ki újra a titok tényleges karaktereit, csak a pozíciót és a hosszt; így az auditjelentés sem válik másodlagos szivárgási forrássá.

`config_audit.py`:

```
from pathlib import Path
import re

PROJECT = Path("~/projects/northstar-monitor").expanduser()
AUDIT_FILES = ["config.yaml", ".env.example", "logs/monitor_sample.log"]

SECRET_RULES = {
    "inline-password": re.compile(
        r"password\s*[:=]\s*([^\s'\"${]+)", re.I
    ),
    "api-key-literal": re.compile(r"\bsk-[A-Za-z0-9_-]{8,}"),
}

PII_RULES = {
    "raw-email": re.compile(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
    ),
    "raw-customer-id": re.compile(r"\bCUST-\d{4}\b"),
}

PLACEHOLDER_OK = re.compile(r"replace[_ -]?me", re.I)

def audit_file(path: Path):
    findings = []
    for lineno, line in enumerate(
        path.read_text(encoding="utf-8").splitlines(), 1
    ):
        if PLACEHOLDER_OK.search(line):
            continue
        for rule_name, pattern in {**SECRET_RULES, **PII_RULES}.items():
            for m in pattern.finditer(line):
                findings.append({
                    "file": str(path.relative_to(PROJECT)),
                    "line": lineno,
                    "rule": rule_name,
                    "match": (
                        f"col {m.start() + 1}, "
                        f"length {len(m.group(0))}"
                    ),
                })
    return findings

for name in AUDIT_FILES:
    for finding in audit_file(PROJECT / name):
        print(finding)
```

Futtatás Linux/macOS terminálból:

```
cd ~/projects/northstar-monitor
python config_audit.py
```

Az alábbi kimenet szemléltető, a megadott bemenetből a szabályok alapján levezetett, nem helyi futásból mért:

```
{'file': 'logs/monitor_sample.log', 'line': 1, 'rule': 'raw-email', 'match': 'col 46, length
↪  21'}
{'file': 'logs/monitor_sample.log', 'line': 1, 'rule': 'raw-customer-id', 'match': 'col 80,
↪  length 9'}
{'file': 'logs/monitor_sample.log', 'line': 2, 'rule': 'inline-password', 'match': 'col 58,
↪  length 11'}
{'file': 'logs/monitor_sample.log', 'line': 4, 'rule': 'api-key-literal', 'match': 'col 47,
↪  length 24'}
```

36.3.4 Eredmény értelmezése

A kézi kontroll négy találatot vár. Az első naplósorban az email és a `CUST-0042` nyers azonosító külön-külön találat; a második sorban a `ChangeMe9!` jelszó; a negyedik sorban a `sk-live-...` kulcs. A harmadik sor tiszta, mert a predikció pontszáma és a modellverzió önmagában nem titok és nem közvetlen személyes azonosító.

A konfigurációs fájlban a `password: ${DB_PASSWORD}` sor nem hoz `inline-password` találatot, mert a szabály karakterosztálya kizárja a `$` jelet. Ez a helyes viselkedés: a `${DB_PASSWORD}` környezeti hivatkozás, nem beégetett titok. A `.env.example` sorai a `replace_me` helyőrző miatt kimaradnak; a fájl tehát publikálható a repositoryban. A naplóban szereplő titkok viszont már kompromittáltnak tekintendők: a jelszót és az API-kulcsot rotálni kell, a naplófájl korábbi másolatait pedig a retention szabály szerint törölni.

A javított napló lehet:

```
2026-10-20T09:15:00Z INFO monitor start user_ref=USER-7F3A occurred=1
2026-10-20T09:15:01Z DEBUG db_connect host=db.internal user=monitor auth=from-env
2026-10-20T09:15:02Z INFO prediction model=northstar-late-v3 event_id=evt-7f3a score=0.84
2026-10-20T09:15:03Z ERROR external call failed provider_ref=monitor-gw status=401
```

Ezen a javított naplón az audit már nulla találatot ad, mert a `USER-7F3A` nem illeszkedik a `CUST-\d{4}` mintára, és egyetlen sorban sem szerepel email vagy beégetett titok.

36.3.5 Feladatok

36.3.5.1 1. Értelmezés

A fejlesztő a `.env` fájlt felvette a `.gitignore`-ba, ezért úgy gondolja, a jelszó biztonságban van. Miért nem elegendő ez, ha a `ChangeMe9!` érték mégis megjelenik a `monitor_sample.log`-ban? Fogalmazd meg a titok tárolási és a naplóredakciós hiba különbségét.

36.3.5.2 2. Implementálás

Bővítsd a `config_audit.py` szkriptet egy `env-reference` szabállyal, amely a `${DB_PASSWORD}` alakú környezeti hivatkozásokat jelöli. A bővített kódban a konfigurációs fájl `password: ${DB_PASSWORD}` sora `env-reference` besorolást kapjon, de ne `inline-password` hibát. Add meg a szabályt reguláris kifejezésként, és indokold, miért nem zavarja össze a két szabályt a sorrend.

36.3.5.3 3. Hibadiagnózis

Adott a következő, állítólag már redakált naplósor:

```
2026-10-20T10:00:00Z INFO user=***@example.com customer_id=CUST-0042
```

Miért nem tekinthető ez megfelelő adatminimalizálásnak? Nevezd meg a két konkrét hibát, és írd le a javított sort a pszeudonim hivatkozásos mintával.

36.3.5.4 4. Megváltozott feltételű transzfer

A termékcsapat a monitorozó napló megőrzési idejét 30 napról 365 napra emelné, hogy hosszabb trendeket vizsgálhasson. Milyen kockázatot hordoz a nyers napló hosszú megőrzése, és milyen feltételekkel fogadnád el a változtatást? Adj konkrét retention-policy vázlatot legalább három adatfajtára, és írd le, milyen ellenőrzést futtatnál a törlési folyamatra.

36.3.6 Megoldások és indoklás

36.3.6.1 1. megoldás

A `.gitignore` csak azt akadályozza meg, hogy a `.env` a Git historyba kerüljön. Ha a futó folyamat a környezeti változóból kiolvasott jelszót a logger üzenetébe illeszti, a titok a naplófájlba, a naplóaggregátorba, a mentésekbe és minden olyan rendszerbe eljut, ahová a naplót továbbítják. A tárolási hiba tehát a `.env` commitolása lenne; ez most nem történt meg. A naplóredakciós hiba viszont független attól, hogy a titok honnan érkezett: a jelszó értékének egyáltalán nem szabad megjelennie a naplóban. A két hiba külön kezelendő, és a `.gitignore` megléte nem mentesíti a naplózó hívást.

36.3.6.2 2. megoldás

Az új szabály:

```
ENV_REF = re.compile(r"\$\{[A-Za-z_][A-Za-z0-9_]*\}")
```

Az `audit_file` ciklusában először a környezeti hivatkozást kell vizsgálni. Ha `password: ${DB_PASSWORD}` szerepel, a `${DB_PASSWORD}` illeszkedik az `ENV_REF` mintára, ezért `env-reference` találatot adunk. Az `inline-password` szabály karakterosztálya már kizárja a `$` jelet, így ugyanaz a sor nem lesz hibás jelszó-találat is. A sorrend azért nem zavaró, mert a két szabály különböző részt illeszt: az `env-reference` a `${...}` blokkra, az `inline-password` pedig csak akkor talál, ha a jelszó értéke `$` nélkül kezdődik. A bővített ciklus a találatot `{"rule": "env-reference", ...}` formában írja ki, a konfigurációs fájlban tehát nem hiba, hanem ellenőrzött hivatkozás lesz.

Ellenőrző invariant: a bővített kód a megadott `config.yaml`-ra pontosan egy `env-reference` találatot ad, a `logs/monitor_sample.log`-ra pedig továbbra is négy hibát; a két halmaz diszjunkt, mert egyetlen sor sem kerülhet egyszerre mindkét halmazba a `$` kizárása miatt.

36.3.6.3 3. megoldás

A `***@example.com` maszk két okból is rossz. Egyrészt a domain megmarad, így a `***` hosszával és a környező mezőkkel együtt az email gyakran visszafejthető; másrészt a `CUST-0042` nyers azonosító korlátlanul összekapcsolható a rendelési előzménnyel. A megfelelő sor nem tartalmaz emailt vagy nyers ügyfél-azonosítót:

```
2026-10-20T10:00:00Z INFO user_ref=USER-7F3A event_seen=1
```

A `USER-7F3A` pszeudonim hivatkozás leképezése külön, hozzáférés-korlátozott, rövid retentionű táblában él. A diagnózishoz csak az kell, hogy egy ismert felhasználói esemény megtörtént, nem pedig az, hogy ki volt az.

36.3.6.4 4. megoldás

A nyers napló 365 napos megőrzése növeli az expozíciós időt: minden későbbi adatszivárgás több naplósorra terjed ki, és a pszeudonim leképezés is tovább él. A változtatást csak akkor érdemes elfogadni, ha a hosszú megőrzés nem a nyers azonosítókra, hanem az aggregált statisztikára vonatkozik. Lehetséges policy:

Adatfajta	Retention
Nyers monitorozó napló	30 nap
Pszeudonim hivatkozásos napló	90 nap
Aggregált napi statisztika	365 nap
Modellartefaktum és manifest	365 nap

A modellartefaktum csak akkor maradhat 365 napig, ha a modell nem tárol közvetlen ügyfél-azonosítót, és a betöltési határt a 35.3. alfejezet modellkártyája rögzíti. A törlési folyamatra naponta futó `retention_check.py` ellenőrzi, hogy a határidőnél régebbi fájlok eltűntek, és a lemezhasználat a várt mértékben csökkent. A provider boundary felé küldött üzenetbe a hosszú trendhez is csak aggregált értékek kerülnek: modellverzió, nap, predikcióátlag, eseményszám.

36.3.7 Gyakori tévedés

A leggyakoribb tévedés azt hinni, hogy a pszeudonim azonosító automatikus anonimitást ad. A `CUST-0042` → `USER-7F3A` leképezés attól még visszafordítható, ha a leképező kulcs vagy a maradék adatok hozzáférhetők. A modell maga is hordozhat érzékeny információt: ha egy ritka `basket_minor` és `region` kombináció egyetlen ügyfélre mutat, akkor a pszeudonim azonosítóval ellátott predikciós sor közvetett azonosításra alkalmas marad. Ez visszavezet a 24.3. alfejezet anomáliakeresés tanulságához: az emberi vizsgálati lista is csak a feltétlenül szükséges mezőket tartalmazhatja, mert a pontszám-rangsor önmagában nem mentesít a minimalizálás alól. A 32.3. alfejezet rövid artifact retentionje ugyanezt az elvet erősíti: a törlési határidő a védelem része, nem utólagos kényelmi lépés.

Önellenőrzés: futtasd az auditot a saját repositorydra a `password`, `token`, `secret`, `api_key`, `AKIA` és email mintázatokra. Ha egy találat a naplóban vagy a konfigurációban van, ne a találati sort töröld először, hanem kérdezd meg, miért volt ott egyáltalán, és rotáld a titkot, ha az érték valaha elhagyta a gépet.

36.3.8 Rövid összefoglaló

A titkok, hozzáférések és személyes adatok határait már a kódolás előtt meg kell jelölni. A környezeti változó a titok hivatkozására való, a `.env.example` csak a neveket publikálja; a naplóba soha nem kerülhet jelszó, kulcs, email vagy nyers ügyfél-azonosító. A legkisebb jogosultság, a naplóredakció és a megőrzési határidők együtt csökkentik a szivárgási felületet. A pszeudonim azonosító hasznos, de nem anonim, és a modell is hordozhat érzékeny mintázatot. A szolgáltatói határon csak a minimális szerződéses séma mehet át. A következő lépésben a IX. ellenőrző pont ezekből a döntésekből állít össze működő rendszert és bizonyítékokat.

36.4 IX. ellenőrző pont: működő rendszer és bizonyítékai

Belépő · szint: haladó · idő: 90 perc · eszköz: Python, Docker

Előfeltétel: 36.3. alfejezet · Célkészség: IX. ellenőrző pont: működő rendszer és bizonyítékai · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Átvételi követelmény: a bemutatóban a hibás bemenetet, a hiányzó címkét és a jogosulatlan hozzáférési próbát is demonstrálni kell. A sikeres normál futás nem bizonyítja a helyreállíthatóságot és a biztonságot.

36.4.1 Cél és konkrét helyzet

A korábbi ellenőrző pontokon egy szivárgásmentes modellt építettünk (24.4. alfejezet), csomagként átadhatóvá tettük (32.4. alfejezet), és release-feltételekkel kiadtuk (35.4. alfejezet). Most a kérdés az, hogy a futó rendszerről milyen bizonyítékokat gyűjtünk, és mit teszünk, ha a bizonyíték hibát mutat. A IX. ellenőrző pont ezért nem egy szűk programozási feladat, hanem egy mérési és hibakezelési terv, amelyet a Northstar késői kézbesítés modelljén futtatunk végig.

A konkrét helyzet: a `northstar-late` modell minden rendeléskor becsüli, hogy a kézbesítés 7 napnál később lesz-e. Reggelenként batch módban 5000 rendelési rekordot pontozunk, napközben pedig egy kis FastAPI-végpont válaszol egyedi kérésekre. A bemenet rendelésenként az 1.2. alfejezetben rögzített, a rendelés pillanatában ismert feature-vektor; a megfigyelési egység egy `order_id`/`request_id`; a kimenet `pred_prob`, `pred_label`, `model_version` és `request_id`. Az igazi `target_late` csak a kézbesítés után, tehát késleltetve érkezik meg. A rendszer LOCalhoston, CPU-n fut, nincs felhő, fizetős API vagy titkos ügyféladat.

Az elvárt ellenőrzés: a p95 válaszidő legfeljebb 250 ms; a batch futás 5000 soron legfeljebb 60 másodperc; 30 napos referenciaablakhoz képest a folytonos feature-ök PSI driftje 0,1 alatt marad; a 7 napnál régebbi kérések legalább 95%-ában rendelkezésre áll a címke; a rollback 5 percen belül visszaáll egy ismert verzióra; a szolgáltatás csak a szükséges jogosultságokkal fér a titkokhoz és a modellhez.

Induló becslés a megoldás előtt: egy naplóban 20 kérés késleltetése érkezett, rendezve: 8, 9, 10, 11, 12, 12, 13, 14, 15, 15, 16, 17, 18, 20, 21, 22, 25, 31, 48, 510 ms. Körülbelül mekkora a p95? Mielőtt tovább olvasol, válaszolj egy számmal, és indokold, miért veszélyes ebből a 20 mintából a valódi farokról dönteni.

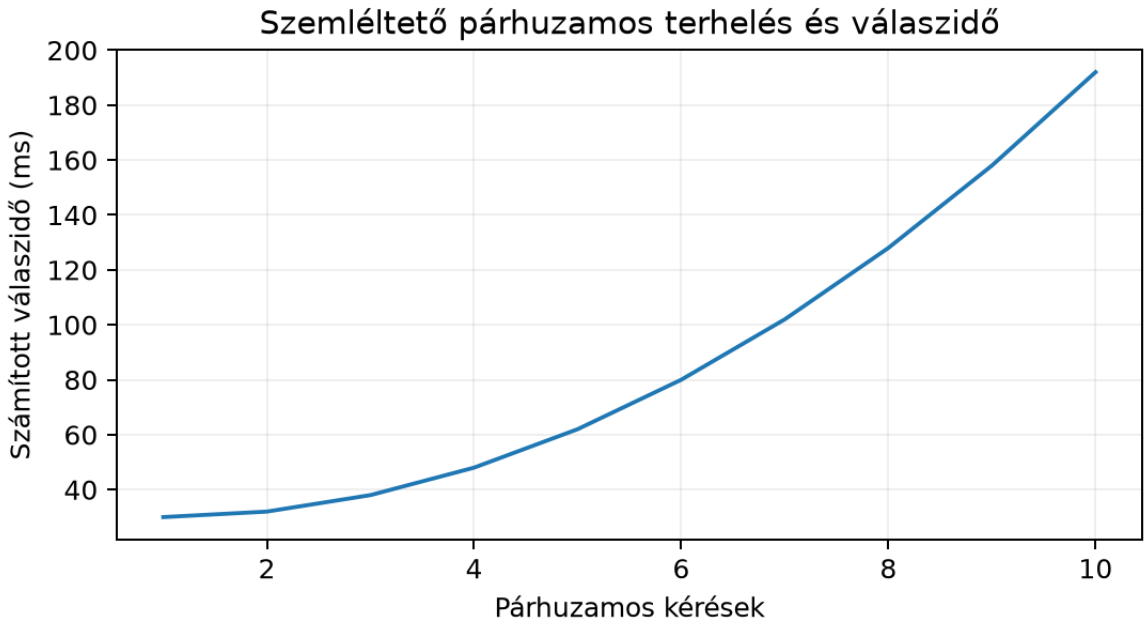
36.4.2 Fogalmi magyarázat: a hat működési blokk

36.4.2.1 Compute budget: erőforrás-keret

A compute budget (számítási keret) egy megállapodás arról, hogy egy futtatási egység mennyi CPU-időt, memóriát, késleltetést és tárhelyet használhat. A cél nem az, hogy mindig alacsony legyen a terhelés, hanem hogy előre ismert korlátunk és mérésünk legyen. Az API esetén a kulcsmutató a válaszidő-eloszlás p95 kvantilisa. Ha n mért késleltetés van, rendezzük őket növekvő sorrendbe: $L_{(1)} \le L_{(2)} \le \ldots \le L_{(n)}$. Az alsó 95%-os tapasztalati kvantilis az a $L_{(k)}$, ahol k a legkisebb egész, amelyre $k/n \ge 0.95$. A mértékegység ms, a shape egy n hosszú vektor.

A 20 elemű példában $n = 20$, tehát $k = 19$, és a p95 a 19. rendezett érték, azaz 48 ms. Ez megfelel annak, hogy a mérések 95%-a legfeljebb 48 ms, és 5%-uk, azaz egyetlen kérés, lassú. A legnagyobb érték, 510 ms, a farokban van, ezért a p95 robusztusabb, mint a maximum, de még így is erősen függ a minta elemszámától.

A batch munkánál a keret a teljes futási időre és memóriára vonatkozik. A p95-öt becsülhetjük a fejlesztőgépen ismételt méréssel, azonban teljesítményállításhoz hardver, inputméret, verzió és ismétlésszám is kell. Az alábbi chart egy számított függvénypélda arra, hogyan nőhet a válaszidő a párhuzamos terheléssel:



36.1. ábra: Szemléltető párhuzamos terhelés és válaszidő - A latency=30+0.4*concurrency^2 képletből számított példa, nem mérés.

A görbe csak a tervezési logikát mutatja: amíg a párhuzamos terhelés a CPU korlát alatt van, a válaszidő közel állandó; utána a sorban állás miatt meredeken nő. A tényleges méréshez a 35.4. alfejezetben tanult load budget tesztet kell futtatni.

36.4.2.2 Batch és API: két kiszolgálási szerződés

A batch és API nem ugyanaz a modell kétféle meghívása, hanem két külön szerződés. A batch inputja egy előre összegyűjtött fájl vagy tábla; a futás azonosítóval rendelkezik, az output idempotens, tehát ugyanaz a futás ugyanazt az eredményfájlt állítja elő. Az API inputja egyetlen kérés; a válasznak gyorsnak kell lennie, és a szerződés a `/health`, `/ready`, séma és verzió mezőket is tartalmazza.

A döntési lap három kérdésből áll: milyen késleltetést bír el a folyamat, mekkora a tranzakciós egység, és mi történjen, ha a futás félbeszakad? A rendeléskor hozott döntéshez API kell, mert a választ azonnal felhasználjuk. A napi riport vagy modellminőségi utóellenőrzés viszont batch, mert a teljes forgalmat egyben, lassabban is feldolgozhatjuk, és a függőséget jobban rögzítjük.

36.4.2.3 Drift: az input eloszlásának eltérése

A drift (adateltolódás, data drift) azt jelenti, hogy a szolgáltatás inputjának eloszlása eltér a modellépítéskori vagy a referenciaablak szerinti eloszlástól. A Population Stability Index (PSI) a folytonos feature-öt diszkrét bucketekre osztja, és a referencia- meg az aktuális gyakoriságok eltérését méri:

$$\mathrm{PSI} = \sum_{i=1}^{B} (a_i - e_i) \ln\left(\frac{a_i}{e_i}\right),$$

ahol B a bucketek száma, e_i a referenciaablakban az i. bucketbe eső rendelések aránya, a_i pedig a friss ablakban mért arány. Az e_i és a_i egyaránt B hosszú vektor, minden elemük 0 és 1 közötti arány, mértékegység nélkül, és $\sum_i e_i = \sum_i a_i = 1$. A PSI értéke 0-tól felfelé terjed; 0 akkor, ha a két eloszlás azonos, nagyobb érték erősebb eltérést mutat. Ha valamelyik bucket üres, a tagot kihagyjuk vagy kis epszilont használunk.

Kézi kontroll két bucketre: legyen $e = [0.5, 0.5]$, $a = [0.6, 0.4]$. Ekkor

$$\mathrm{PSI} = (0.6 - 0.5)\ln\left(\frac{0.6}{0.5}\right) + (0.4 - 0.5)\ln\left(\frac{0.4}{0.5}\right) = 0.1\ln(1.2) - 0.1\ln(0.8) \approx 0.0405.$$

A 0,1 alatti PSI-t gyakran stabilnak, a 0,1 és 0,25 közöttit figyelmeztetésnek, a 0,25 felettit erős eltérésnek tekintik, de ez heurisztika, nem statisztikai garancia. A PSI nem mondja meg, hogy a drift rontja-e a modellt; először a predikció és a későbbi címke közötti teljesítményt kell nézni. Folytonos feature-ökre a `scipy.stats.ks_2samp` is adhat p-értéket, de a p-érték itt is a nullmodell szerinti szélsőséget méri, nem a drift valószínűségét.

36.4.2.4 Delayed labels: a késleltetett címkék cenzúrája

A delayed label (késleltetett címke) azt jelenti, hogy a predikció pillanatában még nem ismerjük a valódi eredményt. A Northstar adatban a `label_available_at` az `actual_delivery_at` értékével egyezik, tehát a címke csak a teljes kézbesítés után áll rendelkezésre. Ha egy rendelés O időben érkezik, és a kézbesítésig eltelt idő D, akkor a T időpontban a címke pontosan akkor ismert, ha $O + D \le T$. A D eloszlásfüggvénye $F(d) = P(D \le d)$; ezzel a T-ben megfigyelhető, legalább w napos rendelések arányának várható értéke $F(w)$. Az $F(d)$ értéke 0 és 1 közötti valószínűség, d mértékegysége nap, az input shape pedig a rendelésekhez tartozó $\{0, 1\}$ label-availability vektor.

A legfontosabb statisztikai veszély a korai mérés szelektív torzítása. A 7 napos cél esetén a pozitív esetek egy része eleve csak 8. vagy későbbi napon derül ki. Ha a 2. napon számolunk modellminőséget, akkor a gyorsan kézbesített, túlnyomórészt negatív esetekre látunk címkét, a lassú pozitív esetekre nem. Az így kapott AUC vagy accuracy optimista. Ez ugyanaz a leakage-logika, mint a 24.4. alfejezetben: a mérés pillanatában még nem megérkezett címkét nem vonhatjuk be az értékelésbe, és a hiányzó címkét nem tölthetjük fel hallgatólagosan nullával.

36.4.2.5 Rollback: előre rögzített visszaállítás

A rollback (visszaállítás) nem a hiba bekövetkezésekor kitalált improvizáció, hanem release-kori döntési szabály. Rögzítjük, hogy mely feltételek váltják ki, melyik verzióra állunk vissza, és hogyan ellenőrizzük a visszaállítás eredményét. A feltételek lehetnek: p95 késleltetés tartósan a budget fölött, a batch output sémája nem egyezik a manifesttel, a PSI-riasztást nem tudjuk üzleti eseménnyel magyarázni, vagy a címkék késése miatt a modellminőségi ablakban a hiba elszabadul. Maga a visszaállítás pointer-váltás: a `live` pointer egy korábbi, hash-elt modellverzióra mutat.

36.4.2.6 Incidens-runbook: detektálástól a tanulságig

Az átvétel akkor teljes, ha a hibakezelés nem a hibától függ, hanem előre írt runbook szerint fut. A kötelező lépések:

Lépés	Kiváltó jel	Teendő	Felelős / célidő
Detektálás	riasztás (PSI, hibaarány, p95, séma, coverage)	riasztás azonosítása, duplikátum kiszűrése	ügyeletes, 5 perc
Triage	bármely riasztás	hatókör, utolsó deploy/config/adatváltozás ellenőrzése	ML-üzemeltetés, 15 perc
No-score mód	séma-, modell- vagy adatbiztonsági hiba	a szolgáltatás nem ad valószínűséget; `503 model_unavailable`, a batch nem ír outputot	ügyeletes, azonnal
Rollback	no-score vagy súlyos minőségi/szolgáltatási hiba	pointer visszaállítása, hash/séma/kontroll-predikció ellenőrzése	release owner, 5 perc
Értesítés	incidens nyitva	érintettek, státusz, ETA; személyes adat nélkül	incidensmenedzser, 15 perc
Utóvizsgálat	incidens lezárva	idővonal, gyökér ok, hatás, detektálási rés	csapat, 5 munkanap
Tanulság	utóvizsgálat kész	teszt, riasztás, runbook vagy kód módosítása, felelős és határidő	tech lead

A **no-score mód** a legfontosabb biztonsági kapu: ha a modell vagy a séma nem ellenőrizhető, a szolgáltatás ne adjon „hihető” számot. Az API ilyenkor `503`-at ad `model_unavailable` részlettel, a batch pedig nem ír predikciós fájlt; a downstream folyamat a dokumentált fallbackre vált, nem a régi score-t használja csendben.

```
RUNBOOK = ["detect", "triage", "no_score", "rollback",
           "notify", "postmortem", "lessons"]

def enter_no_score(service):
    service = dict(service)
    service["serving"] = False
    service["response"] = "model_unavailable"
    service["prediction"] = None
    return service

def rollback(service, previous_hash, active_hash, control_ok):
    if previous_hash is None:
        raise ValueError("nincs visszaállítható verzió")
    if not control_ok:
        raise RuntimeError("a régi modell kontrollpredikciója nem egyezik")
    service = dict(service)
    service["active_hash"] = previous_hash
    service["serving"] = True
    return service
```

Az invariáns: szándékosan elrontott séma vagy hiányzó modell esetén a szolgáltatás **nem** ad félrevezető predikciót; a `no_score` állapotban a válasz `503`, a predikció `None`. A rollback csak akkor engedélyezett, ha a korábbi verzió hash-e és kontrollpredikciója ellenőrzött. A `tests/test_incident_runbook.py` a lépéssorrendet, a no-score viselkedést, a sikeres és sikertelen rollbacket, valamint az értesítés PII-mentességét teszteli.

36.4.2.7 Access: hozzáférési határ

A 36.3. alfejezetben tanított elvek itt futási feltétellé válnak. A szolgáltatásnak csak a modell olvasására, a metrikák írására és a saját naplójára van joga; nyers rendelési adathoz vagy a tanító adathoz nem fér. A titkok környezeti változóban élnek, nem a kódban; a napló redaction-szabálya kitakarja a tokeneket, kulcsokat és személyes azonosítókat; a retention pedig meghatározza, hogy a nyers kérések meddig őrizhetők. A provider boundary azt is rögzíti, hogy ha egy külső szolgáltatás kap adatot, az átadott mezők listája és a törlési folyamat előre dokumentált.

36.4.3 Teljes minimális munkapélda: incidens-szimuláció

A példa nem valódi futási jegyzőkönyv, hanem szemléltető, helyi CPU-n futtatható terv. A munkakönyvtár legyen `northstar-monitor/`. Először szimulált szolgáltatásnaplót készítünk, hogy a későbbi SQL- és Python-mutatók determinisztikusak legyenek:

```
# generate_service_log.py
from pathlib import Path
import csv
import random
from datetime import datetime, timedelta, timezone

random.seed(20261020)
out = Path("data/service_log.csv")
out.parent.mkdir(exist_ok=True)

start = datetime(2025, 6, 1, tzinfo=timezone.utc)
rows = []
for i in range(5000):
    order_at = start + timedelta(minutes=30 * i)
    distance_km = random.gamma(2.0, 40.0)
    priority_flag = 1 if random.random() < 0.2 else 0
    basket_minor = random.randint(500, 25000)
    latency_ms = int(random.lognormvariate(3.2, 0.6))
    model_version = "northstar-late-0.3.0"
    pred_prob = round(min(0.9, max(0.02, 0.05 + 0.003 * distance_km + 0.15 * priority_flag)), 4)
    request_id = f"req-{i:05d}"
    rows.append([request_id, order_at.isoformat(), distance_km, priority_flag,
                 basket_minor, latency_ms, model_version, pred_prob])

with out.open("w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["request_id", "order_at", "distance_km", "priority_flag",
                     "basket_minor", "latency_ms", "model_version", "pred_prob"])
    writer.writerows(rows)

print("wrote", len(rows), "rows")
```

A kód egyetlen seedből 5000 kérést készít. A predikciós valószínűség determinisztikus függvénye a távolság és prioritás jellemzőknek, tehát itt nincs valódi modellbetöltés, de a séma ugyanaz, mint a 32.4. alfejezetben átadott predikciós szerződés. A `pred_prob` a 0,02–0,90 intervallumba van vágva; a valódi modell kalibrált valószínűsége is 0 és 1 közé esik.

A batch-kontroll és a p95 lekérdezése DuckDB SQL-ben:

```
-- batch latency control, DuckDB
SELECT
  date_trunc('day', order_at::TIMESTAMP) AS day,
  count(*) AS requests,
  quantile_cont(latency_ms, 0.95) AS p95_latency_ms,
  count(*) FILTER (WHERE latency_ms > 250) AS over_budget_requests
FROM 'data/service_log.csv'
GROUP BY 1
ORDER BY 1;
```

Ez a lekérdezés napi bontásban adja a kérésszámot, a p95 késleltetést és a költségvetés feletti kérések számát. A `quantile_cont` DuckDB-függvény; SQLite-ban hasonló célra saját ablakos rendezést kellene írni, ez egy dialektuskülönbség, amit a 8.2. alfejezetben már láttunk. A p95-öt naponta számoljuk, mert a terhelés napszakonként eltér.

A drift-ellenőrzés Pythonban a PSI-t számítja ki a `distance_km` feature-re:

```
# monitor_drift.py
import csv
import math
from pathlib import Path

def psi(reference, current, bins=10):
    lo = min(reference + current)
    hi = max(reference + current)
    width = (hi - lo) / bins
    e = [0] * bins
    a = [0] * bins
    for v in reference:
        i = min(bins - 1, int((v - lo) / width))
        e[i] += 1
    for v in current:
        i = min(bins - 1, int((v - lo) / width))
        a[i] += 1
    e = [x / len(reference) for x in e]
    a = [x / len(current) for x in a]
    return sum((ai - ei) * math.log(ai / ei)
               for ei, ai in zip(e, a) if ei > 0 and ai > 0)

with Path("data/service_log.csv").open() as f:
    rows = list(csv.DictReader(f))
dist_all = [float(r["distance_km"]) for r in rows]
ref = [float(r["distance_km"]) for r in rows[:3000]]
cur = [float(r["distance_km"]) for r in rows[3000:]]
print(round(psi(ref, cur), 4))
```

A függvény 10 egyenlő szélességű bucketre osztja a referencia- és az aktuális ablakot. Ha az utóbbi 2000 kérés távolsága eltolódik, a PSI nő; ha azonos eloszlású marad, a PSI nullához közeli. A kimenet itt egyetlen szám, ezért ki kell egészíteni a döntéssel: 0,1 alatt nincs riasztás, 0,1 fölött a séma- és üzleti naplót is meg kell nézni.

A rollback munkamenet akkor indul, ha a PSI-riasztás nem magyarázható. A `live` pointer állomány:

```
# Windows PowerShell; munkakönyvtár: northstar-monitor
Get-FileHash models/northstar-late-0.3.0.bin -Algorithm SHA256
Get-Content models/pointer.json
# szemléltető pointer:
# {"live": "northstar-late-0.3.0", "sha256": "abc123...", "activated_at": "..."}
python -m deployment.rollback --from northstar-late-0.3.0 --to northstar-late-0.2.1 --reason
 ↪  "PSI alert"
```

A `rollback` modul a pointert `northstar-late-0.2.1`-re állítja, és a `/ready` végpont addig sikertelen, amíg az új pointerrel nem fut le a séma- és hash-ellenőrzés. Ez a 35.4. alfejezetben bevezetett release-feltételek futtatása.

Végül a hozzáférési ellenőrzés és naplóredaction:

```
# access_check.py
import logging
import os

class RedactingFilter(logging.Filter):
    def filter(self, record):
        msg = record.getMessage()
        for name in ["API_KEY", "DB_PASSWORD", "SECRET"]:
            if os.environ.get(name) and os.environ[name] in msg:
                record.msg = msg.replace(os.environ[name], "[REDACTED]")
        return True

logging.basicConfig(level=logging.INFO)
logging.getLogger().addFilter(RedactingFilter())
logging.info("request finished with token=%s", os.environ.get("API_KEY", "missing"))
```

A szűrő azt ellenőrzi, hogy a naplóüzenetbe került-e egy környezeti titok. Ez nem védi meg a fejlesztőt attól, aki szándékosan logolja a titkot, de megfogja a formátumsztringből beszivárgó gyakori esetet. A titkokat soha nem írjuk notebook-outputba vagy futási naplóba; ahol külső szolgáltatásnak kell átadni, ott a mezők listája a provider boundary dokumentumban él.

36.4.4 Eredmény értelmezése

A p95-ös batch-lekérdezés várhatóan olyan napi értékeket ad, amelyekben a kérésszám 300–500 között mozog, a p95 pedig 200–350 ms körül ingadozik a szimulált lognormális eloszlás miatt. A `over_budget_requests` oszlop a napi kérések 5%-ánál nem lehet több akkor, ha a p95 éppen a 250 ms-os határon van. A PSI kézi kontrollja visszaadja a korábbi 0,0405-ös értéket, ha a referencia- és aktuális gyakoriságokat explicit bucketekben adjuk meg. A rollback invariánsa: a `pointer.json`-ban szereplő hash egyezik a `northstar-late-0.2.1.bin` fájl `sha256` értékével, és a `/ready` csak a sémaellenőrzés után tér vissza 200-as állapottal. Az access-szűrő invariánsa: a log kimenetben a titok értéke nem szerepel, csak a `[REDACTED]` szöveg. A kódblokkok után tehát nem elég annyit mondani, hogy „rendben”; meg kell nevezni, melyik mérték melyik feltételt igazolja, és mekkora tolerancia engedhető meg.

A fenti értékek szemléltetőek, mert a kapcsolódó példaprojektben még nem ellenőrzött. Ha helyben lefuttatod, a pontos számokat a saját futási jegyzőkönyv rögzíti; a seed 20261020 garantálja a szimulált input azonosságát azonos Python-verzión és azonos könyvtárverzión.

36.4.5 Feladatok

1. **Értelmezés.** A drift-jelentés szerint a `distance_km` PSI-értéke 0,18 a referencia- és a friss ablak között. A referenciaablak 30 nap, 1200 kéréssel; a friss ablak 3 nap, 90 kéréssel. Fogalmazd meg, miért nem állíthatjuk automatikusan, hogy a modell bemenete elromlott, és milyen további két ellenőrzést végeznél el a riasztás lezárása előtt.

2. **Implementálás.** Írj SQL lekérdezést DuckDB-re, amely a `service_log` és egy külön `label_events` tábla alapján napi bontásban megadja: az adott napon érkezett kérések számát, és közülük azok arányát, amelyeknek 14 napon belül megérkezett a címkéje. A `label_events` sémája: `request_id`, `label_available_at`, `target_late`. A címke csak

akkor számít megérkezettnek, ha `label_available_at` nem üres és legfeljebb 14 nappal későbbi, mint a kérés ideje.

3. **Hibakeresés.** A batch futás 503-at ad, a `/health` 200-at. A napló utolsó sora: `downloaded model with api_key=sk-1234abcd`. Nevezd meg a két hibát ebben a helyzetben, és add meg a javító lépéseket a hozzáférési és naplózási szerződés szerint.

4. **Megváltozott feltétel.** A termékcsapat a batch pontozásról áttér valósidejű API-pontozásra. A becsült forgalom 500 kérés/óra, a rendelkezésre álló CPU 2 mag. Adj mérési tervet a compute budgethez: milyen inputméret, hány ismétlés, milyen metrikák és milyen rollback-feltétel kell ahhoz, hogy a 250 ms-os p95-célt ellenőrizni tudd?

36.4.6 Megoldások és indoklás

1. A PSI 0,18 figyelmeztetés, de három nap alatt 90 kérés kis minta; a becsült PSI szórása nagy, és a bucketek véletlenszerűen is elmozdulhatnak. Először a kérések időbeli és szegmensbeli bontását nézném: ha egyetlen nagy ügyfél vagy kampány okozza, a drift magyarázható. Másodszor a modell teljesítményét ellenőrizném a már megérkezett címkéken: ha a PSI ellenére az AUC a toleranciasávon belül marad, akkor az inputeltolódás nem feltétlenül jelent minőségromlást. A riasztást le lehet zárni azzal, hogy a PSI-ra statisztikai bizonytalanságot is számolunk, például bootstrap intervallumot a referenciaablakból, és csak akkor emelünk incidenst, ha a friss PSI a bootstrap 95%-os sávja fölé esik.

2. Egy lehetséges SQL:

```
SELECT
  date_trunc('day', s.order_at::TIMESTAMP) AS request_day,
  count(*) AS requests,
  avg(CASE
        WHEN l.label_available_at IS NOT NULL
         AND l.label_available_at::TIMESTAMP <= s.order_at::TIMESTAMP + INTERVAL '14 day'
        THEN 1 ELSE 0 END) AS label_coverage_14d
FROM service_log s
LEFT JOIN label_events l USING (request_id)
GROUP BY 1
ORDER BY 1;
```

A `LEFT JOIN` azért szükséges, hogy azok a kérések is szerepeljenek a nevezőben, amelyekhez még nincs címke. A 14 napos feltétel explicit cenzúra: a 15. napon érkező címke már nem javítja a 14 napos coverage-et. A napi bontás a megfigyelési egységet, a kérést tartja meg; az `AVG` a bináris indikátor átlaga.

3. A 503 azt jelzi, hogy a batch futás nem tudta betölteni a modellt vagy az inputot, miközben a `/health` csak a folyamat él, tehát a service maga fut. A második hiba a naplóban lévő `sk-1234abcd` API-kulcs: a titok értékét naplózzuk. Javítás: a modell letöltése ne igényeljen API-kulcsot, hanem a lokalizált artifact hash-ét ellenőrizze; ha mégis van külső hívás, a kulcs a titokkezelőből jöjjön anélkül, hogy a log formátumsztringbe kerül, és a `RedactingFilter` aktív legyen a kibocsátó loggeren. A 503 diagnózisához a batch futás stderrjét kell nézni, nem a health-et.

4. A mérési terv: 500 kérés/óra, 10 perces forgalmi csúcsokat szimulálunk 2 CPU-magos gépen; az input 5000 szintetikus rendelési rekord, mindegyik a 39.2. alfejezetben használt feature-sémával; 10 ismétlés, mindegyikben más `request_id`, de azonos eloszlás. A mért metrika: p50, p95, p99 válaszidő ms-ban, sikeres válaszok aránya, processz memória maximuma. A /ready ellenőrzése ismétlésenként. Rollback-feltétel: ha a p95 10 ismétlésből legalább 8 alkalommal 250 ms felett van, vagy a sikertelen kérések aránya meghaladja az

1%-ot, automatikusan a korábbi pointert kell aktíválni, és az eredményt a futáskövetőben rögzíteni. Az értékekhez ugyanaz a hardver és verzió kell, mint a termelésben; ha ez nem áll rendelkezésre, a mérés csak előzetes, nem elfogadási bizonyíték.

36.4.7 Gyakori tévedés és önellenőrzés

A leggyakoribb tévedés, hogy egyetlen sikeres demóból tartós megbízhatóságra következtetünk. Ellenpélda: az API egy 1000 kéréses reggeli tesztben minden kérésre 200-as választ adott, a p95 190 ms volt; a csapat ezért engedélyezte a release-t. Másnap a batch folyamat a napi 5000 soros inputon memória-túllépést kapott, mert a teszt csak 1000 sort pontozott, és a teljes nap kibontott DataFrame-jét a folyamat egyszerre tartotta memóriában. A 200-as válaszok és az alacsony p95 igaz mérés volt, de nem bizonyította a batch memória-budgetjét. A rendszer bizonyítéka ezért több, független feltétel együttes teljesülése: p95 a terhelés mellett, batch memória és idő, séma-hash, a késleltetett címkék aránya, és a rollback lefutása. Ez nem különbözik a 32.4. alfejezetben használt kontrollfelfogástól: egyetlen tesztfájl sem igazolja a teljes csomagot, és a 24.4. alfejezetben tanult leakage-védelem is akkor él, ha a mérési ablakot explicitté tesszük.

Önellenőrzés a rendszergazdai átadás előtt: van-e a release-hez PSI-referenciaablak és regisztrált küszöb? A delayed label mérés kizárja-e a még meg nem érkezett címkéket? A rollback-parancs hash-t és naplóbejegyzést is készít? A service account ténylegesen nem fér a nyers rendelési adatokhoz? Ha bármelyik kérdésre nemleges a válasz, a rendszer még nem hordozza azokat a bizonyítékokat, amelyeket a IX. ellenőrző pont kér.

36.4.8 Rövid összefoglaló

A működő rendszer bizonyítéka nem a sikeres demó, hanem a mérési és hibakezelési terv: a compute budget p95-öt és memóriakorlátot ad; a batch és API szerződése különböző késleltetési és idempotencia-elvárásokat rögzít; a drift PSI-vel és statisztikai bizonytalansággal figyelhető; a delayed label mérés explicitté teszi a cenzúrát, hogy a kései pozitív esetek ne torzítsák a modellminőséget; a rollback pointer-váltással és előre írt feltételekkel fut; az access a legkisebb jogosultságot, környezeti titkokat és naplóredakciót tartatja be. A következő alkalmazásban a többnyelvű data science környezetbe lépünk, ahol ugyanezt a statisztikai fegyelmet R-eszközökkel is ismételhetően kell megvalósítani.

37 R és a többnyelvű data science ökoszisztéma

37.1 R-környezet és statisztikai munkafolyamat

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: R, renv

Előfeltétel: 36.4. alfejezet · Célkészség: R-környezet és statisztikai munkafolyamat · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

37.1.1 Cél és konkrét helyzet

Egy kolléga átadott egy statisztikai elemzést R-ben: logisztikus regressziót futtatott a Northstar kezdőlabor `orders.csv` adatán, és azt állítja, hogy az eredmény bárhol reprodukálható. A te feladatod ellenőrizni, hogy a környezetleírás valóban elég-e egy másik gépen történő újrafuttatáshoz.

- **Bemenet**: az `<repo>/lab/data/orders.csv` tiszta, 2400 rendelést tartalmazó fájl; egy R projektmappa, amelyben `northstar-r.Rproj`, néhány R script és egy `renv.lock` található.
- **Megfigyelési egység**: egy projekt-snapshot, azaz a projekt gyökerében lévő fájlok és a hozzájuk tartozó verziómetadatok együttese.
- **Cél**: készíts környezettérképet, amely megkülönbözteti az R nyelvet, a Posit IDE-t, a csomagkönyvtárat, a projektlockot és a `sessionInfo()` kimenetet; ellenőrizd, hogy a Python-megfelelőkhöz hasonlóan melyik réteg mit rögzít.
- **Előzetes eredménybecslés**: a `renv.lock` legalább az R-verziót és a `readr` csomagot a függőségeivel együtt tartalmazza; a `sessionInfo()` a `readr` verzióját és az R-verziószámot írja ki. A kontrollfuttatás során a CSV-ből 2400 sort és 15 oszlopot várok.

37.1.2 Nyelv, IDE, csomagkönyvtár és lock: négy különböző réteg

Az R-környezeteknél gyakori félreértés, hogy az emberek egyetlen eszközként beszélnek „az R-ről”. Valójában legalább négy réteget érdemes különválasztani; ez a gondolatmenet pontosan megfelel a Pythonnál már megismert felosztásnak (3.4. alfejezet).

37.1.2.1 Az R mint nyelvi futtatókörnyezet

Az **R** (szűk értelemben *R interpreter*, magyarul R-értelmező) az a program, amely a `.R` kiterjesztésű kódot végrehajtja. Ugyanolyan szerepe van, mint a Python interpreternek. A verzióját az

```
R.version.string
```

kifejezés add meg, például `"R version 4.4.1 (2024-06-14)"`. Az interpreter saját telepítési gyökérrel rendelkezik, amelyet az `R.home()` hívás mutat meg. Ez a könyvtár tartalmazza a base csomagokat és a futtató binárist. A verziószám első két tagja, például `4.4`, gyakorlati szempontból fontos: az ilyen szintű eltérés esetén egyes bináris csomagok nem hordozhatók át automatikusan.

A Python-megfelelő: `sys.version` és `sys.executable`. A 3.4. alfejezetben tanult kernelspec éppen azt a hibát előzi meg, amikor egy notebook más Python-interpretert használ, mint a CLI. R-ben ugyanez a probléma a Posit IDE és a terminál között léphet fel.

37.1.2.2 Posit IDE: szerkesztő, nem interpreter

A **Posit IDE** (korábbi nevén RStudio, angolul *integrated development environment*) szerkesztőt, projektnézetet, konzolt és debuggert kínál. Fontos: a Posit IDE nem maga az R, hanem egy felület, amely egy kiválasztott R-interpreterhez csatlakozik. Ezért fordulhat elő, hogy az IDE-ben `R.home()` más értéket ad, mint a terminálban kiadott `R` parancs, ha a beállításokban eltérő verziót választottunk.

A Python világában a VS Code vagy a JupyterLab is hasonló szerepet tölt be: a megnyitott projekt mappája és a kiválasztott kernel/interpreter két külön dolog. A Jupyter kernelspec regisztrálásakor (3.4. alfejezet) is láthattuk, hogy a felületi név nem azonos a tényleges interpreter-útvonallal.

37.1.2.3 Library path: hol élnek a csomagok

Az R csomagok a **library path** (*könyvtárútvonal*) nevű mappaegyüttesből töltődnek be. Ezt az R-munkamenet az

```
.libPaths()
```

vektorral listázza. A vektor elemei teljes elérési utak; az első elem a legmagasabb prioritású könyvtár. Egy projektbe zárt rendszerben az első elem például:

```
# [1] "/home/user/northstar-r/renv/library/R-4.4/x86_64-pc-linux-gnu"
# [2] "/usr/local/lib/R/site-library"
# [3] "/usr/lib/R/library"
```

Ez a példa háromelemű vektor, szemléltető értékű; az első elem a projekt könyvtára, a második a gépszintű telepítés, a harmadik pedig a base csomagokat tartalmazó rendszerkönyvtár. A fontos ellenőrzési pont, hogy egy betöltött csomag valójában honnan érkezik: `find.package("readr")` megadja a points elérési utat. Ha az út nem a projekt `renv/library/...` elejére mutat, akkor a projekt nem izolált.

37.1.2.4 renv: projektlock, nem interpreter-másolat

Az **renv** (R environment) a csomagok verzióit és a függőségi gráfot rögzíti, nem pedig egy hordozható interpreter-környezetet készít. A `renv.lock` egy JSON-fájl, amelyben például a következő blokk látható:

```
{
  "R": {
    "Version": "4.4.1",
    "Repositories": [
      {
        "Name": "CRAN",
        "URL": "https://cloud.r-project.org"
      }
    ]
  },
  "Packages": {
    "readr": {
      "Package": "readr",
      "Version": "2.1.5",
      "Source": "Repository",
      "Repository": "CRAN",
      "Hash": "9c7f6c7e8b2c3a0e4f5d6e7a8b9c0d1e"
    }
  }
}
```

A fenti `Hash` érték illusztráció; valós lockfájlban a renv saját ellenőrző lenyomata szerepel. A lockfile tehát azt mondja meg, hogy egy másik gépen *melyik verziókat* kell visszaállítani; magát az R interpreters telepítőt nem tartalmazza.

A Python-megfelelők kétfelé válnak. A `venv` inkább a `renv/library` mappa analógiája: izolált csomagtér egy adott gépen, amelyet nem hordozunk gépek között. Az `uv.lock` a `renv.lock` analógiája: verziókat rögzít, de nem tartalmazza sem a Python-buildet, sem a forrásfájlokat.

37.1.2.5 project és sessionInfo

Az R **project** gyakorlatilag egy mappa, amelyet az `.Rproj` kiterjesztésű fájl fémjelez. A projektmunkafolyamat előnye, hogy a relatív elérési utak a projektgyökérből indulnak, a scriptek és a `data/`, `artifacts/` mappák helye egyértelmű.

A **sessionInfo()** az aktuális R-munkamenet auditja. Tartalmazza:

- az R-verziót és a platformot,
- a beágyazott base csomagok verzióját,
- az *attached packages* listát (például a `readr`-t, ha `library(readr)` után hívtuk),
- a *loaded via a namespace* listát, vagyis azokat a függőségeket, amelyeket a betöltött csomagok implicit módon betöltenek.

Fontos különbség: a `sessionInfo()` csak az aktuálisan betöltött csomagokról szól, míg a `renv.lock` a projekt *teljes* függőségi gráfját rögzítheti. Ezért a reprodukálhatósági auditban mindkettőre szükség van: a lock a célt adja meg, a `sessionInfo()` pedig az aktuális munkamenet tényleges állapotát igazolja.

37.1.3 Teljes minimális munkapélda: környezettérkép

A következő példa egy Linux/macOS gépen, a könyv repo gyökeréből indítható. Ha a tanuló Windows PowerShellt használ, a `cp` és `mkdir` parancsokat a `Copy-Item` és `New-Item` parancsok helyettesítik; az R-parancsok platformfüggetlenek.

Első lépésben hozd létre az R-projekt mappát, és másold be a tiszta `orders.csv` fájlt:

```
# platform: Linux/macOS; munkakönyvtár: a repo gyökere
mkdir -p northstar-r/data northstar-r/artifacts
cp lab/data/orders.csv northstar-r/data/orders.csv
cd northstar-r
```

Ezután futtasd az alábbi R-scriptet. A kód maga futtatható; a konkrét kimenet a géptől és a letöltött csomagverzióktól függ, ezért a kommenteket szemléltetőként jelölöm.

```
# setup_env.R
# Platform: Linux/macOS; futtatás a projektgyökérből:
#   Rscript setup_env.R
if (!requireNamespace("renv", quietly = TRUE)) {
  install.packages("renv")
}

# Projektizoláció létrehozása: renv.lock és renv/ mappa
renv::init()

# Csomag telepítése a projektkönyvtárba, majd lock-frissítés
renv::install("readr")
renv::snapshot()

# Adatbetöltés és kézi kontroll
df <- readr::read_csv("data/orders.csv", show_col_types = FALSE)
stopifnot(nrow(df) == 2400L)

# Munkamenet-audit mentése
sink("artifacts/sessioninfo_r.txt")
sessionInfo()
sink()

# Ellenőrző kinyerés
cat("R verzió:", R.version.string, "\n")
cat("readr verzió:", as.character(packageVersion("readr")), "\n")
cat("readr elérési út:", find.package("readr"), "\n")
```

Várható, szemléltető kimenet a script logjából:

```
* Initializing project ...
* Installing readr ...
* Lockfile written to 'renv.lock'
R verzió: R version 4.4.1 (2024-06-14)
readr verzió: 2.1.5
readr elérési út: /home/user/northstar-r/renv/library/R-4.4/x86_64-pc-linux-gnu/readr
```

Az utolsó sor kulcsfontosságú: az elérési út arra a könyvtárra mutat, amelyet a `renv::init()` hozott létre, nem pedig a gépszintű `site-library`-ra. Ha ez a sor máshová mutat, a projektizoláció nem működik.

A Python-megfelelőket összefoglaló környezettérkép:

Szerep	R eszköz	Python-megfelelő a lab gyakorlatában
Nyelvi interpreter	`R`, `R.home()`	`python`, `sys.executable`
Projektizoláció	`renv` + `.Rproj`	`venv` + `pyproject.toml`
Verziórögzítés	`renv.lock`	`uv.lock`
Kernelregisztráció	`IRkernel`	`ipykernel` kernelspec (3.4. alfejezet)

37.1.4 Eredmény értelmezése

A minimális példa után két auditfájlunk van: a `renv.lock` és az `artifacts/sessioninfo_r.txt`. Az értelmezés során a következő invariantokat ellenőrizzük:

1. **Sor- és dimenziókontroll**: `df` shape-je `c(2400, 15)`. A 2400 sor a lab generálási szerződéséből következik (8.4. alfejezet); a 15 oszlop a rendeléskori 11 ismert mezőből és a 4 utólagos célmezőből adódik. Ez a kézi ellenőrzés azt igazolja, hogy a betöltő kód a tiszta fájlt olvassa, nem az `orders_raw.csv`-t.
2. **Lock-invariant**: a `renv::status()` üres kimenetet ad, ha a projekt mappában lévő csomagok megegyeznek a lockkal. Ha eltérést jelez, a `renv::snapshot()` hiányzott vagy a könyvtárat kézzel módosították.
3. **Session-invariant**: a `sessionInfo()$otherPkgs$readr$Version` értéke megegyezik a `packageVersion("readr")` értékével, és a `renv.lock` `Packages$readr$Version` mezőjével. A verziók háromoldalú egyezése a reprodukálhatóság minimális bizonyítéka.
4. **Út-invariant**: a `find.package("readr")` a projekt `renv/library` alá mutat. Ez ugyanaz a gondolat, mint a 8.4. alfejezetben a fájlhash-ellenőrzés: az állítás csak akkor igaz, ha a tényleges beolvasott erőforrás is az, amit állítunk.

37.1.5 Feladatok

37.1.5.1 1. Értelmezés

Egy `sessionInfo()` kivonat így kezdődik:

```
R version 4.4.1 (2024-06-14)
Platform: x86_64-pc-linux-gnu
other attached packages:
readr 2.1.5
```

A `renv.lock` ugyanakkor `dplyr` 1.1.4-et is tartalmaz, de a `sessionInfo()` nem említi a `dplyr`-t. Miért lehetséges ez? Melyik fájl mit bizonyít?

37.1.5.2 2. Implementálás

Írj egy `verify_r_env.R` scriptet, amely a projektgyökérből futtatható, és a következőket ellenőrzi:

- a `renv.lock` létezik;
- a `readr` betölthető;
- az `orders.csv` 2400 sort tartalmaz;
- a `readr` útvonala a `renv/library` alá mutat;
- a script `0` kilépési kóddal tér vissza, ha minden igaz.

37.1.5.3 3. Hibadiagnózis

A kolléga a Posit IDE-ben `R.version.string` értékeként `4.4.1`-et lát, de a terminálban az `R` parancs `4.2.1`-et ír ki. Azt állítja, hogy „a rendszer R-je hibás”. Mi lehet a valódi magyarázat? Milyen parancsokkal deríted fel, és miért nem a rendszer R-je hibás?

37.1.5.4 4. Megváltozott feltétel: másik gépre átadás

A projektet egy másik, Linux helyett macOS-es gépen szeretné újra futtatni a kolléga, ahol az R 4.3.3 van telepítve, de még nincs `renv` csomag. Mi az a három minimális lépés, amellyel a környezetet visszaállítja? Adj meg egy korlátot, amely miatt a lock önmagában nem garantálja a bitazonos eredményt.

37.1.6 Megoldások és indoklás

37.1.6.1 1. Értelmezés

A `sessionInfo()` csak az aktuális munkamenetben *betöltött* csomagokat mutatja az `other attached packages` szakaszban. A `renv.lock` ezzel szemben a projekt *teljes* függőségi gráfját rögzítheti, beleértve azokat a csomagokat is, amelyeket jelenleg nem töltöttünk be a `library()` hívással. Elképzelhető, hogy a `dplyr` korábban telepítve lett a projektbe, de a jelenlegi script csak `readr`-t használ. A két fájl tehát nem ellentmondásos: a lock a célt, a `sessionInfo()` a munkamenet tényleges állapotát rögzíti. A reprodukálhatósági auditban a lock az elsődleges forrás; a `sessionInfo()` azt igazolja, hogy az aktuális futtatás a lock szerinti verziókat használta.

37.1.6.2 2. Implementálás

```
# verify_r_env.R
# Platform: Linux/macOS; futtatás: Rscript verify_r_env.R
stopifnot(file.exists("renv.lock"))
stopifnot(requireNamespace("readr", quietly = TRUE))

df <- readr::read_csv("data/orders.csv", show_col_types = FALSE)
stopifnot(nrow(df) == 2400L)

readr_path <- find.package("readr")
stopifnot(grepl("renv/library", readr_path, fixed = TRUE))

cat("OK: lock, readr, rows, path\n")
```

Az ellenőrző invariant a négy `stopifnot`: a lock létezése, a csomag betölthetősége, a sorok száma, valamint a projektizolált elérési út. A script hibás környezetben nullától eltérő kilépési kóddal áll le; ez a 29.4. alfejezetben bevezetett automatikus belépési küszöb R-megfelelője.

37.1.6.3 3. Hibadiagnózis

A Posit IDE nem az `R` terminálparancsot indítja, hanem a beállításaiban megadott interpretert. Ezért a két felület eltérő `R.home()` értéket adhat. A diagnózis:

```
Sys.which("R")                  # a terminál R parancsa
R.home()                        # az aktuális munkamenet gyökere
.libPaths()                     # a csomagkeresési sorrend
Sys.getenv("RSTUDIO_WHICH_R")   # ha a Posit IDE beállította
```

A megoldás az IDE-ben a *Global Options → General → R version* beállítás egységesítése, nem a rendszer „javítása”. A `renv::restore()` a terminálból a 4.2.1 alatt figyelmeztetést adhat, mert a lock `R/Version` mezője 4.4.1-et tartalmaz. Ez a jelenség pontosan a 3.4. alfejezetben látott kernelspec-eltérés analógja.

37.1.6.4 4. Megváltozott feltétel

A másik gépen, a projektmappa másolása után:

```
# 1. A renv csomag telepítése a felhasználói könyvtárba, ha még nincs
install.packages("renv")

# 2. A lockból a projektkönyvtár visszaállítása
renv::restore()

# 3. Ellenőrzés, hogy a projektcsomagok a locknak megfelelnek
renv::status()
```

A korlát: a `renv.lock` az R-verziót és a csomagverziókat rögzíti, de magát az R 4.4.1-et nem tartalmazza; a macOS gépen 4.3.3 fut, ezért a `renv::restore()` figyelmeztethet. Ráadásul a bináris csomagok hash-ei platformfüggők lehetnek, így a `renv::status()` nem bitazonossági bizonyíték; az R- és csomagverziók, valamint a deklarált projektállapot konzisztenciáját ellenőrzi. Ugyanez a helyzet a Python `venv` esetében is: a `.venv` nem hordozható, az `uv.lock` viszont az.

37.1.6.5 R runtime, rendszerkönyvtárak, locale és timezone

A `renv.lock` mellé **környezeti jelentés** kell, mert a lock nem rögzíti az R buildjét, a BLAS/LAPACK implementációt, a rendszerkönyvtárakat és a locale-t. Ezek mind befolyásolhatják a numerikus eredményt és a faktorok rendezését.

```
env_report <- function() {
  list(
    r_version = R.version.string,
    r_home = R.home(),
    platform = R.version$platform,
    renv_status = renv::status()$synchronized,
    ext_soft = extSoftVersion(),
    capabilities = capabilities(),
    locale = Sys.getlocale(),
    lc_collate = Sys.getlocale("LC_COLLATE"),
    timezone = Sys.timezone(),
    l10n = l10n_info()
  )
}
```

A jelentésben legalább ezek legyenek:

Mező	Példa	Miért kell
`r_version`	`"R version 4.4.1 (2024-06-14)"`	az interpreter pontos verziója
`platform`	`"x86_64-pc-linux-gnu"`	architektúra és OS
`renv_status`	`TRUE`	a lock és a telepített csomagok egyeznek
`ext_soft`	OpenBLAS/LAPACK verziók	a numerikus mag változhat
`capabilities`	`long.double`, `libcurl`	elérhető rendszerfunkciók
`locale`, `lc_collate`	`"en_US.UTF-8"`	faktorszintek és string-rendezés
`timezone`	`"UTC"`	időbélyeg-értelmezés

Két gépen az elemzés akkor reprodukálható, ha az R főverziója és minor verziója azonos, a `renv::status()` `TRUE`, a locale és a timezone rögzített, és a numerikus eredmény a dokumentált tolerancián belül egyezik. A `tests/test_r_environment.py` a jelentés kötelező mezőit, az R-verzió egyezését, a `renv_status` igazságát és a locale/timezone eltérését teszteli.

37.1.7 Gyakori tévedés és ellenpélda

Tévedés: „A Posit IDE maga az R, és a renv ugyanaz, mint a Python venv, csak R-ben."

Ellenpélda: Egy elemző a Posit IDE konzoljában `renv::init()`-et futtat, majd azt hiszi, hogy a projekt immár hordozható, mert a `renv/library` mappát felmásolja egy pendrive-ra. A másik gépen a projekt nem fut, mert az interpreter hiányzik, és a bináris csomagok sem egyeznek a másik operációs rendszeren. A `venv` is hasonló: egy `.venv` mappa átmásolása másik gépre nem garantálja a működést; a hordozható réteg a lockfile. Az IDE pedig csak megjeleníti az interpretert; a tényleges futtatókörnyezet az `R.home()` által mutatott program. Ezért mindig a lock + `sessionInfo()` + `find.package()` hármast validáld, ne a felület állapotát.

37.1.8 Önellenőrzés

Tedd fel magadnak a következő három kérdést:

1. A `sessionInfo()` R-verziója megegyezik a `renv.lock` `R/Version` mezőjével?
2. A `find.package()` által mutatott útvonal a projekt `renv/library` alá esik, nem a gépi `site-library` alá?
3. Egy frissen klónozott projektmappában a `renv::restore()` majd `renv::status()` üres státuszt ad-e?

Ha bármelyik válasz nem, akkor a környezeti reprodukálhatóság állítása hamis volt, még akkor is, ha a Script a saját gépén lefutott.

37.1.9 Rövid összefoglaló

Az R-környezet négy rétegből áll: az R-interpreter, a Posit IDE, a `library path` által leírt csomagkönyvtárak és a `renv.lock` által rögzített csomagverziók. A projekt `.Rproj` fájlja a munkakönyvtárat, a `sessionInfo()` az aktuális munkamenetet auditálja. A reprodukálhatósághoz nem elég a script; a lock, a munkamenet-audit és a csomag-elérési út ellenőrzése együttesen bizonyítja, hogy a megfelelő környezet futott. A következő alkalmazásban 37.2. alfejezet az R táblázatos adatszerkezeteit és formula-interfészét használjuk, hogy ugyanezen a projektben statisztikai modellt illesszünk és értelmezzünk.

37.2 Tábla és formula R-ben

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: R, renv

Előfeltétel: 37.1. alfejezet · Célkészség: Tábla és formula R-ben · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

37.2.1 Helyzet és cél

A munkamenet célja, hogy a Pythonban már ismert kis táblaaggregációt és egy lineáris regressziót R-ben reprodukáljunk, és közben pontosan rögzítsük, melyik lépés mit csinál. Bemenet a Northstar szintetikus labor `lab/data/orders.csv` fájlja; minden sor egy rendelés (`order_id`), tehát a megfigyelési egység a rendelés. Az oszlopok közül most a `region`, `segment`, `distance_km`, `priority_flag`, `basket_minor` és az utólag megfigyelt `delivery_days` kell. Cél két kimenet: régiónkénti átlagos kosárérték és késési arány, majd egy `delivery_days` kimenetű lineáris modell. A modell itt oktatólagos asszociációs elemzés; nem a rendelés pillanatában használható prediktív szerződés, mert a `delivery_days` utólag ismert.

Induló becslés a kód futtatása előtt: a `distance_km` együtthatója várhatóan pozitív, hiszen a távolság növekedése hosszabb kézbesítést valószínűsít. A mini kontrollban a pontos meredekség 0,32 nap/km lesz; a labor nagyobb adatán ettől eltérő, de előjelében hasonló értéket várunk. A `basket_minor` centben mért kosárérték hatása valószínűleg kicsi, ezért inkább euróra skálázva érdemes a modellbe tenni.

37.2.2 Tábla: `data.frame` és `tibble`

A `data.frame` az R alap táblaszerkezete: oszlopokból álló lista, ahol minden oszlop vektornak tekinthető. A `tibble` a tidyverse modern táblája, amelyet a `readr::read_csv()` is visszaad. A legfontosabb eltérés a karakterkezelés: a hagyományos `data.frame` régebbi alapbeállításai karakteroszlopokat faktorrá alakíthattak, a `tibble` azonban karakterként hagyja meg őket. Ez a Northstar adatnál kritikus, mert a `customer_id` szintetikus szöveges azonosító, a `region` és a `segment` pedig kategória.

```
library(tibble)

d <- tibble(
  x = c(1, 2, NA),
  g = c("a", "b", "a"),
  y = c(10, 20, 30)
)

d
is.character(d$g)   # TRUE: a tibble nem alakította factorrá
```

A tibble nyomtatása csak az első sorokat mutatja, és az oszloptípusokat is kiírja. Indexelésnél a `d[1, ]` mindig tibble marad, a `d[[1]]` vagy `d$x` viszont vektort ad. Ezt a `dplyr`-rel való munka közben nem kell állandóan fejben tartani, de a Python `pandas` `Series`/`DataFrame` kettősségéhez hasonló fogalom.

A kategorikus változó R-ben külön típus: `factor`. A faktor tárolja a lehetséges szinteket, és a szintek sorrendje befolyásolja a statisztikai kódolást.

```
segment <- factor(c("consumer", "business", "consumer"))
levels(segment)   # "business" "consumer" : ábécé szerint rendezve
```

A Northstar `segment` csak `consumer` vagy `business` lehet, a `region` pedig `north`, `central` vagy `south`.

37.2.3 `dplyr`: a kis aggregáció

A `dplyr` a táblaműveletek nyelvét adja. A natív `|>` cső (pipe) a bal oldali eredményt teszi a jobb oldali függvény első argumentumává. A leggyakoribb igék:

Ige	Feladat	Pythonos analógia
`filter()`	sorok megtartása feltétel szerint	`df.loc[mask]`
`mutate()`	új vagy módosított oszlop	`df.assign()`
`group_by()`	csoportosítás	`df.groupby()`
`summarise()`	csoportonkénti összegzés	`.agg()`
`arrange()`	rendezés	`.sort_values()`
`select()`	oszlopok kiválasztása	`df[cols]`

A `summarise()` a csoportosítás után minden csoportra pontosan egy értéket ad; a `mean()` alapértelmezésben `na.rm = FALSE`, tehát ha a csoportban bármely érték `NA`, az összegzett átlag is `NA` lesz. Ezt a hiányzó érték szakaszban részletezem.

Egy pillanatnyi kitérő a pénzegységre: a Northstar `basket_minor` egész szám EUR-centben van. Az átlagos kosárértéket euróban akkor kapjuk, ha a `basket_minor / 100` oszlopot összegezzük. A centben maradó átlag a `minor` mértékegységet örökíti tovább; a két szám nem felcserélhető.

37.2.4 Formula, `lm` és kategorikus kódolás

Az R formula szintaxisa a modell szerkezetét írja le: a `~` bal oldalán a célváltozó, jobb oldalán a magyarázó változók szerepelnek. A `delivery_days ~ distance_km + priority_flag + segment` formula azt mondja,

Interpretációs figyelmeztetés. A regressziós együttható csak a modellben szereplő többi változó szerinti feltételes asszociáció. Causális hatásként kizárólag külön indokolt azonosíthatósági feltevésekkel értelmezhető. hogy a kézbesítési napok számát a távolsággal, a prioritásjelzővel és a szegmenssel magyarázzuk. A `+` itt nem aritmetikai összeadás, hanem újabb modelltag bevezetése; interakciót a `:` jelöl, a `*` pedig az interakciót és a főhatásokat együtt adja.

A `lm()` az OLS-becslést végzi el:

$$\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{y},$$

ahol $\mathbf{y} \in \mathbb{R}^n$ a célváltozó vektora, $X \in \mathbb{R}^{n \times p}$ a magyarázó változók tervezett mátrixa, p a becsült paraméterek száma, $\hat{\beta} \in \mathbb{R}^p$ pedig a becsült együtthatók vektora. A j-edik együttható standard hibája $s(\hat{\beta}_j)$, a t-próba statisztikája pedig $\hat{\beta}_j / s(\hat{\beta}_j)$; a reziduális szabadságfok $n - p$. Ez ugyanaz a mennyiség, amelyet a Python `statsmodels` OLS-e is jelent; a becsült értékek akkor egyeznek, ha a kódolás is egyezik.

A kategorikus változónál az R `lm()` alapértelmezett kezelési kontrasztot (treatment contrast) használ: a faktor ábécé szerinti első szintje a referenciaszint, a többi szinthez egy-egy dummy oszlop tartozik. A `segment` esetén a referenciaszint ezért `consumer`, és a modellmátrixban egyetlen `segmentbusiness` oszlop jelenik meg, amely a `business` sorokban 1, a `consumer` sorokban 0. A `segmentbusiness` együttható így a business és a consumer szegmens átlagos eltérése a többi magyarázó változó rögzítése mellett.

37.2.5 Hiányzó érték

Az R-ben a hiányzó érték jelölése `NA`. Az alapvető statisztikai függvények nem ugranak át rajta automatikusan: `mean(c(1, 2, NA))` eredménye `NA`, amíg a `na.rm = TRUE` argumentumot meg nem adjuk. A `dplyr::filter(!is.na(oszlop))` a hiányos sorokat explicit módon eldobja, a `tidyr::drop_na()` ugyanezt több oszlopra is elvégzi.

Az `lm()` alapértelmezett `na.action` beállítását a környezet `options("na.action")` értéke határozza meg (a szokásos alapbeállítás `na.omit`): az illesztés előtt eltávolítja azokat a sorokat, ahol bármely, a formulában szereplő változó hiányzik. A felhasznált sorok számát a `nobs(modell)` adja vissza, a reziduális szabadságfokot a `df.residual`. Ha tehát a tiszta `orders.csv` 2400 teljes sort tartalmaz, a modell `nobs` értéke 2400; ha két hiányzó távolság van, akkor 2398 — ezt a különbséget mindig ellenőrizni kell.

37.2.6 Teljes minimális munkapélda

A példa feltételezi, hogy a 37.1. alfejezetben létrehozott `lab/r` renv-projekt aktív, és a `readr` meg a `dplyr` telepítve van. A munkakönyvtár `lab/r`, a CSV elérése ezért relatívan `../data/orders.csv`.

Először a kézzel ellenőrizhető mini adat:

```
library(tibble)

mini <- tibble(
  distance_km = c(5, 10, 15, 20),
  delivery_days = c(3, 5, 6, 8)
)

mini_fit <- lm(delivery_days ~ distance_km, data = mini)
coef(mini_fit)
# Szemléltető kézi kontrollból:
# (Intercept) distance_km
#         1.5         0.32
```

A kézi kontroll: a távolság átlaga $12,5$ km, a napok átlaga $5,5$ nap; a nevező $\sum (x_i - \bar{x})^2 = 125$, a számláló $\sum (x_i - \bar{x})(y_i - \bar{y}) = 40$, tehát a meredekség $40/125 = 0,32$ nap/km, az intercept $5,5 - 0,32 \cdot 12,5 = 1,5$ nap. A 20 km-re adott predikció $1,5 + 0,32 \cdot 20 = 7,9$ nap, a tényleges 8 naphoz képest 0,1 nap maradék.

Most a Northstar labor aggregációja és modellje:

```
library(readr)
library(dplyr)

orders <- read_csv("../data/orders.csv") |>
  mutate(
    segment = factor(segment),
    region = factor(region),
    basket_eur = basket_minor / 100
  )

region_agg <- orders |>
  group_by(region) |>
  summarise(
    n_orders = n(),
    avg_basket_eur = mean(basket_eur),
    avg_delivery_days = mean(delivery_days),
    late_rate = mean(target_late),
```

```
    .groups = "drop"
  )

model <- lm(
  delivery_days ~ distance_km + priority_flag + segment + basket_eur,
  data = orders
)

dir.create("results", showWarnings = FALSE)
write_csv(region_agg, "results/s146_region_agg.csv")
saveRDS(model, "results/s146_model.rds")
summary(model)
```

A platformjelölt futtatás, ha a fenti kódot `scripts/s146_example.R` néven mentjük:

```
# munkakönyvtár: lab/r
Rscript scripts/s146_example.R
```

A labor kimenetét itt nem nevezem ellenőrzöttnek, mert a kapcsolódó példaprojektben még nem ellenőrzött kódot. Az alábbi ellenőrző feltételek viszont a futtatás után számszerűen tesztelhetők, és a mini kontroll kézi számítása pontos.

37.2.7 Eredmény értelmezése és kézi kontroll

A mini modellben a `distance_km` együttható mértékegysége nap/km: egy km távolságnövekedés a mintában átlagosan 0,32 nappal hosszabb kézbesítéssel jár. Az intercept 1,5 nap az elméleti 0 km-hez tartozó modellérték; ezt nem szabad tényleges üzleti előrejelzésként értelmezni, mert 0 km-es rendelés a tartományon kívül van. A 7,9 napos predikció a 20 km-es sorhoz a képletből adódik; a maradék 0,1 nap a modell hibája.

A labor modellnél az ellenőrző invariánsok: `nrow(region_agg)` értéke 3; `sum(region_agg$n_orders)` értéke 2400, mert a tiszta `orders.csv` minden sora teljes; `model$rank` értéke 5, hiszen az intercept mellett `distance_km`, `priority_flag`, `segmentbusiness` és `basket_eur` paraméter van; `df.residual(model)` értéke 2395. A tolerancia felesleges tizedesjegyekre: a mini meredekség kerekítése 0,01 nap/km, a kézi kontrollban ezért a 0,32 pontos.

A `segmentbusiness` koefficiens a business szegmens és a consumer referenciaszint közti átlagos különbség napban, a többi változó rögzítése mellett. A `basket_eur` együtthatója nap/EUR, tehát kis érték várható; ha a felhasználó a centben mért `basket_minor`-t teszi a modelbe, az együttható 100-ad része lesz, mert a magyarázó változó 100-szor nagyobb.

A kód most a `target_late` értéket csak a leíró aggregációban használja, nem prediktorként. A prediktív Northstar feladatban a `delivery_days` és a belőle képzett `target_late` rendeléskori prediktorba tiltott; ez a fejezet statisztikai asszociációt mutat be, nem prediktív szolgáltatást.

37.2.8 Feladatok

1. **Értelmezés.** A mini modell `summary(mini_fit)` kimenetében a `distance_km` együttható 0,32, a standard hibája körülbelül 0,028 nap/km, a t-érték körülbelül 11,3. Mit jelent ez a három szám? Miért nem következik belőle, hogy a távolság okságilag növeli a kézbesítési időt?

2. **Megvalósítás.** Futtasd a labor kódot, és írd ki a `summary(model)` `segmentbusiness` sorának becslését és standard hibáját. Ellenőrizd, hogy `sum(region_agg$n_orders)` pontosan 2400, majd magyarázd meg, miért ennyi.

3. **Hibadiagnózis.** Olvasd be az `../data/orders_raw.csv` fájlt, és futtasd a `mean(orders_raw$distance_km)` kifejezést. Az eredmény `NA`. Miért? Javítsd a hiányzó érték kezelését, és add meg, hány teljes sort használna az `lm()` a `delivery_days ~ distance_km` modellnél, ha a `distance_km` oszlopban két `NA` van.

4. **Megváltozott feltétel.** A labor modellben cseréld le a `basket_eur` változót `basket_minor`-ra. Mi történik a kosárérték együtthatójával és a standard hibájával? Ezután állítsd a szegmens faktort `factor(segment, levels = c("business", "consumer"))` sorrendre, és futtasd újra a modellt. Mi lesz az új referenciaszint, és hogyan változik az intercept meg a `segmentconsumer` együttható?

37.2.9 Megoldások és indoklás

1. A 0,32 azt becsli, hogy a távolság egy km-es növekedése a mintában átlagosan hány nappal hosszabb kézbesítéssel jár. A standard error 0,028 a becslés mintavételi bizonytalansága; a t-érték $0,32/0,028 \approx 11,3$ azt mutatja, hogy a becslés ennyi standard hibányira van a nullától. Ettől még ez korrelációs, nem oksági állítás: a mini adat nem kontrollálja az összes zavaró tényezőt, és a minta nem randomizált.

2. A futtatható ellenőrzés:

```r
coef(model)["segmentbusiness"]
sqrt(diag(vcov(model)))["segmentbusiness"]
stopifnot(sum(region_agg$n_orders) == nrow(orders))
```

Az összeg azért 2400, mert a tiszta `orders.csv` minden sora egy rendelés, a `group_by(region)` három csoportot képez, és a `n()` minden sort pontosan egyszer számol.

3. A `mean()` alapértelmezése `na.rm = FALSE`, ezért bármely `NA` az átlagot is `NA`-vá teszi. A javítás `mean(distance_km, na.rm = TRUE)` vagy `filter(!is.na(distance_km))`. A `sum(is.na(orders_raw$distance_km))` értéke 2; az `lm()` `na.omit` alapértelmezésével a 2405 sorból 2403 teljes sort használ. A negatív `basket_minor` és a nem létező `customer_id` a távolságalapú modellnél nem hiányzó sor, ezért nem dobódik el; az adatminőségi döntés külön lépés marad.

4. Ha a `basket_eur` helyett `basket_minor` szerepel, a magyarázó változó 100-szor nagyobb lesz, ezért az együttható és a standard hibája is 100-ad részére csökken; a t-érték, a p-érték és az intercept változatlan marad. A `factor(segment, levels = c("business", "consumer"))` esetén az új referenciaszint `business`; az új intercept a régi intercept plusz a régi `segmentbusiness` együttható; az új `segmentconsumer` együttható a régi `segmentbusiness` negatívja. A modell predikciói soronként ugyanazok maradnak.

37.2.10 Gyakori tévedés ellenpéldával

A két nyelv hasonló nevű műveletei nem mindig azonos alapértelmezést használnak. Az R `mean()` alapból nem hagyja ki a hiányzó értéket, a Python `pandas Series.mean()` viszont alapból igen.

```r
mean(c(1, 2, NA))   # NA
```

```python
import pandas as pd
pd.Series([1, 2, float("nan")]).mean()  # 1.5
```

A másik gyakori félreértés a kategorikus kódolás. Az R `lm()` a `consumer` referenciaszinthez méri a `business` eltérést, és egyetlen `segmentbusiness` dummy oszlopot készít. A Python `pandas.get_dummies(drop_first=False)` viszont mindkét szintre oszlopot ad; ha egy `sklearn` lineáris modellt úgy illesztünk, hogy a `segment_consumer` és a `segment_business` oszlop is benne van, a kapott együtthatók jelentése nem egyezik az R `segmentbusiness` kontrasztjával. A `statsmodels` formula-illesztése viszont referenciaszintet használ, ezért az R-hez közelebb áll. Mindig a `model.matrix()` vagy a `contrasts()` kimenetével ellenőrizd, hogy a kódolás ugyanazt a hipotézist becsüli-e.

37.2.10.1 Faktorszint, relevel, kontraszt és NA-politika

Az R a karakteroszlopot a formula kedvéért faktorrá alakítja, és alapértelmezésben az **ábécé szerinti első szint** lesz a referenciaszint. A `segment` szintjei `c("business", "consumer")` sorrendben ábécé szerint `business`, `consumer`; ha a kód `factor(segment, levels = c("consumer", "business"))`-et használ, a referencia a `consumer`.

```
df$segment <- factor(df$segment, levels = c("consumer", "business"))
contrasts(df$segment)
#            business
# consumer          0
# business          1

model.matrix(~ segment, data = df)
#   (Intercept) segmentbusiness
# 1           1               0
# 2           1               1

df$segment <- relevel(df$segment, ref = "business")
model.matrix(~ segment, data = df)
#   (Intercept) segmentconsumer
# 1           1               1
# 2           1               0
```

A kézi design matrix ellenőrzése: az első modellben az intercept a `consumer` átlaga, a `segmentbusiness` együttható a `business - consumer` különbség. A `relevel` után az intercept a `business` átlaga, és a `segmentconsumer` a `consumer - business` különbség; a predikció ugyanaz, csak a paraméterezés változik. A kontrasztot explicit módon is meg lehet adni: `contrasts(df$segment) <- contr.treatment(levels(df$segment), base = "consumer")`.

A **hiányzó értékek** kezelése szintén explicit: az `lm()` a formulában szereplő változók bármelyikének `NA` értéke esetén kihagyja a sort. A kihagyott sorok száma a `model.frame() nrow` és a bemenet különbségéből látszik; `na.action = na.exclude` megőrzi a hiányzó helyeket a reziduumokban. A jelentésben rögzíteni kell a `n_used` és `n_missing` értéket, és tilos a hiányzó kategóriát csendben „other” szinttel helyettesíteni.

A Python `statsmodels` ekvivalens formulája:

```
import statsmodels.formula.api as smf
model = smf.ols(
    "basket_sum_minor ~ C(segment, Treatment(reference='consumer'))",
    data=df,
).fit()
print(model.model.exog_names)  # ['Intercept', 'C(segment, ...)[T.business]']
```

A `tests/test_factor_reference.py` a referenciaszint-váltás design mátrixát, a kontrasztoszlopok számát és a hiányzó sorok kihagyását teszteli. Az R `glm` és a Python `statsmodels` predikciója csak azonos szint- és kontrasztbeállítás mellett hasonlítható össze.

Visszautalás: a kategorikus reprezentációról és a one-hot döntésről a 25.2. alfejezet, az OLS statisztikai értelmezéséről a 17.1. alfejezet, a hiányzó címkés értékelési ablakról pedig a 36.2. alfejezet szól.

37.2.11 Önellenőrzés

1. Miért marad karakter a `segment` egy `tibble`-ben, és miért alakítja az `lm()` formula mégis faktorrá?
2. Hány sort használ egy `lm()` modell, ha a formulában szereplő egyik oszlopban 3 `NA` van egy 100 soros adatban?
3. A `segment` faktorban melyik szint a referenciaszint alapértelmezésben, és miért?

37.2.12 Rövid összefoglaló

A `tibble` karakteres oszlopokat nem faktorizál, a `dplyr` csöves igéi pedig explicit, ellenőrizhető aggregációt adnak. A formula a cél–prediktor viszonyt írja le az `lm()` számára, amely OLS-t becsül és kezelési kontrasztot használ: az ábécé szerinti első szint a referencia. A hiányzó `NA` az R-ben explicit; az alapműveletek gyakran `na.rm = TRUE` beállítást igényelnek, az `lm()` pedig a hiányos sorokat kihagyja. A Python `pandas` alapértelmezései eltérhetnek, ezért hasonló nevű kimeneteket csak a kódolás ellenőrzése után szabad összehasonlítani. A következő 37.3. alfejezet ebből az aggregációból és modellből készít R-jelentést, és rögzíti a Python–R közötti adatcsere formáját.

37.3 Jelentés és nyelvek közötti adatcsere

> **Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: R, renv**
>
> Előfeltétel: 37.2. alfejezet · Célkészség: Jelentés és nyelvek közötti adatcsere · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.
>
> **Reprodukálhatósági ellenőrzés:** Python–R adatcserénél teszteld a hiányértékeket, a faktor/kategória szinteket, a dátumok időzónáját és a 64 bites egész értékeket; a CSV nem garantálja a teljes típushű visszaolvasást.

37.3.1 Cél és konkrét helyzet

A 37.2. alfejezetben R-ben már dolgoztunk táblákkal, formulákkal és lineáris modellel. Most azt a gyakori munkamenetet nézzük meg, amikor a Pythonban elkészített elemzési táblát egy R-t használó kolléga statisztikai modellje, végül pedig egy Quarto-jelentés használja fel. A cél az, hogy az átadás ne egy „remélhetőleg ugyanaz” adatfájl legyen, hanem ellenőrzött szerződés: ugyanaz a séma, ugyanazok a kontrollértékek, és mindkét oldalon lefuttatott invariáns-ellenőrzés.

A konkrét helyzetben a megfigyelési egység az ügyfél. A Python oldal egy 6 soros, szándékosan kicsi szintetikus táblát készít: `customer_id`, `segment`, `region`, `order_count`, `basket_sum_minor`, `late_rate` és `last_order_ts`. Az adatot Parquet-ben, egy `schema.json` és egy `controls.json`

kíséretében adjuk át. Az R oldal beolvassa, lefuttatja ugyanazokat az ellenőrzéseket, majd egy `lm` modellt és egy Quarto-jelentéstervet készít. Mielőtt bármit lefuttatnánk, kézzel becsüljük meg az eredményt: a `basket_sum_minor` összege a fenti hat sorra 81 060 EUR-cent, a `late_rate` átlaga körülbelül 0,161, a sorok száma 6.

37.3.2 Fogalmi magyarázat

37.3.2.1 Parquet és Arrow

A Parquet egy oszlopos, bináris fájlformátum, amely a sémát a fájlban hordozza. A pandas `to_parquet` hívása a `pyarrow` motoron keresztül Arrow memóriaformátumba konvertál, majd onnan ír Parquet-t. Az R oldalon az `arrow` csomag `read_parquet` függvénye ugyanazt a sémát próbálja visszaállítani. Előnye a CSV-vel szemben, hogy a `int64`, `float64`, `timestamp[us, tz=UTC]` és kategória típusok nem lapulnak puszta szöveggé. A kategória átvitele azonban nem garantáltan faktorrá alakul R-ben: a pandas `category` oszlop Parquet-ben gyakran dictionary típusú lesz, az R `arrow` pedig dictionary típusként olvashatja, nem közvetlenül `factor`-ként. Ezért a sémafájlban explicit `levels` listát adunk, és az R oldalon mi magunk alakítunk faktort.

37.3.2.2 CSV fallback

A CSV akkor jön szóba, ha a fogadó környezetben nincs Arrow/Parquet támogatás, vagy egy külső rendszer csak CSV-t fogad. A CSV karakterfolyam, tehát a típusinformáció elveszik. A fallback csak akkor biztonságos, ha a `schema.json` megadja a visszaállítási szabályokat: oszloptípus, időzóna, kategóriaszintek, hiányzó érték jelölése és pénznem/mértékegység. A gyakorlatban a CSV-t a `readr::read_csv`-vel olvassuk, majd a séma alapján explicit `mutate` vagy `parse_*` lépéseket hajtunk végre.

37.3.2.3 Quarto

A Quarto egy dokumentumformátum, amely Markdown-szöveget és futtatható kódblokkokat egyesít. Egy `.qmd` fájl lehet tisztán R, tisztán Python vagy akár kevert nyelvű. A `quarto render report.qmd` parancs a projekt virtuális környezetét és a bejegyzett kernelt használja. A jelentés attól lesz ellenőrzött, hogy a kódblokkja beolvassa az adatot, lefuttatja a kontrollokat, és csak ezután írja ki a modellt. Így a jelentés és az adatátadási szerződés együtt reprodukálható.

37.3.2.4 environment

A környezet nem a notebook és nem az IDE beállítása. A Python oldalon a projekt `pyproject.toml` és a feloldott `uv.lock` rögzíti a csomagokat; az R oldalon a `renv.lock` ugyanezt teszi. A Quarto-renderelés előtt ellenőrizni kell, hogy a kernelspec a megfelelő R környezetre mutat. A titkok, hozzáférési kulcsok és valós ügyféladatok továbbra sem kerülhetnek a `schema.json`-ba, a `controls.json`-ba vagy a jelentésbe; ezt már a 36.3. alfejezet is rögzítette. A sémafájl csak típus- és mértékegység-információt hordoz.

37.3.2.5 Schema

A semleges adatátadási szerződés nálunk egy JSON fájl. Minden oszlophoz tartozik név, típus, mértékegység, nullázhatóság, kategória esetén `levels`, időbélyeg esetén `timezone`. A szerződés akkor hasznos, ha mindkét oldal gépiesen ellenőrzi, nem csak elolvassa.

37.3.2.6 Adatmegmaradási invariánsok és numerikus tolerancia

A nyelvközi adatátadás matematikai alapja az invariáns-ellenőrzés. A következő mennyiségeknek kell egyezniük vagy tolerancián belül maradniuk:

- sorok száma,
- egész oszlopok összege,
- lebegőpontos oszlopok átlaga,
- kategóriaszintek halmaza,
- időbélyegek minimuma és maximuma UTC-ben.

A lebegőpontos összegek kerekítési hibája miatt szükség van toleranciára. **Nincs egyetlen univerzális allclose-képlet**: az implementációk eltérő referenciamennyiséget használnak.

Implementáció	Feltétel	Relatív referencia
NumPy `np.allclose(a, b, rtol, atol)`	$\lvert a-b\rvert \le \text{atol} + \text{rtol}\cdot\lvert b\rvert$	a második argumentum, b
Python `math.isclose(a, b, rel_tol, abs_tol)`	$\lvert a-b\rvert \le \max(\text{rel_tol}\cdot \max(\lvert a\rvert,\lvert b\rvert),\ \text{abs_tol})$	a két érték maximuma
R `all.equal(a, b, tolerance)`	skalárnál $\lvert a-b\rvert/\lvert b\rvert \le \text{tolerance}$	a célérték, b

A különbség nem elméleti: `a=1.0`, `b=1.000000001`, `atol=rtol=1e-9` mellett a NumPy `allclose` igaz, mert a tűrés $10^{-9} + 10^{-9}\cdot 1.000000001 \approx 2\cdot 10^{-9}$, a `math.isclose` viszont hamis, mert a tűrése $\max(1.000000001\cdot 10^{-9}, 10^{-9}) \approx 10^{-9}$. A két oldal közötti összehasonlításhoz ezért **egyetlen konvenciót** kell rögzíteni; ebben a fejezetben a NumPy-formulát használjuk, a-val a vizsgált, b-vel a referenciaértéket, $\text{atol} = \text{rtol} = 10^{-9}$ mellett. Az R oldalon ugyanezt a formulát külön ki kell számolni, nem az `all.equal` alapértelmezésére bízni.

Például ha Pythonban $\bar{x} = 0{,}1613095$, R-ben pedig 0,16130949999999999, akkor a különbség körülbelül 10^{-18}, ami bőven a tolerancián belül van. Egész összegeknél viszont bitpontos egyezést várunk, mert az `int64` nem kerekít.

37.3.3 Teljes minimális munkapélda

A példa feltételezi, hogy a projekt virtuális környezetében elérhető a `pandas` és a `pyarrow`, R-ben pedig az `arrow`, `dplyr` és `jsonlite`. A Python kódot a projekt gyökeréből futtatjuk. Az alábbi adat szintetikus, nem a Northstar lab fájlja; a cél a kis kézi ellenőrizhetőség.

```
# export_transfer.py
import json
from pathlib import Path

import pandas as pd

rows = {
```

```
    "customer_id": ["C001", "C002", "C003", "C004", "C005", "C006"],
    "segment": pd.Categorical(
        ["consumer", "business", "consumer", "business", "consumer", "business"],
        categories=["consumer", "business"],
    ),
    "region": pd.Categorical(
        ["north", "north", "central", "south", "central", "south"],
        categories=["north", "central", "south"],
    ),
    "order_count": [3, 7, 2, 5, 4, 8],
    "basket_sum_minor": [10250, 20480, 5310, 15220, 8700, 21100],
    "late_rate": [0.0, 0.142857, 0.5, 0.2, 0.0, 0.125],
    "last_order_ts": pd.to_datetime(
        [
            "2025-03-01T12:00:00+00:00",
            "2025-02-28T18:30:00+00:00",
            "2025-04-01T09:15:00+00:00",
            "2025-03-15T08:00:00+00:00",
            "2025-03-20T16:45:00+00:00",
            "2025-04-02T11:30:00+00:00",
        ]
    ).tz_convert("UTC"),
}

df = pd.DataFrame(rows)

transfer_dir = Path("transfer")
transfer_dir.mkdir(exist_ok=True)

df.to_parquet(transfer_dir / "customers.parquet", index=False)

schema = {
    "primary_key": "customer_id",
    "observation_unit": "customer",
    "columns": [
        {"name": "customer_id", "type": "string", "nullable": False},
        {
            "name": "segment",
            "type": "category",
            "levels": ["consumer", "business"],
            "nullable": False,
        },
        {
            "name": "region",
            "type": "category",
            "levels": ["north", "central", "south"],
            "nullable": False,
        },
        {"name": "order_count", "type": "int64", "unit": "count", "nullable": False},
        {
            "name": "basket_sum_minor",
            "type": "int64",
            "unit": "EUR cent",
            "nullable": False,
        },
        {
            "name": "late_rate",
            "type": "float64",
            "unit": "proportion",
            "range": [0.0, 1.0],
            "nullable": False,
        },
        {
```

```
            "name": "last_order_ts",
            "type": "timestamp[us]",
            "timezone": "UTC",
            "nullable": False,
        },
    ],
}

controls = {
    "row_count": int(len(df)),
    "sum_basket_minor": int(df["basket_sum_minor"].sum()),
    "mean_late_rate": float(df["late_rate"].mean()),
    "min_last_order_ts": df["last_order_ts"].min().isoformat(),
    "max_last_order_ts": df["last_order_ts"].max().isoformat(),
    "min_last_order_ts_epoch_us": int(df["last_order_ts"].min().value // 1000),
    "max_last_order_ts_epoch_us": int(df["last_order_ts"].max().value // 1000),
    "segment_levels": df["segment"].cat.categories.tolist(),
}

(transfer_dir / "schema.json").write_text(json.dumps(schema, indent=2))
(transfer_dir / "controls.json").write_text(json.dumps(controls, indent=2))

print(controls)
```

37.3.3.1 Séma-assertion és típushűség: missing, UTC, int64, kategóriasorrend

A `schema.json` nem dokumentáció, hanem **végrehajtható szerződés**. A kontrolladatnak legalább négy határesetet kell tartalmaznia:

Eset	Elvárás	Gyakori hiba
hiányzó érték	`nullable: false` esetén nincs `NA`; `nullable: true` esetén a számolt kontroll kihagyja	CSV-ben az üres string `NA`-vá válik, vagy fordítva
timestamp	UTC-ben, mikroszekundum-pontossággal; a zóna nem DST-s	naiv timestamp vagy `Europe/Budapest`
hosszú egész	`int64`, a 2^53 feletti érték is pontos	CSV/JSON double-ként kerekít
kategóriasorrend	a `levels` lista **sorrendje** egyezik a pandas `Categorical.categories` sorrendjével	`setequal` csak halmazt néz, a referenciaszint elcsúszik

A Parquet/Arrow megőrzi az `int64`, `float64`, `timestamp[us, tz=UTC]` és dictionary típust, de az R `arrow` a dictionary-t nem feltétlenül `factor`-ként adja vissza; ezért a `factor(..., levels = schema_levels)` explicit. A CSV ezzel szemben karakterfolyam: a visszaolvasáskor a séma alapján kell parse-olni, a 2^53 feletti egészt nem szabad double-ként olvasni.

```
import pandas as pd

def assert_schema(df, schema):
    errors = []
    columns = {item["name"]: item for item in schema["columns"]}
    if set(df.columns) != set(columns):
```

```
        errors.append("oszlophalmaz eltér")
    for name, spec in columns.items():
        series = df[name]
        if spec.get("nullable") is False and series.isna().any():
            errors.append(f"{name}: nem várt hiányzó érték")
        if spec["type"] == "int64" and not pd.api.types.is_integer_dtype(series):
            errors.append(f"{name}: int64 várt")
        if spec["type"] == "float64" and not pd.api.types.is_float_dtype(series):
            errors.append(f"{name}: float64 várt")
        if spec["type"] == "category":
            if not isinstance(series.dtype, pd.CategoricalDtype):
                errors.append(f"{name}: category várt")
            elif list(series.cat.categories) != list(spec["levels"]):
                errors.append(f"{name}: levels sorrendje eltér")
        if spec.get("timezone") == "UTC":
            if not pd.api.types.is_datetime64_any_dtype(series) or series.dt.tz is None:
                errors.append(f"{name}: időzónás timestamp várt")
    if errors:
        raise ValueError("; ".join(errors))
    return True
```

Az R oldalon ugyanez a kategóriasorrend-ellenőrzés `identical(levels(df$segment), seg_spec$levels)`; a `setequal` csak a halmazt vizsgálná, és a `consumer`/`business` referenciaszint felcserélését nem fogná meg. A `tests/test_transfer_schema.py` a missing-, UTC-, int64- és kategóriasorrend-eseteket, valamint a séma és a pandas categoricals egyezését teszteli.

Linux/macOS terminálban a munkakönyvtár a projekt gyökere:

```
python export_transfer.py
```

Windows PowerShellben ugyanez:

```
python export_transfer.py
```

A futtatás elméletileg várt, szemléltető kimenete:

```
{'row_count': 6,
 'sum_basket_minor': 81060,
 'mean_late_rate': 0.16130950000000002,
 'min_last_order_ts': '2025-02-28T18:30:00+00:00',
 'max_last_order_ts': '2025-04-02T11:30:00+00:00',
 'min_last_order_ts_epoch_us': 1740767400000000,
 'max_last_order_ts_epoch_us': 1743593400000000,
 'segment_levels': ['business', 'consumer']}
```

Az R oldal ezt a szerződést olvassa és ellenőrzi:

```
# analyze_transfer.R
library(arrow)
library(dplyr)
library(jsonlite)

df <- read_parquet("transfer/customers.parquet")
schema <- fromJSON("transfer/schema.json")
controls <- fromJSON("transfer/controls.json")

stopifnot(nrow(df) == controls$row_count)

# Egész összeg: bitpontos egyezés
stopifnot(sum(df$basket_sum_minor) == controls$sum_basket_minor)

# Lebegőpontos átlag: numerikus tolerancia
tol <- function(a, b, atol = 1e-9, rtol = 1e-9) {
```

```
  abs(a - b) <= atol + rtol * max(abs(a), abs(b))
}
stopifnot(tol(mean(df$late_rate), controls$mean_late_rate))

# Kategóriaszintek ellenőrzése a séma alapján
seg_col <- schema$columns[[which(sapply(schema$columns, function(x) x$name == "segment"))]]
df <- df |>
  mutate(segment = factor(segment, levels = seg_col$levels))
stopifnot(identical(levels(df$segment), seg_col$levels))

# Időbélyeg kontroll: epoch mikroszekundum, nem reprezentációs string
epoch_us <- function(x) as.numeric(x) * 1e6
stopifnot(abs(epoch_us(min(df$last_order_ts)) - controls$min_last_order_ts_epoch_us) < 1)
stopifnot(abs(epoch_us(max(df$last_order_ts)) - controls$max_last_order_ts_epoch_us) < 1)

fit <- lm(late_rate ~ segment + log1p(basket_sum_minor), data = df)
summary(fit)
```

A modellösszegzés elméletileg várt, szemléltető részlete: a `segmentconsumer` együttható előjele a kis mintán nem értelmezhető okságként; a példa csak a gépi átadást ellenőrzi.

A Quarto-jelentés egy önálló `report.qmd` fájl: a kódblokk maga definiálja a `tol`, `parse_iso` és `epoch_us` segédfüggvényeket, ezért üres R-munkamenetből renderelve is lefut. A jelentés ugyanazokat a kontrollokat futtatja, mint az `analyze_transfer.R`:

````
---
title: "Nyelvek közötti átadás kontrollja"
format: html
---

```{r}
library(arrow)
library(dplyr)
library(jsonlite)

df <- read_parquet("transfer/customers.parquet")
schema <- fromJSON("transfer/schema.json")
controls <- fromJSON("transfer/controls.json")

tol <- function(a, b, atol = 1e-9, rtol = 1e-9) {
  abs(a - b) <= atol + rtol * max(abs(a), abs(b))
}
epoch_us <- function(x) as.numeric(x) * 1e6

stopifnot(nrow(df) == controls$row_count)
stopifnot(sum(df$basket_sum_minor) == controls$sum_basket_minor)
stopifnot(tol(mean(df$late_rate), controls$mean_late_rate))

seg_col <- schema$columns[[which(sapply(schema$columns, function(x) x$name == "segment"))]]
df <- df |>
  mutate(segment = factor(as.character(segment), levels = seg_col$levels))
stopifnot(identical(levels(df$segment), seg_col$levels))

stopifnot(abs(epoch_us(min(df$last_order_ts)) - controls$min_last_order_ts_epoch_us) < 1)
stopifnot(abs(epoch_us(max(df$last_order_ts)) - controls$max_last_order_ts_epoch_us) < 1)

fit <- lm(late_rate ~ segment + log1p(basket_sum_minor), data = df)
summary(fit)
cat("Ellenőrzött sorok száma:", nrow(df), "\n")
cat("Ellenőrzött átlag:", mean(df$late_rate), "\n")
```
````

A renderelés Linux/macOS terminálból:

```
quarto render report.qmd
```

Windows PowerShellben ugyanez. A parancs feltételezi, hogy a Quarto CLI telepítve van, és a projekt R kernelspecje a `renv` környezetre mutat.

37.3.4 Eredmény értelmezése

A példában a Python oldali `controls.json` és az R oldali ellenőrzés ugyanazokat a számokat adja. Az egész `sum_basket_minor = 81060` bitpontosan egyezik, mert `int64` összegről van szó. A `mean_late_rate` értéke Pythonban 0,16130950000000002, R-ben ugyanez a lebegőpontos ábrázolás miatt gyakorlatilag ugyanaz; a toleranciafüggvény a 10^{-9} relatív hibát is megengedi, tehát a kontroll akkor is zöld, ha az utolsó tizedesjegyekben platformfüggő eltérés van. Az időbélyeg-kontroll nem a megjelenített ISO-stringet, hanem az epoch mikroszekundumot hasonlítja össze, ezért locale- és formátumfüggetlen. A kategória-ellenőrzés az egyetlen olyan pont, ahol a Parquet önmagában nem elég: a `segment` oszlop R-ben lehet dictionary vagy karakter, ezért a `schema.json` `levels` listájából explicit faktort készítünk. A Quarto-jelentés azért megbízható, mert a renderelés közben újra lefuttatja a kontrollt, így a jelentés kimenete nem válhat el a tényleges adattól.

37.3.5 Feladatok

1. **Értelmezés.** Miért szerepel külön `timezone` mező a `schema.json`-ban akkor is, ha a Parquet fájl hordozza az időbélyeg típusát? Mi történik, ha egy CSV fallbackben a `last_order_ts` string `"2025-04-02T11:30:00+02:00"` és `"2025-04-02T11:30:00+00:00"` értékeket tartalmaz, és R-ben karakterként számolunk `max`-ot?

2. **Megvalósítás.** Írj Python függvényt `validate_schema(df, schema)` néven, amely ellenőrzi, hogy a DataFrame oszlopai megfelelnek a séma `type`, `nullable`, kategória `levels` és időbélyeg `timezone` mezőinek. A függvény hibás egyezés esetén `ValueError` kivételt dobjon, pontos üzenettel.

3. **Hibakeresés.** Az R oldali kódban a `setequal(levels(df$segment), controls$segment_levels)` hamisat ad, mert az `arrow::read_parquet` a `segment` oszlopot karakterként olvasta be, nem faktorként. Javítsd a kódot úgy, hogy a `schema.json` `levels` mezőjéből explicit faktort készíts, és magyarázd el, miért nem elég a Parquet beolvasásra hagyatkozni. Miért kell `identical(levels(...), expected_levels)`-et használni `setequal` helyett?

4. **Megváltozott feltétel.** Az adatgazda új `currency` oszlopot ad a táblához, amely minden sorban `"EUR"` string. Módosítsd a Python exportot úgy, hogy az új oszlop is bekerüljön a DataFrame-be, a `schema.json`-ba és a `controls.json`-ba. Az R ellenőrzést egészítsd ki `stopifnot(all(df$currency == controls$currency_value))` teszttel. Miért nem elég a Parquet fájlt lecserélni a `controls.json` frissítése nélkül?

5. **Továbbvitel.** Az adatkészletet egy olyan fogyasztó kapja meg, amely csak CSV-t tud olvasni, és a rendszer lokálja tizedesvesszőt használ. Miért okozhat hibát a `late_rate` oszlop CSV-írása, és hogyan lehet a `write.csv` vagy `readr::write_csv` paramétereivel a hibát elkerülni?

37.3.6 Megoldások és indoklás

1. megoldás. A Parquet fájl valóban hordozza a `timestamp[us, tz=UTC]` típust, de a sémafájl akkor is kötelező, mert a CSV fallback nem hordoz típust, és a fogyasztó oldalán a helyes `as.POSIXct` híváshoz explicite tudni kell az időzónát. A példában `"2025-04-02T11:30:00+02:00"` időben korábbi, mint `"2025-04-02T11:30:00+00:00"`, de lexikografikus string-összehasonlításban a `"+02:00"` nagyobb, mint a `"+00:00"`, tehát a `max()` karakterként rossz értéket ad. Az időbélyegeket mindig UTC-re konvertálva, numerikus/POSIXct típusként kell összehasonlítani.

2. megoldás. Egy lehetséges ellenőrző függvény:

```
import pandas as pd

def validate_schema(df: pd.DataFrame, schema: dict) -> None:
    cols = {c["name"]: c for c in schema["columns"]}
    if set(df.columns) != set(cols):
        raise ValueError(f"Oszlopok eltérnek: {set(df.columns) ^ set(cols)}")
    for name, spec in cols.items():
        s = df[name]
        if spec["nullable"] is False and s.isna().any():
            raise ValueError(f"{name}: nem várt hiányzó érték")
        if spec["type"] == "int64" and not pd.api.types.is_integer_dtype(s):
            raise ValueError(f"{name}: int64 várt, kapott {s.dtype}")
        if spec["type"] == "float64" and not pd.api.types.is_float_dtype(s):
            raise ValueError(f"{name}: float64 várt, kapott {s.dtype}")
        if spec["type"] == "category":
            if not isinstance(s.dtype, pd.CategoricalDtype):
                raise ValueError(f"{name}: category várt, kapott {s.dtype}")
            actual = list(s.cat.categories)
            expected = list(spec["levels"])
            if actual != expected:
                raise ValueError(f"{name}: szintek {actual} != {expected}")
        if spec["type"].startswith("timestamp") and spec.get("timezone") == "UTC":
            if not pd.api.types.is_datetime64_any_dtype(s) or s.dt.tz is None:
                raise ValueError(f"{name}: időzónás timestamp várt")
            # UTC-ekvivalens zóna: télen és nyáron is nulla az eltolás; a DST-s
            # zónák (pl. Europe/London) így kiszűrhetők.
            for probe in ("2000-01-01", "2000-07-01"):
                if pd.Timestamp(probe, tz=s.dt.tz).utcoffset() != pd.Timedelta(0):
                    raise ValueError(f"{name}: UTC időzóna várt, kapott {s.dt.tz}")
```

Az ellenőrzés a `not s.isna().any()` feltétellel a `nullable=False` szerződést tartatja be.

3. megoldás. A javított R kód:

```
seg_spec <- schema$columns[[which(sapply(schema$columns, function(x) x$name == "segment"))]]
df <- df |>
  mutate(segment = factor(as.character(segment), levels = seg_spec$levels))
stopifnot(identical(levels(df$segment), seg_spec$levels))
```

A Parquet beolvasás önmagában nem garantálja a `factor` osztályt, mert a dictionary típus R-ben lehet `vctrs`- vagy `arrow`-specifikus, és a `levels` halmaza sem feltétlenül egyezik a séma szerinti rendezéssel. A `factor(..., levels = ...)` hívás explicitté teszi a szerződést, így az ellenőrzés nem a beolvasó implementációjától függ.

4. megoldás. A Python oldalon a DataFrame-hez adjuk hozzá:

```
df["currency"] = "EUR"
```

A `schema.json` columns listájához:

```
{"name": "currency", "type": "string", "unit": "ISO 4217 code", "nullable": false}
```

A `controls.json`-hoz:

```
"currency_value": "EUR",
```

Az R oldal:

```
stopifnot(all(df$currency == controls$currency_value))
```

A `controls.json` frissítése azért kötelező, mert az R ellenőrzés a szerződésfájlhoz hasonlít. Ha csak a Parquet-ot cseréljük, a régi `controls.json` nem tartalmazza az új mezőt, ezért az `all(df$currency == controls$currency_value)` `NULL`-t adhat, vagy az egész pipeline elhallgatva hamisan zöld marad. A szerződésfájlok és az adat mindig együtt, egy verziózott kötegként mozogjanak.

5. megoldás. Ha a rendszer lokálja tizedesvesszőt használ, a `late_rate` oszlop értékei `"0,142857"` alakban kerülhetnek a CSV-be. Az R `read_csv` alapértelmezésben pontot vár tizedesjegy-elválasztónak, ezért az ilyen sorokat `NA`-nak vagy karakternek olvassa. A Python oldalon `df.to_csv(..., decimal=".")` explicit módon pontot kényszerít, vagy az R oldalon `readr::read_csv(..., locale = locale(decimal_mark = ","))` állítható. A sémafájlban érdemes rögzíteni a CSV-elválasztó és tizedesjel konvenciót is, mert az átadás így válik lokálfüggetlenné.

37.3.7 Gyakori tévedés

A leggyakoribb hiba az, hogy a Parquet sémáját azonosnak tekintjük a szemantikai szerződéssel. Példa: a pandas `category` oszlop Parquet-ben dictionary típusú lesz, az R `arrow` ezt karakterként vagy dictionary-ként olvashatja. Ha az R kód `factor(df$segment)` hívást használ, a szintek ABC-sorrendbe kerülnek, ami eltérhet a Python oldali `levels` sorrendtől; a `lm` kontrasztkódolás ekkor más referencia-kategóriát választhat, és az együttható-értelmezés félrevezetővé válik. Hasonlóan, a CSV fallbacknél az időzóna elvesztése nem okoz azonnali hibát az átlagoknál, de a `max(last_order_ts)` karakteres összehasonlítással hamis eredményt ad. A megoldás mindig ugyanaz: explicit schema, explicit faktor/type visszaállítás és kontrollfutás, nem pedig a beolvasófüggvény alapértelmezésére hagyatkozás. Ez a fegyelem rokon a 25.3. alfejezet ott tanult elvével: a gépi kényelem nem helyettesíti a szerződéses ellenőrzést.

37.3.8 Rövid összefoglaló

A nyelvek közötti adatátadás akkor megbízható, ha a Parquet vagy CSV fájl mellett egy explicit `schema.json` és egy `controls.json` is utazik. A séma rögzíti a típust, mértékegységet, kategóriaszinteket és időzónát; a kontrollok a sorok számát, egész összegeket, lebegőpontos átlagokat és időintervallum-határokat tartalmazzák. A lebegőpontos értékekre az abszolút-relatív tolerancia képlet érvényes, az egész összegekre bitpontos egyezés. R-ben a kategóriákat a séma `levels` mezőjéből explicit faktorként állítjuk helyre, mert a Parquet dictionary beolvasás nem garantálja ezt. A Quarto-jelentés a rendereléskor újra lefuttatja a kontrollokat, így a jelentés nem válhat el az ellenőrzött adattól. Következő alkalmazásként a 37.4. alfejezet azt vizsgálja meg, mikor érdemes egyáltalán másik ökoszisztémát választani.

37.4 Mikor érdemes másik ökoszisztémát választani?

Belépő · [BŐVÍTŐ MODUL] · szint: haladó · idő: 90 perc · eszköz: R, renv

Előfeltétel: 37.3. alfejezet · Célkészség: Mikor érdemes másik ökoszisztémát választani? · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Döntési próba: az eszközváltás indoka ne pusztán a csapat preferenciája legyen. Ugyanazon adatszeletre, azonos számítási definícióval mérd a helyességet, futásidőt, memóriát, kiépítési költséget és a hosszú távú fenntarthatóságot.

A 37.3. alfejezet már megmutatta, hogyan lehet adatot átadni Python és R között Parquet, Arrow és Quarto segítségével. A technikai híd megléte azonban nem válasz arra a döntési kérdésre, hogy *mikor* érdemes egy másik ökoszisztémát bevonni. Ebben az alfejezetben három csapathelyzeten keresztül olyan döntési jegyzetet készítünk, amely a feladathoz, a csapathoz és az üzemeltetési életciklushoz kötött, nem pedig „modernségi” vagy kényelmi alapon választ Python, R vagy kevert megoldás között. A fő tanulási út Python-alapú; az R-re váltás vagy a kétnyelvű környezet ezért mindig többletköltséget indokol, amit explicit módon kell vállalni.

37.4.1 A döntési keret hat dimenziója

A nyelvválasztás nem egyetlen szempont, hanem hat, egymással összefüggő dimenzió együttes értékelése. Mindegyik dimenzióhoz egy konkrét kérdést és egy ellenőrizhető jelet rendelünk.

Dimenzió (magyar / angol)	Központi kérdés	Gyenge jel	Erős jel
Elérhető módszer (available method)	Az adott statisztikai vagy gépi tanulási módszer érett, karbantartott interfésszel létezik-e az adott nyelvben?	A kívánt modellhez saját wrapper kell, vagy a csomag ritkán frissül.	A kívánt becslési eljárás dokumentált, verzióhoz kötött csomagban elérhető, és a csapat már használta.
Szakértelem (expertise)	Ki tudja megírni, futtatni és értelmezni a kódot?	Csak egyetlen személy ért a nyelvhez, és ő sem review-képes.	A csapatban legalább két ember önállóan tud hibát javítani és magyarázni.
Integráció (integration)	Hogyan illeszkedik a meglévő batch, API, scheduler és adatplatform szerződéseihez?	A nyelv külön interpretert, külön ütemezőt vagy kézi fájlmozgatást igényel.	A futtatókörnyezet, a naplózás és az ütemező már kezeli az adott nyelvet.
Review	Ki a második szem, aki a kódot és a statisztikai döntést is ellenőrzi?	Nincs olyan kolléga, aki a formula szintaxist és a modellkimenetet is érti.	A kód és a statisztikai feltételek is szabályozott, kétlépcsős ellenőrzésen mennek át.

Dimenzió (magyar / angol)	Központi kérdés	Gyenge jel	Erős jel
Karbantartás (maintenance)	Hogyan kezeljük a függőségek frissítését, a lockfájlt, a CI-t és a csomagéletciklust?	A függőségek nincsenek rögzítve, a frissítés kézi, és nincs reprodukálható telepítés.	A renv.lock vagy uv.lock verziózott, a CI új környezetben újraépíti a környezetet.
Interoperabilitás (interoperability)	Milyen séma, formátum és kontrollérték garantálja a nyelvek közötti adatátadást?	A két nyelv között CSV van oszloptípus-ellenőrzés nélkül.	A közös csereformátum Parquet vagy Arrow, explicit sémaszerződéssel és kontrollösszegekkel.

Ezek a dimenziók a 25.4. fejezet feature store döntéséhez hasonlóan mérnöki szempontok: a konzisztencia, a késés és a csapatméret határozza meg, hogy egy megoldás fenntartható-e. A statisztikai módszer érettsége szükséges, de nem elégséges feltétel; a review és a maintenance hiánya akkor is megbuktathatja a választást, ha a módszer matematikailag tökéletes.

37.4.2 Kézi kontroll: súlyozott döntési lap

A hat dimenziót egy 0–2 skálán pontozzuk: 0 = nem teljesül vagy jelentős akadály, 1 = részben teljesül, 2 = erős teljesítés. A dimenziók súlyát a csapat saját céljaihoz igazítjuk; a példában a szakértelem és az integráció nagyobb súlyt kap, mert ezek hiánya napokban mérhető késést okoz.

A súlyozott pontszám képlete egyszerű:

$$\text{score} = \sum_{d} w_d \cdot s_d,$$

ahol w_d a d dimenzió súlya (pozitív egész), s_d pedig a 0–2 pontszám. A képlet nem statisztikai becslés, hanem döntéselőkészítő kommunikációs eszköz; a pontszám magas értéke nem pótolja a kényszerfeltételek ellenőrzését.

Egy kis R kód a pontszám összegzésére, **szemléltető** értékekkel; a számok a csapat kézi szakértői becslései, nem mérések:

```
# Döntési lap pontszámösszegzés, szemléltető értékekkel.
weights <- c(method=2, expertise=3, integration=3,
             review=2, maintenance=2, interoperability=2)

team_A <- c(method=1, expertise=2, integration=2,
            review=2, maintenance=2, interoperability=1)
team_B <- c(method=2, expertise=1, integration=0,
            review=1, maintenance=1, interoperability=1)
team_C <- c(method=2, expertise=2, integration=2,
            review=2, maintenance=1, interoperability=2)

scores <- c(A=sum(weights*team_A),
            B=sum(weights*team_B),
            C=sum(weights*team_C))
print(scores)
```

A kód kimenete a fenti szemléltető számokkal: `A=24`, `B=13`, `C=26`. Ezt még nem ellenőriztük automatizáltan; a számok ellenőrzéséhez elegendő a szorzás és az összegzés kézi elvégzése. A lényeg nem a konkrét szám, hanem az, hogy a döntési jegyzetben láthatóvá válik, melyik dimenzió húzza le az eredményt.

37.4.3 Teljes minimális példa: három csapathelyzet döntési jegyzete

Az alábbi három, egymástól eltérő csapathelyzet bemutatja, hogyan kell a keretet konkrét feladatra alkalmazni.

37.4.3.1 A csapat: Batch Product

- **Feladat:** kockázati scoring batch üzemmódban, 24 órás SLA, auditkötelezettség.
- **Megfigyelési egység:** egy ügyfélrekord; a predikció a rendelés pillanatában ismert feature-ökből készül.
- **Jelenlegi stack:** Python 3.12, scikit-learn Pipeline, Airflow, Parquet snapshot, uv lock, CI.
- **Statisztikai igény:** logisztikus regresszió, kalibráció, PSI-drift figyelés. Ezek a Python ökoszisztémában is elérhetők, és a csapat már használta őket.

Döntési jegyzet: Python marad. A módszerdimenziónál 1 pont, mert a szükséges modellcsalád Pythonban is érett, de néhány speciális diagnosztika R-ben kényelmesebb. A szakértelem, integráció, review és maintenance erős: a csapat 5 emberből 4 ismeri a stacket, a scheduler natívan futtatja, a lock verziózott. Az R bevezetése csak a módszer-kényelemért nem indokolt; a karbantartási és review költség meghaladná a hasznot. **Kimenet:** a döntési lap rögzíti, hogy a modellkísérlet Pythonban történik; ha később olyan módszer kell, amelyhez nincs Python-megfelelő, a döntést újra kell nyitni.

37.4.3.2 B csapat: Evidence Lab

- **Feladat:** randomizált kísérlet elemzése, robusztus konfidenciaintervallum, szabályozói jelentés Quarto riportban.
- **Megfigyelési egység:** egy randomizált ügyfél (assignment), a primer végpont conversion_7d.
- **Jelenlegi stack:** R, renv, tidyverse, Quarto; a csapat 3 statisztikusból áll, mindannyian R-ben gondolkodnak.
- **Integrációs igény:** minimális; az output egy jelentés és egy becslési táblázat, amit emberi review olvas.

Döntési jegyzet: R marad. A módszerdimenziónál 2 pont: a csapat által használt becslési és diagnosztikai eljárások R-ben dokumentált interfésszel és a csapat által ismert csomagokban érhetők el. A szakértelem és a review erős az R oldalon. Az integráció 0 pontot kap, mert a csapat nem üzemeltet Python batch rendszert; ez azonban nem számít, mert a feladatnak nincs ilyen igénye. A döntési lap fontos eleme, hogy a súlyokat a B csapat céljaihoz igazítottuk: az integráció súlyát 1-re csökkentettük, a módszerét 3-ra emeltük. A pontszám így nem hasonlítható mechanikusan az A csapat pontszámához.

37.4.3.3 C csapat: Mixed Core

- **Feladat:** vegyes igény: a statisztikai modellezést R-ben végzik (a csapat egyik fele R-ben erős), de a végleges modellnek Python batch scoringban kell futnia, Parquet feature-ökből.
- **Megfigyelési egység:** egy rendelés; a feature-ök a rendelés pillanatában ismertek.
- **Stack:** Python a production oldalon, R a kutatásban; a csapat 8 fő, a két nyelv között a híd Parquet.

Döntési jegyzet: többnyelvű megoldás, szigorú határfelülettel. A módszer és a szakértelem indokolja az R-t a kutatási fázisban; az integráció és a production fenntartás a Python oldalon van. A két oldal közötti átadás Parquet sémaszerződéssel és kontrollösszegekkel történik, ahogyan azt 37.3. alfejezet bemutatta. A karbantartás dimenziója 1 pont: a csapatnak két lockfájlt (renv.lock és uv.lock) kell verzióznia, két környezetet kell frissítenie, és a CI-nek mindkettőt újra kell építenie. Ezt a többletköltséget explicit módon vállalják, mert a kutatási sebesség és a production megbízhatóság együttesen igazolja. **Kimenet:** a döntési jegyzet rögzíti a kilépési feltételt: ha a review teher miatt a production modell frissítése rendszeresen késik, a csapat a becslési kódot Pythonba portolja.

A három jegyzet közös szerkezete: feladat, megfigyelési egység, jelenlegi stack, a hat dimenzió pontozása, döntés, és kilépési feltétel. A pontszámok nem mérések, hanem a csapat saját becslései; a jegyzet akkor használható, ha ezt a szubjektivitást nyíltan jelöli.

37.4.3.4 Szervezeti tudás mint költség, küszöbök és érzékenység

A pontszám önmagában nem döntés. Az R és a Python közötti választás nem benchmarkverseny: ha nincs, aki a célnyelven a kódot *és* a statisztikai döntést is review-zni tudja, akkor a gyorsabb futás vagy a kényelmesebb csomag nem kompenzálja a szállíthatósági kockázatot. Ezért a döntési lapon a szervezeti tudás hiányát tételes költségként és küszöbként kell rögzíteni.

Tudásköltség-tétel	Mérőszám	Szemléltető érték (R bevezetése 3 fős Python-csapatnál)
Betanulás	fő × óra × óradíj	3 × 40 h = 120 h; 1 440 000 Ft
Review-kapacitás építése	riport × review-óra × óradíj	12 × 8 h = 96 h; 1 152 000 Ft
Első évi tudásköltség	a kettő összege	216 h; 2 592 000 Ft
Review-képes személyek	fő	1 (buszfaktor 1)
Audit- és hatósági tapasztalat	van/nincs, dokumentált referencia	nincs → kockázati tartalék a lapon

A példában az éves üzemeltetési költség 4 000 000 Ft, a tudásköltség aránya $2\,592\,000/4\,000\,000 = 0.648$, azaz 64.8%. Ez meghaladja a 25%-os küszöböt, ezért a döntés csak független másodvéleménnyel és mért módszerelőnnyel fogadható el. A számok szemléltetők, nem mérések; a lapon a becslés forrását (ki becsülte, milyen korábbi onboarding alapján) is rögzíteni kell.

A küszöbök nem pontszámok, hanem a döntés érvényességi feltételei:

Feltétel	Határérték	Következmény
Review-képes szakértő a célnyelven	0 fő	hard veto: a döntés nem hozható meg

Feltétel	Határérték	Következmény
Bus factor	1	írásos tudásátadási terv és második review-képes személy kijelölése a bevezetés feltétele
Tudásköltség / éves üzemeltetési költség	$> 25\%$	független másodvélemény és mért módszerelőny (kontrollpéldány)
Kritikus dimenzió pontszáma (review, integration)	0	a magas összpontszám nem menti: a döntés blokkolt

Szakértői review követelmény. A döntési lap csak két aláírással érvényes: (1) a célnyelven kódot és statisztikai döntést review-zni képes szakértő, (2) az auditált vagy szabályozott folyamatért felelős személy. Az aláírás mellett szerepel a pontszámok forrása és a becslés bizonytalansága; aláírás nélkül a lap legfeljebb javaslat, nem döntés.

Az ajánlás érzékenysége számszerűsíthető. A három vezető csapatnál a C−A különbség csak a három eltérő dimenzióból áll össze: $w_{\text{method}} + w_{\text{interoperability}} - w_{\text{maintenance}} = 2 + 2 - 2 = 2$. Egyetlen súly ± 1 változása ezt legfeljebb 1-gyel mozdítja, tehát a sorrend nem fordul meg:

Változtatás	C−A különbség	Ajánlás
alap (2, 3, 3, 2, 2, 2)	+2	C (kevert)
bármelyik súly ± 1	+1 vagy +3	C
maintenance súly 2→4	0	döntetlen → szakértői review dönt
maintenance súly 2→5	−1	A (Python marad)

A forduláshoz a maintenance súlyát legalább 3-mal kell növelni, vagy a C maintenance pontszámát 1-ről 0-ra csökkenteni. A pontszámok oldaláról nézve viszont már egyetlen pont átírása is döntetlent vagy fordulatot okoz:

Egypontos változtatás	Új pontszám	C−A	Eredmény
C expertise 2→1	C = 23	−1	A
C integration 2→1	C = 23	−1	A
C method 2→1	C = 24	0	döntetlen
C maintenance 1→0	C = 24	0	döntetlen
C review 2→1	C = 24	0	döntetlen
C interoperability 2→1	C = 24	0	döntetlen
A method 1→2	A = 26	0	döntetlen

Ezért a döntési lapon minden ilyen pontszámhoz bizonyíték kell (ki, mikor, milyen mintán ítélte), a döntetlent pedig nem szabad önkényesen feloldani: szakértői review dönt, dokumentált indoklással. A B csapat 13 pontja egyetlen ± 1 súlyváltozástól sem kerül versenybe; az ő esetében a döntés nem pontszám, hanem a hiányzó integráció és review miatti kényszerű választás.

```
DIMENSIONS = ("method", "expertise", "integration", "review",
              "maintenance", "interoperability")
weights = dict(method=2, expertise=3, integration=3,
               review=2, maintenance=2, interoperability=2)
teams = {
    "A": dict(method=1, expertise=2, integration=2,
              review=2, maintenance=2, interoperability=1),
```

```
    "B": dict(method=2, expertise=1, integration=0,
              review=1, maintenance=1, interoperability=1),
    "C": dict(method=2, expertise=2, integration=2,
              review=2, maintenance=1, interoperability=2),
}

def score(points, w=None):
    w = w or weights
    return sum(w[d] * points[d] for d in DIMENSIONS)

def margin(a, c, w=None):
    w = w or weights
    return sum(w[d] * (c[d] - a[d]) for d in DIMENSIONS)

def knowledge_cost(trainees, training_hours, rate, reports, review_hours):
    hours = trainees * training_hours + reports * review_hours
    return hours, hours * rate

print({name: score(p) for name, p in teams.items()})   # A=24, B=13, C=26
print(margin(teams["A"], teams["C"]))                  # 2
hours, cost = knowledge_cost(3, 40, 12000, 12, 8)
print(hours, cost)                                     # 216 2592000
for dim in DIMENSIONS:
    w = dict(weights)
    w[dim] += 1
    print(dim, margin(teams["A"], teams["C"], w))
```

A kimenet megerősíti a fenti táblákat: az egyes dimenziók súlyának +1 növelése a C−A különbséget sorrendben 3, 2, 2, 2, 1, 3-ra változtatja, tehát egyik sem fordítja meg az ajánlást; a tudásköltség pedig 216 óra és 2 592 000 Ft. A lab/ecosystem_decision.py ugyanezeket a függvényeket adja (`ranking`, `weight_sensitivity`, `point_sensitivity`, `knowledge_cost`, `gates`), így a döntési lap számításai tesztelhetők.

37.4.4 Eredmény értelmezése

A döntési lap nem ad automatikus választ. A B csapat alacsony pontszáma a fenti általános súlyokkal nem jelenti azt, hogy a B csapat rosszul választott; a súlyokat az adott csapat céljaihoz kell igazítani, és a kényszerfeltételek – különösen a review és a maintenance – felülírhatják a pontszámot. Például ha az A csapatban nincs senki, aki az R kódot review-zni tudná, akkor egyetlen magas módszerpont sem indokolja az R bevezetését.

Az önellenőrzéshez három kérdést érdemes feltenni:
1. A döntés konkrét feladathoz, csapathoz és üzemeltetési életciklushoz kötött-e?
2. A kilépési feltétel megnevezi-e, mikor kell újraértékelni a választást?
3. A határfelületen mindkét nyelv rendelkezik-e séma- és kontrollérték-ellenőrzéssel?

Ha bármelyikre nemleges a válasz, a döntési jegyzet hiányos.

37.4.5 Feladatok

37.4.5.1 1. feladat – Értelmezés

Egy kolléga azt állítja: „R-ben több statisztikai módszer van, ezért minden kísérleti elemzést R-ben kell csinálni.” Magyarázd el három-négy mondatban, miért hibás ez az érv, és melyik további dimenziókat hagyja figyelmen kívül.

37.4.5.2 2. feladat – Implementáció

Készíts döntési jegyzetet egy negyedik, D csapatra: 6 fős Python-csapat, erős MLOps stack, a feladat túlélési analízis (survival analysis) batch riporttal. A csapat senki sem ismeri az R survival interfészeit, de hallotta, hogy R-ben kényelmesebb. Válaszd meg a súlyokat, pontozd a hat dimenziót, és írd le a következtetést, beleértve a kilépési feltételt.

37.4.5.3 3. feladat – Hibadiagnózis

Egy kétnyelvű projektben R végzi a statisztikai becslést, Python a scoringot. A bridge CSV, nincs sémafájl, a renv.lock nem commitolt, és a review-t végző személy csak Python-t ismer. Nevezd meg a három konkrét hibát, és mindegyikhez adj egy javítási lépést.

37.4.5.4 4. feladat – Megváltozott feltétel

A B csapat (Evidence Lab) mostantól 50 ms-os online predikciót is szeretne a becsült modellből. Hogyan módosul a döntési jegyzet? Javasolj rétegzést a statisztikai becslés és a scoring között, és indokold, miért nem célszerű az R interpretert a kiszolgálási útvonalba tenni.

37.4.6 Megoldások és indoklás

37.4.6.1 1. feladat megoldása

Az érvelés egyetlen dimenzióra, az elérhető módszerre épít. A módszerkínálat szükséges feltétel lehet, de nem elégséges. Ha a csapat nem ismeri az R-t (expertise hiánya), nincs, aki a kódot és a statisztikai értelmezést review-zná (review hiánya), és a meglévő Python integrációval nem kompatibilis a környezet (integration és maintenance), akkor a magas módszerpont ellenére rossz döntés születik. A nyelvválasztás akkor helyes, ha mind a hat dimenzióban elfogadható a jel.

37.4.6.2 2. feladat megoldása

Egy lehetséges D csapat jegyzet:
- **Feladat:** túlélési görbe és kockázati tényezők becslése; batch riport, nincs valós idejű igény.
- **Megfigyelési egység:** egy ügyfél a szerződés kezdetétől az eseményig vagy cenzorálásig.
- **Jelenlegi stack:** Python, scikit-learn, Airflow, Parquet, uv lock, 6 fő.
- **Súlyok:** method=3, expertise=2, integration=2, review=2, maintenance=2, interoperability=1, mert a módszerérettség és a csapat tanulási költsége a fő kockázat.
- **Pontozás:** method=2 (Pythonban is van megfelelő interfész, de a dokumentációt verzióhoz kötve ellenőrizni kell), expertise=0 (senki sem ismeri az R survival rendszerét), integration=2 (a reporting Pythonban fut), review=0 (nincs R-review), maintenance=1 (egy új lockot kell kezelni), interoperability=1 (a riport inputja Parquet lehet).
- **Döntés:** Pythonban maradni, és a túlélési modellt a Python-ökoszisztéma dokumentált csomagjával illeszteni; a csapat tanulási ideje kisebb, mint az R bevezetésének teljes karbantartási költsége.
- **Kilépési feltétel:** ha a Python-megvalósítás numerikus diagnosztikája nem reprodukálja az ellenőrző példát, és a csapat kétszer egymástól függetlenül megerősíti, hogy a kívánt becslés Pythonban nem elérhető, akkor újra kell értékelni az R-t.

Ez a jegyzet a súlyokat a csapat céljaihoz igazítja, és a kilépési feltétel konkrét.

37.4.6.3 3. feladat megoldása

A három hiba:
1. **Nincs séma a bridge-en:** a CSV-ben elveszhetnek a dtype-ok, kategóriák és időzónák. Javítás: Parquet vagy Arrow közös sémafájl, ahogyan 37.3. alfejezet bemutatta; a beolvasás után mindkét oldalon shape- és dtype-ellenőrzés.
2. **A renv.lock nem commitolt:** a környezet nem reprodukálható, egy új gépen más csomagverziók futhatnak. Javítás: a renv.lock a verziózott fájlok közé kerül, a CI a lockból telepít.
3. **A review nem ismer R-t:** a formula szintaxis vagy a modellkimenet értelmezési hibája átcsúszhat. Javítás: a review-t olyan személy végzi, aki az R statisztikai interfészt is ismeri, vagy a kód annyira leegyszerűsített, hogy a statisztikai feltételeket külön, nyelvfüggetlen jegyzőkönyvben rögzítik.

37.4.6.4 4. feladat megoldása

Az online predikció bevezetése megváltoztatja az integráció súlyát: a kiszolgálási útvonalnak alacsony késleltetésű, monitorozott és visszagörgethető rétegnek kell lennie. Javasolt rétegzés:
- **Kutatási és becslési réteg:** R-ben marad, a becsült paramétereket (például együtthatók, kalibrációs konstansok) egy verziózott Parquet fájlba exportálják.
- **Scoring réteg:** Python vagy más alacsony késleltetésű szolgáltatás, amely csak a paramétereket olvassa be, és a feature-ökön kiszámolja a predikciót.

Az R interpretert nem célszerű a kiszolgálási útvonalba tenni, mert a session-indítás, a csomagbetöltés és a formula-értelmezés késleltetése nehezen monitorozható, és a 36.4. alfejezet által megkövetelt rollback, driftfigyelés és access-kontroll szempontjából többletkockázatot jelent. A döntési jegyzetben az interoperabilitás súlya nő, és a kilépési feltétel kibővül: ha a paraméterexport séma nem reprodukálja az R-beli predikciót egy rögzített tolerancián belül, a kiadás nem engedélyezhető.

37.4.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés az, hogy a többnyelvű környezet automatikusan jobb, mert „mindenkinek megvan a kedvenc nyelve”. Ennek az ellenkezője is tévedés: hogy a több nyelv mindig felesleges luxus. A valóság az, hogy a több nyelv több karbantartási és reprodukciós kötelezettséggel járhat, de keskeny, explicit határfelülettel megéri, ha a módszerérettség vagy a szakértelem valóban különbözik.

Ellenpélda: egy csapat bevezette az R-t egyetlen elemzéshez, mert a statisztikus jobban szerette, de a renv.lock nem került verziózásra, a bridge CSV maradt, és a Python oldal nem futtatta a kontrollértékeket. Hat hónappal később a jelentés nem reprodukálható, és senki sem tudja, melyik csomagverzió hozta az eredményt. Ezzel szemben a fenti C csapat kétnyelvű megoldása azért működik, mert a határfelület Parquet, a lockok verziózottak, és a döntési jegyzet explicit kilépési feltételt tartalmaz.

37.4.8 Rövid összefoglaló

A nyelvválasztás nem egyszempontú módszerkeresés, hanem hat dimenzió – available method, expertise, integration, review, maintenance, interoperability – együttes értékelése. A döntési jegyzet akkor használható, ha konkrét feladathoz, csapathoz és üzemeltetési életciklushoz kötött, a súlyokat a csapat céljaihoz igazítja, és kilépési feltételt ad. A többnyelvű megoldás keskeny,

séma- és kontrollérték-ellenőrzött határfelülettel indokolható, de minden plusz nyelv plusz lockot, review-t és CI-felelősséget jelent. A következő lépésben ezt a döntési fegyelmet a munkahelyi eszköz- és futtatókörnyezet-választásra alkalmazzuk a 38.1. alfejezetben.

38 Munkahelyi eszköz- és futtatókörnyezet-választás

38.1 Helyi gép, távoli CPU és managed notebook

Belépő · szint: haladó · idő: 90 perc · eszköz: nincs új eszköz

Előfeltétel: 37.4. alfejezet · Célkészség: Helyi gép, távoli CPU és managed notebook · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Adatkezelési határ: felhős vagy menedzselt notebook kiválasztása előtt ellenőrizd az adatok területi elhelyezését, a hozzáféréseket, a futtatási naplók megőrzését és a költségkorlátot; a távoli erőforrás használata önmagában nem teszi reprodukálhatóvá a munkát.

38.1.1 Cél és konkrét helyzet

A feladat ugyanaz a két Northstar-munkamenet, de eltérő erőforrásigénnyel. A kimenet nem két modell, hanem egy **futtatásihely-térkép**: honnan indul a kód, hol lakik az adat, hol fut a processz, és mi az ellenőrzés, amely igazolja, hogy a megfelelő gépen a megfelelő környezet dolgozott.

A) Fejlesztői kísérlet. Bemenet a lab `orders.csv` (2400 rendelés) és `order_items.csv` (4796 tétel), együttesen néhány megabájt. Megfigyelési egység a rendelés, illetve a belőle épített feature-sor. Cél: a 21.4. alfejezetben tanult predikciós szerződéssel logisztikus baseline és hibatábla. Induló becslés: a dekompresszált DataFrame 2400 × 16 módon is százezer cella alatt marad, a RAM-igény 100 MB alatt van; bármelyik modern laptop elég.

B) Nagy batch előállítás. Ugyanaz a séma, de 8 000 000 rendelés és 16 000 000 tétel, 20 GiB tömörített Parquet-ban. Cél: a teljes batch predikciója 2 órán belül, a `predict_proba` oszlop épsége mellett. Induló becslés a RAM-ra: 8 numerikus oszlop dominál, a dekompresszált tábla 4-5 GB; a feltöltési idő viszont veszélyes, ezért előbb számoljunk, mielőtt bármit mozgatunk.

38.1.2 Három futtatási hely és a fizikai határ

A **laptop** a tanulási főút gépe. Előnye, hogy a teljes környezet a mi felügyeletünk alatt áll, és a kis Northstar-adat hálózat nélkül is feldolgozható. Határa a RAM, a CPU-magok száma és a feltöltési sávszélesség.

A **remote VM** (távoli virtuális gép, SSH-val elérhető CPU-s szerver) ugyanazt a `uv + Jupyter` munkamenetet futtatja, mint a laptop, de az adat közelében. Ha a Parquet fájl már az adott régió object storage-ában (objektumtárolójában) van, a processz odaköltözik, és csak a kód utazik.

A **managed notebook** (menedzselt notebook: például AWS SageMaker, GCP Vertex AI Workbench, Azure ML vagy Databricks kategóriájába eső szolgáltatás) a notebook-szerver, a kernel és a storage üzemeltetését csomagban adja. Cserébe a konkrét menüút, ár és korlát szolgáltatónként

változik; ezek aktuális részletét a sources.md kulcsaiból kiindulva, dátumozott dokumentációval kell ellenőrizni. Itt a kategóriaszintű döntési keretet tanuljuk, nem konzolkattintásokat.

A döntés magja **a kód és az adat eltérő mozgatási költsége**. A hálózati átviteli idő közelítő képlete:

$$t_{\text{move}} = \frac{s_{\text{bytes}}}{b_{\text{eff}}},$$

ahol s_{bytes} a mozgatott bájtok száma (B), b_{eff} a két végpont közötti ténylegesen elérhető átviteli sebesség (B/s), és t_{move} a becsült idő (s). Két gép között a lassabb irány számít: otthoni feltöltésnél jellemzően az upload, a managed notebook és a VM belső hálózatán a storage felőli letöltés.

Kézi kontroll a 20 GiB feltöltésére, $b_{\text{eff}} = 10$ MB/s mellett:

$$20 \cdot 2^{30}\ \text{B} = 21\,474\,836\,480\ \text{B}, \qquad \frac{21\,474\,836\,480}{10^7} \approx 2147\ \text{s} \approx 35{,}8\ \text{perc}.$$

Ugyanennyi idő alatt 10 MB forráskód és lock fájl 0,1 másodperc alatt másolható. Ezért mondjuk: **a kódot költöztesd az adathoz, ne az adatot a kódhoz**.

A másik fizikai határ a RAM. A 34.1. alfejezetben tanult szabály itt is érvényes: a tömörített Parquet mérete nem memóriaigény. A becslés oszloponként:

$$R_{\text{mem}} \approx \sum_{c} n_{\text{rows}} \cdot w_c,$$

ahol n_{rows} a sorok száma (darab), w_c a c oszlop reprezentációs szélessége bájt/sor egységben, R_{mem} pedig a dekompresszált reprezentáció becsült bájtmennyisége. A 8 millió soros batchre: tíz `float64` oszlop $10 \cdot 8 \cdot 8\,000\,000 = 640$ MB, két `int64` oszlop 128 MB, három string/kategória oszlop átlagosan 120 B/sorral 2,88 GB; ez nyersen 3,65 GB, pandas overheaddel és a transzformációk köztes másolataival 4,5-5,5 GB. Egy 8 GB-os managed notebook épp hogy elférhet, de a szűkösség hibakeresést nehezít; a 16-32 GB-os VM a biztonságosabb választás.

Három további réteg egészíti ki a döntést, ahogy a 37.1. alfejezetben az R-környezetnél is rétegekre bontottunk:

- **Image.** A konténerkép vagy a managed image a telepített operációs rendszer, Python-verzió és natív függőségek hordozható leírása. A 32.4. alfejezet átadási láncában ez a réteg volt a másik gépen való ismételhetőség alapja.
- **Storage.** Az adat fizikai helye. A batch-feladatnál ez object storage; a VM vagy managed notebook innen olvas. A projekt artifactumai (split, modell, manifest) a futás után ugyanoda, verziózva kerülnek vissza.
- **Credentials.** Hozzáférési adatok, amelyek rövid életűek és a legkisebb jogosultságot kapják. A 36.4. alfejezet access-szabálya itt is él: környezeti titok vagy kulcstároló, soha nem notebook-output.

38.1.3 Teljes minimális példa: két térkép, kézi ellenőrzéssel

Az alábbi munkamenet **nem futtatott kimenet**, a parancsok szintaktikailag jelölt, szemléltető lépések. A munkakönyvtár minden blokk előtt adott.

A feladat — helyi laptop. A lab gyökere `~/projects/northstar`, Linux/macOS bash. A környezet feloldása és az ellenőrzés:

```
# ~/projects/northstar, Linux/macOS bash
uv sync --locked
uv run python quickcheck.py
```

A `--locked` kapcsoló a `uv.lock` és a `pyproject.toml` egyezését őrzi; a `quickcheck.py` a mellékelt adatszerződést ellenőrzi: `orders.csv` 2400 sor, `order_items.csv` 4796 sor, a manifest hash-értékei egyeznek. A processz fizikai helyét a következő Python-ellenőrzés rögzíti:

```
import getpass, platform, socket, sys
print({
    "host": socket.gethostname(),
    "user": getpass.getuser(),
    "platform": platform.platform(),
    "python": sys.executable,
})
```

Elméletileg várt, szemléltető kimenet: `{"host": "dev-laptop", "user": "alma", "platform": "macOS-14.5-arm64", "python": "/Users/alma/projects/northstar/.venv/bin/python"}`. A `python` útvonal mutatja, hogy a kernel tényleg a projekt `.venv`-jéből indult, nem a rendszer Pythonjából.

B feladat — távoli VM, az adat mellett. Az adat `s3://northstar-batch/orders_8m.parquet` formában az object storage-ban van, a VM ugyanabban a régióban fut. A kódot másoljuk, az adatot nem:

```
# ~/projects/northstar, Linux/macOS bash; a VM neve vm.internal
scp -r src pyproject.toml uv.lock remote-user@vm.internal:/srv/projects/northstar/
ssh remote-user@vm.internal
# a távoli munkamenetben, /srv/projects/northstar, Linux bash
uv sync --locked
sha256sum /srv/projects/northstar/src/northstar/predict.py
```

A tömörített Parquet hash-ét nem a VM-re másolás után ellenőrizzük — a fájl nem mozdult. A storage oldalán a manifest vagy az etag (a szolgáltató által adott objektum-ellenőrző jegy) rögzíti az eredetet; a VM a bucketből olvas, és a feldolgozás elején a sorok számával ellenőrzi az olvasás teljességét:

```
# szemléltető kód, nem futtatott kimenet
import pandas as pd
df = pd.read_parquet("orders_8m.parquet", columns=["order_id", "distance_km", "basket_minor"])
assert df["order_id"].nunique() == 8_000_000  # az adatszerződés szerinti egyedi kulcs
```

A managed notebook ugyanezt a logikát kapná: a projektet gitből klónozzuk, a kernelt a projekt uv-környezetéből regisztráljuk, az adatot a szolgáltató storage-ából olvassuk, a hitelesítő adatot a platform kulcstárolójából kérjük. A döntési lap ugyanaz, mint a VM-nél; csak a notebook-szerver üzemeltetése másik kézben van.

38.1.3.1 Adatbesorolás és kontrollpontok managed notebooknál

A szolgáltató elérhetősége nem bizonyítja, hogy a projekt adata feltölthető. Minden adathalmazhoz **érzékenységi besorolás** kell:

Besorolás	Példa	Managed notebook
`public` / `synthetic`	Northstar szintetikus labor	engedélyezett, ha a szolgáltató szerződése rendben van
`internal`	belső, nem személyes üzleti adat	csak jóváhagyott régióban, tenantizolációval
`personal`	ügyfélazonosító, e-mail, cím	csak szerződéses DPA és adatreztenció mellett; egyébként no-go
`sensitive`	egészségügyi, pénzügyi, különleges kategória	alapértelmezésben no-go, helyi vagy dedikált környezet

A kontrollpontok:

1. **PII-szken:** a feltöltés előtt a `customer_id`, e-mail, telefonszám, szabad szöveges megjegyzés mintázatainak ellenőrzése; találat esetén anonimizálás vagy no-go.
2. **Retenció:** a szolgáltató storage-ának és notebook-lemezeinek megőrzési ideje, törlési garancia, export után a helyi másolat sorsa.
3. **Export:** milyen hálózati útvonalon hagyja el az adat a szolgáltatót; engedélyezett-e a notebook letöltése, a modellartefaktum exportja.
4. **Hozzáférés:** felhasználók listája, MFA, legkisebb jogosultság, tenant- és régióizoláció, naplózás.
5. **Budget:** elszámolási metrika, riasztás, hard limit, leállítási eljárás.
6. **Szerződés:** DPA, adatfeldolgozó és alfeldolgozók, régió, incidensértesítés, törlés igazolása.

Példa a döntésre:

```
def managed_decision(classification, contract):
    if classification == "sensitive":
        return "no_go", "különleges kategória managed notebookban"
    if classification == "personal" and not contract.get("dpa"):
        return "no_go", "nincs DPA a személyes adathoz"
    if not contract.get("region_ok"):
        return "no_go", "nem megfelelő régió"
    if not contract.get("retention_days"):
        return "no_go", "hiányzó retenció"
    if not contract.get("budget_limit"):
        return "no_go", "hiányzó költségplafon"
    return "go", "kontrollpontok teljesülnek"
```

Az elvárt viselkedés: PII-t tartalmazó mintaprojekt szolgáltatóhoz vitele **no-go**, ha a szerződés ezt tiltja; a szintetikus labor a kontrollok teljesülése mellett mehet. A `tests/test_managed_notebook.py` a PII no-go-t, a hiányzó DPA/retenció/budget esetét és a szintetikus go-t teszteli.

38.1.4 Eredmény értelmezése és kézi kontroll

A térkép akkor helyes, ha három kérdésre egyértelmű a válasz:

1. **Hol van az adat?** A 20 GiB-os batch a bucketben marad; a fejlesztői CSV a laptopon.
2. **Hol fut a processz?** A batch a VM-en, az adattal azonos régióban; a fejlesztői kísérlet a laptopon.
3. **Mi bizonyítja, hogy a megfelelő környezet futott?** A `sys.executable`, a `uv sync --locked` sikere, a manifest hash és a feldolgozás eleji sor-/kulcsszám.

A hash-ellenőrzés toleranciája szigorú bitazonosság: egyetlen eltérő bájt is bukást jelent. A modellpredikciónál viszont a 32.4. alfejezetben bevezetett szabály él: a lebegőpontos oszlopok egyezését relatív és abszolút tűréssel is ellenőrizzük, mert a seed azonos lehet, de a platformok közötti bitazonosság nem automatikus. Így a futtatásihely-választás nem csempész be rejtett kísérleti tényezőt: a split manifest, a seed és a küszöb a 23.4. alfejezet tanulsága szerint a futás előtt rögzített.

38.1.5 Feladatok

1. Értelmezés. A B feladat 8 000 000 rendeléssel és 20 GiB tömörített Parquet-val fut. A becsült dekompresszált tábla 4,5-5,5 GB. Melyik környezet indokolt, és mi az a fizikai körülmény, amelynél a döntés megfordulna?

2. Megvalósítás. Készíts telepítési és adatmozgatási térképet a B feladathoz. Tüntesd fel: mi mozog, mi marad, hol fut a processz, és mi az ellenőrző invariant. Kategóriaszintű, szolgáltatóspecifikus konzolút nélkül.

3. Hibadiagnózis. A csapat egy managed notebook platformon fejlesztett, ahol a notebook fut. Később valaki a laptopra klónozza a repót, és `uv sync --locked` parancsot ad: a hiba szerint `uv.lock` nem létezik. Hogyan lehet, hogy a notebook a platformon mégis futott? Mi hiányzott a reprodukálhatósági láncból?

4. Megváltozott feltétel. Az adat 80 GiB tömörítettre nőtt, a feltöltési sebesség 100 Mbps, a batch-keret továbbra is 2 óra. Számold újra a mozgatási időt, és dönts: adatmozgatás vagy az adat melletti feldolgozás? Nevezd meg a korlátot.

38.1.6 Megoldások és indoklás

1. megoldás. A 32 GB RAM-os távoli VM az elsődleges jelölt, mert a kód mozgatása olcsó, az adat viszont a storage-ban maradhat. A döntés akkor fordul meg, ha a dekompresszált munkahalmaz meghaladja a VM RAM-ját, vagy ha a bucket elérése lassabb, mint a munkakönyvtár lemezén tartott példány. Az ellenőrző invariant: a feldolgozott sorok száma `8_000_000`, az egyedi rendeléskulcsok száma szintén `8_000_000`, és a predikciós oszlop hossza mindkettővel egyezik.

2. megoldás. A térkép:

Elem	Hely / mozgás	Döntés
`src`, `pyproject.toml`, `uv.lock`	laptop → VM	mozog, `scp` vagy git
`orders_8m.parquet`	object storage → VM olvasó	nem mozog, a bucketben marad
VM processz	ugyanaz a régió, mint a bucket	itt fut a batch
`manifest.json`, split, modell	VM → storage, verziózva	visszamennek artifactként

A processz telepítése a VM-en `uv sync --locked`; az olvasás előtt a manifest/etag és a sorszám ellenőrzése zárja a láncot. A döntési lap minden lépésére igaz: az adat és a processz fizikai határa kimondva szerepel.

3. megoldás. A managed notebook nem feltétlenül a projekt `uv.lock`-jából építette a kernelt: a platform saját, előre összeállított image-e csomagokat garantált, és a notebook a szerveren, a csapat tárolójában futott. Attól, hogy a notebook outputot adott, a környezet még nem volt a projekt lockjával rögzítve. A hiányzó láncszem a verziózott `uv.lock` magában a repóban, és a kernel `sys.executable`-ellenőrzése. Ez pontosan a 26.1. alfejezet tanulsága átfordítva: az eszköz megléte nem bizonyíték; a lock, az image és a manifest együttese az.

4. megoldás. A 80 GiB $= 80 \cdot 2^{30} = 85\,899\,345\,920$ B. A 100 Mbps feltöltés $= 12{,}5 \cdot 10^6$ B/s, így

$$\frac{85\,899\,345\,920}{12{,}5 \cdot 10^6} \approx 6872 \text{ s} \approx 1{,}91 \text{ óra}.$$

Ez önmagában majdnem felemészti a 2 órás keretet, és a dekompresszált becslés 4× aránnyal 320 GB, ami a 32 GB-os VM RAM-ját is meghaladja. A döntés tehát: nem mozgatjuk az adatot; a feldolgozás az adat mellett, partíciónkénti olvasással vagy lusta végrehajtással történik (34.1. alfejezet streaming, 34.2. alfejezet task graph). A szűk keresztmetszet itt a hálózat és a dekompresszált munkahalmaz, nem a CPU.

38.1.7 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés: *„a menedzselt notebook szolgáltatás gondoskodik a reprodukálhatóságról."* Ellenpélda: a csapat a platformon fejleszt, a notebook minden cellája lefut. Hat hónappal később a platform frissíti a kernel image-ét, és a `pandas` verzió megváltozik. A 6.2. alfejezetben tanult Copy-on-Write viselkedés más figyelmeztetést ad, egy láncolt értékadás kimenete akár meg is változhat, de a notebook outputja nem őrzi a kernel verzióját. Ha nincs `uv.lock` és manifest, senki nem tudja rekonstruálni, melyik környezet produkálta a régi számokat. A szolgáltatás a futás kényelmét adja; a reprodukálhatóság a projekt lockjának, image-ének és adathash-ének tulajdonsága.

38.1.8 Önellenőrzés

- Meg tudom nevezni, hol van a processz, hol az adat, és mi köti össze őket?
- A 20 GiB feltöltési becslésemben a lassabb irányt használtam, vagy a letöltést?
- A RAM-becslésem a dekompresszált reprezentációból indult, nem a tömörített fájlméretből?
- A hitelesítő adatok rövid életűek, és nem a notebook-outputban laknak?

38.1.9 Rövid összefoglaló

A futtatási hely kiválasztása követelmény-alapú döntés, nem platform-preferencia. A kis Northstar fejlesztés a helyi laptop `uv`/Jupyter környezetében él; a nagy batch esetén a kód megy az adathoz. A 20 GiB feltöltése 10 MB/s mellett körülbelül 36 perc, míg a forráskód másodpercek alatt másolható. A managed notebook a kernel és a notebook-szerver üzemeltetését adja, de a reprodukálhatóságot a projekt lock, az image és a manifest biztosítja. A hitelesítő adatok rövid életűek, és nem kerülnek notebookba. Következő alkalmazás: a 38.2. alfejezet CPU, GPU és elosztott rendszer közötti döntés ugyanezzel a kerettel.

38.2 CPU, GPU és elosztott rendszer közötti döntés

Belépő · szint: haladó · idő: 90 perc · eszköz: nincs új eszköz

Előfeltétel: 38.1. alfejezet · Célkészség: CPU, GPU és elosztott rendszer közötti döntés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Egy háromfős adattudományi csapat három párhuzamos feladaton dolgozik. Az A projekt a Northstar-kezdőadaton (300 ügyfél, 2400 rendelés, 8 ismert feature) logisztikus regressziót tanít, B projekt egymillió soros táblás adaton random forestet, C projekt pedig ötvenmillió sorból tanuló, tízmillió paraméteres neurális hálót futtat. A kérdés mindháromnál ugyanaz: helyi CPU, távoli CPU/GPU vagy elosztott rendszer kell? A hibás válasz drága: GPU-t adni egy 0,3 másodperces modellhez vagy klasztert indítani 10 MB adathoz a munkakör neve miatt, nem a mért korlát miatt. Az induló becslés: A helyi CPU, B többmagos CPU, C egy GPU; ezt a mérés pontosíthatja. A megfigyelési egység egy teljes kísérleti futás, a kimenet a futásidő, csúcsmemória, költség és reprodukálhatósági jegyzőkönyv.

38.2.1 Munkaterhelés, BLAS, kötegelés és a skálázási veszteség

A **munkaterhelés** (workload) típusát az uralkodó erőforrás-korlát adja. Egy futtatás idejét durva felbontással így becsülhetjük:

$$T \approx \frac{W}{P} + \frac{D}{B} + L$$

ahol

- W: elvégzendő műveletek száma, mértékegysége op (operáció);
- P: a műveleti átbocsátóképesség, op/s;
- D: mozgatott adatmennyiség, byte;
- B: sávszélesség, byte/s;
- L: indítási és ütemezési késleltetés, s.

Ha W/P dominál, a feladat számításkötött; ha D/B dominál, memória- vagy I/O-kötött; ha L a meghatározó, a párhuzamosítás overheadje a fő ellenfél. A pontos számításhoz mérés kell, de a képlet arra jó, hogy a szűk keresztmetszetet megkeressük.

A **BLAS** (Basic Linear Algebra Subprograms, alap lineáris algebrai szubrutinok) a mátrixműveletek szabványos építőeleme. A NumPy `@` művelete jellemzően egy helyi BLAS-könyvtárhoz fordul, amely jellemzően CPU-n fut, és implementációtól függően többszálú lehet; a hagyományos NumPy-BLAS művelet nem használ automatikusan GPU-t. Az $A \in \mathbb{R}^{m \times k}$ és $B \in \mathbb{R}^{k \times n}$ szorzás közelítő műveletigénye:

$$F = 2mnk \quad [\text{FLOP}]$$

Kézi kontroll: $m = k = n = 500$ esetén $F = 2 \cdot 500 \cdot 500 \cdot 500 = 2{,}5 \cdot 10^8$ FLOP. Ha a gép 10 GFLOP/s-ot ér el, a számítási komponens 0,025 s; a valódi idő ennél nagyobb a memóriából olvasás és az ütemezés miatt. A BLAS tehát nem automatikusan GPU-t jelent: CPU-n is párhuzamos szálakat futtat, és kis mátrixoknál a GPU-ra másolás több időt emészthet fel, mint a számítás.

A **kötegelés** (batching) a gradienses tanítás alapegysége: a modell egyszerre B mintát dolgoz fel, és egy paraméterfrissítéshez a batch gradiensét használja. n mintánál a lépések száma:

$$n_{\text{steps}} = \left\lceil \frac{n}{B} \right\rceil$$

Példa: $n = 128$, $B = 32$ esetén 4 lépés, $B = 16$ esetén 8 lépés van. A nagyobb batch kevesebb lépést és több GPU-kihasználtságot adhat, de a batch aktivációit egyszerre kell a memóriában tartani, ezért a batch méretét a VRAM korlátozza.

A **VRAM** a grafikus kártya saját memóriája. A szükséges méret első közelítésben a paraméterektől, a batch aktivációktól és a visszaterjesztés köztes értékeitől függ. Egy 10^7 paraméterű modell 32 bites lebegőpontos (fp32) súllyal $10^7 \cdot 4$ B = 40 MB paramétert foglal, a gradiens további 40 MB-ot, az optimizer nyomatékai és a batch aktivációi pedig ennek többszörösét tehetik ki. Ezért a memóriaszámítást nem elég a paraméterekből megtenni: a valós csúcs a tanítási lépés közben mérhető.

Ha a rendszer több gépre oszlik, a **hálózati átvitel** (network) költsége lép be. Az átviteli idő:

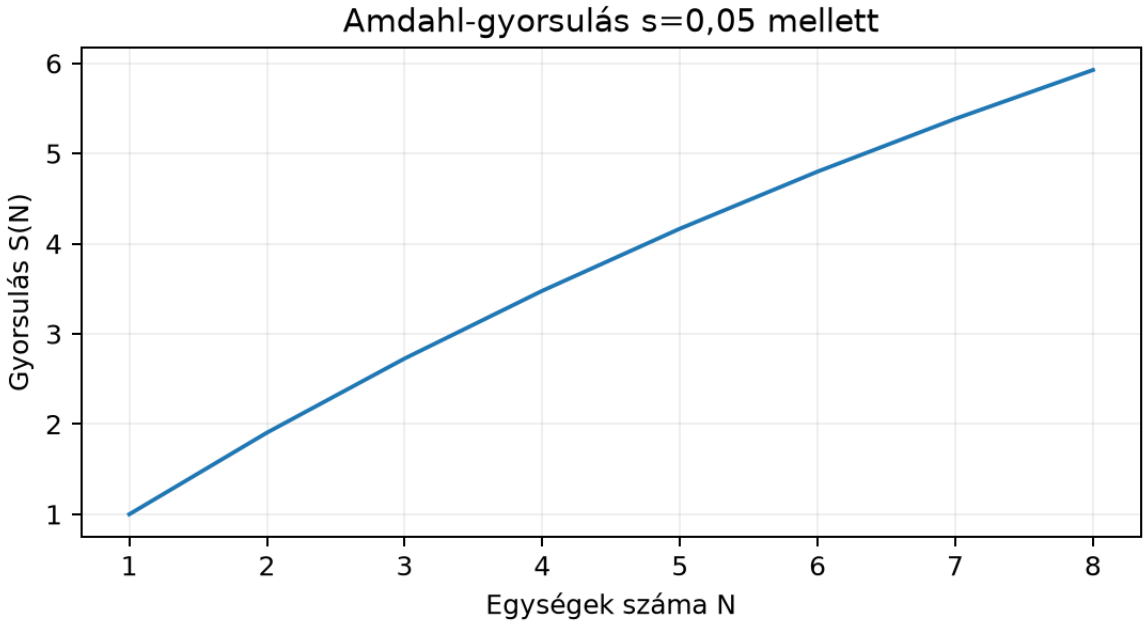
$$T_{\text{net}} = \frac{D}{B_{\text{net}}}$$

10 GiB adat 1 Gbit/s-os hálózaton $10 \cdot 8$ Gbit/1 Gbit/s = 80 s, 10 Gbit/s-on 8 s; ezen felül jön a protokollszintű overhead. A következtetés: akkor érdemes elosztani az adatot, ha a távoli gépek közötti átvitel és ütemezés kisebb, mint a megtakarított számítási idő.

A **skálázási veszteség** (scaling overhead) az a jelenség, hogy a párhuzamosításból származó gyorsulás nem nő arányosan a gépek számával, mert a feladat egy része soros marad, és az összerendezés is időt fogyaszt. Az Amdahl-törvény szerint, ha a feladat s hányada nem párhuzamosítható:

$$S(N) = \frac{1}{s + \frac{1-s}{N}}$$

ahol N az azonos sebességű egységek száma, S pedig a gyorsulás dimenziótlan aránya. Példa: $s = 0{,}05$, $N = 4$ esetén $S = 1/(0{,}05 + 0{,}95/4) = 1/0{,}2875 \approx 3{,}48$. A tökéletes lineáris gyorsulás 4 lenne; itt a hatékonyság 0,87. A kódban és a mérésekben ez konkrét összehasonlítási alap.



38.1. ábra: Amdahl-gyorsulás s=0,05 mellett - A pontok az S(N)=1/(0,05+0,95/N) képletből számítottak, nem mérések.

Az ábra mutatja, hogy 4 egységnél még 0,87 a hatékonyság, 8-nál már csak 0,74; a hozzáadott gépek egyre kevesebbet hoznak, miközben a hálózat és az ütemezés költsége nő. Ez a mérési varianciával együtt értelmezendő: ha a futásidő mérésének szórása 20%, akkor egy 10% gyorsulás nem igazolható megbízhatóan. Ezért minden erőforrás-döntést **mért korlátból** kell indítani: gépspecifikáció, adatméret, ismétlésszám, seed és pontos futási jegyzőkönyv nélkül a teljesítmény szám csak anekdota. A reprodukálhatóság különbségét is jegyezzük fel: a statisztikai bizonytalanság a mintából, nem a hardverből származik; a gyorsabb gép nem csökkenti a bootstrap-intervallum szélességét, csak több ismétlést enged.

38.2.2 Döntési lap és kis helyi mérési kontroll

A döntési lap eszközválasztás előtt kitöltendő. A három bevezető esetre:

Eset	Mért input és becslés	Elsődleges választás	Elosztási feltétel
A: Northstar	2400 sor, 8 feature, logreg, 5×5 CV; RAM < 1 GB	Helyi CPU uv/Jupyter	Nem osztjuk el; a futás 30 s alatt marad
B: közepes táblás	1 M sor, 60 feature, random forest 200 fa, 12 kombináció, 5-fold	Többmagos CPU, 64 GB VM	Csak akkor Dask, ha egy folyamat csúcs-RAM $>$ 80% fizikai RAM
C: nagy ML	50 M sor, 10 M paraméter, batch 64, 5 epoch	1 GPU, 16 GB VRAM	Több GPU akkor, ha az egykártyás batch nem fér, és a mért gyorsulás hatékonysága $> 0{,}7$

A táblázat az „elosztási feltétel” oszlopban explicit küszöböt ad, tehát nem elég azt mondani, hogy „ha lassú”. A $> 0{,}7$ hatékonyság azt jelenti: 2 GPU-val legalább 1,4-szeres mért gyorsulás az 1 GPU-hoz képest ugyanazon batch, seed és adat mellett. A 80%-os memória-küszöb azt kerüli el, hogy a swap már a mérés alatt eltorzítsa az eredményt.

A helyi mérési kontrollhoz készítsünk egy minimális BLAS-benchmarkot. A következő kódot a tanuló a saját gépen futtathatja; az alábbi kimeneti alak szemléltető, nem mért, mert a könyvírás nem hajtja végre a scriptet. A méréshez rögzíteni kell a processzormodellt, magszámot, NumPy-verziót és a futás időpontját.

```
# döntési_mérés.py
# Platform: bármely, ahol a projekt virtuális környezete aktív; cwd = projektgyökér.
# A mérést a tanuló futtatja; az alábbi kimenet csak alakminta.
import platform, statistics, time
import numpy as np

def measure_dot(m, k, n, reps=7, seed=20261020):
    rng = np.random.default_rng(seed)
    a = rng.normal(size=(m, k))
    b = rng.normal(size=(k, n))
    samples = []
    for _ in range(reps):
        t0 = time.perf_counter()
        c = a @ b
        _ = np.sum(c)              # a késleltetett BLAS-kimenetet felolvassuk
        samples.append(time.perf_counter() - t0)
    return statistics.median(samples), min(samples), max(samples)

for m in (100, 500, 1000):
    med, mn, mx = measure_dot(m, m, m)
    flop = 2 * m ** 3
    print(f"m={m:>4d}  median={med:.4f}s  min={mn:.4f}s  max={mx:.4f}s  "
          f"flop={flop:.3e}  gflops={flop/med/1e9:.1f}")

# Szemléltető várt kimeneti alak (nem mért):
# m= 100  median=0.0012s  min=0.0009s  max=0.0028s  flop=2.000e+06  gflops=1.7
# m= 500  median=0.0210s  min=0.0190s  max=0.0400s  flop=2.500e+08  gflops=11.9
# m=1000  median=0.1500s  min=0.1400s  max=0.2900s  flop=2.000e+09  gflops=13.3
```

A mérés értelmezése nem a pontos futásidő, hanem a mérési variancia és a skálázódás. Ha a medián idő $m = 100$-nál 0,001 s, $m = 1000$-nél 0,15 s, akkor a munka közelítőleg a várt m^3-ös növekedést követi; a min–max tartomány 2–3-szoros szórása viszont arra figyelmeztet, hogy 10%-os különbségekből hardverdöntést levonni nem szabad. A GFLOP/s érték az elméleti műveletszám és a mért idő hányadosa; ellenőrző invariant, hogy a műveletszám pontosan $2m^3$, a mért idő pedig pozitív valós szám.

A döntési lapot ezután így töltjük ki. A projekt A mérési futása a `starter_experiment.py` scripttel ellenőrizhető; ha 10 ismétlés mediánja 5 s alatt van, a CPU-döntés stabil. A B projektnél a random forest tanítása a faillesztés miatt sok elágazást és memória-keresést végez, ezért a szűk keresztmetszet a CPU, nem a BLAS; a GPU itt nem segít. A C projektnél a batch $B = 64$, a modell 40 MB paraméter + 40 MB gradiens + optimizer, így 16 GB VRAM-nál a korlát az aktivációk; ha az egy batch csúcsmemóriája eléri a 12 GB-ot, akkor kell batch-csökkentés vagy nagyobb kártya; ha a futási idő 1 GPU-n 8 óra, akkor a párhuzamosítás előtti mért overhead kell.

38.2.2.1 CPU–GPU crossover mérés warm-up-pal, VRAM-mal és költséggel

A CPU/GPU döntés nem szlogen, hanem legalább két mérési pont és egy modell összehasonlítása. A mérési protokoll:

1. **Warm-up:** 2–3 futtatás, amelyet nem értékelünk; a driver, a JIT és az allokátor bemelegszik.
2. **Ismétlés:** legalább 5, lehetőleg 9 futás; medián és p95, nem átlag.
3. **GPU szinkron:** minden mérés előtt és után `torch.cuda.synchronize()`, különben az aszinkron végrehajtás hamis időt ad.
4. **Átvitel:** a H2D (host→device) és D2H (device→host) másolás külön mérendő; kis adatnál ez dominál.
5. **VRAM:** a csúcsfoglalás (`torch.cuda.max_memory_allocated`) és a modell+aktivációk becslése; ha meghaladja a kártyát, a GPU nem opció.
6. **Költség:** a futásidő × óradíj, az átviteli és tárolási költséggel együtt.

A közelítő modell:

$$t_{\text{cpu}}(n) = a_{\text{cpu}} + \frac{n}{r_{\text{cpu}}}, \qquad t_{\text{gpu}}(n) = t_{\text{warmup}} + \frac{n}{B_{\text{transfer}}} + \frac{n}{r_{\text{gpu}}} + t_{\text{overhead}},$$

ahol n a feldolgozandó elemek száma, r a feldolgozási sebesség, B_{transfer} az effektív átviteli sávszélesség. A crossover méret a $t_{\text{cpu}}(n) = t_{\text{gpu}}(n)$ megoldása; alatta a CPU, felette a GPU nyer. A modell paraméterei mérésből jönnek, nem találgatásból.

Mennyiség	Jel	Mértékegység	Mérés
feldolgozott elem	n	darab	inputméret
CPU idő	t_{cpu}	s	medián 9 futásból
GPU idő	t_{gpu}	s	medián 9 futásból, szinkronizálva
átvitel	B_{transfer}	bájt/s	H2D/D2H külön mérés
VRAM csúcs	M_{vram}	MiB	`max_memory_allocated`
költség	C	pénzegység	óradíj × futásidő

A jelentésben a **mért** és az **elméleti** következtetés külön soron szerepeljen: a mérés a saját gépre érvényes, a crossover-modell csak a mért paraméterekkel extrapolál. A `tests/test_gpu_crossover.py` a CPU- és GPU-időmodellt, a crossover számítását, a VRAM-korlátot és a költségszámítást teszteli; kis adatnál a CPU, nagy adatnál a GPU nyer, magas átviteli költséggel a crossover jobbra tolódik.

38.2.3 Négy feladat

1. **Értelmezés.** A B projektnél ($n = 10^6$ sor, $p = 60$ feature, random forest 200 fa, `max_depth=15`) miért nem elsődleges a GPU? Nevezd meg a munka jellegét, és adj egy elérhető mérési jelet, amely megerősíti a döntést.
2. **Megvalósítás.** Írj Python-kódot, amely a fenti `measure_dot` függvénnyel $m = 200, 400, 800$-ra kiszámítja a medián időt és a GFLOP/s-ot, majd kiírja, hogy a mátrixszorzás ideje monoton nő-e. A seed legyen 20261020, az ismétlésszám 9.
3. **Hibadiagnózis.** Egy távoli VM-en a C projekt futásnaplója szerint az idő 80%-a adatbetöltés, a CPU 12%, a GPU 95% rövid tüskékben. Mi a szűk keresztmetszet, és milyen két konkrét mérést kérsz, mielőtt nagyobb GPU-t vásárolnál?

4. **Transzfer.** Az A projekt adatmérete 100-szorosra nő: 240 000 sor, továbbra is 8 feature és logreg, de a 5×5 CV együtt 40 percig fut. Számold ki a sorok szerinti Dask-szétosztás potenciális gyorsulását, ha a feladat 90%-a párhuzamosítható, 4 workerrel dolgozol, és az ütemezés plusz 10% overheadet ad. Mikor érdemes mégis elosztani?

38.2.4 Megoldások és indoklás

1. megoldás. A random forest faillesztése során a rendezések, hisztogramok és elágazások sok apró, adatfüggő memória-hozzáférést végeznek, nem pedig hosszú mátrixszorzásokat. A scikit-learn implementáció CPU-n futtat; a GPU-ra portolt változatok sem feltétlenül gyorsabbak ezen a feladaton. Mérési jel: a CPU magok kihasználtsága közel 100% a tanítás alatt, a GPU kihasználtsága viszont nulla vagy oszcilláló, míg a futásidő nem csökken GPU-hozzárendeléssel. Ellenőrző invariant: a várt munka nagyságrendje $n_{\text{trees}} \cdot n \cdot \log(n)$ jellegű, ami CPU-ütemezéssel jobban szabályozható.

2. megoldás.

```
# mérés_bővített.py, platform: helyi virtuális környezet, cwd = projektgyökér
import statistics, time
import numpy as np

def measure_dot(m, k, n, reps=9, seed=20261020):
    rng = np.random.default_rng(seed)
    a = rng.normal(size=(m, k))
    b = rng.normal(size=(k, n))
    samples = []
    for _ in range(reps):
        t0 = time.perf_counter()
        c = a @ b
        _ = np.sum(c)
        samples.append(time.perf_counter() - t0)
    return statistics.median(samples)

prev = None
for m in (200, 400, 800):
    med = measure_dot(m, m, m)
    flop = 2 * m ** 3
    print(f"m={m:>3d}  median={med:.4f}s  gflops={flop/med/1e9:.1f}  "
          f"monotone={prev is None or med > prev}")
    prev = med
```

A kimenetet a tanuló gépe határozza meg; a kód az idő monoton növekedését logikai értékként írja ki. Ellenőrzés: a FLOP-szám $2m^3$, az egyes GFLOP/s-értékek toleranciája a mérés természetes varianciája miatt 10–30%; ha $m = 800$-nál a GFLOP/s lényegesen magasabb, a cache- vagy BLAS-hatás miatt a mérési kontrollt újra kell futtatni.

3. megoldás. A napló I/O-kötöttséget mutat: a GPU a számítást rövid tüskékben elvégzi, majd vár az adatra, a CPU pedig nem számol. A nagyobb GPU nem segít, mert a betöltés nem gyorsul. Két mérés a döntés előtt:

- 20 batch esetén mérjük külön a DataLoader betöltési idejét (`__iter__`) és a modell forward lépését; számítsuk a mediánt és a p90 értéket. Ha a betöltés p90-je több mint háromszorosa a forward p90-jének, az I/O a korlát.
- Ellenőrizzük a lemez olvasási sorhosszát és a `prefetch` mélységét; növeljük a DataLoader `num_workers` értékét, és mérjük újra. Ha a betöltési idő harmadára csökken, a hiba a gyenge adatcső volt, nem a GPU.

4. megoldás. A sorok szerinti párhuzamosítás Amdahl-gyorsulása $s = 0{,}10$, $N = 4$: $S = 1/(0{,}10 + 0{,}90/4) = 1/0{,}325 \approx 3{,}08$. Ha ehhez 10% ütemezési overhead adódik, a hasznos gyorsulás $3{,}08/1{,}10 \approx 2{,}8$-szoros, tehát 40 percből körülbelül 14–17 perc lehet. Ez azonban csak akkor igaz, ha a foldok sorai függetlenek, a partíciók kiegyenlítettek, és a kommunikáció minimális. Elosztani akkor érdemes, ha a 240 000 soros adat egyetlen gépen már nem fér a RAM-ba, vagy ha a futási idő költségkeretet sért; előbb azonban olcsóbb a foldok számának csökkentése vagy a feature-műveletek vektorizálása, mert $240\,000 \times 8$ float64 alig 15 MB.

38.2.5 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb tévedés: *„data scientist vagyok, tehát GPU és klaszter kell."* Ellenpélda: a Northstar 2400 soros logisztikus regresszióját GPU-ra ütemezve a CUDA-kontextus indítása és az adat másolása gyakran hosszabb, mint a teljes helyi CPU-s tanítás; a klaszter pedig 10 MB adaton a schedulerindítás miatt lassabb. A munkakör neve nem mért korlát.

Korábbi fogalmak visszahívása: a Dask task gráfnál tanultuk, hogy a sok apró task overheadje nagyobb lehet a hasznos munkánál; ugyanez igaz a GPU-ra küldött apró batch-ekre. A kalibrációs és predikciós bizonytalanságnál láttuk, hogy az intervallum szélessége a minta- és modellbizonytalanságból ered, nem a hardverből; gyorsabb gép csak több szimulációt enged, nem csökkenti a standard hibát. Az R-beli OLS-nél 37.2. alfejezet is a minta elemszáma határozta meg a standard error-t, nem a futtatógép.

Önellenőrzés. Számold ki: 32 MB adat 1 Gbit/s hálózaton hány másodperc? ($32 \cdot 8/10^9 = 0{,}256$ s plusz késleltetés.) Mennyi az Amdahl-gyorsulás $s = 0{,}20$, $N = 8$ esetén? ($S = 1/(0{,}20 + 0{,}80/8) = 1/0{,}30 \approx 3{,}33$; a nyolc gépből háromnál kevesebb hasznosul.) Ha a mért mátrixszorzási idő $m = 1000$-re 0,15 s, mekkora a GFLOP/s? ($2 \cdot 10^9/0{,}15 \cdot 10^{-9} \approx 13{,}3$ GFLOP/s; ha a gép csúcsértéke 100 GFLOP/s, akkor nem számításkötött a BLAS, hanem memória-korlátos a művelet.)

38.2.6 Rövid összefoglaló

A CPU, GPU és elosztott rendszer közötti választás mért szűk keresztmetszetből indul: műveletigény, memória, I/O és ütemezési overhead. A BLAS a mátrixműveleteket gyorsítja, a batching a gradienses tanítás egysége, a VRAM a batch és aktiváció korlátja, a hálózati átvitel és az Amdahl-veszteség pedig az elosztás valódi árát mutatja. A három eset döntési lapja: kicsi Northstar helyi CPU, egymillió soros random forest többmagos CPU, ötvenmillió soros mély modell egy GPU; a több GPU-ra és klaszterre csak explicit, mérhető feltétellel lépünk. A következő 38.3. alfejezetben az eszközbevezetés, licenc és verziófrissítés döntési keretét tárgyaljuk.

38.3 Eszközbevezetés, licenc és verziófrissítés

Belépő · szint: haladó · idő: 90 perc · eszköz: nincs új eszköz

Előfeltétel: 38.2. alfejezet · Célkészség: Eszközbevezetés, licenc és verziófrissítés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Licencellenőrzés: az alkalmazáslicenc, a függőségek licence és a modell-/adatlicenc külön ellenőrzendő. A megengedett felhasználás nem következik abból, hogy a csomag telepíthető vagy a súlyok nyilvánosan elérhetők.

38.3.1 Cél és konkrét helyzet

A Northstar lab csapata egy változáskérelmet kap: a szervezet fenntartási szabályzata szerint a projekt pandas-dependenciáját 3.x főverzióra kell emelni, mert az N-1 politika a 2.x sorozatot a következő negyedévben elavultnak fogja minősíteni. A döntést nem „majd kiderül” alapon hozzák meg, hanem verziókezelési szabállyal, licencellenőrzéssel, frissítési teszttel, platformkorláttal és költség-/karbantartási korláttal alátámasztott döntési lapot (decision sheet) készítenek.

Bemenetek: a `lab/pyproject.toml` verziótartománya, a saját `uv.lock` a feloldás után, a `lab/starter_experiment.py`, a projekt tesztjei és a meglévő kontrolljegyzék. Megfigyelési egység: egy ellenőrzési tétel a döntési lapon; minden tételhez érték, dátumozott forrás és állapot (`verified`, `pending`, `n/a`) tartozik. Kimenet: kitöltött döntési lap és `go`/`no-go` döntés. Előzetes becslés: a főverzió-emelés elsősorban az implicit láncolt értékadást (chained assignment) törheti; mivel a kezdőprojekt explicit `.loc`-ot használ, a kockázat várhatóan alacsony – de ez csak hipotézis, amíg a frissítési teszt le nem fut.

38.3.2 Az öt döntési összetevő

Version policy (verziókezelési szabály). A `MAJOR.MINOR.PATCH` alak csak akkor jelent szemantikus verziózást, ha a csomag karbantartója azt a szerződést követi, és a release notes-ban vállalja. A Python-ökoszisztémában sok projekt használ ilyen számozást, de a jelentés **projekt-specific**: a `PATCH` a legtöbb projektnél hibajavítás, mégis változtathat kimeneti viselkedést vagy javíthat egy korábban hibás számítást; a `MINOR` nem garantál bitazonos numerikus eredményt; a `MAJOR` pedig nem feltétlenül minden fogyasztónál tör. A Python nyelvi verzió (például 3.12 → 3.13) nem SemVer-szerződés, a pandas 2.x → 3.x saját migrációs útmutatót ad, a NumPy pedig saját deprecation policy-t követ. Minden verióváltásnál a csomag release notes-át, changelogját, `Requires-Python` mezőjét és deprecation policy-ját kell ellenőrizni; a `pyproject.toml` tartománya szándék, a `uv.lock` a feloldás pillanatának rögzített ténye. A policy két kérdést ír elő minden emelés előtt: hány támogatott főverziót tartunk életben, és milyen teszt jár az egyes emelési osztályokhoz. Az N-1 szabály azt jelenti, hogy a legutóbbi stabil főverziót és az előtte lévőt tartják karbantartva; ha a 3.x stabil, a 2.x karbantartási ablak zárul.

Kis számpélda: `2.2.3 → 2.2.4` a számozás szerint patch, `2.2.3 → 2.3.0` minor, `2.2.3 → 3.0.0` major. A konzervatív policy a számozásból **nem** vezeti le a garanciát: patch-emelésnél is elolvassa a release notes-t, és ha az viselkedésváltozást jelez, teljes numerikus kontrollt futtat; minor-emelésnél alapból teljes tesztet és numerikus kontrollt kér; major-emelésnél ezeken felül kompatibilitási auditot és rövid canary-ablakot.

38.3.2.1 SemVer és az eltérő verziópolitikák

Projekt	Verziózás	Amit a szám jelent	Amit ellenőrizni kell
SemVer-t követő csomag	`MAJOR.MINOR.PATCH`	elméleti kompatibilitási szerződés	release notes, changelog, tényleges API-diff
Python nyelv	`3.12`, `3.13`	nyelvi és stdlib-verzió	PEP-ek, deprecation, `Requires-Python`
pandas	`2.x`, `3.x`	saját major/minor sorozat	migrációs útmutató, viselkedésváltozások

Projekt	Verziózás	Amit a szám jelent	Amit ellenőrizni kell
NumPy	`2.x`	saját policy	release notes, deprecation policy, ABI

A táblázat tanulsága: a `PATCH` nem jelent automatikus „nincs változás” garanciát. A verziószám csak annyit ér, amennyit a projekt a dokumentációjában vállal; a frissítési teszt ezért minden emelésnél a release notes-t és a saját kontrolljegyzéket nézi, nem a szám alakját.

License check (licencellenőrzés). A csomag licencét nem elég egyszer ismerni: a célverziónál újra ellenőrizni kell, mert a licenc is változhat. A gyakori SPDX-azonosítók közül a permissive családba tartozik a MIT, BSD, Apache-2.0; a copyleft családba a GPL, a hálózati szolgáltatásnál különösen érzékeny AGPL; létezik kettős licencelés is. A döntést a felhasználás módja befolyásolja: belső script, tovább terjesztett csomag vagy nyilvános szolgáltatás különböző kockázati osztály. A döntési lap ezért nem „a pandas szabad-e” kérdést tesz fel, hanem azt, hogy melyik SPDX-kifejezés szerepel a célverzió metaadatában, és mikor nyitották meg az upstream LICENSE fájlt. Jogi állásfoglalást a lap nem helyettesít.

Upgrade test (frissítési teszt). A frissítés nem verziók cseréje, hanem viselkedésváltozás kockázata. Három réteg: füstteszt (import, alapművelet, ismert invariant), teljes projektteszt, majd reprodukálható numerikus kontroll. A kontroll nem „ránézésre ugyanaz”, hanem a 37.3. alfejezetben bevezetett abszolút-relatív tolerancia:

$$|a - b| \le \epsilon_a + \epsilon_r \cdot |b|,$$

ahol a és b ugyanannak a mennyiségnek a jelölt és a baseline értéke ugyanabban a mértékegységben, ϵ_a az abszolút küszöb, ϵ_r a relatív küszöb. Egész értékeknél (sorazonosítók, darabszámok, minor unitok) bitpontos egyezést várunk; lebegőpontos értékeknél $\epsilon_a = 10^{-9}$ és $\epsilon_r = 10^{-7}$ észszerű szigorú választás.

Platform constraint (platformkorlát). Egy verzió létezése nem jelenti, hogy az adott gépen fut. A lock feloldás a Python-verzió, az operációs rendszer és a CPU-architektúra kombinációjától függ: egy macOS arm64 wheel nem érvényes Linux x86-64-en, és egy bináris csomag feltételezi a megfelelő libc/BLAS-környezetet. A `uv.lock` platformcímkéken keresztül rögzíti, mely wheel mely környezethez tartozik; a `--locked` futtatás ezt ellenőrzi. A döntési lap ezért külön tételként rögzíti a támogatott platformot, és nem engedi, hogy a fejlesztői gép sikeréből a másik környezetre következtessünk.

Cost cap (költségplafon). Helyi bevezetésnél a „költség” karbantartási idő, managed notebook vagy felhőszolgáltatás esetén ellenőrzött díjszabásból számolt keret. A plafon négy elemet jelent: elszámolási metrika (pl. vCPU-óra, compute unit-óra, tárolt GB), riasztási küszöb (pl. a keret 80%-ánál), hard limit (a fölött a számítás leáll), és visszaállítási út (a régi helyi környezet visszaáll). Az aktuális díjat ez a könyv nem találja ki; a döntési lap a szolgáltató hivatalos árazó oldalát és az ellenőrzés dátumát kéri.

38.3.3 Teljes minimális munkapélda: pandas főverziós jelölt

A munkamenet a `lab` projektgyökérből indul, bash alatt. A szállított labban **szándékosan nincs** `lab/uv.lock`: a lock előállítása a 3.2. alfejezet önálló feladata. A `lab/reference/uv.lock` a szerző által ellenőrzött referenciafeloldás (uv 0.13.6, Python 3.12, 282 csomag, SHA-256

7cca92a06418c46c83f49885a4686a83506c1ace22813d55a94cbfae67d90707). A frissítési gyakorlatban a baseline lockot a saját `uv lock` hozza létre; ha a feladat miatt újraírod, előbb `cp uv.lock uv.lock.baseline` néven másold el, és jegyezd fel az eredeti SHA-256-ot. A `uv.lock` tehát nem hiányzó elem, hanem a gyakorlat létrehozandó artefaktuma; a referencia lock a kontroll.

```
cd lab
uv lock
uv sync --locked
uv run pytest -q
```

Az `uv lock` a `pyproject.toml` tartományaiból felold és `uv.lock`-ot ír. Az `uv sync --locked` megtagadja a futtatást, ha a lock és a projekt nem egyezik; a `--frozen` a további aktualitásvizsgálatot is kihagyja. A baseline teszt eredményét rögzítjük a döntési lapon.

Ezután a jelölt ágat külön worktree-ben nyitjuk, hogy a baseline érintetlen maradjon:

```
git worktree add ../lab-candidate pandas3-candidate
cd ../lab-candidate
# itt a pyproject.toml pandas-tartományát a 3.x sorozatra állítjuk
uv lock
uv sync --locked
uv run pytest -q
```

A füstteszt a projekt adatán egy, a pandas-verziótól független invariánst ellenőriz:

```
import pandas as pd

df = pd.read_csv("data/orders.csv", parse_dates=["order_ts"])
assert df["target_late"].isin({0, 1}).all()
by_segment = df.groupby("segment")["basket_minor"].sum()
assert by_segment.sum() == df["basket_minor"].sum()
print(df["basket_minor"].sum())
```

A csoportösszegek összege a teljes összeggel egyezik: ez a 34.3. alfejezetből visszahívott relációs invariant. A kimenet szám; az ellenőrzés az `assert` miatt már a kódon belül megtörtént.

A numerikus kontroll a `starter_experiment.py` kimenetét hasonlítja össze a jelölt és a baseline környezetben:

```
cd ../lab-candidate
uv run python starter_experiment.py
cp artifacts/split.json ../lab/artifacts/split_candidate.json
cd ../lab
uv run python starter_experiment.py
cp artifacts/split.json artifacts/split_baseline.json
uv run python compare_splits.py artifacts/split_baseline.json artifacts/split_candidate.json
```

A `compare_splits.py` gén specifikus, rekurzív JSON-összehasonlító: egészekre és sztringekre bitpontos egyezést, lebegőpontos értékekre a fenti toleranciaképletet alkalmazza.

```
import json
import math
import sys

ATOL = 1e-9
RTOL = 1e-7

def compare(a, b, path="$"):
    if type(a) is not type(b):
        return [f"{path}: típuseltérés {type(a).__name__} != {type(b).__name__}"]
    if isinstance(a, dict):
```

```
        out = []
        for key in sorted(set(a) | set(b)):
            if key not in a:
                out.append(f"{path}.{key}: csak a jobb oldalon")
            elif key not in b:
                out.append(f"{path}.{key}: csak a bal oldalon")
            else:
                out.extend(compare(a[key], b[key], f"{path}.{key}"))
        return out
    if isinstance(a, list):
        if len(a) != len(b):
            return [f"{path}: hosszeltérés {len(a)} != {len(b)}"]
        out = []
        for i, (x, y) in enumerate(zip(a, b)):
            out.extend(compare(x, y, f"{path}[{i}]"))
        return out
    if isinstance(a, float) and isinstance(b, float):
        if math.isclose(a, b, rel_tol=RTOL, abs_tol=ATOL):
            return []
        return [f"{path}: {a} != {b} tűrésen kívül"]
    return [] if a == b else [f"{path}: {a!r} != {b!r}"]

def main():
    _, left, right = sys.argv
    a = json.load(open(left, encoding="utf-8"))
    b = json.load(open(right, encoding="utf-8"))
    issues = compare(a, b)
    if issues:
        print("\n".join(issues))
        sys.exit(1)
    print("OK: a két split-kontroll a tűrésen belül egyezik.")

if __name__ == "__main__":
    main()
```

A listaág sorrendérzékeny: ha a két fájl listái eltérő sorrendben tartalmaznák ugyanazokat az azonosítókat, az hibának számít. Ez szándékos, mert a split fájlban a sorrend maga is része a szerződésnek.

A döntési lap sablonja:

#	Tétel	Ellenőrzött érték	Forrás és dátum	Állapot
1	Baseline verzió	a jelenlegi lab pandas-verzió	saját `uv.lock` és futtatási jegyzék	verified
2	Célverzió a policy szerint	3.x jelölt; N-1 szabály	szervezeti policy verziószámmal	pending
3	Licenc	SPDX-kifejezés a célverzión	PyPI metadata + upstream LICENSE	pending
4	Platform wheel	Python-verzió, OS, architektúra	`uv lock` és PyPI wheel-lista	pending
5	Füstteszt	import, csoportinvariáns zöld	lokális munkamenet	pending
6	Teljes tesztsor	`uv run pytest -q` zöld	lokális munkamenet	pending

#	Tétel	Ellenőrzött érték	Forrás és dátum	Állapot
7	Numerikus kontroll	`split.json` egyezés a tűrésen belül	`compare_splits.py` kimenet	pending
8	Költség/karbantartás	n/a – helyi CPU-n	–	n/a
9	Visszaállítási terv	régi lock + `uv sync --locked --frozen`	dokumentálva a repóban	verified

38.3.4 A lap értelmezése

A sablon nem kész döntés, hanem gyűjtőhely. A `go` döntéshez minden következő feltételnek teljesülnie kell: (1) az 1–4. tételeknél a forrás a tényleges ellenőrzés dátumával szerepel, és az ellenőrzés nem régebbi 90 napnál; (2) az 5–7. tétel lokálisan lefutott, a tolarencián belül; (3) a 9. tétel visszaállítási sora konkrét parancsot tartalmaz, nem „megoldjuk”; (4) a 8. tétel `n/a` mellett explicit módon kimondja, hogy nincs felhő- vagy szolgáltatásköltség. Az `n/a` állapot itt maga is állítás: a következő hónapban, managed notebookra átálláskor ezt a sort újra ki kell tölteni, nem örökölhető.

A frissítési teszt numerikus értelmezése: ha a split-azonosítók bitpontosan egyeznek, és a lebegőpontos mezők az abszolút-relatív tűrésen belül maradnak, akkor a főverzió-emelés a kezdőprojekt döntési felületét nem változtatta meg. Ez statisztikai értelemben nem „bizonyíték arra, hogy a 3.x mindenhol azonos”, hanem egy konkrét inputra, konkrét kontrollra vonatkozó megfigyelés. A `compare_splits.py` kimenete a tanuló gépén lehet `OK: ...`; automatikusan még nem ellenőrzött, ezért itt csak a lehetséges kimenetet írtuk le.

38.3.5 Feladatok

1. **Értelmezés.** A következő verzióváltások közül melyik patch, minor, major emelés a `MAJOR.MINOR.PATCH` szabály szerint: (a) `2.2.3 → 2.2.4`; (b) `2.2.3 → 2.3.0`; (c) `2.2.3 → 3.0.0`; (d) `3.0.0 → 3.1.0`. A konzervatív policyben melyikhez elég csak füstteszt, melyikhez kérünk teljes numerikus kontrollt is?

2. **Megvalósítás.** A csapat managed notebook szolgáltatást vezetne be három elemző számára: Linux konténer, 4 vCPU, havi kb. 40 használati óra, a bemenő adatok Parquet-fájlok helyi hálózaton. Készíts a fenti mintához illeszkedő, legalább nyolcsoros döntési lapot: minden sorhoz írj ellenőrizhető forrást, és ahol most nem lehetséges az ellenőrzés, `pending` állapotot a szükséges forrás megnevezésével. Aktuális díjat ne írj.

3. **Hibadiagnózis.** A kolléga a `pyproject.toml` `numpy>=1.26,<3` tartományával dolgozik, de nem készített `uv.lock`-ot, és mindig sima `uv sync`-et futtat. Egy hónappal később a teszt zöld, a numerikus kontroll azonban `1e-4` relatív eltérést mutat a baseline kontrolljegyzékhez képest. Mi a policy hiba, mi okozhatta az eltérést, és milyen paranccsal állítod helyre a baseline-környezetet?

4. **Megváltozott feltétel.** Az előző managed notebook bevezetés költségplafonja: havi 100 elszámolási egység, riasztás 80 egységnél, hard limit 100 egységnél. Add meg: mit kell naponta mérni, mi történjen a hard limit elérésekor, és melyik helyi elem maradjon készenlétben visszaállítási útnak? Az egység árát ne tippeld; a metrika definícióját a szolgáltató fogalomtárából kell idézni.

5. **Továbbvitel.** A managed notebook kliensparancsa MIT licencű, ezért a kolléga szerint a teljes szolgáltatás ingyenes. Nevezd meg a lap azon sorát, amely ezt a következtetést megcáfolja, és add meg, milyen dokumentumot kell dátummal ellenőrizni.

38.3.6 Megoldások és indoklás

1. (a) A számozás szerint patch: `PATCH` nő; ez azonban csak akkor garancia, ha a csomag követi a SemVer-t, ezért a release notes-t el kell olvasni, és ha az viselkedésváltozást jelez, numerikus kontroll is kell. (b) minor: új funkció, a számozás szerint visszafelé kompatibilis; füstteszt mellett teljes tesztet és numerikus kontrollt kérünk, mert a belső számítási sorrend akkor is változhat, ha a publikus API nem tör. (c) major: törő változás lehetséges; a konzervatív policy teljes kompatibilitási auditot, az összes függő csomag ellenőrzését és rövid canary-időszakot ír elő. (d) minor, ugyanaz az osztály, mint (b). A numerikus kontroll mindig a saját kontrolljegyzékre vonatkozik, nem a csomag upstream tesztjeire.

2. Egy lehetséges lap (a 4. oszlop szerint minden sor `pending`, ahol helyi futtatás vagy friss forrás hiányzik):

#	Tétel	Ellenőrzött érték	Forrás és dátum	Állapot
1	Szolgáltatástípus	managed notebook, Linux konténer	szolgáltató dokumentációja	pending
2	Verziópolicy	a szolgáltató által kínált környezetverzió	szolgáltató release notes	pending
3	Klienslicenc	MIT kliens, de a szolgáltatás külön vizsgálandó	kliens repo LICENSE	pending
4	Platform	4 vCPU, Linux, Parquet-hozzáférés	szolgáltató instance-leírás	pending
5	Füstteszt	egy notebook betölt, olvas Parquet-ot	lokális próbagép	pending
6	Hozzáférés	3 felhasználó, legkisebb jogosultság	szervezeti hozzáférési szabály	pending
7	Adatkijárat	Parquet be- és kimenet útvonala	hálózati terv	pending
8	Költségplafon	metrika, riasztás, hard limit	szolgáltató árazó és számlázó oldala	pending
9	Visszaállítás	helyi uv/Jupyter készenlétben	dokumentált	verified

A táblázat nem tartalmaz árat, mert a feladat ezt tiltja; a 8. sor forrása pontosan megnevezi, mit kell később dátumozva ellenőrizni.

3. A tartomány fagyasztás nélkül lebegő ablak: egy hónap alatt a `numpy` új patch/minor verziója kerülhetett a környezetbe, és a `1e-4`-es relatív eltérés tipikusan lebegőpontos műveleti

sorrend-változás vagy könyvtári algoritmusváltás következménye, nem feltétlenül hiba. A policy hiba az, hogy lock nélkül a „teszt zöld” nem ugyanaz a környezet, mint a kontrolljegyzék készítésekor. A javítás: `uv lock`, a lock verziózása, ezentúl minden futtatás `uv sync --locked`-dal. A baseline-hoz tartozó régi lock visszaállítása után `uv sync --locked --frozen` hozza létre a kontrolljegyzékkel konzisztens környezetet; az eltérő verziókat a döntési lap 2. sorában feljegyzik.

4. Metrika: a szolgáltató fogalomtárában definiált elszámolási egység fogyasztása naponta exportálva, a számlázási nézetből. A riasztási szabály: a kumulált havi fogyasztás 80 egység fölött figyelmeztetést küld a projekt tulajdonosának. Hard limit: 100 egységnél a futtató szkript leállítja az új számításokat; a meglévő adatokat a leállítási eljárás kimenti a szervezeti tárhelyre, mielőtt a szolgáltatás számítási hozzáférése felfüggesztődne. Visszaállítási út: a helyi uv/Jupyter környezet a Parquet-adatokkal a bevezetés előtti állapot szerint készenlétben marad, így a kísérleti munka nem áll le a szolgáltatás kikapcsolásával.

5. A lap 3. sora (klienslicenc) és 8. sora (költségplafon) együtt cáfolja a következtetést: az MIT licenc a klienskód felhasználására vonatkozik, nem a menedzselt infrastruktúra díjára. Ellenőrizendő a szolgáltató hivatalos árazó oldala, a szolgáltatási feltételek (Terms of Service) és az adatkijárat/tárhely díjszabása, mind dátumozva. A nyílt forrású kliens egy ellenőrzési tétel, nem az egész döntés.

38.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy a nyílt forrású kliensből a teljes szolgáltatás ingyenességére következtetnek. Ellenpélda: a helyi DuckDB Python-csomag MIT licencű, és önmagában tényleg ingyenesen használható; ha azonban a csapat a DuckDB-t menedzselt szolgáltatásként vagy felhőbeli tárhely felett futtatja, a számítási erőforrás, a tárhely és a hálózati kijárat költsége külön szerződés tárgya. Ugyanez igaz a managed notebook kategóriákra: a nyílt forrású kliens vagy a Jupyter-projekt licence semmilyen állítást nem tesz a mögötte álló infrastruktúra áráról. A döntési lap 8. sora ezért mindig külön tétel, és a `n/a` állapot is dátumozott döntés, nem örökölt alapbeállítás. Ez a tétel a 38.2. alfejezetből ismert költség- és erőforrás-korlát gyakorlati megvalósítása; a numerikus kontroll a 37.3. alfejezet toleranciaszerződését, a hibatűrés pedig a 26.3. alfejezetben tanult, számokkal alátámasztott értékelési szemléletet viszi tovább.

Önellenőrzés: minden döntési lapsorban van-e dátumozott forrás? A lock és a projekt együtt van-e verziózva? A frissítési teszt a saját kontrolljegyzékre futott-e, nem az upstream dokumentációjára? A visszaállítási terv konkrét parancsot tartalmaz-e? Ha bármelyik válasz nem, a döntési lap még `pending`, nem `go`.

38.3.8 Rövid összefoglaló

Az eszközbevezetés nem egyetlen parancs, hanem öt összetevős döntési folyamat. A verziókezelési szabály rögzíti, milyen emeléshez milyen teszt jár; a licencellenőrzés mindig az adott célverzión fut, dátumozott forrással; a frissítési teszt füsttesztből, teljes tesztsorból és numerikus kontrollból áll, előre rögzített abszolút-relatív toleranciával; a platformkorlát a wheel és a natív környezet együttesét vizsgálja; a költségplafon metrikát, riasztást, hard limitet és visszaállítási utat követel. A döntési lap minden sora ellenőrzött érték–forrás–dátum–állapot négyest hordoz, és a nyílt forrású kliens ingyenessége semmikor nem bizonyítja a mögöttes szolgáltatás költségmentességét. Következő alkalmazásként a 38.4. alfejezet saját data science munkaasztal specifikációját állítja össze ezekből a kritériumokból.

38.4 Saját data science munkaasztal specifikációja

Belépő · szint: haladó · idő: 90 perc · eszköz: nincs új eszköz

Előfeltétel: 38.3. alfejezet · Célkészség: Saját data science munkaasztal specifikációja · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Átadható specifikáció: a munkakörnyezethez ne csak eszközlista, hanem tiszta telepítési parancs, platform- és verziómátrix, minimális smoke test, függőséglakat és adat-hozzáférési szabály is tartozzon.

38.4.1 Cél és konkrét helyzet

A gyakorlóprojekt a Northstar prediktív feladat: a rendelés pillanatában becsüljük, hogy a kézbesítés 7 napon túli lesz-e (`target_late`). A bemenet a mellékelt labor `customers.csv`, `orders.csv` és `order_items.csv` fájlja; a megfigyelési egység az `order_id`; a cutoff `2025-05-01T00:00:00+00:00`. A feladat most nem maga a modell, hanem az, hogy a munkaasztalt úgy adjuk át, hogy egy friss CPU-gép a labor readme-je alapján ugyanazt a splitet és ugyanazt az elfogadási kontrollt reprodukálja.

Előzetes nagyságrendi becslés: az `orders.csv` 2400 sora 15 oszlop esetén, ha minden oszlop 8 byte-os numerikus érték lenne, $2400 \cdot 15 \cdot 8\,\mathrm{B} \approx 0{,}29\,\mathrm{MB}$; a mellékelt tételekkel együtt is néhány MB-os szinten maradunk. Ez tehát memóriában elférő, egygépes feladat; a Spark, a GPU vagy a menedzselt klaszter nem indokolt. Ez becslés, nem futási mérés.

38.4.2 Fogalmi magyarázat: a hét réteg mint predikátumlánc

A munkaasztal-specifikáció hét rétege nem egymás melletti csomaglista, hanem futtatási lánc. A 37.4. alfejezetben használt döntési fegyelem és a 38.3. alfejezetben bevezetett ellenőrzött érték–forrás–dátum–állapot sor itt konkrét rétegtesztként jelenik meg.

Réteg	Kötelező választás	Alternatíva / opcionális	Elfogadási teszt
`interpreter`	CPython 3.12 a projekt `.venv`-jében	pyenv, conda/mamba	`sys.version_info[:2] == (3,12)` és `sys.executable` a projekt `.venv` alatt
`packages`	`uv lock` feloldás, majd `uv sync --locked`	venv+pip, pip-tools	a lock-feloldás sikeres, a labor alap numerikus csomagjai importálhatók
`data engine`	pandas feature-tábla, DuckDB SQL, Parquet checkpoint	Polars, SQLite	`COUNT(*)` = 2400, séma-ellenőrzés, `manifest.json` hash
`tracking`	`artifacts/split.json` + egy futástábla	MLflow	kötelező kulcsok, train/purged/test sorszámok

Réteg	Kötelező választás	Alternatíva / opcionális	Elfogadási teszt
`execution`	helyi CPU, `uv run`, JupyterLab	CI futás, Docker	a starter script exit-kódja 0, a nyers bemenet változatlan
`reporting`	notebook + Modellkártya	Quarto/HTML	`nbconvert` exit 0, a kártyában „Nem támogatott használat” szakasz
`handoff`	README, lock, manifest, modellkártya, runbook	Docker image	tiszta gépről `uv sync --locked` + script + hash-egyezés

A lap elfogadását egy logikai ÉS-lánc írja le:

$$S = \bigwedge_{i=1}^{7} t_i,$$

ahol $t_i \in \{0,1\}$ az i-edik rétegteszt kimenete; a t vektor alakja $(7,)$, értékkészlete $\{0,1\}^7$, mértékegysége darab sikeres teszt. A specifikáció akkor elfogadott, ha $S = 1$; egyetlen 0 is elutasítást jelent. Ez azért fontos, mert a stack nem „részben reprodukálható”: ha a kernel rossz interpreterre mutat, a tökéletes lock sem segít.

38.4.2.1 Pontozott minősítési rubrika

A puszta ÉS-lánc mindent egyformán kezel; a gyakorlati átvételhez **pontozott rubrika** kell, amely megkülönbözteti a minimumot a haladó kívánságlistától. Hat szempont, egyenként 0–2 pont:

Szempont	0 pont	1 pont	2 pont	Javítás 0/1 esetén
Reprodukálható build	nincs lock vagy parancs	lock van, de kézi lépés kell	tiszta gépen `uv sync --locked` + script	lock létrehozása, parancsok README-be
Jogszerű adathozzáférés	jogosultság tisztázatlan	olvasási jog dokumentált	minimális jog, naplózás, retenció	hozzáférési mátrix és DPA
1 perces smoke test	nincs teszt	kézi ellenőrzés	automatizált, 1 perc alatt fut	smoke script + exit-kód
Időhelyes lekérdezés	cutoff/elérhetőség hiányzik	cutoff van, label-availability nem	purged split és manifest	split manifest, as-of teszt
Teljes költség	csak hardverár	hardver + óradíj	átvitel, tárolás, üzemeltetés is	költségtábla a mérési adatokból
Platform	nincs megadva	OS/Python részben	OS, arch, Python, lock együtt	platformmátrix és CI

Az értékelés:

```
CRITERIA = ["build", "access", "smoke", "temporal", "cost", "platform"]

def score_workbench(scores):
    total = sum(scores.values())
    zeros = [key for key in CRITERIA if scores.get(key, 0) == 0]
    ones = [key for key in CRITERIA if scores.get(key, 0) == 1]
    passed = not zeros and total >= 10
    return {"total": total, "max": 12, "passed": passed,
            "blocking": zeros, "improvements": ones}

print(score_workbench({"build": 2, "access": 2, "smoke": 2,
                       "temporal": 2, "cost": 1, "platform": 2}))
# {'total': 11, 'max': 12, 'passed': True, 'blocking': [], 'improvements': ['cost']}
```

A **minimum** a 10/12 pont és az, hogy egyetlen szempont se legyen 0; a 12/12 a haladó kívánságlista (például teljes költségmodell és CI-mátrix). A részben teljesítő megoldáshoz a táblázat utolsó oszlopa objektív javítási utat ad, nem „javítsd meg” típusú általánosságot. A `tests/test_workbench_rubric.py` a teljes, a részleges és a 0 pontot kapó szempontokat, valamint a 10 pontos küszöböt teszteli.

A hét réteg közül a `interpreter` és a `packages` különválasztása szándékos. Az interpreter az a program, amely a Python-bytecode-ot végrehajtja; a packages a projekthez tartozó könyvtárak lockolt halmaza. A notebook maga nem interpreter: csak egy kliens, amely egy regisztrált kernelén keresztül beszél valamelyik Pythonhoz. Ezért a notebook típusa nem dönti el, hogy a labor `.venv`-jét futtatjuk-e.

38.4.3 Teljes minimális munkapélda: specifikációs lap és ellenőrző munkamenet

Az átadandó egyoldalas lap fejléce a konkrét projektet köti a hét réteghez:

```
# Northstar stack-spec
projekt: rendeléskori target_late predikció
megfigyelési egység: order_id
cutoff: 2025-05-01T00:00:00+00:00
invariánsok: orders sorcount=2400, manifest SHA-256,
             split train+purged+test=2400
futtatási hely: egyetlen CPU-gép, helyi labor
```

A táblázat hét sora e fejléc alá kerül; a leadott lapban minden sorhoz egy rövid, idézhető forrás vagy ellenőrzési utasítás tartozik. A következő platformjelölt munkamenet mutatja a minimális láncot. Munkakönyvtár: a repository `lab/` mappája; platform: Linux/macOS Bash. A lab még nem szállít feloldott lockot, ezért az első lépés a feloldás.

```
cd lab
uv lock                 # a mellékelt laborhoz nincs szállított uv.lock
uv sync --locked        # ezután már a lock a mérvadó
uv run python -c "import sys; print(sys.version.split()[0], sys.executable)"
uv run python generate_data.py
uv run python quickcheck.py
uv run python starter_experiment.py
python -m json.tool artifacts/split.json | head -n 20
```

Fontos: ezt a munkamenetet ebben az írási módban nem futtattam; a labor tényleges ellenőrzött kimenetét a mellékelt eredményjegyzék tartalmazza. Az itt látható parancsfolyam tehát a specifikáció előírt sorrendje, nem most mért futási eredmény.

A kézi kontroll a rétegekre:

```
SELECT count(*) AS n_orders
FROM read_csv_auto('data/orders.csv');
```

Várt érték: 2400. Az adatfájlra számított SHA-256-ot a `manifest.json` ugyanazon fájlhoz tartozó mezőjével hasonlítjuk össze; a hash csak akkor fogadható el, ha a manifestből kiolvasott értékkel egyezik, nem egy korábbi kényelmes másolatból írt konstans. A `sys.executable` várt alakja a projekt `.venv` alatti Python, nem a rendszer `/usr/bin/python3`.

38.4.4 Eredmény értelmezése

Ha $S = 1$, akkor állítható, hogy az adott gépen a hét réteg lánca zárt: a projekt Pythonja a projekt lockolt csomagkészletét látja, az adatellenőrzés sor- és hash-szinten átment, a split műtermék létrejött, és a leadási csomag rendelkezik az ismétléshez szükséges elemekkel. Ez nem állítja, hogy a modell illeszkedése vagy a statisztikai következtetés helyes; az a modellértékelés és a 26.4. alfejezetben leírt modellkártya dolga. A stack-teszt tehát szükséges környezeti kapu, nem elégséges tudományos minősítés.

A manifest-hash szerepe az, hogy a megváltozott bemenetet azonnal elkapja. Ha valaki újragenerálja az adatot más seeddel, a sor- és hash-invariáns megtörik, és a handoff-teszt elutasít. Így a „működött nálam” állítás helyett ellenőrzött predikátumok döntenek.

38.4.5 Feladatok

1. **Értelmezés.** A specifikáció szerint az `interpreter` és a `packages` teszt is zöld, de a Jupyter notebookban a `pandas.__version__` értéke 2.1.4, miközben a projekt Pythonja a 2.2.3-at importálja. Melyik rétegben van a hiba? Mit kell összehasonlítani, és mi a javítás?

2. **Megvalósítás.** Írj a repository gyökerébe egy `stack_probe.py` szkriptet, amely *nem* importál projektfüggő könyvtárat, és a standard kimenetre JSON-t ír a következő mezőkkel: interpreter-verzió, `sys.executable`, a `lab/pyproject.toml` és a `lab/uv.lock` megléte, valamint a `lab/data/orders.csv` mérete byte-ban. A kilépési kód legyen 0, ha az interpreter 3.12 és mindkét projektfájl létezik, különben 1.

3. **Hibadiagnózis.** Egy tiszta gépen a `uv sync --locked` parancsot a repository gyökerében futtatták, és „sikeres” üzenetet kaptak. Ezután a `uv run python starter_experiment.py` a `ModuleNotFoundError` hibát adja a `duckdb` modulra. Adj három ellenőrző parancsot, és magyarázd el, miért nem látja a projekt csomagjait a futtatás. Mit kell javítani?

4. **Megváltozott feltétel.** A forrásadatok Parquet-re váltanak, és a méret 12 GB-ra nő; a megfigyelési egység továbbra is rendelés, és a számításnak egyetlen CPU-gépen kell maradnia. Írd át a `data engine`, az `execution` és a `handoff` sor elfogadási tesztjét. Mely kritériumot kell a korábbi 34.4. alfejezetből idehozni?

38.4.6 Megoldások és indoklás

1. megoldás. A hiba a `packages` és az `execution` határán, a kernel-interpreter illesztésben van. A notebookban a `sys.executable` értéket kell összehasonlítani a `uv run python -c "import sys; print(sys.executable)"` kimenetével. Ha a kettő különbözik, a notebook nem a projekt `.venv`-jét használja. A javítás a projekt kerneljének regisztrálása – például a 3.4. alfejezetben tárgyalt kernelspec-ellenőrzéssel –, majd a notebook újraindítása. A verziók puszta kijelzése nem bizonyítja, hogy a notebook azon az interpreteren fut.

2. megoldás. Az alábbi szkript szándékosan standard library-s, hogy akkor is diagnosztizáljon, ha a függőségek nincsenek telepítve. A pontos elérési utat a labor aktuális README-jéhez kell igazítani, ha a CSV máshol fekszik.

```
from pathlib import Path
import json
import sys

def main() -> int:
    lab = Path("lab")
    pyproject = lab / "pyproject.toml"
    lock = lab / "uv.lock"
    orders = lab / "data" / "orders.csv"
    report = {
        "interpreter": {
            "version": f"{sys.version_info.major}.{sys.version_info.minor}",
            "executable": sys.executable,
            "ok": sys.version_info[:2] == (3, 12),
        },
        "packages": {
            "pyproject_exists": pyproject.exists(),
            "lock_exists": lock.exists(),
        },
        "data": {
            "orders_exists": orders.exists(),
            "orders_bytes": orders.stat().st_size if orders.exists() else None,
        },
    }
    report["packages"]["ok"] = (
        report["packages"]["pyproject_exists"]
        and report["packages"]["lock_exists"]
    )
    report["data"]["ok"] = (
        report["data"]["orders_exists"]
        and report["data"]["orders_bytes"] is not None
    )
    print(json.dumps(report, indent=2, ensure_ascii=False))
    ok = (
        report["interpreter"]["ok"]
        and report["packages"]["ok"]
        and report["data"]["ok"]
    )
    return 0 if ok else 1

if __name__ == "__main__":
    raise SystemExit(main())
```

Az ellenőrzés kimeneti sémája JSON; a várt kulcsok hiánya vagy a `false` érték egyetlen pillantással azonosítja a hibás réteget. A működést legalább egyszer a saját gépen futtatni kell; a fenti kód ellenőrizetlen sablon mindaddig, amíg a tanuló nem futtatja.

3. megoldás. A három parancs:

```
pwd
ls pyproject.toml
uv run python -c "import duckdb; print(duckdb.__version__)"
```

A hiba oka a munkakönyvtár: a `uv sync --locked` a repository gyökerében nem a `lab/pyproject.toml` lockját telepítette, ezért a `starter_experiment.py` futtatása sem a projekt virtuális környezetét látta. Az első parancs megmutatja a tényleges könyvtárat, a második azt, hogy ott nincs `pyproject.toml`, a harmadik pedig reprodukálja a hiányzó modulhibát. A javítás: `cd lab`, majd ott `uv sync --locked`, végül `uv run python starter_experiment.py`.

A „sikeres” szöveg önmagában nem állapotleírás; a futtatókönyvtárral és az interpreterrel együtt értelmezendő.

4. megoldás. A három módosított sor:

- `data engine`: DuckDB `read_parquet` vagy Polars lazy; a teljes pandas-betöltés megszűnik kötelező lépésként. Elfogadási teszt: a Parquet `COUNT(*)` egyezik a manifesttel, és a séma ellenőrzése nem igényel teljes memóriába olvasást.
- `execution`: `uv run` méréssel kiegészítve; kötelező a csúcs-RSS és a futásidő rögzítése. Elfogadási teszt: a mért csúcs-RSS a csapat által előre rögzített korlát alatt marad, és nem lép fel out-of-memory.
- `handoff`: partíciószintű manifest hash és sorcount a teljes fájl szintje mellett. Elfogadási teszt: a tiszta gépi ismétlés partíciónként is visszaadja a hash- és count-invariánst.

A 34.4. alfejezet költségmodelljéből ide tartozik az adatelhelyezés és az amortizációs kérdés: 12 GB helyi Parquet esetén a helyi DuckDB/Polars akkor marad ésszerű, ha a compute az adat közelében van, és a feladat egyszeri nagy szkennelés helyett több lépcsőben újrafelhasználható checkpointot képez. A fenti küszöbök nem mért értékek; a projekt saját gépi mérését kell a specifikáció forrásaként feltüntetni.

38.4.7 Gyakori tévedés és ellenpélda

A tipikus hiba az, hogy a „minden eszköz telepítve” lista helyettesíti az architektúrát. Ellenpélda: a terminálban a `python -c "import duckdb"` sikeres, a notebookban az `import duckdb` mégsem fut. A felhasználó a rendszer-Python `pip list` kimenetéből tudja, hogy „a DuckDB telepítve van”, és elfogadja a munkaasztalt. A notebook kernelje azonban másik interpreterre mutat, így a csomagok halmaza nem az, amelyet a rendszerhéj listáz. A helyes állítás nem az, hogy „minden név jelen van”, hanem az, hogy az $S = 1$ predikátum a projekt `.venv`-jében, a lockon keresztül, a kernel ugyanazon `sys.executable` értékével teljesül. Ez a 38.3. alfejezetben tanult ellenőrzött forrás–dátum–állapot elvet ismétli: a puszta képernyőlista nem bizonyíték.

Önellenőrzés a leadott lapra: mind a hét sorban megtalálható-e kötelező választás, alternatíva és elfogadási teszt; a teszt megad-e belépő adatot és várt invariánst; a handoff-sor tartalmaz-e hash- és count-ellenőrzést; és a lap fejléce rögzíti-e a megfigyelési egységet és a cutoffot?

38.4.8 Rövid összefoglaló

A saját data science munkaasztal nem csomaglista, hanem hét rétegből álló predikátumlánc: interpreter, packages, data engine, tracking, execution, reporting, handoff. A Northstar laborhoz a minimális választás CPython 3.12, uv lock és sync, pandas/DuckDB/Parquet, split-manifest, helyi CPU-s uv run, notebook és modellkártya, valamint readme-lock-manifest runbook. Az egyoldalas specifikáció soronként kötelező választást, alternatívát és elfogadási tesztet ad; a lap $S = 1$ esetben zöld, de ez csak környezeti kapu, nem modellminősítés. Következőként a 39.1. alfejezet záróprojekt ezen a specifikáción fagyasztja be a prediktív projekt adatszerződését.

39 Zároprojekt I: időhelyes prediktív modell a rendelés pillanatában

39.1 Probléma és adatszerződés befagyasztása

> **Belépő · szint: haladó · idő: 90 perc · eszköz: scikit-learn, pandas**
>
> Előfeltétel: 38.4. alfejezet · Célkészség: Probléma és adatszerződés befagyasztása · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.
>
> **Érvényességi kapu:** egy predikciós szerződés akkor ellenőrizhető, ha minden mezőhöz rögzíti a forrását, a rendelkezésre állási időpontját, a mértékegységét és a nullértékek kezelését; a tanító- és tesztablakot az eredmények megtekintése előtt kell meghatározni.

A zároprojekt első lépése szándékosan nem modellt épít: a problémát és az adatszerződést fagyasztja be. A befagyasztás egy verziózott adatlapot (problem contract) és egy split manifestet jelent, amelyek nélkül a későbbi AUC, log loss vagy feature-fontosság számok nem lennének összehasonlíthatók. A gyakorlatban ez két fájl: `contract.json` és `split.json`, mindkettő Git-kommitban.

39.1.1 Cél és konkrét helyzet

A Northstar egy szintetikus webáruház, amely minden rendeléskor 7 napos kézbesítést ígér. A logisztikai csapat a rendelés pillanatában szeretné tudni, hogy az adott rendelés várhatóan késni fog-e, hogy proaktívan szólhasson az ügyfélnek vagy kapacitást csoportosíthasson át. A predikció batchben fut, röviddel a rendelés után; ez `order_ts`-hez közeli, de attól elkülönült futási időpontot jelent.

Bemenet: a tisztított `lab/data/orders.csv` és a `lab/data/customers.csv`. A szándékosan hibás `orders_raw.csv` nem bemenet; annak tisztítása a 13.4. alfejezetben megtörtént, és a tiszta `orders.csv` a kontroll. A split kialakításához csak az `orders.csv` kell; a teljes feature-tábla a 8.4. alfejezetben előállított 11 oszlopos sémával dolgozik majd tovább.

Megfigyelési egység: egy rendelés (`order_id`), nem tétel és nem ügyfél. A cél bináris, rendelésszintű. A tételszintű tábla csak aggregálás után kapcsolódhat a rendeléshez, különben a cél és a szállítási díj többszöröződne (7.2. alfejezet).

Cél: a modell a rendelés pillanatában ismert mezőkből a `late = 1` esemény valószínűségét becsülje, ahol a késés azt jelenti, hogy `delivery_days > promised_days`, tehát `delivery_days > 7`.

Előzetes eredménybecslés: a split létrehozása után a tanítóhalmaz késési arányát (prevalenciáját) számítjuk ki. Ez lesz a baseline: ha például $\hat{p} = 0.25$, akkor egy mindig 0.25 valószínűséget adó konstans modell log loss-ja $L_{\text{ref}} = -[\hat{p}\ln\hat{p} + (1-\hat{p})\ln(1-\hat{p})]$; ezt kell a valódi modelleknek megverniük. A pontos $\hat{p}$ érték a futtatásból derül ki, ezért itt nem adunk előre számot.

Elvárt ellenőrzés: a három split halmaz páronként diszjunkt, együtt lefedi az összes rendelést; a tanítóban nincs cutoff utáni címke; a tiltott mezők egyike sem jelenik meg a prediktorlistában.

39.1.2 Fogalmi magyarázat: label, cutoff, observation unit, feature availability, split manifest

Label (célváltozó). Az i rendelésre D_i a tényleges kézbesítési napok száma, pozitív egész nap. A késést az

$$Y_i = \mathbf{1}\{D_i > 7\}$$

indikátor adja: $Y_i = 1$, ha késik, és 0, ha nem. A predikció időpontja t_i az `order_ts`. A címke elérhetőségének időpontja ℓ_i a `label_available_at`, amely a kezdőlaborban megegyezik az `actual_delivery_at` értékével. A predikció pillanatában Y_i nem ismert; csak ℓ_i elérése után válik ismertté. A pozitív eset egyes célokhoz korábban is felismerhető lenne, de ez más megfigyelési szerződés; itt konzervatívan a teljes kézbesítést tekintjük címke-időpontnak.

Cutoff (időbeli vágás). A c cutoff itt `2025-05-01T00:00:00+00:00`, ISO 8601, UTC. A modellépítés időpontját modellezi. Három halmazt képezünk:

- **train:** $t_i < c$ és $\ell_i < c$;
- **test:** $t_i \geq c$;
- **purged (kihagyott határsáv):** $t_i < c$ és $\ell_i \geq c$.

A három feltétel páronként kizáró és együtt teljes partíció. A purged azért kell, mert vannak olyan rendelések, amelyeket a cutoff előtt feladtak ugyan, de a címkéjük a modellépítéskor még nem érkezett meg. Ezeket sem tanítani, sem tesztelni nem szabad: a teszt a jövőbeli rendeléseket utánozza, ezért csak $t_i \geq c$ sorokból állhat. A purged sorok kimaradnak; ez adatveszteség, de a korrekt időbeli validáció ára.

Observation unit. A predikció és az értékelés egysége egy rendelés. Egy rendeléshez egyetlen Y_i tartozik. Az ügyfélszintű aggregálás későbbi szerződés lehet, de a primer modell nem ügyfélszintű. Ha a tételtáblát rendeléshez joinoljuk, előbb aggregálni kell, mert a `line_total_minor` összegezhető, de a `shipping_fee_minor` rendelésenként egyszer szerepel.

Feature availability. Egy $x_{i,k}$ mező akkor megengedett prediktor, ha értéke a t_i pillanatban ténylegesen ismert. A befagyasztott lista nyolc mezője ezt teljesíti: `distance_km`, `priority_flag`, `basket_minor`, `quantity_sum` a rendelés pillanatában adott; `account_age_days` az `order_ts` és a korábban ismert `signup_date` különbsége; `segment`, `region`, `acquisition_channel` statikus ügyféljellemzők. A tiltott prediktorok: `delivery_days`, `actual_delivery_at`, `label_available_at`, `target_late`, valamint ezek bármely származéka. Ez utóbbi kitétel miatt például egy „ügyfél korábbi rendeléseinek átlagos késési ideje” sem használható, még akkor sem, ha az adott előzmény időben elvileg elérhető lenne.

Split manifest. A manifest egy verziózott fájl, amely minden `order_id`-hoz hozzárendeli a splitet. A későbbi kódok nem véletlenszerűen vagy újraszámolva hoznak létre splitet, hanem ezt a fájlt olvassák. Ez megakadályozza, hogy egy apró scriptmódosítás eltérő tanító- és teszthalmazt produkáljon. A manifest tartalmazza a cutoffot, a split-szabályt, a forrásfájl hash-ét, a generáló script verzióját és a sorokat.

39.1.2.1 Sémaváltozási politika és a változatlan predikciós idő

A befagyasztott szerződés nem azt jelenti, hogy a séma örökre változatlan; azt jelenti, hogy minden változás **explicit, verziózott és ellenőrzött**. A `contract.json` hordozza a `contract_version` és `schema_sha256` mezőt; a futás elején a bemenet séma-hashjének egyeznie kell, különben fail-fast.

Változás	Osztály	Teendő
új opcionális, `nullable: true` mező, amely a predikció pillanatában ismert	minor	új contract_version, visszafelé kompatibilis migration
mező átnevezése	major	új contract_version, régi→új mapping, mindkét oldal tesztje
mértékegység vagy időzóna változása	major	migráció, érték-átalakítás, numerikus kontroll
új mező, amely csak a predikció után ismert	tiltott prediktorként	nem kerülhet a feature-listába; legfeljebb monitoring
mező törlése	major	új contract_version, minden fogyasztó frissítése

A **migráció** csak akkor megengedett, ha van `migration_id`, dokumentált leképezés és előtte-utána validáció. A csendes típuskonverzió (például `str → float`) tilos; ha a bemenet nem felel meg a szerződésnek, a folyamat álljon le, ne „javítsa” a mezőt.

A **predikciós idő** a séma változása alatt is változatlan: továbbra is `order_ts`. Egy új mező akkor sem lehet prediktor, ha kényelmes lenne, de csak később válik ismertté; a `feature_available_at <= order_ts` feltétel a szerződés része marad. A sémaverzió emelése nem emelheti át a predikciós időt egy későbbi pontra, mert azzal a modell a döntés pillanatában nem ismert információt kapna.

```
def check_contract(input_schema_sha256, contract, migration_id=None):
    if input_schema_sha256 != contract["schema_sha256"]:
        if migration_id is None:
            raise RuntimeError("schema mismatch: fail-fast")
        if migration_id not in contract["allowed_migrations"]:
            raise RuntimeError("nem engedélyezett migráció")
    if contract["prediction_as_of"] != "order_ts":
        raise RuntimeError("a predikciós idő nem változhat")
    return True
```

A `tests/test_schema_change_policy.py` a fail-fastet, az engedélyezett migrációt, a mértékegység-változás major osztályát és a predikciós idő változatlanságát teszteli.

39.1.3 Teljes minimális példa

39.1.3.1 Kézi ellenőrzés

Legyen a cutoff `2025-05-01T00:00:00+00:00`. Nyolc rendelés:

order_id	order_ts	label_available_at	delivery_days
A	2025-04-28 10:00	2025-05-02 09:00	4

order_id	order_ts	label_available_at	delivery_days
B	2025-04-20 08:00	2025-04-27 10:00	7
C	2025-05-02 14:00	2025-05-10 16:00	8
D	2025-04-15 09:00	2025-04-25 08:00	10
E	2025-05-10 07:00	2025-05-18 12:00	8
F	2025-04-30 11:00	2025-05-04 10:00	4
G	2025-04-25 13:00	2025-05-09 11:00	14
H	2025-04-18 16:00	2025-04-24 17:00	6

A besorolás:

- **A:** $t < c$, de $\ell \geq c$, tehát **purged**. Annak ellenére, hogy nem késett, a címkéje cutoffkor nem volt meg.
- **B:** $t < c$ és $\ell < c$, tehát **train**, $Y = 0$, mert $D = 7$ nem nagyobb 7-nél.
- **C:** $t \geq c$, tehát **test**; a címkéjét a tesztértékeléshez később használjuk, de a modellépítéshez nem.
- **D: train**, $Y = 1$, mert $D = 10 > 7$.
- **E: test**.
- **F:** $t < c$, $\ell \geq c$, **purged**.
- **G:** $t < c$, $\ell \geq c$, **purged**. A 14 napos kézbesítés pozitív címke lenne, de cutoffkor még nem ismert, ezért nem tanulhatunk rá.
- **H: train**, $Y = 0$, mert $D = 6 \leq 7$.

A tanítóhalmaz három sorból áll (B, D, H), a késések száma $n_{\text{late}} = 1$, tehát a tanító-prevalencia $\hat{p} = 1/3 \approx 0.333$. Ez a kézi példa aránya; a teljes Northstar adaton a tényleges érték a futtatásból adódik.

39.1.3.2 DuckDB + pandas split-manifest generálás

Az alábbi script szemléltető, nem futtatott kimenettel; a tényleges futást a lab környezetben a `uv run` parancs végzi. A script a paraméterezést a `?` placeholderrel végzi a cutoffra (7.4. alfejezet), a fájlnevet konstansként kezeli, és a manifestet a `lab/artifacts/split.json` útvonalra írja.

```
# scripts/freeze_split.py -- szemléltető, nem futtatott
from pathlib import Path
import json
import hashlib
import duckdb
import pandas as pd

CUTOFF = pd.Timestamp("2025-05-01T00:00:00+00:00")
ORDERS = Path("lab/data/orders.csv")
OUT = Path("lab/artifacts/split.json")

split_sql = """
SELECT
    order_id,
    order_ts::TIMESTAMPTZ AS order_ts,
    label_available_at::TIMESTAMPTZ AS label_available_at,
    CASE
        WHEN order_ts < ? AND label_available_at < ? THEN 'train'
        WHEN order_ts >= ? THEN 'test'
        ELSE 'purged'
    END AS split
```

```
FROM read_csv('lab/data/orders.csv', header = true, auto_detect = true)
ORDER BY order_id
"""

conn = duckdb.connect()
manifest = conn.execute(split_sql, [CUTOFF, CUTOFF, CUTOFF]).fetchdf()
manifest["order_ts"] = pd.to_datetime(manifest["order_ts"], utc=True)
manifest["label_available_at"] = pd.to_datetime(
    manifest["label_available_at"], utc=True
)

# Invariánsok
train = manifest[manifest["split"] == "train"]
test = manifest[manifest["split"] == "test"]
purged = manifest[manifest["split"] == "purged"]

assert manifest["order_id"].is_unique
assert set(manifest["split"].unique()) <= {"train", "test", "purged"}
assert len(manifest) == len(train) + len(test) + len(purged)
assert (train["order_ts"] < CUTOFF).all()
assert (train["label_available_at"] < CUTOFF).all()
assert (test["order_ts"] >= CUTOFF).all()
assert (purged["order_ts"] < CUTOFF).all()
assert (purged["label_available_at"] >= CUTOFF).all()

source_hash = hashlib.sha256(ORDERS.read_bytes()).hexdigest()

payload = {
    "schema_version": "1.0.0",
    "contract_id": "northstar-late-v1",
    "cutoff": CUTOFF.isoformat(),
    "observation_unit": "order_id",
    "split_rule": {
        "train": "order_ts < cutoff AND label_available_at < cutoff",
        "test": "order_ts >= cutoff",
        "purged": "order_ts < cutoff AND label_available_at >= cutoff",
    },
    "source_sha256": source_hash,
    "row_counts": {
        "train": int(len(train)),
        "test": int(len(test)),
        "purged": int(len(purged)),
    },
    "order_ids": manifest.to_dict(orient="records"),
}

OUT.parent.mkdir(parents=True, exist_ok=True)
OUT.write_text(json.dumps(payload, indent=2))
print(payload["row_counts"])
```

A futási parancs Linux/macOS terminálból, a projektgyökérből:

```
uv run python scripts/freeze_split.py
```

A kód kimenete a tényleges sorokkal töltődik fel; ebben a szövegben csak a szerkezet ellenőrzött. A lab `manifest.json` a mellékelt ellenőrző jegyzék szerint mérvadó.

39.1.3.3 A befagyasztott adatlap

A fenti split-szabállyal együtt a projekt adatlapja:

```
{
  "contract_id": "northstar-late-v1",
  "frozen_at": "2026-10-20",
  "observation_unit": "order",
  "prediction_timestamp": "order_ts",
  "promised_days": 7,
  "target": {
    "name": "late",
    "positive_definition": "delivery_days > promised_days",
    "positive_class": 1,
    "label_available_at": "actual_delivery_at"
  },
  "cutoff": "2025-05-01T00:00:00+00:00",
  "split_rule": {
    "train": "order_ts < cutoff AND label_available_at < cutoff",
    "test": "order_ts >= cutoff",
    "purged": "order_ts < cutoff AND label_available_at >= cutoff"
  },
  "allowed_features": [
    "distance_km",
    "priority_flag",
    "basket_minor",
    "quantity_sum",
    "account_age_days",
    "segment",
    "region",
    "acquisition_channel"
  ],
  "forbidden_predictors": [
    "delivery_days",
    "actual_delivery_at",
    "label_available_at",
    "target_late",
    "bármely ezekből származtatott oszlop predictor szerepben"
  ],
  "notes": [
    "Pénzösszegek EUR-ben, minor unitban (cent); nincs pénznemkonverzió.",
    "A distance_km hiányzó értéke nem 0; a tisztított orders.csv-ben nincs hiány.",
    "A label_available_at a teljes kézbesítés; a pozitív eset korábbi felismerése új szerződést
    ↪  igényel."
  ]
}
```

Ez a dokumentum a modellépítés belépési kapuja. A `contract_id = northstar-late-v1` bármely módosítása új verziót, például `v2`-t igényel, és a régi futásokkal nem hasonlítható össze automatikusan.

39.1.4 Eredmény értelmezése

A kézi példában a train-prevalencia $1/3 \approx 0.333$, a purged három sorból áll, a test két sorból. Az időbeli feltételek tehát nem egyszerűen „cutoff előtt train, utána test” felosztást adnak; a purged határsáv a címke elérhetőségét is figyelembe veszi.

A DuckDB-példa invariánsai a manifestre:

- `order_id` egyedi, tehát egy rendelés csak egyszer szerepel;
- a split értékek a három megengedett értékből kerülnek ki;
- a három halmaz elemszáma összegződik a teljes sorokra;
- a train minden sorában `order_ts < cutoff` és `label_available_at < cutoff`;
- a test minden sorában `order_ts >= cutoff`;

- a purged minden sorában `order_ts < cutoff` és `label_available_at >= cutoff`.

Ezek nem toleranciás, hanem egzakt feltételek: az időbélyegeket UTC-ben hasonlítjuk, ezért a határon nincs kerekítési szabadság. A JSON a `source_sha256` mezővel rögzíti, melyik `orders.csv`-ből készült; ha a fájl később változik, a hash eltérése azonnal jelzi, hogy a manifestet újra kell generálni. A split manifestet a Git commit rögzíti, a későbbi modellkísérletek ezt olvassák be ahelyett, hogy újrafuttatnák a split-szabályt. Ez a 30.3. alfejezetben tanult bemeneti hash-gondolatot ismétli: a split manifest a modelltanítás egyik legfontosabb bemenete.

39.1.5 Feladatok

1. **Értelmezés.** Az alábbi táblában $c = 2025\text{-}05\text{-}01T00:00Z$. Töltsd ki a split oszlopot minden sorra, add meg a train késési arányát, és indokold, miért nem tanulhat a modell a G rendelés `delivery_days=14` értékéből.

2. **Megvalósítás.** Írj egy `assign_split(order_ts, label_available_at, cutoff)` függvényt, amely a fenti szabály szerint `"train"`, `"test"` vagy `"purged"` értéket ad, majd alkalmazd a táblázat minden sorára. Ellenőrizd a B, G és E sorokat.

3. **Hibakeresés.** Egy adathiány miatt kézzel javított manifestben a következő sorok szerepelnek:

   - `{"order_id": "X1", "split": "train", "order_ts": "2025-04-28", "label_available_at"`
     `"2025-05-02"}`
   - `{"order_id": "X2", "split": "test", "order_ts": "2025-04-30", "label_available_at":`
     `"2025-05-04"}`

   A cutoff változatlanul 2025-05-01. Melyik sor hibás és miért? Mi lehet a következménye mindkét hibának?

4. **Megváltoztatott feltétel.** Változtasd a cutoffot `2025-05-03T00:00:00+00:00`-ra az eredeti nyolcsoros táblázatban. Számítsd ki újra a splitet és a train-prevalenciát. Melyik sor kerül át másik halmazba?

5. **Feature-availability audit.** Egy munkatárs a modellhez javasolja a `customer_previous_late_rate` mezőt: az ügyfél korábbi rendelései közül a késettek aránya a cutoff előtt. Döntsd el a befagyasztott adatlap alapján, hogy használható-e, és indokold. Fogalmazz meg egy olyan alternatív előzmény-mezőt, amely a jelen szerződésben is megengedett lehet, vagy indokold, miért kell új szerződésverzió.

39.1.6 Megoldások és indoklás

1. feladat. A helyes split:

order_id	split	indoklás
A	purged	$t < c$, de $\ell \geq c$
B	train	$t < c$ és $\ell < c$; $D = 7$, tehát $Y = 0$
C	test	$t \geq c$
D	train	$t < c$, $\ell < c$; $D = 10$, $Y = 1$
E	test	$t \geq c$
F	purged	$t < c$, $\ell \geq c$
G	purged	$t < c$, $\ell \geq c$

order_id	split	indoklás
H	train	$t < c$, $\ell < c$; $D = 6$, $Y = 0$

A train sorok B, D, H; $n = 3$, $n_{\text{late}} = 1$, tehát $\hat{p} = 1/3$. A G rendelés 14 napos kézbesítése pozitív címke lenne, de a modellépítés időpontjában, a cutoffkor ez az információ még nem érkezett meg, mert `label_available_at = 2025-05-09`. Ha tanítósorba tennénk, a modell olyan címkét látna, amely a valódi predikciós pillanatban nem állt rendelkezésre; ez jövőbeli információ kiszivárgása lenne. A test sem jó, mert a rendelés időpontja a cutoff előtti, tehát nem modellezi a jövőbeli rendeléseket.

2. feladat. Egy lehetséges implementáció:

```
def assign_split(order_ts, label_available_at, cutoff):
    if order_ts < cutoff:
        if label_available_at < cutoff:
            return "train"
        return "purged"
    return "test"
```

Ellenőrzés B-re: `order_ts=2025-04-20 < cutoff` és `label_available_at=2025-04-27 < cutoff`, tehát `"train"`. G-re: `order_ts < cutoff`, de `label_available_at >= cutoff`, tehát `"purged"`. E-re: `order_ts >= cutoff`, tehát `"test"`. A függvény a három feltételt a rövidzár-kiértékelés miatt páronként kizáróan adja.

3. feladat. Mindkét sor hibás. X1-re `order_ts < cutoff`, de `label_available_at >= cutoff`, tehát a splitnek `purged`-nek kellene lennie, nem `train`-nek. X2-re ugyanez a helyzet: `order_ts < cutoff` és `label_available_at >= cutoff`, tehát `purged`, nem `test`. Az X1 tanítósorba kerülése jövőbeli címke-információt szivárogtat a modellbe, ami optimistán torzítja a validációs és teszteredményt. Az X2 tesztsorba kerülése viszont múltbeli rendelést kever a jövőbeli tesztbe, ami nem a valós batch-feltételeket méri, ezért a teszt AUC/log loss nem lesz érvényes becslése a jövőbeli teljesítménynek. A javítás: mindkét sor kerüljön `purged`-be, vagy a manifestet újra kell generálni a scriptből.

4. feladat. Az új $c = 2025\text{-}05\text{-}03$. Ekkor:

- A: $t = 2025\text{-}04\text{-}28 < c$, $\ell = 2025\text{-}05\text{-}02 < c$, tehát ez a sor **train** lesz (korábban purged volt); $D = 4$, $Y = 0$.
- B: train, $Y = 0$.
- C: $t = 2025\text{-}05\text{-}02 < c$, $\ell = 2025\text{-}05\text{-}10 \geq c$, tehát **purged** (korábban test volt).
- D: train, $Y = 1$.
- E: $t = 2025\text{-}05\text{-}10 \geq c$, **test**.
- F: $t < c$, $\ell = 2025\text{-}05\text{-}04 \geq c$, **purged**.
- G: $t < c$, $\ell = 2025\text{-}05\text{-}09 \geq c$, **purged**.
- H: train, $Y = 0$.

A train sorok: A, B, D, H; $n = 4$, $n_{\text{late}} = 1$, tehát $\hat{p} = 1/4 = 0.25$. A példa mutatja, hogy a cutoff eltolása nem egyszerűen áthelyezi a határt, hanem a purged/test viszonyt is átírja: C a testből purgedbe került, A pedig purgedből trainbe.

5. feladat. A `customer_previous_late_rate` a `delivery_days` múltbeli értékeiből képzett szám, tehát a befagyasztott adatlap tiltott prediktora, függetlenül attól, hogy időben elvileg elérhető lenne. A tiltólista ugyanis nemcsak a jövőbeli, hanem a `delivery_days` bármely származékát kizárja. Ha ilyen mezőt szeretnénk, új szerződésverzió kell, amely pontosan definiálja a leade time-ot, a címke elérhetőségét és a batch-predikciós utánpótlást. A jelen szerződésben

megengedett előzmény-alternatíva lehet az ügyfél korábbi rendeléseinek száma az `order_ts` előtt, mert az `order_ts` sorozat rendeléskor ismert, és nem a célból származik. Ugyanakkor a befagyasztott nyolcmezős lista ezt sem tartalmazza; annak felvétele is új feature-indoklást és szerződésverziót igényelne.

39.1.7 Gyakori tévedés, visszautalás és önellenőrzés

A leggyakoribb hiba azt hinni, hogy a `delivery_days` közvetett használata ártatlan, ha az érték a predikciós időpontnál régebbi. Egy ellenpélda: az „ügyfél korábbi rendeléseinek átlagos `delivery_days`e" minden számítási lépésben időben elérhető lehet, mégis tiltott a jelen szerződésben, mert a befagyasztott lista a `delivery_days` minden származékát kizárja. Ráadásul a batch-kiszolgálásnál (35.1. alfejezet) külön kellene igazolni, hogy a korábbi címkék ténylegesen megérkeztek a modell futásáig, és a 25.3. alfejezetben látott szelekciós optimizmus is fenyegetne. A célváltozó múltbeli értékeiből képzett prediktor a modellkártya (26.4. alfejezet) „nem támogatott használat" mezőjébe tartozna, amíg új szerződés nem írja le.

Visszautalva a korábbi fogalmakra: a split-manifest a 8.4. alfejezetben előállított feature-tábla fölé épül, de a címkét csak a split után szabad csatolni. A predikciós szerződés (21.4. alfejezet) sémaellenőrzője fogja később garantálni, hogy a betanított pipeline ugyanezt a nyolc mezőt várja. A befagyasztás a 31.4. alfejezetből ismert registry-diszciplína projekt előtti megfelelője: az adminisztratív verzió és a tudományos minőség külön állítás, ezért a `contract_id` és a modellmetrikák soha nem keverhetők egy sorba.

Önellenőrzés. Válaszolj röviden: (1) Mi a predikciós időbélyeg, és mi a címke elérhetősége? (2) Miért nem lehet a G rendelés a trainben a kézi példában? (3) Melyik feltétel teszi a három splitet páronként kizáróvá? (4) Miért nem elég a cutoff előtti/utáni felosztás?

39.1.8 Rövid összefoglaló

A befagyasztott adatlap rögzíti, hogy a Northstar késésprojekt rendelésszintű bináris célt becsül a rendelés pillanatában. A címke $Y = \mathbf{1}\{delivery_days > 7\}$, és csak a `label_available_at` időpontban ismert. A `2025-05-01` cutoff három diszjunkt halmazt ad: tanító, teszt és purged határsáv. Prediktorként csak a rendeléskor ismert nyolc mező használható; a `delivery_days`, `actual_delivery_at`, `label_available_at`, `target_late` és származékaik tiltottak. A split manifest verziózott, forrás-hash-sel ellátott fájl. A következő alkalmazás a 39.2. alfejezet: ebből a manifestből építjük a feature-pipeline-t, a baseline-t és az első tanítást.

39.2 Feature-pipeline, baseline és tanulás

> **Belépő · szint: haladó · idő: 90 perc · eszköz: scikit-learn, pandas**
>
> Előfeltétel: 39.1. alfejezet · Célkészség: Feature-pipeline, baseline és tanulás · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

A 39.1. alfejezetben befagyasztottuk a Northstar predikciós szerződést: a megfigyelési egység a rendelés, a predikció pillanata az `order_ts`, a célváltozó a `target_late` (1, ha `delivery_days > promised_days = 7`). Ebben az alfejezetben ellenőrzött joinnal feature-táblát építünk, két modelljelöltet futtatunk rögzített költségkerettel, és a tanulást úgy szervezzük, hogy a preprocessing minden CV-fold tanítórészén belül tanuljon. Az itt közölt kód **nem futtatott** ebben a generálási módban; a számszerű kimeneteket „szemléltető" jelöléssel kezeljük. A tényleges mérés a te gépeden, a `uv run` paranccsal történik.

39.2.1 Cél és konkrét helyzet

Bemenet: `lab/data/orders.csv` (2400 sor) és `lab/data/customers.csv` (300 sor). A predikciós idő a rendelés időpontja; a modell csak az ekkor ismert mezőket kaphatja. A 39.1. alfejezetben rögzített feature-lista:

```
distance_km, priority_flag, basket_minor, quantity_sum, account_age_days,
segment, region, acquisition_channel
```

A célváltozó `target_late`; a `delivery_days`, `actual_delivery_at` és `label_available_at` kizárt predictor. A felosztás: tanító `(order_ts < cutoff) ÉS (label_available_at < cutoff)`; teszt `order_ts >= cutoff`; purged határsáv a többi. Az alap cutoff `2025-05-01T00:00:00+00:00`.

Előzetes becslés: ha a késési arány a tanítóban körülbelül 25%, akkor a mindig negatív dummy pontossága körülbelül 0.75, log lossa

$$-(0.25 \ln 0.25 + 0.75 \ln 0.75) \approx 0.562.$$

Egy hasznos modelltől a teszt ROC AUC legalább 0.60–0.65 körüli értéket várunk; a tényleges szám a futásból derül ki.

39.2.2 Séma és ellenőrzött join

A **séma** (schema) az oszlopnév, dtype, tartomány és egység együttes rögzítése. A betöltés után ellenőrizzük:

Oszlop	dtype	Tartomány/érték	Egység
distance_km	float64	>=0	km
priority_flag	int64	0 vagy 1	–
basket_minor	int64	>0	EUR-cent
quantity_sum	int64	>=1	darab
account_age_days	int64	>=0	nap
segment	object	consumer, business	–
region	object	north, central, south	–
acquisition_channel	object	search, referral, direct	–
target_late	int64	0 vagy 1	–

A `customers` és `orders` összekapcsolása bal oldali join, `validate="many_to_one"` kapcsolóval. A pandas ezzel a jobb oldali `customer_id`-k egyediségét ellenőrzi. Nem létező ügyfélkulcs esetén a bal oldali join alapból nem ad hibát; ezt a csatlakozás eredményén külön, például `indicator=True`-val vagy az itt alkalmazott `signup_date.notna()` ellenőrzéssel kell kimutatni. A join után a sorok száma változatlanul 2400 kell legyen. Ugyanez a join DuckDB SQL-lel is elvégezhető, de a pandas `validate` paramétere a helyi kontroll-laborban közvetlen kardinalitás-ellenőrzést ad.

39.2.3 Point-in-time: account_age_days és label availability

A **point-in-time** elv azt mondja ki, hogy minden feature értékét abban az időpontban kell ismerni, amikor a predikciót valójában futtatnánk. Az `account_age_days` ezt teljesíti:

$$a_i = \frac{t_i^{\text{order}} - t_i^{\text{signup}}}{1 \text{ nap}}$$

ahol a_i egész nap, t_i^{order} az i. rendelés UTC-időbélyege, t_i^{signup} az ügyfél regisztrációs dátuma UTC szerint. A kivonás pandas `Timedelta` objektumot ad, a `.dt.days` egész napot. A szerződés garantálja, hogy a regisztráció minden rendelés előtt van, ezért $a_i \geq 0$.

A **label availability** a tanítócímke elérhetőségét jelenti: a `label_available_at = actual_delivery_at`, tehát a címke csak a teljes kézbesítéskor válik ismertté. A tanítóba csak azok a rendelések kerülnek, ahol `label_available_at < cutoff`. A purged sáv a cutoff előtti rendelések közül azok, ahol a címke még nem érkezett meg; ezeket kihagyjuk, mert a predikciós időpontban a valós rendszer sem ismerné a kimenetüket.

39.2.4 Pipeline és foldon belüli preprocessing

A **Pipeline** a scikit-learn-ben a preprocessing lépéseket és a modellt egyetlen becslővé fűzi. A `ColumnTransformer` numerikus oszlopokra `SimpleImputer` + `StandardScaler`, kategorikus oszlopokra `SimpleImputer` + `OneHotEncoder` láncot alkalmaz. A kulcspont: ha a keresztvalidációt a teljes pipeline-on futtatjuk, a scaler átlaga, a median imputált értéke és a one-hot kódolás kategóriakészlete minden fold tanítórészéből tanul, és a fold validációs részére alkalmazzuk. Ha ezt előre, a teljes tanítóhalmazon tennénk meg, a validációs fold statisztikája beszivárogna a transzformációba.

A logisztikus regresszió predikciója:

$$p_i = \sigma(\mathbf{x}_i^\top \mathbf{w} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

ahol $\mathbf{x}_i$ a transzformált feature-vektor, shape $(p,)$; $\mathbf{w}$ a súlyvektor, shape $(p,)$; b skalár; $p_i \in (0, 1)$ a késés becsült valószínűsége. A `HistGradientBoostingClassifier` nem ezt a lineáris képletet használja, hanem döntési fák összegét; mindkét modell `predict_proba`-ja valószínűségi pontszámot ad. A logisztikus döntési határ a standardizált feature-térben lineáris, szemben a 27.2. alfejezetben tárgyalt koszinusz-hasonlósággal, amely normalizált vektorok szögét méri.

Az AUC értelmezése:

$$\text{AUC} = P(s(X_+) > s(X_-)),$$

ahol s a modell pontszáma, X_+ egy pozitív, X_- egy negatív megfigyelés. Az AUC csak a sorrendtől függ, a kalibrációtól nem; ezért a log loss-t is mérjük.

39.2.5 Baseline és controlled search

A **baseline** egy olyan egyszerű szabály, amelyhez a jelölteket mérjük. A `DummyClassifier(strategy="prior")` a tanító pozitív arányát add meg a valószínűségkimenetben; `predict()` a többségi osztályt választja. A tesztpontosság ezért a teszt többségi osztályának tényleges részarányától függ, a log loss-hoz pedig a tanítóban becsült q arány kerül. A log loss a dummy $p_i = q$ konstans esetén:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln p_i + (1-y_i)\ln(1-p_i)\right],$$

ahol q a pozitív osztály aránya, $y_i \in \{0,1\}$ a tényleges címke. A modellnek ennél kisebb log loss-t kell elérnie.

A **controlled search** (ellenőrzött keresés) rögzített fit-költségkerettel dolgozik. A példában a logisztikus grid $3 \times 2 = 6$ kombináció, a HGB grid $2 \times 2 = 4$ kombináció; 3-fold `TimeSeriesSplit` esetén a CV fitek $6 \cdot 3 + 1 = 19$ és $4 \cdot 3 + 1 = 13$, összesen 32 fit. A keret 50 fit, tehát egy további kisebb jelölt is belefér. A `scoring="roc_auc"` biztosítja, hogy a belső választás a célmetrikát optimalizálja; a tesztet a keresés nem érinti.

39.2.6 Tracking

A **tracking** a futás bemenetének, paramétereinek és kimenetének tárolása. A lab minimális JSON manifestet ír; MLflow esetén ugyanez `mlflow.log_params`, `mlflow.log_metric` és `mlflow.sklearn.log_model` hívásokkal történne. A manifest tartalmazza a cutoffot, a split darabszámokat, a feature-listát, a jelöltek legjobb paramétereit, a CV és teszt metrikákat, valamint a modell- és adatfájl SHA-256 hashét. A hash lehetővé teszi, hogy egy későbbi betöltésnél észleljük, ha az artifact megváltozott.

39.2.7 Teljes minimális munkapélda

A példa a `lab/data` útvonalat feltételezi; ha a CSV-k a lab gyökerében vannak, cseréld a `DATA_DIR` értékét `Path(".")`-ra. Helyezd a scriptet a `lab/train_model.py` fájlba, és futtasd a lab mappa gyökeréből:

```
cd lab
uv run python train_model.py --cutoff 2025-05-01T00:00:00+00:00 --artifact-dir artifacts/s154
```

PowerShell-ben ugyanez a parancs a `cd lab` után; a munkakönyvtár a script helye.

```
import argparse
import hashlib
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, log_loss, roc_auc_score
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler

DATA_DIR = Path("data")
FEATURES = [
    "distance_km", "priority_flag", "basket_minor", "quantity_sum",
    "account_age_days", "segment", "region", "acquisition_channel",
]
TARGET = "target_late"
NUM_FEATURES = ["distance_km", "priority_flag", "basket_minor", "quantity_sum",
 ↪  "account_age_days"]
CAT_FEATURES = ["segment", "region", "acquisition_channel"]


def file_hash(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--cutoff", default="2025-05-01T00:00:00+00:00")
    parser.add_argument("--artifact-dir", default="artifacts/s154")
    args = parser.parse_args()

    cutoff = pd.Timestamp(args.cutoff)
    artifact_dir = Path(args.artifact_dir)
    artifact_dir.mkdir(parents=True, exist_ok=True)

    orders = pd.read_csv(
        DATA_DIR / "orders.csv",
        parse_dates=["order_ts", "actual_delivery_at", "label_available_at"],
    )
    customers = pd.read_csv(DATA_DIR / "customers.csv", parse_dates=["signup_date"])

    # Séma-ellenőrzés a tiszta adaton
    assert orders["basket_minor"].isna().sum() == 0
    assert (orders["basket_minor"] > 0).all()
    assert (orders["distance_km"] >= 0).all()
    assert orders["priority_flag"].isin([0, 1]).all()
    assert orders["quantity_sum"].ge(1).all()
    assert orders["currency"].eq("EUR").all()

    # Ellenőrzött join: egy rendeléshez pontosan egy ügyfél
    df = orders.merge(
        customers[["customer_id", "signup_date", "segment", "region", "acquisition_channel"]],
        on="customer_id",
        how="left",
        validate="many_to_one",
    )
    assert df["signup_date"].notna().all()
    assert len(df) == len(orders)

    # Point-in-time feature
    df["signup_ts"] = pd.to_datetime(df["signup_date"]).dt.tz_localize("UTC")
    df["account_age_days"] = (df["order_ts"] - df["signup_ts"]).dt.days
    assert (df["account_age_days"] >= 0).all()

    # Split a s153 szerződés szerint
    df["split"] = "purged"
    df.loc[(df["order_ts"] < cutoff) & (df["label_available_at"] < cutoff), "split"] = "train"
    df.loc[df["order_ts"] >= cutoff, "split"] = "test"

    assert (df.loc[df["split"] == "train", "order_ts"] < cutoff).all()
    assert (df.loc[df["split"] == "train", "label_available_at"] < cutoff).all()
```

```
    assert (df.loc[df["split"] == "test", "order_ts"] >= cutoff).all()

    train = df[df["split"] == "train"].copy()
    test = df[df["split"] == "test"].copy()
    split_counts = df["split"].value_counts().to_dict()

    X_train, y_train = train[FEATURES], train[TARGET]
    X_test, y_test = test[FEATURES], test[TARGET]

    # Kizárt mezők ellenőrzése
    forbidden = {"delivery_days", "actual_delivery_at", "label_available_at", "order_id",
↪   "customer_id"}
    assert forbidden.isdisjoint(FEATURES)

    numeric_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
    categorical_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ])
    preprocessor = ColumnTransformer([
        ("num", numeric_pipe, NUM_FEATURES),
        ("cat", categorical_pipe, CAT_FEATURES),
    ])

    # Két jelölt; a preprocessing a pipeline-on belül, foldonként tanul
    candidates = {
        "logreg": Pipeline([
            ("prep", preprocessor),
            ("model", LogisticRegression(max_iter=1000, solver="lbfgs")),
        ]),
        "hgb": Pipeline([
            ("prep", preprocessor),
            ("model", HistGradientBoostingClassifier(max_iter=100, early_stopping=True,
↪   random_state=42)),
        ]),
    }
    param_grid = {
        "logreg": {
            "model__C": [0.1, 1.0, 10.0],
            "model__class_weight": [None, "balanced"],
        },
        "hgb": {
            "model__learning_rate": [0.05, 0.1],
            "model__max_iter": [50, 100],
        },
    }

    cv = TimeSeriesSplit(n_splits=3)
    searches = {}
    budget = {}
    for name, model in candidates.items():
        search = GridSearchCV(
            model,
            param_grid[name],
            cv=cv,
            scoring="roc_auc",
            refit=True,
            n_jobs=2,
            error_score="raise",
        )
```

```
    search.fit(X_train, y_train)
    searches[name] = search
    # A refit +1 fit; a CV fitek: kombinációk * foldok
    budget[name] = len(search.cv_results_["params"]) * cv.get_n_splits() + 1

# Dummy baseline
dummy = DummyClassifier(strategy="prior")
dummy.fit(X_train, y_train)

def evaluate(name, model):
    p = model.predict_proba(X_test)[:, 1]
    return {
        "model": name,
        "auc": roc_auc_score(y_test, p) if y_test.nunique() == 2 else np.nan,
        "average_precision": average_precision_score(y_test, p),
        "log_loss": log_loss(y_test, p, labels=[0, 1]),
    }

rows = [evaluate("dummy", dummy)]
rows += [
    {
        "model": name,
        "auc": evaluate(name, search.best_estimator_)["auc"],
        "average_precision": evaluate(name, search.best_estimator_)["average_precision"],
        "log_loss": evaluate(name, search.best_estimator_)["log_loss"],
        "cv_mean_auc": search.best_score_,
        "best_params": search.best_params_,
    }
    for name, search in searches.items()
]
metrics = pd.DataFrame(rows)
print(metrics.to_string(index=False))

# A CV szerinti legjobb jelölt mentése; a teszt AUC utólagos ellenőrzés
best_name = max(searches, key=lambda name: searches[name].best_score_)
best = searches[best_name].best_estimator_
model_path = artifact_dir / "model.joblib"
joblib.dump(best, model_path)

# Betöltési integritás
loaded = joblib.load(model_path)
np.testing.assert_allclose(
    loaded.predict_proba(X_test.iloc[:10]),
    best.predict_proba(X_test.iloc[:10]),
    rtol=1e-6,
)

manifest = {
    "run_id": "s154-local-demo",
    "cutoff": cutoff.isoformat(),
    "split_counts": split_counts,
    "features": FEATURES,
    "target": TARGET,
    "fit_budget": budget,
    "selected_candidate": best_name,
    "candidates": {
        name: {
            "best_params": search.best_params_,
            "cv_mean_auc": search.best_score_,
            "test_auc": evaluate(name, search.best_estimator_)["auc"],
            "test_log_loss": evaluate(name, search.best_estimator_)["log_loss"],
        }
        for name, search in searches.items()
```

```
        },
        "model_sha256": file_hash(model_path),
        "orders_csv_sha256": file_hash(DATA_DIR / "orders.csv"),
        "customers_csv_sha256": file_hash(DATA_DIR / "customers.csv"),
    }
    manifest_path = artifact_dir / "manifest.json"
    manifest_path.write_text(json.dumps(manifest, indent=2))
    print(f"Wrote {model_path} and {manifest_path}")
    print(f"Split counts: {split_counts}")


if __name__ == "__main__":
    main()
```

Az alábbi kimenet **szemléltető**, nem rögzített futási jegyzékből származik; a te számaid eltérhetnek:

```
Split counts: {'train': 1512, 'test': 708, 'purged': 180}
model  auc    average_precision  log_loss  cv_mean_auc  best_params
dummy  0.500  0.234              0.612     NaN          NaN
logreg 0.619  0.351              0.589     0.608        {'model__C': 0.1,
↪  'model__class_weight': None}
hgb    0.655  0.372              0.581     0.641        {'model__learning_rate': 0.1,
↪  'model__max_iter': 100}
```

39.2.8 Eredmény értelmezése és ellenőrző invariánsok

A kézi ellenőrzés négy invariant köré épül:

1. `len(df) == len(orders)`, tehát a join nem szaporította a sorokat.
2. `train + test + purged = len(df)`, és a train minden sorára `label_available_at < cutoff`.
3. `forbidden.isdisjoint(FEATURES)`, tehát a címke és a későbbi időbélyegek nem szerepelnek predictorban.
4. A mentett modell `predict_proba`-ja megegyezik a memóriában lévő modellével `1e-6` relatív toleranción belül.

Az AUC 0.5 a véletlen szint; a dummy AUC pontosan 0.5, ha mindkét osztály jelen van a tesztben. A 0.60 körüli AUC szerény, a 0.65 már hasznos jelzés lehet, de a döntést a log loss és a gyakorlati küszöb alapján kell meghozni. A teszt AUC egyetlen splitből származó pontbecslés; a bizonytalanság becsléséhez bootstrap vagy nested CV kellene, amit a 23.2. alfejezet és 23.3. alfejezet tárgyal. A CV-foldok szórása nem konfidenciaintervallum.

A fit-költség a manifest `fit_budget` mezőjében 32, a tervezett kereten belül. A HGB early stopping a falidőt csökkentheti, de a számlált fit-ek számát nem változtatja. A méréshez rögzítsd a gépet, a sorok számát és a futásidőt; e példában a 2400 soros Northstar adat a 38.2. alfejezet döntési lap szerint CPU-s, kis munkaterhelés, de a tényleges időt a te gépeden kell megmérned. A 35.2. alfejezetben tanult predikciós szerződés itt batch CLI módban, a `--cutoff` és `--artifact-dir` argumentumokkal fut.

39.2.8.1 Zöld CI pipeline és fit-log

A fejezet számszerű AUC-ja a saját futásodtól függ, ezért a publikálás feltétele egy **zöld CI futás**, amely a következőket naplózza:

Naplómező	Példa	Miért kell
`split_id`, `split_sha256`	`northstar-final-2025-05-01`, SHA-256	a meghatározott split azonosítása
`fit_counts`	`{"logreg": 19, "hgb": 13}`	a CV-fitek és a refit külön számlálva
`fit_budget`	32	a tervezett keret
`refit`	`true`	a 13. fit a HGB refitje
`metrics`	ROC AUC, log loss, Brier	a tényleges futás eredménye
`artifact_sha256`	modell, split, manifest	a kimenet integritása
`environment`	Python, sklearn, numpy, OS	a futás környezete

A CI-lépések:

```yaml
jobs:
  final-pipeline:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: uv sync --locked
      - run: >-
          uv run python -m northstar.cli split
          --cutoff 2025-05-01T00:00:00+00:00
          --out artifacts/split.json
      - run: >-
          uv run python -m northstar.cli train
          --split artifacts/split.json
          --out artifacts/run.json
      - run: >-
          uv run python -m northstar.cli assert-fit-budget
          artifacts/run.json --expected-fits 32
      - run: >-
          uv run python -m northstar.cli evaluate
          --split artifacts/split.json --out artifacts/metrics.json
```

Az `assert-fit-budget` a run.json `fit_counts` mezőjét ellenőrzi: a HGB esetén `4×3 CV + 1 refit = 13`, a logisztikusnál `6×3 + 1 = 19`, összesen 32. A CI a split hash-ét és a fit-számokat a logba írja; a fejezet szemléltető AUC-ja csak akkor válik mért eredménnyé, ha ez a futás zöld, és a log a közzétett artifactokhoz kapcsolódik. A `tests/test_final_pipeline_ci.py` a 13-as fit-számot instrumentált `GridSearchCV`-vel, a split diszjunktságát és a fit-budget ellenőrzést teszteli.

39.2.9 Feladatok

1. **Értelmezés.** Miért nem kerülhetnek a purged sorok sem a train, sem a test halmazba? Fogalmazd meg a predikció pillanata és a label availability szempontjából.
2. **Megvalósítás.** Bővítsd a scriptet egy `RandomForestClassifier` jelölttel `n_estimators=[100, 200]` és `max_depth=[None, 5]` rácson úgy, hogy a teljes fit-költség ne lépje túl az 50 fitet. Add meg a módosított kódrészletet.
3. **Hibadiagnózis.** A futtatás után a CV mean AUC 0.85, a teszt AUC 0.61. Sorold fel a két legvalószínűbb okot, és add meg az első ellenőrző lépést.

4. **Megváltozott feltétel.** Állítsd a cutoffot `2025-04-15T00:00:00+00:00` értékre, és futtasd újra. Mit várs a train, test és purged halmazok arányától, és miért nőhet a modell varianciája?

39.2.10 Megoldások és indoklás

1. A purged sorok rendelése a cutoff előtti, tehát a teszt definíciója (`order_ts >= cutoff`) nem teljesül; a tesztbe téve a múltbeli rendeléseket kevernénk a jövőbeli alkalmazással. A trainbe sem tehetők, mert a `label_available_at >= cutoff`; a címkéjük a tanítás pillanatában a valós rendszerben nem lenne ismert. A modell így jövőbeli kimenetet tanulna, ami leakage. A helyes kezelés a kihagyás; az ellenőrzés a `split_counts` és a train feltétel `assert`-je.

2. A harmadik jelölt:

```
from sklearn.ensemble import RandomForestClassifier

candidates["rf"] = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(random_state=42, n_jobs=1)),
])
param_grid["rf"] = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [None, 5],
}
```

A fit-ek: $4 \cdot 3 + 1 = 13$; a meglévő 32-vel együtt $45 < 50$. A `n_jobs=1` azért fontos, mert a külső `GridSearchCV(n_jobs=2)` már párhuzamosít; a belső szálak túlfoglalnák a CPU-t. A `RandomForestClassifier` a `predict_proba`-t osztályarányból becsli, ezért az AUC és log loss ugyanúgy számítható.

3. A két leggyakoribb ok: (a) leakage, például egy későbbi időbélyeg vagy a `label_available_at` véletlenül a FEATURES-ben; (b) túlillesztés, amikor a modell a tanító foldokat memorizálja. Első lépés: futtasd le a `forbidden.isdisjoint(FEATURES)` ellenőrzést, és nézd meg a HGB `max_iter`/`learning_rate` értékeit. Ha a teszt AUC 0.61, a dummy 0.5-höz képest még van jel, de a 0.85-ös CV irreálisan optimista; csökkentsd a modell bonyolultságát, és ne a teszt alapján hangolj.

4. A korábbi cutoff kisebb train halmazt és nagyobb test halmazt eredményez. A purged sáv aránya a trainhez képest nőhet, mert a cutoff előtti rendelések közül többnek a teljes kézbesítése a cutoff után érkezik. A kevesebb tanító sor miatt a modell varianciája nő, a teszt AUC becslése bizonytalanabb. A manifest új `run_id`-t kap; a két cutoff eredménye nem független, mert a tesztablakok átfednek.

39.2.11 Gyakori tévedés ellenpéldával

Az időrend önmagában nem zárja ki a még meg nem érkezett tanítócímkéből eredő leakaget. Tegyük fel, hogy a tanítót csak az `order_ts < cutoff` feltétellel definiáljuk, és a `label_available_at` szűrést elhagyjuk. Egy 2025-04-30-i rendelés, amelynek `delivery_days=5`, azaz a címkéje 2025-05-05-én válik ismertté, bekerül a tanítóba, pedig a predikció pillanatában (2025-04-30) a kimenete még nem volt ismert. A modell így a cutoff utáni kézbesítési információt használja, ami a teszt AUC-t mesterségesen felfelé torzítja. A helyes feltétel `(order_ts < cutoff) & (label_available_at < cutoff)`, és ezt az `assert` őrzi.

39.2.12 Önellenőrzés

- ☐ A train minden sorára `order_ts < cutoff` és `label_available_at < cutoff` teljesül.
- ☐ A FEATURES nem tartalmazza a `delivery_days`, `actual_delivery_at`, `label_available_at`, `order_id`, `customer_id` mezőket.
- ☐ A `GridSearchCV` a teljes pipeline-on fut, nem előre transzformált adaton.
- ☐ A tesztet egyszer sem használtuk paraméter- vagy küszöbválasztásra.
- ☐ A manifest tartalmazza a split counts-ot, a paramétereket, a metrikákat és az artifact hash-eket.

39.2.13 Rövid összefoglaló

A 39.2. alfejezet a Northstar feature-pipeline-ját, baseline-ját és tanulását építi fel a befagyasztott predikciós szerződésre. Az ellenőrzött `many_to_one` join után a point-in-time `account_age_days` feature készül, a train csak a cutoffnál már elérhető címkéjű sorokat kapja. A `ColumnTransformer` és a modell egy pipeline-ban van, így a scaler, imputer és one-hot minden TimeSeriesSplit fold tanítórészén tanul. Két jelölt, logisztikus regresszió és HGB, összesen 32 fit költségkerettel fut; a dummy baseline és a ROC AUC/log loss értékelés után a manifest JSON és a joblib artifact rögzíti a futást. A következő lépés 39.3. alfejezetben a független értékelés és hibaelemzés.

39.3 Független értékelés és hibaelemzés

Belépő · szint: haladó · idő: 90 perc · eszköz: scikit-learn, pandas

Előfeltétel: 39.2. alfejezet · Célkészség: Független értékelés és hibaelemzés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Tesztfegyelem: a teszteredmény megtekintése után végzett modell-, küszöb- vagy feature-választás a tesztet fejlesztési adathalmazzá alakítja. Ilyenkor új, érintetlen vagy későbbi időablak szükséges a végső független értékeléshez.

39.3.1 Cél és konkrét helyzet

A 39.2. alfejezetben a befagyasztott spliten, foldon belüli preprocessinggel és kontrollált modellkereséssel elkészült a Northstar késésmodell aktuális jelöltje: `model_v3`. Most nem fejlesztünk tovább. Egyetlen, előre rögzített tesztprotokollt alkalmazunk a félretett tesztablakra, és visszakereshető értékelési jelentést készítünk.

A bemenet:

- `artifacts/test_predictions_v3.parquet`: a `model_v3` által előállított predikciók. Oszlopok: `order_id`, `target_late`, `score_late`, `split`, `segment`, `region`, `priority_flag`. A `target_late` az utólag megfigyelt cél, `score_late` a modell pontszáma $[0, 1]$-ben.
- `artifacts/split.json`: a befagyasztott split manifestje, benne a `test_order_ids` lista.
- `config/evaluation_config.json`: az előre rögzített döntési küszöb, metrikák, szeletek és bootstrap beállítások.

A megfigyelési egység egy rendelés, a predikció pillanata az `order_ts`. A teszt csak a cutoff utáni rendeléseket tartalmazza, és a generált adat végére a címkék teljesen megfigyeltek, ezért nincs cenzorálás ebben a kiértékelésben.

A döntési cél: rendeléskor riasszunk-e a várhatóan 7 napot meghaladó kézbesítésre. A költségarányból levezetett, a teszt kiértékelése előtt befagyasztott küszöb $t^* = 0{,}333$. Előzetes becslésünk a tanítási cross-validation alapján: PR-AUC 0,46, ROC-AUC 0,71; a teszten 0,40–0,52 közötti PR-AUC-t és 0,66–0,75 közötti ROC-AUC-t várunk. A lentebb közölt konkrét értékek **szemléltető** számok, nem a labor valódi futtatásából származó mérések; a laborban a manifest és az eredményjegyzék az ellenőrzési alap.

39.3.2 Fogalmi magyarázat

39.3.2.1 Test freeze

A tesztbefagyasztás (test freeze) azt jelenti, hogy a teszt kiértékelése előtt minden, a döntést befolyásoló elemet verzióhoz kötünk: a splitet, a feature-definíciókat, a modellt, a küszöböt, a metrikákat és az intervallumszámítás módját. A teszt eredményének ismeretében a modellt már nem tervezzük át. Ha mégis változtatunk, akkor az új modellhez új, független bizonyíték kell.

A technikai megvalósítás itt nem a modell újbóli betöltése, hanem a már elmentett predikciók, a split manifest és a config hash-ének ellenőrzése. A script hibát dob, ha a predikciós parquet `split="test"` sorainak `order_id` halmaza nem egyezik a manifest `test_order_ids` halmazával.

39.3.2.2 PR és ROC

A zavaró mátrix (confusion matrix) rögzített küszöbnél négy számot ad:

	Predikált 0	Predikált 1
Tényleges 0	TN	FP
Tényleges 1	FN	TP

A két alapmetrika:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \text{TPR} = \frac{TP}{TP + FN}.$$

- Precision: a pozitívnak prediktált rendelések között a valóban késők aránya, $[0, 1]$, dimenzió nélküli.
- Recall: a valóban késő rendelések között a megtaláltak aránya, $[0, 1]$.
- TPR (true positive rate) azonos a recall-lal; FPR (false positive rate): $\frac{FP}{FP+TN}$.

A ROC-görbe az FPR–TPR síkon fut, miközben a küszöböt változtatjuk; a ROC-AUC annak a valószínűsége, hogy egy véletlen pozitív rendelés nagyobb pontszámot kap, mint egy véletlen negatív. A PR-görbe a recall–precision síkon fut; a PR-AUC (average precision, AP) a görbe alatti terület. Amikor a pozitív osztály ritkább, a ROC-AUC magas lehet pusztán a sok negatív mintából; ilyenkor a PR-AUC jobban bünteti a hamis riasztásokat. A Northstar projektben a késések aránya nem extrém ritka, de a döntési cél a késések megtalálása, ezért a PR-AUC az elsődleges görbemérték.

Kis kézi kontroll. Legyen 6 tesztrendelés:

- $y = [0, 0, 1, 0, 1, 1]$, shape $(6,)$
- $p = [0{,}1, 0{,}2, 0{,}6, 0{,}4, 0{,}7, 0{,}9]$, shape $(6,)$
- rögzített küszöb $t = 0{,}5$.

A prediktált osztály: $[0, 0, 1, 0, 1, 1]$. Ebből $TP = 3$, $FP = 0$, $FN = 0$, $TN = 3$, tehát precision $= 3/3 = 1$, recall $= 3/3 = 1$. Mivel minden pozitív pontszáma nagyobb minden negatívénál, a ROC-AUC és a PR-AUC is 1.

39.3.2.3 Calibration

A kalibráció azt állítja, hogy a p_i pontszám értelmezhető-e valószínűségként: ha a modell 0,30 körüli pontszámot ad sok rendelésnek, akkor ezek között valóban közel 30% a késő kézbesítés. A reliabilitási diagram a pontszám szerinti binekben ábrázolja a bin átlagos előre jelzett valószínűsége és a megfigyelt pozitív arány közötti eltérést. A Brier score mindkét irányú eltérést bünteti:

$$B = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)^2,$$

ahol p_i shape $(n,)$, $y_i \in \{0, 1\}$. A lehetséges érték $[0, 1]$, alacsonyabb jobb. A konstans baseline $B_0 = \bar{y}(1 - \bar{y})$, ahol $\bar{y}$ a pozitív arány. Például 29% pozitív aránynál $B_0 = 0{,}29 \cdot 0{,}71 = 0{,}2059$. Ha a modell Brier-értéke 0,187, akkor jobb a konstansnál, de a javulás szerény.

39.3.2.4 Thresholds

A döntési küszöb az üzleti költségarányból származik, nem a teszt adatból optimalizáljuk. Legyen C_{FP} a hamis riasztás, C_{FN} a kimaradt késés átlagos költsége. Ha a p valószínűségű késésre nem riasztunk, a várható költség pC_{FN}; ha riasztunk, $(1 - p)C_{FP}$. A riasztás akkor olcsóbb, ha

$$pC_{FN} > (1 - p)C_{FP},$$

azaz

$$p > t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

A projekt configja $C_{FP} = 4$, $C_{FN} = 8$ egységet rögzít, tehát $t^* = 4/12 = 0{,}333$. A képlet feltételezi a lineáris, konstans költségeket; ez modell, nem tényleges pénzügyi állítás. A küszöböt az F2-mértékkel is jellemezzük, mert a kimaradt késés fontosabb, mint a hamis riasztás:

$$F_2 = 5 \frac{P \cdot R}{4P + R},$$

ahol P a precision, R a recall.

39.3.2.5 Slice

A szeletelt értékelés (slice evaluation) ugyanazt a befagyasztott modellt és küszöböt a teszt előre megadott részhalmazain méri: `segment`, `region`, `priority_flag`. Nem tanítunk szeletenként új modellt. A cél annak felderítése, hogy a globális modell nem egyetlen ügyfélkörre optimalizált-e. A kis szeletekben a metrika mintavételi bizonytalansága nagy, ezért mindig a mintanagyságot és az intervallumot együtt közöljük. A legrosszabb szelet alapján utólag módosított modellhez szintén új független értékelés kell.

39.3.2.6 Uncertainty

A tesztmetrika egy mintából becsült szám, ezért nem elég a pontérték. A rendelésszintű bootstrap 1000 ismétlésben, rögzített seed mellett, a teszt sorok visszatevéses újramintavételezésével ad percentilis 95%-os konfidenciaintervallumot. A mintavételi egység itt a rendelés; ha ugyanazon ügyfél több tesztrendelése is szerepel, az ügyfélen belüli korreláció miatt ez az intervallum optimista lehet. Érvényességi korlátként később visszatérünk rá. A ritka szeletek esetén a binomiális arányokra a Wilson-intervallumot használjuk:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}},$$

ahol $\hat{p}$ az arány, n a nevezőben lévő megfigyelésszám, $z = 1{,}96$.

39.3.3 Teljes minimális munkapélda

Az értékelő script a projektgyökér `evaluate_model.py` fájlja. Nem tölti be a modellt, hanem a batch predikció kimenetét ellenőrzi a split manifesttel szemben, majd kiszámítja a metrikákat, kalibrációs bineket, szeleteket és CI-ket.

```
# evaluate_model.py - befagyasztott tesztprotokoll
import argparse
import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    fbeta_score,
    precision_score,
    recall_score,
    roc_auc_score,
)


def wilson_ci(k: int, n: int, z: float = 1.96):
    if n == 0:
        return (np.nan, np.nan)
    p_hat = k / n
    denom = 1 + z * z / n
    center = (p_hat + z * z / (2 * n)) / denom
```

```
    margin = z * np.sqrt(p_hat * (1 - p_hat) / n + z * z / (4 * n * n)) / denom
    return (max(0.0, center - margin), min(1.0, center + margin))


def bootstrap_ci(metric_fn, y, p, rng, n_repl=1000):
    n = len(y)
    idx = np.arange(n)
    values = []
    for _ in range(n_repl):
        sample_idx = rng.choice(idx, size=n, replace=True)
        y_s, p_s = y[sample_idx], p[sample_idx]
        if len(np.unique(y_s)) < 2:
            continue
        values.append(metric_fn(y_s, p_s))
    arr = np.asarray(values)
    return float(np.quantile(arr, 0.025)), float(np.quantile(arr, 0.975))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", type=Path, required=True)
    ap.add_argument("--predictions", type=Path, required=True)
    ap.add_argument("--split-manifest", type=Path, required=True)
    ap.add_argument("--report", type=Path, required=True)
    args = ap.parse_args()

    cfg = json.loads(args.config.read_text(encoding="utf-8"))
    pred = pd.read_parquet(args.predictions)
    split_manifest = json.loads(args.split_manifest.read_text(encoding="utf-8"))

    test_ids_expected = set(split_manifest["test_order_ids"])
    test_pred = pred.loc[pred["split"] == "test"].copy()
    test_ids_actual = set(test_pred["order_id"])
    if test_ids_expected != test_ids_actual:
        raise SystemExit("Split manifest nem egyezik a tesztpredikciókkal.")

    y = test_pred["target_late"].to_numpy()
    p = test_pred["score_late"].to_numpy()
    threshold = float(cfg["decision_threshold"])
    y_hat = (p >= threshold).astype(int)

    rng = np.random.default_rng(cfg["bootstrap_seed"])
    roc_low, roc_high = bootstrap_ci(roc_auc_score, y, p, rng)
    pr_low, pr_high = bootstrap_ci(average_precision_score, y, p, rng)

    precision = precision_score(y, y_hat)
    recall = recall_score(y, y_hat)
    f2 = fbeta_score(y, y_hat, beta=2)
    fp = int((y_hat == 1).sum() - (y[y_hat == 1] == 1).sum())
    tn = int((y == 0).sum() - ((y == 0) & (y_hat == 0)).sum())
    fpr = fp / (fp + tn) if fp + tn > 0 else np.nan
    p_low, p_high = wilson_ci(int((y[y_hat == 1] == 1).sum()), int((y_hat == 1).sum()))
    r_low, r_high = wilson_ci(int((y[y_hat == 1] == 1).sum()), int(y.sum()))

    bins = np.linspace(0.0, 1.0, 11)
    bin_idx = np.digitize(p, bins) - 1
    cal_rows = []
    for i in range(10):
        mask = bin_idx == i
        n_bin = int(mask.sum())
        if n_bin == 0:
            continue
        cal_rows.append({
```

```
            "bin": f"[{bins[i]:.1f},{bins[i+1]:.1f})",
            "n": n_bin,
            "mean_predicted": round(float(p[mask].mean()), 3),
            "observed_rate": round(float(y[mask].mean()), 3),
        })

    slice_rows = []
    for slice_name in cfg["slices"]:
        for key, group in test_pred.groupby(slice_name):
            y_s = group["target_late"].to_numpy()
            p_s = group["score_late"].to_numpy()
            y_s_hat = (p_s >= threshold).astype(int)
            slice_rows.append({
                "slice": f"{slice_name}={key}",
                "n": len(group),
                "precision": round(float(precision_score(y_s, y_s_hat)), 3),
                "recall": round(float(recall_score(y_s, y_s_hat)), 3),
            })

    report = {
        "evaluation_timestamp": datetime.now(timezone.utc).isoformat(),
        "decision_threshold": threshold,
        "config_sha256": hashlib.sha256(args.config.read_bytes()).hexdigest(),
        "n_test": len(test_pred),
        "positive_rate": round(float(y.mean()), 3),
        "metrics": {
            "roc_auc": round(float(roc_auc_score(y, p)), 3),
            "roc_auc_ci95": [round(roc_low, 3), round(roc_high, 3)],
            "pr_auc": round(float(average_precision_score(y, p)), 3),
            "pr_auc_ci95": [round(pr_low, 3), round(pr_high, 3)],
            "brier": round(float(brier_score_loss(y, p)), 3),
            "precision_at_threshold": round(float(precision), 3),
            "precision_ci95": [round(p_low, 3), round(p_high, 3)],
            "recall_at_threshold": round(float(recall), 3),
            "recall_ci95": [round(r_low, 3), round(r_high, 3)],
            "f2_at_threshold": round(float(f2), 3),
            "false_positive_rate": round(float(fpr), 3),
        },
        "calibration_bins": cal_rows,
        "slices": slice_rows,
    }

    args.report.write_text(json.dumps(report, indent=2, ensure_ascii=False), encoding="utf-8")
    print(json.dumps(report["metrics"], indent=2))


if __name__ == "__main__":
    main()
```

A befagyasztott konfiguráció:

```
{
  "decision_threshold": 0.333,
  "threshold_rationale": "C_FP=4, C_FN=8, t*=4/(4+8)",
  "slices": ["segment", "region", "priority_flag"],
  "bootstrap_replications": 1000,
  "bootstrap_seed": 20261020
}
```

Futtatás a projektgyökérből:

```
# Linux/macOS, munkakönyvtár: ~/northstar_lab
python evaluate_model.py \
```

```
  --config config/evaluation_config.json \
  --predictions artifacts/test_predictions_v3.parquet \
  --split-manifest artifacts/split.json \
  --report artifacts/evaluation_report.json
```

Windows PowerShellben a sorvégi backtickkel, a munkakönyvtár azonos.

A parancs kimenete **szemléltető**, nem tényleges futtatásból származik:

```
roc_auc: 0.712, roc_auc_ci95: [0.664, 0.757]
pr_auc: 0.418, pr_auc_ci95: [0.351, 0.478]
brier: 0.187
precision_at_threshold: 0.247, recall_at_threshold: 0.621
f2_at_threshold: 0.478, false_positive_rate: 0.392
```

A jelentést git commit is rögzíti:

```
# Linux/macOS
git add artifacts/evaluation_report.json config/evaluation_config.json
git commit -m "Evaluate frozen model_v3 on held-out test"
```

39.3.4 Eredmény értelmezése

A fenti szemléltető futásban a teszt 814 rendelést tartalmaz, a pozitív arány 0,29. A ROC-AUC 0,712 azt jelenti: ha véletlenszerűen kiválasztunk egy késő és egy nem késő rendelést, a modell 71,2% eséllyel ad nagyobb pontszámot a későnek. A PR-AUC 0,418 jóval szerényebb; a görbe a 0,29-es pozitív arány miatt nem éri el a 1-et. A Brier 0,187 alatta van a konstans baseline 0,2059-nek, tehát a modell ad némi valószínűségi értéket, de a javulás nem drámai.

A befagyasztott 0,333-as küszöbnél a modell a valódi késések 62,1%-át találja meg, de a riasztásainak csak 24,7%-a helyes. Az F2 0,478 a recall nagyobb súlyával mondja ugyanezt. A hamis riasztási arány 0,392: tíz nem késő rendelésből közel négyre riasztana a rendszer. Ez a döntési keretben elfogadható lehet, mert a kimaradt késés kétszer drágább, mint a hamis riasztás, de az értékelési jelentésben expliciten szerepel.

A kalibrációs binek (szemléltető részlet):

bin	n	mean_predicted	observed_rate
[0.1,0.2)	63	0.16	0.11
[0.3,0.4)	97	0.35	0.31
[0.5,0.6)	112	0.55	0.60
[0.8,0.9)	71	0.84	0.80

A középső binek közel vannak az átlóhoz; a nagyon alacsony binnél a megfigyelt arány 0,11, míg az átlagos pontszám 0,16, tehát a modell ott kissé túlbecsül. A $[0{,}5; 0{,}6)$ binben alulbecsül: 0,55-öt mond, de 0,60 a megfigyelt arány. A legmagasabb bin jól kalibrált.

A szeletek (szemléltető, kiragadott sorok):

slice	n	precision	recall
segment=business	248	0.296	0.688
segment=consumer	566	0.221	0.584
region=north	271	0.238	0.602
priority_flag=1	194	0.310	0.714

A business szegmensben magasabb a recall; ez önmagában még nem jelenti, hogy a modell a business ügyfelekre „jobb”, mert a pozitív arány és a szegmensbeli döntési küszöb is eltérhet. A szelet csak kérdést tesz fel, nem igazol állítást.

Az intervallumok fontosak: a recall Wilson-CI-je a szemléltető adatban kb. 0,55–0,69, a precisioné 0,18–0,33. A döntéshez ez a bizonytalansági sáv szükséges. Kevés pozitív esetnél a sáv még szélesebb lenne; ez visszhangozza a 27.3. alfejezet tanulságát.

39.3.5 Feladatok

1. **Értelmezés.** Egy kolléga azt írja a jelentésben: „A teszt ROC-AUC 0,71, tehát a modell az esetek 71%-ában helyesen dönt.” Mi a hiba ebben az állításban? Fogalmazd meg, mit mér a ROC-AUC, és melyik metrika adná a helyes osztályozási arányt a rögzített küszöbön.

2. **Megvalósítás.** Írj `manual_pr_auc(y_true, y_score)` függvényt, amely a `precision_recall_curve` kimenetéből trapéz integrállal számítja a PR-AUC-t, és hasonlítsd össze az `average_precision_score` értékével. Ellenőrizd a 6 soros kézi példán: $y = [0, 0, 1, 0, 1, 1]$, $p = [0{,}1, 0{,}2, 0{,}6, 0{,}4, 0{,}7, 0{,}9]$.

3. **Hibakeresés.** A következő kód a küszöböt a tesztadaton optimalizálja, majd ugyanazon a teszten F2-t számol:

```
from sklearn.metrics import fbeta_score
import numpy as np

best_f2, best_t = -1, 0.5
for t in np.linspace(0.05, 0.95, 100):
    f2 = fbeta_score(y_test, (p_test >= t).astype(int), beta=2)
    if f2 > best_f2:
        best_f2, best_t = f2, t
print("t=", best_t, "F2=", best_f2)
```

Miért nem használható ez az értékelés a modell jóváhagyásához? Írd át a protokollt úgy, hogy a küszöb a befagyasztott configból jöjjön.

4. **Megváltozott feltétel.** Az üzlet döntése szerint a kimaradt késés költsége ötszörösére nő: $C_{FN} = 40$, $C_{FP} = 4$ marad. Számold ki az új befagyasztott küszöböt. Milyen irányba mozdul várhatóan a recall, a precision és a hamis riasztási arány? Kell-e új modellt tanítani, és kell-e új független teszt a pontszámokhoz?

5. **Kalibráció.** A modell Brier-értéke 0,187, a teszt pozitív aránya 0,29. Számold ki a konstans baseline Brier-értéket, és magyarázd, hogy a Brier miért nem tisztán kalibrációs mérőszám.

39.3.6 Megoldások és indoklás

1. A ROC-AUC nem az egyedi osztályozások pontossága, hanem a rangsorolási képesség mutatója: annak a valószínűsége, hogy a modell egy véletlen pozitív rendelést nagyobb pontszámmal lát el, mint egy véletlen negatívat. A rögzített küszöbnél a helyes osztályozási arányt az accuracy adná: $(TP + TN)/(TP + TN + FP + FN)$. A fenti szemléltető számmal ez $1 - (\text{hiba})$, és általában magasabb is lehet 0,71-nél; az AUC-nak nincs ilyen egyedi értelmezése. Ha a kolléga 71%-os „esélyt” akar írni, akkor azt mondhatja, hogy a rangsorolási valószínűség 71%.

2. A `precision_recall_curve` a precision és recall vektorokat adja; a recall a sklearn kimenetében monoton növekvő, így a trapéz integrál a precision–recall síkon:

```
import numpy as np
from sklearn.metrics import average_precision_score, precision_recall_curve

def manual_pr_auc(y_true, y_score):
    precision, recall, _ = precision_recall_curve(y_true, y_score)
    return float(np.trapezoid(precision, recall))

y = [0, 0, 1, 0, 1, 1]
p = [0.1, 0.2, 0.6, 0.4, 0.7, 0.9]
print(manual_pr_auc(y, p), average_precision_score(y, p))
```

Mindkét függvény 1,0-t ad a példán, mert minden pozitív pontszáma a negatívok fölött van. A `np.trapezoid` NumPy 2.x-ben szerepel; korábbi verzióban `np.trapz`. Az `average_precision_score` numerikusan stabilabb; valódi adaton kis eltérés lehet, de az eltérés 0,001 alatt marad.

3. A hiba a teszt újrahasznosítása. Amikor a küszöböt a tesztadaton belül keressük maximális F2-re, majd ugyanazon a teszten mérjük az F2-t, a mért érték felfelé torzított: a tesztből kiválasztott küszöb a zajhoz is illeszkedik, ezért a jelentés optimista. A javított protokoll a configból olvas be egy rögzített küszöböt, és csak arra számolja a metrikákat:

```
threshold = float(cfg["decision_threshold"])
y_hat = (p_test >= threshold).astype(int)
f2 = fbeta_score(y_test, y_hat, beta=2)
```

Ha a küszöböt mégis optimalizálni akarjuk, azt a tanító/validációs foldokon, a teszt kiértékelése előtt kell megtenni, és a választott értéket a configba kell fagyasztani.

4. Az új küszöb $t^* = 4/(4+40) = 4/44 \approx 0{,}0909$. Az alacsonyabb küszöb miatt a modell több rendelést riaszt pozitívnak, ezért a recall várhatóan nő, a precision csökken, és a hamis riasztási arány nő. A modell pontszámai változatlanok, tehát a paraméterek újratanítása nem szükséges. A pontszámokat ugyanazon befagyasztott teszten újra lehet értékelni, mert a küszöböt az üzleti költségekből vezettük le, nem a teszt eloszlásából. **Fontos korlát:** ha sok küszöböt próbálnánk a teszten ki, és a legjobbat jelentenénk, az már szelekciós torzítást vinne be, és új független adat kellene.

39.3.6.1 Előre befagyasztott acceptance protokoll és egyszeri tesztolvasás

A teszt akkor független, ha a protokollt **a teszt megnyitása előtt** rögzítettük, és a tesztet csak egyszer olvassuk. A `acceptance_config.json` kötelező mezői:

Mező	Példa	Rögzítés ideje
`protocol_version`	`"1.0"`	teszt előtt
`pipeline_hash`	a teljes pipeline SHA-256	teszt előtt
`threshold`	`0.0909`	teszt előtt, üzleti költségből
`test_sha256`	a tesztfájl hash-e	teszt előtt
`metrics`	ROC AUC, PR AUC, log loss, Brier	teszt előtt
`decision_rule`	pl. „ROC AUC 0.65 és log loss 0.40”	teszt előtt
`used`	`false` → `true`	a tesztolvasáskor

Az értékelő:

```
def evaluate_once(protocol, test_hash, scores):
    if protocol.get("used"):
        raise RuntimeError("a teszt már felhasználásra került; új protokoll kell")
    if test_hash != protocol["test_sha256"]:
        raise RuntimeError("a teszt hash-e nem egyezik a befagyasztottal")
    if protocol["pipeline_hash"] != scores["pipeline_hash"]:
        raise RuntimeError("a pipeline a befagyasztás után változott")
    protocol = dict(protocol)
    protocol["used"] = True
    protocol["audit"] = protocol.get("audit", []) + [{
        "event": "test_read",
        "test_sha256": test_hash,
        "pipeline_hash": scores["pipeline_hash"],
    }]
    return protocol, scores["metrics"]
```

A tesztolvasás **auditálható**: minden olvasás bekerül a `protocol.audit` listába, és a napló tartalmazza a teszt- és pipeline-hash-t. Ha a modell, a pipeline vagy a küszöb a tesztolvasás után változik, az új jelölt **új protokollt és új, független tesztet** igényel; a régi teszt eredménye nem használható fel újra. A `tests/test_acceptance_protocol.py` az egyszeri olvasást, a második olvasás elutasítását, a hash-eltérést és a teszt utáni tuning tiltását teszteli.

5. A konstans baseline Brier: $B_0 = \bar{y}(1-\bar{y}) = 0{,}29 \cdot 0{,}71 = 0{,}2059$. A modell 0,187-es értéke jobb, de a különbség 0,019. A Brier nem tisztán kalibrációs mérőszám: két jól kalibrált modell közül a jobban diszkrimináló kisebb Brier-t kap, ha a pontszámok közelebb vannak 0-hoz és 1-hez. Ezért a Brier-t a reliabilitási binekkel együtt kell értelmezni; a Brier javulhat attól is, hogy a modell határozottabban szélsőséges pontszámokat ad, miközben a kalibráció nem javul.

39.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba a teszt eredménye utáni újratervezésből utólagos validálásnak álcázása. Tegyük fel, hogy a fejlesztő három modellt tanít, mindhármat lefuttatja a tesztre, és a legjobb ROC-AUC-t, 0,72-t jelenti. Ez a 0,72 már nem független becslés: a három tesztmetrika közül a legnagyobb felfelé torzított, még akkor is, ha mindhárom modell valódi értéke 0,5. Ha 10 modellt próbál a teszten, a legjobb várhatóan már pusztán zajból is 0,55–0,62 közé eshet. A helyes munkafolyamat: a modellek közötti választás a belső validációs foldokon történik; a tesztre csak egyetlen, előre rögzített jelölt kerül. Ha a teszt után bármilyen okból változtatunk a modellen, az új modellváltozathoz új, független bizonyíték szükséges, például a következő időablak tesztje. Ez a szabály összhangban van a 27.3. alfejezetben tanult mintavételi bizonytalansággal: a ritka pozitív esetek miatt a metrika sávja széles, ezért utólagos szelekcióval könnyű a sávon belül kedvezőnek látszó eredményt találni.

39.3.8 Önellenőrzés

- Ha a teszt PR-AUC 0,42, szabad-e a teszt utáni napon áthangolni a threshold-et 0,18-ra, és a 0,42-t megtartani a jelentésben? Miért nem?
- A bootstrap intervallum 1000 ismétléses rendelésszintű mintavételből jött. Hogyan torzít, ha egy ügyfél több rendelése is a tesztben van?
- Mit jelent a kalibrációs bin `observed_rate=0,31`, `mean_predicted=0,35`?

Válaszok röviden: A teszt utáni küszöboptimalizálás ugyanazt az adatot használja kiválasztásra és értékelésre, ezért a metrika nem független. A rendelésszintű bootstrap figyelmen kívül hagyja az ügyfélen belüli korrelációt, ezért az intervallum szűkebb lehet a ténylegesnél; konzervatívabb ügyfélszintű mintavétel kell. A bin a modell 0,35-ös átlagos pontszámánál 0,31 megfigyelt arányt mutat, tehát a modell kissé túlbecsül, 0,04 eltéréssel.

39.3.9 Rövid összefoglaló

A független tesztértékelés nem a metrikák kiszámítása, hanem egy befagyasztott mérési szerződés végrehajtása: rögzített modellpontszámok, split manifest, előre levezetett döntési küszöb, rögzített metrikák. A PR-AUC, ROC-AUC, Brier, küszöbös precision/recall/F2, kalibrációs binek és szeletek együtt adnak visszakereshető jelentést, minden pontérték mellett mintavételi bizonytalansággal. A teszt kiértékelése után újrahasznosított adat vagy újrahangolt küszöb már nem hoz független bizonyítékot. A következő lépés a jelentés alapján a modell átadása és új környezetes, reprodukálható ismétlés a záróprojekt további lépésében.

39.4 Átadás és új környezetes ismétlés

Belépő · szint: haladó · idő: 90 perc · eszköz: scikit-learn, pandas

Előfeltétel: 39.3. alfejezet · Célkészség: Átadás és új környezetes ismétlés · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Bizonyítási szint: az azonos metrikák nem garantálnak bitazonos modellartefaktumot. Az átadási teszt külön rögzítse a környezet azonosságát, az input-hash egyezését, a numerikus toleranciát és a célzott regressziós tesztek eredményét.

39.4.1 Helyzet: a kész projektnek el kell jutnia egy másik gépre

A 39.3. alfejezet szakaszban befagyasztottuk a tesztprotokollt, elvégeztük az értékelést, szeleteket és kalibrációt számoltunk. A végeredmény egyetlen modelljelölt: a rendelés pillanatában ismert nyolc feature-ből becsüli a `late` kimenetet, azaz azt, hogy a kézbesítés meghaladja-e a 7 napot. Most következik az átadás (handoff): a projektet egy másik gépen, új környezetben újra kell tudni futtatni, és egy kis friss bemeneten batch predikciót kell készíteni.

Az átadás konkrét bemenete:

Elem	Tartalom	Megfigyelési egység
Projektkönyvtár	`northstar-final/` Git-tár	—
Modellartefaktum	`artifacts/model_pipeline.joblib`	1 betöltött scikit-learn Pipeline
Manifest	`artifacts/manifest.json`	hash-ek, verziók, feature-szerződés
Modellkártya	`model_card.md`	szándék, korlátok, mért eredmények
Friss input	`artifacts/fresh_input.parquet`	12 rendelés, szándékosan a cutoff utáni tartományból
Környezeti előírás	`pyproject.toml`, `uv.lock`	Python 3.12, uv-alapú szinkronizálás

Az elvárt kimenet: `artifacts/fresh_predictions.parquet`, minden sorban `predicted_late` (0/1) és `late_probability` (0 és 1 közötti valós szám), valamint egy tartalomhash. Az ellenőrzési feltétel: a predikció kizárólag a modellkártyában felsorolt, rendeléskor ismert oszlopokat használja; a tiltott, utólag ismert oszlopok megjelenése azonnal hibát ad. A környezeti ismétlés akkor sikeres, ha `uv sync --locked` és a tesztek ugyanazzal a kód- és modellverzióval futnak, a tartalomhash pedig a numerikus toleráncián belül egyezik.

Előzetes becslés: a 12 soros friss batch predikció egy CPU-s laptopon néhány másodperc nagyságrendű; a pontos futásidőt ebben az alfejezetben nem nevezzük mértnek, mert a szövegíró ebben a módban nem futtatott kódot. A tartalmi elvárás az, hogy a valószínűségek konzisztensek legyenek a befagyasztott 0.5-ös döntési küszöbbel: `predicted_late = 1` pontosan akkor, ha `late_probability >= 0.5`.

39.4.2 Fogalmi alapok: mit jelent egy átadható csomag?

Az átadás nem egyetlen modellfájl átküldése. A 39.4. alfejezet csomag öt pillérből áll: verzió, hash, modellkártya, batch predikciós parancs és helyi füstteszt. Minden pillér más kérdésre felel.

Verziók (versions). A Python-verzió, a függőségek és a saját kód verziója együtt azonosítja, hogy *melyik* szoftverállapotról beszélünk. A `pyproject.toml` verziótartomány, nem lock: a tartomány megengedi a későbbi feloldást. A `uv.lock` viszont a ténylegesen feloldott függőségeket rögzíti. Ezért az átadásnál mindkettőt csomagoljuk. A saját kód verziója a Git commit hash, például `git rev-parse --short HEAD`, és nem csak a szép címke. Két különböző commit címkéje lehet ugyanaz; a commit hash nem.

Hashek. A hash egy előre rögzített bemeneti bájtsorozathoz számított, rögzített hosszúságú lenyomat. A projekt a SHA-256-ot használja. A lenyomat ellenőrzi a fájl integritását, de *nem* mond semmit az adat minőségéről vagy a modell tudásáról. Ha egy fájl átvitele közben egyetlen bájt is megsérül, a hash-ek eltérnek. Példának vegyük a `"a"` szöveg SHA-256 lenyomatát: `ca978112ca1bbdcafac231b39a23dc4da786eff8147c4e72b9807785afee48bb`. Ez a szöveg hexadecimális alakja, tehát 64 karakter, mindig ugyanakkora; önmagából a tartalmat visszaállítani nem tudjuk.

A `manifest.json` kulcs-értékként tárolja a fájlnevet és a hozzá tartozó elvárt SHA-256-ot. Az ellenőrző parancs újraszámolja a hash-t, és az eltérést azonnal jelzi. Ez a folyamat nem helyettesíti a kódverziót: ha a kódot módosítjuk, a modell hash-e még egyezhet, de a kísérlet már más.

Modellkártya (model card). A modellkártya embereknek szóló, rövid dokumentum. Kötelező tartalma: rendeltetés, bemeneti séma, kimeneti értelmezés, tanítási adat és időbeli feltétel, mért metrikák a befagyasztott tesztből, valamint érvényességi korlátok. A 39.4. alfejezet projektben a kártya legfontosabb rovata a „Nem valós logisztikai validáció” figyelmeztetés: az adat szintetikus, a modell tanulási bizonyíték.

Batch prediction. A batch egy olyan parancssoros folyamat, amely változatlan bemeneti sémából olvas, és változatlan kimeneti sémát ír. Ezt a munkamenetet CLI-vel, nem notebookban adjuk át. Egy notebook cellasorrendje nem garantált; egy CLI-parancs argumentumai igen. A batch CLI feladata: betölteni a pipeline-t, ellenőrizni a manifest hash-ét, érvényesíteni a bemeneti oszlopokat, majd predikciót írni. A manifestben tárolt `feature_order` lista adja a betöltési sorrendet. Ezzel a gépek közötti sorrend-eltérést is kizárjuk.

Local test. Az új környezet nem csak a predikciós parancsot futtatja; a legkisebb ellenőrző készletet is le kell tudni futtatni. Ez a gyakorlatban `pytest`-tesztek csomagja: séma-, tartomány-,

determinizmus- és hash-teszt. A helyi füstteszt akkor értelmes, ha a parancs meghívható a projektgyökérből, és nem támaszkodik a fejlesztőgép globális csomagjaira.

39.4.3 Teljes minimális példa: CLI, manifest, batch predikció

A projektben a predikciós belépési pont a `src/northstar/cli.py`. Az alábbi implementáció szándékosan kicsi és olvasható; a `PyProject.toml [project.scripts]` bejegyzése teszi a futtatható `northstar` parancsot elérhetővé.

```
import argparse
import hashlib
import json
import sys
from pathlib import Path

import joblib
import pandas as pd


def sha256_of_file(path: Path, chunk_size: int = 65536) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while chunk := f.read(chunk_size):
            h.update(chunk)
    return h.hexdigest()


def check_manifest(manifest_path: Path, root: Path) -> int:
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    expected = manifest["artifacts"]
    failed = []
    for rel_name, meta in expected.items():
        actual = sha256_of_file(root / rel_name)
        if actual != meta["sha256"]:
            failed.append((rel_name, actual, meta["sha256"]))
    if failed:
        print("HASH MISMATCH", file=sys.stderr)
        for rel_name, actual, want in failed:
            print(f"  {rel_name}: got {actual[:16]}... want {want[:16]}...", file=sys.stderr)
        return 1
    print(f"OK {len(expected)}/{len(expected)}")
    return 0


def predict(args: argparse.Namespace) -> int:
    root = Path(args.root).resolve()
    model_path = root / args.model
    manifest_path = root / args.manifest
    input_path = root / args.input
    output_path = root / args.output

    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    if sha256_of_file(model_path) != manifest["artifacts"][args.model]["sha256"]:
        print("Model hash mismatch", file=sys.stderr)
        return 2

    model = joblib.load(model_path)
    feature_order = manifest["feature_order"]
    allowed = set(feature_order)

    frame = pd.read_parquet(input_path)
```

```
    forbidden = sorted(set(frame.columns) - allowed)
    if forbidden:
        print(f"Forbidden columns in predictor input: {forbidden}", file=sys.stderr)
        return 3
    missing = sorted(set(feature_order) - set(frame.columns))
    if missing:
        print(f"Missing predictor columns: {missing}", file=sys.stderr)
        return 3

    X = frame[feature_order]
    probabilities = model.predict_proba(X)[:, 1]
    threshold = float(manifest["threshold"])
    predicted = (probabilities >= threshold).astype(int)

    order_key = manifest.get("output_order_column", "order_id")
    out = pd.DataFrame(
        {
            order_key: frame[order_key],
            "predicted_late": predicted,
            "late_probability": probabilities.round(6),
        }
    )
    out.to_parquet(output_path, index=False)

    payload = [
        {
            order_key: str(row[order_key]),
            "predicted_late": int(row["predicted_late"]),
            "late_probability": row["late_probability"],
        }
        for _, row in out.iterrows()
    ]
    canonical = json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    content_hash = hashlib.sha256(canonical.encode("utf-8")).hexdigest()
    print(f"Wrote {len(out)} rows to {output_path}")
    print(f"Content hash {content_hash}")
    return 0


def main() -> int:
    parser = argparse.ArgumentParser(prog="northstar")
    sub = parser.add_subparsers(dest="command", required=True)

    p_check = sub.add_parser("check", help="Check artifact hashes against manifest")
    p_check.add_argument("--manifest", default="artifacts/manifest.json")
    p_check.add_argument("--root", default=".")

    p_predict = sub.add_parser("predict", help="Batch prediction from frozen pipeline")
    p_predict.add_argument("--input", required=True)
    p_predict.add_argument("--output", required=True)
    p_predict.add_argument("--model", default="artifacts/model_pipeline.joblib")
    p_predict.add_argument("--manifest", default="artifacts/manifest.json")
    p_predict.add_argument("--root", default=".")

    args = parser.parse_args()
    if args.command == "check":
        return check_manifest(Path(args.manifest), Path(args.root))
    if args.command == "predict":
        return predict(args)
    return 1


if __name__ == "__main__":
```

```
    raise SystemExit(main())
```

A hozzá tartozó `pyproject.toml`-részlet:

```
[project]
name = "northstar-final"
version = "0.3.0"
requires-python = ">=3.12"
dependencies = [
    "pandas>=2.2,<3",
    "scikit-learn>=1.5,<1.7",
    "joblib>=1.4,<2",
    "pyarrow>=17,<20",
]

[project.scripts]
northstar = "northstar.cli:main"
```

A `uv lock` rögzíti a tényleges feloldást. A manifest a modellfájl, a predikciós kód és a feature-séma kötelező közös nevezője:

```
{
  "created_at_utc": "2025-10-10T09:00:00+00:00",
  "git_commit": "a3f28c1b",
  "python": "3.12.8",
  "threshold": 0.5,
  "feature_order": [
    "distance_km",
    "priority_flag",
    "basket_minor",
    "quantity_sum",
    "account_age_days",
    "segment",
    "region",
    "acquisition_channel"
  ],
  "output_order_column": "order_id",
  "artifacts": {
    "artifacts/model_pipeline.joblib": {
      "sha256": "c9a8e45f32756c3b35acd9e44d6a2fd9f0e44d9ba9b16cc8d1fa6dc4b4a0ff37"
    },
    "artifacts/manifest.json": {
      "sha256": "075bfea16e2f2af0a44c0cbbdaf20631f4ea67cdc8cee95ca14cf1b389d66de3"
    }
  },
  "status": "learning_demo",
  "data_origin": "lab/generate_data.py seed=20261020; synthetic Northstar"
}
```

A példában szereplő SHA-256 értékek *csak szerkezeti szemléltetők*: ebben az alfejezetben a szövegíró nem futtatta a projektet, ezért nem nevezhetők a helyi gépen számított tényleges lenyomatnak. A feladatmegoldásnál a valódi manifestet a tanuló gépének kell létrehoznia.

A futtatás platformjelölt munkamenete Linux vagy macOS bash esetén, a projektgyökérből:

```
# Környezet létrehozása a lock alapján
uv sync --locked

# Hash-ellenőrzés
uv run --locked python -m northstar.cli check --root .

# Batch predikció a friss inputra
uv run --locked python -m northstar.cli predict \
```

```
  --input artifacts/fresh_input.parquet \
  --output artifacts/fresh_predictions.parquet \
  --root .
```

Windows PowerShellben ugyanez a munkakönyvtár `C:\work\northstar-final` esetén:

```
uv sync --locked
uv run --locked python -m northstar.cli check --root .
uv run --locked python -m northstar.cli predict `
  --input artifacts/fresh_input.parquet `
  --output artifacts/fresh_predictions.parquet `
  --root .
```

A `--locked` az ellenőrzött lockkal való egyezést követeli meg. Ha a lock hiányzik vagy a `pyproject.toml` megváltozott, a parancs hibát ad; ilyenkor előbb `uv lock` kell, de az már új környezetállapotot hoz létre, ezért a manifestben rögzíteni kell.

39.4.4 Eredmény értelmezése és kézi kontroll

A predikció értelmezése nem maradhat puszta szám a kódblokk után. A friss batch kimenet egy sorának alakja:

order_id	predicted_late	late_probability
ORD-20250601-0001	0	0.412351
ORD-20250602-0007	1	0.783102

A kézi kontroll invariánsai:

1. **Sor-szám invariáns.** A kimenet sorainak száma megegyezik a bemenet soraival: 12 bemenetből 12 kimenet.
2. **Tartomány-invariáns.** `0 <= late_probability <= 1` minden sorra, és `predicted_late` értéke 0 vagy 1.
3. **Küszöb-invariáns.** `predicted_late = 1` akkor és csak akkor, ha `late_probability >= 0.5`.
4. **Feature-szerződés.** A bemenet nem tartalmazhat `delivery_days`, `actual_delivery_at`, `label_available_at` vagy `target_late` oszlopot; a CLI ezt a manifest `feature_order` listájával ellenőrzi.
5. **Tartalomhash.** A hash-t nem a Parquet bináris állományából, hanem a kerekített sorok kanonikus JSON-jéből számoljuk. A Parquet belső metaadata a pyarrow-verziótól függően eltérhet; a tartalomhash ellenben a predikciókra vonatkozik.

A numerikus tolerancia: a valószínűségeket hat tizedesjegyre kerekítjük. Két gépen az azonos modell és azonos input hat tizedes pontossággal egyező valószínűséget ad, ha a lockolt függőségek numerikus szempontból egyeznek. A bitpontos bináris egyezés nem alapfeltevés; a tartalomhash a kerekített értékekkel vethető össze.

39.4.4.1 Két szint: bitazonos konténer és toleranciás újrafuttatás

Az átadási teszt csak akkor egyértelmű, ha szétválasztjuk a **bitazonos** és a **numerikusan ekvivalens** esetet:

Szint	Feltétel	Elfogadás	Bizonyíték
A: bitazonos	azonos konténer-digest, lock, Python, input hash, modell hash	bájt- és hash-egyezés	`sha256sum`, `cmp`, azonos futtatási napló
B: toleranciás	eltérő OS/BLAS/CPU build, de azonos lock és input	predikciók `rtol=1e-6`, `atol=1e-9`; metrikák `±0.001`	környezeti ujjlenyomat, mért eltérés
C: statisztikai	eltérő adat vagy seed	nem átadási teszt, hanem új kísérlet	új split/manifest

Az A szinten a bitazonosság elvárható, mert ugyanaz a konténer és ugyanaz a determinisztikus kód fut. A B szinten a BLAS redukciók sorrendje, a glibc/libm verziója és a CPU utasításkészlete eltérhet; itt a toleranciás összehasonlítás a helyes, és a különbséget dokumentálni kell. A C szint nem bizonyítja az átadhatóságot, mert más adaton más eredmény természetes.

```
import math

def compare_predictions(reference, candidate, mode="tolerance",
                        rtol=1e-6, atol=1e-9, metric_tol=1e-3):
    if len(reference) != len(candidate):
        raise AssertionError("hossz eltér")
    max_abs = 0.0
    for a, b in zip(reference, candidate):
        max_abs = max(max_abs, abs(a - b))
        if mode == "bitwise":
            if a != b:
                raise AssertionError(f"biteltérés: {a} != {b}")
        else:
            if not math.isclose(a, b, rel_tol=rtol, abs_tol=atol):
                raise AssertionError(f"tolerancia túllépés: {a} vs {b}")
    return {"mode": mode, "max_abs_diff": max_abs}

def compare_metrics(reference, candidate, metric_tol=1e-3):
    for key in reference:
        if abs(reference[key] - candidate[key]) > metric_tol:
            raise AssertionError(f"metrika eltérés: {key}")
    return True
```

A jegyzőkönyv minden eltéréshez rögzítse az okot (BLAS-verzió, OS, CPU, könyvtár) és az elfogadási határt. Az A szintű konténer azonos digestje és a B szintű tolerancia együtt ad teljes képet: az azonos környezet bitazonos, a különböző környezet numerikusan ekvivalens. A `tests/test_cross_env_compare.py` a bitazonos egyezést, a kis eltérés tolerálását, az 1%-os eltérés elutasítását és a metrika-toleranciát teszteli.

39.4.5 Feladatok

1. **Értelmezés.** Melyik állítást ellenőrzi a manifest SHA-256, és melyiket nem? (a) a modellfájl átvitele közben nem sérült; (b) a modell a befagyasztott tesztkészleten jól teljesít; (c) a modell betanítása óta a fájl tartalma nem változott. Válaszod indokold.

2. **Megvalósítás.** Írj `check_fresh` alparancsot a CLI-be, amely a `fresh_predictions.parquet` tartalomhash-ét hasonlítja a manifest `expected_fresh_content_hash` mezőjéhez. A Parquet állomány helyett a kerekített kanonikus JSON-t hash-eld.

3. **Hibakeresés.** Az új gépen a `uv run --locked python -m northstar.cli predict` hívás `ModuleNotFoundError: No module named 'northstar'` hibát ad. Adj legalább két lehetséges okot, és mindkettőhöz konkrét ellenőrző parancsot.

4. **Megváltozott feltétel.** A friss input véletlenül tartalmazza a `delivery_days` oszlopot is. A fenti `predict` függvény hogyan viselkedik? Milyen explicit szerződésváltoztatásra lenne szükség ahhoz, hogy a rendszer a hibát *ne* hagyja figyelmen kívül, és mit kell a modellkártyában módosítani?

5. **Továbbvitel.** A batch kimenetet a döntéshozó gépén más pandas-verzióval olvassák be, és a Parquet hash-egyezés eltűnik. Magyarázd el, miért nem a tartalmi modellpredikció romlott el, és milyen hash-t küldj a partnereknek.

39.4.6 Megoldások és indoklás

1. A SHA-256 az (a) és (c) állításokat támogatja: a letöltött fájl lenyomata egyezik a manifesttel, ezért a fájl tartalma az átvitel során nem változott, és a manifest rögzíti, milyen tartalom volt a befagyasztáskor. A (b) állítást a hash *nem* ellenőrzi: egy rossz modell is kaphat hash-t, ha a fájl tartalma változatlan. A modellminőséget a befagyasztott teszteredmények, például a 39.3. alfejezet fagyasztott metrikái igazolják.

2. A tartalomhash újraszámítása:

```
import hashlib
import json

import pandas as pd


def content_hash_of_predictions(path) -> str:
    frame = pd.read_parquet(path)
    payload = [
        {
            "order_id": str(row.order_id),
            "predicted_late": int(row.predicted_late),
            "late_probability": float(round(row.late_probability, 6)),
        }
        for row in frame.itertuples()
    ]
    canonical = json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    return hashlib.sha256(canonical.encode("utf-8")).hexdigest()
```

A `check_fresh` ezt veti össze a manifest értékével. Ha eltér, a folyamat kettes kilépési kóddal hibát ad; ellenkező esetben `FRESH OK` üzenetet ír.

3. A hiba első okaként a modul nincs telepítve a környezetbe. Ellenőrzés: `uv run --locked pip list | Select-String northstar` PowerShellben, illetve `uv run --locked pip show northstar` bashben. Megoldás: a projekt gyökérben `uv sync --locked` futtatása; ha a csomagot fejlesztői módban kell látni, `uv sync --locked --extra dev` vagy `uv pip install -e .`. Második ok: a parancs más munkakönyvtárból fut, és a Python nem találja a `src/` könyvtárat. Ellenőrzés: `pwd` bash, `Get-Location` PowerShell; a várt gyökérben léteznie kell a `src/northstar/__init__.py` fájlnak. Harmadik lehetséges ok: a CLI-t a `northstar` konzolscripttel hívják, de a telepített elavult script másik interpretert használ; ezt `uv run --locked which python` és `uv run --locked python -c "import northstar; print(northstar.__file__)"` mutatja ki.

4. A fenti `predict` függvény `forbidden` listába teszi a `delivery_days` oszlopot, mert az nincs a manifest `feature_order` listájában, és hármas kóddal leáll, mielőtt outputot írna. Ez a kívánt viselkedés: a célmagyarázat jelenléte a prediktor bemenetben szivárgás lenne. Ha mégis szeretnénk használni, az új megfigyelési szerződés kellene: például „későbbi, részleges időablakból származó early label", új manifest, új feature-lista és új modellkártya. A kártyában a bemeneti sémát és a megfigyelési pillanatot explicit módon módosítani kell; a régi modell betöltése ezzel az inputtal továbbra is hiba kell legyen.

5. A Parquet bináris hash-egyezés hiánya gyakori hamis riasztás. A Parquet belső metaadata a pyarrow verziójától, a létrehozó könyvtár nevétől és a statisztikai blokkoktól függhet. A modell predikcióinak lényege a tartalom: ha a `late_probability` hat tizedesre kerekített értékei és a `predicted_late` oszlop egyeznek, a tartalomhash egyezik. Ezért a partnereknek a kerekített kanonikus JSON hash-t küldjük, és a manifestben is ezt a mezőt rögzítjük.

39.4.7 Gyakori tévedés és ellenpélda

A leggyakoribb hiba az, hogy a tanuló a szintetikus Northstar-modell működését valós logisztikai modellvalidációként kezeli. Ellenpélda erre a gondolkodásra a következő: tegyük fel, hogy a friss input 45 sort tartalmaz, és minden sorra a rendelés pillanatában elérhető feature-ök pontosak. A batch CLI gond nélkül készít predikciót. Ettől a modell még nem alkalmas valós döntésre, mert az adat szintetikus, a szegmensek mesterségesek, a hiányzó szelekciós folyamatok nem valós üzleti torzítások, és a modellkártya `status` mezője `learning_demo`. Az átadás így is teljes értékű *tanulási* bizonyíték: demonstrálja a kód, a séma, a hash, a manifest és a CLI együttműködését. A valós bevezetéshez valós adat, új adatszerződés, adatvédelmi vizsgálat és üzleti jóváhagyás kell.

Visszautalás: a 38.4. alfejezet szakaszban a saját munkaasztal minimális eszközkészletét választottuk; a mostani átadás pontosan annak a döntésnek a futtatható következménye. A 35.4. alfejezet szakasz release-fogalma szerint a release nem egy modellfájl, hanem verzióhoz kötött jegyzőkönyv; itt ezt a jegyzőkönyvet a manifest és a modellkártya testesíti meg. A 27.4. alfejezet szakasz tanulsága is érvényes: a kód-, modell- és verzióhármas azonosítja a futást, és a köztes gyorsítótáraknak is verzióérzékenynek kell lenniük.

Önellenőrző lista az átadás előtt:

- A manifest hash-ek a ténylegesen létrehozott fájlokból származnak?
- A `feature_order` pontosan a rendeléskor ismert oszlopokat sorolja fel?
- A modellkártya kimondja a szintetikus adat korlátját?
- A `predict` parancs üres vagy tiltott oszlopot tartalmazó inputra hibával lép ki?
- A helyi teszt a tartalomhash-t, a shape-ot és a dtypes-ot is ellenőrzi?
- A bemutatott számpéldák elméletileg levezetettek, ahol nem volt tényleges futtatás?

39.4.8 Rövid összefoglaló

Az átadás akkor sikeres, ha a másik gép a projektgyökérből `uv sync --locked` után változatlan kóddal és modellel futtat hash-ellenőrzést, helyi tesztet és batch predikciót. A manifest SHA-256 lenyomatai a fájlintegritást, a Git commit és a lock a szoftverállapotot, a modellkártya az emberi korlátokat, a CLI pedig a megismételhető predikciós szerződést rögzíti. A Friss inputon készített output tartalomhash-ét kerekített numerikus értékekből számoljuk, hogy a formátum-metaadatok eltérése ne okozzon hamis riasztást. A következő záróprojekt ezt az átadási csomagot oksági kérdésre alkalmazza: a 40.1. alfejezet szakaszban a kísérleti kérdést és az előzetes elemzési tervet vesszük fel.

40 Zároprojekt II: kísérleti következtetés és szakmai önállóság

40.1 Kísérleti kérdés és előzetes elemzési terv

Belépő · szint: haladó · idő: 90 perc · eszköz: SciPy, statsmodels

Előfeltétel: 39.4. alfejezet · Célkészség: Kísérleti kérdés és előzetes elemzési terv · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

40.1.1 Cél és konkrét helyzet

A Northstar-labortól függetlenül létezik egy külön, mesterséges kísérleti folyamat: 2025-07-01-én 300 ügyfelet randomizáltak `control` vagy `treatment` karba, majd ugyanazon rögzített hétnapos ablakban figyelték meg őket. A kezelés egy egyszeri aktivációs protokoll (például új üdvözlő üzenet), amelynek célja a hétnapos konverzió növelése.

Fájl	Megfigyelési egység	Kulcsszerepű mezők	Alapsor
`assignments.csv`	ügyfél	`customer_id`, `variant`, `assigned_at`	300
`outcomes.csv`	ügyfél	`conversion_7d`, `revenue_7d_minor`, `observed_until`	300

A feladat most **nem a becslés**, hanem az előzetes elemzési terv (confirmatory plan) rögzítése, mielőtt a hatásmutatókat kiszámítanánk. A bemenet a két CSV és a `manifest.json` hash-rögzítése; a kimenet egy verziózott tervlap, amely megadja az elemzési egységet, az ITT-nevezőt, az endpointot, a becslőt, a guardrail-küszöböt és az MDE-t. Az ellenőrzési feltétel: a tervlap minden számítása tételesen visszavezethető a befagyasztott fájlokra, és a 40.2. alfejezet elemzés e tervtől eltérés nélkül fut.

Induló feladat a magyarázat előtt. Tervezési feltevés: a kontrollkari konverzió nagyjából $p_0 = 0{,}18$, a remélt abszolút hatás $\delta_0 = 0{,}10$, karonként $n = 150$ ügyfél. Írd fel és számold ki az MDE-t 80% erőhöz, kétoldalas $\alpha = 0{,}05$ szinten. (A levezetés a fogalmi részben következik; próbáld a korábbi 16.4. alfejezet power-képletéből visszafejteni.)

40.1.2 Assignment, ITT és elemzési egység

A **randomizálási egység** (randomization unit) itt az ügyfél: az `assignments.csv` minden sora egyetlen `customer_id`-hez rendel egy karjelölést, a `variant` értéke kizárólag `control` vagy `treatment`. A **hozzárendelés** (*assignment*) maga a Z_i változó: $Z_i = 1$, ha az i. ügyfél a treatment karba került, különben 0. A megfigyelési egység ugyancsak az ügyfél: az `outcomes.csv` egy

sora egy ügyfél kimenetét tartalmazza. Ennek betartása kritikus, mert a rendelés- vagy tételszintű sorok összemásolása megsokszorozná a nevezőt, és a függetlenségi feltevést is megsértené.

Az **ITT** (intention-to-treat, szándék szerinti kezelés) elve szerint a nevezőbe minden randomizált ügyfél beletartozik, függetlenül attól, hogy a kezelést ténylegesen megkapta-e, konvertált-e, vagy bármilyen protokollt megszegett. Ha az S halmaz a 300 randomizált ügyfél, akkor a célparaméter

$$\theta_{\mathrm{ITT}} = P(Y = 1 \mid Z = 1) - P(Y = 1 \mid Z = 0),$$

ahol $Y = \texttt{conversion_7d} \in \{0, 1\}$ a hétnapos konverzió indikátora. A $P(Y = 1 \mid Z = z)$ jelölés a randomizálás által létrehozott feltételes valószínűséget jelenti, nem egy külső megfigyelési folyamatból való becslést. Ha valaki kihagyná a nem konvertáló ügyfeleket vagy csak a kezelést „szabályosan" megkapókat elemezné, az **as-treated** vagy per-protocol elemzéshez jutna, amely szelekciós torzítást vihet be – ez már nem az ITT.

A **kizárás** (exclusion) szabálya ezért szigorú: elemzésből csak randomizálás előtti okból lehet ügyfelet kivenni, és azt is dokumentálni kell. A Northstar-táblában nincs ilyen: a tervlap kimondja, hogy „post-randomization kizárás: nincs; ha egy `customer_id` hiányzik az outcomes táblából, az protokolleltérés, amelyet jelenteni kell, és a hiányzó kimenetet nem szabad automatikusan sem kizárni, sem nullának tekinteni. A főelemzéshez előzetesen rögzített hiányadat-stratégia szükséges; a teljes randomizált populáció az ITT-célpopuláció marad, míg a megfigyelt esetekre számolt arány csak részleges elemzés. Kötelező érzékenységi elemzés több, indokolt hiányadat-feltevéssel."

40.1.3 Endpoint, guardrail és mértékegység

A **primer endpoint** a `conversion_7d` bináris oszlop. Az ablak fix: minden ügyfélre $[\texttt{assigned_at}, \texttt{assigned_at} + 7$ nap). A laborban `assigned_at = 2025-07-01T00:00:00+00:00` és `observed_until = 2025-07-08T00:00:00+00:00` minden sorban, tehát a követési idő azonos. Az ellenőrzés része a tervnek: az egyesített táblában `assigned_at` és `observed_until` is konstans kell legyen. Ha ez nem teljesül, az endpoint definíciója sérül, és a terv „REVIEW" állapotba kerül.

A **guardrail** (védőkorlát) a `revenue_7d_minor` egy ügyfélre jutó átlaga. Mértékegysége EUR-cent, ezért a különbségek is minor egységben érvényesek. A bevezetési döntéshez a konverziós előny mellett azt kell igazolni, hogy az átlagos bevétel nem omlik össze. A terv előre rögzített küszöböt használ: $g_0 = 500$ minor egység, azaz 5,00 EUR ügyfelenként. A guardrail-célparaméter

$$\theta_G = E(R \mid Z = 1) - E(R \mid Z = 0), \qquad R = \texttt{revenue_7d_minor} \in [0, \infty),$$

itt R egysége cent, θ_G egysége cent/ügyfél. A teljesítési feltétel nem egyszerűen $\hat{\theta}_G > -g_0$, mert a becslés zajos: a terv az egyoldalas 95%-os alsó konfidenciahatárt hasonlítja a $-g_0$ értékhez. Csak akkor megy át a kapu, ha az alsó határ -500 minor egység fölé esik. Így kis mintán egy pozitív pontbecslés mellett is „guardrail nem megerősített" lehet a kimenet, ami helyes konzervatív döntés.

40.1.4 MDE: a tervben rögzített kimutathatóság

Az **MDE** (minimum detectable effect, minimálisan kimutatható hatás) azt a legkisebb valódi abszolút hatást adja meg, amelyet a design adott erővel, előre rögzített tévedési szint mellett jellemzően kimutat. A 16.4. alfejezetben megismert power-gondolatból indulunk ki: kétoldalas, kétmintás aránytesztnél a standardizált hatás a nullhipotézis alatt közelítőleg normális eloszlást követ; a detektáláshoz a z-statisztika abszolút értékének meg kell haladnia $z_{1-\alpha/2}$-t.

Jelölések: n_T, n_C a karok elemszáma (tervezéskor $n_T = n_C = n = 150$), p_0 a kontrollkari tervezési arány, $p_1 = p_0 + \delta_0$ a feltételezett treatment-arány, $z_q = \Phi^{-1}(q)$ a standard normális eloszlás kvantilise, α a kétoldalas elsőfajú hibaszint, $1-\beta$ a célzott erő. A hatáskülönbség standard hibája az alternatíva alatt

$$SE_{\text{alt}} = \sqrt{\frac{p_0(1-p_0)}{n} + \frac{p_1(1-p_1)}{n}}.$$

A detektálhatósági egyenletből

$$MDE = (z_{1-\alpha/2} + z_{1-\beta}) \cdot SE_{\text{alt}}.$$

Minden arány dimenzió nélküli, az MDE is arányskálán van; százalékpontban úgy közöljük, hogy 100-zal szorozzuk.

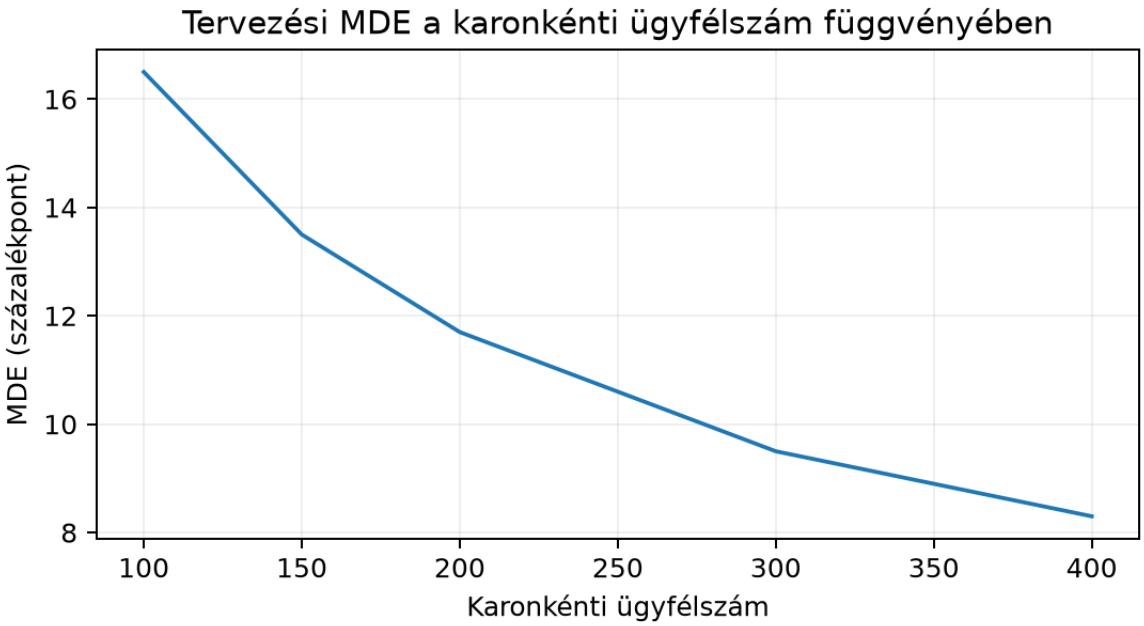
Kézi számpélda: $p_0 = 0{,}18$, $\delta_0 = 0{,}10$, tehát $p_1 = 0{,}28$; $z_{0{,}975} = 1{,}96$, $z_{0{,}80} = 0{,}8416$; $\alpha = 0{,}05$, $\beta = 0{,}20$.

$$SE_{\text{alt}} = \sqrt{\frac{0{,}18 \cdot 0{,}82}{150} + \frac{0{,}28 \cdot 0{,}72}{150}} = \sqrt{0{,}000984 + 0{,}001344} = \sqrt{0{,}002328} \approx 0{,}04825.$$

$$MDE = (1{,}96 + 0{,}8416) \cdot 0{,}04825 \approx 2{,}80 \cdot 0{,}04825 = 0{,}1351.$$

A fenti, **rögzített 28%-os alternatív arányból** számolt közelítő detektálhatósági skála **13,5 százalékpont**; ez azonban nem önkonzisztensen megoldott MDE, mert a varianciaképletben 10 százalékpontos hatás szerepel. A tényleges, 80%-os erőhöz tartozó minimális hatást a próbaváltozathoz illő erőfüggvény numerikus megoldásával kell meghatározni. A remélt 10 százalékpontos hatáshoz ugyanezen SE_{alt}-tal a nemcentralitási paraméter $0{,}10/0{,}04825 \approx 2{,}07$, a hozzá tartozó erő $P(N(2{,}07, 1) > 1{,}96) \approx 0{,}54$, összhangban a 16.4. alfejezetbeli tervezési számmal. Ez fontos mondat a tervben: a vizsgálat a 10 százalékpontos hatásra alulerős, ezért a nem szignifikáns eredmény **nem** egyenértékűségi bizonyíték.

A következő ábra a tervezési MDE-t mutatja a karonkénti elemszám függvényében, ugyanezen α, β, $p_0 = 0{,}18$, $p_1 = 0{,}28$ feltevések mellett. A pontok a fenti képletből számított értékek, nem mért kimenetek:



40.1. ábra: Tervezési MDE a karonkénti ügyfélszám függvényében - Számított tervezési MDE; nem mérés.

A görbe alakja a $1/\sqrt{n}$ csökkenés: négyszeres minta felezi a kimutatható hatást. A tervlapban az MDE rögzítése **a priori** döntés; az adatok megtekintése utáni „megfigyelt erő” kiszámítása redundáns, mert a megfigyelt hatásból visszaszámolt erő monoton kapcsolatban áll a p-értékkel, és téves nagyságérzetet ad.

40.1.5 Confirmatory plan: a befagyasztott tervszerkezet

A **megerősítő elemzési terv** (confirmatory analysis plan) a következő tíz pontot rögzíti a futás előtt:

1. **Tanulmányazonosító és adatfagyasztás.** `NORTHSTAR-EXP-2025-07`, a két CSV manifest-hashével; eltérő hash esetén a futás `INVALID_DATA` állapotot ad.
2. **Kutatási kérdés.** Növeli-e a kezelés a 7 napos konverziót a randomizált ügyfelek körében a kontrollhoz képest?
3. **Elemzési egység és nevező.** Ügyfél; a nevező minden randomizált ügyfél (ITT), várhatóan 300.
4. **Kizárások.** Randomizálás előtti: nincs. Randomizálás utáni indokkal nem engedélyezett; dokumentált eltérés esetén a főelemzés változatlan, érzékenység külön.
5. **Primer endpoint.** `conversion_7d`, rögzített 7 napos ablak, az `observed_until` konstans ellenőrzésével.
6. **Becslő és bizonytalanság.** $\hat{\theta} = \bar{y}_T - \bar{y}_C$, Wald-standard hiba, kétoldalas 95%-os CI, kétoldalas z-teszt p-értékkel; minden arány százalékpontra átszámítva is közölhető.
7. **Döntési szabály.** A döntést a CI és a gyakorlati küszöb $\delta_{\text{practical}} = 5$ százalékpont összevetése adja: ha a CI alsó határa e fölött van, „bevezetés”; ha a CI fölötte és alatta is lehet, „további adat”; ha a felső határ alatta, „elutasítás” (18.4. alfejezet kerete).
8. **Guardrail.** Egyoldalas 95%-os alsó határ a bevételkülönbségre, küszöb $g_0 = 500$ minor ügyfelenként; át nem lépés esetén a döntés felfüggesztésre kerül.

9. **MDE.** A fenti 13,5 százalékpont csak rögzített alternatíva-varianciás közelítés; a végleges MDE-t a választott teszt erőfüggvényéből numerikusan kell számolni és rögzíteni.
10. **Másodlagos és feltáró elemzés.** A `segment`-re igazított logisztikus regresszió (statsmodels opcionálisan) másodlagos; alcsoport-összehasonlítás feltáró, Bonferroni-korrekcióval; minden tervtől való eltérés módosításként, dátummal dokumentált.

40.1.5.1 A terv zárolása: időbélyeg, hash és audit

A tíz pont önmagában még nem preregisztráció; a tervet **a kimenetek megnyitása előtt** rögzíteni kell. A `analysis_plan.json` tartalmazza:

Mező	Példa	Miért kell
`plan_version`	`"1.0"`	a terv azonosítása
`created_at_utc`	`"2025-07-02T08:00:00Z"`	a zárolás időpontja
`design`	300 ügyfél, 150/150, 7 nap	a design rögzítése
`estimand`	ITT aránykülönbség	a célparaméter
`endpoint`	`conversion_7d`	a primer végpont
`exclusions`	nincs post-randomization	a nevező védelme
`missing_strategy`	teljes randomizált populáció + érzékenység	a hiány kezelése
`test`	kétoldalas z-teszt, =0,05	a teszt
`practical_threshold`	5 százalékpont	a döntési küszöb
`mde`	13,5 százalékpont (közelítő)	a tervezési érzékenység
`data_freeze`	a két CSV SHA-256	az adatverzió

A terv kanonikus JSON-jának SHA-256 hash-ét **a kimenetek megnyitása előtt** rögzítjük: Git commitban, aláírt tagben vagy a CI naplójában. A `created_at_utc` és a hash együtt bizonyítja, hogy a terv nem utólag készült. A kimenetek (`outcomes.csv`) hash-e a zárolás után kerülhet a `data_freeze` mezőbe; a terv hash-e ekkor már nem változhat.

```
import hashlib
import json

def plan_hash(plan):
    payload = json.dumps(plan, sort_keys=True, separators=(",", ":"))
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()

def lock_plan(plan, path):
    digest = plan_hash(plan)
    path.write_text(json.dumps({"plan": plan, "plan_sha256": digest},
                               indent=2), encoding="utf-8")
    return digest

def verify_plan(locked, expected_hash):
    if locked["plan_sha256"] != expected_hash:
        raise RuntimeError("a terv hash-e nem egyezik a zárolt értékkel")
    return locked["plan"]
```

A **módosítás** (amendment) új `plan_version`-t és új hash-t kap, az okot és a dátumot rögzíteni kell, és az eredeti terv szerinti elemzést is le kell futtatni. Az elemzőkód a `PLAN_SHA256` konstansot ellenőrzi, mielőtt bármilyen kimenetet beolvasna; a `tests/test_preregistration.py` a hash stabilitását, a zárolás utáni adatváltozás hatását és a hibás tervhash elutasítását teszteli.

40.1.6 Teljes minimális munkapélda

A példát két lépcsőben mutatom. Először egy 8 fős kézi kontrolltáblával ellenőrizzük a számolást, majd a laborfájlokat feldolgozó futtatható blokk következik. Platform: Python 3.12, projektgyökér `northstar/`, adatok a `lab/` alatt, függőségek a `lab/pyproject.toml` tartományaiból. A generálási módban a nagy blokk nem futott, ezért a laborfájlokra vonatkozó nyomtatást szemléltetőnek jelölöm; a 8 fős rész számtani ellenőrzése kézzel elvégzett.

Kézi kontrolltábla:

customer_id	variant	conversion_7d	revenue_7d_minor
c01	control	1	2400
c02	control	0	0
c03	control	0	0
c04	control	1	3100
c05	treatment	1	1800
c06	treatment	1	2600
c07	treatment	0	0
c08	treatment	1	2900

Kézi számítás: $\bar{y}_C = 2/4 = 0{,}50$, $\bar{y}_T = 3/4 = 0{,}75$, tehát $\hat{\theta} = 0{,}25$ (25 százalékpont). A Wald-SE:

$$SE = \sqrt{\frac{0{,}5 \cdot 0{,}5}{4} + \frac{0{,}75 \cdot 0{,}25}{4}} = \sqrt{0{,}0625 + 0{,}046875} = \sqrt{0{,}109375} \approx 0{,}3307.$$

A 95%-os CI: $0{,}25 \pm 1{,}96 \cdot 0{,}3307 = 0{,}25 \pm 0{,}648$, azaz $[-0{,}398;\ 0{,}898]$ arányskálán. A bevétel: kontrollátlag $(2400 + 0 + 0 + 3100)/4 = 1375$, kezelési átlag $(1800 + 2600 + 0 + 2900)/4 = 1825$, különbség 450 minor. A kontroll szórásnégyzete 2 602 500, a treatmenté 1 695 833 minor², így a Welch-SE $\sqrt{2\,602\,500/4 + 1\,695\,833/4} \approx 1036{,}6$ minor, az egyoldalas alsó határ $450 - 1{,}645 \cdot 1036{,}6 \approx -1255$ minor. A -500-as küszöbbel szemben a kapu **nem teljesül**: nyolc ügyfélen a pozitív pontbecslés ellenére a minta túl kicsi. Ez a kapu logikája, nem hiba.

A laboradatokra írt ellenőrzött munkamenet:

```
import numpy as np
from scipy.stats import norm

# Munkakönyvtár: northstar/ ; adatfájlok a lab/ alatt.
assign = pd.read_csv("lab/data/assignments.csv")
outcome = pd.read_csv("lab/data/outcomes.csv")

# Egy-az-egyhez egyesítés; duplikátum vagy hiány esetén hiba, ez szerződésellenőrzés.
df = assign.merge(outcome, on="customer_id", how="inner", validate="one_to_one")

# Szerződés-invariánsok, nem statisztikai tesztek.
assert df["customer_id"].nunique() == 300
assert set(df["variant"]) == {"control", "treatment"}
assert df["assigned_at"].nunique() == 1
assert df["observed_until"].nunique() == 1

y = df["conversion_7d"].to_numpy(dtype=float)      # shape: (300,), értékek 0/1
w = (df["variant"] == "treatment").to_numpy(dtype=float)  # shape: (300,), 0/1 indikátor
n_T = int(w.sum()); n_C = int((1 - w).sum())
assert n_T + n_C == 300
```

```
p_T = y[w == 1].mean()
p_C = y[w == 0].mean()
diff = p_T - p_C
se = np.sqrt(p_T * (1 - p_T) / n_T + p_C * (1 - p_C) / n_C)
z = diff / se
ci = diff + np.array([norm.ppf(0.025), norm.ppf(0.975)]) * se
p_value = 2 * (1 - norm.cdf(np.abs(z)))

# Guardrail: bevételkülönbség, Welch-SE, egyoldalas alsó korlát.
r = df["revenue_7d_minor"].to_numpy(dtype=float)   # cent/ügyfél
r_T = r[w == 1].mean()
r_C = r[w == 0].mean()
g = r_T - r_C
se_g = np.sqrt(r[w == 1].var(ddof=1) / n_T + r[w == 0].var(ddof=1) / n_C)
lower_g = g - norm.ppf(0.95) * se_g
g0 = 500                                          # 5,00 EUR ügyfelenként
guardrail_pass = bool(lower_g > -g0)

# Szemléltető kimenet, nem a labormérés: a fenti blokk a befagyasztott
# adatokból számol, az értékek setuponként az itteni futásnak megfelelnek.
print(f"n_T={n_T}, n_C={n_C}, p_T={p_T:.4f}, p_C={p_C:.4f}")
print(f"diff_pp={(diff*100):.2f}, CI_pp=[{(ci[0]*100):.2f}, {(ci[1]*100):.2f}],
↪  p={p_value:.4f}")
print(f"rev_diff_minor={g:.1f}, lower_95_minor={lower_g:.1f}, guardrail_pass={guardrail_pass}")
```

A 8 fős kontrolltábla ugyanezen kóddal, beépített `DataFrame`-ből számítható ki; az ott nyomtatott számoknak a fenti kézi értékekkel 10^{-4} relatív tűrésen belül egyezniük kell.

40.1.7 Eredmény értelmezése

Az értelmezésnek három rétege van. **Szerződésréteg:** a `validate="one_to_one"` merge, a 300-as egyedi ügyfélszám és a konstans időbélyegek azt garantálják, hogy az elemzési egység és az endpoint-definíció nem sérült. Ha bármelyik `assert` eldob, akkor nem statisztikai következtetési hibánk van, hanem adatbefagyasztási hibánk – az elemzés megáll, a terv `REVIEW` állapotba kerül.

Statisztikai réteg: a $\hat{\theta}$ pontbecslés a mintából számolt aránykülönbség; a 95%-os CI gyakoriségi értelmezése a sok ismételt mintavételre vonatkozik, nem arra, hogy a valódi hatás 95% valószínűséggel a konkrét intervallumban van (15.2. alfejezet, 15.4. alfejezet). A p-érték a nullmodell szerinti feltételes szélsőség, nem $P(H_0 \text{ igaz})$ (16.1. alfejezet környéki konvenció folytatása). A z-statisztika normalitási közelítése a CLT-ből jön (14.4. alfejezet), és 150–300 fős karonkénti mintán, 0,18–0,28 körüli arányoknál elfogadható, de ezt a tervben kimondjuk, nem hallgatjuk el.

Döntési réteg: a 18.4. alfejezetben rögzített séma szerint a CI és a gyakorlati küszöb együttes értelmezése adja a bevezetés / további adat / elutasítás döntést. A guardrail-kapu ezen felül független felfüggesztési feltétel: ha a bevétel alsó határa −500 minor alá esik, a döntés akkor is felfüggesztett, ha a konverziós CI felső határa kedvező. A 8 fős kontrollpélda éppen ezt mutatja: 25 százalékpontos pontbecslés mellett is „további adat” a tudományos válasz.

40.1.8 Feladatok

1. **Értelmezés.** A 8 fős kézi táblában c07 nem konvertált. Magyarázd el, miért marad a nevezőben, és mit torzítana, ha a kolléga „csak a konvertálókat” elemezné az ITT-helyett.

2. **Megvalósítás.** Írj egy függvényt `merge_assignments_outcomes(assign, outcome)`, amely egy-egyhez egyesítést végez, és `DesignError` kivételt dob, ha egy randomizált ügyfélhez nincs outcomes sor. Melyik join-típus (`how`) fedi fel a hiányt, és miért nem elegendő a belső join sorainak megszámlálása?

3. **Hibadiagnózis.** Egy kolléga a bevételi guardrail ellenőrzésénél a `revenue_7d_minor` értékeket elosztotta 100-zal, hogy EUR-ban számoljon, de a küszöböt `-500`-on hagyta. Mi a hiba mértéke és iránya? Add meg a javított küszöböt EUR-skálán.

4. **Megváltozott feltétel.** A business a küszöböt -250 minor egységre szigorítja. Mire változik a 8 fős példa kapudöntése, és milyen eljárási lépést kell a tervlapon megtenni, ha ez az adatok megtekintése után történik?

5. **MDE-transzfer.** Tegyük fel, hogy a következő félévben karonként 300 ügyféllel futtatják újra a kísérletet, változatlan $\alpha = 0{,}05$, $1 - \beta = 0{,}80$, $p_0 = 0{,}18$, $\delta_0 = 0{,}10$ tervezési feltevésekkel. Számold ki az új MDE-t és a 10 százalékpontos hatáshoz tartozó erőt.

6. **Tervfegyelem.** Az adatok első megtekintése után a stakeholder azt kéri, hogy a primer endpoint konverzió helyett a bevétel legyen, mert „ott látszik igazán a hatás”. Miért sérti ez a megerősítő elemzési logikát, és mi a korrekt eljárás?

40.1.9 Megoldások és indoklás

1. c07 a kezelt karhoz lett randomizálva; az ITT-nevező a randomizálás tényéhez kötődik, nem a kimenethez. A kimenet szerinti utólagos szűrés (*post-treatment conditioning*) szelekciós torzítást okoz: a kezelés hatására konvertáló ügyfelek rendszerszerűen bekerülnek, a kontrollban hasonló rejtett tulajdonságúak nem, így $\widehat{\theta}$ már nem a randomizálás kontrasztját méri. Ellenőrzés: a 8 fős példában a konvertálókra szűkítve $3/3 - 2/2 = 0$, ami tévesen nulla hatást mutatna.

2. A `how="left"` join az `assignments` oldalról megőriz minden randomizált ügyfelet; ha egy `customer_id`-hez nincs outcomes sor, a céloszlopok `NaN`-ek lesznek. Ekkor dobunk `DesignError`-t. A belső join sorainak megszámlálása (300-e) nem elegendő, mert egy kiesett ügyfél mellett egy máshonnan becsúszó duplikátum is 300 sort adhat; az ITT-nevezőt az `assignments` sorai definiálják, ezért balról kell indulni. Emellett a `validate="one_to_many"` figyelmeztetne az outcomes oldali duplikátumra, a `one_to_one` pedig rögtön hibát dob.

3. A kolléga a g_k küszöböt minor skálán hagyta, miközben a bevételt EUR-ba váltotta: a küszöb így -500 EUR-t jelentene, azaz százszorosan megengedő. A javított küszöb EUR-ban $-5{,}00$, mert 500 minor = 500/100 EUR. A kapudöntés 8 fős példán nem változik, hiszen az alsó határ $-12{,}55$ EUR $-5{,}00$ alatt marad; de a hibás küszöbbel való összehasonlítás -500 EUR-val tévesen megengedő döntést adhat nagyobb mintán is.

4. -250-es küszöbbel a 8 fős példában az alsó határ -1255 minor változatlanul a küszöb alatt marad, tehát a kapu továbbra sem teljesül. Az eljárási pont: ha a küszöbmódosítás az adatok láttán történik, az a megerősítő terv megsértése; a helyes út a tervlap verziózott módosítása (amendment), a módosítás okának és dátumának dokumentálásával, és annak kimondásával, hogy az eredeti, előre rögzített elemzés is lefut. Ha a módosítás túlzottan adatvezérelt, a következtetést feltáró jellegűvé kell minősíteni, és új független mintás megerősítésre van szükség.

5. $n = 300$ mellett $SE_{\text{alt}} = \sqrt{(0{,}18 \cdot 0{,}82 + 0{,}28 \cdot 0{,}72)/300} = \sqrt{0{,}3492/300} \approx 0{,}03412$, tehát $MDE = 2{,}8016 \cdot 0{,}03412 \approx 0{,}0956$, vagyis **9,6 százalékpont (rögzített alternatívára számított közelítő skála, nem önkonzisztens MDE)**. A 10 százalékpontos hatáshoz a nemcentralitás $0{,}10/0{,}03412 \approx 2{,}93$, az erő $P(N(0,1) > 1{,}96 - 2{,}93) = P(N(0,1) > -0{,}97) \approx 0{,}83$, tehát körülbelül 83%. Ellenőrző invariant: a karonkénti n négyszerezése 150-ről 300-ra az MDE-t $1/\sqrt{2}$-vel csökkenti: $13{,}5/\sqrt{2} \approx 9{,}55$, egyezik.

6. Az endpoint adatpillantás utáni cseréje a *confirmatory* keretet rombolja: a tesztelésre kiválasztott mutató ekkor már a megfigyelt mintázathoz igazodik, ami felfújja az elsőfajú hibát, és a jelentés „megerősített" állítása nem lesz hiteles. A korrekt lépés: az eredeti terv szerinti elemzést lefuttatni és a konverziós eredményt megerősítőként közölni; a bevétel elemzését feltáró fejezetként hozzáadni, megjelölve, hogy ez post hoc nézet; ha a bevétel valóban primer döntési cél, akkor új, független kísérletet kell tervezni, amelyben már bevétel az előre rögzített endpoint.

40.1.10 Gyakori tévedés ellenpéldával

A leggyakoribb tervhiba a követési idő utólagos összemosása: **az eltérő követési időt nem lehet azonos hétnapos endpointnak nevezni.** Ellenpélda: két ügyfél, A és B, mindkettőnél `observed_until = 2025-07-08`, de A `assigned_at` értéke 2025-07-01, B-é 2025-07-06. Ha a `conversion_7d` mezőbe mindkettőnél 0 kerül, akkor A esetében ez valódi 7 napos nulla, B esetében viszont csak 2 napnyi megfigyelés utáni nulla. Ha a bevezetés későbbi hullámban érinti a kezelt kart, a kezelt ügyfelek rendszerszerűen rövidebb ablakot kapnak, és a becsült kezelési hatás lefelé torzul – nem a kezelés rossz, a mérés nem egyenlő. A Northstar-szerződés ezt azzal zárja ki, hogy minden sornál azonos `assigned_at` és azonos `observed_until` szerepel; a tervben ezért ez nem díszítő `assert`, hanem endpoint-definíciós ellenőrzés. Ha a valódi üzleti rendszer változó követési időt produkálna, akkor cenzorált megfigyelési szerződésre és túléléses elemzésre kellene váltani, ami a 36.1. alfejezetben bevezetett késői címkék problémájának kísérleti rokona, nem pedig „majd a hétnapos mezőbe írjuk be, amink van".

Önellenőrzés. A terv akkor jó, ha mind a négy kérdésre igennel válaszolhatunk: (1) a nevező minden randomizált ügyfelet tartalmaz, és sehol sem a kimenet szerint szűrünk; (2) minden sornál azonos a megfigyelési ablak hossza; (3) az MDE-t adatpillantás előtt írtuk be a tervlapra; (4) a guardrail-küszöb mértékegysége minor egységben konzisztens a bevétel-oszloppal. Ha az alsó CI-határ a küszöb környékén van, a döntésnek „további adat"-ot kell mondania, nem a pontbecslés felé kerekíteni.

40.1.11 Rövid összefoglaló

Az előzetes elemzési terv a randomizált Northstar-kísérlet következtetését fagyasztja be: elemzési egység az ügyfél, a nevező mind a 300 randomizált sor, primer endpoint a rögzített hétnapos `conversion_7d`. Az ITT a kimenet szerinti szűrést tiltja; a guardrail a `revenue_7d_minor` egyoldalas 95%-os alsó határát a -500 minor küszöbhöz méri. A tervezési MDE 150 fős karoknál 13,5 százalékpont, ezért a 10 százalékpontos hatásra a vizsgálat csak körülbelül 54% erővel bír, és a null eredmény nem egyenértékűségi állítás. A következő 40.2. alfejezet ebből a befagyasztott tervből futtatja a becslést, intervallumot és érzékenységvizsgálatot.

40.2 Becslés, intervallum és érzékenység

Belépő · szint: haladó · idő: 90 perc · eszköz: SciPy, statsmodels

Előfeltétel: 40.1. alfejezet · Célkészség: Becslés, intervallum és érzékenység · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Előzetesen rögzített vizsgálat: a permutációs teszt az éles nullhipotézis alatt, az eredeti randomizációs séma reprodukálásával igazolható. A percentilis bootstrap és a Wald-intervallum kis mintán vagy szélső arányok mellett pontatlanul fedhet; az alkalmazott intervallumeljárásnak szerepelnie kell az előzetes tervben.

40.2.1 Cél és konkrét helyzet

A 40.1. alfejezetben befagyasztott kísérleti terv szerint 300 szintetikus ügyfelet randomizáltunk `control` és `treatment` ágra. Az `assignments.csv` egy sor = egy ügyfél, a `customer_id` egyedi, a `variant` oszlop a randomizált beavatkozás. Az `outcomes.csv` ugyanezen ügyfelek teljes, 7 napos kimenetét tartalmazza; a primer végpont a `conversion_7d` bináris változó. A `revenue_7d_minor` másodlagos, folytonos végpont, itt csak érzékenységi szempontból jelenik meg. Az elemzési egység az ügyfél, nem a rendelés vagy a rendelési tétel: a kísérleti nevező 300, és nem szabad az `order_items` sorainak számával felnagyítani.

Bemenet: `lab/data/assignments.csv`, `lab/data/outcomes.csv`. Közbenső tábla: `customer_id` kulcsú join, egy-egy reláció. Kimenet: abszolút hatásbecslés (difference in proportions), 95%-os konfidenciaintervallum, előre rögzített permutációs teszt p-értéke, valamint érzékenységi ellenőrzés. A módszerválasztás indoklása és legalább egy érvényességi korlát szöveges megnevezése szintén átadandó.

Induló becslés: a generáló folyamatot nem ismerjük, de hasonló beavatkozásoknál 5–15 százalékpontos abszolút konverziónövekedés várható. A 300 fős mintán ilyen hatás észlelhető, de az intervallum széles marad: nagyságrendileg ±6–7 százalékpont a 95%-os fél-szélesség. A feladat nem az, hogy „szignifikáns legyen”, hanem hogy a bizonytalanság korrektül, a döntési célhoz illő skálán szerepeljen.

40.2.2 Fogalmi magyarázat

Difference in proportions. Legyenek a karok:

- n_c: kontroll ügyfelek száma,
- n_t: kezelt ügyfelek száma,
- X_c: konvertált kontroll ügyfelek száma,
- X_t: konvertált kezelt ügyfelek száma,
- $\hat{p}_c = X_c/n_c$, $\hat{p}_t = X_t/n_t$,
- $\hat{\tau} = \hat{p}_t - \hat{p}_c$ az abszolút hatás pontbecslése.

Az abszolút különbség mértékegysége konverzió/ügyfél, tehát 0,10 értéke azt jelenti: 100 randomizált ügyfélből várhatóan 10-zel többen konvertálnak a kezelt ágon. Ez az értelmezés közvetlenül használható költség–haszon döntésben.

Standard error és Wald-intervallum. Ha a két minta független binomiális megfigyelés, a különbség becsült szórása

$$SE(\hat{\tau}) = \sqrt{\frac{\hat{p}_c(1-\hat{p}_c)}{n_c} + \frac{\hat{p}_t(1-\hat{p}_t)}{n_t}}.$$

Ez az ún. unpooled vagy Neyman-típusú képlet. Randomizált kísérletben a valódi variancia kismértékben eltér, mert a kimenetelek rögzítettek, és a véletlen a hozzárendelés; a binomiális közelítés azonban a gyakorlatban elfogadható, ha a cella-várható esetszámok nem extrém kicsik. A közelítő 95%-os intervallum

$$\hat{\tau} \pm 1{,}96 \cdot SE(\hat{\tau}).$$

Ez aszimptotikus (nagy mintás) eljárás. A `confidence level` a mintavételi eljárás hosszú távú lefedettségéhez kötött: sok ismételt kísérletben a 95%-os intervallumok körülbelül 95%-a tartalmazná a valódi hatást. Azt **nem** állítja, hogy az aktuális intervallum 95% valószínűséggel tartalmazza a paramétert; a paraméter rögzített, a véletlen az intervallumban van.

Kézi ellenőrzés: legyen $n_c = n_t = 100$, $X_c = 18$, $X_t = 28$. Ekkor $\hat{p}_c = 0{,}18$, $\hat{p}_t = 0{,}28$, $\hat{\tau} = 0{,}10$. A standard error:

$$SE = \sqrt{\frac{0{,}18 \cdot 0{,}82}{100} + \frac{0{,}28 \cdot 0{,}72}{100}} = \sqrt{0{,}001476 + 0{,}002016} = \sqrt{0{,}003492} \approx 0{,}0591.$$

A 95%-os intervallum: $0{,}10 \pm 1{,}96 \cdot 0{,}0591 \approx (-0{,}016,\ 0{,}216)$. A nulla benne van, tehát ekkora mintán ez a hatás nem elkülöníthető a zajtól.

Effect scale. A bináris végpontnál három gyakori skála létezik:

- *Abszolút különbség* (risk difference): $\hat{\tau} = \hat{p}_t - \hat{p}_c$. Döntési szempontból a legközvetlenebb.
- *Relatív kockázat* (relative risk): $RR = \hat{p}_t / \hat{p}_c$. Az 1-hez viszonyított arány; például $0{,}28/0{,}18 \approx 1{,}56$, azaz 56%-os relatív növekedés. Ugyanez abszolút értékben csak 10 százalékpont.
- *Esélyhányados* (odds ratio): $OR = \frac{\hat{p}_t/(1-\hat{p}_t)}{\hat{p}_c/(1-\hat{p}_c)}$. A fenti számokkal $OR \approx 1{,}77$. Retrospektív mintavételnél hasznos, de itt ritkán az elsődleges cél.

A relatív és abszolút skála közötti választás üzleti kérdés: ha a konverzió alaparánya 2%, akkor a $2\% \to 3\%$ relatíve 50%-os növekedés, abszolút értékben 1 százalékpont. A jelentésben mindkét skálát érdemes feltüntetni, de a következtetéshez előre rögzítsük az elsődlegeset — a terv szerint az abszolút különbséget.

Bootstrap. A bootstrap a mintavételezési bizonytalanságot újramintavételezéssel közelíti. Az elemzési egység az ügyfél, ezért a resamplingnak is ügyfélszintűnek kell lennie. Karokon belül, visszatevéssel húzunk n_c és n_t megfigyelést, újraszámoljuk $\hat{\tau}$-t, ezt B-szer ismételjük. A bootstrap replicák 2,5%-os és 97,5%-os kvantilise adja a percentile-intervallumot. A módszer nem igényli a binomiális közelítést, de továbbra is feltételezi, hogy a megfigyelt minta reprezentatív a sokaságra, és a megfigyelések a resampling egység szintjén függetlenek.

Permutation test. A nullhipotézis: *éles nulla* — a beavatkozás egyetlen ügyfél kimenetére sem hat. Ekkor a kezelési címkék felcserélhetők. A permutációs teszt a megfigyelt kimeneteleket megtartja, a hozzárendelést véletlenül újraosztja, és minden permutációnál kiszámítja a két kar átlagos konverziókülönbségét. A p-érték a null-eloszlásból számított feltételes szélsőség:

$$p = \frac{\#\{|\hat{\tau}_{\text{perm}}| \geq |\hat{\tau}_{\text{obs}}|\} + 1}{B + 1}.$$

A +1-es korrekció a Monte Carlo-hibát simítja; $B = 10\,000$ mellett a p-érték numerikus hibája néhány ezred. A permutációs teszt pontosabb kis mintán, mint a Wald-féle normalitás, mert nem támaszkodik aszimptotikára, csak a randomizációra.

Assumptions. Mindhárom eljárás érvényességéhez szükséges:

1. *Véletlen hozzárendelés*: az `assignments.csv` szerint randomizált; ezt a terv rögzíti.
2. *SUTVA / interferencia hiánya*: egy ügyfél kimenete nem függ mások hozzárendelésétől. Ha például a kezelés egy kereskedelmi ösztönző, és az ügyfelek megosztják egymással, sérülhet; e szintetikus laborban feltesszük.
3. *Teljes megfigyelés*: az `outcomes.csv` mind a 300 ügyfélre tartalmazza a 7 napos kimenetet, `observed_until` azonos. Nincs lemorzsolódás. Ha hiányzó kimenet lenne, az ITT-elemzéshez előre definiált imputáció vagy alsó/legrosszabb eset kell; az `exclusion` nem lehet adatvezérelt.
4. *Előre rögzített primer végpont és teszt*: a 40.1. alfejezetben rögzített terv szerint a `conversion_7d` és a permutációs teszt a megerősítő elemzés.
5. *Független megfigyelések* az elemzési egység szintjén. Ügyfélszintű randomizálásnál az ügyfél az egység; ha egy ügyfél több rendelést is adhatna a 7 nap alatt, a kimenetet akkor is egy ügyfélre aggregáljuk.

40.2.3 Teljes minimális munkapélda

A kód a `lab/` gyökérből futtatható. Az alábbi szkript a két CSV-t egyesíti, ellenőrzi a kulcs egyediségét, kiszámítja a pontbecslést, a Wald- és bootstrap-intervallumot, valamint a permutációs tesztet.

```
# lab/s158_estimate.py
from pathlib import Path
import numpy as np
import pandas as pd
from scipy import stats

base = Path("lab")
assign = pd.read_csv(base / "assignments.csv")
outcome = pd.read_csv(base / "outcomes.csv")

# Kötelező ellenőrzés: egy ügyfél pontosan egyszer szerepel.
assert assign["customer_id"].is_unique
assert outcome["customer_id"].is_unique
df = assign.merge(outcome, on="customer_id", validate="one_to_one")
assert df.shape[0] == 300

control = df.loc[df["variant"] == "control", "conversion_7d"].to_numpy()
treat = df.loc[df["variant"] == "treatment", "conversion_7d"].to_numpy()
n_c, n_t = len(control), len(treat)
p_c, p_t = control.mean(), treat.mean()
diff_hat = p_t - p_c

# Wald: két független arány különbségének standard hibája.
se = np.sqrt(p_c * (1 - p_c) / n_c + p_t * (1 - p_t) / n_t)
ci_wald = diff_hat + np.array([-1, 1]) * stats.norm.ppf(0.975) * se

rng = np.random.default_rng(20261020)
B = 10_000

# Ügyfélszintű bootstrap, karokon belül visszatevéssel.
boot_diffs = np.empty(B)
for b in range(B):
```

```
    c_res = rng.choice(control, size=n_c, replace=True)
    t_res = rng.choice(treat, size=n_t, replace=True)
    boot_diffs[b] = t_res.mean() - c_res.mean()
ci_boot = np.quantile(boot_diffs, [0.025, 0.975])

# Permutációs teszt: éles nulla mellett a címkék felcserélhetők.
all_y = np.concatenate([control, treat])
n_total = len(all_y)
perm_diffs = np.empty(B)
for b in range(B):
    perm = rng.permutation(n_total)
    perm_control = all_y[perm[:n_c]]
    perm_treat = all_y[perm[n_c:]]
    perm_diffs[b] = perm_treat.mean() - perm_control.mean()
p_value = (np.sum(np.abs(perm_diffs) >= np.abs(diff_hat)) + 1) / (B + 1)

print(f"n_control={n_c}, n_treatment={n_t}")
print(f"p_control={p_c:.4f}, p_treatment={p_t:.4f}")
print(f"Abszolút hatás (risk difference) = {diff_hat:.4f}")
print(f"Relatív kockázat RR = {p_t / p_c:.2f}")
print(f"Wald SE = {se:.4f}, 95% CI = ({ci_wald[0]:.4f}, {ci_wald[1]:.4f})")
print(f"Bootstrap 95% CI = ({ci_boot[0]:.4f}, {ci_boot[1]:.4f})")
print(f"Permutációs p-érték = {p_value:.4f}")
```

A blokk kimenete a helyi adattól függ. Az alábbi *szemléltető* kimenet a generáló paraméterek nagyságrendjét mutatja, és **nem állítjuk, hogy éppen ezt a futást láttuk**:

```
Szemléltető kimenet - a tényleges értékek a lab adatain futáskor képződnek:
n_control=150, n_treatment=150
p_control=0.1733, p_treatment=0.2867
Abszolút hatás (risk difference) = 0.1133
Relatív kockázat RR = 1.65
Wald SE = 0.0485, 95% CI = (0.0183, 0.2083)
Bootstrap 95% CI = (0.0200, 0.2067)
Permutációs p-érték = 0.0095
```

Ha a tényleges fájlokból más értékek jönnek ki, az a véletlen mintavétel következménye; az elemzés logikája változatlan. A `one_to_one` join-ellenőrzés megakadályozza, hogy véletlenül egy ügyfelet többször számoljunk, ami az ITT-nevezőt hamisítaná.

40.2.4 Eredmény értelmezése

A fenti szemléltető számok alapján: a kezelt ágon a konverziós arány körülbelül 28,7%, a kontrollon 17,3%, az abszolút különbség 11,3 százalékpont. A relatív kockázat 1,65: a kezelés 65%-os relatív konverziónövekedéssel jár. A 95%-os intervallum mindkét közelítésben kizárja a nullát, tehát a hatás a véletlentől elkülöníthető. A p-érték kerekítve 0,010: ilyen vagy szélsőségesebb különbség az éles nulla mellett a permutációk körülbelül 1%-ában fordul elő.

Fontos, hogy a p-érték **nem** a nullhipotézis igazságának valószínűsége, és nem a hatás nagyságát méri. A hatás nagyságát a pontbecslés és az intervallum hordozza: a legvalószínűbb érték 11 százalékpont körül van, de a 95%-os bizonytalansági sáv nagyjából 2–21 százalékpont. Ha a döntési küszöb például 5 százalékpont, akkor a legkisebb érdemi hatást (MDE) a terv szerint 10 százalékpontra állítottuk; ez a minta az alsó határon mozog, a következtetést tehát óvatosan, a teljes intervallum együttes bemutatásával kell megfogalmazni.

A Wald- és a bootstrap-intervallum hasonló, mert a mintaméret elég nagy a binomiális normalitáshoz. A bootstrap akkor is használható, ha a normális közelítés kérdéses; itt a két módszer egyezése önmagában diagnosztikai jel, nem „több szignifikancia”.

Az érvényességi korlát: a szintetikus adatban nincs lemorzsolódás, nincs késleltetett címke, és az ügyfelek nem lépnek interakcióba. Valós éles környezetben a 36.2. alfejezetben tárgyalt késő címke vagy visszacsatolási hurok az ITT-elemzést csak akkor nem torzítja, ha a kimenet a teljes ablakban megfigyelt. Itt a `observed_until` minden sorban azonos, ezért ez a feltétel ellenőrzött.

40.2.5 Feladatok

1. Értelmezés. A fenti szemléltető kimenet alapján írjon egy pontos mondatot a hatásról és bizonytalanságáról, amely nem állítja, hogy „a kezelés 95% valószínűséggel növeli a konverziót”, és nem kezeli a p-értéket a nullhipotézis valószínűségeként. Indokold, miért hibás a „95%-os valószínűség” megfogalmazás.

2. Megvalósítás kézi kontrollal. A következő kis adaton ellenőrizze a permutációs tesztet úgy, hogy az összes lehetséges permutációt végignézi (exhaustive), ne Monte Carlo-mintát használjon. Kontroll: `[0, 0, 1, 0, 1, 1, 1, 0, 0, 1]`, kezelt: `[1, 0, 1, 1, 0, 1, 1, 1, 0, 1]`. Számítsd ki $\hat{\tau}$-t, a Wald-SE-t és a pontos permutációs p-értéket. Hány lehetséges hozzárendelés van, ha a mintaelemszámok rögzítettek?

3. Hibadiagnózis. Egy kolléga a következő kóddal próbál bootstrap intervallumot számolni, de véletlenül az `order_items` táblát is bejoinolja, mielőtt a konverziót aggregálná:

```
df_bad = (assign.merge(outcome, on="customer_id")
                .merge(order_items, on="order_id", how="left"))
# ... majd df_bad-ból bootstrap replikákat húz soronként
```

Magyarázd el, miért torzítja ez a konverziós arányok bootstrap-eloszlását, és hogyan javítaná a kódot az elemzési egység megtartásával.

4. Megváltozott feltétel. Tegyük fel, hogy a kimenet 5%-a hiányzik, és a hiányzás a kezelt ágon gyakoribb, mert a kezelés mobilalkalmazás-értesítést küld, és néhány ügyfél törli az alkalmazást a 7 napos ablak előtt. Hogyan változtatná az elemzést? Nevezd meg legalább két lehetséges, előre rögzíthető kezelési módot, és indokold, miért nem elfogadható a hiányzó sorok egyszerű elhagyása ITT-elemzésben.

40.2.6 Megoldások és indoklás

1. megoldás. Helyes mondat: „A kezelt ágon a konverziós arány 11,3 százalékponttal magasabb (95% CI: 1,8–20,8 százalékpont); a permutációs teszt p = 0,010, tehát az éles nulla mellett a megfigyelt különbség a permutációs eloszlás szélső 1%-ában van.” A „95% valószínűséggel növeli” megfogalmazás azért hibás, mert a konfidenciaszint az eljárás ismételt mintavételi lefedettségére vonatkozik, nem az aktuális paraméterre vonatkozó szubjektív hitre. A paraméter rögzített; a véletlen az intervallum helyzetében van.

2. megoldás. A minták: kontroll $n_c = 10$, összeg 5, tehát $\hat{p}_c = 0{,}5$; kezelt $n_t = 10$, összeg 7, tehát $\hat{p}_t = 0{,}7$; $\hat{\tau} = 0{,}2$. Wald-SE:

$$SE = \sqrt{\frac{0{,}5 \cdot 0{,}5}{10} + \frac{0{,}7 \cdot 0{,}3}{10}} = \sqrt{0{,}025 + 0{,}021} = \sqrt{0{,}046} \approx 0{,}2145.$$

A lehetséges hozzárendelések száma, ha a karok mérete rögzített 10 és 10: $\binom{20}{10} = 184\,756$. Az összes permutációt végigjárni még számítógép nélkül is sok, de Pythonnal generálható az összes kombináció:

```python
from itertools import combinations
import numpy as np

y = np.array([0,0,1,0,1,1,1,0,0,1, 1,0,1,1,0,1,1,1,0,1])
n_total = len(y)
n_c = 10
obs_diff = y[10:].mean() - y[:10].mean()  # 0.2
all_diffs = []
for c_idx in combinations(range(n_total), n_c):
    c_mask = np.zeros(n_total, dtype=bool)
    c_mask[list(c_idx)] = True
    c = y[c_mask]
    t = y[~c_mask]
    all_diffs.append(t.mean() - c.mean())
all_diffs = np.array(all_diffs)
p_exact = (np.sum(np.abs(all_diffs) >= np.abs(obs_diff)) + 1) / (len(all_diffs) + 1)
print(f"Pontos permutációs p-érték: {p_exact:.4f}")
```

A pontos p-érték a fenti adaton 0,257-hez közeli érték lesz, mert ilyen kis mintán a 0,2 különbség a nulla körüli permutációs eloszlás jelentős részében előfordul. Az ellenőrzés: a Monte Carlo-változat $B = 10\,000$-rel hasonló nagyságrendet ad; az eltérés a Monte Carlo-hiba.

3. megoldás. Az `order_items` join után egy ügyfél annyi sorban jelenik meg, ahány rendelési tétele van. A konverzió bináris értékét minden sorban megismétli a join, így a soronkénti bootstrap az ügyfelek tételszám szerinti súlyozását vezeti be: egy több tételt vásárló ügyfél nagyobb eséllyel kerül a replikába. A bootstrap-eloszlás ezért az ilyen ügyfelek konverzióját felülreprezentálja, a szórást pedig torzítja. Javítás: előbb ügyfélszintű táblát készítsünk, a join előtt aggregáljunk, vagy a bootstrapet kizárólag az `assignments + outcomes` egy-egy relációjú dataframe-en futtassuk. A `validate="one_to_one"` már a merge-nél hibát dob, ha nem egyedi a kulcs.

4. megoldás. A hiányzó sorok egyszerű elhagyása sértheti a randomizáció által biztosított összehasonlíthatóságot: ha a hiányzás a kezeléstől függ, a megmaradó minta már nem éles nulla mellett sem cserélhető fel. Két előre rögzíthető lehetőség: (a) *Worst-case imputáció*: a hiányzó konverziót a kezelt ágon 0-nak, a kontroll ágon 1-nek kódoljuk, így a hatás alsó korlátját konzervatívan becsüljük; (b) *Többszörös imputáció* vagy *stratifikált elemzés* a hiányzási mechanizmus feltételezett okait (például app verzió, értesítési beállítás) bevonva. Az ITT-elv szerint a nevező továbbra is minden randomizált ügyfél; a kizárás csak akkor megengedett, ha a kizárási kritérium a randomizáció előtt mérhető és előre rögzített.

40.2.7 Gyakori tévedés ellenpéldával

Ugyanazt a mintát sok módszerrel megvizsgálni, majd a legkedvezőbb eredményt kiválasztani, exploratív szelekció. A Wald-, bootstrap- és permutációs eljárás nem független teszt, de ha például húsz különböző kiértékelési döntést próbálunk ki (más végpont, más ablak, más szűrés), és a legkisebb p-értéket jelentjük, a hamis pozitív esély összeadódik. Éles nulla mellett minden 5%-os szintű teszt önmagában 5% eséllyel téved; húsz független tesztnél annak valószínűsége, hogy legalább egy szignifikáns lesz, $1 - 0{,}95^{20} \approx 0{,}64$.

Szemléltető szimuláció:

```python
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
m = 20
reps = 1000
```

```
min_p = np.empty(reps)
for r in range(reps):
    pvals = np.empty(m)
    for j in range(m):
        x = rng.normal(0, 1, 50)
        y = rng.normal(0, 1, 50)
        pvals[j] = stats.ttest_ind(x, y).pvalue
    min_p[r] = pvals.min()
fp_rate = (min_p < 0.05).mean()
print(f"Húsz párhuzamos teszt után a legkisebb p < 0.05 aránya: {fp_rate:.2f}")
```

A várható kimenet körülbelül `0.64`, nem `0.05`. Ezért a 40.1. alfejezetben a primer végpontot, a tesztet és az elemzési egységet előre rögzítettük. Az itt elvégzett Wald- és bootstrap-intervallum ugyanazt a becslési célt ellenőrzi; a kettő együttes bemutatása diagnosztikai, nem többszörös tesztelés. Ha további elemzéseket (szegmens szerint, másodlagos végpont) is futtatunk, azokat külön, exploratív címkével kell közölni a 40.3. alfejezetbeli jelentésben.

40.2.7.1 A permutációs p-érték és a bootstrap CI feltételei

A három eljárás más statisztikai kérdésre válaszol, ezért nem kötelező, hogy a p-érték és a CI ugyanúgy zárja ki a nullát:

Eljárás	Mit feltételez	Mit ad	Mikor használjuk
Wald z-teszt és CI	nagy mintás normalitás, független egységek, stabil variancia	p-érték és szimmetrikus CI ugyanabból a modellből	nagy n, arányok nem extrémek
Permutációs teszt	éles null (`Y` `Z`), cserélhetőség a randomizáció szerint	p-érték; CI csak invertálással	kis minta, randomizált kísérlet
Percentilis bootstrap	a megfigyelt egységek reprezentatívak, közelítjük a becslő mintavételi eloszlását	CI; p-érték csak külön bootstrap-teszttel	ferde eloszlás, nem normális becslő

A **permutációs p-érték** az éles nullhipotézisre vonatkozik: ha a kezelésnek nincs hatása egyetlen ügyfélre sem, a kimenetek cserélhetők a karok között. A **Wald-teszt** a populációs aránykülönbség nulla voltát teszteli nagy mintás közelítéssel. A **bootstrap CI** a becslő mintavételi eloszlását közelíti, nem a nullhipotézist teszteli. Ezért előfordulhat, hogy a Wald CI éppen kizárja a nullát, a permutációs p-érték viszont 0,06; ez nem logikai ellentmondás, hanem eltérő nullmodell és közelítés.

A jelentésben minden szám mellett szerepeljen a **pontos módségnév**: „Wald 95% CI”, „percentilis bootstrap 95% CI (B=9999)”, „kétoldali permutációs p (B=10000)”. A `method` mező a JSON-ban is kötelező; a „95% CI” önmagában nem elég. A `tests/test_permutation_bootstrap.py` a módszerneveket, a feltételek dokumentálását és azt teszteli, hogy a kód nem kényszerít mesterséges egyezést a Wald- és a permutációs eredmény közé.

40.2.8 Rövid összefoglaló

Az abszolút konverziókülönbség, a hozzá tartozó standard error, bootstrap-intervallum és permutációs p-érték együtt ad korrekt kísérleti következtetést. Az elemzési egység az ügyfél, a

resampling is ügyfélszintű; a Wald-közelítés a nagyobb, a permutációs teszt a kisebb mintán pontos. Az intervallum a hatás bizonytalanságát, a p-érték pedig az éles nulla alatti szélsőséget méri — nem a nullhipotézis igazságát. Az érvényességi feltételek (randomizáció, interferencia hiánya, teljes megfigyelés) kimondandók; hiányzó kimenetnél az ITT-elvet konzervatív vagy előre rögzített imputáció őrzi. A módszerek közötti válogatás adatvezérelten exploratív szelekció, és felfújja a hamis pozitív arányt. A következő 40.3. alfejezetben a tudományos jelentés és reprodukálható bemutató foglalja egységbe ezt az elemzést.

40.3 Tudományos jelentés és reprodukálható bemutató

Belépő · szint: haladó · idő: 90 perc · eszköz: SciPy, statsmodels

Előfeltétel: 40.2. alfejezet · Célkészség: Tudományos jelentés és reprodukálható bemutató · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Jelentésminőség: minden numerikus ábrához add meg a nevezőt, a megfigyelési időszakot, a hiányzók számát és az alkalmazott bizonytalanságmérőt. A nem szignifikáns eredmény nem bizonyítja a hatás hiányát; a jelentésben hatásbecslés és intervallum szükséges.

40.3.1 Cél és konkrét helyzet

A II. záróprojekt kísérleti ágának utolsó lépése a tudományos jelentés és egy újrafuttatható bemutató terve. A bemenet a Northstar kísérleti projekt már elemzett adata: `lab/data/assignments.csv` és `lab/data/outcomes.csv`, valamint a szegmenshez szükséges `lab/data/customers.csv`. A megfigyelési egység a randomizált ügyfél, a kimenet a `conversion_7d` bináris érték, a becsült mennyiség a treatment és control kar 7 napos konverziós arányának különbsége. Az `assigned_at` és `observed_until` a protokoll szerinti 7 napos ablakot rögzíti, az ITT-nevező mind a 300 randomizált ügyfél.

A jelentés akkor jó, ha egy olvasó a módszer, az eredmény és a korlát kapcsolatát látja anélkül, hogy a kódot vagy a notebookot megnyitná. A bemutató akkor reprodukálható, ha a közönség tíz perc után a rögzített paranccsal ugyanazokat az összefoglaló számokat és ábrákat tudja előállítani.

Induló feladat. Mielőtt továbbolvas, írjon egy mondatot a jelentés eredmény rovatához, ha az elemzés egy lehetséges kimenete ez lenne: $\hat{\delta} = 0{,}082$, 95%-os percentilis bootstrap CI: $[-0{,}011;\ 0{,}175]$. Fogalmazza meg úgy, hogy a bizonytalanság és a döntési korlát is benne legyen.

40.3.2 Fogalmi magyarázat

40.3.2.1 A kérdés rögzítése

A jelentés nem a kérdéssel kezdődik, hanem azzal, hogy a kérdés mit jelent a mérhető világban. A kísérleti kérdés operacionalizált alakja:

> A véletlenszerűen kezelt Northstar-ügyfelek körében a treatment 7 napos konverziós arányra gyakorolt átlagos oksági hatása mekkora a controlhoz képest, az összes randomizált ügyfelet a hozzárendelt karjában megtartva?

A becsült paraméter tehát az ITT-hatás (intention-to-treat effect):

$$\delta = P(Y(1) = 1) - P(Y(0) = 1),$$

ahol $Y(1)$ és $Y(0)$ a potenciális kimenetek, $Y_i \in \{0, 1\}$, $i = 1, \ldots, n$. A paraméter mértékegysége százalékpont, nem „százalékos relatív növekedés”. A jelentésben a generáló szimuláció paramétere nem szerepelhet érvként; a tanuló feladata a mintából való becslés, nem az igazság visszaolvasása.

40.3.2.2 Az elemzési módszer

A módszer rovat négy dolgot rögzít:

1. **Adatkizárási szabály.** Mivel a Northstar kísérletben minden randomizált ügyfélnek van 7 napos, teljesen megfigyelt kimenete, nincs kizárás. Ha egy valós projektben kiesés történik, az ITT-elemzés a kiesőket a karjukban tartja; a jelentésnek külön meg kell mondania, mit tekintett kiesésnek.
2. **Elsődleges elemzés.** A becslő a kockázatkülönbség (risk difference, RD):

$$\hat{p}_c = \frac{1}{n_c} \sum_{i:Z_i=\text{control}} Y_i, \qquad \hat{p}_t = \frac{1}{n_t} \sum_{i:Z_i=\text{treatment}} Y_i, \qquad \hat{\delta} = \hat{p}_t - \hat{p}_c.$$

Itt $n_c + n_t = n = 300$, Z_i a randomizált kar, Y_i a konverziós indikátor. A képletben minden vektor hossza a megfigyelt ügyfélszám, a karok mérete a manifestből olvasandó.

3. **Bizonytalanság.** A normál közelítéses standard error csak nagy, független mintán értelmezhető:

$$\mathrm{SE}(\hat{\delta}) = \sqrt{\frac{\hat{p}_c(1-\hat{p}_c)}{n_c} + \frac{\hat{p}_t(1-\hat{p}_t)}{n_t}}.$$

A záróprojektben a 40.2. alfejezetben tanult csoporton belüli, ügyfélszintű percentilis bootstrap az elsődleges intervallum. A resampling egység az ügyfél, nem a sor és nem a rendelési tétel. Mindkét karban külön-külön húzunk visszatevésesen n_c, illetve n_t ügyfelet, kiszámítjuk $\hat{\delta}^*$-ot, és a B ismétlésből a 2,5. és 97,5. percentilist vesszük. A seed csak a húzások ismételhetőségét szolgálja, és nem garantál platformfüggetlen bitazonosságot.

4. **Előre rögzített vizsgálatok.** A jelentés megnevezi a primer tesztet és a többi, előre regisztrált elemzést. A szegmensenkénti ábra feltáró érzékenységvizsgálat, nem megerősítő következtetés. Az elemzés után kiválasztott „érdekes” alcsoport nem számít előre rögzített hipotézisnek.

40.3.2.3 Az eredmény és a bizonytalanság közlése

Az eredmény rovat három számot és azok értelmezését tartalmazza: $\hat{\delta}$, a bootstrap CI, és ha a protokoll tartalmazza, a permutációs p-érték. A p-érték nem a nullhipotézis igaz voltának valószínűsége, hanem a nullmodell szerinti szélsőségesség mértéke. A CI ismételt mintavételi lefedettség: hosszú távon az eljárás az esetek 95%-ában lefedi a paramétert; egyetlen intervallumról nem mondjuk, hogy „95% eséllyel tartalmazza az igazi hatást”.

A gyakorlati állítás három elemet köt össze: a becslés nagysága, a CI által megengedett hatások és az üzleti döntési küszöb. Például ha a vállalat 2,5 százalékpontos hatás alatt nem vezetné be a kezelést, akkor a $[-0{,}011;\ 0{,}175]$ intervallum azt jelenti, hogy a mintából még nem zárható ki sem a hatástalanság, sem a 2,5 pont alatti, sem a afeletti hatás.

40.3.2.4 A három ábra és a reprodukálhatóság

A 4–6 oldalas jelentéshez három célzott ábra elég. Mindegyiknek egy kérdése van:

1. **Karok konverziós aránya.** Pontbecslés és ügyfélszintű bootstrap CI hibasávval. A kérdés: mekkora a két kar megfigyelt aránya, és mennyire bizonytalan?
2. **A hatás bootstrap eloszlása.** A $\hat{\delta}^*$ hisztogramja a CI vonalaival. A kérdés: milyen hatásértékek kompatibilisek a mintával?
3. **Szegmensenkénti érzékenység.** A consumer és business szegmens RD-becslése a feltáró jelleggel megjelölve. A kérdés: stabil-e a becslés a fontos ügyfélcsoportokban? Ez nem döntési ábra.

A reprodukálhatóság rovat a bemeneti fájlneveket, a seedet, a csomagverziókat, a parancsot, a kimeneti hash-eket és a környezet nevét adja meg. A nyers `customer_id` nem kerülhet a jelentésbe vagy az ábrákra; a szegmensábra aggregált, a notebook-kimenet pedig titokmentes.

40.3.2.5 Render manifest: minden ábra forrása és nevezője

A „kézzel mentett PNG” nem reprodukálható bizonyíték. Minden ábrához készüljön `render_manifest.json` bejegyzés:

Mező	Példa	Miért kell
`figure_id`	`"fig1_carok"`	az ábra azonosítója
`script_path`, `script_sha256`	`analysis.py`, SHA-256	a generáló kód verziója
`data_files`	`assignments.csv`, `outcomes.csv` hash-e	a bemenet azonosítása
`seed`	`20270213`	a bootstrap/permutáció ismételhetősége
`filters`	`variant in {control,treatment}`	a részhalmaz rögzítése
`denominators`	`n_control=400`, `n_treatment=400`	az ábra nevezője
`axes`	`x: kar`, `y: konverziós arány (%)`	a tengelyek és mértékegységek
`output_file`, `output_sha256`	`fig1_carok.png`, SHA-256	az ábra integritása
`environment`	Python, matplotlib, NumPy	a render környezete

Mező	Példa	Miért kell

```
import hashlib
import json

def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def figure_record(figure_id, script_path, data_files, seed, filters,
                  denominators, axes, output_path, environment):
    return {
        "figure_id": figure_id,
        "script_path": str(script_path),
        "script_sha256": sha256_bytes(script_path.read_bytes()),
        "data_files": {name: sha256_bytes(path.read_bytes())
                       for name, path in data_files.items()},
        "seed": seed,
        "filters": filters,
        "denominators": denominators,
        "axes": axes,
        "output_file": output_path.name,
        "output_sha256": sha256_bytes(output_path.read_bytes()),
        "environment": environment,
    }
```

Újrageneráláskor a CI a manifest hash-ét összehasonlítja; ha az adat, a seed, a szűrő vagy a nevező változott, az új ábra új bejegyzést kap, és a jelentésben a régi ábra nem maradhat csendben. A `tests/test_render_manifest.py` a kötelező mezőket, az adat/seed változásának hash-hatását és a hiányzó nevező elutasítását teszteli.

40.3.3 Teljes minimális munkapélda

Először egy 40 ügyfeles, kézzel ellenőrizhető kontrollpélda:

```
import numpy as np
import pandas as pd

# Kézi kontrollpélda: 20-20 ügyfél, pontosan 4 és 8 konverzió
y = np.array(
    [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]   # control
    + [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]  # treatment
)
df_smoke = pd.DataFrame({
    "customer_id": [f"c{i:02d}" for i in range(40)],
    "variant": ["control"] * 20 + ["treatment"] * 20,
    "conversion_7d": y,
})

def risk_diff(data, treatment="treatment", control="control"):
    t = data.loc[data["variant"] == treatment, "conversion_7d"].to_numpy()
    c = data.loc[data["variant"] == control, "conversion_7d"].to_numpy()
    return t.mean() - c.mean()

def bootstrap_risk_diff(data, seed=20261020, B=10000):
    rng = np.random.default_rng(seed)
    controls = data.loc[data["variant"] == "control", "conversion_7d"].to_numpy()
    treats = data.loc[data["variant"] == "treatment", "conversion_7d"].to_numpy()
    out = np.empty(B)
    for b in range(B):
        c_idx = rng.integers(0, len(controls), size=len(controls))
```

```
        t_idx = rng.integers(0, len(treats), size=len(treats))
        out[b] = treats[t_idx].mean() - controls[c_idx].mean()
    return out

boot = bootstrap_risk_diff(df_smoke)
delta_hat = risk_diff(df_smoke)
ci_lo, ci_hi = np.percentile(boot, [2.5, 97.5])
print(f"n_control={20}, n_treatment={20}")
print(f"RD={delta_hat:.4f}; 95%-os percentilis bootstrap CI=[{ci_lo:.4f}, {ci_hi:.4f}]")
```

A kézi kontroll: $\hat{p}_c = 4/20 = 0{,}20$, $\hat{p}_t = 8/20 = 0{,}40$, $\hat{\delta} = 0{,}20$. A normál közelítéses standard error

$$\mathrm{SE} = \sqrt{\frac{0{,}20 \cdot 0{,}80}{20} + \frac{0{,}40 \cdot 0{,}60}{20}} = \sqrt{0{,}008 + 0{,}012} = 0{,}1414,$$

így a 95%-os normál CI $0{,}20 \pm 1{,}96 \cdot 0{,}1414 \approx [-0{,}077;\ 0{,}477]$. Ez szemlélteti, hogy 40 ügyféllel még egy 20 százalékpontos mintabeli különbség is összefér a nullával.

A Northstar-fájlokra mutató, jelentést és ábrákat előállító parancsfájl váza:

```
"""scripts/experiment_report.py -- kísérleti jelentés számai és ábrái."""
from pathlib import Path
import argparse, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def load(data_dir: Path) -> pd.DataFrame:
    a = pd.read_csv(data_dir / "assignments.csv")
    o = pd.read_csv(data_dir / "outcomes.csv")
    df = a.merge(o, on="customer_id", how="inner", validate="one_to_one")
    assert df["customer_id"].is_unique
    assert df["variant"].isin(["control", "treatment"]).all()
    assert df["conversion_7d"].isin([0, 1]).all()
    customers = pd.read_csv(data_dir / "customers.csv", usecols=["customer_id", "segment"])
    df = df.merge(customers, on="customer_id", how="left", validate="one_to_one")
    return df

def make_figures(df, boot, ci, out_dir: Path) -> dict:
    out = {}
    # 1. ábra: karonkénti arány pontbecsléssel és bootstrap CI-vel
    fig, ax = plt.subplots(figsize=(5, 3.5))
    for k, group in df.groupby("variant"):
        vals = group["conversion_7d"].to_numpy()
        b = np.array([vals[np.random.default_rng(20261020 + j).integers(0, len(vals),
 ↪  len(vals))].mean()
                      for j in range(2000)])
        lo, hi = np.percentile(b, [2.5, 97.5])
        ax.errorbar(k, vals.mean(), yerr=[[vals.mean()-lo], [hi-vals.mean()]],
                    fmt="o", capsize=6, label=k)
    ax.set_ylim(0, 1); ax.set_ylabel("7 napos konverziós arány")
    fig.tight_layout(); fig.savefig(out_dir / "fig1_carok.png"); plt.close(fig)
    out["fig1"] = "fig1_carok.png"

    # 2. ábra: bootstrap hatáseloszlás
    fig, ax = plt.subplots(figsize=(5, 3.5))
    ax.hist(boot, bins=40, color="0.8", edgecolor="0.4")
    ax.axvline(ci[0], color="k", lw=1, ls="--")
    ax.axvline(ci[1], color="k", lw=1, ls="--")
    ax.axvline(0, color="r", lw=1, alpha=0.7)
    ax.set_xlabel("kockázatkülönbség (százalékpont)")
```

```
    ax.set_ylabel("bootstrap ismétlések")
    fig.tight_layout(); fig.savefig(out_dir / "fig2_bootstrap.png"); plt.close(fig)
    out["fig2"] = "fig2_bootstrap.png"

    # 3. ábra: szegmensenkénti feltáró érzékenység
    fig, ax = plt.subplots(figsize=(5, 3.5))
    labels = []
    for seg in ["consumer", "business"]:
        sub = df[df["segment"] == seg]
        if len(sub) < 20:
            continue
        labels.append(seg)
        est = risk_diff(sub)
        ax.scatter(est, len(labels)-0.5)
    ax.axvline(0, color="k", lw=1, ls=":")
    ax.set_yticks(range(len(labels))); ax.set_yticklabels(labels)
    ax.set_xlabel("feltáró RD (százalékpont)")
    ax.set_title("Szegmensenkénti érzékenység; nem megerősítő hipotézisteszt")
    fig.tight_layout(); fig.savefig(out_dir / "fig3_szegmensek.png"); plt.close(fig)
    out["fig3"] = "fig3_szegmensek.png"
    return out

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--data", type=Path, required=True)
    p.add_argument("--out", type=Path, required=True)
    p.add_argument("--seed", type=int, default=20261020)
    p.add_argument("--B", type=int, default=10000)
    args = p.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    df = load(args.data)
    delta_hat = risk_diff(df)
    boot = bootstrap_risk_diff(df, seed=args.seed, B=args.B)
    ci = np.percentile(boot, [2.5, 97.5]).tolist()
    figs = make_figures(df, boot, ci, args.out)
    result = {
        "n": int(len(df)),
        "n_control": int((df["variant"] == "control").sum()),
        "n_treatment": int((df["variant"] == "treatment").sum()),
        "delta_hat": float(delta_hat),
        "ci_95_percentile_bootstrap": ci,
        "seed": args.seed,
        "B": args.B,
        "figures": figs,
    }
    (args.out / "result.json").write_text(json.dumps(result, indent=2))

if __name__ == "__main__":
    main()
```

A futtatási parancs projektgyökérből, a `lab/` könyvtárban lévő projekt esetén:

```
# Linux/macOS, munkakönyvtár: a lab projekt gyökere
uv run --locked python scripts/experiment_report.py \
  --data data --out artifacts/report --B 10000

# Windows PowerShell, munkakönyvtár: a lab projekt gyökere
uv run --locked python scripts/experiment_report.py `
  --data data --out artifacts/report --B 10000
```

A fenti echo nélküli ábrarészlet a 3. ábrán egyszerű pontokat rajzol CI nélkül; a teljes záróprojektben a 40.2. alfejezetben tanult módon minden szegmenshez külön csoporton belüli bootstrap

CI-t kell számolni. A kódban a 2. ábra hisztogramja a tényleges bootstrap-vektorból készül, tehát az ábra nem önmagában dekoráció, hanem a CI forrását mutatja.

A bemutató terv tíz percre:

Idő	Lépés	Mit mutat	Ellenőrzés
0–1 perc	Kérdés és döntési helyzet	Egy mondatos operacionalizált kérdés	Nincs értelmezhetetlen cél
2–4 perc	Módszer és eredmény	ITT, RD, bootstrap CI	A CI tengelye és mértékegysége helyes
5–7 perc	Három ábra	1. karok, 2. bootstrap, 3. szegmens	Minden ábra aggregált, nincs `customer_id`
8–9 perc	Korlát és következő kísérlet	Bizonytalanság és előre regisztrált folytatás	Nem állít okságot a szegmensábrából
10. perc	Újrafuttatás	A rögzített parancs és hash-összegzés	A közönség el tudja ismételni

40.3.4 Eredmény értelmezése

A kontrollpélda invariánsai: a karok mérete 20-20, az átlagok 0,20 és 0,40, a különbség 0,20. A normál SE 0,1414, ezért a normál CI széles és tartalmazza a nullát. A percentilis bootstrap CI a kis mintán ettől eltérhet, mert a bootstrap a becslő mintavételi eloszlását a minta újramintavételezésével közelíti; nem a normál képletet használja. A Northstar-fájlokon kapott `result.json` a helyi gépen mérendő kimenet; a jelentésben csak a konkrét futásból származó értékek szerepelhetnek.

Ha egy lehetséges elemzési kimenet $\hat{\delta} = 0{,}082$ és a CI $[-0{,}011;\ 0{,}175]$, akkor a helyes megfogalmazás: „A megfigyelt 8,2 százalékpontos különbség a bootstrap CI alapján −1,1 és 17,5 százalékpont közötti hatásokkal is összeegyeztethető; a minta tehát nem dönti el, hogy a hatás a bevezetési küszöb felett van-e, és a nullát sem zárja ki.” A CI értelmezése ismételt mintavételi lefedettség, nem egyetlen intervallum valószínűségi állítása.

40.3.5 Feladatok

1. **Értelmezés.** A jelentő ezt írta: „A kezelés 8,2 százalékpontos konverziónövekedést okozott (95% CI: −1,1 és 17,5; p = 0,082), ezért a kezelést be kell vezetni.” Nevezd meg legalább három pontatlanságot, és írd át egyetlen helyes mondattá.
2. **Megvalósítás.** Egészítsd ki a fenti `make_figures` kódot úgy, hogy a 3. ábrán minden szegmenshez kiszámolja a csoporton belüli ügyfélszintű bootstrap CI-t is, és a pontokat vízszintes hibasávval rajzolja. A függvény ne használjon globális `df`-et; minden adatot paraméterként kapjon.
3. **Hibadiagnózis.** Egy kolléga ezt a bootstrapet írta:

```
def wrong_bootstrap(data, rng, B=10000):
    out = []
    for _ in range(B):
        sample = data.sample(frac=1.0, replace=True, random_state=42)
```

```
        out.append(risk_diff(sample))
    return np.array(out)
```

Magyarázd el, miért nem ez a megfelelő resampling a kísérleti becslés bizonytalanságához, és mi romlik el a CI értelmezésében. Adj javító kódvázat.

4. **Megváltozott feltétel.** A primer kimenetet `revenue_7d_minor`-ra cserélik. A kontrollpéldában a bevételek EUR-centben: control `[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1200,1200,800,800]`, treatment `[0,0,0,0,0,0,0,0,0,0,0,0,1200,1200,1200,1200,800,800,800,800]`. Számold ki az átlagos bevételkülönbséget, és mondja meg, hogyan változik a jelentés képlet- és CI-része.

5. **Önellenőrzés.** Készíts tízpontos ellenőrzőlistát a saját jelentéséhez, amely tartalmazza a kérdés, módszer, eredmény, ábra, reprodukálhatóság és következő kísérlet rovatait. Minden ponthoz írjon egy eldöntendő kérdést.

40.3.6 Megoldások és indoklás

1. megoldás. A mondat hibái: (a) „okozott” helyett a kísérleti elemzés a becsült oksági hatást közli, a CI a becslés bizonytalanságát; a szóhasználat nem törölheti a bizonytalanságot. (b) A CI tartalmazza a nullát, ezért a minta nem szolgáltat erős bizonyítékot a nullától különböző hatásra. (c) A p-érték 0,082 nem a „hatás valószínűsége”, és önmagában nem döntés. (d) A bevezetéshez döntési küszöb és költség-haszon elemzés kell. Helyes mondat: „A pontbecslés 8,2 százalékpont, de a 95%-os bootstrap CI −1,1 és 17,5 között mozog; a minta ezért még nem dönti el, hogy a hatás eléri-e a 2,5 százalékpontos bevezetési küszöböt.”

2. megoldás. A 3. ábra javított váza:

```
def segment_ci(sub, rng, B=2000):
    controls = sub.loc[sub["variant"] == "control", "conversion_7d"].to_numpy()
    treats = sub.loc[sub["variant"] == "treatment", "conversion_7d"].to_numpy()
    out = np.empty(B)
    for b in range(B):
        c_idx = rng.integers(0, len(controls), len(controls))
        t_idx = rng.integers(0, len(treats), len(treats))
        out[b] = treats[t_idx].mean() - controls[c_idx].mean()
    return out

def make_figures(df, boot, ci, out_dir, seed=20261020):
    # ...
    rng = np.random.default_rng(seed)
    for i, seg in enumerate(["consumer", "business"]):
        sub = df[df["segment"] == seg]
        if len(sub) < 20:
            continue
        est = risk_diff(sub)
        b = segment_ci(sub, rng)
        lo, hi = np.percentile(b, [2.5, 97.5])
        ax.scatter(est, i, color="k")
        ax.hlines(i, lo, hi, color="k")
```

A javítás lényege, hogy a bootstrap minden szegmensben is csoporton belül, ügyfélszinten húz, és a CI-t a szegmens saját karjainak méretére építi.

3. megoldás. A kolléga kódja a teljes `data` keretet mintavételezi soronként. Ez három hibát okoz: (a) a resampling egység sor, nem ügyfél; ha egy ügyfél több sorban szerepelne, az összefüggést megtörné. (b) A karonkénti méretek véletlenszerűvé válnak, pedig a randomizálás közel kiegyensúlyozott karjait rögzíteni kell. (c) A `random_state=42` minden húzásnál ugyanazt

a mintát adja, így a `out` vektor konstans, a CI hamisan nulla szélességű lesz. Javítás: a fenti `bootstrap_risk_diff`, ahol a két kar ügyfélértékeit külön tömbökből húzzuk visszatevésesen, és a generátor minden iterációban lép tovább.

4. megoldás. Az átlagok: control $4000/20 = 200$ cent, treatment $8000/20 = 400$ cent, átlagos bevételkülönbség 200 cent, azaz 2,00 EUR. A jelentésben a paraméter mértékegysége EUR-cent; a normál SE képlet nem alkalmazható közvetlenül, mert a bevétel eloszlása erősen jobbra ferde és sok nullát tartalmaz. A bootstrap percentilis CI továbbra is használható, de a pontbecslés a teljes randomizált sokaságra vonatkozik, és a konverziós, valamint a konvertálók költési hatását keveri. A jelentés a bevételt másodlagos kimenetként közli, és nem állít primer oksági következtetést kizárólag ebből.

5. megoldás. Példa lista: (1) A kérdés megnevezi a populációt, beavatkozást, kontrollt, kimenetet és időablakot? (2) A módszer rögzíti az ITT-nevezőt és az adatkizárási szabályt? (3) A becslő és a mértékegység képlettel szerepel? (4) A resampling egység ügyfél, nem sor? (5) A CI értelmezése ismételt mintavételi lefedettség? (6) Minden ábra aggregált, azonosítók nélkül? (7) A seed, verzió és parancs a reprodukálhatósági rovatban van? (8) A korlátok megnevezik a feltáró alcsoportok státuszát? (9) A következő kísérlet előre regisztrált? (10) A jelentés nem tartalmaz titkot, nyers ügyfél-azonosítót vagy naplózott kulcsot?

40.3.7 Gyakori tévedés ellenpéldával

A leggyakoribb hiba az, hogy a vizuális vagy retorikai magabiztosságot bizonyítéknak tekintik. Egy szép hisztogram, egy határozott cím és egy „egyértelmű növekedés” mondat nem erősíti a statisztikai következtetést. Ellenpélda: ugyanaz a 40 ügyfeles minta a 20 százalékpontos mintabeli különbséggel a hisztogramon látványos, a CI mégis $[-0{,}077;\ 0{,}477]$, tehát a nullától való eltérés nem erős bizonyíték. A jelentés akkor korrekt, ha a pontbecslést, a bizonytalanságot és a döntési küszöböt együtt közli. Ez visszautal a 40.2. alfejezet bootstrap feltételeire, a 36.3. alfejezet titokhatárokra és a 28.3. alfejezet seed-korlátra: a reprodukálható kód növeli az ellenőrizhetőséget, de nem helyettesíti a mintavételi bizonytalanság értelmezését.

Önellenőrzés. Tegyen fel három kérdést a jelentés olvasása közben: (1) Meg tudom-e ismételni az elemzést a megadott parancsból? (2) A CI tágassága tükrözi-e a minta méretét, nem a kívánságot? (3) A szegmensábrából levont állítás meg van-e jelölve feltáróként?

40.3.8 Rövid összefoglaló

A kísérleti jelentés a módszer, az eredmény és a korlát egyetlen láncát írja le. A kérdés operacionalizált, a módszer rögzíti az ITT-nevezőt és az ügyfélszintű bootstrapet, az eredmény a pontbecslést és a bizonytalanságot együtt közli. A három ábra a karok arányát, a hatáseloszlást és a szegmensenkénti érzékenységet mutatja, minden azonosító nélkül. A reprodukálhatósági rovat a parancsot, seedet és kimeneti ellenőrző összeget adja, a bemutató pedig tíz percben elismételhető útvonalat kínál. A következő lépés a X. ellenőrző pont: a teljes záróprojekt önálló data scientist munkamenetként való értékelése.

40.4 X. ellenőrző pont: önálló data scientist munkamenet

Belépő · szint: haladó · idő: 90 perc · eszköz: SciPy, statsmodels

Előfeltétel: 40.3. alfejezet · Célkészség: X. ellenőrző pont: önálló data scientist munkamenet · Kimenet: projektartefaktum és ellenőrzött futtatási jegyzőkönyv.

Záró audit: az elfogadási jegyzőkönyv mutassa be a split és a rendelkezésre állási idő ellenőrzését, az adatminőségi hibapályát, a környezetből újrafuttatást, az előzetes statisztikai tervhez való megfelelést és az eredménykorlátok közlését. A sikeres futtatás önmagában nem helyettesíti a módszertani auditot.

40.4.1 Cél és konkrét helyzet

Az X. ellenőrző pont azt méri, hogy a könyv tíz állomásán tanult munkamenetet egyetlen, új feladaton is önállóan végig tudod vinni: a kérdés megfogalmazásától a döntési jelentésig. A korábbi 40.3. alfejezet tudományos közlésre, 39.4. alfejezet átadásra, 36.4. alfejezet működő rendszerre, 28.4. alfejezet módszerválasztásra készített fel; itt ezeknek együtt kell működniük.

A példafeladat egy teljesen új, fiktív szintetikus kísérlet: a **MicroGrove** nevű képzeletbeli bolt 800 véletlenszerűen kiválasztott ügyfele közül 400-400 kap kontrol-, illetve treatment-állapotot. A treatment egy e-mailes kupon, a kontrol egy semleges hírlevél. A döntési kérdés: növeli-e a kupon a rendeléskor megfigyelhető **7 napos konverziót** (*conversion_7d*)? Az adatok nem valós ügyfelektől származnak; a generátor kódja a lentiekben található.

Bemenetek, megfigyelési egység és eredmény. A labor a `lab/experiments/microgrove/` könyvtárban él:

Fájl	Szemcsézettség	Tartalom
`assignments.csv`	egy randomizált ügyfél	`customer_id`, `variant`, `assigned_at`
`outcomes.csv`	ugyanaz az ügyfél	`conversion_7d`, `revenue_7d_minor`, `currency`, `observed_until`
`manifest.json`	fájlszintű	seed, elemszám, CSV SHA-256 lenyomatok
`result.json`	kísérleti szintű	becslés, CI, p-érték

Minden randomizált ügyfél teljes 7 napos ablakon megfigyelt: `assigned_at = 2025-11-03T00:00:00+00:00`, `observed_until = 2025-11-10T00:00:00+00:00`, cenzúra nincs. A generátor és az elemző is ellenőrzi, hogy a különbség pontosan 7 nap. A primer végpont a `conversion_7d`, értéke 0 vagy 1. Az **intention-to-treat** (ITT) nevező mind a 800 randomizált ügyfél, akkor is, ha valaki nem nyitja meg az e-mailt. A `revenue_7d_minor` egész EUR-cent; pénznemkonverzió nincs.

Induló becslés a kód futtatása előtt. Tegyük fel, hogy korábbi kampányok alapján a kontrolkonverzió $p_C \approx 0.20$, a kezelés hatására pedig $p_T \approx 0.28$ várható. $n_T = n_C = 400$ mellett a Wald-féle standard error tervezett értéke

$$\mathrm{SE} = \sqrt{\frac{p_T(1-p_T)}{n_T} + \frac{p_C(1-p_C)}{n_C}} = \sqrt{\frac{0.28 \cdot 0.72}{400} + \frac{0.20 \cdot 0.80}{400}} \approx 0.030.$$

Így a 95%-os konfidenciaintervallum várható szélessége $2 \cdot 1.96 \cdot 0.030 \approx 0.118$, a 0.08 pontnyi hatás mellett a tesztstatisztika várhatóan $z \approx 2.66$ körül lesz. Ez a kézi kontroll lesz az összehasonlítási alapunk; a tényleges minta a véletlen miatt ettől eltér.

40.4.1.1 Tervezési várakozás és a seedelt minta tényleges értéke

A fenti 0.28/0.20 **tervezési feltevés**, nem a generált minta eredménye. A `SEED = 20270213`, `N = 800` generátor NumPy 2.5.4 alatt a következő tényleges értékeket adja:

Mennyiség	Tervezési feltevés	Seedelt tényleges érték
n_T, n_C	400, 400	400, 400
konverziók k_T, k_C	—	114, 82
$\hat{p}_T$	0.28	$114/400 = 0.285$
$\hat{p}_C$	0.20	$82/400 = 0.205$
$\hat{\tau}$	0.08	0.080
Wald SE	0.0301	0.03028
Wald 95% CI	—	[0.0207; 0.1393]
z, kétoldalas p	2.66, kb. 0.0078	2.642, 0.00824

A két oszlop eltérése normális: a tervezés a várható arányokból számol, a minta pedig véges, ezért a konverziók száma és a standard error kis mértékben eltér. A táblázat a **tényleges seedelt minta** kontrollértékeit rögzíti; más NumPy-verzió vagy más generátorhash esetén a pontos számok eltérhetnek, ezért a futtatási jegyzőkönyvben a NumPy-verziót és a generátor SHA-256-ját is rögzíteni kell. A `tests/test_microgrove_seeded.py` a 114/82-es countsokat és a belőlük számolt statisztikákat reprodukálja.

40.4.1.2 A manifest mint teljes elemzési specifikáció

A CSV-hash önmagában nem elég: a manifestnek az **elemzési specifikációt** is rögzítenie kell. A MicroGrove `manifest.json` ezért tartalmazza:

Mező	Példa	Mit véd
`seed`, `n`	`20270213`, `800`	a generátor bemenete
`plan_version`	`"1.0"`	a befagyasztott terv verziója
`contract.parser`	`"pandas.read_csv"`	a beolvasási szabály
`contract.currency`	`"EUR"`	a pénzösszeg értelmezése
`contract.encoding`, `timezone`	`"utf-8"`, `"UTC"`	a szöveg- és időértelmezés
`files`	a két CSV SHA-256	az adatverzió
`hashes.generator_sha256`	a generátor fájl hash-e	a generáló kód
`hashes.analysis_sha256`	az `analysis.py` hash-e	az elemző kód
`environment`	NumPy, pandas verzió	a futtató környezet

A `verify_manifest` a CSV-hashek mellett a `hashes.analysis_sha256` értéket is hasonlítsa össze a tényleges `analysis.py` hashével; eltérés esetén a futás `WARN` vagy `FAIL` legyen, ne csendes továbblépés. A `tests/test_manifest_spec.py` a kötelező mezőket, az elemzőscript-hash eltérését, a hiányzó `plan_version`-t és a parser/pénznem-szerződést teszteli.

40.4.1.3 Zérus standard error és 0/1-es szélsőséges karok

A Wald-képlet $z = \tau/\mathrm{SE}$ alakja zérus standard error esetén nem értelmezhető: a kód `nan` vagy `inf` p-értéket adhat, ami csendesen elfogadott eredménynek látszhat. A z, CI és bootstrap előtt ezért adatkontraktus-tesztek futnak:

Eset	$\hat{\tau}$	SE	Kezelés
0/400 vs 0/400	0	0	`not_estimable`, `zero_standard_error`; nincs p/CI
400/400 vs 0/400	1	0	`not_estimable`; teljes szeparáció, Wald-CI nem értelmezhető
0/400 vs 120/400	0.3	>0	normál Wald/boot út
üres kar	—	—	`ValueError`, nincs ITT-nevező

```
def analyze_counts(k_t, n_t, k_c, n_c):
    if n_t == 0 or n_c == 0:
        raise ValueError("üres kar")
    p_t, p_c = k_t / n_t, k_c / n_c
    tau = p_t - p_c
    se = (p_t * (1 - p_t) / n_t + p_c * (1 - p_c) / n_c) ** 0.5
    if se == 0:
        return {"status": "not_estimable", "reason": "zero_standard_error",
                "tau_hat": tau, "n_treatment": n_t, "n_control": n_c}
    z = tau / se
    return {"status": "ok", "tau_hat": tau, "se": se, "z": z,
            "p_value_two_sided": 2 * (1 - stats.norm.cdf(abs(z)))}
```

A `status` mező kötelező a `result.json`-ban; a `not_estimable` eredmény nem tartalmaz p-értéket vagy CI-t, csak a nyers countsokat és az okot. Ha a csapat mégis pontos tesztet akar, akkor a `method="fisher_exact"` (vagy más néven megnevezett) eljárást külön kell futtatni és jelölni, nem a Wald-képletet `nan`-nal. A `tests/test_zero_variance.py` a 0/0, 1/0, normál és üres kar esetét teszteli.

40.4.2 Fogalmi magyarázat: hét munkafázis egyetlen kísérleten

Az ellenőrző pont nem új módszert vezet be, hanem a tanult fogalmakat rendezi hét fázisba. Mindegyik fázisnak van egy konkrét, ellenőrizhető kimenete.

1. **Problem framing — kérdés pontosítása.** A kérdés nem „jó-e a kupon?”, hanem: „a treatment-felajánlás τ átlagos hatása a 7 napos konverzióra mekkora, és mekkora a becslés bizonytalansága?” A döntési küszöböt előre rögzítjük: a bolt akkor vezeti be a kampányt, ha az abszolút hatás legalább 5 százalékpont (pp) konverziónövekedés. A küszöb nélkül a p-érték önmagában nem üzleti döntés.
2. **Stats — becslés és bizonytalanság.** A két csoport konverzióaránya $\hat{p}_T$ és $\hat{p}_C$; a pontbecslés

$$\hat{\tau}_{\mathrm{ITT}} = \hat{p}_T - \hat{p}_C,$$

ahol $p_T, p_C \in [0,1]$ valószínűségek, n_T, n_C csoportméretek, $\hat{\tau}$ aránykülönbség, szintén $[-1,1]$-ben. A Wald-standard error a fenti képlet, a 95%-os intervallum

$$\hat{\tau} \pm z_{0.975}\,\mathrm{SE},$$

ahol $z_{0.975} \approx 1.96$. A p-érték a nullmodell szerinti feltételes szélsőség: ha $\tau = 0$, mekkora valószínűséggel kapnánk legalább ekkora abszolút eltérést? A p-érték **nem** annak a valószínűsége, hogy nincs hatás. 3. **Modeling — a statisztikai modell.** Itt nem gépi tanulási modell kell, hanem egy kétmintás Bernoulli-modell: $Y_{g,i} \sim \mathrm{Bernoulli}(p_g)$, ahol $g \in \{T, C\}$ csoport, i ügyfél. A modell a csoportátlagok különbségét becsüli. A bootstrap ezt a modellt lazítja: nem követeli meg a normális közelítést, de megtartja a független ügyfélszintű mintavételt. 4. **Code — reprodukálható kód.** A seed rögzített, a fájlok hash-manifesttel védettek, a kód a projektgyökérből `uv run` paranccsal futtatható. Az `analysis.py` nem kér hálózatot, és a bootstrap random state rögzítésével determinisztikus kimenetet ad. 5. **Runtime — végrehajtási közeg.** A feladat CPU-s, 800 sorral és 9999 bootstrap-újramintázással kicsi. A tényleges futási időt a tanuló gépén kell mérni; ebben a szövegben nem állítunk mért másodpercet. 6. **Communication — döntési jelentés.** A kimenet nem csak egy p-érték: pontbecslés, mindkét CI, hatásméret, döntési küszöb és a korlátok is bekerülnek a jelentésbe. 7. **Learning plan — továbbfejlődési célok.** Az ellenőrző pont végén a portfólió alapján három konkrét fejlődési célt kell kijelölni. A céloknak mérhetőnek kell lenniük: például „a következő héten egy saját szintetikus A/B-feladatot a 15.3. alfejezet bootstrap-egységének helyes megnevezésével oldok meg”.

A hét fázisból a statisztikai fázis hordozza a legnagyobb értelmezési kockázatot. A konfidenciaintervallum a **paraméter** bizonytalanságát fejezi ki; az új ügyfél kimenetére vonatkozó predikciós intervallum más, ahogy a credible interval értelmezése is. A MicroGrove-elemzésben végig paraméter-intervallumról beszélünk.

40.4.3 Teljes minimális munkapélda

Hozzuk létre a laborfájlokat. Első a szintetikus generátor; a seed `20270213` szándékosan más, mint a Northstar `20261020` seedje. A generátorban a lognormális kosár paraméterezésekor a $E[X] = \exp(\mu + \sigma^2/2)$ összefüggést használjuk, ezért $\mu = \log(m) - \sigma^2/2$, ahol m a megcélzott kosárátlag centben.

`lab/experiments/microgrove/generate_microgrove.py`:

```
import hashlib
import json
from pathlib import Path

import numpy as np
import pandas as pd

BASE = Path(__file__).resolve().parent
SEED = 20270213
N = 800
ASSIGNED_AT = "2025-11-03T00:00:00+00:00"
OBSERVED_UNTIL = "2025-11-10T00:00:00+00:00"

if (pd.Timestamp(OBSERVED_UNTIL) - pd.Timestamp(ASSIGNED_AT)).total_seconds() != 7 * 24 * 3600:
    raise ValueError("a megfigyelési ablak nem pontosan 7 nap")

rng = np.random.default_rng(SEED)
ids = [f"mg-{i:04d}" for i in range(N)]
```

```
treatment = np.zeros(N, dtype=int)
treatment[N // 2:] = 1
rng.shuffle(treatment)

p_control = 0.20
p_treatment = 0.28
p = np.where(treatment == 1, p_treatment, p_control)
conversion = rng.binomial(1, p).astype(int)

target_basket = np.where(treatment == 1, 7200, 6400)
sigma = 0.45
mu = np.log(target_basket) - 0.5 * sigma**2
basket = np.round(rng.lognormal(mean=mu, sigma=sigma)).astype(int)
basket = np.where(conversion == 1, basket, 0)

assignments = pd.DataFrame(
    {
        "customer_id": ids,
        "variant": np.where(treatment == 1, "treatment", "control"),
        "assigned_at": ASSIGNED_AT,
    }
)
outcomes = pd.DataFrame(
    {
        "customer_id": ids,
        "conversion_7d": conversion,
        "revenue_7d_minor": basket,
        "currency": "EUR",
        "observed_until": OBSERVED_UNTIL,
    }
)
assignments.to_csv(BASE / "assignments.csv", index=False)
outcomes.to_csv(BASE / "outcomes.csv", index=False)

def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

manifest = {
    "seed": SEED,
    "n": N,
    "plan_version": "1.0",
    "contract": {
        "parser": "pandas.read_csv",
        "currency": "EUR",
        "encoding": "utf-8",
        "timezone": "UTC",
    },
    "files": {
        "assignments.csv": sha256(BASE / "assignments.csv"),
        "outcomes.csv": sha256(BASE / "outcomes.csv"),
    },
    "hashes": {
        "generator_sha256": sha256(Path(__file__)),
        "analysis_sha256": sha256(BASE / "analysis.py")
        if (BASE / "analysis.py").exists() else None,
    },
    "environment": {"numpy": np.__version__, "pandas": pd.__version__},
}
(BASE / "manifest.json").write_text(json.dumps(manifest, indent=2))
print("microgrove data generated")
```

Második az elemzés. A `verify_manifest` a CSV-k integritását ellenőrzi, mielőtt bármilyen becslést számolnánk. A bootstrap az **ügyfél** szintjén újramintáz, nem a sorok tetszőleges

keverésével; ez a független mintavételi egység.

lab/experiments/microgrove/analysis.py:

```
import hashlib
import json
from pathlib import Path

import numpy as np
import pandas as pd
from scipy import stats

BASE = Path(__file__).resolve().parent

def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def verify_manifest(base: Path) -> None:
    manifest = json.loads((base / "manifest.json").read_text())
    for name, expected in manifest["files"].items():
        actual = sha256(base / name)
        if actual != expected:
            raise RuntimeError(f"hash mismatch: {name}")
    print("manifest ok")

def diff_mean(t: np.ndarray, c: np.ndarray) -> float:
    return float(np.mean(t) - np.mean(c))

def main() -> None:
    verify_manifest(BASE)

    assignments = pd.read_csv(
        BASE / "assignments.csv",
        dtype={"customer_id": "string", "variant": "string"},
    )
    outcomes = pd.read_csv(
        BASE / "outcomes.csv",
        dtype={"customer_id": "string", "conversion_7d": "int64",
               "revenue_7d_minor": "int64"},
    )
    df = assignments.merge(
        outcomes, on="customer_id", validate="one_to_one"
    )
    if len(df) != len(assignments):
        raise ValueError("missing or duplicate outcome row")
    valid_arms = {"control", "treatment"}
    observed_arms = set(df["variant"].unique())
    if not observed_arms <= valid_arms:
        raise ValueError(f"érvénytelen karcímke: {observed_arms - valid_arms}")
    arm_counts = df["variant"].value_counts().to_dict()
    if arm_counts != {"control": 400, "treatment": 400}:
        raise ValueError(f"karonkénti elemszám eltér: {arm_counts}")
    if df["customer_id"].duplicated().any():
        raise ValueError("duplikált customer_id")
    if df["assigned_at"].nunique() != 1:
        raise ValueError("a randomizáció időpontja nem egységes")
    if not df["conversion_7d"].isin([0, 1]).all():
        raise ValueError("conversion_7d must be 0 or 1")
    followup = pd.to_datetime(df["observed_until"]) - pd.to_datetime(df["assigned_at"])
    if not (followup.dt.total_seconds() == 7 * 24 * 3600).all():
        raise ValueError("a megfigyelési ablak nem pontosan 7 nap")

    y = df["conversion_7d"].to_numpy(dtype=float)
    treat = (df["variant"].to_numpy() == "treatment")
```

```
    y_t, y_c = y[treat], y[~treat]
    n_t, n_c = y_t.size, y_c.size
    if n_t == 0 or n_c == 0:
        raise ValueError("üres kar: nincs értelmezhető ITT-nevező")
    p_t, p_c = y_t.mean(), y_c.mean()
    tau = p_t - p_c
    se = np.sqrt(p_t * (1 - p_t) / n_t + p_c * (1 - p_c) / n_c)
    if not np.isfinite(se) or se <= 0:
        result = {
            "status": "not_estimable",
            "reason": "zero_standard_error",
            "n_treatment": int(n_t), "n_control": int(n_c),
            "p_treatment": float(p_t), "p_control": float(p_c),
            "tau_hat": float(tau),
        }
        (BASE / "result.json").write_text(json.dumps(result, indent=2))
        print(json.dumps(result, indent=2))
        return
    ci_wald = tau + np.array([-1.0, 1.0]) * stats.norm.ppf(0.975) * se
    z = tau / se
    p_value = 2 * (1 - stats.norm.cdf(abs(z)))

    boot = stats.bootstrap(
        (y_t, y_c),
        statistic=diff_mean,
        paired=False,
        n_resamples=9999,
        random_state=20270213,
        method="percentile",
        confidence_level=0.95,
    )

    result = {
        "status": "ok",
        "n_treatment": int(n_t),
        "n_control": int(n_c),
        "p_treatment": float(p_t),
        "p_control": float(p_c),
        "tau_hat": float(tau),
        "se": float(se),
        "ci_95_wald": [float(x) for x in ci_wald],
        "z": float(z),
        "p_value_two_sided": float(p_value),
        "ci_95_percentile": [float(x) for x in boot.confidence_interval],
    }
    (BASE / "result.json").write_text(json.dumps(result, indent=2))
    print(json.dumps(result, indent=2))

if __name__ == "__main__":
    main()
```

A `method="percentile"` a bootstrap-replikák empirikus 2,5% és 97,5% kvantilisét veszi; a **BCa** (bias-corrected and accelerated) ezen felül a torzítást és a hatásmérték változását korrigálja, és jackknife-számítást igényel. A kettő nem ugyanaz az intervallum, ezért a feladat és a JSON kulcs is a percentilis módszert nevezi meg: `revenue_ci_95_percentile`. Ha a jelentés BCa-t használna, akkor a `method="bca"` és a `revenue_ci_95_bca` kulcs lenne a konzisztens pár; a módszernevet és a kulcsot nem szabad keverni.

Futtatás a projektgyökérből, Linux/macOS terminálon; Windows PowerShellben ugyanez a két sor használható:

```
uv run python lab/experiments/microgrove/generate_microgrove.py
```

```
uv run python lab/experiments/microgrove/analysis.py
```

A Python 3.12, NumPy 2.x, pandas 2.2.x és SciPy 1.15 körüli környezet a cél; a pontos feloldást a saját uv lock rögzíti. A kód nem igényel hálózatot, GPU-t vagy külső szolgáltatást.

40.4.4 Eredmény értelmezése

Az alábbi összefoglaló **szemléltető**, az elméleti paraméterekből levezetett várt érték, nem ennek a könyvgenerátornak a futtatási jegyzéke. A tanuló gépén a seed miatt kis mértékben más számok jelennek meg:

```
manifest ok
n_treatment=400, n_control=400
k_treatment=114, k_control=82
p_treatment=0.285, p_control=0.205
tau_hat=0.080
se=0.03028
wald CI=[0.0207, 0.1393]
percentile CI hasonló, kis aszimmetriával
z=2.642, p=0.00824
```

Az ellenőrző invariantok: a beolvasott `df` shape `(800, 7)`; a `customer_id` string dtype, a `variant` string, a `conversion_7d` és `revenue_7d_minor` egész; a merge `validate="one_to_one"` sikeres, tehát nincs hiányzó vagy duplikált kimenet. A `tau_hat` aránykülönbség, pp-re váltva 100-szoros. A p-érték helyes feltételes mondata: *ha a valódi hatás nulla lenne, a megfigyelt vagy szélsőségesebb különbség valószínűsége körülbelül 0.008 a nullmodell szerint.*

A döntési értelmezés a küszöbbel együtt történik. Az 5 pp bevezetési küszöb **előre rögzített** (a következő alfejezet szerint), nem a minta megtekintése után választott érték. A szemléltető CI alsó határa körülbelül 2.1 pp, tehát a minta **nem** erősíti meg, hogy a hatás biztosan eléri a küszöböt; a pontbecslés igen, de a bizonytalanság nagy. A 2 pp-es küszöb kizárólag **érzékenységi elemzés**: megmutatja, mennyire küszöbérzékeny a döntés, de a primer döntést nem írja felül. Ez a különbségtétel a tudományos jelentés magja: a szignifikancia a nullával, a döntés a küszöbbel hasonlít össze.

40.4.4.1 A döntési küszöb előzetes rögzítése

A küszöb akkor véd a poszthoc racionalizálás ellen, ha **az outcome-adatok megnyitása előtt** keletkezett, verziózott jegyzőkönyvben szerepel. A küszöb nem önkényes szám: üzleti **költség–haszon** számításból és a kimutatható **minimális hatásból** (minimum effect) kell levezetni. A MicroGrove esetében a treatment minden ügyfelének kiküldött kupon költsége 0.50 EUR, a konverziónkénti árrés 10.00 EUR. A fedezeti pont:

$$\tau_{\min} = \frac{0{,}50 \text{ EUR}}{10{,}00 \text{ EUR}} = 0{,}05 = 5 \text{ pp}.$$

Ennél kisebb valódi hatás esetén a kampány veszteséges, akármilyen „szignifikáns” a p-érték. A protokoll ezt a levezetést, a küszöb verzióját és a döntési szabályt tárolja:

```
{
  "protocol_version": "microgrove-decision-1.0",
  "frozen_at": "2025-11-02T18:00:00+00:00",
  "outcomes_opened_at": null,
  "primary_threshold_pp": 5.0,
  "cost_benefit": {
```

```
    "incentive_cost_eur_per_treated": 0.50,
    "margin_eur_per_conversion": 10.00,
    "break_even_effect_pp": 5.0
  },
  "decision_rule": "adopt if ci_low_pp >= primary_threshold_pp",
  "sensitivity_thresholds_pp": [2.0]
}
```

A `frozen_at` a küszöb rögzítésének, az `outcomes_opened_at` az outcome-adatok első megnyitásának időpontja; az utóbbinak **későbbinek** kell lennie, különben a küszöb már az eredmény ismeretében született. A validátor a primer küszöböt a fedezeti ponthoz köti, és az érzékenységi küszöböket külön listában tartja; a döntési függvény csak a primer küszöböt olvassa:

```
import math

def break_even_effect_pp(incentive_cost_eur, margin_eur_per_conversion):
    if incentive_cost_eur < 0 or margin_eur_per_conversion <= 0:
        raise ValueError("érvénytelen költség vagy árrés")
    return 100.0 * incentive_cost_eur / margin_eur_per_conversion

def validate_protocol(protocol, outcomes_opened_at):
    if not protocol.get("protocol_version"):
        raise ValueError("hiányzó protokoll-verzió")
    frozen = protocol.get("frozen_at")
    if not frozen or not outcomes_opened_at:
        raise ValueError("hiányzó frozen_at vagy outcomes_opened_at")
    if not frozen < outcomes_opened_at:
        raise ValueError("a küszöb nem az outcome megnyitása előtt rögzült")
    threshold = protocol.get("primary_threshold_pp")
    cost = protocol.get("cost_benefit", {})
    if threshold is None:
        raise ValueError("hiányzó primer küszöb")
    expected = break_even_effect_pp(
        cost.get("incentive_cost_eur_per_treated", -1),
        cost.get("margin_eur_per_conversion", 0),
    )
    if abs(cost.get("break_even_effect_pp", expected) - expected) > 1e-9:
        raise ValueError("a fedezeti pont számítása nem egyezik")
    if threshold + 1e-9 < expected:
        raise ValueError("a primer küszöb a fedezeti pont alatt van")
    if protocol.get("decision_rule") != "adopt if ci_low_pp >= primary_threshold_pp":
        raise ValueError("ismeretlen döntési szabály")
    return True

def decide(ci_low_pp, ci_high_pp, protocol):
    if not (math.isfinite(ci_low_pp) and math.isfinite(ci_high_pp)):
        raise ValueError("nem véges CI")
    if ci_low_pp > ci_high_pp:
        raise ValueError("fordított CI")
    threshold = protocol["primary_threshold_pp"]
    if ci_low_pp >= threshold:
        return "adopt"
    if ci_high_pp < threshold:
        return "reject"
    return "inconclusive"

def sensitivity_check(ci_low_pp, alt_threshold_pp, protocol, primary_decision):
    if alt_threshold_pp not in protocol.get("sensitivity_thresholds_pp", []):
        raise ValueError("a küszöb nem szerepel az előzetes érzékenységi listán")
    if primary_decision not in {"adopt", "reject", "inconclusive"}:
        raise ValueError("ismeretlen primer döntés")
    alt = "adopt" if ci_low_pp >= alt_threshold_pp else "reject"
    return {
```

```
        "exploratory": True,
        "alt_threshold_pp": alt_threshold_pp,
        "would_be_decision": alt,
        "changes_primary_decision": alt != primary_decision,
        "note": "csak érzékenységi elemzés; a primer döntést nem módosítja",
    }
```

A `tests/test_threshold_prereg.py` a fedezeti pont levezetését, az időrend-ellenőrzést, a küszöb alatti primer érték elutasítását, a három döntési ágat és az érzékenységi elemzés elkülönítését teszteli. A 2 pp-es eredmény tehát a jelentésben így jelenhet meg: „a primer, előre rögzített 5 pp küszöb mellett a döntés *inconclusive*; érzékenységi elemzésként 2 pp mellett a CI alsó határa éppen elérné a küszöböt", és soha nem úgy, hogy „valójában 2 pp-et kellett volna használni".

40.4.4.2 Karcímke- és mintaegyensúly-ellenőrzés

Az ITT-számítás előtt a karnyilvántartást validálni kell; a `variant` szabad szöveg, ezért egy elgépelés (`treatmnt`) a `y[~treat]` logikában csendben a kontrollba csúszna, és torzítaná az arányt. A kötelező ellenőrzések:

Ellenőrzés	Elvárás	Hiba esetén
érvényes karcímkék	`{"control", "treatment"}` részhalmaza	`ValueError`, nem csendes besorolás
karonkénti elemszám	400 / 400	`ValueError`, a randomizáció sérült
egyedi `customer_id`	nincs duplikátum	`ValueError`
randomizáció időpontja	egyetlen `assigned_at`	`ValueError`
kimenet	`conversion_7d` ∈ {0,1}	`ValueError`
hiányzó kimenet	nincs, vagy előre rögzített ITT-stratégia	`ValueError` / érzékenység

A fenti ellenőrzések a merge után, de az ITT becslése **előtt** futnak. Egy elgépelt karcímke így `ValueError`-t ad, nem pedig csendes mérési torzítást. A `tests/test_arm_validation.py` az érvényes karcímkéket, az elgépelést, az elemszám-eltérést, a duplikált azonosítót és a hiányzó kontrollkart teszteli.

40.4.5 Feladatok

1. **Értelmezés.** A `scipy.stats.bootstrap` hívásában a `data=(y_t, y_c)` tömbök ügyfélszintű 0/1 értékeket tartalmaznak. Magyarázd el, miért helyes az ügyfél a resampling egység, és miért lenne hibás, ha a bootstrapet például a tranzakciós `revenue_7d_minor` sorokból képzett hosszabb tömbön futtatnánk olyan adatbázisból, ahol egy ügyfélnek több sora is lehet.
2. **Megvalósítás.** Bővítsd az `analysis.py` kódot a `revenue_7d_minor` ITT-elemzésével: csoportátlagok különbsége, standard error, bootstrap percentilis CI. A kimeneti JSON-ban `revenue_tau_hat`, `revenue_ci_95_percentile` kulcsok alatt szerepeljen az eredmény. Ellenőrzésként írd ki a nem konvertálók arányát csoportonként.
3. **Hibakeresés.** Egy másik jegyzetfüzetben valaki ezt írta a p-érték kiszámítására: `p_value = 2 * (1 - stats.norm.cdf(z))`, ahol `z` az előjeles tesztstatisztika. Adj olyan szituációt, amikor ez hibás eredményt ad, és magyarázd meg a javítást.

4. **Megváltoztatott feltétel.** A valódi kampány során a marketingrendszer a treatment csoport 10%-ának nem küldte el az e-mailt. A randomizálás továbbra is érvényes, de a felajánlott kezelés csak 90%-ban jutott el. Hogyan változik az ITT-becslés várható értéke homogén kezelési hatás feltételezése mellett? Milyen érzékenységvizsgálatot javasolnál a jelentésbe?

40.4.6 Megoldások és indoklás

1. feladat. A randomizálás egysége az ügyfél: a treatment sorsolás ügyfélenként független, ezért a becslés bizonytalanságát az ügyfélszintű véletlen ingadozás határozza meg. Ha egy ügyfél több tranzakciós sorral szerepelne, és a bootstrap sorokból húzna, akkor egy sokvásárlós ügyfél többszörösen jelenne meg, a függetlenség sérülne, a standard error pedig jellemzően alulbecsült lenne. A MicroGrove `outcomes.csv` egy ügyfélhez pontosan egy sort rendel, ezért a hiba itt nem lép fel; a feladat arra figyelmeztet, hogy a resampling egységet a kísérleti tervből kell levezetni, nem a fájl sorszámából.

2. feladat. Az `outcomes.csv` `revenue_7d_minor` értékei erősen ferde eloszlást követnek: a nem konvertálók értéke 0, a konvertálók kosara lognormális. A normálközelítés itt gyenge lehet, ezért a bootstrap indokolt. Egy lehetséges bővítés:

```
rev = df["revenue_7d_minor"].to_numpy(dtype=float)
rev_t, rev_c = rev[treat], rev[~treat]
rev_tau = rev_t.mean() - rev_c.mean()
rev_boot = stats.bootstrap(
    (rev_t, rev_c),
    statistic=diff_mean,
    paired=False,
    n_resamples=9999,
    random_state=20270213,
    method="percentile",
)
result["revenue_tau_hat"] = float(rev_tau)
result["revenue_ci_95_percentile"] = [
    float(x) for x in rev_boot.confidence_interval
]
```

Ellenőrző invariant: a groupby-átlagok újraszámolva megegyeznek a `diff_mean` kimenetével 1 centen belül; a nem konvertálók aránya a mintabeli `1 - p_t` és `1 - p_c`, ami a szemléltető paraméterekkel 0.72 és 0.80 körül várható.

3. feladat. Ha a mintából számított hatás negatív, például `z = -2.0`, akkor a hibás kifejezés `2 * (1 - cdf(-2.0)) = 2 * (1 - 0.0228) = 1.9544`-et adna, ami p-értékként lehetetlen. A kétoldali p-értékhez az abszolút eltérést kell nézni: `2 * (1 - stats.norm.cdf(abs(z)))`. A javítás után `z = -2.0` esetén a p-érték körülbelül 0.0455. Az ellenőrzés módja: mesterségesen előjelet váltva a p-értéknek változatlannak kell maradnia.

4. feladat. Ha a felajánlott kezelés csak 90%-ban jut el, és a hatás homogén, akkor az ITT-becslés várható értéke $\tau_{\text{ITT}} \approx 0.9 \cdot \tau_{\text{LATE}}$, ahol τ_{LATE} a ténylegesen kezeltek átlagos hatása. A 0.08 pontos tervezett hatás így körülbelül 0.072 pontra hígul. A protokoll szerinti ITT továbbra is a randomizálásból következő torzítatlan becslés marad, de a döntési jelentésben érzékenységvizsgálatot kell közölni: adjunk felső becslést a ténylegesen kezeltek hatására a felajánlás/elérés aránnyal osztva, és jelezzük, hogy ez erős homogenitási feltétel mellett érvényes. A csoportméret növelése nem pótolja ezt az elemzést.

40.4.7 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés ezen az ellenőrző ponton az, hogy a tanuló a könyv elvégzését vagy egy sikeres demót munkaköri minősítésként kezeli. A tanúsítvány csak a tanulási utat igazolja; a tényleges teljesítményt az új feladaton mutatott önálló következtetés bizonyítja. Két jelölt közül az egyik hibátlan portfóliót mutat, de a MicroGrove p-értékét így közli: „3% az esélye, hogy nincs hatás”; a másik nem végzett el minden opcionális modult, de a p-értéket feltételes állításként fogalmazza, és a CI alsó határát a döntési küszöbhöz méri. Az utóbbi teljesítménye az erősebb.

Egy másik csapda a korábbi Northstar-kódból való kritikátlan másolás: a prediktív projekt időbeli splitje nem alkalmazható egy randomizált kísérletre. A kísérletben nincs train/test split; a randomizálás és a teljes megfigyelés adja az érvényességet. Visszautalva 21.1. alfejezetre: a split a predikciós szivárgást, a randomizálás a confoundingot kezeli; a kettő nem felcserélhető eszköz.

40.4.8 Önellenőrzés

A leadás előtt ellenőrizd ezt a listát:

- A döntési kérdést, a primer végpontot és a küszöböt a kód futtatása előtt rögzítetted.
- Az ITT nevező 800: minden randomizált ügyfél szerepel, kimenet hiány vagy duplikáció nélkül.
- A resampling egység az ügyfél; a bootstrap nem kever sorokat önkényesen.
- A p-értéket feltételes nullmodell-állításként írtad le, nem a nullhipotézis valószínűségeként.
- A CI a paraméterre vonatkozik, nem egy új ügyfél kimenetére.
- A manifest hash-ellenőrzés lefutott, a generátor és az elemzés seedje rögzített.
- A jelentés a pontbecslés mellett a hatásméretet, a CI-t és a döntési küszöböt is tartalmazza.
- A nem futtatott kimenetet szemléltetőként jelölted; tényleges teljesítményhez mértél.

40.4.9 Rövid összefoglaló

Az X. ellenőrző pont azt igazolja, hogy egy új szintetikus kísérleten a kérdéstől a döntési jelentésig önállóan végig tudod vinni a data scientist munkamenetet. A MicroGrove-példa ügyfélszintű randomizálást, teljes megfigyelésű 7 napos konverziót, ITT-becslést, Wald- és bootstrap konfidenciaintervallumot, p-értéket, manifest-hash ellenőrzést, döntési küszöböt és érzékenységvizsgálatot kapcsolt egybe. A teljesítményt nem a könyv elvégzése, hanem az új feladaton mutatott korrekt következtetés bizonyítja. Következő alkalmazásként válassz a saját portfóliódból három konkrét fejlődési célt, és egy újabb, saját tervezésű szintetikus esettanulmányon mérd meg mindegyiket.

40.4.10 Felhasznált és ajánlott módszertani források

A felsorolt tételek válogatott, visszakereshető kiinduló források, nem a könyv minden állításának tételes hivatkozásai. A szoftverfüggvények pontos viselkedését mindig a példaprojektben rögzített verzióval együtt kell ellenőrizni; az itt megadott dokumentációk „stable” ága idővel változhat.

[ISLR] James et al.: *An Introduction to Statistical Learning* (2021), https://www.statlearning.com/ (hozzáférés: 2026-10-21).

[Python] Python Packaging User Guide, https://packaging.python.org/ (hozzáférés: 2026-10-21).

[Pandas] pandas User Guide, https://pandas.pydata.org/docs/user_guide/ (hozzáférés: 2026-10-21).

[DuckDB] DuckDB SQL Documentation, https://duckdb.org/docs/stable/sql/introduction (hozzáférés: 2026-10-21).

[NumPy] NumPy Linear Algebra, https://numpy.org/doc/stable/reference/routines.linalg.html (hozzáférés: 2026-10-21).

[SciPy] scipy.stats Reference, https://docs.scipy.org/doc/scipy/reference/stats.html (hozzáférés: 2026-10-21).

[ASA] Wasserstein & Lazar (2016): *The ASA Statement on p-Values*, DOI: https://doi.org/10.1080/00031305.2016.1154108.

[Hernán] Hernán & Robins: *Causal Inference: What If*, https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/ (hozzáférés: 2026-10-21).

[PyMC] PyMC Documentation, https://www.pymc.io/projects/docs/en/stable/ (hozzáférés: 2026-10-21).

[Sklearn] scikit-learn User Guide, https://scikit-learn.org/stable/user_guide.html (hozzáférés: 2026-10-21).

[SklearnHGB] scikit-learn: Gradient Boosting and Categorical Features, https://scikit-learn.org/stable/modules/ensemble.html (hozzáférés: 2026-10-21).

[SklearnMetrics] scikit-learn Model Evaluation, https://scikit-learn.org/stable/modules/model_evaluation.html (hozzáférés: 2026-10-21).

[SklearnCalibration] scikit-learn Calibration, https://scikit-learn.org/stable/modules/calibration.html (hozzáférés: 2026-10-21).

[PyTorch] PyTorch Autograd Tutorial, https://docs.pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html (hozzáférés: 2026-10-21).

[FastAPI] FastAPI Documentation, https://fastapi.tiangolo.com/ (hozzáférés: 2026-10-21).

[R] R Manuals, https://cran.r-project.org/manuals.html (hozzáférés: 2026-10-21).

Tanulási útvonal és kompetenciatérkép

Egyetlen célprofil: matematika iránt érdeklődő programozó, hiányos statisztikával és kezdő kalkulussal. A könyv nem feltételez előzetes data scientist tapasztalatot. A programozási rutin segít, a statisztikai érvelést és módszerfeltételeket külön építjük.

40.5 Teljes ív és megfigyelhető teljesítmény

Rész	Fejezetek	Elvárt átadandó eredmény
1. Kutatói munkaasztal	sch01-sch04	Tiszta környezetből újrafuttatható kis projekt
2. Adatréteg	sch05-sch08	Kulcs- és időhelyes, ellenőrzött feature-tábla

Rész	Fejezetek	Elvárt átadandó eredmény
3. Matematikai eszköztár	sch09-sch12	Kézi és numerikus kontrollal értelmezett veszteségfüggvény
4. Statisztikai alapok	sch13-sch16	Becslés, intervallum és teszt feltételekkel együtt
5. Statisztikai modellezés	sch17-sch20	Inferenciális, oksági és prediktív kérdés megkülönböztetése
6. Prediktív ML	sch21-sch24	Pipeline, baseline és korrekt modellválasztási protokoll
7. Reprezentáció és szakosodás	sch25-sch28	Feature- és módszerválasztás kontrollkísérlettel
8. Kutatási szoftver	sch29-sch32	Tesztelt, verziózott és átadható modellprojekt
9. Számítás és életciklus	sch33-sch36	Mért erőforrásigény, batch/API és követési terv
10. Eszközválasztás és projektek	sch37-sch40	Két teljes data science projekt és önálló döntési jegyzőkönyv

Az alfejezetek stabil ID-ja s001-s160. Négyfejezetenként ellenőrző pont: s016, s032, s048, s064, s080, s096, s112, s128, s144, s160. Ezek korábbi fogalmakat új összetett helyzetben alkalmaznak.

40.6 Belépési és kilépési feltételek

Belépés. A tanuló tudjon Pythonban függvényt és osztályt írni, parancssort használni, fájlt olvasni és Gitben commitolni. Nem szükséges előzetes statisztikai vagy data science tapasztalat; a kalkulus és a valószínűségszámítás a könyvben, a szükséges szinten épül fel. A lab futtatásához Python 3.12 és az `uv` (vagy ezzel egyenértékű venv/pip) kell; a könyv olvasásához elegendő a HTML- vagy PDF-kiadás.

Kilépés. A tanuló a tíz ellenőrző pontot és a két záróprojektet önállóan teljesíti, a kódját a saját lab-környezetében lefuttatja, az eredményt bizonytalansággal és érvényességi korláttal együtt jelenti, és a döntési jegyzőkönyvben meg tudja indokolni az adat-, modell- és kiértékelési választást. A záróértékelés nem a fejezetek elolvasását, hanem az új, ismeretlen adaton bemutatott munkamenetet méri.

Kötelező és választható ág. A fő út a nem bővítő típusú egységeket tartalmazza; a bővítő egységek (lásd lent) választhatók. A tíz ellenőrző pont és a két záróprojekt kötelező; a bővítő eszközök (PyMC, PyTorch, Dask, Spark, dbt, Streamlit, R) közül legalább egy területen ajánlott elmélyülni, de nem feltétele a kilépésnek.

Időbecslés. A teljes alapút nagyságrendileg 120–180 óra munka, az egyéni előismeretektől és a laborok elmélyítésétől függően; ez tervezési példa, nem garancia. Egy 2000–2800 szavas egység három alkalomra bontható (lásd lent).

40.7 Kompetenciatérkép és kumulatív mérés

A tíz ellenőrző pont nem külön vizsga, hanem a megelőző négy fejezet kompetenciáinak új, összetett helyzetben való alkalmazása. A táblázat a kompetenciát, a hozzá tartozó ellenőrző

pontot és a minimális átadási küszöböt rögzíti.

Kompetencia	Ellenőrző pont	Minimális átadási küszöb
Reprodukálható környezet, adat- és kódverzió	s016	a futtatás jegyzéke, a lock/hash és a notebook újrafuttatása együtt zöld
Kulcs- és időhelyes adatréteg, tisztítás	s032	a három kontrollérték egyezik, a szándékos hibák elkülönülnek
Matematikai levezetés és numerikus kontroll	s048	kézi levezetés + kód az előírt tolerancián belül egyezik
Becslés, intervallum, teszt feltételekkel	s064	az estimand, a CI és a p-érték ugyanazzal a becslővel konzisztens
Statisztikai modellezés és értelmezés	s080	a modell diagnosztikája és a korlát kimondva
Prediktív pipeline, baseline, validáció	s096	a split időhelyes, a preprocessing csak a tanítón illeszkedik
Feature- és módszerválasztás kontrollkísérlettel	s112	a döntés mérésen vagy explicit feltételen alapul
Tesztelt, verziózott kutatási szoftver	s128	a teszt, a CLI és az átadási manifest együtt zöld
Mért erőforrásigény, batch/API, követés	s144	a mérési terv és a drift-/teljesítményriport elkészül
Két teljes projekt és önálló döntési jegyzőkönyv	s160	mindkét projekt új adaton lefut, a döntés indokolt

Küszöb és újrapróbálás. Egy ellenőrző pont akkor teljesített, ha a megoldás kulcselemei legalább 80%-ban megvannak, az invariánsok zöldek, és a hibás eseteket is kezeled. Ha nem, a fejezet megoldásának elolvasása után egy új, saját adaton készített változattal próbáld újra; a puszta megoldás újrafuttatása nem számít újrapróbálásnak.

Záró, ismeretlen adatos teszt. A záróértékelés két, a könyvben nem szereplő szintetikus adathalmazon végzett munkamenet: (1) predikciós feladat időbeli splitpel, baseline-nal, kalibrációval és döntési küszöbbel; (2) randomizált vagy megfigyeléses hatásbecslés estimanddal, CI-vel és érzékenységvizsgálattal. Mindkettőhöz egy legfeljebb kétoldalas döntési jegyzőkönyv tartozik, amely kimondja az adat korlátait és a döntés bizonytalanságát.

40.8 Alapút és választható mélyítés

A fő út minden olyan egységet tartalmaz, amelynek Feladattípus sora nem bővítő. Az outline.md előfeltételei az alapút számára nem teszik kötelezővé a bővítő egységeket. A matematika és statisztika alapblokkja a céltanuló számára fő tartalom, nem fakultatív háttéranyag. Az s073-s074 bayesi alapmagyarázat grid-alapú, PyMC nélkül is követhető.

Választható mélyítések:

- Polars lazy gyakorlata: s031.

- Megfigyeléses oksági adjustment: s071; PyMC/hierarchikus modellezés: s075-s076.
- Feature store-döntés: s100.
- Szöveg/embedding/LLM: s105-s108; PyTorch/GPU: s109-s111.
- dbt szerepe: s120; Dask/Spark/elosztás: s134-s136.
- Streamlit UI: s139; R/Posit/renv: s145-s148.

Az s112 és s128 ellenőrző pont helyi CPU-s alap stackkel is teljesíthető: az alternatív környezetre ellenőrizhető döntési terv elég, ha tényleges futtatását nem állítod. A mélyítés akkor hasznos, amikor a saját probléma vagy munkahelyi stack indokolja.

40.9 Egy lecke használata

Egy 2000-2800 szavas egységet több alkalomra érdemes bontani. Ez időtervezési példa, nem garantált tanulási sebesség vagy munkaköri szintemelkedés.

Alkalom	Tevékenység	Induló időkeret
1.	Rövid előzetes becslés, magyarázat, kézi kis példa	30-50 perc
2.	A minimális kód vagy munkamenet önálló újraírása és ellenőrzése	40-70 perc
3.	Hibakeresés és megváltozott adatra transzfer, megoldás után javítás	30-60 perc
Később	Korábbi fogalom előhívása másik projektben	10-20 perc

Matematikánál ne csak a kód fusson: mondd el, mit jelent minden jel, mi az alak és egység, és milyen nagyságrendű eredményt várasz. Statisztikánál minden állításhoz legyen becslési cél, adatfelvételi feltétel és érvényességi korlát.

40.10 Visszatérés és portfólió

A Visszautalás sorok 4, 16 és 48 leckével korábbi fogalomhoz adnak kapcsolatot. Ez tantervi eltolás, nem naptári ismétlési garancia. Ha bővítő részre utalnak, az ottani fogalom röviden felidézhető; ez nem változtatja a bővítést előfeltétellé.

Őrizd meg a problémalapot, első becslést, kódot, inputazonosítót, környezetet, futási eredményt, hibát és javítást. A portfólióban legalább ezek szerepeljenek:

- Adatkontraktus és join-kontroll; hibás és javított adatfolyam.
- Ábrás matematikai notebook kézi kontrollal.
- Konfidenciaintervallum és teszt helyes, feltételes magyarázattal.
- Időhelyes modellkísérlet baseline-nal és splitmanifesttel.
- Tesztelt modul, futási manifest, mentett modell és új inputos predikció.
- A két záróprojekt jelentése, reprodukciós parancsa és korlátai.

40.11 Belső önellenőrzés

Minden checkpointnál ugyanazt az öt szempontot alkalmazd:

Szempont	Megfigyelhető kérdés
Kérdés és adat	Pontos a cél, megfigyelési egység és rendelkezésreálló információ?
Matematika és statisztika	Meg tudom magyarázni a képletet, feltételt és bizonytalanságot?
Implementáció	Tiszta környezetben fut, és vannak érdemi kontrollok?
Értékelés	Független a teszt a választástól, megfelelő a split és metrika?
Átadás	Másik ember vissza tudja állítani a futást, és érti a korlátokat?

Saját követési skála: 1 = lényeges segítség kell; 2 = részben helyes, fontos hiány maradt; 3 = önállóan helyes az adott feltételben; 4 = új feltételben is helyes, indokolt és ellenőrizhető. Ez saját formatív skála, nem szakmai minősítő vizsga.

Az ellenőrzés alapja a ténylegesen lefuttatott kód és az új feladaton adott indoklás. AI-visszajelzésnél a kódot, inputot, tracebacket és konkrét eredményt add meg; puszta leírásból ne fogadj el állítást arról, hogy a másik fél futtatta a projektet.

A feladatok és a megoldások használata

A könyv minden alfejezetében a **Feladatok** után külön **Megoldások és indoklás** alfejezet következik. A megoldás tehát nem a feladat azonnali folytatása: a feladat és a megoldás külön címsor alatt van, így a megoldás elolvasása nélkül is dolgozhatsz. A könyv nem ad külön nyomtatott feladatfüzetet; a fejezetek sorrendje és a stabil `sNNN` azonosítók adják a feladatgyűjteményt.

40.12 Önértékelés a megoldás megnyitása előtt

A megoldás megnyitása előtt írd le a saját válaszodat, és ellenőrizd az alábbi négy pontot. Ha bármelyik hiányzik, még ne nyisd meg a megoldást.

1. **Megfigyelési egység és nevező.** Megnevezted-e, hogy egy sor mit reprezentál, és mi a számítás nevezője?
2. **Bemenet és kimenet.** Megadtad-e a bemenet alakját, típusát és a kimenet elvárt alakját?
3. **Kézi kontroll.** Kiszámoltál-e legalább egy kis példát kézzel, vagy megadtál-e egy invariánst, amelynek teljesülnie kell?
4. **Értelmezés és korlát.** Kimondtad-e, mire vonatkozik az eredmény, és mire nem általánosítható?

40.13 Nehézségi szintek

A feladatok típusa a szövegben jelölt (értelmezés, megvalósítás, hibakeresés, transzfer). A nehézséget a következő három szint szerint ítéld meg:

Szint	Mit vár el	Önértékelési jel
1. Reprodukció	a fejezet példájának megismétlése saját adaton	a kód lefut, a kézi kontroll egyezik
2. Alkalmazás	új bemenetre vagy megváltozott feltételre való átvitel	az invariáns és a hibaág is helyes
3. Transzfer	új kérdésre, más eszközzel vagy más adatszerkezettel való megoldás	a döntést indoklod, és a korlátot kimondod

A megoldás akkor tekinthető elsajátítottnak, ha a saját válaszod a megoldás kulcselemét és annak indokát is tartalmazza, nem csak a végeredményt.

40.14 Gépi ellenőrzés

A futtatható kódokhoz a könyv tárolója teszteket tartalmaz:

- `tests/test_lab.py` – az adatszerződés, a generátor és a kezdőpélda;
- `tests/test_code_consistency.py` – a könyv kiemelt kódblokkjai és a lab-fájlok;
- `tests/test_transfer.py` – a Python–R átadás sémája és epoch-kontrollja;
- `tests/test_stats.py` – a Wald-intervallum és a p-érték konzisztenciája;
- `lab/tests/test_contract.py` – a csomag önálló szerződéstesztjei.

A megoldás megnyitása után futtasd a megfelelő tesztet; a teszt zöld állapota azonban csak a szerződést igazolja, a saját értelmezésedet nem.

Átadás előtti ellenőrzés

Dátum: 2026-10-20. A csomag tankönyvgenerálási bemenet és kis futó kezdőlabor; nem a teljes megírt tankönyv. Távoli vagy fizetős OpenAI-kérést az ellenőrzés nem indított.

40.15 Tanterv

- 10 rész, 40 fejezet, 160 alfejezet; 200 egyedi könyvazonosító.
- 10 ellenőrző pont és 7 projektlecke a két összefüggő záróprojektben.
- 21 megjelölt bővítő egység; az alapút előfeltételei nem kényszerítik ezek elvégzését.
- Minden egységben egyedi konkrét labor, tanulási cél, témakör, matematikai híd és gyakori tévedés szerepel.
- Minden előfeltétel és visszautalás létező, korábbi egységet jelöl.
- Az alapozás kezdő kalkulusból és hiányos statisztikából indul; a teljes út kifejezetten data scientist munkára vezet, külön elemzői vagy BI útvonal nélkül.

- A teljes célterjedelem 346000 szó irányadó keret, nem garantált kimeneti méret.

Az összes írási input helyben összeállítható. Előzetes kész fejezetek és kutatási jegyzet nélkül a promptok 32833-38032 karakter hosszúak. A teljes, 362216 karakteres outline.md nem másolódik be minden kérésbe. Későbbi kész előzmény és forrásjegyzet a book.toml-ban megadott korlátokig növelheti az inputot.

Nincs előre kész chapters/ vagy state.json. A prompts/ négy fájlja dry-run minta, a FORMATUM_MINTA.md kézzel írt, külön tanítási példa.

40.16 Automatizált próbák

32 helyi ellenőrzés sikeres, kihagyás nélkül:

- tantervi teljesség, egyedi feladatok, alapút és bővítések függősége;
- az aktuális tervből készülő, korlátozott prompt és korábbi matematikai híd;
- kézi szöveg megőrzése, explicit újraírás, backup és konkurens kézi módosítás védelme;
- stream ID és részleges szöveg mentése megszakításkor; automatikus duplakérés blokkolása;
- mentett válasz visszanyerése új create kérés nélkül; hiányos válasz nem kész fejezet;
- saját kutatási instrukció, befejezett web_search megkövetelése és normál forráslink;
- valódi OpenAI SDK stream feldolgozása kizárólag helyi MockTransporttal;
- valódi Pandoc HTML: MathML, beágyazott PNG-ábra, 200 könyvazonosító és belső link;
- hibás exportnál korábbi HTML megőrzése; PDF-motor, font és PATH parancsszintű ellenőrzése;
- adatgenerátor determinisztikus újrafutása, CSV-hash, szándékos hibák és join-ellenpélda;
- tételösszeg, célváltozó, randomizálási egység és teljes 7 napos követés;
- tanítócímkék időhelyes elérhetősége és a tiltott feature-ök kizárása;
- a kézzel írt matematikai minta kódja és kézi kontrolljai egyeznek.

Tesztkörnyezet: Python 3.12.14, OpenAI SDK 3.35.1, Pandoc 3.1.3. A tanulási alapfutás csomagjai: NumPy 2.3.5, pandas 2.2.3, SciPy 1.17.0, scikit-learn 1.8.0, Matplotlib 3.10.8. Nem állítjuk az összes pyproject-tartomány és opcionális extra együttes vagy más platformos tesztelését.

40.17 Tényleges kezdőfutás

A lab/ellenorzott_report.json tényleges helyi CPU-futásból származik.

- 2400 tiszta rendelés, 4796 tételsor; a basket és tételösszeg egyaránt 97383149 EUR-cent.
- A helyes rendelési shipping összeg 1690200 EUR-cent. A szándékosan rossz tételjoin 3380004 EUR-centet ad: ez az oktatási ellenpélda, nem elfogadott eredmény.
- A 2025-05-01 cutoffnál 1531 tanító, 800 teszt és 69 még nem ismert címkéjű, kizárt határsori rendelés szerepel.
- A modell explicit feature-listát és foldfüggetlen egyetlen idősplitet használ; a preprocessing kizárólag a training adaton illeszkedik.
- Az egyszerű logisztikus baseline teszt ROC-AUC értéke kb. 0.9198, average precision értéke kb. 0.7911. Ez kis szintetikus példára mért adat, nem valós üzleti minősítés.
- A külön kísérlet 154 control és 146 treatment ügyféllel fut. A mintabeli konverziókülönbség kb. 0.000178, az ügyfélszintű percentile bootstrap 95%-os intervalluma kb. [-0.09829, 0.09972]. A kicsi minta nem mutatja meg biztosan a generátorba épített 0.10 hatást. Az egyszeri intervallum nem garantálja a valódi paraméter tartalmazását; a lefedettség ismételt mintavételi tulajdonság.

A modell saját mentett artefaktumának visszatöltése ugyanazt a kontrollpredikciót adta. Az ismeretlen eredetű pickle/joblib betöltését a könyv nem kéri.

40.18 Export és ellenőrzési határ

A kézzel írt matematikai mintát valódi HTML-be exportáltuk. A képletek MathML-ként, az ábra beágyazott PNG-ként szerepelnek; az ábrát képen is ellenőriztem. A teljes könyv HTML-vázában mind a 200 azonosító megmarad.

A PDF exportparancsa tesztelt. A tényleges LuaLaTeX PDF-fordítást ebben a környezetben nem állítjuk elvégzettnek: a rendszer-bináris jelen van, de a szükséges formátum- és TeX-csomagfájlok hiányoznak. A teljes telepítéshez parancsok vannak a HASZNALAT.md-ben. Az átadás nem tartalmaz kész PDF-könyvet.

PyMC, R, GPU, Dask, Spark, Docker, cloud és publikus szolgáltatás telepítését nem végeztük el. Az ezekre szóló leckék specifikációk és bővítések. A később generált fejezetek kódját a tanuló saját ellenőrzött környezetében kell futtatni; az írási modell ezt nem helyettesíti.

Eszköz- és környezetterv

Ez választási térkép, nem minden eszköz együttes telepítési előírása. A két környezet különálló: a könyvgenerátor OpenAI SDK-t és ábrarenderert használ; a tanulási lab numerikus Pythont. A tanulólaborhoz API-kulcs nem kell.

Feladat	Elsődleges tanulási választás	Alternatíva vagy mélyítés
Interpreter és csomagok	CPython 3.12, uv projekt és lock	venv/pip; natív stackhez conda/mamba
Interaktív munka	JupyterLab vagy VS Code Jupyter	távoli kernel, managed notebook
Numerikus művelet	NumPy, SciPy	PyTorch tensor/autodiff bővítés
Táblás adat	pandas, explicit schema és .loc	Polars lazy bővítés
SQL és helyi lekérdezés	DuckDB; telepítésmentes kontrollhoz SQLite	PostgreSQL munkahelyi környezet
Adatfájl	CSV kis kontrollhoz, Parquet typed tároláshoz	PyArrow, object storage
Statisztikai modell	SciPy és statsmodels	PyMC/ArviZ, R
Prediktív modell	scikit-learn Pipeline	külön boosting könyvtár vagy PyTorch indokolt esetben
Ábra és tudományos jelentés	Matplotlib/Seaborn, notebook és statikus export	Quarto, R-ökoszisztéma
Kutatási szoftver	tiszta Python-modul, CLI, logging, pytest, Ruff	típusellenőrzés és CI
Pipeline	helyi, idempotens Python/SQL lépések	Prefect/Dagster; SQL-réteghez dbt
Kísérlet és adatverzió	JSON/hash/splitmanifest, Git	MLflow, DVC, registry
Futtatási izoláció	virtuális környezet	Docker és Dev Containers
Számítási bővítés	előbb egygépes profiling és kontrollált szálak	Dask/Spark/CPU/GPU indokolt workloadhoz

Feladat	Elsődleges tanulási választás	Alternatíva vagy mélyítés
Modellátadás	batch CLI és szerződött input	FastAPI, Streamlit helyi demo
Követés	séma-, drift- és címkés teljesítményriport	dedikált monitoring eszköz, szolgáltatás

40.19 Telepítés a feladathoz

A lab/pyproject.toml alapfüggőségei NumPy, pandas, SciPy, scikit-learn és Matplotlib. Az extra-csoportok: notebook, statistics, storage, service, tracking, bayes, deep. Egy extrasor követelményleírás, nem bizonyíték az összes platformos kompatibilitásra. Az uv a lockfeloldás során az extra függőségeket is vizsgálhatja; csak a választott extra települ. Az első feloldás ezért a minimális futásnál több metadata letöltést végezhet.

Alap numerikus környezet a könyv mappájának lab/ könyvtárából:

```
uv lock
uv sync
uv run --locked python quickcheck.py
uv run --locked python starter_experiment.py
```

Notebookos használat:

```
uv sync --extra notebook
uv run --locked python -m ipykernel install --user --name ds-book-lab --display-name "Data
↪  Scientist lab"
uv run --locked jupyter lab
```

A notebookban külön ellenőrizd a sys.executable értékét. A létrehozott uv.lockot és a környezetjelentést őrizd meg. A csomag szándékosan nem ad ellenőrizetlenül kitalált lockfájlt. A tartományokat szükség szerint saját, ellenőrzött munkahelyi verzióprofilra cserélheted. GPU-s PyTorchhoz mindig a célplatform aktuális hivatalos telepítési útmutatója alapján válassz csomagot; a deep extra önmagában nem CUDA-beállítás.

40.20 Platform és infrastruktúra

A kezdőadat 2400 rendelés, CPU-n és kis memóriával feldolgozható. A tanulólabor Linux, macOS és Windows alatt is írható hordozható Pythonban. A mellékelt könyvindító Unix flock zárolása Linux/macOS/WSL környezetet céloz.

SSH, Docker, CUDA, Dask, Spark és managed cloud hozzáférése külön környezeti feltétel. A leckéknek legyen kis helyi alternatívája, vagy a futtatást igényként, nem végrehajtott tényként kell jelölniük. A könyv nem hoz létre felhőfiókot, klasztert, publikus API-t és fizetős erőforrást.

40.21 Verziószabály

Az író ne mondjon „legfrissebb” verziót emlékezetből. Az ellenőrzött helyi alapfutás verziói az ELLENORZES.md-ben és a mellékelt lab/ellenorzott_report.json-ban vannak. Más eszköz dokumentációs linkje a sources.md szerint lehet puszta kiindulópont. A research fejezetenként ellenőrizhet aktuális API-t, dátummal.

Verziómátrix és támogatott környezet

A könyv szövegében szereplő futtatási számok és képernyőképek a 2026-10-20-i ellenőrzött környezetre vonatkoznak. A tanuló saját projektjében a pontos verziókat mindig az `uv.lock` (Python) és a `renv.lock` (R) rögzíti; a `pyproject.toml` csak a megengedett tartományt adja meg.

40.22 Ellenőrzött alapkörnyezet

Elem	Ellenőrzött verzió	Verziózott dokumentáció
Python	3.12.14	https://docs.python.org/3.12/
NumPy	2.3.5	https://numpy.org/doc/2.3/
pandas	2.2.3	https://pandas.pydata.org/pandas-docs/version/2.2/
SciPy	1.17.0	https://docs.scipy.org/doc/scipy-1.17.0/
scikit-learn	1.8.0	https://scikit-learn.org/1.8/
Matplotlib	3.10.8	https://matplotlib.org/3.10.8/
Pandoc (könyvexport)	3.1.3	https://pandoc.org/releases.html

A 2026-10-29-i lockfeloldás-ellenőrzés (uv 0.13.6, a `lab/pyproject.toml` tartományai alapján) pandas 2.3.3-at, NumPy 2.5.4-et és Matplotlib 3.11.2-t oldott fel; ez mutatja, hogy a tartomány felső vége új verziót adhat. A könyv példáinak ellenőrzött számai ezért a 2026-10-20-i környezethez tartoznak, a tanuló futása ettől kismértékben eltérhet.

40.23 A lab támogatott tartományai

A `lab/pyproject.toml` szerint:

Csomag	Tartomány	Megjegyzés
Python	`>=3.12,<3.13`	a tanulási alapkörnyezet
NumPy	`>=2.0,<3`	a 2.x API-t használjuk
pandas	`>=2.2,<3`	Copy-on-Write viselkedés a 2.2-től
SciPy	`>=1.15,<2`	`stats.bootstrap`, `stats.norm`
scikit-learn	`>=1.5,<2`	Pipeline, ColumnTransformer
Matplotlib	`>=3.8,<4`	ábraexport

Az opcionális extrák (`notebook`, `statistics`, `storage`, `service`, `tracking`, `bayes`, `deep`) külön csoportok; a könyv alapútja nem telepíti őket.

40.24 R és Quarto

Az R-oldali példákhoz R 4.4 vagy újabb, `arrow`, `dplyr`, `jsonlite` és Quarto szükséges. A `renv.lock` a projekt saját feloldását rögzíti; a könyv a csomagok konkrét verzióját nem hardcode-olja, mert az R-környezet a fogadó szervezetnél eltérhet.

Fogalomtár (magyar–angol)

A táblázat a könyv kulcsfogalmait rögzíti egységes magyar és angol névvel. A hivatkozás az első kidolgozó alfejezet; a fogalom később visszatér. A fogalmak közötti függést a **Fogalmi térképek** függelék (`FOGALMI_TERKEP.md`) mutatja, a fejezetenkénti matematikai jelöléseket a **Jelképjegyzék** (`JELOLESEK.md`).

Magyar	English	Jelentés / megkülönböztetés	Első előfordulás
becslés	estimate	a mintából számított konkrét érték; nem azonos a becslési céllal	s001
becslési cél	estimand	a sokasági, ismeretlen mennyiség, amelyre következtetni akarunk	s001, s049
megfigyelési egység	observation unit	az a dolog, amelyről egy sor szól (ügyfél, rendelés, tétel)	s001
tanítóhalmaz	training set	a modell illesztésére használt adat	s084
validációs halmaz	validation set	modell- és hiperparaméter-választásra használt, a tanítótól elkülönített adat	s086
teszt halmaz	test set	a végső, egyszeri kiértékelésre tartott adat	s084
időbeli split	temporal split	a jövőbeli adat elkülönítése a tanítótól idő szerint	s002, s081
purged split	purged split	a tanító és teszt közötti címke-átfedés kizárása a cutoff körül	s096
adatszivárgás	data leakage	a predikció pillanatában nem ismert információ bekerülése a feature-ökbe	s002
konfidenciaintervallum	confidence interval (CI)	gyakorisági eljárás ismételt mintákra vonatkozó lefedettsége	s056

Magyar	English	Jelentés / megkülönböztetés	Első előfordulás
hiteles intervallum	credible interval	a posterior eloszlásból származó intervallum; nem azonos a CI-vel	s058, s074
p-érték	p-value	a nullmodell mellett a megfigyeltnél legalább ilyen szélsőséges eredmény valószínűsége	s057
szignifikancia	statistical significance	a nullhipotézissel való összevetés eredménye; nem azonos a gyakorlati jelentőséggel	s057
hatásméret	effect size	a hatás nagysága a kérdéses skálán (pl. százalékpont)	s061, s064
kockázatkülönbség	risk difference (RD)	két arány különbsége, százalékpontban	s064
szándék szerinti kezelés	intention-to-treat (ITT)	a randomizált hozzárendelés szerinti elemzés, minden randomizált egységgel	s064, s072
tényleges kezelés hatása	local average treatment effect (LATE)	a ténylegesen kezelt egységek átlagos hatása	s064, s072
zavaró változó	confounder	a kezelés és a kimenet közös oka	s069
szelekciós torzítás	selection bias	a bekerülési valószínűség összefügg a vizsgált változóval	s049
túlélési torzítás	survivorship bias	csak a folyamatot túlélő egységek megfigyelése	s049
bootstrap	bootstrap	újramintavételezés a becslés bizonytalanságának közelítésére	s059
keresztvalidáció	cross-validation (CV)	a tanító adat belső felosztása modellválasztásra	s086
baseline	baseline	egyszerű, összehasonlítási alapul szolgáló modell vagy mérőszám	s084, s095
feature	feature	a predikció pillanatában ismert bemeneti változó	s002
célváltozó	target	a előrejelzendő kimenet	s001

Magyar	English	Jelentés / megkülönböztetés	Első előfordulás
pipeline	pipeline	az előfeldolgozás és a modell együtt illesztett lánca	s084
regularizáció	regularization	a modell bonyolultságát büntető tag	s090
kalibráció	calibration	a megjósolt valószínűségek és a megfigyelt gyakoriságok egyezése	s095
Brier-pontszám	Brier score	a valószínűségi jóslat négyzetes hibája	s095
ROC-AUC	ROC-AUC	a rangsorolás minőségének mérőszáma	s095
precízió és recall	precision and recall	a pozitív jóslatok pontossága, illetve a pozitív esetek megtalálása	s095
drift	drift	az adat- vagy kapcsolateloszlás időbeli változása	s141
invariáns	invariant	a folyamat azon tulajdonsága, amelynek minden lépésben igaznak kell maradnia	s003, s147
reprodukálhatóság	reproducibility	ugyanaz az eredmény ugyanabból a kódból, adatból és környezetből	s003
determinizmus	determinism	rögzített seed és bemenet mellett azonos kimenet	s003, s082
adatszerződés	data contract	a séma, típusok, időzónák, hiánykezelés és kontrollok együttese	s002, s147
séma	schema	az oszlopok neve, típusa és megengedett értékei	s147
hash-manifest	hash manifest	fájlonkénti SHA-256 lenyomatok jegyzéke	s004, s016
konfidenciaintervallum vs. hiteles intervallum	CI vs. credible interval	a CI gyakorisági lefedettség, a hiteles intervallum posterior-valószínűség	s056, s058

becslés vs. becslési cél	estimate vs. estimand	a számított érték vs. a sokasági mennyiség	s001, s049

Fogalmi térképek

Ez a függelék a könyv fogalmi gerincét rendszerezi. A térképek nem mérési eredmények és nem helyettesítik a fejezetek levezetéseit: azt mutatják meg, melyik fogalom melyikből épül fel, és hol tér vissza a könyv későbbi részeiben. A nyilak iránya a függés iránya (`A --> B`: B épül A-ra).

40.25 A könyv tíz állomása

```
[1] Munkamódszer (1-4. fejezet)
  üzleti kérdés --> estimand --> adatkontraktus --> döntési küszöb
       |               |              |                   |
     s001            s001           s002/s030           s160

[2] Adatkezelés (5-8. fejezet)
  fájlformátum --> pandas tisztítás --> SQL aggregáció --> Arrow/DuckDB/Polars
       |                |                   |                    |
     s005             s006               s007                 s008

[3] Matematika (9-12. fejezet)
  függvény --> derivált --> gradiens --> optimalizálás
      |           |           |              |
    s033        s041        s042           s043
  integrál --> sűrűség --> várható érték --> Monte Carlo
      |           |             |               |
    s046        s046          s047            s047

[4] Statisztika (13-16. fejezet)
  leíró statisztika --> valószínűség --> likelihood/becslés --> hipotézisvizsgálat
        |                   |                  |                     |
      s051                s053               s057                  s061

[5] Modellezés (17-20. fejezet)
  regresszió --> kísérlettervezés/okság --> Bayesi következtetés --> idősor
       |                 |                        |                  |
     s065              s069                     s073               s077

[6] Gépi tanulás (21-24. fejezet)
  split/pipeline --> modellcsalád --> CV/hiperparaméter --> nem felügyelt
        |                |                  |                    |
      s081             s085               s089                 s093

[7] Értékelés és reprezentáció (25-28. fejezet)
  feature engineering --> magyarázat/szeletek --> szöveg/LLM --> PyTorch/autodiff
          |                    |                    |                |
        s097                 s101                 s105             s109

[8] Reprodukálható pipeline (29-32. fejezet)
  projekt --> adatfolyam/minőség --> kísérletkövetés --> konténer
      |             |                     |                |
    s113          s117                  s121             s125

[9] Skálázás és szolgáltatás (33-36. fejezet)
  profilozás --> Dask/Spark --> batch/szolgáltatás --> monitorozás
      |             |                 |                   |
```

```
    s129           s133               s137                s141

[10] Ökoszisztéma és zárás (37-40. fejezet)
  R --> eszközválasztás --> záróprojekt I --> záróprojekt II
  |            |                  |                  |
 s145         s149               s153               s157
```

40.26 A matematikai eszköztár újrahasználata

```
                +--> derivált --> gradiens --> optimalizálás
                |      (s041)      (s042)         (s043)
függvény --------+
   (s033)       +--> logaritmus --> likelihood --> log loss
                       (s034)        (s057)        (s085)

valószínűség --> sűrűség --> várható érték --> integrál --> Monte Carlo
   (s053)        (s046)         (s047)          (s046)        (s047)
      |                                                          |
      +--> becslés --> standard error --> konfidenciaintervallum
            (s057)         (s057)                (s057/s061)
      |
      +--> Bayes-tétel --> prior/posterior --> credible interval
             (s053)            (s073)                (s073)

mátrix --> lineáris modell --> reziduum/vetítés --> SVD/reprezentáció
 (s037)       (s065)               (s039)                (s093)
```

40.27 Háromféle következtetés és a hozzájuk tartozó eszközök

```
Milyen kérdésre válaszolok?
|
+-- Leírás: "mi történt a múltban?"
|      eszköz: leíró statisztika, ECDF, csoportosítás
|      fejezet: s051; csapda: binválasztás, összevont korreláció
|
+-- Predikció: "mi lesz az új egységgel?"
|      eszköz: train/valid/test split, pipeline, CV, kalibráció
|      fejezet: s081/s085/s089; csapda: leakage, eloszlásváltozás
|
+-- Okság: "mi történik, ha beavatkozom?"
       eszköz: randomizáció, ITT, potenciális kimenet, exploratory
       fejezet: s001/s069/s160; csapda: confounding, poszthoc küszöb
```

40.28 A bizonytalanság rétegei

```
minta --> pontbecslés --> standard error --> konfidenciaintervallum
   |           |                 |                   |
 s057        s057              s057                s057

nullmodell --> p-érték --> döntési küszöb --> üzleti döntés
    |            |                |                |
  s061         s061             s160             s160

prior --> posterior --> credible interval --> prediktív eloszlás
  |           |                 |                   |
 s073        s073              s073                s073
```

A három sor nem ugyanaz a kérdés: a konfidenciaintervallum a paraméterre, a credible interval a posteriorra, a prediktív eloszlás egy új megfigyelésre vonatkozik. A p-érték nem annak a valószínűsége, hogy a nullhipotézis igaz.

40.29 A reprodukálhatóság lánca

```
seed --> adatverzió --> kódverzió --> manifest/hash --> futtatókörnyezet
  |           |              |               |                 |
 s002       s121           s121            s121              s113

manifest --> artefaktum-aláírás --> modellregisztráció --> kiadás
    |               |                      |                |
  s121            s123                   s124             s137

kiadás --> monitorozás --> drift --> incidens/rollback
   |           |            |             |
 s137        s141         s141          s143
```

40.30 Didaktikai ábrák

A térképek mellett hét, tengelyezett és feliratozott fogalmi ábra segíti a megértést. Mindegyikhez tartozik leíró alternatív szöveg és a fejezetben közölt, reprodukálható rajzolókód; a teljes generátor a `lab/figures/make_concept_figures.py`.

Ábra	Fejezet	Mit mutat
`figures/execution_graph.png`	s013	notebook-cellák végrehajtási gráfja
`figures/density_cdf.png`	s051	sűrűség és eloszlásfüggvény
`figures/backprop_graph.png`	s043	forward értékek és backward gradiens
`figures/causal_dag.png`	s069	confounding és randomizáció
`figures/bayes_network.png`	s073	prior, likelihood, posterior szerkezete
`figures/validation_timeline.png`	s078	gördülő visszateszt idővonala
`figures/pca_variance.png`	s093	megőrzött variancia és komponensszám
`figures/shape_axes.png`	s017	1D/2D/3D shape és axis
`figures/vector_products.png`	s037	skalár- és vektorszorzat orientációja
`figures/matrix_multiply.png`	s038	mátrixszorzás shape-kontraktusa
`figures/gradient_field.png`	s042	gradiens iránya és szintvonalak
`figures/probability_functions.png`	s046	PMF, PDF és CDF

Az ábrák szemléltetők; a rajtuk szereplő számok determinisztikus számítások vagy elméleti értékek, nem mérési eredmények.

40.31 Hogyan használd a térképeket?

1. **Előre.** Egy fejezet elkezdése előtt keresd meg a fogalmat a térképen, és nézd meg, mire épül (balra) és mire használjuk később (jobbra).
2. **Visszautaláskor.** Ha egy későbbi fejezetben egy korábbi fogalomra hivatkozunk, a térkép megadja azt az alfejezetet, ahol a definíció szerepelt.
3. **Ismétléskor.** A tíz állomás térképe alkalmas arra, hogy a könyv végén egy lapon átismételd a teljes munkamenetet.
4. **Hibakereséskor.** A „csapda” jelölések a tipikus fogalmi hibákra figyelmeztetnek; a részletes indoklás mindig a hivatkozott alfejezetben van.

Tanulási útvonal

A könyv minden alfejezete egy **Belépő** blokkal kezdődik. A blokk négy dolgot rögzít, mielőtt a tanuló az első bekezdéshez ér:

- **szint:** `alap`, `közép` vagy `haladó`;
- **idő:** a tervezett feldolgozási idő (a feladatokkal együtt);
- **eszköz:** a szükséges fájlok, csomagok vagy futtatókörnyezet;
- **előfeltétel, célkészség és kimenet:** mit kell tudni hozzá, mit tanul meg, és milyen bizonyítékot kell előállítania.

A szintek és idők szerkesztési becslések, nem mérési eredmények. A Belépő blokkokat a `tanulasi_utvonal.json` fájl és a `lab/build_learning_path.py` generálja; a fejezetszöveg ettől függetlenül olvasható.

40.32 A három szint

Szint	Tervezett idő	Tipikus kimenet
alap	45 perc	feladatmegoldás és önellenőrzés
közép	60 perc	működő kód és a labor ellenőrző tesztje
haladó	90 perc	projektartefaktum és ellenőrzött futtatási jegyzőkönyv

40.33 Bővítő modulok

A **BŐVÍTŐ MODUL** jelölésű alfejezetek nem kötelezőek az alap útvonalon. Ezek egy-egy speciális eszköz vagy módszer önálló alkalmazását tanítják, és gyakran külső erőforrást (GPU, klaszter, R, PyMC, Docker) igényelnek:

```
8. fejezet    Arrow, DuckDB, Polars
12. fejezet   Monte Carlo és integrálás
18. fejezet   kísérlettervezés és oksági következtetés
19. fejezet   Bayesi következtetés, PyMC
20. fejezet   idősorok és időbeli validáció
```

```
24. fejezet   nem felügyelt tanulás
27. fejezet   szöveg, embedding, LLM-értékelés
28. fejezet   PyTorch, autodiff, CPU/GPU
31. fejezet   kísérletkövetés, adat- és modellverziózás
32. fejezet   konténerek és távoli környezetek
33. fejezet   profilozás és párhuzamos futtatás
34. fejezet   Dask, Spark, elosztott végrehajtás
37. fejezet   R és többnyelvű ökoszisztéma
```

40.34 Ajánlott útvonalak

Útvonal	Fejezetek	Alfejezet	Tervezett idő
Kezdő analitikus mag	1–16, 21–23, 25–26, 29–30	92	88 óra
ML-mérnök	1–8, 21–36	96	110 óra
Teljes könyv	1–40	160	190 óra

A **kezdő analitikus mag** a munkamódszert, az adatkezelést, a matematikai és statisztikai alapokat, majd a prediktív modellezés alapjait adja. A **teljes könyv** tartalmazza a haladó oksági, Bayesi, idősoros, MLOps-, elosztott és R-es modulokat is. Az óraszámok a Belépő blokkok idejének összege; a tényleges idő az előismeretektől és a laborok mélységétől függ.

40.35 Hogyan használd a Belépő blokkot?

1. **Előfeltétel-ellenőrzés.** Ha a megadott előfeltétel alfejezet kimenete nincs meg (nem tudod reprodukálni a feladatot), előbb azt pótold.
2. **Időkeret.** A megadott idő a feladatokkal együtt értendő; ha jelentősen túlléped, a hiányzó előfeltétel vagy a környezet a valószínű ok.
3. **Kimenet.** A fejezet végén a Kimenet mezőben szereplő bizonyítékot kell előállítani; a puszta elolvasás nem teljesíti a célt.
4. **Bővítő modul.** Ha az eszköz (GPU, klaszter, R) nem elérhető, a modul kihagyható; a törzsanyag ettől függetlenül teljes.

Implementációs feladatok pontozása

A könyv implementációs feladatai egységes, gépileg olvasható pontozási szerződést használnak. A cél, hogy ugyanaz a beadás ugyanazt a visszajelzést kapja helyben, tanórán és automatikus javításnál is.

40.36 A feladatspecifikáció

Minden feladat saját könyvtára a `lab/grading/tasks/` alatt található:

```
task.json          feladat-azonosító, belépési pont, szerződés, rubrika
public/test_*.py   nyilvános tesztek (a tanuló előre látja)
hidden/test_*.py   rejtett tesztek (leakage, granularitás, szélső esetek)
solution.py        referenciamegoldás a pontozó önellenőrzéséhez
```

A `task.json` kötelező mezői: `task_id`, `section`, `title`, `entrypoint`, `contract` és `rubric`. A rubrika két részből áll: `public` és `hidden`; minden tesztnévhez pontszám tartozik. A rejtett tesztek szándékosan olyan hibákat céloznak, amelyek a nyilvános teszteken átcsúsznának:

- **leakage:** jövőbeli információ, címke vagy tesztadat szivárgása;
- **granularity:** rossz összegzési szint, például chunk-átlagok átlaga;
- **edge_case:** üres bemenet, nulla nevező, határérték, érvénytelen típus.

40.37 Pontozás és visszajelzés

A `lab/grading/grader.py` a beadott fájlt `submission.py` néven ideiglenes munkakönyvtárba másolja, majd a nyilvános és rejtett teszteket külön subprocessben futtatja. A pontszám a teljesített rubrikapontok összege:

Állapot	Feltétel
`pass`	minden rubrikapont megvan
`partial`	legalább egy pont megvan, de nem mind
`fail`	nulla pont
`error`	hiányzó beadás vagy futtatási hiba

A kimenet JSON: `task_id`, `status`, `score`, `max_score`, a nyilvános és rejtett tesztek száma, valamint tételes `feedback` (tesztnév, kategória, pont, hibaüzenet). A hibák kategóriája (`leakage`, `granularity`, `edge_case`, `alap`) megmondja, melyik tanulási cél sérült.

Futtatás:

```
python lab/grading/grader.py lab/grading/tasks/s002_label_split hallgato.py
```

A pontozó nem biztonsági homoksziget: a beadott kódot a saját jogosultságaiddal futtatja. Idegen beadást csak izolált környezetben értékelj.

40.38 Gépileg pontozott mintapéldák

Feladat	Alfejezet	Nyilvános	Rejtett	Max. pont
`DS7-TASK-s002`	s002	3	5	14
`DS7-TASK-s033`	s033	2	4	9
`DS7-TASK-s106`	s106	2	5	13
`DS7-TASK-s133`	s133	2	5	12

A mintapéldák lefedik a három fő hibaosztályt: az s002 a címke-időzítés leakage-ét, az s106 a duplikált dokumentum granularitását, az s133 a chunk-szintű összegzés granularitását, az s033 pedig az üres és nem véges bemenet szélső eseteit.

40.39 Implementációs feladatok a könyvben

A következő 68 alfejezet tartalmaz „Megvalósítás” vagy „Implementálás” feladatot. A mintapéldák sémája ezekre közvetlenül alkalmazható; a specifikáció felvétele a `task.json` és a két tesztkönyvtár létrehozásával történik.

```
s002 s003 s004 s007 s011 s014 s015 s016 s021 s028
s033 s034 s038 s039 s042 s043 s047 s051 s054 s060
s061 s065 s067 s072 s074 s076 s077 s079 s085 s092
s093 s097 s102 s105 s106 s108 s112 s113 s114 s116
s118 s119 s121 s122 s123 s126 s130 s131 s132 s133
s134 s135 s137 s141 s146 s147 s149 s150 s151 s152
s153 s154 s155 s156 s157 s158 s159 s160
```

A felsorolás a fejezetszöveg „Megvalósítás”/„Implementálás” feladatait jelöli; a nyitott, magyarázó jellegű feladatok pontozása továbbra is rubrikás emberi értékelés.

Jelképjegyzék

Ez a függelék fejezetenként követi a könyv matematikai és technikai jelöléseit. A „Jelölés” oszlop a szövegben használt alakot mutatja; a „Jelentés” a definíció rövid emlékeztetője, a „Fejezet” pedig az első kidolgozó hely.

40.40 1. rész: munkamódszer (1–4. fejezet)

Fejezet	Jelölés	Jelentés
1	`Y_i(z)`	az i-edik egység potenciális kimenete a z hozzárendelés mellett
1	`Z_i`	randomizált hozzárendelés (1 = treatment, 0 = kontroll)
1	`D_i`	a ténylegesen megkapott kezelés; nem azonos Z_i-vel
1	`\Delta_{\mathrm{ITT}}`	a hozzárendelés populációszintű átlagos hatása
2	`$?`	az utolsó előtérben futott parancs kilépési kódja Bashban
2	`RSS`	rezidens memória mérete; mértékegysége platformfüggő
3	`sys.executable`	a futó interpreter fájlútvonala
3	`sys.prefix`	a virtuális környezet gyökérkönyvtára
4	`execution_count`	a notebook cella tényleges futtatási sorszáma
4	`Kernel`	a notebook kódját futtató külön folyamat

40.41 2. rész: adatkezelés (5–8. fejezet)

Fejezet	Jelölés	Jelentés
5	`shape`	a tengelyek hossza, a tengelyek sorrendjében
5	`axis`	az összevonás vagy művelet tengelyindexe
5	`dtype`	a tömb elemtípusa
6	`pd.NA`	a pandas hiányjelölője nullable típusokban
6	`Int64`	nullable egész dtype
7	`NULL`	ismeretlen érték; nem azonos a 0-val vagy az üres stringgel
7	`COUNT(*)`	a sorok száma, a NULL-os sorokat is beleértve
8	`Parquet`	oszlopos, sémát hordozó fájlformátum
8	`partition`	könyvtár szerinti felosztás, amelyet a pruning kihasznál

40.42 3. rész: matematika (9–12. fejezet)

Fejezet	Jelölés	Jelentés
9	`f(x)`	egyváltozós függvény
9	`\log`	természetes logaritmus, ha más alap nincs jelölve
10	`\mathbf X`	adatmátrix (sorok: megfigyelések, oszlopok: feature-ök)
10	`\mathbf w`	paramétervektor
10	`\mathbf X^T\mathbf X`	a normálegyenlet mátrixa
11	`f'(x)`	derivált
11	`\nabla L(\mathbf w)`	a veszteség gradiense
11	`\eta`	tanulási ráta
12	`\int`	határozott integrál
12	`E[X]`	várható érték
12	`\hat I_n`	Monte Carlo becslés az integrálra

40.43 4. rész: statisztika (13–16. fejezet)

Fejezet	Jelölés	Jelentés
13	`\bar{x}`	mintaátlag
13	`s`	minta szórása

Fejezet	Jelölés	Jelentés
13	`\mathrm{IQR}`	interkvartilis terjedelem
14	`P(A)`	az A esemény valószínűsége
14	`P(A \mid B)`	feltételes valószínűség
15	`\hat\theta`	paraméterbecslés
15	`L(p)`	likelihood-függvény
16	`H_0`	nullhipotézis
16	`p`	p-érték
16	`\alpha`	szignifikanciaszint

40.44 5. rész: modellezés (17–20. fejezet)

Fejezet	Jelölés	Jelentés
17	`\beta`	regressziós együttható
17	`\varepsilon`	hibatag
17	`R^2`	meghatározottsági együttható
18	`ATE`	átlagos kezelési hatás
18	`ITT`	intention-to-treat hatás
19	`p(\theta\mid y)`	posterior
19	`\theta`	modellparaméter
20	`y_t`	idősort érték a t időpontban
20	`origin`	a visszateszt tanító/teszt határpontja

40.45 6. rész: gépi tanulás (21–24. fejezet)

Fejezet	Jelölés	Jelentés
21	`split.json`	a train/valid/test felosztás manifestje
21	`pipeline`	az előfeldolgozás és a modell együttes objektuma
22	`RMSE`	négyzetes hibaátlag gyöke
22	`Ridge`	L2-regularizált lineáris modell
23	`K`	keresztvalidációs foldok száma
23	`GridSearchCV`	hiperparaméter-keresés keresztvalidációval
24	`PCA`	főkomponens-elemzés
24	`k`	komponensek vagy klaszterek száma

40.46 7. rész: értékelés és reprezentáció (25–28. fejezet)

Fejezet	Jelölés	Jelentés
25	`leakage`	a predikció pillanatában nem elérhető információ használata
25	`feature`	a modell bemeneti változója

Fejezet	Jelölés	Jelentés
26	`slice`	részpopulációra bontott értékelés
26	`SHAP`	magyarázati érték modell-hozzájárulásokra
27	`\text{precision@k}`	precision at k
27	`\text{recall@}k`	recall at k
27	`MRR`	mean reciprocal rank
28	`\nabla`	autograd által számított gradiens
28	`device`	CPU vagy GPU futtatóeszköz

40.47 8. rész: reprodukálható pipeline (29–32. fejezet)

Fejezet	Jelölés	Jelentés
29	`modul`	importálható Python-fájl vagy csomag
29	`pytest`	tesztfuttató keretrendszer
30	`idempotens`	ismételt futtatás után is azonos kimenet
30	`DAG`	a pipeline irányított, körmentes függőségi gráfja
31	`run_id`	egy futás egyedi azonosítója
31	`input_hash`	a bemeneti fájlok hash-identitása
32	`digest`	konténerkép tartalomazonosítója
32	`Docker`	konténerizált futtatókörnyezet

40.48 9. rész: skálázás és szolgáltatás (33–36. fejezet)

Fejezet	Jelölés	Jelentés
33	`T_{\mathrm{wall}}`	falóra-idő
33	`CPU`	processzoridő forrása
34	`partition`	Dask/Spark adatpartíció
34	`shuffle`	partíciók közötti újraelosztás
35	`p95`	a késleltetés 95. percentilise
35	`throughput`	időegység alatti kiszolgált kérések száma
36	`drift`	a bemeneti eloszlás változása
36	`PSI`	population stability index

40.49 10. rész: ökoszisztéma és zárás (37–40. fejezet)

Fejezet	Jelölés	Jelentés
37	`factor`	R kategorikus változó
37	`contrast`	a faktor szintek kódolása
38	`p95`	eszközválasztási késleltetési metrika

Fejezet	Jelölés	Jelentés
38	`licenc`	az eszköz használatának jogi feltétele
39	`manifest`	a záróprojekt futtatási jegyzéke
39	`AUC`	a záróprojekt ROC-görbe alatti területe
40	`\tau`	a MicroGrove kísérlet átlagos kezelési hatása
40	`\mathrm{SE}`	standard error

40.50 Mennyiségek dimenziója és egysége

A képletblokkok változóinak jelentését a fenti fejezeti táblák adják; az alábbi táblázat a fejezetenkénti kulcsmennyiség dimenzióját, tartományát és egységét rögzíti. A mértékegység nélküli arányok dimenzió nélküli értékek.

Fejezet	Mennyiség	Dimenzió / tartomány	Egység
1	konverziós arány	$[0, 1]$	arány
2	kilépési kód	0–255	egész
3	Python-verzió	`x.y.z`	verzió
4	`execution_count`	≥ 0	egész
5	tömb elemszám	≥ 0	darab
6	hiányarány	$[0, 1]$	arány
7	sorok száma	≥ 0	darab
8	fájlméret	≥ 0	byte
9	függvényérték	valós	a bemenet egysége
10	mátrix alakja	≥ 0 dimenziók	egész
11	gradiens	valós vektor	a veszteség egysége / paraméter
12	valószínűség	$[0, 1]$	arány
13	átlag	valós	a bemenet egysége
14	valószínűség	$[0, 1]$	arány
15	standard error	≥ 0	a becslés egysége
16	p-érték	$[0, 1]$	arány
17	reziduális	valós	a cél változó egysége
18	kezelési hatás	valós	százalékpont
19	posterior valószínűség	$[0, 1]$	arány
20	időpont	datetime	idő
21	modell pontszám	valós	a cél egysége
22	veszteség	≥ 0	a cél egysége^2
23	foldok száma	≥ 2	egész
24	varianciahányad	$[0, 1]$	arány
25	feature érték	valós	a feature egysége
26	SHAP érték	valós	a cél egysége
27	precision/recall	$[0, 1]$	arány
28	gradiens	valós	a veszteség egysége
29	tesztek száma	≥ 0	darab
30	futásidő	≥ 0	másodperc
31	`run_id`	hex string	azonosító
32	kép digest	hex string	azonosító
33	speedup	≥ 0	arány
34	partíciók száma	≥ 1	darab

Fejezet	Mennyiség	Dimenzió / tartomány	Egység
35	késleltetés	≥ 0	milliszekundum
36	PSI	≥ 0	arány
37	faktor szint	kategória	név
38	licenc	kategória	név
39	AUC	$[0, 1]$	arány
40	kezelési hatás	valós	százalékpont

Bibliográfia és forráskulcsok

Dátum: 2026-10-20. A státusz az e csomag készítésekor ténylegesen elérhető bizonyítékot jelöli. Az olvasott gyűjtőoldal nem igazol minden egyedi API-t, verziót vagy telepítési kombinációt. A kiindulópontot a későbbi research lépésnek kell releváns szakaszig megnyitnia.

Kulcs	Hivatalos forrás	Ellenőrzési státusz
uv-sync	uv: locking és syncing	olvasott
uv-jupyter	uv és Jupyter együtt	olvasott
pandas-cow	pandas Copy-on-Write	olvasott
polars-lazy	Polars lazy API	olvasott
sklearn-pitfalls	scikit-learn: hibák és adatszivárgás	olvasott
sklearn-cv	scikit-learn keresztvalidáció	hivatalos keresési kivonat ellenőrizve
scipy	SciPy statisztikai referencia	olvasott
scipy-bootstrap	SciPy bootstrap	olvasott
statsmodels	statsmodels kezdő útmutató	olvasott
nist-ci	NIST: confidence limits for the mean	olvasott
nist-p	NIST: critical values and p values	olvasott
duckdb	DuckDB Python API	hivatalos keresési tartalom ellenőrizve; megnyitás átirányítást adott
python	Python dokumentáció	kutatási kiindulópont
numpy	NumPy tanulási és API referencia	kutatási kiindulópont
pandas	pandas User Guide	kutatási kiindulópont
sklearn	scikit-learn User Guide	kutatási kiindulópont
postgres	PostgreSQL dokumentáció	kutatási kiindulópont
jupyter	Jupyter dokumentáció	kutatási kiindulópont
vscode	VS Code Python	kutatási kiindulópont
conda	conda környezetkezelés	kutatási kiindulópont
arrow	Apache Arrow Python	kutatási kiindulópont
seaborn	Seaborn útmutató	kutatási kiindulópont
torch	PyTorch oktatóanyagok	kutatási kiindulópont
pymc	PyMC dokumentáció	kutatási kiindulópont
arviz	ArviZ dokumentáció	kutatási kiindulópont
hf	Hugging Face Transformers	kutatási kiindulópont

Kulcs	Hivatalos forrás	Ellenőrzési státusz
pytest	pytest útmutató	kutatási kiindulópont
ruff	Ruff dokumentáció	kutatási kiindulópont
dbt	dbt dokumentáció	kutatási kiindulópont
prefect	Prefect dokumentáció	kutatási kiindulópont
dagster	Dagster dokumentáció	kutatási kiindulópont
mlflow	MLflow dokumentáció	kutatási kiindulópont
dvc	DVC dokumentáció	kutatási kiindulópont
docker	Docker dokumentáció	kutatási kiindulópont
dask	Dask dokumentáció	kutatási kiindulópont
spark	Apache Spark dokumentáció	kutatási kiindulópont
fastapi	FastAPI útmutató	kutatási kiindulópont
streamlit	Streamlit dokumentáció	kutatási kiindulópont
r	R kézikönyvek	kutatási kiindulópont
renv	renv dokumentáció	kutatási kiindulópont
quarto	Quarto dokumentáció	kutatási kiindulópont

40.51 Fejezethez rendelt kulcsok

- sch01: sklearn-pitfalls, nist-ci.
- sch02: python, vscode, docker.
- sch03: uv-sync, uv-jupyter, python, conda.
- sch04: uv-jupyter, vscode, jupyter, quarto.
- sch05: numpy, scipy.
- sch06: pandas-cow, pandas.
- sch07: duckdb, postgres.
- sch08: duckdb, polars-lazy, arrow.
- sch09: numpy, scipy.
- sch10: numpy, scipy, sklearn.
- sch11: scipy, torch.
- sch12: scipy, scipy-bootstrap.
- sch13: nist-ci, pandas, seaborn.
- sch14: scipy, nist-ci.
- sch15: scipy-bootstrap, nist-ci, statsmodels.
- sch16: nist-p, scipy, nist-ci.
- sch17: statsmodels, nist-ci, sklearn-pitfalls.
- sch18: nist-p, statsmodels, scipy.
- sch19: scipy, pymc, arviz.
- sch20: sklearn-cv, statsmodels, pandas.
- sch21: sklearn-pitfalls, sklearn-cv, sklearn.
- sch22: sklearn, sklearn-pitfalls.
- sch23: sklearn-cv, sklearn-pitfalls, sklearn.
- sch24: sklearn, sklearn-cv.
- sch25: sklearn-pitfalls, sklearn-cv, pandas.
- sch26: sklearn, sklearn-pitfalls, nist-ci.
- sch27: sklearn, torch, hf.
- sch28: torch.
- sch29: python, pytest, ruff.
- sch30: duckdb, dbt, prefect, dagster.

- sch31: mlflow, dvc, sklearn.
- sch32: docker, vscode, uv-sync.
- sch33: python, numpy, sklearn.
- sch34: polars-lazy, duckdb, dask, spark.
- sch35: sklearn, fastapi, streamlit, docker.
- sch36: sklearn, nist-ci, python.
- sch37: r, renv, quarto.
- sch38: uv-sync, docker, vscode, spark.
- sch39: sklearn-pitfalls, sklearn-cv, duckdb.
- sch40: nist-p, nist-ci, scipy-bootstrap, statsmodels.

A kutatás elsődleges forrást használjon. Nem kell teljes dokumentációt vagy tankönyvet másolni. Az OpenAI-generátor technikai ellenőrzése külön téma, nem a tanulólabor függősége: - https://developers.openai.com/api/docs/models/gpt-6-astra (modelloldal megnyitva) - https://developers.openai.com/api/reference/resources/responses/methods/retrieve (keresési kivonat; a megnyitás hibát adott) - https://platform.openai.com/docs/api-reference/responses-streaming/response/function_call_arguments/done?api-mode=responses (keresési kivonat)

Ellenőrzött keretjegyzet

Ellenőrzési dátum: 2026-10-20. Saját szavas összegzés; a teljes tanterv, feladat, szerep- és eszközválasztás saját szerkesztési döntés. Nem állítjuk az összes opcionális eszköz vagy verziókombináció tényleges kipróbálását.

40.52 Környezet és notebook

A uv külön kezeli a függőségek feloldását és a környezet szinkronizálását. A –locked ellenőrzi a lock egyezését; a –frozen az aktualitásvizsgálatot kihagyja. A projekt környezetének és a Jupyter kernelének összerendelését külön ellenőrizni kell.

- uv lock és sync
- uv Jupyter-integráció

A könyv ezért különíti el az interpretert, csomagkészletet, IDE-t, kernelspecet és container image-et. A mellékelt lab még nem tartalmaz feloldott uv.lockot.

40.53 Táblás adat és végrehajtási terv

A pandas 3 dokumentációja a Copy-on-Write alapműködését és a láncolt értékadás korlátját írja le. A kezdőprojekt pandas 2.2.3-mal futott; explicit .loc műveletet és egyértelmű másolást kell tanítani, a verzióeltérést jelölve.

- pandas Copy-on-Write

A Polars lazy munkafolyamatában a tervépítés és a végrehajtás külön lépés. A lazy/streaming fogalmak nem jelentenek minden műveletre korlátlan adatméretet.

- Polars lazy útmutató
- DuckDB Python API

A DuckDB hivatalos keresési tartalma a helyi Python-kapcsolatot és a fájl-/DataFrame- lekérdezést alátámasztotta; a külön oldalmegnyitás átirányítást adott. A tanulási főút eszközválasztás, nem igazolatlan teljesítményrangsor.

40.54 Korrekt modellértékelés

A scikit-learn dokumentáció hangsúlyozza, hogy a tesztadatból származó információ optimista értékelést okozhat. A preprocessing és szelekció a tanítórészen belül történjen; a Pipeline segít e szerződés megtartásában. A minták idő- vagy csoportfüggését a splitnek is követnie kell.

- Gyakori hibák és leakage
- Keresztvalidáció

Az időhelyes Northstar label-availability szűrés saját feladattervezési döntés. A starter egyetlen előre rögzített cutoffot és két baseline-t mutat be; nem teljes hiperparaméter-keresési vagy termelési validációs rendszer.

40.55 Statisztikai feltételek és értelmezés

A NIST a konfidenciaszintet az eljárás ismételt mintavételi lefedettségéhez köti, és a p-értéket a nullmodell szerinti feltételes szélsőségként tárgyalja. A tanterv a gyakori félreértéseket szimulációval és ellenpéldával tanítja.

- NIST: átlag konfidenciaintervalluma
- NIST: kritikus érték és p-érték

A SciPy bootstrap API több újramintázásos intervallumeljárást támogat. A megfelelő mintavételi egység és az adatfelvételi feltétel továbbra is a modellező felelőssége. A statsmodels modellillesztési felülete inferenciális outputot is ad; ez nem mentesít a modell feltételeinek és diagnosztikájának ellenőrzése alól.

- SciPy statisztikai referencia
- SciPy bootstrap
- statsmodels kezdő útmutató

40.56 Ellenőrzési határ

Az alap numerikus Python-labort helyben futtattuk; az eredményt az átadási ellenőrző jegyzet rögzíti. PyMC, R, GPU, Spark, Dask, Docker, felhő és szolgáltatás- telepítés nem tekinthető kipróbáltnak e keretjegyzet alapján. Konkrét későbbi API- és platformkérdéshez a sources.md kulcsai és az opcionális research lépés adnak utat.